

第二十一届全国大学生智能汽车竞赛

技 术 报 告

基于多视野连续感知与有线空地 协同的飞跃雷区系统设计

飞跃雷区组

学 校：哈尔滨工程大学

队伍名称：济海追风 1 队

参赛队员：秦华骏、窦林禹吉、魏志豪、潘雅琴

带队教师：张立伟

关于技术报告和研究论文使用授权的说明

本人完全了解全国大学生智能汽车竞赛有关保留、使用技术报告和研究论文的规定，即：参赛作品著作权归参赛者本人，比赛组委会可以在相关主页上收录并公开参赛作品的设计方案、技术报告以及参赛模型车的视频、图像资料，并将相关内容编纂收录在组委会出版论文集中。

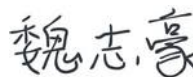
参赛队员签名：



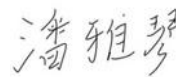
秦华骏



窦林禹吉

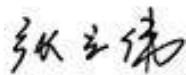


魏志豪



潘雅琴

带队教师签名：



张立伟

日期：2026年8月16日

摘要

飞跃雷区项目由悬停飞行器、不可离地的麦轮地面车以及二者之间的短线缆共同构成。地面车不能配置环境感知传感器，飞行器与地面车之间又禁止无线通信，因此系统必须同时处理视野切换、异构执行体协同、多节点通信和故障安全问题。本项目围绕上述约束，设计了一套由五个双核 CYT4BB7 控制节点组成的生产系统：Core0 负责飞行控制与机载传感器，Core1 负责下视视觉、三来源目标连续性和运动规划，Core2 负责麦轮底盘控制，Core3 与 Core4 分别提供后向和前向观察。硬件载板层面，Core0 与 Core1 的两颗 CYT4BB7 集成在同一张四旋翼飞行控制板上；“五板十核”是项目沿用的五个逻辑控制节点简称，不表示五张彼此独立的主控 PCB。另设 CoreT3 原始图像录制板作为开发工具，不参与比赛运行时的生产控制闭环。

系统的第一项核心设计是多视野分布式感知与统一车体系目标连续性。Core1、Core3、Core4 三路信标视觉源先在各自相机 owner 处完成目标检测和轨迹确认，再由 Core1 将各来源候选变换为统一的车体前向、横向误差。当前 `source/session/trackId` 仍有效时保持既有目标；精确身份消失后，在目标推进门开放的条件下，按照历史目标在车体系中的预测位置对所有来源重新评分。普通遮挡、真实越界和系统故障分别进入 EdgeHold、可靠边界拒绝和 fail-closed 路径，避免把三种不同情况混为一次普通丢帧。

第二项核心设计是同一运动意图驱动的有线空地协同。Core1 以固定规划节拍形成唯一 `vx/vy/wz` 计划，经 RS-485 发送给 Core2，同时经 Core0/Core1 有线链路发送给飞行控制。麦轮车依据该计划完成全向运动学和航向、轮速、电流级联控制；飞行器将同一计划解释为水平速度目标、角度前馈和偏置转心补偿。两套执行系统不共享控制器，但共享经过身份、序号、新鲜度、飞行阶段和安全状态约束的任务意图。

第三项核心设计是面向多节点系统的分级安全降级与受控恢复。飞行任务序列、姿态保护、ToF 失效保护和底盘运行状态分别由明确 owner 管理。短时 ToF 失效进入 HOLD，长期失效进入按固定斜率降低油门偏移的 RAMP；链路、协议、配置、相机、位姿和计划停更分别产生可定位的故障事实。旧

session、重复序号和过期 snapshot 不能恢复控制权，Core1 → Core0 MOTION 与 Core2 owner 重建均设置零输出栅栏。

第四项核心设计是五板参数库与可恢复的分布式参数事务。Core0~Core4 分别维护 Live RAM、独立 Runtime 和五个 Library Slot，并以双银行、CRC 和 selector journal 保存持久状态。Core1 通过 PREPARE、COMMIT、ACTIVATE、FINISH/ABORT 协调整机参数加载；节点复位或写入应答不确定时，根据持久决策继续完成或补偿恢复。该机制已在限定范围内取得五板联合硬件验收。本文只给出代码、确定性 Host 场景、模型仿真和既有登记证据能够支持的结论，不以源码或构建结果替代实机性能。

量化验证中，麦轮模型完成 6 类确定性工况和 61 点饱和扫描；三来源生产 C 场景以 16 拍覆盖 Source1、Source3、Source4 的身份保持、接管、遮挡、冻结和边界拒绝；75 帧固定合成真值夹具同时给出检出计数与像素中心误差；ToF 生产 C 依赖桩场景验证 3 s HOLD、2 个百分点每秒 RAMP 及停机请求边界。空地协同和五板参数事务分别通过启动/重入模型和成功/拒绝/重启恢复模型补齐直接证据。队伍确认报告采用的板上配置随当前固件烧录后能够完成比赛流程；该结论没有附带本轮完整赛程日志，因而不外推为所有场地和故障组合均已实测。

关键词：飞跃雷区；分布式控制；多视野感知；目标连续性；空地协同；麦轮底盘；故障降级；参数事务

目录

目录将在打开文档时自动更新

主要符号表

符号或缩写	含义	单位或取值
Core0~Core4	生产控制系统的五个 CYT4BB7 双核逻辑节点；Core0/Core1 共载于主飞控 PCB	—
CoreT3	独立原始图像录制与调试板	—
vx、vy	统一规划中的车体系前向、横向速度意图	m/s
wz	统一规划中的偏航角速度意图	rad/s
errFwd、errRight	目标相对动态中心的车体系前向、横向误差	规划内部量
u、v	图像平面横向、纵向坐标	pixel
ToF	Time of Flight，飞行高度测距传感器	mm
IMU	Inertial Measurement Unit，惯性测量单元	—
KF	Kalman Filter，水平速度估计中的卡尔曼滤波	—
TD	Tracking Differentiator，规划输出整形环节	—
KVM	Core0 实体按键到多板菜单的租约式路由机制	—
DTO	跨核传输的数据对象	—
CRC	循环冗余校验	—
Live RAM	当前编辑并被运行消费者使用的参数工作副本	—
Runtime	独立于参数库、可在上电时恢复的运行参数	—
Library Slot	整机参数库槽，编号 1~5	—
CODE-CONFIRMED	当前源码和调用链已核实	证据等级
BUILD-CONFIRMED	指定提交和 IAR 配置构建通过	证据等级
HW-CONFIRMED	指定固件、板卡、接线和场景实测通过	证据等级
NEEDS-EVIDENCE	当前证据不足或不适用于当前基线	证据等级
Host/回放	指定主机回归或记录回放；作为验证方法绑定到提交和输入范围，不单独构成正式证据等级	支撑证据

第 1 章 引言

1.1 赛题约束与系统对象

飞跃雷区不是一辆传统循迹车，也不是一架独立飞行器。参赛作品由悬停飞行器、地面麦轮车和二者之间的短线缆构成。车体不得离开地面，也不能安装环境感知传感器；飞行器不能降落到地面或车辆上；飞机与车辆之间的业务通信必须使用有线链路。由此形成了一个明确的系统边界：环境目标首先由飞行器侧视觉系统观测，运动意图必须经有线网络传到两个动力学不同的执行体，任何单板失效或陈旧数据都不能直接演化为持续运动命令。报告的组成、附录和排版边界依据组委会提交通知及技术报告细则确定。^{[1][2]}

无人机的前、后、下三个方向安装了相互补充的视觉观察单元。下视相机适于观察机体下方和车辆附近目标，前向、后向相机用于扩展车辆运动方向上的覆盖。当目标由一个画面离开并进入另一个画面时，三个视觉源的像素坐标、安装方向和观察模型不同，不能直接比较。因此，本项目没有把三幅图像简单拼接，而是先在各板本地完成检测和跟踪，再将候选转换到统一车体系进行选择。

地面执行体采用四轮麦克纳姆轮底盘。麦轮能够在不改变车体朝向的情况下完成前后、横向和斜向移动，适合在有限场地内响应二维速度计划，但也带来四轮速度分配、方向归一化、航向保持和电流闭环等问题。其全向轮结构与运动学来源可追溯至 Ilon 公开的早期技术文献；本文只将该文献作为外部技术来源，不表示本队拥有或申请相关专利。^[5] 飞行器则需要维持姿态与高度，同时跟随车辆运动。两者若分别根据局部信息独立决策，容易出现车辆提前启动、飞机滞后或停止条件不一致。项目因此将规划 owner 集中在 Core1，由同一 vx/vy/wz 意图驱动两个执行体，再由各自控制系统完成动力学闭环。

1.2 相关工作与方案比较

四旋翼姿态估计通常以陀螺仪提供短时角速度，以加速度方向约束长期倾角漂移。Mahony 等提出的非线性互补滤波为在旋转群上融合这两类信息提供了结构化方法。^[3] 本工程采用 Mahony 六轴姿态解算作为姿态链的一部分，但没

有把该基础算法包装为本队原创理论；工程工作主要集中在原始量校准、控制周期分层、保护门以及与 ToF 倾角补偿和速度估计的共享。

水平速度估计需要在 IMU 高频但会积分漂移、光流直接反映地面相对运动但受高度和纹理影响之间权衡。Kalman 滤波给出了基于状态转移、过程噪声和量测噪声组织递推估计的经典框架。^[4]本工程在这一框架上处理光流时延、机体旋转像移、传感器偏置与有效性门控；报告只主张这些面向当前装机条件的工程实现，不宣称提出新的滤波理论。

传统单相机信标跟踪可在每帧选择最近中心、最大面积或最高置信度候选，但目标离开画面后只能重新确认，新画面中的候选也缺乏与旧目标直接可比的几何语义。简单多相机方案还可能把各相机选择结果直接竞争，造成换源时目标跳变。本工程让 Core1、Core3、Core4 先在本地完成检测和时间一致性跟踪，再把结构化轨迹统一到车体系，通过身份保持、全来源重新评分、EdgeHold、SEARCH 和边界拒绝组织连续性。该机制没有跨相机全局 ID，也不等于三维重建或自动标定。

空地协同的另一种直接实现是飞机和车辆各自根据局部误差规划，再用启停信号粗略同步；多板参数也可以由操作员逐板保存。这两种做法分别容易产生运动意图冲突和混合配置。本工程改为由 Core1 产生唯一运动计划，并用分布式参数事务组织五板准备、提交、应用与恢复。其价值在于减少系统级状态歧义，而不是声称各执行体瞬时运动完全一致，或 Flash 写入具备硬件瞬时原子性。当前架构与证据边界以活动源码、项目文档和证据台账为依据。^[8]

表 1.1 常规实现与本工程方案比较

系统问题	常规实现	本工程机制	直接价值	适用边界
目标跨出单相机视野	丢失后在新画面重新确认，或每拍选择最近候选	三源本地跟踪、统一车体系、精确身份保持与全来源重新评分	减少重复确认和计划硬复位的机会	不具备跨相机全局 ID，多灯交叉仍可能误接管
飞机与车辆协同	两端分别规划，用启停量粗同步	Core1 产生唯一 vx/vy/wz，两端分别执行动力学闭环	统一任务意图，减少互相冲突的局部决策	不保证瞬时速度相同或轨迹误差为零
多节点故障恢复	仅依据在线位或收到任意新帧恢复	分类故障、首因锁存、session/	防止旧身份和陈旧命令跨重连复活	只覆盖已经建模的故障类别

系统问题	常规实现	本工程机制	直接价值	适用边界
		sequence/freshness 与 零帧 fence		
多板参数 保存	操作员逐板保存和加 载	五板 PREPARE、 COMMIT、 ACTIVATE、 FINISH/ABORT 事务	避免部分板卡加载造 成混合配置	不等于 Flash 瞬时原 子写入或任意断电均 已验证

1.3 主要技术问题

本项目首先面对的是跨视野目标连续性。最直接的做法是每拍选择当前距离中心最近或亮度最高的候选，但这种策略会在重叠视野和多灯场景中频繁换灯。另一种做法是在原相机丢失后等待新相机重新累计确认，又会制造不必要的停顿。系统需要在“保持现有目标”和“允许新来源及时接管”之间建立明确条件，同时区分普通遮挡、画面越界和通信故障。

第二个问题是空地协同。飞行器与车辆的控制周期、状态变量和安全约束均不相同，不能共享一个执行控制器；但比赛任务又要求二者围绕同一目标运动。为此，系统把“任务意图”和“执行控制”分层：Core1 只发布经过限幅、斜率和状态门处理的运动计划，Core0 与 Core2 分别把计划映射到飞行速度/姿态和四轮电机目标。

第三个问题是多节点失效。系统包含五个生产控制节点和十个 M7 核，节点间有 UART、RS-485，双核之间还有非一致缓存条件下的共享内存。单独检查一个 linkAlive 位不足以证明数据可用：链路可能在线但配置不一致，相机可能在线但轨迹停更，snapshot 可能格式正确但属于旧 boot generation。安全设计必须同时考虑 owner、session、sequence、freshness、finite、状态和任务阶段。

第四个工作问题是整机调参。飞控、视觉、规划和底盘参数分散在五块板上。如果一次 LOAD 只在部分板成功，系统会进入难以定位的混合配置。项目因此在每板双银行持久化之上增加跨板事务，使参数切换能够验证、提交、应用、完成，并在复位或不确定写入后恢复到可解释状态。

1.4 总体技术贡献

围绕上述问题，本文总结四项面向赛题的工程机制：

1. 将三路独立视觉轨迹转换为统一车体系，在精确身份保持、全来源重新评分、EdgeHold、SEARCH 和可靠边界拒绝之间建立清晰次序；
2. 由 Core1 形成唯一运动意图，通过有线链路同时驱动飞行器和麦轮车，并在两端保留各自控制与安全 owner；
3. 对已建模的传感器、链路、协议、配置、身份和新鲜度异常实施分级降级，恢复前重新建立有效身份和零输出屏障；
4. 将五板 Live RAM、Runtime 和五个参数库槽分离，以持久协调日志组织可恢复的 SAVE/LOAD。

这些贡献不依赖“智能”“先进”等抽象形容词。后续章节将从实物、数据流、状态机、公式、源码结构和验证证据逐项说明其实现与边界。

1.5 报告结构

第二章给出任务阶段和五板总体方案；第三章说明飞行器、麦轮底盘和自研电路；第四章分析五板十核分工、共享内存和 Observer 专线；第五章说明飞行感知、速度估计和控制；第六章展开三视野目标连续性；第七章说明统一计划和麦轮闭环；第八章总结安全降级和恢复；第九章介绍参数事务、KVM、录制工具以及制作安装调试过程；第十章给出可复现实验与分层证据；第十一章归纳创新、局限和后续工作。

第2章 任务分析与总体方案

2.1 比赛任务的状态分解

系统运行不是从“开始”直接跳到“追踪”。上电后，五个生产控制节点分别初始化本地外设和双核共享区，再建立 Core0/Core1、Core1/Core2、Core1/Core3、Core1/Core4 链路。观察板必须完成协议 HELLO、能力校验、配置接收和帧边界应用，才能把轨迹事实纳入规划。飞行器进入起飞序列后，先完成执行器准备和缓升；只有达到规定任务高度并发布 altReached，地面车的 LAUNCH 门才可能打开。

任务运行阶段，Core1、Core3、Core4 三路信标视觉源持续产生候选。Core1 将候选变换到统一车体系，维护当前目标并生成 vx/vy/wz。计划一路发送至 Core2，另一路发送至 Core0。车辆进行麦轮逆解算、航向和电机闭环，飞机进行水平速度跟随、角度前馈和高度控制。目标暂时遮挡时，系统可以短时保持预测；目标真正越界时撤销旧轨迹；全来源均无候选后转入搜索。

安全状态贯穿整个流程。人工降落请求、必要链路失效、必需视觉故障、ToF 长时间失效和执行器保护不会被普通规划覆盖。停止或失效后，系统也不会因为收到一个新的非零帧立即恢复：MOTION 需要显式零帧 fence，Core2 从本地 MODE4 回到 RS-485 owner 时需要稳定期、新通信序号和一个零输出控制拍。

2.2 五板十核生产架构

生产系统由 Core0~Core4 五个 CYT4BB7 双核逻辑节点组成，每个节点使用两个 Cortex-M7 核，但两个核不共享模糊的“共同职责”，而是按照外设 owner 和实时负载分工。硬件实装中，Core0 与 Core1 对应的两颗 CYT4BB7 集成在同一张四旋翼飞行控制板上，Core2 位于随动车控制板，Core3 与 Core4 分别位于两张独立视觉处理板。因此，全文“板卡”在软件架构和参数事务语境中沿用项目节点命名；讨论 PCB、供电和器件时则按实际载板区分。

板卡	系统角色	cm7_0	cm7_1
Core0	无人机主控	飞行控制、IMU、光流、ToF、	灯板视觉、菜单、参数和本地

第二十一届全国大学生智能汽车邀请赛技术报告

板卡	系统角色	cm7_0	cm7_1
		电机和飞行序列	显示
Core1	视觉决策与通信中枢	Core0/Core2 有线链路、状态与计划派发	下视视觉、三来源连续性、规划、Core3/Core4 专线和远程交互
Core2	麦轮底盘	500 μ s 电机控制、编码器、电流、IMU 和本地菜单	RS-485、LoRa、协议解析和核间发布
Core3	后向观察板	4 Mbps Observer 专线、wire codec 和核间桥	后视相机、视觉、tracker、参数、显示和 KVM
Core4	前向观察板	4 Mbps Observer 专线、wire codec 和核间桥	前视相机、视觉、tracker、参数、显示和 KVM

CoreT3 也使用 CYT4BB7，但它是独立录制板，只负责采集 188×120 Gray8 原始帧并经 WiFi SPI 上传 PC。CoreT3 不参与比赛运行时的目标连续性、运动规划或执行器控制，因此不计入“五板十核生产架构”。

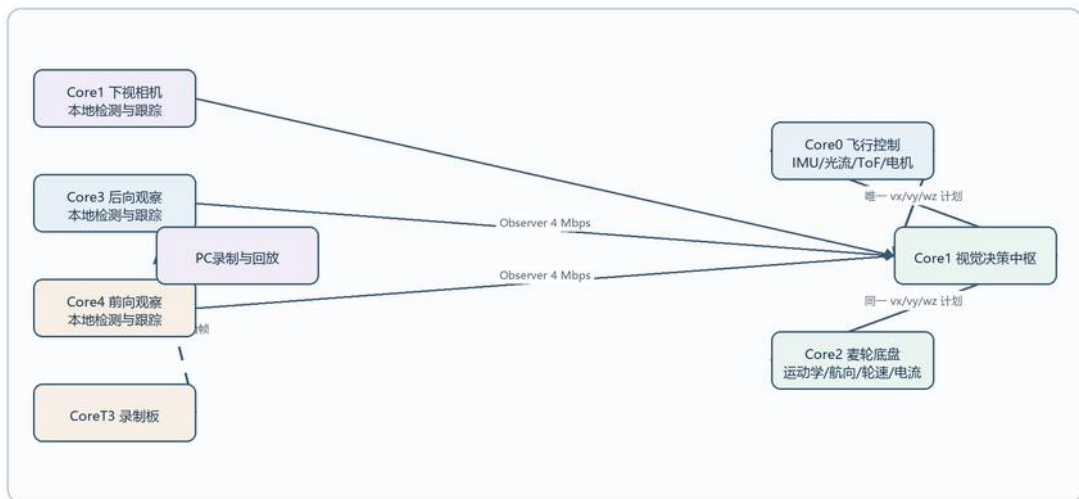


图 2.1 五板十核生产系统与独立录制链

2.3 三类数据流

系统内的数据可以分为感知事实、控制意图和安全事实。感知事实描述“观察到了什么”，包括 tracker 轨迹、相机状态、Core0 位姿和传感器测量；控制意图描述“希望如何运动”，主要是 Core1 产生的 vx/vy/wz 和 Core0 内部形成的姿

态、高度目标；安全事实回答“当前是否允许消费这些数据”，包括 boot generation、session、sequence、freshness、配置身份、有限数检查、飞行阶段和故障类别。

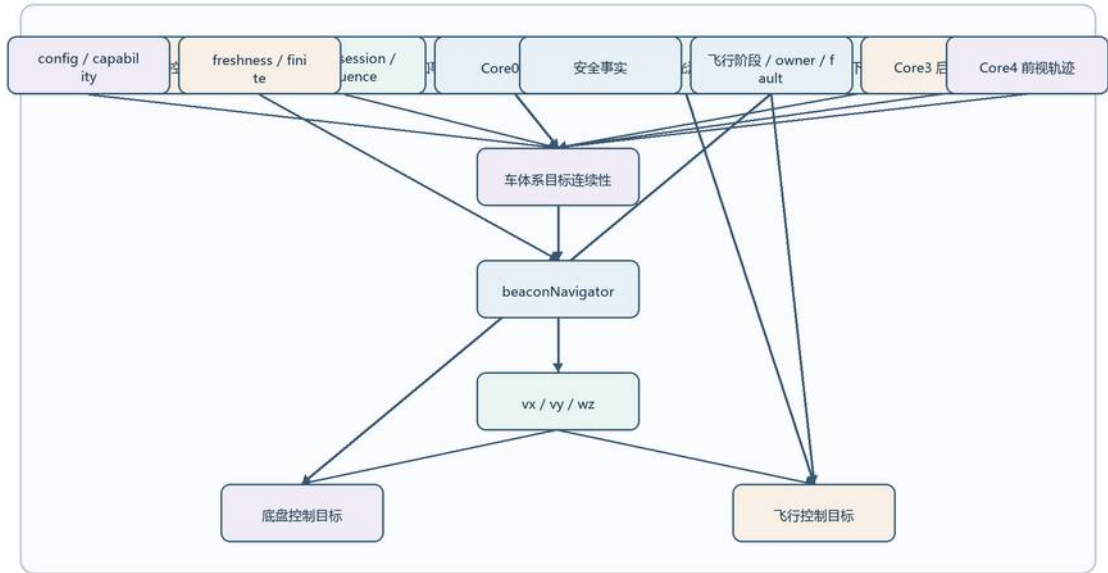


图 2.2 感知事实、控制意图和安全事实的数据流

2.4 设计原则

项目首先坚持单一 owner。相机、UART、执行器、Flash 和控制状态分别由明确模块初始化、更新和发布，调用方只通过 facade、消息或 snapshot 消费结果。这样可以从调用链上回答“谁能直接改变电机输出”“谁能宣布配置已应用”以及“谁能恢复控制权”。

第二个原则是结构化跨核。双核之间不返回仍在变化的内部对象指针，也不依赖一个 volatile 标志解决缓存一致性。共享区包含 magic、ABI、generation、sequence 等身份信息，并在发布和读取时执行 DCache Clean/Invalidate 及一致性检查。读取碰撞保留 last-good，但 last-good 不能自动满足安全新鲜度。

第三个原则是分离正常连续性与安全完整性。相机间正常切换尽量保留 slew 和 TD 状态，不人为制造零拍；而链路、配置、身份或安全资格变化必须归零并重建屏障。二者采用不同路径，避免为了“平滑”保留危险命令，也避免把普通换源当成故障停机。

第四个原则是证据不越级。当前源码可以证明机制存在，Host 回归可以证明给定输入下的状态机行为，IAR 构建可以证明指定配置可编译，只有绑定固件、板卡、接线和场景的记录才能形成硬件结论。本文对四类证据分别标注。

第 3 章 机械结构与自研硬件

3.1 飞行器结构

飞行器采用四旋翼布局。俯视图能够看到机臂、四个动力单元和中央电子系统的相对位置；侧视图则用于说明机载板卡、传感器、观察方向和线缆的高度关系。

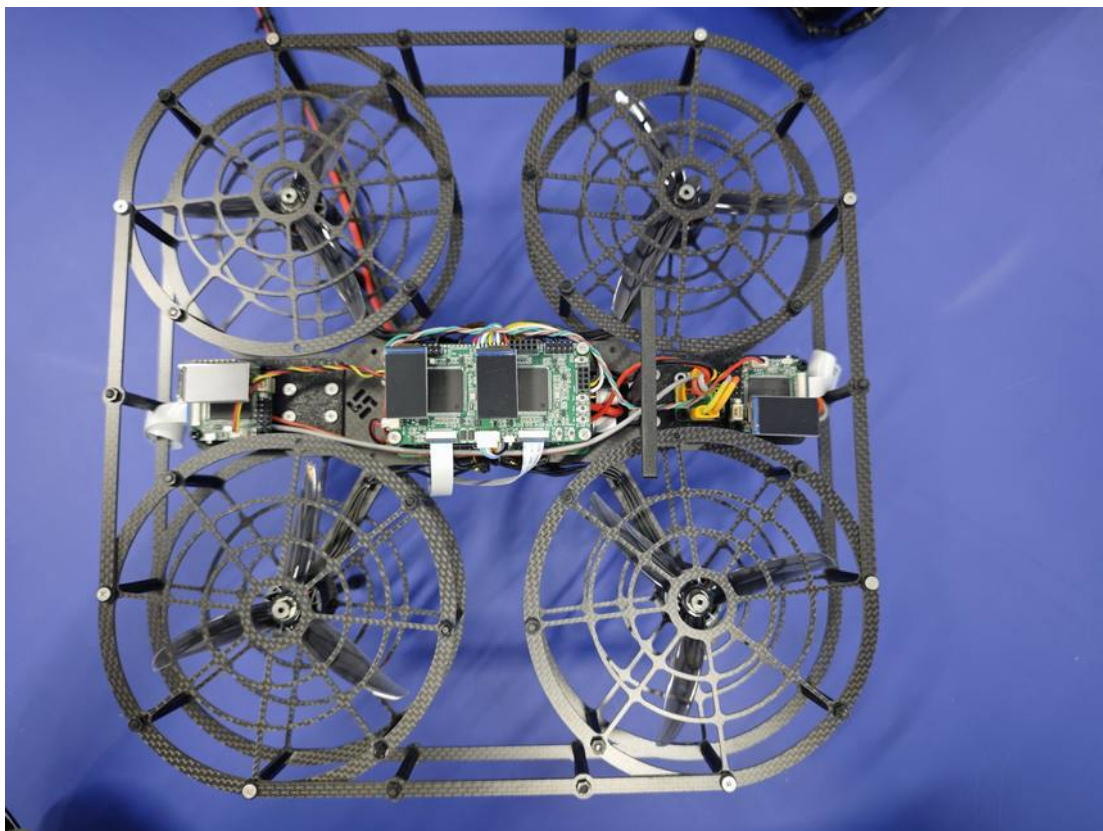


图 3.1 无人机俯视实物

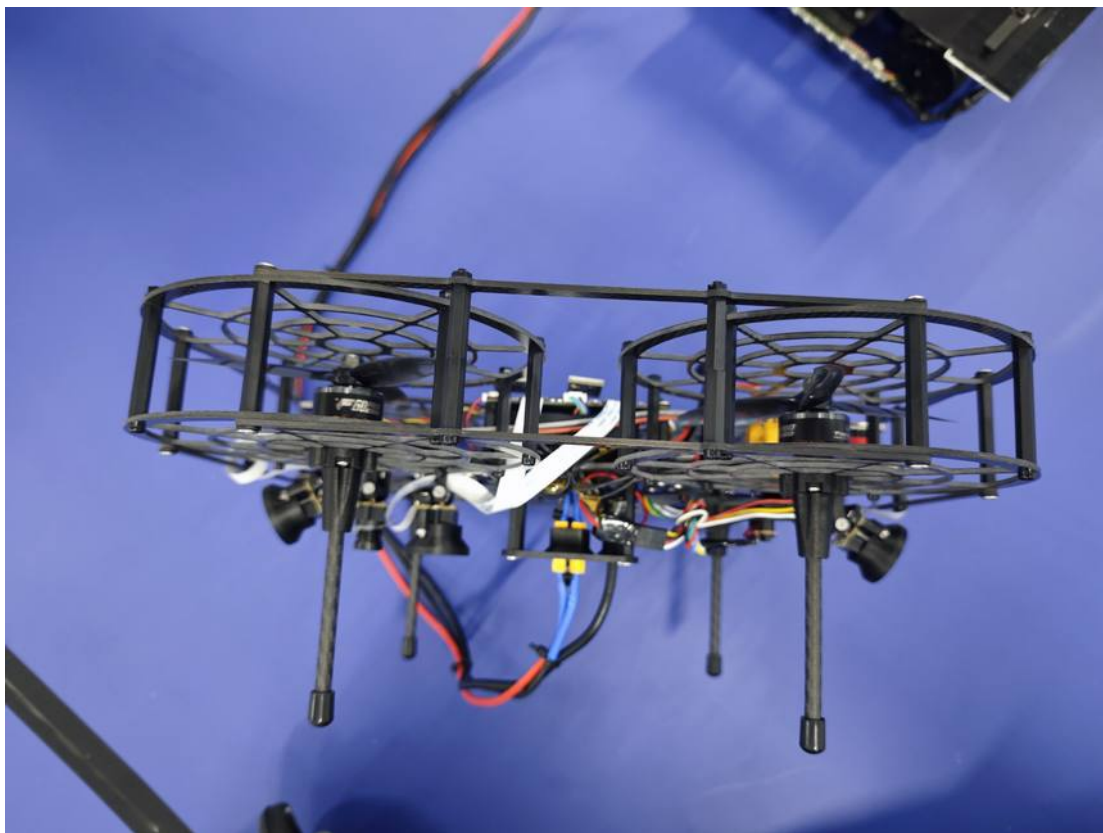


图 3.2 无人机侧视实物

实物采用碳纤维上下框架与立柱形成承力和防护结构，四个动力单元分别位于框架四角，圆形桨叶保护罩通过螺钉与隔离柱固定。主飞控板布置在中央层板，观察板分置于前、后观察方向；板卡使用螺钉和隔离柱保持电气间隙，排线与插接线束沿框架内侧布置并用束线件约束。侧视图可见上下保护层、中央电子层和起落支撑相互分离，使桨叶平面不直接穿过控制板与线束区域。该布置将质量较大的控制与电源部件集中在几何中心附近，降低偏置安装对姿态控制和线缆摆动的影响。

中央控制部分由主飞控板承担飞行控制、传感器和多板接口。该板集成 Core0、Core1 两颗 CYT4BB7：前者连接飞行执行器、BMI270、VL53L8CX ToF、LC-302 光流及一路本地视觉，后者连接下视信标相机、RS-485 和两路观察板专线。四合一电子调速器把四路动力驱动集中在一块板上，前、后观察板与四旋翼筒易分电板构成扩展视野单元。飞行器使用四个 F60PRO 无感无刷电机和四副 5 英寸 3 叶桨，电机由自研四合一无刷电调驱动。申请材料 and 硬件工程用于确认装机构成。^[9]

第二十一届全国大学生智能汽车邀请赛技术报告

队伍调试记录中，单个 F60PRO 在对应桨叶和电源条件下取得约 1700 g 静态推力、约 40 A 电流；四合一电调完成过四路满指令运行，正常比赛任务很少持续处于满输出。由于没有保留同步电压、电流、转速和推力原始曲线，本文不把该记录扩展为持续额定推力、电调连续载流能力或整机效率结论。

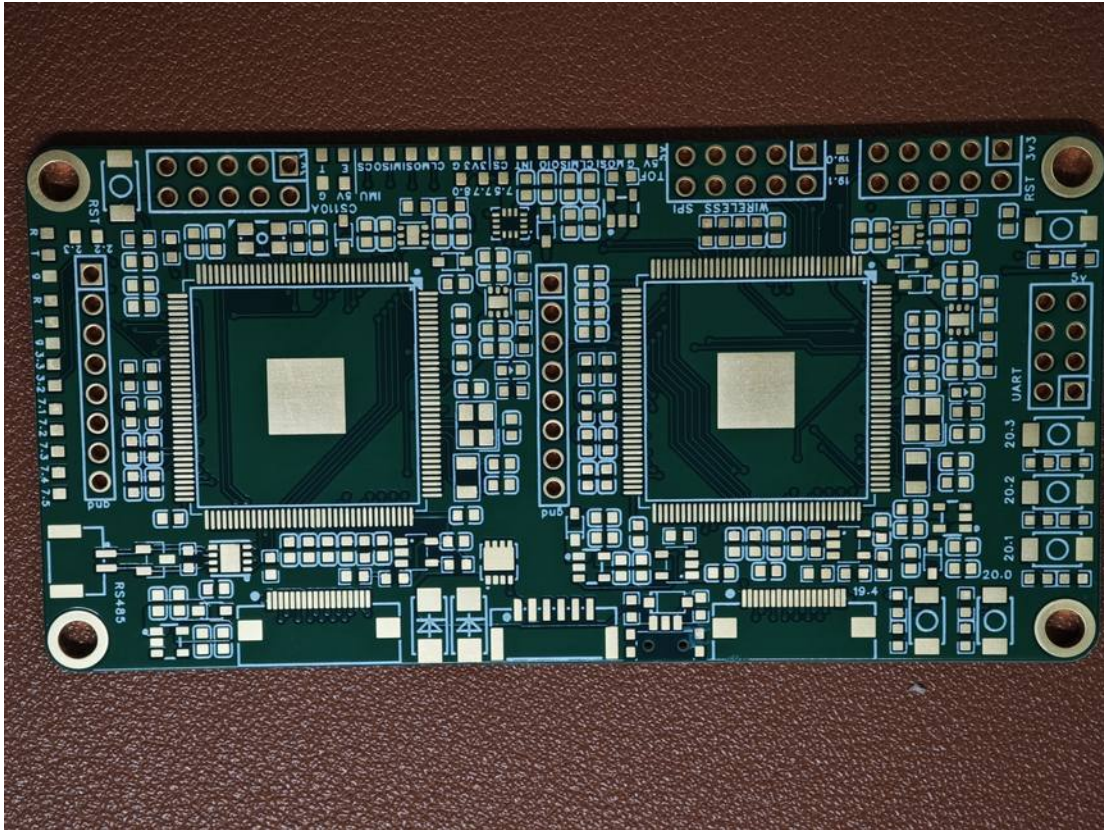


图 3.3 主飞控板正面实物

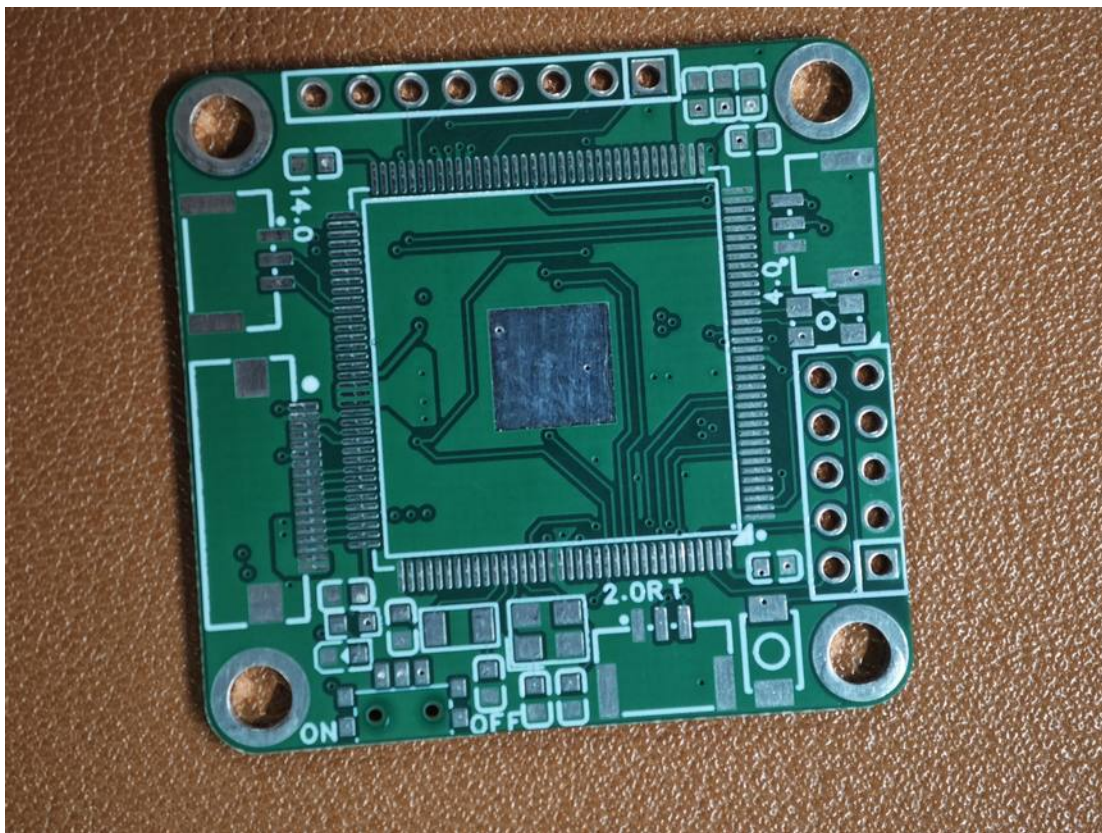


图 3.4 前向、后向观察板所用硬件

3.2 麦轮底盘结构

地面车使用四轮麦克纳姆轮。四个轮子的滚子方向按全向运动学要求布置，使车体能够在保持朝向的同时产生纵向和横向速度。底盘主控板、电机驱动板、分电板和灯板固定在车架上，编码器与驱动电机形成轮速反馈。

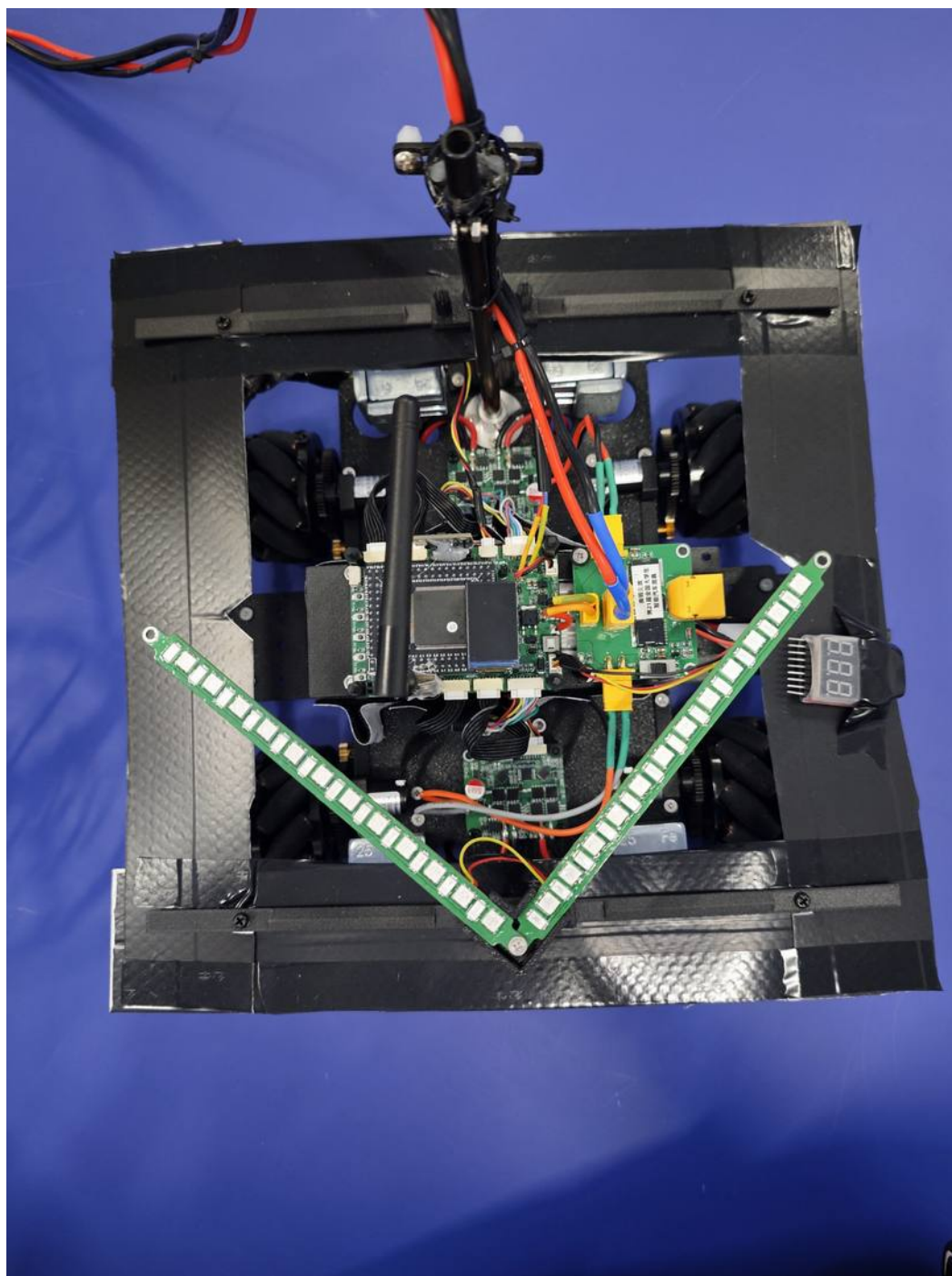


图 3.5 麦轮底盘俯视实物

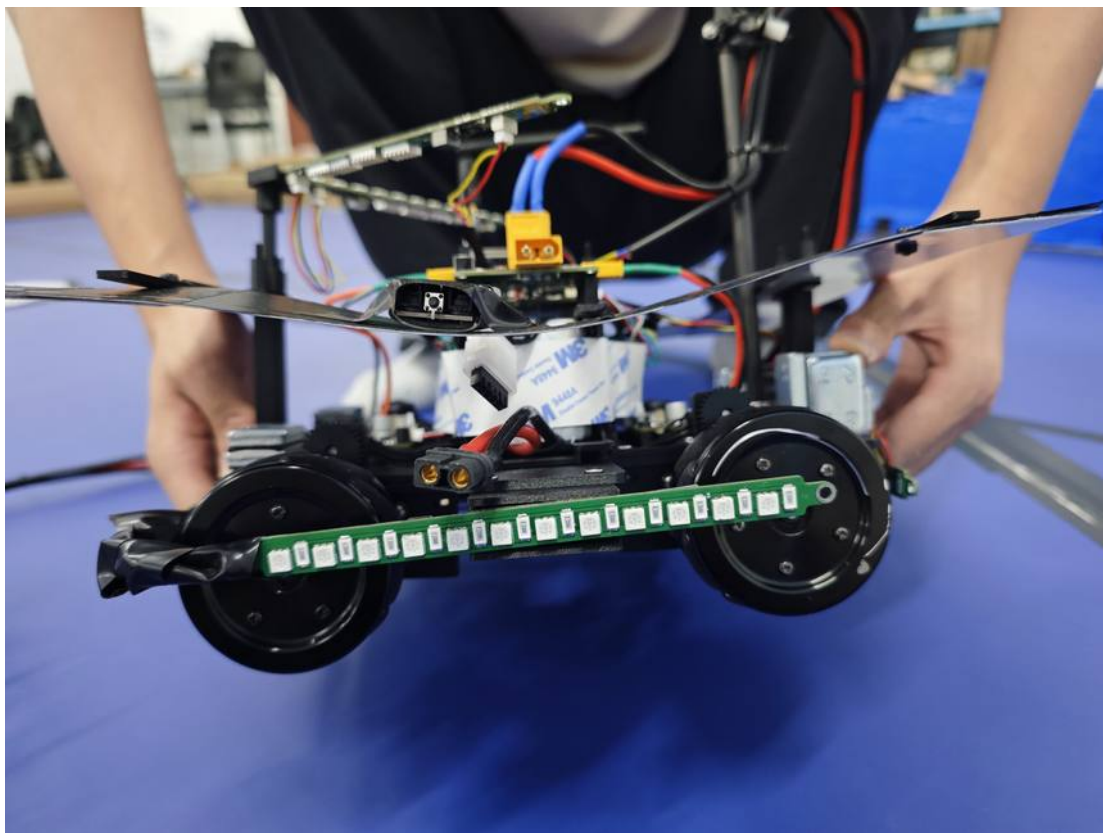


图 3.6 麦轮底盘侧视实物

底盘使用上下层板和立柱形成电子舱，四个 RC380 减速电机固定在车架四角，麦轮滚子方向按 FL、FR、BL、BR 运动学符号成对布置。底盘主控板、总分电板和两张双路驱动板位于车体内部，采用螺钉、隔离柱或板级固定孔与底板连接；大电流动力线使用独立插接端子，编码器和控制线束通过小型插座接入，并与轮胎旋转空间保持分离。车体灯板固定在外缘，既便于飞行器视觉观察，也不占用中央动力和控制器件的安装空间。系留连接从车体上方引出，电源与 RS-485 信号在接口处保持明确端点，避免把临时飞线作为正式连接方式。

底盘主控板的 cm7_0 直接管理 PWM、编码器和电流采样，cm7_1 管理 RS-485 与 LoRa。两块二合一有刷电机驱动板合计驱动四个博思威龙 RC380 有刷电机，车轮反馈来自四路 1024 线正交抗磁编码器。每轮控制采用电流内环串联速度外环：速度环根据轮速误差给出电流目标，电流环根据采样反馈形成 PWM 输出；所有模式最终经同一 `motorControlOutput()` 出口作用于执行器。总分电板为电池、底盘控制板、驱动板和系留供电提供集中端点。ILD8150 灯条构成车体光学标志，而不是环境目标传感器；其原理图和 PCB 图纳入硬件附

录。433 MHz LoRa 仅用于异常情况下的独立紧急制动，不参与正常控制，也不承担车—机通信。

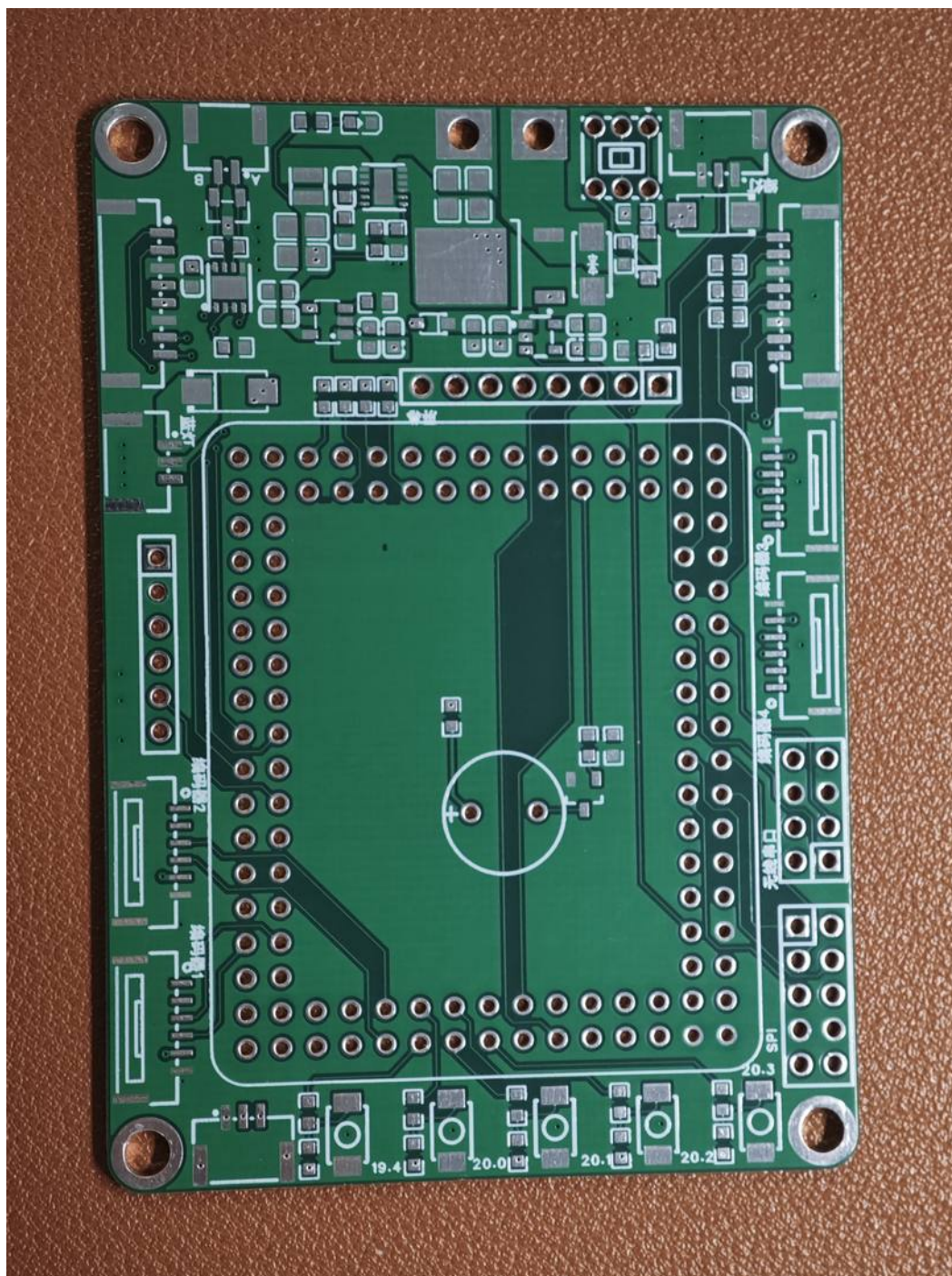


图 3.7 底盘主控板正面实物

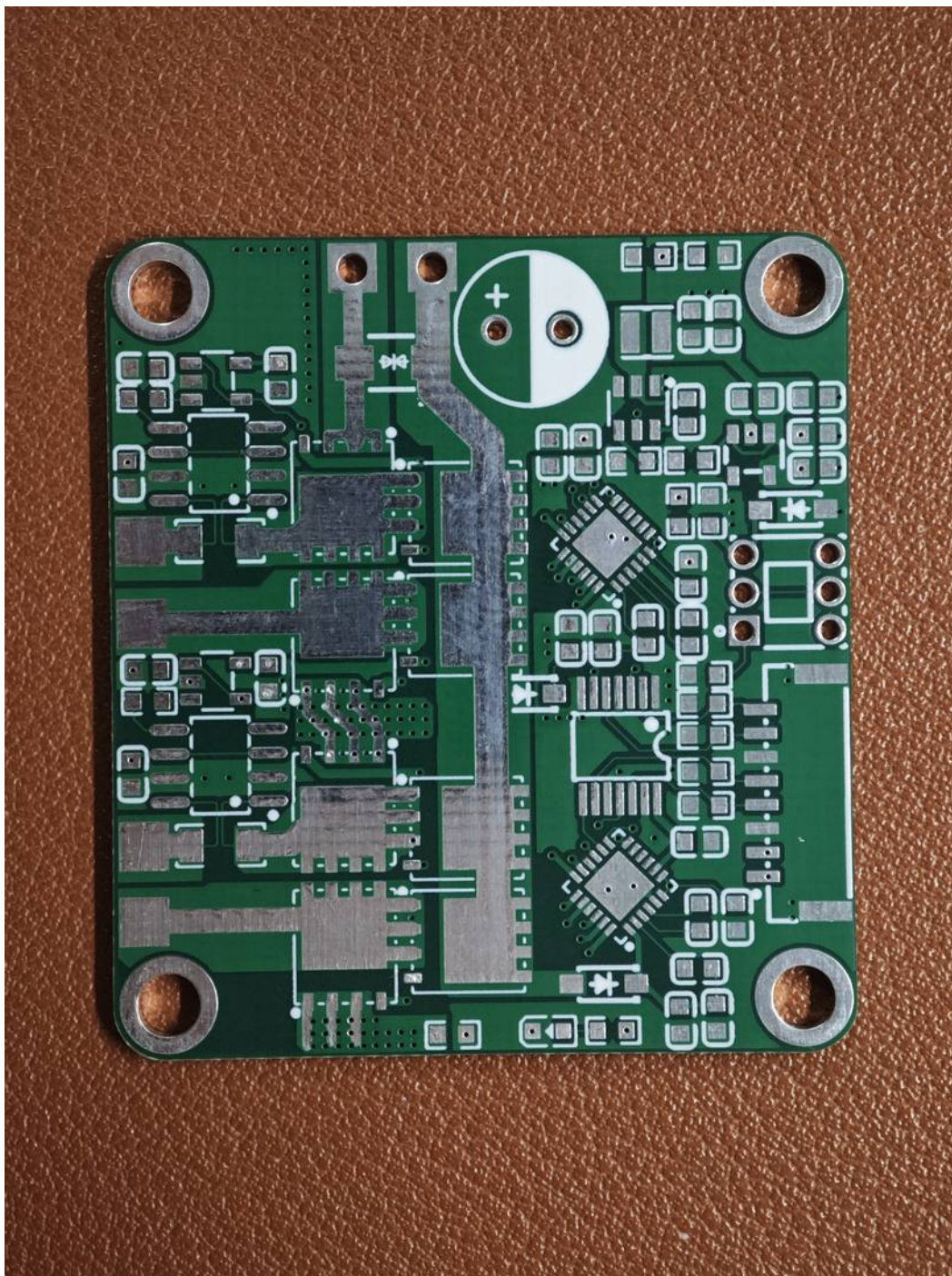


图 3.8 二合一有刷电机驱动板正面实物

3.3 自研电路系统

本项目的硬件不是围绕一块通用核心板简单接线，而是按飞行控制、动力驱动、视觉观察、底盘控制和电源分配形成板卡族。嘉立创 EDA 专业版工程包

第二十一届全国大学生智能汽车邀请赛技术报告

含 8 套板级设计和 1 个拼板文档；同一设计可以有多张实装板，例如双路有刷驱动板和视觉处理板均各使用两张。板级组成、主要器件型号和实装数量与开源奖项申请材料交叉核对。^[9]

子系统	板卡	主要职责
飞行控制	四旋翼飞行控制板	集成 Core0/Core1 两颗 CYT4BB7、BMI270、相机/光流/ToF 接口、四路电调 PWM 与跨板通信
飞行动力	四合一无刷电子调速器	四个 AI8051U 控制通道及四组三相功率级
视觉观察	视觉处理控制板×2	每板一颗 CYT4BB7 和一路 MT9V034，承担前、后观察与 4 Mbps 专线
飞端配电	四旋翼简易分电板	电池母线向两张视觉处理板分配供电
底盘控制	随动车控制板	编码器、电流、PWM、IMU660RB、RS-485、LoRa 安全输入和本地交互
底盘驱动	双路有刷电机驱动板×2	每板两路 DRV8701 全桥，合计驱动四个 RC380 电机
总电源分配	总分电板	4S 电池输入、缓启动及底盘、飞行器支路分配
车体光学标志	ILD8150 灯条	蓝、绿 LED 恒流驱动及 PWM/DIM 调节

正文选取主飞控板和底盘主控板原理图说明功能分区，完整多页原理图与 PCB 各层放入附录。

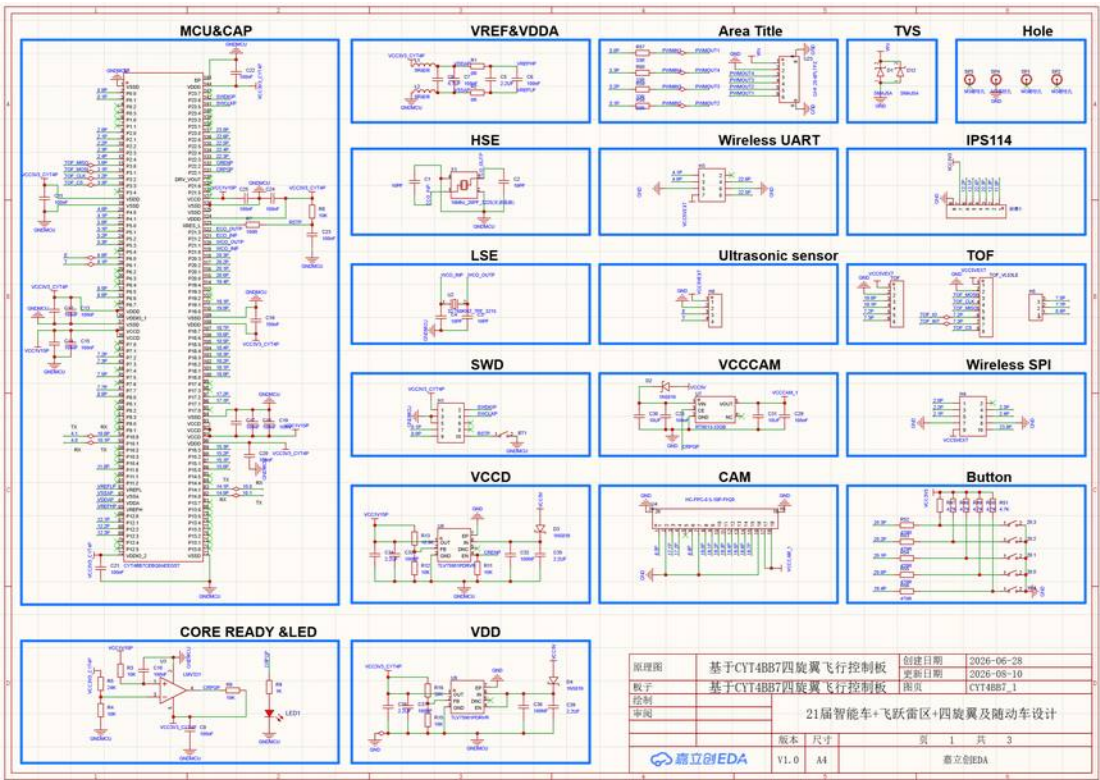


图 3.9 主飞控板原理图第一页

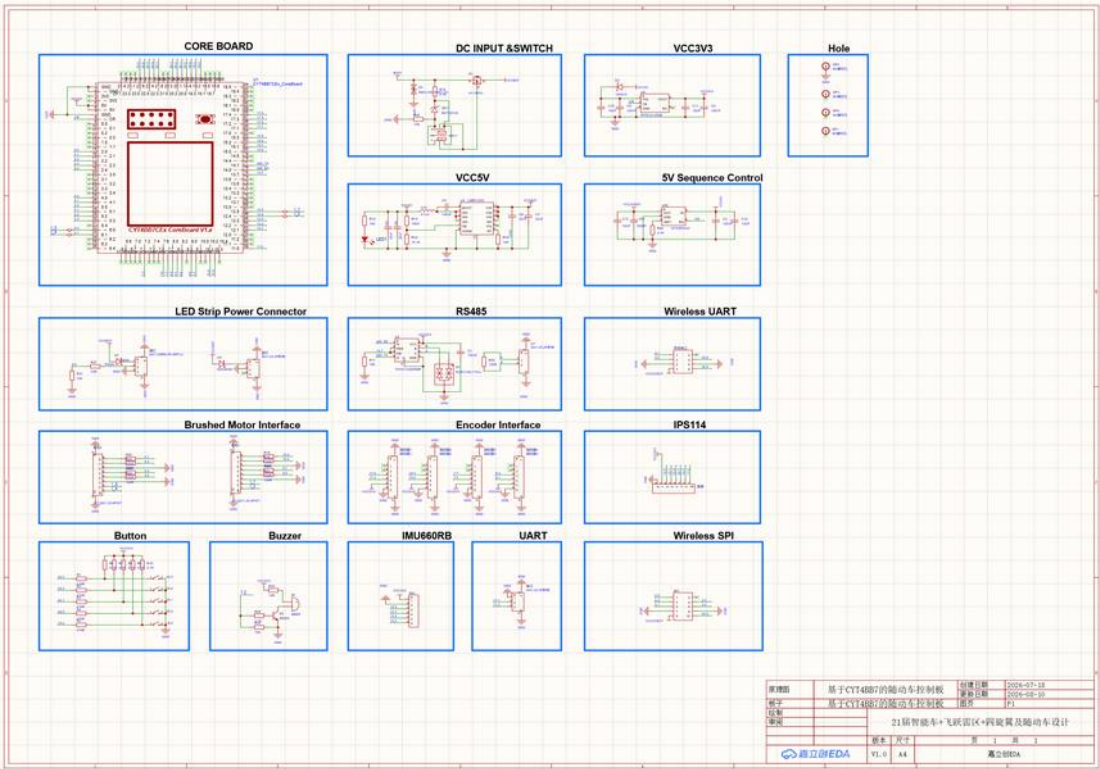


图 3.10 底盘主控板原理图

3.4 控制、通信与传感器电路

随动车控制板以 CYT4BB7 为主控。4S 电池输入端设置 TVS、MOSFET、稳压二极管和按键控制网络；LMR51450 生成 5 V，RT9013-33GB 生成 3.3 V，SY6280AAC 控制外部 5 V 支路。板上引出四路编码器、两张双路驱动板、IMU660RB、IPS114、按键、蜂鸣器和通用串口。车—机链路采用 THVD1450DRBR 差分收发器，并以 NUP2105L1TG-L 进行总线保护；该 RS-485 链路是随动车与飞行器的正常通信通道。

四旋翼飞行控制板集成两颗 CYT4BB7CEBQ0AEEGST。两颗处理器各自具有相机 FPC 与 VCCCAM 供电，分别处理一路 MT9V034；Core0 侧还连接四路电调 PWM、VL53L8CX ToF、LC-302 光流、屏幕和按键，Core1 侧连接 RS-485、两路观察板 UART、RGB 和本地交互。处理器电源使用 TLV75901PDRVR，板载 BMI270 位于独立 VCCIMU 电源域，由 RT9013-33GB 生成 3.3 V 并通过 SPI 连接。模拟地与数字地经 0 Ω 电阻单点连接，用于控制功率回路对惯性测量的耦合路径。

两张视觉处理控制板各集成一颗 CYT4BB7 和一路 MT9V034，并提供 18 针 FPC、串口、屏幕接口以及 LMR51450、TLV75901、RT9013 等电源器件。由此，飞行器共有四路实体相机：Core0 本地相机用于车体光学标志视觉，Core1 下视、Core3 后视和 Core4 前视三路相机才进入第 6 章的信标目标连续性。实体相机总数为四，不应把“三视野连续性”误写成整机只有三台相机。

按整机功能统计，传感器包括 MT9V034 相机 4 台、IMU 2 套（飞行器 BMI270 与底盘 IMU660RB 各 1 套）、VL53L8CX ToF 1 套、LC-302 光流 1 套以及四轮正交编码器 4 套。底盘驱动还具有 4 路电机电流采样；这里将物理传感器和电流采样通道分开计数，避免把一张多通道驱动板误写成一个反馈量。地面车上的 IMU、编码器和电流采样只服务本车姿态与执行器闭环，不构成对赛场环境的感知。

3.5 动力驱动、电源分配与保护

每张双路有刷电机驱动板使用两颗 DRV8701ERGET；每个通道由四颗 TPH1R403NL N 沟道 MOSFET 组成全桥，单板共八颗功率 MOSFET。桥臂低阻分流信号由 INA240A2EDRQ1 差分放大，再经 RC 与 TL081HIDBVR 有源低通

网络送入控制器，为第7章的电流采样和级联闭环提供硬件入口。驱动板另以TPS54302生成5 V，并以ME6231C33生成模拟3.3 V电源域。器件库中的耐压、电流和导通电阻是元件额定或特定测试条件数据，不等同于整板连续工作能力。

四合一无刷电调的每个通道采用一颗AI8051U-34K64-QFP48控制器、一颗FD6288Q三相栅极驱动器和六颗NCEP4090GU功率MOSFET，并以2.5 mΩ分流电阻和INA199A1DCKR形成母线电流采样。LMR51450支路向主飞控板提供5 V，TPS54302分别生成板载5 V和10 V。功率区采用宽1.5 mm、厚1 mm的焊接铜条，并预留散热片和风扇供电接口。该电调参考lcet7122发布的“STC电调（AM32固件移植）”，复用了面向AI8051U的引脚配置和驱动程序后按四合一形式修改；公开与提交时应保留来源署名，并继续核对相关附件的许可证和再发布要求。^[10]

整机以4S电池为总电源。总分电板通过XT60、SMCJ18CA以及MOSFET、三极管、稳压二极管和RC网络完成输入保护与缓启动，再向随动车控制板、两张有刷驱动板和四合一电调分配电源；电调同时向主飞控板提供5 V，并把电池母线引至四旋翼简易分电板，后者经四个XT30AW接口和SMAJ20CA保护向两张视觉处理板供电。灯条采用ILD8150XUMA1、47 μH电感和SS54构成降压恒流驱动，通过PWMIN/DIM调节蓝、绿LED状态。

表 3.1 汇总申请材料引用的关键器件额定信息，用于说明选型范围和电路拓扑，不作为整板安全额定值

器件	申请材料引用的器件级参数	在本系统中的用途
THVD1450D RBR	3~5.5 V，最高 50 Mbps	系留 RS-485 差分物理层
DRV8701ERG ET	47 V H 桥栅极驱动器	随动车有刷电机全桥驱动
TPH1R403NL	30 V；器件库标注 150 A、3.3 mΩ 特定条件值	有刷电机全桥功率管
NCEP4090GU	40 V；器件库标注 90 A、2.2 mΩ 特定条件值	无刷电调三相半桥功率管
ILD8150XUM A1	8~80 V、最高 2 MHz、1.5 A 器件额定值	车体 LED 灯条恒流驱动

第二十一届全国大学生智能汽车邀请赛技术报告

器件	申请材料引用的器件级参数	在本系统中的用途
LMR51450	4~36 V、最高 5 A 器件额定值	控制板与电调降压电源
TPS54302DD CR	4.5~28 V、5 V/3 A 器件额定值	驱动板及电调辅助电源

申请材料提示，少数器件的图纸显示值、制造商型号与器件库描述可能存在不一致。报告因此以原理图实际参数和最终 BOM 为裁决依据，并在制造前再次核对采样电阻、电容、功率电感和替代料；上表不能替代热设计、铜厚、连接器、散热和整板试验形成的降额边界。

3.6 机械、电气与观察方向协同

三路相机的安装方向直接决定后续几何模型。Core1 下视相机随飞机航向变化，其动态中心由 Core0 位姿和 Core1 offset 共同确定；Core3 后向相机使用固定观察外参；Core4 前向相机在固定外参基础上结合当前 heading。三个来源因此不能只通过“统一旋转图片”获得等价坐标，而必须分别保存 scale、mount yaw、镜像和前/右符号。

光流与 ToF 的安装也进入控制计算。光流速度需要考虑安装映射、俯仰/横滚角速度以及偏心位置引起的 yaw 切向速度；ToF 斜距需要根据飞机姿态换算垂直高度。机械装配不是软件之外的静态背景，而是估计器和规划坐标的参数来源。

空地短线缆同时承担供电与 RS-485 数据连接。申请材料记录随动车投影尺寸为 30 cm×30 cm、四旋翼外形为 35 cm×35 cm×20 cm、保护罩网眼最大宽度小于 40 mm，以及 4S 7000 mAh 80C 电池、满电 16.8 V 和单次连续运行约 2 min；这些数据作为现有装机材料列入报告。^[9]

队伍装机记录给出飞行器质量 870 g、底盘质量 2500 g、系留线质量 100 g，整套空地系统总质量为 3470 g。整车所有电路板上离散电容的名义容量合计约 2000 μ F，统计范围不包括电池等效容量和电机绕组。系统常用输入功率约 600 W；飞机与底盘进行大幅换向等剧烈机动时，瞬时峰值约 1500 W。由于没有保留同步电压电流原始曲线，这些功率只作为队伍运行记录，不用于证明效率、续航裕量、持续 1500 W 能力或整板连续安全额定值。队伍同时确认，报告

所述板上配置已经随当前固件烧录并能够完成比赛流程；本轮没有附加原始完赛日志，因此该结论只覆盖队伍确认的装机、配置和比赛流程，不外推为所有场地与故障组合均已实测。

按细则中的附加执行器口径，整机额外伺服电机数量为 0。飞机的四个 F60PRO 无感无刷电机和底盘的四个 RC380 有刷电机均属于主动力执行器，不计入“除原有驱动电机、舵机之外的伺服电机”。三路生产信标视觉均使用 188×120 输入，图像坐标的标称离散分辨率为 1 像素；板上相机配置为 50 fps，对应 20 ms 帧周期，Core1 规划周期为 10 ms，对应 100 Hz 计划更新上限。配置值由固件证据基线源码复核，像素定位误差由固定合成真值夹具给出；二者共同作为报告检测指标，但不换算成未经标定的赛场毫米误差。

第4章 五板十核分布式架构

4.1 以所有权和实时性划分双核职责

系统的五个 CYT4BB7 生产控制节点均采用双 Cortex-M7 处理器，由此形成项目所称的“五板十核”逻辑架构；其中 Core0 与 Core1 两颗器件共载于四旋翼飞行控制 PCB。处理器、缓存和外设能力依据 TRAVEO T2G 技术资料核对。^[6] 双核不是为了把同一段业务任意拆成两半，而是用于隔离控制、通信、视觉、存储和交互等时间特征不同的任务。Core0 的 cm7_0 独占 IMU、光流、ToF、四路飞行电机和飞行安全状态，cm7_1 负责本地视觉、参数 Flash、按键和显示；Core1 的 cm7_0 负责跨板有线通信和消息派发，cm7_1 负责下视视觉、三来源协调和运动规划；Core2 的 cm7_0 负责麦轮底盘的周期控制，cm7_1 负责 RS-485、LoRa 和模式通信；Core3 与 Core4 则分别把 Observer 链路和相机视觉放在不同处理器上。

这种划分首先维护硬件唯一所有者。相机、串口、Flash、执行器和显示器均只由一个处理器完成初始化、ISR 和直接访问，另一核通过公开 facade、快照或队列获得结果。其次，它把高频控制与耗时不确定的业务隔离：图像处理、Flash 擦写、LCD 刷新和协议格式化不会进入飞控或底盘控制中断。最后，它使系统能够分别描述“链路是否在线”“业务事实是否新鲜”“配置是否真实应用”和“控制是否放行”，不把一个粗粒度在线标志扩展为整机健康结论。

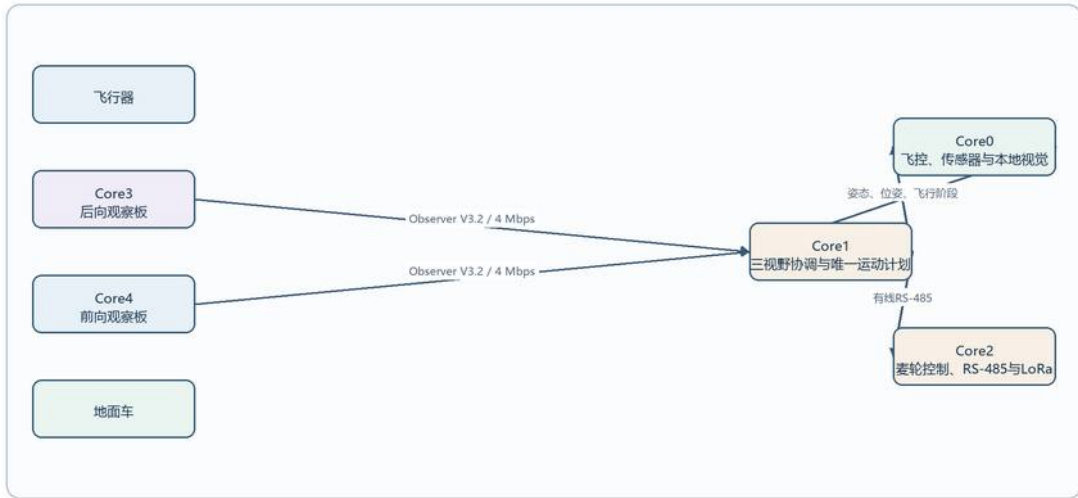


图 4.1 五板十核职责与主要数据流

CoreT3 录制板不计入上述五板十核生产架构。它用于从 MT9V03X 采集原始图像并通过 WiFi SPI 上传至上位机，以形成可复现的视觉素材；其掉线不会进入生产控制链，也不能被当作生产系统的冗余视觉节点。

4.2 核间共享快照与有界事件队列

CYT4BB7 两个 M7 核均启用数据缓存。若发布端只修改本核 Cache 而没有清理到共享存储，另一核可能读到旧数据；若读取期间发布端更新了结构体，还可能获得由两个时刻字段拼成的撕裂快照。各板因此将共享区访问集中到板级数据传输 facade，不允许业务代码直接持有共享地址。

共享快照头包含 magic、ABI 版本、payload 长度、producer 启动代次和发布序号。区域按 Cache Line 对齐，发布端先填写 payload，再执行 DCache Clean，最后提交发布身份；读取端先 Invalidate 并探测头部，仅在启动代次或序号变化时读取完整 DTO，再次读取头部以确认前后身份一致。读取碰撞返回失败，调用方按业务语义保留 last-good 或等待超时，不能把碰撞伪造成一份“合法全零”事实。

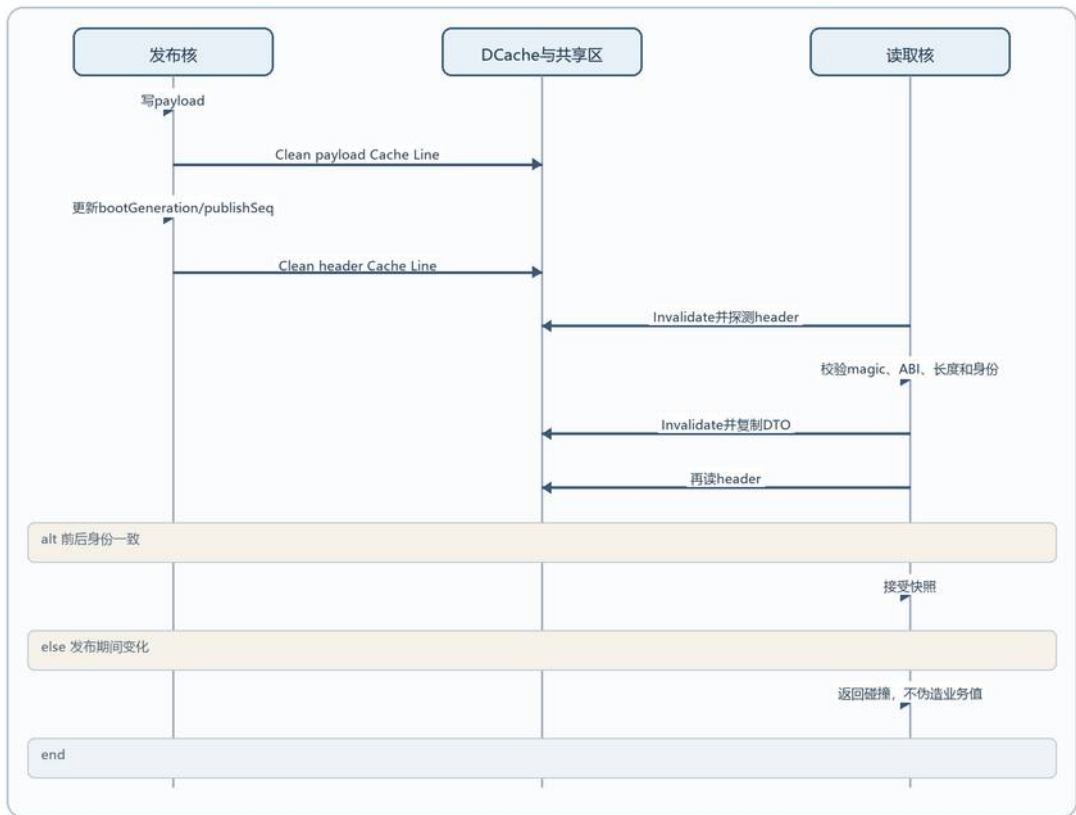


图 4.2 一致性共享快照时序

对 CONTROL、EVENT、ACK、MENU_STATUS 和边界拒绝等不可被“最新值覆盖”替代的边沿事件，系统采用双向独立的有界 SPSC 队列。队列通过逐槽序号和 commit 标志区分已发布与未完成写入，producer 重启不会擅自改写 consumer 的消费位置。物理绑定边沿另用 bindingEpoch 表示，即使设备重启后 session 数值偶然复用，旧 KVM 租约、配置确认和轨迹资格也不能跨绑定继承。

4.3 Observer V3.2 双独立专线

Core1 通过两条相互独立的 4 Mbps UART 专线分别连接 Core3 和 Core4。两条 endpoint 不共享 session、接收环形缓冲、分片重组状态或发送预算，因此一条链路的重启、积压和非法帧不会直接污染另一条链路。UART 接收中断只搬运字节到有界环形缓冲；帧解析、配置处理、轨迹分片和发送调度均在前台完成。

Observer V3.2 建链时先交换 HELLO，并核对协议主次版本、来源编号、角色和 capability。在线状态下，延迟到达且 session 不同的 HELLO 不能以“数值更大”为由抢占当前 peer；只有进入 600 ms 离线栅栏后，新的合格 HELLO 才能重新绑定。每个业务包携带推进的 packet sequence，重复包和旧包既不更新业务事实，也不延长新鲜度。

Core1 是共享视觉配置的 owner，分别向 Core3 和 Core4 发送公共算法配置与本来源 profile。观察板先暂存配置，在下一次成功取得相机帧的边界组合应用；只有真实应用后的 revision、hash、DetectMode 和来源身份完全匹配时，才回复 APPLIED。因此，串口在线不等于算法配置已生效。

轨迹采用 planning snapshot 分片传输。一个 snapshot 在发送期间冻结，所有分片携带相同的来源、session、配置 tuple、tracker 帧序号、分片总数和记录总数。Core1 完整收齐后检查分片一致性、轨迹数量、重复 ID、浮点有限性和状态组合。缺片、冲突、越界数量或非法拒绝记录均使该 snapshot 无效。边界拒绝记录只携带原 trackId 与 BORDER_REJECTED 状态，不伪造坐标，也不占普通目标发布配额。



图 4.3 Observer 建链、配置和轨迹发布时序

重连后旧事实不会自动复活。新 peer 必须重新完成 HELLO、取得精确的 CONFIG/APPLIED、发布新鲜 HEALTH 并形成新的完整 planning snapshot。配置不匹配、相机超时、核间 bridge 停更和 session 变化分别保留为不同故障类别。Core1 在资格重建期间保持计划无效或输出为零。

本章的 owner 划分、共享快照、一致性读取和 Observer 状态机属于 CODE-CONFIRMED。Observer codec 与指定历史提交已有 Host 回归和 IAR 构建证据，但尚不能证明当前整机在双链并发、长期 Cache 竞争或频繁复位下无丢包、无超时，相关性保持 NEEDS-EVIDENCE。

第 5 章 飞行感知与控制

5.1 多速率串级飞控

Core0 是飞行控制的唯一执行 owner。其 cm7_0 汇总姿态、速度、高度、视觉误差和外部运动意图，经单一入口形成电机输出；视觉核只发布观测和参数快照，不直接修改飞控内部状态。飞控采用多速率串级结构：角速度环和四旋翼混控以 1 ms 周期运行，俯仰/横滚姿态角环和偏航计算以 3 ms 周期运行，水平速度环以 9 ms 周期运行，视觉位置外环以 27 ms 周期运行。高频内环优先抑制姿态扰动，较慢的外环只生成参考量，避免在每个 1 ms 控制拍重复运行全部算法。

Mahony 六轴姿态解算直接使用校准后的原始角速度，以减少额外滤波的相位滞后；角速度 PID 使用 1 ms 一阶低通后的角速度，降低高频噪声进入电机控制量。四元数转换得到物理俯仰角和横滚角，供姿态保护、ToF 倾角补偿和速度估计共同使用。控制参数统一从运行时参数对象读取，并在安全状态允许时应用，调用层不能绕过参数 owner 直接改写 PID 字段。

外部引导通过 pidFlightControllerSetGuidanceCommand() 单次提交，同一命令携带视觉位置误差、KF 水平速度反馈、MOTION 速度前馈以及姿态前馈。飞行序列协调器先完成任务阶段、外部运动和安全门仲裁，再将同一拍的最终结果交给控制器，避免位置、速度和前馈来自不同时间代。

设位置外环给出的前向、右向速度目标为 $(v_{p,f}, v_{p,r})$ ，外部运动前馈为 $(v_{ff,f}, v_{ff,r})$ ，则速度环的合成目标为

$$v_{f*} = \text{sat}(v_{p,f} + v_{ff,f}, v_{\max}), \quad v_{r*} = \text{sat}(v_{p,r} + v_{ff,r}, v_{\max}). \quad \text{公式 1}$$

横滚物理正方向与右向速度正方向相反，因此横滚速度环输出取反后构成负反馈；俯仰前倾与前向速度同号，不作相同反转。最终姿态参考为

$$\varphi^* = \varphi_0 + \Delta\varphi_v + \varphi_{ff}, \quad \theta^* = \theta_0 + \Delta\theta_v + \theta_{ff}. \quad \text{公式 2}$$

速度估计短时无效时，速度环暂时保持上一拍角度修正；持续无效后修正归零，使飞行器退回基准姿态与定高控制，而不长期消费陈旧速度。混控以基础油门和定高偏移之和作为推力基准，再叠加俯仰、横滚和偏航控制量：

$$\begin{aligned} u_{FL} &= u_b - u_p - u_r + u_y, \quad u_{FR} = u_b - u_p + u_r - u_y, \quad u_{BL} = u_b + u_p - u_r - u_y, \\ u_{BR} &= u_b + u_p + u_r + u_y. \end{aligned} \quad \text{公式 3}$$

混控结果还需经过飞行状态门和输出范围限制，才能写入电机 PWM。

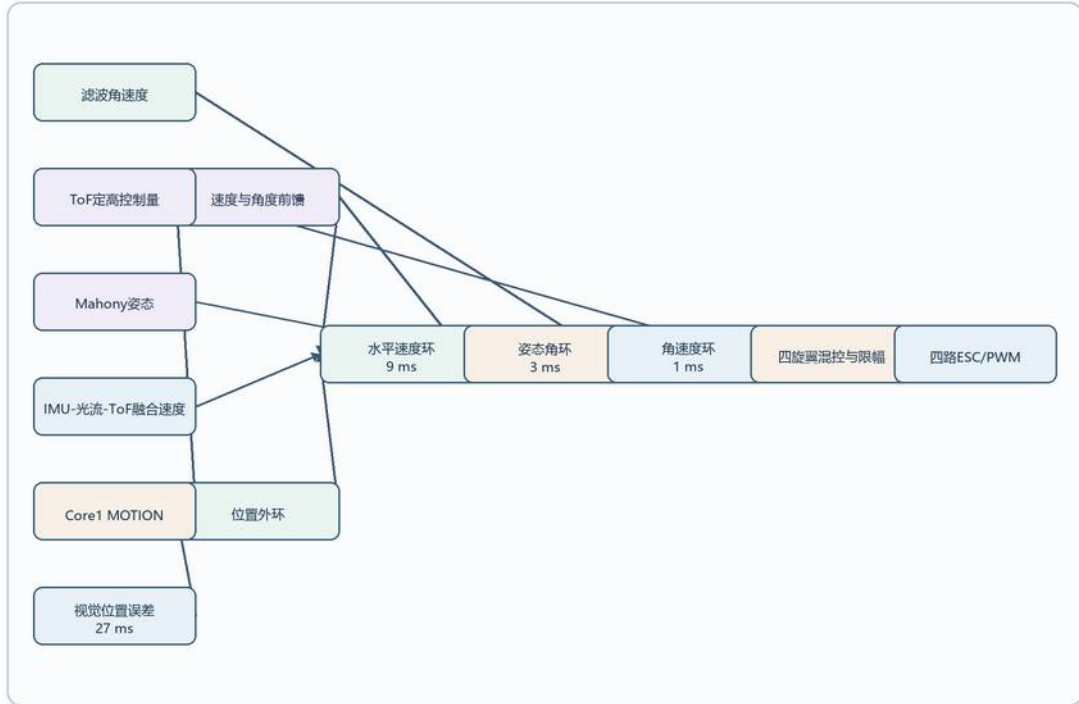


图 5.1 多速率飞行控制结构

5.2 IMU—光流—ToF 水平速度估计

水平速度不是由单一传感器直接给出。IMU 更新快但积分会漂移；光流提供相对地面的运动量，却依赖高度、姿态、安装方向和传输时延；ToF 提供尺度，但会受到倾斜、野值与短时失帧影响。Core0 因此采用“IMU 高频预测、光流低频校正、ToF 提供尺度和资格”的融合路径。

IMU 预测以 1 ms 节拍执行。程序先根据四元数计算传感器系重力方向，再从实测加速度扣除重力：

$$a_{lin,s} = a_s - g \cdot g_s(q). \quad \text{公式 4}$$

系统需要相对于当前机头的前向、右向速度，而不是世界固定坐标速度。六轴 IMU 的 yaw 长期不可观，若直接使用完整旋转矩阵，yaw 漂移会进入水平速度。程序从四元数矩阵中消除 yaw，只保留俯仰和横滚倾斜变换，再映射到

前向为正、右向为正的平面航向系。每轴估计状态包含速度 $\mathbf{v}$ 和等效加速度偏置 $\mathbf{b}$ ：

$$\mathbf{v}_k[k-1] = \mathbf{v}_{k-1}[k-1] + (\mathbf{a}_k - \mathbf{b}_{k-1})\Delta t, \quad \mathbf{b}_k[k-1] = \mathbf{b}_{k-1}[k-1]. \quad \text{公式 5}$$

上电后在静止条件下累计约 2 s 水平加速度以获得偏置初值；若期间出现明显运动，则重新累计。所有积分量进行有限数检查和物理范围钳制，姿态或加速度异常时不继续污染状态。

光流串口中断只喂入解析器，完整帧处理在前台完成。光流积分量按真实积分时间换算为角流率，使用 IMU 原始角速度补偿俯仰和横滚旋转像移，再以 ToF 高度 h 换算线速度 $\mathbf{v}_{\text{flow}} \approx h\boldsymbol{\omega}_{\text{flow}}$ 。偏航时，非旋转中心处的光流传感器会产生切向速度；程序根据安装偏移 $\mathbf{r}$ 扣除 $\boldsymbol{\omega} \times \mathbf{r}$ ，再通过标定矩阵完成轴向与符号映射。

同一角流误差在高处对应更大的线速度误差，因此量测方差随高度平方增加：

$$R(h) = \text{sat}(R_0 + k_h h^2, R_{\min}, R_{\max}). \quad \text{公式 6}$$

为补偿采集、积分和传输时延，估计器维护历史速度环形缓冲，按约定延迟查找 $\hat{\mathbf{v}}_{k-d}$ 并计算创新 $\nu_k = \mathbf{z}_k - \hat{\mathbf{v}}_{k-d}$ 。状态先在短临界区内复制为局部快照，KF 计算在临界区外进行，只有预测代次仍一致时才提交，从而不以长时间关中断扰动 1 ms 飞控环。

光流校正必须依次通过完整帧、积分时间、ToF 资格、高度、姿态倾角、有限数、创新限幅和 KF 数值检查，两个水平轴同时合法才共同提交。系统分别记录“收到完整串口帧”和“成功形成可信校正”的年龄，所以链路在线不等于速度估计有效。

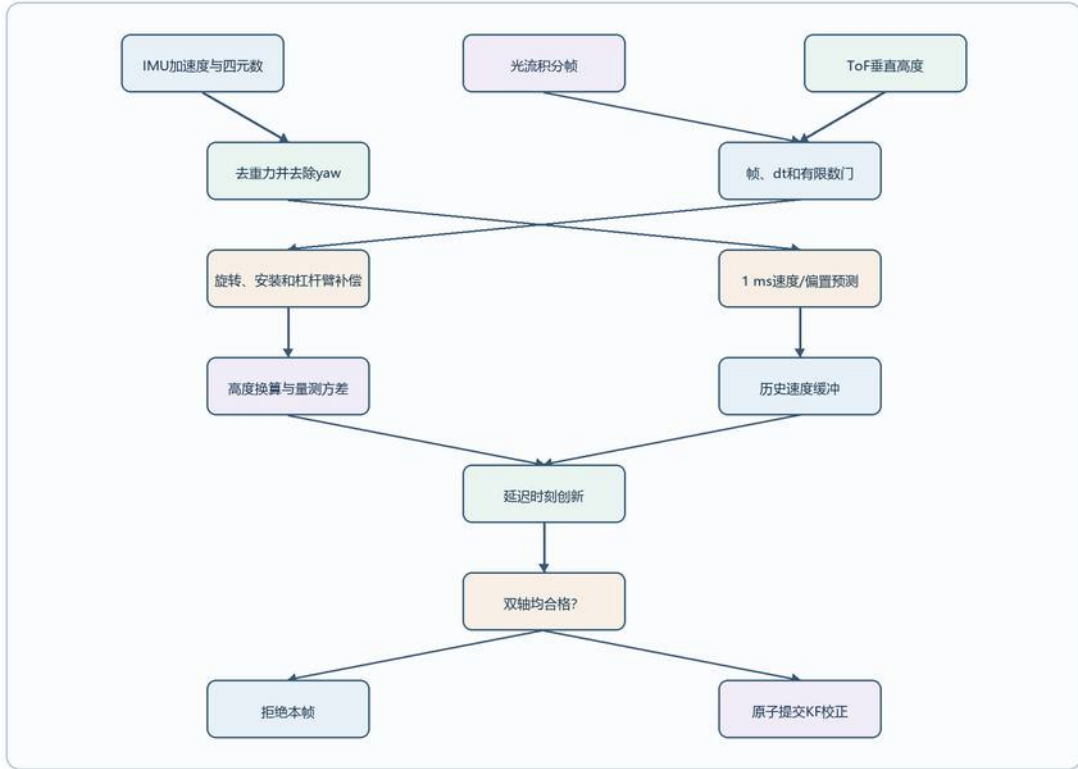


图 5.2 水平速度融合估计流程

5.3 ToF 高度处理与定高

ToF 原始输出是沿光轴的斜距，程序用同拍姿态换算垂直高度：

$$h=d_scos\theta cos\varphi. \quad \text{公式 7}$$

合成倾角过大时，激光可能落在侧方目标，投影误差也会放大，因此该帧直接无效。姿态补偿后的高度还通过量程和时间域创新门：

$$|h_k-h_{k-1}|\leq\max(\Delta h_min,v_z,max\Delta t). \quad \text{公式 8}$$

越界单帧被冻结，不因连续出现相近野值就自动把野值簇重建为新基线；无效持续到超时后才丢弃旧基线，以允许真实的大幅高度变化重新建立观测。恢复首帧只重建速度差分基线，不跨整个故障间隔计算差分，避免虚假垂直速度尖峰。

定高采用高度外环和垂直速度内环：

$$v_z^*=sat(C_h(h^*-h),v_z,max), \quad \text{公式 9}$$

$$\Delta u_h = \text{sat}(C_v(v_z^* - v_z) + K_{ffv_z} v_z^*, \Delta u_{\max}). \quad \text{公式 10}$$

该输出只是基础油门偏移，不替代姿态稳定环。待机发车时，姿态安全而 ToF 不健康仍不能进入闭环。巡航后短时 ToF 失效先保持最后的定高偏移，长期失效则撤销水平运动并按固定斜率降低油门偏移，第 8 章给出其安全状态机。

5.4 飞行序列的单一任务级仲裁

飞行行为不由按键、定高、视觉和通信模块各自推进。flightSequenceCoordinator 是任务序列唯一 owner，每拍消费安全状态、ToF、视觉、MOTION、链路、模式和人工请求等同拍事实，输出阶段、高度目标、位置源、前馈、灯光、蜂鸣和停机请求。调用层每拍只执行一次 flightSequenceStep()。

默认阶段包括 IDLE、WAIT、RAMP、CRUISE、DESCENDING 和 LANDED。起飞接受后先在 WAIT 保持起始条件，再按

$$h^*(t) = h_0 + \text{sat}(\text{fract}\{T_{\text{rise}}\}, 0, 1)(h_c - h_0) \quad \text{公式 11}$$

缓升至巡航目标。只有进入 CRUISE 后才发布 altReached，供 Core1 和地面车约束运动放行。短时 ToF HOLD 为避免重放起飞过程，可暂时保留该任务 episode 资格；长期失效或正式缓降才撤销。

flightSequencePolicy.c 是现场任务触发和阶段轨迹的有限编辑面，只能选择发车、起飞、缓降触发，并给出允许阶段的高度、机体系水平速度和偏航速度；它不能直接写姿态参考、油门、PWM 或 PID 内部状态。Coordinator 校验 action、override mask、来源和事件序号，持续高电平不会重复触发同一行为。

系统明确区分三个 owner：Coordinator 管理任务阶段和控制门；modeFlight 独占 DISARMED/CLOSED_LOOP/PROTECTED、PID 生命周期和停桨；altitudeHold 独占 ToF failsafe。正常缓降到达地板目标并不等于已经接地。只有 DESCENDING 中 ToF 健康且实测高度不高于 100 mm，才进入 LANDED，下一拍请求停机。该 100 mm 是软件触地判据，不是经当前版本硬件测量保证的物理停桨高度。

第6章 三视野感知与连续规划

6.1 分布式检测与观测发布

三路信标视觉分别来自 Core1 下视相机、Core3 后向相机和 Core4 前向相机。三个视觉逻辑节点均在各自相机 owner 处处理 188×120 灰度图像，不把原始图像集中上传到 Core1。高带宽图像处理留在相机 owner 附近，节点间只传递身份化候选轨迹和健康事实，Core1 则集中负责来源资格、目标连续性和唯一运动计划。

各视觉板可依当前 DetectMode 选择 Binary Connected Components 或 Legacy Canny 路径。检测阶段使用面积、亮度、偏心率、填充率和 Hu 特征等参数过滤候选，再由时间一致性 tracker 完成关联、确认、预测和编号。trackId 仅在同一来源和同一 session 内表示轨迹身份，不具有跨相机的全局含义。只有已经确认、当前新鲜且未被边界策略拒绝的轨迹，才进入三来源候选池。

Core3 和 Core4 的配置分为本板私有参数和 Core1 下发的共享参数。曝光、局部阈值与显示周期由观察板保存；DetectMode、形状过滤、tracker、发布边界和 EdgeBand 等由 Core1 协调。共享参数只在成功图像帧边界应用，避免同一帧的检测器、tracker 和发布层消费不同版本。模式真实变化时重置检测器和 tracker；重复配置 tuple 不反复清轨迹。

Core4 在相机快照 owner 处统一执行一次 180°采集方向归一化，使 HOME 原图、阈值预览、检测、跟踪和发往 Core1 的坐标共用画面方向。该归一化不改变 LCD 文字、协议字段或控制外参，下游不得再次旋转。

6.2 动态中心与三种几何路径

三路观测必须先转换到统一的车体前向—横向语义。设轨迹中心为 $((x,y))$ ，来源动态中心为 $((c_x,c_y))$ ，像素比例为 (s) ，则

$$u=(x-c_x)s, \quad v=(y-c_y)s. \quad \text{公式 12}$$

三路动态中心都由 Core0 当前 pose 与本来源独立偏置形成：

$$c_x=p_x+\Delta x_i, \quad c_y=p_y+\Delta y_i. \quad \text{公式 13}$$

Core1 下视源使用当前 heading ψ 逆旋转为车体系误差：

$$e_F = \cos\psi u + \sin\psi v, \quad e_R = -\sin\psi u + \cos\psi v. \quad \text{公式 14}$$

Core3 后视源先按 mirrorX 决定 (u_m) 的符号，再结合安装角 (θ_3) 和方向符号：

$$e_R = \sigma_R(\cos\theta_3 u_m - \sin\theta_3 v), \quad \text{公式 15}$$

$$e_F = \sigma_F(\sin\theta_3 u_m + \cos\theta_3 v). \quad \text{公式 16}$$

Core3 当前映射不再消费 heading 数值，固定外参直接描述后视相机与车体轴的关系；但 heading 有效性仍作为目标推进和 TRACK 输出的安全资格。Core4 则先应用固定外参得到 $(e_F^{\text{fix}}, e_R^{\text{fix}})$ ，再用当前 heading 变换：

$$e_F = \cos\psi e_F^{\text{fix}} + \sin\psi e_R^{\text{fix}}, \quad \text{公式 17}$$

$$e_R = -\sin\psi e_F^{\text{fix}} + \cos\psi e_R^{\text{fix}}. \quad \text{公式 18}$$

所以三路并非“动态中心或固定外参”二选一：Core1 为动态中心加 heading；Core3 为动态中心加固定外参；Core4 为动态中心加固定外参后再使用 heading。每个 profile 在使用前检查中心、比例、安装角和符号的有限性与范围，标定非法时撤销来源资格，而不是退回默认零值。

6.3 精确身份保持与全来源重新评分

Core1 每 10 ms 执行一次规划拍。该拍只读一次 Core0 pose，再取得本地观测以及 Core3、Core4 的完整 planning snapshot。候选必须满足来源新鲜、tracker 确认、trackId 非零、位置与预测有限、profile 合法且未被边界拒绝。一个来源的非法候选不能屏蔽其它来源的合法候选。

选择过程先尝试精确身份保持。如果当前目标的 source/session/trackId 仍存在且合格，系统直接保持该目标，即使另一来源出现更靠近动态中心的候选，也不因“更近”抢占 owner。这减少了重叠视野中的频繁切换。只有精确身份消失后，系统才根据上一目标的位置和运动趋势建立车体系参考 $(\hat{e}_F, \hat{e}_R)$ ，对三来源全部候选计算

$$J_i = (e_{F,i} - \hat{e}_F)^2 + (e_{R,i} - \hat{e}_R)^2, \quad \text{公式 19}$$

选择 $\backslash(J_i)$ 最小者。完全同分时先保持当前来源，再按来源、session 和 trackId 形成稳定裁决。推进门 targetAdvanceAllowed 开放时，新候选可在同一规划拍以完整意图接管，无需额外等待旧来源的 EdgeHold。

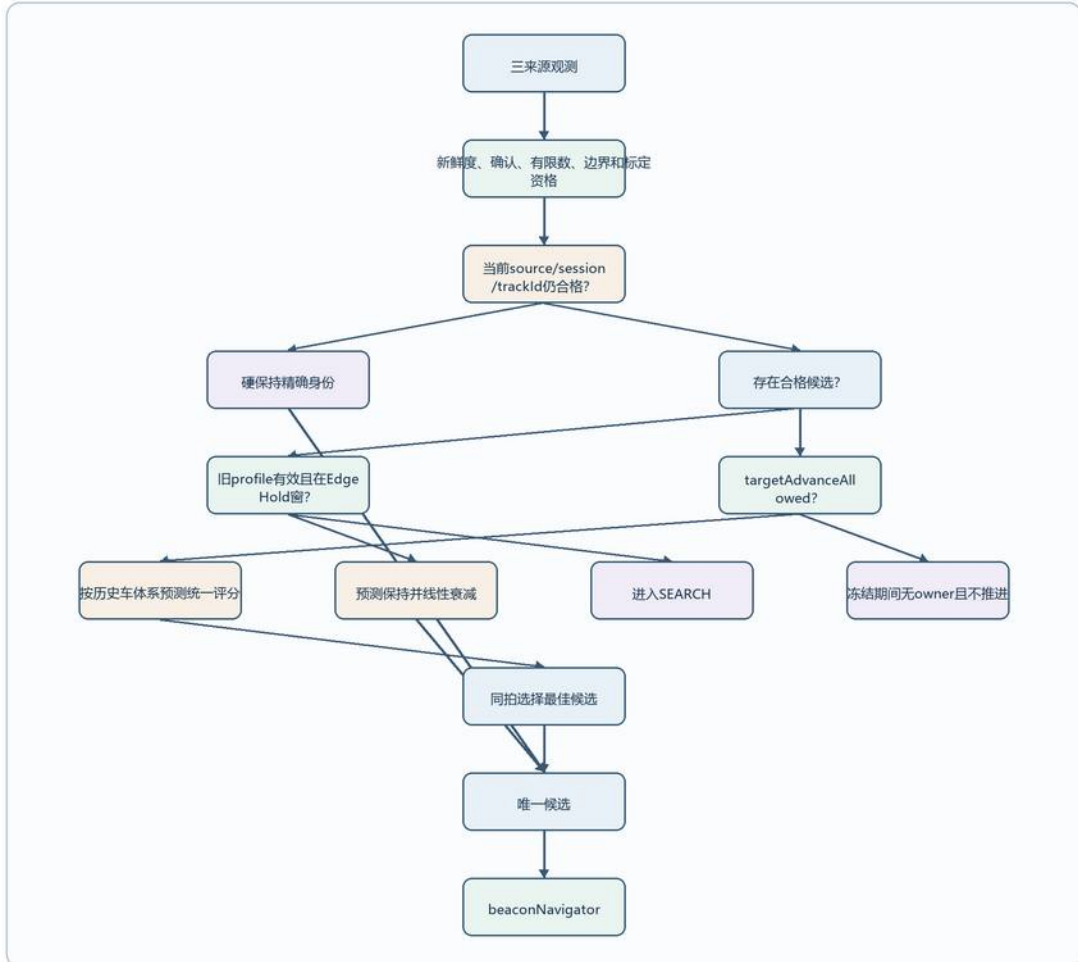


图 6.1 三来源目标连续性状态流

该机制是“基于车体系历史预测的连续选择”，不是全局目标重识别。协议没有跨相机全局 ID、三维重建或多目标交叉消歧模型；多个信标在统一坐标中接近、交叉或重合时，最近历史预测的候选不一定仍是原物理目标。因此本文不使用“保证识别同一信标”的表述，相关边界必须通过多灯交叉实验呈现。

6.4 边界拒绝、普通遮挡与冻结门

画面边缘存在两种不同情形：目标越过可信成像区时，不应继续作为控制依据；短时遮挡或漏检则可能下一帧恢复。tracker 用 Enter/Exit 两组边界形成施

密特门。Enter 边界外的新检测不建立轨迹；已发布轨迹若连续两帧越过 Exit 边界，立即退休并生成一次 BORDER_REJECTED。拒绝事实只携带原 trackId，不带伪造坐标，不计入 fresh 目标数量，也不占 MaxBeacons 配额。

当拒绝记录与当前 source/session/trackId 完全匹配且观测序号前进时，continuity 立即撤销旧 owner 和趋势历史。该撤权是安全优先动作，在起飞延时、位姿等待或重入屏障等冻结门内也可发生；但冻结期间不能借此换 owner。只有 targetAdvanceAllowed 开放时，系统才在同一拍评分并接管其它 fresh 来源。

普通遮挡不产生边界拒绝。全来源暂时无合格候选、上一来源 profile 仍有效且保持窗未到期时，系统用 tracker 最后预测点形成 EdgeHold 候选，并按

$$\alpha(t)=\max(0,1-\text{fract}\{T_hold\}) \quad \text{公式 20}$$

线性衰减运动意图。预测点每个规划拍仍结合当前 pose 与来源 profile 重算车体系误差。任一来源恢复 fresh 候选后立即使用真实观测，并令意图比例回到 1；保持窗到期仍无候选，才进入 SEARCH。

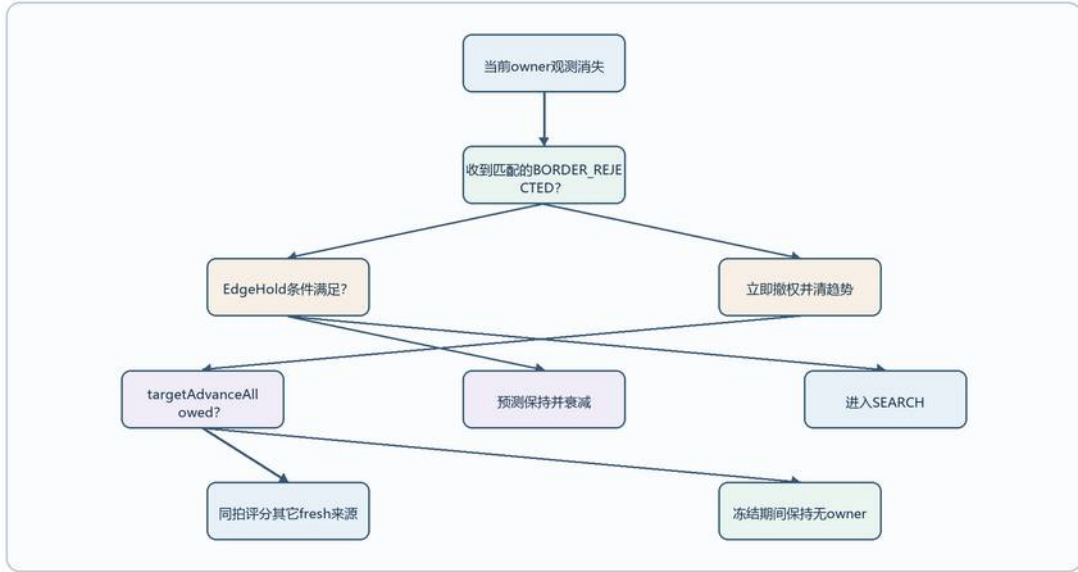


图 6.2 边界拒绝与普通遮挡的分叉处理

6.5 唯一计划与两类输出整形

continuity 只输出一个已统一几何化的候选，navigator 不再二次选择。TRACK 对象先经平面死区，再按来源使用独立的前向、横向增益与方向符号：

$$v_{x^*} = \sigma_x K_{pxe_F}, \quad v_{y^*} = \sigma_y K_{pye_R}. \quad \text{公式 21}$$

三路非 SEARCH 状态的角速度目标均为精确零，观察相机安装角不直接产生底盘旋转命令。平移依次通过“P 控制—intent scale—逐轴 Max—Slew—可选 TD—再次 Max—最低速度椭圆”。TD 启停时以当前输出同步动态状态，不把热切换解释为控制复位；TD 输出非有限数时执行硬零。

当 V_{xMin} 和 V_{yMin} 均为正时构成椭圆半轴 $\backslash(a,b\backslash)$ 。非零输出位于椭圆内部，即

$$q = (\text{frac}v_{xa})^2 + (\text{frac}v_{yb})^2 < 1, \quad \text{公式 22}$$

则沿原方向投影到边界：

$$(v_{x'}, v_{y'}) = \text{frac}1\sqrt{q}(v_x, v_y). \quad \text{公式 23}$$

该处理保持运动方向比例；精确零仍为零，不根据历史方向重新启动。只有一个半轴有效时退回单轴旧语义，两个均为零时关闭最低平移速度限制。

SEARCH 与 TRACK 分路。SEARCH 强制 $(v_x=v_y=0)$ ，只根据最近目标横向方向生成 (w_z) ，并经过 Wz Max、Slew、可选 TD、再次 Max 和 WzMin；它不消费目标位置、动态中心或平移最小速度椭圆。任一来源恢复 fresh 候选时结束 SEARCH，搜索超时则锁停。

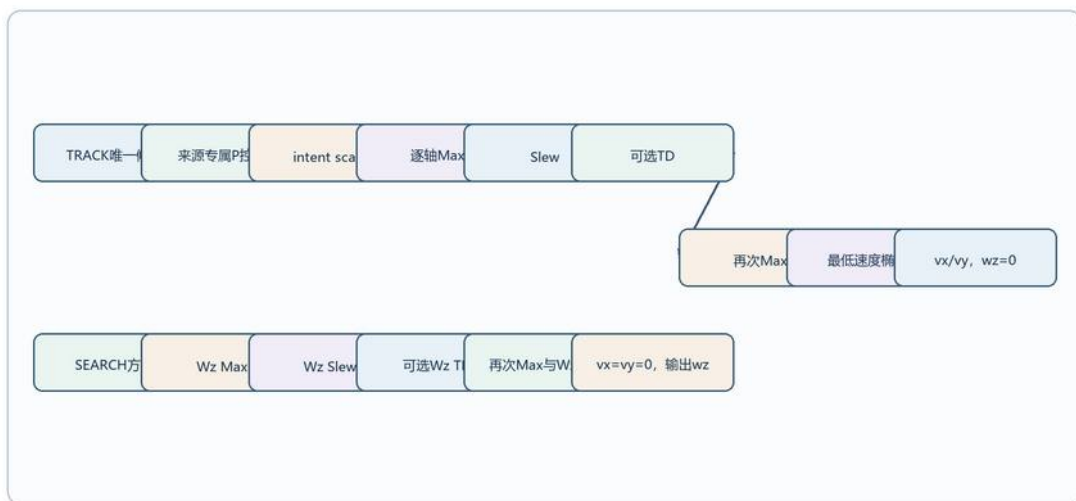


图 6.3 TRACK 平移与 SEARCH 角速度的分路整形

正常同源换灯、跨源接管和 SEARCH 转 TRACK 不清空 Slew/TD，也不人为插入零计划拍，变化由既有整形链约束。协议、配置、标定、相机、共享桥、位姿或安全状态失去资格时则必须归零并建立重入屏障。两条路径不能混为“任何情况下都连续输出”。

本章的三来源资格化、动态中心、统一评分、EdgeHold、SEARCH 和 navigator 调用链为 CODE-CONFIRMED。指定历史提交中，全来源目标锁定、边界拒绝和最低速度椭圆具备 Host、视觉回放或 IAR 构建证据；历史回放的数值只适用于其绑定素材和配置，不写作当前整场识别率。即时满比例跨源接管、三摄高速切换、多灯交叉和真实故障重入仍为 NEEDS-EVIDENCE，不得宣称“三摄冗余容错”“无缝无冲击”或“始终保持同一物理信标”。

第7章 有线空地协同与麦轮控制

7.1 同一运动意图驱动空中与地面执行

空地系统的关键并不是分别制造一套“飞机跟踪器”和一套“车辆跟踪器”，而是让 Core1 在完成三视野目标连续性之后只产生一份运动计划。该计划以车体系前向速度 (v_x) 、横向速度 (v_y) 和偏航角速度 (w_z) 表示，经有线链路同时作用于地面车与飞行器。Core2 将其解释为麦轮底盘速度目标；Core0 将其解释为水平速度前馈、姿态角前馈和偏置转心补偿。两端使用同一运动意图，但保留各自的动力学控制器和安全门。

这种结构避免两个执行体各自观察信标后产生相互矛盾的轨迹。目标身份保持、边界拒绝、SEARCH 和输出整形只在 Core1 完成一次，Core0 与 Core2 不再重新选择目标。与此同时，共享计划不意味着执行输出完全相同：飞机需要姿态稳定和定高，车辆需要轮速与电流闭环；计划是否有效还分别受飞行阶段、链路新鲜度和底盘模式约束。

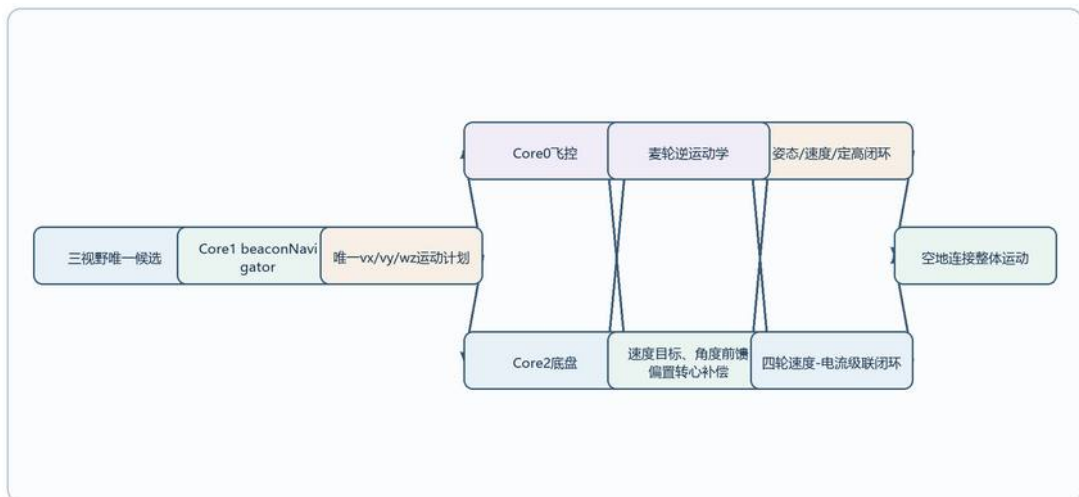


图 7.1 同一运动计划驱动空中与地面系统

计划发布前还需通过任务门。未发车、飞行器尚未到达巡航阶段、必要位姿无效、参数或配置未就绪以及故障重入屏障未解除时，Core1 不发布有效非零计划。Core0 收到计划后仍依据本机 CLOSED_LOOP、任务阶段和 ToF 状态钳制；Core2 本地执行门依据运行模式、RS-485 新鲜度、ENABLE、ESTOP、LAUNCH 和命令合法性决定是否驱动车轮。Core2 姿态/IMU 异常会撤销本地航

向闭环修正并上报故障，但不会单独禁止合法平移命令；整机任务层再依据上报故障决定缓降等系统动作。

7.2 有线链路上的命令与状态闭环

Core1 与 Core2 通过物理 RS-485 有线链路交换运动命令和底盘状态。命令不是无期限保持的寄存器值，而是带序号和新鲜度的消息。Core2 只有在接收到合法、序号推进且未超时的命令时才允许进入有线运行路径；收到非法数值、陈旧序号或链路静默时，不能沿用最后一个非零速度。

必须区分“初次断链停止”和“从 LoRa 模式返回有线模式”两个时序。物理 RS-485 命令新鲜度超过 500 ms 后，Core2 撤销有线命令资格并进入 WAIT_LINK 停止状态，这是断链后的直接安全行为。另一条路径发生在 Core2 曾因模式条件进入 MODE4 后重新回到 RS-485：它要求约 2 s 稳定期、观察新的 commSeq，并先输出一个 500 μ s 零控制拍，再允许真实非零速度。这两者不是一套统一的“2 s 断链停止”规则。

Core1 到 Core0 的 MOTION 重入也有独立契约。静默、producer 重建、producer 变化或物理重连后，core1MotionTxFence 使首次真实请求先发送显式零 MOTION，下一拍才允许非零；它不要求复用 Core2 的 2 s 稳定期，也不把配置重绑泛化为所有 MOTION 恢复条件。Core0 只接受 fence 基线以后 wrap-aware 前进的新序号。

状态回传使 Core1 能够区分车辆链路、运行模式、姿态/估算故障和发车资格。其它类型数据不能替 STATUS 业务事实续鲜。这样，物理字节流尚有活动但 STATUS 停更时，系统仍会撤销依赖该事实的任务资格。

7.3 两级运动前馈与偏置转心补偿

飞行器如果只以位置误差追随车辆，会在车辆加速或转向后才作出响应，系留连接中容易形成滞后。Core0 因此从同一 MOTION 计划生成两级前馈。第一层把 (v_x, v_y) 直接加入水平速度目标，使速度环在位置误差尚小的时候就获得车辆运动方向；第二层依据速度命令的变化形成俯仰和横滚角前馈，为机体加速建立更早的姿态趋势。

若飞行器相对车辆旋转中心存在水平偏置 $\mathbf{r}=(r_x, r_y)$ ，车辆偏航还会产生切向速度：

$$\mathbf{v}_{\text{rot}} = [-w_{zr_y} \ w_{zr_x}] \quad \text{公式 24}$$

该项与平移计划合成后再进入飞控机体系映射，减少车辆原地转向时飞机只收到偏航而没有相应平移速度的问题。所有前馈均受有限数、范围、任务阶段和新鲜度约束；MOTION 失效、ToF 长期故障或正式缓降时，速度前馈和微分历史被清除，不能把旧加速度冲击带入恢复阶段。

前馈只改善命令响应，不替代反馈稳定。飞行器仍由视觉位置、融合速度、IMU 姿态和 ToF 高度闭环修正；本报告不会在没有对比实验时宣称前馈“消除全部滞后”。

7.4 麦轮运动学与级联闭环

地面车采用四麦克纳姆轮全向底盘。设车体系目标为前向速度 (v_x) 、横向速度 (v_y) 、角速度 (w_z) ，轴距为 (L) 、轮距为 (W) ，当前源码的几何旋转系数为

$$k = \frac{L}{W^2} \quad \text{公式 25}$$

则一种与当前轮序和电机方向约定相符的逆运动学可写为

$$\begin{aligned} \omega_{FL}^* &= c(v_x - v_y - kw_z), \omega_{FR}^* = c(v_x + v_y + kw_z), \omega_{RL}^* = c(v_x + v_y - kw_z), \\ \omega_{RR}^* &= c(v_x - v_y + kw_z), \end{aligned} \quad \text{公式 26}$$

其中 (c) 汇总轮半径、减速比和单位换算。实际程序再依据电机安装方向、编码器极性和驱动通道映射处理符号。报告中的公式只表达运动学结构，最终符号表必须以实车逐轮验证为准。

底盘控制由 Core2 cm7_0 以 500 μs 固定节拍推进。外层速度环根据编码器反馈生成期望电流，内层电流环根据采样电流生成 PWM，占空比最终还经过饱和、方向和运行状态钳制。级联结构将“轮速跟踪”和“电机力矩/电流约束”分开，避免速度环直接驱动功率级。

为抑制车辆在无旋转命令时受轮差和地面扰动产生的航向漂移，底盘在满足模式和姿态资格时使用 IMU 航向保持。显式非零 (w_z) 具有更高优先级；

进入停车、断链或故障状态时，速度 PID、航向保持和输出状态按 owner 规则复位，不能让积分残留在重新启动时形成突跳。

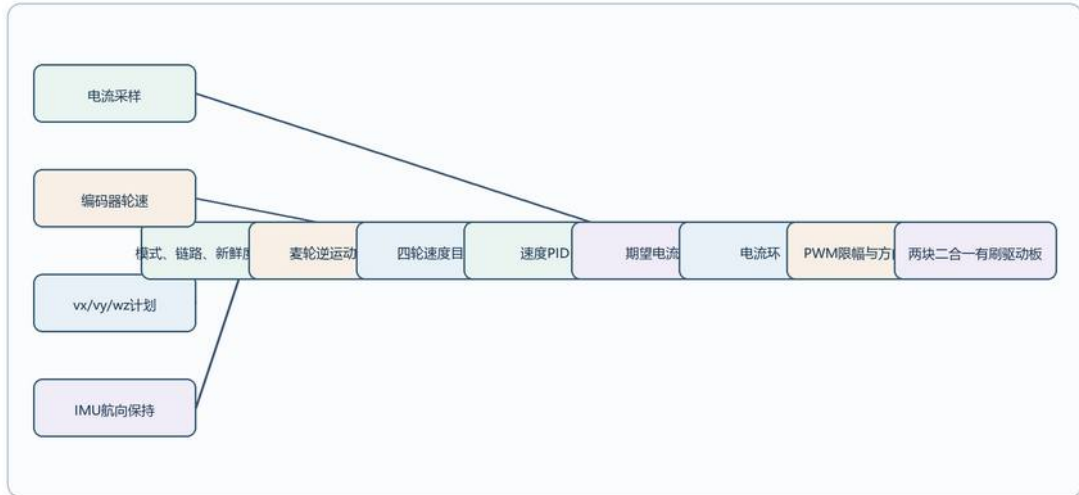


图 7.2 麦轮运动学与速度—电流级联控制

7.5 联合启动与停止时序

联合启动遵循“事实先成立，执行后放行”。Core0 完成初始化、姿态和 ToF 检查后接受飞行任务；Core1 等待有效 pose、观察来源配置和任务阶段；Core2 等待合法有线命令与本机安全状态。只有飞行器进入巡航并发布 altReached、规划目标有效且两端链路事实新鲜时，非零 MOTION 才能同时到达空中和地面控制入口。

停止原因按来源分类。正常任务结束由飞行序列进入缓降并撤销运动；物理 RS-485 命令超时使 Core2 在 500 ms 新鲜度边界后停止；Core1—Core0 MOTION 超时使 Core0 在 300 ms 边界后得到无效零值；姿态保护则由本机安全 owner 立即停桨。不同原因不被压缩为单一“掉线停机”，因为其动作和恢复条件不同。

当前源码确认同一 vx/vy/wz 计划分别到达 Core0 与 Core2，确认两级飞行前馈、麦轮运动学、级联控制以及命令新鲜度门。它不能替代连接线缆约束下的整机联合实验，因此本文只将该结构表述为“共享运动意图的空地协同”，不宣称两执行体完全同步或轨迹误差优于其它方案。

第 8 章 安全降级与故障恢复

8.1 分层安全所有权

系统不把“通信在线”直接解释为“允许运动”。发车前同时要求初始化、控制器、参数、姿态、ToF 和必要链路满足资格；运行中再按故障类型选择立即停桨、受控缓降、运动归零或有限时间保持。安全结论只覆盖源码明确建模的异常，不外推到所有物理故障。

Core0 安全职责分为三层。第一层由 modeFlight 管理执行安全状态。上电保持 DISARMED；发车请求合法且姿态、角速度和 ToF 满足门限后，才进入 CLOSED_LOOP 并启动 PID。闭环中姿态、角速度或数值异常会进入 PROTECTED 并立即停止电机；扶正后不自动恢复，必须重新发车。

第二层是 flightSequenceCoordinator 的任务级仲裁。Core1 或 Core2 失联、Core2 明确姿态/估算故障和必需视觉故障，在地面阻止发车，在闭环飞行中按优先级进入 DESCENDING。触发拍立即撤销 altReached 和外部 MOTION，清除前馈历史，随后输出零水平运动与下降高度目标。故障原因只在锁存为空时写入，后续连锁症状不能覆盖首因；视觉故障还保留来源和类别，以区分相机、链路、协议、配置、标定、bridge 或位姿问题。

第三层是 altitudeHold 独占的 ToF failsafe。它在定高更新后根据本拍新鲜度推进 NORMAL/HOLD/RAMP。Coordinator 必须在高度目标和控制门设置完成后调用定高 owner，再用其本拍结果最终钳制，不能以旧状态决定新一拍运动。

Core1 STATUS 和 MOTION 使用不同新鲜度。STATUS 业务事实与相关链路有独立超时，MOTION 超过 300 ms 后在 Core0 侧无效并返回零值；其它消息不能替 STATUS 中 Core2、视觉等业务位续鲜。所有外部浮点还经过有限数和范围检查。

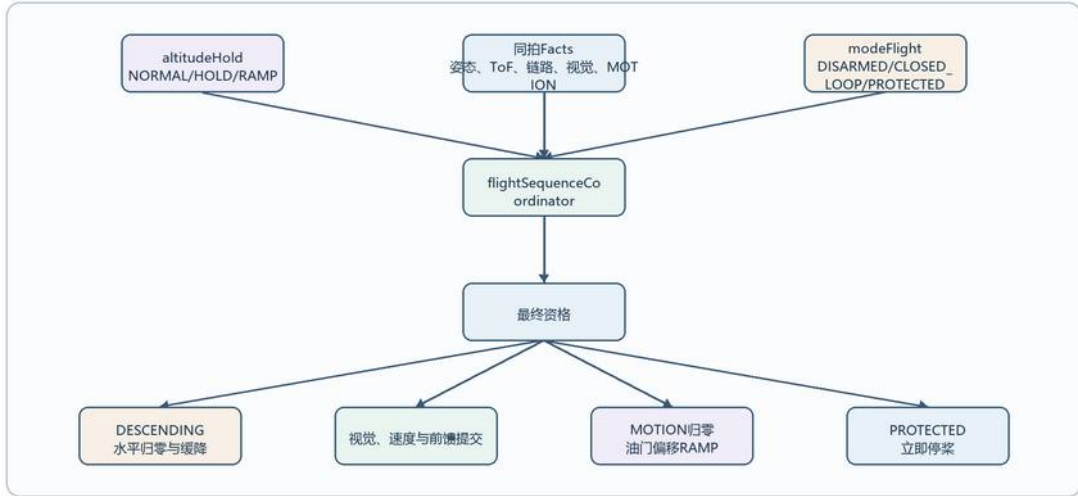


图 8.1 多所有者安全仲裁与动作收敛

8.2 ToF 短时保持与长期失效

ToF 单帧离群、短时遮挡或偶发丢帧若立即切断定高修正，可能产生推力阶跃；长期冻结又会无限延用陈旧输出。系统采用有限 HOLD 后进入 RAMP 的两级策略。

定高已启用而本拍没有新鲜高度时，若底层 ToF 仍处于健康等待窗口，控制状态不推进；健康资格撤销后，failsafe 从 NORMAL 进入 HOLD。HOLD 保留最后一拍定高油门偏移，定高高度外环和垂直速度内环不继续推进积分。已经巡航且未正式缓降时，水平速度估计可在短窗内使用最后可信高度作为光流尺度，并保留合格的视觉与 MOTION。该策略只避免短时动作突变，不代表失效期间高度不变。

若 ToF 在 3 s 内恢复，系统清除 failsafe，从当前可信高度重置定高 PID，并重建速度差分基线。若连续失效达到 3 s，则进入 RAMP：撤销 altReached，强制位置源接管并归零视觉误差、机体系水平速度命令、偏航速率命令和姿态前馈，同时清除运动微分历史。定高油门偏移按固定 $(2\%/s)$ 下降：

$$\Delta u_h(t+\Delta t) = \Delta u_h(t) - 2\% \Delta t. \quad \text{公式 27}$$

单拍 (Δt) 被限制，避免调度长停顿导致一次大幅降油门。偏移到达“最小允许油门减基础油门”的下限后，定高 owner 发布 requestDisarm，再由

modeFlight 停止闭环。RAMP 是固定油门偏移下降，不同于正常 DESCENDING 中按高度目标闭环缓降。

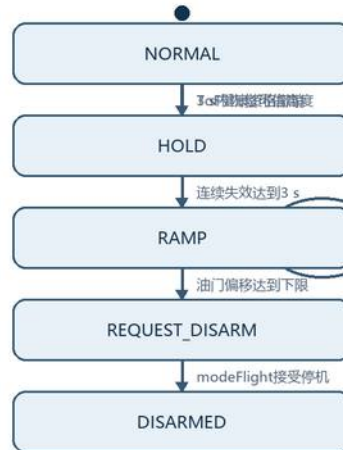


图 8.2 ToF HOLD—RAMP—停机状态机

正常缓降的触地是另一条路径。DESCENDING 期间持续闭环，只有 ToF 健康且当前实测高度不高于 100 mm 才进入 LANDED，下一拍请求停机；目标值到达地板不直接证明接地。ToF 不健康时不得利用冻结高度触发着地，而由 HOLD/RAMP 接管。

8.3 多来源故障分类与首因锁存

多节点系统中，一处根因常引出多个表象。Core1 失联后，Core2 和视觉业务事实也会因 STATUS 停更而陈旧；若显示不断覆盖为最新症状，操作者无法定位起点。Core0 因而只在锁存为空时写入首个原因，必需视觉故障再携带来源与类别。首次 K5 操作用于确认故障，不在同一输入边沿重新发车，故障确认与下一次启动分离。

恢复不是收到任意一帧。STATUS 需通过长度、校验、保留位、业务 tuple、producer session 和 wrap-aware 序号检查；MOTION 还需要显式零帧 fence。共享内存读取碰撞可保留 last-good，但不得借此刷新年龄或无限延长资格。ToF 恢复首帧重建速度差分基线；KF 恢复时预置 PID 上一误差以抑制微分

冲击；PROTECTED 恢复必须等待新的发车请求。不同 owner 的恢复契约不能概括为完全相同的“稳定期+重绑”。

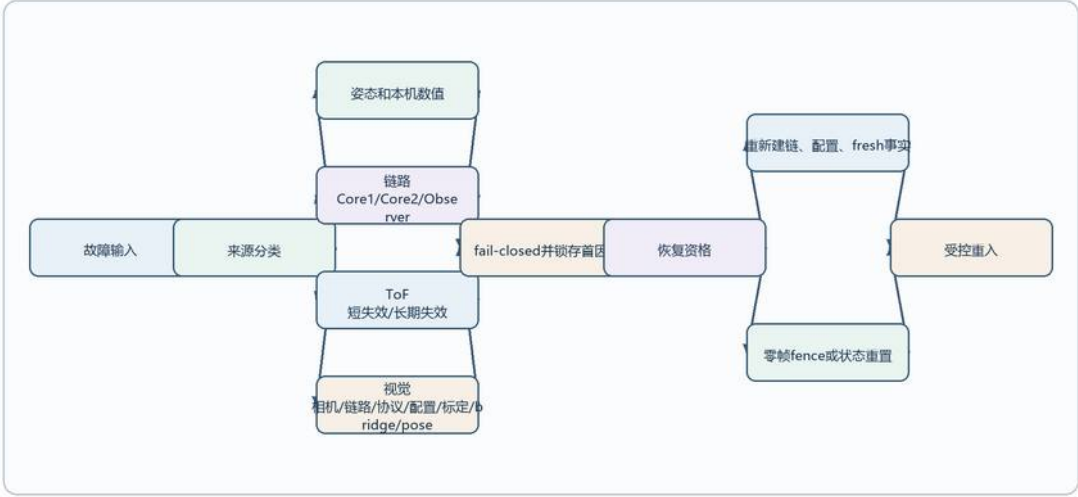


图 8.3 视觉、链路、ToF 和姿态故障传播与受控重入

表 8.1 故障类别与系统动作

故障	立即动作	持续动作	恢复条件
姿态/角速度保护	进入 PROTECTED、停桨	保持锁存	扶正并重新发车
必需视觉或任务链路故障	撤销运动与巡航资格	正常高度闭环缓降	对应来源重新建立完整资格
MOTION 超时	外部运动无效并归零	保留本机安全闭环	零帧 fence 后的新鲜前进序号
ToF 短时失效	冻结定高偏移	HOLD 有限等待	3 s 内可信高度恢复并重建基线
ToF 长期失效	撤销水平运动	2%/s RAMP 至请求停机	不自动恢复当前飞行 episode
Core2 RS-485 命令超时	WAIT_LINK 停车	输出保持安全零值	合法新鲜命令及本机模式资格

当前源码可确认 owner、状态门和输出收敛关系，但不能据此证明真实飞行的下降轨迹、停桨时刻、故障检出延迟或机械安全性。EV-C01-TOF-HOLD-

CONTINUITY 只对其历史提交构成 IAR 构建证据，EV-C0-LAND-TOUCHDOWN 只覆盖源码和 Host 边界；第 10 章因此只给出生产 C 依赖桩场景，不提升为硬件结论。

第9章 参数事务、交互与开发工具

9.1 五板参数库的本地可靠存储

Core0~Core4 五个生产控制节点均维护本节点运行参数，参数使用者不直接从 Flash 地址读取。每节点参数库区分运行态对象、持久化银行和选择日志：运行态供控制、视觉和通信模块在确定边界读取；两个持久化银行保存带 schema/version、payload 长度、generation 和 CRC 的镜像；选择日志以追加方式指出当前有效银行。启动时先验证头部、长度、版本和 CRC，再按 generation 与选择记录确定候选；验证失败时不会把随机 Flash 内容解释为合法参数。

双银行设计避免新值写入一半就覆盖唯一旧值。保存时先向非活动银行写入完整镜像并校验，再追加选择记录；旧银行在新记录生效前仍可作为回退。这里的“可恢复”指源码显式覆盖的事务阶段和启动选择逻辑，不等于任意时刻断电都已经完成板测，也不表示 Flash 多字节写入具有瞬时原子性。

每个参数对象带有范围和有限数约束。控制参数在 apply 时重置或同步受影响的 PID、滤波器、TD 和派生量，视觉参数在图像帧边界应用，通信参数按协议会话和配置 tuple 生效。参数“已经保存”、“已经进入运行态”和“所有相关节点已经应用”是三个不同事实。

9.2 Core1 协调的可恢复分布式事务

跨板参数操作由 Core1 协调，覆盖 Core0、Core1、Core2、Core3 和 Core4。五板共享事务身份和目标槽号，但每板参数 schema、payload、CRC/hash 与 generation 可以不同，协调器不会要求这些板内属性跨板相等。

SAVE 与 LOAD 是两条不同链。SAVE 开始时，各板冻结本板当前 Live RAM 快照，以同一非零 transaction 写入同号 Library Slot；保存完成不自动改变 Runtime。LOAD 开始前，协调器确认五板目标槽均有效并属于同一 transaction，再让各板把各自槽位内容提交到本板 Runtime，并调用本板 apply。Core2 若处于不适合应用的执行状态，会先按本机规则安全收敛，再报告应用结果。

若任一参与者拒绝、持久化失败、应用失败或超时，协调器不会把事务标记为整体完成。根据阶段，系统保留旧状态、执行补偿恢复，或在重启后依据日志处理未决事务。完成条件是事务身份、槽号、参与板状态以及各板实际保存/应用结果符合本次 SAVE 或 LOAD 的协议，不要求五板 generation、CRC 或参数 hash 数值相同。

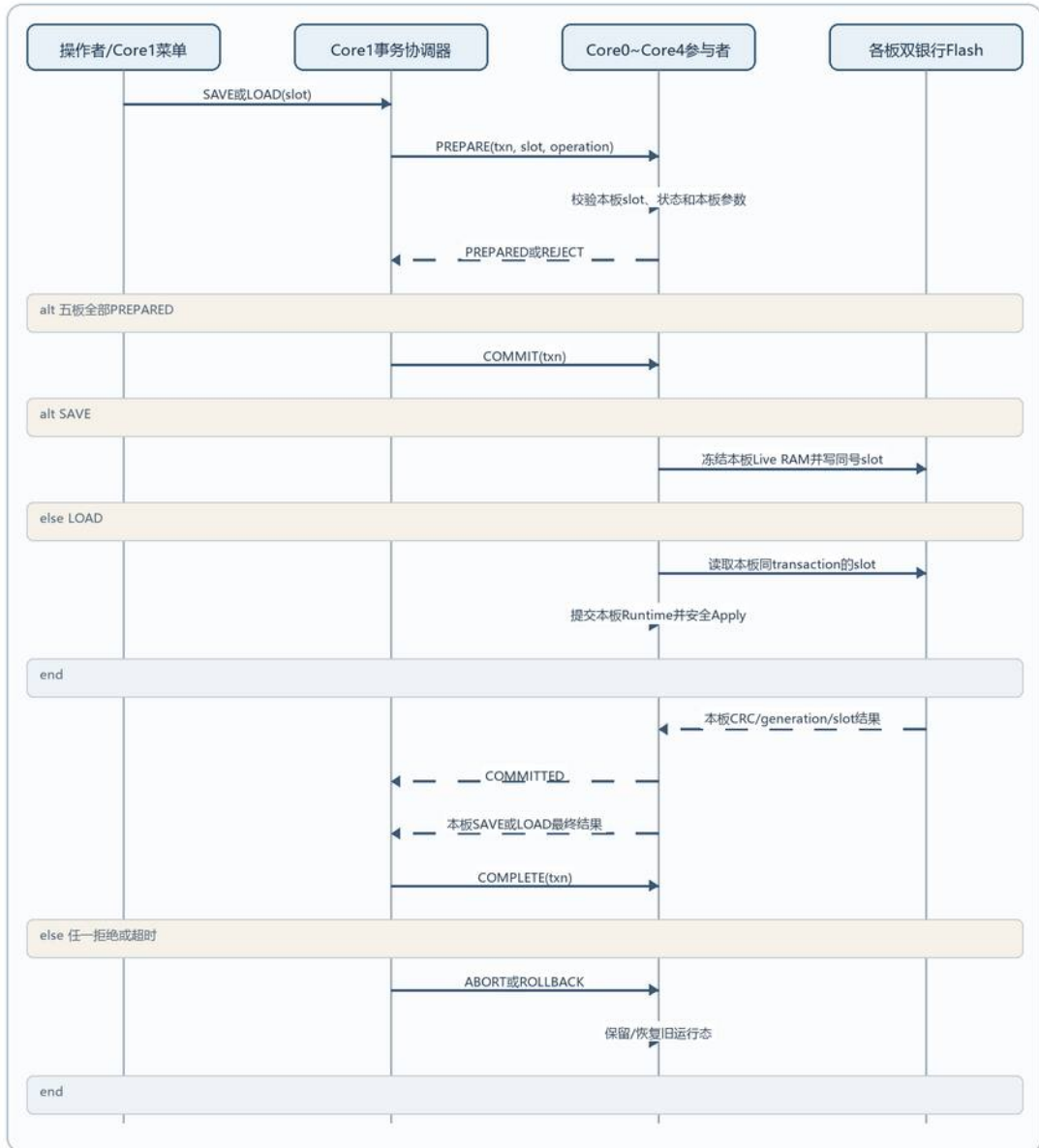


图 9.1 五板参数事务时序

该事务的价值在于把“操作员按下保存键”扩展为可审计的分布式状态，而不是把五块板的成功概率简单相乘。它能明确指出哪块板停在准备、持久化或

应用阶段，并让重启后的节点依据日志恢复；但本文不使用“瞬时原子写入”“任意断电绝不丢失”或“所有板同时生效”等超出证据的表述。

现有证据 EV-C01234-PARAM-SLOT-LOAD-SYNC-20260806 对绑定提交 555ad813... 的五板参数槽位加载与同步路径标记为限定范围内 HW-CONFIRMED。该结论不能自动覆盖当前基线、任意断电相位、Flash 磨损寿命或所有参数组合。

9.3 本地菜单与远程 KVM

参数交互采用板级本地菜单与远程 KVM 结合。Core0 提供根界面和本地页面；Core1、Core3、Core4 可通过受控远程会话显示页面、接收按键并返回状态。远程 KVM 不覆盖 Core2，Core2 保持本地菜单交互；报告不得把“五板参数协调”与“KVM 覆盖范围”混为一谈。

远程会话不是透明地转发任意内存。控制请求携带目标板、会话、租约、按键边沿和序号；显示返回使用受限的菜单状态或 overlay DTO。session、bindingEpoch、配置身份或新鲜度变化时，旧租约失效。按键采用边沿事件和确认机制，避免重传把一次操作重复执行。远程显示的丢帧只影响交互可见性，不应直接改变飞控和底盘实时控制。

菜单修改作用于本板 Live RAM，并通过参数 owner 的应用链热应用到运行消费者；Library Slot 不会因普通参数编辑自动更新。联合 SAVE 冻结五板各自的 Live RAM 写入所选槽，联合 LOAD 才把完整槽位提交为各板 Runtime 并应用。菜单代码通过公开参数接口发起修改和事务，不直接写 PID、tracker 或通信内部状态。

9.4 CoreT3 原始图像录制与回放

CoreT3 是独立调试录制板，不属于生产五板架构，也不参与飞行和底盘控制。它从 MT9V03X 获取原始图像，经 WiFi SPI 与上位机链路上传，用于保存难以在实验室重复构造的视野、遮挡和灯板素材。将图像输入从实车时序中解耦后，同一素材可在参数调整前后重复回放，比较检测、tracker、边界拒绝和候选发布变化。

回放能够证明算法在指定素材、参数和提交上的输出，不等于实车控制闭环通过。素材需要同时记录采集板、相机方向、曝光、场景、时间、分辨率和对应固件；回放结果需要记录期望标注、实际输出、工具版本和原始日志。只有这些元数据完整时，历史 Precision、Recall 或边界统计才可在限定范围引用。

9.5 分层验证工具与开发闭环

工程验证按“源码审查—Host 回归—IAR 构建—板端/整机”分层。纯协议 codec、几何变换、选择器和状态机可在 Host 构造边界输入，验证序号回绕、非法数值、分片冲突和目标切换；IAR Embedded Workbench 用于各双核工程的编译、链接和目标调试。^[7] IAR 构建证明指定配置能由目标工具链编译链接；板端记录才回答真实串口、Cache、传感器、执行器和时序问题。任何一层均不能替代更高层。

远程 KVM 提升多板调参效率，CoreT3 提供可复现视觉输入，参数事务保证配置身份可追踪，证据台账把结论绑定到提交、配置和范围。四者共同构成工程闭环，其优势是缩短“发现现象—复现—调整—验证—回滚”的路径，而非宣称工具本身属于新的控制理论。

9.6 制作、安装与调试过程

本系统采用“先模型、后单机、再联合”的开发顺序。前期先在 PC 端设计并验证视觉算法，将二值化、连通域、形状判别、时间一致性跟踪和麦轮运动学拆成可重复输入的模型；随后分别建立飞行器和底盘的独立调试路径，最后才接入三路信标观察、有线空地通信和系统级安全门。该顺序把传感器方向、控制极性、算法状态和跨板时序分开定位，避免整机首次联调时同时面对全部变量。

表 9.1 开发阶段与 Git 历史锚点

阶段	主要工作	代表性历史提交
仿真与算法原型	整理仿真入口，以 Python 调用生产 C 或镜像模型；建立麦轮解算和视觉算法原型	2d18d179、bdd7ae4a、5ff1dcdd

阶段	主要工作	代表性历史提交
飞控独立调试	建立无人机调试工程，依次完成传感器、无刷电调、定高与飞控链，再迁入 Core0 生产结构	d9378c76、fd82d030、fad73b03
底盘独立调试	完成麦轮库、轮速与电流级联控制、轨迹试验和物理量反馈	5ff1dcdd、23a4090e
视觉仿真回归	用合成夹具验证时间一致性跟踪与 BinaryCC 检测，分离单帧候选和时域状态	8b8633a5、3229d717
三视野与空地联调	建立三摄初次完赛基线，接入前后观察板、统一车体系连续性和空地共享计划	5c6a8e09、80b86f94、40c7a84a
安全与参数收口	修正参数事务同步与恢复，合入新底盘几何和槽位验证结果	555ad813、16d670d2

PCB 制作完成后，装配按电源域和功能域分阶段进行。首先检查裸板短路、主电源与各级稳压输出，再安装处理器、传感器、通信收发器和功率器件；控制板与驱动板分别限流上电，确认没有异常温升后才接入相机、编码器、IMU、ToF、光流和显示。飞行器侧按前、后、下视方向固定相机与观察板，并记录镜像、安装偏航和坐标正方向；底盘侧逐轮确认 FL、FR、BL、BR 通道、编码器正方向、电机极性和麦轮滚子方向。系留线最后接入，分别检查供电极性、RS-485 差分线和公共参考地。

飞控单独调试从停桨状态、IMU 静止零偏和姿态方向开始，再依次检查四路电机序号、转向、最小油门、姿态内环、速度估计、定高和任务序列。单电机推力测试记录约 1700 g、约 40 A；四合一电调完成四路满指令运行，但正常任务很少持续满输出。该记录没有原始推力—电流曲线，因此只用于说明调试覆盖，不用于推导持续推重比或电调额定能力。

底盘单独调试先让四轮离地或处于安全支撑状态，逐轮核对 PWM、方向、编码器和电流采样；随后先闭合电流内环，再闭合速度外环，最后检查麦轮前进、横移、旋转、复合运动和饱和归一化。模型正逆解用于提前确认代数方向，实装核对负责确认轮序、极性、机械摩擦和编码器反馈，二者不能相互替代。

第二十一届全国大学生智能汽车邀请赛技术报告

联合调试先建立 Core0/Core1、Core1/Core2 和两条 Observer 专线，再逐项开放位姿、轨迹、规划和执行器门。每次只增加一类事实：先确认三来源坐标和身份，再观察唯一计划，随后分别检查飞机和车辆的本机闭环，最后验证联合启动、停止、断链、陈旧命令、视觉失效和 ToF 降级。发现异常时，按“原始输入—本地算法—跨核快照—有线协议—计划 owner—执行器”逐层回溯，并用 CoreT3 录制回放、Host 场景、IAR 构建和板端现象分别回答不同证据层的问题。当前报告只把 Git 历史能够证明的开发演进、源码能够证明的机制和队伍明确提供的装机记录写成事实，不补造未保存的波形、故障次数或整机指标。

第 10 章 可复现实验与证据

10.1 证据口径与生成基线

本章只收录由活动源码、确定性 Host 场景、模型仿真和现有证据台账能够复查的结果。固件证据基线固定为 16d670d2b7141bfc37bc3d2403d695ffdda1a428。所有 CSV、JSON、PNG、源码片段和哈希清单由 tools/technical_report/generate_report_assets.py 生成，生成清单见 technical_report_assets/generated/generation_manifest.json。

CODE-CONFIRMED、BUILD-CONFIRMED、HW-CONFIRMED 和 NEEDS-EVIDENCE 是正式证据状态。Host 执行、合成夹具和模型仿真只说明验证方法与输入范围，不构成新的证据等级。模型结果不能证明实车轨迹精度，Host 状态机结果不能证明真实相机切换效果，历史硬件记录也不能自动提升当前基线的全部功能。

表 10.1 可复现结果概览

项目	当前结果	证据边界
麦轮运动学	6 个确定性场景及 61 点饱和扫描；轮半径 0.05 m、轴距 0.169 m、轮距 0.21 m	运动学模型；不等于实车轨迹精度
三来源连续性	活动 beaconTargetContinuity.c 经 Host 编译执行，三个独立场景共输出 16 个状态拍并覆盖 Source1/3/4	生产 C 确定性场景；不等于真实跨相机效果
信标检测与定位	75 帧固定合成真值夹具给出检出计数及平均、P95 和最大中心误差；关联门读取板上配置	模型证据；像素域指标不换算为赛场毫米误差
ToF 分级降级	活动 altitudeHold.c 经 Host 编译执行，覆盖 HOLD 前恢复、RAMP 中恢复和持续故障	生产 C 加依赖桩的确定性场景；不是高度或着陆实测
空地共享计划	9 拍规则模型覆盖启动、Core0 重连和 Core2 MODE4 重入	源码规则驱动模型；不代表实物链路时延或轨迹误差
五板参数事	12 拍协议模型覆盖正常 LOAD、参与	协议状态模型；限定历史五板硬件证

项目	当前结果	证据边界
务	者拒绝和协调器重启恢复	据另行登记
必要程序源码	从 12 个活动 IAR 工程收集 416 个项目自有 C/H 文件	CODE-CONFIRMED；现场网络凭据与 6 个 ST 传感器组件排除；不证明 IAR 构建或硬件运行

10.2 麦轮运动学模型

麦轮仿真直接复用 tools/mecanum_sim/mecanum_sim.py 的逆解、归一化和正解。停车、前进、左移、逆时针旋转和未饱和复合运动中，正解能够恢复输入的 $v_x/v_y/w_z$ ；饱和场景则保持四轮比例并整体缩放，因此正解结果不再等于原始大幅输入。这一结果验证了模型内部的代数一致性和归一化方向，不评价轮胎打滑、负载、电机死区或地面轨迹误差。

麦克纳姆轮运动学：四轮命令与饱和归一化

活动模型参数 · 轮速单位为每 2 ms 采样周期的编码器脉冲冲数

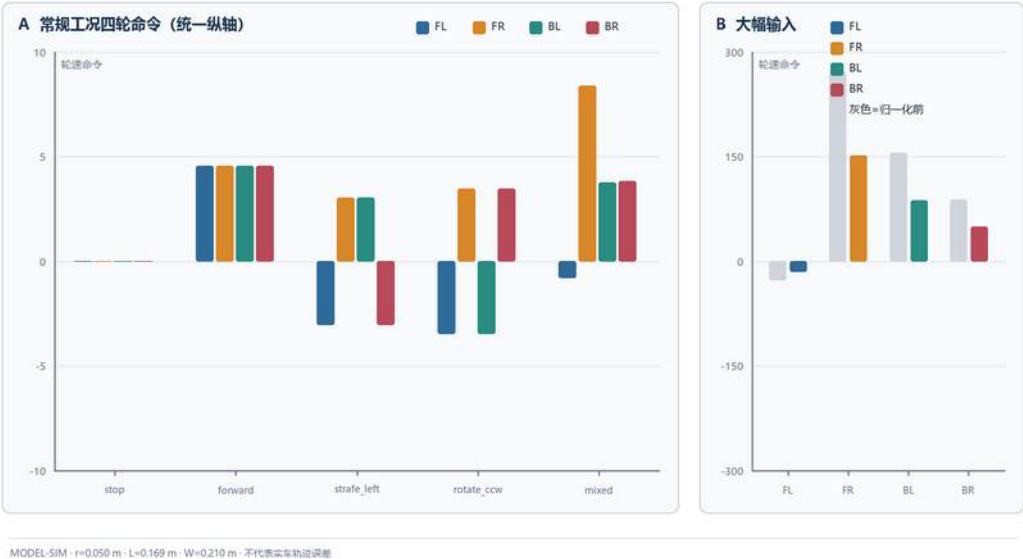


图 10.1 麦轮运动学六场景四轮命令

图 10.1 中 FL、FR、BL、BR 分别为左前、右前、左后、右后轮的离散速度命令。完整输入、轮速和正解结果见 `technical_report_assets/generated/mecanum_results.csv` 与 `mecanum_results.json`

麦克纳姆轮饱和和扫描：限幅点与方向保持

沿(vx, vy, wz)=(8, 6, 20)等比例放大输入，比较归一化前后轮速峰值

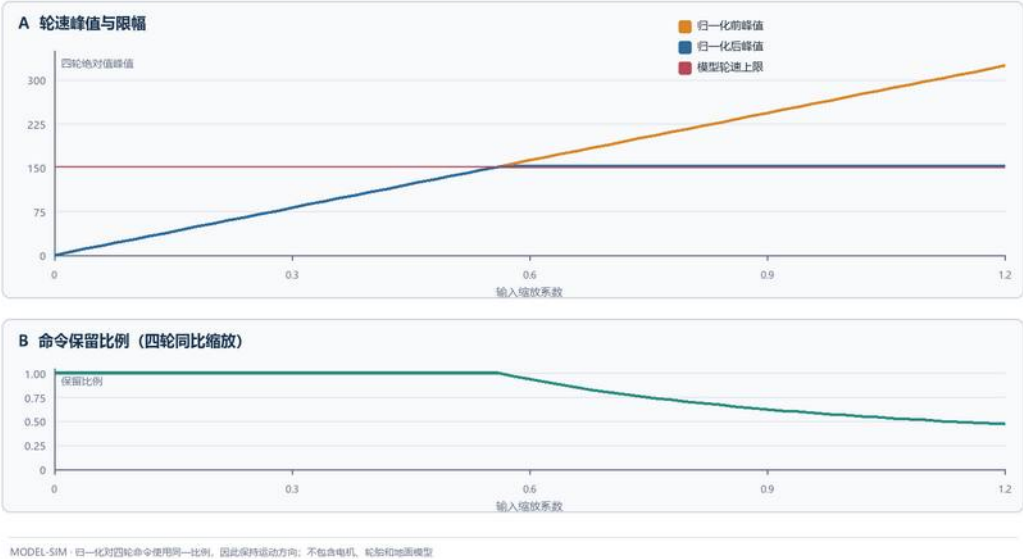


图 10.2 麦轮归一化饱和和扫描

图 10.2 沿 $(v_x, v_y, w_z)=(8, 6, 20)$ 方向对输入做 61 点等比例扫描。归一化前的四轮绝对值峰值随输入线性增加，触达模型上限后，输出峰值保持在上限，命令保留比例随输入继续增大而降低。在原始大幅输入点，保留比例为 0.5621，正解得到 $(4.4969, 3.3727, 11.2423)$ ；三个分量均使用相同缩放比例，因此模型内运动方向保持不变。扫描数据见 mecanum_saturation_sweep.csv。这一结论不包含电机饱和、轮胎打滑或电池压降

表 10.2 模型输入与正解结果

场景	输入 (v_x, v_y, w_z)	正解 (v_x, v_y, w_z)
stop	$(0, 0, 0)$	$(0, 0, 0)$
forward	$(0.3, 0, 0)$	$(0.3, 0, 0)$
strafe_left	$(0, 0.2, 0)$	$(0, 0.2, 0)$
rotate_ccw	$(0, 0, 1.2)$	$(0, 0, 1.2)$
mixed	$(0.25, 0.15, 0.8)$	$(0.25, 0.15, 0.8)$
saturated	$(8, 6, 20)$	$(4.4969, 3.3727, 11.2423)$

10.3 三来源目标连续性生产 C 场景

报告专用 Host 入口没有复写连续性算法，而是把活动 beaconTargetContinuity.c 与一个只负责构造输入和输出 CSV 的 C 文件共同编译。场景依次覆盖同一 source/session/trackId 保持、原身份消失后的跨来源接管、全来源失效后的 EdgeHold—SEARCH、推进冻结以及匹配 owner 且序号前进的边界拒绝。

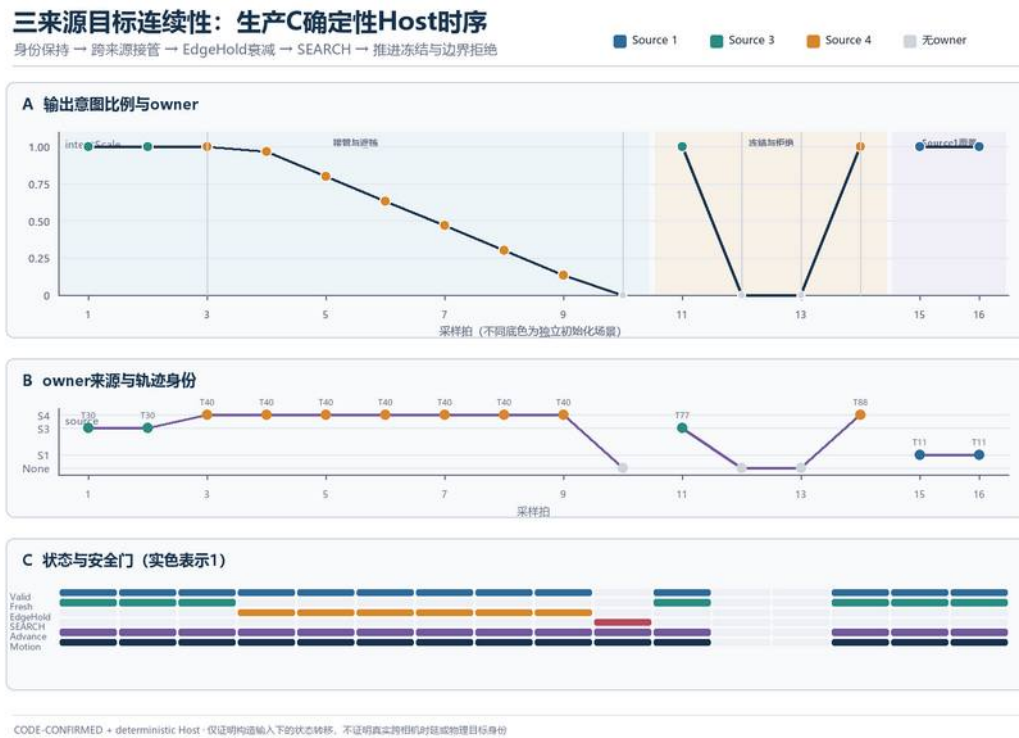


图 10.3 生产 C 三来源连续性确定性时序

图 10.3 分三层展示 intentScale、owner/track 以及 Valid、Fresh、EdgeHold、SEARCH、Advance、Motion 六条状态通道。横轴是采样拍，不是等间隔物理时间；三种底色分别表示经过独立初始化的“接管与遮挡”“冻结与拒绝”“Source1 覆盖”场景，场景之间不连线。16 个采样拍覆盖 Source1/3/4，其中包含 6 个 EdgeHold 衰减样本和 2 个推进冻结样本。结果表明：精确身份存在时保持原 owner；身份消失且推进门开放时可由另一 fresh 来源接管；全来源失效后保持比例随时间下降，保持窗结束进入 SEARCH；推进门关闭时其它 fresh 来源不能换 owner；匹配当前 source/session/trackId 且观察序号前进的边界拒绝仍能撤销旧 owner，只有推进门重新开放后才允许新候选取得 owner。原始逐拍结果见 continuity_trace.csv，其中保留每拍 tick_ms；统计摘要见 continuity_summary.json

该用例证明的是活动生产 C 在这些构造输入下的状态转移，不证明真实物理信标身份、相机标定精度或跨视野切换时延。

10.4 合成信标检测、定位与时域跟踪

固定夹具生成 75 帧、每帧 3 个带已知像素中心的运动信标，并叠加噪声和两类高亮条状干扰。生成器从固件证据基线读取板上 VISION_TRACK_MAX_ASSOC_DIST_DEFAULT=14 px，再以相同关联门运行时域模型。单帧方法得到 TP=222、FP=2、FN=3；时域跟踪得到 TP=219、

FP=0、FN=6。这组结果用于量化当前报告中的图像域检测性能，不换算为未经相机标定和距离约束的赛场毫米误差。

合成信标夹具：单帧检测与时域跟踪的取舍

固定合成输入·同时呈现计数与比例，避免只展示接近100%的百分比

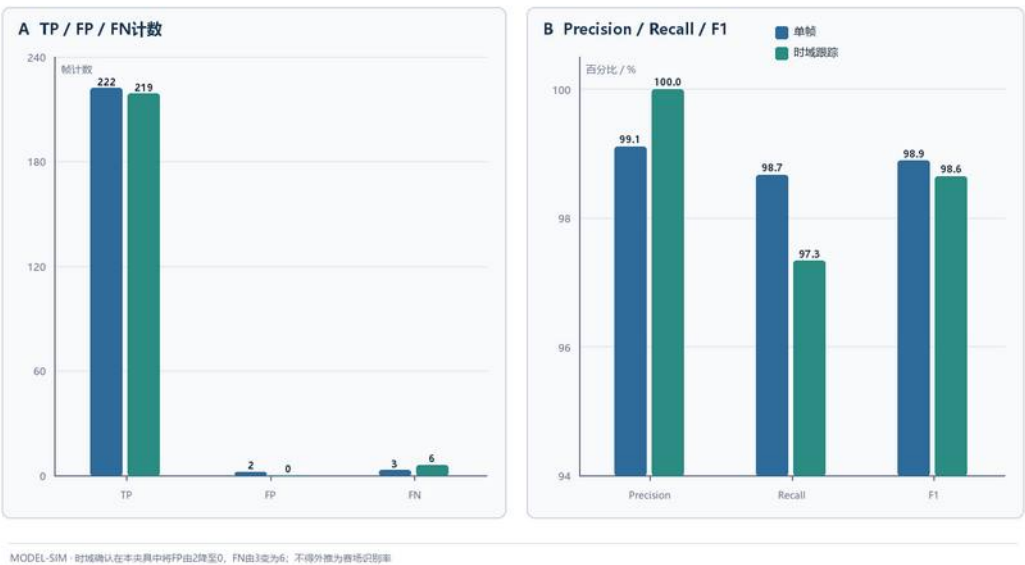


图 10.4 合成信标夹具指标对比

表 10.3 合成夹具完整指标

方法	TP	FP	FN	Precision	Recall	F1	误发现率	漏检率
单帧	222	2	3	99.1%	98.7%	98.9%	0.9%	1.3%
时域跟踪	219	0	6	100.0%	97.3%	98.6%	0.0%	2.7%

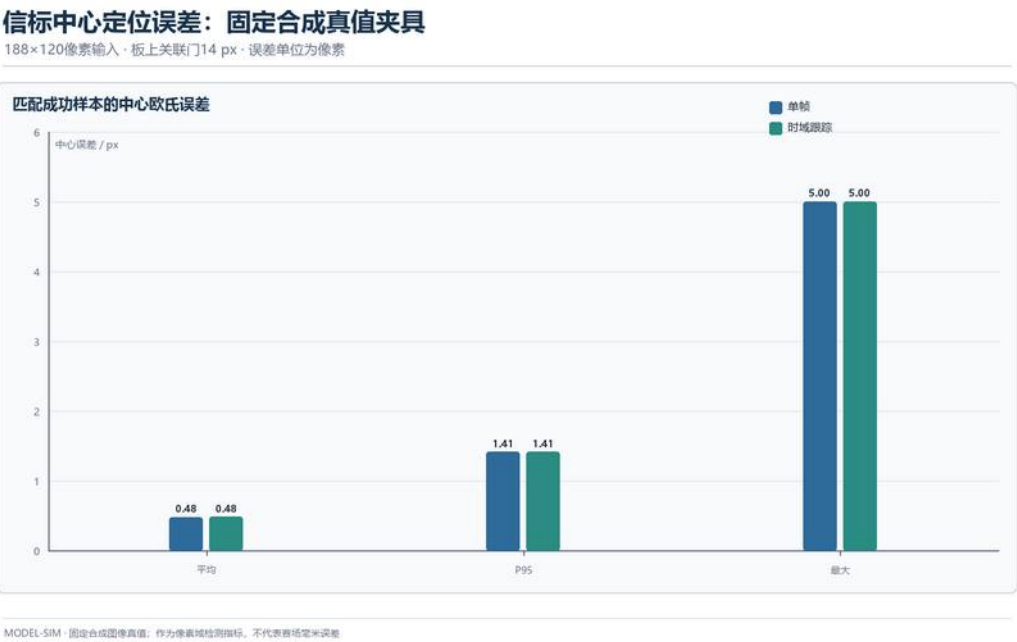


图 10.5 固定合成真值夹具的中心定位误差

图 10.5 统计匹配成功样本的中心欧氏误差。单帧检测平均误差为 0.48 px、P95 为 1.41 px、最大值为 5.00 px；时域跟踪平均误差为 0.48 px、P95 为 1.41 px、最大值为 5.00 px。图像坐标自身以 1 像素离散，板上采集配置为 50 fps；因此报告采用“像素域合成真值误差+板上配置频率”回答检测精度与频率，不虚构物理距离精度。机器可读结果见 beacon_tracking_metrics.csv/json

10.5 空地共享计划与独立执行门

空地协同模型使用同一 0.60 归一化运动意图构造 9 个采样拍，覆盖首次启动、Core0 MOTION 失鲜后重连以及 Core2 由 MODE4 返回 RS-485 owner。模型直接采用源码规则：Core0 MOTION 新鲜度门为 300 ms，重连后的首次真实请求先提交零帧；Core2 有线命令新鲜度门为 500 ms，MODE4 重入在稳定期和新 commSeq 成立后先经历一个 500 μ s 零控制拍。两端分别判定资格，因此一端失效不会把另一端已有的新鲜事实伪造成失效，也不会让旧命令跨重连恢复。

空地共享运动意图：独立执行门与重入栅栏

归一化意图模型·同一Core1计划分别通过Core0与Core2本机资格门

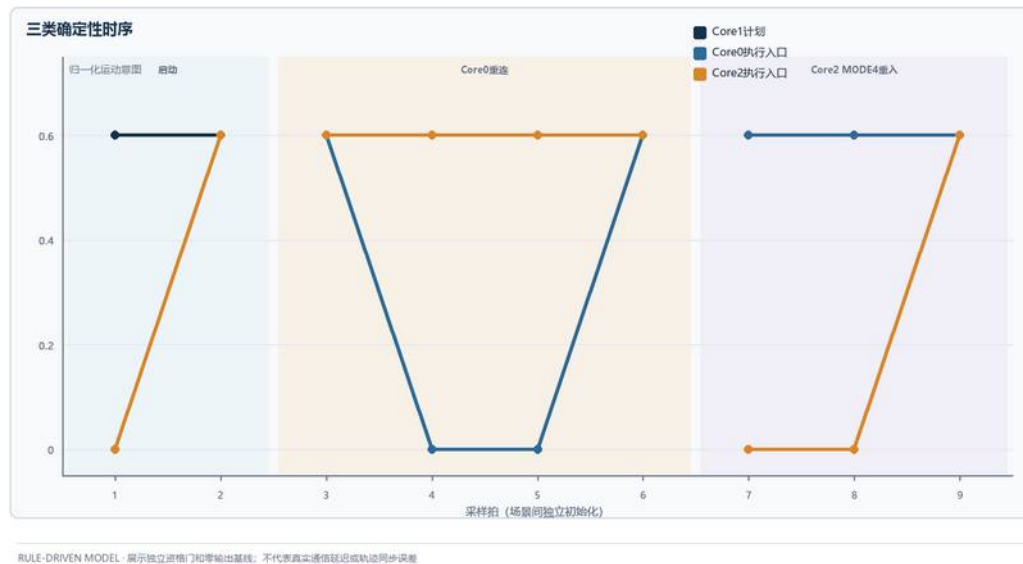


图 10.6 空地共享计划及独立重入门

图 10.6 表明，Core1 计划保持同一语义时，Core0 与 Core2 仍可因本机状态分别输出零或接受非零命令。该图是源码规则驱动的确定性模型，用于验证状态关系，不是实物通信延迟、飞机—底盘同步误差或机械系留响应曲线。逐拍数据见

[air_ground_coordination_trace.csv/json](#)

10.6 五板参数事务路径

参数事务模型以 Core0~Core4 五个参与节点构造正常 LOAD、参与者拒绝和协调器重启恢复三条路径。正常路径依次达到 5 板 PREPARE、5 板 COMMIT、5 板 ACTIVATE 并 FINISH；拒绝路径在 PREPARE 阶段只有 4 板接受，随后进入 REJECT 和 ABORT，不把部分准备解释为整机成功；重启路径在 5 板 COMMIT 后重建协调状态，再依据持久决策继续 ACTIVATE 和 FINISH。

五板参数事务：成功、拒绝与重启恢复

协议阶段确定性模型·纵轴为已准备、已提交和已应用节点数



图 10.7 五板参数事务成功、拒绝与恢复模型

图 10.7 展示已准备、已提交和已应用节点数随阶段变化。它证明协议阶段模型没有把部分参与者成功当成全局完成；限定历史提交上的五板硬件验收进一步证明已覆盖场景能够在真实板卡执行，但二者都不支持“任意断电瞬时原子”结论。机器可读结果见 `parameter_transaction_trace.csv/json`

10.7 ToF 分级故障逻辑

报告入口把活动 `altitudeHold.c` 与 ToF、飞行安全、参数仓库、PID 和飞控输出依赖桩共同编译，以一个 5 pp 的确定性初始油门偏移构造三条轨迹：2.1 s 时在 HOLD 内恢复、4.3 s 时在 RAMP 中恢复，以及持续故障至 11 s。前两条恢复后均返回 NORMAL 并重建 PID 基线；持续故障在 3 s 前保持原偏移，达到 3 s 后按 2 pp/s 下降，每次推进受 100 ms 步长上限约束，10.6 s 触达 -10 pp 下限并置位 `requestDisarm`。

ToF故障降级：生产C依赖桩Host时序

三种确定性场景·曲线为油门偏移输出，不是高度或着陆轨迹

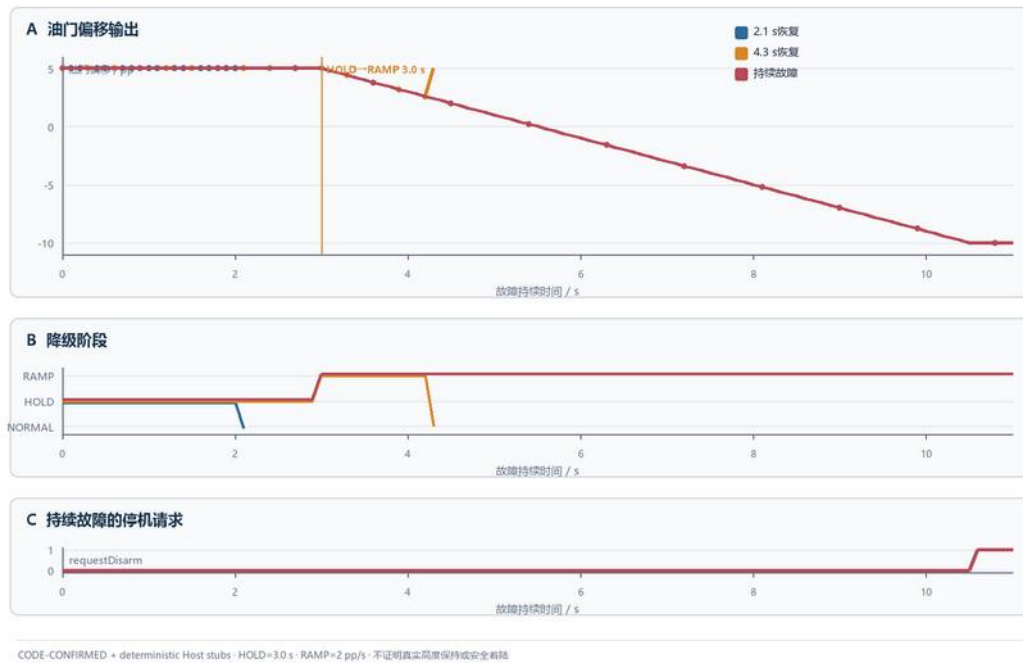


图 10.8 ToF 故障 HOLD—RAMP 生产 CHost 曲线

图 10.8 实际执行生产函数，但传感器、参数、PID 反馈和安全状态均为依赖桩，因此不是传感器闭环实测，也不是飞行高度或着陆曲线。图中停机请求只表示源码状态机在该桩参数数下达到油门偏移下限，不能外推为实机已经接地或可以安全停桨。原始序列见 tof_failsafe_logic.csv，状态常量、场景终值和首次请求时刻见 tof_failsafe_logic.json

表 10.4 ToF 确定性场景终值

场景	最后采样时刻	最后阶段	油门偏移	requestDisarm
HOLD 内恢复	2.1 s	NORMAL	5.0 pp	0
RAMP 中恢复	4.3 s	NORMAL	5.0 pp	0
持续故障	11.0 s	RAMP	-10.0 pp	1

10.8 板上配置、完赛确认与已登记硬件证据

证据台账中的 EV-C012-NEW-CHASSIS-SLOT5-20260812 记录了新底盘几何、参数槽 5 默认值、对应六个 IAR Debug 配置以及用户限定范围的板端验收；结论只绑定提交 46a0835db8d1062b69af06a2ca8c090fd23ae7c5 和该批范

围 。 EV-C01234-PARAM-SLOT-LOAD-SYNC-20260806 记录 了 提交 555ad8139c9212d1ac12b1355ed0122e00ab5896 上五板参数 LOAD 同步、事务锁、复位恢复和安全态应用的用户联合硬件验收。两项均没有原始串口或示波器附件，因此不得外推 Flash 寿命、任意断电窗口或全工况性能。

队伍于 2026-08-16 确认，报告采用的板上配置已经随当前固件烧录并能够完成比赛流程。该确认使板上实际配置优先于旧 Host 脚本中已经过时的默认值期望；报告生成器直接从固件证据基线提取配置并驱动仿真，不再用旧常量覆盖板上值。本轮没有附加完整赛程日志、录像或同步测量曲线，因此该结论只覆盖队伍确认的装机、配置和比赛流程，不表示所有光照、场地和故障组合均已逐项实测。

10.9 证据矩阵

Claim	Mechanism	当前证据	允许表述	禁止表述
三来源形成连续控制候选	精确身份保持、统一评分、EdgeHold、SEARCH、边界拒绝	当前代码；16 拍、三个独立生产 CHost 场景覆盖 Source1/3/4	实现并在构造场景验证状态转移	真实赛场无缝切换、始终保持同一物理目标
空地共享运动意图	Core1 唯一计划分别进入 Core0 与 Core2	当前代码；9 拍规则驱动模型；队伍完赛确认	同一计划驱动两个控制入口并分别受本机门控	飞机与车辆完全同步、误差为零
麦轮运动学方向和归一化代数一致	逆解、比例归一化、正解	6 场景模型仿真、61 点饱和扫描；历史限定硬件证据	模型内输入—正解一致，饱和时保持轮速比例	实车轨迹精度、全工况稳定
ToF 短失效保持、长期失效 RAMP	NORMAL/HOLD/RAMP 状态和源码常量	当前代码；生产 C 依赖桩 Host 场景	实现 3 s HOLD 及 2 pp/s RAMP 规则	保证高度不变或安全着陆
五板参数事务具备恢复机制	双银行、CRC、selector journal 和协调日志	当前代码；12 拍协议状态模型；限定范围历史 HW-CONFIRMED	五板已覆盖场景内完成事务与恢复	任意断电瞬时原子、永不丢失
信标检测具有可量化像素域	固定真值图像、板上关联门、单帧与	75 帧合成真值夹具；板上配置由基	报告表 10.3 及图 10.5 给出的像素域	赛场毫米精度或所有光照下保持相同

第二十一届全国大学生智能汽车邀请赛技术报告

Claim	Mechanism	当前证据	允许表述	禁止表述
指标	时域处理	线源码提取	结果	指标

所有生成图表均保留机器可读输入与 SHA-256。生成器连续两次运行后若清单哈希一致，即可证明报告资产在同一基线和工具环境下可重复生成；这一重复性仍不提升其物理保真等级。

第 11 章 技术创新、局限与总结

11.1 工程机制创新

本系统的创新定位是面向飞跃雷区任务的工程机制组合，而不是提出新的控制理论、SLAM、自动标定或全局目标识别算法。四项主创新从感知连续性、空地执行、安全恢复和多板参数一致性四个方面构成闭环。

第一项创新是多视野分布式感知与统一车体系目标连续性。Core1、Core3、Core4 三个视觉逻辑节点在相机附近完成检测和 tracker，只向连续性模块发布结构化候选；Core1 分别按下视、后视、前视几何路径转换到统一车体系，先硬保持精确 source/session/trackId，身份消失后才对全来源候选按历史预测重新评分。与当前 source/session/trackId 完全匹配且观测序号前进的边界拒绝会立即撤销旧 owner，普通遮挡则用有限 EdgeHold 衰减，最后才进入 SEARCH。这一结构减少视野切换时重复选目标和输出硬复位的机会，同时明确保留“没有跨相机全局 ID、无法保证同一物理信标”的边界。

第二项创新是同一运动意图驱动的有线空地协同。Core1 只生成一份 vx/vy/wz，通过有线链路同时进入 Core0 飞控和 Core2 麦轮控制。飞行端把它转化为速度目标、角度前馈和偏置转心补偿，地面端把它转化为四轮目标并经速度—电流闭环执行。目标选择、输出整形和 SEARCH 不在两端重复实现，因此飞机和车辆共享“要往哪里运动”的计划，同时各自保留符合动力学的反馈控制和故障门。

第三项创新是面向多节点系统的分级安全降级与受控恢复。系统区分姿态保护、任务链路/视觉故障、MOTION 失效、ToF 短失效、ToF 长失效和底盘断链，分别选择停桨、缓降、归零、HOLD、RAMP 或停车。首因锁存避免根因被连锁症状覆盖；session、序号、新鲜度、配置身份、重建基线和零帧 fence 防止旧命令在重连后自动复活。创新重点是让不确定输入沿明确路径收敛到受控状态，而不是承诺“绝对安全”。

第四项创新是五板参数库与可恢复的分布式参数事务。每板通过运行态、双银行、CRC 与选择日志管理本地参数，Core1 再协调准备、提交、应用、完成、回滚和重启恢复。事务把多板调参从五次独立按键操作提升为带身份、阶

段和结果的系统事件。限定历史提交已有五板硬件证据，但当前报告仍不把它称为瞬时原子写入或任意断电完全验证。

表 11.1 四项创新的闭环关系

任务问题	核心机制	直接价值	主要证据	当前边界
单相机视野有限、切换易停顿	三源资格化、统一几何、身份保持、EdgeHold	保持目标与计划的时间连续性	代码、生产CHost 场景	无全局 ID；证据不覆盖真实高速切换性能
飞机与车辆各自规划易冲突	唯一运动计划、双端不同执行器映射	共享运动意图并保留本机闭环	代码	证据不覆盖联合轨迹误差
多节点故障传播和旧命令复活	分类降级、首因、fence、受控重入	使异常沿可解释路径收敛	代码、部分历史证据	证据不覆盖全部物理故障组合
多板参数难以保持一致	双银行与分布式事务	可追踪准备、应用、回滚和恢复	限定历史五板硬件记录	证据不覆盖当前基线任意断电窗口

11.2 支撑性工程优势

四项主创新建立在一组相互配合的基础能力上。多速率串级飞控把 27 ms 视觉位置、9 ms 水平速度、3 ms 姿态和 1 ms 角速度/混控按带宽分层；IMU、光流与 ToF 的融合同时处理 yaw 不可观、旋转像移、杠杆臂、时延和高度相关量测噪声；麦轮底盘通过全向运动学、航向保持和速度—电流级联闭环将计划落实到四轮。

系统架构方面，五板十核按硬件 owner 与实时性隔离职责，M7 共享区使用 DCache 一致性快照和有界 SPSC 队列；Observer V3.2 通过两条独立 4 Mbps 专线传输轨迹、健康、配置和 KVM；控制权仲裁、新鲜度与序号门防止“有字节就算健康”。开发工具方面，远程 KVM、CoreT3 原始图像录制回放和分层证据台账降低了多板调试与复现成本。这些能力是主创新的机制支撑，不单独包装为未经对比实验支持的理论原创。

11.3 局限性

当前三来源连续性依赖安装外参、动态中心和 pose 有效性，尚无自动标定。trackId 只在单来源 session 内有效，多灯交叉时使用历史几何最近原则，可

能发生物理目标误接管。EdgeHold 只适用于短时观测空洞，不提供环境级预测。系统不进行三维重建，也没有 SLAM 地图。

飞行速度估计依赖光流纹理和 ToF 高度资格，六轴 IMU 的 yaw 不可长期观测。短时 ToF HOLD 只是有限连续性策略，不能证明高度不变；固定油门 RAMP 也不同于带健康高度反馈的精确着陆。100 mm 仅是软件触地判据。

有线空地协同共享运动意图，但飞机与车辆动力学、控制周期、机械连接和扰动不同，并不保证两者瞬时速度相同。麦轮打滑、轮径误差和滚子接触会影响里程和航向，当前证据不支持“轨迹精度领先”的结论。

分布式参数事务有明确恢复协议，但现有证据不覆盖 Flash 写入寿命和所有断电相位。远程 KVM 只覆盖 Core1、Core3、Core4，不覆盖 Core2；CoreT3 只用于调试录制，不是生产冗余。历史证据必须按绑定提交使用，不能外推为当前整机结论。

11.4 总结

本文完成了飞跃雷区空地系留系统的总体设计论证。系统以五个 CYT4BB7 生产控制节点、十个 M7 核心分担飞控、三视野感知、通信、麦轮执行和人机交互，其中 Core0/Core1 两颗器件集成在同一张主飞控 PCB 上；CoreT3 作为独立录制工具提供可复现的视觉素材。三路信标相机经各自几何路径统一到车体系，目标连续性模块产生唯一运动计划；第四路 Core0 本地相机服务车体光学标志视觉，不进入三来源连续性。该计划通过有线链路同时驱动飞行器与地面车，并分别进入多速率飞控和麦轮级联闭环。

装机记录给出的飞行器、底盘和系留线质量分别为 870 g、2500 g 和 100 g，整套系统总质量 3470 g；整车所有电路板离散电容名义容量合计约 2000 μF 。整套系统常用输入功率约 600 W，大幅换向等剧烈机动瞬时峰值约 1500 W。飞行器使用四个 F60PRO 无感无刷电机，底盘使用四个 RC380 有刷电机，额外伺服电机数量为 0。上述物理数据没有配套原始称重单、同步电压电流曲线或 BOM 统计表，因此只作为队伍装机与运行记录，不用于证明效率、持续峰值能力或未测安全额定值。

三路信标视觉的板上配置为 188×120 输入和 50 fps 采集，图像坐标标称分辨率为 1 像素；Core1 以 10 ms 周期更新规划，对应 100 Hz 计划更新上限。配置

值由固件证据基线源码复核，像素定位误差由固定合成真值夹具给出，第 10 章将二者作为报告检测指标并保留来源边界。队伍确认当前板上配置随固件烧录后能够完成比赛流程；本轮未提供原始完赛日志，因此不外推所有场地和故障组合。

面对多节点不确定性，系统没有依赖一个笼统在线标志，而是对姿态、ToF、视觉、配置、链路、共享内存和命令新鲜度建立分层资格；异常发生时按类别进入保护、归零、缓降、HOLD、RAMP 或停车，恢复时重新建立身份、序号、配置和基线。五板参数事务进一步使运行配置能够被验证、应用、回滚和恢复。

本文已经给出四项创新的机制、数据流、可复现实验、证据与边界。未形成原始记录的物理性能不在正文中补造；后续若取得绑定最终固件和明确工况的曲线、日志或视频，只按其实际覆盖范围提升相应结论，不追溯性扩大当前证据等级。

参考文献

- [1] 全国大学生智能汽车竞赛组委会. 第二十一届全国大学生智能汽车竞赛技术报告提交通知 (<https://zhuoqing.blog.csdn.net/article/details/163552826>) [EB/OL]. 访问日期: 2026-08-12.
- [2] 全国大学生智能汽车竞赛组委会. 第二十一届全国大学生智能汽车竞赛技术报告要求细则 (<https://zhuoqing.blog.csdn.net/article/details/163553155>) [EB/OL]. 访问日期: 2026-08-12.
- [3] Robert Mahony, Tarek Hamel, Jean-Michel Pflimlin. Nonlinear Complementary Filters on the Special Orthogonal Group[J]. IEEE Transactions on Automatic Control, 2008, 53(5): 1203-1218.
- [4] R. E. Kalman. A New Approach to Linear Filtering and Prediction Problems[J]. Journal of Basic Engineering, 1960, 82(1): 35-45.
- [5] Bengt Ilon. Wheels for a Course Stable Selfpropelling Vehicle Movable in Any Desired Direction on the Ground: US, 3876255[P]. 1975.
- [6] Infineon Technologies AG. TRAVEO T2G CYT4BB Series Technical Reference Manual[Z].
- [7] IAR Systems. IAR Embedded Workbench for Arm User Guide[Z].
- [8] 项目组. docs/EVIDENCE_STATUS.md、Core0 ~ Core4 PROJECT.md 与 ARCHITECTURE.md、Observer V3.2 协议源码、参数持久化文档 [Z]. 基线 main@16d670d2b7141bfc37bc3d2403d695ffdda1a428, 2026-08-12.
- [9] 济海追风 1 队. 《第 21 届智能车赛_嘉立创开源奖项申请文本》 [Z]. 本地材料 SHA-256 : 754971e7727226807e7d728a80d133c507241f0af1737880fc3062996f408c97 , 核对日期: 2026-08-13.
- [10] lcet7122. STC 电调 (AM32 固件移植) (<https://oshwhub.com/lcet7122/stc-am32-firmware-porting>) [EB/OL]. 申请材料记载参考工程页面标注 CC BY-SA 3.0; 正式公开前仍须核对驱动程序附件自身许可证与再发布要求, 核对日期: 2026-08-13.

附录 A 详细原理图与 PCB 图

Purpose: 保存技术报告附录使用的完整原理图与 PCB 图片导航。

Authority: 图片内容以 hardwareInfoPicture/ 原始资产为准。

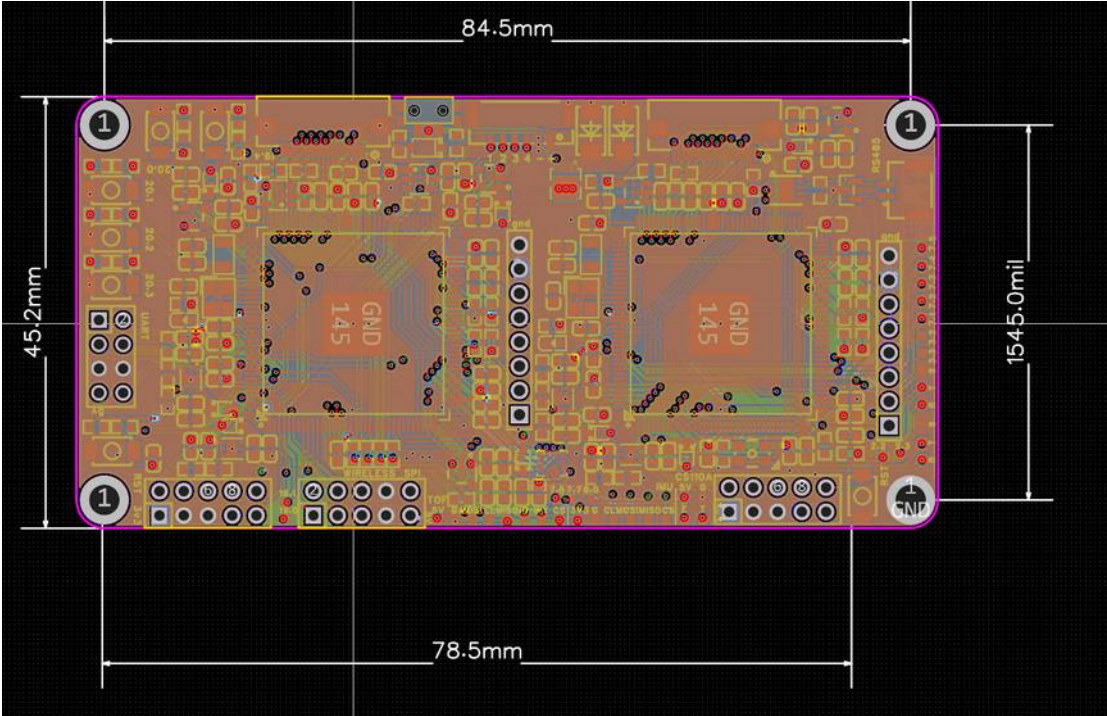
Sources: 主飞控板、电调、观察板、驱动板、分电板和灯板原图。

Not owned: 不从图片外观推断未标注电气性能，不复制或压缩原图。

Evidence status: 图片路径和文件存在性已核对；电气性能不由本导航证明。

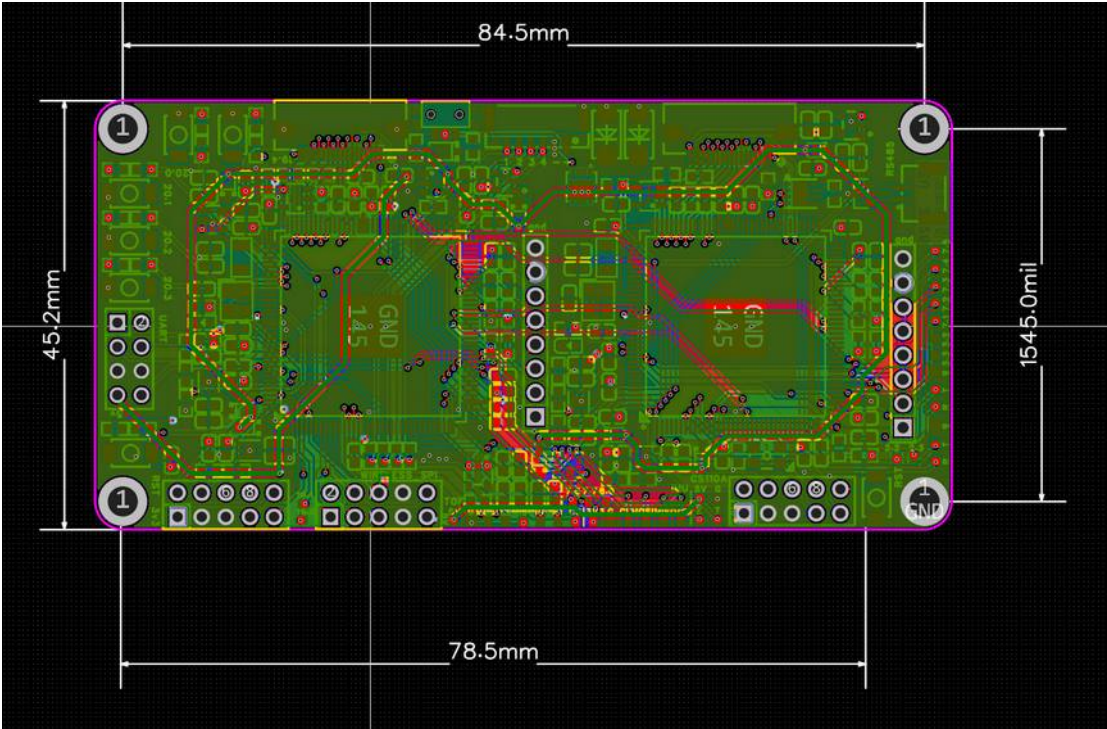
以下图片直接引用原始资产，未复制、压缩或重绘。图题仅标识板卡、原理图页或 PCB 层。

主飞控板



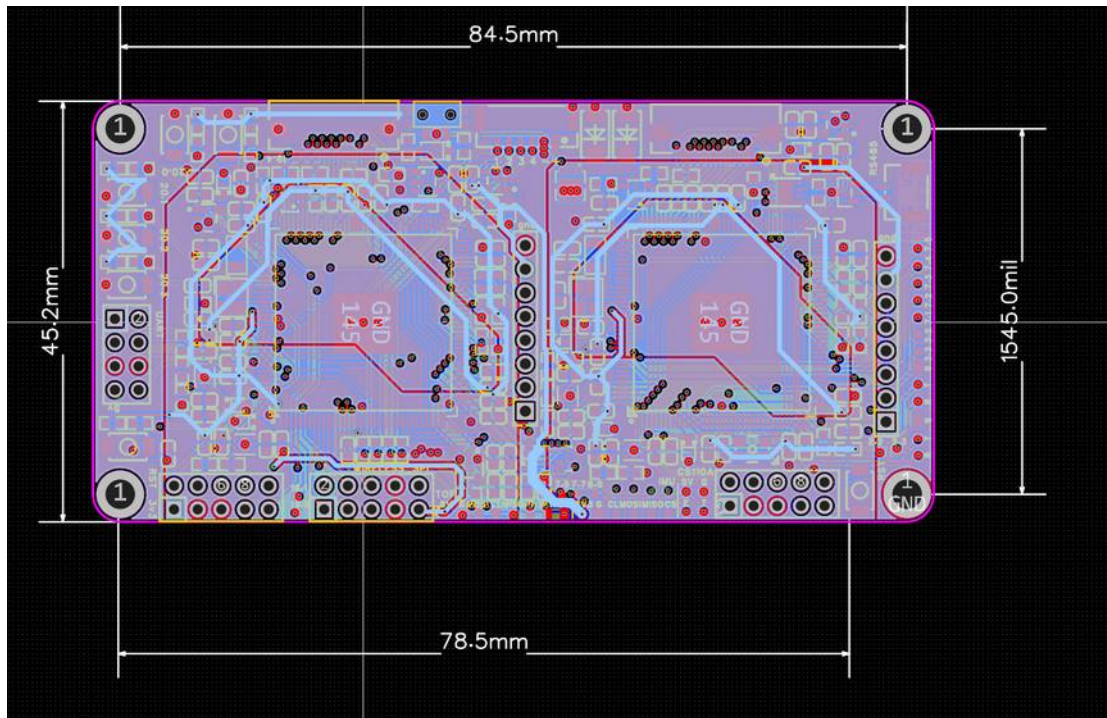
附图 A-1 主飞控板-PCB 内层 1

图 A-1 主飞控板-PCB 内层 1



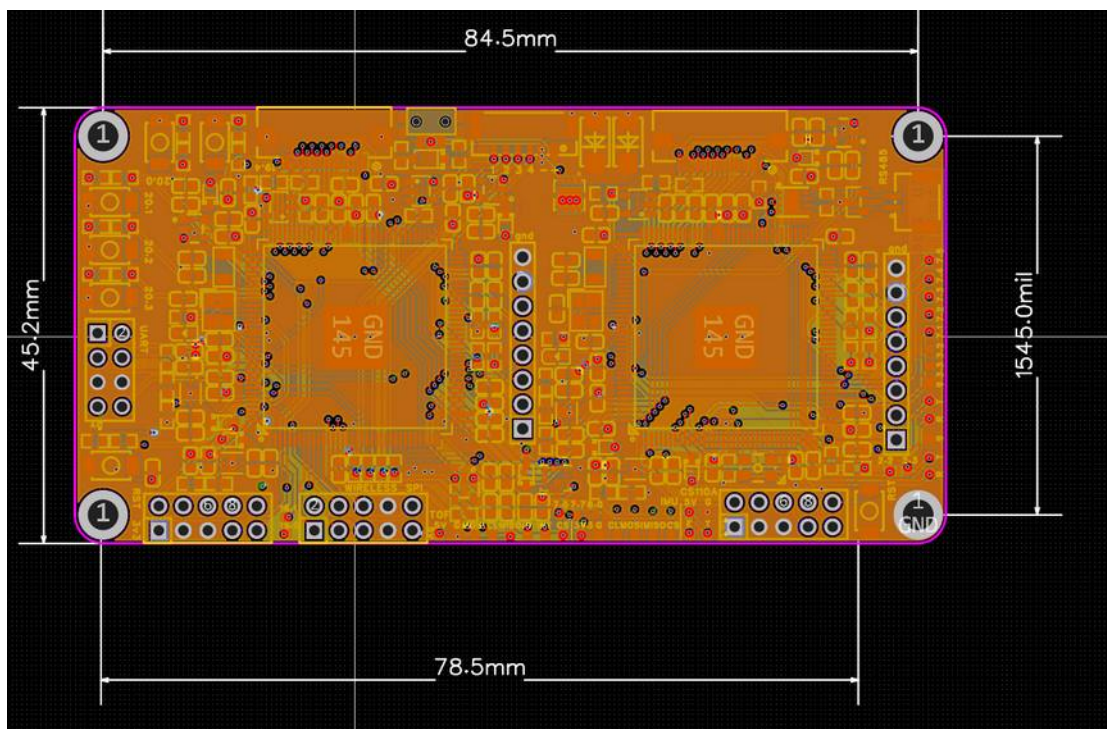
附图 A-2 主飞控板-PCB 内层 2

图 A-2 主飞控板-PCB 内层 2



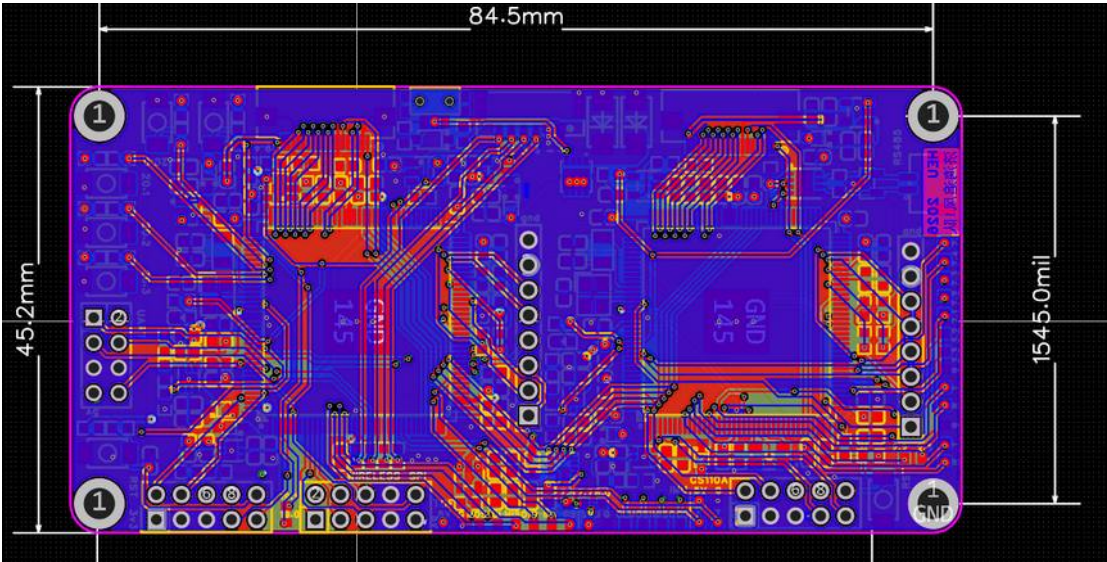
附图 A-3 主飞控板-PCB 内层 3

图 A-3 主飞控板-PCB 内层 3



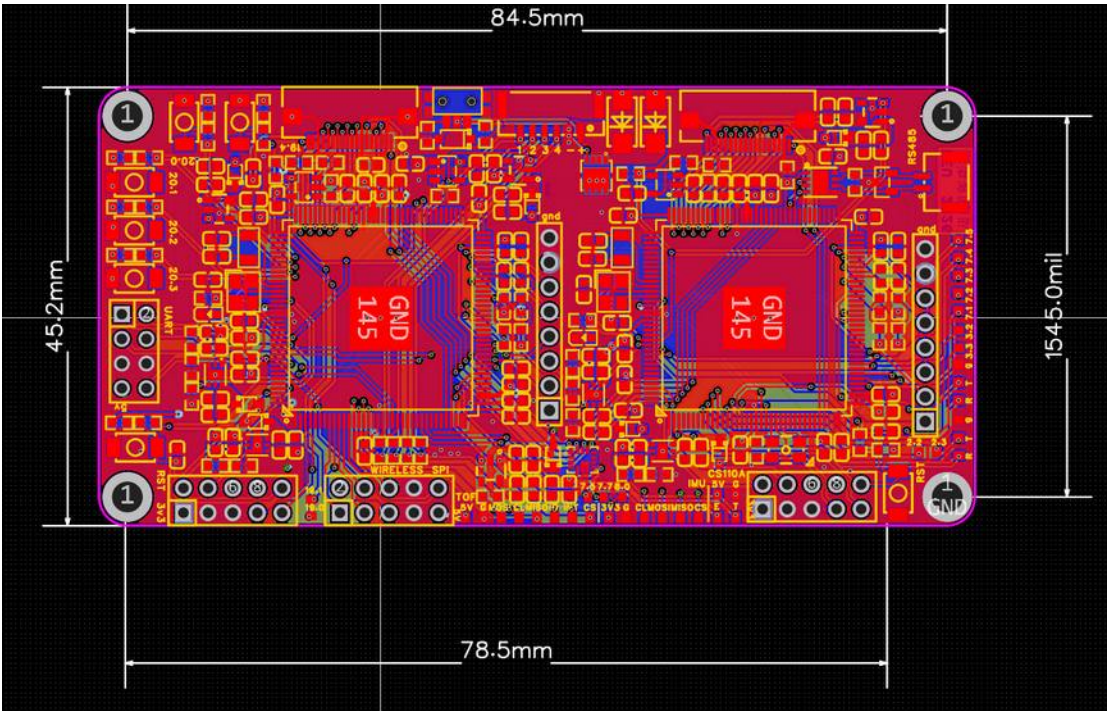
附图 A-4 主飞控板-PCB 内层 4

图 A-4 主飞控板-PCB 内层 4



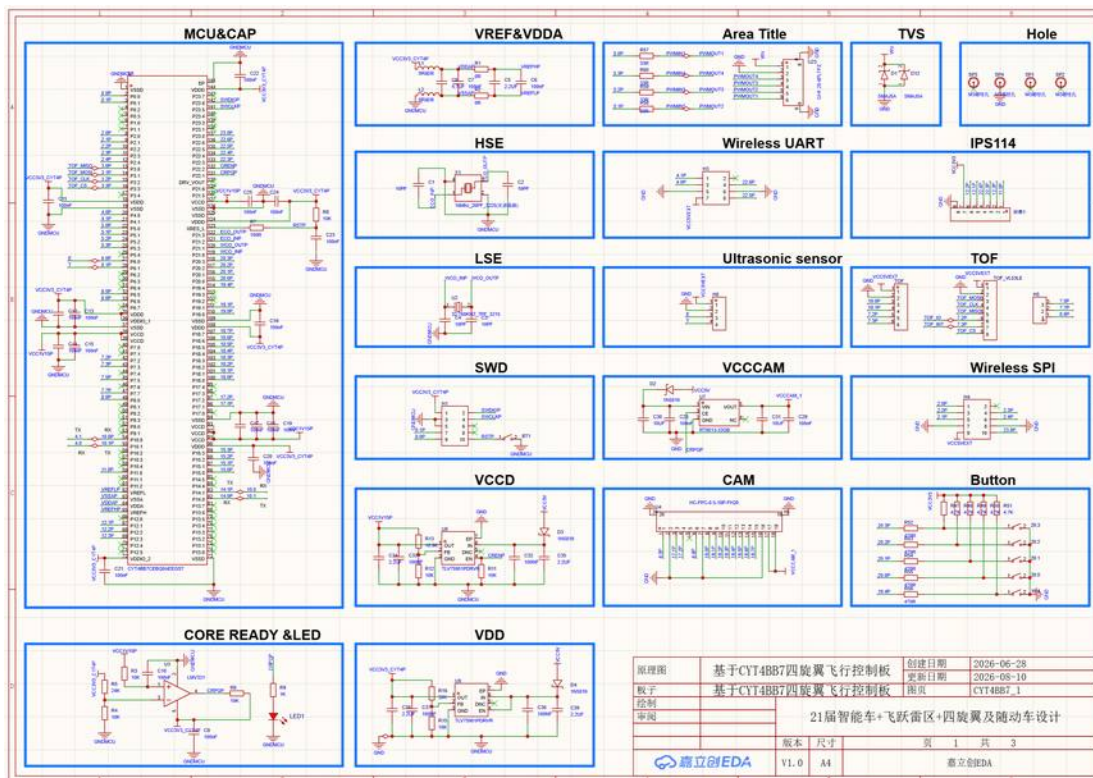
附图 A-5 主飞控板-PCB 底层

图 A-5 主飞控板-PCB 底层



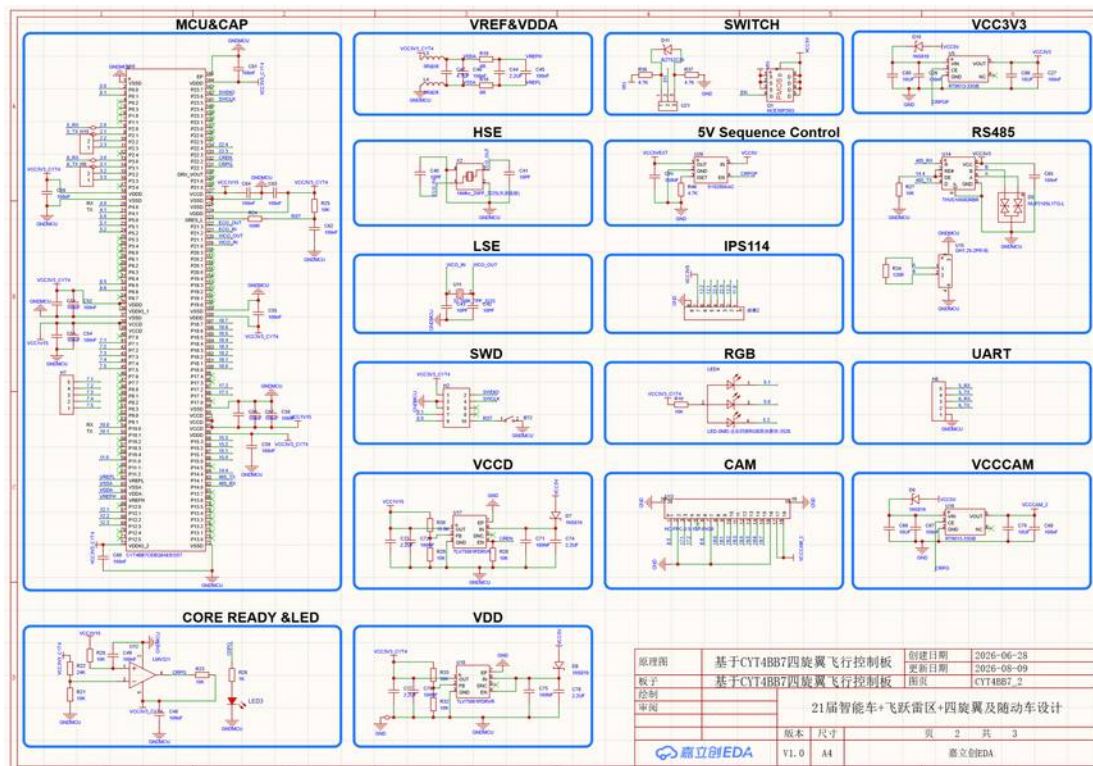
附图 A-6 主飞控板-PCB 顶层

图 A-6 主飞控板-PCB 顶层



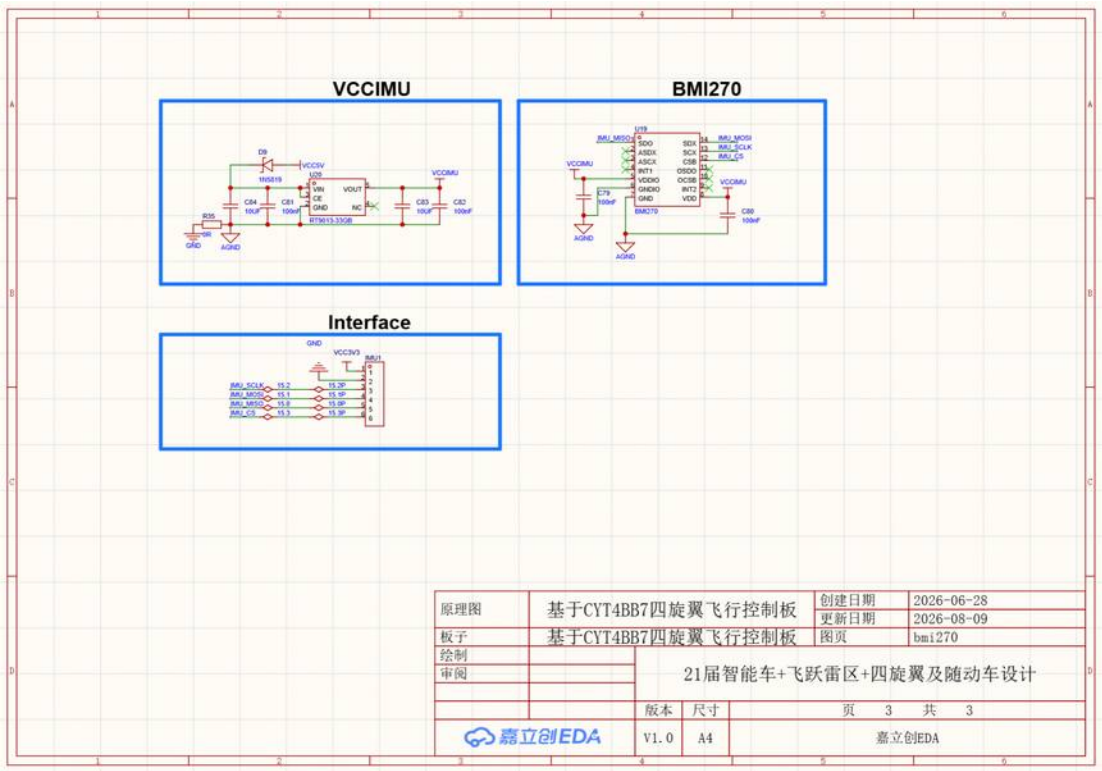
附图 A-7 主飞控板-原理图 P1

图 A-7 主飞控板-原理图 P1



附图 A-8 主飞控板-原理图 P2

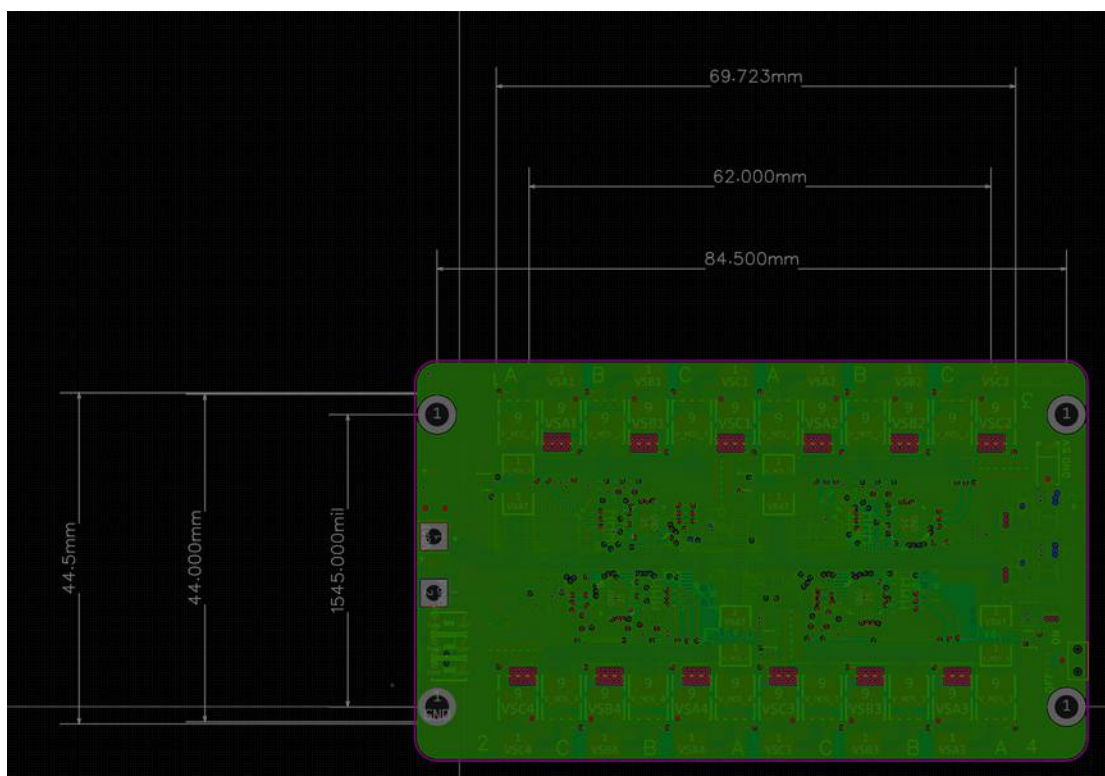
图 A-8 主飞控板-原理图 P2



附图 A-9 主飞控板-原理图 P3

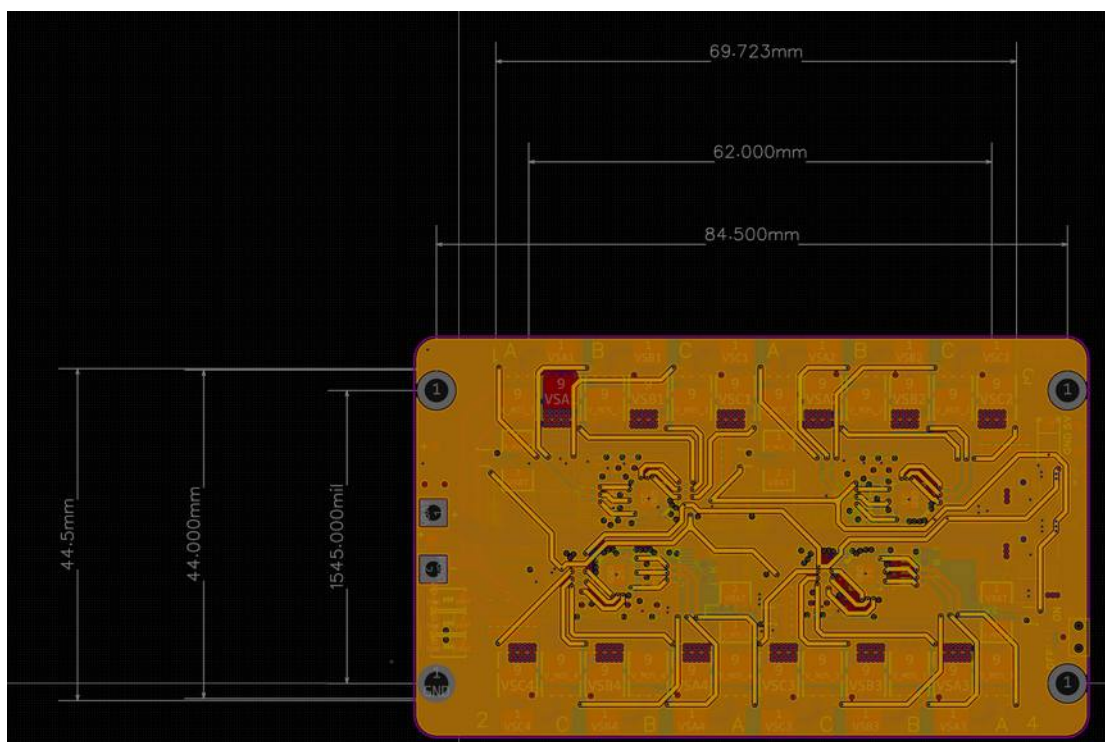
图 A-9 主飞控板-原理图 P3

四合一电子调速器



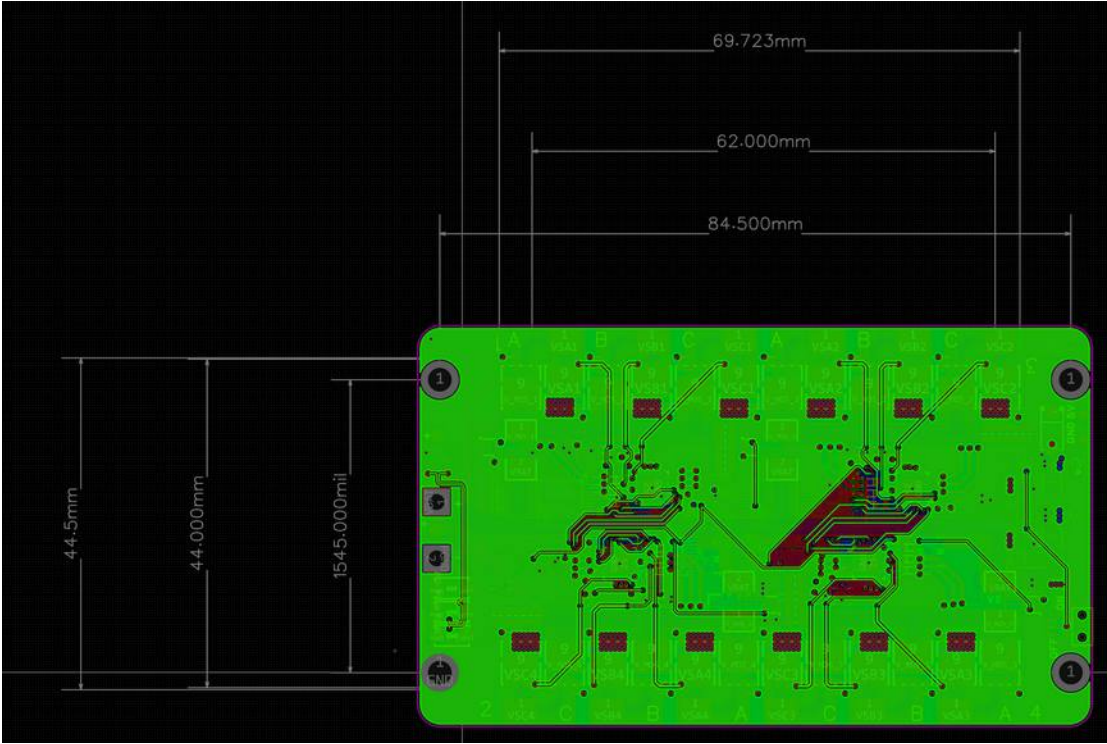
附图 A-10 四合一电子调速器-PCB 内层 1

图 A-10 四合一电子调速器-PCB 内层 1



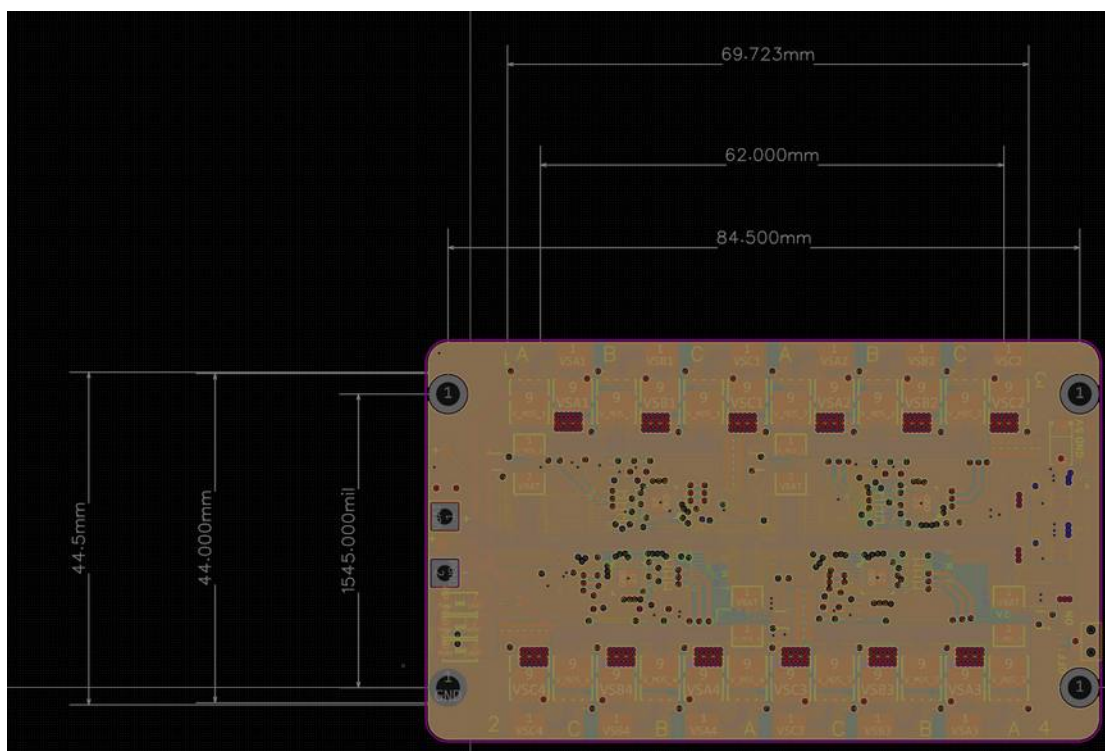
附图 A-11 四合一电子调速器-PCB 内层 2

图 A-11 四合一电子调速器-PCB 内层 2

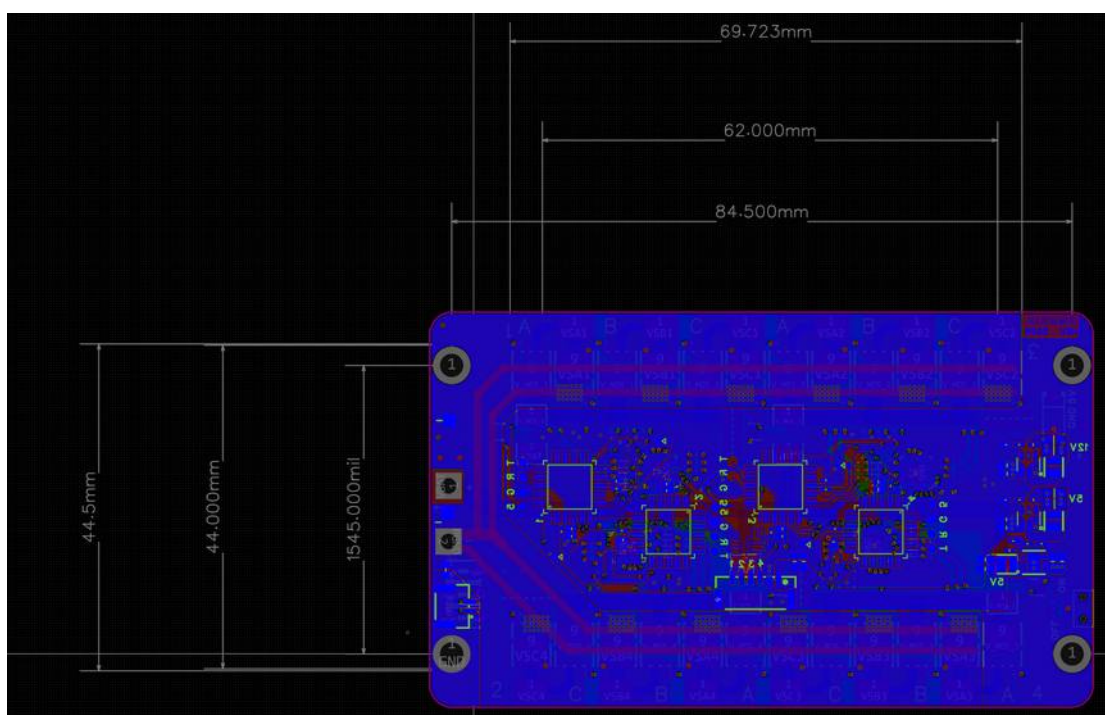


附图 A-12 四合一电子调速器-PCB 内层 3

图 A-12 四合一电子调速器-PCB 内层 3

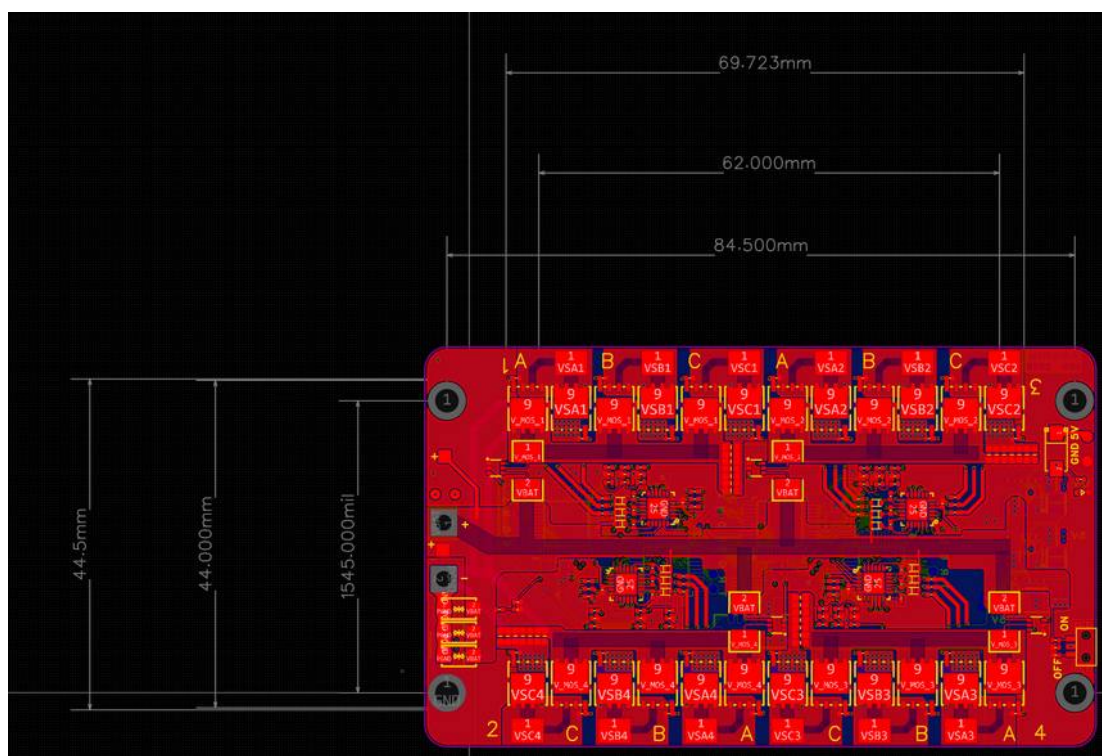


附图 A-13 四合一电子调速器-PCB 内层 4



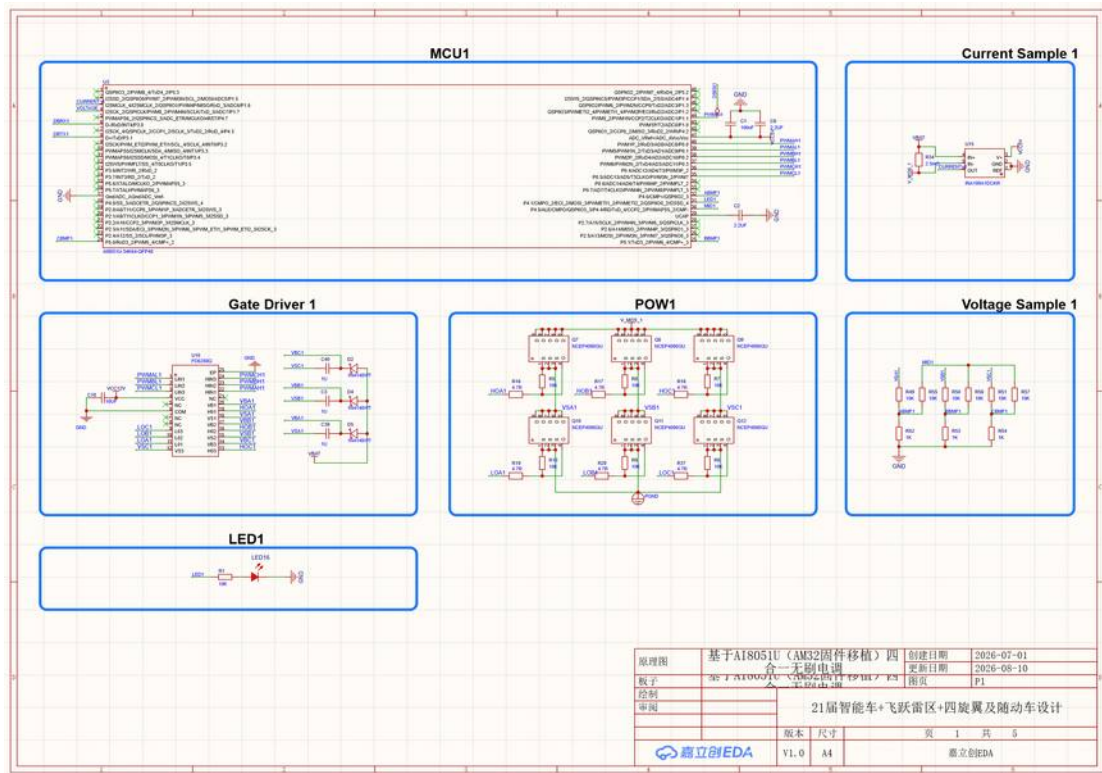
附图 A-14 四合一电子调速器-PCB 底层

图 A-14 四合一电子调速器-PCB 底层



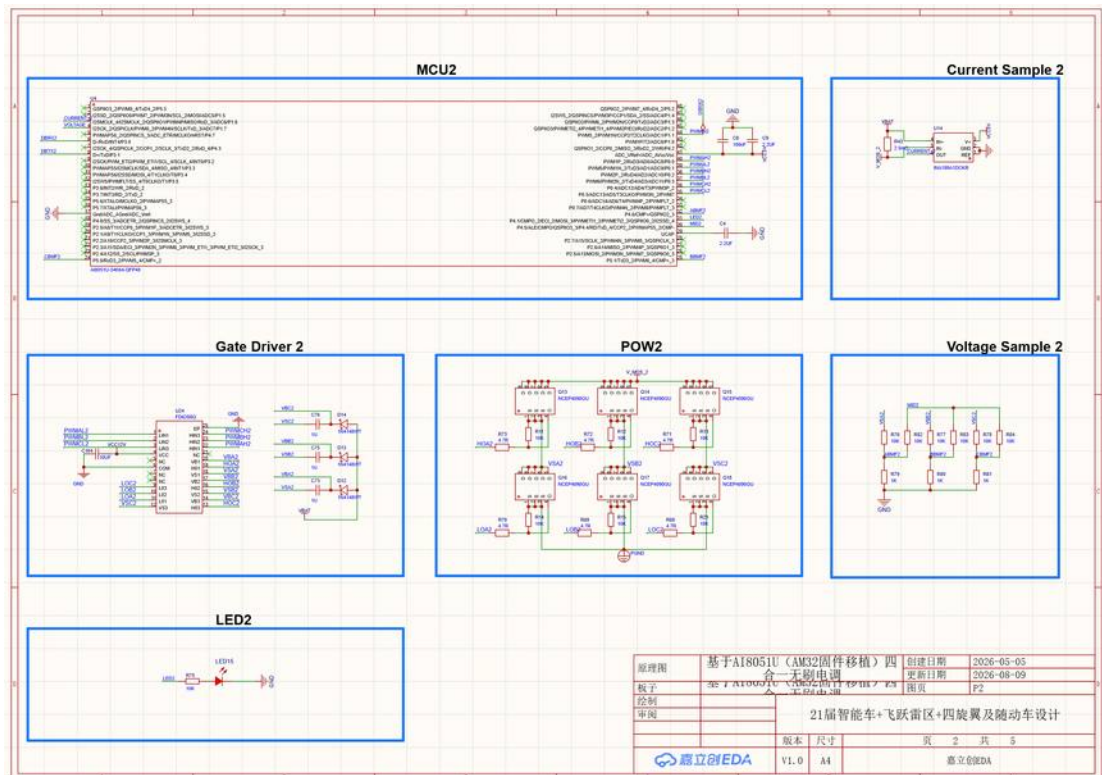
附图 A-15 四合一电子调速器-PCB 顶层

图 A-15 四合一电子调速器-PCB 顶层



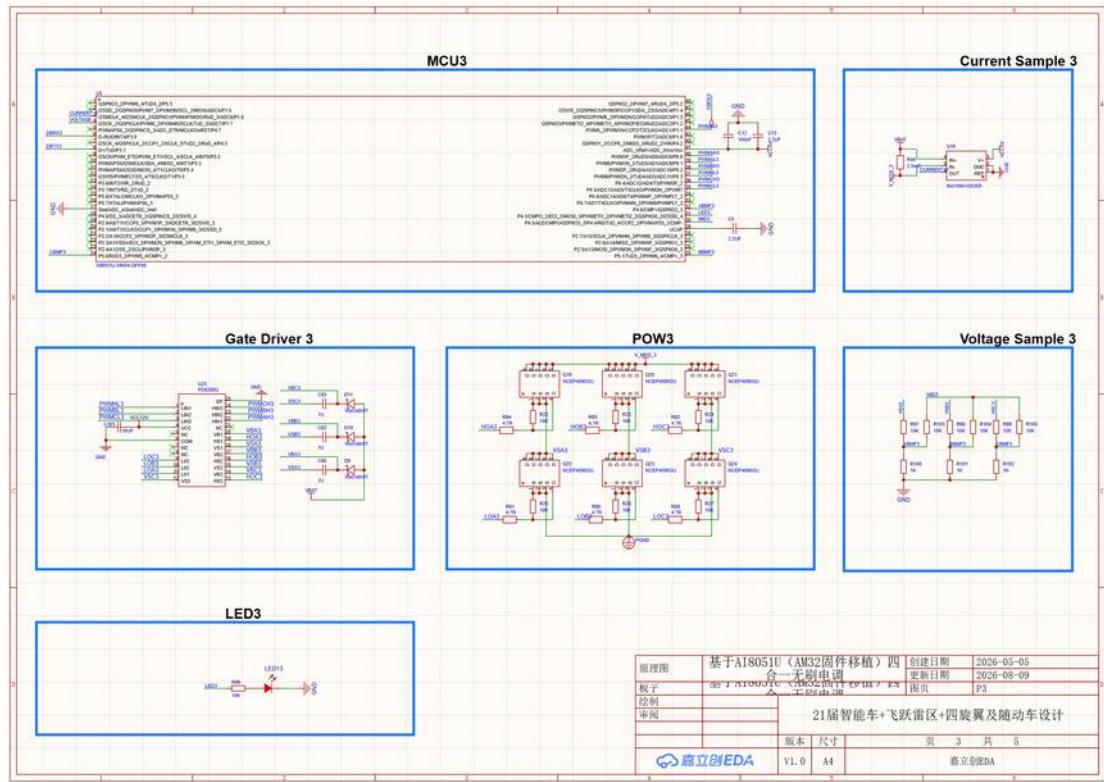
附图 A-16 四合一电子调速器-原理图 P1

图 A-16 四合一电子调速器-原理图 P1



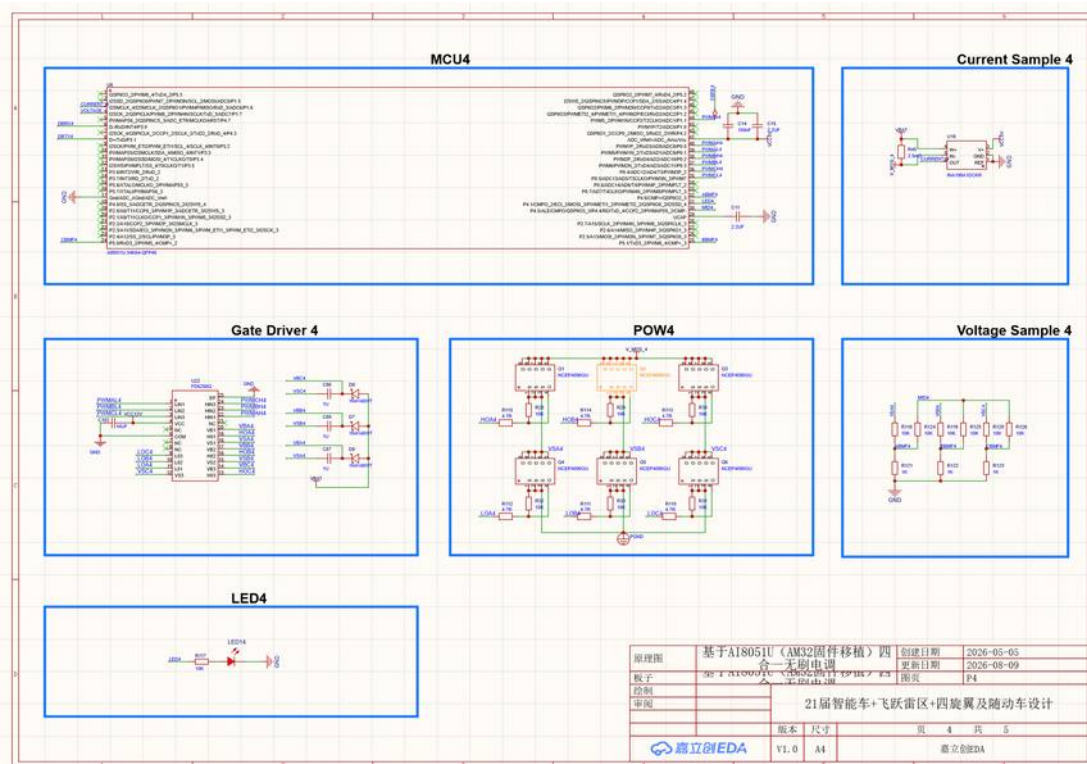
附图 A-17 四合一电子调速器-原理图 P2

图 A-17 四合一电子调速器-原理图 P2



附图 A-18 四合一电子调速器-原理图 P3

图 A-18 四合一电子调速器-原理图 P3



附图 A-19 四合一电子调速器-原理图 P4

图 A-19 四合一电子调速器-原理图 P4

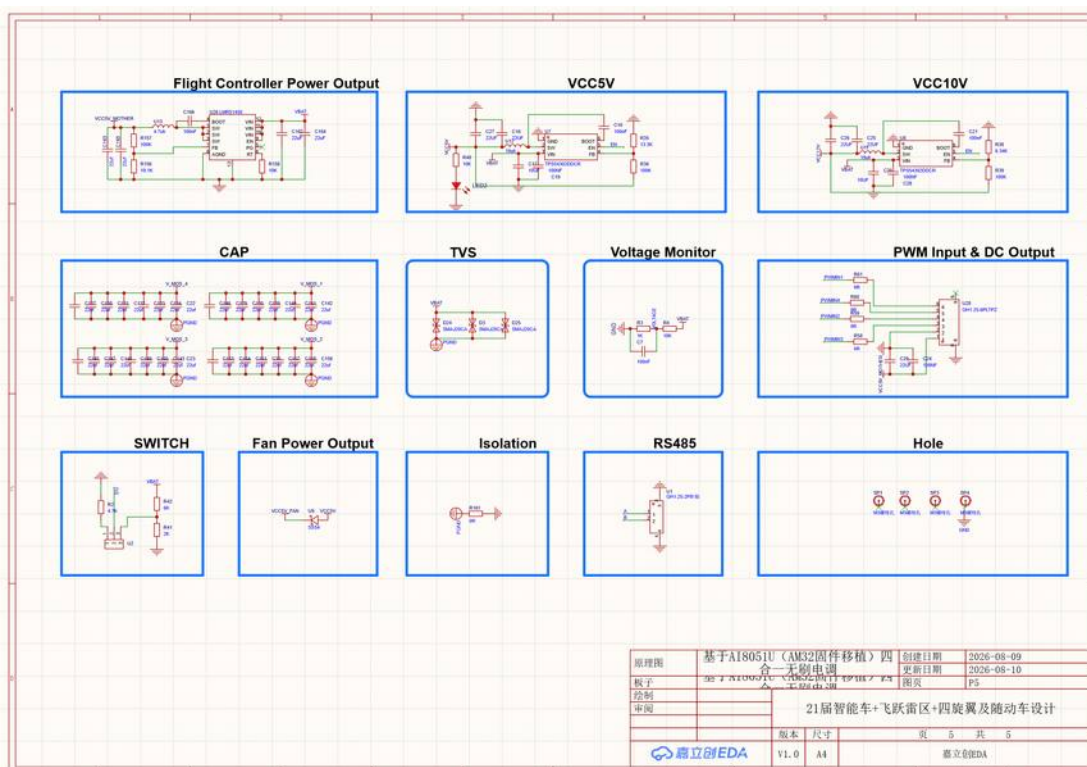
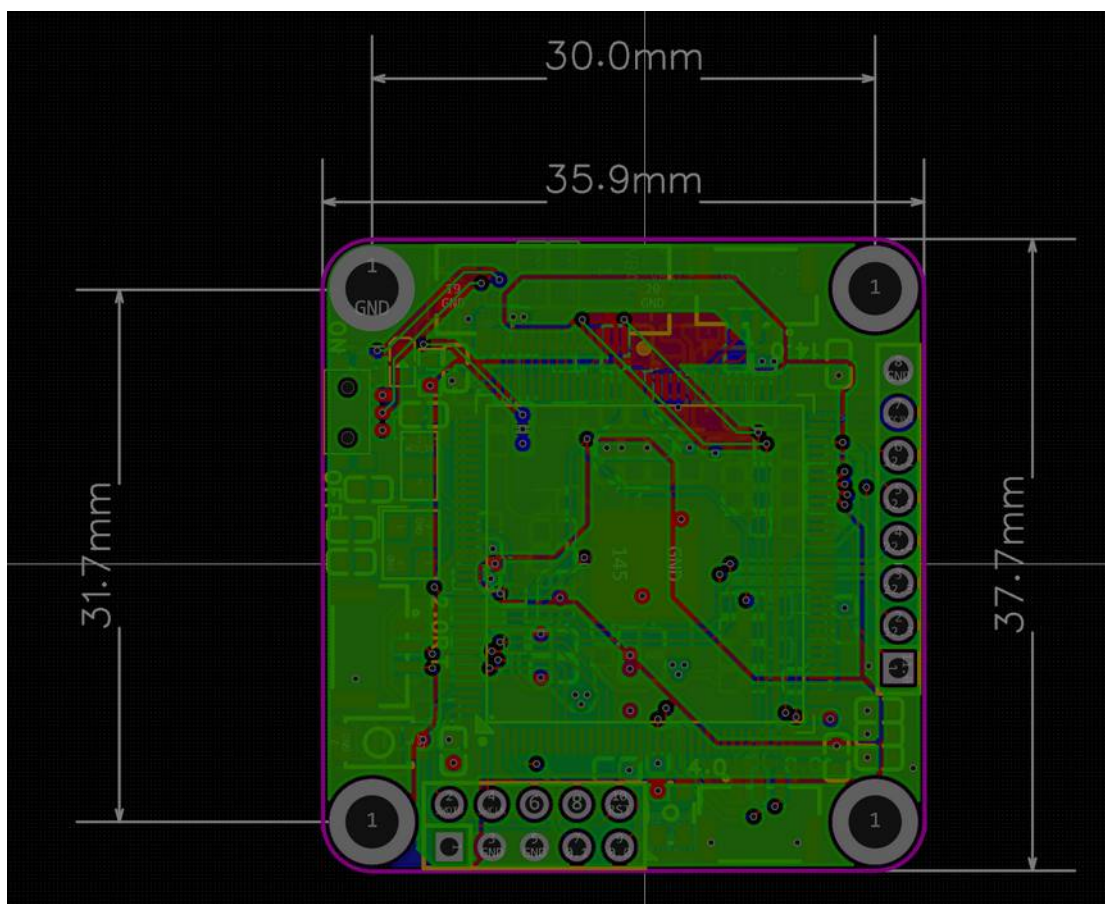


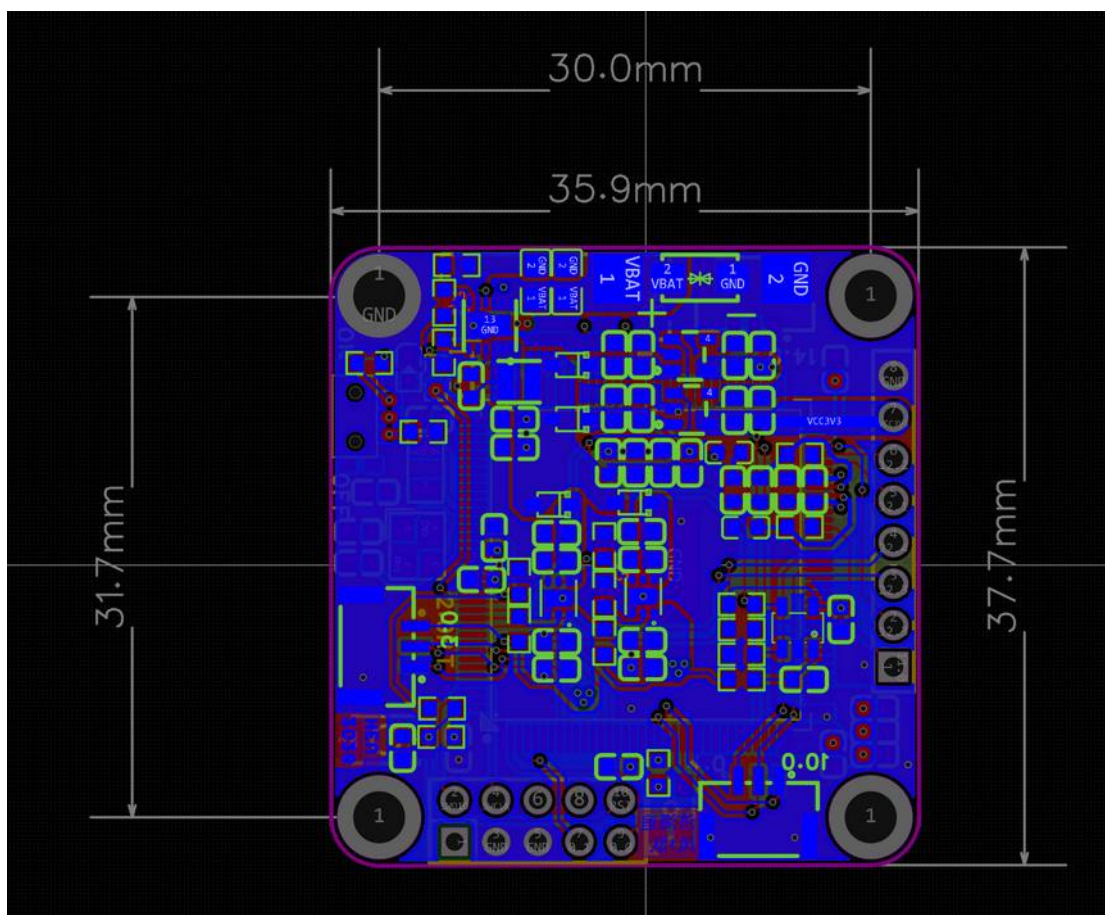
图 A-20 四合一电子调速器-原理图 P5

图 A-21 观察板-PCB 内层 1



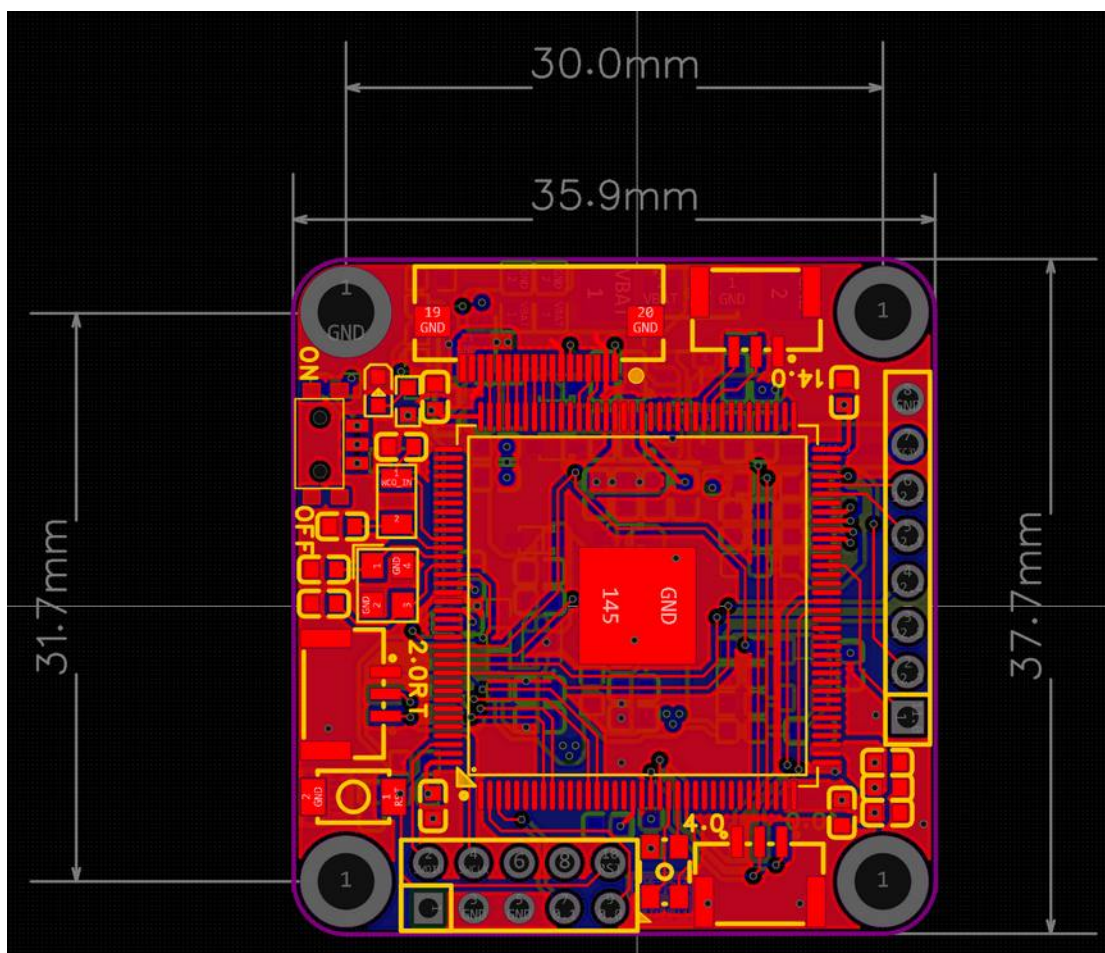
附图 A-22 观察板-PCB 内层 2

图 A-22 观察板-PCB 内层 2



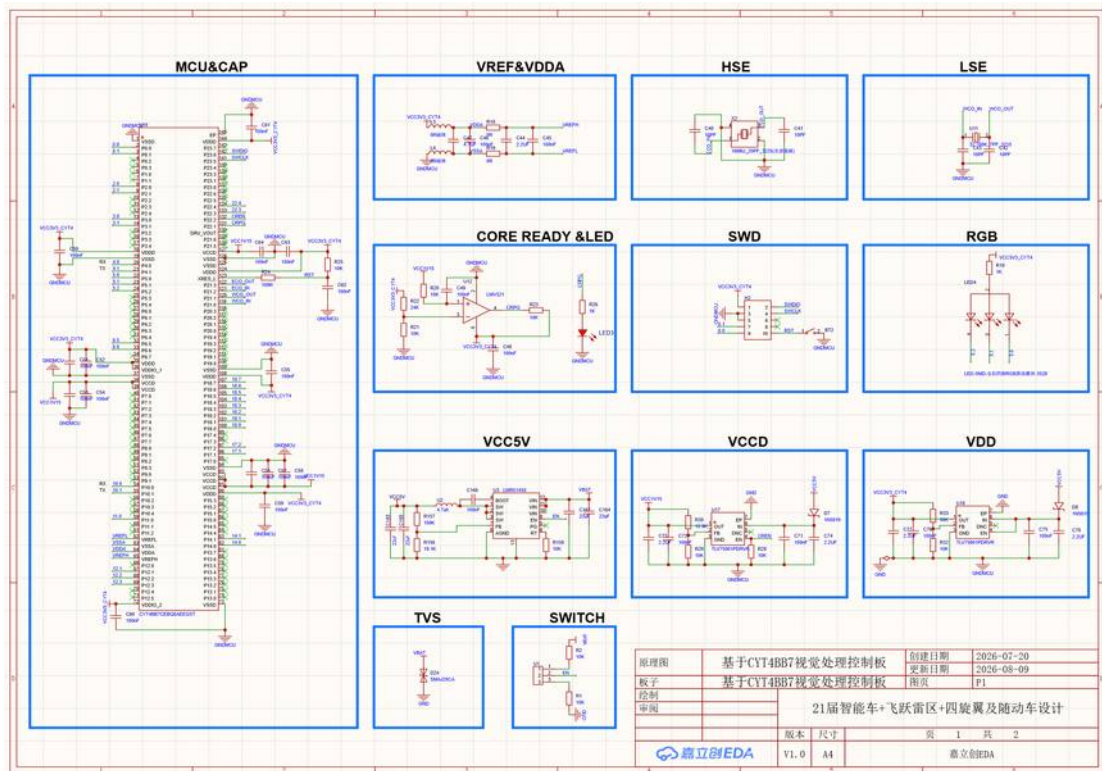
附图 A-23 观察板-PCB 底层

图 A-23 观察板-PCB 底层



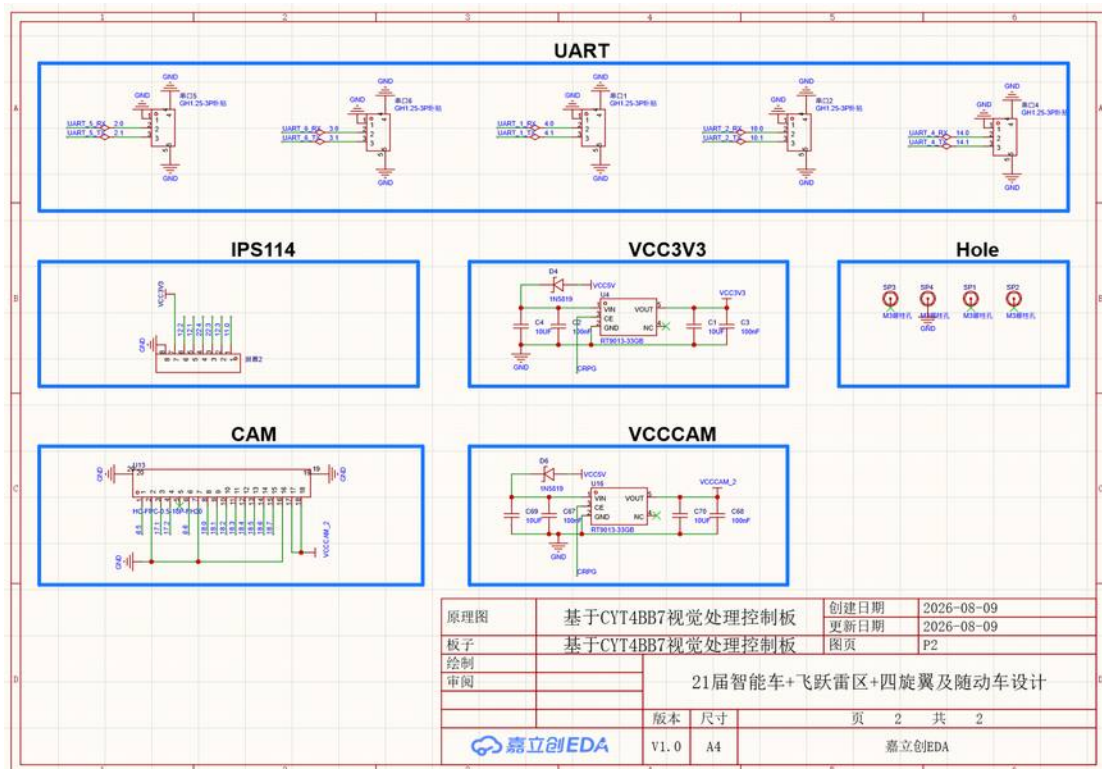
附图 A-24 观察板-PCB 顶层

图 A-24 观察板-PCB 顶层



附图 A-25 观察板-原理图 P1

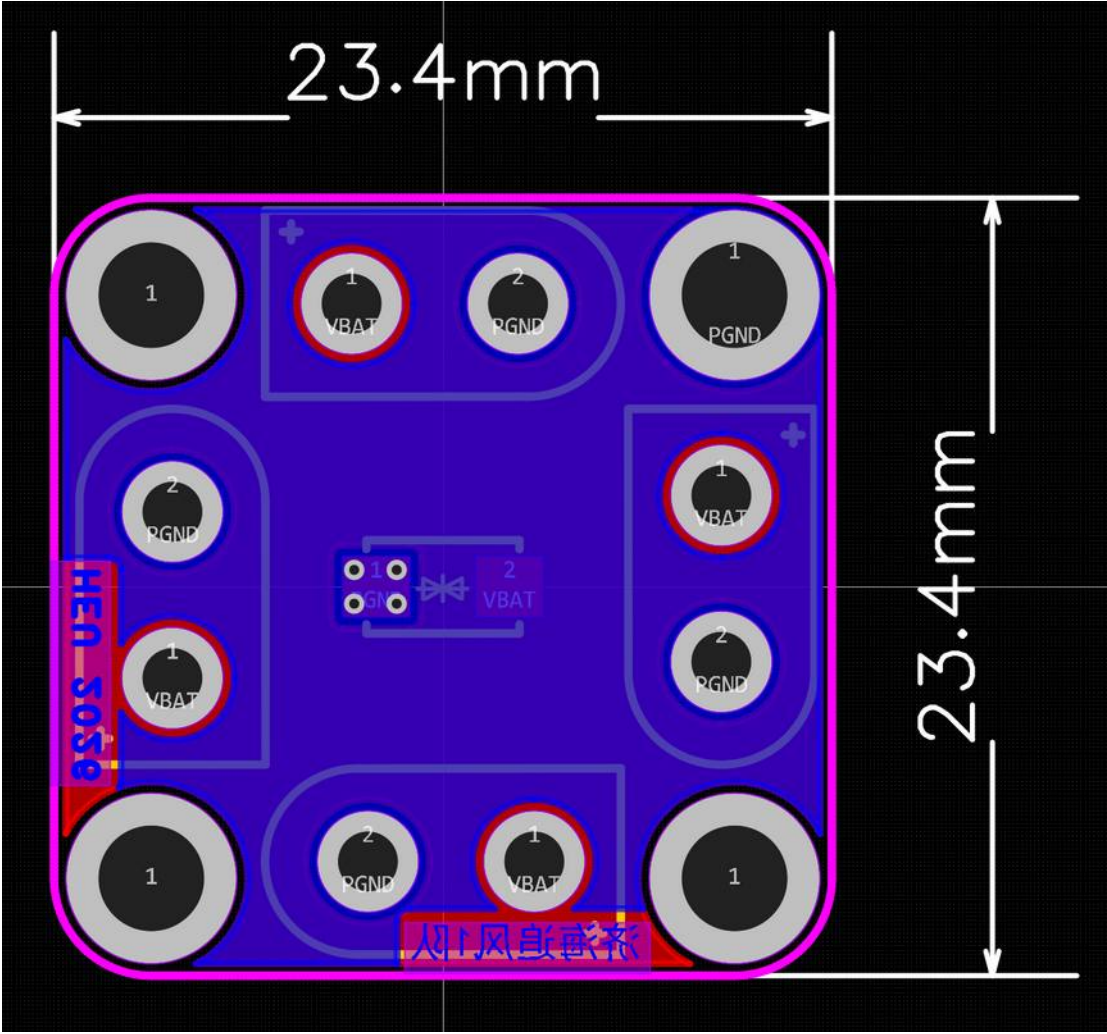
图 A-25 观察板-原理图 P1



附图 A-26 观察板-原理图 P2

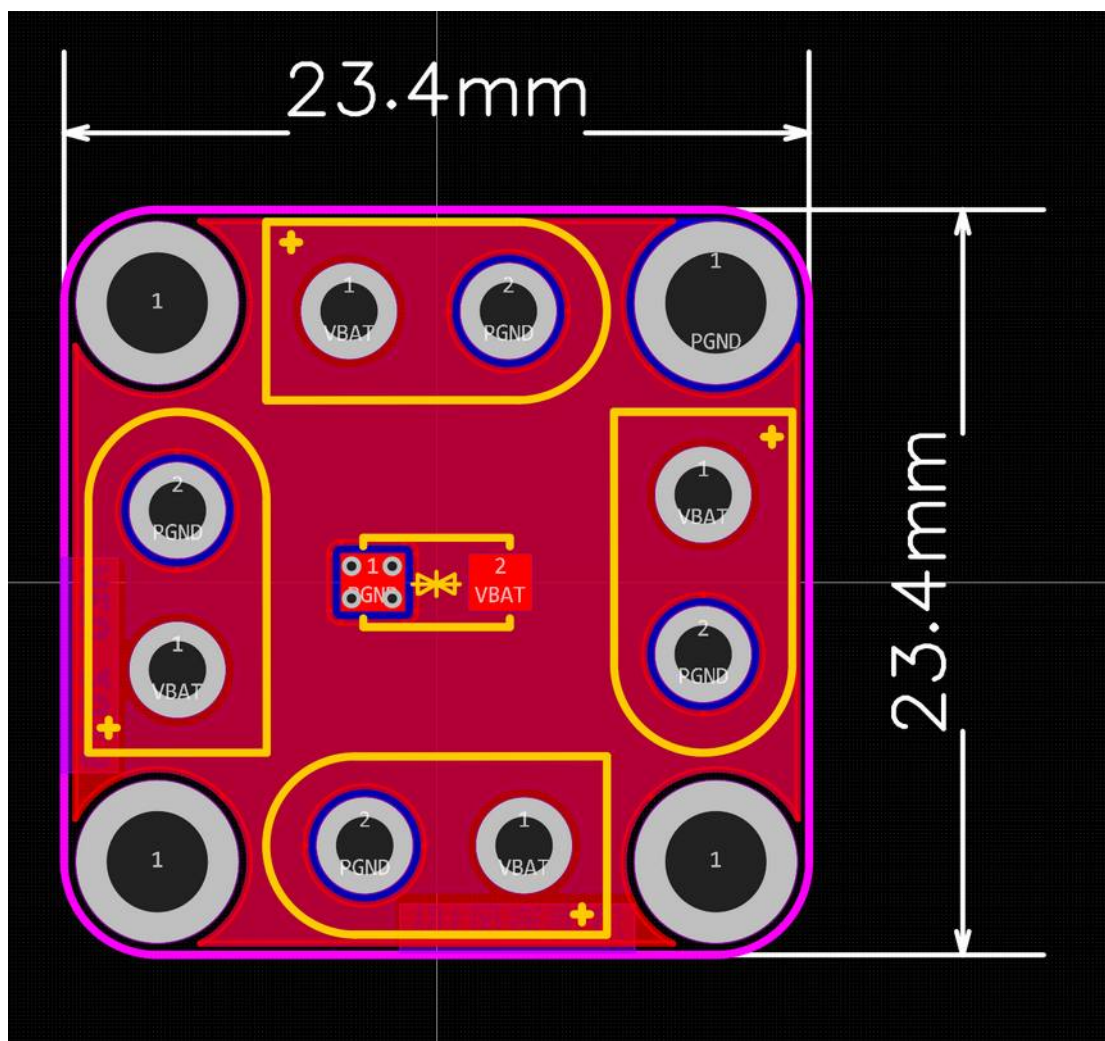
图 A-26 观察板-原理图 P2

观察板分电板



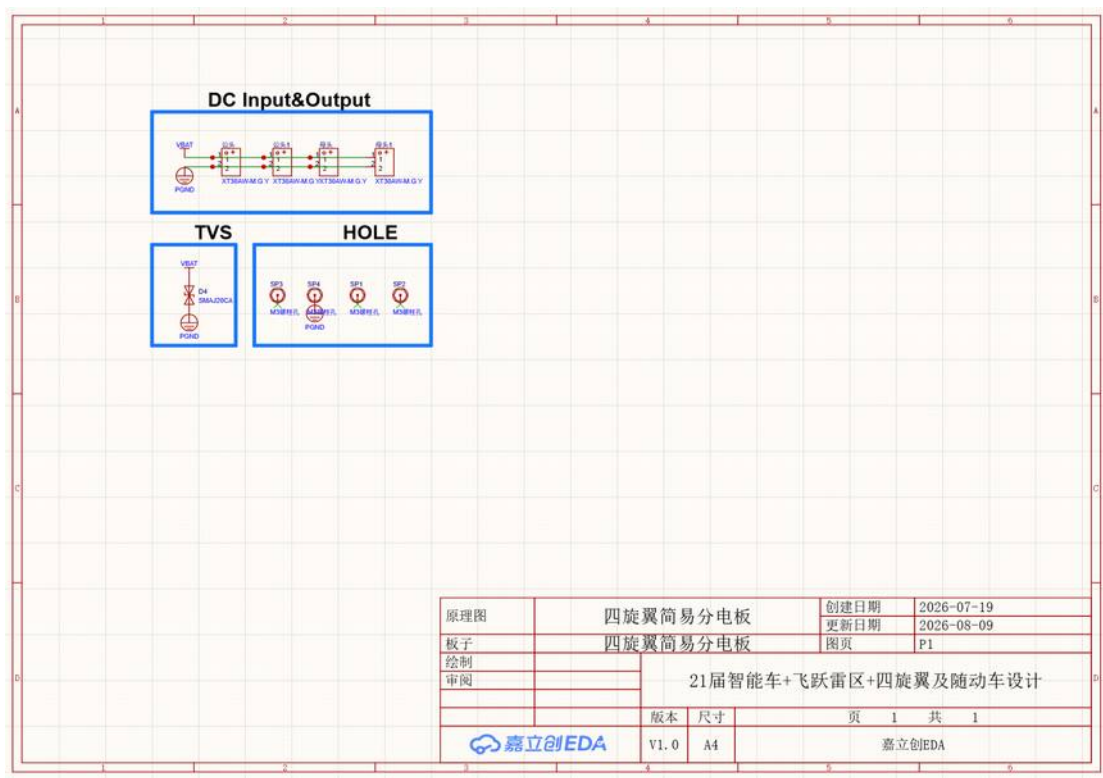
附图 A-27 观察板分电板-PCB 底层

图 A-27 观察板分电板-PCB 底层



附图 A-28 观察板分电板-PCB 顶层

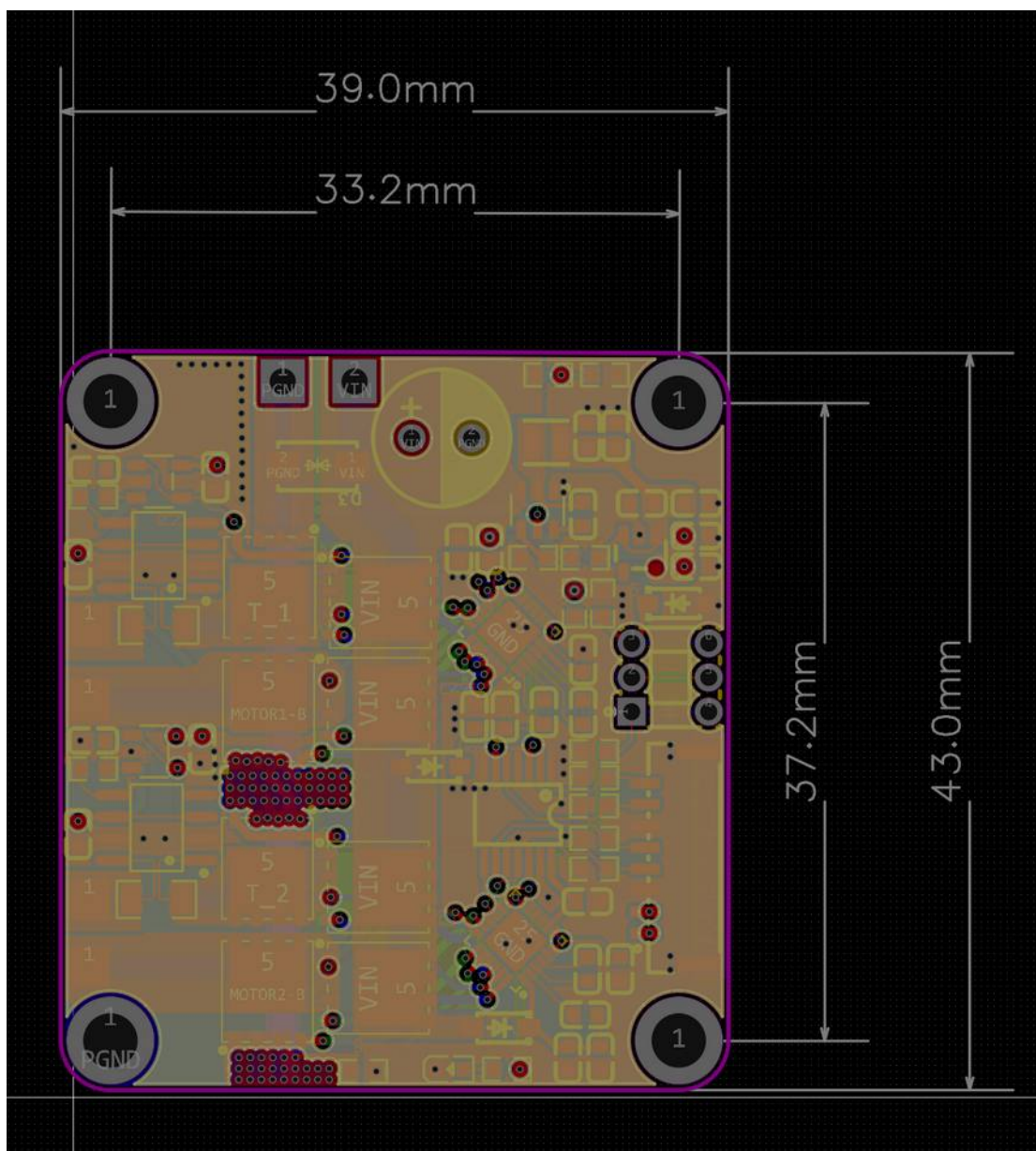
图 A-28 观察板分电板-PCB 顶层



附图 A-29 观察板分电板-原理图

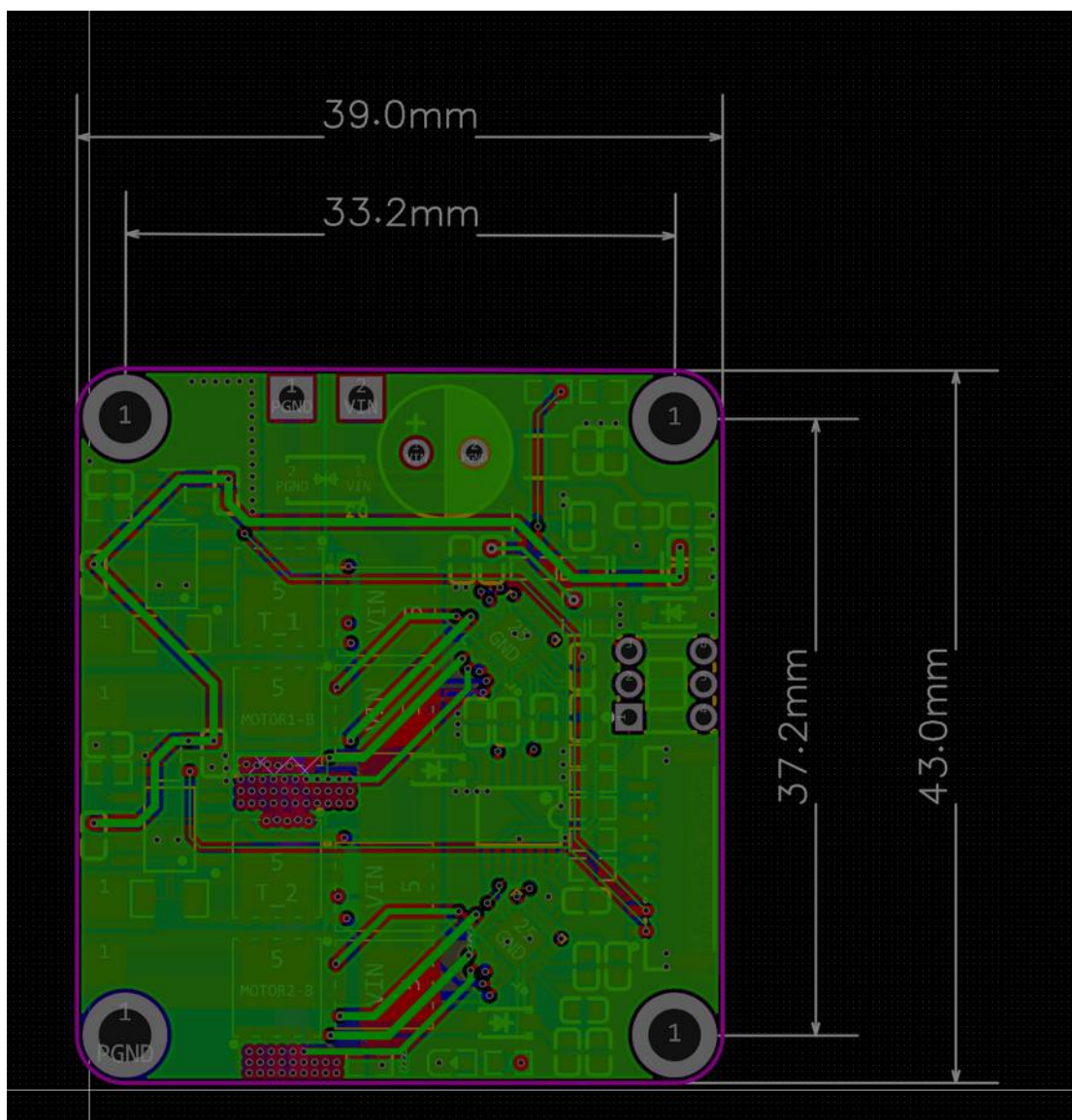
图 A-29 观察板分电板-原理图

二合一有刷电机驱动板



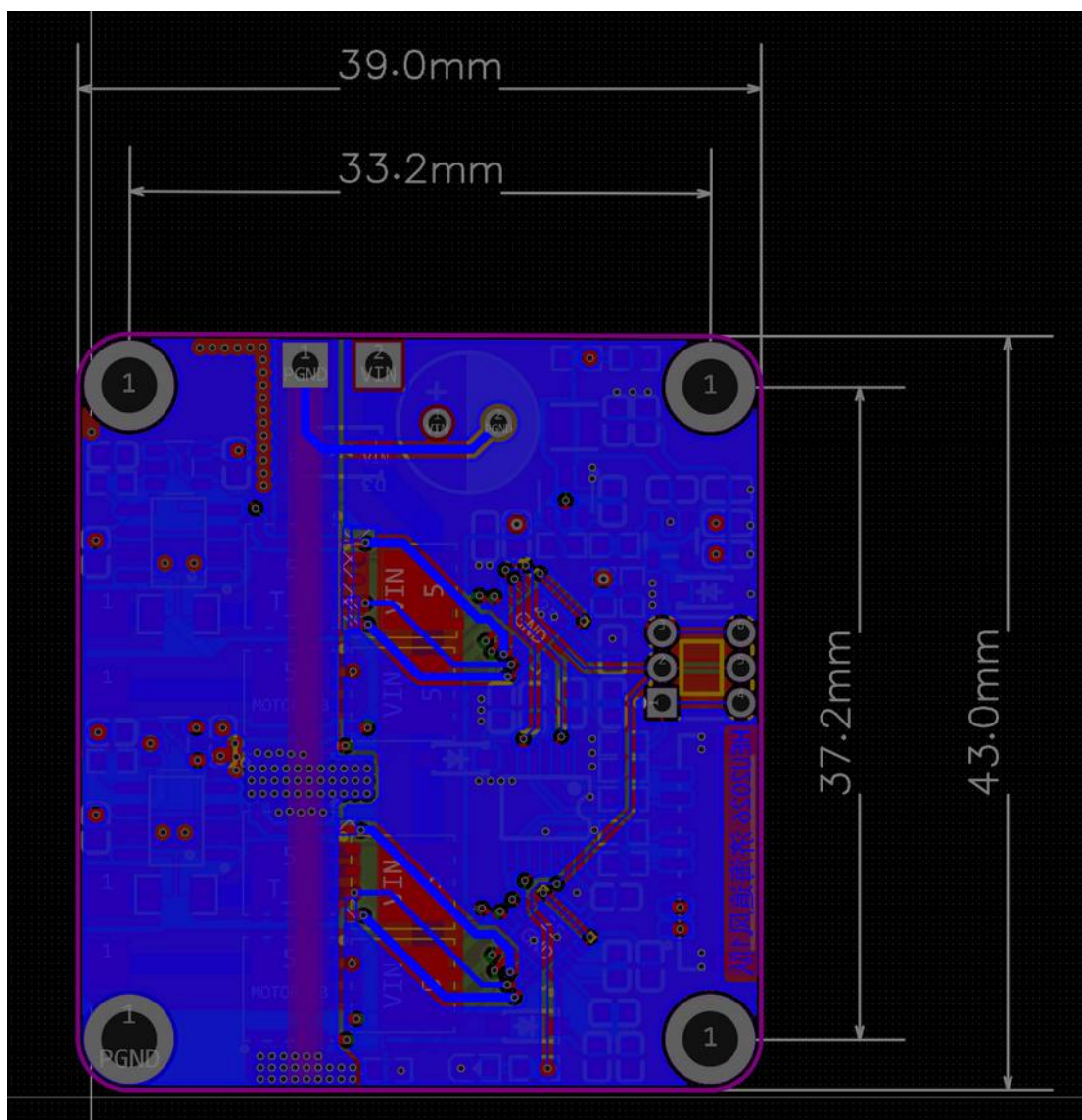
附图 A-30 二合一有刷电机驱动板-PCB 内层 1

图 A-30 二合一有刷电机驱动板-PCB 内层 1



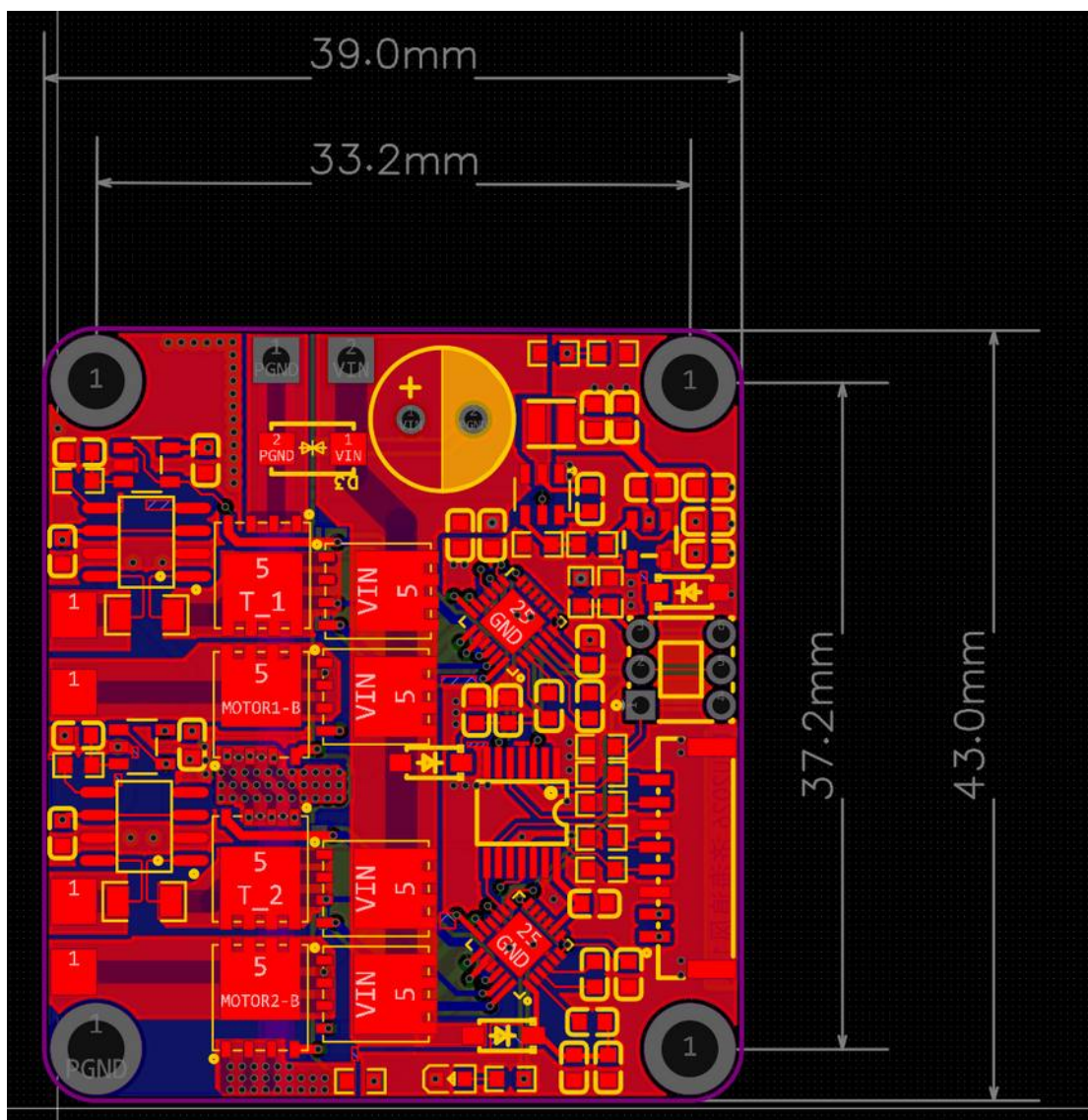
附图 A-31 二合一有刷电机驱动板-PCB 内层 2

图 A-31 二合一有刷电机驱动板-PCB 内层 2



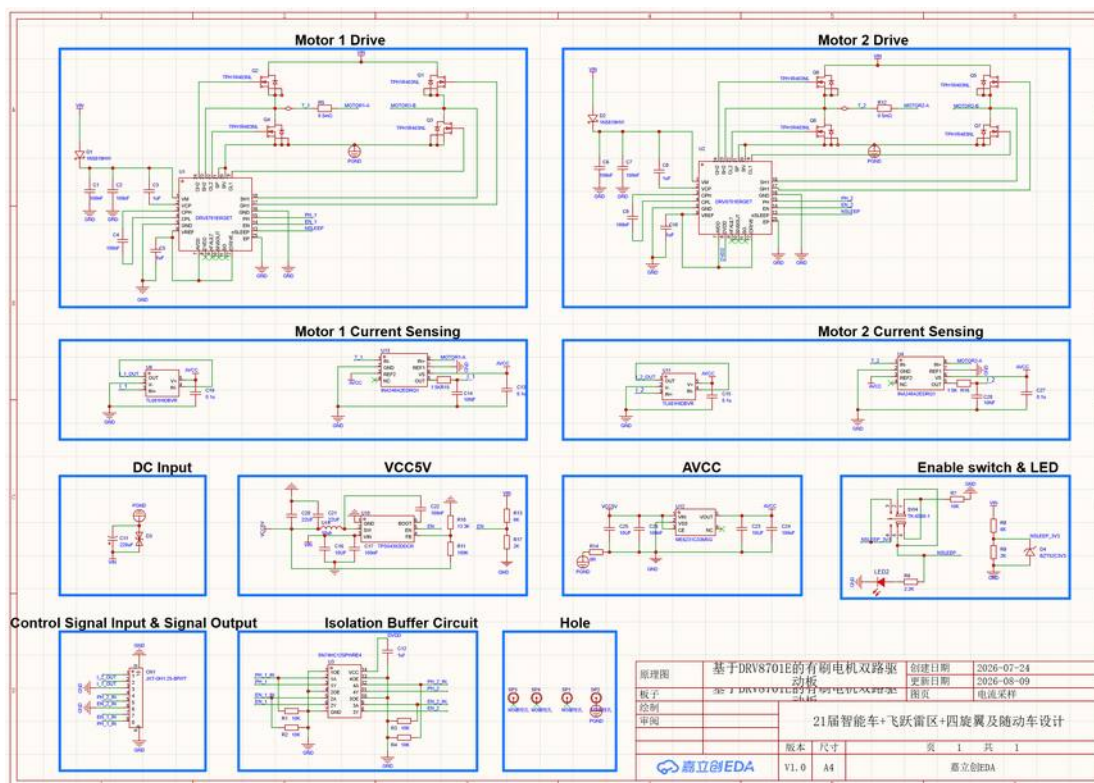
附图 A-32 二合一有刷电机驱动板-PCB 底层

图 A-32 二合一有刷电机驱动板-PCB 底层



附图 A-33 二合一有刷电机驱动板-PCB 顶层

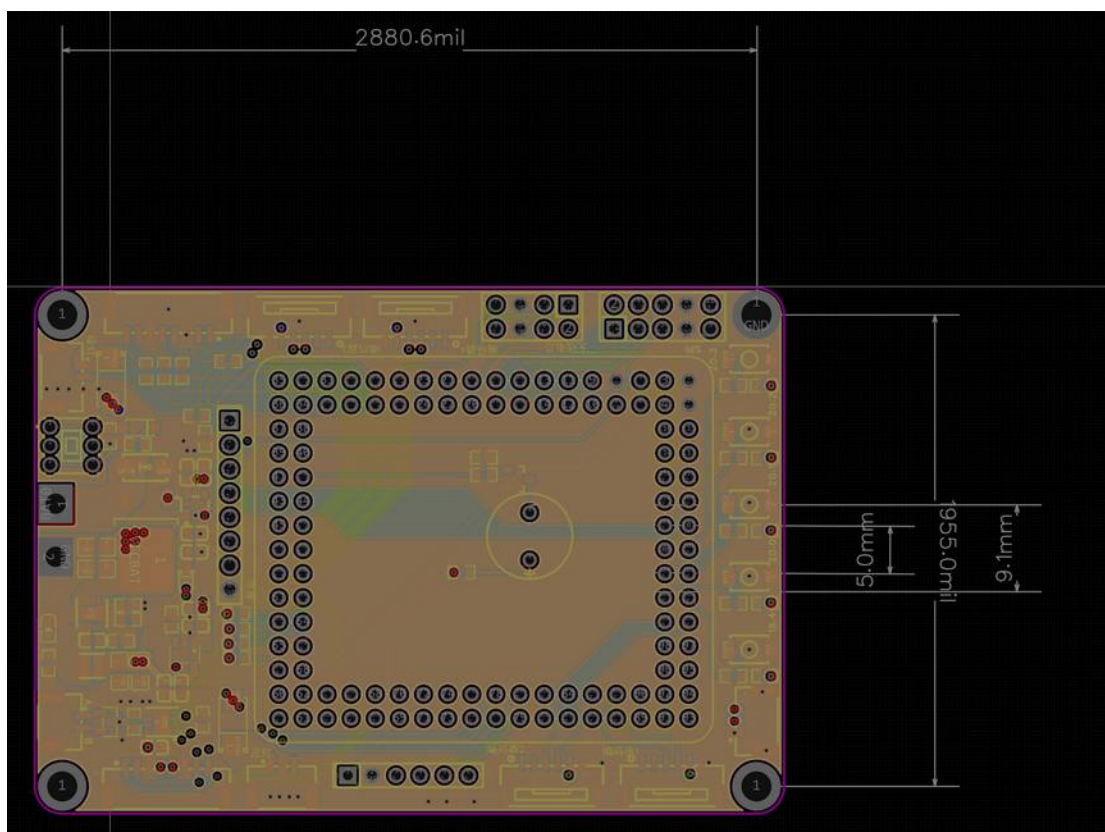
图 A-33 二合一有刷电机驱动板-PCB 顶层



附图 A-34 二合一有刷电机驱动板-原理图

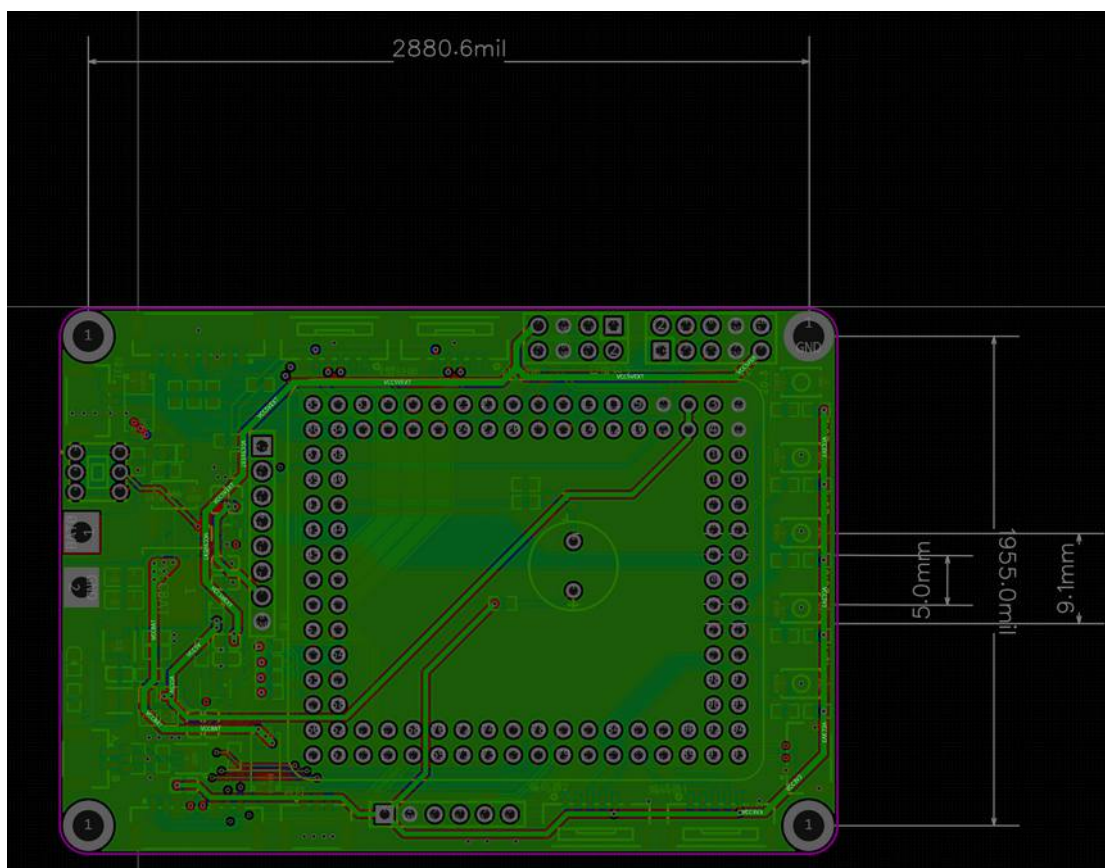
图 A-34 二合一有刷电机驱动板-原理图

底盘主控板



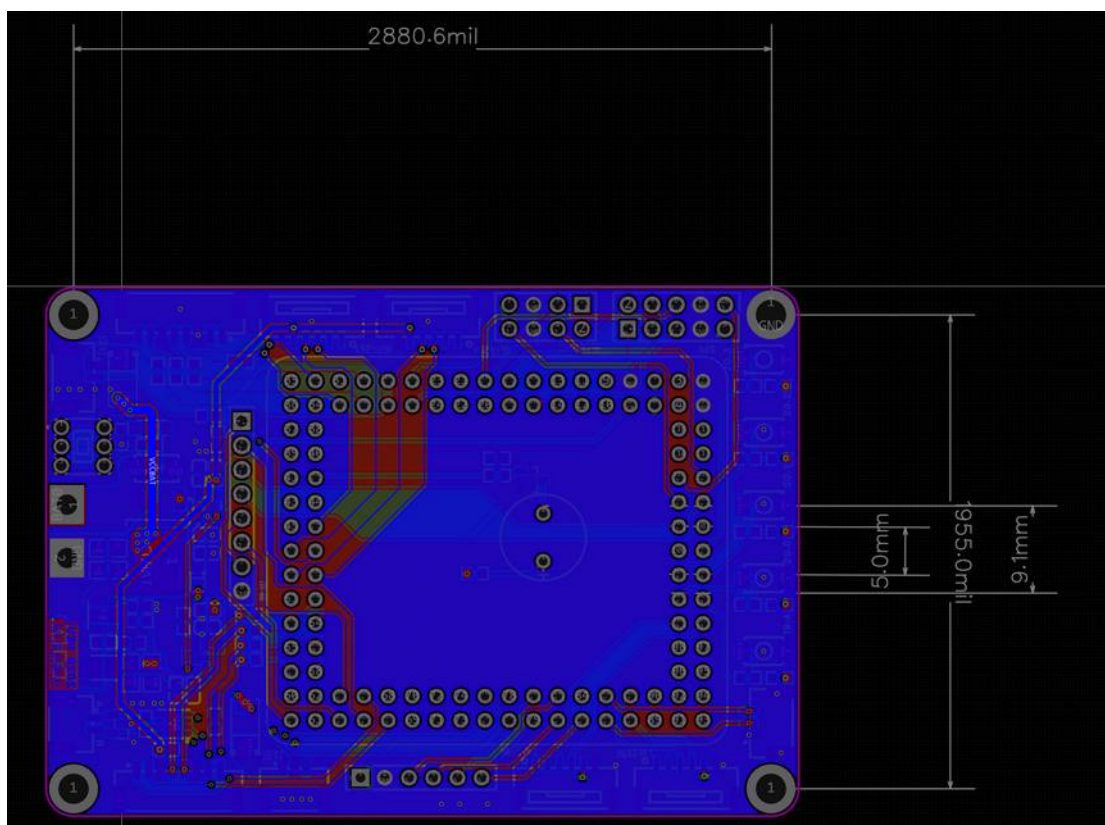
附图 A-35 底盘主控板-PCB 内层 1

图 A-35 底盘主控板-PCB 内层 1



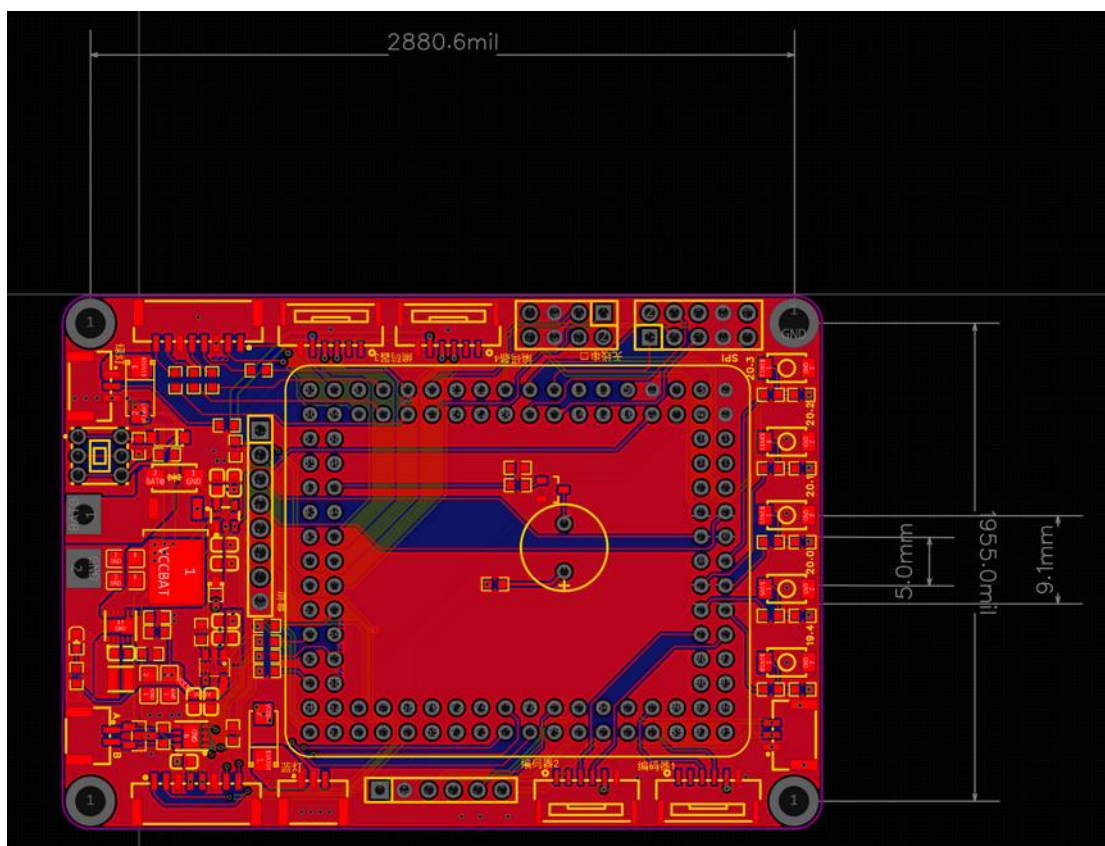
附图 A-36 底盘主控板-PCB 内层 2

图 A-36 底盘主控板-PCB 内层 2



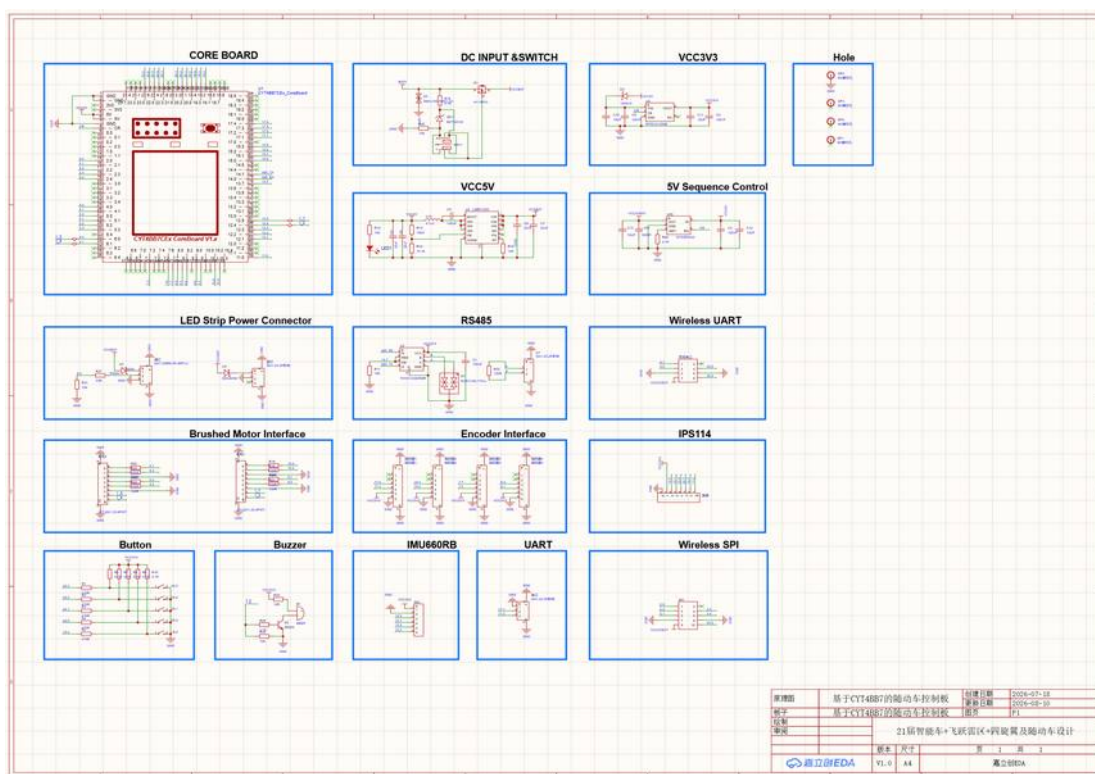
附图 A-37 底盘主控板-PCB 底层

图 A-37 底盘主控板-PCB 底层



附图 A-38 底盘主控板-PCB 顶层

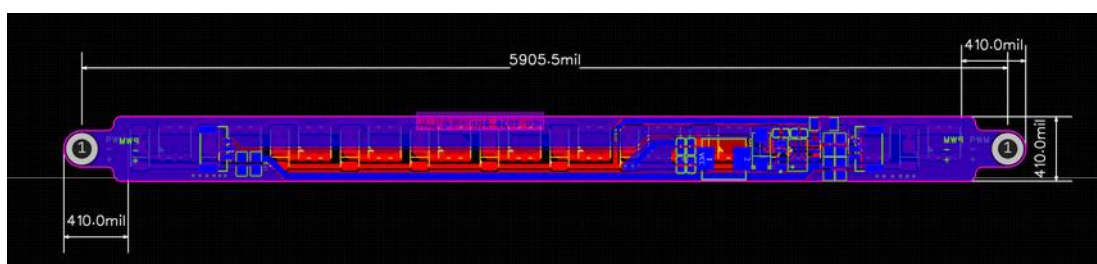
图 A-38 底盘主控板-PCB 顶层



附图 A-39 底盘主控板-原理图

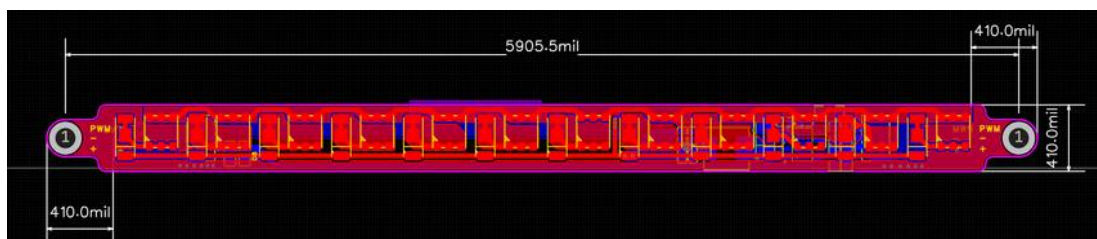
图 A-39 底盘主控板-原理图

灯板



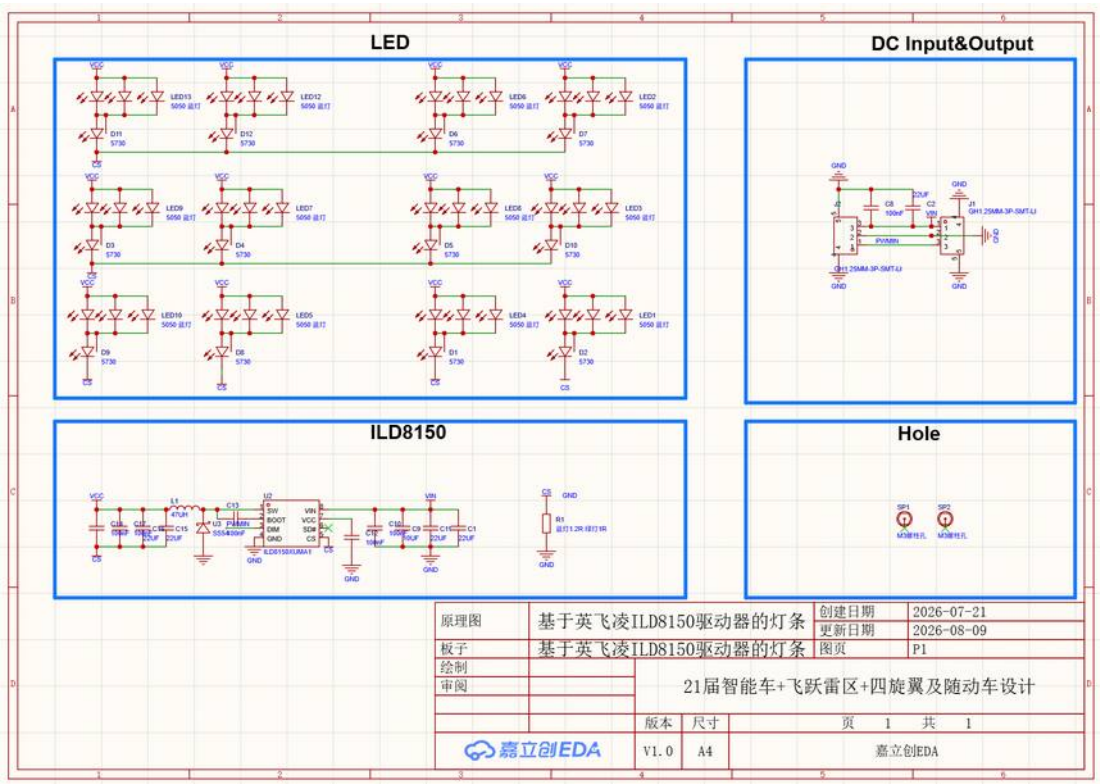
附图 A-40 灯板-PCB 底层

图 A-40 灯板-PCB 底层



附图 A-41 灯板-PCB 顶层

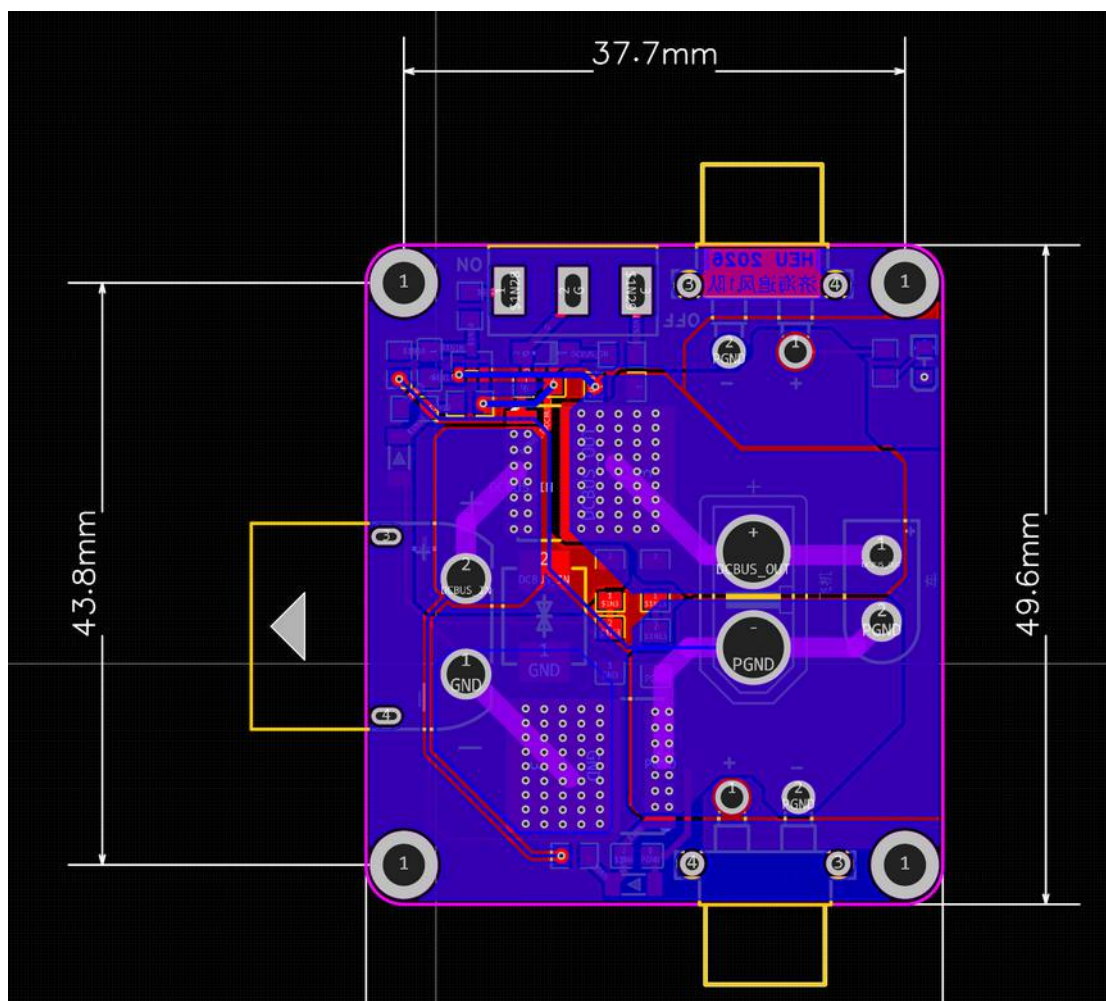
图 A-41 灯板-PCB 顶层



附图 A-42 灯板-原理图

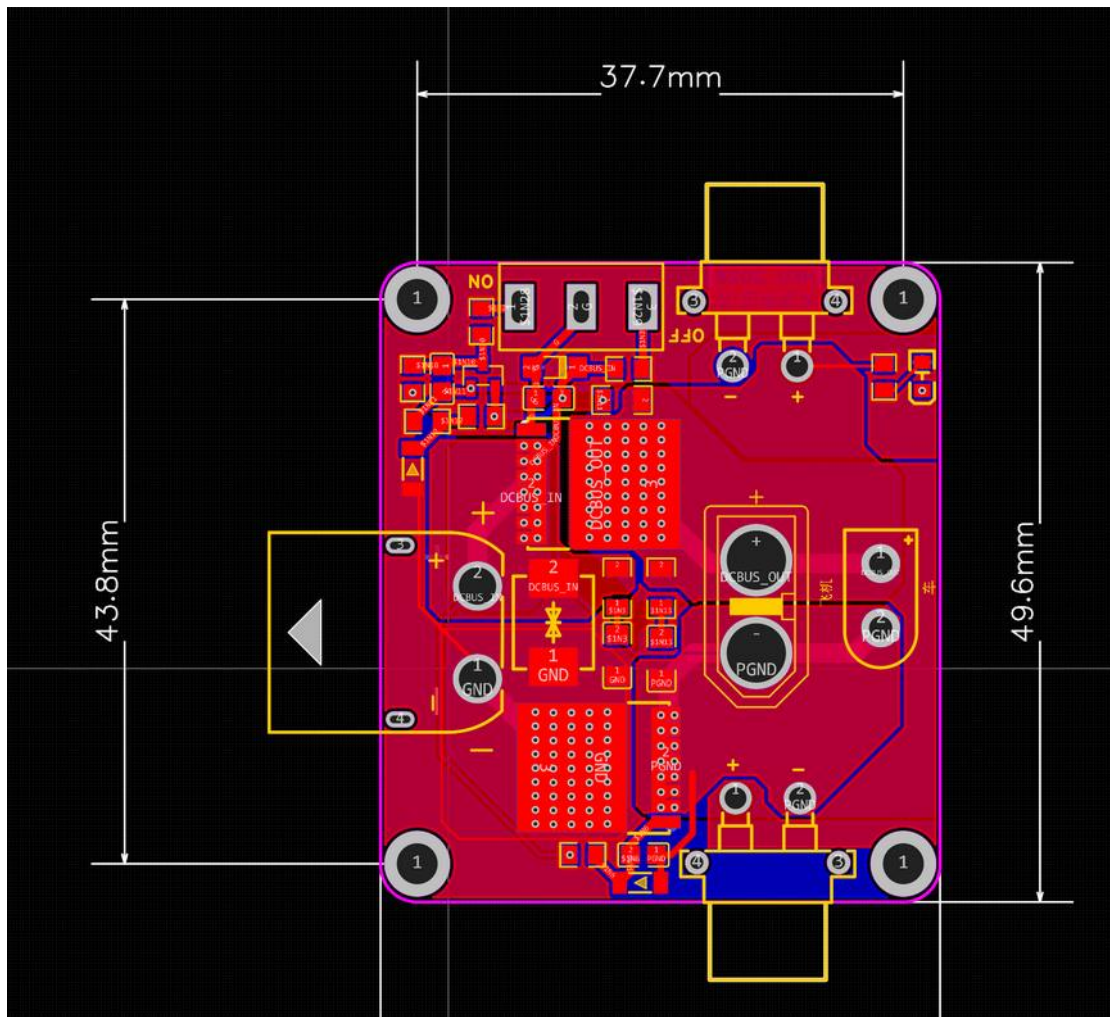
图 A-42 灯板-原理图

空地系统分电板



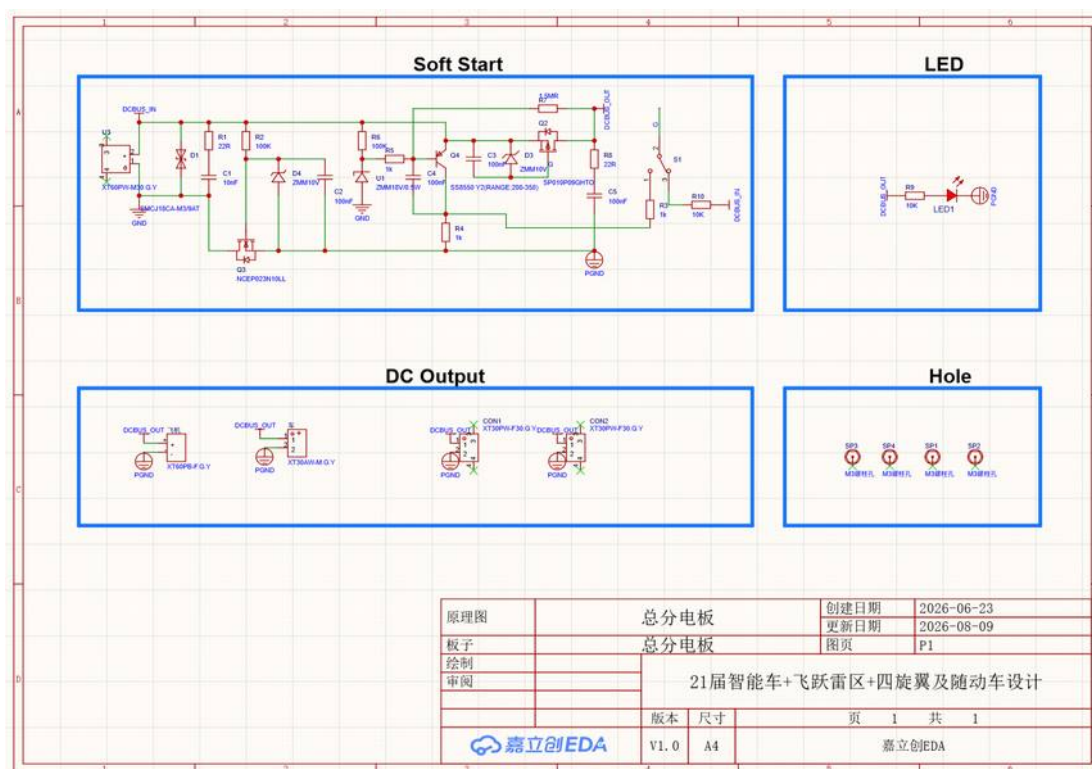
附图 A-43 空地系统分电板-PCB 底层

图 A-43 空地系统分电板-PCB 底层



附图 A-44 空地系统分电板-PCB 顶层

图 A-44 空地系统分电板-PCB 顶层



附图 A-45 空地系统分电板-原理图

图 A-45 空地系统分电板-原理图

附录 B 核心算法子程序

Purpose: 为技术报告提供可复查的活动源码函数片段。

Authority: 片段由符号提取器从活动 C 源码逐字提取。

Sources: 固件证据基线 16d670d2b7141bfc37bc3d2403d695ffdda1a428 的活动源码。

Not owned: 不替代完整源码附件，不证明构建或硬件运行。

Evidence status: CODE-CONFIRMED。

Core1 三来源目标连续性

来源： core1_cyt4bb7/project/code/planning/beaconTargetContinuity.c ， 符号： beaconTargetContinuityStep ， 片段 SHA-256： 21498d937fa1fe2362256a7fb2e764ea068a015d7bbecca5dbe952988879ccd1。

代码 (c)

```
void beaconTargetContinuityStep(beaconTargetContinuity *continuity,
    const beaconSourceObservationView *views,
    uint8_t viewCount,
    uint32_t tickMs,
    uint16_t edgeHoldMs,
    uint16_t searchTimeoutMs,
    uint8_t searchEnabled,
    uint8_t targetAdvanceAllowed,
    uint8_t searchAdvanceAllowed,
    beaconTargetContinuityOutput *output)
{
    beaconControlCandidate best;
    float referenceFwd = 0.0f;
    float referenceRight = 0.0f;
    uint32_t clockElapsedMs;
    uint8_t haveBest = 0u;
    uint8_t referenceValid = 1u;
    uint8_t i;
    if ((continuity == 0) || (views == 0) || (output == 0)) return;
    memset(output, 0, sizeof(*output));
    clockElapsedMs = updateClock(continuity, tickMs);
    (void) consumeBorderRejection(continuity, views, viewCount);
    for (i = 0u; i < viewCount; i++)
    {
        uint16_t j;
        if ((views[i].fresh == 0u) || (views[i].tracks == 0)) continue;
        for (j = 0u; j < views[i].count; j++)
        {
            beaconControlCandidate qualified;
            if (convertCandidate(&views[i], &views[i].tracks[j],
                &qualified) == 0u) continue;
            if (output->qualifiedCount < 255u) output->qualifiedCount++;
        }
    }
}
```

```

    }
}
if (continuity->last.valid != 0u)
{
    referenceValid = buildScoringReference(views, viewCount, continuity,
        &referenceFwd, &referenceRight);
}
if (targetAdvanceAllowed == 0u)
{
    if (searchAdvanceAllowed != 0u)
    {
        uint8_t edgeHoldExpired = (uint8_t) ((continuity->last.valid == 0u) ||
            (edgeHoldMs == 0u) ||
            ((uint32_t) (tickMs - continuity->lastTickMs) > edgeHoldMs));
        if (edgeHoldExpired != 0u)
        {
            continuity->edgeHoldActive = 0u;
        }
        advanceSearch(continuity, tickMs, searchTimeoutMs, searchEnabled,
            edgeHoldExpired);
    }
    else
    {
        pauseTargetTimer(continuity, clockElapsedMs);
        pauseSearchTimer(continuity, clockElapsedMs);
    }
    output->searchActive = (uint8_t) (searchEnabled &&
        continuity->searchRunning &&
        !continuity->searchTimedOut);
    output->searchDirection = (continuity->searchDirection == 0) ? 1 :
        continuity->searchDirection;
    return;
}
if (referenceValid == 0u)
{
    continuity->last.valid = 0u;
    continuity->trendValid = 0u;
}
haveBest = selectExactCurrentCandidate(continuity, views, viewCount, &best);
if (haveBest == 0u)
    haveBest = selectBestCandidate(continuity, views, viewCount, &best);
if (haveBest != 0u)
{
    continuity->searchRunning = 0u;
    continuity->searchTimedOut = 0u;
    continuity->trendValid = continuity->last.valid;
    continuity->last = best;
    continuity->lastTickMs = tickMs;
    continuity->edgeHoldActive = 0u;
    continuity->searchDirection = (best.errRight < 0.0f) ? -1 : 1;
    output->candidate = best;
}
else if ((continuity->last.valid != 0u) &&
    ((uint32_t) (tickMs - continuity->lastTickMs) <= edgeHoldMs) &&
    (edgeHoldMs != 0u) &&
    (findCurrentView(views, viewCount, &continuity->last) != 0))
{
    uint32_t elapsedMs = (uint32_t) (tickMs - continuity->lastTickMs);
    float intentScale = edgeHoldIntentScale(elapsedMs, edgeHoldMs);
    continuity->searchRunning = 0u;
    continuity->edgeHoldActive = 1u;
    if (buildEdgeHoldCandidate(views, viewCount, &continuity->last,
        intentScale, &output->candidate) == 0u)
    {
        memset(&output->candidate, 0, sizeof(output->candidate));
        continuity->edgeHoldActive = 0u;
    }
}

```

```

    }
    else
    {
        continuity->edgeHoldActive = 0u;
        advanceSearch(continuity, tickMs, searchTimeoutMs, searchEnabled, 1u);
        output->searchActive = (uint8_t) (searchEnabled &&
            continuity->searchRunning && !continuity->searchTimedOut);
    }
    output->searchDirection = (continuity->searchDirection == 0) ? 1 :
        continuity->searchDirection;
}

```

Core0 ToF 定高与故障降级

来 源 ： core0_cyt4bb7/project/code/flight/altitudeHold.c ， 符 号 ：
altitudeHoldUpdate ， 片 段 SHA-256 ：
bd03d727ecb47eea89724440e04ed69e2d65ace7396034d57da42d3b317238b4。

代码 (c)

```

void altitudeHoldUpdate(uint32_t nowMs)
{
    float heightMm = 0.0f;
    float velFiltered = 0.0f;
    uint8_t fresh;

    /* facade 完成取帧/姿态补偿/范围/门控/速度低通；本层只取结果。 */
    altitudeToFUpdate();
    fresh = tofGetHeight(&heightMm, &velFiltered);
    altitudeHold.distanceMm = (uint16_t) heightMm; /* 遥测/调试用，始终更新 */
    altitudeHold.seq++;

    /* 安全状态门控：未发车 / 姿态保护停机时不输出，下次发车从当前高度起算。 */
    {
        flightSafetySnapshotStruct safety;
        modeFlightGetSnapshot(&safety);
        if (!holdControlEnabled || safety.state != SAFETY_STATE_CLOSED_LOOP)
        {
            altitudeContext.failsafeStage = ALTITUDE_FAILSAFE_NORMAL;
            altitudeContext.failsafeRequestDisarm = 0u;
            altitudeContext.faultAgeMs = 0u;
            clearRuntimeOutput();
            return;
        }
    }

    /* 冻结期（无新鲜健康帧）：保持上一拍 altitudeOffsetPct，速度+高度均沿用 last-valid，
    * 不推进 PID（积分天然冻结），避免油门偏移骤降致高度剧变。 */
    if (!fresh)
    {
        uint32_t dtMs;
        float lowerOffset;
        if (altitudeTofsHealthy() != 0u) return;
        if (altitudeContext.failsafeStage == ALTITUDE_FAILSAFE_NORMAL)
        {
            altitudeContext.failsafeStage = ALTITUDE_FAILSAFE_HOLD;
            altitudeContext.faultStartMs = nowMs;
            altitudeContext.failsafeLastMs = nowMs;
        }
    }
}

```



```

    uint32_t age = (uint32_t)(nowMs - altitudeContext.faultStartMs);
    altitudeContext.faultAgeMs = (age > 0xFFFFu) ? 0xFFFFu : (uint16_t)age;
}
if ((altitudeContext.failsafeStage == ALTITUDE_FAILSAFE_HOLD) &&
    ((uint32_t)(nowMs - altitudeContext.faultStartMs) >= ALTITUDE_FAILSAFE_HOLD_MS))
{
    altitudeContext.failsafeStage = ALTITUDE_FAILSAFE_RAMP;
    altitudeContext.failsafeLastMs = nowMs;
}
if (altitudeContext.failsafeStage != ALTITUDE_FAILSAFE_RAMP) return;

dtMs = (uint32_t)(nowMs - altitudeContext.failsafeLastMs);
if (dtMs > ALTITUDE_FAILSAFE_MAX_STEP_MS) dtMs = ALTITUDE_FAILSAFE_MAX_STEP_MS;
altitudeContext.failsafeLastMs = nowMs;
lowerOffset = altitudeConfig.throttleMinPct - altitudeConfig.baseThrottlePct;
altitudeHold.outputPct -= ALTITUDE_FAILSAFE_RAMP_PCT_PER_S * ((float)dtMs / 1000.0f);
if (altitudeHold.outputPct <= lowerOffset)
{
    altitudeHold.outputPct = lowerOffset;
    altitudeContext.failsafeRequestDisarm = 1u;
}
pidFlightControllerSetAltitudeOffset(altitudeHold.outputPct);
return;
}

if (altitudeContext.failsafeStage != ALTITUDE_FAILSAFE_NORMAL)
{
    altitudeContext.failsafeStage = ALTITUDE_FAILSAFE_NORMAL;
    altitudeContext.failsafeRequestDisarm = 0u;
    altitudeContext.faultAgeMs = 0u;
    resetPids(heightMm);
    wasInClosedLoop = 1u;
    outerDivCount = 0u;
}

if (!wasInClosedLoop)
{
    resetPids(currentTarget);
    wasInClosedLoop = 1u;
    outerDivCount = 0u;
}

/* 外环分频：每 ALTITUDE_HOLD_OUTER_DIV 拍跑一次位置环。 */
outerDivCount++;
if (outerDivCount >= ALTITUDE_HOLD_OUTER_DIV)
{
    outerDivCount = 0u;
    altPosPid.expectedValue = currentTarget;
    pidProcess(&altPosPid, heightMm);
    velTarget = clampf(altPosPid.controlValue,
        -altitudeConfig.velCmdLimit, altitudeConfig.velCmdLimit);
}

/* 内环每拍 */
altVelPid.expectedValue = velTarget;
pidProcess(&altVelPid, velFiltered);
altitudeHold.outputPct = clampf(altVelPid.controlValue + altitudeConfig.velFeedforward * velTarget,
    -altitudeConfig.offsetLimitPct, altitudeConfig.offsetLimitPct);

pidFlightControllerSetAltitudeOffset(altitudeHold.outputPct);
}

```

Core2 麦轮逆运动学

来源：core2_cyt4bb7/project/code/control/chassisMecanum.c，符号：
chassisMecanumInverse，片段SHA-256：
87c2907f6d10626ee5a366344b7ccad9ffddc8fe04357634d6af6034697206e3。

代码 (c)

```
wheelSpeedStruct chassisMecanumInverse(const chassisMecanumConfigStruct* config, chassisSpeedStruct speed)
{
    wheelSpeedStruct wheelSpeed = {0.0f, 0.0f, 0.0f, 0.0f};
    float rotateFactor;

    if (!config) return wheelSpeed;
    if (config->wheelLinearToPulse <= 0.0f) return wheelSpeed;

    rotateFactor = getRotateFactor(config);

    wheelSpeed.fl = (speed.vx - speed.vy - rotateFactor * speed.wz) * config->wheelLinearToPulse;
    wheelSpeed.fr = (speed.vx + speed.vy + rotateFactor * speed.wz) * config->wheelLinearToPulse;
    wheelSpeed.bl = (speed.vx + speed.vy - rotateFactor * speed.wz) * config->wheelLinearToPulse;
    wheelSpeed.br = (speed.vx - speed.vy + rotateFactor * speed.wz) * config->wheelLinearToPulse;

    if (config->maxWheelPulseSpeed > 0.0f)
    {
        wheelSpeed = chassisMecanumNormalize(wheelSpeed, config->maxWheelPulseSpeed);
    }

    return wheelSpeed;
}
```

Core2 麦轮正运动学

来源：core2_cyt4bb7/project/code/control/chassisMecanum.c，符号：
chassisMecanumForward，片段SHA-256：
edb07225de8a6be0a46dc00f6a2bed2f2055ebfcfc5de9496b30f6aa59bebfaa。

代码 (c)

```
chassisSpeedStruct chassisMecanumForward(const chassisMecanumConfigStruct* config, wheelSpeedStruct speed)
{
    chassisSpeedStruct chassisSpeed = {0.0f, 0.0f, 0.0f};
    float rotateFactor;
    float speedFactor;

    if (!config) return chassisSpeed;

    rotateFactor = getRotateFactor(config);
    if (config->wheelPulseToLinear <= 0.0f) return chassisSpeed;
    if (rotateFactor <= 0.0f) return chassisSpeed;

    speedFactor = 0.25f * config->wheelPulseToLinear;

    chassisSpeed.vx = (speed.fl + speed.fr + speed.bl + speed.br) * speedFactor;
    chassisSpeed.vy = (-speed.fl + speed.fr + speed.bl - speed.br) * speedFactor;
```

```

chassisSpeed.wz = (-speed.fl + speed.fr - speed.bl + speed.br) * speedFactor / rotateFactor;

return chassisSpeed;
}

```

Core1 运动计划整形与最终计划填充

来源：core1_cyt4bb7/project/code/planning/beaconNavigator.c，符号：
fillPlan，片 段 SHA-256：
29f4e9cc19849834c30e8c1d3701c4d269184a0d62d5edffb1efd025d248ffab。

代码 (c)

```

static void fillPlan(planToLinkStruct *out)
{
    memset(out, 0, sizeof(*out));
    out->magic = CORE1_PLAN_READY_MAGIC;
    out->state = nav.state;
    out->valid = nav.valid;
    out->beaconCount = nav.beaconCount;
    out->targetX = nav.controlX;
    out->targetY = nav.controlY;
    out->targetSize = nav.targetSize;
    out->errX = nav.errX;
    out->errY = nav.errY;
    out->vx = nav.vx;
    out->vy = nav.vy;
    out->wz = (nav.state == (uint8_t) BEACON_NAV_SEARCH) ? nav.wz : 0.0f;
}

```

Core0 水平速度估计更新

来源：core0_cyt4bb7/project/code/sensor/velocityEstimator.c，符号：
velocityEstimatorUpdate，片 段 SHA-256：
dc5ac7b35dccddca1ec381689ca2f770773e1ba66777a1d153bf9dec1eb70506。

代码 (c)

```

void velocityEstimatorUpdate(uint8_t allowFrozenTofHeight)
{
    opticalFlowSampleStruct sample;
    float heightMm = 0.0f;
    float verticalVelocityMmS = 0.0f;
    float heightM;
    uint8_t tofFresh = tofGetHeight(&heightMm, &verticalVelocityMmS);
    uint8_t tofHealthy = altitudeTofIsHealthy();
    uint32_t primask;

    (void) verticalVelocityMmS; /* 垂直速度本阶段不入水平融合 */
    heightM = heightMm * 0.001f;

    primask = __get_PRIMASK();
    __disable_irq();
    velocityEstimatorState.heightM = heightM;
}

```

```

velocityEstimatorState.toffFresh = toffFresh;
__set_PRIMASK(primask);

if (!opticalFlowPoll(&sample))
{
    return;
}
/* 收到任意完整帧 → 清链路帧年龄（含无效帧，表示链路活）。
* 有效观测年龄 validAgeMs 仅在通过全部门控 + 校正后才清零（见下）。
* 连续被拒计数：每收一帧先 +1（饱和不回卷），成功校正时在提交块归零。*/
primask = __get_PRIMASK();
__disable_irq();
velocityEstimatorState.frameAgeMs = 0u;
velocityEstimatorState.lastRawFlowX = (float) sample.flowX;
velocityEstimatorState.lastRawFlowY = (float) sample.flowY;
if (velocityEstimatorState.consecutiveInvalidFrames < 0xFFFFu)
{
    velocityEstimatorState.consecutiveInvalidFrames++;
}
__set_PRIMASK(primask);

/* 帧无效 / dt 越界 / 无可用高度 / 高度越界 → 本帧不校正、不刷新有效年龄。
* 已进入 CRUISE 后的 ToF HOLD 允许使用 facade 冻结的最后可信高度维持短窗光流尺度；
* RAMP 及其它调用场景仍要求实时健康，避免长期陈旧高度掩盖安全失效。*/
if (!sample.valid ||
    sample.dtUs < VELOCITY_ESTIMATOR_FLOW_DT_MIN_US ||
    sample.dtUs > VELOCITY_ESTIMATOR_FLOW_DT_MAX_US ||
    (!tofHealthy && allowFrozenToFHeight == 0u) ||
    heightM < VELOCITY_ESTIMATOR_HEIGHT_MIN_M ||
    heightM > VELOCITY_ESTIMATOR_HEIGHT_MAX_M)
{
    return;
}

{
    float q[4];
    float gyroX;
    float gyroY;
    float gyroZ;
    float physicalPitch;
    float physicalRoll;
    float invDtS;
    float flowAngularRateX;
    float flowAngularRateY;
    float flowBodyX;
    float flowBodyY;
    float flowSensorX;
    float flowSensorY;
    float flowLocal[VELOCITY_ESTIMATOR_AXES];
    float measurementVariance;
    float snapVel[VELOCITY_ESTIMATOR_AXES];
    float snapBias[VELOCITY_ESTIMATOR_AXES];
    float snapP00[VELOCITY_ESTIMATOR_AXES];
    float snapP01[VELOCITY_ESTIMATOR_AXES];
    float snapP11[VELOCITY_ESTIMATOR_AXES];
    float delayedVel[VELOCITY_ESTIMATOR_AXES];
    float innovation[VELOCITY_ESTIMATOR_AXES];
    velocityEstimatorKalmanResultStruct result[VELOCITY_ESTIMATOR_AXES];
    uint16_t delaySlot;
    uint32_t snapPredictGeneration;
    uint8_t updateAccepted;
    uint8_t a;

    velocityEstimatorSnapshotImu(
        q, &gyroX, &gyroY, &gyroZ, &physicalPitch, &physicalRoll);

/* 大倾角时 v=h×flowRate 近似失效；门限放宽至约 65° 以容忍机动平移大倾角（大倾角帧测速误差增大）。*/

```

```

if (fabsf(physicalPitch) > VELOCITY_ESTIMATOR_TILT_LIMIT_RAD ||
    fabsf(physicalRoll) > VELOCITY_ESTIMATOR_TILT_LIMIT_RAD)
{
    return;
}

invDtS = 1 000 000.0f / (float) sample.dtUs; /* 1/积分时长(秒) */
flowAngularRateX = VELOCITY_ESTIMATOR_FLOW_SCALE_X_RAD_PER_COUNT
    * (float) sample.flowX * invDtS;
flowAngularRateY = VELOCITY_ESTIMATOR_FLOW_SCALE_Y_RAD_PER_COUNT
    * (float) sample.flowY * invDtS;

/* 倾摆去旋转：flowX 配 pitch(gyroX)、flowY 配 roll(gyroY)，板端"减负"=加回。 */
flowAngularRateX -= VELOCITY_ESTIMATOR_GYRO_COMP_X * gyroX;
flowAngularRateY -= VELOCITY_ESTIMATOR_GYRO_COMP_Y * gyroY;
flowBodyX = heightM * flowAngularRateX;
flowBodyY = heightM * flowAngularRateY;

/* 杠杆臂切向解耦：光流偏离旋转中心 r=(LEVER_X,LEVER_Y)，yaw 转动时光流测到
* 真实切向速度需扣除以还原质心速度。注意 gyroZ=imuData.gz 为 CCW 正，而 FRD
* 右手系(Z 向下)的  $\omega_z = -\text{gyroZ}$ ，故  $v_{\text{tan}} = (\text{gyroZ} \cdot dy, -\text{gyroZ} \cdot dx)$ ，
* 质心  $v = v_{\text{光流}} - v_{\text{tan}} \rightarrow \text{forward} -= \text{gyroZ} \cdot dy$ ； $\text{right} += \text{gyroZ} \cdot dx$ 。确定性补偿，非标定。 */
flowBodyX -= (gyroZ * VELOCITY_ESTIMATOR_FLOW_LEVER_Y_M);
flowBodyY += (gyroZ * VELOCITY_ESTIMATOR_FLOW_LEVER_X_M);

/* 已标定 B(FRD)→S 安装映射：sensorX=-bodyRight，sensorY=-bodyForward。 */
flowSensorX = VELOCITY_ESTIMATOR_BODY_TO_SENSOR_XX * flowBodyX
    + VELOCITY_ESTIMATOR_BODY_TO_SENSOR_XY * flowBodyY;
flowSensorY = VELOCITY_ESTIMATOR_BODY_TO_SENSOR_YX * flowBodyX
    + VELOCITY_ESTIMATOR_BODY_TO_SENSOR_YY * flowBodyY;
velocityEstimatorRotateSensorToLocal(
    q[0], q[1], q[2], q[3],
    flowSensorX, flowSensorY, 0.0f,
    &flowLocal[0], &flowLocal[1]);

/* 物理速度有限性守卫（高度/dt 异常组合可能产生 Inf/NaN）。 */
if (!velocityEstimatorIsFinite(flowLocal[0]) ||
    !velocityEstimatorIsFinite(flowLocal[1]))
{
    return;
}

/* 量测方差 R 随高度平方增长：同样角流噪声在更高处对应更大线速度噪声。
* 首版不按 innovation 调 R（避免真实加速时残差变大而错误压低光流权重）。 */
measurementVariance = VELOCITY_ESTIMATOR_KF_R_BASE
    + VELOCITY_ESTIMATOR_KF_R_HEIGHT_COEFF * heightM * heightM;
measurementVariance = clampf(
    measurementVariance, VELOCITY_ESTIMATOR_KF_P_MIN, VELOCITY_ESTIMATOR_KF_P_MAX);

/* 延迟补偿 + 缩短临界区：
* P2 修复——关中断仅保护"快照"和"提交"两个短瞬间，中间 KF 计算（纯函数、
* 基于局部快照）放临界区外，避免长时间关中断扰动 1ms 飞控环。
* 延迟补偿——光流量测代表约 FLOW_DELAY_MS 前的状态，故 innovation 用"延迟时刻"
* 的融合速度（环形历史）而非当前值，使量测与估计同时刻，消除时间错位。 */

/* 临界区 1（短）：快照当前 v/bias/P + 取延迟时刻融合速度。 */
primask = __get_PRIMASK();
__disable_irq();
snapPredictGeneration = velocityEstimatorState.predictGeneration;
/* histWriteIdx 指向下一个待写槽，最近写入的是 idx-1；延迟 D 帧即 idx-1-D。 */
delaySlot = (uint16_t)((velocityEstimatorState.histWriteIdx
    + VELOCITY_ESTIMATOR_VEL_HIST_LEN
    - 1u - VELOCITY_ESTIMATOR_FLOW_DELAY_MS) % VELOCITY_ESTIMATOR_VEL_HIST_LEN);
for (a = 0u; a < VELOCITY_ESTIMATOR_AXES; a++)
{
    snapVel[a] = velocityEstimatorState.axis[a].fusedVelocity;
    snapBias[a] = velocityEstimatorState.axis[a].bias;
}

```

```

        snapP00[a] = velocityEstimatorState.axis[a].p00;
        snapP01[a] = velocityEstimatorState.axis[a].p01;
        snapP11[a] = velocityEstimatorState.axis[a].p11;
        delayedVel[a] = velocityEstimatorState.axis[a].fusedVelHist[delaySlot];
    }
    __set_PRIMASK(primask);

    /* 临界区外：innovation 对齐延迟时刻估计（延迟补偿核心），KF 更新纯函数计算。
    * 两轴同拍：任一轴坏值（innovation 越限或更新失败）则整帧不提交。*/
    updateAccepted = 1u;
    for (a = 0u; a < VELOCITY_ESTIMATOR_AXES; a++)
    {
        innovation[a] = flowLocal[a] - delayedVel[a];
        if (velocityEstimatorBiasCalibrationCount < VELOCITY_ESTIMATOR_BIAS_CALIBRATION_SAMPLES ||
            fabsf(innovation[a]) > VELOCITY_ESTIMATOR_INNOVATION_LIMIT_MPS ||
            !velocityEstimatorKalmanUpdateAxis(
                snapVel[a], snapBias[a], snapP00[a], snapP01[a], snapP11[a],
                innovation[a], measurementVariance, &result[a]))
        {
            updateAccepted = 0u;
        }
    }

    /* 临界区 2（短）：提交结果 + 调试量。两轴同拍，单轴坏值则整帧不更新。*/
    primask = __get_PRIMASK();
    __disable_irq();
    for (a = 0u; a < VELOCITY_ESTIMATOR_AXES; a++)
    {
        velocityEstimatorState.axis[a].flowVelocity = flowLocal[a];
        velocityEstimatorState.axis[a].innovation = innovation[a];
    }
    if (updateAccepted &&
        velocityEstimatorState.predictGeneration == snapPredictGeneration)
    {
        for (a = 0u; a < VELOCITY_ESTIMATOR_AXES; a++)
        {
            velocityEstimatorState.axis[a].fusedVelocity = result[a].velocity;
            velocityEstimatorState.axis[a].bias = result[a].bias;
            velocityEstimatorState.axis[a].p00 = result[a].p00;
            velocityEstimatorState.axis[a].p01 = result[a].p01;
            velocityEstimatorState.axis[a].p11 = result[a].p11;
        }
        velocityEstimatorState.flowValid = 1u;
        velocityEstimatorState.validAgeMs = 0u; /* 有效观测年龄仅在更新成功时清零 */
        velocityEstimatorState.consecutiveInvalidFrames = 0u; /* 成功校正 → 连续被拒计数归零 */
    }
    __set_PRIMASK(primask);
}
}

```

Core0 飞行任务序列推进

来源：core0_cyt4bb7/project/code/flight/flightSequenceCoordinator.c，符号：
flightSequenceStep，片 段 SHA-256：
38b02fc51581a753ba123b7016af124276e6d1609b9f8d3e15b0d9fd8493765a。

代码（c）

```

void flightSequenceStep(const flightSequenceFactsStruct *facts,
    flightSequenceOutputStruct *output)

```



```

{
    flightSequencePolicyContextStruct policyContext;
    flightSequencePolicyDecisionStruct decision;
    flightSequenceFactsStruct policyFacts;
    flightSequenceStageEnum renderStage;
    uint32_t dtMs;
    uint32_t landingTrigger = 0u;
    uint32_t policyActionMask;
    uint32_t policyTriggerSources = 0u;
    uint8_t policyPrimaryReason = CORE0_FAULT_REASON_NONE;
    uint8_t landingReason = CORE0_FAULT_REASON_NONE;
    uint8_t closedLoop;
    uint8_t mode3Event;
    uint8_t mode2Event;
    uint8_t landingTriggeredThisStep = 0u;
    uint8_t directDisarmActive = 0u;

    if (output == 0) return;
    memset(output, 0, sizeof(*output));
    if (facts == 0) return;

    dtMs = facts->nowMs - sequenceContext.lastNowMs;
    sequenceContext.lastNowMs = facts->nowMs;
    sequenceContext.cycleSeq++;
    policyFacts = *facts;
    policyFacts.dtMs = dtMs;
    policyFacts.coordinatorCycleSeq = sequenceContext.cycleSeq;
    policyFacts.coordinatorTransitionSeq = sequenceContext.transitionSeq;
    policyFacts.coordinatorEpisodeSeq = sequenceContext.episodeSeq;
    closedLoop = (facts->safety.state == SAFETY_STATE_CLOSED_LOOP) ? 1u : 0u;

    if (facts->systemMode != sequenceContext.lastSystemMode)
    {
        sequenceContext.lastSystemMode = facts->systemMode;
        sequenceContext.mode3Armed = 0u;
        sequenceContext.mode2Armed = 0u;
        sequenceContext.mode3LowTiming = 0u;
        sequenceContext.mode2LowTiming = 0u;
        if (sequenceContext.stage == FLIGHT_SEQUENCE_STAGE_DESCENDING)
            sequenceContext.stageEntryAltitudeMm = (float) facts->altitude.distanceMm;
        flightSequencePolicyReset(FLIGHT_SEQUENCE_RESET_MODE);
    }
    if (facts->rejoinEdge != 0u)
    {
        sequenceContext.mode3Armed = 0u;
        sequenceContext.mode2Armed = 0u;
        if (sequenceContext.stage == FLIGHT_SEQUENCE_STAGE_DESCENDING)
            sequenceContext.stageEntryAltitudeMm = (float) facts->altitude.distanceMm;
        flightSequencePolicyReset(FLIGHT_SEQUENCE_RESET_REJOIN);
    }

    if (facts->safety.disarmAcceptedSeq != sequenceContext.lastDisarmAcceptedSeq)
    {
        sequenceContext.lastDisarmAcceptedSeq = facts->safety.disarmAcceptedSeq;
        flightSequenceReset(FLIGHT_SEQUENCE_RESET_DISARM);
    }
    if (facts->safety.protectAcceptedSeq != sequenceContext.lastProtectAcceptedSeq)
    {
        sequenceContext.lastProtectAcceptedSeq = facts->safety.protectAcceptedSeq;
        flightSequenceReset(FLIGHT_SEQUENCE_RESET_PROTECT);
    }
    if (facts->safety.launchAcceptedSeq != sequenceContext.lastLaunchAcceptedSeq)
    {
        sequenceContext.lastLaunchAcceptedSeq = facts->safety.launchAcceptedSeq;
        sequenceContext.episodeSeq++;
        flightSequencePolicyReset(FLIGHT_SEQUENCE_RESET_INIT);
        if (sequenceContext.pendingAutoTakeoff != 0u)

```

```

    {
        sequenceContext.pendingAutoTakeoff = 0u;
        enterStage(FLIGHT_SEQUENCE_STAGE_WAIT,
            (float) facts->altitude.distanceMm);
    }
}

mode3Event = consumeLevelRequest(facts->mode3RequestValid,
    facts->mode3RequestLevel, (uint8_t) (facts->core2LinkLost == 0u),
    &sequenceContext.mode3Armed, &sequenceContext.mode3LowTiming,
    &sequenceContext.mode3LowStartMs, facts->nowMs);
mode2Event = consumeLevelRequest(facts->mode2RequestValid,
    facts->mode2RequestLevel, (uint8_t) (facts->core2LinkLost == 0u),
    &sequenceContext.mode2Armed, &sequenceContext.mode2LowTiming,
    &sequenceContext.mode2LowStartMs, facts->nowMs);
policyFacts.mode3RequestArmed = sequenceContext.mode3Armed;
policyFacts.mode2RequestArmed = sequenceContext.mode2Armed;
policyFacts.coordinatorTransitionSeq = sequenceContext.transitionSeq;
policyFacts.coordinatorEpisodeSeq = sequenceContext.episodeSeq;

memset(&policyContext, 0, sizeof(policyContext));
policyContext.stage = sequenceContext.stage;
policyContext.stageElapsedMs = sequenceContext.stageElapsedMs;
policyContext.dtMs = dtMs;
policyContext.stageEntryAltitudeMm = sequenceContext.stageEntryAltitudeMm;
policyContext.cruiseAltitudeMm = sequenceContext.cruiseAltitudeMm;
policyContext.mode3RequestEvent = (uint8_t) (mode3Event &&
    facts->systemMode == (uint8_t) UART_LINK_SYS_MODE_LORA_FWD);
policyContext.mode2RequestEvent = (uint8_t) (mode2Event &&
    facts->systemMode == (uint8_t) UART_LINK_SYS_MODE_RS485_AUTO);
memset(&decision, 0, sizeof(decision));
flightSequencePolicySelectTriggers(&policyFacts, &policyContext, &decision);
policyActionMask = decision.actionMask & FLIGHT_SEQUENCE_POLICY_ACTION_ALL;
if (policyActionMask != decision.actionMask)
{
    output->safetyClampMask |= FLIGHT_SEQUENCE_CLAMP_POLICY_INVALID;
    policyActionMask = 0u;
}
if (policyActionMask != 0u)
{
    uint8_t triggerMetadataValid = (uint8_t)
        (decision.triggerEventValid == 1u && decision.triggerSources != 0u &&
            (decision.triggerSources & ~FLIGHT_SEQUENCE_TRIGGER_POLICY_ALL) == 0u);
    uint8_t risingAction = (uint8_t)
        ((policyActionMask & ~sequenceContext.policyActionLevelMask) != 0u);
    uint8_t repeatedToken = (uint8_t)
        (sequenceContext.policyTriggerPrimed != 0u &&
            sequenceContext.lastPolicyTriggerEventSeq == decision.triggerEventSeq &&
            sequenceContext.lastPolicyTriggerSources == decision.triggerSources);
    if (triggerMetadataValid && risingAction && !repeatedToken)
    {
        policyTriggerSources = decision.triggerSources;
        policyPrimaryReason = decision.primaryReason;
        sequenceContext.policyTriggerPrimed = 1u;
        sequenceContext.lastPolicyTriggerEventSeq = decision.triggerEventSeq;
        sequenceContext.lastPolicyTriggerSources = decision.triggerSources;
    }
    else
    {
        if (!triggerMetadataValid)
            output->safetyClampMask |= FLIGHT_SEQUENCE_CLAMP_POLICY_INVALID;
        policyActionMask = 0u;
    }
}
}
sequenceContext.policyActionLevelMask = decision.actionMask &
    FLIGHT_SEQUENCE_POLICY_ACTION_ALL;

```

```

if (facts->k5StandbyEdge != 0u)
{
    output->requestDisarm = 1u;
    directDisarmActive = 1u;
    output->triggerSources |= FLIGHT_SEQUENCE_TRIGGER_K5_STANDBY;
    sequenceContext.pendingAutoTakeoff = 0u;
}
if (((policyActionMask & FLIGHT_SEQUENCE_POLICY_ACTION_ARM) != 0u) &&
    (directDisarmActive == 0u))
{
    output->triggerSources |= policyTriggerSources;
    sequenceContext.latchedTriggerSources |= policyTriggerSources;
    if (policyPrimaryReason != CORE0_FAULT_REASON_NONE)
        sequenceContext.primaryReason = policyPrimaryReason;
    if (facts->faultLatched != 0u) output->acknowledgeFault = 1u;
    else if ((facts->safety.state == SAFETY_STATE_DISARMED ||
        facts->safety.state == SAFETY_STATE_PROTECTED) &&
        (facts->initialized != 0u) && (facts->controlReady != 0u) &&
        (requiredLinkLost(facts) == 0u))
    {
        output->requestLaunch = 1u;
        if ((policyActionMask & FLIGHT_SEQUENCE_POLICY_ACTION_TAKEOFF) != 0u)
            sequenceContext.pendingAutoTakeoff = 1u;
    }
    else
    {
        sequenceContext.pendingAutoTakeoff = 0u;
        output->safetyClampMask |= (requiredLinkLost(facts) != 0u) ?
            FLIGHT_SEQUENCE_CLAMP_REQUIRED_LINK : FLIGHT_SEQUENCE_CLAMP_SAFETY_STATE;
    }
}
if (((policyActionMask & FLIGHT_SEQUENCE_POLICY_ACTION_TAKEOFF) != 0u) &&
    ((policyActionMask & FLIGHT_SEQUENCE_POLICY_ACTION_ARM) == 0u) && closedLoop &&
    (sequenceContext.stage == FLIGHT_SEQUENCE_STAGE_IDLE))
{
    if ((facts->systemPhase == CORE_UX_PHASE_HOME) && (facts->standalone == 0u) &&
        (requiredLinkLost(facts) == 0u))
    {
        output->triggerSources |= policyTriggerSources;
        sequenceContext.latchedTriggerSources |= policyTriggerSources;
        if (policyPrimaryReason != CORE0_FAULT_REASON_NONE)
            sequenceContext.primaryReason = policyPrimaryReason;
        enterStage(FLIGHT_SEQUENCE_STAGE_WAIT, (float) facts->altitude.distanceMm);
    }
}

if (sequenceContext.stageElapsedMs < FLIGHT_SEQUENCE_ELAPSED_CAP_MS)
{
    uint32_t room = FLIGHT_SEQUENCE_ELAPSED_CAP_MS - sequenceContext.stageElapsedMs;
    sequenceContext.stageElapsedMs += (dtMs < room) ? dtMs : room;
}
renderStage = sequenceContext.stage;
renderBaseStage(facts, output, renderStage);

policyContext.stage = renderStage;
policyContext.stageElapsedMs = sequenceContext.stageElapsedMs;
policyContext.dtMs = dtMs;
policyContext.stageEntryAltitudeMm = sequenceContext.stageEntryAltitudeMm;
memset(&decision, 0, sizeof(decision));
policyFacts.coordinatorTransitionSeq = sequenceContext.transitionSeq;
policyFacts.coordinatorEpisodeSeq = sequenceContext.episodeSeq;
flightSequencePolicyApplyStage(&policyFacts, &policyContext, &decision);
mergePolicy(&decision, renderStage, output);
if (output->rejectedOverrideMask != 0u)
    output->safetyClampMask |= FLIGHT_SEQUENCE_CLAMP_POLICY_INVALID;

if (renderStage == FLIGHT_SEQUENCE_STAGE_WAIT &&

```

```

(((decision.overrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION) != 0u) ?
(((output->rejectedOverrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION) == 0u) &&
decision.stageTransitionReady) :
(sequenceContext.stageElapsedMs >= TAKEOFF_WAIT_MS)))
enterStage(FLIGHT_SEQUENCE_STAGE_RAMP, sequenceContext.stageEntryAltitudeMm);
else if (renderStage == FLIGHT_SEQUENCE_STAGE_RAMP &&
(((decision.overrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION) != 0u) ?
(((output->rejectedOverrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION) == 0u) &&
decision.stageTransitionReady) :
(sequenceContext.stageElapsedMs >= TAKEOFF_RAMP_MS)))
enterStage(FLIGHT_SEQUENCE_STAGE_CRUISE, sequenceContext.cruiseAltitudeMm);
else if (renderStage == FLIGHT_SEQUENCE_STAGE_DESCENDING &&
(decision.touchdown ||
((decision.overrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION) != 0u)))
{
uint8_t transitionOverrideValid = (uint8_t)
((decision.overrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION) != 0u &&
(output->rejectedOverrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION) == 0u);
uint8_t landReady = (uint8_t) (decision.touchdown ||
(transitionOverrideValid && decision.stageTransitionReady));
if (landReady) enterStage(FLIGHT_SEQUENCE_STAGE_LANDED, output->altitudeTargetMm);
}
else if (renderStage == FLIGHT_SEQUENCE_STAGE_LANDED)
output->requestDisarm = 1u;

if (closedLoop && confirmedLandingLinkLost(facts, closedLoop,
&landingReason, &landingTrigger))
{
if ((sequenceContext.stage != FLIGHT_SEQUENCE_STAGE_DESCENDING) &&
(sequenceContext.stage != FLIGHT_SEQUENCE_STAGE_LANDED))
{
sequenceContext.primaryReason = landingReason;
sequenceContext.latchedTriggerSources |= landingTrigger;
enterStage(FLIGHT_SEQUENCE_STAGE_DESCENDING,
(float) facts->altitude.distanceMm);
flightSequencePolicyReset(FLIGHT_SEQUENCE_RESET_ABORT);
landingTriggeredThisStep = 1u;
}
}
else if (closedLoop &&
((policyActionMask & FLIGHT_SEQUENCE_POLICY_ACTION_LAND) != 0u) &&
(sequenceContext.stage != FLIGHT_SEQUENCE_STAGE_DESCENDING) &&
(sequenceContext.stage != FLIGHT_SEQUENCE_STAGE_LANDED))
{
landingReason = (policyPrimaryReason != CORE0_FAULT_REASON_NONE) ?
policyPrimaryReason : (policyContext.mode2RequestEvent ?
CORE0_FAULT_REASON_LAND_LORA_MODE2 : CORE0_FAULT_REASON_LAND_LORA_MODE3);
landingTrigger = policyTriggerSources;
sequenceContext.primaryReason = landingReason;
sequenceContext.latchedTriggerSources |= landingTrigger;
enterStage(FLIGHT_SEQUENCE_STAGE_DESCENDING,
(float) facts->altitude.distanceMm);
flightSequencePolicyReset(FLIGHT_SEQUENCE_RESET_ABORT);
landingTriggeredThisStep = 1u;
}
if ((sequenceContext.stage == FLIGHT_SEQUENCE_STAGE_DESCENDING) ||
(sequenceContext.stage == FLIGHT_SEQUENCE_STAGE_LANDED))
{
output->landingActive = 1u;
output->altReached = 0u;
output->safetyClampMask |= FLIGHT_SEQUENCE_CLAMP_LANDING;
output->triggerSources |= landingTrigger;
output->primaryReason = sequenceContext.primaryReason;
if (landingTriggeredThisStep != 0u)
{
/* 兼容既有语义：触发拍立即撤销运动资格，高度/RGB/蜂鸣从下一拍缓降输出接管。 */
output->policyPositionActive = 1u;
}
}

```

```

    }
}
if (output->requestDisarm != 0u) directDisarmActive = 1u;

if ((facts->safety.state != SAFETY_STATE_CLOSED_LOOP) ||
    (facts->safety.valid == 0u) || (facts->paramsValid == 0u) ||
    (facts->initialized == 0u) || (facts->controlReady == 0u) ||
    (directDisarmActive != 0u))
{
    output->altitudeControlEnable = 0u;
    output->idleThrottleMode = 0u;
    output->altReached = 0u;
    output->policyPositionActive = 1u;
    output->safetyClampMask |= FLIGHT_SEQUENCE_CLAMP_SAFETY_STATE;
}
/* 唯一 owner 在目标/门设置后推进 altitudeHold，并以本拍 fresh failsafe 做最终钳制。*/
pidFlightControllerSetIdleThrottleMode(output->idleThrottleMode);
altitudeHoldSetControlEnabled(output->altitudeControlEnable);
altitudeHoldSetTargetMm(output->altitudeTargetMm);
altitudeHoldUpdate(facts->nowMs);
altitudeHoldGetSnapshot(&output->altitude);
if (output->altitude.failsafeStage != ALTITUDE_FAILSAFE_NORMAL)
{
    output->rgbIntent = UART_LINK_RGB_DISCONNECT_WARN;
    output->buzzerPattern = UART_LINK_BUZZER_WARN;
}
if (output->altitude.failsafeStage == ALTITUDE_FAILSAFE_RAMP)
{
    output->altReached = 0u;
    output->policyPositionActive = 1u;
    output->safetyClampMask |= FLIGHT_SEQUENCE_CLAMP_TOF_RAMP;
    output->triggerSources |= FLIGHT_SEQUENCE_TRIGGER_TOF_FAILSAFE;
    if (output->primaryReason == CORE0_FAULT_REASON_NONE)
        output->primaryReason = CORE0_FAULT_REASON_LAND_TOF_FAILSAFE;
}
if (output->altitude.requestDisarm != 0u)
{
    output->requestDisarm = 1u;
    output->altReached = 0u;
    output->policyPositionActive = 1u;
    output->safetyClampMask |= FLIGHT_SEQUENCE_CLAMP_SAFETY_STATE;
}
if ((facts->readyBeepActive != 0u) && (output->landingActive == 0u) &&
    (output->altitude.failsafeStage == ALTITUDE_FAILSAFE_NORMAL) &&
    ((output->buzzerPattern == UART_LINK_BUZZER_SILENT) ||
     (output->buzzerPattern == UART_LINK_BUZZER_3S)))
    output->buzzerPattern = UART_LINK_BUZZER_THREE_BEEPS;

/* 最终水平引导仲裁：Policy 本地 > 普通视觉/MOTION；安全钳制已将 Policy 源置零。*/
if (output->policyPositionActive == 0u)
{
    uint8_t externalAllowed = (uint8_t) (output->allowExternalMotion && facts->motionValid &&
        facts->motionFenceOpen && !facts->standalone &&
        facts->systemPhase == CORE_UX_PHASE_HOME && !output->landingActive &&
        output->altitude.failsafeStage != ALTITUDE_FAILSAFE_RAMP);
    if (externalAllowed)
    {
        motionFeedforwardDiffUpdate(facts->motionVx, facts->motionVy,
            facts->motionSeq, facts->nowMs);
        if (renderStage == FLIGHT_SEQUENCE_STAGE_CRUISE)
        {
            output->bodyVelocityFwdMps = facts->motionVx *
                sequenceParams.motionVelFfVxToFwd;
            output->bodyVelocityRightMps = facts->motionVy *
                FLIGHT_MOTION_VEL_FF_VY_TO_RIGHT_SIGN *
                sequenceParams.motionVelFfVyToRight;
            output->bodyVelocityFwdMps += facts->motionWz *

```

```

        sequenceParams.motionYawOrbitOffsetRightM;
        output->bodyVelocityRightMps += facts->motionWz *
            sequenceParams.motionYawOrbitOffsetFwdM;
    }
    output->pitchFeedforwardRad = motionFeedforwardDiffGetDvX() *
        sequenceParams.motionFfVxToPitch;
    output->rollFeedforwardRad = motionFeedforwardDiffGetDvY() *
        sequenceParams.motionFfVyToRoll;
    output->yawRateRadS = facts->motionWz * sequenceParams.motionFfWzToYaw;
}
else motionFeedforwardDiffReset();

if (output->allowVisionPosition &&
    (renderStage == FLIGHT_SEQUENCE_STAGE_CRUISE) &&
    sequenceParams.visionPosErrEnable && facts->visionFrameValid)
{
    uint8_t visionFinite = (uint8_t) (finiteFloat(facts->visionErrX) &&
        finiteFloat(facts->visionErrY) && finiteFloat(facts->visionYawErr) &&
        facts->visionErrX >= -FLIGHT_SEQUENCE_MAX_VISION_ERROR_PX &&
        facts->visionErrX <= FLIGHT_SEQUENCE_MAX_VISION_ERROR_PX &&
        facts->visionErrY >= -FLIGHT_SEQUENCE_MAX_VISION_ERROR_PX &&
        facts->visionErrY <= FLIGHT_SEQUENCE_MAX_VISION_ERROR_PX &&
        facts->visionYawErr >= -FLIGHT_SEQUENCE_MAX_VISION_YAW_RAD &&
        facts->visionYawErr <= FLIGHT_SEQUENCE_MAX_VISION_YAW_RAD);
    if (facts->visionTargetValid && visionFinite)
    {
        sequenceContext.lastVisionHadTarget = 1u;
        output->visionErrX = facts->visionErrX;
        output->visionErrY = facts->visionErrY;
        output->visionYawErr = facts->visionYawErr;
        output->visionValid = 1u;
    }
    else if (sequenceContext.lastVisionHadTarget && visionFinite &&
        facts->targetLostAgeMs < 3000u)
    {
        output->visionErrX = facts->visionErrX;
        output->visionErrY = facts->visionErrY;
        output->visionValid = 1u;
    }
}
}
else
{
    if (output->landingActive != 0u)
    {
        /* 缓降保留 Policy 显式机体系命令，但撤销外部 MOTION 微分前馈及其历史。*/
        motionFeedforwardDiffReset();
        output->pitchFeedforwardRad = 0.0f;
        output->rollFeedforwardRad = 0.0f;
    }
    else
    {
        motionFeedforwardDiffUpdate(output->bodyVelocityFwdMps,
            output->bodyVelocityRightMps * FLIGHT_MOTION_VEL_FF_VY_TO_RIGHT_SIGN,
            sequenceContext.cycleSeq, facts->nowMs);
        output->pitchFeedforwardRad = motionFeedforwardDiffGetDvX() *
            sequenceParams.motionFfVxToPitch;
        output->rollFeedforwardRad = motionFeedforwardDiffGetDvY() *
            sequenceParams.motionFfVyToRoll;
    }
}

if ((output->safetyClampMask & (FLIGHT_SEQUENCE_CLAMP_SAFETY_STATE |
    FLIGHT_SEQUENCE_CLAMP_TOF_RAMP)) != 0u)
{
    output->visionValid = 0u;
    output->visionErrX = 0.0f;

```



```

    output->visionErrY = 0.0f;
    output->visionYawErr = 0.0f;
    output->bodyVelocityFwdMps = 0.0f;
    output->bodyVelocityRightMps = 0.0f;
    output->yawRateRadS = 0.0f;
    output->pitchFeedforwardRad = 0.0f;
    output->rollFeedforwardRad = 0.0f;
    motionFeedforwardDiffReset();
}

output->cycleSeq = sequenceContext.cycleSeq;
output->episodeSeq = sequenceContext.episodeSeq;
output->transitionSeq = sequenceContext.transitionSeq;
output->triggerSources |= sequenceContext.latchedTriggerSources;
if (output->primaryReason == CORE0_FAULT_REASON_NONE)
    output->primaryReason = sequenceContext.primaryReason;
output->valid = 1u;
}

```

Core1 MOTION 重入零帧栅栏

来源：[core1_cyt4bb7/project/code/app/core1MotionTxFence.c](https://github.com/ARM-software/psa-architectures/blob/master/core1/cy4bb7/project/code/app/core1MotionTxFence.c)，符号：
core1MotionTxFencePrepare，片段 SHA-256：
80ba2c89e4754f5e6f5fa44208f0e4ab8f5d12a82572fd5403bc0d25041d6f7c。

代码 (c)

```

uint8_t core1MotionTxFencePrepare(core1MotionTxFenceState *state,
    uint32_t nowMs, float vx, float vy, float wz, uint8_t flags,
    core1MotionTxFenceFrame *frame)
{
    uint8_t nonzero;
    if ((state == 0) || (frame == 0)) return 0u;
    nonzero = (uint8_t) ((vx != 0.0f) || (vy != 0.0f) ||
        (wz != 0.0f) || (flags != 0u));
    if ((nonzero != 0u) &&
        ((state->primed == 0u) ||
            ((uint32_t) (nowMs - state->lastTxMs) >= CORE1_MOTION_TX_REARM_MS)))
    {
        frame->vx = 0.0f;
        frame->vy = 0.0f;
        frame->wz = 0.0f;
        frame->flags = 0u;
        state->primed = 1u;
        state->lastTxMs = nowMs;
        return 0u;
    }
    frame->vx = vx;
    frame->vy = vy;
    frame->wz = wz;
    frame->flags = flags;
    state->primed = 1u;
    state->lastTxMs = nowMs;
    return 1u;
}

```

Core1 参数事务阶段推进

来源：core1_cyt4bb7/project/code/comm/paramSyncGateway.c，符号：
advanceGlobal，片 段 SHA-256：
233ee0ea6a7b7db4ceb36b2369172e1ddc658f24277238b68cc03781a920b635。

代码 (c)

```
static void advanceGlobal(void)
{
    uint8_t next;
    if (globalPhase == PARAM_GLOBAL_DECIDE)
    {
        persistRecoveryDecision(PARAM_SYNC_OP_FINISH, globalTransactionId,
                                globalTargetMask);
        globalPhase = PARAM_GLOBAL_FINISH;
        globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
        if (!dispatchGlobalCurrent())
            finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
        return;
    }
    if (globalPhase == PARAM_GLOBAL_INTENT)
    {
        persistRecoveryDecision(PARAM_SYNC_OP_ABORT, globalTransactionId,
                                globalTargetMask);
        globalPhase = PARAM_GLOBAL_LOCK;
        globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
        if (!dispatchGlobalCurrent())
            finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
        return;
    }
    next = nextGlobalTarget(globalPhase, globalTargetMask, globalTarget);
    if (next < 5u)
    {
        globalTarget = next;
        /* Core1 owns the durable coordinator journal and is deliberately the
         * last cleanup participant. If any earlier FINISH failed, do not let
         * Core1 clear that journal: retain the full FINISH decision and replay
         * every participant after recovery instead. */
        if ((globalPhase == PARAM_GLOBAL_FINISH ||
             globalPhase == PARAM_GLOBAL_ABORT) && (next == 1u) &&
            (globalFailedMask != 0u))
        {
            globalFailedMask |= (uint8_t) (1u << 1);
            advanceGlobal();
            return;
        }
        if (!dispatchGlobalCurrent())
            finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
        return;
    }
    if (globalPhase == PARAM_GLOBAL_LOCK)
    {
        if (globalAction == PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY)
            globalPhase = PARAM_GLOBAL_PREPARE_LIBRARY;
        else if (globalAction == PARAM_SYNC_GLOBAL_ACTION_LOAD_RUNTIME)
            globalPhase = PARAM_GLOBAL_VERIFY_LOCK;
        else
            globalPhase = PARAM_GLOBAL_PREPARE_RUNTIME;
        globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
    }
}
```

```

    if (!dispatchGlobalCurrent())
        finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}
else if (globalPhase == PARAM_GLOBAL_VERIFY_LOCK)
{
    globalPhase = PARAM_GLOBAL_PREPARE_RUNTIME;
    globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
    if (!dispatchGlobalCurrent())
        finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}
else if (globalPhase == PARAM_GLOBAL_PREPARE_LIBRARY)
{
    globalPhase = PARAM_GLOBAL_COMMIT;
    globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
    if (!dispatchGlobalCurrent())
        finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}
else if (globalPhase == PARAM_GLOBAL_PREPARE_RUNTIME)
{
    globalPhase = PARAM_GLOBAL_COMMIT_RUNTIME;
    globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
    if (!dispatchGlobalCurrent())
        finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}
else if (globalPhase == PARAM_GLOBAL_COMMIT_RUNTIME)
{
    globalPhase = PARAM_GLOBAL_ACTIVATE;
    globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
    if (!dispatchGlobalCurrent())
        finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}
else if (globalPhase == PARAM_GLOBAL_COMMIT || globalPhase == PARAM_GLOBAL_ACTIVATE)
{
    globalPhase = PARAM_GLOBAL_DECIDE;
    globalTarget = 1u;
    if (!dispatchGlobalCurrent())
        finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}
else if (globalPhase == PARAM_GLOBAL_FINISH)
{
    if (globalFailedMask != 0u)
        rememberRecovery(PARAM_SYNC_OP_FINISH, globalTransactionId,
            globalTargetMask);
    else
        clearRecovery();
    globalCommittedMask = globalFailedMask;
    globalTouchedMask = 0u;
    globalPreparedMask = 0u;
    globalFinishRequired = 0u;
    if (globalFailedMask == 0u &&
        globalConflictCleanupActive != 0u)
    {
        if (restartAfterConflictRecovery() == 0u)
            sendGlobalFinal(PARAM_SYNC_STATUS_FAILED, "RETRY_ROUTE");
        return;
    }
    sendGlobalFinal(globalFailedMask ? PARAM_SYNC_STATUS_FAILED : PARAM_SYNC_STATUS_OK,
        globalFailedMask ? "FINISH" : NULL);
}
else if (globalPhase == PARAM_GLOBAL_ABORT)
{
    uint8_t abortFailed = (uint8_t) (globalFailedMask != 0u);
    uint8_t responseFailed;
    if (abortFailed)
        rememberRecovery(PARAM_SYNC_OP_ABORT, globalTransactionId,
            globalTargetMask);
    else

```

```

        clearRecovery();
        globalCommittedMask &= globalFailedMask;
        globalTouchedMask &= globalFailedMask;
        globalPreparedMask &= globalFailedMask;
        globalFinishRequired = 0u;
        if (abortFailed == 0u && globalConflictCleanupActive != 0u)
        {
            if (restartAfterConflictRecovery() == 0u)
                sendGlobalFinal(PARAM_SYNC_STATUS_FAILED, "RETRY_ROUTE");
            return;
        }
        /* A successful rollback does not turn the operation which triggered it
        * into success. Board-originated transactions are intentionally
        * silent on UART, but still need a FAILED terminal state. */
        responseFailed = (uint8_t) (abortFailed || globalOperationFailed);
        sendGlobalFinal(responseFailed ? PARAM_SYNC_STATUS_FAILED : PARAM_SYNC_STATUS_OK,
            abortFailed ? "ROLLBACK" : (responseFailed ? "ABORTED" : NULL));
    }
    else if (globalPhase == PARAM_GLOBAL_INSPECT_LIBRARY)
    {
        globalTransactionId = paramSyncGlobalPackSlotSummary(
            globalSlotValidMask, globalSlotEmptyMask, globalSlotInvalidMask);
        if (globalTransactionId == 0u)
        {
            globalStateError = PARAM_SYNC_GLOBAL_ERROR_INVALID;
            sendGlobalFinal(PARAM_SYNC_STATUS_FAILED, "INSPECT");
        }
        else
            sendGlobalFinal(PARAM_SYNC_STATUS_OK, NULL);
    }
    else
    {
        sendGlobalFinal(PARAM_SYNC_STATUS_OK, NULL);
    }
}

```

附录 C 关键协议与状态定义

表 C.1 协议域、关键身份与安全语义

域	关键身份/状态	安全语义
Observer V3.2	source、role、capability、session、packet sequence	身份不匹配、重复或陈旧包不续鲜
Observer 配置	revision、hash、DetectMode、source profile、APPLIED	只有精确应用 tuple 才取得规划资格
目标身份	source/session/trackId	仅单来源会话内身份，不是全局 ID
Core1 → Core0 MOTION	producer、sequence、freshness、zero fence	300 ms 失效；重建后先零帧
Core1 ↔ Core2 RS-485	commSeq、mode、500 ms freshness	陈旧命令停车；MODE4 重入另有稳定期
飞行阶段	IDLE、WAIT、RAMP、CRUISE、DESCENDING、LANDED	由 flightSequenceCoordinator 独占
ToF failsafe	NORMAL、HOLD、RAMP、requestDisarm	由 altitudeHold 独占
参数事务	PREPARE、COMMIT、ACTIVATE、FINISH、ABORT	五板精确状态一致后才完成

表 C.2 关键消息字段及接收资格

消息或事实	关键字段	接收与失效规则
Observer 轨迹	source、session、trackId、observation sequence、confirmed、位置与预测	来源与会话必须匹配，序号必须前进，数值有限且在新鲜窗内；边界拒绝只撤销完全匹配的当前身份
Observer 配置	revision、hash、DetectMode、source profile、APPLIED	peer 报告的实际应用 tuple 与 Core1 期望完全一致后才允许进入规划候选池
Core1 → Core0	producer generation、sequence、	静默或 producer 变化后先建立零帧

消息或事实	关键字段	接收与失效规则
MOTION	valid、vx/vy/wz、age	基线；Core0 仅接受基线之后的新序号，超过 300 ms 撤销资格
Core1 → Core2 命令	commSeq、mode、ENABLE、ESTOP、LAUNCH、速度量	非法数值、重复/陈旧序号或超过 500 ms 均停车；MODE4 返回有线 owner 另需稳定期和零控制拍
双核 snapshot	magic、ABI、payload length、boot generation、publish sequence	发布端 Clean、读取端 Invalidate，头部前后身份一致才接受；读取碰撞不得刷新数据年龄
参数事务消息	transaction、slot、operation、phase、participant result	非零事务身份和目标槽必须一致；任一参与者拒绝、超时或应用失败均不能宣布全局完成

表 C.3 主要恢复路径的差异

恢复对象	恢复前条件	首个安全动作	允许恢复非零输出的条件
Core1 → Core0 MOTION	静默、producer 重建/变化或物理重连	首次真实请求发送显式零 MOTION	下一前进序号且 Core0 本机飞行、ToF 和任务门仍合格
Core2 由 MODE4 返回 RS-485	本地 owner 退出，观察到有线链路恢复	约 2 s 稳定期后执行一个 500 μ s 零控制拍	新 commSeq、模式与安全门均合格后的下一控制拍
Observer 轨迹	HELLO 或物理绑定重建	清除旧 session、配置资格和轨迹身份	新 session 完成能力校验、配置 APPLIED 并发布 fresh 确认轨迹
ToF	HOLD 或 RAMP 期间重新获得可信高度	重建高度差分与控制基线	高度资格和飞行阶段允许；不继承失效期间的伪速度
参数事务	协调器或参与节点在未决事务中复位	从持久决策与本地日志恢复阶段	五板事务身份、目标槽和最终结果一致后 FINISH

附录 D 必要程序源代码清单与提交标识

完整项目自有必要源码由
TECHNICAL_REPORT_SOURCE_APPENDIX.md

(TECHNICAL_REPORT_SOURCE_APPENDIX.md) 维护。该附件从 Core0～Core4 和 CoreT3 的 12 个活动 IAR 工程编译单元生成，共收录 416 个项目自有 C/H 文件；每个文件均带原始相对路径和 SHA-256。STMicroelectronics VL53L/VL53LMZ 组件、SeekFree、芯片 SDK、构建产物、二进制和 _legacy 历史目录不复制正文，而是在附件的第三方依赖表中登记；含 SSID、口令和现场 IP 的 CoreT3 网络配置也不写入报告附件。本次最终 Word 已将该附件全文并入“附录 H 完整程序源代码”，并非仅保留文件索引或外部链接。

附录 E 技术参数汇总

类别	参数	数值/说明
架构	生产控制节点	Core0~Core4，共五个 CYT4BB7 双核逻辑节点；Core0/Core1 共载于主飞控 PCB
调试工具	录制板	CoreT3，不参与生产控制
板级设计	嘉立创 EDA 工程	8 套板级设计、1 个拼板文档；双路驱动板和视觉处理板各实装 2 张
整机质量	飞行器/底盘/系留线/总质量	870 g / 2500 g / 100 g / 3470 g（队伍装机记录）
电路容量	整车电容总容量	约 2000 μ F；统计整车所有电路板上的离散电容名义容量，不含电池等效容量和电机绕组
整机功耗	常用/大幅换向峰值	约 600 W / 约 1500 W（队伍运行记录，无同步电压电流原始曲线）
视觉硬件	实体相机	MT9V034 $\times$ 4；其中 Core1/Core3/Core4 三路进入信标连续性，Core0 一路用于车体光学标志视觉
视觉标称指标	坐标分辨率/采集/规划	188 $\times$ 120 输入，图像坐标标称分辨率 1 像素；相机配置 50 fps（20 ms/帧）；Core1 规划周期 10 ms（100 Hz 更新上限），均非赛场毫米级实测精度或实测帧率
传感器总表	种类与数量	MT9V034 相机 $\times$ 4、IMU $\times$ 2（BMI270 $\times$ 1、IMU660RB $\times$ 1）、VL53L8CX ToF $\times$ 1、LC-302 光流 $\times$ 1、正交编码器 $\times$ 4、电机电流采样通道 $\times$ 4；物理器件与采样通道分开计数
飞行动力	电机/桨叶/驱动	F60PRO 无感无刷电机 $\times$ 4；5 英寸 3 叶桨 $\times$ 4；四合一无刷电调
底盘动力	电机/编码器/控制	RC380 有刷电机 $\times$ 4；1024 线正交抗磁编码器 $\times$ 4（申请材料型号描述）；电流内环串联速度外环

类别	参数	数值/说明
附加伺服	数量	0；上述八个动力电机不计作附加伺服电机
动力调试	单电机推力/电流	约 1700 g / 约 40 A；四合一电调完成四路满指令运行，正常任务很少持续满输出；无原始推力曲线，不作持续额定推力或效率结论
总电源	动力电池	鹰氩 4S 7000 mAh 80C；满电 16.8 V（申请材料记载）
实装尺寸	随动车/四旋翼	30 cm×30 cm 投影；35 cm×35 cm×20 cm（申请材料记载，本轮未复测）
防护结构	保护罩网眼	最大宽度小于 40 mm（申请材料记载，本轮未复测）
运行记录	连续运行	单次约 2 min（申请材料记载，无本轮原始测量记录）
完赛确认	当前烧录配置	队伍确认当前板上配置随固件烧录后可完成比赛流程；无本轮原始日志，不外推全部场地和故障组合
观察链	Core1—Core3/Core4	两条独立 4 Mbps UART
飞控周期	角速度/混控、姿态、速度、位置	1/3/9/27 ms
底盘周期	Core2 控制入口	500 μ s
底盘几何	轮半径/轴距/轮距	50/169/210 mm
编码器换算	控制代码单轮每转脉冲	2385；这是当前控制参数，不等同于申请材料中的编码器原始“1024 线”型号描述
MOTION 新鲜度	Core1 $\rightarrow$ Core0	300 ms
底盘命令新鲜度	Core1 $\rightarrow$ Core2 RS-485	500 ms
ToF 分级	HOLD 转 RAMP	连续失效达到 3 s

类别	参数	数值/说明
ToF RAMP	油门偏移下降率	2 pp/s

附录 F 扩展术语与协议字段

术语	含义
owner	对硬件、状态或协议事实拥有唯一写入与生命周期职责的模块
facade	向调用方提供有限生命周期、命令或快照接口的边界
snapshot	带身份、序号、新鲜度的一次性一致性数据副本
session	协议 peer 的一次启动或绑定身份
bindingEpoch	物理绑定边沿代次，用于隔离重启前后的旧租约和事件
fresh	数据身份合法且年龄未越过业务超时
last-good	最近一次一致读取成功的快照；不能据此刷新新鲜度
EdgeHold	普通短时遮挡时对预测目标进行有限时间衰减保持
BORDER_REJECTED	轨迹跨越可信边界后的身份化拒绝事实
fence	重连后先建立安全零基线，再接受新非零命令的屏障

附录 G 可复现资产与验证记录

资产	内容
generation_manifest.json	生成器、固件基线、全部产物 SHA-256 和字节数
mecanum_results.csv/json	六个麦轮场景的输入、轮速和正解
mecanum_saturation_sweep.csv	61 点输入缩放、归一化前后轮速峰值、保留比例与正解
continuity_trace.csv / continuity_summary.json	生产 C 连续性状态机三个独立场景 16 拍输出及状态计数
beacon_tracking_metrics.csv/json	75 帧合成真值夹具的计数、Precision、Recall、F1 及中心定位误差；关联门由板上配置提取
air_ground_coordination_trace.csv/json	空地共享计划、Core0 零帧 fence 和 Core2 MODE4 重入的 9 拍规则模型
parameter_transaction_trace.csv/json	五板正常 LOAD、参与者拒绝和协调器重启恢复的 12 拍协议状态模型
tof_failsafe_logic.csv/json	ToF 源码常量、三条逻辑轨迹及首次停机请求时刻
source_manifest.json	416 个项目自有必要程序源码文件、6 个第三方排除项及哈希
core_algorithm_snippets.md	九个活动源码函数的逐字提取结果
hardware_schematic_appendix.md	45 张原理图和 PCB 铜层的直接引用

本轮未运行 IAR 构建或 Pandoc/DOCX 转换。队伍确认当前板上配置随固件烧录后可完成比赛流程；报告其余新增图表由固件证据基线、Host 或模型生成，并在各节保留来源边界。

固件证据基线：main@16d670d2b7141bfc37bc3d2403d695ffdda1a428。报告
资产生成日期：2026-08-13。

附录 H 完整程序源代码

排除的第三方依赖

- `core0_cyt4bb7/project/code/sensor/tof/VL53L1_register_map.h` : STMicroelectronics VL53 sensor component, 不在报告中复制其正文。
- `core0_cyt4bb7/project/code/sensor/tof/vl53lmz_api.c` : STMicroelectronics VL53 sensor component, 不在报告中复制其正文。
- `core0_cyt4bb7/project/code/sensor/tof/vl53lmz_api.h` : STMicroelectronics VL53 sensor component, 不在报告中复制其正文。
- `core0_cyt4bb7/project/code/sensor/tof/vl53lmz_buffers.h` : STMicroelectronics VL53 sensor component, 不在报告中复制其正文。
- `core0_cyt4bb7/project/code/sensor/tof/vl53lmz_platform.c` : STMicroelectronics VL53 sensor component, 不在报告中复制其正文。
- `core0_cyt4bb7/project/code/sensor/tof/vl53lmz_platform.h` : STMicroelectronics VL53 sensor component, 不在报告中复制其正文。

活动工程入口

- `core0_cyt4bb7/project/iar/project_config/cyt4bb7_cm_7_0.ewp`
- `core0_cyt4bb7/project/iar/project_config/cyt4bb7_cm_7_1.ewp`
- `core1_cyt4bb7/project/iar/project_config/cyt4bb7_cm_7_0.ewp`
- `core1_cyt4bb7/project/iar/project_config/cyt4bb7_cm_7_1.ewp`
- `core2_cyt4bb7/project/iar/project_config/cyt4bb7_cm_7_0.ewp`
- `core2_cyt4bb7/project/iar/project_config/cyt4bb7_cm_7_1.ewp`
- `core3_cyt4bb7/project/iar/project_config/cyt4bb7_cm_7_0.ewp`
- `core3_cyt4bb7/project/iar/project_config/cyt4bb7_cm_7_1.ewp`
- `core4_cyt4bb7/project/iar/project_config/cyt4bb7_cm_7_0.ewp`
- `core4_cyt4bb7/project/iar/project_config/cyt4bb7_cm_7_1.ewp`
- `coreT3_cyt4bb7/project/iar/project_config/cyt4bb7_cm_7_0.ewp`
- `coreT3_cyt4bb7/project/iar/project_config/cyt4bb7_cm_7_1.ewp`

文件清单与正文

core0_cyt4bb7

core0_cyt4bb7/project/code/app/core0Loop.c

SHA-256

:

de8906b100a90bed37be87d1bceb6aad94cf18550a90d5ecd001d1693e1166c8

代码 (c)

```
#include "core0Loop.h"

#include <stdint.h>
#include <string.h>

#include "zf_driver_pit.h"
#include "zf_device_mt9v03x.h"

#include "motor.h"
#include "IMU.h"
#include "pidFlightController.h"
#include "altitudeHold.h"
#include "flightSequenceCoordinator.h"
#include "modeFlight.h"
#include "opticalFlow.h"
#include "velocityEstimator.h"
#include "configFlightControlDefaults.h"
#include "configAltitudeTof.h"
#include "configVelocityEstimator.h"
#include "commonDef.h"
#include "factFreshness.h"
#include "coreDataTransfer.h"
#include "flightParams.h"
#include "crossCoreLink.h"
#include "uartLinkProto.h" /* UART_LINK_C0_FLAG_ * 状态位 + UART_LINK_SYS_MODE_ * 空地模式线值 */

/*
 * core0 cm_7_0 飞控主循环（定高飞控）
 * PIT_CH0 1ms : IMU + 安全状态机（消费发车指令）
 * PIT_CH1 1ms : PID 级联控制 + 混控 + 电机输出
 * 主循环 : VL53L8CX 定高 + 读视觉误差 + 参数热重载 + 共享内存收发
 *
 * 取消急停：cm_7_0 不再读物理按键。发车/恢复/清空/调参全部由 cm_7_1 菜单经
 * 共享内存第三区 paramToFlight 下发：
 * - cmdLaunchSeq/cmdStandbySeq 边沿 → Coordinator 解释后向 modeFlight 发布轻量请求
 * - paramSeq 变化 → 非 CLOSED_LOOP 态热重载 flightParams + applyParams
 * - cmdClearSeq 边沿 → 同 paramSeq（参数已在 params 内回默认）
 *
 * 跨核初始化时序：cm_7_0 从 ESC arming 阶段开始发布 startupStateValid 启动快照，
 * 最终初始化完成后才置 cm70Ready 并发布健康汇总；必须等到 cm_7_1 视觉 cm71Ready 后才允许
 * 安全状态机运行（controlAllowed=1），确保两核就绪。
 */

#define VISION_STALE_TIMEOUT_MS (3000u) /* 视觉误差陈旧阈值：超过则弃用旧误差（相机/视觉核卡死降级） */
#define CHEVRON_FACT_STALE_TIMEOUT_MS (200u)
typedef struct
{
    volatile uint8_t initReady;
    volatile uint8_t controlAllowed;
    volatile uint16_t visionStaleMs;
    volatile uint16_t targetLostMs;
    volatile uint32_t startupInitMs;
    volatile uint32_t loopMs;
    uint32_t lastParamSeq;
    uint32_t lastCmdLaunchSeq;
    uint32_t lastCmdStandbySeq;
    uint32_t lastCmdClearSeq;
    uint32_t lastBootEpoch;
    uint8_t paramHandshakeDone;
    uint8_t paramApplyPending;
    uint32_t pendingParamSeq;
```

```

uint32_t pendingBootEpoch;
uint32_t lastVisionSeq;
uint8_t systemPhase;
uint8_t standalone;
uint8_t motionFenceWaiting;
uint8_t latchedFaultReason;
uint8_t lastSafetyStateForFault;
uint8_t faultAckSuppressActive;
uint8_t previousSystemPhase;
uint32_t lastChevronSeq;
uint32_t lastChevronAgeTickMs;
uint16_t chevronAgeMs;
uint8_t chevronSeqPrimed;
uint32_t readyStableStartMs;
uint32_t motionFenceSeq;
uint32_t readyBeepUntilMs;
uint8_t imuReady;
uint8_t initFailedMask;
visionToFlightStruct visionRx;
flightToVisionStruct flightTx;
paramToFlightStruct paramRx;
visionDebugToFlightStruct chevronFacts;
flightParamsStruct params;
} core0RuntimeContextStruct;

static core0RuntimeContextStruct runtimeContext;

#define CORE0_INITIAL_LINK_WAIT_MS (2000u)
#define CORE0_REJOIN_STABLE_MS (2000u)

#define initReadyFlag runtimeContext.initReady
#define controlAllowed runtimeContext.controlAllowed
#define visionStaleMs runtimeContext.visionStaleMs
#define targetLostMs runtimeContext.targetLostMs
#define startupInitMs runtimeContext.startupInitMs
#define loopMs runtimeContext.loopMs
#define lastParamSeq runtimeContext.lastParamSeq
#define lastCmdLaunchSeq runtimeContext.lastCmdLaunchSeq
#define lastCmdStandbySeq runtimeContext.lastCmdStandbySeq
#define lastCmdClearSeq runtimeContext.lastCmdClearSeq
#define lastBootEpoch runtimeContext.lastBootEpoch
#define paramHandshakeDone runtimeContext.paramHandshakeDone
#define paramApplyPending runtimeContext.paramApplyPending
#define pendingParamSeq runtimeContext.pendingParamSeq
#define pendingBootEpoch runtimeContext.pendingBootEpoch
#define lastVisionSeq runtimeContext.lastVisionSeq
#define visionRx runtimeContext.visionRx
#define flightTx runtimeContext.flightTx
#define paramRx runtimeContext.paramRx
#define runtimeParams runtimeContext.params
#define imuReady runtimeContext.imuReady
#define faultReasonLatch runtimeContext.latchedFaultReason
#define lastSafetyStateForFault runtimeContext.lastSafetyStateForFault
#define faultAckSuppressActive runtimeContext.faultAckSuppressActive

/* C1/C3/C4 均为必需视觉；Core1 自身断链由既有主链路保护独立覆盖，
 * 这里把远端通用 source/class 事实并入发车门和运行中缓降。*/
static uint8_t requiredVisionFault(uint8_t *source, uint8_t *faultClass)
{
    if (!crossCoreLinkIsCore1Alive())
    {
        if (source != NULL) *source = 2u;
        if (faultClass != NULL) *faultClass = 2u;
        return 1u;
    }
    return crossCoreLinkGetRequiredVisionFault(source, faultClass);
}

static uint8_t anyRequiredLinkLost(void)
{
    return (uint8_t) (!crossCoreLinkIsCore1Alive() ||
        crossCoreLinkGetCore2LinkLost() || requiredVisionFault(NULL, NULL));
}

static void core0FaultLatch(uint8_t reason)
{
    if ((reason != CORE0_FAULT_REASON_NONE) &&
        (faultAckSuppressActive == 0u) &&
        (faultReasonLatch == CORE0_FAULT_REASON_NONE))

```

```

    {
        if (reason == CORE0_FAULT_REASON_REQUIRED_VISION)
        {
            uint8_t source = 0u;
            uint8_t faultClass = 0u;
            (void) crossCoreLinkGetRequiredVisionFault(&source, &faultClass);
            faultReasonLatch = CORE0_FAULT_REASON_VISION(source, faultClass);
        }
        else faultReasonLatch = reason;
    }
}

static uint8_t sequenceNewer(uint32_t value, uint32_t baseline)
{
    return (uint8_t) ((value != baseline) && ((int32_t) (value - baseline) > 0));
}

/* LoRa 组合键是电平传输、事件消费。只有连续新鲜的真实低电平才能重新武装；
 * 链路陈旧、Core2 失联合成低值和模式切换均不得把同一次物理按压拆成两次事件。 */
static void core0RuntimeUpdateSystemPhase(void)
{
    uint8_t ready = (uint8_t) ((controlAllowed != 0u) &&
        (anyRequiredLinkLost() == 0u));
    if ((runtimeContext.systemPhase == CORE_UX_PHASE_REJOIN) &&
        (runtimeContext.standalone == 0u))
    {
        if (pidFlightControllerMotionBarrierPending() == 0u)
        {
            runtimeContext.systemPhase = CORE_UX_PHASE_HOME;
            runtimeContext.readyBeepUntilMs = loopMs + 700u;
        }
        return;
    }
    if (runtimeContext.systemPhase == CORE_UX_PHASE_LINK_WAIT)
    {
        if (ready != 0u)
        {
            runtimeContext.systemPhase = CORE_UX_PHASE_HOME;
            runtimeContext.standalone = 0u;
            runtimeContext.readyBeepUntilMs = loopMs + 700u;
        }
        else if (loopMs >= CORE0_INITIAL_LINK_WAIT_MS)
        {
            runtimeContext.systemPhase = CORE_UX_PHASE_STANDALONE;
            runtimeContext.standalone = 1u;
        }
        return;
    }
    if (runtimeContext.standalone == 0u)
    {
        if (ready == 0u)
        {
            runtimeContext.systemPhase = CORE_UX_PHASE_STANDALONE;
            runtimeContext.standalone = 1u;
            runtimeContext.readyStableStartMs = 0u;
            runtimeContext.motionFenceSeq = crossCoreLinkFenceMotion();
            runtimeContext.motionFenceWaiting = 1u;
        }
        return;
    }
    if (ready == 0u)
    {
        runtimeContext.systemPhase = CORE_UX_PHASE_STANDALONE;
        runtimeContext.readyStableStartMs = 0u;
        return;
    }
    if (runtimeContext.readyStableStartMs == 0u)
        runtimeContext.readyStableStartMs = loopMs;
    runtimeContext.systemPhase = CORE_UX_PHASE_REJOIN;
    if (((uint32_t) (loopMs - runtimeContext.readyStableStartMs)) >= CORE0_REJOIN_STABLE_MS)
    {
        runtimeContext.motionFenceSeq = crossCoreLinkFenceMotion();
        runtimeContext.motionFenceWaiting = 1u;
        pidFlightControllerRequestMotionBarrier(1u);
        runtimeContext.standalone = 0u;
        runtimeContext.readyStableStartMs = 0u;
        runtimeContext.systemPhase = CORE_UX_PHASE_REJOIN;
    }
}

```

```

/* 相机模式是独立安全控制量：参数整包在 CLOSED_LOOP 延后时，它仍在合法 seq/epoch 边沿即时生效。 */
static uint8_t applyFlightParamsSnapshot(const paramToFlightStruct* src)
{
    if (!modeFlightTryBeginConfigUpdate()) return 0u;
    (void) flightParamsCommit(&src->params);
    flightParamsGetActive(&runtimeParams);
    pidFlightControllerApplyParams();
    altitudeHoldApplyParams();
    IMUApplyParams();
    modeFlightApplyParams();
    flightSequenceApplyParams();
    paramApplyPending = 0u;
    pendingParamSeq = src->paramSeq;
    pendingBootEpoch = src->cm71BootEpoch;
    modeFlightEndConfigUpdate();
    return 1u;
}

static void markFlightParamsPending(const paramToFlightStruct* src)
{
    paramApplyPending = 1u;
    pendingParamSeq = src->paramSeq;
    pendingBootEpoch = src->cm71BootEpoch;
}

/* ESC arming 进度回调：arming 阻塞期间持续把进度写共享内存，cm_7_1 据此画进度条。
 * 启动阶段 safetyState 约定值的单一真相源在 coreDataTransfer.h。 */
static void armProgressPublish(uint8_t progress)
{
    {
        flightTx.safetyState = FLIGHT_SAFETY_ARMING;
        flightTx.armProgress = progress;
        flightTx.tofHealthy = 0u;
        flightTx.startupStateValid = 1u;
        coreDataWriteFlightToVision(&flightTx);
    }
}

/* ESC arming 完成后才进入 ToF 循环初始化。每轮完整初始化前发布启动阶段。 */
static void tofInitPublish(void)
{
    {
        flightTx.safetyState = FLIGHT_SAFETY_TOF_INIT;
        flightTx.armProgress = 0u;
        flightTx.tofHealthy = 0u;
        flightTx.startupStateValid = 1u;
        coreDataWriteFlightToVision(&flightTx);
    }
}

/* =====
 * processInit
 * ===== */
void processInit(void)
{
    runtimeContext.systemPhase = CORE_UX_PHASE_LINK_WAIT;
    runtimeContext.standalone = 0u;
    runtimeContext.readyStableStartMs = 0u;
    runtimeContext.motionFenceWaiting = 0u;
    faultAckSuppressActive = 0u;
    runtimeContext.previousSystemPhase = CORE_UX_PHASE_LINK_WAIT;
    runtimeContext.lastChevronSeq = 0u;
    runtimeContext.lastChevronAgeTickMs = loopMs;
    runtimeContext.chevronAgeMs = 0xFFFFu;
    runtimeContext.chevronSeqPrimed = 0u;
    memset(&runtimeContext.chevronFacts, 0, sizeof(runtimeContext.chevronFacts));
    runtimeContext.initFailedMask = 0u;
    startupInitMs = 0u;
    imuReady = 0u;
    /* 飞控参数 RAM 镜像必须最先加载：IMUInit(gyroLpf)/pidFlightControllerInit/
     * altitudeHoldInit 均依赖 flightParams 默认值。Phase 5 后由 cm_7_1 经共享
     * 内存覆盖，但上电首帧先用 configFlightControlDefaults.h 默认值兜底。 */
    flightParamsLoadDefaults(&runtimeParams);
    flightParamsSetInitial(&runtimeParams);
    flightParamsGetActive(&runtimeParams);
    modeFlightApplyParams();

    memset(&flightTx, 0, sizeof(flightTx));

    /* PIT_CH0 在 initReady 前只提供启动毫秒计数，不运行飞控或外设周期逻辑。 */
    pit_ms_init(PIT_CH0, 1u);
}

```

```

/* 电调优先：motorInit 先把四路 PWM 驱动到 950us 确定停机态，
 * 再阻塞完成 arming。ToF 重试不得占用 PWM 引脚未初始化的上电窗口。 */
motorInit(&armProgressPublish); /* 阻塞 ~5s arming，回调持续把进度写共享内存供 cm_7_1 画进度条 */

```

```

{
    uint32_t tofInitStartMs = startupInitMs;
    uint8_t tofReady = 0u;

    do
    {
        /* altitudeHoldInit 每轮都会进入 tofL8cxInit，并通过 LPN（代码沿用 XSHUT 宏名）完整复位。 */
        tofInitPublish();
        tofReady = altitudeHoldInit();
        if (tofReady != 0u) break;
    }
    while ((uint32_t)(startupInitMs - tofInitStartMs) < ALT_TOF_INIT_TIMEOUT_MS);

    if (tofReady == 0u) runtimeContext.initFailedMask |= CORE0_INIT_FAIL_TOF;
}

```

```

{
    uint8_t imuResult = IMUInit();
    if (imuResult)
    {
        /* IMU 失败：电机保持停机，但不中断前台链路/共享状态服务。 */
        motorStopAll();
        runtimeContext.initFailedMask |= CORE0_INIT_FAIL_IMU;
    }
    else imuReady = 1u;
}

```

```

pidFlightControllerApplyParams();
pidFlightControllerInit();
flightSequenceInit(0u);
if (opticalFlowInit() == 0u) runtimeContext.initFailedMask |= CORE0_INIT_FAIL_OPTICAL_FLOW;
velocityEstimatorInit(); /* IMU+光流双轴速度估计器（KF 反馈已接水平速度内环） */

```

```

/* 核间链路(UART_2/P10 ↔ core1 P04): 发按键事件透传 core1 + 双向心跳。
 * 收发在前台 whileProcess, 心跳分频/存活计数在 pitProcess0(轻量), 不扰 400Hz 飞控。 */
crossCoreLinkInit();

```

```

/* 初始化共享内存发送区并标记 cm_7_0 就绪（握手） */
memset(&flightTx, 0, sizeof(flightTx));
flightTx.cm70Ready = CORE_DATA_READY_MAGIC;
flightTx.initFailedMask = runtimeContext.initFailedMask;
coreDataWriteFlightToVision(&flightTx);

```

```

pit_us_init(PIT_CH1, 1000u);

```

```

    initReadyFlag = 1u;
}

```

```

/* =====
 * pitProcess0 — 1ms: IMU + 安全状态机
 * ===== */

```

```

void pitProcess0(void)

```

```

{
    if (!initReadyFlag)
    {
        startupInitMs++;
        return;
    }

    if (imuReady != 0u)
    {
        IMUUpdate();
        velocityEstimatorPredictTick(); /* IMU 去重力+水平投影+双轴速度预测（1ms） */
    }
    altitudeHoldTickMs(); /* ToF 失活看门狗（运行期拔线降级） */
    loopMs++; /* 自由 ms 计时：Coordinator 按无符号差值计算 wrap-safe dt。 */
}

```

```

/* 视觉新鲜度看门狗：1ms 累加，whileProcess 收到新 visionSeq 时清零 */
if (visionStaleMs < 0xFFFFu) visionStaleMs++;

```

```

/* 目标丢失保持窗口：1ms 累加，whileProcess 收到有效目标时清零 */
if (targetLostMs < 0xFFFFu) targetLostMs++;

```

```

/* 仅在跨核握手通过后运行硬安全状态机；前台 Coordinator 只发布轻量请求标志。 */
if ((controlAllowed != 0u) && (imuReady != 0u))

```

```

    {
        modeFlightSafetyTick();
    }

/* 核间链路心跳分频 + core1 存活超时计数(仅计数+置标志, 轻量, 不发送/不阻塞)。 */
crossCoreLinkPit1ms();
}

/* =====
* pitProcess1 — 1ms: 控制环
* ===== */
void pitProcess1(void)
{
    if (!initReadyFlag) return;

    if (imuReady != 0u) pidFlightControllerTick();
}

/* =====
* whileProcess — 主循环: 定高 + 跨核握手 + 共享内存收发
* ===== */
void whileProcess(void)
{
    flightSequenceFactsStruct sequenceFacts;
    flightSequenceOutputStruct sequenceOutput;
    crossCoreFlightFactsSnapshotStruct linkFacts;
    visionDebugToFlightStruct chevronFacts;
    velocityEstimatorSnapshotStruct velocitySnapshot;
    pidFlightGuidanceCommandStruct guidance;
    uint8_t launchCommandEdge = 0u;
    uint8_t standbyCommandEdge = 0u;
    float motionVx = 0.0f;
    float motionVy = 0.0f;
    float motionWz = 0.0f;
    uint32_t motionSeq = 0u;
    uint32_t chevronElapsedMs;
    uint8_t motionFresh = 0u;
    flightSafetySnapshotStruct safety;
    pidFlightSnapshotStruct pidSnapshot;
    altitudeHoldSnapshotStruct altitudeSnapshot;
    if (!initReadyFlag) return;
    /* 先排空 UART 并更新存活事实, 再解释任何远端请求或运动量。 */
    crossCoreLinkPoll();
    modeFlightGetSnapshot(&safety);
    pidFlightControllerGetSnapshot(&pidSnapshot);
    altitudeHoldGetSnapshot(&altitudeSnapshot);

    /* 跨核握手: 等待 cm_7_1 视觉就绪后放行控制 */
    if (!controlAllowed)
    {
        coreDataReadVisionToFlight(&visionRx);
        if ((visionRx.cm71Ready == CORE_DATA_READY_MAGIC) && (imuReady != 0u))
        {
            controlAllowed = 1u;
        }
    }

    /* 参数区轮询: 热重载 + 发车/清空指令 (cm_7_1 菜单下发) */
    if (coreDataReadParamToFlight(&paramRx) != 0u)
    {
        if (!paramHandshakeDone)
        {
            /* 首次握手: 以 cm_7_1 持久化参数为准; 闭环态只挂起参数 apply。 */
            paramHandshakeDone = 1u;
            lastParamSeq = paramRx.paramSeq;
            lastCmdClearSeq = paramRx.cmdClearSeq;
            lastCmdLaunchSeq = paramRx.cmdLaunchSeq; /* 忽略上电前的陈旧发车计数 */
            lastCmdStandbySeq = paramRx.cmdStandbySeq; /* 忽略上电前的陈旧回待机计数 */
            lastBootEpoch = paramRx.cm71BootEpoch;
            if (!applyFlightParamsSnapshot(&paramRx))
            {
                markFlightParamsPending(&paramRx);
            }
        }
        else if (paramRx.cm71BootEpoch != lastBootEpoch ||
            paramRx.paramSeq < lastParamSeq ||
            paramRx.cmdLaunchSeq < lastCmdLaunchSeq ||
            paramRx.cmdStandbySeq < lastCmdStandbySeq ||
            paramRx.cmdClearSeq < lastCmdClearSeq)

```



```

{
    /* 对端(cm_7_1)复位检测——双判据，任一成立即视为复位：
     * (1) cm71BootEpoch 变化 (cm_7_1 由共享区上一帧 epoch+1 派生，单调递增)；
     * (2) 任一 seq 相对 last 后退（确定性兜底：cm_7_1 复位后 seq 从 N 跌回 0，
     *     单调计数器永不正常后退，故 seq<last 必为复位）。
     * 复位时只重同步基线并挂起参数 apply，绝不合成发车/回待机边沿，
     * 杜绝把计数器回退误判成“未操作的发车/回待机”。等真实 K5/菜单操作再增量。 */
    lastBootEpoch = paramRx.cm71BootEpoch;
    lastParamSeq = paramRx.paramSeq;
    lastCmdClearSeq = paramRx.cmdClearSeq;
    lastCmdLaunchSeq = paramRx.cmdLaunchSeq;
    lastCmdStandbySeq = paramRx.cmdStandbySeq;
    runtimeContext.chevronSeqPrimed = 0u;
    runtimeContext.chevronAgeMs = 0xFFFFu;
    markFlightParamsPending(&paramRx);
}
else
{
    /* 参数热重载：闭环态只记录待应用，退出闭环后再套用最新快照。 */
    if (paramRx.paramSeq != lastParamSeq || paramRx.cmdClearSeq != lastCmdClearSeq)
    {
        lastParamSeq = paramRx.paramSeq;
        lastCmdClearSeq = paramRx.cmdClearSeq;
        if (!applyFlightParamsSnapshot(&paramRx))
        {
            markFlightParamsPending(&paramRx);
        }
    }

    /* 发车指令边沿：置 pending，ISR 安全状态机消费 */
    if (paramRx.cmdLaunchSeq != lastCmdLaunchSeq)
    {
        lastCmdLaunchSeq = paramRx.cmdLaunchSeq;
        launchCommandEdge = 1u;
    }

    /* 回待机指令边沿：飞行中 K5 长按下发，置 disarm pending 让 ISR 安全状态机停机 */
    if (paramRx.cmdStandbySeq != lastCmdStandbySeq)
    {
        lastCmdStandbySeq = paramRx.cmdStandbySeq;
        standbyCommandEdge = 1u;
    }
}

if (paramApplyPending &&
    paramRx.paramSeq == pendingParamSeq &&
    paramRx.cm71BootEpoch == pendingBootEpoch)
{
    (void) applyFlightParamsSnapshot(&paramRx);
}

/* KVM 路由与事件仅在前台快照给链路 owner；UART 重试/ACK 不进入 ISR。 */
crossCoreLinkSetKvmRoute(paramRx.cm71BootEpoch,
    paramRx.kvmRouteEpoch,
    paramRx.kvmTarget,
    paramRx.kvmEventSeq,
    paramRx.kvmKeyId,
    paramRx.kvmKeyState,
    (uint8_t) (paramRx.kvmFlags & 1u));
}

/* 参数/命令处理可能跨过 PIT_CH0 状态转换；控制源与缓降决策使用就近安全快照。 */
modeFlightGetSnapshot(&safety);
if ((lastSafetyStateForFault == (uint8_t) SAFETY_STATE_CLOSED_LOOP) &&
    (safety.state == SAFETY_STATE_PROTECTED))
{
    core0FaultLatch(CORE0_FAULT_REASON_PROTECT_ATTITUDE);
}
if ((faultAckSuppressActive != 0u) &&
    (lastSafetyStateForFault != (uint8_t) SAFETY_STATE_CLOSED_LOOP) &&
    (safety.state == SAFETY_STATE_CLOSED_LOOP))
{
    /* 新的闭环飞行 episode 重新武装故障页；只抑制已 ACK 的上一事件。 */
    faultAckSuppressActive = 0u;
}
lastSafetyStateForFault = (uint8_t) safety.state;
core0RuntimeUpdateSystemPhase();

/* 先刷新视觉/MOTION/链路事实，再由唯一 Coordinator 解释触发、推进阶段并执行最终仲裁。 */

```

```

coreDataReadVisionToFlight(&visionRx);
if (visionRx.visionSeq != lastVisionSeq)
{
    lastVisionSeq = visionRx.visionSeq;
    visionStaleMs = 0u;
}
if (visionRx.visionTargetValid != 0u) targetLostMs = 0u;
chevronElapsedMs = loopMs - runtimeContext.lastChevronAgeTickMs;
runtimeContext.lastChevronAgeTickMs = loopMs;
runtimeContext.chevronAgeMs = factFreshnessAdvanceAgeU16(
    runtimeContext.chevronAgeMs, chevronElapsedMs);
(void) coreDataTryReadVisionDebugToFlight(&runtimeContext.chevronFacts);
chevronFacts = runtimeContext.chevronFacts;
if ((chevronFacts.magic == CORE_DATA_READY_MAGIC) &&
    (factFreshnessAcceptSeq(&runtimeContext.chevronSeqPrimed,
        &runtimeContext.lastChevronSeq, chevronFacts.seq) != 0u))
{
    runtimeContext.chevronAgeMs = 0u;
}
crossCoreLinkGetFlightFactsSnapshot(&linkFacts);
motionFresh = crossCoreLinkGetMotionSnapshot(&motionVx, &motionVy, &motionWz, &motionSeq);
if ((motionFresh != 0u) && (runtimeContext.motionFenceWaiting != 0u))
{
    if (sequenceNewer(motionSeq, runtimeContext.motionFenceSeq) != 0u)
        runtimeContext.motionFenceWaiting = 0u;
    else motionFresh = 0u;
}
velocityEstimatorGetSnapshot(&velocitySnapshot);

memset(&sequenceFacts, 0, sizeof(sequenceFacts));
sequenceFacts.nowMs = loopMs;
sequenceFacts.callerCycleSeq = loopMs;
sequenceFacts.safety = safety;
sequenceFacts.initialized = initReadyFlag;
sequenceFacts.controlReady = (uint8_t) (controlAllowed && imuReady);
sequenceFacts.systemMode = runtimeParams.systemMode;
sequenceFacts.systemPhase = runtimeContext.systemPhase;
sequenceFacts.standalone = runtimeContext.standalone;
sequenceFacts.paramsValid = 1u;
sequenceFacts.readyBeepActive = ((int32_t) (runtimeContext.readyBeepUntilMs - loopMs) > 0) ? 1u : 0u;
sequenceFacts.k5LaunchEdge = launchCommandEdge;
sequenceFacts.k5StandbyEdge = standbyCommandEdge;
sequenceFacts.faultLatched = (faultReasonLatch != CORE0_FAULT_REASON_NONE) ? 1u : 0u;
sequenceFacts.rejoinEdge = (uint8_t) (runtimeContext.previousSystemPhase != CORE_UX_PHASE_REJOIN) &&
    (runtimeContext.systemPhase == CORE_UX_PHASE_REJOIN);
runtimeContext.previousSystemPhase = runtimeContext.systemPhase;
sequenceFacts.mode3RequestValid = linkFacts.mode3RequestValid;
sequenceFacts.mode3RequestLevel = linkFacts.mode3RequestLevel;
sequenceFacts.mode3RequestAgeMs = linkFacts.motionAgeMs;
sequenceFacts.mode3RequestSeq = linkFacts.motionSeq;
sequenceFacts.mode2RequestValid = linkFacts.mode2RequestValid;
sequenceFacts.mode2RequestLevel = linkFacts.mode2RequestLevel;
sequenceFacts.mode2RequestAgeMs = linkFacts.statusAgeMs;
sequenceFacts.mode2RequestSeq = linkFacts.statusSeq;
sequenceFacts.core1Online = crossCoreLinkIsCore1Alive();
sequenceFacts.core1FactValid = initReadyFlag;
sequenceFacts.core2FactValid = linkFacts.statusValid;
sequenceFacts.requiredVisionFactValid = linkFacts.statusValid;
sequenceFacts.core2LinkLost = crossCoreLinkGetCore2LinkLost();
sequenceFacts.core2AttitudeFault = linkFacts.core2AttitudeFault;
sequenceFacts.requiredVisionFault = crossCoreLinkGetRequiredVisionFault(
    &sequenceFacts.visionFaultSource, &sequenceFacts.visionFaultClass);
sequenceFacts.core1AgeMs = linkFacts.core1AgeMs;
sequenceFacts.core2AgeMs = linkFacts.statusAgeMs;
sequenceFacts.requiredVisionAgeMs = linkFacts.statusAgeMs;
sequenceFacts.linkSeq = linkFacts.statusSeq;
sequenceFacts.core1Seq = linkFacts.core1Seq;
sequenceFacts.core2Seq = linkFacts.statusSeq;
sequenceFacts.requiredVisionSeq = linkFacts.statusSeq;
sequenceFacts.altitude = altitudeSnapshot;
sequenceFacts.visionErrX = visionRx.visionErrX;
sequenceFacts.visionErrY = visionRx.visionErrY;
sequenceFacts.visionYawErr = visionRx.visionYawErr;
sequenceFacts.visionTargetValid = visionRx.visionTargetValid;
sequenceFacts.visionFrameValid = (visionStaleMs < VISION_STALE_TIMEOUT_MS) ? 1u : 0u;
sequenceFacts.visionAgeMs = visionStaleMs;
sequenceFacts.targetLostAgeMs = targetLostMs;
sequenceFacts.visionSeq = visionRx.visionSeq;
sequenceFacts.chevronX = chevronFacts.filteredCenterX;

```

```

sequenceFacts.chevronY = chevronFacts.filteredCenterY;
sequenceFacts.chevronOrientationRad = chevronFacts.orientation;
sequenceFacts.chevronOrientationConfidence = chevronFacts.orientationConfidence;
sequenceFacts.chevronPositionValid = (uint8_t) (chevronFacts.magic == CORE_DATA_READY_MAGIC &&
    runtimeContext.chevronAgeMs < CHEVRON_FACT_STALE_TIMEOUT_MS &&
    chevronFacts.trackingState == 2u && chevronFacts.targetLost == 0u &&
    chevronFacts.filteredCenterX >= 0.0f &&
    chevronFacts.filteredCenterX <= (float) (MT9V03X_W - 1u) &&
    chevronFacts.filteredCenterY >= 0.0f &&
    chevronFacts.filteredCenterY <= (float) (MT9V03X_H - 1u));
sequenceFacts.chevronOrientationValid = (uint8_t) (sequenceFacts.chevronPositionValid &&
    chevronFacts.orientProtectState == 0u &&
    chevronFacts.orientation >= -3.1415927f && chevronFacts.orientation <= 3.1415927f &&
    chevronFacts.orientationConfidence >= 0.0f && chevronFacts.orientationConfidence <= 1.0f);
sequenceFacts.chevronAgeMs = runtimeContext.chevronAgeMs;
sequenceFacts.chevronSeq = chevronFacts.seq;
sequenceFacts.beaconCount = linkFacts.beaconCount;
sequenceFacts.beaconAny = (linkFacts.beaconCount != 0u) ? 1u : 0u;
sequenceFacts.beaconSearching = linkFacts.beaconSearching;
sequenceFacts.beaconSearchTimedOut = linkFacts.beaconSearchTimedOut;
sequenceFacts.beaconStateValid = linkFacts.beaconStateValid;
sequenceFacts.beaconStateAgeMs = linkFacts.statusAgeMs;
sequenceFacts.beaconStateSeq = linkFacts.beaconStateSeq;
sequenceFacts.motionVx = motionVx;
sequenceFacts.motionVy = motionVy;
sequenceFacts.motionWz = motionWz;
sequenceFacts.motionValid = motionFresh;
sequenceFacts.motionFenceOpen = runtimeContext.motionFenceWaiting ? 0u : 1u;
sequenceFacts.motionAgeMs = linkFacts.motionAgeMs;
sequenceFacts.motionSeq = motionSeq;
sequenceFacts.kfVelocityFwdMps = velocitySnapshot.velocityFwd;
sequenceFacts.kfVelocityRightMps = velocitySnapshot.velocityRight;
sequenceFacts.kfVelocityValid = velocitySnapshot.valid;

flightSequenceStep(&sequenceFacts, &sequenceOutput);
altitudeSnapshot = sequenceOutput.altitude;
if (sequenceOutput.acknowledgeFault != 0u)
{
    faultReasonLatch = CORE0_FAULT_REASON_NONE;
    faultAckSuppressActive = 1u;
}
if (sequenceOutput.primaryReason != CORE0_FAULT_REASON_NONE)
    core0FaultLatch(sequenceOutput.primaryReason);
if (sequenceOutput.requestLaunch != 0u) modeFlightRequestLaunch();
if (sequenceOutput.requestDisarm != 0u) modeFlightRequestDisarm();
if ((faultAckSuppressActive != 0u) &&
    (safety.state == SAFETY_STATE_DISARMED) &&
    (sequenceOutput.landingActive == 0u) &&
    (altitudeSnapshot.failsafeStage == ALTITUDE_FAILSAFE_NORMAL))
    faultAckSuppressActive = 0u;

velocityEstimatorUpdate((uint8_t) (
    altitudeSnapshot.failsafeStage == ALTITUDE_FAILSAFE_HOLD &&
    sequenceOutput.altReached != 0u && sequenceOutput.landingActive == 0u &&
    safety.state == SAFETY_STATE_CLOSED_LOOP));
velocityEstimatorGetSnapshot(&velocitySnapshot);
memset(&guidance, 0, sizeof(guidance));
guidance.visionErrX = sequenceOutput.visionErrX;
guidance.visionErrY = sequenceOutput.visionErrY;
guidance.visionYawErr = sequenceOutput.visionYawErr;
guidance.visionValid = sequenceOutput.visionValid;
guidance.visionHeld = sequenceOutput.visionHeld;
guidance.kfVelFwd = velocitySnapshot.velocityFwd;
guidance.kfVelRight = velocitySnapshot.velocityRight;
guidance.kfVelValid = velocitySnapshot.valid;
guidance.ffVelFwd = sequenceOutput.bodyVelocityFwdMps;
guidance.ffVelRight = sequenceOutput.bodyVelocityRightMps;
guidance.ffPitch = sequenceOutput.pitchFeedforwardRad;
guidance.ffRoll = sequenceOutput.rollFeedforwardRad;
guidance.ffYawRate = sequenceOutput.yawRateRadS;
pidFlightControllerSetGuidanceCommand(&guidance);
flightTx.velX = velocitySnapshot.velocityFwd;
flightTx.velY = velocitySnapshot.velocityRight;
flightTx.velValid = velocitySnapshot.valid;

/* 写飞控调试数据供 cm_7_1 显示 */
pidFlightControllerGetSnapshot(&pidSnapshot);
modeFlightGetSnapshot(&safety);
altitudeHoldGetSnapshot(&altitudeSnapshot);

```

```

{
    imuSnapshotStruct imu;
    IMUGetSnapshot(&imu);
    flightTx.pitch = imu.physicalPitch;
    flightTx.roll = imu.physicalRoll;
    flightTx.yaw = imu.yaw;
}
flightTx.altMm = (float) altitudeSnapshot.distanceMm;
flightTx.baseThr = pidSnapshot.baseThrottlePct;
flightTx.motorPct[0] = pidSnapshot.motorPct[0];
flightTx.motorPct[1] = pidSnapshot.motorPct[1];
flightTx.motorPct[2] = pidSnapshot.motorPct[2];
flightTx.motorPct[3] = pidSnapshot.motorPct[3];
flightTx.safetyState = (imuReady != 0u) ? (uint8_t) safety.state : FLIGHT_SAFETY_IMUFAIL;
flightTx.armProgress = motorArmProgress;
flightTx.tofHealthy = altitudeHoldIsHealthy();
flightTx.cm70Ready = CORE_DATA_READY_MAGIC;
{
    uint8_t core1Alive = crossCoreLinkIsCore1Alive();
    flightTx.core1LinkUp = core1Alive; /* 远程菜单: core1 链路存活回报 cm_7_1(断联保护/禁发车) */
    flightTx.core2LinkLost = core1Alive ? crossCoreLinkGetCore2LinkLost() : 1u;
    flightTx.requiredVisionFault = requiredVisionFault(NULL, NULL);
    flightTx.systemPhase = runtimeContext.systemPhase;
    flightTx.primaryReason = !core1Alive ? 1u :
        (flightTx.core2LinkLost ? 2u :
            (linkFacts.core2AttitudeFault ? 4u :
                (flightTx.requiredVisionFault ? 3u : 0u)));
    flightTx.initFailedMask = runtimeContext.initFailedMask;
    flightTx.latchedFaultReason = faultReasonLatch;
}
coreDataWriteFlightToVision(&flightTx);

/* RGB 状态灯: 打包 core0 飞控状态经 CORE0_STATUS 帧发 core1(crossCoreLinkPoll 前台发送)。
 * safetyState/armProgress 直接用上面已填的 flightTx; statusFlags 按位汇总 4 个健康维度。
 * cm70Ready 恒置位(cm_7_0 运行到此即已就绪)。 */
{
    uint8_t c0Flags = UART_LINK_C0_FLAG_CM70_READY;
    if (flightTx.velValid) c0Flags |= UART_LINK_C0_FLAG_VEL_VALID;
    if (visionRx.visionTargetValid) c0Flags |= UART_LINK_C0_FLAG_TARGET_VALID;
    if (flightTx.tofHealthy) c0Flags |= UART_LINK_C0_FLAG_TOF_HEALTHY;
    /* 飞控进入闭环=已发车: 经 CORE0_STATUS 状态位传 core1, 由 core1 中转为 RS-485 cmd LAUNCH 给 core2。
     * 电平语义: 闭环期间持续置位, 解除闭环(落地/急停回 DISARM)即清零, 车随之停发车。 */
    if (safety.state == SAFETY_STATE_CLOSED_LOOP) c0Flags |= UART_LINK_C0_FLAG_LAUNCHED;
    crossCoreLinkSetCore0Status(flightTx.safetyState, flightTx.armProgress, c0Flags);

    /* 空地系统下发 core1 (CORE0_STATUS sysCtrl + buzzer) :
     * sysCtrl bit0-1 = systemMode (core1 据此选 core2 控制源 485/LoRa) ;
     * sysCtrl bit2-4 = 起飞时序 rgbIntent (core1 rgbAnimator 自演动画) ;
     * sysCtrl bit5 = altReached (到顶; core1 据此放行车发车 LAUNCH) ;
     * sysCtrl bit6/7 = padding, 恒为 0;
     * buzzer 字节 = 起飞时序蜂鸣模式码 (core1 中转 core2 buzzer 库 pattern 层播放) 。
     * 每拍填真值: core1 据此门控车发车(LAUNCH = launched && altReached)。altReached 表示
     * 本架次已获得 CRUISE 资格且尚未进入 RAMP/正式缓降, 不等价于每拍 ToF 健康; WAIT/RAMP 为 0,
     * CRUISE 后的短时 ToF HOLD 保持 1。 */
    {
        uint8_t sysCtrl = (uint8_t) (runtimeParams.systemMode & UART_LINK_SYS_MODE_MASK);
        sysCtrl |= (uint8_t) ((sequenceOutput.rgbIntent & UART_LINK_SYS_RGB_MASK) <<
            UART_LINK_SYS_RGB_SHIFT);
        if (sequenceOutput.altReached) sysCtrl |= UART_LINK_SYS_ALT_REACHED;
        crossCoreLinkSetSysControl(sysCtrl, sequenceOutput.buzzerPattern);
    }
}
}

```

core0_cyt4bb7/project/code/app/core0Loop.h

SHA-256

:

dec9dfe59a297beb5bd5b993d21812f34c016fa5892f68ed1c296cbe2bd255fb

代码 (c)

```

#ifndef _CORE0LOOP_H_
#define _CORE0LOOP_H_

```

```
#include <stdint.h>

void processInit(void);
void pitProcess0(void);
void pitProcess1(void);
void whileProcess(void);

#endif /* _CORE0LOOP_H_ */
```

core0_cyt4bb7/project/code/app/core1Loop.c

SHA-256

:

b9a7cc27ae090c865c34be9e4b9c67a97082dec4a253ca5e30a821d8e55c1fbd

代码 (c)

```
#include "core1Loop.h"
#include <string.h>
#include <math.h>

#include "zf_driver_pit.h"

#include "zf_device_mt9v03x.h"
#include "commonDef.h"
#include "coreDataTransfer.h"
#include "image.h"
#include "menu.h"
#include "paramSyncOwnerService.h"
#include "schmitt.h"

/* =====
 * core0 cm_7_1 视觉核（调度层，UI/菜单在 menu 模块： menu/menuModel/menuPersistence/menuRender/menuInput）
 * - 运行 ^ 灯板视觉管线（image.c）
 * - 每帧计算图像误差写与共享内存供 cm_7_0 飞控
 * - 菜单 + IPS114 显示 + 按键 + flash 由 menu 模块封装
 *
 * 本文件只做：外设/视觉/菜单 init、PIT 节拍、whileProcess 调度。
 * ===== */

volatile uint8_t initReadyFlag = 0u;
static uint8_t cameraReady = 0u;

#define MENU_PIT_TICK_MS (10u) /* PIT_CH2 菜单节拍周期 ms： pit_ms_init 与目标丢失冻结计时单一真相源 */
#define VISION_LOST_FREEZE_TICK_MS (MENU_PIT_TICK_MS) /* 目标丢失冻结每拍累加，恒等于 PIT_CH2 周期 */

/* 共享内存发送缓存（主循环使用，非 ISR） */
static visionToFlightStruct visionTx;

/* 外环误差施密特(滞回)死区：复用通用 schmitt 库（util/schmitt）。
 * 判据=质心到目标中点合成半径平方 r²； upper=大圆半径(进入,转高)、 lower=小圆半径(退出,归零)。
 * outputState： 0=低(已到位,误差归零)、 1=高(跟踪中,正常输出)。BSS 初值 0=初始低状态
 * (刚锁定要求先越过大圆才驱动)； 阈值随菜单 deadzone% 每帧刷新。 */
static SchmittTrigger visionDeadzone;

/* 目标丢失外推：持续维护最后一次有效目标质心与最后一次已发布 X/Y 误差（无论施密特高/低状态都记录）。
 * 丢失后先冻结输出一段可调时间，再按最后质心相对 exitR 分类为归零或沿最后视向外推。 */
static float lastValidCx = 0.0f;
static float lastValidCy = 0.0f;
static float lastOutputErrX = 0.0f;
static float lastOutputErrY = 0.0f;
static uint8_t lastValidHas = 0u; /* 是否已有有效质心历史（无则丢失时不外推） */
static volatile uint8_t visionLostActive = 0u;
static volatile uint16_t visionLostElapsedMs = 0u;

/* 计算图像误差并写共享内存（每帧调用，与显示解耦）。
 * 目标中点/极性取自菜单可调镜像 menuVisionTuning（运行时可改 + flash 持久化）。 */
static void publishVisionError(const imageObservationSnapshotStruct* observation)
{
    visionTuningStruct tuning;
    menuGetVisionTuning(&tuning);
    if (observation->valid)
    {
        /* 位置发布当前质心(OM_POSITION_ALPHA=1.0 默认不过滤)；速度发布“原始质心差分→低通”的结果。 */
        float cx = observation->filteredCenterX;
        float cy = observation->filteredCenterY;
```

```

/* 持续维护最后有效质心（先于施密特门控，保证死区低状态也记录真实质心，供丢失外推用方向）。*/
lastValidCx = cx;
lastValidCy = cy;
lastValidHas = 1u;
visionLostActive = 0u;
visionLostElapsedMs = 0u;

/* 施密特死区：半径用原始像素偏差(与极性无关)；用 r² 比较省 sqrt。
* 半径基准=图像高 H 的百分比直径(半径=0.5×pct/100×H)。阈值用半径²。*/
float dx = cx - tuning.targetCenterX;
float dy = cy - tuning.targetCenterY;
float r2 = dx * dx + dy * dy;
float enterR = 0.5f * (tuning.deadzoneEnterPct * 0.01f) * (float) MT9V03X_H;
float exitR = 0.5f * (tuning.deadzoneExitPct * 0.01f) * (float) MT9V03X_H;
/* 依赖 menuModel 钳位保证 enter≥exit(故 enterR²≥exitR²)：upper≥lower 时库不交换阈值，
* 施密特进入/退出语义正确；若该钳位被破坏致 upper<lower，库会交换→死区逻辑反转。*/
schmittSetThresholds(&visionDeadzone, enterR * enterR, exitR * exitR);
schmittUpdate(&visionDeadzone, r2);

if (visionDeadzone.outputState)
{
    visionTx.visionErrX = tuning.errXSign * dx;
    visionTx.visionErrY = tuning.errYSign * dy;
}
else
{
    /* 死区内：位置误差归零(已到位)，valid 仍 1 → 位置外环速度目标归零、
    * 速度内环常驻把速度收向 0(悬停抗漂)，不触发失效降级。*/
    visionTx.visionErrX = 0.0f;
    visionTx.visionErrY = 0.0f;
}
lastOutputErrX = visionTx.visionErrX;
lastOutputErrY = visionTx.visionErrY;
visionTx.visionVelX = tuning.errXSign * observation->velocityX;
visionTx.visionVelY = tuning.errYSign * observation->velocityY;
/* yaw 朝向误差 不受位置死区影响；朝向基线未确认时不做偏航修正。*/
visionTx.visionYawErr = observation->orientBaselineValid
    ? (tuning.yawErrSign * wrapToPi(observation->orientation - tuning.targetAngleRad))
    : 0.0f;
visionTx.visionTargetValid = 1u;
}
else
{
    /* 目标丢失两阶段：
    * 1) 丢失后 lostFreezeMs 内冻结最后一次已发布 X/Y 误差，避免从真实误差阶跃到外推圆误差；
    * 2) 超时后，若最后质心在 exitR 内则归零；否则沿*目标中点 C → 最后有效质心 P*
    * 射线投影到外推圆周作虚拟靶点，驱动飞机朝最后视向找回。
    * 契约：丢失期 visionTargetValid=0，但 visionErrX/Y 可非零；偏航/速度丢失期归零。
    * 目标丢失安全上限仍由 cm_7_0 TARGET_LOST_HOLD_MS(3s) 兜底。*/
    float extrapErrX = 0.0f;
    float extrapErrY = 0.0f;

    visionDeadzone.outputState = 0u; /* 重捕从低状态开始(保留)*/

    if (!visionLostActive)
    {
        visionLostActive = 1u;
        visionLostElapsedMs = 0u;
    }

    if (lastValidHas)
    {
        uint16_t lostElapsedMs = visionLostElapsedMs;
        if ((float) lostElapsedMs < tuning.lostFreezeMs)
        {
            extrapErrX = lastOutputErrX;
            extrapErrY = lastOutputErrY;
        }
        else
        {
            float dx = lastValidCx - tuning.targetCenterX;
            float dy = lastValidCy - tuning.targetCenterY;
            float dist = sqrtf(dx * dx + dy * dy);
            float exitR = 0.5f * (tuning.deadzoneExitPct * 0.01f) * (float) MT9V03X_H;
            if (dist >= exitR && dist > 1e-3f)
            {
                float R = 0.5f * tuning.lostExtrapDiameterPx; /* 外推圆半径*/
                extrapErrX = tuning.errXSign * R * dx / dist; /* 虚拟靶点沿最后视向，距中点 R*/
            }
        }
    }
}

```

```

        extrapErrY = tuning.errYSign * R * dy / dist;
    }
}
}

visionTx.visionErrX = extrapErrX;
visionTx.visionErrY = extrapErrY;
visionTx.visionVelX = 0.0f;
visionTx.visionVelY = 0.0f;
visionTx.visionYawErr = 0.0f; /* 丢失期偏航不修正 */
visionTx.visionTargetValid = 0u;
}
visionTx.cm71Ready = CORE_DATA_READY_MAGIC;
visionTx.visionSeq++; /* 每发布一帧自增, 供 cm_7_0 判新鲜度 */
coreDataWriteVisionToFlight(&visionTx);
}

/* 每帧把 ^ 灯板业务事实写入保留 ABI 名称的 visionDebugRegion, 供 cm_7_0 生成 Policy 事实
* 与 Core0 Pose。该共享区不再连接任何串口调试输出。与 publishVisionError 同源
* (chevronDetector)、同拍(仅新帧调), cm_7_0 只接受一致快照。 */
static void publishVisionDebug(const imageObservationSnapshotStruct* observation)
{
    static uint32_t visionDebugSeq = 0u;
    visionDebugToFlightStruct dbg;

    dbg.magic = CORE_DATA_READY_MAGIC;
    visionDebugSeq++;
    if (visionDebugSeq == 0u) visionDebugSeq = 1u;
    dbg.seq = visionDebugSeq;
    dbg.trackingState = observation->trackingState;
    dbg.contourCount = observation->contourCount;
    dbg.selectedIndex = observation->selectedIndex;
    dbg.targetLost = observation->targetLost;
    /* 无候选(contourCount==0)时 contourData[si] 的 center/size/Hu 是上帧残留(om_contour 每帧只清 contourSize,
    * 不清 center/Hu), 整组置 0 避免调试通道显示陈旧值, 与 CH11=0/CH13=1 自治。
    * contourCount>0(含 targetLost 但有真实轮廓)则照常显示选中轮廓观测值, 保留诊断价值。 */
    if (observation->contourCount > 0u)
    {
        dbg.selectedCenterX = observation->selectedCenterX;
        dbg.selectedCenterY = observation->selectedCenterY;
        dbg.selectedSize = observation->selectedSize;
        dbg.selectedHuDistance = observation->selectedHuDistance;
    }
    else
    {
        dbg.selectedCenterX = 0u;
        dbg.selectedCenterY = 0u;
        dbg.selectedSize = 0u;
        dbg.selectedHuDistance = 0.0f;
    }
    dbg.filteredCenterX = observation->filteredCenterX;
    dbg.filteredCenterY = observation->filteredCenterY;
    dbg.orientationFresh = observation->orientationFresh;
    dbg.padding0 = 0u;
    dbg.orientation = observation->orientation;
    dbg.orientationConfidence = observation->orientationConfidence;
    dbg.orientProtectState = observation->orientProtectState;
    dbg.orientSpikeFrameCount = observation->orientSpikeFrameCount;
    dbg.lostFrameCount = observation->lostFrameCount;
    dbg.lockFrameCount = observation->lockFrameCount;
    coreDataWriteVisionDebugToFlight(&dbg);
}

/* =====
* processInit
* ===== */
void processInit(void)
{
    paramSyncOwnerServiceInit();
    cameraReady = imageInit();

    /* 菜单接管 IPS114 + 按键 + flash + 参数下发 + 握手 */
    menuInit();
    menuSetInitHealth(cameraReady ? 0u : CORE0_INIT_FAIL_CAMERA, 0u);

    /* 初始化共享内存发送区并标记 cm_7_1 就绪 */
    memset(&visionTx, 0, sizeof(visionTx));
    visionTx.cm71Ready = CORE_DATA_READY_MAGIC;
    coreDataWriteVisionToFlight(&visionTx);
}

```



```

/* 菜单节拍用 PIT_CH2: PIT_CH0/CH1 已被 cm_7_0 飞控占用（同一片 TCPWM0 硬件定时器
* 跨双核共享），cm_7_1 若复用 PIT_CH1 会与 cm_7_0 的 1ms 控制环争用同一计数器，
* 把按键扫描节拍拉快~10 倍 → 短按难触发且被误判长按。改用空闲的 PIT_CH2。*/
pit_ms_init(PIT_CH2, MENU_PIT_TICK_MS);

    initReadyFlag = 1u;
}

/* =====
* pitProcess0 — 10ms (PIT_CH2): 菜单按键扫描 + 节拍（轻量）
* ===== */
void pitProcess0(void)
{
    if (!initReadyFlag) return;
    menuPitTick();
    if (visionLostActive)
    {
        if (visionLostElapsedMs <= (uint16_t) (0xFFFFu - VISION_LOST_FREEZE_TICK_MS))
        {
            visionLostElapsedMs = (uint16_t) (visionLostElapsedMs + VISION_LOST_FREEZE_TICK_MS);
        }
        else
        {
            visionLostElapsedMs = 0xFFFFu;
        }
    }
}

/* =====
* whileProcess — 视觉处理 + 误差发布 + 菜单调度
* ===== */
void whileProcess(void)
{
    if (!initReadyFlag) return;

    paramSyncOwnerServicePoll();

    /* 先消费按键并热应用参数；IPS114、反馈与 Flash 仍统一留在循环尾。*/
    menuServiceInput();

    /* 视觉处理（无新帧直接进菜单调度，保证 UI 仍响应）*/
    if ((cameraReady != 0u) && imageProcess())
    {
        imageObservationSnapshotStruct observation;
        imageGetObservationSnapshot(&observation);
        publishVisionError(&observation);
        publishVisionDebug(&observation); /* 保留 ABI 名称：同拍发布 Policy 灯板事实与 Core0 Pose 业务快照 */
    }

    /* 菜单尾服务（Flash/反馈/渲染），自身按刷新节拍限流。*/
    menuTick();
}

```

core0_cyt4bb7/project/code/app/core1Loop.h

SHA-256

:

030adc393b85cebd984ca84f8e555a05274425c2f79499f0902079bc7db9f319

代码 (c)

```

#ifndef _CORE1LOOP_H_
#define _CORE1LOOP_H_

#include <stdint.h>

#include "cy_device_headers.h"

void processInit(void);
void pitProcess0(void);
void whileProcess(void);

#endif /* _CORE1LOOP_H_ */

```

core0_cyt4bb7/project/code/app/menu.c

SHA-256

:

c86ccd7bf7b75b625a23828065c0609dc9575fe13c8ea9646b0df859bdba80a5

代码 (c)

```
#include "menu.h"
#include "menuInternal.h"
#include "paramSyncOwnerService.h"

#include <string.h>
#include <math.h>
#include <stdio.h>

#include "menuKey.h"
#include "zf_device_ips114.h"
#include "zf_device_mt9v03x.h"
#include "zf_driver_flash.h"
#include "zf_common_font.h"

#include "configVision.h"
#include "image.h"
#include "coreDataTransfer.h"
#include "flightParams.h"

/*
 * core0 cm_7_1 菜单 + 屏幕显示 (core1Loop 仅调度)。本模块拆分为：
 * menu.c      共享状态唯一定义 + 公共 API(menuInit/PitTick/Tick) + 状态机分发 + arming 门控
 * menuModel.c 参数默认值/本地热重载/下发 cm_7_0 + 页表构建
 * menuPersistence.c flash blob/CRC/版本迁移 + 读写/擦除
 * menuRender.c IPS114 绘制原语 + 各状态渲染
 * menuInput.c 按键 handler + 反馈/清屏/滚动维护
 * 共享宏/类型在 menuInternal.h，跨文件状态与函数原型亦在该头集中声明。
 *
 * 参数三类：
 * 1) 视觉/相机参数 (菜单本地应用，不下发 cm_7_0)：menuVisionTuning + DetectMode/BinaryCC/Canny + 曝光
 * 2) 飞控参数 (编辑后经共享内存推给 cm_7_0 热重载)：menuFlightWork
 * 3) 指令：发车(cmdLaunch) / 回待机(cmdStandby) / 清空恢复默认(cmdClear)
 *
 * 屏幕：IPS114 240×135 (横向)，8×16 字体 (30 列×8 行)，飞行中持续刷新。
 * 开机：先显示约 5s ESC arming 进度，再显示 cm_7_0 ToF 初始化状态；
 * 两阶段均忽略按键，完成后才进入发车页 (可交互)。
 */

/* 刷新：10ms tick → 100ms 刷新 */
#define LCD_REFRESH_DIV (10u)
#define FLIGHT_READ_FAIL_CLOSED_MS (600u)
#define KVM_READ_FAIL_CLOSED_MS (600u)
#define MENU_STORAGE_NONE (0u)
#define MENU_STORAGE_SAVE (1u)
#define MENU_STORAGE_RESTORE (2u)

/* =====
 * 共享状态唯一定义点 (跨文件量在 menuInternal.h extern 声明)
 * ===== */
menuContextStruct menuContext;

static void discardAllMenuKeys(void)
{
    uint8_t keyIndex;
    for (keyIndex = 0u; keyIndex < (uint8_t) MENU_KEY_COUNT; keyIndex++)
    {
        menuKeyDiscardUntilRelease((menuKeyIndexEnum) keyIndex);
    }
}

static uint8_t faultReasonValid(uint8_t reason)
{
    if ((reason & CORE0_FAULT_REASON_VISION_FLAG) != 0u)
    {
        uint8_t source = CORE0_FAULT_REASON_VISION_SOURCE(reason);
        uint8_t faultClass = CORE0_FAULT_REASON_VISION_CLASS(reason);
    }
}
```

```

        return (uint8_t) ((source >= 1u) && (source <= 4u) &&
            (faultClass >= 1u) && (faultClass <= 7u));
    }
    return (uint8_t) ((reason >= CORE0_FAULT_REASON_PROTECT_ATTITUDE) &&
        (reason <= CORE0_FAULT_REASON_LAND_C2_ATTITUDE));
}

/* =====
* 开机启动门控（ESC arming 后进入 ToF 初始化，两个阶段均禁止按键）
* ===== */
static void updateArmingGate(void)
{
    /* cm_7_0 依次回报 ARMING 与 TOF_INIT；两个阶段均未完成启动。
    * 完成判据：cm_7_0 最终置 cm70Ready=MAGIC —— 说明启动
    * 已结束（无论 cm_7_1 是否亲眼见过 ARMING 窗口）。这样可覆盖 cm_7_1 启动慢、
    * 单独复位、错过 100% 帧等情形，避免永久卡在进度条。
    * cm_7_0 未最终就绪时保持当前启动页。 */
    if (!bootArmingDone)
    {
        if (flightRx.cm70Ready != CORE_DATA_READY_MAGIC)
        {
            return;
        }
        /* cm_7_0 就绪且已离开启动阶段 */
        bootArmingDone = 1u;
        clearScreenForState(); /* arming 结束清屏进发车页 */
    }
}

/* =====
* 对外接口
* ===== */
void menuInit(void)
{
    menuState = MENU_HOME;
    pagIdx = 0;
    optIdx = 0;
    scrollTop = 0u;
    forceRedraw = 1u;
    bootArmingDone = 0u;
    bootRenderState = 0xFFu;
    armBarLast = 0xFFu;
    feedbackHoldTicks = 0u;
    feedbackWasActive = 0u;
    menuInitFailedMask = 0u;
    menuInitUnverifiedMask = 0u;
    menuInitSummaryTicks = 0u;
    menuInitSummaryLastSecond = 0xFFFFu;
    menuInitSummaryPresented = 0u;
    menuInputServiced = 0u;
    menuPendingScreenClear = 0u;
    menuPendingFeedback = 0u;
    menuPendingFeedbackColor = RGB565_BLACK;
    menuPendingStorageAction = MENU_STORAGE_NONE;
    menuParamGlobalReset();
    menuFaultPageActive = 0u;
    menuFaultAckPending = 0u;
    menuFaultDisplayReason = CORE0_FAULT_REASON_NONE;
    menuFaultRenderedReason = CORE0_FAULT_REASON_NONE;
    menuFaultRenderedSafetyState = 0xFFu;
    remoteDeferredKeyValid = 0u;
    remoteDeferredKeyId = 0u;
    remoteDeferredKeyState = 0u;
    remoteDeferredKeyTickMs = 0u;
    memset(&flightRx, 0, sizeof(flightRx));
    flightLastGeneration = 0u;
    flightLastAdvanceMs = 0u;
    flightGenerationPrimed = 0u;
    flightSnapshotAvailable = 0u;
    mutationLockLast = 0u;
    menuTickMs = 0u;
    kvmReadFailureStartMs = 0u;
    kvmReadFailureActive = 0u;
    /* 取本启动唯一 epoch：读共享参数区上一帧 cm71BootEpoch + 1。共享区(0x28001200)
    * 不在 cm_7_1 SRAM 的 startup ECC 清零范围内，跨 cm_7_1 复位保留，故 epoch 单调递增，
    * 每次 cm_7_1 复位后必与上次不同。最高位置 1 保证非 0（cm_7_0 lastBootEpoch 初值 0，
    * 0 epoch 会漏判首次复位）。首帧(共享区未握手)用固定非 0 种子。 */
}

```

```

    paramToFlightStruct priorFrame;
    if (coreDataReadParamToFlight(&priorFrame) != 0u)
    {
        bootEpoch = (priorFrame.cm71BootEpoch + 1u) | 0x80 000 000u;
    }
    else
    {
        bootEpoch = 0x80 000 001u;
    }
}
seqParam = 0u;
seqLaunch = 0u;
seqStandby = 0u;
seqClear = 0u;
menuKvmRouteEpoch = (bootEpoch ^ 0x4B564D31u) | 1u;
remoteKeySeq = 0u;
remoteKeyPending = 0u;
remoteRouteRestartPending = 0u;
remoteEnterThresholdPending = 0u;
menuKeyTargetCore = UART_LINK_KVM_TARGET_CORE0;

menuLoadDefaults();
menuBuildPages();

/* IPS114 8×16 横屏 180，黑底绿字 */
ips114_set_dir(IPS114_PORTAIT_180);
ips114_set_font(IPS114_8X16_FONT);
ips114_init();
ips114_set_color(RGB565_GREEN, RGB565_BLACK);
ips114_clear();
menuRenderResetListSnapshot();

menuKeyInit(10u);

/* flash 载入：magic/version/CRC 校验，失败回退默认 → 本地应用 → 推 cm_7_0 */
flash_init();
menuFlashLoad();
applyVisionLocal();
menuPushParams();
}

void menuPitTick(void)
{
    menuTickMs += 10u;
    menuKeyScan();

    if (menuInitSummaryTicks > 0u) menuInitSummaryTicks--;

    /* 填充反馈倒计时（10ms 确定时长，不受前台帧率影响） */
    if (feedbackHoldTicks > 0u) feedbackHoldTicks--;

    lcdRefreshCount++;
    if (lcdRefreshCount >= LCD_REFRESH_DIV)
    {
        lcdRefreshCount = 0u;
        lcdRefreshFlag = 1u;
    }
}

void menuRequestScreenClear(void)
{
    menuPendingScreenClear = 1u;
    forceRedraw = 1u;
}

void menuRequestFeedback(uint16_t color)
{
    menuPendingFeedbackColor = color;
    menuPendingFeedbackAction = PARAM_SYNC_GLOBAL_ACTION_NONE;
    menuPendingFeedbackSlot = 0u;
    menuPendingFeedback = 1u;
}

void menuRequestParamFeedback(uint16_t color, uint8_t action, uint8_t slot)
{
    menuPendingFeedbackColor = color;
    menuPendingFeedbackAction = action;
    menuPendingFeedbackSlot = slot;
}

```

```

    menuPendingFeedback = 1u;
}

void menuRequestSaveSettings(void)
{
    if (paramSyncOwnerServiceMutationLocked() != 0u) return;
    menuPendingStorageAction = MENU_STORAGE_SAVE;
}

void menuRequestRestoreDefaults(void)
{
    if (paramSyncOwnerServiceMutationLocked() != 0u) return;
    menuPendingStorageAction = MENU_STORAGE_RESTORE;
}

static void applyPendingTailEffects(void)
{
    if (paramSyncOwnerServiceMutationLocked() != 0u)
    {
        menuPendingStorageAction = MENU_STORAGE_NONE;
        (void) imageTakeCalibrationSaveRequest();
        return;
    }
    if (imageTakeCalibrationSaveRequest())
    {
        menuFlashSave();
    }
    if (menuPendingStorageAction == MENU_STORAGE_SAVE)
    {
        menuPendingStorageAction = MENU_STORAGE_NONE;
        menuSaveSettings();
    }
    else if (menuPendingStorageAction == MENU_STORAGE_RESTORE)
    {
        menuPendingStorageAction = MENU_STORAGE_NONE;
        menuRestoreDefaults();
    }
}

if (menuPendingFeedback != 0u)
{
    menuPendingFeedback = 0u;
    menuPendingScreenClear = 0u;
    ips114_full(menuPendingFeedbackColor);
    if (menuPendingFeedbackAction == PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY ||
        menuPendingFeedbackAction == PARAM_SYNC_GLOBAL_ACTION_LOAD_RUNTIME)
    {
        char slotText[16];
        ips114_set_color((menuPendingFeedbackColor == RGB565_GREEN) ?
            RGB565_BLACK : RGB565_WHITE, menuPendingFeedbackColor);
        ips114_show_string(80u, 42u,
            (menuPendingFeedbackAction ==
                PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY) ?
                "SAVE OK" : "LOAD OK");
        (void) snprintf(slotText, sizeof(slotText), "SLOT %u",
            (unsigned) menuPendingFeedbackSlot);
        ips114_show_string(88u, 66u, slotText);
        ips114_set_color(RGB565_GREEN, RGB565_BLACK);
    }
    menuPendingFeedbackAction = PARAM_SYNC_GLOBAL_ACTION_NONE;
    menuPendingFeedbackSlot = 0u;
    menuRenderResetListSnapshot();
    feedbackHoldTicks = MENU_FEEDBACK_HOLD_TICKS;
    return;
}
if (menuPendingScreenClear != 0u)
{
    menuPendingScreenClear = 0u;
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
    ips114_clear();
    menuRenderResetListSnapshot();
    forceRedraw = 1u;
}
}

void menuServiceInput(void)
{
    flightToVisionStruct nextFlight;
    uint32_t nextFlightGeneration;
    kvmStatusToMenuStruct nextKvm;

```

```

uint32_t nowMs = menuTickMs;

if (menuInputServiced != 0u) return;
menuInputServiced = 1u;

/* 读 cm_7_0 状态 → arming 门控。读写碰撞只保留 last-good;
 * 已确认 generation 连续 600ms 未前进才按真实静默 fail-closed。 */
if (coreDataTryReadFlightToVision(&nextFlight,
    &nextFlightGeneration) != 0u)
{
    if ((flightGenerationPrimed == 0u) ||
        (nextFlightGeneration != flightLastGeneration))
    {
        flightRx = nextFlight;
        flightLastGeneration = nextFlightGeneration;
        flightLastAdvanceMs = nowMs;
        flightGenerationPrimed = 1u;
        flightSnapshotAvailable = 1u;
    }
}
if ((flightGenerationPrimed == 0u) ||
    ((uint32_t) (nowMs - flightLastAdvanceMs) >=
        FLIGHT_READ_FAIL_CLOSED_MS))
{
    flightSnapshotAvailable = 0u;
}
if (coreDataTryReadKvmStatusToMenu(&nextKvm) != 0u)
{
    kvmRx = nextKvm;
    kvmReadFailureActive = 0u;
}
else if (kvmReadFailureActive == 0u)
{
    kvmReadFailureActive = 1u;
    kvmReadFailureStartMs = nowMs;
}
else if ((uint32_t) (nowMs - kvmReadFailureStartMs) >= KVM_READ_FAIL_CLOSED_MS)
{
    memset(&kvmRx, 0, sizeof(kvmRx));
}
updateArmingGate();
if ((menuFaultAckPending == 0u) &&
    (menuFaultDisplayReason == CORE0_FAULT_REASON_NONE) &&
    (faultReasonValid(flightRx.latchedFaultReason) != 0u))
{
    menuFaultDisplayReason = flightRx.latchedFaultReason;
}

/* 等 cm_7_0 完成初始化发布后再一次性汇总，避免本地相机结果先到而漏掉
 * IMU/ToF/光流等较晚到达的失败事实。 */
if ((menuInitSummaryPresented == 0u) &&
    (flightRx.cm70Ready == CORE_DATA_READY_MAGIC) &&
    ((menuInitFailedMask | menuInitUnverifiedMask | flightRx.initFailedMask |
        flightRx.initUnverifiedMask) != 0u))
{
    menuInitSummaryPresented = 1u;
    menuInitSummaryTicks = 300u;
    menuInitSummaryLastSecond = 0xFFFFu;
    forceRedraw = 1u;
}

if (menuInitSummaryTicks > 0u)
{
    discardAllMenuKeys();
    return;
}
if (menuInitSummaryLastSecond != 0xFFFFu)
{
    menuInitSummaryLastSecond = 0xFFFFu;
    clearScreenForState();
}

if (!bootArmingDone)
{
    discardAllMenuKeys();
    return;
}

if ((flightRx.cm70Ready == CORE_DATA_READY_MAGIC) &&

```

```

(menuFaultDisplayReason != CORE0_FAULT_REASON_NONE) &&
(menuFaultAckPending == 0u))
{
    menuState = MENU_HOME;
    if (menuFaultAckPending == 0u)
    {
        (void) handleFaultReasonAck();
    }
    else
    {
        discardAllMenuKeys();
    }
    return;
}
if (menuFaultAckPending != 0u)
{
    if (flightRx.latchedFaultReason == CORE0_FAULT_REASON_NONE)
    {
        menuFaultAckPending = 0u;
        menuFaultDisplayReason = CORE0_FAULT_REASON_NONE;
        menuFaultRenderedReason = CORE0_FAULT_REASON_NONE;
        menuFaultRenderedSafetyState = 0xFFu;
    }
    else discardAllMenuKeys();
    return;
}

/* === 填充反馈保持期: 屏幕维持填充色, 吞掉按键; 倒计时(10ms PIT 递减)结束后清屏重绘 === */
if (feedbackHoldTicks > 0u)
{
    feedbackWasActive = 1u;
    /* 吞掉保持期内的按键, 避免反馈结束后误触发陈旧事件 */
    discardAllMenuKeys();
    return;
}
if (feedbackWasActive)
{
    feedbackWasActive = 0u;
    clearScreenForState(); /* 保持期刚结束: 清屏 + forceRedraw, 下方按当前 menuState 重绘 */
}

/* === 按键 === */
if (handleFlightStandbyKey())
{
    return;
}

/* 远程操控模式: 按键转发给 core1, 不进本地菜单状态机。core0 屏保持发车页(红边框)。
 * 强制回发车页, 使切换瞬间本地不停留在子菜单。 */
if (menuKeyTargetCore)
{
    if (menuState != MENU_HOME)
    {
        menuState = MENU_HOME;
        clearScreenForState();
    }
    handleRemoteForward();
}
else
{
    switch (menuState)
    {
        {
            case MENU_HOME: handleHomeKeys(); break;
            case MENU_PAGE_LIST: handlePageListKeys(); break;
            case MENU_OPTION_LIST: handleOptionListKeys(); break;
            case MENU_EDIT: handleEditKeys(); break;
            case MENU_THRESH_EDIT: handleThreshEditKeys(); break;
            case MENU_PARAM_SLOTS: handleParamSlotKeys(); break;
            default: break;
        }
    }
}

}

void menuTick(void)
{
    uint8_t mutationLocked;
    menuParamGlobalPoll();
    if (menuPendingFeedback != 0u)

```



```

{
    discardAllMenuKeys();
    menuInputServiced = 1u;
}
mutationLocked = paramSyncOwnerServiceMutationLocked();
if (mutationLocked != mutationLockLast)
{
    discardAllMenuKeys();
    menuPendingStorageAction = MENU_STORAGE_NONE;
    (void) imageTakeCalibrationSaveRequest();
    mutationLockLast = mutationLocked;
}
if (menuInputServiced == 0u) menuServiceInput();
menuInputServiced = 0u;
applyPendingTailEffects();

if (menuInitSummaryTicks > 0u)
{
    uint16_t second = (uint16_t)((menuInitSummaryTicks + 99u) / 100u);
    if ((second != menuInitSummaryLastSecond) || forceRedraw)
    {
        forceRedraw = 0u;
        renderInitFaultSummary((uint8_t)(menuInitFailedMask | flightRx.initFailedMask),
            (uint8_t)(menuInitUnverifiedMask | flightRx.initUnverifiedMask),
            (uint8_t)second);
        menuInitSummaryLastSecond = second;
    }
    return;
}
if (!bootArmingDone)
{
    uint8_t bootState = (flightRx.startupStateValid != 0u) ?
        flightRx.safetyState : FLIGHT_SAFETY_ARMING;

    if (bootState == FLIGHT_SAFETY_TOF_INIT)
    {
        if ((bootRenderState != FLIGHT_SAFETY_TOF_INIT) || forceRedraw)
        {
            forceRedraw = 0u;
            renderToInit();
            bootRenderState = FLIGHT_SAFETY_TOF_INIT;
        }
    }
    else
    {
        uint8_t prog = flightRx.armProgress;
        if (bootRenderState != FC_STATE_ARMING)
        {
            ips114_clear();
            menuRenderResetListSnapshot();
            armBarLast = 0xFFu;
            bootRenderState = FC_STATE_ARMING;
        }
        if ((prog != armBarLast) || forceRedraw)
        {
            forceRedraw = 0u;
            renderArming(prog);
            armBarLast = prog;
        }
    }
    return;
}
if ((flightRx.cm70Ready == CORE_DATA_READY_MAGIC) &&
    (menuFaultDisplayReason != CORE0_FAULT_REASON_NONE) &&
    (menuFaultAckPending == 0u))
{
    uint8_t fullRedraw = (menuFaultPageActive == 0u) ? 1u : 0u;
    uint8_t reasonChanged = (menuFaultRenderedReason !=
        menuFaultDisplayReason) ? 1u : 0u;
    uint8_t stateChanged = (menuFaultRenderedSafetyState !=
        flightRx.safetyState) ? 1u : 0u;
    lcdRefreshFlag = 0u;
    if ((fullRedraw != 0u) || (reasonChanged != 0u) ||
        (stateChanged != 0u))
    {
        forceRedraw = 0u;
        menuFaultPageActive = 1u;
        renderFaultReasonPage(menuFaultDisplayReason,
            flightRx.safetyState, fullRedraw,

```

```

        reasonChanged, stateChanged);
        menuFaultRenderedReason = menuFaultDisplayReason;
        menuFaultRenderedSafetyState = flightRx.safetyState;
    }
    return;
}
if (menuFaultPageActive != 0u)
{
    menuFaultPageActive = 0u;
    menuFaultRenderedReason = CORE0_FAULT_REASON_NONE;
    menuFaultRenderedSafetyState = 0xFFu;
    clearScreenForState();
    return;
}
if ((feedbackHoldTicks > 0u) || (menuPendingFeedback != 0u)) return;

/* === 渲染（仅在刷新节拍或强制重绘时，限 SPI 流量） === */
if (!lcdRefreshFlag && !forceRedraw) return;
lcdRefreshFlag = 0u;
forceRedraw = 0u;

switch (menuState)
{
    case MENU_HOME: renderHome(); break;
    case MENU_PAGE_LIST: renderPageList(); break;
    case MENU_OPTION_LIST: renderOptionList(0u); break;
    case MENU_EDIT: renderOptionList(1u); break;
    case MENU_THRESH_EDIT: renderThreshEdit(); break;
    case MENU_PARAM_SLOTS: renderParamSlots(); break;
    default: break;
}
}

void menuSetInitHealth(uint8_t failedMaskValue, uint8_t unverifiedMaskValue)
{
    menuInitFailedMask = failedMaskValue;
    menuInitUnverifiedMask = unverifiedMaskValue;
}

void menuGetVisionTuning(visionTuningStruct* out)
{
    if (out == NULL) return;
    *out = menuVisionTuning;
}

```

core0_cyt4bb7/project/code/app/menu.h

SHA-256

:

afb097316e1429d3ec02073fa44da3c7161c7669a522480b9bb135720433be26

代码 (c)

```

#ifndef _MENU_H_
#define _MENU_H_

#include <stdint.h>

/*
 * core0 cm_7_1 菜单 + 屏幕显示（独占 IPS114 + 5 按键 + flash）
 *
 * 职责拆分为五个 TU（共享状态/类型/原型见 menuInternal.h）：
 * menu.c      共享状态定义 + 公共 API + 状态机分发 + 开机 arming 门控
 * menuModel.c 参数默认/本地热重载/下发 cm_7_0 + 分页页表构建
 * menuPersistence.c flash blob/CRC/版本迁移 + 读写/擦除
 * menuRender.c 8×16 渲染（绘制原语 + 各状态画面）
 * menuInput.c 按键状态机 handler + 清屏/填充反馈/滚动窗口维护
 *
 * core1Loop.c 只做调度：
 * processInit → imageInit + menuInit
 * pitProcess0 → menuPitTick（PIT_CH2 10ms ISR：key_scanner + 计数。
 *      注意 PIT_CH0/CH1 是 cm_7_0 飞控独占的同片 TCPWM0 定时器，
 *      cm_7_1 必须用 PIT_CH2，复用 CH1 会跨核争用导致按键节拍失准）
 * whileProcess → menuServiceInput + imageProcess/publishVisionError + menuTick
 *
 * 按键映射：

```

* 发车页短按: K1 进菜单 / K2 标定(成功后自动持久化) / K3 进阈值编辑器 /
* K4 短:手动保存全部参数(绿屏反馈) 长:清空恢复默认(红屏反馈) / K5 短:发车 长(飞行态):回待机
* 发车页长按: K1 直达 Core1 / K2 直达 Core3 阈值页 / K3 直达 Core4 阈值页; 远端 K1 长按返回 Core0
* 页列表: K1 进入选项 / K2 下 / K3 上 / K4 回发车页 / K5 保存回发车页
* 选项列表:K1 进入编辑 / K2 下 / K3 上 / K4 回页列表 / K5 保存回发车页
* 编辑: K1 +大 / K2 +小 / K3 -小 / K4 -大 / K5 保存回选项列表
* 阈值编辑器: K1 +大 / K2 +小 / K3 -小 / K4 -大 / K5 短:切项(Lo/Hi/Exp) 长:保存回发车页
* 开机 ESC arming 期间显示进度条并忽略按键; 当前调试配置在飞行中保持屏显。
*/

/* 视觉误差调参 (菜单可调, 供 core1Loop publishVisionError 计算误差读取)。
* 与原 configVision.h 宏一一对应, 运行时可改 + flash 持久化。
* 仅在前台 whileProcess 上下文访问 (菜单编辑 + 误差发布同线程), 无需 volatile。 */
typedef struct

```
{  
    float targetCenterX; /* ^ 目标像素中点 X (0..MT9V03X_W-1) */  
    float targetCenterY; /* ^ 目标像素中点 Y (0..MT9V03X_H-1) */  
    float targetAngleRad; /* ^ 期望朝向参考角 rad */  
    float errXSign; /* errX 极性 (±1) */  
    float errYSign; /* errY 极性 (±1) */  
    float yawErrSign; /* yawErr 极性 (±1) */  
    float deadzoneEnterPct; /* 施密特大圆(进入,转高)直径占图像高%, 须 > exit */  
    float deadzoneExitPct; /* 施密特小圆(退出,归零)直径占图像高% */  
    float lostFreezeMs; /* 目标丢失后冻结最后输出误差的时间 ms */  
    float lostExtrapDiameterPx; /* 目标丢失外推圆直径 px (沿最后视向投影找回目标) */  
} visionTuningStruct;  
  
/* 生命周期 */  
void menuInit(void); /* ips114+key 初始化、加载默认/flash、推参数给 cm_7_0、握手 */  
void menuSetInitHealth(uint8_t failedMask, uint8_t unverifiedMask);  
void menuPitTick(void); /* PIT_CH2 10ms ISR: key_scanner + 刷新计数 (轻量) */  
void menuServiceInput(void); /* 前台前段: 门控 + 按键状态机; 不访问 IPS114/Flash */  
void menuTick(void); /* 前台尾段: Flash/反馈/渲染服务 */  
void menuGetVisionTuning(visionTuningStruct* out);  
  
#endif /* _MENU_H_ */
```

core0_cyt4bb7/project/code/app/menuFloatDisplayInternal.h

SHA-256 :
e5c06fecc348bcaa914fc246d7a0df4c1c07169fdbf396c61be6a168d0ec125f

代码 (c)

```
#ifndef MENU_FLOAT_DISPLAY_INTERNAL_H  
#define MENU_FLOAT_DISPLAY_INTERNAL_H  
  
#include <stdint.h>  
  
#include "zf_device_ips114.h"  
  
#define MENU_FLOAT_DISPLAY_DECIMALS_MAX (8u)  
#define MENU_FLOAT_DISPLAY_CHARS (15u)  
  
static uint8_t menuFloatDisplayDigitsU64(uint64_t value)  
{  
    uint8_t digits = 1u;  
  
    while (value >= 10u)  
    {  
        value /= 10u;  
        digits++;  
    }  
    return digits;  
}  
  
static void menuFloatDisplayShowLiteral(uint16_t x,  
    uint16_t y,  
    const char *literal)  
{  
    char buf[MENU_FLOAT_DISPLAY_CHARS + 1u];  
    uint8_t i;  
    uint8_t length = 0u;  
  
    while ((literal[length] != '\0') && (length < MENU_FLOAT_DISPLAY_CHARS))
```

```

    {
        length++;
    }
    for (i = 0u; i < MENU_FLOAT_DISPLAY_CHARS; i++)
    {
        buf[i] = ' ';
    }
    for (i = 0u; i < length; i++)
    {
        buf[MENU_FLOAT_DISPLAY_CHARS - length + i] = literal[i];
    }
    buf[MENU_FLOAT_DISPLAY_CHARS] = '\0';
    ips114_show_string(x, y, buf);
}

static uint8_t menuFloatDisplayDecimalsForStep(float step)
{
    uint8_t decimals = 0u;
    double scaled = (step < 0.0f) ? -(double) step : (double) step;

    if (!(scaled > 0.0))
    {
        return MENU_FLOAT_DISPLAY_DECIMALS_MAX;
    }
    while (decimals < MENU_FLOAT_DISPLAY_DECIMALS_MAX)
    {
        uint64_t rounded;
        double difference;

        if (!(scaled < (double) UINT64_MAX) ||
            !((scaled + 0.5) < (double) UINT64_MAX))
        {
            return MENU_FLOAT_DISPLAY_DECIMALS_MAX;
        }
        rounded = (uint64_t) (scaled + 0.5);
        difference = scaled - (double) rounded;

        if (difference < 0.0)
        {
            difference = -difference;
        }
        if (difference <= 0.000001)
        {
            break;
        }
        scaled *= 10.0;
        decimals++;
    }
    return decimals;
}

static uint8_t menuFloatDisplayDecimalsForValue(double value)
{
    uint8_t decimals;
    double scale = 1.0;
    double tolerance = 0.0000001;

    if (value < 0.0)
    {
        value = -value;
    }
    if (!(value >= 0.0) || !(value < (double) UINT64_MAX))
    {
        return MENU_FLOAT_DISPLAY_DECIMALS_MAX;
    }
    for (decimals = 0u; decimals < MENU_FLOAT_DISPLAY_DECIMALS_MAX; decimals++)
    {
        double scaled = value * scale;
        uint64_t rounded;
        double difference;

        if (!(scaled < (double) UINT64_MAX) ||
            !((scaled + 0.5) < (double) UINT64_MAX))
        {
            return MENU_FLOAT_DISPLAY_DECIMALS_MAX;
        }
        rounded = (uint64_t) (scaled + 0.5);
        difference = value - ((double) rounded / scale);
    }
}

```

```

        if (difference < 0.0)
        {
            difference = -difference;
        }
        if (difference <= tolerance)
        {
            return decimals;
        }
        scale *= 10.0;
    }
    return MENU_FLOAT_DISPLAY_DECIMALS_MAX;
}

static uint8_t menuFloatDisplayDecimalsForValueAndStep(double value, float step)
{
    uint8_t stepDecimals = menuFloatDisplayDecimalsForStep(step);
    uint8_t valueDecimals = menuFloatDisplayDecimalsForValue(value);

    return (stepDecimals > valueDecimals) ? stepDecimals : valueDecimals;
}

static void menuFloatDisplayShow(uint16_t x,
    uint16_t y,
    double value,
    uint8_t requestedDecimals)
{
    static const uint64_t scaleTable[MENU_FLOAT_DISPLAY_DECIMALS_MAX + 1u] =
    {
        1u, 10u, 100u, 1000u, 10 000u,
        100 000u, 1 000 000u, 10 000 000u, 100 000 000u
    };
    char buf[MENU_FLOAT_DISPLAY_CHARS + 1u];
    uint8_t decimals;
    uint8_t selectedDecimals = 0xFFu;
    uint8_t negative = 0u;
    uint8_t pos;
    uint8_t i;
    uint64_t scale = 1u;
    uint64_t scaled = 0u;
    uint64_t intPart;
    uint64_t fracPart;

    if (value != value)
    {
        menuFloatDisplayShowLiteral(x, y, "NaN");
        return;
    }
    if (value < 0.0)
    {
        negative = 1u;
        value = -value;
    }

    decimals = (requestedDecimals <= MENU_FLOAT_DISPLAY_DECIMALS_MAX)
        ? requestedDecimals
        : MENU_FLOAT_DISPLAY_DECIMALS_MAX;
    for (; decimals-->)
    {
        double scaledDouble;
        double roundedDouble;
        uint8_t required;

        scale = scaleTable[decimals];
        scaledDouble = value * (double) scale;
        roundedDouble = scaledDouble + 0.5;
        if ((scaledDouble < (double) UINT64_MAX) &&
            (roundedDouble < (double) UINT64_MAX))
        {
            scaled = (uint64_t) roundedDouble;
            intPart = scaled / scale;
            required = (uint8_t) (negative + menuFloatDisplayDigitsU64(intPart));
            if (decimals != 0u)
            {
                required = (uint8_t) (required + 1u + decimals);
            }
            if (required <= MENU_FLOAT_DISPLAY_CHARS)
            {
                selectedDecimals = decimals;
                break;
            }
        }
    }
}

```

```

    }
}
if (decimals == 0u)
{
    break;
}
}
if (selectedDecimals == 0xFFu)
{
    menuFloatDisplayShowLiteral(x, y, "OVERFLOW");
    return;
}

for (i = 0u; i < MENU_FLOAT_DISPLAY_CHARS; i++)
{
    buf[i] = ' ';
}
buf[MENU_FLOAT_DISPLAY_CHARS] = '\0';
intPart = scaled / scale;
fracPart = scaled % scale;
pos = MENU_FLOAT_DISPLAY_CHARS;
for (i = 0u; i < selectedDecimals; i++)
{
    buf[--pos] = (char) ('0' + (fracPart % 10u));
    fracPart /= 10u;
}
if (selectedDecimals != 0u)
{
    buf[--pos] = '.';
}
do
{
    buf[--pos] = (char) ('0' + (intPart % 10u));
    intPart /= 10u;
} while ((intPart != 0u) && (pos > 0u));
if ((negative != 0u) && (pos > 0u))
{
    buf[--pos] = '-';
}

ips114_show_string(x, y, buf);
}

#endif

```

core0_cyt4bb7/project/code/app/menuInput.c

SHA-256

:

7cf6805b3fba0819cdc91d6cc434c2492bb1f881f10388c583c5a95fa6dda1f6

代码 (c)

```

#include "menu.h"
#include "menuInternal.h"
#include "menuParamSlots.h"
#include "paramSyncOwnerService.h"

#include <string.h>

#include "menuKey.h"
#include "zf_common_font.h"

#include "boardConfig.h" /* GLOBAL_PARAM_PERSIST_ENABLE 编译期开关 */
#include "configVision.h"
#include "image.h"

/*
 * menuInput.c — 按键状态机 handler（按 menuState 分发）+ 屏幕清屏/填充反馈/滚动窗口维护。
 * 编辑即时生效：视觉项调 applyVisionLocal(), 飞控项 applyAfterEdit() 推 cm_7_0（见 menuModel.c）。
 */

/* =====
 * 滚动窗口维护
 * ===== */
static void clampSelAndScroll(uint8_t count)

```

```

{
    if (count == 0u)
    {
        optIdx = 0;
        scrollTop = 0u;
        return;
    }
    if (optIdx < 0) optIdx = (int8_t) (count - 1u);
    else if (optIdx >= (int8_t) count) optIdx = 0;

    if ((uint8_t) optIdx < scrollTop) scrollTop = (uint8_t) optIdx;
    else if ((uint8_t) optIdx >= scrollTop + VISIBLE_ROWS)
        scrollTop = (uint8_t) ((uint8_t) optIdx - VISIBLE_ROWS + 1u);
}

static void pageListMove(int8_t delta);
static void menuFlashFeedback(uint16_t color);
static uint8_t remoteStatusHome(void);
static uint8_t remoteStatusLaunchHome(void);

static void discardAllPhysicalKeysUntilRelease(void)
{
    uint8_t k;
    for (k = 0u; k < (uint8_t) MENU_KEY_COUNT; k++)
    {
        menuKeyDiscardUntilRelease((menuKeyIndexEnum) k);
    }
}

#define REMOTE_DEFERRED_KEY_MAX_AGE_MS (600u)

static void clearDeferredRemoteKey(void)
{
    remoteDeferredKeyValid = 0u;
    remoteDeferredKeyId = 0u;
    remoteDeferredKeyState = 0u;
    remoteDeferredKeyTickMs = 0u;
}

static void captureOneSafeRemoteKeyWhilePending(void)
{
    uint8_t k;

    /* 只允许一个同路由菜单短按跨 ACK 窗口。K5 可能触发保存或发车，始终禁止
     * 排队；K4 仅在非 HOME 页暂存，并在发送前再次核对页面仍未回 HOME。*/
    if (remoteDeferredKeyValid == 0u)
    {
        for (k = 0u; k <= (uint8_t) MENU_KEY_4; k++)
        {
            if ((menuKeyGetState((menuKeyIndexEnum) k) == MENU_KEY_SHORT_PRESS) &&
                !((k == (uint8_t) MENU_KEY_4) &&
                  (remoteStatusLaunchHome() != 0u)))
            {
                remoteDeferredKeyValid = 1u;
                remoteDeferredKeyId = k;
                remoteDeferredKeyState = (uint8_t) MENU_KEY_SHORT_PRESS;
                remoteDeferredKeyTickMs = menuTickMs;
                break;
            }
        }
    }
    discardAllPhysicalKeysUntilRelease();
}

static void selectKvmTarget(uint8_t target, uint8_t enterThresholdAfterOwner)
{
    menuKeyTargetCore = target;
    menuKvmRouteEpoch++;
    if (menuKvmRouteEpoch == 0u) menuKvmRouteEpoch = 1u;
    remoteKeySeq = 0u;
    remoteKeyPending = 0u;
    remoteRouteRestartPending = 0u;
    remoteEnterThresholdPending = enterThresholdAfterOwner ? 1u : 0u;
    clearDeferredRemoteKey();
    menuState = MENU_HOME;
    discardAllPhysicalKeysUntilRelease();
    menuPushParams();
    clearScreenForState();
}

```

```

static void restartRemoteRoute(void)
{
    /* 目标核是操作员意图，协议重同步只能重建同一目标的租约，不能擅自改回 Core0。 */
    menuKvmRouteEpoch++;
    if (menuKvmRouteEpoch == 0u) menuKvmRouteEpoch = 1u;
    remoteKeySeq = 0u;
    remoteKeyPending = 0u;
    remoteRouteRestartPending = 0u;
    clearDeferredRemoteKey();
    discardAllPhysicalKeysUntilRelease();
    menuPushParams();
    forceRedraw = 1u;
}

static uint8_t remoteStatusFresh(void)
{
    uint8_t selectedValid;
    if ((kvmRx.magic != CORE_DATA_READY_MAGIC) ||
        (kvmRx.routeSession != bootEpoch) ||
        (kvmRx.routeEpoch != menuKvmRouteEpoch) ||
        (kvmRx.target != menuKeyTargetCore) ||
        (kvmRx.activeTarget != menuKeyTargetCore) ||
        ((kvmRx.flags & UART_LINK_KVM_FLAG_PEER_RESTART) != 0u) ||
        ((kvmRx.flags & (UART_LINK_KVM_FLAG_ROUTE_ACCEPTED | UART_LINK_KVM_FLAG_LEASE_VALID)) !=
         (UART_LINK_KVM_FLAG_ROUTE_ACCEPTED | UART_LINK_KVM_FLAG_LEASE_VALID)))
    {
        return 0u;
    }
    /* Core4 当前 STATUS 无独立菜单 VALID 位；route accepted + lease + activeTarget
     * 已构成 owner 建立事实，也是自动进入阈值页的触发边界。 */
    if (menuKeyTargetCore == UART_LINK_KVM_TARGET_CORE4) return 1u;
    selectedValid = (menuKeyTargetCore == UART_LINK_KVM_TARGET_CORE1)
        ? UART_LINK_KVM_FLAG_CORE1_VALID : UART_LINK_KVM_FLAG_CORE3_VALID;
    return (uint8_t) ((kvmRx.flags & selectedValid) != 0u);
}

static uint8_t remoteStatusHome(void)
{
    uint8_t homeFlag = (menuKeyTargetCore == UART_LINK_KVM_TARGET_CORE1)
        ? UART_LINK_KVM_FLAG_CORE1_HOME : UART_LINK_KVM_FLAG_CORE3_HOME;
    return (uint8_t) ((kvmRx.flags & homeFlag) != 0u);
}

static uint8_t remoteStatusPeerRestarted(void)
{
    return (uint8_t) ((kvmRx.magic == CORE_DATA_READY_MAGIC) &&
        (kvmRx.routeSession == bootEpoch) &&
        (kvmRx.routeEpoch == menuKvmRouteEpoch) &&
        (kvmRx.target == menuKeyTargetCore) &&
        ((kvmRx.flags & UART_LINK_KVM_FLAG_PEER_RESTART) != 0u));
}

void clearScreenForState(void)
{
    menuRequestScreenClear();
}

static void scheduleParamSlotInspect(void);

static void enterPageList(uint8_t resetSelection)
{
    menuState = MENU_PAGE_LIST;
    if (resetSelection)
    {
        pageIndex = 0;
        scrollTop = 0u;
    }
    else
    {
        scrollTop = 0u;
        pageListMove(0);
    }
    clearScreenForState();
}

static void enterOptionList(void)
{
    if ((pageIndex >= 0) && ((uint8_t) pageIndex == paramSlotPageIndex))

```



```

{
    if (selectedParamSlot < 1u || selectedParamSlot > MENU_PARAM_SLOT_COUNT)
        selectedParamSlot = 1u;
    menuState = MENU_PARAM_SLOTS;
    scheduleParamSlotInspect();
    clearScreenForState();
    return;
}
if (pageIdx < 0 || (uint8_t) pageIdx >= menuPageCount
    || menuPages[pageIdx].optionCount == 0u)
{
    forceRedraw = 1u;
    return;
}
menuState = MENU_OPTION_LIST;
optIdx = 0;
scrollTop = 0u;
clearScreenForState();
}

static uint8_t remoteStatusLaunchHome(void)
{
    if (menuKeyTargetCore == UART_LINK_KVM_TARGET_CORE4) return 0u;
    return remoteStatusHome();
}

static uint32_t paramGlobalRequestSequence;
static uint32_t pendingParamGlobalCoordinatorBoot;
static paramSyncGlobalRequest pendingParamGlobalRequest;
static menuParamGlobalUiSnapshot paramGlobalUi;
static uint8_t pendingParamGlobalRequestValid;
static uint8_t pendingParamSlotInspect;
static uint8_t requestGlobalParamAction(uint8_t action, uint8_t slot);

static void scheduleParamSlotInspect(void)
{
    pendingParamSlotInspect = 1u;
    paramGlobalUi.slotUiState = PARAM_SYNC_SLOT_UI_CHECKING;
    forceRedraw = 1u;
}

void menuParamGlobalReset(void)
{
    memset(&pendingParamGlobalRequest, 0, sizeof(pendingParamGlobalRequest));
    memset(&paramGlobalUi, 0, sizeof(paramGlobalUi));
    paramGlobalUi.target = 0xFFu;
    pendingParamGlobalRequestValid = 0u;
    pendingParamSlotInspect = 0u;
    pendingParamGlobalCoordinatorBoot = 0u;
    paramGlobalUi.slotUiState = PARAM_SYNC_SLOT_UI_UNKNOWN;
}

void menuParamGlobalGetUi(menuParamGlobalUiSnapshot *snapshot)
{
    if (snapshot != 0) *snapshot = paramGlobalUi;
}

void menuParamGlobalPoll(void)
{
    paramSyncGlobalState state;
    uint8_t nextUi;
    if (pendingParamGlobalRequestValid == 0u)
    {
        if (pendingParamSlotInspect != 0u &&
            coreDataTryReadParamGlobalState(&state) != 0u &&
            paramSyncGlobalStateReady(&state) != 0u &&
            requestGlobalParamAction(PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY,
                selectedParamSlot) != 0u)
            pendingParamSlotInspect = 0u;
        return;
    }
    if (coreDataTryReadParamGlobalState(&state) == 0u) return;
    if (state.coordinatorBootGeneration != pendingParamGlobalCoordinatorBoot)
    {
        nextUi = MENU_PARAM_GLOBAL_FAILED;
        state.phase = PARAM_SYNC_GLOBAL_PHASE_FAILED;
        state.target = 0u;
        state.error = PARAM_SYNC_GLOBAL_ERROR_PEER_RESTART;
        pendingParamGlobalRequestValid = 0u;
    }
}

```

```

    }
    else if (paramSyncGlobalStateMatchesRequest(&state,
        &pendingParamGlobalRequest) != 0u &&
        (state.flags & PARAM_SYNC_GLOBAL_FLAG_TERMINAL) != 0u)
    {
        uint8_t completedAction = pendingParamGlobalRequest.action;
        uint8_t completedSlot = pendingParamGlobalRequest.slot;
        nextUi = ((state.flags & PARAM_SYNC_GLOBAL_FLAG_OK) != 0u) ?
            MENU_PARAM_GLOBAL_SUCCESS : MENU_PARAM_GLOBAL_FAILED;
        paramGlobalUi.action = completedAction;
        paramGlobalUi.slot = completedSlot;
        if (completedAction == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
        {
            if (completedSlot == selectedParamSlot)
                paramGlobalUi.slotUiState = (nextUi == MENU_PARAM_GLOBAL_SUCCESS) ?
                    paramSyncGlobalClassifySlotSummary(state.transactionId) :
                    PARAM_SYNC_SLOT_UI_UNAVAILABLE;
        }
        else if (nextUi == MENU_PARAM_GLOBAL_SUCCESS)
        {
            menuRequestParamFeedback((completedAction ==
                PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY) ? RGB565_GREEN :
                RGB565_RED, completedAction, completedSlot);
            pendingParamSlotInspect = 1u;
        }
        pendingParamGlobalRequestValid = 0u;
    }
    else if (paramSyncGlobalStateMatchesRequest(&state,
        &pendingParamGlobalRequest) != 0u &&
        (state.flags & PARAM_SYNC_GLOBAL_FLAG_BUSY) != 0u)
        nextUi = MENU_PARAM_GLOBAL_PROGRESS;
    else if (paramSyncGlobalStateReady(&state) == 0u)
    {
        nextUi = MENU_PARAM_GLOBAL_FAILED;
        state.phase = PARAM_SYNC_GLOBAL_PHASE_FAILED;
        state.target = 0u;
        state.error = ((state.flags & PARAM_SYNC_GLOBAL_FLAG_RECOVERY) != 0u) ?
            PARAM_SYNC_GLOBAL_ERROR_RECOVERY :
            (((state.flags & PARAM_SYNC_GLOBAL_FLAG_BUSY) != 0u) ?
                PARAM_SYNC_GLOBAL_ERROR_BUSY : PARAM_SYNC_GLOBAL_ERROR_ROUTE);
        pendingParamGlobalRequestValid = 0u;
    }
    }
    else
        return;
    if (paramGlobalUi.state != nextUi || paramGlobalUi.phase != state.phase ||
        paramGlobalUi.target != state.target || paramGlobalUi.error != state.error)
    {
        paramGlobalUi.state = nextUi;
        paramGlobalUi.phase = state.phase;
        paramGlobalUi.target = state.target;
        paramGlobalUi.error = state.error;
        forceRedraw = 1u;
    }
}

static uint8_t requestGlobalParamAction(uint8_t action, uint8_t slot)
{
    paramSyncGlobalState state;
    paramSyncGlobalRequest request;
    if (paramSyncOwnerServiceMutationLocked() != 0u ||
        pendingParamGlobalRequestValid != 0u ||
        coreDataTryReadParamGlobalState(&state) == 0u ||
        paramSyncGlobalStateReady(&state) == 0u) return 0u;
    paramGlobalRequestSequence++;
    if (paramGlobalRequestSequence == 0u) paramGlobalRequestSequence = 1u;
    request.producerBootGeneration = bootEpoch != 0u ? bootEpoch : 1u;
    request.sequence = paramGlobalRequestSequence;
    request.origin = 0u;
    request.action = action;
    request.slot = slot;
    request.reserved = 0u;
    if (paramSyncGlobalRequestIsValid(&request) == 0u) return 0u;
    coreDataWriteParamGlobalRequest(&request);
    pendingParamGlobalRequest = request;
    pendingParamGlobalCoordinatorBoot = state.coordinatorBootGeneration;
    pendingParamGlobalRequestValid = 1u;
    paramGlobalUi.state = MENU_PARAM_GLOBAL_QUEUED;
    paramGlobalUi.phase = PARAM_SYNC_GLOBAL_PHASE_REQUESTED;
    paramGlobalUi.target = 0xFFu;
}

```

```

paramGlobalUi.error = PARAM_SYNC_GLOBAL_ERROR_NONE;
paramGlobalUi.action = action;
paramGlobalUi.slot = slot;
if (action == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
    paramGlobalUi.slotUiState = PARAM_SYNC_SLOT_UI_CHECKING;
forceRedraw = 1u;
return 1u;
}

void handleParamSlotKeys(void)
{
    uint8_t ok;
    if (menuKeyGetState(MENU_KEY_5) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_5);
        enterPageList(0u);
        return;
    }
    if (menuKeyGetState(MENU_KEY_2) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_2);
        selectedParamSlot = (selectedParamSlot <= 1u) ?
            MENU_PARAM_SLOT_COUNT : (uint8_t) (selectedParamSlot - 1u);
        scheduleParamSlotInspect();
        return;
    }
    if (menuKeyGetState(MENU_KEY_3) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_3);
        selectedParamSlot = (selectedParamSlot >= MENU_PARAM_SLOT_COUNT) ?
            1u : (uint8_t) (selectedParamSlot + 1u);
        scheduleParamSlotInspect();
        return;
    }
    if (menuKeyGetState(MENU_KEY_4) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_4);
        if (paramSyncOwnerServiceMutationLocked() != 0u ||
            paramGlobalUi.slotUiState != PARAM_SYNC_SLOT_UI_FULL) return;
        ok = requestGlobalParamAction(
            PARAM_SYNC_GLOBAL_ACTION_LOAD_RUNTIME, selectedParamSlot);
        (void) ok;
        return;
    }
    if (menuKeyGetState(MENU_KEY_1) == MENU_KEY_LONG_PRESS)
    {
        menuKeyClear(MENU_KEY_1);
        if (paramSyncOwnerServiceMutationLocked() != 0u) return;
        ok = requestGlobalParamAction(
            PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY, selectedParamSlot);
        (void) ok;
    }
}

/* 长按/保存操作请求循环尾整屏反馈，让操作员确认按键被成功触发。
* menuTick 应用请求并设置 feedbackHoldTicks，在 10ms PIT 倒计时数百 ms 后清屏重绘。
* color: 保存/确认用绿，清空/危险用红。MENU_FEEDBACK_HOLD_TICKS 见 menuInternal.h。*/
static void menuFlashFeedback(uint16_t color)
{
    menuRequestFeedback(color);
}

static void requestLaunchFromHome(void)
{
    if (flightSnapshotAvailable == 0u)
    {
        menuFlashFeedback(RGB565_RED);
        return;
    }
#ifdef INTER_CORE_LINK_GUARD_ENABLE
    if ((flightRx.core1LinkUp == 0u) || (flightRx.core2LinkLost != 0u))
    {
        menuFlashFeedback(RGB565_RED);
        return;
    }
#endif
    seqLaunch++;
    menuPushParams();
}

```

```

uint8_t handleFlightStandbyKey(void)
{
    /* 远程路由时五键全部属于目标菜单；K5 长按待机只在 Core0 本地目标生效。
    * 操作员仍可先 K1 长按切回 Core0，再执行待机，避免吞掉远端菜单的保存键。 */
    if ((menuKeyTargetCore != UART_LINK_KVM_TARGET_CORE0) ||
        (flightRx.safetyState != FC_STATE_FLYING))
    {
        return 0u;
    }
    if (menuKeyGetState(MENU_KEY_5) != MENU_KEY_LONG_PRESS)
    {
        return 0u;
    }

    menuKeyClear(MENU_KEY_5);
    menuKeyTargetCore = UART_LINK_KVM_TARGET_CORE0;
    menuKvmRouteEpoch++;
    if (menuKvmRouteEpoch == 0u) menuKvmRouteEpoch = 1u;
    remoteKeyPending = 0u;
    remoteRouteRestartPending = 0u;
    remoteEnterThresholdPending = 0u;
    menuState = MENU_HOME;
    seqStandby++;
    menuPushParams();
    menuFlashFeedback(RGB565_RED);
    return 1u;
}

uint8_t handleFaultReasonAck(void)
{
    menuKeyStateEnum k5State;
    if ((flightRx.cm70Ready != CORE_DATA_READY_MAGIC) ||
        (menuFaultDisplayReason == CORE0_FAULT_REASON_NONE))
    {
        return 0u;
    }
    k5State = menuKeyGetState(MENU_KEY_5);
    if ((k5State == MENU_KEY_SHORT_PRESS) || (k5State == MENU_KEY_LONG_PRESS))
    {
        menuKeyClear(MENU_KEY_5);
        menuKeyTargetCore = UART_LINK_KVM_TARGET_CORE0;
        menuKvmRouteEpoch++;
        if (menuKvmRouteEpoch == 0u) menuKvmRouteEpoch = 1u;
        remoteKeyPending = 0u;
        remoteRouteRestartPending = 0u;
        remoteEnterThresholdPending = 0u;
        menuState = MENU_HOME;
        seqLaunch++;
        menuFaultAckPending = 1u;
        menuFaultPageActive = 0u;
        menuPushParams();
        clearScreenForState();
        /* K5 短按在物理释放时形成；ACK 成功后再丢弃所有残留键，并要求各键经过
        * 真实释放才可在 HOME 重新计时，保证本次确认绝不穿透成发车/KVM 动作。 */
        discardAllPhysicalKeysUntilRelease();
    }
    /* 未形成 K5 SHORT/LONG 时不得清扫扫描状态。短按依赖按下累计后在释放沿生成；
    * 长按达到 1s 即确认，避免 LONG 锁存后无法再形成 SHORT。 */
    return 1u;
}

void handleHomeKeys(void)
{
    if (menuKeyGetState(MENU_KEY_5) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_5);
        requestLaunchFromHome();
        return;
    }
    if (paramSyncOwnerServiceMutationLocked() != 0u)
    {
        uint8_t key;
        if (menuKeyGetState(MENU_KEY_1) == MENU_KEY_SHORT_PRESS)
        {
            menuKeyClear(MENU_KEY_1);
            enterPageList(1u);
            return;
        }
    }
}

```

```

    for (key = 0u; key < (uint8_t) MENU_KEY_COUNT; key++)
    {
        if (menuKeyGetState((menuKeyIndexEnum) key) != MENU_KEY_RELEASE)
            menuKeyDiscardUntilRelease((menuKeyIndexEnum) key);
    }
    return;
}
/* HOME 长按直达：K1=Core1；K2=Core3 阈值；K3=Core4 阈值。*/
if (menuKeyGetState(MENU_KEY_1) == MENU_KEY_LONG_PRESS)
{
    selectKvmTarget(UART_LINK_KVM_TARGET_CORE1, 0u);
    return;
}
if (menuKeyGetState(MENU_KEY_2) == MENU_KEY_LONG_PRESS)
{
    selectKvmTarget(UART_LINK_KVM_TARGET_CORE3, 1u);
    return;
}
if (menuKeyGetState(MENU_KEY_3) == MENU_KEY_LONG_PRESS)
{
    selectKvmTarget(UART_LINK_KVM_TARGET_CORE4, 1u);
    return;
}
if (menuKeyGetState(MENU_KEY_1) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_1);
    enterPageList(1u);
}
else if (menuKeyGetState(MENU_KEY_2) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_2);
    imageCalibTrigger();
}
else if (menuKeyGetState(MENU_KEY_3) == MENU_KEY_SHORT_PRESS)
{
    /* K3 进入阈值编辑器 */
    menuKeyClear(MENU_KEY_3);
    menuState = MENU_THRESH_EDIT;
    threshSel = 0u;
    clearScreenForState();
}
else if (menuKeyGetState(MENU_KEY_4) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_4);
    menuRequestSaveSettings();
    menuFlashFeedback(RGB565_GREEN); /* 保存确认：循环尾绿屏保持数帧后自动重绘 */
}
else if (menuKeyGetState(MENU_KEY_4) == MENU_KEY_LONG_PRESS)
{
    menuKeyClear(MENU_KEY_4);
    menuRequestRestoreDefaults();
    menuFlashFeedback(RGB565_RED); /* 清空恢复：红屏保持数帧后自动重绘 */
}
}

/* 远端目标一次只允许一个在途事件；ACK 前禁止覆盖。*/
void handleRemoteForward(void)
{
    uint8_t fresh;
    uint8_t ackClosedThisPass = 0u;
    uint8_t restartRequired = remoteRouteRestartPending;

    /* 参数事务锁必须先于重绑、ACK 等早退路径撤销旧菜单意图；
    * HOME 发车仍在后续专用分支保持正交。*/
    if (paramSyncOwnerServiceMutationLocked() != 0u)
    {
        remoteEnterThresholdPending = 0u;
        clearDeferredRemoteKey();
    }

    if (remoteStatusPeerRestarted() != 0u)
    {
        remoteRouteRestartPending = 1u;
        restartRequired = 1u;
    }

    if ((remoteKeyPending != 0u) &&
        (kvmRx.routeSession == bootEpoch) &&
        (kvmRx.routeEpoch == menuKvmRouteEpoch) &&

```

```

(kvmRx.ackEventSeq == remoteKeySeq) &&
((kvmRx.flags & UART_LINK_KVM_FLAG_ACK_TERMINAL) != 0u))
{
    remoteKeyPending = 0u;
    ackClosedThisPass = 1u;
    menuPushParams();
    if ((kvmRx.ackResult == UART_LINK_KVM_ACK_PEER_RESTART) ||
        (kvmRx.ackResult == UART_LINK_KVM_ACK_NO_LEASE) ||
        (kvmRx.ackResult == UART_LINK_KVM_ACK_WRONG_TARGET) ||
        (kvmRx.ackResult == UART_LINK_KVM_ACK_STALE_ROUTE) ||
        (kvmRx.ackResult == UART_LINK_KVM_ACK_GLOBAL_EVENT_REJECTED) ||
        (kvmRx.ackResult == UART_LINK_KVM_ACK_GAP) ||
        (kvmRx.ackResult == UART_LINK_KVM_ACK_OLD_EVENT) ||
        (kvmRx.ackResult == UART_LINK_KVM_ACK_BAD_PAYLOAD))
    {
        remoteRouteRestartPending = 1u;
        restartRequired = 1u;
    }
}
/* HOME 的 K1 长按是操作员退出远程路由的本地逃生动作。 */
if (menuKeyGetState(MENU_KEY_1) == MENU_KEY_LONG_PRESS &&
    remoteStatusHome())
{
    selectKvmTarget(UART_LINK_KVM_TARGET_CORE0, 0u);
    return;
}
if (restartRequired != 0u)
{
    restartRemoteRoute();
    return;
}
fresh = remoteStatusFresh();
if (fresh == 0u)
{
    /* 资格失效时不保留尚未发送的第二事件，避免断链后陈旧菜单动作重放。 */
    clearDeferredRemoteKey();
    discardAllPhysicalKeysUntilRelease();
    return;
}
if (ackClosedThisPass != 0u)
{
    /* ACK 与实体事件同拍时无法证明事件发生在 ACK 之后，只截取一个允许
     * 延后的菜单短按；HOME 发车与全部长按仍不穿透安全边界。 */
    captureOneSafeRemoteKeyWhilePending();
    return;
}
if (remoteKeyPending != 0u)
{
    captureOneSafeRemoteKeyWhilePending();
    return;
}
if (paramSyncOwnerServiceMutationLocked() != 0u)
{
    remoteEnterThresholdPending = 0u;
    clearDeferredRemoteKey();
    if ((menuKeyGetState(MENU_KEY_5) == MENU_KEY_SHORT_PRESS) &&
        remoteStatusLaunchHome())
    {
        menuKeyClear(MENU_KEY_5);
        requestLaunchFromHome();
        return;
    }
    discardAllPhysicalKeysUntilRelease();
    return;
}
if (remoteEnterThresholdPending != 0u)
{
    /* 复用目标菜单既有 K3 短按入口，并沿用 eventSeq/在途锁/终态 ACK。 */
    remoteEnterThresholdPending = 0u;
    remoteKeyId = (uint8_t) MENU_KEY_3;
    remoteKeyState = (uint8_t) MENU_KEY_SHORT_PRESS;
    remoteKeySeq++;
    if (remoteKeySeq == 0u) remoteKeySeq = 1u;
    remoteKeyPending = 1u;
    discardAllPhysicalKeysUntilRelease();
    menuPushParams();
    return;
}
if ((menuKeyGetState(MENU_KEY_5) == MENU_KEY_SHORT_PRESS) &&

```

```

        remoteStatusLaunchHome()
    {
        menuKeyClear(MENU_KEY_5);
        requestLaunchFromHome();
        return;
    }
    if (remoteDeferredKeyValid != 0u)
    {
        if ((remoteDeferredKeyId == (uint8_t) MENU_KEY_4) &&
            (remoteStatusLaunchHome() != 0u))
        {
            clearDeferredRemoteKey();
        }
        if ((uint32_t) (menuTickMs - remoteDeferredKeyTickMs) <=
            REMOTE_DEFERRED_KEY_MAX_AGE_MS &&
            (remoteDeferredKeyValid != 0u))
        {
            remoteKeyId = remoteDeferredKeyId;
            remoteKeyState = remoteDeferredKeyState;
            clearDeferredRemoteKey();
            remoteKeySeq++;
            if (remoteKeySeq == 0u) remoteKeySeq = 1u;
            remoteKeyPending = 1u;
            menuPushParams();
            return;
        }
        clearDeferredRemoteKey();
    }
}

for (uint8_t k = 0u; k < (uint8_t) MENU_KEY_COUNT; k++)
{
    menuKeyStateEnum st = menuKeyGetState((menuKeyIndexEnum) k);
    if (st == MENU_KEY_SHORT_PRESS || st == MENU_KEY_LONG_PRESS)
    {
        menuKeyClear((menuKeyIndexEnum) k);
        remoteKeyId = k;
        remoteKeyState = (uint8_t) st;
        remoteKeySeq++;
        if (remoteKeySeq == 0u) remoteKeySeq = 1u;
        remoteKeyPending = 1u;
        menuPushParams();
        break;
    }
}
}

static void pageListMove(int8_t delta)
{
    if (menuPageCount == 0u)
    {
        pageIndex = 0;
        scrollTop = 0u;
        return;
    }
    pageIndex = (int8_t) (pageIndex + delta);
    if (pageIndex < 0) pageIndex = (int8_t) (menuPageCount - 1u);
    else if (pageIndex >= (int8_t) menuPageCount) pageIndex = 0;

    if ((uint8_t) pageIndex < scrollTop) scrollTop = (uint8_t) pageIndex;
    else if ((uint8_t) pageIndex >= scrollTop + VISIBLE_ROWS)
        scrollTop = (uint8_t) ((uint8_t) pageIndex - VISIBLE_ROWS + 1u);
    forceRedraw = 1u;
}

void handlePageListKeys(void)
{
    if (menuKeyGetState(MENU_KEY_1) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_1);
        enterOptionList();
    }
    else if (menuKeyGetState(MENU_KEY_2) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_2);
        pageListMove(+1);
    }
    else if (menuKeyGetState(MENU_KEY_3) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_3);
    }
}

```

```

        pageListMove(-1);
    }
    else if (menuKeyGetState(MENU_KEY_4) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_4);
        menuState = MENU_HOME;
        clearScreenForState();
    }
    else if (menuKeyGetState(MENU_KEY_5) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_5);
        if (paramSyncOwnerServiceMutationLocked() == 0u)
        {
            menuRequestSaveSettings();
            menuState = MENU_HOME;
            menuFlashFeedback(RGB565_GREEN); /* 保持数帧后自动清屏重绘发车页 */
        }
    }
}

void handleOptionListKeys(void)
{
    menuPageStruct* pg;

    if (pageIdx < 0 || (uint8_t) pageIdx >= menuPageCount)
    {
        enterPageList(0u);
        return;
    }
    pg = &menuPages[pageIdx];

    if (menuKeyGetState(MENU_KEY_1) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_1);
        if (paramSyncOwnerServiceMutationLocked() == 0u &&
            pg->optionCount > 0u && optIdx >= 0 && optIdx < (int8_t) pg->optionCount
            && pg->opt[optIdx] != 0)
        {
            menuState = MENU_EDIT;
            forceRedraw = 1u;
        }
    }
    else if (menuKeyGetState(MENU_KEY_2) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_2);
        optIdx++;
        clampSelAndScroll(pg->optionCount);
        forceRedraw = 1u;
    }
    else if (menuKeyGetState(MENU_KEY_3) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_3);
        optIdx--;
        clampSelAndScroll(pg->optionCount);
        forceRedraw = 1u;
    }
    else if (menuKeyGetState(MENU_KEY_4) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_4);
        enterPageList(0u);
    }
    else if (menuKeyGetState(MENU_KEY_5) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_5);
        if (paramSyncOwnerServiceMutationLocked() == 0u)
        {
            menuRequestSaveSettings();
            menuState = MENU_HOME;
            menuFlashFeedback(RGB565_GREEN); /* 保持数帧后自动清屏重绘发车页 */
        }
    }
}

void handleEditKeys(void)
{
    menuPageStruct* pg;
    float delta = 0.0f;
    uint8_t hasDelta = 0u;

```



```

if (pageIndex < 0 || (uint8_t) pageIndex >= menuPageCount)
{
    enterPageList(0u);
    return;
}
pg = &menuPages[pageIndex];

if (paramSyncOwnerServiceMutationLocked() != 0u)
{
    menuState = MENU_OPTION_LIST;
    clearScreenForState();
    return;
}

if (optIdx < 0 || optIdx >= (int8_t) pg->optionCount || pg->opt[optIdx] == 0)
{
    menuState = MENU_OPTION_LIST;
    clearScreenForState();
    return;
}

/* K1 +大 / K2 +小 / K3 -小 / K4 -大 */
if (menuKeyGetState(MENU_KEY_1) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_1);
    delta = pg->stepL[optIdx];
    hasDelta = 1u;
}
else if (menuKeyGetState(MENU_KEY_2) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_2);
    delta = pg->stepS[optIdx];
    hasDelta = 1u;
}
else if (menuKeyGetState(MENU_KEY_3) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_3);
    delta = -pg->stepS[optIdx];
    hasDelta = 1u;
}
else if (menuKeyGetState(MENU_KEY_4) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_4);
    delta = -pg->stepL[optIdx];
    hasDelta = 1u;
}
else if (menuKeyGetState(MENU_KEY_5) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_5);
    menuState = MENU_OPTION_LIST;
    clearScreenForState();
    return;
}

if (hasDelta)
{
    #if GLOBAL_PARAM_PERSIST_ENABLE
        *pg->opt[optIdx] += delta;
        applyAfterEdit(pg->isVision[optIdx]);
        forceRedraw = 1u;
    #else
        /* 持久化关闭：参数锁定，改值键已被消费但不改值、不下发。 */
        (void) delta;
    #endif
}

/* =====
* 检测阈值编辑器按键（与编辑页加减逻辑一致）：
* K1 +大 / K2 +小 / K3 -小 / K4 -大（作用于当前 threshSel 项 Lo/Hi/Exp）
* K5 短按 切项(Lo→Hi→Exp 循环) / K5 长按 保存回发车页
* 每次调值即 applyVisionLocal() 热重载，下一帧两图即更新。
* 步进：Lo/Hi 按 DetectMode 选择 BinaryCC 或 Legacy Canny；
* Exposure 大=VISION_EXPOSURE_STEP_LONG/小=VISION_EXPOSURE_STEP_SHORT。
* ===== */
void handleThreshEditKeys(void)
{
    float stepL;
    float stepS;

```

```

float delta = 0.0f;

if (paramSyncOwnerServiceMutationLocked() != 0u)
{
    menuState = MENU_HOME;
    clearScreenForState();
    return;
}

/* K5 长按：保存回发车页（绿屏反馈）；短按：切项 */
if (menuKeyGetState(MENU_KEY_5) == MENU_KEY_LONG_PRESS)
{
    menuKeyClear(MENU_KEY_5);
    menuRequestSaveSettings(); /* 循环尾写 flash（含当前 DetectMode 阈值/Exposure 字段） */
    menuState = MENU_HOME;
    menuFlashFeedback(RGB565_GREEN); /* 保持数帧后自动清屏重绘发车页 */
    return;
}
if (menuKeyGetState(MENU_KEY_5) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_5);
    threshSel = (uint8_t)((threshSel + 1u) % 3u);
    forceRedraw = 1u;
    return;
}

/* 当前项步进 */
if (threshSel == 2u)
{
    stepL = VISION_EXPOSURE_STEP_LONG;
    stepS = VISION_EXPOSURE_STEP_SHORT;
} /* Exposure */
else
{
    if (detectModeF < 0.5f)
    {
        stepL = (float) VISION_BINARYCC_STEP_LONG;
        stepS = (float) VISION_BINARYCC_STEP_SHORT;
    }
    else
    {
        stepL = (float) VISION_CANNY_STEP_LONG;
        stepS = (float) VISION_CANNY_STEP_SHORT;
    }
} /* Lo/Hi */

/* 加减：K1 +大 / K2 +小 / K3 -小 / K4 -大（同编辑页方向） */
if (menuKeyGetState(MENU_KEY_1) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_1);
    delta = stepL;
}
else if (menuKeyGetState(MENU_KEY_2) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_2);
    delta = stepS;
}
else if (menuKeyGetState(MENU_KEY_3) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_3);
    delta = -stepS;
}
else if (menuKeyGetState(MENU_KEY_4) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_4);
    delta = -stepL;
}

if (delta != 0.0f)
{
    #if GLOBAL_PARAM_PERSIST_ENABLE
    if (threshSel == 0u)
    {
        if (detectModeF < 0.5f) binaryCcLowF += delta;
        else cannyLowF += delta;
    }
    else if (threshSel == 1u)
    {
        if (detectModeF < 0.5f) binaryCcHighF += delta;
    }
    #endif
}

```

```

        else cannyHighF += delta;
    }
    else exposureF += delta;
    applyVisionLocal(); /* 钳位 + 热重载，下一帧两图即更新 */
    forceRedraw = 1u;
#else
    /* 持久化关闭：阈值页仍显示默认值，K1-K4 已被消费但改值无效。 */
    (void) delta;
#endif
}
}
}

```

core0_cyt4bb7/project/code/app/menuInternal.h

SHA-256

:

feacd11eebf09bc55f9bf61298934d20d11f5a46fe5a7d138435c52c1f6d889e

代码 (c)

```

#ifndef _MENU_INTERNAL_H_
#define _MENU_INTERNAL_H_

/*
 * menu 模块内部共享头（不对外，仅 menu*.c 引用）
 * menu.c      状态唯一定义点 + 公共 API + 状态机分发 + arming 门控
 * menuModel.c 参数默认值/本地热重载/下发 + 页表构建
 * menuPersistence.c Flash blob/CRC/版本迁移 + 读写/擦除
 * menuRender.c IPS114 绘制原语 + 各状态渲染
 * menuInput.c  按键 handler + 反馈/清屏/滚动维护
 *
 * 共享状态在 menu.c 唯一定义，此处 extern 声明并按归属分组。模块私有量
 * （menuModel 的 paramTx、menuPersistence 的 flash blob、menu.c 的刷新计数等）不在此暴露。
 */

#include <stdint.h>

#include "menu.h"          /* visionTuningStruct */
#include "flightParams.h"   /* flightParamsStruct */
#include "coreDataTransfer.h" /* flightToVisionStruct */
#include "uartLinkProto.h"

/* =====
 * 屏幕几何（8×16 字体，IPS114 240×135 横向）
 * ===== */
#define LCD_W      (240u)
#define LCD_H      (135u)
#define FONT_W     (8u)
#define FONT_H     (16u)
#define ROW_Y(r)   ((uint16_t)((r) * FONT_H))
#define COL_X(c)   ((uint16_t)((c) * FONT_W))
#define VISIBLE_ROWS (7u) /* 标题占第 0 行，列表可见 7 行 */

/* 滚动条（列表页右侧）几何：贴右内边框 */
#define SCROLLBAR_W (4u)
#define SCROLLBAR_X ((uint16_t)(LCD_W - 1u - SCROLLBAR_W)) /* 235 */

/* 发车页右侧文字区起始列（左图区 x2..117 + 竖分隔 x120) */
#define HOME_TXT_X (124u)

/* 阈值编辑器双图布局（240×135）：标题 row0；两图并排 y=18 起，各 118×75，
 * 最大化利用图片区；底部参数行。左原图 x=1，右检测预览图 x=121。 */
#define THR_IMG_Y (18u)
#define THR_IMG_W (118u)
#define THR_IMG_H (75u)
#define THR_IMG_L_X (1u)
#define THR_IMG_R_X (121u)
#define THR_TXT_Y (112u) /* 参数行 y（第 7 行 ROW_Y(7)=112） */

/* cm_7_0 safetyState 约定值（与 core0Loop / pidFlightController 对齐）。
 * 非枚举启动约定值单一真相源在 coreDataTransfer.h，此处引用避免重复硬编码。 */
#define FC_STATE_DISARM (0u)
#define FC_STATE_FLYING (1u)
#define FC_STATE_PROTECT (2u)
#define FC_STATE_ARMING FLIGHT_SAFETY_ARMING
#define FC_STATE_IMUFAIL FLIGHT_SAFETY_IMUFAIL

```

```

/* 长按/保存填充反馈保持时长：×10ms PIT = 300ms，确定时长不受前台帧率影响 */
#define MENU_FEEDBACK_HOLD_TICKS (30u)

/* =====
* 菜单数据结构
* ===== */
#define MENU_MAX_OPTIONS (16u)
#define MENU_MAX_PAGES (24u) /* menuPages[] 容量；新增页前确认不超此值 */

typedef struct
{
    const char* name;
    uint8_t optionCount;
    const char* optName[MENU_MAX_OPTIONS];
    float* opt[MENU_MAX_OPTIONS];
    float stepS[MENU_MAX_OPTIONS];
    float stepL[MENU_MAX_OPTIONS];
    uint8_t isVision[MENU_MAX_OPTIONS]; /* 1=视觉/相机本地参数；0=飞控参数推 cm_7_0 */
} menuPageStruct;

typedef enum
{
    MENU_HOME = 0,
    MENU_PAGE_LIST,
    MENU_OPTION_LIST,
    MENU_EDIT,
    MENU_THRESH_EDIT, /* 检测阈值编辑器：双图实时对照 + Lo/Hi/Exp 热调 */
    MENU_PARAM_SLOTS
} menuStateEnum;

typedef enum
{
    MENU_PARAM_GLOBAL_IDLE = 0u,
    MENU_PARAM_GLOBAL_QUEUED,
    MENU_PARAM_GLOBAL_PROGRESS,
    MENU_PARAM_GLOBAL_SUCCESS,
    MENU_PARAM_GLOBAL_FAILED
} menuParamGlobalUiState;

typedef struct
{
    uint8_t state;
    uint8_t phase;
    uint8_t target;
    uint8_t error;
    uint8_t action;
    uint8_t slot;
    uint8_t slotUiState;
    uint8_t reserved;
} menuParamGlobalUiSnapshot;

/* =====
* 跨文件共享状态（唯一定义在 menu.c）
* menuVisionTuning 见 menu.h（视觉侧亦消费）。此处声明其余跨模块量。
* ===== */
/* menu owner 跨实现文件共享的私有 context。调用方只见 menu.h facade。 */
typedef struct
{
    visionTuningStruct visionTuning;
    float cannyLowF;
    float cannyHighF;
    float exposureF;
    float detectModeF;
    float binaryCcLowF;
    float binaryCcHighF;
    float binaryCcAreaMinF;
    float binaryCcAreaMaxF;
    float binaryCcBrightnessMinF;
    float binaryCcEccMaxF;
    float binaryCcFillMinF;
    float binaryCcHuMaxF;
    float binaryCcOrientBrightMinF;
    float binaryCcOrientBorderMarginF;
    float binaryCcOrientConfidenceMinF;
    float binaryCcOrientHoldFramesF;
    float orientJumpMaxF;
    float orientRecoverF;
    float orientConfirmFramesF;

```

```

float contourSizeMinF;
float contourSizeMaxF;
float contourHuLostF;
float contourBrightnessMaxF;
flightParamsStruct flightWork;
float visionPosErrEnableF;
float systemModeF;
menuStateEnum state;
int8_t pageIdx;
int8_t optIdx;
uint8_t scrollTop;
uint8_t forceRedraw;
uint8_t threshSel;
uint8_t paramSlotPageIndex;
uint8_t selectedParamSlot;
uint8_t keyTargetCore;
uint32_t kvmRouteEpoch;
uint32_t remoteKeySeq;
uint8_t remoteKeyId;
uint8_t remoteKeyState;
uint8_t remoteKeyPending;
uint8_t remoteRouteRestartPending;
uint8_t remoteEnterThresholdPending;
uint8_t remoteDeferredKeyValid;
uint8_t remoteDeferredKeyId;
uint8_t remoteDeferredKeyState;
uint32_t remoteDeferredKeyTickMs;
uint8_t armBarLast;
volatile uint8_t feedbackHoldTicks;
uint32_t seqParam;
uint32_t seqLaunch;
uint32_t seqStandby;
uint32_t seqClear;
uint32_t bootEpoch;
flightToVisionStruct flightRx;
uint32_t flightLastGeneration;
uint32_t flightLastAdvanceMs;
uint8_t flightGenerationPrimed;
uint8_t flightSnapshotAvailable;
uint8_t mutationLockLast;
kvmStatusToMenuStruct kvmRx;
volatile uint32_t tickMs;
uint32_t kvmReadFailureStartMs;
uint8_t kvmReadFailureActive;
menuPageStruct pages[MENU_MAX_PAGES];
uint8_t pageCount;
uint8_t bootArmingDone;
uint8_t bootRenderState;
uint8_t feedbackWasActive;
volatile uint8_t lcdRefreshFlag;
uint8_t lcdRefreshCount;
uint8_t initFailedMask;
uint8_t initUnverifiedMask;
volatile uint16_t initSummaryTicks;
uint16_t initSummaryLastSecond;
uint8_t initSummaryPresented;
uint8_t inputServiced;
uint8_t pendingScreenClear;
uint8_t pendingFeedback;
uint16_t pendingFeedbackColor;
uint8_t pendingFeedbackAction;
uint8_t pendingFeedbackSlot;
uint8_t pendingStorageAction;
uint8_t faultPageActive;
uint8_t faultAckPending;
uint8_t faultDisplayReason;
uint8_t faultRenderedReason;
uint8_t faultRenderedSafetyState;
} menuContextStruct;

extern menuContextStruct menuContext;

#define menuVisionTuning      menuContext.visionTuning
#define cannyLowF             menuContext.cannyLowF
#define cannyHighF            menuContext.cannyHighF
#define exposureF             menuContext.exposureF
#define detectModeF           menuContext.detectModeF
#define binaryCcLowF          menuContext.binaryCcLowF
#define binaryCcHighF         menuContext.binaryCcHighF

```

```

#define binaryCcAreaMinF      menuContext.binaryCcAreaMinF
#define binaryCcAreaMaxF      menuContext.binaryCcAreaMaxF
#define binaryCcBrightnessMinF menuContext.binaryCcBrightnessMinF
#define binaryCcEccMaxF       menuContext.binaryCcEccMaxF
#define binaryCcFillMinF      menuContext.binaryCcFillMinF
#define binaryCcHuMaxF        menuContext.binaryCcHuMaxF
#define binaryCcOrientBrightMinF menuContext.binaryCcOrientBrightMinF
#define binaryCcOrientBorderMarginF menuContext.binaryCcOrientBorderMarginF
#define binaryCcOrientConfidenceMinF menuContext.binaryCcOrientConfidenceMinF
#define binaryCcOrientHoldFramesF menuContext.binaryCcOrientHoldFramesF
#define orientJumpMaxF        menuContext.orientJumpMaxF
#define orientRecoverF        menuContext.orientRecoverF
#define orientConfirmFramesF  menuContext.orientConfirmFramesF
#define contourSizeMinF       menuContext.contourSizeMinF
#define contourSizeMaxF       menuContext.contourSizeMaxF
#define contourHuLostF        menuContext.contourHuLostF
#define contourBrightnessMaxF  menuContext.contourBrightnessMaxF
#define menuFlightWork        menuContext.flightWork
#define visionPosErrEnableF    menuContext.visionPosErrEnableF
#define systemModeF          menuContext.systemModeF
#define menuState             menuContext.state
#define pageIdx               menuContext.pageIdx
#define optIdx                menuContext.optIdx
#define scrollTop              menuContext.scrollTop
#define forceRedraw           menuContext.forceRedraw
#define threshSel             menuContext.threshSel
#define paramSlotPageIndex    menuContext.paramSlotPageIndex
#define selectedParamSlot     menuContext.selectedParamSlot
#define menuKeyTargetCore     menuContext.keyTargetCore
#define menuKvmRouteEpoch    menuContext.kvmRouteEpoch
#define remoteKeySeq          menuContext.remoteKeySeq
#define remoteKeyId           menuContext.remoteKeyId
#define remoteKeyState        menuContext.remoteKeyState
#define remoteKeyPending      menuContext.remoteKeyPending
#define remoteRouteRestartPending menuContext.remoteRouteRestartPending
#define remoteEnterThresholdPending menuContext.remoteEnterThresholdPending
#define remoteDeferredKeyValid menuContext.remoteDeferredKeyValid
#define remoteDeferredKeyId    menuContext.remoteDeferredKeyId
#define remoteDeferredKeyState menuContext.remoteDeferredKeyState
#define remoteDeferredKeyTickMs menuContext.remoteDeferredKeyTickMs
#define armBarLast            menuContext.armBarLast
#define feedbackHoldTicks     menuContext.feedbackHoldTicks
#define seqParam              menuContext.seqParam
#define seqLaunch              menuContext.seqLaunch
#define seqStandby             menuContext.seqStandby
#define seqClear               menuContext.seqClear
#define bootEpoch            menuContext.bootEpoch
#define flightRx               menuContext.flightRx
#define flightLastGeneration   menuContext.flightLastGeneration
#define flightLastAdvanceMs    menuContext.flightLastAdvanceMs
#define flightGenerationPrimed menuContext.flightGenerationPrimed
#define flightSnapshotAvailable menuContext.flightSnapshotAvailable
#define mutationLockLast      menuContext.mutationLockLast
#define kvmRx                  menuContext.kvmRx
#define menuTickMs             menuContext.tickMs
#define kvmReadFailureStartMs  menuContext.kvmReadFailureStartMs
#define kvmReadFailureActive   menuContext.kvmReadFailureActive
#define menuPages              menuContext.pages
#define menuPageCount          menuContext.pageCount
#define bootArmingDone         menuContext.bootArmingDone
#define bootRenderState        menuContext.bootRenderState
#define feedbackWasActive      menuContext.feedbackWasActive
#define lcdRefreshFlag         menuContext.lcdRefreshFlag
#define lcdRefreshCount        menuContext.lcdRefreshCount
#define menuInitFailedMask     menuContext.initFailedMask
#define menuInitUnverifiedMask menuContext.initUnverifiedMask
#define menuInitSummaryTicks   menuContext.initSummaryTicks
#define menuInitSummaryLastSecond menuContext.initSummaryLastSecond
#define menuInitSummaryPresented menuContext.initSummaryPresented
#define menuInputServiced      menuContext.inputServiced
#define menuPendingScreenClear menuContext.pendingScreenClear
#define menuPendingFeedback    menuContext.pendingFeedback
#define menuPendingFeedbackColor menuContext.pendingFeedbackColor
#define menuPendingFeedbackAction menuContext.pendingFeedbackAction
#define menuPendingFeedbackSlot menuContext.pendingFeedbackSlot
#define menuPendingStorageAction menuContext.pendingStorageAction
#define menuFaultPageActive    menuContext.faultPageActive
#define menuFaultAckPending     menuContext.faultAckPending
#define menuFaultDisplayReason menuContext.faultDisplayReason

```

```

#define menuFaultRenderedReason    menuContext.faultRenderedReason
#define menuFaultRenderedSafetyState menuContext.faultRenderedSafetyState

/* =====
* 跨文件函数原型
* ===== */
/* menuModel.c: 参数模型 + 页表 */
void menuFillVisionTuningDefaults(visionTuningStruct* tuning);
void menuFillBinaryCcDefaults(void);
void menuLoadDefaults(void);
void applyVisionLocal(void);
void menuPushParams(void);
void menuBuildPages(void);
void applyAfterEdit(uint8_t isVisionOpt);
void menuSaveSettings(void);
void menuRestoreDefaults(void);
void menuRequestScreenClear(void);
void menuRequestFeedback(uint16_t color);
void menuRequestParamFeedback(uint16_t color, uint8_t action, uint8_t slot);
void menuRequestSaveSettings(void);
void menuRequestRestoreDefaults(void);

/* menuPersistence.c: Flash blob/CRC/版本迁移 + 读写/擦除 */
void menuFlashLoad(void);
void menuFlashSave(void);
void menuFlashErase(void);

/* menuRender.c: 各状态渲染 */
void renderToInit(void);
void renderArming(uint8_t progress);
void renderInitFaultSummary(uint8_t failedMask, uint8_t unverifiedMask, uint8_t secondsLeft);
void renderFaultReasonPage(uint8_t reason, uint8_t safetyState,
    uint8_t fullRedraw, uint8_t reasonChanged, uint8_t stateChanged);
void renderHome(void);
void renderThreshEdit(void);
void renderPageList(void);
void renderOptionList(uint8_t editing);
void renderParamSlots(void);
void menuRenderResetListSnapshot(void); /* 切状态/清屏复位列表渲染快照 → 下次列表渲染为结构性整页重绘 */

/* menuInput.c: 按键 handler + 共享清屏 */
void clearScreenForState(void);
uint8_t handleFlightStandbyKey(void); /* 飞行态 K5 长按: 立即请求回 DISARMED 待机 */
uint8_t handleFaultReasonAck(void); /* 故障原因页 K5 短按: 只确认原因, 不发车 */
void handleHomeKeys(void);
void handleRemoteForward(void); /* 远程操控模式: 转发按键给 core1(不进本地菜单) */
void handlePageListKeys(void);
void handleOptionListKeys(void);
void handleEditKeys(void);
void handleThreshEditKeys(void);
void handleParamSlotKeys(void);
void menuParamGlobalReset(void);
void menuParamGlobalPoll(void);
void menuParamGlobalGetUi(menuParamGlobalUiSnapshot* snapshot);

#endif /* _MENU_INTERNAL_H_ */

```

core0_cyt4bb7/project/code/app/menuKey.c

SHA-256

:

12dd206cd9b327f1dbd504076c96cfbc790e80d7c1690a51132958afcd598fa1

代码 (c)

```

#include "menuKey.h"

#include "pinMap.h"
#include "zf_driver_gpio.h"

/*
* 实现: 5 路 GPI 上拉, 按下 = GPIO_LOW。
* menuKeyScan 在 10ms PIT ISR 调用, 按 zf_device_key 等价语义实现:
* - KEY_RELEASE_LEVEL = GPIO_HIGH (释放)
* - KEY_MAX_SHOCK_PERIOD = 10ms (消抖)
* - KEY_LONG_PRESS_PERIOD = 1000ms (短按/长按门限)

```

```

* 对原逆飞库的功能扩展：
* - 短按状态锁存到 menuKeyClear 调用（防止短按事件被相邻 scan 周期覆盖）
* - 短按事件超时（达长按周期）自动清除，防陈旧事件误触发
* - 5 键全覆盖（原库 key_clear_all_state 仅清前 4 键，不在本封装提供 all_clear）
*/

#define MENU_KEY_RELEASE_LEVEL  (GPIO_HIGH)
#define MENU_KEY_SHOCK_PERIOD_MS (10u)
#define MENU_KEY_LONG_PERIOD_MS (1000u)

static const gpio_pin_enum keyPins[MENU_KEY_COUNT] = {MENU_KEY1_PIN, MENU_KEY2_PIN, MENU_KEY3_PIN, MENU_KEY4_PIN,
MENU_KEY5_PIN};

#include "menuKeyStateMachineImpl.h"

```

core0_cyt4bb7/project/code/app/menuKey.h

SHA-256 :
a71f025ba33422e2a02223dfdc92f51f39ceb72cfa815431f504fe115f48465f

代码 (c)

```

#ifndef _MENU_KEY_H_
#define _MENU_KEY_H_

#include <stdint.h>

/*
 * core0 cm_7_1 菜单专用按键扫描封装
 *
 * 独立于逆飞 zf_device_key 库，避免修改逆飞原始代码以便后续库同步。
 * 实现：
 * - 5 路 GPIO 直读（KEY_LIST 引脚）+ 周期消抖
 * - 短按 / 长按 区分（释放时根据持续时长判定）
 * - 短按锁存：短按事件保留至 menuKeyGetState 读取后由 menuKeyClear 清除
 * - 短按长时间未消费时自动超时清除（防止误触发陈旧事件）
 * - 长按消费(menuKeyClear)后置“等释放”：按住期间不再重复触发，必须物理释放
 *   后才允许下一次事件（防长按操作在按住期间反复触发）
 *
 * 线程模型：
 * menuKeyScan() 在 PIT ISR 周期调用（10ms）；
 * menuKeyGetState/Clear 在 whileProcess 前台消费；
 * 状态数组声明 volatile 跨 ISR/前台共享。
 */

typedef enum
{
    MENU_KEY_1 = 0u,
    MENU_KEY_2,
    MENU_KEY_3,
    MENU_KEY_4,
    MENU_KEY_5,
    MENU_KEY_COUNT
} menuKeyIndexEnum;

typedef enum
{
    MENU_KEY_RELEASE = 0u,
    MENU_KEY_SHORT_PRESS,
    MENU_KEY_LONG_PRESS
} menuKeyStateEnum;

/* 初始化：配置 GPI 上拉 + 清状态。periodMs = 调用 menuKeyScan 的周期(毫秒) */
void menuKeyInit(uint32_t periodMs);

/* 扫描：在 PIT ISR 周期调用 */
void menuKeyScan(void);

/* 查询按键状态（短按事件保留至 menuKeyClear 调用） */
menuKeyStateEnum menuKeyGetState(menuKeyIndexEnum k);

/* 清除指定按键状态 */
void menuKeyClear(menuKeyIndexEnum k);

/* 丢弃当前事件并禁止重新计时，直到检测到一次物理释放。用于页面切换/安全 ACK，

```



```
/* 防仍按住的键在新页面重新累计并穿透成菜单或发车动作。 */
void menuKeyDiscardUntilRelease(menuKeyIndexEnum k);
```

```
#endif /* _MENU_KEY_H_ */
```

core0_cyt4bb7/project/code/app/menuKeyStateMachineImpl.h

SHA-256

:

59535b1e043685925409616f368a3a31b77aec3c8e1a5054def43f33e20be6e5

代码 (c)

```
#ifndef MENU_KEY_STATE_MACHINE_IMPL_H
#define MENU_KEY_STATE_MACHINE_IMPL_H
```

```
/* 单翻译单元实现片段：由 menuKey.c 在目标专用 keyPins 定义后包含一次。 */
static volatile uint32_t scanPeriodMs = 0u;
static volatile uint32_t pressTimeMs[MENU_KEY_COUNT];
static volatile uint32_t shortHoldMs[MENU_KEY_COUNT];
static volatile menuKeyStateEnum keyState[MENU_KEY_COUNT];
static volatile uint8_t waitRelease[MENU_KEY_COUNT]; /* 消费后等物理释放，防长按按住期间重复触发 */
```

```
void menuKeyInit(uint32_t periodMs)
{
    if (periodMs == 0u) periodMs = 1u;
    scanPeriodMs = periodMs;
    for (uint8_t i = 0u; i < MENU_KEY_COUNT; i++)
    {
        gpio_init(keyPins[i], GPI, GPIO_HIGH, GPI_PULL_UP);
        keyState[i] = MENU_KEY_RELEASE;
        pressTimeMs[i] = 0u;
        shortHoldMs[i] = 0u;
        waitRelease[i] = 0u;
    }
}
```

```
void menuKeyScan(void)
{
    for (uint8_t i = 0u; i < MENU_KEY_COUNT; i++)
    {
        uint8_t pressed = (gpio_get_level(keyPins[i]) != MENU_KEY_RELEASE_LEVEL) ? 1u : 0u;

        /* 等释放期间：忽略按键直到物理释放，期间不累计、不产状态 */
        if (waitRelease[i])
        {
            if (!pressed)
            {
                waitRelease[i] = 0u;
                keyState[i] = MENU_KEY_RELEASE;
                pressTimeMs[i] = 0u;
                shortHoldMs[i] = 0u;
            }
            continue;
        }

        /* 短按状态锁存超时清除 */
        if (keyState[i] == MENU_KEY_SHORT_PRESS)
        {
            shortHoldMs[i] += scanPeriodMs;
            if (shortHoldMs[i] >= MENU_KEY_LONG_PERIOD_MS)
            {
                keyState[i] = MENU_KEY_RELEASE;
                shortHoldMs[i] = 0u;
            }
        }

        if (pressed)
        {
            /* 按下：累计时长，达长按门限即升级为长按 */
            pressTimeMs[i] += scanPeriodMs;
            if (pressTimeMs[i] >= MENU_KEY_LONG_PERIOD_MS)
            {
                keyState[i] = MENU_KEY_LONG_PRESS;
            }
        }
    }
}
```

```

else
{
    /* 释放：根据持续时长判定短按/释放 */
    if (keyState[i] != MENU_KEY_LONG_PRESS &&
        pressTimeMs[i] >= MENU_KEY_SHOCK_PERIOD_MS &&
        pressTimeMs[i] < MENU_KEY_LONG_PERIOD_MS)
    {
        if (keyState[i] != MENU_KEY_SHORT_PRESS)
        {
            keyState[i] = MENU_KEY_SHORT_PRESS;
            shortHoldMs[i] = 0u;
        }
    }
    else if ((keyState[i] != MENU_KEY_SHORT_PRESS) &&
        (keyState[i] != MENU_KEY_LONG_PRESS))
    {
        keyState[i] = MENU_KEY_RELEASE;
    }
    pressTimeMs[i] = 0u;
}
}
}

menuKeyStateEnum menuKeyGetState(menuKeyIndexEnum k)
{
    if (k >= MENU_KEY_COUNT) return MENU_KEY_RELEASE;
    return keyState[k];
}

void menuKeyClear(menuKeyIndexEnum k)
{
    if (k >= MENU_KEY_COUNT) return;
    /* 长按消费后置等释放：防按住期间 scan 立刻再次升级为 LONG_PRESS 重复触发。
     * 短按消费正常清零即可（短按事件本就一次性锁存）。 */
    if (keyState[k] == MENU_KEY_LONG_PRESS) waitRelease[k] = 1u;
    keyState[k] = MENU_KEY_RELEASE;
    pressTimeMs[k] = 0u;
    shortHoldMs[k] = 0u;
}

void menuKeyDiscardUntilRelease(menuKeyIndexEnum k)
{
    if (k >= MENU_KEY_COUNT) return;
    waitRelease[k] = 1u;
    keyState[k] = MENU_KEY_RELEASE;
    pressTimeMs[k] = 0u;
    shortHoldMs[k] = 0u;
}

#endif

```

core0_cyt4bb7/project/code/app/menuModel.c

SHA-256 :
993adc6510d4962f122056ef0af3536ede18b6ed7b792593bc4bd78650038589

代码 (c)

```

#include "menu.h"
#include "menuInternal.h"

#include <string.h>

#include "zf_device_mt9v03x.h"

#include "boardConfig.h" /* GLOBAL_PARAM_PERSIST_ENABLE 编译期开关 */
#include "commonDef.h" /* clampf / wrapToPi / PI */
#include "configFlightControlDefaults.h"
#include "configVision.h"
#include "image.h"
#include "coreDataTransfer.h"
#include "flightParams.h"

/*
 * menuModel.c — 参数模型层：默认值加载、本地视觉热重载、向 cm_7_0 下发飞控参数/指令、

```

```

* 分页菜单页表构建；Flash 布局/迁移/读写由 menuPersistence.c 负责。
* 不触屏、不读按键；状态机/渲染/按键见 menu.c / menuRender.c / menuInput.c。
*/

/* 下发缓存（非 ISR，本 TU 私有） */
static paramToFlightStruct paramTx;

/* 视觉目标/极性/死区/外推默认值（configVision 宏单一真相源）：
 * menuLoadDefaults 与 menuFlashSave 的 PERSIST-off 占位共用，避免两处逐字段重复。 */
void menuFillVisionTuningDefaults(visionTuningStruct* v)
{
    v->targetCenterX = (float) VISION_TARGET_CENTER_X;
    v->targetCenterY = (float) VISION_TARGET_CENTER_Y;
    v->targetAngleRad = VISION_TARGET_ANGLE_RAD;
    v->errXSign = VISION_ERR_X_SIGN;
    v->errYSign = VISION_ERR_Y_SIGN;
    v->yawErrSign = VISION_YAW_ERR_SIGN;
    v->deadzoneEnterPct = VISION_DEADZONE_ENTER_PCT_DEFAULT;
    v->deadzoneExitPct = VISION_DEADZONE_EXIT_PCT_DEFAULT;
    v->lostFreezeMs = VISION_LOST_FREEZE_MS_DEFAULT;
    v->lostExtrapDiameterPx = VISION_LOST_EXTRAP_DIAMETER_PX_DEFAULT;
}

void menuFillBinaryCcDefaults(void)
{
    detectModeF = (float) VISION_DETECT_MODE_DEFAULT;
    binaryCcLowF = (float) VISION_BINARYCC_LOW_DEFAULT;
    binaryCcHighF = (float) VISION_BINARYCC_HIGH_DEFAULT;
    binaryCcAreaMinF = (float) VISION_BINARYCC_AREA_MIN_DEFAULT;
    binaryCcAreaMaxF = (float) VISION_BINARYCC_AREA_MAX_DEFAULT;
    binaryCcBrightnessMinF = (float) VISION_BINARYCC_BRIGHT_MIN_DEFAULT;
    binaryCcEccMaxF = VISION_BINARYCC_ECC_MAX_DEFAULT;
    binaryCcFillMinF = VISION_BINARYCC_FILL_MIN_DEFAULT;
    binaryCcHuMaxF = VISION_BINARYCC_HU_MAX_DEFAULT;
    binaryCcOrientBrightMinF = (float) VISION_BINARYCC_ORI_BRIGHT_MIN_DEFAULT;
    binaryCcOrientBorderMarginF = (float) VISION_BINARYCC_ORI_BORDER_MARGIN_DEFAULT;
    binaryCcOrientConfidenceMinF = VISION_BINARYCC_ORI_CONF_MIN_DEFAULT;
    binaryCcOrientHoldFramesF = (float) VISION_BINARYCC_ORI_HOLD_FRAMES_DEFAULT;
}

/* =====
 * 默认值加载（视觉从 configVision 宏，飞控从 flightParamsLoadDefaults）
 * ===== */
void menuLoadDefaults(void)
{
    imageOrientationProtectTuningStruct orientDefaults;
    menuFillVisionTuningDefaults(&menuVisionTuning);

    cannyLowF = (float) VISION_CANNY_LOW_DEFAULT;
    cannyHighF = (float) VISION_CANNY_HIGH_DEFAULT;
    exposureF = (float) VISION_EXPOSURE_DEFAULT;
    menuFillBinaryCcDefaults();

    imageGetOrientationProtectDefaults(&orientDefaults);
    orientJumpMaxF = orientDefaults.jumpMax;
    orientRecoverF = orientDefaults.recoverRad;
    orientConfirmFramesF = (float) orientDefaults.confirmFrames;
    /* 轮廓检测默认（image.h 宏） */
    contourSizeMinF = (float) VISION_CONTOUR_SIZE_MIN_DEFAULT;
    contourSizeMaxF = (float) VISION_CONTOUR_SIZE_MAX_DEFAULT;
    contourHuLostF = VISION_CONTOUR_HU_LOST_DEFAULT;
    contourBrightnessMaxF = (float) VISION_CONTOUR_BRIGHTNESS_MAX_DEF;

    flightParamsLoadDefaults(&menuFlightWork);
    visionPosErrEnableF = (float) menuFlightWork.visionPosErrEnable;
    systemModeF = (float) menuFlightWork.systemMode;
}

/* =====
 * 本地热重载（视觉/相机参数即时生效）
 * ===== */
void applyVisionLocal(void)
{
    /* Canny 钳位（与原 core1Loop 一致：MIN/MAX/GAP） */
    if (cannyLowF < (float) VISION_CANNY_THRESHOLD_MIN) cannyLowF = (float) VISION_CANNY_THRESHOLD_MIN;
    if (cannyHighF > (float) VISION_CANNY_THRESHOLD_MAX) cannyHighF = (float) VISION_CANNY_THRESHOLD_MAX;
    if (cannyLowF + (float) VISION_CANNY_THRESHOLD_GAP > cannyHighF)

```

```

{
    if (cannyHighF > (float) (VISION_CANNY_THRESHOLD_MIN + VISION_CANNY_THRESHOLD_GAP))
    {
        cannyLowF = cannyHighF - (float) VISION_CANNY_THRESHOLD_GAP;
    }
    else
    {
        cannyLowF = (float) VISION_CANNY_THRESHOLD_MIN;
        cannyHighF = (float) (VISION_CANNY_THRESHOLD_MIN + VISION_CANNY_THRESHOLD_GAP);
    }
}
imageApplyCannyThresholds((uint16_t) cannyLowF, (uint16_t) cannyHighF);

detectModeF = (detectModeF >= 0.5f) ? (float) VISION_DETECT_MODE_LEGACY_CANNY : (float) VISION_DETECT_MODE_BINARYCC;

binaryCcLowF = clampf(binaryCcLowF, (float) VISION_BINARYCC_THRESHOLD_MIN, (float) VISION_BINARYCC_THRESHOLD_MAX);
binaryCcHighF = clampf(binaryCcHighF, (float) VISION_BINARYCC_THRESHOLD_MIN, (float) VISION_BINARYCC_THRESHOLD_MAX);
if (binaryCcLowF > binaryCcHighF) binaryCcLowF = binaryCcHighF;
binaryCcAreaMinF = clampf(binaryCcAreaMinF, 1.0f, 59999.0f);
binaryCcAreaMaxF = clampf(binaryCcAreaMaxF, binaryCcAreaMinF + 1.0f, 60000.0f);
binaryCcBrightnessMinF = clampf(binaryCcBrightnessMinF, 0.0f, 255.0f);
binaryCcEccMaxF = clampf(binaryCcEccMaxF, 0.0f, 100.0f);
binaryCcFillMinF = clampf(binaryCcFillMinF, 0.0f, 1.0f);
binaryCcHuMaxF = clampf(binaryCcHuMaxF, 0.0f, 200.0f);
binaryCcOrientBrightMinF = clampf(binaryCcOrientBrightMinF, 0.0f, 255.0f);
binaryCcOrientBorderMarginF = clampf(binaryCcOrientBorderMarginF, 0.0f, 20.0f);
binaryCcOrientConfidenceMinF = clampf(binaryCcOrientConfidenceMinF, 0.0f, 1.0f);
binaryCcOrientHoldFramesF = clampf(binaryCcOrientHoldFramesF, 1.0f, 60.0f);
{
    imageBinaryCcTuningStruct binaryCfg;
    binaryCfg.thresholdLow = (uint8_t) (binaryCcLowF + 0.5f);
    binaryCfg.thresholdHigh = (uint8_t) (binaryCcHighF + 0.5f);
    binaryCfg.areaMin = (uint16_t) binaryCcAreaMinF;
    binaryCfg.areaMax = (uint16_t) binaryCcAreaMaxF;
    binaryCfg.brightnessMin = (uint8_t) (binaryCcBrightnessMinF + 0.5f);
    binaryCfg.eccMax = binaryCcEccMaxF;
    binaryCfg.fillMin = binaryCcFillMinF;
    binaryCfg.huMax = binaryCcHuMaxF;
    binaryCfg.orientationBrightMin = (uint8_t) (binaryCcOrientBrightMinF + 0.5f);
    binaryCfg.orientationBorderMargin = (uint8_t) (binaryCcOrientBorderMarginF + 0.5f);
    binaryCfg.orientationConfidenceMin = binaryCcOrientConfidenceMinF;
    binaryCfg.orientationHoldFrames = (uint8_t) (binaryCcOrientHoldFramesF + 0.5f);
    binaryCfg.referenceCenterX = (uint16_t) clampf(menuVisionTuning.targetCenterX, 0.0f, (float) (MT9V03X_W - 1u));
    binaryCfg.referenceCenterY = (uint16_t) clampf(menuVisionTuning.targetCenterY, 0.0f, (float) (MT9V03X_H - 1u));
    imageApplyBinaryCcTuning(&binaryCfg);
    imageSetDetectMode((uint8_t) detectModeF);
}

/* 曝光钳位并下发相机 */
exposureF = clampf(exposureF, (float) VISION_EXPOSURE_MIN, (float) VISION_EXPOSURE_MAX);
mt9v03x_set_exposure_time((uint16_t) exposureF);

/* 极性规整为 ±1 */
menuVisionTuning.errXSign = (menuVisionTuning.errXSign >= 0.0f) ? 1.0f : -1.0f;
menuVisionTuning.errYSign = (menuVisionTuning.errYSign >= 0.0f) ? 1.0f : -1.0f;
menuVisionTuning.yawErrSign = (menuVisionTuning.yawErrSign >= 0.0f) ? 1.0f : -1.0f;
/* 死区百分比钳位：enter∈[1,100]，exit∈[0,enter-1]，保证大圆>小圆(回差非负) */
menuVisionTuning.deadzoneEnterPct = clampf(menuVisionTuning.deadzoneEnterPct, 1.0f, 100.0f);
menuVisionTuning.deadzoneExitPct = clampf(menuVisionTuning.deadzoneExitPct, 0.0f, menuVisionTuning.deadzoneEnterPct - 1.0f);
menuVisionTuning.lostFreezeMs = clampf(menuVisionTuning.lostFreezeMs, 0.0f, 3000.0f);
/* 外推圆直径钳位 [2,200]px：下限防退化点（方向无意义），上限留余量（虚拟靶点可超图像） */
menuVisionTuning.lostExtrapDiameterPx = clampf(menuVisionTuning.lostExtrapDiameterPx, 2.0f, 200.0f);
menuVisionTuning.targetCenterX = clampf(menuVisionTuning.targetCenterX, 0.0f, (float) (MT9V03X_W - 1u));
menuVisionTuning.targetCenterY = clampf(menuVisionTuning.targetCenterY, 0.0f, (float) (MT9V03X_H - 1u));
menuVisionTuning.targetAngleRad = wrapToPi(menuVisionTuning.targetAngleRad);

/* 朝向跳变保护钳位 + 下发（jumpMax∈(0,PI]，recover∈(0,jumpMax)，confirm≥1） */
orientJumpMaxF = clampf(orientJumpMaxF, 0.05f, PI);
orientRecoverF = clampf(orientRecoverF, 0.01f, orientJumpMaxF);
orientConfirmFramesF = clampf(orientConfirmFramesF, 1.0f, 60.0f);
{
    imageOrientationProtectTuningStruct orientConfig;
    orientConfig.jumpMax = orientJumpMaxF;
    orientConfig.recoverRad = orientRecoverF;
    orientConfig.confirmFrames = (uint8_t) (orientConfirmFramesF + 0.5f);
    imageApplyOrientationProtect(&orientConfig);
}

/* 轮廓检测钳位 + 下发（经 image.c 封装，^ Hu 模板保持 image 私有）。

```

```

    * 先把 min 钳到 [1,59999], 再把 max 钳到 [min+1,60000], 保证 min<max 且二者
    * 都在 uint16 范围内, 杜绝 sizeMin>sizeMax 或转 uint16_t 回绕 (极端菜单/flash 值)。 */
    contourSizeMinF = clampf(contourSizeMinF, 1.0f, 59 999.0f);
    contourSizeMaxF = clampf(contourSizeMaxF, contourSizeMinF + 1.0f, 60 000.0f);
    if (contourHuLostF < 0.0f) contourHuLostF = 0.0f;
    contourBrightnessMaxF = clampf(contourBrightnessMaxF, 0.0f, 255.0f);
    imageApplyContourTuning((uint16_t) contourSizeMinF, (uint16_t) contourSizeMaxF,
        contourHuLostF, (uint8_t) (contourBrightnessMaxF + 0.5f));
}

/* =====
* 向 cm_7_0 推送飞控参数 / 指令
* ===== */
void menuPushParams(void)
{
    menuFlightWork.visionPosErrEnable = (visionPosErrEnableF >= 0.5f) ? 1u : 0u;

    /* 空地系统模式: 影子 → 工作副本 (钳 1..3)。顶层模式只下发给 core1/core2 决定车是否运动
    * 以及车控制源; 不改写独立调试开关 VisPosErr。 */
    {
        uint8_t newSystemMode = (uint8_t) clampf(systemModeF, 1.0f, 3.0f);
        menuFlightWork.systemMode = newSystemMode;
    }
    /* 推送前硬校验: 修正菜单可能调出的非法值 (负增益/上下限反转等),
    * 修正结果回写工作副本, 使屏幕显示与实际下发一致 */
    flightParamsSanitize(&menuFlightWork);
    visionPosErrEnableF = (float) menuFlightWork.visionPosErrEnable;
    systemModeF = (float) menuFlightWork.systemMode;

    memcpy(&paramTx.params, &menuFlightWork, sizeof(flightParamsStruct));
    paramTx.paramSeq = seqParam;
    paramTx.cmdLaunchSeq = seqLaunch;
    paramTx.cmdStandbySeq = seqStandby;
    paramTx.cmdClearSeq = seqClear;
    paramTx.cm71BootEpoch = bootEpoch;
    paramTx.cm71ParamReady = 0u; /* 事务 token 由 coreDataTransfer owner 填充 */
    paramTx.kvmRouteEpoch = menuKvmRouteEpoch;
    paramTx.kvmEventSeq = remoteKeySeq;
    paramTx.kvmTarget = menuKeyTargetCore;
    paramTx.kvmKeyId = remoteKeyId;
    paramTx.kvmKeyState = remoteKeyState;
    paramTx.kvmFlags = remoteKeyPending ? 1u : 0u;
    coreDataWriteParamToFlight(&paramTx);
}

/* =====
* 页表构建
* ===== */
static void addPage(const char* name)
{
    if (menuPageCount >= MENU_MAX_PAGES) return;
    menuPages[menuPageCount].name = name;
    menuPages[menuPageCount].optionCount = 0u;
}
static void addOpt(const char* nm, float* p, float s, float l, uint8_t vision)
{
    menuPageStruct* pg;
    uint8_t i;
    if (menuPageCount >= MENU_MAX_PAGES) return;
    pg = &menuPages[menuPageCount];
    i = pg->optionCount;
    if (i >= MENU_MAX_OPTIONS) return;
    pg->optName[i] = nm;
    pg->opt[i] = p;
    pg->stepS[i] = s;
    pg->stepL[i] = l;
    pg->isVision[i] = vision;
    pg->optionCount = (uint8_t) (i + 1u);
}
static void endPage(void)
{
    if (menuPageCount >= MENU_MAX_PAGES) return;
    menuPageCount++;
}

void menuBuildPages(void)
{
    menuPageCount = 0u;
}

```

```

addPage("Vision/Cam");
addOpt("DetectMode", &detectModeF, 1.0f, 1.0f, 1u);
addOpt("TgtCenterX", &menuVisionTuning.targetCenterX, 1.0f, 10.0f, 1u);
addOpt("TgtCenterY", &menuVisionTuning.targetCenterY, 1.0f, 10.0f, 1u);
addOpt("TgtAngle", &menuVisionTuning.targetAngleRad, 0.01f, 0.1f, 1u);
addOpt("ErrX Sign", &menuVisionTuning.errXSign, 2.0f, 2.0f, 1u);
addOpt("ErrY Sign", &menuVisionTuning.errYSign, 2.0f, 2.0f, 1u);
addOpt("YawE Sign", &menuVisionTuning.yawErrSign, 2.0f, 2.0f, 1u);
addOpt("Canny Low", &cannyLowF,
(float) VISION_CANNY_STEP_SHORT,
(float) VISION_CANNY_STEP_LONG,
1u);
addOpt("Canny High", &cannyHighF,
(float) VISION_CANNY_STEP_SHORT,
(float) VISION_CANNY_STEP_LONG,
1u);
addOpt("Exposure", &exposureF, VISION_EXPOSURE_STEP_SHORT, VISION_EXPOSURE_STEP_LONG, 1u);
endPage();

```

```

addPage("BinaryCC");
addOpt("Bin Low", &binaryCcLowF,
(float) VISION_BINARYCC_STEP_SHORT,
(float) VISION_BINARYCC_STEP_LONG,
1u);
addOpt("Bin High", &binaryCcHighF,
(float) VISION_BINARYCC_STEP_SHORT,
(float) VISION_BINARYCC_STEP_LONG,
1u);
addOpt("AreaMin", &binaryCcAreaMinF, 10.0f, 100.0f, 1u);
addOpt("AreaMax", &binaryCcAreaMaxF, 100.0f, 1000.0f, 1u);
addOpt("BrightMin", &binaryCcBrightnessMinF, 5.0f, 20.0f, 1u);
addOpt("EccMax", &binaryCcEccMaxF, 0.1f, 0.5f, 1u);
addOpt("FillMin", &binaryCcFillMinF, 0.01f, 0.05f, 1u);
addOpt("HuMax", &binaryCcHuMaxF, 0.5f, 2.0f, 1u);
addOpt("OriBright", &binaryCcOrientBrightMinF, 5.0f, 20.0f, 1u);
addOpt("OriEdge", &binaryCcOrientBorderMarginF, 1.0f, 5.0f, 1u);
addOpt("OriConf", &binaryCcOrientConfidenceMinF, 0.01f, 0.05f, 1u);
addOpt("OriHold", &binaryCcOrientHoldFramesF, 1.0f, 5.0f, 1u);
endPage();

```

```

addPage("Vision Track");
addOpt("OriJumpMax", &orientJumpMaxF, 0.05f, 0.2f, 1u);
addOpt("OriRecover", &orientRecoverF, 0.02f, 0.1f, 1u);
addOpt("OriCfmFrm", &orientConfirmFramesF, 1.0f, 5.0f, 1u);
addOpt("SizeMin", &contourSizeMinF, 1.0f, 10.0f, 1u);
addOpt("SizeMax", &contourSizeMaxF, 100.0f, 1000.0f, 1u);
addOpt("HuLost", &contourHuLostF, 0.5f, 2.0f, 1u);
addOpt("BrightMax", &contourBrightnessMaxF, 5.0f, 20.0f, 1u);
addOpt("DZ Enter%", &menuVisionTuning.deadzoneEnterPct, 0.5f, 5.0f, 1u);
addOpt("DZ Exit%", &menuVisionTuning.deadzoneExitPct, 0.5f, 5.0f, 1u);
addOpt("LostFrzMs", &menuVisionTuning.lostFreezeMs, 10.0f, 100.0f, 1u);
addOpt("LostExtrD", &menuVisionTuning.lostExtrapDiameterPx, 1.0f, 10.0f, 1u);
endPage();

```

```

addPage("Throttle");
addOpt("Base Thr", &menuFlightWork.baseThrottlePct, 0.5f, 5.0f, 0u);
addOpt("Thr Min", &menuFlightWork.throttleMinPct, 0.5f, 5.0f, 0u);
addOpt("Thr Max", &menuFlightWork.throttleMaxPct, 1.0f, 10.0f, 0u);
addOpt("Pitch Ref", &menuFlightWork.pitchRefRad, 0.01f, 0.1f, 0u);
addOpt("Roll Ref", &menuFlightWork.rollRefRad, 0.01f, 0.1f, 0u);
endPage();

```

```

addPage("Angle PID");
addOpt("P Ang Kp", &menuFlightWork.pitchAngle.kp, 0.1f, 1.0f, 0u);
addOpt("P Ang Ki", &menuFlightWork.pitchAngle.ki, 0.01f, 0.1f, 0u);
addOpt("P Ang Kd", &menuFlightWork.pitchAngle.kd, 0.1f, 1.0f, 0u);
addOpt("P Ang Imax", &menuFlightWork.pitchAngle.integralMax, 0.1f, 1.0f, 0u);
addOpt("R Ang Kp", &menuFlightWork.rollAngle.kp, 0.1f, 1.0f, 0u);
addOpt("R Ang Ki", &menuFlightWork.rollAngle.ki, 0.01f, 0.1f, 0u);
addOpt("R Ang Kd", &menuFlightWork.rollAngle.kd, 0.1f, 1.0f, 0u);
addOpt("R Ang Imax", &menuFlightWork.rollAngle.integralMax, 0.1f, 1.0f, 0u);
addOpt("RateLim P", &menuFlightWork.rateCmdLimitPitch, 10.0f, 100.0f, 0u);
addOpt("RateLim R", &menuFlightWork.rateCmdLimitRoll, 10.0f, 100.0f, 0u);
endPage();

```

```

/* 姿态去旋转增益即使在位置 PID 临时隐藏时仍需可调，用于隔离旋转光流。*/
addPage("Vision Comp");
addOpt("AttComp X", &menuFlightWork.posAttCompXGainPxPerRad, 1.0f, 10.0f, 0u);
addOpt("AttComp Y", &menuFlightWork.posAttCompYGainPxPerRad, 1.0f, 10.0f, 0u);

```

```

endPage();

addPage("Rate PID PR");
addOpt("P Rate Kp", &menuFlightWork.pitchRate.kp, 0.1f, 1.0f, 0u);
addOpt("P Rate Ki", &menuFlightWork.pitchRate.ki, 0.001f, 0.01f, 0u);
addOpt("P Rate Kd", &menuFlightWork.pitchRate.kd, 0.1f, 1.0f, 0u);
addOpt("P Rate Imax", &menuFlightWork.pitchRate.integralMax, 0.5f, 5.0f, 0u);
addOpt("R Rate Kp", &menuFlightWork.rollRate.kp, 0.1f, 1.0f, 0u);
addOpt("R Rate Ki", &menuFlightWork.rollRate.ki, 0.001f, 0.01f, 0u);
addOpt("R Rate Kd", &menuFlightWork.rollRate.kd, 0.1f, 1.0f, 0u);
addOpt("R Rate Imax", &menuFlightWork.rollRate.integralMax, 0.5f, 5.0f, 0u);
addOpt("CtrlLim P", &menuFlightWork.ctrlLimitPctPitch, 1.0f, 10.0f, 0u);
addOpt("CtrlLim R", &menuFlightWork.ctrlLimitPctRoll, 1.0f, 10.0f, 0u);
addOpt("P RateFF", &menuFlightWork.pitchRateFF, 0.01f, 0.1f, 0u);
addOpt("R RateFF", &menuFlightWork.rollRateFF, 0.01f, 0.1f, 0u);
endPage();

addPage("Yaw Rate");
addOpt("Y Rate Kp", &menuFlightWork.yawRate.kp, 0.1f, 1.0f, 0u);
addOpt("Y Rate Ki", &menuFlightWork.yawRate.ki, 0.01f, 0.1f, 0u);
addOpt("Y Rate Kd", &menuFlightWork.yawRate.kd, 0.1f, 1.0f, 0u);
addOpt("Y Rate Imax", &menuFlightWork.yawRate.integralMax, 0.1f, 1.0f, 0u);
addOpt("Y Rate Ref", &menuFlightWork.yawRateRefRadS, 0.01f, 0.1f, 0u);
addOpt("CtrlLim Y", &menuFlightWork.ctrlLimitPctYaw, 1.0f, 10.0f, 0u);
addOpt("Gyro LPF", &menuFlightWork.gyroLpfAlpha, 0.01f, 0.1f, 0u);
endPage();

addPage("Pos Loop");
addOpt("PosX Kp", &menuFlightWork.posX.kp, 0.0001f, 0.001f, 0u);
addOpt("PosX Ki", &menuFlightWork.posX.ki, 0.0001f, 0.001f, 0u);
addOpt("PosX Kd", &menuFlightWork.posX.kd, 0.001f, 0.01f, 0u);
addOpt("PosY Kp", &menuFlightWork.posY.kp, 0.0001f, 0.001f, 0u);
addOpt("PosY Ki", &menuFlightWork.posY.ki, 0.0001f, 0.001f, 0u);
addOpt("PosY Kd", &menuFlightWork.posY.kd, 0.001f, 0.01f, 0u);
addOpt("PosVelLim", &menuFlightWork.posVelCmdLimit, 0.01f, 0.1f, 0u);
addOpt("PosAngLim", &menuFlightWork.posAngleLimitRad, 0.001f, 0.01f, 0u);
endPage();

addPage("Pos Vel");
addOpt("PVx Kp", &menuFlightWork.posVelX.kp, 0.01f, 0.1f, 0u);
addOpt("PVx Ki", &menuFlightWork.posVelX.ki, 0.00001f, 0.0001f, 0u);
addOpt("PVx Kd", &menuFlightWork.posVelX.kd, 0.01f, 0.1f, 0u);
addOpt("PVx Imax", &menuFlightWork.posVelX.integralMax, 0.01f, 0.1f, 0u);
addOpt("PVy Kp", &menuFlightWork.posVelY.kp, 0.01f, 0.1f, 0u);
addOpt("PVy Ki", &menuFlightWork.posVelY.ki, 0.00001f, 0.0001f, 0u);
addOpt("PVy Kd", &menuFlightWork.posVelY.kd, 0.01f, 0.1f, 0u);
addOpt("PVy Imax", &menuFlightWork.posVelY.integralMax, 0.01f, 0.1f, 0u);
endPage();

addPage("Yaw Angle");
addOpt("YAng Kp", &menuFlightWork.yawAngle.kp, 0.01f, 0.1f, 0u);
addOpt("YAng Ki", &menuFlightWork.yawAngle.ki, 0.001f, 0.01f, 0u);
addOpt("YAng Kd", &menuFlightWork.yawAngle.kd, 0.01f, 0.1f, 0u);
addOpt("YRateCmdLim", &menuFlightWork.yawRateCmdLimit, 0.1f, 1.0f, 0u);
endPage();

addPage("Alt Gain");
addOpt("AltPos Kp", &menuFlightWork.altPos.kp, 0.1f, 1.0f, 0u);
addOpt("AltPos Ki", &menuFlightWork.altPos.ki, 0.001f, 0.01f, 0u);
addOpt("AltPos Kd", &menuFlightWork.altPos.kd, 0.1f, 1.0f, 0u);
addOpt("AltVel Kp", &menuFlightWork.altVel.kp, 0.001f, 0.01f, 0u);
addOpt("AltVel Ki", &menuFlightWork.altVel.ki, 0.0001f, 0.001f, 0u);
addOpt("AltVel Kd", &menuFlightWork.altVel.kd, 0.001f, 0.01f, 0u);
addOpt("AltVel Imax", &menuFlightWork.altVel.integralMax, 0.5f, 5.0f, 0u);
addOpt("AltVelFF", &menuFlightWork.altVelFF, 0.001f, 0.01f, 0u);
addOpt("OffsetLim", &menuFlightWork.altOffsetLimitPct, 0.5f, 5.0f, 0u);
endPage();

addPage("Alt Config");
addOpt("Target mm", &menuFlightWork.altTargetMm, 10.0f, 100.0f, 0u);
addOpt("Dist Min", &menuFlightWork.altDistMinMm, 10.0f, 50.0f, 0u);
addOpt("Dist Max", &menuFlightWork.altDistMaxMm, 100.0f, 500.0f, 0u);
addOpt("Timeout ms", &menuFlightWork.altTimeoutMs, 100.0f, 500.0f, 0u);
addOpt("VelLpf a", &menuFlightWork.altVelLpfAlpha, 0.01f, 0.1f, 0u);
addOpt("VelCmdLim", &menuFlightWork.altVelCmdLimit, 1000.0f, 10000.0f, 0u);
addOpt("TofMaxV", &menuFlightWork.altTofMaxSpeedMmS, 100.0f, 1000.0f, 0u);
endPage();

addPage("Posture");

```

```

addOpt("PitchMin", &menuFlightWork.postureSafePitchMinRad, 0.05f, 0.1f, 0u);
addOpt("PitchMax", &menuFlightWork.postureSafePitchMaxRad, 0.05f, 0.1f, 0u);
addOpt("RollMin", &menuFlightWork.postureSafeRollMinRad, 0.05f, 0.1f, 0u);
addOpt("RollMax", &menuFlightWork.postureSafeRollMaxRad, 0.05f, 0.1f, 0u);
addOpt("GyroMax", &menuFlightWork.postureSafeGyroMaxRadS, 1.0f, 10.0f, 0u);
endPage();

addPage("FF/Switch");
addOpt("VFF Vx>Fwd", &menuFlightWork.motionVelFFVxToFwd, 0.001f, 0.01f, 0u);
addOpt("VFF Vy>Rgt", &menuFlightWork.motionVelFFVyToRight, 0.001f, 0.01f, 0u);
addOpt("FF Vx>Pit", &menuFlightWork.motionFFVxToPitch, 0.0001f, 0.001f, 0u);
addOpt("FF Vy>Rol", &menuFlightWork.motionFFVyToRoll, 0.0001f, 0.001f, 0u);
addOpt("FF Wz>Yaw", &menuFlightWork.motionFFWzToYaw, 0.001f, 0.01f, 0u);
addOpt("Vx dLim", &menuFlightWork.motionFFVxDiffLimitMps2, 0.1f, 1.0f, 0u);
addOpt("Vy dLim", &menuFlightWork.motionFFVyDiffLimitMps2, 0.1f, 1.0f, 0u);
addOpt("Vx dLpf", &menuFlightWork.motionFFVxDiffLpfAlpha, 0.01f, 0.1f, 0u);
addOpt("Vy dLpf", &menuFlightWork.motionFFVyDiffLpfAlpha, 0.01f, 0.1f, 0u);
addOpt("Orb FwdM", &menuFlightWork.motionYawOrbitOffsetFwdM, 0.01f, 0.1f, 0u);
addOpt("Orb RgtM", &menuFlightWork.motionYawOrbitOffsetRightM, 0.01f, 0.1f, 0u);
addOpt("VisPosErr", &visionPosErrEnableF, 1.0f, 1.0f, 0u);
endPage();

/* 空地系统模式（1=车不动 2=RS485 自主 3=LoRa 转发）： 步长 1 整数循环编辑。
 * VisPosErr 为独立调试开关，模式切换不覆盖。 */
addPage("System");
addOpt("SysMode", &systemModeF, 1.0f, 1.0f, 0u);
endPage();

addPage("Param Slots");
paramSlotPageIndex = (uint8_t) (menuPageCount - 1u);
endPage();
}

/* =====
 * 编辑应用：本地视觉即时生效；飞控推 cm_7_0
 * ===== */
void applyAfterEdit(uint8_t isVisionOpt)
{
    if (isVisionOpt)
    {
        applyVisionLocal();
    }
    else
    {
        seqParam++;
        menuPushParams();
    }
}

/* =====
 * 保存 / 清空恢复默认
 * ===== */
void menuSaveSettings(void)
{
    #if GLOBAL_PARAM_PERSIST_ENABLE
        applyVisionLocal();
        seqParam++;
        menuPushParams();
        menuFlashSave();
    #else
        /* 持久化关闭：人工保存键无效（可调参数不落盘）。
         * 标定自动落盘走 menuTick 直接调 menuFlashSave，不经本路径，不受影响。 */
    #endif
}

void menuRestoreDefaults(void)
{
    menuLoadDefaults();
    #if GLOBAL_PARAM_PERSIST_ENABLE
        imageResetActiveTemplate(); /* 清两套 RAM 标定模板，回退各自出厂默认 */
    #endif
    applyVisionLocal();
    #if GLOBAL_PARAM_PERSIST_ENABLE
        menuFlashErase();
    #endif
    seqParam++;
    #if GLOBAL_PARAM_PERSIST_ENABLE
        seqClear++;

```



```

#endif
/* seq 增量须在 menuPushParams 之前——它一次性快照 seqParam/seqClear/seqLaunch 下发 cm_7_0。
 * 持久化关闭时不擦 flash、不清标定模板、不动 seqClear（标定不受开关影响；
 * seqParam 边沿已足以触发 cm_7_0 整块回默认）。*/
menuPushParams();
}

```

core0_cyt4bb7/project/code/app/menuPersistence.c

SHA-256

:

f55e288df3989c27d111566bc29b38c8c89b91cd42b2b2754f090e184ad619b3

代码 (c)

```

#include "menu.h"
#include "menuInternal.h"

#include <math.h>
#include <stddef.h>
#include <string.h>

#include "zf_driver_flash.h"

#include "boardConfig.h" /* GLOBAL_PARAM_PERSIST_ENABLE 编译期开关 */
#include "commonDef.h" /* clampf */
#include "configFlightControlDefaults.h"
#include "configVision.h"
#include "crc32leee.h"
#include "flightParams.h"
#include "image.h"
#include "menuParamSlots.h"
#include "paramSlotStore.h"

/*
 * menuPersistence.c — core0 cm_7_1 菜单参数 Flash 持久化：
 * blob 布局、CRC 及 load/save/erase。非当前版本统一回退默认值。
 * 公共菜单 API 不变，模型默认值仍由 menuModel.c 提供。
 */

/* =====
 * flash 持久化布局（单页，magic+version+CRC）
 * ===== */
#define MENU_FLASH_PAGE (0u)
#define MENU_FLASH_MAGIC (0x4D314B30u) /* "M1K0" core0 cm_7_1 菜单参数 */
#define MENU_FLASH_VERSION (42u)

typedef struct
{
    uint32_t magic;
    uint32_t version;
    uint32_t payloadBytes; /* sizeof(载荷区)，用于一致性校验 */
    visionTuningStruct vision; /* 视觉目标中点/极性 */
    float cannyLow;
    float cannyHigh;
    float exposure;
    float detectMode;
    float binaryLow;
    float binaryHigh;
    float binaryAreaMin;
    float binaryAreaMax;
    float binaryBrightnessMin;
    float binaryEccMax;
    float binaryFillMin;
    float binaryHuMax;
    float binaryOrientBrightMin;
    float binaryOrientBorderMargin;
    float binaryOrientConfidenceMin;
    float binaryOrientHoldFrames;
    /* 朝向跳变保护 + Legacy 轮廓检测调参 */
    float orientJumpMax;
    float orientRecover;
    float orientConfirmFrames;
    float contourSizeMin;
    float contourSizeMax;
    float contourHuLost;

```

```

float contourBrightnessMax;
uint32_t legacyCalibValid;
float legacyCalibHu[7];
float legacyCalibSizeMin;
float legacyCalibSizeMax;
uint32_t binaryCalibValid;
float binaryCalibHu[7];
float binaryCalibSizeMin;
float binaryCalibSizeMax;
flightParamsStruct flight; /* 全量飞控参数 */
uint32_t crc; /* CRC32 over [magic..flight] */
} menuFlashBlobStruct;

typedef char menu_flash_blob_size_assert[(sizeof(menuFlashBlobStruct) == 648u) ? 1 : -1];

static uint8_t menuFlashBlobCrcValid(const void* blob, uint32_t blobBytes)
{
    uint32_t storedCrc;
    uint32_t crcLen;
    uint32_t crc;

    if (blobBytes <= sizeof(uint32_t)) return 0u;
    crcLen = blobBytes - (uint32_t) sizeof(uint32_t);
    memcpy(&storedCrc, ((const uint8_t*) blob) + crcLen, sizeof(storedCrc));
    crc = crc32leeeCalc((const uint8_t*) blob, crcLen);
    return (crc == storedCrc) ? 1u : 0u;
}

static uint8_t menuFlashBlobValid(const menuFlashBlobStruct* blob)
{
    if (blob->payloadBytes != (uint32_t) sizeof(menuFlashBlobStruct)) return 0u;

    return menuFlashBlobCrcValid(blob, (uint32_t) sizeof(menuFlashBlobStruct));
}

#ifdef GLOBAL_PARAM_PERSIST_ENABLE
static void fillBinaryCcBlobDefaults(menuFlashBlobStruct* blob);
#endif

static paramSlotStore g_paramSlots;
static uint32_t g_localTransactionId = 1u;

static uint8_t menuFlashBlobSlotValid(const void *data, uint32_t bytes)
{
    const menuFlashBlobStruct *blob = (const menuFlashBlobStruct *) data;
    return (uint8_t) ((bytes == sizeof(*blob)) &&
        (blob->magic == MENU_FLASH_MAGIC) &&
        (blob->version == MENU_FLASH_VERSION) &&
        (menuFlashBlobValid(blob) != 0u));
}

static uint8_t menuFlashBlobExactValid(const menuFlashBlobStruct *blob)
{
    const uint32_t *words = (const uint32_t *) (const void *) blob;
    flightParamsStruct sanitized;
    uint32_t i;

    if (menuFlashBlobSlotValid(blob, sizeof(*blob)) == 0u) return 0u;
    for (i = 0u; i < (uint32_t) (offsetof(menuFlashBlobStruct, crc) / 4u); i++)
    {
        float value;
        memcpy(&value, &words[i], sizeof(value));
        if (!isfinite(value)) return 0u;
    }
    if ((blob->legacyCalibValid > 1u) || (blob->binaryCalibValid > 1u)) return 0u;
    if ((blob->legacyCalibSizeMin < 0.0f) || (blob->legacyCalibSizeMin > 65 535.0f) ||
        (blob->legacyCalibSizeMax < 0.0f) || (blob->legacyCalibSizeMax > 65 535.0f) ||
        (blob->binaryCalibSizeMin < 0.0f) || (blob->binaryCalibSizeMin > 65 535.0f) ||
        (blob->binaryCalibSizeMax < 0.0f) || (blob->binaryCalibSizeMax > 65 535.0f))
        return 0u;
    sanitized = blob->flight;
    flightParamsSanitize(&sanitized);
    return (uint8_t) (memcmp(&sanitized, &blob->flight, sizeof(sanitized)) == 0);
}

static uint32_t nextLocalTransaction(void)
{
    g_localTransactionId++;
    if (g_localTransactionId == 0u) g_localTransactionId = 1u;
}

```

```

    return g_localTransactionId;
}

static void menuFlashBlobBuild(menuFlashBlobStruct *blob)
{
    uint32_t crcLen;

    memset(blob, 0, sizeof(*blob));
    blob->magic = MENU_FLASH_MAGIC;
    blob->version = MENU_FLASH_VERSION;
    blob->payloadBytes = (uint32_t) sizeof(*blob);
#if GLOBAL_PARAM_PERSIST_ENABLE
    flightParamsStruct flight = menuFlightWork;
    flight.visionPosErrEnable = (visionPosErrEnableF >= 0.5f) ? 1u : 0u;
    flight.systemMode = (uint8_t) clampf(systemModeF, 1.0f, 3.0f);
    flightParamsSanitize(&flight);
    blob->vision = menuVisionTuning;
    blob->cannyLow = cannyLowF;
    blob->cannyHigh = cannyHighF;
    blob->exposure = exposureF;
    blob->detectMode = detectModeF;
    blob->binaryLow = binaryCcLowF;
    blob->binaryHigh = binaryCcHighF;
    blob->binaryAreaMin = binaryCcAreaMinF;
    blob->binaryAreaMax = binaryCcAreaMaxF;
    blob->binaryBrightnessMin = binaryCcBrightnessMinF;
    blob->binaryEccMax = binaryCcEccMaxF;
    blob->binaryFillMin = binaryCcFillMinF;
    blob->binaryHuMax = binaryCcHuMaxF;
    blob->binaryOrientBrightMin = binaryCcOrientBrightMinF;
    blob->binaryOrientBorderMargin = binaryCcOrientBorderMarginF;
    blob->binaryOrientConfidenceMin = binaryCcOrientConfidenceMinF;
    blob->binaryOrientHoldFrames = binaryCcOrientHoldFramesF;
    blob->orientJumpMax = orientJumpMaxF;
    blob->orientRecover = orientRecoverF;
    blob->orientConfirmFrames = orientConfirmFramesF;
    blob->contourSizeMin = contourSizeMinF;
    blob->contourSizeMax = contourSizeMaxF;
    blob->contourHuLost = contourHuLostF;
    blob->contourBrightnessMax = contourBrightnessMaxF;
    blob->flight = flight;
#else
    {
        flightParamsStruct defFlight;
        imageOrientationProtectTuningStruct orientDefaults;
        flightParamsLoadDefaults(&defFlight);
        imageGetOrientationProtectDefaults(&orientDefaults);
        menuFillVisionTuningDefaults(&blob->vision);
        blob->cannyLow = (float) VISION_CANNY_LOW_DEFAULT;
        blob->cannyHigh = (float) VISION_CANNY_HIGH_DEFAULT;
        blob->exposure = (float) VISION_EXPOSURE_DEFAULT;
        fillBinaryCcBlobDefaults(blob);
        blob->orientJumpMax = orientDefaults.jumpMax;
        blob->orientRecover = orientDefaults.recoverRad;
        blob->orientConfirmFrames = (float) orientDefaults.confirmFrames;
        blob->contourSizeMin = (float) VISION_CONTOUR_SIZE_MIN_DEFAULT;
        blob->contourSizeMax = (float) VISION_CONTOUR_SIZE_MAX_DEFAULT;
        blob->contourHuLost = VISION_CONTOUR_HU_LOST_DEFAULT;
        blob->contourBrightnessMax = (float) VISION_CONTOUR_BRIGHTNESS_MAX_DEF;
        blob->flight = defFlight;
    }
#endif
    {
        uint16_t sizeMin = 0u;
        uint16_t sizeMax = 0u;
        blob->legacyCalibValid = (uint32_t) imageGetActiveTemplateForMode(
            VISION_DETECT_MODE_LEGACY_CANNY, blob->legacyCalibHu, &sizeMin, &sizeMax);
        blob->legacyCalibSizeMin = (float) sizeMin;
        blob->legacyCalibSizeMax = (float) sizeMax;
        sizeMin = 0u;
        sizeMax = 0u;
        blob->binaryCalibValid = (uint32_t) imageGetActiveTemplateForMode(
            VISION_DETECT_MODE_BINARYCC, blob->binaryCalibHu, &sizeMin, &sizeMax);
        blob->binaryCalibSizeMin = (float) sizeMin;
        blob->binaryCalibSizeMax = (float) sizeMax;
    }
    crcLen = (uint32_t) (sizeof(*blob) - sizeof(uint32_t));
    blob->crc = crc32IeeeCalc((const uint8_t *) blob, crcLen);
}

```

```

#if !GLOBAL_PARAM_PERSIST_ENABLE
static void fillBinaryCcBlobDefaults(menuFlashBlobStruct* blob)
{
    blob->detectMode = (float) VISION_DETECT_MODE_DEFAULT;
    blob->binaryLow = (float) VISION_BINARYCC_LOW_DEFAULT;
    blob->binaryHigh = (float) VISION_BINARYCC_HIGH_DEFAULT;
    blob->binaryAreaMin = (float) VISION_BINARYCC_AREA_MIN_DEFAULT;
    blob->binaryAreaMax = (float) VISION_BINARYCC_AREA_MAX_DEFAULT;
    blob->binaryBrightnessMin = (float) VISION_BINARYCC_BRIGHT_MIN_DEFAULT;
    blob->binaryEccMax = VISION_BINARYCC_ECC_MAX_DEFAULT;
    blob->binaryFillMin = VISION_BINARYCC_FILL_MIN_DEFAULT;
    blob->binaryHuMax = VISION_BINARYCC_HU_MAX_DEFAULT;
    blob->binaryOrientBrightMin = (float) VISION_BINARYCC_ORI_BRIGHT_MIN_DEFAULT;
    blob->binaryOrientBorderMargin = (float) VISION_BINARYCC_ORI_BORDER_MARGIN_DEFAULT;
    blob->binaryOrientConfidenceMin = VISION_BINARYCC_ORI_CONF_MIN_DEFAULT;
    blob->binaryOrientHoldFrames = (float) VISION_BINARYCC_ORI_HOLD_FRAMES_DEFAULT;
}
#endif

/* =====
* flash 持久化 (magic+version+CRC, 失败回退默认)
* FLASH_PAGE_LENGTH = 512 words (2KB), blob 远小于一页。
* blob 经 flash_union_buffer 中转 (库要求), 按 word 对齐写入。
* ===== */
static void menuFlashApplyBlob(const menuFlashBlobStruct *blob)
{
    #if GLOBAL_PARAM_PERSIST_ENABLE
        menuVisionTuning = blob->vision;
        cannyLowF = blob->cannyLow;
        cannyHighF = blob->cannyHigh;
        exposureF = blob->exposure;
        detectModeF = blob->detectMode;
        binaryCcLowF = blob->binaryLow;
        binaryCcHighF = blob->binaryHigh;
        binaryCcAreaMinF = blob->binaryAreaMin;
        binaryCcAreaMaxF = blob->binaryAreaMax;
        binaryCcBrightnessMinF = blob->binaryBrightnessMin;
        binaryCcEccMaxF = blob->binaryEccMax;
        binaryCcFillMinF = blob->binaryFillMin;
        binaryCcHuMaxF = blob->binaryHuMax;
        binaryCcOrientBrightMinF = blob->binaryOrientBrightMin;
        binaryCcOrientBorderMarginF = blob->binaryOrientBorderMargin;
        binaryCcOrientConfidenceMinF = blob->binaryOrientConfidenceMin;
        binaryCcOrientHoldFramesF = blob->binaryOrientHoldFrames;
        orientJumpMaxF = blob->orientJumpMax;
        orientRecoverF = blob->orientRecover;
        orientConfirmFramesF = blob->orientConfirmFrames;
        contourSizeMinF = blob->contourSizeMin;
        contourSizeMaxF = blob->contourSizeMax;
        contourHuLostF = blob->contourHuLost;
        contourBrightnessMaxF = blob->contourBrightnessMax;
        menuFlightWork = blob->flight;
    #if (FLIGHT_SYSTEM_MODE_FORCE_DEFAULT_ON_BOOT != 0u)
        menuFlightWork.systemMode = FLIGHT_SYSTEM_MODE_DEFAULT;
    #endif
        flightParamsSanitize(&menuFlightWork);
        visionPosErrEnableF = (float) menuFlightWork.visionPosErrEnable;
        systemModeF = (float) menuFlightWork.systemMode;
    #endif

    if (blob->legacyCalibValid == 1u)
    {
        imageSetActiveTemplateForMode(VISION_DETECT_MODE_LEGACY_CANNY, blob->legacyCalibHu,
            (uint16_t) blob->legacyCalibSizeMin, (uint16_t) blob->legacyCalibSizeMax);
    }
    if (blob->binaryCalibValid == 1u)
    {
        imageSetActiveTemplateForMode(VISION_DETECT_MODE_BINARYCC, blob->binaryCalibHu,
            (uint16_t) blob->binaryCalibSizeMin, (uint16_t) blob->binaryCalibSizeMax);
    }
}

void menuFlashLoad(void)
{
    menuFlashBlobStruct blob;
    menuFlashBlobStruct defaults;
    uint8_t valid = 0u;

    if (sizeof(menuFlashBlobStruct) > FLASH_PAGE_SIZE) return;

```

```

menuFlashBlobBuild(&defaults);
if (paramSlotStoreInit(&g_paramSlots, sizeof(menuFlashBlobStruct),
    menuFlashBlobSlotValid, &defaults) != PARAM_SLOT_OK) return;
valid = (uint8_t) (paramSlotStoreReadRuntime(&g_paramSlots,
    &blob, sizeof(blob)) == PARAM_SLOT_OK);

if (valid)
{
    menuFlashApplyBlob(&blob);
    return;
}

/* Selector 指向的数据损坏时保持调用方已加载的完整源码默认值。 */
}

void menuFlashSave(void)
{
    menuFlashBlobStruct blob;
    menuFlashBlobBuild(&blob);
    (void) paramSlotStoreSaveRuntime(&g_paramSlots, &blob, sizeof(blob));
}

void menuFlashErase(void)
{
    /* menuRestoreDefaults 已先恢复 RAM 默认和清空标定；只覆盖 runtime。 */
    menuFlashSave();
}

uint8_t menuParamSlotsGetActive(void)
{
    return paramSlotStoreGetActive(&g_paramSlots);
}

uint8_t menuParamSlotsHasPending(void)
{
    paramSlotPendingInfo pending;
    paramSlotStatus status = paramSlotStoreGetPending(&g_paramSlots, &pending);
    if (status == PARAM_SLOT_EMPTY) return 0u;
    return (uint8_t) ((status != PARAM_SLOT_OK) || (pending.valid != 0u));
}

uint32_t menuParamSlotsGetActiveBlobCrc(void)
{
    paramRuntimeMeta meta;
    return paramSlotStoreGetRuntimeMeta(&g_paramSlots, &meta) == PARAM_SLOT_OK ?
        meta.blobCrc : 0u;
}

uint32_t menuParamCurrentBlobCrc(void)
{
    menuFlashBlobStruct blob;
    menuFlashBlobBuild(&blob);
    return crc32IeeeCalc((const uint8_t *) &blob, sizeof(blob));
}

uint8_t menuParamRuntimeIsDirty(void)
{
    paramRuntimeMeta meta;
    if (paramSlotStoreGetRuntimeMeta(&g_paramSlots, &meta) != PARAM_SLOT_OK)
        return 1u;
    return (uint8_t) (menuParamCurrentBlobCrc() != meta.blobCrc);
}

uint8_t menuParamSlotsGetMeta(uint8_t slot, menuParamSlotMeta *meta)
{
    paramSlotMeta generic;
    paramSlotStatus status;
    if (meta == NULL) return 0u;
    status = paramSlotStoreGetMeta(&g_paramSlots, slot, &generic);
    memset(meta, 0, sizeof(*meta));
    meta->slot = slot;
    if ((status != PARAM_SLOT_OK) && (status != PARAM_SLOT_EMPTY)) return 0u;
    meta->valid = generic.valid;
    meta->active = generic.active;
    meta->bank = generic.bank;
    meta->generation = generic.generation;
    meta->blobCrc = generic.blobCrc;
    meta->transactionId = generic.transactionId;
    return 1u;
}

```

```

}

uint8_t menuParamSlotsRead(uint8_t slot, uint8_t *blob, uint32_t blobBytes)
{
    return (uint8_t) (paramSlotStoreRead(&g_paramSlots, slot, blob, blobBytes) ==
        PARAM_SLOT_OK);
}

uint8_t menuParamSlotsSaveCurrent(uint8_t slot)
{
    menuFlashBlobStruct blob;
    menuFlashBlobBuild(&blob);
    return (uint8_t) (paramSlotStoreSave(&g_paramSlots, slot, &blob, sizeof(blob),
        nextLocalTransaction(), 0u) == PARAM_SLOT_OK);
}

uint8_t menuParamSlotsActivate(uint8_t slot)
{
    menuFlashBlobStruct blob;
    if (paramSlotStoreRead(&g_paramSlots, slot, &blob, sizeof(blob)) != PARAM_SLOT_OK)
        return 0u;
    if (paramSlotStoreSetActive(&g_paramSlots, slot) != PARAM_SLOT_OK) return 0u;
    menuFlashApplyBlob(&blob);
    applyVisionLocal();
    seqParam++;
    menuPushParams();
    return 1u;
}

uint8_t menuParamSlotsActivatePrepared(uint8_t slot, uint32_t transactionId)
{
    menuFlashBlobStruct blob;
    if (paramSlotStoreRead(&g_paramSlots, slot, &blob, sizeof(blob)) != PARAM_SLOT_OK)
        return 0u;
    if (paramSlotStoreActivatePrepared(&g_paramSlots, slot, transactionId) !=
        PARAM_SLOT_OK) return 0u;
    menuFlashApplyBlob(&blob);
    applyVisionLocal();
    seqParam++;
    menuPushParams();
    return 1u;
}

uint8_t menuParamSlotsActivatePending(uint8_t slot)
{
    menuFlashBlobStruct blob;
    if (paramSlotStoreRead(&g_paramSlots, slot, &blob, sizeof(blob)) != PARAM_SLOT_OK)
        return 0u;
    if (paramSlotStoreSetActivePending(&g_paramSlots, slot) != PARAM_SLOT_OK) return 0u;
    menuFlashApplyBlob(&blob);
    applyVisionLocal();
    seqParam++;
    menuPushParams();
    return 1u;
}

uint8_t menuParamSlotsAbortPendingActivation(void)
{
    paramSlotPendingInfo pending;
    menuFlashBlobStruct rollback;
    paramSlotStatus status = paramSlotStoreGetPending(&g_paramSlots, &pending);
    if (status == PARAM_SLOT_EMPTY) return 1u;
    if (status != PARAM_SLOT_OK || (pending.valid == 0u) ||
        (pending.activationOnly == 0u)) return 0u;
    if (pending.active != 0u)
    {
        if (paramSlotStoreReadRuntime(&g_paramSlots, &rollback,
            sizeof(rollback)) != PARAM_SLOT_OK) return 0u;
        menuFlashApplyBlob(&rollback);
        applyVisionLocal();
        seqParam++;
        menuPushParams();
    }
    if (paramSlotStoreAbortPendingActivation(&g_paramSlots) != PARAM_SLOT_OK)
        return 0u;
    return 1u;
}

uint8_t menuParamSlotsFinishPendingActivation(void)

```

```

{
    paramSlotPendingInfo pending;
    menuFlashBlobStruct blob;
    paramSlotStatus status = paramSlotStoreGetPending(&g_paramSlots, &pending);
    if (status == PARAM_SLOT_EMPTY) return 1u;
    if (status != PARAM_SLOT_OK || (pending.valid == 0u) ||
        (pending.active == 0u) ||
        (pending.activationOnly == 0u) ||
        (paramSlotStoreReadPreparedRuntime(&g_paramSlots, UINT32_MAX,
            &blob, sizeof(blob)) != PARAM_SLOT_OK)) return 0u;
    menuFlashApplyBlob(&blob);
    applyVisionLocal();
    seqParam++;
    menuPushParams();
    if (paramSlotStoreFinishPendingActivation(&g_paramSlots) != PARAM_SLOT_OK)
        return 0u;
    return 1u;
}

uint8_t menuParamSlotsPrepare(uint8_t slot, const uint8_t *blob,
    uint32_t blobBytes, uint32_t transactionId)
{
    if ((blob == NULL) || (blobBytes != sizeof(menuFlashBlobStruct)) ||
        (menuFlashBlobExactValid((const menuFlashBlobStruct *) (const void *) blob) == 0u))
        return 0u;
    return (uint8_t) (paramSlotStorePrepare(&g_paramSlots, slot, blob, blobBytes,
        transactionId) == PARAM_SLOT_OK);
}

uint8_t menuParamSlotsPrepareCurrent(uint8_t slot, uint32_t transactionId)
{
    menuFlashBlobStruct blob;
    menuFlashBlobBuild(&blob);
    return (uint8_t) (paramSlotStorePrepare(&g_paramSlots, slot, &blob,
        sizeof(blob), transactionId) == PARAM_SLOT_OK);
}

uint8_t menuParamSlotsCommit(uint8_t slot, uint32_t transactionId)
{
    return (uint8_t) (paramSlotStoreCommit(&g_paramSlots, slot, transactionId, 0u) ==
        PARAM_SLOT_OK);
}

uint8_t menuParamSlotsAbort(uint32_t transactionId)
{
    paramSlotPendingInfo pending;
    paramSlotStatus pendingStatus = paramSlotStoreGetPending(&g_paramSlots,
        &pending);
    menuFlashBlobStruct rollback;
    uint8_t runtimePending;

    if ((pendingStatus != PARAM_SLOT_OK) &&
        (pendingStatus != PARAM_SLOT_EMPTY)) return 0u;
    if ((pendingStatus == PARAM_SLOT_OK) && (pending.valid != 0u) &&
        (pending.active != 0u) &&
        ((pending.activationOnly != 0u) ||
        (pending.transactionId != transactionId))) return 0u;
    runtimePending = (uint8_t) ((pendingStatus == PARAM_SLOT_OK) &&
        (pending.valid != 0u) && (pending.active != 0u) &&
        (pending.activationOnly == 0u) &&
        (pending.transactionId == transactionId));
    if (runtimePending != 0u)
    {
        if (paramSlotStoreReadRuntime(&g_paramSlots, &rollback,
            sizeof(rollback)) != PARAM_SLOT_OK) return 0u;
        menuFlashApplyBlob(&rollback);
        applyVisionLocal();
        seqParam++;
        menuPushParams();
    }
    if (paramSlotStoreAbort(&g_paramSlots, transactionId) != PARAM_SLOT_OK)
        return 0u;
    return 1u;
}

uint8_t menuParamSlotsFinish(uint32_t transactionId)
{
    paramSlotPendingInfo pending;
    paramSlotStatus pendingStatus = paramSlotStoreGetPending(&g_paramSlots,

```

```

        &pending);
menuFlashBlobStruct blob;
uint8_t runtimePending;

if ((pendingStatus != PARAM_SLOT_OK) &&
    (pendingStatus != PARAM_SLOT_EMPTY)) return 0u;
runtimePending = (uint8_t) ((pendingStatus == PARAM_SLOT_OK) &&
    (pending.valid != 0u) && (pending.active != 0u));
if (runtimePending != 0u)
{
    if (paramSlotStoreReadPreparedRuntime(&g_paramSlots, transactionId,
        &blob, sizeof(blob)) != PARAM_SLOT_OK) return 0u;
    menuFlashApplyBlob(&blob);
    applyVisionLocal();
    seqParam++;
    menuPushParams();
}
if (paramSlotStoreFinish(&g_paramSlots, transactionId) != PARAM_SLOT_OK)
    return 0u;
return 1u;
}

uint8_t menuParamSlotsInspect(uint8_t slot, menuParamLibraryMeta *meta)
{
    paramSlotMeta raw;
    menuFlashBlobStruct blob;
    paramSlotStatus status;
    if (meta == NULL) return 0u;
    memset(meta, 0, sizeof(*meta));
    status = paramSlotStoreGetMeta(&g_paramSlots, slot, &raw);
    if (status == PARAM_SLOT_EMPTY)
    {
        meta->state = MENU_PARAM_SLOT_EMPTY;
        return 1u;
    }
    if (status != PARAM_SLOT_OK) return 0u;
    meta->bank = raw.bank;
    meta->generation = raw.generation;
    meta->blobCrc = raw.blobCrc;
    meta->transactionId = raw.transactionId;
    status = paramSlotStoreRead(&g_paramSlots, slot, &blob, sizeof(blob));
    if (status == PARAM_SLOT_OK) meta->state = MENU_PARAM_SLOT_VALID;
    else if ((status == PARAM_SLOT_INVALID) ||
        (status == PARAM_SLOT_VERIFY_FAILED))
        meta->state = MENU_PARAM_SLOT_INVALID;
    else return 0u;
    return 1u;
}

uint8_t menuParamRuntimeRead(uint8_t *blob, uint32_t blobBytes)
{
    return (uint8_t) (paramSlotStoreReadRuntime(&g_paramSlots, blob,
        blobBytes) == PARAM_SLOT_OK);
}

uint8_t menuParamRuntimeGetMeta(menuParamRuntimeMeta *meta)
{
    paramRuntimeMeta raw;
    if ((meta == NULL) ||
        (paramSlotStoreGetRuntimeMeta(&g_paramSlots, &raw) != PARAM_SLOT_OK))
        return 0u;
    memset(meta, 0, sizeof(*meta));
    meta->valid = raw.valid;
    meta->bank = raw.bank;
    meta->pending = raw.pending;
    meta->sourceSlot = raw.sourceSlot;
    meta->diagnosticFlags = paramSlotStoreRuntimeDefaultsRewritten(&g_paramSlots) ?
        MENU_PARAM_RUNTIME_DIAGNOSTIC_DEFAULTS_REWRITTEN : 0u;
    meta->generation = raw.generation;
    meta->blobCrc = raw.blobCrc;
    meta->transactionId = raw.transactionId;
    return 1u;
}

uint8_t menuParamRuntimeSaveCurrent(void)
{
    menuFlashBlobStruct blob;
    menuFlashBlobBuild(&blob);
    return (uint8_t) (paramSlotStoreSaveRuntime(&g_paramSlots, &blob,

```



```

        sizeof(blob)) == PARAM_SLOT_OK);
    }

uint8_t menuParamRuntimePrepare(const uint8_t *blob, uint32_t blobBytes,
    uint32_t transactionId, uint8_t sourceSlot)
{
    if ((blob == NULL) || (blobBytes != sizeof(menuFlashBlobStruct)) ||
        (menuFlashBlobExactValid((const menuFlashBlobStruct *)
            (const void *) blob) == 0u)) return 0u;
    return (uint8_t) (paramSlotStorePrepareRuntime(&g_paramSlots, blob,
        blobBytes, transactionId, sourceSlot) == PARAM_SLOT_OK);
}

uint8_t menuParamRuntimePrepareFromSlot(uint8_t slot,
    uint32_t transactionId)
{
    return (uint8_t) (paramSlotStorePrepareRuntimeFromSlot(&g_paramSlots,
        slot, transactionId) == PARAM_SLOT_OK);
}

uint8_t menuParamRuntimeApplyPrepared(uint32_t transactionId)
{
    menuFlashBlobStruct blob;
    if (paramSlotStoreReadPreparedRuntime(&g_paramSlots, transactionId,
        &blob, sizeof(blob)) != PARAM_SLOT_OK) return 0u;
    menuFlashApplyBlob(&blob);
    applyVisionLocal();
    seqParam++;
    menuPushParams();
    return 1u;
}

uint8_t menuParamRuntimeCommit(uint32_t transactionId)
{
    return (uint8_t) (paramSlotStoreCommitRuntime(&g_paramSlots,
        transactionId) == PARAM_SLOT_OK);
}

```

core0_cyt4bb7/project/code/app/menuRender.c

SHA-256

:

5bfa4319a91bb148a39aada00877c847f0cc95e028a3c525cb91a54344b5fa2d

代码 (c)

```

#include "menu.h"
#include "menuInternal.h"
#include "menuParamSlots.h"

```

```

#include <math.h>
#include <stdarg.h>
#include <stdio.h>
#include <string.h>

```

```

#include "zf_device_ips114.h"
#include "zf_device_mt9v03x.h"
#include "zf_common_font.h"

```

```

#include "commonDef.h"
#include "image.h"
#include "coreDataTransfer.h"
#include "configVision.h"
#include "menuFloatDisplayInternal.h"

```

```

/* 三态列表行配色 (统一规范 docs/MENU_UI_CONVENTION.md §3: 名/值始终不同前景色) :
 * 普通 底黑/名绿/值白 ; 选中 底青/名黑/值蓝 ; 编辑 底黄/名黑/值红 */
#define MENU_NORM_NAME_FG (RGB565_GREEN)
#define MENU_NORM_VAL_FG (RGB565_WHITE)
#define MENU_SEL_BG      (RGB565_CYAN)
#define MENU_SEL_NAME_FG (RGB565_BLACK)
#define MENU_SEL_VAL_FG  (RGB565_BLUE)
#define MENU_EDIT_BG     (RGB565_YELLOW)
#define MENU_EDIT_NAME_FG (RGB565_BLACK)
#define MENU_EDIT_VAL_FG (RGB565_RED)
#define MENU_PARAM_VALUE_X      (COL_X(14))

```

```

#define MENU_MARK_X          (COL_X(1))
#define MENU_ITEM_X          (COL_X(2))
#define MENU_SOLID_FILL_PIXELS  (240u)
#define THRESH_SLOT_LOW_X    (2u)
#define THRESH_SLOT_HIGH_X   (82u)
#define THRESH_SLOT_EXPOSURE_X (162u)
#define THRESH_SLOT_LOW_WIDTH (72u)
#define THRESH_SLOT_HIGH_WIDTH (72u)
#define THRESH_SLOT_EXPOSURE_WIDTH (76u)
#define THRESH_NAME_LOW_X    (10u)
#define THRESH_NAME_HIGH_X   (90u)
#define THRESH_NAME_EXPOSURE_X (170u)
#define THRESH_VALUE_LOW_X   (42u)
#define THRESH_VALUE_HIGH_X  (122u)
#define THRESH_VALUE_EXPOSURE_X (202u)

/*
 * menuRender.c — IPS114 8×16 渲染（绘制原语 + 各状态渲染），仅消费共享状态不修改。
 * 布局原则（消除闪烁/伪影）：任何分隔线都落在文字/图像的空白间隙，
 * 绝不与字模或图像矩形像素重叠。8×16 字体行高固定 16px。
 */

/* 相机图像坐标 → 发车页相机显示区坐标（显示区 116×72，左上偏移 (2,2)）。
 * 目标中点 / 轮廓质心叠加共用，消除重复魔数。 */
#define HOME_CAM_X (2u)
#define HOME_CAM_Y (17u)
#define HOME_CAM_W (130u)
#define HOME_CAM_H (67u)
#define HOME_VLINE_X (133u)
#define HOME_STATUS_X (135u)
#define HOME_SLOT_COUNT (13u)
#define HOME_SLOT_CHARS (39u)

static char homeSlotCache[HOME_SLOT_COUNT][HOME_SLOT_CHARS + 1u];
static uint8_t homeSlotValid[HOME_SLOT_COUNT];
static uint16_t homeSlotColor[HOME_SLOT_COUNT];
static uint16_t homeSlotBackground[HOME_SLOT_COUNT];
static uint16_t homeTitleColor = 0xFFFFu;
static uint16_t menuSolidFillBuffer[MENU_SOLID_FILL_PIXELS];

static void fillSolidRect(uint16_t x, uint16_t y, uint16_t w, uint16_t h,
    uint16_t color)
{
    uint16_t rowsPerTransfer;
    uint16_t rows;
    uint16_t i;

    if ((w == 0u) || (h == 0u) || (x >= LCD_W) || (y >= LCD_H)) return;
    if ((uint32_t) x + w > LCD_W) w = (uint16_t) (LCD_W - x);
    if ((uint32_t) y + h > LCD_H) h = (uint16_t) (LCD_H - y);
    if (w > MENU_SOLID_FILL_PIXELS) return;
    rowsPerTransfer = (uint16_t) (MENU_SOLID_FILL_PIXELS / w);
    for (i = 0u; i < MENU_SOLID_FILL_PIXELS; i++) menuSolidFillBuffer[i] = color;
    while (h > 0u)
    {
        rows = (h < rowsPerTransfer) ? h : rowsPerTransfer;
        ips114_show_rgb565_image(x, y, menuSolidFillBuffer,
            w, rows, w, rows, 0u);
        y = (uint16_t) (y + rows);
        h = (uint16_t) (h - rows);
    }
}

static void drawHomeSlot(uint8_t slot, uint16_t x, uint16_t y,
    uint8_t width, uint16_t color, uint16_t background, const char *format, ...)
{
    char line[HOME_SLOT_CHARS + 1u];
    char formatted[48];
    size_t length;
    va_list args;
    if ((slot >= HOME_SLOT_COUNT) || (width > HOME_SLOT_CHARS)) return;
    va_start(args, format);
    (void) vsnprintf(formatted, sizeof(formatted), format, args);
    va_end(args);
    memset(line, 0, sizeof(line));
    memset(line, ' ', width);
    line[width] = '\0';
    length = strlen(formatted);
    if (length > width) length = width;

```

```

memcpy(line, formatted, length);
if ((homeSlotValid[slot] != 0u) &&
    (homeSlotColor[slot] == color) &&
    (homeSlotBackground[slot] == background) &&
    (memcmp(homeSlotCache[slot], line, sizeof(line)) == 0)) return;
memcpy(homeSlotCache[slot], line, sizeof(line));
homeSlotValid[slot] = 1u;
homeSlotColor[slot] = color;
homeSlotBackground[slot] = background;
ips114_set_font(IPS114_6X8_FONT);
ips114_set_color(color, background);
ips114_show_string(x, y, line);
}

static inline uint16_t imgToDispX(uint32_t imgX)
{
    return (uint16_t) (HOME_CAM_X + imgX * HOME_CAM_W / (uint32_t) MT9V03X_W);
}
static inline uint16_t imgToDispY(uint32_t imgY)
{
    return (uint16_t) (HOME_CAM_Y + imgY * HOME_CAM_H / (uint32_t) MT9V03X_H);
}

static void drawHomeBinaryCcEdgeFrame(void)
{
    uint32_t m;
    uint16_t x0;
    uint16_t y0;
    uint16_t x1;
    uint16_t y1;

    if (imageGetDetectMode() != VISION_DETECT_MODE_BINARYCC) return;

    m = (uint32_t) (binaryCcOrientBorderMarginF + 0.5f);
    if (m == 0u) return;
    if (m == (uint32_t) (MT9V03X_W / 2u) || m >= (uint32_t) (MT9V03X_H / 2u)) return;

    x0 = imgToDispX(m);
    y0 = imgToDispY(m);
    x1 = imgToDispX((uint32_t) (MT9V03X_W - 1u) - m);
    y1 = imgToDispY((uint32_t) (MT9V03X_H - 1u) - m);

    if (x0 < HOME_CAM_X) x0 = HOME_CAM_X;
    if (y0 < HOME_CAM_Y) y0 = HOME_CAM_Y;
    if (x1 > HOME_CAM_X + HOME_CAM_W - 1u) x1 = HOME_CAM_X + HOME_CAM_W - 1u;
    if (y1 > HOME_CAM_Y + HOME_CAM_H - 1u) y1 = HOME_CAM_Y + HOME_CAM_H - 1u;
    if (x0 >= x1 || y0 >= y1) return;

    ips114_draw_line(x0, y0, x1, y0, RGB565_RED);
    ips114_draw_line(x0, y1, x1, y1, RGB565_RED);
    ips114_draw_line(x0, y0, x0, y1, RGB565_RED);
    ips114_draw_line(x1, y0, x1, y1, RGB565_RED);
}

static uint16_t menuOwnerColor(void)
{
    uint8_t owner = UART_LINK_KVM_TARGET_NONE;
    if ((flightSnapshotAvailable != 0u) &&
        (flightRx.cm70Ready == CORE_DATA_READY_MAGIC) &&
        (flightRx.core1LinkUp != 0u) &&
        (kvmRx.magic == CORE_DATA_READY_MAGIC) &&
        (kvmRx.routeSession == bootEpoch) &&
        (kvmRx.routeEpoch == menuKvmRouteEpoch) &&
        (kvmRx.target == menuKeyTargetCore) &&
        ((kvmRx.flags & (UART_LINK_KVM_FLAG_ROUTE_ACCEPTED | UART_LINK_KVM_FLAG_LEASE_VALID)) ==
         (UART_LINK_KVM_FLAG_ROUTE_ACCEPTED | UART_LINK_KVM_FLAG_LEASE_VALID)))
    {
        owner = kvmRx.activeTarget;
    }
    if (owner == UART_LINK_KVM_TARGET_NONE) return RGB565_YELLOW;
    return (owner == UART_LINK_KVM_TARGET_CORE0) ? RGB565_GREEN : RGB565_RED;
}

static void drawFrameBorder(void)
{
    uint16_t color = menuOwnerColor();
    ips114_draw_line(0, 0, LCD_W - 1u, 0, color); /* 顶 */
    ips114_draw_line(0, LCD_H - 1u, LCD_W - 1u, LCD_H - 1u, color); /* 底 */
    ips114_draw_line(0, 0, 0, LCD_H - 1u, color); /* 左 */

```

```

        ips114_draw_line(LCD_W - 1u, 0, LCD_W - 1u, LCD_H - 1u, color); /* 右 */
    }

/* 中点(Bresenham)画圆，逐点钳在发车页相机显示区 [x0,x1]×[y0,y1] 内。
 * 仅用于发车页死区圆标注；SDK 无 ips114_draw_circle，用 8 对称 draw_point 实现。 */
static void drawCircleClamped(int16_t cx, int16_t cy, int16_t r, uint16_t color,
    int16_t x0, int16_t y0, int16_t x1, int16_t y1)
{
    int16_t x = r;
    int16_t y = 0;
    int16_t err = 1 - r;
    if (r <= 0) return;

    while (x >= y)
    {
        int16_t px[8];
        int16_t py[8];
        uint8_t i;
        px[0] = cx + x; py[0] = cy + y;
        px[1] = cx + y; py[1] = cy + x;
        px[2] = cx - y; py[2] = cy + x;
        px[3] = cx - x; py[3] = cy + y;
        px[4] = cx - x; py[4] = cy - y;
        px[5] = cx - y; py[5] = cy - x;
        px[6] = cx + y; py[6] = cy - x;
        px[7] = cx + x; py[7] = cy - y;
        for (i = 0u; i < 8u; i++)
        {
            if (px[i] >= x0 && px[i] <= x1 && py[i] >= y0 && py[i] <= y1)
            {
                ips114_draw_point((uint16_t) px[i], (uint16_t) py[i], color);
            }
        }
        y++;
        if (err < 0)
        {
            err += 2 * y + 1;
        }
        else
        {
            x--;
            err += 2 * (y - x) + 1;
        }
    }
}

/* 反相标题条：标准页用黑字绿底，等待/报警等特殊页可传入异色背景。 */
static void drawTitleBar(const char* title, uint16_t fg, uint16_t bg)
{
    char bounded[40];
    size_t len = strlen(title);
    uint16_t x;

    if (len > 39u) len = 39u;
    memcpy(bounded, title, len);
    bounded[len] = '\0';
    x = (uint16_t) ((LCD_W - (uint16_t) (len * 6u)) / 2u);
    if (x < 2u) x = 2u;
    fillSolidRect(1u, 1u, (uint16_t) (LCD_W - 2u), 15u, bg);
    ips114_set_font(IPS114_6X8_FONT);
    ips114_set_color(fg, bg);
    ips114_show_string(x, 4u, bounded);
    ips114_set_font(IPS114_8X16_FONT);
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
}

static void fillRect(uint16_t x, uint16_t y, uint16_t w, uint16_t h, uint16_t color)
{
    {
        fillSolidRect(x, y, w, h, color);
    }
}

/* 列表右侧滚动滑块：轨道贴右内框，滑块高/位按 visible/total 比例。
 * 仅在 total>visible 时绘制。轨道 x 区间 [SCROLLBAR_X, LCD_W-2]，
 * y 区间 [trackTop, trackBot]，与文字列不重叠（文字止于数值列）。 */
static void drawScrollbar(uint8_t total, uint8_t top)
{
    {
        uint16_t trackTop = ROW_Y(1);
        uint16_t trackBot = (uint16_t) (LCD_H - 2u);
        uint16_t trackH;

```

```

uint16_t thumbH;
uint16_t thumbY;

if (total <= VISIBLE_ROWS) return;
trackH = (uint16_t) (trackBot - trackTop);

/* 自清槽：先清整个槽区(轨道 + 旧滑块) → 消滚动时旧滑块横线残留。不触右外框(x=LCD_W-1)。 */
for (uint16_t cy = trackTop; cy < trackBot; cy++)
{
    ips114_draw_line(SCROLLBAR_X, cy, (uint16_t) (LCD_W - 2u), cy, RGB565_BLACK);
}

/* 轨道竖线（左右两条勾勒细槽） */
ips114_draw_line(SCROLLBAR_X, trackTop, SCROLLBAR_X, trackBot, RGB565_GREEN);
ips114_draw_line((uint16_t) (LCD_W - 2u), trackTop, (uint16_t) (LCD_W - 2u), trackBot, RGB565_GREEN);

/* 滑块高度 = 轨道 × visible/total, 位置 = 轨道 × top/total */
thumbH = (uint16_t) ((uint32_t) trackH * VISIBLE_ROWS / total);
if (thumbH < 6u) thumbH = 6u;
thumbY = (uint16_t) (trackTop + (uint32_t) (trackH - thumbH) * top / (uint16_t) (total - VISIBLE_ROWS));

/* 实心滑块：逐行横线填充槽内（x 留 1px 内缩，避免压槽壁） */
for (uint16_t yy = thumbY; yy < thumbY + thumbH && yy < trackBot; yy++)
{
    ips114_draw_line((uint16_t) (SCROLLBAR_X + 1u), yy, (uint16_t) (LCD_W - 3u), yy, RGB565_GREEN);
}
}

static uint8_t homeFooterDirty = 1u;
static uint8_t homeLastKeyTarget = 0xFFu;
static uint8_t homeLastC1Up = 0xFFu;
static uint8_t homeLastC2Up = 0xFFu;
static uint8_t homeLastVisionUp = 0xFFu;
static uint8_t homeLastInputOwner = 0xFFu;

static void drawHomeFooter(void)
{
    uint8_t c1up = (flightRx.cm70Ready == CORE_DATA_READY_MAGIC) ? flightRx.core1LinkUp : 0u;
    uint8_t c2up = (uint8_t) ((c1up != 0u) && (flightRx.core2LinkLost == 0u));
    uint8_t visionUp = (uint8_t) ((c1up != 0u) && (flightRx.requiredVisionFault == 0u));
    uint8_t target = menuKeyTargetCore;
    const char *targetText = (target == UART_LINK_KVM_TARGET_CORE0) ? "C0" :
        ((target == UART_LINK_KVM_TARGET_CORE1) ? "C1" :
        ((target == UART_LINK_KVM_TARGET_CORE3) ? "C3" : "C4"));
    const char *ownerText = (kvmRx.activeTarget == UART_LINK_KVM_TARGET_CORE0) ? "C0" :
        ((kvmRx.activeTarget == UART_LINK_KVM_TARGET_CORE1) ? "C1" :
        ((kvmRx.activeTarget == UART_LINK_KVM_TARGET_CORE3) ? "C3" :
        ((kvmRx.activeTarget == UART_LINK_KVM_TARGET_CORE4) ? "C4" : "NONE")));
    uint8_t statusChanged = (uint8_t) (homeFooterDirty
        || homeLastKeyTarget != target || homeLastC1Up != c1up || homeLastC2Up != c2up
        || homeLastVisionUp != visionUp || homeLastInputOwner != kvmRx.activeTarget);

    if (statusChanged)
    {
        drawHomeSlot(11u, 2u, 106u, 39u, menuOwnerColor(), RGB565_BLACK,
            "KEY:%s>%s C1:%s C2:%s VIS:%s", targetText, ownerText,
            c1up ? "OK" : "LOST", c2up ? "OK" : "LOST",
            visionUp ? "OK" : "LOST");

        homeFooterDirty = 0u;
        homeLastKeyTarget = target;
        homeLastC1Up = c1up;
        homeLastC2Up = c2up;
        homeLastVisionUp = visionUp;
        homeLastInputOwner = kvmRx.activeTarget;
    }

    {
        const char *phase = (flightRx.systemPhase == 2u) ? "LINK WAIT" :
            ((flightRx.systemPhase == 4u) ? "REJOIN" :
            ((flightRx.systemPhase == 5u) ? "STANDALONE" : "HOME"));
        const char *reason = (flightRx.primaryReason == 1u) ? "C1 LINK" :
            ((flightRx.primaryReason == 2u) ? "C2 LINK" :
            ((flightRx.primaryReason == 3u) ? "VISION" :
            ((flightRx.primaryReason == 4u) ? "C2 ATT" : "READY")));
        uint16_t color = (flightRx.primaryReason == 0u) ? RGB565_GRAY : RGB565_YELLOW;
        drawHomeSlot(12u, 2u, 121u, 39u, color, RGB565_BLACK,
            "%s %s", phase, reason);
    }
}

```

```

}

/* =====
* 渲染：ToF 上电复位与初始化重试。
* ===== */
void renderTofInit(void)
{
    ips114_clear();
    menuRenderResetListSnapshot();
    ips114_set_color(GB565_GREEN, GB565_BLACK);
    drawTitleBar("TOF INIT", GB565_BLACK, GB565_YELLOW);
    ips114_show_string(COL_X(8), ROW_Y(4), "Please wait...");
    drawFrameBorder();
}

/* =====
* 渲染：ESC arming 进度条（开机阻塞期间，cm_7_0 回报 0xFE+progress）
* 居中标题 + 中部进度条 + 百分比。整屏布局，无图像。
* ===== */
void renderArming(uint8_t progress)
{
    /* 进度条几何：居中，宽 200，高 20 */
    const uint16_t barX = 20u;
    const uint16_t barY = 60u;
    const uint16_t barW = 200u;
    const uint16_t barH = 20u;
    uint16_t fillW = (uint16_t)((uint32_t)(barW - 2u) * progress / 100u);

    /* 静态元素（标题/分隔线/提示/外框/边框）仅首帧绘制一次：armBarLast==0xFF 表示尚未画过。
    * 进度变化时只刷填充条与百分比，避免每帧重画标题导致闪烁。 */
    if (armBarLast == 0xFFu)
    {
        ips114_set_color(GB565_GREEN, GB565_BLACK);
        drawTitleBar("ESC ARMING", GB565_BLACK, GB565_YELLOW);
        ips114_show_string(COL_X(8), ROW_Y(2), "Please wait...");
        ips114_draw_line(barX, barY, (uint16_t)(barX + barW), barY, GB565_GREEN);
        ips114_draw_line(barX, (uint16_t)(barY + barH), (uint16_t)(barX + barW), (uint16_t)(barY + barH), GB565_GREEN);
        ips114_draw_line(barX, barY, barX, (uint16_t)(barY + barH), GB565_GREEN);
        ips114_draw_line((uint16_t)(barX + barW), barY, (uint16_t)(barX + barW), (uint16_t)(barY + barH), GB565_GREEN);
        ips114_show_string(COL_X(16), ROW_Y(6), "%");
        drawFrameBorder();
    }

    ips114_set_color(GB565_GREEN, GB565_BLACK);

    /* 填充（逐行横线，框内 barX+1..barX+barW-1） */
    for (uint16_t yy = (uint16_t)(barY + 2u); yy < (uint16_t)(barY + barH - 1u); yy++)
    {
        ips114_draw_line((uint16_t)(barX + 1u), yy, (uint16_t)(barX + 1u + fillW), yy, GB565_GREEN);
    }

    /* 百分比（条下方居中，3 位定宽覆盖旧值） */
    ips114_show_int(COL_X(13), ROW_Y(6), (int32_t)progress, 3u);
}

/* =====
* 渲染：发车页（左灰度图 + 右数据 + 底部状态/按键）
* 布局（240×135，无坐标重叠）：
* 左图 x2..117 y2..73 （116×72）
* 竖分隔 x120 y1..76
* 右数据 x124.. y2,17,32,47,62(5 行：ToF/Vld/姿态 P R/Y/速度 V)
* 横分隔 y78 x1..238
* 底部 y80(状态) y96(键提示 1) y112(键提示 2)
* ===== */
void renderHome(void)
{
    imageObservationSnapshotStruct observation;
    imageFrameViewStruct originalView;
    const char *flightState;
    uint16_t titleColor;
    imageGetObservationSnapshot(&observation);

    switch (flightRx.safetyState)
    {
        case FC_STATE_DISARM: flightState = "DISARM"; break;
        case FC_STATE_FLYING: flightState = "FLYING"; break;
        case FC_STATE_PROTECT: flightState = "PROTECT"; break;
        case FC_STATE_IMUFAIL: flightState = "IMUFAIL"; break;
        case FC_STATE_ARMING: flightState = "ARMING"; break;
    }

```

```

        default: flightState = "WAIT"; break;
    }
    titleColor = ((flightSnapshotAvailable != 0u) &&
        ((flightRx.safetyState == FC_STATE_PROTECT) ||
        (flightRx.safetyState == FC_STATE_IMUFAIL))) ? RGB565_RED :
        ((flightSnapshotAvailable == 0u) ||
        (flightRx.cm70Ready != CORE_DATA_READY_MAGIC) ||
        (flightRx.safetyState == FC_STATE_ARMING) ||
        (flightRx.systemPhase != CORE_UX_PHASE_HOME) ||
        (flightRx.core1LinkUp == 0u)) ? RGB565_YELLOW : RGB565_GREEN;
    if (homeTitleColor != titleColor)
    {
        drawTitleBar("CORE0 FLIGHT", RGB565_BLACK, titleColor);
        homeTitleColor = titleColor;
    }

/* 左侧灰度图（内缩进框，避免与外框/竖分隔重叠；判空防相机未就绪/图像指针异常崩溃，仿 core1） */
{
    const uint8_t* pixels = imageGetFrameView(IMAGE_FRAME_ORIGINAL, &originalView) ?
        originalView.pixels : 0;
    if (pixels != 0)
    {
        ips114_show_gray_image(HOME_CAM_X, HOME_CAM_Y, pixels,
            originalView.width, originalView.height, HOME_CAM_W, HOME_CAM_H, 0u);
    }
}

/* BinaryCC 朝向边缘门安全框：红框外侧区域不再采信新朝向，仅保持/失效旧朝向。 */
drawHomeBinaryCcEdgeFrame();

/* 叠加目标参考：目标中点青色十字 + 目标角度黄色短线。 */
{
    uint16_t tx = imgToDispX((uint32_t) menuVisionTuning.targetCenterX);
    uint16_t ty = imgToDispY((uint32_t) menuVisionTuning.targetCenterY);
    if (tx < HOME_CAM_X) tx = HOME_CAM_X;
    else if (tx > HOME_CAM_X + HOME_CAM_W - 1u) tx = HOME_CAM_X + HOME_CAM_W - 1u;
    if (ty < HOME_CAM_Y) ty = HOME_CAM_Y;
    else if (ty > HOME_CAM_Y + HOME_CAM_H - 1u) ty = HOME_CAM_Y + HOME_CAM_H - 1u;

    {
        uint16_t x0 = (tx > HOME_CAM_X + 3u) ? (uint16_t) (tx - 3u) : HOME_CAM_X;
        uint16_t x1 = (tx + 3u < HOME_CAM_X + HOME_CAM_W) ?
            (uint16_t) (tx + 3u) : (uint16_t) (HOME_CAM_X + HOME_CAM_W - 1u);
        uint16_t y0 = (ty > HOME_CAM_Y + 3u) ? (uint16_t) (ty - 3u) : HOME_CAM_Y;
        uint16_t y1 = (ty + 3u < HOME_CAM_Y + HOME_CAM_H) ?
            (uint16_t) (ty + 3u) : (uint16_t) (HOME_CAM_Y + HOME_CAM_H - 1u);
        ips114_draw_line(x0, ty, x1, ty, RGB565_CYAN);
        ips114_draw_line(tx, y0, tx, y1, RGB565_CYAN);
    }

/* 施密特死区双圆标注（圆心=目标中点）：大圆黄(进入阈值)、小圆品红(退出阈值)。
 * 半径按显示高 72 缩放(死区基准=图像高 H)： r_disp = 0.5*pct/100*72 = pct*0.36。 */
{
    int16_t enterR = (int16_t) (menuVisionTuning.deadzoneEnterPct *
        (0.5f * (float) HOME_CAM_H / 100.0f) + 0.5f);
    int16_t exitR = (int16_t) (menuVisionTuning.deadzoneExitPct *
        (0.5f * (float) HOME_CAM_H / 100.0f) + 0.5f);
    drawCircleClamped((int16_t) tx, (int16_t) ty, enterR, RGB565_YELLOW,
        HOME_CAM_X, HOME_CAM_Y, HOME_CAM_X + HOME_CAM_W - 1u,
        HOME_CAM_Y + HOME_CAM_H - 1u);
    drawCircleClamped((int16_t) tx, (int16_t) ty, exitR, RGB565_MAGENTA,
        HOME_CAM_X, HOME_CAM_Y, HOME_CAM_X + HOME_CAM_W - 1u,
        HOME_CAM_Y + HOME_CAM_H - 1u);
}

{
    float ox = cosf(menuVisionTuning.targetAngleRad);
    float oy = sinf(menuVisionTuning.targetAngleRad);
    int16_t ex = (int16_t) ((float) tx + ox * 12.0f);
    int16_t ey = (int16_t) ((float) ty + oy * 12.0f);
    if (ex < (int16_t) HOME_CAM_X) ex = (int16_t) HOME_CAM_X;
    else if (ex > (int16_t) (HOME_CAM_X + HOME_CAM_W - 1u)) ex =
        (int16_t) (HOME_CAM_X + HOME_CAM_W - 1u);
    if (ey < (int16_t) HOME_CAM_Y) ey = (int16_t) HOME_CAM_Y;
    else if (ey > (int16_t) (HOME_CAM_Y + HOME_CAM_H - 1u)) ey =
        (int16_t) (HOME_CAM_Y + HOME_CAM_H - 1u);
    ips114_draw_line(tx, ty, (uint16_t) ex, (uint16_t) ey, RGB565_YELLOW);
}
}
}

```

```

/* 叠加识别结果：^ 质心十字 + 朝向短线（图像坐标→显示坐标缩放，红色突出于灰度图）。
* 显示区 (2,2)..(117,73)，116×72；相机 MT9V03X_W×H。 */
if (observation.contourCount > 0u && !observation.targetLost)
{
    uint16_t cxImg = observation.selectedCenterX;
    uint16_t cyImg = observation.selectedCenterY;
    uint16_t sx = imgToDispX(cxImg);
    uint16_t sy = imgToDispY(cyImg);
    if (sx < HOME_CAM_X) sx = HOME_CAM_X;
    else if (sx > HOME_CAM_X + HOME_CAM_W - 1u) sx = HOME_CAM_X + HOME_CAM_W - 1u;
    if (sy < HOME_CAM_Y) sy = HOME_CAM_Y;
    else if (sy > HOME_CAM_Y + HOME_CAM_H - 1u) sy = HOME_CAM_Y + HOME_CAM_H - 1u;

    /* 质心十字（±4px，钳在显示区内） */
    {
        uint16_t x0 = (sx > HOME_CAM_X + 4u) ? (uint16_t) (sx - 4u) : HOME_CAM_X;
        uint16_t x1 = (sx + 4u < HOME_CAM_X + HOME_CAM_W) ?
            (uint16_t) (sx + 4u) : (uint16_t) (HOME_CAM_X + HOME_CAM_W - 1u);
        uint16_t y0 = (sy > HOME_CAM_Y + 4u) ? (uint16_t) (sy - 4u) : HOME_CAM_Y;
        uint16_t y1 = (sy + 4u < HOME_CAM_Y + HOME_CAM_H) ?
            (uint16_t) (sy + 4u) : (uint16_t) (HOME_CAM_Y + HOME_CAM_H - 1u);
        ips114_draw_line(x0, sy, x1, sy, RGB565_RED);
        ips114_draw_line(sx, y0, sx, y1, RGB565_RED);
    }

    /* 朝向短线：从质心沿 orientation 方向延伸 ~14px（屏幕 y 向下，取 +sin） */
    {
        float ox = cosf(observation.orientation);
        float oy = sinf(observation.orientation);
        int16_t ex = (int16_t) ((float) sx + ox * 14.0f);
        int16_t ey = (int16_t) ((float) sy + oy * 14.0f);
        if (ex < (int16_t) HOME_CAM_X) ex = (int16_t) HOME_CAM_X;
        else if (ex > (int16_t) (HOME_CAM_X + HOME_CAM_W - 1u)) ex =
            (int16_t) (HOME_CAM_X + HOME_CAM_W - 1u);
        if (ey < (int16_t) HOME_CAM_Y) ey = (int16_t) HOME_CAM_Y;
        else if (ey > (int16_t) (HOME_CAM_Y + HOME_CAM_H - 1u)) ey =
            (int16_t) (HOME_CAM_Y + HOME_CAM_H - 1u);
        ips114_draw_line(sx, sy, (uint16_t) ex, (uint16_t) ey,
            observation.orientBaselineValid ? RGB565_RED : RGB565_GRAY);
    }
}

drawHomeSlot(0u, HOME_STATUS_X, 18u, 17u,
    flightRx.toHealthy ? RGB565_GREEN : RGB565_RED, RGB565_BLACK,
    "ALT:%ldmm %s", (long) flightRx.altMm,
    flightRx.toHealthy ? "OK" : "BAD");
drawHomeSlot(1u, HOME_STATUS_X, 26u, 17u, RGB565_WHITE, RGB565_BLACK,
    "P:%ld R:%ld", (long) toDeg(flightRx.pitch), (long) toDeg(flightRx.roll));
drawHomeSlot(2u, HOME_STATUS_X, 34u, 17u, RGB565_WHITE, RGB565_BLACK,
    "Y:%ld", (long) toDeg(flightRx.yaw));
drawHomeSlot(3u, HOME_STATUS_X, 42u, 17u,
    flightRx.velValid ? RGB565_GREEN : RGB565_RED, RGB565_BLACK,
    flightRx.velValid ? "VX:%+.1f VY:%+.1f" : "VX:-- VY:--",
    (double) flightRx.velX, (double) flightRx.velY);
drawHomeSlot(4u, HOME_STATUS_X, 50u, 17u,
    observation.valid ? RGB565_GREEN : RGB565_RED, RGB565_BLACK,
    "VISION:%s", observation.valid ? "TARGET" : "LOST");
drawHomeSlot(5u, HOME_STATUS_X, 58u, 17u, RGB565_CYAN, RGB565_BLACK,
    "SYS:%u", menuFlightWork.systemMode);
drawHomeSlot(6u, HOME_STATUS_X, 66u, 17u,
    (flightRx.safetyState == FC_STATE_PROTECT || flightRx.safetyState == FC_STATE_IMUFAIL) ?
        RGB565_RED : RGB565_WHITE, RGB565_BLACK, "FC:%s T:%u", flightState,
    (unsigned int) flightRx.baseThr);
drawHomeSlot(7u, HOME_STATUS_X, 74u, 17u,
    (flightRx.toHealthy && flightRx.core1LinkUp) ? RGB565_GREEN : RGB565_RED,
    RGB565_BLACK, "GATE:%s",
    (flightRx.toHealthy && flightRx.core1LinkUp) ? "READY" : "BLOCK");
drawHomeSlot(8u, HOME_STATUS_X, 82u, 17u,
    flightRx.core1LinkUp ? RGB565_GREEN : RGB565_RED, RGB565_BLACK,
    "C1:%s C2:%s", flightRx.core1LinkUp ? "OK" : "LOST",
    (!flightRx.core2LinkLost && flightRx.core1LinkUp) ? "OK" : "LOST");
drawHomeSlot(9u, HOME_STATUS_X, 90u, 17u,
    flightRx.requiredVisionFault ? RGB565_RED : RGB565_GREEN,
    RGB565_BLACK, "VIS:%s",
    flightRx.requiredVisionFault ? "FAULT" : "OK");
/* 底部两行做局部刷新：第二行固定指南只在清屏后重画，避免 100ms 空刷闪烁。 */
drawHomeFooter();

```



```

/* 外框与发车页内部分隔线颜色按操控权:
* 本地操控-绿, 转发给远端目标时为红。 */
{
    uint16_t bc = menuOwnerColor();
    ips114_draw_line(0, 0, LCD_W - 1u, 0, bc);
    ips114_draw_line(0, LCD_H - 1u, LCD_W - 1u, LCD_H - 1u, bc);
    ips114_draw_line(0, 0, 0, LCD_H - 1u, bc);
    ips114_draw_line(LCD_W - 1u, 0, LCD_W - 1u, LCD_H - 1u, bc);
    ips114_draw_line(HOME_VLINE_X, 17u, HOME_VLINE_X, 103u, bc);
    ips114_draw_line(1u, 104u, LCD_W - 2u, 104u, RGB565_GRAY);
    ips114_draw_line(1u, 119u, LCD_W - 2u, 119u, RGB565_GRAY);
}

static const char* faultReasonText(uint8_t reason)
{
    if ((reason & CORE0_FAULT_REASON_VISION_FLAG) != 0u)
    {
        switch (CORE0_FAULT_REASON_VISION_CLASS(reason))
        {
            case 1u: return "VISION CAMERA";
            case 2u: return "VISION LINK";
            case 3u: return "VISION BRIDGE";
            case 4u: return "VISION PROTO";
            case 5u: return "VISION CONFIG";
            case 6u: return "VISION CALIB";
            case 7u: return "VISION POSE";
            default: return "VISION UNKNOWN";
        }
    }
    switch (reason)
    {
        case CORE0_FAULT_REASON_PROTECT_ATTITUDE: return "PROTECT ATT";
        case CORE0_FAULT_REASON_LAND_C1_LINK: return "LAND C1 LINK";
        case CORE0_FAULT_REASON_LAND_C2_LINK: return "LAND C2 LINK";
        case CORE0_FAULT_REASON_REQUIRED_VISION: return "LAND VISION";
        case CORE0_FAULT_REASON_LAND_LORA_MODE3: return "LAND LORA M3";
        case CORE0_FAULT_REASON_LAND_LORA_MODE2: return "LAND LORA M2";
        case CORE0_FAULT_REASON_LAND_TOF_FAILSAFE: return "LAND TOF";
        case CORE0_FAULT_REASON_LAND_C2_ATTITUDE: return "LAND C2 ATT";
        default: return "UNKNOWN";
    }
}

static const char* faultSourceText(uint8_t reason)
{
    if ((reason & CORE0_FAULT_REASON_VISION_FLAG) == 0u) return "";
    switch (CORE0_FAULT_REASON_VISION_SOURCE(reason))
    {
        case 1u: return "SRC: CORE0";
        case 2u: return "SRC: CORE1";
        case 3u: return "SRC: CORE3";
        case 4u: return "SRC: CORE4";
        default: return "SRC: ?";
    }
}

static const char* safetyStateText(uint8_t state)
{
    switch (state)
    {
        case FC_STATE_DISARM: return "DISARM";
        case FC_STATE_FLYING: return "FLYING";
        case FC_STATE_PROTECT: return "PROTECT";
        case FC_STATE_IMUFAIL: return "IMUFAIL";
        case FC_STATE_ARMING: return "ARMING";
        default: return "WAIT";
    }
}

void renderFaultReasonPage(uint8_t reason, uint8_t state,
    uint8_t fullRedraw, uint8_t reasonChanged, uint8_t stateChanged)
{
    uint16_t titleColor = (reason == CORE0_FAULT_REASON_PROTECT_ATTITUDE) ?
        RGB565_RED : RGB565_YELLOW;

    if (fullRedraw != 0u)
    {
        drawTitleBar("CORE0 FAULT", RGB565_BLACK, titleColor);
    }
}

```

```

        fillRect(1u, 16u, (uint16_t) (LCD_W - 2u),
            (uint16_t) (LCD_H - 17u), RGB565_BLACK);
        ips114_set_color(RGB565_RED, RGB565_BLACK);
        ips114_show_string(12u, 32u, "FAULT LATCHED");
        ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
        ips114_show_string(12u, 56u, "RSN:");
        ips114_set_color(RGB565_CYAN, RGB565_BLACK);
        ips114_show_string(12u, 80u, "STATE:");
        ips114_set_color(RGB565_GREEN, RGB565_BLACK);
        ips114_show_string(12u, 112u, "K5 ACK -> HOME");
        ips114_draw_line(0u, 0u, LCD_W - 1u, 0u, titleColor);
        ips114_draw_line(0u, LCD_H - 1u, LCD_W - 1u,
            LCD_H - 1u, titleColor);
        ips114_draw_line(0u, 0u, 0u, LCD_H - 1u, titleColor);
        ips114_draw_line(LCD_W - 1u, 0u, LCD_W - 1u,
            LCD_H - 1u, titleColor);
        reasonChanged = 1u;
        stateChanged = 1u;
    }
    else if (reasonChanged != 0u)
    {
        /* 原因类别改变时标题颜色也可能由红转黄；仅按事件重画标题与边框。*/
        drawTitleBar("CORE0 FAULT", RGB565_BLACK, titleColor);
        ips114_draw_line(0u, 0u, LCD_W - 1u, 0u, titleColor);
        ips114_draw_line(0u, LCD_H - 1u, LCD_W - 1u,
            LCD_H - 1u, titleColor);
        ips114_draw_line(0u, 0u, 0u, LCD_H - 1u, titleColor);
        ips114_draw_line(LCD_W - 1u, 0u, LCD_W - 1u,
            LCD_H - 1u, titleColor);
    }
    if (reasonChanged != 0u)
    {
        fillRect(52u, 56u, 144u, 16u, RGB565_BLACK);
        ips114_set_font(IPS114_8X16_FONT);
        ips114_set_color(RGB565_WHITE, RGB565_BLACK);
        ips114_show_string(52u, 56u, faultReasonText(reason));
        fillRect(12u, 72u, 120u, 8u, RGB565_BLACK);
        ips114_set_font(IPS114_6X8_FONT);
        ips114_show_string(12u, 72u, faultSourceText(reason));
    }
    if (stateChanged != 0u)
    {
        fillRect(68u, 80u, 96u, 16u, RGB565_BLACK);
        ips114_set_font(IPS114_8X16_FONT);
        ips114_set_color(RGB565_WHITE, RGB565_BLACK);
        ips114_show_string(68u, 80u, safetyStateText(state));
    }
}

/* =====
* 渲染：检测阈值编辑器（原图 + 当前 DetectMode 预览 + Lo/Hi/Exp 热调）
* ===== */
typedef struct
{
    uint32_t originalFrameSeq;
    uint32_t previewFrameSeq;
    int32_t lowValue;
    int32_t highValue;
    int32_t exposureValue;
    uint16_t ownerColor;
    uint8_t detectMode;
    uint8_t selection;
    uint8_t originalValid;
    uint8_t previewValid;
    uint8_t valid;
} thresholdRenderSnapshotStruct;

static thresholdRenderSnapshotStruct thresholdRenderSnapshot;

static int32_t thresholdDisplayValue(float value, int32_t maximum)
{
    if (!(value == value) || (value <= 0.0f)) return 0;
    if (value >= (float) maximum) return maximum;
    return (int32_t) value;
}

static void drawThresholdValue(uint16_t slotX, uint16_t slotWidth,
    uint16_t nameX, uint16_t valueX, const char *name, uint8_t selected,
    int32_t value, uint8_t digits)

```

```

{
    char valueText[5];
    uint16_t background = selected ? MENU_SEL_BG : RGB565_BLACK;
    uint16_t nameForeground = selected ? MENU_SEL_NAME_FG : MENU_NORM_NAME_FG;
    uint16_t valueForeground = selected ? MENU_SEL_VAL_FG : MENU_NORM_VAL_FG;

    if (digits == 3u)
    {
        (void) snprintf(valueText, sizeof(valueText), "%3ld", (long) value);
    }
    else
    {
        (void) snprintf(valueText, sizeof(valueText), "%4ld", (long) value);
    }
    fillSolidRect(slotX, THR_TXT_Y, slotWidth, FONT_H, background);
    ips114_set_font(IPS114_8X16_FONT);
    ips114_set_color(nameForeground, background);
    ips114_show_string(nameX, THR_TXT_Y, name);
    ips114_set_color(valueForeground, background);
    ips114_show_string(valueX, THR_TXT_Y, valueText);
}

void renderThreshEdit(void)
{
    imageFrameViewStruct originalView;
    imageFrameViewStruct previewView;
    uint8_t originalValid = imageGetFrameView(IMAGE_FRAME_ORIGINAL, &originalView) &&
        (originalView.pixels != 0);
    uint8_t previewValid = imageGetFrameView(IMAGE_FRAME_DETECT_PREVIEW, &previewView) &&
        (previewView.pixels != 0);
    uint8_t binaryMode = (detectModeF < 0.5f) ? 1u : 0u;
    float lowValue = binaryMode ? binaryCcLowF : cannyLowF;
    float highValue = binaryMode ? binaryCcHighF : cannyHighF;
    int32_t lowDisplay = thresholdDisplayValue(lowValue, 999);
    int32_t highDisplay = thresholdDisplayValue(highValue, 999);
    int32_t exposureDisplay = thresholdDisplayValue(exposureF, 9999);
    uint16_t ownerColor = menuOwnerColor();
    uint8_t first = (thresholdRenderSnapshot.valid == 0u) ? 1u : 0u;
    uint8_t selectionChanged = (uint8_t) (first ||
        (thresholdRenderSnapshot.selection != threshSel));

    if (first || (thresholdRenderSnapshot.detectMode != binaryMode))
    {
        drawTitleBar(binaryMode ? "BINARY THR" : "CANNY THR", RGB565_BLACK, RGB565_GREEN);
    }

    if (first || (thresholdRenderSnapshot.originalValid != originalValid) ||
        (originalValid && (thresholdRenderSnapshot.originalFrameSeq != originalView.frameSeq)))
    {
        if (originalValid)
        {
            ips114_show_gray_image(THR_IMG_L_X, THR_IMG_Y, originalView.pixels,
                originalView.width, originalView.height, THR_IMG_W, THR_IMG_H, 0u);
        }
        else
        {
            fillRect(THR_IMG_L_X, THR_IMG_Y, THR_IMG_W, THR_IMG_H, RGB565_BLACK);
        }
    }
    if (first || (thresholdRenderSnapshot.previewValid != previewValid) ||
        (previewValid && (thresholdRenderSnapshot.previewFrameSeq != previewView.frameSeq)))
    {
        if (previewValid)
        {
            ips114_show_gray_image(THR_IMG_R_X, THR_IMG_Y, previewView.pixels,
                previewView.width, previewView.height, THR_IMG_W, THR_IMG_H, 0u);
        }
        else
        {
            fillRect(THR_IMG_R_X, THR_IMG_Y, THR_IMG_W, THR_IMG_H, RGB565_BLACK);
        }
    }

    if (first || (thresholdRenderSnapshot.lowValue != lowDisplay) ||
        (selectionChanged && ((threshSel == 0u) ||
            (thresholdRenderSnapshot.selection == 0u))))
    drawThresholdValue(THRESH_SLOT_LOW_X, THRESH_SLOT_LOW_WIDTH,
        THRESH_NAME_LOW_X, THRESH_VALUE_LOW_X, "Lo:",
        (uint8_t) (threshSel == 0u), lowDisplay, 3u);
}

```

```

if (first || (thresholdRenderSnapshot.highValue != highDisplay) ||
    (selectionChanged && ((threshSel == 1u) ||
        (thresholdRenderSnapshot.selection == 1u))))
    drawThresholdValue(THRESH_SLOT_HIGH_X, THRESH_SLOT_HIGH_WIDTH,
        THRESH_NAME_HIGH_X, THRESH_VALUE_HIGH_X, "Hi.",
        (uint8_t) (threshSel == 1u), highDisplay, 3u);
if (first || (thresholdRenderSnapshot.exposureValue != exposureDisplay) ||
    (selectionChanged && ((threshSel == 2u) ||
        (thresholdRenderSnapshot.selection == 2u))))
    drawThresholdValue(THRESH_SLOT_EXPOSURE_X, THRESH_SLOT_EXPOSURE_WIDTH,
        THRESH_NAME_EXPOSURE_X, THRESH_VALUE_EXPOSURE_X, "Ex.",
        (uint8_t) (threshSel == 2u), exposureDisplay, 4u);

if (first || (thresholdRenderSnapshot.ownerColor != ownerColor))
{
    drawFrameBorder();
    ips114_draw_line(120u, ROW_Y(1), 120u, 96u, ownerColor);
    ips114_draw_line(1u, 96u, LCD_W - 2u, 96u, ownerColor);
}

thresholdRenderSnapshot.originalFrameSeq = originalValid ? originalView.frameSeq : 0u;
thresholdRenderSnapshot.previewFrameSeq = previewValid ? previewView.frameSeq : 0u;
thresholdRenderSnapshot.lowValue = lowDisplay;
thresholdRenderSnapshot.highValue = highDisplay;
thresholdRenderSnapshot.exposureValue = exposureDisplay;
thresholdRenderSnapshot.ownerColor = ownerColor;
thresholdRenderSnapshot.detectMode = binaryMode;
thresholdRenderSnapshot.selection = threshSel;
thresholdRenderSnapshot.originalValid = originalValid;
thresholdRenderSnapshot.previewValid = previewValid;
thresholdRenderSnapshot.valid = 1u;
}

/* 整行铺底右界：有滚动条→滚动条左(SCROLLBAR_X-1)；无→右内框(LCD_W-2)。渲染前按 total 设。 */
static uint16_t listFillRightX = (uint16_t) (LCD_W - 2u);

static void fillBottomGutter(uint16_t y, uint16_t color)
{
    if (y != ROW_Y(VISIBLE_ROWS)) return;
    fillSolidRect(1u, (uint16_t) (y + FONT_H), listFillRightX,
        (uint16_t) ((LCD_H - 1u) - (y + FONT_H)), color);
}

/* 列表空行清理：铺黑到右界（含右侧空隙）。 */
static void clearListRow(uint16_t y)
{
    fillSolidRect(1u, y, listFillRightX, FONT_H, RGB565_BLACK);
    fillBottomGutter(y, RGB565_BLACK);
}

/* 整行铺底色，顶到右界，并为随后绘制的行标记和名称设置字色。 */
static void fillListRowBg(uint16_t y, uint16_t fg, uint16_t bg)
{
    ips114_set_color(fg, bg);
    fillSolidRect(1u, y, listFillRightX, FONT_H, bg);
    fillBottomGutter(y, bg);
}

/* 列表渲染快照（变化检测防闪烁，对齐 core2 chassisMenuRender）：仅结构变化(进/出编辑、换页、
 * 换选中项、滚动、切状态后)才整页重绘列表；EDIT 态值变只局部重绘当前值。
 * clearScreenForState(切状态/清屏)调 menuRenderResetListSnapshot 复位 → 下次列表渲染为结构性。 */
static menuStateEnum listLastState = (menuStateEnum) 0xFF;
static int16_t listLastPageIdx = -1;
static int16_t listLastOptIdx = -1;
static uint8_t listLastScrollTop = 0xFFu;
static uint16_t listLastBorderColor = 0xFFFFFu;
static uint8_t paramSlotsSurfaceValid;
static uint8_t paramSlotsLastSelection = 0xFFu;
static menuParamGlobalUiSnapshot paramSlotsLastUi;

void menuRenderResetListSnapshot(void)
{
    listLastState = (menuStateEnum) 0xFF;
    listLastPageIdx = -1;
    listLastOptIdx = -1;
    listLastScrollTop = 0xFFu;
    listLastBorderColor = 0xFFFFFu;
    paramSlotsSurfaceValid = 0u;
    paramSlotsLastSelection = 0xFFu;
}

```

```

memset(&paramSlotsLastUi, 0xFF, sizeof(paramSlotsLastUi));
homeFooterDirty = 1u;
homeLastKeyTarget = 0xFFu;
homeLastC1Up = 0xFFu;
homeLastC2Up = 0xFFu;
homeLastVisionUp = 0xFFu;
homeLastInputOwner = 0xFFu;
memset(homeSlotValid, 0, sizeof(homeSlotValid));
memset(homeSlotColor, 0, sizeof(homeSlotColor));
memset(homeSlotBackground, 0, sizeof(homeSlotBackground));
homeTitleColor = 0xFFFFu;
memset(&thresholdRenderSnapshot, 0, sizeof(thresholdRenderSnapshot));
}

void renderInitFaultSummary(uint8_t failedMask, uint8_t unverifiedMask, uint8_t secondsLeft)
{
    static const char *const names[4] = {"IMU", "TOF", "OPT FLOW", "CAMERA"};
    uint8_t row = 0u;
    uint8_t i;

    if (secondsLeft >= 3u)
    {
        ips114_clear();
        menuRenderResetListSnapshot();
        drawTitleBar((failedMask != 0u) ? "HW INIT FAIL" : "HW INIT CHECK",
            RGB565_BLACK, (failedMask != 0u) ? RGB565_RED : RGB565_YELLOW);
        for (i = 0u; i < 4u; i++)
        {
            {
                uint8_t bit = (uint8_t) (1u << i);
                if ((failedMask & bit) != 0u)
                {
                    ips114_set_color(RGB565_RED, RGB565_BLACK);
                    ips114_show_string(8u, (uint16_t) (18u + row * 16u), names[i]);
                    ips114_show_string(128u, (uint16_t) (18u + row * 16u), "FAIL ");
                    row++;
                }
            }
        }
        for (i = 0u; i < 4u; i++)
        {
            {
                uint8_t bit = (uint8_t) (1u << i);
                if (((unverifiedMask & bit) != 0u) && ((failedMask & bit) == 0u))
                {
                    ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
                    ips114_show_string(8u, (uint16_t) (18u + row * 16u), names[i]);
                    ips114_show_string(128u, (uint16_t) (18u + row * 16u), "CHECK");
                    row++;
                }
            }
        }
        drawFrameBorder();
    }
    ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
    ips114_show_string(8u, 114u, "CONTINUE 0s");
    ips114_show_uint(80u, 114u, secondsLeft, 1u);
}

/* =====
* 渲染：页列表（标题居中 + 滚动滑块）
* ===== */
void renderPageList(void)
{
    uint16_t borderColor = menuOwnerColor();
    /* 结构变化才整页重绘(防 100ms 空刷闪烁)：切入页列表 / 换选中页 / 滚动。 */
    uint8_t structural = (uint8_t) (listLastState != MENU_PAGE_LIST
        || listLastPageIdx != (int16_t) pageIdx || listLastScrollTop != scrollTop
        || listLastBorderColor != borderColor);
    uint8_t selectionOnly = (uint8_t) ((listLastState == MENU_PAGE_LIST) &&
        (listLastScrollTop == scrollTop) &&
        (listLastBorderColor == borderColor) &&
        (listLastPageIdx != (int16_t) pageIdx));
    if (!structural) return;

    listFillRightX = (menuPageCount > VISIBLE_ROWS) ? (uint16_t) (SCROLLBAR_X - 1u) : (uint16_t) (LCD_W - 2u);
    if (selectionOnly == 0u)
        drawTitleBar("PAGES", RGB565_BLACK, RGB565_GREEN);

    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
    uint8_t start = scrollTop;
    for (uint8_t i = 0u; i < VISIBLE_ROWS; i++)
    {

```

```

uint8_t pi = (uint8_t) (start + i);
uint16_t y = ROW_Y((uint8_t) (i + 1u));
if ((selectionOnly != 0u) && (pi != (uint8_t) pageIndex) &&
    (pi != (uint8_t) listLastPageIndex)) continue;
if (pi >= menuPageCount)
{
    clearListRow(y);
    continue;
}
if (pi == (uint8_t) pageIndex)
{
    fillListRowBg(y, MENU_SEL_NAME_FG, MENU_SEL_BG); /* 选中整行青底，名黑(页列表无数值) */
    ips114_show_string(MENU_MARK_X, y, ">");
    ips114_show_string(MENU_ITEM_X, y, menuPages[pi].name);
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
}
else
{
    clearListRow(y);
    ips114_set_color(MENU_NORM_NAME_FG, RGB565_BLACK);
    ips114_show_string(MENU_ITEM_X, y, menuPages[pi].name);
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
}
}

if (selectionOnly == 0u)
{
    drawFrameBorder();
    drawScrollbar(menuPageCount, scrollTop);
}
listLastState = MENU_PAGE_LIST;
listLastPageIndex = (int16_t) pageIndex;
listLastScrollTop = scrollTop;
listLastOptIdx = -1;
listLastBorderColor = borderColor;
}

/* =====
* 渲染：选项列表 / 编辑（共用，editing 改箭头 + 标题居中 + 滚动滑块）
* ===== */
void renderOptionList(uint8_t editing)
{
    menuPageStruct* pg = &menuPages[pageIndex];
    uint16_t borderColor = menuOwnerColor();
    uint8_t curIsEdit = (uint8_t) (editing != 0u);
    /* 结构变化判定(对齐 core2)：切入选项列表 / 进出编辑 / 换选中项 / 滚动 → 整页重绘；
    * EDIT 态值变(同态同项同滚动)只局部重绘当前值，消 100ms 全页重绘闪烁(用户重点:core0 调节页)。 */
    uint8_t enteringList = (uint8_t) (listLastState != MENU_OPTION_LIST && listLastState != MENU_EDIT);
    uint8_t editTransition = (uint8_t) ((listLastState == MENU_EDIT) != curIsEdit);
    uint8_t structural = (uint8_t) (enteringList || editTransition
        || listLastPageIndex != (int16_t) pageIndex
        || listLastOptIdx != (int16_t) optIdx || listLastScrollTop != scrollTop
        || listLastBorderColor != borderColor);
    uint8_t selectionOnly = (uint8_t) (!(enteringList) && (!editTransition) &&
        (listLastPageIndex == (int16_t) pageIndex) &&
        (listLastScrollTop == scrollTop) &&
        (listLastBorderColor == borderColor) &&
        (listLastOptIdx != (int16_t) optIdx));

    if (structural)
    {
        listFillRightX = (pg->optionCount > VISIBLE_ROWS) ? (uint16_t) (SCROLLBAR_X - 1u) : (uint16_t) (LCD_W - 2u);
        if (selectionOnly == 0u)
            drawTitleBar(pg->name, RGB565_BLACK, RGB565_GREEN);

        ips114_set_color(RGB565_GREEN, RGB565_BLACK);
        uint8_t start = scrollTop;
        for (uint8_t i = 0u; i < VISIBLE_ROWS; i++)
        {
            uint8_t oi = (uint8_t) (start + i);
            uint16_t y = ROW_Y((uint8_t) (i + 1u));
            if ((selectionOnly != 0u) && (oi != (uint8_t) optIdx) &&
                (oi != (uint8_t) listLastOptIdx)) continue;
            if (oi >= pg->optionCount)
            {
                clearListRow(y);
                continue;
            }
            if (oi == (uint8_t) optIdx && editing)

```

```

{
    /* 编辑行：整行黄底强调，名黑/值红（非仅数值反相） */
    fillListRowBg(y, MENU_EDIT_NAME_FG, MENU_EDIT_BG);
    ips114_show_string(MENU_MARK_X, y, "");
    ips114_show_string(MENU_ITEM_X, y, pg->optName[oi]);
    ips114_set_color(MENU_EDIT_VAL_FG, MENU_EDIT_BG);
    menuFloatDisplayShow(MENU_PARAM_VALUE_X, y,
        (double) (*pg->opt[oi]),
        menuFloatDisplayDecimalsForValueAndStep(
            (double) (*pg->opt[oi]), pg->stepS[oi]));
    ips114_set_color(GB565_GREEN, GB565_BLACK);
}
else if (oi == (uint8_t) optIdx)
{
    /* 选中行：整行青底，名黑/值蓝 */
    fillListRowBg(y, MENU_SEL_NAME_FG, MENU_SEL_BG);
    ips114_show_string(MENU_MARK_X, y, ">");
    ips114_show_string(MENU_ITEM_X, y, pg->optName[oi]);
    ips114_set_color(MENU_SEL_VAL_FG, MENU_SEL_BG);
    menuFloatDisplayShow(MENU_PARAM_VALUE_X, y,
        (double) (*pg->opt[oi]),
        menuFloatDisplayDecimalsForValueAndStep(
            (double) (*pg->opt[oi]), pg->stepS[oi]));
    ips114_set_color(GB565_GREEN, GB565_BLACK);
}
else
{
    /* 普通行：黑底，名绿/值白 */
    clearListRow(y);
    ips114_set_color(MENU_NORM_NAME_FG, GB565_BLACK);
    ips114_show_string(MENU_ITEM_X, y, pg->optName[oi]);
    ips114_set_color(MENU_NORM_VAL_FG, GB565_BLACK);
    menuFloatDisplayShow(MENU_PARAM_VALUE_X, y,
        (double) (*pg->opt[oi]),
        menuFloatDisplayDecimalsForValueAndStep(
            (double) (*pg->opt[oi]), pg->stepS[oi]));
    ips114_set_color(GB565_GREEN, GB565_BLACK);
}
}

if (selectionOnly == 0u)
{
    drawFrameBorder();
    /* 名称列/数值列竖分隔线已删除：名/值现用三态颜色区分，竖线多余且参与闪烁感（codex 点 6）。 */
    drawScrollbar(pg->optionCount, scrollTop);
}
}
else if (editing)
{
    /* 值局部刷新：仅当前选中项数值(黄底红字)，不动整行/名/边框/滚动条 → 消整页重绘闪烁 */
    uint16_t y = ROW_Y((uint8_t) ((uint8_t) optIdx - scrollTop + 1u));
    ips114_set_color(MENU_EDIT_VAL_FG, MENU_EDIT_BG);
    menuFloatDisplayShow(MENU_PARAM_VALUE_X, y,
        (double) (*pg->opt[optIdx]),
        menuFloatDisplayDecimalsForValueAndStep(
            (double) (*pg->opt[optIdx]), pg->stepS[optIdx]));
    ips114_set_color(GB565_GREEN, GB565_BLACK);
}

listLastState = curIsEdit ? MENU_EDIT : MENU_OPTION_LIST;
listLastPageIdx = (int16_t) pageIdx;
listLastOptIdx = (int16_t) optIdx;
listLastScrollTop = scrollTop;
listLastBorderColor = borderColor;
}

void renderParamSlots(void)
{
    char line[32];
    menuParamGlobalUiSnapshot globalUi;
    uint8_t fullRedraw;
    menuParamGlobalGetUi(&globalUi);

    fullRedraw = (uint8_t) (paramSlotsSurfaceValid == 0u);
    if (fullRedraw != 0u)
    {
        ips114_set_color(GB565_WHITE, GB565_BLACK);
        listFillRightX = (uint16_t) (LCD_W - 2u);
        drawTitleBar("PARAM SLOTS", GB565_BLACK, GB565_GREEN);
    }
}

```

```

ips114_set_color(GB565_GREEN, GB565_BLACK);
ips114_show_string(8u, ROW_Y(5u), "K2/K3 SLOT");
ips114_show_string(8u, ROW_Y(6u), "K1L SAVE K4 LOAD");
ips114_show_string(8u, ROW_Y(7u), "K5 BACK");
drawFrameBorder();
paramSlotsSurfaceValid = 1u;
}
if (fullRedraw != 0u || paramSlotsLastSelection != selectedParamSlot)
{
fillListRowBg(ROW_Y(1u), MENU_SEL_NAME_FG, MENU_SEL_BG);
(void) snprintf(line, sizeof(line), "> SELECT    SLOT %u",
(unsigned) selectedParamSlot);
ips114_show_string(MENU_MARK_X, ROW_Y(1u), line);
paramSlotsLastSelection = selectedParamSlot;
}
if (fullRedraw != 0u || paramSlotsLastUi.slotUiState != globalUi.slotUiState)
{
static const char *const slotText[] = {
"UNKNOWN", "CHECKING", "FULL", "EMPTY", "PARTIAL",
"INVALID", "UNAVAILABLE"};
uint8_t state = globalUi.slotUiState;
if (state > PARAM_SYNC_SLOT_UI_UNAVAILABLE)
state = PARAM_SYNC_SLOT_UI_UNKNOWN;
clearListRow(ROW_Y(2u));
(void) snprintf(line, sizeof(line), "SLOT: %s", slotText[state]);
ips114_set_color((state == PARAM_SYNC_SLOT_UI_FULL) ? GB565_GREEN :
((state == PARAM_SYNC_SLOT_UI_EMPTY) ? GB565_CYAN : GB565_YELLOW),
GB565_BLACK);
ips114_show_string(24u, ROW_Y(2u), line);
}
if (globalUi.state == MENU_PARAM_GLOBAL_QUEUED)
(void) snprintf(line, sizeof(line), "TASK: QUEUED");
else if (globalUi.state == MENU_PARAM_GLOBAL_PROGRESS)
(void) snprintf(line, sizeof(line), "RUN: C%u PH%u",
(unsigned) globalUi.target, (unsigned) globalUi.phase);
else if (globalUi.state == MENU_PARAM_GLOBAL_SUCCESS)
(void) snprintf(line, sizeof(line), "TASK: SUCCESS");
else if (globalUi.state == MENU_PARAM_GLOBAL_FAILED)
(void) snprintf(line, sizeof(line), "FAIL: C%u E%u",
(unsigned) globalUi.target, (unsigned) globalUi.error);
else
(void) snprintf(line, sizeof(line), "TASK: READY");
if (fullRedraw != 0u || memcmp(&paramSlotsLastUi, &globalUi,
sizeof(globalUi)) != 0)
{
clearListRow(ROW_Y(3u));
ips114_set_color(globalUi.state == MENU_PARAM_GLOBAL_SUCCESS ? GB565_GREEN :
(globalUi.state == MENU_PARAM_GLOBAL_FAILED ? GB565_RED : GB565_CYAN),
GB565_BLACK);
ips114_show_string(24u, ROW_Y(3u), line);
}
paramSlotsLastUi = globalUi;
}

```

core0_cyt4bb7/project/code/app/paramSyncOwnerService.c

SHA-256

:

72210ad1d5eebadf683037eda7fd85b67183109c88b707b248196e7e7cff81c1

代码 (c)

```

#include "paramSyncOwnerService.h"

#include <stddef.h>
#include <string.h>

#include "crc32Ieee.h"
#include "menuParamSlots.h"
#include "paramSyncMailbox.h"

#define CORE0_PARAM_SYNC_TARGET    (0u)
#define CORE0_PARAM_SCHEMA_HASH    (0xFC40 993Au)
#define CORE0_PARAM_FLASH_VERSION  (42u)
#define CORE0_PARAM_BLOB_MAGIC      (0x4D314B30u)
#define CORE0_PARAM_BLOB_BYTES      (648u)

```



```

#define CORE0_PARAM_BLOB_CRC_OFFSET (644u)
#define CORE0_ACTIVATE_ROLLBACK_MAGIC (0xA0B20 000u)
#define PARAM_STAGE_NONE (0u)
#define PARAM_STAGE_LIBRARY (1u)
#define PARAM_STAGE_RUNTIME (2u)

typedef char core0ParamBlobSizeAssert[(MENU_PARAM_SLOT_BLOB_BYTES == CORE0_PARAM_BLOB_BYTES) ? 1 : -1];

typedef struct
{
    uint32_t transactionId;
    uint32_t expectedBlobCrc;
    uint16_t expectedBytes;
    uint16_t receivedBytes;
    uint8_t slot;
    uint8_t active;
    uint8_t prepared;
    uint8_t committed;
    uint8_t activated;
    uint8_t kind;
    uint8_t applied;
    uint8_t blob[CORE0_PARAM_BLOB_BYTES];
} ownerPrepareContext;

static ownerPrepareContext g_prepare;
static uint32_t g_lastRequestBoot;
static uint32_t g_lastRequestSequence;
static uint32_t g_responseSequence;
static uint32_t g_ownerBootGeneration;
static uint32_t g_lastFinishedTransaction;
static uint32_t g_lastAbortedTransaction;
static paramSyncMailboxResponse g_lastResponse;
static uint8_t g_lastResponseValid;
static uint8_t g_activationRecoveryChecked;
static uint8_t g_syncMutationLocked;
static uint8_t g_syncLockedSlot;
static uint32_t g_syncLockedTransaction;
#ifdef __ICCARM__
__no_init uint32_t core0ParamSyncOwnerBootRetained;
__no_init uint32_t core0ParamSyncOwnerBootInverseRetained;
__no_init uint32_t core0ParamSyncActivationRollbackRetained;
__no_init uint32_t core0ParamSyncActivationRollbackInverseRetained;
#else
static uint32_t core0ParamSyncOwnerBootRetained;
static uint32_t core0ParamSyncOwnerBootInverseRetained;
static uint32_t core0ParamSyncActivationRollbackRetained;
static uint32_t core0ParamSyncActivationRollbackInverseRetained;
#endif

static uint8_t activationContext(uint8_t *oldSlot, uint8_t *newSlot)
{
    uint32_t word = core0ParamSyncActivationRollbackRetained;
    if (core0ParamSyncActivationRollbackInverseRetained != ~word ||
        (word & 0xFFFF0000u) != CORE0_ACTIVATE_ROLLBACK_MAGIC)
        return 0u;
    *oldSlot = (uint8_t) word;
    *newSlot = (uint8_t) (word >> 8);
    return (uint8_t) (*oldSlot >= 1u && *oldSlot <= MENU_PARAM_SLOT_COUNT &&
        *newSlot >= 1u && *newSlot <= MENU_PARAM_SLOT_COUNT);
}

static void setActivationContext(uint8_t oldSlot, uint8_t newSlot)
{
    uint32_t word = CORE0_ACTIVATE_ROLLBACK_MAGIC |
        (uint32_t) oldSlot | ((uint32_t) newSlot << 8);
    core0ParamSyncActivationRollbackRetained = word;
    core0ParamSyncActivationRollbackInverseRetained = ~word;
}

static void clearActivationContext(void)
{
    core0ParamSyncActivationRollbackRetained = 0u;
    core0ParamSyncActivationRollbackInverseRetained = UINT32_MAX;
}

static uint8_t rollbackActivation(void)
{
    uint8_t oldSlot, newSlot;
    (void) activationContext(&oldSlot, &newSlot);
}

```

```

    if (menuParamSlotsAbortPendingActivation() == 0u) return 0u;
    clearActivationContext();
    return 1u;
}

static uint8_t recoverActivationAfterOwnerBoot(void)
{
    if (g_activationRecoveryChecked != 0u) return 1u;
    g_activationRecoveryChecked = 1u;
    return 1u;
}

static uint16_t readU16(const uint8_t *data)
{
    return (uint16_t) (data[0] | ((uint16_t) data[1] << 8));
}

static uint32_t readU32(const uint8_t *data)
{
    return (uint32_t) data[0] |
        ((uint32_t) data[1] << 8) |
        ((uint32_t) data[2] << 16) |
        ((uint32_t) data[3] << 24);
}

static void writeU16(uint8_t *data, uint16_t value)
{
    data[0] = (uint8_t) value;
    data[1] = (uint8_t) (value >> 8);
}

static void writeU32(uint8_t *data, uint32_t value)
{
    data[0] = (uint8_t) value;
    data[1] = (uint8_t) (value >> 8);
    data[2] = (uint8_t) (value >> 16);
    data[3] = (uint8_t) (value >> 24);
}

static uint32_t getOwnerBootGeneration(void)
{
    uint32_t previous;
    if (g_ownerBootGeneration != 0u) return g_ownerBootGeneration;
    previous = core0ParamSyncOwnerBootRetained;
    if ((previous == 0u) ||
        (core0ParamSyncOwnerBootInverseRetained != ~previous)) previous = 0u;
    previous++;
    if (previous == 0u) previous = 1u;
    core0ParamSyncOwnerBootRetained = previous;
    core0ParamSyncOwnerBootInverseRetained = ~previous;
    g_ownerBootGeneration = previous;
    return g_ownerBootGeneration;
}

static void clearMutationLock(void)
{
    g_syncMutationLocked = 0u;
    g_syncLockedSlot = 0u;
    g_syncLockedTransaction = 0u;
}

static uint8_t decodeGlobalTransaction(const paramSyncMailboxRequest *request,
    uint8_t allowDefaultSlot, uint8_t *slot, uint32_t *transactionId)
{
    if (request->dataLength != 5u) return 0u;
    *slot = request->data[0];
    *transactionId = readU32(&request->data[1]);
    return (uint8_t) ((*transactionId != 0u) &&
        (*transactionId != UINT32_MAX) &&
        ((*slot >= 1u && *slot <= MENU_PARAM_SLOT_COUNT) ||
            (allowDefaultSlot != 0u && *slot == 0u)));
}

static uint8_t transactionLocked(uint8_t slot, uint32_t transactionId)
{
    return (uint8_t) (g_syncMutationLocked != 0u &&
        g_syncLockedSlot == slot &&
        g_syncLockedTransaction == transactionId);
}

```

```

static uint8_t producerBootChangeAllowed(
    const paramSyncMailboxRequest *request)
{
    menuParamRuntimeMeta runtimeMeta;
    uint8_t oldSlot, newSlot;
    uint8_t activationPending;
    uint8_t transactionPending;
    uint32_t transactionId;

    if (menuParamRuntimeGetMeta(&runtimeMeta) == 0u) return 0u;
    activationPending = (uint8_t) (activationContext(&oldSlot, &newSlot) ||
        (runtimeMeta.pending != 0u &&
            runtimeMeta.transactionId == UINT32_MAX));
    transactionPending = (uint8_t) (g_syncMutationLocked ||
        g_prepare.active || g_prepare.prepared || g_prepare.committed ||
            runtimeMeta.pending);
    if ((activationPending == 0u) && (transactionPending == 0u)) return 1u;
    if ((request->op != PARAM_SYNC_OP_FINISH &&
        request->op != PARAM_SYNC_OP_ABORT) ||
        (request->dataLength != 4u)) return 0u;

    transactionId = readU32(request->data);
    if (activationPending != 0u)
    {
        if ((transactionId != 0u) || g_prepare.active ||
            g_prepare.prepared || g_prepare.committed) return 0u;
        return 1u;
    }
    if ((g_syncMutationLocked != 0u) &&
        (transactionId != g_syncLockedTransaction)) return 0u;
    if ((g_prepare.active || g_prepare.prepared || g_prepare.committed) &&
        (transactionId != g_prepare.transactionId)) return 0u;
    if ((runtimeMeta.pending != 0u) &&
        (transactionId != runtimeMeta.transactionId)) return 0u;
    return 1u;
}

uint8_t paramSyncOwnerServiceMutationLocked(void)
{
    return (uint8_t) (g_syncMutationLocked || g_prepare.active ||
        g_prepare.prepared || g_prepare.committed ||
            menuParamSlotsHasPending());
}

static void publishResponse(const paramSyncMailboxRequest *request,
    uint8_t status, const uint8_t *data, uint16_t length, uint32_t payloadCrc)
{
    paramSyncMailboxResponse response;

    memset(&response, 0, sizeof(response));
    g_responseSequence++;
    if (g_responseSequence == 0u) g_responseSequence = 1u;
    response.producerBootGeneration = getOwnerBootGeneration();
    response.sequence = g_responseSequence;
    response.transactionId = request->transactionId;
    response.offset = request->offset;
    response.payloadCrc32 = payloadCrc;
    response.dataLength = length;
    response.op = request->op;
    response.slot = request->slot;
    response.target = request->target;
    response.status = status;
    if ((data != NULL) && (length != 0u)) memcpy(response.data, data, length);
    g_lastResponse = response;
    g_lastResponseValid = 1u;
    (void) paramSyncMailboxPublishResponse(&response);
}

static uint8_t blobValid(const uint8_t *blob, uint32_t expectedBlobCrc)
{
    uint32_t internalStored = readU32(&blob[CORE0_PARAM_BLOB_CRC_OFFSET]);
    uint32_t internalComputed = crc32leecalc(blob,
        CORE0_PARAM_BLOB_CRC_OFFSET);

    return (uint8_t) ((readU32(&blob[0]) == CORE0_PARAM_BLOB_MAGIC) &&
        (readU32(&blob[4]) == CORE0_PARAM_FLASH_VERSION) &&
        (readU32(&blob[8]) == CORE0_PARAM_BLOB_BYTES) &&
        (internalStored == internalComputed) &&

```

```

        (internalStored == expectedBlobCrc));
    }

static void handleReadMeta(const paramSyncMailboxRequest *request)
{
    menuParamLibraryMeta meta;
    uint8_t output[26u];
    uint8_t blob[CORE0_PARAM_BLOB_BYTES];
    uint32_t blobCrc = 0u;
    uint8_t slot;

    if (request->dataLength != 1u)
    {
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    slot = request->data[0];
    memset(&meta, 0, sizeof(meta));
    if ((slot < 1u) || (slot > MENU_PARAM_SLOT_COUNT) ||
        (menuParamSlotsInspect(slot, &meta) == 0u))
    {
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (meta.state == MENU_PARAM_SLOT_VALID)
    {
        if (menuParamSlotsRead(slot, blob, sizeof(blob)) == 0u)
        {
            publishResponse(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
            return;
        }
        blobCrc = readU32(&blob[CORE0_PARAM_BLOB_CRC_OFFSET]);
    }
    memset(output, 0, sizeof(output));
    output[0] = CORE0_PARAM_SYNC_TARGET;
    output[1] = slot;
    output[2] = meta.state;
    output[3] = 0u;
    writeU32(&output[4], CORE0_PARAM_FLASH_VERSION);
    writeU32(&output[8], meta.generation);
    writeU32(&output[12], blobCrc);
    writeU32(&output[16], meta.transactionId);
    writeU32(&output[20], CORE0_PARAM_SCHEMA_HASH);
    writeU16(&output[24], CORE0_PARAM_BLOB_BYTES);
    publishResponse(request, PARAM_SYNC_STATUS_OK, output, sizeof(output), 0u);
}

static void handleReadRuntimeMeta(const paramSyncMailboxRequest *request)
{
    menuParamRuntimeMeta meta;
    uint8_t output[32u];
    uint8_t blob[CORE0_PARAM_BLOB_BYTES];

    if (request->dataLength != 0u || menuParamRuntimeGetMeta(&meta) == 0u ||
        menuParamRuntimeRead(blob, sizeof(blob)) == 0u)
    {
        publishResponse(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    memset(output, 0, sizeof(output));
    output[0] = CORE0_PARAM_SYNC_TARGET;
    output[1] = meta.valid;
    output[2] = meta.bank;
    output[3] = meta.pending;
    output[4] = menuParamRuntimeIsDirty();
    output[5] = meta.sourceSlot;
    output[6] = meta.dagnosticFlags;
    writeU32(&output[8], CORE0_PARAM_FLASH_VERSION);
    writeU32(&output[12], meta.generation);
    writeU32(&output[16], readU32(&blob[CORE0_PARAM_BLOB_CRC_OFFSET]));
    writeU32(&output[20], meta.transactionId);
    writeU32(&output[24], CORE0_PARAM_SCHEMA_HASH);
    writeU16(&output[28], CORE0_PARAM_BLOB_BYTES);
    publishResponse(request, PARAM_SYNC_STATUS_OK, output, sizeof(output), 0u);
}

static void handleReadRuntimeChunk(const paramSyncMailboxRequest *request)
{
    uint8_t blob[CORE0_PARAM_BLOB_BYTES];

```

```

uint8_t output[PARAM_SYNC_MAILBOX_DATA_MAX];
uint16_t offset;
uint16_t length;

if (request->dataLength != 4u)
{
    publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
    return;
}
offset = readU16(&request->data[0]);
length = readU16(&request->data[2]);
if (length == 0u || length > (PARAM_SYNC_MAILBOX_DATA_MAX - 4u) ||
    (uint32_t) offset + length > sizeof(blob) ||
    menuParamRuntimeRead(blob, sizeof(blob)) == 0u)
{
    publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
    return;
}
writeU16(&output[0], offset);
writeU16(&output[2], sizeof(blob));
memcpy(&output[4], &blob[offset], length);
publishResponse(request, PARAM_SYNC_STATUS_OK, output,
    (uint16_t) (4u + length),
    readU32(&blob[CORE0_PARAM_BLOB_CRC_OFFSET]));
}

static void handleReadChunk(const paramSyncMailboxRequest *request)
{
    uint8_t blob[CORE0_PARAM_BLOB_BYTES];
    uint8_t output[PARAM_SYNC_MAILBOX_DATA_MAX];
    menuParamSlotMeta meta;
    uint16_t offset;
    uint16_t length;
    uint8_t slot;

    if (request->dataLength != 5u)
    {
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    slot = request->data[0];
    offset = readU16(&request->data[1]);
    length = readU16(&request->data[3]);
    if ((slot < 1u) || (slot > MENU_PARAM_SLOT_COUNT) || (length == 0u) ||
        (length > (PARAM_SYNC_MAILBOX_DATA_MAX - 5u)) ||
        ((uint32_t) offset + length > CORE0_PARAM_BLOB_BYTES) ||
        (menuParamSlotsGetMeta(slot, &meta) == 0u) || (meta.valid == 0u) ||
        (menuParamSlotsRead(slot, blob, sizeof(blob)) == 0u))
    {
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    output[0] = slot;
    writeU16(&output[1], offset);
    writeU16(&output[3], CORE0_PARAM_BLOB_BYTES);
    memcpy(&output[5], &blob[offset], length);
    publishResponse(request, PARAM_SYNC_STATUS_OK, output,
        (uint16_t) (length + 5u), readU32(&blob[CORE0_PARAM_BLOB_BYTES - 4u]));
}

static void clearPrepare(void)
{
    memset(&g_prepare, 0, sizeof(g_prepare));
}

static uint8_t abortPrepare(uint32_t transactionId)
{
    if (g_prepare.active == 0u) return 0u;
    if ((transactionId != 0u) &&
        (transactionId != g_prepare.transactionId)) return 0u;
    if ((g_prepare.prepared != 0u) &&
        (menuParamSlotsAbort(g_prepare.transactionId) == 0u)) return 0u;
    g_lastAbortedTransaction = g_prepare.transactionId;
    clearPrepare();
    return 1u;
}

static void handlePrepareLibraryFromLive(
    const paramSyncMailboxRequest *request)

```

```

{
    uint8_t slot;
    uint32_t transactionId;
    uint32_t blobCrc;

    if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u)
    {
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (g_prepare.active != 0u)
    {
        publishResponse(request, (g_prepare.kind == PARAM_STAGE_LIBRARY &&
            g_prepare.slot == slot &&
            g_prepare.transactionId == transactionId) ? PARAM_SYNC_STATUS_OK :
            PARAM_SYNC_STATUS_BUSY, NULL, 0u, g_prepare.expectedBlobCrc);
        return;
    }
    blobCrc = menuParamCurrentBlobCrc();
    if (menuParamSlotsPrepareCurrent(slot, transactionId) == 0u)
    {
        publishResponse(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    clearPrepare();
    g_prepare.slot = slot;
    g_prepare.transactionId = transactionId;
    g_prepare.expectedBlobCrc = blobCrc;
    g_prepare.expectedBytes = CORE0_PARAM_BLOB_BYTES;
    g_prepare.receivedBytes = CORE0_PARAM_BLOB_BYTES;
    g_prepare.active = 1u;
    g_prepare.prepared = 1u;
    g_prepare.kind = PARAM_STAGE_LIBRARY;
    publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, blobCrc);
}

static void handlePrepareRuntime(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;
    uint8_t ok;

    if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u ||
        slot == 0u)
    {
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (g_prepare.active != 0u)
    {
        publishResponse(request, (g_prepare.kind == PARAM_STAGE_RUNTIME &&
            g_prepare.slot == slot &&
            g_prepare.transactionId == transactionId) ? PARAM_SYNC_STATUS_OK :
            PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
        return;
    }
    ok = menuParamRuntimePrepareFromSlot(slot, transactionId);
    if (ok == 0u)
    {
        publishResponse(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    clearPrepare();
    g_prepare.slot = slot;
    g_prepare.transactionId = transactionId;
    g_prepare.active = 1u;
    g_prepare.prepared = 1u;
    g_prepare.kind = PARAM_STAGE_RUNTIME;
    publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleCommitRuntime(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;

    if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||

```

```

transactionLocked(slot, transactionId) == 0u ||
g_prepare.active == 0u || g_prepare.prepared == 0u ||
g_prepare.kind != PARAM_STAGE_RUNTIME || g_prepare.slot != slot ||
g_prepare.transactionId != transactionId)
{
    publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
    return;
}
if (g_prepare.committed == 0u &&
    menuParamRuntimeCommit(transactionId) == 0u)
{
    publishResponse(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}
g_prepare.committed = 1u;
publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleApplyRuntime(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;

    if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u ||
        g_prepare.active == 0u || g_prepare.committed == 0u ||
        g_prepare.kind != PARAM_STAGE_RUNTIME || g_prepare.slot != slot ||
        g_prepare.transactionId != transactionId)
    {
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (g_prepare.applied == 0u &&
        menuParamRuntimeApplyPrepared(transactionId) == 0u)
    {
        publishResponse(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    g_prepare.applied = 1u;
    publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handlePrepareBegin(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;

    if (request->dataLength != 19u)
    {
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    slot = request->data[0];
    transactionId = readU32(&request->data[1]);
    if ((slot < 1u) || (slot > MENU_PARAM_SLOT_COUNT) ||
        (transactionId == 0u) || (transactionId == UINT32_MAX) ||
        (readU32(&request->data[5]) != CORE0_PARAM_FLASH_VERSION) ||
        (readU32(&request->data[9]) != CORE0_PARAM_SCHEMA_HASH) ||
        (readU16(&request->data[17]) != CORE0_PARAM_BLOB_BYTES))
    {
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if ((g_prepare.active != 0u) && (slot == g_prepare.slot) &&
        (transactionId == g_prepare.transactionId) &&
        (readU32(&request->data[13]) == g_prepare.expectedBlobCrc) &&
        (readU16(&request->data[17]) == g_prepare.expectedBytes))
    {
        publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
        return;
    }
    if ((g_prepare.active != 0u) && abortPrepare(g_prepare.transactionId) == 0u)
    {
        publishResponse(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    clearPrepare();
    g_prepare.slot = slot;
    g_prepare.transactionId = transactionId;
}

```

```

g_prepare.expectedBlobCrc = readU32(&request->data[13]);
g_prepare.expectedBytes = CORE0_PARAM_BLOB_BYTES;
g_prepare.active = 1u;
g_prepare.kind = PARAM_STAGE_LIBRARY;
publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handlePrepareChunk(const paramSyncMailboxRequest *request)
{
    uint16_t offset;
    uint16_t length;

    if ((g_prepare.active == 0u) || (request->dataLength < 5u))
    {
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    offset = readU16(&request->data[1]);
    length = readU16(&request->data[3]);
    if ((request->data[0] == g_prepare.slot) &&
        ((uint32_t) offset + length <= g_prepare.receivedBytes) &&
        (length == (uint16_t) (request->dataLength - 5u)) &&
        (memcmp(&g_prepare.blob[offset], &request->data[5], length) == 0))
    {
        publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
        return;
    }
    if ((request->data[0] != g_prepare.slot) ||
        (offset != g_prepare.receivedBytes) ||
        (length != (uint16_t) (request->dataLength - 5u)) ||
        ((uint32_t) offset + length > g_prepare.expectedBytes))
    {
        (void) abortPrepare(g_prepare.transactionId);
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    memcpy(&g_prepare.blob[offset], &request->data[5], length);
    g_prepare.receivedBytes = (uint16_t) (g_prepare.receivedBytes + length);
    publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handlePrepareFinish(const paramSyncMailboxRequest *request)
{
    uint32_t transactionId;
    uint32_t crc;

    if ((g_prepare.prepared != 0u) && (request->dataLength == 1u) &&
        (request->data[0] == g_prepare.slot))
    {
        publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u,
            g_prepare.expectedBlobCrc);
        return;
    }
    if ((g_prepare.active == 0u) || (request->dataLength != 1u) ||
        (request->data[0] != g_prepare.slot) ||
        (g_prepare.receivedBytes != g_prepare.expectedBytes) ||
        (blobValid(g_prepare.blob, g_prepare.expectedBlobCrc) == 0u))
    {
        transactionId = g_prepare.transactionId;
        if (transactionId != 0u) (void) abortPrepare(transactionId);
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (menuParamSlotsPrepare(g_prepare.slot, g_prepare.blob,
        sizeof(g_prepare.blob), g_prepare.transactionId) == 0u)
    {
        transactionId = g_prepare.transactionId;
        (void) abortPrepare(transactionId);
        publishResponse(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    g_prepare.prepared = 1u;
    crc = g_prepare.expectedBlobCrc;
    publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, crc);
}

static uint8_t decodeSlotTransaction(const paramSyncMailboxRequest *request,
    uint8_t *slot, uint32_t *transactionId)
{

```



```

    if (request->dataLength != 5u) return 0u;
    *slot = request->data[0];
    *transactionId = readU32(&request->data[1]);
    return (uint8_t) ((*slot >= 1u) && (*slot <= MENU_PARAM_SLOT_COUNT));
}

static void handleCommit(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;
    if ((decodeSlotTransaction(request, &slot, &transactionId) != 0u) &&
        (g_prepare.committed != 0u) && (slot == g_prepare.slot) &&
        (transactionId == g_prepare.transactionId))
    {
        publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
        return;
    }
    if ((decodeSlotTransaction(request, &slot, &transactionId) == 0u) ||
        (g_prepare.kind != PARAM_STAGE_LIBRARY) ||
        (g_prepare.prepared == 0u) || (slot != g_prepare.slot) ||
        (transactionId != g_prepare.transactionId))
    {
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (menuParamSlotsCommit(slot, transactionId) == 0u)
    {
        publishResponse(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    g_prepare.committed = 1u;
    publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleActivate(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint8_t oldSlot, pendingSlot;
    uint32_t transactionId;
    if (decodeSlotTransaction(request, &slot, &transactionId) == 0u)
    {
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (((transactionId != 0u) && ((g_prepare.committed == 0u) ||
        (slot != g_prepare.slot) || (transactionId != g_prepare.transactionId)))
        ||
        (publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (transactionId == 0u)
    {
        if (!g_syncMutationLocked || g_syncLockedSlot != slot ||
            g_prepare.active || g_prepare.prepared ||
            g_prepare.committed)
        {
            publishResponse(request, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
            return;
        }
        if (activationContext(&oldSlot, &pendingSlot))
        {
            publishResponse(request, pendingSlot == slot ? PARAM_SYNC_STATUS_OK : PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
            return;
        }
        oldSlot = menuParamSlotsGetActive();
        if (oldSlot < 1u || oldSlot > MENU_PARAM_SLOT_COUNT)
        {
            publishResponse(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
            return;
        }
        setActivationContext(oldSlot, slot);
    }
    if (((transactionId == 0u ? menuParamSlotsActivatePending(slot) :
        menuParamSlotsActivatePrepared(slot, transactionId)) == 0u)
        ||
        (if (transactionId == 0u) (void) rollbackActivation();
        publishResponse(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
}

```

```

    if (transactionId != 0u) g_prepare.activated = 1u;
    publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleAbort(const paramSyncMailboxRequest *request)
{
    uint32_t transactionId;
    if (request->dataLength != 4u)
    {
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    transactionId = readU32(request->data);
    if (transactionId == 0u)
    {
        uint8_t ok = rollbackActivation();
        if (ok) clearMutationLock();
        publishResponse(request, ok ? PARAM_SYNC_STATUS_OK : PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    if ((g_prepare.active == 0u) &&
        (transactionId == g_lastAbortedTransaction))
    {
        publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
        return;
    }
    if (((g_prepare.active != 0u) && (abortPrepare(transactionId) == 0u)) ||
        ((g_prepare.active == 0u) && (menuParamSlotsAbort(transactionId) == 0u)))
    {
        publishResponse(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    clearMutationLock();
    publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleFinish(const paramSyncMailboxRequest *request)
{
    uint32_t transactionId;
    if (request->dataLength != 4u)
    {
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    transactionId = readU32(request->data);
    if (transactionId == 0u)
    {
        if (menuParamSlotsFinishPendingActivation() == 0u)
        {
            publishResponse(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
            return;
        }
        clearActivationContext();
        clearMutationLock();
        publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
        return;
    }
    if ((g_prepare.active == 0u) &&
        (transactionId == g_lastFinishedTransaction))
    {
        publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
        return;
    }
    if ((g_prepare.active != 0u) &&
        (((g_prepare.kind == PARAM_STAGE_LIBRARY) &&
          (g_prepare.committed == 0u)) ||
         ((g_prepare.kind == PARAM_STAGE_RUNTIME) &&
          ((g_prepare.committed == 0u) || (g_prepare.applied == 0u))) ||
          (transactionId != g_prepare.transactionId)))
    {
        publishResponse(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (menuParamSlotsFinish(transactionId) == 0u)
    {
        publishResponse(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    g_lastFinishedTransaction = transactionId;
}

```

```

clearPrepare();
clearMutationLock();
publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleLock(const paramSyncMailboxRequest *request)
{
    uint8_t oldSlot, newSlot;
    uint32_t transactionId = (request->dataLength == 5u) ?
        readU32(&request->data[1]) : 0u;
    if (request->dataLength != 5u || request->data[0] > 5u ||
        transactionId == 0u || transactionId == UINT32_MAX ||
        (g_syncMutationLocked && (g_syncLockedSlot != request->data[0] ||
            g_syncLockedTransaction != transactionId)) ||
        g_prepare.active || menuParamSlotsHasPending() ||
        activationContext(&oldSlot, &newSlot))
    {
        publishResponse(request, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
        return;
    }
    g_syncMutationLocked = 1u;
    g_syncLockedSlot = request->data[0];
    g_syncLockedTransaction = transactionId;
    publishResponse(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static uint8_t retainedRequestMayResume(
    const paramSyncMailboxRequest *request)
{
    if (request->target != CORE0_PARAM_SYNC_TARGET) return 0u;
    return (uint8_t) (request->op == PARAM_SYNC_OP_QUERY ||
        request->op == PARAM_SYNC_OP_READ_META ||
        request->op == PARAM_SYNC_OP_READ_CHUNK ||
        request->op == PARAM_SYNC_OP_READ_RUNTIME_META ||
        request->op == PARAM_SYNC_OP_READ_RUNTIME_CHUNK ||
        request->op == PARAM_SYNC_OP_RECOVERY ||
        request->op == PARAM_SYNC_OP_ABORT ||
        request->op == PARAM_SYNC_OP_FINISH);
}

void paramSyncOwnerServiceInit(void)
{
    paramSyncMailboxRequest retainedRequest;
    g_lastRequestBoot = 0u;
    g_lastRequestSequence = 0u;
    g_responseSequence = 0u;
    g_lastResponseValid = 0u;
    (void) getOwnerBootGeneration();
    if (paramSyncMailboxTryReadRequest(&retainedRequest) != 0u &&
        retainedRequestMayResume(&retainedRequest) == 0u)
    {
        g_lastRequestBoot = retainedRequest.producerBootGeneration;
        g_lastRequestSequence = retainedRequest.sequence;
        if (retainedRequest.target == CORE0_PARAM_SYNC_TARGET)
            publishResponse(&retainedRequest, PARAM_SYNC_STATUS_BUSY,
                NULL, 0u, 0u);
    }
}

void paramSyncOwnerServicePoll(void)
{
    paramSyncMailboxRequest request;

    if (paramSyncMailboxTryReadRequest(&request) == 0u) return;
    if ((request.producerBootGeneration == g_lastRequestBoot) &&
        (request.sequence == g_lastRequestSequence))
    {
        if (g_lastResponseValid != 0u)
            (void) paramSyncMailboxPublishResponse(&g_lastResponse);
        return;
    }
    if (request.producerBootGeneration == g_lastRequestBoot &&
        (int32_t) (request.sequence - g_lastRequestSequence) <= 0) return;
    if ((g_lastRequestBoot != 0u) &&
        (request.producerBootGeneration != g_lastRequestBoot))
    {
        if (producerBootChangeAllowed(&request) == 0u)
        {
            if (request.target == CORE0_PARAM_SYNC_TARGET)

```

```

        publishResponse(&request, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
    return;
}
}
g_lastRequestBoot = request.producerBootGeneration;
g_lastRequestSequence = request.sequence;
if (request.target != CORE0_PARAM_SYNC_TARGET) return;
if (recoverActivationAfterOwnerBoot() == 0u)
{
    publishResponse(&request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}
switch (request.op)
{
    case PARAM_SYNC_OP_READ_META: handleReadMeta(&request); break;
    case PARAM_SYNC_OP_READ_CHUNK: handleReadChunk(&request); break;
    case PARAM_SYNC_OP_READ_RUNTIME_META:
        handleReadRuntimeMeta(&request); break;
    case PARAM_SYNC_OP_READ_RUNTIME_CHUNK:
        handleReadRuntimeChunk(&request); break;
    case PARAM_SYNC_OP_PREPARE_LIBRARY_FROM_LIVE:
        handlePrepareLibraryFromLive(&request); break;
    case PARAM_SYNC_OP_PREPARE_RUNTIME_FROM_SLOT:
        handlePrepareRuntime(&request); break;
    case PARAM_SYNC_OP_COMMIT_RUNTIME:
        handleCommitRuntime(&request); break;
    case PARAM_SYNC_OP_APPLY_RUNTIME:
        handleApplyRuntime(&request); break;
    case PARAM_SYNC_OP_PREPARE_BEGIN: handlePrepareBegin(&request); break;
    case PARAM_SYNC_OP_PREPARE_CHUNK: handlePrepareChunk(&request); break;
    case PARAM_SYNC_OP_PREPARE_VERIFY: handlePrepareFinish(&request); break;
    case PARAM_SYNC_OP_COMMIT: handleCommit(&request); break;
    case PARAM_SYNC_OP_ACTIVATE: handleActivate(&request); break;
    case PARAM_SYNC_OP_ABORT: handleAbort(&request); break;
    case PARAM_SYNC_OP_FINISH: handleFinish(&request); break;
    case PARAM_SYNC_OP_LOCK: handleLock(&request); break;
    default:
        publishResponse(&request, PARAM_SYNC_STATUS_REJECTED, NULL, 0u, 0u);
        break;
}
}
}

```

core0_cyt4bb7/project/code/comm/coreDataTransfer.c

SHA-256

:

b92cd5be7aac0428123a6fe6cc6bc434ac1099baac36222522fcce0260eb5d76

代码 (c)

```

#include "coreDataTransfer.h"

#include "cy_device_headers.h"

#include <string.h>

#define CORE_DATA_CACHE_LINE_SIZE    (32u)
#define CORE_DATA_PARAM_TXN_TAG      (0xC03A0000u)
#define CORE_DATA_PARAM_TXN_TAG_MASK (0xFFFF0001u)
#define CORE_DATA_PARAM_TXN_COUNTER_MASK (0x0000FFFEu)
#define CORE_DATA_VISDBG_BUSY_MAGIC  (0u)

/* 用 #pragma location 把两个结构体强制放在共享 RAM 区固定地址。
 * 让链接器为其保留空间 (沿用旧 sharedObject 的 __no_init 绝对放置机制)。
 * 两区相距 256 字节, 各自独立 cache line, 杜绝 false-sharing。 */
#pragma location = CORE_DATA_VISION_ADDR
__no_init volatile visionToFlightStruct visionRegion;

#pragma location = CORE_DATA_FLIGHT_ADDR
__no_init volatile flightToVisionStruct flightRegion;

#pragma location = CORE_DATA_KVM_STATUS_ADDR
__no_init volatile kvmStatusToMenuStruct kvmStatusRegion;

#pragma location = CORE_DATA_PARAM_GLOBAL_STATE_ADDR
__no_init volatile coreDataParamGlobalStateStruct paramGlobalStateRegion;

```

```

#pragma location = CORE_DATA_PARAM_GLOBAL_REQUEST_ADDR
__no_init volatile coreDataParamGlobalRequestStruct paramGlobalRequestRegion;

#pragma location = CORE_DATA_FLIGHT_MARKER_ADDR
__no_init volatile coreDataFlightMarkerStruct flightMarkerRegion;

#pragma location = CORE_DATA_PARAM_ADDR
__no_init volatile paramToFlightStruct paramRegion;

static uint32_t paramPublishCounter;
static uint32_t flightPublishGeneration;
static uint8_t flightPublishPrimed;

static uint32_t nextFlightPublishGeneration(void)
{
    coreDataFlightMarkerStruct marker;
    if (flightPublishPrimed == 0u)
    {
        SCB_InvalidateDCache_by_Addr((volatile void*) &flightMarkerRegion,
            sizeof(flightMarkerRegion));
        __DSB();
        memcpy(&marker, (const void*) &flightMarkerRegion, sizeof(marker));
        if ((marker.magic == CORE_DATA_READY_MAGIC) &&
            (marker.generation != 0u))
            flightPublishGeneration = marker.generation;
        flightPublishPrimed = 1u;
    }
    if ((flightPublishGeneration & 1u) != 0u)
        flightPublishGeneration += 1u;
    else
        flightPublishGeneration += 2u;
    if (flightPublishGeneration == 0u) flightPublishGeneration = 2u;
    return flightPublishGeneration;
}

static void publishFlightMarker(uint32_t generation)
{
    coreDataFlightMarkerStruct marker;
    memset(&marker, 0, sizeof(marker));
    marker.magic = CORE_DATA_READY_MAGIC;
    marker.generation = generation;
    memcpy((void*) &flightMarkerRegion, &marker, sizeof(marker));
    SCB_CleanDCache_by_Addr((volatile void*) &flightMarkerRegion,
        sizeof(flightMarkerRegion));
    __DSB();
}

static volatile void* paramMarkerCacheLine(void)
{
    uintptr_t marker = (uintptr_t) &paramRegion.cm71ParamReady;
    return (volatile void*) (marker & ~((uintptr_t) CORE_DATA_CACHE_LINE_SIZE - 1u));
}

static void cleanParamMarker(void)
{
    SCB_CleanDCache_by_Addr(paramMarkerCacheLine(), CORE_DATA_CACHE_LINE_SIZE);
    __DSB();
}

static void invalidateParamMarker(void)
{
    SCB_InvalidateDCache_by_Addr(paramMarkerCacheLine(), CORE_DATA_CACHE_LINE_SIZE);
    __DSB();
}

static uint32_t nextParamCommitToken(void)
{
    paramPublishCounter = (paramPublishCounter + 2u) & CORE_DATA_PARAM_TXN_COUNTER_MASK;
    if (paramPublishCounter == 0u) paramPublishCounter = 2u;
    return CORE_DATA_PARAM_TXN_TAG | paramPublishCounter;
}

static uint8_t paramTokenIsCommitted(uint32_t token)
{
    return (uint8_t) (((token & CORE_DATA_PARAM_TXN_TAG_MASK) == CORE_DATA_PARAM_TXN_TAG) &&
        ((token & CORE_DATA_PARAM_TXN_COUNTER_MASK) != 0u));
}

```

```

#pragma location = CORE_DATA_VISDBG_ADDR
__no_init volatile visionDebugToFlightStruct visionDebugRegion;

static volatile void* visionDebugMarkerCacheLine(void)
{
    uintptr_t marker = (uintptr_t) &visionDebugRegion.magic;
    return (volatile void*) (marker & ~((uintptr_t) CORE_DATA_CACHE_LINE_SIZE - 1u));
}

static void cleanVisionDebugMarker(void)
{
    SCB_CleanDCache_by_Addr(visionDebugMarkerCacheLine(), CORE_DATA_CACHE_LINE_SIZE);
    __DSB();
}

static void invalidateVisionDebugMarker(void)
{
    SCB_InvalidateDCache_by_Addr(visionDebugMarkerCacheLine(), CORE_DATA_CACHE_LINE_SIZE);
    __DSB();
}

void coreDataWriteVisionToFlight(const visionToFlightStruct* src)
{
    if (0 == src) return;
    memcpy((void*) &visionRegion, src, sizeof(visionToFlightStruct));
    SCB_CleanDCache_by_Addr((volatile void*) &visionRegion, sizeof(visionToFlightStruct));
}

void coreDataReadVisionToFlight(visionToFlightStruct* dst)
{
    if (0 == dst) return;
    SCB_InvalidateDCache_by_Addr((volatile void*) &visionRegion, sizeof(visionToFlightStruct));
    memcpy(dst, (const void*) &visionRegion, sizeof(visionToFlightStruct));
}

void coreDataWriteFlightToVision(const flightToVisionStruct* src)
{
    uint32_t generation;
    if (0 == src) return;
    generation = nextFlightPublishGeneration();
    publishFlightMarker(generation - 1u);
    memcpy((void*) &flightRegion, src, sizeof(flightToVisionStruct));
    SCB_CleanDCache_by_Addr((volatile void*) &flightRegion, sizeof(flightToVisionStruct));
    __DSB();
    publishFlightMarker(generation);
}

uint8_t coreDataTryReadFlightToVision(flightToVisionStruct* dst,
uint32_t* generation)
{
    coreDataFlightMarkerStruct before;
    coreDataFlightMarkerStruct after;
    flightToVisionStruct snapshot;

    if ((dst == 0) || (generation == 0)) return 0u;
    SCB_InvalidateDCache_by_Addr((volatile void*) &flightMarkerRegion,
        sizeof(flightMarkerRegion));
    __DSB();
    memcpy(&before, (const void*) &flightMarkerRegion, sizeof(before));
    if ((before.magic != CORE_DATA_READY_MAGIC) ||
        (before.generation == 0u) || ((before.generation & 1u) != 0u))
        return 0u;

    SCB_InvalidateDCache_by_Addr((volatile void*) &flightRegion,
        sizeof(flightToVisionStruct));
    __DSB();
    memcpy(&snapshot, (const void*) &flightRegion, sizeof(snapshot));

    SCB_InvalidateDCache_by_Addr((volatile void*) &flightMarkerRegion,
        sizeof(flightMarkerRegion));
    __DSB();
    memcpy(&after, (const void*) &flightMarkerRegion, sizeof(after));
    if ((after.magic != before.magic) ||
        (after.generation != before.generation) ||
        ((after.generation & 1u) != 0u))
        return 0u;

    *dst = snapshot;
    *generation = after.generation;
}

```

```

    return 1u;
}

void coreDataWriteKvmStatusToMenu(const kvmStatusToMenuStruct* src)
{
    if (0 == src) return;
    memcpy((void*) &kvmStatusRegion, src, sizeof(kvmStatusToMenuStruct));
    SCB_CleanDCache_by_Addr((volatile void*) &kvmStatusRegion, sizeof(kvmStatusToMenuStruct));
}

uint8_t coreDataTryReadKvmStatusToMenu(kvmStatusToMenuStruct* dst)
{
    kvmStatusToMenuStruct first;
    kvmStatusToMenuStruct second;
    uint8_t attempt;
    if (0 == dst) return 0u;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        SCB_InvalidateDCache_by_Addr((volatile void*) &kvmStatusRegion, sizeof(kvmStatusToMenuStruct));
        memcpy(&first, (const void*) &kvmStatusRegion, sizeof(first));
        SCB_InvalidateDCache_by_Addr((volatile void*) &kvmStatusRegion, sizeof(kvmStatusToMenuStruct));
        memcpy(&second, (const void*) &kvmStatusRegion, sizeof(second));
        if (memcmp(&first, &second, sizeof(first)) == 0)
        {
            *dst = second;
            return 1u;
        }
    }
    return 0u;
}

void coreDataWriteParamGlobalState(const paramSyncGlobalState *src)
{
    coreDataParamGlobalStateStruct frame;
    if ((src == 0) || (paramSyncGlobalStateIsValid(src) == 0u)) return;
    memset(&frame, 0, sizeof(frame));
    frame.magic = CORE_DATA_PARAM_GLOBAL_MAGIC;
    frame.value = *src;
    memcpy((void *) &paramGlobalStateRegion, &frame, sizeof(frame));
    SCB_CleanDCache_by_Addr((volatile void *) &paramGlobalStateRegion,
        sizeof(paramGlobalStateRegion));
    __DSB();
}

void coreDataClearParamGlobalState(void)
{
    coreDataParamGlobalStateStruct frame;
    memset(&frame, 0, sizeof(frame));
    memcpy((void *) &paramGlobalStateRegion, &frame, sizeof(frame));
    SCB_CleanDCache_by_Addr((volatile void *) &paramGlobalStateRegion,
        sizeof(paramGlobalStateRegion));
    __DSB();
}

uint8_t coreDataTryReadParamGlobalState(paramSyncGlobalState *dst)
{
    coreDataParamGlobalStateStruct first, second;
    uint8_t attempt;
    if (dst == 0) return 0u;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        SCB_InvalidateDCache_by_Addr((volatile void *) &paramGlobalStateRegion,
            sizeof(paramGlobalStateRegion));
        __DSB();
        memcpy(&first, (const void *) &paramGlobalStateRegion, sizeof(first));
        SCB_InvalidateDCache_by_Addr((volatile void *) &paramGlobalStateRegion,
            sizeof(paramGlobalStateRegion));
        __DSB();
        memcpy(&second, (const void *) &paramGlobalStateRegion, sizeof(second));
        if ((memcmp(&first, &second, sizeof(first)) == 0) &&
            (second.magic == CORE_DATA_PARAM_GLOBAL_MAGIC) &&
            (paramSyncGlobalStateIsValid(&second.value) != 0u))
        {
            *dst = second.value;
            return 1u;
        }
    }
    return 0u;
}

```

```

void coreDataWriteParamGlobalRequest(const paramSyncGlobalRequest *src)
{
    coreDataParamGlobalRequestStruct frame;
    if ((src == 0) || (paramSyncGlobalRequestIsValid(src) == 0u)) return;
    memset(&frame, 0, sizeof(frame));
    frame.magic = CORE_DATA_PARAM_GLOBAL_MAGIC;
    frame.value = *src;
    memcpy((void *) &paramGlobalRequestRegion, &frame, sizeof(frame));
    SCB_CleanDCache_by_Addr((volatile void *) &paramGlobalRequestRegion,
        sizeof(paramGlobalRequestRegion));
    __DSB();
}

uint8_t coreDataTryReadParamGlobalRequest(paramSyncGlobalRequest *dst)
{
    coreDataParamGlobalRequestStruct first, second;
    uint8_t attempt;
    if (dst == 0) return 0u;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        SCB_InvalidateDCache_by_Addr((volatile void *) &paramGlobalRequestRegion,
            sizeof(paramGlobalRequestRegion));
        __DSB();
        memcpy(&first, (const void *) &paramGlobalRequestRegion, sizeof(first));
        SCB_InvalidateDCache_by_Addr((volatile void *) &paramGlobalRequestRegion,
            sizeof(paramGlobalRequestRegion));
        __DSB();
        memcpy(&second, (const void *) &paramGlobalRequestRegion, sizeof(second));
        if ((memcmp(&first, &second, sizeof(first)) == 0) &&
            (second.magic == CORE_DATA_PARAM_GLOBAL_MAGIC) &&
            (paramSyncGlobalRequestIsValid(&second.value) != 0u))
        {
            *dst = second.value;
            return 1u;
        }
    }
    return 0u;
}

void coreDataWriteParamToFlight(const paramToFlightStruct* src)
{
    paramToFlightStruct frame;
    uint32_t committed;

    if (0 == src) return;

    frame = *src;
    committed = nextParamCommitToken();

    /* 先单独发布奇数 marker，使跨多 cache line 的后续复制全程不可被接受。 */
    paramRegion.cm71ParamReady = committed | 1u;
    cleanParamMarker();

    frame.cm71ParamReady = committed | 1u;
    memcpy((void*) &paramRegion, &frame, sizeof(frame));
    SCB_CleanDCache_by_Addr((volatile void*) &paramRegion, sizeof(paramToFlightStruct));
    __DSB();

    /* payload 全部可见后，最后把 marker 提交为对应偶数。 */
    paramRegion.cm71ParamReady = committed;
    cleanParamMarker();
}

uint8_t coreDataReadParamToFlight(paramToFlightStruct* dst)
{
    uint8_t attempt;

    if (0 == dst) return 0u;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        uint32_t before;
        uint32_t after;

        invalidateParamMarker();
        before = paramRegion.cm71ParamReady;
        if (paramTokenIsCommitted(before) == 0u) continue;

        SCB_InvalidateDCache_by_Addr((volatile void*) &paramRegion, sizeof(paramToFlightStruct));
    }
}

```



```

__DSB();
memcpy(dst, (const void*) &paramRegion, sizeof(*dst));

invalidateParamMarker();
after = paramRegion.cm71ParamReady;
if ((before == dst->cm71ParamReady) && (before == after) &&
    (paramTokenIsCommitted(after) != 0u))
{
    return 1u;
}
}

memset(dst, 0, sizeof(*dst));
return 0u;
}

void coreDataWriteVisionDebugToFlight(const visionDebugToFlightStruct* src)
{
    visionDebugToFlightStruct frame;
    if (0 == src) return;

    frame = *src;
    if (frame.seq == 0u) return;

    /* 先发布 BUSY，再写跨两条 cache line 的 payload；最后单独提交 READY。
     * payload 阶段必须继续携带 BUSY，避免第一条 line 先 clean 时提前公开半帧。 */
    visionDebugRegion.magic = CORE_DATA_VISDBG_BUSY_MAGIC;
    cleanVisionDebugMarker();
    frame.magic = CORE_DATA_VISDBG_BUSY_MAGIC;
    memcpy((void*) &visionDebugRegion, &frame, sizeof(frame));
    SCB_CleanDCache_by_Addr((volatile void*) &visionDebugRegion, sizeof(visionDebugToFlightStruct));
    __DSB();
    visionDebugRegion.magic = CORE_DATA_READY_MAGIC;
    cleanVisionDebugMarker();
}

uint8_t coreDataTryReadVisionDebugToFlight(visionDebugToFlightStruct* dst)
{
    uint8_t attempt;
    if (0 == dst) return 0u;

    for (attempt = 0u; attempt < 3u; attempt++)
    {
        visionDebugToFlightStruct frame;
        uint32_t beforeMagic;
        uint32_t beforeSeq;
        uint32_t afterMagic;
        uint32_t afterSeq;

        invalidateVisionDebugMarker();
        beforeMagic = visionDebugRegion.magic;
        beforeSeq = visionDebugRegion.seq;
        if ((beforeMagic != CORE_DATA_READY_MAGIC) || (beforeSeq == 0u)) continue;

        SCB_InvalidateDCache_by_Addr((volatile void*) &visionDebugRegion,
            sizeof(visionDebugToFlightStruct));
        __DSB();
        memcpy(&frame, (const void*) &visionDebugRegion, sizeof(frame));

        invalidateVisionDebugMarker();
        afterMagic = visionDebugRegion.magic;
        afterSeq = visionDebugRegion.seq;
        if ((frame.magic == CORE_DATA_READY_MAGIC) &&
            (beforeMagic == afterMagic) &&
            (beforeSeq == frame.seq) && (beforeSeq == afterSeq))
        {
            *dst = frame;
            return 1u;
        }
    }
    return 0u;
}

```

core0_cyt4bb7/project/code/comm/coreDataTransfer.h

SHA-256

:

f87bae3f4fba18d06c3bbaf7d51f32d062268fd792e78de70c3425b6716f978d

代码 (c)

```
/*
 * 核间共享内存 (cm_7_0 飞控 ↔ cm_7_1 视觉)
 *
 * 物理区域: section .global_ram_data
 * startAddr = 0x28 001 000, size = 0x2000 (0x28 001 000 ~ 0x28 002 FFF)
 * 保留地址须避开: mt9v03x_h_num @ 0x28 006 BF0, mt9v03x_w_num @ 0x28 006 BF2
 *
 * 双核 D-Cache 一致性约定 (Cortex-M7, cache line = 32 字节):
 * - 两个传输方向放在各自独立、32 字节对齐、互不共享 cache line 的区域,
 *   避免 false-sharing (一核 clean 把另一核刚写的字节冲掉)。
 * - 写方: memcpy 进共享区 → SCB_CleanDCache_by_Addr (把脏行刷回 RAM)。
 * - 读方: SCB_InvalidateDCache_by_Addr (丢弃过期缓存) → memcpy 出。
 * - 握手用 32 位 magic (CORE_DATA_READY_MAGIC), 防 __no_init 上电垃圾误判。
 *
 * visionToFlight @ 0x28 001 000 (cm_7_1 写, cm_7_0 读)
 * flightToVision @ 0x28 001 100 (cm_7_0 写, cm_7_1 读) 两区相距 256B, 绝不共享 line
 */
#ifndef CORE_DATA_TRANSFER_H
#define CORE_DATA_TRANSFER_H

#include <stdint.h>

#include "flightParams.h"
#include "paramSyncMailbox.h"
#include "paramSyncGlobal.h"

#define CORE_DATA_VISION_ADDR (0x28 001 000u)
#define CORE_DATA_FLIGHT_ADDR (0x28 001 100u)
#define CORE_DATA_KVM_STATUS_ADDR (0x28 001 140u)
#define CORE_DATA_PARAM_GLOBAL_STATE_ADDR (0x28 001 160u)
#define CORE_DATA_PARAM_GLOBAL_REQUEST_ADDR (0x28 001 180u)
#define CORE_DATA_FLIGHT_MARKER_ADDR (0x280 011 C0u)
#define CORE_DATA_PARAM_ADDR (0x28 001 200u)
/* 0x28 001 300 不可用: paramRegion(paramToFlightStruct≈448B)从 0x28 001 200 起跨过该 256B 边界,
 * 故 ^ 灯板视觉调试快照另起 0x28 001 400(paramRegion 止于 ~0x280 013 C0, 独立 cache line, 编译期断言守护)。 */
#define CORE_DATA_VISDBG_ADDR (0x28 001 400u)
#define CORE_DATA_READY_MAGIC (0x5A5AA5A6u)

#define CORE0_FAULT_REASON_NONE (0u)
#define CORE0_FAULT_REASON_PROTECT_ATTITUDE (1u)
#define CORE0_FAULT_REASON_LAND_C1_LINK (2u)
#define CORE0_FAULT_REASON_LAND_C2_LINK (3u)
#define CORE0_FAULT_REASON_REQUIRED_VISION (4u)
#define CORE0_FAULT_REASON_LAND_LORA_MODE3 (5u)
#define CORE0_FAULT_REASON_LAND_LORA_MODE2 (6u)
#define CORE0_FAULT_REASON_LAND_TOF_FAILSAFE (7u)
#define CORE0_FAULT_REASON_LAND_C2_ATTITUDE (8u)
#define CORE0_FAULT_REASON_VISION_FLAG (0x80u)
#define CORE0_FAULT_REASON_VISION(source, faultClass) \
    (((uint8_t) (CORE0_FAULT_REASON_VISION_FLAG | (((source) & 0x07u) << 3) | ((faultClass) & 0x07u))))
#define CORE0_FAULT_REASON_VISION_SOURCE(reason) (((uint8_t) (((reason) >> 3) & 0x07u))
#define CORE0_FAULT_REASON_VISION_CLASS(reason) (((uint8_t) ((reason) & 0x07u))

/* cm_7_0 飞控 safetyState 跨核约定的非枚举值 (区别于枚举 0=DISARM/1=CLOSED_LOOP/2=PROTECTED) :
 * cm_7_0 写、cm_7_1 菜单读, 单一真相源 (消除两 TU 各自硬编码 0xFE/0xFF)。 */
#define FLIGHT_SAFETY_TOF_INIT (0xFDu) /* ToF 上电复位与初始化重试中 */
#define FLIGHT_SAFETY_ARMING (0xFEu) /* arming 进行中 (ESC 上电阻塞期) */
#define FLIGHT_SAFETY_IMUFAIL (0xFFu) /* IMU 初始化失败挂起 */

/* flightToVision.systemPhase: 运行 owner 裁决, 菜单只读显示。 */
#define CORE_UX_PHASE_LINK_WAIT (2u)
#define CORE_UX_PHASE_HOME (3u)
#define CORE_UX_PHASE_REJOIN (4u)
#define CORE_UX_PHASE_STANDALONE (5u)

/* 启动健康清单位。failedMask 只记录具有可靠返回值的初始化失败: void 型 SDK
 * 只能进入 unverifiedMask, 禁止伪造成 FAIL。两核各自填本地位后由屏幕 owner 合并。 */
```

```

#define CORE0_INIT_FAIL_IMU      (0x01u)
#define CORE0_INIT_FAIL_TOF      (0x02u)
#define CORE0_INIT_FAIL_OPTICAL_FLOW (0x04u)
#define CORE0_INIT_FAIL_CAMERA    (0x08u)

/* cm_7_1 视觉 → cm_7_0 飞控 (32 字节, 单 cache line) */
typedef struct
{
    float visionErrX; /* 像素误差 X (目标中点可调), 已乘极性 */
    float visionErrY; /* 像素误差 Y */
    float visionYawErr; /* ^ 朝向误差 (rad, wrapToPi), 已乘极性 */
    uint8_t visionTargetValid; /* 1=锁定有效, 0=丢失 */
    uint8_t padding0[3];
    uint32_t cm71Ready; /* = CORE_DATA_READY_MAGIC 表示 cm_7_1 初始化完成 */
    uint32_t visionSeq; /* 每发布一帧 +1, cm_7_0 据此判新鲜度 (陈旧则弃用误差) */
    float visionVelX; /* 原始质心按实测帧间隔差分后的低通速度 X (px/s), 已乘极性 */
    float visionVelY; /* 原始质心按实测帧间隔差分后的低通速度 Y (px/s), 已乘极性 */
} visionToFlightStruct;

/* cm_7_0 飞控 → cm_7_1 视觉 (64 字节, 2 cache line; 调试显示用) */
typedef struct
{
    float pitch; /* rad */
    float roll; /* rad */
    float yaw; /* rad */
    float altMm; /* 定高距离 mm */
    float baseThr; /* 基准油门 % */
    float motorPct[4]; /* FL/FR/BL/BR 油门 % */
    uint8_t safetyState; /* 安全状态机枚举 */
    uint8_t armProgress; /* arming 进度 0~100 */
    uint8_t tofHealthy; /* = altitudeHoldIsHealthy(): 1=链路活&&测距可信&&突变保护未悬挂(可发车);
    * 0 覆盖 init 失败/无效 range_status/距离越界/突变悬挂/链路失活 */
    uint8_t startupStateValid; /* 1=ARMING/TOF_INIT 启动状态有效; 不等价于最终初始化完成 */
    uint32_t cm70Ready; /* = CORE_DATA_READY_MAGIC 表示 cm_7_0 初始化完成 */
    /* 速度观测器状态 (KF 机体水平航向系 m/s, 发车页显示), 保持 64B/2 cache line. */
    float velX; /* KF 前向速度 m/s (axis[0]) */
    float velY; /* KF 右向速度 m/s (axis[1]) */
    uint8_t velValid; /* 1=KF 估计可信 (velocityEstimatorIsValid) */
    uint8_t padding1[3];
    uint8_t core1LinkUp; /* 远程菜单: 1=core1 核间链路在线(cm_7_0 crossCoreLink 心跳未超时), 供 cm_7_1 菜单断联保护/禁发车 */
    uint8_t core2LinkLost; /* 1=core1 已报告 RS-485↔core2 断联; 供 cm_7_1 发车页显示/禁发车 */
    uint8_t requiredVisionFault; /* 1=Core1 报告任一必需视觉故障 */
    uint8_t systemPhase; /* 2=LINK_WAIT 3=HOME 4=REJOIN 5=STANDALONE */
    uint8_t primaryReason; /* 0=READY 1=C1 2=C2 3=VISION 4=C2 ATT */
    uint8_t initFailedMask; /* CORE0_INIT_FAIL_*, cm_7_0 可观测失败 */
    uint8_t initUnverifiedMask; /* 无可靠自检的活动外设; 只显示 CHECK */
    uint8_t latchedFaultReason; /* CORE0_FAULT_REASON_*; cm_7_0 首因锁存, K5 ACK 清除 */
} flightToVisionStruct;

typedef char coreData_flightToVision_size_assert[(sizeof(flightToVisionStruct) == 64u) ? 1 : -1];

/* cm_7_0 链路核 → cm_7_1 菜单核: 独立 KVM 状态 cache line. */
typedef struct
{
    uint32_t magic;
    uint32_t targetSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t ackEventSeq;
    uint32_t statusSeq;
    uint16_t core3AgeMs;
    uint8_t target;
    uint8_t activeTarget;
    uint8_t menuStates;
    uint8_t flags;
    uint8_t ackResult;
} kvmStatusToMenuStruct;

typedef char coreData_kvm_status_size_assert[(sizeof(kvmStatusToMenuStruct) == 32u) ? 1 : -1];
typedef char coreData_kvm_status_align_assert[(((CORE_DATA_KVM_STATUS_ADDR % 32u) == 0u) ? 1 : -1)];
typedef char coreData_kvm_status_after_flight_assert[(CORE_DATA_FLIGHT_ADDR + sizeof(flightToVisionStruct) <=
CORE_DATA_KVM_STATUS_ADDR) ? 1 : -1];

#define CORE_DATA_PARAM_GLOBAL_MAGIC (0x50474C42u)
typedef struct
{
    uint32_t magic;
    paramSyncGlobalState value;
} coreDataParamGlobalStateStruct;

```

```

typedef struct
{
    uint32_t magic;
    paramSyncGlobalRequest value;
    uint8_t padding[16];
} coreDataParamGlobalRequestStruct;
typedef char coreData_param_global_state_size_assert[(sizeof(coreDataParamGlobalStateStruct) == 32u) ? 1 : -1];
typedef char coreData_param_global_request_size_assert[(sizeof(coreDataParamGlobalRequestStruct) == 32u) ? 1 : -1];
typedef char coreData_param_global_state_align_assert[(((CORE_DATA_PARAM_GLOBAL_STATE_ADDR & 31u) == 0u) ? 1 : -1)];
typedef char coreData_param_global_request_align_assert[(((CORE_DATA_PARAM_GLOBAL_REQUEST_ADDR & 31u) == 0u) ? 1 : -1)];
typedef char coreData_param_global_after_kvm_assert[
    ((CORE_DATA_KVM_STATUS_ADDR + sizeof(kvmStatusToMenuStruct)) <=
    CORE_DATA_PARAM_GLOBAL_STATE_ADDR) ? 1 : -1];
typedef char coreData_param_global_state_before_request_assert[
    ((CORE_DATA_PARAM_GLOBAL_STATE_ADDR + sizeof(coreDataParamGlobalStateStruct)) <=
    CORE_DATA_PARAM_GLOBAL_REQUEST_ADDR) ? 1 : -1];

/* flightToVision 跨两条 cache line，独立 marker 提供 odd/even 发布事务。 */
typedef struct
{
    uint32_t magic;
    uint32_t reserved;
    uint32_t generation;
    uint8_t padding[20];
} coreDataFlightMarkerStruct;
typedef char coreData_flight_marker_size_assert[(sizeof(coreDataFlightMarkerStruct) == 32u) ? 1 : -1];
typedef char coreData_flight_marker_align_assert[(((CORE_DATA_FLIGHT_MARKER_ADDR & 31u) == 0u) ? 1 : -1)];
typedef char coreData_flight_marker_after_param_global_assert[
    ((CORE_DATA_PARAM_GLOBAL_REQUEST_ADDR + sizeof(coreDataParamGlobalRequestStruct)) <=
    CORE_DATA_FLIGHT_MARKER_ADDR) ? 1 : -1];
typedef char coreData_flight_marker_before_param_assert[
    ((CORE_DATA_FLIGHT_MARKER_ADDR + sizeof(coreDataFlightMarkerStruct)) <=
    CORE_DATA_PARAM_ADDR) ? 1 : -1];

/* cm_7_1 菜单 → cm_7_0 飞控 (参数热重载 + 指令通道) @ 0x28001200
* - params: 全量飞控参数，cm_7_0 在 paramSeq 变化时整块 热重载
* - paramSeq: 参数 dirty 计数器，cm_7_1 每次下发参数 +1
* - cmdLaunchSeq: 发车指令边沿计数器，cm_7_0 检测增量触发起控
* - cmdStandbySeq: 回待机指令边沿计数器，cm_7_0 检测增量触发 CLOSED_LOOP→DISARMED
* - cmdClearSeq: 清空/恢复默认指令边沿计数器 (参数已在 params 内回默认)
* - cm71BootEpoch: cm_7_1 每次启动由共享区上一帧 cm71BootEpoch+1 派生 (共享区跨 cm_7_1
* 复位保留，故单调递增，每次复位必不同)。cm_7_0 据"epoch 变化 或 任一 seq 相对 last
* 回退"判定对端复位，只重同步 lastParamSeq/lastCmdSeq，绝不产生发车/热重载，杜绝
* cm_7_1 重启把计数器回退(N→0)被误判成新边沿 (会清突变悬挂 + 误恢复闭环)。
* - cm71ParamReady: 由共享区 owner 写入的奇偶事务 token；调用方不得自行赋值或解释
* 单向 (cm_7_1 写 / cm_7_0 读)，整块 clean/invalidate。注意这不是硬件原子事务：本结构
* 跨多 cache line，cm_7_0 读到的是某次写之后的一致快照 (写方整块 clean、读方整块
* invalidate)，靠 epoch/seq 边沿与回退判据防误触发，而非依赖单拍原子性。
* 起始 0x28001200 与 flightToVision(0x28001100) 相距 256B，独立 cache line。 */
typedef struct
{
    flightParamsStruct params;
    uint32_t paramSeq;
    uint32_t cmdLaunchSeq;
    uint32_t cmdStandbySeq;
    uint32_t cmdClearSeq;
    uint32_t cm71BootEpoch;
    uint32_t cm71ParamReady; /* 私有事务 token；仅 coreDataTransfer 读写 */
    uint32_t kvmRouteEpoch; /* 每次切目标递增；旧 route 的事件/ACK 自动失效 */
    uint32_t kvmEventSeq; /* 当前 route 内事件序号，0=尚无事件 */
    uint8_t kvmTarget; /* 0=Core0, 1=Core1, 3=Core3 */
    uint8_t kvmKeyId; /* 0..4 = K1..K5 */
    uint8_t kvmKeyState; /* menuKeyStateEnum: 1=SHORT 2=LONG */
    uint8_t kvmFlags; /* bit0=本 route 有待确认事件 */
} paramToFlightStruct;

typedef char coreData_flight_params_size_assert[(sizeof(flightParamsStruct) == 420u) ? 1 : -1];
typedef char coreData_param_to_flight_size_assert[(sizeof(paramToFlightStruct) == 456u) ? 1 : -1];

/* cm_7_1 视觉 → cm_7_0 (^ 灯板轮廓快照, 单向) @ 0x28001400
* cm_7_1 每帧从 chevronDetector 写本区；cm_7_0 用于 Policy 事实与 Core0 Pose，不连接串口调试输出。
* 结构跨两条 cache line，写方以 magic BUSY→payload→READY 两阶段提交；读方还校验前后 seq，
* TryRead 碰撞失败不得修改 last-good。seq 每个已提交帧+1 并跳过 0。
* 与 paramRegion(0x28001200)独立 cache line，无 false-sharing。字段布局按 4 字节对齐(padding0 补 selectedSize 后空隙)。 */
typedef struct
{
    uint32_t magic; /* = CORE_DATA_READY_MAGIC 表示 cm_7_1 已写过 */
    uint32_t seq; /* 每个已提交帧+1 并跳过 0；同时参与事务一致性确认 */
    uint8_t trackingState; /* OM_TRACK_SEARCHING/ACQUIRING/LOCKED/LOST_HOLDING */

```

```

uint8_t contourCount; /* 有效闭合轮廓数 */
uint8_t selectedIndex; /* 选中轮廓索引 */
uint8_t targetLost; /* 1=目标本体丢失(无候选/Hu 超阈/触边) */
uint16_t selectedCenterX; /* 选中轮廓原始质心 X */
uint16_t selectedCenterY; /* 选中轮廓原始质心 Y */
float filteredCenterX; /* 发布质心 X (字段名保留; 默认不过滤) */
float filteredCenterY; /* 发布质心 Y (字段名保留; 默认不过滤) */
uint16_t selectedSize; /* 选中轮廓像素数 */
uint8_t orientationFresh; /* 1=本帧朝向样本被接受且已有可信基线 */
uint8_t padding0; /* 对齐: 补 selectedSize 后到 selectedHuDistance 的 4 字节边界 */
float selectedHuDistance; /* 选中轮廓 Hu 距离(0=未参与 Hu) */
float orientation; /* 朝向 rad(-PI-PI) */
float orientationConfidence; /* 本帧朝向置信度 0~1(低置信只拒绝朝向更新) */
uint8_t orientProtectState; /* OM_ORIENT_NORMAL/SUSPENDED */
uint8_t orientSpikeFrameCount; /* 朝向跳变保护确认帧计数 */
uint8_t lostFrameCount;
uint8_t lockFrameCount;
} visionDebugToFlightStruct;

/* 编译期断言(照搬 core1 typedef-char 负长度惯用法); 地址不与 paramRegion 重叠 + 32 字节对齐 + 不越共享段尾 + 结构不超 128B。 */
typedef char coreData_visdbg_after_param_assert[(CORE_DATA_PARAM_ADDR + sizeof(paramToFlightStruct) <= CORE_DATA_VISDBG_ADDR) ? 1 :
-1];
typedef char coreData_visdbg_align_assert[((CORE_DATA_VISDBG_ADDR % 32u) == 0u) ? 1 : -1];
typedef char coreData_visdbg_end_assert[(CORE_DATA_VISDBG_ADDR + sizeof(visionDebugToFlightStruct) <= 0x28 003 000u) ? 1 : -1];
typedef char coreData_visdbg_exact_size_assert[(sizeof(visionDebugToFlightStruct) == 44u) ? 1 : -1];
typedef char coreData_visdbg_size_assert[(sizeof(visionDebugToFlightStruct) <= 128u) ? 1 : -1];
typedef char coreData_visdbg_before_param_sync_assert[
((CORE_DATA_VISDBG_ADDR + sizeof(visionDebugToFlightStruct)) <=
PARAM_SYNC_MAILBOX_REGION_ADDR) ? 1 : -1];
typedef char coreData_param_sync_region_end_assert[
((PARAM_SYNC_MAILBOX_REGION_ADDR + PARAM_SYNC_MAILBOX_REGION_BYTES) <=
0x28 002 000u) ? 1 : -1];

/* cm_7_1 调用: 写参数+指令 (Clean) */
void coreDataWriteParamToFlight(const paramToFlightStruct* src);
/* cm_7_0 调用: 读参数+指令 (Invalidate) */
uint8_t coreDataReadParamToFlight(paramToFlightStruct* dst);

/* cm_7_1 调用: 写视觉结果 (Clean) */
void coreDataWriteVisionToFlight(const visionToFlightStruct* src);
/* cm_7_0 调用: 读视觉结果 (Invalidate) */
void coreDataReadVisionToFlight(visionToFlightStruct* dst);

/* cm_7_0 调用: 写飞控调试数据 (Clean) */
void coreDataWriteFlightToVision(const flightToVisionStruct* src);
/* marker 事务读: 成功返回 1 并返回已确认的非零偶数 generation;
* 未提交/读写碰撞时返回 0, 且不修改 dst 或 generation。 */
uint8_t coreDataTryReadFlightToVision(flightToVisionStruct* dst,
uint32_t* generation);

void coreDataWriteKvmStatusToMenu(const kvmStatusToMenuStruct* src);
/* 事务读: 成功返回 1; 连续三轮读碰撞时返回 0 且不修改 dst。 */
uint8_t coreDataTryReadKvmStatusToMenu(kvmStatusToMenuStruct* dst);

/* cm_7_0 链路核与 cm_7_1 菜单核之间的全局五槽事务状态/请求桥。 */
void coreDataWriteParamGlobalState(const paramSyncGlobalState* src);
void coreDataClearParamGlobalState(void);
uint8_t coreDataTryReadParamGlobalState(paramSyncGlobalState* dst);
void coreDataWriteParamGlobalRequest(const paramSyncGlobalRequest* src);
uint8_t coreDataTryReadParamGlobalRequest(paramSyncGlobalRequest* dst);

/* cm_7_1 调用: 写 ^ 灯板视觉调试快照 (Clean) */
void coreDataWriteVisionDebugToFlight(const visionDebugToFlightStruct* src);
/* cm_7_0 调用: 一致性事务读; 成功返回 1, 碰撞/未提交返回 0 且不修改 dst。 */
uint8_t coreDataTryReadVisionDebugToFlight(visionDebugToFlightStruct* dst);

#endif // CORE_DATA_TRANSFER_H

```

core0_cyt4bb7/project/code/comm/crossCoreLink.c

SHA-256

:

1f7b298532b5b54dde0b27325535d0bce99034c2ed812fc839efe807ae237562

代码 (c)

```
#include "crossCoreLink.h"

#include <string.h>

#include "zf_driver_uart.h"
#include "zf_common_interrupt.h"

#include "pinMap.h"
#include "coreDataTransfer.h"
#include "uartLinkProto.h"
#include "paramSyncMailbox.h"
#include "zf_device_mt9v03x.h"

/*
 * crossCoreLink core0 cm_7_0 端实现。单线双向(UART_2/P10 ↔ core1 P04)。
 * 发按键事件+心跳; 收 core1 心跳维护存活。收发在前台, 绝不进飞控 ISR。
 */

#define CCL_UART_NUM (UART_2)
#define CCL_BAUDRATE (UART_LINK_BAUDRATE)
#if UART_LINK_BAUDRATE != 4000000u
#error "Core0-Core1 link baud contract requires 4 Mbps"
#endif
#define CCL_RX_QUEUE_SIZE (16u)
#define CCL_PARAM_RX_QUEUE_SIZE (4u)
#define CCL_PARAM_TRANSPORT_TIMEOUT_MS (600u)
#define CCL_PARAM_GLOBAL_STATE_LOST_MS (300u)
#define CCL_PARAM_GLOBAL_REQUEST_TX_MS (50u)
#define CCL_HEARTBEAT_MS (200u) /* 心跳发送周期 */
#define CCL_KVM_CONTROL_REFRESH_MS (50u) /* 独立于心跳续租, 降低前台抖动导致的假失权 */
#define CCL_LINK_LOST_MS (600u) /* 无 core1 帧超时判断断联 */
#define CCL_STATUS_LOST_MS (600u) /* STATUS 独立新鲜度; 其它帧不得续鲜其业务位 */
#define CCL_MOTION_LOST_MS (300u) /* 无 MOTION 帧超时→运动前馈失效(喂零防遥控失联失控) */
#define CCL_MOTION_ABS_MAX (200.0f) /* wire 数值完整性门; 正常底盘命令远小于此上限 */
#define CCL_CORE0_POSE_TX_MS (20u) /* core0 灯板同坐标位姿回传周期: 50Hz */
#define CCL_KVM_LEASE_MS (600u)
#define CCL_KVM_EVENT_RETRY_MS (100u)
#define CCL_KVM_STATUS_LOST_MS (600u)
#define CCL_CORE0_POSE_MAX_X ((float)(MT9V03X_W - 1u))
#define CCL_CORE0_POSE_MAX_Y ((float)(MT9V03X_H - 1u))
#define CCL_POSE_PI (3.14159265358979323846f)
#define CCL_OM_TRACK_LOCKED (2u) /* 与 openMorph.h OM_TRACK_LOCKED 同线值; 避免 cm_7_0 引入视觉模块头 */
#define CCL_OM_ORIENT_NORMAL (0u) /* 与 openMorph.h OM_ORIENT_NORMAL 同线值 */

typedef struct
{
    uartLinkParserStruct parser;
    uartLinkFrameStruct rxQueue[CCL_RX_QUEUE_SIZE];
    uartLinkFrameStruct paramRxQueue[CCL_PARAM_RX_QUEUE_SIZE];
    volatile uint8_t rxHead;
    volatile uint8_t rxTail;
    volatile uint8_t rxCount;
    volatile uint8_t paramRxHead;
    volatile uint8_t paramRxTail;
    volatile uint8_t paramRxCount;
    volatile uint8_t initReadyFlag;
    volatile uint8_t heartbeatSendFlag;
    volatile uint8_t kvmControlSendFlag;
    volatile uint8_t statusSendFlag;
    volatile uint8_t poseSendFlag;
    volatile uint16_t heartbeatTickCount;
    volatile uint16_t kvmControlTickCount;
    volatile uint16_t poseTickCount;
    volatile uint16_t core1QuietMs; /* 距上次收到 core1 合法帧 ms(ISR 累加, 收帧清零) */
    /* 运动前馈(MOTION 帧, core1 转发 core2 LoRa 遥控值): 前台 Poll 存值, Pit1ms 累加超时计时。 */
    float motionVx; /* 前台单写(Poll)/单读(GetMotion), 不进 ISR, 无需 volatile */
    float motionVy;
    float motionWz;
    uint8_t motionTakeoffReq; /* MOTION 帧 flags bit0: 模式 3 车 S3+S4 请求(前台 Poll 写/Get 读) */
    uint32_t motionRxSeq; /* 本地 MOTION 接收序号: 每收到一帧自增, 供调用方区分新帧与旧值保持 */
    volatile uint16_t motionQuietMs; /* 距上次 MOTION 帧 ms(Pit1ms 累加, Poll 收帧清零); 超时判遥控失联 */
    volatile uint8_t motionWirePrimed;
    uint8_t motionSessionFenced;
    uint8_t motionWireSeq;
    volatile uint16_t statusQuietMs; /* 距上次前进 STATUS 帧 ms; 与整链路存活正交 */
    uint8_t statusWirePrimed;
```

```

uint8_t statusWireSeq;
uint32_t producerBootSession;
uint8_t mode2LoraLandReqArmed; /* 新鲜低电平建立基线后才允许下一次高电平 */
uint8_t mode2RequestBaselineValid; /* 仅前进的真实低 STATUS 建立; stale 后禁止伪低累计 rearm */
uint8_t core1ReportedCore2Lost; /* STATUS 帧 linkFlags bit1: core1 报告 RS-485↔core2 断联(前台 Poll 写/Get 读)。
* core1 自身死时此值陈旧, 由 IsCore1Alive() 在缓降/发车拦截中兜底。 */

uint8_t core1ReportedCore2AttitudeFault;
uint8_t core1ReportedRequiredVisionFault;
uint8_t core1ReportedFaultSource;
uint8_t core1ReportedFaultClass;
uint8_t core1ReportedMode2LoraLandReq; /* STATUS 帧 linkFlags bit2: 模式 2 LoRa 双肩键调试缓降请求(前台 Poll 写/Get 读)。 */
uint16_t core1ReportedBeaconCount;
uint32_t statusRxSeq;
uint32_t beaconStateSeq;
uint8_t core1ReportedBeaconSearching;
uint8_t core1ReportedBeaconSearchTimedOut;
uint8_t core1ReportedBeaconStateValid;
uint8_t beaconStatePrimed;
volatile uint8_t controlActive; /* 1=core0 远程模式(操控 core1), 心跳携带给 core1 画绿框 */
/* CORE0_STATUS 帧内容(前台 SetCore0Status 写, 前台发送; 与心跳同周期发出供 core1 RGB 灯) */
volatile uint8_t c0SafetyState;
volatile uint8_t c0ArmProgress;
volatile uint8_t c0StatusFlags;
volatile uint8_t c0SysCtrl; /* 空地系统模式/RGB 意图/到顶(crossCoreLinkSetSysControl 写) */
volatile uint8_t c0Buzzer; /* 蜂鸣模式码(中转 core2) */
uint8_t statusTxSequence;
uint8_t poseTxSequence;
uint8_t poseEpoch;
uint8_t posePrimed; /* 1=已记录本次 core0 启动后的 visionDebug 基线 seq */
uint8_t heartbeatSequence;
uint8_t kvmControlSequence;
uint8_t kvmEventSequence;
uint32_t poseLastSeq; /* 已处理的 visionDebugRegion.seq */
uint32_t rxFrameCount;
uint32_t rxErrorCount;
uint32_t kvmRouteSession;
uint32_t kvmRouteEpoch;
uint32_t kvmEventSeq;
uint32_t kvmTargetSession;
volatile uint16_t kvmRetryTickMs;
volatile uint16_t kvmStatusQuietMs;
uint8_t kvmTarget;
uint8_t kvmKeyId;
uint8_t kvmKeyState;
volatile uint8_t kvmEventPending;
volatile uint8_t kvmEventSendFlag;
uint8_t kvmPeerRestartLatched;
kvmStatusToMenuStruct kvmStatus;
paramSyncMailboxRequest paramAssembly;
paramSyncMailboxResponse paramResponse;
uint32_t paramRequestSequence;
uint32_t paramWireRequestId;
uint32_t paramOwnerBootGeneration;
uint16_t paramExpectedBytes;
uint16_t paramReceivedBytes;
volatile uint16_t paramQuietMs;
uint8_t paramAssemblyActive;
uint8_t paramRequestPublished;
uint8_t paramResponseValid;
uint8_t paramTxSequence;
paramSyncGlobalState paramGlobalState;
paramSyncGlobalRequest paramGlobalRequest;
uint32_t paramGlobalRequestBoot;
uint32_t paramGlobalRequestSequence;
volatile uint16_t paramGlobalStateQuietMs;
volatile uint16_t paramGlobalRequestTxMs;
uint8_t paramGlobalStateValid;
uint8_t paramGlobalRequestAcked;
uint8_t paramGlobalTxSequence;
} crossCoreLinkStateStruct;

static crossCoreLinkStateStruct cclState;
__no_init uint8_t cclPoseEpochRetained;
__no_init uint32_t cclParamBridgeBootRetained;
__no_init uint32_t cclParamMailboxRequestRetained;

static uint8_t cclStatusFrameSemanticValid(const uartLinkFrameStruct *frame);
static uint8_t cclBindProducerSessionFromStatus(uint32_t session);
static uint8_t cclMotionSessionMatches(uint32_t session);

```

```

static uint8_t cclWireSeqAdvanced(uint8_t sequence, uint8_t previous)
{
    uint8_t delta = (uint8_t)(sequence - previous);
    return (uint8_t)((delta != 0u) && (delta < 128u));
}

static uint8_t cclMotionFrameUsable(const uartLinkFrameStruct *frame)
{
    if ((frame == 0) ||
        ((frame->motionFlags & (uint8_t)~UART_LINK_MOTION_FLAG_TAKEOFF_REQ) != 0u))
    {
        return 0u;
    }
    /* NaN 使全部有序比较为 false, Inf 越界; 无需在高频前台引入 libm。 */
    return (uint8_t)((frame->motionVx > -CCL_MOTION_ABS_MAX) &&
        (frame->motionVx < CCL_MOTION_ABS_MAX) &&
        (frame->motionVy > -CCL_MOTION_ABS_MAX) &&
        (frame->motionVy < CCL_MOTION_ABS_MAX) &&
        (frame->motionWz > -CCL_MOTION_ABS_MAX) &&
        (frame->motionWz < CCL_MOTION_ABS_MAX));
}

static uint8_t cclMotionFrameIsZeroFence(const uartLinkFrameStruct *frame)
{
    return (uint8_t)((frame != 0) &&
        (frame->motionVx == 0.0f) && (frame->motionVy == 0.0f) &&
        (frame->motionWz == 0.0f) && (frame->motionFlags == 0u));
}

static uint8_t cclStatusFrameSemanticValid(const uartLinkFrameStruct *frame)
{
    const uint8_t allowedFlags = UART_LINK_STATUS_FLAG_LINK_OK |
        UART_LINK_STATUS_FLAG_CORE2_LOST |
        UART_LINK_STATUS_FLAG_MODE2_LORA_LAND_REQ |
        UART_LINK_STATUS_FLAG_REQUIRED_VISION_FAULT |
        UART_LINK_STATUS_FLAG_BEACON_SEARCHING |
        UART_LINK_STATUS_FLAG_BEACON_SEARCH_TIMEOUT |
        UART_LINK_STATUS_FLAG_BEACON_VALID |
        UART_LINK_STATUS_FLAG_CORE2_ATTITUDE_FAULT;

    uint8_t faultActive;
    uint8_t beaconValid;
    uint8_t searchFlags;
    if ((frame == 0) || (frame->type != UART_LINK_TYPE_STATUS) ||
        (frame->producerBootSession == 0u) ||
        ((frame->linkFlags & UART_LINK_STATUS_FLAG_LINK_OK) == 0u) ||
        ((frame->linkFlags & (uint8_t)~allowedFlags) != 0u)) return 0u;
    faultActive = (frame->linkFlags &
        UART_LINK_STATUS_FLAG_REQUIRED_VISION_FAULT)
        ? 1u
        : 0u;
    if (((faultActive == 0u) &&
        ((frame->faultSource != 0u) || (frame->faultClass != 0u))) ||
        ((faultActive != 0u) &&
        (((frame->faultSource < 1u) || (frame->faultSource > 4u)) ||
        ((frame->faultClass < 1u) || (frame->faultClass > 7u))))) return 0u;
    beaconValid = (frame->linkFlags & UART_LINK_STATUS_FLAG_BEACON_VALID) ? 1u : 0u;
    searchFlags = frame->linkFlags & (UART_LINK_STATUS_FLAG_BEACON_SEARCHING |
        UART_LINK_STATUS_FLAG_BEACON_SEARCH_TIMEOUT);
    if ((beaconValid == 0u) &&
        ((frame->beaconCount != 0u) || (searchFlags != 0u))) return 0u;
    if (searchFlags == (UART_LINK_STATUS_FLAG_BEACON_SEARCHING |
        UART_LINK_STATUS_FLAG_BEACON_SEARCH_TIMEOUT)) return 0u;
    return 1u;
}

static void cclEnterStatusProtocolFault(void)
{
    cclState.statusQuietMs = CCL_STATUS_LOST_MS;
    cclState.core1ReportedCore2Lost = 1u;
    cclState.core1ReportedCore2AttitudeFault = 0u;
    cclState.core1ReportedRequiredVisionFault = 1u;
    cclState.core1ReportedFaultSource = 2u;
    cclState.core1ReportedFaultClass = 4u;
    cclState.core1ReportedMode2LoraLandReq = 0u;
    cclState.mode2LoraLandReqArmed = 0u;
    cclState.mode2RequestBaselineValid = 0u;
    cclState.motionWirePrimed = 0u;
    cclState.motionSessionFenced = 0u;
}

```



```

cclState.motionVx = 0.0f;
cclState.motionVy = 0.0f;
cclState.motionWz = 0.0f;
cclState.motionTakeoffReq = 0u;
cclState.motionQuietMs = CCL_MOTION_LOST_MS;
cclState.rxErrorCount++;
}

static void cclCommitBeaconState(const uartLinkFrameStruct *frame)
{
    uint16_t count = frame->beaconCount;
    uint8_t searching =
        (frame->linkFlags & UART_LINK_STATUS_FLAG_BEACON_SEARCHING) ? 1u : 0u;
    uint8_t timedOut =
        (frame->linkFlags & UART_LINK_STATUS_FLAG_BEACON_SEARCH_TIMEOUT) ? 1u : 0u;
    uint8_t valid =
        (frame->linkFlags & UART_LINK_STATUS_FLAG_BEACON_VALID) ? 1u : 0u;

    if ((cclState.beaconStatePrimed == 0u) ||
        (count != cclState.core1ReportedBeaconCount) ||
        (searching != cclState.core1ReportedBeaconSearching) ||
        (timedOut != cclState.core1ReportedBeaconSearchTimedOut) ||
        (valid != cclState.core1ReportedBeaconStateValid))
    {
        cclState.core1ReportedBeaconCount = count;
        cclState.core1ReportedBeaconSearching = searching;
        cclState.core1ReportedBeaconSearchTimedOut = timedOut;
        cclState.core1ReportedBeaconStateValid = valid;
        cclState.beaconStatePrimed = 1u;
        cclState.beaconStateSeq++;
        if (cclState.beaconStateSeq == 0u) cclState.beaconStateSeq = 1u;
    }
}

static void cclHandleStatusFrame(const uartLinkFrameStruct *frame)
{
    uint8_t mode2High =
        (frame->linkFlags & UART_LINK_STATUS_FLAG_MODE2_LORA_LAND_REQ) ? 1u : 0u;
    uint8_t statusStale =
        (cclState.statusQuietMs >= CCL_STATUS_LOST_MS) ? 1u : 0u;
    uint8_t sequenceAdvanced = (cclState.statusWirePrimed == 0u) ? 1u : cclWireSeqAdvanced(frame->sequence, cclState.statusWireSeq);

    if (statusStale != 0u)
    {
        cclState.core1ReportedMode2LoraLandReq = 0u;
        cclState.mode2LoraLandReqArmed = 0u;
        cclState.mode2RequestBaselineValid = 0u;
    }
    if (sequenceAdvanced != 0u)
    {
        uint8_t faultActive = (frame->linkFlags &
                               UART_LINK_STATUS_FLAG_REQUIRED_VISION_FAULT)
                               ? 1u
                               : 0u;
        cclState.statusWirePrimed = 1u;
        cclState.statusWireSeq = frame->sequence;
        cclState.statusRxSeq++;
        if (cclState.statusRxSeq == 0u) cclState.statusRxSeq = 1u;
        cclState.statusQuietMs = 0u;
        cclCommitBeaconState(frame);
        cclState.core1ReportedCore2Lost =
            (frame->linkFlags & UART_LINK_STATUS_FLAG_CORE2_LOST) ? 1u : 0u;
        cclState.core1ReportedCore2AttitudeFault =
            (frame->linkFlags & UART_LINK_STATUS_FLAG_CORE2_ATTITUDE_FAULT) ? 1u : 0u;
        if (((faultActive == 0u) &&
              ((frame->faultSource != 0u) || (frame->faultClass != 0u))) ||
            ((faultActive != 0u) &&
              (((frame->faultSource < 1u) || (frame->faultSource > 4u)) ||
               ((frame->faultClass < 1u) || (frame->faultClass > 7u)))))
        {
            cclState.core1ReportedRequiredVisionFault = 1u;
            cclState.core1ReportedFaultSource = 2u;
            cclState.core1ReportedFaultClass = 4u;
        }
    }
    else
    {
        cclState.core1ReportedRequiredVisionFault = faultActive;
        cclState.core1ReportedFaultSource = frame->faultSource;
        cclState.core1ReportedFaultClass = frame->faultClass;
    }
}

```

```

    }
    if (mode2High == 0u)
    {
        cclState.core1ReportedMode2LoraLandReq = 0u;
        cclState.mode2LoraLandReqArmed = 1u;
        cclState.mode2RequestBaselineValid = 1u;
    }
    else if (cclState.mode2LoraLandReqArmed != 0u)
    {
        cclState.core1ReportedMode2LoraLandReq = 1u;
        cclState.mode2LoraLandReqArmed = 0u;
    }
}
else if ((statusStale != 0u) && (mode2High == 0u))
{
    /* 隔离重建帧只重锚 wire 身份，不刷新任何业务事实或本地业务序号。
    * 下一前进低帧才恢复 STATUS，并重新开始 Coordinator 的 400ms rearm。 */
    cclState.statusWirePrimed = 1u;
    cclState.statusWireSeq = frame->sequence;
}
}

static uint8_t cclNextIndex(uint8_t index)
{
    index++;
    if (index >= CCL_RX_QUEUE_SIZE) index = 0u;
    return index;
}

static uint8_t cclNextParamIndex(uint8_t index)
{
    index++;
    if (index >= CCL_PARAM_RX_QUEUE_SIZE) index = 0u;
    return index;
}

/* ISR 上下文: 解析一字节, 成帧入队; 轻量, 不做共享内存/DCache。 */
static void cclHandleByte(uint8_t data)
{
    {
        uartLinkFrameStruct frame;
        uartLinkProtoStatusEnum status = uartLinkProtoInputByte(&cclState.parser, data, &frame);

        if (UART_LINK_PROTO_STATUS_FRAME_READY != status)
        {
            if (UART_LINK_PROTO_STATUS_CHECK_ERROR == status || UART_LINK_PROTO_STATUS_TAIL_ERROR == status ||
                UART_LINK_PROTO_STATUS_TYPE_ERROR == status)
            {
                cclState.rxErrorCount++;
            }
            return;
        }

        /* 参数同步不得挤占控制队列，也不得给控制/状态链路续鲜。参数队列满时
        * 丢弃本帧并依靠发送端重试，绝不淘汰既有控制帧。 */
        if (UART_LINK_TYPE_PARAM_REQUEST == frame.type)
        {
            if (cclState.paramRxCount >= CCL_PARAM_RX_QUEUE_SIZE)
            {
                cclState.rxErrorCount++;
                return;
            }
            cclState.paramRxQueue[cclState.paramRxTail] = frame;
            cclState.paramRxTail = cclNextParamIndex(cclState.paramRxTail);
            cclState.paramRxCount++;
            cclState.rxFrameCount++;
            return;
        }
        if (UART_LINK_TYPE_PARAM_RESPONSE == frame.type)
        {
            cclState.rxErrorCount++;
            return;
        }
        if (UART_LINK_TYPE_PARAM_GLOBAL_STATE == frame.type)
        {
            if (cclState.rxCount >= CCL_RX_QUEUE_SIZE)
            {
                cclState.rxErrorCount++;
                return;
            }
        }
    }
}

```

```

        cclState.rxQueue[cclState.rxTail] = frame;
        cclState.rxTail = cclNextIndex(cclState.rxTail);
        cclState.rxCount++;
        return;
    }
    if (UART_LINK_TYPE_PARAM_GLOBAL_REQUEST == frame.type)
    {
        cclState.rxErrorCount++;
        return;
    }
    cclState.core1QuietMs = 0u; /* 仅业务/控制合法帧证明 core1 控制链在线 */

    if (cclState.rxCount >= CCL_RX_QUEUE_SIZE)
    {
        cclState.rxHead = cclNextIndex(cclState.rxHead);
        cclState.rxCount--;
    }
    cclState.rxQueue[cclState.rxTail] = frame;
    cclState.rxTail = cclNextIndex(cclState.rxTail);
    cclState.rxCount++;
    cclState.rxFrameCount++;
}

static uint8_t cclPopFrame(uartLinkFrameStruct *frame)
{
    uint8_t got = 0u;
    uart_rx_interrupt(CCL_UART_NUM, 0u);
    if (cclState.rxCount > 0u)
    {
        *frame = cclState.rxQueue[cclState.rxHead];
        cclState.rxHead = cclNextIndex(cclState.rxHead);
        cclState.rxCount--;
        got = 1u;
    }
    uart_rx_interrupt(CCL_UART_NUM, 1u);
    return got;
}

static void cclSendHeartbeat(void)
{
    uint8_t buffer[UART_LINK_FRAME_LENGTH];
    uartLinkFrameStruct frame;
    frame.type = UART_LINK_TYPE_HEARTBEAT;
    frame.sequence = cclState.heartbeatSequence;
    /* value 携带操控态: 1=core0 远程模式正操控 core1, 0=本地模式。core1 据此画绿/红边框。 */
    frame.value = cclState.controlActive ? 1u : 0u;
    if (UART_LINK_PROTO_STATUS_OK == uartLinkProtoEncode(&frame, buffer, (uint8_t) sizeof(buffer)))
    {
        uart_write_buffer(CCL_UART_NUM, buffer, UART_LINK_FRAME_LENGTH);
        cclState.heartbeatSequence++;
    }
}

static uint8_t cclPopParamFrame(uartLinkFrameStruct *frame)
{
    uint8_t got = 0u;
    uart_rx_interrupt(CCL_UART_NUM, 0u);
    if (cclState.paramRxCount > 0u)
    {
        *frame = cclState.paramRxQueue[cclState.paramRxHead];
        cclState.paramRxHead = cclNextParamIndex(cclState.paramRxHead);
        cclState.paramRxCount--;
        got = 1u;
    }
    uart_rx_interrupt(CCL_UART_NUM, 1u);
    return got;
}

static void cclSendParamGlobalRequest(const paramSyncGlobalRequest *request)
{
    uint8_t buffer[UART_LINK_FRAME_LENGTH_PARAM_GLOBAL_REQUEST];
    uartLinkFrameStruct frame;
    if ((request == 0) || (paramSyncGlobalRequestIsValid(request) == 0u)) return;
    memset(&frame, 0, sizeof(frame));
    frame.type = UART_LINK_TYPE_PARAM_GLOBAL_REQUEST;
    frame.sequence = cclState.paramGlobalTxSequence++;
    frame.paramGlobalRequest = *request;
    if (uartLinkProtoEncode(&frame, buffer, sizeof(buffer)) == UART_LINK_PROTO_STATUS_OK)
        uart_write_buffer(CCL_UART_NUM, buffer, sizeof(buffer));
}

```

```

}

static void cclCancelPendingParamGlobalRequest(void)
{
    if ((cclState.paramGlobalRequestAcked == 0u) &&
        (paramSyncGlobalRequestIsValid(&cclState.paramGlobalRequest) != 0u))
        cclState.paramGlobalRequestAcked = 1u;
}

static void cclAcceptParamGlobalState(const paramSyncGlobalState *state)
{
    uint8_t accept;
    if (paramSyncGlobalStateIsValid(state) == 0u)
    {
        cclState.paramGlobalStateValid = 0u;
        cclState.paramGlobalStateQuietMs = CCL_PARAM_GLOBAL_STATE_LOST_MS;
        coreDataClearParamGlobalState();
        cclCancelPendingParamGlobalRequest();
        cclState.rxErrorCount++;
        return;
    }
    accept = (uint8_t) ((cclState.paramGlobalStateValid == 0u) ||
        (state->coordinatorBootGeneration !=
            cclState.paramGlobalState.coordinatorBootGeneration) ||
        ((uint32_t) (state->sequence - cclState.paramGlobalState.sequence) <
            0x80 000 000u && state->sequence != cclState.paramGlobalState.sequence));
    if (accept == 0u) return;
    if ((paramSyncGlobalRequestIsValid(&cclState.paramGlobalRequest) != 0u) &&
        (cclState.paramGlobalRequestAcked == 0u))
    {
        if ((cclState.paramGlobalState.coordinatorBootGeneration != 0u) &&
            (state->coordinatorBootGeneration !=
                cclState.paramGlobalState.coordinatorBootGeneration))
            cclState.paramGlobalRequestAcked = 1u;
        else if (paramSyncGlobalStateMatchesRequest(state,
            &cclState.paramGlobalRequest) != 0u)
            cclState.paramGlobalRequestAcked = 1u;
    }
    cclState.paramGlobalState = *state;
    cclState.paramGlobalStateValid = 1u;
    cclState.paramGlobalStateQuietMs = 0u;
    coreDataWriteParamGlobalState(state);
}

static void cclSendParamResponse(const uartLinkFrameStruct *request,
    uint8_t status, const uint8_t *data, uint8_t length, uint16_t offset)
{
    uint8_t buffer[UART_LINK_FRAME_LENGTH_PARAM];
    uartLinkFrameStruct response;
    uint8_t i;
    memset(&response, 0, sizeof(response));
    response.type = UART_LINK_TYPE_PARAM_RESPONSE;
    response.sequence = cclState.paramTxSequence++;
    response.paramOp = (request->paramOp == UART_LINK_PARAM_OP_POLL &&
        cclState.paramResponseValid != 0u) ?
        cclState.paramAssembly.op : request->paramOp;
    response.paramReserved = request->paramReserved;
    response.paramTarget = request->paramTarget;
    response.paramStatus = status;
    response.paramRequestId = request->paramRequestId;
    response.paramOffset = offset;
    response.paramLength = length;
    for (i = 0u; i < length; i++) response.paramData[i] = data[i];
    if (uartLinkProtoEncode(&response, buffer, sizeof(buffer)) == UART_LINK_PROTO_STATUS_OK)
        uart_write_buffer(CCL_UART_NUM, buffer, sizeof(buffer));
}

static void cclResetParamTransport(void)
{
    cclState.paramAssemblyActive = 0u;
    cclState.paramRequestPublished = 0u;
    cclState.paramResponseValid = 0u;
    cclState.paramExpectedBytes = 0u;
    cclState.paramReceivedBytes = 0u;
    cclState.paramWireRequestId = 0u;
    cclState.paramQuietMs = CCL_PARAM_TRANSPORT_TIMEOUT_MS;
}

static uint8_t cclParamIdentityMatches(const uartLinkFrameStruct *frame)

```

```

{
    return (uint8_t) ((frame->paramRequestId ==
        cclState.paramWireRequestId) &&
        (frame->paramOp == cclState.paramAssembly.op) &&
        (frame->paramReserved == 0u) &&
        (frame->paramTarget == cclState.paramAssembly.target));
}

static uint8_t cclPublishParamAssembly(void)
{
    cclState.paramAssembly.dataLength = cclState.paramExpectedBytes;
    cclState.paramRequestSequence++;
    if (cclState.paramRequestSequence == 0u) cclState.paramRequestSequence = 1u;
    cclParamMailboxRequestRetained++;
    if (cclParamMailboxRequestRetained == 0u)
        cclParamMailboxRequestRetained = 1u;
    cclState.paramAssembly.producerBootGeneration = cclParamBridgeBootRetained;
    cclState.paramAssembly.sequence = cclState.paramRequestSequence;
    cclState.paramAssembly.transactionId = cclParamMailboxRequestRetained;
    cclState.paramAssembly.offset = cclState.paramRequestSequence;
    cclState.paramAssembly.slot = (cclState.paramExpectedBytes != 0u) ?
        cclState.paramAssembly.data[0] : 0u;
    if (paramSyncMailboxPublishRequest(&cclState.paramAssembly) == 0u)
    {
        cclResetParamTransport();
        return 0u;
    }
    cclState.paramAssemblyActive = 0u;
    cclState.paramRequestPublished = 1u;
    cclState.paramResponseValid = 0u;
    cclState.paramQuietMs = 0u;
    return 1u;
}

static void cclHandleParamRequest(const uartLinkFrameStruct *frame)
{
    paramSyncMailboxResponse response;
    uint8_t status = PARAM_SYNC_STATUS_PENDING;
    uint8_t length = 0u;
    uint8_t data[UART_LINK_PARAM_DATA_BYTES];
    uint16_t offset = frame->paramOffset;
    if (frame->paramOp == UART_LINK_PARAM_OP_POLL)
    {
        uint16_t remaining;
        if ((frame->paramLength != 0u) || (frame->paramStatus != 0u) ||
            (cclState.paramRequestPublished == 0u) ||
            (frame->paramRequestId != cclState.paramWireRequestId) ||
            (frame->paramReserved != 0u) ||
            (frame->paramTarget != cclState.paramAssembly.target))
        {
            cclResetParamTransport();
            status = PARAM_SYNC_STATUS_INVALID;
        }
        else
        {
            cclState.paramQuietMs = 0u;
            if (!cclState.paramResponseValid &&
                paramSyncMailboxTryReadResponse(&response) &&
                response.transactionId == cclState.paramAssembly.transactionId &&
                response.offset == cclState.paramAssembly.offset &&
                response.op == cclState.paramAssembly.op &&
                response.slot == cclState.paramAssembly.slot &&
                response.target == cclState.paramAssembly.target)
            {
                if ((response.producerBootGeneration == 0u) ||
                    ((cclState.paramOwnerBootGeneration != 0u) &&
                     (response.producerBootGeneration !=
                      cclState.paramOwnerBootGeneration)))
                {
                    if (response.producerBootGeneration != 0u)
                        cclState.paramOwnerBootGeneration =
                            response.producerBootGeneration;
                    cclResetParamTransport();
                    status = UART_LINK_PARAM_OWNER_RESTART;
                }
                else
                {
                    cclState.paramOwnerBootGeneration =
                        response.producerBootGeneration;
                }
            }
        }
    }
}

```

```

        cclState.paramResponse = response;
        cclState.paramResponseValid = 1u;
    }
}
if (cclState.paramResponseValid)
{
    status = (uint8_t) (cclState.paramResponse.status &
        UART_LINK_PARAM_STATUS_MASK);
    if ((status > PARAM_SYNC_STATUS_FAILED) ||
        (offset > cclState.paramResponse.dataLength))
    {
        cclResetParamTransport();
        status = PARAM_SYNC_STATUS_INVALID;
    }
    else
    {
        remaining = (uint16_t) (cclState.paramResponse.dataLength - offset);
        length = (remaining > UART_LINK_PARAM_DATA_BYTES) ? UART_LINK_PARAM_DATA_BYTES : (uint8_t) remaining;
        memcpy(data, &cclState.paramResponse.data[offset], length);
        if (remaining > length) status |= UART_LINK_PARAM_TRANSPORT_MORE;
    }
}
}
}
else if ((frame->paramStatus & UART_LINK_PARAM_TRANSPORT_BEGIN) != 0u)
{
    uint8_t sameRequest = (uint8_t) (cclParamIdentityMatches(frame) &&
        (frame->paramOffset == cclState.paramExpectedBytes) &&
        ((cclState.paramAssemblyActive != 0u) ||
            (cclState.paramRequestPublished != 0u)));
    if ((frame->paramStatus != UART_LINK_PARAM_TRANSPORT_BEGIN) ||
        (frame->paramLength != 0u) ||
        (frame->paramReserved != 0u) ||
        (frame->paramTarget != 0u) ||
        (frame->paramOffset > PARAM_SYNC_MAILBOX_DATA_MAX))
    {
        cclResetParamTransport();
        status = PARAM_SYNC_STATUS_INVALID;
    }
    else if (sameRequest != 0u)
        cclState.paramQuietMs = 0u;
    else
    {
        cclResetParamTransport();
        memset(&cclState.paramAssembly, 0, sizeof(cclState.paramAssembly));
        cclState.paramExpectedBytes = frame->paramOffset;
        cclState.paramReceivedBytes = 0u;
        cclState.paramAssembly.op = frame->paramOp;
        cclState.paramAssembly.target = frame->paramTarget;
        cclState.paramWireRequestId = frame->paramRequestId;
        cclState.paramAssemblyActive = 1u;
        cclState.paramQuietMs = 0u;
        if (cclState.paramExpectedBytes == 0u && !cclPublishParamAssembly())
            status = PARAM_SYNC_STATUS_FAILED;
    }
}
else if ((frame->paramStatus == 0u) && (frame->paramLength != 0u) &&
    cclParamIdentityMatches(frame) &&
    ((cclState.paramAssemblyActive != 0u) ||
        (cclState.paramRequestPublished != 0u)) &&
    ((uint32_t) frame->paramOffset + frame->paramLength <=
        cclState.paramExpectedBytes))
{
    if (frame->paramOffset < cclState.paramReceivedBytes)
    {
        if (((uint32_t) frame->paramOffset + frame->paramLength >
            cclState.paramReceivedBytes) ||
            (memcmp(&cclState.paramAssembly.data[frame->paramOffset],
                frame->paramData, frame->paramLength) != 0))
        {
            cclResetParamTransport();
            status = PARAM_SYNC_STATUS_INVALID;
        }
        else cclState.paramQuietMs = 0u;
    }
    else if ((cclState.paramAssemblyActive != 0u) &&
        (frame->paramOffset == cclState.paramReceivedBytes))
    {
        memcpy(&cclState.paramAssembly.data[cclState.paramReceivedBytes],

```

```

        frame->paramData, frame->paramLength);
    cclState.paramReceivedBytes = (uint16_t) (cclState.paramReceivedBytes +
        frame->paramLength);
    cclState.paramQuietMs = 0u;
    if (cclState.paramReceivedBytes == cclState.paramExpectedBytes &&
        !cclPublishParamAssembly()) status = PARAM_SYNC_STATUS_FAILED;
    }
    else
    {
        cclResetParamTransport();
        status = PARAM_SYNC_STATUS_INVALID;
    }
}
else
{
    cclResetParamTransport();
    status = PARAM_SYNC_STATUS_INVALID;
}
cclSendParamResponse(frame, status, data, length, offset);
}

static void cclSendKvmControl(void)
{
    uint8_t buffer[UART_LINK_FRAME_LENGTH_KVM_CONTROL];
    uartLinkFrameStruct frame;
    frame.type = UART_LINK_TYPE_KVM_CONTROL;
    frame.sequence = cclState.kvmControlSequence;
    frame.kvmTargetSession = cclState.kvmTargetSession;
    frame.kvmRouteSession = cclState.kvmRouteSession;
    frame.kvmRouteEpoch = cclState.kvmRouteEpoch;
    frame.kvmTarget = cclState.kvmTarget;
    frame.kvmLeaseMs = CCL_KVM_LEASE_MS;
    if (UART_LINK_PROTO_STATUS_OK == uartLinkProtoEncode(&frame, buffer, (uint8_t) sizeof(buffer)))
    {
        uart_write_buffer(CCL_UART_NUM, buffer, (uint8_t) sizeof(buffer));
        cclState.kvmControlSequence++;
    }
}

static void cclSendKvmEvent(void)
{
    uint8_t buffer[UART_LINK_FRAME_LENGTH_KVM_EVENT];
    uartLinkFrameStruct frame;
    if ((cclState.kvmEventPending == 0u) || (cclState.kvmTargetSession == 0u)) return;
    frame.type = UART_LINK_TYPE_KVM_EVENT;
    frame.sequence = cclState.kvmEventSequence;
    frame.kvmTargetSession = cclState.kvmTargetSession;
    frame.kvmRouteSession = cclState.kvmRouteSession;
    frame.kvmRouteEpoch = cclState.kvmRouteEpoch;
    frame.kvmEventSeq = cclState.kvmEventSeq;
    frame.kvmTarget = cclState.kvmTarget;
    frame.kvmKeyId = cclState.kvmKeyId;
    frame.kvmKeyState = cclState.kvmKeyState;
    if (UART_LINK_PROTO_STATUS_OK == uartLinkProtoEncode(&frame, buffer, (uint8_t) sizeof(buffer)))
    {
        uart_write_buffer(CCL_UART_NUM, buffer, (uint8_t) sizeof(buffer));
        cclState.kvmEventSequence++;
    }
}

static void cclPublishKvmStatus(void)
{
    cclState.kvmStatus.magic = CORE_DATA_READY_MAGIC;
    cclState.kvmStatus.statusSeq++;
    coreDataWriteKvmStatusToMenu(&cclState.kvmStatus);
}

static void cclHandleKvmStatus(const uartLinkFrameStruct *frame)
{
    uint32_t previousSession;
    if ((frame->kvmRouteSession != cclState.kvmRouteSession) ||
        (frame->kvmRouteEpoch != cclState.kvmRouteEpoch) ||
        (frame->kvmTarget != cclState.kvmTarget))
    {
        return;
    }
    previousSession = cclState.kvmTargetSession;
    cclState.kvmTargetSession = frame->kvmTargetSession;
    if ((previousSession == 0u) && (frame->kvmTargetSession != 0u))

```

```

{
    /* 首帧只用于发现目标 session; 学到真实身份后同轮补发有效 CONTROL。 */
    cclState.kvmControlSendFlag = 1u;
}
cclState.kvmStatusQuietMs = 0u;
cclState.kvmStatus.targetSession = frame->kvmTargetSession;
cclState.kvmStatus.routeSession = frame->kvmRouteSession;
cclState.kvmStatus.routeEpoch = frame->kvmRouteEpoch;
cclState.kvmStatus.ackEventSeq = frame->kvmEventSeq;
cclState.kvmStatus.core3AgeMs = frame->kvmCore3AgeMs;
cclState.kvmStatus.target = frame->kvmTarget;
cclState.kvmStatus.activeTarget = frame->kvmActiveTarget;
cclState.kvmStatus.menuStates = frame->kvmMenuStates;
cclState.kvmStatus.flags = frame->kvmFlags;
cclState.kvmStatus.ackResult = frame->kvmAckResult;
if ((previousSession != 0u) && (previousSession != frame->kvmTargetSession))
{
    cclState.kvmPeerRestartLatched = 1u;
    if (cclState.kvmEventPending != 0u)
    {
        cclState.kvmEventPending = 0u;
        cclState.kvmStatus.flags |= UART_LINK_KVM_FLAG_ACK_TERMINAL;
        cclState.kvmStatus.ackEventSeq = cclState.kvmEventSeq;
        cclState.kvmStatus.ackResult = UART_LINK_KVM_ACK_PEER_RESTART;
    }
}
else if ((cclState.kvmEventPending != 0u) &&
        ((frame->kvmFlags & UART_LINK_KVM_FLAG_ACK_TERMINAL) != 0u) &&
        (frame->kvmEventSeq == cclState.kvmEventSeq))
{
    cclState.kvmEventPending = 0u;
}
if (cclState.kvmPeerRestartLatched != 0u)
    cclState.kvmStatus.flags |= UART_LINK_KVM_FLAG_PEER_RESTART;
cclPublishKvmStatus();
}

/* 发 CORE0_STATUS 帧(10 字节)给 core1: 飞控安全态 + arming 进度 + 状态位(供 RGB 状态灯)
* + sysCtrl(空地系统模式/RGB 意图/到顶) + buzzer(蜂鸣模式码)。与心跳同周期(前台), 绝不进飞控 ISR。 */
static void cclSendCore0Status(void)
{
    uint8_t buffer[UART_LINK_FRAME_LENGTH_C0STATUS];
    uartLinkFrameStruct frame;
    frame.type = UART_LINK_TYPE_CORE0_STATUS;
    frame.sequence = cclState.statusTxSequence;
    frame.safetyState = cclState.c0SafetyState;
    frame.armProgress = cclState.c0ArmProgress;
    frame.statusFlags = cclState.c0StatusFlags;
    frame.sysCtrl = cclState.c0SysCtrl; /* 空地系统模式/RGB 意图/到顶(crossCoreLinkSetSysControl 写, 默认 0) */
    frame.buzzer = cclState.c0Buzzer; /* 蜂鸣模式码(默认 SILENT=0) */
    if (UART_LINK_PROTO_STATUS_OK == uartLinkProtoEncode(&frame, buffer, (uint8_t) sizeof(buffer)))
    {
        uart_write_buffer(CCL_UART_NUM, buffer, UART_LINK_FRAME_LENGTH_C0STATUS);
        cclState.statusTxSequence++;
    }
}

static uint8_t cclPoseFinite(float v)
{
    {
        return (v == v && v > -100 000.0f && v < 100 000.0f) ? 1u : 0u;
    }
}

static float cclPoseClamp(float v, float lo, float hi)
{
    {
        if (v < lo) return lo;
        if (v > hi) return hi;
        return v;
    }
}

static uint16_t cclPoseQ8(float v)
{
    {
        if (v < 0.0f) v = 0.0f;
        if (v > 255.996f) v = 255.996f;
        return (uint16_t) (v * 256.0f + 0.5f);
    }
}

static uint8_t cclPoseConfidenceQ8(float conf)
{
    {
        conf = cclPoseClamp(conf, 0.0f, 1.0f);
    }
}

```



```

    return (uint8_t) (conf * 255.0f + 0.5f);
}

static void cclSendCore0PoseInvalid(void)
{
    uint8_t buffer[UART_LINK_FRAME_LENGTH_C0POSE];
    uartLinkFrameStruct frame;
    frame.type = UART_LINK_TYPE_CORE0_POSE;
    frame.sequence = cclState.poseTxSequence;
    frame.poseEpoch = cclState.poseEpoch;
    frame.poseXQ8 = 0u;
    frame.poseYQ8 = 0u;
    frame.poseOrientationRad = 0.0f;
    frame.poseConfidenceQ8 = 0u;
    frame.poseFlags = 0u;
    if (UART_LINK_PROTO_STATUS_OK == uartLinkProtoEncode(&frame, buffer, (uint8_t) sizeof(buffer)))
    {
        uart_write_buffer(CCL_UART_NUM, buffer, UART_LINK_FRAME_LENGTH_C0POSE);
        cclState.poseTxSequence++;
    }
}

static void cclBuildCore0PoseFrame(const visionDebugToFlightStruct *dbg,
    uint8_t sequence, uint8_t epoch, uartLinkFrameStruct *frame)
{
    uint8_t flags = 0u;
    uint8_t trackLocked;
    uint8_t sourceOk;
    uint8_t positionOk;
    uint8_t headingOk;

    if (frame == 0) return;
    memset(frame, 0, sizeof(*frame));
    frame->type = UART_LINK_TYPE_CORE0_POSE;
    frame->sequence = sequence;
    frame->poseEpoch = epoch;
    if (dbg == 0) return;

    trackLocked = (dbg->trackingState == CCL_OM_TRACK_LOCKED && dbg->contourCount > 0u) ? 1u : 0u;
    sourceOk = (uint8_t) (trackLocked && (dbg->targetLost == 0u));
    positionOk = (uint8_t) (sourceOk &&
        cclPoseFinite(dbg->filteredCenterX) &&
        cclPoseFinite(dbg->filteredCenterY) &&
        (dbg->filteredCenterX >= 0.0f) &&
        (dbg->filteredCenterX <= CCL_CORE0_POSE_MAX_X) &&
        (dbg->filteredCenterY >= 0.0f) &&
        (dbg->filteredCenterY <= CCL_CORE0_POSE_MAX_Y));
    headingOk = (uint8_t) ((dbg->orientationFresh != 0u) &&
        (dbg->orientProtectState == CCL_OM_ORIENT_NORMAL) &&
        cclPoseFinite(dbg->orientation) &&
        (dbg->orientation >= -CCL_POSE_PI) &&
        (dbg->orientation <= CCL_POSE_PI) &&
        cclPoseFinite(dbg->orientationConfidence) &&
        (dbg->orientationConfidence >= 0.0f) &&
        (dbg->orientationConfidence <= 1.0f));

    if (trackLocked) flags |= UART_LINK_POSE_FLAG_TRACK_LOCKED;
    if (dbg->targetLost) flags |= UART_LINK_POSE_FLAG_TARGET_LOST;
    if (positionOk != 0u)
    {
        frame->poseXQ8 = cclPoseQ8(dbg->filteredCenterX);
        frame->poseYQ8 = cclPoseQ8(dbg->filteredCenterY);
        flags |= UART_LINK_POSE_FLAG_POSITION_NUMERIC_VALID;
        flags |= UART_LINK_POSE_FLAG_POSITION_FRESH;
    }
    if (headingOk != 0u)
    {
        frame->poseOrientationRad = dbg->orientation;
        frame->poseConfidenceQ8 = cclPoseConfidenceQ8(
            dbg->orientationConfidence);
        flags |= UART_LINK_POSE_FLAG_HEADING_FRESH;
    }
    frame->poseFlags = flags;
}

static void cclSendCore0PoseFromDebug(const visionDebugToFlightStruct *dbg)
{
    uint8_t buffer[UART_LINK_FRAME_LENGTH_C0POSE];
    uartLinkFrameStruct frame;

```

```

    cclBuildCore0PoseFrame(dbg, cclState.poseTxSequence,
        cclState.poseEpoch, &frame);
    if (UART_LINK_PROTO_STATUS_OK == uartLinkProtoEncode(&frame, buffer, (uint8_t) sizeof(buffer)))
    {
        uart_write_buffer(CCL_UART_NUM, buffer, UART_LINK_FRAME_LENGTH_C0POSE);
        cclState.poseTxSequence++;
    }
}

static void cclSendCore0PoseIfDue(void)
{
    visionDebugToFlightStruct dbg;
    if (!cclState.poseSendFlag) return;
    cclState.poseSendFlag = 0u;

    if (coreDataTryReadVisionDebugToFlight(&dbg) == 0u) return;
    if (dbg.magic != CORE_DATA_READY_MAGIC)
    {
        cclState.posePrimed = 0u;
        cclSendCore0PoseInvalid();
        return;
    }

    if (!cclState.posePrimed)
    {
        cclState.posePrimed = 1u;
        cclState.poseLastSeq = dbg.seq;
        cclSendCore0PoseInvalid();
        return;
    }

    if (dbg.seq == cclState.poseLastSeq)
    {
        return;
    }
    cclState.poseLastSeq = dbg.seq;
    cclSendCore0PoseFromDebug(&dbg);
}

void crossCoreLinkInit(void)
{
    cclState.rxHead = 0u;
    cclState.rxTail = 0u;
    cclState.rxCount = 0u;
    cclState.paramRxHead = 0u;
    cclState.paramRxTail = 0u;
    cclState.paramRxCount = 0u;
    cclState.heartbeatSendFlag = 0u;
    cclState.kvmControlSendFlag = 0u;
    cclState.statusSendFlag = 0u;
    cclState.poseSendFlag = 0u;
    cclState.heartbeatTickCount = 0u;
    cclState.kvmControlTickCount = 0u;
    cclState.poseTickCount = 0u;
    cclState.core1QuietMs = CCL_LINK_LOST_MS; /* 上电默认断联 */
    cclState.motionVx = 0.0f;
    cclState.motionVy = 0.0f;
    cclState.motionWz = 0.0f;
    cclState.motionRxSeq = 0u;
    cclState.motionQuietMs = CCL_MOTION_LOST_MS; /* 上电默认无遥控前馈 */
    cclState.motionWirePrimed = 0u;
    cclState.motionSessionFenced = 0u;
    cclState.motionWireSeq = 0u;
    cclState.motionTakeoffReq = 0u;
    cclState.statusQuietMs = CCL_STATUS_LOST_MS;
    cclState.statusWirePrimed = 0u;
    cclState.statusWireSeq = 0u;
    cclState.producerBootSession = 0u;
    cclState.statusRxSeq = 0u;
    cclState.mode2RequestBaselineValid = 0u;
    cclState.beaconStateSeq = 0u;
    cclState.beaconStatePrimed = 0u;
    cclState.core1ReportedBeaconCount = 0u;
    cclState.core1ReportedBeaconSearching = 0u;
    cclState.core1ReportedBeaconSearchTimedOut = 0u;
    cclState.core1ReportedBeaconStateValid = 0u;
    cclState.mode2LoraLandReqArmed = 0u;
    cclState.core1ReportedCore2Lost = 1u; /* 上电默认 core2 未确认在线; 收到 core1 STATUS 后刷新 */
}

```

```

cclState.core1ReportedCore2AttitudeFault = 0u;
cclState.core1ReportedRequiredVisionFault = 1u;
cclState.core1ReportedFaultSource = 2u;
cclState.core1ReportedFaultClass = 2u;
cclState.core1ReportedMode2LoraLandReq = 0u;
cclState.controlActive = 0u;
cclState.c0SafetyState = 0u;
cclState.c0ArmProgress = 0u;
cclState.c0StatusFlags = 0u;
cclState.c0SysCtrl = 0u;
cclState.c0Buzzer = 0u;
cclState.statusTxSequence = 0u;
cclState.poseTxSequence = 0u;
cclState.heartbeatSequence = 0u;
cclState.kvmControlSequence = 0u;
cclState.kvmEventSequence = 0u;
cclResetParamTransport();
cclState.paramRequestSequence = 0u;
cclState.paramOwnerBootGeneration = 0u;
cclState.paramTxSequence = 0u;
memset(&cclState.paramGlobalState, 0, sizeof(cclState.paramGlobalState));
memset(&cclState.paramGlobalRequest, 0,
    sizeof(cclState.paramGlobalRequest));
cclState.paramGlobalStateQuietMs = CCL_PARAM_GLOBAL_STATE_LOST_MS;
cclState.paramGlobalRequestTxMs = CCL_PARAM_GLOBAL_REQUEST_TX_MS;
cclState.paramGlobalStateValid = 0u;
cclState.paramGlobalRequestAcked = 0u;
cclState.paramGlobalRequestBoot = 0u;
cclState.paramGlobalRequestSequence = 0u;
cclState.paramGlobalTxSequence = 0u;
cclParamBridgeBootRetained++;
if (cclParamBridgeBootRetained == 0u) cclParamBridgeBootRetained = 1u;
cclPoseEpochRetained++;
if (0u == cclPoseEpochRetained) cclPoseEpochRetained = 1u;
cclState.poseEpoch = cclPoseEpochRetained;
cclState.posePrimed = 0u;
cclState.poseLastSeq = 0u;
cclState.rxFrameCount = 0u;
cclState.rxErrorCount = 0u;
cclState.kvmRouteSession = 0u;
cclState.kvmRouteEpoch = 0u;
cclState.kvmEventSeq = 0u;
cclState.kvmTargetSession = 0u;
cclState.kvmRetryTickMs = 0u;
cclState.kvmStatusQuietMs = CCL_KVM_STATUS_LOST_MS;
cclState.kvmTarget = UART_LINK_KVM_TARGET_CORE0;
cclState.kvmKeyId = 0u;
cclState.kvmKeyState = 0u;
cclState.kvmEventPending = 0u;
cclState.kvmEventSendFlag = 0u;
cclState.kvmPeerRestartLatched = 0u;
memset(&cclState.kvmStatus, 0, sizeof(cclState.kvmStatus));
cclState.kvmStatus.activeTarget = UART_LINK_KVM_TARGET_NONE;
cclPublishKvmStatus();
coreDataClearParamGlobalState();
uartLinkProtoInit(&cclState.parser);

uart_init(CCL_UART_NUM, CCL_BAUDRATE, CROSS_CORE_UART_TX_PIN, CROSS_CORE_UART_RX_PIN);
uart_rx_interrupt(CCL_UART_NUM, 1u);

cclState.initReadyFlag = 1u;
}

void crossCoreLinkPit1ms(void)
{
    if (!cclState.initReadyFlag) return;

    cclState.heartbeatTickCount++;
    if (cclState.heartbeatTickCount >= CCL_HEARTBEAT_MS)
    {
        cclState.heartbeatTickCount = 0u;
        cclState.heartbeatSendFlag = 1u;
    }
    cclState.kvmControlTickCount++;
    if (cclState.kvmControlTickCount >= CCL_KVM_CONTROL_REFRESH_MS)
    {
        cclState.kvmControlTickCount = 0u;
        cclState.kvmControlSendFlag = 1u;
    }
}

```

```

cclState.poseTickCount++;
if (cclState.poseTickCount >= CCL_CORE0_POSE_TX_MS)
{
    cclState.poseTickCount = 0u;
    cclState.poseSendFlag = 1u;
}

if (cclState.core1QuietMs < CCL_LINK_LOST_MS)
{
    cclState.core1QuietMs++;
}
if (cclState.motionQuietMs < CCL_MOTION_LOST_MS)
{
    cclState.motionQuietMs++;
}
if (cclState.statusQuietMs < CCL_STATUS_LOST_MS)
{
    cclState.statusQuietMs++;
}
if (cclState.kvmEventPending != 0u)
{
    cclState.kvmRetryTickMs++;
    if (cclState.kvmRetryTickMs >= CCL_KVM_EVENT_RETRY_MS)
    {
        cclState.kvmRetryTickMs = 0u;
        cclState.kvmEventSendFlag = 1u;
    }
}
if (cclState.kvmStatusQuietMs < CCL_KVM_STATUS_LOST_MS)
{
    cclState.kvmStatusQuietMs++;
}
if (cclState.paramQuietMs < CCL_PARAM_TRANSPORT_TIMEOUT_MS)
{
    cclState.paramQuietMs++;
}
if (cclState.paramGlobalStateQuietMs < CCL_PARAM_GLOBAL_STATE_LOST_MS)
    cclState.paramGlobalStateQuietMs++;
if (cclState.paramGlobalRequestTxMs < CCL_PARAM_GLOBAL_REQUEST_TX_MS)
    cclState.paramGlobalRequestTxMs++;
}

void crossCoreLinkPoll(void)
{
    uartLinkFrameStruct frame;
    if (!cclState.initReadyFlag) return;

    /* STATUS 身份进入离线栅栏即同步撤销 MOTION，不等待独立 300ms 运动超时。
    * 后续只能由合法 STATUS 重绑并重新经过显式零 MOTION fence。 */
    if ((cclState.statusQuietMs >= CCL_STATUS_LOST_MS) &&
        (cclState.motionSessionFenced != 0u))
    {
        cclState.motionWirePrimed = 0u;
        cclState.motionSessionFenced = 0u;
        cclState.motionWireSeq = 0u;
        cclState.motionVx = 0.0f;
        cclState.motionVy = 0.0f;
        cclState.motionWz = 0.0f;
        cclState.motionTakeoffReq = 0u;
        cclState.motionQuietMs = CCL_MOTION_LOST_MS;
    }
    while (cclPopFrame(&frame))
    {
        /* MOTION 帧: core1 转发的 core2 LoRa 遥控前馈 vx/vy/wz, 存最新值 + 清超时计时。
        * 其余(心跳/STATUS)收帧已在 ISR 刷新存活计时。 */
        if (UART_LINK_TYPE_MOTION == frame.type)
        {
            uint8_t frameUsable = cclMotionFrameUsable(&frame);
            uint8_t sequenceAdvanced;
            uint8_t zeroFence = cclMotionFrameIsZeroFence(&frame);
            if (cclMotionSessionMatches(frame.producerBootSession) == 0u)
            {
                continue;
            }
            if ((zeroFence == 0u) && (cclState.motionSessionFenced == 0u)) continue;
            if (cclState.motionWirePrimed == 0u)
            {
                sequenceAdvanced = 1u;
            }
        }
    }
}

```

```

else
{
    sequenceAdvanced = cclWireSeqAdvanced(
        frame.sequence, cclState.motionWireSeq);
}
if (sequenceAdvanced == 0u) continue;
if (zeroFence != 0u)
{
    cclState.motionSessionFenced = 1u;
    cclState.motionWirePrimed = 1u;
    cclState.motionWireSeq = frame.sequence;
    cclState.motionVx = 0.0f;
    cclState.motionVy = 0.0f;
    cclState.motionWz = 0.0f;
    cclState.motionTakeoffReq = 0u;
    cclState.motionQuietMs = 0u;
    cclState.motionRxSeq++;
    if (cclState.motionRxSeq == 0u) cclState.motionRxSeq = 1u;
    continue;
}
if ((sequenceAdvanced != 0u) && (frameUsable != 0u))
{
    cclState.motionWirePrimed = 1u;
    cclState.motionWireSeq = frame.sequence;
    cclState.motionVx = frame.motionVx;
    cclState.motionVy = frame.motionVy;
    cclState.motionWz = frame.motionWz;
    cclState.motionTakeoffReq =
        (frame.motionFlags & UART_LINK_MOTION_FLAG_TAKEOFF_REQ) ? 1u : 0u;
    cclState.motionQuietMs = 0u;
    cclState.motionRxSeq++;
    if (0u == cclState.motionRxSeq) cclState.motionRxSeq = 1u; /* 回绕跳过 0: 初值/无帧态用 0, 故有效序号恒非 0 */
}
else if ((sequenceAdvanced != 0u) && (frameUsable == 0u))
{
    /* CRC 通过但业务值非法: 立即撤销前馈, 不刷新新鲜度或接收序号。 */
    cclState.motionVx = 0.0f;
    cclState.motionVy = 0.0f;
    cclState.motionWz = 0.0f;
    cclState.motionTakeoffReq = 0u;
    cclState.motionQuietMs = CCL_MOTION_LOST_MS;
    cclState.rxErrorCount++;
}
}
/* STATUS 帧有独立 wire 序列与新鲜度。心跳/KVM/MOTION 等其它合法帧只能证明物理链路,
* 不得替 STATUS 业务事实续鲜。发送端重启/旧 seq 时, 只允许显式 M2 低电平安全重建基线。 */
else if (UART_LINK_TYPE_STATUS == frame.type)
{
    if (cclStatusFrameSemanticValid(&frame) == 0u)
    {
        cclEnterStatusProtocolFault();
        continue;
    }
    if (cclBindProducerSessionFromStatus(
        frame.producerBootSession) == 0u) continue;
    cclHandleStatusFrame(&frame);
}
else if (UART_LINK_TYPE_KVM_STATUS == frame.type)
{
    cclHandleKvmStatus(&frame);
}
else if (UART_LINK_TYPE_PARAM_GLOBAL_STATE == frame.type)
{
    cclAcceptParamGlobalState(&frame.paramGlobalState);
}
}

if ((cclState.paramGlobalStateValid != 0u) &&
    (cclState.paramGlobalStateQuietMs >= CCL_PARAM_GLOBAL_STATE_LOST_MS))
{
    cclState.paramGlobalStateValid = 0u;
    coreDataClearParamGlobalState();
    cclCancelPendingParamGlobalRequest();
}
if (cclState.paramGlobalRequestTxMs >= CCL_PARAM_GLOBAL_REQUEST_TX_MS)
{
    paramSyncGlobalRequest request;
    cclState.paramGlobalRequestTxMs = 0u;
    if (coreDataTryReadParamGlobalRequest(&request) != 0u)

```

```

    {
        if ((request.producerBootGeneration != cclState.paramGlobalRequestBoot) ||
            (request.sequence != cclState.paramGlobalRequestSequence))
        {
            cclState.paramGlobalRequest = request;
            cclState.paramGlobalRequestBoot = request.producerBootGeneration;
            cclState.paramGlobalRequestSequence = request.sequence;
            cclState.paramGlobalRequestAcked = 0u;
        }
        if (cclState.paramGlobalRequestAcked == 0u)
            cclSendParamGlobalRequest(&request);
    }
}

cclSendCore0PoseIfDue();

/* 路由边沿先立即发布 CONTROL，再允许同路由 EVENT 发送。 */
if (cclState.kvmControlSendFlag != 0u)
{
    cclState.kvmControlSendFlag = 0u;
    cclSendKvmControl();
}

if (cclState.kvmEventSendFlag != 0u)
{
    cclState.kvmEventSendFlag = 0u;
    cclSendKvmEvent();
}

if (cclState.heartbeatSendFlag)
{
    cclState.heartbeatSendFlag = 0u;
    cclSendHeartbeat();
    cclSendCore0Status(); /* 与心跳同周期发 core0 状态(供 core1 RGB 灯) */
    cclState.statusSendFlag = 0u;
    cclSendKvmControl();
}
else if (cclState.statusSendFlag != 0u)
{
    cclState.statusSendFlag = 0u;
    cclSendCore0Status();
}

if (!crossCoreLinkIsCore1Alive())
{
    if (cclState.kvmStatus.flags != 0u)
    {
        cclState.kvmStatus.flags = 0u;
        cclState.kvmStatus.activeTarget = UART_LINK_KVM_TARGET_NONE;
        cclPublishKvmStatus();
    }
}
else if ((cclState.kvmStatusQuietMs >= CCL_KVM_STATUS_LOST_MS) &&
         (cclState.kvmStatus.flags != 0u))
{
    cclState.kvmStatus.flags = 0u;
    cclState.kvmStatus.activeTarget = UART_LINK_KVM_TARGET_NONE;
    cclPublishKvmStatus();
}

/* 参数桥接是最低优先级：控制队列和本拍全部安全/心跳发送完成后，
 * 每次 Poll 最多处理一帧。静默超时只撤销参数传输，不影响控制事实。 */
if ((cclState.paramQuietMs >= CCL_PARAM_TRANSPORT_TIMEOUT_MS) &&
    ((cclState.paramAssemblyActive != 0u) ||
     (cclState.paramRequestPublished != 0u) ||
     (cclState.paramResponseValid != 0u)))
{
    cclResetParamTransport();
}
if (cclPopParamFrame(&frame)) cclHandleParamRequest(&frame);
}

uint8_t crossCoreLinkGetParamGlobalState(paramSyncGlobalState *out)
{
    uint32_t primask;
    uint8_t valid;
    if (out == 0) return 0u;
    primask = __get_PRIMASK();
    __disable_irq();

```

```

    valid = (uint8_t) ((cclState.paramGlobalStateValid != 0u) &&
        (cclState.paramGlobalStateQuietMs < CCL_PARAM_GLOBAL_STATE_LOST_MS) &&
        (paramSyncGlobalStateIsValid(&cclState.paramGlobalState) != 0u));
    if (valid != 0u) *out = cclState.paramGlobalState;
    __set_PRIMASK(primask);
    return valid;
}

void crossCoreLinkRxIrqHandler(void)
{
    uint8_t data;
    if (uart_query_byte(CCL_UART_NUM, &data))
    {
        cclHandleByte(data);
    }
}

void crossCoreLinkSetKvmRoute(uint32_t routeSession, uint32_t routeEpoch,
    uint8_t target, uint32_t eventSeq, uint8_t keyId,
    uint8_t keyState, uint8_t eventPending)
{
    uint8_t routeChanged = (uint8_t) ((routeSession != cclState.kvmRouteSession) ||
        (routeEpoch != cclState.kvmRouteEpoch) || (target != cclState.kvmTarget));
    if (routeChanged != 0u)
    {
        cclState.kvmRouteSession = routeSession;
        cclState.kvmRouteEpoch = routeEpoch;
        cclState.kvmTarget = target;
        cclState.kvmTargetSession = 0u;
        cclState.kvmPeerRestartLatched = 0u;
        cclState.kvmStatusQuietMs = CCL_KVM_STATUS_LOST_MS;
        cclState.kvmEventPending = 0u;
        cclState.kvmControlSendFlag = 1u;
        /* 路由切换后的首个心跳必须携带新的 controlActive，避免旧心跳令
        * Core1 短暂推导出 NONE 并显示黄色。下一次前台 poll 立即发送。 */
        cclState.heartbeatSendFlag = 1u;
        memset(&cclState.kvmStatus, 0, sizeof(cclState.kvmStatus));
        cclState.kvmStatus.magic = CORE_DATA_READY_MAGIC;
        cclState.kvmStatus.routeSession = routeSession;
        cclState.kvmStatus.routeEpoch = routeEpoch;
        cclState.kvmStatus.target = target;
        cclState.kvmStatus.activeTarget = UART_LINK_KVM_TARGET_NONE;
        cclPublishKvmStatus();
    }
    cclState.controlActive = (target != UART_LINK_KVM_TARGET_CORE0) ? 1u : 0u;
    if ((target != UART_LINK_KVM_TARGET_CORE0) && (eventPending != 0u) &&
        (eventSeq != 0u) &&
        ((cclState.kvmEventPending == 0u) || (eventSeq != cclState.kvmEventSeq)))
    {
        cclState.kvmEventSeq = eventSeq;
        cclState.kvmKeyId = keyId;
        cclState.kvmKeyState = keyState;
        cclState.kvmEventPending = 1u;
        cclState.kvmRetryTickMs = 0u;
        cclState.kvmEventSendFlag = 1u;
    }
    else if (eventPending == 0u)
    {
        cclState.kvmEventPending = 0u;
    }
}

void crossCoreLinkSetCore0Status(uint8_t safetyState, uint8_t armProgress, uint8_t statusFlags)
{
    cclState.c0SafetyState = safetyState;
    cclState.c0ArmProgress = armProgress;
    cclState.c0StatusFlags = statusFlags;
}

/* 设置 CORE0_STATUS 的 sysCtrl(空地系统模式/RGB 意图/到顶, UART_LINK_SYS.* 打包) + buzzer(蜂鸣模式码)。
* 前台每拍调：随心跳周期发给 core1。空地系统模式下发 + 起飞 RGB/蜂鸣编排 + 到顶车禁动门控用。 */
void crossCoreLinkSetSysControl(uint8_t sysCtrl, uint8_t buzzer)
{
    if ((cclState.c0SysCtrl != sysCtrl) || (cclState.c0Buzzer != buzzer))
    {
        cclState.c0SysCtrl = sysCtrl;
        cclState.c0Buzzer = buzzer;
        cclState.statusSendFlag = 1u;
    }
}

```

```

}

uint8_t crossCoreLinkIsCore1Alive(void)
{
    return (cclState.core1QuietMs < CCL_LINK_LOST_MS) ? 1u : 0u;
}

uint8_t crossCoreLinkGetMotionSnapshot(float *vx, float *vy, float *wz, uint32_t *rxSeq)
{
    if (cclState.motionQuietMs >= CCL_MOTION_LOST_MS)
    {
        return 0u; /* 超时/无遥控前馈, 调用方喂零 */
    }
    if (vx != 0) *vx = cclState.motionVx;
    if (vy != 0) *vy = cclState.motionVy;
    if (wz != 0) *wz = cclState.motionWz;
    if (rxSeq != 0) *rxSeq = cclState.motionRxSeq;
    return 1u;
}

uint32_t crossCoreLinkFenceMotion(void)
{
    uint32_t baseline = cclState.motionRxSeq;
    /* 这些字段只由前台 parser 与本前台 API 写; UART ISR 仅入字节 FIFO, 无需扩大为全局临界区。 */
    cclState.motionVx = 0.0f;
    cclState.motionVy = 0.0f;
    cclState.motionWz = 0.0f;
    cclState.motionTakeoffReq = 0u;
    cclState.motionQuietMs = CCL_MOTION_LOST_MS;
    return baseline;
}

/* core1 经 STATUS 帧报告的 RS-485↔core2 断联位 (最近一帧的原值)。
 * 仅当 core1 在线时有意义; core1 死时此值可能陈旧, 调用方(缓降/发车拦截)须用
 * crossCoreLinkIsCore1Alive() 兜底: anyLinkLost != !IsCore1Alive() || GetCore2LinkLost()。 */
uint8_t crossCoreLinkGetCore2LinkLost(void)
{
    if (cclState.statusQuietMs >= CCL_STATUS_LOST_MS) return 1u;
    return cclState.core1ReportedCore2Lost;
}

/* Core1 经固定 14B STATUS bit3 报告的通用必需视觉故障事实; plan 缺失或停更都会
 * fail-closed 置位。C1/C3/C4 永久必需, Core1 自身断联仍由
 * !crossCoreLinkIsCore1Alive() 独立兜底。 */
uint8_t crossCoreLinkGetRequiredVisionFault(uint8_t *source, uint8_t *faultClass)
{
    if (source != 0) *source = cclState.core1ReportedFaultSource;
    if (faultClass != 0) *faultClass = cclState.core1ReportedFaultClass;
    if (cclState.statusQuietMs >= CCL_STATUS_LOST_MS)
    {
        if (source != 0) *source = 2u;
        if (faultClass != 0) *faultClass = 3u;
        return 1u;
    }
    return cclState.core1ReportedRequiredVisionFault;
}

static void cclResetProducerSession(uint32_t session)
{
    cclResetParamTransport();
    cclState.producerBootSession = session;
    cclState.motionWirePrimed = 0u;
    cclState.motionSessionFenced = 0u;
    cclState.statusWirePrimed = 0u;
    cclState.motionVx = 0.0f;
    cclState.motionVy = 0.0f;
    cclState.motionWz = 0.0f;
    cclState.motionTakeoffReq = 0u;
    cclState.motionQuietMs = CCL_MOTION_LOST_MS;
    cclState.statusQuietMs = CCL_STATUS_LOST_MS;
    cclState.core1ReportedCore2AttitudeFault = 0u;
    cclState.core1ReportedRequiredVisionFault = 1u;
    cclState.core1ReportedFaultSource = 2u;
    cclState.core1ReportedFaultClass = 2u;
}

static uint8_t cclBindProducerSessionFromStatus(uint32_t session)
{
    uint8_t decision = uartLinkRxSessionClassify(

```



```

        (uint8_t) (cclState.statusQuietMs >= CCL_STATUS_LOST_MS),
        cclState.producerBootSession, session);
    if (decision == UART_LINK_RX_SESSION_REJECT) return 0u;
    if (decision == UART_LINK_RX_SESSION_REBIND)
        cclResetProducerSession(session);
    return 1u;
}

static uint8_t cclMotionSessionMatches(uint32_t session)
{
    return (uint8_t) ((cclState.statusQuietMs < CCL_STATUS_LOST_MS) &&
        (session != 0u) && (session == cclState.producerBootSession));
}

void crossCoreLinkGetFlightFactsSnapshot(crossCoreFlightFactsSnapshotStruct *out)
{
    uint32_t primask;
    if (out == 0) return;
    primask = __get_PRIMASK();
    __disable_irq();
    out->motionSeq = cclState.motionRxSeq;
    out->statusSeq = cclState.statusRxSeq;
    out->beaconStateSeq = cclState.beaconStateSeq;
    out->core1Seq = cclState.rxFrameCount;
    out->motionAgeMs = cclState.motionQuietMs;
    out->statusAgeMs = cclState.statusQuietMs;
    out->core1AgeMs = cclState.core1QuietMs;
    out->beaconCount = cclState.core1ReportedBeaconCount;
    out->mode3RequestLevel = cclState.motionTakeoffReq;
    out->mode3RequestValid = (cclState.motionQuietMs < CCL_MOTION_LOST_MS) ? 1u : 0u;
    out->statusValid = (uint8_t) (crossCoreLinksCore1Alive() &&
        cclState.statusQuietMs < CCL_STATUS_LOST_MS);
    out->core2AttitudeFault = out->statusValid ? cclState.core1ReportedCore2AttitudeFault : 0u;
    out->mode2RequestLevel = cclState.core1ReportedMode2LoraLandReq;
    out->mode2RequestValid = (uint8_t) (out->statusValid &&
        cclState.mode2RequestBaselineValid);
    out->beaconSearching = cclState.core1ReportedBeaconSearching;
    out->beaconSearchTimedOut = cclState.core1ReportedBeaconSearchTimedOut;
    out->beaconStateValid = (uint8_t) (out->statusValid &&
        cclState.core1ReportedBeaconStateValid);
    __set_PRIMASK(primask);
}

```

core0_cyt4bb7/project/code/comm/crossCoreLink.h

SHA-256

:

f8d2d0e20bb174aa186a3e384916cde20c56f81bd39ee979a030708ef388f864

代码 (c)

```

#ifndef _CROSS_CORE_LINK_H_
#define _CROSS_CORE_LINK_H_

#include <stdint.h>
#include "paramSyncGlobal.h"

typedef struct
{
    uint32_t motionSeq;
    uint32_t statusSeq;
    uint32_t beaconStateSeq;
    uint32_t core1Seq;
    uint16_t motionAgeMs;
    uint16_t statusAgeMs;
    uint16_t core1AgeMs;
    uint16_t beaconCount;
    uint8_t mode3RequestLevel;
    uint8_t mode3RequestValid;
    uint8_t mode2RequestLevel;
    uint8_t mode2RequestValid;
    uint8_t statusValid;
    uint8_t core2AttitudeFault;
    uint8_t beaconSearching;
    uint8_t beaconSearchTimedOut;
    uint8_t beaconStateValid;
}

```

```

} crossCoreFlightFactsSnapshotStruct;

/*
 * crossCoreLink (core0 cm_7_0 端) - 核间单线双向链路 控制端
 *
 * 物理: UART_2 / P10 (group0) ↔ core1 P04 / UART_1, 4 Mbps, 全双工单线双向。
 * - TX: 周期发 HEARTBEAT/状态/姿态, 并通过事务 KVM 帧路由按键。
 * - RX: 收 core1 心跳/状态 → 维护 core1 存活计时(断联检测)。
 *
 * 全部收发在前台 whileProcess(crossCoreLinkPoll), 绝不进 400Hz 飞控 PIT ISR。
 * crossCoreLinkPit1ms 仅做心跳分频 + 存活超时计数(轻量, 可入 1ms ISR)。
 * RX 在 UART_2 中断仅取一字节喂解析器(轻量)。
 */

void crossCoreLinkInit(void); /* 配 UART_2 + 开 RX 中断 + 复位状态 */
void crossCoreLinkPit1ms(void); /* 1ms tick: 心跳分频 + core1 存活超时计数 */
void crossCoreLinkPoll(void); /* 前台: 排空 RX → 维护存活; 待发按键事件 + 心跳 */
void crossCoreLinkRxIrqHandler(void); /* UART_2 RX 中断: 仅取一字节喂解析器 */

/* cm_7_1 菜单前台快照路由意图; cm_7_0 负责周期 CONTROL 与单在途 EVENT 重试。 */
void crossCoreLinkSetKvmRoute(uint32_t routeSession, uint32_t routeEpoch,
    uint8_t target, uint32_t eventSeq, uint8_t keyId,
    uint8_t keyState, uint8_t eventPending);

/* 设置 core0 飞控状态(供 core1 RGB 状态灯): safetyState/armProgress/statusFlags(UART_LINK_C0_FLAG_)。
 * 与心跳同周期经 CORE0_STATUS 帧发给 core1。前台调用。 */
void crossCoreLinkSetCore0Status(uint8_t safetyState, uint8_t armProgress, uint8_t statusFlags);

/* 设置 CORE0_STATUS 的 sysCtrl(空地系统模式/RGB 意图/到顶, UART_LINK_SYS_ 打包) + buzzer(蜂鸣模式码,
 * UART_LINK_BUZZER_)。空地系统模式下发 + 起飞 RGB/蜂鸣编排 + 到顶车禁动门控用。前台每拍调。 */
void crossCoreLinkSetSysControl(uint8_t sysCtrl, uint8_t buzzer);

uint8_t crossCoreLinkIsCore1Alive(void); /* 1=core1 链路在线(心跳未超时) */

/* 取 core1 转发来的运动前馈 vx/vy/wz(MOTION 帧, 来自 core2 LoRa 遥控值)。
 * 返回 1=有效(近期收到且未超时), 0=超时/无(调用方应喂零, 防遥控失联失控)。 */
/* 取 MOTION 最新值与本地接收序号。
 * rxSeq 仅在 core0 本地每收一帧 MOTION 后自增, 不改变线协议; 调用方用它区分“新帧”与“旧值仍未超时”。 */
uint8_t crossCoreLinkGetMotionSnapshot(float *vx, float *vy, float *wz, uint32_t *rxSeq);

/* 热重连事务: 前台 owner 清陈旧运动值/请求并返回当前接收序列基线; 不重置 UART、parser 或 session。 */
uint32_t crossCoreLinkFenceMotion(void);

/* 取模式 3 车 LoRa S3+S4 请求电平(MOTION 帧 flags bit0; 超时返回 0)。
 * core0Loop 以低电平重装、每个高电平只消费一次, 并按 TKO 阶段解释为起飞或缓降。 */
/* 取 core1 经 STATUS 帧上报的 RS-485↔core2 断联位(最近一帧原值; 200ms 周期刷新)。
 * 仅 core1 在线时有意义; Coordinator 仍须以!crossCoreLinkIsCore1Alive()兜底 core1 自身断联。 */
uint8_t crossCoreLinkGetCore2LinkLost(void);

/* 取 Core1 经 STATUS bit3 上报的通用必需视觉故障事实 (
 * plan 缺失/停更均 fail-closed)。C1/C3/C4 永久必需, 不再受相机数量模式门控。 */
uint8_t crossCoreLinkGetRequiredVisionFault(uint8_t *source, uint8_t *faultClass);

/* 起降 Coordinator 一次性读取的链路业务事实; 各来源保留独立 seq/age/valid。 */
void crossCoreLinkGetFlightFactsSnapshot(crossCoreFlightFactsSnapshotStruct *out);

/* Core1 全核参数协调状态; 独立于控制 STATUS/linkFlags, 新鲜度失效时返回 0。 */
uint8_t crossCoreLinkGetParamGlobalState(paramSyncGlobalState *out);

#endif /* _CROSS_CORE_LINK_H_ */

```

core0_cyt4bb7/project/code/comm/paramSyncMailbox.c

SHA-256

:

87df05e7d6c118bce04dffe55c188f9fb5d6f94149cabe052009b7e883236345

代码 (c)

```

#include "paramSyncMailbox.h"

#include "cy_device_headers.h"

#include <string.h>

```

```

#define PARAM_SYNC_REQUEST_MAGIC (0x50535251u) /* PSRQ */
#define PARAM_SYNC_RESPONSE_MAGIC (0x50535253u) /* PSRS */
#define PARAM_SYNC_MAILBOX_ABI (1u)

typedef struct
{
    uint32_t beginSequence;
    uint32_t magic;
    uint32_t abiVersion;
    uint32_t payloadBytes;
    uint32_t producerBootGeneration;
    uint32_t sequence;
    uint32_t transactionId;
    uint32_t offset;
    uint32_t payloadCrc32;
    uint16_t dataLength;
    uint8_t op;
    uint8_t slot;
    uint8_t target;
    uint8_t state;
    uint8_t reservedHeader[6];
    uint8_t data[PARAM_SYNC_MAILBOX_DATA_MAX];
    uint8_t reservedTail[12];
    uint32_t endSequence;
} paramSyncMailboxSharedFrame;

typedef char paramSyncMailbox_frame_size_assert[
    (sizeof(paramSyncMailboxSharedFrame) == PARAM_SYNC_MAILBOX_FRAME_BYTES) ? 1 : -1];

#pragma location = PARAM_SYNC_MAILBOX_REQUEST_ADDR
__no_init static volatile paramSyncMailboxSharedFrame paramSyncRequestRegion;
#pragma location = PARAM_SYNC_MAILBOX_RESPONSE_ADDR
__no_init static volatile paramSyncMailboxSharedFrame paramSyncResponseRegion;

static uint8_t frameInputValid(uint32_t bootGeneration, uint32_t sequence,
    uint16_t dataLength)
{
    return (uint8_t) ((bootGeneration != 0u) && (sequence != 0u) &&
        (dataLength <= PARAM_SYNC_MAILBOX_DATA_MAX));
}

static void cleanRegion(volatile paramSyncMailboxSharedFrame *region)
{
    SCB_CleanDCache_by_Addr((volatile void *) region, sizeof(*region));
    __DSB();
}

static void invalidateRegion(volatile paramSyncMailboxSharedFrame *region)
{
    SCB_InvalidateDCache_by_Addr((volatile void *) region, sizeof(*region));
    __DSB();
}

static uint8_t publishFrame(volatile paramSyncMailboxSharedFrame *region,
    const paramSyncMailboxSharedFrame *frame)
{
    paramSyncMailboxSharedFrame staged;

    staged = *frame;
    staged.endSequence = 0u;

    region->beginSequence = 0u;
    region->endSequence = 0u;
    cleanRegion(region);

    memcpy((void *) region, &staged, sizeof(staged));
    cleanRegion(region);

    region->endSequence = staged.sequence;
    SCB_CleanDCache_by_Addr((volatile void *)
        (((uintptr_t) &region->endSequence) & ~((uintptr_t) 31u)), 32u);
    __DSB();
    return 1u;
}

static uint8_t sharedFrameValid(const paramSyncMailboxSharedFrame *frame,
    uint32_t magic)
{
    return (uint8_t) ((frame->beginSequence != 0u) &&

```

```

        (frame->beginSequence == frame->sequence) &&
        (frame->endSequence == frame->sequence) &&
        (frame->magic == magic) &&
        (frame->abiVersion == PARAM_SYNC_MAILBOX_ABI) &&
        (frame->payloadBytes == frame->dataLength) &&
        frameInputValid(frame->producerBootGeneration, frame->sequence,
            frame->dataLength));
    }

static uint8_t tryReadFrame(volatile paramSyncMailboxSharedFrame *region,
    uint32_t magic, paramSyncMailboxSharedFrame *frame)
{
    paramSyncMailboxSharedFrame first;
    paramSyncMailboxSharedFrame second;
    uint8_t attempt;

    for (attempt = 0u; attempt < 3u; attempt++)
    {
        invalidateRegion(region);
        memcpy(&first, (const void *) region, sizeof(first));
        invalidateRegion(region);
        memcpy(&second, (const void *) region, sizeof(second));
        if (sharedFrameValid(&first, magic) &&
            sharedFrameValid(&second, magic) &&
            (memcmp(&first, &second, sizeof(first)) == 0))
        {
            *frame = second;
            return 1u;
        }
    }
    return 0u;
}

uint8_t paramSyncMailboxPublishRequest(const paramSyncMailboxRequest *request)
{
    paramSyncMailboxSharedFrame frame;
    if ((request == 0) || !frameInputValid(request->producerBootGeneration,
        request->sequence, request->dataLength)) return 0u;
    memset(&frame, 0, sizeof(frame));
    frame.beginSequence = request->sequence;
    frame.magic = PARAM_SYNC_REQUEST_MAGIC;
    frame.abiVersion = PARAM_SYNC_MAILBOX_ABI;
    frame.payloadBytes = request->dataLength;
    frame.producerBootGeneration = request->producerBootGeneration;
    frame.sequence = request->sequence;
    frame.transactionId = request->transactionId;
    frame.offset = request->offset;
    frame.payloadCrc32 = request->payloadCrc32;
    frame.dataLength = request->dataLength;
    frame.op = request->op;
    frame.slot = request->slot;
    frame.target = request->target;
    frame.state = request->flags;
    memcpy(frame.data, request->data, request->dataLength);
    return publishFrame(&paramSyncRequestRegion, &frame);
}

uint8_t paramSyncMailboxTryReadRequest(paramSyncMailboxRequest *request)
{
    paramSyncMailboxSharedFrame frame;
    paramSyncMailboxRequest candidate;
    if ((request == 0) || !tryReadFrame(&paramSyncRequestRegion,
        PARAM_SYNC_REQUEST_MAGIC, &frame)) return 0u;
    memset(&candidate, 0, sizeof(candidate));
    candidate.producerBootGeneration = frame.producerBootGeneration;
    candidate.sequence = frame.sequence;
    candidate.transactionId = frame.transactionId;
    candidate.offset = frame.offset;
    candidate.payloadCrc32 = frame.payloadCrc32;
    candidate.dataLength = frame.dataLength;
    candidate.op = frame.op;
    candidate.slot = frame.slot;
    candidate.target = frame.target;
    candidate.flags = frame.state;
    memcpy(candidate.data, frame.data, frame.dataLength);
    *request = candidate;
    return 1u;
}

```

```
uint8_t paramSyncMailboxPublishResponse(const paramSyncMailboxResponse *response)
{
    paramSyncMailboxSharedFrame frame;
    if ((response == 0) || !frameInputValid(response->producerBootGeneration,
        response->sequence, response->dataLength)) return 0u;
    memset(&frame, 0, sizeof(frame));
    frame.beginSequence = response->sequence;
    frame.magic = PARAM_SYNC_RESPONSE_MAGIC;
    frame.abiVersion = PARAM_SYNC_MAILBOX_ABI;
    frame.payloadBytes = response->dataLength;
    frame.producerBootGeneration = response->producerBootGeneration;
    frame.sequence = response->sequence;
    frame.transactionId = response->transactionId;
    frame.offset = response->offset;
    frame.payloadCrc32 = response->payloadCrc32;
    frame.dataLength = response->dataLength;
    frame.op = response->op;
    frame.slot = response->slot;
    frame.target = response->target;
    frame.state = response->status;
    memcpy(frame.data, response->data, response->dataLength);
    return publishFrame(&paramSyncResponseRegion, &frame);
}
```

```
uint8_t paramSyncMailboxTryReadResponse(paramSyncMailboxResponse *response)
{
    paramSyncMailboxSharedFrame frame;
    paramSyncMailboxResponse candidate;
    if ((response == 0) || !tryReadFrame(&paramSyncResponseRegion,
        PARAM_SYNC_RESPONSE_MAGIC, &frame)) return 0u;
    memset(&candidate, 0, sizeof(candidate));
    candidate.producerBootGeneration = frame.producerBootGeneration;
    candidate.sequence = frame.sequence;
    candidate.transactionId = frame.transactionId;
    candidate.offset = frame.offset;
    candidate.payloadCrc32 = frame.payloadCrc32;
    candidate.dataLength = frame.dataLength;
    candidate.op = frame.op;
    candidate.slot = frame.slot;
    candidate.target = frame.target;
    candidate.status = frame.state;
    memcpy(candidate.data, frame.data, frame.dataLength);
    *response = candidate;
    return 1u;
}
```

core0_cyt4bb7/project/code/comm/uartLinkProto.c

SHA-256

:

48b3216267bf537a48b2af3bafdcdb79827286b7d84d04349d307831fd19fd11e

代码 (c)

```
#include "uartLinkProto.h"
```

```
/* float ↔ 4 字节小端互转 (core0/core1 同为小端 ARM, 按字节显式拼装传输) 。 */
typedef union { float f; uint8_t b[4]; } uartLinkFloatBytes;
```

```
static uint8_t crc8AtmUpdate(uint8_t crc, uint8_t data)
{
    uint8_t bit;
    crc ^= data;
    for (bit = 0u; bit < 8u; bit++)
    {
        crc = (crc & 0x80u) ? (uint8_t)((crc << 1u) ^ 0x07u) : (uint8_t)(crc << 1u);
    }
    return crc;
}
```

```
static uint8_t xorU32Le(uint8_t value, uint32_t data)
{
    return (uint8_t)(value ^ (uint8_t)data ^ (uint8_t)(data >> 8)
        ^ (uint8_t)(data >> 16) ^ (uint8_t)(data >> 24));
}
```

```

static uint8_t xorU16Le(uint8_t value, uint16_t data)
{
    return (uint8_t)(value ^ (uint8_t)data ^ (uint8_t)(data >> 8));
}

static void putU32Le(uint8_t *buffer, uint8_t offset, uint32_t value)
{
    buffer[offset] = (uint8_t)value;
    buffer[offset + 1u] = (uint8_t)(value >> 8);
    buffer[offset + 2u] = (uint8_t)(value >> 16);
    buffer[offset + 3u] = (uint8_t)(value >> 24);
}

static uint32_t getU32Le(const uint8_t *buffer, uint8_t offset)
{
    return (uint32_t)buffer[offset]
        | ((uint32_t)buffer[offset + 1u] << 8)
        | ((uint32_t)buffer[offset + 2u] << 16)
        | ((uint32_t)buffer[offset + 3u] << 24);
}

/* 按 type 返回帧总长（含 HEAD/TAIL）；未知 type 返回 0。*/
static uint8_t uartLinkProtoFrameLen(uint8_t type)
{
    switch (type)
    {
        case UART_LINK_TYPE_PING:
        case UART_LINK_TYPE_ACK:
        case UART_LINK_TYPE_HEARTBEAT:
            return UART_LINK_FRAME_LENGTH;
        case UART_LINK_TYPE_STATUS:
            return UART_LINK_FRAME_LENGTH_STATUS;
        case UART_LINK_TYPE_CORE0_STATUS:
            return UART_LINK_FRAME_LENGTH_C0STATUS;
        case UART_LINK_TYPE_CORE0_POSE:
            return UART_LINK_FRAME_LENGTH_C0POSE;
        case UART_LINK_TYPE_MOTION:
            return UART_LINK_FRAME_LENGTH_MOTION;
        case UART_LINK_TYPE_KVM_CONTROL:
            return UART_LINK_FRAME_LENGTH_KVM_CONTROL;
        case UART_LINK_TYPE_KVM_EVENT:
            return UART_LINK_FRAME_LENGTH_KVM_EVENT;
        case UART_LINK_TYPE_KVM_STATUS:
        case UART_LINK_TYPE_PARAM_REQUEST:
        case UART_LINK_TYPE_PARAM_RESPONSE:
            return UART_LINK_FRAME_LENGTH_KVM_STATUS;
        case UART_LINK_TYPE_PARAM_GLOBAL_REQUEST:
            return UART_LINK_FRAME_LENGTH_PARAM_GLOBAL_REQUEST;
        case UART_LINK_TYPE_PARAM_GLOBAL_STATE:
            return UART_LINK_FRAME_LENGTH_PARAM_GLOBAL_STATE;
        default:
            return 0u;
    }
}

void uartLinkProtoInit(uartLinkParserStruct *parser)
{
    uint8_t i;

    if (0 == parser)
    {
        return;
    }

    parser->index = 0u;
    parser->expectLen = 0u;
    for (i = 0u; i < UART_LINK_FRAME_LENGTH_MAX; ++i)
    {
        parser->buffer[i] = 0u;
    }
}

uint8_t uartLinkProtoChecksum(const uartLinkFrameStruct *frame)
{
    if (0 == frame)
    {
        return 0u;
    }
}

```

```

if (UART_LINK_TYPE_STATUS == frame->type)
{
    uint8_t beaconLo = (uint8_t)(frame->beaconCount & 0xFFu);
    uint8_t beaconHi = (uint8_t)((frame->beaconCount >> 8) & 0xFFu);
    return (uint8_t)(frame->type ^ frame->sequence ^ beaconLo ^ beaconHi ^
        frame->linkFlags ^ frame->faultSource ^ frame->faultClass ^
        (uint8_t)frame->producerBootSession ^ (uint8_t)(frame->producerBootSession >> 8) ^
        (uint8_t)(frame->producerBootSession >> 16) ^ (uint8_t)(frame->producerBootSession >> 24));
}

if (UART_LINK_TYPE_CORE0_STATUS == frame->type)
{
    uint8_t crc = 0u;
    crc = crc8AtmUpdate(crc, frame->type);
    crc = crc8AtmUpdate(crc, frame->sequence);
    crc = crc8AtmUpdate(crc, frame->safetyState);
    crc = crc8AtmUpdate(crc, frame->armProgress);
    crc = crc8AtmUpdate(crc, frame->statusFlags);
    crc = crc8AtmUpdate(crc, frame->sysCtrl);
    return crc8AtmUpdate(crc, frame->buzzer);
}

if (UART_LINK_TYPE_MOTION == frame->type)
{
    uartLinkFloatBytes vx, vy, wz;
    uint8_t i;
    uint8_t x = 0u;
    vx.f = frame->motionVx;
    vy.f = frame->motionVy;
    wz.f = frame->motionWz;
    x = crc8AtmUpdate(x, frame->type);
    x = crc8AtmUpdate(x, frame->sequence);
    for (i = 0u; i < 4u; i++) x = crc8AtmUpdate(x, vx.b[i]);
    for (i = 0u; i < 4u; i++) x = crc8AtmUpdate(x, vy.b[i]);
    for (i = 0u; i < 4u; i++) x = crc8AtmUpdate(x, wz.b[i]);
    x = crc8AtmUpdate(x, frame->motionFlags);
    x = crc8AtmUpdate(x, (uint8_t)frame->producerBootSession);
    x = crc8AtmUpdate(x, (uint8_t)(frame->producerBootSession >> 8));
    x = crc8AtmUpdate(x, (uint8_t)(frame->producerBootSession >> 16));
    return crc8AtmUpdate(x, (uint8_t)(frame->producerBootSession >> 24));
}

if (UART_LINK_TYPE_CORE0_POSE == frame->type)
{
    uartLinkFloatBytes yaw;
    uint8_t i;
    uint8_t x = (uint8_t)(frame->type ^ frame->sequence ^ frame->poseEpoch
        ^ (uint8_t)(frame->poseXQ8 & 0xFFu) ^ (uint8_t)((frame->poseXQ8 >> 8) & 0xFFu)
        ^ (uint8_t)(frame->poseYQ8 & 0xFFu) ^ (uint8_t)((frame->poseYQ8 >> 8) & 0xFFu));
    yaw.f = frame->poseOrientationRad;
    for (i = 0u; i < 4u; i++) x = (uint8_t)(x ^ yaw.b[i]);
    x = (uint8_t)(x ^ frame->poseConfidenceQ8 ^ frame->poseFlags);
    return x;
}

if (UART_LINK_TYPE_KVM_CONTROL == frame->type)
{
    uint8_t x = (uint8_t)(frame->type ^ frame->sequence);
    x = xorU32Le(x, frame->kvmTargetSession);
    x = xorU32Le(x, frame->kvmRouteSession);
    x = xorU32Le(x, frame->kvmRouteEpoch);
    x = (uint8_t)(x ^ frame->kvmTarget);
    return xorU16Le(x, frame->kvmLeaseMs);
}

if (UART_LINK_TYPE_KVM_EVENT == frame->type)
{
    uint8_t x = (uint8_t)(frame->type ^ frame->sequence);
    x = xorU32Le(x, frame->kvmTargetSession);
    x = xorU32Le(x, frame->kvmRouteSession);
    x = xorU32Le(x, frame->kvmRouteEpoch);
    x = xorU32Le(x, frame->kvmEventSeq);
    return (uint8_t)(x ^ frame->kvmTarget ^ frame->kvmKeyId ^ frame->kvmKeyState);
}

if (UART_LINK_TYPE_KVM_STATUS == frame->type)
{
    uint8_t x = (uint8_t)(frame->type ^ frame->sequence);
    x = xorU32Le(x, frame->kvmTargetSession);
}

```

```

x = xorU32Le(x, frame->kvmRouteSession);
x = xorU32Le(x, frame->kvmRouteEpoch);
x = xorU32Le(x, frame->kvmEventSeq);
x = (uint8_t)(x ^ frame->kvmTarget ^ frame->kvmActiveTarget
    ^ frame->kvmMenuStates ^ frame->kvmFlags ^ frame->kvmAckResult);
return xorU16Le(x, frame->kvmCore3AgeMs);
}

if ((UART_LINK_TYPE_PARAM_REQUEST == frame->type) ||
    (UART_LINK_TYPE_PARAM_RESPONSE == frame->type))
{
    uint8_t crc = 0u;
    uint8_t i;
    crc = crc8AtmUpdate(crc, frame->type);
    crc = crc8AtmUpdate(crc, frame->sequence);
    crc = crc8AtmUpdate(crc, frame->paramOp);
    crc = crc8AtmUpdate(crc, frame->paramReserved);
    crc = crc8AtmUpdate(crc, frame->paramTarget);
    crc = crc8AtmUpdate(crc, frame->paramStatus);
    for (i = 0u; i < 4u; i++) crc = crc8AtmUpdate(crc,
        (uint8_t)(frame->paramRequestId >> (8u * i)));
    crc = crc8AtmUpdate(crc, (uint8_t)frame->paramOffset);
    crc = crc8AtmUpdate(crc, (uint8_t)(frame->paramOffset >> 8));
    crc = crc8AtmUpdate(crc, frame->paramLength);
    for (i = 0u; i < UART_LINK_PARAM_DATA_BYTES; i++)
        crc = crc8AtmUpdate(crc,
            (i < frame->paramLength) ? frame->paramData[i] : 0u);
    return crc;
}

if (UART_LINK_TYPE_PARAM_GLOBAL_REQUEST == frame->type)
{
    uint8_t crc = 0u;
    uint8_t i;
    crc = crc8AtmUpdate(crc, frame->type);
    crc = crc8AtmUpdate(crc, frame->sequence);
    for (i = 0u; i < 4u; i++) crc = crc8AtmUpdate(crc,
        (uint8_t)(frame->paramGlobalRequest.producerBootGeneration >> (8u * i)));
    for (i = 0u; i < 4u; i++) crc = crc8AtmUpdate(crc,
        (uint8_t)(frame->paramGlobalRequest.sequence >> (8u * i)));
    crc = crc8AtmUpdate(crc, frame->paramGlobalRequest.origin);
    crc = crc8AtmUpdate(crc, frame->paramGlobalRequest.action);
    crc = crc8AtmUpdate(crc, frame->paramGlobalRequest.slot);
    return crc8AtmUpdate(crc, frame->paramGlobalRequest.reserved);
}

if (UART_LINK_TYPE_PARAM_GLOBAL_STATE == frame->type)
{
    const uint32_t words[5] = {
        frame->paramGlobalState.coordinatorBootGeneration,
        frame->paramGlobalState.sequence,
        frame->paramGlobalState.requestBootGeneration,
        frame->paramGlobalState.requestSequence,
        frame->paramGlobalState.transactionId
    };
    const uint8_t bytes[8] = {
        frame->paramGlobalState.origin, frame->paramGlobalState.action,
        frame->paramGlobalState.slot, frame->paramGlobalState.phase,
        frame->paramGlobalState.target, frame->paramGlobalState.error,
        frame->paramGlobalState.readyMask, frame->paramGlobalState.flags
    };
    uint8_t crc = 0u;
    uint8_t i, j;
    crc = crc8AtmUpdate(crc, frame->type);
    crc = crc8AtmUpdate(crc, frame->sequence);
    for (i = 0u; i < 5u; i++)
        for (j = 0u; j < 4u; j++) crc = crc8AtmUpdate(crc,
            (uint8_t)(words[i] >> (8u * j)));
    for (i = 0u; i < 8u; i++) crc = crc8AtmUpdate(crc, bytes[i]);
    return crc;
}

/* 基础帧: type ^ seq ^ value */
return (uint8_t)(frame->type ^ frame->sequence ^ frame->value);
}

uartLinkProtoStatusEnum uartLinkProtoEncode(const uartLinkFrameStruct *frame, uint8_t *buffer, uint8_t bufferSize)
{
    uint8_t frameLen;

```



```

if (0 == frame || 0 == buffer)
{
    return UART_LINK_PROTO_STATUS_INVALID_PARAM;
}

frameLen = uartLinkProtoFrameLen(frame->type);
if (0u == frameLen)
{
    return UART_LINK_PROTO_STATUS_TYPE_ERROR;
}
if (bufferSize < frameLen)
{
    return UART_LINK_PROTO_STATUS_INVALID_PARAM;
}

buffer[0] = UART_LINK_FRAME_HEAD;
buffer[1] = frame->type;
buffer[2] = frame->sequence;

if (UART_LINK_TYPE_STATUS == frame->type)
{
    buffer[3] = (uint8_t)(frame->beaconCount & 0xFFu);
    buffer[4] = (uint8_t)((frame->beaconCount >> 8) & 0xFFu);
    buffer[5] = frame->linkFlags;
    buffer[6] = frame->faultSource;
    buffer[7] = frame->faultClass;
    putU32Le(buffer, 8u, frame->producerBootSession);
    buffer[12] = uartLinkProtoChecksum(frame);
    buffer[13] = UART_LINK_FRAME_TAIL;
}
else if (UART_LINK_TYPE_CORE0_STATUS == frame->type)
{
    buffer[3] = frame->safetyState;
    buffer[4] = frame->armProgress;
    buffer[5] = frame->statusFlags;
    buffer[6] = frame->sysCtrl;
    buffer[7] = frame->buzzer;
    buffer[8] = uartLinkProtoChecksum(frame);
    buffer[9] = UART_LINK_FRAME_TAIL;
}
else if (UART_LINK_TYPE_MOTION == frame->type)
{
    uartLinkFloatBytes vx, vy, wz;
    uint8_t i;
    vx.f = frame->motionVx;
    vy.f = frame->motionVy;
    wz.f = frame->motionWz;
    for (i = 0u; i < 4u; i++) buffer[3u + i] = vx.b[i];
    for (i = 0u; i < 4u; i++) buffer[7u + i] = vy.b[i];
    for (i = 0u; i < 4u; i++) buffer[11u + i] = wz.b[i];
    buffer[15] = frame->motionFlags;
    putU32Le(buffer, 16u, frame->producerBootSession);
    buffer[20] = uartLinkProtoChecksum(frame);
    buffer[21] = UART_LINK_FRAME_TAIL;
}
else if (UART_LINK_TYPE_CORE0_POSE == frame->type)
{
    uartLinkFloatBytes yaw;
    uint8_t i;
    yaw.f = frame->poseOrientationRad;
    buffer[3] = frame->poseEpoch;
    buffer[4] = (uint8_t)(frame->poseXQ8 & 0xFFu);
    buffer[5] = (uint8_t)((frame->poseXQ8 >> 8) & 0xFFu);
    buffer[6] = (uint8_t)(frame->poseYQ8 & 0xFFu);
    buffer[7] = (uint8_t)((frame->poseYQ8 >> 8) & 0xFFu);
    for (i = 0u; i < 4u; i++) buffer[8u + i] = yaw.b[i];
    buffer[12] = frame->poseConfidenceQ8;
    buffer[13] = frame->poseFlags;
    buffer[14] = uartLinkProtoChecksum(frame);
    buffer[15] = UART_LINK_FRAME_TAIL;
}
else if (UART_LINK_TYPE_KVM_CONTROL == frame->type)
{
    putU32Le(buffer, 3u, frame->kvmTargetSession);
    putU32Le(buffer, 7u, frame->kvmRouteSession);
    putU32Le(buffer, 11u, frame->kvmRouteEpoch);
    buffer[15] = frame->kvmTarget;
    buffer[16] = (uint8_t)frame->kvmLeaseMs;
}

```

```

    buffer[17] = (uint8_t)(frame->kvmLeaseMs >> 8);
    buffer[18] = uartLinkProtoChecksum(frame);
    buffer[19] = UART_LINK_FRAME_TAIL;
}
else if (UART_LINK_TYPE_KVM_EVENT == frame->type)
{
    putU32Le(buffer, 3u, frame->kvmTargetSession);
    putU32Le(buffer, 7u, frame->kvmRouteSession);
    putU32Le(buffer, 11u, frame->kvmRouteEpoch);
    putU32Le(buffer, 15u, frame->kvmEventSeq);
    buffer[19] = frame->kvmTarget;
    buffer[20] = frame->kvmKeyId;
    buffer[21] = frame->kvmKeyState;
    buffer[22] = uartLinkProtoChecksum(frame);
    buffer[23] = UART_LINK_FRAME_TAIL;
}
else if (UART_LINK_TYPE_KVM_STATUS == frame->type)
{
    putU32Le(buffer, 3u, frame->kvmTargetSession);
    putU32Le(buffer, 7u, frame->kvmRouteSession);
    putU32Le(buffer, 11u, frame->kvmRouteEpoch);
    putU32Le(buffer, 15u, frame->kvmEventSeq);
    buffer[19] = frame->kvmTarget;
    buffer[20] = frame->kvmActiveTarget;
    buffer[21] = frame->kvmMenuStates;
    buffer[22] = frame->kvmFlags;
    buffer[23] = frame->kvmAckResult;
    buffer[24] = (uint8_t)frame->kvmCore3AgeMs;
    buffer[25] = (uint8_t)(frame->kvmCore3AgeMs >> 8);
    buffer[26] = uartLinkProtoChecksum(frame);
    buffer[27] = UART_LINK_FRAME_TAIL;
}
else if ((UART_LINK_TYPE_PARAM_REQUEST == frame->type) ||
        (UART_LINK_TYPE_PARAM_RESPONSE == frame->type))
{
    uint8_t i;
    if (frame->paramLength > UART_LINK_PARAM_DATA_BYTES)
        return UART_LINK_PROTO_STATUS_INVALID_PARAM;
    buffer[3] = frame->paramOp;
    buffer[4] = frame->paramReserved;
    buffer[5] = frame->paramTarget;
    buffer[6] = frame->paramStatus;
    putU32Le(buffer, 7u, frame->paramRequestId);
    buffer[11] = (uint8_t)frame->paramOffset;
    buffer[12] = (uint8_t)(frame->paramOffset >> 8);
    buffer[13] = frame->paramLength;
    for (i = 0u; i < UART_LINK_PARAM_DATA_BYTES; i++)
        buffer[14u + i] = (i < frame->paramLength) ? frame->paramData[i] : 0u;
    buffer[26] = uartLinkProtoChecksum(frame);
    buffer[27] = UART_LINK_FRAME_TAIL;
}
else if (UART_LINK_TYPE_PARAM_GLOBAL_REQUEST == frame->type)
{
    putU32Le(buffer, 3u, frame->paramGlobalRequest.producerBootGeneration);
    putU32Le(buffer, 7u, frame->paramGlobalRequest.sequence);
    buffer[11] = frame->paramGlobalRequest.origin;
    buffer[12] = frame->paramGlobalRequest.action;
    buffer[13] = frame->paramGlobalRequest.slot;
    buffer[14] = frame->paramGlobalRequest.reserved;
    buffer[15] = uartLinkProtoChecksum(frame);
    buffer[16] = UART_LINK_FRAME_TAIL;
}
else if (UART_LINK_TYPE_PARAM_GLOBAL_STATE == frame->type)
{
    putU32Le(buffer, 3u, frame->paramGlobalState.coordinatorBootGeneration);
    putU32Le(buffer, 7u, frame->paramGlobalState.sequence);
    putU32Le(buffer, 11u, frame->paramGlobalState.requestBootGeneration);
    putU32Le(buffer, 15u, frame->paramGlobalState.requestSequence);
    putU32Le(buffer, 19u, frame->paramGlobalState.transactionId);
    buffer[23] = frame->paramGlobalState.origin;
    buffer[24] = frame->paramGlobalState.action;
    buffer[25] = frame->paramGlobalState.slot;
    buffer[26] = frame->paramGlobalState.phase;
    buffer[27] = frame->paramGlobalState.target;
    buffer[28] = frame->paramGlobalState.error;
    buffer[29] = frame->paramGlobalState.readyMask;
    buffer[30] = frame->paramGlobalState.flags;
    buffer[31] = uartLinkProtoChecksum(frame);
    buffer[32] = UART_LINK_FRAME_TAIL;
}

```

```

    }
    else
    {
        buffer[3] = frame->value;
        buffer[4] = uartLinkProtoChecksum(frame);
        buffer[5] = UART_LINK_FRAME_TAIL;
    }

    return UART_LINK_PROTO_STATUS_OK;
}

uartLinkProtoStatusEnum uartLinkProtoInputByte(uartLinkParserStruct *parser, uint8_t data, uartLinkFrameStruct *frame)
{
    uint8_t frameLen;

    if (0 == parser || 0 == frame)
    {
        return UART_LINK_PROTO_STATUS_INVALID_PARAM;
    }

    /* 等待帧头：index==0 时只接受 HEAD，其余字节丢弃。 */
    if (0u == parser->index && UART_LINK_FRAME_HEAD != data)
    {
        return UART_LINK_PROTO_STATUS_OK;
    }

    parser->buffer[parser->index] = data;
    parser->index++;

    /* 读到 type（index==2 即第 2 字节落位后）确定期望帧长。 */
    if (2u == parser->index)
    {
        parser->expectLen = uartLinkProtoFrameLen(parser->buffer[1]);
        if (0u == parser->expectLen)
        {
            /* 未知 type：复位重新找帧头，报类型错。 */
            parser->index = 0u;
            return UART_LINK_PROTO_STATUS_TYPE_ERROR;
        }
    }

    /* expectLen 未定或未集满，继续收。 */
    if (parser->index < 2u || parser->index < parser->expectLen)
    {
        return UART_LINK_PROTO_STATUS_OK;
    }

    /* 集满一帧，解析。 */
    frameLen = parser->expectLen;
    parser->index = 0u;
    parser->expectLen = 0u;

    frame->type = parser->buffer[1];
    frame->sequence = parser->buffer[2];
    frame->value = 0u;
    frame->beaconCount = 0u;
    frame->linkFlags = 0u;
    frame->faultSource = 0u;
    frame->faultClass = 0u;
    frame->producerBootSession = 0u;
    frame->safetyState = 0u;
    frame->armProgress = 0u;
    frame->statusFlags = 0u;
    frame->sysCtrl = 0u;
    frame->buzzer = 0u;
    frame->poseEpoch = 0u;
    frame->poseXQ8 = 0u;
    frame->poseYQ8 = 0u;
    frame->poseOrientationRad = 0.0f;
    frame->poseConfidenceQ8 = 0u;
    frame->poseFlags = 0u;
    frame->motionVx = 0.0f;
    frame->motionVy = 0.0f;
    frame->motionWz = 0.0f;
    frame->motionFlags = 0u;
    frame->kvmTargetSession = 0u;
    frame->kvmRouteSession = 0u;
    frame->kvmRouteEpoch = 0u;
    frame->kvmEventSeq = 0u;

```

```

frame->kvmLeaseMs = 0u;
frame->kvmCore3AgeMs = 0u;
frame->kvmTarget = UART_LINK_KVM_TARGET_CORE0;
frame->kvmActiveTarget = UART_LINK_KVM_TARGET_CORE0;
frame->kvmKeyId = 0u;
frame->kvmKeyState = 0u;
frame->kvmMenuStates = 0xFFu;
frame->kvmFlags = 0u;
frame->kvmAckResult = UART_LINK_KVM_ACK_NONE;
frame->paramRequestId = 0u;
frame->paramOffset = 0u;
frame->paramOp = 0u;
frame->paramReserved = 0u;
frame->paramTarget = 0u;
frame->paramStatus = 0u;
frame->paramLength = 0u;
{ uint8_t i; for (i = 0u; i < UART_LINK_PARAM_DATA_BYTES; i++) frame->paramData[i] = 0u; }
frame->paramGlobalRequest.producerBootGeneration = 0u;
frame->paramGlobalRequest.sequence = 0u;
frame->paramGlobalRequest.origin = 0u;
frame->paramGlobalRequest.action = 0u;
frame->paramGlobalRequest.slot = 0u;
frame->paramGlobalRequest.reserved = 0u;
frame->paramGlobalState.coordinatorBootGeneration = 0u;
frame->paramGlobalState.sequence = 0u;
frame->paramGlobalState.requestBootGeneration = 0u;
frame->paramGlobalState.requestSequence = 0u;
frame->paramGlobalState.transactionId = 0u;
frame->paramGlobalState.origin = 0u;
frame->paramGlobalState.action = 0u;
frame->paramGlobalState.slot = 0u;
frame->paramGlobalState.phase = 0u;
frame->paramGlobalState.target = 0u;
frame->paramGlobalState.error = 0u;
frame->paramGlobalState.readyMask = 0u;
frame->paramGlobalState.flags = 0u;

if (UART_LINK_TYPE_STATUS == frame->type)
{
    frame->beaconCount = (uint16_t)(parser->buffer[3] | ((uint16_t)parser->buffer[4] << 8));
    frame->linkFlags = parser->buffer[5];
    frame->faultSource = parser->buffer[6];
    frame->faultClass = parser->buffer[7];
    frame->producerBootSession = getU32Le(parser->buffer, 8u);
}
else if (UART_LINK_TYPE_CORE0_STATUS == frame->type)
{
    frame->safetyState = parser->buffer[3];
    frame->armProgress = parser->buffer[4];
    frame->statusFlags = parser->buffer[5];
    frame->sysCtrl = parser->buffer[6];
    frame->buzzer = parser->buffer[7];
}
else if (UART_LINK_TYPE_MOTION == frame->type)
{
    uartLinkFloatBytes vx, vy, wz;
    uint8_t i;
    for (i = 0u; i < 4u; i++) vx.b[i] = parser->buffer[3u + i];
    for (i = 0u; i < 4u; i++) vy.b[i] = parser->buffer[7u + i];
    for (i = 0u; i < 4u; i++) wz.b[i] = parser->buffer[11u + i];
    frame->motionVx = vx.f;
    frame->motionVy = vy.f;
    frame->motionWz = wz.f;
    frame->motionFlags = parser->buffer[15];
    frame->producerBootSession = getU32Le(parser->buffer, 16u);
}
else if (UART_LINK_TYPE_CORE0_POSE == frame->type)
{
    uartLinkFloatBytes yaw;
    uint8_t i;
    frame->poseEpoch = parser->buffer[3];
    frame->poseXQ8 = (uint16_t)(parser->buffer[4] | ((uint16_t)parser->buffer[5] << 8));
    frame->poseYQ8 = (uint16_t)(parser->buffer[6] | ((uint16_t)parser->buffer[7] << 8));
    for (i = 0u; i < 4u; i++) yaw.b[i] = parser->buffer[8u + i];
    frame->poseOrientationRad = yaw.f;
    frame->poseConfidenceQ8 = parser->buffer[12];
    frame->poseFlags = parser->buffer[13];
}
else if (UART_LINK_TYPE_KVM_CONTROL == frame->type)

```

```

{
    frame->kvmTargetSession = getU32Le(parser->buffer, 3u);
    frame->kvmRouteSession = getU32Le(parser->buffer, 7u);
    frame->kvmRouteEpoch = getU32Le(parser->buffer, 11u);
    frame->kvmTarget = parser->buffer[15];
    frame->kvmLeaseMs = (uint16_t)(parser->buffer[16] | ((uint16_t)parser->buffer[17] << 8));
}
else if (UART_LINK_TYPE_KVM_EVENT == frame->type)
{
    frame->kvmTargetSession = getU32Le(parser->buffer, 3u);
    frame->kvmRouteSession = getU32Le(parser->buffer, 7u);
    frame->kvmRouteEpoch = getU32Le(parser->buffer, 11u);
    frame->kvmEventSeq = getU32Le(parser->buffer, 15u);
    frame->kvmTarget = parser->buffer[19];
    frame->kvmKeyId = parser->buffer[20];
    frame->kvmKeyState = parser->buffer[21];
}
else if (UART_LINK_TYPE_KVM_STATUS == frame->type)
{
    frame->kvmTargetSession = getU32Le(parser->buffer, 3u);
    frame->kvmRouteSession = getU32Le(parser->buffer, 7u);
    frame->kvmRouteEpoch = getU32Le(parser->buffer, 11u);
    frame->kvmEventSeq = getU32Le(parser->buffer, 15u);
    frame->kvmTarget = parser->buffer[19];
    frame->kvmActiveTarget = parser->buffer[20];
    frame->kvmMenuStates = parser->buffer[21];
    frame->kvmFlags = parser->buffer[22];
    frame->kvmAckResult = parser->buffer[23];
    frame->kvmCore3AgeMs = (uint16_t)(parser->buffer[24] | ((uint16_t)parser->buffer[25] << 8));
}
else if ((UART_LINK_TYPE_PARAM_REQUEST == frame->type) ||
         (UART_LINK_TYPE_PARAM_RESPONSE == frame->type))
{
    uint8_t i;
    frame->paramOp = parser->buffer[3];
    frame->paramReserved = parser->buffer[4];
    frame->paramTarget = parser->buffer[5];
    frame->paramStatus = parser->buffer[6];
    frame->paramRequestId = getU32Le(parser->buffer, 7u);
    frame->paramOffset = (uint16_t)(parser->buffer[11] | ((uint16_t)parser->buffer[12] << 8));
    frame->paramLength = parser->buffer[13];
    if (frame->paramLength > UART_LINK_PARAM_DATA_BYTES)
        return UART_LINK_PROTO_STATUS_INVALID_PARAM;
    for (i = 0u; i < UART_LINK_PARAM_DATA_BYTES; i++)
        frame->paramData[i] = parser->buffer[14u + i];
}
else if (UART_LINK_TYPE_PARAM_GLOBAL_REQUEST == frame->type)
{
    frame->paramGlobalRequest.producerBootGeneration = getU32Le(parser->buffer, 3u);
    frame->paramGlobalRequest.sequence = getU32Le(parser->buffer, 7u);
    frame->paramGlobalRequest.origin = parser->buffer[11];
    frame->paramGlobalRequest.action = parser->buffer[12];
    frame->paramGlobalRequest.slot = parser->buffer[13];
    frame->paramGlobalRequest.reserved = parser->buffer[14];
    if (paramSyncGlobalRequestsValid(&frame->paramGlobalRequest) == 0u)
        return UART_LINK_PROTO_STATUS_INVALID_PARAM;
}
else if (UART_LINK_TYPE_PARAM_GLOBAL_STATE == frame->type)
{
    frame->paramGlobalState.coordinatorBootGeneration = getU32Le(parser->buffer, 3u);
    frame->paramGlobalState.sequence = getU32Le(parser->buffer, 7u);
    frame->paramGlobalState.requestBootGeneration = getU32Le(parser->buffer, 11u);
    frame->paramGlobalState.requestSequence = getU32Le(parser->buffer, 15u);
    frame->paramGlobalState.transactionId = getU32Le(parser->buffer, 19u);
    frame->paramGlobalState.origin = parser->buffer[23];
    frame->paramGlobalState.action = parser->buffer[24];
    frame->paramGlobalState.slot = parser->buffer[25];
    frame->paramGlobalState.phase = parser->buffer[26];
    frame->paramGlobalState.target = parser->buffer[27];
    frame->paramGlobalState.error = parser->buffer[28];
    frame->paramGlobalState.readyMask = parser->buffer[29];
    frame->paramGlobalState.flags = parser->buffer[30];
}
else
{
    frame->value = parser->buffer[3];
}
}

if (uartLinkProtoChecksum(frame) != parser->buffer[frameLen - 2u])

```

```

    {
        return UART_LINK_PROTO_STATUS_CHECK_ERROR;
    }
    if (UART_LINK_FRAME_TAIL != parser->buffer[frameLen - 1u])
    {
        return UART_LINK_PROTO_STATUS_TAIL_ERROR;
    }

    return UART_LINK_PROTO_STATUS_FRAME_READY;
}

```

core0_cyt4bb7/project/code/comm/uartLinkProto.h

SHA-256

:

c06329a6c679a4526c3ed656c142cdbcb35a539a0978991bbd1c31f2ecdcb75d

代码 (c)

```

#ifndef _UART_LINK_PROTO_H_
#define _UART_LINK_PROTO_H_

/*
 * SYNC: core0 ↔ core1 两份 uartLinkProto.c/.h 必须逐字节同源。改任一份后 diff 校验另一份。
 *
 * 帧族（按 type 定长解析，HEAD/TAIL 不变）：
 * 基础帧 6 字节 [HEAD|type|seq|value|xor|TAIL]
 * - PING/ACK/HEARTBEAT：原有，value 留白
 * 状态帧 14 字节 [HEAD|type|seq|beaconLo|beaconHi|linkFlags|faultSource|faultClass|producerBootSession(4)|xor|TAIL]
 * - STATUS(0x40)：core1→core0 回传信标数、链路标志和通用必需视觉故障来源/类别
 *   linkFlags bit1=core2LinkLost(core1 测 RS-485↔core2 断联，core0 据此聚合缓降并禁 K5 发车)；
 *   linkFlags bit2=mode2LoraLandReq(模式 2 调试 LoRa 双肩键缓降请求，仅转 manualLand，不请求起飞)；
 *   linkFlags bit3=requiredVisionFault(任一必需视觉故障；plan 缺失/停更或必需视觉链路、相机、配置、标定故障均 fail-closed，由 core0 聚合缓降并禁 K5 发车)；
 *   与心跳同周期(200ms)常发，不受 MOTION 帧 gotMotion 门控影响(故 core2 断联上报走此帧而非 MOTION flags)
 * core0 状态帧 10 字节 [HEAD|0x44|seq|safetyState|armProgress|statusFlags|sysCtrl|buzzer|crc8|TAIL]
 * - CORE0_STATUS(0x44)：core0→core1 回传 飞控安全态 + arming 进度 + 状态位(供 RGB 状态灯)
 *   + sysCtrl(空地系统模式/RGB 意图/到顶标志) + buzzer(蜂鸣模式码，core1 中转 RS-485 给 core2 buzzer 库 pattern 层播放)
 * core0 位姿帧 16 字节 [HEAD|0x42|seq|epoch|xQ8(2)|yQ8(2)|yaw(4)|confQ8|flags|xor|TAIL]
 * - CORE0_POSE(0x42)：core0→core1 回传 core0 视觉灯板同坐标/方向，供 core1 车坐标误差生成和屏幕叠加
 * 运动帧 22 字节 [HEAD|0x51|seq|vx(4)|vy(4)|wz(4)|flags|producerBootSession(4)|crc8|TAIL]
 * - MOTION(0x51)：core1→core0 运动前馈 vx/vy/wz (3×float 小端，来源=core1 规划(beaconNavigator) 或 LoRa 遥控转发)
 *   + flags(bit0=remoteTakeoffReq：模式 3 车 LoRa S3+S4 请求，core0 按 TKO 阶段解释为起飞或缓降)
 * CORE0_STATUS/MOTION 使用 CRC-8/ATM(poly=0x07, init=0)，覆盖 type+seq+payload；
 * 其余帧保持 xor = type ^ seq ^ 其余 payload 各字节。
 * 字节序：多字节字段显式按字节小端拼装，不依赖结构体布局。
 */

#include <stdint.h>
#include "paramSyncGlobal.h"

#define UART_LINK_RX_SESSION_REJECT (0u)
#define UART_LINK_RX_SESSION_ACCEPT (1u)
#define UART_LINK_RX_SESSION_REBIND (2u)

/* Core0 只在 STATUS 已进入离线栅栏时允许任意非零 producer session 重绑。
 * 在线期间其它 session 一律拒绝，避免延迟旧帧抢占当前 producer。 */
static inline uint8_t uartLinkRxSessionClassify(uint8_t statusFenced,
        uint32_t currentSession, uint32_t incomingSession)
{
    if (incomingSession == 0u) return UART_LINK_RX_SESSION_REJECT;
    if (statusFenced != 0u) return UART_LINK_RX_SESSION_REBIND;
    return (incomingSession == currentSession) ? UART_LINK_RX_SESSION_ACCEPT : UART_LINK_RX_SESSION_REJECT;
}

#define UART_LINK_BAUDRATE (4000 000u)

#define UART_LINK_FRAME_HEAD (0xA5u)
#define UART_LINK_FRAME_TAIL (0xA5Au)

#define UART_LINK_FRAME_LENGTH      (6u) /* 基础帧 (PING/ACK/HEARTBEAT) */
#define UART_LINK_FRAME_LENGTH_STATUS (14u) /* STATUS 含故障事实及 Core1 启动身份 */
typedef char uartLink_status_frame_length_assert[(UART_LINK_FRAME_LENGTH_STATUS == 14u) ? 1 : -1];
#define UART_LINK_FRAME_LENGTH_C0STATUS (10u) /* core0 状态帧 (CORE0_STATUS 0x44, 含 sysCtrl+buzzer) */
#define UART_LINK_FRAME_LENGTH_C0POSE (16u) /* core0 位姿帧 [HEAD|0x42|seq|epoch|xQ8|yQ8|yaw|conf|flags|xor|TAIL] */
#define UART_LINK_FRAME_LENGTH_MOTION (22u) /* 运动帧含 Core1 启动身份 */

```

```

#define UART_LINK_FRAME_LENGTH_KVM_CONTROL (20u)
#define UART_LINK_FRAME_LENGTH_KVM_EVENT (24u)
#define UART_LINK_FRAME_LENGTH_KVM_STATUS (28u)
#define UART_LINK_FRAME_LENGTH_PARAM (28u)
#define UART_LINK_FRAME_LENGTH_PARAM_GLOBAL_REQUEST (17u)
#define UART_LINK_FRAME_LENGTH_PARAM_GLOBAL_STATE (33u)
#define UART_LINK_PARAM_DATA_BYTES (12u)
#define UART_LINK_FRAME_LENGTH_MAX (33u)

#define UART_LINK_TYPE_PING (0x10u)
#define UART_LINK_TYPE_ACK (0x11u)
#define UART_LINK_TYPE_HEARTBEAT (0x20u)
#define UART_LINK_TYPE_KVM_CONTROL (0x31u) /* core0→core1 KVM 路由租约 */
#define UART_LINK_TYPE_KVM_EVENT (0x32u) /* core0→core1 KVM exactly-once 事件 */
#define UART_LINK_TYPE_STATUS (0x40u) /* core1→core0 状态回传 */
#define UART_LINK_TYPE_CORE0_STATUS (0x44u) /* core0→core1 飞控状态; CRC-8/ATM */
#define UART_LINK_TYPE_CORE0_POSE (0x42u) /* core0→core1 灯板同坐标/方向, 供 core1 车坐标误差和屏幕叠加 */
#define UART_LINK_TYPE_KVM_STATUS (0x43u) /* core1→core0 KVM 菜单状态与终态 ACK */
#define UART_LINK_TYPE_MOTION (0x51u) /* core1→core0 运动前馈指令; CRC-8/ATM */
#define UART_LINK_TYPE_PARAM_REQUEST (0x60u)
#define UART_LINK_TYPE_PARAM_RESPONSE (0x61u)
#define UART_LINK_TYPE_PARAM_GLOBAL_STATE (0x62u)
#define UART_LINK_TYPE_PARAM_GLOBAL_REQUEST (0x63u)
#define UART_LINK_PARAM_TRANSPORT_BEGIN (0x80u)
#define UART_LINK_PARAM_TRANSPORT_MORE (0x40u)
#define UART_LINK_PARAM_OWNER_RESTART (0x20u)
#define UART_LINK_PARAM_STATUS_MASK (0x3Fu)
#define UART_LINK_PARAM_OP_POLL (0x7Fu)

/* 三目标 KVM 线值。Core3 使用物理节点编号 3, 禁止压缩成索引 2。 */
#define UART_LINK_KVM_TARGET_CORE0 (0u)
#define UART_LINK_KVM_TARGET_CORE1 (1u)
#define UART_LINK_KVM_TARGET_CORE3 (3u)
#define UART_LINK_KVM_TARGET_CORE4 (4u)
#define UART_LINK_KVM_TARGET_NONE (0xFFu)

/* KVM_STATUS flags。HOME 位只有对应 VALID 且状态未陈旧时才可用于 K5 判定。 */
#define UART_LINK_KVM_FLAG_ROUTE_ACCEPTED (0x01u)
#define UART_LINK_KVM_FLAG_LEASE_VALID (0x02u)
#define UART_LINK_KVM_FLAG_CORE1_VALID (0x04u)
#define UART_LINK_KVM_FLAG_CORE1_HOME (0x08u)
#define UART_LINK_KVM_FLAG_CORE3_VALID (0x10u)
#define UART_LINK_KVM_FLAG_CORE3_HOME (0x20u)
#define UART_LINK_KVM_FLAG_ACK_TERMINAL (0x40u)
#define UART_LINK_KVM_FLAG_PEER_RESTART (0x80u)

/* KVM 终态/拒绝结果。同一 boot session 内重复事件只重放 ACK, 不重复副作用。 */
#define UART_LINK_KVM_ACK_NONE (0u)
#define UART_LINK_KVM_ACK_EXECUTED (1u)
#define UART_LINK_KVM_ACK_DUPLICATE (2u)
#define UART_LINK_KVM_ACK_UNHANDLED (3u)
#define UART_LINK_KVM_ACK_BUSY (4u)
#define UART_LINK_KVM_ACK_GLOBAL_EVENT_REJECTED (5u)
#define UART_LINK_KVM_ACK_NO_LEASE (6u)
#define UART_LINK_KVM_ACK_WRONG_TARGET (7u)
#define UART_LINK_KVM_ACK_STALE_ROUTE (8u)
#define UART_LINK_KVM_ACK_GAP (9u)
#define UART_LINK_KVM_ACK_OLD_EVENT (10u)
#define UART_LINK_KVM_ACK_BAD_PAYLOAD (11u)
#define UART_LINK_KVM_ACK_PEER_RESTART (12u)

/* STATUS 的 linkFlags 位定义 (core1→core0; core1 cm_7_0 发, core0 crossCoreLinkPoll 取) */
#define UART_LINK_STATUS_FLAG_LINK_OK (0x01u) /* bit0: core1 认为核间链路在线 */
#define UART_LINK_STATUS_FLAG_CORE2_LOST (0x02u) /* bit1: core1 测 RS-485↔core2 断联(→core1 RS485 超时), core0 缓降聚合用 */
#define UART_LINK_STATUS_FLAG_MODE2_LORA_LAND_REQ (0x04u) /* bit2: 模式 2 LoRa 双肩键调试缓降请求 */
#define UART_LINK_STATUS_FLAG_REQUIRED_VISION_FAULT (0x08u)
#define UART_LINK_STATUS_FLAG_BEACON_SEARCHING (0x10u) /* bit4: Core1 正在执行寻灯 */
#define UART_LINK_STATUS_FLAG_BEACON_SEARCH_TIMEOUT (0x20u) /* bit5: 本次寻灯已超时 */
#define UART_LINK_STATUS_FLAG_BEACON_VALID (0x40u) /* bit6: beaconCount/寻灯字段来自 fresh 计划 */
#define UART_LINK_STATUS_FLAG_CORE2_ATTITUDE_FAULT (0x80u) /* bit7: Core2 姿态诊断故障 */

/* CORE0_STATUS 的 statusFlags 位定义 */
#define UART_LINK_C0_FLAG_CM70_READY (0x01u) /* cm_7_0 初始化完成 */
#define UART_LINK_C0_FLAG_VEL_VALID (0x02u) /* 速度估计有效 */
#define UART_LINK_C0_FLAG_TARGET_VALID (0x04u) /* ^灯板视觉目标锁定 */
#define UART_LINK_C0_FLAG_TOF_HEALTHY (0x08u) /* ToF 定高健康 */
#define UART_LINK_C0_FLAG_LAUNCHED (0x10u) /* 飞控已发车(safetyState==CLOSED_LOOP): core1 读此位中转为 RS-485 cmd LAUNCH 给 core2 */

```

```

/* CORE0_STATUS 的 sysCtrl 字节位布局 (core0 填 / core1 取) :
* bit0-1 = systemMode; bit2-4 = rgbIntent; bit5 = altReached;
* bit6-7 = padding, 发送端恒为 0, 接收端不得赋予业务语义。 */
#define UART_LINK_SYS_MODE_MASK (0x03u)
#define UART_LINK_SYS_RGB_SHIFT (2u)
#define UART_LINK_SYS_RGB_MASK (0x07u) /* 取值: (sysCtrl>>SHIFT)&MASK */
#define UART_LINK_SYS_ALT_REACHED (0x20u)
/* 空地系统模式线值 (systemMode, CORE0_STATUS sysCtrl bit0-1; core0/core1 共用, 本宏为语义单一真相源) :
* 1=车不动 2=RS-485 自主(车随视觉误差) 3=LoRa 转发(车 LoRa 遥控)。0=未设。 */
#define UART_LINK_SYS_MODE_CAR_STILL (1u)
#define UART_LINK_SYS_MODE_RS485_AUTO (2u)
#define UART_LINK_SYS_MODE_LORA_FWD (3u)

/* rgbIntent 意图码 (core0 产生 → core1 rgbAnimator 渲染: 3 bit, 0..7) */
#define UART_LINK_RGB_NORMAL (0u) /* 无动画, 走 core1 原状态灯优先级链 */
#define UART_LINK_RGB_TKO_WAIT (1u) /* 起飞 5s 等待: 快闪频率渐增→常亮 */
#define UART_LINK_RGB_TKO_RAMP (2u) /* 起飞 2s 爬高: 常亮→线性渐灭 */
#define UART_LINK_RGB_DISCONNECT_WARN (3u) /* 断联告警: 加速闪烁 */

/* MOTION 帧 flags 字节位定义 (core2 → core1 → core0 反向信号; core1 中转填位) */
#define UART_LINK_MOTION_FLAG_TAKEOFF_REQ (0x01u) /* 模式 3 车 LoRa S3+S4 请求: TKO_IDLE 起飞, WAIT/RAMP/CRUISE 缓降 */

/* CORE0_POSE flags 位定义 (位置与 heading 独立资格; confidence 仅作 heading 更新门控)。 */
#define UART_LINK_POSE_FLAG_TRACK_LOCKED (0x01u) /* bit0: core0 视觉跟踪锁定 */
#define UART_LINK_POSE_FLAG_TARGET_LOST (0x02u) /* bit1: core0 视觉报告丢失 */
#define UART_LINK_POSE_FLAG_POSITION_NUMERIC_VALID (0x04u) /* bit2: x/y 数值与范围有效 */
#define UART_LINK_POSE_FLAG_POSITION_FRESH (0x08u) /* bit3: 本帧位置可作为 core1 姿态候选 */
#define UART_LINK_POSE_FLAG_HEADING_FRESH (0x10u) /* bit4: 本帧方向样本独立有效 */

/* buzzer 蜂鸣模式码 (core0 产生 → core1 中转 RS-485 → core2 buzzer 库 pattern 层播放; 与 chassisLink CHASSIS_LINK_BUZZER_* 同值 SYNC) */
#define UART_LINK_BUZZER_SILENT (0u)
#define UART_LINK_BUZZER_1HZ (1u) /* 起飞上升期: 每秒一次 */
#define UART_LINK_BUZZER_LONG (2u) /* 到顶: 长鸣一声 */
#define UART_LINK_BUZZER_3S (3u) /* 巡航: 每 3s 短促一声 */
#define UART_LINK_BUZZER_WARN (4u) /* 断联告警: 急促鸣 */
#define UART_LINK_BUZZER_ARMED_IDLE (5u) /* 模式 3 已 arm 待车 S3+S4 请求: 每 ~2s 短鸣警示"已待命"(桨在转) */
#define UART_LINK_BUZZER_WAITING (6u) /* 等待连接: 每 ~3s 短鸣(core2 本地 WAIT_RS485 用; core0 不下发此码) */
#define UART_LINK_BUZZER_THREE_BEEPS (7u) /* 完整系统资格上升沿: 三声一次 */

typedef enum
{
    UART_LINK_PROTO_STATUS_OK = 0u,
    UART_LINK_PROTO_STATUS_FRAME_READY,
    UART_LINK_PROTO_STATUS_INVALID_PARAM,
    UART_LINK_PROTO_STATUS_TYPE_ERROR,
    UART_LINK_PROTO_STATUS_CHECK_ERROR,
    UART_LINK_PROTO_STATUS_TAIL_ERROR
} uartLinkProtoStatusEnum;

typedef struct
{
    uint8_t type;
    uint8_t sequence;
    uint8_t value; /* 基础帧 payload */
    uint16_t beaconCount; /* STATUS 帧: 信标数 */
    uint8_t linkFlags; /* STATUS 帧: 链路标志 */
    uint8_t faultSource; /* STATUS: 必需视觉故障来源, 0=无 */
    uint8_t faultClass; /* STATUS: 必需视觉故障类别, 0=无 */
    uint32_t producerBootSession; /* STATUS/MOTION: Core1 cm7_0 启动身份, 0 非法 */
    uint8_t safetyState; /* CORE0_STATUS 帧: 飞控安全态 */
    uint8_t armProgress; /* CORE0_STATUS 帧: arming 进度 0-100 */
    uint8_t statusFlags; /* CORE0_STATUS 帧: 状态位(UART_LINK_C0_FLAG_*) */
    uint8_t sysCtrl; /* CORE0_STATUS 帧: 空地系统模式/RGB 意图/到顶标志(UART_LINK_SYS_*) */
    uint8_t buzzer; /* CORE0_STATUS 帧: 蜂鸣模式码(UART_LINK_BUZZER_*, 中转 core2 buzzer 库 pattern 层播放) */
    uint8_t poseEpoch; /* CORE0_POSE 帧: core0 链路初始化 epoch, 变更表示 core0 重启/共享区需重新锁存 */
    uint16_t poseXQ8; /* CORE0_POSE 帧: core0 灯板 X, 同坐标 Q8.8 */
    uint16_t poseYQ8; /* CORE0_POSE 帧: core0 灯板 Y, 同坐标 Q8.8 */
    float poseOrientationRad; /* CORE0_POSE 帧: 灯板方向 rad, 范围约 [-pi, pi] */
    uint8_t poseConfidenceQ8; /* CORE0_POSE 帧: 方向置信度 Q0.8 (0.255) */
    uint8_t poseFlags; /* CORE0_POSE 帧: flags(UART_LINK_POSE_FLAG_*) */
    float motionVx; /* MOTION 帧: 速度指令 vx; core0 速度前馈直用, pitch 前馈用其指令微分 */
    float motionVy; /* MOTION 帧: 速度指令 vy; core0 速度前馈直用, roll 前馈用其指令微分 */
    float motionWz; /* MOTION 帧: 角速度指令 wz; core0 yaw 前馈直通 */
    uint8_t motionFlags; /* MOTION 帧: flags(UART_LINK_MOTION_FLAG_*, bit0=remoteTakeoffReq) */
    /* KVM_CONTROL/EVENT/STATUS: 显式字段按帧类型编码, 禁止裸结构体传输。 */
    uint32_t kvmTargetSession; /* 目标路由 owner boot session echo, 重启后旧事件拒绝 */
    uint32_t kvmRouteSession; /* Core0 cm7_1 菜单 boot epoch */
    uint32_t kvmRouteEpoch; /* 每次切目标递增 */
    uint32_t kvmEventSeq; /* 每 route epoch 从 1 起; STATUS 中为最终终态 ACK seq */

```



```

uint16_t kvmLeaseMs;    /* CONTROL 租约上限 */
uint16_t kvmCore3AgeMs; /* STATUS: Core3 菜单状态年龄 */
uint8_t kvmTarget;
uint8_t kvmActiveTarget;
uint8_t kvmKeyId;
uint8_t kvmKeyState;
uint8_t kvmMenuStates; /* STATUS: 低 4 位 Core1、高 4 位 Core3 原始菜单态 */
uint8_t kvmFlags;
uint8_t kvmAckResult;
uint32_t paramRequestId;
uint16_t paramOffset;
uint8_t paramOp;
uint8_t paramReserved;
uint8_t paramTarget;
uint8_t paramStatus;
uint8_t paramLength;
uint8_t paramData[UART_LINK_PARAM_DATA_BYTES];
paramSyncGlobalRequest paramGlobalRequest;
paramSyncGlobalState paramGlobalState;
} uartLinkFrameStruct;

typedef struct
{
    uint8_t index;
    uint8_t expectLen; /* 收到 type 后据此定帧长 (0=未定, 等 type 字节) */
    uint8_t buffer[UART_LINK_FRAME_LENGTH_MAX];
} uartLinkParserStruct;

void uartLinkProtoInit(uartLinkParserStruct *parser);
uint8_t uartLinkProtoChecksum(const uartLinkFrameStruct *frame);
uartLinkProtoStatusEnum uartLinkProtoEncode(const uartLinkFrameStruct *frame, uint8_t *buffer, uint8_t bufferSize);
uartLinkProtoStatusEnum uartLinkProtoInputByte(uartLinkParserStruct *parser, uint8_t data, uartLinkFrameStruct *frame);

#endif /* _UART_LINK_PROTO_H_ */

```

core0_cyt4bb7/project/code/config/boardConfig.h

SHA-256

:

036f1174c022dd4d1433fb2cea661ba906bad2ea5e4ebc2e8d333706f1d64cc9

代码 (c)

```

#ifndef _BOARD_CONFIG_H_
#define _BOARD_CONFIG_H_

/*
 * core0 编译期硬件/工程选择器 (单一汇集点)
 *
 * 本文件只放**编译期锁定**的板级选择器: 改这些值会切换条件编译、改变
 * 外设接线/驱动选择或调试链路, 绝不可在运行时改 (不进 flightParams/菜单/flash)。
 *
 * 与飞控参数子头的分工:
 * - boardConfig.h (本文件) = 板级编译期选择器 (型号/通信/持久化开关)
 * - configFlightControlDefaults.h = 进 flightParams 的可调默认值 (PID/限幅/参考)
 * - configFlightTiming.h / configVelocityEstimator.h / configAltitudeToF.h =
 *   飞控算法侧的编译期锁定项 (调度分频/通道数/估计器/ToF)
 *
 * 调用方必须按实际消费直接包含本头或对应参数子头, 不再依赖旧聚合头转包。
 */

/* =====
 * 1. 全局可调参数持久化开关
 * ===== */
/* 1 = 菜单可调参数从 Flash 加载、允许调节与人工保存 (现有行为)。
 * 0 = 上电只载入默认可调参数; 菜单改值键与人工保存键无效。
 * 注意: 视觉自标定模板 (Legacy/BinaryCC 两槽 Hu+尺寸, 人类不可调) 不受本开关影响, 仍正常持久化。*/
#ifndef GLOBAL_PARAM_PERSIST_ENABLE
#define GLOBAL_PARAM_PERSIST_ENABLE (1)
#endif

/* =====
 * 1.5 核间链路断联保护开关(远程菜单系统)
 * ===== */
/* 1 = 启用核间链路(core0↔core1)断联保护: 任一方检测对方心跳超时(600ms)→屏显告警 + 禁止发车。
 * core0 还会据 core1 上报的 core2LinkLost 禁止发车, 避免 core2 未入链时误发车。

```

```

* 0 = 仅显示链路状态, 不阻断发车(便于无 core1 单核调试 core0)。
* core0 据 flightToVision.core1LinkUp/core2LinkLost 判全链路; core1 据共享内存 linkUp 判 core0 在线。
* 注意: core1 在 menuInternal.h 有同名宏, 两核改值须同步(语义对称, 缺一端则保护不对等)。 */
#ifndef INTER_CORE_LINK_GUARD_ENABLE
#define INTER_CORE_LINK_GUARD_ENABLE (1)
#endif

/* =====
* 2. ToF 传感器型号选择
* ===== */
/* VL53L1X 单区 / VL53L8CX 4x4。驱动 altitudeToF.c / tofBus.c 据此条件编译,
* 只编入被选芯片一套代码。 */
#define ALT_TOF_SENSOR_VL53L1X (1u)
#define ALT_TOF_SENSOR_VL53L8CX (2u)
#ifndef ALT_TOF_SENSOR_SELECT
#define ALT_TOF_SENSOR_SELECT (ALT_TOF_SENSOR_VL53L8CX)
#endif

/* =====
* 3. VL53L8CX 通信协议选择
* ===== */
/* 软件 IIC(P19_0/1) / 硬件 SPI(SPI_3/P03_x)。仅 L8CX 生效; VL53L1X 恒为软件 IIC。
* 当前默认 HARD_SPI——传感器总线接 P03_2/1/0(SCK/MOSI/MISO)、SPI_I2C_N(C1)→IOVDD 上拉、
* NCS(C2)→P03_3、MISO(C5)→P03_0 (详见 docs/PINOUT.md)。若硬件仍按 P19 软 IIC 接线,
* 将读不到 L8CX(ToF NG)。 */
#define ALT_TOF_L8CX_COMM_SOFT_IIC (1u)
#define ALT_TOF_L8CX_COMM_HARD_SPI (2u)
#ifndef ALT_TOF_L8CX_COMM_SELECT
#define ALT_TOF_L8CX_COMM_SELECT (ALT_TOF_L8CX_COMM_HARD_SPI)
#endif

#endif /* _BOARD_CONFIG_H_ */

```

core0_cyt4bb7/project/code/config/commonDef.h

SHA-256

:

cfc8375acd05484d0bdee4bc0ef934f38d3524aff19ee0a1a36a45851da32dd9

代码 (c)

```

#ifndef COMMON_DEF_H
#define COMMON_DEF_H

#include <stdint.h>

/*
* 跨模块公共定义: 纯常量 / 数学工具 / 图像类型。
* 本文件不含编译期硬件选择器 (在 boardConfig.h, 需要者直接包含)、也不含运行时可调参数
* (在 configFlightControlDefaults.h / flightParams.h)。这里全是无副作用、可安全包含的常量与工具。
*/

/* =====
* 1. 数学宏 (钳位 / 角度换算 / 圆周率)
* ===== */
#define range(x, min, max) (((x) < (min)) ? (min) : (((x) > (max)) ? (max) : (x)))
#define unsignRange(x, max) (((x) > (max)) ? (max) : (x))

#define toRad(x) ((x) * 3.14159265358979323846f / 180.0f) /* 储备: 当前无消费者, 与在用的 toDeg 对称保留 */
#define toDeg(x) ((x) * 180.0f / 3.14159265358979323846f)

/* PI 可能已由 CMSIS DSP 的 arm_math.h 定义 (值在 float 精度下等价),
* 用 #ifndef 守卫避免 Pa181 宏重定义警告 */
#ifndef PI
#define PI (3.14159265358979323846f)
#endif
#define TWO_PI (2.0f * PI)
#define HALF_PI (0.5f * PI)

/* =====
* 2. 二值图像颜色
* ===== */
#define BLACK 0 /* 二值背景色 */
#define WHITE 255 /* 二值前景色 */
#define cReverse(x) ((x) ? BLACK : WHITE) /* 颜色翻转 */

```

```

/* =====
* 3. 跨模块 inline 工具（钳位 / 角度归一化）
* ===== */
static inline float clampf(float v, float lo, float hi)
{
    if (v < lo) return lo;
    if (v > hi) return hi;
    return v;
}

static inline uint8_t floatIsFinite(float v)
{
    return (uint8_t) ((v == v) && ((v - v) == 0.0f));
}

static inline float wrapToPi(float a)
{
    while (a > PI) a -= TWO_PI;
    while (a < -PI) a += TWO_PI;
    return a;
}

/* =====
* 4. 通用图像 typedef
*   u8Image 为视觉管线主力类型。
* ===== */
typedef struct
{
    uint8_t* ptr;

    uint32_t xLen;
    uint32_t yLen;

    uint32_t size;
} u8Image;

#endif

```

core0_cyt4bb7/project/code/config/configAltitudeTof.h

SHA-256

:

9a60c00ae4173e0a28076a7c22226da51813329359aeb6cf7c18d001b0dbf5a2

代码 (c)

```

#ifndef _CONFIG_ALTITUDE_TOF_H_
#define _CONFIG_ALTITUDE_TOF_H_

/*
* core0 定高 ToF 驱动/facade 编译期参数。
*
* 板级芯片型号与 L8CX 通信方式选择仍在 boardConfig.h；本头只放被选 ToF
* 驱动与 altitudeTof facade 消费的采样、轮询、空间融合和时间域创新门控参数。
*/

/* ---- VL53L1X：单区 30Hz 左右 ---- */
#define ALT_TOF_L1X_SAMPLE_MS      (33u)
#define ALT_TOF_L1X_TIMING_BUDGET_US (33 000u)
#define ALT_TOF_L1X_IIC_DELAY      (100u)

/* ---- VL53L8CX：4x4@60Hz，驱动内部融合为单一高度 ---- */
#define ALT_TOF_L8CX_FREQUENCY_HZ  (60u)
#define ALT_TOF_L8CX_IIC_DELAY      (50u)
#define ALT_TOF_POLL_PERIOD_MS      (8u)
#define ALT_TOF_L8CX_MIN_VALID_ZONES (6u)
#define ALT_TOF_L8CX_OUTLIER_ABS_MM (120u)
#define ALT_TOF_L8CX_OUTLIER_REL_PERMILLE (180u)
#define ALT_TOF_L8CX_HARD_MIN_MM    (30u)
#define ALT_TOF_L8CX_HARD_MAX_MM    (4000u)

/* ---- ToF facade 通用时间域创新门控 ---- */
#define ALT_TOF_INIT_TIMEOUT_MS      (3000u)
#define ALT_TOF_MAX_SPEED_MM_S      (4000u)
#define ALT_TOF_MIN_STEP_MM         (50u)
#define ALT_TOF_HEALTH_TIMEOUT_MS    (200u)

```

```

#define ALT_TOF_MIN_TILT_COS    (0.80f)

#if (ALT_TOF_INIT_TIMEOUT_MS == 0u)
#error "ALT_TOF_INIT_TIMEOUT_MS must be > 0"
#endif
#if (ALT_TOF_MAX_SPEED_MM_S == 0u)
#error "ALT_TOF_MAX_SPEED_MM_S must be > 0"
#endif
#if (ALT_TOF_MIN_STEP_MM == 0u)
#error "ALT_TOF_MIN_STEP_MM must be > 0"
#endif

#endif /* _CONFIG_ALTITUDE_TOF_H_ */

```

core0_cyt4bb7/project/code/config/configFlightControlDefaults.h

SHA-256

:

903bc482de0a1f097b4fe9531d87f326fd75f90bb1da66676961ebbc676b9a27

代码 (c)

```

#ifndef _CONFIG_FLIGHT_CONTROL_DEFAULTS_H_
#define _CONFIG_FLIGHT_CONTROL_DEFAULTS_H_

/*
 * core0 flightParams 运行时可调默认值。
 *
 * 量纲约定：角度 rad；角速度 rad/s；油门 %；控制量 pp；高度 mm；
 * 低通 alpha 属 (0,1]，alpha >= 1.0f 表示不启用 LPF。
 * 本头只放会进入 flightParams RAM 镜像 / 菜单 / flash 的默认值与运行时钳位边界。
 */

/* ---- 油门 (%) ---- */
#define PID_FLIGHT_BASE_THROTTLE_PCT (19.0f)
#define PID_FLIGHT_THROTTLE_PCT_MIN (1.0f)
#define PID_FLIGHT_THROTTLE_PCT_MAX (85.0f)

/* ---- 角度参考 (rad, 无位置环修正时的基准) ---- */
#define PID_FLIGHT_PITCH_REF_RAD (0.0f)
#define PID_FLIGHT_ROLL_REF_RAD (0.0f)

/* ---- 角度外环 PID (周期 3ms；输入角度 rad -> 输出目标角速度 rad/s) ---- */
#define PID_FLIGHT_PITCH_ANGLE_KP (5.5f)
#define PID_FLIGHT_PITCH_ANGLE_KI (0.0f)
#define PID_FLIGHT_PITCH_ANGLE_KD (0.0f)
#define PID_FLIGHT_PITCH_ANGLE_INTEGRAL_MAX (1.0f)
#define PID_FLIGHT_PITCH_ANGLE_DLPF_ALPHA (1.0f)

#define PID_FLIGHT_ROLL_ANGLE_KP (5.5f)
#define PID_FLIGHT_ROLL_ANGLE_KI (0.0f)
#define PID_FLIGHT_ROLL_ANGLE_KD (0.0f)
#define PID_FLIGHT_ROLL_ANGLE_INTEGRAL_MAX (1.0f)
#define PID_FLIGHT_ROLL_ANGLE_DLPF_ALPHA (1.0f)

#define PID_FLIGHT_RATE_CMD_LIMIT_PITCH (1000.0f)
#define PID_FLIGHT_RATE_CMD_LIMIT_ROLL (1000.0f)

/* ---- 角速度内环 PID (周期 1ms；输入角速度 rad/s -> 输出控制量 pp) ---- */
#define PID_FLIGHT_PITCH_RATE_KP (3.80000 019f)
#define PID_FLIGHT_PITCH_RATE_KI (0.00899 999 961f)
#define PID_FLIGHT_PITCH_RATE_KD (14.0f)
#define PID_FLIGHT_PITCH_RATE_INTEGRAL_MAX (5.5f)
#define PID_FLIGHT_PITCH_RATE_DLPF_ALPHA (1.0f)

#define PID_FLIGHT_ROLL_RATE_KP (3.80000 019f)
#define PID_FLIGHT_ROLL_RATE_KI (0.00899 999 961f)
#define PID_FLIGHT_ROLL_RATE_KD (14.0f)
#define PID_FLIGHT_ROLL_RATE_INTEGRAL_MAX (5.5f)
#define PID_FLIGHT_ROLL_RATE_DLPF_ALPHA (1.0f)

#define PID_FLIGHT_YAW_RATE_REF_RAD_S (0.0f)
#define PID_FLIGHT_YAW_RATE_KP (15.0f)
#define PID_FLIGHT_YAW_RATE_KI (0.0f)
#define PID_FLIGHT_YAW_RATE_KD (0.0f)
#define PID_FLIGHT_YAW_RATE_INTEGRAL_MAX (5.0f)

```

```

#define PID_FLIGHT_YAW_RATE_DLPF_ALPHA (1.0f)

#define PID_FLIGHT_PITCH_RATE_FF (0.0f)
#define PID_FLIGHT_ROLL_RATE_FF (0.0f)

#define PID_FLIGHT_CTRL_LIMIT_PCT_PITCH (30.0f)
#define PID_FLIGHT_CTRL_LIMIT_PCT_ROLL (30.0f)
#define PID_FLIGHT_CTRL_LIMIT_PCT_YAW (15.0f)

/* ---- IMU 角速度低通（内环反馈用；Mahony 仍用 raw gyro） ---- */
#define GYRO_LPF_ALPHA (1.0f)

/* ---- 姿态安全保护（超限进 PROTECTED） ---- */
#define POSTURE_SAFE_PITCH_MIN_RAD (-1.5f)
#define POSTURE_SAFE_PITCH_MAX_RAD (1.5f)
#define POSTURE_SAFE_ROLL_MIN_RAD (-1.5f)
#define POSTURE_SAFE_ROLL_MAX_RAD (1.5f)
#define POSTURE_SAFE_GYRO_MAX_RAD_S (50.0f)

/* ---- 定高双环（位置环 + 速度环） ---- */
#define ALTITUDE_HOLD_TARGET_MM (1700u)
#define ALTITUDE_HOLD_DIST_MIN_MM (50u)
#define ALTITUDE_HOLD_DIST_MAX_MM (4000u)
#define ALTITUDE_HOLD_TIMEOUT_MS (1000u)

#define ALTITUDE_HOLD_VEL_LPF_ALPHA (0.1f)

#define ALTITUDE_HOLD_POS_KP (0.85f)
#define ALTITUDE_HOLD_POS_KI (0.0f)
#define ALTITUDE_HOLD_POS_KD (0.0f)
#define ALTITUDE_HOLD_POS_INTEGRAL_MAX (1.0f)
#define ALTITUDE_HOLD_POS_DLPF_ALPHA (1.0f)

#define ALTITUDE_HOLD_VEL_KP (0.016f)
#define ALTITUDE_HOLD_VEL_KI (0.0002999999985f)
#define ALTITUDE_HOLD_VEL_KD (0.016f)
#define ALTITUDE_HOLD_VEL_INTEGRAL_MAX (15.0f)
#define ALTITUDE_HOLD_VEL_DLPF_ALPHA (1.0f)
#define ALTITUDE_HOLD_VEL_FF (0.0f)

#define ALTITUDE_HOLD_OFFSET_LIMIT_PCT (25.0f)
#define ALTITUDE_HOLD_VEL_CMD_LIMIT (500000.0f)

/* ---- 视觉定点调试开关与空地系统默认模式 ---- */
#define FLIGHT_VISION_POS_ERR_ENABLE (1u)
#define FLIGHT_SYSTEM_MODE_DEFAULT (2u)
#define FLIGHT_SYSTEM_MODE_FORCE_DEFAULT_ON_BOOT (0u)
/* ---- 位置外环（像素误差 -> 物理速度目标 m/s） ---- */
#define PID_FLIGHT_POS_X_KP (0.007f)
#define PID_FLIGHT_POS_X_KI (0.0f)
#define PID_FLIGHT_POS_X_KD (0.0f)
#define PID_FLIGHT_POS_X_INTEGRAL_MAX (1.0f)
#define PID_FLIGHT_POS_X_DLPF_ALPHA (1.0f)

#define PID_FLIGHT_POS_Y_KP (0.007f)
#define PID_FLIGHT_POS_Y_KI (0.0f)
#define PID_FLIGHT_POS_Y_KD (0.0f)
#define PID_FLIGHT_POS_Y_INTEGRAL_MAX (1.0f)
#define PID_FLIGHT_POS_Y_DLPF_ALPHA (1.0f)

/* ---- 速度内环（物理速度目标 m/s -> 角度修正 rad） ---- */
#define PID_FLIGHT_POS_VX_KP (0.91f)
#define PID_FLIGHT_POS_VX_KI (0.0f)
#define PID_FLIGHT_POS_VX_KD (1.6f)
#define PID_FLIGHT_POS_VX_INTEGRAL_MAX (0.1f)
#define PID_FLIGHT_POS_VX_DLPF_ALPHA (1.0f)

#define PID_FLIGHT_POS_VY_KP (0.91f)
#define PID_FLIGHT_POS_VY_KI (0.0f)
#define PID_FLIGHT_POS_VY_KD (1.6f)
#define PID_FLIGHT_POS_VY_INTEGRAL_MAX (0.1f)
#define PID_FLIGHT_POS_VY_DLPF_ALPHA (1.0f)

#define PID_FLIGHT_POS_VEL_CMD_LIMIT (4.0f)
#define PID_FLIGHT_POS_ANGLE_LIMIT_RAD (1.0f)

/* KF 速度失效降级：速度环周期 9ms，17 tick 覆盖 150ms。 */
#define PID_FLIGHT_KF_INVALID_FREEZE_MS (150u)
#define PID_FLIGHT_KF_INVALID_FREEZE_TICKS (17u)

```

```
/* ---- 姿态补偿（位置误差 errComp = err - K*物理倾角） ----
 * 默认值取自当前上板完赛基线，X/Y 沿用既有物理轴极性。 */
#define PID_FLIGHT_POS_ATT_COMP_X_GAIN_PX_PER_RAD (180.722222f)
#define PID_FLIGHT_POS_ATT_COMP_Y_GAIN_PX_PER_RAD (-180.722222f)

/* ---- 偏航角度环（^ 朝向对准） ---- */
#define PID_FLIGHT_YAW_ANGLE_KP (5.6f)
#define PID_FLIGHT_YAW_ANGLE_KI (0.0f)
#define PID_FLIGHT_YAW_ANGLE_KD (0.0f)
#define PID_FLIGHT_YAW_ANGLE_INTEGRAL_MAX (1.0f)
#define PID_FLIGHT_YAW_ANGLE_DLPF_ALPHA (1.0f)

#define PID_FLIGHT_YAW_RATE_CMD_LIMIT (3.0f)

/* ---- 运动前馈接口（车辆运动指令 -> 飞机从动） ---- */
#define FLIGHT_MOTION_VEL_FF_VX_TO_FWD (0.396000 117f)
#define FLIGHT_MOTION_VEL_FF_VY_TO_RIGHT (0.396000 117f)
/* core0 物理右向速度目标与外部 MOTION vy 语义桥接；协议 vy 仍保持右向为正。 */
#define FLIGHT_MOTION_VEL_FF_VY_TO_RIGHT_SIGN (-1.0f)
#define FLIGHT_MOTION_FF_VX_TO_PITCH (0.0294 999 965f)
#define FLIGHT_MOTION_FF_VY_TO_ROLL (0.0294 999 965f)
#define FLIGHT_MOTION_FF_WZ_TO_YAW (0.456f)
#define FLIGHT_MOTION_FF_VX_DIFF_LIMIT_MPS2 (10.0f)
#define FLIGHT_MOTION_FF_VY_DIFF_LIMIT_MPS2 (10.0f)
#define FLIGHT_MOTION_FF_VX_DIFF_LPF_ALPHA (0.1f)
#define FLIGHT_MOTION_FF_VY_DIFF_LPF_ALPHA (0.1f)
/* 底盘转心 -> 飞机期望投影的偏置，前/右为正，wz 偏航时生成切向速度前馈。 */
#define FLIGHT_MOTION_YAW_ORBIT_OFFSET_FWD_M (0.0f)
#define FLIGHT_MOTION_YAW_ORBIT_OFFSET_RIGHT_M (0.0f)
#define FLIGHT_MOTION_FF_DIFF_DT_MIN_MS (5u)
#define FLIGHT_LPF_ALPHA_MIN (0.001f)
#define FLIGHT_LPF_ALPHA_MAX (1.0f)

#if (PID_FLIGHT_KF_INVALID_FREEZE_TICKS * 9u < PID_FLIGHT_KF_INVALID_FREEZE_MS) || \
    ((PID_FLIGHT_KF_INVALID_FREEZE_TICKS - 1u) * 9u >= PID_FLIGHT_KF_INVALID_FREEZE_MS)
#error "PID_FLIGHT_KF_INVALID_FREEZE_TICKS must equal ceil(FREEZE_MS / 9ms)"
#endif

#endif /* _CONFIG_FLIGHT_CONTROL_DEFAULTS_H_ */
```

core0_cyt4bb7/project/code/config/configFlightSequence.h

SHA-256 : 870ae6549c7ef1f02ca8d809cb4ee822304d7b6b9ca00bc6cae54a66092e6f95

代码 (c)

```
#ifndef _CONFIG_FLIGHT_SEQUENCE_H_
#define _CONFIG_FLIGHT_SEQUENCE_H_

/*
 * core0 自动起飞与断联缓降时序默认值。
 *
 * 这些值不改变 ISR 调度，当前不进 flightParams，可后续再菜单化。
 */

#define TAKEOFF_WAIT_MS (5000u)
#define TAKEOFF_RAMP_MS (2000u)
#define TAKEOFF_APEX_BEEP_MS (800u)
#define TAKEOFF_CRUISE_BEEP_MS (3000u)

#define LAND_DESCEND_RATE_MM_S (200.0f)
#define LAND_TOUCHDOWN_MM (100.0f)
#define LAND_FLOOR_MM (0.0f)

#endif /* _CONFIG_FLIGHT_SEQUENCE_H_ */
```

core0_cyt4bb7/project/code/config/configFlightTiming.h

SHA-256 :
17e595ba144325ef23b37653884e1484c409fb4018eb2357f3f010b0799d2306

代码 (c)

```
#ifndef _CONFIG_FLIGHT_TIMING_H_
#define _CONFIG_FLIGHT_TIMING_H_

/*
 * core0 飞控调度编译期锁定项。
 *
 * 这些宏与 ISR 分频、控制环节拍绑定，不进入 flightParams、menu 或 flash；
 * 改动前须同步检查所有派生周期。
 */

#define PID_FLIGHT_CASCADE_DIV    (3u)          /* 相邻级联控制环统一三分频 */
#define PID_FLIGHT_ANGLE_LOOP_DIV  (PID_FLIGHT_CASCADE_DIV) /* 角速度内环 -> 角度外环 (1ms*3=3ms) */
#define PID_FLIGHT_POS_VEL_LOOP_DIV (PID_FLIGHT_CASCADE_DIV) /* 姿态角环 -> 物理速度环 (3ms*3=9ms) */
#define PID_FLIGHT_POS_LOOP_DIV    (PID_FLIGHT_CASCADE_DIV) /* 物理速度环 -> 视觉位置环 (9ms*3=27ms) */
#define PID_FLIGHT_YAW_ANGLE_LOOP_DIV (PID_FLIGHT_CASCADE_DIV) /* 偏航角速度内环 -> 偏航角度环 (1ms*3=3ms) */
#define ALTITUDE_HOLD_OUTER_DIV    (3u)          /* 定高速度环 -> 位置环分频 */

#endif /* _CONFIG_FLIGHT_TIMING_H_ */
```

core0_cyt4bb7/project/code/config/configSerialDebug.h

SHA-256 :
05082182ee930f412c2c2af6abb81ffc03990535805a7943a60c3e0041a3c157

代码 (c)

```
#ifndef _CONFIG_SERIAL_DEBUG_H_
#define _CONFIG_SERIAL_DEBUG_H_

/* 逐飞底层断言/异常文本串口配置；不承载项目级周期调试流。 */

/* 默认 debug UART: 若改串口且保留中断接收, 需同步更换 zf_common_debug 的中断 handler。 */
#define SERIAL_DEBUG_UART_INDEX      (UART_0)
#define SERIAL_DEBUG_UART_BAUDRATE   (1 000 000)
#define SERIAL_DEBUG_UART_TX_PIN     (UART0_TX_P00_1)
#define SERIAL_DEBUG_UART_RX_PIN     (UART0_RX_P00_0)
#define SERIAL_DEBUG_UART_USE_INTERRUPT (1)

/* 数值可比较的编译期保护；UART 枚举不在预处理期比较。 */
#if (SERIAL_DEBUG_UART_BAUDRATE < 1)
#error "SERIAL_DEBUG_UART_BAUDRATE must be > 0"
#endif

#endif /* _CONFIG_SERIAL_DEBUG_H_ */
```

core0_cyt4bb7/project/code/config/configVelocityEstimator.h

SHA-256 :
604679a29d0428804ee952df65e8f38a291089801da8ada8cbc6006181a093d7

代码 (c)

```
#ifndef _CONFIG_VELOCITY_ESTIMATOR_H_
#define _CONFIG_VELOCITY_ESTIMATOR_H_

/*
 * core0 水平速度估计器编译期参数。
 *
 * 本头全部宏与 IMU + 光流双轴 2 状态 KF 绑定，不进入 menu/flash。
 */
```

```
*/

#define VELOCITY_ESTIMATOR_PREDICT_DT_S      (0.001f)
#define VELOCITY_ESTIMATOR_GRAVITY_MPS2      (9.81f)
#define VELOCITY_ESTIMATOR_ACCEL_LIMIT_MPS2   (30.0f)
#define VELOCITY_ESTIMATOR_VELOCITY_LIMIT_MPS (15.0f)

#define VELOCITY_ESTIMATOR_FLOW_DELAY_MS      (18u)
#define VELOCITY_ESTIMATOR_VEL_HIST_LEN       (40u)

#define VELOCITY_ESTIMATOR_BIAS_CALIBRATION_SAMPLES (2000u)
#define VELOCITY_ESTIMATOR_BIAS_LIMIT_MPS2        (3.0f)
#define VELOCITY_ESTIMATOR_CALIB_MOTION_MPS2      (1.5f)

#define VELOCITY_ESTIMATOR_KF_Q_VEL (2.0e-8f)
#define VELOCITY_ESTIMATOR_KF_Q_BIAS (2.0e-7f)

#define VELOCITY_ESTIMATOR_KF_R_BASE      (0.0001558f)
#define VELOCITY_ESTIMATOR_KF_R_HEIGHT_COEFF (0.0009055f)

#define VELOCITY_ESTIMATOR_KF_P_INIT_VEL   (1.0f)
#define VELOCITY_ESTIMATOR_KF_P_INIT_BIAS   (0.25f)
#define VELOCITY_ESTIMATOR_KF_P_MIN         (1.0e-8f)
#define VELOCITY_ESTIMATOR_KF_P_MAX         (100.0f)
#define VELOCITY_ESTIMATOR_KF_GAIN_VEL_MAX  (1.0f)
#define VELOCITY_ESTIMATOR_KF_GAIN_BIAS_MAX (5.0f)

#define VELOCITY_ESTIMATOR_INNOVATION_LIMIT_MPS (5.0f)

#define VELOCITY_ESTIMATOR_FLOW_SCALE_X_RAD_PER_COUNT (0.000105f)
#define VELOCITY_ESTIMATOR_FLOW_SCALE_Y_RAD_PER_COUNT (0.000105f)

#define VELOCITY_ESTIMATOR_GYRO_COMP_X (-1.0f)
#define VELOCITY_ESTIMATOR_GYRO_COMP_Y (1.0f)

#define VELOCITY_ESTIMATOR_FLOW_LEVER_X_M (-0.115f)
#define VELOCITY_ESTIMATOR_FLOW_LEVER_Y_M (0.0f)

#define VELOCITY_ESTIMATOR_FLOW_DT_MIN_US (10 000u)
#define VELOCITY_ESTIMATOR_FLOW_DT_MAX_US (40 000u)
#define VELOCITY_ESTIMATOR_FLOW_TIMEOUT_MS (100u)
#define VELOCITY_ESTIMATOR_HEIGHT_MIN_M   (0.0f)
#define VELOCITY_ESTIMATOR_HEIGHT_MAX_M   (3.50f)
#define VELOCITY_ESTIMATOR_TILT_LIMIT_RAD  (1.1345f)

#define VELOCITY_ESTIMATOR_BODY_TO_SENSOR_XX (0.0f)
#define VELOCITY_ESTIMATOR_BODY_TO_SENSOR_XY (-1.0f)
#define VELOCITY_ESTIMATOR_BODY_TO_SENSOR_YX (-1.0f)
#define VELOCITY_ESTIMATOR_BODY_TO_SENSOR_YY (0.0f)

#if (VELOCITY_ESTIMATOR_BIAS_CALIBRATION_SAMPLES == 0u) || \
    (VELOCITY_ESTIMATOR_BIAS_CALIBRATION_SAMPLES > 65 535u)
#error "VELOCITY_ESTIMATOR_BIAS_CALIBRATION_SAMPLES must be in [1,65535]"
#endif
#if (VELOCITY_ESTIMATOR_FLOW_DT_MIN_US == 0u) || \
    (VELOCITY_ESTIMATOR_FLOW_DT_MAX_US <= VELOCITY_ESTIMATOR_FLOW_DT_MIN_US)
#error "invalid velocity estimator optical-flow dt range"
#endif
#if (VELOCITY_ESTIMATOR_FLOW_DELAY_MS >= VELOCITY_ESTIMATOR_VEL_HIST_LEN)
#error "VELOCITY_ESTIMATOR_FLOW_DELAY_MS must be < VEL_HIST_LEN"
#endif
#endif /* _CONFIG_VELOCITY_ESTIMATOR_H_ */
```

core0_cyt4bb7/project/code/config/configVision.h

SHA-256 :
fa6a04409d80968b4fe36c451058620cd26828f38ee1f3862264a4e1042ef750

代码 (c)

```
#ifndef _CONFIG_VISION_H_
#define _CONFIG_VISION_H_

/*
```



```

* core0 cm_7_1 视觉调参中心（默认值由 menu.c 加载，运行时可改 + flash 持久化）
*
* 图像坐标系：MT9V03X 188×120 灰度，x∈[0,187]，y∈[0,119]。
* 误差定义（写入共享内存供 cm_7_0 飞控位置环/偏航环消费）：
* visionErrX = VISION_ERR_X_SIGN * (centerX - VISION_TARGET_CENTER_X)
* visionErrY = VISION_ERR_Y_SIGN * (centerY - VISION_TARGET_CENTER_Y)
* visionYawErr = VISION_YAW_ERR_SIGN * wrapToPi(orientation - VISION_TARGET_ANGLE_RAD)
*
* 注意：底层轮廓跟踪/朝向滤波参数（OM_*）在 openMorph_config.h，本文件只放
* 应用级可调项（目标中点、角度中点、极性、DetectMode 与检测阈值默认值）。
* 屏显策略由菜单模块固定为飞行中持续刷新，无需编译期屏幕开关宏。
*/

/* =====
* 1. 图像目标中点（飞机要把 ^ 顶到此像素点正上方）
* 默认值为当前上板整定中点。
* ===== */
#define VISION_TARGET_CENTER_X (80) /* 0.187 */
#define VISION_TARGET_CENTER_Y (70) /* 0.119 */

/* =====
* 2. 角度中点（^ 期望朝向参考角，rad）
* 屏幕 y 向下，当前实车装配参考方向 = -PI/2
* yawErr = orientation - 本值
* ===== */
#define VISION_TARGET_ANGLE_RAD (-1.5707963f) /* -PI/2 */

/* =====
* 3. 误差极性（首飞前上调试架标定，确保闭环负反馈）
* 极性判据（屏幕上端=机体前、右端=机体右）：
* errX = SIGN-(cx-中点)，目标偏右(cx>中点) 须令飞机右飞才能顶回 → 位置环须输出向右速度目标(>0)；
* 位置环 PID expected=0 → 目标速度 = -Kp·errX，故须 errX<0 ⇒ X_SIGN=-1。
* errY = SIGN-(cy-中点)，目标偏下(屏下=机体后) 须令飞机后飞 → 前向速度目标<0 = -Kp·errY 须<0
* ⇒ errY>0 时成立 ⇒ Y_SIGN=+1。
* ===== */
#define VISION_ERR_X_SIGN (-1.0f) /* X(右向)：目标偏右须右飞，位置环负号要求 errX<0，故取 -1 */
#define VISION_ERR_Y_SIGN (1.0f) /* Y(前向)：目标偏后须后飞，+1 已满足闭环负反馈 */
#define VISION_YAW_ERR_SIGN (1.0f)

/* =====
* 4. 目标检测模式与阈值（菜单可调 + flash 持久化）
* ===== */
#define VISION_DETECT_MODE_BINARYCC (0u)
#define VISION_DETECT_MODE_LEGACY_CANNY (1u)
#define VISION_DETECT_MODE_DEFAULT VISION_DETECT_MODE_BINARYCC

#define VISION_CANNY_LOW_DEFAULT (170u)
#define VISION_CANNY_HIGH_DEFAULT (220u)
#define VISION_CANNY_THRESHOLD_MIN (5u)
#define VISION_CANNY_THRESHOLD_MAX (500u)
#define VISION_CANNY_THRESHOLD_GAP (5u)
#define VISION_CANNY_STEP_SHORT (1)
#define VISION_CANNY_STEP_LONG (10)

#define VISION_BINARYCC_LOW_DEFAULT (210u)
#define VISION_BINARYCC_HIGH_DEFAULT (255u)
#define VISION_BINARYCC_THRESHOLD_MIN (0u)
#define VISION_BINARYCC_THRESHOLD_MAX (255u)
#define VISION_BINARYCC_STEP_SHORT (1)
#define VISION_BINARYCC_STEP_LONG (10)
#define VISION_BINARYCC_AREA_MIN_DEFAULT (15u)
#define VISION_BINARYCC_AREA_MAX_DEFAULT (3000u)
#define VISION_BINARYCC_BRIGHT_MIN_DEFAULT (70u)
#define VISION_BINARYCC_ECC_MAX_DEFAULT (5.0f)
#define VISION_BINARYCC_FILL_MIN_DEFAULT (0.05f)
#define VISION_BINARYCC_HU_MAX_DEFAULT (32.0f)
#define VISION_BINARYCC_ORI_BRIGHT_MIN_DEFAULT (100u)
#define VISION_BINARYCC_ORI_BORDER_MARGIN_DEFAULT (0u)
#define VISION_BINARYCC_ORI_CONF_MIN_DEFAULT (0.05f)
#define VISION_BINARYCC_ORI_HOLD_FRAMES_DEFAULT (5u)

/* =====
* 4.5 相机曝光（MT9V03X set_exposure_time 行单位，菜单可调 + flash 持久化）
* 默认值随上板调参基线维护；同版本合法 Flash 仍优先，只有空白/损坏或恢复默认采用本值。
* ===== */
#define VISION_EXPOSURE_DEFAULT (51u)
#define VISION_EXPOSURE_MIN (1u)
#define VISION_EXPOSURE_MAX (2000u)
#define VISION_EXPOSURE_STEP_SHORT (1.0f)

```

```
#define VISION_EXPOSURE_STEP_LONG (10.0f)

/* =====
* 5. 外环误差施密特(滞回)死区：抑制目标近中心抖动（菜单可调 + flash 持久化）
* 判据：质心到目标中点的合成半径  $r = \sqrt{(dx^2 + dy^2)}$ ，与极性无关。
* 双圆滞回（大圆直径 > 小圆直径，形成回差）：
*   · 低状态(已到位/误差归零)：r 超过大圆(进入阈值) → 转高状态，误差正常输出；否则归零。
*   · 高状态(跟踪中)：r 回到小圆(退出阈值)内 → 转低状态、误差归零；否则正常输出。
* 初始低状态：刚锁定要求先明显偏离(超过大圆)才驱动平移，回到小圆内即判到位归零。
* 边界以图像高 H 的百分比直径表示（半径 =  $0.5 \times \text{pct}/100 \times H$ ）。
* ===== */
#define VISION_DEADZONE_ENTER_PCT_DEFAULT (40.0f) /* 施密特大圆(进入,转高)直径占图像高% */
#define VISION_DEADZONE_EXIT_PCT_DEFAULT (20.0f) /* 施密特小圆(退出,归零)直径占图像高%；须 < enter */

/* =====
* 6. 目标丢失冻结 + 外推定位（菜单可调 + flash 持久化）
* 目标丢失后先冻结最后一次已发布 X/Y 输出误差，超过冻结时间再分类处理。
* 若最后有效质心在 exitR 内，误差归零悬停；否则沿*目标中点 C → 最后有效质心 P*射线方向，
* 取该射线与圆心 C、半径=直径/2 的圆的交点作为虚拟靶点，据此算误差驱动飞机朝最后视向平移去找回目标。
* 方向有意义判据：|P-C| ≥ 施密特小圆退出半径 exitR；否则（中心附近丢失，方向是噪声）误差归零悬停。
* 外推为虚拟靶点，仅用于算方向×半径的误差，不采样像素，故直径可超图像尺寸。
* 丢失外推上限由 cm_7_0 的 TARGET_LOST_HOLD_MS(3s) 兜底：超时归零悬停。
* ===== */
#define VISION_LOST_FREEZE_MS_DEFAULT (400.0f) /* 目标丢失后冻结最后输出误差的时间 ms */
#define VISION_LOST_EXTRAP_DIAMETER_PX_DEFAULT (148.0f) /* 外推圆直径 px（半径=直径/2，丢失期沿最后视向投影到此圆周） */

#endif /* _CONFIG_VISION_H_ */
```

core0_cyt4bb7/project/code/config/pinMap.h

SHA-256

:

d394679dde60d5db7890331bbc82f807cd69d4893b6a6fa23da36511e23512d2

代码 (c)

```
#ifndef _PIN_MAP_H_
#define _PIN_MAP_H_

/*
* core0 全工程自定义引脚集中管理（纯编译期 #define，零依赖）。
* 右值为逆飞 GPIO/外设复用枚举（TCPWM/SPI/UART 通道名或裸 Pxx_y），
* 仅在各消费.c 的预处理期展开；本头不 include 任何驱动头，
* token 的实际定义由消费方自带的 zf_driver_*.h 头提供。
*/

/* =====
* 电机 ESC PWM（cm_7_0）
* 实际物理方位映射：P5.0/P5.1/P5.2/P5.3 = BL/BR/FL/FR
* ===== */
#define ESC_CH_PHYSICAL_BL TCPWM_CH09_P05_0
#define ESC_CH_PHYSICAL_BR TCPWM_CH10_P05_1
#define ESC_CH_PHYSICAL_FL TCPWM_CH11_P05_2
#define ESC_CH_PHYSICAL_FR TCPWM_CH12_P05_3

/* =====
* UPixels 光流串口（cm_7_0，UART_1，19200 8N1，50Hz）
* 光流模块 TX 接 MCU RX = P04_0；MCU TX = P04_1 仅占位可不接。
* ===== */
#define OPTICAL_FLOW_UART_TX_PIN (UART1_TX_P04_1)
#define OPTICAL_FLOW_UART_RX_PIN (UART1_RX_P04_0)

/* =====
* 核间链路串口（cm_7_0，UART_2，P10 ↔ core1 P04/UART_1，4 Mbps）
* 单线双向(group0): core0 发按键事件/心跳, 收 core1 心跳/状态。
* 远程菜单系统用(core0 按键透传给 core1)。group1(P14) 不用于核间。
* ===== */
#define CROSS_CORE_UART_TX_PIN (UART2_TX_P10_1)
#define CROSS_CORE_UART_RX_PIN (UART2_RX_P10_0)

/* =====
* ToF 硬件 SPI 总线（cm_7_0，VL53L8CX，SPI_3）
* 仅 ALT_TOF_L8CX_COMM_SELECT == HARD_SPI 时由 tofBus.c 引用
* ===== */
#define TOF_SPI_SCK_PIN (SPI3_CLK_P03_2)
#define TOF_SPI_MOSI_PIN (SPI3_MOSI_P03_1)
```

```

#define TOF_SPI_MISO_PIN (SPI3_MISO_P03_0)
#define TOF_SPI_NCS_PIN (P03_3)

/* =====
 * ToF 软件 IIC 总线 (cm_7_0, 位翻)
 * 硬件 SPI 未选中时由 tofBus.c 引用
 * ===== */
#define TOF_IIC_SDA_PIN (P19_0)
#define TOF_IIC_SCL_PIN (P19_1)

/* =====
 * VL53L1X 控制脚 (数据就绪 INT / 复位 XSHUT)
 * VL53L8CX 控制脚 (数据就绪 INT / 复位 XSHUT)
 * IAR 工程会同时编译两个驱动源, 但上层由 ALT_TOF_SENSOR_SELECT 在 init 时
 * 只绑定一种运行路径。INT/SHUT 共用同一物理引脚 (P07_2/P07_3), 保留独立
 * 命名防误用——勿合并为单一宏。
 * ===== */
#define VL53L1X_IIC_INT_PIN (P07_2)
#define VL53L1X_IIC_SHUT_PIN (P07_3)
#define VL53LMZ_INT_PIN (P07_2)
#define VL53LMZ_SHUT_PIN (P07_3)

/* =====
 * 菜单按键 (cm_7_1, 5 路 GPI 上拉)
 * ===== */
#define MENU_KEY1_PIN (P20_3)
#define MENU_KEY2_PIN (P20_2)
#define MENU_KEY3_PIN (P20_1)
#define MENU_KEY4_PIN (P20_0)
#define MENU_KEY5_PIN (P19_4)

#endif /* _PIN_MAP_H_ */

```

core0_cyt4bb7/project/code/flight/altitudeHold.c

SHA-256

:

4fc30db8102b8cb05fed2af945aed383957eaeff1afe21cb3743ea90d68d539f

代码 (c)

```

#include "altitudeHold.h"

#include "pidFlightController.h"
#include "altitudeTof.h"
#include "PID.h"
#include "commonDef.h"
#include "configFlightTiming.h"
#include "flightParams.h"
#include "modeFlight.h"
#include "zf_common_interrupt.h"

typedef struct
{
    flightPidParamStruct posPid;
    flightPidParamStruct velPid;
    float targetMm;
    float velCmdLimit;
    float velFeedforward;
    float offsetLimitPct;
    float baseThrottlePct;
    float throttleMinPct;
} altitudeHoldConfigStruct;

typedef struct
{
    volatile uint32_t seq;
    volatile uint16_t distanceMm;
    volatile float outputPct;
    PID posPid;
    PID velPid;
    uint8_t outerDivCount;
    uint8_t wasInClosedLoop;
    uint8_t controlEnabled;
    uint8_t failsafeStage;
    uint8_t failsafeRequestDisarm;
}

```

```

uint32_t faultStartMs;
uint32_t failsafeLastMs;
uint16_t faultAgeMs;
float currentTarget;
float velTarget;
altitudeHoldConfigStruct config;
} altitudeHoldContextStruct;

static altitudeHoldContextStruct altitudeContext;

#define altitudeHold    altitudeContext
#define altPosPid      altitudeContext.posPid
#define altVelPid      altitudeContext.velPid
#define outerDivCount  altitudeContext.outerDivCount
#define wasInClosedLoop altitudeContext.wasInClosedLoop
#define holdControlEnabled altitudeContext.holdControlEnabled
#define currentTarget  altitudeContext.currentTarget
#define velTarget      altitudeContext.velTarget
#define altitudeConfig altitudeContext.config

#define ALTITUDE_FAILSAFE_HOLD_MS    (3000u)
#define ALTITUDE_FAILSAFE_RAMP_PCT_PER_S (2.0f)
#define ALTITUDE_FAILSAFE_MAX_STEP_MS (100u)

static void loadConfig(void)
{
    flightParamsStruct params;
    flightParamsGetActive(&params);
    altitudeConfig.posPid = params.altPos;
    altitudeConfig.velPid = params.altVel;
    altitudeConfig.targetMm = params.altTargetMm;
    altitudeConfig.velCmdLimit = params.altVelCmdLimit;
    altitudeConfig.velFeedforward = params.altVelFF;
    altitudeConfig.offsetLimitPct = params.altOffsetLimitPct;
    altitudeConfig.baseThrottlePct = params.baseThrottlePct;
    altitudeConfig.throttleMinPct = params.throttleMinPct;
}

static void resetPids(float target)
{
    currentTarget = target;
    pidInitFromParams(&altPosPid, &altitudeConfig.posPid, target);
    pidInitFromParams(&altVelPid, &altitudeConfig.velPid, 0.0f);
    velTarget = 0.0f;
}

static void clearRuntimeOutput(void)
{
    wasInClosedLoop = 0u;
    outerDivCount = 0u;
    velTarget = 0.0f;
    altitudeHold.outputPct = 0.0f;
    pidFlightControllerSetAltitudeOffset(0.0f);
}

uint8_t altitudeHoldInit(void)
{
    uint8_t tofReady;
    loadConfig();
    /* facade 绑定芯片并初始化; 链路/测距健康态由 facade 维护, 本层不再持有。 */
    tofReady = altitudeToFInit();

    resetPids(altitudeConfig.targetMm);

    altitudeHold.distanceMm = 0u;
    altitudeHold.outputPct = 0.0f;
    outerDivCount = 0u;
    wasInClosedLoop = 0u;
    holdControlEnabled = 1u;
    altitudeContext.failsafeStage = ALTITUDE_FAILSAFE_NORMAL;
    altitudeContext.failsafeRequestDisarm = 0u;
    altitudeContext.faultStartMs = 0u;
    altitudeContext.failsafeLastMs = 0u;
    altitudeContext.faultAgeMs = 0u;

    pidFlightControllerSetAltitudeOffset(0.0f);
    return tofReady;
}

```

```

void altitudeHoldTickMs(void)
{
    altitudeTofTickMs();
}

void altitudeHoldUpdate(uint32_t nowMs)
{
    float heightMm = 0.0f;
    float velFiltered = 0.0f;
    uint8_t fresh;

    /* facade 完成取帧/姿态补偿/范围/门控/速度低通；本层只取结果。 */
    altitudeTofUpdate();
    fresh = tofGetHeight(&heightMm, &velFiltered);
    altitudeHold.distanceMm = (uint16_t) heightMm; /* 遥测/调试用，始终更新 */
    altitudeHold.seq++;

    /* 安全状态门控：未发车 / 姿态保护停机时不输出，下次发车从当前高度起算。 */
    {
        flightSafetySnapshotStruct safety;
        modeFlightGetSnapshot(&safety);
        if (!holdControlEnabled || safety.state != SAFETY_STATE_CLOSED_LOOP)
        {
            altitudeContext.failsafeStage = ALTITUDE_FAILSAFE_NORMAL;
            altitudeContext.failsafeRequestDisarm = 0u;
            altitudeContext.faultAgeMs = 0u;
            clearRuntimeOutput();
            return;
        }
    }

    /* 冻结期（无新鲜健康帧）：保持上一拍 altitudeOffsetPct，速度+高度均沿用 last-valid，
     * 不推进 PID（积分天然冻结），避免油门偏移骤降致高度剧变。 */
    if (!fresh)
    {
        uint32_t dtMs;
        float lowerOffset;
        if (altitudeTofIsHealthy() != 0u) return;
        if (altitudeContext.failsafeStage == ALTITUDE_FAILSAFE_NORMAL)
        {
            altitudeContext.failsafeStage = ALTITUDE_FAILSAFE_HOLD;
            altitudeContext.faultStartMs = nowMs;
            altitudeContext.failsafeLastMs = nowMs;
        }
        {
            uint32_t age = (uint32_t)(nowMs - altitudeContext.faultStartMs);
            altitudeContext.faultAgeMs = (age > 0xFFFFu) ? 0xFFFFu : (uint16_t)age;
        }
        if ((altitudeContext.failsafeStage == ALTITUDE_FAILSAFE_HOLD) &&
            ((uint32_t)(nowMs - altitudeContext.faultStartMs) >= ALTITUDE_FAILSAFE_HOLD_MS))
        {
            altitudeContext.failsafeStage = ALTITUDE_FAILSAFE_RAMP;
            altitudeContext.failsafeLastMs = nowMs;
        }
        if (altitudeContext.failsafeStage != ALTITUDE_FAILSAFE_RAMP) return;

        dtMs = (uint32_t)(nowMs - altitudeContext.failsafeLastMs);
        if (dtMs > ALTITUDE_FAILSAFE_MAX_STEP_MS) dtMs = ALTITUDE_FAILSAFE_MAX_STEP_MS;
        altitudeContext.failsafeLastMs = nowMs;
        lowerOffset = altitudeConfig.throttleMinPct - altitudeConfig.baseThrottlePct;
        altitudeHold.outputPct -= ALTITUDE_FAILSAFE_RAMP_PCT_PER_S * ((float)dtMs / 1000.0f);
        if (altitudeHold.outputPct <= lowerOffset)
        {
            altitudeHold.outputPct = lowerOffset;
            altitudeContext.failsafeRequestDisarm = 1u;
        }
        pidFlightControllerSetAltitudeOffset(altitudeHold.outputPct);
        return;
    }

    if (altitudeContext.failsafeStage != ALTITUDE_FAILSAFE_NORMAL)
    {
        altitudeContext.failsafeStage = ALTITUDE_FAILSAFE_NORMAL;
        altitudeContext.failsafeRequestDisarm = 0u;
        altitudeContext.faultAgeMs = 0u;
        resetPids(heightMm);
        wasInClosedLoop = 1u;
        outerDivCount = 0u;
    }
}

```

```

if (!wasInClosedLoop)
{
    resetPids(currentTarget);
    wasInClosedLoop = 1u;
    outerDivCount = 0u;
}

/* 外环分频：每 ALTITUDE_HOLD_OUTER_DIV 拍跑一次位置环。 */
outerDivCount++;
if (outerDivCount >= ALTITUDE_HOLD_OUTER_DIV)
{
    outerDivCount = 0u;
    altPosPid.expectedValue = currentTarget;
    pidProcess(&altPosPid, heightMm);
    velTarget = clampf(altPosPid.controlValue,
        -altitudeConfig.velCmdLimit, altitudeConfig.velCmdLimit);
}

/* 内环每拍 */
altVelPid.expectedValue = velTarget;
pidProcess(&altVelPid, velFiltered);
altitudeHold.outputPct = clampf(altVelPid.controlValue + altitudeConfig.velFeedforward * velTarget,
    -altitudeConfig.offsetLimitPct, altitudeConfig.offsetLimitPct);

pidFlightControllerSetAltitudeOffset(altitudeHold.outputPct);
}

uint8_t altitudeHoldIsHealthy(void)
{
    return altitudeTofIsHealthy();
}

void altitudeHoldGetSnapshot(altitudeHoldSnapshotStruct *out)
{
    uint32_t primask;
    if (out == 0) return;
    primask = __get_PRIMASK();
    __disable_irq();
    out->seq = altitudeHold.seq;
    out->distanceMm = altitudeHold.distanceMm;
    out->outputPct = altitudeHold.outputPct;
    out->controlEnabled = holdControlEnabled;
    out->healthy = altitudeTofIsHealthy();
    out->failsafeStage = altitudeContext.failsafeStage;
    out->requestDisarm = altitudeContext.failsafeRequestDisarm;
    out->faultAgeMs = altitudeContext.faultAgeMs;
    __set_PRIMASK(primask);
}

/* 外部目标覆盖：自动起飞时序每拍喂目标，替代直接读 flightParams.altTargetMm。
 * 与 altitudeHoldUpdate 同在前台上下文（无 ISR 竞争）；只写 currentTarget，
 * 由 update 的位置外环分支(line: altPosPid.expectedValue = currentTarget)采用。 */
void altitudeHoldSetTargetMm(float targetMm)
{
    currentTarget = targetMm;
}

void altitudeHoldSetControlEnabled(uint8_t enabled)
{
    holdControlEnabled = enabled ? 1u : 0u;
    if (!holdControlEnabled)
    {
        clearRuntimeOutput();
    }
}

/* 参数热重载：用 flightParams 最新值重置定高 PID/目标高度，并级联刷新 facade 滤波参数。
 * 仅在非 CLOSED_LOOP 态由前台调用（与 pidFlightControllerApplyParams 同时），
 * 保证菜单改 Target mm / 定高增益后，下一次发车按新目标与新增益飞行。 */
void altitudeHoldApplyParams(void)
{
    loadConfig();
    resetPids(altitudeConfig.targetMm);
    altitudeTofApplyParams();
    holdControlEnabled = 1u;
    clearRuntimeOutput();
}

```

core0_cyt4bb7/project/code/flight/altitudeHold.h

SHA-256

:

da257dd4ee3eb002bae71facb265437b173e3bf7f11c8e98b29dfd0b633fb634

代码 (c)

```
#ifndef _ALTITUDE_HOLD_H_
#define _ALTITUDE_HOLD_H_

#include <stdint.h>

/*
 * core0 ToF 定高双环闭环本体（数据加工已下沉到 altitudeTof facade）：
 * 外环：高度位置 PID（内环三分频），输入 = facade 垂直高度，输出 = 目标速度
 * 内环：每个有效测量执行，输入 = facade 速度低通，输出 = 油门偏移 pp
 * 本层只保留：安全态门控 + 位置环 PID + 速度环 PID + 输出限幅 + 写 altitudeOffsetPct。
 * 姿态补偿 / 范围检查 / 时间域门控 / 无效冻结 / 速度低通 / 链路看门狗 / 健康态
 * 全部在 altitudeTof facade 完成，本层经 tofGetHeight() 一次性取高度+速度低通。
 */

typedef struct
{
    uint32_t seq;
    uint16_t distanceMm; /* facade 当前垂直高度（遥测/调试镜像） */
    uint8_t healthy;
    uint8_t controlEnabled;
    uint8_t failsafeStage;
    uint8_t requestDisarm;
    uint16_t faultAgeMs;
    float outputPct; /* 定高 PID 输出（pp，镜像 altitudeOffsetPct，调试用） */
} altitudeHoldSnapshotStruct;

typedef enum
{
    ALTITUDE_FAILSAFE_NORMAL = 0u,
    ALTITUDE_FAILSAFE_HOLD = 1u,
    ALTITUDE_FAILSAFE_RAMP = 2u
} altitudeFailsafeStageEnum;

uint8_t altitudeHoldInit(void); /* 1=ToF facade 初始化成功，0=失败；控制层仍保持安全失效态 */
void altitudeHoldUpdate(uint32_t nowMs);

/* 外部目标高度覆盖（mm）：自动起飞时序每拍喂目标，替代直接读 flightParams.altTargetMm。
 * 仅写缓存 currentTarget；下一次位置外环拍（altitudeHoldUpdate 的 OUTER_DIV 分支）据此更新
 * altPosPid.expectedValue。非 CLOSED_LOOP 态写入无副作用（update 早返回）。前台调用，非 ISR。 */
void altitudeHoldSetTargetMm(float targetMm);

/* 自动起飞 WAIT 期门控：0=只更新 ToF 遥测，定高 PID 不推进且输出清零。 */
void altitudeHoldSetControlEnabled(uint8_t enabled);

/* ToF 失活看门狗 + 轮询节流计时：1ms ISR 周期调用，转发到 altitudeTof facade。 */
void altitudeHoldTickMs(void);

/* 参数热重载：重置定高 PID/目标高度并级联刷新 facade 滤波参数（非 CLOSED_LOOP 态调用）。 */
void altitudeHoldApplyParams(void);

/* ToF 是否健康（发车门槛）：转发 altitudeTofIsHealthy()
 * = 链路活 且 测距可信 且 门控未悬挂 且 已有有效基线。安全状态机据此拒绝发车。 */
uint8_t altitudeHoldIsHealthy(void);
void altitudeHoldGetSnapshot(altitudeHoldSnapshotStruct *out);

#endif /* _ALTITUDE_HOLD_H_ */
```

core0_cyt4bb7/project/code/flight/flightParams.c

SHA-256

:

00b1be3bb71e83080e93f793573d5547854f9acb663c6ed391c37a0b99e5720a

代码 (c)

```
#include "flightParams.h"

#include <stddef.h>
#include <string.h>

#include "commonDef.h"
#include "configAltitudeTof.h" /* ALT_TOF_MAX_SPEED_MM_S 作 altTofMaxSpeedMmS 默认来源 */
#include "configFlightControlDefaults.h"

/*
 * core0 飞控参数 RAM 镜像实现。
 * 活动参数由本模块私有持有；各 owner 只在 Init/ApplyParams 时缓存自己的窄配置。
 * cm_7_1 也编译本文件（共用 flightParamsLoadDefaults 给菜单初始化默认值）。
 */

static flightParamsStruct activeParams;
static uint32_t activeRevision;

typedef char flight_params_padding_prefix_layout_assert[
    ((offsetof(flightParamsStruct, padding) == 2u) &&
    (offsetof(flightParamsStruct, baseThrottlePct) == 4u)) ? 1 : -1];

void flightParamsLoadDefaults(flightParamsStruct* p)
{
    if (0 == p) return;

    p->visionPosErrEnable = FLIGHT_VISION_POS_ERR_ENABLE;
    p->systemMode = FLIGHT_SYSTEM_MODE_DEFAULT;
    p->padding[0] = 0u;
    p->padding[1] = 0u;

    /* 油门 */
    p->baseThrottlePct = PID_FLIGHT_BASE_THROTTLE_PCT;
    p->throttleMinPct = PID_FLIGHT_THROTTLE_PCT_MIN;
    p->throttleMaxPct = PID_FLIGHT_THROTTLE_PCT_MAX;

    /* 角度参考 */
    p->pitchRefRad = PID_FLIGHT_PITCH_REF_RAD;
    p->rollRefRad = PID_FLIGHT_ROLL_REF_RAD;

    /* 角度外环 PID */
    p->pitchAngle.kp = PID_FLIGHT_PITCH_ANGLE_KP;
    p->pitchAngle.ki = PID_FLIGHT_PITCH_ANGLE_KI;
    p->pitchAngle.kd = PID_FLIGHT_PITCH_ANGLE_KD;
    p->pitchAngle.integralMax = PID_FLIGHT_PITCH_ANGLE_INTEGRAL_MAX;
    p->pitchAngle.dlpfAlpha = PID_FLIGHT_PITCH_ANGLE_DLPF_ALPHA;

    p->rollAngle.kp = PID_FLIGHT_ROLL_ANGLE_KP;
    p->rollAngle.ki = PID_FLIGHT_ROLL_ANGLE_KI;
    p->rollAngle.kd = PID_FLIGHT_ROLL_ANGLE_KD;
    p->rollAngle.integralMax = PID_FLIGHT_ROLL_ANGLE_INTEGRAL_MAX;
    p->rollAngle.dlpfAlpha = PID_FLIGHT_ROLL_ANGLE_DLPF_ALPHA;

    p->rateCmdLimitPitch = PID_FLIGHT_RATE_CMD_LIMIT_PITCH;
    p->rateCmdLimitRoll = PID_FLIGHT_RATE_CMD_LIMIT_ROLL;

    /* 角速度内环 PID */
    p->pitchRate.kp = PID_FLIGHT_PITCH_RATE_KP;
    p->pitchRate.ki = PID_FLIGHT_PITCH_RATE_KI;
    p->pitchRate.kd = PID_FLIGHT_PITCH_RATE_KD;
    p->pitchRate.integralMax = PID_FLIGHT_PITCH_RATE_INTEGRAL_MAX;
    p->pitchRate.dlpfAlpha = PID_FLIGHT_PITCH_RATE_DLPF_ALPHA;

    p->rollRate.kp = PID_FLIGHT_ROLL_RATE_KP;
    p->rollRate.ki = PID_FLIGHT_ROLL_RATE_KI;
    p->rollRate.kd = PID_FLIGHT_ROLL_RATE_KD;
    p->rollRate.integralMax = PID_FLIGHT_ROLL_RATE_INTEGRAL_MAX;
    p->rollRate.dlpfAlpha = PID_FLIGHT_ROLL_RATE_DLPF_ALPHA;

    p->yawRate.kp = PID_FLIGHT_YAW_RATE_KP;
    p->yawRate.ki = PID_FLIGHT_YAW_RATE_KI;
    p->yawRate.kd = PID_FLIGHT_YAW_RATE_KD;
    p->yawRate.integralMax = PID_FLIGHT_YAW_RATE_INTEGRAL_MAX;
    p->yawRate.dlpfAlpha = PID_FLIGHT_YAW_RATE_DLPF_ALPHA;

    p->yawRateRefRadS = PID_FLIGHT_YAW_RATE_REF_RAD_S;
```



```

p->pitchRateFF = PID_FLIGHT_PITCH_RATE_FF;
p->rollRateFF = PID_FLIGHT_ROLL_RATE_FF;
p->ctrlLimitPctPitch = PID_FLIGHT_CTRL_LIMIT_PCT_PITCH;
p->ctrlLimitPctRoll = PID_FLIGHT_CTRL_LIMIT_PCT_ROLL;
p->ctrlLimitPctYaw = PID_FLIGHT_CTRL_LIMIT_PCT_YAW;

/* IMU 角速度 LPF */
p->gyroLpfAlpha = GYRO_LPF_ALPHA;

/* 定高双环 */
p->altTargetMm = (float) ALTITUDE_HOLD_TARGET_MM;
p->altDistMinMm = (float) ALTITUDE_HOLD_DIST_MIN_MM;
p->altDistMaxMm = (float) ALTITUDE_HOLD_DIST_MAX_MM;
p->altTimeoutMs = (float) ALTITUDE_HOLD_TIMEOUT_MS;
p->altVelLpfAlpha = ALTITUDE_HOLD_VEL_LPF_ALPHA;

p->altPos.kp = ALTITUDE_HOLD_POS_KP;
p->altPos.ki = ALTITUDE_HOLD_POS_KI;
p->altPos.kd = ALTITUDE_HOLD_POS_KD;
p->altPos.integralMax = ALTITUDE_HOLD_POS_INTEGRAL_MAX;
p->altPos.dlpfAlpha = ALTITUDE_HOLD_POS_DLPF_ALPHA;

p->altVel.kp = ALTITUDE_HOLD_VEL_KP;
p->altVel.ki = ALTITUDE_HOLD_VEL_KI;
p->altVel.kd = ALTITUDE_HOLD_VEL_KD;
p->altVel.integralMax = ALTITUDE_HOLD_VEL_INTEGRAL_MAX;
p->altVel.dlpfAlpha = ALTITUDE_HOLD_VEL_DLPF_ALPHA;

p->altVelFF = ALTITUDE_HOLD_VEL_FF;
p->altOffsetLimitPct = ALTITUDE_HOLD_OFFSET_LIMIT_PCT;
p->altVelCmdLimit = ALTITUDE_HOLD_VEL_CMD_LIMIT;

/* 姿态安全保护 */
p->postureSafePitchMinRad = POSTURE_SAFE_PITCH_MIN_RAD;
p->postureSafePitchMaxRad = POSTURE_SAFE_PITCH_MAX_RAD;
p->postureSafeRollMinRad = POSTURE_SAFE_ROLL_MIN_RAD;
p->postureSafeRollMaxRad = POSTURE_SAFE_ROLL_MAX_RAD;
p->postureSafeGyroMaxRadS = POSTURE_SAFE_GYRO_MAX_RAD_S;

/* 位置环 */
p->posX.kp = PID_FLIGHT_POS_X_KP;
p->posX.ki = PID_FLIGHT_POS_X_KI;
p->posX.kd = PID_FLIGHT_POS_X_KD;
p->posX.integralMax = PID_FLIGHT_POS_X_INTEGRAL_MAX;
p->posX.dlpfAlpha = PID_FLIGHT_POS_X_DLPF_ALPHA;

p->posY.kp = PID_FLIGHT_POS_Y_KP;
p->posY.ki = PID_FLIGHT_POS_Y_KI;
p->posY.kd = PID_FLIGHT_POS_Y_KD;
p->posY.integralMax = PID_FLIGHT_POS_Y_INTEGRAL_MAX;
p->posY.dlpfAlpha = PID_FLIGHT_POS_Y_DLPF_ALPHA;

p->posAngleLimitRad = PID_FLIGHT_POS_ANGLE_LIMIT_RAD;

/* 位置速度环（内侧）+ 姿态补偿 */
p->posVelX.kp = PID_FLIGHT_POS_VX_KP;
p->posVelX.ki = PID_FLIGHT_POS_VX_KI;
p->posVelX.kd = PID_FLIGHT_POS_VX_KD;
p->posVelX.integralMax = PID_FLIGHT_POS_VX_INTEGRAL_MAX;
p->posVelX.dlpfAlpha = PID_FLIGHT_POS_VX_DLPF_ALPHA;

p->posVelY.kp = PID_FLIGHT_POS_VY_KP;
p->posVelY.ki = PID_FLIGHT_POS_VY_KI;
p->posVelY.kd = PID_FLIGHT_POS_VY_KD;
p->posVelY.integralMax = PID_FLIGHT_POS_VY_INTEGRAL_MAX;
p->posVelY.dlpfAlpha = PID_FLIGHT_POS_VY_DLPF_ALPHA;

p->posVelCmdLimit = PID_FLIGHT_POS_VEL_CMD_LIMIT;
p->posAttCompXGainPxPerRad = PID_FLIGHT_POS_ATT_COMP_X_GAIN_PX_PER_RAD;
p->posAttCompYGainPyPerRad = PID_FLIGHT_POS_ATT_COMP_Y_GAIN_PX_PER_RAD;

/* 偏航角度环 */
p->yawAngle.kp = PID_FLIGHT_YAW_ANGLE_KP;
p->yawAngle.ki = PID_FLIGHT_YAW_ANGLE_KI;
p->yawAngle.kd = PID_FLIGHT_YAW_ANGLE_KD;
p->yawAngle.integralMax = PID_FLIGHT_YAW_ANGLE_INTEGRAL_MAX;
p->yawAngle.dlpfAlpha = PID_FLIGHT_YAW_ANGLE_DLPF_ALPHA;

p->yawRateCmdLimit = PID_FLIGHT_YAW_RATE_CMD_LIMIT;

```

```

/* 运动前馈接口：三模式常开，默认增益来自 configFlightControlDefaults.h；无 MOTION/超时则喂零 */
p->motionVelFfVxToFwd = FLIGHT_MOTION_VEL_FF_VX_TO_FWD;
p->motionVelFfVyToRight = FLIGHT_MOTION_VEL_FF_VY_TO_RIGHT;
p->motionFfVxToPitch = FLIGHT_MOTION_FF_VX_TO_PITCH;
p->motionFfVyToRoll = FLIGHT_MOTION_FF_VY_TO_ROLL;
p->motionFfWzToYaw = FLIGHT_MOTION_FF_WZ_TO_YAW;
p->motionFfVxDiffLimitMps2 = FLIGHT_MOTION_FF_VX_DIFF_LIMIT_MPS2;
p->motionFfVyDiffLimitMps2 = FLIGHT_MOTION_FF_VY_DIFF_LIMIT_MPS2;
p->motionFfVxDiffLpfAlpha = FLIGHT_MOTION_FF_VX_DIFF_LPF_ALPHA;
p->motionFfVyDiffLpfAlpha = FLIGHT_MOTION_FF_VY_DIFF_LPF_ALPHA;
p->motionYawOrbitOffsetFwdM = FLIGHT_MOTION_YAW_ORBIT_OFFSET_FWD_M;
p->motionYawOrbitOffsetRightM = FLIGHT_MOTION_YAW_ORBIT_OFFSET_RIGHT_M;

/* ToF 时间域门控最大垂直速度默认（configAltitudeToF.h 宏作单一真相源） */
p->altToFMaxSpeedMmS = (float) ALT_TOF_MAX_SPEED_MM_S;
}

/* PID 子参数硬约束：增益/积分限幅非负，D 项 Lpf alpha 落在 (0,1] 或 >=1(不启用) */
/* ===== 硬安全上限（防菜单把安全边界放空） ===== */
#define SAN_GAIN_MAX (500.0f) /* PID 增益绝对上限 */
#define SAN_INTEGRAL_MAX (5000.0f) /* 积分限幅绝对上限 */
#define SAN_CTRL_PCT_MAX (100.0f) /* 控制量百分点上限 */
#define SAN_RATE_CMD_MAX (2000.0f) /* 角速度指令限幅上限 rad/s 量纲(留宽) */
#define SAN_POSTURE_ABS_MAX (1.6f) /* 姿态保护窗口绝对上限 ≈91°(rad)，容纳出厂 ±1.5 默认且不得放空到翻倒 */
#define SAN_POSTURE_GYRO_MAX (50.0f) /* 姿态保护角速度上限 rad/s */
#define SAN_ALT_DIST_MAX (4000.0f) /* 定高距离上限 mm(VL53L1X 量程内) */
#define SAN_ALT_OFFSET_MAX (100.0f) /* 定高油门偏移上限 % */
#define SAN_FF_ABS_MAX (10.0f) /* 前馈系数绝对上限 */
#define SAN_POS_ANGLE_MAX (1.20f) /* 位置环输出角度限幅上限 rad(对齐默认 POS_ANGLE_LIMIT_RAD，原 1.0 会把默认钳回) */
#define SAN_POS_VEL_CMD_MAX (8.0f) /* 位置环物理速度指令限幅上限 m/s(像素域已弃用,默认 4.0,留余量) */
#define SAN_POS_VEL_CMD_MIN (0.1f) /* 下限 m/s：防菜单/默认误置 0 静默清零速度目标(位置保持失效无告警),类比
SAN_POS_INTEGRAL_MIN */
#define SAN_POS_ATT_COMP_MAX (500.0f) /* 姿态补偿增益绝对上限 px/rad(可负) */
#define SAN_POS_INTEGRAL_MIN (0.001f) /* 定点四环 integralMax 非零下限：=0 会令 PID.c 选增量式，违反“位置式”硬约束 */
#define SAN_ALT_TIMEOUT_MAX (2000.0f) /* 定高无效测距保留旧输出窗口上限 ms(防菜单调过长) */
#define SAN_ALT_VELCMD_MAX (1000000.0f) /* 定高速度指令限幅上限 mm/s 量纲(高于出厂 500000，仅防绝对溢出) */
#define SAN_MOTION_DIFF_LIMIT_MAX (10.0f) /* 运动指令微分限幅上限 m/s^2 */
#define SAN_ALT_TOF_MAXSPEED_MIN (500.0f) /* ToF 门控最大垂直速度下限 mm/s：过小会误杀真实爬升 */
#define SAN_ALT_TOF_MAXSPEED_MAX (10000.0f) /* ToF 门控最大垂直速度上限 mm/s：过大门控失效 */
#define SAN_MOTION_ORBIT_OFFSET_MAX (1.5f) /* 底盘转心到飞机期望投影的可调偏置上限 m */

static uint8_t flightParamsAllFinite(const flightParamsStruct* p)
{
    const float* values = &p->baseThrottlePct;
    uint32_t count = (uint32_t)((sizeof(*p) - offsetof(flightParamsStruct, baseThrottlePct)) / sizeof(float));
    uint32_t i;

    for (i = 0u; i < count; i++)
    {
        if (floatIsFinite(values[i]) == 0u) return 0u;
    }
    return 1u;
}

static void sanitizePid(flightPidParamStruct* g)
{
    g->kp = clampf(g->kp, 0.0f, SAN_GAIN_MAX);
    g->ki = clampf(g->ki, 0.0f, SAN_GAIN_MAX);
    g->kd = clampf(g->kd, 0.0f, SAN_GAIN_MAX);
    g->integralMax = clampf(g->integralMax, 0.0f, SAN_INTEGRAL_MAX);
    /* dlpfAlpha 必须 >0；>=1 表示不启用 LPF，合法 */
    if (g->dlpfAlpha <= 0.0f) g->dlpfAlpha = 1.0f;
}

/*
* 硬安全校验/不变量修正：菜单下发的参数可能被调成负值或上下限反转，
* 直接喂给控制环会导致电机钳位失效/积分发散。cm_7_1 推送前与 cm_7_0 接收后
* 都调用本函数，保证 flightParams 始终满足物理不变量（就地修正，不依赖调用方）。
*/
void flightParamsSanitize(flightParamsStruct* p)
{
    if (0 == p) return;

    if (flightParamsAllFinite(p) == 0u)
    {
        flightParamsLoadDefaults(p);
        p->systemMode = 1u;
        p->padding[0] = 0u;
    }
}

```

```

    p->padding[1] = 0u;
}

p->visionPosErrEnable = (p->visionPosErrEnable != 0u) ? 1u : 0u;

/* 空地系统模式钳位 1.3（防菜单/flash 注入非法值，越界回安全默认 1=车不动）*/
if ((p->systemMode < 1u) || (p->systemMode > 3u))
{
    p->systemMode = FLIGHT_SYSTEM_MODE_DEFAULT;
}
p->padding[0] = 0u;
p->padding[1] = 0u;

/* 油门：范围 [0,100]，min<=base<=max，且 min<max */
p->throttleMinPct = clampf(p->throttleMinPct, 0.0f, 100.0f);
p->throttleMaxPct = clampf(p->throttleMaxPct, 0.0f, 100.0f);
if (p->throttleMaxPct < p->throttleMinPct)
{
    float t = p->throttleMinPct;
    p->throttleMinPct = p->throttleMaxPct;
    p->throttleMaxPct = t;
}
p->baseThrottlePct = clampf(p->baseThrottlePct, p->throttleMinPct, p->throttleMaxPct);

/* 角度环：限幅非负且有上限 */
sanitizePid(&p->pitchAngle);
sanitizePid(&p->rollAngle);
p->rateCmdLimitPitch = clampf(p->rateCmdLimitPitch, 0.0f, SAN_RATE_CMD_MAX);
p->rateCmdLimitRoll = clampf(p->rateCmdLimitRoll, 0.0f, SAN_RATE_CMD_MAX);

/* 角速度环 */
sanitizePid(&p->pitchRate);
sanitizePid(&p->rollRate);
sanitizePid(&p->yawRate);
p->ctrlLimitPctPitch = clampf(p->ctrlLimitPctPitch, 0.0f, SAN_CTRL_PCT_MAX);
p->ctrlLimitPctRoll = clampf(p->ctrlLimitPctRoll, 0.0f, SAN_CTRL_PCT_MAX);
p->ctrlLimitPctYaw = clampf(p->ctrlLimitPctYaw, 0.0f, SAN_CTRL_PCT_MAX);
p->pitchRateFF = clampf(p->pitchRateFF, -SAN_FF_ABS_MAX, SAN_FF_ABS_MAX);
p->rollRateFF = clampf(p->rollRateFF, -SAN_FF_ABS_MAX, SAN_FF_ABS_MAX);

/* IMU 角速度 LPF alpha ∈ (0,1) */
p->gyroLpfAlpha = clampf(p->gyroLpfAlpha, 0.001f, 1.0f);

/* 定高：距离范围 [0,上限] 且 min<max；目标落入范围；限幅有界；LPF alpha 合法。
* min 钳到 [0, MAX-1] 保证 max=min+1 时仍不突破 SAN_ALT_DIST_MAX 硬上限。*/
p->altDistMinMm = clampf(p->altDistMinMm, 0.0f, SAN_ALT_DIST_MAX - 1.0f);
p->altDistMaxMm = clampf(p->altDistMaxMm, 0.0f, SAN_ALT_DIST_MAX);
if (p->altDistMaxMm < p->altDistMinMm + 1.0f) p->altDistMaxMm = p->altDistMinMm + 1.0f;
p->altTargetMm = clampf(p->altTargetMm, p->altDistMinMm, p->altDistMaxMm);
p->altTimeoutMs = clampf(p->altTimeoutMs, 0.0f, SAN_ALT_TIMEOUT_MAX);
p->altVelLpfAlpha = clampf(p->altVelLpfAlpha, 0.001f, 1.0f);
sanitizePid(&p->altPos);
sanitizePid(&p->altVel);
p->altVelFF = clampf(p->altVelFF, -SAN_FF_ABS_MAX, SAN_FF_ABS_MAX); /* 定高速度前馈系数有界，与姿态/运动前馈一致 */
p->altOffsetLimitPct = clampf(p->altOffsetLimitPct, 0.0f, SAN_ALT_OFFSET_MAX);
p->altVelCmdLimit = clampf(p->altVelCmdLimit, 0.0f, SAN_ALT_VELCMD_MAX);
p->altToFMaxSpeedMmS = clampf(p->altToFMaxSpeedMmS, SAN_ALT_TOF_MAXSPEED_MIN, SAN_ALT_TOF_MAXSPEED_MAX);

/* 姿态保护：先钳绝对范围（防放空到翻倒角），min 取负侧、max 取正侧；再纠反转。
* Min 钳到 [-ABS_MAX, 0]，Max 钳到 [0, ABS_MAX]，保证窗口不超过 ±91°。*/
p->postureSafePitchMinRad = clampf(p->postureSafePitchMinRad, -SAN_POSTURE_ABS_MAX, 0.0f);
p->postureSafePitchMaxRad = clampf(p->postureSafePitchMaxRad, 0.0f, SAN_POSTURE_ABS_MAX);
p->postureSafeRollMinRad = clampf(p->postureSafeRollMinRad, -SAN_POSTURE_ABS_MAX, 0.0f);
p->postureSafeRollMaxRad = clampf(p->postureSafeRollMaxRad, 0.0f, SAN_POSTURE_ABS_MAX);
if (p->postureSafeGyroMaxRadS < 0.0f) p->postureSafeGyroMaxRadS = -p->postureSafeGyroMaxRadS;
p->postureSafeGyroMaxRadS = clampf(p->postureSafeGyroMaxRadS, 0.0f, SAN_POSTURE_GYRO_MAX);

/* 定点串级（位置外环 + 速度内环 + 姿态补偿）/ 偏航角度环 */
sanitizePid(&p->posX);
sanitizePid(&p->posY);
sanitizePid(&p->posVelX);
sanitizePid(&p->posVelY);
/* 四环强制位置式：integralMax==0 会令 pidInit 选增量式，违反定点“位置式”硬约束 */
if (p->posX.integralMax < SAN_POS_INTEGRAL_MIN) p->posX.integralMax = SAN_POS_INTEGRAL_MIN;
if (p->posY.integralMax < SAN_POS_INTEGRAL_MIN) p->posY.integralMax = SAN_POS_INTEGRAL_MIN;
if (p->posVelX.integralMax < SAN_POS_INTEGRAL_MIN) p->posVelX.integralMax = SAN_POS_INTEGRAL_MIN;
if (p->posVelY.integralMax < SAN_POS_INTEGRAL_MIN) p->posVelY.integralMax = SAN_POS_INTEGRAL_MIN;
p->posVelCmdLimit = clampf(p->posVelCmdLimit, SAN_POS_VEL_CMD_MIN, SAN_POS_VEL_CMD_MAX);
p->posAngleLimitRad = clampf(p->posAngleLimitRad, 0.0f, SAN_POS_ANGLE_MAX);

```

```

p->posAttCompXGainPxPerRad = clampf(p->posAttCompXGainPxPerRad, -SAN_POS_ATT_COMP_MAX, SAN_POS_ATT_COMP_MAX);
p->posAttCompYGainPxPerRad = clampf(p->posAttCompYGainPxPerRad, -SAN_POS_ATT_COMP_MAX, SAN_POS_ATT_COMP_MAX);
sanitizePid(&p->yawAngle);
if (p->yawRateCmdLimit < 0.0f) p->yawRateCmdLimit = 0.0f;
p->yawRateCmdLimit = clampf(p->yawRateCmdLimit, 0.0f, SAN_RATE_CMD_MAX);

/* 运动前馈系数有界 */
p->motionVelFfVxToFwd = clampf(p->motionVelFfVxToFwd, -SAN_FF_ABS_MAX, SAN_FF_ABS_MAX);
p->motionVelFfVyToRight = clampf(p->motionVelFfVyToRight, 0.0f, SAN_FF_ABS_MAX); /* v17: 正幅值, 符号由 config 宏固定 */
p->motionFfVxToPitch = clampf(p->motionFfVxToPitch, -SAN_FF_ABS_MAX, SAN_FF_ABS_MAX);
p->motionFfVyToRoll = clampf(p->motionFfVyToRoll, -SAN_FF_ABS_MAX, SAN_FF_ABS_MAX);
p->motionFfWzToYaw = clampf(p->motionFfWzToYaw, -SAN_FF_ABS_MAX, SAN_FF_ABS_MAX);
p->motionFfVxDiffLimitMps2 = clampf(p->motionFfVxDiffLimitMps2, 0.0f, SAN_MOTION_DIFF_LIMIT_MAX);
p->motionFfVyDiffLimitMps2 = clampf(p->motionFfVyDiffLimitMps2, 0.0f, SAN_MOTION_DIFF_LIMIT_MAX);
p->motionFfVxDiffLpfAlpha = clampf(p->motionFfVxDiffLpfAlpha, FLIGHT_LPF_ALPHA_MIN, FLIGHT_LPF_ALPHA_MAX);
p->motionFfVyDiffLpfAlpha = clampf(p->motionFfVyDiffLpfAlpha, FLIGHT_LPF_ALPHA_MIN, FLIGHT_LPF_ALPHA_MAX);
p->motionYawOrbitOffsetFwdM = clampf(p->motionYawOrbitOffsetFwdM,
-SAN_MOTION_ORBIT_OFFSET_MAX, SAN_MOTION_ORBIT_OFFSET_MAX);
p->motionYawOrbitOffsetRightM = clampf(p->motionYawOrbitOffsetRightM,
-SAN_MOTION_ORBIT_OFFSET_MAX, SAN_MOTION_ORBIT_OFFSET_MAX);

/* 末尾再次硬钳: min=max+1 等边界修正可能微越界, 最后强制回到硬上限内 */
p->altDistMaxMm = clampf(p->altDistMaxMm, 0.0f, SAN_ALT_DIST_MAX);
p->altTargetMm = clampf(p->altTargetMm, p->altDistMinMm, p->altDistMaxMm);
}

void flightParamsSetInitial(const flightParamsStruct* initial)
{
    flightParamsStruct next;
    if (initial == NULL) return;
    memcpy(&next, initial, sizeof(next));
    flightParamsSanitize(&next);
    memcpy(&activeParams, &next, sizeof(activeParams));
    activeRevision = 1u;
}

uint8_t flightParamsCommit(const flightParamsStruct* candidate)
{
    flightParamsStruct next;
    if (candidate == NULL) return 0u;
    memcpy(&next, candidate, sizeof(next));
    flightParamsSanitize(&next);
    memcpy(&activeParams, &next, sizeof(activeParams));
    activeRevision++;
    if (activeRevision == 0u) activeRevision = 1u;
    return 1u;
}

void flightParamsGetActive(flightParamsStruct* out)
{
    if (out == NULL) return;
    memcpy(out, &activeParams, sizeof(*out));
}

```

core0_cyt4bb7/project/code/flight/flightParams.h

SHA-256

:

f17467169b4bbdc11a05a08d4fb5d13c8c8b0768db4c25793aa59e93a4527ce7

代码 (c)

```

#ifndef FLIGHT_PARAMS_H_
#define FLIGHT_PARAMS_H_

#include <stdint.h>

#include "PID.h" /* pidInitFromParams 用 PID 类型 + pidInit */

/*
 * core0 飞控可调参数 RAM 镜像 (替代飞控默认值编译期直读)
 *
 * 背景: 菜单系统 (cm_7_1) 需在运行时编辑全部飞控参数并经共享内存下发给
 * cm_7_0 热重载。原先这些参数是编译期宏, 无法运行时改, 故抽出为本结构。
 *
 * 默认值单一真相源: 多数飞控参数来自 configFlightControlDefaults.h;

```

```

* ToF 时间域门控最大速度 altTofMaxSpeedMmS 来自
* configAltitudeTof.h::ALT_TOF_MAX_SPEED_MM_S。flightParamsLoadDefaults() 从宏
* 初始化本结构，菜单/flash 在此基础上覆盖。改默认值只改对应 config*.h。
*
* 编译期锁定项（不进本结构，仍是宏，改运行时会破坏 ISR 调度/ToF 时序）：
* PID_FLIGHT_CASCADE_DIV / PID_FLIGHT_ANGLE_LOOP_DIV /
* PID_FLIGHT_POS_VEL_LOOP_DIV / PID_FLIGHT_POS_LOOP_DIV /
* PID_FLIGHT_YAW_ANGLE_LOOP_DIV /
* ALT_TOF_SENSOR_SELECT / ALT_TOF_L8CX_COMM_SELECT /
* configAltitudeTof.h 中除 ALT_TOF_MAX_SPEED_MM_S 外的 ALT_TOF_*/
* configFlightTiming.h 的 ALTITUDE_HOLD_OUTER_DIV
*
* 并发约定：flightParams 被 ISR（pitProcess0 姿态保护 / pitProcess1 控制环）
* 与前台（core0Loop 热重载、altitudeHold）共享，声明 volatile。热重载（整块
* memcpy + apply）仅在非 CLOSED_LOOP 态由前台执行；float 字段 4 字节对齐，
* 单字写原子，ISR 读到的恒为合法浮点（不会撕裂为 NaN）。
*/

/* PID 参数子结构（kp/ki/kd/积分限幅/D 项 LPF 系数），对应 pidInit 入参 */
typedef struct
{
    float kp;
    float ki;
    float kd;
    float integralMax; /* != 0 → 位置式；== 0 → 增量式 */
    float dlpfAlpha; /* >=1 不启用 D 项 LPF */
} flightPidParamStruct;

/* 用 PID 子结构初始化一个 PID 实例（kp/ki/kd/积分限幅/D 项 LPF 直接映射 pidInit 入参，
* target 另传）。活动参数由仓库私有持有，owner 缓存后按 const 指针读。
* 公共内联（原在 pidFlightController.c）：pidFlightController 与 altitudeHold 共用，避免手工展开 pidInit。*/
static inline void pidInitFromParams(PID* obj, const flightPidParamStruct* p, float target)
{
    pidInit(obj, p->integralMax, target, p->kp, p->ki, p->kd, p->dlpfAlpha);
}

typedef struct
{
    /* ---- 调试开关（原编译期 #if，改运行时）---- */
    uint8_t visionPosErrEnable; /* 1=位置闭环用视觉误差；0=误差恒 0 仅前馈 */
    uint8_t systemMode; /* 空地系统模式 1=车不动 2=RS485 自主 3=LoRa 转发（语义见 uartLinkProto.h UART_LINK_SYS_MODE_*) */
    uint8_t padding[2]; /* 不承载业务语义；保持后续 float 对齐。*/

    /* ---- 油门 ---- */
    float baseThrottlePct;
    float throttleMinPct;
    float throttleMaxPct;

    /* ---- 角度参考 ---- */
    float pitchRefRad;
    float rollRefRad;

    /* ---- 角度外环 PID ---- */
    flightPidParamStruct pitchAngle;
    flightPidParamStruct rollAngle;
    float rateCmdLimitPitch;
    float rateCmdLimitRoll;

    /* ---- 角速度内环 PID ---- */
    flightPidParamStruct pitchRate;
    flightPidParamStruct rollRate;
    flightPidParamStruct yawRate;
    float yawRateRefRadS;
    float pitchRateFF;
    float rollRateFF;
    float ctrlLimitPctPitch;
    float ctrlLimitPctRoll;
    float ctrlLimitPctYaw;

    /* ---- IMU 角速度 LPF ---- */
    float gyroLpfAlpha;

    /* ---- 定高双环 ---- */
    float altTargetMm;
    float altDistMinMm;
    float altDistMaxMm;
    float altTimeoutMs;
    float altVelLpfAlpha;
    flightPidParamStruct altPos;

```

```

flightPidParamStruct altVel;
float altVelFF;
float altOffsetLimitPct;
float altVelCmdLimit;

/* ---- 姿态安全保护 ---- */
float postureSafePitchMinRad;
float postureSafePitchMaxRad;
float postureSafeRollMinRad;
float postureSafeRollMaxRad;
float postureSafeGyroMaxRadS;

/* ---- 定点串级：位置环(外) + 速度环(内，常驻闭环) ---- */
flightPidParamStruct posX; /* 位置环 X: err → 物理速度目标 m/s */
flightPidParamStruct posY; /* 位置环 Y */
flightPidParamStruct posVelX; /* 速度环 X: KF 速度误差 m/s → 角度修正 */
flightPidParamStruct posVelY; /* 速度环 Y */
float posVelCmdLimit; /* 位置环输出(物理速度目标 m/s)限幅 */
float posAngleLimitRad; /* 速度环输出(角度修正)限幅 */
float posAttCompXGainPxPerRad; /* X 轴位置误差姿态去旋转增益，可负，px/rad */
float posAttCompYGainPyPerRad; /* Y 轴位置误差姿态去旋转增益，可负，px/rad */

/* ---- 偏航角度环（^ 朝向对准） ---- */
flightPidParamStruct yawAngle;
float yawRateCmdLimit;

/* ---- 运动前馈接口（车辆运动指令 → 飞机从动；接口常开，无 MOTION/超时则喂零）
 * motionVelFf 直接使用 vx/vy；yawOrbit 用 wz 与飞机相对底盘转心偏置生成切向速度；
 * motionFfVx/Vy 使用 vx/vy 指令微分；motionFfWz 仍直通 wz。 ---- */
float motionVelFfVxToFwd;
float motionVelFfVyToRight;
float motionFfVxToPitch;
float motionFfVyToRoll;
float motionFfWzToYaw;
float motionFfVxDiffLimitMps2;
float motionFfVyDiffLimitMps2;
float motionFfVxDiffLpfAlpha;
float motionFfVyDiffLpfAlpha;
float motionYawOrbitOffsetFwdM; /* 底盘转心 -> 飞机期望投影，前为正、后为负，m */
float motionYawOrbitOffsetRightM; /* 底盘转心 -> 飞机期望投影，右为正、左为负，m */

/* ---- ToF 时间域门控（运行时可调 + flash 持久化） ---- */
float altToFMaxSpeedMmS; /* ToF 创新门单帧最大垂直速度 mm/s: maxStep=本值×实测帧间隔(下限 ALT_TOF_MIN_STEP_MM)，超此判离群冻结 */
} flightParamsStruct;

/* 用对应 config 头的默认宏初始化目标结构（多数来自 configFlightControlDefaults.h；
 * altToFMaxSpeedMmS 来自 configAltitudeToF.h）。 */
void flightParamsLoadDefaults(flightParamsStruct* p);

/* 硬安全校验/不变量修正：就地钳位非法值（负增益、上下限反转、LPF alpha 越界等）。
 * cm_7_1 菜单推送前 + cm_7_0 接收后都应调用，防止菜单调出危险参数。 */
void flightParamsSanitize(flightParamsStruct* p);

/* 参数仓库 facade。活动参数仅由本模块持有；仅允许初始化和前台配置维护 owner 调用，
 * 不得从 ISR 调用 Get/Commit。调用方取得值副本，不得持有内部指针。 */
void flightParamsSetInitial(const flightParamsStruct* initial);
uint8_t flightParamsCommit(const flightParamsStruct* candidate);
void flightParamsGetActive(flightParamsStruct* out);

#endif /* _FLIGHT_PARAMS_H_ */

```

core0_cyt4bb7/project/code/flight/flightSequenceCoordinator.c

SHA-256

:

d1f895ba792473011cabce4e839d4a3c0492e5821a74312e66eacf38e08ecf19

代码 (c)

```
#include "flightSequenceCoordinator.h"
```

```
#include <string.h>
```

```
#include "flightSequencePolicyInternal.h"
#include "flightParams.h"
```

```

#include "motionFeedforwardDiff.h"
#include "pidFlightController.h"
#include "configFlightSequence.h"
#include "configFlightControlDefaults.h"
#include "coreDataTransfer.h"
#include "uartLinkProto.h"

#define FLIGHT_SEQUENCE_ELAPSED_CAP_MS (60 000u)
#define FLIGHT_SEQUENCE_LORA_REARM_MS (400u)
#define FLIGHT_SEQUENCE_MAX_ALTITUDE_MM (4000.0f)
#define FLIGHT_SEQUENCE_MAX_VISION_ERROR_PX (1000.0f)
#define FLIGHT_SEQUENCE_MAX_VISION_YAW_RAD (3.1415927f)
#define FLIGHT_SEQUENCE_POLICY_ACTION_ARM | \
    FLIGHT_SEQUENCE_POLICY_ACTION_TAKEOFF | FLIGHT_SEQUENCE_POLICY_ACTION_LAND
#define FLIGHT_SEQUENCE_POLICY_OVERRIDE_ALL (FLIGHT_SEQUENCE_POLICY_OVERRIDE_ALTITUDE | \
    FLIGHT_SEQUENCE_POLICY_OVERRIDE_POSITION | FLIGHT_SEQUENCE_POLICY_OVERRIDE_RGB | \
    FLIGHT_SEQUENCE_POLICY_OVERRIDE_BUZZER | FLIGHT_SEQUENCE_POLICY_OVERRIDE_REACHED | \
    FLIGHT_SEQUENCE_POLICY_OVERRIDE_DISARM | FLIGHT_SEQUENCE_POLICY_OVERRIDE_CONTROL | \
    FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION)
#define FLIGHT_SEQUENCE_TRIGGER_BUILTIN_ALL (FLIGHT_SEQUENCE_TRIGGER_K5_LAUNCH | \
    FLIGHT_SEQUENCE_TRIGGER_K5_STANDBY | FLIGHT_SEQUENCE_TRIGGER_LORA_MODE3 | \
    FLIGHT_SEQUENCE_TRIGGER_LORA_MODE2 | FLIGHT_SEQUENCE_TRIGGER_CORE1_LOST | \
    FLIGHT_SEQUENCE_TRIGGER_CORE2_LOST | FLIGHT_SEQUENCE_TRIGGER_REQUIRED_VISION_FAULT | \
    FLIGHT_SEQUENCE_TRIGGER_TOF_FAILSAFE | FLIGHT_SEQUENCE_TRIGGER_CORE2_ATTITUDE_FAULT)
#define FLIGHT_SEQUENCE_TRIGGER_POLICY_ALL (FLIGHT_SEQUENCE_TRIGGER_BUILTIN_ALL | \
    FLIGHT_SEQUENCE_TRIGGER_POLICY_MASK)

typedef struct
{
    uint32_t lastNowMs;
    uint32_t cycleSeq;
    uint32_t episodeSeq;
    uint32_t transitionSeq;
    uint32_t lastLaunchAcceptedSeq;
    uint32_t lastDisarmAcceptedSeq;
    uint32_t lastProtectAcceptedSeq;
    uint32_t stageElapsedMs;
    uint32_t mode3LowStartMs;
    uint32_t mode2LowStartMs;
    uint32_t latchedTriggerSources;
    uint32_t policyActionLevelMask;
    uint32_t lastPolicyTriggerEventSeq;
    uint32_t lastPolicyTriggerSources;
    float stageEntryAltitudeMm;
    float cruiseAltitudeMm;
    flightSequenceStageEnum stage;
    uint8_t primaryReason;
    uint8_t pendingAutoTakeoff;
    uint8_t mode3Armed;
    uint8_t mode2Armed;
    uint8_t mode3LowTiming;
    uint8_t mode2LowTiming;
    uint8_t lastVisionHadTarget;
    uint8_t lastSystemMode;
    uint8_t policyTriggerPrimed;
} flightSequenceContextStruct;

static flightSequenceContextStruct sequenceContext;
static flightParamsStruct sequenceParams;

static uint8_t finiteFloat(float value)
{
    {
        return (uint8_t) ((value == value) &&
            (value <= 3.4028234e38f) && (value >= -3.4028234e38f));
    }
}

static void enterStage(flightSequenceStageEnum stage, float entryAltitudeMm)
{
    {
        sequenceContext.stage = stage;
        sequenceContext.stageElapsedMs = 0u;
        sequenceContext.stageEntryAltitudeMm = entryAltitudeMm;
        sequenceContext.transitionSeq++;
    }
}

static uint8_t consumeLevelRequest(uint8_t valid, uint8_t level,
    uint8_t rearmAllowed, uint8_t *armed, uint8_t *lowTiming,
    uint32_t *lowStartMs, uint32_t nowMs)
{
    {
        if (valid == 0u)

```

```

{
    *lowTiming = 0u;
    return 0u;
}
if (level == 0u)
{
    if (rearmAllowed == 0u)
    {
        *lowTiming = 0u;
        return 0u;
    }
    if (*lowTiming == 0u)
    {
        *lowTiming = 1u;
        *lowStartMs = nowMs;
    }
    else if ((uint32_t) (nowMs - *lowStartMs) >= FLIGHT_SEQUENCE_LORA_REARM_MS)
    {
        *armed = 1u;
    }
    return 0u;
}
*lowTiming = 0u;
if (*armed == 0u) return 0u;
*armed = 0u;
return 1u;
}

static uint8_t requiredLinkLost(const flightSequenceFactsStruct *facts)
{
    return (uint8_t) ((facts->core1Online == 0u) ||
        (facts->core2LinkLost != 0u) ||
        (facts->core2AttitudeFault != 0u) ||
        (facts->requiredVisionFault != 0u));
}

static uint8_t confirmedLandingLinkLost(const flightSequenceFactsStruct *facts,
    uint8_t closedLoop, uint8_t *reason, uint32_t *trigger)
{
    if (facts->core1Online == 0u)
    {
        *reason = CORE0_FAULT_REASON_LAND_C1_LINK;
        *trigger = FLIGHT_SEQUENCE_TRIGGER_CORE1_LOST;
        return 1u;
    }
    if (facts->core2LinkLost != 0u)
    {
        *reason = CORE0_FAULT_REASON_LAND_C2_LINK;
        *trigger = FLIGHT_SEQUENCE_TRIGGER_CORE2_LOST;
        return 1u;
    }
    if (facts->core2AttitudeFault != 0u)
    {
        *reason = CORE0_FAULT_REASON_LAND_C2_ATTITUDE;
        *trigger = FLIGHT_SEQUENCE_TRIGGER_CORE2_ATTITUDE_FAULT;
        return 1u;
    }
    if ((closedLoop == 0u) || (facts->requiredVisionFault == 0u))
    {
        return 0u;
    }
    *reason = CORE0_FAULT_REASON_REQUIRED_VISION;
    *trigger = FLIGHT_SEQUENCE_TRIGGER_REQUIRED_VISION_FAULT;
    return 1u;
}

void flightSequenceReset(flightSequenceResetReasonEnum reason)
{
    sequenceContext.pendingAutoTakeoff = 0u;
    sequenceContext.mode3Armed = 0u;
    sequenceContext.mode2Armed = 0u;
    sequenceContext.mode3LowTiming = 0u;
    sequenceContext.mode2LowTiming = 0u;
    sequenceContext.lastVisionHadTarget = 0u;
    sequenceContext.primaryReason = CORE0_FAULT_REASON_NONE;
    sequenceContext.latchedTriggerSources = 0u;
    sequenceContext.policyActionLevelMask = 0u;
    enterStage(FLIGHT_SEQUENCE_STAGE_IDLE, 0.0f);
    motionFeedforwardDiffReset();
}

```



```

    flightSequencePolicyReset(reason);
}

void flightSequenceInit(uint32_t nowMs)
{
    memset(&sequenceContext, 0, sizeof(sequenceContext));
    sequenceContext.lastNowMs = nowMs;
    sequenceContext.stage = FLIGHT_SEQUENCE_STAGE_IDLE;
    sequenceContext.lastSystemMode = 0xFFu;
    motionFeedforwardDiffInit();
    flightSequenceApplyParams();
    flightSequencePolicyReset(FLIGHT_SEQUENCE_RESET_INIT);
}

void flightSequenceApplyParams(void)
{
    flightParamsGetActive(&sequenceParams);
    sequenceContext.cruiseAltitudeMm = sequenceParams.altTargetMm;
    motionFeedforwardDiffApplyParams();
}

static void renderBaseStage(const flightSequenceFactsStruct *facts,
    flightSequenceOutputStruct *output, flightSequenceStageEnum renderStage)
{
    output->stage = renderStage;
    output->stageElapsedMs = sequenceContext.stageElapsedMs;
    output->altitudeTargetMm = (float) facts->altitude.distanceMm;
    output->rgbIntent = UART_LINK_RGB_NORMAL;
    output->buzzerPattern = UART_LINK_BUZZER_SILENT;
    output->altReached = 0u;
    output->altitudeControlEnable = 0u;
    output->allowExternalMotion = 1u; /* 保留现有非巡航角度前馈兼容语义。 */

    switch (renderStage)
    {
        case FLIGHT_SEQUENCE_STAGE_WAIT:
            output->altitudeTargetMm = sequenceContext.stageEntryAltitudeMm;
            output->idleThrottleMode = 1u;
            output->rgbIntent = UART_LINK_RGB_TKO_WAIT;
            output->buzzerPattern = UART_LINK_BUZZER_1HZ;
            break;
        case FLIGHT_SEQUENCE_STAGE_RAMP:
            output->altitudeControlEnable = 1u;
            break;
        case FLIGHT_SEQUENCE_STAGE_CRUISE:
            output->altitudeControlEnable = 1u;
            output->allowVisionPosition = 1u;
            output->altitudeTargetMm = sequenceContext.cruiseAltitudeMm;
            output->altReached = 1u;
            if (sequenceContext.stageElapsedMs < TAKEOFF_APEX_BEEP_MS)
                output->buzzerPattern = UART_LINK_BUZZER_LONG;
            else if (sequenceContext.stageElapsedMs <
                (uint32_t) (TAKEOFF_APEX_BEEP_MS + TAKEOFF_CRUISE_BEEP_MS))
                output->buzzerPattern = UART_LINK_BUZZER_3S;
            break;
        case FLIGHT_SEQUENCE_STAGE_DESCENDING:
            output->altitudeControlEnable = 1u;
            output->landingActive = 1u;
            break;
        case FLIGHT_SEQUENCE_STAGE_LANDED:
            output->altitudeControlEnable = 1u;
            output->landingActive = 1u;
            output->altitudeTargetMm = LAND_FLOOR_MM;
            output->rgbIntent = UART_LINK_RGB_DISCONNECT_WARN;
            output->buzzerPattern = UART_LINK_BUZZER_WARN;
            break;
        case FLIGHT_SEQUENCE_STAGE_IDLE:
        default:
            if ((facts->systemMode == (uint8_t) UART_LINK_SYS_MODE_LORA_FWD) &&
                (facts->safety.state == SAFETY_STATE_CLOSED_LOOP))
            {
                output->idleThrottleMode = 1u;
                output->buzzerPattern = UART_LINK_BUZZER_ARMED_IDLE;
            }
            break;
    }
}

static void mergePolicy(const flightSequencePolicyDecisionStruct *decision,

```

```

flightSequenceStageEnum renderStage,
flightSequenceOutputStruct *output)
{
    output->rejectedOverrideMask |= decision->overrideMask &
        ~FLIGHT_SEQUENCE_POLICY_OVERRIDE_ALL;
    if ((decision->overrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_ALTITUDE) != 0u)
    {
        if (finiteFloat(decision->altitudeTargetMm) &&
            decision->altitudeTargetMm >= 0.0f &&
            decision->altitudeTargetMm <= FLIGHT_SEQUENCE_MAX_ALTITUDE_MM)
            output->altitudeTargetMm = decision->altitudeTargetMm;
        else output->rejectedOverrideMask |= FLIGHT_SEQUENCE_POLICY_OVERRIDE_ALTITUDE;
    }
    if ((decision->overrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_RGB) != 0u)
    {
        if (decision->rgbIntent <= UART_LINK_RGB_DISCONNECT_WARN)
            output->rgbIntent = decision->rgbIntent;
        else output->rejectedOverrideMask |= FLIGHT_SEQUENCE_POLICY_OVERRIDE_RGB;
    }
    if ((decision->overrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_BUZZER) != 0u)
    {
        if (decision->buzzerPattern <= UART_LINK_BUZZER_THREE_BEEPS)
            output->buzzerPattern = decision->buzzerPattern;
        else output->rejectedOverrideMask |= FLIGHT_SEQUENCE_POLICY_OVERRIDE_BUZZER;
    }
    if ((decision->overrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_REACHED) != 0u)
    {
        if (decision->altReached <= 1u)
            output->altReached = decision->altReached;
        else output->rejectedOverrideMask |= FLIGHT_SEQUENCE_POLICY_OVERRIDE_REACHED;
    }

    if ((decision->overrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_POSITION) != 0u)
    {
        if (finiteFloat(decision->bodyVelocityFwdMps) &&
            finiteFloat(decision->bodyVelocityRightMps) &&
            finiteFloat(decision->yawRateRadS) &&
            decision->bodyVelocityFwdMps >= -sequenceParams.posVelCmdLimit &&
            decision->bodyVelocityFwdMps <= sequenceParams.posVelCmdLimit &&
            decision->bodyVelocityRightMps >= -sequenceParams.posVelCmdLimit &&
            decision->bodyVelocityRightMps <= sequenceParams.posVelCmdLimit &&
            decision->yawRateRadS >= -sequenceParams.yawRateCmdLimit &&
            decision->yawRateRadS <= sequenceParams.yawRateCmdLimit &&
            decision->allowVisionPosition <= 1u &&
            decision->allowExternalMotion <= 1u &&
            decision->localPositionValid <= 1u)
        {
            output->policyPositionActive = decision->localPositionValid ? 1u : 0u;
            if (output->policyPositionActive != 0u)
            {
                output->bodyVelocityFwdMps = decision->bodyVelocityFwdMps;
                output->bodyVelocityRightMps = decision->bodyVelocityRightMps;
                output->yawRateRadS = decision->yawRateRadS;
            }
            else
            {
                output->bodyVelocityFwdMps = 0.0f;
                output->bodyVelocityRightMps = 0.0f;
                output->yawRateRadS = 0.0f;
            }
            output->allowVisionPosition = decision->allowVisionPosition ? 1u : 0u;
            output->allowExternalMotion = decision->allowExternalMotion ? 1u : 0u;
        }
        else output->rejectedOverrideMask |= FLIGHT_SEQUENCE_POLICY_OVERRIDE_POSITION;
    }
    if (decision->requestDisarm != 0u) output->requestDisarm = 1u;
    if ((decision->overrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_CONTROL) != 0u)
    {
        if (decision->altitudeControlEnable <= 1u &&
            decision->idleThrottleMode <= 1u &&
            !(decision->altitudeControlEnable && decision->idleThrottleMode))
        {
            output->altitudeControlEnable = decision->altitudeControlEnable;
            output->idleThrottleMode = decision->idleThrottleMode;
        }
        else output->rejectedOverrideMask |= FLIGHT_SEQUENCE_POLICY_OVERRIDE_CONTROL;
    }
    if ((decision->overrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION) != 0u)
    {

```

```

        if (decision->stageTransitionReady > 1u ||
            (renderStage != FLIGHT_SEQUENCE_STAGE_WAIT &&
             renderStage != FLIGHT_SEQUENCE_STAGE_RAMP &&
             renderStage != FLIGHT_SEQUENCE_STAGE_DESCENDING))
        {
            output->rejectedOverrideMask |= FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION;
        }
    }
}

void flightSequenceStep(const flightSequenceFactsStruct *facts,
    flightSequenceOutputStruct *output)
{
    flightSequencePolicyContextStruct policyContext;
    flightSequencePolicyDecisionStruct decision;
    flightSequenceFactsStruct policyFacts;
    flightSequenceStageEnum renderStage;
    uint32_t dtMs;
    uint32_t landingTrigger = 0u;
    uint32_t policyActionMask;
    uint32_t policyTriggerSources = 0u;
    uint8_t policyPrimaryReason = CORE0_FAULT_REASON_NONE;
    uint8_t landingReason = CORE0_FAULT_REASON_NONE;
    uint8_t closedLoop;
    uint8_t mode3Event;
    uint8_t mode2Event;
    uint8_t landingTriggeredThisStep = 0u;
    uint8_t directDisarmActive = 0u;

    if (output == 0) return;
    memset(output, 0, sizeof(*output));
    if (facts == 0) return;

    dtMs = facts->nowMs - sequenceContext.lastNowMs;
    sequenceContext.lastNowMs = facts->nowMs;
    sequenceContext.cycleSeq++;
    policyFacts = *facts;
    policyFacts.dtMs = dtMs;
    policyFacts.coordinatorCycleSeq = sequenceContext.cycleSeq;
    policyFacts.coordinatorTransitionSeq = sequenceContext.transitionSeq;
    policyFacts.coordinatorEpisodeSeq = sequenceContext.episodeSeq;
    closedLoop = (facts->safety.state == SAFETY_STATE_CLOSED_LOOP) ? 1u : 0u;

    if (facts->systemMode != sequenceContext.lastSystemMode)
    {
        sequenceContext.lastSystemMode = facts->systemMode;
        sequenceContext.mode3Armed = 0u;
        sequenceContext.mode2Armed = 0u;
        sequenceContext.mode3LowTiming = 0u;
        sequenceContext.mode2LowTiming = 0u;
        if (sequenceContext.stage == FLIGHT_SEQUENCE_STAGE_DESCENDING)
            sequenceContext.stageEntryAltitudeMm = (float) facts->altitude.distanceMm;
        flightSequencePolicyReset(FLIGHT_SEQUENCE_RESET_MODE);
    }
    if (facts->rejoinEdge != 0u)
    {
        sequenceContext.mode3Armed = 0u;
        sequenceContext.mode2Armed = 0u;
        if (sequenceContext.stage == FLIGHT_SEQUENCE_STAGE_DESCENDING)
            sequenceContext.stageEntryAltitudeMm = (float) facts->altitude.distanceMm;
        flightSequencePolicyReset(FLIGHT_SEQUENCE_RESET_REJOIN);
    }

    if (facts->safety.disarmAcceptedSeq != sequenceContext.lastDisarmAcceptedSeq)
    {
        sequenceContext.lastDisarmAcceptedSeq = facts->safety.disarmAcceptedSeq;
        flightSequenceReset(FLIGHT_SEQUENCE_RESET_DISARM);
    }
    if (facts->safety.protectAcceptedSeq != sequenceContext.lastProtectAcceptedSeq)
    {
        sequenceContext.lastProtectAcceptedSeq = facts->safety.protectAcceptedSeq;
        flightSequenceReset(FLIGHT_SEQUENCE_RESET_PROTECT);
    }
    if (facts->safety.launchAcceptedSeq != sequenceContext.lastLaunchAcceptedSeq)
    {
        sequenceContext.lastLaunchAcceptedSeq = facts->safety.launchAcceptedSeq;
        sequenceContext.episodeSeq++;
        flightSequencePolicyReset(FLIGHT_SEQUENCE_RESET_INIT);
        if (sequenceContext.pendingAutoTakeoff != 0u)

```

```

    {
        sequenceContext.pendingAutoTakeoff = 0u;
        enterStage(FLIGHT_SEQUENCE_STAGE_WAIT,
            (float) facts->altitude.distanceMm);
    }
}

mode3Event = consumeLevelRequest(facts->mode3RequestValid,
    facts->mode3RequestLevel, (uint8_t) (facts->core2LinkLost == 0u),
    &sequenceContext.mode3Armed, &sequenceContext.mode3LowTiming,
    &sequenceContext.mode3LowStartMs, facts->nowMs);
mode2Event = consumeLevelRequest(facts->mode2RequestValid,
    facts->mode2RequestLevel, (uint8_t) (facts->core2LinkLost == 0u),
    &sequenceContext.mode2Armed, &sequenceContext.mode2LowTiming,
    &sequenceContext.mode2LowStartMs, facts->nowMs);
policyFacts.mode3RequestArmed = sequenceContext.mode3Armed;
policyFacts.mode2RequestArmed = sequenceContext.mode2Armed;
policyFacts.coordinatorTransitionSeq = sequenceContext.transitionSeq;
policyFacts.coordinatorEpisodeSeq = sequenceContext.episodeSeq;

memset(&policyContext, 0, sizeof(policyContext));
policyContext.stage = sequenceContext.stage;
policyContext.stageElapsedMs = sequenceContext.stageElapsedMs;
policyContext.dtMs = dtMs;
policyContext.stageEntryAltitudeMm = sequenceContext.stageEntryAltitudeMm;
policyContext.cruiseAltitudeMm = sequenceContext.cruiseAltitudeMm;
policyContext.mode3RequestEvent = (uint8_t) (mode3Event &&
    facts->systemMode == (uint8_t) UART_LINK_SYS_MODE_LORA_FWD);
policyContext.mode2RequestEvent = (uint8_t) (mode2Event &&
    facts->systemMode == (uint8_t) UART_LINK_SYS_MODE_RS485_AUTO);
memset(&decision, 0, sizeof(decision));
flightSequencePolicySelectTriggers(&policyFacts, &policyContext, &decision);
policyActionMask = decision.actionMask & FLIGHT_SEQUENCE_POLICY_ACTION_ALL;
if (policyActionMask != decision.actionMask)
{
    output->safetyClampMask |= FLIGHT_SEQUENCE_CLAMP_POLICY_INVALID;
    policyActionMask = 0u;
}
if (policyActionMask != 0u)
{
    uint8_t triggerMetadataValid = (uint8_t)
        (decision.triggerEventValid == 1u && decision.triggerSources != 0u &&
        (decision.triggerSources & ~FLIGHT_SEQUENCE_TRIGGER_POLICY_ALL) == 0u);
    uint8_t risingAction = (uint8_t)
        ((policyActionMask & ~sequenceContext.policyActionLevelMask) != 0u);
    uint8_t repeatedToken = (uint8_t)
        (sequenceContext.policyTriggerPrimed != 0u &&
        sequenceContext.lastPolicyTriggerEventSeq == decision.triggerEventSeq &&
        sequenceContext.lastPolicyTriggerSources == decision.triggerSources);
    if (triggerMetadataValid && risingAction && !repeatedToken)
    {
        policyTriggerSources = decision.triggerSources;
        policyPrimaryReason = decision.primaryReason;
        sequenceContext.policyTriggerPrimed = 1u;
        sequenceContext.lastPolicyTriggerEventSeq = decision.triggerEventSeq;
        sequenceContext.lastPolicyTriggerSources = decision.triggerSources;
    }
    else
    {
        if (!triggerMetadataValid)
            output->safetyClampMask |= FLIGHT_SEQUENCE_CLAMP_POLICY_INVALID;
        policyActionMask = 0u;
    }
}
}
sequenceContext.policyActionLevelMask = decision.actionMask &
    FLIGHT_SEQUENCE_POLICY_ACTION_ALL;

if (facts->k5StandbyEdge != 0u)
{
    output->requestDisarm = 1u;
    directDisarmActive = 1u;
    output->triggerSources |= FLIGHT_SEQUENCE_TRIGGER_K5_STANDBY;
    sequenceContext.pendingAutoTakeoff = 0u;
}
if (((policyActionMask & FLIGHT_SEQUENCE_POLICY_ACTION_ARM) != 0u) &&
    (directDisarmActive == 0u))
{
    output->triggerSources |= policyTriggerSources;
    sequenceContext.latchedTriggerSources |= policyTriggerSources;
}

```

```

    if (policyPrimaryReason != CORE0_FAULT_REASON_NONE)
        sequenceContext.primaryReason = policyPrimaryReason;
    if (facts->faultLatched != 0u) output->acknowledgeFault = 1u;
    else if ((facts->safety.state == SAFETY_STATE_DISARMED ||
             facts->safety.state == SAFETY_STATE_PROTECTED) &&
             (facts->initialized != 0u) && (facts->controlReady != 0u) &&
             (requiredLinkLost(facts) == 0u))
    {
        output->requestLaunch = 1u;
        if ((policyActionMask & FLIGHT_SEQUENCE_POLICY_ACTION_TAKEOFF) != 0u)
            sequenceContext.pendingAutoTakeoff = 1u;
    }
    else
    {
        sequenceContext.pendingAutoTakeoff = 0u;
        output->safetyClampMask |= (requiredLinkLost(facts) != 0u) ?
            FLIGHT_SEQUENCE_CLAMP_REQUIRED_LINK : FLIGHT_SEQUENCE_CLAMP_SAFETY_STATE;
    }
}

if (((policyActionMask & FLIGHT_SEQUENCE_POLICY_ACTION_TAKEOFF) != 0u) &&
    ((policyActionMask & FLIGHT_SEQUENCE_POLICY_ACTION_ARM) == 0u) && closedLoop &&
    (sequenceContext.stage == FLIGHT_SEQUENCE_STAGE_IDLE))
{
    if ((facts->systemPhase == CORE_UX_PHASE_HOME) && (facts->standalone == 0u) &&
        (requiredLinkLost(facts) == 0u))
    {
        output->triggerSources |= policyTriggerSources;
        sequenceContext.latchedTriggerSources |= policyTriggerSources;
        if (policyPrimaryReason != CORE0_FAULT_REASON_NONE)
            sequenceContext.primaryReason = policyPrimaryReason;
        enterStage(FLIGHT_SEQUENCE_STAGE_WAIT, (float) facts->altitude.distanceMm);
    }
}

if (sequenceContext.stageElapsedMs < FLIGHT_SEQUENCE_ELAPSED_CAP_MS)
{
    uint32_t room = FLIGHT_SEQUENCE_ELAPSED_CAP_MS - sequenceContext.stageElapsedMs;
    sequenceContext.stageElapsedMs += (dtMs < room) ? dtMs : room;
}

renderStage = sequenceContext.stage;
renderBaseStage(facts, output, renderStage);

policyContext.stage = renderStage;
policyContext.stageElapsedMs = sequenceContext.stageElapsedMs;
policyContext.dtMs = dtMs;
policyContext.stageEntryAltitudeMm = sequenceContext.stageEntryAltitudeMm;
memset(&decision, 0, sizeof(decision));
policyFacts.coordinatorTransitionSeq = sequenceContext.transitionSeq;
policyFacts.coordinatorEpisodeSeq = sequenceContext.episodeSeq;
flightSequencePolicyApplyStage(&policyFacts, &policyContext, &decision);
mergePolicy(&decision, renderStage, output);
if (output->rejectedOverrideMask != 0u)
    output->safetyClampMask |= FLIGHT_SEQUENCE_CLAMP_POLICY_INVALID;

if (renderStage == FLIGHT_SEQUENCE_STAGE_WAIT &&
    (((decision.overrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION) != 0u) ?
     ((output->rejectedOverrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION) == 0u) &&
     decision.stageTransitionReady) :
    (sequenceContext.stageElapsedMs >= TAKEOFF_WAIT_MS))
    enterStage(FLIGHT_SEQUENCE_STAGE_RAMP, sequenceContext.stageEntryAltitudeMm);
else if (renderStage == FLIGHT_SEQUENCE_STAGE_RAMP &&
    (((decision.overrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION) != 0u) ?
     ((output->rejectedOverrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION) == 0u) &&
     decision.stageTransitionReady) :
    (sequenceContext.stageElapsedMs >= TAKEOFF_RAMP_MS))
    enterStage(FLIGHT_SEQUENCE_STAGE_CRUISE, sequenceContext.cruiseAltitudeMm);
else if (renderStage == FLIGHT_SEQUENCE_STAGE_DESCENDING &&
    (decision.touchdown ||
     (decision.overrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION) != 0u))
{
    uint8_t transitionOverrideValid = (uint8_t)
        ((decision.overrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION) != 0u &&
         (output->rejectedOverrideMask & FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION) == 0u);
    uint8_t landReady = (uint8_t) (decision.touchdown ||
        (transitionOverrideValid && decision.stageTransitionReady));
    if (landReady) enterStage(FLIGHT_SEQUENCE_STAGE_LANDED, output->altitudeTargetMm);
}
else if (renderStage == FLIGHT_SEQUENCE_STAGE_LANDED)

```

```

output->requestDisarm = 1u;

if (closedLoop && confirmedLandingLinkLost(facts, closedLoop,
&landingReason, &landingTrigger))
{
    if ((sequenceContext.stage != FLIGHT_SEQUENCE_STAGE_DESCENDING) &&
        (sequenceContext.stage != FLIGHT_SEQUENCE_STAGE_LANDED))
    {
        sequenceContext.primaryReason = landingReason;
        sequenceContext.latchedTriggerSources |= landingTrigger;
        enterStage(FLIGHT_SEQUENCE_STAGE_DESCENDING,
            (float) facts->altitude.distanceMm);
        flightSequencePolicyReset(FLIGHT_SEQUENCE_RESET_ABORT);
        landingTriggeredThisStep = 1u;
    }
}
else if (closedLoop &&
        ((policyActionMask & FLIGHT_SEQUENCE_POLICY_ACTION_LAND) != 0u) &&
        (sequenceContext.stage != FLIGHT_SEQUENCE_STAGE_DESCENDING) &&
        (sequenceContext.stage != FLIGHT_SEQUENCE_STAGE_LANDED))
{
    landingReason = (policyPrimaryReason != CORE0_FAULT_REASON_NONE) ?
        policyPrimaryReason : (policyContext.mode2RequestEvent ?
            CORE0_FAULT_REASON_LAND_LORA_MODE2 : CORE0_FAULT_REASON_LAND_LORA_MODE3);
    landingTrigger = policyTriggerSources;
    sequenceContext.primaryReason = landingReason;
    sequenceContext.latchedTriggerSources |= landingTrigger;
    enterStage(FLIGHT_SEQUENCE_STAGE_DESCENDING,
        (float) facts->altitude.distanceMm);
    flightSequencePolicyReset(FLIGHT_SEQUENCE_RESET_ABORT);
    landingTriggeredThisStep = 1u;
}
if ((sequenceContext.stage == FLIGHT_SEQUENCE_STAGE_DESCENDING) ||
    (sequenceContext.stage == FLIGHT_SEQUENCE_STAGE_LANDED))
{
    output->landingActive = 1u;
    output->altReached = 0u;
    output->safetyClampMask |= FLIGHT_SEQUENCE_CLAMP_LANDING;
    output->triggerSources |= landingTrigger;
    output->primaryReason = sequenceContext.primaryReason;
    if (landingTriggeredThisStep != 0u)
    {
        /* 兼容既有语义：触发拍立即撤销运动资格，高度/RGB/蜂鸣从下一拍缓冲输出接管。*/
        output->policyPositionActive = 1u;
    }
}
}
if (output->requestDisarm != 0u) directDisarmActive = 1u;

if ((facts->safety.state != SAFETY_STATE_CLOSED_LOOP) ||
    (facts->safety.valid == 0u) || (facts->paramsValid == 0u) ||
    (facts->initialized == 0u) || (facts->controlReady == 0u) ||
    (directDisarmActive != 0u))
{
    output->altitudeControlEnable = 0u;
    output->idleThrottleMode = 0u;
    output->altReached = 0u;
    output->policyPositionActive = 1u;
    output->safetyClampMask |= FLIGHT_SEQUENCE_CLAMP_SAFETY_STATE;
}
/* 唯一 owner 在目标/门设置后推进 altitudeHold，并以本拍 fresh failsafe 做最终钳制。*/
pidFlightControllerSetIdleThrottleMode(output->idleThrottleMode);
altitudeHoldSetControlEnabled(output->altitudeControlEnable);
altitudeHoldSetTargetMm(output->altitudeTargetMm);
altitudeHoldUpdate(facts->nowMs);
altitudeHoldGetSnapshot(&output->altitude);
if (output->altitude.failsafeStage != ALTITUDE_FAILSAFE_NORMAL)
{
    output->rgbIntent = UART_LINK_RGB_DISCONNECT_WARN;
    output->buzzerPattern = UART_LINK_BUZZER_WARN;
}
}
if (output->altitude.failsafeStage == ALTITUDE_FAILSAFE_RAMP)
{
    output->altReached = 0u;
    output->policyPositionActive = 1u;
    output->safetyClampMask |= FLIGHT_SEQUENCE_CLAMP_TOF_RAMP;
    output->triggerSources |= FLIGHT_SEQUENCE_TRIGGER_TOF_FAILSAFE;
    if (output->primaryReason == CORE0_FAULT_REASON_NONE)
        output->primaryReason = CORE0_FAULT_REASON_LAND_TOF_FAILSAFE;
}
}

```

```

if (output->altitude.requestDisarm != 0u)
{
    output->requestDisarm = 1u;
    output->altReached = 0u;
    output->policyPositionActive = 1u;
    output->safetyClampMask |= FLIGHT_SEQUENCE_CLAMP_SAFETY_STATE;
}
if ((facts->readyBeepActive != 0u) && (output->landingActive == 0u) &&
    (output->altitude.failsafeStage == ALTITUDE_FAILSAFE_NORMAL) &&
    ((output->buzzerPattern == UART_LINK_BUZZER_SILENT) ||
     (output->buzzerPattern == UART_LINK_BUZZER_3S)))
    output->buzzerPattern = UART_LINK_BUZZER_THREE_BEEPS;

/* 最终水平引导仲裁：Policy 本地 > 普通视觉/MOTION；安全钳制已将 Policy 源置零。 */
if (output->policyPositionActive == 0u)
{
    uint8_t externalAllowed = (uint8_t) (output->allowExternalMotion && facts->motionValid &&
        facts->motionFenceOpen && !facts->standalone &&
        facts->systemPhase == CORE_UX_PHASE_HOME && !output->landingActive &&
        output->altitude.failsafeStage != ALTITUDE_FAILSAFE_RAMP);
    if (externalAllowed)
    {
        motionFeedforwardDiffUpdate(facts->motionVx, facts->motionVy,
            facts->motionSeq, facts->nowMs);
        if (renderStage == FLIGHT_SEQUENCE_STAGE_CRUISE)
        {
            output->bodyVelocityFwdMps = facts->motionVx *
                sequenceParams.motionVelFfVxToFwd;
            output->bodyVelocityRightMps = facts->motionVy *
                FLIGHT_MOTION_VEL_FF_VY_TO_RIGHT_SIGN *
                sequenceParams.motionVelFfVyToRight;
            output->bodyVelocityFwdMps += facts->motionWz *
                sequenceParams.motionYawOrbitOffsetRightM;
            output->bodyVelocityRightMps += facts->motionWz *
                sequenceParams.motionYawOrbitOffsetFwdM;
        }
        output->pitchFeedforwardRad = motionFeedforwardDiffGetDvX() *
            sequenceParams.motionFfVxToPitch;
        output->rollFeedforwardRad = motionFeedforwardDiffGetDvY() *
            sequenceParams.motionFfVyToRoll;
        output->yawRateRadS = facts->motionWz * sequenceParams.motionFfWzToYaw;
    }
    else motionFeedforwardDiffReset();

    if (output->allowVisionPosition &&
        (renderStage == FLIGHT_SEQUENCE_STAGE_CRUISE) &&
        sequenceParams.visionPosErrEnable && facts->visionFrameValid)
    {
        uint8_t visionFinite = (uint8_t) (finiteFloat(facts->visionErrX) &&
            finiteFloat(facts->visionErrY) && finiteFloat(facts->visionYawErr) &&
            facts->visionErrX >= -FLIGHT_SEQUENCE_MAX_VISION_ERROR_PX &&
            facts->visionErrX <= FLIGHT_SEQUENCE_MAX_VISION_ERROR_PX &&
            facts->visionErrY >= -FLIGHT_SEQUENCE_MAX_VISION_ERROR_PX &&
            facts->visionErrY <= FLIGHT_SEQUENCE_MAX_VISION_ERROR_PX &&
            facts->visionYawErr >= -FLIGHT_SEQUENCE_MAX_VISION_YAW_RAD &&
            facts->visionYawErr <= FLIGHT_SEQUENCE_MAX_VISION_YAW_RAD);
        if (facts->visionTargetValid && visionFinite)
        {
            sequenceContext.lastVisionHadTarget = 1u;
            output->visionErrX = facts->visionErrX;
            output->visionErrY = facts->visionErrY;
            output->visionYawErr = facts->visionYawErr;
            output->visionValid = 1u;
        }
        else if (sequenceContext.lastVisionHadTarget && visionFinite &&
            facts->targetLostAgeMs < 3000u)
        {
            output->visionErrX = facts->visionErrX;
            output->visionErrY = facts->visionErrY;
            output->visionValid = 1u;
        }
    }
}
else
{
    if (output->landingActive != 0u)
    {
        /* 缓降保留 Policy 显式机体系命令，但撤销外部 MOTION 微分前馈及其历史。 */
        motionFeedforwardDiffReset();
    }
}

```

```

        output->pitchFeedforwardRad = 0.0f;
        output->rollFeedforwardRad = 0.0f;
    }
    else
    {
        motionFeedforwardDiffUpdate(output->bodyVelocityFwdMps,
            output->bodyVelocityRightMps * FLIGHT_MOTION_VEL_FF_VY_TO_RIGHT_SIGN,
            sequenceContext.cycleSeq, facts->nowMs);
        output->pitchFeedforwardRad = motionFeedforwardDiffGetDvX() *
            sequenceParams.motionFFVxToPitch;
        output->rollFeedforwardRad = motionFeedforwardDiffGetDvY() *
            sequenceParams.motionFFVyToRoll;
    }
}

if ((output->safetyClampMask & (FLIGHT_SEQUENCE_CLAMP_SAFETY_STATE |
    FLIGHT_SEQUENCE_CLAMP_TOF_RAMP)) != 0u)
{
    output->visionValid = 0u;
    output->visionErrX = 0.0f;
    output->visionErrY = 0.0f;
    output->visionYawErr = 0.0f;
    output->bodyVelocityFwdMps = 0.0f;
    output->bodyVelocityRightMps = 0.0f;
    output->yawRateRadS = 0.0f;
    output->pitchFeedforwardRad = 0.0f;
    output->rollFeedforwardRad = 0.0f;
    motionFeedforwardDiffReset();
}

output->cycleSeq = sequenceContext.cycleSeq;
output->episodeSeq = sequenceContext.episodeSeq;
output->transitionSeq = sequenceContext.transitionSeq;
output->triggerSources |= sequenceContext.latchedTriggerSources;
if (output->primaryReason == CORE0_FAULT_REASON_NONE)
    output->primaryReason = sequenceContext.primaryReason;
output->valid = 1u;
}

```

core0_cyt4bb7/project/code/flight/flightSequenceCoordinator.h

SHA-256

:

d8054aca2a975fc67218e002275ac6f87b33811963d97c9ca2ba078b3a0bb888

代码 (c)

```

#ifndef FLIGHT_SEQUENCE_COORDINATOR_H_
#define FLIGHT_SEQUENCE_COORDINATOR_H_

#include <stdint.h>

#include "modeFlight.h"
#include "altitudeHold.h"

typedef enum
{
    FLIGHT_SEQUENCE_STAGE_IDLE = 0u,
    FLIGHT_SEQUENCE_STAGE_WAIT = 1u,
    FLIGHT_SEQUENCE_STAGE_RAMP = 2u,
    FLIGHT_SEQUENCE_STAGE_CRUISE = 3u,
    FLIGHT_SEQUENCE_STAGE_DESCENDING = 4u,
    FLIGHT_SEQUENCE_STAGE_LANDED = 5u
} flightSequenceStageEnum;

typedef enum
{
    FLIGHT_SEQUENCE_RESET_INIT = 0u,
    FLIGHT_SEQUENCE_RESET_DISARM = 1u,
    FLIGHT_SEQUENCE_RESET_PROTECT = 2u,
    FLIGHT_SEQUENCE_RESET_MODE = 3u,
    FLIGHT_SEQUENCE_RESET_REJOIN = 4u,
    FLIGHT_SEQUENCE_RESET_ABORT = 5u
} flightSequenceResetReasonEnum;

#define FLIGHT_SEQUENCE_TRIGGER_K5_LAUNCH    (1u << 0)

```



```

#define FLIGHT_SEQUENCE_TRIGGER_K5_STANDBY    (1u << 1)
#define FLIGHT_SEQUENCE_TRIGGER_LORA_MODE3    (1u << 2)
#define FLIGHT_SEQUENCE_TRIGGER_LORA_MODE2    (1u << 3)
#define FLIGHT_SEQUENCE_TRIGGER_CORE1_LOST    (1u << 4)
#define FLIGHT_SEQUENCE_TRIGGER_CORE2_LOST    (1u << 5)
#define FLIGHT_SEQUENCE_TRIGGER_REQUIRED_VISION_FAULT (1u << 6)
#define FLIGHT_SEQUENCE_TRIGGER_TOF_FAILSAFE  (1u << 7)
#define FLIGHT_SEQUENCE_TRIGGER_CORE2_ATTITUDE_FAULT (1u << 8)
#define FLIGHT_SEQUENCE_TRIGGER_POLICY_CUSTOM0 (1u << 16)
#define FLIGHT_SEQUENCE_TRIGGER_POLICY_CUSTOM1 (1u << 17)
#define FLIGHT_SEQUENCE_TRIGGER_POLICY_CUSTOM2 (1u << 18)
#define FLIGHT_SEQUENCE_TRIGGER_POLICY_CUSTOM3 (1u << 19)
#define FLIGHT_SEQUENCE_TRIGGER_POLICY_MASK    (0x00FF0000u)

#define FLIGHT_SEQUENCE_REASON_POLICY_CUSTOM0  (0x80u)
#define FLIGHT_SEQUENCE_REASON_POLICY_CUSTOM1  (0x81u)

#define FLIGHT_SEQUENCE_CLAMP_NONE              (0u)
#define FLIGHT_SEQUENCE_CLAMP_SAFETY_STATE      (1u << 0)
#define FLIGHT_SEQUENCE_CLAMP_TOF_RAMP          (1u << 1)
#define FLIGHT_SEQUENCE_CLAMP_LANDING           (1u << 2)
#define FLIGHT_SEQUENCE_CLAMP_POLICY_INVALID    (1u << 3)
#define FLIGHT_SEQUENCE_CLAMP_REQUIRED_LINK     (1u << 4)

/* 完整前台事实快照。每个外部事实保留独立 valid/age/seq, Policy 不得把它们合成总 valid。 */
typedef struct
{
    uint32_t nowMs;
    uint32_t callerCycleSeq;
    uint32_t dtMs;
    uint32_t coordinatorCycleSeq;
    uint32_t coordinatorTransitionSeq;
    uint32_t coordinatorEpisodeSeq;
    flightSafetySnapshotStruct safety;
    uint8_t initialized;
    uint8_t controlReady;
    uint8_t systemMode;
    uint8_t systemPhase;
    uint8_t standalone;
    uint8_t paramsValid;
    uint8_t readyBeepActive;

    uint8_t k5LaunchEdge;
    uint8_t k5StandbyEdge;
    uint8_t faultLatched;
    uint8_t rejoinEdge;

    uint8_t mode3RequestValid;
    uint8_t mode3RequestLevel;
    uint16_t mode3RequestAgeMs;
    uint32_t mode3RequestSeq;
    uint8_t mode3RequestArmed;
    uint8_t mode2RequestValid;
    uint8_t mode2RequestLevel;
    uint16_t mode2RequestAgeMs;
    uint32_t mode2RequestSeq;
    uint8_t mode2RequestArmed;

    uint8_t core1Online;
    uint8_t core1FactValid;
    uint8_t core2FactValid;
    uint8_t requiredVisionFactValid;
    uint8_t core2LinkLost;
    uint8_t core2AttitudeFault;
    uint8_t requiredVisionFault;
    uint8_t visionFaultSource;
    uint8_t visionFaultClass;
    uint16_t core1AgeMs;
    uint16_t core2AgeMs;
    uint16_t requiredVisionAgeMs;
    uint32_t linkSeq;
    uint32_t core1Seq;
    uint32_t core2Seq;
    uint32_t requiredVisionSeq;

    altitudeHoldSnapshotStruct altitude;

    float visionErrX;
    float visionErrY;

```

```

float visionYawErr;
uint8_t visionTargetValid;
uint8_t visionFrameValid;
uint16_t visionAgeMs;
uint16_t targetLostAgeMs;
uint32_t visionSeq;

/* ^ 灯板原始位置事实，来自独立视觉调试快照。 */
float chevronX;
float chevronY;
float chevronOrientationRad;
float chevronOrientationConfidence;
uint8_t chevronPositionValid;
uint8_t chevronOrientationValid;
uint16_t chevronAgeMs;
uint32_t chevronSeq;

/* 信标/寻灯状态由 Core1 STATUS 的独立业务新鲜度发布。 */
uint16_t beaconCount;
uint8_t beaconAny;
uint8_t beaconSearching;
uint8_t beaconSearchTimedOut;
uint8_t beaconStateValid;
uint16_t beaconStateAgeMs;
uint32_t beaconStateSeq;

float motionVx;
float motionVy;
float motionWz;
uint8_t motionValid;
uint8_t motionFenceOpen;
uint16_t motionAgeMs;
uint32_t motionSeq;

float kfVelocityFwdMps;
float kfVelocityRightMps;
uint8_t kfVelocityValid;
} flightSequenceFactsStruct;

typedef struct
{
    uint32_t cycleSeq;
    uint32_t episodeSeq;
    uint32_t transitionSeq;
    uint32_t triggerSources;
    uint32_t rejectedOverrideMask;
    uint32_t safetyClampMask;
    uint32_t stageElapsedMs;
    flightSequenceStageEnum stage;
    uint8_t primaryReason;
    uint8_t valid;
    uint8_t requestLaunch;
    uint8_t requestDisarm;
    uint8_t acknowledgeFault;
    uint8_t landingActive;
    uint8_t altitudeControlEnable;
    uint8_t idleThrottleMode;
    uint8_t altReached;
    uint8_t rgbIntent;
    uint8_t buzzerPattern;
    float altitudeTargetMm;

    /* 最终引导命令；调用层只允许机械提交，不得再解释来源或组合门。 */
    float visionErrX;
    float visionErrY;
    float visionYawErr;
    uint8_t visionValid;
    uint8_t visionHeld;
    uint8_t allowVisionPosition;
    uint8_t allowExternalMotion;
    float bodyVelocityFwdMps;
    float bodyVelocityRightMps;
    float yawRateRadS;
    float pitchFeedforwardRad;
    float rollFeedforwardRad;
    uint8_t policyPositionActive;

    altitudeHoldSnapshotStruct altitude;
} flightSequenceOutputStruct;

```

```

void flightSequenceInit(uint32_t nowMs);
void flightSequenceApplyParams(void);
void flightSequenceStep(const flightSequenceFactsStruct *facts,
    flightSequenceOutputStruct *output);
void flightSequenceReset(flightSequenceResetReasonEnum reason);

```

```

#endif /* _FLIGHT_SEQUENCE_COORDINATOR_H_ */

```

core0_cyt4bb7/project/code/flight/flightSequencePolicy.c

SHA-256

:

c6b6b5ec97a49d44c4c22cce41717a25db099a9dabf038338a9f7701e716cf5e

代码 (c)

```

#include "flightSequencePolicyInternal.h"

```

```

#include <string.h>

```

```

#include "configFlightSequence.h"
#include "coreDataTransfer.h"
#include "uartLinkProto.h"

```

```

/*
 * 现场唯一编辑面：触发选择与阶段位置命令均集中在本文件。
 * 不注册回调、不使用 weak/动态内存，也不进入 ISR。
 */

```

```

typedef struct
{
    float descentTargetMm;
    uint8_t descentPrimed;
    uint8_t descentReanchorPending;
} flightSequencePolicyPrivateStruct;

```

```

static flightSequencePolicyPrivateStruct policyPrivate;

```

```

static inline void flightSequencePolicySlowRise(
    const flightSequenceFactsStruct *facts,
    const flightSequencePolicyContextStruct *context,
    flightSequencePolicyDecisionStruct *decision)
{
    float fraction;
    (void) facts;
    fraction = (TAKEOFF_RAMP_MS > 0u) ?
        ((float) context->stageElapsedMs / (float) TAKEOFF_RAMP_MS) : 1.0f;
    if (fraction > 1.0f) fraction = 1.0f;
    decision->overrideMask |= FLIGHT_SEQUENCE_POLICY_OVERRIDE_ALTITUDE |
        FLIGHT_SEQUENCE_POLICY_OVERRIDE_POSITION |
        FLIGHT_SEQUENCE_POLICY_OVERRIDE_RGB |
        FLIGHT_SEQUENCE_POLICY_OVERRIDE_BUZZER |
        FLIGHT_SEQUENCE_POLICY_OVERRIDE_REACHED;
    decision->altitudeTargetMm = context->stageEntryAltitudeMm + fraction *
        (context->cruiseAltitudeMm - context->stageEntryAltitudeMm);
    decision->bodyVelocityFwdMps = 0.0f;
    decision->bodyVelocityRightMps = 0.0f;
    decision->yawRateRadS = 0.0f;
    decision->allowVisionPosition = 0u;
    decision->allowExternalMotion = 1u;
    decision->localPositionValid = 0u; /* 默认保留 HEAD 的非巡航角度/yaw 前馈；改为 1 即可启用本地命令并排他接管。 */
    decision->rgbIntent = UART_LINK_RGB_TKO_RAMP;
    decision->buzzerPattern = UART_LINK_BUZZER_1HZ;
    decision->altReached = 0u;
    decision->overrideMask |= FLIGHT_SEQUENCE_POLICY_OVERRIDE_CONTROL;
    decision->altitudeControlEnable = 1u;
    decision->idleThrottleMode = 0u;
}

```

```

static inline void flightSequencePolicySlowDescent(
    const flightSequenceFactsStruct *facts,
    const flightSequencePolicyContextStruct *context,
    flightSequencePolicyDecisionStruct *decision)
{
    if (policyPrivate.descentPrimed == 0u)
    {

```

```

    policyPrivate.descentTargetMm = context->stageEntryAltitudeMm;
    policyPrivate.descentPrimed = 1u;
    policyPrivate.descentReanchorPending = facts->altitude.healthy ? 0u : 1u;
}
if (facts->altitude.healthy == 0u)
{
    policyPrivate.descentReanchorPending = 1u;
}
else if (policyPrivate.descentReanchorPending != 0u)
{
    policyPrivate.descentTargetMm = (float) facts->altitude.distanceMm;
    policyPrivate.descentReanchorPending = 0u;
}
else
{
    policyPrivate.descentTargetMm -= LAND_DESCEND_RATE_MM_S *
        ((float) context->dtMs / 1000.0f);
    if (policyPrivate.descentTargetMm < LAND_FLOOR_MM)
        policyPrivate.descentTargetMm = LAND_FLOOR_MM;
}

decision->overrideMask |= FLIGHT_SEQUENCE_POLICY_OVERRIDE_ALTITUDE |
    FLIGHT_SEQUENCE_POLICY_OVERRIDE_POSITION |
    FLIGHT_SEQUENCE_POLICY_OVERRIDE_RGB |
    FLIGHT_SEQUENCE_POLICY_OVERRIDE_BUZZER |
    FLIGHT_SEQUENCE_POLICY_OVERRIDE_REACHED;
decision->altitudeTargetMm = policyPrivate.descentTargetMm;
decision->bodyVelocityFwdMps = 0.0f;
decision->bodyVelocityRightMps = 0.0f;
decision->yawRateRadS = 0.0f;
decision->allowVisionPosition = 0u;
decision->allowExternalMotion = 0u;
decision->localPositionValid = 1u;
decision->rgbIntent = UART_LINK_RGB_DISCONNECT_WARN;
decision->buzzerPattern = UART_LINK_BUZZER_WARN;
decision->altReached = 0u;
decision->overrideMask |= FLIGHT_SEQUENCE_POLICY_OVERRIDE_CONTROL;
decision->altitudeControlEnable = 1u;
decision->idleThrottleMode = 0u;
/* 目标到地板只表示继续保持最低高度，不能证明飞机已经触地。
 * 仅允许健康 ToF 的实测高度触发停桨，避免实际下降跟随滞后时高空 disarm。 */
if (facts->altitude.healthy &&
    ((float) facts->altitude.distanceMm <= LAND_TOUCHDOWN_MM))
{
    decision->touchdown = 1u;
}
}

void flightSequencePolicyReset(flightSequenceResetReasonEnum reason)
{
    (void) reason;
    memset(&policyPrivate, 0, sizeof(policyPrivate));
}

void flightSequencePolicySelectTriggers(const flightSequenceFactsStruct *facts,
    const flightSequencePolicyContextStruct *context,
    flightSequencePolicyDecisionStruct *decision)
{
    if ((facts == 0) || (context == 0) || (decision == 0)) return;

    if (facts->k5LaunchEdge != 0u)
    {
        decision->actionMask |= FLIGHT_SEQUENCE_POLICY_ACTION_ARM;
        decision->triggerSources |= FLIGHT_SEQUENCE_TRIGGER_K5_LAUNCH;
        decision->triggerEventSeq = facts->coordinatorCycleSeq;
        decision->triggerEventValid = 1u;
        if (facts->systemMode != (uint8_t) UART_LINK_SYS_MODE_LORA_FWD)
            decision->actionMask |= FLIGHT_SEQUENCE_POLICY_ACTION_TAKEOFF;
    }
    if (context->mode3RequestEvent != 0u)
    {
        decision->triggerSources |= FLIGHT_SEQUENCE_TRIGGER_LORA_MODE3;
        decision->triggerEventSeq = facts->mode3RequestSeq;
        decision->triggerEventValid = 1u;
        if (context->stage == FLIGHT_SEQUENCE_STAGE_IDLE)
            decision->actionMask |= FLIGHT_SEQUENCE_POLICY_ACTION_TAKEOFF;
        else
        {
            decision->actionMask |= FLIGHT_SEQUENCE_POLICY_ACTION_LAND;

```

```

        decision->primaryReason = CORE0_FAULT_REASON_LAND_LORA_MODE3;
    }
}
if ((context->mode2RequestEvent != 0u) &&
    (context->stage != FLIGHT_SEQUENCE_STAGE_IDLE))
{
    decision->actionMask |= FLIGHT_SEQUENCE_POLICY_ACTION_LAND;
    decision->triggerSources |= FLIGHT_SEQUENCE_TRIGGER_LORA_MODE2;
    decision->triggerEventSeq = facts->mode2RequestSeq;
    decision->triggerEventValid = 1u;
    decision->primaryReason = CORE0_FAULT_REASON_LAND_LORA_MODE2;
}
}

void flightSequencePolicyApplyStage(const flightSequenceFactsStruct *facts,
    const flightSequencePolicyContextStruct *context,
    flightSequencePolicyDecisionStruct *decision)
{
    if ((facts == 0) || (context == 0) || (decision == 0)) return;
    if (context->stage == FLIGHT_SEQUENCE_STAGE_RAMP)
        flightSequencePolicySlowRise(facts, context, decision);
    else if (context->stage == FLIGHT_SEQUENCE_STAGE_DESCENDING)
        flightSequencePolicySlowDescent(facts, context, decision);
}

```

core0_cyt4bb7/project/code/flight/flightSequencePolicyInternal.h

SHA-256

:

8566bace2c8e8d2e2ade6445369d2b23ca71cd62ae2f78cd8ff6bd152d5242ca

代码 (c)

```

#ifndef FLIGHT_SEQUENCE_POLICY_INTERNAL_H_
#define FLIGHT_SEQUENCE_POLICY_INTERNAL_H_

#include "flightSequenceCoordinator.h"

#define FLIGHT_SEQUENCE_POLICY_ACTION_ARM    (1u << 0)
#define FLIGHT_SEQUENCE_POLICY_ACTION_TAKEOFF (1u << 1)
#define FLIGHT_SEQUENCE_POLICY_ACTION_LAND   (1u << 2)

#define FLIGHT_SEQUENCE_POLICY_OVERRIDE_ALTITUDE (1u << 0)
#define FLIGHT_SEQUENCE_POLICY_OVERRIDE_POSITION (1u << 1)
#define FLIGHT_SEQUENCE_POLICY_OVERRIDE_RGB      (1u << 2)
#define FLIGHT_SEQUENCE_POLICY_OVERRIDE_BUZZER   (1u << 3)
#define FLIGHT_SEQUENCE_POLICY_OVERRIDE_REACHED  (1u << 4)
#define FLIGHT_SEQUENCE_POLICY_OVERRIDE_DISARM   (1u << 5)
#define FLIGHT_SEQUENCE_POLICY_OVERRIDE_CONTROL (1u << 6)
#define FLIGHT_SEQUENCE_POLICY_OVERRIDE_TRANSITION (1u << 7)

typedef struct
{
    flightSequenceStageEnum stage;
    uint32_t stageElapsedMs;
    uint32_t dtMs;
    float stageEntryAltitudeMm;
    float cruiseAltitudeMm;
    uint8_t mode3RequestEvent;
    uint8_t mode2RequestEvent;
} flightSequencePolicyContextStruct;

typedef struct
{
    uint32_t actionMask;
    uint32_t triggerSources;
    uint32_t triggerEventSeq;
    uint32_t overrideMask;
    float altitudeTargetMm;
    float bodyVelocityFwdMps;
    float bodyVelocityRightMps;
    float yawRateRadS;
    uint8_t allowVisionPosition;
    uint8_t allowExternalMotion;
    uint8_t localPositionValid;
    uint8_t rgbIntent;
}

```

```

uint8_t buzzerPattern;
uint8_t altReached;
uint8_t requestDisarm;
uint8_t touchdown;
uint8_t altitudeControlEnable;
uint8_t idleThrottleMode;
uint8_t stageTransitionReady;
uint8_t triggerEventValid;
uint8_t primaryReason;
} flightSequencePolicyDecisionStruct;

void flightSequencePolicyReset(flightSequenceResetReasonEnum reason);
void flightSequencePolicySelectTriggers(const flightSequenceFactsStruct *facts,
    const flightSequencePolicyContextStruct *context,
    flightSequencePolicyDecisionStruct *decision);
void flightSequencePolicyApplyStage(const flightSequenceFactsStruct *facts,
    const flightSequencePolicyContextStruct *context,
    flightSequencePolicyDecisionStruct *decision);

#endif /* _FLIGHT_SEQUENCE_POLICY_INTERNAL_H_ */

```

core0_cyt4bb7/project/code/flight/IMU.c

SHA-256

:

d604818e51ce8401b3a87d061d66f722d06165a27150778394951e121d8d7cef

代码 (c)

```

#include "IMU.h"

#include "zf_driver_delay.h"
#include "zf_common_interrupt.h"
#include "zf_device_imu660ra.h"

#include "commonDef.h"
#include "lowPassFilter.h"
#include "flightParams.h"
#include "mahony.h"

/* imu660ra: ±8G / ±2000DPS 默认配置（与驱动 IMU660RA_ SAMPLE_DEFAULT 保持一致）
 * 加速度满量程 ±8g → 32768 LSB / 8g → 4096 LSB/g → 1g = 9.81 m/s^2
 * 增益 = 9.81 / 4096 ≈ 0.002395 m/s^2 / LSB
 * 陀螺仪满量程 ±2000DPS → 32768 LSB / 2000 → 16.384 LSB/(°/s)
 * 增益 = (1/16.384) * π/180 rad/s/LSB
 */
#define IMU660RA_ACC_GAIN_MPS2 (9.81f / 4096.0f)
#define IMU660RA_GYRO_GAIN_RADPS ((1.0f / 16.384f) * (PI / 180.0f))

#define IMUObjectCount (2000)
#define IMUObjectDelayMs (2u)
#define IMU_FREEZE_TIMEOUT_MS (100u)
#define IMU_FREEZE_RECOVERY_SAMPLES (20u)

typedef struct
{
    volatile _6IMUDATA data;
    volatile uint32_t snapshotSeq;
    volatile uint8_t snapshotValid;
    volatile uint8_t healthReason;
    volatile uint16_t unchangedMs;
    uint8_t recoverySamples;
    uint8_t rawPrimed;
    int16_t lastRaw[6];
    int16_t gyroXOffset;
    int16_t gyroYOffset;
    int16_t gyroZOffset;
    float gyroXRaw;
    float gyroYRaw;
    float gyroZRaw;
    volatile float gyroXFiltered;
    volatile float gyroYFiltered;
    volatile float gyroZFiltered;
    float accX;
    float accY;
    float accZ;
}

```

```

    LowPassFilterData gyroLpfX;
    LowPassFilterData gyroLpfY;
    LowPassFilterData gyroLpfZ;
} imuContextStruct;

static imuContextStruct imuContext;

#define imuData      imuContext.data
#define imuSnapshotSeq  imuContext.snapshotSeq
#define imuSnapshotValid imuContext.snapshotValid
#define imuHealthReason  imuContext.healthReason
#define imuUnchangedMs  imuContext.unchangedMs
#define gyroXOffset     imuContext.gyroXOffset
#define gyroYOffset     imuContext.gyroYOffset
#define gyroZOffset     imuContext.gyroZOffset
#define gyroXRaw        imuContext.gyroXRaw
#define gyroYRaw        imuContext.gyroYRaw
#define gyroZRaw        imuContext.gyroZRaw
#define gyroXFiltered   imuContext.gyroXFiltered
#define gyroYFiltered   imuContext.gyroYFiltered
#define gyroZFiltered   imuContext.gyroZFiltered
#define accX            imuContext.accX
#define accY            imuContext.accY
#define accZ            imuContext.accZ
#define gyroLpfX        imuContext.gyroLpfX
#define gyroLpfY        imuContext.gyroLpfY
#define gyroLpfZ        imuContext.gyroLpfZ

static void getOffset(void);

uint8_t IMUInit(void)
{
    uint8_t imuInitState = imu660ra_init();
    flightParamsStruct params;

    imuSnapshotValid = 0u;
    imuHealthReason = IMU_HEALTH_OK;
    imuUnchangedMs = 0u;
    imuContext.recoverySamples = 0u;
    imuContext.rawPrimed = 0u;
    imuSnapshotSeq = 0u;
    if (imuInitState) return imuInitState;

    getOffset();

    flightParamsGetActive(&params);
    lowPassFilterInit(&gyroLpfX, params.gyroLpfAlpha);
    lowPassFilterInit(&gyroLpfY, params.gyroLpfAlpha);
    lowPassFilterInit(&gyroLpfZ, params.gyroLpfAlpha);

    mahony6Init((_6IMUDATA *) &imuData, &gyroXRaw, &gyroYRaw, &gyroZRaw, &accX, &accY, &accZ);
    imuSnapshotValid = 1u;

    return imuInitState;
}

void IMUApplyParams(void)
{
    flightParamsStruct params;
    flightParamsGetActive(&params);
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    gyroLpfX.alpha = params.gyroLpfAlpha;
    gyroLpfY.alpha = params.gyroLpfAlpha;
    gyroLpfZ.alpha = params.gyroLpfAlpha;
    __set_PRIMASK(primask);
}

inline void IMUUpdate(void)
{
    uint8_t changed;
    imu660ra_get_gyro();
    imu660ra_get_acc();

    changed = (uint8_t)((imuContext.rawPrimed == 0u) ||
        (imu660ra_gyro_x != imuContext.lastRaw[0]) ||
        (imu660ra_gyro_y != imuContext.lastRaw[1]) ||
        (imu660ra_gyro_z != imuContext.lastRaw[2]) ||
        (imu660ra_acc_x != imuContext.lastRaw[3]) ||

```

```

        (imu660ra_acc_y != imuContext.lastRaw[4]) ||
        (imu660ra_acc_z != imuContext.lastRaw[5]));
if (changed != 0u)
{
    imuContext.lastRaw[0] = imu660ra_gyro_x;
    imuContext.lastRaw[1] = imu660ra_gyro_y;
    imuContext.lastRaw[2] = imu660ra_gyro_z;
    imuContext.lastRaw[3] = imu660ra_acc_x;
    imuContext.lastRaw[4] = imu660ra_acc_y;
    imuContext.lastRaw[5] = imu660ra_acc_z;
    imuContext.rawPrimed = 1u;
    imuUnchangedMs = 0u;
    if (imuHealthReason == IMU_HEALTH_FROZEN)
    {
        if (imuContext.recoverySamples < IMU_FREEZE_RECOVERY_SAMPLES)
            imuContext.recoverySamples++;
        if (imuContext.recoverySamples >= IMU_FREEZE_RECOVERY_SAMPLES)
        {
            imuHealthReason = IMU_HEALTH_OK;
            imuSnapshotValid = 1u;
            imuContext.recoverySamples = 0u;
        }
    }
}
else
{
    if (imuUnchangedMs < IMU_FREEZE_TIMEOUT_MS) imuUnchangedMs++;
    if (imuUnchangedMs >= IMU_FREEZE_TIMEOUT_MS)
    {
        imuHealthReason = IMU_HEALTH_FROZEN;
        imuSnapshotValid = 0u;
        imuContext.recoverySamples = 0u;
    }
}

gyroXRaw = (float) (imu660ra_gyro_x - gyroXOffset) * IMU660RA_GYRO_GAIN_RADPS;
gyroYRaw = (float) (imu660ra_gyro_y - gyroYOffset) * IMU660RA_GYRO_GAIN_RADPS;
gyroZRaw = (float) (imu660ra_gyro_z - gyroZOffset) * IMU660RA_GYRO_GAIN_RADPS;

lowPassFilter(&gyroLpfX, gyroXRaw);
lowPassFilter(&gyroLpfY, gyroYRaw);
lowPassFilter(&gyroLpfZ, gyroZRaw);

gyroXFiltered = gyroLpfX.filteredValue;
gyroYFiltered = gyroLpfY.filteredValue;
gyroZFiltered = gyroLpfZ.filteredValue;

accX = (float) imu660ra_acc_x * IMU660RA_ACC_GAIN_MPS2;
accY = (float) imu660ra_acc_y * IMU660RA_ACC_GAIN_MPS2;
accZ = (float) imu660ra_acc_z * IMU660RA_ACC_GAIN_MPS2;

mahony6Update((_6IMUDATA *) &imuData);
imuSnapshotSeq++;
}

void IMUGetSnapshot(imuSnapshotStruct *out)
{
    uint32_t primask;
    uint8_t i;
    if (out == 0) return;

    primask = __get_PRIMASK();
    __disable_irq();
    out->seq = imuSnapshotSeq;
    out->valid = imuSnapshotValid;
    out->healthReason = imuHealthReason;
    out->unchangedMs = imuUnchangedMs;
    for (i = 0u; i < 4u; i++) out->q[i] = imuData.q[i];
    out->physicalPitch = imuData.roll;
    out->physicalRoll = imuData.pitch;
    out->yaw = imuData.yaw;
    out->gyroPitchRaw = gyroXRaw;
    out->gyroRollRaw = gyroYRaw;
    out->gyroYawRaw = gyroZRaw;
    out->gyroPitchFiltered = gyroXFiltered;
    out->gyroRollFiltered = gyroYFiltered;
    out->gyroYawFiltered = gyroZFiltered;
    out->accelSensorX = accX;
    out->accelSensorY = accY;

```



```

        out->accelSensorZ = accZ;
        __set_PRIMASK(primask);
    }

static void getOffset(void)
{
    system_delay_ms(500u);

    int xTemp = 0, yTemp = 0, zTemp = 0;

    for (uint16_t i = 0; i < IMUOffsetCount; i++)
    {
        imu660ra_get_gyro();

        xTemp += (int) imu660ra_gyro_x;
        yTemp += (int) imu660ra_gyro_y;
        zTemp += (int) imu660ra_gyro_z;

        system_delay_ms(IMUOffsetDelayMs);
    }

    gyroXOffset = (int16_t) (xTemp / IMUOffsetCount);
    gyroYOffset = (int16_t) (yTemp / IMUOffsetCount);
    gyroZOffset = (int16_t) (zTemp / IMUOffsetCount);
}

```

core0_cyt4bb7/project/code/flight/IMU.h

SHA-256

:

9403dba93679ccf55d41fae5df5e29fc0c444ddfca6d04896aa90624f9573712

代码 (c)

```

#ifndef _IMU_H_
#define _IMU_H_

#include <stdint.h>

/*
 * core0 IMU (imu660ra) 装配方向 → 物理姿态映射 (机械零点均为 0°)
 * 物理 pitch (前倾正) ← imuData.roll 与 *imuData.gx
 * 物理 roll (左倾正) ← imuData.pitch 与 *imuData.gy
 * 物理 yaw (CW 为负) ← imuData.yaw 与 *imuData.gz
 *
 * Mahony 解算逻辑保持不变, 只在使用层 (pidFlightController / core0Loop) 按上述映射重新绑定。
 *
 * Gyro 双路:
 * - imuData.gx/gy/gz 指针指向 raw (未滤波) gyro, 喂给 Mahony, 避免低通带来的相位滞后。
 * - imuGyroFilteredX/Y/Z 指针指向 1ms 一阶 LPF 滤波后的 gyro, 专供角速度内环 PID 使用。
 * LPF 系数由运行时 flightParams.gyroLpfAlpha 控制, 可经 IMUApplyParams() 热重载。
 */
typedef struct
{
    uint32_t seq;
    uint8_t valid;
    uint8_t healthReason;
    uint16_t unchangedMs;
    float q[4];
    float physicalPitch;
    float physicalRoll;
    float yaw;
    float gyroPitchRaw;
    float gyroRollRaw;
    float gyroYawRaw;
    float gyroPitchFiltered;
    float gyroRollFiltered;
    float gyroYawFiltered;
    float accelSensorX;
    float accelSensorY;
    float accelSensorZ;
} imuSnapshotStruct;

#define IMU_HEALTH_OK (0u)
#define IMU_HEALTH_FROZEN (1u)

```

```
/* 一次短临界区返回同拍姿态、角速度和加速度；调用方不再依赖 Mahony/厂商类型。 */
void IMUGetSnapshot(imuSnapshotStruct *out);
```

```
uint8_t IMUInit(void);
void IMUApplyParams(void);
void IMUUpdate(void);
```

```
#endif // _IMU_H_
```

core0_cyt4bb7/project/code/flight/mahony.c

SHA-256

:

94857297a91065efa204fe1d479bc0613bb088b8ea10af587480584865751d6a

代码 (c)

```
/*
 * mahony.c
 *
 * Created: 20 250 116
 * Author: Hecate
 */
#include "mahony.h"

#include <math.h>
#include <stddef.h>

//-----
// Definitions

#define sampleFreq 1000.0f // sample frequency in Hz
#define twoKpDef (1.0f) // 2 * proportional gain
#define twoKiDef (0.0f) // 2 * integral gain

//-----
// Variable definitions

static volatile float twoKp = twoKpDef; // 2 * proportional gain (Kp)
static volatile float twoKi = twoKiDef; // 2 * integral gain (Ki)
static volatile float integralFBx = 0.0f, integralFBy = 0.0f, integralFBz = 0.0f; // integral error terms scaled by Ki

//-----
// Function declarations

static inline float invSqrt(float x);

#ifndef M_PI
#define M_PI 3.14159265358979323846f
#endif

#ifndef M_2PI
#define M_2PI 6.28318530717958647692f
#endif

static void MahonyAHRSupdateIMU(float q[4], float gx, float gy, float gz, float ax, float ay, float az);

void mahony6Init(_6IMUDATA* obj, float* gx, float* gy, float* gz, float* ax, float* ay, float* az)
{
    obj->q[0] = 1.0f;
    obj->q[1] = 0.0f;
    obj->q[2] = 0.0f;
    obj->q[3] = 0.0f;
    obj->gx = gx;
    obj->gy = gy;
    obj->gz = gz;
    obj->ax = ax;
    obj->ay = ay;
    obj->az = az;

    obj->pitch = 0.0f;
    obj->roll = 0.0f;
    obj->yaw = 0.0f;
}

static inline void quaternionToEuler(const float q[4], float* pitch, float* roll, float* yaw);
```

```

static volatile const float yawfre = (1.0f / sampleFreq);
void mahony6Update( _6IMUDATA* obj)
{
    MahonyAHRSupdateIMU(obj->q, *(obj->gx), *(obj->gy), *(obj->gz), *(obj->ax), *(obj->ay), *(obj->az));

    quaternionToEuler(obj->q, &(obj->pitch), &(obj->roll), &(obj->yaw));
}

//=====================================================
// Functions
//-----
// IMU algorithm update

static void MahonyAHRSupdateIMU(float q[4], float gx, float gy, float gz, float ax, float ay, float az)
{
    float recipNorm;
    float halfvx, halfvy, halfvz;
    float halfex, halfey, halfez;
    float qa, qb, qc;

    static float half_errorInv;
    static int count = 0;
    /******参数规划*****/

    if (count < 3000)
    {
        count += 2;
        twoKp = 20;
    }
    else
    {
        if (half_errorInv < 20)
            twoKp = 0.1f;
        else
            twoKp = 4;
    }

    /******

    // Compute feedback only if accelerometer measurement valid (avoids NaN in accelerometer normalisation)
    if (!((ax == 0.0f) && (ay == 0.0f) && (az == 0.0f)))
    {
        // Normalise accelerometer measurement
        recipNorm = invSqrt(ax * ax + ay * ay + az * az);
        ax *= recipNorm;
        ay *= recipNorm;
        az *= recipNorm;

        // Estimated direction of gravity and vector perpendicular to magnetic flux
        halfvx = q[1] * q[3] - q[0] * q[2];
        halfvy = q[0] * q[1] + q[2] * q[3];
        halfvz = q[0] * q[0] - 0.5f + q[3] * q[3];

        // Error is sum of cross product between estimated and measured direction of gravity
        halfex = (ay * halfvz - az * halfvy);
        halfey = (az * halfvx - ax * halfvz);
        halfez = (ax * halfvy - ay * halfvx);
        half_errorInv = invSqrt(halfex * halfex + halfey * halfey + halfez * halfez);
        // Compute and apply integral feedback if enabled
        if (twoKi > 0.0f)
        {
            integralFBx += twoKi * halfex * (1.0f / sampleFreq); // integral error scaled by Ki
            integralFBy += twoKi * halfey * (1.0f / sampleFreq);
            integralFBz += twoKi * halfez * (1.0f / sampleFreq);
            gx += integralFBx; // apply integral feedback
            gy += integralFBy;
            gz += integralFBz;
        }
        else
        {
            integralFBx = 0.0f; // prevent integral windup
            integralFBy = 0.0f;
            integralFBz = 0.0f;
        }

        // Apply proportional feedback
        gx += twoKp * halfex;
        gy += twoKp * halfey;

```

```

        gz += twoKp * halfez;
    }

    // Integrate rate of change of quaternion
    gx *= (0.5f * (1.0f / sampleFreq)); // pre-multiply common factors
    gy *= (0.5f * (1.0f / sampleFreq));
    gz *= (0.5f * (1.0f / sampleFreq));
    qa = q[0];
    qb = q[1];
    qc = q[2];
    q[0] += (-qb * gx - qc * gy - q[3] * gz);
    q[1] += (qa * gx + qc * gz - q[3] * gy);
    q[2] += (qa * gy - qb * gz + q[3] * gx);
    q[3] += (qa * gz + qb * gy - qc * gx);

    // Normalise quaternion
    recipNorm = invSqrt(q[0] * q[0] + q[1] * q[1] + q[2] * q[2] + q[3] * q[3]);
    q[0] *= recipNorm;
    q[1] *= recipNorm;
    q[2] *= recipNorm;
    q[3] *= recipNorm;
}

//-----
// Fast inverse square-root
// See: http://en.wikipedia.org/wiki/Fast_inverse_square_root

static inline float invSqrt(float x)
{
    float halfx = 0.5f * x;
    float y = x;
    long i = *(long*) &y;
    i = 0x5f3759df - (i >> 1);
    y = *(float*) &i;
    y = y * (1.5f - (halfx * y * y));
    return y;
}

static inline void quaternionToEuler(const float q[4], float* pitch, float* roll, float* yaw)
{
    if (q == NULL)
        return;

    const float q0 = q[0]; // w
    const float q1 = q[1]; // x
    const float q2 = q[2]; // y
    const float q3 = q[3]; // z

    const float sinr_cosp = 2.0f * (q0 * q1 + q2 * q3);
    const float cosr_cosp = 1.0f - 2.0f * (q1 * q1 + q2 * q2);
    *roll = atan2f(sinr_cosp, cosr_cosp);

    float sinp = 2.0f * (q0 * q2 - q1 * q3);
    if (sinp > 1.0f)
        sinp = 1.0f;
    else if (sinp < -1.0f)
        sinp = -1.0f;
    *pitch = asinf(sinp);

    const float siny_cosp = 2.0f * (q0 * q3 + q1 * q2);
    const float cosy_cosp = 1.0f - 2.0f * (q2 * q2 + q3 * q3);
    *yaw = atan2f(siny_cosp, cosy_cosp);
}

```

core0_cyt4bb7/project/code/flight/mahony.h

SHA-256

:

e4b1f615b8df284ccdb010fdca7e99a0dbcd a0bc2d73e92e2d9f65071f597a1a

代码 (c)

```

/*
 * mahony.h
 *
 * Created: 20 250 116

```

```

* Author: Hecate
*/

#ifndef CODE_MAHONY_H_
#define CODE_MAHONY_H_

// Mahony 滤波以 1 ms 周期调用，角速度单位为弧度每秒

// 6 轴 IMU 姿态解算数据
typedef struct
{
    float q[4]; // quaternion
    float *gx, *gy, *gz; // gyroscope rates
    float *ax, *ay, *az; // accelerometer readings
    float pitch, roll, yaw; // Euler angles
} _6IMUDATA;

// 初始化 6 轴 Mahony 滤波器
void mahony6Init(_6IMUDATA* obj, float* gx, float* gy, float* gz, float* ax, float* ay, float* az);

// 更新 6 轴 Mahony 滤波器并更新姿态
void mahony6Update(_6IMUDATA* obj);

#endif /* CODE_MAHONY_H_ */

```

core0_cyt4bb7/project/code/flight/modeFlight.c

SHA-256

:

9fb3ff19541c9b2845c43b6c412d0f6f4ade55ba1bfc29635f266ff804a88a99

代码 (c)

```

#include "modeFlight.h"

#include "IMU.h"
#include "pidFlightController.h"
#include "altitudeHold.h"
#include "flightParams.h"
#include "commonDef.h"
#include "zf_common_interrupt.h"

/*
 * core0 定高飞控安全状态机
 * cm_7_0 不拥有 IPS114，LCD 显示由 cm_7_1 经共享内存完成，本文件不含任何 ips114_*。
 */

typedef struct
{
    float pitchMin;
    float pitchMax;
    float rollMin;
    float rollMax;
    float gyroMax;
} modeFlightConfigStruct;

typedef struct
{
    volatile safetyStateEnum safetyState;
    volatile uint8_t launchPending;
    volatile uint8_t disarmPending;
    volatile uint8_t protectionPending;
    volatile uint32_t seq;
    volatile uint32_t transitionSeq;
    volatile uint32_t launchAcceptedSeq;
    volatile uint32_t disarmAcceptedSeq;
    volatile uint32_t protectAcceptedSeq;
    volatile uint8_t lastPostureSafe;
    volatile uint8_t lastToffHealthy;
    volatile uint8_t configUpdateActive;
    modeFlightConfigStruct config;
} flightSafetyContextStruct;

static flightSafetyContextStruct safetyContext = {
    .safetyState = SAFETY_STATE_DISARMED
};

```

```

#define flightSafetyState    safetyContext.safetyState
#define launchRequestPending safetyContext.launchPending
#define disarmRequestPending safetyContext.disarmPending
#define protectionRequestPending safetyContext.protectionPending
#define flightSafetySeq     safetyContext.seq
#define flightTransitionSeq  safetyContext.transitionSeq
#define flightLaunchAcceptedSeq safetyContext.launchAcceptedSeq
#define flightDisarmAcceptedSeq safetyContext.disarmAcceptedSeq
#define flightProtectAcceptedSeq safetyContext.protectAcceptedSeq
#define lastPostureSafe     safetyContext.lastPostureSafe
#define lastTofHealthy      safetyContext.lastTofHealthy
#define modeConfig          safetyContext.config

static inline uint8_t postureSafeCheck(const imuSnapshotStruct* imu)
{
    float physicalPitch = imu->physicalPitch;
    float physicalRoll = imu->physicalRoll;
    if (!imu->valid) return 0u;
    if ((floatIsFinite(physicalPitch) == 0u) || (floatIsFinite(physicalRoll) == 0u) ||
        (floatIsFinite(modeConfig.pitchMin) == 0u) ||
        (floatIsFinite(modeConfig.pitchMax) == 0u) ||
        (floatIsFinite(modeConfig.rollMin) == 0u) ||
        (floatIsFinite(modeConfig.rollMax) == 0u) ||
        (floatIsFinite(modeConfig.gyroMax) == 0u)) return 0u;
    if (physicalPitch < modeConfig.pitchMin || physicalPitch > modeConfig.pitchMax ||
        physicalRoll < modeConfig.rollMin || physicalRoll > modeConfig.rollMax)
    {
        return 0u;
    }

    float gx = imu->gyroPitchRaw;
    float gy = imu->gyroRollRaw;
    float gz = imu->gyroYawRaw;
    float gyroMax = modeConfig.gyroMax;
    if ((floatIsFinite(gx) == 0u) || (floatIsFinite(gy) == 0u) ||
        (floatIsFinite(gz) == 0u)) return 0u;
    if (gx > gyroMax || gx < -gyroMax ||
        gy > gyroMax || gy < -gyroMax ||
        gz > gyroMax || gz < -gyroMax)
    {
        return 0u;
    }

    return 1u;
}

void modeFlightRequestLaunch(void)
{
    launchRequestPending = 1u;
}

void modeFlightRequestDisarm(void)
{
    disarmRequestPending = 1u;
}

void modeFlightRequestProtection(void)
{
    protectionRequestPending = 1u;
}

uint8_t modeFlightTryBeginConfigUpdate(void)
{
    uint8_t accepted = 0u;
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    if ((flightSafetyState != SAFETY_STATE_CLOSED_LOOP) && !safetyContext.configUpdateActive)
    {
        safetyContext.configUpdateActive = 1u;
        accepted = 1u;
    }
    __set_PRIMASK(primask);
    return accepted;
}

void modeFlightEndConfigUpdate(void)
{
    uint32_t primask = __get_PRIMASK();
    __disable_irq();

```

```

    safetyContext.configUpdateActive = 0u;
    __set_PRIMASK(primask);
}

void modeFlightApplyParams(void)
{
    flightParamsStruct params;
    modeFlightConfigStruct next;
    uint32_t primask;
    flightParamsGetActive(&params);
    next.pitchMin = params.postureSafePitchMinRad;
    next.pitchMax = params.postureSafePitchMaxRad;
    next.rollMin = params.postureSafeRollMinRad;
    next.rollMax = params.postureSafeRollMaxRad;
    next.gyroMax = params.postureSafeGyroMaxRadS;
    primask = __get_PRIMASK();
    __disable_irq();
    modeConfig = next;
    __set_PRIMASK(primask);
}

void modeFlightSafetyTick(void)
{
    imuSnapshotStruct imu;
    uint8_t launchRequested = 0u;
    uint8_t disarmRequested = disarmRequestPending;
    uint8_t safe;
    if (!safetyContext.configUpdateActive)
    {
        launchRequested = launchRequestPending;
        launchRequestPending = 0u;
    }
    disarmRequestPending = 0u;
    IMUGetSnapshot(&imu);
    safe = postureSafeCheck(&imu);
    /* 发车额外条件：ToF 健康。ToF 链路/测距/突变保护异常时禁止起飞，
    * 避免仅靠基准油门 + 姿态环进入无可靠高度反馈的飞行状态。 */
    uint8_t canLaunch = (uint8_t)(safe && altitudeHoldIsHealthy());
    lastPostureSafe = safe;
    lastToFHealthy = altitudeHoldIsHealthy();
    flightSafetySeq++;

    if (protectionRequestPending)
    {
        protectionRequestPending = 0u;
        if (flightSafetyState != SAFETY_STATE_PROTECTED)
        {
            pidFlightControllerStop();
            flightSafetyState = SAFETY_STATE_PROTECTED;
            flightTransitionSeq++;
            flightProtectAcceptedSeq++;
        }
        else pidFlightControllerStop();
        return;
    }

    /* 回待机/断联缓降落地请求（core0Loop disarm 标志）：优先于发车处理。
    * 仅闭环态接受（回待机或缓降只在 CLOSED_LOOP 触发，请求回 DISARMED，支持随后重新发车）。
    * pidFlightControllerStop 停机 + 清 PID/输出/电机（同姿态保护停机路径）；置 DISARMED；
    * 中止起飞时序回 IDLE（避免残留时序在下次发车前误编排）。本拍 disarm 后不再处理发车。 */
    if (disarmRequested && flightSafetyState == SAFETY_STATE_CLOSED_LOOP)
    {
        pidFlightControllerStop();
        flightSafetyState = SAFETY_STATE_DISARMED;
        flightTransitionSeq++;
        flightDisarmAcceptedSeq++;
        return;
    }

    switch (flightSafetyState)
    {
        case SAFETY_STATE_DISARMED:
            /* 上电不自动起控；收到发车指令且姿态安全 + ToF 健康才进闭环 */
            if (launchRequested && canLaunch)
            {
                pidFlightControllerInit();
                flightSafetyState = SAFETY_STATE_CLOSED_LOOP;
                pidFlightControllerSetEnabled(1u);
                flightTransitionSeq++;
            }

```

```

        flightLaunchAcceptedSeq++;
    }
    break;

case SAFETY_STATE_CLOSED_LOOP:
    if (!safe)
    {
        pidFlightControllerStop();
        flightSafetyState = SAFETY_STATE_PROTECTED;
        flightTransitionSeq++;
        flightProtectAcceptedSeq++;
    }
    break;

case SAFETY_STATE_PROTECTED:
    /* 扶正后再次发车重新 init 闭环起飞（支持多次发车）。 */
    if (launchRequested && canLaunch)
    {
        pidFlightControllerInit();
        flightSafetyState = SAFETY_STATE_CLOSED_LOOP;
        pidFlightControllerSetEnabled(1u);
        flightTransitionSeq++;
        flightLaunchAcceptedSeq++;
    }
    break;

default:
    break;
}
}

void modeFlightGetSnapshot(flightSafetySnapshotStruct *out)
{
    uint32_t primask;
    if (out == 0) return;
    primask = __get_PRIMASK();
    __disable_irq();
    out->seq = flightSafetySeq;
    out->transitionSeq = flightTransitionSeq;
    out->launchAcceptedSeq = flightLaunchAcceptedSeq;
    out->disarmAcceptedSeq = flightDisarmAcceptedSeq;
    out->protectAcceptedSeq = flightProtectAcceptedSeq;
    out->state = flightSafetyState;
    out->postureSafe = lastPostureSafe;
    out->tofHealthy = lastTofHealthy;
    out->valid = 1u;
    out->configUpdateActive = safetyContext.configUpdateActive;
    __set_PRIMASK(primask);
}

```

core0_cyt4bb7/project/code/flight/modeFlight.h

SHA-256

:

b706312b9f643cfbb9449a88e7668b5da75322c0ac10dfb985c7ef4ad191179

代码 (c)

```

#ifndef _MODE_FLIGHT_H_
#define _MODE_FLIGHT_H_

#include "zf_common_typedef.h"

typedef enum
{
    SAFETY_STATE_DISARMED = 0,
    SAFETY_STATE_CLOSED_LOOP = 1,
    SAFETY_STATE_PROTECTED = 2
} safetyStateEnum;

typedef struct
{
    uint32_t seq;
    uint32_t transitionSeq;
    uint32_t launchAcceptedSeq;
    uint32_t disarmAcceptedSeq;

```



```

uint32_t protectAcceptedSeq;
safetyStateEnum state;
uint8_t postureSafe;
uint8_t toffHealthy;
uint8_t valid;
uint8_t configUpdateActive;
} flightSafetySnapshotStruct;

/* cm_7_0 定高飞控安全状态机（无 LCD，显示由 cm_7_1 负责）
* launchRequested: 1=本拍收到 cm_7_1 发车指令边沿（DISARMED/PROTECTED→CLOSED_LOOP）。
* disarmRequested: 1=本拍收到回待机/断联缓降落地请求（CLOSED_LOOP→DISARMED，core0Loop 下发）。*/
void modeFlightRequestLaunch(void);
void modeFlightRequestDisarm(void);
void modeFlightRequestProtection(void);
uint8_t modeFlightTryBeginConfigUpdate(void);
void modeFlightEndConfigUpdate(void);
void modeFlightApplyParams(void);
void modeFlightSafetyTick(void);
void modeFlightGetSnapshot(flightSafetySnapshotStruct *out);

#endif /* _MODE_FLIGHT_H_ */

```

core0_cyt4bb7/project/code/flight/motionFeedforwardDiff.c

SHA-256

:

41c345e6179424374e15435fbba0488f35a0334b54f36e3fb7d6a493bc18feeb

代码（c）

```

#include "motionFeedforwardDiff.h"

#include "commonDef.h"
#include "configFlightControlDefaults.h"
#include "flightParams.h"
#include "lowPassFilter.h"

static float motionCmdDvX = 0.0f; /* MOTION vx 指令微分 m/s^2，经限幅+低通；首帧/超时为 0 */
static float motionCmdDvY = 0.0f; /* MOTION vy 指令微分 m/s^2，经限幅+低通；首帧/超时为 0 */
static LowPassFilterData motionDiffLpfVx;
static LowPassFilterData motionDiffLpfVy;
static uint8_t motionDiffPrimed = 0u;
static uint32_t motionDiffLastSeq = 0u;
static uint32_t motionDiffLastMs = 0u;
static uint32_t motionDiffLastFilterMs = 0u;
static float motionDiffLastVx = 0.0f;
static float motionDiffLastVy = 0.0f;
static float motionDiffLimitX;
static float motionDiffLimitY;
static float motionDiffAlphaX;
static float motionDiffAlphaY;

static float motionDiffFilterAxis(LowPassFilterData *lpf, float raw, float limit, float alpha)
{
    if (limit <= 0.0f)
    {
        lowPassFilterReset(lpf, 0.0f);
        return 0.0f;
    }
    lpf->alpha = clampf(alpha, FLIGHT_LPF_ALPHA_MIN, FLIGHT_LPF_ALPHA_MAX);
    raw = clampf(raw, -limit, limit);
    lowPassFilter(lpf, raw);
    return lpf->filteredValue;
}

void motionFeedforwardDiffReset(void)
{
    motionCmdDvX = 0.0f;
    motionCmdDvY = 0.0f;
    motionDiffPrimed = 0u;
    motionDiffLastSeq = 0u;
    motionDiffLastMs = 0u;
    motionDiffLastFilterMs = 0u;
    motionDiffLastVx = 0.0f;
    motionDiffLastVy = 0.0f;
    lowPassFilterReset(&motionDiffLpfVx, 0.0f);
}

```

```

    lowPassFilterReset(&motionDiffLpfVy, 0.0f);
}

void motionFeedforwardDiffInit(void)
{
    lowPassFilterInit(&motionDiffLpfVx, FLIGHT_MOTION_FF_VX_DIFF_LPF_ALPHA);
    lowPassFilterInit(&motionDiffLpfVy, FLIGHT_MOTION_FF_VY_DIFF_LPF_ALPHA);
    motionFeedforwardDiffReset();
    motionFeedforwardDiffApplyParams();
}

void motionFeedforwardDiffApplyParams(void)
{
    flightParamsStruct params;
    flightParamsGetActive(&params);
    motionDiffLimitX = params.motionFfVxDiffLimitMps2;
    motionDiffLimitY = params.motionFfVyDiffLimitMps2;
    motionDiffAlphaX = params.motionFfVxDiffLpfAlpha;
    motionDiffAlphaY = params.motionFfVyDiffLpfAlpha;
}

static void motionFeedforwardDiffDecay(uint32_t nowMs)
{
    if (!motionDiffPrimed) return;

    if ((nowMs - motionDiffLastFilterMs) >= (uint32_t) FLIGHT_MOTION_FF_DIFF_DT_MIN_MS)
    {
        motionCmdDvX = motionDiffFilterAxis(&motionDiffLpfVx, 0.0f,
            motionDiffLimitX, motionDiffAlphaX);
        motionCmdDvY = motionDiffFilterAxis(&motionDiffLpfVy, 0.0f,
            motionDiffLimitY, motionDiffAlphaY);
        motionDiffLastFilterMs = nowMs;
    }
}

void motionFeedforwardDiffUpdate(float vx, float vy, uint32_t rxSeq, uint32_t nowMs)
{
    if (!motionDiffPrimed)
    {
        motionDiffPrimed = 1u;
        motionDiffLastSeq = rxSeq;
        motionDiffLastMs = nowMs;
        motionDiffLastFilterMs = nowMs;
        motionDiffLastVx = vx;
        motionDiffLastVy = vy;
        motionCmdDvX = 0.0f;
        motionCmdDvY = 0.0f;
        lowPassFilterReset(&motionDiffLpfVx, 0.0f);
        lowPassFilterReset(&motionDiffLpfVy, 0.0f);
        return;
    }

    if (rxSeq != motionDiffLastSeq)
    {
        uint32_t dtMs = nowMs - motionDiffLastMs;
        float invDtS;
        float rawDx;
        float rawDy;
        if (dtMs < (uint32_t) FLIGHT_MOTION_FF_DIFF_DT_MIN_MS)
        {
            dtMs = (uint32_t) FLIGHT_MOTION_FF_DIFF_DT_MIN_MS;
        }
        invDtS = 1000.0f / (float) dtMs;
        rawDx = (vx - motionDiffLastVx) * invDtS;
        rawDy = (vy - motionDiffLastVy) * invDtS;

        motionCmdDvX = motionDiffFilterAxis(&motionDiffLpfVx, rawDx,
            motionDiffLimitX, motionDiffAlphaX);
        motionCmdDvY = motionDiffFilterAxis(&motionDiffLpfVy, rawDy,
            motionDiffLimitY, motionDiffAlphaY);

        motionDiffLastSeq = rxSeq;
        motionDiffLastMs = nowMs;
        motionDiffLastFilterMs = nowMs;
        motionDiffLastVx = vx;
        motionDiffLastVy = vy;
    }
    else
    {

```

```

        motionFeedforwardDiffDecay(nowMs);
    }
}

float motionFeedforwardDiffGetDvX(void)
{
    return motionCmdDvX;
}

float motionFeedforwardDiffGetDvY(void)
{
    return motionCmdDvY;
}

```

core0_cyt4bb7/project/code/flight/motionFeedforwardDiff.h

SHA-256

:

59d8eff13c7521c80e7c986e2956011b4fea33d007106366cdca72396587c99a

代码 (c)

```

#ifndef _MOTION_FEEDFORWARD_DIFF_H_
#define _MOTION_FEEDFORWARD_DIFF_H_

#include <stdint.h>

/*
 * 运动前馈微分 (core0 飞控)：对 MOTION 帧的 vx/vy 速度指令做指令微分 (dt 下限保护 + 分轴限幅 +
 * 一阶低通)，输出 m/s^2 量纲的 dvx/dvy，供 core0Loop 并入 pitch/roll 角度前馈。
 *
 * 状态机自包含 (原内联于 core0Loop.c，前台 whileProcess 调用，非 ISR)：
 * Init —— processInit 一次性初始化低通并复位状态。
 * Update —— 收到新 MOTION 快照时按 rxSeq 边沿求微分；同 seq (无新帧) 则衰减。
 * Reset —— 链路超时/无运动信号时复位 (dvx/dvy 归零、状态清空)。
 * GetDvX/GetDvY —— 前台读取最新微分值 (生产消费同前台，无需 volatile)。
 * 参数 (限幅/低通 alpha) 取自 flightParams，运行时菜单可调；alpha 区间见 FLIGHT_LPF_ALPHA_MIN/MAX。
 */

void motionFeedforwardDiffInit(void);
void motionFeedforwardDiffApplyParams(void);
void motionFeedforwardDiffReset(void);
void motionFeedforwardDiffUpdate(float vx, float vy, uint32_t rxSeq, uint32_t nowMs);
float motionFeedforwardDiffGetDvX(void);
float motionFeedforwardDiffGetDvY(void);

#endif /* _MOTION_FEEDFORWARD_DIFF_H_ */

```

core0_cyt4bb7/project/code/flight/motor.c

SHA-256

:

f9d032dc6d0a226fdc3ca672c3e6478eaa55cb918688457d91f8a73e26314c17

代码 (c)

```

#include "motor.h"

#include "pinMap.h"
#include "zf_driver_pwm.h"
#include "zf_driver_delay.h"

/*
 * 400 Hz Servo PWM
 * 周期 2500µs；脉宽 950-2000µs 映射到 duty 寄存器 3800-8000
 * (PWM_DUTY_MAX=10 000 → 100% duty = 周期；950/2500*10 000=3800)
 *
 * core0 移植说明：cm_7_0 不拥有 IPS114 (屏幕归 cm_7_1)，故 arming 全程
 * 不调用任何 ips114_*。进度通过 volatile motorArmProgress (0~100) 暴露，
 * 并经 motorInit(onProgress) 回调由 cm_7_0 写共享内存 (safetyState=0xFE +
 * armProgress)，cm_7_1 据此在屏上画 ESC arming 进度条。*/
#define ESC_PWM_FREQ_HZ (400u)

```

```

#define ESC_PWM_PERIOD_US (1000000u / ESC_PWM_FREQ_HZ) /* 2500 */
#define ESC_PWM_REG_MIN ((MOTOR_STOP_PULSE_US * PWM_DUTY_MAX) / ESC_PWM_PERIOD_US) /* 3800 */
#define ESC_PWM_REG_MAX ((MOTOR_FULL_PULSE_US * PWM_DUTY_MAX) / ESC_PWM_PERIOD_US) /* 8000 */
#define ESC_PWM_REG_SPAN (ESC_PWM_REG_MAX - ESC_PWM_REG_MIN) /* 4200 */

/* 上电 arming: 1050→950μs 扫描后保持 950μs 3s */
#define ESC_ARM_SWEEP_START_US (1050u)
#define ESC_ARM_SWEEP_STEPS (101u)
#define ESC_ARM_SWEEP_STEP_MS (20u)
#define ESC_ARM_HOLD_MS (3000u)
#define ESC_ARM_TOTAL_MS (ESC_ARM_SWEEP_STEPS * ESC_ARM_SWEEP_STEP_MS + ESC_ARM_HOLD_MS)

/* ESC 物理方位映射 (P5.0/1/2/3 = BL/BR/FL/FR) 见 pinMap.h */

/* arming 进度 0~100, 供跨核调试显示读取 */
volatile uint8_t motorArmProgress = 0u;

/* =====
* 内部 helper: duty→PWM 寄存器 / 寄存器→脉宽
* ===== */

static inline uint32_t rawDutyToReg(int16_t duty)
{
    if (duty <= 0) return (uint32_t) ESC_PWM_REG_MIN;
    return (uint32_t) ESC_PWM_REG_MIN + (uint32_t) ((duty * (int32_t) ESC_PWM_REG_SPAN) / (int32_t) MOTOR_DUTY_MAX);
}

/* =====
* 百分比油门 API
* 0~100% ←→ MOTOR_ACTIVE_PULSE_MIN_US~MOTOR_ACTIVE_PULSE_MAX_US
* pct<=0 → 停机 (950μs)
* pct>100 → 钳到上限
* ===== */

static int16_t motorPercentToDuty(float pct)
{
    /* NaN 唯一满足 x!=x 的浮点数; NaN 视为停机 */
    if (pct != pct) return 0;
    if (pct <= 0.0f) return 0;
    if (pct > 100.0f) pct = 100.0f;

    float span = (float) (MOTOR_ACTIVE_DUTY_MAX - MOTOR_ACTIVE_DUTY_MIN);
    float d = (float) MOTOR_ACTIVE_DUTY_MIN + (pct * 0.01f) * span;

    if (d < (float) MOTOR_ACTIVE_DUTY_MIN) d = (float) MOTOR_ACTIVE_DUTY_MIN;
    if (d > (float) MOTOR_ACTIVE_DUTY_MAX) d = (float) MOTOR_ACTIVE_DUTY_MAX;
    return (int16_t) (d + 0.5f);
}

void motorSetThrottlePercent(float flPct, float frPct, float blPct, float brPct)
{
    pwm_set_duty(ESC_CH_PHYSICAL_FL, rawDutyToReg(motorPercentToDuty(flPct)));
    pwm_set_duty(ESC_CH_PHYSICAL_FR, rawDutyToReg(motorPercentToDuty(frPct)));
    pwm_set_duty(ESC_CH_PHYSICAL_BL, rawDutyToReg(motorPercentToDuty(blPct)));
    pwm_set_duty(ESC_CH_PHYSICAL_BR, rawDutyToReg(motorPercentToDuty(brPct)));
}

/* =====
* 生命周期
* ===== */

void motorInit(motorArmProgressCb onProgress)
{
    uint8_t i;
    uint32_t elapsedMs = 0u;

    motorArmProgress = 0u;

    /* 先让 4 路 PWM 进入确定的停机脉宽, 再发布 arming 状态。
    * 避免共享状态写入期间 ESC 控制脚仍处于未初始化/悬空状态。 */
    pwm_init(ESC_CH_PHYSICAL_FL, ESC_PWM_FREQ_HZ, (uint32_t) ESC_PWM_REG_MIN);
    pwm_init(ESC_CH_PHYSICAL_FR, ESC_PWM_FREQ_HZ, (uint32_t) ESC_PWM_REG_MIN);
    pwm_init(ESC_CH_PHYSICAL_BL, ESC_PWM_FREQ_HZ, (uint32_t) ESC_PWM_REG_MIN);
    pwm_init(ESC_CH_PHYSICAL_BR, ESC_PWM_FREQ_HZ, (uint32_t) ESC_PWM_REG_MIN);
    if (onProgress != 0) onProgress(0u);

    /* 阶段 1: 1050→950μs 扫描 */
    for (i = 0u; i < ESC_ARM_SWEEP_STEPS; i++)
    {

```

```

uint32_t pulseUs = (uint32_t) ESC_ARM_SWEEP_START_US - (uint32_t) i;
uint32_t reg = (pulseUs * (uint32_t) PWM_DUTY_MAX) / (uint32_t) ESC_PWM_PERIOD_US;
pwm_set_duty(ESC_CH_PHYSICAL_FL, reg);
pwm_set_duty(ESC_CH_PHYSICAL_FR, reg);
pwm_set_duty(ESC_CH_PHYSICAL_BL, reg);
pwm_set_duty(ESC_CH_PHYSICAL_BR, reg);
system_delay_ms(ESC_ARM_SWEEP_STEP_MS);
elapsedMs += ESC_ARM_SWEEP_STEP_MS;
motorArmProgress = (uint8_t) ((elapsedMs * 100u) / ESC_ARM_TOTAL_MS);
if (onProgress != 0) onProgress(motorArmProgress);
}

/* 阶段 2: 保持 950μs 3s */
pwm_set_duty(ESC_CH_PHYSICAL_FL, (uint32_t) ESC_PWM_REG_MIN);
pwm_set_duty(ESC_CH_PHYSICAL_FR, (uint32_t) ESC_PWM_REG_MIN);
pwm_set_duty(ESC_CH_PHYSICAL_BL, (uint32_t) ESC_PWM_REG_MIN);
pwm_set_duty(ESC_CH_PHYSICAL_BR, (uint32_t) ESC_PWM_REG_MIN);
{
    uint32_t holdMs = 0u;
    uint32_t holdStep = 100u;
    while (holdMs < ESC_ARM_HOLD_MS)
    {
        system_delay_ms(holdStep);
        holdMs += holdStep;
        elapsedMs += holdStep;
        motorArmProgress = (uint8_t) ((elapsedMs * 100u) / ESC_ARM_TOTAL_MS);
        if (onProgress != 0) onProgress(motorArmProgress);
    }
}

motorArmProgress = 100u;
if (onProgress != 0) onProgress(100u);
}

void motorStopAll(void)
{
    /* 全通道写停机脉宽 950μs */
    pwm_set_duty(ESC_CH_PHYSICAL_FL, (uint32_t) ESC_PWM_REG_MIN);
    pwm_set_duty(ESC_CH_PHYSICAL_FR, (uint32_t) ESC_PWM_REG_MIN);
    pwm_set_duty(ESC_CH_PHYSICAL_BL, (uint32_t) ESC_PWM_REG_MIN);
    pwm_set_duty(ESC_CH_PHYSICAL_BR, (uint32_t) ESC_PWM_REG_MIN);
}

```

core0_cyt4bb7/project/code/flight/motor.h

SHA-256

:

bf27e2d742c7915d94dac00ad1aeab1a43a022c0266ddac15706c94599a4ee83

代码 (c)

```

#ifndef _MOTOR_H_
#define _MOTOR_H_

#include <stdint.h>

/*
 * core0 ESC 驱动: 4 路 400Hz Servo PWM (AM32 移植版自动识别为 servo 协议)
 *
 * 实际物理方位映射 (用户硬件接线):
 * FL → TCPWM_CH11 / P5.2, 顺时针
 * FR → TCPWM_CH12 / P5.3, 逆时针
 * BL → TCPWM_CH09 / P5.0, 逆时针
 * BR → TCPWM_CH10 / P5.1, 顺时针
 *
 * ===== 控制接口 (百分比油门) =====
 * 闭环控制链路统一使用 motorSetThrottlePercent, 输入为百分比 (0~100):
 * pct <= 0      → 停机 (950μs)
 * 0 < pct < 下限  → 抬到 1100μs
 * 下限..100     → 线性映射到 MOTOR_ACTIVE_PULSE_MIN~MAX
 * pct > 100     → 钳到上限
 *
 * ===== 例外路径 =====
 * motorInit 内部 arming 与 motorStopAll 直接写 950μs 脉宽,
 * 绕过百分比映射, 是上电/急停安全的最后兜底。
 */

```

```

#define MOTOR_DUTY_MAX (10000)

#define MOTOR_STOP_PULSE_US    (950u)
#define MOTOR_FULL_PULSE_US    (2000u)
#define MOTOR_ACTIVE_PULSE_MIN_US (1100u)
/* PID 飞控正油门上限：1940μs */
#define MOTOR_ACTIVE_PULSE_MAX_US (1940u)
#define MOTOR_ACTIVE_DUTY_MIN    ((int16_t) (((MOTOR_ACTIVE_PULSE_MIN_US - MOTOR_STOP_PULSE_US) * (uint32_t)
MOTOR_DUTY_MAX) / (MOTOR_FULL_PULSE_US - MOTOR_STOP_PULSE_US)))
#define MOTOR_ACTIVE_DUTY_MAX    ((int16_t) (((MOTOR_ACTIVE_PULSE_MAX_US - MOTOR_STOP_PULSE_US) * (uint32_t)
MOTOR_DUTY_MAX) / (MOTOR_FULL_PULSE_US - MOTOR_STOP_PULSE_US)))

/* ===== 百分比油门接口 =====
* 输入：0~100（%），对应脉宽线性映射到 MOTOR_ACTIVE_PULSE_MIN_US ~ MOTOR_ACTIVE_PULSE_MAX_US
* 特殊值：
*   pct <= 0      → 停机（950μs）
*   0 < pct < 实际下限 → 抬到 MOTOR_ACTIVE_PULSE_MIN_US
*   pct > 上限      → 钳到 MOTOR_ACTIVE_PULSE_MAX_US 等效百分比
* 钳位逻辑在 motor.c 内部，上层不再重复钳位。
*/
void motorSetThrottlePercent(float flPct, float frPct, float blPct, float brPct);

/* arming 进度 0~100（cm_7_0 无 LCD，进度供跨核调试显示读取） */
extern volatile uint8_t motorArmProgress;

/* arming 进度回调：每步进度更新时调用，供 cm_7_0 把进度写共享内存给 cm_7_1 画进度条。
* 解耦设计：motor.c 不依赖 coreDataTransfer，由调用方注入。NULL 表示不回调。 */
typedef void (*motorArmProgressCb)(uint8_t progress);

/* ===== 生命周期 ===== */
void motorInit(motorArmProgressCb onProgress); /* 阻塞 ~5s arming（1050→950 扫描 + 3s 保持），无 LCD */
void motorStopAll(void); /* 全部输出停机脉宽（950μs） */

#endif /* _MOTOR_H_ */

```

core0_cyt4bb7/project/code/flight/PID.c

SHA-256

:

1266d5bf73d259544a3b7d31f61a1b22adfb1f1ed04eb60974e1cfa3f64b7d01

代码（c）

```

#include "PID.h"

#include <math.h>

static void incrementalPidProcessHandler(PID *obj, float currentValue)
{
    float error = obj->expectedValue - currentValue;
    float deltaError = error - obj->lastError;

    obj->controlValue +=
        obj->kp * deltaError +
        obj->ki * error +
        obj->kd * (deltaError - (obj->lastError - obj->beforeLastError));

    obj->beforeLastError = obj->lastError;
    obj->lastError = error;
}

static void positionalPidHandler(PID *obj, float currentValue)
{
    float error = obj->expectedValue - currentValue;

    obj->integral += error;

    if (obj->integralMax)
    {
        if (obj->integral > obj->integralMax) obj->integral = obj->integralMax;
        else if (obj->integral < -obj->integralMax) obj->integral = -obj->integralMax;
    }

    obj->controlValue =
        obj->kp * error +

```

```

    obj->ki * obj->integral +
    obj->kd * (error - obj->lastError);

    obj->lastError = error;
}

/* 位置式 PID + D 项一阶低通：对 (error - lastError) 过 LPF 后乘 kd */
static void positionalPidWithDLpfHandler(PID *obj, float currentValue)
{
    float error = obj->expectedValue - currentValue;

    obj->integral += error;

    if (obj->integralMax)
    {
        if (obj->integral > obj->integralMax) obj->integral = obj->integralMax;
        else if (obj->integral < -obj->integralMax) obj->integral = -obj->integralMax;
    }

    lowPassFilter(&obj->dLpf, error - obj->lastError);

    obj->controlValue =
        obj->kp * error +
        obj->ki * obj->integral +
        obj->kd * obj->dLpf.filteredValue;

    obj->lastError = error;
}

void pidInit(PID *obj, float integralMax, float target, float kp, float ki, float kd, float derivativeLpfAlpha)
{
    obj->expectedValue = target;
    obj->kp = kp;
    obj->ki = ki;
    obj->kd = kd;

    if (ki != 0.0f && integralMax != 0.0f) obj->integralMax = fabsf(integralMax / ki);
    else obj->integralMax = 0.0f;

    obj->lastError = 0.0f;
    obj->beforeLastError = 0.0f;
    obj->integral = 0.0f;
    obj->controlValue = 0.0f;

    /* D 项 LPF：仅在位置式 + alpha∈(0,1) 时启用 */
    if (integralMax != 0.0f && derivativeLpfAlpha > 0.0f && derivativeLpfAlpha < 1.0f)
    {
        lowPassFilterInit(&obj->dLpf, derivativeLpfAlpha);
        obj->dLpfEnabled = 1u;
        obj->process = positionalPidWithDLpfHandler;
    }
    else
    {
        lowPassFilterInit(&obj->dLpf, 1.0f);
        obj->dLpfEnabled = 0u;
        if (integralMax == 0.0f) obj->process = incrementalPidProcessHandler;
        else obj->process = positionalPidHandler;
    }
}

inline void pidProcess(PID *obj, float currentValue)
{
    obj->process(obj, currentValue);
}

void pidReset(PID *obj)
{
    obj->expectedValue = 0.0f;
    obj->lastError = 0.0f;
    obj->beforeLastError = 0.0f;
    obj->integral = 0.0f;
    obj->controlValue = 0.0f;
    lowPassFilterInit(&obj->dLpf, obj->dLpf.alpha);
}

```

core0_cyt4bb7/project/code/flight/PID.h

SHA-256

:

03c43f6b80090576d596ebd60803501f199042ce06c312d410a1f0dc31e206e6

代码 (c)

```
#ifndef _PID_H_
#define _PID_H_

#include <stdint.h>

#include "lowPassFilter.h"

/*
 * 通用 PID 控制器
 * - integralMax == 0 → 增量式（用于电流/速度类无稳态的环；D 项 LPF 强制不启用）
 * - integralMax > 0 → 位置式（积分限幅自动按 ki 折算）
 *
 * D 项低通：
 * - derivativeLpfAlpha >= 1.0f → 不启用（位置式直接乘 kd）
 * - derivativeLpfAlpha ∈ (0,1) 且 位置式 → 启用，对 (error - lastError) 过一阶 LPF 后再乘 kd
 * - 增量式无视 alpha
 *
 * 用法：
 * PID pitchAnglePid;
 * pidInit(&pitchAnglePid, 5.0f, 0.0f, 4.0f, 0.0f, 0.02f, 1.0f); // 位置式无 D 项 LPF
 * pidInit(&pitchRatePid, 5.0f, 0.0f, 17.1f, 0.0f, 17.0f, 0.3f); // 位置式 + D 项 LPF
 * pitchAnglePid.expectedValue = targetAngle;
 * pidProcess(&pitchAnglePid, currentAngle);
 * float out = pitchAnglePid.controlValue;
 */

typedef struct PID PID;
typedef void (*pidProcessHandlerFn)(PID *obj, float currentValue);

struct PID
{
    float expectedValue;
    float kp;
    float ki;
    float kd;

    float lastError;
    float beforeLastError;
    float integral;
    float integralMax;

    float controlValue;

    LowPassFilterData dLpf;
    uint8_t dLpfEnabled;

    pidProcessHandlerFn process;
};

/* 初始化：integralMax==0 选增量式，否则位置式。
 * integralMax 的含义：积分项在控制量单位下的最大贡献（内部换算为 |integral| ≤ integralMax/|ki|）
 * derivativeLpfAlpha：D 项一阶 LPF 系数；语义见文件头注释 */
void pidInit(PID *obj, float integralMax, float target, float kp, float ki, float kd, float derivativeLpfAlpha);
void pidProcess(PID *obj, float currentValue);
/* 清零 PID 运行状态（expectedValue/lastError/beforeLastError/integral/controlValue + 重置 D 项 LPF），
 * 保留 kp/ki/kd/integralMax/process 等配置；用于使能切换、急停、回放等需复位累加器的场景。 */
void pidReset(PID *obj);

#endif /* _PID_H_ */
```

core0_cyt4bb7/project/code/flight/pidFlightController.c

SHA-256

:

b3f7d96d63cda6d138fbf04ec30bdc2dbef8c2838bbcca9e3b9a58644ab45e6e

代码 (c)

```
#include "pidFlightController.h"

#include "PID.h"
#include "IMU.h"
#include "motor.h"
#include "commonDef.h"
#include "configFlightControlDefaults.h"
#include "configFlightTiming.h"
#include "flightParams.h"
#include "modeFlight.h"
#include "zf_common_interrupt.h"

/*
 * core0 定高飞控 PID 控制器（仅 FULL_ALT_HOLD，已删除全部调试模式分支）
 * 视觉速度环 9ms + 位置环 27ms（视觉 errX/Y → pitch/roll 角度参考）
 *   → 外环 3ms 角度 PID（pitch/roll）+ 偏航角度环 3ms（yawErr → yawRateRef）
 *   → 内环 1ms 角速度 PID（pitch/roll/yaw）→ 混控
 * 定高偏移 altitudeOffsetPct 由 altitudeHold 写入，叠加到基准油门
 * 运动前馈 ffVel* 叠加到速度目标，ff* 叠加到角度参考 / 偏航参考
 */

typedef struct
{
    flightPidParamStruct pitchAngle;
    flightPidParamStruct rollAngle;
    flightPidParamStruct pitchRate;
    flightPidParamStruct rollRate;
    flightPidParamStruct yawRate;
    flightPidParamStruct posX;
    flightPidParamStruct posY;
    flightPidParamStruct posVelX;
    flightPidParamStruct posVelY;
    flightPidParamStruct yawAngle;
    float pitchRefRad;
    float rollRefRad;
    float yawRateRefRadS;
    float posAttCompXGainPxPerRad;
    float posAttCompYGainPxPerRad;
    float posVelCmdLimit;
    float posAngleLimitRad;
    float yawRateCmdLimit;
    float rateCmdLimitPitch;
    float rateCmdLimitRoll;
    float pitchRateFF;
    float rollRateFF;
    float ctrlLimitPctPitch;
    float ctrlLimitPctRoll;
    float ctrlLimitPctYaw;
    float throttleMinPct;
    float throttleMaxPct;
    float baseThrottlePct;
} pidFlightConfigStruct;

typedef struct
{
    volatile pidFlightSnapshotStruct snapshot;
    pidFlightConfigStruct config;
    PID pitchAnglePid;
    PID pitchRatePid;
    PID rollAnglePid;
    PID rollRatePid;
    PID yawRatePid;
    PID posXPid;
    PID posYPid;
    PID posVelXPid;
    PID posVelYPid;
    PID yawAnglePid;
    float posHeldRollCorr;
    float posHeldPitchCorr;
    uint8_t posTargetPrimed;
    uint8_t posVelPrimed;
    uint8_t posPrevKfValid;
    uint16_t posKfInvalidTicks;
    uint8_t posPrevVisionActive;
    float posVelTargetX;
    float posVelTargetY;
}
```

```

uint8_t angleLoopCount;
uint8_t posVelLoopCount;
uint8_t posLoopCount;
uint8_t yawAngleLoopCount;
volatile uint8_t idleThrottleMode;
volatile uint8_t idleThrottleWasActive;
volatile uint8_t motionBarrierTicks;
} pidFlightControllerContextStruct;

static pidFlightControllerContextStruct controllerContext;

#define pidFlight      controllerContext.snapshot
#define pidConfig      controllerContext.config
#define pitchAnglePid  controllerContext.pitchAnglePid
#define pitchRatePid   controllerContext.pitchRatePid
#define rollAnglePid   controllerContext.rollAnglePid
#define rollRatePid    controllerContext.rollRatePid
#define yawRatePid     controllerContext.yawRatePid
#define posXPid        controllerContext.posXPid
#define posYPid        controllerContext.posYPid
#define posVelXPid     controllerContext.posVelXPid
#define posVelYPid     controllerContext.posVelYPid
#define yawAnglePid    controllerContext.yawAnglePid
#define posHeldRollCorr controllerContext.posHeldRollCorr
#define posHeldPitchCorr controllerContext.posHeldPitchCorr
#define posTargetPrimed controllerContext.posTargetPrimed
#define posVelPrimed   controllerContext.posVelPrimed
#define posPrevKfValid controllerContext.posPrevKfValid
#define posKfInvalidTicks controllerContext.posKfInvalidTicks
#define posPrevVisionActive controllerContext.posPrevVisionActive
#define posVelTargetX  controllerContext.posVelTargetX
#define posVelTargetY  controllerContext.posVelTargetY
#define angleLoopCount controllerContext.angleLoopCount
#define posVelLoopCount controllerContext.posVelLoopCount
#define posLoopCount   controllerContext.posLoopCount
#define yawAngleLoopCount controllerContext.yawAngleLoopCount
#define idleThrottleMode controllerContext.idleThrottleMode
#define idleThrottleWasActive controllerContext.idleThrottleWasActive
#define motionBarrierTicks controllerContext.motionBarrierTicks

static void loadConfig(void)
{
    flightParamsStruct params;
    flightParamsGetActive(&params);
    pidConfig.pitchAngle = params.pitchAngle;
    pidConfig.rollAngle = params.rollAngle;
    pidConfig.pitchRate = params.pitchRate;
    pidConfig.rollRate = params.rollRate;
    pidConfig.yawRate = params.yawRate;
    pidConfig.posX = params.posX;
    pidConfig.posY = params.posY;
    pidConfig.posVelX = params.posVelX;
    pidConfig.posVelY = params.posVelY;
    pidConfig.yawAngle = params.yawAngle;
    pidConfig.pitchRefRad = params.pitchRefRad;
    pidConfig.rollRefRad = params.rollRefRad;
    pidConfig.yawRateRefRadS = params.yawRateRefRadS;
    pidConfig.posAttCompXGainPxPerRad = params.posAttCompXGainPxPerRad;
    pidConfig.posAttCompYGainPxPerRad = params.posAttCompYGainPxPerRad;
    pidConfig.posVelCmdLimit = params.posVelCmdLimit;
    pidConfig.posAngleLimitRad = params.posAngleLimitRad;
    pidConfig.yawRateCmdLimit = params.yawRateCmdLimit;
    pidConfig.rateCmdLimitPitch = params.rateCmdLimitPitch;
    pidConfig.rateCmdLimitRoll = params.rateCmdLimitRoll;
    pidConfig.pitchRateFF = params.pitchRateFF;
    pidConfig.rollRateFF = params.rollRateFF;
    pidConfig.ctrlLimitPctPitch = params.ctrlLimitPctPitch;
    pidConfig.ctrlLimitPctRoll = params.ctrlLimitPctRoll;
    pidConfig.ctrlLimitPctYaw = params.ctrlLimitPctYaw;
    pidConfig.throttleMinPct = params.throttleMinPct;
    pidConfig.throttleMaxPct = params.throttleMaxPct;
    pidConfig.baseThrottlePct = params.baseThrottlePct;
}

/* 速度环已与视觉位置环解耦：速度内环在 CLOSED_LOOP 下与姿态环同级常驻运行（velocityLoopUpdate），
 * 位置外环只负责生成/冻结/归零速度目标 posVelTarget（positionTargetUpdate），速度环再叠加 MOTION 速度前馈。*/

/* ===== 内部工具 ===== */

```

```

/* PID 单拍：写期望 → 处理反馈 → 输出钳幅 ±limit。 */
static inline float runPidClamped(PID *pid, float ref, float feedback, float limit)
{
    pid->expectedValue = ref;
    pidProcess(pid, feedback);
    return clampf(pid->controlValue, -limit, limit);
}

/* PID 单拍 + 角速度前馈：输出 = clampf(pidOut + ffCoeff*ref, ±limit)。
 * 仅角速度内环 pitch/roll 用（前馈量 = 目标角速度 = ref = expectedValue）。 */
static inline float runPidFF(PID *pid, float ref, float feedback, float ffCoeff, float limit)
{
    pid->expectedValue = ref;
    pidProcess(pid, feedback);
    return clampf(pid->controlValue + ffCoeff * ref, -limit, limit);
}

/* 定点串级复位：外环位置 PID + 内环速度 PID + 调度/保持状态。 */
/* 位置串级的非 PID 辅助状态清零（首测/重捕标记、保持修正、KF/视觉态、速度目标、分频计数）。
 * 重载增益版 resetPositionCascade 与仅清状态版 resetAllPidState 共用。 */
static void resetPositionCascadeState(void)
{
    {
        posTargetPrimed = 0u;
        posVelPrimed = 0u;
        posHeldRollCorr = 0.0f;
        posHeldPitchCorr = 0.0f;
        posPrevKfValid = 0u;
        posKfInvalidTicks = 0u;
        posPrevVisionActive = 0u;
        posVelTargetX = 0.0f;
        posVelTargetY = 0.0f;
        posVelLoopCount = 0u;
        posLoopCount = 0u;
    }
}

/* 定点串级复位（重载增益版）：4 个位置/速度 PID 从 flightParams 重载增益 + 清状态 + 辅助状态清零。 */
static void resetPositionCascade(void)
{
    {
        pidInitFromParams(&posXPid, &pidConfig.posX, 0.0f); /* 目标误差为 0（^ 顶到目标中点） */
        pidInitFromParams(&posYPid, &pidConfig.posY, 0.0f);
        pidInitFromParams(&posVelXPid, &pidConfig.posVelX, 0.0f);
        pidInitFromParams(&posVelYPid, &pidConfig.posVelY, 0.0f);

        resetPositionCascadeState();
    }
}

/* 全 PID 重载增益版（pidInitFromParams：从 flightParams 重载增益 + 清状态 + 设 expectedValue）。
 * 仅用于 Init / ApplyParams（增益可能变）。 */
static void resetAllPid(void)
{
    {
        pidInitFromParams(&pitchAnglePid, &pidConfig.pitchAngle, pidConfig.pitchRefRad);
        pidInitFromParams(&rollAnglePid, &pidConfig.rollAngle, pidConfig.rollRefRad);
        pidInitFromParams(&pitchRatePid, &pidConfig.pitchRate, 0.0f);
        pidInitFromParams(&rollRatePid, &pidConfig.rollRate, 0.0f);
        pidInitFromParams(&yawRatePid, &pidConfig.yawRate, 0.0f);

        resetPositionCascade();

        pidInitFromParams(&yawAnglePid, &pidConfig.yawAngle, 0.0f); /* 目标 yawErr 为 0（机头对准 ^ 朝向） */
    }
}

/* 全 PID 仅清状态版（pidReset：清累加器/lastError/controlValue/expectedValue + 重置 dLpf，保留增益）。
 * 用于 Stop（急停/disarm）+ idleThrottle 进/出边沿：增益不变只需清状态，各 PID expectedValue
 * 由下一控制拍 angle/rate/velocity/positionTarget 环重设，故清零无残留影响。 */
static void resetAllPidState(void)
{
    {
        pidReset(&pitchAnglePid);
        pidReset(&rollAnglePid);
        pidReset(&pitchRatePid);
        pidReset(&rollRatePid);
        pidReset(&yawRatePid);

        pidReset(&posXPid);
        pidReset(&posYPid);
        pidReset(&posVelXPid);
        pidReset(&posVelYPid);

        resetPositionCascadeState();
    }
}

```

```

    pidReset(&yawAnglePid);
}

/* 三个生命周期函数共有的参考值 + 循环计数器复位。 */
static void resetControlReferences(void)
{
    pidFlight.pitchRef = pidConfig.pitchRefRad;
    pidFlight.rollRef = pidConfig.rollRefRad;
    pidFlight.yawRateRef = pidConfig.yawRateRefRadS;
    angleLoopCount = 0u;
    yawAngleLoopCount = 0u;
}

/* Init/Stop 共有的控制输出清零（参考速率 + 控制量 + 电机 + 定高偏移）。 */
static void resetControlOutputs(void)
{
    pidFlight.pitchRateRef = 0.0f;
    pidFlight.rollRateRef = 0.0f;
    pidFlight.pitchCtrlPct = 0.0f;
    pidFlight.rollCtrlPct = 0.0f;
    pidFlight.yawCtrlPct = 0.0f;
    pidFlight.motorPct[0] = 0.0f;
    pidFlight.motorPct[1] = 0.0f;
    pidFlight.motorPct[2] = 0.0f;
    pidFlight.motorPct[3] = 0.0f;
    pidFlight.altitudeOffsetPct = 0.0f;
}

/* ===== 位置外环目标生成：27ms（速度内环 3 分频） =====
* 把视觉像素误差换算为物理速度目标 posVelTargetX/Y（m/s, ZOH 供速度内环消费）。
* 三态语义（速度环始终运行，仅目标来源不同）：
* ① 视觉有效(active=valid&&!held) → 跑位置 PID 更新目标 → 视觉指哪走哪
* ② 丢失保持期(held) → 不跑 PID，冻结 posVelTarget 上次值（维持最后速度意图 ≤3s）
* ③ 无视觉/超时(!valid) → posVelTarget 置 0（零速度位置保持/抗漂移）
* errX↔roll(X 右向)、errY↔pitch(Y 前向)。 */
static void positionTargetUpdate(const imuSnapshotStruct* imu)
{
    uint8_t visionActive = (pidFlight.visionValid && !pidFlight.visionHeld) ? 1u : 0u;
    uint8_t firstTarget = 0u;

    /* 态 ③：无视觉/超时 → 目标归零（区别于 held 的冻结）。位置 PID 状态待下次有效时重置。 */
    if (!pidFlight.visionValid)
    {
        posVelTargetX = 0.0f;
        posVelTargetY = 0.0f;
        posPrevVisionActive = 0u;
        return;
    }

    /* 态 ②：丢失保持期 → 冻结 posVelTarget 上次值，不跑 PID。 */
    if (!visionActive)
    {
        posPrevVisionActive = 0u;
        return;
    }

    /* 态 ①：视觉有效。首次识别 / 从无效或保持恢复 → 本拍立即跑一次位置外环并预置 lastError 防 D 冲击。 */
    if (!posTargetPrimed || !posPrevVisionActive)
    {
        firstTarget = 1u;
        posTargetPrimed = 1u;
        posLoopCount = 0u;
    }
    posPrevVisionActive = 1u;

    posLoopCount++;
    if (posLoopCount >= PID_FLIGHT_POS_LOOP_DIV || firstTarget)
    {
        float physicalPitch = imu->physicalPitch;
        float physicalRoll = imu->physicalRoll;
        float rawErrX = pidFlight.visionErrX;
        float rawErrY = pidFlight.visionErrY;
        float errCompX = rawErrX - pidConfig.posAttCompXGainPxPerRad * physicalRoll; /* errX↔roll */
        float errCompY = rawErrY - pidConfig.posAttCompYGainPxPerRad * physicalPitch; /* errY↔pitch；增益默认 0 退化纯级联 */
        posLoopCount = 0u;

        if (firstTarget) posXPid.lastError = -errCompX;
        posVelTargetX = runPidClamped(&posXPid, 0.0f, errCompX, pidConfig.posVelCmdLimit);
    }
}

```

```

        if (firstTarget) posYPid.lastError = -errCompY;
        posVelTargetY = runPidClamped(&posYPid, 0.0f, errCompY, pidConfig.posVelCmdLimit);
    }
}

/* ===== 速度内环：9ms，CLOSED_LOOP 下与姿态环同级常驻 =====
* 期望=posVelTarget + ffVel(ZOH, m/s)，反馈=KF 物理速度 m/s → 角度修正（钳 ±posAngleLimitRad）。
* 轴对齐：roll 环用 KF 向右、pitch 环用 KF 前向（KF 内部已去 yaw+倾摆，反馈不再做像素域姿态补偿）。
* 轴极性：roll(左倾正) 与 向右速度(右正) 反号 → roll 输出取反才是负反馈；pitch(前倾正) 与 前向(前正) 同号 → 不取反。
* 仅更新 posHeldRollCorr/PitchCorr；rollRef/pitchRef 由 positionReferenceUpdate 每 3ms 合成（含前馈）。
* KF 失效(kfVelValid=0)：FREEZE_TICKS(≈150ms) 内冻结 posHeldRollCorr/PitchCorr 上次值（容忍光流瞬时丢帧），
* 超时仍未恢复 → 角度修正归零（rollRef/pitchRef 退回纯基准姿态），保证长时间失效下姿态安全；恢复时预置 lastError 防 D 冲击。 */
static void velocityLoopUpdate(void)
{
    float velX = pidFlight.kfVelRight; /* roll 环反馈 m/s */
    float velY = pidFlight.kfVelFwd; /* pitch 环反馈 m/s */
    float velCmdLimit = pidConfig.posVelCmdLimit;
    float velTargetX = clampf(posVelTargetX + pidFlight.ffVelRight, -velCmdLimit, velCmdLimit);
    float velTargetY = clampf(posVelTargetY + pidFlight.ffVelFwd, -velCmdLimit, velCmdLimit);

    /* KF 失效降级：短时(< FREEZE_TICKS)冻结上拍角度修正，保持 posVelTarget，不跑 PID；
    * 超时则归零角度修正，让姿态退回基准（纯姿态/定高仍在，水平不再依赖陈旧速度修正）。 */
    if (!pidFlight.kfVelValid)
    {
        posPrevKfValid = 0u;
        if (posKfInvalidTicks < PID_FLIGHT_KF_INVALID_FREEZE_TICKS)
        {
            posKfInvalidTicks++; /* 冻结期：posHeldRollCorr/PitchCorr 保持上次值不变 */
        }
        else
        {
            posHeldRollCorr = 0.0f; /* 超时：归零角度修正，保姿态安全 */
            posHeldPitchCorr = 0.0f;
        }
        return;
    }

    posKfInvalidTicks = 0u; /* KF 有效：清失效计数 */

    /* 首测 / KF 恢复：预置 lastError 防微分冲击。 */
    if (!posVelPrimed || !posPrevKfValid)
    {
        posVelXPid.lastError = velTargetX - velX;
        posVelYPid.lastError = velTargetY - velY;
        posVelPrimed = 1u;
        posPrevKfValid = 1u;
    }

    /* 极性修正（roll 轴）：物理 roll 左倾正，而速度轴向右正——二者反号。
    * +rollRef(左倾) 使向右速度减小 ⇒ roll 环对地速度阻尼的回路增益为负，
    * 须对 PID 输出取反才构成负反馈（否则正反馈发散/跑飞）。
    * pitch 环“前倾正”与“前向正”同号、回路增益为正，故 velocityLoop 仅 roll 取反、pitch 不取反。 */
    posVelXPid.expectedValue = velTargetX;
    pidProcess(&posVelXPid, velX);
    posHeldRollCorr = clampf(-posVelXPid.controlValue,
        -pidConfig.posAngleLimitRad, pidConfig.posAngleLimitRad);

    posHeldPitchCorr = runPidClamped(&posVelYPid, velTargetY, velY, pidConfig.posAngleLimitRad);
}

/* ===== 位置/速度参考合成：3ms，与姿态角环同拍 =====
* 速度内环在 CLOSED_LOOP 下常驻运行（不再被视觉有效性掐断），按 9ms 分频。
* 位置外环（27ms）只在速度环拍内生/冻结/归零速度目标。
* rollRef/pitchRef 每 3ms 合成（基准 + 速度环角度修正 ZOH + 运动前馈），保证前馈 3ms 刷新。 */
static void positionReferenceUpdate(const imuSnapshotStruct* imu)
{
    posVelLoopCount++;
    if (posVelLoopCount >= PID_FLIGHT_POS_VEL_LOOP_DIV)
    {
        posVelLoopCount = 0u;
        positionTargetUpdate(imu); /* 27ms 分频在内部：生成/冻结/归零速度目标 */
        velocityLoopUpdate(); /* 9ms：更新 posHeldRollCorr/PitchCorr */
    }

    pidFlight.rollRef = pidConfig.rollRefRad + posHeldRollCorr + pidFlight.ffRoll;
    pidFlight.pitchRef = pidConfig.pitchRefRad + posHeldPitchCorr + pidFlight.ffPitch;
}

/* ===== 偏航角度环：3ms（视觉 yawErr → yawRateRef）=====

```

```

/* 目标丢失时仅用运动前馈；输出钳位到 ±YAW_RATE_CMD_LIMIT。 */
static void yawAngleLoopUpdate(void)
{
    float yawRateCmd;
    if (!pidFlight.visionValid)
    {
        pidReset(&yawAnglePid); /* 视觉失效仅清累加器(增益不变,expectedValue 恒 0)，防重捕 windup/D 冲击；原每拍全量重载属冗余 */
        yawRateCmd = 0.0f;
    }
    else
    {
        yawRateCmd = runPidClamped(&yawAnglePid, 0.0f, pidFlight.visionYawErr,
                                   pidConfig.yawRateCmdLimit);
    }
    pidFlight.yawRateRef = yawRateCmd + pidFlight.ffYawRate;
}

/* ===== 外环：3ms 角度 PID (pitch/roll) =====
 * pitchRef/rollRef 由位置环写入 (含基准 + 位置修正 + 前馈)。 */
static void angleLoopUpdate(const imuSnapshotStruct* imu)
{
    pidFlight.pitchRateRef = runPidClamped(&pitchAnglePid, pidFlight.pitchRef,
                                           imu->physicalPitch, pidConfig.rateCmdLimitPitch);
    pidFlight.rollRateRef = runPidClamped(&rollAnglePid, pidFlight.rollRef,
                                           imu->physicalRoll, pidConfig.rateCmdLimitRoll);
}

/* ===== 内环：1ms 角速度 PID (pitch/roll/yaw) ===== */
static void rateLoopUpdate(const imuSnapshotStruct* imu)
{
    pidFlight.pitchCtrlPct = runPidFF(&pitchRatePid, pidFlight.pitchRateRef,
                                       imu->gyroPitchFiltered, pidConfig.pitchRateFF, pidConfig.ctrlLimitPctPitch);
    pidFlight.rollCtrlPct = runPidFF(&rollRatePid, pidFlight.rollRateRef,
                                       imu->gyroRollFiltered, pidConfig.rollRateFF, pidConfig.ctrlLimitPctRoll);
    pidFlight.yawCtrlPct = runPidClamped(&yawRatePid, pidFlight.yawRateRef,
                                         imu->gyroYawFiltered, pidConfig.ctrlLimitPctYaw);
}

/* ===== 混控：基准油门 + 控制量 → 4 电机百分比 =====
 * 混控符号严格对齐 LQR 历史 T_ALLOC 物理约束：
 * FL = base - pitchCtrl - rollCtrl + yawCtrl   (CW)
 * FR = base - pitchCtrl + rollCtrl - yawCtrl   (CCW)
 * BL = base + pitchCtrl - rollCtrl - yawCtrl   (CCW)
 * BR = base + pitchCtrl + rollCtrl + yawCtrl   (CW)
 * 约定：正 pitchCtrl → 前倾力矩；正 rollCtrl → 左倾力矩；正 yawCtrl → CCW
 */
static void applyMixerAndOutput(void)
{
    float base = pidFlight.baseThrottlePct + pidFlight.altitudeOffsetPct;

    float pitchCmd = pidFlight.pitchCtrlPct;
    float rollCmd = pidFlight.rollCtrlPct;
    float yawCmd = pidFlight.yawCtrlPct;

    if ((floatIsFinite(base) == 0u) || (floatIsFinite(pitchCmd) == 0u) ||
        (floatIsFinite(rollCmd) == 0u) || (floatIsFinite(yawCmd) == 0u) ||
        (floatIsFinite(pidConfig.throttleMinPct) == 0u) ||
        (floatIsFinite(pidConfig.throttleMaxPct) == 0u))
    {
        pidFlightControllerStop();
        modeFlightRequestProtection();
        return;
    }

    float flPct = base - pitchCmd - rollCmd + yawCmd;
    float frPct = base - pitchCmd + rollCmd - yawCmd;
    float blPct = base + pitchCmd - rollCmd - yawCmd;
    float brPct = base + pitchCmd + rollCmd + yawCmd;

    if ((floatIsFinite(flPct) == 0u) || (floatIsFinite(frPct) == 0u) ||
        (floatIsFinite(blPct) == 0u) || (floatIsFinite(brPct) == 0u))
    {
        pidFlightControllerStop();
        modeFlightRequestProtection();
        return;
    }

    flPct = clampf(flPct, pidConfig.throttleMinPct, pidConfig.throttleMaxPct);
    frPct = clampf(frPct, pidConfig.throttleMinPct, pidConfig.throttleMaxPct);

```

```

    blPct = clampf(blPct, pidConfig.throttleMinPct, pidConfig.throttleMaxPct);
    brPct = clampf(brPct, pidConfig.throttleMinPct, pidConfig.throttleMaxPct);

    pidFlight.motorPct[0] = flPct;
    pidFlight.motorPct[1] = frPct;
    pidFlight.motorPct[2] = blPct;
    pidFlight.motorPct[3] = brPct;

    motorSetThrottlePercent(flPct, frPct, blPct, brPct);
}

/* WAIT 期输出：四路固定 Thr Min，不叠加任何闭环控制量。*/
static void outputIdleThrottle(void)
{
    if ((floatIsFinite(pidConfig.throttleMinPct) == 0u) ||
        (floatIsFinite(pidConfig.throttleMaxPct) == 0u))
    {
        pidFlightControllerStop();
        modeFlightRequestProtection();
        return;
    }
    float idlePct = clampf(pidConfig.throttleMinPct, 0.0f, pidConfig.throttleMaxPct);

    pidFlight.pitchRateRef = 0.0f;
    pidFlight.rollRateRef = 0.0f;
    pidFlight.pitchCtrlPct = 0.0f;
    pidFlight.rollCtrlPct = 0.0f;
    pidFlight.yawCtrlPct = 0.0f;
    pidFlight.altitudeOffsetPct = 0.0f;

    pidFlight.motorPct[0] = idlePct;
    pidFlight.motorPct[1] = idlePct;
    pidFlight.motorPct[2] = idlePct;
    pidFlight.motorPct[3] = idlePct;

    motorSetThrottlePercent(idlePct, idlePct, idlePct, idlePct);
}

/* ===== 外部接口 ===== */

void pidFlightControllerInit(void)
{
    resetAllPid();
    resetControlReferences();
    resetControlOutputs();
    idleThrottleWasActive = 0u;
    motionBarrierTicks = 0u;

    pidFlight.baseThrottlePct = pidConfig.baseThrottlePct;
    pidFlight.visionErrX = 0.0f;
    pidFlight.visionErrY = 0.0f;
    pidFlight.visionYawErr = 0.0f;
    pidFlight.visionValid = 0u;
    pidFlight.visionHeld = 0u;
    pidFlight.kfVelFwd = 0.0f;
    pidFlight.kfVelRight = 0.0f;
    pidFlight.kfVelValid = 0u;
    pidFlight.ffVelFwd = 0.0f;
    pidFlight.ffVelRight = 0.0f;
    pidFlight.ffPitch = 0.0f;
    pidFlight.ffRoll = 0.0f;
    pidFlight.ffYawRate = 0.0f;
    pidFlight.enabled = 0u;
    idleThrottleMode = 0u;
    idleThrottleWasActive = 0u;
}

void pidFlightControllerStop(void)
{
    /* enabled 必须先置零，防止 pitProcess1 中 Tick() 在 Stop 中途重写电机输出 */
    pidFlight.enabled = 0u;
    idleThrottleMode = 0u;
    idleThrottleWasActive = 0u;

    resetAllPidState(); /* 急停/disarm 只需清累加器(增益不变)；下次发车 Init 重载增益 */
    resetControlReferences();
    resetControlOutputs();

    motorStopAll();

```

```

}

void pidFlightControllerSetEnabled(uint8_t enabled)
{
    if (!enabled)
    {
        pidFlightControllerStop();
        return;
    }
    pidFlight.enabled = 1u;
}

void pidFlightControllerSetIdleThrottleMode(uint8_t active)
{
    idleThrottleMode = active ? 1u : 0u;
}

void pidFlightControllerApplyParams(void)
{
    /* 热重载：用 flightParams 最新值重置所有 PID 与基准油门。
     * 仅在非 CLOSED_LOOP 态由前台调用（cm_7_0 按 safetyState 门控，飞行中不触发）。
     * enabled 不变（停机态保持停机），仅刷新增益/限幅/参考。 */
    loadConfig();
    resetAllPid();
    resetControlReferences();
    pidFlight.baseThrottlePct = pidConfig.baseThrottlePct;
}

void pidFlightControllerSetGuidanceCommand(
    const pidFlightGuidanceCommandStruct *command)
{
    uint32_t primask;
    if (command == 0) return;
    primask = __get_PRIMASK();
    __disable_irq();
    pidFlight.visionErrX = command->visionErrX;
    pidFlight.visionErrY = command->visionErrY;
    pidFlight.visionYawErr = command->visionYawErr;
    pidFlight.visionValid = command->visionValid;
    pidFlight.visionHeld = command->visionHeld;
    pidFlight.kfVelFwd = command->kfVelFwd;
    pidFlight.kfVelRight = command->kfVelRight;
    pidFlight.kfVelValid = command->kfVelValid;
    pidFlight.ffVelFwd = command->ffVelFwd;
    pidFlight.ffVelRight = command->ffVelRight;
    pidFlight.ffPitch = command->ffPitch;
    pidFlight.ffRoll = command->ffRoll;
    pidFlight.ffYawRate = command->ffYawRate;
    __set_PRIMASK(primask);
}

void pidFlightControllerSetAltitudeOffset(float value)
{
    pidFlight.altitudeOffsetPct = value;
}

void pidFlightControllerGetSnapshot(pidFlightSnapshotStruct *out)
{
    uint32_t primask;
    if (out == 0) return;
    primask = __get_PRIMASK();
    __disable_irq();
    *out = pidFlight;
    __set_PRIMASK(primask);
}

void pidFlightControllerRequestMotionBarrier(uint8_t ticks)
{
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    if (ticks > motionBarrierTicks) motionBarrierTicks = ticks;
    __set_PRIMASK(primask);
}

uint8_t pidFlightControllerMotionBarrierPending(void)
{
    uint32_t primask = __get_PRIMASK();
    uint8_t pending;

```



```

__disable_irq();
pending = motionBarrierTicks;
__set_PRIMASK(primask);
return pending;
}

void pidFlightControllerTick(void)
{
    imuSnapshotStruct imu;
    IMUGetSnapshot(&imu);
    pidFlight.seq++;
    if (motionBarrierTicks != 0u)
    {
        pidFlight.ffVelFwd = 0.0f;
        pidFlight.ffVelRight = 0.0f;
        pidFlight.ffPitch = 0.0f;
        pidFlight.ffRoll = 0.0f;
        pidFlight.ffYawRate = 0.0f;
        motionBarrierTicks--;
    }
    if (!pidFlight.enabled)
    {
        idleThrottleWasActive = 0u;
        motorStopAll();
        return;
    }

    if (idleThrottleMode)
    {
        if (!idleThrottleWasActive)
        {
            resetAllPidState(); /* 进起转旁路：清累加器(增益不变) */
            resetControlReferences();
            resetControlOutputs();
            idleThrottleWasActive = 1u;
        }
        outputIdleThrottle();
        return;
    }

    if (idleThrottleWasActive)
    {
        resetAllPidState(); /* 出起转旁路恢复闭环：清累加器(增益不变)，expectedValue 下一拍重设 */
        resetControlReferences();
        resetControlOutputs();
        idleThrottleWasActive = 0u;
    }

    /* 偏航角度环：每 3 拍 = 3ms (yawErr → yawRateRef) */
    yawAngleLoopCount++;
    if (yawAngleLoopCount >= PID_FLIGHT_YAW_ANGLE_LOOP_DIV)
    {
        yawAngleLoopCount = 0u;
        yawAngleLoopUpdate();
    }

    /* 姿态角环：每 3 拍 = 3ms。positionReferenceUpdate 内部再分频：
    * 速度内环 9ms (CLOSED_LOOP 下常驻，不被视觉拍断)、位置外环 27ms (仅生成/冻结/归零速度目标)。
    * 同一到期拍严格按 位置目标 → 速度内环 (写 rollRef/pitchRef) → 姿态角环 执行。 */
    angleLoopCount++;
    if (angleLoopCount >= PID_FLIGHT_ANGLE_LOOP_DIV)
    {
        angleLoopCount = 0u;
        positionReferenceUpdate(&imu);
        angleLoopUpdate(&imu);
    }

    rateLoopUpdate(&imu);
    applyMixerAndOutput();
}

```

core0_cyt4bb7/project/code/flight/pidFlightController.h

SHA-256

:

a36dc6304dabc7660ef40cc83cb6dbc9bdf338f85ece06c9740937a09b3a435f

代码 (c)

```
#ifndef _PID_FLIGHT_CONTROLLER_H_
#define _PID_FLIGHT_CONTROLLER_H_

#include <stdint.h>

/*
 * core0 定高飞控 PID (仅 FULL_ALT_HOLD, 无调试模式)
 * - 外环: 角度 PID, 3ms; 位置式 (pitch/roll)
 * - 内环: 角速度 PID, 1ms; 位置式 (pitch/roll/yaw) , 反馈用 imuGyroFiltered* (1ms LPF)
 * - 角速度前馈: PID 输出 + FF 系数 × 目标角速度, 结果钳位
 * - yaw 作为角速度差速闭环, yawRateRef 由偏航角度环 (Phase 3) 写入
 * - 混控: 基准油门 + 控制量 (百分点) 叠加到 FL/FR/BL/BR
 * - 输出: motorSetThrottlePercent (油门行程 1100~1940μs 映射为 0~100%)
 *
 * ===== 物理轴映射 (与 IMU.h 一致) =====
 * 物理 pitch (前倾正) ← imuData.roll & *imuGyroFilteredX
 * 物理 roll (左倾正) ← imuData.pitch & *imuGyroFilteredY
 * 物理 yaw (CCW 正) ← imuData.yaw & *imuGyroFilteredZ
 *
 * ===== 混控符号 (X 四旋翼, 对齐 LQR 历史 T_ALLOC 物理约束) =====
 * base 是 [0,100] 百分比; pitchCtrl / rollCtrl / yawCtrl 单位为"百分点"(pp)
 * 电机旋向: FL=CW, FR=CCW, BL=CCW, BR=CW
 * 本工程沿用 LQR 历史 T_ALLOC 的反号约定; 若改用实际力矩方向为正, 混控正负需全部翻转。
 * 正 pitchCtrl → 前倾力矩 (T_ALLOC 反号); 正 rollCtrl → 左倾力矩; 正 yawCtrl → CCW
 * FL = base - pitchCtrl - rollCtrl + yawCtrl (CW)
 * FR = base - pitchCtrl + rollCtrl - yawCtrl (CCW)
 * BL = base + pitchCtrl - rollCtrl - yawCtrl (CCW)
 * BR = base + pitchCtrl + rollCtrl + yawCtrl (CW)
 *
 * ===== 调参入口 =====
 * 所有 PID 参数 / 限幅 / 滤波系数默认值集中于 configFlightControlDefaults.h,
 * 本文件仅暴露状态机与生命周期。
 */

typedef struct
{
    uint32_t seq;
    float pitchRef; /* 目标角 rad (基准 + 位置环修正) */
    float rollRef;
    float pitchRateRef; /* 外环输出的目标角速度 rad/s */
    float rollRateRef;
    float yawRateRef; /* yaw 角速度目标 rad/s (偏航角度环输出 + 前馈) */

    float pitchCtrlPct; /* 内环 pitch 控制量 (pp) */
    float rollCtrlPct;
    float yawCtrlPct; /* 内环 yaw 控制量 (pp) */

    float baseThrottlePct;
    float motorPct[4]; /* FL/FR/BL/BR 混控后百分比 */
    float altitudeOffsetPct; /* 定高 PID 输出 (pp, 叠加到基准油门) */

    /* 视觉/遥控输入 (由 core0Loop 在前台写入, ISR 内消费) */
    float visionErrX; /* 像素误差 X (已含极性) */
    float visionErrY; /* 像素误差 Y */
    float visionYawErr; /* ^ 朝向误差 rad (已含极性) */
    uint8_t visionValid; /* 1=视觉目标有效 (含丢失保持期: 仍按上次误差/冻结输出闭环) */
    uint8_t visionHeld; /* 1=目标丢失保持期: 位置外环冻结速度目标, 速度内环仍常驻运行 */
    /* KF 物理速度反馈 (机体水平航向系 m/s, 由 core0Loop 前台写入 velocityEstimatorGet/IsValid) 。
     * 速度内环反馈用此替代像素速度: kfVelFwd=前向(对齐 pitch 环)、kfVelRight=右向(对齐 roll 环)。 */
    float kfVelFwd; /* KF 前向速度 m/s (axis[0]) */
    float kfVelRight; /* KF 右向速度 m/s (axis[1]) */
    uint8_t kfVelValid; /* 1=KF 估计可信 (velocityEstimatorIsValid) ; 0=失效→速度环冻结上次输出降级 */
    float ffVelFwd; /* MOTION 速度目标前馈: 前向速度 m/s, 叠加到 pitch 速度环目标 */
    float ffVelRight; /* MOTION 速度目标前馈: 右向速度 m/s, 叠加到 roll 速度环目标 */
    float ffPitch; /* 运动前馈 pitch 角度增量 rad (core0Loop 由 vx 指令微分换算) */
    float ffRoll; /* 运动前馈 roll 角度增量 rad (core0Loop 由 vy 指令微分换算) */
    float ffYawRate; /* 运动前馈 yaw 角速度 rad/s (core0Loop 由 wz 直通换算) */

    uint8_t enabled;
    uint8_t padding[3];
} pidFlightSnapshotStruct;

typedef struct
{
    float visionErrX;
    float visionErrY;
```

```

    float visionYawErr;
    uint8_t visionValid;
    uint8_t visionHeld;
    float kfVelFwd;
    float kfVelRight;
    uint8_t kfVelValid;
    float ffVelFwd;
    float ffVelRight;
    float ffPitch;
    float ffRoll;
    float ffYawRate;
} pidFlightGuidanceCommandStruct;

/* ----- 生命周期 ----- */
void pidFlightControllerInit(void);
void pidFlightControllerStop(void);
void pidFlightControllerSetEnabled(uint8_t enabled);

/* 自动起飞 WAIT 期输出旁路：1=四路固定 Thr Min，跳过姿态/速度/位置/偏航闭环。 */
void pidFlightControllerSetIdleThrottleMode(uint8_t active);

/* 热重载：用 flightParams 最新值重置所有 PID 与基准油门/参考。
 * 仅允许在非 CLOSED_LOOP 态调用（cm_7_0 按 safetyState 门控，飞行中不触发）。 */
void pidFlightControllerApplyParams(void);

/* 同拍引导量批量发布：一次短 PRIMASK 提交，PIT 不得观察到视觉/速度/角度前馈的混合拍。 */
void pidFlightControllerSetGuidanceCommand(
    const pidFlightGuidanceCommandStruct *command);

/* 热重连安全屏障由 1ms 控制 owner 消费，调用方不得用前台时间窗冒充控制拍。 */
void pidFlightControllerRequestMotionBarrier(uint8_t ticks);
uint8_t pidFlightControllerMotionBarrierPending(void);

void pidFlightControllerSetAltitudeOffset(float value);
void pidFlightControllerGetSnapshot(pidFlightSnapshotStruct *out);

/* 角速度环 tick：1ms 周期；内部按统一三分频依次调度姿态角环、
 * 物理速度环和视觉位置环。双 PIT(P1) 下读取 PIT_CH0 最近一次刷新的 imuGyroFiltered”。 */
void pidFlightControllerTick(void);

#endif /* _PID_FLIGHT_CONTROLLER_H_ */

```

core0_cyt4bb7/project/code/sensor/altitudeTof.c

SHA-256 :

8f31389404f59be103a245bfa1b4d704da5901f49aaed5c799b49cf137a6ea82

代码 (c)

```

#include "altitudeTof.h"

#include <math.h>

#include "boardConfig.h" /* ALT_TOF_SENSOR_SELECT 等编译期选择器 */
#include "configAltitudeTof.h"
#include "flightParams.h"
#include "lowPassFilter.h"
#include "IMU.h"

#if ALT_TOF_SENSOR_SELECT == ALT_TOF_SENSOR_VL53L1X
#include "tofL1x.h"
#elif ALT_TOF_SENSOR_SELECT == ALT_TOF_SENSOR_VL53L8CX
#include "tofL8cx.h"
#else
#error "Unsupported ALT_TOF_SENSOR_SELECT"
#endif

/* 两款驱动统一暴露的契约：取帧 + 采样周期。init 时绑定其一，业务路径全走指针。
 * vtable 随 ALT_TOF_SENSOR_SELECT 编译期只定义被选芯片一套（未选芯片头未 include、
 * 函数未声明，不能无条件取地址）。 */
typedef struct
{
    uint8_t (*init)(void);
    uint8_t (*poll)(uint16_t *distanceMm, uint8_t *valid);
    float (*samplePeriodS)(void);
}

```

```

}tofChipOps;

#if ALT_TOF_SENSOR_SELECT == ALT_TOF_SENSOR_VL53L1X
static const tofChipOps opsSelected = {tofL1xInit, tofL1xPoll, tofL1xSamplePeriodS};
#else
static const tofChipOps opsSelected = {tofL8cxInit, tofL8cxPoll, tofL8cxSamplePeriodS};
#endif

typedef enum
{
    TOF_PROTECT_NORMAL = 0,
    TOF_PROTECT_SUSPENDED
}tofProtectStateEnum;

typedef struct
{
    float frozenHeightMm; /* 访问器输出的高度 (last-valid, 冻结期保持) */
    float frozenVelMmS; /* 访问器输出的速度低通 (last-valid, 冻结期保持) */
    float lastAcceptedHeightMm; /* 门控基线：上一可信高度 */
    float lastDiffHeightMm; /* 速度差分基线 */
    LowPassFilterData vellPf;

    uint8_t hasAccepted; /* 是否已有有效基线 */
    uint8_t hasDiffBaseline; /* 速度差分基线是否就绪 */
    uint8_t resetVelBaseline; /* 恢复首帧标志：不跨冻结期做差分 */
    uint8_t linkAlive; /* 链路活 (init 成功 + 看门狗未超时) */
    uint8_t rangeHealthy; /* 最近一次测距可信 */
    uint8_t freshHealthy; /* 本拍是否产出新鲜健康帧 */
    tofProtectStateEnum protectState;

    float invalidAccumMs; /* 无效累计，达 altTimeoutMs 后丢弃基线 */
    float sampleElapsedS; /* 节流/丢帧合并后的真实帧间隔累计 */
    volatile uint16_t silentMs; /* 距上次出测量的毫秒数 (ISR 累加)，超时判链路死 */
    volatile uint16_t pollAccumMs; /* 轮询节流计时 (ISR 累加，update 消费) */
}tofStateStruct;

static const tofChipOps *ops;
static tofStateStruct tof;

typedef struct
{
    float maxSpeedMmS;
    float timeoutMs;
    float minMm;
    float maxMm;
    float velAlpha;
}altitudeTofConfigStruct;

static altitudeTofConfigStruct tofConfig;

static void loadConfig(void)
{
    flightParamsStruct params;
    flightParamsGetActive(&params);
    tofConfig.maxSpeedMmS = params.altTofMaxSpeedMmS;
    tofConfig.timeoutMs = params.altTimeoutMs;
    tofConfig.minMm = params.altDistMinMm;
    tofConfig.maxMm = params.altDistMaxMm;
    tofConfig.velAlpha = params.altVelLpfAlpha;
}

static void resetState(void)
{
    tof.frozenHeightMm = 0.0f;
    tof.frozenVelMmS = 0.0f;
    tof.lastAcceptedHeightMm = 0.0f;
    tof.lastDiffHeightMm = 0.0f;
    tof.hasAccepted = 0u;
    tof.hasDiffBaseline = 0u;
    tof.resetVelBaseline = 1u;
    tof.linkAlive = 0u;
    tof.rangeHealthy = 0u;
    tof.freshHealthy = 0u;
    tof.protectState = TOF_PROTECT_NORMAL;
    tof.invalidAccumMs = 0.0f;
    tof.sampleElapsedS = 0.0f;
    tof.silentMs = 0u;
    tof.pollAccumMs = ALT_TOF_POLL_PERIOD_MS; /* 允许上电后首个 update 立即轮询一次 */
    lowPassFilterInit(&tof.vellPf, tofConfig.velAlpha);
}

```

```

}

/* 姿态补偿：斜距 → 垂直高度 height = slant × cos(pitch) × cos(roll)。
* 合成倾角 cos 低于阈值（激光易打斜侧、投影失真放大）则判无效。 */
static float compensateTilt(uint16_t slantMm, uint8_t *valid)
{
    imuSnapshotStruct imu;
    float tiltCos;
    IMUGetSnapshot(&imu);
    tiltCos = cosf(imu.physicalPitch) * cosf(imu.physicalRoll);
    if (tiltCos < ALT_TOF_MIN_TILT_COS)
    {
        *valid = 0u;
        return 0.0f;
    }
    *valid = 1u;
    return (float) slantMm * tiltCos;
}

/* 时间域创新门控（L1X/L8CX 共用）：单帧相对上一可信高度变化超过
* 物理最大垂直速度 × 实测帧间隔 即判离群丢弃。离群帧一律拒绝(返回 0)、不再聚簇重建基线
* （旧「连续 N 帧聚簇即采纳」会让 L8CX 野值簇被当新基线产生突发高度骤变，已彻底剔除）。
* 持续离群由 altitudeTofUpdate 累加 invalidAccumMs，达 altTimeoutMs 后丢弃基线从头确认
* （纯超时兜底，防真实大高度变化永久锁死）。返回 1=接受，0=丢弃。 */
static uint8_t temporalGate(float heightMm, float dtS)
{
    float maxStep;
    float diff;

    if (!tof.hasAccepted)
    {
        tof.protectState = TOF_PROTECT_NORMAL;
        tof.resetVelBaseline = 1u;
        return 1u;
    }

    maxStep = tofConfig.maxSpeedMmS * dtS;
    if (maxStep < (float) ALT_TOF_MIN_STEP_MM)
    {
        maxStep = (float) ALT_TOF_MIN_STEP_MM;
    }

    diff = fabsf(heightMm - tof.lastAcceptedHeightMm);
    if (diff <= maxStep)
    {
        tof.protectState = TOF_PROTECT_NORMAL;
        return 1u;
    }

    /* 离群帧：直接拒绝冻结，不重建基线；持续离群靠 update 累加 invalidAccumMs 超时兜底。 */
    tof.protectState = TOF_PROTECT_SUSPENDED;
    return 0u;
}

/* 接受有效帧：差分 + 低通求速度，更新冻结值与门控基线。
* 恢复首帧（resetVelBaseline）不跨冻结期做差分，速度低通锚定到上一有效值。 */
static void acceptFrame(float heightMm, float dtS)
{
    if (tof.resetVelBaseline || !tof.hasDiffBaseline || dtS <= 0.0f)
    {
        tof.hasDiffBaseline = 1u;
        tof.resetVelBaseline = 0u;
        lowPassFilterReset(&tof.velLpf, tof.frozenVelMmS);
    }
    else
    {
        lowPassFilter(&tof.velLpf, (heightMm - tof.lastDiffHeightMm) / dtS);
    }

    tof.lastAcceptedHeightMm = heightMm;
    tof.lastDiffHeightMm = heightMm;
    tof.frozenHeightMm = heightMm;
    tof.frozenVelMmS = tof.velLpf.filteredValue;
    tof.hasAccepted = 1u;
    tof.rangeHealthy = 1u;
    tof.freshHealthy = 1u;
}

/* 无效帧：清测距健康态、冻结高度+速度（frozen* 不动），并标记恢复首帧重建速度基线、

```

```

* 清本次累计时基——保证恢复帧不跨冻结 gap 做速度差分（决策 A：避免恢复速度尖峰）。
* 累计达 altTimeoutMs 则进一步丢弃门控基线，使门控从头确认。*/
static void handleInvalid(float dtS)
{
    tof.rangeHealthy = 0u;
    tof.freshHealthy = 0u;
    tof.resetVelBaseline = 1u;
    tof.sampleElapsedS = 0.0f;
    tof.invalidAccumMs += dtS * 1000.0f;

    if (tof.invalidAccumMs >= tofConfig.timeoutMs)
    {
        tof.hasAccepted = 0u;
        tof.hasDiffBaseline = 0u;
        tof.protectState = TOF_PROTECT_NORMAL;
        tof.invalidAccumMs = 0.0f;
    }
}

uint8_t altitudeTofInit(void)
{
    ops = &opsSelected;
    loadConfig();
    resetState();
    tof.linkAlive = ops->init();
    return tof.linkAlive;
}

void altitudeTofTickMs(void)
{
    /* 轮询节流计时无条件累加（即使链路失活也要继续，否则节流后无法恢复轮询）*/
    if (tof.pollAccumMs < 0xFFFFu) tof.pollAccumMs++;

    if (!tof.linkAlive && !tof.rangeHealthy) return; /* 已失效，无需再累计*/
    if (tof.silentMs < ALT_TOF_HEALTH_TIMEOUT_MS)
    {
        tof.silentMs++;
        if (tof.silentMs >= ALT_TOF_HEALTH_TIMEOUT_MS)
        {
            /* 链路失活：清健康态，冻结高度+速度（frozen* 不动），拒绝发车。
            * 标记恢复首帧重建速度基线，与 invalid 恢复路径一致（防 dead recovery 速度尖峰）。*/
            tof.linkAlive = 0u;
            tof.rangeHealthy = 0u;
            tof.freshHealthy = 0u;
            tof.resetVelBaseline = 1u;
        }
    }
}

void altitudeTofUpdate(void)
{
    uint16_t slantMm = 0u;
    uint8_t rawValid = 0u;
    uint8_t tiltValid = 0u;
    float heightMm;
    float dtS;

    tof.freshHealthy = 0u;

    /* 软件 IIC 轮询节流：每 ALT_TOF_POLL_PERIOD_MS 才放行一次取帧。*/
    if (tof.pollAccumMs < ALT_TOF_POLL_PERIOD_MS) return;
    tof.pollAccumMs = 0u;

    if (ops == 0 || !ops->poll(&slantMm, &rawValid)) return;

    tof.silentMs = 0u; /* 出测量即喂看门狗，与距离是否有效无关*/
    tof.linkAlive = 1u;
    dtS = ops->samplePeriodS();
    tof.sampleElapsedS += dtS;

    /* 姿态补偿在范围检查之前：量程判定针对真实垂直高度*/
    heightMm = compensateTilt(slantMm, &tiltValid);
    if (!rawValid || !tiltValid ||
        heightMm < tofConfig.minMm || heightMm > tofConfig.maxMm)
    {
        handleInvalid(dtS); /* 无效超时累计用单帧 dt（防累计值二次膨胀提前超时）*/
        return;
    }
}

```

```

if (!temporalGate(heightMm, tof.sampleElapsedS))
{
    /* 离群拒绝期：冻结(frozen* 不动)，标记恢复首帧重建速度基线、清累计时基。
    * 纯超时兜底：累加 invalidAccumMs(单帧 dt)，持续离群达 altTimeoutMs 则丢弃基线，
    * 下一帧因 !hasAccepted 无条件接受真值，防真实大高度变化永久锁死。*/
    tof.rangeHealthy = 0u;
    tof.resetVelBaseline = 1u;
    tof.sampleElapsedS = 0.0f;
    tof.invalidAccumMs += dtS * 1000.0f;
    if (tof.invalidAccumMs >= tofConfig.timeoutMs)
    {
        tof.hasAccepted = 0u;
        tof.protectState = TOF_PROTECT_NORMAL;
        tof.invalidAccumMs = 0.0f;
    }
    return;
}

tof.invalidAccumMs = 0.0f;
acceptFrame(heightMm, tof.sampleElapsedS);
tof.sampleElapsedS = 0.0f;
}

uint8_t tofGetHeight(float *heightMm, float *velFiltered)
{
    if (heightMm != 0) *heightMm = tof.frozenHeightMm;
    if (velFiltered != 0) *velFiltered = tof.frozenVelMmS;
    return tof.freshHealthy;
}

uint8_t altitudeTofsHealthy(void)
{
    return (uint8_t) (tof.linkAlive && tof.rangeHealthy &&
        tof.protectState == TOF_PROTECT_NORMAL && tof.hasAccepted);
}

void altitudeToFApplyParams(void)
{
    loadConfig();
    lowPassFilterInit(&tof.vellpf, tofConfig.velAlpha);
    lowPassFilterReset(&tof.velLpf, tof.frozenVelMmS);
    tof.invalidAccumMs = 0.0f;
    tof.protectState = TOF_PROTECT_NORMAL;
    tof.resetVelBaseline = 1u;

    /* 用新距离上下限保守复核缓存高度：若参数把合法区间收窄到当前冻结高度之外，
    * 立即清健康态，避免 altitudeTofsHealthy() 在下一帧 ToF 到来前继续返回 OK
    * （旧边界下接受的 frozenHeightMm 可能已越界）。下一帧 acceptFrame 会重新置位。*/
    if (tof.hasAccepted &&
        (tof.frozenHeightMm < tofConfig.minMm ||
         tof.frozenHeightMm > tofConfig.maxMm))
    {
        tof.rangeHealthy = 0u;
        tof.freshHealthy = 0u;
    }
}

```

core0_cyt4bb7/project/code/sensor/altitudeToF.h

SHA-256 :
a0eebabe3cbd8bc272d84637796861cb1820648296586045f4e1c4ebb80c5ef2

代码 (c)

```

#ifndef ALTITUDE_TOF_H_
#define ALTITUDE_TOF_H_

#include <stdint.h>

/*
* 定高 ToF facade：芯片分派 + 全部传感器数据加工。
* - init 时按 ALT_TOF_SENSOR_SELECT 绑定 L1X/L8CX 函数表，运行期不切换
* - 姿态补偿（斜距→垂直高度）、范围检查、时间域创新门控、无效冻结、
* 差分速度低通、链路看门狗、健康态全部在本层完成
* - 定高闭环只通过 tofGetHeight() 取“当前高度 + 速度低通”，不接触任一芯片细节

```

```

*/

uint8_t altitudeToFInit(void);

/* 1ms ISR 周期调用：累加轮询节流计时 + 链路失活看门狗。 */
void altitudeToFTickMs(void);

/* 前台轮询：按节流周期放行一次取帧，做姿态补偿/范围/门控/速度低通，更新冻结值。 */
void altitudeToFUpdate(void);

/* 访问器：填入当前高度(mm)与速度低通(mm/s)。返回 1=本拍新鲜健康；
 * 0=冻结中（仍填上一有效高度与速度，距离与速度都不清零）。 */
uint8_t tofGetHeight(float *heightMm, float *velFiltered);

/* 发车门槛：链路活 且 测距可信 且 门控未悬挂 且 已有有效基线。 */
uint8_t altitudeToFIsHealthy(void);

/* 参数热重载：刷新速度低通 alpha 并清门控/无效累计（非 CLOSED_LOOP 态调用）。 */
void altitudeToFApplyParams(void);

#endif /* _ALTITUDE_TOF_H_ */

```

core0_cyt4bb7/project/code/sensor/opticalFlow.c

SHA-256

:

f5a2345cc6a82f9a2e43482a619a34eac7e567e869cdd4857d07c8f2eff526d2

代码 (c)

```

#include "opticalFlow.h"

#include "pinMap.h"
#include "zf_driver_uart.h"

#define OPTICAL_FLOW_UART_NUM      (UART_1)
#define OPTICAL_FLOW_UART_BAUDRATE (19200u)
#define OPTICAL_FLOW_UART_RX_TRIGGER_LEVEL (0u) /* 0 = 每收到 1 字节触发一次 RX 中断 */

#define OPTICAL_FLOW_FRAME_HEAD      (0xFEu)
#define OPTICAL_FLOW_FRAME_TAIL      (0x55u)
#define OPTICAL_FLOW_PAYLOAD_LENGTH   (0x0Au) /* 帧载荷长度字段固定值 = 10 */
#define OPTICAL_FLOW_VALID_MARKER     (0xF5u)
#define OPTICAL_FLOW_SAMPLE_PERIOD_SEC (0.02f) /* 名义 50Hz */

/* 流式解析状态机：帧头 -> 长度 -> 10 字节载荷 -> 校验 -> 帧尾 */
#define OPTICAL_FLOW_PARSE_WAIT_HEAD (0u)
#define OPTICAL_FLOW_PARSE_WAIT_LENGTH (1u)
#define OPTICAL_FLOW_PARSE_PAYLOAD (2u)
#define OPTICAL_FLOW_PARSE_CHECKSUM (3u)
#define OPTICAL_FLOW_PARSE_TAIL (4u)

typedef struct
{
    uint8_t state;
    uint8_t payloadIndex;
    uint8_t checksum;
    uint8_t payload[OPTICAL_FLOW_PAYLOAD_LENGTH];
} opticalFlowParserStruct;

static opticalFlowParserStruct opticalFlowParser;

/* ISR 与主循环共享：最新样本 + 发布序号声明为私有 volatile（公共 struct 不加 volatile）。 */
static volatile opticalFlowSampleStruct opticalFlowLatestSample;
static volatile uint32_t opticalFlowPublishedSequence = 0u;
static uint32_t opticalFlowConsumedSequence = 0u;

static void opticalFlowParserReset(void)
{
    opticalFlowParser.state = OPTICAL_FLOW_PARSE_WAIT_HEAD;
    opticalFlowParser.payloadIndex = 0u;
    opticalFlowParser.checksum = 0u;
}

/* 校验失败 / 帧尾错误时的重同步：若当前字节恰为帧头，直接作为下一帧头继续。 */
static void opticalFlowParserRestartFromByte(uint8_t byte)

```



```

{
    opticalFlowParser.payloadIndex = 0u;
    opticalFlowParser.checksum = 0u;
    opticalFlowParser.state = (OPTICAL_FLOW_FRAME_HEAD == byte)
        ? OPTICAL_FLOW_PARSE_WAIT_LENGTH
        : OPTICAL_FLOW_PARSE_WAIT_HEAD;
}

static uint16_t opticalFlowDecodeU16(const uint8_t *data)
{
    return (uint16_t)((uint16_t)data[0] | ((uint16_t)data[1] << 8u));
}

static void opticalFlowPublishSample(void)
{
    uint16_t flowXRaw = opticalFlowDecodeU16(&opticalFlowParser.payload[0]);
    uint16_t flowYRaw = opticalFlowDecodeU16(&opticalFlowParser.payload[2]);
    uint32_t nextSequence = opticalFlowPublishedSequence + 1u;

    /* 显式小端拼成 uint16 再转 int16，不用 packed struct / 指针强转 / memcpy 解释整数。*/
    opticalFlowLatestSample.flowX = (int16_t)flowXRaw;
    opticalFlowLatestSample.flowY = (int16_t)flowYRaw;
    opticalFlowLatestSample.dtUs = opticalFlowDecodeU16(&opticalFlowParser.payload[4]);
    opticalFlowLatestSample.distanceMm = opticalFlowDecodeU16(&opticalFlowParser.payload[6]);
    opticalFlowLatestSample.valid =
        (OPTICAL_FLOW_VALID_MARKER == opticalFlowParser.payload[8]) ? 1u : 0u;
    opticalFlowLatestSample.version = opticalFlowParser.payload[9];
    opticalFlowLatestSample.sequence = nextSequence;

    /* 序号最后发布：主循环据此判断整帧快照是否已完整写入。*/
    opticalFlowPublishedSequence = nextSequence;
}

static void opticalFlowParserInputByte(uint8_t byte)
{
    switch (opticalFlowParser.state)
    {
    case OPTICAL_FLOW_PARSE_WAIT_HEAD:
    {
        if (OPTICAL_FLOW_FRAME_HEAD == byte)
        {
            opticalFlowParser.state = OPTICAL_FLOW_PARSE_WAIT_LENGTH;
        }
        break;
    }

    case OPTICAL_FLOW_PARSE_WAIT_LENGTH:
    {
        if (OPTICAL_FLOW_PAYLOAD_LENGTH == byte)
        {
            opticalFlowParser.payloadIndex = 0u;
            opticalFlowParser.checksum = 0u;
            opticalFlowParser.state = OPTICAL_FLOW_PARSE_PAYLOAD;
        }
        else
        {
            opticalFlowParser.RestartFromByte(byte);
        }
        break;
    }

    case OPTICAL_FLOW_PARSE_PAYLOAD:
    {
        opticalFlowParser.payload[opticalFlowParser.payloadIndex] = byte;
        opticalFlowParser.checksum ^= byte;
        opticalFlowParser.payloadIndex++;

        if (opticalFlowParser.payloadIndex >= OPTICAL_FLOW_PAYLOAD_LENGTH)
        {
            opticalFlowParser.state = OPTICAL_FLOW_PARSE_CHECKSUM;
        }
        break;
    }

    case OPTICAL_FLOW_PARSE_CHECKSUM:
    {
        if (opticalFlowParser.checksum == byte)
        {
            opticalFlowParser.state = OPTICAL_FLOW_PARSE_TAIL;
        }
    }
}

```

```

    }
    else
    {
        opticalFlowParserRestartFromByte(byte);
    }
    break;
}

case OPTICAL_FLOW_PARSE_TAIL:
{
    if (OPTICAL_FLOW_FRAME_TAIL == byte)
    {
        opticalFlowPublishSample();
        opticalFlowParserReset();
    }
    else
    {
        opticalFlowParserRestartFromByte(byte);
    }
    break;
}

default:
{
    opticalFlowParserRestartFromByte(byte);
    break;
}
}
}

uint8_t opticalFlowInit(void)
{
    opticalFlowParserReset();

    opticalFlowLatestSample.flowX = 0;
    opticalFlowLatestSample.flowY = 0;
    opticalFlowLatestSample.dtUs = 0u;
    opticalFlowLatestSample.distanceMm = 0u;
    opticalFlowLatestSample.sequence = 0u;
    opticalFlowLatestSample.valid = 0u;
    opticalFlowLatestSample.version = 0u;
    opticalFlowPublishedSequence = 0u;
    opticalFlowConsumedSequence = 0u;

    /* TX 脚仅占位（光流模块只发不收，可不接线）；RX=P04_0 收模块 TX。 */
    uart_init(
        OPTICAL_FLOW_UART_NUM,
        OPTICAL_FLOW_UART_BAUDRATE,
        OPTICAL_FLOW_UART_TX_PIN,
        OPTICAL_FLOW_UART_RX_PIN);
    uart_rx_trigger_interrupt(
        OPTICAL_FLOW_UART_NUM,
        OPTICAL_FLOW_UART_RX_TRIGGER_LEVEL);
    uart_rx_interrupt(OPTICAL_FLOW_UART_NUM, 1u);

    return 1u;
}

void opticalFlowRxIrqHandler(void)
{
    uint8_t byte;

    while (uart_query_byte(OPTICAL_FLOW_UART_NUM, &byte))
    {
        opticalFlowParserInputByte(byte);
    }
}

uint8_t opticalFlowPoll(opticalFlowSampleStruct *sample)
{
    uint8_t hasNewSample = 0u;

    if (0 == sample)
    {
        return 0u;
    }

    /* 短暂屏蔽 UART_1 RX 中断做多字段一致性快照，不关全局中断。 */
    uart_rx_interrupt(OPTICAL_FLOW_UART_NUM, 0u);

```

```
if (opticalFlowPublishedSequence != opticalFlowConsumedSequence)
{
    sample->flowX = opticalFlowLatestSample.flowX;
    sample->flowY = opticalFlowLatestSample.flowY;
    sample->dtUs = opticalFlowLatestSample.dtUs;
    sample->distanceMm = opticalFlowLatestSample.distanceMm;
    sample->sequence = opticalFlowLatestSample.sequence;
    sample->valid = opticalFlowLatestSample.valid;
    sample->version = opticalFlowLatestSample.version;
    opticalFlowConsumedSequence = opticalFlowPublishedSequence;
    hasNewSample = 1u;
}
uart_rx_interrupt(OPTICAL_FLOW_UART_NUM, 1u);

return hasNewSample;
}
```

core0_cyt4bb7/project/code/sensor/opticalFlow.h

SHA-256

:

1bdc08dd5cda74605af7e71938192ad082818d17e8bf8f004a5ec4a525323258

代码 (c)

```
#ifndef _OPTICAL_FLOW_H_
#define _OPTICAL_FLOW_H_

#include <stdint.h>

/* UPixels 光流模块驱动（UART_1/P04，19200 8N1，50Hz，上电即输出无需初始化命令）。
 * 14 字节定长小端帧，RX 中断内流式解析，发布单槽原子快照。
 * 驱动负责初始化与帧解析；数据由 velocityEstimator 前台独占消费，并作为水平速度内环的 KF 量测源。
 * 方向映射（已实测，依赖安装姿态）：flowX 前为正、flowY 右为正，已对齐机体 FRD。 */

typedef struct
{
    int16_t flowX; /* X 方向光流积分（前为正） */
    int16_t flowY; /* Y 方向光流积分（右为正） */
    uint16_t dtUs; /* 积分时长（微秒），速度换算以此为准，不可用名义周期替代 */
    uint16_t distanceMm; /* 模块自带距离；无 ToF 时恒 999mm 占位，不可作真实高度 */
    uint32_t sequence; /* 每个协议完整帧（含 valid=0）递增，用于漏帧 / 新鲜度检测 */
    uint8_t valid; /* 测量有效性：1=有效(0xF5) / 0=无效(质量不足或静止) */
    uint8_t version; /* 固件版本字段，原样上报 */
} opticalFlowSampleStruct;

/* 初始化 UART_1 接收链路；返回 1 表示本地外设配置完成，不代表链路已有数据。 */
uint8_t opticalFlowInit(void);

/* 仅由 uart1_isr() 调用：排空逐飞 UART 软件缓冲并喂入流式解析器。 */
void opticalFlowRxIrqHandler(void);

/* 返回 1 表示取得一帧尚未消费的新数据；sample->valid 表示测量有效性。 */
uint8_t opticalFlowPoll(opticalFlowSampleStruct *sample);

/* 返回传感器名义输出周期 0.02s；实际积分时长以 sample->dtUs 为准。 */

#endif /* _OPTICAL_FLOW_H_ */
```

core0_cyt4bb7/project/code/sensor/tof/tofBus.c

SHA-256

:

ec30ac0d52abe37467f0bbf5a32948411aec4b53a9263cddad0ba9a476c96207

代码 (c)

```
#include "tofBus.h"

#include <string.h>

#include "boardConfig.h" /* ALT_TOF_SENSOR_SELECT / ALT_TOF_L8CX_COMM_SELECT 等编译期选择器 */
```

```

#include "configAltitudeToF.h"
#include "pinMap.h"
#include "zf_driver_delay.h"
#include "zf_driver_gpio.h"

/* 后端选择： 仅 VL53L8CX + 硬件 SPI 走 SPI， 其余（含 VL53L1X）走软件 IIC */
#if (ALT_TOF_SENSOR_SELECT == ALT_TOF_SENSOR_VL53L8CX) && \
    (ALT_TOF_L8CX_COMM_SELECT == ALT_TOF_L8CX_COMM_HARD_SPI)
#define TOF_BUS_BACKEND_SPI (1)
#else
#define TOF_BUS_BACKEND_SPI (0)
#endif

/* =====
* 硬件 SPI 后端（WiFi SPI 引脚，NCS 软件管理）
* ===== */
#if TOF_BUS_BACKEND_SPI

#include "zf_driver_spi.h"

#define TOF_SPI_INDEX    (SPI_3)
/* SPI3 SCK/MOSI/MISO/NCS 引脚见 pinMap.h */
#define TOF_SPI_BAUD      (2000000u) /* 2MHz： 低于手册 ≤3MHz 上限留裕量，避免分频取整超频/信号完整性问题 */
#define TOF_SPI_WRITE_FLAG (0x80u) /* 地址高字节 MSB： 写=1， 读=0 */
#define TOF_SPI_SCRATCH   (64u) /* 全双工分块缓冲： 写时丢弃 RX / 读时发 0 哑数据 */

static uint8_t tofSpiScratch[TOF_SPI_SCRATCH];

/* 全双工写： 发 tx， 同步收到的 RX 丢进 scratch。 spi_transfer_8bit 逐字节读 RX FIFO，
* 避免只写不读导致全双工 RX FIFO 溢出/残留（L8CX 固件上传为大块写）。 NCS 由调用方维持。 */
static void tofSpiWrite(const uint8_t *tx, uint32_t len)
{
    uint32_t off = 0u;

    while (off < len)
    {
        uint32_t n = (len - off > TOF_SPI_SCRATCH) ? TOF_SPI_SCRATCH : (len - off);
        spi_transfer_8bit(TOF_SPI_INDEX, &tx[off], tofSpiScratch, n);
        off += n;
    }
}

/* 全双工读： 发 0 哑数据， 收 RX 到 rx。 */
static void tofSpiRead(uint8_t *rx, uint32_t len)
{
    uint32_t off = 0u;

    while (off < len)
    {
        uint32_t n = (len - off > TOF_SPI_SCRATCH) ? TOF_SPI_SCRATCH : (len - off);
        memset(tofSpiScratch, 0, n);
        spi_transfer_8bit(TOF_SPI_INDEX, tofSpiScratch, &rx[off], n);
        off += n;
    }
}

void tofBusInit(void)
{
    /* CS 传 SPI_CS_NULL： 禁用驱动自动 CS， 改由 NCS GPIO 跨“地址+数据”事务手动保持低 */
    spi_init(TOF_SPI_INDEX, SPI_MODE3, TOF_SPI_BAUD,
        TOF_SPI_SCK_PIN, TOF_SPI_MOSI_PIN, TOF_SPI_MISO_PIN, SPI_CS_NULL);
    gpio_init(TOF_SPI_NCS_PIN, GPO, GPIO_HIGH, GPO_PUSH_PULL);
}

void tofBusSetAddress(uint8_t addr7)
{
    (void) addr7; /* SPI 经 NCS 选片， 无从机地址 */
}

uint8_t tofBusWriteReg16(uint16_t reg, const uint8_t *buf, uint32_t size)
{
    uint8_t addr2[2];

    /* 非法 buffer： 与软件 IIC 后端口径一致， 返回错误而非伪装成功，
    * 避免平台层错误被 ST ULD 当成成功（防后续驱动扩展踩坑）。 */
    if (size != 0u && buf == NULL)
    {
        return 1u;
    }
}

```

```

addr2[0] = (uint8_t) (((reg >> 8) & 0x7Fu) | TOF_SPI_WRITE_FLAG);
addr2[1] = (uint8_t) reg;

gpio_low(TOF_SPI_NCS_PIN);
tofSpiWrite(addr2, 2u);
if (size != 0u)
{
    tofSpiWrite(buf, size);
}
gpio_high(TOF_SPI_NCS_PIN);
return 0u;
}

uint8_t tofBusReadReg16(uint16_t reg, uint8_t *buf, uint32_t size)
{
    uint8_t addr2[2];

    /* 非法 buffer：与软件 IIC 后端口径一致，返回错误而非伪装成功。 */
    if (size != 0u && buf == NULL)
    {
        return 1u;
    }

    addr2[0] = (uint8_t) ((reg >> 8) & 0x7Fu);
    addr2[1] = (uint8_t) reg;

    gpio_low(TOF_SPI_NCS_PIN);
    tofSpiWrite(addr2, 2u);
    if (size != 0u)
    {
        tofSpiRead(buf, size);
    }
    gpio_high(TOF_SPI_NCS_PIN);
    return 0u;
}

/* =====
* 软件 IIC 后端（P19_0/P19_1 位翻）
* ===== */
#else

#include "zf_driver_soft_iic.h"

/* 软件 IIC SDA/SCL 引脚（P19_0/1）见 pinMap.h */
#define TOF_IIC_DEFAULT_ADDR (0x52u >> 1u)
#define TOF_IIC_WRITE_CHUNK (256u) /* ULD 固件上传大块写按此分块（与地址各 2 字节前缀） */

#if (ALT_TOF_SENSOR_SELECT == ALT_TOF_SENSOR_VL53L1X)
#define TOF_IIC_DELAY (ALT_TOF_L1X_IIC_DELAY)
#else
#define TOF_IIC_DELAY (ALT_TOF_L8CX_IIC_DELAY)
#endif

static soft_iic_info_struct toflic;
static uint8_t toflicTxBuf[TOF_IIC_WRITE_CHUNK + 2u];

void tofBusInit(void)
{
    soft_iic_init(&toflic, TOF_IIC_DEFAULT_ADDR, TOF_IIC_DELAY,
        TOF_IIC_SCL_PIN, TOF_IIC_SDA_PIN);
}

void tofBusSetAddress(uint8_t addr7)
{
    toflic.addr = addr7;
}

uint8_t tofBusWriteReg16(uint16_t reg, const uint8_t *buf, uint32_t size)
{
    uint32_t offset = 0u;

    /* 分块 ≤256，每块自带 16-bit 起始地址（地址随偏移递增）；size==0 时不发送 */
    if (size != 0u && buf == NULL)
    {
        return 1u;
    }
    while (offset < size)
    {

```

```

uint32_t remaining = size - offset;
uint32_t chunk = (remaining > TOF_IIC_WRITE_CHUNK) ? TOF_IIC_WRITE_CHUNK : remaining;
uint16_t chunkReg = (uint16_t) (reg + offset);

toflcTxBuf[0] = (uint8_t) (chunkReg >> 8);
toflcTxBuf[1] = (uint8_t) chunkReg;
memcpy(&toflcTxBuf[2], &buf[offset], chunk);
soft_iic_write_8bit_array(&toflc, toflcTxBuf, chunk + 2u);
offset += chunk;
}
return 0u;
}

uint8_t tofBusReadReg16(uint16_t reg, uint8_t *buf, uint32_t size)
{
    uint8_t addr2[2];

    if (size == 0u)
    {
        return 0u;
    }
    if (buf == NULL)
    {
        return 1u;
    }
    addr2[0] = (uint8_t) (reg >> 8);
    addr2[1] = (uint8_t) reg;
    soft_iic_transfer_8bit_array(&toflc, addr2, 2u, buf, size);
    return 0u;
}

#endif /* TOF_BUS_BACKEND_SPI */

void tofBusDelayMs(uint32_t ms)
{
    system_delay_ms(ms);
}

```

core0_cyt4bb7/project/code/sensor/tof/tofBus.h

SHA-256

:

7fc07accd40e89ffc1c619fda43227e535684eca3086398730a7959b6f03441a

代码 (c)

```

#ifndef _TOF_BUS_H_
#define _TOF_BUS_H_

#include <stdint.h>

/*
 * ToF 统一通信传输层
 * 按 boardConfig.h / configAltitudeToF.h 宏在 [软件 IIC] / [硬件 SPI] 间编译期切换;
 * VL53L1X / VL53L8CX 两款芯片共用同一组寄存器读写原语
 * (16-bit 寄存器地址 + 字节缓冲, 地址大端)。
 *
 * - 软件 IIC: 逐飞 zf_driver_soft_iic, P19_0(SDA)/P19_1(SCL), 从机地址 0x29
 * - 硬件 SPI: 逐飞 zf_driver_spi (SPI_3, 引脚 P03_x), NCS 软件管理(P03_3)
 * L8CX SPI 协议: 地址高字节 MSB=1 写 / =0 读; Mode3(CPOL=1,CPHA=1); ≤3MHz
 *
 * 仅 L8CX + HARD_SPI 使用 SPI 后端, 其余 (含 VL53L1X) 使用软件 IIC。
 * 返回值 0 = 成功 (与 ST ULD VL53LMZ_STATUS_OK 约定一致)。
 */

void tofBusInit(void);
/* 设置软件 IIC 从机 7-bit 地址; SPI 后端经 NCS 选片, 忽略此值 */
void tofBusSetAddress(uint8_t addr7);
uint8_t tofBusWriteReg16(uint16_t reg, const uint8_t *buf, uint32_t size);
uint8_t tofBusReadReg16(uint16_t reg, uint8_t *buf, uint32_t size);
void tofBusDelayMs(uint32_t ms);

#endif /* _TOF_BUS_H_ */

```

core0_cyt4bb7/project/code/sensor/tof/tofL1x.c

SHA-256

:

df7e6f80464a35f0cd48e46b0e023e8904aebb20efbae2ea642811aa34cbeadb

代码 (c)

```
#include "tofL1x.h"

#include <string.h>

#include "VL53L1_register_map.h"
#include "configAltitudeToF.h"
#include "pinMap.h"
#include "tofBus.h"
#include "zf_common_typedef.h"
#include "zf_driver_delay.h"
#include "zf_driver_gpio.h"

/* ===== 私有类型与状态码 (原 VL53L1X.h 下沉, 仅本文件使用) ===== */
typedef enum
{
    VL53L1X_DISTANCE_MODE_SHORT = 0u,
    VL53L1X_DISTANCE_MODE_MEDIUM,
    VL53L1X_DISTANCE_MODE_LONG,
    VL53L1X_DISTANCE_MODE_UNKNOWN,
} vl53l1x_distance_mode_enum;

typedef enum
{
    VL53L1X_STATUS_OK = 0u,
    VL53L1X_STATUS_TIMEOUT,
    VL53L1X_STATUS_INVALID_PARAM,
    VL53L1X_STATUS_NOT_DETECTED,
    VL53L1X_STATUS_NOT_INITIALIZED,
    VL53L1X_STATUS_DATA_NOT_READY,
} vl53l1x_status_enum;

#define VL53L1_RANGESTATUS_RANGE_VALID (0u)
#define VL53L1_RANGESTATUS_SIGMA_FAIL (1u)
#define VL53L1_RANGESTATUS_SIGNAL_FAIL (2u)
#define VL53L1_RANGESTATUS_RANGE_VALID_MIN_RANGE_CLIPPED (3u)
#define VL53L1_RANGESTATUS_OUTOFBOUNDS_FAIL (4u)
#define VL53L1_RANGESTATUS_HARDWARE_FAIL (5u)
#define VL53L1_RANGESTATUS_RANGE_VALID_NO_WRAP_CHECK_FAIL (6u)
#define VL53L1_RANGESTATUS_WRAP_TARGET_FAIL (7u)
#define VL53L1_RANGESTATUS_PROCESSING_FAIL (8u)
#define VL53L1_RANGESTATUS_XTALK_SIGNAL_FAIL (9u)
#define VL53L1_RANGESTATUS_SYNCRONISATION_INT (10u)
#define VL53L1_RANGESTATUS_RANGE_VALID_MERGED_PULSE (11u)
#define VL53L1_RANGESTATUS_TARGET_PRESENT_LACK_OF_SIGNAL (12u)
#define VL53L1_RANGESTATUS_MIN_RANGE_FAIL (13u)
#define VL53L1_RANGESTATUS_RANGE_INVALID (14u)
#define VL53L1_RANGESTATUS_NONE (255u)

typedef struct
{
    uint16 distance_mm;
    uint8 range_status;
    float peak_signal_count_rate_mcps;
    float ambient_count_rate_mcps;
    uint8 last_status;
    uint8 did_timeout;
    uint8 is_initialized;
    uint8 is_continuous_mode;
    vl53l1x_distance_mode_enum distance_mode;
    uint16 io_timeout_ms;
    uint32 intermeasurement_period_ms;
} VL53L1X_DATA;

/* 芯片控制脚 INT/XSHUT 见 pinMap.h; 数据总线(P19 软件 IIC)由 tofBus 管理 */
#define VL53L1X_IIC_ADDR (0x52u >> 1u)

#define VL53L1X_DEFAULT_TIMEOUT_MS (100u)
```

```

#define VL53L1X_DEFAULT_INTERMEASUREMENT_PERIOD_MS (50u)
#define VL53L1X_TIMING_GUARD_US (4528u)
#define VL53L1X_TARGET_RATE (0x0A00u)

typedef struct
{
    uint8 range_status;
    uint8 stream_count;
    uint16 dss_actual_effective_spads_sd0;
    uint16 ambient_count_rate_mcps_sd0;
    uint16 final_crosstalk_corrected_range_mm_sd0;
    uint16 peak_signal_count_rate_crosstalk_corrected_mcps_sd0;
} vl53l1x_result_buffer_struct;

typedef struct
{
    uint16 fast_osc_frequency;
    uint16 osc_calibrate_val;
    uint8 calibrated;
    uint8 saved_vhv_init;
    uint8 saved_vhv_timeout;
    uint8 is_started;
    vl53l1x_result_buffer_struct results;
} vl53l1x_context_struct;

/* ===== 私有寄存器驱动原型 (static → 内部链接, 仅 tofL1x* 契约对外公开) ===== */
static uint8 VL53L1XInit(void);
static uint8 VL53L1XSetDistanceMode(vl53l1x_distance_mode_enum mode);
static uint32 VL53L1XGetMeasurementTimingBudget(void);
static uint8 VL53L1XSetMeasurementTimingBudget(uint32 budget_us);
static uint8 VL53L1XDataReady(void);
static uint8 VL53L1XStartContinuous(uint32 period_ms);
static uint16 VL53L1XRead(uint8 blocking);

static vl53l1x_context_struct vl53l1xContext;
static VL53L1X_DATA vl53l1xData;

static void vl53l1xWriteReg(uint16 reg, uint8 value)
{
    tofBusWriteReg16(reg, &value, 1u);
    vl53l1xData.last_status = VL53L1X_STATUS_OK;
}

static void vl53l1xWriteReg16(uint16 reg, uint16 value)
{
    uint8 buffer[2];

    buffer[0] = (uint8)(value >> 8);
    buffer[1] = (uint8)value;
    tofBusWriteReg16(reg, buffer, 2u);
    vl53l1xData.last_status = VL53L1X_STATUS_OK;
}

static void vl53l1xWriteReg32(uint16 reg, uint32 value)
{
    uint8 buffer[4];

    buffer[0] = (uint8)(value >> 24);
    buffer[1] = (uint8)(value >> 16);
    buffer[2] = (uint8)(value >> 8);
    buffer[3] = (uint8)value;
    tofBusWriteReg16(reg, buffer, 4u);
    vl53l1xData.last_status = VL53L1X_STATUS_OK;
}

static uint8 vl53l1xReadReg(uint16 reg)
{
    uint8 value = 0u;

    tofBusReadReg16(reg, &value, 1u);
    vl53l1xData.last_status = VL53L1X_STATUS_OK;
    return value;
}

static uint16 vl53l1xReadReg16(uint16 reg)
{
    uint8 read_buffer[2];

    tofBusReadReg16(reg, read_buffer, 2u);

```



```

    vl53l1xData.last_status = VL53L1X_STATUS_OK;
    return (uint16)((((uint16)read_buffer[0] << 8) | read_buffer[1]));
}

static void vl53l1xReadBlock(uint16 reg, uint8 *data, uint32 len)
{
    tofBusReadReg16(reg, data, len);
    vl53l1xData.last_status = VL53L1X_STATUS_OK;
}

static float vl53l1xCountRateFixedToFloat(uint16 count_rate_fixed)
{
    return (float)count_rate_fixed / (float)(1u << 7);
}

static uint32 vl53l1xDecodeTimeout(uint16 reg_val)
{
    return (((uint32)(reg_val & 0x00FFu) << (reg_val >> 8)) + 1u);
}

static uint16 vl53l1xEncodeTimeout(uint32 timeout_mclks)
{
    uint32 ls_byte = 0u;
    uint16 ms_byte = 0u;

    if (0u == timeout_mclks)
    {
        return 0u;
    }

    ls_byte = timeout_mclks - 1u;
    while ((ls_byte & 0xFFFFF00u) > 0u)
    {
        ls_byte >>= 1;
        ms_byte++;
    }

    return (uint16)((ms_byte << 8) | (ls_byte & 0xFFu));
}

static uint32 vl53l1xTimeoutMclksToMicroseconds(uint32 timeout_mclks, uint32 macro_period_us)
{
    return (uint32)((((uint64)timeout_mclks * macro_period_us + 0x800u) >> 12));
}

static uint32 vl53l1xTimeoutMicrosecondsToMclks(uint32 timeout_us, uint32 macro_period_us)
{
    if (0u == macro_period_us)
    {
        return 0u;
    }

    return (((uint32)timeout_us << 12) + (macro_period_us >> 1)) / macro_period_us;
}

static uint32 vl53l1xCalcMacroPeriod(uint8 vcsel_period)
{
    uint32 pll_period_us;
    uint8 vcsel_period_pclks;
    uint32 macro_period_us;

    if (0u == vl53l1xContext.fast_osc_frequency)
    {
        return 0u;
    }

    pll_period_us = ((uint32)1u << 30) / vl53l1xContext.fast_osc_frequency;
    vcsel_period_pclks = (uint8)((vcsel_period + 1u) << 1);

    macro_period_us = 2304u * pll_period_us;
    macro_period_us >>= 6;
    macro_period_us *= vcsel_period_pclks;
    macro_period_us >>= 6;

    return macro_period_us;
}

static uint8 vl53l1xWaitForBootCompletion(void)
{

```

```

uint16 elapsed_ms = 0u;

while (0u == (vl53l1xReadReg(VL53L1_FIRMWARE__SYSTEM_STATUS) & 0x01u))
{
    if ((vl53l1xData.io_timeout_ms > 0u) && (elapsed_ms >= vl53l1xData.io_timeout_ms))
    {
        vl53l1xData.did_timeout = ZF_TRUE;
        vl53l1xData.last_status = VL53L1X_STATUS_TIMEOUT;
        return ZF_FALSE;
    }

    system_delay_ms(1u);
    elapsed_ms++;
}

return ZF_TRUE;
}

static uint8 vl53l1xWaitForDataReady(void)
{
    uint16 elapsed_ms = 0u;

    while (ZF_FALSE == VL53L1XDataReady())
    {
        if ((vl53l1xData.io_timeout_ms > 0u) && (elapsed_ms >= vl53l1xData.io_timeout_ms))
        {
            vl53l1xData.did_timeout = ZF_TRUE;
            vl53l1xData.last_status = VL53L1X_STATUS_TIMEOUT;
            return ZF_FALSE;
        }

        system_delay_ms(1u);
        elapsed_ms++;
    }

    return ZF_TRUE;
}

static void vl53l1xSetupManualCalibration(void)
{
    vl53l1xContext.saved_vhv_init = vl53l1xReadReg(VL53L1_VHV_CONFIG__INIT);
    vl53l1xContext.saved_vhv_timeout = vl53l1xReadReg(VL53L1_VHV_CONFIG__TIMEOUT_MACROP_LOOP_BOUND);

    vl53l1xWriteReg(VL53L1_VHV_CONFIG__INIT, (uint8)(vl53l1xContext.saved_vhv_init & 0x7Fu));
    vl53l1xWriteReg(
        VL53L1_VHV_CONFIG__TIMEOUT_MACROP_LOOP_BOUND,
        (uint8)((vl53l1xContext.saved_vhv_timeout & 0x03u) + (3u << 2)));
    vl53l1xWriteReg(VL53L1_PHASECAL_CONFIG__OVERRIDE, 0x01u);
    vl53l1xWriteReg(VL53L1_CAL_CONFIG__VCSEL_START, vl53l1xReadReg(VL53L1_PHASECAL_RESULT__VCSEL_START));
}

static void vl53l1xReadResults(void)
{
    uint8 result_buffer[17];

    vl53l1xReadBlock(VL53L1_RESULT__RANGE_STATUS, result_buffer, 17u);
    vl53l1xContext.results.range_status = result_buffer[0];
    vl53l1xContext.results.stream_count = result_buffer[2];
    vl53l1xContext.results.dss_actual_effective_spads_sd0 = (uint16)((((uint16)result_buffer[3] << 8) | result_buffer[4]));
    vl53l1xContext.results.ambient_count_rate_mcps_sd0 = (uint16)((((uint16)result_buffer[7] << 8) | result_buffer[8]));
    vl53l1xContext.results.final_crosstalk_corrected_range_mm_sd0 = (uint16)((((uint16)result_buffer[13] << 8) | result_buffer[14]));
    vl53l1xContext.results.peak_signal_count_rate_crosstalk_corrected_mcps_sd0 = (uint16)((((uint16)result_buffer[15] << 8) | result_buffer[16]));
}

static void vl53l1xUpdateDss(void)
{
    uint16 spad_count = vl53l1xContext.results.dss_actual_effective_spads_sd0;

    if (0u != spad_count)
    {
        uint32 total_rate_per_spad = (uint32)vl53l1xContext.results.peak_signal_count_rate_crosstalk_corrected_mcps_sd0
            + vl53l1xContext.results.ambient_count_rate_mcps_sd0;

        if (total_rate_per_spad > 0xFFFFu)
        {
            total_rate_per_spad = 0xFFFFu;
        }

        total_rate_per_spad <= 16;
    }
}

```

```

total_rate_per_spad /= spad_count;

if (0u != total_rate_per_spad)
{
    uint32 required_spads = ((uint32)VL53L1X_TARGET_RATE << 16) / total_rate_per_spad;

    if (required_spads > 0xFFFFu)
    {
        required_spads = 0xFFFFu;
    }

    vl53l1xWriteReg16(VL53L1_DSS_CONFIG__MANUAL_EFFECTIVE_SPADS_SELECT, (uint16)required_spads);
    return;
}
}

vl53l1xWriteReg16(VL53L1_DSS_CONFIG__MANUAL_EFFECTIVE_SPADS_SELECT, 0x8000u);
}

static void vl53l1xGetRangingData(void)
{
    uint16 range = vl53l1xContext.results.final_crosstalk_corrected_range_mm_sd0;

    vl53l1xData.distance_mm = (uint16)(((uint32)range * 2011u + 0x0400u) / 0x0800u);

    switch (vl53l1xContext.results.range_status)
    {
        case 17:
        case 2:
        case 1:
        case 3:
            vl53l1xData.range_status = VL53L1_RANGESTATUS_HARDWARE_FAIL;
            break;

        case 13:
            vl53l1xData.range_status = VL53L1_RANGESTATUS_MIN_RANGE_FAIL;
            break;

        case 18:
            vl53l1xData.range_status = VL53L1_RANGESTATUS_SYNCRONISATION_INT;
            break;

        case 5:
            vl53l1xData.range_status = VL53L1_RANGESTATUS_OUTOFBOUNDS_FAIL;
            break;

        case 4:
            vl53l1xData.range_status = VL53L1_RANGESTATUS_SIGNAL_FAIL;
            break;

        case 6:
            vl53l1xData.range_status = VL53L1_RANGESTATUS_SIGMA_FAIL;
            break;

        case 7:
            vl53l1xData.range_status = VL53L1_RANGESTATUS_WRAP_TARGET_FAIL;
            break;

        case 12:
            vl53l1xData.range_status = VL53L1_RANGESTATUS_XTALK_SIGNAL_FAIL;
            break;

        case 8:
            vl53l1xData.range_status = VL53L1_RANGESTATUS_RANGE_VALID_MIN_RANGE_CLIPPED;
            break;

        case 9:
            if (0u == vl53l1xContext.results.stream_count)
            {
                vl53l1xData.range_status = VL53L1_RANGESTATUS_RANGE_VALID_NO_WRAP_CHECK_FAIL;
            }
            else
            {
                vl53l1xData.range_status = VL53L1_RANGESTATUS_RANGE_VALID;
            }
            break;

        default:
            vl53l1xData.range_status = VL53L1_RANGESTATUS_NONE;
    }
}

```

```

        break;
    }

    vl53l1xData.peak_signal_count_rate_mcps = vl53l1xCountRateFixedToFloat(
        vl53l1xContext.results.peak_signal_count_rate_crosstalk_corrected_mcps_sd0);
    vl53l1xData.ambient_count_rate_mcps = vl53l1xCountRateFixedToFloat(
        vl53l1xContext.results.ambient_count_rate_mcps_sd0);
}

static void vl53l1xResetState(void)
{
    memset(&vl53l1xContext, 0, sizeof(vl53l1xContext));
    memset(&vl53l1xData, 0, sizeof(vl53l1xData));
    vl53l1xData.range_status = VL53L1_RANGESTATUS_NONE;
    vl53l1xData.distance_mode = VL53L1X_DISTANCE_MODE_UNKNOWN;
    vl53l1xData.io_timeout_ms = VL53L1X_DEFAULT_TIMEOUT_MS;
    vl53l1xData.intermeasurement_period_ms = VL53L1X_DEFAULT_INTERMEASUREMENT_PERIOD_MS;
    vl53l1xData.last_status = VL53L1X_STATUS_NOT_INITIALIZED;
}

uint8 VL53L1XInit(void)
{
    vl53l1xResetState();
    tofBusInit();
    tofBusSetAddress(VL53L1X_IIC_ADDR);
    gpio_init(VL53L1X_IIC_SHUT_PIN, GPIO, GPIO_HIGH, GPIO_PUSH_PULL);
    gpio_init(VL53L1X_IIC_INT_PIN, GPI, GPIO_HIGH, GPI_PULL_UP);

    gpio_low(VL53L1X_IIC_SHUT_PIN);
    system_delay_ms(2u);
    gpio_high(VL53L1X_IIC_SHUT_PIN);
    system_delay_ms(10u);

    if (0xEACCu != vl53l1xReadReg16(VL53L1_IDENTIFICATION_MODEL_ID))
    {
        vl53l1xData.last_status = VL53L1X_STATUS_NOT_DETECTED;
        return ZF_FALSE;
    }

    vl53l1xWriteReg(VL53L1_SOFT_RESET, 0x00u);
    system_delay_us(100u);
    vl53l1xWriteReg(VL53L1_SOFT_RESET, 0x01u);

    if (ZF_FALSE == vl53l1xWaitForBootCompletion())
    {
        return ZF_FALSE;
    }

    vl53l1xWriteReg(
        VL53L1_PAD_I2C_HV_EXTSUP_CONFIG,
        (uint8)(vl53l1xReadReg(VL53L1_PAD_I2C_HV_EXTSUP_CONFIG) | 0x01u));

    vl53l1xContext.fast_osc_frequency = vl53l1xReadReg16(VL53L1_OSC_MEASURED_FAST_OSC_FREQUENCY);
    vl53l1xContext.osc_calibrate_val = vl53l1xReadReg16(VL53L1_RESULT_OSC_CALIBRATE_VAL);

    vl53l1xWriteReg16(VL53L1_DSS_CONFIG_TARGET_TOTAL_RATE_MCPS, VL53L1X_TARGET_RATE);
    vl53l1xWriteReg(VL53L1_GPIO_TIO_HV_STATUS, 0x02u);
    vl53l1xWriteReg(VL53L1_SIGMA_ESTIMATOR_EFFECTIVE_PULSE_WIDTH_NS, 8u);
    vl53l1xWriteReg(VL53L1_SIGMA_ESTIMATOR_EFFECTIVE_AMBIENT_WIDTH_NS, 16u);
    vl53l1xWriteReg(VL53L1_ALGO_CROSSTALK_COMPENSATION_VALID_HEIGHT_MM, 0x01u);
    vl53l1xWriteReg(VL53L1_ALGO_RANGE_IGNORE_VALID_HEIGHT_MM, 0xFFu);
    vl53l1xWriteReg(VL53L1_ALGO_RANGE_MIN_CLIP, 0u);
    vl53l1xWriteReg(VL53L1_ALGO_CONSISTENCY_CHECK_TOLERANCE, 2u);
    vl53l1xWriteReg16(VL53L1_SYSTEM_THRESH_RATE_HIGH, 0x0000u);
    vl53l1xWriteReg16(VL53L1_SYSTEM_THRESH_RATE_LOW, 0x0000u);
    vl53l1xWriteReg(VL53L1_DSS_CONFIG_APERTURE_ATTENUATION, 0x38u);
    vl53l1xWriteReg16(VL53L1_RANGE_CONFIG_SIGMA_THRESH, 360u);
    vl53l1xWriteReg16(VL53L1_RANGE_CONFIG_MIN_COUNT_RATE_RTN_LIMIT_MCPS, 192u);
    vl53l1xWriteReg(VL53L1_SYSTEM_GROUPED_PARAMETER_HOLD_0, 0x01u);
    vl53l1xWriteReg(VL53L1_SYSTEM_GROUPED_PARAMETER_HOLD_1, 0x01u);
    vl53l1xWriteReg(VL53L1_SD_CONFIG_QUANTIFIER, 2u);
    vl53l1xWriteReg(VL53L1_SYSTEM_GROUPED_PARAMETER_HOLD, 0x00u);
    vl53l1xWriteReg(VL53L1_SYSTEM_SEED_CONFIG, 1u);
    vl53l1xWriteReg(VL53L1_SYSTEM_SEQUENCE_CONFIG, 0x8Bu);
    vl53l1xWriteReg16(VL53L1_DSS_CONFIG_MANUAL_EFFECTIVE_SPADS_SELECT, (uint16)(200u << 8));
    vl53l1xWriteReg(VL53L1_DSS_CONFIG_ROI_MODE_CONTROL, 2u);

    vl53l1xData.is_initialized = ZF_TRUE;
}

```

```

if (ZF_FALSE == VL53L1XSetDistanceMode(VL53L1X_DISTANCE_MODE_LONG))
{
    vl53l1xData.is_initialized = ZF_FALSE;
    return ZF_FALSE;
}

if (ZF_FALSE == VL53L1XSetMeasurementTimingBudget(50 000u))
{
    vl53l1xData.is_initialized = ZF_FALSE;
    return ZF_FALSE;
}

vl53l1xWriteReg16(
    VL53L1_ALGO_PART_TO_PART_RANGE_OFFSET_MM,
    (uint16)(vl53l1xReadReg16(VL53L1_MM_CONFIG_OUTER_OFFSET_MM) * 4u));

vl53l1xData.last_status = VL53L1X_STATUS_OK;
return ZF_TRUE;
}

void VL53L1XSetTimeout(uint16 timeout_ms)
{
    vl53l1xData.io_timeout_ms = timeout_ms;
}

uint16 VL53L1XGetTimeout(void)
{
    return vl53l1xData.io_timeout_ms;
}

uint8 VL53L1XTimeoutOccurred(void)
{
    uint8 did_timeout = vl53l1xData.did_timeout;

    vl53l1xData.did_timeout = ZF_FALSE;
    return did_timeout;
}

uint8 VL53L1XSetDistanceMode(vl53l1x_distance_mode_enum mode)
{
    uint32 budget_us;

    if (ZF_FALSE == vl53l1xData.is_initialized)
    {
        vl53l1xData.last_status = VL53L1X_STATUS_NOT_INITIALIZED;
        return ZF_FALSE;
    }

    budget_us = VL53L1XGetMeasurementTimingBudget();

    switch (mode)
    {
        case VL53L1X_DISTANCE_MODE_SHORT:
            vl53l1xWriteReg(VL53L1_RANGE_CONFIG_VCSEL_PERIOD_A, 0x07u);
            vl53l1xWriteReg(VL53L1_RANGE_CONFIG_VCSEL_PERIOD_B, 0x05u);
            vl53l1xWriteReg(VL53L1_RANGE_CONFIG_VALID_PHASE_HIGH, 0x38u);
            vl53l1xWriteReg(VL53L1_SD_CONFIG_WOI_SD0, 0x07u);
            vl53l1xWriteReg(VL53L1_SD_CONFIG_WOI_SD1, 0x05u);
            vl53l1xWriteReg(VL53L1_SD_CONFIG_INITIAL_PHASE_SD0, 6u);
            vl53l1xWriteReg(VL53L1_SD_CONFIG_INITIAL_PHASE_SD1, 6u);
            break;

        case VL53L1X_DISTANCE_MODE_MEDIUM:
            vl53l1xWriteReg(VL53L1_RANGE_CONFIG_VCSEL_PERIOD_A, 0x0Bu);
            vl53l1xWriteReg(VL53L1_RANGE_CONFIG_VCSEL_PERIOD_B, 0x09u);
            vl53l1xWriteReg(VL53L1_RANGE_CONFIG_VALID_PHASE_HIGH, 0x78u);
            vl53l1xWriteReg(VL53L1_SD_CONFIG_WOI_SD0, 0x0Bu);
            vl53l1xWriteReg(VL53L1_SD_CONFIG_WOI_SD1, 0x09u);
            vl53l1xWriteReg(VL53L1_SD_CONFIG_INITIAL_PHASE_SD0, 10u);
            vl53l1xWriteReg(VL53L1_SD_CONFIG_INITIAL_PHASE_SD1, 10u);
            break;

        case VL53L1X_DISTANCE_MODE_LONG:
            vl53l1xWriteReg(VL53L1_RANGE_CONFIG_VCSEL_PERIOD_A, 0x0Fu);
            vl53l1xWriteReg(VL53L1_RANGE_CONFIG_VCSEL_PERIOD_B, 0x0Du);
            vl53l1xWriteReg(VL53L1_RANGE_CONFIG_VALID_PHASE_HIGH, 0xB8u);
            vl53l1xWriteReg(VL53L1_SD_CONFIG_WOI_SD0, 0x0Fu);
            vl53l1xWriteReg(VL53L1_SD_CONFIG_WOI_SD1, 0x0Du);
            vl53l1xWriteReg(VL53L1_SD_CONFIG_INITIAL_PHASE_SD0, 14u);

```

```

        vl53l1xWriteReg(VL53L1_SD_CONFIG__INITIAL_PHASE_SD1, 14u);
        break;

    default:
        vl53l1xData.last_status = VL53L1X_STATUS_INVALID_PARAM;
        return ZF_FALSE;
    }

    vl53l1xData.distance_mode = mode;

    if ((0u != budget_us) && (ZF_FALSE == VL53L1XSetMeasurementTimingBudget(budget_us)))
    {
        return ZF_FALSE;
    }

    vl53l1xData.last_status = VL53L1X_STATUS_OK;
    return ZF_TRUE;
}

vl53l1x_distance_mode_enum VL53L1XGetDistanceMode(void)
{
    return vl53l1xData.distance_mode;
}

uint8 VL53L1XSetMeasurementTimingBudget(uint32 budget_us)
{
    uint32 range_config_timeout_us;
    uint32 macro_period_us;
    uint32 phasecal_timeout_mclks;

    if (ZF_FALSE == vl53l1xData.is_initialized)
    {
        vl53l1xData.last_status = VL53L1X_STATUS_NOT_INITIALIZED;
        return ZF_FALSE;
    }

    if (budget_us <= VL53L1X_TIMING_GUARD_US)
    {
        vl53l1xData.last_status = VL53L1X_STATUS_INVALID_PARAM;
        return ZF_FALSE;
    }

    range_config_timeout_us = budget_us - VL53L1X_TIMING_GUARD_US;
    if (range_config_timeout_us > 1 100 000u)
    {
        vl53l1xData.last_status = VL53L1X_STATUS_INVALID_PARAM;
        return ZF_FALSE;
    }

    range_config_timeout_us /= 2u;
    macro_period_us = vl53l1xCalcMacroPeriod(vl53l1xReadReg(VL53L1_RANGE_CONFIG__VCSEL_PERIOD_A));
    if (0u == macro_period_us)
    {
        vl53l1xData.last_status = VL53L1X_STATUS_INVALID_PARAM;
        return ZF_FALSE;
    }

    phasecal_timeout_mclks = vl53l1xTimeoutMicrosecondsToMclks(1000u, macro_period_us);
    if (phasecal_timeout_mclks > 0xFFu)
    {
        phasecal_timeout_mclks = 0xFFu;
    }

    vl53l1xWriteReg(VL53L1_PHASECAL_CONFIG__TIMEOUT_MACROP, (uint8)phasecal_timeout_mclks);
    vl53l1xWriteReg16(
        VL53L1_MM_CONFIG__TIMEOUT_MACROP_A,
        vl53l1xEncodeTimeout(vl53l1xTimeoutMicrosecondsToMclks(1u, macro_period_us)));
    vl53l1xWriteReg16(
        VL53L1_RANGE_CONFIG__TIMEOUT_MACROP_A,
        vl53l1xEncodeTimeout(vl53l1xTimeoutMicrosecondsToMclks(range_config_timeout_us, macro_period_us)));

    macro_period_us = vl53l1xCalcMacroPeriod(vl53l1xReadReg(VL53L1_RANGE_CONFIG__VCSEL_PERIOD_B));
    if (0u == macro_period_us)
    {
        vl53l1xData.last_status = VL53L1X_STATUS_INVALID_PARAM;
        return ZF_FALSE;
    }

    vl53l1xWriteReg16(

```

```

        VL53L1_MM_CONFIG_TIMEOUT_MACROP_B,
        vl53l1xEncodeTimeout(vl53l1xTimeoutMicrosecondsToMcIs(1u, macro_period_us)));
vl53l1xWriteReg16(
    VL53L1_RANGE_CONFIG_TIMEOUT_MACROP_B,
    vl53l1xEncodeTimeout(vl53l1xTimeoutMicrosecondsToMcIs(range_config_timeout_us, macro_period_us)));

    vl53l1xData.last_status = VL53L1X_STATUS_OK;
    return ZF_TRUE;
}

uint32 VL53L1XGetMeasurementTimingBudget(void)
{
    uint32 macro_period_us;
    uint32 range_config_timeout_us;

    if (ZF_FALSE == vl53l1xData.is_initialized)
    {
        return 0u;
    }

    macro_period_us = vl53l1xCalcMacroPeriod(vl53l1xReadReg(VL53L1_RANGE_CONFIG_VCSEL_PERIOD_A));
    if (0u == macro_period_us)
    {
        return 0u;
    }

    range_config_timeout_us = vl53l1xTimeoutMcIsToMicroseconds(
        vl53l1xDecodeTimeout(vl53l1xReadReg16(VL53L1_RANGE_CONFIG_TIMEOUT_MACROP_A)),
        macro_period_us);

    return (2u * range_config_timeout_us) + VL53L1X_TIMING_GUARD_US;
}

uint8 VL53L1XDataReady(void)
{
    if (ZF_FALSE == vl53l1xData.is_initialized)
    {
        return ZF_FALSE;
    }

    return (uint8)((vl53l1xReadReg(VL53L1_GPIO__TIO_HV_STATUS) & 0x01u) == 0u);
}

uint8 VL53L1XStartContinuous(uint32 period_ms)
{
    if (ZF_FALSE == vl53l1xData.is_initialized)
    {
        vl53l1xData.last_status = VL53L1X_STATUS_NOT_INITIALIZED;
        return ZF_FALSE;
    }

    if (0u == period_ms)
    {
        period_ms = vl53l1xData.intermeasurement_period_ms;
    }

    if (0u == period_ms)
    {
        period_ms = VL53L1X_DEFAULT_INTERMEASUREMENT_PERIOD_MS;
    }

    if (0u == vl53l1xContext.osc_calibrate_val)
    {
        vl53l1xData.last_status = VL53L1X_STATUS_INVALID_PARAM;
        return ZF_FALSE;
    }

    vl53l1xData.intermeasurement_period_ms = period_ms;
    vl53l1xWriteReg32(
        VL53L1_SYSTEM__INTERMEASUREMENT_PERIOD,
        period_ms * (uint32)vl53l1xContext.osc_calibrate_val);
    vl53l1xWriteReg(VL53L1_SYSTEM__INTERRUPT_CLEAR, 0x01u);
    vl53l1xWriteReg(VL53L1_SYSTEM__MODE_START, 0x40u);

    vl53l1xContext.is_started = ZF_TRUE;
    vl53l1xData.is_continuous_mode = ZF_TRUE;
    vl53l1xData.last_status = VL53L1X_STATUS_OK;
    return ZF_TRUE;
}

```

```

}

void VL53L1XStopContinuous(void)
{
    if (ZF_FALSE == vl53l1xData.is_initialized)
    {
        return;
    }

    vl53l1xWriteReg(VL53L1_SYSTEM__MODE_START, 0x80u);
    vl53l1xContext.calibrated = ZF_FALSE;

    if (0u != vl53l1xContext.saved_vhv_init)
    {
        vl53l1xWriteReg(VL53L1_VHV_CONFIG__INIT, vl53l1xContext.saved_vhv_init);
    }

    if (0u != vl53l1xContext.saved_vhv_timeout)
    {
        vl53l1xWriteReg(VL53L1_VHV_CONFIG__TIMEOUT_MACROP_LOOP_BOUND, vl53l1xContext.saved_vhv_timeout);
    }

    vl53l1xWriteReg(VL53L1_PHASECAL_CONFIG__OVERRIDE, 0x00u);
    vl53l1xContext.is_started = ZF_FALSE;
    vl53l1xData.is_continuous_mode = ZF_FALSE;
    vl53l1xData.last_status = VL53L1X_STATUS_OK;
}

uint16 VL53L1XRead(uint8 blocking)
{
    if (ZF_FALSE == vl53l1xData.is_initialized)
    {
        vl53l1xData.last_status = VL53L1X_STATUS_NOT_INITIALIZED;
        return 0u;
    }

    if (ZF_FALSE == vl53l1xContext.is_started)
    {
        if (ZF_FALSE == VL53L1XStartContinuous(vl53l1xData.intermeasurement_period_ms))
        {
            return 0u;
        }
    }

    if (blocking)
    {
        if (ZF_FALSE == vl53l1xWaitForDataReady())
        {
            vl53l1xData.range_status = VL53L1_RANGESTATUS_NONE;
            vl53l1xData.distance_mm = 0u;
            vl53l1xData.peak_signal_count_rate_mcps = 0.0f;
            vl53l1xData.ambient_count_rate_mcps = 0.0f;
            return 0u;
        }
    }
    else if (ZF_FALSE == VL53L1XDataReady())
    {
        vl53l1xData.last_status = VL53L1X_STATUS_DATA_NOT_READY;
        return vl53l1xData.distance_mm;
    }

    vl53l1xReadResults();

    if (ZF_FALSE == vl53l1xContext.calibrated)
    {
        vl53l1xSetupManualCalibration();
        vl53l1xContext.calibrated = ZF_TRUE;
    }

    vl53l1xUpdateDss();
    vl53l1xGetRangingData();
    vl53l1xWriteReg(VL53L1_SYSTEM__INTERRUPT_CLEAR, 0x01u);
    vl53l1xData.last_status = VL53L1X_STATUS_OK;
    return vl53l1xData.distance_mm;
}

const char *VL53L1XRangeStatusToString(uint8 status)
{
    switch (status)

```



```

{
    case VL53L1_RANGESTATUS_RANGE_VALID:
        return "RANGE VALID";

    case VL53L1_RANGESTATUS_SIGMA_FAIL:
        return "SIGMA FAIL";

    case VL53L1_RANGESTATUS_SIGNAL_FAIL:
        return "SIGNAL FAIL";

    case VL53L1_RANGESTATUS_RANGE_VALID_MIN_RANGE_CLIPPED:
        return "RANGE VALID MIN RANGE CLIPPED";

    case VL53L1_RANGESTATUS_OUTOFBOUNDS_FAIL:
        return "OUTOFBOUNDS FAIL";

    case VL53L1_RANGESTATUS_HARDWARE_FAIL:
        return "HARDWARE FAIL";

    case VL53L1_RANGESTATUS_RANGE_VALID_NO_WRAP_CHECK_FAIL:
        return "RANGE VALID NO WRAP CHECK FAIL";

    case VL53L1_RANGESTATUS_WRAP_TARGET_FAIL:
        return "WRAP TARGET FAIL";

    case VL53L1_RANGESTATUS_PROCESSING_FAIL:
        return "PROCESSING FAIL";

    case VL53L1_RANGESTATUS_XTALK_SIGNAL_FAIL:
        return "XTALK SIGNAL FAIL";

    case VL53L1_RANGESTATUS_SYNCRONISATION_INT:
        return "SYNCRONISATION INT";

    case VL53L1_RANGESTATUS_RANGE_VALID_MERGED_PULSE:
        return "RANGE VALID MERGED PULSE";

    case VL53L1_RANGESTATUS_TARGET_PRESENT_LACK_OF_SIGNAL:
        return "TARGET PRESENT LACK OF SIGNAL";

    case VL53L1_RANGESTATUS_MIN_RANGE_FAIL:
        return "MIN RANGE FAIL";

    case VL53L1_RANGESTATUS_RANGE_INVALID:
        return "RANGE INVALID";

    case VL53L1_RANGESTATUS_NONE:
    default:
        return "NONE";
}
}

void VL53L1XGetData(void)
{
    (void)VL53L1XRead(ZF_TRUE);
}

/* =====
* 统一对外契约（与 tofL8cx 对齐，由 altitudeTof 调度）
* ===== */
uint8_t tofL1xInit(void)
{
    if (ZF_FALSE == VL53L1XInit())
    {
        return 0u;
    }
    if (ZF_FALSE == VL53L1XSetMeasurementTimingBudget(ALT_TOF_L1X_TIMING_BUDGET_US))
    {
        return 0u;
    }
    if (ZF_FALSE == VL53L1XStartContinuous(ALT_TOF_L1X_SAMPLE_MS))
    {
        return 0u;
    }
    return 1u;
}

uint8_t tofL1xPoll(uint16_t *distanceMm, uint8_t *valid)
{

```

```

if (ZF_FALSE == VL53L1XDataReady())
{
    return 0u;
}
*distanceMm = VL53L1XRead(0u);
*valid = (uint8_t) (vl53l1xData.range_status == VL53L1_RANGESTATUS_RANGE_VALID ||
    vl53l1xData.range_status == VL53L1_RANGESTATUS_RANGE_VALID_MIN_RANGE_CLIPPED ||
    vl53l1xData.range_status == VL53L1_RANGESTATUS_RANGE_VALID_NO_WRAP_CHECK_FAIL ||
    vl53l1xData.range_status == VL53L1_RANGESTATUS_RANGE_VALID_MERGED_PULSE);
return 1u;
}

float tofL1xSamplePeriodS(void)
{
    return (float) ALT_TOF_L1X_SAMPLE_MS / 1000.0f;
}

```

core0_cyt4bb7/project/code/sensor/tof/tofL1x.h

SHA-256

:

d036f087fb13247199280b14b09e7377293aed82256be819d14b684a8f47f128

代码 (c)

```

#ifndef _TOF_L1X_H_
#define _TOF_L1X_H_

#include <stdint.h>

/* VL53L1X 驱动（单区，软件 IIC）。统一契约： Init / Poll / SamplePeriodS，
 * 与 tofL8cx 对齐，由 altitudeTof 调度。内部寄存器驱动与状态码私有于 tofL1x.c。 */

uint8_t tofL1xInit(void);

/* 返回 1 表示产生了一帧； valid=0 时该帧不进入定高闭环。 */
uint8_t tofL1xPoll(uint16_t *distanceMm, uint8_t *valid);
float tofL1xSamplePeriodS(void);

#endif /* _TOF_L1X_H_ */

```

core0_cyt4bb7/project/code/sensor/tof/tofL8cx.c

SHA-256

:

05eac1d11e97284cdb8be860147abd301743b19ca30476d403c4ef3b6f1cf3b6

代码 (c)

```

#include "tofL8cx.h"

#include <string.h>

#include "configAltitudeTof.h"
#include "vl53lmz_api.h"

#define VL53L8CX_ZONE_COUNT      (16u)
#define VL53L8CX_STATUS_VALID    (5u)
#define VL53L8CX_STATUS_VALID_LARGE_PULSE (9u)

#if ALT_TOF_L8CX_FREQUENCY_HZ == 0u || ALT_TOF_L8CX_FREQUENCY_HZ > 60u
#error "VL53L8CX 4x4 ranging frequency must be in 1..60Hz"
#endif

#if ALT_TOF_L8CX_MIN_VALID_ZONES == 0u || ALT_TOF_L8CX_MIN_VALID_ZONES > VL53L8CX_ZONE_COUNT
#error "ALT_TOF_L8CX_MIN_VALID_ZONES must be in 1..16"
#endif

static VL53LMZ_Configuration device;
static VL53LMZ_ResultsData results;
static uint8_t lastStreamCount;
static uint8_t hasStreamCount;

```

```

static float samplePeriodS = 1.0f / (float) ALT_TOF_L8CX_FREQUENCY_HZ;

static void sortDistances(uint16_t *values, uint8_t count)
{
    uint8_t i;

    for (i = 1u; i < count; i++)
    {
        uint16_t value = values[i];
        uint8_t j = i;
        while (j > 0u && values[j - 1u] > value)
        {
            values[j] = values[j - 1u];
            j--;
        }
        values[j] = value;
    }
}

static uint16_t medianDistance(const uint16_t *values, uint8_t count)
{
    uint8_t middle = (uint8_t) (count / 2u);
    if ((count & 1u) != 0u)
    {
        return values[middle];
    }
    return (uint16_t) (((uint32_t) values[middle - 1u] + values[middle]) / 2u);
}

static uint8_t fuseHeight(uint16_t *distanceMm)
{
    uint16_t distances[VL53L8CX_ZONE_COUNT];
    uint8_t count = 0u;
    uint8_t zone;

    for (zone = 0u; zone < VL53L8CX_ZONE_COUNT; zone++)
    {
        uint8_t status = results.target_status[zone];
        int16_t distance = results.distance_mm[zone];
        if (results.nb_target_detected[zone] != 0u &&
            (status == VL53L8CX_STATUS_VALID || status == VL53L8CX_STATUS_VALID_LARGE_PULSE) &&
            distance >= (int16_t) ALT_TOF_L8CX_HARD_MIN_MM &&
            distance <= (int16_t) ALT_TOF_L8CX_HARD_MAX_MM)
        {
            distances[count++] = (uint16_t) distance;
        }
    }

    if (count < ALT_TOF_L8CX_MIN_VALID_ZONES)
    {
        return 0u;
    }

    sortDistances(distances, count);
    uint16_t preliminary = medianDistance(distances, count);
    uint16_t relativeTolerance = (uint16_t) (((uint32_t) preliminary * ALT_TOF_L8CX_OUTLIER_REL_PERMILLE) / 1000u);
    uint16_t tolerance = relativeTolerance > ALT_TOF_L8CX_OUTLIER_ABS_MM
        ? relativeTolerance
        : ALT_TOF_L8CX_OUTLIER_ABS_MM;

    uint8_t first = 0u;
    uint8_t last = count;

    while (first < last &&
        ((uint32_t) distances[first] + tolerance < preliminary))
    {
        first++;
    }
    while (last > first &&
        distances[last - 1u] > (uint32_t) preliminary + tolerance)
    {
        last--;
    }

    uint8_t inlierCount = (uint8_t) (last - first);
    if (inlierCount < ALT_TOF_L8CX_MIN_VALID_ZONES)
    {
        return 0u;
    }
}

```

```

        *distanceMm = medianDistance(&distances[first], inlierCount);
        return 1u;
    }

uint8_t tofL8cxInit(void)
{
    uint8_t alive = 0u;
    uint8_t status;

    memset(&device, 0, sizeof(device));
    memset(&results, 0, sizeof(results));
    lastStreamCount = 0u;
    hasStreamCount = 0u;
    samplePeriodS = 1.0f / (float) ALT_TOF_L8CX_FREQUENCY_HZ;
    device.platform.address = VL53LMZ_DEFAULT_I2C_ADDRESS;

    status = vl53lmzPlatformInit();
    status |= vl53lmz_is_alive(&device, &alive);
    if (status != VL53LMZ_STATUS_OK || !alive)
    {
        return 0u;
    }

    status = vl53lmz_init(&device);
    if (status != VL53LMZ_STATUS_OK || device.module_type != VL53LMZ_MODULE_TYPE_L8)
    {
        return 0u;
    }

    status |= vl53lmz_set_resolution(&device, VL53LMZ_RESOLUTION_4X4);
    status |= vl53lmz_set_ranging_mode(&device, VL53LMZ_RANGING_MODE_CONTINUOUS);
    status |= vl53lmz_set_ranging_frequency_hz(&device, ALT_TOF_L8CX_FREQUENCY_HZ);
    status |= vl53lmz_start_ranging(&device);
    return (uint8_t) (status == VL53LMZ_STATUS_OK);
}

/* 时间域创新门控已统一上移到 altitudeTof facade (L1X/L8CX 共用) ;
 * 本驱动只做 4x4 空间中值 + 离群融合，输出单帧候选高度。 */
uint8_t tofL8cxPoll(uint16_t *distanceMm, uint8_t *valid)
{
    {
        uint8_t ready = 0u;
        uint8_t status = vl53lmz_check_data_ready(&device, &ready);

        if (status != VL53LMZ_STATUS_OK || !ready)
        {
            return 0u;
        }

        status = vl53lmz_get_ranging_data(&device, &results);
        uint8_t streamCount = device.streamcount;
        if (hasStreamCount)
        {
            uint8_t frameDelta = streamCount >= lastStreamCount
                ? (uint8_t) (streamCount - lastStreamCount)
                : (uint8_t) ((256u - lastStreamCount) + streamCount);
            if (frameDelta == 0u) frameDelta = 1u;
            samplePeriodS = (float) frameDelta / (float) ALT_TOF_L8CX_FREQUENCY_HZ;
        }
        lastStreamCount = streamCount;
        hasStreamCount = 1u;

        *valid = 0u;
        if (status == VL53LMZ_STATUS_OK)
        {
            uint16_t fused;
            /* 仅做 4x4 空间中值融合；时间域门控由 altitudeTof facade 统一处理。 */
            if (fuseHeight(&fused))
            {
                *distanceMm = fused;
                *valid = 1u;
            }
        }
        return 1u;
    }
}

float tofL8cxSamplePeriodS(void)
{
    return samplePeriodS;
}

```

core0_cyt4bb7/project/code/sensor/tof/tofL8cx.h

SHA-256 :

378aa1503780b016e6dc32048bec602a49c140f8cdf3564e3c7673ffb86fa293

代码 (c)

```
#ifndef _TOF_L8CX_H_
#define _TOF_L8CX_H_

#include <stdint.h>

/* VL53L8CX 驱动（4x4 多区 + 中值/离群融合，基于 ST ULD）。
 * 统一契约：Init / Poll / SamplePeriodS，与 tofL1x 对齐，由 altitudeToF 调度。*/

uint8_t tofL8cxInit(void);

/* 返回 1 表示消费了一帧；valid=0 表示该帧无法形成可信单高度。*/
uint8_t tofL8cxPoll(uint16_t *distanceMm, uint8_t *valid);
float tofL8cxSamplePeriodS(void);

#endif /* _TOF_L8CX_H_ */
```

core0_cyt4bb7/project/code/sensor/velocityEstimator.c

SHA-256 :

fd1af0193cf2b3dd6fcb32c901b7b9ecc2df514d1cca8672b82ec05562873734

代码 (c)

```
#include "velocityEstimator.h"

#include <math.h>

#include "zf_common_interrupt.h"

#include "IMU.h"
#include "altitudeToF.h"
#include "commonDef.h"
#include "configVelocityEstimator.h"
#include "opticalFlow.h"

#define VELOCITY_ESTIMATOR_AXES 2u /* [0]=X 前向, [1]=Y 右向 */

/* 单轴融合状态：KF 状态 [v,b] + 对称协方差三项 P00/P01/P11 + 诊断量 + 延迟补偿历史。*/
typedef struct
{
    volatile float acceleration; /* 机体水平航向系去重力加速度 m/s² */
    volatile float imuOnlyVelocity; /* 纯 IMU 积分参考（固定 bias，不被光流校正，影子对比漂移）*/
    volatile float fusedVelocity; /* KF 融合速度 m/s */
    volatile float bias; /* 等效加速度 bias m/s² */
    volatile float p00;
    volatile float p01;
    volatile float p11;
    volatile float flowVelocity; /* 光流换算物理速度（诊断）*/
    volatile float innovation; /* 量测残差（诊断）*/
    /* 融合速度环形历史（1ms 预测每拍写入），供光流帧延迟补偿取“延迟时刻”估计。*/
    volatile float fusedVelHist[VELOCITY_ESTIMATOR_VEL_HIST_LEN];
} velocityEstimatorAxisStruct;

typedef struct
{
    velocityEstimatorAxisStruct axis[VELOCITY_ESTIMATOR_AXES];
    volatile uint16_t histWritIdx; /* X/Y 共享：两轴同拍写入后统一前进一格*/
    volatile float heightM;
    volatile uint16_t frameAgeMs; /* 最后一帧（含无效帧）年龄：链路活性*/
    volatile uint16_t validAgeMs; /* 最后一帧通过全部门控的有效观测年龄*/
    volatile uint16_t consecutiveInvalidFrames; /* 距上次成功 KF 校正以来连续被拒的光流帧数（每收一帧+1，成功校正归零）*/
    volatile uint32_t predictGeneration; /* 1ms 预测拍代数，拒绝提交基于旧预测的前台校正*/
    volatile float lastRawFlowX; /* 最近一帧原始光流计数 flowX（前正，未换算，常态调试用）*/
```

```

    volatile float lastRawFlowY; /* 最近一帧原始光流计数 flowY（右正，未换算） */
    volatile uint8_t flowValid;
    volatile uint8_t toFresh;
} velocityEstimatorStateStruct;

/* 单轴 KF 量测更新结果（纯函数式产出，校验通过后才提交到 state）。 */
typedef struct
{
    float velocity;
    float bias;
    float p00;
    float p01;
    float p11;
} velocityEstimatorKalmanResultStruct;

static volatile velocityEstimatorStateStruct velocityEstimatorState;
static volatile uint16_t velocityEstimatorBiasCalibrationCount; /* X/Y 共享单计数 */
static float velocityEstimatorBiasCalibrationSum[VELOCITY_ESTIMATOR_AXES];
static float velocityEstimatorImuOnlyBias[VELOCITY_ESTIMATOR_AXES];

static float velocityEstimatorClamp(float value, float limit)
{
    return clampf(value, -limit, limit);
}

/* 有限性检查：NaN/Inf 返回 0（clampf 挡不住 NaN，需独立守卫防污染积分）。 */
static uint8_t velocityEstimatorIsFinite(float v)
{
    /* NaN != 自身；Inf 的差为 Inf。用乘 0 判断：有限数×0=0，NaN/Inf×0=NaN/Inf。 */
    float z = v * 0.0f;
    return (z == 0.0f) ? 1u : 0u;
}

/* 每轴状态 x=[v,b]^T, F=[[1,-dt],[0,1]], P 仅存对称项 P00/P01/P11。 */
static void velocityEstimatorKalmanResetCovarianceAxis(
    volatile float *p00, volatile float *p01, volatile float *p11)
{
    *p00 = VELOCITY_ESTIMATOR_KF_P_INIT_VEL;
    *p01 = 0.0f;
    *p11 = VELOCITY_ESTIMATOR_KF_P_INIT_BIAS;
}

/* 对角项上下限保护；若浮点误差使 det(P)=P00*P11-P01^2<0，清交叉项恢复保守对角。 */
static uint8_t velocityEstimatorKalmanSanitizeCovariance(
    float *p00, float *p01, float *p11)
{
    float determinant;

    if (!velocityEstimatorIsFinite(*p00) ||
        !velocityEstimatorIsFinite(*p01) ||
        !velocityEstimatorIsFinite(*p11))
    {
        return 0u;
    }

    *p00 = clampf(*p00, VELOCITY_ESTIMATOR_KF_P_MIN, VELOCITY_ESTIMATOR_KF_P_MAX);
    *p01 = clampf(*p01, -VELOCITY_ESTIMATOR_KF_P_MAX, VELOCITY_ESTIMATOR_KF_P_MAX);
    *p11 = clampf(*p11, VELOCITY_ESTIMATOR_KF_P_MIN, VELOCITY_ESTIMATOR_KF_P_MAX);
    determinant = *p00 * *p11 - *p01 * *p01;
    if (!velocityEstimatorIsFinite(determinant))
    {
        return 0u;
    }
    if (determinant < 0.0f)
    {
        *p01 = 0.0f;
    }
    return 1u;
}

/* P=F*P*F^T+Q 标量展开 (F=[[1,-dt],[0,1]]) :
 * P00' = P00 - 2*dt*P01 + dt^2*P11 + qv
 * P01' = P01 - dt*P11
 * P11' = P11 + qb
 * 无 sqrtf / 除法，适配 1ms ISR。 */
static void velocityEstimatorKalmanPredictCovarianceAxis(
    volatile float *p00State, volatile float *p01State, volatile float *p11State)
{
    float p00 = *p00State;

```

```

float p01 = *p01State;
float p11 = *p11State;
float dt = VELOCITY_ESTIMATOR_PREDICT_DT_S;
float nextP00;
float nextP01;
float nextP11;

if (!velocityEstimatorKalmanSanitizeCovariance(&p00, &p01, &p11))
{
    velocityEstimatorKalmanResetCovarianceAxis(p00State, p01State, p11State);
    return;
}

nextP00 = p00 - 2.0f * dt * p01 + dt * dt * p11 + VELOCITY_ESTIMATOR_KF_Q_VEL;
nextP01 = p01 - dt * p11;
nextP11 = p11 + VELOCITY_ESTIMATOR_KF_Q_BIAS;

if (!velocityEstimatorKalmanSanitizeCovariance(&nextP00, &nextP01, &nextP11))
{
    velocityEstimatorKalmanResetCovarianceAxis(p00State, p01State, p11State);
    return;
}

*p00State = nextP00;
*p01State = nextP01;
*p11State = nextP11;
}

/* H=[1,0], S=P00+R, K=[P00/S,P01/S]^T。Joseph 协方差更新标量展开：
* P00' = (1-K0)^2*P00 + K0^2*R
* P01' = (1-K0)*(P01-K1*P00) + K0*K1*R
* P11' = P11 - 2*K1*P01 + K1^2*P00 + K1^2*R
* 纯函数式：只写 result，校验失败返回 0（调用方不提交）。*/
static uint8_t velocityEstimatorKalmanUpdateAxis(
    float velocity, float bias,
    float p00, float p01, float p11,
    float innovation, float measurementVariance,
    velocityEstimatorKalmanResultStruct *result)
{
    float innovationVariance;
    float gainVelocity;
    float gainBias;
    float oneMinusGainVelocity;
    float nextP00;
    float nextP01;
    float nextP11;

    if (!velocityEstimatorIsFinite(velocity) ||
        !velocityEstimatorIsFinite(bias) ||
        !velocityEstimatorIsFinite(innovation) ||
        !velocityEstimatorIsFinite(measurementVariance) ||
        !velocityEstimatorKalmanSanitizeCovariance(&p00, &p01, &p11))
    {
        return 0u;
    }

    innovationVariance = p00 + measurementVariance;
    if (innovationVariance <= VELOCITY_ESTIMATOR_KF_P_MIN ||
        !velocityEstimatorIsFinite(innovationVariance))
    {
        return 0u;
    }

    gainVelocity = clampf(
        p00 / innovationVariance, 0.0f, VELOCITY_ESTIMATOR_KF_GAIN_VEL_MAX);
    gainBias = clampf(
        p01 / innovationVariance,
        -VELOCITY_ESTIMATOR_KF_GAIN_BIAS_MAX,
        VELOCITY_ESTIMATOR_KF_GAIN_BIAS_MAX);

    result->velocity = velocityEstimatorClamp(
        velocity + gainVelocity * innovation, VELOCITY_ESTIMATOR_VELOCITY_LIMIT_MPS);
    result->bias = velocityEstimatorClamp(
        bias + gainBias * innovation, VELOCITY_ESTIMATOR_BIAS_LIMIT_MPS2);

    oneMinusGainVelocity = 1.0f - gainVelocity;
    nextP00 = oneMinusGainVelocity * oneMinusGainVelocity * p00
        + gainVelocity * gainVelocity * measurementVariance;
    nextP01 = oneMinusGainVelocity * (p01 - gainBias * p00)

```

```

    + gainVelocity * gainBias * measurementVariance;
nextP11 = p11 - 2.0f * gainBias * p01
    + gainBias * gainBias * p00
    + gainBias * gainBias * measurementVariance;

if (!velocityEstimatorIsFinite(result->velocity) ||
    !velocityEstimatorIsFinite(result->bias) ||
    !velocityEstimatorKalmanSanitizeCovariance(&nextP00, &nextP01, &nextP11))
{
    return 0u;
}
result->p00 = nextP00;
result->p01 = nextP01;
result->p11 = nextP11;
return 1u;
}

/* Mahony q 为传感器系 S 到本地系 L 的姿态。目标输出“机体水平航向系”速度
* （前向/右向，跟随机头 yaw），因此须去掉 q 的 yaw 分量，只保留 pitch/roll 倾斜补偿。
* 否则用完整 q（含 yaw）会把速度转到世界固定系，导致 xy 速度与 yaw 强耦合
* （机体匀速前飞，yaw 转 90° 后输出却变成“向右飞”）；且 6 轴 yaw 不可观会漂移，
* 双重污染。这里构造 R_tilt = R_yaw^T · R(q) 消 yaw：
* yaw 由 body-X 轴在世界水平面投影定， $\cos\psi=r00/h$ ,  $\sin\psi=r10/h$ ,  $h=\sqrt{(r00^2+r10^2)}$ 。
* 输出端再做固定 L_S→L_FRD 重映射（前=X 正/右=Y 正）：newX=-tiltY、newY=-tiltX。
* IMU(真实 S 去重力加速度) 与光流(经 BODY_TO_SENSOR 转成 S) 共用本 helper。*/
static void velocityEstimatorRotateSensorToLocal(
    float q0, float q1, float q2, float q3,
    float sensorX, float sensorY, float sensorZ,
    float *localX, float *localY)
{
    float r00 = 1.0f - 2.0f * (q2 * q2 + q3 * q3);
    float r01 = 2.0f * (q1 * q2 - q0 * q3);
    float r02 = 2.0f * (q1 * q3 + q0 * q2);
    float r10 = 2.0f * (q1 * q2 + q0 * q3);
    float r11 = 1.0f - 2.0f * (q1 * q1 + q3 * q3);
    float r12 = 2.0f * (q2 * q3 - q0 * q1);

    /* 去 yaw：R_tilt = R_yaw^T · R。yaw 由 body-X 世界水平投影定。*/
    float h = sqrtf(r00 * r00 + r10 * r10);
    float cosPsi;
    float sinPsi;
    float tilt00;
    float tilt01;
    float tilt02;
    float tilt10;
    float tilt11;
    float tilt12;
    float localSX;
    float localSY;

    if (h < 1.0e-6f)
    {
        /* 接近垂直（body-X 指天/地），yaw 无定义，退化为不去 yaw（极端姿态，已被倾角门控挡）*/
        cosPsi = 1.0f;
        sinPsi = 0.0f;
    }
    else
    {
        cosPsi = r00 / h;
        sinPsi = r10 / h;
    }

    /* R_tilt 行 0 = cosPsi·R[0] + sinPsi·R[1]；行 1 = -sinPsi·R[0] + cosPsi·R[1] */
    tilt00 = cosPsi * r00 + sinPsi * r10; /* = h */
    tilt01 = cosPsi * r01 + sinPsi * r11;
    tilt02 = cosPsi * r02 + sinPsi * r12;
    tilt10 = -sinPsi * r00 + cosPsi * r10; /* = 0 */
    tilt11 = -sinPsi * r01 + cosPsi * r11;
    tilt12 = -sinPsi * r02 + cosPsi * r12;

    localSX = tilt00 * sensorX + tilt01 * sensorY + tilt02 * sensorZ;
    localSY = tilt10 * sensorX + tilt11 * sensorY + tilt12 * sensorZ;

    /* L_S → L_FRD：前向(X)=-L_S.Y，右向(Y)=-L_S.X。*/
    *localX = -localSY;
    *localY = -localSX;
}

/* 前台用：关中断快照 imuData（四元数/gyro/姿态），保证多字段一致。*/

```



```

* physicalPitch/physicalRoll 出参允许 NULL（标定 store 路径不需要姿态角）。*/
static void velocityEstimatorSnapshotImu(
    float q[4], float *gyroX, float *gyroY, float *gyroZ,
    float *physicalPitch, float *physicalRoll)
{
    imuSnapshotStruct snapshot;
    IMUGetSnapshot(&snapshot);
    q[0] = snapshot.q[0];
    q[1] = snapshot.q[1];
    q[2] = snapshot.q[2];
    q[3] = snapshot.q[3];
    *gyroX = snapshot.gyroPitchRaw;
    *gyroY = snapshot.gyroRollRaw;
    *gyroZ = snapshot.gyroYawRaw;
    if (physicalPitch != 0)
    {
        *physicalPitch = snapshot.physicalPitch;
    }
    if (physicalRoll != 0)
    {
        *physicalRoll = snapshot.physicalRoll;
    }
}

void velocityEstimatorInit(void)
{
    uint8_t a;
    uint16_t histIdx;

    for (a = 0u; a < VELOCITY_ESTIMATOR_AXES; a++)
    {
        velocityEstimatorState.axis[a].acceleration = 0.0f;
        velocityEstimatorState.axis[a].imuOnlyVelocity = 0.0f;
        velocityEstimatorState.axis[a].fusedVelocity = 0.0f;
        velocityEstimatorState.axis[a].bias = 0.0f;
        velocityEstimatorKalmanResetCovarianceAxis(
            &velocityEstimatorState.axis[a].p00,
            &velocityEstimatorState.axis[a].p01,
            &velocityEstimatorState.axis[a].p11);
        velocityEstimatorState.axis[a].flowVelocity = 0.0f;
        velocityEstimatorState.axis[a].innovation = 0.0f;
        for (histIdx = 0u; histIdx < VELOCITY_ESTIMATOR_VEL_HIST_LEN; histIdx++)
        {
            velocityEstimatorState.axis[a].fusedVelHist[histIdx] = 0.0f;
        }
        velocityEstimatorBiasCalibrationSum[a] = 0.0f;
        velocityEstimatorImuOnlyBias[a] = 0.0f;
    }
    velocityEstimatorState.histWriteIdx = 0u;
    velocityEstimatorState.heightM = 0.0f;
    velocityEstimatorState.frameAgeMs = VELOCITY_ESTIMATOR_FLOW_TIMEOUT_MS;
    velocityEstimatorState.validAgeMs = VELOCITY_ESTIMATOR_FLOW_TIMEOUT_MS;
    velocityEstimatorState.consecutiveInvalidFrames = 0u;
    velocityEstimatorState.predictGeneration = 0u;
    velocityEstimatorState.lastRawFlowX = 0.0f;
    velocityEstimatorState.lastRawFlowY = 0.0f;
    velocityEstimatorState.flowValid = 0u;
    velocityEstimatorState.toffFresh = 0u;

    velocityEstimatorBiasCalibrationCount = 0u;
}

/* 1ms ISR 热路径：三轴去重力 + 四元数矩阵展开 + 双轴积分；
* 无三角函数 / 除法（除标定结束一次）/ malloc / 递归。M7 硬件 FPU 可承受。*/
void velocityEstimatorPredictTick(void)
{
    float accelSensorX;
    float accelSensorY;
    float accelSensorZ;
    float linearSensorX;
    float linearSensorY;
    float linearSensorZ;
    float acceleration[VELOCITY_ESTIMATOR_AXES];
    imuSnapshotStruct imuSnapshot;
    float q0;
    float q1;
    float q2;
    float q3;
    float gravitySensorX;

```

```

float gravitySensorY;
float gravitySensorZ;
uint8_t a;

velocityEstimatorState.predictGeneration++;
IMUGetSnapshot(&imuSnapshot);
q0 = imuSnapshot.q[0];
q1 = imuSnapshot.q[1];
q2 = imuSnapshot.q[2];
q3 = imuSnapshot.q[3];
accelSensorX = imuSnapshot.accelSensorX;
accelSensorY = imuSnapshot.accelSensorY;
accelSensorZ = imuSnapshot.accelSensorZ;

if (velocityEstimatorState.frameAgeMs < 0xFFFFu)
{
    velocityEstimatorState.frameAgeMs++;
}
if (velocityEstimatorState.validAgeMs < 0xFFFFu)
{
    velocityEstimatorState.validAgeMs++;
}
if (velocityEstimatorState.validAgeMs >= VELOCITY_ESTIMATOR_FLOW_TIMEOUT_MS)
{
    velocityEstimatorState.flowValid = 0u;
}

/* 机体系重力单位向量（与 Mahony halfv×2 同式），扣除得线加速度。*/
gravitySensorX = 2.0f * (q1 * q3 - q0 * q2);
gravitySensorY = 2.0f * (q0 * q1 + q2 * q3);
gravitySensorZ = 2.0f * (q0 * q0 - 0.5f + q3 * q3);

linearSensorX = accelSensorX - VELOCITY_ESTIMATOR_GRAVITY_MPS2 * gravitySensorX;
linearSensorY = accelSensorY - VELOCITY_ESTIMATOR_GRAVITY_MPS2 * gravitySensorY;
linearSensorZ = accelSensorZ - VELOCITY_ESTIMATOR_GRAVITY_MPS2 * gravitySensorZ;

velocityEstimatorRotateSensorToLocal(
    q0, q1, q2, q3,
    linearSensorX, linearSensorY, linearSensorZ,
    &acceleration[0], &acceleration[1]);

/* NaN/Inf 守卫：四元数/加速度异常时冻结本拍预测并清有效标志，防 NaN 永久污染积分。*/
if (!velocityEstimatorIsFinite(acceleration[0]) ||
    !velocityEstimatorIsFinite(acceleration[1]))
{
    velocityEstimatorState.flowValid = 0u;
    return;
}

for (a = 0u; a < VELOCITY_ESTIMATOR_AXES; a++)
{
    acceleration[a] = velocityEstimatorClamp(
        acceleration[a], VELOCITY_ESTIMATOR_ACCEL_LIMIT_MPS2);
    velocityEstimatorState.axis[a].acceleration = acceleration[a];
}

/* 上电静止 ~2s：平均去重力后的水平加速度作等效 bias 初值。
* 标定期检测到运动（任一轴水平线加速度超阈值）即清零累计重新开始，
* 避免把启动晃动 / Mahony 高增益收敛期误差写入 bias。双轴同时完成。*/
if (velocityEstimatorBiasCalibrationCount < VELOCITY_ESTIMATOR_BIAS_CALIBRATION_SAMPLES)
{
    if (acceleration[0] > VELOCITY_ESTIMATOR_CALIB_MOTION_MPS2 ||
        acceleration[0] < -VELOCITY_ESTIMATOR_CALIB_MOTION_MPS2 ||
        acceleration[1] > VELOCITY_ESTIMATOR_CALIB_MOTION_MPS2 ||
        acceleration[1] < -VELOCITY_ESTIMATOR_CALIB_MOTION_MPS2)
    {
        velocityEstimatorBiasCalibrationSum[0] = 0.0f;
        velocityEstimatorBiasCalibrationSum[1] = 0.0f;
        velocityEstimatorBiasCalibrationCount = 0u;
        return;
    }

    velocityEstimatorBiasCalibrationSum[0] += acceleration[0];
    velocityEstimatorBiasCalibrationSum[1] += acceleration[1];
    velocityEstimatorBiasCalibrationCount++;

    if (velocityEstimatorBiasCalibrationCount == VELOCITY_ESTIMATOR_BIAS_CALIBRATION_SAMPLES)
    {
        float invSamples = 1.0f / (float) VELOCITY_ESTIMATOR_BIAS_CALIBRATION_SAMPLES;

```

```

    for (a = 0u; a < VELOCITY_ESTIMATOR_AXES; a++)
    {
        float bias = velocityEstimatorClamp(
            velocityEstimatorBiasCalibrationSum[a] * invSamples,
            VELOCITY_ESTIMATOR_BIAS_LIMIT_MPS2);
        velocityEstimatorState.axis[a].bias = bias;
        velocityEstimatorImuOnlyBias[a] = bias;
        velocityEstimatorState.axis[a].imuOnlyVelocity = 0.0f;
        velocityEstimatorState.axis[a].fusedVelocity = 0.0f;
        velocityEstimatorKalmanResetCovarianceAxis(
            &velocityEstimatorState.axis[a].p00,
            &velocityEstimatorState.axis[a].p01,
            &velocityEstimatorState.axis[a].p11);
    }
}
return;
}

for (a = 0u; a < VELOCITY_ESTIMATOR_AXES; a++)
{
    /* 纯 IMU 积分参考（固定 bias，不被光流校正，用于影子对比漂移）。*/
    velocityEstimatorState.axis[a].imuOnlyVelocity = velocityEstimatorClamp(
        velocityEstimatorState.axis[a].imuOnlyVelocity
        + (acceleration[a] - velocityEstimatorImuOnlyBias[a])
        * VELOCITY_ESTIMATOR_PREDICT_DT_S,
        VELOCITY_ESTIMATOR_VELOCITY_LIMIT_MPS);

    /* KF 状态预测：v += (a-b)*dt，b 为随机游走状态，本拍均值保持不变。*/
    velocityEstimatorState.axis[a].fusedVelocity = velocityEstimatorClamp(
        velocityEstimatorState.axis[a].fusedVelocity
        + (acceleration[a] - velocityEstimatorState.axis[a].bias)
        * VELOCITY_ESTIMATOR_PREDICT_DT_S,
        VELOCITY_ESTIMATOR_VELOCITY_LIMIT_MPS);

    /* 协方差预测：每轴约 10 次乘法，无除法/矩阵库；250MHz M7 FPU 1ms 预算充足。*/
    velocityEstimatorKalmanPredictCovarianceAxis(
        &velocityEstimatorState.axis[a].p00,
        &velocityEstimatorState.axis[a].p01,
        &velocityEstimatorState.axis[a].p11);

    /* 写融合速度环形历史（供光流帧延迟补偿取延迟时刻估计）。本拍在 ISR 内，
    * 单写者，前台 Update 只读，无需额外保护。*/
    velocityEstimatorState.axis[a].fusedVelHist[velocityEstimatorState.histWriteIdx] =
        velocityEstimatorState.axis[a].fusedVelocity;
}

/* 两轴写入同一槽后统一前进（X/Y 共享单索引）。*/
velocityEstimatorState.histWriteIdx =
    (uint16_t)((velocityEstimatorState.histWriteIdx + 1u) % VELOCITY_ESTIMATOR_VEL_HIST_LEN);
}

void velocityEstimatorUpdate(uint8_t allowFrozenToFHeight)
{
    opticalFlowSampleStruct sample;
    float heightMm = 0.0f;
    float verticalVelocityMmS = 0.0f;
    float heightM;
    uint8_t tofFresh = tofGetHeight(&heightMm, &verticalVelocityMmS);
    uint8_t tofHealthy = altitudeToflsHealthy();
    uint32_t primask;

    (void) verticalVelocityMmS; /* 垂直速度本阶段不入水平融合 */
    heightM = heightMm * 0.001f;

    primask = __get_PRIMASK();
    __disable_irq();
    velocityEstimatorState.heightM = heightM;
    velocityEstimatorState.tofFresh = tofFresh;
    __set_PRIMASK(primask);

    if (!opticalFlowPoll(&sample))
    {
        return;
    }
}
/* 收到任意完整帧 → 清链路帧年龄（含无效帧，表示链路活）。
* 有效观测年龄 validAgeMs 仅在通过全部门控 + 校正后才清零（见下）。
* 连续被拒计数：每收一帧先 +1（饱和和回卷），成功校正时在提交块归零。*/
primask = __get_PRIMASK();
__disable_irq();

```

```

velocityEstimatorState.frameAgeMs = 0u;
velocityEstimatorState.lastRawFlowX = (float) sample.flowX;
velocityEstimatorState.lastRawFlowY = (float) sample.flowY;
if (velocityEstimatorState.consecutiveInvalidFrames < 0xFFFFu)
{
    velocityEstimatorState.consecutiveInvalidFrames++;
}
__set_PRIMASK(primask);

/* 帧无效 / dt 越界 / 无可用高度 / 高度越界 → 本帧不校正、不刷新有效年龄。
 * 已进入 CRUISE 后的 ToF HOLD 允许使用 facade 冻结的最后可信高度维持短窗光流尺度；
 * RAMP 及其它调用场景仍要求实时健康，避免长期陈旧高度掩盖安全失效。 */
if (!sample.valid ||
    sample.dtUs < VELOCITY_ESTIMATOR_FLOW_DT_MIN_US ||
    sample.dtUs > VELOCITY_ESTIMATOR_FLOW_DT_MAX_US ||
    (!tofHealthy && allowFrozenToFHeight == 0u) ||
    heightM < VELOCITY_ESTIMATOR_HEIGHT_MIN_M ||
    heightM > VELOCITY_ESTIMATOR_HEIGHT_MAX_M)
{
    return;
}

{
    float q[4];
    float gyroX;
    float gyroY;
    float gyroZ;
    float physicalPitch;
    float physicalRoll;
    float invDtS;
    float flowAngularRateX;
    float flowAngularRateY;
    float flowBodyX;
    float flowBodyY;
    float flowSensorX;
    float flowSensorY;
    float flowLocal[VELOCITY_ESTIMATOR_AXES];
    float measurementVariance;
    float snapVel[VELOCITY_ESTIMATOR_AXES];
    float snapBias[VELOCITY_ESTIMATOR_AXES];
    float snapP00[VELOCITY_ESTIMATOR_AXES];
    float snapP01[VELOCITY_ESTIMATOR_AXES];
    float snapP11[VELOCITY_ESTIMATOR_AXES];
    float delayedVel[VELOCITY_ESTIMATOR_AXES];
    float innovation[VELOCITY_ESTIMATOR_AXES];
    velocityEstimatorKalmanResultStruct result[VELOCITY_ESTIMATOR_AXES];
    uint16_t delaySlot;
    uint32_t snapPredictGeneration;
    uint8_t updateAccepted;
    uint8_t a;

    velocityEstimatorSnapshotImu(
        q, &gyroX, &gyroY, &gyroZ, &physicalPitch, &physicalRoll);

    /* 大倾角时 v=h×flowRate 近似失效；门限放宽至约 65° 以容忍机动平移大倾角（大倾角帧测速误差增大）。 */
    if (fabsf(physicalPitch) > VELOCITY_ESTIMATOR_TILT_LIMIT_RAD ||
        fabsf(physicalRoll) > VELOCITY_ESTIMATOR_TILT_LIMIT_RAD)
    {
        return;
    }

    invDtS = 1 000 000.0f / (float) sample.dtUs; /* 1/积分时长(秒) */
    flowAngularRateX = VELOCITY_ESTIMATOR_FLOW_SCALE_X_RAD_PER_COUNT
        * (float) sample.flowX * invDtS;
    flowAngularRateY = VELOCITY_ESTIMATOR_FLOW_SCALE_Y_RAD_PER_COUNT
        * (float) sample.flowY * invDtS;

    /* 倾摆去旋转：flowX 配 pitch(gyroX)、flowY 配 roll(gyroY)，板端“减负”=加回。 */
    flowAngularRateX -= VELOCITY_ESTIMATOR_GYRO_COMP_X * gyroX;
    flowAngularRateY -= VELOCITY_ESTIMATOR_GYRO_COMP_Y * gyroY;
    flowBodyX = heightM * flowAngularRateX;
    flowBodyY = heightM * flowAngularRateY;

    /* 杠杆臂切向解耦：光流偏离旋转中心 r=(LEVER_X,LEVER_Y)，yaw 转动时光流测到
     * 真实切向速度需扣除以还原质心速度。注意 gyroZ=imuData.gz 为 CCW 正，而 FRD
     * 右手系(Z 向下)的  $\omega_z = -\text{gyroZ}$ ，故  $v_{\text{tan}} = (\text{gyroZ} \cdot dy, -\text{gyroZ} \cdot dx)$ ，
     * 质心  $v = v_{\text{光流}} - v_{\text{tan}} \rightarrow \text{forward} -= \text{gyroZ} \cdot dy$ ； $\text{right} += \text{gyroZ} \cdot dx$ 。确定性补偿，非标定。 */
    flowBodyX -= (gyroZ * VELOCITY_ESTIMATOR_FLOW_LEVER_Y_M);
    flowBodyY += (gyroZ * VELOCITY_ESTIMATOR_FLOW_LEVER_X_M);

```

```

/* 已标定 B(FRD)→S 安装映射: sensorX=-bodyRight, sensorY=-bodyForward。 */
flowSensorX = VELOCITY_ESTIMATOR_BODY_TO_SENSOR_XX * flowBodyX
    + VELOCITY_ESTIMATOR_BODY_TO_SENSOR_XY * flowBodyY;
flowSensorY = VELOCITY_ESTIMATOR_BODY_TO_SENSOR_YX * flowBodyX
    + VELOCITY_ESTIMATOR_BODY_TO_SENSOR_YY * flowBodyY;
velocityEstimatorRotateSensorToLocal(
    q[0], q[1], q[2], q[3],
    flowSensorX, flowSensorY, 0.0f,
    &flowLocal[0], &flowLocal[1]);

/* 物理速度有限性守卫 (高度/dt 异常组合可能产生 Inf/NaN) 。 */
if (!velocityEstimatorIsFinite(flowLocal[0]) ||
    !velocityEstimatorIsFinite(flowLocal[1]))
{
    return;
}

/* 量测方差 R 随高度平方增长: 同样角流噪声在更高处对应更大线速度噪声。
* 首版不按 innovation 调 R (避免真实加速时残差变大而错误压低光流权重) 。 */
measurementVariance = VELOCITY_ESTIMATOR_KF_R_BASE
    + VELOCITY_ESTIMATOR_KF_R_HEIGHT_COEFF * heightM * heightM;
measurementVariance = clampf(
    measurementVariance, VELOCITY_ESTIMATOR_KF_P_MIN, VELOCITY_ESTIMATOR_KF_P_MAX);

/* 延迟补偿 + 缩短临界区:
* P2 修复——关中断仅保护“快照”和“提交”两个短瞬间, 中间 KF 计算 (纯函数、
* 基于局部快照) 放临界区外, 避免长时间关中断扰动 1ms 飞控环。
* 延迟补偿——光流量测代表约 FLOW_DELAY_MS 前的状态, 故 innovation 用“延迟时刻”
* 的融合速度 (环形历史) 而非当前值, 使量测与估计同时刻, 消除时间错位。 */

/* 临界区 1 (短): 快照当前 v/bias/P + 取延迟时刻融合速度。 */
primask = __get_PRIMASK();
__disable_irq();
snapPredictGeneration = velocityEstimatorState.predictGeneration;
/* histWriteldx 指向下一个待写槽, 最近写入的是 idx-1; 延迟 D 帧即 idx-1-D。 */
delaySlot = (uint16_t)((velocityEstimatorState.histWriteldx
    + VELOCITY_ESTIMATOR_VEL_HIST_LEN
    - 1u - VELOCITY_ESTIMATOR_FLOW_DELAY_MS) % VELOCITY_ESTIMATOR_VEL_HIST_LEN);
for (a = 0u; a < VELOCITY_ESTIMATOR_AXES; a++)
{
    snapVel[a] = velocityEstimatorState.axis[a].fusedVelocity;
    snapBias[a] = velocityEstimatorState.axis[a].bias;
    snapP00[a] = velocityEstimatorState.axis[a].p00;
    snapP01[a] = velocityEstimatorState.axis[a].p01;
    snapP11[a] = velocityEstimatorState.axis[a].p11;
    delayedVel[a] = velocityEstimatorState.axis[a].fusedVelHist[delaySlot];
}
__set_PRIMASK(primask);

/* 临界区外: innovation 对齐延迟时刻估计 (延迟补偿核心), KF 更新纯函数计算。
* 两轴同拍: 任一轴坏值 (innovation 越限或更新失败) 则整帧不提交。 */
updateAccepted = 1u;
for (a = 0u; a < VELOCITY_ESTIMATOR_AXES; a++)
{
    innovation[a] = flowLocal[a] - delayedVel[a];
    if (velocityEstimatorBiasCalibrationCount < VELOCITY_ESTIMATOR_BIAS_CALIBRATION_SAMPLES ||
        fabsf(innovation[a]) > VELOCITY_ESTIMATOR_INNOVATION_LIMIT_MPS ||
        !velocityEstimatorKalmanUpdateAxis(
            snapVel[a], snapBias[a], snapP00[a], snapP01[a], snapP11[a],
            innovation[a], measurementVariance, &result[a]))
    {
        updateAccepted = 0u;
    }
}

/* 临界区 2 (短): 提交结果 + 调试量。两轴同拍, 单轴坏值则整帧不更新。 */
primask = __get_PRIMASK();
__disable_irq();
for (a = 0u; a < VELOCITY_ESTIMATOR_AXES; a++)
{
    velocityEstimatorState.axis[a].flowVelocity = flowLocal[a];
    velocityEstimatorState.axis[a].innovation = innovation[a];
}
if (updateAccepted &&
    velocityEstimatorState.predictGeneration == snapPredictGeneration)
{
    for (a = 0u; a < VELOCITY_ESTIMATOR_AXES; a++)
    {

```

```

        velocityEstimatorState.axis[a].fusedVelocity = result[a].velocity;
        velocityEstimatorState.axis[a].bias = result[a].bias;
        velocityEstimatorState.axis[a].p00 = result[a].p00;
        velocityEstimatorState.axis[a].p01 = result[a].p01;
        velocityEstimatorState.axis[a].p11 = result[a].p11;
    }
    velocityEstimatorState.flowValid = 1u;
    velocityEstimatorState.validAgeMs = 0u; /* 有效观测年龄仅在更新成功时清零 */
    velocityEstimatorState.consecutiveInvalidFrames = 0u; /* 成功校正 → 连续被拒计数归零 */
}
__set_PRIMASK(primask);
}
}

void velocityEstimatorGetSnapshot(velocityEstimatorSnapshotStruct *out)
{
    uint32_t primask = __get_PRIMASK();

    if (out == 0)
    {
        return;
    }

    __disable_irq();
    out->velocityFwd = velocityEstimatorState.axis[0].fusedVelocity;
    out->velocityRight = velocityEstimatorState.axis[1].fusedVelocity;
    out->validAgeMs = velocityEstimatorState.validAgeMs;
    out->valid =
        (velocityEstimatorBiasCalibrationCount >= VELOCITY_ESTIMATOR_BIAS_CALIBRATION_SAMPLES &&
         velocityEstimatorState.flowValid) ? 1u : 0u;
    out->padding = 0u;
    out->predictGeneration = velocityEstimatorState.predictGeneration;
    __set_PRIMASK(primask);
}

uint16_t velocityEstimatorGetConsecutiveInvalidFrames(void)
{
    uint16_t count;
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    count = velocityEstimatorState.consecutiveInvalidFrames;
    __set_PRIMASK(primask);
    return count;
}

void velocityEstimatorGetRawFlow(float *flowX, float *flowY)
{
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    if (flowX != 0)
    {
        *flowX = velocityEstimatorState.lastRawFlowX;
    }
    if (flowY != 0)
    {
        *flowY = velocityEstimatorState.lastRawFlowY;
    }
    __set_PRIMASK(primask);
}
}

```

core0_cyt4bb7/project/code/sensor/velocityEstimator.h

SHA-256

:

c98b905e37dad9105eeba859dc12ec1f87efbbfe4e86815b9400cce6b90d89bb

代码 (c)

```

#ifndef VELOCITY_ESTIMATOR_H_
#define VELOCITY_ESTIMATOR_H_

```

```

#include <stdint.h>

```

```

/* IMU + 光流水平速度估计器（双轴独立 2 状态线性 KF）。

```

```

* - 1ms IMU 预测：去重力 + 倾斜补偿(去 yaw, 保留 pitch/roll)转机体水平航向系 + 双轴状态/协方差预测
* - 光流帧异步校正：物理速度换算 + 标量量测更新 (Joseph) + bias 状态估计

```

```
* -输出位于**机体水平航向系**（前向/右向，跟随机头 yaw），单位 m/s
* 注意：非世界固定系——去 yaw 避免 xy 速度与航向耦合，正是光流定位所需的对地机体速度。
* 输出经一次 snapshot 接入水平速度内环闭环；valid=0 时速度环冻结降级。*/

typedef struct
{
    float velocityFwd;
    float velocityRight;
    uint16_t validAgeMs;
    uint8_t valid;
    uint8_t padding;
    uint32_t predictGeneration;
} velocityEstimatorSnapshotStruct;

void velocityEstimatorInit(void);

/* 1ms PIT 调用（紧跟 IMUUpdate()）：仅四元数展开、去重力和双轴预测，
* 无阻塞 / 动态内存 / UART / 三角函数。*/
void velocityEstimatorPredictTick(void);

/* 前台调用：消费光流新帧 + ToF 高度，物理速度换算 + 双轴 KF 量测更新。
* allowFrozenToFHeight 仅用于已进入 CRUISE 后的 ToF HOLD：以最后可信高度维持短窗光流尺度；
* RAMP/正式缓降必须传 0，不能用冻结高度掩盖长期失效。*/
void velocityEstimatorUpdate(uint8_t allowFrozenToFHeight);

/* 一次短临界区返回同拍融合速度、有效性与预测代数。*/
void velocityEstimatorGetSnapshot(velocityEstimatorSnapshotStruct *out);

/* 距上次成功 KF 校正以来连续被拒的光流帧数（每收一帧+1 饱和于 0xFFFF，成功校正归零）。
* 调试观测光流/KF 健康度：持续增大说明光流帧一直进不了融合（无效/门控拒/倾角越界等）。*/
uint16_t velocityEstimatorGetConsecutiveInvalidFrames(void);

/* 读取最近一帧原始光流计数（flowX 前正 / flowY 右正，未经 K_flow 换算）；允许任一指针为空。
* 供调试通道观测光流是否在动；光流帧被 velocityEstimator 前台独占消费，外部不可再 poll。*/
void velocityEstimatorGetRawFlow(float *flowX, float *flowY);

#endif /* _VELOCITY_ESTIMATOR_H_ */
```

core0_cyt4bb7/project/code/util/crc32leee.c

SHA-256

:

363e996887ef0e2c68b3518217986f209f0760bcd89ff2ca5cdd3a4801583aba

代码 (c)

```
#include "crc32leee.h"

uint32_t crc32leeeCalc(const uint8_t* data, uint32_t len)
{
    uint32_t crc = 0xFFFFFFFFu;
    uint32_t i;

    for (i = 0u; i < len; i++)
    {
        uint8_t b;
        crc ^= (uint32_t) data[i];
        for (b = 0u; b < 8u; b++)
        {
            uint32_t mask = (uint32_t) (~(int32_t) (crc & 1u));
            crc = (crc >> 1) ^ (0xEDB88320u & mask);
        }
    }
    return ~crc;
}
```

core0_cyt4bb7/project/code/util/crc32leee.h

SHA-256

:

5442cded78a9802395246ceee07532bb7e5de239f12b419b4ecc309d2c9e1001

代码 (c)

```
#ifndef CRC32_IEEE_H
#define CRC32_IEEE_H

#include <stdint.h>

uint32_t crc32IeeeCalc(const uint8_t* data, uint32_t len);

#endif
```

core0_cyt4bb7/project/code/util/factFreshness.h

SHA-256

:

21e85a6d697ae0d1b61f0b07a61d890e9c908eec4007a75ed192b371fa666c3d

代码 (c)

```
#ifndef FACT_FRESHNESS_H_
#define FACT_FRESHNESS_H_

#include <stdint.h>

/* 调用方以相邻采样点的 uint32 模减传入 elapsedMs; 饱和后仅显式新事实可清零。 */
static inline uint16_t factFreshnessAdvanceAgeU16(uint16_t ageMs, uint32_t elapsedMs)
{
    if (elapsedMs >= (uint32_t) (0xFFFFu - ageMs)) return 0xFFFFu;
    return (uint16_t) (ageMs + elapsedMs);
}

/* 首次一致快照只建立当前会话基线，避免__no_init 历史 READY 被当作新事实。 */
static inline uint8_t factFreshnessAcceptSeq(uint8_t *primed,
    uint32_t *baselineSeq, uint32_t candidateSeq)
{
    if (*primed == 0u)
    {
        *primed = 1u;
        *baselineSeq = candidateSeq;
        return 0u;
    }
    if (candidateSeq == *baselineSeq) return 0u;
    *baselineSeq = candidateSeq;
    return 1u;
}

#endif
```

core0_cyt4bb7/project/code/util/lowPassFilter.c

SHA-256

:

36e5dbbded0830f3b91308821132c824fa42a1411f3817fc1e7c4ce7a4a2ba85

代码 (c)

```
/*
 * lowPassFilter.c
 *
 * Created: 20 250 319
 * Author: Hecate
 */
#include "lowPassFilter.h"

void lowPassFilterInit(LowPassFilterData *obj, float alpha)
{
    obj->alpha = alpha;

    obj->filteredValue = 0;
    obj->firstFlag = 1;
    obj->lastOutput = 0;
}
```



```

void lowPassFilterReset(LowPassFilterData *obj, float value)
{
    obj->firstFlag = 0u;
    obj->lastOutput = value;
    obj->filteredValue = value;
}

void lowPassFilter(LowPassFilterData *obj, const float value)
{
    if ((obj)->firstFlag == 1u)
    {
        (obj)->firstFlag = 0u;
        (obj)->lastOutput = value;
    }
    (obj)->filteredValue = (obj)->alpha * value + (1.0f - (obj)->alpha) * (obj)->lastOutput;
    (obj)->lastOutput = (obj)->filteredValue;
}

```

core0_cyt4bb7/project/code/util/lowPassFilter.h

SHA-256

:

e5deedc753466e971994dc6d6a19c97a2853d175c5a17578994096c17b64ec50

代码 (c)

```

/*
 * lowPassFilter.h
 *
 * Created: 20 250 319
 * Author: Hecate
 */
#ifndef CODE_LOWPASSFILTER_H_
#define CODE_LOWPASSFILTER_H_

#include <stdint.h>

// 一阶低通滤波器句柄
typedef struct
{
    float alpha;
    float lastOutput;
    float filteredValue;
    uint8_t firstFlag;
} LowPassFilterData;

// 初始化滤波器参数
void lowPassFilterInit(LowPassFilterData *obj, float alpha);

// 将滤波器状态强制锚定到指定值 (firstFlag 清零, 下一拍从此值连续滤波)
// 用于"非锁定态直接跟随原始值"场景, 保证后续进入滤波时无突跳
void lowPassFilterReset(LowPassFilterData *obj, float value);

// 输入新样本并更新滤波值
void lowPassFilter(LowPassFilterData *obj, const float value);

#endif /* CODE_LOWPASSFILTER_H_ */

```

core0_cyt4bb7/project/code/util/paramSlotStore.c

SHA-256

:

2c3154f0f25f12f17c08688758426562a2ef3d4e37a2730bb144652604a4f3b5

代码 (c)

```

#include "paramSlotStore.h"

#include <stddef.h>
#include <string.h>

```

```

#include "crc32leec.h"
#include "zf_driver_flash.h"

#define PARAM_SLOT_SELECTOR_MAGIC      (0x31534C50u) /* PLS1 */
#define PARAM_SLOT_SELECTOR_VERSION    (1u)
#define PARAM_SLOT_SELECTOR_BYTES      (96u)
#define PARAM_SLOT_SELECTOR_PAGE_A     (10u)
#define PARAM_SLOT_SELECTOR_PAGE_B     (11u)
#define PARAM_SLOT_SELECTOR_RECORD_WORDS (PARAM_SLOT_SELECTOR_BYTES / 4u)
#define PARAM_SLOT_SELECTOR_RECORDS_PER_PAGE \
    (FLASH_PAGE_SIZE / PARAM_SLOT_SELECTOR_BYTES)
#define PARAM_SLOT_DATA_PAGE_FIRST     (0u)
#define PARAM_SLOT_DATA_PAGE_LAST      (PARAM_SLOT_COUNT * 2u - 1u)
#define PARAM_SLOT_COORDINATOR_PAGE_A  (12u)
#define PARAM_SLOT_COORDINATOR_PAGE_B  (13u)
#define PARAM_SLOT_PENDING_MAGIC       (0x50520000u)
#define PARAM_SLOT_PENDING_MAGIC_MASK  (0xFFFF0000u)
#define PARAM_SLOT_PENDING_ACTIVATION  (1u << 10)
#define PARAM_SLOT_PENDING_FIELDS_MASK (0x000007FFu)
#define PARAM_SLOT_MASK                 ((1u << PARAM_SLOT_COUNT) - 1u)

#define PARAM_RUNTIME_SELECTOR_MAGIC    (0x31544E52u) /* RNT1 */
#define PARAM_RUNTIME_SELECTOR_VERSION  (1u)
#define PARAM_RUNTIME_SELECTOR_BYTES    (64u)
#define PARAM_RUNTIME_DATA_PAGE_A       (14u)
#define PARAM_RUNTIME_DATA_PAGE_B       (15u)
#define PARAM_RUNTIME_SELECTOR_PAGE_A   (16u)
#define PARAM_RUNTIME_SELECTOR_PAGE_B   (17u)
#define PARAM_RUNTIME_SELECTOR_RECORD_WORDS (PARAM_RUNTIME_SELECTOR_BYTES / 4u)
#define PARAM_RUNTIME_SELECTOR_RECORDS_PER_PAGE \
    (FLASH_PAGE_SIZE / PARAM_RUNTIME_SELECTOR_BYTES)
#define PARAM_RUNTIME_TX_ACTIVATION_ONLY (UINT32_MAX)

#ifndef PARAM_SLOT_FLASH_ADDRESS_TYPE
#define PARAM_SLOT_FLASH_ADDRESS_TYPE uint32_t
#endif

typedef struct
{
    uint32_t magic;
    uint32_t version;
    uint32_t recordBytes;
    uint32_t sequence;
    uint8_t activeSlot;
    uint8_t validMask;
    uint8_t bankMask;
    uint8_t reserved0;
    uint32_t generation[PARAM_SLOT_COUNT];
    uint32_t blobCrc[PARAM_SLOT_COUNT];
    uint32_t transactionId[PARAM_SLOT_COUNT];
    uint32_t reserved[3u];
    uint32_t crc;
} paramSlotSelector;

typedef struct
{
    uint32_t magic;
    uint32_t version;
    uint32_t recordBytes;
    uint32_t sequence;
    uint8_t bank;
    uint8_t valid;
    uint8_t pending;
    uint8_t sourceSlot;
    uint32_t generation;
    uint32_t blobCrc;
    uint32_t transactionId;
    uint8_t rollbackBank;
    uint8_t rollbackValid;
    uint8_t rollbackSourceSlot;
    uint8_t reserved0;
    uint32_t rollbackGeneration;
    uint32_t rollbackBlobCrc;
    uint32_t rollbackTransactionId;
    uint32_t reserved[3u];
    uint32_t crc;
} paramRuntimeSelector;

typedef char paramSlotSelectorSizeAssert[

```

```

    (sizeof(paramSlotSelector) == PARAM_SLOT_SELECTOR_BYTES) ? 1 : -1];
typedef char paramRuntimeSelectorSizeAssert[
    (sizeof(paramRuntimeSelector) == PARAM_RUNTIME_SELECTOR_BYTES) ? 1 : -1];
typedef char paramSlotPagesExistAssert[(FLASH_PAGE_NUM >= 18u) ? 1 : -1];
typedef char paramSlotDataBeforeSelectorAssert[
    (PARAM_SLOT_DATA_PAGE_FIRST <= PARAM_SLOT_DATA_PAGE_LAST &&
     PARAM_SLOT_DATA_PAGE_LAST < PARAM_SLOT_SELECTOR_PAGE_A) ? 1 : -1];
typedef char paramSlotSelectorBeforeCoordinatorAssert[
    (PARAM_SLOT_SELECTOR_PAGE_A + 1u == PARAM_SLOT_SELECTOR_PAGE_B &&
     PARAM_SLOT_SELECTOR_PAGE_B < PARAM_SLOT_COORDINATOR_PAGE_A) ? 1 : -1];
typedef char paramSlotCoordinatorBeforeRuntimeAssert[
    (PARAM_SLOT_COORDINATOR_PAGE_A + 1u == PARAM_SLOT_COORDINATOR_PAGE_B &&
     PARAM_SLOT_COORDINATOR_PAGE_B < PARAM_RUNTIME_DATA_PAGE_A) ? 1 : -1];
typedef char paramSlotRuntimeLayoutAssert[
    (PARAM_RUNTIME_DATA_PAGE_A + 1u == PARAM_RUNTIME_DATA_PAGE_B &&
     PARAM_RUNTIME_DATA_PAGE_B < PARAM_RUNTIME_SELECTOR_PAGE_A &&
     PARAM_RUNTIME_SELECTOR_PAGE_A + 1u == PARAM_RUNTIME_SELECTOR_PAGE_B &&
     PARAM_RUNTIME_SELECTOR_PAGE_B < FLASH_PAGE_NUM) ? 1 : -1];
typedef char paramSlotSelectorRecordsFitAssert[
    ((PARAM_SLOT_SELECTOR_BYTES <= FLASH_PAGE_SIZE) &&
     (PARAM_SLOT_SELECTOR_RECORDS_PER_PAGE > 0u) &&
     (PARAM_RUNTIME_SELECTOR_BYTES <= FLASH_PAGE_SIZE) &&
     (PARAM_RUNTIME_SELECTOR_RECORDS_PER_PAGE > 0u)) ? 1 : -1];

static paramSlotSelector *selectorMutable(paramSlotStore *store)
{
    return (paramSlotSelector *) (void *) store->selectorBytes;
}

static const paramSlotSelector *selectorConst(const paramSlotStore *store)
{
    return (const paramSlotSelector *) (const void *) store->selectorBytes;
}

static const paramRuntimeSelector *runtimeSelectorConst(
    const paramSlotStore *store)
{
    return (const paramRuntimeSelector *) (const void *)
        store->runtimeSelectorBytes;
}

static uint8_t slotValid(uint8_t slot)
{
    return (uint8_t) ((slot >= PARAM_SLOT_FIRST) && (slot <= PARAM_SLOT_LAST));
}

static uint32_t dataPage(uint8_t slot, uint8_t bank)
{
    return (uint32_t) ((slot - 1u) * 2u + (bank & 1u));
}

static uint8_t sequenceNewer(uint32_t candidate, uint32_t reference)
{
    return (uint8_t) ((int32_t) (candidate - reference) > 0);
}

static uint8_t selectorValid(const paramSlotSelector *selector)
{
    uint32_t crc;
    uint32_t packed;
    uint8_t slot;
    uint8_t oldActive;
    uint8_t oldValid;
    uint8_t oldBank;
    uint8_t i;

    if ((selector->magic != PARAM_SLOT_SELECTOR_MAGIC) ||
        (selector->version != PARAM_SLOT_SELECTOR_VERSION) ||
        (selector->recordBytes != PARAM_SLOT_SELECTOR_BYTES) ||
        (selector->sequence == 0u) ||
        (selector->activeSlot > PARAM_SLOT_LAST) ||
        (selector->reserved0 != 0u) ||
        ((selector->validMask & (uint8_t) ~PARAM_SLOT_MASK) != 0u) ||
        ((selector->bankMask & (uint8_t) ~PARAM_SLOT_MASK) != 0u) ||
        ((selector->activeSlot != 0u) &&
         ((selector->validMask &
          (uint8_t) (1u << (selector->activeSlot - 1u))) == 0u)))
    {
        return 0u;
    }

```

```

    }
    for (i = 0u; i < PARAM_SLOT_COUNT; i++)
    {
        uint8_t valid = (uint8_t) ((selector->validMask >> i) & 1u);
        if (valid != 0u)
        {
            if (selector->generation[i] == 0u ||
                selector->transactionId[i] == PARAM_RUNTIME_TX_ACTIVATION_ONLY)
                return 0u;
        }
        else if (((selector->bankMask >> i) & 1u) != 0u ||
                 selector->generation[i] != 0u ||
                 selector->blobCrc[i] != 0u ||
                 selector->transactionId[i] != 0u)
        {
            return 0u;
        }
    }
    packed = selector->reserved[2u];
    if ((packed & PARAM_SLOT_PENDING_MAGIC_MASK) != PARAM_SLOT_PENDING_MAGIC)
    {
        if (selector->reserved[0u] != 0u || selector->reserved[1u] != 0u ||
            packed != 0u) return 0u;
    }
    else
    {
        if (((packed & ~(PARAM_SLOT_PENDING_MAGIC_MASK |
                        PARAM_SLOT_PENDING_FIELDS_MASK)) != 0u)
            return 0u;
        slot = (uint8_t) (packed & 0x0Fu);
        oldActive = (uint8_t) ((packed >> 4) & 0x0Fu);
        oldValid = (uint8_t) ((packed >> 8) & 1u);
        oldBank = (uint8_t) ((packed >> 9) & 1u);
        if (slotValid(slot) == 0u || slotValid(oldActive) == 0u ||
            (selector->activeSlot != slot &&
             selector->activeSlot != oldActive) ||
            ((selector->validMask >> (slot - 1u)) & 1u) == 0u ||
            ((oldActive != slot) &&
             (((selector->validMask >> (oldActive - 1u)) & 1u) == 0u)))
            return 0u;
        if ((packed & PARAM_SLOT_PENDING_ACTIVATION) != 0u)
        {
            uint8_t index = (uint8_t) (slot - 1u);
            /* The former activation-only writer switched activeSlot while
             * retaining an exact snapshot of the target slot in reserved[].
             * Accept that durable shape so an upgrade can materialize the
             * rollback runtime; a zeroed pseudo-snapshot was never emitted by
             * production firmware and must not make a corrupt record valid. */
            if (selector->activeSlot != slot || oldValid == 0u ||
                oldBank != ((selector->bankMask >> index) & 1u) ||
                selector->reserved[0u] != selector->transactionId[index] ||
                selector->reserved[1u] != selector->generation[index])
                return 0u;
        }
        else
        {
            if (selector->transactionId[slot - 1u] == 0u ||
                selector->transactionId[slot - 1u] ==
                    PARAM_RUNTIME_TX_ACTIVATION_ONLY)
                return 0u;
            if (oldValid != 0u)
            {
                if (selector->reserved[1u] == 0u) return 0u;
            }
            else if (oldBank != 0u || selector->reserved[0u] != 0u ||
                     selector->reserved[1u] != 0u)
            {
                return 0u;
            }
        }
    }
    crc = crc32IeeeCalc((const uint8_t *) selector,
                        (uint32_t) offsetof(paramSlotSelector, crc));
    return (uint8_t) (crc == selector->crc);
}

static const uint32_t *flashWord(uint32_t page, uint32_t word)
{
    uintptr_t address = (uintptr_t) FLASH_BASE_ADDR +

```

```

        (uintptr_t) page * FLASH_PAGE_SIZE +
        (uintptr_t) word * sizeof(uint32_t);
    return (const uint32_t *) address;
}

static void cleanFlashSource(const void *source, uint32_t bytes)
{
    #if CY_CORE_CM7_0 || CY_CORE_CM7_1
        uintptr_t start = ((uintptr_t) source) & ~(uintptr_t) 31u;
        uintptr_t end = ((uintptr_t) source + bytes + 31u) & ~(uintptr_t) 31u;
        SCB_CleanDCache_by_Addr((volatile void *) start,
            (int32_t) (end - start));
    #else
        (void) source;
        (void) bytes;
    #endif
}

typedef enum
{
    PARAM_SLOT_FLASH_BLANK = 0,
    PARAM_SLOT_FLASH_PROGRAMMED,
    PARAM_SLOT_FLASH_IO_FAILED
} paramSlotFlashState;

static paramSlotFlashState blankCheckWords(uint32_t page,
    uint32_t firstWord,
    uint32_t wordCount)
{
    {
        cy_stc_flash_blankcheck_config_t config = {0};
        cy_en_flashdrv_status_t status;

        if ((page >= FLASH_PAGE_NUM) || (wordCount == 0u) ||
            (firstWord >= FLASH_PAGE_LENGTH) ||
            (wordCount > (FLASH_PAGE_LENGTH - firstWord)))
        {
            return PARAM_SLOT_FLASH_IO_FAILED;
        }
        config.addrToBeChecked = flashWord(page, firstWord);
        config.numOfWordsToBeChecked = wordCount;
        status = Cy_Flash_BlankCheck(NULL, &config, CY_FLASH_DRIVER_BLOCKING);
        if (status == CY_FLASH_DRV_SUCCESS) return PARAM_SLOT_FLASH_BLANK;
        if (status == CY_FLASH_DRV_FLASH_NOT_ERASED) return PARAM_SLOT_FLASH_PROGRAMMED;
        return PARAM_SLOT_FLASH_IO_FAILED;
    }
}

static uint8_t readFlashBytes(uint32_t page, uint32_t byteOffset,
    void *data, uint32_t bytes)
{
    {
        uint32_t firstWord;
        uint32_t wordCount;
        uint32_t i;
        uint8_t *output = (uint8_t *) data;

        if ((data == NULL) || (bytes == 0u) || ((byteOffset & 3u) != 0u) ||
            (byteOffset + bytes > FLASH_PAGE_SIZE)) return 0u;
        firstWord = byteOffset / sizeof(uint32_t);
        wordCount = (bytes + 3u) / sizeof(uint32_t);
        if ((page >= FLASH_PAGE_NUM) || (firstWord >= FLASH_PAGE_LENGTH) ||
            (wordCount > (FLASH_PAGE_LENGTH - firstWord))) return 0u;
        for (i = 0u; i < wordCount; i++)
        {
            const uint32_t *address;
            uint32_t value;
            uint32_t copyBytes;
            paramSlotFlashState state = blankCheckWords(page, firstWord + i, 1u);
            if (state == PARAM_SLOT_FLASH_BLANK)
            {
                value = UINT32_MAX;
                copyBytes = bytes - i * sizeof(uint32_t);
                if (copyBytes > sizeof(uint32_t)) copyBytes = sizeof(uint32_t);
                memcpy(&output[i * sizeof(uint32_t)], &value, copyBytes);
                continue;
            }
            if (state != PARAM_SLOT_FLASH_PROGRAMMED) return 0u;
            address = flashWord(page, firstWord + i);
            #if CY_CORE_CM7_0 || CY_CORE_CM7_1
                {
                    uintptr_t line = ((uintptr_t) address) & ~(uintptr_t) 31u;

```

```

        SCB_InvalidateDCache_by_Addr((uint32_t *) line, 32);
    }
#endif
    value = *((const volatile uint32_t *) address);
    copyBytes = bytes - i * sizeof(uint32_t);
    if (copyBytes > sizeof(uint32_t)) copyBytes = sizeof(uint32_t);
    memcpy(&output[i * sizeof(uint32_t)], &value, copyBytes);
}
return 1u;
}

static uint8_t recordBlank(uint32_t page, uint32_t recordIndex)
{
    paramSlotFlashState state = blankCheckWords(page,
        recordIndex * PARAM_SLOT_SELECTOR_RECORD_WORDS,
        PARAM_SLOT_SELECTOR_RECORD_WORDS);
    if (state == PARAM_SLOT_FLASH_BLANK) return 1u;
    if (state == PARAM_SLOT_FLASH_PROGRAMMED) return 0u;
    return 2u;
}

static uint32_t firstBlankRecord(uint32_t page)
{
    uint32_t record;
    for (record = 0u; record < PARAM_SLOT_SELECTOR_RECORDS_PER_PAGE; record++)
    {
        uint8_t state = recordBlank(page, record);
        if (state == 1u) return record;
        if (state == 2u) return PARAM_SLOT_SELECTOR_RECORDS_PER_PAGE + 1u;
    }
    return PARAM_SLOT_SELECTOR_RECORDS_PER_PAGE;
}

static uint8_t loadLatestSelector(paramSlotSelector *latest,
    uint32_t *latestPage, uint32_t *nextRecord)
{
    uint32_t pages[2u] = {PARAM_SLOT_SELECTOR_PAGE_A, PARAM_SLOT_SELECTOR_PAGE_B};
    uint8_t found = 0u;
    uint8_t ioFailed = 0u;
    uint8_t programmedInvalid = 0u;
    uint8_t committedInvalid = 0u;
    uint32_t p;

    *latestPage = PARAM_SLOT_SELECTOR_PAGE_A;
    *nextRecord = 0u;
    for (p = 0u; p < 2u; p++)
    {
        uint32_t record;
        for (record = 0u; record < PARAM_SLOT_SELECTOR_RECORDS_PER_PAGE; record++)
        {
            paramSlotSelector candidate;
            uint8_t blank = recordBlank(pages[p], record);
            if (blank == 2u)
            {
                ioFailed = 1u;
                continue;
            }
            if (blank == 1u) continue;
            if (readFlashBytes(pages[p], record * PARAM_SLOT_SELECTOR_BYTES,
                &candidate, sizeof(candidate)) == 0u)
            {
                ioFailed = 1u;
                continue;
            }
            if (selectorValid(&candidate) == 0u)
            {
                programmedInvalid = 1u;
                if (candidate.magic == PARAM_SLOT_SELECTOR_MAGIC)
                    committedInvalid = 1u;
            }
            else if ((found == 0u) ||
                sequenceNewer(candidate.sequence, latest->sequence))
            {
                *latest = candidate;
                *latestPage = pages[p];
                *nextRecord = firstBlankRecord(pages[p]);
                if (*nextRecord > PARAM_SLOT_SELECTOR_RECORDS_PER_PAGE)
                    ioFailed = 1u;
                found = 1u;
            }
        }
    }
}

```

```

    }
}
}
if ((found == 0u) && (ioFailed == 0u) &&
    (programmedInvalid == 0u))
{
    *latestPage = PARAM_SLOT_SELECTOR_PAGE_A;
    *nextRecord = firstBlankRecord(PARAM_SLOT_SELECTOR_PAGE_A);
    if (*nextRecord >= PARAM_SLOT_SELECTOR_RECORDS_PER_PAGE)
    {
        *latestPage = PARAM_SLOT_SELECTOR_PAGE_B;
        *nextRecord = firstBlankRecord(PARAM_SLOT_SELECTOR_PAGE_B);
    }
    if (*nextRecord > PARAM_SLOT_SELECTOR_RECORDS_PER_PAGE) ioFailed = 1u;
}
if (ioFailed != 0u) return 2u;
if (committedInvalid != 0u) return (found != 0u) ? 3u : 4u;
if ((found == 0u) && (programmedInvalid != 0u)) return 4u;
return found;
}

static uint8_t programWord(uint32_t page, uint32_t wordIndex, uint32_t value)
{
    uint32_t readback;
    cleanFlashSource(&value, sizeof(value));
    Cy_FlashWriteWork((PARAM_SLOT_FLASH_ADDRESS_TYPE) (uintptr_t)
        flashWord(page, wordIndex),
        &value, CY_FLASH_DRIVER_BLOCKING);
    if (readFlashBytes(page, wordIndex * sizeof(uint32_t),
        &readback, sizeof(readback)) == 0u) return 0u;
    return (uint8_t) (readback == value);
}

static uint8_t resolveUncertainSelector(paramSlotStore *store)
{
    paramSlotSelector latest;
    uint32_t page;
    uint32_t record;
    uint8_t loadStatus;
    if (store->selectorUncertain == 0u) return 1u;
    memset(&latest, 0, sizeof(latest));
    loadStatus = loadLatestSelector(&latest, &page, &record);
    if (loadStatus >= 2u) return 0u;
    /* An initialized store had a durable selector before this write, so an
     * empty rescan cannot prove rollback and remains fail-closed. During the
     * very first selector creation, however, empty is the expected durable
     * state when the magic word never committed; allow the caller to retry. */
    if (loadStatus == 0u)
    {
        if (store->initialized != 0u) return 0u;
        memset(store->selectorBytes, 0, sizeof(store->selectorBytes));
        store->selectorUncertain = 0u;
        return 1u;
    }
    memcpy(store->selectorBytes, &latest, sizeof(latest));
    store->selectorUncertain = 0u;
    return 1u;
}

static uint8_t reconcileSelectorWrite(paramSlotStore *store,
    const paramSlotSelector *expected)
{
    store->selectorUncertain = 1u;
    if (resolveUncertainSelector(store) == 0u) return 0u;
    return (uint8_t) (memcmp(store->selectorBytes, expected,
        sizeof(*expected)) == 0);
}

static uint8_t appendSelector(paramSlotStore *store,
    paramSlotSelector *next)
{
    paramSlotSelector latest;
    uint32_t page;
    uint32_t record;
    uint32_t baseWord;
    const uint32_t *words = (const uint32_t *) (const void *) next;
    uint32_t i;

    if (store->selectorUncertain != 0u)

```

```

{
    if (resolveUncertainSelector(store) == 0u) return 0u;
    return 0u;
}

memset(&latest, 0, sizeof(latest));
{
    uint8_t loadStatus = loadLatestSelector(&latest, &page, &record);
    if (loadStatus >= 2u) return 0u;
    if (loadStatus != 0u)
    {
        next->sequence = latest.sequence + 1u;
        if (next->sequence == 0u) next->sequence = 1u;
    }
    else
    {
        next->sequence = 1u;
    }
}
if (record >= PARAM_SLOT_SELECTOR_RECORDS_PER_PAGE)
{
    page = (page == PARAM_SLOT_SELECTOR_PAGE_A) ?
        PARAM_SLOT_SELECTOR_PAGE_B : PARAM_SLOT_SELECTOR_PAGE_A;
    if (flash_check(0u, page) != 0u) flash_erase_page(0u, page);
    record = 0u;
}
next->magic = PARAM_SLOT_SELECTOR_MAGIC;
next->version = PARAM_SLOT_SELECTOR_VERSION;
next->recordBytes = PARAM_SLOT_SELECTOR_BYTES;
next->crc = crc32IeeeCalc((const uint8_t *) next,
    (uint32_t) offsetof(paramSlotSelector, crc));
baseWord = record * PARAM_SLOT_SELECTOR_RECORD_WORDS;
for (i = 1u; i < PARAM_SLOT_SELECTOR_RECORD_WORDS; i++)
{
    if (programWord(page, baseWord + i, words[i]) == 0u) return 0u;
}
if (programWord(page, baseWord, words[0]) == 0u)
    return reconcileSelectorWrite(store, next);
{
    paramSlotSelector verified;
    if (readFlashBytes(page, record * PARAM_SLOT_SELECTOR_BYTES,
        &verified, sizeof(verified)) == 0u)
        return reconcileSelectorWrite(store, next);
    if ((selectorValid(&verified) == 0u) ||
        (memcmp(&verified, next, sizeof(verified)) != 0))
        return reconcileSelectorWrite(store, next);
}
memcpy(store->selectorBytes, next, sizeof(*next));
store->selectorUncertain = 0u;
return 1u;
}

static uint32_t runtimeDataPage(uint8_t bank)
{
    return bank != 0u ? PARAM_RUNTIME_DATA_PAGE_B :
        PARAM_RUNTIME_DATA_PAGE_A;
}

static uint8_t runtimeSelectorValid(const paramRuntimeSelector *selector)
{
    uint32_t crc;
    if ((selector->magic != PARAM_RUNTIME_SELECTOR_MAGIC) ||
        (selector->version != PARAM_RUNTIME_SELECTOR_VERSION) ||
        (selector->recordBytes != PARAM_RUNTIME_SELECTOR_BYTES) ||
        (selector->sequence == 0u) ||
        (selector->bank > 1u) || (selector->valid > 1u) ||
        (selector->pending > 1u) ||
        ((selector->sourceSlot != 0u) &&
        (slotValid(selector->sourceSlot) == 0u)) ||
        (selector->rollbackBank > 1u) ||
        (selector->rollbackValid > 1u) ||
        ((selector->rollbackSourceSlot != 0u) &&
        (slotValid(selector->rollbackSourceSlot) == 0u)) ||
        (selector->reserved0 != 0u) || selector->reserved[0u] != 0u ||
        selector->reserved[1u] != 0u || selector->reserved[2u] != 0u)
    {
        return 0u;
    }
    if ((selector->valid == 0u) || (selector->generation == 0u)) return 0u;
}

```



```

    if ((selector->pending != 0u) &&
        ((selector->transactionId == 0u) ||
         (selector->rollbackValid == 0u) ||
         (selector->rollbackGeneration == 0u) ||
         (selector->bank == selector->rollbackBank))) return 0u;
    if ((selector->pending == 0u) &&
        (selector->rollbackBank != 0u || selector->rollbackValid != 0u ||
         selector->rollbackSourceSlot != 0u ||
         selector->rollbackGeneration != 0u ||
         selector->rollbackBlobCrc != 0u ||
         selector->rollbackTransactionId != 0u)) return 0u;
    crc = crc32leeeCalc((const uint8_t *) selector,
        (uint32_t) offsetof(paramRuntimeSelector, crc));
    return (uint8_t) (crc == selector->crc);
}

static uint8_t runtimeRecordBlank(uint32_t page, uint32_t recordIndex)
{
    paramSlotFlashState state = blankCheckWords(page,
        recordIndex * PARAM_RUNTIME_SELECTOR_RECORD_WORDS,
        PARAM_RUNTIME_SELECTOR_RECORD_WORDS);
    if (state == PARAM_SLOT_FLASH_BLANK) return 1u;
    if (state == PARAM_SLOT_FLASH_PROGRAMMED) return 0u;
    return 2u;
}

static uint32_t firstBlankRuntimeRecord(uint32_t page)
{
    uint32_t record;
    for (record = 0u; record < PARAM_RUNTIME_SELECTOR_RECORDS_PER_PAGE;
        record++)
    {
        uint8_t state = runtimeRecordBlank(page, record);
        if (state == 1u) return record;
        if (state == 2u) return PARAM_RUNTIME_SELECTOR_RECORDS_PER_PAGE + 1u;
    }
    return PARAM_RUNTIME_SELECTOR_RECORDS_PER_PAGE;
}

static uint8_t loadLatestRuntimeSelector(paramRuntimeSelector *latest,
    uint32_t *latestPage, uint32_t *nextRecord)
{
    const uint32_t pages[2u] = {PARAM_RUNTIME_SELECTOR_PAGE_A,
        PARAM_RUNTIME_SELECTOR_PAGE_B};
    uint8_t found = 0u;
    uint8_t ioFailed = 0u;
    uint8_t programmedInvalid = 0u;
    uint8_t committedInvalid = 0u;
    uint32_t p;

    *latestPage = PARAM_RUNTIME_SELECTOR_PAGE_A;
    *nextRecord = 0u;
    for (p = 0u; p < 2u; p++)
    {
        uint32_t record;
        for (record = 0u; record < PARAM_RUNTIME_SELECTOR_RECORDS_PER_PAGE;
            record++)
        {
            paramRuntimeSelector candidate;
            uint8_t blank = runtimeRecordBlank(pages[p], record);
            if (blank == 2u)
            {
                ioFailed = 1u;
                continue;
            }
            if (blank == 1u) continue;
            if (readFlashBytes(pages[p],
                record * PARAM_RUNTIME_SELECTOR_BYTES,
                &candidate, sizeof(candidate)) == 0u)
            {
                ioFailed = 1u;
                continue;
            }
            if (runtimeSelectorValid(&candidate) == 0u)
            {
                programmedInvalid = 1u;
                if (candidate.magic == PARAM_RUNTIME_SELECTOR_MAGIC)
                    committedInvalid = 1u;
            }
        }
    }
}

```

```

        else if ((found == 0u) ||
            sequenceNewer(candidate.sequence, latest->sequence))
        {
            *latest = candidate;
            *latestPage = pages[p];
            *nextRecord = firstBlankRuntimeRecord(pages[p]);
            if (*nextRecord > PARAM_RUNTIME_SELECTOR_RECORDS_PER_PAGE)
                ioFailed = 1u;
            found = 1u;
        }
    }
}

if ((found == 0u) && (ioFailed == 0u) &&
    (programmedInvalid == 0u))
{
    *latestPage = PARAM_RUNTIME_SELECTOR_PAGE_A;
    *nextRecord = firstBlankRuntimeRecord(PARAM_RUNTIME_SELECTOR_PAGE_A);
    if (*nextRecord >= PARAM_RUNTIME_SELECTOR_RECORDS_PER_PAGE)
    {
        *latestPage = PARAM_RUNTIME_SELECTOR_PAGE_B;
        *nextRecord = firstBlankRuntimeRecord(PARAM_RUNTIME_SELECTOR_PAGE_B);
    }
    if (*nextRecord > PARAM_RUNTIME_SELECTOR_RECORDS_PER_PAGE)
        ioFailed = 1u;
}

return (uint8_t) ((ioFailed != 0u) || (committedInvalid != 0u) ||
    ((found == 0u) && (programmedInvalid != 0u)) ? 2u : found);
}

static uint8_t resolveUncertainRuntimeSelector(paramSlotStore *store)
{
    paramRuntimeSelector latest;
    uint32_t page;
    uint32_t record;
    uint8_t loadStatus;
    if (store->runtimeSelectorUncertain == 0u) return 1u;
    memset(&latest, 0, sizeof(latest));
    loadStatus = loadLatestRuntimeSelector(&latest, &page, &record);
    if (loadStatus == 2u) return 0u;
    if (loadStatus == 0u)
    {
        if (store->runtimeInitialized != 0u) return 0u;
        memset(store->runtimeSelectorBytes, 0,
            sizeof(store->runtimeSelectorBytes));
        store->runtimeSelectorUncertain = 0u;
        return 1u;
    }
    memcpy(store->runtimeSelectorBytes, &latest, sizeof(latest));
    store->runtimeSelectorUncertain = 0u;
    return 1u;
}

static uint8_t reconcileRuntimeSelectorWrite(paramSlotStore *store,
    const paramRuntimeSelector *expected)
{
    store->runtimeSelectorUncertain = 1u;
    if (resolveUncertainRuntimeSelector(store) == 0u) return 0u;
    return (uint8_t) (memcmp(store->runtimeSelectorBytes, expected,
        sizeof(*expected)) == 0);
}

static uint8_t appendRuntimeSelector(paramSlotStore *store,
    paramRuntimeSelector *next)
{
    paramRuntimeSelector latest;
    uint32_t page;
    uint32_t record;
    uint32_t baseWord;
    const uint32_t *words = (const uint32_t *) (const void *) next;
    uint32_t i;

    if (store->runtimeSelectorUncertain != 0u)
    {
        if (resolveUncertainRuntimeSelector(store) == 0u) return 0u;
        return 0u;
    }
    memset(&latest, 0, sizeof(latest));
    {
        uint8_t loadStatus = loadLatestRuntimeSelector(&latest, &page, &record);

```

```

        if (loadStatus == 2u) return 0u;
        next->sequence = (loadStatus != 0u) ? latest.sequence + 1u : 1u;
        if (next->sequence == 0u) next->sequence = 1u;
    }
    if (record >= PARAM_RUNTIME_SELECTOR_RECORDS_PER_PAGE)
    {
        page = (page == PARAM_RUNTIME_SELECTOR_PAGE_A) ?
            PARAM_RUNTIME_SELECTOR_PAGE_B : PARAM_RUNTIME_SELECTOR_PAGE_A;
        if (flash_check(0u, page) != 0u) flash_erase_page(0u, page);
        record = 0u;
    }
    next->magic = PARAM_RUNTIME_SELECTOR_MAGIC;
    next->version = PARAM_RUNTIME_SELECTOR_VERSION;
    next->recordBytes = PARAM_RUNTIME_SELECTOR_BYTES;
    next->crc = crc32IeeeCalc((const uint8_t *) next,
        (uint32_t) offsetof(paramRuntimeSelector, crc));
    baseWord = record * PARAM_RUNTIME_SELECTOR_RECORD_WORDS;
    for (i = 1u; i < PARAM_RUNTIME_SELECTOR_RECORD_WORDS; i++)
    {
        if (programWord(page, baseWord + i, words[i]) == 0u) return 0u;
    }
    if (programWord(page, baseWord, words[0]) == 0u)
        return reconcileRuntimeSelectorWrite(store, next);
    {
        paramRuntimeSelector verified;
        if (readFlashBytes(page, record * PARAM_RUNTIME_SELECTOR_BYTES,
            &verified, sizeof(verified)) == 0u)
            return reconcileRuntimeSelectorWrite(store, next);
        if ((runtimeSelectorValid(&verified) == 0u) ||
            (memcmp(&verified, next, sizeof(verified)) != 0))
            return reconcileRuntimeSelectorWrite(store, next);
    }
    memcpy(store->runtimeSelectorBytes, next, sizeof(*next));
    store->runtimeSelectorUncertain = 0u;
    return 1u;
}

static uint8_t readDataPage(const paramSlotStore *store,
    uint32_t page, void *blob)
{
    {
        uint32_t paddedBytes = (store->blobBytes + 3u) & ~3u;
        uint8_t *scratch = (uint8_t *) (void *) flash_union_buffer;
        /* SeekFree's flash_buffer_clear() only clears FLASH_PAGE_LENGTH bytes,
         * although flash_union_buffer occupies FLASH_PAGE_SIZE bytes. Fill the
         * complete source page here so a non-word-aligned blob cannot inherit
         * stale tail bytes. */
        memset((void *) flash_union_buffer, 0xFF, FLASH_PAGE_SIZE);
        if (readFlashBytes(page, 0u, scratch, paddedBytes) == 0u) return 0u;
        if (store->validate(scratch, store->blobBytes) == 0u) return 0u;
        if (blob != scratch) memcpy(blob, scratch, store->blobBytes);
        return 1u;
    }
}

static paramSlotStatus verifyPendingLibraryCandidate(
    const paramSlotStore *store, uint32_t transactionId)
{
    {
        const paramSlotSelector *selector = selectorConst(store);
        uint32_t packed = selector->reserved[2u];
        uint8_t *blob = (uint8_t *) (void *) flash_union_buffer;
        uint8_t slot;
        uint8_t index;
        uint8_t bank;

        if ((packed & PARAM_SLOT_PENDING_MAGIC_MASK) != PARAM_SLOT_PENDING_MAGIC)
            return PARAM_SLOT_OK;
        slot = (uint8_t) (packed & 0x0Fu);
        if (slotValid(slot) == 0u) return PARAM_SLOT_VERIFY_FAILED;
        index = slot - 1u;
        if (transactionId == PARAM_RUNTIME_TX_ACTIVATION_ONLY)
        {
            if ((packed & PARAM_SLOT_PENDING_ACTIVATION) == 0u)
                return PARAM_SLOT_TRANSACTION_MISMATCH;
        }
        else if (((packed & PARAM_SLOT_PENDING_ACTIVATION) != 0u) ||
            (selector->transactionId[index] != transactionId))
        {
            return PARAM_SLOT_TRANSACTION_MISMATCH;
        }
    }
    if (((selector->validMask >> index) & 1u) == 0u ||

```

```

        selector->generation[index] == 0u)
        return PARAM_SLOT_VERIFY_FAILED;
    bank = (uint8_t)((selector->bankMask >> index) & 1u);
    if (readDataPage(store, dataPage(slot, bank), blob) == 0u ||
        crc32IeeeCalc(blob, store->blobBytes) != selector->blobCrc[index])
        return PARAM_SLOT_VERIFY_FAILED;
    return PARAM_SLOT_OK;
}

static uint8_t rollbackPendingSelector(paramSlotStore *store)
{
    const paramSlotSelector *current = selectorConst(store);
    paramSlotSelector rollback;
    uint32_t packed = current->reserved[2u];
    uint8_t slot;
    uint8_t oldActive;
    uint8_t oldValid;
    uint8_t oldBank;
    uint8_t *blob = (uint8_t *) (void *) flash_union_buffer;
    if ((packed & PARAM_SLOT_PENDING_MAGIC_MASK) != PARAM_SLOT_PENDING_MAGIC) return 1u;
    slot = (uint8_t)(packed & 0x0Fu);
    oldActive = (uint8_t)((packed >> 4) & 0x0Fu);
    oldValid = (uint8_t)((packed >> 8) & 0x01u);
    oldBank = (uint8_t)((packed >> 9) & 0x01u);
    if (!slotValid(slot) || !slotValid(oldActive)) return 0u;
    rollback = *current;
    rollback.activeSlot = (oldValid != 0u) ? oldActive : 0u;
    if (oldValid)
    {
        if (!readDataPage(store, dataPage(slot, oldBank), blob)) return 0u;
        if (oldBank) rollback.bankMask |= (uint8_t)(1u << (slot - 1u));
        else rollback.bankMask &= (uint8_t)~(1u << (slot - 1u));
        rollback.validMask |= (uint8_t)(1u << (slot - 1u));
        rollback.generation[slot - 1u] = current->reserved[1u];
        rollback.blobCrc[slot - 1u] = crc32IeeeCalc(blob, store->blobBytes);
        rollback.transactionId[slot - 1u] = current->reserved[0u];
    }
    else
    {
        rollback.validMask &= (uint8_t)~(1u << (slot - 1u));
        rollback.bankMask &= (uint8_t)~(1u << (slot - 1u));
        rollback.generation[slot - 1u] = 0u;
        rollback.blobCrc[slot - 1u] = 0u;
        rollback.transactionId[slot - 1u] = 0u;
    }
    memset(rollback.reserved, 0, sizeof(rollback.reserved));
    return appendSelector(store, &rollback);
}

static uint8_t writeDataPage(const paramSlotStore *store,
    uint32_t page, const void *blob)
{
    uint32_t words = (store->blobBytes + 3u) / 4u;
    void *verify = (void *) flash_union_buffer;

    if ((store->blobBytes > FLASH_PAGE_SIZE) ||
        (store->validate(blob, store->blobBytes) == 0u)) return 0u;
    memset((void *) flash_union_buffer, 0xFF, FLASH_PAGE_SIZE);
    memcpy((void *) flash_union_buffer, blob, store->blobBytes);
    cleanFlashSource((const void *) flash_union_buffer, words * 4u);
    if (flash_check(0u, page) != 0u) flash_erase_page(0u, page);
    if (flash_write_page_from_buffer(0u, page, words) != 0u) return 0u;
    memset((void *) flash_union_buffer, 0xFF, FLASH_PAGE_SIZE);
    if (readDataPage(store, page, verify) == 0u) return 0u;
    return (uint8_t)(memcmp(verify, blob, store->blobBytes) == 0);
}

static void recoverSelectorFromDataPages(const paramSlotStore *store,
    paramSlotSelector *selector)
{
    uint8_t *blob = (uint8_t *) (void *) flash_union_buffer;
    uint8_t slot;

    memset(selector, 0, sizeof(*selector));
    for (slot = PARAM_SLOT_FIRST; slot <= PARAM_SLOT_LAST; slot++)
    {
        uint8_t bank;
        for (bank = 0u; bank < 2u; bank++)
        {

```

```

        if (readDataPage(store, dataPage(slot, bank), blob) != 0u)
        {
            uint8_t index = (uint8_t) (slot - 1u);
            uint8_t mask = (uint8_t) (1u << index);
            selector->validMask |= mask;
            if (bank != 0u) selector->bankMask |= mask;
            selector->generation[index] = 1u;
            selector->blobCrc[index] = crc32IeeeCalc(blob,
                store->blobBytes);
            if (selector->activeSlot == 0u) selector->activeSlot = slot;
            break;
        }
    }
}
}
}

```

```

static paramSlotStatus readRuntimeWithSelector(const paramSlotStore *store,
    const paramRuntimeSelector *selector, void *blob)
{
    uint32_t paddedBytes;
    uint8_t *scratch = (uint8_t *) (void *) flash_union_buffer;
    if ((selector->valid == 0u) || (selector->bank > 1u))
        return PARAM_SLOT_EMPTY;
    paddedBytes = (store->blobBytes + 3u) & ~3u;
    memset((void *) flash_union_buffer, 0xFF, FLASH_PAGE_SIZE);
    if (readFlashBytes(runtimeDataPage(selector->bank), 0u, scratch,
        paddedBytes) == 0u) return PARAM_SLOT_IO_FAILED;
    if (store->validate(scratch, store->blobBytes) == 0u)
        return PARAM_SLOT_INVALID;
    if (crc32IeeeCalc(scratch, store->blobBytes) !=
        selector->blobCrc) return PARAM_SLOT_VERIFY_FAILED;
    if (blob != scratch) memcpy(blob, scratch, store->blobBytes);
    return PARAM_SLOT_OK;
}

```

```

static paramSlotStatus prepareRuntimeBlob(paramSlotStore *store,
    const void *blob, uint32_t transactionId, uint8_t sourceSlot)
{
    const paramRuntimeSelector *current;
    uint8_t bank;
    uint32_t generation;
    if ((store == NULL) || (blob == NULL) ||
        ((sourceSlot != 0u) && (slotValid(sourceSlot) == 0u)) ||
        (store->validate(blob, store->blobBytes) == 0u))
        return PARAM_SLOT_BAD_ARGUMENT;
    if (resolveUncertainRuntimeSelector(store) == 0u)
        return PARAM_SLOT_IO_FAILED;
    current = runtimeSelectorConst(store);
    if ((store->runtimeStagedValid != 0u) ||
        ((store->runtimeInitialized != 0u) && (current->pending != 0u)))
        return PARAM_SLOT_NO_PREPARE;
    bank = (store->runtimeInitialized != 0u && current->valid != 0u) ?
        (uint8_t) (current->bank ^ 1u) : 0u;
    generation = (store->runtimeInitialized != 0u && current->valid != 0u) ?
        current->generation + 1u : 1u;
    if (generation == 0u) generation = 1u;
    if (writeDataPage(store, runtimeDataPage(bank), blob) == 0u)
        return PARAM_SLOT_VERIFY_FAILED;
    store->runtimeStagedValid = 1u;
    store->runtimeStagedCommitted = 0u;
    store->runtimeStagedBank = bank;
    store->runtimeStagedSourceSlot = sourceSlot;
    store->runtimeStagedGeneration = generation;
    store->runtimeStagedBlobCrc = crc32IeeeCalc((const uint8_t *) blob,
        store->blobBytes);
    store->runtimeStagedTransactionId = transactionId;
    return PARAM_SLOT_OK;
}

```

```

static paramSlotStatus commitRuntimeBlob(paramSlotStore *store,
    uint32_t transactionId, uint8_t pending)
{
    paramRuntimeSelector next;
    const paramRuntimeSelector *current;
    if ((store == NULL) || (store->runtimeStagedValid == 0u))
        return PARAM_SLOT_NO_PREPARE;
    if (store->runtimeStagedTransactionId != transactionId)
        return PARAM_SLOT_TRANSACTION_MISMATCH;
    if ((pending != 0u) && (transactionId == 0u))

```

```

        return PARAM_SLOT_BAD_ARGUMENT;
current = runtimeSelectorConst(store);
memset(&next, 0, sizeof(next));
if (store->runtimeInitialized != 0u) next = *current;
if (pending != 0u)
{
    if ((store->runtimeInitialized == 0u) || (current->valid == 0u))
        return PARAM_SLOT_NO_PREPARE;
    next.rollbackBank = current->bank;
    next.rollbackValid = current->valid;
    next.rollbackSourceSlot = current->sourceSlot;
    next.rollbackGeneration = current->generation;
    next.rollbackBlobCrc = current->blobCrc;
    next.rollbackTransactionId = current->transactionId;
}
else
{
    next.rollbackBank = 0u;
    next.rollbackValid = 0u;
    next.rollbackSourceSlot = 0u;
    next.rollbackGeneration = 0u;
    next.rollbackBlobCrc = 0u;
    next.rollbackTransactionId = 0u;
}
next.bank = store->runtimeStagedBank;
next.valid = 1u;
next.pending = (uint8_t) (pending != 0u);
next.sourceSlot = store->runtimeStagedSourceSlot;
next.generation = store->runtimeStagedGeneration;
next.blobCrc = store->runtimeStagedBlobCrc;
next.transactionId = transactionId;
memset(next.reserved, 0, sizeof(next.reserved));
if (appendRuntimeSelector(store, &next) == 0u)
    return PARAM_SLOT_IO_FAILED;
store->runtimeInitialized = 1u;
store->runtimeStagedCommitted = 1u;
return PARAM_SLOT_OK;
}

static void clearRuntimeStage(paramSlotStore *store)
{
    store->runtimeStagedValid = 0u;
    store->runtimeStagedCommitted = 0u;
    store->runtimeStagedBank = 0u;
    store->runtimeStagedSourceSlot = 0u;
    store->runtimeStagedGeneration = 0u;
    store->runtimeStagedBlobCrc = 0u;
    store->runtimeStagedTransactionId = 0u;
}

static paramSlotStatus abortRuntimeTransaction(paramSlotStore *store,
uint32_t transactionId)
{
    paramRuntimeSelector next;
    const paramRuntimeSelector *current;
    if (resolveUncertainRuntimeSelector(store) == 0u)
        return PARAM_SLOT_IO_FAILED;
    current = runtimeSelectorConst(store);
    if ((store->runtimeInitialized != 0u) && (current->pending != 0u))
    {
        if (current->transactionId != transactionId)
            return PARAM_SLOT_TRANSACTION_MISMATCH;
        if (current->rollbackValid == 0u) return PARAM_SLOT_INVALID;
        next = *current;
        next.bank = current->rollbackBank;
        next.valid = current->rollbackValid;
        next.pending = 0u;
        next.sourceSlot = current->rollbackSourceSlot;
        next.generation = current->rollbackGeneration;
        next.blobCrc = current->rollbackBlobCrc;
        next.transactionId = current->rollbackTransactionId;
        next.rollbackBank = 0u;
        next.rollbackValid = 0u;
        next.rollbackSourceSlot = 0u;
        next.rollbackGeneration = 0u;
        next.rollbackBlobCrc = 0u;
        next.rollbackTransactionId = 0u;
        memset(next.reserved, 0, sizeof(next.reserved));
        if (appendRuntimeSelector(store, &next) == 0u)

```

```

        return PARAM_SLOT_IO_FAILED;
    }
    else if ((store->runtimeStagedValid != 0u) &&
             (store->runtimeStagedTransactionId != transactionId))
    {
        return PARAM_SLOT_TRANSACTION_MISMATCH;
    }
    clearRuntimeStage(store);
    return PARAM_SLOT_OK;
}

static paramSlotStatus finishRuntimeTransaction(paramSlotStore *store,
        uint32_t transactionId)
{
    paramRuntimeSelector next;
    const paramRuntimeSelector *current;
    uint8_t candidate[FLASH_PAGE_SIZE];
    if (resolveUncertainRuntimeSelector(store) == 0u)
        return PARAM_SLOT_IO_FAILED;
    current = runtimeSelectorConst(store);
    if ((store->runtimeInitialized != 0u) && (current->pending != 0u))
    {
        if (current->transactionId != transactionId)
            return PARAM_SLOT_TRANSACTION_MISMATCH;
        /* FINISH is the durable point of no return. Re-read the candidate
         * before dropping rollback metadata so a damaged or unreadable bank
         * remains abortable after reboot. */
        if (readRuntimeWithSelector(store, current, candidate) != PARAM_SLOT_OK)
            return PARAM_SLOT_VERIFY_FAILED;
        next = *current;
        next.pending = 0u;
        next.rollbackBank = 0u;
        next.rollbackValid = 0u;
        next.rollbackSourceSlot = 0u;
        next.rollbackGeneration = 0u;
        next.rollbackBlobCrc = 0u;
        next.rollbackTransactionId = 0u;
        memset(next.reserved, 0, sizeof(next.reserved));
        if (appendRuntimeSelector(store, &next) == 0u)
            return PARAM_SLOT_IO_FAILED;
    }
    else if ((store->runtimeStagedValid != 0u) &&
             (store->runtimeStagedTransactionId != transactionId))
    {
        return PARAM_SLOT_TRANSACTION_MISMATCH;
    }
    clearRuntimeStage(store);
    return PARAM_SLOT_OK;
}

static paramSlotStatus initializeRuntime(paramSlotStore *store,
        const void *defaultBlob)
{
    paramRuntimeSelector selector;
    const paramSlotSelector *library;
    uint32_t page;
    uint32_t record;
    uint32_t packed;
    uint8_t blob[FLASH_PAGE_SIZE];
    uint8_t sourceSlot = 0u;
    uint8_t loadStatus;
    paramSlotStatus status;

    memset(&selector, 0, sizeof(selector));
    loadStatus = loadLatestRuntimeSelector(&selector, &page, &record);
    if (loadStatus == 2u) return PARAM_SLOT_IO_FAILED;
    if (loadStatus != 0u)
    {
        paramRuntimeSelector committed;
        memcpy(store->runtimeSelectorBytes, &selector, sizeof(selector));
        store->runtimeInitialized = 1u;
        status = readRuntimeWithSelector(store, &selector, blob);
        if (selector.pending != 0u)
        {
            if (selector.rollbackValid == 0u) return PARAM_SLOT_INVALID;
            committed = selector;
            committed.bank = selector.rollbackBank;
            committed.valid = selector.rollbackValid;
            committed.pending = 0u;

```

```

    committed.sourceSlot = selector.rollbackSourceSlot;
    committed.generation = selector.rollbackGeneration;
    committed.blobCrc = selector.rollbackBlobCrc;
    committed.transactionId = selector.rollbackTransactionId;
    /* A pending candidate is not the startup source. Even if its
     * bank is corrupt, preserve the selector and boot the rollback
     * bank so the coordinator can still issue ABORT. */
    return readRuntimeWithSelector(store, &committed, blob);
}
if (status == PARAM_SLOT_OK) return PARAM_SLOT_OK;
if (selector.pending != 0u) return status;
if ((status == PARAM_SLOT_IO_FAILED) || (defaultBlob == NULL) ||
    (store->validate(defaultBlob, store->blobBytes) == 0u))
    return status;
/* A structurally readable but invalid runtime is replaced by the
 * current compiled defaults. The library pages remain untouched. */
selector.pending = 0u;
selector.rollbackValid = 0u;
memcpy(store->runtimeSelectorBytes, &selector, sizeof(selector));
clearRuntimeStage(store);
status = prepareRuntimeBlob(store, defaultBlob, 0u, 0u);
if (status == PARAM_SLOT_OK) status = commitRuntimeBlob(store, 0u, 0u);
if (status == PARAM_SLOT_OK)
{
    clearRuntimeStage(store);
    store->runtimeDefaultsRewritten = 1u;
}
return status;
}

library = selectorConst(store);
sourceSlot = library->activeSlot;
packed = library->reserved[2u];

/* Migrate a transaction that had already switched the legacy active-slot
 * selector. Materialize its previous startup source first, then recreate
 * the candidate as a pending runtime transaction. This preserves the
 * coordinator's post-reboot ABORT/FINISH decision instead of silently
 * finalizing whichever slot happened to be active at upgrade time. */
if (((packed & PARAM_SLOT_PENDING_MAGIC_MASK) == PARAM_SLOT_PENDING_MAGIC) &&
    slotValid((uint8_t) (packed & 0x0Fu)) &&
    (sourceSlot == (uint8_t) (packed & 0x0Fu)))
{
    uint8_t targetSlot = (uint8_t) (packed & 0x0Fu);
    uint8_t oldActive = (uint8_t) ((packed >> 4) & 0x0Fu);
    uint8_t oldValid = 0u;
    uint8_t oldBank = 0u;
    uint32_t transactionId;

    if (slotValid(oldActive))
    {
        if (oldActive == targetSlot)
        {
            oldValid = (uint8_t) ((packed >> 8) & 1u);
            oldBank = (uint8_t) ((packed >> 9) & 1u);
            status = (oldValid != 0u) &&
                readDataPage(store, dataPage(oldActive, oldBank), blob) ?
                PARAM_SLOT_OK : PARAM_SLOT_EMPTY;
        }
        else
        {
            status = paramSlotStoreRead(store, oldActive, blob,
                store->blobBytes);
        }
    }
    else
    {
        status = PARAM_SLOT_EMPTY;
    }
}
if ((status != PARAM_SLOT_OK) && (defaultBlob != NULL) &&
    (store->validate(defaultBlob, store->blobBytes) != 0u))
{
    memcpy(blob, defaultBlob, store->blobBytes);
    oldActive = 0u;
    status = PARAM_SLOT_OK;
}
if (status != PARAM_SLOT_OK) return status;
status = prepareRuntimeBlob(store, blob, 0u, oldActive);
if (status == PARAM_SLOT_OK) status = commitRuntimeBlob(store, 0u, 0u);

```



```

if (status != PARAM_SLOT_OK) return status;
clearRuntimeStage(store);

status = paramSlotStoreRead(store, targetSlot, blob, store->blobBytes);
if (status != PARAM_SLOT_OK)
{
    /* Keep the valid rollback runtime and the legacy pending marker;
     * the coordinator can still ABORT the damaged transaction. */
    return PARAM_SLOT_OK;
}
transactionId = ((packed & PARAM_SLOT_PENDING_ACTIVATION) != 0u) ?
    PARAM_RUNTIME_TX_ACTIVATION_ONLY :
    library->transactionId[targetSlot - 1u];
if (transactionId == 0u) return PARAM_SLOT_OK;
status = prepareRuntimeBlob(store, blob, transactionId, targetSlot);
if (status == PARAM_SLOT_OK)
    status = commitRuntimeBlob(store, transactionId, 1u);
return status;
}

if ((slotValid(sourceSlot) != 0u) &&
    (paramSlotStoreRead(store, sourceSlot, blob, store->blobBytes) ==
     PARAM_SLOT_OK))
{
    status = prepareRuntimeBlob(store, blob, 0u, sourceSlot);
}
else if ((defaultBlob != NULL) &&
    (store->validate(defaultBlob, store->blobBytes) != 0u))
{
    status = prepareRuntimeBlob(store, defaultBlob, 0u, 0u);
}
else
{
    return PARAM_SLOT_EMPTY;
}
if (status == PARAM_SLOT_OK) status = commitRuntimeBlob(store, 0u, 0u);
if (status == PARAM_SLOT_OK) clearRuntimeStage(store);
return status;
}

paramSlotStatus paramSlotStoreInit(paramSlotStore *store,
    uint32_t blobBytes,
    paramSlotBlobValidator validate,
    const void *defaultBlob)
{
    paramSlotSelector selector;
    uint32_t page;
    uint32_t record;
    uint8_t legacy[FLASH_PAGE_SIZE];

    if ((store == NULL) || (validate == NULL) ||
        (blobBytes == 0u) ||
        (blobBytes > FLASH_PAGE_SIZE)) return PARAM_SLOT_BAD_ARGUMENT;
    memset(store, 0, sizeof(*store));
    store->blobBytes = blobBytes;
    store->validate = validate;
    flash_init();
    memset(&selector, 0, sizeof(selector));
    {
        uint8_t loadStatus = loadLatestSelector(&selector, &page, &record);
        if (loadStatus == 2u)
        {
            /* Runtime is the sole startup source. A corrupt library journal
             * remains unavailable/fail-closed, but must not strand a valid
             * committed runtime or prevent a verified defaults repair. */
            memset(store->selectorBytes, 0, sizeof(store->selectorBytes));
            store->initialized = 0u;
            return initializeRuntime(store, defaultBlob);
        }
        if (loadStatus == 3u)
        {
            /* A durable malformed record must not poison an independently
             * valid last-good selector forever. Rebuild only the selector
             * journal; library data pages and runtime pages are untouched. */
            if (flash_check(0u, PARAM_SLOT_SELECTOR_PAGE_A) != 0u)
                flash_erase_page(0u, PARAM_SLOT_SELECTOR_PAGE_A);
            if (flash_check(0u, PARAM_SLOT_SELECTOR_PAGE_B) != 0u)
                flash_erase_page(0u, PARAM_SLOT_SELECTOR_PAGE_B);
            memset(store->selectorBytes, 0, sizeof(store->selectorBytes));

```

```

        if (appendSelector(store, &selector) == 0u)
            return PARAM_SLOT_IO_FAILED;
        loadStatus = 1u;
    }
    else if (loadStatus == 4u)
    {
        /* No selector record survived validation. Reconstruct an
         * overwritable directory from independently valid data banks.
         * Generation/transaction identity cannot be trusted without a
         * selector and is restarted locally; the next five-board SAVE
         * assigns one common transaction identity. Runtime remains the
         * independent startup source. */
        if (flash_check(0u, PARAM_SLOT_SELECTOR_PAGE_A) != 0u)
            flash_erase_page(0u, PARAM_SLOT_SELECTOR_PAGE_A);
        if (flash_check(0u, PARAM_SLOT_SELECTOR_PAGE_B) != 0u)
            flash_erase_page(0u, PARAM_SLOT_SELECTOR_PAGE_B);
        recoverSelectorFromDataPages(store, &selector);
        memset(store->selectorBytes, 0, sizeof(store->selectorBytes));
        if (appendSelector(store, &selector) == 0u)
            return PARAM_SLOT_IO_FAILED;
        loadStatus = 1u;
    }
    if (loadStatus != 0u)
    {
        uint32_t packed;
        uint8_t slot;
        memcpy(store->selectorBytes, &selector, sizeof(selector));
        store->initialized = 1u;
        packed = selector.reserved[2u];
        if ((packed & PARAM_SLOT_PENDING_MAGIC_MASK) == PARAM_SLOT_PENDING_MAGIC &&
            (packed & PARAM_SLOT_PENDING_ACTIVATION) == 0u)
        {
            slot = (uint8_t) (packed & 0x0Fu);
            if (!slotValid(slot)) return PARAM_SLOT_IO_FAILED;
            store->stagedValid = 1u;
            store->stagedCommitted = 1u;
            store->stagedSlot = slot;
            store->stagedBank = (uint8_t) ((selector.bankMask >>
                (slot - 1u)) & 1u);
            store->stagedGeneration = selector.generation[slot - 1u];
            store->stagedBlobCrc = selector.blobCrc[slot - 1u];
            store->stagedTransactionId = selector.transactionId[slot - 1u];
        }
        return initializeRuntime(store, defaultBlob);
    }
}
memset(legacy, 0xFF, blobBytes);
if (readFlashBytes(0u, 0u, legacy, (blobBytes + 3u) & ~3u) == 0u)
    return PARAM_SLOT_IO_FAILED;
memset(&selector, 0, sizeof(selector));
if (validate(legacy, blobBytes) != 0u)
{
    selector.activeSlot = 1u;
    selector.validMask = 1u;
    selector.generation[0] = 1u;
    selector.blobCrc[0] = crc32IeeeCalc(legacy, blobBytes);
}
if (appendSelector(store, &selector) == 0u) return PARAM_SLOT_IO_FAILED;
store->initialized = 1u;
return initializeRuntime(store, defaultBlob);
}

paramSlotStatus paramSlotStoreGetMeta(const paramSlotStore *store,
uint8_t slot, paramSlotMeta *meta)
{
    const paramSlotSelector *selector;
    uint8_t index;
    if ((store == NULL) || (meta == NULL) || (store->initialized == 0u) ||
        (slotValid(slot) == 0u)) return PARAM_SLOT_BAD_ARGUMENT;
    selector = selectorConst(store);
    index = slot - 1u;
    memset(meta, 0, sizeof(*meta));
    meta->slot = slot;
    meta->valid = (uint8_t) ((selector->validMask >> index) & 1u);
    meta->active = 0u;
    meta->bank = (uint8_t) ((selector->bankMask >> index) & 1u);
    meta->generation = selector->generation[index];
    meta->blobCrc = selector->blobCrc[index];
    meta->transactionId = selector->transactionId[index];
}

```

```

    return meta->valid ? PARAM_SLOT_OK : PARAM_SLOT_EMPTY;
}

paramSlotStatus paramSlotStoreRead(const paramSlotStore *store,
uint8_t slot, void *blob, uint32_t blobBytes)
{
    paramSlotMeta meta;
    if ((blob == NULL) || (store == NULL) || (blobBytes != store->blobBytes))
        return PARAM_SLOT_BAD_ARGUMENT;
    if (paramSlotStoreGetMeta(store, slot, &meta) != PARAM_SLOT_OK) return PARAM_SLOT_EMPTY;
    if (readDataPage(store, dataPage(slot, meta.bank), blob) == 0u) return PARAM_SLOT_INVALID;
    if (crc32IeeeCalc((const uint8_t *) blob, blobBytes) != meta.blobCrc)
        return PARAM_SLOT_VERIFY_FAILED;
    return PARAM_SLOT_OK;
}

paramSlotStatus paramSlotStoreReadRuntime(const paramSlotStore *store,
void *blob, uint32_t blobBytes)
{
    paramRuntimeSelector committed;
    const paramRuntimeSelector *selector;
    if ((store == NULL) || (blob == NULL) ||
        (blobBytes != store->blobBytes) ||
        (store->runtimeInitialized == 0u)) return PARAM_SLOT_BAD_ARGUMENT;
    selector = runtimeSelectorConst(store);
    if (selector->pending == 0u)
        return readRuntimeWithSelector(store, selector, blob);
    if (selector->rollbackValid == 0u) return PARAM_SLOT_INVALID;
    committed = *selector;
    committed.bank = selector->rollbackBank;
    committed.valid = selector->rollbackValid;
    committed.pending = 0u;
    committed.sourceSlot = selector->rollbackSourceSlot;
    committed.generation = selector->rollbackGeneration;
    committed.blobCrc = selector->rollbackBlobCrc;
    committed.transactionId = selector->rollbackTransactionId;
    return readRuntimeWithSelector(store, &committed, blob);
}

paramSlotStatus paramSlotStoreReadPreparedRuntime(paramSlotStore *store,
uint32_t transactionId, void *blob, uint32_t blobBytes)
{
    const paramRuntimeSelector *selector;
    if ((store == NULL) || (blob == NULL) ||
        (blobBytes != store->blobBytes) || (transactionId == 0u))
        return PARAM_SLOT_BAD_ARGUMENT;
    if (resolveUncertainRuntimeSelector(store) == 0u)
        return PARAM_SLOT_IO_FAILED;
    selector = runtimeSelectorConst(store);
    if ((store->runtimeInitialized == 0u) || (selector->pending == 0u))
        return PARAM_SLOT_NO_PREPARE;
    if (selector->transactionId != transactionId)
        return PARAM_SLOT_TRANSACTION_MISMATCH;
    return readRuntimeWithSelector(store, selector, blob);
}

paramSlotStatus paramSlotStoreSaveRuntime(paramSlotStore *store,
const void *blob, uint32_t blobBytes)
{
    paramSlotStatus status;
    if ((store == NULL) || (blob == NULL) ||
        (blobBytes != store->blobBytes)) return PARAM_SLOT_BAD_ARGUMENT;
    status = prepareRuntimeBlob(store, blob, 0u, 0u);
    if (status == PARAM_SLOT_OK) status = commitRuntimeBlob(store, 0u, 0u);
    if (status == PARAM_SLOT_OK) clearRuntimeStage(store);
    else if (store->runtimeStagedCommitted == 0u) clearRuntimeStage(store);
    return status;
}

paramSlotStatus paramSlotStorePrepareRuntime(paramSlotStore *store,
const void *blob, uint32_t blobBytes, uint32_t transactionId,
uint8_t sourceSlot)
{
    if ((store == NULL) || (blob == NULL) ||
        (blobBytes != store->blobBytes) || (transactionId == 0u) ||
        (transactionId == PARAM_RUNTIME_TX_ACTIVATION_ONLY))
        return PARAM_SLOT_BAD_ARGUMENT;
    return prepareRuntimeBlob(store, blob, transactionId, sourceSlot);
}

```

```

paramSlotStatus paramSlotStorePrepareRuntimeFromSlot(paramSlotStore *store,
uint8_t slot, uint32_t transactionId)
{
    uint8_t blob[FLASH_PAGE_SIZE];
    paramSlotStatus status;
    if ((store == NULL) || (transactionId == 0u) ||
        (transactionId == PARAM_RUNTIME_TX_ACTIVATION_ONLY) ||
        (slotValid(slot) == 0u)) return PARAM_SLOT_BAD_ARGUMENT;
    status = paramSlotStoreRead(store, slot, blob, store->blobBytes);
    if (status != PARAM_SLOT_OK) return status;
    return prepareRuntimeBlob(store, blob, transactionId, slot);
}

paramSlotStatus paramSlotStoreCommitRuntime(paramSlotStore *store,
uint32_t transactionId)
{
    if ((store == NULL) || (transactionId == 0u) ||
        (transactionId == PARAM_RUNTIME_TX_ACTIVATION_ONLY))
        return PARAM_SLOT_BAD_ARGUMENT;
    return commitRuntimeBlob(store, transactionId, 1u);
}

paramSlotStatus paramSlotStoreGetRuntimeMeta(const paramSlotStore *store,
paramRuntimeMeta *meta)
{
    const paramRuntimeSelector *selector;
    if ((store == NULL) || (meta == NULL) ||
        (store->runtimeInitialized == 0u)) return PARAM_SLOT_BAD_ARGUMENT;
    selector = runtimeSelectorConst(store);
    memset(meta, 0, sizeof(*meta));
    meta->valid = selector->valid;
    meta->pending = selector->pending;
    if (selector->pending != 0u)
    {
        meta->valid = selector->rollbackValid;
        meta->bank = selector->rollbackBank;
        meta->sourceSlot = selector->rollbackSourceSlot;
        meta->generation = selector->rollbackGeneration;
        meta->blobCrc = selector->rollbackBlobCrc;
        meta->transactionId = selector->rollbackTransactionId;
    }
    else
    {
        meta->bank = selector->bank;
        meta->sourceSlot = selector->sourceSlot;
        meta->generation = selector->generation;
        meta->blobCrc = selector->blobCrc;
        meta->transactionId = selector->transactionId;
    }
    return selector->valid != 0u ? PARAM_SLOT_OK : PARAM_SLOT_EMPTY;
}

uint8_t paramSlotStoreRuntimeDefaultsRewritten(const paramSlotStore *store)
{
    return (uint8_t) ((store != NULL) &&
        (store->runtimeDefaultsRewritten != 0u));
}

paramSlotStatus paramSlotStorePrepare(paramSlotStore *store,
uint8_t slot, const void *blob, uint32_t blobBytes, uint32_t transactionId)
{
    paramSlotMeta meta;
    uint8_t bank = 0u;
    uint32_t generation = 1u;
    if ((store == NULL) || (store->initialized == 0u) || (blob == NULL) ||
        (blobBytes != store->blobBytes) ||
        (slotValid(slot) == 0u) || (transactionId == 0u) ||
        (transactionId == PARAM_RUNTIME_TX_ACTIVATION_ONLY))
        return PARAM_SLOT_BAD_ARGUMENT;
    if (store->stagedValid != 0u) return PARAM_SLOT_NO_PREPARE;
    if ((selectorConst(store)->reserved[2u] & PARAM_SLOT_PENDING_MAGIC_MASK) ==
        PARAM_SLOT_PENDING_MAGIC) return PARAM_SLOT_NO_PREPARE;
    if (paramSlotStoreGetMeta(store, slot, &meta) == PARAM_SLOT_OK)
    {
        bank = (uint8_t) (meta.bank ^ 1u);
        generation = meta.generation + 1u;
        if (generation == 0u) generation = 1u;
    }
}

```

```

    if (writeDataPage(store, dataPage(slot, bank), blob) == 0u)
        return PARAM_SLOT_VERIFY_FAILED;
    store->stagedValid = 1u;
    store->stagedSlot = slot;
    store->stagedBank = bank;
    store->stagedGeneration = generation;
    store->stagedBlobCrc = crc32IeeeCalc((const uint8_t *) blob, blobBytes);
    store->stagedTransactionId = transactionId;
    store->stagedCommitted = 0u;
    memcpy(store->rollbackSelectorBytes, store->selectorBytes,
           sizeof(store->rollbackSelectorBytes));
    return PARAM_SLOT_OK;
}

paramSlotStatus paramSlotStoreCommit(paramSlotStore *store,
uint8_t slot, uint32_t transactionId, uint8_t activate)
{
    paramSlotSelector next;
    uint8_t *candidate = (uint8_t *) (void *) flash_union_buffer;
    uint8_t mask;
    uint8_t index;
    (void) activate;
    if ((store == NULL) || (store->stagedValid == 0u)) return PARAM_SLOT_NO_PREPARE;
    if ((store->stagedSlot != slot) || (store->stagedTransactionId != transactionId))
        return PARAM_SLOT_TRANSACTION_MISMATCH;
    if ((store->stagedBank > 1u) || (store->stagedGeneration == 0u) ||
        (readDataPage(store, dataPage(slot, store->stagedBank), candidate) == 0u) ||
        (crc32IeeeCalc(candidate, store->blobBytes) != store->stagedBlobCrc))
        return PARAM_SLOT_VERIFY_FAILED;
    next = *selectorMutable(store);
    index = slot - 1u;
    mask = (uint8_t) (1u << index);
    next.validMask |= mask;
    if (store->stagedBank != 0u) next.bankMask |= mask;
    else next.bankMask &= (uint8_t) ~mask;
    next.generation[index] = store->stagedGeneration;
    next.blobCrc[index] = store->stagedBlobCrc;
    next.transactionId[index] = transactionId;
    if (next.activeSlot == 0u) next.activeSlot = slot;
    next.reserved[0u] = selectorConst(store)->transactionId[index];
    next.reserved[1u] = selectorConst(store)->generation[index];
    next.reserved[2u] = PARAM_SLOT_PENDING_MAGIC | (uint32_t) slot |
        ((uint32_t) ((selectorConst(store)->activeSlot != 0u) ?
            selectorConst(store)->activeSlot : slot) << 4) |
        ((uint32_t) ((selectorConst(store)->validMask >> index) & 1u) << 8) |
        ((uint32_t) ((selectorConst(store)->bankMask >> index) & 1u) << 9);
    if (appendSelector(store, &next) == 0u) return PARAM_SLOT_IO_FAILED;
    store->stagedCommitted = 1u;
    return PARAM_SLOT_OK;
}

paramSlotStatus paramSlotStoreSave(paramSlotStore *store,
uint8_t slot, const void *blob, uint32_t blobBytes,
uint32_t transactionId, uint8_t activate)
{
    paramSlotStatus status = paramSlotStorePrepare(store, slot, blob, blobBytes,
transactionId);
    if (status != PARAM_SLOT_OK) return status;
    status = paramSlotStoreCommit(store, slot, transactionId, activate);
    if (status == PARAM_SLOT_OK)
        status = paramSlotStoreFinish(store, transactionId);
    if (status != PARAM_SLOT_OK &&
        paramSlotStoreAbort(store, transactionId) != PARAM_SLOT_OK)
        return PARAM_SLOT_IO_FAILED;
    return status;
}

static paramSlotStatus activateLibrarySlot(paramSlotStore *store,
uint8_t slot, uint32_t transactionId, uint8_t pending)
{
    uint8_t blob[FLASH_PAGE_SIZE];
    paramSlotStatus status;
    const paramRuntimeSelector *runtime;
    if ((store == NULL) || (slotValid(slot) == 0u))
        return PARAM_SLOT_BAD_ARGUMENT;
    runtime = runtimeSelectorConst(store);
    if ((store->runtimeInitialized != 0u) && (runtime->pending != 0u))
    {
        if ((runtime->transactionId == transactionId) &&

```

```

        (runtime->sourceSlot == slot)) return PARAM_SLOT_OK;
    return PARAM_SLOT_NO_PREPARE;
}

status = paramSlotStoreRead(store, slot, blob, store->blobBytes);
if (status != PARAM_SLOT_OK) return status;
status = prepareRuntimeBlob(store, blob, transactionId, slot);
if (status == PARAM_SLOT_OK)
    status = commitRuntimeBlob(store, transactionId, pending);
if ((status == PARAM_SLOT_OK) && (pending == 0u)) clearRuntimeStage(store);
else if ((status != PARAM_SLOT_OK) &&
        (store->runtimeStagedCommitted == 0u)) clearRuntimeStage(store);
return status;
}

paramSlotStatus paramSlotStoreSetActive(paramSlotStore *store, uint8_t slot)
{
    if (store == NULL || store->stagedValid != 0u)
        return PARAM_SLOT_NO_PREPARE;
    if ((selectorConst(store)->reserved[2u] & PARAM_SLOT_PENDING_MAGIC_MASK) ==
        PARAM_SLOT_PENDING_MAGIC) return PARAM_SLOT_NO_PREPARE;
    return activateLibrarySlot(store, slot, 0u, 0u);
}

paramSlotStatus paramSlotStoreActivatePrepared(paramSlotStore *store,
uint8_t slot, uint32_t transactionId)
{
    paramSlotMeta meta;
    if (store == NULL || transactionId == 0u ||
        transactionId == PARAM_RUNTIME_TX_ACTIVATION_ONLY ||
        store->stagedValid == 0u || store->stagedCommitted == 0u)
        return PARAM_SLOT_NO_PREPARE;
    if (store->stagedSlot != slot ||
        store->stagedTransactionId != transactionId)
        return PARAM_SLOT_TRANSACTION_MISMATCH;
    if ((selectorConst(store)->reserved[2u] & PARAM_SLOT_PENDING_MAGIC_MASK) !=
        PARAM_SLOT_PENDING_MAGIC)
        return PARAM_SLOT_NO_PREPARE;
    if (paramSlotStoreGetMeta(store, slot, &meta) != PARAM_SLOT_OK ||
        meta.transactionId != transactionId)
        return PARAM_SLOT_TRANSACTION_MISMATCH;
    return activateLibrarySlot(store, slot, transactionId, 1u);
}

paramSlotStatus paramSlotStoreSetActivePending(paramSlotStore *store, uint8_t slot)
{
    if (store == NULL || store->stagedValid != 0u) return PARAM_SLOT_NO_PREPARE;
    if ((selectorConst(store)->reserved[2u] & PARAM_SLOT_PENDING_MAGIC_MASK) ==
        PARAM_SLOT_PENDING_MAGIC) return PARAM_SLOT_NO_PREPARE;
    return activateLibrarySlot(store, slot, PARAM_RUNTIME_TX_ACTIVATION_ONLY, 1u);
}

paramSlotStatus paramSlotStoreAbortPendingActivation(paramSlotStore *store)
{
    uint32_t packed;
    paramSlotStatus runtimeStatus;
    if (store == NULL) return PARAM_SLOT_BAD_ARGUMENT;
    if (resolveUncertainSelector(store) == 0u)
        return PARAM_SLOT_IO_FAILED;
    packed = store->initialized != 0u ?
        selectorConst(store)->reserved[2u] : 0u;
    if ((packed & PARAM_SLOT_PENDING_MAGIC_MASK) == PARAM_SLOT_PENDING_MAGIC)
    {
        if ((packed & PARAM_SLOT_PENDING_ACTIVATION) == 0u)
            return PARAM_SLOT_TRANSACTION_MISMATCH;
        if (rollbackPendingSelector(store) == 0u)
            return PARAM_SLOT_IO_FAILED;
        store->stagedValid = 0u;
        store->stagedCommitted = 0u;
    }
    runtimeStatus = abortRuntimeTransaction(store,
        PARAM_RUNTIME_TX_ACTIVATION_ONLY);
    return runtimeStatus;
}

paramSlotStatus paramSlotStoreFinishPendingActivation(paramSlotStore *store)
{
    uint32_t packed;
    paramSlotStatus libraryStatus;
    paramSlotStatus runtimeStatus;

```

```

if (store == NULL) return PARAM_SLOT_BAD_ARGUMENT;
if (resolveUncertainSelector(store) == 0u)
    return PARAM_SLOT_IO_FAILED;
libraryStatus = verifyPendingLibraryCandidate(store,
    PARAM_RUNTIME_TX_ACTIVATION_ONLY);
if (libraryStatus != PARAM_SLOT_OK) return libraryStatus;
packed = store->initialized != 0u ?
    selectorConst(store)->reserved[2u] : 0u;
if ((packed & PARAM_SLOT_PENDING_MAGIC_MASK) == PARAM_SLOT_PENDING_MAGIC)
{
    paramSlotSelector next = *selectorMutable(store);
    const paramRuntimeSelector *runtime = runtimeSelectorConst(store);
    if ((packed & PARAM_SLOT_PENDING_ACTIVATION) == 0u ||
        (store->runtimeInitialized == 0u) ||
        (runtime->pending == 0u) ||
        (runtime->transactionId != PARAM_RUNTIME_TX_ACTIVATION_ONLY))
        return PARAM_SLOT_NO_PREPARE;
    memset(next.reserved, 0, sizeof(next.reserved));
    if (appendSelector(store, &next) == 0u)
        return PARAM_SLOT_IO_FAILED;
    store->stagedValid = 0u;
    store->stagedCommitted = 0u;
}
runtimeStatus = finishRuntimeTransaction(store,
    PARAM_RUNTIME_TX_ACTIVATION_ONLY);
return runtimeStatus;
}

paramSlotStatus paramSlotStoreAbort(paramSlotStore *store, uint32_t transactionId)
{
    paramSlotStatus runtimeStatus;
    if ((store == NULL) ||
        (transactionId == PARAM_RUNTIME_TX_ACTIVATION_ONLY))
        return PARAM_SLOT_BAD_ARGUMENT;
    runtimeStatus = abortRuntimeTransaction(store, transactionId);
    if (runtimeStatus != PARAM_SLOT_OK) return runtimeStatus;
    if (resolveUncertainSelector(store) == 0u)
        return PARAM_SLOT_IO_FAILED;
    if (store->stagedValid == 0u)
        return PARAM_SLOT_OK;
    if ((transactionId != 0u) && (store->stagedTransactionId != transactionId))
        return PARAM_SLOT_TRANSACTION_MISMATCH;
    if ((selectorConst(store)->reserved[2u] & PARAM_SLOT_PENDING_MAGIC_MASK) ==
        PARAM_SLOT_PENDING_MAGIC)
    {
        if (rollbackPendingSelector(store) == 0u) return PARAM_SLOT_IO_FAILED;
    }
    store->stagedValid = 0u;
    store->stagedCommitted = 0u;
    return PARAM_SLOT_OK;
}

paramSlotStatus paramSlotStoreFinish(paramSlotStore *store, uint32_t transactionId)
{
    paramSlotStatus runtimeStatus;
    paramSlotStatus libraryStatus;
    if ((store == NULL) ||
        (transactionId == PARAM_RUNTIME_TX_ACTIVATION_ONLY))
        return PARAM_SLOT_BAD_ARGUMENT;
    if (resolveUncertainSelector(store) == 0u)
        return PARAM_SLOT_IO_FAILED;
    if ((store->stagedValid != 0u) &&
        (store->stagedTransactionId != transactionId))
        return PARAM_SLOT_TRANSACTION_MISMATCH;
    libraryStatus = verifyPendingLibraryCandidate(store, transactionId);
    if (libraryStatus != PARAM_SLOT_OK) return libraryStatus;
    {
        paramSlotSelector next = *selectorMutable(store);
        if ((next.reserved[2u] & PARAM_SLOT_PENDING_MAGIC_MASK) ==
            PARAM_SLOT_PENDING_MAGIC)
        {
            memset(next.reserved, 0, sizeof(next.reserved));
            if (appendSelector(store, &next) == 0u) return PARAM_SLOT_IO_FAILED;
        }
        store->stagedValid = 0u;
        store->stagedCommitted = 0u;
    }
    runtimeStatus = finishRuntimeTransaction(store, transactionId);
    return runtimeStatus;
}

```

```

}

uint8_t paramSlotStoreGetActive(const paramSlotStore *store)
{
    (void) store;
    return 0u;
}

paramSlotStatus paramSlotStoreGetPending(const paramSlotStore *store,
    paramSlotPendingInfo *pending)
{
    const paramSlotSelector *selector;
    uint32_t packed;
    uint8_t slot;
    if (store == NULL || pending == NULL ||
        ((store->initialized == 0u) && (store->runtimeInitialized == 0u)))
        return PARAM_SLOT_BAD_ARGUMENT;
    memset(pending, 0, sizeof(*pending));
    selector = selectorConst(store);
    packed = store->initialized != 0u ? selector->reserved[2u] : 0u;
    if ((packed & PARAM_SLOT_PENDING_MAGIC_MASK) != PARAM_SLOT_PENDING_MAGIC)
    {
        const paramRuntimeSelector *runtime = runtimeSelectorConst(store);
        if ((store->runtimeInitialized == 0u) || (runtime->pending == 0u))
            return PARAM_SLOT_EMPTY;
        if ((runtime->sourceSlot != 0u) &&
            (slotValid(runtime->sourceSlot) == 0u)) return PARAM_SLOT_INVALID;
        pending->valid = 1u;
        pending->slot = runtime->sourceSlot;
        pending->active = 1u;
        pending->activationOnly = (uint8_t) (
            runtime->transactionId == PARAM_RUNTIME_TX_ACTIVATION_ONLY);
        pending->transactionId = pending->activationOnly ? 0u :
            runtime->transactionId;
        return PARAM_SLOT_OK;
    }
    slot = (uint8_t) (packed & 0x0Fu);
    if (!slotValid(slot)) return PARAM_SLOT_INVALID;
    pending->valid = 1u;
    pending->slot = slot;
    pending->active = (uint8_t) ((store->runtimeInitialized != 0u) &&
        (runtimeSelectorConst(store)->pending != 0u) &&
        (runtimeSelectorConst(store)->sourceSlot == slot));
    pending->activationOnly = (uint8_t) ((packed &
        PARAM_SLOT_PENDING_ACTIVATION) != 0u);
    pending->transactionId = pending->activationOnly ? 0u :
        selector->transactionId[slot - 1u];
    return PARAM_SLOT_OK;
}

```

core0_cyt4bb7/project/code/util/paramSlotStore.h

SHA-256

:

8b3b702c892d2775d93d55a43df6484346602ecc537bfb1b839c699820f4f472

代码 (c)

```

#ifndef PARAM_SLOT_STORE_H
#define PARAM_SLOT_STORE_H

#include <stdint.h>

#define PARAM_SLOT_COUNT (5u)
#define PARAM_SLOT_FIRST (1u)
#define PARAM_SLOT_LAST (5u)

typedef enum
{
    PARAM_SLOT_OK = 0,
    PARAM_SLOT_BAD_ARGUMENT,
    PARAM_SLOT_EMPTY,
    PARAM_SLOT_INVALID,
    PARAM_SLOT_IO_FAILED,
    PARAM_SLOT_VERIFY_FAILED,
    PARAM_SLOT_NO_PREPARE,

```



```

    PARAM_SLOT_TRANSACTION_MISMATCH
} paramSlotStatus;

typedef uint8_t (*paramSlotBlobValidator)(const void *blob, uint32_t blobBytes);

typedef struct
{
    uint8_t slot;
    uint8_t valid;
    uint8_t active;
    uint8_t bank;
    uint32_t generation;
    uint32_t blobCrc;
    uint32_t transactionId;
} paramSlotMeta;

typedef struct
{
    uint32_t transactionId;
    uint8_t slot;
    uint8_t valid;
    uint8_t activationOnly;
    uint8_t active;
} paramSlotPendingInfo;

typedef struct
{
    uint8_t valid;
    uint8_t bank;
    uint8_t pending;
    uint8_t sourceSlot;
    uint32_t generation;
    uint32_t blobCrc;
    uint32_t transactionId;
} paramRuntimeMeta;

typedef struct
{
    uint32_t blobBytes;
    paramSlotBlobValidator validate;
    uint8_t initialized;
    uint8_t stagedValid;
    uint8_t stagedCommitted;
    uint8_t selectorUncertain;
    uint8_t runtimeInitialized;
    uint8_t runtimeStagedValid;
    uint8_t runtimeStagedCommitted;
    uint8_t runtimeSelectorUncertain;
    uint8_t runtimeDefaultsRewritten;
    uint8_t stagedSlot;
    uint8_t stagedBank;
    uint8_t runtimeStagedBank;
    uint8_t runtimeStagedSourceSlot;
    uint32_t stagedGeneration;
    uint32_t stagedBlobCrc;
    uint32_t stagedTransactionId;
    uint32_t runtimeStagedGeneration;
    uint32_t runtimeStagedBlobCrc;
    uint32_t runtimeStagedTransactionId;
    uint8_t rollbackSelectorBytes[96u];
    uint8_t selectorBytes[96u];
    uint8_t runtimeSelectorBytes[64u];
} paramSlotStore;

paramSlotStatus paramSlotStoreInit(paramSlotStore *store,
    uint32_t blobBytes,
    paramSlotBlobValidator validate,
    const void *defaultBlob);
paramSlotStatus paramSlotStoreGetMeta(const paramSlotStore *store,
    uint8_t slot,
    paramSlotMeta *meta);
paramSlotStatus paramSlotStoreRead(const paramSlotStore *store,
    uint8_t slot,
    void *blob,
    uint32_t blobBytes);
paramSlotStatus paramSlotStoreReadRuntime(const paramSlotStore *store,
    void *blob,
    uint32_t blobBytes);
paramSlotStatus paramSlotStoreReadPreparedRuntime(paramSlotStore *store,

```

```

        uint32_t transactionId,
        void *blob,
        uint32_t blobBytes);
paramSlotStatus paramSlotStoreSaveRuntime(paramSlotStore *store,
        const void *blob,
        uint32_t blobBytes);
paramSlotStatus paramSlotStorePrepareRuntime(paramSlotStore *store,
        const void *blob,
        uint32_t blobBytes,
        uint32_t transactionId,
        uint8_t sourceSlot);
paramSlotStatus paramSlotStorePrepareRuntimeFromSlot(paramSlotStore *store,
        uint8_t slot,
        uint32_t transactionId);
paramSlotStatus paramSlotStoreCommitRuntime(paramSlotStore *store,
        uint32_t transactionId);
paramSlotStatus paramSlotStoreGetRuntimeMeta(const paramSlotStore *store,
        paramRuntimeMeta *meta);
uint8_t paramSlotStoreRuntimeDefaultsRewritten(const paramSlotStore *store);
paramSlotStatus paramSlotStorePrepare(paramSlotStore *store,
        uint8_t slot,
        const void *blob,
        uint32_t blobBytes,
        uint32_t transactionId);
paramSlotStatus paramSlotStoreCommit(paramSlotStore *store,
        uint8_t slot,
        uint32_t transactionId,
        uint8_t activate);
paramSlotStatus paramSlotStoreSave(paramSlotStore *store,
        uint8_t slot,
        const void *blob,
        uint32_t blobBytes,
        uint32_t transactionId,
        uint8_t activate);
paramSlotStatus paramSlotStoreSetActive(paramSlotStore *store, uint8_t slot);
paramSlotStatus paramSlotStoreActivatePrepared(paramSlotStore *store,
        uint8_t slot, uint32_t transactionId);
paramSlotStatus paramSlotStoreSetActivePending(paramSlotStore *store, uint8_t slot);
paramSlotStatus paramSlotStoreAbortPendingActivation(paramSlotStore *store);
paramSlotStatus paramSlotStoreFinishPendingActivation(paramSlotStore *store);
paramSlotStatus paramSlotStoreAbort(paramSlotStore *store, uint32_t transactionId);
paramSlotStatus paramSlotStoreFinish(paramSlotStore *store, uint32_t transactionId);
paramSlotStatus paramSlotStoreGetPending(const paramSlotStore *store,
        paramSlotPendingInfo *pending);
uint8_t paramSlotStoreGetActive(const paramSlotStore *store);

#endif

```

core0_cyt4bb7/project/code/util/schmitt.c

SHA-256

:

039ffb8a50027ef0bf6ed2f5f970b97818e2d800c13b77aa0a98ed98caef7eca

代码 (c)

```

/*
 * schmitt.c
 *
 * Created: 20 250 718
 * Author: Hecate
 */
#include "schmitt.h"

void schmittSetThresholds(SchmittTrigger *obj, float upperThresh, float lowerThresh)
{
    if (upperThresh < lowerThresh)
    {
        // 交换阈值
        obj->upperThreshold = lowerThresh;
        obj->lowerThreshold = upperThresh;
    }
    else
    {
        obj->upperThreshold = upperThresh;
        obj->lowerThreshold = lowerThresh;
    }
}

```

```
    }  
}  
  
/* 去掉外部库原 inline 限定：C99 下 .c 内裸 inline 不保证产生外部符号，  
* 跨 TU 调用会链接期 undefined；普通函数 IAR 自会按需内联，无性能损失。*/  
void schmittUpdate(SchmittTrigger *obj, float inputValue)  
{  
    const float val = (inputValue);  
    (obj)->outputState = (obj)->outputState  
        ? ((val <= (obj)->lowerThreshold) ? 0u : 1u)  
        : ((val >= (obj)->upperThreshold) ? 1u : 0u);  
}
```

core0_cyt4bb7/project/code/util/schmitt.h

SHA-256

:

577a6b3e13f18ff01cb3f547fef2770bd9d208db550036227fe6b035c936e7c1

代码 (c)

```
/*  
 * schmitt.h  
 *  
 * Created: 20 250 718  
 * Author: Hecate  
 */  
#ifndef CODE_SCHMITT_H_  
#define CODE_SCHMITT_H_  
  
#include <stdint.h>  
  
// 施密特触发器对象  
typedef struct  
{  
    float upperThreshold;  
    float lowerThreshold;  
    uint8_t outputState;  
} SchmittTrigger;  
  
// 重新设置上下阈值  
void schmittSetThresholds(SchmittTrigger *obj, float upperThresh, float lowerThresh);  
  
// 输入新采样值并更新输出。迟滞语义（边界含等号）：  
// outputState=1（高）时，输入 <= lowerThreshold 才转 0；  
// outputState=0（低）时，输入 >= upperThreshold 才转 1。  
void schmittUpdate(SchmittTrigger *obj, float inputValue);  
  
#endif /* CODE_SCHMITT_H_ */
```

core0_cyt4bb7/project/code/vision/image.c

SHA-256

:

75903fc9bcc1a751601e307ce07c4d141b0a7beb0d05feb8bdf38e84f30e19a3

代码 (c)

```
#include "image.h"  
  
#include <math.h>  
#include <string.h>  
  
#include "cy_device_headers.h"  
#include "zf_device_mt9v03x.h"  
#include "zf_driver_timer.h"  
  
#include "openMorph.h"  
#include "om_binarycc.h"  
#include "configVision.h"  
  
#define IMAGE_TIMER_WRAP_US (536870912u) /* 32 位 8MHz TCPWM 经 /8 后的微秒回绕周期 */
```

```

#define IMAGE_CAMERA_INT_VALID_MASK (1u << 31)
/* 相机 ISR 每帧整帧覆写 SDK 公共缓冲 mt9v03x_image；高帧率下其 memcpy 会与视觉管线、
 * LCD 刷屏并发，拷到半新半旧的撕裂帧。视觉与显示统一只读本地一致性快照。*/
static uint8_t imageFrameBuffer[MT9V03X_IMAGE_SIZE];
static u8Image mt9v03xOriImg = {imageFrameBuffer, MT9V03X_W, MT9V03X_H, MT9V03X_IMAGE_SIZE};

static uint8_t gaussianBlurImgBuffer[MT9V03X_IMAGE_SIZE];
static u8Image gaussianBlurImg = {gaussianBlurImgBuffer, MT9V03X_W, MT9V03X_H, MT9V03X_IMAGE_SIZE};

static uint8_t cannyEdgeImgBuffer[MT9V03X_IMAGE_SIZE];
static u8Image cannyEdgeImg = {cannyEdgeImgBuffer, MT9V03X_W, MT9V03X_H, MT9V03X_IMAGE_SIZE};

/* 当前生效的 Hu 模板与标定收紧的尺寸范围（标定成功后用实测值覆盖）。
 * Legacy 初值为几何模板；BinaryCC Hu 模板源自历史 exposure=101、low=230/high=255 素材
 * 当前 v35 阈值默认见 configVision.h（low=160/high=255）
 * 实机 PGM 亮核心域仿真模板。
 * 菜单调 Canny/曝光等触发的 imageApplyContourTuning 不得回退此模板，
 * 否则标定结果会被默认模板冲掉（P2 修复）。*/
static const float legacyDefaultHu[7] = {
    9.2866892706e-01f, 8.3634088034e-02f, 5.2070075699e-01f, 3.6007068223e-02f,
    -4.9303272544e-03f, -1.0413085262e-02f, 0.0000000000e+00f};
static const float binaryCcDefaultHu[7] = {
    3.316533319847e-01f, 4.568476825756e-02f, 1.961961823615e-02f, 2.277107169141e-03f,
    -5.942349246482e-06f, -3.229190620859e-04f, 9.155863456097e-06f};
typedef struct
{
    CONTOUR_DETECTOR_STRUCT detector;
    uint32_t frameSeq;
    CalibState calibState;
    uint16_t calibSize;
    uint8_t calibrationSavePending;
    float contourHuLost;
    uint8_t detectMode;
    float legacyActiveHu[7];
    float binaryCcActiveHu[7];
    uint16_t legacyActiveSizeMin;
    uint16_t legacyActiveSizeMax;
    uint16_t binaryCcActiveSizeMin;
    uint16_t binaryCcActiveSizeMax;
    uint8_t legacyCalibrated;
    uint8_t binaryCcCalibrated;
    BINARYCC_CONFIG_STRUCT binaryCcActiveConfig;
    uint32_t lastFrameTimestampUs;
    uint8_t frameTimestampPrimed;
} imageContextStruct;

static imageContextStruct imageContext = {
    .calibState = CALIB_WAITING,
    .contourHuLost = VISION_CONTOUR_HU_LOST_DEFAULT,
    .detectMode = VISION_DETECT_MODE_DEFAULT,
    .legacyActiveHu = {
        9.2866892706e-01f, 8.3634088034e-02f, 5.2070075699e-01f, 3.6007068223e-02f,
        -4.9303272544e-03f, -1.0413085262e-02f, 0.0000000000e+00f},
    .binaryCcActiveHu = {
        3.316533319847e-01f, 4.568476825756e-02f, 1.961961823615e-02f, 2.277107169141e-03f,
        -5.942349246482e-06f, -3.229190620859e-04f, 9.155863456097e-06f},
    .legacyActiveSizeMin = VISION_CONTOUR_SIZE_MIN_DEFAULT,
    .legacyActiveSizeMax = VISION_CONTOUR_SIZE_MAX_DEFAULT,
    .binaryCcActiveSizeMin = VISION_BINARYCC_AREA_MIN_DEFAULT,
    .binaryCcActiveSizeMax = VISION_BINARYCC_AREA_MAX_DEFAULT,
    .binaryCcActiveConfig = {
        VISION_BINARYCC_LOW_DEFAULT, VISION_BINARYCC_HIGH_DEFAULT,
        VISION_BINARYCC_AREA_MIN_DEFAULT, VISION_BINARYCC_AREA_MAX_DEFAULT,
        VISION_BINARYCC_BRIGHT_MIN_DEFAULT, VISION_BINARYCC_ECC_MAX_DEFAULT,
        VISION_BINARYCC_FILL_MIN_DEFAULT, VISION_BINARYCC_HU_MAX_DEFAULT,
        VISION_BINARYCC_ORI_BRIGHT_MIN_DEFAULT, VISION_BINARYCC_ORI_BORDER_MARGIN_DEFAULT,
        VISION_BINARYCC_ORI_CONF_MIN_DEFAULT, VISION_BINARYCC_ORI_HOLD_FRAMES_DEFAULT,
        VISION_TARGET_CENTER_X, VISION_TARGET_CENTER_Y}
};

#define chevronDetector      imageContext.detector
#define imageFrameSeq        imageContext.frameSeq
#define chevronCalibState    imageContext.calibState
#define chevronCalibSize     imageContext.calibSize
#define calibrationSavePending imageContext.calibrationSavePending
#define contourHuLost        imageContext.contourHuLost
#define imageDetectMode      imageContext.detectMode
#define legacyActiveHu        imageContext.legacyActiveHu
#define binaryCcActiveHu      imageContext.binaryCcActiveHu

```

```

#define legacyActiveSizeMin  imageContext.legacyActiveSizeMin
#define legacyActiveSizeMax  imageContext.legacyActiveSizeMax
#define binaryCcActiveSizeMin imageContext.binaryCcActiveSizeMin
#define binaryCcActiveSizeMax imageContext.binaryCcActiveSizeMax
#define legacyCalibrated     imageContext.legacyCalibrated
#define binaryCcCalibrated   imageContext.binaryCcCalibrated
#define binaryCcActiveConfig imageContext.binaryCcActiveConfig
#define lastFrameTimestampUs imageContext.lastFrameTimestampUs
#define frameTimestampPrimed imageContext.frameTimestampPrimed

static volatile uint32_t* imageCameraSystemIntControl(void)
{
    #if defined(CY_CORE_CM7_0) && (CY_CORE_CM7_0 != 0)
        return &CPUSS->unCM7_0_SYSTEM_INT_CTL[tc_pwm_0_interrupts_59_IRQn].u32Register;
    #else
        return &CPUSS->unCM7_1_SYSTEM_INT_CTL[tc_pwm_0_interrupts_59_IRQn].u32Register;
    #endif
}

static uint8_t imageNormalizeDetectMode(uint8_t mode)
{
    return (mode == VISION_DETECT_MODE_LEGACY_CANNY) ? VISION_DETECT_MODE_LEGACY_CANNY : VISION_DETECT_MODE_BINARYCC;
}

static void imageClampTemplateRange(uint16_t* sizeMin, uint16_t* sizeMax)
{
    if (*sizeMin < 1u) *sizeMin = 1u;
    else if (*sizeMin > 59999u) *sizeMin = 59999u;
    if (*sizeMax > 60000u) *sizeMax = 60000u;
    if (*sizeMax <= *sizeMin) *sizeMax = (uint16_t) (*sizeMin + 1u);
}

static void imageApplyLegacyTemplate(void)
{
    contourDetectSetSizeRange(legacyActiveSizeMin, legacyActiveSizeMax);
    contourDetectSetTemplate(legacyActiveHu, contourHuLost);
}

static void imageApplyBinaryTemplate(void)
{
    BINARYCC_CONFIG_STRUCT cfg = binaryCcActiveConfig;
    if (binaryCcCalibrated)
    {
        cfg.areaMin = binaryCcActiveSizeMin;
        cfg.areaMax = binaryCcActiveSizeMax;
    }
    binaryCcSetConfig(&cfg);
    binaryCcSetTemplate(binaryCcActiveHu, cfg.huMax);
}

static uint8_t imageTakeFrameSnapshot(float* sampleDtS)
{
    volatile uint32_t* intControl;
    uint32_t savedIntControl;

    if (!mt9v03x_finish_flag) return 0u;

    /* CPUIntIdx3 还承载 UART，只屏蔽 MT9V03X 的 TCPWM59 系统中断源。*/
    intControl = imageCameraSystemIntControl();
    savedIntControl = *intControl;
    *intControl = savedIntControl & ~IMAGE_CAMERA_INT_VALID_MASK;
    (void) *intControl;
    __DSB();
    if (mt9v03x_finish_flag != 0u)
    {
        mt9v03x_finish_flag = 0u;
        /* 相机物理旋转 180°：整帧线性反转 = 水平+垂直翻转 = 180° 旋转。
         * 在快照处统一矫正朝向，下游检测/显示/闭环极性全部无需改动。*/
        const uint8_t* src = mt9v03x_image[0];
        for (uint32_t i = 0u; i < MT9V03X_IMAGE_SIZE; i++)
        {
            imageFrameBuffer[i] = src[MT9V03X_IMAGE_SIZE - 1u - i];
        }
    }
    else
    {
        if ((savedIntControl & IMAGE_CAMERA_INT_VALID_MASK) != 0u)
        {
            *intControl = savedIntControl;
        }
    }
}

```

```

        (void) *intControl;
        __DSB();
    }
    return 0u;
}
if ((savedIntControl & IMAGE_CAMERA_INT_VALID_MASK) != 0u)
{
    *intControl = savedIntControl;
    (void) *intControl;
    __DSB();
}

uint32_t nowUs = timer_get(TC_TIME2_CH0);
if (!frameTimestampPrimed)
{
    *sampleDts = 0.0f;
    frameTimestampPrimed = 1u;
}
else
{
    uint32_t elapsedUs = (nowUs >= lastFrameTimestampUs)
        ? (nowUs - lastFrameTimestampUs)
        : ((IMAGE_TIMER_WRAP_US - lastFrameTimestampUs) + nowUs);
    *sampleDts = (float) elapsedUs * 0.000001f;
}
lastFrameTimestampUs = nowUs;
return 1u;
}

void imageApplyContourTuning(uint16_t sizeMin, uint16_t sizeMax, float huLost, uint8_t brightnessMax)
{
    contourHuLost = huLost;
    /* 未标定时尺寸范围跟随菜单；已标定时保持标定收紧值（菜单不再覆盖尺寸） */
    if (!legacyCalibrated)
    {
        legacyActiveSizeMin = sizeMin;
        legacyActiveSizeMax = sizeMax;
    }
    contourDetectSetSizeRange(legacyActiveSizeMin, legacyActiveSizeMax);
    /* 始终用当前生效模板（默认或标定后），不回退硬编码默认 */
    contourDetectSetTemplate(legacyActiveHu, huLost);
    contourDetectSetBrightnessMax(brightnessMax);
}

void imageApplyBinaryCcTuning(const imageBinaryCcTuningStruct* config)
{
    BINARYCC_CONFIG_STRUCT next;
    if (config == 0) return;
    next.thresholdLow = config->thresholdLow;
    next.thresholdHigh = config->thresholdHigh;
    next.areaMin = config->areaMin;
    next.areaMax = config->areaMax;
    next.brightnessMin = config->brightnessMin;
    next.eccMax = config->eccMax;
    next.fillMin = config->fillMin;
    next.huMax = config->huMax;
    next.orientationBrightMin = config->orientationBrightMin;
    next.orientationBorderMargin = config->orientationBorderMargin;
    next.orientationConfidenceMin = config->orientationConfidenceMin;
    next.orientationHoldFrames = config->orientationHoldFrames;
    next.referenceCenterX = config->referenceCenterX;
    next.referenceCenterY = config->referenceCenterY;
    binaryCcActiveConfig = next;
    if (!binaryCcCalibrated)
    {
        binaryCcActiveSizeMin = config->areaMin;
        binaryCcActiveSizeMax = config->areaMax;
    }
    imageApplyBinaryTemplate();
}

void imageApplyCannyThresholds(uint16_t thresholdLow, uint16_t thresholdHigh)
{
    cannySetThreshold(thresholdLow, thresholdHigh);
}

void imageGetOrientationProtectDefaults(imageOrientationProtectTuningStruct* out)
{
    if (out == 0) return;

```

```

        out->jumpMax = OM_ORIENTATION_JUMP_MAX;
        out->recoverRad = OM_ORIENTATION_RECOVER_RAD;
        out->confirmFrames = OM_ORIENTATION_CONFIRM_FRAMES;
    }

void imageApplyOrientationProtect(const imageOrientationProtectTuningStruct* config)
{
    if (config == 0) return;
    contourDetectSetOrientProtect(config->jumpMax, config->recoverRad, config->confirmFrames);
}

void imageSetDetectMode(uint8_t mode)
{
    imageDetectMode = imageNormalizeDetectMode(mode);
    if (imageDetectMode == VISION_DETECT_MODE_LEGACY_CANNY)
        imageApplyLegacyTemplate();
    else
        imageApplyBinaryTemplate();
}

uint8_t imageGetDetectMode(void)
{
    return imageDetectMode;
}

/* 标定/flash 恢复：写入实测 Hu 模板 + 收紧尺寸范围，并即时下发生效 */
void imageSetActiveTemplateForMode(uint8_t mode, const float hu[7], uint16_t sizeMin, uint16_t sizeMax)
{
    if (0 == hu) return;
    /* 入口钳位：防未来 blob 字段变更/手工刷写传入非法范围(min>=max 或 0),
     * 与菜单侧 applyVisionLocal 同口径(min∈[1,59999], max∈[min+1,60000])。 */
    imageClampTemplateRange(&sizeMin, &sizeMax);
    if (imageNormalizeDetectMode(mode) == VISION_DETECT_MODE_LEGACY_CANNY)
    {
        for (uint8_t i = 0u; i < 7u; i++) legacyActiveHu[i] = hu[i];
        legacyActiveSizeMin = sizeMin;
        legacyActiveSizeMax = sizeMax;
        legacyCalibrated = 1u;
        if (imageDetectMode == VISION_DETECT_MODE_LEGACY_CANNY) imageApplyLegacyTemplate();
    }
    else
    {
        for (uint8_t i = 0u; i < 7u; i++) binaryCcActiveHu[i] = hu[i];
        binaryCcActiveSizeMin = sizeMin;
        binaryCcActiveSizeMax = sizeMax;
        binaryCcCalibrated = 1u;
        if (imageDetectMode == VISION_DETECT_MODE_BINARYCC) imageApplyBinaryTemplate();
    }
}

/* 供 flash 持久化读取当前模板（返回 1=已标定值，0=出厂默认） */
uint8_t imageGetActiveTemplateForMode(uint8_t mode, float huOut[7], uint16_t* sizeMinOut, uint16_t* sizeMaxOut)
{
    uint8_t legacy = (imageNormalizeDetectMode(mode) == VISION_DETECT_MODE_LEGACY_CANNY) ? 1u : 0u;
    if (huOut != 0)
    {
        for (uint8_t i = 0u; i < 7u; i++) huOut[i] = legacy ? legacyActiveHu[i] : binaryCcActiveHu[i];
    }
    if (sizeMinOut != 0) *sizeMinOut = legacy ? legacyActiveSizeMin : binaryCcActiveSizeMin;
    if (sizeMaxOut != 0) *sizeMaxOut = legacy ? legacyActiveSizeMax : binaryCcActiveSizeMax;
    return legacy ? legacyCalibrated : binaryCcCalibrated;
}

/* 清空恢复出厂默认：丢弃两套 RAM 标定模板，回退默认模板 + 默认尺寸范围，并即时下发生效。 */
void imageResetActiveTemplate(void)
{
    for (uint8_t i = 0u; i < 7u; i++)
    {
        legacyActiveHu[i] = legacyDefaultHu[i];
        binaryCcActiveHu[i] = binaryCcDefaultHu[i];
    }
    legacyActiveSizeMin = VISION_CONTOUR_SIZE_MIN_DEFAULT;
    legacyActiveSizeMax = VISION_CONTOUR_SIZE_MAX_DEFAULT;
    binaryCcActiveSizeMin = VISION_BINARYCC_AREA_MIN_DEFAULT;
    binaryCcActiveSizeMax = VISION_BINARYCC_AREA_MAX_DEFAULT;
    legacyCalibrated = 0u;
    binaryCcCalibrated = 0u;
    calibrationSavePending = 0u;
    imageApplyLegacyTemplate();
}

```

```

        imageApplyBinaryTemplate();
    }

/* ---- 稳定性检测参数 ---- */
#define STABLE_WINDOW    15u /* 连续稳定帧数 */
#define STABLE_POS_THRESH 3u /* 坐标极差阈值（像素） */
#define STABLE_ANG_THRESH 0.15f /* 朝向角极差阈值（弧度 ≈ 8.6°） */

/* 环形缓冲区 */
static uint16_t stabBufX[STABLE_WINDOW];
static uint16_t stabBufY[STABLE_WINDOW];
static float stabBufAng[STABLE_WINDOW];
static uint8_t stabBufIdx = 0u;
static uint8_t stabBufFull = 0u;

/* 圆周角差（处理 -PI/PI 环绕） */
static inline float angleDiff(float a, float b)
{
    float d = a - b;
    if (d > PI) d -= 2.0f * PI;
    else if (d < -PI) d += 2.0f * PI;
    return fabsf(d);
}

/* 检查环形缓冲区内数据是否稳定 */
static uint8_t checkStability(void)
{
    uint16_t xMin = stabBufX[0], xMax = stabBufX[0];
    uint16_t yMin = stabBufY[0], yMax = stabBufY[0];
    for (uint8_t i = 1u; i < STABLE_WINDOW; i++)
    {
        if (stabBufX[i] < xMin) xMin = stabBufX[i];
        if (stabBufX[i] > xMax) xMax = stabBufX[i];
        if (stabBufY[i] < yMin) yMin = stabBufY[i];
        if (stabBufY[i] > yMax) yMax = stabBufY[i];
    }
    if ((xMax - xMin) > STABLE_POS_THRESH) return 0u;
    if ((yMax - yMin) > STABLE_POS_THRESH) return 0u;
    /* 角度用圆周角差：以第一个样本为参考，检查最大偏差 */
    float refAng = stabBufAng[0];
    for (uint8_t i = 1u; i < STABLE_WINDOW; i++)
    {
        if (angleDiff(stabBufAng[i], refAng) > STABLE_ANG_THRESH) return 0u;
    }
    return 1u;
}

uint8_t imageInit(void)
{
    uint8_t initState = mt9v03x_init();

    /* mt9v03x_init() 返回 0 表示成功，非零表示失败 */
    if (initState) return 0u;

    timer_init(TC_TIME2_CH0, TIMER_US);
    timer_start(TC_TIME2_CH0);
    lastFrameTimestampUs = 0u;
    frameTimestampPrimed = 0u;

    cannyInit(&mt9v03xOriImg, 0.75f, 0u, 0u, mt9v03xOriImg.xLen - 1u, mt9v03xOriImg.yLen - 1u,
        VISION_CANNY_LOW_DEFAULT, VISION_CANNY_HIGH_DEFAULT);

    /* ^ 轮廓检测：先 init 固定项（质心特征/ROI/视觉中心），再用默认调参喂可调项。
    * sizeMin/sizeMax/huLost/brightnessMax 走 imageApplyContourTuning，
    * 与菜单热重载共用同一入口，保证默认值与运行时调参一致。 */
    contourDetectInit(
        VISION_CONTOUR_SIZE_MIN_DEFAULT, VISION_CONTOUR_SIZE_MAX_DEFAULT, 255u,
        mt9v03xOriImg.xLen / 2u, mt9v03xOriImg.yLen / 2u,
        0u, 0u, VISION_CONTOUR_BRIGHTNESS_MAX_DEF);
    imageApplyContourTuning(
        VISION_CONTOUR_SIZE_MIN_DEFAULT, VISION_CONTOUR_SIZE_MAX_DEFAULT,
        VISION_CONTOUR_HU_LOST_DEFAULT, VISION_CONTOUR_BRIGHTNESS_MAX_DEF);
    binaryCcInit(&binaryCcActiveConfig);
    imageApplyBinaryTemplate();
    imageSetDetectMode(VISION_DETECT_MODE_DEFAULT);

    return 1u;
}

```



```

uint8_t imageGetFrameView(imageFrameKindEnum kind, imageFrameViewStruct* out)
{
    const u8Image* selected;
    if (out == 0) return 0u;
    out->frameSeq = imageFrameSeq;
    switch (kind)
    {
        case IMAGE_FRAME_ORIGINAL:
            selected = &mt9v03xOrImg;
            break;
        case IMAGE_FRAME_DETECT_PREVIEW:
        case IMAGE_FRAME_CANNY:
            selected = &cannyEdgeImg;
            break;
        default:
            out->pixels = 0;
            out->width = 0u;
            out->height = 0u;
            return 0u;
    }
    out->pixels = selected->ptr;
    out->width = selected->xLen;
    out->height = selected->yLen;
    return (out->pixels != 0) ? 1u : 0u;
}

void imageGetObservationSnapshot(imageObservationSnapshotStruct* out)
{
    uint8_t selectedIndex;
    if (out == 0) return;

    selectedIndex = chevronDetector.selectedIndex;
    out->frameSeq = imageFrameSeq;
    out->trackingState = chevronDetector.trackingState;
    out->contourCount = chevronDetector.contourCount;
    out->selectedIndex = selectedIndex;
    out->targetLost = chevronDetector.targetLost;
    out->valid = (chevronDetector.trackingState == OM_TRACK_LOCKED &&
        chevronDetector.contourCount > 0u && selectedIndex < chevronDetector.contourCount &&
        !chevronDetector.targetLost) ? 1u : 0u;
    out->orientBaselineValid = chevronDetector.orientBaselineValid;
    out->orientProtectState = chevronDetector.orientProtectState;
    out->orientationFresh = (uint8_t) (
        (chevronDetector.orientBaselineValid != 0u) &&
        (chevronDetector.orientRejectFrameCount == 0u));
    if ((chevronDetector.contourCount > 0u) &&
        (selectedIndex < chevronDetector.contourCount))
    {
        out->selectedCenterX = chevronDetector.contourData[selectedIndex].centerX;
        out->selectedCenterY = chevronDetector.contourData[selectedIndex].centerY;
        out->selectedSize = chevronDetector.contourData[selectedIndex].contourSize;
        out->selectedHuDistance = chevronDetector.contourData[selectedIndex].huDistance;
    }
    else
    {
        out->selectedCenterX = 0u;
        out->selectedCenterY = 0u;
        out->selectedSize = 0u;
        out->selectedHuDistance = 0.0f;
    }
    out->lostFrameCount = chevronDetector.lostFrameCount;
    out->lockFrameCount = chevronDetector.lockFrameCount;
    out->orientSpikeFrameCount = chevronDetector.orientSpikeFrameCount;
    out->filteredCenterX = chevronDetector.centerXFilter.filteredValue;
    out->filteredCenterY = chevronDetector.centerYFilter.filteredValue;
    out->velocityX = chevronDetector.velocityXFilter.filteredValue;
    out->velocityY = chevronDetector.velocityYFilter.filteredValue;
    out->orientation = chevronDetector.orientation;
    out->orientationConfidence = chevronDetector.orientationConfidence;
}

uint8_t imageProcess(void)
{
    float sampleDtS;
    if (!imageTakeFrameSnapshot(&sampleDtS)) return 0u;
    imageFrameSeq++;

    if (imageDetectMode == VISION_DETECT_MODE_LEGACY_CANNY)
    {

```

```

        cannyEdgeDetection(&mt9v03xOriImg, &gaussianBlurImg, &cannyEdgeImg);
        contourDetect(&cannyEdgeImg, &mt9v03xOriImg, sampleDtS, &chevronDetector);
    }
    else
    {
        binaryCcDetect(&mt9v03xOriImg, &cannyEdgeImg, sampleDtS, &chevronDetector);
    }

    /* 标定状态机：等待稳定后提取实测 Hu 矩 */
    if (chevronCalibState == CALIB_DETECTING)
    {
        if (chevronDetector.contourCount > 0 && !chevronDetector.targetLost)
        {
            uint8_t si = chevronDetector.selectedIndex;
            stabBufX[stabBufIdx] = chevronDetector.contourData[si].centerX;
            stabBufY[stabBufIdx] = chevronDetector.contourData[si].centerY;
            stabBufAng[stabBufIdx] = chevronDetector.orientation;
            stabBufIdx++;
            if (stabBufIdx >= STABLE_WINDOW)
            {
                stabBufIdx = 0u;
                stabBufFull = 1u;
            }

            if (stabBufFull && checkStability())
            {
                float calibHu[7];
                uint8_t extracted = (imageDetectMode == VISION_DETECT_MODE_LEGACY_CANNY)
                    ? contourExtractHuMoments(&chevronDetector, si, calibHu)
                    : binaryCcExtractHuMoments(&chevronDetector, si, calibHu);

                if (extracted)
                {
                    /* 记录像素数并收紧尺寸范围 (允许 50%~200%) */
                    chevronCalibSize = chevronDetector.contourData[si].contourSize;
                    uint16_t sMin = (chevronCalibSize > 4u) ? (chevronCalibSize / 2u) : 2u;
                    uint16_t sMax = chevronCalibSize * 2u;
                    /* 写入生效模板：标定 Hu + 收紧尺寸，后续菜单热重载不再覆盖。
                     * HuLost 沿用当前菜单设定，供 flash 持久化读取。 */
                    imageSetActiveTemplateForMode(imageDetectMode, calibHu, sMin, sMax);

                    chevronCalibState = CALIB_DONE;
                    calibrationSavePending = 1u;
                }
            }
        }
        else
        {
            /* 目标丢失时重置稳定缓冲区 */
            stabBufIdx = 0u;
            stabBufFull = 0u;
        }
    }

    return 1u;
}

void imageCalibTrigger(void)
{
    stabBufIdx = 0u;
    stabBufFull = 0u;
    calibrationSavePending = 0u;
    chevronCalibState = CALIB_DETECTING;
}

uint8_t imageTakeCalibrationSaveRequest(void)
{
    uint8_t pending = calibrationSavePending;
    calibrationSavePending = 0u;
    return pending;
}

```

core0_cyt4bb7/project/code/vision/image.h

SHA-256

:

78c774fe8b767f87d12c716068f012946fc8347cbdf737b0d07060e9e0d87e07

代码 (c)

```
#ifndef IMAGE_H
#define IMAGE_H

#include <stdint.h>

/* 标定状态 */
typedef enum
{
    CALIB_WAITING, /* 上电默认 / 标定完成后, 等待按键触发 */
    CALIB_DETECTING, /* 用预设模板检测中, 等待稳定 */
    CALIB_DONE /* 标定完成, 用实测模板继续检测 */
} CalibState;

typedef enum
{
    IMAGE_FRAME_ORIGINAL = 0,
    IMAGE_FRAME_DETECT_PREVIEW = 1,
    IMAGE_FRAME_CANNY = 2
} imageFrameKindEnum;

typedef struct
{
    const uint8_t* pixels;
    uint16_t width;
    uint16_t height;
    uint32_t frameSeq;
} imageFrameViewStruct;

typedef struct
{
    uint32_t frameSeq;
    uint8_t valid;
    uint8_t trackingState;
    uint8_t contourCount;
    uint8_t selectedIndex;
    uint8_t targetLost;
    uint8_t orientBaselineValid;
    uint8_t orientProtectState;
    uint8_t orientationFresh;
    uint16_t selectedCenterX;
    uint16_t selectedCenterY;
    uint16_t selectedSize;
    uint16_t lostFrameCount;
    uint16_t lockFrameCount;
    uint16_t orientSpikeFrameCount;
    float selectedHuDistance;
    float filteredCenterX;
    float filteredCenterY;
    float velocityX;
    float velocityY;
    float orientation;
    float orientationConfidence;
} imageObservationSnapshotStruct;

typedef struct
{
    uint8_t thresholdLow;
    uint8_t thresholdHigh;
    uint16_t areaMin;
    uint16_t areaMax;
    uint8_t brightnessMin;
    float eccMax;
    float fillMin;
    float huMax;
    uint8_t orientationBrightMin;
    uint8_t orientationBorderMargin;
    float orientationConfidenceMin;
    uint8_t orientationHoldFrames;
    uint16_t referenceCenterX;
    uint16_t referenceCenterY;
} imageBinaryCcTuningStruct;

typedef struct
{
    float jumpMax;
    float recoverRad;
```

```

    uint8_t confirmFrames;
} imageOrientationProtectTuningStruct;

/* 只读图像访问器（原始灰度图 / 检测预览图，均无识别标注）
 * DetectMode=BinaryCC 时检测预览为 binary mask；Legacy 时为 Canny 边缘图。 */
uint8_t imageGetFrameView(imageFrameKindEnum kind, imageFrameViewStruct* out);
void imageGetObservationSnapshot(imageObservationSnapshotStruct* out);

/* ^ 轮廓检测默认值（菜单从这里加载，运行时可改 + flash 持久化）
 * sizeMin=15: 滤掉小杂散非闭合轮廓（保守值，原 30 过严）
 * sizeMax=5000: 上限
 * brightnessMax=255: 关闭质心亮度过滤（硬件未验证）
 * huLost=15.0: 选中轮廓 Hu 距离超此值即 targetLost（保守值）
 */
#define VISION_CONTOUR_SIZE_MIN_DEFAULT (15u)
#define VISION_CONTOUR_SIZE_MAX_DEFAULT (5000u)
#define VISION_CONTOUR_BRIGHTNESS_MAX_DEF (255u)
#define VISION_CONTOUR_HU_LOST_DEFAULT (15.0f)

uint8_t imageInit(void);
uint8_t imageProcess(void);
void imageCalibTrigger(void); /* 按键触发一次标定 */
/* 取走一次标定成功事件，供菜单前台自动持久化；上电 Flash 恢复不会产生事件。 */
uint8_t imageTakeCalibrationSaveRequest(void);

void imageSetDetectMode(uint8_t mode);
uint8_t imageGetDetectMode(void);

/* 轮廓检测调参热重载封装：^ Hu 模板由 image.c 私有持有，菜单只传可调标量。
 * 内部依次调 contourDetectSetSizeRange / SetTemplate / SetBrightnessMax。 */
void imageApplyContourTuning(uint16_t sizeMin, uint16_t sizeMax, float huLost, uint8_t brightnessMax);
void imageApplyBinaryCcTuning(const imageBinaryCcTuningStruct* config);
void imageApplyCannyThresholds(uint16_t thresholdLow, uint16_t thresholdHigh);
void imageGetOrientationProtectDefaults(imageOrientationProtectTuningStruct* out);
void imageApplyOrientationProtect(const imageOrientationProtectTuningStruct* config);

/* 标定/flash 恢复：写入当前 DetectMode 的实测 Hu 模板 + 收紧尺寸范围并即时生效。 */
void imageSetActiveTemplateForMode(uint8_t mode, const float hu[7], uint16_t sizeMin, uint16_t sizeMax);

/* 读当前 DetectMode 的 Hu 模板 + 尺寸范围（供 flash 持久化）。返回 1=标定值，0=出厂默认。 */
uint8_t imageGetActiveTemplateForMode(uint8_t mode, float huOut[7], uint16_t* sizeMinOut, uint16_t* sizeMaxOut);

/* 清空恢复出厂默认：丢弃 RAM 内两套已标定 Hu 模板，回退默认模板 + 默认尺寸并即时生效。 */
void imageResetActiveTemplate(void);

#endif // IMAGE_H

```

core0_cyt4bb7/project/code/vision/om_binarycc.c

SHA-256

:

06485d29e0b30ee6dfdafb27db0ced25b70352bab06bc5cf4f8e3c0a28cbb3c3

代码 (c)

```

/*
 * om_binarycc.c - BinaryCC ^ 灯板检测
 *
 * 双阈值二值化 + 八连通候选 + 矩/Hu/朝向筛选，输出兼容 CONTOUR_DETECTOR_STRUCT。
 */

#include "om_binarycc.h"

#include "om_internal.h"
#include "om_tracking_internal.h"

#define BINARYCC_HU_TIE_THRESHOLD (1.0f)
#define BINARYCC_BRIGHT_MIN_COUNT (12u)

typedef struct
{
    uint16_t area;
    uint16_t bboxMinX;
    uint16_t bboxMinY;
    uint16_t bboxMaxX;
    uint16_t bboxMaxY;

```

```

float centroidX;
float centroidY;
float meanBrightness;
float m20;
float m02;
float m11;
float m30;
float m03;
float m21;
float m12;
float hu[7];
float huDistance;
float ecc;
float fillRatio;
float orientation;
float orientationConfidence;
uint8_t orientationEdgeClipped;
} BINARYCC_CANDIDATE_STRUCT;

static BINARYCC_CONFIG_STRUCT binaryCcConfig;
static uint8_t binaryCcTemplateReady = 0u;
static float binaryCcHuTemplate[7];
static BINARYCC_CANDIDATE_STRUCT binaryCcCandidates[OM_MAX_OBJECTS];
static uint8_t binaryCcCandidateCount = 0u;

static void binaryCcComputeHu(float area, float mu20, float mu02, float mu11, float mu30, float mu03, float mu21, float mu12, float hu[7]);
static float binaryCcHuDistance(const float hu[7], const float templ[7]);
static void binaryCcThreshold(const u8Image* graySrc, u8Image* maskDst);
static uint8_t binaryCcVisited(uint32_t index);
static void binaryCcMarkVisited(uint32_t index);
static uint8_t binaryCcScanComponent(const u8Image* graySrc, const u8Image* maskDst, uint32_t startIndex, BINARYCC_CANDIDATE_STRUCT* out);
static void binaryCcFinishCandidate(const u8Image* graySrc, uint32_t queueCount, BINARYCC_CANDIDATE_STRUCT* c,
    float originX, float originY,
    float sumX, float sumY, float sumX2, float sumY2, float sumXY, float sumX3, float sumY3,
    float sumX2Y, float sumXY2, float sumGray);
static float binaryCcComputeOrientation(const u8Image* graySrc, uint32_t queueCount, float cx, float cy,
    float covXX, float covYY, float covXY, float* outConfidence);
static uint8_t binaryCcComputeBrightCoreOrientation(const u8Image* graySrc, uint32_t queueCount, float* outOrientation, float* outConfidence);
static float binaryCcSideSpreadScore(const u8Image* graySrc, uint32_t queueCount, float cx, float cy,
    float axisC, float axisS, float sign, float extent, uint8_t brightMin);
static uint8_t binaryCcTouchesOrientationBorder(const u8Image* graySrc, const BINARYCC_CANDIDATE_STRUCT* c);
static uint32_t binaryCcReferenceDistSq(const CONTOUR_DETECTOR_STRUCT* obj, uint8_t idx);
static void binaryCcUpdateTracking(CONTOUR_DETECTOR_STRUCT* obj, uint8_t detected, float sampleDtS);
static void binaryCcHandleLost(CONTOUR_DETECTOR_STRUCT* obj);

void binaryCcInit(const BINARYCC_CONFIG_STRUCT* config)
{
    memset(&binaryCcConfig, 0, sizeof(binaryCcConfig));
    binaryCcCandidateCount = 0u;
    binaryCcTemplateReady = 0u;
    if (config != 0) binaryCcConfig = *config;
}

void binaryCcSetConfig(const BINARYCC_CONFIG_STRUCT* config)
{
    if (config != 0) binaryCcConfig = *config;
}

void binaryCcSetTemplate(const float hu[7], float huMax)
{
    if (hu == 0)
    {
        binaryCcTemplateReady = 0u;
        return;
    }
    for (uint8_t i = 0u; i < 7u; i++) binaryCcHuTemplate[i] = hu[i];
    binaryCcConfig.huMax = huMax;
    binaryCcTemplateReady = 1u;
}

uint16_t binaryCcDetect(const u8Image* graySrc, u8Image* maskDst, float sampleDtS, CONTOUR_DETECTOR_STRUCT* obj)
{
    if (obj == 0 || graySrc == 0 || maskDst == 0 || graySrc->ptr == 0 || maskDst->ptr == 0 ||
        graySrc->size > OM_BUFFER_SIZE || maskDst->size < graySrc->size)
    {
        if (obj != 0) binaryCcHandleLost(obj);
        return 0u;
    }
}

```

```

binaryCcThreshold(graySrc, maskDst);
binaryCcCandidateCount = 0u;
obj->selectedIndex = 0u;
obj->contourCount = 0u;
obj->targetLost = 1u;

contourGeneration++;
if (contourGeneration == 0u)
{
    memset(inContourMap, 0, graySrc->size * sizeof(inContourMap[0]));
    contourGeneration = 1u;
}

for (uint32_t i = 0u; i < graySrc->size; i++)
{
    BINARYCC_CANDIDATE_STRUCT c;
    if (maskDst->ptr[i] != WHITE || binaryCcVisited(i)) continue;
    memset(&c, 0, sizeof(c));
    if (!binaryCcScanComponent(graySrc, maskDst, i, &c))
    {
        continue;
    }

    if (c.area < binaryCcConfig.areaMin)
    {
        continue;
    }
    if (binaryCcConfig.areaMax > 0u && c.area > binaryCcConfig.areaMax)
    {
        continue;
    }
    if (binaryCcConfig.brightnessMin > 0u && c.meanBrightness < (float) binaryCcConfig.brightnessMin)
    {
        continue;
    }
    if (binaryCcConfig.eccMax > 0.0f && c.ecc > binaryCcConfig.eccMax)
    {
        continue;
    }
    if (binaryCcConfig.fillMin > 0.0f && c.fillRatio < binaryCcConfig.fillMin)
    {
        continue;
    }
    if (binaryCcTemplateReady && binaryCcConfig.huMax > 0.0f && c.huDistance > binaryCcConfig.huMax)
    {
        continue;
    }

    if (binaryCcCandidateCount < OM_MAX_OBJECTS)
    {
        uint8_t dst = binaryCcCandidateCount;
        binaryCcCandidates[dst] = c;
        obj->contourData[dst].centerX = (uint16_t) (c.centroidX + 0.5f);
        obj->contourData[dst].centerY = (uint16_t) (c.centroidY + 0.5f);
        obj->contourData[dst].contourSize = c.area;
        obj->contourData[dst].huDistance = c.huDistance;
        binaryCcCandidateCount++;
    }
    else
    {
        continue;
    }
}

obj->contourCount = binaryCcCandidateCount;
for (uint8_t i = binaryCcCandidateCount; i < OM_MAX_OBJECTS; i++) obj->contourData[i].contourSize = 0u;

if (binaryCcCandidateCount == 0u)
{
    binaryCcHandleLost(obj);
    return 0u;
}

if (binaryCcTemplateReady)
{
    float huMin = binaryCcCandidates[0u].huDistance;
    for (uint8_t i = 1u; i < binaryCcCandidateCount; i++)
        if (binaryCcCandidates[i].huDistance < huMin) huMin = binaryCcCandidates[i].huDistance;
}

```

```

uint32_t bestDist = 0xFFFFFFFFu;
for (uint8_t i = 0u; i < binaryCcCandidateCount; i++)
{
    if ((binaryCcCandidates[i].huDistance - huMin) < BINARYCC_HU_TIE_THRESHOLD)
    {
        uint32_t d = binaryCcReferenceDistSq(obj, i);
        if (d < bestDist)
        {
            bestDist = d;
            obj->selectedIndex = i;
        }
    }
}
}
else
{
    uint32_t bestDist = binaryCcReferenceDistSq(obj, 0u);
    obj->selectedIndex = 0u;
    for (uint8_t i = 1u; i < binaryCcCandidateCount; i++)
    {
        uint32_t d = binaryCcReferenceDistSq(obj, i);
        if (d < bestDist)
        {
            bestDist = d;
            obj->selectedIndex = i;
        }
    }
}

obj->targetLost = 0u;
binaryCcUpdateTracking(obj, 1u, sampleDtS);
return binaryCcCandidateCount;
}

uint8_t binaryCcExtractHuMoments(const CONTOUR_DETECTOR_STRUCT* obj, uint8_t contourIndex, float huOut[7])
{
    if (obj == 0 || huOut == 0) return 0u;
    if (contourIndex >= obj->contourCount || contourIndex >= binaryCcCandidateCount) return 0u;
    for (uint8_t i = 0u; i < 7u; i++) huOut[i] = binaryCcCandidates[contourIndex].hu[i];
    return 1u;
}

static void binaryCcThreshold(const u8Image* graySrc, u8Image* maskDst)
{
    for (uint32_t i = 0u; i < graySrc->size; i++)
    {
        uint8_t v = graySrc->ptr[i];
        maskDst->ptr[i] = (v >= binaryCcConfig.thresholdLow && v <= binaryCcConfig.thresholdHigh) ? WHITE : BLACK;
    }
}

static uint8_t binaryCcVisited(uint32_t index)
{
    return ((inContourMap[index] >> 16) == contourGeneration) ? 1u : 0u;
}

static void binaryCcMarkVisited(uint32_t index)
{
    inContourMap[index] = ((uint32_t) contourGeneration << 16) | 1u;
}

static uint8_t binaryCcScanComponent(const u8Image* graySrc, const u8Image* maskDst, uint32_t startIndex, BINARYCC_CANDIDATE_STRUCT* out)
{
    uint32_t head = 0u;
    uint32_t tail = 0u;
    float sumX = 0.0f, sumY = 0.0f, sumX2 = 0.0f, sumY2 = 0.0f, sumXY = 0.0f;
    float sumX3 = 0.0f, sumY3 = 0.0f, sumX2Y = 0.0f, sumXY2 = 0.0f, sumGray = 0.0f;
    // 以种子像素为原点累加相对坐标: 中心矩平移不变, 但幅值大幅减小, 避免 float 高阶矩灾难性抵消
    uint32_t originX = startIndex % graySrc->xLen;
    uint32_t originY = startIndex / graySrc->xLen;

    out->bboxMinX = graySrc->xLen;
    out->bboxMinY = graySrc->yLen;
    if (tail >= OM_BUFFER_SIZE) return 0u;
    contourQueue[tail++] = startIndex;
    binaryCcMarkVisited(startIndex);

    while (head < tail)

```

```

{
    uint32_t cur = contourQueue[head++];
    uint32_t x = cur % graySrc->xLen;
    uint32_t y = cur / graySrc->xLen;
    float xf = (float) ((int32_t) x - (int32_t) originX);
    float yf = (float) ((int32_t) y - (int32_t) originY);
    float gray = (float) graySrc->ptr[cur];

    out->area++;
    if (x < out->bboxMinX) out->bboxMinX = (uint16_t) x;
    if (y < out->bboxMinY) out->bboxMinY = (uint16_t) y;
    if (x > out->bboxMaxX) out->bboxMaxX = (uint16_t) x;
    if (y > out->bboxMaxY) out->bboxMaxY = (uint16_t) y;

    sumX += xf;
    sumY += yf;
    sumX2 += xf * xf;
    sumY2 += yf * yf;
    sumXY += xf * yf;
    sumX3 += xf * xf * xf;
    sumY3 += yf * yf * yf;
    sumX2Y += xf * xf * yf;
    sumXY2 += xf * yf * yf;
    sumGray += gray;

    for (uint8_t ni = 0u; ni < 8u; ni++)
    {
        int32_t nx = (int32_t) x + contourDx[ni];
        int32_t ny = (int32_t) y + contourDy[ni];
        uint32_t next;
        if (nx < 0 || ny < 0 || (uint32_t) nx >= graySrc->xLen || (uint32_t) ny >= graySrc->yLen) continue;
        next = (uint32_t) ny * graySrc->xLen + (uint32_t) nx;
        if (maskDst->ptr[next] == WHITE && !binaryCcVisited(next))
        {
            if (tail >= OM_BUFFER_SIZE) return 0u;
            binaryCcMarkVisited(next);
            contourQueue[tail++] = next;
        }
    }
}

if (out->area == 0u) return 0u;
binaryCcFinishCandidate(graySrc, tail, out, (float) originX, (float) originY,
    sumX, sumY, sumX2, sumY2, sumXY, sumX3, sumY3, sumX2Y, sumXY2, sumGray);
return 1u;
}

static void binaryCcFinishCandidate(const u8Image* graySrc, uint32_t queueCount, BINARYCC_CANDIDATE_STRUCT* c,
    float originX, float originY,
    float sumX, float sumY, float sumX2, float sumY2, float sumXY, float sumX3, float sumY3,
    float sumX2Y, float sumXY2, float sumGray)
{
    float area = (float) c->area;
    // cx/cy 为相对 seed 原点的质心, 供中心矩使用(平移不变); 绝对质心 = 原点 + 相对质心
    float cx = sumX / area;
    float cy = sumY / area;
    float centroidXAbs = originX + cx;
    float centroidYAbs = originY + cy;
    float mu20 = sumX2 - cx * sumX;
    float mu02 = sumY2 - cy * sumY;
    float mu11 = sumXY - cx * sumY;
    float mu30 = sumX3 - 3.0f * cx * sumX2 + 3.0f * cx * cx * sumX - cx * cx * cx * area;
    float mu03 = sumY3 - 3.0f * cy * sumY2 + 3.0f * cy * cy * sumY - cy * cy * cy * area;
    float mu21 = sumX2Y - cy * sumX2 - 2.0f * cx * sumXY + 2.0f * cx * cy * sumX + cx * cx * sumY - cx * cx * cy * area;
    float mu12 = sumXY2 - cx * sumY2 - 2.0f * cy * sumXY + 2.0f * cx * cy * sumY + cy * cy * sumX - cx * cy * cy * area;
    float covXX = mu20 / area;
    float covYY = mu02 / area;
    float covXY = mu11 / area;
    float trace = covXX + covYY;
    float delta = sqrtf((covXX - covYY) * (covXX - covYY) + 4.0f * covXY * covXY);
    float lambdaA = (trace + delta) * 0.5f;
    float lambdaB = (trace - delta) * 0.5f;
    float bboxArea = (float) ((c->bboxMaxX - c->bboxMinX + 1u) * (c->bboxMaxY - c->bboxMinY + 1u));

    if (lambdaA < lambdaB)
    {
        float t = lambdaA;
        lambdaA = lambdaB;
        lambdaB = t;
    }
}

```



```

    }

    c->centroidX = centroidXAbs;
    c->centroidY = centroidYAbs;
    c->meanBrightness = sumGray / area;
    c->m20 = mu20;
    c->m02 = mu02;
    c->m11 = mu11;
    c->m30 = mu30;
    c->m03 = mu03;
    c->m21 = mu21;
    c->m12 = mu12;
    c->ecc = (lambdaB > 1.0e-6f) ? sqrtf(lambdaA / lambdaB) : 9999.0f;
    c->fillRatio = (bboxArea > 0.0f) ? (area / bboxArea) : 0.0f;
    binaryCcComputeHu(area, mu20, mu02, mu11, mu30, mu03, mu21, mu12, c->hu);
    c->huDistance = binaryCcTemplateReady ? binaryCcHuDistance(c->hu, binaryCcHuTemplate) : -1.0f;
    c->orientation = binaryCcComputeOrientation(graySrc, queueCount, centroidXAbs, centroidYAbs, covXX, covYY, covXY, &c->orientationConfidence);
    c->orientationEdgeClipped = binaryCcTouchesOrientationBorder(graySrc, c);
    if (c->orientationEdgeClipped) c->orientationConfidence = 0.0f;
}

```

```

static void binaryCcComputeHu(float area, float mu20, float mu02, float mu11, float mu30, float mu03, float mu21, float mu12, float hu[7])
{

```

```

    float invArea2 = 1.0f / (area * area);
    float invArea25 = invArea2 / sqrtf(area);
    float eta20 = mu20 * invArea2;
    float eta02 = mu02 * invArea2;
    float eta11 = mu11 * invArea2;
    float eta30 = mu30 * invArea25;
    float eta03 = mu03 * invArea25;
    float eta21 = mu21 * invArea25;
    float eta12 = mu12 * invArea25;
    float a = eta30 + eta12;
    float b = eta21 + eta03;
    float c = eta30 - 3.0f * eta12;
    float d = 3.0f * eta21 - eta03;

```

```

    hu[0] = eta20 + eta02;
    hu[1] = (eta20 - eta02) * (eta20 - eta02) + 4.0f * eta11 * eta11;
    hu[2] = c * c + d * d;
    hu[3] = a * a + b * b;
    hu[4] = c * a * (a * a - 3.0f * b * b) + d * b * (3.0f * a * a - b * b);
    hu[5] = (eta20 - eta02) * (a * a - b * b) + 4.0f * eta11 * a * b;
    hu[6] = d * a * (a * a - 3.0f * b * b) - c * b * (3.0f * a * a - b * b);
}

```

```

static float binaryCcHuDistance(const float hu[7], const float templ[7])
{

```

```

    {
        static const float weights[7] = {2.0f, 2.0f, 1.5f, 1.5f, 0.5f, 0.5f, 0.02f};
        float distance = 0.0f;
        for (uint8_t i = 0u; i < 7u; i++)
        {
            float a = fabsf(hu[i]);
            float b = fabsf(templ[i]);
            if (a < 1.0e-12f) a = 1.0e-12f;
            if (b < 1.0e-12f) b = 1.0e-12f;
            distance += weights[i] * fabsf(logf(a) - logf(b));
        }
        return distance;
    }
}

```

```

static float binaryCcComputeOrientation(const u8Image* graySrc, uint32_t queueCount, float cx, float cy,
float covXX, float covYY, float covXY, float* outConfidence)
{

```

```

    {
        float axis = wrapToPi(0.5f * atan2f(2.0f * covXY, covXX - covYY) + HALF_PI);
        if (binaryCcConfig.orientationBrightMin > 0u)
        {
            float brightOrient;
            if (binaryCcComputeBrightCoreOrientation(graySrc, queueCount, &brightOrient, outConfidence)) return brightOrient;
            *outConfidence = 0.0f;
            return axis;
        }
    }

```

```

    float axisC = cosf(axis);
    float axisS = sinf(axis);
    float posExtent = 0.0f;
    float negExtent = 0.0f;
    for (uint32_t i = 0u; i < queueCount; i++)
    {

```

```

        uint32_t index = contourQueue[i];
        float dx = (float) (index % graySrc->xLen) - cx;
        float dy = (float) (index / graySrc->xLen) - cy;
        float t = dx * axisC + dy * axisS;
        if (t > posExtent) posExtent = t;
        if (-t > negExtent) negExtent = -t;
    }
    if (posExtent <= 1.0f || negExtent <= 1.0f)
    {
        *outConfidence = 0.0f;
        return axis;
    }
    *outConfidence = 1.0f;
    return axis;
}

static uint8_t binaryCcComputeBrightCoreOrientation(const u8Image* graySrc, uint32_t queueCount, float* outOrientation, float* outConfidence)
{
    float count = 0.0f, sumX = 0.0f, sumY = 0.0f, sumX2 = 0.0f, sumY2 = 0.0f, sumXY = 0.0f;
    float cx, cy, covXX, covYY, covXY, axis, axisC, axisS, posExtent = 0.0f, negExtent = 0.0f;
    float posScore, negScore, denom;

    for (uint32_t i = 0u; i < queueCount; i++)
    {
        uint32_t index = contourQueue[i];
        if (graySrc->ptr[index] < binaryCcConfig.orientationBrightMin) continue;
        float x = (float) (index % graySrc->xLen);
        float y = (float) (index / graySrc->xLen);
        count += 1.0f;
        sumX += x;
        sumY += y;
        sumX2 += x * x;
        sumY2 += y * y;
        sumXY += x * y;
    }
    if (count < (float) BINARYCC_BRIGHT_MIN_COUNT) return 0u;

    cx = sumX / count;
    cy = sumY / count;
    covXX = sumX2 / count - cx * cx;
    covYY = sumY2 / count - cy * cy;
    covXY = sumXY / count - cx * cy;
    axis = wrapToPi(0.5f * atan2f(2.0f * covXY, covXX - covYY) + HALF_PI);
    axisC = cosf(axis);
    axisS = sinf(axis);

    for (uint32_t i = 0u; i < queueCount; i++)
    {
        uint32_t index = contourQueue[i];
        if (graySrc->ptr[index] < binaryCcConfig.orientationBrightMin) continue;
        float dx = (float) (index % graySrc->xLen) - cx;
        float dy = (float) (index / graySrc->xLen) - cy;
        float t = dx * axisC + dy * axisS;
        if (t > posExtent) posExtent = t;
        if (-t > negExtent) negExtent = -t;
    }
    if (posExtent <= 1.0f || negExtent <= 1.0f) return 0u;

    posScore = binaryCcSideSpreadScore(graySrc, queueCount, cx, cy, axisC, axisS, 1.0f, posExtent, binaryCcConfig.orientationBrightMin);
    negScore = binaryCcSideSpreadScore(graySrc, queueCount, cx, cy, axisC, axisS, -1.0f, negExtent, binaryCcConfig.orientationBrightMin);
    if (posScore >= 1.0e8f || negScore >= 1.0e8f) return 0u;
    *outOrientation = (posScore <= negScore) ? axis : wrapToPi(axis + PI);
    denom = fabsf(posScore) + fabsf(negScore) + 1.0e-6f;
    *outConfidence = fabsf(posScore - negScore) / denom;
    if (*outConfidence > 1.0f) *outConfidence = 1.0f;
    return 1u;
}

static float binaryCcSideSpreadScore(const u8Image* graySrc, uint32_t queueCount, float cx, float cy,
    float axisC, float axisS, float sign, float extent, uint8_t brightMin)
{
    float farStart = 0.45f * extent;
    float farEnd = 0.98f * extent;
    float absSum = 0.0f, squareSum = 0.0f, absMax = 0.0f, sampleCount = 0.0f;
    for (uint32_t i = 0u; i < queueCount; i++)
    {
        uint32_t index = contourQueue[i];
        if (graySrc->ptr[index] < brightMin) continue;
        float dx = (float) (index % graySrc->xLen) - cx;

```

```

        float dy = (float) (index / graySrc->xLen) - cy;
        float t = dx * axisC + dy * axisS;
        float p = -dx * axisS + dy * axisC;
        float signedT = sign * t;
        if (signedT < farStart || signedT > farEnd) continue;
        float absP = fabsf(p);
        absSum += absP;
        squareSum += p * p;
        if (absP > absMax) absMax = absP;
        sampleCount += 1.0f;
    }
    if (sampleCount < 3.0f) return 1.0e9f;
    return (absSum / sampleCount) + 0.35f * sqrtf(squareSum / sampleCount) + 0.10f * absMax;
}

static uint8_t binaryCcTouchesOrientationBorder(const u8Image* graySrc, const BINARYCC_CANDIDATE_STRUCT* c)
{
    uint8_t m = binaryCcConfig.orientationBorderMargin;
    if (m == 0u) return 0u;
    if (c->bboxMinX <= m || c->bboxMinY <= m) return 1u;
    if ((uint16_t) (c->bboxMaxX + m) >= (graySrc->xLen - 1u)) return 1u;
    if ((uint16_t) (c->bboxMaxY + m) >= (graySrc->yLen - 1u)) return 1u;
    return 0u;
}

static uint32_t binaryCcReferenceDistSq(const CONTOUR_DETECTOR_STRUCT* obj, uint8_t idx)
{
    uint16_t refX = (obj->lastValidSize != 0u && obj->trackingState != OM_TRACK_SEARCHING)
        ? obj->lastValidCenterX
        : binaryCcConfig.referenceCenterX;
    uint16_t refY = (obj->lastValidSize != 0u && obj->trackingState != OM_TRACK_SEARCHING)
        ? obj->lastValidCenterY
        : binaryCcConfig.referenceCenterY;
    int32_t dx = (int32_t) obj->contourData[idx].centerX - (int32_t) refX;
    int32_t dy = (int32_t) obj->contourData[idx].centerY - (int32_t) refY;
    return (uint32_t) (dx * dx + dy * dy);
}

static void binaryCcUpdateTracking(CONTOUR_DETECTOR_STRUCT* obj, uint8_t detected, float sampleDts)
{
    if (!detected)
    {
        binaryCcHandleLost(obj);
        return;
    }

    uint8_t si = obj->selectedIndex;
    BINARYCC_CANDIDATE_STRUCT* c = &binaryCcCandidates[si];
    uint16_t selCx = obj->contourData[si].centerX;
    uint16_t selCy = obj->contourData[si].centerY;
    uint16_t selSize = obj->contourData[si].contourSize;
    uint8_t acceptOrient = (c->orientationConfidence >= binaryCcConfig.orientationConfidenceMin) ? 1u : 0u;
    uint8_t holdFrames = (binaryCcConfig.orientationHoldFrames == 0u) ? 1u : binaryCcConfig.orientationHoldFrames;

    obj->orientationConfidence = c->orientationConfidence;
    obj->lostFrameCount = 0u;
    obj->lockFrameCount = (obj->lockFrameCount < 255u) ? obj->lockFrameCount + 1u : 255u;
    if (obj->trackingState == OM_TRACK_SEARCHING && obj->lockFrameCount >= OM_LOCK_THRESHOLD) obj->trackingState =
OM_TRACK_ACQUIRING;
    else if (obj->trackingState == OM_TRACK_ACQUIRING && obj->lockFrameCount >= (OM_LOCK_THRESHOLD + OM_LOCK_CONFIRM)) obj-
>trackingState = OM_TRACK_LOCKED;
    else if (obj->trackingState == OM_TRACK_LOST_HOLDING) obj->trackingState = OM_TRACK_LOCKED;

    if (acceptOrient && obj->orientBaselineValid)
    {
        float dOri = fabsf(wrapToPi(c->orientation - obj->lastValidOrientation));
        if (contourDetectOrientationJumpAllowed(dOri) == 0u) acceptOrient = 0u;
    }

    omTrackingUpdateVelocity(obj, (float) selCx, (float) selCy, sampleDts);
    obj->lastValidCenterX = selCx;
    obj->lastValidCenterY = selCy;
    obj->lastValidSize = selSize;

    if (obj->trackingState == OM_TRACK_LOCKED)
    {
        omTrackingLowPass(&obj->centerXFilter, (float) selCx, OM_POSITION_ALPHA);
        omTrackingLowPass(&obj->centerYFilter, (float) selCy, OM_POSITION_ALPHA);
    }
}

```

```

else
{
    lowPassFilterReset(&obj->centerXFilter, (float) selCx);
    lowPassFilterReset(&obj->centerYFilter, (float) selCy);
}

if (!acceptOrient)
{
    obj->orientAcquireFrameCount = 0u;
    obj->orientRejectFrameCount = (obj->orientRejectFrameCount < 255u) ? obj->orientRejectFrameCount + 1u : 255u;
    if (obj->orientRejectFrameCount >= holdFrames) obj->orientBaselineValid = 0u;
    if (obj->orientBaselineValid) obj->orientation = obj->orientFilter.filteredValue;
    return;
}

obj->orientRejectFrameCount = 0u;
if (!obj->orientBaselineValid)
{
    obj->orientAcquireFrameCount = (obj->orientAcquireFrameCount < 255u) ? obj->orientAcquireFrameCount + 1u : 255u;
    lowPassFilterReset(&obj->orientFilter, c->orientation);
    if (obj->orientAcquireFrameCount >= holdFrames)
    {
        obj->orientBaselineValid = 1u;
        obj->orientAcquireFrameCount = 0u;
        obj->lastValidOrientation = c->orientation;
        obj->orientation = obj->orientFilter.filteredValue;
    }
    return;
}

obj->lastValidOrientation = c->orientation;
if (obj->trackingState == OM_TRACK_LOCKED)
{
    omTrackingAngleLowPass(&obj->orientFilter, c->orientation, OM_ORIENTATION_ALPHA);
}
else
{
    lowPassFilterReset(&obj->orientFilter, c->orientation);
}
obj->orientation = obj->orientFilter.filteredValue;
}

static void binaryCcHandleLost(CONTOUR_DETECTOR_STRUCT* obj)
{
    obj->targetLost = 1u;
    obj->lockFrameCount = 0u;
    obj->lostFrameCount = (obj->lostFrameCount < 255u) ? obj->lostFrameCount + 1u : 255u;
    obj->orientationConfidence = 0.0f;
    obj->orientRejectFrameCount = (obj->orientRejectFrameCount < 255u) ? obj->orientRejectFrameCount + 1u : 255u;
    if (obj->orientRejectFrameCount >= ((binaryCcConfig.orientationHoldFrames == 0u) ? 1u : binaryCcConfig.orientationHoldFrames))
        obj->orientBaselineValid = 0u;
    omTrackingResetVelocity(obj);
    if (obj->trackingState == OM_TRACK_LOCKED && obj->lostFrameCount >= OM_LOST_THRESHOLD) obj->trackingState =
OM_TRACK_LOST_HOLDING;
    else if (obj->trackingState == OM_TRACK_LOST_HOLDING && obj->lostFrameCount >= OM_LOST_TIMEOUT) obj->trackingState =
OM_TRACK_SEARCHING;
    else if (obj->trackingState == OM_TRACK_ACQUIRING) obj->trackingState = OM_TRACK_SEARCHING;
    if (obj->orientBaselineValid) obj->orientation = obj->orientFilter.filteredValue;
}

```

core0_cyt4bb7/project/code/vision/om_binarycc.h

SHA-256

:

1a8ce28885355325574c68e4a98ad7e1dc87f533038f4deec02b903ebf0c11e8

代码 (c)

```

#ifndef OM_BINARYCC_H
#define OM_BINARYCC_H

```

```

#include <stdint.h>

```

```

#include "openMorph.h"

```

```

typedef struct

```

```

{
    uint8_t thresholdLow;
    uint8_t thresholdHigh;
    uint16_t areaMin;
    uint16_t areaMax;
    uint8_t brightnessMin;
    float eccMax;
    float fillMin;
    float huMax;
    uint8_t orientationBrightMin;
    uint8_t orientationBorderMargin;
    float orientationConfidenceMin;
    uint8_t orientationHoldFrames;
    uint16_t referenceCenterX;
    uint16_t referenceCenterY;
} BINARYCC_CONFIG_STRUCT;

void binaryCcInit(const BINARYCC_CONFIG_STRUCT* config);
void binaryCcSetConfig(const BINARYCC_CONFIG_STRUCT* config);
void binaryCcSetTemplate(const float hu[7], float huMax);
uint16_t binaryCcDetect(const u8Image* graySrc, u8Image* maskDst, float sampleDts, CONTOUR_DETECTOR_STRUCT* obj);
uint8_t binaryCcExtractHuMoments(const CONTOUR_DETECTOR_STRUCT* obj, uint8_t contourIndex, float huOut[7]);
#endif /* OM_BINARYCC_H */

```

core0_cyt4bb7/project/code/vision/om_canny.c

SHA-256

:

4c84b294f3345b1e280a99322c1a99e614a8d3b009b2003af2ea2a6e35f095a5

代码 (c)

```

/*
 * om_canny.c - [4] 高斯模糊及 Canny 边缘检测
 *
 * 3x3 定点高斯核 / 逐行流水线 Sobel+NMS+滞后门限 边缘检测
 */

#include "om_internal.h"

// ---- define ----

#define CANNY_RING_WIDTH (OM_CANNY_RING_WIDTH)

// ---- typedef ----

typedef struct
{
    uint32_t width;
    uint32_t height;
    uint32_t pixelCount;
    uint32_t cropStartX;
    uint32_t cropStartY;
    uint32_t cropEndX;
    uint32_t cropEndY;
    uint32_t blurStartX;
    uint32_t blurStartY;
    uint32_t blurEndX;
    uint32_t blurEndY;
} gaussConfigStruct;

typedef struct
{
    uint32_t width;
    uint32_t height;
    uint32_t pixelCount;
    uint32_t sobelStartX;
    uint32_t sobelStartY;
    uint32_t sobelEndX;
    uint32_t sobelEndY;
    uint32_t nmsStartX;
    uint32_t nmsStartY;
    uint32_t nmsEndX;
    uint32_t nmsEndY;
    uint16_t thresholdLow;
    uint16_t thresholdHigh;
}

```

```

uint32_t thresholdLowSq;
uint32_t thresholdHighSq;
int32_t trackOffsets[8];
} cannyConfigStruct;

// ---- 内部变量 ----

static uint16_t gaussKernelPreValueInt0[256];
static uint16_t gaussKernelPreValueInt1[256];
static uint16_t gaussKernelPreValueInt2[256];
static gaussConfigStruct gaussConfig = {0u};
static cannyConfigStruct cannyConfig = {0u};
static uint32_t sobelMapRing[3][CANNY_RING_WIDTH] = {0};
static uint8_t sobelDirMapRing[3][CANNY_RING_WIDTH] = {0};
static uint16_t cannyEdgeQueue[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE] = {0};

// ---- 外部函数实现 ----

void gaussianInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY)
{
    // gaussianKernelTemp[0] : 四个角点对应的核值
    // gaussianKernelTemp[1] : 四个边点对应的核值
    // gaussianKernelTemp[2] : 中心点对应的核值
    float gaussianKernelTemp[3] = {0.0f, 0.0f, 0.0f};

    memset(&gaussConfig, 0, sizeof(gaussConfig));

    if ((NULL == src) || (NULL == src->ptr) || (0u == src->xLen) || (0u == src->yLen) ||
        (src->xLen > CANNY_RING_WIDTH) || ((src->xLen * src->yLen) > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE))
    {
        memset(gaussKernelPreValueInt0, 0, sizeof(gaussKernelPreValueInt0));
        memset(gaussKernelPreValueInt1, 0, sizeof(gaussKernelPreValueInt1));
        memset(gaussKernelPreValueInt2, 0, sizeof(gaussKernelPreValueInt2));
        return;
    }

    gaussConfig.width = src->xLen;
    gaussConfig.height = src->yLen;
    gaussConfig.pixelCount = src->xLen * src->yLen;

    if (startX > endX)
    {
        uint32_t temp = startX;
        startX = endX;
        endX = temp;
    }
    if (startY > endY)
    {
        uint32_t temp = startY;
        startY = endY;
        endY = temp;
    }

    gaussConfig.cropStartX = unsignRange(startX, gaussConfig.width - 1u);
    gaussConfig.cropStartY = unsignRange(startY, gaussConfig.height - 1u);
    gaussConfig.cropEndX = range(endX, gaussConfig.cropStartX, gaussConfig.width - 1u);
    gaussConfig.cropEndY = range(endY, gaussConfig.cropStartY, gaussConfig.height - 1u);

    gaussConfig.blurStartX = 1u;
    gaussConfig.blurStartY = 1u;
    gaussConfig.blurEndX = 0u;
    gaussConfig.blurEndY = 0u;

    if ((gaussConfig.width >= 3u) && (gaussConfig.height >= 3u) &&
        ((gaussConfig.cropEndX - gaussConfig.cropStartX) >= 2u) &&
        ((gaussConfig.cropEndY - gaussConfig.cropStartY) >= 2u))
    {
        gaussConfig.blurStartX = (0u == gaussConfig.cropStartX) ? 1u : (gaussConfig.cropStartX + 1u);
        gaussConfig.blurStartY = (0u == gaussConfig.cropStartY) ? 1u : (gaussConfig.cropStartY + 1u);
        gaussConfig.blurEndX = (gaussConfig.cropEndX >= (gaussConfig.width - 1u)) ? (gaussConfig.width - 2u) : (gaussConfig.cropEndX - 1u);
        gaussConfig.blurEndY = (gaussConfig.cropEndY >= (gaussConfig.height - 1u)) ? (gaussConfig.height - 2u) : (gaussConfig.cropEndY - 1u);
    }

    if (sigma <= 0.0f)
    {
        memset(gaussKernelPreValueInt0, 0, sizeof(gaussKernelPreValueInt0));
        memset(gaussKernelPreValueInt1, 0, sizeof(gaussKernelPreValueInt1));
        memset(gaussKernelPreValueInt2, 0, sizeof(gaussKernelPreValueInt2));
        return;
    }
}

```

```

}

float sigma2 = sigma * sigma;
float coeff = 1.0f / (TWO_PI * sigma2);
float sum = 0.0f;

// 生成 3x3 高斯核，并提取 3 个唯一值（角点/边点/中心）
for (uint8_t y = 0u; y < 3; y++)
{
    for (uint8_t x = 0u; x < 3; x++)
    {
        int16_t dx = (int16_t) x - 1;
        int16_t dy = (int16_t) y - 1;
        float exponent = -((float) (dx * dx + dy * dy)) / (2.0f * sigma2);
        float value = (float) exp((double) exponent) * coeff;

        if (0 == dx && 0 == dy) gaussianKernelTemp[2] = value;
        else if (0 == dx || 0 == dy) gaussianKernelTemp[1] = value;
        else gaussianKernelTemp[0] = value;

        sum += value;
    }
}

if (sum <= 0.0f)
{
    memset(gaussKernelPreValueInt0, 0, sizeof(gaussKernelPreValueInt0));
    memset(gaussKernelPreValueInt1, 0, sizeof(gaussKernelPreValueInt1));
    memset(gaussKernelPreValueInt2, 0, sizeof(gaussKernelPreValueInt2));
    return;
}

// 归一化 3 个唯一核值
gaussianKernelTemp[0] /= sum;
gaussianKernelTemp[1] /= sum;
gaussianKernelTemp[2] /= sum;

// 预计算核值与 8bit 灰度值相乘的定点数 (放缩 256 倍)，消除内层浮点开销
for (uint16_t gray = 0u; gray < 256; gray++)
{
    gaussKernelPreValueInt0[gray] = (uint16_t) (gaussianKernelTemp[0] * (float) gray * 256.0f);
    gaussKernelPreValueInt1[gray] = (uint16_t) (gaussianKernelTemp[1] * (float) gray * 256.0f);
    gaussKernelPreValueInt2[gray] = (uint16_t) (gaussianKernelTemp[2] * (float) gray * 256.0f);
}
}

void cannyInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY, uint16_t thresholdLow, uint16_t thresholdHigh)
{
    gaussianInit(src, sigma, startX, startY, endX, endY);

    memset(&cannyConfig, 0, sizeof(cannyConfig));

    if ((NULL == src) || (NULL == src->ptr) || (0u == src->xLen) || (0u == src->yLen) ||
        (src->xLen > CANNY_RING_WIDTH) || ((src->xLen * src->yLen) > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)) return;

    cannyConfig.width = src->xLen;
    cannyConfig.height = src->yLen;
    cannyConfig.pixelCount = src->xLen * src->yLen;

    if (thresholdLow > thresholdHigh)
    {
        uint16_t temp = thresholdLow;
        thresholdLow = thresholdHigh;
        thresholdHigh = temp;
    }

    cannyConfig.thresholdLow = thresholdLow;
    cannyConfig.thresholdHigh = thresholdHigh;

    // 提前计算平方
    cannyConfig.thresholdLowSq = (uint32_t) thresholdLow * thresholdLow;
    cannyConfig.thresholdHighSq = (uint32_t) thresholdHigh * thresholdHigh;

    cannyConfig.sobelStartX = 1u;
    cannyConfig.sobelStartY = 1u;
    cannyConfig.sobelEndX = 0u;
    cannyConfig.sobelEndY = 0u;
    cannyConfig.nmsStartX = 1u;

```

```

cannyConfig.nmsStartY = 1u;
cannyConfig.nmsEndX = 0u;
cannyConfig.nmsEndY = 0u;

if ((gaussConfig.blurEndX > (gaussConfig.blurStartX + 1u)) &&
    (gaussConfig.blurEndY > (gaussConfig.blurStartY + 1u)))
{
    cannyConfig.sobelStartX = gaussConfig.blurStartX + 1u;
    cannyConfig.sobelStartY = gaussConfig.blurStartY + 1u;
    cannyConfig.sobelEndX = gaussConfig.blurEndX - 1u;
    cannyConfig.sobelEndY = gaussConfig.blurEndY - 1u;
}

if ((cannyConfig.sobelEndX > (cannyConfig.sobelStartX + 1u)) &&
    (cannyConfig.sobelEndY > (cannyConfig.sobelStartY + 1u)))
{
    cannyConfig.nmsStartX = cannyConfig.sobelStartX + 1u;
    cannyConfig.nmsStartY = cannyConfig.sobelStartY + 1u;
    cannyConfig.nmsEndX = cannyConfig.sobelEndX - 1u;
    cannyConfig.nmsEndY = cannyConfig.sobelEndY - 1u;
}

// 预先计算滞后门限追踪(连边)阶段需要的直接邻信偏移
cannyConfig.trackOffsets[0] = -(int32_t) gaussConfig.width - 1;
cannyConfig.trackOffsets[1] = -(int32_t) gaussConfig.width;
cannyConfig.trackOffsets[2] = -(int32_t) gaussConfig.width + 1;
cannyConfig.trackOffsets[3] = -1;
cannyConfig.trackOffsets[4] = 1;
cannyConfig.trackOffsets[5] = (int32_t) gaussConfig.width - 1;
cannyConfig.trackOffsets[6] = (int32_t) gaussConfig.width;
cannyConfig.trackOffsets[7] = (int32_t) gaussConfig.width + 1;
}

/* 运行时仅更新 Canny 双阈值，不重算高斯核与 ROI，便于按键调参 */
void cannySetThreshold(uint16_t thresholdLow, uint16_t thresholdHigh)
{
    if (thresholdLow > thresholdHigh)
    {
        uint16_t temp = thresholdLow;
        thresholdLow = thresholdHigh;
        thresholdHigh = temp;
    }

    cannyConfig.thresholdLow = thresholdLow;
    cannyConfig.thresholdHigh = thresholdHigh;
    cannyConfig.thresholdLowSq = (uint32_t) thresholdLow * thresholdLow;
    cannyConfig.thresholdHighSq = (uint32_t) thresholdHigh * thresholdHigh;
}

void cannyEdgeDetection(const u8Image* src, u8Image* dstGaussBlur, u8Image* dst)
{
    if (NULL == src || NULL == src->ptr || NULL == dstGaussBlur || NULL == dstGaussBlur->ptr ||
        NULL == dst || NULL == dst->ptr) return;
    if (0u == cannyConfig.pixelCount || 0u == gaussConfig.width) return;

    uint32_t queueHead = 0u;
    uint32_t queueTail = 0u;

    // 清零目标画布、ring buffer 残留、高斯模糊边界脏值
    memset(dst->ptr, BLACK, gaussConfig.width * gaussConfig.height);
    memset(dstGaussBlur->ptr, 0, gaussConfig.width * gaussConfig.height);
    memset(sobelMapRing, 0, sizeof(sobelMapRing));
    memset(sobelDirMapRing, 0, sizeof(sobelDirMapRing));

    for (uint32_t y = gaussConfig.blurStartY; y <= gaussConfig.blurEndY; y++)
    {
        // ===== 第 1 阶段: 高斯模糊 =====
        const uint8_t* row0 = src->ptr + (y - 1u) * gaussConfig.width;
        const uint8_t* row1 = src->ptr + y * gaussConfig.width;
        const uint8_t* row2 = src->ptr + (y + 1u) * gaussConfig.width;
        uint8_t* dstRow = dstGaussBlur->ptr + y * gaussConfig.width;

        for (uint32_t x = gaussConfig.blurStartX; x <= gaussConfig.blurEndX; x++)
        {
            uint32_t result = gaussKernelPreValueInt0[row0[x - 1u]] + gaussKernelPreValueInt1[row0[x]] + gaussKernelPreValueInt0[row0[x + 1u]] +
                gaussKernelPreValueInt1[row1[x - 1u]] + gaussKernelPreValueInt2[row1[x]] + gaussKernelPreValueInt1[row1[x + 1u]] +
                gaussKernelPreValueInt0[row2[x - 1u]] + gaussKernelPreValueInt1[row2[x]] + gaussKernelPreValueInt0[row2[x + 1u]];

            result = (result + 128) >> 8;

```



```

        dstRow[x] = (uint8_t) ((result > 255) ? 255 : result);
    }

// ===== 第2阶段: Sobel 计算 =====
if ((cannyConfig.sobelStartX <= cannyConfig.sobelEndX) &&
    (cannyConfig.sobelStartY <= cannyConfig.sobelEndY) &&
    (y >= (cannyConfig.sobelStartY + 1u)) &&
    ((y - 1u) <= cannyConfig.sobelEndY))
{
    const uint8_t* s_row0 = dstGaussBlur->ptr + (y - 2u) * gaussConfig.width;
    const uint8_t* s_row1 = dstGaussBlur->ptr + (y - 1u) * gaussConfig.width;
    const uint8_t* s_row2 = dstGaussBlur->ptr + y * gaussConfig.width;

    uint32_t sobelRow = sobelMapRing[(y - 1u) % 3];
    uint8_t dirRow = sobelDirMapRing[(y - 1u) % 3];

    for (uint32_t x = cannyConfig.sobelStartX; x <= cannyConfig.sobelEndX; x++)
    {
        int32_t gx = (int32_t) s_row0[x + 1u] + ((int32_t) s_row1[x + 1u] << 1) + (int32_t) s_row2[x + 1u] - (int32_t) s_row0[x - 1u] - ((int32_t) s_row1[x - 1u] << 1) - (int32_t) s_row2[x - 1u];
        int32_t gy = (int32_t) s_row0[x - 1u] + ((int32_t) s_row0[x] << 1) + (int32_t) s_row0[x + 1u] - (int32_t) s_row2[x - 1u] - ((int32_t) s_row2[x] << 1) - (int32_t) s_row2[x + 1u];

        // 去除掉极其耗时的 sqrtf 运算
        uint32_t mag_sq = (uint32_t) (gx * gx) + (uint32_t) (gy * gy);
        sobelRow[x] = mag_sq;

        // 完全消除浮点除法, 只用整数比较判定 4 大区面方向
        int32_t ax = (gx >= 0) ? gx : -gx;
        int32_t ay = (gy >= 0) ? gy : -gy;
        int32_t ay256 = ay << 8;

        if (ay256 <= ax * 106)
        {
            dirRow[x] = 0; // 水平梯度主导 (纵向边缘), 沿水平方向抑制
        }
        else if (ay256 >= ax * 618)
        {
            dirRow[x] = 2; // 垂直梯度主导 (横向边缘), 沿垂直方向抑制
        }
        else if ((gx ^ gy) >= 0)
        {
            dirRow[x] = 1; // 45 度梯度
        }
        else
        {
            dirRow[x] = 3; // 135 度梯度
        }
    }
}

// ===== 第3阶段: 非极大值抑制 =====
if ((cannyConfig.nmsStartX <= cannyConfig.nmsEndX) &&
    (cannyConfig.nmsStartY <= cannyConfig.nmsEndY) &&
    (y >= (cannyConfig.nmsStartY + 2u)) &&
    ((y - 2u) <= cannyConfig.nmsEndY))
{
    const uint32_t* n_row0 = sobelMapRing[(y - 3u) % 3];
    const uint32_t* n_row1 = sobelMapRing[(y - 2u) % 3];
    const uint32_t* n_row2 = sobelMapRing[(y - 1u) % 3];
    const uint8_t dirRow = sobelDirMapRing[(y - 2u) % 3];

    uint8_t* final_dstRow = dst->ptr + (y - 2u) * gaussConfig.width + cannyConfig.nmsStartX;

    for (uint32_t x = cannyConfig.nmsStartX; x <= cannyConfig.nmsEndX; x++)
    {
        uint32_t edge0, edge1;
        uint32_t magnitude = n_row1[x];
        uint8_t dir = dirRow[x];

        if (dir == 0)
        {
            edge0 = n_row1[x - 1u];
            edge1 = n_row1[x + 1u];
        }
        else if (dir == 2)
        {
            edge0 = n_row0[x];
            edge1 = n_row2[x];
        }
    }
}

```

```

    }
    else if (dir == 1)
    {
        // gx, gy 同号 → 梯度方向 ↗ (亮度梯度指向右上) → NMS 比较 ↗ 与 ↘
        edge0 = n_row0[x + 1u];
        edge1 = n_row2[x - 1u];
    }
    else
    {
        // gx, gy 异号 → 梯度方向 ↘ (亮度梯度指向右下) → NMS 比较 ↘ 与 ↗
        edge0 = n_row0[x - 1u];
        edge1 = n_row2[x + 1u];
    }

    if ((magnitude > edge0) && (magnitude >= edge1))
    {
        if (magnitude >= cannyConfig.thresholdHighSq)
        {
            *final_dstRow = WHITE; // 确信强边缘
            if (queueTail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
            {
                cannyEdgeQueue[queueTail++] = (uint16_t) ((y - 2u) * gaussConfig.width + x);
            }
        }
        else if (magnitude >= cannyConfig.thresholdLowSq)
        {
            *final_dstRow = 127; // 先标记为弱边缘，用于第四阶段追踪，避免大数组分配
        }
    }

    final_dstRow++;
}
}

// ===== 第4阶段: 滞后门限追踪(连边) =====
while (queueHead < queueTail)
{
    uint32_t index = cannyEdgeQueue[queueHead++];

    for (uint8_t i = 0; i < 8; i++)
    {
        int32_t trackIndex = (int32_t) index + cannyConfig.trackOffsets[i];

        if (trackIndex >= 0 && trackIndex < (int32_t) cannyConfig.pixelCount)
        {
            if (dst->ptr[trackIndex] == 127)
            {
                dst->ptr[trackIndex] = WHITE; // 连通升级为强边缘

                // 防溢出保护
                if (queueTail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
                {
                    cannyEdgeQueue[queueTail++] = (uint16_t) trackIndex;
                }
            }
        }
    }
}

// ===== 收尾: 清理死路 =====
uint32_t totalPixels = gaussConfig.width * gaussConfig.height;
for (uint32_t i = 0; i < totalPixels; i++)
{
    if (dst->ptr[i] == 127) dst->ptr[i] = BLACK; // 无法与强边缘连通被消除
}
}

```

core0_cyt4bb7/project/code/vision/om_contour.c

SHA-256

:

f7d57c64c8ff5447dfbca9699ee6ddb5bab43e1007d074c68b7f53420448c8c2

代码 (c)

```

/*
 * om_contour.c - [5] 轮廓提取
 *
 * 8 邻域 BFS 闭合轮廓检测、Hu 矩特征匹配与几何特征
 * 本文件同时对外暴露 om_beacon.c 共用的 BFS 基础设施 (inContourMap 等)
 */

#include "om_internal.h"
#include "om_tracking_internal.h"

// ---- define ----

#define CONTOUR_ECC_MIN (1.5f)
#define CONTOUR_ECC_MAX (6.0f) /* 偏心率上限: ^ 笔画粗 ecc<=4, 细长开放曲线 ecc>=20 */
/* CONTOUR_HU_TOPK 已废弃: 现对所有候选全量计算 Hu, 不做 Top-K 预筛 */
#define CONTOUR_USE_FAST_LOG (1u)
#define CONTOUR_RAY_FAR_START_RATIO (0.28f)
#define CONTOUR_RAY_FAR_END_RATIO (0.96f)
#define CONTOUR_RAY_CONE_RATIO (0.24f)
#define CONTOUR_RAY_CONE_MIN (1.5f)
#define CONTOUR_RAY_RADIUS_SPLIT (24.0f)
#define CONTOUR_RAY_HIT_WEIGHT (2.0f)
#define CONTOUR_RAY_NEAR_WEIGHT (0.08f)
#define CONTOUR_RAY_OFF_PENALTY (0.05f)

// ---- typedef ----

typedef struct
{
    uint16_t sizeMax;
    uint16_t sizeMin;

    uint16_t visionCenterX; // 视觉中心 X
    uint16_t visionCenterY; // 视觉中心 Y

    uint16_t roiRadius; // ROI 检测半径 (0=全图)
    uint16_t earlyStopRadius; // 质心距视觉中心 <= 此值时立即停止
    uint32_t earlyStopRadiusSq;

    uint8_t centroidFeature; // 0: 质心在内 1: 质心在外

    uint8_t huTemplateReady; // Hu 模板是否有效: 1=已设置, 0=未设置

    float huTemplate[7]; // 目标形状 Hu 矩模板
    float huLostThreshold; // 目标丢失 Hu 距离阈值 (选中轮廓 Hu 距离>阈值 → targetLost)
    uint8_t brightnessMax; // 质心亮度允许上限 (255=禁用, ^ 笔画位于暗背景上)
} CONTOUR_DATA_CONFIG;

// =====
// 共用 BFS 基础设施 (om_beacon.c 通过 extern 复用)
// =====

uint32_t inContourMap[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE];
uint16_t contourGeneration = 0;
const int8_t contourDx[8] = {0, 1, 1, 1, 0, -1, -1, -1};
const int8_t contourDy[8] = {-1, -1, 0, 1, 1, 1, 0, -1};
uint32_t contourQueue[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE];

// ---- 内部数据 ----

static CONTOUR_DATA_CONFIG contourDataConfig;

// ---- 轮廓选择缓冲区 (内存换性能: 全帧候选对应 Hu 特征) ----
static float contourHuBuf[OM_MAX_OBJECTS]; // 每条轮廓的 Hu 加权匹配距离
static uint32_t contourDistSqBuf[OM_MAX_OBJECTS]; // 每条轮廓质心距视觉中心的距离平方
static float contourMomBuf[OM_MAX_OBJECTS * 7]; // 每条轮廓的中心矩 [mxx,myy,mxy,m30,m03,m21,m12]
static float contourOrientBuf[OM_MAX_OBJECTS]; // 朝向: ^ 尖端几何射线定向 (-PI ~ PI)
static float contourOrientConfBuf[OM_MAX_OBJECTS]; /* 尖端几何射线定向置信度 */

/* 朝向跳变保护运行时阈值, 初值取 openMorph_config.h 宏 */
static float omOrientJumpMax = OM_ORIENTATION_JUMP_MAX;
static float omOrientRecoverRad = OM_ORIENTATION_RECOVER_RAD;
static uint8_t omOrientConfirmFrames = OM_ORIENTATION_CONFIRM_FRAMES;

/* Hu 选择策略: candidate 与最佳 Hu 距离差 < CONTOUR_HU_TIE_THRESHOLD 时
 * 使用位置距离 tiebreak; 收紧则更依赖 Hu */
#define CONTOUR_HU_TIE_THRESHOLD 1.0f

// ---- 内部函数声明 ----

```

```

static inline uint16_t eightNeighborhoodIterativeCall(const u8Image* obj, uint16_t startX, uint16_t startY, uint32_t genTag, CONTOUR_DATA_STRUCT*
contour, float* outMxx, float* outMyy, float* outMxy, float* outM30, float* outM03, float* outM21, float* outM12, float* outOrientation, float*
outOrientConfidence);
static float contourComputeRayBalanceOrientation(const u8Image* obj, uint32_t genTag, const uint32_t* queue, uint16_t count, float meanX, float meanY,
float mxx, float myy, float mxy, float* outConfidence);
static float contourRayBalanceSideScore(const u8Image* obj, uint32_t genTag, const uint32_t* queue, uint16_t count, float meanX, float meanY, float axisC,
float axisS, uint8_t positiveSide, float extent, float coneWidth, uint8_t rayRadius);
static uint8_t contourRayHasTaggedPixel(const u8Image* obj, uint32_t genTag, int32_t rx, int32_t ry, uint8_t radius);
static int32_t contourRoundToInt(float v);
static void computeHuMoments(float mxx, float myy, float mxy, float m30, float m03, float m21, float m12, uint16_t n, float hu[7]);
static float contourHuDistanceByIndex(const CONTOUR_DETECTOR_STRUCT* obj, uint16_t contourIdx);
static float huMatchDistance(const float huA[7], const float huB[7]);
static inline float fastLogPositive(float x);
static uint32_t contourCandidateReferenceDistSq(const CONTOUR_DETECTOR_STRUCT* obj, uint16_t contourIdx);
static void contourHandleLostSample(CONTOUR_DETECTOR_STRUCT* obj, uint8_t resetOrientProtect);

/* 跟踪建立后优先保持上一目标连续性；搜索态才回退到固定图像中心。 */
static uint32_t contourCandidateReferenceDistSq(const CONTOUR_DETECTOR_STRUCT* obj, uint16_t contourIdx)
{
    if (obj->lastValidSize == 0u || obj->trackingState == OM_TRACK_SEARCHING)
    {
        return contourDistSqBuf[contourIdx];
    }

    int32_t dx = (int32_t) obj->contourData[contourIdx].centerX - (int32_t) obj->lastValidCenterX;
    int32_t dy = (int32_t) obj->contourData[contourIdx].centerY - (int32_t) obj->lastValidCenterY;
    return (uint32_t) (dx * dx + dy * dy);
}

static void contourHandleLostSample(CONTOUR_DETECTOR_STRUCT* obj, uint8_t resetOrientProtect)
{
    obj->lockFrameCount = 0u;
    obj->lostFrameCount = (obj->lostFrameCount < 255u) ? obj->lostFrameCount + 1u : 255u;
    if (resetOrientProtect)
    {
        obj->orientProtectState = OM_ORIENT_NORMAL;
        obj->orientSpikeFrameCount = 0u;
    }
    omTrackingResetVelocity(obj);
    obj->orientAcquireFrameCount = 0u;
    obj->orientRejectFrameCount = (obj->orientRejectFrameCount < 255u) ? obj->orientRejectFrameCount + 1u : 255u;
    if (obj->orientRejectFrameCount >= ((omOrientConfirmFrames == 0u) ? 1u : omOrientConfirmFrames))
    {
        obj->orientBaselineValid = 0u;
    }

    switch (obj->trackingState)
    {
        case OM_TRACK_SEARCHING:
            break;
        case OM_TRACK_ACQUIRING:
            obj->trackingState = OM_TRACK_SEARCHING;
            break;
        case OM_TRACK_LOCKED:
            if (obj->lostFrameCount >= OM_LOST_THRESHOLD)
                obj->trackingState = OM_TRACK_LOST_HOLDING;
            break;
        case OM_TRACK_LOST_HOLDING:
            if (obj->lostFrameCount >= OM_LOST_TIMEOUT)
                obj->trackingState = OM_TRACK_SEARCHING;
            break;
        default:
            obj->trackingState = OM_TRACK_SEARCHING;
            break;
    }

    obj->orientation = obj->orientBaselineValid ? obj->orientFilter.filteredValue : 0.0f;
}

// ---- 外部函数实现 ----

void contourDetectInit(uint16_t sizeMin, uint16_t sizeMax, uint8_t centroidFeature, uint16_t visionCenterX, uint16_t visionCenterY, uint16_t roiRadius,
uint16_t earlyStopRadius, uint8_t brightnessMax)
{
    contourDataConfig.sizeMin = sizeMin;
    contourDataConfig.sizeMax = sizeMax;
    contourDataConfig.centroidFeature = centroidFeature;
    contourDataConfig.visionCenterX = visionCenterX;

```

```

    contourDataConfig.visionCenterY = visionCenterY;
    contourDataConfig.roiRadius = roiRadius;
    contourDataConfig.earlyStopRadius = earlyStopRadius;
    contourDataConfig.brightnessMax = brightnessMax;
    contourDataConfig.earlyStopRadiusSq = (uint32_t) earlyStopRadius * earlyStopRadius;
    contourDataConfig.huTemplateReady = 0u;
    contourDataConfig.huLostThreshold = 0.0f;
}

void contourDetectSetSizeRange(uint16_t sizeMin, uint16_t sizeMax)
{
    contourDataConfig.sizeMin = sizeMin;
    contourDataConfig.sizeMax = sizeMax;
}

void contourDetectSetBrightnessMax(uint8_t brightnessMax)
{
    contourDataConfig.brightnessMax = brightnessMax;
}

void contourDetectSetOrientProtect(float jumpMax, float recoverRad, uint8_t confirmFrames)
{
    omOrientJumpMax = jumpMax;
    omOrientRecoverRad = recoverRad;
    omOrientConfirmFrames = confirmFrames;
}

uint8_t contourDetectOrientationJumpAllowed(float deltaRad)
{
    return (deltaRad <= omOrientJumpMax) ? 1u : 0u;
}

void contourDetectSetTemplate(const float* huMoments, float huLostThreshold)
{
    if (NULL == huMoments)
    {
        for (uint8_t i = 0; i < 7; i++) contourDataConfig.huTemplate[i] = 0.0f;
        contourDataConfig.huTemplateReady = 0u;
    }
    else
    {
        for (uint8_t i = 0; i < 7; i++) contourDataConfig.huTemplate[i] = huMoments[i];
        contourDataConfig.huTemplateReady = 1u;
    }
    contourDataConfig.huLostThreshold = huLostThreshold;
}

uint16_t contourDetect(const u8Image* src, const u8Image* graySrc, float sampleDtS, CONTOUR_DETECTOR_STRUCT* obj)
{
    if (src->size > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
    {
        obj->contourCount = 0;
        obj->selectedIndex = 0;
        obj->targetLost = 1;
        obj->orientation = 0.0f;
        obj->orientationConfidence = 0.0f;
        omTrackingResetVelocity(obj);
        return 0;
    }

    // generation 替代 memset: 每帧自增, 匹配当前 generation 才视为已访问
    contourGeneration++;
    if (contourGeneration == 0)
    {
        memset(inContourMap, 0, sizeof(inContourMap));
        contourGeneration = 1;
    }
    uint32_t genBase = (uint32_t) contourGeneration << 16;

    uint16_t contourCount = 0;
    uint32_t tagCounter = 0;
    obj->selectedIndex = 0;
    obj->orientation = 0.0f;
    obj->orientationConfidence = 0.0f; /* 每帧起底 0: 无候选/目标本体丢失时保持 0 */
    obj->targetLost = 1;

    /* huEnabled 仅以模板是否就绪为准 */
    uint8_t huEnabled = (contourDataConfig.huTemplateReady) ? 1u : 0u;

```

```

// ROI 边界
uint32_t roiStartX = 0, roiStartY = 0;
uint32_t roiEndX = src->xLen - 1, roiEndY = src->yLen - 1;
if (contourDataConfig.roiRadius > 0)
{
    int32_t rs = (int32_t) contourDataConfig.visionCenterX - (int32_t) contourDataConfig.roiRadius;
    int32_t re = (int32_t) contourDataConfig.visionCenterX + (int32_t) contourDataConfig.roiRadius;
    int32_t ts = (int32_t) contourDataConfig.visionCenterY - (int32_t) contourDataConfig.roiRadius;
    int32_t te = (int32_t) contourDataConfig.visionCenterY + (int32_t) contourDataConfig.roiRadius;
    roiStartX = (rs < 0) ? 0 : (uint32_t) rs;
    roiStartY = (ts < 0) ? 0 : (uint32_t) ts;
    roiEndX = ((uint32_t) re >= src->xLen) ? src->xLen - 1 : (uint32_t) re;
    roiEndY = ((uint32_t) te >= src->yLen) ? src->yLen - 1 : (uint32_t) te;
}

// ===== 第一阶段: 扫描全部轮廓并收集特征 =====
for (uint32_t y = roiStartY; y <= roiEndY && contourCount < OM_MAX_OBJECTS - 1; y++)
{
    uint32_t rowOff = y * src->xLen;
    for (uint32_t x = roiStartX; x <= roiEndX && contourCount < OM_MAX_OBJECTS - 1; x++)
    {
        uint32_t i = rowOff + x;
        if (src->ptr[i] != WHITE || (inContourMap[i] >> 16) == contourGeneration) continue;

        tagCounter++;
        if (tagCounter > 0xFFFFu) goto phaseTwoSelect;

        uint32_t genTag = genBase | (uint32_t) tagCounter;

        float curMxx = 0.0f, curMyy = 0.0f, curMxy = 0.0f;
        float curM30 = 0.0f, curM03 = 0.0f, curM21 = 0.0f, curM12 = 0.0f;
        uint16_t endpointCount = eightNeighborhoodIterativeCall(src, (uint16_t) x, (uint16_t) y, genTag, &obj->contourData[contourCount], &curMxx,
        &curMyy, &curMxy, &curM30, &curM03, &curM21, &curM12, &contourOrientBuf[contourCount], &contourOrientConfBuf[contourCount]);

        /* 闭合判定收紧: 真闭环=0 端点, ^=2 端点; 其余开放噪声曲线一律弃 */
        if (endpointCount != 0u && endpointCount != 2u) continue;

        /* BFS 内 abandon (触边/超尺寸/超端点) 时 contourSize 已被置 0, 下方 sizeMin 检查会拒绝 */

        // 尺寸过滤
        uint16_t sz = obj->contourData[contourCount].contourSize;
        if (sz < contourDataConfig.sizeMin || sz > contourDataConfig.sizeMax) continue;

        // ^形状几何预筛: 偏心率检查 (^ 主轴比/次轴比 在[1.5, 6])
        float eigSum = curMxx + curMyy;
        float eigDiff2 = (curMxx - curMyy) * (curMxx - curMyy) + 4.0f * curMxy * curMxy;
        if (eigSum > 0.0f && eigDiff2 > 0.0f)
        {
            // 免 sqrt 偏心率判定:
            // eigMax/eigMin < r <=> diff^2 < sum^2 * ((r-1)/(r+1))^2
            // eigMax/eigMin > r <=> diff^2 > sum^2 * ((r-1)/(r+1))^2
            const float ratioKMin = (CONTOUR_ECC_MIN - 1.0f) / (CONTOUR_ECC_MIN + 1.0f);
            const float ratioKMax = (CONTOUR_ECC_MAX - 1.0f) / (CONTOUR_ECC_MAX + 1.0f);
            float boundMin = eigSum * ratioKMin;
            float boundMax = eigSum * ratioKMax;
            if (eigDiff2 < boundMin * boundMin) continue;
            if (eigDiff2 > boundMax * boundMax) continue;
        }

        // 质心亮度上限过滤: ^笔画必处于暗背景上, 中心过亮必非^
        if (contourDataConfig.brightnessMax < 255)
        {
            uint16_t ccx = obj->contourData[contourCount].centerX;
            uint16_t ccy = obj->contourData[contourCount].centerY;
            uint8_t centroidBright = graySrc->ptr[(uint32_t) ccy * graySrc->xLen + (uint32_t) ccx];
            if (centroidBright > contourDataConfig.brightnessMax) continue;
        }

        // 质心内外特征 (射线法: 由质心向右发射水平射线, 统计与本轮廓边缘的交点奇偶)
        if (contourDataConfig.centroidFeature <= 1)
        {
            uint16_t crossings = 0;
            uint8_t prevEdge = 0;
            for (uint16_t cx = obj->contourData[contourCount].centerX + 1; cx < src->xLen; cx++)
            {
                uint16_t idx = (uint16_t) (obj->contourData[contourCount].centerY * src->xLen + cx);
                uint8_t isEdge = ((inContourMap[idx] & 0xFFFFu) == tagCounter && (inContourMap[idx] >> 16) == contourGeneration) ? 1u : 0u;
                if (isEdge && !prevEdge) crossings++;
                prevEdge = isEdge;
            }
        }
    }
}

```

```

    }
    uint8_t centroidInside = crossings & 1u;
    if (contourDataConfig.centroidFeature == 0 && !centroidInside) continue;
    if (contourDataConfig.centroidFeature == 1 && centroidInside) continue;
}

// 存储距离平方
int32_t dx = (int32_t) obj->contourData[contourCount].centerX - (int32_t) contourDataConfig.visionCenterX;
int32_t dy = (int32_t) obj->contourData[contourCount].centerY - (int32_t) contourDataConfig.visionCenterY;
contourDistSqBuf[contourCount] = (uint32_t) (dx * dx + dy * dy);

// 存储中心矩 (第二阶段成败选择需要)
uint32_t mBase = (uint32_t) contourCount * 7u;
contourMomBuf[mBase] = curMxx;
contourMomBuf[mBase + 1] = curMyy;
contourMomBuf[mBase + 2] = curMxy;
contourMomBuf[mBase + 3] = curM30;
contourMomBuf[mBase + 4] = curM03;
contourMomBuf[mBase + 5] = curM21;
contourMomBuf[mBase + 6] = curM12;

// Hu 距离在第二阶段进入 Top-K 候选后才算, 第一阶段仅收集几何特征
obj->contourData[contourCount].huDistance = 0.0f;

contourCount++;

// 非 Hu 模式下按当前选择参考 early-stop; 跟踪态靠近上一目标, 搜索态靠近画面中心。
if (!huEnabled && contourDataConfig.earlyStopRadius > 0
    && contourCandidateReferenceDistSq(obj, contourCount - 1u) <= contourDataConfig.earlyStopRadiusSq)
    goto phaseTwoSelect;
}
}

phaseTwoSelect:
obj->contourCount = contourCount;

// ===== 第二阶段: 在已收集轮廓中选出最佳轮廓 =====
if (contourCount > 0)
{
    if (huEnabled && contourCount > 1)
    {
        // Hu-first: 对所有候选计算 Hu 距离, 不做 Top-K 预筛
        float huMin = 3.402823 466e+38f;
        for (uint16_t k = 0; k < contourCount; k++)
        {
            float dHu = contourHuDistanceByIndex(obj, k);
            contourHuBuf[k] = dHu;
            obj->contourData[k].huDistance = dHu;
            if (dHu < huMin) huMin = dHu;
        }

        /* Hu 优先选择: 最小 Hu; 同 Hu 带内 (差 < TIE_THRESHOLD) 按位置 tiebreak */
        uint32_t bestTieDist = 0xFFFFFFFFu;
        for (uint16_t k = 0; k < contourCount; k++)
        {
            if ((contourHuBuf[k] - huMin) < CONTOUR_HU_TIE_THRESHOLD)
            {
                uint32_t referenceDist = contourCandidateReferenceDistSq(obj, k);
                if (referenceDist < bestTieDist)
                {
                    bestTieDist = referenceDist;
                    obj->selectedIndex = (uint8_t) k;
                }
            }
        }
    }
    else if (!huEnabled)
    {
        // 纯距离选择
        uint32_t bestDist = contourCandidateReferenceDistSq(obj, 0u);
        obj->selectedIndex = 0;
        for (uint16_t k = 1; k < contourCount; k++)
        {
            uint32_t referenceDist = contourCandidateReferenceDistSq(obj, k);
            if (referenceDist < bestDist)
            {
                bestDist = referenceDist;
                obj->selectedIndex = (uint8_t) k;
            }
        }
    }
}

```

```

    }
}
// contourCount==1 时 selectedIndex 已为 0

// 目标丢失判定: Hu 距离超阈值 → 形状不匹配模板
if (huEnabled && contourDataConfig.huLostThreshold > 0.0f)
{
    float selHu = contourHuDistanceByIndex(obj, obj->selectedIndex);
    contourHuBuf[obj->selectedIndex] = selHu;
    obj->contourData[obj->selectedIndex].huDistance = selHu;
    obj->targetLost = (selHu > contourDataConfig.huLostThreshold) ? 1u : 0u;
}
else
{
    obj->targetLost = 0;
}

/* 选中轮廓朝向: BFS 末尾用 ^ 几何特征定向, 无跨帧状态 */
obj->orientation = contourOrientBuf[obj->selectedIndex];
}

// 清空后段未使用位置
for (uint16_t i = contourCount; i < OM_MAX_OBJECTS; i++) obj->contourData[i].contourSize = 0;

// ===== 跟踪状态机 =====
{
    uint8_t detected = (obj->targetLost == 0u && contourCount > 0u) ? 1u : 0u;
    uint16_t selCx = obj->contourData[obj->selectedIndex].centerX;
    uint16_t selCy = obj->contourData[obj->selectedIndex].centerY;
    float selOrient = contourOrientBuf[obj->selectedIndex];
    float selOrientConf = contourOrientConfBuf[obj->selectedIndex];
    uint16_t selSize = obj->contourData[obj->selectedIndex].contourSize;

    // 画幅边缘接触检测: 质心过于靠近边缘视为丢失
    if (detected)
    {
        if (selCx <= OM_BORDER_MARGIN || selCx >= (src->xLen - 1u - OM_BORDER_MARGIN) ||
            selCy <= OM_BORDER_MARGIN || selCy >= (src->yLen - 1u - OM_BORDER_MARGIN))
        {
            detected = 0u;
            obj->targetLost = 1u;
        }
    }
    if (detected)
    {
        obj->orientationConfidence = selOrientConf;
        obj->lostFrameCount = 0u;
        obj->lockFrameCount = (obj->lockFrameCount < 255u) ? obj->lockFrameCount + 1u : 255u;

        switch (obj->trackingState)
        {
            case OM_TRACK_SEARCHING:
                if (obj->lockFrameCount >= OM_LOCK_THRESHOLD)
                    obj->trackingState = OM_TRACK_ACQUIRING;
                break;
            case OM_TRACK_ACQUIRING:
                if (obj->lockFrameCount >= (OM_LOCK_THRESHOLD + OM_LOCK_CONFIRM))
                    obj->trackingState = OM_TRACK_LOCKED;
                break;
            case OM_TRACK_LOCKED:
                break;
            case OM_TRACK_LOST_HOLDING:
                obj->trackingState = OM_TRACK_LOCKED;
                break;
            default:
                obj->trackingState = OM_TRACK_SEARCHING;
                break;
        }
    }

    // 主轴方向(PCA)实时跟随; 低置信只拒绝朝向样本, 不影响目标位置有效性。
    // 小目标仍按上一帧对齐 180 度, 不冻结主轴角。
    float finalOrient = selOrient;
    if (obj->trackingState >= OM_TRACK_ACQUIRING && obj->orientBaselineValid)
    {
        if (selSize < OM_ORIENTATION_MIN_SIZE)
        {
            float diff = selOrient - obj->lastValidOrientation;
            diff = wrapToPi(diff);
            // selOrient 与上一帧反向(尖端选反) -> 翻转 180 度对齐, 主轴角仍随时

```



```

        if (diff > HALF_PI)
            finalOrient = selOrient - PI;
        else if (diff < -HALF_PI)
            finalOrient = selOrient + PI;
        finalOrient = wrapToPi(finalOrient);
    }
}

// 朝向跳变保护 (逻辑同 ToF 定高跳变保护): 几何对齐后再判定 finalOrient
// 相对上一基线的圆周跳变。仅 ACQUIRING/LOCKED 且已有可信基线时生效。
// acceptSample=0 时拒绝当前朝向样本: 位置目标仍有效, 朝向沿用上一滤波值。
uint8_t acceptSample = 1u;
uint8_t hasOrientBaseline = obj->orientBaselineValid ? 1u : 0u;
if (selOrientConf < OM_RAY_BALANCE_CONFIDENCE_MIN)
{
    acceptSample = 0u;
}

if (acceptSample && obj->trackingState >= OM_TRACK_ACQUIRING && hasOrientBaseline)
{
    float dOri = finalOrient - obj->lastValidOrientation;
    dOri = wrapToPi(dOri);
    if (dOri < 0.0f) dOri = -dOri;

    if (obj->orientProtectState == OM_ORIENT_NORMAL)
    {
        if (dOri > omOrientJumpMax)
        {
            // 异常突跳 -> 挂起, 拒绝该样本
            obj->orientProtectState = OM_ORIENT_SUSPENDED;
            obj->orientSpikeFrameCount = 0u;
            acceptSample = 0u;
        }
    }
    else /* OM_ORIENT_SUSPENDED */
    {
        if (dOri > omOrientJumpMax)
        {
            // 持续异常: 计数清零, 继续沿用基线
            obj->orientSpikeFrameCount = 0u;
            acceptSample = 0u;
        }
        else if (dOri < omOrientRecoverRad)
        {
            // 窄带恢复: 接受当前朝向
            obj->orientProtectState = OM_ORIENT_NORMAL;
        }
        else
        {
            // 介于二者之间: 累计确认帧, 达阈值后接受新基线
            obj->orientSpikeFrameCount = (obj->orientSpikeFrameCount < 255u) ? obj->orientSpikeFrameCount + 1u : 255u;
            if (obj->orientSpikeFrameCount >= omOrientConfirmFrames)
            {
                obj->orientProtectState = OM_ORIENT_NORMAL;
                obj->orientSpikeFrameCount = 0u;
            }
            else
            {
                acceptSample = 0u;
            }
        }
    }
}

}

if (!acceptSample)
{
    /* 朝向质量不足只冻结朝向基线; 目标质心/速度仍按本帧更新。 */
    omTrackingUpdateVelocity(obj, (float) selCx, (float) selCy, sampleDtS);

    obj->lastValidCenterX = selCx;
    obj->lastValidCenterY = selCy;
    obj->lastValidSize = selSize;
    obj->orientAcquireFrameCount = 0u;
    obj->orientRejectFrameCount = (obj->orientRejectFrameCount < 255u) ? obj->orientRejectFrameCount + 1u : 255u;
    if (obj->orientRejectFrameCount >= ((omOrientConfirmFrames == 0u) ? 1u : omOrientConfirmFrames))
    {
        obj->orientBaselineValid = 0u;
    }
}

```

```

        if (obj->trackingState == OM_TRACK_LOCKED)
        {
            omTrackingLowPass(&obj->centerXFilter, (float) selCx, OM_POSITION_ALPHA);
            omTrackingLowPass(&obj->centerYFilter, (float) selCy, OM_POSITION_ALPHA);
        }
        else
        {
            lowPassFilterReset(&obj->centerXFilter, (float) selCx);
            lowPassFilterReset(&obj->centerYFilter, (float) selCy);
        }

        obj->orientation = obj->orientBaselineValid ? obj->orientFilter.filteredValue : 0.0f;
    }
    else
    {
        /* 先用本帧原始整数质心更新速度；位置发布与速度低通相互独立。 */
        omTrackingUpdateVelocity(obj, (float) selCx, (float) selCy, sampleDtS);
        uint8_t hadOrientBaseline = obj->orientBaselineValid ? 1u : 0u;
        uint8_t baselineConfirmFrames = (omOrientConfirmFrames == 0u) ? 1u : omOrientConfirmFrames;

        obj->lastValidCenterX = selCx;
        obj->lastValidCenterY = selCy;
        obj->lastValidOrientation = finalOrient;
        obj->lastValidSize = selSize;
        obj->orientRejectFrameCount = 0u;
        if (hadOrientBaseline)
        {
            obj->orientAcquireFrameCount = 0u;
        }
        else
        {
            obj->orientAcquireFrameCount = (obj->orientAcquireFrameCount < 255u) ? obj->orientAcquireFrameCount + 1u : 255u;
            if (obj->orientAcquireFrameCount >= baselineConfirmFrames)
            {
                obj->orientBaselineValid = 1u;
                obj->orientAcquireFrameCount = 0u;
            }
        }
    }

    // 低通滤波 (仅 LOCKED 态), 系数由 OM_*_ALPHA 宏调节
    if (obj->trackingState == OM_TRACK_LOCKED && hadOrientBaseline)
    {
        omTrackingLowPass(&obj->centerXFilter, (float) selCx, OM_POSITION_ALPHA);
        omTrackingLowPass(&obj->centerYFilter, (float) selCy, OM_POSITION_ALPHA);
        omTrackingAngleLowPass(&obj->orientFilter, finalOrient, OM_ORIENTATION_ALPHA);
    }
    else
    {
        // 非 LOCKED 态直接跟随, 锚定滤波器状态防止入锁瞬间突跳
        lowPassFilterReset(&obj->centerXFilter, (float) selCx);
        lowPassFilterReset(&obj->centerYFilter, (float) selCy);
        lowPassFilterReset(&obj->orientFilter, finalOrient);
    }

    obj->orientation = obj->orientBaselineValid ? obj->orientFilter.filteredValue : 0.0f;
}
}
else
{
    {
        contourHandleLostSample(obj, 1u);
    }
}
}

return contourCount;
}

// ---- 提取 Hu 实现 ----

uint8_t contourExtractHuMoments(const CONTOUR_DETECTOR_STRUCT* obj, uint8_t contourIndex, float huOut[7])
{
    {
        if (obj == NULL || huOut == NULL) return 0u;
        if (contourIndex >= obj->contourCount || obj->contourData[contourIndex].contourSize == 0) return 0u;
        uint32_t mBase = (uint32_t) contourIndex * 7u;
        computeHuMoments(contourMomBuf[mBase], contourMomBuf[mBase + 1], contourMomBuf[mBase + 2],
            contourMomBuf[mBase + 3], contourMomBuf[mBase + 4], contourMomBuf[mBase + 5], contourMomBuf[mBase + 6],
            obj->contourData[contourIndex].contourSize, huOut);
        return 1u;
    }
}

```

// ---- 内部函数实现 ----

// 8 邻域 BFS: 统计连通区域大小、质心、二阶矩、三阶矩及端点数; 返回端点数
#include "om_contour_geometry_impl.h"

core0_cyt4bb7/project/code/vision/om_contour_geometry_impl.h

SHA-256

:

2297fbb13be4d89ebed81f7598f660462412069a60294526615368c2a394e61a

代码 (c)

```
#ifndef OM_CONTOUR_GEOMETRY_IMPL_H
#define OM_CONTOUR_GEOMETRY_IMPL_H

/*
 * om_contour.c 单翻译单元实现片段: BFS、ray-balance 朝向与 Hu 数学内核。
 * 依赖 om_contour.c 的私有类型、宏和状态, 只允许在该文件原内部实现位置包含一次。
 */

static inline uint16_t eightNeighborhoodIterativeCall(const u8Image* obj, uint16_t startX, uint16_t startY, uint32_t genTag, CONTOUR_DATA_STRUCT*
contour, float* outMxx, float* outMyy, float* outMxy, float* outM30, float* outM03, float* outM21, float* outM12, float* outOrientation, float*
outOrientConfidence)
{
    uint32_t width = obj->xLen;
    uint32_t height = obj->yLen;

    contour->contourSize = 0;
    contour->centerX = 0;
    contour->centerY = 0;

    uint16_t head = 0;
    uint16_t tail = 0;

    // 起点入队并标记 (packed: high16=y, low16=x)
    inContourMap[startY * width + startX] = genTag;
    contourQueue[tail++] = ((uint32_t) startY << 16) | (uint32_t) startX;

    uint32_t sumX = 0;
    uint32_t sumY = 0;
    uint32_t sumXX = 0;
    uint32_t sumYY = 0;
    uint32_t sumXY = 0;
    uint64_t sumXXX = 0;
    uint64_t sumYYY = 0;
    uint64_t sumXXY = 0;
    uint64_t sumXYY = 0;
    uint16_t totalCount = 0;
    uint16_t endpointCount = 0;
    uint8_t abandon = 0;

    while (head < tail)
    {
        uint16_t cx = (uint16_t) (contourQueue[head] & 0xFFFFu);
        uint16_t cy = (uint16_t) (contourQueue[head] >> 16);
        head++;

        totalCount++;

        if (!abandon)
        {
            sumX += cx;
            sumY += cy;
            uint32_t cxx = (uint32_t) cx * cx;
            uint32_t cyy = (uint32_t) cy * cy;
            sumXX += cxx;
            sumYY += cyy;
            sumXY += (uint32_t) cx * cy;
            sumXXX += (uint64_t) cxx * cx;
            sumYYY += (uint64_t) cyy * cy;
            sumXXY += (uint64_t) cxx * cy;
            sumXYY += (uint64_t) cx * cyy;
        }
    }
}
```

```

// 统计原图中当前像素的白色 8 邻域数量 (用于端点检测)
uint8_t whiteNeighborCount = 0;

for (uint8_t i = 0; i < 8; i++)
{
    int32_t nx = (int32_t) cx + contourDx[i];
    int32_t ny = (int32_t) cy + contourDy[i];

    if (nx < 0 || (uint32_t) nx >= width || ny < 0 || (uint32_t) ny >= height) continue;

    uint16_t nIndex = (uint16_t) ((uint32_t) ny * width + (uint32_t) nx);

    if (obj->ptr[nIndex] == WHITE)
    {
        whiteNeighborCount++;

        // 未访问的白色像素入队
        if ((inContourMap[nIndex] >> 16) != contourGeneration)
        {
            inContourMap[nIndex] = genTag;
            if (tail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
                contourQueue[tail++] = ((uint32_t) ny << 16) | (uint32_t) nx;
        }
    }
}

// Canny 后边缘上端点像素白色邻居数 < 2
if (!abandon && whiteNeighborCount < 2) endpointCount++;

/* 触图像边界即 abandon: 边缘截断的开放曲线刚好有 2 个端点会冒充闭合 ^ */
if (!abandon && (cx == 0u || cy == 0u || (uint32_t) cx == width - 1u || (uint32_t) cy == height - 1u)) abandon = 1;

if (!abandon && (totalCount > contourDataConfig.sizeMax || endpointCount > 2)) abandon = 1;
}

/* abandon 时 contourSize 置 0, 让 caller 的 sizeMin 检查直接拒绝该轮廓 */
contour->contourSize = abandon ? 0u : totalCount;

if (totalCount > 0 && !abandon)
{
    float fN = (float) totalCount;
    float meanX = (float) sumX / fN;
    float meanY = (float) sumY / fN;
    contour->centerX = (uint16_t) (sumX / totalCount);
    contour->centerY = (uint16_t) (sumY / totalCount);

    *outMxx = (float) sumXX / fN - meanX * meanX;
    *outMyy = (float) sumYY / fN - meanY * meanY;
    *outMxy = (float) sumXY / fN - meanX * meanY;

    // 计算中心矩
    *outM30 = (float) sumXXX / fN - 3.0f * meanX * (float) sumXX / fN + 2.0f * meanX * meanX * meanX;
    *outM03 = (float) sumYYY / fN - 3.0f * meanY * (float) sumYY / fN + 2.0f * meanY * meanY * meanY;
    *outM21 = (float) sumXXY / fN - 2.0f * meanX * (float) sumXY / fN - meanY * (float) sumXX / fN + 2.0f * meanX * meanX * meanY;
    *outM12 = (float) sumXYY / fN - 2.0f * meanY * (float) sumXY / fN - meanX * (float) sumYY / fN + 2.0f * meanX * meanY * meanY;

    /* ^ 尖端朝向: 主轴由二阶矩给出, 尖端由质心射线两侧几何评分决定。
    * 尖端侧远端像素应沿主轴收敛, 反向开口侧远端像素离轴发散。
    */
    *outOrientation = contourComputeRayBalanceOrientation(obj, genTag, contourQueue, tail,
        meanX, meanY, *outMxx, *outMyy, *outMxy, outOrientConfidence);
}
else
{
    *outMxx = 0.0f;
    *outMyy = 0.0f;
    *outMxy = 0.0f;
    *outM30 = 0.0f;
    *outM03 = 0.0f;
    *outM21 = 0.0f;
    *outM12 = 0.0f;
    *outOrientation = 0.0f;
    *outOrientConfidence = 0.0f;
}

return endpointCount;
}

static float contourComputeRayBalanceOrientation(const u8Image* obj, uint32_t genTag, const uint32_t* queue, uint16_t count, float meanX, float meanY,

```

```

float mxx, float myy, float mxy, float* outConfidence)
{
    float theta = 0.5f * atan2f(2.0f * mxy, mxx - myy);
    float axis = wrapToPi(theta + HALF_PI);
    if (count == 0u)
    {
        *outConfidence = 0.0f;
        return axis;
    }

    float axisC = cosf(axis);
    float axisS = sinf(axis);
    float posExtent = 0.0f;
    float negExtent = 0.0f;

    for (uint16_t i = 0u; i < count; i++)
    {
        uint16_t x = (uint16_t) (queue[i] & 0xFFFFu);
        uint16_t y = (uint16_t) (queue[i] >> 16);
        float dx = (float) x - meanX;
        float dy = (float) y - meanY;
        float t = dx * axisC + dy * axisS;
        if (t > posExtent) posExtent = t;
        if (-t > negExtent) negExtent = -t;
    }

    if (posExtent <= 1.0f || negExtent <= 1.0f)
    {
        *outConfidence = 0.0f;
        return axis;
    }

    float halfExtent = (posExtent > negExtent) ? posExtent : negExtent;
    float minExtent = (posExtent < negExtent) ? posExtent : negExtent;
    float coneWidth = CONTOUR_RAY_CONE_RATIO * minExtent;
    if (coneWidth < CONTOUR_RAY_CONE_MIN) coneWidth = CONTOUR_RAY_CONE_MIN;
    uint8_t rayRadius = (halfExtent < CONTOUR_RAY_RADIUS_SPLIT) ? 1u : 2u;

    float posScore = contourRayBalanceSideScore(obj, genTag, queue, count, meanX, meanY, axisC, axisS, 1u, posExtent, coneWidth, rayRadius);
    float negScore = contourRayBalanceSideScore(obj, genTag, queue, count, meanX, meanY, axisC, axisS, 0u, negExtent, coneWidth, rayRadius);
    float denom = fabsf(posScore) + fabsf(negScore) + 1e-6f;
    float confidence = fabsf(posScore - negScore) / denom;
    if (confidence > 1.0f) confidence = 1.0f;
    *outConfidence = confidence;

    return (posScore >= negScore) ? axis : wrapToPi(axis + PI);
}

static float contourRayBalanceSideScore(const u8Image* obj, uint32_t genTag, const uint32_t* queue, uint16_t count, float meanX, float meanY, float axisC,
float axisS, uint8_t positiveSide, float extent, float coneWidth, uint8_t rayRadius)
{
    float farStart = CONTOUR_RAY_FAR_START_RATIO * extent;
    float farEnd = CONTOUR_RAY_FAR_END_RATIO * extent;
    float coneScore = 0.0f;
    uint16_t nearCount = 0u;
    uint16_t farTotal = 0u;
    float offAxisPenalty = 0.0f;
    float sign = positiveSide ? 1.0f : -1.0f;

    for (uint16_t i = 0u; i < count; i++)
    {
        uint16_t x = (uint16_t) (queue[i] & 0xFFFFu);
        uint16_t y = (uint16_t) (queue[i] >> 16);
        float dx = (float) x - meanX;
        float dy = (float) y - meanY;
        float t = dx * axisC + dy * axisS;
        float p = -dx * axisS + dy * axisC;
        float signedT = sign * t;
        if (signedT < farStart) continue;

        farTotal++;
        float absP = fabsf(p);
        float farWeight = signedT / (extent + 1e-6f);
        if (farWeight > 1.0f) farWeight = 1.0f;
        if (absP <= coneWidth)
        {
            nearCount++;
            coneScore += (1.0f - absP / coneWidth) * (0.5f + 0.5f * farWeight);
        }
    }
}

```

```

        else
        {
            float off = absP / coneWidth;
            if (off > 3.0f) off = 3.0f;
            offAxisPenalty += off * CONTOUR_RAY_OFF_PENALTY;
        }
    }

    float rayHits = 0.0f;
    int32_t steps = (int32_t) (farEnd - farStart) + 1;
    if (steps < 1) steps = 1;
    for (int32_t i = 0; i < steps; i++)
    {
        float ratio = (steps > 1) ? ((float) i / (float) (steps - 1)) : 0.0f;
        float d = farStart + (farEnd - farStart) * ratio;
        int32_t rx = contourRoundToInt(meanX + sign * axisC * d);
        int32_t ry = contourRoundToInt(meanY + sign * axisS * d);
        if (contourRayHasTaggedPixel(obj, genTag, rx, ry, rayRadius))
        {
            rayHits += 0.5f + 0.5f * (d / (extent + 1e-6f));
        }
    }

    float densityNorm = sqrtf((farTotal > 0u) ? (float) farTotal : 1.0f);
    return (CONTOUR_RAY_HIT_WEIGHT * rayHits) + (coneScore / densityNorm) +
        (CONTOUR_RAY_NEAR_WEIGHT * (float) nearCount) - offAxisPenalty;
}

static uint8_t contourRayHasTaggedPixel(const u8Image* obj, uint32_t genTag, int32_t rx, int32_t ry, uint8_t radius)
{
    int32_t r = (int32_t) radius;
    for (int32_t yy = ry - r; yy <= ry + r; yy++)
    {
        if (yy < 0 || (uint32_t) yy >= obj->yLen) continue;
        for (int32_t xx = rx - r; xx <= rx + r; xx++)
        {
            if (xx < 0 || (uint32_t) xx >= obj->xLen) continue;
            uint32_t idx = (uint32_t) yy * obj->xLen + (uint32_t) xx;
            if (inContourMap[idx] == genTag) return 1u;
        }
    }
    return 0u;
}

static int32_t contourRoundToInt(float v)
{
    return (v >= 0.0f) ? (int32_t) (v + 0.5f) : (int32_t) (v - 0.5f);
}

// 计算 7 个 Hu 不变矩 (输入为中心矩, n 为像素数)
static void computeHuMoments(float mxx, float myy, float mxy, float m30, float m03, float m21, float m12, uint16_t n, float hu[7])
{
    float fN = (float) n;
    float sqrtN = sqrtf(fN);
    float invN = 1.0f / fN;
    float invN15 = invN / sqrtN;

    // 归一化中心矩
    float eta20 = mxx * invN;
    float eta02 = myy * invN;
    float eta11 = mxy * invN;
    float eta30 = m30 * invN15;
    float eta03 = m03 * invN15;
    float eta21 = m21 * invN15;
    float eta12 = m12 * invN15;

    // 7 个 Hu 不变矩
    hu[0] = eta20 + eta02;
    hu[1] = (eta20 - eta02) * (eta20 - eta02) + 4.0f * eta11 * eta11;

    float t1 = eta30 - 3.0f * eta12;
    float t2 = 3.0f * eta21 - eta03;
    hu[2] = t1 * t1 + t2 * t2;

    float t3 = eta30 + eta12;
    float t4 = eta21 + eta03;
    hu[3] = t3 * t3 + t4 * t4;

    float t3sq = t3 * t3;

```

```

float t4sq = t4 * t4;
hu[4] = t1 * t3 * (t3sq - 3.0f * t4sq) + t2 * t4 * (3.0f * t3sq - t4sq);
hu[5] = (eta20 - eta02) * (t3sq - t4sq) + 4.0f * eta11 * t3 * t4;
hu[6] = t2 * t3 * (t3sq - 3.0f * t4sq) - t1 * t4 * (3.0f * t3sq - t4sq);
}

static float contourHuDistanceByIndex(const CONTOUR_DETECTOR_STRUCT* obj, uint16_t contourIdx)
{
    float hu[7];
    uint32_t mBase = (uint32_t) contourIdx * 7u;
    computeHuMoments(contourMomBuf[mBase], contourMomBuf[mBase + 1], contourMomBuf[mBase + 2],
        contourMomBuf[mBase + 3], contourMomBuf[mBase + 4], contourMomBuf[mBase + 5], contourMomBuf[mBase + 6],
        obj->contourData[contourIdx].contourSize, hu);
    return huMatchDistance(hu, contourDataConfig.huTemplate);
}

static inline float fastLogPositive(float x)
{
    #if (CONTOUR_USE_FAST_LOG)
        union
        {
            float f;
            uint32_t u;
        } v;
        v.f = x;

        int32_t exp2 = (int32_t) ((v.u >> 23) & 0xFFu) - 127;
        uint32_t mant = (v.u & 0x7FFFFFFu) | 0x3F800000u; // 1.m
        union
        {
            float f;
            uint32_t u;
        } vm;
        vm.u = mant;
        float m = vm.f;

        float y = m - 1.0f; // y in [0,1)
        // ln(1+y) 三阶截断
        float ln_m = y - 0.5f * y * y + (1.0f / 3.0f) * y * y * y;
        return ln_m + (float) exp2 * 0.69314718056f;
    #else
        return logf(x);
    #endif
}

// Hu 矩加权匹配距离 (log 空间加权 L1, 低阶高权重, h6 符号无意义)
static float huMatchDistance(const float huA[7], const float huB[7])
{
    float a0 = fabsf(huA[0]);
    if (a0 < 1e-12f) a0 = 1e-12f;
    float b0 = fabsf(huB[0]);
    if (b0 < 1e-12f) b0 = 1e-12f;
    float a1 = fabsf(huA[1]);
    if (a1 < 1e-12f) a1 = 1e-12f;
    float b1 = fabsf(huB[1]);
    if (b1 < 1e-12f) b1 = 1e-12f;
    float a2 = fabsf(huA[2]);
    if (a2 < 1e-12f) a2 = 1e-12f;
    float b2 = fabsf(huB[2]);
    if (b2 < 1e-12f) b2 = 1e-12f;
    float a3 = fabsf(huA[3]);
    if (a3 < 1e-12f) a3 = 1e-12f;
    float b3 = fabsf(huB[3]);
    if (b3 < 1e-12f) b3 = 1e-12f;
    float a4 = fabsf(huA[4]);
    if (a4 < 1e-12f) a4 = 1e-12f;
    float b4 = fabsf(huB[4]);
    if (b4 < 1e-12f) b4 = 1e-12f;
    float a5 = fabsf(huA[5]);
    if (a5 < 1e-12f) a5 = 1e-12f;
    float b5 = fabsf(huB[5]);
    if (b5 < 1e-12f) b5 = 1e-12f;

    float d0 = fastLogPositive(a0) - fastLogPositive(b0);
    float d1 = fastLogPositive(a1) - fastLogPositive(b1);
    float d2 = fastLogPositive(a2) - fastLogPositive(b2);
    float d3 = fastLogPositive(a3) - fastLogPositive(b3);
    float d4 = fastLogPositive(a4) - fastLogPositive(b4);
    float d5 = fastLogPositive(a5) - fastLogPositive(b5);

```

```
float dist = 2.0f * fabsf(d0) + 2.0f * fabsf(d1) + 1.5f * fabsf(d2) + 1.5f * fabsf(d3) + 0.5f * fabsf(d4) + 0.5f * fabsf(d5);
return dist;
}

#endif /* OM_CONTOUR_GEOMETRY_IMPL_H */
```

core0_cyt4bb7/project/code/vision/om_internal.h

SHA-256 :

e2e73c91a1fef2535f5ffb038efcee4f2566a907cd4fafd11515113ceec46fdf

代码 (c)

```
/*
 * om_internal.h - openMorph 内部共享头文件 (不对外暴露)
 *
 * 提供各子模块共用的宏定义、标准库包含及共享 BFS 基础设施声明。
 * 外部用户代码只需包含 openMorph.h。
 */
#ifndef OM_INTERNAL_H
#define OM_INTERNAL_H

// =====
// 公共接口 + 基础设施
// =====

#include "openMorph.h" // 公共类型定义与 API 声明
#include "openMorph_config.h" // 可移植尺寸/数量配置 (移植时只改此文件)
#include "commonDef.h" // u8Image, BLACK, WHITE, cReverse, range 等

#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

// =====
// 共享宏定义
// =====

#define UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE (OM_BUFFER_SIZE)

// =====
// 共享 BFS 基础设施 (om_contour.c 定义, om_beacon.c 使用)
// =====

extern uint32_t inContourMap[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE];
extern uint16_t contourGeneration;
extern const int8_t contourDx[8];
extern const int8_t contourDy[8];
extern uint32_t contourQueue[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE];

#endif // OM_INTERNAL_H
```

core0_cyt4bb7/project/code/vision/om_tracking_internal.h

SHA-256 :

b13d09423b6fafbc87ac6d254d4184542509dc6371f0e7550ad2e25482d3793a

代码 (c)

```
#ifndef OM_TRACKING_INTERNAL_H
#define OM_TRACKING_INTERNAL_H

/*
 * openMorph 跟踪滤波内部助手。
 * 必须在 om_internal.h 之后包含； 普通 static 保持各翻译单元原有本地链接与调用语义。
 */

static void omTrackingLowPass(LowPassFilterData* f, float sample, float alpha)
```



```

{
    f->alpha = alpha;
    lowPassFilter(f, sample);
}

static void omTrackingAngleLowPass(LowPassFilterData* f, float sample, float alpha)
{
    f->alpha = alpha;
    if (f->firstFlag == 1u)
    {
        lowPassFilterReset(f, sample);
        return;
    }
    float diff = sample - f->lastOutput;
    diff = wrapToPi(diff);
    lowPassFilter(f, f->lastOutput + diff);
    float out = f->filteredValue;
    out = wrapToPi(out);
    f->filteredValue = out;
    f->lastOutput = out;
}

static void omTrackingUpdateVelocity(CONTOUR_DETECTOR_STRUCT* obj, float rawX, float rawY, float sampleDtS)
{
    if (!obj->velocityPrimed)
    {
        obj->lastRawCenterX = rawX;
        obj->lastRawCenterY = rawY;
        lowPassFilterReset(&obj->velocityXFilter, 0.0f);
        lowPassFilterReset(&obj->velocityYFilter, 0.0f);
        obj->velocityPrimed = 1u;
        return;
    }

    if (sampleDtS >= OM_VELOCITY_DT_MIN_S && sampleDtS <= OM_VELOCITY_DT_MAX_S)
    {
        obj->velocityXFilter.alpha = OM_VELOCITY_ALPHA;
        obj->velocityYFilter.alpha = OM_VELOCITY_ALPHA;
        lowPassFilter(&obj->velocityXFilter, (rawX - obj->lastRawCenterX) / sampleDtS);
        lowPassFilter(&obj->velocityYFilter, (rawY - obj->lastRawCenterY) / sampleDtS);
    }
    else
    {
        lowPassFilterReset(&obj->velocityXFilter, 0.0f);
        lowPassFilterReset(&obj->velocityYFilter, 0.0f);
    }

    obj->lastRawCenterX = rawX;
    obj->lastRawCenterY = rawY;
}

static void omTrackingResetVelocity(CONTOUR_DETECTOR_STRUCT* obj)
{
    obj->velocityPrimed = 0u;
    obj->lastRawCenterX = 0.0f;
    obj->lastRawCenterY = 0.0f;
    lowPassFilterReset(&obj->velocityXFilter, 0.0f);
    lowPassFilterReset(&obj->velocityYFilter, 0.0f);
}

#endif /* OM_TRACKING_INTERNAL_H */

```

core0_cyt4bb7/project/code/vision/openMorph.h

SHA-256

:

a295094409912d9f3349a3d0848dc1ea49336254b43161302ed85003f2ea8ee3

代码 (c)

```

/*
 * openMorph.h - 图像处理 头文件
 *
 * Created: 20 250 804
 * Author: Hecate
 */

```

```

* 活跃模块: 高斯/Canny 与绿色灯板轮廓提取。
*/
#ifndef OPENMORPH_H
#define OPENMORPH_H

// =====
// 包含文件
// =====

#include <stdint.h>

#include "commonDef.h"
#include "openMorph_config.h"
#include "lowPassFilter.h"

// =====
// [4] 高斯模糊与 Canny 边缘检测
// =====

// ---- 函数声明 ----

void gaussianInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY);
void cannyInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY, uint16_t thresholdLow, uint16_t thresholdHigh);
void cannySetThreshold(uint16_t thresholdLow, uint16_t thresholdHigh);
void cannyEdgeDetection(const u8Image* src, u8Image* dstGaussBlur, u8Image* dst);

// =====
// [5] 轮廓提取
// =====

// ---- typedef ----

// 单个轮廓数据
typedef struct
{
    uint16_t centerX;    // 质心 X
    uint16_t centerY;    // 质心 Y
    uint16_t contourSize; // 全部像素数
    float huDistance;     /* Hu 距离: 0 = 未参与 Hu (未进 Top-K); >0 = 实际匹配距离 */
} CONTOUR_DATA_STRUCT;

// 跟踪状态枚举
#define OM_TRACK_SEARCHING (0u)
#define OM_TRACK_ACQUIRING (1u)
#define OM_TRACK_LOCKED (2u)
#define OM_TRACK_LOST_HOLDING (3u)

// 朝向跳变保护态 (逻辑同 ToF 定高跳变保护)
#define OM_ORIENT_NORMAL (0u)
#define OM_ORIENT_SUSPENDED (1u)

// 轮廓检测器数据
typedef struct
{
    CONTOUR_DATA_STRUCT contourData[OM_MAX_OBJECTS]; // 轮廓数据缓存

    uint8_t selectedIndex; // 当前选中轮廓索引
    uint8_t contourCount;  // 有效闭合轮廓数量
    uint8_t targetLost;    // 1= 目标丢失(无候选/Hu 超阈值/触边), 0=目标本体有效
    float orientation;      // 选中轮廓的几何朝向 (弧度, -PI ~ PI)
    float orientationConfidence; // 选中轮廓本帧朝向置信度 (0-1, 低置信只拒绝朝向更新, 不代表目标丢失)

    /* 跨帧状态保持 */
    uint16_t lastValidCenterX;
    uint16_t lastValidCenterY;
    float lastValidOrientation;
    uint16_t lastValidSize;
    uint8_t lostFrameCount;
    uint8_t lockFrameCount;
    uint8_t trackingState;
    uint8_t orientProtectState; // OM_ORIENT_NORMAL / OM_ORIENT_SUSPENDED
    uint8_t orientSpikeFrameCount; // SUSPENDED 态累计帧数 (达确认帧数后接受新朝向)
    uint8_t orientBaselineValid; // 已有可用于跳变保护/低通输出的可信朝向基线
    uint8_t orientAcquireFrameCount; // 无基线时连续接受朝向样本计数
    uint8_t orientRejectFrameCount; // 连续拒绝/丢失朝向样本计数
    LowPassFilterData orientFilter; // 朝向低通 (角度域, 系数 OM_ORIENTATION_ALPHA)
    LowPassFilterData centerXFilter; // 发布质心 X 状态 (OM_POSITION_ALPHA=1.0 默认不过滤)
    LowPassFilterData centerYFilter; // 发布质心 Y 状态 (OM_POSITION_ALPHA=1.0 默认不过滤)
}

```

```

    LowPassFilterData velocityXFilter; // 原始质心 X 差分速度低通 (px/s)
    LowPassFilterData velocityYFilter; // 原始质心 Y 差分速度低通 (px/s)
    float lastRawCenterX;           // 上一个有效原始质心 X
    float lastRawCenterY;           // 上一个有效原始质心 Y
    uint8_t velocityPrimed;         // 速度差分基线已建立
} CONTOUR_DETECTOR_STRUCT;

// ---- 函数声明 ----

void contourDetectInit(uint16_t sizeMin, uint16_t sizeMax, uint8_t centroidFeature, uint16_t visionCenterX, uint16_t visionCenterY, uint16_t roiRadius,
uint16_t earlyStopRadius, uint8_t brightnessMax);

// 运行时更新尺寸过滤范围 (标定后收紧)
void contourDetectSetSizeRange(uint16_t sizeMin, uint16_t sizeMax);

// 运行时更新质心亮度过滤上限 (255=关闭过滤)
void contourDetectSetBrightnessMax(uint8_t brightnessMax);

/* 朝向跳变保护参数由 contour owner 私有持有，仅通过本命令热重载。 */
void contourDetectSetOrientProtect(float jumpMax, float recoverRad, uint8_t confirmFrames);
uint8_t contourDetectOrientationJumpAllowed(float deltaRad);

// 设置目标形状 Hu 矩模板
// huLostThreshold: 选中轮廓 Hu 距离超阈值则置 targetLost=1, 建议 20.0
// huMoments: 可传 NULL 以禁用 Hu 模板(此时退化为纯距离逻辑)
void contourDetectSetTemplate(const float* huMoments, float huLostThreshold);

// 检测: src=Canny 边缘图, graySrc=灰度原图 (用于亮度筛选), sampleDtS=相邻处理帧实测间隔
// brightnessMax=255 时跳过亮度筛选; 返回检测到的闭合轮廓数量
uint16_t contourDetect(const u8Image* src, const u8Image* graySrc, float sampleDtS, CONTOUR_DETECTOR_STRUCT* obj);

// 从最近一次 contourDetect() 结果中提取指定轮廓的 7 维 Hu 矩
// 返回 1=成功, 0=索引越界或无数据 (必须在同一帧 contourDetect 之后调用)
uint8_t tcontourExtractHuMoments(const CONTOUR_DETECTOR_STRUCT* obj, uint8_t contourIndex, float huOut[7]);

#endif // OPENMORPH_H

```

core0_cyt4bb7/project/code/vision/openMorph_config.h

SHA-256

:

3a37588be7fdc4de1027347c7e4bf1dd0ee64d093ef62731b5a1a7be7ef0cb07

代码 (c)

```

/*
 * openMorph_config.h - openMorph 库可移植配置头
 *
 * 移植到新工程时只需修改本文件即可适配不同图像分辨率与对象数量上限。
 * openMorph 内部其他源文件不应再出现关于尺寸的字面量。
 */
#ifndef OPENMORPH_CONFIG_H
#define OPENMORPH_CONFIG_H

// =====
// 图像最大尺寸 (静态缓冲区按此分配)
// =====
// 当前默认 188 x 120 = 22560 像素 (与本工程实际相机分辨率匹配)
// 如需适配 320x240 等更高分辨率, 按比例上调即可。
// 本配置与具体相机驱动解耦, 不引入 SDK 头依赖。
// -----
#define OM_IMAGE_MAX_WIDTH (188u)
#define OM_IMAGE_MAX_HEIGHT (120u)
#define OM_BUFFER_SIZE (OM_IMAGE_MAX_WIDTH * OM_IMAGE_MAX_HEIGHT)

// =====
// 对象检测数量上限 (轮廓 / 信标 / 连通域共用)
// =====
// contourData[] / beaconData[] / sbea[] 等数组的成员数量。
// 超过此上限的候选对象会被忽略 (只保留前 N 个有效结果)。
// -----
#define OM_MAX_OBJECTS (101u)

// =====
// Canny 流水线环形缓冲宽度
// =====

```

```
// Canny 边缘检测内部 sobel 环形缓冲的列宽, 必须 >= OM_IMAGE_MAX_WIDTH。
// -----
#define OM_CANNY_RING_WIDTH (256u)

// =====
// 轮廓跟踪状态机参数
// =====
#define OM_LOCK_THRESHOLD    (2u)  // 连续检测到帧数: SEARCHING → ACQUIRING
#define OM_LOCK_CONFIRM     (1u)  // 继续确认帧数: ACQUIRING → LOCKED
#define OM_LOST_THRESHOLD   (5u)  // 连续丢失帧数: LOCKED → LOST_HOLDING
#define OM_LOST_TIMEOUT     (30u)  // 超时帧数: LOST_HOLDING → SEARCHING
#define OM_RAY_BALANCE_CONFIDENCE_MIN (0.50f) // ^ 尖端几何定向最低置信度: 低于此值拒绝更新朝向, 目标位置仍有效
#define OM_ORIENTATION_MIN_SIZE (8u)  // 朝向可靠的最小像素数; 分辨率迁移不改变朝向冻结门
#define OM_ORIENTATION_ALPHA  (1.0f) // 朝向低通滤波系数 (1.0=不低通, 直接跟随确认后的有效朝向)
#define OM_POSITION_ALPHA    (1.0f) // 质心位置不过滤
#define OM_VELOCITY_ALPHA    (0.1f) // 原始质心差分速度低通滤波系数
#define OM_VELOCITY_DT_MIN_S  (0.001f) // 有效图像样本间隔下限
#define OM_VELOCITY_DT_MAX_S  (0.200f) // 超过此间隔只重建速度基线
#define OM_BORDER_MARGIN     (1u)  // 画幅边缘安全边距

// =====
// 朝向跳变保护默认值 (运行时可由菜单热重载, 逻辑同 ToF 定高跳变保护)
// NORMAL 态: |Δθ| > JUMP_MAX 视为异常突跳, 拒绝该样本进 SUSPENDED
// SUSPEND 态: |Δθ| < RECOVER_RAD 即时恢复; 介于二者间累计 CONFIRM_FRAMES 帧后
// 接受新朝向为基线; 持续 > JUMP_MAX 则计数清零继续沿用旧朝向
// -----
#define OM_ORIENTATION_JUMP_MAX  (0.785398f) // PI/4, 朝向跳变阈值 (rad)
#define OM_ORIENTATION_RECOVER_RAD (0.196349f) // PI/16, 恢复窄带 (rad)
#define OM_ORIENTATION_CONFIRM_FRAMES (3u)    // 异常持续这么多帧后接受新朝向

#endif // OPENMORPH_CONFIG_H
```

core0_cyt4bb7/project/user/cm7_0_isr.c

SHA-256

41e1a4f768b252f171eb5e4b37f97a0a98799fa0c1a6a53177a06f36bdc43fa8

代码 (c)

```
/*=====
 * CYT4BB Opensourec Library 即 ( CYT4BB 开源库) 是一个基于官方 SDK 接口的第三方开源库
 * Copyright (c) 2022 SEEKFREE 逐飞科技
 *
 * 本文件是 CYT4BB 开源库的一部分
 *
 * CYT4BB 开源库是免费软件
 * 您可以根据自由软件基金会发布的 GPL (GNU General Public License, 即 GNU 通用公共许可证) 的条款
 * 即 GPL 的第 3 版 (即 GPL3.0) 或 (您选择的) 任何后来的版本, 重新发布和/或修改它
 *
 * 本开源库的发布是希望它能发挥作用, 但并未对其作任何的保证
 * 甚至没有隐含的适销性或适合特定用途的保证
 * 更多细节请参见 GPL
 *
 * 您应该在收到本开源库的同时收到一份 GPL 的副本
 * 如果没有, 请参阅<https://www.gnu.org/licenses/>
 *
 * 额外注明:
 * 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
 * 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
 * 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
 * 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明 (即本声明)
 *
 * 文件名称      cm7_0_isr
 * 公司名称      成都逐飞科技有限公司
 * 版本信息      查看 libraries/doc 文件夹内 version 文件 版本说明
 * 开发环境      IAR 9.40.1
 * 适用平台      CYT4BB
 * 店铺链接      https://seekfree.taobao.com/
 *
 * 修改记录
 * 日期          作者          备注
 * 2024-1-9      pudding      first version
 * 2024-5-14      pudding      新增 12 个 pit 周期中断 增加部分注释说明
 * 2025-2-4      pudding      优化串口中断逻辑, 防止意外干扰导致的卡死问题, 优化串口波特率计算逻辑
 * 2025-2-4      pudding      新增两个串口接口
=====*/
```

*****/

#include "zf_common_headfile.h"

#include "core0Loop.h"
#include "commonDef.h"
#include "opticalFlow.h"
#include "crossCoreLink.h"

// ***** PIT 中断函数 *****

void pit0_ch0_isr() // 定时器通道 0 周期中断服务函数

{
 pit_isr_flag_clear(PIT_CH0);

 pitProcess0();
}

void pit0_ch1_isr() // 定时器通道 1 周期中断服务函数

{
 pit_isr_flag_clear(PIT_CH1);

 pitProcess1();
}

void pit0_ch2_isr() // 定时器通道 2 周期中断服务函数

{
 pit_isr_flag_clear(PIT_CH2);
}

void pit0_ch10_isr() // 定时器通道 10 周期中断服务函数

{
 pit_isr_flag_clear(PIT_CH10);
}

void pit0_ch11_isr() // 定时器通道 11 周期中断服务函数

{
 pit_isr_flag_clear(PIT_CH11);
}

void pit0_ch12_isr() // 定时器通道 12 周期中断服务函数

{
 pit_isr_flag_clear(PIT_CH12);
}

void pit0_ch13_isr() // 定时器通道 13 周期中断服务函数

{
 pit_isr_flag_clear(PIT_CH13);
}

void pit0_ch14_isr() // 定时器通道 14 周期中断服务函数

{
 pit_isr_flag_clear(PIT_CH14);
}

void pit0_ch15_isr() // 定时器通道 15 周期中断服务函数

{
 pit_isr_flag_clear(PIT_CH15);
}

void pit0_ch16_isr() // 定时器通道 16 周期中断服务函数

{
 pit_isr_flag_clear(PIT_CH16);
}

void pit0_ch17_isr() // 定时器通道 17 周期中断服务函数

{
 pit_isr_flag_clear(PIT_CH17);
}

void pit0_ch18_isr() // 定时器通道 18 周期中断服务函数

{
 pit_isr_flag_clear(PIT_CH18);
}

void pit0_ch19_isr() // 定时器通道 19 周期中断服务函数

{
 pit_isr_flag_clear(PIT_CH19);
}

```

void pit0_ch20_isr() // 定时器通道 20 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH20);
}

void pit0_ch21_isr() // 定时器通道 21 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH21);
    tsl1401_collect_pit_handler();
}
// ***** PIT 中断函数 *****

// ***** 串口中断函数 *****
// 串口 0 默认作为调试串口
void uart0_isr(void)
{
    if (uart_isr_mask(UART_0)) // 串口 0 接收中断
    {
        #if DEBUG_UART_USE_INTERRUPT // 如果开启 debug 串口中断
            debug_interrupr_handler(); // 调用 debug 串口接收处理函数 数据会被 debug 环形缓冲区读取
        #endif // 如果修改了 DEBUG_UART_INDEX 那这段代码需要放到对应的串口中断去
    }
    else // 串口 0 发送中断
    {
        {
        }
    }
}

void uart1_isr(void)
{
    if (uart_isr_mask(UART_1)) // 串口 1 接收中断
    {
        opticalFlowRxIrqHandler(); // UPixels 光流：排空 UART 软缓冲喂流式解析器
    }
    else // 串口 1 发送中断
    {
        {
        }
    }
}

void uart2_isr(void)
{
    if (uart_isr_mask(UART_2)) // 串口 2 接收中断
    {
        crossCoreLinkRxIrqHandler(); // 核间链路：排空 UART_2 软缓冲喂帧解析器
    }
    else // 串口 2 发送中断
    {
        {
        }
    }
}

void uart3_isr(void)
{
    if (uart_isr_mask(UART_3)) // 串口 3 接收中断
    {
        {
        }
    }
    else // 串口 3 发送中断
    {
        {
        }
    }
}

void uart4_isr(void)
{
    if (uart_isr_mask(UART_4)) // 串口 4 接收中断
    {
        {
            uart_receiver_handler(); // 串口接收机回调函数
        }
    }
    else // 串口 4 发送中断
    {
        {
        }
    }
}

void uart5_isr(void)
{
    if (uart_isr_mask(UART_5)) // 串口 5 接收中断
    {
        {
        }
    }
    else // 串口 5 发送中断
    {
        {
        }
    }
}

```

```

}

void uart6_isr(void)
{
    if (uart_isr_mask(UART_6)) // 串口 6 接收中断
    {
    }
    else // 串口 6 发送中断
    {
    }
}
// ***** 串口中断函数 *****

// ***** 外部中断函数 *****
void gpio_0_exti_isr() // 外部 GPIO_0 中断服务函数
{
}

void gpio_1_exti_isr() // 外部 GPIO_1 中断服务函数
{
    if (exti_flag_get(P01_0)) // 示例 P1_0 端口外部中断判断
    {
    }
    if (exti_flag_get(P01_1))
    {
    }
}

void gpio_2_exti_isr() // 外部 GPIO_2 中断服务函数
{
    if (exti_flag_get(P02_0))
    {
    }
    if (exti_flag_get(P02_4))
    {
    }
}

void gpio_3_exti_isr() // 外部 GPIO_3 中断服务函数
{
}

void gpio_4_exti_isr() // 外部 GPIO_4 中断服务函数
{
}

void gpio_5_exti_isr() // 外部 GPIO_5 中断服务函数
{
}

void gpio_6_exti_isr() // 外部 GPIO_6 中断服务函数
{
}

void gpio_7_exti_isr() // 外部 GPIO_7 中断服务函数
{
}

void gpio_8_exti_isr() // 外部 GPIO_8 中断服务函数
{
}

void gpio_9_exti_isr() // 外部 GPIO_9 中断服务函数
{
}

void gpio_10_exti_isr() // 外部 GPIO_10 中断服务函数
{
}

void gpio_11_exti_isr() // 外部 GPIO_11 中断服务函数
{
}

void gpio_12_exti_isr() // 外部 GPIO_12 中断服务函数
{
}

```

```

void gpio_13_exti_isr() // 外部 GPIO_13 中断服务函数
{
}

void gpio_14_exti_isr() // 外部 GPIO_14 中断服务函数
{
}

void gpio_15_exti_isr() // 外部 GPIO_15 中断服务函数
{
}

void gpio_16_exti_isr() // 外部 GPIO_16 中断服务函数
{
}

void gpio_17_exti_isr() // 外部 GPIO_17 中断服务函数
{
}

void gpio_18_exti_isr() // 外部 GPIO_18 中断服务函数
{
}

void gpio_19_exti_isr() // 外部 GPIO_19 中断服务函数
{
}

void gpio_20_exti_isr() // 外部 GPIO_20 中断服务函数
{
}

void gpio_21_exti_isr() // 外部 GPIO_21 中断服务函数
{
}

void gpio_22_exti_isr() // 外部 GPIO_22 中断服务函数
{
}

void gpio_23_exti_isr() // 外部 GPIO_23 中断服务函数
{
}
// ***** 外部中断函数 *****

```

core0_cyt4bb7/project/user/cm7_1_isr.c

SHA-256

:

a943bbc7aa59d3ccef8bf0382bc220649d56e4d4957de7bfaad18b582e8f5911

代码 (c)

```

/*****
* CYT4BB Opensourec Library 即 (CYT4BB 开源库) 是一个基于官方 SDK 接口的第三方开源库
* Copyright (c) 2022 SEEKFREE 逐飞科技
*
* 本文件是 CYT4BB 开源库的一部分
*
* CYT4BB 开源库 是免费软件
* 您可以根据自由软件基金会发布的 GPL (GNU General Public License, 即 GNU 通用公共许可证) 的条款
* 即 GPL 的第 3 版 (即 GPL3.0) 或 (您选择的) 任何后来的版本, 重新发布和/或修改它
*
* 本开源库的发布是希望它能发挥作用, 但并未对其作任何的保证
* 甚至没有隐含的适销性或适合特定用途的保证
* 更多细节请参见 GPL
*
* 您应该在收到本开源库的同时收到一份 GPL 的副本
* 如果没有, 请参阅<https://www.gnu.org/licenses/>
*
* 额外注明:
* 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
* 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
* 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
* 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明 (即本声明)
*

```


* 文件名称 cm7_1_isr
* 公司名称 成都逐飞科技有限公司
* 版本信息 查看 libraries/doc 文件夹内 version 文件 版本说明
* 开发环境 IAR 9.40.1
* 适用平台 CYT4BB
* 店铺链接 <https://seekfree.taobao.com/>
*
* 修改记录
* 日期 作者 备注
* 2024-1-9 pudding first version
* 2024-5-14 pudding 新增 12 个 pit 周期中断 增加部分注释说明
* 2025-2-4 pudding 优化串口中断逻辑，防止意外干扰导致的卡死问题，优化串口波特率计算逻辑
* 2025-2-4 pudding 新增两个串口接口
*****/

```
#include "zf_common_headfile.h"

#include "core1Loop.h"

// ***** PIT 中断函数 *****
void pit0_ch0_isr() // 定时器通道 0 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH0);
}

void pit0_ch1_isr() // 定时器通道 1 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH1);
}

void pit0_ch2_isr() // 定时器通道 2 周期中断服务函数（cm_7_1 菜单按键扫描 10ms）
{
    pit_isr_flag_clear(PIT_CH2);

    pitProcess0();
}

void pit0_ch10_isr() // 定时器通道 10 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH10);
}

void pit0_ch11_isr() // 定时器通道 11 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH11);
}

void pit0_ch12_isr() // 定时器通道 12 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH12);
}

void pit0_ch13_isr() // 定时器通道 13 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH13);
}

void pit0_ch14_isr() // 定时器通道 14 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH14);
}

void pit0_ch15_isr() // 定时器通道 15 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH15);
}

void pit0_ch16_isr() // 定时器通道 16 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH16);
}

void pit0_ch17_isr() // 定时器通道 17 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH17);
}

void pit0_ch18_isr() // 定时器通道 18 周期中断服务函数
{

```

```

    pit_isr_flag_clear(PIT_CH18);
}

void pit0_ch19_isr() // 定时器通道 19 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH19);
}

void pit0_ch20_isr() // 定时器通道 20 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH20);
}

void pit0_ch21_isr() // 定时器通道 21 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH21);
    tsl1401_collect_pit_handler();
}
// ***** PIT 中断函数 *****

// ***** 串口中断函数 *****
// 串口 0 默认作为调试串口
void uart0_isr(void)
{
    if (uart_isr_mask(UART_0)) // 串口 0 接收中断
    {
        #if DEBUG_UART_USE_INTERRUPT // 如果开启 debug 串口中断
            debug_interrupr_handler(); // 调用 debug 串口接收处理函数 数据会被 debug 环形缓冲区读取
        #endif // 如果修改了 DEBUG_UART_INDEX 那这段代码需要放到对应的串口中断去
    }
    else // 串口 0 发送中断
    {
    }
}

void uart1_isr(void)
{
    if (uart_isr_mask(UART_1)) // 串口 1 接收中断
    {
        wireless_module_uart_handler();
    }
    else // 串口 1 发送中断
    {
    }
}

void uart2_isr(void)
{
    if (uart_isr_mask(UART_2)) // 串口 2 接收中断
    {
        gnss_uart_callback();
    }
    else // 串口 2 发送中断
    {
    }
}

void uart3_isr(void)
{
    if (uart_isr_mask(UART_3)) // 串口 3 接收中断
    {
    }
    else // 串口 3 发送中断
    {
    }
}

void uart4_isr(void)
{
    if (uart_isr_mask(UART_4)) // 串口 4 接收中断
    {
        uart_receiver_handler(); // 串口接收机回调函数
    }
    else // 串口 4 发送中断
    {
    }
}

```

```

void uart5_isr(void)
{
    if (uart_isr_mask(UART_5)) // 串口 5 接收中断
    {
    }
    else // 串口 5 发送中断
    {
    }
}

void uart6_isr(void)
{
    if (uart_isr_mask(UART_6)) // 串口 6 接收中断
    {
    }
    else // 串口 6 发送中断
    {
    }
}
// ***** 串口中断函数 *****

// ***** 外部中断函数 *****
void gpio_0_exti_isr() // 外部 GPIO_0 中断服务函数
{
}

void gpio_1_exti_isr() // 外部 GPIO_1 中断服务函数
{
    if (exti_flag_get(P01_0)) // 示例 P1_0 端口外部中断判断
    {
    }
    if (exti_flag_get(P01_1))
    {
    }
}

void gpio_2_exti_isr() // 外部 GPIO_2 中断服务函数
{
    if (exti_flag_get(P02_0))
    {
    }
    if (exti_flag_get(P02_4))
    {
    }
}

void gpio_3_exti_isr() // 外部 GPIO_3 中断服务函数
{
}

void gpio_4_exti_isr() // 外部 GPIO_4 中断服务函数
{
}

void gpio_5_exti_isr() // 外部 GPIO_5 中断服务函数
{
}

void gpio_6_exti_isr() // 外部 GPIO_6 中断服务函数
{
}

void gpio_7_exti_isr() // 外部 GPIO_7 中断服务函数
{
}

void gpio_8_exti_isr() // 外部 GPIO_8 中断服务函数
{
}

void gpio_9_exti_isr() // 外部 GPIO_9 中断服务函数
{
}

void gpio_10_exti_isr() // 外部 GPIO_10 中断服务函数
{
}

```

```

void gpio_11_exti_isr() // 外部 GPIO_11 中断服务函数
{
}

void gpio_12_exti_isr() // 外部 GPIO_12 中断服务函数
{
}

void gpio_13_exti_isr() // 外部 GPIO_13 中断服务函数
{
}

void gpio_14_exti_isr() // 外部 GPIO_14 中断服务函数
{
}

void gpio_15_exti_isr() // 外部 GPIO_15 中断服务函数
{
}

void gpio_16_exti_isr() // 外部 GPIO_16 中断服务函数
{
}

void gpio_17_exti_isr() // 外部 GPIO_17 中断服务函数
{
}

void gpio_18_exti_isr() // 外部 GPIO_18 中断服务函数
{
}

void gpio_19_exti_isr() // 外部 GPIO_19 中断服务函数
{
}

void gpio_20_exti_isr() // 外部 GPIO_20 中断服务函数
{
}

void gpio_21_exti_isr() // 外部 GPIO_21 中断服务函数
{
}

void gpio_22_exti_isr() // 外部 GPIO_22 中断服务函数
{
}

void gpio_23_exti_isr() // 外部 GPIO_23 中断服务函数
{
}
// ***** 外部中断函数 *****

```

core0_cyt4bb7/project/user/main_cm7_0.c

SHA-256

:

242b7ae29f849aa3e046f4825283205c56b64b525bd46c859bf86110f750abf1

代码 (c)

```

/*
 * *****
 * CYT4BB Opensourec Library 即 ( CYT4BB 开源库) 是一个基于官方 SDK 接口的第三方开源库
 * Copyright (c) 2022 SEEKFREE 逐飞科技
 *
 * 本文件是 CYT4BB 开源库的一部分
 *
 * CYT4BB 开源库 是免费软件
 * 您可以根据自由软件基金会发布的 GPL (GNU General Public License, 即 GNU 通用公共许可证) 的条款
 * 即 GPL 的第 3 版 (即 GPL3.0) 或 (您选择的) 任何后来的版本, 重新发布和/或修改它
 *
 * 本开源库的发布是希望它能发挥作用, 但并未对其作任何的保证
 * 甚至没有隐含的适销性或适合特定用途的保证
 * 更多细节请参见 GPL
 *
 * 您应该在收到本开源库的同时收到一份 GPL 的副本

```

```
* 如果没有，请参阅<https://www.gnu.org/licenses/>
*
* 额外注明：
* 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
* 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
* 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
* 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明（即本声明）
*
* 文件名称      main_cm7_0
* 公司名称      成都逐飞科技有限公司
* 版本信息      查看 libraries/doc 文件夹内 version 文件 版本说明
* 开发环境      IAR 9.40.1
* 适用平台      CYT4BB
* 店铺链接      https://seekfree.taobao.com/
*
* 修改记录
* 日期          作者          备注
* 2024-1-4      pudding      first version
*****/

#include "zf_common_headfile.h"

#include "core0Loop.h"

// 打开新的工程或者工程移动了位置务必执行以下操作
// 第一步 关闭上面所有打开的文件
// 第二步 project->clean 等待下方进度条走完

// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设

// ***** 代码区域 *****

/* =====
* BEFORE EDITING THIS CORE — READ THESE TWO DOCS FIRST:
* 1. <repo-root>/CLAUDE.md — multi-core architecture, naming rules, git workflow
* 2. ./project/code/PROJECT.md — this core's modules, boundaries, hazards
* Each core has the SAME filenames (motor.c / IMU.c / PID.c) but DIFFERENT semantics.
* Do NOT copy patterns across cores. Do NOT edit two cores in one change.
* ===== */
int main(void)
{
    clock_init(SYSTEM_CLOCK_250M); // 时钟配置及系统初始化<务必保留>
    debug_init(); // 调试串口信息初始化
    // 此处编写用户代码 例如外设初始化代码等

    processInit();

    // 此处编写用户代码 例如外设初始化代码等
    while (true)
    {
        // 此处编写需要循环执行的代码

        whileProcess();

        // 此处编写需要循环执行的代码
    }
}

// ***** 代码区域 *****
```

core0_cyt4bb7/project/user/main_cm7_1.c

SHA-256

:

363e25e84c266e455ba510f21d00be6a76c6b61238c8b3cf827431da3e8084e8

代码（c）

```
/* =====
* CYT4BB Opensource Library 即（CYT4BB 开源库）是一个基于官方 SDK 接口的第三方开源库
* Copyright (c) 2022 SEEKFREE 逐飞科技
*
* 本文件是 CYT4BB 开源库的一部分
*
* =====
```

* CYT4BB 开源库 是免费软件
* 您可以根据自由软件基金会发布的 GPL（GNU General Public License，即 GNU 通用公共许可证）的条款
* 即 GPL 的第 3 版（即 GPL3.0）或（您选择的）任何后来的版本，重新发布和/或修改它
*
* 本开源库的发布是希望它能发挥作用，但并未对其作任何的保证
* 甚至没有隐含的适销性或适合特定用途的保证
* 更多细节请参见 GPL
*
* 您应该在收到本开源库的同时收到一份 GPL 的副本
* 如果没有，请参阅<<https://www.gnu.org/licenses/>>
*
* 额外注明：
* 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
* 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
* 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
* 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明（即本声明）
*
* 文件名称 main_cm7_1
* 公司名称 成都逐飞科技有限公司
* 版本信息 查看 libraries/doc 文件夹内 version 文件 版本说明
* 开发环境 IAR 9.40.1
* 适用平台 CYT4BB
* 店铺链接 <https://seekfree.taobao.com/>
*
* 修改记录
* 日期 作者 备注
* 2024-1-4 pudding first version
***** /

```
#include "zf_common_headfile.h"

#include "core1Loop.h"

// 打开新的工程或者工程移动了位置务必执行以下操作
// 第一步 关闭上面所有打开的文件
// 第二步 project->clean 等待下方进度条走完

// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设

// ***** 代码区域 *****

/* =====
* BEFORE EDITING THIS CORE — READ THESE TWO DOCS FIRST:
* 1. <repo-root>/CLAUDE.md — multi-core architecture, naming rules, git workflow
* 2. ./project/code/PROJECT.md — this core's modules, boundaries, hazards
* Each core has the SAME filenames (motor.c / IMU.c / PID.c) but DIFFERENT semantics.
* Do NOT copy patterns across cores. Do NOT edit two cores in one change.
* ===== */
int main(void)
{
    clock_init(SYSTEM_CLOCK_250M); // 时钟配置及系统初始化<务必保留>
    debug_info_init();              // 调试串口信息初始化

    // 此处编写用户代码 例如外设初始化代码等

    processInit();

    // 此处编写用户代码 例如外设初始化代码等
    while (true)
    {
        // 此处编写需要循环执行的代码

        whileProcess();

        // 此处编写需要循环执行的代码
    }
}
```

// ***** 代码区域 *****

core1_cyt4bb7

core1_cyt4bb7/project/code/app/core0Loop.c

SHA-256

:

eb568e932a9783e718400673e259f2238c507f60a5b8395d70ae4cea5ffe6fa3

代码 (c)

```
#include "core0Loop.h"

#include <math.h>
#include <string.h>

#include "zf_driver_pit.h"

#include "crossCoreLink.h"
#include "uartLinkProto.h" /* UART_LINK_SYS_MODE_*(系统模式线值, 决定下发 core2 控制源) */
#include "chassisLink.h" /* RS-485 协议常量(MSG/cmd 位) + chassisLinkSendCommand(core1 作总线主发命令) */
#include "coreDataTransfer.h" /* planToLinkStruct/coreDataReadPlan */
#include "core1MotionTxFence.h"
#include "core1TakeoffEpochIdentity.h"
#include "paramSyncGateway.h"
#include "paramSyncPlatformPolicy.h"

typedef char core1_buzzer_bridge_value_assert[(UART_LINK_BUZZER_SILENT == CHASSIS_LINK_BUZZER_SILENT && UART_LINK_BUZZER_1HZ == CHASSIS_LINK_BUZZER_1HZ && UART_LINK_BUZZER_LONG == CHASSIS_LINK_BUZZER_LONG && UART_LINK_BUZZER_3S == CHASSIS_LINK_BUZZER_3S && UART_LINK_BUZZER_WARN == CHASSIS_LINK_BUZZER_WARN && UART_LINK_BUZZER_ARMED_IDLE == CHASSIS_LINK_BUZZER_ARMED_IDLE && UART_LINK_BUZZER_WAITING == CHASSIS_LINK_BUZZER_WAITING && UART_LINK_BUZZER_THREE_BEEPS == CHASSIS_LINK_BUZZER_THREE_BEEPS) ? 1 : -1];
typedef char core1_takeoff_flag_bridge_value_assert[(UART_LINK_MOTION_FLAG_TAKEOFF_REQ == CHASSIS_LINK_LORA_FLAG_TAKEOFF_REQ) ? 1 : -1];

/* core1 作 RS-485 总线主: 固定周期发 MSG_COMMAND 作"请求", core2 收到后才回一帧应答
 * (半双工串行化, 避免碰撞)。周期取 10ms: 命令(~1.56ms)+core2 应答(~1.56ms) 往返 <4ms,
 * 10ms 周期留足余量; core2 链路超时 500ms, 10ms 远快于超时, 保活无忧。 */
#define RS485_COMMAND_PERIOD_MS (10u)

/* core2 链路断联判据(core1 测): RS-485 应答静默超此时长 → 判 core2 断联, 经 STATUS 帧上报 core0 缓降。
 * core1 每 10ms 轮询、core2 收到命令即应答, 500ms(50 次轮询无应答)足够稳健, 远快于飞行缓降时间尺度。
 * 注: core0 链路丢时 core1 仍发 ESTOP 命令(仍轮询), core2 仍应答 → 不会被误判为 core2 断联。 */
#define AG_CORE1_RS485_LOST_MS (500u)

/* core2 的 LoRa 双肩键请求保持约 700ms。core1 仅在短时无 TELEMETRY 时保留最近置位,
 * 以覆盖 STATUS 200ms 周期; 收到清零 TELEMETRY、模式切走或 core2 失链立即清零。 */
#define MODE2_LORA_LAND_REQ_STALE_MS (300u)

typedef struct
{
    volatile uint32_t loopMs;
    uint32_t rs485LastCommandMs;
    uint32_t rs485LastReplyMs;
    uint32_t mode2LoraLandReqLastMs;
    uint32_t rs485CommandRequestMs;
    uint32_t planLastSeq;
    uint32_t planProducerBootGeneration;
    uint32_t planLastUpdateMs;
    planToLinkStruct lastGoodPlan;
    volatile uint8_t initReady;
    uint8_t rs485ReplyPrimed;
    uint8_t mode2LoraLandReqPrimed;
    uint8_t mode2LoraLandReqLevel;
    uint8_t mode2LoraLandReqPublished;
    uint8_t core2AttitudeFaultLevel;
    uint8_t rs485CommandReplyOutstanding;
    uint8_t planSeqPrimed;
    uint8_t lastGoodPlanValid;
    uint8_t mode2MotionNonzeroSent;
    uint8_t mode2MotionGateReady;
    core1MotionTxFenceState motionTxFence;
    core1TakeoffEpochIdentityState takeoffEpochIdentity;
    core1ParamSyncObserverBridgeCache observerParamBridge[2];
} core0RuntimeContextStruct;
```

```

static core0RuntimeContextStruct runtimeContext;

#define initReadyFlag          runtimeContext.initReady
#define rs485LastCommandMs     runtimeContext.rs485LastCommandMs
#define rs485LastReplyMs       runtimeContext.rs485LastReplyMs
#define mode2LoraLandReqLastMs runtimeContext.mode2LoraLandReqLastMs
#define rs485ReplyPrimed       runtimeContext.rs485ReplyPrimed
#define mode2LoraLandReqPrimed runtimeContext.mode2LoraLandReqPrimed
#define runtimeMode2LoraLandReqLevel runtimeContext.mode2LoraLandReqLevel
#define mode2LoraLandReqPublished runtimeContext.mode2LoraLandReqPublished
#define planLastSeq            runtimeContext.planLastSeq
#define planLastUpdateMs       runtimeContext.planLastUpdateMs
#define planSeqPrimed          runtimeContext.planSeqPrimed
#define loopMs                 runtimeContext.loopMs

static void core1ResetMotionTxFence(void)
{
    core1MotionTxFenceReset(&runtimeContext.motionTxFence);
    runtimeContext.mode2MotionNonzeroSent = 0u;
}

static uint8_t core1SendMotionWithZeroFence(float vx, float vy, float wz, uint8_t flags)
{
    core1MotionTxFenceFrame frame;
    uint8_t requestSent = core1MotionTxFencePrepare(
        &runtimeContext.motionTxFence, loopMs, vx, vy, wz, flags, &frame);
    crossCoreLinkSendMotion(frame.vx, frame.vy, frame.wz, frame.flags);
    return requestSent;
}
#define REQUIRED_VISION_PLAN_STALE_MS (600u) /* cm_7_1 plan 停更时无法证明必需视觉安全，fail-closed */

/* core2 是否在线(RS-485 应答未超时)。 供经 crossCoreLink STATUS 帧上报 core0 断联聚合缓降。 */
static uint8_t core1Core2Alive(void)
{
    return (rs485ReplyPrimed &&
        (uint32_t) (loopMs - rs485LastReplyMs) < AG_CORE1_RS485_LOST_MS)
        ? 1u
        : 0u;
}

static uint16_t core1AgeMs(uint8_t primed, uint32_t lastMs, uint16_t limit)
{
    uint32_t age = primed ? (uint32_t) (loopMs - lastMs) : (uint32_t) limit;
    return (age < (uint32_t) limit) ? (uint16_t) age : limit;
}

void processInit(void)
{
    memset(runtimeContext.observerParamBridge, 0,
        sizeof(runtimeContext.observerParamBridge));
    paramSyncGatewayInit();
    /* 核间单线双向链路(UART_1/P04 ↔ core0 P10): 收 core0 透传按键→共享内存, 周期发心跳。
     * 替代原 uartLinkTest RESPONDER(已移除)。 */
    crossCoreLinkInit();
    /* RS-485(UART_4/P14, THVD1450): 收 core2 LoRa 遥控值, 中转给 core0 做无人机前馈。 */
    chassisLinkInit();
    pit_ms_init(PIT_CH0, 1u);
    initReadyFlag = 1u;
}

void pitProcess0(void)
{
    if (!initReadyFlag)
    {
        return;
    }

    crossCoreLinkPit1ms();

    loopMs++; /* 自由 ms 计时， 供链路静默与请求年龄计算。 */

    /* 周期与静默年龄都由单调 loopMs 派生， ISR 不再与前台竞争清零标志。 */
}

void whileProcess(void)
{
    planToLinkStruct plan;
    planToLinkStruct sampledPlan;

```



```

crossCoreLinkStatusStruct linkBeforePoll;
crossCoreLinkStatusStruct linkAfterPoll;
crossCoreLinkLocalFactsStruct localFacts;
uint8_t requiredVisionFault = 0u;
uint8_t visionFaultSource = REQUIRED_VISION_SOURCE_NONE;
uint8_t visionFaultClass = REQUIRED_VISION_FAULT_NONE;
float loraReplyVx = 0.0f;
float loraReplyVy = 0.0f;
float loraReplyWz = 0.0f;
uint8_t loraReplyValid = 0u;
uint8_t remoteTakeoffReq = 0u;
uint8_t mode2MotionGateReady = 0u;
uint8_t takeoffIdentityReady;
uint8_t rs485CommandSent = 0u;
core1TakeoffEpochIdentityInput takeoffIdentityInput;

if (!initReadyFlag)
{
    paramSyncGatewayPoll(loopMs);
    return;
}

/* planRegion 本轮只读一次：既供 Core3 必需链路安全事实，也供 10ms RS-485 派发，避免同拍多次
 * invalidate/read 得到不同快照。读取碰撞与重复/旧 seq 保持 last-good；只有 wrap-aware 新 seq 更新缓存和时间戳。 */
if ((coreDataReadPlan(&sampledPlan) != 0u) &&
    (sampledPlan.magic == CORE1_PLAN_READY_MAGIC) &&
    (sampledPlan.producerBootGeneration != 0u))
{
    if ((planSeqPrimed == 0u) ||
        (sampledPlan.producerBootGeneration != runtimeContext.planProducerBootGeneration) ||
        ((int32_t) (sampledPlan.seq - planLastSeq) > 0))
    {
        if (sampledPlan.producerBootGeneration != runtimeContext.planProducerBootGeneration)
        {
            memset(&runtimeContext.lastGoodPlan, 0, sizeof(runtimeContext.lastGoodPlan));
            runtimeContext.lastGoodPlanValid = 0u;
            runtimeContext.mode2MotionGateReady = 0u;
            core1ResetMotionTxFence();
            runtimeContext.planProducerBootGeneration = sampledPlan.producerBootGeneration;
        }
        runtimeContext.lastGoodPlan = sampledPlan;
        runtimeContext.lastGoodPlanValid = 1u;
        planLastSeq = sampledPlan.seq;
        planLastUpdateMs = loopMs;
        planSeqPrimed = 1u;
    }
}
if (runtimeContext.lastGoodPlanValid != 0u)
{
    plan = runtimeContext.lastGoodPlan;
}
else
{
    memset(&plan, 0, sizeof(plan));
}
crossCoreLinkGetStatus(&linkBeforePoll);
/* STATUS bit3 始终携带通用必需视觉故障事实。plan 身份无效或停更时本核无法证明安全，
 * fail-closed 置 1；有效 plan 持续刷新该事实。 */
if ((plan.magic != CORE1_PLAN_READY_MAGIC) ||
    (core1AgeMs(planSeqPrimed, planLastUpdateMs, REQUIRED_VISION_PLAN_STALE_MS) >=
        REQUIRED_VISION_PLAN_STALE_MS))
{
    requiredVisionFault = 1u;
    visionFaultSource = REQUIRED_VISION_SOURCE_CORE1;
    visionFaultClass = REQUIRED_VISION_FAULT_BRIDGE;
}
else if (plan.requiredVisionFault != 0u)
{
    requiredVisionFault = 1u;
    visionFaultSource = plan.faultSource;
    visionFaultClass = plan.faultClass;
}

/* 先排空上一条 RS-485 请求的 Core2 应答，再冻结并发布本拍本地事实。旧实现先发 STATUS、
 * 后消费清零 TELEMETRY，恰逢 200ms 发送点时会把上一拍 M2 高电平多发一帧。 */
{
    uint8_t mode = linkBeforePoll.systemMode;
    chassisLinkReplyStruct decoded;
    while (chassisLinkPollReply(&decoded) != 0u)

```

```

{
    runtimeContext.rs485CommandReplyOutstanding = 0u;
    rs485LastReplyMs = loopMs;
    rs485ReplyPrimed = 1u;
    runtimeContext.core2AttitudeFaultLevel = decoded.attitudeFault;
    if (decoded.motionValid)
    {
        loraReplyVx = decoded.vx;
        loraReplyVy = decoded.vy;
        loraReplyWz = decoded.wz;
        loraReplyValid = 1u;
    }
    if (decoded.remoteTakeoffReq != 0u)
    {
        remoteTakeoffReq = 1u;
    }
    if (decoded.mode2LoraLandReqPresent != 0u)
    {
        if (decoded.mode2LoraLandReqLevel != 0u)
        {
            runtimeMode2LoraLandReqLevel = 1u;
            mode2LoraLandReqLastMs = loopMs;
            mode2LoraLandReqPrimed = 1u;
        }
        else
        {
            runtimeMode2LoraLandReqLevel = 0u;
            mode2LoraLandReqPrimed = 0u;
        }
    }
}
}
{
    uint8_t core2Alive = core1Core2Alive();
    if ((mode != UART_LINK_SYS_MODE_RS485_AUTO) || !core2Alive ||
        (core1AgeMs(mode2LoraLandReqPrimed, mode2LoraLandReqLastMs,
            MODE2_LORA_LAND_REQ_STALE_MS) >= MODE2_LORA_LAND_REQ_STALE_MS))
    {
        runtimeMode2LoraLandReqLevel = 0u;
        if ((mode != UART_LINK_SYS_MODE_RS485_AUTO) || !core2Alive)
        {
            mode2LoraLandReqPrimed = 0u;
        }
    }
    mode2LoraLandReqPublished = (uint8_t) ((mode == UART_LINK_SYS_MODE_RS485_AUTO) && core2Alive &&
        (runtimeMode2LoraLandReqLevel != 0u));
}
}

/* 上报 core2 链路存活给 crossCoreLink: 经 STATUS 帧(与心跳同周期 200ms 常发)转发 core0,
 * core0 landingSupervisor 聚合 (!core1Alive || core2Lost) → 任一断联缓降。每轮刷新, 发由 Poll 节流。 */
localFacts.core2LinkLost = core1Core2Alive() ? 0u : 1u;
localFacts.core2AttitudeFault = localFacts.core2LinkLost ? 0u : runtimeContext.core2AttitudeFaultLevel;
localFacts.requiredVisionFault = requiredVisionFault;
localFacts.faultSource = visionFaultSource;
localFacts.faultClass = visionFaultClass;
localFacts.mode2LoraLandReq = mode2LoraLandReqPublished;
localFacts.beaconStateValid = (uint8_t) ((plan.magic == CORE1_PLAN_READY_MAGIC) &&
    planSeqPrimed && (plan.planTimeoutMs != 0u) &&
    (core1AgeMs(planSeqPrimed, planLastUpdateMs, 0xFFFFu) < plan.planTimeoutMs));
localFacts.beaconCount = localFacts.beaconStateValid ? plan.beaconCount : 0u;
localFacts.beaconSearching = localFacts.beaconStateValid ? plan.beaconSearching : 0u;
localFacts.beaconSearchTimedOut = localFacts.beaconStateValid ? plan.beaconSearchTimedOut : 0u;
crossCoreLinkPublishLocalFacts(&localFacts);

crossCoreLinkPoll();
if (crossCoreLinkTakeCore0PeerRearm() != 0u)
{
    core1ResetMotionTxFence();
}
crossCoreLinkGetStatus(&linkAfterPoll);
/* 本拍唯一一次 plan 采样和两条安全链路的 RX 排空均已完成。网关复用
 * 最新 last-good 与停车事实, 避免读碰撞清桥或沿用上一拍状态接收请求。 */
paramSyncGatewayPoll(loopMs);
if (linkAfterPoll.core0Alive == 0u)
{
    /* 物理失链期间不发送 MOTION, 并撤销发送身份。重连后的第一次真实请求
     * 必经 core1SendMotionWithZeroFence 先发零, 闭合 Core0 超时后的 rearm 契约。 */
    core1ResetMotionTxFence();
}
}

```

```

/* 模式2 每个飞行 episode 必须观察到 cm_7_1 发布的新
 * (producerBootGeneration,takeoffMotionEpoch)元组和新鲜解锁 plan。
 * producer 重启时即使 epoch 数值与旧代碰撞，新代身份仍可在自身安全门解锁后恢复；
 * STATUS 空窗及飞行中 altReached 短暂撤销只归零，不重登记活动 episode 基线。 */
memset(&takeoffIdentityInput, 0, sizeof(takeoffIdentityInput));
takeoffIdentityInput.planIdentityValid = (uint8_t) (plan.magic == CORE1_PLAN_READY_MAGIC &&
    plan.producerBootGeneration != 0u);
takeoffIdentityInput.planProducerGeneration =
    plan.producerBootGeneration;
takeoffIdentityInput.planEpoch = plan.takeoffMotionEpoch;
takeoffIdentityInput.core0StatusUsable = (uint8_t) (linkAfterPoll.core0Alive != 0u &&
    linkAfterPoll.core0StatusValid != 0u);
takeoffIdentityInput.mode2Active = (uint8_t) (linkAfterPoll.systemMode == UART_LINK_SYS_MODE_RS485_AUTO);
takeoffIdentityInput.launched = linkAfterPoll.core0Launched;
takeoffIdentityInput.altReached = linkAfterPoll.altReached;
takeoffIdentityReady = core1TakeoffEpochIdentityStep(
    &runtimeContext.takeoffEpochIdentity, &takeoffIdentityInput);
mode2MotionGateReady = (uint8_t) (takeoffIdentityReady != 0u &&
    plan.magic == CORE1_PLAN_READY_MAGIC && plan.valid != 0u &&
    plan.dispatchEnable != 0u && plan.motionUnlocked != 0u &&
    plan.takeoffMotionEpoch != 0u &&
    core1AgeMs(planSeqPrimed, plan.LastUpdateMs, 0xFFFFu) < plan.planTimeoutMs &&
    chassisLinkMotionValid(plan.vx, plan.vy, plan.wz));
runtimeContext.mode2MotionGateReady = mode2MotionGateReady;

/* RS-485 总线主: 周期发 MSG_COMMAND(请求), 驱动 core2 响应帧。命令携带发车门控位 + sysByte:
 * core0 链路活 → ENABLE; core0 已发车(闭环) → LAUNCH; core0 链路丢 → ESTOP(车停)。
 * sysByte: 控制源按 core0 系统模式(模式3→LoRa, 模式1/2/未设→485, 覆盖 core2 本地源) + 中转 core0 蜂鸣码。
 * 运动 vx/vy/wz: 模式2(RS485 自主)且 cm_7_1 规划新鲜有效时取 planRegion; 模式1/3/未设保持零速。
 * 先发命令再 Poll 收 core2 应答(本轮发完即切回收, 应答多在后续 whileProcess 轮次收到)。 */
if ((uint32_t) (loopMs - rs485LastCommandMs) >= RS485_COMMAND_PERIOD_MS &&
    paramSyncGatewayCore2ReplyTurnaroundActive(loopMs) == 0u)
{
    uint8_t cmd = 0u;
    uint8_t sysByte = 0u;
    uint8_t buzzer;
    uint8_t mode = linkAfterPoll.systemMode;
    float vx = 0.0f, vy = 0.0f, wz = 0.0f; /* 下发 core2 的运动量, 默认零速 */
    rs485LastCommandMs = loopMs;
    if (linkAfterPoll.core0Alive &&
        linkAfterPoll.core0StatusValid)
    {
        cmd |= CHASSIS_LINK_CMD_ENABLE;
        cmd |= CHASSIS_LINK_CMD_CORE0_ALIVE; /* 转告 core2: core0 核间链路活 (供 core2 等待屏全链路门控 + 全通蜂鸣三声) */
        /* 车发车门控接飞机到顶信号: 飞机已发车(闭环) 且 自动起飞时序已到巡航高(altReached)
         * 才置 LAUNCH。到顶前 altReached=0 → 车禁主动移动, 避免起飞期(等待/爬高)车先动撞系留机。 */
        if (linkAfterPoll.core0Launched && linkAfterPoll.altReached &&
            (mode != UART_LINK_SYS_MODE_RS485_AUTO ||
             mode2MotionGateReady != 0u))
        {
            cmd |= CHASSIS_LINK_CMD_LAUNCH;
        }
    }
    else
    {
        cmd |= CHASSIS_LINK_CMD_ESTOP; /* core0 链路丢 → 命令车急停/等待 */
    }
    /* 控制源: core0 系统模式3(LoRa 转发) → LoRa, 其余(车不动/RS-485 自主/未设) → RS-485。 */
    if (mode == UART_LINK_SYS_MODE_LORA_FWD)
    {
        sysByte |= CHASSIS_LINK_SYS_SOURCE_LORA;
    }
    /* 中转 core0 蜂鸣模式码(uartLinkProto 与 chassisLink BUZZER_* 同值)给 core2 代鸣。 */
    buzzer = (uint8_t) (linkAfterPoll.buzzer & CHASSIS_LINK_SYS_BUZZER_MASK);
    sysByte |= (uint8_t) (buzzer << CHASSIS_LINK_SYS_BUZZER_SHIFT);

    /* 运动来源: 仅"模式2(RS485 自主) + 规划新鲜有效 + 派发开关开 + 三轴有效"才用 cm_7_1 规划值; 否则零速。
     * 派发总开关/超时改运行时(随 planToLinkStruct 跨核传来, 无编译期宏)。模式1(车不动)/模式3(LoRa 转发)
     * 恒零速——模式3车用本地 LoRa, 不消费此 vx/vy/wz。 Core1 beaconNavigator 负责规划幅值/斜率/Min/TD;
     * Core2 仍按车体物理边界钳位并做麦轮归一化。非法(NaN/Inf/越界) plan fail-closed 归零。 */
    if (mode == UART_LINK_SYS_MODE_RS485_AUTO && mode2MotionGateReady != 0u && plan.magic == CORE1_PLAN_READY_MAGIC &&
        plan.valid != 0u && plan.dispatchEnable != 0u && core1AgeMs(planSeqPrimed, plan.LastUpdateMs, 0xFFFFu) < plan.planTimeoutMs &&
        chassisLinkMotionValid(plan.vx, plan.vy, plan.wz))
    {
        vx = plan.vx;
        vy = plan.vy;
        wz = (plan.state == CORE1_PLAN_STATE_SEARCH) ? plan.wz : 0.0f;
    }
}

```

```

chassisLinkSendCommand(vx, vy, wz, cmd, sysByte);
runtimeContext.rs485CommandRequestMs = loopMs;
runtimeContext.rs485CommandReplyOutstanding = 1u;
rs485CommandSent = 1u;

/* 抄送 core0 飞控前馈：模式 2 非零值实时发送；非零→零、规划失效或退出模式 2 时
 * 只发一次显式零边沿，稳态零不周期续鲜。首次/静默 300ms 后的真实值先经过零 fence，
 * 下一拍再发送非零。模式 3 LoRa→MOTION 在下方应答块处理，与此按模式互斥。*/
if (linkAfterPoll.core0Alive && mode == UART_LINK_SYS_MODE_RS485_AUTO &&
    (vx != 0.0f || vy != 0.0f || wz != 0.0f))
{
    runtimeContext.mode2MotionNonzeroSent =
        core1SendMotionWithZeroFence(vx, vy, wz, 0u);
}
else if (runtimeContext.mode2MotionNonzeroSent != 0u)
{
    (void) core1SendMotionWithZeroFence(0.0f, 0.0f, 0.0f, 0u);
    runtimeContext.mode2MotionNonzeroSent = 0u;
}
}

/* 参数事务只能占用本拍未发送控制命令的 RS-485 空隙；命令/心跳始终优先。*/
paramSyncGatewayServiceExternal(loopMs, (uint8_t) (
    rs485CommandSent == 0u &&
    (runtimeContext.rs485CommandReplyOutstanding == 0u ||
    (uint32_t) (loopMs - runtimeContext.rs485CommandRequestMs) >=
        PARAM_SYNC_GATEWAY_RS485_REPLY_GUARD_MS) &&
    (uint32_t) (loopMs - rs485LastCommandMs) +
        PARAM_SYNC_GATEWAY_RS485_REPLY_GUARD_MS <
        RS485_COMMAND_PERIOD_MS));

/* 仅在收到有效运动帧时发 MOTION。应答已在本拍发布 STATUS 事实前排空；这里等待
 * crossCoreLinkPoll 更新 Core0 链路身份后再决定是否转发，保持重连 zero-fence 契约。*/
if ((loraReplyValid != 0u) &&
    (linkAfterPoll.core0Alive != 0u) &&
    (linkAfterPoll.systemMode == UART_LINK_SYS_MODE_LORA_FWD))
{
    (void) core1SendMotionWithZeroFence(
        loraReplyVx, loraReplyVy, loraReplyWz, remoteTakeoffReq);
}

}

uint8_t paramSyncGatewayPlatformTargetMask(void)
{
    crossCoreLinkStatusStruct link;
    uint8_t mask = 0u;
    crossCoreLinkGetStatus(&link);
    if (initReadyFlag != 0u) mask |= 0x02u;
    if (link.core0Alive && link.core0StatusValid) mask |= 0x01u;
    if (core1Core2Alive()) mask |= 0x04u;
    if (core1ParamSyncObserverBridgeReady(
        &runtimeContext.observerParamBridge[0], loopMs) != 0u)
        mask |= 0x08u;
    if (core1ParamSyncObserverBridgeReady(
        &runtimeContext.observerParamBridge[1], loopMs) != 0u)
        mask |= 0x10u;
    return mask;
}

void paramSyncGatewayPlatformPoll(uint32_t nowMs)
{
    core1ParamObserverBridgeStruct bridge;
    uint8_t i;
    for (i = 0u; i < 2u; i++)
    {
        core1ParamSyncObserverBridgeCache *cache =
            &runtimeContext.observerParamBridge[i];
        uint8_t source = (uint8_t) (i + 3u);
        uint8_t bridgeRead = 0u;
        bridgeRead = coreDataTryReadParamObserverBridge(source, &bridge);
        core1ParamSyncObserverBridgePoll(cache,
            bridgeRead != 0u ? &bridge : 0, nowMs);
    }
}

```

core1_cyt4bb7/project/code/app/core0Loop.h

SHA-256

:

8683a3836b2f98b97cb3fce05de2fe209dae77db57fa7426359f248f15982d4f

代码 (c)

```
#ifndef _CORE0LOOP_H_
#define _CORE0LOOP_H_

#include <stdint.h>

void processInit(void);
void pitProcess0(void);
void whileProcess(void);

#endif
```

core1_cyt4bb7/project/code/app/core1ContinuityIdentity.c

SHA-256

:

5c58f74bba3a9a8c3d3f82441dfc826c117775e0f5563d7b1e1430a0bb7af25d

代码 (c)

```
#include "core1ContinuityIdentity.h"

#include <string.h>

void core1ContinuityIdentityReset(core1ContinuityIdentityState *state)
{
    if (state != 0) memset(state, 0, sizeof(*state));
}

uint8_t core1ContinuityIdentityObserve(core1ContinuityIdentityState *state,
    const core1ContinuityIdentityInput *input)
{
    uint8_t changed;
    if ((state == 0) || (input == 0)) return 0u;
    if (state->primed == 0u)
    {
        state->last = *input;
        state->primed = 1u;
        return 0u;
    }
    changed = (uint8_t) (memcmp(&state->last, input, sizeof(*input)) != 0);
    state->last = *input;
    return changed;
}

uint8_t core1ContinuityIdentityApply(core1ContinuityIdentityState *state,
    const core1ContinuityIdentityInput *input,
    beaconTargetContinuity *continuity,
    uint8_t *barrierPending,
    uint8_t *planForceStep)
{
    if ((continuity == 0) || (barrierPending == 0) ||
        (planForceStep == 0)) return 0u;
    if (core1ContinuityIdentityObserve(state, input) == 0u) return 0u;
    beaconTargetContinuityInit(continuity);
    *barrierPending = 1u;
    *planForceStep = 1u;
    return 1u;
}
```

core1_cyt4bb7/project/code/app/core1ContinuityIdentity.h

SHA-256

:

521a7bebc13696edf6075d0c743baf90af43e2aad45b3bf2d11ff7c164ae6ea4

代码 (c)

```
#ifndef CORE1_CONTINUITY_IDENTITY_H
#define CORE1_CONTINUITY_IDENTITY_H

#include <stdint.h>

#include "beaconTargetContinuity.h"

typedef struct
{
    uint8_t launched;
    uint8_t core3ConfigValid;
    uint8_t core4ConfigValid;
    uint8_t padding;
    uint32_t core3Session;
    uint32_t core4Session;
    uint32_t core3Revision;
    uint32_t core4Revision;
    uint32_t core3Hash;
    uint32_t core4Hash;
} core1ContinuityIdentityInput;

typedef struct
{
    core1ContinuityIdentityInput last;
    uint8_t primed;
} core1ContinuityIdentityState;

void core1ContinuityIdentityReset(core1ContinuityIdentityState *state);
uint8_t core1ContinuityIdentityObserve(core1ContinuityIdentityState *state,
    const core1ContinuityIdentityInput *input);
uint8_t core1ContinuityIdentityApply(core1ContinuityIdentityState *state,
    const core1ContinuityIdentityInput *input,
    beaconTargetContinuity *continuity,
    uint8_t *barrierPending,
    uint8_t *planForceStep);

#endif /* CORE1_CONTINUITY_IDENTITY_H */
```

core1_cyt4bb7/project/code/app/core1KvmRouter.c

SHA-256

:

2d566a5400fed3bc1b00b7199b44039abbef6d3624f40b1b4abe98679f3d4340

代码 (c)

```
#include "core1KvmRouter.h"

#include <string.h>

#include "core3LinkCore1.h"
#include "core4LinkCore1.h"
#include "coreDataTransfer.h"
#include "menu.h"
#include "menuKeyRemote.h"
#include "uartLinkProto.h"

#define CORE1_KVM_MAX_LEASE_MS (600u)
#define CORE1_KVM_CORE3_FRESH_MS (600u)
#define CORE1_KVM_BASE_READ_TIMEOUT_MS (600u)

typedef struct
{
    core1KvmStatusStruct status;
```

```

uint32_t lastIngressSeq;
uint32_t lastIngressEventSeq;
uint32_t leaseDeadlineMs;
uint32_t expectedSeq;
uint32_t completedSeq;
uint32_t localEventSeq;
uint32_t localEventMenuTick;
uint32_t causalMenuTick;
uint32_t lastPeerSession;
uint32_t lastCore4PeerSession;
uint8_t target;
uint8_t routeAccepted;
uint8_t leaseValid;
uint8_t localEventPending;
uint8_t core3EventPending;
uint8_t core3ControlPending;
uint8_t localKeyId;
uint8_t completedResult;
uint8_t causalHomeValid;
uint8_t causalHome;
uint8_t causalMenuState;
uint8_t upstreamControlActive;
keyEventToMenuStruct baseSnapshot;
uint32_t baseLastGoodTickMs;
uint16_t baseSnapshotAgeMs;
uint8_t baseSnapshotValid;
uint32_t snapshotSeq;
} core1KvmRouterState;

static core1KvmRouterState g_router;
static volatile uint8_t g_displayState =
    CORE1_KVM_DISPLAY_PACK(CORE1_KVM_DISPLAY_TARGET_CORE0, CORE1_KVM_DISPLAY_TARGET_NONE);
static uint8_t inputOwner(void);

static uint8_t displayTarget(uint8_t target)
{
    if (target == UART_LINK_KVM_TARGET_CORE0) return CORE1_KVM_DISPLAY_TARGET_CORE0;
    if (target == UART_LINK_KVM_TARGET_CORE1) return CORE1_KVM_DISPLAY_TARGET_CORE1;
    if (target == UART_LINK_KVM_TARGET_CORE3) return CORE1_KVM_DISPLAY_TARGET_CORE3;
    if (target == UART_LINK_KVM_TARGET_CORE4) return CORE1_KVM_DISPLAY_TARGET_CORE4;
    return CORE1_KVM_DISPLAY_TARGET_NONE;
}

static void publishDisplayState(void)
{
    g_displayState = CORE1_KVM_DISPLAY_PACK(displayTarget(g_router.target),
        displayTarget(inputOwner()));
}

static uint8_t deadlineReached(uint32_t nowMs, uint32_t deadlineMs)
{
    return (uint8_t) (((int32_t) (nowMs - deadlineMs)) >= 0);
}

static uint8_t observerTargetKvmEligible(uint8_t target,
    const core3LinkCore1Status *link, const core4LinkCore1Status *core4)
{
    const observerLinkCore1EndpointStatus *status;
    const uint32_t required = OBSERVER_LINK_CAP_KVM |
        OBSERVER_LINK_CAP_MENU_STATUS;

    if (target == UART_LINK_KVM_TARGET_CORE3) status = link;
    else if (target == UART_LINK_KVM_TARGET_CORE4) status = core4;
    else return 1u;
    return (uint8_t) ((status->online != 0u) &&
        (status->protocolCompatible != 0u) &&
        ((status->peerCapabilities & required) == required));
}

static uint8_t inputOwner(void)
{
    uint8_t requiredValid;

    if ((g_router.routeAccepted == 0u) || (g_router.leaseValid == 0u))
        return UART_LINK_KVM_TARGET_NONE;
    if (g_router.target == UART_LINK_KVM_TARGET_CORE0)
    {
        return ((g_router.status.flags & UART_LINK_KVM_FLAG_CORE1_VALID) != 0u) &&
            (g_router.upstreamControlActive == 0u)
    }
}

```

```

        ? UART_LINK_KVM_TARGET_CORE0 : UART_LINK_KVM_TARGET_NONE;
    }
    requiredValid = (g_router.target == UART_LINK_KVM_TARGET_CORE1)
        ? UART_LINK_KVM_FLAG_CORE1_VALID : UART_LINK_KVM_FLAG_CORE3_VALID;
    return ((g_router.upstreamControlActive != 0u) &&
        ((g_router.status.flags & requiredValid) != 0u))
        ? g_router.target : UART_LINK_KVM_TARGET_NONE;
}

static uint8_t mapCore3Result(uint8_t result, uint8_t flags)
{
    if ((flags & OBSERVER_LINK_KVM_ACK_FLAG_DUPLICATE) != 0u) return UART_LINK_KVM_ACK_DUPLICATE;
    switch (result)
    {
        case OBSERVER_LINK_KVM_RESULT_EXECUTED: return UART_LINK_KVM_ACK_EXECUTED;
        case OBSERVER_LINK_KVM_RESULT_UNHANDLED: return UART_LINK_KVM_ACK_UNHANDLED;
        case OBSERVER_LINK_KVM_RESULT_NO_LEASE: return UART_LINK_KVM_ACK_NO_LEASE;
        case OBSERVER_LINK_KVM_RESULT_WRONG_TARGET: return UART_LINK_KVM_ACK_WRONG_TARGET;
        case OBSERVER_LINK_KVM_RESULT_STALE_EPOCH: return UART_LINK_KVM_ACK_STALE_ROUTE;
        case OBSERVER_LINK_KVM_RESULT_GLOBAL_EVENT_REJECTED: return UART_LINK_KVM_ACK_GLOBAL_EVENT_REJECTED;
        case OBSERVER_LINK_KVM_RESULT_GAP: return UART_LINK_KVM_ACK_GAP;
        case OBSERVER_LINK_KVM_RESULT_OLD_SEQ: return UART_LINK_KVM_ACK_OLD_EVENT;
        default: return UART_LINK_KVM_ACK_BAD_PAYLOAD;
    }
}

static uint32_t nextEventSeq(uint32_t value)
{
    value++;
    return (value != 0u) ? value : 1u;
}

static uint8_t eventSeqNewer(uint32_t value, uint32_t reference)
{
    return (uint8_t)((value != reference) &&
        ((uint32_t)(value - reference) < 0x80 000 000u));
}

static uint8_t eventSeqReached(uint32_t value, uint32_t reference)
{
    if (reference == 0u) return 1u;
    if (value == 0u) return 0u;
    return (uint8_t)((value == reference) ||
        eventSeqNewer(value, reference));
}

static void setTerminalAck(uint32_t eventSeq, uint8_t result)
{
    g_router.status.ackEventSeq = eventSeq;
    g_router.status.ackResult = result;
    g_router.status.flags |= UART_LINK_KVM_FLAG_ACK_TERMINAL;
    g_router.completedSeq = eventSeq;
    g_router.completedResult = result;
}

static void completeCurrentEvent(uint32_t eventSeq, uint8_t result,
    uint8_t menuState, uint8_t homeReady)
{
    menuStatusSnapshotStruct menuStatus;
    menuGetStatus(&menuStatus);
    setTerminalAck(eventSeq, result);
    g_router.expectedSeq = nextEventSeq(g_router.expectedSeq);
    g_router.causalHomeValid = 1u;
    g_router.causalHome = homeReady;
    g_router.causalMenuState = menuState;
    g_router.causalMenuTick = menuStatus.tickSeq;
}

static void cancelPendingEvent(uint8_t result)
{
    uint32_t eventSeq = 0u;
    if (g_router.localEventPending != 0u) eventSeq = g_router.localEventSeq;
    else if (g_router.core3EventPending != 0u) eventSeq = g_router.expectedSeq;
    menuKeyRemoteRevoke();
    g_router.localEventPending = 0u;
    g_router.core3EventPending = 0u;
    g_router.core3ControlPending = 0u;
    g_router.causalHomeValid = 0u;
    if (eventSeq != 0u) setTerminalAck(eventSeq, result);
}

```



```

}

static void enforceActiveObserverEligibility(
    const core3LinkCore1Status *link, const core4LinkCore1Status *core4)
{
    if (((g_router.target != UART_LINK_KVM_TARGET_CORE3) &&
        (g_router.target != UART_LINK_KVM_TARGET_CORE4)) ||
        (observerTargetKvmEligible(g_router.target, link, core4) != 0u))
    {
        return;
    }
    if ((g_router.routeAccepted != 0u) || (g_router.leaseValid != 0u) ||
        (g_router.localEventPending != 0u) ||
        (g_router.core3EventPending != 0u) ||
        (g_router.core3ControlPending != 0u))
    {
        cancelPendingEvent(UART_LINK_KVM_ACK_NO_LEASE);
    }
    g_router.routeAccepted = 0u;
    g_router.leaseValid = 0u;
    g_router.core3ControlPending = 0u;
    g_router.causalHomeValid = 0u;
    g_router.status.activeTarget = UART_LINK_KVM_TARGET_NONE;
    g_router.status.flags &= (uint8_t) ~(UART_LINK_KVM_FLAG_CORE3_VALID |
        UART_LINK_KVM_FLAG_CORE3_HOME |
        UART_LINK_KVM_FLAG_ROUTE_ACCEPTED |
        UART_LINK_KVM_FLAG_LEASE_VALID);
    g_router.status.menuStates &= 0x0Fu;
    g_router.status.core3AgeMs = 0xFFFFu;
    publishDisplayState();
}

static void resetRoute(uint32_t routeSession, uint32_t routeEpoch, uint8_t target)
{
    menuKeyRemoteRevoke();
    g_router.status.routeSession = routeSession;
    g_router.status.routeEpoch = routeEpoch;
    g_router.status.ackEventSeq = 0u;
    g_router.status.ackResult = UART_LINK_KVM_ACK_NONE;
    g_router.status.flags = 0u;
    g_router.expectedSeq = 1u;
    g_router.completedSeq = 0u;
    g_router.completedResult = UART_LINK_KVM_ACK_NONE;
    g_router.localEventPending = 0u;
    g_router.core3EventPending = 0u;
    g_router.core3ControlPending = 0u;
    g_router.causalHomeValid = 0u;
    g_router.target = target;
    g_router.routeAccepted = 0u;
    g_router.leaseValid = 0u;
    g_router.lastIngressEventSeq = 0u;
}

static uint32_t targetSession(uint8_t target, const core3LinkCore1Status *link,
    const core4LinkCore1Status *core4)
{
    if (target == UART_LINK_KVM_TARGET_CORE3) return link->peerSession;
    if (target == UART_LINK_KVM_TARGET_CORE4) return core4->peerSession;
    return g_router.status.routerSession;
}

static void applyControl(const core1KvmIngressStruct *ingress,
    const core3LinkCore1Status *link, const core4LinkCore1Status *core4,
    uint32_t tickMs)
{
    uint32_t session;
    uint8_t routeChanged = (uint8_t) ((ingress->routeSession != g_router.status.routeSession) ||
        (ingress->routeEpoch != g_router.status.routeEpoch) ||
        (ingress->target != g_router.target));
    if (routeChanged != 0u)
    {
        resetRoute(ingress->routeSession, ingress->routeEpoch, ingress->target);
    }
    g_router.status.target = ingress->target;
    session = targetSession(ingress->target, link, core4);
    g_router.status.targetSession = session;
    if ((ingress->leaseMs == 0u) || (ingress->leaseMs > CORE1_KVM_MAX_LEASE_MS) ||
        ((ingress->target != UART_LINK_KVM_TARGET_CORE0) &&
        (ingress->target != UART_LINK_KVM_TARGET_CORE1) &&

```

```

        (ingress->target != UART_LINK_KVM_TARGET_CORE3) &&
        (ingress->target != UART_LINK_KVM_TARGET_CORE4)) ||
        (session == 0u) ||
        ((ingress->target != UART_LINK_KVM_TARGET_CORE0) &&
        (ingress->targetSession != session)))
    {
        g_router.routeAccepted = 0u;
        g_router.leaseValid = 0u;
        return;
    }
    if (observerTargetKvmEligible(ingress->target, link, core4) == 0u)
    {
        g_router.routeAccepted = 0u;
        g_router.leaseValid = 0u;
        return;
    }
    g_router.leaseValid = 1u;
    g_router.leaseDeadlineMs = tickMs + ingress->leaseMs;
    if ((ingress->target == UART_LINK_KVM_TARGET_CORE3) ||
        (ingress->target == UART_LINK_KVM_TARGET_CORE4))
    {
        observerLinkKvmControl control;
        memset(&control, 0, sizeof(control));
        control.targetSession = session;
        control.routeSession = ingress->routeSession;
        control.routeEpoch = ingress->routeEpoch;
        control.leaseMs = ingress->leaseMs;
        control.target = ingress->target;
        g_router.core3ControlPending = (ingress->target == UART_LINK_KVM_TARGET_CORE3) ?
            core3LinkCore1QueueKvmControl(&control) :
            core4LinkCore1QueueKvmControl(&control);
        if (g_router.core3ControlPending == 0u)
        {
            g_router.leaseValid = 0u;
            g_router.routeAccepted = 0u;
        }
    }
    else g_router.routeAccepted = 1u;
}

static void rejectEvent(const core1KvmIngressStruct *ingress, uint8_t result)
{
    setTerminalAck(ingress->eventSeq, result);
}

static void applyEvent(const core1KvmIngressStruct *ingress,
    const core3LinkCore1Status *link, const core4LinkCore1Status *core4)
{
    menuStatusSnapshotStruct menuStatus;
    uint32_t session = targetSession(ingress->target, link, core4);
    menuGetStatus(&menuStatus);
    if ((ingress->routeSession != g_router.status.routeSession) ||
        (ingress->routeEpoch != g_router.status.routeEpoch))
    {
        rejectEvent(ingress, UART_LINK_KVM_ACK_STALE_ROUTE);
        return;
    }
    if ((ingress->target != g_router.target) || (ingress->targetSession != session))
    {
        rejectEvent(ingress, UART_LINK_KVM_ACK_WRONG_TARGET);
        return;
    }
    if (observerTargetKvmEligible(ingress->target, link, core4) == 0u)
    {
        rejectEvent(ingress, UART_LINK_KVM_ACK_NO_LEASE);
        return;
    }
    if ((ingress->keyId >= 5u) || (ingress->keyState < 1u) || (ingress->keyState > 2u))
    {
        rejectEvent(ingress, UART_LINK_KVM_ACK_BAD_PAYLOAD);
        return;
    }
    if ((g_router.routeAccepted == 0u) || (g_router.leaseValid == 0u))
    {
        rejectEvent(ingress, UART_LINK_KVM_ACK_NO_LEASE);
        return;
    }
    if ((ingress->eventSeq == g_router.completedSeq) &&
        (g_router.completedSeq != 0u))

```

```

{
    setTerminalAck(ingress->eventSeq, UART_LINK_KVM_ACK_DUPLICATE);
    return;
}
if ((ingress->eventSeq == 0u) ||
    ((ingress->eventSeq != g_router.expectedSeq) &&
     (eventSeqNewer(ingress->eventSeq, g_router.expectedSeq) == 0u)))
{
    rejectEvent(ingress, UART_LINK_KVM_ACK_OLD_EVENT);
    return;
}
if (ingress->eventSeq != g_router.expectedSeq)
{
    rejectEvent(ingress, UART_LINK_KVM_ACK_GAP);
    return;
}
if (g_router.localEventPending != 0u) return;
g_router.status.flags &= (uint8_t) ~UART_LINK_KVM_FLAG_ACK_TERMINAL;
g_router.causalHomeValid = 0u;
if (ingress->target == UART_LINK_KVM_TARGET_CORE1)
{
    if ((menuStatus.inputReady == 0u) ||
        (menuKeyRemoteInject(ingress->keyId, ingress->keyState) == 0u))
    {
        completeCurrentEvent(ingress->eventSeq, UART_LINK_KVM_ACK_BUSY,
                               menuStatus.state, menuStatus.homeReady);
        return;
    }
    g_router.localEventPending = 1u;
    g_router.localEventSeq = ingress->eventSeq;
    g_router.localEventMenuTick = menuStatus.tickSeq;
    g_router.localKeyId = ingress->keyId;
}
else
{
    observerLinkKvmEvent event;
    memset(&event, 0, sizeof(event));
    event.targetSession = session;
    event.routeSession = ingress->routeSession;
    event.routeEpoch = ingress->routeEpoch;
    event.eventSeq = ingress->eventSeq;
    event.target = ingress->target;
    event.keyId = ingress->keyId;
    event.keyState = (uint8_t) (ingress->keyState - 1u);
    if (ingress->target == UART_LINK_KVM_TARGET_CORE3)
        (void) core3LinkCore1QueueKvmEvent(&event);
    else
        (void) core4LinkCore1QueueKvmEvent(&event);
    g_router.core3EventPending = 1u;
}
}

static void finishEvents(void)
{
    observerLinkKvmAck ack;
    menuStatusSnapshotStruct menuStatus;
    menuGetStatus(&menuStatus);
    if (g_router.localEventPending != 0u)
    {
        if (menuStatus.tickSeq != g_router.localEventMenuTick)
        {
            uint8_t consumed = menuKeyRemoteTakeConsumed(g_router.localKeyId);
            if (consumed == 0u) menuKeyRemoteRevoke();
            completeCurrentEvent(g_router.localEventSeq,
                                consumed ? UART_LINK_KVM_ACK_EXECUTED : UART_LINK_KVM_ACK_UNHANDLED,
                                menuStatus.state, menuStatus.homeReady);
            g_router.localEventPending = 0u;
        }
    }
}

while (((g_router.target == UART_LINK_KVM_TARGET_CORE3) ?
        core3LinkCore1TakeKvmAck(&ack) : core4LinkCore1TakeKvmAck(&ack)) != 0u)
{
    if ((ack.targetSession == g_router.status.targetSession) &&
        (ack.routeSession == g_router.status.routeSession) &&
        (ack.routeEpoch == g_router.status.routeEpoch) &&
        (ack.eventSeq == g_router.expectedSeq) &&
        (g_router.core3EventPending != 0u))
    {
        uint8_t result = mapCore3Result(ack.result, ack.flags);
    }
}

```

```

uint8_t ackValid = 1u;
uint32_t requiredExpected = g_router.expectedSeq;
if ((result == UART_LINK_KVM_ACK_EXECUTED) ||
    (result == UART_LINK_KVM_ACK_UNHANDLED) ||
    (result == UART_LINK_KVM_ACK_DUPLICATE))
    requiredExpected = nextEventSeq(requiredExpected);
if (ack.expectedSeq != requiredExpected)
{
    result = UART_LINK_KVM_ACK_BAD_PAYLOAD;
    requiredExpected = g_router.expectedSeq;
    ackValid = 0u;
}
setTerminalAck(ack.eventSeq, result);
g_router.expectedSeq = requiredExpected;
g_router.core3EventPending = 0u;
g_router.causalHomeValid = ackValid;
if (ackValid != 0u)
{
    g_router.causalHome = (uint8_t) (
        (ack.flags & OBSERVER_LINK_KVM_ACK_FLAG_HOME_READY) != 0u);
    g_router.causalMenuState = ack.menuState;
}
}
}

static uint8_t updateBaseSnapshot(uint32_t tickMs)
{
    keyEventToMenuStruct next;

    if (coreDataTryReadKeyEvent(&next) != 0u)
    {
        g_router.baseSnapshot = next;
        g_router.baseLastGoodTickMs = tickMs;
        g_router.baseSnapshotAgeMs = 0u;
        g_router.baseSnapshotValid = 1u;
    }
    else if ((g_router.baseSnapshotValid == 0u) ||
        ((uint32_t) (tickMs - g_router.baseLastGoodTickMs) >=
            CORE1_KVM_BASE_READ_TIMEOUT_MS))
    {
        memset(&g_router.baseSnapshot, 0, sizeof(g_router.baseSnapshot));
        g_router.baseSnapshotAgeMs = 0xFFFFu;
        g_router.baseSnapshotValid = 0u;
    }
    else
    {
        uint32_t age = (uint32_t) (tickMs - g_router.baseLastGoodTickMs);
        g_router.baseSnapshotAgeMs = (age > 0xFFFFu) ? 0xFFFFu :
            (uint16_t) age;
    }
    return g_router.baseSnapshotValid;
}

static void publishStatus(uint32_t tickMs, const core3LinkCore1Status *link,
    const core4LinkCore1Status *core4)
{
    observerLinkKvmMenuStatus core3Menu;
    uint32_t core3Age = UINT32_MAX;
    const keyEventToMenuStruct *base = &g_router.baseSnapshot;
    menuStatusSnapshotStruct menuStatus;
    uint8_t core1Home;
    uint8_t observerMenuAvailable = 0u;
    uint8_t observerEligible = observerTargetKvmEligible(
        g_router.target, link, core4);
    menuGetStatus(&menuStatus);
    core1Home = menuStatus.homeReady;
    uint8_t useCausal = (uint8_t) (
        ((g_router.status.flags & UART_LINK_KVM_FLAG_ACK_TERMINAL) != 0u) &&
        (g_router.causalHomeValid != 0u));

    if ((useCausal != 0u) &&
        (g_router.target == UART_LINK_KVM_TARGET_CORE1) &&
        (menuStatus.tickSeq != g_router.causalMenuTick))
    {
        g_router.causalHomeValid = 0u;
        useCausal = 0u;
    }
}

```

```

g_router.status.magic = CORE1_DATA_READY_MAGIC;
g_router.status.target = g_router.target;
g_router.status.activeTarget = UART_LINK_KVM_TARGET_NONE;
g_router.status.targetSession = targetSession(g_router.target, link, core4);
g_router.status.menuStates = (uint8_t) (menuStatus.state & 0x0Fu);
g_router.status.paramTargetMask = 0x02u;
if (link->online && link->protocolCompatible &&
    ((link->peerCapabilities & OBSERVER_LINK_CAP_PARAM_SYNC) != 0u))
    g_router.status.paramTargetMask |= 0x08u;
if (core4->online && core4->protocolCompatible &&
    ((core4->peerCapabilities & OBSERVER_LINK_CAP_PARAM_SYNC) != 0u))
    g_router.status.paramTargetMask |= 0x10u;
g_router.status.flags &= UART_LINK_KVM_FLAG_ACK_TERMINAL | UART_LINK_KVM_FLAG_PEER_RESTART;
if ((g_router.baseSnapshotValid != 0u) &&
    (base->ready == CORE1_DATA_READY_MAGIC) && (base->linkUp != 0u))
{
    g_router.status.flags |= UART_LINK_KVM_FLAG_CORE1_VALID;
    if (((useCausal != 0u) && (g_router.target == UART_LINK_KVM_TARGET_CORE1))
        ? g_router.causalHome : core1Home)
        g_router.status.flags |= UART_LINK_KVM_FLAG_CORE1_HOME;
}
if ((observerEligible != 0u) && (g_router.leaseValid != 0u) &&
    ((g_router.target == UART_LINK_KVM_TARGET_CORE3) ||
     (g_router.target == UART_LINK_KVM_TARGET_CORE4)))
{
    observerMenuAvailable = (g_router.target == UART_LINK_KVM_TARGET_CORE4) ?
        core4LinkCore1GetKvmMenuStatus(&core3Menu, tickMs, &core3Age) :
        core3LinkCore1GetKvmMenuStatus(&core3Menu, tickMs, &core3Age);
}
if ((observerMenuAvailable != 0u) &&
    (core3Age < CORE1_KVM_CORE3_FRESH_MS) &&
    (core3Menu.targetSession == ((g_router.target == UART_LINK_KVM_TARGET_CORE4) ?
        core4->peerSession : link->peerSession)) &&
    (core3Menu.routeSession == g_router.status.routeSession) &&
    (core3Menu.routeEpoch == g_router.status.routeEpoch) &&
    (core3Menu.target == g_router.target) &&
    ((core3Menu.flags & OBSERVER_LINK_KVM_MENU_FLAG_LEASE_VALID) != 0u) &&
    (eventSeqReached(core3Menu.lastEventSeq, g_router.completedSeq) ||
     ((useCausal != 0u) && ((g_router.target == UART_LINK_KVM_TARGET_CORE3) ||
        (g_router.target == UART_LINK_KVM_TARGET_CORE4)))))
{
    uint8_t core3State;
    if (((g_router.target == UART_LINK_KVM_TARGET_CORE3) ||
        (g_router.target == UART_LINK_KVM_TARGET_CORE4)) &&
        (g_router.completedSeq != 0u) &&
        eventSeqReached(core3Menu.lastEventSeq,
            g_router.completedSeq))
    {
        g_router.causalHomeValid = 0u;
        useCausal = 0u;
    }
    if (((g_router.target == UART_LINK_KVM_TARGET_CORE3) ||
        (g_router.target == UART_LINK_KVM_TARGET_CORE4)) &&
        (g_router.leaseValid != 0u))
    {
        g_router.routeAccepted = 1u;
        g_router.core3ControlPending = 0u;
    }
    g_router.status.flags |= UART_LINK_KVM_FLAG_CORE3_VALID;
    if (((useCausal != 0u) && ((g_router.target == UART_LINK_KVM_TARGET_CORE3) ||
        (g_router.target == UART_LINK_KVM_TARGET_CORE4)))
        ? g_router.causalHome
        : ((core3Menu.flags & OBSERVER_LINK_KVM_MENU_FLAG_HOME_READY) != 0u))
        g_router.status.flags |= UART_LINK_KVM_FLAG_CORE3_HOME;
    core3State = ((useCausal != 0u) &&
        ((g_router.target == UART_LINK_KVM_TARGET_CORE3) ||
         (g_router.target == UART_LINK_KVM_TARGET_CORE4)))
        ? g_router.causalMenuState : core3Menu.menuState;
    g_router.status.menuStates |= (uint8_t) ((core3State & 0x0Fu) << 4);
}
g_router.status.activeTarget = inputOwner();
publishDisplayState();
if (g_router.routeAccepted != 0u) g_router.status.flags |= UART_LINK_KVM_FLAG_ROUTE_ACCEPTED;
if (g_router.leaseValid != 0u) g_router.status.flags |= UART_LINK_KVM_FLAG_LEASE_VALID;
g_router.status.core3AgeMs = (core3Age > 0xFFFFu) ? 0xFFFFu : (uint16_t) core3Age;
coreDataWriteKvmStatus(&g_router.status);
}

void core1KvmRouterInit(uint32_t tickMs)

```

```

{
    core3LinkCore1Status link;
    core4LinkCore1Status core4;
    (void) tickMs;
    memset(&g_router, 0, sizeof(g_router));
    core3LinkCore1GetStatus(&link);
    core4LinkCore1GetStatus(&core4);
    g_router.status.routerSession = (link.localSession ^ 0x4B564D31u) | 1u;
    g_router.status.targetSession = g_router.status.routerSession;
    g_router.status.target = UART_LINK_KVM_TARGET_CORE0;
    g_router.status.activeTarget = UART_LINK_KVM_TARGET_NONE;
    publishDisplayState();
    g_router.expectedSeq = 1u;
    g_router.target = UART_LINK_KVM_TARGET_CORE0;
    g_router.lastPeerSession = link.peerSession;
    g_router.lastCore4PeerSession = core4.peerSession;
    g_router.status.magic = CORE1_DATA_READY_MAGIC;
    coreDataWriteKvmStatus(&g_router.status);
}

void core1KvmRouterIngress(uint32_t tickMs)
{
    core1KvmIngressStruct ingress;
    core3LinkCore1Status link;
    core4LinkCore1Status core4;
    const keyEventToMenuStruct *base;
    uint8_t baseValid;

    core3LinkCore1GetStatus(&link);
    core4LinkCore1GetStatus(&core4);
    if (link.peerSession != g_router.lastPeerSession)
    {
        if ((g_router.lastPeerSession != 0u) &&
            (g_router.target == UART_LINK_KVM_TARGET_CORE3))
        {
            cancelPendingEvent(UART_LINK_KVM_ACK_PEER_RESTART);
            g_router.routeAccepted = 0u;
            g_router.leaseValid = 0u;
            g_router.core3ControlPending = 0u;
            g_router.status.flags |= UART_LINK_KVM_FLAG_PEER_RESTART;
        }
        g_router.lastPeerSession = link.peerSession;
    }
    if (core4.peerSession != g_router.lastCore4PeerSession)
    {
        if ((g_router.lastCore4PeerSession != 0u) &&
            (g_router.target == UART_LINK_KVM_TARGET_CORE4))
        {
            cancelPendingEvent(UART_LINK_KVM_ACK_PEER_RESTART);
            g_router.routeAccepted = 0u;
            g_router.leaseValid = 0u;
            g_router.core3ControlPending = 0u;
            g_router.status.flags |= UART_LINK_KVM_FLAG_PEER_RESTART;
        }
        g_router.lastCore4PeerSession = core4.peerSession;
    }
    baseValid = updateBaseSnapshot(tickMs);
    base = &g_router.baseSnapshot;
    if ((baseValid != 0u) && (base->ready == CORE1_DATA_READY_MAGIC))
    {
        g_router.upstreamControlActive = base->controlActive;
        if (base->linkUp == 0u)
        {
            cancelPendingEvent(UART_LINK_KVM_ACK_NO_LEASE);
            g_router.routeAccepted = 0u;
            g_router.leaseValid = 0u;
            g_router.core3ControlPending = 0u;
        }
    }
    else if (baseValid == 0u)
    {
        cancelPendingEvent(UART_LINK_KVM_ACK_NO_LEASE);
        g_router.routeAccepted = 0u;
        g_router.leaseValid = 0u;
        g_router.core3ControlPending = 0u;
    }
    if ((g_router.leaseValid != 0u) && deadlineReached(tickMs, g_router.leaseDeadlineMs))
    {
        cancelPendingEvent(UART_LINK_KVM_ACK_NO_LEASE);
    }
}

```

```

    g_router.leaseValid = 0u;
    g_router.routeAccepted = 0u;
    g_router.core3ControlPending = 0u;
}
enforceActiveObserverEligibility(&link, &core4);
coreDataReadKvmIngress(&ingress);
if ((ingress.magic == CORE1_DATA_READY_MAGIC) &&
    (ingress.updateSeq != g_router.lastIngressSeq))
{
    g_router.lastIngressSeq = ingress.updateSeq;
    if ((ingress.flags & CORE1_KVM_INGRESS_CONTROL_VALID) != 0u)
        applyControl(&ingress, &link, &core4, tickMs);
    if (((ingress.flags & CORE1_KVM_INGRESS_EVENT_VALID) != 0u) &&
        ((ingress.eventSeq != g_router.lastIngressEventSeq) ||
         (ingress.flags & CORE1_KVM_INGRESS_EVENT_EDGE) != 0u)))
    {
        g_router.lastIngressEventSeq = ingress.eventSeq;
        applyEvent(&ingress, &link, &core4);
    }
}
menuKeyRemoteSetLinkState(
    (uint8_t) ((baseValid != 0u) &&
               (base->ready == CORE1_DATA_READY_MAGIC) && (base->linkUp != 0u)),
    (uint8_t) (inputOwner() == UART_LINK_KVM_TARGET_CORE1),
    (baseValid != 0u) ? base->core2LinkLost : 1u);
}

void core1KvmRouterFinalize(uint32_t tickMs)
{
    core3LinkCore1Status link;
    core4LinkCore1Status core4;

    core3LinkCore1GetStatus(&link);
    core4LinkCore1GetStatus(&core4);
    enforceActiveObserverEligibility(&link, &core4);
    finishEvents();
    publishStatus(tickMs, &link, &core4);
    g_router.snapshotSeq++;
}

void core1KvmRouterGetSnapshot(core1KvmRouterSnapshot *out)
{
    if (out == 0) return;
    out->seq = g_router.snapshotSeq;
    out->routeSession = g_router.status.routeSession;
    out->routeEpoch = g_router.status.routeEpoch;
    out->targetSession = g_router.status.targetSession;
    out->core3AgeMs = g_router.status.core3AgeMs;
    out->desiredTarget = g_router.target;
    out->inputOwner = inputOwner();
    out->routeAccepted = g_router.routeAccepted;
    out->leaseValid = g_router.leaseValid;
    out->core1Valid = (uint8_t) ((g_router.status.flags & UART_LINK_KVM_FLAG_CORE1_VALID) != 0u);
    out->core3Valid = (uint8_t) ((g_router.status.flags & UART_LINK_KVM_FLAG_CORE3_VALID) != 0u);
    out->eventPending = (uint8_t) ((g_router.localEventPending != 0u) ||
                                   (g_router.core3EventPending != 0u) || (g_router.core3ControlPending != 0u));
    out->ackResult = g_router.status.ackResult;
    out->valid = 1u;
}

void core1KvmRouterGetSystemFacts(core1SystemFactsSnapshot *out)
{
    const keyEventToMenuStruct *base;
    if (out == 0) return;
    memset(out, 0, sizeof(*out));
    if (g_router.baseSnapshotValid == 0u) return;
    base = &g_router.baseSnapshot;
    out->ready = (uint8_t) (base->ready == CORE1_DATA_READY_MAGIC);
    out->linkUp = base->linkUp;
    out->core0StatusValid = base->core0StatusValid;
    out->core2LinkLost = base->core2LinkLost;
    out->core0SafetyState = base->core0SafetyState;
    out->core0StatusFlags = base->core0StatusFlags;
    out->core0RgbIntent = base->core0RgbIntent;
    out->core0SystemMode = base->core0SystemMode;
    out->core0AltReached = base->core0AltReached;
    out->baseAgeMs = g_router.baseSnapshotAgeMs;
}

```

```
uint8_t core1KvmRouterGetDisplayStateIsr(void)
{
    return g_displayState;
}
```

core1_cyt4bb7/project/code/app/core1KvmRouter.h

SHA-256

:

5fd1b5e015a5f3b700c76bf51c82ae4292581c560868cc9d66bc19252de63b89

代码 (c)

```
#ifndef CORE1_KVM_ROUTER_H
#define CORE1_KVM_ROUTER_H

#include <stdint.h>

typedef struct
{
    uint32_t seq;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t targetSession;
    uint16_t core3AgeMs;
    uint8_t desiredTarget;
    uint8_t inputOwner;
    uint8_t routeAccepted;
    uint8_t leaseValid;
    uint8_t core1Valid;
    uint8_t core3Valid;
    uint8_t eventPending;
    uint8_t ackResult;
    uint8_t valid;
} core1KvmRouterSnapshot;

typedef struct
{
    uint8_t ready;
    uint8_t linkUp;
    uint8_t core0StatusValid;
    uint8_t core2LinkLost;
    uint8_t core0SafetyState;
    uint8_t core0StatusFlags;
    uint8_t core0RgbIntent;
    uint8_t core0SystemMode;
    uint8_t core0AltReached;
    uint16_t baseAgeMs;
} core1SystemFactsSnapshot;

/* Snapshot 中的 desiredTarget/inputOwner 使用这一公共语义；NONE 保留 wire 侧 0xFF，
 * 调用方无需包含 uartLinkProto.h。 */
#define CORE1_KVM_TARGET_CORE0 (0u)
#define CORE1_KVM_TARGET_CORE1 (1u)
#define CORE1_KVM_TARGET_CORE3 (3u)
#define CORE1_KVM_TARGET_CORE4 (4u)
#define CORE1_KVM_TARGET_NONE (0xFFu)

#define CORE1_KVM_DISPLAY_TARGET_CORE0 (0u)
#define CORE1_KVM_DISPLAY_TARGET_CORE1 (1u)
#define CORE1_KVM_DISPLAY_TARGET_NONE (2u)
#define CORE1_KVM_DISPLAY_TARGET_CORE3 (3u)
#define CORE1_KVM_DISPLAY_TARGET_CORE4 (4u)
#define CORE1_KVM_DISPLAY_PACK(desired, owner) \
    (((uint8_t) (((uint8_t) (desired) & 0x07u) << 3) | ((uint8_t) (owner) & 0x07u)))
#define CORE1_KVM_DISPLAY_DESIRED(value) ((uint8_t) (((uint8_t) (value) >> 3) & 0x07u))
#define CORE1_KVM_DISPLAY_OWNER(value) ((uint8_t) ((uint8_t) (value) & 0x07u))

void core1KvmRouterInit(uint32_t tickMs);
/* 前台两阶段：Ingress 在 menuTick 前接收路由/事件；Finalize 在 menuTick 后完成 ACK 并发布。 */
void core1KvmRouterIngress(uint32_t tickMs);
void core1KvmRouterFinalize(uint32_t tickMs);
void core1KvmRouterGetSnapshot(core1KvmRouterSnapshot *out);
void core1KvmRouterGetSystemFacts(core1SystemFactsSnapshot *out);
/* PIT ISR 只读取这一字节，不复制前台多字状态。 */
uint8_t core1KvmRouterGetDisplayStateIsr(void);
```



```
#endif /* CORE1_KVM_ROUTER_H */
```

core1_cyt4bb7/project/code/app/core1Loop.c

SHA-256

:

7723434c6029a8e10e1abc752385c760e3b987e60bcb690c30a83b4d6f66ec7f

代码 (c)

```
#include "core1Loop.h"
#include "paramSyncOwnerService.h"
#include "paramSyncObserverRouter.h"

#include <string.h>
#include <math.h>

#include "zf_driver_pit.h"

#include "image.h"
#include "visionParams.h"
#include "menu.h"
#include "rgbStatusLed.h"
#include "beaconNavigator.h"
#include "takeoffMotionGate.h"
#include "core0PoseSupervisor.h"
#include "beaconTargetContinuity.h"
#include "coreDataTransfer.h"
#include "configBeaconFusion.h"
#include "configBeaconPlanning.h"
#include "core3LinkCore1.h"
#include "core4LinkCore1.h"
#include "uartLinkProto.h"
#include "core1KvmRouter.h"
#include "menuKeyRemote.h"
#include "core1ContinuityIdentity.h"
#include "core1NavigationDecision.h"
#include "observerOverlayPoseMapping.h"

#include "zf_driver_flash.h"

typedef char core1PlanSearchStateMustMatch[((uint8_t) BEACON_NAV_SEARCH == CORE1_PLAN_STATE_SEARCH) ? 1 : -1];

#define CORE1_CORE0_SAFETY_DISARMED (0u)

/*
 * core1_cm_7_1 主循环调度。视觉管线 + 远程菜单系统(IPS114) + RGB 状态灯。
 * processInit → imageInit + visionParamsLoad(flash) + menuInit + rgbStatusLedInit
 * pitProcess0 → menuPitTick + rgbStatusLedUpdate(PIT_CH2 10ms: 按键扫描 + 灯动效, 轻量)
 * whileProcess → KVM Ingress + menuServiceInput + imageProcess/规划 + menuTick + KVM Finalize
 * 显示由菜单统一接管(发车页即全屏信标图); 原 beaconDisplay 已移除。
 * RGB 灯反映两核(core0+core1)综合状态, 详见 rgbStatusLed.c。
 */

typedef struct
{
    volatile uint32_t tickMs; /* PIT 单写, 前台只读 */
    volatile uint16_t beaconCount;
    volatile uint8_t initReady;
    uint32_t planLastStepMs;
    uint32_t planSeq;
    planToLinkStruct planTx;
    uint8_t planForceStep;
    uint8_t lastDetectMode;
    uint8_t localObservationPending;
    uint32_t localObservationTickMs;
    uint32_t localObservationRevision;
    uint32_t localObservationHash;
    uint8_t localObservationDetectMode;
    uint32_t core3OverlaySeq;
    uint32_t core4OverlaySeq;
    uint32_t core3OverlayLastQueueMs;
    uint32_t core4OverlayLastQueueMs;
    uint32_t core3TrackerFrameSeq;
    uint32_t core4TrackerFrameSeq;
```

```

uint32_t readyStableStartMs;
uint8_t systemPhase;
uint8_t rejoinBarrierPending;
uint8_t rejoinBarrierActive;
uint8_t standalone;
uint8_t cameraReady;
beaconControlCandidate displayTarget;
core1ContinuityIdentityState continuityIdentity;
/* 大轨迹 DTO 由编排 owner 持有，避免每个 10ms 规划拍把 101 条本地轨迹和两份
 * 32 条远端轨迹同时压入 8KiB 栈；对其它 owner 仍只通过 snapshot/Step facade 交互。 */
imageObservationSnapshotStruct observation;
} core1RuntimeContextStruct;

static core1RuntimeContextStruct runtimeContext;
static beaconTargetContinuity runtimeTargetContinuity;
static beaconTargetContinuity runtimeDisplayContinuity;
static beaconConfirmedStruct runtimeCore3Tracks[OBSERVER_LINK_MAX_TRACKS];
static beaconConfirmedStruct runtimeCore4Tracks[OBSERVER_LINK_MAX_TRACKS];
static void core1ContinuityReset(void)
{
    beaconTargetContinuityInit(&runtimeTargetContinuity);
    beaconTargetContinuityInit(&runtimeDisplayContinuity);
}

static uint8_t continuityConfigIdentityValid(
    const observerLinkCore1EndpointStatus *status)
{
    if (status == NULL) return 0u;
    return (uint8_t) ((status->online != 0u) &&
        (status->protocolCompatible != 0u) &&
        (status->planningCompatible != 0u) &&
        (status->configCompatible != 0u) &&
        (status->configAckStatus == OBSERVER_LINK_CONFIG_APPLIED) &&
        (status->configAckRevision == status->desiredRevision) &&
        (status->configAckHash == status->desiredHash) &&
        (status->appliedMode == status->desiredMode));
}

static void core1ContinuityObserveIdentity(
    const core3LinkCore1Status *core3,
    const core4LinkCore1Status *core4,
    uint8_t launched)
{
    core1ContinuityIdentityInput input;
    if ((core3 == NULL) || (core4 == NULL)) return;
    memset(&input, 0, sizeof(input));
    input.launched = launched;
    input.core3ConfigValid = continuityConfigIdentityValid(core3);
    input.core4ConfigValid = continuityConfigIdentityValid(core4);
    input.core3Session = core3->peerSession;
    input.core4Session = core4->peerSession;
    input.core3Revision = core3->desiredRevision;
    input.core4Revision = core4->desiredRevision;
    input.core3Hash = core3->desiredHash;
    input.core4Hash = core4->desiredHash;
    (void) core1ContinuityIdentityApply(&runtimeContext.continuityIdentity,
        &input, &runtimeTargetContinuity,
        &runtimeContext.rejoinBarrierPending,
        &runtimeContext.planForceStep);
}

static uint16_t adaptObserverTracks(const observerLinkPlanningTrackSnapshot *wire,
    uint8_t expectedSource,
    beaconConfirmedStruct *tracks)
{
    uint8_t i;
    if ((wire == NULL) || (tracks == NULL) || (wire->valid == 0u) ||
        (wire->sourceId != expectedSource) ||
        (wire->retainedCount > OBSERVER_LINK_MAX_TRACKS)) return 0u;
    memset(tracks, 0, sizeof(runtimeCore4Tracks));
    for (i = 0u; i < wire->retainedCount; i++)
    {
        tracks[i].centerX = wire->tracks[i].centerX;
        tracks[i].centerY = wire->tracks[i].centerY;
        tracks[i].predX = wire->tracks[i].predX;
        tracks[i].predY = wire->tracks[i].predY;
        tracks[i].size = wire->tracks[i].size;
        tracks[i].predSize = wire->tracks[i].predSize;
        tracks[i].trackId = wire->tracks[i].trackId;
    }
}

```

```

        tracks[i].misses = wire->tracks[i].misses;
        tracks[i].fresh = (wire->tracks[i].flags &
                           OBSERVER_LINK_TRACK_FLAG_FRESH)
                           ? 1u
                           : 0u;
        tracks[i].flags = wire->tracks[i].flags;
        tracks[i].frameSeq = wire->trackerFrameSeq;
        tracks[i].velX = (float) wire->tracks[i].velXQ7 / 128.0f;
        tracks[i].velY = (float) wire->tracks[i].velYQ7 / 128.0f;
        tracks[i].areaVel = (float) wire->tracks[i].areaVelQ0;
    }
    return wire->retainedCount;
}

#define CORE1_INITIAL_LINK_WAIT_MS (2000u)
#define CORE1_REJOIN_STABLE_MS    (2000u)

#define runtimeInitReady          runtimeContext.initReady
#define runtimeBeaconCount        runtimeContext.beaconCount
#define runtimeTickMs             runtimeContext.tickMs
#define runtimePlanLastStepMs     runtimeContext.planLastStepMs
#define runtimePlanForceStep      runtimeContext.planForceStep
#define runtimePlanSeq            runtimeContext.planSeq
#define runtimePlanTx             runtimeContext.planTx
#define runtimeLastDetectMode     runtimeContext.lastDetectMode
#define runtimeLocalObservationPending runtimeContext.localObservationPending
#define runtimeLocalObservationTickMs runtimeContext.localObservationTickMs
#define runtimeLocalObservationRevision runtimeContext.localObservationRevision
#define runtimeLocalObservationHash runtimeContext.localObservationHash
#define runtimeLocalObservationDetectMode runtimeContext.localObservationDetectMode

typedef char core3_source_core1_wire_assert[(BEACON_SOURCE_CORE1 == OBSERVER_LINK_SOURCE_CORE1) ? 1 : -1];
typedef char core3_source_core3_wire_assert[(BEACON_SOURCE_CORE3 == OBSERVER_LINK_SOURCE_CORE3) ? 1 : -1];
typedef char core4_source_core4_wire_assert[(BEACON_SOURCE_CORE4 == OBSERVER_LINK_SOURCE_CORE4) ? 1 : -1];
typedef struct
{
    uint8_t active;
    uint8_t source;
    uint8_t faultClass;
    uint8_t inhibit;
} requiredVisionSnapshot;

typedef struct
{
    uint32_t revision;
    uint32_t changedTickMs;
} observerConfigGrace;

static observerConfigGrace g_core3ConfigGrace;
static observerConfigGrace g_core4ConfigGrace;

static uint8_t signValid(int8_t value)
{
    return (uint8_t) ((value == 1) || (value == -1));
}

static uint8_t sourceCalibrationValid(const visionParamsStruct *params,
                                     uint8_t source)
{
    if (params == NULL) return 0u;
    if (source == REQUIRED_VISION_SOURCE_CORE1)
    {
        return (uint8_t) (isfinite(params->c1Core0PoseOffsetX) &&
                           isfinite(params->c1Core0PoseOffsetY) &&
                           isfinite(params->planningKpVx) && isfinite(params->planningKpVy) &&
                           signValid(params->planningVxSign) &&
                           signValid(params->planningVySign) && signValid(params->planningWzSign));
    }
    if (source == REQUIRED_VISION_SOURCE_CORE3)
    {
        return (uint8_t) (isfinite(params->core3CenterX) &&
                           isfinite(params->core3CenterY) &&
                           (params->core3CenterX >= 0.0f) && (params->core3CenterX < 188.0f) &&
                           (params->core3CenterY >= 0.0f) && (params->core3CenterY < 120.0f) &&
                           isfinite(params->core3Scale) &&
                           (params->core3Scale >= CORE3_CALIBRATION_SCALE_MIN) &&
                           (params->core3Scale <= CORE3_CALIBRATION_SCALE_MAX) &&
                           isfinite(params->core3MountYawRad) &&
                           (params->core3MountYawRad >= CORE3_CALIBRATION_YAW_RAD_MIN) &&

```

```

        (params->core3MountYawRad <= CORE3_CALIBRATION_YAW_RAD_MAX) &&
        (params->core3MirrorX <= 1u) &&
        signValid(params->core3ErrorFwdSign) &&
        signValid(params->core3ErrorRightSign) &&
        isfinite(params->core3Core0PoseOffsetX) &&
        isfinite(params->core3Core0PoseOffsetY) &&
        isfinite(params->core3PlanningKpVx) &&
        isfinite(params->core3PlanningKpVy) &&
        signValid(params->core3PlanningVxSign) &&
        signValid(params->core3PlanningVySign));
    }
    return (uint8_t) (isfinite(params->core4CenterX) &&
        isfinite(params->core4CenterY) &&
        (params->core4CenterX >= 0.0f) && (params->core4CenterX < 188.0f) &&
        (params->core4CenterY >= 0.0f) && (params->core4CenterY < 120.0f) &&
        isfinite(params->core4Scale) &&
        (params->core4Scale >= CORE3_CALIBRATION_SCALE_MIN) &&
        (params->core4Scale <= CORE3_CALIBRATION_SCALE_MAX) &&
        isfinite(params->core4MountYawRad) &&
        (params->core4MountYawRad >= CORE3_CALIBRATION_YAW_RAD_MIN) &&
        (params->core4MountYawRad <= CORE3_CALIBRATION_YAW_RAD_MAX) &&
        (params->core4MirrorX <= 1u) &&
        signValid(params->core4ErrorFwdSign) &&
        signValid(params->core4ErrorRightSign) &&
        isfinite(params->core4Core0PoseOffsetX) &&
        isfinite(params->core4Core0PoseOffsetY) &&
        isfinite(params->core4PlanningKpVx) &&
        isfinite(params->core4PlanningKpVy) &&
        signValid(params->core4PlanningVxSign) &&
        signValid(params->core4PlanningVySign));
}

static requiredVisionSnapshot observerRequiredFault(
    const observerLinkCore1EndpointStatus *link,
    observerConfigGrace *grace,
    uint8_t source)
{
    requiredVisionSnapshot fault = {0u, 0u, 0u, 0u};
    uint8_t configApplied;
    if (link->desiredRevision != grace->revision)
    {
        grace->revision = link->desiredRevision;
        grace->changedTickMs = runtimeTickMs;
    }
    fault.active = 1u;
    fault.source = source;
    if (link->online == 0u) fault.faultClass = REQUIRED_VISION_FAULT_LINK;
    else if ((link->protocolCompatible == 0u) ||
        (link->planningCompatible == 0u) ||
        (link->algorithmsReady == 0u))
        fault.faultClass = REQUIRED_VISION_FAULT_PROTOCOL;
    else if (link->configProviderValid == 0u)
        fault.faultClass = REQUIRED_VISION_FAULT_CALIBRATION;
    else
    {
        configApplied = (uint8_t) ((link->configCompatible != 0u) &&
            (link->configAckStatus == OBSERVER_LINK_CONFIG_APPLIED) &&
            (link->configAckRevision == link->desiredRevision) &&
            (link->configAckHash == link->desiredHash) &&
            (link->appliedMode == link->desiredMode));
        if (configApplied == 0u)
        {
            if ((uint32_t) (runtimeTickMs - grace->changedTickMs) < 600u)
            {
                fault.faultClass = REQUIRED_VISION_FAULT_NONE;
                fault.inhibit = 1u;
            }
            else
                fault.faultClass = REQUIRED_VISION_FAULT_CONFIG;
        }
    }
    else if ((link->healthValid == 0u) ||
        (link->health.ownerSession != link->localSession) ||
        ((uint32_t) (runtimeTickMs - link->healthTickMs) >= 600u) ||
        ((link->health.flags & OBSERVER_LINK_HEALTH_FLAG_BRIDGE_FRESH) == 0u) ||
        (link->health.bridgeAgeMs >= 600u))
        fault.faultClass = REQUIRED_VISION_FAULT_BRIDGE;
    else if ((link->health.cameraState != OBSERVER_LINK_CAMERA_RUNNING) ||
        ((link->health.flags & OBSERVER_LINK_HEALTH_FLAG_FRAME_FRESH) == 0u) ||
        (link->health.frameAgeMs >= 200u))

```

```

        fault.faultClass = REQUIRED_VISION_FAULT_CAMERA;
    else
        fault.faultClass = REQUIRED_VISION_FAULT_NONE;
    }
    if (fault.faultClass == REQUIRED_VISION_FAULT_NONE) fault.active = 0u;
    return fault;
}

static requiredVisionSnapshot core1RequiredVisionFault(void)
{
    requiredVisionSnapshot fault = {0u, 0u, 0u, 0u};
    requiredVisionSnapshot core3Fault;
    requiredVisionSnapshot core4Fault;
    core3LinkCore1Status core3;
    core4LinkCore1Status core4;
    visionParamsStruct params;
    visionParamsGetActive(&params);
    if (sourceCalibrationValid(&params, REQUIRED_VISION_SOURCE_CORE1) == 0u)
    {
        fault.active = 1u;
        fault.source = REQUIRED_VISION_SOURCE_CORE1;
        fault.faultClass = REQUIRED_VISION_FAULT_CALIBRATION;
        return fault;
    }
    if (sourceCalibrationValid(&params, REQUIRED_VISION_SOURCE_CORE3) == 0u)
    {
        fault.active = 1u;
        fault.source = REQUIRED_VISION_SOURCE_CORE3;
        fault.faultClass = REQUIRED_VISION_FAULT_CALIBRATION;
        return fault;
    }
    if (sourceCalibrationValid(&params, REQUIRED_VISION_SOURCE_CORE4) == 0u)
    {
        fault.active = 1u;
        fault.source = REQUIRED_VISION_SOURCE_CORE4;
        fault.faultClass = REQUIRED_VISION_FAULT_CALIBRATION;
        return fault;
    }
    if ((runtimeContext.cameraReady == 0u) ||
        ((uint32_t) (runtimeTickMs - runtimeLocalObservationTickMs) >= 200u))
    {
        fault.active = 1u;
        fault.source = REQUIRED_VISION_SOURCE_CORE1;
        fault.faultClass = REQUIRED_VISION_FAULT_CAMERA;
        return fault;
    }
    core3LinkCore1GetStatus(&core3);
    core3Fault = observerRequiredFault(&core3, &g_core3ConfigGrace,
        REQUIRED_VISION_SOURCE_CORE3);
    if (core3Fault.active != 0u) return core3Fault;
    core4LinkCore1GetStatus(&core4);
    core4Fault = observerRequiredFault(&core4, &g_core4ConfigGrace,
        REQUIRED_VISION_SOURCE_CORE4);
    if (core4Fault.active != 0u) return core4Fault;
    if (core3Fault.inhibit != 0u) return core3Fault;
    return core4Fault;
}

static uint8_t core1RuntimeRequiredLinksReady(core1SystemFactsSnapshot *factsOut)
{
    core1SystemFactsSnapshot facts;
    requiredVisionSnapshot visionFault;
    uint8_t ready;
    core1KvmRouterGetSystemFacts(&facts);
    ready = (uint8_t) ((facts.ready != 0u) &&
        (facts.linkUp != 0u) && (facts.core0StatusValid != 0u) &&
        (facts.core2LinkLost == 0u));
    visionFault = core1RequiredVisionFault();
    if ((visionFault.active != 0u) || (visionFault.inhibit != 0u)) ready = 0u;
    if (factsOut != NULL) *factsOut = facts;
    return ready;
}

static void core1RuntimeUpdateSystemPhase(void)
{
    uint8_t ready = core1RuntimeRequiredLinksReady(NULL);
    if (runtimeContext.systemPhase == OBSERVER_LINK_SYSTEM_PHASE_LINK_WAIT)
    {
        if (ready != 0u)

```

```

    {
        runtimeContext.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_HOME;
        runtimeContext.standalone = 0u;
    }
    else if (runtimeTickMs >= CORE1_INITIAL_LINK_WAIT_MS)
    {
        runtimeContext.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_STANDALONE;
        runtimeContext.standalone = 1u;
    }
    return;
}
if (runtimeContext.standalone == 0u)
{
    if (ready == 0u)
    {
        runtimeContext.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_STANDALONE;
        runtimeContext.standalone = 1u;
        runtimeContext.readyStableStartMs = 0u;
    }
    return;
}
if (ready == 0u)
{
    runtimeContext.readyStableStartMs = 0u;
    runtimeContext.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_STANDALONE;
    return;
}
if (runtimeContext.readyStableStartMs == 0u)
{
    runtimeContext.readyStableStartMs = runtimeTickMs;
}
runtimeContext.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_REJOIN;
if ((uint32_t) (runtimeTickMs - runtimeContext.readyStableStartMs) >=
    CORE1_REJOIN_STABLE_MS)
{
    /* 只复位规划动态历史，不重初始化相机、UART、屏幕、参数或 PIT。 */
    core3LinkCore1FencePlanningFacts();
    core4LinkCore1FencePlanningFacts();
    beaconNavigatorResetForRejoin();
    core1ContinuityReset();
    core1ContinuityIdentityReset(&runtimeContext.continuityIdentity);
    runtimeLocalObservationPending = 0u;
    runtimePlanForceStep = 1u;
    runtimeContext.rejoinBarrierPending = 1u;
    runtimeContext.standalone = 0u;
    runtimeContext.readyStableStartMs = 0u;
    /* REJOIN 保持到零输出屏障拍已实际发布；不能提前向 UI 宣称 HOME。 */
    runtimeContext.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_REJOIN;
}
}

static uint8_t overlayPixel(float value, uint16_t limit, uint16_t *out)
{
    if ((out == NULL) || (!isfinite(value)) || (value < 0.0f) ||
        (value > (float) (limit - 1u))) return 0u;
    *out = (uint16_t) (value + 0.5f);
    return 1u;
}

typedef struct
{
    uint8_t sourceId;
    uint8_t mirrorX;
    uint8_t geometryMode;
    int8_t errorFwdSign;
    int8_t errorRightSign;
    float centerX;
    float centerY;
    float poseOffsetX;
    float poseOffsetY;
    float scale;
    float mountYawRad;
    float bodyHeadingRad;
} observerOverlayProfile;

static uint8_t observerOverlayProfileGet(const visionParamsStruct *params,
    uint8_t sourceId, observerOverlayProfile *profile)
{
    if ((params == NULL) || (profile == NULL) ||

```

```

        ((sourceId != OBSERVER_LINK_SOURCE_CORE3) &&
         (sourceId != OBSERVER_LINK_SOURCE_CORE4))) return 0u;
memset(profile, 0, sizeof(*profile));
profile->sourceId = sourceId;
if (sourceId == OBSERVER_LINK_SOURCE_CORE4)
{
    profile->geometryMode = BEACON_SOURCE_GEOMETRY_OBSERVER_HEADING;
    profile->centerX = params->core4CenterX;
    profile->centerY = params->core4CenterY;
    profile->poseOffsetX = params->core4Core0PoseOffsetX;
    profile->poseOffsetY = params->core4Core0PoseOffsetY;
    profile->scale = params->core4Scale;
    profile->mountYawRad = params->core4MountYawRad;
    profile->mirrorX = params->core4MirrorX;
    profile->errorFwdSign = params->core4ErrorFwdSign;
    profile->errorRightSign = params->core4ErrorRightSign;
}
else
{
    profile->geometryMode = BEACON_SOURCE_GEOMETRY_OBSERVER_FIXED;
    profile->centerX = params->core3CenterX;
    profile->centerY = params->core3CenterY;
    profile->poseOffsetX = params->core3Core0PoseOffsetX;
    profile->poseOffsetY = params->core3Core0PoseOffsetY;
    profile->scale = params->core3Scale;
    profile->mountYawRad = params->core3MountYawRad;
    profile->mirrorX = params->core3MirrorX;
    profile->errorFwdSign = params->core3ErrorFwdSign;
    profile->errorRightSign = params->core3ErrorRightSign;
}
return (uint8_t) (isfinite(profile->centerX) && isfinite(profile->centerY) &&
                  isfinite(profile->poseOffsetX) && isfinite(profile->poseOffsetY) &&
                  isfinite(profile->scale) && isfinite(profile->mountYawRad) &&
                  (profile->centerX >= 0.0f) && (profile->centerX < 188.0f) &&
                  (profile->centerY >= 0.0f) && (profile->centerY < 120.0f) &&
                  (profile->scale >= CORE3_CALIBRATION_SCALE_MIN) &&
                  (profile->scale <= CORE3_CALIBRATION_SCALE_MAX) &&
                  (profile->mountYawRad >= CORE3_CALIBRATION_YAW_RAD_MIN) &&
                  (profile->mountYawRad <= CORE3_CALIBRATION_YAW_RAD_MAX) &&
                  (profile->mirrorX <= 1u) &&
                  ((profile->errorFwdSign == 1) || (profile->errorFwdSign == -1)) &&
                  ((profile->errorRightSign == 1) || (profile->errorRightSign == -1)));
}

static uint8_t bodyErrorToObserverVector(const observerOverlayProfile *profile,
float errFwd, float errRight, float *u, float *v)
{
    beaconSourceProfile sourceProfile;
    if ((profile == NULL) || (u == NULL) || (v == NULL) ||
        (!isfinite(errFwd)) || (!isfinite(errRight))) return 0u;
    memset(&sourceProfile, 0, sizeof(sourceProfile));
    sourceProfile.centerX = profile->centerX;
    sourceProfile.centerY = profile->centerY;
    sourceProfile.scale = profile->scale;
    sourceProfile.mountYawRad = profile->mountYawRad;
    sourceProfile.bodyHeadingRad = profile->bodyHeadingRad;
    sourceProfile.mirrorX = profile->mirrorX;
    sourceProfile.errorFwdSign = profile->errorFwdSign;
    sourceProfile.errorRightSign = profile->errorRightSign;
    sourceProfile.geometryMode = profile->geometryMode;
    return beaconTargetContinuityBodyToObserver(&sourceProfile,
        errFwd, errRight, u, v);
}

static uint8_t bodyErrorToObserverPixel(const observerOverlayProfile *profile,
float centerX, float centerY, float errFwd, float errRight,
uint16_t *x, uint16_t *y)
{
    float u;
    float v;
    if (bodyErrorToObserverVector(profile, errFwd, errRight, &u, &v) == 0u)
        return 0u;
    return (uint8_t) (overlayPixel(centerX + u / profile->scale, 188u, x) &&
                     overlayPixel(centerY + v / profile->scale, 120u, y));
}

static void publishObserverViewOverlay(uint8_t observerSource,
const beaconNavigatorSnapshotStruct *nav)
{

```

```

observerLinkCore1EndpointStatus link;
observerLinkViewOverlay overlay;
observerOverlayProfile profile;
visionParamsStruct params;
float centerX;
float centerY;
observerOverlayPoseMapping poseMapping;
uint32_t sourceFrameSeq;
uint32_t *overlaySeq;
uint32_t *lastQueueMs;

if (nav == NULL) return;
if (observerSource == OBSERVER_LINK_SOURCE_CORE4)
{
    overlaySeq = &runtimeContext.core4OverlaySeq;
    lastQueueMs = &runtimeContext.core4OverlayLastQueueMs;
    sourceFrameSeq = runtimeContext.core4TrackerFrameSeq;
    core4LinkCore1GetStatus(&link);
}
else
{
    overlaySeq = &runtimeContext.core3OverlaySeq;
    lastQueueMs = &runtimeContext.core3OverlayLastQueueMs;
    sourceFrameSeq = runtimeContext.core3TrackerFrameSeq;
    core3LinkCore1GetStatus(&link);
}
if ((uint32_t) (runtimeTickMs - *lastQueueMs) < 100u) return;
if ((link.online == 0u) || (link.protocolCompatible == 0u) ||
    (link.configAckStatus != OBSERVER_LINK_CONFIG_APPLIED) ||
    (link.configAckRevision != link.desiredRevision) ||
    (link.configAckHash != link.desiredHash) ||
    (link.appliedMode != link.desiredMode)) return;
visionParamsGetActive(&params);
if (observerOverlayProfileGet(&params, observerSource, &profile) == 0u) return;
if (profile.geometryMode == BEACON_SOURCE_GEOMETRY_OBSERVER_HEADING)
    profile.bodyHeadingRad = nav->poseHeadingRad;
centerX = (nav->poseValid != 0u) ? nav->poseRawX + profile.poseOffsetX : profile.centerX;
centerY = (nav->poseValid != 0u) ? nav->poseRawY + profile.poseOffsetY : profile.centerY;
if (!isfinite(centerX) || !isfinite(centerY)) return;

memset(&overlay, 0, sizeof(overlay));
overlay.overlaySeq = ++(*overlaySeq);
if (overlay.overlaySeq == 0u) overlay.overlaySeq = ++(*overlaySeq);
overlay.sourceFrameSeq = sourceFrameSeq;
overlay.ownerSession = link.localSession;
overlay.configRevision = link.desiredRevision;
overlay.configHash = link.desiredHash;
overlay.detectMode = link.desiredMode;
overlay.flags = OBSERVER_LINK_OVERLAY_FLAG_VALID;

if (overlayPixel(centerX, 188u, &overlay.activeCenterX) &&
    overlayPixel(centerY, 120u, &overlay.activeCenterY))
{
    overlay.flags |= OBSERVER_LINK_OVERLAY_FLAG_CENTER;
    if (isfinite(params.planningDeadzoneEnterPx) &&
        isfinite(params.planningDeadzoneExitPx) &&
        (profile.scale > 0.0f))
    {
        float enterRadius = params.planningDeadzoneEnterPx / profile.scale;
        float exitRadius = params.planningDeadzoneExitPx / profile.scale;
        if ((enterRadius >= 0.0f) && (enterRadius <= 188.0f) &&
            (exitRadius >= 0.0f) && (exitRadius <= enterRadius))
        {
            overlay.deadzoneEnterRadius = (uint16_t) (enterRadius + 0.5f);
            overlay.deadzoneExitRadius = (uint16_t) (exitRadius + 0.5f);
            overlay.flags |= OBSERVER_LINK_OVERLAY_FLAG_DEADZONE;
        }
    }
}

if ((sourceFrameSeq != 0u) &&
    (nav->targetValid != 0u) &&
    (nav->targetTrackId != 0u) && (nav->targetSize <= 22560u))
{
    uint8_t selectedOk;
    if (nav->sourceId == observerSource)
    {
        selectedOk = (uint8_t) (overlayPixel((float) nav->controlX, 188u,
            &overlay.selectedX) &&

```



```

        overlayPixel((float) nav->controlY, 120u,
            &overlay.selectedY));
    }
    else
    {
        selectedOk = (uint8_t) (((profile.geometryMode !=
            BEACON_SOURCE_GEOMETRY_OBSERVER_HEADING) ||
            ((nav->poseHeadingValid != 0u) && isfinite(profile.bodyHeadingRad))) &&
            bodyErrorToObserverPixel(&profile, centerX, centerY,
            nav->errFwd, nav->errRight, &overlay.selectedX, &overlay.selectedY));
    }
    if (selectedOk != 0u)
    {
        overlay.selectedTrackId = nav->targetTrackId;
        overlay.selectedSize = nav->targetSize;
        overlay.selectedFlags = (nav->targetHolding != 0u) ? OBSERVER_LINK_OVERLAY_SELECTED_COAST :
OBSERVER_LINK_OVERLAY_SELECTED_FRESH;
        overlay.flags |= OBSERVER_LINK_OVERLAY_FLAG_SELECTED;
    }
}

observerOverlayPoseMap(nav->poseRawX, nav->poseRawY,
    profile.poseOffsetX, profile.poseOffsetY, nav->poseValid,
    nav->poseHeadingRad, nav->poseHeadingValid, &poseMapping);
if (poseMapping.poseMapped != 0u)
{
    overlay.poseX = poseMapping.x;
    overlay.poseY = poseMapping.y;
    overlay.poseFlags = nav->poseHolding ? OBSERVER_LINK_OVERLAY_POSE_HOLD : OBSERVER_LINK_OVERLAY_POSE_FRESH;
    overlay.flags |= OBSERVER_LINK_OVERLAY_FLAG_POSE;
    if (poseMapping.headingMapped != 0u)
    {
        overlay.poseHeadingQ8 = poseMapping.headingQ8;
        overlay.poseFlags |= OBSERVER_LINK_OVERLAY_POSE_HEADING_VALID;
        if (nav->poseHeadingHolding != 0u)
        {
            overlay.poseFlags &= (uint8_t) ~OBSERVER_LINK_OVERLAY_POSE_FRESH;
            overlay.poseFlags |= OBSERVER_LINK_OVERLAY_POSE_HOLD;
        }
    }
}

if (((observerSource == OBSERVER_LINK_SOURCE_CORE4) ? core4LinkCore1QueueViewOverlay(&overlay, runtimeTickMs) :
core3LinkCore1QueueViewOverlay(&overlay, runtimeTickMs)) != 0u)
    *lastQueueMs = runtimeTickMs;
}

void processInit(void)
{
    visionParamsStruct params;
    paramSyncOwnerServiceInit();
    /* PIT_CH2 10ms: 菜单按键扫描 + 刷新计数(对齐 core0 menuPitTick; PIT_CH1 留给图像/通用)。
    * cm_7_1 用 PIT_CH2, 不与飞控核(cm_7_0)的 CH0/CH1 跨核争用。 */
    pit_ms_init(PIT_CH2, 10u);

    /* 信标视觉管线初始化(mt9v03x + DetectMode 检测器 + 跟踪)。失败不阻塞主循环。 */
    runtimeContext.cameraReady = imageInit();

    /* 视觉参数: flash 加载(校验失败回退默认)→热重载到 DetectMode/track。
    * 须在 imageInit(已建两套检测默认配置)之后, 覆盖其默认阈值。 */
    flash_init();
    visionParamsLoad();
    visionParamsGetActive(&params);
    runtimeLastDetectMode = params.detectMode;

    /* 后/前观察板专线: 两实例独立收发, 协议状态机均在前台。 */
    core3LinkCore1Init(0u);
    core4LinkCore1Init(0u);
    core1KvmRouterInit(0u);
    paramSyncObserverRouterInit();

    /* 远程菜单(IPS114 占屏 cm_7_1): 上电先等 core0 链路在线才进发车页。 */
    menuInit();
    menuSetInitHealth(runtimeContext.cameraReady ? 0u : 1u, 0u);

    /* RGB 状态灯(P05 三路 PWM, 共阳极, 限亮 30%): 反映两核综合状态, 默认灭。 */
    rgbStatusLedInit();
    rgbStatusLedSetHardwareInitFault(runtimeContext.cameraReady ? 0u : 1u);

```

```

/* 路径规划(beaconNavigator): 复位状态机 + 输出滤波器。须在 imageInit 之后(消费确认信标)。 */
beaconNavigatorInit();
beaconTargetContinuityInit(&runtimeTargetContinuity);
beaconTargetContinuityInit(&runtimeDisplayContinuity);
core0PoseSupervisorInit();
takeoffMotionGateInit();
takeoffMotionGateApplyParams(params.planningPostTakeoffDelayMs);

runtimeContext.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_LINK_WAIT;
runtimeContext.readyStableStartMs = 0u;
runtimeContext.rejoinBarrierPending = 0u;
runtimeContext.rejoinBarrierActive = 0u;
core1ContinuityIdentityReset(&runtimeContext.continuityIdentity);
runtimeContext.standalone = 0u;

runtimeInitReady = 1u;
}

void pitProcess0(void)
{
    if (!runtimeInitReady) return;

    /* 菜单 10ms tick: 读核间透传按键(menuKeyScan) + 刷新分频。轻量, 渲染在前台。 */
    menuPitTick();

    /* RGB 状态灯 10ms 刷新(稳定节拍软生成闪烁/呼吸): 读本地 + 共享内存 core0 状态, 设三路 PWM。
     * 仅寄存器写 + 共享内存读(DCache invalidate 32B), 轻量, 与 menuKeyScan 同级开销。 */
    rgbStatusLedUpdate(runtimeInitReady, (uint8_t) 0u, runtimeBeaconCount);

    runtimeTickMs += 10u;
}

static void publishCore3ControlStatus(
    const beaconNavigatorSnapshotStruct *navSnapshot)
{
    core3LinkCore1Status linkStatus;
    core4LinkCore1Status core4LinkStatus;
    observerLinkControlStatus status;
    observerLinkControlStatus core4Status;
    core1KvmRouterSnapshot kvm;
    core1SystemFactsSnapshot facts;
    uint8_t core0Up;
    uint8_t core0Parked;
    uint8_t baseReason;
    uint8_t requiredLinksReady;

    memset(&status, 0, sizeof(status));
    core3LinkCore1GetStatus(&linkStatus);
    status.detectMode = runtimeLastDetectMode;
    status.protocolCompatible = linkStatus.protocolCompatible;
    status.algorithmsReady = linkStatus.algorithmsReady;
    status.configCompatible = linkStatus.configCompatible;
    core1KvmRouterGetSnapshot(&kvm);
    status.desiredTarget = kvm.desiredTarget;
    status.ownerTarget = (kvm.inputOwner == UART_LINK_KVM_TARGET_NONE) ? OBSERVER_LINK_KVM_TARGET_NONE : kvm.inputOwner;
    core0Up = menuKeyRemoteLinkUp();
    status.systemPhase = runtimeContext.systemPhase;
    status.systemFlags = 0u;
    if (runtimeContext.systemPhase == OBSERVER_LINK_SYSTEM_PHASE_STANDALONE)
        status.systemFlags |= OBSERVER_LINK_SYSTEM_FLAG_STANDALONE;
    else if (runtimeContext.systemPhase == OBSERVER_LINK_SYSTEM_PHASE_REJOIN)
        status.systemFlags |= OBSERVER_LINK_SYSTEM_FLAG_REJOIN;
    requiredLinksReady = core1RuntimeRequiredLinksReady(&facts);
    core0Parked = (uint8_t) ((facts.core0StatusValid != 0u) &&
        (facts.core0SafetyState == CORE1_CORE0_SAFETY_DISARMED) &&
        ((facts.core0StatusFlags & UART_LINK_C0_FLAG_LAUNCHED) == 0u));
    if ((facts.core0StatusFlags & UART_LINK_C0_FLAG_LAUNCHED) != 0u)
        status.systemFlags |= OBSERVER_LINK_SYSTEM_FLAG_LAUNCHED;
    if ((facts.core0StatusFlags & UART_LINK_C0_FLAG_VEL_VALID) != 0u)
        status.systemFlags |= OBSERVER_LINK_SYSTEM_FLAG_VEL_VALID;
    if ((requiredLinksReady != 0u) &&
        ((facts.core0StatusFlags & (UART_LINK_C0_FLAG_CM70_READY |
            UART_LINK_C0_FLAG_TOF_HEALTHY)) ==
            (UART_LINK_C0_FLAG_CM70_READY | UART_LINK_C0_FLAG_TOF_HEALTHY)))
        status.systemFlags |= OBSERVER_LINK_SYSTEM_FLAG_LAUNCH_ELIGIBLE;
    if (core0Up == 0u)
    {
        baseReason = OBSERVER_LINK_REASON_CORE0_LINK;
    }
}

```

```

else if (facts.core2LinkLost != 0u)
{
    baseReason = OBSERVER_LINK_REASON_CORE2_LINK;
}
else if (runtimePlanTx.requiredVisionFault != 0u)
{
    if (runtimePlanTx.faultClass == REQUIRED_VISION_FAULT_LINK ||
        runtimePlanTx.faultClass == REQUIRED_VISION_FAULT_BRIDGE)
        baseReason = OBSERVER_LINK_REASON_OBSERVER_LINK;
    else if (runtimePlanTx.faultClass == REQUIRED_VISION_FAULT_PROTOCOL)
        baseReason = OBSERVER_LINK_REASON_PROTOCOL;
    else if (runtimePlanTx.faultClass == REQUIRED_VISION_FAULT_CONFIG)
        baseReason = OBSERVER_LINK_REASON_CONFIG;
    else
        baseReason = OBSERVER_LINK_REASON_SAFETY;
}
else
{
    baseReason = OBSERVER_LINK_REASON_NONE;
}
status.primaryReason = baseReason;

/* 观察板本地参数安全门只接受这一份由 Core0 当前 DISARMED 事实生成的
 * canonical SAFE DTO。ARMING/TOF_INIT/CLOSED_LOOP 及状态无效均发布 RUN。 */
if (core0Parked != 0u)
{
    status.cameraControlState = OBSERVER_LINK_CAMERA_CONTROL_SAFE;
    core4Status = status;
    core4LinkCore1GetStatus(&core4LinkStatus);
    core4Status.protocolCompatible = core4LinkStatus.protocolCompatible;
    core4Status.algorithmsReady = core4LinkStatus.algorithmsReady;
    core4Status.configCompatible = core4LinkStatus.configCompatible;
    core3LinkCore1PublishControlStatus(&status, runtimeTickMs);
    core4LinkCore1PublishControlStatus(&core4Status, runtimeTickMs);
    return;
}

status.cameraControlState = OBSERVER_LINK_CAMERA_CONTROL_RUN;
status.flags = OBSERVER_LINK_CONTROL_FLAG_STATUS_VALID |
    OBSERVER_LINK_CONTROL_FLAG_REQUIRED_VISION;
if (navSnapshot == NULL)
{
    core4Status = status;
    core4LinkCore1GetStatus(&core4LinkStatus);
    core4Status.protocolCompatible = core4LinkStatus.protocolCompatible;
    core4Status.algorithmsReady = core4LinkStatus.algorithmsReady;
    core4Status.configCompatible = core4LinkStatus.configCompatible;
    core3LinkCore1PublishControlStatus(&status, runtimeTickMs);
    core4LinkCore1PublishControlStatus(&core4Status, runtimeTickMs);
    return;
}
if (runtimePlanTx.valid != 0u)
{
    status.flags |= OBSERVER_LINK_CONTROL_FLAG_DECISION_VALID;
}
if (runtimeContext.rejoinBarrierActive != 0u)
{
    status.flags |= OBSERVER_LINK_CONTROL_FLAG_BARRIER_REQUIRED;
}
if (runtimePlanTx.dispatchEnable != 0u)
{
    status.flags |= OBSERVER_LINK_CONTROL_FLAG_DISPATCH_ENABLED;
}
if (runtimePlanTx.requiredVisionFault == 0u)
{
    status.flags |= OBSERVER_LINK_CONTROL_FLAG_REMOTE_ELIGIBLE;
}
if (core0Up != 0u)
{
    status.flags |= OBSERVER_LINK_CONTROL_FLAG_CORE0_LINK_UP;
    if (menuKeyRemoteCore2LinkLost() == 0u)
    {
        status.flags |= OBSERVER_LINK_CONTROL_FLAG_CORE2_LINK_UP;
    }
}
if (navSnapshot->targetValid != 0u)
{
    status.systemFlags |= OBSERVER_LINK_SYSTEM_FLAG_TARGET_VALID;
}

```

```

if ((core0Up != 0u) && (menuKeyRemoteCore2LinkLost() != 0u))
{
    status.primaryReason = OBSERVER_LINK_REASON_CORE2_LINK;
}
else if (runtimeContext.rejoinBarrierActive != 0u)
{
    status.primaryReason = OBSERVER_LINK_REASON_BARRIER;
}
else if ((runtimePlanTx.valid != 0u) &&
         (runtimePlanTx.dispatchEnable == 0u))
{
    status.primaryReason = OBSERVER_LINK_REASON_DISPATCH;
}
else if (navSnapshot->targetValid == 0u)
{
    status.primaryReason = OBSERVER_LINK_REASON_TARGET;
}

status.activeSource = navSnapshot->sourceId;
if (runtimePlanTx.beaconSearching != 0u)
{
    status.activeSource = OBSERVER_LINK_SOURCE_NONE;
    status.controlClass = OBSERVER_LINK_CONTROL_CLASS_SEARCH;
}
else if (navSnapshot->targetValid != 0u)
{
    status.controlClass = (navSnapshot->sourceId == OBSERVER_LINK_SOURCE_CORE1) ? OBSERVER_LINK_CONTROL_CLASS_CORE1_TRACK :
OBSERVER_LINK_CONTROL_CLASS_OBSERVER_TRACK;
}
else
{
    status.activeSource = OBSERVER_LINK_SOURCE_NONE;
    status.controlClass = OBSERVER_LINK_CONTROL_CLASS_NONE;
}

status.navState = navSnapshot->state;
status.beaconCount = navSnapshot->beaconCount;
status.targetX = navSnapshot->controlX;
status.targetY = navSnapshot->controlY;
status.targetSize = navSnapshot->targetSize;
if ((observerLinkQuantizeSigned(navSnapshot->errX, 16.0f,
    -2048.0f, 2047.9375f, &status.errXQ4) == 0u) ||
    (observerLinkQuantizeSigned(navSnapshot->errY, 16.0f,
    -2048.0f, 2047.9375f, &status.errYQ4) == 0u))
{
    status.cameraControlState = OBSERVER_LINK_CAMERA_CONTROL_RUN;
    status.activeSource = OBSERVER_LINK_SOURCE_NONE;
    status.controlClass = OBSERVER_LINK_CONTROL_CLASS_NONE;
    status.flags = OBSERVER_LINK_CONTROL_FLAG_STATUS_VALID |
OBSERVER_LINK_CONTROL_FLAG_REQUIRED_VISION;
    status.navState = 0u;
    status.beaconCount = 0u;
    status.targetX = 0u;
    status.targetY = 0u;
    status.targetSize = 0u;
    status.errXQ4 = 0;
    status.errYQ4 = 0;
    status.systemPhase = runtimeContext.systemPhase;
    status.primaryReason = baseReason;
    status.desiredTarget = kvm.desiredTarget;
    status.ownerTarget = (kvm.inputOwner == UART_LINK_KVM_TARGET_NONE) ? OBSERVER_LINK_KVM_TARGET_NONE : kvm.inputOwner;
}
core4Status = status;
core4LinkCore1GetStatus(&core4LinkStatus);
core4Status.protocolCompatible = core4LinkStatus.protocolCompatible;
core4Status.algorithmsReady = core4LinkStatus.algorithmsReady;
core4Status.configCompatible = core4LinkStatus.configCompatible;
core3LinkCore1PublishControlStatus(&status, runtimeTickMs);
core4LinkCore1PublishControlStatus(&core4Status, runtimeTickMs);
}

void whileProcess(void)
{
    if (!runtimeInitReady) return;

    core3LinkCore1Poll(runtimeTickMs);
    core4LinkCore1Poll(runtimeTickMs);
    paramSyncObserverRouterPoll(runtimeTickMs);
    core1KvmRouterIngress(runtimeTickMs);
}

```

```

core3LinkCore1FlushControl(runtimeTickMs);
core4LinkCore1FlushControl(runtimeTickMs);
{
    core1SystemFactsSnapshot facts;
    core1KvmRouterGetSystemFacts(&facts);
    rgbStatusLedPublishCore0Facts(facts.ready, facts.linkUp,
        facts.core0StatusValid, facts.core0SafetyState,
        facts.core0StatusFlags, facts.core0RgbIntent);
}
/* KVM 事实进入后立即消费按键；屏幕与 Flash 副作用仍留在循环尾。 */
menuServiceInput();
/* 菜单动作一旦提交即生成终态 ACK，不等待视觉、规划或 IPS114 同步绘制。 */
core1KvmRouterFinalize(runtimeTickMs);
core3LinkCore1FlushControl(runtimeTickMs);
/* 同一 surface 的选项/数值变化立即脏区绘制，避免等待本拍重视觉；不清屏、不写 Flash。 */
menuRenderInputFeedback();
core1RuntimeUpdateSystemPhase();

if (runtimeLastDetectMode != imageGetDetectMode())
{
    runtimeLastDetectMode = imageGetDetectMode();
    runtimeBeaconCount = 0u;
    runtimeLocalObservationPending = 0u;
    beaconNavigatorInit();
    core1ContinuityReset();
    runtimeContext.rejoinBarrierPending = 1u;
    runtimePlanForceStep = 1u;
}

/* 前台跑信标视觉管线：取帧→BinaryCC→时间一致性跟踪。
 * 连通域/Hu 计算必须在前台，绝不进 PIT ISR。 */
if ((runtimeContext.cameraReady != 0u) && imageProcess())
{
    visionParamsStruct frameParams;
    imageGetObservationSnapshot(&runtimeContext.observation);
    visionParamsGetActive(&frameParams);
    runtimeBeaconCount = 0u;
    for (uint16_t i = 0u; i < runtimeContext.observation.count; i++)
    {
        if ((runtimeContext.observation.tracks[i].flags &
            BEACON_CONFIRMED_FLAG_BORDER_REJECTED) == 0u)
            runtimeBeaconCount++;
    }
    runtimeLocalObservationPending = 1u;
    runtimeLocalObservationTickMs = runtimeTickMs;
    runtimeLocalObservationRevision = frameParams.sharedConfigRevision;
    runtimeLocalObservationHash = frameParams.sharedConfigHash;
    runtimeLocalObservationDetectMode = frameParams.detectMode;
}

/* 路径规划 10ms 节拍：取最新确认信标 → beaconNavigator(选目标→误差→控制律→整形)
 * → 写跨核反向区(planRegion)供 cm_7_0 按空地系统模式派发。重计算全在前台。 */
if (runtimePlanForceStep || runtimeTickMs != runtimePlanLastStepMs)
{
    beaconNavigatorCommandStruct navCommand;
    beaconNavigatorOutputStruct navOutput;
    takeoffMotionGateInputStruct gateInput;
    takeoffMotionGateDecisionStruct gateDecision;
    core1SystemFactsSnapshot facts;
    uint8_t requiredLinksReady;
    uint8_t freshTargetAccepted;
    uint8_t continuityAdvanceAllowed;
    uint8_t searchOutputAllowed;
    uint8_t standbyDisplayAllowed;
    visionParamsStruct params;
    core0PoseSupervisorSnapshot poseSnapshot;
    beaconSourceObservationView sourceViews[BEACON_CONTINUITY_MAX_SOURCES];
    beaconTargetContinuityOutput continuityOutput;
    beaconTargetContinuityOutput displayContinuityOutput;
    core1NavigationDecisionInput navigationDecisionInput;
    core1NavigationDecisionKind navigationDecision;
    observerLinkPlanningTrackSnapshot core4Wire;
    observerLinkPlanningTrackSnapshot core3Wire;
    core3LinkCore1Status core3Status;
    core4LinkCore1Status core4Status;
    uint32_t core4Age = 0xFFFFFFFFu;
    uint32_t core3Age = 0xFFFFFFFFu;
    uint16_t core4Count = 0u;
    uint16_t core3Count = 0u;

```

```

imageGetObservationSnapshot(&runtimeContext.observation);
memset(&navCommand, 0, sizeof(navCommand));
memset(&navOutput, 0, sizeof(navOutput));
memset(&gateInput, 0, sizeof(gateInput));
memset(&gateDecision, 0, sizeof(gateDecision));
memset(sourceViews, 0, sizeof(sourceViews));
memset(&continuityOutput, 0, sizeof(continuityOutput));
memset(&displayContinuityOutput, 0, sizeof(displayContinuityOutput));
memset(&navigationDecisionInput, 0, sizeof(navigationDecisionInput));
memset(&core4Wire, 0, sizeof(core4Wire));
memset(&core3Wire, 0, sizeof(core3Wire));
navCommand.dtS = PLANNING_DT_S;
visionParamsGetActive(&params);
core0PoseSupervisorStep(runtimeTickMs,
    params.poseHoldMs,
    params.planningCore0PoseMinConfidence,
    params.planningCore0PoseHeadingSign,
    params.planningCore0PoseHeadingOffsetRad,
    &poseSnapshot);
navCommand.pose = &poseSnapshot;
runtimePlanForceStep = 0u;
runtimePlanLastStepMs = runtimeTickMs;
runtimeContext.rejoinBarrierActive = 0u;
core3LinkCore1GetStatus(&core3Status);
core4LinkCore1GetStatus(&core4Status);
core1KvmRouterGetSystemFacts(&facts);
core1ContinuityObserveIdentity(&core3Status, &core4Status,
    (uint8_t) (facts.core0StatusFlags & UART_LINK_C0_FLAG_LAUNCHED) != 0u);
if (core3LinkCore1GetPlanningTrackSnapshot(&core3Wire,
    runtimeTickMs, params.core3ObservationTimeoutMs, &core3Age) != 0u)
{
    runtimeContext.core3TrackerFrameSeq = core3Wire.trackerFrameSeq;
    core3Count = adaptObserverTracks(&core3Wire,
        OBSERVER_LINK_SOURCE_CORE3, runtimeCore3Tracks);
}
if (core4LinkCore1GetPlanningTrackSnapshot(&core4Wire,
    runtimeTickMs, params.c4ObservationTimeoutMs, &core4Age) != 0u)
{
    runtimeContext.core4TrackerFrameSeq = core4Wire.trackerFrameSeq;
    core4Count = adaptObserverTracks(&core4Wire,
        OBSERVER_LINK_SOURCE_CORE4, runtimeCore4Tracks);
}
sourceViews[0].sourceId = OBSERVER_LINK_SOURCE_CORE1;
sourceViews[0].fresh = (uint8_t) (poseSnapshot.headingValid &&
    sourceCalibrationValid(&params, REQUIRED_VISION_SOURCE_CORE1) &&
    (runtimeLocalObservationRevision == params.sharedConfigRevision) &&
    (runtimeLocalObservationHash == params.sharedConfigHash) &&
    (runtimeLocalObservationDetectMode == params.detectMode) &&
    ((uint32_t) (runtimeTickMs - runtimeLocalObservationTickMs) <=
        params.c1FreshGateMs));
sourceViews[0].session = core3Status.localSession;
sourceViews[0].observationSeq = runtimeContext.observation.frameSeq;
sourceViews[0].tracks = runtimeContext.observation.tracks;
sourceViews[0].count = runtimeContext.observation.count;
sourceViews[0].profile.centerX = poseSnapshot.x + params.c1Core0PoseOffsetX;
sourceViews[0].profile.centerY = poseSnapshot.y + params.c1Core0PoseOffsetY;
sourceViews[0].profile.scale = 1.0f;
sourceViews[0].profile.bodyHeadingRad = poseSnapshot.headingRad;
sourceViews[0].profile.geometryMode =
    BEACON_SOURCE_GEOMETRY_DOWNWARD_HEADING;
sourceViews[1].sourceId = OBSERVER_LINK_SOURCE_CORE3;
sourceViews[1].fresh = (uint8_t) (sourceCalibrationValid(&params, REQUIRED_VISION_SOURCE_CORE3) &&
    (core3Age <= params.c3FreshGateMs));
sourceViews[1].session = core3Status.peerSession;
sourceViews[1].observationSeq = core3Wire.trackerFrameSeq;
sourceViews[1].tracks = runtimeCore3Tracks;
sourceViews[1].count = core3Count;
sourceViews[1].profile.centerX = poseSnapshot.x + params.core3Core0PoseOffsetX;
sourceViews[1].profile.centerY = poseSnapshot.y + params.core3Core0PoseOffsetY;
sourceViews[1].profile.scale = params.core3Scale;
sourceViews[1].profile.mountYawRad = params.core3MountYawRad;
sourceViews[1].profile.mirrorX = params.core3MirrorX;
sourceViews[1].profile.errorFwdSign = params.core3ErrorFwdSign;
sourceViews[1].profile.errorRightSign = params.core3ErrorRightSign;
sourceViews[1].profile.geometryMode =
    BEACON_SOURCE_GEOMETRY_OBSERVER_FIXED;
sourceViews[2].sourceId = OBSERVER_LINK_SOURCE_CORE4;
sourceViews[2].fresh = (uint8_t) (sourceCalibrationValid(&params, REQUIRED_VISION_SOURCE_CORE4) &&
    (core4Age <= params.c4FreshGateMs));

```

```

sourceViews[2].session = core4Status.peerSession;
sourceViews[2].observationSeq = core4Wire.trackerFrameSeq;
sourceViews[2].tracks = runtimeCore4Tracks;
sourceViews[2].count = core4Count;
sourceViews[2].profile.centerX = poseSnapshot.x + params.core4Core0PoseOffsetX;
sourceViews[2].profile.centerY = poseSnapshot.y + params.core4Core0PoseOffsetY;
sourceViews[2].profile.scale = params.core4Scale;
sourceViews[2].profile.mountYawRad = params.core4MountYawRad;
sourceViews[2].profile.bodyHeadingRad = poseSnapshot.headingRad;
sourceViews[2].profile.mirrorX = params.core4MirrorX;
sourceViews[2].profile.errorFwdSign = params.core4ErrorFwdSign;
sourceViews[2].profile.errorRightSign = params.core4ErrorRightSign;
sourceViews[2].profile.geometryMode =
    BEACON_SOURCE_GEOMETRY_OBSERVER_HEADING;
requiredLinksReady = core1RuntimeRequiredLinksReady(&facts);
takeoffMotionGateApplyParams(params.planningPostTakeoffDelayMs);
gateInput.nowMs = runtimeTickMs;
gateInput.systemMode = facts.core0SystemMode;
gateInput.core0StatusValid = facts.core0StatusValid;
gateInput.launched = (uint8_t)((facts.core0StatusFlags &
    UART_LINK_C0_FLAG_LAUNCHED) != 0u);
gateInput.cruiseQualified = facts.core0AltReached;
gateInput.runtimePermitted = (uint8_t)(requiredLinksReady != 0u && runtimeContext.cameraReady != 0u &&
    runtimeContext.standalone == 0u &&
    runtimeContext.rejoinBarrierPending == 0u &&
    runtimeContext.rejoinBarrierActive == 0u);
takeoffMotionGatePrepare(&gateInput, &gateDecision);
searchOutputAllowed = (uint8_t)(gateDecision.motionUnlocked != 0u &&
    poseSnapshot.motionPermitted != 0u);
continuityAdvanceAllowed = (uint8_t)(searchOutputAllowed != 0u &&
    poseSnapshot.headingValid != 0u);
if (poseSnapshot.stage == CORE0_POSE_STAGE_WAIT_FIRST)
{
    core1ContinuityReset();
}
beaconTargetContinuityStep(&runtimeTargetContinuity, sourceViews,
    BEACON_CONTINUITY_MAX_SOURCES, runtimeTickMs,
    params.planningEdgePredictionHoldMs,
    params.planningSearchTimeoutMs, params.planningSearchEnable,
    continuityAdvanceAllowed, searchOutputAllowed, &continuityOutput);
if (searchOutputAllowed == 0u) continuityOutput.searchActive = 0u;
memset(&runtimeContext.displayTarget, 0,
    sizeof(runtimeContext.displayTarget));
standbyDisplayAllowed = (uint8_t)((gateInput.launched == 0u) &&
    (requiredLinksReady != 0u) &&
    (runtimeContext.cameraReady != 0u) &&
    (runtimeContext.standalone == 0u) &&
    (runtimeContext.rejoinBarrierPending == 0u) &&
    (runtimeContext.rejoinBarrierActive == 0u) &&
    (poseSnapshot.headingValid != 0u));
if (standbyDisplayAllowed != 0u)
{
    beaconTargetContinuityStep(&runtimeDisplayContinuity, sourceViews,
        BEACON_CONTINUITY_MAX_SOURCES, runtimeTickMs,
        params.planningEdgePredictionHoldMs,
        params.planningSearchTimeoutMs, params.planningSearchEnable,
        1u, 1u, &displayContinuityOutput);
    runtimeContext.displayTarget = displayContinuityOutput.candidate;
}
else
{
    beaconTargetContinuityInit(&runtimeDisplayContinuity);
    if (continuityOutput.candidate.valid != 0u)
        runtimeContext.displayTarget = continuityOutput.candidate;
}
navCommand.qualifiedCount = continuityOutput.qualifiedCount;
navCommand.searchActive = continuityOutput.searchActive;
navCommand.searchDirection = continuityOutput.searchDirection;
runtimeLocalObservationPending = 0u;

if (runtimeContext.rejoinBarrierPending != 0u)
{
    runtimeContext.rejoinBarrierActive = 1u;
}
navigationDecisionInput.barrierRequired = (uint8_t)((runtimeContext.rejoinBarrierPending != 0u) ||
    (gateDecision.suppressNavigator != 0u));
navigationDecisionInput.headingValid = poseSnapshot.headingValid;
navigationDecisionInput.lastTargetValid =
    runtimeTargetContinuity.last.valid;

```

```

navigationDecisionInput.startupForwardActive =
    gateDecision.startupForwardActive;
navigationDecisionInput.candidateValid =
    continuityOutput.candidate.valid;
navigationDecisionInput.searchActive = continuityOutput.searchActive;
navigationDecision = core1NavigationDecisionSelect(
    &navigationDecisionInput);
if (navigationDecision == CORE1_NAV_DECISION_BARRIER)
{
    navCommand.kind = BEACON_NAV_COMMAND_BARRIER;
}
else if (navigationDecision == CORE1_NAV_DECISION_STARTUP_FORWARD)
{
    navCommand.kind = BEACON_NAV_COMMAND_STARTUP_FORWARD;
}
else if (navigationDecision == CORE1_NAV_DECISION_CANDIDATE)
{
    navCommand.kind = BEACON_NAV_COMMAND_CANDIDATE;
    navCommand.candidate = &continuityOutput.candidate;
}
else
{
    navCommand.kind = BEACON_NAV_COMMAND_INVALID;
}
beaconNavigatorStep(&navCommand, &navOutput);
freshTargetAccepted = (uint8_t) (navOutput.snapshot.state == (uint8_t) BEACON_NAV_TRACK &&
    navOutput.snapshot.targetTrackId != 0u &&
    navOutput.snapshot.targetFresh != 0u &&
    navOutput.snapshot.targetValid != 0u);
takeoffMotionGateFinalize(freshTargetAccepted, &gateDecision);
runtimePlanTx = navOutput.plan;
runtimePlanTx.planTimeoutMs = params.planningPlanTimeoutMs;
runtimePlanTx.motionUnlocked = gateDecision.motionUnlocked;
runtimePlanTx.startupPhase = gateDecision.phase;
runtimePlanTx.takeoffMotionEpoch = gateDecision.epoch;
runtimePlanTx.dispatchEnable = (uint8_t) ((runtimePlanTx.valid != 0u) ? params.planningDispatchEnable : 0u);
if (runtimeContext.standalone != 0u)
{
    runtimePlanTx.valid = 0u;
    runtimePlanTx.vx = 0.0f;
    runtimePlanTx.vy = 0.0f;
    runtimePlanTx.wz = 0.0f;
    runtimePlanTx.dispatchEnable = 0u;
    runtimePlanTx.motionUnlocked = 0u;
}
if (runtimeContext.cameraReady == 0u)
{
    runtimePlanTx.valid = 0u;
    runtimePlanTx.vx = 0.0f;
    runtimePlanTx.vy = 0.0f;
    runtimePlanTx.wz = 0.0f;
    runtimePlanTx.dispatchEnable = 0u;
    runtimePlanTx.motionUnlocked = 0u;
}
if (runtimeContext.rejoinBarrierPending != 0u)
{
    runtimePlanTx.valid = 0u;
    runtimePlanTx.vx = 0.0f;
    runtimePlanTx.vy = 0.0f;
    runtimePlanTx.wz = 0.0f;
    runtimePlanTx.dispatchEnable = 0u;
    runtimePlanTx.motionUnlocked = 0u;
    runtimeContext.rejoinBarrierPending = 0u;
}
{
    requiredVisionSnapshot visionFault = core1RequiredVisionFault();
    if ((visionFault.active == 0u) &&
        (poseSnapshot.stage == CORE0_POSE_STAGE_FAULT))
    {
        visionFault.active = 1u;
        visionFault.source = REQUIRED_VISION_SOURCE_CORE0;
        visionFault.faultClass = REQUIRED_VISION_FAULT_POSE;
    }
    runtimePlanTx.requiredVisionFault = visionFault.active;
    runtimePlanTx.faultSource = visionFault.source;
    runtimePlanTx.faultClass = visionFault.faultClass;
    runtimePlanTx.poseStage = poseSnapshot.stage;
    if (poseSnapshot.motionPermitted == 0u)
    {

```



```

        runtimePlanTx.valid = 0u;
        runtimePlanTx.dispatchEnable = 0u;
        runtimePlanTx.vx = 0.0f;
        runtimePlanTx.vy = 0.0f;
        runtimePlanTx.wz = 0.0f;
    }
    if (visionFault.active != 0u)
    {
        runtimePlanTx.valid = 0u;
        runtimePlanTx.dispatchEnable = 0u;
        runtimePlanTx.vx = 0.0f;
        runtimePlanTx.vy = 0.0f;
        runtimePlanTx.wz = 0.0f;
    }
}
runtimePlanTx.beaconSearching = continuityOutput.searchActive;
runtimePlanTx.beaconSearchTimedOut = runtimeTargetContinuity.searchTimedOut;
runtimePlanTx.seq = ++runtimePlanSeq;
coreDataWritePlan(&runtimePlanTx);
publishCore3ControlStatus(&navOutput.snapshot);
publishObserverViewOverlay(OBSERVER_LINK_SOURCE_CORE3, &navOutput.snapshot);
publishObserverViewOverlay(OBSERVER_LINK_SOURCE_CORE4, &navOutput.snapshot);
if (runtimeContext.rejoinBarrierActive != 0u)
{
    runtimeContext.rejoinBarrierActive = 0u;
    runtimeContext.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_HOME;
}
}

/* 参数 owner 只读取已提交 runtime，不参与停车或发车状态判定。 */
paramSyncOwnerServicePoll();

/* 重视觉链之后再排空一次 RX，覆盖最多两个 10ms epoch 的规划突发。 */
core3LinkCore1DrainRx(runtimeTickMs);
core4LinkCore1DrainRx(runtimeTickMs);
/* Core3 回程 ACK 在渲染前闭合，缩短 Core0 单事件在途窗口。 */
core1KvmRouterFinalize(runtimeTickMs);
core3LinkCore1FlushControl(runtimeTickMs);
core4LinkCore1FlushControl(runtimeTickMs);

/* 菜单: 链路门控 + 按键状态机 + 渲染(发车页信标图 / 阈值双图 / 参数列表)。
 * HUD 事实 20ms，HOME 整幅灰图+标注 40ms，内部读 image.c 最新快照与确认信标。 */
menuTick();
core3LinkCore1FlushTx(runtimeTickMs);
core4LinkCore1FlushTx(runtimeTickMs);
}

void core1RuntimeGetPlanSnapshot(core1RuntimePlanSnapshotStruct *out)
{
    if (out == NULL) return;
    out->seq = runtimePlanTx.seq;
    out->valid = runtimePlanTx.valid;
    out->dispatchEnable = runtimePlanTx.dispatchEnable;
    out->barrier = (uint8_t) ((runtimeContext.rejoinBarrierPending != 0u) ||
        (runtimeContext.rejoinBarrierActive != 0u));
    out->standalone = runtimeContext.standalone;
    out->systemPhase = runtimeContext.systemPhase;
}

void core1RuntimeGetDisplayTargetSnapshot(
    core1RuntimeDisplayTargetSnapshotStruct *out)
{
    if (out == NULL) return;
    out->valid = runtimeContext.displayTarget.valid;
    out->fresh = runtimeContext.displayTarget.fresh;
    out->holding = runtimeContext.displayTarget.holding;
    out->sourceId = runtimeContext.displayTarget.sourceId;
    out->trackId = runtimeContext.displayTarget.trackId;
    out->size = runtimeContext.displayTarget.size;
    out->controlX = runtimeContext.displayTarget.fresh ? runtimeContext.displayTarget.centerX : runtimeContext.displayTarget.predictedX;
    out->controlY = runtimeContext.displayTarget.fresh ? runtimeContext.displayTarget.centerY : runtimeContext.displayTarget.predictedY;
}

uint32_t core1RuntimeGetTickMs(void)
{
    return runtimeTickMs;
}

```

core1_cyt4bb7/project/code/app/core1Loop.h

SHA-256

:

7316ff2b2f6a2c195b209429efc3d644ed09f05e99aed4560cd5d2c133ce4e10

代码 (c)

```
#ifndef _CORE1LOOP_H_
#define _CORE1LOOP_H_

#include <stdint.h>

typedef struct
{
    uint32_t seq;
    uint8_t valid;
    uint8_t dispatchEnable;
    uint8_t barrier;
    uint8_t standalone;
    uint8_t systemPhase;
} core1RuntimePlanSnapshotStruct;

typedef struct
{
    uint8_t valid;
    uint8_t fresh;
    uint8_t holding;
    uint8_t sourceId;
    uint16_t trackId;
    uint16_t size;
    uint16_t controlX;
    uint16_t controlY;
} core1RuntimeDisplayTargetSnapshotStruct;

void processInit(void);
void pitProcess0(void);
void whileProcess(void);
void core1RuntimeGetPlanSnapshot(core1RuntimePlanSnapshotStruct *out);
void core1RuntimeGetDisplayTargetSnapshot(
    core1RuntimeDisplayTargetSnapshotStruct *out);
uint32_t core1RuntimeGetTickMs(void);

#endif
```

core1_cyt4bb7/project/code/app/core1MotionTxFence.c

SHA-256

:

f8cf78e9f158a3425f7482240de36f32f78642e198add48e58c8de1722826b1c

代码 (c)

```
#include "core1MotionTxFence.h"

#include <string.h>

void core1MotionTxFenceReset(core1MotionTxFenceState *state)
{
    if (state != 0) memset(state, 0, sizeof(*state));
}

uint8_t core1MotionTxFencePrepare(core1MotionTxFenceState *state,
    uint32_t nowMs, float vx, float vy, float wz, uint8_t flags,
    core1MotionTxFenceFrame *frame)
{
    uint8_t nonzero;
    if ((state == 0) || (frame == 0)) return 0u;
    nonzero = (uint8_t) ((vx != 0.0f) || (vy != 0.0f) ||
        (wz != 0.0f) || (flags != 0u));
    if ((nonzero != 0u) &&
        ((state->primed == 0u) ||
```

```

        ((uint32_t) (nowMs - state->lastTxMs) >= CORE1_MOTION_TX_REARM_MS)))
    {
        frame->vx = 0.0f;
        frame->vy = 0.0f;
        frame->wz = 0.0f;
        frame->flags = 0u;
        state->primed = 1u;
        state->lastTxMs = nowMs;
        return 0u;
    }
    frame->vx = vx;
    frame->vy = vy;
    frame->wz = wz;
    frame->flags = flags;
    state->primed = 1u;
    state->lastTxMs = nowMs;
    return 1u;
}

```

core1_cyt4bb7/project/code/app/core1MotionTxFence.h

SHA-256

:

bb176c47085c90b9b0838bc305e7eb6e8d818ede102d1fd9bd11eb41cd152640

代码 (c)

```

#ifndef CORE1_MOTION_TX_FENCE_H
#define CORE1_MOTION_TX_FENCE_H

#include <stdint.h>

#define CORE1_MOTION_TX_REARM_MS (300u)

typedef struct
{
    uint32_t lastTxMs;
    uint8_t primed;
} core1MotionTxFenceState;

typedef struct
{
    float vx;
    float vy;
    float wz;
    uint8_t flags;
} core1MotionTxFenceFrame;

void core1MotionTxFenceReset(core1MotionTxFenceState *state);
uint8_t core1MotionTxFencePrepare(core1MotionTxFenceState *state,
    uint32_t nowMs, float vx, float vy, float wz, uint8_t flags,
    core1MotionTxFenceFrame *frame);

#endif /* CORE1_MOTION_TX_FENCE_H */

```

core1_cyt4bb7/project/code/app/core1NavigationDecision.c

SHA-256

:

fac88220d7749c1a8cac4bd9ebe6923b886d4b8c75d0a13f827cc7d4b314536e

代码 (c)

```

#include "core1NavigationDecision.h"

core1NavigationDecisionKind core1NavigationDecisionSelect(
    const core1NavigationDecisionInput *input)
{
    if (input == 0) return CORE1_NAV_DECISION_BARRIER;
    if (input->barrierRequired != 0u) return CORE1_NAV_DECISION_BARRIER;

    /* heading 保持窗耗尽后，TRACK 和首灯前直行都不能盖过已经成立的

```

```

* 纯 wz SEARCH; 该规则同样适用于尚未建立历史 owner 的冷启动。 */
if ((input->headingValid == 0u) && (input->searchActive != 0u))
    return CORE1_NAV_DECISION_CANDIDATE;
if ((input->headingValid == 0u) && (input->lastTargetValid != 0u))
    return CORE1_NAV_DECISION_BARRIER;
if ((input->startupForwardActive != 0u) &&
    (input->candidateValid == 0u))
    return CORE1_NAV_DECISION_STARTUP_FORWARD;
if (input->searchActive != 0u) return CORE1_NAV_DECISION_CANDIDATE;
if ((input->candidateValid != 0u) && (input->headingValid == 0u))
    return CORE1_NAV_DECISION_BARRIER;
if (input->candidateValid != 0u) return CORE1_NAV_DECISION_CANDIDATE;
if (input->startupForwardActive != 0u)
    return CORE1_NAV_DECISION_STARTUP_FORWARD;
return CORE1_NAV_DECISION_INVALID;
}

```

core1_cyt4bb7/project/code/app/core1NavigationDecision.h

SHA-256

:

5ad7472e51102fa4c906e539de7cad5504ce338cb1f1f2f26c5db723a7ac7b3a

代码 (c)

```

#ifndef CORE1_NAVIGATION_DECISION_H
#define CORE1_NAVIGATION_DECISION_H

#include <stdint.h>

typedef enum
{
    CORE1_NAV_DECISION_BARRIER = 0u,
    CORE1_NAV_DECISION_STARTUP_FORWARD,
    CORE1_NAV_DECISION_CANDIDATE,
    CORE1_NAV_DECISION_INVALID
} core1NavigationDecisionKind;

typedef struct
{
    uint8_t barrierRequired;
    uint8_t headingValid;
    uint8_t lastTargetValid;
    uint8_t startupForwardActive;
    uint8_t candidateValid;
    uint8_t searchActive;
} core1NavigationDecisionInput;

core1NavigationDecisionKind core1NavigationDecisionSelect(
    const core1NavigationDecisionInput *input);

#endif /* CORE1_NAVIGATION_DECISION_H */

```

core1_cyt4bb7/project/code/app/core1TakeoffEpochIdentity.c

SHA-256

:

00e75dd4a587cbf6e1d808d022757c7ad6c377816790b7647e83a9a0a33be341

代码 (c)

```

#include "core1TakeoffEpochIdentity.h"

#include <string.h>

static void captureBaseline(core1TakeoffEpochIdentityState *state)
{
    state->baselineProducerGeneration = state->observedProducerGeneration;
    state->baselineEpoch = state->observedEpoch;
}

void core1TakeoffEpochIdentityReset(core1TakeoffEpochIdentityState *state)

```

```

{
    if (state != 0) memset(state, 0, sizeof(*state));
}

uint8_t core1TakeoffEpochIdentityStep(core1TakeoffEpochIdentityState *state,
    const core1TakeoffEpochIdentityInput *input)
{
    if ((state == 0) || (input == 0)) return 0u;
    if ((input->planIdentityValid != 0u) &&
        (input->planProducerGeneration != 0u))
    {
        state->observedProducerGeneration = input->planProducerGeneration;
        state->observedEpoch = input->planEpoch;
    }

    if (input->core0StatusUsable == 0u) return 0u;
    if ((input->mode2Active == 0u) || (input->launched == 0u))
    {
        state->episodeActive = 0u;
        captureBaseline(state);
        return 0u;
    }
    if (input->altReached == 0u)
    {
        /* 飞行中到顶事实短暂撤销时保留已建立 episode；真正进入 episode 前
         * 则持续吸收当前 producer+epoch 作为旧身份基线。 */
        if (state->episodeActive == 0u) captureBaseline(state);
        return 0u;
    }
    if (state->episodeActive == 0u) state->episodeActive = 1u;
    if ((state->observedProducerGeneration == 0u) ||
        (state->observedEpoch == 0u)) return 0u;
    return (uint8_t) (
        (state->observedProducerGeneration !=
            state->baselineProducerGeneration) ||
        (state->observedEpoch != state->baselineEpoch));
}

```

core1_cyt4bb7/project/code/app/core1TakeoffEpochIdentity.h

SHA-256

:

c86f0ec9faa60aa3e156cce986532546ec178880bd399f475ac7c4e3e396d1ef

代码 (c)

```

#ifndef CORE1_TAKEOFF_EPOCH_IDENTITY_H
#define CORE1_TAKEOFF_EPOCH_IDENTITY_H

#include <stdint.h>

typedef struct
{
    uint32_t observedProducerGeneration;
    uint32_t observedEpoch;
    uint32_t baselineProducerGeneration;
    uint32_t baselineEpoch;
    uint8_t episodeActive;
} core1TakeoffEpochIdentityState;

typedef struct
{
    uint32_t planProducerGeneration;
    uint32_t planEpoch;
    uint8_t planIdentityValid;
    uint8_t core0StatusUsable;
    uint8_t mode2Active;
    uint8_t launched;
    uint8_t altReached;
} core1TakeoffEpochIdentityInput;

void core1TakeoffEpochIdentityReset(core1TakeoffEpochIdentityState *state);
uint8_t core1TakeoffEpochIdentityStep(core1TakeoffEpochIdentityState *state,
    const core1TakeoffEpochIdentityInput *input);

#endif /* CORE1_TAKEOFF_EPOCH_IDENTITY_H */

```

core1_cyt4bb7/project/code/app/menu.c

SHA-256

:

6e51227409e077e9b89820d5cb24f851a869a8083ea9ec6f253f552c05da0855

代码 (c)

```
#include "menu.h"
#include "menuInternal.h"
#include "../comm/paramSyncOwnerService.h"
#include "../comm/paramSyncGlobal.h"

#include "menuKeyRemote.h"
#include "core1KvmRouter.h"
#include "../vision/visionParams.h"
#include "zf_device_ips114.h"
#include "zf_common_font.h"

#include <stdio.h>

/*
 * core1 cm_7_1 菜单调度核心: 共享状态唯一定义 + 公共 API + 状态机分发 + 链路握手门控。
 * 移植自 core0 menu.c, 去掉 arming/飞控, 改为*等 core0 链路在线*门控(MENU_WAIT_LINK)。
 */

#define LCD_REFRESH_DIV (2u) /* 10ms tick → 20ms 遥测; 按键 dirty 仍同拍刷新 */
#define HOME_VISUAL_REFRESH_DIV (4u) /* 10ms tick → 40ms 整幅灰图+标注, 独立于按键 dirty */
#define MENU_INITIAL_LINK_WAIT_TICKS (200u) /* 10ms × 200 = 2s */
#define MENU_STORAGE_NONE (0u)
#define MENU_STORAGE_SAVE (1u)
#define MENU_STORAGE_RESTORE (2u)

static volatile uint16_t initialLinkWaitTicks;

/* =====
 * 共享状态唯一定义点
 * ===== */
/* BinaryCC 阈值 + 相机曝光 (menuModel applyVisionLocal 钳位后写回 visionParams, 热重载)。 */
menuContextStruct menuContext;

static void discardAllMenuKeys(void)
{
    uint8_t keyIndex;
    for (keyIndex = 0u; keyIndex < (uint8_t) MENU_KEY_COUNT; keyIndex++)
    {
        menuKeyClear((menuKeyIndexEnum) keyIndex);
    }
}

static uint8_t isValidKvmTarget(uint8_t target)
{
    return (uint8_t) ((target == CORE1_KVM_TARGET_CORE0) ||
        (target == CORE1_KVM_TARGET_CORE1) ||
        (target == CORE1_KVM_TARGET_CORE3) ||
        (target == CORE1_KVM_TARGET_CORE4));
}

void menuObserveKvmDesiredTarget(uint8_t snapshotValid, uint8_t desiredTarget)
{
    if (snapshotValid == 0u) return;
    if (isValidKvmTarget(desiredTarget) == 0u) return;

    if ((kvmDesiredTargetPrimed != 0u) &&
        (lastValidKvmDesiredTarget == CORE1_KVM_TARGET_CORE1) &&
        (desiredTarget != CORE1_KVM_TARGET_CORE1))
    {
        menuCurrentState = MENU_HOME;
        pageIndex = 0;
        optIdx = 0;
        scrollTop = 0u;
        threshSel = THRESH_SEL_CANNY_LOW;
        feedbackHoldTicks = 0u;
        feedbackWasActive = 0u;
        menuPendingFeedback = 0u;
    }
}
```

```

        discardAllMenuKeys();
        clearScreenForState();
    }
    lastValidKvmDesiredTarget = desiredTarget;
    kvmDesiredTargetPrimed = 1u;
}

uint16_t menuBorderColor(void)
{
    core1KvmRouterSnapshot kvm;
    core1KvmRouterGetSnapshot(&kvm);
    if ((kvm.valid == 0u) || (kvm.inputOwner == CORE1_KVM_TARGET_NONE))
        return RGB565_YELLOW;
    return (kvm.inputOwner == CORE1_KVM_TARGET_CORE1) ? RGB565_GREEN : RGB565_RED;
}

void menuInit(void)
{
    menuCurrentState = MENU_WAIT_LINK;
    pageIndex = 0;
    optIdx = 0;
    scrollTop = 0u;
    forceRedraw = 1u;
    threshSel = THRESH_SEL_CANNY_LOW;
    controlOwned = 0u;
    feedbackHoldTicks = 0u;
    feedbackWasActive = 0u;
    lcdRefreshFlag = 0u;
    homeVisualRefreshFlag = 0u;
    lcdRefreshCount = 0u;
    homeVisualRefreshCount = 0u;
    waitLinkRendered = 0u;
    kvmTickSeq = 0u;
    initialLinkWaitTicks = 0u;
    initFailedMask = 0u;
    initUnverifiedMask = 0u;
    initSummaryTicks = 0u;
    initSummaryLastSecond = 0xFFFFu;
    menuInputServiced = 0u;
    menuPendingScreenClear = 0u;
    menuPendingFeedback = 0u;
    menuPendingFeedbackColor = RGB565_BLACK;
    menuPendingStorageAction = MENU_STORAGE_NONE;
    mutationLockLast = paramSyncOwnerServiceMutationLocked();
    menuParamGlobalReset();
    kvmDesiredTargetPrimed = 0u;
    lastValidKvmDesiredTarget = CORE1_KVM_TARGET_NONE;

    menuSyncFromParams(); /* 工作副本 ← 当前 visionParams(已由 visionParamsLoad 从 flash 载入) */
    menuContext.paramsRevision = visionParamsGetRevision();
    menuBuildPages();

    /* IPS114 8×16 横屏 180, 黑底绿字 */
    ips114_set_dir(IPS114_PORTAIT_180);
    ips114_set_font(IPS114_8X16_FONT);
    ips114_init();
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
    ips114_clear();
    menuRenderResetListSnapshot();

    menuKeyInit(10u);
}

void menuPitTick(void)
{
    menuKeyScan();

    if (initSummaryTicks > 0u) initSummaryTicks--;

    if (initialLinkWaitTicks < MENU_INITIAL_LINK_WAIT_TICKS)
        initialLinkWaitTicks++;

    if (feedbackHoldTicks > 0u) feedbackHoldTicks--;

    lcdRefreshCount++;
    if (lcdRefreshCount >= LCD_REFRESH_DIV)
    {
        lcdRefreshCount = 0u;
        lcdRefreshFlag = 1u;
    }
}

```

```

    }

    homeVisualRefreshCount++;
    if (homeVisualRefreshCount >= HOME_VISUAL_REFRESH_DIV)
    {
        homeVisualRefreshCount = 0u;
        homeVisualRefreshFlag = 1u;
    }
}

void menuRequestScreenClear(void)
{
    menuPendingScreenClear = 1u;
    forceRedraw = 1u;
}

void menuRequestFeedback(uint16_t color)
{
    menuPendingFeedbackColor = color;
    menuPendingFeedbackAction = PARAM_SYNC_GLOBAL_ACTION_NONE;
    menuPendingFeedbackSlot = 0u;
    menuPendingFeedback = 1u;
}

void menuRequestParamFeedback(uint16_t color, uint8_t action, uint8_t slot)
{
    menuPendingFeedbackColor = color;
    menuPendingFeedbackAction = action;
    menuPendingFeedbackSlot = slot;
    menuPendingFeedback = 1u;
}

void menuRequestSaveSettings(void)
{
    if (paramSyncOwnerServiceMutationLocked() != 0u) return;
    menuPendingStorageAction = MENU_STORAGE_SAVE;
}

void menuRequestRestoreDefaults(void)
{
    if (paramSyncOwnerServiceMutationLocked() != 0u) return;
    menuPendingStorageAction = MENU_STORAGE_RESTORE;
}

static void applyPendingTailEffects(void)
{
    if (paramSyncOwnerServiceMutationLocked() != 0u)
    {
        if (menuPendingStorageAction != MENU_STORAGE_NONE &&
            menuPendingFeedbackAction == PARAM_SYNC_GLOBAL_ACTION_NONE)
            menuPendingFeedback = 0u;
        menuPendingStorageAction = MENU_STORAGE_NONE;
        return;
    }
    if (menuPendingStorageAction == MENU_STORAGE_SAVE)
    {
        menuPendingStorageAction = MENU_STORAGE_NONE;
        menuSaveSettings();
    }
    else if (menuPendingStorageAction == MENU_STORAGE_RESTORE)
    {
        menuPendingStorageAction = MENU_STORAGE_NONE;
        menuRestoreDefaults();
    }

    if (menuPendingFeedback != 0u)
    {
        menuPendingFeedback = 0u;
        menuPendingScreenClear = 0u;
        ips114_full(menuPendingFeedbackColor);
        if (menuPendingFeedbackAction == PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY ||
            menuPendingFeedbackAction == PARAM_SYNC_GLOBAL_ACTION_LOAD_RUNTIME)
        {
            char slotText[16];
            ips114_set_color((menuPendingFeedbackColor == RGB565_GREEN) ?
                RGB565_BLACK : RGB565_WHITE, menuPendingFeedbackColor);
            ips114_show_string(80u, 42u,
                (menuPendingFeedbackAction ==
                    PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY) ?

```



```

        "SAVE OK" : "LOAD OK");
        (void) snprintf(slotText, sizeof(slotText), "SLOT %u",
            (unsigned) menuPendingFeedbackSlot);
        ips114_show_string(88u, 66u, slotText);
        ips114_set_color(RGB565_GREEN, RGB565_BLACK);
    }
    menuPendingFeedbackAction = PARAM_SYNC_GLOBAL_ACTION_NONE;
    menuPendingFeedbackSlot = 0u;
    menuRenderResetListSnapshot();
    feedbackHoldTicks = MENU_FEEDBACK_HOLD_TICKS;
    return;
}
if (menuPendingScreenClear != 0u)
{
    menuPendingScreenClear = 0u;
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
    ips114_clear();
    menuRenderResetListSnapshot();
    forceRedraw = 1u;
}
}

void menuServiceInput(void)
{
    core1KvmRouterSnapshot kvm;
    uint8_t linkUp;

    if (menuInputServiced != 0u) return;
    menuInputServiced = 1u;
    kvmTickSeq++;
    /* 链路在线 = 可显示菜单(门控); 操控权(controlActive) = core0 远程模式正操控 core1
    * (决定边框 绿/红)。core0 本地模式时 linkUp=1 但 controlActive=0: 菜单可见但边框红。 */
    linkUp = menuKeyRemoteLinkUp();
    controlOwned = menuKeyRemoteControlActive();
    core1KvmRouterGetSnapshot(&kvm);
    menuObserveKvmDesiredTarget(kvm.valid, kvm.desiredTarget);

    if (initSummaryTicks > 0u)
    {
        discardAllMenuKeys();
        return;
    }
    if (initSummaryLastSecond != 0xFFFFu)
    {
        initSummaryLastSecond = 0xFFFFu;
        clearScreenForState();
    }

    /* === 上电链路握手门控: 等 core0 链路在线才进 HOME === */
    if (menuCurrentState == MENU_WAIT_LINK)
    {
        if (linkUp || (initialLinkWaitTicks >= MENU_INITIAL_LINK_WAIT_TICKS))
        {
            menuCurrentState = MENU_HOME;
            waitLinkRendered = 0u;
            clearScreenForState();
        }
        else
        {
            discardAllMenuKeys();
            return;
        }
    }

    /* 初始窗口结束后保持 HOME 连续观测; 断链/重连由标题、链路槽和安全 owner 表达,
    * 禁止切整页 WAIT 造成图像消失和两次清屏黑闪。 */

    /* === 填充反馈保持期: 吞按键, 倒计时结束清屏重绘 === */
    if (feedbackHoldTicks > 0u)
    {
        feedbackWasActive = 1u;
        discardAllMenuKeys();
        return;
    }
    if (feedbackWasActive)
    {
        feedbackWasActive = 0u;
        clearScreenForState();
    }
}

```

```

/* === 按键 === */
switch (menuCurrentState)
{
    case MENU_HOME: handleHomeKeys(); break;
    case MENU_PAGE_LIST: handlePageListKeys(); break;
    case MENU_OPTION_LIST: handleOptionListKeys(); break;
    case MENU_EDIT: handleEditKeys(); break;
    case MENU_THRESH_EDIT: handleThreshEditKeys(); break;
    default: break;
}

}

void menuTick(void)
{
    uint8_t redrawRequested;
    uint8_t visualDue;
    uint8_t mutationLocked;
    uint32_t paramsRevision;

    menuParamGlobalPoll();
    if (menuPendingFeedback != 0u)
    {
        discardAllMenuKeys();
        menuInputServiced = 1u;
    }
    mutationLocked = paramSyncOwnerServiceMutationLocked();
    if (mutationLocked != mutationLockLast)
    {
        discardAllMenuKeys();
        if (menuPendingStorageAction != MENU_STORAGE_NONE &&
            menuPendingFeedbackAction == PARAM_SYNC_GLOBAL_ACTION_NONE)
            menuPendingFeedback = 0u;
        menuPendingStorageAction = MENU_STORAGE_NONE;
        mutationLockLast = mutationLocked;
    }
    if (mutationLocked == 0u)
    {
        paramsRevision = visionParamsGetRevision();
        if (paramsRevision != menuContext.paramsRevision)
        {
            menuSyncFromParams();
            menuContext.paramsRevision = paramsRevision;
            forceRedraw = 1u;
        }
    }
    if (menuInputServiced == 0u) menuServiceInput();
    menuInputServiced = 0u;
    applyPendingTailEffects();

    if (initSummaryTicks > 0u)
    {
        uint16_t second = (uint16_t) ((initSummaryTicks + 99u) / 100u);
        if ((second != initSummaryLastSecond) || forceRedraw)
        {
            forceRedraw = 0u;
            renderInitFaultSummary(initFailedMask, initUnverifiedMask, (uint8_t) second);
            initSummaryLastSecond = second;
        }
        return;
    }
    if (menuCurrentState == MENU_WAIT_LINK)
    {
        if (forceRedraw || !waitLinkRendered)
        {
            forceRedraw = 0u;
            renderWaitLink();
            waitLinkRendered = 1u;
        }
        return;
    }
    if ((feedbackHoldTicks > 0u) || (menuPendingFeedback != 0u)) return;

    /* === 渲染(仅刷新节拍或强制重绘) === */
    if (!lcdRefreshFlag && !forceRedraw) return;
    lcdRefreshFlag = 0u;
    redrawRequested = forceRedraw;
    forceRedraw = 0u;

```

```

visualDue = (uint8_t)((menuCurrentState == MENU_HOME) && redrawRequested);
if (((menuCurrentState == MENU_HOME) ||
    (menuCurrentState == MENU_THRESH_EDIT)) &&
    (homeVisualRefreshFlag != 0u))
{
    homeVisualRefreshFlag = 0u;
    visualDue = 1u;
}

switch (menuCurrentState)
{
    case MENU_HOME: renderHome(visualDue); break;
    case MENU_PAGE_LIST: renderPageList(); break;
    case MENU_OPTION_LIST: renderOptionList(0u); break;
    case MENU_EDIT: renderOptionList(1u); break;
    case MENU_THRESH_EDIT: renderThreshEdit(visualDue); break;
    default: break;
}

}

void menuRenderInputFeedback(void)
{
    if ((forceRedraw == 0u) ||
        (menuPendingScreenClear != 0u) ||
        (menuPendingFeedback != 0u) ||
        (menuPendingStorageAction != MENU_STORAGE_NONE) ||
        (feedbackHoldTicks != 0u) ||
        (initSummaryTicks != 0u) ||
        (menuCurrentState == MENU_WAIT_LINK) ||
        (menuCurrentState == MENU_HOME))
    {
        return;
    }

    /* 只绘制当前 surface 的 dirty 槽；页面切换清屏、反馈和 Flash 仍由尾段 owner 处理。 */
    forceRedraw = 0u;
    switch (menuCurrentState)
    {
        case MENU_PAGE_LIST: renderPageList(); break;
        case MENU_OPTION_LIST: renderOptionList(0u); break;
        case MENU_EDIT: renderOptionList(1u); break;
        case MENU_THRESH_EDIT: renderThreshEdit(0u); break;
        default: break;
    }
}

void menuSetInitHealth(uint8_t failedMaskValue, uint8_t unverifiedMaskValue)
{
    {
        initFailedMask = failedMaskValue;
        initUnverifiedMask = unverifiedMaskValue;
        if ((failedMaskValue | unverifiedMaskValue) != 0u)
        {
            initSummaryTicks = 300u;
            initSummaryLastSecond = 0xFFFFu;
            forceRedraw = 1u;
        }
    }
}

void menuGetStatus(menuStatusSnapshotStruct *out)
{
    {
        uint8_t holdTicks;
        if (out == 0) return;
        holdTicks = feedbackHoldTicks;
        out->tickSeq = kvmTickSeq;
        out->state = (uint8_t) menuCurrentState;
        out->homeReady = (uint8_t)((menuCurrentState == MENU_HOME) && (holdTicks == 0u));
        out->inputReady = (uint8_t)((menuCurrentState != MENU_WAIT_LINK) && (holdTicks == 0u));
        out->valid = 1u;
    }
}

```

core1_cyt4bb7/project/code/app/menu.h

SHA-256

:

95479e3c231fd1dd976c51b8e3abe91c7c8820657dba08e173cd63c0ad2ad232

代码 (c)

```
#ifndef _MENU_H_
#define _MENU_H_

#include <stdint.h>

/*
 * core1 cm_7_1 菜单 + 屏幕显示(独占 IPS114; 按键来自核间串口透传, core1 无物理按键)。
 *
 * 远程 KVM 模型: core0 发车页长按切换“按键操控目标”到 core1 后, core0 按键事件经核间
 * 串口透传到 core1, 驱动本菜单。core1 在自己的屏显示发车页/阈值编辑器/参数页, 交互/UI
 * 对齐 core0。UI 分割线/边框颜色表示操控权: 被 core0 操控=绿, 未操控=红。
 *
 * 职责拆分(共享状态/类型/原型见 menuInternal.h):
 * menu.c      共享状态定义 + 公共 API + 状态机分发 + 链路握手门控
 * menuModel.c 视觉/规划参数工作副本/默认/热重载到 visionParams + 页表 + flash
 * menuRender.c 8×16 渲染(发车页全屏信标图 + 阈值双图 + 列表) + 绿/红边框
 * menuKeyRemote 远程按键源(接口同 core0 menuKey)
 *
 * core1Loop.c 调度:
 * processInit  → imageInit + visionParamsLoad + menuInit
 * pitProcess0  → menuPitTick(PIT_CH2 10ms ISR: menuKeyScan + 刷新计数)
 * whileProcess → KVM Ingress + menuServiceInput + imageProcess/planning + menuTick + KVM Finalize
 *
 * 上电先 MENU_WAIT_LINK 等 core0 链路在线(menuKeyRemoteLinkUp), 确认后进 MENU_HOME。
 *
 * 按键映射(对齐 core0):
 * 发车页: K1 进菜单 / K3 进阈值编辑器 / K4 短:保存 长:恢复默认 / (K2/K5 预留)
 * 页列表: K1 进选项 / K2 下 / K3 上 / K4 回发车页 / K5 保存回发车页
 * 选项列表: K1 进编辑 / K2 下 / K3 上 / K4 回页列表 / K5 保存回发车页
 * 编辑:    K1 +大 / K2 +小 / K3 -小 / K4 -大 / K5 回选项列表
 * 阈值编辑器: K1 +大 / K2 +小 / K3 -小 / K4 -大 / K5 短:切项(Lo/Hi/Exp) 长:保存回发车页
 */

void menuInit(void); /* ips114+ 按键源初始化, 同步 visionParams 工作副本, 建页表 */
void menuSetInitHealth(uint8_t failedMask, uint8_t unverifiedMask);
void menuPitTick(void); /* PIT_CH2 10ms ISR: menuKeyScan + 刷新计数(轻量) */
void menuServiceInput(void); /* 前前台段: 链路门控 + 按键状态机, 不访问 IPS114/Flash */
void menuRenderInputFeedback(void); /* 同页选择/数值变化的轻量即时反馈, 不处理 Flash/清屏 */
void menuTick(void); /* 前后台尾段: Flash/反馈/渲染 */
typedef struct
{
    uint32_t tickSeq;
    uint8_t state;
    uint8_t homeReady;
    uint8_t inputReady;
    uint8_t valid;
} menuStatusSnapshotStruct;

void menuGetStatus(menuStatusSnapshotStruct *out);

#endif /* _MENU_H_ */
```

core1_cyt4bb7/project/code/app/menuFloatDisplayInternal.h

SHA-256

:

e5c06fecc348bcaa914fc246d7a0df4c1c07169fdbf396c61be6a168d0ec125f

代码 (c)

```
#ifndef MENU_FLOAT_DISPLAY_INTERNAL_H
#define MENU_FLOAT_DISPLAY_INTERNAL_H

#include <stdint.h>

#include "zf_device_ips114.h"

#define MENU_FLOAT_DISPLAY_DECIMALS_MAX (8u)
#define MENU_FLOAT_DISPLAY_CHARS      (15u)

static uint8_t menuFloatDisplayDigitsU64(uint64_t value)
{
    uint8_t digits = 1u;
```

```

    while (value >= 10u)
    {
        value /= 10u;
        digits++;
    }
    return digits;
}

static void menuFloatDisplayShowLiteral(uint16_t x,
    uint16_t y,
    const char *literal)
{
    char buf[MENU_FLOAT_DISPLAY_CHARS + 1u];
    uint8_t i;
    uint8_t length = 0u;

    while ((literal[length] != '\0') && (length < MENU_FLOAT_DISPLAY_CHARS))
    {
        length++;
    }
    for (i = 0u; i < MENU_FLOAT_DISPLAY_CHARS; i++)
    {
        buf[i] = ' ';
    }
    for (i = 0u; i < length; i++)
    {
        buf[MENU_FLOAT_DISPLAY_CHARS - length + i] = literal[i];
    }
    buf[MENU_FLOAT_DISPLAY_CHARS] = '\0';
    ips114_show_string(x, y, buf);
}

static uint8_t menuFloatDisplayDecimalsForStep(float step)
{
    uint8_t decimals = 0u;
    double scaled = (step < 0.0f) ? -(double) step : (double) step;

    if (!(scaled > 0.0))
    {
        return MENU_FLOAT_DISPLAY_DECIMALS_MAX;
    }
    while (decimals < MENU_FLOAT_DISPLAY_DECIMALS_MAX)
    {
        uint64_t rounded;
        double difference;

        if (!(scaled < (double) UINT64_MAX) ||
            !(scaled + 0.5) < (double) UINT64_MAX)
        {
            return MENU_FLOAT_DISPLAY_DECIMALS_MAX;
        }
        rounded = (uint64_t) (scaled + 0.5);
        difference = scaled - (double) rounded;

        if (difference < 0.0)
        {
            difference = -difference;
        }
        if (difference <= 0.000001)
        {
            break;
        }
        scaled *= 10.0;
        decimals++;
    }
    return decimals;
}

static uint8_t menuFloatDisplayDecimalsForValue(double value)
{
    uint8_t decimals;
    double scale = 1.0;
    double tolerance = 0.0000001;

    if (value < 0.0)
    {
        value = -value;
    }

```

```

if (!(value >= 0.0) || !(value < (double) UINT64_MAX))
{
    return MENU_FLOAT_DISPLAY_DECIMALS_MAX;
}
for (decimals = 0u; decimals < MENU_FLOAT_DISPLAY_DECIMALS_MAX; decimals++)
{
    double scaled = value * scale;
    uint64_t rounded;
    double difference;

    if (!(scaled < (double) UINT64_MAX) ||
        !((scaled + 0.5) < (double) UINT64_MAX))
    {
        return MENU_FLOAT_DISPLAY_DECIMALS_MAX;
    }
    rounded = (uint64_t) (scaled + 0.5);
    difference = value - ((double) rounded / scale);

    if (difference < 0.0)
    {
        difference = -difference;
    }
    if (difference <= tolerance)
    {
        return decimals;
    }
    scale *= 10.0;
}
return MENU_FLOAT_DISPLAY_DECIMALS_MAX;
}

static uint8_t menuFloatDisplayDecimalsForValueAndStep(double value, float step)
{
    uint8_t stepDecimals = menuFloatDisplayDecimalsForStep(step);
    uint8_t valueDecimals = menuFloatDisplayDecimalsForValue(value);

    return (stepDecimals > valueDecimals) ? stepDecimals : valueDecimals;
}

static void menuFloatDisplayShow(uint16_t x,
    uint16_t y,
    double value,
    uint8_t requestedDecimals)
{
    static const uint64_t scaleTable[MENU_FLOAT_DISPLAY_DECIMALS_MAX + 1u] =
    {
        1u, 10u, 100u, 1000u, 10 000u,
        100 000u, 1 000 000u, 10 000 000u, 100 000 000u
    };
    char buf[MENU_FLOAT_DISPLAY_CHARS + 1u];
    uint8_t decimals;
    uint8_t selectedDecimals = 0xFFu;
    uint8_t negative = 0u;
    uint8_t pos;
    uint8_t i;
    uint64_t scale = 1u;
    uint64_t scaled = 0u;
    uint64_t intPart;
    uint64_t fracPart;

    if (value != value)
    {
        menuFloatDisplayShowLiteral(x, y, "NAN");
        return;
    }
    if (value < 0.0)
    {
        negative = 1u;
        value = -value;
    }

    decimals = (requestedDecimals <= MENU_FLOAT_DISPLAY_DECIMALS_MAX)
        ? requestedDecimals
        : MENU_FLOAT_DISPLAY_DECIMALS_MAX;
    for (; decimals-->)
    {
        double scaledDouble;
        double roundedDouble;
        uint8_t required;

```

```

scale = scaleTable[decimals];
scaledDouble = value * (double) scale;
roundedDouble = scaledDouble + 0.5;
if ((scaledDouble < (double) UINT64_MAX) &&
    (roundedDouble < (double) UINT64_MAX))
{
    scaled = (uint64_t) roundedDouble;
    intPart = scaled / scale;
    required = (uint8_t) (negative + menuFloatDisplayDigitsU64(intPart));
    if (decimals != 0u)
    {
        required = (uint8_t) (required + 1u + decimals);
    }
    if (required <= MENU_FLOAT_DISPLAY_CHARS)
    {
        selectedDecimals = decimals;
        break;
    }
}
if (decimals == 0u)
{
    break;
}
}
if (selectedDecimals == 0xFFu)
{
    menuFloatDisplayShowLiteral(x, y, "OVERFLOW");
    return;
}

for (i = 0u; i < MENU_FLOAT_DISPLAY_CHARS; i++)
{
    buf[i] = ' ';
}
buf[MENU_FLOAT_DISPLAY_CHARS] = '\0';
intPart = scaled / scale;
fracPart = scaled % scale;
pos = MENU_FLOAT_DISPLAY_CHARS;
for (i = 0u; i < selectedDecimals; i++)
{
    buf[--pos] = (char) ('0' + (fracPart % 10u));
    fracPart /= 10u;
}
if (selectedDecimals != 0u)
{
    buf[--pos] = '.';
}
do
{
    buf[--pos] = (char) ('0' + (intPart % 10u));
    intPart /= 10u;
} while ((intPart != 0u) && (pos > 0u));
if ((negative != 0u) && (pos > 0u))
{
    buf[--pos] = '-';
}

ips114_show_string(x, y, buf);
}

#endif

```

core1_cyt4bb7/project/code/app/menuInput.c

SHA-256

:

9f16b9a503f4fa17be69e56c7921e664103353aa4e1502d5dcbcf7d108983b18

代码 (c)

```

#include "menu.h"
#include "menuInternal.h"

#include <string.h>

```

```

#include "menuKeyRemote.h"
#include "zf_common_font.h"

#include "configBeaconDetect.h"
#include "core1Loop.h"
#include "coreDataTransfer.h"
#include "../comm/paramSyncOwnerService.h"
#include "visionParams.h"

/*
 * core1 menuInput.c - 按键状态机 handler(按 menuState 分发) + 清屏/填充反馈/滚动维护。
 * 移植自 core0, 编辑即时生效调 applyVisionLocal()。按键经 menuKeyRemote(核间透传)。
 */

static void moveSelectionAndScroll(int8_t* index, uint8_t count, int8_t delta)
{
    if (count == 0u)
    {
        *index = 0;
        scrollTop = 0u;
        return;
    }
    *index = (int8_t) (*index + delta);
    if (*index < 0) *index = (int8_t) (count - 1u);
    else if (*index >= (int8_t) count) *index = 0;

    if ((uint8_t) *index < scrollTop) scrollTop = (uint8_t) *index;
    else if ((uint8_t) *index >= scrollTop + VISIBLE_ROWS)
        scrollTop = (uint8_t) ((uint8_t) *index - VISIBLE_ROWS + 1u);
}

static void pageListMove(int8_t delta);

static uint32_t paramGlobalRequestSequence;
static uint32_t pendingParamGlobalCoordinatorBoot;
static paramSyncGlobalRequest pendingParamGlobalRequest;
static menuParamGlobalUiSnapshot paramGlobalUi;
static uint8_t pendingParamGlobalRequestValid;
static paramSyncGlobalState latestParamGlobalState;
static uint32_t latestParamGlobalStateLastAdvanceMs;
static uint8_t latestParamGlobalStatePrimed;
static uint8_t latestParamGlobalStateValid;
static uint8_t pendingParamSlotInspect;
static uint8_t requestGlobalParamAction(uint8_t action, uint8_t slot);

static void scheduleParamSlotInspect(void)
{
    pendingParamSlotInspect = 1u;
    paramGlobalUi.slotUiState = PARAM_SYNC_SLOT_UI_CHECKING;
    forceRedraw = 1u;
}

#define MENU_PARAM_GLOBAL_STATE_FRESH_MS (300u)

static uint8_t tryReadFreshParamGlobalState(paramSyncGlobalState *state)
{
    paramSyncGlobalState sampled;
    uint32_t nowMs = core1RuntimeGetTickMs();
    if (state == 0) return 0u;
    if (coreDataTryReadParamGlobalState(&sampled) != 0u)
    {
        if (latestParamGlobalStatePrimed == 0u ||
            sampled.coordinatorBootGeneration !=
                latestParamGlobalState.coordinatorBootGeneration)
        {
            latestParamGlobalState = sampled;
            latestParamGlobalStateLastAdvanceMs = nowMs;
            latestParamGlobalStatePrimed = 1u;
            latestParamGlobalStateValid = 0u;
        }
        else if ((int32_t) (sampled.sequence -
            latestParamGlobalState.sequence) > 0)
        {
            latestParamGlobalState = sampled;
            latestParamGlobalStateLastAdvanceMs = nowMs;
            latestParamGlobalStateValid = 1u;
        }
    }
    if (latestParamGlobalStatePrimed == 0u ||

```



```

    latestParamGlobalStateValid == 0u ||
    (uint32_t) (nowMs - latestParamGlobalStateLastAdvanceMs) >=
        MENU_PARAM_GLOBAL_STATE_FRESH_MS)
{
    latestParamGlobalStateValid = 0u;
    return 0u;
}
*state = latestParamGlobalState;
return 1u;
}

void menuParamGlobalReset(void)
{
    memset(&pendingParamGlobalRequest, 0, sizeof(pendingParamGlobalRequest));
    memset(&paramGlobalUi, 0, sizeof(paramGlobalUi));
    paramGlobalUi.target = 0xFFu;
    pendingParamGlobalRequestValid = 0u;
    pendingParamGlobalCoordinatorBoot = 0u;
    memset(&latestParamGlobalState, 0, sizeof(latestParamGlobalState));
    latestParamGlobalStateLastAdvanceMs = 0u;
    latestParamGlobalStatePrimed = 0u;
    latestParamGlobalStateValid = 0u;
    pendingParamSlotInspect = 0u;
    paramGlobalUi.slotUiState = PARAM_SYNC_SLOT_UI_UNKNOWN;
}

void menuParamGlobalGetUi(menuParamGlobalUiSnapshot *snapshot)
{
    if (snapshot != 0) *snapshot = paramGlobalUi;
}

void menuParamGlobalPoll(void)
{
    paramSyncGlobalState state;
    uint8_t nextUi;
    uint32_t nextDiagnostic = 0u;
    if (pendingParamGlobalRequestValid == 0u)
    {
        if (pendingParamSlotInspect != 0u &&
            tryReadFreshParamGlobalState(&state) != 0u &&
            paramSyncGlobalStateReady(&state) != 0u &&
            requestGlobalParamAction(PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY,
                (uint8_t) paramSlotSelection) != 0u)
            pendingParamSlotInspect = 0u;
        return;
    }
    if (tryReadFreshParamGlobalState(&state) == 0u)
    {
        memset(&state, 0, sizeof(state));
        nextUi = MENU_PARAM_GLOBAL_FAILED;
        state.phase = PARAM_SYNC_GLOBAL_PHASE_FAILED;
        state.target = 1u;
        state.error = (latestParamGlobalStatePrimed != 0u &&
            latestParamGlobalState.coordinatorBootGeneration !=
                pendingParamGlobalCoordinatorBoot) ?
            PARAM_SYNC_GLOBAL_ERROR_PEER_RESTART :
            PARAM_SYNC_GLOBAL_ERROR_ROUTE;
        pendingParamGlobalRequestValid = 0u;
    }
    else if (state.coordinatorBootGeneration != pendingParamGlobalCoordinatorBoot)
    {
        nextUi = MENU_PARAM_GLOBAL_FAILED;
        state.phase = PARAM_SYNC_GLOBAL_PHASE_FAILED;
        state.target = 1u;
        state.error = PARAM_SYNC_GLOBAL_ERROR_PEER_RESTART;
        pendingParamGlobalRequestValid = 0u;
    }
    else if (paramSyncGlobalStateMatchesRequest(&state,
        &pendingParamGlobalRequest) != 0u &&
        (state.flags & PARAM_SYNC_GLOBAL_FLAG_TERMINAL) != 0u)
    {
        uint8_t completedAction = pendingParamGlobalRequest.action;
        uint8_t completedSlot = pendingParamGlobalRequest.slot;
        nextUi = ((state.flags & PARAM_SYNC_GLOBAL_FLAG_OK) != 0u) ?
            MENU_PARAM_GLOBAL_SUCCESS : MENU_PARAM_GLOBAL_FAILED;
        nextDiagnostic = (nextUi == MENU_PARAM_GLOBAL_FAILED) ?
            state.transactionId : 0u;
        paramGlobalUi.action = completedAction;
        paramGlobalUi.slot = completedSlot;
    }
}

```

```

    if (completedAction == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
    {
        if (completedSlot == (uint8_t) paramSlotSelection)
            paramGlobalUi.slotUiState = (nextUi == MENU_PARAM_GLOBAL_SUCCESS) ?
                paramSyncGlobalClassifySlotSummary(state.transactionId) :
                PARAM_SYNC_SLOT_UI_UNAVAILABLE;
    }
    else if (nextUi == MENU_PARAM_GLOBAL_SUCCESS)
    {
        menuRequestParamFeedback((completedAction ==
            PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY) ? RGB565_GREEN :
            RGB565_RED, completedAction, completedSlot);
        pendingParamSlotInspect = 1u;
    }
    pendingParamGlobalRequestValid = 0u;
}
else if (paramSyncGlobalStateMatchesRequest(&state,
    &pendingParamGlobalRequest) != 0u &&
    (state.flags & PARAM_SYNC_GLOBAL_FLAG_BUSY) != 0u)
    nextUi = MENU_PARAM_GLOBAL_PROGRESS;
else if (paramSyncGlobalStateReady(&state) == 0u)
{
    nextUi = MENU_PARAM_GLOBAL_FAILED;
    state.phase = PARAM_SYNC_GLOBAL_PHASE_FAILED;
    state.target = 1u;
    state.error = ((state.flags & PARAM_SYNC_GLOBAL_FLAG_RECOVERY) != 0u) ?
        PARAM_SYNC_GLOBAL_ERROR_RECOVERY :
        (((state.flags & PARAM_SYNC_GLOBAL_FLAG_BUSY) != 0u) ?
            PARAM_SYNC_GLOBAL_ERROR_BUSY : PARAM_SYNC_GLOBAL_ERROR_ROUTE);
    pendingParamGlobalRequestValid = 0u;
}
else
    return;
if (paramGlobalUi.state != nextUi || paramGlobalUi.phase != state.phase ||
    paramGlobalUi.target != state.target || paramGlobalUi.error != state.error ||
    paramGlobalUi.diagnostic != nextDiagnostic)
{
    paramGlobalUi.state = nextUi;
    paramGlobalUi.phase = state.phase;
    paramGlobalUi.target = state.target;
    paramGlobalUi.error = state.error;
    paramGlobalUi.diagnostic = nextDiagnostic;
    forceRedraw = 1u;
}
}

static uint8_t requestGlobalParamAction(uint8_t action, uint8_t slot)
{
    paramSyncGlobalState state;
    paramSyncGlobalRequest request;
    if (pendingParamGlobalRequestValid != 0u ||
        paramSyncOwnerServiceMutationLocked() != 0u ||
        tryReadFreshParamGlobalState(&state) == 0u ||
        paramSyncGlobalStateReady(&state) == 0u) return 0u;
    paramGlobalRequestSequence++;
    if (paramGlobalRequestSequence == 0u) paramGlobalRequestSequence = 1u;
    memset(&request, 0, sizeof(request));
    request.producerBootGeneration = coreDataGetPlanBootGeneration();
    request.sequence = paramGlobalRequestSequence;
    request.origin = 1u;
    request.action = action;
    request.slot = slot;
    if (paramSyncGlobalRequestIsValid(&request) == 0u) return 0u;
    coreDataWriteParamGlobalRequest(&request);
    pendingParamGlobalRequest = request;
    pendingParamGlobalCoordinatorBoot = state.coordinatorBootGeneration;
    pendingParamGlobalRequestValid = 1u;
    paramGlobalUi.state = MENU_PARAM_GLOBAL_QUEUED;
    paramGlobalUi.phase = PARAM_SYNC_GLOBAL_PHASE_REQUESTED;
    paramGlobalUi.target = 0xFFu;
    paramGlobalUi.error = PARAM_SYNC_GLOBAL_ERROR_NONE;
    paramGlobalUi.diagnostic = 0u;
    paramGlobalUi.action = action;
    paramGlobalUi.slot = slot;
    if (action == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
        paramGlobalUi.slotUiState = PARAM_SYNC_SLOT_UI_CHECKING;
    forceRedraw = 1u;
    return 1u;
}

```

```

void clearScreenForState(void)
{
    menuRequestScreenClear();
}

static void enterPageList(uint8_t resetSelection)
{
    menuCurrentState = MENU_PAGE_LIST;
    if (resetSelection)
    {
        pageIndex = 0;
        scrollTop = 0u;
    }
    else
    {
        scrollTop = 0u;
        pageListMove(0);
    }
    clearScreenForState();
}

static void enterOptionList(void)
{
    if (pageIndex < 0 || (uint8_t) pageIndex >= menuPageCount
        || menuPages[pageIndex].optionCount == 0u)
    {
        forceRedraw = 1u;
        return;
    }
    menuCurrentState = MENU_OPTION_LIST;
    optIdx = 0;
    scrollTop = 0u;
    if ((uint8_t) pageIndex == paramSlotsPageIndex)
        scheduleParamSlotInspect();
    clearScreenForState();
}

/* 填充反馈请求由循环尾执行，保持 MENU_FEEDBACK_HOLD_TICKS×10ms 后清屏重绘。 */
static void menuFlashFeedback(uint16_t color)
{
    menuRequestFeedback(color);
}

static void pageListMove(int8_t delta)
{
    if (menuPageCount == 0u)
    {
        moveSelectionAndScroll(&pageIndex, menuPageCount, delta);
        return;
    }
    moveSelectionAndScroll(&pageIndex, menuPageCount, delta);
    forceRedraw = 1u;
}

/* 发车页: K1 进菜单 / K3 进阈值编辑器 / K4 短:保存 长:恢复默认。(K2/K5 预留) */
void handleHomeKeys(void)
{
    if (paramSyncOwnerServiceMutationLocked() != 0u) return;

    if (menuKeyGetState(MENU_KEY_1) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_1);
        enterPageList(1u);
    }
    else if (menuKeyGetState(MENU_KEY_3) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_3);
        menuCurrentState = MENU_THRESH_EDIT;
        threshSel = THRESH_SEL_CANNY_LOW;
        clearScreenForState();
    }
    else if (menuKeyGetState(MENU_KEY_4) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_4);
        menuRequestSaveSettings();
        menuFlashFeedback(RGB565_GREEN); /* 保存确认绿屏 */
    }
    else if (menuKeyGetState(MENU_KEY_4) == MENU_KEY_LONG_PRESS)

```

```

    {
        menuKeyClear(MENU_KEY_4);
        menuRequestRestoreDefaults();
        menuFlashFeedback(RGB565_RED); /* 恢复默认红屏 */
    }
}

void handlePageListKeys(void)
{
    if (menuKeyGetState(MENU_KEY_1) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_1);
        enterOptionList();
    }
    else if (menuKeyGetState(MENU_KEY_2) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_2);
        pageListMove(+1);
    }
    else if (menuKeyGetState(MENU_KEY_3) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_3);
        pageListMove(-1);
    }
    else if (menuKeyGetState(MENU_KEY_4) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_4);
        menuCurrentState = MENU_HOME;
        clearScreenForState();
    }
    else if (menuKeyGetState(MENU_KEY_5) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_5);
        if (paramSyncOwnerServiceMutationLocked() == 0u)
        {
            menuRequestSaveSettings();
            menuCurrentState = MENU_HOME;
            menuFlashFeedback(RGB565_GREEN);
        }
    }
}

void handleOptionListKeys(void)
{
    menuPageStruct* pg;

    if (pageIndex < 0 || (uint8_t) pageIndex >= menuPageCount)
    {
        enterPageList(0u);
        return;
    }
    pg = &menuPages[pageIndex];

    if ((uint8_t) pageIndex == paramSlotsPageIndex)
    {
        uint8_t slot = (uint8_t) paramSlotSelection;
        uint8_t ok = 0u;
        if (menuKeyGetState(MENU_KEY_4) == MENU_KEY_SHORT_PRESS)
        {
            menuKeyClear(MENU_KEY_4);
            if (paramGlobalUi.slotUiState == PARAM_SYNC_SLOT_UI_FULL)
            {
                ok = requestGlobalParamAction(
                    PARAM_SYNC_GLOBAL_ACTION_LOAD_RUNTIME, slot);
            }
            (void) ok;
        }
    }
    else if (menuKeyGetState(MENU_KEY_2) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_2);
        paramSlotSelection = (float) ((slot < 5u) ? slot + 1u : 1u);
        scheduleParamSlotInspect();
    }
    else if (menuKeyGetState(MENU_KEY_3) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_3);
        paramSlotSelection = (float) ((slot > 1u) ? slot - 1u : 5u);
        scheduleParamSlotInspect();
    }
}

```

```

else if (menuKeyGetState(MENU_KEY_5) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_5);
    enterPageList(0u);
}
else if (menuKeyGetState(MENU_KEY_1) == MENU_KEY_LONG_PRESS)
{
    menuKeyClear(MENU_KEY_1);
    ok = requestGlobalParamAction(
        PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY, slot);
    (void) ok;
}
return;
}

if (menuKeyGetState(MENU_KEY_1) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_1);
    if (paramSyncOwnerServiceMutationLocked() == 0u &&
        pg->optionCount > 0u && optIdx >= 0 && optIdx < (int8_t) pg->optionCount
        && pg->opt[optIdx] != 0)
    {
        menuCurrentState = MENU_EDIT;
        forceRedraw = 1u;
    }
}
else if (menuKeyGetState(MENU_KEY_2) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_2);
    moveSelectionAndScroll(&optIdx, pg->optionCount, +1);
    forceRedraw = 1u;
}
else if (menuKeyGetState(MENU_KEY_3) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_3);
    moveSelectionAndScroll(&optIdx, pg->optionCount, -1);
    forceRedraw = 1u;
}
else if (menuKeyGetState(MENU_KEY_4) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_4);
    enterPageList(0u);
}
else if (menuKeyGetState(MENU_KEY_5) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_5);
    if (paramSyncOwnerServiceMutationLocked() == 0u)
    {
        menuRequestSaveSettings();
        menuCurrentState = MENU_HOME;
        menuFlashFeedback(RGB565_GREEN);
    }
}
}

void handleEditKeys(void)
{
    menuPageStruct* pg;
    float delta = 0.0f;
    uint8_t hasDelta = 0u;

    if (pageIdx < 0 || (uint8_t) pageIdx >= menuPageCount)
    {
        enterPageList(0u);
        return;
    }
    pg = &menuPages[pageIdx];

    if (paramSyncOwnerServiceMutationLocked() != 0u)
    {
        menuCurrentState = MENU_OPTION_LIST;
        clearScreenForState();
        return;
    }

    if (optIdx < 0 || optIdx >= (int8_t) pg->optionCount || pg->opt[optIdx] == 0)
    {
        menuCurrentState = MENU_OPTION_LIST;
        clearScreenForState();
    }
}

```

```

    return;
}

/* K1 +大 / K2 +小\ / K3 -小\ / K4 -大 / K5 回选项列表 */
if (menuKeyGetState(MENU_KEY_1) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_1);
    delta = pg->stepL[optIdx];
    hasDelta = 1u;
}
else if (menuKeyGetState(MENU_KEY_2) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_2);
    delta = pg->stepS[optIdx];
    hasDelta = 1u;
}
else if (menuKeyGetState(MENU_KEY_3) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_3);
    delta = -pg->stepS[optIdx];
    hasDelta = 1u;
}
else if (menuKeyGetState(MENU_KEY_4) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_4);
    delta = -pg->stepL[optIdx];
    hasDelta = 1u;
}
else if (menuKeyGetState(MENU_KEY_5) == MENU_KEY_SHORT_PRESS)
{
    menuKeyClear(MENU_KEY_5);
    menuCurrentState = MENU_OPTION_LIST;
    clearScreenForState();
    return;
}

if (hasDelta)
{
    *pg->opt[optIdx] += delta;
    applyVisionLocal(); /* 钳位 + 热重载, 下一帧即生效 */
    forceRedraw = 1u;
}
}

/* 阈值编辑器: K1 +大 / K2 +小\ / K3 -小\ / K4 -大(作用于当前 DetectMode 的 Lo/Hi/Exp);
 * K5 短:切项(Lo→Hi→Exp) 长:保存回发车页。 */
void handleThreshEditKeys(void)
{
    float stepL;
    float stepS;
    float delta = 0.0f;
    uint8_t hasDelta = 0u;

    if (paramSyncOwnerServiceMutationLocked() != 0u)
    {
        menuCurrentState = MENU_HOME;
        clearScreenForState();
        return;
    }

    if (menuKeyGetState(MENU_KEY_5) == MENU_KEY_LONG_PRESS)
    {
        menuKeyClear(MENU_KEY_5);
        menuRequestSaveSettings();
        menuCurrentState = MENU_HOME;
        menuFlashFeedback(RGB565_GREEN);
        return;
    }
    if (menuKeyGetState(MENU_KEY_5) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_5);
        threshSel = (threshSelEnum)((threshSel + 1u) % THRESH_SEL_COUNT); /* Lo → Hi → Exp */
        forceRedraw = 1u;
        return;
    }

    if (threshSel == THRESH_SEL_EXPOSURE)
    {
        stepL = VISION_EXPOSURE_STEP_LONG;

```

```

        stepS = VISION_EXPOSURE_STEP_SHORT;
    }
    else
    {
        stepL = VISION_CANNY_STEP_LONG;
        stepS = VISION_CANNY_STEP_SHORT;
    }

    if (menuKeyGetState(MENU_KEY_1) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_1);
        delta = stepL;
        hasDelta = 1u;
    }
    else if (menuKeyGetState(MENU_KEY_2) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_2);
        delta = stepS;
        hasDelta = 1u;
    }
    else if (menuKeyGetState(MENU_KEY_3) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_3);
        delta = -stepS;
        hasDelta = 1u;
    }
    else if (menuKeyGetState(MENU_KEY_4) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(MENU_KEY_4);
        delta = -stepL;
        hasDelta = 1u;
    }
}

if (hasDelta)
{
    if (threshSel == THRESH_SEL_CANNY_LOW)
    {
        if (detectModeF >= 0.5f) legacyCannyLowF += delta;
        else cannyLowF += delta;
    }
    else if (threshSel == THRESH_SEL_CANNY_HIGH)
    {
        if (detectModeF >= 0.5f) legacyCannyHighF += delta;
        else cannyHighF += delta;
    }
    else
    {
        exposureF += delta;
    }
    applyVisionLocal(); /* 钳位 + 热重载, 下一帧两图即更新 */
    forceRedraw = 1u;
}
}

```

core1_cyt4bb7/project/code/app/menuInternal.h

SHA-256 :
9a521a4e84aa5e8236f35418799a9b8174917c52cee78375d7232b653030b691

代码 (c)

```

#ifndef _MENU_INTERNAL_H_
#define _MENU_INTERNAL_H_

/*
 * core1 cm_7_1 菜单模块内部共享头（不对外，仅 menu*.c 引用）。
 * 移植自 core0 菜单框架, 精简为信标视觉参数(无飞控参数), 按键来自核间串口透传。
 * menu.c      状态唯一定义 + 公共 API + 状态机分发 + 链路握手门控
 * menuModel.c 视觉参数工作副本/默认/热重载到 visionParams + 页表 + flash(经 visionParams)
 * menuRender.c IPS114 8×16 渲染(发车页左图右字 + 阈值双图 + 列表) + 操控权绿/红边框
 * menuInput.c  按键 handler + 清屏/反馈/滚动维护
 */

#include <stdint.h>

```

```

/* =====
* 屏幕几何 (8×16 字体, IPS114 240×135 横向, 与 core0 一致)
* ===== */
#define LCD_W      (240u)
#define LCD_H      (135u)
#define FONT_W     (8u)
#define FONT_H     (16u)
#define ROW_Y(r)   ((uint16_t)((r) * FONT_H))
#define COL_X(c)   ((uint16_t)((c) * FONT_W))
#define VISIBLE_ROWS (7u)

#define SCROLLBAR_W (4u)
#define SCROLLBAR_X ((uint16_t)(LCD_W - 1u - SCROLLBAR_W))

/* 阈值编辑器双图布局: 标题 row0; 两图并排 y=18, 各 110×70(贴近 188:120≈1.57 比例);
* 左原图 x=2, 右 BinaryCC mask x=124; 底部参数行 row7。 */
#define THR_IMG_Y   (18u)
#define THR_IMG_W   (110u)
#define THR_IMG_H   (70u)
#define THR_IMG_L_X (2u)
#define THR_IMG_R_X (124u)
#define THR_TXT_Y   (112u)

/* 长按/保存填充反馈保持时长: ×10ms PIT = 300ms */
#define MENU_FEEDBACK_HOLD_TICKS (30u)

/* 核间链路断联保护开关(与 core0 boardConfig.h 同名同义): 1=断联(core0 链路超时)→回等待页+提示;
* 0=断联仅停止操控不强制回退(便于调试)。core1 据共享内存 linkUp 判 core0 在线。
* 注意: core0 boardConfig.h 有同名宏, 两核改值须同步(语义对称, 缺一端则保护不对等)。 */
#ifndef INTER_CORE_LINK_GUARD_ENABLE
#define INTER_CORE_LINK_GUARD_ENABLE (1)
#endif

/* =====
* 菜单数据结构 (与 core0 同构)
* ===== */
#define MENU_MAX_OPTIONS (16u)
#define MENU_MAX_PAGES   (20u)

typedef struct
{
    const char* name;
    uint8_t optionCount;
    const char* optName[MENU_MAX_OPTIONS];
    float* opt[MENU_MAX_OPTIONS];
    float stepS[MENU_MAX_OPTIONS];
    float stepL[MENU_MAX_OPTIONS];
    uint8_t decimals[MENU_MAX_OPTIONS]; /* 0=按当前值与细调步长自动推导 */
} menuPageStruct;

typedef enum
{
    MENU_WAIT_LINK = 0, /* 上电等核间通信确认(core0 链路在线)才进 HOME */
    MENU_HOME,
    MENU_PAGE_LIST,
    MENU_OPTION_LIST,
    MENU_EDIT,
    MENU_THRESH_EDIT
} menuStateEnum;

/* =====
* 跨文件共享状态 (唯一定义在 menu.c)
* ===== */
typedef struct
{
    float detectModeF, cannyLowF, cannyHighF, legacyCannyLowF, legacyCannyHighF, exposureF;
    float beaconSizeMinF, beaconSizeMaxF, beaconEccMaxF, beaconBrightMinF;
    float beaconMaxEndpointsF, beaconFillMinF, beaconHuMaxF, beaconHuAreaMinF;
    float trackAssocDistF, trackSizeRatioF, trackConfirmHitsF, trackDropMissesF, trackCoastMissesF;
    float trackPositionAlphaF, trackSizeAlphaF, trackAreaVelAlphaF;
    float trackAssocScoreMaxF, trackAssocAreaWeightF, trackAssocAreaVelWeightF;
    float planningEdgePredictionHoldMsF;
    float c1EdgeBandLeftF, c1EdgeBandRightF, c1EdgeBandTopF, c1EdgeBandBottomF;
    float observerEdgeBandLeftF, observerEdgeBandRightF, observerEdgeBandTopF, observerEdgeBandBottomF;
    float c1BorderEnterLeftF, c1BorderExitLeftF, c1BorderEnterRightF, c1BorderExitRightF;
    float c1BorderEnterTopF, c1BorderExitTopF, c1BorderEnterBottomF, c1BorderExitBottomF;
    float observerBorderEnterLeftF, observerBorderExitLeftF, observerBorderEnterRightF, observerBorderExitRightF;
    float observerBorderEnterTopF, observerBorderExitTopF, observerBorderEnterBottomF, observerBorderExitBottomF;
    float planningKpVxF, planningKpVyF;

```



```

float core3PlanningKpVxF, core3PlanningKpVyF;
float planningVxMaxF, planningVyMaxF, planningWzMaxF;
float planningVxSlewPerSF, planningVySlewPerSF, planningWzSlewPerSF;
float planningVxMinF, planningVyMinF, planningWzMinF;
float planningDeadzoneEnterF, planningDeadzoneExitF;
float planningCore0PoseMinConfidenceF;
float c1Core0PoseOffsetXF, c1Core0PoseOffsetYF;
float planningCore0PoseHeadingOffsetRadF, planningCore0PoseHeadingSignF, poseHoldMsF;
float planningVxSignF, planningVySignF, planningWzSignF;
float planningDispatchEnableF, planningSearchEnableF, planningPlanTimeoutMsF;
float planningSearchWzF, planningSearchTimeoutMsF;
float planningPostTakeoffDelayMsF, planningFirstBeaconForwardSpeedF;
float planningFirstBeaconForwardEnableF;
float planningTdRvxF, planningTdRvyF, planningTdRwzF, planningTdNF;
float planningTdVxEnableF, planningTdVyEnableF, planningTdWzEnableF;
float maxBeaconsF, core3ObservationTimeoutMsF;
float core3CenterXF, core3CenterYF, core3ScaleF, core3MountYawRadF, core3MirrorXF;
float core3ErrorFwdSignF, core3ErrorRightSignF;
float core3Core0PoseOffsetXF, core3Core0PoseOffsetYF;
float core3PlanningVxSignF, core3PlanningVySignF;
float c1FreshGateMsF, c3FreshGateMsF;
float c4FreshGateMsF, c4ObservationTimeoutMsF;
float c4EdgeBandLeftF, c4EdgeBandRightF, c4EdgeBandTopF, c4EdgeBandBottomF;
float c4BorderEnterLeftF, c4BorderExitLeftF, c4BorderEnterRightF, c4BorderExitRightF;
float c4BorderEnterTopF, c4BorderExitTopF, c4BorderEnterBottomF, c4BorderExitBottomF;
float core4PlanningKpVxF, core4PlanningKpVyF;
float core4CenterXF, core4CenterYF, core4ScaleF, core4MountYawRadF, core4MirrorXF;
float core4ErrorFwdSignF, core4ErrorRightSignF;
float core4Core0PoseOffsetXF, core4Core0PoseOffsetYF;
float core4PlanningVxSignF, core4PlanningVySignF;
} menuWorkingParamsStruct;
/* 阈值编辑器当前项；枚举值即原 0/1/2，COUNT 供循环切换取模。*/
typedef enum
{
    THRESH_SEL_CANNY_LOW = 0,
    THRESH_SEL_CANNY_HIGH = 1,
    THRESH_SEL_EXPOSURE = 2,
    THRESH_SEL_COUNT = 3
} threshSelEnum;

typedef enum
{
    MENU_PARAM_GLOBAL_IDLE = 0u,
    MENU_PARAM_GLOBAL_QUEUED,
    MENU_PARAM_GLOBAL_PROGRESS,
    MENU_PARAM_GLOBAL_SUCCESS,
    MENU_PARAM_GLOBAL_FAILED
} menuParamGlobalUiState;

typedef struct
{
    uint8_t state;
    uint8_t phase;
    uint8_t target;
    uint8_t error;
    uint8_t action;
    uint8_t slot;
    uint8_t slotUiState;
    uint8_t reserved;
    uint32_t diagnostic;
} menuParamGlobalUiSnapshot;
typedef struct
{
    menuWorkingParamsStruct working;
    menuStateEnum state;
    int8_t pageIdx;
    int8_t optIdx;
    uint8_t scrollTop;
    uint8_t forceRedraw;
    threshSelEnum threshSel;
    uint8_t controlOwned;
    volatile uint8_t feedbackHoldTicks;
    volatile uint8_t lcdRefreshFlag;
    volatile uint8_t homeVisualRefreshFlag;
    menuPageStruct pages[MENU_MAX_PAGES];
    uint32_t kvmTickSeq;
    uint8_t pageCount;
    uint8_t feedbackWasActive;
    uint8_t lcdRefreshCount;

```

```

uint8_t homeVisualRefreshCount;
uint8_t waitLinkRendered;
uint8_t initFailedMask;
uint8_t initUnverifiedMask;
volatile uint16_t initSummaryTicks;
uint16_t initSummaryLastSecond;
uint8_t inputServiced;
uint8_t pendingScreenClear;
uint8_t pendingFeedback;
uint16_t pendingFeedbackColor;
uint8_t pendingFeedbackAction;
uint8_t pendingFeedbackSlot;
uint8_t pendingStorageAction;
uint8_t mutationLockLast;
uint8_t kvmDesiredTargetPrimed;
uint8_t lastValidKvmDesiredTarget;
uint8_t paramSlotsPageIndex;
float paramSlotSelection;
uint32_t paramsRevision;
} menuContextStruct;

```

```

extern menuContextStruct menuContext;

```

```

#define detectModeF menuContext.working.detectModeF
#define cannyLowF menuContext.working.cannyLowF
#define cannyHighF menuContext.working.cannyHighF
#define legacyCannyLowF menuContext.working.legacyCannyLowF
#define legacyCannyHighF menuContext.working.legacyCannyHighF
#define exposureF menuContext.working.exposureF
#define beaconSizeMinF menuContext.working.beaconSizeMinF
#define beaconSizeMaxF menuContext.working.beaconSizeMaxF
#define beaconEccMaxF menuContext.working.beaconEccMaxF
#define beaconBrightMinF menuContext.working.beaconBrightMinF
#define beaconMaxEndpointsF menuContext.working.beaconMaxEndpointsF
#define beaconFillMinF menuContext.working.beaconFillMinF
#define beaconHuMaxF menuContext.working.beaconHuMaxF
#define beaconHuAreaMinF menuContext.working.beaconHuAreaMinF
#define trackAssocDistF menuContext.working.trackAssocDistF
#define trackSizeRatioF menuContext.working.trackSizeRatioF
#define trackConfirmHitsF menuContext.working.trackConfirmHitsF
#define trackDropMissesF menuContext.working.trackDropMissesF
#define trackCoastMissesF menuContext.working.trackCoastMissesF
#define trackPositionAlphaF menuContext.working.trackPositionAlphaF
#define trackSizeAlphaF menuContext.working.trackSizeAlphaF
#define trackAreaVelAlphaF menuContext.working.trackAreaVelAlphaF
#define trackAssocScoreMaxF menuContext.working.trackAssocScoreMaxF
#define trackAssocAreaWeightF menuContext.working.trackAssocAreaWeightF
#define trackAssocAreaVelWeightF menuContext.working.trackAssocAreaVelWeightF
#define planningEdgePredictionHoldMsF menuContext.working.planningEdgePredictionHoldMsF
#define c1EdgeBandLeftF menuContext.working.c1EdgeBandLeftF
#define c1EdgeBandRightF menuContext.working.c1EdgeBandRightF
#define c1EdgeBandTopF menuContext.working.c1EdgeBandTopF
#define c1EdgeBandBottomF menuContext.working.c1EdgeBandBottomF
#define observerEdgeBandLeftF menuContext.working.observerEdgeBandLeftF
#define observerEdgeBandRightF menuContext.working.observerEdgeBandRightF
#define observerEdgeBandTopF menuContext.working.observerEdgeBandTopF
#define observerEdgeBandBottomF menuContext.working.observerEdgeBandBottomF
#define c1BorderEnterLeftF menuContext.working.c1BorderEnterLeftF
#define c1BorderExitLeftF menuContext.working.c1BorderExitLeftF
#define c1BorderEnterRightF menuContext.working.c1BorderEnterRightF
#define c1BorderExitRightF menuContext.working.c1BorderExitRightF
#define c1BorderEnterTopF menuContext.working.c1BorderEnterTopF
#define c1BorderExitTopF menuContext.working.c1BorderExitTopF
#define c1BorderEnterBottomF menuContext.working.c1BorderEnterBottomF
#define c1BorderExitBottomF menuContext.working.c1BorderExitBottomF
#define observerBorderEnterLeftF menuContext.working.observerBorderEnterLeftF
#define observerBorderExitLeftF menuContext.working.observerBorderExitLeftF
#define observerBorderEnterRightF menuContext.working.observerBorderEnterRightF
#define observerBorderExitRightF menuContext.working.observerBorderExitRightF
#define observerBorderEnterTopF menuContext.working.observerBorderEnterTopF
#define observerBorderExitTopF menuContext.working.observerBorderExitTopF
#define observerBorderEnterBottomF menuContext.working.observerBorderEnterBottomF
#define observerBorderExitBottomF menuContext.working.observerBorderExitBottomF
#define planningKpVxF menuContext.working.planningKpVxF
#define planningKpVyF menuContext.working.planningKpVyF
#define core3PlanningKpVxF menuContext.working.core3PlanningKpVxF
#define core3PlanningKpVyF menuContext.working.core3PlanningKpVyF
#define planningVxMaxF menuContext.working.planningVxMaxF
#define planningVyMaxF menuContext.working.planningVyMaxF

```

```

#define planningWzMaxF menuContext.working.planningWzMaxF
#define planningVxSlewPerSF menuContext.working.planningVxSlewPerSF
#define planningVySlewPerSF menuContext.working.planningVySlewPerSF
#define planningWzSlewPerSF menuContext.working.planningWzSlewPerSF
#define planningVxMinF menuContext.working.planningVxMinF
#define planningVyMinF menuContext.working.planningVyMinF
#define planningWzMinF menuContext.working.planningWzMinF
#define planningDeadzoneEnterF menuContext.working.planningDeadzoneEnterF
#define planningDeadzoneExitF menuContext.working.planningDeadzoneExitF
#define planningCore0PoseMinConfidenceF menuContext.working.planningCore0PoseMinConfidenceF
#define c1Core0PoseOffsetXF menuContext.working.c1Core0PoseOffsetXF
#define c1Core0PoseOffsetYF menuContext.working.c1Core0PoseOffsetYF
#define planningCore0PoseHeadingOffsetRadF menuContext.working.planningCore0PoseHeadingOffsetRadF
#define planningCore0PoseHeadingSignF menuContext.working.planningCore0PoseHeadingSignF
#define poseHoldMsF menuContext.working.poseHoldMsF
#define planningVxSignF menuContext.working.planningVxSignF
#define planningVySignF menuContext.working.planningVySignF
#define planningWzSignF menuContext.working.planningWzSignF
#define planningDispatchEnableF menuContext.working.planningDispatchEnableF
#define planningSearchEnableF menuContext.working.planningSearchEnableF
#define planningPlanTimeoutMsF menuContext.working.planningPlanTimeoutMsF
#define planningSearchWzF menuContext.working.planningSearchWzF
#define planningSearchTimeoutMsF menuContext.working.planningSearchTimeoutMsF
#define planningPostTakeoffDelayMsF menuContext.working.planningPostTakeoffDelayMsF
#define planningFirstBeaconForwardSpeedF menuContext.working.planningFirstBeaconForwardSpeedF
#define planningFirstBeaconForwardEnableF menuContext.working.planningFirstBeaconForwardEnableF
#define planningTdRvxvF menuContext.working.planningTdRvxvF
#define planningTdRvyvF menuContext.working.planningTdRvyvF
#define planningTdRwzF menuContext.working.planningTdRwzF
#define planningTdNF menuContext.working.planningTdNF
#define planningTdVxEnableF menuContext.working.planningTdVxEnableF
#define planningTdVyEnableF menuContext.working.planningTdVyEnableF
#define planningTdWzEnableF menuContext.working.planningTdWzEnableF
#define maxBeaconsF menuContext.working.maxBeaconsF
#define core3ObservationTimeoutMsF menuContext.working.core3ObservationTimeoutMsF
#define core3CenterXF menuContext.working.core3CenterXF
#define core3CenterYF menuContext.working.core3CenterYF
#define core3ScaleF menuContext.working.core3ScaleF
#define core3MountYawRadF menuContext.working.core3MountYawRadF
#define core3MirrorXF menuContext.working.core3MirrorXF
#define core3ErrorFwdSignF menuContext.working.core3ErrorFwdSignF
#define core3ErrorRightSignF menuContext.working.core3ErrorRightSignF
#define core3Core0PoseOffsetXF menuContext.working.core3Core0PoseOffsetXF
#define core3Core0PoseOffsetYF menuContext.working.core3Core0PoseOffsetYF
#define core3PlanningVxSignF menuContext.working.core3PlanningVxSignF
#define core3PlanningVySignF menuContext.working.core3PlanningVySignF
#define c1FreshGateMsF menuContext.working.c1FreshGateMsF
#define c3FreshGateMsF menuContext.working.c3FreshGateMsF
#define c4FreshGateMsF menuContext.working.c4FreshGateMsF
#define c4ObservationTimeoutMsF menuContext.working.c4ObservationTimeoutMsF
#define c4EdgeBandLeftF menuContext.working.c4EdgeBandLeftF
#define c4EdgeBandRightF menuContext.working.c4EdgeBandRightF
#define c4EdgeBandTopF menuContext.working.c4EdgeBandTopF
#define c4EdgeBandBottomF menuContext.working.c4EdgeBandBottomF
#define c4BorderEnterLeftF menuContext.working.c4BorderEnterLeftF
#define c4BorderExitLeftF menuContext.working.c4BorderExitLeftF
#define c4BorderEnterRightF menuContext.working.c4BorderEnterRightF
#define c4BorderExitRightF menuContext.working.c4BorderExitRightF
#define c4BorderEnterTopF menuContext.working.c4BorderEnterTopF
#define c4BorderExitTopF menuContext.working.c4BorderExitTopF
#define c4BorderEnterBottomF menuContext.working.c4BorderEnterBottomF
#define c4BorderExitBottomF menuContext.working.c4BorderExitBottomF
#define core4PlanningKpVxF menuContext.working.core4PlanningKpVxF
#define core4PlanningKpVyF menuContext.working.core4PlanningKpVyF
#define core4CenterXF menuContext.working.core4CenterXF
#define core4CenterYF menuContext.working.core4CenterYF
#define core4ScaleF menuContext.working.core4ScaleF
#define core4MountYawRadF menuContext.working.core4MountYawRadF
#define core4MirrorXF menuContext.working.core4MirrorXF
#define core4ErrorFwdSignF menuContext.working.core4ErrorFwdSignF
#define core4ErrorRightSignF menuContext.working.core4ErrorRightSignF
#define core4Core0PoseOffsetXF menuContext.working.core4Core0PoseOffsetXF
#define core4Core0PoseOffsetYF menuContext.working.core4Core0PoseOffsetYF
#define core4PlanningVxSignF menuContext.working.core4PlanningVxSignF
#define core4PlanningVySignF menuContext.working.core4PlanningVySignF
#define menuCurrentState menuContext.state
#define pageIndex menuContext.pageIdx
#define optIdx menuContext.optIdx
#define scrollTop menuContext.scrollTop

```

```

#define forceRedraw menuContext.forceRedraw
#define threshSel menuContext.threshSel
#define controlOwned menuContext.controlOwned
#define feedbackHoldTicks menuContext.feedbackHoldTicks
#define lcdRefreshFlag menuContext.lcdRefreshFlag
#define homeVisualRefreshFlag menuContext.homeVisualRefreshFlag
#define menuPages menuContext.pages
#define menuPageCount menuContext.pageCount
#define kvmTickSeq menuContext.kvmTickSeq
#define feedbackWasActive menuContext.feedbackWasActive
#define lcdRefreshCount menuContext.lcdRefreshCount
#define homeVisualRefreshCount menuContext.homeVisualRefreshCount
#define waitLinkRendered menuContext.waitLinkRendered
#define initFailedMask menuContext.initFailedMask
#define initUnverifiedMask menuContext.initUnverifiedMask
#define initSummaryTicks menuContext.initSummaryTicks
#define initSummaryLastSecond menuContext.initSummaryLastSecond
#define menuInputServiced menuContext.inputServiced
#define menuPendingScreenClear menuContext.pendingScreenClear
#define menuPendingFeedback menuContext.pendingFeedback
#define menuPendingFeedbackColor menuContext.pendingFeedbackColor
#define menuPendingFeedbackAction menuContext.pendingFeedbackAction
#define menuPendingFeedbackSlot menuContext.pendingFeedbackSlot
#define menuPendingStorageAction menuContext.pendingStorageAction
#define mutationLockLast menuContext.mutationLockLast
#define kvmDesiredTargetPrimed menuContext.kvmDesiredTargetPrimed
#define lastValidKvmDesiredTarget menuContext.lastValidKvmDesiredTarget
#define paramSlotsPageIndex menuContext.paramSlotsPageIndex
#define paramSlotSelection menuContext.paramSlotSelection

/* =====
 * 跨文件函数原型
 * ===== */
/* menuModel.c */
void menuSyncFromParams(void); /* 工作副本 ← 当前 visionParams */
void applyVisionLocal(void); /* 工作副本钳位 → 写回 visionParams → visionParamsApply 热重载 */
void menuBuildPages(void);
void menuSaveSettings(void); /* applyVisionLocal + visionParamsSave(flash) */
void menuRestoreDefaults(void); /* visionParamsRestoreDefaults(载默认→apply→擦盘) + 同步工作副本 */
void menuRequestScreenClear(void);
void menuRequestFeedback(uint16_t color);
void menuRequestParamFeedback(uint16_t color, uint8_t action, uint8_t slot);
void menuRequestSaveSettings(void);
void menuRequestRestoreDefaults(void);
void menuObserveKvmDesiredTarget(uint8_t snapshotValid, uint8_t desiredTarget);

/* menuRender.c */
void renderWaitLink(void);
void renderInitFaultSummary(uint8_t failedMask, uint8_t unverifiedMask, uint8_t secondsLeft);
void renderHome(uint8_t visualDue);
void renderThreshEdit(uint8_t visualDue);
void renderPageList(void);
void renderOptionList(uint8_t editing);
void menuRenderResetListSnapshot(void); /* 切状态/清屏复位列表渲染快照 → 下次列表渲染为结构性整页重绘 */

/* menuInput.c */
void clearScreenForState(void);
void handleHomeKeys(void);
void handlePageListKeys(void);
void handleOptionListKeys(void);
void handleEditKeys(void);
void handleThreshEditKeys(void);
void menuParamGlobalReset(void);
void menuParamGlobalPoll(void);
void menuParamGlobalGetUi(menuParamGlobalUiSnapshot *snapshot);

/* 当前菜单分割线/边框颜色(操控权: 被控=绿 未控=红)。render 各页统一调用。 */
uint16_t menuBorderColor(void);

#endif /* _MENU_INTERNAL_H_ */

```

core1_cyt4bb7/project/code/app/menuKeyRemote.c

SHA-256

:

b96f2e5df55aa8fe88d2397c633aa0b8faa8d653e68ec7834da58d346e7bdb6c

代码 (c)

```
#include "menuKeyRemote.h"

/*
 * core1 远程按键源实现。事务 KVM router 在前台通过 menuKeyRemoteInject 投递，
 * 本模块只拥有按键锁存与超时，供菜单 handler 用 menuKeyGetState/Clear 消费。
 *
 * 与 core0 menuKey 的差异：不读 GPIO、不消抖(生产端已消抖识别)，只 seq 去重 + 锁存 + 超时。
 * 与 core0 一致：短按锁存至 Clear；长按消费后置等释放；短按超时自动清除防陈旧。
 */

#define MENU_KEY_LONG_PERIOD_MS (1000u) /* 短按锁存超时(与 core0 一致)，防陈旧事件 */

static volatile uint32_t scanPeriodMs = 0u;
static volatile menuKeyStateEnum keyState[MENU_KEY_COUNT]; /* menuKeyClear 显式消费证据 */
static volatile uint8_t keyConsumed[MENU_KEY_COUNT]; /* 短按锁存已保持时长 */
static volatile uint32_t shortHoldMs[MENU_KEY_COUNT]; /* 最近读到的 core0 链路在线标志 */
static volatile uint8_t linkUpCache = 0u; /* core0 远程模式正操控 core1(决定绿/红边框) */
static volatile uint8_t controlActiveCache = 0u; /* core1 cm_7_0 观测到的 core2 链路断联标志 */
static volatile uint8_t core2LinkLostCache = 0u;

void menuKeyInit(uint32_t periodMs)
{
    if (periodMs == 0u) periodMs = 1u;
    scanPeriodMs = periodMs;
    for (uint8_t i = 0u; i < (uint8_t) MENU_KEY_COUNT; i++)
    {
        keyState[i] = MENU_KEY_RELEASE;
        shortHoldMs[i] = 0u;
    }
    linkUpCache = 0u;
    controlActiveCache = 0u;
    core2LinkLostCache = 0u;
}

void menuKeyScan(void)
{
    /* 短按锁存超时清除(与 core0 menuKey 同语义)：短按事件长时间未被消费自动失效。 */
    for (uint8_t i = 0u; i < (uint8_t) MENU_KEY_COUNT; i++)
    {
        if (keyState[i] == MENU_KEY_SHORT_PRESS)
        {
            shortHoldMs[i] += scanPeriodMs;
            if (shortHoldMs[i] >= MENU_KEY_LONG_PERIOD_MS)
            {
                keyState[i] = MENU_KEY_RELEASE;
                shortHoldMs[i] = 0u;
            }
        }
    }
}

uint8_t menuKeyRemoteInject(uint8_t keyId, uint8_t state)
{
    if ((keyId >= (uint8_t) MENU_KEY_COUNT) ||
        ((state != (uint8_t) MENU_KEY_SHORT_PRESS) &&
         (state != (uint8_t) MENU_KEY_LONG_PRESS)) ||
        (keyState[keyId] != MENU_KEY_RELEASE))
    {
        return 0u;
    }
    keyState[keyId] = (menuKeyStateEnum) state;
    shortHoldMs[keyId] = 0u;
    keyConsumed[keyId] = 0u;
    return 1u;
}

uint8_t menuKeyRemoteTakeConsumed(uint8_t keyId)
{
    uint8_t consumed;
    if (keyId >= (uint8_t) MENU_KEY_COUNT) return 0u;
    consumed = keyConsumed[keyId];
    keyConsumed[keyId] = 0u;
    return consumed;
}
```

```

void menuKeyRemoteRevoke(void)
{
    uint8_t i;
    for (i = 0u; i < (uint8_t) MENU_KEY_COUNT; i++)
    {
        keyState[i] = MENU_KEY_RELEASE;
        shortHoldMs[i] = 0u;
        keyConsumed[i] = 0u;
    }
}

void menuKeyRemoteSetLinkState(uint8_t linkUp, uint8_t controlActive,
    uint8_t core2LinkLost)
{
    linkUpCache = linkUp ? 1u : 0u;
    controlActiveCache = controlActive ? 1u : 0u;
    core2LinkLostCache = core2LinkLost ? 1u : 0u;
}

menuKeyStateEnum menuKeyGetState(menuKeyIndexEnum k)
{
    if (k >= MENU_KEY_COUNT) return MENU_KEY_RELEASE;
    return keyState[k];
}

void menuKeyClear(menuKeyIndexEnum k)
{
    if (k >= MENU_KEY_COUNT) return;
    /* 远程事件无"按住"概念(每个事务事件单次投递), 消费即清。 */
    if (keyState[k] != MENU_KEY_RELEASE) keyConsumed[k] = 1u;
    keyState[k] = MENU_KEY_RELEASE;
    shortHoldMs[k] = 0u;
}

uint8_t menuKeyRemoteLinkUp(void)
{
    return linkUpCache;
}

uint8_t menuKeyRemoteControlActive(void)
{
    return controlActiveCache;
}

uint8_t menuKeyRemoteCore2LinkLost(void)
{
    return core2LinkLostCache;
}

```

core1_cyt4bb7/project/code/app/menuKeyRemote.h

SHA-256

:

922ba08c8895ccb2c90c7a0d40e346dfb40b38d9c58e50faf5fe3ba42e6a7e42

代码 (c)

```

#ifndef _MENU_KEY_REMOTE_H_
#define _MENU_KEY_REMOTE_H_

#include <stdint.h>

/*
 * core1 cm_7_1 远程按键源 —— 接口与 core0 menuKey 完全一致(menuKeyGetState/Clear),
 * 但按键状态不读本地 GPIO(core1 无物理按键), 而来自事务 KVM router:
 * 生产者: core1KvmRouter 在前台校验 session/epoch/lease 后调用 menuKeyRemoteInject
 * 消费者: 菜单 handler 调 menuKeyGetState/Clear(与 core0 同语义)
 *
 * 保留 core0 的锁存 + 超时清除语义: 短按事件锁存至 menuKeyClear; 长时间未消费自动清除
 * 防陈旧。生产者已是"消抖识别后的事件"(core0 menuKeyScan 产出), 故本端不再消抖, 只做
 * exactly-once 由 router 负责; 本模块只做锁存与超时。
 *
 * 线程模型: menuKeyScan 在 PIT_CH2 10ms ISR 更新超时; Inject/Get/Clear 在前台调用,
 * 状态数组 volatile 跨 ISR/前台共享。
 */

```

```

typedef enum
{
    MENU_KEY_1 = 0u,
    MENU_KEY_2,
    MENU_KEY_3,
    MENU_KEY_4,
    MENU_KEY_5,
    MENU_KEY_COUNT
} menuKeyIndexEnum;

typedef enum
{
    MENU_KEY_RELEASE = 0u,
    MENU_KEY_SHORT_PRESS,
    MENU_KEY_LONG_PRESS
} menuKeyStateEnum;

/* 初始化: 清状态。periodMs = 调用 menuKeyRemoteTick 的周期(毫秒) */
void menuKeyInit(uint32_t periodMs);

/* tick: PIT ISR 只做本地锁存超时，不读共享内存/DCache。 */
void menuKeyScan(void);

/* 仅由前台 KVM router 调用。 */
uint8_t menuKeyRemoteInject(uint8_t keyId, uint8_t keyState);
uint8_t menuKeyRemoteTakeConsumed(uint8_t keyId);
void menuKeyRemoteRevoke(void);
void menuKeyRemoteSetLinkState(uint8_t linkUp, uint8_t controlActive,
    uint8_t core2LinkLost);

/* 查询/清除按键状态(短按事件保留至 menuKeyClear) —— 与 core0 menuKey 同名同义 */
menuKeyStateEnum menuKeyGetState(menuKeyIndexEnum k);
void menuKeyClear(menuKeyIndexEnum k);

/* core0 链路是否在线(共享内存 linkUp)。菜单据此做断联门控/提示。 */
uint8_t menuKeyRemoteLinkUp(void);

/* core0 是否远程模式正操控 core1(共享内存 controlActive)。菜单据此画绿(被控)/红(未控)边框。 */
uint8_t menuKeyRemoteControlActive(void);

/* core1 cm_7_0 观测到 core2 RS-485 链路断联=1；供 HOME HUD 显示。 */
uint8_t menuKeyRemoteCore2LinkLost(void);

#endif /* _MENU_KEY_REMOTE_H_ */

```

core1_cyt4bb7/project/code/app/menuModel.c

SHA-256

:

01105ee3599ad8ab437a5c52256dcf0e4677c269579d42faa20b5ecaeea0fdfc

代码 (c)

```

#include "menu.h"
#include "menuInternal.h"

#include "commonDef.h" /* clampf */
#include "visionParams.h" /* 私有参数仓库 facade + DTO */
#include "configBeaconDetect.h" /* VISION_CANNY_THRESHOLD_MIN/MAX/GAP + VISION_EXPOSURE_ */
#include "configBeaconTrack.h" /* VISION_TRACK * 范围/步长 + 关联 alpha */
#include "configBeaconPlanning.h" /* PLANNING_ * 规划运行时默认/约束 + TD/sign/派发 */
#include "configBeaconFusion.h"
#include "core3LinkCore1.h"
#include "core4LinkCore1.h"
#include "core1Loop.h"

/*
 * core1 menuModel.c — 视觉参数模型层: float 工作副本 ↔ visionParams 结构, 分页页表,
 * 热重载(写回 visionParams + visionParamsApply), flash 持久化(经 visionParams)。
 * 不触屏、不读按键。参数覆盖信标视觉(BinaryCC/beacon/track)与规划(planning), 无飞控参数。
 *
 * 工作副本用 float 是为复用 core0 菜单"float* opt[]" 指针编辑 + 步进"的通用 EDIT 框架;
 * 应用时钳位 + 转回 visionParams 的定宽类型。
 */

/* BinaryCC 阈值编辑约束(MIN/MAX/GAP)已移至 config/configBeaconDetect.h(VISION_CANNY_THRESHOLD_)。 */

```

```

static float sanitizeMenuFloat(float v, float def, float lo, float hi)
{
    if (v != v || v > 1.0e30f || v < -1.0e30f) v = def;
    return clampf(v, lo, hi);
}

static void clampBorderPairF(float *enter, float *exit, float maxValue)
{
    if ((enter == 0) || (exit == 0)) return;

    if (*enter < 0.5f)
    {
        *enter = 0.0f;
        *exit = 0.0f;
        return;
    }
    *enter = clampf(*enter, 1.0f, maxValue);
    *exit = clampf(*exit, 0.0f, *enter - 1.0f);
}

/* 工作副本 ← 当前 visionParams(定宽→float) */
void menuSyncFromParams(void)
{
    visionParamsStruct candidate;
    visionParamsGetActive(&candidate);
    detectModeF = (float) candidate.detectMode;
    cannyLowF = (float) candidate.cannyLow;
    cannyHighF = (float) candidate.cannyHigh;
    legacyCannyLowF = (float) candidate.legacyCannyLow;
    legacyCannyHighF = (float) candidate.legacyCannyHigh;
    exposureF = (float) candidate.exposure;
    beaconSizeMinF = (float) candidate.beaconSizeMin;
    beaconSizeMaxF = (float) candidate.beaconSizeMax;
    beaconEccMaxF = candidate.beaconEccMax;
    beaconBrightMinF = (float) candidate.beaconBrightnessMin;
    beaconMaxEndpointsF = (float) candidate.beaconMaxEndpoints;
    beaconFillMinF = candidate.beaconFillMin;
    beaconHuMaxF = candidate.beaconHuMax;
    beaconHuAreaMinF = (float) candidate.beaconHuAreaMin;
    trackAssocDistF = candidate.trackMaxAssocDist;
    trackSizeRatioF = candidate.trackSizeRatioMax;
    trackConfirmHitsF = (float) candidate.trackConfirmHits;
    trackDropMissesF = (float) candidate.trackDropMisses;
    trackCoastMissesF = (float) candidate.trackCoastMisses;
    trackPositionAlphaF = candidate.trackPositionAlpha;
    trackSizeAlphaF = candidate.trackSizeAlpha;
    trackAreaVelAlphaF = candidate.trackAreaVelAlpha;
    trackAssocScoreMaxF = candidate.trackAssocScoreMax;
    trackAssocAreaWeightF = candidate.trackAssocAreaWeight;
    trackAssocAreaVelWeightF = candidate.trackAssocAreaVelWeight;
    planningEdgePredictionHoldMsF = (float) candidate.planningEdgePredictionHoldMs;
    c1EdgeBandLeftF = (float) candidate.c1EdgeBandLeft;
    c1EdgeBandRightF = (float) candidate.c1EdgeBandRight;
    c1EdgeBandTopF = (float) candidate.c1EdgeBandTop;
    c1EdgeBandBottomF = (float) candidate.c1EdgeBandBottom;
    observerEdgeBandLeftF = (float) candidate.observerEdgeBandLeft;
    observerEdgeBandRightF = (float) candidate.observerEdgeBandRight;
    observerEdgeBandTopF = (float) candidate.observerEdgeBandTop;
    observerEdgeBandBottomF = (float) candidate.observerEdgeBandBottom;
    c1BorderEnterLeftF = (float) candidate.c1BorderEnterLeft;
    c1BorderExitLeftF = (float) candidate.c1BorderExitLeft;
    c1BorderEnterRightF = (float) candidate.c1BorderEnterRight;
    c1BorderExitRightF = (float) candidate.c1BorderExitRight;
    c1BorderEnterTopF = (float) candidate.c1BorderEnterTop;
    c1BorderExitTopF = (float) candidate.c1BorderExitTop;
    c1BorderEnterBottomF = (float) candidate.c1BorderEnterBottom;
    c1BorderExitBottomF = (float) candidate.c1BorderExitBottom;
    observerBorderEnterLeftF = (float) candidate.observerBorderEnterLeft;
    observerBorderExitLeftF = (float) candidate.observerBorderExitLeft;
    observerBorderEnterRightF = (float) candidate.observerBorderEnterRight;
    observerBorderExitRightF = (float) candidate.observerBorderExitRight;
    observerBorderEnterTopF = (float) candidate.observerBorderEnterTop;
    observerBorderExitTopF = (float) candidate.observerBorderExitTop;
    observerBorderEnterBottomF = (float) candidate.observerBorderEnterBottom;
    observerBorderExitBottomF = (float) candidate.observerBorderExitBottom;
    planningKpVxF = candidate.planningKpVx;
    planningKpVyF = candidate.planningKpVy;
    core3PlanningKpVxF = candidate.core3PlanningKpVx;

```



```

core3PlanningKpVyF = candidate.core3PlanningKpVy;
planningVxMaxF = candidate.planningVxMax;
planningVyMaxF = candidate.planningVyMax;
planningWzMaxF = candidate.planningWzMax;
planningVxSlewPerSF = candidate.planningVxSlewPerS;
planningVySlewPerSF = candidate.planningVySlewPerS;
planningWzSlewPerSF = candidate.planningWzSlewPerS;
planningVxMinF = candidate.planningVxMin;
planningVyMinF = candidate.planningVyMin;
planningWzMinF = candidate.planningWzMin;
planningDeadzoneEnterF = candidate.planningDeadzoneEnterPx;
planningDeadzoneExitF = candidate.planningDeadzoneExitPx;
planningCore0PoseMinConfidenceF = candidate.planningCore0PoseMinConfidence;
c1Core0PoseOffsetXF = candidate.c1Core0PoseOffsetX;
c1Core0PoseOffsetYF = candidate.c1Core0PoseOffsetY;
planningCore0PoseHeadingOffsetRadF = candidate.planningCore0PoseHeadingOffsetRad;
planningCore0PoseHeadingSignF = (float) candidate.planningCore0PoseHeadingSign;
poseHoldMsF = (float) candidate.poseHoldMs;
planningVxSignF = (float) candidate.planningVxSign;
planningVySignF = (float) candidate.planningVySign;
planningWzSignF = (float) candidate.planningWzSign;
planningDispatchEnableF = (float) candidate.planningDispatchEnable;
planningSearchEnableF = (float) candidate.planningSearchEnable;
planningPlanTimeoutMsF = (float) candidate.planningPlanTimeoutMs;
planningSearchWzF = candidate.planningSearchWz;
planningSearchTimeoutMsF = (float) candidate.planningSearchTimeoutMs;
planningPostTakeoffDelayMsF = (float) candidate.planningPostTakeoffDelayMs;
planningFirstBeaconForwardSpeedF = candidate.planningFirstBeaconForwardSpeed;
planningFirstBeaconForwardEnableF =
    (float) candidate.planningFirstBeaconForwardEnable;
planningTdRvxF = candidate.planningTdRvx;
planningTdRvyF = candidate.planningTdRvy;
planningTdRwzF = candidate.planningTdRwz;
planningTdNF = candidate.planningTdN;
planningTdVxEnableF = (float) candidate.planningTdVxEnable;
planningTdVyEnableF = (float) candidate.planningTdVyEnable;
planningTdWzEnableF = (float) candidate.planningTdWzEnable;
maxBeaconsF = (float) candidate.maxBeacons;
core3ObservationTimeoutMsF = (float) candidate.core3ObservationTimeoutMs;
core3CenterXF = candidate.core3CenterX;
core3CenterYF = candidate.core3CenterY;
core3ScaleF = candidate.core3Scale;
core3MountYawRadF = candidate.core3MountYawRad;
core3MirrorXF = (float) candidate.core3MirrorX;
core3ErrorFwdSignF = (float) candidate.core3ErrorFwdSign;
core3ErrorRightSignF = (float) candidate.core3ErrorRightSign;
core3Core0PoseOffsetXF = candidate.core3Core0PoseOffsetX;
core3Core0PoseOffsetYF = candidate.core3Core0PoseOffsetY;
core3PlanningVxSignF = (float) candidate.core3PlanningVxSign;
core3PlanningVySignF = (float) candidate.core3PlanningVySign;
c1FreshGateMsF = (float) candidate.c1FreshGateMs;
c3FreshGateMsF = (float) candidate.c3FreshGateMs;
c4FreshGateMsF = (float) candidate.c4FreshGateMs;
c4ObservationTimeoutMsF = (float) candidate.c4ObservationTimeoutMs;
c4EdgeBandLeftF = (float) candidate.c4EdgeBandLeft;
c4EdgeBandRightF = (float) candidate.c4EdgeBandRight;
c4EdgeBandTopF = (float) candidate.c4EdgeBandTop;
c4EdgeBandBottomF = (float) candidate.c4EdgeBandBottom;
c4BorderEnterLeftF = (float) candidate.c4BorderEnterLeft;
c4BorderExitLeftF = (float) candidate.c4BorderExitLeft;
c4BorderEnterRightF = (float) candidate.c4BorderEnterRight;
c4BorderExitRightF = (float) candidate.c4BorderExitRight;
c4BorderEnterTopF = (float) candidate.c4BorderEnterTop;
c4BorderExitTopF = (float) candidate.c4BorderExitTop;
c4BorderEnterBottomF = (float) candidate.c4BorderEnterBottom;
c4BorderExitBottomF = (float) candidate.c4BorderExitBottom;
core4PlanningKpVxF = candidate.core4PlanningKpVx;
core4PlanningKpVyF = candidate.core4PlanningKpVy;
core4CenterXF = candidate.core4CenterX;
core4CenterYF = candidate.core4CenterY;
core4ScaleF = candidate.core4Scale;
core4MountYawRadF = candidate.core4MountYawRad;
core4MirrorXF = (float) candidate.core4MirrorX;
core4ErrorFwdSignF = (float) candidate.core4ErrorFwdSign;
core4ErrorRightSignF = (float) candidate.core4ErrorRightSign;
core4Core0PoseOffsetXF = candidate.core4Core0PoseOffsetX;
core4Core0PoseOffsetYF = candidate.core4Core0PoseOffsetY;
core4PlanningVxSignF = (float) candidate.core4PlanningVxSign;
core4PlanningVySignF = (float) candidate.core4PlanningVySign;

```

```

}

/* 工作副本钳位 → 写回 visionParams → 热重载到 BinaryCC/track 检测器(立即生效)。 */
void applyVisionLocal(void)
{
    visionParamsStruct candidate;
    visionParamsGetActive(&candidate);
    detectModeF = (detectModeF >= 0.5f) ? (float) VISION_DETECT_MODE_LEGACY_CANNY
        : (float) VISION_DETECT_MODE_BINARYCC;

    /* BinaryCC 阈值钳位: MIN/MAX/GAP(low+GAP<=high)。 */
    if (cannyLowF < VISION_CANNY_THRESHOLD_MIN) cannyLowF = VISION_CANNY_THRESHOLD_MIN;
    if (cannyHighF > VISION_CANNY_THRESHOLD_MAX) cannyHighF = VISION_CANNY_THRESHOLD_MAX;
    if (cannyLowF + VISION_CANNY_THRESHOLD_GAP > cannyHighF)
    {
        if (cannyHighF > (VISION_CANNY_THRESHOLD_MIN + VISION_CANNY_THRESHOLD_GAP))
        {
            cannyLowF = cannyHighF - VISION_CANNY_THRESHOLD_GAP;
        }
        else
        {
            cannyLowF = VISION_CANNY_THRESHOLD_MIN;
            cannyHighF = VISION_CANNY_THRESHOLD_MIN + VISION_CANNY_THRESHOLD_GAP;
        }
    }
    if (legacyCannyLowF < VISION_LEGACY_CANNY_THRESHOLD_MIN) legacyCannyLowF = VISION_LEGACY_CANNY_THRESHOLD_MIN;
    if (legacyCannyHighF > VISION_LEGACY_CANNY_THRESHOLD_MAX) legacyCannyHighF = VISION_LEGACY_CANNY_THRESHOLD_MAX;
    if (legacyCannyLowF + VISION_LEGACY_CANNY_THRESHOLD_GAP > legacyCannyHighF)
    {
        if (legacyCannyHighF > (VISION_LEGACY_CANNY_THRESHOLD_MIN + VISION_LEGACY_CANNY_THRESHOLD_GAP))
        {
            legacyCannyLowF = legacyCannyHighF - VISION_LEGACY_CANNY_THRESHOLD_GAP;
        }
        else
        {
            legacyCannyLowF = VISION_LEGACY_CANNY_THRESHOLD_MIN;
            legacyCannyHighF = VISION_LEGACY_CANNY_THRESHOLD_MIN + VISION_LEGACY_CANNY_THRESHOLD_GAP;
        }
    }
    exposureF = clampf(exposureF, (float) VISION_EXPOSURE_MIN, (float) VISION_EXPOSURE_MAX);

    /* beacon 钳位: sizeMin<sizeMax 且都在 uint16; ecc>=1; bright/endpoints 范围。 */
    beaconSizeMinF = clampf(beaconSizeMinF, 1.0f, 59 999.0f);
    beaconSizeMaxF = clampf(beaconSizeMaxF, beaconSizeMinF + 1.0f, 60 000.0f);
    beaconEccMaxF = clampf(beaconEccMaxF, 0.0f, 20.0f);
    beaconBrightMinF = clampf(beaconBrightMinF, 0.0f, 255.0f);
    beaconMaxEndpointsF = clampf(beaconMaxEndpointsF, 0.0f, 200.0f);
    beaconFillMinF = clampf(beaconFillMinF, 0.0f, 1.0f);
    beaconHuMaxF = clampf(beaconHuMaxF, VISION_BEACON_HU_MAX_MIN, VISION_BEACON_HU_MAX_MAX);
    beaconHuAreaMinF = clampf(beaconHuAreaMinF,
        VISION_BEACON_HU_AREA_MIN_MIN, VISION_BEACON_HU_AREA_MIN_MAX);

    /* track 钳位: assocDist>0; sizeRatio>1; confirm/drop/coast>=1; 关联 alpha 0.01..1。 */
    trackAssocDistF = clampf(trackAssocDistF, 1.0f, 100.0f);
    trackSizeRatioF = clampf(trackSizeRatioF, 1.1f, 10.0f);
    trackConfirmHitsF = clampf(trackConfirmHitsF, 1.0f, 60.0f);
    trackDropMissesF = clampf(trackDropMissesF, 1.0f, 60.0f);
    trackCoastMissesF = clampf(trackCoastMissesF, 1.0f, 60.0f);
    trackPositionAlphaF = clampf(trackPositionAlphaF, VISION_TRACK_ALPHA_MIN, VISION_TRACK_ALPHA_MAX);
    trackSizeAlphaF = clampf(trackSizeAlphaF, VISION_TRACK_ALPHA_MIN, VISION_TRACK_ALPHA_MAX);
    trackAreaVelAlphaF = clampf(trackAreaVelAlphaF, VISION_TRACK_ALPHA_MIN, VISION_TRACK_ALPHA_MAX);
    trackAssocScoreMaxF = clampf(trackAssocScoreMaxF,
        VISION_TRACK_ASSOC_SCORE_MAX_MIN, VISION_TRACK_ASSOC_SCORE_MAX_MAX);
    trackAssocAreaWeightF = clampf(trackAssocAreaWeightF,
        VISION_TRACK_ASSOC_WEIGHT_MIN, VISION_TRACK_ASSOC_WEIGHT_MAX);
    trackAssocAreaVelWeightF = clampf(trackAssocAreaVelWeightF,
        VISION_TRACK_ASSOC_WEIGHT_MIN, VISION_TRACK_ASSOC_WEIGHT_MAX);
    planningEdgePredictionHoldMsF = clampf(planningEdgePredictionHoldMsF,
        VISION_TRACK_EDGE_PREDICTION_HOLD_MS_MIN,
        VISION_TRACK_EDGE_PREDICTION_HOLD_MS_MAX);
    c1EdgeBandLeftF = clampf(c1EdgeBandLeftF, 0.0f, VISION_TRACK_EDGE_BAND_X_MAX);
    c1EdgeBandRightF = clampf(c1EdgeBandRightF, 0.0f, VISION_TRACK_EDGE_BAND_X_MAX);
    c1EdgeBandTopF = clampf(c1EdgeBandTopF, 0.0f, VISION_TRACK_EDGE_BAND_Y_MAX);
    c1EdgeBandBottomF = clampf(c1EdgeBandBottomF, 0.0f, VISION_TRACK_EDGE_BAND_Y_MAX);
    observerEdgeBandLeftF = clampf(observerEdgeBandLeftF, 0.0f, VISION_TRACK_EDGE_BAND_X_MAX);
    observerEdgeBandRightF = clampf(observerEdgeBandRightF, 0.0f, VISION_TRACK_EDGE_BAND_X_MAX);
    observerEdgeBandTopF = clampf(observerEdgeBandTopF, 0.0f, VISION_TRACK_EDGE_BAND_Y_MAX);
    observerEdgeBandBottomF = clampf(observerEdgeBandBottomF, 0.0f, VISION_TRACK_EDGE_BAND_Y_MAX);
    clampBorderPairF(&c1BorderEnterLeftF, &c1BorderExitLeftF, (float) VISION_TRACK_BORDER_X_MAX);

```

```

clampBorderPairF(&c1BorderEnterRightF, &c1BorderExitRightF, (float) VISION_TRACK_BORDER_X_MAX);
clampBorderPairF(&c1BorderEnterTopF, &c1BorderExitTopF, (float) VISION_TRACK_BORDER_Y_MAX);
clampBorderPairF(&c1BorderEnterBottomF, &c1BorderExitBottomF, (float) VISION_TRACK_BORDER_Y_MAX);
clampBorderPairF(&observerBorderEnterLeftF, &observerBorderExitLeftF, (float) VISION_TRACK_BORDER_X_MAX);
clampBorderPairF(&observerBorderEnterRightF, &observerBorderExitRightF, (float) VISION_TRACK_BORDER_X_MAX);
clampBorderPairF(&observerBorderEnterTopF, &observerBorderExitTopF, (float) VISION_TRACK_BORDER_Y_MAX);
clampBorderPairF(&observerBorderEnterBottomF, &observerBorderExitBottomF, (float) VISION_TRACK_BORDER_Y_MAX);

```

```

/* planning 钳位：中心/增益/限幅/Min/施密特死区/保持阈值/TD 均为运行时参数。 */
planningKpVxF = clampf(planningKpVxF, PLANNING_KP_LINEAR_MIN, PLANNING_KP_LINEAR_MAX);
planningKpVyF = clampf(planningKpVyF, PLANNING_KP_LINEAR_MIN, PLANNING_KP_LINEAR_MAX);
core3PlanningKpVxF = clampf(core3PlanningKpVxF, PLANNING_KP_LINEAR_MIN, PLANNING_KP_LINEAR_MAX);
core3PlanningKpVyF = clampf(core3PlanningKpVyF, PLANNING_KP_LINEAR_MIN, PLANNING_KP_LINEAR_MAX);
planningVxMaxF = sanitizeMenuFloat(planningVxMaxF,
    PLANNING_VX_MAX_DEFAULT, PLANNING_V_LINEAR_LIMIT_MIN, PLANNING_V_LINEAR_LIMIT_MAX);
planningVyMaxF = sanitizeMenuFloat(planningVyMaxF,
    PLANNING_VY_MAX_DEFAULT, PLANNING_V_LINEAR_LIMIT_MIN, PLANNING_V_LINEAR_LIMIT_MAX);
planningWzMaxF = sanitizeMenuFloat(planningWzMaxF,
    PLANNING_WZ_MAX_DEFAULT, PLANNING_WZ_LIMIT_MIN, PLANNING_WZ_LIMIT_MAX);
planningVxSlewPerSF = sanitizeMenuFloat(planningVxSlewPerSF,
    PLANNING_VX_SLEW_DEFAULT, PLANNING_SLEW_MIN, PLANNING_SLEW_MAX);
planningVySlewPerSF = sanitizeMenuFloat(planningVySlewPerSF,
    PLANNING_VY_SLEW_DEFAULT, PLANNING_SLEW_MIN, PLANNING_SLEW_MAX);
planningWzSlewPerSF = sanitizeMenuFloat(planningWzSlewPerSF,
    PLANNING_WZ_SLEW_DEFAULT, PLANNING_SLEW_MIN, PLANNING_SLEW_MAX);
planningVxMinF = sanitizeMenuFloat(planningVxMinF, PLANNING_VX_MIN_DEFAULT, 0.0f, planningVxMaxF);
planningVyMinF = sanitizeMenuFloat(planningVyMinF, PLANNING_VY_MIN_DEFAULT, 0.0f, planningVyMaxF);
planningWzMinF = sanitizeMenuFloat(planningWzMinF, PLANNING_WZ_MIN_DEFAULT, 0.0f, planningWzMaxF);
/* deadzone: 先钳 enter- 后钳 exit (exit 上界依赖已钳位的 enter, exit <= enter - GAP_MIN) , 顺序不可换。 */
planningDeadzoneEnterF = clampf(planningDeadzoneEnterF,
    PLANNING_DEADZONE_ENTER_MIN, PLANNING_DEADZONE_ENTER_MAX);
planningDeadzoneExitF = clampf(planningDeadzoneExitF,
    PLANNING_DEADZONE_EXIT_MIN, planningDeadzoneEnterF - PLANNING_DEADZONE_GAP_MIN);
planningCore0PoseMinConfidenceF = clampf(planningCore0PoseMinConfidenceF,
    PLANNING_CORE0_POSE_MIN_CONF_MIN, PLANNING_CORE0_POSE_MIN_CONF_MAX);
c1Core0PoseOffsetXF = clampf(c1Core0PoseOffsetXF,
    PLANNING_CORE0_POSE_OFFSET_X_MIN, PLANNING_CORE0_POSE_OFFSET_X_MAX);
c1Core0PoseOffsetYF = clampf(c1Core0PoseOffsetYF,
    PLANNING_CORE0_POSE_OFFSET_Y_MIN, PLANNING_CORE0_POSE_OFFSET_Y_MAX);
planningCore0PoseHeadingOffsetRadF = clampf(planningCore0PoseHeadingOffsetRadF,
    PLANNING_CORE0_POSE_HEADING_OFFSET_MIN, PLANNING_CORE0_POSE_HEADING_OFFSET_MAX);
planningCore0PoseHeadingSignF = (planningCore0PoseHeadingSignF >= 0.0f) ? 1.0f : -1.0f;
poseHoldMsF = clampf(poseHoldMsF,
    PLANNING_CORE0_POSE_HOLD_MS_MIN, PLANNING_CORE0_POSE_HOLD_MS_MAX);
/* 离散 sign 归一化 -1/+1 (step 2 翻转) ; 派发开关归一化 0/1; 派发超时/TD 钳到范围。 */
planningVxSignF = (planningVxSignF >= 0.0f) ? 1.0f : -1.0f;
planningVySignF = (planningVySignF >= 0.0f) ? 1.0f : -1.0f;
planningWzSignF = (planningWzSignF >= 0.0f) ? 1.0f : -1.0f;
planningDispatchEnableF = (planningDispatchEnableF >= 0.5f) ? 1.0f : 0.0f;
planningSearchEnableF = (planningSearchEnableF >= 0.5f) ? 1.0f : 0.0f;
planningPlanTimeoutMsF = clampf(planningPlanTimeoutMsF,
    PLANNING_PLAN_TIMEOUT_MS_MIN, PLANNING_PLAN_TIMEOUT_MS_MAX);
planningSearchWzF = sanitizeMenuFloat(planningSearchWzF,
    PLANNING_SEARCH_WZ_DEFAULT, PLANNING_SEARCH_WZ_MIN, PLANNING_SEARCH_WZ_MAX);
planningSearchTimeoutMsF = clampf(planningSearchTimeoutMsF,
    PLANNING_SEARCH_TIMEOUT_MS_MIN, PLANNING_SEARCH_TIMEOUT_MS_MAX);
planningPostTakeoffDelayMsF = sanitizeMenuFloat(
    planningPostTakeoffDelayMsF,
    (float) PLANNING_POST_TAKEOFF_DELAY_MS_DEFAULT,
    PLANNING_POST_TAKEOFF_DELAY_MS_MIN,
    PLANNING_POST_TAKEOFF_DELAY_MS_MAX);
planningFirstBeaconForwardSpeedF = sanitizeMenuFloat(
    planningFirstBeaconForwardSpeedF,
    PLANNING_FIRST_BEACON_FORWARD_SPEED_DEFAULT,
    PLANNING_FIRST_BEACON_FORWARD_SPEED_MIN, planningVxMaxF);
planningFirstBeaconForwardEnableF =
    (planningFirstBeaconForwardEnableF >= 0.5f) ? 1.0f : 0.0f;
planningTdRvxF = clampf(planningTdRvxF, PLANNING_TD_R_MIN, PLANNING_TD_R_MAX);
planningTdRvyF = clampf(planningTdRvyF, PLANNING_TD_R_MIN, PLANNING_TD_R_MAX);
planningTdRwzF = clampf(planningTdRwzF, PLANNING_TD_R_MIN, PLANNING_TD_R_MAX);
planningTdNF = clampf(planningTdNF, PLANNING_TD_N_MIN, PLANNING_TD_N_MAX);
planningTdVxEnableF = (planningTdVxEnableF >= 0.5f) ? 1.0f : 0.0f;
planningTdVyEnableF = (planningTdVyEnableF >= 0.5f) ? 1.0f : 0.0f;
planningTdWzEnableF = (planningTdWzEnableF >= 0.5f) ? 1.0f : 0.0f;
maxBeaconsF = clampf(maxBeaconsF,
    (float) BEACON_MAX_BEACONS_MIN, (float) BEACON_MAX_BEACONS_MAX);
core3ObservationTimeoutMsF = clampf(core3ObservationTimeoutMsF,
    (float) CORE3_OBSERVATION_TIMEOUT_MS_MIN,
    (float) CORE3_OBSERVATION_TIMEOUT_MS_MAX);

```

```

core3CenterXF = sanitizeMenuFloat(core3CenterXF,
    CORE3_CALIBRATION_CENTER_X_DEFAULT,
    CORE3_CALIBRATION_CENTER_X_MIN, CORE3_CALIBRATION_CENTER_X_MAX);
core3CenterYF = sanitizeMenuFloat(core3CenterYF,
    CORE3_CALIBRATION_CENTER_Y_DEFAULT,
    CORE3_CALIBRATION_CENTER_Y_MIN, CORE3_CALIBRATION_CENTER_Y_MAX);
core3ScaleF = sanitizeMenuFloat(core3ScaleF,
    CORE3_CALIBRATION_SCALE_DEFAULT,
    CORE3_CALIBRATION_SCALE_MIN, CORE3_CALIBRATION_SCALE_MAX);
core3MountYawRadF = sanitizeMenuFloat(core3MountYawRadF,
    CORE3_CALIBRATION_YAW_RAD_DEFAULT,
    CORE3_CALIBRATION_YAW_RAD_MIN, CORE3_CALIBRATION_YAW_RAD_MAX);
core3MirrorXF = (core3MirrorXF >= 0.5f) ? 1.0f : 0.0f;
core3ErrorFwdSignF = (core3ErrorFwdSignF >= 0.0f) ? 1.0f : -1.0f;
core3ErrorRightSignF = (core3ErrorRightSignF >= 0.0f) ? 1.0f : -1.0f;
core3Core0PoseOffsetXF = sanitizeMenuFloat(core3Core0PoseOffsetXF,
    CORE3_CORE0_POSE_OFFSET_X_DEFAULT,
    PLANNING_CORE0_POSE_OFFSET_X_MIN, PLANNING_CORE0_POSE_OFFSET_X_MAX);
core3Core0PoseOffsetYF = sanitizeMenuFloat(core3Core0PoseOffsetYF,
    CORE3_CORE0_POSE_OFFSET_Y_DEFAULT,
    PLANNING_CORE0_POSE_OFFSET_Y_MIN, PLANNING_CORE0_POSE_OFFSET_Y_MAX);
core3PlanningVxSignF = (core3PlanningVxSignF >= 0.0f) ? 1.0f : -1.0f;
core3PlanningVySignF = (core3PlanningVySignF >= 0.0f) ? 1.0f : -1.0f;
c1FreshGateMsF = clampf(c1FreshGateMsF,
    (float) SOURCE_FRESH_GATE_MS_MIN, (float) SOURCE_FRESH_GATE_MS_MAX);
c3FreshGateMsF = clampf(c3FreshGateMsF,
    (float) SOURCE_FRESH_GATE_MS_MIN, (float) SOURCE_FRESH_GATE_MS_MAX);
c4FreshGateMsF = clampf(c4FreshGateMsF,
    (float) SOURCE_FRESH_GATE_MS_MIN, (float) SOURCE_FRESH_GATE_MS_MAX);
c4ObservationTimeoutMsF = clampf(c4ObservationTimeoutMsF,
    (float) CORE4_OBSERVATION_TIMEOUT_MS_MIN,
    (float) CORE4_OBSERVATION_TIMEOUT_MS_MAX);
c4EdgeBandLeftF = clampf(c4EdgeBandLeftF, 0.0f, VISION_TRACK_EDGE_BAND_X_MAX);
c4EdgeBandRightF = clampf(c4EdgeBandRightF, 0.0f, VISION_TRACK_EDGE_BAND_X_MAX);
c4EdgeBandTopF = clampf(c4EdgeBandTopF, 0.0f, VISION_TRACK_EDGE_BAND_Y_MAX);
c4EdgeBandBottomF = clampf(c4EdgeBandBottomF, 0.0f, VISION_TRACK_EDGE_BAND_Y_MAX);
clampBorderPairF(&c4BorderEnterLeftF, &c4BorderExitLeftF, (float) VISION_TRACK_BORDER_X_MAX);
clampBorderPairF(&c4BorderEnterRightF, &c4BorderExitRightF, (float) VISION_TRACK_BORDER_X_MAX);
clampBorderPairF(&c4BorderEnterTopF, &c4BorderExitTopF, (float) VISION_TRACK_BORDER_Y_MAX);
clampBorderPairF(&c4BorderEnterBottomF, &c4BorderExitBottomF, (float) VISION_TRACK_BORDER_Y_MAX);
core4PlanningKpVxF = clampf(core4PlanningKpVxF, PLANNING_KP_LINEAR_MIN, PLANNING_KP_LINEAR_MAX);
core4PlanningKpVyF = clampf(core4PlanningKpVyF, PLANNING_KP_LINEAR_MIN, PLANNING_KP_LINEAR_MAX);
core4CenterXF = sanitizeMenuFloat(core4CenterXF, CORE4_CALIBRATION_CENTER_X_DEFAULT,
    CORE3_CALIBRATION_CENTER_X_MIN, CORE3_CALIBRATION_CENTER_X_MAX);
core4CenterYF = sanitizeMenuFloat(core4CenterYF, CORE4_CALIBRATION_CENTER_Y_DEFAULT,
    CORE3_CALIBRATION_CENTER_Y_MIN, CORE3_CALIBRATION_CENTER_Y_MAX);
core4ScaleF = sanitizeMenuFloat(core4ScaleF, CORE4_CALIBRATION_SCALE_DEFAULT,
    CORE3_CALIBRATION_SCALE_MIN, CORE3_CALIBRATION_SCALE_MAX);
core4MountYawRadF = sanitizeMenuFloat(core4MountYawRadF,
    CORE4_CALIBRATION_YAW_RAD_DEFAULT,
    CORE3_CALIBRATION_YAW_RAD_MIN, CORE3_CALIBRATION_YAW_RAD_MAX);
core4MirrorXF = (core4MirrorXF >= 0.5f) ? 1.0f : 0.0f;
core4ErrorFwdSignF = (core4ErrorFwdSignF >= 0.0f) ? 1.0f : -1.0f;
core4ErrorRightSignF = (core4ErrorRightSignF >= 0.0f) ? 1.0f : -1.0f;
core4Core0PoseOffsetXF = clampf(core4Core0PoseOffsetXF,
    PLANNING_CORE0_POSE_OFFSET_X_MIN, PLANNING_CORE0_POSE_OFFSET_X_MAX);
core4Core0PoseOffsetYF = clampf(core4Core0PoseOffsetYF,
    PLANNING_CORE0_POSE_OFFSET_Y_MIN, PLANNING_CORE0_POSE_OFFSET_Y_MAX);
core4PlanningVxSignF = (core4PlanningVxSignF >= 0.0f) ? 1.0f : -1.0f;
core4PlanningVySignF = (core4PlanningVySignF >= 0.0f) ? 1.0f : -1.0f;

```

```

/* 写回 visionParams(float→定宽, +0.5f 四舍五入) */
candidate.detectMode = (uint8_t) detectModeF;
candidate.cannyLow = (uint16_t) (cannyLowF + 0.5f);
candidate.cannyHigh = (uint16_t) (cannyHighF + 0.5f);
candidate.legacyCannyLow = (uint16_t) (legacyCannyLowF + 0.5f);
candidate.legacyCannyHigh = (uint16_t) (legacyCannyHighF + 0.5f);
candidate.exposure = (uint16_t) (exposureF + 0.5f);
candidate.beaconSizeMin = (uint16_t) (beaconSizeMinF + 0.5f);
candidate.beaconSizeMax = (uint16_t) (beaconSizeMaxF + 0.5f);
candidate.beaconEccMax = beaconEccMaxF;
candidate.beaconBrightnessMin = (uint8_t) (beaconBrightMinF + 0.5f);
candidate.beaconMaxEndpoints = (uint16_t) (beaconMaxEndpointsF + 0.5f);
candidate.beaconFillMin = beaconFillMinF;
candidate.beaconHuMax = beaconHuMaxF;
candidate.beaconHuAreaMin = (uint16_t) (beaconHuAreaMinF + 0.5f);
candidate.trackMaxAssocDist = trackAssocDistF;
candidate.trackSizeRatioMax = trackSizeRatioF;
candidate.trackConfirmHits = (uint16_t) (trackConfirmHitsF + 0.5f);

```

```

candidate.trackDropMisses = (uint16_t) (trackDropMissesF + 0.5f);
candidate.trackCoastMisses = (uint16_t) (trackCoastMissesF + 0.5f);
candidate.trackPositionAlpha = trackPositionAlphaF;
candidate.trackSizeAlpha = trackSizeAlphaF;
candidate.trackAreaVelAlpha = trackAreaVelAlphaF;
candidate.trackAssocScoreMax = trackAssocScoreMaxF;
candidate.trackAssocAreaWeight = trackAssocAreaWeightF;
candidate.trackAssocAreaVelWeight = trackAssocAreaVelWeightF;
candidate.planningEdgePredictionHoldMs = (uint16_t) (planningEdgePredictionHoldMsF + 0.5f);
candidate.c1EdgeBandLeft = (uint8_t) (c1EdgeBandLeftF + 0.5f);
candidate.c1EdgeBandRight = (uint8_t) (c1EdgeBandRightF + 0.5f);
candidate.c1EdgeBandTop = (uint8_t) (c1EdgeBandTopF + 0.5f);
candidate.c1EdgeBandBottom = (uint8_t) (c1EdgeBandBottomF + 0.5f);
candidate.observerEdgeBandLeft = (uint8_t) (observerEdgeBandLeftF + 0.5f);
candidate.observerEdgeBandRight = (uint8_t) (observerEdgeBandRightF + 0.5f);
candidate.observerEdgeBandTop = (uint8_t) (observerEdgeBandTopF + 0.5f);
candidate.observerEdgeBandBottom = (uint8_t) (observerEdgeBandBottomF + 0.5f);
candidate.c1BorderEnterLeft = (uint8_t) (c1BorderEnterLeftF + 0.5f);
candidate.c1BorderExitLeft = (uint8_t) (c1BorderExitLeftF + 0.5f);
candidate.c1BorderEnterRight = (uint8_t) (c1BorderEnterRightF + 0.5f);
candidate.c1BorderExitRight = (uint8_t) (c1BorderExitRightF + 0.5f);
candidate.c1BorderEnterTop = (uint8_t) (c1BorderEnterTopF + 0.5f);
candidate.c1BorderExitTop = (uint8_t) (c1BorderExitTopF + 0.5f);
candidate.c1BorderEnterBottom = (uint8_t) (c1BorderEnterBottomF + 0.5f);
candidate.c1BorderExitBottom = (uint8_t) (c1BorderExitBottomF + 0.5f);
candidate.observerBorderEnterLeft = (uint8_t) (observerBorderEnterLeftF + 0.5f);
candidate.observerBorderExitLeft = (uint8_t) (observerBorderExitLeftF + 0.5f);
candidate.observerBorderEnterRight = (uint8_t) (observerBorderEnterRightF + 0.5f);
candidate.observerBorderExitRight = (uint8_t) (observerBorderExitRightF + 0.5f);
candidate.observerBorderEnterTop = (uint8_t) (observerBorderEnterTopF + 0.5f);
candidate.observerBorderExitTop = (uint8_t) (observerBorderExitTopF + 0.5f);
candidate.observerBorderEnterBottom = (uint8_t) (observerBorderEnterBottomF + 0.5f);
candidate.observerBorderExitBottom = (uint8_t) (observerBorderExitBottomF + 0.5f);
candidate.planningKpVx = planningKpVxF;
candidate.planningKpVy = planningKpVyF;
candidate.core3PlanningKpVx = core3PlanningKpVxF;
candidate.core3PlanningKpVy = core3PlanningKpVyF;
candidate.planningVxMax = planningVxMaxF;
candidate.planningVyMax = planningVyMaxF;
candidate.planningWzMax = planningWzMaxF;
candidate.planningVxSlewPerS = planningVxSlewPerSF;
candidate.planningVySlewPerS = planningVySlewPerSF;
candidate.planningWzSlewPerS = planningWzSlewPerSF;
candidate.planningVxMin = planningVxMinF;
candidate.planningVyMin = planningVyMinF;
candidate.planningWzMin = planningWzMinF;
candidate.planningDeadzoneEnterPx = planningDeadzoneEnterF;
candidate.planningDeadzoneExitPx = planningDeadzoneExitF;
candidate.planningCore0PoseMinConfidence = planningCore0PoseMinConfidenceF;
candidate.c1Core0PoseOffsetX = c1Core0PoseOffsetXF;
candidate.c1Core0PoseOffsetY = c1Core0PoseOffsetYF;
candidate.planningCore0PoseHeadingOffsetRad = planningCore0PoseHeadingOffsetRadF;
candidate.planningCore0PoseHeadingSign = (int8_t) planningCore0PoseHeadingSignF;
candidate.poseHoldMs = (uint16_t) (poseHoldMsF + 0.5f);
candidate.planningVxSign = (int8_t) planningVxSignF;
candidate.planningVySign = (int8_t) planningVySignF;
candidate.planningWzSign = (int8_t) planningWzSignF;
candidate.planningDispatchEnable = (uint8_t) planningDispatchEnableF;
candidate.planningSearchEnable = (uint8_t) planningSearchEnableF;
candidate.planningPlanTimeoutMs = (uint16_t) (planningPlanTimeoutMsF + 0.5f);
candidate.planningSearchWz = planningSearchWzF;
candidate.planningSearchTimeoutMs = (uint16_t) (planningSearchTimeoutMsF + 0.5f);
candidate.planningPostTakeoffDelayMs =
    (uint16_t) (planningPostTakeoffDelayMsF + 0.5f);
candidate.planningFirstBeaconForwardSpeed =
    planningFirstBeaconForwardSpeedF;
candidate.planningFirstBeaconForwardEnable =
    (uint8_t) planningFirstBeaconForwardEnableF;
candidate.planningTdRvx = planningTdRvxF;
candidate.planningTdRvy = planningTdRvyF;
candidate.planningTdRwz = planningTdRwzF;
candidate.planningTdN = planningTdNF;
candidate.planningTdVxEnable = (uint8_t) planningTdVxEnableF;
candidate.planningTdVyEnable = (uint8_t) planningTdVyEnableF;
candidate.planningTdWzEnable = (uint8_t) planningTdWzEnableF;
candidate.maxBeacons = (uint8_t) (maxBeaconsF + 0.5f);
candidate.core3ObservationTimeoutMs = (uint16_t) (core3ObservationTimeoutMsF + 0.5f);
candidate.core3CenterX = core3CenterXF;
candidate.core3CenterY = core3CenterYF;

```

```

candidate.core3Scale = core3ScaleF;
candidate.core3MountYawRad = core3MountYawRadF;
candidate.core3MirrorX = (uint8_t) core3MirrorXF;
candidate.core3ErrorFwdSign = (int8_t) core3ErrorFwdSignF;
candidate.core3ErrorRightSign = (int8_t) core3ErrorRightSignF;
candidate.core3Core0PoseOffsetX = core3Core0PoseOffsetXF;
candidate.core3Core0PoseOffsetY = core3Core0PoseOffsetYF;
candidate.core3PlanningVxSign = (int8_t) core3PlanningVxSignF;
candidate.core3PlanningVySign = (int8_t) core3PlanningVySignF;
candidate.c1FreshGateMs = (uint16_t) (c1FreshGateMsF + 0.5f);
candidate.c3FreshGateMs = (uint16_t) (c3FreshGateMsF + 0.5f);
candidate.c4FreshGateMs = (uint16_t) (c4FreshGateMsF + 0.5f);
candidate.c4ObservationTimeoutMs = (uint16_t) (c4ObservationTimeoutMsF + 0.5f);
candidate.c4EdgeBandLeft = (uint8_t) (c4EdgeBandLeftF + 0.5f);
candidate.c4EdgeBandRight = (uint8_t) (c4EdgeBandRightF + 0.5f);
candidate.c4EdgeBandTop = (uint8_t) (c4EdgeBandTopF + 0.5f);
candidate.c4EdgeBandBottom = (uint8_t) (c4EdgeBandBottomF + 0.5f);
candidate.c4BorderEnterLeft = (uint8_t) (c4BorderEnterLeftF + 0.5f);
candidate.c4BorderExitLeft = (uint8_t) (c4BorderExitLeftF + 0.5f);
candidate.c4BorderEnterRight = (uint8_t) (c4BorderEnterRightF + 0.5f);
candidate.c4BorderExitRight = (uint8_t) (c4BorderExitRightF + 0.5f);
candidate.c4BorderEnterTop = (uint8_t) (c4BorderEnterTopF + 0.5f);
candidate.c4BorderExitTop = (uint8_t) (c4BorderExitTopF + 0.5f);
candidate.c4BorderEnterBottom = (uint8_t) (c4BorderEnterBottomF + 0.5f);
candidate.c4BorderExitBottom = (uint8_t) (c4BorderExitBottomF + 0.5f);
candidate.core4PlanningKpVx = core4PlanningKpVxF;
candidate.core4PlanningKpVy = core4PlanningKpVyF;
candidate.core4CenterX = core4CenterXF;
candidate.core4CenterY = core4CenterYF;
candidate.core4Scale = core4ScaleF;
candidate.core4MountYawRad = core4MountYawRadF;
candidate.core4MirrorX = (uint8_t) core4MirrorXF;
candidate.core4ErrorFwdSign = (int8_t) core4ErrorFwdSignF;
candidate.core4ErrorRightSign = (int8_t) core4ErrorRightSignF;
candidate.core4Core0PoseOffsetX = core4Core0PoseOffsetXF;
candidate.core4Core0PoseOffsetY = core4Core0PoseOffsetYF;
candidate.core4PlanningVxSign = (int8_t) core4PlanningVxSignF;
candidate.core4PlanningVySign = (int8_t) core4PlanningVySignF;

(void) visionParamsCommit(&candidate); /* 单次提交并热重载全部 owner */
}

/* 保存: 钳位应用 + 落 flash。 */
void menuSaveSettings(void)
{
    applyVisionLocal();
    core3LinkCore1SyncDesiredIdentity();
    core4LinkCore1SyncDesiredIdentity();
    visionParamsSave();
}

/* 恢复默认: visionParams 次序(载默认→apply→擦盘, 擦后立即按默认运行) + 同步工作副本。 */
void menuRestoreDefaults(void)
{
    visionParamsRestoreDefaults();
    menuSyncFromParams();
}

/* =====
* 分页页表(信标视觉 + 规划参数)
* ===== */
static menuPageStruct* addPage(const char* name)
{
    menuPageStruct* pg;
    if (menuPageCount >= MENU_MAX_PAGES) return 0;
    pg = &menuPages[menuPageCount];
    pg->name = name;
    pg->optionCount = 0u;
    menuPageCount++;
    return pg;
}

static void addPageOption(menuPageStruct* pg, uint8_t* index, const char* name,
    float* value, float stepShort, float stepLong)
{
    uint8_t i = *index;
    pg->optName[i] = name;
    pg->opt[i] = value;
    pg->stepS[i] = stepShort;

```

```

pg->stepL[i] = stepLong;
pg->decimals[i] = 0u;
*index = (uint8_t) (i + 1u);
}

static void addPageOptionDecimal(menuPageStruct* pg, uint8_t* index, const char* name,
float* value, float stepShort, float stepLong, uint8_t decimals)
{
    uint8_t i = *index;
    pg->optName[i] = name;
    pg->opt[i] = value;
    pg->stepS[i] = stepShort;
    pg->stepL[i] = stepLong;
    pg->decimals[i] = decimals;
    *index = (uint8_t) (i + 1u);
}

void menuBuildPages(void)
{
    menuPageCount = 0u;

    /* 页 1: DetectMode + Threshold + Camera */
    {
        menuPageStruct* pg = addPage("Bin/Cam");
        uint8_t i = 0u;
        if (pg == 0) return;
        addPageOption(pg, &i, "Mode", &detectModeF, 1.0f, 1.0f);
        addPageOption(pg, &i, "Bin Low", &cannyLowF, VISION_CANNY_STEP_SHORT, VISION_CANNY_STEP_LONG);
        addPageOption(pg, &i, "Bin High", &cannyHighF, VISION_CANNY_STEP_SHORT, VISION_CANNY_STEP_LONG);
        addPageOption(pg, &i, "Can Low", &legacyCannyLowF, VISION_CANNY_STEP_SHORT, VISION_CANNY_STEP_LONG);
        addPageOption(pg, &i, "Can High", &legacyCannyHighF, VISION_CANNY_STEP_SHORT, VISION_CANNY_STEP_LONG);
        addPageOption(pg, &i, "Exposure", &exposureF, VISION_EXPOSURE_STEP_SHORT, VISION_EXPOSURE_STEP_LONG);
        pg->optionCount = i;
    }

    /* 页 2: BeaconCC 检测 */
    {
        menuPageStruct* pg = addPage("Beacon");
        uint8_t i = 0u;
        if (pg == 0) return;
        addPageOption(pg, &i, "SizeMin", &beaconSizeMinF, 1.0f, 10.0f);
        addPageOption(pg, &i, "SizeMax", &beaconSizeMaxF, 10.0f, 100.0f);
        addPageOption(pg, &i, "EccMax", &beaconEccMaxF, 0.1f, 1.0f);
        addPageOption(pg, &i, "BrightMin", &beaconBrightMinF, 1.0f, 10.0f);
        addPageOption(pg, &i, "MaxEndpt", &beaconMaxEndpointsF, 1.0f, 5.0f);
        addPageOptionDecimal(pg, &i, "FillMin", &beaconFillMinF, 0.01f, 0.10f, 2u);
        addPageOptionDecimal(pg, &i, "HuMax", &beaconHuMaxF, VISION_BEACON_HU_MAX_STEP_SHORT, VISION_BEACON_HU_MAX_STEP_LONG,
1u);
        addPageOption(pg, &i, "HuAreaMin", &beaconHuAreaMinF, VISION_BEACON_HU_AREA_MIN_STEP_SHORT,
VISION_BEACON_HU_AREA_MIN_STEP_LONG);
        pg->optionCount = i;
    }

    /* 页 3: Track 跟踪 */
    {
        menuPageStruct* pg = addPage("Track");
        uint8_t i = 0u;
        if (pg == 0) return;
        addPageOption(pg, &i, "AssocDist", &trackAssocDistF, 0.5f, 2.0f);
        addPageOption(pg, &i, "SizeRatio", &trackSizeRatioF, 0.1f, 0.5f);
        addPageOption(pg, &i, "ConfirmHit", &trackConfirmHitsF, 1.0f, 2.0f);
        addPageOption(pg, &i, "DropMiss", &trackDropMissesF, 1.0f, 2.0f);
        addPageOption(pg, &i, "CoastMiss", &trackCoastMissesF, 1.0f, 2.0f);
        addPageOptionDecimal(pg, &i, "PosAlpha", &trackPositionAlphaF, VISION_TRACK_ALPHA_STEP_SHORT, VISION_TRACK_ALPHA_STEP_LONG,
2u);
        addPageOptionDecimal(pg, &i, "SizeAlpha", &trackSizeAlphaF, VISION_TRACK_ALPHA_STEP_SHORT, VISION_TRACK_ALPHA_STEP_LONG, 2u);
        addPageOptionDecimal(pg, &i, "AreaVelA", &trackAreaVelAlphaF, VISION_TRACK_ALPHA_STEP_SHORT, VISION_TRACK_ALPHA_STEP_LONG,
2u);
        addPageOptionDecimal(pg, &i, "AssocScr", &trackAssocScoreMaxF, VISION_TRACK_ASSOC_SCORE_MAX_STEP_SHORT,
VISION_TRACK_ASSOC_SCORE_MAX_STEP_LONG, 2u);
        addPageOptionDecimal(pg, &i, "AreaW", &trackAssocAreaWeightF, VISION_TRACK_ASSOC_WEIGHT_STEP_SHORT,
VISION_TRACK_ASSOC_WEIGHT_STEP_LONG, 2u);
        addPageOptionDecimal(pg, &i, "AreaVelW", &trackAssocAreaVelWeightF, VISION_TRACK_ASSOC_WEIGHT_STEP_SHORT,
VISION_TRACK_ASSOC_WEIGHT_STEP_LONG, 2u);
        addPageOption(pg, &i, "EdgeHoldMs", &planningEdgePredictionHoldMsF,
VISION_TRACK_EDGE_PREDICTION_HOLD_STEP_SHORT,
VISION_TRACK_EDGE_PREDICTION_HOLD_STEP_LONG);
        pg->optionCount = i;
    }
}

```

```

/* Core1 独立四边发布边界施密特门。 */
{
    menuPageStruct* pg = addPage("C1 Border");
    uint8_t i = 0u;
    if (pg == 0) return;
    addPageOption(pg, &i, "LIn", &c1BorderEnterLeftF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "LOut", &c1BorderExitLeftF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "RIn", &c1BorderEnterRightF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "ROut", &c1BorderExitRightF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "TIn", &c1BorderEnterTopF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "TOut", &c1BorderExitTopF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "BIn", &c1BorderEnterBottomF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "BOut", &c1BorderExitBottomF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "BandL", &c1EdgeBandLeftF,
        VISION_TRACK_EDGE_BAND_STEP_SHORT, VISION_TRACK_EDGE_BAND_STEP_LONG);
    addPageOption(pg, &i, "BandR", &c1EdgeBandRightF,
        VISION_TRACK_EDGE_BAND_STEP_SHORT, VISION_TRACK_EDGE_BAND_STEP_LONG);
    addPageOption(pg, &i, "BandT", &c1EdgeBandTopF,
        VISION_TRACK_EDGE_BAND_STEP_SHORT, VISION_TRACK_EDGE_BAND_STEP_LONG);
    addPageOption(pg, &i, "BandB", &c1EdgeBandBottomF,
        VISION_TRACK_EDGE_BAND_STEP_SHORT, VISION_TRACK_EDGE_BAND_STEP_LONG);
    pg->optionCount = i;
}

/* Core3 独立四边发布边界施密特门，经 Core1 专线同步。 */
{
    menuPageStruct* pg = addPage("C3 Border");
    uint8_t i = 0u;
    if (pg == 0) return;
    addPageOption(pg, &i, "LIn", &observerBorderEnterLeftF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "LOut", &observerBorderExitLeftF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "RIn", &observerBorderEnterRightF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "ROut", &observerBorderExitRightF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "TIn", &observerBorderEnterTopF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "TOut", &observerBorderExitTopF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "BIn", &observerBorderEnterBottomF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "BOut", &observerBorderExitBottomF,
        VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "BandL", &observerEdgeBandLeftF,
        VISION_TRACK_EDGE_BAND_STEP_SHORT, VISION_TRACK_EDGE_BAND_STEP_LONG);
    addPageOption(pg, &i, "BandR", &observerEdgeBandRightF,
        VISION_TRACK_EDGE_BAND_STEP_SHORT, VISION_TRACK_EDGE_BAND_STEP_LONG);
    addPageOption(pg, &i, "BandT", &observerEdgeBandTopF,
        VISION_TRACK_EDGE_BAND_STEP_SHORT, VISION_TRACK_EDGE_BAND_STEP_LONG);
    addPageOption(pg, &i, "BandB", &observerEdgeBandBottomF,
        VISION_TRACK_EDGE_BAND_STEP_SHORT, VISION_TRACK_EDGE_BAND_STEP_LONG);
    pg->optionCount = i;
}

/* Core4 独立四边发布边界与 EdgeBand，经独立 endpoint 同步。 */
{
    menuPageStruct* pg = addPage("C4 Border");
    uint8_t i = 0u;
    if (pg == 0) return;
    addPageOption(pg, &i, "LIn", &c4BorderEnterLeftF, VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "LOut", &c4BorderExitLeftF, VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "RIn", &c4BorderEnterRightF, VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "ROut", &c4BorderExitRightF, VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "TIn", &c4BorderEnterTopF, VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "TOut", &c4BorderExitTopF, VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "BIn", &c4BorderEnterBottomF, VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "BOut", &c4BorderExitBottomF, VISION_TRACK_BORDER_STEP_SHORT, VISION_TRACK_BORDER_STEP_LONG);
    addPageOption(pg, &i, "BandL", &c4EdgeBandLeftF, VISION_TRACK_EDGE_BAND_STEP_SHORT, VISION_TRACK_EDGE_BAND_STEP_LONG);
}

```



```

        addPageOption(pg, &i, "BandR", &c4EdgeBandRightF, VISION_TRACK_EDGE_BAND_STEP_SHORT, VISION_TRACK_EDGE_BAND_STEP_LONG);
        addPageOption(pg, &i, "BandT", &c4EdgeBandTopF, VISION_TRACK_EDGE_BAND_STEP_SHORT, VISION_TRACK_EDGE_BAND_STEP_LONG);
        addPageOption(pg, &i, "BandB", &c4EdgeBandBottomF, VISION_TRACK_EDGE_BAND_STEP_SHORT, VISION_TRACK_EDGE_BAND_STEP_LONG);
        pg->optionCount = i;
    }

/* 三来源 TRACK 只编辑平移增益； 旋转只由 SEARCH 产生。 */
{
    menuPageStruct* pg = addPage("C1 Gain");
    uint8_t i = 0u;
    if (pg == 0) return;
    addPageOptionDecimal(pg, &i, "KpVx", &planningKpVxF, PLANNING_KP_VX_STEP_SHORT, PLANNING_KP_VX_STEP_LONG, 3u);
    addPageOptionDecimal(pg, &i, "KpVy", &planningKpVyF, PLANNING_KP_VY_STEP_SHORT, PLANNING_KP_VY_STEP_LONG, 3u);
    pg->optionCount = i;
}

{
    menuPageStruct* pg = addPage("C3 Gain");
    uint8_t i = 0u;
    if (pg == 0) return;
    addPageOptionDecimal(pg, &i, "KpVx", &core3PlanningKpVxF, PLANNING_KP_VX_STEP_SHORT, PLANNING_KP_VX_STEP_LONG, 3u);
    addPageOptionDecimal(pg, &i, "KpVy", &core3PlanningKpVyF, PLANNING_KP_VY_STEP_SHORT, PLANNING_KP_VY_STEP_LONG, 3u);
    pg->optionCount = i;
}

{
    menuPageStruct* pg = addPage("C4 Gain");
    uint8_t i = 0u;
    if (pg == 0) return;
    addPageOptionDecimal(pg, &i, "KpVx", &core4PlanningKpVxF, PLANNING_KP_VX_STEP_SHORT, PLANNING_KP_VX_STEP_LONG, 3u);
    addPageOptionDecimal(pg, &i, "KpVy", &core4PlanningKpVyF, PLANNING_KP_VY_STEP_SHORT, PLANNING_KP_VY_STEP_LONG, 3u);
    pg->optionCount = i;
}

/* Plan Limit: Max 对齐 Core2 接收上限, Min 是可选 TD 之后的最终非零下发约束。 */
{
    menuPageStruct* pg = addPage("Plan Limit");
    uint8_t i = 0u;
    if (pg == 0) return;
    addPageOptionDecimal(pg, &i, "VxMax", &planningVxMaxF, PLANNING_V_LIMIT_STEP_SHORT, PLANNING_V_LIMIT_STEP_LONG, 2u);
    addPageOptionDecimal(pg, &i, "VyMax", &planningVyMaxF, PLANNING_V_LIMIT_STEP_SHORT, PLANNING_V_LIMIT_STEP_LONG, 2u);
    addPageOptionDecimal(pg, &i, "WzMax", &planningWzMaxF, PLANNING_WZ_LIMIT_STEP_SHORT, PLANNING_WZ_LIMIT_STEP_LONG, 2u);
    addPageOptionDecimal(pg, &i, "VxMin", &planningVxMinF, PLANNING_V_MIN_STEP_SHORT, PLANNING_V_MIN_STEP_LONG, 2u);
    addPageOptionDecimal(pg, &i, "VyMin", &planningVyMinF, PLANNING_V_MIN_STEP_SHORT, PLANNING_V_MIN_STEP_LONG, 2u);
    addPageOptionDecimal(pg, &i, "WzMin", &planningWzMinF, PLANNING_WZ_MIN_STEP_SHORT, PLANNING_WZ_MIN_STEP_LONG, 2u);
    addPageOptionDecimal(pg, &i, "VxSlew", &planningVxSlewPerSF, PLANNING_SLEW_STEP_SHORT, PLANNING_SLEW_STEP_LONG, 1u);
    addPageOptionDecimal(pg, &i, "VySlew", &planningVySlewPerSF, PLANNING_SLEW_STEP_SHORT, PLANNING_SLEW_STEP_LONG, 1u);
    addPageOptionDecimal(pg, &i, "WzSlew", &planningWzSlewPerSF, PLANNING_SLEW_STEP_SHORT, PLANNING_SLEW_STEP_LONG, 1u);
    pg->optionCount = i;
}

/* Plan Shape: 固定中心已删除, 仅保留共享输出形状参数。 */
{
    menuPageStruct* pg = addPage("Plan Shape");
    uint8_t i = 0u;
    if (pg == 0) return;
    addPageOptionDecimal(pg, &i, "DZ Enter", &planningDeadzoneEnterF, PLANNING_DEADZONE_STEP_SHORT,
PLANNING_DEADZONE_STEP_LONG, 1u);
    addPageOptionDecimal(pg, &i, "DZ Exit", &planningDeadzoneExitF, PLANNING_DEADZONE_STEP_SHORT,
PLANNING_DEADZONE_STEP_LONG, 1u);
    pg->optionCount = i;
}

/* Plan Pose: 三来源中心永久由 Core0RawPose + 各自 offset 得到。 */
{
    menuPageStruct* pg = addPage("Plan Pose");
    uint8_t i = 0u;
    if (pg == 0) return;
    addPageOptionDecimal(pg, &i, "HeadConf", &planningCore0PoseMinConfidenceF, PLANNING_CORE0_POSE_CONF_STEP_SHORT,
PLANNING_CORE0_POSE_CONF_STEP_LONG, 2u);
    addPageOptionDecimal(pg, &i, "C1 PoseX", &c1Core0PoseOffsetXF, PLANNING_CORE0_POSE_OFFSET_STEP_SHORT,
PLANNING_CORE0_POSE_OFFSET_STEP_LONG, 1u);
    addPageOptionDecimal(pg, &i, "C1 PoseY", &c1Core0PoseOffsetYF, PLANNING_CORE0_POSE_OFFSET_STEP_SHORT,
PLANNING_CORE0_POSE_OFFSET_STEP_LONG, 1u);
    addPageOptionDecimal(pg, &i, "C3 PoseX", &core3Core0PoseOffsetXF, PLANNING_CORE0_POSE_OFFSET_STEP_SHORT,
PLANNING_CORE0_POSE_OFFSET_STEP_LONG, 1u);
    addPageOptionDecimal(pg, &i, "C3 PoseY", &core3Core0PoseOffsetYF, PLANNING_CORE0_POSE_OFFSET_STEP_SHORT,
PLANNING_CORE0_POSE_OFFSET_STEP_LONG, 1u);
    addPageOptionDecimal(pg, &i, "C4 PoseX", &core4Core0PoseOffsetXF, PLANNING_CORE0_POSE_OFFSET_STEP_SHORT,

```

```

PLANNING_CORE0_POSE_OFFSET_STEP_LONG, 1u);
    addPageOptionDecimal(pg, &i, "C4 PoseY", &core4Core0PoseOffsetYF, PLANNING_CORE0_POSE_OFFSET_STEP_SHORT,
PLANNING_CORE0_POSE_OFFSET_STEP_LONG, 1u);
    addPageOptionDecimal(pg, &i, "HeadOff", &planningCore0PoseHeadingOffsetRadF, PLANNING_CORE0_POSE_HEADING_STEP_SHORT,
PLANNING_CORE0_POSE_HEADING_STEP_LONG, 2u);
    addPageOption(pg, &i, "HeadSign", &planningCore0PoseHeadingSignF, PLANNING_SIGN_STEP, PLANNING_SIGN_STEP);
    addPageOption(pg, &i, "HoldMs", &poseHoldMsF, PLANNING_CORE0_POSE_HOLD_MS_STEP_SHORT,
PLANNING_CORE0_POSE_HOLD_MS_STEP_LONG);
    addPageOption(pg, &i, "C1 Fresh", &c1FreshGateMsF, SOURCE_FRESH_GATE_STEP_SHORT, SOURCE_FRESH_GATE_STEP_LONG);
    addPageOption(pg, &i, "C3 Fresh", &c3FreshGateMsF, SOURCE_FRESH_GATE_STEP_SHORT, SOURCE_FRESH_GATE_STEP_LONG);
    pg->optionCount = i;
}

/* 页 9: Plan Sign （离散极性 -1/+1， step 2 翻转） */
{
    menuPageStruct* pg = addPage("Plan Sign");
    uint8_t i = 0u;
    if (pg == 0) return;
    addPageOption(pg, &i, "SignVx", &planningVxSignF, PLANNING_SIGN_STEP, PLANNING_SIGN_STEP);
    addPageOption(pg, &i, "SignVy", &planningVySignF, PLANNING_SIGN_STEP, PLANNING_SIGN_STEP);
    addPageOption(pg, &i, "SearchSign", &planningWzSignF, PLANNING_SIGN_STEP, PLANNING_SIGN_STEP);
    addPageOption(pg, &i, "C3 SignVx", &core3PlanningVxSignF, PLANNING_SIGN_STEP, PLANNING_SIGN_STEP);
    addPageOption(pg, &i, "C3 SignVy", &core3PlanningVySignF, PLANNING_SIGN_STEP, PLANNING_SIGN_STEP);
    addPageOption(pg, &i, "C4 SignVx", &core4PlanningVxSignF, PLANNING_SIGN_STEP, PLANNING_SIGN_STEP);
    addPageOption(pg, &i, "C4 SignVy", &core4PlanningVySignF, PLANNING_SIGN_STEP, PLANNING_SIGN_STEP);
    pg->optionCount = i;
}

/* 页 10: Plan TD （输出跟踪微分器 R 三轴独立 + N 共享） */
{
    menuPageStruct* pg = addPage("Plan TD");
    uint8_t i = 0u;
    if (pg == 0) return;
    addPageOptionDecimal(pg, &i, "TD Rvx", &planningTdRvxF, PLANNING_TD_R_STEP_SHORT, PLANNING_TD_R_STEP_LONG, 2u);
    addPageOptionDecimal(pg, &i, "TD Rvy", &planningTdRvyF, PLANNING_TD_R_STEP_SHORT, PLANNING_TD_R_STEP_LONG, 2u);
    addPageOptionDecimal(pg, &i, "TD Rwz", &planningTdRwzF, PLANNING_TD_R_STEP_SHORT, PLANNING_TD_R_STEP_LONG, 2u);
    addPageOptionDecimal(pg, &i, "TD N", &planningTdNF, PLANNING_TD_N_STEP_SHORT, PLANNING_TD_N_STEP_LONG, 1u);
    addPageOption(pg, &i, "TD Vx", &planningTdVxEnableF, PLANNING_ENABLE_STEP, PLANNING_ENABLE_STEP);
    addPageOption(pg, &i, "TD Vy", &planningTdVyEnableF, PLANNING_ENABLE_STEP, PLANNING_ENABLE_STEP);
    addPageOption(pg, &i, "TD Wz", &planningTdWzEnableF, PLANNING_ENABLE_STEP, PLANNING_ENABLE_STEP);
    pg->optionCount = i;
}

/* 页 11: Plan Disp （派发总开关/搜索参数/启动运动门/派发超时） */
{
    menuPageStruct* pg = addPage("Plan Disp");
    uint8_t i = 0u;
    if (pg == 0) return;
    addPageOption(pg, &i, "Dispatch", &planningDispatchEnableF, PLANNING_ENABLE_STEP, PLANNING_ENABLE_STEP);
    addPageOption(pg, &i, "Search", &planningSearchEnableF, PLANNING_ENABLE_STEP, PLANNING_ENABLE_STEP);
    addPageOptionDecimal(pg, &i, "SearchWz", &planningSearchWzF, PLANNING_SEARCH_WZ_STEP_SHORT,
PLANNING_SEARCH_WZ_STEP_LONG, 2u);
    addPageOption(pg, &i, "SearchTo", &planningSearchTimeoutMsF, PLANNING_SEARCH_TIMEOUT_STEP_SHORT,
PLANNING_SEARCH_TIMEOUT_STEP_LONG);
    addPageOption(pg, &i, "Timeout", &planningPlanTimeoutMsF, PLANNING_PLAN_TIMEOUT_STEP_SHORT,
PLANNING_PLAN_TIMEOUT_STEP_LONG);
    addPageOption(pg, &i, "UnlockMs", &planningPostTakeoffDelayMsF,
    PLANNING_POST_TAKEOFF_DELAY_STEP_SHORT,
    PLANNING_POST_TAKEOFF_DELAY_STEP_LONG);
    addPageOptionDecimal(pg, &i, "FirstVx",
    &planningFirstBeaconForwardSpeedF,
    PLANNING_FIRST_BEACON_FORWARD_SPEED_STEP_SHORT,
    PLANNING_FIRST_BEACON_FORWARD_SPEED_STEP_LONG, 2u);
    addPageOption(pg, &i, "FirstGo",
    &planningFirstBeaconForwardEnableF,
    PLANNING_ENABLE_STEP, PLANNING_ENABLE_STEP);
    pg->optionCount = i;
}

/* Core3 独立标定/观察超时。 */
{
    menuPageStruct* pg = addPage("C3 Profile");
    uint8_t i = 0u;
    if (pg == 0) return;
    addPageOption(pg, &i, "MaxB", &maxBeaconsF, 1.0f, 4.0f);
    addPageOption(pg, &i, "Timeout", &core3ObservationTimeoutMsF,
    CORE3_OBSERVATION_TIMEOUT_STEP_SHORT, CORE3_OBSERVATION_TIMEOUT_STEP_LONG);
    addPageOption(pg, &i, "C3 CenterX", &core3CenterXF,
    CORE3_CALIBRATION_CENTER_STEP_SHORT, CORE3_CALIBRATION_CENTER_STEP_LONG);
}

```

```

        addPageOption(pg, &i, "C3 CenterY", &core3CenterYF,
            CORE3_CALIBRATION_CENTER_STEP_SHORT, CORE3_CALIBRATION_CENTER_STEP_LONG);
        addPageOptionDecimal(pg, &i, "C3 Scale", &core3ScaleF,
            CORE3_CALIBRATION_SCALE_STEP_SHORT, CORE3_CALIBRATION_SCALE_STEP_LONG, 2u);
        addPageOptionDecimal(pg, &i, "C3 Yaw", &core3MountYawRadF,
            CORE3_CALIBRATION_YAW_STEP_SHORT, CORE3_CALIBRATION_YAW_STEP_LONG, 2u);
        addPageOption(pg, &i, "C3 Mirror", &core3MirrorXF, 1.0f, 1.0f);
        addPageOption(pg, &i, "C3 FSign", &core3ErrorFwdSignF, 2.0f, 2.0f);
        addPageOption(pg, &i, "C3 RSign", &core3ErrorRightSignF, 2.0f, 2.0f);
        pg->optionCount = i;
    }

/* Core4 独立标定/新鲜度，不能复用 Core3 工作副本。 */
{
    menuPageStruct* pg = addPage("C4 Profile");
    uint8_t i = 0u;
    if (pg == 0) return;
    addPageOption(pg, &i, "Fresh", &c4FreshGateMsF,
        SOURCE_FRESH_GATE_STEP_SHORT, SOURCE_FRESH_GATE_STEP_LONG);
    addPageOption(pg, &i, "Timeout", &c4ObservationTimeoutMsF,
        SOURCE_OBSERVATION_TIMEOUT_STEP_SHORT, SOURCE_OBSERVATION_TIMEOUT_STEP_LONG);
    addPageOption(pg, &i, "CenterX", &core4CenterXF,
        CORE3_CALIBRATION_CENTER_STEP_SHORT, CORE3_CALIBRATION_CENTER_STEP_LONG);
    addPageOption(pg, &i, "CenterY", &core4CenterYF,
        CORE3_CALIBRATION_CENTER_STEP_SHORT, CORE3_CALIBRATION_CENTER_STEP_LONG);
    addPageOptionDecimal(pg, &i, "Scale", &core4ScaleF,
        CORE3_CALIBRATION_SCALE_STEP_SHORT, CORE3_CALIBRATION_SCALE_STEP_LONG, 2u);
    addPageOptionDecimal(pg, &i, "Yaw", &core4MountYawRadF,
        CORE3_CALIBRATION_YAW_STEP_SHORT, CORE3_CALIBRATION_YAW_STEP_LONG, 2u);
    addPageOption(pg, &i, "Mirror", &core4MirrorXF, 1.0f, 1.0f);
    addPageOption(pg, &i, "FSign", &core4ErrorFwdSignF, 2.0f, 2.0f);
    addPageOption(pg, &i, "RSign", &core4ErrorRightSignF, 2.0f, 2.0f);
    pg->optionCount = i;
}

paramSlotsPageIndex = menuPageCount;
paramSlotSelection = 1.0f;
{
    menuPageStruct* pg = addPage("Param Slots");
    uint8_t i = 0u;
    if (pg == 0) return;
    addPageOption(pg, &i, "Slot", &paramSlotSelection, 1.0f, 1.0f);
    pg->optionCount = i;
}
}

```

core1_cyt4bb7/project/code/app/menuRender.c

SHA-256

:

d38857e296559460e1aea206ed62f49760951c962863d9480d501bc89fe135b3

代码 (c)

```

#include "menu.h"
#include "menuInternal.h"

#include <math.h>
#include <stdarg.h>
#include <stdio.h>
#include <string.h>

#include "zf_device_ips114.h"
#include "zf_device_mt9v03x.h"
#include "zf_common_font.h"

#include "image.h"
#include "beaconTrack.h"
#include "beaconNavigator.h"
#include "core3LinkCore1.h"
#include "core4LinkCore1.h"
#include "core1KvmRouter.h"
#include "core1Loop.h"
#include "menuKeyRemote.h"
#include "menuFloatDisplayInternal.h"
#include "uartLinkProto.h"

```

```
#include "visionParams.h"
```

```
/* 三态列表行配色（统一规范 docs/MENU_UI_CONVENTION.md §3： 名/值始终不同前景色）：
```

```
* 普通 底黑/名绿/值白；选中 底青/名黑/值蓝；编辑 底黄/名黑/值红 */
```

```
#define MENU_NORM_NAME_FG (RGB565_GREEN)
```

```
#define MENU_NORM_VAL_FG (RGB565_WHITE)
```

```
#define MENU_SEL_BG (RGB565_CYAN)
```

```
#define MENU_SEL_NAME_FG (RGB565_BLACK)
```

```
#define MENU_SEL_VAL_FG (RGB565_BLUE)
```

```
#define MENU_EDIT_BG (RGB565_YELLOW)
```

```
#define MENU_EDIT_NAME_FG (RGB565_BLACK)
```

```
#define MENU_EDIT_VAL_FG (RGB565_RED)
```

```
#define MENU_PARAM_VALUE_X (COL_X(14))
```

```
#define MENU_MARK_X (COL_X(1))
```

```
#define MENU_ITEM_X (COL_X(2))
```

```
#define THRESH_SLOT_LOW_X (2u)
```

```
#define THRESH_SLOT_HIGH_X (82u)
```

```
#define THRESH_SLOT_EXPOSURE_X (162u)
```

```
#define THRESH_SLOT_LOW_WIDTH (72u)
```

```
#define THRESH_SLOT_HIGH_WIDTH (72u)
```

```
#define THRESH_SLOT_EXPOSURE_WIDTH (76u)
```

```
#define THRESH_NAME_LOW_X (10u)
```

```
#define THRESH_NAME_HIGH_X (90u)
```

```
#define THRESH_NAME_EXPOSURE_X (170u)
```

```
#define THRESH_VALUE_LOW_X (42u)
```

```
#define THRESH_VALUE_HIGH_X (122u)
```

```
#define THRESH_VALUE_EXPOSURE_X (202u)
```

```
/*
```

```
* core1 menuRender.c — IPS114 8×16 渲染。移植自 core0, 适配:
```

```
* - 相机 188×120, 缩放宏按本核分辨率重算
```

```
* - 发车页显示信标灰度图视口 + 右侧 HUD + 左下来源/误差三行状态(非 core0 飞控数据)
```

```
* - 阈值编辑器双图: 原图 + BinaryCC mask(imageGetFrameView), Lo/Hi/Exp 热调
```

```
* - 分割线/边框颜色按操控权: 被控=绿 未控=红(menuBorderColor)
```

```
* 仅消费状态不修改。8×16 字模占 y0..15, 横线避开字模行防闪烁。
```

```
*/
```

```
/* 发车页布局: 左图像块 + 右文字区。动态文字定宽覆盖, 避免整块清黑闪烁。 */
```

```
#define HOME_IMG_X (2u)
```

```
#define HOME_IMG_Y (17u)
```

```
#define HOME_IMG_W (130u)
```

```
#define HOME_CAM_X (HOME_IMG_X)
```

```
#define HOME_CAM_Y (HOME_IMG_Y)
```

```
#define HOME_CAM_W (HOME_IMG_W)
```

```
#define HOME_CAM_H (67u)
```

```
#define HOME_TXT_X (135u)
```

```
#define HOME_VLINE_X (133u)
```

```
#define HOME_SLOT_COUNT (13u)
```

```
#define HOME_SLOT_CHARS (39u)
```

```
static char homeSlotCache[HOME_SLOT_COUNT][HOME_SLOT_CHARS + 1u];
```

```
static uint8_t homeSlotValid[HOME_SLOT_COUNT];
```

```
static uint16_t homeSlotColor[HOME_SLOT_COUNT];
```

```
static uint16_t homeSlotBackground[HOME_SLOT_COUNT];
```

```
static uint16_t homeTitleColor = 0xFFFFu;
```

```
static uint16_t homeLayoutColor = 0xFFFFu;
```

```
#define MENU_SOLID_FILL_PIXELS (240u)
```

```
static uint16_t menuSolidFillBuffer[MENU_SOLID_FILL_PIXELS];
```

```
static void fillSolidRect(uint16_t x, uint16_t y, uint16_t w, uint16_t h,
```

```
uint16_t color)
```

```
{
```

```
uint16_t rowsPerTransfer;
```

```
uint16_t rows;
```

```
uint16_t i;
```

```
if ((w == 0u) || (h == 0u) || (x >= LCD_W) || (y >= LCD_H)) return;
```

```
if ((uint32_t) x + w > LCD_W) w = (uint16_t) (LCD_W - x);
```

```
if ((uint32_t) y + h > LCD_H) h = (uint16_t) (LCD_H - y);
```

```
if (w > MENU_SOLID_FILL_PIXELS) return;
```

```
rowsPerTransfer = (uint16_t) (MENU_SOLID_FILL_PIXELS / w);
```

```
for (i = 0u; i < MENU_SOLID_FILL_PIXELS; i++) menuSolidFillBuffer[i] = color;
```

```
while (h > 0u)
```

```
{
```

```
rows = (h < rowsPerTransfer) ? h : rowsPerTransfer;
```

```
ips114_show_rgb565_image(x, y, menuSolidFillBuffer,
```

```
w, rows, w, rows, 0u);
```

```
y = (uint16_t) (y + rows);
```

```
h = (uint16_t) (h - rows);
```

```

    }
}

static void drawHomeSlot(uint8_t slot, uint16_t x, uint16_t y,
    uint8_t width, uint16_t color, uint16_t background, const char *format, ...)
{
    char line[HOME_SLOT_CHARS + 1u];
    char formatted[48];
    size_t length;
    va_list args;
    if ((slot >= HOME_SLOT_COUNT) || (width > HOME_SLOT_CHARS)) return;
    va_start(args, format);
    (void) vsnprintf(formatted, sizeof(formatted), format, args);
    va_end(args);
    memset(line, 0, sizeof(line));
    memset(line, ' ', width);
    line[width] = '\0';
    length = strlen(formatted);
    if (length > width) length = width;
    memcpy(line, formatted, length);
    if ((homeSlotValid[slot] != 0u) &&
        (homeSlotColor[slot] == color) &&
        (homeSlotBackground[slot] == background) &&
        (memcmp(homeSlotCache[slot], line, sizeof(line)) == 0)) return;
    memcpy(homeSlotCache[slot], line, sizeof(line));
    homeSlotValid[slot] = 1u;
    homeSlotColor[slot] = color;
    homeSlotBackground[slot] = background;
    ips114_set_font(IPS114_6X8_FONT);
    ips114_set_color(color, background);
    ips114_show_string(x, y, line);
}

static inline uint16_t homeMapX(uint32_t imgX)
{
    uint32_t v = (uint32_t) HOME_CAM_X + imgX * HOME_CAM_W / (uint32_t) MT9V03X_W;
    uint32_t xMax = (uint32_t) HOME_CAM_X + HOME_CAM_W - 1u;
    if (v > xMax) v = xMax;
    return (uint16_t) v;
}

static inline uint16_t homeMapY(uint32_t imgY)
{
    uint32_t v = (uint32_t) HOME_CAM_Y + imgY * HOME_CAM_H / (uint32_t) MT9V03X_H;
    uint32_t yMax = (uint32_t) HOME_CAM_Y + HOME_CAM_H - 1u;
    if (v > yMax) v = yMax;
    return (uint16_t) v;
}

static uint16_t homeMapFloatX(float imgX)
{
    if (imgX < 0.0f) imgX = 0.0f;
    else if (imgX > (float) (MT9V03X_W - 1u)) imgX = (float) (MT9V03X_W - 1u);
    return homeMapX((uint32_t) (imgX + 0.5f));
}

static uint16_t homeMapFloatY(float imgY)
{
    if (imgY < 0.0f) imgY = 0.0f;
    else if (imgY > (float) (MT9V03X_H - 1u)) imgY = (float) (MT9V03X_H - 1u);
    return homeMapY((uint32_t) (imgY + 0.5f));
}

static uint8_t homeImagePointVisible(float imgX, float imgY)
{
    return (uint8_t) (isfinite(imgX) && isfinite(imgY) &&
        (imgX >= 0.0f) && (imgX <= (float) (MT9V03X_W - 1u)) &&
        (imgY >= 0.0f) && (imgY <= (float) (MT9V03X_H - 1u)));
}

static int16_t homeRadiusToDisp(float radiusPx)
{
    if (radiusPx <= 0.0f) return 0;
    return (int16_t) (radiusPx * (float) HOME_CAM_H / (float) MT9V03X_H + 0.5f);
}

/* 外边框(四边), 颜色由操控权决定。坐标落屏内, 不与图像视口/HUD 冲突。 */
static void drawFrameBorder(uint16_t color)
{
    fillSolidRect(0u, 0u, LCD_W, 1u, color);
}

```

```

        fillSolidRect(0u, (uint16_t) (LCD_H - 1u), LCD_W, 1u, color);
        fillSolidRect(0u, 1u, 1u, (uint16_t) (LCD_H - 2u), color);
        fillSolidRect((uint16_t) (LCD_W - 1u), 1u, 1u,
            (uint16_t) (LCD_H - 2u), color);
    }

static void fillRect(uint16_t x, uint16_t y, uint16_t w, uint16_t h, uint16_t color)
{
    fillSolidRect(x, y, w, h, color);
}

static void drawTitleBar(const char* title, uint16_t fg, uint16_t bg)
{
    char bounded[40];
    size_t len = strlen(title);
    uint16_t x;

    if (len > 39u) len = 39u;
    memcpy(bounded, title, len);
    bounded[len] = '\0';
    x = (uint16_t) ((LCD_W - (uint16_t) (len * 6u)) / 2u);
    if (x < 2u) x = 2u;
    fillSolidRect(1u, 1u, (uint16_t) (LCD_W - 2u), 15u, bg);
    ips114_set_font(IPS114_6X8_FONT);
    ips114_set_color(fg, bg);
    ips114_show_string(x, 4u, bounded);
    ips114_set_font(IPS114_8X16_FONT);
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
}

static void drawScrollbar(uint8_t total, uint8_t top, uint16_t color)
{
    uint16_t trackTop = ROW_Y(1);
    uint16_t trackBot = (uint16_t) (LCD_H - 2u);
    uint16_t trackH;
    uint16_t thumbH;
    uint16_t thumbY;

    if (total <= VISIBLE_ROWS) return;
    trackH = (uint16_t) (trackBot - trackTop);

    /* 自清槽：先清整个槽区(轨道 + 旧滑块) → 滚动时旧滑块横线残留。不触右外框(x=LCD_W-1)。 */
    fillSolidRect(SCROLLBAR_X, trackTop,
        (uint16_t) (LCD_W - 2u) - SCROLLBAR_X + 1u,
        (uint16_t) (trackBot - trackTop + 1u), RGB565_BLACK);
    fillSolidRect(SCROLLBAR_X, trackTop, 1u,
        (uint16_t) (trackBot - trackTop + 1u), color);
    fillSolidRect((uint16_t) (LCD_W - 2u), trackTop, 1u,
        (uint16_t) (trackBot - trackTop + 1u), color);

    thumbH = (uint16_t) ((uint32_t) trackH * VISIBLE_ROWS / total);
    if (thumbH < 6u) thumbH = 6u;
    thumbY = (uint16_t) (trackTop + (uint32_t) (trackH - thumbH) * top / (uint16_t) (total - VISIBLE_ROWS));

    if ((uint16_t) (thumbY + thumbH) > trackBot)
        thumbH = (uint16_t) (trackBot - thumbY);
    fillSolidRect((uint16_t) (SCROLLBAR_X + 1u), thumbY,
        (uint16_t) (LCD_W - 3u) - (SCROLLBAR_X + 1u) + 1u, thumbH, color);
}

/* 在 (sx,sy) 画十字, 四端钳到 HOME camera viewport 内。 */
static void drawCross(uint16_t sx, uint16_t sy, uint16_t half, uint16_t color)
{
    uint16_t xMin = HOME_CAM_X;
    uint16_t yMin = HOME_CAM_Y;
    uint16_t xMax = (uint16_t) (HOME_CAM_X + HOME_CAM_W - 1u);
    uint16_t yMax = (uint16_t) (HOME_CAM_Y + HOME_CAM_H - 1u);
    uint16_t x0 = (sx > (uint16_t) (xMin + half)) ? (uint16_t) (sx - half) : xMin;
    uint16_t x1 = (sx + half < xMax) ? (uint16_t) (sx + half) : xMax;
    uint16_t y0 = (sy > (uint16_t) (yMin + half)) ? (uint16_t) (sy - half) : yMin;
    uint16_t y1 = (sy + half < yMax) ? (uint16_t) (sy + half) : yMax;
    fillSolidRect(x0, sy, (uint16_t) (x1 - x0 + 1u), 1u, color);
    fillSolidRect(sx, y0, 1u, (uint16_t) (y1 - y0 + 1u), color);
}

static uint16_t clampHomeX(int16_t x)
{
    int16_t xMin = (int16_t) HOME_CAM_X;
    int16_t xMax = (int16_t) (HOME_CAM_X + HOME_CAM_W - 1u);

```

```

    if (x < xMin) x = xMin;
    if (x > xMax) x = xMax;
    return (uint16_t) x;
}

static uint16_t clampHomeY(int16_t y)
{
    int16_t yMin = (int16_t) HOME_CAM_Y;
    int16_t yMax = (int16_t) (HOME_CAM_Y + HOME_CAM_H - 1u);
    if (y < yMin) y = yMin;
    if (y > yMax) y = yMax;
    return (uint16_t) y;
}

static void drawCircleClamped(int16_t cx, int16_t cy, int16_t r, uint16_t color)
{
    const int16_t xMin = (int16_t) HOME_CAM_X;
    const int16_t yMin = (int16_t) HOME_CAM_Y;
    const int16_t xMax = (int16_t) (HOME_CAM_X + HOME_CAM_W - 1u);
    const int16_t yMax = (int16_t) (HOME_CAM_Y + HOME_CAM_H - 1u);
    int16_t x = r;
    int16_t y = 0;
    int16_t err = 1 - r;

    if (r <= 0) return;

    while (x >= y)
    {
        int16_t px[8] = {
            (int16_t) (cx + x), (int16_t) (cx + y), (int16_t) (cx - y), (int16_t) (cx - x),
            (int16_t) (cx - x), (int16_t) (cx - y), (int16_t) (cx + y), (int16_t) (cx + x)
        };
        int16_t py[8] = {
            (int16_t) (cy + y), (int16_t) (cy + x), (int16_t) (cy - x), (int16_t) (cy - y),
            (int16_t) (cy - y), (int16_t) (cy - x), (int16_t) (cy + x), (int16_t) (cy + y)
        };
        for (uint8_t i = 0u; i < 8u; i++)
        {
            if (px[i] >= xMin && px[i] <= xMax && py[i] >= yMin && py[i] <= yMax)
            {
                ips114_draw_point((uint16_t) px[i], (uint16_t) py[i], color);
            }
        }
        y++;
        if (err < 0)
        {
            err = (int16_t) (err + 2 * y + 1);
        }
        else
        {
            x--;
            err = (int16_t) (err + 2 * (y - x + 1));
        }
    }
}

/* =====
 * 等待链路页(上电门控): 等 core0 链路在线
 * ===== */
void renderWaitLink(void)
{
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
    menuRenderResetListSnapshot(); /* 启动 surface 复用; 只复位后续 HOME/list 的 dirty 快照。 */
    drawTitleBar("WAIT LINK", RGB565_BLACK, RGB565_RED);
    ips114_set_color(RGB565_RED, RGB565_BLACK);
    ips114_show_string(COL_X(4), ROW_Y(3), "Waiting core0 link");
    ips114_show_string(COL_X(7), ROW_Y(5), "(no control)");
    drawFrameBorder(RGB565_YELLOW); /* NONE/待连接不冒充操控权, 统一黄框 */
}

/* 规划态 → 3 字符缩写(底行观测显示用)。 */
static const char* planStateStr(uint8_t s)
{
    switch (s)
    {
        {
            case (uint8_t) BEACON_NAV_TRACK:    return "TRK";
            case (uint8_t) BEACON_NAV_STARTUP_FORWARD: return "1ST";
            case (uint8_t) BEACON_NAV_SEARCH:    return "SRC";
            default:                             return "IDL";
        }
    }
}

```

```

    }
}

static void drawHomeImage(void)
{
    imageFrameViewStruct view;
    if (imageGetFrameView(IMAGE_FRAME_ORIGINAL, &view) && (view.pixels != 0))
    {
        ips114_show_gray_image(HOME_CAM_X, HOME_CAM_Y, view.pixels,
                               view.width, view.height, HOME_CAM_W, HOME_CAM_H, 0u);
    }
    else
    {
        fillRect(HOME_CAM_X, HOME_CAM_Y, HOME_CAM_W, HOME_CAM_H, RGB565_BLACK);
    }
}

static void drawHomeTrackBorder(const beaconTrackBorderMarginsStruct *margin, uint16_t color)
{
    uint16_t left;
    uint16_t right;
    uint16_t top;
    uint16_t bottom;

    if (margin == 0) return;
    left = homeMapX(margin->left);
    right = homeMapX((uint32_t) (MT9V03X_W - 1u - margin->right));
    top = homeMapY(margin->top);
    bottom = homeMapY((uint32_t) (MT9V03X_H - 1u - margin->bottom));
    if (margin->top != 0u) ips114_draw_line(left, top, right, top, color);
    if (margin->bottom != 0u) ips114_draw_line(left, bottom, right, bottom, color);
    if (margin->left != 0u) ips114_draw_line(left, top, left, bottom, color);
    if (margin->right != 0u) ips114_draw_line(right, top, right, bottom, color);
}

static void drawHomeEdgePredictionBand(const beaconTrackEdgeBandsStruct *bands)
{
    uint16_t left = homeMapX(0u);
    uint16_t right = homeMapX(MT9V03X_W - 1u);
    uint16_t top = homeMapY(0u);
    uint16_t bottom = homeMapY(MT9V03X_H - 1u);
    if (bands == 0) return;
    if (bands->left != 0u)
        ips114_draw_line(homeMapX(bands->left), top, homeMapX(bands->left), bottom,
                        RGB565_MAGENTA);
    if (bands->right != 0u)
        ips114_draw_line(homeMapX(MT9V03X_W - bands->right), top,
                        homeMapX(MT9V03X_W - bands->right), bottom, RGB565_MAGENTA);
    if (bands->top != 0u)
        ips114_draw_line(left, homeMapY(bands->top), right, homeMapY(bands->top),
                        RGB565_MAGENTA);
    if (bands->bottom != 0u)
        ips114_draw_line(left, homeMapY(MT9V03X_H - bands->bottom), right,
                        homeMapY(MT9V03X_H - bands->bottom), RGB565_MAGENTA);
}

static void drawHomeTrackBorders(void)
{
    visionParamsStruct params;
    beaconTrackBorderConfigStruct border;
    beaconTrackEdgeBandsStruct bands;

    visionParamsGetActive(&params);
    border.enter.left = params.c1BorderEnterLeft;
    border.enter.right = params.c1BorderEnterRight;
    border.enter.top = params.c1BorderEnterTop;
    border.enter.bottom = params.c1BorderEnterBottom;
    border.exit.left = params.c1BorderExitLeft;
    border.exit.right = params.c1BorderExitRight;
    border.exit.top = params.c1BorderExitTop;
    border.exit.bottom = params.c1BorderExitBottom;
    drawHomeTrackBorder(&border.exit, RGB565_YELLOW);
    drawHomeTrackBorder(&border.enter, RGB565_CYAN);
    bands.left = params.c1EdgeBandLeft;
    bands.right = params.c1EdgeBandRight;
    bands.top = params.c1EdgeBandTop;
    bands.bottom = params.c1EdgeBandBottom;
    drawHomeEdgePredictionBand(&bands);
}

```



```

static int32_t homeErrorToInt(float value)
{
    int32_t out = (value >= 0.0f) ? (int32_t) (value + 0.5f) : (int32_t) (value - 0.5f);
    if (out > 999) out = 999;
    else if (out < -999) out = -999;
    return out;
}

void renderInitFaultSummary(uint8_t failedMask, uint8_t unverifiedMask, uint8_t secondsLeft)
{
    if (secondsLeft >= 3u)
    {
        ips114_clear();
        menuRenderResetListSnapshot();
        drawTitleBar((failedMask != 0u) ? "HW INIT FAIL" : "HW INIT CHECK",
            RGB565_BLACK, (failedMask != 0u) ? RGB565_RED : RGB565_YELLOW);
        ips114_set_color((failedMask != 0u) ? RGB565_RED : RGB565_YELLOW, RGB565_BLACK);
        ips114_show_string(8u, 34u, "CAMERA");
        ips114_show_string(128u, 34u,
            (failedMask != 0u) ? "FAIL" : ((unverifiedMask != 0u) ? "CHECK" : "OK  "));
        drawFrameBorder((failedMask != 0u) ? RGB565_RED : RGB565_YELLOW);
    }
    ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
    ips114_show_string(8u, 114u, "CONTINUE 0s");
    ips114_show_uint(80u, 114u, secondsLeft, 1u);
}

static uint8_t beaconIsSelectedTrack(const beaconConfirmedStruct* bea,
    const core1RuntimeDisplayTargetSnapshotStruct* target)
{
    if ((bea == 0) || (target == 0) || (target->valid == 0u) ||
        (target->sourceId != BEACON_SOURCE_CORE1) ||
        (target->trackId == 0u)) return 0u;
    return (bea->trackId == target->trackId) ? 1u : 0u;
}

static void drawHomeBeacons(void)
{
    imageObservationSnapshotStruct observation;
    core1RuntimeDisplayTargetSnapshotStruct target;
    uint8_t skippedSelected = 0u;
    imageGetObservationSnapshot(&observation);
    core1RuntimeGetDisplayTargetSnapshot(&target);

    if (observation.valid != 0u)
    {
        for (uint16_t i = 0u; i < observation.count; i++)
        {
            if ((observation.tracks[i].flags &
                BEACON_CONFIRMED_FLAG_BORDER_REJECTED) != 0u) continue;
            if (!skippedSelected && beaconIsSelectedTrack(&observation.tracks[i],
                &target))
            {
                skippedSelected = 1u;
                continue;
            }
            drawCross(homeMapX(observation.tracks[i].centerX),
                homeMapY(observation.tracks[i].centerY), 4u, RGB565_YELLOW);
        }
    }

    /* HOME 只显示 Core1 本地图像坐标； C3/C4 目标不做伪像素投影。 */
    if ((target.valid == 0u) || (target.sourceId != BEACON_SOURCE_CORE1) ||
        (target.size == 0u)) return;
    drawCross(homeMapX(target.controlX), homeMapY(target.controlY), 5u,
        (target.holding != 0u) ? RGB565_MAGENTA : RGB565_RED);
}

static void drawHomePlanningDeadzone(const beaconNavigatorSnapshotStruct* snap)
{
    visionParamsStruct params;
    float centerX;
    float centerY;
    if (snap == 0) return;
    visionParamsGetActive(&params);
    centerX = snap->activeCenterX;
    centerY = snap->activeCenterY;
    if (homeImagePointVisible(centerX, centerY) == 0u) return;

```

```

uint16_t cx = homeMapFloatX(centerX);
uint16_t cy = homeMapFloatY(centerY);
int16_t enterR = homeRadiusToDisp(params.planningDeadzoneEnterPx);
int16_t exitR = homeRadiusToDisp(params.planningDeadzoneExitPx);

drawCross(cx, cy, 4u, RGB565_CYAN);
drawCircleClamped((int16_t) cx, (int16_t) cy, enterR, RGB565_YELLOW);
drawCircleClamped((int16_t) cx, (int16_t) cy, exitR, RGB565_MAGENTA);
}

static void drawHomeCore0Pose(const beaconNavigatorSnapshotStruct* snap)
{
    visionParamsStruct params;
    float posX;
    float posY;
    uint16_t color;
    uint16_t headingColor;
    uint16_t sx;
    uint16_t sy;
    int16_t ex;
    int16_t ey;
    const float lineLen = 12.0f;
    if ((snap == 0) || (snap->poseValid == 0u)) return;

    visionParamsGetActive(&params);
    posX = snap->poseRawX + params.c1Core0PoseOffsetX;
    posY = snap->poseRawY + params.c1Core0PoseOffsetY;
    if (homeImagePointVisible(posX, posY) == 0u) return;

    color = snap->poseHolding ? RGB565_BLUE : RGB565_GREEN;
    sx = homeMapFloatX(posX);
    sy = homeMapFloatY(posY);
    drawCross(sx, sy, 6u, color);
    if (snap->poseHeadingValid)
    {
        headingColor = snap->poseHeadingHolding ? RGB565_BLUE : RGB565_GREEN;
        ex = (int16_t)((float) sx + cosf(snap->poseHeadingRad) * lineLen + 0.5f);
        ey = (int16_t)((float) sy + sinf(snap->poseHeadingRad) * lineLen + 0.5f);
        ips114_draw_line(sx, sy, clampHomeX(ex), clampHomeY(ey), headingColor);
    }
}

static void drawHomeHud(const beaconNavigatorSnapshotStruct* snap)
{
    core3LinkCore1Status core3;
    core4LinkCore1Status core4;
    core1KvmRouterSnapshot kvm;
    core1RuntimePlanSnapshotStruct plan;
    uint8_t c0Up = menuKeyRemoteLinkUp();
    uint8_t c2Lost = menuKeyRemoteCore2LinkLost();
    uint8_t state = (snap != 0) ? snap->state : (uint8_t) BEACON_NAV_IDLE;
    const char *source;
    uint16_t title;
    uint8_t observersReady;
    const char *controlClass;

    core3LinkCore1GetStatus(&core3);
    core4LinkCore1GetStatus(&core4);
    core1KvmRouterGetSnapshot(&kvm);
    core1RuntimeGetPlanSnapshot(&plan);
    source = (snap != NULL && snap->sourceId == 4u) ? "C4" :
        ((snap != NULL && snap->sourceId == 3u) ? "C3" :
        ((snap != NULL && snap->sourceId == 1u) ? "C1" : "--"));
    observersReady = (uint8_t) (core3.online && core4.online &&
        core3.configCompatible && core4.configCompatible);
    controlClass = (state == (uint8_t) BEACON_NAV_TRACK) ? "TRACK" :
        ((state == (uint8_t) BEACON_NAV_SEARCH) ? "SEARCH" : "NONE");
    title = (!c0Up || c2Lost) ? RGB565_RED :
        ((plan.systemPhase != OBSERVER_LINK_SYSTEM_PHASE_HOME) ? RGB565_YELLOW :
        ((plan.barrier || !observersReady) ?
            RGB565_YELLOW : RGB565_GREEN));
    if (homeTitleColor != title)
    {
        drawTitleBar("CORE1 PLANNER", RGB565_BLACK, title);
        homeTitleColor = title;
    }

    drawHomeSlot(0u, HOME_TXT_X, 18u, 17u, RGB565_CYAN, RGB565_BLACK,
        "VISION:ALL SRC:%s", source);
}

```

```

drawHomeSlot(1u, HOME_TXT_X, 26u, 17u,
(c0Up && !c2Lost) ? RGB565_GREEN : RGB565_RED, RGB565_BLACK,
"C0:%s C2:%s", c0Up ? "OK" : "LOST", c2Lost ? "LOST" : "OK");
drawHomeSlot(2u, HOME_TXT_X, 34u, 17u,
(core3.online && core3.configCompatible) ? RGB565_GREEN : RGB565_YELLOW,
RGB565_BLACK, "C3:%s CFG:%s", core3.online ? "OK" : "LOST",
core3.configCompatible ? "OK" : "SYNC");
drawHomeSlot(3u, HOME_TXT_X, 42u, 17u, RGB565_WHITE, RGB565_BLACK,
"NAV:%s CLS:%s", planStateStr(state),
controlClass);
drawHomeSlot(4u, HOME_TXT_X, 50u, 17u,
plan.barrier ? RGB565_YELLOW : RGB565_GREEN, RGB565_BLACK,
"BAR:%c DSP:%c", plan.barrier ? 'Y' : 'N',
plan.dispatchEnable ? 'Y' : 'N');
drawHomeSlot(5u, HOME_TXT_X, 58u, 17u,
(core4.online && core4.configCompatible) ? RGB565_GREEN : RGB565_YELLOW,
RGB565_BLACK, "C4:%s CFG:%s", core4.online ? "OK" : "LOST",
core4.configCompatible ? "OK" : "SYNC");
drawHomeSlot(6u, HOME_TXT_X, 66u, 17u, RGB565_WHITE, RGB565_BLACK,
"EX:%ld", (long) ((snap != 0) ? homeErrorToInt(snap->errX) : 0));
drawHomeSlot(7u, HOME_TXT_X, 74u, 17u, RGB565_WHITE, RGB565_BLACK,
"EY:%ld", (long) ((snap != 0) ? homeErrorToInt(snap->errY) : 0));
drawHomeSlot(8u, HOME_TXT_X, 82u, 17u, RGB565_WHITE, RGB565_BLACK,
"VX:%+.1f VY:%+.1f", (double) ((snap != 0) ? snap->vx : 0.0f),
(double) ((snap != 0) ? snap->vy : 0.0f));
drawHomeSlot(9u, HOME_TXT_X, 90u, 17u, RGB565_WHITE, RGB565_BLACK,
"WZ:%+.1f", (double) ((snap != 0) ? snap->wz : 0.0f));
drawHomeSlot(11u, 2u, 106u, 39u,
(!plan.dispatchEnable && plan.valid) ? RGB565_YELLOW : RGB565_GREEN,
RGB565_BLACK, "DSP:%s TARGET:%s KEY:%u>%u",
plan.dispatchEnable ? "ON" : "LOCK",
(snap != 0 && snap->valid) ? "VALID" : "LOST",
kvm.desiredTarget, kvm.inputOwner);
drawHomeSlot(12u, 2u, 121u, 39u,
(!c0Up || c2Lost) ? RGB565_RED : RGB565_GRAY, RGB565_BLACK,
(!c0Up) ? "BLOCK: CORE0 LINK" : (c2Lost ? "BLOCK: CORE2 LINK" :
((plan.systemPhase == OBSERVER_LINK_SYSTEM_PHASE_REJOIN) ?
"REJOIN: VERIFY + ZERO BARRIER" :
(plan.standalone ? "STANDALONE: NO DISPATCH" : "K1/KVM MENU READY"))));
}

static void drawHomeLayoutLines(uint16_t color)
{
    if (homeLayoutColor == color) return;
    fillSolidRect(HOME_VLINE_X, 17u, 1u, 87u, color);
    fillSolidRect(1u, 104u, 238u, 1u, RGB565_GRAY);
    fillSolidRect(1u, 119u, 238u, 1u, RGB565_GRAY);
    drawFrameBorder(color);
    homeLayoutColor = color;
}

/* =====
* 发车页: 左图像视口 + 右文字区 + 操控权边框
* ===== */
void renderHome(uint8_t visualDue)
{
    beaconNavigatorSnapshotStruct snap;
    beaconNavigatorGetSnapshot(&snap);

    if (visualDue != 0u)
    {
        drawHomeImage();
        drawHomeTrackBorders();
        drawHomePlanningDeadzone(&snap);
        drawHomeBeacons();
        drawHomeCore0Pose(&snap);
    }
    drawHomeHud(&snap);
    drawHomeLayoutLines(menuBorderColor());

    /* 边框与静态分隔线仅在操控权颜色变化时重绘。 */
}

/* =====
* 阈值编辑器: 左原图 + 右检测预览(Binary mask 或 Canny edge) + 底部 Lo/Hi/Exp 参数行
* ===== */
typedef struct
{
    uint32_t originalFrameSeq;

```

```

uint32_t previewFrameSeq;
int32_t lowValue;
int32_t highValue;
int32_t exposureValue;
uint16_t ownerColor;
uint8_t detectMode;
uint8_t selection;
uint8_t originalValid;
uint8_t previewValid;
uint8_t valid;
} thresholdRenderSnapshotStruct;

static thresholdRenderSnapshotStruct thresholdRenderSnapshot;

static int32_t thresholdDisplayValue(float value, int32_t maximum)
{
    if (!(value == value) || (value <= 0.0f)) return 0;
    if (value >= (float) maximum) return maximum;
    return (int32_t) value;
}

static void drawThresholdValue(uint16_t slotX, uint16_t slotWidth,
    uint16_t nameX, uint16_t valueX, const char *name, uint8_t selected,
    int32_t value, uint8_t digits)
{
    char valueText[5];
    uint16_t background = selected ? MENU_SEL_BG : RGB565_BLACK;
    uint16_t nameForeground = selected ? MENU_SEL_NAME_FG : MENU_NORM_NAME_FG;
    uint16_t valueForeground = selected ? MENU_SEL_VAL_FG : MENU_NORM_VAL_FG;

    if (digits == 3u)
    {
        (void) snprintf(valueText, sizeof(valueText), "%3ld", (long) value);
    }
    else
    {
        (void) snprintf(valueText, sizeof(valueText), "%4ld", (long) value);
    }
    fillSolidRect(slotX, THR_TXT_Y, slotWidth, FONT_H, background);
    ips114_set_font(IPS114_8X16_FONT);
    ips114_set_color(nameForeground, background);
    ips114_show_string(nameX, THR_TXT_Y, name);
    ips114_set_color(valueForeground, background);
    ips114_show_string(valueX, THR_TXT_Y, valueText);
}

void renderThreshEdit(uint8_t visualDue)
{
    uint16_t bc = menuBorderColor();
    uint8_t legacyMode = (detectModeF >= 0.5f) ? 1u : 0u;
    float lowValue = legacyMode ? legacyCannyLowF : cannyLowF;
    float highValue = legacyMode ? legacyCannyHighF : cannyHighF;
    int32_t lowDisplay = thresholdDisplayValue(lowValue, 999);
    int32_t highDisplay = thresholdDisplayValue(highValue, 999);
    int32_t exposureDisplay = thresholdDisplayValue(exposureF, 9999);

    imageFrameViewStruct ori;
    imageFrameViewStruct detectPreview;
    uint8_t oriValid = (imageGetFrameView(IMAGE_FRAME_ORIGINAL, &ori) &&
        (ori.pixels != 0));
    uint8_t previewValid = (uint8_t) (imageGetFrameView(IMAGE_FRAME_DETECT_PREVIEW, &detectPreview) &&
        (detectPreview.pixels != 0));
    uint8_t first = (thresholdRenderSnapshot.valid == 0u) ? 1u : 0u;
    uint8_t selectionChanged = (uint8_t) (first ||
        (thresholdRenderSnapshot.selection != threshSel));

    if (first || (thresholdRenderSnapshot.detectMode != legacyMode))
    {
        drawTitleBar(legacyMode ? "CANNY EDIT" : "BINARY EDIT", RGB565_BLACK, RGB565_GREEN);
    }

    if ((visualDue != 0u) &&
        (first || (thresholdRenderSnapshot.originalValid != oriValid) ||
            (oriValid && (thresholdRenderSnapshot.originalFrameSeq != ori.frameSeq))))
    {
        if (oriValid)
        {
            ips114_show_gray_image(THR_IMG_L_X, THR_IMG_Y, ori.pixels,
                ori.width, ori.height, THR_IMG_W, THR_IMG_H, 0u);
        }
    }
}

```

```

    }
    else
    {
        fillRect(THR_IMG_L_X, THR_IMG_Y, THR_IMG_W, THR_IMG_H, RGB565_BLACK);
    }
}

if ((visualDue != 0u) &&
    (first || (thresholdRenderSnapshot.previewValid != previewValid) ||
     (previewValid && (thresholdRenderSnapshot.previewFrameSeq != detectPreview.frameSeq))))
{
    if (previewValid)
    {
        ips114_show_gray_image(THR_IMG_R_X, THR_IMG_Y, detectPreview.pixels,
                               detectPreview.width, detectPreview.height, THR_IMG_W, THR_IMG_H, 0u);
    }
    else
    {
        fillRect(THR_IMG_R_X, THR_IMG_Y, THR_IMG_W, THR_IMG_H, RGB565_BLACK);
    }
}

if (first || (thresholdRenderSnapshot.lowValue != lowDisplay) ||
    (selectionChanged && ((threshSel == THRESH_SEL_CANNY_LOW) ||
     (thresholdRenderSnapshot.selection == THRESH_SEL_CANNY_LOW))))
    drawThresholdValue(THRESH_SLOT_LOW_X, THRESH_SLOT_LOW_WIDTH,
                      THRESH_NAME_LOW_X, THRESH_VALUE_LOW_X, "Lo:",
                      (uint8_t) (threshSel == THRESH_SEL_CANNY_LOW), lowDisplay, 3u);
if (first || (thresholdRenderSnapshot.highValue != highDisplay) ||
    (selectionChanged && ((threshSel == THRESH_SEL_CANNY_HIGH) ||
     (thresholdRenderSnapshot.selection == THRESH_SEL_CANNY_HIGH))))
    drawThresholdValue(THRESH_SLOT_HIGH_X, THRESH_SLOT_HIGH_WIDTH,
                      THRESH_NAME_HIGH_X, THRESH_VALUE_HIGH_X, "Hi:",
                      (uint8_t) (threshSel == THRESH_SEL_CANNY_HIGH), highDisplay, 3u);
if (first || (thresholdRenderSnapshot.exposureValue != exposureDisplay) ||
    (selectionChanged && ((threshSel == THRESH_SEL_EXPOSURE) ||
     (thresholdRenderSnapshot.selection == THRESH_SEL_EXPOSURE))))
    drawThresholdValue(THRESH_SLOT_EXPOSURE_X, THRESH_SLOT_EXPOSURE_WIDTH,
                      THRESH_NAME_EXPOSURE_X, THRESH_VALUE_EXPOSURE_X, "Ex:",
                      (uint8_t) (threshSel == THRESH_SEL_EXPOSURE), exposureDisplay, 4u);

if (first || (thresholdRenderSnapshot.ownerColor != bc))
{
    /* 竖分隔 + 横分隔落在两图之间/下方空隙，与外框同步操控权色。*/
    fillSolidRect(120u, ROW_Y(1), 1u,
                 (uint16_t) (THR_IMG_Y + THR_IMG_H - ROW_Y(1) + 1u), bc);
    fillSolidRect(1u, (uint16_t) (THR_IMG_Y + THR_IMG_H + 2u),
                 (uint16_t) (LCD_W - 2u), 1u, bc);
    drawFrameBorder(bc);
}

if (visualDue != 0u)
{
    thresholdRenderSnapshot.originalFrameSeq = oriValid ? ori.frameSeq : 0u;
    thresholdRenderSnapshot.previewFrameSeq = previewValid ? detectPreview.frameSeq : 0u;
    thresholdRenderSnapshot.originalValid = oriValid;
    thresholdRenderSnapshot.previewValid = previewValid;
}
thresholdRenderSnapshot.lowValue = lowDisplay;
thresholdRenderSnapshot.highValue = highDisplay;
thresholdRenderSnapshot.exposureValue = exposureDisplay;
thresholdRenderSnapshot.ownerColor = bc;
thresholdRenderSnapshot.detectMode = legacyMode;
thresholdRenderSnapshot.selection = threshSel;
thresholdRenderSnapshot.valid = 1u;
}

/* 列表空行清理使用内框 RGB565 块覆盖，既清旧内容又不触碰外框。
 * 替代每帧全屏 ips114_clear()——全屏清屏会在清→重绘间产生整屏黑闪。*/
/* 整行铺底右界：有滚动条→滚动条左(SCROLLBAR_X-1); 无→右内框(LCD_W-2)。渲染前按 total 设。*/
static uint16_t listFillRightX = (uint16_t) (LCD_W - 2u);

static void fillBottomGutter(uint16_t y, uint16_t color)
{
    uint16_t top;
    if (y != ROW_Y(VISIBLE_ROWS)) return;
    top = (uint16_t) (y + FONT_H);
    if (top >= (uint16_t) (LCD_H - 1u)) return;
    fillSolidRect(1u, top, listFillRightX,
                 (uint16_t) (LCD_H - 1u - top), color);
}

```

```

}

static void clearListRow(uint16_t y)
{
    fillSolidRect(1u, y, listFillRightX, FONT_H, RGB565_BLACK);
    fillBottomGutter(y, RGB565_BLACK);
}

/* 整行铺底色，顶到右界：选中/编辑整行强调，名/值由调用方另设色。
 * 修原选中态只反色字符、中间黑底不连成整行选中条的问题。 */
static void fillListRowBg(uint16_t y, uint16_t fg, uint16_t bg)
{
    ips114_set_color(fg, bg);
    fillSolidRect(1u, y, listFillRightX, FONT_H, bg);
    fillBottomGutter(y, bg);
}

/* 列表/HOME 局部渲染快照（变化检测防闪烁，对齐 core2/core0）：仅结构变化(进/出编辑、换页、换选中项、滚动、
 * 操控权边框色变、切状态后)才整页重绘；EDIT 态值变只局部重绘当前值。
 * clearScreenForState 调 menuRenderResetListSnapshot 复位 → 下次列表渲染为结构性，HOME 来源条强制重绘。 */
static menuStateEnum listLastState = (menuStateEnum) 0xFF;
static int16_t listLastPageIdx = -1;
static int16_t listLastOptIdx = -1;
static uint8_t listLastScrollTop = 0xFFu;
static uint16_t listLastBc = 0xFFFFu;
static uint8_t paramSlotsSurfaceValid;
static uint8_t paramSlotsLastSelection = 0xFFu;
static menuParamGlobalUiSnapshot paramSlotsLastUi;

void menuRenderResetListSnapshot(void)
{
    listLastState = (menuStateEnum) 0xFF;
    listLastPageIdx = -1;
    listLastOptIdx = -1;
    listLastScrollTop = 0xFFu;
    listLastBc = 0xFFFFu;
    paramSlotsSurfaceValid = 0u;
    paramSlotsLastSelection = 0xFFu;
    memset(&paramSlotsLastUi, 0xFF, sizeof(paramSlotsLastUi));
    homeLayoutColor = 0xFFFFu;
    memset(homeSlotValid, 0, sizeof(homeSlotValid));
    memset(homeSlotColor, 0, sizeof(homeSlotColor));
    memset(homeSlotBackground, 0, sizeof(homeSlotBackground));
    homeTitleColor = 0xFFFFu;
    memset(&thresholdRenderSnapshot, 0, sizeof(thresholdRenderSnapshot));
}

/* =====
 * 页列表 / 选项列表(editing=1 时为编辑页, 高亮当前项显值)
 * 不全屏 clear(防黑闪): 进入页面时 clearScreenForState 已清屏一次,
 * 页内 20ms 事实刷新仅逐行脏区覆盖(clearListRow)。
 * ===== */
void renderPageList(void)
{
    uint16_t bc = menuBorderColor();
    /* 结构变化才整页重绘(防周期空刷闪烁)：切入页列表 / 换选中页 / 滚动 / 边框色变。 */
    uint8_t structural = (uint8_t) (listLastState != MENU_PAGE_LIST
        || listLastPageIdx != (int16_t) pageIdx || listLastScrollTop != scrollTop
        || listLastBc != bc);
    uint8_t selectionOnly = (uint8_t) ((listLastState == MENU_PAGE_LIST) &&
        (listLastScrollTop == scrollTop) && (listLastBc == bc) &&
        (listLastPageIdx != (int16_t) pageIdx));
    if (!structural) return;

    listFillRightX = (menuPageCount > VISIBLE_ROWS) ? (uint16_t) (SCROLLBAR_X - 1u) : (uint16_t) (LCD_W - 2u);
    if (selectionOnly == 0u)
        drawTitleBar("PAGES", RGB565_BLACK, RGB565_GREEN);

    uint8_t top = scrollTop;
    for (uint8_t r = 0u; r < VISIBLE_ROWS; r++)
    {
        uint8_t idx = (uint8_t) (top + r);
        uint16_t y = ROW_Y((uint16_t) (r + 1u));
        if ((selectionOnly != 0u) && (idx != (uint8_t) pageIdx) &&
            (idx != (uint8_t) listLastPageIdx)) continue;
        if (idx >= menuPageCount) { clearListRow(y); continue; }
        if (idx == (uint8_t) pageIdx)
        {
            fillListRowBg(y, MENU_SEL_NAME_FG, MENU_SEL_BG); /* 选中整行青底，名黑(页列表无数值) */

```

```

        ips114_show_string(MENU_MARK_X, y, ">");
        ips114_show_string(MENU_ITEM_X, y, menuPages[idx].name);
    }
    else
    {
        clearListRow(y);
        ips114_set_color(MENU_NORM_NAME_FG, RGB565_BLACK);
        ips114_show_string(MENU_ITEM_X, y, menuPages[idx].name);
    }
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
}
ips114_set_color(RGB565_GREEN, RGB565_BLACK);
if (selectionOnly == 0u)
{
    drawScrollbar(menuPageCount, scrollTop, bc);
    drawFrameBorder(bc);
}
listLastState = MENU_PAGE_LIST;
listLastPageldx = (int16_t) pageldx;
listLastScrollTop = scrollTop;
listLastOptIdx = -1;
listLastBc = bc;
}

void renderOptionList(uint8_t editing)
{
    if ((editing == 0u) && (pageldx >= 0) &&
        ((uint8_t) pageldx == paramSlotsPageIndex))
    {
        char line[32];
        menuParamGlobalUiSnapshot globalUi;
        uint16_t bc = menuBorderColor();
        uint8_t fullRedraw;
        menuParamGlobalGetUi(&globalUi);
        listFillRightX = (uint16_t) (LCD_W - 2u);
        fullRedraw = (uint8_t) (paramSlotsSurfaceValid == 0u);
        if (fullRedraw != 0u)
        {
            uint8_t row;
            drawTitleBar("PARAM SLOTS", RGB565_BLACK, RGB565_GREEN);
            for (row = 1u; row < 8u; row++) clearListRow(ROW_Y(row));
            ips114_set_color(RGB565_GREEN, RGB565_BLACK);
            ips114_show_string(8u, ROW_Y(5u), "K2/K3 SLOT");
            ips114_show_string(8u, ROW_Y(6u), "K1L SAVE K4 LOAD");
            ips114_show_string(8u, ROW_Y(7u), "K5 BACK");
            drawFrameBorder(bc);
            paramSlotsSurfaceValid = 1u;
        }
        if (fullRedraw != 0u || paramSlotsLastSelection !=
            (uint8_t) paramSlotSelection)
        {
            fillListRowBg(ROW_Y(1u), MENU_SEL_NAME_FG, MENU_SEL_BG);
            (void) snprintf(line, sizeof(line), "> SELECT    SLOT %u",
                (unsigned) ((uint8_t) paramSlotSelection));
            ips114_show_string(MENU_MARK_X, ROW_Y(1u), line);
            paramSlotsLastSelection = (uint8_t) paramSlotSelection;
        }
        if (fullRedraw != 0u || paramSlotsLastUi.slotUiState !=
            globalUi.slotUiState)
        {
            static const char *const slotText[] = {
                "UNKNOWN", "CHECKING", "FULL", "EMPTY", "PARTIAL",
                "INVALID", "UNAVAILABLE"};
            uint8_t state = globalUi.slotUiState;
            if (state > PARAM_SYNC_SLOT_UI_UNAVAILABLE)
                state = PARAM_SYNC_SLOT_UI_UNKNOWN;
            clearListRow(ROW_Y(2u));
            (void) snprintf(line, sizeof(line), "SLOT: %s", slotText[state]);
            ips114_set_color((state == PARAM_SYNC_SLOT_UI_FULL) ? RGB565_GREEN :
                ((state == PARAM_SYNC_SLOT_UI_EMPTY) ? RGB565_CYAN : RGB565_YELLOW),
                RGB565_BLACK);
            ips114_show_string(24u, ROW_Y(2u), line);
        }
        if (globalUi.state == MENU_PARAM_GLOBAL_QUEUED)
            (void) snprintf(line, sizeof(line), "TASK: QUEUED");
        else if (globalUi.state == MENU_PARAM_GLOBAL_PROGRESS)
            (void) snprintf(line, sizeof(line), "RUN: C%u PH%u",
                (unsigned) globalUi.target, (unsigned) globalUi.phase);
        else if (globalUi.state == MENU_PARAM_GLOBAL_SUCCESS)

```

```

        (void) snprintf(line, sizeof(line), "TASK: SUCCESS");
    else if (globalUi.state == MENU_PARAM_GLOBAL_FAILED)
    {
        uint32_t diagnostic = globalUi.diagnostic;
        (void) snprintf(line, sizeof(line),
            "F:C%uE%u P%uO%uS%uD%u",
            (unsigned) globalUi.target, (unsigned) globalUi.error,
            (unsigned) ((diagnostic >> 16) & 0xFFu),
            (unsigned) (diagnostic & 0xFFu),
            (unsigned) ((diagnostic >> 8) & 0xFFu),
            (unsigned) ((diagnostic >> 24) & 0xFFu));
    }
    else
        (void) snprintf(line, sizeof(line), "TASK: READY");
    if (fullRedraw != 0u || memcmp(&paramSlotsLastUi, &globalUi,
        sizeof(globalUi)) != 0)
    {
        clearListRow(ROW_Y(3u));
        ips114_set_color(globalUi.state == MENU_PARAM_GLOBAL_SUCCESS ? RGB565_GREEN :
            (globalUi.state == MENU_PARAM_GLOBAL_FAILED ? RGB565_RED : RGB565_CYAN),
            RGB565_BLACK);
        ips114_show_string(24u, ROW_Y(3u), line);
    }
    paramSlotsLastUi = globalUi;
    listLastState = MENU_OPTION_LIST;
    listLastPagIdx = (int16_t) pagIdx;
    listLastOptIdx = (int16_t) optIdx;
    listLastScrollTop = scrollTop;
    listLastBc = bc;
    return;
}
uint16_t bc = menuBorderColor();
menuPageStruct* pg = &menuPages[pagIdx];
uint8_t curIsEdit = (uint8_t) (editing != 0u);
/* 结构变化判定(对齐 core2/core0): 切入 / 进出编辑 / 换选中项 / 滚动 / 边框色变 → 整页重绘;
 * EDIT 态值变(同态同项同滚动同边框色)只局部重绘当前值, 消周期全页重绘闪烁。 */
uint8_t enteringList = (uint8_t) (listLastState != MENU_OPTION_LIST && listLastState != MENU_EDIT);
uint8_t editTransition = (uint8_t) ((listLastState == MENU_EDIT) != curIsEdit);
uint8_t structural = (uint8_t) (enteringList || editTransition
    || listLastPagIdx != (int16_t) pagIdx
    || listLastOptIdx != (int16_t) optIdx || listLastScrollTop != scrollTop
    || listLastBc != bc);
uint8_t selectionOnly = (uint8_t) ((!enteringList) && (!editTransition) &&
    (listLastPagIdx == (int16_t) pagIdx) &&
    (listLastScrollTop == scrollTop) && (listLastBc == bc) &&
    (listLastOptIdx != (int16_t) optIdx));

if (structural)
{
    listFillRightX = (pg->optionCount > VISIBLE_ROWS) ? (uint16_t) (SCROLLBAR_X - 1u) : (uint16_t) (LCD_W - 2u);
    if (selectionOnly == 0u)
        drawTitleBar(pg->name, RGB565_BLACK, RGB565_GREEN);

    uint8_t top = scrollTop;
    for (uint8_t r = 0u; r < VISIBLE_ROWS; r++)
    {
        uint8_t idx = (uint8_t) (top + r);
        uint16_t y = ROW_Y((uint16_t) (r + 1u));
        uint8_t sel;
        if ((selectionOnly != 0u) && (idx != (uint8_t) optIdx) &&
            (idx != (uint8_t) listLastOptIdx)) continue;
        if (idx >= pg->optionCount) { clearListRow(y); continue; }
        sel = (idx == (uint8_t) optIdx);

        if (sel && editing)
        {
            /* 编辑行: 整行黄底强调, 名黑/值红 (非仅数值反相) */
            fillListRowBg(y, MENU_EDIT_NAME_FG, MENU_EDIT_BG);
            ips114_show_string(MENU_MARK_X, y, "");
            ips114_show_string(MENU_ITEM_X, y, pg->optName[idx]);
            ips114_set_color(MENU_EDIT_VAL_FG, MENU_EDIT_BG);
            menuFloatDisplayShow(MENU_PARAM_VALUE_X, y,
                (double) (*pg->opt[idx]),
                (pg->decimals[idx] != 0u)
                ? pg->decimals[idx]
                : menuFloatDisplayDecimalsForValueAndStep(
                    (double) (*pg->opt[idx]), pg->stepS[idx]));
        }
    }
}

```



```

else if (sel)
{
    /* 选中行：整行青底，名黑/值蓝（整行铺底，修原只反色字符问题） */
    fillListRowBg(y, MENU_SEL_NAME_FG, MENU_SEL_BG);
    ips114_show_string(MENU_MARK_X, y, ">");
    ips114_show_string(MENU_ITEM_X, y, pg->optName[idx]);
    ips114_set_color(MENU_SEL_VAL_FG, MENU_SEL_BG);
    menuFloatDisplayShow(MENU_PARAM_VALUE_X, y,
        (double) (*pg->opt[idx]),
        (pg->decimals[idx] != 0u)
            ? pg->decimals[idx]
            : menuFloatDisplayDecimalsForValueAndStep(
                (double) (*pg->opt[idx]), pg->stepS[idx]));
}
else
{
    /* 普通行：黑底，名绿/值白 */
    clearListRow(y);
    ips114_set_color(MENU_NORM_NAME_FG, RGB565_BLACK);
    ips114_show_string(MENU_ITEM_X, y, pg->optName[idx]);
    ips114_set_color(MENU_NORM_VAL_FG, RGB565_BLACK);
    menuFloatDisplayShow(MENU_PARAM_VALUE_X, y,
        (double) (*pg->opt[idx]),
        (pg->decimals[idx] != 0u)
            ? pg->decimals[idx]
            : menuFloatDisplayDecimalsForValueAndStep(
                (double) (*pg->opt[idx]), pg->stepS[idx]));
}
ips114_set_color(RGB565_GREEN, RGB565_BLACK);
}
ips114_set_color(RGB565_GREEN, RGB565_BLACK);
if (selectionOnly == 0u)
{
    drawScrollbar(pg->optionCount, scrollTop, bc);
    drawFrameBorder(bc);
}
}
else if (editing)
{
    /* 值局部刷新：仅当前选中项数值(黄底红字)，不动整行/名/边框/滚动条 → 消整页重绘闪烁 */
    uint16_t y = ROW_Y((uint16_t) optIdx - scrollTop + 1u);
    ips114_set_color(MENU_EDIT_VAL_FG, MENU_EDIT_BG);
    menuFloatDisplayShow(MENU_PARAM_VALUE_X, y,
        (double) (*pg->opt[optIdx]),
        (pg->decimals[optIdx] != 0u)
            ? pg->decimals[optIdx]
            : menuFloatDisplayDecimalsForValueAndStep(
                (double) (*pg->opt[optIdx]), pg->stepS[optIdx]));
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
}

listLastState = curIsEdit ? MENU_EDIT : MENU_OPTION_LIST;
listLastPageIdx = (int16_t) pageIdx;
listLastOptIdx = (int16_t) optIdx;
listLastScrollTop = scrollTop;
listLastBc = bc;
}

```

core1_cyt4bb7/project/code/app/observerOverlayPoseMapping.h

SHA-256

:

e6d32c6f97f710dc854bc277cc221ba203d6e865596492a6129d440db9618794

代码 (c)

```

#ifndef OBSERVER_OVERLAY_POSE_MAPPING_H
#define OBSERVER_OVERLAY_POSE_MAPPING_H

```

```

#include <math.h>
#include <stdint.h>

```

```

typedef struct
{
    uint16_t x;
    uint16_t y;

```

```

    int16_t headingQ8;
    uint8_t poseMapped;
    uint8_t headingMapped;
} observerOverlayPoseMapping;

static inline uint8_t observerOverlayMapCoordinate(float value,
    uint16_t limit, uint16_t *pixel)
{
    if ((pixel == 0) || (!isfinite(value)) ||
        (value < 0.0f) || (value > (float) (limit - 1u))) return 0u;
    *pixel = (uint16_t) (value + 0.5f);
    return 1u;
}

static inline void observerOverlayPoseMap(float rawX, float rawY,
    float offsetX, float offsetY, uint8_t poseValid,
    float rawHeadingRad, uint8_t headingValid,
    observerOverlayPoseMapping *mapping)
{
    if (mapping == 0) return;
    mapping->x = 0u;
    mapping->y = 0u;
    mapping->headingQ8 = 0;
    mapping->poseMapped = 0u;
    mapping->headingMapped = 0u;
    if ((poseValid == 0u) ||
        (observerOverlayMapCoordinate(rawX + offsetX, 188u,
            &mapping->x) == 0u) ||
        (observerOverlayMapCoordinate(rawY + offsetY, 120u,
            &mapping->y) == 0u)) return;
    mapping->poseMapped = 1u;
    if ((headingValid == 0u) || (!isfinite(rawHeadingRad)) ||
        (rawHeadingRad < -3.1415927f) ||
        (rawHeadingRad > 3.1415927f)) return;
    mapping->headingQ8 = (int16_t) (rawHeadingRad * 256.0f);
    mapping->headingMapped = 1u;
}

#endif

```

core1_cyt4bb7/project/code/app/paramSyncPlatformPolicy.h

SHA-256

:

50f464f40bb5c432c5aae25f6af5b98c29969b141802c44e71e81d2bb796a8ee

代码 (c)

```

#ifndef PARAM_SYNC_PLATFORM_POLICY_H
#define PARAM_SYNC_PLATFORM_POLICY_H

#include <stdint.h>
#include <string.h>

#include "coreDataTransfer.h"

#define CORE1_PARAM_SYNC_OBSERVER_TARGET_FRESH_MS (600u)

typedef struct
{
    uint32_t producerBootGeneration;
    uint32_t publishSequence;
    uint32_t peerSession;
    uint32_t lastAdvanceMs;
    uint8_t primed;
    uint8_t ready;
} core1ParamSyncObserverBridgeCache;

static inline void core1ParamSyncObserverBridgeReset(
    core1ParamSyncObserverBridgeCache *cache)
{
    if (cache != 0) memset(cache, 0, sizeof(*cache));
}

static inline uint8_t core1ParamSyncObserverBridgeAccept(
    core1ParamSyncObserverBridgeCache *cache,

```

```

    const core1ParamObserverBridgeStruct *bridge, uint32_t nowMs)
{
    if (cache == 0 || bridge == 0 ||
        coreDataParamObserverBridgeIsValid(bridge) == 0u)
    {
        return 0u;
    }
    if (cache->primed != 0u &&
        cache->producerBootGeneration == bridge->producerBootGeneration &&
        cache->peerSession == bridge->peerSession &&
        (int32_t) (bridge->publishSequence - cache->publishSequence) <= 0)
    {
        return 0u;
    }
    if (cache->primed == 0u ||
        cache->producerBootGeneration != bridge->producerBootGeneration ||
        cache->peerSession != bridge->peerSession)
    {
        core1ParamSyncObserverBridgeReset(cache);
    }
    cache->producerBootGeneration = bridge->producerBootGeneration;
    cache->publishSequence = bridge->publishSequence;
    cache->peerSession = bridge->peerSession;
    cache->lastAdvanceMs = nowMs;
    cache->primed = 1u;
    cache->ready = (uint8_t) ((bridge->flags &
        CORE1_PARAM_OBSERVER_FLAG_READY) != 0u);
    return 1u;
}

static inline void core1ParamSyncObserverBridgeExpire(
    core1ParamSyncObserverBridgeCache *cache, uint32_t nowMs)
{
    if (cache == 0 || cache->primed == 0u ||
        (uint32_t) (nowMs - cache->lastAdvanceMs) <
            CORE1_PARAM_SYNC_OBSERVER_TARGET_FRESH_MS)
    {
        return;
    }
    cache->ready = 0u;
}

static inline uint8_t core1ParamSyncObserverBridgeReady(
    const core1ParamSyncObserverBridgeCache *cache, uint32_t nowMs)
{
    return (uint8_t) (cache != 0 && cache->primed != 0u &&
        cache->ready != 0u &&
        (uint32_t) (nowMs - cache->lastAdvanceMs) <
            CORE1_PARAM_SYNC_OBSERVER_TARGET_FRESH_MS);
}

static inline void core1ParamSyncObserverBridgePoll(
    core1ParamSyncObserverBridgeCache *cache,
    const core1ParamObserverBridgeStruct *bridge, uint32_t nowMs)
{
    if (cache == 0) return;
    if (bridge != 0)
        (void) core1ParamSyncObserverBridgeAccept(cache, bridge, nowMs);
    core1ParamSyncObserverBridgeExpire(cache, nowMs);
}

#endif

```

core1_cyt4bb7/project/code/app/rgbAnimator.c

SHA-256

:

1f19e0b0ef65ec3ec1a893ef14f695b15dcf22e114d4d6b37c0a7d251358eca4

代码 (c)

```

#include "rgbAnimator.h"

#include "uartLinkProto.h" /* UART_LINK_RGB_x 意图码 (与 core0 同源同值) */

/*

```

```

* RGB 起飞编排动画本体。详见 rgbAnimator.h 顶注。
* 闪烁用 uint16 相位累加器：每 tick += step（自然回绕），on = (phase < 32768) → 50% 占空方波；
* step 越大频率越高。亮度档由 rgbStatusLed setRgb 内部钳 30%，本层用 0~100 相对值。
*/

/* 动画时长(ms)：对齐 core0 configFlight 的 TAKEOFF_WAIT_MS/RAMP_MS，仅控制本地动画形状(频率/亮度斜坡)。
* 无需与 core0 字节级一致——意图码由 core0 真实相位驱动，相位切换时本层据新意图重置；时长只决定形状。*/
#define RGB_ANIM_TICK_MS   (10u) /* rgbAnimTick 调用周期(PIT_CH2 10ms) */
#define RGB_ANIM_WAIT_MS   (5000u) /* ~对齐 TAKEOFF_WAIT_MS */
#define RGB_ANIM_RAMP_MS   (2000u) /* ~对齐 TAKEOFF_RAMP_MS */
#define RGB_ANIM_WAIT_SOLID_MS (700u) /* WAIT 末段强制常亮时长(频率封顶→常亮过渡，提示即将爬升) */
#define RGB_ANIM_BRIGHT_PCT (100u) /* 满档相对亮度(setRgb 内部钳 30%) */

/* 闪烁相位步进：WAIT 期随 elapsed 线性增(慢闪→快闪)，WARN 固定高频。值越大频率越高。*/
#define RGB_ANIM_WAIT_STEP_SLOW (1000u) /* 起始 ~1.5Hz */
#define RGB_ANIM_WAIT_STEP_FAST (6000u) /* 末段 ~9Hz */
#define RGB_ANIM_WARN_STEP      (8000u) /* 断联告警固定高频红闪 */

static uint8_t curIntent;
static uint32_t animElapsedMs;
static uint16_t animPhase;

void rgbAnimInit(void)
{
    curIntent = UART_LINK_RGB_NORMAL;
    animElapsedMs = 0u;
    animPhase = 0u;
}

void rgbAnimSet(uint8_t intent)
{
    if (intent != curIntent)
    {
        curIntent = intent;
        animElapsedMs = 0u;
        animPhase = 0u;
    }
}

void rgbAnimTick(void)
{
    uint16_t step = 0u;

    if (animElapsedMs < (0xFFFFFFFFu - RGB_ANIM_TICK_MS))
    {
        animElapsedMs += RGB_ANIM_TICK_MS;
    }

    switch (curIntent)
    {
        case UART_LINK_RGB_TKO_WAIT:
        {
            /* 步进随 elapsed 从 SLOW 线性升到 FAST（频率渐增），到时长封顶。*/
            uint32_t frac256 = (animElapsedMs >= RGB_ANIM_WAIT_MS)
                ? 256u : (animElapsedMs * 256u / RGB_ANIM_WAIT_MS);
            step = (uint16_t) (RGB_ANIM_WAIT_STEP_SLOW
                + (uint32_t) (RGB_ANIM_WAIT_STEP_FAST - RGB_ANIM_WAIT_STEP_SLOW) * frac256 / 256u);
            break;
        }
        case UART_LINK_RGB_DISCONNECT_WARN:
            step = RGB_ANIM_WARN_STEP;
            break;
        default:
            step = 0u; /* RAMP/NORMAL 不用闪烁相位 */
            break;
    }

    animPhase = (uint16_t) (animPhase + step);
}

uint8_t rgbAnimActive(void)
{
    return (curIntent != UART_LINK_RGB_NORMAL) ? 1u : 0u;
}

void rgbAnimRender(uint8_t* rPct, uint8_t* gPct, uint8_t* bPct)
{
    uint8_t r = 0u;
    uint8_t g = 0u;

```

```
uint8_t b = 0u;

switch (curIntent)
{
    case UART_LINK_RGB_TKO_WAIT:
    {
        /* 白色快闪(频率随 elapsed 渐增); 未 RGB_ANIM_WAIT_SOLID_MS 段强制常亮(过渡到爬升)。 */
        uint8_t on;
        if (animElapsedMs >= (RGB_ANIM_WAIT_MS - RGB_ANIM_WAIT_SOLID_MS))
        {
            on = 1u;
        }
        else
        {
            on = (animPhase < 32 768u) ? 1u : 0u;
        }
        r = on ? RGB_ANIM_BRIGHT_PCT : 0u;
        g = r;
        b = r;
        break;
    }
    case UART_LINK_RGB_TKO_RAMP:
    {
        /* 白色常亮 → 线性渐灭(RAMP_MS)。 */
        uint32_t frac256 = (animElapsedMs >= RGB_ANIM_RAMP_MS)
            ? 256u : (animElapsedMs * 256u / RGB_ANIM_RAMP_MS);
        uint8_t bright = (uint8_t) ((uint32_t) RGB_ANIM_BRIGHT_PCT * (256u - frac256) / 256u);
        r = bright;
        g = bright;
        b = bright;
        break;
    }
    case UART_LINK_RGB_DISCONNECT_WARN:
    {
        /* 红色高频快闪(断联告警)。 */
        uint8_t on = (animPhase < 32 768u) ? 1u : 0u;
        r = on ? RGB_ANIM_BRIGHT_PCT : 0u;
        break;
    }
    default:
        break; /* NORMAL: 全 0 (rgbAnimActive=0, 渲染不会被调用) */
}

*rPct = r;
*gPct = g;
*bPct = b;
}
```

core1_cyt4bb7/project/code/app/rgbAnimator.h

SHA-256 :

c07a8c8d11e4cec14b40bdf69386247df259439cf004a09dff2d4d4d673829fd

代码 (c)

```
#ifndef_RGB_ANIMATOR_H_
#define_RGB_ANIMATOR_H_

#include <stdint.h>

/*
 * core1 cm_7_1 RGB 起飞编排动画层 (决策/渲染分离)。
 * core0 takeoffSequencer 产“意图码”(UART_LINK_RGB_*) → CORE0_STATUS sysCtrl 下发 → 共享内存
 * keyEventToMenuStruct.core0RgbIntent → 本层据意图本地自演动画(频率/亮度斜坡), 输出 RGB 期望
 * 亮度, 由 rgbStatusLed 用其 setRgb 原语应用。意图=NORMAL 时本层 inactive, 放行原状态灯优先级链。
 * 纯 cm_7_1 前台 (rgbStatusLedUpdate 内 PIT_CH2 10ms 调), 无 ISR / 无跨核共享。
 * 单向依赖: rgbStatusLed → rgbAnimator (本层不反向调 rgbStatusLed, 仅算色返回)。
 */

void rgbAnimInit(void);
void rgbAnimSet(uint8_t intent); /* 设意图码(UART_LINK_RGB_*); 意图变化即重置动画相位/计时 */
void rgbAnimTick(void); /* 每 10ms: 推进动画相位与计时 */
uint8_t rgbAnimActive(void); /* 1=非 NORMAL(本层接管渲染); 0=放行原状态灯链 */
void rgbAnimRender(uint8_t* rPct, uint8_t* gPct, uint8_t* bPct); /* 算当前 RGB 期望亮度 0~100 */
```

```
#endif /* _RGB_ANIMATOR_H_ */
```

core1_cyt4bb7/project/code/app/rgbStatusLed.c

SHA-256

:

26ae45665d1db373e0b84a7f003970a175b12df1ea7bae5974c4d98cd3d4c1f3

代码 (c)

```
#include "rgbStatusLed.h"

#include "zf_driver_pwm.h"

#include "core1KvmRouter.h"
#include "uartLinkProto.h" /* UART_LINK_C0_FLAG_ 位定义 + UART_LINK_RGB_ 意图码 */
#include "rgbAnimator.h" /* 起飞编排动画层 (意图码→灯效) */
#include "pinMap.h" /* RGB 三路 TCPWM 通道令牌单一真相源 */

/*
 * core1 RGB 状态灯实现。共阳极低电平点亮 → 占空比反向(亮度% 越高 duty 越小)。
 * 最大亮度 30% 硬钳位。动效(闪烁/呼吸)由 PIT_CH2 10ms tick 每次 update 累加软生成。
 */

#define RGB_PWM_R (RGB_LED_R_PWM)
#define RGB_PWM_G (RGB_LED_G_PWM)
#define RGB_PWM_B (RGB_LED_B_PWM)
#define RGB_PWM_FREQ_HZ (1000u)

#define RGB_MAX_BRIGHT_PCT (30u) /* 最大亮度上限(用户要求) */
#define RGB_DIM_PCT (8u) /* "暗"档亮度(明暗维的暗) */

/* PIT_CH2 10ms tick 每次 update 自增, 决定动效相位(闪烁方波/呼吸三角波)。10ms 节拍稳定,
 * 相位阈值以"调用次数"计(如 flashOn(40) ≈ 400ms 半周期), 肉眼可辨即可。 */
static uint32_t rgbTick = 0u;

/* core0 核间链路"曾连接"锁存: 区分上电从未连接(等待 core0, 红慢闪) vs 曾连接后断开(断联告警, 红快闪 WARN)。
 * rgbStatusLedInit 清 0; 一旦 linkUp 见过 1 即锁存, 不复位(断联后保持告警, 重连后清意图自然恢复)。 */
static uint8_t everLinkedCore0 = 0u;
static uint8_t hardwareInitFaultActive = 0u;
static volatile uint32_t core0FactsPacked = 0u;

/* 亮度% → 占空比(共阳极反向 + 30% 硬钳位)。pct: 期望亮度 0~100。 */
static uint32_t brightToDuty(uint8_t pct)
{
    if (pct > RGB_MAX_BRIGHT_PCT) pct = RGB_MAX_BRIGHT_PCT; /* 硬钳位, 任何状态不超 30% */
    /* 共阳极: 亮度 pct% → 低电平时间 pct% → duty = PWM_DUTY_MAX*(100-pct)/100。 */
    return (uint32_t)(PWM_DUTY_MAX - (uint32_t) pct * (PWM_DUTY_MAX / 100u));
}

/* 设三路颜色(各通道期望亮度 0~100, 内部钳 30%)。 */
static void setRgb(uint8_t rPct, uint8_t gPct, uint8_t bPct)
{
    pwm_set_duty(RGB_PWM_R, brightToDuty(rPct));
    pwm_set_duty(RGB_PWM_G, brightToDuty(gPct));
    pwm_set_duty(RGB_PWM_B, brightToDuty(bPct));
}

/* 动效相位: 方波(闪烁)。periodTicks=半周期调用次数; 返回 1=亮相 0=灭相。 */
static uint8_t flashOn(uint32_t periodTicks)
{
    if (periodTicks == 0u) return 1u;
    return ((rgbTick / periodTicks) & 1u) ? 0u : 1u;
}

/* 动效相位: 三角波(呼吸), 返回 0~100 亮度系数。periodTicks=半周期。 */
static uint8_t breathePct(uint32_t periodTicks)
{
    if (periodTicks == 0u) return 100u;
    uint32_t ph = rgbTick % (periodTicks * 2u);
    uint32_t up = (ph < periodTicks) ? ph : (periodTicks * 2u - ph); /* 0..periodTicks 三角 */
    return (uint8_t)(up * 100u / periodTicks);
}

void rgbStatusLedInit(void)
```

```

{
    rgbTick = 0u;
    everLinkedCore0 = 0u;
    hardwareInitFaultActive = 0u;
    core0FactsPacked = 0u;
    rgbAnimInit(); /* 起飞编排动画层复位 */
    /* 共阳极: 初始全灭(duty=满, 高电平) 防上电乱闪。 */
    pwm_init(RGB_PWM_R, RGB_PWM_FREQ_HZ, PWM_DUTY_MAX);
    pwm_init(RGB_PWM_G, RGB_PWM_FREQ_HZ, PWM_DUTY_MAX);
    pwm_init(RGB_PWM_B, RGB_PWM_FREQ_HZ, PWM_DUTY_MAX);
}

void rgbStatusLedUpdate(uint8_t core1Ready, uint8_t menuStateVal, uint16_t beaconCount)
{
    (void) menuStateVal;
    (void) beaconCount;
    rgbTick++;

    /* 前台发布窄原子事实; PIT ISR 禁止做 D-cache 事务和共享结构双读。 */
    uint32_t facts = core0FactsPacked;
    uint8_t shmReady = (uint8_t) (facts & 1u);
    uint8_t linkUp = (uint8_t) ((shmReady != 0u) && (((facts >> 1) & 1u) != 0u));
    uint8_t c0Valid = (uint8_t) ((shmReady != 0u) && (((facts >> 2) & 1u) != 0u));
    uint8_t c0Safety = (uint8_t) (facts >> 8);
    uint8_t c0Flags = (uint8_t) (facts >> 16);
    uint8_t c0RgbIntent = (uint8_t) (facts >> 24);

    uint8_t cm70Ready = (c0Valid && (c0Flags & UART_LINK_C0_FLAG_CM70_READY)) ? 1u : 0u;
    uint8_t velValid = (c0Valid && (c0Flags & UART_LINK_C0_FLAG_VEL_VALID)) ? 1u : 0u;
    uint8_t targetValid = (c0Valid && (c0Flags & UART_LINK_C0_FLAG_TARGET_VALID)) ? 1u : 0u;
    uint8_t kvmState = core1KvmRouterGetDisplayStateIsr();
    uint8_t desired = CORE1_KVM_DISPLAY_DESIRED(kvmState);
    uint8_t owner = CORE1_KVM_DISPLAY_OWNER(kvmState);

    /* 安全优先级高于起飞编排动画: 本地硬故障/相机故障 > 飞控硬故障 >
     * 必需链路 > PROTECT > 初始化等待 > 起飞编排。 */
    if (!core1Ready)
    {
        setRgb(0u, 0u, 0u);
        return;
    }
    if (linkUp && c0Valid && c0Safety == 0xFFu)
    {
        uint8_t on = flashOn(10u);
        setRgb(on ? RGB_MAX_BRIGHT_PCT : 0u, 0u, 0u);
        return;
    }
    if (hardwareInitFaultActive != 0u)
    {
        uint32_t phase = rgbTick % 200u;
        uint8_t on = (uint8_t) ((phase < 10u) ||
            ((phase >= 25u) && (phase < 35u)));
        setRgb(on ? RGB_MAX_BRIGHT_PCT : 0u, 0u, 0u);
        return;
    }
    if (linkUp) everLinkedCore0 = 1u;
    if (!linkUp)
    {
        uint8_t on = flashOn(everLinkedCore0 ? 10u : 40u);
        setRgb(on ? RGB_MAX_BRIGHT_PCT : 0u, 0u, 0u);
        return;
    }
    if (c0Safety == 0x02u)
    {
        uint8_t on = flashOn(15u);
        setRgb(on ? RGB_MAX_BRIGHT_PCT : 0u, 0u,
            on ? RGB_MAX_BRIGHT_PCT : 0u);
        return;
    }
    if (!cm70Ready)
    {
        uint8_t b = breathePct(60u);
        setRgb((uint8_t)(RGB_MAX_BRIGHT_PCT * b / 100u),
            (uint8_t)(RGB_MAX_BRIGHT_PCT * b / 100u), 0u);
        return;
    }
    if (c0Safety == 0xFEu)
    {
        uint8_t on = flashOn(15u);

```

```

setRgb(on ? RGB_MAX_BRIGHT_PCT : 0u,
      on ? RGB_MAX_BRIGHT_PCT : 0u, 0u);
return;
}

/* ---- 起飞编排动画层(最高优先): core0 经 sysCtrl 下发 rgbIntent, 非 NORMAL 时本核自演动画 ----
* 每拍眼意图(共享内存就绪才信, 否则 NORMAL)+推进相位: core1 就绪且意图有效时接管渲染并返回,
* 否则放行下方原状态灯优先级链(NORMAL 时本层 inactive, 原灯照常)。 */
{
    uint8_t intent;
    /* 链路上升沿已在安全优先级入口锁存。 */
    /* 意图选择(区分等待 vs 断联, 并防陈旧意图残留):
    * 共享内存未就绪 → NORMAL;
    * 链路在线 → 用 core0 下发意图(起飞编排 / core0 缓降下发的 DISCONNECT_WARN);
    * 曾连接后断开 → 统一红快闪 DISCONNECT_WARN(断联告警; 不沿用陈旧 core0 意图, 防残留起飞动画);
    * 上电从未连接 → NORMAL, 放行下方优先级链 #3 红慢闪作"等待 core0 连接"指示。 */
    if (!shmReady)
    {
        intent = UART_LINK_RGB_NORMAL;
    }
    else if (linkUp)
    {
        intent = c0RgbIntent;
    }
    else
    {
        intent = everLinkedCore0 ? UART_LINK_RGB_DISCONNECT_WARN : UART_LINK_RGB_NORMAL;
    }
    rgbAnimSet(intent);
    rgbAnimTick();
    if (core1Ready && rgbAnimActive())
    {
        uint8_t ar;
        uint8_t ag;
        uint8_t ab;
        rgbAnimRender(&ar, &ag, &ab);
        setRgb(ar, ag, ab);
        return;
    }
}

/* 正常职责: KVM 颜色只表征实际输入 owner: C0 白、C1 绿、C3 蓝、NONE 黄。
* pending 呼吸; 目标不可达时黄快闪。目标无效仅插入短红缺口, 速度无效降亮。 */
{
    uint8_t bright = velValid ? RGB_MAX_BRIGHT_PCT : RGB_DIM_PCT;
    uint8_t r = 0u;
    uint8_t g = 0u;
    uint8_t b = 0u;
    uint8_t shown = owner;

    if (!targetValid && ((rgbTick % 100u) < 8u))
    {
        setRgb(bright, 0u, 0u);
        return;
    }
    if ((owner == CORE1_KVM_DISPLAY_TARGET_NONE) &&
        (desired != CORE1_KVM_DISPLAY_TARGET_NONE))
    {
        uint8_t on = flashOn(10u);
        setRgb(on ? bright : 0u, on ? bright : 0u, 0u);
        return;
    }
    if (desired != owner)
    {
        shown = desired;
        bright = (uint8_t)((uint16_t) bright * breathePct(60u) / 100u);
    }
    if (shown == CORE1_KVM_DISPLAY_TARGET_CORE0) r = g = b = bright;
    else if (shown == CORE1_KVM_DISPLAY_TARGET_CORE1) g = bright;
    else if (shown == CORE1_KVM_DISPLAY_TARGET_CORE3) b = bright;
    else r = g = bright;
    setRgb(r, g, b);
}
}

void rgbStatusLedSetHardwareInitFault(uint8_t active)
{
    hardwareInitFaultActive = active ? 1u : 0u;
}

```



```
void rgbStatusLedPublishCore0Facts(uint8_t ready, uint8_t linkUp,
    uint8_t statusValid, uint8_t safetyState, uint8_t statusFlags,
    uint8_t rgbIntent)
{
    core0FactsPacked = ((uint32_t) (ready != 0u)) |
        ((uint32_t) (linkUp != 0u) << 1) |
        ((uint32_t) (statusValid != 0u) << 2) |
        ((uint32_t) safetyState << 8) |
        ((uint32_t) statusFlags << 16) |
        ((uint32_t) rgbIntent << 24);
}
```

core1_cyt4bb7/project/code/app/rgbStatusLed.h

SHA-256

:

cd00f88039ff774eae80553af8794fbdb4ced0793751e50d2de7e8fa01adf0e0

代码（c）

```
#ifndef_RGB_STATUS_LED_H_
#define_RGB_STATUS_LED_H_

#include <stdint.h>

/*
 * rgbStatusLed - core1 cm_7_1 RGB 状态灯（两核共用状态标志硬件）
 *
 * 硬件: P05_0(B)/P05_1(R)/P05_2(G) → TCPWM CH09/CH10/CH11，共阳极低电平点亮。
 * 占空比越小越亮(duty 0=全亮, 10 000=灭)。最大亮度限 30%(RGB_MAX_BRIGHT_PCT)。
 *
 * 状态来源(综合 core0+core1 整机):
 * - 本地(cm_7_1 直接读): initReadyFlag / menuState / beaconCount
 * - core0(由前台路由器发布一份打包事实): linkUp / core0SafetyState /
 *   core0StatusFlags(cm70Ready/velValid/targetValid/tofHealthy)
 *
 * 双维编码(用户要求): 飞行/待命档用 颜色(^灯板目标有效=绿/无=红) + 明暗(速度估计有效=亮/失效=暗)
 * 同时表达两个飞控健康维度。其余档用颜色+动效(常亮/闪烁/呼吸)区分。
 *
 * 全部经 core1 cm_7_1 的 PIT_CH2 10ms tick(pitProcess0)调用, 稳定节拍软生成动效。
 */

/* 初始化三路 PWM(默认灭)。须在 pwm 相关时钟就绪后调用。 */
void rgbStatusLedInit(void);
void rgbStatusLedSetHardwareInitFault(uint8_t active);
void rgbStatusLedPublishCore0Facts(uint8_t ready, uint8_t linkUp,
    uint8_t statusValid, uint8_t safetyState, uint8_t statusFlags,
    uint8_t rgbIntent);

/* 刷新一次(PIT_CH2 10ms ISR 每 tick 调)。内部只读本地状态和前台原子发布的打包事实，
 * 不在 ISR 访问跨核共享区；软生成动效并设置三路占空比。
 * core1Ready: cm_7_1 初始化完成标志(initReadyFlag)
 * menuStateVal: 当前菜单态(menuStateEnum, 仅用于区分 WAIT_LINK 等; 传 uint8_t 避免头依赖)
 * beaconCount: 当前确认信标数 */
void rgbStatusLedUpdate(uint8_t core1Ready, uint8_t menuStateVal, uint16_t beaconCount);

#endif /*_RGB_STATUS_LED_H_*/
```

core1_cyt4bb7/project/code/comm/chassisLink.c

SHA-256

:

294c62c30dd841ce3f45383f48127787ee58c0122cd3637c0ea197057829ec18

代码（c）

```
#include "chassisLink.h"
#include "chassisLinkInternal.h"
#include "chassisLinkWireInternal.h"

#include <string.h>
```

```

#include "THVD1450.h"
#include "chassisParamProtocol.h"

/*
 * core1 端 RS-485 命令发送 (core1→core2) 。 core1 作总线主，周期发 MSG_COMMAND。
 * 帧布局与 core2 chassisLink.c 解析侧逐字节对齐 (SYNC)：
 * payload[0]=0x02, [1:4]=vx, [5:8]=vy, [9:12]=wz (float32 小端)，[13]=cmd 位，[14]=sysByte。
 */

#define CHASSIS_LINK_MOTION_ABS_MAX (200.0f)

static chassisParamFrame paramResponse;
static uint8_t paramResponsePending;
static uint8_t paramGlobalStateBytes[CHASSIS_PARAM_GLOBAL_STATE_BYTES];
static uint8_t paramGlobalRequestBytes[CHASSIS_PARAM_GLOBAL_REQUEST_BYTES];
static paramSyncGlobalRequest paramGlobalRequest;
static uint32_t paramGlobalStateSequence;
static uint8_t paramGlobalStateStreamSequence;
static uint8_t paramGlobalStateFragmentIndex;
static uint8_t paramGlobalStateCyclePending;
static uint8_t paramGlobalRequestStreamSequence;
static uint8_t paramGlobalRequestMask;
static uint8_t paramGlobalRequestPending;

uint8_t chassisLinkMotionValid(float vx, float vy, float wz)
{
    return (vx > -CHASSIS_LINK_MOTION_ABS_MAX && vx < CHASSIS_LINK_MOTION_ABS_MAX && vy > -CHASSIS_LINK_MOTION_ABS_MAX
    && vy < CHASSIS_LINK_MOTION_ABS_MAX && wz > -CHASSIS_LINK_MOTION_ABS_MAX && wz < CHASSIS_LINK_MOTION_ABS_MAX) ? 1u :
    0u;
}

uint8_t chassisLinkDecodeReply(const thvd1450FrameStruct *frame,
    chassisLinkReplyStruct *decoded)
{
    uint8_t type;
    chassisLinkReplyStruct candidate;
    if ((frame == NULL) || (decoded == NULL)) return 0u;
    memset(decoded, 0, sizeof(*decoded));
    memset(&candidate, 0, sizeof(candidate));
    if (frame->length == 0u) return 0u;
    type = frame->payload[CHASSIS_LINK_TYPE_OFFSET];

    if (frame->length >= CHASSIS_LINK_LORA_BASE_LENGTH && type == CHASSIS_LINK_MSG_LORA_MOTION)
    {
        chassisLinkWireDecodeVector(frame->payload,
            &candidate.vx, &candidate.vy, &candidate.wz);
        candidate.motionValid = chassisLinkMotionValid(candidate.vx, candidate.vy, candidate.wz);
        if (candidate.motionValid == 0u) return 0u;
        if (frame->length >= CHASSIS_LINK_LORA_LENGTH)
        {
            uint8_t flags = frame->payload[CHASSIS_LINK_CONTROL_OFFSET];
            const uint8_t allowedFlags = CHASSIS_LINK_LORA_FLAG_TAKEOFF_REQ |
                CHASSIS_LINK_LORA_FLAG_ATTITUDE_FAULT;
            if ((flags & (uint8_t)~allowedFlags) != 0u) return 0u;
            candidate.remoteTakeoffReq = (flags & CHASSIS_LINK_LORA_FLAG_TAKEOFF_REQ) ? 1u : 0u;
            candidate.attitudeFault =
                (flags & CHASSIS_LINK_LORA_FLAG_ATTITUDE_FAULT) ? 1u : 0u;
        }
        candidate.replyType = CHASSIS_LINK_MSG_LORA_MOTION;
        *decoded = candidate;
        return 1u;
    }
    else if (frame->length >= CHASSIS_LINK_TELEMETRY_LENGTH && type == CHASSIS_LINK_MSG_TELEMETRY)
    {
        uint8_t status = frame->payload[CHASSIS_LINK_CONTROL_OFFSET];
        chassisLinkWireDecodeVector(frame->payload,
            &candidate.vx, &candidate.vy, &candidate.wz);
        if (chassisLinkMotionValid(candidate.vx, candidate.vy, candidate.wz) == 0u) return 0u;
        if ((status & (uint8_t)~(CHASSIS_LINK_STATUS_ENABLED |
            CHASSIS_LINK_STATUS_LINK_OK | CHASSIS_LINK_STATUS_ESTOP |
            CHASSIS_LINK_STATUS_LAUNCHED |
            CHASSIS_LINK_STATUS_MODE2_LORA_LAND_REQ |
            CHASSIS_LINK_STATUS_ATTITUDE_FAULT)) != 0u) return 0u;
        candidate.mode2LoraLandReqPresent = 1u;
        candidate.mode2LoraLandReqLevel =
            (status & CHASSIS_LINK_STATUS_MODE2_LORA_LAND_REQ) ? 1u : 0u;
        candidate.attitudeFault =
            (status & CHASSIS_LINK_STATUS_ATTITUDE_FAULT) ? 1u : 0u;
        candidate.replyType = CHASSIS_LINK_MSG_TELEMETRY;
    }
}

```

```

        candidate.status = status;
        *decoded = candidate;
        return 1u;
    }
    return 0u;
}

void chassisLinkInit(void)
{
    thvd1450Init();
    paramResponsePending = 0u;
    memset(paramGlobalStateBytes, 0, sizeof(paramGlobalStateBytes));
    memset(paramGlobalRequestBytes, 0, sizeof(paramGlobalRequestBytes));
    memset(&paramGlobalRequest, 0, sizeof(paramGlobalRequest));
    paramGlobalStateSequence = 0u;
    paramGlobalStateStreamSequence = 0u;
    paramGlobalStateFragmentIndex = 0u;
    paramGlobalStateCyclePending = 0u;
    paramGlobalRequestStreamSequence = 0u;
    paramGlobalRequestMask = 0u;
    paramGlobalRequestPending = 0u;
}

uint8_t chassisLinkPollReply(chassisLinkReplyStruct *out)
{
    thvd1450FrameStruct frame;
    if (out == NULL) return 0u;
    thvd1450Poll();
    while (thvd1450ReadFrame(&frame) == THVD1450_STATUS_OK)
    {
        chassisParamFrame decodedParam;
        chassisParamGlobalFragment globalFragment;
        if (chassisParamDecode(frame.payload, frame.length,
            CHASSIS_LINK_MSG_PARAM_RESPONSE, &decodedParam) != 0u)
        {
            paramResponse = decodedParam;
            paramResponsePending = 1u;
            continue;
        }
        if (chassisParamGlobalFragmentDecode(frame.payload, frame.length,
            CHASSIS_LINK_MSG_PARAM_GLOBAL_REQUEST,
            &globalFragment) != 0u)
        {
            uint8_t expectedCount = (uint8_t)
                ((CHASSIS_PARAM_GLOBAL_REQUEST_BYTES +
                 CHASSIS_PARAM_GLOBAL_DATA_BYTES - 1u) /
                 CHASSIS_PARAM_GLOBAL_DATA_BYTES);
            uint8_t offset = (uint8_t) (globalFragment.fragmentIndex *
                CHASSIS_PARAM_GLOBAL_DATA_BYTES);
            if ((globalFragment.fragmentCount != expectedCount) ||
                ((uint16_t) offset + globalFragment.length >
                 CHASSIS_PARAM_GLOBAL_REQUEST_BYTES))
            {
                paramGlobalRequestMask = 0u;
                continue;
            }
            if (globalFragment.streamSequence !=
                paramGlobalRequestStreamSequence)
            {
                paramGlobalRequestStreamSequence =
                    globalFragment.streamSequence;
                paramGlobalRequestMask = 0u;
            }
            memcpy(&paramGlobalRequestBytes[offset], globalFragment.data,
                globalFragment.length);
            paramGlobalRequestMask |= (uint8_t)
                (1u << globalFragment.fragmentIndex);
            if (paramGlobalRequestMask ==
                (uint8_t) ((1u << expectedCount) - 1u))
            {
                if (paramSyncGlobalDecodeRequest(paramGlobalRequestBytes,
                    &paramGlobalRequest) != 0u)
                {
                    paramGlobalRequestPending = 1u;
                    paramGlobalRequestMask = 0u;
                }
                continue;
            }
            if (chassisLinkDecodeReply(&frame, out) != 0u) return 1u;
        }
    }
}

```

```

        return 0u;
    }

uint8_t chassisLinkSendParamGlobalState(const paramSyncGlobalState *state)
{
    chassisParamGlobalFragment fragment;
    uint8_t payload[CHASSIS_PARAM_GLOBAL_FRAME_BYTES];
    uint8_t count = (uint8_t)((CHASSIS_PARAM_GLOBAL_STATE_BYTES +
        CHASSIS_PARAM_GLOBAL_DATA_BYTES - 1u) /
        CHASSIS_PARAM_GLOBAL_DATA_BYTES);
    uint8_t offset;
    uint8_t remaining;
    if ((state == 0) || (paramSyncGlobalStateIsValid(state) == 0u)) return 0u;
    if (state->sequence != paramGlobalStateSequence)
    {
        paramSyncGlobalEncodeState(state, paramGlobalStateBytes);
        paramGlobalStateSequence = state->sequence;
        paramGlobalStateFragmentIndex = 0u;
        paramGlobalStateStreamSequence++;
        if (paramGlobalStateStreamSequence == 0u)
            paramGlobalStateStreamSequence = 1u;
        paramGlobalStateCyclePending = 1u;
    }
    if (paramGlobalStateCyclePending == 0u) return 0u;
    memset(&fragment, 0, sizeof(fragment));
    fragment.streamSequence = paramGlobalStateStreamSequence;
    fragment.fragmentIndex = paramGlobalStateFragmentIndex;
    fragment.fragmentCount = count;
    offset = (uint8_t)(fragment.fragmentIndex * CHASSIS_PARAM_GLOBAL_DATA_BYTES);
    remaining = (uint8_t)(CHASSIS_PARAM_GLOBAL_STATE_BYTES - offset);
    fragment.length = (remaining > CHASSIS_PARAM_GLOBAL_DATA_BYTES) ?
        CHASSIS_PARAM_GLOBAL_DATA_BYTES : remaining;
    memcpy(fragment.data, &paramGlobalStateBytes[offset], fragment.length);
    if ((chassisParamGlobalFragmentEncode(
        CHASSIS_LINK_MSG_PARAM_GLOBAL_STATE, &fragment, payload) == 0u) ||
        (thvd1450SendBytes(payload, sizeof(payload)) != THVD1450_STATUS_OK))
        return 0u;
    paramGlobalStateFragmentIndex++;
    if (paramGlobalStateFragmentIndex >= count)
    {
        paramGlobalStateFragmentIndex = 0u;
        paramGlobalStateCyclePending = 0u;
    }
    return 1u;
}

uint8_t chassisLinkTryReadParamGlobalRequest(paramSyncGlobalRequest *request)
{
    if ((request == 0) || (paramGlobalRequestPending == 0u)) return 0u;
    if (paramSyncGlobalRequestIsValid(&paramGlobalRequest) == 0u)
    {
        paramGlobalRequestPending = 0u;
        return 0u;
    }
    *request = paramGlobalRequest;
    paramGlobalRequestPending = 0u;
    return 1u;
}

uint8_t chassisLinkSendParamRequest(const chassisParamFrame *request)
{
    uint8_t payload[CHASSIS_PARAM_FRAME_BYTES];
    if (chassisParamEncode(CHASSIS_LINK_MSG_PARAM_REQUEST, request, payload) == 0u)
        return 0u;
    return (thvd1450SendBytes(payload, CHASSIS_PARAM_FRAME_BYTES) ==
        THVD1450_STATUS_OK)
        ? 1u
        : 0u;
}

uint8_t chassisLinkTryReadParamResponse(chassisParamFrame *response)
{
    if ((response == 0) || (paramResponsePending == 0u)) return 0u;
    *response = paramResponse;
    paramResponsePending = 0u;
    return 1u;
}

void chassisLinkSendCommand(float vx, float vy, float wz, uint8_t cmd, uint8_t sysByte)

```

```
{
    uint8_t payload[CHASSIS_LINK_COMMAND_LENGTH];
    chassisLinkWireEncodeVector(payload, CHASSIS_LINK_MSG_COMMAND, vx, vy, wz);
    payload[CHASSIS_LINK_CONTROL_OFFSET] = cmd;
    payload[CHASSIS_LINK_SYSBYTE_OFFSET] = sysByte; /* bit0 控制源(0=485/1=LoRa) + bit1-3 蜂鸣模式码 */
    /* thvd1450SendBytes 内部封帧(7A|len|payload|xor|55) + 半双工 DE 自管理, 阻塞至发完切回收。 */
    (void) thvd1450SendBytes(payload, CHASSIS_LINK_COMMAND_LENGTH);
}
```

core1_cyt4bb7/project/code/comm/chassisLink.h

SHA-256

:

b1f5deb07c4cbcfabb36b56f2e1944cb28f955bb400a6a5385c2cef06155eb96

代码 (c)

```
#ifndef _CHASSIS_LINK_H_
#define _CHASSIS_LINK_H_

#include <stdint.h>

#include "chassisLinkProtocol.h"
#include "chassisParamProtocol.h"

typedef struct
{
    float vx;
    float vy;
    float wz;
    uint8_t motionValid;
    uint8_t remoteTakeoffReq;
    uint8_t mode2LoraLandReqPresent;
    uint8_t mode2LoraLandReqLevel;
    uint8_t attitudeFault;
    uint8_t replyType;
    uint8_t status;
} chassisLinkReplyStruct;

/*
 * core1 端 RS-485 链路层 (core1 ↔ core2, 基于 THVD1450 帧层)。core1 是总线主:
 * 发 MSG_COMMAND(core1 → core2): 目标 vx/vy/wz + 指令字(ENABLE/ESTOP/LAUNCH)。
 * 收 MSG_TELEMETRY / MSG_LORA_MOTION(core2 → core1): chassisLinkInternal 单帧解码, core0Loop 前台聚合。
 *
 * 请求-应答: core1 周期发 MSG_COMMAND 作“轮询”, core2 收到有效命令后才回一帧
 * (REMOTE_485 → 遥测 / REMOTE_LORA → LoRa 遥控值), 半双工总线串行化、避免碰撞。
 *
 * 协议常量与帧布局须与 core2 chassisLink.h 逐字节同源 (SYNC: 改一处必同步另一处)。
 * payload[0]=消息类型; 其后小端 float32 + 指令/状态字节(手填, 不用 PackFloats 以容纳类型字节)。
 */

/* 发送 MSG_COMMAND (15B payload: [0x02|vx(4)|vy(4)|wz(4)|cmd|sysByte], float32 小端)。
 * sysByte = 控制源位(CHASSIS_LINK_SYS_SOURCE_LORA) | (蜂鸣码 << CHASSIS_LINK_SYS_BUZZER_SHIFT)。
 * 半双工阻塞发送, DE 由 THVD1450 自管理; 发完自动切回收以接 core2 的应答帧。 */
void chassisLinkSendCommand(float vx, float vy, float wz, uint8_t cmd, uint8_t sysByte);
void chassisLinkInit(void);
uint8_t chassisLinkPollReply(chassisLinkReplyStruct *out);
uint8_t chassisLinkMotionValid(float vx, float vy, float wz);
uint8_t chassisLinkSendParamRequest(const chassisParamFrame *request);
uint8_t chassisLinkTryReadParamResponse(chassisParamFrame *response);
uint8_t chassisLinkSendParamGlobalState(const paramSyncGlobalState *state);
uint8_t chassisLinkTryReadParamGlobalRequest(paramSyncGlobalRequest *request);

#endif /* _CHASSIS_LINK_H_ */
```

core1_cyt4bb7/project/code/comm/chassisLinkInternal.h

SHA-256

:

f9e3e67cb20360fee34f8bff8205e17e1830147bc1baa32945afb91f5407c2de

代码 (c)

```
#ifndef CHASSIS_LINK_INTERNAL_H
#define CHASSIS_LINK_INTERNAL_H

#include <stdint.h>

#include "chassisLink.h"
#include "THVD1450_protocol.h"

uint8_t chassisLinkDecodeReply(const thvd1450FrameStruct* frame,
    chassisLinkReplyStruct* decoded);

#endif
```

core1_cyt4bb7/project/code/comm/chassisLinkProtocol.h

SHA-256 : 9ad1367a6cbe7fb9c9c9a61ae4dc8d2103ecc74ef0be36c0eb59aa948ad3a5af

代码 (c)

```
#ifndef CHASSIS_LINK_PROTOCOL_H
#define CHASSIS_LINK_PROTOCOL_H

#include <stdint.h>

#define CHASSIS_LINK_MSG_TELEMETRY (0x01u)
#define CHASSIS_LINK_MSG_COMMAND (0x02u)
#define CHASSIS_LINK_MSG_LORA_MOTION (0x03u)

#define CHASSIS_LINK_TYPE_OFFSET (0u)
#define CHASSIS_LINK_VX_OFFSET (1u)
#define CHASSIS_LINK_VY_OFFSET (5u)
#define CHASSIS_LINK_WZ_OFFSET (9u)
#define CHASSIS_LINK_CONTROL_OFFSET (13u)
#define CHASSIS_LINK_SYSBYTE_OFFSET (14u)

#define CHASSIS_LINK_LORA_BASE_LENGTH (13u)
#define CHASSIS_LINK_LORA_LENGTH (14u)
#define CHASSIS_LINK_TELEMETRY_LENGTH (14u)
#define CHASSIS_LINK_COMMAND_BASE_LENGTH (14u)
#define CHASSIS_LINK_COMMAND_LENGTH (15u)

#define CHASSIS_LINK_LORA_FLAG_TAKEOFF_REQ (0x01u)
#define CHASSIS_LINK_LORA_FLAG_ATTITUDE_FAULT (0x02u)

#define CHASSIS_LINK_STATUS_ENABLED (0x01u)
#define CHASSIS_LINK_STATUS_LINK_OK (0x02u)
#define CHASSIS_LINK_STATUS_ESTOP (0x04u)
#define CHASSIS_LINK_STATUS_LAUNCHED (0x08u)
#define CHASSIS_LINK_STATUS_MODE2_LORA_LAND_REQ (0x10u)
#define CHASSIS_LINK_STATUS_ATTITUDE_FAULT (0x20u)
#define CHASSIS_LINK_CMD_ESTOP (0x01u)
#define CHASSIS_LINK_CMD_ENABLE (0x02u)
#define CHASSIS_LINK_CMD_LAUNCH (0x08u)
#define CHASSIS_LINK_CMD_CORE0_ALIVE (0x10u)

#define CHASSIS_LINK_SYS_SOURCE_LORA (0x01u)
#define CHASSIS_LINK_SYS_BUZZER_SHIFT (1u)
#define CHASSIS_LINK_SYS_BUZZER_MASK (0x07u)

#define CHASSIS_LINK_BUZZER_SILENT (0u)
#define CHASSIS_LINK_BUZZER_1HZ (1u)
#define CHASSIS_LINK_BUZZER_LONG (2u)
#define CHASSIS_LINK_BUZZER_3S (3u)
#define CHASSIS_LINK_BUZZER_WARN (4u)
#define CHASSIS_LINK_BUZZER_ARMED_IDLE (5u)
#define CHASSIS_LINK_BUZZER_WAITING (6u)
#define CHASSIS_LINK_BUZZER_THREE_BEEPS (7u)

typedef char chassisLink_float_size_assert[(sizeof(float) == 4u) ? 1 : -1];

#endif
```

core1_cyt4bb7/project/code/comm/chassisLinkWireInternal.h

SHA-256

:

a5d3b8eef02b0730d1659ecf4a429dea95901503427c84a0474bf38daf506cf0

代码 (c)

```
#ifndef CHASSIS_LINK_WIRE_INTERNAL_H
#define CHASSIS_LINK_WIRE_INTERNAL_H

#include <string.h>

#include "chassisLinkProtocol.h"

static void chassisLinkWireEncodeVector(uint8_t* payload, uint8_t type,
    float vx, float vy, float wz)
{
    payload[CHASSIS_LINK_TYPE_OFFSET] = type;
    memcpy(&payload[CHASSIS_LINK_VX_OFFSET], &vx, sizeof(vx));
    memcpy(&payload[CHASSIS_LINK_VY_OFFSET], &vy, sizeof(vy));
    memcpy(&payload[CHASSIS_LINK_WZ_OFFSET], &wz, sizeof(wz));
}

static void chassisLinkWireDecodeVector(const uint8_t* payload,
    float* vx, float* vy, float* wz)
{
    memcpy(vx, &payload[CHASSIS_LINK_VX_OFFSET], sizeof(*vx));
    memcpy(vy, &payload[CHASSIS_LINK_VY_OFFSET], sizeof(*vy));
    memcpy(wz, &payload[CHASSIS_LINK_WZ_OFFSET], sizeof(*wz));
}

#endif
```

core1_cyt4bb7/project/code/comm/core3LinkCore1.c

SHA-256

:

c418697d59052d8630a83e2f37160aeb62c6d7b5be58c0d75b20ff58b63a3385

代码 (c)

```
#include "core3LinkCore1.h"

#include "cy_device_headers.h"
#include "observerLinkConfigCore1.h"
#include "zf_driver_uart.h"
#include "pinMap.h"

static observerLinkCore1EndpointContext g_core3Endpoint;

static const observerLinkCore1EndpointDescriptor g_core3Descriptor = {
    OBSERVER_CORE3_UART_NUM,
    OBSERVER_CORE3_UART_RX_PIN,
    OBSERVER_CORE3_UART_TX_PIN,
    OBSERVER_CORE3_UART_SCB,
    OBSERVER_LINK_ROLE_CORE3,
    OBSERVER_LINK_SOURCE_CORE3,
    OBSERVER_LINK_KVM_TARGET_CORE3,
    0u,
    observerLinkConfigCore1Provide,
    0};

void core3LinkCore1Init(uint32_t tickMs)
{
    (void) observerLinkCore1EndpointInit(&g_core3Endpoint, &g_core3Descriptor, tickMs);
}

void core3LinkCore1Poll(uint32_t tickMs)
{
    observerLinkCore1EndpointPoll(&g_core3Endpoint, tickMs);
}

void core3LinkCore1DrainRx(uint32_t tickMs)
```

```

{
    observerLinkCore1EndpointDrainRx(&g_core3Endpoint, tickMs);
}
void core3LinkCore1FlushControl(uint32_t tickMs)
{
    observerLinkCore1EndpointFlushControl(&g_core3Endpoint, tickMs);
}
void core3LinkCore1FlushTx(uint32_t tickMs)
{
    observerLinkCore1EndpointFlushTx(&g_core3Endpoint, tickMs);
}
void core3LinkCore1RxIsr(void)
{
    observerLinkCore1EndpointRxIsr(&g_core3Endpoint);
}
void core3LinkCore1GetStatus(core3LinkCore1Status *status)
{
    observerLinkCore1EndpointGetStatus(&g_core3Endpoint, status);
}
void core3LinkCore1FencePlanningFacts(void)
{
    observerLinkCore1EndpointFencePlanningFacts(&g_core3Endpoint);
}
void core3LinkCore1SyncDesiredIdentity(void)
{
    (void) observerLinkCore1EndpointSyncConfig(&g_core3Endpoint);
}
void core3LinkCore1PublishControlStatus(const observerLinkControlStatus *status, uint32_t tickMs)
{
    observerLinkCore1EndpointPublishControlStatus(&g_core3Endpoint, status, tickMs);
}
uint8_t core3LinkCore1QueueViewOverlay(const observerLinkViewOverlay *overlay, uint32_t tickMs)
{
    return observerLinkCore1EndpointQueueViewOverlay(&g_core3Endpoint, overlay, tickMs);
}
uint8_t core3LinkCore1QueueKvmControl(const observerLinkKvmControl *control)
{
    return observerLinkCore1EndpointQueueKvmControl(&g_core3Endpoint, control);
}
uint8_t core3LinkCore1QueueKvmEvent(const observerLinkKvmEvent *event)
{
    return observerLinkCore1EndpointQueueKvmEvent(&g_core3Endpoint, event);
}
uint8_t core3LinkCore1TakeKvmAck(observerLinkKvmAck *ack)
{
    return observerLinkCore1EndpointTakeKvmAck(&g_core3Endpoint, ack);
}
uint8_t core3LinkCore1GetKvmMenuStatus(observerLinkKvmMenuStatus *status, uint32_t tickMs, uint32_t *ageMs)
{
    return observerLinkCore1EndpointGetKvmMenuStatus(&g_core3Endpoint, status, tickMs, ageMs);
}
uint8_t core3LinkCore1GetPlanningTrackSnapshot(observerLinkPlanningTrackSnapshot *snapshot, uint32_t tickMs, uint32_t timeoutMs, uint32_t *ageMs)
{
    return observerLinkCore1EndpointGetPlanningTrackSnapshot(&g_core3Endpoint, snapshot, tickMs, timeoutMs, ageMs);
}
uint8_t core3LinkCore1QueueParamRequest(const uint8_t *payload, uint16_t length)
{
    return observerLinkCore1EndpointQueueParamRequest(&g_core3Endpoint, payload, length);
}
uint8_t core3LinkCore1TakeParamResponse(uint8_t *payload, uint16_t capacity, uint16_t *length)
{
    return observerLinkCore1EndpointTakeParamResponse(&g_core3Endpoint, payload, capacity, length);
}

uint8_t core3LinkCore1QueueParamGlobalState(
    const paramSyncGlobalState *state)
{
    return observerLinkCore1EndpointQueueParamGlobalState(
        &g_core3Endpoint, state);
}

uint8_t core3LinkCore1TakeParamGlobalRequest(
    paramSyncGlobalRequest *request)
{
    return observerLinkCore1EndpointTakeParamGlobalRequest(
        &g_core3Endpoint, request);
}

```


core1_cyt4bb7/project/code/comm/core3LinkCore1.h

SHA-256

:

2ec4fa6f32ed5c2f2205a480a8e18b69f337b72d28918e439c5d99009a7c8148

代码 (c)

```
#ifndef CORE3_LINK_CORE1_H
#define CORE3_LINK_CORE1_H

#include "observerLinkCore1Endpoint.h"

typedef observerLinkCore1EndpointStatus core3LinkCore1Status;

void core3LinkCore1Init(uint32_t tickMs);
void core3LinkCore1Poll(uint32_t tickMs);
void core3LinkCore1DrainRx(uint32_t tickMs);
void core3LinkCore1FlushControl(uint32_t tickMs);
void core3LinkCore1FlushTx(uint32_t tickMs);
void core3LinkCore1RxIsr(void);
void core3LinkCore1GetStatus(core3LinkCore1Status *status);
void core3LinkCore1FencePlanningFacts(void);
void core3LinkCore1SyncDesiredIdentity(void);
void core3LinkCore1PublishControlStatus(
    const observerLinkControlStatus *status, uint32_t tickMs);
uint8_t core3LinkCore1QueueViewOverlay(
    const observerLinkViewOverlay *overlay, uint32_t tickMs);
uint8_t core3LinkCore1QueueKvmControl(const observerLinkKvmControl *control);
uint8_t core3LinkCore1QueueKvmEvent(const observerLinkKvmEvent *event);
uint8_t core3LinkCore1TakeKvmAck(observerLinkKvmAck *ack);
uint8_t core3LinkCore1GetKvmMenuStatus(observerLinkKvmMenuStatus *status,
    uint32_t tickMs, uint32_t *ageMs);
uint8_t core3LinkCore1GetPlanningTrackSnapshot(
    observerLinkPlanningTrackSnapshot *snapshot,
    uint32_t tickMs, uint32_t timeoutMs, uint32_t *ageMs);
uint8_t core3LinkCore1QueueParamRequest(const uint8_t *payload, uint16_t length);
uint8_t core3LinkCore1TakeParamResponse(uint8_t *payload,
    uint16_t capacity, uint16_t *length);
uint8_t core3LinkCore1QueueParamGlobalState(
    const paramSyncGlobalState *state);
uint8_t core3LinkCore1TakeParamGlobalRequest(
    paramSyncGlobalRequest *request);
#endif /* CORE3_LINK_CORE1_H */
```

core1_cyt4bb7/project/code/comm/core4LinkCore1.c

SHA-256

:

6a48e136ddae2d2d1b57fb2bead184e39e985a59c0c374d990c1ec01a59e6a17

代码 (c)

```
#include "core4LinkCore1.h"

#include "cy_device_headers.h"
#include "observerLinkConfigCore1.h"
#include "zf_driver_uart.h"
#include "pinMap.h"

static observerLinkCore1EndpointContext g_core4Endpoint;

static const observerLinkCore1EndpointDescriptor g_core4Descriptor = {
    OBSERVER_CORE4_UART_NUM,
    OBSERVER_CORE4_UART_RX_PIN,
    OBSERVER_CORE4_UART_TX_PIN,
    OBSERVER_CORE4_UART_SCB,
    OBSERVER_LINK_ROLE_CORE4,
    OBSERVER_LINK_SOURCE_CORE4,
    OBSERVER_LINK_KVM_TARGET_CORE4,
    0u,
    observerLinkConfigCore1Provide,
```

```

    0};

void core4LinkCore1Init(uint32_t tickMs)
{
    (void) observerLinkCore1EndpointInit(&g_core4Endpoint, &g_core4Descriptor, tickMs);
}
void core4LinkCore1Poll(uint32_t tickMs)
{
    observerLinkCore1EndpointPoll(&g_core4Endpoint, tickMs);
}
void core4LinkCore1DrainRx(uint32_t tickMs)
{
    observerLinkCore1EndpointDrainRx(&g_core4Endpoint, tickMs);
}
void core4LinkCore1FlushControl(uint32_t tickMs)
{
    observerLinkCore1EndpointFlushControl(&g_core4Endpoint, tickMs);
}
void core4LinkCore1FlushTx(uint32_t tickMs)
{
    observerLinkCore1EndpointFlushTx(&g_core4Endpoint, tickMs);
}
void core4LinkCore1RxIsr(void)
{
    observerLinkCore1EndpointRxIsr(&g_core4Endpoint);
}
void core4LinkCore1GetStatus(core4LinkCore1Status *status)
{
    observerLinkCore1EndpointGetStatus(&g_core4Endpoint, status);
}
void core4LinkCore1FencePlanningFacts(void)
{
    observerLinkCore1EndpointFencePlanningFacts(&g_core4Endpoint);
}
void core4LinkCore1SyncDesiredIdentity(void)
{
    (void) observerLinkCore1EndpointSyncConfig(&g_core4Endpoint);
}
void core4LinkCore1PublishControlStatus(const observerLinkControlStatus *status, uint32_t tickMs)
{
    observerLinkCore1EndpointPublishControlStatus(&g_core4Endpoint, status, tickMs);
}
uint8_t core4LinkCore1QueueViewOverlay(const observerLinkViewOverlay *overlay, uint32_t tickMs)
{
    return observerLinkCore1EndpointQueueViewOverlay(&g_core4Endpoint, overlay, tickMs);
}
uint8_t core4LinkCore1QueueKvmControl(const observerLinkKvmControl *control)
{
    return observerLinkCore1EndpointQueueKvmControl(&g_core4Endpoint, control);
}
uint8_t core4LinkCore1QueueKvmEvent(const observerLinkKvmEvent *event)
{
    return observerLinkCore1EndpointQueueKvmEvent(&g_core4Endpoint, event);
}
uint8_t core4LinkCore1TakeKvmAck(observerLinkKvmAck *ack)
{
    return observerLinkCore1EndpointTakeKvmAck(&g_core4Endpoint, ack);
}
uint8_t core4LinkCore1GetKvmMenuStatus(observerLinkKvmMenuStatus *status, uint32_t tickMs, uint32_t *ageMs)
{
    return observerLinkCore1EndpointGetKvmMenuStatus(&g_core4Endpoint, status, tickMs, ageMs);
}
uint8_t core4LinkCore1GetPlanningTrackSnapshot(observerLinkPlanningTrackSnapshot *snapshot, uint32_t tickMs, uint32_t timeoutMs, uint32_t *ageMs)
{
    return observerLinkCore1EndpointGetPlanningTrackSnapshot(&g_core4Endpoint, snapshot, tickMs, timeoutMs, ageMs);
}
uint8_t core4LinkCore1QueueParamRequest(const uint8_t *payload, uint16_t length)
{
    return observerLinkCore1EndpointQueueParamRequest(&g_core4Endpoint, payload, length);
}
uint8_t core4LinkCore1TakeParamResponse(uint8_t *payload, uint16_t capacity, uint16_t *length)
{
    return observerLinkCore1EndpointTakeParamResponse(&g_core4Endpoint, payload, capacity, length);
}

uint8_t core4LinkCore1QueueParamGlobalState(
    const paramSyncGlobalState *state)
{
    return observerLinkCore1EndpointQueueParamGlobalState(

```

```

        &g_core4Endpoint, state);
    }

uint8_t core4LinkCore1TakeParamGlobalRequest(
    paramSyncGlobalRequest *request)
{
    return observerLinkCore1EndpointTakeParamGlobalRequest(
        &g_core4Endpoint, request);
}

```

core1_cyt4bb7/project/code/comm/core4LinkCore1.h

SHA-256

:

2e2994a120bbc15c0e00c1ead9f0e227cfd97aedb3041781fb46a72b03364ed2

代码 (c)

```

#ifndef CORE4_LINK_CORE1_H
#define CORE4_LINK_CORE1_H

#include "observerLinkCore1Endpoint.h"

typedef observerLinkCore1EndpointStatus core4LinkCore1Status;

void core4LinkCore1Init(uint32_t tickMs);
void core4LinkCore1Poll(uint32_t tickMs);
void core4LinkCore1DrainRx(uint32_t tickMs);
void core4LinkCore1FlushControl(uint32_t tickMs);
void core4LinkCore1FlushTx(uint32_t tickMs);
void core4LinkCore1RxIsr(void);
void core4LinkCore1GetStatus(core4LinkCore1Status *status);
void core4LinkCore1FencePlanningFacts(void);
void core4LinkCore1SyncDesiredIdentity(void);
void core4LinkCore1PublishControlStatus(
    const observerLinkControlStatus *status, uint32_t tickMs);
uint8_t core4LinkCore1QueueViewOverlay(
    const observerLinkViewOverlay *overlay, uint32_t tickMs);
uint8_t core4LinkCore1QueueKvmControl(const observerLinkKvmControl *control);
uint8_t core4LinkCore1QueueKvmEvent(const observerLinkKvmEvent *event);
uint8_t core4LinkCore1TakeKvmAck(observerLinkKvmAck *ack);
uint8_t core4LinkCore1GetKvmMenuStatus(observerLinkKvmMenuStatus *status,
    uint32_t tickMs, uint32_t *ageMs);
uint8_t core4LinkCore1GetPlanningTrackSnapshot(
    observerLinkPlanningTrackSnapshot *snapshot,
    uint32_t tickMs, uint32_t timeoutMs, uint32_t *ageMs);
uint8_t core4LinkCore1QueueParamRequest(const uint8_t *payload, uint16_t length);
uint8_t core4LinkCore1TakeParamResponse(uint8_t *payload,
    uint16_t capacity, uint16_t *length);
uint8_t core4LinkCore1QueueParamGlobalState(
    const paramSyncGlobalState *state);
uint8_t core4LinkCore1TakeParamGlobalRequest(
    paramSyncGlobalRequest *request);
#endif /* CORE4_LINK_CORE1_H */

```

core1_cyt4bb7/project/code/comm/coreDataTransfer.c

SHA-256

:

06c61e92c55bd072663b05ba7c6318bfd8b46ef2547376bf2e49db8f895274f7

代码 (c)

```

#include "coreDataTransfer.h"

#include "cy_device_headers.h"

#include <string.h>

#define CORE1_PLAN_CACHE_LINE_SIZE (32u)

/* #pragma location 把共享结构强制放共享 RAM 固定地址(__no_init 绝对放置),

```

```

/* 让链接器保留空间。照搬 core0 coreDataTransfer 机制。 */
#pragma location = CORE1_DATA_KEY_ADDR
__no_init volatile keyEventToMenuStruct keyEventRegion;

void coreDataWriteKeyEvent(const keyEventToMenuStruct* src)
{
    if (0 == src) return;
    memcpy((void*) &keyEventRegion, src, sizeof(keyEventToMenuStruct));
    SCB_CleanDCache_by_Addr((volatile void*) &keyEventRegion, sizeof(keyEventToMenuStruct));
}

uint8_t coreDataTryReadKeyEvent(keyEventToMenuStruct* dst)
{
    keyEventToMenuStruct first;
    keyEventToMenuStruct second;
    uint8_t attempt;
    if (0 == dst) return 0u;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        SCB_InvalidateDCache_by_Addr((volatile void*) &keyEventRegion,
            sizeof(keyEventToMenuStruct));
        memcpy(&first, (const void*) &keyEventRegion, sizeof(first));
        SCB_InvalidateDCache_by_Addr((volatile void*) &keyEventRegion,
            sizeof(keyEventToMenuStruct));
        memcpy(&second, (const void*) &keyEventRegion, sizeof(second));
        if (memcmp(&first, &second, sizeof(first)) == 0)
        {
            *dst = second;
            return 1u;
        }
    }
    return 0u;
}

void coreDataReadKeyEvent(keyEventToMenuStruct* dst)
{
    if (0 == dst) return;
    if (coreDataTryReadKeyEvent(dst) == 0u)
    {
        /* 兼容旧 consumer: 未迁移调用仍保持原 fail-closed 语义。 */
        memset(dst, 0, sizeof(*dst));
    }
}

/* core0 位姿区: cm_7_0 链路核 → cm_7_1 规划/显示核。 */
#pragma location = CORE1_DATA_C0POSE_ADDR
__no_init volatile core0PoseToVisionStruct core0PoseRegion;

#pragma location = CORE1_DATA_KVM_STATUS_ADDR
__no_init volatile core1KvmStatusStruct kvmStatusRegion;

#pragma location = CORE1_DATA_KVM_INGRESS_ADDR
__no_init volatile core1KvmIngressStruct kvmIngressRegion;

#pragma location = CORE1_DATA_PARAM_GLOBAL_REQUEST_ADDR
__no_init volatile core1ParamGlobalRequestStruct paramGlobalRequestRegion;

#pragma location = CORE1_DATA_PARAM_GLOBAL_STATE_ADDR
__no_init volatile core1ParamGlobalStateStruct paramGlobalStateRegion;

#pragma location = CORE1_DATA_PARAM_OBSERVER3_ADDR
__no_init volatile core1ParamObserverBridgeStruct paramObserver3Region;

#pragma location = CORE1_DATA_PARAM_OBSERVER4_ADDR
__no_init volatile core1ParamObserverBridgeStruct paramObserver4Region;

void coreDataWriteCore0Pose(const core0PoseToVisionStruct* src)
{
    if (0 == src) return;
    memcpy((void*) &core0PoseRegion, src, sizeof(core0PoseToVisionStruct));
    SCB_CleanDCache_by_Addr((volatile void*) &core0PoseRegion, sizeof(core0PoseToVisionStruct));
}

void coreDataReadCore0Pose(core0PoseToVisionStruct* dst)
{
    if (0 == dst) return;
    if (coreDataTryReadCore0Pose(dst) == 0u) memset(dst, 0, sizeof(*dst));
}

uint8_t coreDataTryReadCore0Pose(core0PoseToVisionStruct* dst)

```

```

{
    core0PoseToVisionStruct first;
    core0PoseToVisionStruct second;
    uint8_t attempt;
    if (dst == 0) return 0u;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        SCB_InvalidateDCache_by_Addr((volatile void*) &core0PoseRegion,
            sizeof(core0PoseToVisionStruct));
        memcpy(&first, (const void*) &core0PoseRegion, sizeof(first));
        SCB_InvalidateDCache_by_Addr((volatile void*) &core0PoseRegion,
            sizeof(core0PoseToVisionStruct));
        memcpy(&second, (const void*) &core0PoseRegion, sizeof(second));
        if (memcmp(&first, &second, sizeof(first)) == 0)
        {
            {
                *dst = second;
                return 1u;
            }
        }
    }
    return 0u;
}

void coreDataWriteKvmStatus(const core1KvmStatusStruct* src)
{
    if (0 == src) return;
    memcpy((void*) &kvmStatusRegion, src, sizeof(core1KvmStatusStruct));
    SCB_CleanDCache_by_Addr((volatile void*) &kvmStatusRegion, sizeof(core1KvmStatusStruct));
}

uint8_t coreDataTryReadKvmStatus(core1KvmStatusStruct* dst)
{
    core1KvmStatusStruct first;
    core1KvmStatusStruct second;
    uint8_t attempt;
    if (0 == dst) return 0u;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        SCB_InvalidateDCache_by_Addr((volatile void*) &kvmStatusRegion, sizeof(core1KvmStatusStruct));
        memcpy(&first, (const void*) &kvmStatusRegion, sizeof(first));
        SCB_InvalidateDCache_by_Addr((volatile void*) &kvmStatusRegion, sizeof(core1KvmStatusStruct));
        memcpy(&second, (const void*) &kvmStatusRegion, sizeof(second));
        if (memcmp(&first, &second, sizeof(first)) == 0)
        {
            {
                *dst = second;
                return 1u;
            }
        }
    }
    return 0u;
}

void coreDataWriteKvmIngress(const core1KvmIngressStruct* src)
{
    if (0 == src) return;
    memcpy((void*) &kvmIngressRegion, src, sizeof(core1KvmIngressStruct));
    SCB_CleanDCache_by_Addr((volatile void*) &kvmIngressRegion, sizeof(core1KvmIngressStruct));
}

void coreDataReadKvmIngress(core1KvmIngressStruct* dst)
{
    core1KvmIngressStruct first;
    core1KvmIngressStruct second;
    uint8_t attempt;
    if (0 == dst) return;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        SCB_InvalidateDCache_by_Addr((volatile void*) &kvmIngressRegion, sizeof(core1KvmIngressStruct));
        memcpy(&first, (const void*) &kvmIngressRegion, sizeof(first));
        SCB_InvalidateDCache_by_Addr((volatile void*) &kvmIngressRegion, sizeof(core1KvmIngressStruct));
        memcpy(&second, (const void*) &kvmIngressRegion, sizeof(second));
        if (memcmp(&first, &second, sizeof(first)) == 0)
        {
            {
                *dst = second;
                return;
            }
        }
    }
    memset(dst, 0, sizeof(*dst));
}

void coreDataWriteParamObserverBridge(

```

```

    const core1ParamObserverBridgeStruct *src)
{
    volatile core1ParamObserverBridgeStruct *region;
    if (coreDataParamObserverBridgeIsValid(src) == 0u) return;
    region = (src->source == 3u) ? &paramObserver3Region :
        &paramObserver4Region;
    memcpy((void *) region, src, sizeof(*src));
    SCB_CleanDCache_by_Addr((volatile void *) region, sizeof(*src));
    __DSB();
}

uint8_t coreDataTryReadParamObserverBridge(uint8_t source,
    core1ParamObserverBridgeStruct *dst)
{
    volatile core1ParamObserverBridgeStruct *region;
    core1ParamObserverBridgeStruct first, second;
    uint8_t attempt;
    if (dst == 0 || (source != 3u && source != 4u)) return 0u;
    region = (source == 3u) ? &paramObserver3Region :
        &paramObserver4Region;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        SCB_InvalidateDCache_by_Addr((volatile void *) region, sizeof(*dst));
        __DSB();
        memcpy(&first, (const void *) region, sizeof(first));
        SCB_InvalidateDCache_by_Addr((volatile void *) region, sizeof(*dst));
        __DSB();
        memcpy(&second, (const void *) region, sizeof(second));
        if (memcmp(&first, &second, sizeof(first)) == 0 &&
            second.source == source &&
            coreDataParamObserverBridgeIsValid(&second) != 0u)
        {
            *dst = second;
            return 1u;
        }
    }
    return 0u;
}

/* 反向区: cm_7_1 规划核 → cm_7_0 链路核(运动意图)。另起一固定地址, 与 keyEventRegion 无重叠。 */
#pragma location = CORE1_DATA_PLAN_ADDR
__no_init volatile planToLinkStruct planRegion;

static uint32_t planPublishGeneration;
__no_init uint32_t planBootGenerationRetained;
__no_init uint32_t planBootGenerationInverseRetained;
static uint32_t planBootGeneration;

uint32_t coreDataGetPlanBootGeneration(void)
{
    if (planBootGeneration == 0u)
    {
        {
            uint32_t previous = planBootGenerationRetained;
            if ((previous == 0u) ||
                (planBootGenerationInverseRetained != ~previous))
            {
                previous = 0u;
            }
            previous++;
            if (previous == 0u) previous = 1u;
            planBootGenerationRetained = previous;
            planBootGenerationInverseRetained = ~previous;
            planBootGeneration = previous;
        }
        return planBootGeneration;
    }
}

static volatile void* planGenerationCacheLine(void)
{
    {
        uintptr_t marker = (uintptr_t) &planRegion.publishGeneration;
        return (volatile void*) (marker & ~(uintptr_t) CORE1_PLAN_CACHE_LINE_SIZE - 1u);
    }
}

static void cleanPlanGeneration(void)
{
    {
        SCB_CleanDCache_by_Addr(planGenerationCacheLine(), CORE1_PLAN_CACHE_LINE_SIZE);
        __DSB();
    }
}

```

```

static void invalidatePlanGeneration(void)
{
    SCB_InvalidateDCache_by_Addr(planGenerationCacheLine(), CORE1_PLAN_CACHE_LINE_SIZE);
    __DSB();
}

static uint32_t nextPlanGeneration(void)
{
    planPublishGeneration += 2u;
    if (planPublishGeneration == 0u) planPublishGeneration = 2u;
    return planPublishGeneration;
}

void coreDataWritePlan(const planToLinkStruct* src)
{
    planToLinkStruct frame;
    uint32_t committed;

    if (0 == src) return;

    frame = *src;
    frame.producerBootGeneration = coreDataGetPlanBootGeneration();
    committed = nextPlanGeneration();

    planRegion.publishGeneration = committed | 1u;
    cleanPlanGeneration();

    frame.publishGeneration = committed | 1u;
    memcpy((void*) &planRegion, &frame, sizeof(frame));
    SCB_CleanDCache_by_Addr((volatile void*) &planRegion, sizeof(planToLinkStruct));
    __DSB();

    planRegion.publishGeneration = committed;
    cleanPlanGeneration();
}

uint8_t coreDataReadPlan(planToLinkStruct* dst)
{
    uint8_t attempt;

    if (0 == dst) return 0u;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        uint32_t before;
        uint32_t after;

        invalidatePlanGeneration();
        before = planRegion.publishGeneration;
        if ((before == 0u) || ((before & 1u) != 0u)) continue;

        SCB_InvalidateDCache_by_Addr((volatile void*) &planRegion, sizeof(planToLinkStruct));
        __DSB();
        memcpy(dst, (const void*) &planRegion, sizeof(*dst));

        invalidatePlanGeneration();
        after = planRegion.publishGeneration;
        if ((before == dst->publishGeneration) && (before == after) &&
            ((after & 1u) == 0u) && (dst->magic == CORE1_PLAN_READY_MAGIC))
        {
            return 1u;
        }
    }

    memset(dst, 0, sizeof(*dst));
    return 0u;
}

void coreDataWriteParamGlobalRequest(const paramSyncGlobalRequest *src)
{
    core1ParamGlobalRequestStruct frame;
    if ((src == 0) || (paramSyncGlobalRequestIsValid(src) == 0u)) return;
    memset(&frame, 0, sizeof(frame));
    frame.magic = CORE1_PARAM_GLOBAL_MAGIC;
    frame.value = *src;
    memcpy((void *) &paramGlobalRequestRegion, &frame, sizeof(frame));
    SCB_CleanDCache_by_Addr((volatile void *) &paramGlobalRequestRegion,
        sizeof(paramGlobalRequestRegion));
    __DSB();
}

```

```

uint8_t coreDataTryReadParamGlobalRequest(paramSyncGlobalRequest *dst)
{
    core1ParamGlobalRequestStruct first, second;
    uint8_t attempt;
    if (dst == 0) return 0u;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        SCB_InvalidateDCache_by_Addr((volatile void *) &paramGlobalRequestRegion,
            sizeof(paramGlobalRequestRegion));
        __DSB();
        memcpy(&first, (const void *) &paramGlobalRequestRegion, sizeof(first));
        SCB_InvalidateDCache_by_Addr((volatile void *) &paramGlobalRequestRegion,
            sizeof(paramGlobalRequestRegion));
        __DSB();
        memcpy(&second, (const void *) &paramGlobalRequestRegion, sizeof(second));
        if ((memcmp(&first, &second, sizeof(first)) == 0) &&
            second.magic == CORE1_PARAM_GLOBAL_MAGIC &&
            paramSyncGlobalRequestsValid(&second.value) != 0u)
        {
            *dst = second.value;
            return 1u;
        }
    }
    return 0u;
}

```

```

void coreDataWriteParamGlobalState(const paramSyncGlobalState *src)
{
    core1ParamGlobalStateStruct frame;
    if ((src == 0) || (paramSyncGlobalStatesValid(src) == 0u)) return;
    memset(&frame, 0, sizeof(frame));
    frame.magic = CORE1_PARAM_GLOBAL_MAGIC;
    frame.value = *src;
    memcpy((void *) &paramGlobalStateRegion, &frame, sizeof(frame));
    SCB_CleanDCache_by_Addr((volatile void *) &paramGlobalStateRegion,
        sizeof(paramGlobalStateRegion));
    __DSB();
}

```

```

uint8_t coreDataTryReadParamGlobalState(paramSyncGlobalState *dst)
{
    core1ParamGlobalStateStruct first, second;
    uint8_t attempt;
    if (dst == 0) return 0u;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        SCB_InvalidateDCache_by_Addr((volatile void *) &paramGlobalStateRegion,
            sizeof(paramGlobalStateRegion));
        __DSB();
        memcpy(&first, (const void *) &paramGlobalStateRegion, sizeof(first));
        SCB_InvalidateDCache_by_Addr((volatile void *) &paramGlobalStateRegion,
            sizeof(paramGlobalStateRegion));
        __DSB();
        memcpy(&second, (const void *) &paramGlobalStateRegion, sizeof(second));
        if ((memcmp(&first, &second, sizeof(first)) == 0) &&
            second.magic == CORE1_PARAM_GLOBAL_MAGIC &&
            paramSyncGlobalStatesValid(&second.value) != 0u)
        {
            *dst = second.value;
            return 1u;
        }
    }
    return 0u;
}

```

core1_cyt4bb7/project/code/comm/coreDataTransfer.h

SHA-256

:

f5e40902a85b0a6364c04d16f9e65e8364ea3c890936b09d4d2c2b2d117f7507

代码 (c)

/*
 * core1 核间共享内存 (cm_7_0 链路核 ↔ cm_7_1 菜单/视觉核)


```

*
* 物理区域: section.global_ram_data
* startAddr = 0x28001000, size = 0x2000 (core1 此段经 .map 确认空闲)
* 保留地址须避开: mt9v03x_h_num @ 0x28006BF0, mt9v03x_w_num @ 0x28006BF2 (远离, 无冲突)
*
* 用途:
* - status 区: core0 经核间 UART 把链路、飞控和系统状态发给 cm_7_0, cm_7_1 按值读取。
* - pose 区: core0 经核间 UART 把灯板同坐标/方向发给 core1 cm_7_0, cm_7_1 规划核读出生成车坐标误差。
* key/pose 均同时携带链路在线语义或用于链路在线门控。core1 内 cm_7_0→cm_7_1 单向。
*
* 双核 D-Cache 一致性 (Cortex-M7, cache line=32B), 照搬 core0 coreDataTransfer:
* 写方 memcpy→SCB_CleanDCache_by_Addr; key/KVM 安全快照读方最多三次双读一致校验, 失败整帧清零。
* 握手 32 位 magic 防 __no_init 上电垃圾。
*/
#ifndef _CORE_DATA_TRANSFER_H_
#define _CORE_DATA_TRANSFER_H_

#include <stdint.h>

#include "paramSyncMailbox.h"
#include "paramSyncGlobal.h"

#define CORE1_DATA_KEY_ADDR (0x28001000u)
#define CORE1_DATA_READY_MAGIC (0x5A5AA5A6u)

/* cm_7_0 链路核 → cm_7_1 菜单核 (32 字节, 单 cache line)
* - linkUp: core0 链路在线(心跳未超时)=1; cm_7_1 据此判断远程操控可用 + 断联保护
* - core2LinkLost: core1 cm_7_0 观测到 RS-485↔core2 断联=1, 供 cm_7_1 HOME HUD 显示
* - ready: = MAGIC 表示 cm_7_0 链路核已初始化(防 __no_init 上电垃圾) */
typedef struct
{
    uint32_t ready; /* = CORE1_DATA_READY_MAGIC */
    uint8_t linkUp; /* 1=core0 链路在线(心跳未超时) */
    uint8_t controlActive; /* 1=core0 处于远程模式(正用按键操控 core1); 0=core0 本地模式 */
    /* core0 飞控状态(经 CORE0_STATUS 帧, 供 cm_7_1 RGB 状态灯)。cm_7_0 写 cm_7_1 读。 */
    uint8_t core0SafetyState; /* 0x00 DISARM/0x01 CLOSED_LOOP/0x02 PROTECT/0xFE ARMING/0xFF IMUFAIL */
    uint8_t core0ArmProgress; /* arming 进度 0~100 */
    uint8_t core0StatusFlags; /* UART_LINK_C0_FLAG_*; cm70Ready/velValid/targetValid/tofHealthy */
    uint8_t core0StatusValid; /* 1=已收到过 core0 CORE0_STATUS 帧(否则上面三字段无意义) */
    /* 空地系统模式 + RGB 意图(经 CORE0_STATUS sysCtrl, 供 cm_7_1 菜单显示 + rgbAnimator)。cm_7_0 写 cm_7_1 读。 */
    uint8_t core0SystemMode; /* 空地系统模式 1/2/3(0=未设) */
    uint8_t core0RgbIntent; /* RGB 意图码(UART_LINK_RGB_*, 供 cm_7_1 rgbAnimator) */
    uint8_t core2LinkLost; /* 1=core1 RS-485↔core2 断联; 0=在线/未判失联 */
    uint8_t core0AltReached; /* CORE0_STATUS sysCtrl bit5; 仅 statusValid 时有意义 */
    uint8_t padding[18];
} keyEventToMenuStruct;

typedef char coreData_keyEvent_size_assert[(sizeof(keyEventToMenuStruct) == 32u) ? 1 : -1];
typedef char coreData_keyEvent_align_assert[(((CORE1_DATA_KEY_ADDR % 32u) == 0u) ? 1 : -1)];

/* cm_7_0 调用: 写按键事件/链路状态 (Clean) */
void coreDataWriteKeyEvent(const keyEventToMenuStruct* src);
/* cm_7_1 调用: 读按键事件/链路状态 (Invalidate) */
void coreDataReadKeyEvent(keyEventToMenuStruct* dst);
/* 事务读: 成功返回 1; 连续三轮读碰撞时返回 0 且不修改 dst。 */
uint8_t coreDataTryReadKeyEvent(keyEventToMenuStruct* dst);

/* =====
* core0 位姿区: cm_7_0 链路核 → cm_7_1 规划/显示核
* cm_7_0 收 CORE0_POSE 帧后写本区; cm_7_1 规划核按 epoch/flags 与 heading 置信度分别锁存/保持。
* 地址接在 keyEventRegion 后一个 cache line, 与 planRegion 保持大间隔。
* ===== */
#define CORE1_DATA_C0POSE_ADDR (0x28001020u)
#define CORE1_DATA_KVM_STATUS_ADDR (0x28001040u)
#define CORE1_DATA_KVM_INGRESS_ADDR (0x28001060u)
#define CORE1_DATA_PARAM_GLOBAL_REQUEST_ADDR (0x28001080u)
#define CORE1_DATA_PARAM_GLOBAL_STATE_ADDR (0x280010A0u)
#define CORE1_DATA_PARAM_OBSERVER3_ADDR (0x280010C0u)
#define CORE1_DATA_PARAM_OBSERVER4_ADDR (0x280010E0u)
#define CORE1_DATA_PLAN_ADDR (0x28001100u)
#define CORE1_C0POSE_READY_MAGIC (0xA5A5A5A5u)

typedef struct
{
    uint32_t magic; /* = CORE1_C0POSE_READY_MAGIC */
    uint32_t seq32; /* core1 本地接收 CORE0_POSE 帧计数, 每帧 +1 */
    uint8_t poseEpoch; /* core0 链路初始化 epoch, 变更即清旧锁存 */
    uint8_t confidenceQ8; /* core0 方向置信度 Q0.8 */
    uint8_t flags; /* UART_LINK_POSE_FLAG_* */

```

```

uint8_t uartSeq;    /* 原始 CORE0_POSE 帧 seq, 调试用 */
uint16_t xQ8;      /* core0 灯板 X, 同坐标 Q8.8 */
uint16_t yQ8;      /* core0 灯板 Y, 同坐标 Q8.8 */
float   orientationRad; /* core0 灯板方向 rad */
uint32_t padding[3];
} core0PoseToVisionStruct;

typedef char coreData_c0pose_size_assert[(sizeof(core0PoseToVisionStruct) == 32u) ? 1 : -1];
typedef char coreData_c0pose_align_assert[(((CORE1_DATA_C0POSE_ADDR % 32u) == 0u) ? 1 : -1)];
typedef char coreData_c0pose_after_key_assert[(CORE1_DATA_KEY_ADDR + sizeof(keyEventToMenuStruct) <= CORE1_DATA_C0POSE_ADDR) ? 1 :
-1];
typedef char coreData_plan_after_c0pose_assert[(CORE1_DATA_C0POSE_ADDR + sizeof(core0PoseToVisionStruct) <= CORE1_DATA_PLAN_ADDR) ? 1 :
-1];

/* cm_7_1 KVM router → cm_7_0 UART owner。 */
typedef struct
{
    uint32_t magic;
    uint32_t routerSession;
    uint32_t targetSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t ackEventSeq;
    uint16_t core3AgeMs;
    uint8_t target;
    uint8_t activeTarget;
    uint8_t menuStates;
    uint8_t flags;
    uint8_t ackResult;
    uint8_t paramTargetMask; /* cm7_1 实时确认的 Core1/Core3/Core4 参数能力位。 */
} core1KvmStatusStruct;

/* cm_7_0 UART owner → cm_7_1 KVM router。 */
typedef struct
{
    uint32_t magic;
    uint32_t targetSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t eventSeq;
    uint32_t leaseMs;
    uint32_t updateSeq;
    uint8_t target;
    uint8_t keyId;
    uint8_t keyState;
    uint8_t flags;
} core1KvmIngressStruct;

#define CORE1_KVM_INGRESS_CONTROL_VALID (1u << 0)
#define CORE1_KVM_INGRESS_EVENT_VALID  (1u << 1)
#define CORE1_KVM_INGRESS_EVENT_EDGE   (1u << 2)

typedef char coreData_kvm_status_size_assert[(sizeof(core1KvmStatusStruct) == 32u) ? 1 : -1];
typedef char coreData_kvm_ingress_size_assert[(sizeof(core1KvmIngressStruct) == 32u) ? 1 : -1];
typedef char coreData_kvm_status_align_assert[(((CORE1_DATA_KVM_STATUS_ADDR % 32u) == 0u) ? 1 : -1)];
typedef char coreData_kvm_ingress_align_assert[(((CORE1_DATA_KVM_INGRESS_ADDR % 32u) == 0u) ? 1 : -1)];
typedef char coreData_kvm_status_after_pose_assert[(CORE1_DATA_C0POSE_ADDR + sizeof(core0PoseToVisionStruct) <=
CORE1_DATA_KVM_STATUS_ADDR) ? 1 : -1];
typedef char coreData_kvm_ingress_after_status_assert[(CORE1_DATA_KVM_STATUS_ADDR + sizeof(core1KvmStatusStruct) <=
CORE1_DATA_KVM_INGRESS_ADDR) ? 1 : -1];
typedef char coreData_plan_after_kvm_ingress_assert[(CORE1_DATA_KVM_INGRESS_ADDR + sizeof(core1KvmIngressStruct) <=
CORE1_DATA_PLAN_ADDR) ? 1 : -1];

#define CORE1_PARAM_GLOBAL_MAGIC (0x50474C42u)
#define CORE1_PARAM_OBSERVER_MAGIC (0x504F4252u)
#define CORE1_PARAM_OBSERVER_FLAG_READY (1u << 0)
#define CORE1_PARAM_OBSERVER_FLAG_REQUEST_VALID (1u << 1)
#define CORE1_PARAM_OBSERVER_ALLOWED_FLAGS \
    (CORE1_PARAM_OBSERVER_FLAG_READY | CORE1_PARAM_OBSERVER_FLAG_REQUEST_VALID)
typedef struct
{
    uint32_t magic;
    paramSyncGlobalRequest value;
    uint8_t padding[16];
} core1ParamGlobalRequestStruct;
typedef struct
{
    uint32_t magic;
    paramSyncGlobalState value;

```

```

} core1ParamGlobalStateStruct;
/* cm_7_1 Observer UART owner -> cm_7_0 gateway。每个来源独占一个 cache
* line，发布链路能力新鲜度及尚未被协调器确认的板端请求。*/
typedef struct
{
    uint32_t magic;
    uint32_t producerBootGeneration;
    uint32_t publishSequence;
    uint32_t peerSession;
    paramSyncGlobalRequest request;
    uint8_t source;
    uint8_t flags;
    uint8_t reserved[2];
} core1ParamObserverBridgeStruct;
typedef char coreData_param_global_request_size_assert[(sizeof(core1ParamGlobalRequestStruct) == 32u) ? 1 : -1];
typedef char coreData_param_global_state_size_assert[(sizeof(core1ParamGlobalStateStruct) == 32u) ? 1 : -1];
typedef char coreData_param_observer_size_assert[(sizeof(core1ParamObserverBridgeStruct) == 32u) ? 1 : -1];
typedef char coreData_param_global_request_align_assert[(((CORE1_DATA_PARAM_GLOBAL_REQUEST_ADDR & 31u) == 0u) ? 1 : -1)];
typedef char coreData_param_global_state_align_assert[(((CORE1_DATA_PARAM_GLOBAL_STATE_ADDR & 31u) == 0u) ? 1 : -1)];
typedef char coreData_param_observer3_align_assert[(((CORE1_DATA_PARAM_OBSERVER3_ADDR & 31u) == 0u) ? 1 : -1)];
typedef char coreData_param_observer4_align_assert[(((CORE1_DATA_PARAM_OBSERVER4_ADDR & 31u) == 0u) ? 1 : -1)];
typedef char coreData_param_global_after_kvm_assert[(((CORE1_DATA_KVM_INGRESS_ADDR + sizeof(core1KvmIngressStruct)) <=
CORE1_DATA_PARAM_GLOBAL_REQUEST_ADDR) ? 1 : -1)];
typedef char coreData_param_observer3_after_state_assert[(((CORE1_DATA_PARAM_GLOBAL_STATE_ADDR + sizeof(core1ParamGlobalStateStruct)) <=
CORE1_DATA_PARAM_OBSERVER3_ADDR) ? 1 : -1)];
typedef char coreData_param_observer4_after_observer3_assert[(((CORE1_DATA_PARAM_OBSERVER3_ADDR +
sizeof(core1ParamObserverBridgeStruct)) <= CORE1_DATA_PARAM_OBSERVER4_ADDR) ? 1 : -1)];
typedef char coreData_param_observer_before_plan_assert[(((CORE1_DATA_PARAM_OBSERVER4_ADDR + sizeof(core1ParamObserverBridgeStruct)) <=
CORE1_DATA_PLAN_ADDR) ? 1 : -1)];

static inline uint8_t coreDataParamObserverBridgeIsValid(
    const core1ParamObserverBridgeStruct *value)
{
    uint8_t requestValid;
    if (value == 0 || value->magic != CORE1_PARAM_OBSERVER_MAGIC ||
        value->producerBootGeneration == 0u || value->publishSequence == 0u ||
        (value->source != 3u && value->source != 4u) ||
        (value->flags & (uint8_t)~CORE1_PARAM_OBSERVER_ALLOWED_FLAGS) != 0u ||
        value->reserved[0] != 0u || value->reserved[1] != 0u)
        return 0u;
    requestValid = (uint8_t)((value->flags &
        CORE1_PARAM_OBSERVER_FLAG_REQUEST_VALID) != 0u);
    if ((value->flags & CORE1_PARAM_OBSERVER_FLAG_READY) != 0u)
    {
        if (value->peerSession == 0u) return 0u;
    }
    else if (value->peerSession != 0u || requestValid != 0u)
        return 0u;
    if (requestValid != 0u)
        return (uint8_t)(paramSyncGlobalRequestIsValid(&value->request) != 0u &&
            value->request.origin == value->source);
    return (uint8_t)(value->request.producerBootGeneration == 0u &&
        value->request.sequence == 0u && value->request.origin == 0u &&
        value->request.action == 0u && value->request.slot == 0u &&
        value->request.reserved == 0u);
}

void coreDataWriteParamGlobalRequest(const paramSyncGlobalRequest *src);
uint8_t coreDataTryReadParamGlobalRequest(paramSyncGlobalRequest *dst);
void coreDataWriteParamGlobalState(const paramSyncGlobalState *src);
uint8_t coreDataTryReadParamGlobalState(paramSyncGlobalState *dst);

void coreDataWriteParamObserverBridge(
    const core1ParamObserverBridgeStruct *src);
uint8_t coreDataTryReadParamObserverBridge(uint8_t source,
    core1ParamObserverBridgeStruct *dst);

void coreDataWriteKvmStatus(const core1KvmStatusStruct* src);
uint8_t coreDataTryReadKvmStatus(core1KvmStatusStruct* dst);
void coreDataWriteKvmIngress(const core1KvmIngressStruct* src);
void coreDataReadKvmIngress(core1KvmIngressStruct* dst);

/* cm_7_0 调用: 写 core0 位姿快照 (Clean) */
void coreDataWriteCore0Pose(const core0PoseToVisionStruct* src);
/* cm_7_1 调用: 读 core0 位姿快照 (Invalidate) */
void coreDataReadCore0Pose(core0PoseToVisionStruct* dst);
uint8_t coreDataTryReadCore0Pose(core0PoseToVisionStruct* dst);

/* =====

```

```

* 反向区: cm_7_1 规划核(beaconNavigator) → cm_7_0 链路核(派发)
* cm_7_1 把选中信标算出的统一运动意图 vx/vy/wz 写本区, cm_7_0 读出后按空地系统
* 模式派发: 模式 2 → RS-485(MSG_COMMAND) 给 core2 执行 + MOTION 帧抄送 core0 前馈。
* 方向与 keyEventRegion 相反(cm_7_1 单写 Clean, cm_7_0 单读 Invalidate)。
* 地址另起一 cache line, 与 32B 的 keyEventRegion(0x28 001 000) 无重叠。
* ===== */
#define CORE1_PLAN_READY_MAGIC (0xA5A55A5Cu)
typedef enum
{
    CORE1_PLAN_STATE_IDLE = 0u,
    CORE1_PLAN_STATE_TRACK = 2u,
    CORE1_PLAN_STATE_STARTUP_FORWARD = 3u,
    CORE1_PLAN_STATE_SEARCH = 4u
} core1PlanStateEnum;

typedef enum
{
    REQUIRED_VISION_FAULT_NONE = 0u,
    REQUIRED_VISION_FAULT_CAMERA = 1u,
    REQUIRED_VISION_FAULT_LINK = 2u,
    REQUIRED_VISION_FAULT_BRIDGE = 3u,
    REQUIRED_VISION_FAULT_PROTOCOL = 4u,
    REQUIRED_VISION_FAULT_CONFIG = 5u,
    REQUIRED_VISION_FAULT_CALIBRATION = 6u,
    REQUIRED_VISION_FAULT_POSE = 7u
} requiredVisionFaultClassEnum;

#define REQUIRED_VISION_SOURCE_NONE (0u)
#define REQUIRED_VISION_SOURCE_CORE0 (1u)
#define REQUIRED_VISION_SOURCE_CORE1 (2u)
#define REQUIRED_VISION_SOURCE_CORE3 (3u)
#define REQUIRED_VISION_SOURCE_CORE4 (4u)

/* cm_7_1 规划核 → cm_7_0 链路核 (64 字节, 2 cache line)
* - magic: = CORE1_PLAN_READY_MAGIC 表示规划核已写过(防 __no_init 上电垃圾)
* - seq: 每次规划更新 +1, cm_7_0 据 seq 变化判数据新鲜(超时无变化→零速)
* - state: beaconNavigatorStateEnum (0 IDLE/2 TRACK/3 首灯前等待或直行/4 SEARCH)
* - valid: 1= vx/vy/wz 可下发; 0=应作零速处理
* - target / err 字段: 观测量(供 cm_7_1 上屏 + 调试), targetX/Y/Size 为 continuity 控制点/面积;
* EdgeHold 时 targetSize 为最后目标面积, 不一定来自当前 fresh 观测。
* cm_7_0 派发只读 valid/vx/vy/wz */
typedef struct
{
    uint32_t magic; /* = CORE1_PLAN_READY_MAGIC */
    uint32_t seq; /* 规划更新边沿计数 */
    uint8_t state; /* 运动调度状态机当前态 */
    uint8_t valid; /* 1=运动意图有效可下发 */
    uint8_t beaconCount; /* 当前确认信标数(观测) */
    uint8_t padding0;
    uint16_t targetX; /* continuity 控制点 X(实测或保持) */
    uint16_t targetY; /* continuity 控制点 Y(实测或保持) */
    uint16_t targetSize; /* continuity 目标面积 */
    uint16_t padding1;
    float errX; /* 像素误差 X(观测) */
    float errY; /* 像素误差 Y(观测) */
    float vx; /* 规划输出: 前向速度意图 */
    float vy; /* 规划输出: 横向速度意图 */
    float wz; /* 规划输出: 偏航角速度意图 */
    /* 运行时派发控制 (cm_7_1 从 visionParams 填, cm_7_0 派发门控用; 取代原编译期宏) 。 */
    uint8_t dispatchEnable; /* 1=允许模式 2 下发运动(总开关) */
    uint8_t motionUnlocked; /* 模式 2 本次 altReached episode 已通过启动门 */
    uint16_t planTimeoutMs; /* cm_7_0 判规划 seq 新鲜度超时(ms) */
    uint8_t requiredVisionFault;
    uint8_t startupPhase; /* takeoffMotionPhaseEnum, 仅 Core1 双核内部观测 */
    uint8_t beaconSearching; /* 1=continuity 正在寻灯 */
    uint8_t beaconSearchTimedOut; /* 1=当前寻灯 episode 已超时 */
    uint32_t takeoffMotionEpoch; /* 每次模式 2 altReached 新 episode 递增 */
    uint32_t publishGeneration; /* 共享区奇偶事务代数; 调用方不赋值 */
    uint8_t faultSource;
    uint8_t faultClass;
    uint8_t poseStage;
    uint8_t paddingTail;
    uint32_t producerBootGeneration; /* cm_7_1 启动代次; 先于 seq 判新鲜, 0 非法 */
} planToLinkStruct;

/* 跨核 ABI 固定 64 字节 (两核 cm_7_0/cm_7_1 须同步烧录同一布局)。改字段务必守住 64B。 */
typedef char coreDataTransfer_planToLink_size_assert[(sizeof(planToLinkStruct) == 64u) ? 1 : -1];

/* cm_7_1 调用: 写规划运动意图 (Clean) */

```

```

uint32_t coreDataGetPlanBootGeneration(void);
void coreDataWritePlan(const planToLinkStruct* src);
/* cm_7_0 调用: 读规划运动意图 (Invalidate) */
uint8_t coreDataReadPlan(planToLinkStruct* dst);

typedef char coreData_plan_before_param_sync_assert[
    ((CORE1_DATA_PLAN_ADDR + sizeof(planToLinkStruct)) <=
    PARAM_SYNC_MAILBOX_REGION_ADDR) ? 1 : -1];
typedef char coreData_param_sync_region_end_assert[
    ((PARAM_SYNC_MAILBOX_REGION_ADDR + PARAM_SYNC_MAILBOX_REGION_BYTES) <=
    0x28002000u) ? 1 : -1];

#endif /* _CORE_DATA_TRANSFER_H_ */

```

core1_cyt4bb7/project/code/comm/crossCoreLink.c

SHA-256

:

6dae87120b5fa0819808bb6a026f007a176a781cdf211c71dc32a1ebc3c587ac

代码 (c)

```

#include "crossCoreLink.h"

#include <string.h>

#include "zf_driver_uart.h"

static uint8_t crossCoreLinkIsCore0Alive(void);
static void crossCoreLinkSetMode2LoraLandReq(uint8_t req);
static void crossCoreLinkSetCore2LinkLost(uint8_t lost);

#include "uartLinkProto.h"
#include "coreDataTransfer.h"
#include "pinMap.h" /* 核间 UART 编号/引脚单一真相源 */

/*
 * crossCoreLink core1 cm_7_0 端实现。单线双向(UART_1/P04 ↔ core0 P10)。
 * 收状态/位姿/KVM 帧并写共享内存；周期发心跳，维护 core0 存活超时。
 */

#define CCL_UART_NUM    (LINK_GROUP0_UART_NUM)
#define CCL_UART_TX_PIN (LINK_GROUP0_UART_TX_PIN)
#define CCL_UART_RX_PIN (LINK_GROUP0_UART_RX_PIN)
#define CCL_BAUDRATE    (UART_LINK_BAUDRATE)
#if UART_LINK_BAUDRATE != 4000000u
#error "Core0-Core1 link baud contract requires 4 Mbps"
#endif
#define CCL_RX_QUEUE_SIZE    (16u)
#define CCL_HEARTBEAT_MS     (200u) /* 心跳发送周期 */
#define CCL_LINK_LOST_MS     (600u) /* 无 core0 帧超时判断断联(3 个心跳周期) */
#define CCL_CORE0_STATUS_LOST_MS (600u)
#define CCL_PARAM_GLOBAL_STATE_TX_MS (50u)

typedef struct
{
    uartLinkParserStruct parser;
    uartLinkFrameStruct rxQueue[CCL_RX_QUEUE_SIZE];
    volatile uint8_t rxHead;
    volatile uint8_t rxTail;
    volatile uint8_t rxCount;
    volatile uint8_t initReadyFlag;
    volatile uint32_t tickMs;
    volatile uint32_t lastRxTickMs;
    uint32_t lastHeartbeatTickMs;
    volatile uint8_t linkPrimed;
    volatile uint8_t core0ControlActive; /* core0 心跳携带: 1=core0 远程模式正操控 core1 */
    volatile uint8_t core0StatusFlags; /* 最近 CORE0_STATUS 的 statusFlags(含 LAUNCHED 位), 供 cm_7_0 取发车态 */
    volatile uint8_t core0SysCtrl; /* 最近 CORE0_STATUS 的 sysCtrl(系统模式/RGB 意图/到顶), 供 cm_7_0 取 */
    volatile uint8_t core0Buzzer; /* 最近 CORE0_STATUS 的蜂鸣模式码, 供 cm_7_0 中转 RS-485 */
    volatile uint8_t core0SafetyState; /* 最近 CORE0_STATUS 的 safetyState 字节, 供 cm_7_0 判 core0 地面态(断链视觉调试 gate) */
    volatile uint8_t core0StatusValid; /* 1=已收到过 CORE0_STATUS(断链 Poll 清 0), fail-closed 地面态判据基 */
    uint32_t lastCore0StatusTickMs;
    uint8_t core0StatusWirePrimed;
    uint8_t core0StatusWireSeq;
    uint8_t core0StatusRearmRequired;

```

```

uint8_t core0PeerRearmPending;
uint8_t heartbeatSequence;
uint8_t motionSequence; /* MOTION 帧序号(转发给 core0 的运动前馈) */
uint8_t statusSequence; /* STATUS 帧序号(周期上报 core2 断联位给 core0, 与心跳同周期发) */
uint8_t kvmStatusSequence;
uint32_t kvmIngressSeq;
uint8_t kvmIngressBatchFlags;
core1KvmStatusStruct lastSentKvmStatus;
uint8_t lastSentKvmStatusValid;
uint8_t core2LinkLost; /* RS-485↔core2 断联位(core0Loop 前台 set, cclSendStatus 读发); 全前台单上下文 */
uint8_t core2AttitudeFault; /* Core2 姿态诊断故障; 独立于链路和 ESTOP */
uint8_t requiredVisionFault;
uint8_t faultSource;
uint8_t faultClass;
uint32_t producerBootSession;
uint8_t statusDirty;
uint8_t mode2LoraLandReq; /* 模式 2 LoRa 双肩键缓降请求位; STATUS linkFlags bit2 上报 core0 */
uint16_t beaconCount;
uint8_t beaconSearching;
uint8_t beaconSearchTimedOut;
uint8_t beaconStateValid;
uint32_t poseSeq32; /* 已写入 poseRegion 的 CORE0_POSE 接收计数 */
uint32_t rxFrameCount;
uint32_t rxErrorCount;
uartLinkFrameStruct paramResponse;
uint8_t paramResponsePending;
uint8_t paramTxSequence;
paramSyncGlobalRequest paramGlobalRequest;
paramSyncGlobalState paramGlobalState;
uint32_t lastParamGlobalStateTickMs;
volatile uint8_t paramGlobalRequestPending;
uint8_t paramGlobalStateValid;
uint8_t paramGlobalStateDirty;
uint8_t paramGlobalTxSequence;
} crossCoreLinkStateStruct;

static crossCoreLinkStateStruct cclState;
__no_init static uint32_t cclBootCounterRetained;
__no_init static uint32_t cclBootCounterInverseRetained;

static uint32_t cclNextBootSession(void)
{
    uint32_t value = cclBootCounterRetained;
    if ((value == 0u) || (cclBootCounterInverseRetained != ~value)) value = 0u;
    value++;
    if (value == 0u) value = 1u;
    cclBootCounterRetained = value;
    cclBootCounterInverseRetained = ~value;
    return value;
}

static uint8_t cclWireSeqAdvanced(uint8_t sequence, uint8_t previous)
{
    uint8_t delta = (uint8_t) (sequence - previous);
    return (uint8_t) ((delta != 0u) && (delta < 128u));
}

static uint8_t cclCore0StatusSemanticValid(const uartLinkFrameStruct *frame)
{
    uint8_t launched;
    uint8_t altReached;
    uint8_t mode;
    uint8_t rgb;
    if (frame == 0) return 0u;
    if (!(frame->safetyState <= 2u) || (frame->safetyState == 0xFEu) ||
        (frame->safetyState == 0xFFu)) return 0u;
    if (frame->armProgress > 100u) return 0u;
    if ((frame->statusFlags & (uint8_t) ~0x1Fu) != 0u) return 0u;
    if (frame->buzzer > UART_LINK_BUZZER_THREE_BEEPS) return 0u;
    mode = (uint8_t) (frame->sysCtrl & UART_LINK_SYS_MODE_MASK);
    rgb = (uint8_t) ((frame->sysCtrl >> UART_LINK_SYS_RGB_SHIFT) & UART_LINK_SYS_RGB_MASK);
    if ((mode > UART_LINK_SYS_MODE_LORA_FWD) || (rgb > UART_LINK_RGB_DISCONNECT_WARN)) return 0u;
    launched = (frame->statusFlags & UART_LINK_C0_FLAG_LAUNCHED) ? 1u : 0u;
    altReached = (frame->sysCtrl & UART_LINK_SYS_ALT_REACHED) ? 1u : 0u;
    if (launched != (uint8_t) (frame->safetyState == 1u)) return 0u;
    if ((altReached != 0u) && (launched == 0u)) return 0u;
    return 1u;
}

```

```

static uint8_t cclCore0StatusIsSafeRearm(const uartLinkFrameStruct *frame)
{
    return (uint8_t) ((frame != 0) &&
        (frame->safetyState != 1u) &&
        ((frame->statusFlags & UART_LINK_C0_FLAG_LAUNCHED) == 0u) &&
        ((frame->sysCtrl & UART_LINK_SYS_ALT_REACHED) == 0u));
}

static void cclInvalidateCore0Status(uint8_t requireSafeRearm)
{
    keyEventToMenuStruct ev;
    cclState.core0StatusValid = 0u;
    cclState.core0StatusFlags = 0u;
    cclState.core0SysCtrl = 0u;
    cclState.core0Buzzer = 0u;
    cclState.core0SafetyState = CCL_CORE0_SAFETY_UNKNOWN;
    if (requireSafeRearm != 0u) cclState.core0StatusRearmRequired = 1u;
    coreDataReadKeyEvent(&ev);
    if (ev.ready == CORE1_DATA_READY_MAGIC)
    {
        ev.core0SafetyState = CCL_CORE0_SAFETY_UNKNOWN;
        ev.core0ArmProgress = 0u;
        ev.core0StatusFlags = 0u;
        ev.core0StatusValid = 0u;
        ev.core0SystemMode = 0u;
        ev.core0RgbIntent = 0u;
        ev.core0AltReached = 0u;
        coreDataWriteKeyEvent(&ev);
    }
}

static uint8_t cclNextIndex(uint8_t index)
{
    index++;
    if (index >= CCL_RX_QUEUE_SIZE) index = 0u;
    return index;
}

/* ISR 上下文: 解析一字节, 成帧则入队(丢弃溢出)。仅轻量, 不做 DCache/共享内存。 */
static void cclHandleByte(uint8_t data)
{
    uartLinkFrameStruct frame;
    uartLinkProtoStatusEnum status = uartLinkProtoInputByte(&cclState.parser, data, &frame);

    if (UART_LINK_PROTO_STATUS_FRAME_READY != status)
    {
        if (UART_LINK_PROTO_STATUS_CHECK_ERROR == status || UART_LINK_PROTO_STATUS_TAIL_ERROR == status ||
            UART_LINK_PROTO_STATUS_TYPE_ERROR == status)
        {
            cclState.rxErrorCount++;
        }
        return;
    }

    if (UART_LINK_TYPE_PARAM_GLOBAL_REQUEST == frame.type)
    {
        cclState.paramGlobalRequest = frame.paramGlobalRequest;
        cclState.paramGlobalRequestPending = 1u;
        return;
    }
    if (UART_LINK_TYPE_PARAM_GLOBAL_STATE == frame.type)
    {
        cclState.rxErrorCount++;
        return;
    }

    /* 收到任意控制域合法帧才视为 core0 在线; 参数域不为控制链续帧。 */
    cclState.lastRxTickMs = cclState.tickMs;
    cclState.linkPrimed = 1u;

    /* 心跳帧 value 携带 core0 操控态(1=远程模式正操控 core1)。心跳不入队列。
    * 仅更新 controlActive(ISR 内单字节写, 原子); 队列放需前台处理的状态/位姿/KVM 帧。 */
    if (UART_LINK_TYPE_HEARTBEAT == frame.type)
    {
        cclState.core0ControlActive = (frame.value != 0u) ? 1u : 0u;
        return;
    }
}

```

```

if (cclState.rxCount >= CCL_RX_QUEUE_SIZE)
{
    /* 队列满: 丢最旧, 保最新(按键事件最新优先)。
    * 本函数在 UART ISR 上下文; rxCount-- 与下方 ++ 净零, 同核无同级中断抢占
    * (PIT ISR 不碰本队列), 故无需原子化。 */
    cclState.rxHead = cclNextIndex(cclState.rxHead);
    cclState.rxCount--;
}
cclState.rxQueue[cclState.rxTail] = frame;
cclState.rxTail = cclNextIndex(cclState.rxTail);
cclState.rxCount++;
cclState.rxFrameCount++;
}

static uint8_t cclPopFrame(uartLinkFrameStruct *frame)
{
    uint8_t got = 0u;
    uart_rx_interrupt(CCL_UART_NUM, 0u);
    if (cclState.rxCount > 0u)
    {
        *frame = cclState.rxQueue[cclState.rxHead];
        cclState.rxHead = cclNextIndex(cclState.rxHead);
        cclState.rxCount--;
        got = 1u;
    }
    uart_rx_interrupt(CCL_UART_NUM, 1u);
    return got;
}

static void cclEncodeWriteAdvance(const uartLinkFrameStruct *frame,
    uint8_t *buffer, uint8_t frameLength, uint8_t *sequence)
{
    if (UART_LINK_PROTO_STATUS_OK == uartLinkProtoEncode(frame, buffer, frameLength))
    {
        uart_write_buffer(CCL_UART_NUM, buffer, frameLength);
        (*sequence)++;
    }
}

static void cclSendHeartbeat(void)
{
    uint8_t buffer[UART_LINK_FRAME_LENGTH];
    uartLinkFrameStruct frame;
    frame.type = UART_LINK_TYPE_HEARTBEAT;
    frame.sequence = cclState.heartbeatSequence;
    frame.value = (uint8_t) (cclState.heartbeatSequence ^ 0xA5u);
    cclEncodeWriteAdvance(&frame, buffer, UART_LINK_FRAME_LENGTH,
        &cclState.heartbeatSequence);
}

static void cclSendParamGlobalState(void)
{
    uint8_t buffer[UART_LINK_FRAME_LENGTH_PARAM_GLOBAL_STATE];
    uartLinkFrameStruct frame;
    if (cclState.paramGlobalStateValid == 0u) return;
    memset(&frame, 0, sizeof(frame));
    frame.type = UART_LINK_TYPE_PARAM_GLOBAL_STATE;
    frame.sequence = cclState.paramGlobalTxSequence++;
    frame.paramGlobalState = cclState.paramGlobalState;
    if (uartLinkProtoEncode(&frame, buffer, sizeof(buffer)) == UART_LINK_PROTO_STATUS_OK)
        uart_write_buffer(CCL_UART_NUM, buffer, sizeof(buffer));
    cclState.lastParamGlobalStateTickMs = cclState.tickMs;
    cclState.paramGlobalStateDirty = 0u;
}

/* 发 STATUS 帧(14 字节)给 core0: 上报 core2 断联(bit1)、模式 2 调试缓降请求(bit2)及通用必需
* 视觉故障事实(bit3, Core0 始终消费)。与心跳同周期(前台 200ms),
* 常发不受 MOTION 帧 gotMotion 门控影响; beaconCount 与寻灯状态来自 cm_7_1 现有 plan 共享快照。 */
static void cclSendStatus(void)
{
    uint8_t buffer[UART_LINK_FRAME_LENGTH_STATUS];
    uartLinkFrameStruct frame;
    frame.type = UART_LINK_TYPE_STATUS;
    frame.sequence = cclState.statusSequence;
    frame.beaconCount = cclState.beaconCount;
    frame.linkFlags = UART_LINK_STATUS_FLAG_LINK_OK; /* core1 在发即认为核间链路在线 */
    frame.faultSource = cclState.faultSource;
    frame.faultClass = cclState.faultClass;
    frame.producerBootSession = cclState.producerBootSession;
}

```



```

    if (cclState.core2LinkLost)
    {
        frame.linkFlags |= UART_LINK_STATUS_FLAG_CORE2_LOST;
    }
    if (cclState.core2AttitudeFault)
    {
        frame.linkFlags |= UART_LINK_STATUS_FLAG_CORE2_ATTITUDE_FAULT;
    }
    if (cclState.mode2LoraLandReq)
    {
        frame.linkFlags |= UART_LINK_STATUS_FLAG_MODE2_LORA_LAND_REQ;
    }
    if (cclState.requiredVisionFault)
    {
        frame.linkFlags |= UART_LINK_STATUS_FLAG_REQUIRED_VISION_FAULT;
    }
    if (cclState.beaconSearching)
        frame.linkFlags |= UART_LINK_STATUS_FLAG_BEACON_SEARCHING;
    if (cclState.beaconSearchTimedOut)
        frame.linkFlags |= UART_LINK_STATUS_FLAG_BEACON_SEARCH_TIMEOUT;
    if (cclState.beaconStateValid)
        frame.linkFlags |= UART_LINK_STATUS_FLAG_BEACON_VALID;
    cclEncodeWriteAdvance(&frame, buffer, UART_LINK_FRAME_LENGTH_STATUS,
        &cclState.statusSequence);
    cclState.statusDirty = 0;
}

static void cclSendKvmStatus(const core1KvmStatusStruct *status)
{
    uint8_t buffer[UART_LINK_FRAME_LENGTH_KVM_STATUS];
    uartLinkFrameStruct frame;
    if ((status == 0) || (status->magic != CORE1_DATA_READY_MAGIC)) return;
    frame.type = UART_LINK_TYPE_KVM_STATUS;
    frame.sequence = cclState.kvmStatusSequence;
    frame.kvmTargetSession = status->targetSession;
    frame.kvmRouteSession = status->routeSession;
    frame.kvmRouteEpoch = status->routeEpoch;
    frame.kvmEventSeq = status->ackEventSeq;
    frame.kvmTarget = status->target;
    frame.kvmActiveTarget = status->activeTarget;
    frame.kvmMenuStates = status->menuStates;
    frame.kvmFlags = status->flags;
    frame.kvmAckResult = status->ackResult;
    frame.kvmCore3AgeMs = status->core3AgeMs;
    cclEncodeWriteAdvance(&frame, buffer, UART_LINK_FRAME_LENGTH_KVM_STATUS,
        &cclState.kvmStatusSequence);
    cclState.lastSentKvmStatus = *status;
    cclState.lastSentKvmStatusValid = 1;
}

static uint8_t cclKvmStatusSemanticChanged(const core1KvmStatusStruct *status)
{
    const core1KvmStatusStruct *last = &cclState.lastSentKvmStatus;
    if ((status == 0) || (status->magic != CORE1_DATA_READY_MAGIC)) return 0;
    if (cclState.lastSentKvmStatusValid == 0) return 1;
    return (uint8_t) ((status->targetSession != last->targetSession) ||
        (status->routeSession != last->routeSession) ||
        (status->routeEpoch != last->routeEpoch) ||
        (status->ackEventSeq != last->ackEventSeq) ||
        (status->target != last->target) ||
        (status->activeTarget != last->activeTarget) ||
        (status->menuStates != last->menuStates) ||
        (status->flags != last->flags) ||
        (status->ackResult != last->ackResult));
}

static void cclHandleKvmIngress(const uartLinkFrameStruct *frame)
{
    core1KvmIngressStruct ingress;
    coreDataReadKvmIngress(&ingress);
    if (ingress.magic != CORE1_DATA_READY_MAGIC)
    {
        memset(&ingress, 0, sizeof(ingress));
        ingress.magic = CORE1_DATA_READY_MAGIC;
    }
    if ((ingress.routeSession != frame->kvmRouteSession) ||
        (ingress.routeEpoch != frame->kvmRouteEpoch) ||
        (ingress.target != frame->kvmTarget))
    {

```

```

    ingress.flags = 0u;
    ingress.eventSeq = 0u;
    cclState.kvmIngressBatchFlags = 0u;
}
if (frame->type == UART_LINK_TYPE_KVM_CONTROL)
{
    ingress.targetSession = frame->kvmTargetSession;
    ingress.routeSession = frame->kvmRouteSession;
    ingress.routeEpoch = frame->kvmRouteEpoch;
    ingress.leaseMs = frame->kvmLeaseMs;
    ingress.target = frame->kvmTarget;
    cclState.kvmIngressBatchFlags |= CORE1_KVM_INGRESS_CONTROL_VALID;
}
else
{
    ingress.targetSession = frame->kvmTargetSession;
    ingress.routeSession = frame->kvmRouteSession;
    ingress.routeEpoch = frame->kvmRouteEpoch;
    ingress.eventSeq = frame->kvmEventSeq;
    ingress.target = frame->kvmTarget;
    ingress.keyId = frame->kvmKeyId;
    ingress.keyState = frame->kvmKeyState;
    cclState.kvmIngressBatchFlags |= (CORE1_KVM_INGRESS_EVENT_VALID |
        CORE1_KVM_INGRESS_EVENT_EDGE);
}
ingress.flags = (uint8_t)((ingress.flags & CORE1_KVM_INGRESS_EVENT_VALID) |
    cclState.kvmIngressBatchFlags);
ingress.updateSeq = ++cclState.kvmIngressSeq;
coreDataWriteKvmIngress(&ingress);
}

/* 处理一帧: CORE0_STATUS 更新 core0 飞控状态(供 RGB 灯)。
 * 均 RMW(读出共享区→改本帧相关字段→写回), cm_7_0 单写, 保留其他字段不被互相清零。 */
static void cclHandleFrame(const uartLinkFrameStruct *frame)
{
    keyEventToMenuStruct ev;
    uint8_t statusWasRearming = 0u;
    if (frame->type == UART_LINK_TYPE_PARAM_RESPONSE)
    {
        cclState.paramResponse = *frame;
        cclState.paramResponsePending = 1u;
        return;
    }
    if ((frame->type == UART_LINK_TYPE_KVM_CONTROL) ||
        (frame->type == UART_LINK_TYPE_KVM_EVENT))
    {
        cclHandleKvmIngress(frame);
        return;
    }
    if (UART_LINK_TYPE_CORE0_STATUS == frame->type)
    {
        statusWasRearming = cclState.core0StatusRearmRequired;
        if (cclCore0StatusSemanticValid(frame) == 0u)
        {
            /* 语义非法帧撤销业务事实, 后续只能由安全非发车帧重建序列基线。 */
            if ((cclState.core0StatusValid != 0u) ||
                (cclState.core0SafetyState != CCL_CORE0_SAFETY_UNKNOWN))
            {
                cclInvalidateCore0Status(1u);
            }
            else
            {
                cclState.core0StatusRearmRequired = 1u;
            }
            cclState.rxErrorCount++;
            return;
        }
    }
    if (cclState.core0StatusRearmRequired != 0u)
    {
        /* 单消息静默超时但 wire 序列仍连续时, 允许前进的新帧在飞行中恢复。
         * 只有重复/旧 seq 或对端重启回退时, 才要求安全未发车帧重建基线。 */
        uint8_t forward = (uint8_t)((cclState.core0StatusWirePrimed != 0u) &&
            (cclWireSeqAdvanced(frame->sequence, cclState.core0StatusWireSeq) != 0u));
        if ((forward == 0u) && (cclCore0StatusIsSafeRearm(frame) == 0u)) return;
    }
    else if ((cclState.core0StatusWirePrimed != 0u) &&
        (cclWireSeqAdvanced(frame->sequence, cclState.core0StatusWireSeq) == 0u))
    {
        return;
    }
}

```

```

    }
}
coreDataReadKeyEvent(&ev);
if (ev.ready != CORE1_DATA_READY_MAGIC)
{
    /* 上电首帧前共享区可能未握手: 用零值起底 + 置 ready, 防 __no_init 垃圾。 */
    ev.linkUp = 1u;
    ev.controlActive = 0u;
    ev.core0SafetyState = 0u;
    ev.core0ArmProgress = 0u;
    ev.core0StatusFlags = 0u;
    ev.core0StatusValid = 0u;
    ev.core0SystemMode = 0u;
    ev.core0RgbIntent = 0u;
    ev.core2LinkLost = cclState.core2LinkLost;
    ev.core0AltReached = 0u;
    for (uint8_t i = 0u; i < sizeof(ev.padding); i++) ev.padding[i] = 0u;
    ev.ready = CORE1_DATA_READY_MAGIC;
}

if (UART_LINK_TYPE_CORE0_STATUS == frame->type)
{
    ev.core0SafetyState = frame->safetyState;
    ev.core0ArmProgress = frame->armProgress;
    ev.core0StatusFlags = frame->statusFlags;
    ev.core0StatusValid = 1u;
    ev.core0SystemMode = (uint8_t) (frame->sysCtrl & UART_LINK_SYS_MODE_MASK);
    ev.core0RgbIntent = (uint8_t) ((frame->sysCtrl >> UART_LINK_SYS_RGB_SHIFT) & UART_LINK_SYS_RGB_MASK);
    ev.core0AltReached = ((frame->sysCtrl & UART_LINK_SYS_ALT_REACHED) != 0u) ? 1u : 0u;
    ev.linkUp = 1u; /* 收到 core0 帧即在线 */
    coreDataWriteKeyEvent(&ev);
    /* 本地镜像供 cm_7_0 取: 发车态(LAUNCHED) / 系统模式 / 到顶 / 蜂鸣码 / 安全态 */
    cclState.core0StatusFlags = frame->statusFlags;
    cclState.core0SysCtrl = frame->sysCtrl;
    cclState.core0Buzzer = frame->buzzer;
    cclState.core0SafetyState = frame->safetyState;
    cclState.core0StatusValid = 1u; /* 收到有效 CORE0_STATUS: 地面态判据置有效(断链 Poll 会清回 0) */
    cclState.lastCore0StatusTickMs = cclState.tickMs;
    cclState.core0StatusWireSeq = frame->sequence;
    cclState.core0StatusWirePrimed = 1u;
    cclState.core0StatusRearmRequired = 0u;
    if (statusWasRearming != 0u)
        cclState.core0PeerRearmPending = 1u;
}
else if (UART_LINK_TYPE_CORE0_POSE == frame->type)
{
    core0PoseToVisionStruct pose;
    coreDataReadCore0Pose(&pose);
    if (pose.magic != CORE1_C0POSE_READY_MAGIC)
    {
        pose.seq32 = 0u;
        pose.padding[0] = 0u;
        pose.padding[1] = 0u;
        pose.padding[2] = 0u;
    }
    cclState.poseSeq32++;
    pose.magic = CORE1_C0POSE_READY_MAGIC;
    pose.seq32 = cclState.poseSeq32;
    pose.poseEpoch = frame->poseEpoch;
    pose.confidenceQ8 = frame->poseConfidenceQ8;
    pose.flags = frame->poseFlags;
    pose.uartSeq = frame->sequence;
    pose.xQ8 = frame->poseXQ8;
    pose.yQ8 = frame->poseYQ8;
    pose.orientationRad = frame->poseOrientationRad;
    coreDataWriteCore0Pose(&pose);
}
/* 其他帧(心跳)在 cclHandleByte 已处理, 不入此函数 */
}

void crossCoreLinkInit(void)
{
    cclState.rxHead = 0u;
    cclState.rxTail = 0u;
    cclState.rxCount = 0u;
    cclState.tickMs = 0u;
    cclState.lastRxTickMs = 0u;
    cclState.lastHeartbeatTickMs = 0u;
    cclState.linkPrimed = 0u;
}

```

```

cclState.heartbeatSequence = 0u;
cclState.motionSequence = 0u;
cclState.statusSequence = 0u;
cclState.producerBootSession = cclNextBootSession();
cclState.kvmStatusSequence = 0u;
cclState.kvmIngressSeq = 0u;
cclState.core2LinkLost = 0u; /* 上电默认未失联; core0Loop 每轮按 RS-485 存活刷新 */
cclState.requiredVisionFault = 1u;
cclState.faultSource = REQUIRED_VISION_SOURCE_CORE1;
cclState.faultClass = REQUIRED_VISION_FAULT_LINK;
cclState.statusDirty = 1u;
cclState.mode2LoraLandReq = 0u;
cclState.beaconCount = 0u;
cclState.beaconSearching = 0u;
cclState.beaconSearchTimedOut = 0u;
cclState.beaconStateValid = 0u;
cclState.poseSeq32 = 0u;
cclState.rxFrameCount = 0u;
cclState.rxErrorCount = 0u;
cclState.paramResponsePending = 0u;
cclState.paramTxSequence = 0u;
memset(&cclState.paramGlobalRequest, 0, sizeof(cclState.paramGlobalRequest));
memset(&cclState.paramGlobalState, 0, sizeof(cclState.paramGlobalState));
cclState.paramGlobalRequestPending = 0u;
cclState.paramGlobalStateValid = 0u;
cclState.paramGlobalStateDirty = 0u;
cclState.paramGlobalTxSequence = 0u;
cclState.lastParamGlobalStateTickMs = 0u;
cclState.core0ControlActive = 0u;
cclState.core0StatusFlags = 0u;
cclState.core0SysCtrl = 0u;
cclState.core0Buzzer = 0u;
cclState.core0SafetyState = CCL_CORE0_SAFETY_UNKNOWN; /* 未收 CORE0_STATUS 前非 DISARMED(fail-closed) */
cclState.core0StatusValid = 0u;
cclState.lastCore0StatusTickMs = 0u;
cclState.core0StatusWirePrimed = 0u;
cclState.core0StatusWireSeq = 0u;
cclState.core0StatusRearmRequired = 1u;
cclState.core0PeerRearmPending = 0u;
uartLinkProtoInit(&cclState.parser);

uart_init(CCL_UART_NUM, CCL_BAUDRATE, CCL_UART_TX_PIN, CCL_UART_RX_PIN);
uart_rx_interrupt(CCL_UART_NUM, 1u);

/* 上电先写一帧共享内存(linkUp=0), 让菜单核握手 ready 有效。 */
{
    keyEventToMenuStruct ev;
    ev.linkUp = 0u;
    ev.controlActive = 0u;
    ev.ready = CORE1_DATA_READY_MAGIC;
    ev.core0SafetyState = 0u;
    ev.core0ArmProgress = 0u;
    ev.core0StatusFlags = 0u;
    ev.core0StatusValid = 0u;
    ev.core0SystemMode = 0u;
    ev.core0RgbIntent = 0u;
    ev.core2LinkLost = 0u;
    ev.core0AltReached = 0u;
    for (uint8_t i = 0u; i < sizeof(ev.padding); i++) ev.padding[i] = 0u;
    coreDataWriteKeyEvent(&ev);
}

{
    core0PoseToVisionStruct pose;
    pose.magic = CORE1_C0POSE_READY_MAGIC;
    pose.seq32 = 0u;
    pose.poseEpoch = 0u;
    pose.confidenceQ8 = 0u;
    pose.flags = 0u;
    pose.uartSeq = 0u;
    pose.xQ8 = 0u;
    pose.yQ8 = 0u;
    pose.orientationRad = 0.0f;
    pose.padding[0] = 0u;
    pose.padding[1] = 0u;
    pose.padding[2] = 0u;
    coreDataWriteCore0Pose(&pose);
}

```

```

{
    core1KvmIngressStruct ingress;
    memset(&ingress, 0, sizeof(ingress));
    ingress.magic = CORE1_DATA_READY_MAGIC;
    coreDataWriteKvmIngress(&ingress);
}

cclState.initReadyFlag = 1u;
}

void crossCoreLinkPit1ms(void)
{
    if (!cclState.initReadyFlag) return;

    cclState.tickMs++;
}

void crossCoreLinkPoll(void)
{
    core1KvmStatusStruct kvmStatus;
    uint8_t kvmStatusValid;
    uint8_t kvmStatusSent = 0u;
    if (!cclState.initReadyFlag) return;

    uartLinkFrameStruct frame;
    uint8_t linkAlive = crossCoreLinkIsCore0Alive();

    /* EVENT 保持到 cm7_1 按事件身份消费；CONTROL 仍只代表本次 Poll 的新租约事实。 */
    cclState.kvmIngressBatchFlags = 0u;
    while (cclPopFrame(&frame))
    {
        cclHandleFrame(&frame);
    }
    linkAlive = crossCoreLinkIsCore0Alive();

    /* fail-closed: core0 断链即作废 CORE0_STATUS 派生态。
    * 重连后 core0StatusValid 保持 0，直到新 CORE0_STATUS 帧到达才重新置 1，
    * 杜绝“刚重连未收状态帧却沿用断链前发车/模式/蜂鸣/安全态”。 */
    if (!crossCoreLinkIsCore0Alive() ||
        ((cclState.core0StatusValid != 0u) &&
         ((uint32_t) (cclState.tickMs - cclState.lastCore0StatusTickMs) >= CCL_CORE0_STATUS_LOST_MS))) &&
        ((cclState.core0StatusValid != 0u) ||
         (cclState.core0SafetyState != CCL_CORE0_SAFETY_UNKNOWN)))
    {
        cclInvalidateCore0Status(1u);
    }

    /* 链路存活态/操控态变化时刷新共享内存(linkUp + controlActive)，让菜单核及时感知。
    * 断联时 controlActive 强制为 0；共享区 cm7_0 单写/cm7_1 单读。 */
    {
        uint8_t ctrl = (linkAlive && cclState.core0ControlActive) ? 1u : 0u;
        keyEventToMenuStruct ev;
        coreDataReadKeyEvent(&ev);
        if (ev.ready == CORE1_DATA_READY_MAGIC && (ev.linkUp != linkAlive || ev.controlActive != ctrl ||
            ev.core2LinkLost != cclState.core2LinkLost))
        {
            ev.linkUp = linkAlive;
            ev.controlActive = ctrl;
            ev.core2LinkLost = cclState.core2LinkLost;
            coreDataWriteKeyEvent(&ev);
        }
    }

    kvmStatusValid = coreDataTryReadKvmStatus(&kvmStatus);
    if ((kvmStatusValid != 0u) && (cclKvmStatusSemanticChanged(&kvmStatus) != 0u))
    {
        cclSendKvmStatus(&kvmStatus);
        kvmStatusSent = 1u;
    }

    if (cclState.statusDirty != 0u) cclSendStatus();
    if ((uint32_t) (cclState.tickMs - cclState.lastHeartbeatTickMs) >= CCL_HEARTBEAT_MS)
    {
        cclState.lastHeartbeatTickMs = cclState.tickMs;
        cclSendHeartbeat();
        cclSendStatus(); /* 与心跳同周期上报 core2 断联位给 core0(常发, 不受 MOTION 门控) */
        if ((kvmStatusValid != 0u) && (kvmStatusSent == 0u))
        {
            cclSendKvmStatus(&kvmStatus);
        }
    }
}

```

```

    }
}
if ((cclState.paramGlobalStateValid != 0u) &&
    ((cclState.paramGlobalStateDirty != 0u) ||
     ((uint32_t) (cclState.tickMs - cclState.lastParamGlobalStateTickMs) >=
      CCL_PARAM_GLOBAL_STATE_TX_MS)))
    cclSendParamGlobalState();
}

void crossCoreLinkRxIrqHandler(void)
{
    uint8_t data;
    if (uart_query_byte(CCL_UART_NUM, &data))
    {
        cclHandleByte(data);
    }
}

uint8_t crossCoreLinkSendParamRequest(const uartLinkFrameStruct *request)
{
    uint8_t buffer[UART_LINK_FRAME_LENGTH_KVM_STATUS];
    uartLinkFrameStruct frame;
    if (request == NULL || request->paramLength > UART_LINK_PARAM_DATA_BYTES)
        return 0u;
    frame = *request;
    frame.type = UART_LINK_TYPE_PARAM_REQUEST;
    frame.sequence = cclState.paramTxSequence++;
    if (uartLinkProtoEncode(&frame, buffer, sizeof(buffer)) != UART_LINK_PROTO_STATUS_OK)
        return 0u;
    uart_write_buffer(CCL_UART_NUM, buffer, sizeof(buffer));
    return 1u;
}

uint8_t crossCoreLinkTakeParamResponse(uartLinkFrameStruct *response)
{
    if (response == NULL || !cclState.paramResponsePending) return 0u;
    *response = cclState.paramResponse;
    cclState.paramResponsePending = 0u;
    return 1u;
}

void crossCoreLinkPublishParamGlobalState(const paramSyncGlobalState *state)
{
    if ((state == 0) || (paramSyncGlobalStateIsValid(state) == 0u)) return;
    cclState.paramGlobalState = *state;
    cclState.paramGlobalStateValid = 1u;
    cclState.paramGlobalStateDirty = 1u;
}

uint8_t crossCoreLinkTakeParamGlobalRequest(paramSyncGlobalRequest *request)
{
    uint8_t valid = 0u;
    if (request == 0) return 0u;
    uart_rx_interrupt(CCL_UART_NUM, 0u);
    if (cclState.paramGlobalRequestPending == 0u)
    {
        uart_rx_interrupt(CCL_UART_NUM, 1u);
        return 0u;
    }
    if (paramSyncGlobalRequestIsValid(&cclState.paramGlobalRequest) == 0u)
    {
        cclState.paramGlobalRequestPending = 0u;
    }
    else
    {
        *request = cclState.paramGlobalRequest;
        cclState.paramGlobalRequestPending = 0u;
        valid = 1u;
    }
    uart_rx_interrupt(CCL_UART_NUM, 1u);
    return valid;
}

void crossCoreLinkSendMotion(float vx, float vy, float wz, uint8_t remoteTakeoffReq)
{
    uint8_t buffer[UART_LINK_FRAME_LENGTH_MOTION];
    uartLinkFrameStruct frame;
    if (!cclState.initReadyFlag) return;
    frame.type = UART_LINK_TYPE_MOTION;

```

```

    frame.sequence = cclState.motionSequence;
    frame.motionVx = vx;
    frame.motionVy = vy;
    frame.motionWz = wz;
    /* flags bit0=remoteTakeoffReq: 模式3车 LoRa S3+S4 请求中转给 core0, 由 core0 按阶段解释。 */
    frame.motionFlags = remoteTakeoffReq ? UART_LINK_MOTION_FLAG_TAKEOFF_REQ : 0u;
    frame.producerBootSession = cclState.producerBootSession;
    cclEncodeWriteAdvance(&frame, buffer, UART_LINK_FRAME_LENGTH_MOTION,
        &cclState.motionSequence);
}

static uint8_t crossCoreLinkIsCore0Alive(void)
{
    return (cclState.linkPrimed &&
        (uint32_t) (cclState.tickMs - cclState.lastRxTickMs) < CCL_LINK_LOST_MS)
        ? 1u
        : 0u;
}

/* 设置模式2 LoRa 双肩键缓降请求位。该位与 MOTION flags bit0 解耦:
 * 模式2 只经 STATUS bit2 转发给 core0 进入缓降, 绝不触发起飞时序。 */
static void crossCoreLinkSetMode2LoraLandReq(uint8_t req)
{
    cclState.mode2LoraLandReq = req ? 1u : 0u;
}

/* 设置 RS-485↔core2 断联位(core0Loop 前台每轮按 core1Core2Alive() 调)。
 * cclSendStatus 与心跳同周期(200ms)把此位经 STATUS 帧 linkFlags bit1 上报 core0 供缓降聚合。 */
static void crossCoreLinkSetCore2LinkLost(uint8_t lost)
{
    {
        uint8_t val = lost ? 1u : 0u;
        if (cclState.core2LinkLost != val)
        {
            keyEventToMenuStruct ev;
            cclState.core2LinkLost = val;
            coreDataReadKeyEvent(&ev);
            if (ev.ready == CORE1_DATA_READY_MAGIC)
            {
                ev.core2LinkLost = val;
                coreDataWriteKeyEvent(&ev);
            }
        }
    }
}

/* 设置通用必需视觉故障事实。调用方合并物理链路、协议、相机、配置、标定与 plan ready/新鲜度门;
 * 本 owner 只缓存并随固定 14B STATUS bit3 发送, Core0 始终解释。 */
void crossCoreLinkGetStatus(crossCoreLinkStatusStruct *out)
{
    {
        uint32_t now;
        uint32_t lastRx;
        uint8_t alive;
        uint8_t statusValid;
        uint8_t sysCtrl;
        if (out == NULL) return;

        /* 单次捕获存活时基与状态字, 避免恰逢 600ms 边沿时由多个碎 getter
         * 拼出跨时刻组合。状态字段只由本前台 Poll 提交, tick/lastRx 为 32 位原子读。 */
        now = cclState.tickMs;
        lastRx = cclState.lastRxTickMs;
        alive = (uint8_t) ((cclState.linkPrimed != 0u) &&
            ((uint32_t) (now - lastRx) < CCL_LINK_LOST_MS));
        statusValid = (uint8_t) ((alive != 0u) &&
            (cclState.core0StatusValid != 0u) &&
            ((uint32_t) (now - cclState.lastCore0StatusTickMs) < CCL_CORE0_STATUS_LOST_MS));
        sysCtrl = cclState.core0SysCtrl;

        out->core0Alive = alive;
        out->core0StatusValid = statusValid;
        out->core0SafetyState = (statusValid != 0u) ? cclState.core0SafetyState : CCL_CORE0_SAFETY_UNKNOWN;
        out->core0Launched = (uint8_t) ((statusValid != 0u) &&
            ((cclState.core0StatusFlags & UART_LINK_C0_FLAG_LAUNCHED) != 0u));
        out->systemMode = (statusValid != 0u) ? (uint8_t) (sysCtrl & UART_LINK_SYS_MODE_MASK) : 0u;
        out->rgbIntent = (statusValid != 0u) ? (uint8_t) ((sysCtrl >> UART_LINK_SYS_RGB_SHIFT) & UART_LINK_SYS_RGB_MASK) : 0u;
        out->altReached = (uint8_t) ((statusValid != 0u) &&
            ((sysCtrl & UART_LINK_SYS_ALT_REACHED) != 0u));
        out->buzzer = (statusValid != 0u) ? cclState.core0Buzzer : 0u;
    }
}

uint8_t crossCoreLinkTakeCore0PeerRearm(void)

```

```

{
    uint8_t pending = cclState.core0PeerRearmPending;
    cclState.core0PeerRearmPending = 0u;
    return pending;
}

void crossCoreLinkPublishLocalFacts(const crossCoreLinkLocalFactsStruct *facts)
{
    uint8_t active;
    uint8_t source;
    uint8_t faultClass;
    if (facts == NULL) return;
    crossCoreLinkSetCore2LinkLost(facts->core2LinkLost);
    active = facts->core2AttitudeFault ? 1u : 0u;
    if (cclState.core2AttitudeFault != active)
    {
        cclState.core2AttitudeFault = active;
        cclState.statusDirty = 1u;
    }
    active = facts->requiredVisionFault ? 1u : 0u;
    source = active ? facts->faultSource : REQUIRED_VISION_SOURCE_NONE;
    faultClass = active ? facts->faultClass : REQUIRED_VISION_FAULT_NONE;
    if ((cclState.requiredVisionFault != active) ||
        (cclState.faultSource != source) ||
        (cclState.faultClass != faultClass))
    {
        cclState.requiredVisionFault = active;
        cclState.faultSource = source;
        cclState.faultClass = faultClass;
        cclState.statusDirty = 1u;
    }
    crossCoreLinkSetMode2LoraLandReq(facts->mode2LoraLandReq);
    cclState.beaconCount = facts->beaconCount;
    cclState.beaconSearching = facts->beaconSearching ? 1u : 0u;
    cclState.beaconSearchTimedOut = facts->beaconSearchTimedOut ? 1u : 0u;
    cclState.beaconStateValid = facts->beaconStateValid ? 1u : 0u;
}

```

core1_cyt4bb7/project/code/comm/crossCoreLink.h

SHA-256

:

3c8cffb6cb2f0ed5a3faa9b8fb7e59119c1d2b3e09201d605629bed46e14428b

代码 (c)

```

#ifndef _CROSS_CORE_LINK_H_
#define _CROSS_CORE_LINK_H_

#include <stdint.h>
#include "uartLinkProto.h"

/*
 * crossCoreLink (core1 cm_7_0 端) - 核间单线双向链路
 *
 * 物理: UART_1 / P04 (group0) ↔ core0 P10 / UART_2, 4 Mbps, 全双工单线双向。
 * - RX: 收 core0 状态、位姿与事务 KVM 帧; 合法帧刷新 core0 存活计时。
 * - TX: 周期发 HEARTBEAT 帧(前台, 200ms) 给 core0 作存活信标。
 *
 * 替代原 uartLinkTest 的 RESPONDER 角色(已移除)。
 * RX 在 ISR 仅取字节喂解析器(轻量); 写共享内存(含 DCache)/发送均在前台 Poll, 绝不进 ISR。
 *
 * 链路状态由 crossCoreLinkGetStatus() 一次返回同拍快照, 供调用方作断联保护(>600ms 无帧→断联)。
 */

void crossCoreLinkInit(void); /* 配 UART_1 + 开 RX 中断 + 复位状态 */
void crossCoreLinkPit1ms(void); /* 1ms tick: 心跳分频 + core0 存活超时计数(ISR 安全) */
void crossCoreLinkPoll(void); /* 前台: 排空 RX 队列→写共享内存; 发心跳 */
void crossCoreLinkRxIrqHandler(void); /* UART_1 RX 中断: 仅取一字节喂解析器 */

typedef struct
{
    uint8_t core0Alive;
    uint8_t core0StatusValid;
    uint8_t core0SafetyState;
    uint8_t core0Launched;

```



```
uint8_t systemMode;
uint8_t rgbIntent;
uint8_t altReached;
uint8_t buzzer;
} crossCoreLinkStatusStruct;

typedef struct
{
    uint8_t core2LinkLost;
    uint8_t core2AttitudeFault;
    uint8_t requiredVisionFault;
    uint8_t faultSource;
    uint8_t faultClass;
    uint8_t mode2LoraLandReq;
    uint16_t beaconCount;
    uint8_t beaconSearching;
    uint8_t beaconSearchTimedOut;
    uint8_t beaconStateValid;
} crossCoreLinkLocalFactsStruct;

void crossCoreLinkGetStatus(crossCoreLinkStatusStruct *out);
/* CORE0 STATUS 在初始化/超时/序号回退后重新建立时返回一次 1；调用方据此撤销
 * MOTION 发送 priming，使下一次非零请求先发送显式零 fence。 */
uint8_t crossCoreLinkTakeCore0PeerRearm(void);
void crossCoreLinkPublishLocalFacts(const crossCoreLinkLocalFactsStruct *facts);
uint8_t crossCoreLinkSendParamRequest(const uartLinkFrameStruct *request);
uint8_t crossCoreLinkTakeParamResponse(uartLinkFrameStruct *response);
void crossCoreLinkPublishParamGlobalState(const paramSyncGlobalState *state);
uint8_t crossCoreLinkTakeParamGlobalRequest(paramSyncGlobalRequest *request);

/* core0 飞控安全态字节(经 CORE0 STATUS 帧透传, 与 core0 pidFlightController.h safetyStateEnum 同值):
 * 0=DISARMED, 1=CLOSED_LOOP, 2=PROTECTED, 0xFE=ARMING, 0xFF=IMUFAIL。
 * UNKNOWN=0xFD(与任一真实值不同且≠DISARMED), 表示“尚未收到有效 CORE0 STATUS 或 core0 已断链”。 */
#define CCL_CORE0_SAFETY_DISARMED (0x00u)
#define CCL_CORE0_SAFETY_UNKNOWN (0xFDu)

/* fail-closed 地面态判据(供 cm_7_0 断链信标视觉调试 gate):
 * IsCore0StatusValid() = 已收到过 CORE0 STATUS 且 core0 链路在线(断链即失效: 重连未收新帧前不为真)。
 * GetCore0SafetyState() = 最近 CORE0 STATUS 的 safetyState 字节; status invalid 时返回 UNKNOWN(≠DISARMED),
 * 保证 invalid 时绝不返回 DISARMED, 杜绝断链/刚重连误判为地面态。 */
/* 1=core0 飞控已发车(最近 CORE0 STATUS 的 LAUNCHED 位)。供 cm_7_0 中转为 RS-485 cmd LAUNCH 给 core2。
 * CORE0 STATUS invalid(断链/刚重连未收新状态帧)时返回 0, fail-closed 判未发车。 */
/* 取 core0 下发的空地系统控制(最近 CORE0 STATUS 的 sysCtrl/buzzer), 供 cm_7_0 设 core2 控制源/中转蜂鸣/门控。
 * CORE0 STATUS invalid 时均返回 0(未设/不蜂鸣/未到顶), 避免复用断链前镜像。 */
/* 前台发送 MOTION 帧给 core0: vx/vy/wz(float 透传, 来自 core2 RS-485 LoRa 遥控值, 供无人机前馈)
 * + remoteTakeoffReq(模式 3 车 LoRa S3+S4 请求, core0 按 TKO 阶段解释为起飞或缓降)。 */
void crossCoreLinkSendMotion(float vx, float vy, float wz, uint8_t remoteTakeoffReq);

/* 设置模式 2 LoRa 双肩键缓降请求位(core0Loop 前台按 TELEMETRY status bit4 + 模式/链路门控刷新)。
 * 经 STATUS 帧 linkFlags bit2 上报 core0; core0 只解释为 manualLandRequest, 不请求起飞。 */
/* 设置 RS-485↔core2 断联位(core0Loop 前台每轮按 core1Core2Alive() 调)。与心跳同周期经 STATUS 帧
 * linkFlags bit1 上报 core0(常发, 不受 MOTION 门控), 供 core0 landingSupervisor 聚合缓降。 */
#endif /* _CROSS_CORE_LINK_H_ */
```

core1_cyt4bb7/project/code/comm/observerLinkConfigCore1.c

SHA-256 :

8c52d723da639eedc7fb5339d321eaeddc76fa17b87d973d8d84cc65d491539f

代码 (c)

```
#include "observerLinkConfigCore1.h"

#include <limits.h>
#include <string.h>

#include "configBeaconDetect.h"
#include "core4LinkCore1.h"
#include "visionParams.h"

typedef struct
{
    uint8_t initialized;
    uint32_t revision;
    uint32_t hash;
```

```

    uint32_t session;
} observerEndpointConfigIdentity;

static observerEndpointConfigIdentity g_core3Identity;
static observerEndpointConfigIdentity g_core4Identity;

static void buildConfig(const visionParamsStruct *params,
    uint8_t sourceId,
    observerLinkSharedConfig *config)
{
    memset(config, 0, sizeof(*config));
    config->detectMode = params->detectMode;
    config->maxBeacons = params->maxBeacons;
    config->edgePredictionHoldMs = params->planningEdgePredictionHoldMs;
    config->sourceId = sourceId;
    config->profileVersion = 1u;
    config->c1BorderEnterLeft = params->c1BorderEnterLeft;
    config->c1BorderEnterRight = params->c1BorderEnterRight;
    config->c1BorderEnterTop = params->c1BorderEnterTop;
    config->c1BorderEnterBottom = params->c1BorderEnterBottom;
    config->c1BorderExitLeft = params->c1BorderExitLeft;
    config->c1BorderExitRight = params->c1BorderExitRight;
    config->c1BorderExitTop = params->c1BorderExitTop;
    config->c1BorderExitBottom = params->c1BorderExitBottom;
    config->observerBorderEnterLeft = (sourceId == OBSERVER_LINK_SOURCE_CORE4) ?
        params->c4BorderEnterLeft : params->observerBorderEnterLeft;
    config->observerBorderEnterRight = (sourceId == OBSERVER_LINK_SOURCE_CORE4) ?
        params->c4BorderEnterRight : params->observerBorderEnterRight;
    config->observerBorderEnterTop = (sourceId == OBSERVER_LINK_SOURCE_CORE4) ?
        params->c4BorderEnterTop : params->observerBorderEnterTop;
    config->observerBorderEnterBottom = (sourceId == OBSERVER_LINK_SOURCE_CORE4) ?
        params->c4BorderEnterBottom : params->observerBorderEnterBottom;
    config->observerBorderExitLeft = (sourceId == OBSERVER_LINK_SOURCE_CORE4) ?
        params->c4BorderExitLeft : params->observerBorderExitLeft;
    config->observerBorderExitRight = (sourceId == OBSERVER_LINK_SOURCE_CORE4) ?
        params->c4BorderExitRight : params->observerBorderExitRight;
    config->observerBorderExitTop = (sourceId == OBSERVER_LINK_SOURCE_CORE4) ?
        params->c4BorderExitTop : params->observerBorderExitTop;
    config->observerBorderExitBottom = (sourceId == OBSERVER_LINK_SOURCE_CORE4) ?
        params->c4BorderExitBottom : params->observerBorderExitBottom;
    config->c1EdgeBandLeft = params->c1EdgeBandLeft;
    config->c1EdgeBandRight = params->c1EdgeBandRight;
    config->c1EdgeBandTop = params->c1EdgeBandTop;
    config->c1EdgeBandBottom = params->c1EdgeBandBottom;
    config->observerEdgeBandLeft = (sourceId == OBSERVER_LINK_SOURCE_CORE4) ?
        params->c4EdgeBandLeft : params->observerEdgeBandLeft;
    config->observerEdgeBandRight = (sourceId == OBSERVER_LINK_SOURCE_CORE4) ?
        params->c4EdgeBandRight : params->observerEdgeBandRight;
    config->observerEdgeBandTop = (sourceId == OBSERVER_LINK_SOURCE_CORE4) ?
        params->c4EdgeBandTop : params->observerEdgeBandTop;
    config->observerEdgeBandBottom = (sourceId == OBSERVER_LINK_SOURCE_CORE4) ?
        params->c4EdgeBandBottom : params->observerEdgeBandBottom;
    config->beaconSizeMin = params->beaconSizeMin;
    config->beaconSizeMax = params->beaconSizeMax;
    config->beaconEccMax = params->beaconEccMax;
    config->beaconBrightnessMin = params->beaconBrightnessMin;
    config->beaconMaxEndpoints = params->beaconMaxEndpoints;
    config->beaconFillMin = params->beaconFillMin;
    config->beaconHuMax = params->beaconHuMax;
    config->beaconHuAreaMin = params->beaconHuAreaMin;
    config->trackMaxAssocDist = params->trackMaxAssocDist;
    config->trackSizeRatioMax = params->trackSizeRatioMax;
    config->trackConfirmHits = params->trackConfirmHits;
    config->trackDropMisses = params->trackDropMisses;
    config->trackCoastMisses = params->trackCoastMisses;
    config->trackPositionAlpha = params->trackPositionAlpha;
    config->trackSizeAlpha = params->trackSizeAlpha;
    config->trackAreaVelAlpha = params->trackAreaVelAlpha;
    config->trackAssocScoreMax = params->trackAssocScoreMax;
    config->trackAssocAreaWeight = params->trackAssocAreaWeight;
    config->trackAssocAreaVelWeight = params->trackAssocAreaVelWeight;
    config->beaconHuTemplate[0] = VISION_BEACON_HU_TEMPLATE_0;
    config->beaconHuTemplate[1] = VISION_BEACON_HU_TEMPLATE_1;
    config->beaconHuTemplate[2] = VISION_BEACON_HU_TEMPLATE_2;
    config->beaconHuTemplate[3] = VISION_BEACON_HU_TEMPLATE_3;
    config->beaconHuTemplate[4] = VISION_BEACON_HU_TEMPLATE_4;
    config->beaconHuTemplate[5] = VISION_BEACON_HU_TEMPLATE_5;
    config->beaconHuTemplate[6] = VISION_BEACON_HU_TEMPLATE_6;
}

```

```

static uint32_t nextRevision(uint32_t revision)
{
    return (revision == UINT32_MAX) ? 1u : (revision + 1u);
}

static uint32_t refreshCore3Identity(const visionParamsStruct *params,
uint32_t hash)
{
    if (g_core3Identity.initialized == 0u)
    {
        g_core3Identity.revision = ((params->sharedConfigRevision != 0u) &&
(params->sharedConfigHash == hash)) ?
params->sharedConfigRevision : nextRevision(params->sharedConfigRevision);
        g_core3Identity.initialized = 1u;
    }
    else if (g_core3Identity.hash != hash)
    {
        g_core3Identity.revision = nextRevision(g_core3Identity.revision);
    }
    g_core3Identity.hash = hash;
    if ((params->sharedConfigRevision != g_core3Identity.revision) ||
(params->sharedConfigHash != hash))
    {
        (void) visionParamsUpdateSharedIdentity(g_core3Identity.revision, hash);
    }
    return g_core3Identity.revision;
}

static uint32_t refreshCore4Identity(uint32_t session, uint32_t hash)
{
    if (session == 0u) return 0u;
    if ((g_core4Identity.initialized == 0u) ||
(g_core4Identity.session != session))
    {
        /* Core4 没有 Flash 身份槽；每个独立 endpoint session 建立自己的运行时 seed。 */
        g_core4Identity.revision = session;
        g_core4Identity.session = session;
        g_core4Identity.initialized = 1u;
    }
    else if (g_core4Identity.hash != hash)
    {
        g_core4Identity.revision = nextRevision(g_core4Identity.revision);
    }
    g_core4Identity.hash = hash;
    return g_core4Identity.revision;
}

uint8_t observerLinkConfigCore1Provide(void *providerContext,
uint8_t sourceId,
observerLinkSharedConfig *config)
{
    visionParamsStruct params;
    observerLinkSharedConfig desired;
    uint32_t revision;
    uint32_t hash;
    (void) providerContext;
    if ((config == NULL) ||
((sourceId != OBSERVER_LINK_SOURCE_CORE3) &&
(sourceId != OBSERVER_LINK_SOURCE_CORE4))) return 0u;
    visionParamsGetActive(&params);
    buildConfig(&params, sourceId, &desired);
    hash = observerLinkSharedConfigHash(&desired);
    if (sourceId == OBSERVER_LINK_SOURCE_CORE3)
    {
        revision = refreshCore3Identity(&params, hash);
    }
    else
    {
        core4LinkCore1Status status;
        core4LinkCore1GetStatus(&status);
        revision = refreshCore4Identity(status.localSession, hash);
    }
    if (revision == 0u) return 0u;
    *config = desired;
    config->revision = revision;
    config->hash = hash;
    return observerLinkSharedConfigIsValid(config);
}

```

core1_cyt4bb7/project/code/comm/observerLinkConfigCore1.h

SHA-256

:

499f91ee80c7c45b3b1d8a9da11c725233a7a018b11806a0349a2980787b3910

代码 (c)

```
#ifndef OBSERVER_LINK_CONFIG_CORE1_H
#define OBSERVER_LINK_CONFIG_CORE1_H

#include <stdint.h>

#include "observerLinkProtocol.h"

/* C3 使用当前 v46 持久化 sharedConfigRevision seed; C4 按独立 endpoint localSession
 * 建立本次运行 revision seed。两端 hash/APPLIED 身份互不递增或覆盖。 */
uint8_t observerLinkConfigCore1Provide(void *providerContext,
    uint8_t sourceId,
    observerLinkSharedConfig *config);

#endif /* OBSERVER_LINK_CONFIG_CORE1_H */
```

core1_cyt4bb7/project/code/comm/observerLinkCore1Endpoint.c

SHA-256

:

026b8c7cf019aa0b603b4e4621bda231e68cc6d0f2ac9568ce95e41ff8144d1d

代码 (c)

```
#include "observerLinkCore1Endpoint.h"

#include <stddef.h>
#include <string.h>

#include "zf_driver_uart.h"

#if defined(_ICCARM_)
#include "cy_device_headers.h"
#include "scb/cy_scb_uart.h"
#define ENDPOINT_UART(value) ((uart_index_enum)(value))
#define ENDPOINT_TX_PIN(value) ((uart_tx_pin_enum)(value))
#define ENDPOINT_RX_PIN(value) ((uart_rx_pin_enum)(value))
#else
#define ENDPOINT_UART(value) (value)
#define ENDPOINT_TX_PIN(value) (value)
#define ENDPOINT_RX_PIN(value) (value)
#endif

#define ENDPOINT_MAGIC (0x4F4C4531u)
#define ENDPOINT_RX_RING_BYTES (2048u)
#define ENDPOINT_RX_RING_MASK (ENDPOINT_RX_RING_BYTES - 1u)
#define ENDPOINT_HELLO_PERIOD_MS (500u)
#define ENDPOINT_HEARTBEAT_PERIOD_MS (200u)
#define ENDPOINT_CONFIG_PERIOD_MS (200u)
#define ENDPOINT_CONFIG_REFRESH_MS (1000u)
#define ENDPOINT_DIAG_PERIOD_MS (1000u)
#define ENDPOINT_TIMEOUT_MS (600u)
#define ENDPOINT_OVERLAY_MAX_AGE_MS (100u)

typedef struct
{
    uint32_t magic;
    observerLinkCore1EndpointDescriptor descriptor;
    observerLinkParser parser;
    observerLinkCore1EndpointStatus status;
    observerLinkSharedConfig desired;
    observerLinkKvmControl kvmControl;
    observerLinkKvmEvent kvmEvent;
    observerLinkKvmAck kvmAck;
    observerLinkKvmMenuStatus kvmMenuStatus;
```

```

observerLinkControlStatus controlStatus;
observerLinkViewOverlay viewOverlay;
observerLinkPlanningTrackSnapshot planningBuild;
observerLinkPlanningTrackSnapshot planningPublished;
uint32_t packetSeq;
uint32_t rxPacketSeq;
uint8_t rxPacketSeqPrimed;
uint32_t lastHelloTickMs;
uint32_t lastHeartbeatTickMs;
uint32_t lastConfigTickMs;
uint32_t lastDiagTickMs;
uint32_t txEpochTickMs;
uint32_t lastDeferredTickMs;
uint32_t kvmMenuTickMs;
uint32_t planningPublishedTickMs;
uint32_t planningTransportAgeMs;
uint32_t viewOverlayTickMs;
volatile uint32_t rxRingOverflow;
volatile uint16_t rxHead;
volatile uint16_t rxTail;
uint8_t rxRing[ENDPOINT_RX_RING_BYTES];
uint8_t kvmControlPending;
uint8_t kvmEventPending;
uint8_t kvmAckReady;
uint8_t kvmMenuValid;
uint8_t kvmControlArmed;
uint8_t planningBuildMask;
uint8_t planningExpectedMask;
uint8_t planningPublishedValid;
uint8_t viewOverlayPending;
uint8_t txDataCredits;
uint8_t txControlCredits;
uint8_t paramRequest[OBSERVER_LINK_MAX_PAYLOAD_BYTES];
uint8_t paramResponse[OBSERVER_LINK_MAX_PAYLOAD_BYTES];
uint16_t paramRequestLength;
uint16_t paramResponseLength;
uint8_t paramRequestPending;
uint8_t paramResponseReady;
paramSyncGlobalState paramGlobalState;
paramSyncGlobalRequest paramGlobalRequest;
uint8_t paramGlobalStatePending;
uint8_t paramGlobalRequestReady;
} endpointPrivate;

typedef char endpoint_context_size_assert[(sizeof(endpointPrivate) <= OBSERVER_LINK_CORE1_ENDPOINT_CONTEXT_BYTES) ? 1 : -1];
typedef char endpoint_ring_size_assert[(ENDPOINT_RX_RING_BYTES & ENDPOINT_RX_RING_MASK) == 0u ? 1 : -1];

static endpointPrivate *privateFromContext(observerLinkCore1EndpointContext *context)
{
    if (context == NULL) return NULL;
    return (endpointPrivate *) (void *) context->bytes;
}

static endpointPrivate *initializedPrivate(observerLinkCore1EndpointContext *context)
{
    endpointPrivate *endpoint = privateFromContext(context);
    if ((endpoint == NULL) || (endpoint->magic != ENDPOINT_MAGIC)) return NULL;
    return endpoint;
}

static void publishBarrier(void)
{
    {
#ifdef __ICCARM__
        __DMB();
#elif defined(__GNUC__)
        __atomic_thread_fence(__ATOMIC_RELEASE);
#endif
    }
}

static uint8_t descriptorIsValid(
    const observerLinkCore1EndpointDescriptor *descriptor)
{
    if ((descriptor == NULL) || (descriptor->configProvider == NULL) ||
        ((descriptor->expectedRole != OBSERVER_LINK_ROLE_CORE3) &&
         (descriptor->expectedRole != OBSERVER_LINK_ROLE_CORE4)) ||
        ((descriptor->sourceId != OBSERVER_LINK_SOURCE_CORE3) &&
         (descriptor->sourceId != OBSERVER_LINK_SOURCE_CORE4)) ||
        (descriptor->expectedRole != descriptor->sourceId) ||
        (descriptor->kvmTarget != descriptor->sourceId))

```

```

    {
        return 0u;
    }
    return 1u;
}

#if defined(__ICCARM__)
__no_init uint32_t endpointBootCounterRetained;
__no_init uint32_t endpointBootCounterInverseRetained;
#else
static uint32_t endpointBootCounterRetained;
static uint32_t endpointBootCounterInverseRetained;
#endif
static uint32_t endpointBootGeneration;

static uint32_t getEndpointBootGeneration(void)
{
    if (endpointBootGeneration == 0u)
    {
        uint32_t value = endpointBootCounterRetained;
        if ((value == 0u) || (endpointBootCounterInverseRetained != ~value)) value = 0u;
        value++;
        if (value == 0u) value = 1u;
        endpointBootCounterRetained = value;
        endpointBootCounterInverseRetained = ~value;
        endpointBootGeneration = value;
    }
    return endpointBootGeneration;
}

static uint32_t makeBootSession(uint32_t tickMs)
{
    (void) tickMs;
    return getEndpointBootGeneration();
}

static uint8_t helloIdentifiesPeer(const endpointPrivate *endpoint,
    const observerLinkFrame *frame)
{
    uint8_t role;
    uint32_t capabilities;
    const uint32_t required = OBSERVER_LINK_CAP_BINARY_CC |
        OBSERVER_LINK_CAP_LEGACY_CANNY |
        OBSERVER_LINK_CAP_CONFIG_STAGING |
        OBSERVER_LINK_CAP_CONFIG_APPLY |
        OBSERVER_LINK_CAP_PLANNING_TRACKS |
        OBSERVER_LINK_CAP_CAMERA_CONTROL |
        OBSERVER_LINK_CAP_CONTROL_STATUS;
    return (uint8_t) ((frame->major == OBSERVER_LINK_PROTOCOL_MAJOR) &&
        (frame->minor == OBSERVER_LINK_PROTOCOL_MINOR) &&
        (observerLinkDecodeHello(frame->payload, frame->payloadLength,
            &role, &capabilities) != 0u) &&
        (role == endpoint->descriptor.expectedRole) &&
        ((capabilities & required) == required));
}

static void clearPlanningState(endpointPrivate *endpoint)
{
    memset(&endpoint->planningBuild, 0, sizeof(endpoint->planningBuild));
    memset(&endpoint->planningPublished, 0, sizeof(endpoint->planningPublished));
    endpoint->planningBuildMask = 0u;
    endpoint->planningExpectedMask = 0u;
    endpoint->planningPublishedValid = 0u;
    endpoint->planningPublishedTickMs = 0u;
    endpoint->planningTransportAgeMs = 0u;
}

static uint8_t kvmEligible(const endpointPrivate *endpoint)
{
    const uint32_t required = OBSERVER_LINK_CAP_KVM |
        OBSERVER_LINK_CAP_MENU_STATUS;
    return (uint8_t) ((endpoint->status.online != 0u) &&
        (endpoint->status.protocolCompatible != 0u) &&
        ((endpoint->status.peerCapabilities & required) == required));
}

static uint8_t paramSyncEligible(const endpointPrivate *endpoint)
{
    return (uint8_t) ((endpoint->status.online != 0u) &&

```

```

        (endpoint->status.protocolCompatible != 0u) &&
        ((endpoint->status.peerCapabilities & OBSERVER_LINK_CAP_PARAM_SYNC) != 0u));
    }

static void clearParamPeerState(endpointPrivate *endpoint)
{
    memset(endpoint->paramRequest, 0, sizeof(endpoint->paramRequest));
    memset(endpoint->paramResponse, 0, sizeof(endpoint->paramResponse));
    memset(&endpoint->paramGlobalRequest, 0,
        sizeof(endpoint->paramGlobalRequest));
    endpoint->paramRequestLength = 0u;
    endpoint->paramResponseLength = 0u;
    endpoint->paramRequestPending = 0u;
    endpoint->paramResponseReady = 0u;
    endpoint->paramGlobalRequestReady = 0u;
}

static void clearKvmState(endpointPrivate *endpoint)
{
    memset(&endpoint->kvmControl, 0, sizeof(endpoint->kvmControl));
    memset(&endpoint->kvmEvent, 0, sizeof(endpoint->kvmEvent));
    memset(&endpoint->kvmAck, 0, sizeof(endpoint->kvmAck));
    memset(&endpoint->kvmMenuStatus, 0, sizeof(endpoint->kvmMenuStatus));
    endpoint->kvmControlPending = 0u;
    endpoint->kvmEventPending = 0u;
    endpoint->kvmAckReady = 0u;
    endpoint->kvmMenuValid = 0u;
    endpoint->kvmControlArmed = 0u;
    endpoint->kvmMenuTickMs = 0u;
}

static void clearPeerState(endpointPrivate *endpoint)
{
    endpoint->status.protocolCompatible = 0u;
    endpoint->status.planningCompatible = 0u;
    endpoint->status.algorithmsReady = 0u;
    endpoint->status.configCompatible = 0u;
    endpoint->status.configAckStatus = OBSERVER_LINK_CONFIG_NONE;
    endpoint->status.configAckReason = OBSERVER_LINK_REJECT_NONE;
    endpoint->status.configAckDesiredMode = 0xFFu;
    endpoint->status.configAckRevision = 0u;
    endpoint->status.configAckHash = 0u;
    endpoint->status.configAckTickMs = 0u;
    endpoint->status.appliedMode = 0xFFu;
    endpoint->status.peerCapabilities = 0u;
    memset(&endpoint->status.peerDiagnostics, 0,
        sizeof(endpoint->status.peerDiagnostics));
    endpoint->status.peerDiagnosticsTickMs = 0u;
    endpoint->status.healthValid = 0u;
    memset(&endpoint->status.health, 0, sizeof(endpoint->status.health));
    endpoint->status.healthTickMs = 0u;
    clearKvmState(endpoint);
    memset(&endpoint->viewOverlay, 0, sizeof(endpoint->viewOverlay));
    endpoint->viewOverlayTickMs = 0u;
    endpoint->viewOverlayPending = 0u;
    clearParamPeerState(endpoint);
    clearPlanningState(endpoint);
}

static void invalidateConfig(endpointPrivate *endpoint)
{
    endpoint->status.configProviderValid = 0u;
    endpoint->status.configCompatible = 0u;
    endpoint->status.configAckStatus = OBSERVER_LINK_CONFIG_NONE;
    endpoint->status.configAckReason = OBSERVER_LINK_REJECT_NONE;
    endpoint->status.configAckDesiredMode = 0xFFu;
    endpoint->status.configAckRevision = 0u;
    endpoint->status.configAckHash = 0u;
    endpoint->status.configAckTickMs = 0u;
    clearPlanningState(endpoint);
}

static uint8_t syncConfig(endpointPrivate *endpoint)
{
    observerLinkSharedConfig next;
    uint8_t identityChanged;

    memset(&next, 0, sizeof(next));
    if ((endpoint->descriptor.configProvider(

```

```

        endpoint->descriptor.configProviderContext,
        endpoint->descriptor.sourceId,
        &next) == 0u) ||
    (next.sourceId != endpoint->descriptor.sourceId) ||
    (next.revision == 0u) ||
    (next.hash != observerLinkSharedConfigHash(&next)) ||
    (observerLinkSharedConfigsValid(&next) == 0u))
{
    invalidateConfig(endpoint);
    return 0u;
}

identityChanged = (uint8_t) ((endpoint->status.configProviderValid == 0u) ||
    (next.revision != endpoint->desired.revision) ||
    (next.hash != endpoint->desired.hash) ||
    (next.detectMode != endpoint->desired.detectMode) ||
    (next.sourceId != endpoint->desired.sourceId));
endpoint->desired = next;
endpoint->status.configProviderValid = 1u;
endpoint->status.desiredRevision = next.revision;
endpoint->status.desiredHash = next.hash;
endpoint->status.desiredMode = next.detectMode;
if (identityChanged != 0u)
{
    endpoint->status.configCompatible = 0u;
    endpoint->status.configAckStatus = OBSERVER_LINK_CONFIG_NONE;
    endpoint->status.configAckReason = OBSERVER_LINK_REJECT_NONE;
    endpoint->status.configAckDesiredMode = 0xFFu;
    endpoint->status.configAckRevision = 0u;
    endpoint->status.configAckHash = 0u;
    endpoint->status.configAckTickMs = 0u;
    clearPlanningState(endpoint);
}
return 1u;
}

static uint8_t sendMessageBudgeted(endpointPrivate *endpoint,
    uint8_t type,
    const uint8_t *payload,
    uint16_t payloadLength,
    uint32_t tickMs,
    uint8_t controlLane)
{
    observerLinkFrame frame;
    uint8_t wire[OBSERVER_LINK_MAX_FRAME_BYTES];
    uint16_t wireLength;
    uint8_t *credits = (controlLane != 0u) ? &endpoint->txControlCredits : &endpoint->txDataCredits;

    if ((payloadLength > OBSERVER_LINK_MAX_PAYLOAD_BYTES) ||
        ((payloadLength != 0u) && (payload == NULL)) || (*credits == 0u))
    {
        return 0u;
    }
    memset(&frame, 0, sizeof(frame));
    frame.major = OBSERVER_LINK_PROTOCOL_MAJOR;
    frame.minor = OBSERVER_LINK_PROTOCOL_MINOR;
    frame.type = type;
    frame.payloadLength = payloadLength;
    frame.bootSession = endpoint->status.localSession;
    endpoint->packetSeq++;
    if (endpoint->packetSeq == 0u) endpoint->packetSeq = 1u;
    frame.packetSeq = endpoint->packetSeq;
    frame.tickMs = tickMs;
    if (payloadLength != 0u)
    {
        memcpy(frame.payload, payload, payloadLength);
    }
    wireLength = observerLinkEncodeFrame(&frame, wire, (uint16_t) sizeof(wire));
    if (wireLength == 0u)
    {
        endpoint->status.txFailures++;
        return 0u;
    }
    uart_write_buffer(ENDPOINT_UART(endpoint->descriptor.uart), wire, wireLength);
    (*credits)--;
    endpoint->status.txFrames++;
    return 1u;
}

```



```

static uint8_t sendMessage(endpointPrivate *endpoint, uint8_t type,
    const uint8_t *payload, uint16_t length, uint32_t tickMs)
{
    return sendMessageBudgeted(endpoint, type, payload, length, tickMs, 0u);
}

static uint8_t sendControlMessage(endpointPrivate *endpoint, uint8_t type,
    const uint8_t *payload, uint16_t length, uint32_t tickMs)
{
    return sendMessageBudgeted(endpoint, type, payload, length, tickMs, 1u);
}

static void handleHello(endpointPrivate *endpoint,
    const observerLinkFrame *frame)
{
    uint8_t role;
    uint32_t capabilities;
    uint32_t requiredAlgorithms = OBSERVER_LINK_CAP_BINARY_CC |
        OBSERVER_LINK_CAP_LEGACY_CANNY |
        OBSERVER_LINK_CAP_CONFIG_APPLY;
    uint32_t configRelevantCapabilities = requiredAlgorithms |
        OBSERVER_LINK_CAP_CONFIG_STAGING |
        OBSERVER_LINK_CAP_PLANNING_TRACKS |
        OBSERVER_LINK_CAP_CAMERA_CONTROL |
        OBSERVER_LINK_CAP_CONTROL_STATUS;

    uint8_t protocolCompatible;
    uint8_t planningCompatible;
    uint8_t algorithmsReady;

    if ((observerLinkDecodeHello(frame->payload, frame->payloadLength,
        &role, &capabilities) == 0u) ||
        (role != endpoint->descriptor.expectedRole))
    {
        clearPeerState(endpoint);
        return;
    }

    protocolCompatible = (uint8_t) ((frame->minor == OBSERVER_LINK_PROTOCOL_MINOR) &&
        (frame->payload[2] == OBSERVER_LINK_PROTOCOL_MINOR) &&
        ((capabilities & OBSERVER_LINK_CAP_CONFIG_STAGING) != 0u));
    algorithmsReady = (uint8_t) ((capabilities & requiredAlgorithms) == requiredAlgorithms);
    planningCompatible = (uint8_t) ((protocolCompatible != 0u) &&
        ((capabilities & OBSERVER_LINK_CAP_PLANNING_TRACKS) != 0u) &&
        ((capabilities & OBSERVER_LINK_CAP_CAMERA_CONTROL) != 0u) &&
        ((capabilities & OBSERVER_LINK_CAP_CONTROL_STATUS) != 0u));

    if (((capabilities ^ endpoint->status.peerCapabilities) &
        configRelevantCapabilities) != 0u) ||
        (protocolCompatible == 0u) || (algorithmsReady == 0u))
    {
        endpoint->status.configCompatible = 0u;
    }

    endpoint->status.peerCapabilities = capabilities;
    endpoint->status.protocolCompatible = protocolCompatible;
    endpoint->status.planningCompatible = planningCompatible;
    endpoint->status.algorithmsReady = algorithmsReady;
    if (planningCompatible == 0u) clearPlanningState(endpoint);
    if (kvmEligible(endpoint) == 0u) clearKvmState(endpoint);
    if (paramSyncEligible(endpoint) == 0u) clearParamPeerState(endpoint);
}

static void handleAck(endpointPrivate *endpoint,
    const observerLinkFrame *frame,
    uint32_t tickMs)
{
    observerLinkConfigAck ack;

    if ((observerLinkDecodeConfigAck(frame->payload, frame->payloadLength,
        &ack) == 0u) ||
        (ack.ownerSession != endpoint->status.localSession))
    {
        return;
    }

    endpoint->status.configAckStatus = ack.status;
    endpoint->status.configAckReason = ack.reason;
    endpoint->status.configAckDesiredMode = ack.desiredMode;
    endpoint->status.configAckRevision = ack.revision;
    endpoint->status.configAckHash = ack.hash;
    endpoint->status.appliedMode = ack.appliedMode;
    endpoint->status.configAckTickMs = tickMs;
    endpoint->status.configCompatible = (uint8_t) ((endpoint->status.configProviderValid != 0u) &&

```

```

        (endpoint->status.online != 0u) &&
        (endpoint->status.protocolCompatible != 0u) &&
        (endpoint->status.algorithmsReady != 0u) &&
        (ack.status == OBSERVER_LINK_CONFIG_APPLIED) &&
        (ack.reason == OBSERVER_LINK_REJECT_NONE) &&
        (ack.revision == endpoint->desired.revision) &&
        (ack.hash == endpoint->desired.hash) &&
        (ack.desiredMode == endpoint->desired.detectMode) &&
        (ack.appliedMode == endpoint->desired.detectMode));
    if (endpoint->status.configCompatible == 0u) clearPlanningState(endpoint);
}

static uint8_t planningFragmentMatches(endpointPrivate *endpoint,
    const observerLinkPlanningTrackFragment *fragment)
{
    const observerLinkPlanningTrackSnapshot *build = &endpoint->planningBuild;
    return (uint8_t) ((build->valid != 0u) &&
        (fragment->sourceId == build->sourceId) &&
        (fragment->detectMode == build->detectMode) &&
        (fragment->truncated == build->truncated) &&
        (fragment->ownerSession == build->ownerSession) &&
        (fragment->revision == build->revision) &&
        (fragment->hash == build->hash) &&
        (fragment->appFrameSeq == build->appFrameSeq) &&
        (fragment->captureTickMs == build->captureTickMs) &&
        (fragment->trackerFrameSeq == build->trackerFrameSeq) &&
        (fragment->confirmedTotal == build->confirmedTotal) &&
        (fragment->freshTotal == build->freshTotal) &&
        (fragment->freshLeft == build->freshLeft) &&
        (fragment->freshRight == build->freshRight) &&
        (fragment->retainedCount == build->retainedCount));
}

static void startPlanningBuild(endpointPrivate *endpoint,
    const observerLinkPlanningTrackFragment *fragment)
{
    observerLinkPlanningTrackSnapshot *build = &endpoint->planningBuild;
    memset(build, 0, sizeof(*build));
    build->valid = 1u;
    build->sourceId = fragment->sourceId;
    build->detectMode = fragment->detectMode;
    build->truncated = fragment->truncated;
    build->ownerSession = fragment->ownerSession;
    build->revision = fragment->revision;
    build->hash = fragment->hash;
    build->appFrameSeq = fragment->appFrameSeq;
    build->captureTickMs = fragment->captureTickMs;
    build->trackerFrameSeq = fragment->trackerFrameSeq;
    build->confirmedTotal = fragment->confirmedTotal;
    build->freshTotal = fragment->freshTotal;
    build->freshLeft = fragment->freshLeft;
    build->freshRight = fragment->freshRight;
    build->retainedCount = fragment->retainedCount;
    endpoint->planningBuildMask = 0u;
    endpoint->planningExpectedMask =
        (uint8_t) ((1u << fragment->fragmentCount) - 1u);
}

static uint8_t planningBuildsValid(const endpointPrivate *endpoint)
{
    const observerLinkPlanningTrackSnapshot *build = &endpoint->planningBuild;
    uint8_t ordinaryRetained = 0u;
    uint8_t i;
    for (i = 0u; i < build->retainedCount; i++)
    {
        uint8_t j;
        if ((build->tracks[i].flags &
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) == 0u)
            ordinaryRetained++;
        for (j = 0u; j < i; j++)
        {
            if (build->tracks[j].trackId == build->tracks[i].trackId)
                return 0u;
        }
    }
    return (uint8_t) ((ordinaryRetained <= endpoint->desired.maxBeacons) &&
        (ordinaryRetained <= build->confirmedTotal));
}

```

```

static void handlePlanningFragment(endpointPrivate *endpoint,
    const observerLinkFrame *frame,
    uint32_t tickMs)
{
    observerLinkPlanningTrackFragment fragment;
    uint8_t bit;
    uint8_t i;
    uint8_t j;

    if ((frame->minor != OBSERVER_LINK_PROTOCOL_MINOR) ||
        (endpoint->status.planningCompatible == 0u) ||
        (endpoint->status.configCompatible == 0u) ||
        ((endpoint->status.peerCapabilities &
            OBSERVER_LINK_CAP_PLANNING_TRACKS) == 0u) ||
        (observerLinkDecodePlanningTrackFragment(frame->payload,
            frame->payloadLength, &fragment) == 0u) ||
        (fragment.sourceId != endpoint->descriptor.sourceId) ||
        (fragment.ownerSession != endpoint->status.localSession) ||
        (fragment.revision != endpoint->desired.revision) ||
        (fragment.hash != endpoint->desired.hash) ||
        (fragment.detectMode != endpoint->desired.detectMode))
    {
        endpoint->status.planningReassemblyDrops++;
        return;
    }
    if ((endpoint->planningPublishedValid != 0u) &&
        (observerLinkSequenceIsNewer(fragment.trackerFrameSeq,
            endpoint->planningPublished.trackerFrameSeq) == 0u)) return;
    if (endpoint->planningBuild.valid == 0u)
    {
        startPlanningBuild(endpoint, &fragment);
    }
    else if (observerLinkSequenceIsNewer(fragment.trackerFrameSeq,
        endpoint->planningBuild.trackerFrameSeq) != 0u)
    {
        startPlanningBuild(endpoint, &fragment);
    }
    else if (fragment.trackerFrameSeq !=
        endpoint->planningBuild.trackerFrameSeq)
    {
        return;
    }
    else if (planningFragmentMatches(endpoint, &fragment) == 0u)
    {
        endpoint->status.planningReassemblyDrops++;
        memset(&endpoint->planningBuild, 0, sizeof(endpoint->planningBuild));
        endpoint->planningBuildMask = 0u;
        return;
    }
    bit = (uint8_t) (1u << fragment.fragmentIndex);
    if ((endpoint->planningBuildMask & bit) != 0u)
    {
        for (i = 0u; i < fragment.trackCount; i++)
        {
            if (memcmp(
                &endpoint->planningBuild.tracks[fragment.firstTrack + i],
                &fragment.tracks[i], sizeof(fragment.tracks[i])) != 0)
            {
                endpoint->status.planningReassemblyDrops++;
                memset(&endpoint->planningBuild, 0,
                    sizeof(endpoint->planningBuild));
                endpoint->planningBuildMask = 0u;
                return;
            }
        }
    }
    return;
}
for (i = 0u; i < fragment.trackCount; i++)
{
    for (j = 0u; j < endpoint->planningBuild.retainedCount; j++)
    {
        uint8_t ownerFragment = (uint8_t) (j / OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT);
        if (((endpoint->planningBuildMask &
            (uint8_t) (1u << ownerFragment)) != 0u) &&
            (endpoint->planningBuild.tracks[j].trackId ==
                fragment.tracks[i].trackId))
        {
            endpoint->status.planningReassemblyDrops++;
            memset(&endpoint->planningBuild, 0,

```

```

        sizeof(endpoint->planningBuild));
        endpoint->planningBuildMask = 0u;
        return;
    }
}
endpoint->planningBuild.tracks[fragment.firstTrack + i] =
    fragment.tracks[i];
}
endpoint->planningBuildMask |= bit;
if (endpoint->planningBuildMask == endpoint->planningExpectedMask)
{
    if (planningBuildIsValid(endpoint) == 0u)
    {
        endpoint->status.planningReassemblyDrops++;
        memset(&endpoint->planningBuild, 0,
            sizeof(endpoint->planningBuild));
        endpoint->planningBuildMask = 0u;
        return;
    }
    endpoint->planningPublished = endpoint->planningBuild;
    endpoint->planningPublishedValid = 1u;
    endpoint->planningPublishedTickMs = tickMs;
    endpoint->planningTransportAgeMs =
        frame->tickMs - fragment.captureTickMs;
    endpoint->status.planningSnapshots++;
    memset(&endpoint->planningBuild, 0,
        sizeof(endpoint->planningBuild));
    endpoint->planningBuildMask = 0u;
}
}

static void handleFrame(endpointPrivate *endpoint,
    const observerLinkFrame *frame,
    uint32_t tickMs)
{
    uint8_t qualifiedHello = (uint8_t) ((frame->type == OBSERVER_LINK_MSG_HELLO) &&
        (frame->major == OBSERVER_LINK_PROTOCOL_MAJOR) &&
        (frame->minor == OBSERVER_LINK_PROTOCOL_MINOR) &&
        (helloIdentifiesPeer(endpoint, frame) != 0u));
    uint8_t sessionDecision = observerLinkRxSessionClassify(
        endpoint->status.online, endpoint->status.peerSession,
        endpoint->rxPacketSeq, endpoint->rxPacketSeqPrimed,
        frame->bootSession, frame->packetSeq, qualifiedHello);
    if (sessionDecision == OBSERVER_LINK_RX_SESSION_REJECT) return;
    if (sessionDecision == OBSERVER_LINK_RX_SESSION_REBIND)
    {
        endpoint->status.peerSession = frame->bootSession;
        endpoint->rxPacketSeqPrimed = 0u;
        clearPeerState(endpoint);
    }
    endpoint->rxPacketSeq = frame->packetSeq;
    endpoint->rxPacketSeqPrimed = 1u;
    endpoint->status.lastValidRxTickMs = tickMs;
    endpoint->status.online = 1u;
    endpoint->status.rxFrames++;

    if ((frame->major != OBSERVER_LINK_PROTOCOL_MAJOR) ||
        (frame->minor != OBSERVER_LINK_PROTOCOL_MINOR))
    {
        clearPeerState(endpoint);
        return;
    }
    if (frame->type == OBSERVER_LINK_MSG_HELLO)
    {
        handleHello(endpoint, frame);
    }
    else if (frame->type == OBSERVER_LINK_MSG_CONFIG_ACK)
    {
        handleAck(endpoint, frame, tickMs);
    }
    else if (frame->type == OBSERVER_LINK_MSG_DIAGNOSTICS)
    {
        if (observerLinkDecodeDiagnostics(frame->payload, frame->payloadLength,
            &endpoint->status.peerDiagnostics) != 0u)
        {
            endpoint->status.peerDiagnosticsTickMs = tickMs;
        }
    }
    else if (frame->type == OBSERVER_LINK_MSG_HEALTH)

```

```

{
    observerLinkHealth health;
    if ((observerLinkDecodeHealth(frame->payload, frame->payloadLength,
        &health) != 0u) &&
        (health.sourceId == endpoint->descriptor.sourceId) &&
        (health.ownerSession == endpoint->status.localSession))
    {
        endpoint->status.health = health;
        endpoint->status.healthValid = 1u;
        endpoint->status.healthTickMs = tickMs;
    }
}
else if (frame->type == OBSERVER_LINK_MSG_KVM_ACK)
{
    observerLinkKvmAck ack;
    if ((kvmEligible(endpoint) != 0u) &&
        (endpoint->kvmControlArmed != 0u) &&
        (observerLinkDecodeKvmAck(frame->payload, frame->payloadLength,
            &ack) != 0u) &&
        (ack.target == endpoint->descriptor.kvmTarget) &&
        (ack.targetSession == endpoint->status.peerSession))
    {
        endpoint->kvmAck = ack;
        endpoint->kvmAckReady = 1u;
    }
}
else if (frame->type == OBSERVER_LINK_MSG_KVM_MENU_STATUS)
{
    observerLinkKvmMenuStatus status;
    if ((kvmEligible(endpoint) != 0u) &&
        (endpoint->kvmControlArmed != 0u) &&
        (observerLinkDecodeKvmMenuStatus(frame->payload,
            frame->payloadLength, &status) != 0u) &&
        (status.target == endpoint->descriptor.kvmTarget) &&
        (status.targetSession == endpoint->status.peerSession))
    {
        endpoint->kvmMenuStatus = status;
        endpoint->kvmMenuTickMs = tickMs;
        endpoint->kvmMenuValid = 1u;
    }
}
else if (frame->type == OBSERVER_LINK_MSG_PLANNING_TRACK_FRAGMENT)
{
    handlePlanningFragment(endpoint, frame, tickMs);
}
else if ((frame->type == OBSERVER_LINK_MSG_PARAM_RESPONSE) &&
    (paramSyncEligible(endpoint) != 0u) &&
    (frame->payloadLength <= OBSERVER_LINK_MAX_PAYLOAD_BYTES))
{
    memcpy(endpoint->paramResponse, frame->payload, frame->payloadLength);
    endpoint->paramResponseLength = frame->payloadLength;
    endpoint->paramResponseReady = 1u;
}
else if ((frame->type == OBSERVER_LINK_MSG_PARAM_GLOBAL_REQUEST) &&
    (paramSyncEligible(endpoint) != 0u) &&
    (observerParamGlobalRequestDecode(frame->payload,
        frame->payloadLength, &endpoint->paramGlobalRequest) != 0u) &&
    (endpoint->paramGlobalRequest.origin == endpoint->descriptor.sourceId))
{
    endpoint->paramGlobalRequestReady = 1u;
}
}

```

```

static void pollHardwareRxErrors(endpointPrivate *endpoint)
{
#ifdef __ICCARM__
    volatile stc_SCB_t *base = (volatile stc_SCB_t *) endpoint->descriptor.scb;
    uint32_t status;
    uint32_t errors;
    if (base == NULL) return;
    status = Cy_SCB_UART_GetRxFifoStatus(base);
    errors = status & CY_SCB_UART_RECEIVE_ERR;
    if (errors == 0u) return;
    if ((errors & CY_SCB_RX_INTR_UART_FRAME_ERROR) != 0u)
    {
        endpoint->status.localDiagnostics.rxFrameErrors++;
    }
    if ((errors & CY_SCB_RX_INTR_UART_PARITY_ERROR) != 0u)
    {

```

```

        endpoint->status.localDiagnostics.rxParityErrors++;
    }
    Cy_SCB_UART_ClearRxFifoStatus(base, errors);
#else
    (void) endpoint;
#endif
}

static void receiveFrames(endpointPrivate *endpoint, uint32_t tickMs)
{
    observerLinkFrame frame;
    uint16_t budget = OBSERVER_LINK_RX_BYTE_BUDGET;

    pollHardwareRxErrors(endpoint);
    while ((endpoint->rxTail != endpoint->rxHead) && (budget != 0u))
    {
        uint8_t byte = endpoint->rxRing[endpoint->rxTail];
        endpoint->rxTail =
            (uint16_t)((endpoint->rxTail + 1u) & ENDPOINT_RX_RING_MASK);
        budget--;
        if (observerLinkParserFeed(&endpoint->parser, byte, &frame) != 0u)
        {
            handleFrame(endpoint, &frame, tickMs);
        }
    }
}

static uint8_t sendHello(endpointPrivate *endpoint, uint32_t tickMs)
{
    uint8_t payload[8u];
    uint16_t length;
    uint32_t capabilities = OBSERVER_LINK_CAP_BINARY_CC |
        OBSERVER_LINK_CAP_LEGACY_CANNY |
        OBSERVER_LINK_CAP_CONFIG_STAGING |
        OBSERVER_LINK_CAP_CONFIG_APPLY |
        OBSERVER_LINK_CAP_KVM |
        OBSERVER_LINK_CAP_MENU_STATUS |
        OBSERVER_LINK_CAP_PLANNING_TRACKS |
        OBSERVER_LINK_CAP_CAMERA_CONTROL |
        OBSERVER_LINK_CAP_CONTROL_STATUS |
        OBSERVER_LINK_CAP_VIEW_OVERLAY |
        OBSERVER_LINK_CAP_PARAM_SYNC;
    if (observerLinkEncodeHello(OBSERVER_LINK_ROLE_CORE1,
        capabilities, payload, &length) == 0u) return 0u;
    return sendMessage(endpoint, OBSERVER_LINK_MSG_HELLO,
        payload, length, tickMs);
}

static void makeFailClosedControlStatus(observerLinkControlStatus *status)
{
    memset(status, 0, sizeof(*status));
    status->cameraControlState = OBSERVER_LINK_CAMERA_CONTROL_RUN;
    status->flags = OBSERVER_LINK_CONTROL_FLAG_STATUS_VALID |
        OBSERVER_LINK_CONTROL_FLAG_REQUIRED_VISION;
    status->desiredTarget = OBSERVER_LINK_KVM_TARGET_NONE;
    status->ownerTarget = OBSERVER_LINK_KVM_TARGET_NONE;
}

static uint8_t sendHeartbeat(endpointPrivate *endpoint, uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_CORE1_HEARTBEAT_BYTES];
    uint16_t length;
    observerLinkControlStatus status = endpoint->controlStatus;

    status.detectMode = endpoint->desired.detectMode;
    status.protocolCompatible = endpoint->status.protocolCompatible;
    status.algorithmsReady = endpoint->status.algorithmsReady;
    status.configCompatible = endpoint->status.configCompatible;
    if (observerLinkEncodeControlStatus(&status, payload, &length) == 0u)
    {
        makeFailClosedControlStatus(&status);
        status.detectMode = endpoint->desired.detectMode;
        status.protocolCompatible = endpoint->status.protocolCompatible;
        status.algorithmsReady = endpoint->status.algorithmsReady;
        status.configCompatible = endpoint->status.configCompatible;
        if (observerLinkEncodeControlStatus(&status, payload, &length) == 0u)
        {
            endpoint->status.txFailures++;
            return 0u;
        }
    }
}

```

```

    }
}
endpoint->status.controlStatus = status;
endpoint->status.cameraControlState = status.cameraControlState;
return sendMessage(endpoint, OBSERVER_LINK_MSG_HEARTBEAT,
    payload, length, tickMs);
}

static uint8_t sendConfig(endpointPrivate *endpoint, uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_SHARED_CONFIG_BYTES];
    uint16_t length;
    if ((endpoint->status.configProviderValid == 0u) ||
        (observerLinkEncodeSharedConfig(&endpoint->desired,
            payload, &length) == 0u)) return 0u;
    return sendMessage(endpoint, OBSERVER_LINK_MSG_CONFIG_SNAPSHOT,
        payload, length, tickMs);
}

static uint8_t sendDiagnostics(endpointPrivate *endpoint, uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_DIAGNOSTICS_BYTES];
    uint16_t length;
    observerLinkDiagnostics diagnostics;
    endpoint->status.localDiagnostics.validFrames = endpoint->parser.validFrames;
    endpoint->status.localDiagnostics.crcErrors = endpoint->parser.crcErrors;
    endpoint->status.localDiagnostics.lengthErrors = endpoint->parser.lengthErrors;
    endpoint->status.localDiagnostics.versionErrors = endpoint->parser.versionErrors;
    endpoint->status.localDiagnostics.txDeferred = endpoint->status.txDeferred;
    diagnostics = endpoint->status.localDiagnostics;
    diagnostics.rxOverflowErrors += endpoint->rxRingOverflow;
    if (observerLinkEncodeDiagnostics(&diagnostics, payload, &length) == 0u)
    {
        return 0u;
    }
    return sendMessage(endpoint, OBSERVER_LINK_MSG_DIAGNOSTICS,
        payload, length, tickMs);
}

static uint8_t sendViewOverlay(endpointPrivate *endpoint, uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_VIEW_OVERLAY_BYTES];
    uint16_t length;
    if (observerLinkEncodeViewOverlay(&endpoint->viewOverlay,
        payload, &length) == 0u) return 0u;
    return sendMessage(endpoint, OBSERVER_LINK_MSG_VIEW_OVERLAY,
        payload, length, tickMs);
}

static uint8_t sendKvmControl(endpointPrivate *endpoint, uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_KVM_CONTROL_BYTES];
    uint16_t length;
    if (observerLinkEncodeKvmControl(&endpoint->kvmControl,
        payload, &length) == 0u) return 0u;
    return sendControlMessage(endpoint, OBSERVER_LINK_MSG_KVM_CONTROL,
        payload, length, tickMs);
}

static uint8_t sendKvmEvent(endpointPrivate *endpoint, uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_KVM_EVENT_BYTES];
    uint16_t length;
    if (observerLinkEncodeKvmEvent(&endpoint->kvmEvent,
        payload, &length) == 0u) return 0u;
    return sendControlMessage(endpoint, OBSERVER_LINK_MSG_KVM_EVENT,
        payload, length, tickMs);
}

static void refreshTxEpoch(endpointPrivate *endpoint, uint32_t tickMs)
{
    uint32_t elapsedEpochs =
        (uint32_t) (tickMs - endpoint->txEpochTickMs) /
        OBSERVER_LINK_TX_EPOCH_MS;
    uint32_t credits;
    if (elapsedEpochs == 0u) return;
    endpoint->txEpochTickMs += elapsedEpochs * OBSERVER_LINK_TX_EPOCH_MS;
    credits = (uint32_t) endpoint->txDataCredits +
        elapsedEpochs * OBSERVER_LINK_TX_FRAMES_PER_EPOCH;
}

```

```

    endpoint->txDataCredits = (uint8_t) ((credits > OBSERVER_LINK_TX_DATA_CREDIT_CAP) ? OBSERVER_LINK_TX_DATA_CREDIT_CAP : credits);
    credits = (uint32_t) endpoint->txControlCredits +
        elapsedEpochs * OBSERVER_LINK_TX_CONTROL_FRAMES_PER_EPOCH;
    endpoint->txControlCredits = (uint8_t) ((credits > OBSERVER_LINK_TX_CONTROL_CREDIT_CAP) ? OBSERVER_LINK_TX_CONTROL_CREDIT_CAP
: credits);
}

static void serviceControlTransmit(endpointPrivate *endpoint, uint32_t tickMs)
{
    if (kvmEligible(endpoint) == 0u)
    {
        clearKvmState(endpoint);
        return;
    }
    refreshTxEpoch(endpoint, tickMs);
    while (endpoint->txControlCredits != 0u)
    {
        if ((endpoint->kvmControlPending != 0u) &&
            (sendKvmControl(endpoint, tickMs) != 0u))
        {
            endpoint->kvmControlPending = 0u;
            continue;
        }
        if ((endpoint->kvmEventPending != 0u) &&
            (sendKvmEvent(endpoint, tickMs) != 0u))
        {
            endpoint->kvmEventPending = 0u;
            continue;
        }
        break;
    }
}

static uint8_t transmitOne(endpointPrivate *endpoint,
    uint32_t tickMs,
    uint32_t configPeriod)
{
    if (((endpoint->status.online == 0u) ||
        (endpoint->status.protocolCompatible == 0u)) &&
        ((uint32_t) (tickMs - endpoint->lastHelloTickMs) >=
            ENDPOINT_HELLO_PERIOD_MS) &&
        (sendHello(endpoint, tickMs) != 0u))
    {
        endpoint->lastHelloTickMs = tickMs;
        return 1u;
    }
    if (((uint32_t) (tickMs - endpoint->lastHeartbeatTickMs) >=
        ENDPOINT_HEARTBEAT_PERIOD_MS) &&
        (sendHeartbeat(endpoint, tickMs) != 0u))
    {
        endpoint->lastHeartbeatTickMs = tickMs;
        return 1u;
    }
    if ((endpoint->status.protocolCompatible != 0u) &&
        (endpoint->status.configProviderValid != 0u) &&
        ((uint32_t) (tickMs - endpoint->lastConfigTickMs) >= configPeriod) &&
        (sendConfig(endpoint, tickMs) != 0u))
    {
        endpoint->lastConfigTickMs = tickMs;
        return 1u;
    }
    if (((uint32_t) (tickMs - endpoint->lastHelloTickMs) >=
        ENDPOINT_HELLO_PERIOD_MS) &&
        (sendHello(endpoint, tickMs) != 0u))
    {
        endpoint->lastHelloTickMs = tickMs;
        return 1u;
    }
    if (((uint32_t) (tickMs - endpoint->lastDiagTickMs) >=
        ENDPOINT_DIAG_PERIOD_MS) &&
        (sendDiagnostics(endpoint, tickMs) != 0u))
    {
        endpoint->lastDiagTickMs = tickMs;
        return 1u;
    }
    if ((endpoint->viewOverlayPending != 0u) &&
        ((endpoint->status.peerCapabilities &
            OBSERVER_LINK_CAP_VIEW_OVERLAY) != 0u) &&
        (sendViewOverlay(endpoint, tickMs) != 0u))

```



```

{
    endpoint->viewOverlayPending = 0u;
    return 1u;
}
/* 参数同步严格位于心跳、配置、诊断和业务数据之后。 */
if ((endpoint->paramRequestPending != 0u) &&
    (paramSyncEligible(endpoint) != 0u) &&
    (sendMessage(endpoint, OBSERVER_LINK_MSG_PARAM_REQUEST,
        endpoint->paramRequest, endpoint->paramRequestLength, tickMs) != 0u))
{
    endpoint->paramRequestPending = 0u;
    return 1u;
}
if ((endpoint->paramGlobalStatePending != 0u) &&
    (paramSyncEligible(endpoint) != 0u))
{
    uint8_t payload[OBSERVER_PARAM_GLOBAL_STATE_BYTES];
    uint16_t length;
    if ((observerParamGlobalStateEncode(&endpoint->paramGlobalState,
        payload, &length) != 0u) &&
        (sendMessage(endpoint, OBSERVER_LINK_MSG_PARAM_GLOBAL_STATE,
            payload, length, tickMs) != 0u))
    {
        endpoint->paramGlobalStatePending = 0u;
        return 1u;
    }
}
return 0u;
}

static uint8_t transmitWorkPending(endpointPrivate *endpoint, uint32_t tickMs)
{
    uint32_t configPeriod = (endpoint->status.configCompatible != 0u) ? ENDPOINT_CONFIG_REFRESH_MS : ENDPOINT_CONFIG_PERIOD_MS;
    return (uint8_t) ((endpoint->viewOverlayPending != 0u) ||
        (endpoint->paramRequestPending != 0u) ||
        (endpoint->paramGlobalStatePending != 0u) ||
        ((uint32_t) (tickMs - endpoint->lastHeartbeatTickMs) >=
            ENDPOINT_HEARTBEAT_PERIOD_MS) ||
        ((uint32_t) (tickMs - endpoint->lastHelloTickMs) >=
            ENDPOINT_HELLO_PERIOD_MS) ||
        (endpoint->status.protocolCompatible != 0u) &&
        (endpoint->status.configProviderValid != 0u) &&
        ((uint32_t) (tickMs - endpoint->lastConfigTickMs) >= configPeriod)) ||
        ((uint32_t) (tickMs - endpoint->lastDiagTickMs) >=
            ENDPOINT_DIAG_PERIOD_MS));
}

uint8_t observerLinkCore1EndpointQueueParamRequest(
    observerLinkCore1EndpointContext *context, const uint8_t *payload,
    uint16_t length)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    if ((endpoint == NULL) || (payload == NULL) || (length == 0u) ||
        (length > OBSERVER_LINK_MAX_PAYLOAD_BYTES) || (endpoint->paramRequestPending != 0u) ||
        (paramSyncEligible(endpoint) == 0u)) return 0u;
    memcpy(endpoint->paramRequest, payload, length);
    endpoint->paramRequestLength = length;
    endpoint->paramRequestPending = 1u;
    return 1u;
}

uint8_t observerLinkCore1EndpointQueueParamGlobalState(
    observerLinkCore1EndpointContext *context,
    const paramSyncGlobalState *state)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    if ((endpoint == NULL) || (state == NULL) ||
        (paramSyncGlobalStateIsValid(state) == 0u) ||
        (paramSyncEligible(endpoint) == 0u)) return 0u;
    endpoint->paramGlobalState = *state;
    endpoint->paramGlobalStatePending = 1u;
    return 1u;
}

uint8_t observerLinkCore1EndpointTakeParamGlobalRequest(
    observerLinkCore1EndpointContext *context,
    paramSyncGlobalRequest *request)
{
    endpointPrivate *endpoint = initializedPrivate(context);

```

```

if ((endpoint == NULL) || (request == NULL) ||
    (endpoint->paramGlobalRequestReady == 0u)) return 0u;
if ((paramSyncEligible(endpoint) == 0u) ||
    (paramSyncGlobalRequestIsValid(&endpoint->paramGlobalRequest) == 0u))
{
    memset(&endpoint->paramGlobalRequest, 0,
        sizeof(endpoint->paramGlobalRequest));
    endpoint->paramGlobalRequestReady = 0u;
    return 0u;
}
*request = endpoint->paramGlobalRequest;
endpoint->paramGlobalRequestReady = 0u;
return 1u;
}

uint8_t observerLinkCore1EndpointTakeParamResponse(
    observerLinkCore1EndpointContext *context, uint8_t *payload,
    uint16_t capacity, uint16_t *length)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    if ((endpoint == NULL) || (payload == NULL) || (length == NULL) ||
        (endpoint->paramResponseReady == 0u) ||
        (paramSyncEligible(endpoint) == 0u) ||
        (capacity < endpoint->paramResponseLength))
    {
        if (endpoint != NULL && paramSyncEligible(endpoint) == 0u)
        {
            memset(endpoint->paramResponse, 0, sizeof(endpoint->paramResponse));
            endpoint->paramResponseLength = 0u;
            endpoint->paramResponseReady = 0u;
        }
        return 0u;
    }
    memcpy(payload, endpoint->paramResponse, endpoint->paramResponseLength);
    *length = endpoint->paramResponseLength;
    endpoint->paramResponseReady = 0u;
    return 1u;
}

static void serviceTransmit(endpointPrivate *endpoint, uint32_t tickMs)
{
    uint32_t configPeriod = (endpoint->status.configCompatible != 0u) ? ENDPOINT_CONFIG_REFRESH_MS : ENDPOINT_CONFIG_PERIOD_MS;
    refreshTxEpoch(endpoint, tickMs);
    while ((endpoint->txDataCredits != 0u) &&
        (transmitOne(endpoint, tickMs, configPeriod) != 0u))
    {
    }
    if ((transmitWorkPending(endpoint, tickMs) != 0u) &&
        (endpoint->txDataCredits == 0u) &&
        (endpoint->lastDeferredTickMs != tickMs))
    {
        endpoint->lastDeferredTickMs = tickMs;
        endpoint->status.txDeferred++;
    }
}

uint8_t observerLinkCore1EndpointInit(observerLinkCore1EndpointContext *context,
    const observerLinkCore1EndpointDescriptor *descriptor,
    uint32_t tickMs)
{
    endpointPrivate *endpoint;
    if ((context == NULL) || (descriptorIsValid(descriptor) == 0u)) return 0u;
    memset(context, 0, sizeof(*context));
    endpoint = privateFromContext(context);
    endpoint->magic = ENDPOINT_MAGIC;
    endpoint->descriptor = *descriptor;
    endpoint->status.sourceId = descriptor->sourceId;
    endpoint->status.localSession = makeBootSession(tickMs);
    endpoint->status.appliedMode = 0xFFu;
    endpoint->status.configAckDesiredMode = 0xFFu;
    endpoint->status.cameraControlState = OBSERVER_LINK_CAMERA_CONTROL_RUN;
    observerLinkParserInit(&endpoint->parser);
    endpoint->lastHelloTickMs = tickMs - ENDPOINT_HELLO_PERIOD_MS;
    endpoint->lastHeartbeatTickMs = tickMs - ENDPOINT_HEARTBEAT_PERIOD_MS;
    endpoint->lastConfigTickMs = tickMs - ENDPOINT_CONFIG_PERIOD_MS;
    endpoint->lastDiagTickMs = tickMs - ENDPOINT_DIAG_PERIOD_MS;
    endpoint->lastDeferredTickMs = tickMs - 1u;
    endpoint->txEpochTickMs = tickMs;
    endpoint->txDataCredits = OBSERVER_LINK_TX_FRAMES_PER_EPOCH;

```

```

    endpoint->txControlCredits = OBSERVER_LINK_TX_CONTROL_FRAMES_PER_EPOCH;
    makeFailClosedControlStatus(&endpoint->controlStatus);
    endpoint->status.controlStatus = endpoint->controlStatus;
    (void) syncConfig(endpoint);
    uart_init(ENDPOINT_UART(descriptor->uart), OBSERVER_LINK_UART_BAUDRATE,
        ENDPOINT_TX_PIN(descriptor->txPin), ENDPOINT_RX_PIN(descriptor->rxPin));
    uart_rx_interrupt(ENDPOINT_UART(descriptor->uart), 1u);
    return 1u;
}

void observerLinkCore1EndpointRxIsr(observerLinkCore1EndpointContext *context)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    uint8_t byte;
    if (endpoint == NULL) return;
    while (uart_query_byte(ENDPOINT_UART(endpoint->descriptor.uart),
        &byte) != 0u)
    {
        uint16_t next =
            (uint16_t)((endpoint->rxHead + 1u) & ENDPOINT_RX_RING_MASK);
        if (next == endpoint->rxTail)
        {
            endpoint->rxRingOverflow++;
            continue;
        }
        endpoint->rxRing[endpoint->rxHead] = byte;
        publishBarrier();
        endpoint->rxHead = next;
    }
}

void observerLinkCore1EndpointPoll(observerLinkCore1EndpointContext *context,
    uint32_t tickMs)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    if (endpoint == NULL) return;
    (void) syncConfig(endpoint);
    receiveFrames(endpoint, tickMs);
    if ((endpoint->status.online != 0u) &&
        ((uint32_t)(tickMs - endpoint->status.lastValidRxTickMs) >=
            ENDPOINT_TIMEOUT_MS))
    {
        endpoint->status.online = 0u;
        endpoint->rxPacketSeqPrimed = 0u;
        clearPeerState(endpoint);
    }
    if ((endpoint->viewOverlayPending != 0u) &&
        (((uint32_t)(tickMs - endpoint->viewOverlayTickMs) >
            ENDPOINT_OVERLAY_MAX_AGE_MS) ||
            ((endpoint->status.online != 0u) &&
                (endpoint->status.protocolCompatible != 0u) &&
                ((endpoint->status.peerCapabilities &
                    OBSERVER_LINK_CAP_VIEW_OVERLAY) == 0u))))
    {
        endpoint->viewOverlayPending = 0u;
    }
    serviceControlTransmit(endpoint, tickMs);
    serviceTransmit(endpoint, tickMs);
}

void observerLinkCore1EndpointDrainRx(observerLinkCore1EndpointContext *context,
    uint32_t tickMs)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    if (endpoint != NULL) receiveFrames(endpoint, tickMs);
}

void observerLinkCore1EndpointFlushControl(
    observerLinkCore1EndpointContext *context,
    uint32_t tickMs)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    if (endpoint != NULL) serviceControlTransmit(endpoint, tickMs);
}

void observerLinkCore1EndpointFlushTx(observerLinkCore1EndpointContext *context,
    uint32_t tickMs)
{
    endpointPrivate *endpoint = initializedPrivate(context);

```

```

    if (endpoint != NULL) serviceTransmit(endpoint, tickMs);
}

void observerLinkCore1EndpointGetStatus(
    observerLinkCore1EndpointContext *context,
    observerLinkCore1EndpointStatus *status)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    if (status == NULL) return;
    memset(status, 0, sizeof(*status));
    if (endpoint == NULL) return;
    endpoint->status.localDiagnostics.validFrames = endpoint->parser.validFrames;
    endpoint->status.localDiagnostics.crcErrors = endpoint->parser.crcErrors;
    endpoint->status.localDiagnostics.lengthErrors = endpoint->parser.lengthErrors;
    endpoint->status.localDiagnostics.versionErrors = endpoint->parser.versionErrors;
    endpoint->status.localDiagnostics.txDeferred = endpoint->status.txDeferred;
    *status = endpoint->status;
    status->localDiagnostics.rxOverflowErrors += endpoint->rxRingOverflow;
}

void observerLinkCore1EndpointFencePlanningFacts(
    observerLinkCore1EndpointContext *context)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    if (endpoint == NULL) return;
    clearPlanningState(endpoint);
    memset(&endpoint->viewOverlay, 0, sizeof(endpoint->viewOverlay));
    endpoint->viewOverlayTickMs = 0u;
    endpoint->viewOverlayPending = 0u;
}

uint8_t observerLinkCore1EndpointSyncConfig(
    observerLinkCore1EndpointContext *context)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    return (endpoint != NULL) ? syncConfig(endpoint) : 0u;
}

void observerLinkCore1EndpointPublishControlStatus(
    observerLinkCore1EndpointContext *context,
    const observerLinkControlStatus *status,
    uint32_t tickMs)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    uint8_t payload[OBSERVER_LINK_CORE1_HEARTBEAT_BYTES];
    uint16_t length;
    if ((endpoint == NULL) || (status == NULL) ||
        (observerLinkEncodeControlStatus(status, payload, &length) == 0u))
    {
        if (endpoint != NULL)
        {
            makeFailClosedControlStatus(&endpoint->controlStatus);
            endpoint->status.controlStatus = endpoint->controlStatus;
            endpoint->status.cameraControlState =
                OBSERVER_LINK_CAMERA_CONTROL_RUN;
            endpoint->status.controlStatusTickMs = tickMs;
        }
        return;
    }
    endpoint->controlStatus = *status;
    endpoint->status.controlStatus = *status;
    endpoint->status.cameraControlState = status->cameraControlState;
    endpoint->status.controlStatusTickMs = tickMs;
}

uint8_t observerLinkCore1EndpointQueueViewOverlay(
    observerLinkCore1EndpointContext *context,
    const observerLinkViewOverlay *overlay,
    uint32_t tickMs)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    uint8_t payload[OBSERVER_LINK_VIEW_OVERLAY_BYTES];
    uint16_t length;
    if ((endpoint == NULL) || (overlay == NULL) ||
        (observerLinkEncodeViewOverlay(overlay, payload, &length) == 0u))
    {
        return 0u;
    }
    endpoint->viewOverlay = *overlay;
}

```

```

    endpoint->viewOverlayTickMs = tickMs;
    endpoint->viewOverlayPending = 1u;
    return 1u;
}

uint8_t observerLinkCore1EndpointQueueKvmControl(
    observerLinkCore1EndpointContext *context,
    const observerLinkKvmControl *control)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    if ((endpoint == NULL) || (control == NULL) ||
        (kvmEligible(endpoint) == 0u) ||
        (control->target != endpoint->descriptor.kvmTarget) ||
        (control->targetSession != endpoint->status.peerSession)) return 0u;
    endpoint->kvmControl = *control;
    endpoint->kvmControlArmed = 1u;
    endpoint->kvmControlPending = 1u;
    return 1u;
}

uint8_t observerLinkCore1EndpointQueueKvmEvent(
    observerLinkCore1EndpointContext *context,
    const observerLinkKvmEvent *event)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    if ((endpoint == NULL) || (event == NULL) ||
        (kvmEligible(endpoint) == 0u) ||
        (endpoint->kvmControlArmed == 0u) ||
        (event->target != endpoint->descriptor.kvmTarget) ||
        (event->targetSession != endpoint->status.peerSession)) return 0u;
    endpoint->kvmEvent = *event;
    endpoint->kvmEventPending = 1u;
    return 1u;
}

uint8_t observerLinkCore1EndpointTakeKvmAck(
    observerLinkCore1EndpointContext *context,
    observerLinkKvmAck *ack)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    if ((endpoint != NULL) && (kvmEligible(endpoint) == 0u))
    {
        {
            clearKvmState(endpoint);
            return 0u;
        }
        if ((endpoint == NULL) || (ack == NULL) ||
            (endpoint->kvmControlArmed == 0u) ||
            (endpoint->kvmAckReady == 0u) ||
            (endpoint->kvmAck.targetSession != endpoint->status.peerSession))
        {
            return 0u;
        }
        *ack = endpoint->kvmAck;
        endpoint->kvmAckReady = 0u;
        return 1u;
    }
}

uint8_t observerLinkCore1EndpointGetKvmMenuStatus(
    observerLinkCore1EndpointContext *context,
    observerLinkKvmMenuStatus *status,
    uint32_t tickMs,
    uint32_t *ageMs)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    if ((endpoint != NULL) && (kvmEligible(endpoint) == 0u))
    {
        {
            clearKvmState(endpoint);
            return 0u;
        }
        if ((endpoint == NULL) || (status == NULL) ||
            (endpoint->kvmControlArmed == 0u) ||
            (endpoint->kvmMenuValid == 0u) ||
            (endpoint->kvmMenuStatus.targetSession !=
             endpoint->status.peerSession)) return 0u;
        *status = endpoint->kvmMenuStatus;
        if (ageMs != NULL) *ageMs = tickMs - endpoint->kvmMenuTickMs;
        return 1u;
    }
}

```

```

uint8_t observerLinkCore1EndpointGetPlanningTrackSnapshot(
    observerLinkCore1EndpointContext *context,
    observerLinkPlanningTrackSnapshot *snapshot,
    uint32_t tickMs,
    uint32_t timeoutMs,
    uint32_t *ageMs)
{
    endpointPrivate *endpoint = initializedPrivate(context);
    uint32_t age;
    if ((endpoint == NULL) || (snapshot == NULL) || (timeoutMs == 0u) ||
        (endpoint->planningPublishedValid == 0u) ||
        (endpoint->status.online == 0u) ||
        (endpoint->status.protocolCompatible == 0u) ||
        (endpoint->status.planningCompatible == 0u) ||
        (endpoint->status.configCompatible == 0u) ||
        ((endpoint->status.peerCapabilities &
            OBSERVER_LINK_CAP_PLANNING_TRACKS) == 0u) ||
        (endpoint->planningPublished.sourceId !=
            endpoint->descriptor.sourceId) ||
        (endpoint->planningPublished.ownerSession !=
            endpoint->status.localSession) ||
        (endpoint->planningPublished.revision != endpoint->desired.revision) ||
        (endpoint->planningPublished.hash != endpoint->desired.hash) ||
        (endpoint->planningPublished.detectMode !=
            endpoint->desired.detectMode)) return 0u;
    age = endpoint->planningTransportAgeMs +
        (tickMs - endpoint->planningPublishedTickMs);
    *snapshot = endpoint->planningPublished;
    if (ageMs != NULL) *ageMs = age;
    return (uint8_t) (age < timeoutMs);
}

```

core1_cyt4bb7/project/code/comm/observerLinkCore1Endpoint.h

SHA-256

:

d91ba65d9df0440d864f465c76489a00f54c4b9df8d49cf15b2416bb7c214002

代码 (c)

```

#ifndef OBSERVER_LINK_CORE1_ENDPOINT_H
#define OBSERVER_LINK_CORE1_ENDPOINT_H

#include <stdint.h>

#include "observerLinkProtocol.h"

#define OBSERVER_LINK_CORE1_ENDPOINT_CONTEXT_BYTES (8192u)

typedef uint8_t (*observerLinkCore1ConfigProvider)(void *providerContext,
    uint8_t sourceId,
    observerLinkSharedConfig *config);

typedef struct
{
    uint32_t uart;
    uint32_t rxPin;
    uint32_t txPin;
    volatile void *scb;
    uint8_t expectedRole;
    uint8_t sourceId;
    uint8_t kvmTarget;
    uint8_t padding;
    observerLinkCore1ConfigProvider configProvider;
    void *configProviderContext;
} observerLinkCore1EndpointDescriptor;

typedef struct
{
    uint8_t sourceId;
    uint8_t online;
    uint8_t protocolCompatible;
    uint8_t planningCompatible;
    uint8_t algorithmsReady;
    uint8_t configProviderValid;
    uint8_t configCompatible;
}

```

```

uint8_t configAckStatus;
uint8_t configAckReason;
uint8_t configAckDesiredMode;
uint8_t desiredMode;
uint8_t appliedMode;
uint8_t cameraControlState;
uint8_t healthValid;
uint16_t padding;
uint32_t localSession;
uint32_t peerSession;
uint32_t peerCapabilities;
uint32_t desiredRevision;
uint32_t desiredHash;
uint32_t configAckRevision;
uint32_t configAckHash;
uint32_t lastValidRxTickMs;
uint32_t rxFrames;
uint32_t txFrames;
uint32_t txFailures;
uint32_t txDeferred;
observerLinkDiagnostics localDiagnostics;
observerLinkDiagnostics peerDiagnostics;
uint32_t peerDiagnosticsTickMs;
uint32_t configAckTickMs;
uint32_t planningSnapshots;
uint32_t planningReassemblyDrops;
observerLinkControlStatus controlStatus;
uint32_t controlStatusTickMs;
observerLinkHealth health;
uint32_t healthTickMs;
} observerLinkCore1EndpointStatus;

typedef union
{
    uint64_t align64;
    void *alignPointer;
    uint8_t bytes[OBSERVER_LINK_CORE1_ENDPOINT_CONTEXT_BYTES];
} observerLinkCore1EndpointContext;

uint8_t observerLinkCore1EndpointInit(observerLinkCore1EndpointContext *context,
    const observerLinkCore1EndpointDescriptor *descriptor,
    uint32_t tickMs);
void observerLinkCore1EndpointPoll(observerLinkCore1EndpointContext *context,
    uint32_t tickMs);
void observerLinkCore1EndpointDrainRx(observerLinkCore1EndpointContext *context,
    uint32_t tickMs);
void observerLinkCore1EndpointFlushControl(observerLinkCore1EndpointContext *context,
    uint32_t tickMs);
void observerLinkCore1EndpointFlushTx(observerLinkCore1EndpointContext *context,
    uint32_t tickMs);
void observerLinkCore1EndpointRxIsr(observerLinkCore1EndpointContext *context);
void observerLinkCore1EndpointGetStatus(
    observerLinkCore1EndpointContext *context,
    observerLinkCore1EndpointStatus *status);
void observerLinkCore1EndpointFencePlanningFacts(
    observerLinkCore1EndpointContext *context);
uint8_t observerLinkCore1EndpointSyncConfig(
    observerLinkCore1EndpointContext *context);
void observerLinkCore1EndpointPublishControlStatus(
    observerLinkCore1EndpointContext *context,
    const observerLinkControlStatus *status,
    uint32_t tickMs);
uint8_t observerLinkCore1EndpointQueueViewOverlay(
    observerLinkCore1EndpointContext *context,
    const observerLinkViewOverlay *overlay,
    uint32_t tickMs);
uint8_t observerLinkCore1EndpointQueueKvmControl(
    observerLinkCore1EndpointContext *context,
    const observerLinkKvmControl *control);
uint8_t observerLinkCore1EndpointQueueKvmEvent(
    observerLinkCore1EndpointContext *context,
    const observerLinkKvmEvent *event);
uint8_t observerLinkCore1EndpointTakeKvmAck(
    observerLinkCore1EndpointContext *context,
    observerLinkKvmAck *ack);
uint8_t observerLinkCore1EndpointGetKvmMenuStatus(
    observerLinkCore1EndpointContext *context,
    observerLinkKvmMenuStatus *status,
    uint32_t tickMs,

```

```

    uint32_t *ageMs);
uint8_t observerLinkCore1EndpointGetPlanningTrackSnapshot(
    observerLinkCore1EndpointContext *context,
    observerLinkPlanningTrackSnapshot *snapshot,
    uint32_t tickMs,
    uint32_t timeoutMs,
    uint32_t *ageMs);
uint8_t observerLinkCore1EndpointQueueParamRequest(
    observerLinkCore1EndpointContext *context, const uint8_t *payload,
    uint16_t length);
uint8_t observerLinkCore1EndpointTakeParamResponse(
    observerLinkCore1EndpointContext *context, uint8_t *payload,
    uint16_t capacity, uint16_t *length);
uint8_t observerLinkCore1EndpointQueueParamGlobalState(
    observerLinkCore1EndpointContext *context,
    const paramSyncGlobalState *state);
uint8_t observerLinkCore1EndpointTakeParamGlobalRequest(
    observerLinkCore1EndpointContext *context,
    paramSyncGlobalRequest *request);
#endif /* OBSERVER_LINK_CORE1_ENDPOINT_H */

```

core1_cyt4bb7/project/code/comm/observerLinkProtocol.c

SHA-256

:

e019e39df722dc745661f3a59859e5cbeb121fd354d13928214448bcab16b271

代码 (c)

```

#include "observerLinkProtocol.h"

#include <string.h>

#include "crc32leee.h"

static void putU16(uint8_t *dst, uint16_t value)
{
    dst[0] = (uint8_t) value;
    dst[1] = (uint8_t) (value >> 8);
}

static void putU32(uint8_t *dst, uint32_t value)
{
    dst[0] = (uint8_t) value;
    dst[1] = (uint8_t) (value >> 8);
    dst[2] = (uint8_t) (value >> 16);
    dst[3] = (uint8_t) (value >> 24);
}

static uint16_t getU16(const uint8_t *src)
{
    return (uint16_t) ((uint16_t) src[0] | ((uint16_t) src[1] << 8));
}

static uint32_t getU32(const uint8_t *src)
{
    return (uint32_t) src[0] |
        ((uint32_t) src[1] << 8) |
        ((uint32_t) src[2] << 16) |
        ((uint32_t) src[3] << 24);
}

static void putFloat(uint8_t *dst, float value)
{
    uint32_t bits;
    memcpy(&bits, &value, sizeof(bits));
    putU32(dst, bits);
}

static float getFloat(const uint8_t *src)
{
    uint32_t bits = getU32(src);
    float value;
    memcpy(&value, &bits, sizeof(value));
    return value;
}

```



```

static uint8_t finiteInRange(float value, float minimum, float maximum)
{
    if ((value != value) || (value > 1.0e30f) || (value < -1.0e30f))
    {
        return 0u;
    }
    return (uint8_t) ((value >= minimum) && (value <= maximum));
}

uint8_t observerLinkQuantizeSigned(float value, float scale,
float minimum, float maximum, int16_t *quantized)
{
    float scaled;

    if ((quantized == NULL) ||
        (finiteInRange(value, minimum, maximum) == 0u) ||
        (finiteInRange(scale, 0.000001f, 1.0e30f) == 0u))
    {
        return 0u;
    }
    scaled = value * scale;
    if (finiteInRange(scaled, -32768.0f, 32767.0f) == 0u)
    {
        return 0u;
    }
    scaled += (scaled >= 0.0f) ? 0.5f : -0.5f;
    *quantized = (int16_t) scaled;
    return 1u;
}

static uint8_t updateExpected(observerLinkParser *parser)
{
    uint16_t payloadLength;

    parser->expected = 0u;
    if (parser->used < 8u)
    {
        return 1u;
    }
    payloadLength = getU16(&parser->bytes[6]);
    if (payloadLength > OBSERVER_LINK_MAX_PAYLOAD_BYTES)
    {
        return 0u;
    }
    parser->expected = (uint16_t) (OBSERVER_LINK_HEADER_BYTES +
        payloadLength +
        OBSERVER_LINK_CRC_BYTES);
    return 1u;
}

static void recoverParser(observerLinkParser *parser)
{
    uint16_t i;
    uint8_t found;

    for (;;)
    {
        found = 0u;
        for (i = 1u; (uint16_t) (i + 1u) < parser->used; i++)
        {
            if ((parser->bytes[i] == OBSERVER_LINK_SOF0) &&
                (parser->bytes[i + 1u] == OBSERVER_LINK_SOF1))
            {
                memmove(parser->bytes, &parser->bytes[i], parser->used - i);
                parser->used = (uint16_t) (parser->used - i);
                if (updateExpected(parser) != 0u)
                {
                    return;
                }
                parser->lengthErrors++;
                found = 1u;
                break;
            }
        }
        if (found != 0u)
        {
            continue;
        }
    }
}

```

```

    if ((parser->used != 0u) &&
        (parser->bytes[parser->used - 1u] == OBSERVER_LINK_SOF0))
    {
        parser->bytes[0] = OBSERVER_LINK_SOF0;
        parser->used = 1u;
    }
    else
    {
        parser->used = 0u;
    }
    parser->expected = 0u;
    return;
}
}

void observerLinkParserInit(observerLinkParser *parser)
{
    if (parser != NULL)
    {
        memset(parser, 0, sizeof(*parser));
    }
}

uint16_t observerLinkEncodeFrame(const observerLinkFrame *frame, uint8_t *output, uint16_t capacity)
{
    uint16_t total;
    uint32_t crc;

    if ((frame == NULL) || (output == NULL) ||
        (frame->payloadLength > OBSERVER_LINK_MAX_PAYLOAD_BYTES))
    {
        return 0u;
    }
    total = (uint16_t) (OBSERVER_LINK_HEADER_BYTES + frame->payloadLength +
        OBSERVER_LINK_CRC_BYTES);
    if (capacity < total)
    {
        return 0u;
    }

    output[0] = OBSERVER_LINK_SOF0;
    output[1] = OBSERVER_LINK_SOF1;
    output[2] = frame->major;
    output[3] = frame->minor;
    output[4] = frame->type;
    output[5] = frame->flags;
    putU16(&output[6], frame->payloadLength);
    putU32(&output[8], frame->bootSession);
    putU32(&output[12], frame->packetSeq);
    putU32(&output[16], frame->tickMs);
    if (frame->payloadLength != 0u)
    {
        memcpy(&output[OBSERVER_LINK_HEADER_BYTES],
            frame->payload,
            frame->payloadLength);
    }
    crc = crc32IeeeCalc(&output[2],
        (uint32_t) (OBSERVER_LINK_HEADER_BYTES - 2u + frame->payloadLength));
    putU32(&output[OBSERVER_LINK_HEADER_BYTES + frame->payloadLength], crc);
    return total;
}

uint8_t observerLinkParserFeed(observerLinkParser *parser, uint8_t byte, observerLinkFrame *frame)
{
    uint16_t payloadLength;
    uint32_t expectedCrc;
    uint32_t actualCrc;

    if ((parser == NULL) || (frame == NULL))
    {
        return 0u;
    }

    if (parser->used == 0u)
    {
        if (byte == OBSERVER_LINK_SOF0)
        {
            parser->bytes[0] = byte;
            parser->used = 1u;

```

```

    }
    return 0u;
}
if (parser->used == 1u)
{
    if (byte == OBSERVER_LINK_SOF1)
    {
        parser->bytes[1] = byte;
        parser->used = 2u;
    }
    else if (byte != OBSERVER_LINK_SOF0)
    {
        parser->used = 0u;
    }
    return 0u;
}

if (parser->used >= OBSERVER_LINK_MAX_FRAME_BYTES)
{
    parser->lengthErrors++;
    parser->used = 0u;
    parser->expected = 0u;
    return 0u;
}
parser->bytes[parser->used++] = byte;

if ((parser->used >= 8u) && (parser->expected == 0u))
{
    if (updateExpected(parser) == 0u)
    {
        parser->lengthErrors++;
        recoverParser(parser);
        return 0u;
    }
}
if ((parser->expected == 0u) || (parser->used < parser->expected))
{
    return 0u;
}

validateFrame:
payloadLength = getU16(&parser->bytes[6]);
expectedCrc = getU32(&parser->bytes[OBSERVER_LINK_HEADER_BYTES + payloadLength]);
actualCrc = crc32IeeeCalc(&parser->bytes[2],
    (uint32_t) (OBSERVER_LINK_HEADER_BYTES - 2u + payloadLength));
if (expectedCrc != actualCrc)
{
    parser->crcErrors++;
    recoverParser(parser);
    if ((parser->expected != 0u) && (parser->used >= parser->expected))
    {
        goto validateFrame;
    }
    return 0u;
}

frame->major = parser->bytes[2];
frame->minor = parser->bytes[3];
frame->type = parser->bytes[4];
frame->flags = parser->bytes[5];
frame->payloadLength = payloadLength;
frame->bootSession = getU32(&parser->bytes[8]);
frame->packetSeq = getU32(&parser->bytes[12]);
frame->tickMs = getU32(&parser->bytes[16]);
if (payloadLength != 0u)
{
    memcpy(frame->payload,
        &parser->bytes[OBSERVER_LINK_HEADER_BYTES],
        payloadLength);
}
if (frame->major != OBSERVER_LINK_PROTOCOL_MAJOR)
{
    parser->versionErrors++;
}
parser->validFrames++;
if (parser->used > parser->expected)
{
    uint16_t remaining = (uint16_t) (parser->used - parser->expected);
    memmove(parser->bytes, &parser->bytes[parser->expected], remaining);
}

```

```

    parser->used = remaining;
    if (updateExpected(parser) == 0u)
    {
        parser->lengthErrors++;
        recoverParser(parser);
    }
}
else
{
    parser->used = 0u;
    parser->expected = 0u;
}
return 1u;
}

```

```

uint8_t observerLinkEncodeHello(uint8_t role, uint32_t capabilities, uint8_t *payload, uint16_t *length)
{
    if ((payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    payload[0] = role;
    payload[1] = OBSERVER_LINK_PROTOCOL_MAJOR;
    payload[2] = OBSERVER_LINK_PROTOCOL_MINOR;
    payload[3] = 0u;
    putU32(&payload[4], capabilities);
    *length = 8u;
    return 1u;
}

```

```

uint8_t observerLinkDecodeHello(const uint8_t *payload, uint16_t length, uint8_t *role, uint32_t *capabilities)
{
    if ((payload == NULL) || (role == NULL) || (capabilities == NULL) ||
        (length != 8u))
    {
        return 0u;
    }
    if ((payload[1] != OBSERVER_LINK_PROTOCOL_MAJOR) ||
        (payload[2] != OBSERVER_LINK_PROTOCOL_MINOR))
    {
        return 0u;
    }
    *role = payload[0];
    *capabilities = getU32(&payload[4]);
    return 1u;
}

```

```

uint8_t observerLinkEncodeSharedConfig(const observerLinkSharedConfig *config, uint8_t *payload, uint16_t *length)
{
    uint8_t i;
    if ((config == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    memset(payload, 0, OBSERVER_LINK_SHARED_CONFIG_BYTES);
    putU32(&payload[0], config->revision);
    putU32(&payload[4], config->hash);
    payload[8] = config->detectMode;
    payload[9] = config->maxBeacons;
    putU16(&payload[10], config->edgePredictionHoldMs);
    putU16(&payload[12], config->beaconSizeMin);
    putU16(&payload[14], config->beaconSizeMax);
    putFloat(&payload[16], config->beaconEccMax);
    payload[20] = config->beaconBrightnessMin;
    payload[21] = config->sourceId;
    putU16(&payload[22], config->beaconMaxEndpoints);
    putFloat(&payload[24], config->beaconFillMin);
    putFloat(&payload[28], config->beaconHuMax);
    putU16(&payload[32], config->beaconHuAreaMin);
    payload[34] = config->profileVersion;
    putFloat(&payload[36], config->trackMaxAssocDist);
    putFloat(&payload[40], config->trackSizeRatioMax);
    putU16(&payload[44], config->trackConfirmHits);
    putU16(&payload[46], config->trackDropMisses);
    putU16(&payload[48], config->trackCoastMisses);
    putFloat(&payload[52], config->trackPositionAlpha);
    putFloat(&payload[56], config->trackSizeAlpha);
    putFloat(&payload[60], config->trackAreaVelAlpha);
    putFloat(&payload[64], config->trackAssocScoreMax);
}

```

```

    putFloat(&payload[68], config->trackAssocAreaWeight);
    putFloat(&payload[72], config->trackAssocAreaVelWeight);
    for (i = 0u; i < 7u; i++)
    {
        putFloat(&payload[76u + (uint16_t) i * 4u], config->beaconHuTemplate[i]);
    }
    payload[104] = config->c1BorderEnterLeft;
    payload[105] = config->c1BorderEnterRight;
    payload[106] = config->c1BorderEnterTop;
    payload[107] = config->c1BorderEnterBottom;
    payload[108] = config->c1BorderExitLeft;
    payload[109] = config->c1BorderExitRight;
    payload[110] = config->c1BorderExitTop;
    payload[111] = config->c1BorderExitBottom;
    payload[112] = config->observerBorderEnterLeft;
    payload[113] = config->observerBorderEnterRight;
    payload[114] = config->observerBorderEnterTop;
    payload[115] = config->observerBorderEnterBottom;
    payload[116] = config->observerBorderExitLeft;
    payload[117] = config->observerBorderExitRight;
    payload[118] = config->observerBorderExitTop;
    payload[119] = config->observerBorderExitBottom;
    payload[120] = config->c1EdgeBandLeft;
    payload[121] = config->c1EdgeBandRight;
    payload[122] = config->c1EdgeBandTop;
    payload[123] = config->c1EdgeBandBottom;
    payload[124] = config->observerEdgeBandLeft;
    payload[125] = config->observerEdgeBandRight;
    payload[126] = config->observerEdgeBandTop;
    payload[127] = config->observerEdgeBandBottom;
    *length = OBSERVER_LINK_SHARED_CONFIG_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeSharedConfig(const uint8_t *payload, uint16_t length, observerLinkSharedConfig *config)
{
    uint8_t i;
    if ((payload == NULL) || (config == NULL) ||
        (length != OBSERVER_LINK_SHARED_CONFIG_BYTES))
    {
        return 0u;
    }
    memset(config, 0, sizeof(*config));
    config->revision = getU32(&payload[0]);
    config->hash = getU32(&payload[4]);
    config->detectMode = payload[8];
    config->maxBeacons = payload[9];
    config->edgePredictionHoldMs = getU16(&payload[10]);
    config->beaconSizeMin = getU16(&payload[12]);
    config->beaconSizeMax = getU16(&payload[14]);
    config->beaconEccMax = getFloat(&payload[16]);
    config->beaconBrightnessMin = payload[20];
    config->sourceId = payload[21];
    config->beaconMaxEndpoints = getU16(&payload[22]);
    config->beaconFillMin = getFloat(&payload[24]);
    config->beaconHuMax = getFloat(&payload[28]);
    config->beaconHuAreaMin = getU16(&payload[32]);
    config->profileVersion = payload[34];
    config->trackMaxAssocDist = getFloat(&payload[36]);
    config->trackSizeRatioMax = getFloat(&payload[40]);
    config->trackConfirmHits = getU16(&payload[44]);
    config->trackDropMisses = getU16(&payload[46]);
    config->trackCoastMisses = getU16(&payload[48]);
    config->trackPositionAlpha = getFloat(&payload[52]);
    config->trackSizeAlpha = getFloat(&payload[56]);
    config->trackAreaVelAlpha = getFloat(&payload[60]);
    config->trackAssocScoreMax = getFloat(&payload[64]);
    config->trackAssocAreaWeight = getFloat(&payload[68]);
    config->trackAssocAreaVelWeight = getFloat(&payload[72]);
    for (i = 0u; i < 7u; i++)
    {
        config->beaconHuTemplate[i] = getFloat(&payload[76u + (uint16_t) i * 4u]);
    }
    config->c1BorderEnterLeft = payload[104];
    config->c1BorderEnterRight = payload[105];
    config->c1BorderEnterTop = payload[106];
    config->c1BorderEnterBottom = payload[107];
    config->c1BorderExitLeft = payload[108];
    config->c1BorderExitRight = payload[109];

```

```

config->c1BorderExitTop = payload[110];
config->c1BorderExitBottom = payload[111];
config->observerBorderEnterLeft = payload[112];
config->observerBorderEnterRight = payload[113];
config->observerBorderEnterTop = payload[114];
config->observerBorderEnterBottom = payload[115];
config->observerBorderExitLeft = payload[116];
config->observerBorderExitRight = payload[117];
config->observerBorderExitTop = payload[118];
config->observerBorderExitBottom = payload[119];
config->c1EdgeBandLeft = payload[120];
config->c1EdgeBandRight = payload[121];
config->c1EdgeBandTop = payload[122];
config->c1EdgeBandBottom = payload[123];
config->observerEdgeBandLeft = payload[124];
config->observerEdgeBandRight = payload[125];
config->observerEdgeBandTop = payload[126];
config->observerEdgeBandBottom = payload[127];
return 1u;
}

static uint8_t borderSideValid(uint8_t enter, uint8_t exit, uint8_t maxValue)
{
    if ((enter == 0u) && (exit == 0u)) return 1u;
    return (uint8_t) ((enter >= 1u) && (enter <= maxValue) && (exit < enter));
}

static uint8_t sharedConfigBorderValid(const observerLinkSharedConfig *config)
{
    if (config == NULL) return 0u;
    return (uint8_t) (
        (config->edgePredictionHoldMs <= OBSERVER_LINK_EDGE_PREDICTION_HOLD_MS_MAX) &&
        ((config->sourceId == OBSERVER_LINK_SOURCE_CORE3) ||
         (config->sourceId == OBSERVER_LINK_SOURCE_CORE4)) &&
        (config->profileVersion == 1u) &&
        (config->c1EdgeBandLeft <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
        (config->c1EdgeBandRight <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
        (config->c1EdgeBandTop <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        (config->c1EdgeBandBottom <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        (config->observerEdgeBandLeft <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
        (config->observerEdgeBandRight <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
        (config->observerEdgeBandTop <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        (config->observerEdgeBandBottom <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        borderSideValid(config->c1BorderEnterLeft, config->c1BorderExitLeft, 93u) &&
        borderSideValid(config->c1BorderEnterRight, config->c1BorderExitRight, 93u) &&
        borderSideValid(config->c1BorderEnterTop, config->c1BorderExitTop, 59u) &&
        borderSideValid(config->c1BorderEnterBottom, config->c1BorderExitBottom, 59u) &&
        borderSideValid(config->observerBorderEnterLeft, config->observerBorderExitLeft, 93u) &&
        borderSideValid(config->observerBorderEnterRight, config->observerBorderExitRight, 93u) &&
        borderSideValid(config->observerBorderEnterTop, config->observerBorderExitTop, 59u) &&
        borderSideValid(config->observerBorderEnterBottom, config->observerBorderExitBottom, 59u));
}

uint32_t observerLinkSharedConfigHash(const observerLinkSharedConfig *config)
{
    uint8_t payload[OBSERVER_LINK_SHARED_CONFIG_BYTES];
    uint16_t length;

    if ((config == NULL) ||
        (observerLinkEncodeSharedConfig(config, payload, &length) == 0u))
    {
        return 0u;
    }
    return crc32IeeeCalc(&payload[8], (uint32_t) (length - 8u));
}

uint8_t observerLinkSharedConfigIsValid(const observerLinkSharedConfig *config)
{
    uint8_t i;
    if ((config == NULL) || (config->revision == 0u) ||
        (config->hash != observerLinkSharedConfigHash(config)) ||
        (config->detectMode > 1u) ||
        (config->maxBeacons < 1u) || (config->maxBeacons > 32u) ||
        (sharedConfigBorderValid(config) == 0u) ||
        (config->beaconSizeMin < 1u) ||
        (config->beaconSizeMax <= config->beaconSizeMin) ||
        (config->beaconSizeMax > 60000u) ||
        (config->beaconMaxEndpoints > 200u) ||
        (config->beaconHuAreaMin < 1u) ||

```

```

        (config->beaconHuAreaMin > 100u) ||
        (config->trackConfirmHits < 1u) || (config->trackConfirmHits > 60u) ||
        (config->trackDropMisses < 1u) || (config->trackDropMisses > 60u) ||
        (config->trackCoastMisses < 1u) || (config->trackCoastMisses > 60u))
    {
        return 0u;
    }
    for (i = 0u; i < 7u; i++)
    {
        if (finiteInRange(config->beaconHuTemplate[i], -1 000 000.0f, 1 000 000.0f) == 0u)
        {
            return 0u;
        }
    }
    return (uint8_t) (
        finiteInRange(config->beaconEccMax, 0.0f, 20.0f) &&
        finiteInRange(config->beaconFillMin, 0.0f, 1.0f) &&
        finiteInRange(config->beaconHuMax, 0.0f, 200.0f) &&
        finiteInRange(config->trackMaxAssocDist, 1.0f, 100.0f) &&
        finiteInRange(config->trackSizeRatioMax, 1.1f, 10.0f) &&
        finiteInRange(config->trackPositionAlpha, 0.01f, 1.0f) &&
        finiteInRange(config->trackSizeAlpha, 0.01f, 1.0f) &&
        finiteInRange(config->trackAreaVelAlpha, 0.01f, 1.0f) &&
        finiteInRange(config->trackAssocScoreMax, 0.1f, 10.0f) &&
        finiteInRange(config->trackAssocAreaWeight, 0.0f, 3.0f) &&
        finiteInRange(config->trackAssocAreaVelWeight, 0.0f, 3.0f));
}

uint8_t observerLinkEncodeConfigAck(const observerLinkConfigAck *ack, uint8_t *payload, uint16_t *length)
{
    if ((ack == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], ack->ownerSession);
    putU32(&payload[4], ack->revision);
    putU32(&payload[8], ack->hash);
    payload[12] = ack->desiredMode;
    payload[13] = ack->appliedMode;
    payload[14] = ack->status;
    payload[15] = ack->reason;
    *length = OBSERVER_LINK_CONFIG_ACK_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeConfigAck(const uint8_t *payload, uint16_t length, observerLinkConfigAck *ack)
{
    if ((payload == NULL) || (ack == NULL) ||
        (length != OBSERVER_LINK_CONFIG_ACK_BYTES))
    {
        return 0u;
    }
    ack->ownerSession = getU32(&payload[0]);
    ack->revision = getU32(&payload[4]);
    ack->hash = getU32(&payload[8]);
    ack->desiredMode = payload[12];
    ack->appliedMode = payload[13];
    ack->status = payload[14];
    ack->reason = payload[15];
    return 1u;
}

uint8_t observerLinkEncodeDiagnostics(const observerLinkDiagnostics *diagnostics, uint8_t *payload, uint16_t *length)
{
    if ((diagnostics == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], diagnostics->validFrames);
    putU32(&payload[4], diagnostics->crcErrors);
    putU32(&payload[8], diagnostics->lengthErrors);
    putU32(&payload[12], diagnostics->versionErrors);
    putU32(&payload[16], diagnostics->rxOverflowErrors);
    putU32(&payload[20], diagnostics->rxFrameErrors);
    putU32(&payload[24], diagnostics->rxParityErrors);
    putU32(&payload[28], diagnostics->txDeferred);
    *length = OBSERVER_LINK_DIAGNOSTICS_BYTES;
    return 1u;
}

```

```

uint8_t observerLinkDecodeDiagnostics(const uint8_t *payload, uint16_t length, observerLinkDiagnostics *diagnostics)
{
    if ((payload == NULL) || (diagnostics == NULL) ||
        (length != OBSERVER_LINK_DIAGNOSTICS_BYTES))
    {
        return 0u;
    }
    diagnostics->validFrames = getU32(&payload[0]);
    diagnostics->crcErrors = getU32(&payload[4]);
    diagnostics->lengthErrors = getU32(&payload[8]);
    diagnostics->versionErrors = getU32(&payload[12]);
    diagnostics->rxOverflowErrors = getU32(&payload[16]);
    diagnostics->rxFrameErrors = getU32(&payload[20]);
    diagnostics->rxParityErrors = getU32(&payload[24]);
    diagnostics->txDeferred = getU32(&payload[28]);
    return 1u;
}

uint8_t observerLinkEncodeKvmControl(const observerLinkKvmControl *control, uint8_t *payload, uint16_t *length)
{
    if ((control == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], control->targetSession);
    putU32(&payload[4], control->routeSession);
    putU32(&payload[8], control->routeEpoch);
    putU32(&payload[12], control->leaseMs);
    payload[16] = control->target;
    *length = OBSERVER_LINK_KVM_CONTROL_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeKvmControl(const uint8_t *payload, uint16_t length, observerLinkKvmControl *control)
{
    if ((payload == NULL) || (control == NULL) || (length != OBSERVER_LINK_KVM_CONTROL_BYTES))
    {
        return 0u;
    }
    memset(control, 0, sizeof(*control));
    control->targetSession = getU32(&payload[0]);
    control->routeSession = getU32(&payload[4]);
    control->routeEpoch = getU32(&payload[8]);
    control->leaseMs = getU32(&payload[12]);
    control->target = payload[16];
    return 1u;
}

uint8_t observerLinkEncodeKvmEvent(const observerLinkKvmEvent *event, uint8_t *payload, uint16_t *length)
{
    if ((event == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], event->targetSession);
    putU32(&payload[4], event->routeSession);
    putU32(&payload[8], event->routeEpoch);
    putU32(&payload[12], event->eventSeq);
    payload[16] = event->target;
    payload[17] = event->keyId;
    payload[18] = event->keyState;
    payload[19] = event->flags;
    *length = OBSERVER_LINK_KVM_EVENT_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeKvmEvent(const uint8_t *payload, uint16_t length, observerLinkKvmEvent *event)
{
    if ((payload == NULL) || (event == NULL) || (length != OBSERVER_LINK_KVM_EVENT_BYTES))
    {
        return 0u;
    }
    memset(event, 0, sizeof(*event));
    event->targetSession = getU32(&payload[0]);
    event->routeSession = getU32(&payload[4]);
    event->routeEpoch = getU32(&payload[8]);
    event->eventSeq = getU32(&payload[12]);
    event->target = payload[16];

```



```

    event->keyId = payload[17];
    event->keyState = payload[18];
    event->flags = payload[19];
    return 1u;
}

uint8_t observerLinkEncodeKvmAck(const observerLinkKvmAck *ack, uint8_t *payload, uint16_t *length)
{
    if ((ack == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], ack->targetSession);
    putU32(&payload[4], ack->routeSession);
    putU32(&payload[8], ack->routeEpoch);
    putU32(&payload[12], ack->eventSeq);
    putU32(&payload[16], ack->expectedSeq);
    payload[20] = ack->target;
    payload[21] = ack->result;
    payload[22] = ack->menuState;
    payload[23] = ack->flags;
    *length = OBSERVER_LINK_KVM_ACK_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeKvmAck(const uint8_t *payload, uint16_t length, observerLinkKvmAck *ack)
{
    if ((payload == NULL) || (ack == NULL) || (length != OBSERVER_LINK_KVM_ACK_BYTES))
    {
        return 0u;
    }
    memset(ack, 0, sizeof(*ack));
    ack->targetSession = getU32(&payload[0]);
    ack->routeSession = getU32(&payload[4]);
    ack->routeEpoch = getU32(&payload[8]);
    ack->eventSeq = getU32(&payload[12]);
    ack->expectedSeq = getU32(&payload[16]);
    ack->target = payload[20];
    ack->result = payload[21];
    ack->menuState = payload[22];
    ack->flags = payload[23];
    return 1u;
}

uint8_t observerLinkEncodeKvmMenuStatus(const observerLinkKvmMenuStatus *status, uint8_t *payload, uint16_t *length)
{
    if ((status == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], status->targetSession);
    putU32(&payload[4], status->routeSession);
    putU32(&payload[8], status->routeEpoch);
    putU32(&payload[12], status->lastEventSeq);
    putU32(&payload[16], status->screenRevision);
    payload[20] = status->target;
    payload[21] = status->menuState;
    payload[22] = status->flags;
    payload[23] = status->lastAckResult;
    *length = OBSERVER_LINK_KVM_MENU_STATUS_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeKvmMenuStatus(const uint8_t *payload, uint16_t length, observerLinkKvmMenuStatus *status)
{
    if ((payload == NULL) || (status == NULL) || (length != OBSERVER_LINK_KVM_MENU_STATUS_BYTES))
    {
        return 0u;
    }
    memset(status, 0, sizeof(*status));
    status->targetSession = getU32(&payload[0]);
    status->routeSession = getU32(&payload[4]);
    status->routeEpoch = getU32(&payload[8]);
    status->lastEventSeq = getU32(&payload[12]);
    status->screenRevision = getU32(&payload[16]);
    status->target = payload[20];
    status->menuState = payload[21];
    status->flags = payload[22];
    status->lastAckResult = payload[23];
}

```

```

    return 1u;
}

static uint8_t controlStatusIsValid(const observerLinkControlStatus *status)
{
    uint8_t statusValid;
    uint8_t decisionValid;
    uint8_t barrier;
    uint8_t dispatch;
    uint8_t core0LinkUp;
    uint8_t core2LinkUp;
    uint8_t requiredVision;

    if ((status == NULL) || (status->detectMode > 1u) ||
        (status->protocolCompatible > 1u) || (status->algorithmsReady > 1u) ||
        (status->configCompatible > 1u) ||
        (status->cameraControlState > OBSERVER_LINK_CAMERA_CONTROL_RUN) ||
        ((status->activeSource != OBSERVER_LINK_SOURCE_NONE) &&
         (status->activeSource != OBSERVER_LINK_SOURCE_CORE1) &&
         (status->activeSource != OBSERVER_LINK_SOURCE_CORE3) &&
         (status->activeSource != OBSERVER_LINK_SOURCE_CORE4)) ||
        (status->controlClass > OBSERVER_LINK_CONTROL_CLASS_SEARCH) ||
        ((status->navState != 0u) && (status->navState != 2u) &&
         (status->navState != 3u) && (status->navState != 4u)) ||
        (status->systemPhase > OBSERVER_LINK_SYSTEM_PHASE_RUNNING) ||
        ((status->systemFlags & 0x80u) != 0u) ||
        (status->primaryReason > OBSERVER_LINK_REASON_LANDING) ||
        ((status->desiredTarget != OBSERVER_LINK_KVM_TARGET_CORE0) &&
         (status->desiredTarget != OBSERVER_LINK_KVM_TARGET_CORE1) &&
         (status->desiredTarget != OBSERVER_LINK_KVM_TARGET_CORE3) &&
         (status->desiredTarget != OBSERVER_LINK_KVM_TARGET_CORE4) &&
         (status->desiredTarget != OBSERVER_LINK_KVM_TARGET_NONE)) ||
        ((status->ownerTarget != OBSERVER_LINK_KVM_TARGET_CORE0) &&
         (status->ownerTarget != OBSERVER_LINK_KVM_TARGET_CORE1) &&
         (status->ownerTarget != OBSERVER_LINK_KVM_TARGET_CORE3) &&
         (status->ownerTarget != OBSERVER_LINK_KVM_TARGET_CORE4) &&
         (status->ownerTarget != OBSERVER_LINK_KVM_TARGET_NONE)) ||
        (status->targetX >= 188u) ||
        (status->targetY >= 120u) || (status->targetSize > 22560u))
    {
        return 0u;
    }

    statusValid = (uint8_t) ((status->flags &
        OBSERVER_LINK_CONTROL_FLAG_STATUS_VALID) != 0u);
    decisionValid = (uint8_t) ((status->flags &
        OBSERVER_LINK_CONTROL_FLAG_DECISION_VALID) != 0u);
    barrier = (uint8_t) ((status->flags &
        OBSERVER_LINK_CONTROL_FLAG_BARRIER_REQUIRED) != 0u);
    dispatch = (uint8_t) ((status->flags &
        OBSERVER_LINK_CONTROL_FLAG_DISPATCH_ENABLED) != 0u);
    core0LinkUp = (uint8_t) ((status->flags &
        OBSERVER_LINK_CONTROL_FLAG_CORE0_LINK_UP) != 0u);
    core2LinkUp = (uint8_t) ((status->flags &
        OBSERVER_LINK_CONTROL_FLAG_CORE2_LINK_UP) != 0u);
    requiredVision = (uint8_t) ((status->flags &
        OBSERVER_LINK_CONTROL_FLAG_REQUIRED_VISION) != 0u);

    if (((status->systemFlags & OBSERVER_LINK_SYSTEM_FLAG_STANDALONE) != 0u) !=
        (status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_STANDALONE)) ||
        (((status->systemFlags & OBSERVER_LINK_SYSTEM_FLAG_REJOIN) != 0u) !=
         (status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_REJOIN)) ||
        ((status->systemFlags & OBSERVER_LINK_SYSTEM_FLAG_LANDING) != 0u) &&
        ((status->systemFlags & OBSERVER_LINK_SYSTEM_FLAG_LAUNCH_ELIGIBLE) != 0u)))
    {
        return 0u;
    }

    if (statusValid == 0u)
    {
        return (uint8_t) ((status->cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_SAFE) &&
            (status->activeSource == OBSERVER_LINK_SOURCE_NONE) &&
            (status->controlClass == OBSERVER_LINK_CONTROL_CLASS_NONE) &&
            (status->navState == 0u) && (status->beaconCount == 0u) &&
            (status->targetX == 0u) && (status->targetY == 0u) &&
            (status->targetSize == 0u) && (status->errXQ4 == 0) &&
            (status->errYQ4 == 0) && (status->flags == 0u));
    }

    if ((status->cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_SAFE) ||

```

```

((status->cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_RUN) !=
 (requiredVision != 0u)) ||
((core2LinkUp != 0u) && (core0LinkUp == 0u)) ||
((decisionValid != 0u) &&
 (status->controlClass == OBSERVER_LINK_CONTROL_CLASS_NONE)) ||
((status->controlClass == OBSERVER_LINK_CONTROL_CLASS_NONE) &&
 (status->activeSource != OBSERVER_LINK_SOURCE_NONE)) ||
((status->controlClass == OBSERVER_LINK_CONTROL_CLASS_CORE1_TRACK) &&
 (status->activeSource != OBSERVER_LINK_SOURCE_CORE1)) ||
((status->controlClass == OBSERVER_LINK_CONTROL_CLASS_OBSERVER_TRACK) &&
 (status->activeSource != OBSERVER_LINK_SOURCE_CORE3) &&
 (status->activeSource != OBSERVER_LINK_SOURCE_CORE4)) ||
((status->controlClass == OBSERVER_LINK_CONTROL_CLASS_SEARCH) &&
 (status->activeSource != OBSERVER_LINK_SOURCE_NONE)) ||
((dispatch != 0u) && ((decisionValid == 0u) || (barrier != 0u) ||
 (status->controlClass == OBSERVER_LINK_CONTROL_CLASS_NONE))))
{
    return 0u;
}
return 1u;
}

```

```

uint8_t observerLinkEncodeControlStatus(const observerLinkControlStatus *status,
uint8_t *payload, uint16_t *length)

```

```

{
    if ((payload == NULL) || (length == NULL) ||
        (controlStatusIsValid(status) == 0u))
    {
        return 0u;
    }
    payload[0] = status->detectMode;
    payload[1] = status->protocolCompatible;
    payload[2] = status->algorithmsReady;
    payload[3] = status->configCompatible;
    payload[4] = status->cameraControlState;
    payload[5] = status->activeSource;
    payload[6] = status->controlClass;
    payload[7] = status->flags;
    payload[8] = status->navState;
    payload[9] = status->beaconCount;
    putU16(&payload[10], status->targetX);
    putU16(&payload[12], status->targetY);
    putU16(&payload[14], status->targetSize);
    putU16(&payload[16], (uint16_t) status->errXQ4);
    putU16(&payload[18], (uint16_t) status->errYQ4);
    payload[20] = status->systemPhase;
    payload[21] = status->systemFlags;
    payload[22] = status->primaryReason;
    payload[23] = status->desiredTarget;
    payload[24] = status->ownerTarget;
    *length = OBSERVER_LINK_CORE1_HEARTBEAT_BYTES;
    return 1u;
}

```

```

uint8_t observerLinkDecodeControlStatus(const uint8_t *payload, uint16_t length,
observerLinkControlStatus *status)

```

```

{
    if ((payload == NULL) || (status == NULL) ||
        (length != OBSERVER_LINK_CORE1_HEARTBEAT_BYTES))
    {
        return 0u;
    }
    memset(status, 0, sizeof(*status));
    status->detectMode = payload[0];
    status->protocolCompatible = payload[1];
    status->algorithmsReady = payload[2];
    status->configCompatible = payload[3];
    status->cameraControlState = payload[4];
    status->activeSource = payload[5];
    status->controlClass = payload[6];
    status->flags = payload[7];
    status->navState = payload[8];
    status->beaconCount = payload[9];
    status->targetX = getU16(&payload[10]);
    status->targetY = getU16(&payload[12]);
    status->targetSize = getU16(&payload[14]);
    status->errXQ4 = (int16_t) getU16(&payload[16]);
    status->errYQ4 = (int16_t) getU16(&payload[18]);
    status->systemPhase = payload[20];
}

```

```

    status->systemFlags = payload[21];
    status->primaryReason = payload[22];
    status->desiredTarget = payload[23];
    status->ownerTarget = payload[24];
    return controlStatusIsValid(status);
}

static uint8_t observerSourceIsValid(uint8_t sourceId)
{
    return (uint8_t) ((sourceId == OBSERVER_LINK_SOURCE_CORE3) ||
        (sourceId == OBSERVER_LINK_SOURCE_CORE4));
}

uint8_t observerLinkEncodeHealth(const observerLinkHealth *health,
    uint8_t *payload, uint16_t *length)
{
    if ((health == NULL) || (payload == NULL) || (length == NULL) ||
        (observerSourceIsValid(health->sourceId) == 0u) ||
        (health->cameraState > 2u) ||
        (health->configState > OBSERVER_LINK_CONFIG_REJECTED))
    {
        return 0u;
    }
    putU32(&payload[0], health->ownerSession);
    putU32(&payload[4], health->appFrameSeq);
    putU32(&payload[8], health->captureTickMs);
    putU32(&payload[12], health->bridgeSeq);
    putU16(&payload[16], health->frameAgeMs);
    putU16(&payload[18], health->bridgeAgeMs);
    payload[20] = health->sourceId;
    payload[21] = health->cameraState;
    payload[22] = health->configState;
    payload[23] = health->flags;
    *length = OBSERVER_LINK_HEALTH_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeHealth(const uint8_t *payload, uint16_t length,
    observerLinkHealth *health)
{
    if ((payload == NULL) || (health == NULL) ||
        (length != OBSERVER_LINK_HEALTH_BYTES))
    {
        return 0u;
    }
    health->ownerSession = getU32(&payload[0]);
    health->appFrameSeq = getU32(&payload[4]);
    health->captureTickMs = getU32(&payload[8]);
    health->bridgeSeq = getU32(&payload[12]);
    health->frameAgeMs = getU16(&payload[16]);
    health->bridgeAgeMs = getU16(&payload[18]);
    health->sourceId = payload[20];
    health->cameraState = payload[21];
    health->configState = payload[22];
    health->flags = payload[23];
    return (uint8_t) ((observerSourceIsValid(health->sourceId) != 0u) &&
        (health->cameraState <= 2u) &&
        (health->configState <= OBSERVER_LINK_CONFIG_REJECTED));
}

static uint8_t viewOverlayIsValid(const observerLinkViewOverlay *overlay)
{
    uint16_t knownFlags = OBSERVER_LINK_OVERLAY_FLAG_VALID |
        OBSERVER_LINK_OVERLAY_FLAG_CENTER | OBSERVER_LINK_OVERLAY_FLAG_DEADZONE |
        OBSERVER_LINK_OVERLAY_FLAG_SELECTED | OBSERVER_LINK_OVERLAY_FLAG_POSE;
    uint8_t selectedPresent;
    uint8_t posePresent;

    if ((overlay == NULL) || (overlay->overlaySeq == 0u) ||
        (overlay->ownerSession == 0u) || (overlay->configRevision == 0u) ||
        (overlay->detectMode > 1u) || ((overlay->flags & ~knownFlags) != 0u) ||
        ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_VALID) == 0u)) return 0u;
    selectedPresent = (uint8_t) ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_SELECTED) != 0u);
    posePresent = (uint8_t) ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_POSE) != 0u);
    if ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_CENTER) != 0u)
    {
        if ((overlay->activeCenterX >= 188u) || (overlay->activeCenterY >= 120u)) return 0u;
    }
    else if ((overlay->activeCenterX != 0u) || (overlay->activeCenterY != 0u)) return 0u;
}

```

```

if ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_DEADZONE) != 0u)
{
    if ((overlay->deadzoneEnterRadius > 188u) ||
        (overlay->deadzoneExitRadius > 188u) ||
        (overlay->deadzoneExitRadius > overlay->deadzoneEnterRadius)) return 0u;
}
else if ((overlay->deadzoneEnterRadius != 0u) ||
        (overlay->deadzoneExitRadius != 0u)) return 0u;
if (selectedPresent != 0u)
{
    if ((overlay->sourceFrameSeq == 0u) ||
        (overlay->selectedTrackId == 0u) || (overlay->selectedX >= 188u) ||
        (overlay->selectedY >= 120u) || (overlay->selectedSize > 22 560u) ||
        ((overlay->selectedFlags & (uint8_t) ~(OBSERVER_LINK_OVERLAY_SELECTED_FRESH |
        OBSERVER_LINK_OVERLAY_SELECTED_COAST)) != 0u) ||
        (overlay->selectedFlags == 0u) ||
        (overlay->selectedFlags == (OBSERVER_LINK_OVERLAY_SELECTED_FRESH |
        OBSERVER_LINK_OVERLAY_SELECTED_COAST))) return 0u;
}
else if ((overlay->selectedTrackId != 0u) || (overlay->selectedX != 0u) ||
        (overlay->selectedY != 0u) || (overlay->selectedSize != 0u) ||
        (overlay->selectedFlags != 0u)) return 0u;
if (posePresent != 0u)
{
    uint8_t poseAgeFlags = (uint8_t) (overlay->poseFlags &
        (OBSERVER_LINK_OVERLAY_POSE_FRESH | OBSERVER_LINK_OVERLAY_POSE_HOLD));
    if ((overlay->poseX >= 188u) || (overlay->poseY >= 120u) ||
        ((overlay->poseFlags & (uint8_t) ~(OBSERVER_LINK_OVERLAY_POSE_FRESH |
        OBSERVER_LINK_OVERLAY_POSE_HOLD |
        OBSERVER_LINK_OVERLAY_POSE_HEADING_VALID)) != 0u) ||
        (poseAgeFlags == 0u) ||
        (poseAgeFlags == (OBSERVER_LINK_OVERLAY_POSE_FRESH |
        OBSERVER_LINK_OVERLAY_POSE_HOLD))) return 0u;
    if ((overlay->poseFlags & OBSERVER_LINK_OVERLAY_POSE_HEADING_VALID) != 0u)
    {
        if ((overlay->poseHeadingQ8 < -805) || (overlay->poseHeadingQ8 > 805)) return 0u;
    }
    else if (overlay->poseHeadingQ8 != 0) return 0u;
}
else if ((overlay->poseX != 0u) || (overlay->poseY != 0u) ||
        (overlay->poseHeadingQ8 != 0) || (overlay->poseFlags != 0u)) return 0u;
return 1u;
}

uint8_t observerLinkEncodeViewOverlay(const observerLinkViewOverlay *overlay,
uint8_t *payload, uint16_t *length)
{
    if ((payload == NULL) || (length == NULL) ||
        (viewOverlayIsValid(overlay) == 0u)) return 0u;
    putU32(&payload[0], overlay->overlaySeq);
    putU32(&payload[4], overlay->sourceFrameSeq);
    putU32(&payload[8], overlay->ownerSession);
    putU32(&payload[12], overlay->configRevision);
    putU32(&payload[16], overlay->configHash);
    putU16(&payload[20], overlay->flags);
    putU16(&payload[22], overlay->selectedTrackId);
    putU16(&payload[24], overlay->activeCenterX);
    putU16(&payload[26], overlay->activeCenterY);
    putU16(&payload[28], overlay->deadzoneEnterRadius);
    putU16(&payload[30], overlay->deadzoneExitRadius);
    putU16(&payload[32], overlay->selectedX);
    putU16(&payload[34], overlay->selectedY);
    putU16(&payload[36], overlay->selectedSize);
    putU16(&payload[38], overlay->poseX);
    putU16(&payload[40], overlay->poseY);
    putU16(&payload[42], (uint16_t) overlay->poseHeadingQ8);
    payload[44] = overlay->selectedFlags;
    payload[45] = overlay->poseFlags;
    payload[46] = overlay->detectMode;
    *length = OBSERVER_LINK_VIEW_OVERLAY_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeViewOverlay(const uint8_t *payload, uint16_t length,
observerLinkViewOverlay *overlay)
{
    if ((payload == NULL) || (overlay == NULL) ||
        (length != OBSERVER_LINK_VIEW_OVERLAY_BYTES)) return 0u;
    memset(overlay, 0, sizeof(*overlay));

```

```

overlay->overlaySeq = getU32(&payload[0]);
overlay->sourceFrameSeq = getU32(&payload[4]);
overlay->ownerSession = getU32(&payload[8]);
overlay->configRevision = getU32(&payload[12]);
overlay->configHash = getU32(&payload[16]);
overlay->flags = getU16(&payload[20]);
overlay->selectedTrackId = getU16(&payload[22]);
overlay->activeCenterX = getU16(&payload[24]);
overlay->activeCenterY = getU16(&payload[26]);
overlay->deadzoneEnterRadius = getU16(&payload[28]);
overlay->deadzoneExitRadius = getU16(&payload[30]);
overlay->selectedX = getU16(&payload[32]);
overlay->selectedY = getU16(&payload[34]);
overlay->selectedSize = getU16(&payload[36]);
overlay->poseX = getU16(&payload[38]);
overlay->poseY = getU16(&payload[40]);
overlay->poseHeadingQ8 = (int16_t) getU16(&payload[42]);
overlay->selectedFlags = payload[44];
overlay->poseFlags = payload[45];
overlay->detectMode = payload[46];
return viewOverlayIsValid(overlay);
}

uint8_t observerLinkPlanningTrackFragmentCount(uint8_t retainedCount)
{
    if (retainedCount > OBSERVER_LINK_MAX_TRACKS)
    {
        return 0u;
    }
    return (retainedCount == 0u) ? 1u :
        (uint8_t) ((retainedCount + OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT - 1u) /
            OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT);
}

static uint8_t planningTrackIsValid(const observerLinkPlanningTrack *track)
{
    uint8_t fresh;
    uint8_t coast;
    uint8_t edgeGrace;
    uint8_t rejected;

    if ((track == NULL) || (track->centerX >= 188u) ||
        (track->centerY >= 120u) || (track->predX >= 188u) ||
        (track->predY >= 120u) || (track->size > 22 560u) ||
        (track->predSize > 22 560u) || (track->trackId == 0u) ||
        (track->misses > OBSERVER_LINK_PLANNING_TRACK_MISSES_MAX) ||
        ((track->flags & (uint8_t) ~(OBSERVER_LINK_TRACK_FLAG_FRESH |
            OBSERVER_LINK_TRACK_FLAG_COAST |
            OBSERVER_LINK_TRACK_FLAG_EDGE_GRACE |
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED)) != 0u))
    {
        return 0u;
    }
    fresh = (uint8_t) ((track->flags & OBSERVER_LINK_TRACK_FLAG_FRESH) != 0u);
    coast = (uint8_t) ((track->flags & OBSERVER_LINK_TRACK_FLAG_COAST) != 0u);
    edgeGrace = (uint8_t) ((track->flags & OBSERVER_LINK_TRACK_FLAG_EDGE_GRACE) != 0u);
    rejected = (uint8_t) ((track->flags &
        OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) != 0u);
    if (rejected != 0u)
    {
        return (uint8_t) ((track->flags ==
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) &&
            (track->centerX == 0u) && (track->centerY == 0u) &&
            (track->predX == 0u) && (track->predY == 0u) &&
            (track->size == 0u) && (track->predSize == 0u) &&
            (track->misses == 0u) && (track->velXQ7 == 0) &&
            (track->velYQ7 == 0) && (track->areaVelQ0 == 0));
    }
    if ((edgeGrace != 0u) && ((fresh != 0u) || (coast == 0u))) return 0u;
    return (uint8_t) (((fresh != 0u) && (track->misses == 0u) && (coast == 0u)) ||
        ((fresh == 0u) && (track->misses != 0u) && (coast != 0u)));
}

uint8_t observerLinkEncodePlanningTrackFragment(
    const observerLinkPlanningTrackSnapshot *snapshot,
    uint8_t fragmentIndex, uint8_t *payload, uint16_t *length)
{
    uint8_t fragmentCount;
    uint8_t first;

```

```

uint8_t count;
uint8_t i;
uint8_t ordinaryRetained = 0u;
uint16_t offset;

if ((snapshot == NULL) || (payload == NULL) || (length == NULL) ||
    (snapshot->valid == 0u) || (observerSourceIsValid(snapshot->sourceId) == 0u) ||
    (snapshot->detectMode > 1u) ||
    (snapshot->retainedCount > OBSERVER_LINK_MAX_TRACKS) ||
    (snapshot->freshTotal > snapshot->confirmedTotal) ||
    ((uint16_t) snapshot->freshLeft + snapshot->freshRight != snapshot->freshTotal))
{
    return 0u;
}
for (i = 0u; i < snapshot->retainedCount; i++)
{
    uint8_t j;
    if (planningTrackIsValid(&snapshot->tracks[i]) == 0u) return 0u;
    if ((snapshot->tracks[i].flags &
        OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) == 0u)
        ordinaryRetained++;
    for (j = 0u; j < i; j++)
    {
        if (snapshot->tracks[j].trackId == snapshot->tracks[i].trackId)
            return 0u;
    }
}
if (ordinaryRetained > snapshot->confirmedTotal) return 0u;
fragmentCount = observerLinkPlanningTrackFragmentCount(snapshot->retainedCount);
if (fragmentIndex >= fragmentCount)
{
    return 0u;
}
first = (uint8_t) (fragmentIndex * OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT);
count = (uint8_t) (snapshot->retainedCount - first);
if (count > OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT)
{
    count = OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT;
}
memset(payload, 0, OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_MAX_BYTES);
payload[0] = snapshot->sourceId;
payload[1] = snapshot->detectMode;
payload[2] = (uint8_t) (OBSERVER_LINK_TRACK_SNAPSHOT_VALID |
    ((snapshot->truncated != 0u) ? OBSERVER_LINK_TRACK_SNAPSHOT_TRUNCATED : 0u));
payload[3] = fragmentIndex;
payload[4] = fragmentCount;
payload[5] = first;
payload[6] = count;
payload[7] = snapshot->retainedCount;
putU32(&payload[8], snapshot->ownerSession);
putU32(&payload[12], snapshot->revision);
putU32(&payload[16], snapshot->hash);
putU32(&payload[20], snapshot->appFrameSeq);
putU32(&payload[24], snapshot->captureTickMs);
payload[28] = snapshot->confirmedTotal;
payload[29] = snapshot->freshTotal;
payload[30] = snapshot->freshLeft;
payload[31] = snapshot->freshRight;
putU32(&payload[32], snapshot->trackerFrameSeq);
offset = OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES;
for (i = 0u; i < count; i++)
{
    const observerLinkPlanningTrack *track = &snapshot->tracks[first + i];
    uint8_t j;
    if (planningTrackIsValid(track) == 0u)
    {
        return 0u;
    }
    for (j = 0u; j < (uint8_t) (first + i); j++)
    {
        if (snapshot->tracks[j].trackId == track->trackId)
        {
            return 0u;
        }
    }
    payload[offset + 0u] = track->centerX;
    payload[offset + 1u] = track->centerY;
    payload[offset + 2u] = track->predX;
    payload[offset + 3u] = track->predY;
}

```

```

        putU16(&payload[offset + 4u], track->size);
        putU16(&payload[offset + 6u], track->predSize);
        putU16(&payload[offset + 8u], track->trackId);
        putU16(&payload[offset + 10u], (uint16_t) (track->misses |
            ((uint16_t) track->flags << 9u)));
        putU16(&payload[offset + 12u], (uint16_t) track->velXQ7);
        putU16(&payload[offset + 14u], (uint16_t) track->velYQ7);
        putU16(&payload[offset + 16u], (uint16_t) track->areaVelQ0);
        offset = (uint16_t) (offset + OBSERVER_LINK_PLANNING_TRACK_BYTES);
    }
    *length = offset;
    return 1u;
}

uint8_t observerLinkDecodePlanningTrackFragment(const uint8_t *payload,
    uint16_t length, observerLinkPlanningTrackFragment *fragment)
{
    uint8_t expectedFragments;
    uint8_t expectedFirst;
    uint8_t expectedCount;
    uint8_t i;
    uint16_t offset;

    if ((payload == NULL) || (fragment == NULL) ||
        (length < OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES) ||
        (observerSourceIsValid(payload[0]) == 0u) || (payload[1] > 1u) ||
        ((payload[2] & OBSERVER_LINK_TRACK_SNAPSHOT_VALID) == 0u) ||
        ((payload[2] & (uint8_t) ~(OBSERVER_LINK_TRACK_SNAPSHOT_VALID |
            OBSERVER_LINK_TRACK_SNAPSHOT_TRUNCATED)) != 0u) ||
        (payload[7] > OBSERVER_LINK_MAX_TRACKS) ||
        (payload[29] > payload[28]) ||
        ((uint16_t) payload[30] + payload[31] != payload[29]))
    {
        return 0u;
    }
    expectedFragments = observerLinkPlanningTrackFragmentCount(payload[7]);
    if ((payload[4] != expectedFragments) || (payload[3] >= expectedFragments))
    {
        return 0u;
    }
    expectedFirst = (uint8_t) (payload[3] * OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT);
    expectedCount = (uint8_t) (payload[7] - expectedFirst);
    if (expectedCount > OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT)
    {
        expectedCount = OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT;
    }
    if ((payload[5] != expectedFirst) || (payload[6] != expectedCount) ||
        (length != (uint16_t) (OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES +
            expectedCount * OBSERVER_LINK_PLANNING_TRACK_BYTES)))
    {
        return 0u;
    }
    memset(fragment, 0, sizeof(*fragment));
    fragment->sourceId = payload[0];
    fragment->detectMode = payload[1];
    fragment->truncated = (uint8_t) ((payload[2] & OBSERVER_LINK_TRACK_SNAPSHOT_TRUNCATED) != 0u);
    fragment->fragmentIndex = payload[3];
    fragment->fragmentCount = payload[4];
    fragment->firstTrack = payload[5];
    fragment->trackCount = payload[6];
    fragment->retainedCount = payload[7];
    fragment->ownerSession = getU32(&payload[8]);
    fragment->revision = getU32(&payload[12]);
    fragment->hash = getU32(&payload[16]);
    fragment->appFrameSeq = getU32(&payload[20]);
    fragment->captureTickMs = getU32(&payload[24]);
    fragment->confirmedTotal = payload[28];
    fragment->freshTotal = payload[29];
    fragment->freshLeft = payload[30];
    fragment->freshRight = payload[31];
    fragment->trackerFrameSeq = getU32(&payload[32]);
    offset = OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES;
    for (i = 0u; i < expectedCount; i++)
    {
        observerLinkPlanningTrack *track = &fragment->tracks[i];
        uint8_t j;
        track->centerX = payload[offset + 0u];
        track->centerY = payload[offset + 1u];
        track->predX = payload[offset + 2u];

```



```

track->predY = payload[offset + 3u];
track->size = getU16(&payload[offset + 4u]);
track->predSize = getU16(&payload[offset + 6u]);
track->trackId = getU16(&payload[offset + 8u]);
{
    uint16_t state = getU16(&payload[offset + 10u]);
    if ((state & 0xe000u) != 0u)
    {
        return 0u;
    }
    track->misses = (uint16_t) (state & 0x01ffu);
    track->flags = (uint8_t) ((state >> 9u) & 0x0fu);
}
track->velXQ7 = (int16_t) getU16(&payload[offset + 12u]);
track->velYQ7 = (int16_t) getU16(&payload[offset + 14u]);
track->areaVelQ0 = (int16_t) getU16(&payload[offset + 16u]);
if (planningTrackIsValid(track) == 0u)
{
    return 0u;
}
for (j = 0u; j < i; j++)
{
    if (fragment->tracks[j].trackId == track->trackId)
    {
        return 0u;
    }
}
offset = (uint16_t) (offset + OBSERVER_LINK_PLANNING_TRACK_BYTES);
}
return 1u;
}

```

core1_cyt4bb7/project/code/comm/observerLinkProtocol.h

SHA-256

:

f96f51f0dcdad2d9b76945ae34e036142bae8d76aa9e0262593d15c5ed3e8695

代码 (c)

```

#ifndef OBSERVER_LINK_PROTOCOL_H
#define OBSERVER_LINK_PROTOCOL_H

#include <stdint.h>
#include <string.h>
#include "paramSyncGlobal.h"

#define OBSERVER_LINK_SOF0 (0xA5u)
#define OBSERVER_LINK_SOF1 (0x5Au)
#define OBSERVER_LINK_PROTOCOL_MAJOR (3u)
#define OBSERVER_LINK_PROTOCOL_MINOR (2u)
#define OBSERVER_LINK_UART_BAUDRATE (4 000 000u)
#define OBSERVER_LINK_TX_EPOCH_MS (10u)
#define OBSERVER_LINK_TX_FRAMES_PER_EPOCH (4u)
#define OBSERVER_LINK_TX_CONTROL_FRAMES_PER_EPOCH (2u)
#define OBSERVER_LINK_TX_DATA_CREDIT_CAP (8u)
#define OBSERVER_LINK_TX_CONTROL_CREDIT_CAP (4u)
#define OBSERVER_LINK_RX_BYTE_BUDGET (1536u)
#define OBSERVER_LINK_MAX_PAYLOAD_BYTES (128u)
#define OBSERVER_LINK_HEADER_BYTES (20u)
#define OBSERVER_LINK_CRC_BYTES (4u)
#define OBSERVER_LINK_MAX_FRAME_BYTES (OBSERVER_LINK_HEADER_BYTES + OBSERVER_LINK_MAX_PAYLOAD_BYTES + OBSERVER_LINK_CRC_BYTES)
#define OBSERVER_LINK_SHARED_CONFIG_BYTES (128u)
#define OBSERVER_LINK_EDGE_PREDICTION_HOLD_MS_MAX (10 000u)
#define OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX (93u)
#define OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX (59u)
#define OBSERVER_LINK_CONFIG_ACK_BYTES (16u)
#define OBSERVER_LINK_DIAGNOSTICS_BYTES (32u)
#define OBSERVER_LINK_KVM_CONTROL_BYTES (17u)
#define OBSERVER_LINK_KVM_EVENT_BYTES (20u)
#define OBSERVER_LINK_KVM_ACK_BYTES (24u)
#define OBSERVER_LINK_KVM_MENU_STATUS_BYTES (24u)
#define OBSERVER_LINK_VIEW_OVERLAY_BYTES (47u)
#define OBSERVER_LINK_MAX_TRACKS (32u)
#define OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES (36u)

```

```

#define OBSERVER_LINK_PLANNING_TRACK_BYTES          (18u)
#define OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT (5u)
#define OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_MAX_BYTES (OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES +
OBSERVER_LINK_PLANNING_TRACK_BYTES * OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT)
#define OBSERVER_LINK_PLANNING_TRACK_MISSES_MAX      (500u)

/* RFC1982 式 32 位序号比较：0 保留给未初始化态，跨 UINT32_MAX 时仍保持半窗单调。 */
static inline uint8_t observerLinkSequenceIsNewer(
    uint32_t value, uint32_t reference)
{
    uint32_t delta = value - reference;
    return (uint8_t) ((value != 0u) && (delta != 0u) &&
        (delta < 0x80 000 000u));
}

#define OBSERVER_LINK_RX_SESSION_REJECT (0u)
#define OBSERVER_LINK_RX_SESSION_ACCEPT (1u)
#define OBSERVER_LINK_RX_SESSION_REBIND (2u)

/* 在线时身份不可被其它 session 抢占；只有接收端已进入离线栅栏后，合格 HELLO
 * 才能用任意非零 session（含冷启动回退/同值）重建 packet 序号基线。 */
static inline uint8_t observerLinkRxSessionClassify(uint8_t online,
    uint32_t currentSession, uint32_t currentPacketSeq,
    uint8_t packetSeqPrimed, uint32_t incomingSession,
    uint32_t incomingPacketSeq, uint8_t qualifiedHello)
{
    if ((incomingSession == 0u) || (incomingPacketSeq == 0u))
        return OBSERVER_LINK_RX_SESSION_REJECT;
    if (online == 0u)
        return (qualifiedHello != 0u) ? OBSERVER_LINK_RX_SESSION_REBIND : OBSERVER_LINK_RX_SESSION_REJECT;
    if (incomingSession != currentSession)
        return OBSERVER_LINK_RX_SESSION_REJECT;
    if ((packetSeqPrimed != 0u) &&
        (observerLinkSequenceIsNewer(incomingPacketSeq,
            currentPacketSeq) == 0u))
        return OBSERVER_LINK_RX_SESSION_REJECT;
    return OBSERVER_LINK_RX_SESSION_ACCEPT;
}

#define OBSERVER_LINK_CAP_BINARY_CC      (1u << 0)
#define OBSERVER_LINK_CAP_LEGACY_CANNY   (1u << 1)
#define OBSERVER_LINK_CAP_CONFIG_STAGING (1u << 2)
#define OBSERVER_LINK_CAP_CONFIG_APPLY   (1u << 3)
#define OBSERVER_LINK_CAP_KVM            (1u << 4)
#define OBSERVER_LINK_CAP_MENU_STATUS     (1u << 5)
#define OBSERVER_LINK_CAP_PLANNING_TRACKS (1u << 6)
#define OBSERVER_LINK_CAP_CAMERA_CONTROL (1u << 7)
#define OBSERVER_LINK_CAP_CONTROL_STATUS (1u << 8)
#define OBSERVER_LINK_CAP_VIEW_OVERLAY    (1u << 9)
#define OBSERVER_LINK_CAP_PARAM_SYNC      (1u << 10)
#define OBSERVER_PARAM_HEADER_BYTES        (17u)
#define OBSERVER_PARAM_DATA_BYTES          (OBSERVER_LINK_MAX_PAYLOAD_BYTES - OBSERVER_PARAM_HEADER_BYTES)
#define OBSERVER_PARAM_TRANSPORT_BEGIN    (0x80u)
#define OBSERVER_PARAM_TRANSPORT_MORE     (0x40u)
#define OBSERVER_PARAM_STATUS_MASK        (0x3Fu)
#define OBSERVER_PARAM_OP_POLL             (0x7Fu)

typedef struct
{
    uint32_t transactionId;
    uint32_t payloadCrc32;
    uint16_t offset;
    uint16_t totalLength;
    uint8_t op;
    uint8_t slot;
    uint8_t target;
    uint8_t status;
    uint8_t length;
    uint8_t data[OBSERVER_PARAM_DATA_BYTES];
} observerParamFrame;

static inline uint8_t observerParamEncode(const observerParamFrame *frame,
    uint8_t *payload, uint16_t *length)
{
    if (frame == NULL || payload == NULL || length == NULL ||
        frame->length > OBSERVER_PARAM_DATA_BYTES) return 0u;
    payload[0] = frame->op;
    payload[1] = frame->slot;
    payload[2] = frame->target;

```

```

payload[3] = frame->status;
payload[4] = (uint8_t) frame->transactionId;
payload[5] = (uint8_t) (frame->transactionId >> 8);
payload[6] = (uint8_t) (frame->transactionId >> 16);
payload[7] = (uint8_t) (frame->transactionId >> 24);
payload[8] = (uint8_t) frame->offset;
payload[9] = (uint8_t) (frame->offset >> 8);
payload[10] = (uint8_t) frame->totalLength;
payload[11] = (uint8_t) (frame->totalLength >> 8);
payload[12] = (uint8_t) frame->payloadCrc32;
payload[13] = (uint8_t) (frame->payloadCrc32 >> 8);
payload[14] = (uint8_t) (frame->payloadCrc32 >> 16);
payload[15] = (uint8_t) (frame->payloadCrc32 >> 24);
payload[16] = frame->length;
if (frame->length != 0u) memcpy(&payload[17], frame->data, frame->length);
*length = (uint16_t) (OBSERVER_PARAM_HEADER_BYTES + frame->length);
return 1u;
}

static inline uint8_t observerParamDecode(const uint8_t *payload,
uint16_t length, observerParamFrame *frame)
{
    observerParamFrame candidate;
    if (payload == NULL || frame == NULL || length < OBSERVER_PARAM_HEADER_BYTES ||
        payload[16] > OBSERVER_PARAM_DATA_BYTES ||
        length != (uint16_t) (OBSERVER_PARAM_HEADER_BYTES + payload[16])) return 0u;
    memset(&candidate, 0, sizeof(candidate));
    candidate.op = payload[0];
    candidate.slot = payload[1];
    candidate.target = payload[2];
    candidate.status = payload[3];
    candidate.transactionId = (uint32_t) payload[4] | ((uint32_t) payload[5] << 8) |
        ((uint32_t) payload[6] << 16) | ((uint32_t) payload[7] << 24);
    candidate.offset = (uint16_t) (payload[8] | ((uint16_t) payload[9] << 8));
    candidate.totalLength = (uint16_t) (payload[10] | ((uint16_t) payload[11] << 8));
    candidate.payloadCrc32 = (uint32_t) payload[12] |
        ((uint32_t) payload[13] << 8) | ((uint32_t) payload[14] << 16) |
        ((uint32_t) payload[15] << 24);
    candidate.length = payload[16];
    if (candidate.length != 0u) memcpy(candidate.data, &payload[17], candidate.length);
    *frame = candidate;
    return 1u;
}

#define OBSERVER_PARAM_GLOBAL_REQUEST_BYTES (12u)
#define OBSERVER_PARAM_GLOBAL_STATE_BYTES (28u)
typedef char observerParamGlobalRequestSizeAssert[
    (sizeof(paramSyncGlobalRequest) == OBSERVER_PARAM_GLOBAL_REQUEST_BYTES) ? 1 : -1];
typedef char observerParamGlobalStateSizeAssert[
    (sizeof(paramSyncGlobalState) == OBSERVER_PARAM_GLOBAL_STATE_BYTES) ? 1 : -1];
static inline void observerParamGlobalPutU32(uint8_t *p, uint32_t value)
{
    p[0] = (uint8_t) value; p[1] = (uint8_t) (value >> 8);
    p[2] = (uint8_t) (value >> 16); p[3] = (uint8_t) (value >> 24);
}
static inline uint32_t observerParamGlobalGetU32(const uint8_t *p)
{
    return (uint32_t) p[0] | ((uint32_t) p[1] << 8) |
        ((uint32_t) p[2] << 16) | ((uint32_t) p[3] << 24);
}
static inline uint8_t observerParamGlobalRequestEncode(
    const paramSyncGlobalRequest *value, uint8_t *payload, uint16_t *length)
{
    if ((value == NULL) || (payload == NULL) || (length == NULL) ||
        (paramSyncGlobalRequestIsValid(value) == 0u)) return 0u;
    observerParamGlobalPutU32(&payload[0], value->producerBootGeneration);
    observerParamGlobalPutU32(&payload[4], value->sequence);
    payload[8] = value->origin; payload[9] = value->action;
    payload[10] = value->slot; payload[11] = 0u;
    *length = OBSERVER_PARAM_GLOBAL_REQUEST_BYTES;
    return 1u;
}
static inline uint8_t observerParamGlobalRequestDecode(const uint8_t *payload,
uint16_t length, paramSyncGlobalRequest *value)
{
    paramSyncGlobalRequest candidate;
    if ((payload == NULL) || (value == NULL) ||
        (length != OBSERVER_PARAM_GLOBAL_REQUEST_BYTES)) return 0u;
    candidate.producerBootGeneration = observerParamGlobalGetU32(&payload[0]);

```

```

    candidate.sequence = observerParamGlobalGetU32(&payload[4]);
    candidate.origin = payload[8]; candidate.action = payload[9];
    candidate.slot = payload[10]; candidate.reserved = payload[11];
    if (paramSyncGlobalRequestsValid(&candidate) == 0u) return 0u;
    *value = candidate;
    return 1u;
}
static inline uint8_t observerParamGlobalStateEncode(
    const paramSyncGlobalState *value, uint8_t *payload, uint16_t *length)
{
    if ((value == NULL) || (payload == NULL) || (length == NULL) ||
        paramSyncGlobalStateIsValid(value) == 0u) return 0u;
    observerParamGlobalPutU32(&payload[0], value->coordinatorBootGeneration);
    observerParamGlobalPutU32(&payload[4], value->sequence);
    observerParamGlobalPutU32(&payload[8], value->requestBootGeneration);
    observerParamGlobalPutU32(&payload[12], value->requestSequence);
    observerParamGlobalPutU32(&payload[16], value->transactionId);
    payload[20] = value->origin; payload[21] = value->action;
    payload[22] = value->slot; payload[23] = value->phase;
    payload[24] = value->target; payload[25] = value->error;
    payload[26] = value->readyMask; payload[27] = value->flags;
    *length = OBSERVER_PARAM_GLOBAL_STATE_BYTES;
    return paramSyncGlobalStateIsValid(value);
}
static inline uint8_t observerParamGlobalStateDecode(const uint8_t *payload,
    uint16_t length, paramSyncGlobalState *value)
{
    paramSyncGlobalState candidate;
    if ((payload == NULL) || (value == NULL) ||
        (length != OBSERVER_PARAM_GLOBAL_STATE_BYTES)) return 0u;
    memset(&candidate, 0, sizeof(candidate));
    candidate.coordinatorBootGeneration = observerParamGlobalGetU32(&payload[0]);
    candidate.sequence = observerParamGlobalGetU32(&payload[4]);
    candidate.requestBootGeneration = observerParamGlobalGetU32(&payload[8]);
    candidate.requestSequence = observerParamGlobalGetU32(&payload[12]);
    candidate.transactionId = observerParamGlobalGetU32(&payload[16]);
    candidate.origin = payload[20]; candidate.action = payload[21];
    candidate.slot = payload[22]; candidate.phase = payload[23];
    candidate.target = payload[24]; candidate.error = payload[25];
    candidate.readyMask = payload[26]; candidate.flags = payload[27];
    if (paramSyncGlobalStateIsValid(&candidate) == 0u) return 0u;
    *value = candidate;
    return 1u;
}

#define OBSERVER_LINK_CAMERA_CONTROL_SAFE    (0u)
#define OBSERVER_LINK_CAMERA_CONTROL_RUN     (1u)
#define OBSERVER_LINK_CORE1_HEARTBEAT_BYTES  (25u)
#define OBSERVER_LINK_HEALTH_BYTES           (24u)
#define OBSERVER_LINK_CAMERA_UNKNOWN         (0u)
#define OBSERVER_LINK_CAMERA_RUNNING         (1u)
#define OBSERVER_LINK_CAMERA_FAULT           (2u)
#define OBSERVER_LINK_HEALTH_FLAG_FRAME_FRESH (1u << 0)
#define OBSERVER_LINK_HEALTH_FLAG_BRIDGE_FRESH (1u << 1)

#define OBSERVER_LINK_SYSTEM_PHASE_INIT      (0u)
#define OBSERVER_LINK_SYSTEM_PHASE_INIT_FAULT (1u)
#define OBSERVER_LINK_SYSTEM_PHASE_LINK_WAIT (2u)
#define OBSERVER_LINK_SYSTEM_PHASE_HOME      (3u)
#define OBSERVER_LINK_SYSTEM_PHASE_REJOIN     (4u)
#define OBSERVER_LINK_SYSTEM_PHASE_STANDALONE (5u)
#define OBSERVER_LINK_SYSTEM_PHASE_RUNNING    (6u)

#define OBSERVER_LINK_SYSTEM_FLAG_LAUNCHED    (1u << 0)
#define OBSERVER_LINK_SYSTEM_FLAG_LANDING     (1u << 1)
#define OBSERVER_LINK_SYSTEM_FLAG_VEL_VALID    (1u << 2)
#define OBSERVER_LINK_SYSTEM_FLAG_TARGET_VALID (1u << 3)
#define OBSERVER_LINK_SYSTEM_FLAG_STANDALONE    (1u << 4)
#define OBSERVER_LINK_SYSTEM_FLAG_REJOIN       (1u << 5)
#define OBSERVER_LINK_SYSTEM_FLAG_LAUNCH_ELIGIBLE (1u << 6)

#define OBSERVER_LINK_REASON_NONE             (0u)
#define OBSERVER_LINK_REASON_LOCAL_INIT       (1u)
#define OBSERVER_LINK_REASON_CORE0_LINK       (2u)
#define OBSERVER_LINK_REASON_CORE1_LINK       (3u)
#define OBSERVER_LINK_REASON_CORE2_LINK       (4u)
#define OBSERVER_LINK_REASON_OBSERVER_LINK    (5u)
#define OBSERVER_LINK_REASON_PROTOCOL         (6u)
#define OBSERVER_LINK_REASON_CONFIG           (7u)

```

```

#define OBSERVER_LINK_REASON_CAMERA      (8u)
#define OBSERVER_LINK_REASON_TARGET      (9u)
#define OBSERVER_LINK_REASON_BARRIER    (10u)
#define OBSERVER_LINK_REASON_DISPATCH    (11u)
#define OBSERVER_LINK_REASON_SAFETY       (12u)
#define OBSERVER_LINK_REASON_LANDING     (13u)

#define OBSERVER_LINK_OVERLAY_FLAG_VALID  (1u << 0)
#define OBSERVER_LINK_OVERLAY_FLAG_CENTER (1u << 1)
#define OBSERVER_LINK_OVERLAY_FLAG_DEADZONE (1u << 2)
#define OBSERVER_LINK_OVERLAY_FLAG_SELECTED (1u << 3)
#define OBSERVER_LINK_OVERLAY_FLAG_POSE   (1u << 4)
#define OBSERVER_LINK_OVERLAY_SELECTED_FRESH (1u << 0)
#define OBSERVER_LINK_OVERLAY_SELECTED_COAST (1u << 1)
#define OBSERVER_LINK_OVERLAY_POSE_FRESH (1u << 0)
#define OBSERVER_LINK_OVERLAY_POSE_HOLD (1u << 1)
#define OBSERVER_LINK_OVERLAY_POSE_HEADING_VALID (1u << 2)

#define OBSERVER_LINK_CONTROL_FLAG_STATUS_VALID (1u << 0)
#define OBSERVER_LINK_CONTROL_FLAG_DECISION_VALID (1u << 1)
#define OBSERVER_LINK_CONTROL_FLAG_BARRIER_REQUIRED (1u << 2)
#define OBSERVER_LINK_CONTROL_FLAG_DISPATCH_ENABLED (1u << 3)
#define OBSERVER_LINK_CONTROL_FLAG_REMOTE_ELIGIBLE (1u << 4)
#define OBSERVER_LINK_CONTROL_FLAG_CORE0_LINK_UP (1u << 5)
#define OBSERVER_LINK_CONTROL_FLAG_CORE2_LINK_UP (1u << 6)
#define OBSERVER_LINK_CONTROL_FLAG_REQUIRED_VISION (1u << 7)

#define OBSERVER_LINK_SOURCE_NONE (0u)
#define OBSERVER_LINK_SOURCE_CORE1 (1u)
#define OBSERVER_LINK_SOURCE_CORE3 (3u)
#define OBSERVER_LINK_SOURCE_CORE4 (4u)

#define OBSERVER_LINK_CONTROL_CLASS_NONE (0u)
#define OBSERVER_LINK_CONTROL_CLASS_CORE1_TRACK (1u)
#define OBSERVER_LINK_CONTROL_CLASS_OBSERVER_TRACK (2u)
#define OBSERVER_LINK_CONTROL_CLASS_SEARCH (3u)

#define OBSERVER_LINK_TRACK_FLAG_FRESH (1u << 0)
#define OBSERVER_LINK_TRACK_FLAG_COAST (1u << 1)
#define OBSERVER_LINK_TRACK_FLAG_EDGE_GRACE (1u << 2)
#define OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED (1u << 3)
#define OBSERVER_LINK_TRACK_SNAPSHOT_VALID (1u << 0)
#define OBSERVER_LINK_TRACK_SNAPSHOT_TRUNCATED (1u << 1)

#define OBSERVER_LINK_KVM_TARGET_CORE0 (0u)
#define OBSERVER_LINK_KVM_TARGET_CORE1 (1u)
#define OBSERVER_LINK_KVM_TARGET_CORE3 (3u)
#define OBSERVER_LINK_KVM_TARGET_CORE4 (4u)
#define OBSERVER_LINK_KVM_TARGET_NONE (0xFFu)

#define OBSERVER_LINK_KVM_ACK_FLAG_HOME_READY (1u << 0)
#define OBSERVER_LINK_KVM_ACK_FLAG_ACTION_OK (1u << 1)
#define OBSERVER_LINK_KVM_ACK_FLAG_ACTION_SAVE (1u << 2)
#define OBSERVER_LINK_KVM_ACK_FLAG_DUPLICATE (1u << 3)

#define OBSERVER_LINK_KVM_MENU_FLAG_HOME_READY (1u << 0)
#define OBSERVER_LINK_KVM_MENU_FLAG_FEEDBACK_ACTIVE (1u << 1)
#define OBSERVER_LINK_KVM_MENU_FLAG_LEASE_VALID (1u << 2)
#define OBSERVER_LINK_KVM_MENU_FLAG_ACTION_OK (1u << 3)
#define OBSERVER_LINK_KVM_MENU_FLAG_ACTION_SAVE (1u << 4)

typedef enum
{
    OBSERVER_LINK_ROLE_CORE1 = 1,
    OBSERVER_LINK_ROLE_CORE3 = 3,
    OBSERVER_LINK_ROLE_CORE4 = 4
} observerLinkRole;

typedef enum
{
    OBSERVER_LINK_MSG_HELLO = 1,
    OBSERVER_LINK_MSG_HEARTBEAT = 2,
    OBSERVER_LINK_MSG_DIAGNOSTICS = 3,
    OBSERVER_LINK_MSG_CONFIG_SNAPSHOT = 4,
    OBSERVER_LINK_MSG_CONFIG_ACK = 5,
    OBSERVER_LINK_MSG_KVM_CONTROL = 6,
    OBSERVER_LINK_MSG_KVM_EVENT = 7,
    OBSERVER_LINK_MSG_KVM_ACK = 8,
    OBSERVER_LINK_MSG_KVM_MENU_STATUS = 9,

```

```

OBSERVER_LINK_MSG_PLANNING_TRACK_FRAGMENT = 10,
OBSERVER_LINK_MSG_VIEW_OVERLAY = 11,
OBSERVER_LINK_MSG_HEALTH = 12,
OBSERVER_LINK_MSG_PARAM_REQUEST = 13,
OBSERVER_LINK_MSG_PARAM_RESPONSE = 14,
OBSERVER_LINK_MSG_PARAM_GLOBAL_STATE = 15,
OBSERVER_LINK_MSG_PARAM_GLOBAL_REQUEST = 16
} observerLinkMessageType;

typedef enum
{
    OBSERVER_LINK_KVM_RESULT_NONE = 0,
    OBSERVER_LINK_KVM_RESULT_EXECUTED = 1,
    OBSERVER_LINK_KVM_RESULT_UNHANDLED = 2,
    OBSERVER_LINK_KVM_RESULT_NO_LEASE = 3,
    OBSERVER_LINK_KVM_RESULT_BAD_PAYLOAD = 4,
    OBSERVER_LINK_KVM_RESULT_WRONG_TARGET = 5,
    OBSERVER_LINK_KVM_RESULT_STALE_EPOCH = 6,
    OBSERVER_LINK_KVM_RESULT_GLOBAL_EVENT_REJECTED = 7,
    OBSERVER_LINK_KVM_RESULT_GAP = 8,
    OBSERVER_LINK_KVM_RESULT_OLD_SEQ = 9
} observerLinkKvmResult;

typedef enum
{
    OBSERVER_LINK_CONFIG_NONE = 0,
    OBSERVER_LINK_CONFIG_RECEIVED = 1,
    OBSERVER_LINK_CONFIG_STAGED = 2,
    OBSERVER_LINK_CONFIG_APPLIED = 3,
    OBSERVER_LINK_CONFIG_REJECTED = 4
} observerLinkConfigStatus;

typedef enum
{
    OBSERVER_LINK_REJECT_NONE = 0,
    OBSERVER_LINK_REJECT_BAD_PAYLOAD = 1,
    OBSERVER_LINK_REJECT_BAD_HASH = 2,
    OBSERVER_LINK_REJECT_OLD_REVISION = 3,
    OBSERVER_LINK_REJECT_REVISION_CONFLICT = 4,
    OBSERVER_LINK_REJECT_UNSUPPORTED = 5
} observerLinkRejectReason;

typedef struct
{
    uint8_t major;
    uint8_t minor;
    uint8_t type;
    uint8_t flags;
    uint16_t payloadLength;
    uint32_t bootSession;
    uint32_t packetSeq;
    uint32_t tickMs;
    uint8_t payload[OBSERVER_LINK_MAX_PAYLOAD_BYTES];
} observerLinkFrame;

typedef struct
{
    uint8_t bytes[OBSERVER_LINK_MAX_FRAME_BYTES];
    uint16_t used;
    uint16_t expected;
    uint32_t validFrames;
    uint32_t crcErrors;
    uint32_t lengthErrors;
    uint32_t versionErrors;
} observerLinkParser;

typedef struct
{
    uint32_t revision;
    uint32_t hash;
    uint8_t detectMode;
    uint8_t maxBeacons;
    uint16_t edgePredictionHoldMs;
    uint8_t sourceId;
    uint8_t profileVersion;
    uint16_t beaconSizeMin;
    uint16_t beaconSizeMax;
    float beaconEccMax;
    uint8_t beaconBrightnessMin;

```

```

uint16_t beaconMaxEndpoints;
float beaconFillMin;
float beaconHuMax;
uint16_t beaconHuAreaMin;
float trackMaxAssocDist;
float trackSizeRatioMax;
uint16_t trackConfirmHits;
uint16_t trackDropMisses;
uint16_t trackCoastMisses;
float trackPositionAlpha;
float trackSizeAlpha;
float trackAreaVelAlpha;
float trackAssocScoreMax;
float trackAssocAreaWeight;
float trackAssocAreaVelWeight;
float beaconHuTemplate[7];
uint8_t c1BorderEnterLeft;
uint8_t c1BorderEnterRight;
uint8_t c1BorderEnterTop;
uint8_t c1BorderEnterBottom;
uint8_t c1BorderExitLeft;
uint8_t c1BorderExitRight;
uint8_t c1BorderExitTop;
uint8_t c1BorderExitBottom;
uint8_t observerBorderEnterLeft;
uint8_t observerBorderEnterRight;
uint8_t observerBorderEnterTop;
uint8_t observerBorderEnterBottom;
uint8_t observerBorderExitLeft;
uint8_t observerBorderExitRight;
uint8_t observerBorderExitTop;
uint8_t observerBorderExitBottom;
uint8_t c1EdgeBandLeft;
uint8_t c1EdgeBandRight;
uint8_t c1EdgeBandTop;
uint8_t c1EdgeBandBottom;
uint8_t observerEdgeBandLeft;
uint8_t observerEdgeBandRight;
uint8_t observerEdgeBandTop;
uint8_t observerEdgeBandBottom;
} observerLinkSharedConfig;

```

```

typedef struct
{
    uint32_t ownerSession;
    uint32_t revision;
    uint32_t hash;
    uint8_t desiredMode;
    uint8_t appliedMode;
    uint8_t status;
    uint8_t reason;
} observerLinkConfigAck;

```

```

typedef struct
{
    uint32_t validFrames;
    uint32_t crcErrors;
    uint32_t lengthErrors;
    uint32_t versionErrors;
    uint32_t rxOverflowErrors;
    uint32_t rxFrameErrors;
    uint32_t rxParityErrors;
    uint32_t txDeferred;
} observerLinkDiagnostics;

```

```

typedef struct
{
    uint32_t targetSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t leaseMs;
    uint8_t target;
} observerLinkKvmControl;

```

```

typedef struct
{
    uint32_t targetSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
}

```

```

    uint32_t eventSeq;
    uint8_t target;
    uint8_t keyId;
    uint8_t keyState;
    uint8_t flags;
} observerLinkKvmEvent;

```

```

typedef struct
{
    uint32_t targetSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t eventSeq;
    uint32_t expectedSeq;
    uint8_t target;
    uint8_t result;
    uint8_t menuState;
    uint8_t flags;
} observerLinkKvmAck;

```

```

typedef struct
{
    uint32_t targetSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t lastEventSeq;
    uint32_t screenRevision;
    uint8_t target;
    uint8_t menuState;
    uint8_t flags;
    uint8_t lastAckResult;
} observerLinkKvmMenuStatus;

```

```

typedef struct
{
    uint8_t detectMode;
    uint8_t protocolCompatible;
    uint8_t algorithmsReady;
    uint8_t configCompatible;
    uint8_t cameraControlState;
    uint8_t activeSource;
    uint8_t controlClass;
    uint8_t flags;
    uint8_t navState;
    uint8_t beaconCount;
    uint16_t targetX;
    uint16_t targetY;
    uint16_t targetSize;
    int16_t errXQ4;
    int16_t errYQ4;
    uint8_t systemPhase;
    uint8_t systemFlags;
    uint8_t primaryReason;
    uint8_t desiredTarget;
    uint8_t ownerTarget;
} observerLinkControlStatus;

```

```

typedef struct
{
    uint32_t ownerSession;
    uint32_t appFrameSeq;
    uint32_t captureTickMs;
    uint32_t bridgeSeq;
    uint16_t frameAgeMs;
    uint16_t bridgeAgeMs;
    uint8_t sourceId;
    uint8_t cameraState;
    uint8_t configState;
    uint8_t flags;
} observerLinkHealth;

```

/* ownerSession 是发送端 Core1 boot session; 接收端只接受与当前 peer session 一致的 DTO。 */

```

typedef struct
{
    uint32_t overlaySeq;
    uint32_t sourceFrameSeq;
    uint32_t ownerSession;
    uint32_t configRevision;
    uint32_t configHash;

```



```

uint16_t flags;
uint16_t selectedTrackId;
uint16_t activeCenterX;
uint16_t activeCenterY;
uint16_t deadzoneEnterRadius;
uint16_t deadzoneExitRadius;
uint16_t selectedX;
uint16_t selectedY;
uint16_t selectedSize;
uint16_t poseX;
uint16_t poseY;
int16_t poseHeadingQ8;
uint8_t selectedFlags;
uint8_t poseFlags;
uint8_t detectMode;
} observerLinkViewOverlay;

typedef struct
{
    uint8_t centerX;
    uint8_t centerY;
    uint8_t predX;
    uint8_t predY;
    uint16_t size;
    uint16_t predSize;
    uint16_t trackId;
    uint16_t misses;
    uint8_t flags;
    int16_t velXQ7;
    int16_t velYQ7;
    int16_t areaVelQ0;
} observerLinkPlanningTrack;

typedef struct
{
    uint8_t valid;
    uint8_t sourceId;
    uint8_t detectMode;
    uint8_t truncated;
    uint32_t ownerSession;
    uint32_t revision;
    uint32_t hash;
    uint32_t appFrameSeq;
    uint32_t captureTickMs;
    uint32_t trackerFrameSeq;
    uint8_t confirmedTotal;
    uint8_t freshTotal;
    uint8_t freshLeft;
    uint8_t freshRight;
    uint8_t retainedCount;
    observerLinkPlanningTrack tracks[OBSERVER_LINK_MAX_TRACKS];
} observerLinkPlanningTrackSnapshot;

typedef struct
{
    uint8_t sourceId;
    uint8_t detectMode;
    uint8_t truncated;
    uint8_t fragmentIndex;
    uint8_t fragmentCount;
    uint8_t firstTrack;
    uint8_t trackCount;
    uint8_t retainedCount;
    uint32_t ownerSession;
    uint32_t revision;
    uint32_t hash;
    uint32_t appFrameSeq;
    uint32_t captureTickMs;
    uint32_t trackerFrameSeq;
    uint8_t confirmedTotal;
    uint8_t freshTotal;
    uint8_t freshLeft;
    uint8_t freshRight;
    observerLinkPlanningTrack tracks[OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT];
} observerLinkPlanningTrackFragment;

void observerLinkParserInit(observerLinkParser *parser);
uint8_t observerLinkParserFeed(observerLinkParser *parser, uint8_t byte, observerLinkFrame *frame);
uint16_t observerLinkEncodeFrame(const observerLinkFrame *frame, uint8_t *output, uint16_t capacity);

```

```
uint8_t observerLinkEncodeHello(uint8_t role, uint32_t capabilities, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeHello(const uint8_t *payload, uint16_t length, uint8_t *role, uint32_t *capabilities);
uint8_t observerLinkEncodeSharedConfig(const observerLinkSharedConfig *config, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeSharedConfig(const uint8_t *payload, uint16_t length, observerLinkSharedConfig *config);
uint32_t observerLinkSharedConfigHash(const observerLinkSharedConfig *config);
uint8_t observerLinkSharedConfigIsValid(const observerLinkSharedConfig *config);
uint8_t observerLinkEncodeConfigAck(const observerLinkConfigAck *ack, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeConfigAck(const uint8_t *payload, uint16_t length, observerLinkConfigAck *ack);
uint8_t observerLinkEncodeDiagnostics(const observerLinkDiagnostics *diagnostics, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeDiagnostics(const uint8_t *payload, uint16_t length, observerLinkDiagnostics *diagnostics);
uint8_t observerLinkEncodeKvmControl(const observerLinkKvmControl *control, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeKvmControl(const uint8_t *payload, uint16_t length, observerLinkKvmControl *control);
uint8_t observerLinkEncodeKvmEvent(const observerLinkKvmEvent *event, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeKvmEvent(const uint8_t *payload, uint16_t length, observerLinkKvmEvent *event);
uint8_t observerLinkEncodeKvmAck(const observerLinkKvmAck *ack, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeKvmAck(const uint8_t *payload, uint16_t length, observerLinkKvmAck *ack);
uint8_t observerLinkEncodeKvmMenuStatus(const observerLinkKvmMenuStatus *status, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeKvmMenuStatus(const uint8_t *payload, uint16_t length, observerLinkKvmMenuStatus *status);
uint8_t observerLinkEncodeControlStatus(const observerLinkControlStatus *status,
    uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeControlStatus(const uint8_t *payload, uint16_t length,
    observerLinkControlStatus *status);
uint8_t observerLinkEncodeHealth(const observerLinkHealth *health,
    uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeHealth(const uint8_t *payload, uint16_t length,
    observerLinkHealth *health);
uint8_t observerLinkEncodeViewOverlay(const observerLinkViewOverlay *overlay,
    uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeViewOverlay(const uint8_t *payload, uint16_t length,
    observerLinkViewOverlay *overlay);
uint8_t observerLinkPlanningTrackFragmentCount(uint8_t retainedCount);
uint8_t observerLinkEncodePlanningTrackFragment(
    const observerLinkPlanningTrackSnapshot *snapshot,
    uint8_t fragmentIndex, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodePlanningTrackFragment(const uint8_t *payload,
    uint16_t length, observerLinkPlanningTrackFragment *fragment);
uint8_t observerLinkQuantizeSigned(float value, float scale,
    float minimum, float maximum, int16_t *quantized);

#endif /* OBSERVER_LINK_PROTOCOL_H */
```

core1_cyt4bb7/project/code/comm/paramSyncCoordinatorJournal.c

SHA-256 :
70470215e854b182dbaeb40468a8c481bf912d55673078821bc35a3c57eb440

代码 (c)

```
#include "paramSyncCoordinatorJournal.h"

#include <stddef.h>
#include <string.h>

#include "crc32leee.h"
#include "paramSyncMailbox.h"
#include "zf_driver_flash.h"

#define PARAM_COORD_MAGIC      (0x314A4350u) /* PCJ1 */
#define PARAM_COORD_VERSION    (1u)
#define PARAM_COORD_RECORD_BYTES (32u)
#define PARAM_COORD_PAGE_A     (12u)
#define PARAM_COORD_PAGE_B     (13u)
#define PARAM_COORD_RECORD_WORDS (PARAM_COORD_RECORD_BYTES / 4u)
#define PARAM_COORD_RECORDS_PER_PAGE (FLASH_PAGE_SIZE / PARAM_COORD_RECORD_BYTES)
#define PARAM_COORD_TARGET_MASK (0x1Fu)

#ifndef PARAM_COORD_FLASH_ADDRESS_TYPE
#define PARAM_COORD_FLASH_ADDRESS_TYPE uint32_t
#endif

typedef struct
{
    uint32_t magic;
    uint32_t version;
    uint32_t recordBytes;
```

```

uint32_t sequence;
uint32_t transactionId;
uint8_t decision;
uint8_t targetMask;
uint8_t slot;
uint8_t reserved0;
uint32_t reserved1;
uint32_t crc;
} paramCoordinatorRecord;

typedef char paramCoordinatorRecordSizeAssert[
    (sizeof(paramCoordinatorRecord) == PARAM_COORD_RECORD_BYTES) ? 1 : -1];
typedef char paramCoordinatorPagesExistAssert[(FLASH_PAGE_NUM >= 14u) ? 1 : -1];

static paramCoordinatorRecord g_latest;
static uint8_t g_ready;
static uint8_t g_uncertain;
static uint8_t g_committedInvalid;

static const uint32_t *flashWord(uint32_t page, uint32_t word)
{
    uintptr_t address = (uintptr_t) FLASH_BASE_ADDR +
        (uintptr_t) page * FLASH_PAGE_SIZE +
        (uintptr_t) word * sizeof(uint32_t);
    return (const uint32_t *) address;
}

static uint8_t readWord(uint32_t page, uint32_t word, uint32_t *value)
{
    cy_stc_flash_blankcheck_config_t config = {0};
    cy_en_flashdrv_status_t status;
    const uint32_t *address;
    if (page >= FLASH_PAGE_NUM || word >= FLASH_PAGE_LENGTH || value == NULL)
        return 0u;
    address = flashWord(page, word);
    config.addrToBeChecked = address;
    config.numOfWordsToBeChecked = 1u;
    status = Cy_Flash_BlankCheck(NULL, &config, CY_FLASH_DRIVER_BLOCKING);
    if (status == CY_FLASH_DRV_SUCCESS)
    {
        *value = UINT32_MAX;
        return 1u;
    }
    if (status != CY_FLASH_DRV_FLASH_NOT_ERASED) return 0u;
#if CY_CORE_CM7_0 || CY_CORE_CM7_1
    {
        uintptr_t line = ((uintptr_t) address) & ~(uintptr_t) 31u;
        SCB_InvalidateDCache_by_Addr((uint32_t *) line, 32);
    }
#endif
    *value = *((const volatile uint32_t *) address);
    return 1u;
}

static uint8_t readRecord(uint32_t page, uint32_t index,
    paramCoordinatorRecord *record)
{
    uint32_t *words = (uint32_t *) (void *) record;
    uint32_t i;
    if (record == NULL || index >= PARAM_COORD_RECORDS_PER_PAGE) return 0u;
    for (i = 0u; i < PARAM_COORD_RECORD_WORDS; i++)
        if (!readWord(page, index * PARAM_COORD_RECORD_WORDS + i, &words[i]))
            return 0u;
    return 1u;
}

static uint8_t recordValid(const paramCoordinatorRecord *record)
{
    uint32_t crc;
    if (record->magic != PARAM_COORD_MAGIC ||
        record->version != PARAM_COORD_VERSION ||
        record->recordBytes != PARAM_COORD_RECORD_BYTES ||
        record->sequence == 0u || record->reserved0 != 0u ||
        record->reserved1 != 0u ||
        (record->targetMask & (uint8_t) ~PARAM_COORD_TARGET_MASK) != 0u)
        return 0u;
    if (record->decision != PARAM_SYNC_OP_NONE &&
        record->decision != PARAM_SYNC_OP_ABORT &&
        record->decision != PARAM_SYNC_OP_FINISH) return 0u;

```

```

if (record->decision == PARAM_SYNC_OP_NONE)
{
    if (record->transactionId != 0u || record->targetMask != 0u ||
        record->slot != 0u) return 0u;
}
else if (record->transactionId == UINT32_MAX ||
    record->targetMask == 0u || record->slot > 5u ||
    (record->decision == PARAM_SYNC_OP_FINISH &&
    record->targetMask != PARAM_COORD_TARGET_MASK))
{
    return 0u;
}
crc = crc32leeeCalc((const uint8_t *) record,
    (uint32_t) offsetof(paramCoordinatorRecord, crc));
return (uint8_t) (crc == record->crc);
}

static uint8_t recordBlank(uint32_t page, uint32_t index)
{
    uint32_t i;
    if (index >= PARAM_COORD_RECORDS_PER_PAGE) return 2u;
    for (i = 0u; i < PARAM_COORD_RECORD_WORDS; i++)
    {
        uint32_t value;
        if (!readWord(page, index * PARAM_COORD_RECORD_WORDS + i, &value))
            return 2u;
        if (value != UINT32_MAX) return 0u;
    }
    return 1u;
}

static uint8_t sequenceNewer(uint32_t candidate, uint32_t reference)
{
    return (uint8_t) ((int32_t) (candidate - reference) > 0);
}

static uint8_t loadLatest(uint32_t *pageOut, uint32_t *indexOut)
{
    const uint32_t pages[2] = {PARAM_COORD_PAGE_A, PARAM_COORD_PAGE_B};
    uint8_t found = 0u;
    uint8_t committedInvalid = 0u;
    uint32_t p;
    g_committedInvalid = 0u;
    memset(&g_latest, 0, sizeof(g_latest));
    *pageOut = PARAM_COORD_PAGE_A;
    *indexOut = 0u;
    for (p = 0u; p < 2u; p++)
    {
        uint32_t index;
        for (index = 0u; index < PARAM_COORD_RECORDS_PER_PAGE; index++)
        {
            paramCoordinatorRecord candidate;
            if (!readRecord(pages[p], index, &candidate)) return 0u;
            {
                uint8_t blank = recordBlank(pages[p], index);
                if (blank == 2u) return 0u;
                if (blank == 1u) continue;
            }
            if (!recordValid(&candidate))
            {
                if (candidate.magic == PARAM_COORD_MAGIC)
                    committedInvalid = 1u;
            }
            else if (!found ||
                sequenceNewer(candidate.sequence, g_latest.sequence))
            {
                g_latest = candidate;
                *pageOut = pages[p];
                found = 1u;
            }
        }
    }
    if (committedInvalid != 0u)
    {
        g_committedInvalid = 1u;
        return 0u;
    }
    {
        uint32_t index;

```

```

    for (index = 0u; index < PARAM_COORD_RECORDS_PER_PAGE; index++)
    {
        uint8_t blank = recordBlank(*pageOut, index);
        if (blank == 2u) return 0u;
        if (blank == 1u)
        {
            *indexOut = index;
            return 1u;
        }
    }
    *indexOut = PARAM_COORD_RECORDS_PER_PAGE;
}
return 1u;
}

static uint8_t programWord(uint32_t page, uint32_t word, uint32_t value)
{
    uint32_t readback;
#ifdef CY_CORE_CM7_0 || CY_CORE_CM7_1
    {
        uintptr_t start = ((uintptr_t) &value) & ~(uintptr_t) 31u;
        uintptr_t end = ((uintptr_t) &value + sizeof(value) + 31u) &
            ~(uintptr_t) 31u;
        SCB_CleanDCache_by_Addr((volatile void *) start,
            (int32_t) (end - start));
    }
#endif
    Cy_FlashWriteWork((PARAM_COORD_FLASH_ADDRESS_TYPE) (uintptr_t)
        flashWord(page, word), &value, CY_FLASH_DRIVER_BLOCKING);
    return (uint8_t) (readWord(page, word, &readback) && readback == value);
}

static uint8_t reloadLatest(void)
{
    uint32_t page;
    uint32_t index;
    if (!loadLatest(&page, &index))
    {
        g_ready = 0u;
        g_uncertain = 1u;
        return 0u;
    }
    g_ready = 1u;
    g_uncertain = 0u;
    return 1u;
}

static uint8_t reconcileRecordWrite(const paramCoordinatorRecord *expected)
{
    g_ready = 0u;
    g_uncertain = 1u;
    if (!reloadLatest()) return 0u;
    return (uint8_t) (recordValid(&g_latest) &&
        memcmp(&g_latest, expected, sizeof(*expected)) == 0);
}

static uint8_t appendRecord(paramCoordinatorRecord *record)
{
    uint32_t page, index, i, base;
    paramCoordinatorRecord verified;
    const uint32_t *words = (const uint32_t *) (const void *) record;
    if (g_uncertain != 0u)
    {
        if (!reloadLatest()) return 0u;
        return 0u;
    }
    if (!loadLatest(&page, &index))
    {
        g_ready = 0u;
        g_uncertain = 1u;
        return 0u;
    }
    g_ready = 1u;
    if (index >= PARAM_COORD_RECORDS_PER_PAGE)
    {
        page = (page == PARAM_COORD_PAGE_A) ? PARAM_COORD_PAGE_B :
            PARAM_COORD_PAGE_A;
        if (flash_check(0u, page) != 0u) flash_erase_page(0u, page);
        index = 0u;
    }

```

```

    }
    record->magic = PARAM_COORD_MAGIC;
    record->version = PARAM_COORD_VERSION;
    record->recordBytes = PARAM_COORD_RECORD_BYTES;
    record->sequence = g_latest.sequence + 1u;
    if (record->sequence == 0u) record->sequence = 1u;
    record->crc = crc32IeeeCalc((const uint8_t *) record,
        (uint32_t) offsetof(paramCoordinatorRecord, crc));
    base = index * PARAM_COORD_RECORD_WORDS;
    for (i = 1u; i < PARAM_COORD_RECORD_WORDS; i++)
        if (!programWord(page, base + i, words[i]))
            return reconcileRecordWrite(record);
    if (!programWord(page, base, words[0]))
        return reconcileRecordWrite(record);
    if (!readRecord(page, index, &verified) || !recordValid(&verified) ||
        memcmp(&verified, record, sizeof(*record)) != 0)
        return reconcileRecordWrite(record);
    g_latest = verified;
    g_ready = 1u;
    g_uncertain = 0u;
    return 1u;
}

uint8_t paramSyncCoordinatorJournalInit(void)
{
    paramCoordinatorRecord recovered;
    uint8_t recoveredValid;
    flash_init();
    if (reloadLatest()) return 1u;
    if (g_committedInvalid == 0u) return 0u;

    /* Magic is programmed last, so this branch is reserved for a durable but
     * internally inconsistent journal record, not a transient read failure.
     * Rebuild both coordinator-only pages from the newest independently valid
     * decision when one exists. If none survived, use a verified NONE record;
     * library/runtime pending state remains in its own selector, so recovery
     * observes that identity and chooses ABORT. This restores SAVE-overwrite
     * without ever treating an INVALID slot as loadable. */
    recovered = g_latest;
    recoveredValid = recordValid(&recovered);
    flash_erase_page(0u, PARAM_COORD_PAGE_A);
    flash_erase_page(0u, PARAM_COORD_PAGE_B);
    memset(&g_latest, 0, sizeof(g_latest));
    g_ready = 1u;
    g_uncertain = 0u;
    g_committedInvalid = 0u;
    if (recoveredValid == 0u) memset(&recovered, 0, sizeof(recovered));
    return appendRecord(&recovered);
}

uint8_t paramSyncCoordinatorJournalGet(paramSyncCoordinatorDecision *decision)
{
    if ((g_uncertain != 0u || g_ready == 0u) &&
        !paramSyncCoordinatorJournalInit()) return 0u;
    if (decision == NULL) return 0u;
    memset(decision, 0, sizeof(*decision));
    if (!recordValid(&g_latest) || g_latest.decision == PARAM_SYNC_OP_NONE)
        return 1u;
    decision->valid = 1u;
    decision->decision = g_latest.decision;
    decision->transactionId = g_latest.transactionId;
    decision->targetMask = g_latest.targetMask;
    decision->slot = g_latest.slot;
    return 1u;
}

uint8_t paramSyncCoordinatorJournalWrite(uint8_t decision,
    uint32_t transactionId, uint8_t targetMask, uint8_t slot)
{
    paramCoordinatorRecord record;
    if ((decision != PARAM_SYNC_OP_FINISH && decision != PARAM_SYNC_OP_ABORT) ||
        transactionId == UINT32_MAX ||
        targetMask == 0u ||
        (targetMask & (uint8_t) ~PARAM_COORD_TARGET_MASK) != 0u ||
        (decision == PARAM_SYNC_OP_FINISH &&
            targetMask != PARAM_COORD_TARGET_MASK) ||
        slot > 5u) return 0u;
    memset(&record, 0, sizeof(record));
    record.decision = decision;

```

```

        record.transactionId = transactionId;
        record.targetMask = targetMask;
        record.slot = slot;
        return appendRecord(&record);
    }

    uint8_t paramSyncCoordinatorJournalClear(void)
    {
        paramCoordinatorRecord record;
        memset(&record, 0, sizeof(record));
        return appendRecord(&record);
    }

```

core1_cyt4bb7/project/code/comm/paramSyncCoordinatorJournal.h

SHA-256

:

fde60cf5aaedc3bd999b8fae97e00a26d38b81a33526c979d98fbabfa0ea6acc

代码 (c)

```

#ifndef PARAM_SYNC_COORDINATOR_JOURNAL_H
#define PARAM_SYNC_COORDINATOR_JOURNAL_H

#include <stdint.h>

typedef struct
{
    uint32_t transactionId;
    uint8_t decision;
    uint8_t targetMask;
    uint8_t slot;
    uint8_t valid;
} paramSyncCoordinatorDecision;

uint8_t paramSyncCoordinatorJournalInit(void);
uint8_t paramSyncCoordinatorJournalGet(paramSyncCoordinatorDecision *decision);
uint8_t paramSyncCoordinatorJournalWrite(uint8_t decision,
    uint32_t transactionId, uint8_t targetMask, uint8_t slot);
uint8_t paramSyncCoordinatorJournalClear(void);

#endif

```

core1_cyt4bb7/project/code/comm/paramSyncGateway.c

SHA-256

:

cf846ae44c437f770bbcb81160c01736223a7ace50d8ac5ba8503803a24177dc

代码 (c)

```

#include "paramSyncGateway.h"

#include <string.h>

#include "chassisLink.h"
#include "coreDataTransfer.h"
#include "crossCoreLink.h"
#include "paramSyncMailbox.h"
#include "zf_driver_uart.h"

#define PARAM_GATEWAY_MAGIC    (0x4E595350u)
#define PARAM_GATEWAY_HEADER_BYTES (22u)
#define PARAM_GATEWAY_TIMEOUT_MS (5000u)
#define PARAM_GATEWAY_CAP_READ   (1u << 0)
#define PARAM_GATEWAY_CAP_LIBRARY_RUNTIME_SPLIT (1u << 2)
#define PARAM_GATEWAY_INTERNAL_PREPARE_BEGIN    (9u)
#define PARAM_GATEWAY_INTERNAL_PREPARE_CHUNK    (10u)
#define PARAM_GATEWAY_INTERNAL_PREPARE_FINISH   (11u)
#define PARAM_GATEWAY_INTERNAL_COMMIT           (12u)
#define PARAM_GATEWAY_INTERNAL_ACTIVATE         (13u)
#define PARAM_GATEWAY_INTERNAL_ABORT            (14u)

```

```

#define PARAM_GATEWAY_INTERNAL_ACK (15u)
#define PARAM_GATEWAY_INTERNAL_READ_TRANSACTION_STATUS (21u)
#define PARAM_GATEWAY_INTERNAL_TRANSACTION_STATUS (22u)
#define PARAM_GATEWAY_TARGET_MASK (0x1Fu)
#define PARAM_GATEWAY_CORE2_TARGET (2u)
#define PARAM_GATEWAY_RETRY_MS (25u)
#define PARAM_GATEWAY_RETRY_MAX (4u)
#define PARAM_GATEWAY_RECOVERY_MAGIC (0xB5C30 000u)
/* Core0/Core2 均在 600 ms 参数静默后清除普通线级事务缓存。网关每次
 * 启动至少静默 700 ms，再允许向这两个外部 owner 发首帧：这同时覆盖
 * 正常热重启、Core1 冷启动和 retained value/inverse 写裂后事务号回到 1。 */
#define PARAM_GATEWAY_EXTERNAL_CACHE_TIMEOUT_MS (600u)
#define PARAM_GATEWAY_EXTERNAL_REPLAY_FENCE_MS (700u)

#if PARAM_GATEWAY_EXTERNAL_REPLAY_FENCE_MS <= \
    PARAM_GATEWAY_EXTERNAL_CACHE_TIMEOUT_MS
#error "External replay fence must exceed every remote parameter cache timeout"
#endif

typedef enum
{
    PARAM_EXT_IDLE = 0u,
    PARAM_EXT_WAIT_BEGIN,
    PARAM_EXT_WAIT_DATA,
    PARAM_EXT_WAIT_POLL
} paramExternalState;

typedef enum
{
    PARAM_GLOBAL_NONE = 0u,
    PARAM_GLOBAL_COMMIT,
    PARAM_GLOBAL_INTENT,
    PARAM_GLOBAL_LOCK,
    PARAM_GLOBAL_VERIFY_LOCK,
    PARAM_GLOBAL_INSPECT_LIBRARY,
    PARAM_GLOBAL_PREPARE_LIBRARY,
    PARAM_GLOBAL_PREPARE_RUNTIME,
    PARAM_GLOBAL_COMMIT_RUNTIME,
    PARAM_GLOBAL_ACTIVATE,
    PARAM_GLOBAL_DECIDE,
    PARAM_GLOBAL_FINISH,
    PARAM_GLOBAL_ABORT,
    PARAM_GLOBAL_RECOVERY
} paramGlobalPhase;

static volatile uint16_t rxHead, rxTail;
static volatile uint32_t rxOverflowSequence;
static volatile uint32_t rxOverflowAcknowledged;
static uint8_t rxRing[PARAM_SYNC_GATEWAY_RING_BYTES];
static uint8_t rxEncodedFrame[PARAM_SYNC_GATEWAY_WIRE_MAX];
static uint8_t txEncodedFrame[PARAM_SYNC_GATEWAY_WIRE_MAX];
static uint16_t rxEncodedLength;
static uint8_t rxDiscardUntilDelimiter;
static uint32_t activeSession, lastActivityMs, lastPcSequence;
static uint32_t mailboxSequence, pendingMailboxSequence, gatewayBoot;
#if defined(_ICCARM_)
__no_init uint32_t paramSyncGatewayBootRetained;
__no_init uint32_t paramSyncGatewayBootInverseRetained;
__no_init uint32_t paramSyncGatewayWireTransactionRetained;
__no_init uint32_t paramSyncGatewayWireTransactionInverseRetained;
__no_init uint32_t paramSyncGatewayRecoveryRetained;
__no_init uint32_t paramSyncGatewayRecoveryInverseRetained;
__no_init uint32_t paramSyncGatewayRecoveryTransactionRetained;
__no_init uint32_t paramSyncGatewayRecoveryTransactionInverseRetained;
#else
static uint32_t paramSyncGatewayBootRetained;
static uint32_t paramSyncGatewayBootInverseRetained;
static uint32_t paramSyncGatewayWireTransactionRetained;
static uint32_t paramSyncGatewayWireTransactionInverseRetained;
static uint32_t paramSyncGatewayRecoveryRetained;
static uint32_t paramSyncGatewayRecoveryInverseRetained;
static uint32_t paramSyncGatewayRecoveryTransactionRetained;
static uint32_t paramSyncGatewayRecoveryTransactionInverseRetained;
#endif
static uint8_t requestPending;
static paramSyncGatewayFrame pendingRequest;
static paramExternalState externalState;
static chassisParamFrame externalWire;
static uint8_t externalResponse[PARAM_SYNC_GATEWAY_PAYLOAD_MAX];

```



```

static uint16_t externalRequestOffset, externalResponseLength;
static uint32_t externalLastSendMs;
static uint8_t externalRetryCount, externalSendPending;
static paramExternalState core0State;
static uartLinkFrameStruct core0Wire;
static uint8_t core0Response[PARAM_SYNC_GATEWAY_PAYLOAD_MAX];
static uint16_t core0RequestOffset, core0ResponseLength;
static uint32_t core0LastSendMs;
static uint8_t core0RetryCount, core0SendPending;
static paramSyncGatewayFrame globalRequest;
static paramGlobalPhase globalPhase;
static uint32_t globalTransactionId;
static uint8_t globalSlot, globalTarget, globalCommittedMask;
static uint8_t globalOperationFailed, globalFinishRequired;
static uint8_t globalTouchedMask, globalPreparedMask, globalTargetMask;
static uint8_t globalSlotValidMask, globalSlotEmptyMask, globalSlotInvalidMask;
static uint32_t globalSlotTransactionId;
static uint8_t globalSlotTransactionValid;
static uint8_t globalSilent, gatewayCleanupFailed, globalFailedMask;
static uint8_t recoveryTargetMask, recoveryOp;
static uint32_t recoveryTransactionId;
static uint8_t recoveryDiscoveryDone;
static uint8_t targetPreparedSlot[5];
static uint32_t targetTransactionId[5];
static uint32_t globalExpectedCrc[5];
static uint32_t globalLibraryTransactionId;
static uint8_t globalLibraryIdentitySet;
static uint32_t ownerBootGeneration[5];
static uint32_t mailboxResponseBoot[5];
static uint32_t mailboxResponseSequence[5];
static uint8_t currentInternalOp;
static uint8_t prepareIntentPending;
static paramSyncGatewayFrame deferredPrepareRequest;
static paramSyncGlobalState publishedGlobalState;
static uint32_t globalStateSequence, globalStateLastPublishMs;
static uint32_t rs485GatewayTxMs;
static uint32_t boardRequestBoot[5], boardRequestSequence[5];
static uint8_t globalAction, globalOrigin;
static uint8_t globalTerminal, globalTerminalOk, globalStateError;
static uint8_t globalStateFailedTarget;
static uint32_t globalFailureDiagnostic;
static uint32_t globalConflictRetryTransactionId;
static uint8_t globalConflictRecoveryAttempted;
static uint8_t globalConflictCleanupActive;
static uint8_t globalBoardInitiated;
static uint8_t globalTerminalPublished;
static uint8_t rs485GatewayTxPrimed;
static uint32_t globalTerminalPublishedMs;
static uint32_t externalReplayFenceStartMs;
static uint8_t externalReplayFencePrimed;
static uint8_t globalTerminalObservedByPc;

```

```

#define PARAM_GATEWAY_GLOBAL_TERMINAL_HOLD_MS (1000u)

```

```

static void finishRequest(uint8_t status, uint32_t crc,
    const uint8_t* data, uint16_t length);

```

```

static uint32_t nextGatewayBootGeneration(void)
{
    uint32_t previous = paramSyncGatewayBootRetained;
    if ((previous == 0u) ||
        (paramSyncGatewayBootInverseRetained != ~previous)) previous = 0u;
    previous++;
    if (previous == 0u) previous = 1u;
    paramSyncGatewayBootRetained = previous;
    paramSyncGatewayBootInverseRetained = ~previous;
    return previous;
}

```

```

static uint32_t nextGatewayWireTransactionId(void)
{
    uint32_t previous = paramSyncGatewayWireTransactionRetained;
    if (previous == 0u ||
        paramSyncGatewayWireTransactionInverseRetained != ~previous)
        previous = 0u;
    previous++;
    if (previous == 0u || previous == UINT32_MAX) previous = 1u;
    paramSyncGatewayWireTransactionRetained = previous;
    paramSyncGatewayWireTransactionInverseRetained = ~previous;
}

```

```

    return previous;
}

static uint8_t externalReplayFenceActive(uint32_t now)
{
    if (externalReplayFencePrimed == 0u)
    {
        externalReplayFenceStartMs = now;
        externalReplayFencePrimed = 1u;
    }
    return (uint8_t) ((uint32_t) (now - externalReplayFenceStartMs) <
        PARAM_GATEWAY_EXTERNAL_REPLAY_FENCE_MS);
}

static void discardExternalReplayResidue(void)
{
    chassisParamFrame staleCore2;
    uartLinkFrameStruct staleCore0;
    /* 首发隔离期间尚未发送本会话请求，因此此处能读到的应答只可能
    * 属于重启前会话。每拍各消费一个，防止本地接收快照在隔离结束后
    * 与重新从 1 开始的线级事务号碰撞。 */
    (void) chassisLinkTryReadParamResponse(&staleCore2);
    (void) crossCoreLinkTakeParamResponse(&staleCore0);
}

static void putU16(uint8_t* p, uint16_t v)
{
    p[0] = (uint8_t) v;
    p[1] = (uint8_t) (v >> 8);
}
static void putU32(uint8_t* p, uint32_t v)
{
    p[0] = (uint8_t) v;
    p[1] = (uint8_t) (v >> 8);
    p[2] = (uint8_t) (v >> 16);
    p[3] = (uint8_t) (v >> 24);
}
static uint16_t getU16(const uint8_t* p)
{
    return (uint16_t) (p[0] | ((uint16_t) p[1] << 8));
}
static uint32_t getU32(const uint8_t* p)
{
    return (uint32_t) p[0] | ((uint32_t) p[1] << 8) | ((uint32_t) p[2] << 16) | ((uint32_t) p[3] << 24);
}

static uint8_t sequenceAdvances(uint32_t candidate, uint32_t current)
{
    return (uint8_t) (candidate != 0u &&
        (current == 0u || (int32_t) (candidate - current) > 0));
}

static uint8_t requestShapeValid(const paramSyncGatewayFrame *frame)
{
    if (frame == NULL || frame->session == 0u || frame->sequence == 0u ||
        frame->flags != 0u) return 0u;
    switch (frame->messageType)
    {
        case PARAM_GATEWAY_MSG_HELLO:
            return (uint8_t) (frame->target == 0xFFu &&
                frame->payloadLength == 4u &&
                getU32(frame->payload) == frame->session);
        case PARAM_GATEWAY_MSG_ENUM_TARGETS:
            return (uint8_t) (frame->target == 0xFFu &&
                frame->payloadLength == 0u);
        case PARAM_GATEWAY_MSG_READ_META:
            return (uint8_t) (frame->target < 5u &&
                frame->payloadLength == 1u && frame->payload[0] >= 1u &&
                frame->payload[0] <= 5u);
        case PARAM_GATEWAY_MSG_READ_CHUNK:
            return (uint8_t) (frame->target < 5u &&
                frame->payloadLength == 5u && frame->payload[0] >= 1u &&
                frame->payload[0] <= 5u && getU16(&frame->payload[3]) != 0u);
        case PARAM_GATEWAY_MSG_READ_RUNTIME_META:
            return (uint8_t) (frame->target < 5u &&
                frame->payloadLength == 0u);
        case PARAM_GATEWAY_MSG_READ_RUNTIME_CHUNK:
            return (uint8_t) (frame->target < 5u &&
                frame->payloadLength == 4u && getU16(&frame->payload[2]) != 0u);
    }
}

```

```

        default:
            return 0u;
        }
    }

static uint8_t mailboxResponseFresh(const paramSyncMailboxResponse *response)
{
    uint8_t target;
    if (response == NULL || response->target >= 5u ||
        (response->target != 1u && response->target != 3u &&
         response->target != 4u) || response->producerBootGeneration == 0u ||
         response->sequence == 0u) return 0u;
    target = response->target;
    if (mailboxResponseBoot[target] == response->producerBootGeneration)
    {
        if (sequenceAdvances(response->sequence,
                             mailboxResponseSequence[target]) == 0u) return 0u;
    }
    mailboxResponseBoot[target] = response->producerBootGeneration;
    mailboxResponseSequence[target] = response->sequence;
    return 1u;
}

static uint32_t runtimeLoadTransactionId(
    const paramSyncGatewayFrame *request, uint8_t slot)
{
    uint32_t transactionId = gatewayBoot ^ request->session ^
        request->sequence ^ ((uint32_t) slot << 24) ^ 0x52554E54u;
    if (transactionId == 0u || transactionId == UINT32_MAX)
        transactionId = 1u;
    return transactionId;
}

uint32_t paramSyncGatewayCrc32(const uint8_t* d, uint16_t n)
{
    uint32_t c = 0xFFFFFFFFu;
    uint16_t i;
    uint8_t b;
    if (!d) return 0u;
    for (i = 0; i < n; i++)
    {
        c ^= d[i];
        for (b = 0; b < 8; b++) c = (c & 1u) ? ((c >> 1) ^ 0xEDB88320u) : (c >> 1);
    }
    return ~c;
}

static uint16_t cobsEncode(const uint8_t* i, uint16_t n, uint8_t* o, uint16_t cap)
{
    uint16_t r = 0, w = 1, ci = 0;
    uint8_t c = 1;
    if (!i || !o || cap < 2) return 0;
    while (r < n)
    {
        if (i[r] == 0)
        {
            o[ci] = c;
            c = 1;
            ci = w++;
            r++;
            if (w > cap) return 0;
        }
        else
        {
            if (w >= cap) return 0;
            o[w++] = i[r++];
            c++;
            if (c == 0xFF)
            {
                o[ci] = c;
                c = 1;
                ci = w++;
                if (w > cap) return 0;
            }
        }
    }
    o[ci] = c;
    return w;
}

```

```

static uint16_t cobsDecode(const uint8_t* i, uint16_t n, uint8_t* o, uint16_t cap)
{
    uint16_t r = 0, w = 0;
    while (r < n)
    {
        uint8_t c = i[r++], j;
        if (!c) return 0;
        for (j = 1; j < c; j++)
        {
            if (r >= n || w >= cap) return 0;
            o[w++] = i[r++];
        }
        if (c != 0xFF && r < n)
        {
            if (w >= cap) return 0;
            o[w++] = 0;
        }
    }
    return w;
}

uint16_t paramSyncGatewayEncode(const paramSyncGatewayFrame* f, uint8_t* out, uint16_t cap)
{
    uint8_t raw[PARAM_SYNC_GATEWAY_WIRE_MAX];
    uint16_t rn, en;
    if (!f || !out || cap < 2 || f->payloadLength > PARAM_SYNC_GATEWAY_PAYLOAD_MAX) return 0;
    rn = (uint16_t) (PARAM_GATEWAY_HEADER_BYTES + f->payloadLength + 4u);
    putU32(raw, PARAM_GATEWAY_MAGIC);
    raw[4] = PARAM_SYNC_GATEWAY_MAJOR;
    raw[5] = PARAM_SYNC_GATEWAY_MINOR;
    raw[6] = f->messageType;
    raw[7] = f->target;
    putU32(&raw[8], f->session);
    putU32(&raw[12], f->sequence);
    putU32(&raw[16], f->flags);
    putU16(&raw[20], f->payloadLength);
    memcpy(&raw[22], f->payload, f->payloadLength);
    putU32(&raw[22u + f->payloadLength], paramSyncGatewayCrc32(raw, (uint16_t) (22u + f->payloadLength)));
    en = cobsEncode(raw, rn, out, (uint16_t) (cap - 1u));
    if (!en || en >= cap) return 0;
    out[en++] = 0;
    return en;
}

uint8_t paramSyncGatewayDecode(const uint8_t* in, uint16_t n, paramSyncGatewayFrame* f)
{
    uint8_t raw[PARAM_SYNC_GATEWAY_WIRE_MAX];
    paramSyncGatewayFrame x;
    uint16_t rn, pn;
    if (!in || !f || !n) return 0;
    rn = cobsDecode(in, n, raw, sizeof(raw));
    if (rn < 26u) return 0;
    pn = getU16(&raw[20]);
    if (getU32(raw) != PARAM_GATEWAY_MAGIC || raw[4] != PARAM_SYNC_GATEWAY_MAJOR || raw[5] > PARAM_SYNC_GATEWAY_MINOR ||
        pn > PARAM_SYNC_GATEWAY_PAYLOAD_MAX || rn != (uint16_t) (22u + pn + 4u) ||
        getU32(&raw[22u + pn]) != paramSyncGatewayCrc32(raw, (uint16_t) (22u + pn))) return 0;
    memset(&x, 0, sizeof(x));
    x.messageType = raw[6];
    x.target = raw[7];
    x.session = getU32(&raw[8]);
    x.sequence = getU32(&raw[12]);
    x.flags = getU32(&raw[16]);
    x.payloadLength = pn;
    memcpy(x.payload, &raw[22], pn);
    *f = x;
    return 1;
}

#if defined(__ICCARM__)
__weak uint8_t paramSyncGatewayPlatformTargetMask(void)
{
    return 0u;
}
__weak void paramSyncGatewayPlatformPoll(uint32_t nowMs)
{
    (void) nowMs;
}
__weak uint8_t paramSyncGatewayPlatformTakeObserverRequest(uint8_t origin,
    paramSyncGlobalRequest *request)

```

```

{
    (void) origin;
    (void) request;
    return 0u;
}
#elif defined(PARAM_SYNC_GATEWAY_HOST_TEST)
static uint8_t paramSyncGatewayHostTargetMask = PARAM_GATEWAY_TARGET_MASK;
static paramSyncGlobalRequest paramSyncGatewayHostObserverRequest[2];
static uint8_t paramSyncGatewayHostObserverRequestReady[2];
uint8_t paramSyncGatewayPlatformTargetMask(void)
{
    return paramSyncGatewayHostTargetMask;
}
void paramSyncGatewayPlatformPoll(uint32_t nowMs)
{
    (void) nowMs;
}
uint8_t paramSyncGatewayPlatformTakeObserverRequest(uint8_t origin,
    paramSyncGlobalRequest *request)
{
    uint8_t index;
    if (request == NULL || (origin != 3u && origin != 4u)) return 0u;
    index = (uint8_t) (origin - 3u);
    if (paramSyncGatewayHostObserverRequestReady[index] == 0u) return 0u;
    *request = paramSyncGatewayHostObserverRequest[index];
    paramSyncGatewayHostObserverRequestReady[index] = 0u;
    return 1u;
}
#else
__attribute__((weak)) uint8_t paramSyncGatewayPlatformTargetMask(void)
{
    return 0u;
}
__attribute__((weak)) void paramSyncGatewayPlatformPoll(uint32_t nowMs)
{
    (void) nowMs;
}
__attribute__((weak)) uint8_t
paramSyncGatewayPlatformTakeObserverRequest(uint8_t origin,
    paramSyncGlobalRequest *request)
{
    (void) origin;
    (void) request;
    return 0u;
}
#endif

static void sendFrame(paramSyncGatewayFrame* f)
{
    uint16_t n = paramSyncGatewayEncode(f, txEncodedFrame,
        sizeof(txEncodedFrame));
    if (n) uart_write_buffer(UART_0, txEncodedFrame, n);
}
static void sendError(const paramSyncGatewayFrame* r, const char* msg)
{
    paramSyncGatewayFrame x = *r;
    uint16_t n = 0;
    x.messageType = PARAM_GATEWAY_MSG_ERROR;
    while (msg[n] && n < PARAM_SYNC_GATEWAY_PAYLOAD_MAX)
    {
        x.payload[n] = (uint8_t) msg[n];
        n++;
    }
    x.payloadLength = n;
    sendFrame(&x);
}
static uint8_t responseType(uint8_t request)
{
    if (request == PARAM_GATEWAY_MSG_READ_META) return PARAM_GATEWAY_MSG_META;
    if (request == PARAM_GATEWAY_MSG_READ_CHUNK) return PARAM_GATEWAY_MSG_CHUNK;
    if (request == PARAM_GATEWAY_MSG_READ_RUNTIME_META)
        return PARAM_GATEWAY_MSG_RUNTIME_META;
    if (request == PARAM_GATEWAY_MSG_READ_RUNTIME_CHUNK)
        return PARAM_GATEWAY_MSG_RUNTIME_CHUNK;
    if (request == PARAM_GATEWAY_INTERNAL_READ_TRANSACTION_STATUS)
        return PARAM_GATEWAY_INTERNAL_TRANSACTION_STATUS;
    return PARAM_GATEWAY_INTERNAL_ACK;
}

```

```

/* This predicate is consumed by the normal control path. It deliberately
 * describes only whether the coordinator can guarantee that no parameter
 * mutation/recovery is in flight. Flight/drive state and the current online
 * target mask are mutation-entry preconditions, not reasons to suppress the
 * normal Core1 control bridge. */
static uint8_t mutationStagingEmpty(void)
{
    return (uint8_t) (globalTouchedMask == 0u &&
        globalPreparedMask == 0u && globalCommittedMask == 0u &&
        globalFinishRequired == 0u && prepareIntentPending == 0u);
}

static uint8_t runtimeCoordinatorReady(void)
{
    return (uint8_t) (globalPhase == PARAM_GLOBAL_NONE &&
        recoveryDiscoveryDone != 0u && recoveryTargetMask == 0u &&
        gatewayCleanupFailed == 0u && mutationStagingEmpty() != 0u);
}

static uint8_t mutationMenuReady(void)
{
    return (uint8_t) (runtimeCoordinatorReady() != 0u &&
        globalTerminal == 0u && activeSession == 0u &&
        requestPending == 0u &&
        paramSyncGatewayPlatformTargetMask() == PARAM_GATEWAY_TARGET_MASK);
}

uint8_t paramSyncGatewayGlobalReady(void)
{
    return runtimeCoordinatorReady();
}

uint8_t paramSyncGatewayGetGlobalState(paramSyncGlobalState *state)
{
    if (state == NULL ||
        paramSyncGlobalStateIsValid(&publishedGlobalState) == 0u)
        return 0u;
    *state = publishedGlobalState;
    return 1u;
}

static uint8_t publicGlobalPhase(void)
{
    if (globalPhase != PARAM_GLOBAL_NONE &&
        globalAction == PARAM_SYNC_GLOBAL_ACTION_NONE)
        return PARAM_SYNC_GLOBAL_PHASE_RECOVERING;
    switch (globalPhase)
    {
        case PARAM_GLOBAL_INTENT: return PARAM_SYNC_GLOBAL_PHASE_REQUESTED;
        case PARAM_GLOBAL_LOCK: return PARAM_SYNC_GLOBAL_PHASE_LOCKING;
        case PARAM_GLOBAL_VERIFY_LOCK: return PARAM_SYNC_GLOBAL_PHASE_PREPARING;
        case PARAM_GLOBAL_INSPECT_LIBRARY: return PARAM_SYNC_GLOBAL_PHASE_PREPARING;
        case PARAM_GLOBAL_PREPARE_LIBRARY:
        case PARAM_GLOBAL_PREPARE_RUNTIME: return PARAM_SYNC_GLOBAL_PHASE_PREPARING;
        case PARAM_GLOBAL_COMMIT:
        case PARAM_GLOBAL_COMMIT_RUNTIME: return PARAM_SYNC_GLOBAL_PHASE_COMMITTING;
        case PARAM_GLOBAL_ACTIVATE: return PARAM_SYNC_GLOBAL_PHASE_APPLYING;
        case PARAM_GLOBAL_DECIDE: return PARAM_SYNC_GLOBAL_PHASE_DECIDING;
        case PARAM_GLOBAL_FINISH: return PARAM_SYNC_GLOBAL_PHASE_FINISHING;
        case PARAM_GLOBAL_ABORT: return PARAM_SYNC_GLOBAL_PHASE_ABORTING;
        case PARAM_GLOBAL_RECOVERY: return PARAM_SYNC_GLOBAL_PHASE_RECOVERING;
        default:
            if (globalTerminal != 0u)
                return globalTerminalOk != 0u ? PARAM_SYNC_GLOBAL_PHASE_SUCCEEDED :
                    PARAM_SYNC_GLOBAL_PHASE_FAILED;
            return mutationMenuReady() != 0u ? PARAM_SYNC_GLOBAL_PHASE_READY :
                PARAM_SYNC_GLOBAL_PHASE_UNINITIALIZED;
    }
}

static uint32_t packFailureDiagnostic(uint8_t phase, uint8_t op,
    uint8_t status, uint8_t detail)
{
    return (uint32_t) op | ((uint32_t) status << 8) |
        ((uint32_t) phase << 16) | ((uint32_t) detail << 24);
}

static void publishGlobalState(uint32_t nowMs, uint8_t force)
{

```

```

paramSyncGlobalState next;
memset(&next, 0, sizeof(next));
next.coordinatorBootGeneration = gatewayBoot;
next.requestBootGeneration = globalRequest.session;
next.requestSequence = globalRequest.sequence;
next.transactionId = globalTransactionId;
next.origin = globalOrigin;
next.action = globalAction;
next.slot = globalSlot;
next.phase = publicGlobalPhase();
next.target = 0xFFu;
next.error = globalStateError;
next.readyMask = paramSyncGatewayPlatformTargetMask();
if (next.phase == PARAM_SYNC_GLOBAL_PHASE_UNINITIALIZED ||
    next.phase == PARAM_SYNC_GLOBAL_PHASE_READY)
{
    next.requestBootGeneration = 0u;
    next.requestSequence = 0u;
    next.transactionId = 0u;
    next.origin = 0xFFu;
    next.action = PARAM_SYNC_GLOBAL_ACTION_NONE;
    next.slot = 0u;
    next.target = 0xFFu;
    next.error = PARAM_SYNC_GLOBAL_ERROR_NONE;
}
else if (next.phase == PARAM_SYNC_GLOBAL_PHASE_RECOVERING)
{
    next.requestBootGeneration = 0u;
    next.requestSequence = 0u;
    next.origin = 0xFFu;
    next.action = PARAM_SYNC_GLOBAL_ACTION_NONE;
    next.slot = 0u;
    next.target = 0xFFu;
    next.error = (globalStateError == PARAM_SYNC_GLOBAL_ERROR_NONE) ?
        PARAM_SYNC_GLOBAL_ERROR_NONE : PARAM_SYNC_GLOBAL_ERROR_RECOVERY;
}
else if (next.phase == PARAM_SYNC_GLOBAL_PHASE_FAILED)
{
    next.target = globalStateFailedTarget;
    next.transactionId = globalFailureDiagnostic;
}
else if (next.phase >= PARAM_SYNC_GLOBAL_PHASE_REQUESTED &&
    next.phase <= PARAM_SYNC_GLOBAL_PHASE_ABORTING)
    next.target = (globalTarget <= 4u) ? globalTarget : 0xFFu;
if (next.phase == PARAM_SYNC_GLOBAL_PHASE_READY)
    next.flags = PARAM_SYNC_GLOBAL_FLAG_READY;
else if (next.phase == PARAM_SYNC_GLOBAL_PHASE_RECOVERING)
    next.flags = PARAM_SYNC_GLOBAL_FLAG_BUSY |
        PARAM_SYNC_GLOBAL_FLAG_RECOVERY;
else if (next.phase == PARAM_SYNC_GLOBAL_PHASE_SUCCEEDED)
    next.flags = PARAM_SYNC_GLOBAL_FLAG_TERMINAL |
        PARAM_SYNC_GLOBAL_FLAG_OK;
else if (next.phase == PARAM_SYNC_GLOBAL_PHASE_FAILED)
    next.flags = PARAM_SYNC_GLOBAL_FLAG_TERMINAL;
else if (next.phase != PARAM_SYNC_GLOBAL_PHASE_UNINITIALIZED)
    next.flags = PARAM_SYNC_GLOBAL_FLAG_BUSY;
if (runtimeCoordinatorReady() != 0u)
    next.flags |= PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY;
next.sequence = publishedGlobalState.sequence;
if (force == 0u && memcmp(&next, &publishedGlobalState, sizeof(next)) == 0 &&
    (uint32_t) (nowMs - globalStateLastPublishMs) < 50u) return;
globalStateSequence++;
if (globalStateSequence == 0u) globalStateSequence = 1u;
next.sequence = globalStateSequence;
publishedGlobalState = next;
globalStateLastPublishMs = nowMs;
if (next.phase == PARAM_SYNC_GLOBAL_PHASE_SUCCEEDED ||
    next.phase == PARAM_SYNC_GLOBAL_PHASE_FAILED)
{
    if (globalTerminalPublished == 0u)
    {
        globalTerminalPublished = 1u;
        globalTerminalPublishedMs = nowMs;
    }
}
else
    globalTerminalPublished = 0u;
coreDataWriteParamGlobalState(&next);
crossCoreLinkPublishParamGlobalState(&next);

```

```

}

static uint8_t requestOp(uint8_t messageType)
{
    switch (messageType)
    {
        case PARAM_GATEWAY_MSG_READ_META: return PARAM_SYNC_OP_READ_META;
        case PARAM_GATEWAY_MSG_READ_CHUNK: return PARAM_SYNC_OP_READ_CHUNK;
        case PARAM_GATEWAY_MSG_READ_RUNTIME_META:
            return PARAM_SYNC_OP_READ_RUNTIME_META;
        case PARAM_GATEWAY_MSG_READ_RUNTIME_CHUNK:
            return PARAM_SYNC_OP_READ_RUNTIME_CHUNK;
        case PARAM_GATEWAY_INTERNAL_PREPARE_BEGIN: return PARAM_SYNC_OP_PREPARE_BEGIN;
        case PARAM_GATEWAY_INTERNAL_PREPARE_CHUNK: return PARAM_SYNC_OP_PREPARE_CHUNK;
        case PARAM_GATEWAY_INTERNAL_PREPARE_FINISH: return PARAM_SYNC_OP_PREPARE_VERIFY;
        case PARAM_GATEWAY_INTERNAL_COMMIT: return PARAM_SYNC_OP_COMMIT;
        case PARAM_GATEWAY_INTERNAL_ACTIVATE: return PARAM_SYNC_OP_ACTIVATE;
        case PARAM_GATEWAY_INTERNAL_ABORT: return PARAM_SYNC_OP_ABORT;
        default: return PARAM_SYNC_OP_NONE;
    }
}

static uint8_t routeMailbox(const paramSyncGatewayFrame* f)
{
    paramSyncMailboxRequest r;
    memset(&r, 0, sizeof(r));
    mailboxSequence++;
    if (!mailboxSequence) mailboxSequence = 1;
    r.producerBootGeneration = gatewayBoot;
    r.sequence = mailboxSequence;
    r.transactionId = f->session;
    /* offset is an internal request nonce echoed verbatim by every owner.
     * It must not reuse the PC sequence, because a reconnect may repeat the
     * same public session/sequence tuple while a retained response remains. */
    r.offset = mailboxSequence;
    r.payloadCrc32 = f->flags;
    r.dataLength = f->payloadLength;
    r.op = currentInternalOp;
    r.slot = f->payloadLength ? f->payload[0] : 0u;
    r.target = f->target;
    memcpy(r.data, f->payload, f->payloadLength);
    if (paramSyncMailboxPublishRequest(&r) == 0u) return 0u;
    pendingMailboxSequence = r.sequence;
    return 1u;
}

static uint8_t startCore2Request(const paramSyncGatewayFrame* f)
{
    uint8_t op = currentInternalOp;
    if (op == PARAM_SYNC_OP_NONE) return 0u;
    memset(&externalWire, 0, sizeof(externalWire));
    externalWire.requestId = nextGatewayWireTransactionId();
    externalWire.offset = f->payloadLength;
    externalWire.op = op;
    externalWire.reserved = 0u;
    externalWire.target = PARAM_GATEWAY_CORE2_TARGET;
    externalWire.status = CHASSIS_PARAM_TRANSPORT_BEGIN;
    externalRequestOffset = 0u;
    externalResponseLength = 0u;
    externalRetryCount = 0u;
    externalSendPending = 1u;
    externalState = PARAM_EXT_WAIT_BEGIN;
    return 1u;
}

static uint8_t startCore0Request(const paramSyncGatewayFrame* f)
{
    uint8_t op = currentInternalOp;
    if (op == PARAM_SYNC_OP_NONE) return 0u;
    memset(&core0Wire, 0, sizeof(core0Wire));
    core0Wire.paramRequestId = nextGatewayWireTransactionId();
    core0Wire.paramOffset = f->payloadLength;
    core0Wire.paramOp = op;
    core0Wire.paramReserved = 0u;
    core0Wire.paramTarget = 0u;
    core0Wire.paramStatus = UART_LINK_PARAM_TRANSPORT_BEGIN;
    core0RequestOffset = 0u;
    core0ResponseLength = 0u;
    core0RetryCount = 0u;

```



```

    core0SendPending = 1u;
    core0State = PARAM_EXT_WAIT_BEGIN;
    return 1u;
}

static uint8_t core2ReplyStatusValid(uint8_t rawStatus)
{
    uint8_t status;
    uint8_t transport;
    if (rawStatus == CHASSIS_PARAM_OWNER_RESTART) return 1u;
    status = (uint8_t) (rawStatus & CHASSIS_PARAM_STATUS_MASK);
    transport = (uint8_t) (rawStatus &
        (uint8_t) ~CHASSIS_PARAM_STATUS_MASK);
    return (uint8_t) (status <= PARAM_SYNC_STATUS_FAILED &&
        (transport == 0u || transport == CHASSIS_PARAM_TRANSPORT_MORE));
}

static uint8_t core2ReplyMatchesCurrentPhase(
    const chassisParamFrame *reply)
{
    uint8_t status;
    uint8_t more;
    if (reply == 0 || reply->requestId != externalWire.requestId ||
        reply->reserved != 0u ||
        reply->target != PARAM_GATEWAY_CORE2_TARGET ||
        core2ReplyStatusValid(reply->status) == 0u) return 0u;
    status = (uint8_t) (reply->status & CHASSIS_PARAM_STATUS_MASK);
    more = (uint8_t) (reply->status & CHASSIS_PARAM_TRANSPORT_MORE);
    if (externalState == PARAM_EXT_WAIT_BEGIN ||
        externalState == PARAM_EXT_WAIT_DATA)
    {
        return (uint8_t) (reply->status != CHASSIS_PARAM_OWNER_RESTART &&
            status != PARAM_SYNC_STATUS_OK && more == 0u &&
            reply->op == currentInternalOp &&
            reply->offset == externalWire.offset && reply->length == 0u);
    }
    if (externalState != PARAM_EXT_WAIT_POLL ||
        reply->offset != externalResponseLength) return 0u;
    if (reply->status == CHASSIS_PARAM_OWNER_RESTART)
    {
        return (uint8_t) (reply->op == CHASSIS_PARAM_OP_POLL &&
            reply->length == 0u);
    }
    if (status == PARAM_SYNC_STATUS_PENDING)
    {
        return (uint8_t) (reply->status == PARAM_SYNC_STATUS_PENDING &&
            reply->op == CHASSIS_PARAM_OP_POLL && reply->length == 0u);
    }
    if (reply->op == CHASSIS_PARAM_OP_POLL)
    {
        return (uint8_t) ((reply->status == CHASSIS_PARAM_OWNER_RESTART ||
            (status >= PARAM_SYNC_STATUS_REJECTED &&
            status <= PARAM_SYNC_STATUS_FAILED)) &&
            more == 0u && reply->length == 0u);
    }
    if (reply->op != currentInternalOp ||
        (uint32_t) externalResponseLength + reply->length >
        sizeof(externalResponse)) return 0u;
    return (uint8_t) ((more != 0u) ?
        (reply->length == CHASSIS_PARAM_DATA_BYTES) :
        (reply->length <= CHASSIS_PARAM_DATA_BYTES));
}

static uint8_t core0ReplyStatusValid(uint8_t rawStatus)
{
    uint8_t status;
    uint8_t transport;
    if (rawStatus == UART_LINK_PARAM_OWNER_RESTART) return 1u;
    status = (uint8_t) (rawStatus & UART_LINK_PARAM_STATUS_MASK);
    transport = (uint8_t) (rawStatus &
        (uint8_t) ~UART_LINK_PARAM_STATUS_MASK);
    return (uint8_t) (status <= PARAM_SYNC_STATUS_FAILED &&
        (transport == 0u || transport == UART_LINK_PARAM_TRANSPORT_MORE));
}

static uint8_t core0ReplyMatchesCurrentPhase(
    const uartLinkFrameStruct *reply)
{
    uint8_t status;

```

```

uint8_t more;
if (reply == 0 ||
    reply->paramRequestId != core0Wire.paramRequestId ||
    reply->paramReserved != 0u || reply->paramTarget != 0u ||
    core0ReplyStatusValid(reply->paramStatus) == 0u) return 0u;
status = (uint8_t) (reply->paramStatus & UART_LINK_PARAM_STATUS_MASK);
more = (uint8_t) (reply->paramStatus & UART_LINK_PARAM_TRANSPORT_MORE);
if (core0State == PARAM_EXT_WAIT_BEGIN ||
    core0State == PARAM_EXT_WAIT_DATA)
{
    return (uint8_t) (reply->paramStatus !=
        UART_LINK_PARAM_OWNER_RESTART &&
        status != PARAM_SYNC_STATUS_OK && more == 0u &&
        reply->paramOp == currentInternalOp &&
        reply->paramOffset == core0Wire.paramOffset &&
        reply->paramLength == 0u);
}
if (core0State != PARAM_EXT_WAIT_POLL ||
    reply->paramOffset != core0ResponseLength) return 0u;
if (reply->paramStatus == UART_LINK_PARAM_OWNER_RESTART)
{
    return (uint8_t) (reply->paramOp == UART_LINK_PARAM_OP_POLL &&
        reply->paramLength == 0u);
}
if (status == PARAM_SYNC_STATUS_PENDING)
{
    return (uint8_t) (reply->paramStatus == PARAM_SYNC_STATUS_PENDING &&
        reply->paramOp == UART_LINK_PARAM_OP_POLL &&
        reply->paramLength == 0u);
}
if (reply->paramOp == UART_LINK_PARAM_OP_POLL)
{
    return (uint8_t) ((reply->paramStatus ==
        UART_LINK_PARAM_OWNER_RESTART ||
        (status >= PARAM_SYNC_STATUS_REJECTED &&
        status <= PARAM_SYNC_STATUS_FAILED)) &&
        more == 0u && reply->paramLength == 0u);
}
if (reply->paramOp != currentInternalOp ||
    (uint32_t) core0ResponseLength + reply->paramLength >
    sizeof(core0Response)) return 0u;
return (uint8_t) ((more != 0u) ?
    (reply->paramLength == UART_LINK_PARAM_DATA_BYTES) :
    (reply->paramLength <= UART_LINK_PARAM_DATA_BYTES));
}

static void completeExternal(uint8_t status)
{
    externalState = PARAM_EXT_IDLE;
    externalSendPending = 0u;
    core0State = PARAM_EXT_IDLE;
    core0SendPending = 0u;
    finishRequest(status, 0u, externalResponse, externalResponseLength);
}

static uint8_t dispatchPendingRequest(void)
{
    if (pendingRequest.target == 0u)
        return startCore0Request(&pendingRequest);
    if (pendingRequest.target == PARAM_GATEWAY_CORE2_TARGET)
        return startCore2Request(&pendingRequest);
    if (pendingRequest.target == 1u || pendingRequest.target == 3u ||
        pendingRequest.target == 4u)
        return routeMailbox(&pendingRequest);
    return 0u;
}

static uint8_t startPrepareIntent(const paramSyncGatewayFrame *request)
{
    uint8_t targetMask = (uint8_t) (globalTouchedMask |
        (uint8_t) (1u << request->target));
    uint32_t transactionId = getU32(&request->payload[1]);
    deferredPrepareRequest = *request;
    pendingRequest = *request;
    pendingRequest.target = 1u;
    pendingRequest.payloadLength = 7u;
    pendingRequest.payload[0] = PARAM_SYNC_OP_ABORT;
    pendingRequest.payload[1] = request->payload[0];
    putU32(&pendingRequest.payload[2], transactionId);
}

```

```

    pendingRequest.payload[6] = targetMask;
    currentInternalOp = PARAM_SYNC_OP_DECIDE;
    if (!routeMailbox(&pendingRequest)) return 0u;
    prepareIntentPending = 1u;
    requestPending = 1u;
    return 1u;
}

static uint8_t dispatchGlobalCurrent(void)
{
    pendingRequest = globalRequest;
    pendingRequest.target = globalTarget;
    pendingRequest.flags = 0u;
    if (globalPhase == PARAM_GLOBAL_COMMIT || globalPhase == PARAM_GLOBAL_LOCK ||
        globalPhase == PARAM_GLOBAL_PREPARE_LIBRARY ||
        globalPhase == PARAM_GLOBAL_PREPARE_RUNTIME ||
        globalPhase == PARAM_GLOBAL_COMMIT_RUNTIME ||
        globalPhase == PARAM_GLOBAL_ACTIVATE)
    {
        pendingRequest.messageType =
            (globalPhase == PARAM_GLOBAL_COMMIT) ? PARAM_GATEWAY_INTERNAL_COMMIT :
            PARAM_GATEWAY_INTERNAL_ACTIVATE;
        pendingRequest.payloadLength = 5u;
        pendingRequest.payload[0] = globalSlot;
        putU32(&pendingRequest.payload[1], globalTransactionId);
        currentInternalOp =
            (globalPhase == PARAM_GLOBAL_COMMIT) ? PARAM_SYNC_OP_COMMIT :
            (globalPhase == PARAM_GLOBAL_LOCK) ? PARAM_SYNC_OP_LOCK :
            (globalPhase == PARAM_GLOBAL_PREPARE_LIBRARY) ?
                PARAM_SYNC_OP_PREPARE_LIBRARY_FROM_LIVE :
            (globalPhase == PARAM_GLOBAL_PREPARE_RUNTIME) ?
                PARAM_SYNC_OP_PREPARE_RUNTIME_FROM_SLOT :
            (globalPhase == PARAM_GLOBAL_COMMIT_RUNTIME) ?
                PARAM_SYNC_OP_COMMIT_RUNTIME : PARAM_SYNC_OP_APPLY_RUNTIME;
    }
    else if (globalPhase == PARAM_GLOBAL_INTENT)
    {
        pendingRequest.messageType = PARAM_GATEWAY_INTERNAL_ABORT;
        pendingRequest.payloadLength = 7u;
        pendingRequest.payload[0] = PARAM_SYNC_OP_ABORT;
        pendingRequest.payload[1] = globalSlot;
        putU32(&pendingRequest.payload[2], globalTransactionId);
        pendingRequest.payload[6] = globalTargetMask;
        currentInternalOp = PARAM_SYNC_OP_DECIDE;
    }
    else if (globalPhase == PARAM_GLOBAL_VERIFY_LOCK)
    {
        pendingRequest.messageType = PARAM_GATEWAY_MSG_READ_META;
        pendingRequest.payloadLength = 1u;
        pendingRequest.payload[0] = globalSlot;
        currentInternalOp = PARAM_SYNC_OP_READ_META;
    }
    else if (globalPhase == PARAM_GLOBAL_INSPECT_LIBRARY)
    {
        pendingRequest.messageType = PARAM_GATEWAY_MSG_READ_META;
        pendingRequest.payloadLength = 1u;
        pendingRequest.payload[0] = globalSlot;
        currentInternalOp = PARAM_SYNC_OP_READ_META;
    }
    else if (globalPhase == PARAM_GLOBAL_DECIDE)
    {
        pendingRequest.messageType = PARAM_GATEWAY_INTERNAL_COMMIT;
        pendingRequest.payloadLength = 7u;
        pendingRequest.payload[0] = PARAM_SYNC_OP_FINISH;
        pendingRequest.payload[1] = globalSlot;
        putU32(&pendingRequest.payload[2], globalTransactionId);
        pendingRequest.payload[6] = globalTargetMask;
        currentInternalOp = PARAM_SYNC_OP_DECIDE;
    }
    else if (globalPhase == PARAM_GLOBAL_RECOVERY)
    {
        pendingRequest.messageType = PARAM_GATEWAY_INTERNAL_ABORT;
        pendingRequest.payloadLength = 0u;
        currentInternalOp = PARAM_SYNC_OP_RECOVERY;
    }
    else
    {
        pendingRequest.messageType = PARAM_GATEWAY_INTERNAL_ABORT;
        pendingRequest.payloadLength = 4u;
    }
}

```

```

        putU32(pendingRequest.payload, globalTransactionId);
        currentInternalOp = (globalPhase == PARAM_GLOBAL_FINISH) ? PARAM_SYNC_OP_FINISH : PARAM_SYNC_OP_ABORT;
    }
    if (!dispatchPendingRequest()) return 0u;
    requestPending = 1u;
    return 1u;
}

static void sendGlobalFinal(uint8_t status, const char* error)
{
    paramSyncGatewayFrame response = globalRequest;
    uint16_t length = 0u;
    if (status == PARAM_SYNC_STATUS_OK)
    {
        response.messageType = PARAM_GATEWAY_INTERNAL_ACK;
        response.payloadLength = 0u;
    }
    else
    {
        response.messageType = PARAM_GATEWAY_MSG_ERROR;
        while (error != NULL && error[length] != '\0' &&
            length < PARAM_SYNC_GATEWAY_PAYLOAD_MAX)
        {
            response.payload[length] = (uint8_t) error[length];
            length++;
        }
        response.payloadLength = length;
    }
    if (globalSilent != 0u)
    {
        if (globalBoardInitiated == 0u)
            gatewayCleanupFailed = (uint8_t) (status != PARAM_SYNC_STATUS_OK);
        globalSilent = 0u;
    }
    else
        sendFrame(&response);
    if (globalAction != PARAM_SYNC_GLOBAL_ACTION_NONE &&
        globalRequest.session != 0u && globalRequest.sequence != 0u)
    {
        globalTerminal = 1u;
        globalTerminalOk = (uint8_t) (status == PARAM_SYNC_STATUS_OK);
        globalTerminalObservedByPc = 0u;
        if (status != PARAM_SYNC_STATUS_OK &&
            globalStateError == PARAM_SYNC_GLOBAL_ERROR_NONE)
            globalStateError = PARAM_SYNC_GLOBAL_ERROR_STORAGE;
    }
    else
    {
        globalTerminal = 0u;
        globalTerminalOk = 0u;
        globalStateError = PARAM_SYNC_GLOBAL_ERROR_NONE;
        globalStateFailedTarget = 0xFFu;
        globalTransactionId = 0u;
        globalSlot = 0u;
        globalOrigin = 0xFFu;
        globalAction = PARAM_SYNC_GLOBAL_ACTION_NONE;
        globalRequest.session = 0u;
        globalRequest.sequence = 0u;
    }
    requestPending = 0u;
    globalPhase = PARAM_GLOBAL_NONE;
    globalBoardInitiated = 0u;
}

static uint8_t recoveryDecisionValid(uint8_t op, uint32_t transactionId,
    uint8_t targetMask)
{
    return (uint8_t) ((op == PARAM_SYNC_OP_ABORT ||
        op == PARAM_SYNC_OP_FINISH) && transactionId != UINT32_MAX &&
        targetMask != 0u &&
        (targetMask & (uint8_t) ~PARAM_GATEWAY_TARGET_MASK) == 0u &&
        (op != PARAM_SYNC_OP_FINISH ||
            targetMask == PARAM_GATEWAY_TARGET_MASK));
}

static void persistRecoveryDecision(uint8_t op, uint32_t transactionId,
    uint8_t targetMask)
{
    uint32_t word;

```

```

if (op == PARAM_SYNC_OP_NONE)
{
    paramSyncGatewayRecoveryRetained = 0u;
    paramSyncGatewayRecoveryInverseRetained = UINT32_MAX;
    paramSyncGatewayRecoveryTransactionRetained = 0u;
    paramSyncGatewayRecoveryTransactionInverseRetained = UINT32_MAX;
    return;
}
if (recoveryDecisionValid(op, transactionId, targetMask) == 0u)
    return;
word = PARAM_GATEWAY_RECOVERY_MAGIC | ((uint32_t) op << 8) | targetMask;
paramSyncGatewayRecoveryRetained = 0u;
paramSyncGatewayRecoveryInverseRetained = UINT32_MAX;
paramSyncGatewayRecoveryTransactionRetained = transactionId;
paramSyncGatewayRecoveryTransactionInverseRetained = ~transactionId;
paramSyncGatewayRecoveryRetained = word;
paramSyncGatewayRecoveryInverseRetained = ~word;
}

static uint8_t loadRecoveryDecision(uint8_t *op, uint32_t *transactionId,
uint8_t *targetMask)
{
    uint32_t word = paramSyncGatewayRecoveryRetained;
    uint8_t storedOp;
    uint8_t storedMask;
    if (paramSyncGatewayRecoveryInverseRetained != ~word ||
        paramSyncGatewayRecoveryTransactionInverseRetained !=
            ~paramSyncGatewayRecoveryTransactionRetained ||
        (word & 0xFFFF0000u) != PARAM_GATEWAY_RECOVERY_MAGIC)
        return 0u;
    storedOp = (uint8_t) (word >> 8);
    storedMask = (uint8_t) word;
    if (recoveryDecisionValid(storedOp,
        paramSyncGatewayRecoveryTransactionRetained, storedMask) == 0u)
        return 0u;
    *op = storedOp;
    *transactionId = paramSyncGatewayRecoveryTransactionRetained;
    *targetMask = storedMask;
    return 1u;
}

static void rememberRecovery(uint8_t op, uint32_t transactionId,
uint8_t targetMask)
{
    if (recoveryDecisionValid(op, transactionId, targetMask) == 0u)
    {
        gatewayCleanupFailed = 1u;
        return;
    }
    recoveryOp = op;
    recoveryTransactionId = transactionId;
    recoveryTargetMask = targetMask;
    gatewayCleanupFailed = 1u;
    persistRecoveryDecision(op, transactionId, targetMask);
}

static void clearRecovery(void)
{
    recoveryOp = PARAM_SYNC_OP_NONE;
    recoveryTransactionId = 0u;
    recoveryTargetMask = 0u;
    gatewayCleanupFailed = 0u;
    persistRecoveryDecision(PARAM_SYNC_OP_NONE, 0u, 0u);
}

static uint8_t firstGlobalTarget(paramGlobalPhase phase, uint8_t mask)
{
    static const uint8_t forwardOrder[5] = {1u, 0u, 2u, 3u, 4u};
    static const uint8_t cleanupOrder[5] = {0u, 2u, 3u, 4u, 1u};
    const uint8_t *order = (phase == PARAM_GLOBAL_FINISH ||
        phase == PARAM_GLOBAL_ABORT) ? cleanupOrder : forwardOrder;
    uint8_t i;
    for (i = 0u; i < 5u; i++)
        if ((mask & (uint8_t) (1u << order[i])) != 0u) return order[i];
    return 5u;
}

static uint8_t nextGlobalTarget(paramGlobalPhase phase, uint8_t mask,
uint8_t current)

```

```

{
    static const uint8_t forwardOrder[5] = {1u, 0u, 2u, 3u, 4u};
    static const uint8_t cleanupOrder[5] = {0u, 2u, 3u, 4u, 1u};
    const uint8_t *order = (phase == PARAM_GLOBAL_FINISH ||
        phase == PARAM_GLOBAL_ABORT) ? cleanupOrder : forwardOrder;
    uint8_t i, seen = 0u;
    for (i = 0u; i < 5u; i++)
    {
        if (seen && (mask & (uint8_t) (1u << order[i])) != 0u)
            return order[i];
        if (order[i] == current) seen = 1u;
    }
    return 5u;
}

static uint8_t restartAfterConflictRecovery(void)
{
    clearRecovery();
    globalConflictCleanupActive = 0u;
    globalTransactionId = globalConflictRetryTransactionId;
    globalSlot = globalRequest.payload[0];
    globalTargetMask = PARAM_GATEWAY_TARGET_MASK;
    globalTouchedMask = 0u;
    globalPreparedMask = 0u;
    globalCommittedMask = 0u;
    globalFailedMask = 0u;
    globalOperationFailed = 0u;
    globalFinishRequired = 1u;
    globalStateError = PARAM_SYNC_GLOBAL_ERROR_NONE;
    globalStateFailedTarget = 0xFFu;
    globalFailureDiagnostic = 0u;
    globalPhase = PARAM_GLOBAL_INTENT;
    persistRecoveryDecision(PARAM_SYNC_OP_ABORT, globalTransactionId,
        globalTargetMask);
    globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
    return dispatchGlobalCurrent();
}

static void advanceGlobal(void)
{
    uint8_t next;
    if (globalPhase == PARAM_GLOBAL_DECIDE)
    {
        persistRecoveryDecision(PARAM_SYNC_OP_FINISH, globalTransactionId,
            globalTargetMask);
        globalPhase = PARAM_GLOBAL_FINISH;
        globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
        if (!dispatchGlobalCurrent())
            finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
        return;
    }
    if (globalPhase == PARAM_GLOBAL_INTENT)
    {
        persistRecoveryDecision(PARAM_SYNC_OP_ABORT, globalTransactionId,
            globalTargetMask);
        globalPhase = PARAM_GLOBAL_LOCK;
        globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
        if (!dispatchGlobalCurrent())
            finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
        return;
    }
    next = nextGlobalTarget(globalPhase, globalTargetMask, globalTarget);
    if (next < 5u)
    {
        globalTarget = next;
        /* Core1 owns the durable coordinator journal and is deliberately the
         * last cleanup participant. If any earlier FINISH failed, do not let
         * Core1 clear that journal: retain the full FINISH decision and replay
         * every participant after recovery instead. */
        if ((globalPhase == PARAM_GLOBAL_FINISH ||
            globalPhase == PARAM_GLOBAL_ABORT) && (next == 1u) &&
            (globalFailedMask != 0u))
        {
            globalFailedMask |= (uint8_t) (1u << 1);
            advanceGlobal();
            return;
        }
    }
    if (!dispatchGlobalCurrent())
        finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}

```

```

    return;
}
if (globalPhase == PARAM_GLOBAL_LOCK)
{
    if (globalAction == PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY)
        globalPhase = PARAM_GLOBAL_PREPARE_LIBRARY;
    else if (globalAction == PARAM_SYNC_GLOBAL_ACTION_LOAD_RUNTIME)
        globalPhase = PARAM_GLOBAL_VERIFY_LOCK;
    else
        globalPhase = PARAM_GLOBAL_PREPARE_RUNTIME;
    globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
    if (!dispatchGlobalCurrent())
        finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}
else if (globalPhase == PARAM_GLOBAL_VERIFY_LOCK)
{
    globalPhase = PARAM_GLOBAL_PREPARE_RUNTIME;
    globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
    if (!dispatchGlobalCurrent())
        finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}
else if (globalPhase == PARAM_GLOBAL_PREPARE_LIBRARY)
{
    globalPhase = PARAM_GLOBAL_COMMIT;
    globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
    if (!dispatchGlobalCurrent())
        finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}
else if (globalPhase == PARAM_GLOBAL_PREPARE_RUNTIME)
{
    globalPhase = PARAM_GLOBAL_COMMIT_RUNTIME;
    globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
    if (!dispatchGlobalCurrent())
        finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}
else if (globalPhase == PARAM_GLOBAL_COMMIT_RUNTIME)
{
    globalPhase = PARAM_GLOBAL_ACTIVATE;
    globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
    if (!dispatchGlobalCurrent())
        finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}
else if (globalPhase == PARAM_GLOBAL_COMMIT || globalPhase == PARAM_GLOBAL_ACTIVATE)
{
    globalPhase = PARAM_GLOBAL_DECIDE;
    globalTarget = 1u;
    if (!dispatchGlobalCurrent())
        finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}
else if (globalPhase == PARAM_GLOBAL_FINISH)
{
    if (globalFailedMask != 0u)
        rememberRecovery(PARAM_SYNC_OP_FINISH, globalTransactionId,
            globalTargetMask);
    else
        clearRecovery();
    globalCommittedMask = globalFailedMask;
    globalTouchedMask = 0u;
    globalPreparedMask = 0u;
    globalFinishRequired = 0u;
    if (globalFailedMask == 0u &&
        globalConflictCleanupActive != 0u)
    {
        if (restartAfterConflictRecovery() == 0u)
            sendGlobalFinal(PARAM_SYNC_STATUS_FAILED, "RETRY_ROUTE");
        return;
    }
    sendGlobalFinal(globalFailedMask ? PARAM_SYNC_STATUS_FAILED : PARAM_SYNC_STATUS_OK,
        globalFailedMask ? "FINISH" : NULL);
}
else if (globalPhase == PARAM_GLOBAL_ABORT)
{
    uint8_t abortFailed = (uint8_t) (globalFailedMask != 0u);
    uint8_t responseFailed;
    if (abortFailed)
        rememberRecovery(PARAM_SYNC_OP_ABORT, globalTransactionId,
            globalTargetMask);
    else
        clearRecovery();
}

```

```

globalCommittedMask &= globalFailedMask;
globalTouchedMask &= globalFailedMask;
globalPreparedMask &= globalFailedMask;
globalFinishRequired = 0u;
if (abortFailed == 0u && globalConflictCleanupActive != 0u)
{
    if (restartAfterConflictRecovery() == 0u)
        sendGlobalFinal(PARAM_SYNC_STATUS_FAILED, "RETRY_ROUTE");
    return;
}
/* A successful rollback does not turn the operation which triggered it
 * into success. Board-originated transactions are intentionally
 * silent on UART, but still need a FAILED terminal state. */
responseFailed = (uint8_t) (abortFailed || globalOperationFailed);
sendGlobalFinal(responseFailed ? PARAM_SYNC_STATUS_FAILED : PARAM_SYNC_STATUS_OK,
    abortFailed ? "ROLLBACK" : (responseFailed ? "ABORTED" : NULL));
}
else if (globalPhase == PARAM_GLOBAL_INSPECT_LIBRARY)
{
    globalTransactionId = paramSyncGlobalPackSlotSummary(
        globalSlotValidMask, globalSlotEmptyMask, globalSlotInvalidMask);
    if (globalTransactionId == 0u)
    {
        globalStateError = PARAM_SYNC_GLOBAL_ERROR_INVALID;
        sendGlobalFinal(PARAM_SYNC_STATUS_FAILED, "INSPECT");
    }
    else
        sendGlobalFinal(PARAM_SYNC_STATUS_OK, NULL);
}
else
{
    sendGlobalFinal(PARAM_SYNC_STATUS_OK, NULL);
}
}

static void finishRequest(uint8_t status, uint32_t crc,
    const uint8_t* data, uint16_t length)
{
    if (prepareIntentPending != 0u)
    {
        paramSyncGatewayFrame original = deferredPrepareRequest;
        uint8_t bit = (uint8_t) (1u << original.target);
        uint32_t transactionId = getU32(&original.payload[1]);
        prepareIntentPending = 0u;
        requestPending = 0u;
        if (status != PARAM_SYNC_STATUS_OK)
        {
            sendError(&original, "INTENT");
            return;
        }
        globalTouchedMask |= bit;
        targetPreparedSlot[original.target] = original.payload[0];
        targetTransactionId[original.target] = transactionId;
        persistRecoveryDecision(PARAM_SYNC_OP_ABORT, transactionId,
            globalTouchedMask);
        pendingRequest = original;
        currentInternalOp = PARAM_SYNC_OP_PREPARE_BEGIN;
        if (!dispatchPendingRequest())
        {
            sendError(&original, "ROUTE");
            return;
        }
        requestPending = 1u;
        return;
    }
    if (globalPhase == PARAM_GLOBAL_NONE)
    {
        paramSyncGatewayFrame response = pendingRequest;
        response.messageType = (status == PARAM_SYNC_STATUS_OK) ? responseType(pendingRequest.messageType) : PARAM_GATEWAY_MSG_ERROR;
        response.flags = crc;
        response.payloadLength = length;
        if (data != NULL && length != 0u) memcpy(response.payload, data, length);
        if (status == PARAM_SYNC_STATUS_OK && pendingRequest.target < 5u)
        {
            uint8_t bit = (uint8_t) (1u << pendingRequest.target);
            if (pendingRequest.messageType == PARAM_GATEWAY_INTERNAL_PREPARE_BEGIN &&
                pendingRequest.payloadLength == 19u)
            {
                globalTouchedMask |= bit;
            }
        }
    }
}

```



```

        targetPreparedSlot[pendingRequest.target] = pendingRequest.payload[0];
        targetTransactionId[pendingRequest.target] =
            getU32(&pendingRequest.payload[1]);
    }
    else if (pendingRequest.messageType == PARAM_GATEWAY_INTERNAL_PREPARE_FINISH)
        globalPreparedMask |= bit;
    else if (pendingRequest.messageType == PARAM_GATEWAY_INTERNAL_ABORT)
    {
        globalTouchedMask &= (uint8_t) ~bit;
        globalPreparedMask &= (uint8_t) ~bit;
        globalCommittedMask &= (uint8_t) ~bit;
        if ((globalTouchedMask | globalPreparedMask |
            globalCommittedMask) == 0u) clearRecovery();
    }
}
sendFrame(&response);
requestPending = 0u;
return;
}

if (status != PARAM_SYNC_STATUS_OK && globalPhase != PARAM_GLOBAL_NONE &&
    globalFailureDiagnostic == 0u)
{
    uint8_t detail = (uint8_t) ((data != NULL && length >= 1u) ?
        data[0] : 0u);
    globalFailureDiagnostic = packFailureDiagnostic((uint8_t) globalPhase,
        currentInternalOp, status, detail);
}
if (status == PARAM_SYNC_STATUS_FAILED &&
    globalPhase == PARAM_GLOBAL_INTENT && globalTarget == 1u &&
    currentInternalOp == PARAM_SYNC_OP_DECIDE && data != NULL &&
    length == 8u && data[0] == 2u &&
    globalConflictRecoveryAttempted == 0u)
{
    uint8_t recoveryDecision = data[1];
    uint32_t recoveryTransaction = getU32(&data[2]);
    uint8_t recoveryMask = data[6];
    uint8_t recoverySlot = data[7];
    if (recoveryDecisionValid(recoveryDecision, recoveryTransaction,
        recoveryMask) != 0u && recoverySlot <= 5u)
    {
        globalConflictRetryTransactionId = globalTransactionId;
        globalConflictRecoveryAttempted = 1u;
        globalConflictCleanupActive = 1u;
        globalOperationFailed = 1u;
        globalStateError = PARAM_SYNC_GLOBAL_ERROR_RECOVERY;
        globalStateFailedTarget = 1u;
        globalTransactionId = recoveryTransaction;
        globalSlot = recoverySlot;
        /* The durable decision is owned by Core1. An ABORT decision may
         * legitimately describe only the participants touched before the
         * original failure, so its stored mask need not include bit 1.
         * Recovery must still visit Core1 last to consume that journal;
         * otherwise cleanup appears successful and the retry encounters
         * the same decision forever. */
        globalTargetMask = (uint8_t) (recoveryMask | (1u << 1));
        globalTouchedMask = 0u;
        globalPreparedMask = 0u;
        globalCommittedMask = 0u;
        globalFailedMask = 0u;
        requestPending = 0u;
        rememberRecovery(recoveryDecision, recoveryTransaction,
            globalTargetMask);
        globalPhase = (recoveryDecision == PARAM_SYNC_OP_FINISH) ?
            PARAM_GLOBAL_FINISH : PARAM_GLOBAL_ABORT;
        globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
        if (dispatchGlobalCurrent() == 0u)
            sendGlobalFinal(PARAM_SYNC_STATUS_FAILED, "RECOVERY_ROUTE");
        return;
    }
}
requestPending = 0u;
if (status != PARAM_SYNC_STATUS_OK)
{
    globalStateFailedTarget = globalTarget;
    globalStateError = (status == PARAM_SYNC_STATUS_REJECTED) ?
        PARAM_SYNC_GLOBAL_ERROR_REJECTED : PARAM_SYNC_GLOBAL_ERROR_STORAGE;
}
if (globalPhase == PARAM_GLOBAL_RECOVERY)

```

```

{
    if (status == PARAM_SYNC_STATUS_OK && data != NULL && length == 7u &&
        (data[0] == PARAM_SYNC_OP_NONE || data[0] == PARAM_SYNC_OP_ABORT ||
         data[0] == PARAM_SYNC_OP_FINISH) &&
        (data[0] == PARAM_SYNC_OP_NONE ||
         (data[5] != 0u &&
          (data[5] & ~PARAM_GATEWAY_TARGET_MASK) == 0u &&
          data[6] <= 5u &&
          (data[0] != PARAM_SYNC_OP_FINISH ||
           data[5] == PARAM_GATEWAY_TARGET_MASK))))
    {
        if (data[0] != PARAM_SYNC_OP_NONE)
        {
            globalSlot = data[6];
            rememberRecovery(data[0], getU32(&data[1]), data[5]);
        }
        else if (recoveryTargetMask == 0u)
            clearRecovery();
        recoveryDiscoveryDone = 1u;
    }
    else
        gatewayCleanupFailed = 1u;
    globalPhase = PARAM_GLOBAL_NONE;
    globalSilent = 0u;
    return;
}
if (globalPhase == PARAM_GLOBAL_INSPECT_LIBRARY &&
    status == PARAM_SYNC_STATUS_OK)
{
    uint8_t bit = (uint8_t) (1u << globalTarget);
    uint32_t slotTransactionId;
    if (data == NULL || length != 26u || data[0] != globalTarget ||
        data[1] != globalSlot || data[2] > 2u || data[3] != 0u)
    {
        status = PARAM_SYNC_STATUS_FAILED;
        globalStateFailedTarget = globalTarget;
        globalStateError = PARAM_SYNC_GLOBAL_ERROR_INVALID;
    }
    else if (data[2] == 0u)
        globalSlotEmptyMask |= bit;
    else if (data[2] == 2u)
        globalSlotInvalidMask |= bit;
    else
    {
        slotTransactionId = getU32(&data[16]);
        if (slotTransactionId == 0u || slotTransactionId == UINT32_MAX)
            globalSlotInvalidMask |= bit;
        else if (globalSlotTransactionValid == 0u)
        {
            globalSlotTransactionId = slotTransactionId;
            globalSlotTransactionValid = 1u;
            globalSlotValidMask |= bit;
        }
        else if (slotTransactionId == globalSlotTransactionId)
            globalSlotValidMask |= bit;
        else
            globalSlotInvalidMask |= bit;
    }
}
if (globalPhase == PARAM_GLOBAL_VERIFY_LOCK && status == PARAM_SYNC_STATUS_OK)
{
    uint32_t libraryTransactionId;
    if (data == NULL || length != 26u || data[0] != globalTarget ||
        data[1] != globalSlot || data[2] != 1u || data[3] != 0u ||
        (globalOrigin == 0xFFu &&
         getU32(&data[12]) != globalExpectedCrc[globalTarget]))
    {
        status = PARAM_SYNC_STATUS_FAILED;
        globalStateFailedTarget = globalTarget;
        globalStateError = PARAM_SYNC_GLOBAL_ERROR_INCOMPLETE;
    }
    else
    {
        libraryTransactionId = getU32(&data[16]);
        if (libraryTransactionId == 0u ||
            libraryTransactionId == UINT32_MAX)
        {
            status = PARAM_SYNC_STATUS_FAILED;
            globalStateFailedTarget = globalTarget;

```

```

        globalStateError = PARAM_SYNC_GLOBAL_ERROR_INCOMPLETE;
    }
    else if (globalLibraryIdentitySet == 0u)
    {
        globalLibraryTransactionId = libraryTransactionId;
        globalLibraryIdentitySet = 1u;
    }
    else if (libraryTransactionId != globalLibraryTransactionId)
    {
        status = PARAM_SYNC_STATUS_FAILED;
        globalStateFailedTarget = globalTarget;
        globalStateError = PARAM_SYNC_GLOBAL_ERROR_INCOMPLETE;
    }
}
}
if (status != PARAM_SYNC_STATUS_OK && globalFailureDiagnostic == 0u)
{
    globalFailureDiagnostic = packFailureDiagnostic(
        (uint8_t) globalPhase, currentInternalOp, status,
        globalStateError);
}
if (status == PARAM_SYNC_STATUS_OK)
{
    if (globalPhase == PARAM_GLOBAL_COMMIT ||
        globalPhase == PARAM_GLOBAL_COMMIT_RUNTIME ||
        globalPhase == PARAM_GLOBAL_ACTIVATE)
        globalCommittedMask |= (uint8_t) (1u << globalTarget);
    else if (globalPhase == PARAM_GLOBAL_LOCK)
        globalTouchedMask |= (uint8_t) (1u << globalTarget);
    else if (globalPhase == PARAM_GLOBAL_PREPARE_LIBRARY ||
        globalPhase == PARAM_GLOBAL_PREPARE_RUNTIME)
        globalPreparedMask |= (uint8_t) (1u << globalTarget);
    else if (globalPhase == PARAM_GLOBAL_FINISH)
        globalCommittedMask &= (uint8_t) ~(1u << globalTarget);
    else if (globalPhase == PARAM_GLOBAL_ABORT)
    {
        uint8_t bit = (uint8_t) (1u << globalTarget);
        globalTouchedMask &= (uint8_t) ~bit;
        globalPreparedMask &= (uint8_t) ~bit;
        globalCommittedMask &= (uint8_t) ~bit;
    }
    advanceGlobal();
    return;
}
if (globalPhase == PARAM_GLOBAL_INSPECT_LIBRARY)
{
    if (globalFailureDiagnostic == 0u)
    {
        globalFailureDiagnostic = packFailureDiagnostic(
            (uint8_t) globalPhase, currentInternalOp, status,
            globalStateError);
    }
    globalOperationFailed = 1u;
    sendGlobalFinal(PARAM_SYNC_STATUS_FAILED, "INSPECT");
    return;
}
if (globalPhase == PARAM_GLOBAL_DECIDE)
{
    recoveryDiscoveryDone = 0u;
    clearRecovery();
    globalOperationFailed = 1u;
    sendGlobalFinal(PARAM_SYNC_STATUS_FAILED, "DECISION");
    return;
}
if (globalPhase == PARAM_GLOBAL_ABORT)
{
    globalFailedMask |= (uint8_t) (1u << globalTarget);
    advanceGlobal();
    return;
}
if (globalPhase == PARAM_GLOBAL_FINISH)
{
    globalOperationFailed = 1u;
    globalFailedMask |= (uint8_t) (1u << globalTarget);
    advanceGlobal();
    return;
}
globalOperationFailed = 1u;
if (globalPhase == PARAM_GLOBAL_ACTIVATE ||

```

```

        globalPhase == PARAM_GLOBAL_COMMIT_RUNTIME ||
        globalPhase == PARAM_GLOBAL_COMMIT)
    globalCommittedMask |= (uint8_t) (1u << globalTarget);
else if (globalPhase == PARAM_GLOBAL_LOCK ||
        globalPhase == PARAM_GLOBAL_PREPARE_LIBRARY ||
        globalPhase == PARAM_GLOBAL_PREPARE_RUNTIME)
    globalTouchedMask |= (uint8_t) (1u << globalTarget);
globalPhase = PARAM_GLOBAL_ABORT;
globalTargetMask = (uint8_t) (globalTouchedMask | globalPreparedMask |
    globalCommittedMask);
if (globalTargetMask == 0u) globalTargetMask = 0x1Fu;
persistRecoveryDecision(PARAM_SYNC_OP_ABORT, globalTransactionId,
    globalTargetMask);
globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
if (!dispatchGlobalCurrent())
    finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}

static uint8_t startGlobal(const paramSyncGatewayFrame* request)
{
    uint32_t transactionId;
    uint8_t requestedSlot;
    if (globalTerminal != 0u) return 0u;
    if (request->messageType == PARAM_GATEWAY_INTERNAL_ABORT)
    {
        uint8_t i;
        if (request->payloadLength != 4u) return 0u;
        transactionId = getU32(request->payload);
        if (recoveryTargetMask != 0u)
        {
            if (recoveryOp != PARAM_SYNC_OP_ABORT ||
                transactionId != recoveryTransactionId) return 0u;
            globalTargetMask = recoveryTargetMask;
        }
    }
    else
    {
        globalTargetMask = (uint8_t) (globalTouchedMask | globalPreparedMask |
            globalCommittedMask);
        for (i = 0u; transactionId != 0u && i < 5u; i++)
        {
            if ((globalTargetMask & (uint8_t) (1u << i)) != 0u &&
                targetTransactionId[i] != transactionId) return 0u;
        }
    }
    if (transactionId == 0u &&
        ((globalFinishRequired && globalTransactionId != 0u) ||
        (globalTouchedMask != 0u || globalPreparedMask != 0u ||
        globalCommittedMask != 0u) && globalTransactionId != 0u) ||
        (recoveryTargetMask != 0u && recoveryOp == PARAM_SYNC_OP_FINISH))
        return 0u;
    globalSlot = 0u;
    globalAction = PARAM_SYNC_GLOBAL_ACTION_NONE;
    globalOrigin = 0xFFu;
    globalPhase = PARAM_GLOBAL_ABORT;
    globalOperationFailed = 0u;
    if (transactionId == 0u) globalTargetMask = 0x1Fu;
    if (globalTargetMask == 0u)
    {
        globalRequest = *request;
        globalTransactionId = transactionId;
        sendGlobalFinal(PARAM_SYNC_STATUS_OK, NULL);
        return 1u;
    }
}
else
{
    if ((request->payloadLength != 5u && request->payloadLength != 25u) ||
        request->payload[0] < 1u ||
        request->payload[0] > 5u) return 0u;
    requestedSlot = request->payload[0];
    transactionId = getU32(&request->payload[1]);
    if (recoveryTargetMask != 0u) return 0u;
    if (request->messageType == PARAM_GATEWAY_INTERNAL_COMMIT)
    {
        uint8_t i;
        if (transactionId == 0u || transactionId == UINT32_MAX)
            return 0u;
        if (globalPreparedMask != 0x1Fu) return 0u;
        for (i = 0u; i < 5u; i++)

```

```

    {
        if (targetPreparedSlot[i] != requestedSlot ||
            targetTransactionId[i] != transactionId) return 0u;
    }
    globalCommittedMask = 0u;
    globalFinishRequired = 0u;
    globalPhase = PARAM_GLOBAL_COMMIT;
    globalAction = PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY;
    globalOrigin = 0xFFu;
    globalSlot = requestedSlot;
    globalTargetMask = 0x1Fu;
}
else if (request->messageType == PARAM_GATEWAY_INTERNAL_ACTIVATE)
{
    uint8_t i;
    if (transactionId != 0u || request->payloadLength != 25u ||
        globalFinishRequired || globalTouchedMask != 0u ||
        globalPreparedMask != 0u || globalCommittedMask != 0u)
        return 0u;
    transactionId = runtimeLoadTransactionId(request, requestedSlot);
    globalCommittedMask = 0u;
    globalFinishRequired = 1u;
    globalSlot = requestedSlot;
    globalPhase = PARAM_GLOBAL_INTENT;
    globalAction = PARAM_SYNC_GLOBAL_ACTION_LOAD_RUNTIME;
    globalOrigin = 0xFFu;
    globalLibraryIdentitySet = 0u;
    globalLibraryTransactionId = 0u;
    for (i = 0u; i < 5u; i++)
        globalExpectedCrc[i] = getU32(&request->payload[5u + i * 4u]);
    globalTargetMask = 0x1Fu;
}
else return 0u;
globalOperationFailed = 0u;
}

globalRequest = *request;
globalTransactionId = transactionId;
globalTerminal = 0u;
globalTerminalOk = 0u;
globalTerminalObservedByPc = 0u;
globalStateError = PARAM_SYNC_GLOBAL_ERROR_NONE;
globalStateFailedTarget = 0xFFu;
globalFailureDiagnostic = 0u;
globalConflictRetryTransactionId = 0u;
globalConflictRecoveryAttempted = 0u;
globalConflictCleanupActive = 0u;
globalFailedMask = 0u;
globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
if (globalPhase == PARAM_GLOBAL_ABORT)
    persistRecoveryDecision(PARAM_SYNC_OP_ABORT, globalTransactionId,
        globalTargetMask);
else if (globalPhase == PARAM_GLOBAL_LOCK ||
        globalPhase == PARAM_GLOBAL_INTENT)
    persistRecoveryDecision(PARAM_SYNC_OP_ABORT, globalTransactionId,
        globalTargetMask);
return dispatchGlobalCurrent();
}

static uint8_t startBoardGlobal(const paramSyncGlobalRequest *request)
{
    uint32_t transactionId;
    if (paramSyncGlobalRequestIsValid(request) == 0u ||
        globalTerminal != 0u ||
        mutationMenuReady() == 0u ||
        requestPending != 0u || globalPhase != PARAM_GLOBAL_NONE)
        return 0u;
    transactionId = gatewayBoot ^ request->producerBootGeneration ^
        request->sequence ^ ((uint32_t) request->origin << 28) ^
        ((uint32_t) request->action << 20) ^ 0x42524454u;
    if (transactionId == 0u || transactionId == UINT32_MAX)
        transactionId = 1u;
    memset(&globalRequest, 0, sizeof(globalRequest));
    globalRequest.messageType = (request->action ==
        PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY) ? PARAM_GATEWAY_INTERNAL_COMMIT :
        PARAM_GATEWAY_INTERNAL_ACTIVATE;
    globalRequest.target = 0xFFu;
    globalRequest.session = request->producerBootGeneration;
    globalRequest.sequence = request->sequence;
    globalRequest.payloadLength = 5u;
}

```

```

globalRequest.payload[0] = request->slot;
putU32(&globalRequest.payload[1], transactionId);
globalTransactionId = transactionId;
globalSlot = request->slot;
globalOrigin = request->origin;
globalAction = request->action;
globalTargetMask = PARAM_GATEWAY_TARGET_MASK;
globalTouchedMask = 0u;
globalPreparedMask = 0u;
globalCommittedMask = 0u;
globalFailedMask = 0u;
globalOperationFailed = 0u;
globalFinishRequired = (uint8_t) (request->action !=
    PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY);
globalLibraryIdentitySet = 0u;
globalLibraryTransactionId = 0u;
globalSlotValidMask = 0u;
globalSlotEmptyMask = 0u;
globalSlotInvalidMask = 0u;
globalSlotTransactionId = 0u;
globalSlotTransactionValid = 0u;
globalTerminal = 0u;
globalTerminalOk = 0u;
globalTerminalObservedByPc = 0u;
globalStateError = PARAM_SYNC_GLOBAL_ERROR_NONE;
globalStateFailedTarget = 0xFFu;
globalFailureDiagnostic = 0u;
globalConflictRetryTransactionId = 0u;
globalConflictRecoveryAttempted = 0u;
globalConflictCleanupActive = 0u;
globalBoardInitiated = 1u;
globalSilent = 1u;
globalPhase = (request->action == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY) ?
    PARAM_GLOBAL_INSPECT_LIBRARY : PARAM_GLOBAL_INTENT;
if (globalPhase == PARAM_GLOBAL_INTENT)
    persistRecoveryDecision(PARAM_SYNC_OP_ABORT, transactionId,
        globalTargetMask);
globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
return dispatchGlobalCurrent();
}

static uint8_t considerBoardRequest(const paramSyncGlobalRequest *request,
    uint8_t expectedOrigin)
{
    if (request == NULL || request->origin != expectedOrigin ||
        paramSyncGlobalRequestsValid(request) == 0u) return 0u;
    if (boardRequestBoot[expectedOrigin] == request->producerBootGeneration &&
        (int32_t) (request->sequence -
            boardRequestSequence[expectedOrigin]) <= 0) return 0u;
    /* READY 边沿与各板按键不是同一总线原子事件：另一请求可能已经让协调器
    * 进入 BUSY，但并发按键仍在共享区/桥内排队。此类请求必须在当前事务期间
    * 消费并记入去重身份；发起端看到同 coordinator 的非 READY 状态后本地形成
    * BUSY 终态。若保留到当前事务结束，它会在无人再次按键时自动执行。 */
    if (globalTerminal != 0u || globalPhase != PARAM_GLOBAL_NONE ||
        requestPending != 0u)
    {
        boardRequestBoot[expectedOrigin] = request->producerBootGeneration;
        boardRequestSequence[expectedOrigin] = request->sequence;
        return 1u;
    }
    if (startBoardGlobal(request) != 0u)
    {
        boardRequestBoot[expectedOrigin] = request->producerBootGeneration;
        boardRequestSequence[expectedOrigin] = request->sequence;
    }
    else
    {
        boardRequestBoot[expectedOrigin] = request->producerBootGeneration;
        boardRequestSequence[expectedOrigin] = request->sequence;
        globalRequest.session = request->producerBootGeneration;
        globalRequest.sequence = request->sequence;
        globalOrigin = request->origin;
        globalAction = request->action;
        globalSlot = request->slot;
        globalTransactionId = 0u;
        globalTerminal = 1u;
        globalTerminalOk = 0u;
        globalTerminalObservedByPc = 0u;
        globalStateError = (paramSyncGatewayPlatformTargetMask() !=

```

```

        PARAM_GATEWAY_TARGET_MASK) ? PARAM_SYNC_GLOBAL_ERROR_INCOMPLETE :
        ((recoveryDiscoveryDone == 0u || recoveryTargetMask != 0u ||
        gatewayCleanupFailed != 0u) ?
        PARAM_SYNC_GLOBAL_ERROR_RECOVERY :
        PARAM_SYNC_GLOBAL_ERROR_BUSY);
        globalStateFailedTarget = request->origin;
    }
    return 1u;
}

static void pollBoardRequests(void)
{
    paramSyncGlobalRequest request;
    if (crossCoreLinkTakeParamGlobalRequest(&request) != 0u)
        if (considerBoardRequest(&request, 0u) != 0u) return;
    if (coreDataTryReadParamGlobalRequest(&request) != 0u)
        if (considerBoardRequest(&request, 1u) != 0u) return;
    if (chassisLinkTryReadParamGlobalRequest(&request) != 0u)
        if (considerBoardRequest(&request, 2u) != 0u) return;
    if (paramSyncGatewayPlatformTakeObserverRequest(3u, &request) != 0u)
        if (considerBoardRequest(&request, 3u) != 0u) return;
    if (paramSyncGatewayPlatformTakeObserverRequest(4u, &request) != 0u)
        (void) considerBoardRequest(&request, 4u);
}

static void invalidateSession(void)
{
    uint32_t transactionId =
        (globalPhase != PARAM_GLOBAL_NONE) ? globalTransactionId : 0u;
    uint32_t invalidatedSession = activeSession;
    uint8_t targetMask = (uint8_t) (globalTouchedMask | globalPreparedMask |
        globalCommittedMask);

    uint8_t i;
    uint8_t zeroActivationRollback = (uint8_t) ((globalPhase == PARAM_GLOBAL_LOCK ||
        globalPhase == PARAM_GLOBAL_VERIFY_LOCK ||
        globalPhase == PARAM_GLOBAL_ACTIVATE ||
        globalPhase == PARAM_GLOBAL_FINISH) &&
        globalTransactionId == 0u);

    if (requestPending && pendingRequest.target < 5u &&
        (globalPhase == PARAM_GLOBAL_LOCK ||
        globalPhase == PARAM_GLOBAL_PREPARE_LIBRARY ||
        globalPhase == PARAM_GLOBAL_PREPARE_RUNTIME ||
        globalPhase == PARAM_GLOBAL_COMMIT ||
        globalPhase == PARAM_GLOBAL_COMMIT_RUNTIME ||
        globalPhase == PARAM_GLOBAL_ACTIVATE))
    {
        targetMask |= (uint8_t) (1u << pendingRequest.target);
    }

    if (requestPending && pendingRequest.target < 5u &&
        pendingRequest.messageType == PARAM_GATEWAY_INTERNAL_PREPARE_BEGIN &&
        pendingRequest.payloadLength == 19u)
    {
        uint8_t bit = (uint8_t) (1u << pendingRequest.target);
        targetMask |= bit;
        globalTouchedMask |= bit;
        targetPreparedSlot[pendingRequest.target] = pendingRequest.payload[0];
        targetTransactionId[pendingRequest.target] =
            getU32(&pendingRequest.payload[1]);
    }

    if (requestPending && pendingRequest.target < 5u &&
        pendingRequest.messageType == PARAM_GATEWAY_INTERNAL_ACTIVATE &&
        pendingRequest.payloadLength == 5u &&
        getU32(&pendingRequest.payload[1]) == 0u &&
        (currentInternalOp == PARAM_SYNC_OP_LOCK ||
        currentInternalOp == PARAM_SYNC_OP_ACTIVATE))
    {
        targetMask |= (uint8_t) (1u << pendingRequest.target);
        zeroActivationRollback = 1u;
    }

    if (transactionId == 0u && zeroActivationRollback == 0u)
    {
        for (i = 0u; i < 5u; i++)
        {
            if ((targetMask & (uint8_t) (1u << i)) != 0u &&
                targetTransactionId[i] != 0u)
            {
                transactionId = targetTransactionId[i];
                break;
            }
        }
    }
}

```

```

    }
}
activeSession = 0u;
lastPcSequence = 0u;
rxEncodedLength = 0u;
requestPending = 0u;
prepareIntentPending = 0u;
externalState = PARAM_EXT_IDLE;
externalSendPending = 0u;
core0State = PARAM_EXT_IDLE;
core0SendPending = 0u;
if (globalPhase == PARAM_GLOBAL_INSPECT_LIBRARY)
{
    globalStateFailedTarget = globalTarget;
    globalStateError = PARAM_SYNC_GLOBAL_ERROR_PEER_RESTART;
    globalFailureDiagnostic = packFailureDiagnostic(
        (uint8_t) globalPhase, currentInternalOp,
        PARAM_SYNC_STATUS_FAILED, globalStateError);
    globalOperationFailed = 1u;
    sendGlobalFinal(PARAM_SYNC_STATUS_FAILED, "INSPECT");
    return;
}
if (globalPhase == PARAM_GLOBAL_DECIDE ||
    currentInternalOp == PARAM_SYNC_OP_DECIDE)
{
    recoveryDiscoveryDone = 0u;
    clearRecovery();
    globalPhase = PARAM_GLOBAL_NONE;
    return;
}
if (recoveryTargetMask != 0u)
{
    globalPhase = PARAM_GLOBAL_NONE;
    gatewayCleanupFailed = 1u;
    return;
}
if (targetMask == 0u ||
    (transactionId == 0u && zeroActivationRollback == 0u))
{
    globalPhase = PARAM_GLOBAL_NONE;
    return;
}
/* Core1 has already persisted FINISH before cleanup starts. From this
 * point the decision is irreversible: a session timeout, owner reboot or
 * other invalidation may defer cleanup, but must never downgrade the
 * remaining participants to ABORT. Replaying FINISH for the full mask is
 * idempotent and also covers a response that was lost after completion. */
if (globalPhase == PARAM_GLOBAL_FINISH)
{
    rememberRecovery(PARAM_SYNC_OP_FINISH, transactionId, targetMask);
    globalPhase = PARAM_GLOBAL_NONE;
    return;
}
memset(&globalRequest, 0, sizeof(globalRequest));
globalRequest.messageType = PARAM_GATEWAY_INTERNAL_ABORT;
globalRequest.target = 0xFFu;
globalRequest.session = invalidatedSession;
globalRequest.payloadLength = 4u;
putU32(globalRequest.payload, transactionId);
globalTransactionId = transactionId;
globalTargetMask = targetMask;
globalOperationFailed = 1u;
globalFailedMask = 0u;
globalPhase = PARAM_GLOBAL_ABORT;
persistRecoveryDecision(PARAM_SYNC_OP_ABORT, transactionId, targetMask);
globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
if (!dispatchGlobalCurrent())
    finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}

static void startSilentCleanup(const paramSyncGatewayFrame* hello)
{
    globalRequest = *hello;
    globalRequest.messageType = PARAM_GATEWAY_INTERNAL_ABORT;
    globalRequest.target = 0xFFu;
    globalRequest.payloadLength = 4u;
    putU32(globalRequest.payload, recoveryTargetMask ? recoveryTransactionId : 0u);
    globalTransactionId = recoveryTargetMask ? recoveryTransactionId : 0u;
    globalTargetMask = recoveryTargetMask ? recoveryTargetMask : 0x1Fu;
}

```



```

    globalCommittedMask = 0u;
    globalOperationFailed = 0u;
    globalFailedMask = 0u;
    globalFinishRequired = 0u;
    globalPhase = (recoveryTargetMask && recoveryOp == PARAM_SYNC_OP_FINISH) ?
        PARAM_GLOBAL_FINISH : PARAM_GLOBAL_ABORT;
    globalSilent = 1u;
    globalTarget = firstGlobalTarget(globalPhase, globalTargetMask);
    if (!dispatchGlobalCurrent())
        finishRequest(PARAM_SYNC_STATUS_FAILED, 0u, NULL, 0u);
}

static void startRecoveryDiscovery(const paramSyncGatewayFrame *hello)
{
    globalRequest = *hello;
    globalTargetMask = (uint8_t) (1u << 1);
    globalTarget = 1u;
    globalPhase = PARAM_GLOBAL_RECOVERY;
    globalSilent = 1u;
    if (!dispatchGlobalCurrent())
    {
        globalPhase = PARAM_GLOBAL_NONE;
        globalSilent = 0u;
        gatewayCleanupFailed = 1u;
    }
}

static void handleFrame(const paramSyncGatewayFrame* f, uint32_t now)
{
    paramSyncGatewayFrame x = *f;
    if (requestShapeValid(f) == 0u)
    {
        sendError(f, "FRAME");
        return;
    }
    if (f->messageType == PARAM_GATEWAY_MSG_HELLO)
    {
        uint8_t mask = paramSyncGatewayPlatformTargetMask();
        uint8_t safe = (uint8_t) (f->session != 0u &&
            mask == PARAM_GATEWAY_TARGET_MASK);
        uint8_t startCleanup = 0u;
        uint8_t startDiscovery = 0u;
        if (activeSession == f->session &&
            sequenceAdvances(f->sequence, lastPcSequence) == 0u)
        {
            sendError(f, "SEQUENCE");
            return;
        }
        if (safe)
        {
            uint8_t newSession = (uint8_t) (activeSession != f->session);
            if (newSession && globalPhase == PARAM_GLOBAL_NONE &&
                !requestPending && globalTerminal == 0u &&
                mutationStagingEmpty() != 0u)
            {
                activeSession = f->session;
                requestPending = 0u;
            }
            else if (newSession)
                safe = 0u;
            if (!newSession || activeSession == f->session) lastActivityMs = now;
            if (activeSession == f->session && globalPhase == PARAM_GLOBAL_NONE &&
                !requestPending && globalTerminal == 0u &&
                mutationStagingEmpty() != 0u)
            {
                if (!recoveryDiscoveryDone)
                {
                    safe = 0u;
                    startDiscovery = 1u;
                }
                else if (recoveryTargetMask != 0u || gatewayCleanupFailed != 0u)
                {
                    safe = 0u;
                    startCleanup = 1u;
                }
            }
        }
        else if (globalPhase != PARAM_GLOBAL_NONE || requestPending ||
            globalTerminal != 0u || mutationStagingEmpty() == 0u)
            safe = 0u;
    }
}

```

```

    }
    if (activeSession == f->session) lastPcSequence = f->sequence;
    x.messageType = PARAM_GATEWAY_MSG_HELLO_ACK;
    x.payloadLength = 10u;
    putU32(&x.payload[0], gatewayBoot);
    putU32(&x.payload[4], PARAM_GATEWAY_CAP_READ |
        PARAM_GATEWAY_CAP_LIBRARY_RUNTIME_SPLIT);
    x.payload[8] = safe;
    x.payload[9] = mask;
    sendFrame(&x);
    if (startDiscovery)
        startRecoveryDiscovery(f);
    else if (startCleanup)
        startSilentCleanup(f);
    return;
}
if (!activeSession || f->session != activeSession)
{
    sendError(f, "SESSION");
    return;
}
if (sequenceAdvances(f->sequence, lastPcSequence) == 0u)
{
    sendError(f, "SEQUENCE");
    return;
}
lastPcSequence = f->sequence;
lastActivityMs = now;
if (!recoveryDiscoveryDone)
{
    sendError(f, "RECOVERY");
    return;
}
if (requestPending)
{
    sendError(f, "BUSY");
    return;
}
if (gatewayCleanupFailed != 0u &&
    !(f->target == 0xFFu && f->messageType == PARAM_GATEWAY_INTERNAL_ABORT))
{
    sendError(f, "RECOVERY");
    return;
}
if (f->messageType == PARAM_GATEWAY_MSG_ENUM_TARGETS)
{
    x.messageType = PARAM_GATEWAY_MSG_TARGETS;
    x.payloadLength = 1u;
    x.payload[0] = paramSyncGatewayPlatformTargetMask();
    sendFrame(&x);
    return;
}
if (f->target == 0xFFu &&
    f->messageType == PARAM_GATEWAY_INTERNAL_READ_TRANSACTION_STATUS)
{
    x.messageType = PARAM_GATEWAY_INTERNAL_TRANSACTION_STATUS;
    x.payloadLength = sizeof(paramSyncGlobalState);
    memcpy(x.payload, &publishedGlobalState, sizeof(paramSyncGlobalState));
    sendFrame(&x);
    if (globalTerminal != 0u && globalOrigin == 0xFFu)
    {
        globalTerminalObservedByPc = 1u;
        globalTerminalPublishedMs = now -
            PARAM_GATEWAY_GLOBAL_TERMINAL_HOLD_MS;
    }
    return;
}
if (globalTerminal != 0u &&
    f->messageType >= PARAM_GATEWAY_INTERNAL_PREPARE_BEGIN &&
    f->messageType <= PARAM_GATEWAY_INTERNAL_ABORT)
{
    sendError(f, "BUSY");
    return;
}
if (f->target != 0xFFu &&
    (f->messageType == PARAM_GATEWAY_INTERNAL_COMMIT ||
    f->messageType == PARAM_GATEWAY_INTERNAL_ACTIVATE))
{
    sendError(f, "GLOBAL_ONLY");
}

```

```

    return;
}
if (f->target == 0xFFu &&
    (f->messageType == PARAM_GATEWAY_INTERNAL_COMMIT ||
     f->messageType == PARAM_GATEWAY_INTERNAL_ACTIVATE ||
     f->messageType == PARAM_GATEWAY_INTERNAL_ABORT))
{
    if (!startGlobal(f)) sendError(f, "GLOBAL");
    return;
}
if (requestOp(f->messageType) == PARAM_SYNC_OP_NONE)
{
    sendError(f, "OP");
    return;
}
if (f->messageType == PARAM_GATEWAY_INTERNAL_PREPARE_BEGIN &&
    f->payloadLength == 19u)
{
    uint32_t transactionId = getU32(&f->payload[1]);
    uint8_t i;
    if (transactionId == 0u || transactionId == UINT32_MAX ||
        f->target >= 5u)
    {
        sendError(f, "TRANSACTION");
        return;
    }
    for (i = 0u; i < 5u; i++)
    {
        if ((globalTouchedMask & (uint8_t) (1u << i)) != 0u &&
            targetTransactionId[i] != transactionId)
        {
            sendError(f, "TRANSACTION");
            return;
        }
    }
    if (!startPrepareIntent(f)) sendError(f, "INTENT");
    return;
}
currentInternalOp = requestOp(f->messageType);
if (f->target == 0u)
{
    if (!startCore0Request(f))
    {
        sendError(f, "CORE0");
        return;
    }
}
else if (f->target == PARAM_GATEWAY_CORE2_TARGET)
{
    if (!startCore2Request(f))
    {
        sendError(f, "CORE2");
        return;
    }
}
else if (f->target == 1u || f->target == 3u || f->target == 4u)
{
    if (!routeMailbox(f))
    {
        sendError(f, "MAILBOX");
        return;
    }
}
else
{
    sendError(f, "ROUTE");
    return;
}
pendingRequest = *f;
requestPending = 1u;
}

void paramSyncGatewayInit(void)
{
    paramSyncMailboxRequest retainedRequest;
    paramSyncMailboxResponse retainedResponse;
    rxHead = rxTail = rxEncodedLength = 0;
    rxOverflowSequence = 0u;
    rxOverflowAcknowledged = 0u;
}

```

```

rxDiscardUntilDelimiter = 0u;
activeSession = requestPending = mailboxSequence = 0;
lastPcSequence = 0u;
pendingMailboxSequence = 0u;
externalState = PARAM_EXT_IDLE;
externalSendPending = 0u;
core0State = PARAM_EXT_IDLE;
core0SendPending = 0u;
globalPhase = PARAM_GLOBAL_NONE;
globalTransactionId = 0u;
globalSlot = 0u;
globalTarget = 0u;
globalCommittedMask = 0u;
globalOperationFailed = 0u;
globalFinishRequired = 0u;
globalTouchedMask = 0u;
globalPreparedMask = 0u;
globalTargetMask = 0u;
globalSilent = 0u;
gatewayCleanupFailed = 0u;
globalFailedMask = 0u;
currentInternalOp = PARAM_SYNC_OP_NONE;
recoveryTargetMask = 0u;
recoveryOp = PARAM_SYNC_OP_NONE;
recoveryTransactionId = 0u;
prepareIntentPending = 0u;
recoveryDiscoveryDone = 0u;
if (loadRecoveryDecision(&recoveryOp, &recoveryTransactionId,
    &recoveryTargetMask))
{
    gatewayCleanupFailed = 1u;
}
memset(ownerBootGeneration, 0, sizeof(ownerBootGeneration));
memset(mailboxResponseBoot, 0, sizeof(mailboxResponseBoot));
memset(mailboxResponseSequence, 0, sizeof(mailboxResponseSequence));
if (paramSyncMailboxTryReadRequest(&retainedRequest) != 0u &&
    retainedRequest.sequence != 0u)
    mailboxSequence = retainedRequest.sequence;
if (paramSyncMailboxTryReadResponse(&retainedResponse) != 0u &&
    retainedResponse.target < 5u &&
    retainedResponse.producerBootGeneration != 0u &&
    retainedResponse.sequence != 0u)
{
    mailboxResponseBoot[retainedResponse.target] =
        retainedResponse.producerBootGeneration;
    mailboxResponseSequence[retainedResponse.target] =
        retainedResponse.sequence;
}
memset(targetPreparedSlot, 0, sizeof(targetPreparedSlot));
memset(targetTransactionId, 0, sizeof(targetTransactionId));
memset(globalExpectedCrc, 0, sizeof(globalExpectedCrc));
globalLibraryTransactionId = 0u;
globalLibraryIdentitySet = 0u;
globalSlotValidMask = 0u;
globalSlotEmptyMask = 0u;
globalSlotInvalidMask = 0u;
globalSlotTransactionId = 0u;
globalSlotTransactionValid = 0u;
memset(boardRequestBoot, 0, sizeof(boardRequestBoot));
memset(boardRequestSequence, 0, sizeof(boardRequestSequence));
memset(&publishedGlobalState, 0, sizeof(publishedGlobalState));
memset(&globalRequest, 0, sizeof(globalRequest));
globalStateSequence = 0u;
globalStateLastPublishMs = 0u;
rs485GatewayTxMs = 0u;
rs485GatewayTxPrimed = 0u;
globalAction = PARAM_SYNC_GLOBAL_ACTION_NONE;
globalOrigin = 0xFFu;
globalTerminal = globalTerminalOk = globalStateError = 0u;
globalStateFailedTarget = 0xFFu;
globalFailureDiagnostic = 0u;
globalConflictRetryTransactionId = 0u;
globalConflictRecoveryAttempted = 0u;
globalConflictCleanupActive = 0u;
globalBoardInitiated = 0u;
globalTerminalPublished = 0u;
globalTerminalObservedByPc = 0u;
globalTerminalPublishedMs = 0u;
externalReplayFenceStartMs = 0u;

```

```

    externalReplayFencePrimed = 0u;
    gatewayBoot = nextGatewayBootGeneration();
}
void paramSyncGatewayRxIrqHandler(void)
{
    uint8_t batch[16u];
    uint8_t count = uart_query_buffer(UART_0, batch);
    uint8_t i;
    for (i = 0u; i < count; i++)
    {
        uint16_t next;
        if (rxOverflowSequence != rxOverflowAcknowledged) continue;
        next = (uint16_t)((rxHead + 1u) &
            (PARAM_SYNC_GATEWAY_RING_BYTES - 1u));
        if (next == rxTail)
        {
            uint32_t sequence = rxOverflowSequence + 1u;
            if (sequence == 0u) sequence = 1u;
            rxOverflowSequence = sequence;
            continue;
        }
        rxRing[rxHead] = batch[i];
        rxHead = next;
    }
}
void paramSyncGatewayPoll(uint32_t now)
{
    paramSyncMailboxResponse m;
    paramSyncGatewayFrame automatic;
    (void) externalReplayFenceActive(now);
    paramSyncGatewayPlatformPoll(now);
    if (rxOverflowAcknowledged != rxOverflowSequence)
    {
        uint32_t observed = rxOverflowSequence;
        rxTail = rxHead;
        rxEncodedLength = 0u;
        rxDiscardUntilDelimiter = 1u;
        rxOverflowAcknowledged = observed;
    }
    while (rxTail != rxHead)
    {
        uint8_t b = rxRing[rxTail];
        rxTail = (uint16_t)((rxTail + 1u) & (PARAM_SYNC_GATEWAY_RING_BYTES - 1u));
        if (rxDiscardUntilDelimiter != 0u)
        {
            if (b == 0u) rxDiscardUntilDelimiter = 0u;
            continue;
        }
        if (!b)
        {
            paramSyncGatewayFrame f;
            if (rxEncodedLength && paramSyncGatewayDecode(rxEncodedFrame,
                rxEncodedLength, &f)) handleFrame(&f, now);
            rxEncodedLength = 0;
        }
        else if (rxEncodedLength < sizeof(rxEncodedFrame))
            rxEncodedFrame[rxEncodedLength++] = b;
        else
        {
            rxEncodedLength = 0u;
            rxDiscardUntilDelimiter = 1u;
        }
    }
    if (activeSession &&
        (uint32_t)(now - lastActivityMs) >= PARAM_GATEWAY_TIMEOUT_MS)
    {
        invalidateSession();
    }
    if (globalTerminal != 0u && globalPhase == PARAM_GLOBAL_NONE &&
        globalTerminalPublished != 0u &&
        (globalOrigin != 0xFFu || globalTerminalObservedByPc != 0u ||
        activeSession == 0u) &&
        (uint32_t)(now - globalTerminalPublishedMs) >=
            PARAM_GATEWAY_GLOBAL_TERMINAL_HOLD_MS)
    {
        globalTerminal = 0u;
        globalTerminalOk = 0u;
        globalStateError = PARAM_SYNC_GLOBAL_ERROR_NONE;
        globalStateFailedTarget = 0xFFu;
    }
}

```

```

    globalFailureDiagnostic = 0u;
    globalConflictRetryTransactionId = 0u;
    globalConflictRecoveryAttempted = 0u;
    globalConflictCleanupActive = 0u;
    globalAction = PARAM_SYNC_GLOBAL_ACTION_NONE;
    globalOrigin = 0xFFu;
    globalSlot = 0u;
    globalTransactionId = 0u;
    globalTerminalObservedByPc = 0u;
    memset(&globalRequest, 0, sizeof(globalRequest));
}
/* 在任何协调状态下都排空板端入口。忙时 considerBoardRequest 只消费并
 * 去重，不改写当前公开事务；避免队列中的旧按键在终态保持结束后重放。 */
pollBoardRequests();
memset(&automatic, 0, sizeof(automatic));
automatic.target = 0xFFu;
automatic.session = gatewayBoot;
automatic.sequence = globalStateSequence + 1u;
if (globalTerminal == 0u && globalPhase == PARAM_GLOBAL_NONE &&
    requestPending == 0u)
{
    if (recoveryDiscoveryDone == 0u || recoveryTargetMask != 0u ||
        gatewayCleanupFailed != 0u)
    {
        /* Requests left in a bridge/shared mailbox across coordinator
         * restart or recovery are stale by definition. Consume them while
         * the public state is not READY so they receive a matched failure
         * instead of executing automatically after recovery. */
        if (globalTerminal == 0u && globalPhase == PARAM_GLOBAL_NONE &&
            requestPending == 0u &&
            paramSyncGatewayPlatformTargetMask() ==
                PARAM_GATEWAY_TARGET_MASK)
        {
            if (recoveryDiscoveryDone == 0u)
                startRecoveryDiscovery(&automatic);
            else
                startSilentCleanup(&automatic);
        }
    }
    else
    {
        /* READY 且入口已在本拍统一轮询；无额外动作。 */
    }
}
publishGlobalState(now, 0u);
if (!requestPending || (pendingRequest.target != 1u &&
    pendingRequest.target != 3u && pendingRequest.target != 4u) ||
    !paramSyncMailboxTryReadResponse(&m)) return;
if (mailboxResponseFresh(&m) == 0u ||
    m.target != pendingRequest.target ||
    m.op != currentInternalOp ||
    m.slot != (pendingRequest.payloadLength ? pendingRequest.payload[0] : 0u) ||
    m.transactionId != pendingRequest.session ||
    m.offset != pendingMailboxSequence) return;
if (m.producerBootGeneration == 0u)
{
    sendError(&pendingRequest, "OWNER_BOOT");
    invalidateSession();
    return;
}
if (ownerBootGeneration[pendingRequest.target] != 0u &&
    ownerBootGeneration[pendingRequest.target] != m.producerBootGeneration)
{
    /* 本次会话仍 fail-closed，但采纳新 owner 代际供下一会话恢复。 */
    ownerBootGeneration[pendingRequest.target] =
        m.producerBootGeneration;
    sendError(&pendingRequest, "OWNER_BOOT");
    invalidateSession();
    return;
}
ownerBootGeneration[pendingRequest.target] = m.producerBootGeneration;
finishRequest(m.status, m.payloadCrc32, m.data, m.dataLength);
}

void paramSyncGatewayServiceExternal(uint32_t now, uint8_t rs485Allowed)
{
    chassisParamFrame reply;
    uint8_t status;
    uint8_t replayFenceActive = externalReplayFenceActive(now);

```

```

if (paramSyncGatewayCore2ReplyTurnaroundActive(now) != 0u) return;
if (replayFenceActive != 0u) discardExternalReplayResidue();
if ((!requestPending || pendingRequest.target != PARAM_GATEWAY_CORE2_TARGET ||
    replayFenceActive != 0u) &&
    rs485Allowed != 0u && publishedGlobalState.coordinatorBootGeneration != 0u &&
    chassisLinkSendParamGlobalState(&publishedGlobalState) != 0u)
{
    rs485GatewayTxMs = now;
    rs485GatewayTxPrimed = 1u;
    return;
}
if (!requestPending) return;
if ((pendingRequest.target == 0u ||
    pendingRequest.target == PARAM_GATEWAY_CORE2_TARGET) &&
    replayFenceActive != 0u) return;
if (pendingRequest.target == 0u)
{
    uartLinkFrameStruct reply0;
    uint8_t status0;
    if (core0State == PARAM_EXT_IDLE) return;
    if (crossCoreLinkTakeParamResponse(&reply0))
    {
        if (reply0.paramRequestId != core0Wire.paramRequestId)
            return;
        if (core0ReplyMatchesCurrentPhase(&reply0) == 0u)
        {
            completeExternal(PARAM_SYNC_STATUS_INVALID);
            return;
        }
        if (reply0.paramStatus == UART_LINK_PARAM_OWNER_RESTART)
        {
            sendError(&pendingRequest, "OWNER_BOOT");
            invalidateSession();
            return;
        }
        status0 = (uint8_t) (reply0.paramStatus & UART_LINK_PARAM_STATUS_MASK);
        if (core0State == PARAM_EXT_WAIT_BEGIN)
        {
            if (status0 != PARAM_SYNC_STATUS_PENDING)
            {
                completeExternal(status0);
                return;
            }
            if (pendingRequest.payloadLength == 0u)
            {
                core0Wire.paramOp = UART_LINK_PARAM_OP_POLL;
                core0Wire.paramOffset = 0u;
                core0Wire.paramLength = 0u;
                core0Wire.paramStatus = 0u;
                core0State = PARAM_EXT_WAIT_POLL;
            }
            else
            {
                uint16_t remaining0 = (uint16_t) (pendingRequest.payloadLength - core0RequestOffset);
                core0Wire.paramOffset = core0RequestOffset;
                core0Wire.paramLength = (remaining0 > UART_LINK_PARAM_DATA_BYTES) ? UART_LINK_PARAM_DATA_BYTES : (uint8_t) remaining0;
                memcpy(core0Wire.paramData, &pendingRequest.payload[core0RequestOffset],
                    core0Wire.paramLength);
                core0Wire.paramStatus = 0u;
                core0State = PARAM_EXT_WAIT_DATA;
            }
        }
    }
    else if (core0State == PARAM_EXT_WAIT_DATA)
    {
        if (status0 != PARAM_SYNC_STATUS_PENDING)
        {
            completeExternal(status0);
            return;
        }
        core0RequestOffset = (uint16_t) (core0RequestOffset + core0Wire.paramLength);
        if (core0RequestOffset >= pendingRequest.payloadLength)
        {
            core0Wire.paramOp = UART_LINK_PARAM_OP_POLL;
            core0Wire.paramOffset = 0u;
            core0Wire.paramLength = 0u;
            core0Wire.paramStatus = 0u;
            core0State = PARAM_EXT_WAIT_POLL;
        }
    }
    else

```

```

    {
        uint16_t remaining0 = (uint16_t) (pendingRequest.payloadLength - core0RequestOffset);
        core0Wire.paramOffset = core0RequestOffset;
        core0Wire.paramLength = (remaining0 > UART_LINK_PARAM_DATA_BYTES) ? UART_LINK_PARAM_DATA_BYTES : (uint8_t) remaining0;
        memcpy(core0Wire.paramData, &pendingRequest.payload[core0RequestOffset],
            core0Wire.paramLength);
    }
}
else if (status0 == PARAM_SYNC_STATUS_PENDING)
{
    core0Wire.paramOffset = core0ResponseLength;
}
else if (reply0.paramOffset != core0ResponseLength ||
    (uint32_t) core0ResponseLength + reply0.paramLength > sizeof(core0Response))
{
    completeExternal(PARAM_SYNC_STATUS_INVALID);
    return;
}
else
{
    memcpy(&core0Response[core0ResponseLength], reply0.paramData,
        reply0.paramLength);
    core0ResponseLength = (uint16_t) (core0ResponseLength + reply0.paramLength);
    if ((reply0.paramStatus & UART_LINK_PARAM_TRANSPORT_MORE) == 0u)
    {
        memcpy(externalResponse, core0Response, core0ResponseLength);
        externalResponseLength = core0ResponseLength;
        completeExternal(status0);
        return;
    }
    core0Wire.paramOffset = core0ResponseLength;
}
}
core0RetryCount = 0u;
core0SendPending = 1u;
}
if (!core0SendPending && (uint32_t) (now - core0LastSendMs) >= PARAM_GATEWAY_RETRY_MS)
{
    if (core0RetryCount >= PARAM_GATEWAY_RETRY_MAX)
    {
        completeExternal(PARAM_SYNC_STATUS_FAILED);
        return;
    }
    core0RetryCount++;
    core0SendPending = 1u;
}
if (core0SendPending && crossCoreLinkSendParamRequest(&core0Wire))
{
    core0LastSendMs = now;
    core0SendPending = 0u;
}
return;
}
}
if ((pendingRequest.target != PARAM_GATEWAY_CORE2_TARGET) ||
    (externalState == PARAM_EXT_IDLE)) return;

if (chassisLinkTryReadParamResponse(&reply) != 0u)
{
    if (reply.requestId != externalWire.requestId) return;
    if (core2ReplyMatchesCurrentPhase(&reply) == 0u)
    {
        completeExternal(PARAM_SYNC_STATUS_INVALID);
        return;
    }
}
if (reply.status == CHASSIS_PARAM_OWNER_RESTART)
{
    sendError(&pendingRequest, "OWNER_BOOT");
    invalidateSession();
    return;
}
status = (uint8_t) (reply.status & CHASSIS_PARAM_STATUS_MASK);
if (externalState == PARAM_EXT_WAIT_BEGIN)
{
    if (status != CHASSIS_PARAM_STATUS_PENDING)
    {
        completeExternal(status);
        return;
    }
}
if (pendingRequest.payloadLength == 0u)
{

```



```

        externalWire.op = CHASSIS_PARAM_OP_POLL;
        externalWire.offset = 0u;
        externalWire.length = 0u;
        externalWire.status = 0u;
        externalState = PARAM_EXT_WAIT_POLL;
    }
    else
    {
        uint16_t remaining = (uint16_t) (pendingRequest.payloadLength - externalRequestOffset);
        externalWire.offset = externalRequestOffset;
        externalWire.length = (remaining > CHASSIS_PARAM_DATA_BYTES) ? CHASSIS_PARAM_DATA_BYTES : (uint8_t) remaining;
        memcpy(externalWire.data, &pendingRequest.payload[externalRequestOffset],
            externalWire.length);
        externalWire.status = 0u;
        externalState = PARAM_EXT_WAIT_DATA;
    }
}
else if (externalState == PARAM_EXT_WAIT_DATA)
{
    if (status != CHASSIS_PARAM_STATUS_PENDING)
    {
        completeExternal(status);
        return;
    }
    externalRequestOffset = (uint16_t) (externalRequestOffset + externalWire.length);
    if (externalRequestOffset >= pendingRequest.payloadLength)
    {
        externalWire.op = CHASSIS_PARAM_OP_POLL;
        externalWire.offset = 0u;
        externalWire.length = 0u;
        externalWire.status = 0u;
        externalState = PARAM_EXT_WAIT_POLL;
    }
    else
    {
        uint16_t remaining = (uint16_t) (pendingRequest.payloadLength - externalRequestOffset);
        externalWire.offset = externalRequestOffset;
        externalWire.length = (remaining > CHASSIS_PARAM_DATA_BYTES) ? CHASSIS_PARAM_DATA_BYTES : (uint8_t) remaining;
        memcpy(externalWire.data, &pendingRequest.payload[externalRequestOffset],
            externalWire.length);
    }
}
else
{
    if (status == CHASSIS_PARAM_STATUS_PENDING)
    {
        externalWire.offset = externalResponseLength;
    }
    else if ((reply.offset != externalResponseLength) ||
        ((uint32_t) externalResponseLength + reply.length > sizeof(externalResponse)))
    {
        completeExternal(PARAM_SYNC_STATUS_INVALID);
        return;
    }
    else
    {
        memcpy(&externalResponse[externalResponseLength], reply.data, reply.length);
        externalResponseLength = (uint16_t) (externalResponseLength + reply.length);
        if ((reply.status & CHASSIS_PARAM_TRANSPORT_MORE) == 0u)
        {
            completeExternal(status);
            return;
        }
        externalWire.offset = externalResponseLength;
    }
}
externalRetryCount = 0u;
externalSendPending = 1u;
}

if ((!externalSendPending) &&
    ((uint32_t) (now - externalLastSendMs) >= PARAM_GATEWAY_RETRY_MS))
{
    if (externalRetryCount >= PARAM_GATEWAY_RETRY_MAX)
    {
        completeExternal(PARAM_SYNC_STATUS_FAILED);
        return;
    }
    externalRetryCount++;
}

```

```

        externalSendPending = 1u;
    }
    if (externalSendPending && rs485Allowed &&
        chassisLinkSendParamRequest(&externalWire))
    {
        externalLastSendMs = now;
        externalSendPending = 0u;
        rs485GatewayTxMs = now;
        rs485GatewayTxPrimed = 1u;
    }
}

uint8_t paramSyncGatewayCore2ReplyTurnaroundActive(uint32_t nowMs)
{
    return (uint8_t) (rs485GatewayTxPrimed != 0u &&
        (uint32_t) (nowMs - rs485GatewayTxMs) <
            PARAM_SYNC_GATEWAY_RS485_REPLY_GUARD_MS);
}

uint8_t paramSyncGatewayIsSessionActive(void)
{
    return activeSession ? 1u : 0u;
}

```

core1_cyt4bb7/project/code/comm/paramSyncMailbox.c

SHA-256

:

774a74f34cc5eaab467a692cd9faa2c10c8c111750dc3e38cd1b56c2694384ea

代码 (c)

```

#include "paramSyncMailbox.h"

#include "cy_device_headers.h"

#include <string.h>

#define PARAM_SYNC_REQUEST_MAGIC (0x50535251u) /* PSRQ */
#define PARAM_SYNC_RESPONSE_MAGIC (0x50535253u) /* PSRS */
#define PARAM_SYNC_MAILBOX_ABI (1u)

typedef struct
{
    uint32_t beginSequence;
    uint32_t magic;
    uint32_t abiVersion;
    uint32_t payloadBytes;
    uint32_t producerBootGeneration;
    uint32_t sequence;
    uint32_t transactionId;
    uint32_t offset;
    uint32_t payloadCrc32;
    uint16_t dataLength;
    uint8_t op;
    uint8_t slot;
    uint8_t target;
    uint8_t state;
    uint8_t reservedHeader[6];
    uint8_t data[PARAM_SYNC_MAILBOX_DATA_MAX];
    uint8_t reservedTail[12];
    uint32_t endSequence;
} paramSyncMailboxSharedFrame;

typedef char paramSyncMailbox_frame_size_assert[
    (sizeof(paramSyncMailboxSharedFrame) == PARAM_SYNC_MAILBOX_FRAME_BYTES) ? 1 : -1];

#pragma location = PARAM_SYNC_MAILBOX_REQUEST_ADDR
__no_init static volatile paramSyncMailboxSharedFrame paramSyncRequestRegion;
#pragma location = PARAM_SYNC_MAILBOX_RESPONSE_ADDR
__no_init static volatile paramSyncMailboxSharedFrame paramSyncResponseRegion;

static uint8_t frameInputValid(uint32_t bootGeneration, uint32_t sequence,
    uint16_t dataLength)
{
    return (uint8_t) ((bootGeneration != 0u) && (sequence != 0u) &&
        (dataLength <= PARAM_SYNC_MAILBOX_DATA_MAX));
}

```

```

}
static void cleanRegion(volatile paramSyncMailboxSharedFrame *region)
{
    SCB_CleanDCache_by_Addr((volatile void *) region, sizeof(*region));
    __DSB();
}

static void invalidateRegion(volatile paramSyncMailboxSharedFrame *region)
{
    SCB_InvalidateDCache_by_Addr((volatile void *) region, sizeof(*region));
    __DSB();
}

static uint8_t publishFrame(volatile paramSyncMailboxSharedFrame *region,
    const paramSyncMailboxSharedFrame *frame)
{
    paramSyncMailboxSharedFrame staged;

    staged = *frame;
    staged.endSequence = 0u;

    region->beginSequence = 0u;
    region->endSequence = 0u;
    cleanRegion(region);

    memcpy((void *) region, &staged, sizeof(staged));
    cleanRegion(region);

    region->endSequence = staged.sequence;
    SCB_CleanDCache_by_Addr((volatile void *)
        (((uintptr_t) &region->endSequence) & ~((uintptr_t) 31u)), 32u);
    __DSB();
    return 1u;
}

static uint8_t sharedFrameValid(const paramSyncMailboxSharedFrame *frame,
    uint32_t magic)
{
    return (uint8_t) ((frame->beginSequence != 0u) &&
        (frame->beginSequence == frame->sequence) &&
        (frame->endSequence == frame->sequence) &&
        (frame->magic == magic) &&
        (frame->abiVersion == PARAM_SYNC_MAILBOX_ABI) &&
        (frame->payloadBytes == frame->dataLength) &&
        frameInputValid(frame->producerBootGeneration, frame->sequence,
            frame->dataLength));
}

static uint8_t tryReadFrame(volatile paramSyncMailboxSharedFrame *region,
    uint32_t magic, paramSyncMailboxSharedFrame *frame)
{
    paramSyncMailboxSharedFrame first;
    paramSyncMailboxSharedFrame second;
    uint8_t attempt;

    for (attempt = 0u; attempt < 3u; attempt++)
    {
        invalidateRegion(region);
        memcpy(&first, (const void *) region, sizeof(first));
        invalidateRegion(region);
        memcpy(&second, (const void *) region, sizeof(second));
        if (sharedFrameValid(&first, magic) &&
            sharedFrameValid(&second, magic) &&
            (memcmp(&first, &second, sizeof(first)) == 0))
        {
            *frame = second;
            return 1u;
        }
    }
    return 0u;
}

uint8_t paramSyncMailboxPublishRequest(const paramSyncMailboxRequest *request)
{
    paramSyncMailboxSharedFrame frame;
    if ((request == 0) || !frameInputValid(request->producerBootGeneration,
        request->sequence, request->dataLength)) return 0u;
    memset(&frame, 0, sizeof(frame));
    frame.beginSequence = request->sequence;

```

```

    frame.magic = PARAM_SYNC_REQUEST_MAGIC;
    frame.abiVersion = PARAM_SYNC_MAILBOX_ABI;
    frame.payloadBytes = request->dataLength;
    frame.producerBootGeneration = request->producerBootGeneration;
    frame.sequence = request->sequence;
    frame.transactionId = request->transactionId;
    frame.offset = request->offset;
    frame.payloadCrc32 = request->payloadCrc32;
    frame.dataLength = request->dataLength;
    frame.op = request->op;
    frame.slot = request->slot;
    frame.target = request->target;
    frame.state = request->flags;
    memcpy(frame.data, request->data, request->dataLength);
    return publishFrame(&paramSyncRequestRegion, &frame);
}

uint8_t paramSyncMailboxTryReadRequest(paramSyncMailboxRequest *request)
{
    paramSyncMailboxSharedFrame frame;
    paramSyncMailboxRequest candidate;
    if ((request == 0) || !tryReadFrame(&paramSyncRequestRegion,
        PARAM_SYNC_REQUEST_MAGIC, &frame)) return 0u;
    memset(&candidate, 0, sizeof(candidate));
    candidate.producerBootGeneration = frame.producerBootGeneration;
    candidate.sequence = frame.sequence;
    candidate.transactionId = frame.transactionId;
    candidate.offset = frame.offset;
    candidate.payloadCrc32 = frame.payloadCrc32;
    candidate.dataLength = frame.dataLength;
    candidate.op = frame.op;
    candidate.slot = frame.slot;
    candidate.target = frame.target;
    candidate.flags = frame.state;
    memcpy(candidate.data, frame.data, frame.dataLength);
    *request = candidate;
    return 1u;
}

uint8_t paramSyncMailboxPublishResponse(const paramSyncMailboxResponse *response)
{
    paramSyncMailboxSharedFrame frame;
    if ((response == 0) || !frameInputValid(response->producerBootGeneration,
        response->sequence, response->dataLength)) return 0u;
    memset(&frame, 0, sizeof(frame));
    frame.beginSequence = response->sequence;
    frame.magic = PARAM_SYNC_RESPONSE_MAGIC;
    frame.abiVersion = PARAM_SYNC_MAILBOX_ABI;
    frame.payloadBytes = response->dataLength;
    frame.producerBootGeneration = response->producerBootGeneration;
    frame.sequence = response->sequence;
    frame.transactionId = response->transactionId;
    frame.offset = response->offset;
    frame.payloadCrc32 = response->payloadCrc32;
    frame.dataLength = response->dataLength;
    frame.op = response->op;
    frame.slot = response->slot;
    frame.target = response->target;
    frame.state = response->status;
    memcpy(frame.data, response->data, response->dataLength);
    return publishFrame(&paramSyncResponseRegion, &frame);
}

uint8_t paramSyncMailboxTryReadResponse(paramSyncMailboxResponse *response)
{
    paramSyncMailboxSharedFrame frame;
    paramSyncMailboxResponse candidate;
    if ((response == 0) || !tryReadFrame(&paramSyncResponseRegion,
        PARAM_SYNC_RESPONSE_MAGIC, &frame)) return 0u;
    memset(&candidate, 0, sizeof(candidate));
    candidate.producerBootGeneration = frame.producerBootGeneration;
    candidate.sequence = frame.sequence;
    candidate.transactionId = frame.transactionId;
    candidate.offset = frame.offset;
    candidate.payloadCrc32 = frame.payloadCrc32;
    candidate.dataLength = frame.dataLength;
    candidate.op = frame.op;
    candidate.slot = frame.slot;
    candidate.target = frame.target;

```

```

    candidate.status = frame.state;
    memcpy(candidate.data, frame.data, frame.dataLength);
    *response = candidate;
    return 1u;
}

```

core1_cyt4bb7/project/code/comm/paramSyncObserverRouter.c

SHA-256

:

338b3b99a39f0ccdbabcdda7749f5031b2dfd0caf8a5d61ff909772d7ea9986f

代码 (c)

```

#include "paramSyncObserverRouter.h"

#include <string.h>

#include "core3LinkCore1.h"
#include "core4LinkCore1.h"
#include "coreDataTransfer.h"
#include "paramSyncMailbox.h"

#define OBSERVER_ROUTER_TIMEOUT_MS (800u)
#define OBSERVER_GLOBAL_BRIDGE_HEARTBEAT_MS (50u)
#define OBSERVER_GLOBAL_STATE_FRESH_MS (300u)

typedef enum
{
    ROUTER_IDLE = 0u,
    ROUTER_WAIT_BEGIN,
    ROUTER_WAIT_DATA,
    ROUTER_WAIT_POLL
} observerRouterState;

static observerRouterState g_state;
static paramSyncMailboxRequest g_request;
static observerParamFrame g_wire;
static uint8_t g_responseData[PARAM_SYNC_MAILBOX_DATA_MAX];
static uint16_t g_requestOffset, g_responseLength, g_responseExpectedLength;
static uint32_t g_deadlineMs, g_lastBoot, g_lastSequence, g_responseSequence;
static uint32_t g_routerBootGeneration, g_requestPeerSession;
static uint32_t g_wireTransactionSequence, g_wireTransactionId;
static uint32_t g_responsePayloadCrc32;
static uint8_t g_sendPending, g_responseIdentityPrimed;

typedef struct
{
    paramSyncGlobalRequest request;
    uint32_t requestCoordinatorBoot;
    uint32_t peerSession;
    uint32_t queuedStateBoot;
    uint32_t queuedStateSequence;
    uint32_t bridgeSequence;
    uint32_t bridgeLastPublishMs;
    uint8_t source;
    uint8_t ready;
    uint8_t requestPending;
    uint8_t bridgeDirty;
} observerGlobalRoute;

static observerGlobalRoute g_globalRoutes[2];
static paramSyncGlobalState g_latestGlobalState;
static uint8_t g_latestGlobalStateValid;
static uint8_t g_latestGlobalStatePrimed;
static uint32_t g_latestGlobalStateLastAdvanceMs;

static void getObserverStatus(uint8_t source,
    observerLinkCore1EndpointStatus *status)
{
    if (source == 3u) core3LinkCore1GetStatus(status);
    else core4LinkCore1GetStatus(status);
}

static uint8_t queueGlobalState(uint8_t source,
    const paramSyncGlobalState *state)

```

```

{
    return (source == 3u) ?
        core3LinkCore1QueueParamGlobalState(state) :
        core4LinkCore1QueueParamGlobalState(state);
}

static uint8_t takeGlobalRequest(uint8_t source,
    paramSyncGlobalRequest *request)
{
    return (source == 3u) ?
        core3LinkCore1TakeParamGlobalRequest(request) :
        core4LinkCore1TakeParamGlobalRequest(request);
}

static uint8_t sequenceNewer(uint32_t value, uint32_t previous)
{
    return (uint8_t) ((int32_t) (value - previous) > 0);
}

static uint8_t observerReady(
    const observerLinkCore1EndpointStatus *status)
{
    return (uint8_t) (status->online != 0u &&
        status->protocolCompatible != 0u && status->peerSession != 0u &&
        (status->peerCapabilities & OBSERVER_LINK_CAP_PARAM_SYNC) != 0u);
}

static void clearGlobalRequest(observerGlobalRoute *route)
{
    if (route->requestPending != 0u)
    {
        memset(&route->request, 0, sizeof(route->request));
        route->requestCoordinatorBoot = 0u;
        route->requestPending = 0u;
        route->bridgeDirty = 1u;
    }
}

static void publishGlobalBridge(observerGlobalRoute *route,
    uint32_t tickMs, uint8_t force)
{
    core1ParamObserverBridgeStruct bridge;
    if (force == 0u && route->bridgeDirty == 0u &&
        (uint32_t) (tickMs - route->bridgeLastPublishMs) <
            OBSERVER_GLOBAL_BRIDGE_HEARTBEAT_MS) return;
    memset(&bridge, 0, sizeof(bridge));
    route->bridgeSequence++;
    if (route->bridgeSequence == 0u) route->bridgeSequence = 1u;
    bridge.magic = CORE1_PARAM_OBSERVER_MAGIC;
    bridge.producerBootGeneration = coreDataGetPlanBootGeneration();
    bridge.publishSequence = route->bridgeSequence;
    bridge.source = route->source;
    if (route->ready != 0u)
    {
        bridge.peerSession = route->peerSession;
        bridge.flags = CORE1_PARAM_OBSERVER_FLAG_READY;
        if (route->requestPending != 0u)
        {
            bridge.request = route->request;
            bridge.flags |= CORE1_PARAM_OBSERVER_FLAG_REQUEST_VALID;
        }
    }
    coreDataWriteParamObserverBridge(&bridge);
    route->bridgeLastPublishMs = tickMs;
    route->bridgeDirty = 0u;
}

static void acceptLatestGlobalState(uint32_t tickMs)
{
    paramSyncGlobalState state;
    uint8_t i;
    if (coreDataTryReadParamGlobalState(&state) == 0u) return;
    if (g_latestGlobalStatePrimed == 0u ||
        state.coordinatorBootGeneration !=
            g_latestGlobalState.coordinatorBootGeneration)
    {
        if (g_latestGlobalStatePrimed != 0u)
        {

```

```

        for (i = 0u; i < 2u; i++)
            clearGlobalRequest(&g_globalRoutes[i]);
    }
    g_latestGlobalState = state;
    g_latestGlobalStateValid = 0u;
    g_latestGlobalStatePrimed = 1u;
    g_latestGlobalStateLastAdvanceMs = tickMs;
    for (i = 0u; i < 2u; i++)
    {
        g_globalRoutes[i].queuedStateBoot = 0u;
        g_globalRoutes[i].queuedStateSequence = 0u;
    }
    return;
}
if (sequenceNewer(state.sequence, g_latestGlobalState.sequence) == 0u)
    return;
g_latestGlobalState = state;
g_latestGlobalStateValid = 1u;
g_latestGlobalStatePrimed = 1u;
g_latestGlobalStateLastAdvanceMs = tickMs;
for (i = 0u; i < 2u; i++)
{
    g_globalRoutes[i].queuedStateBoot = 0u;
    g_globalRoutes[i].queuedStateSequence = 0u;
}
}

static void pollGlobalRoute(observerGlobalRoute *route, uint32_t tickMs)
{
    observerLinkCore1EndpointStatus status;
    paramSyncGlobalRequest request;
    uint8_t ready;
    getObserverStatus(route->source, &status);
    ready = (uint8_t) (observerReady(&status) != 0u &&
        g_latestGlobalStateValid != 0u);
    if (ready == 0u || route->ready == 0u ||
        route->peerSession != status.peerSession)
    {
        if (ready == 0u || route->peerSession != status.peerSession)
            clearGlobalRequest(route);
        route->queuedStateBoot = 0u;
        route->queuedStateSequence = 0u;
    }
    if (route->ready != ready ||
        route->peerSession != (ready != 0u ? status.peerSession : 0u))
        route->bridgeDirty = 1u;
    route->ready = ready;
    route->peerSession = (ready != 0u) ? status.peerSession : 0u;

    if (ready != 0u && g_latestGlobalStateValid != 0u &&
        (route->queuedStateBoot !=
            g_latestGlobalState.coordinatorBootGeneration ||
            route->queuedStateSequence != g_latestGlobalState.sequence) &&
        queueGlobalState(route->source, &g_latestGlobalState) != 0u)
    {
        route->queuedStateBoot =
            g_latestGlobalState.coordinatorBootGeneration;
        route->queuedStateSequence = g_latestGlobalState.sequence;
    }

    if (takeGlobalRequest(route->source, &request) != 0u &&
        ready != 0u && request.origin == route->source &&
        paramSyncGlobalRequestIsValid(&request) != 0u &&
        g_latestGlobalStateValid != 0u &&
        paramSyncGlobalStateReady(&g_latestGlobalState) != 0u)
    {
        if (route->requestPending == 0u ||
            (route->request.producerBootGeneration ==
                request.producerBootGeneration &&
                route->request.sequence == request.sequence))
        {
            route->request = request;
            route->requestCoordinatorBoot =
                g_latestGlobalState.coordinatorBootGeneration;
            route->requestPending = 1u;
            route->bridgeDirty = 1u;
        }
    }
}
}

```

```

    if (route->requestPending != 0u &&
        (ready == 0u || g_latestGlobalStateValid == 0u ||
         route->requestCoordinatorBoot !=
          g_latestGlobalState.coordinatorBootGeneration ||
         paramSyncGlobalStateMatchesRequest(&g_latestGlobalState,
          &route->request) != 0u ||
         paramSyncGlobalStateReady(&g_latestGlobalState) == 0u))
        clearGlobalRequest(route);
    publishGlobalBridge(route, tickMs, 0u);
}

static void pollGlobalRoutes(uint32_t tickMs)
{
    uint8_t i;
    acceptLatestGlobalState(tickMs);
    if (g_latestGlobalStateValid != 0u &&
        (uint32_t)(tickMs - g_latestGlobalStateLastAdvanceMs) >=
         OBSERVER_GLOBAL_STATE_FRESH_MS)
    {
        g_latestGlobalStateValid = 0u;
        for (i = 0u; i < 2u; i++)
            clearGlobalRequest(&g_globalRoutes[i]);
    }
    pollGlobalRoute(&g_globalRoutes[0], tickMs);
    pollGlobalRoute(&g_globalRoutes[1], tickMs);
}

static uint8_t queueRequest(const uint8_t *payload, uint16_t length)
{
    return (g_request.target == 3u) ? core3LinkCore1QueueParamRequest(payload, length) : core4LinkCore1QueueParamRequest(payload, length);
}

static uint8_t takeResponse(uint8_t *payload, uint16_t capacity, uint16_t *length)
{
    return (g_request.target == 3u) ? core3LinkCore1TakeParamResponse(payload, capacity, length) : core4LinkCore1TakeParamResponse(payload, capacity, length);
}

static void publish(uint8_t status)
{
    paramSyncMailboxResponse response;
    memset(&response, 0, sizeof(response));
    g_responseSequence++;
    if (g_responseSequence == 0u) g_responseSequence = 1u;
    if (g_routerBootGeneration == 0u) return;
    response.producerBootGeneration = g_routerBootGeneration;
    response.sequence = g_responseSequence;
    response.transactionId = g_request.transactionId;
    response.offset = g_request.offset;
    response.payloadCrc32 = g_responseIdentityPrimed != 0u ?
        g_responsePayloadCrc32 : 0u;
    response.dataLength = g_responseLength;
    response.op = g_request.op;
    response.slot = g_request.slot;
    response.target = g_request.target;
    response.status = status;
    memcpy(response.data, g_responseData, g_responseLength);
    (void) paramSyncMailboxPublishResponse(&response);
    g_state = ROUTER_IDLE;
    g_sendPending = 0u;
}

static void setPoll(void)
{
    g_wire.op = OBSERVER_PARAM_OP_POLL;
    g_wire.offset = g_responseLength;
    g_wire.totalLength = 0u;
    g_wire.payloadCrc32 = 0u;
    g_wire.length = 0u;
    g_wire.status = 0u;
    g_state = ROUTER_WAIT_POLL;
    g_sendPending = 1u;
}

static void setData(void)
{
    uint16_t remaining = (uint16_t)(g_request.dataLength - g_requestOffset);
    g_wire.offset = g_requestOffset;
    g_wire.totalLength = g_request.dataLength;
}

```



```

g_wire.payloadCrc32 = g_request.payloadCrc32;
g_wire.length = (remaining > OBSERVER_PARAM_DATA_BYTES) ? OBSERVER_PARAM_DATA_BYTES : (uint8_t) remaining;
g_wire.status = 0u;
memcpy(g_wire.data, &g_request.data[g_requestOffset], g_wire.length);
g_state = ROUTER_WAIT_DATA;
g_sendPending = 1u;
}

static uint32_t nextWireTransactionId(void)
{
    g_wireTransactionSequence++;
    if (g_wireTransactionSequence == 0u ||
        g_wireTransactionSequence == 0xFFFFFFFFu)
        g_wireTransactionSequence = 1u;
    return g_wireTransactionSequence;
}

static uint8_t replyStatusValid(const observerParamFrame *reply)
{
    uint8_t status;
    uint8_t transport;
    if (reply == 0) return 0u;
    status = (uint8_t) (reply->status & OBSERVER_PARAM_STATUS_MASK);
    transport = (uint8_t) (reply->status &
        (uint8_t) ~OBSERVER_PARAM_STATUS_MASK);
    return (uint8_t) (status <= PARAM_SYNC_STATUS_FAILED &&
        (transport == 0u || transport == OBSERVER_PARAM_TRANSPORT_MORE));
}

static uint8_t replyMatchesCurrentPhase(const observerParamFrame *reply)
{
    uint8_t status;
    uint8_t more;
    uint16_t remaining;
    uint8_t expectedLength;
    if (reply == 0 || reply->transactionId != g_wireTransactionId ||
        reply->slot != g_request.slot || reply->target != g_request.target ||
        replyStatusValid(reply) == 0u) return 0u;
    status = (uint8_t) (reply->status & OBSERVER_PARAM_STATUS_MASK);
    more = (uint8_t) (reply->status & OBSERVER_PARAM_TRANSPORT_MORE);
    if (g_state == ROUTER_WAIT_BEGIN || g_state == ROUTER_WAIT_DATA)
    {
        return (uint8_t) (status != PARAM_SYNC_STATUS_OK &&
            reply->op == g_request.op &&
            reply->offset == g_wire.offset &&
            reply->totalLength == g_request.dataLength &&
            reply->payloadCrc32 == 0u && reply->length == 0u && more == 0u);
    }
    if (g_state != ROUTER_WAIT_POLL ||
        reply->offset != g_responseLength) return 0u;
    if (status == PARAM_SYNC_STATUS_PENDING)
    {
        return (uint8_t) (reply->op == OBSERVER_PARAM_OP_POLL &&
            reply->totalLength == 0u && reply->payloadCrc32 == 0u &&
            reply->length == 0u && more == 0u);
    }
    if (reply->op == OBSERVER_PARAM_OP_POLL)
    {
        return (uint8_t) (status != PARAM_SYNC_STATUS_OK &&
            reply->totalLength == 0u && reply->payloadCrc32 == 0u &&
            reply->length == 0u && more == 0u);
    }
    if (reply->op != g_request.op ||
        reply->totalLength > sizeof(g_responseData) ||
        reply->offset > reply->totalLength ||
        (g_responseIdentityPrimed != 0u &&
            (reply->totalLength != g_responseExpectedLength ||
                reply->payloadCrc32 != g_responsePayloadCrc32))) return 0u;
    remaining = (uint16_t) (reply->totalLength - reply->offset);
    expectedLength = (remaining > OBSERVER_PARAM_DATA_BYTES) ?
        OBSERVER_PARAM_DATA_BYTES : (uint8_t) remaining;
    return (uint8_t) (reply->length == expectedLength &&
        ((remaining > expectedLength) ?
            (more != 0u) : (more == 0u)));
}

static void startRequest(const paramSyncMailboxRequest *request, uint32_t tickMs)
{
    observerLinkCore1EndpointStatus status;

```

```

g_request = *request;
memset(&g_wire, 0, sizeof(g_wire));
g_wire.transactionId = nextWireTransactionId();
g_wire.transactionId = g_wire.transactionId;
g_wire.op = request->op;
g_wire.slot = request->slot;
g_wire.target = request->target;
g_wire.status = OBSERVER_PARAM_TRANSPORT_BEGIN;
g_wire.totalLength = request->dataLength;
g_wire.payloadCrc32 = request->payloadCrc32;
g_requestOffset = 0u;
g_responseLength = 0u;
g_responseExpectedLength = 0u;
g_responsePayloadCrc32 = 0u;
g_responseIdentityPrimed = 0u;
g_deadlineMs = tickMs + OBSERVER_ROUTER_TIMEOUT_MS;
g_state = ROUTER_WAIT_BEGIN;
g_sendPending = 1u;
getObserverStatus(request->target, &status);
if (observerReady(&status) == 0u)
{
    publish(PARAM_SYNC_STATUS_FAILED);
    return;
}
g_requestPeerSession = status.peerSession;
}

void paramSyncObserverRouterInit(void)
{
    paramSyncMailboxRequest retainedRequest;
    g_state = ROUTER_IDLE;
    g_sendPending = 0u;
    g_lastBoot = g_lastSequence = g_responseSequence = 0u;
    if (paramSyncMailboxTryReadRequest(&retainedRequest) != 0u)
    {
        /* 共享邮箱是保留快照；重启只建立基线，不能重放旧请求。 */
        g_lastBoot = retainedRequest.producerBootGeneration;
        g_lastSequence = retainedRequest.sequence;
    }
    g_routerBootGeneration = coreDataGetPlanBootGeneration();
    g_requestPeerSession = 0u;
    g_wireTransactionSequence = 0u;
    g_wire.transactionId = 0u;
    memset(g_globalRoutes, 0, sizeof(g_globalRoutes));
    memset(&g_latestGlobalState, 0, sizeof(g_latestGlobalState));
    g_latestGlobalStateValid = 0u;
    g_latestGlobalStatePrimed = 0u;
    g_latestGlobalStateLastAdvanceMs = 0u;
    g_globalRoutes[0].source = 3u;
    g_globalRoutes[1].source = 4u;
    g_globalRoutes[0].bridgeDirty = 1u;
    g_globalRoutes[1].bridgeDirty = 1u;
    publishGlobalBridge(&g_globalRoutes[0], 0u, 1u);
    publishGlobalBridge(&g_globalRoutes[1], 0u, 1u);
}

void paramSyncObserverRouterPoll(uint32_t tickMs)
{
    paramSyncMailboxRequest request;
    observerParamFrame reply;
    uint8_t payload[OBSERVER_LINK_MAX_PAYLOAD_BYTES];
    uint16_t length;
    uint8_t status;
    observerLinkCore1EndpointStatus ownerStatus;

    pollGlobalRoutes(tickMs);

    if (g_state == ROUTER_IDLE && paramSyncMailboxTryReadRequest(&request) &&
        (request.producerBootGeneration != g_lastBoot ||
         sequenceNewer(request.sequence, g_lastSequence) != 0u))
    {
        g_lastBoot = request.producerBootGeneration;
        g_lastSequence = request.sequence;
        if (request.target == 3u || request.target == 4u) startRequest(&request, tickMs);
    }
    if (g_state == ROUTER_IDLE) return;
    getObserverStatus(g_request.target, &ownerStatus);
    if (observerReady(&ownerStatus) == 0u ||
        ownerStatus.peerSession != g_requestPeerSession)

```

```

{
    publish(PARAM_SYNC_STATUS_FAILED);
    return;
}
if ((int32_t) (tickMs - g_deadlineMs) >= 0)
{
    publish(PARAM_SYNC_STATUS_FAILED);
    return;
}
if (takeResponse(payload, sizeof(payload), &length) &&
    observerParamDecode(payload, length, &reply) &&
    reply.transactionId == g_wireTransactionId)
{
    if (replyMatchesCurrentPhase(&reply) == 0u)
    {
        publish(PARAM_SYNC_STATUS_INVALID);
        return;
    }
    status = (uint8_t) (reply.status & OBSERVER_PARAM_STATUS_MASK);
    if (g_state == ROUTER_WAIT_BEGIN)
    {
        if (status != PARAM_SYNC_STATUS_PENDING)
        {
            publish(status);
            return;
        }
        if (g_request.dataLength == 0u) setPoll();
        else setData();
    }
    else if (g_state == ROUTER_WAIT_DATA)
    {
        if (status != PARAM_SYNC_STATUS_PENDING)
        {
            publish(status);
            return;
        }
        g_requestOffset = (uint16_t) (g_requestOffset + g_wire.length);
        if (g_requestOffset >= g_request.dataLength) setPoll();
        else setData();
    }
    else if (status == PARAM_SYNC_STATUS_PENDING)
    {
        setPoll();
    }
    else if (reply.offset != g_responseLength ||
        (uint32_t) g_responseLength + reply.length > sizeof(g_responseData))
    {
        publish(PARAM_SYNC_STATUS_INVALID);
        return;
    }
    else
    {
        if (g_responseIdentityPrimed == 0u)
        {
            g_responseExpectedLength = reply.totalLength;
            g_responsePayloadCrc32 = reply.payloadCrc32;
            g_responseIdentityPrimed = 1u;
        }
        memcpy(&g_responseData[g_responseLength], reply.data, reply.length);
        g_responseLength = (uint16_t) (g_responseLength + reply.length);
        if ((reply.status & OBSERVER_PARAM_TRANSPORT_MORE) != 0u) setPoll();
        else
        {
            publish(status);
            return;
        }
    }
}
}
if (g_sendPending && observerParamEncode(&g_wire, payload, &length) &&
    queueRequest(payload, length))
    g_sendPending = 0u;
}

```

core1_cy4bb7/project/code/comm/paramSyncOwnerService.c

SHA-256

:

ee7b7fa4f406f586989309045fb8c7fef5699d0d8c5659bf5d056a5b90c08a9d

代码 (c)

```
#include "paramSyncOwnerService.h"

#include <stddef.h>
#include <string.h>

#include "coreDataTransfer.h"
#include "crc32Ieee.h"
#include "paramSyncGateway.h"
#include "paramSyncMailbox.h"
#include "paramSyncCoordinatorJournal.h"
#include "uartLinkProto.h"
#include "visionParams.h"
#include "visionParamsFlashSchemaInternal.h"

#define CORE1_SCHEMA_HASH (0x9869F400u)
#define CORE1_ACTIVATE_ROLLBACK_MAGIC (0xA1B20 000u)
#define PARAM_STAGE_NONE (0u)
#define PARAM_STAGE_LIBRARY (1u)
#define PARAM_STAGE_RUNTIME (2u)

static uint32_t lastBoot, lastSequence, responseSequence;
#if defined(__ICCARM__)
__no_init uint32_t paramSyncOwnerBootRetained;
__no_init uint32_t paramSyncOwnerBootInverseRetained;
#else
static uint32_t paramSyncOwnerBootRetained;
static uint32_t paramSyncOwnerBootInverseRetained;
#endif
static uint32_t ownerBootGeneration;
static visionFlashBlobStruct stagedBlob;
static uint32_t stagedTransaction, stagedExpectedCrc;
static uint16_t stagedBytes, stagedReceived;
static uint8_t stagedSlot, stagedActive, stagedPrepared, stagedCommitted;
static uint8_t stagedKind, stagedApplied;
static uint8_t activationRecoveryChecked;
static uint8_t syncMutationLocked;
static uint8_t syncLockedSlot;
static uint32_t syncLockedTransaction;
static uint32_t lastFinishedTransaction;
static uint32_t lastAbortedTransaction;

enum
{
    PARAM_DIAG_JOURNAL_GET = 1u,
    PARAM_DIAG_JOURNAL_CONFLICT = 2u,
    PARAM_DIAG_JOURNAL_WRITE = 3u,
    PARAM_DIAG_FINISH_PREREQUISITE = 4u,
    PARAM_DIAG_LOCK_CONFLICT = 5u,
    PARAM_DIAG_PREPARE_SLOT = 6u,
    PARAM_DIAG_COMMIT_SLOT = 7u
};
#if defined(__ICCARM__)
__no_init uint32_t paramSyncActivationRollbackRetained;
__no_init uint32_t paramSyncActivationRollbackInverseRetained;
#else
static uint32_t paramSyncActivationRollbackRetained;
static uint32_t paramSyncActivationRollbackInverseRetained;
#endif

static uint8_t activationContext(uint8_t *oldSlot, uint8_t *newSlot)
{
    uint32_t word = paramSyncActivationRollbackRetained;
    if (paramSyncActivationRollbackInverseRetained != ~word ||
        (word & 0xFFFF0000u) != CORE1_ACTIVATE_ROLLBACK_MAGIC)
        return 0u;
    *oldSlot = (uint8_t) word;
    *newSlot = (uint8_t) (word >> 8);
}
```

```

        return (uint8_t) (*oldSlot >= 1u && *oldSlot <= 5u &&
            *newSlot >= 1u && *newSlot <= 5u);
    }

static void setActivationContext(uint8_t oldSlot, uint8_t newSlot)
{
    uint32_t word = CORE1_ACTIVATE_ROLLBACK_MAGIC |
        ((uint32_t) oldSlot | ((uint32_t) newSlot << 8));
    paramSyncActivationRollbackRetained = word;
    paramSyncActivationRollbackInverseRetained = ~word;
}

static void clearActivationContext(void)
{
    paramSyncActivationRollbackRetained = 0u;
    paramSyncActivationRollbackInverseRetained = UINT32_MAX;
}

static uint8_t rollbackActivation(void)
{
    uint8_t oldSlot, newSlot;
    (void) activationContext(&oldSlot, &newSlot);
    if (visionParamsAbortPendingActivation() != VISION_PARAMS_SLOT_OK) return 0u;
    clearActivationContext();
    return 1u;
}

static uint8_t recoverActivationAfterOwnerBoot(void)
{
    if (activationRecoveryChecked) return 1u;
    activationRecoveryChecked = 1u;
    return 1u;
}

static void putU16(uint8_t *p, uint16_t v)
{
    p[0] = (uint8_t) v;
    p[1] = (uint8_t) (v >> 8);
}

static void putU32(uint8_t *p, uint32_t v)
{
    p[0] = (uint8_t) v;
    p[1] = (uint8_t) (v >> 8);
    p[2] = (uint8_t) (v >> 16);
    p[3] = (uint8_t) (v >> 24);
}

static uint16_t getU16(const uint8_t *p)
{
    return (uint16_t) (p[0] | ((uint16_t) p[1] << 8));
}

static uint32_t getU32(const uint8_t *p)
{
    return (uint32_t) p[0] | ((uint32_t) p[1] << 8) | ((uint32_t) p[2] << 16) | ((uint32_t) p[3] << 24);
}

static uint32_t getOwnerBootGeneration(void)
{
    uint32_t previous;
    if (ownerBootGeneration != 0u) return ownerBootGeneration;
    previous = paramSyncOwnerBootRetained;
    if ((previous == 0u) ||
        (paramSyncOwnerBootInverseRetained != ~previous)) previous = 0u;
    previous++;
    if (previous == 0u) previous = 1u;
    paramSyncOwnerBootRetained = previous;
    paramSyncOwnerBootInverseRetained = ~previous;
    ownerBootGeneration = previous;
    return ownerBootGeneration;
}

static void encodeBlob(const visionParamsStruct *params, visionFlashBlobStruct *blob)
{
    memset(blob, 0, sizeof(*blob));
    blob->magic = VISION_FLASH_MAGIC;
    blob->version = VISION_FLASH_VERSION;
    blob->payloadBytes = sizeof(*blob);
    blob->param = *params;
    blob->crc = crc32IeeeCalc((const uint8_t *) blob, (uint32_t) offsetof(visionFlashBlobStruct, crc));
}

```

```

static uint8_t finishCoordinatorDecision(uint8_t operation,
uint32_t transactionId)
{
    paramSyncCoordinatorDecision decision;
    if (!paramSyncCoordinatorJournalGet(&decision)) return 0u;
    if (!decision.valid) return 1u;
    if (decision.decision != operation ||
        decision.transactionId != transactionId) return 0u;
    return paramSyncCoordinatorJournalClear();
}

static uint8_t coordinatorAllowsFinish(uint32_t transactionId)
{
    paramSyncCoordinatorDecision decision;
    return (uint8_t) (paramSyncCoordinatorJournalGet(&decision) &&
        decision.valid && decision.decision == PARAM_SYNC_OP_FINISH &&
        decision.transactionId == transactionId);
}

static uint8_t coordinatorAllowsAbort(uint32_t transactionId)
{
    paramSyncCoordinatorDecision decision;
    return (uint8_t) (paramSyncCoordinatorJournalGet(&decision) &&
        (!decision.valid ||
        (decision.decision == PARAM_SYNC_OP_ABORT &&
        decision.transactionId == transactionId)));
}

static uint8_t producerBootChangeAllowed(
const paramSyncMailboxRequest *request)
{
    paramSyncCoordinatorDecision decision;
    visionParamsPendingInfo pending;
    uint8_t oldSlot, newSlot;
    uint8_t activationPending;
    uint8_t transactionPending;
    uint32_t transactionId;

    if (!paramSyncCoordinatorJournalGet(&decision) ||
        visionParamsGetPending(&pending) != VISION_PARAMS_SLOT_OK) return 0u;
    activationPending = (uint8_t) (activationContext(&oldSlot, &newSlot) ||
        (pending.valid != 0u && pending.activationOnly != 0u));
    transactionPending = (uint8_t) (syncMutationLocked || stagedActive ||
        stagedPrepared || stagedCommitted || pending.valid || decision.valid);
    if ((activationPending == 0u) && (transactionPending == 0u)) return 1u;
    if ((request->op == PARAM_SYNC_OP_RECOVERY) &&
        (request->dataLength == 0u)) return 1u;
    if ((request->op != PARAM_SYNC_OP_FINISH &&
        request->op != PARAM_SYNC_OP_ABORT) ||
        (request->dataLength != 4u)) return 0u;

    transactionId = getU32(request->data);
    if (activationPending != 0u)
    {
        if ((transactionId != 0u) || stagedActive || stagedPrepared ||
            stagedCommitted) return 0u;
        if ((decision.valid != 0u) &&
            (request->op != decision.decision ||
            decision.transactionId != 0u)) return 0u;
        return 1u;
    }
    if ((pending.valid != 0u) &&
        (transactionId != pending.transactionId)) return 0u;
    if ((syncMutationLocked != 0u) &&
        (transactionId != syncLockedTransaction)) return 0u;
    if ((stagedActive || stagedPrepared || stagedCommitted) &&
        (transactionId != stagedTransaction)) return 0u;
    if ((decision.valid != 0u) &&
        (request->op != decision.decision ||
        transactionId != decision.transactionId)) return 0u;
    return 1u;
}

static uint8_t persistentMutationPending(void)
{
    visionParamsPendingInfo pending;
    visionParamsSlotStatus pendingStatus;
    uint8_t persistentPending = 1u;

```

```

    pendingStatus = visionParamsGetPending(&pending);
    /* GetPending 将 EMPTY 映射为 OK + valid=0; 真实读取错误保持 fail-closed。 */
    if (pendingStatus == VISION_PARAMS_SLOT_OK)
        persistentPending = pending.valid;
    return persistentPending;
}

uint8_t paramSyncOwnerServiceMutationLocked(void)
{
    return (uint8_t) (syncMutationLocked || stagedActive || stagedPrepared ||
        stagedCommitted || persistentMutationPending());
}

static void clearStaging(void)
{
    memset(&stagedBlob, 0, sizeof(stagedBlob));
    stagedTransaction = 0u;
    stagedExpectedCrc = 0u;
    stagedBytes = 0u;
    stagedReceived = 0u;
    stagedSlot = 0u;
    stagedActive = 0u;
    stagedPrepared = 0u;
    stagedCommitted = 0u;
    stagedKind = PARAM_STAGE_NONE;
    stagedApplied = 0u;
}

static void clearMutationLock(void)
{
    syncMutationLocked = 0u;
    syncLockedSlot = 0u;
    syncLockedTransaction = 0u;
}

static uint8_t decodeGlobalTransaction(const paramSyncMailboxRequest *request,
    uint8_t allowDefaultSlot, uint8_t *slot, uint32_t *transactionId)
{
    if (request->dataLength != 5u) return 0u;
    *slot = request->data[0];
    *transactionId = getU32(&request->data[1]);
    return (uint8_t) ((*transactionId != 0u) &&
        (*transactionId != UINT32_MAX) &&
        ((*slot >= 1u && *slot <= 5u) ||
            (allowDefaultSlot != 0u && *slot == 0u)));
}

static uint8_t transactionLocked(uint8_t slot, uint32_t transactionId)
{
    return (uint8_t) (syncMutationLocked != 0u &&
        syncLockedSlot == slot && syncLockedTransaction == transactionId);
}

static void publish(const paramSyncMailboxRequest *request, uint8_t status,
    const uint8_t *data, uint16_t length, uint32_t crc)
{
    paramSyncMailboxResponse r;
    memset(&r, 0, sizeof(r));
    responseSequence++;
    if (!responseSequence) responseSequence = 1;
    r.producerBootGeneration = getOwnerBootGeneration();
    r.sequence = responseSequence;
    r.transactionId = request->transactionId;
    r.offset = request->offset;
    r.payloadCrc32 = crc;
    r.dataLength = length;
    r.op = request->op;
    r.slot = request->slot;
    r.target = request->target;
    r.status = status;
    if (data && length) memcpy(r.data, data, length);
    (void) paramSyncMailboxPublishResponse(&r);
}

static void publishDiagnostic(const paramSyncMailboxRequest *request,
    uint8_t status, uint8_t detail)
{
    publish(request, status, &detail, 1u, 0u);
}

```

```

static void publishCoordinatorConflict(const paramSyncMailboxRequest *request,
    const paramSyncCoordinatorDecision *current)
{
    uint8_t out[8u];
    out[0] = PARAM_DIAG_JOURNAL_CONFLICT;
    out[1] = current->decision;
    putU32(&out[2], current->transactionId);
    out[6] = current->targetMask;
    out[7] = current->slot;
    publish(request, PARAM_SYNC_STATUS_FAILED, out, sizeof(out), 0u);
}

static void handleReadMeta(const paramSyncMailboxRequest *request)
{
    uint8_t out[26];
    visionParamsLibraryMeta meta;
    visionParamsStruct params;
    visionFlashBlobStruct blob;
    uint32_t blobCrc = 0u;
    memset(&meta, 0, sizeof(meta));
    if (request->dataLength != 1u || request->data[0] < 1u || request->data[0] > 5u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, 0, 0, 0);
        return;
    }
    if (visionParamsInspectSlot(request->data[0], &meta) !=
        VISION_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    if (meta.state == VISION_PARAMS_LIBRARY_VALID)
    {
        if (visionParamsReadSlot(request->data[0], &params, NULL) !=
            VISION_PARAMS_SLOT_OK)
        {
            publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
            return;
        }
        encodeBlob(&params, &blob);
        blobCrc = blob.crc;
    }
    out[0] = 1u;
    out[1] = request->data[0];
    out[2] = meta.state;
    out[3] = 0u;
    putU32(&out[4], VISION_FLASH_VERSION);
    putU32(&out[8], meta.generation);
    putU32(&out[12], blobCrc);
    putU32(&out[16], meta.transactionId);
    putU32(&out[20], CORE1_SCHEMA_HASH);
    putU16(&out[24], sizeof(visionFlashBlobStruct));
    publish(request, PARAM_SYNC_STATUS_OK, out, sizeof(out), 0u);
}

static void handleReadRuntimeMeta(const paramSyncMailboxRequest *request)
{
    uint8_t out[32];
    visionParamsStruct runtime, active;
    visionParamsRuntimeMeta meta;
    visionFlashBlobStruct blob;

    if (request->dataLength != 0u ||
        visionParamsReadRuntime(&runtime, &meta) != VISION_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    encodeBlob(&runtime, &blob);
    visionParamsGetActive(&active);
    memset(out, 0, sizeof(out));
    out[0] = 1u;
    out[1] = meta.valid;
    out[2] = meta.bank;
    out[3] = meta.pending;
    out[4] = (uint8_t) (memcmp(&runtime, &active, sizeof(runtime)) != 0);
    out[5] = meta.sourceSlot;
    out[6] = meta.dagnosticFlags;

```



```

        putU32(&out[8], VISION_FLASH_VERSION);
        putU32(&out[12], meta.generation);
        putU32(&out[16], blob.crc);
        putU32(&out[20], meta.transactionId);
        putU32(&out[24], CORE1_SCHEMA_HASH);
        putU16(&out[28], sizeof(blob));
        publish(request, PARAM_SYNC_STATUS_OK, out, sizeof(out), 0u);
    }

static void handleReadRuntimeChunk(const paramSyncMailboxRequest *request)
{
    visionParamsStruct params;
    visionFlashBlobStruct blob;
    uint8_t out[PARAM_SYNC_MAILBOX_DATA_MAX];
    uint16_t offset, length;

    if (request->dataLength != 4u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    offset = getU16(&request->data[0]);
    length = getU16(&request->data[2]);
    if (length == 0u || length > (PARAM_SYNC_MAILBOX_DATA_MAX - 4u) ||
        (uint32_t) offset + length > sizeof(blob) ||
        visionParamsReadRuntime(&params, NULL) != VISION_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    encodeBlob(&params, &blob);
    putU16(&out[0], offset);
    putU16(&out[2], sizeof(blob));
    memcpy(&out[4], ((const uint8_t *) &blob) + offset, length);
    publish(request, PARAM_SYNC_STATUS_OK, out, (uint16_t) (4u + length),
            blob.crc);
}

static void handleReadChunk(const paramSyncMailboxRequest *request)
{
    visionParamsStruct params;
    visionParamsSlotMeta meta;
    visionFlashBlobStruct blob;
    uint8_t out[PARAM_SYNC_MAILBOX_DATA_MAX];
    uint8_t slot;
    uint16_t offset, length;
    if (request->dataLength != 5u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, 0, 0, 0);
        return;
    }
    slot = request->data[0];
    offset = getU16(&request->data[1]);
    length = getU16(&request->data[3]);
    if (slot < 1u || slot > 5u || length == 0u || length > (PARAM_SYNC_MAILBOX_DATA_MAX - 5u) ||
        offset >= sizeof(blob) || ((uint32_t) offset + length) > sizeof(blob) ||
        visionParamsReadSlot(slot, &params, &meta) != VISION_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, 0, 0, 0);
        return;
    }
    encodeBlob(&params, &blob);
    out[0] = slot;
    putU16(&out[1], offset);
    putU16(&out[3], sizeof(blob));
    memcpy(&out[5], ((const uint8_t *) &blob) + offset, length);
    publish(request, PARAM_SYNC_STATUS_OK, out, (uint16_t) (5u + length), blob.crc);
}

static void handlePrepareLibraryFromLive(
    const paramSyncMailboxRequest *request)
{
    visionParamsStruct active;
    visionFlashBlobStruct blob;
    uint8_t slot;
    uint32_t transactionId;

    if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u)

```

```

{
    publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
    return;
}
if (stagedPrepared != 0u || stagedCommitted != 0u)
{
    publish(request, (stagedKind == PARAM_STAGE_LIBRARY &&
        stagedSlot == slot && stagedTransaction == transactionId) ?
        PARAM_SYNC_STATUS_OK : PARAM_SYNC_STATUS_BUSY, NULL, 0u,
        stagedExpectedCrc);
    return;
}
visionParamsGetActive(&active);
encodeBlob(&active, &blob);
if (visionParamsPrepareSlot(slot, &active, transactionId) !=
    VISION_PARAMS_SLOT_OK)
{
    publishDiagnostic(request, PARAM_SYNC_STATUS_FAILED,
        PARAM_DIAG_PREPARE_SLOT);
    return;
}
clearStaging();
stagedBlob = blob;
stagedSlot = slot;
stagedTransaction = transactionId;
stagedExpectedCrc = blob.crc;
stagedBytes = sizeof(blob);
stagedReceived = sizeof(blob);
stagedPrepared = 1u;
stagedKind = PARAM_STAGE_LIBRARY;
publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, blob.crc);
}

static void handlePrepareRuntime(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;
    visionParamsSlotStatus status;

    if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u ||
        slot == 0u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (stagedPrepared != 0u || stagedCommitted != 0u)
    {
        publish(request, (stagedKind == PARAM_STAGE_RUNTIME &&
            stagedSlot == slot && stagedTransaction == transactionId) ?
            PARAM_SYNC_STATUS_OK : PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
        return;
    }
    status = visionParamsPrepareRuntimeFromSlot(slot, transactionId);
    if (status != VISION_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    clearStaging();
    stagedSlot = slot;
    stagedTransaction = transactionId;
    stagedPrepared = 1u;
    stagedKind = PARAM_STAGE_RUNTIME;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleCommitRuntime(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;
    if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u ||
        stagedKind != PARAM_STAGE_RUNTIME || stagedPrepared == 0u ||
        stagedSlot != slot || stagedTransaction != transactionId)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
}

```

```

    if (stagedCommitted == 0u &&
        visionParamsCommitRuntime(transactionId) != VISION_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    stagedCommitted = 1u;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleApplyRuntime(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;
    if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u ||
        stagedKind != PARAM_STAGE_RUNTIME || stagedCommitted == 0u ||
        stagedSlot != slot || stagedTransaction != transactionId)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (stagedApplied == 0u &&
        visionParamsApplyPreparedRuntime(transactionId) !=
            VISION_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    stagedApplied = 1u;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handlePrepareBegin(const paramSyncMailboxRequest *r)
{
    uint32_t transactionId = r->dataLength == 19u ?
        getU32(&r->data[1]) : 0u;
    if (r->dataLength != 19u || r->data[0] < 1u || r->data[0] > 5u ||
        transactionId == 0u || transactionId == UINT32_MAX ||
        getU32(&r->data[5]) != VISION_FLASH_VERSION ||
        getU32(&r->data[9]) != CORE1_SCHEMA_HASH ||
        getU16(&r->data[17]) != sizeof(stagedBlob))
    {
        publish(r, PARAM_SYNC_STATUS_INVALID, 0, 0, 0);
        return;
    }
    if (stagedPrepared || stagedCommitted)
    {
        if (visionParamsAbortPreparedSlot(stagedTransaction) != VISION_PARAMS_SLOT_OK)
        {
            publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
            return;
        }
    }
    clearStaging();
    memset(&stagedBlob, 0, sizeof(stagedBlob));
    stagedSlot = r->data[0];
    stagedTransaction = transactionId;
    stagedExpectedCrc = getU32(&r->data[13]);
    stagedBytes = getU16(&r->data[17]);
    stagedReceived = 0u;
    stagedActive = 1u;
    stagedPrepared = 0u;
    stagedCommitted = 0u;
    stagedKind = PARAM_STAGE_LIBRARY;
    publish(r, PARAM_SYNC_STATUS_OK, 0, 0, 0);
}

static void handlePrepareChunk(const paramSyncMailboxRequest *r)
{
    uint8_t slot;
    uint16_t offset, length;
    if (!stagedActive || r->dataLength < 5u)
    {
        publish(r, PARAM_SYNC_STATUS_INVALID, 0, 0, 0);
        return;
    }
    slot = r->data[0];
    offset = getU16(&r->data[1]);

```

```

length = getU16(&r->data[3]);
if (slot != stagedSlot || offset != stagedReceived || length != (uint16_t) (r->dataLength - 5u) ||
    ((uint32_t) offset + length) > stagedBytes)
{
    stagedActive = 0u;
    publish(r, PARAM_SYNC_STATUS_INVALID, 0, 0, 0);
    return;
}
memcpy(((uint8_t *) &stagedBlob) + offset, &r->data[5], length);
stagedReceived = (uint16_t) (stagedReceived + length);
publish(r, PARAM_SYNC_STATUS_OK, 0, 0, 0);
}

static void handlePrepareFinish(const paramSyncMailboxRequest *r)
{
    uint32_t computed;
    if (!stagedActive || r->dataLength != 1u || r->data[0] != stagedSlot || stagedReceived != stagedBytes ||
        stagedBlob.magic != VISION_FLASH_MAGIC || stagedBlob.version != VISION_FLASH_VERSION ||
        stagedBlob.payloadBytes != sizeof(stagedBlob))
    {
        stagedActive = 0u;
        publish(r, PARAM_SYNC_STATUS_INVALID, 0, 0, 0);
        return;
    }
    computed = crc32leeeCalc((const uint8_t *) &stagedBlob, (uint32_t) offsetof(visionFlashBlobStruct, crc));
    if (computed != stagedBlob.crc || computed != stagedExpectedCrc ||
        visionParamsPrepareSlot(stagedSlot, &stagedBlob.param, stagedTransaction) != VISION_PARAMS_SLOT_OK)
    {
        stagedActive = 0u;
        publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
    stagedActive = 0u;
    stagedPrepared = 1u;
    publish(r, PARAM_SYNC_STATUS_OK, 0, 0, computed);
}

static void handleCommit(const paramSyncMailboxRequest *r)
{
    uint8_t slot;
    uint32_t transactionId;
    if (r->dataLength != 5u || r->data[0] < 1u || r->data[0] > 5u)
    {
        publish(r, PARAM_SYNC_STATUS_INVALID, 0, 0, 0);
        return;
    }
    slot = r->data[0];
    transactionId = getU32(&r->data[1]);
    if (stagedKind != PARAM_STAGE_LIBRARY || !stagedPrepared ||
        slot != stagedSlot || transactionId != stagedTransaction)
    {
        publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
    if (!stagedCommitted &&
        visionParamsCommitPreparedSlot(slot, transactionId, 0u) != VISION_PARAMS_SLOT_OK)
    {
        publishDiagnostic(r, PARAM_SYNC_STATUS_FAILED,
            PARAM_DIAG_COMMIT_SLOT);
        return;
    }
    stagedCommitted = 1u;
    publish(r, PARAM_SYNC_STATUS_OK, 0, 0, 0);
}

static void handleActivate(const paramSyncMailboxRequest *r)
{
    uint8_t slot;
    uint8_t oldSlot, pendingSlot;
    uint32_t transactionId;
    if (r->dataLength != 5u || r->data[0] < 1u || r->data[0] > 5u)
    {
        publish(r, PARAM_SYNC_STATUS_INVALID, 0, 0, 0);
        return;
    }
    slot = r->data[0];
    transactionId = getU32(&r->data[1]);
    if (transactionId == 0u)

```

```

{
    if (!syncMutationLocked || syncLockedSlot != slot || stagedActive ||
        stagedPrepared || stagedCommitted)
    {
        publish(r, PARAM_SYNC_STATUS_BUSY, 0, 0, 0);
        return;
    }
    if (activationContext(&oldSlot, &pendingSlot))
    {
        publish(r, pendingSlot == slot ? PARAM_SYNC_STATUS_OK : PARAM_SYNC_STATUS_BUSY, 0, 0, 0);
        return;
    }
    oldSlot = visionParamsGetActiveSlot();
    if (oldSlot < 1u || oldSlot > 5u)
    {
        publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
    setActivationContext(oldSlot, slot);
}
if ((transactionId != 0u &&
    (!stagedCommitted || transactionId != stagedTransaction || slot != stagedSlot)) ||
    (transactionId == 0u ? visionParamsActivateSlotPending(slot) :
        visionParamsActivatePreparedSlot(slot, transactionId)) !=
        VISION_PARAMS_SLOT_OK)
{
    if (transactionId == 0u) (void) rollbackActivation();
    publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
    return;
}
publish(r, PARAM_SYNC_STATUS_OK, 0, 0, 0);
}

```

```

static void handleAbort(const paramSyncMailboxRequest *r)
{
    uint32_t transactionId;
    if (r->dataLength != 4u)
    {
        publish(r, PARAM_SYNC_STATUS_INVALID, 0, 0, 0);
        return;
    }
    transactionId = getU32(r->data);
    if (transactionId == 0u)
    {
        uint8_t ok = (uint8_t) (
            finishCoordinatorDecision(PARAM_SYNC_OP_ABORT, 0u) &&
            rollbackActivation());
        if (ok) clearMutationLock();
        publish(r, ok ? PARAM_SYNC_STATUS_OK : PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
    if (!stagedActive && !stagedPrepared && !stagedCommitted &&
        transactionId == lastAbortedTransaction)
    {
        publish(r, finishCoordinatorDecision(PARAM_SYNC_OP_ABORT,
            transactionId) ? PARAM_SYNC_STATUS_OK :
            PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
    if (!stagedActive && !stagedPrepared && !stagedCommitted)
    {
        visionParamsPendingInfo pending;
        if (visionParamsGetPending(&pending) != VISION_PARAMS_SLOT_OK)
        {
            publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
            return;
        }
    }
    if (!pending.valid)
    {
        lastAbortedTransaction = transactionId;
        clearStaging();
        clearMutationLock();
        publish(r, finishCoordinatorDecision(PARAM_SYNC_OP_ABORT,
            transactionId) ? PARAM_SYNC_STATUS_OK :
            PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
}
if ((stagedActive || stagedPrepared || stagedCommitted) &&

```

```

        transactionId != stagedTransaction)
    {
        publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
    if (!coordinatorAllowsAbort(transactionId))
    {
        publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
    if (visionParamsAbortPreparedSlot(transactionId) != VISION_PARAMS_SLOT_OK)
    {
        publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
    lastAbortedTransaction = transactionId;
    clearStaging();
    clearMutationLock();
    if (!finishCoordinatorDecision(PARAM_SYNC_OP_ABORT, transactionId))
    {
        publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
    publish(r, PARAM_SYNC_STATUS_OK, 0, 0, 0);
}

static void handleFinish(const paramSyncMailboxRequest *r)
{
    uint32_t transactionId;
    if (r->dataLength != 4u)
    {
        publish(r, PARAM_SYNC_STATUS_INVALID, 0, 0, 0);
        return;
    }
    transactionId = getU32(r->data);
    if (transactionId == 0u)
    {
        visionParamsPendingInfo pending;
        if (visionParamsGetPending(&pending) != VISION_PARAMS_SLOT_OK)
        {
            publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
            return;
        }
        if (!pending.valid)
        {
            clearActivationContext();
            clearMutationLock();
            publish(r, finishCoordinatorDecision(PARAM_SYNC_OP_FINISH, 0u) ?
                PARAM_SYNC_STATUS_OK : PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
            return;
        }
        if (!pending.activationOnly || !coordinatorAllowsFinish(0u) ||
            visionParamsFinishPendingActivation() != VISION_PARAMS_SLOT_OK)
        {
            publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
            return;
        }
        clearActivationContext();
        clearMutationLock();
        publish(r, finishCoordinatorDecision(PARAM_SYNC_OP_FINISH, 0u) ?
            PARAM_SYNC_STATUS_OK : PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
    if (!stagedActive && !stagedPrepared && !stagedCommitted &&
        transactionId == lastFinishedTransaction)
    {
        publish(r, finishCoordinatorDecision(PARAM_SYNC_OP_FINISH,
            transactionId) ? PARAM_SYNC_STATUS_OK :
            PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
    if ((stagedActive || stagedPrepared || stagedCommitted) &&
        transactionId != stagedTransaction)
    {
        publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
    {
        visionParamsPendingInfo pending;

```

```

    if (visionParamsGetPending(&pending) != VISION_PARAMS_SLOT_OK ||
        (pending.valid && !coordinatorAllowsFinish(transactionId)))
    {
        publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
    if (!pending.valid && !stagedActive && !stagedPrepared && !stagedCommitted)
    {
        lastFinishedTransaction = transactionId;
        clearStaging();
        clearMutationLock();
        publish(r, finishCoordinatorDecision(PARAM_SYNC_OP_FINISH,
            transactionId) ? PARAM_SYNC_STATUS_OK :
                PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
}
if ((stagedKind == PARAM_STAGE_LIBRARY && stagedCommitted == 0u) ||
    (stagedKind == PARAM_STAGE_RUNTIME &&
        (stagedCommitted == 0u || stagedApplied == 0u)) ||
    visionParamsFinishPreparedSlot(transactionId) != VISION_PARAMS_SLOT_OK)
{
    publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
    return;
}
lastFinishedTransaction = transactionId;
clearStaging();
clearMutationLock();
if (!finishCoordinatorDecision(PARAM_SYNC_OP_FINISH, transactionId))
{
    publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
    return;
}
publish(r, PARAM_SYNC_STATUS_OK, 0, 0, 0);
}
}

```

```

static void handleDecide(const paramSyncMailboxRequest *r)
{
    uint8_t operation, slot, targetMask;
    uint32_t transactionId;
    visionParamsPendingInfo pending;
    paramSyncCoordinatorDecision current;
    if (r->dataLength != 7u)
    {
        publish(r, PARAM_SYNC_STATUS_INVALID, 0, 0, 0);
        return;
    }
    operation = r->data[0];
    slot = r->data[1];
    transactionId = getU32(&r->data[2]);
    targetMask = r->data[6];
    if (slot > 5u || targetMask == 0u ||
        (targetMask & ~0x1Fu) != 0u)
    {
        publish(r, PARAM_SYNC_STATUS_INVALID, 0, 0, 0);
        return;
    }
    if (!paramSyncCoordinatorJournalGet(&current))
    {
        publishDiagnostic(r, PARAM_SYNC_STATUS_FAILED,
            PARAM_DIAG_JOURNAL_GET);
        return;
    }
    if (operation == PARAM_SYNC_OP_ABORT)
    {
        if (current.valid &&
            (current.decision != PARAM_SYNC_OP_ABORT ||
                current.transactionId != transactionId || current.slot != slot))
        {
            publishCoordinatorConflict(r, &current);
            return;
        }
        if (!paramSyncCoordinatorJournalWrite(operation, transactionId,
            targetMask, slot))
        {
            publishDiagnostic(r, PARAM_SYNC_STATUS_FAILED,
                PARAM_DIAG_JOURNAL_WRITE);
            return;
        }
    }
}

```

```

    }
    else if (operation == PARAM_SYNC_OP_FINISH)
    {
        if (targetMask != 0x1Fu ||
            visionParamsGetPending(&pending) != VISION_PARAMS_SLOT_OK ||
            !pending.valid || pending.slot != slot ||
            ((transactionId == 0u) ? (!pending.activationOnly) :
                (pending.activationOnly || pending.transactionId != transactionId)) ||
            (current.valid && current.transactionId != transactionId))
        {
            publishDiagnostic(r, PARAM_SYNC_STATUS_FAILED,
                PARAM_DIAG_FINISH_PREREQUISITE);
            return;
        }
        if (!paramSyncCoordinatorJournalWrite(operation, transactionId,
            targetMask, slot))
        {
            publishDiagnostic(r, PARAM_SYNC_STATUS_FAILED,
                PARAM_DIAG_JOURNAL_WRITE);
            return;
        }
    }
    else
    {
        publish(r, PARAM_SYNC_STATUS_INVALID, 0, 0, 0);
        return;
    }
    publish(r, PARAM_SYNC_STATUS_OK, 0, 0, 0);
}

```

```

static void handleRecovery(const paramSyncMailboxRequest *r)
{
    uint8_t out[7];
    paramSyncCoordinatorDecision decision;
    visionParamsPendingInfo pending;
    if (r->dataLength != 0u ||
        !paramSyncCoordinatorJournalGet(&decision) ||
        visionParamsGetPending(&pending) != VISION_PARAMS_SLOT_OK)
    {
        publish(r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
    memset(out, 0, sizeof(out));
    if (decision.valid)
    {
        out[0] = decision.decision;
        putU32(&out[1], decision.transactionId);
        out[5] = decision.targetMask;
        out[6] = decision.slot;
    }
    else if (pending.valid)
    {
        out[0] = PARAM_SYNC_OP_ABORT;
        putU32(&out[1], pending.activationOnly ? 0u : pending.transactionId);
        out[5] = 0x1Fu;
        out[6] = pending.slot;
    }
    publish(r, PARAM_SYNC_STATUS_OK, out, sizeof(out), 0u);
}

```

```

static void handleLock(const paramSyncMailboxRequest *r)
{
    uint8_t oldSlot, newSlot;
    uint32_t transactionId = (r->dataLength == 5u) ?
        getU32(&r->data[1]) : 0u;
    if (r->dataLength != 5u || r->data[0] > 5u ||
        transactionId == 0u || transactionId == UINT32_MAX ||
        (syncMutationLocked && (syncLockedSlot != r->data[0] ||
            syncLockedTransaction != transactionId)) ||
        stagedActive || stagedPrepared || stagedCommitted ||
        persistentMutationPending() ||
        activationContext(&oldSlot, &newSlot))
    {
        publishDiagnostic(r, PARAM_SYNC_STATUS_BUSY,
            PARAM_DIAG_LOCK_CONFLICT);
        return;
    }
    syncMutationLocked = 1u;
    syncLockedSlot = r->data[0];
}

```



```

        syncLockedTransaction = transactionId;
        publish(r, PARAM_SYNC_STATUS_OK, 0, 0, 0);
    }

static uint8_t retainedRequestMayResume(
    const paramSyncMailboxRequest *request)
{
    if (request->target != 1u) return 0u;
    return (uint8_t) (request->op == PARAM_SYNC_OP_QUERY ||
        request->op == PARAM_SYNC_OP_READ_META ||
        request->op == PARAM_SYNC_OP_READ_CHUNK ||
        request->op == PARAM_SYNC_OP_READ_RUNTIME_META ||
        request->op == PARAM_SYNC_OP_READ_RUNTIME_CHUNK ||
        request->op == PARAM_SYNC_OP_RECOVERY ||
        request->op == PARAM_SYNC_OP_ABORT ||
        request->op == PARAM_SYNC_OP_FINISH ||
        request->op == PARAM_SYNC_OP_DECIDE);
}

void paramSyncOwnerServiceInit(void)
{
    paramSyncMailboxRequest retainedRequest;
    lastBoot = 0u;
    lastSequence = 0u;
    responseSequence = 0u;
    (void) getOwnerBootGeneration();
    if (paramSyncMailboxTryReadRequest(&retainedRequest) != 0u &&
        retainedRequestMayResume(&retainedRequest) == 0u)
    {
        lastBoot = retainedRequest.producerBootGeneration;
        lastSequence = retainedRequest.sequence;
        if (retainedRequest.target == 1u)
            publish(&retainedRequest, PARAM_SYNC_STATUS_BUSY, 0, 0, 0);
    }
}

void paramSyncOwnerServicePoll(void)
{
    paramSyncMailboxRequest r;
    if (!paramSyncMailboxTryReadRequest(&r)) return;
    if (r.producerBootGeneration == lastBoot && r.sequence == lastSequence) return;
    if (r.producerBootGeneration == lastBoot &&
        (int32_t) (r.sequence - lastSequence) <= 0) return;
    if (lastBoot != 0u && r.producerBootGeneration != lastBoot)
    {
        if (producerBootChangeAllowed(&r) == 0u)
        {
            if (r.target == 1u) publish(&r, PARAM_SYNC_STATUS_BUSY, 0, 0, 0);
            return;
        }
    }
    lastBoot = r.producerBootGeneration;
    lastSequence = r.sequence;
    if (r.target != 1u) return;
    if (!recoverActivationAfterOwnerBoot())
    {
        publish(&r, PARAM_SYNC_STATUS_FAILED, 0, 0, 0);
        return;
    }
    switch (r.op)
    {
        case PARAM_SYNC_OP_READ_META: handleReadMeta(&r); break;
        case PARAM_SYNC_OP_READ_CHUNK: handleReadChunk(&r); break;
        case PARAM_SYNC_OP_READ_RUNTIME_META: handleReadRuntimeMeta(&r); break;
        case PARAM_SYNC_OP_READ_RUNTIME_CHUNK: handleReadRuntimeChunk(&r); break;
        case PARAM_SYNC_OP_PREPARE_LIBRARY_FROM_LIVE:
            handlePrepareLibraryFromLive(&r); break;
        case PARAM_SYNC_OP_PREPARE_RUNTIME_FROM_SLOT:
            handlePrepareRuntime(&r); break;
        case PARAM_SYNC_OP_COMMIT_RUNTIME: handleCommitRuntime(&r); break;
        case PARAM_SYNC_OP_APPLY_RUNTIME: handleApplyRuntime(&r); break;
        case PARAM_SYNC_OP_PREPARE_BEGIN: handlePrepareBegin(&r); break;
        case PARAM_SYNC_OP_PREPARE_CHUNK: handlePrepareChunk(&r); break;
        case PARAM_SYNC_OP_PREPARE_VERIFY: handlePrepareFinish(&r); break;
        case PARAM_SYNC_OP_COMMIT: handleCommit(&r); break;
        case PARAM_SYNC_OP_ACTIVATE: handleActivate(&r); break;
        case PARAM_SYNC_OP_ABORT: handleAbort(&r); break;
        case PARAM_SYNC_OP_FINISH: handleFinish(&r); break;
        case PARAM_SYNC_OP_LOCK: handleLock(&r); break;
    }
}

```

```

        case PARAM_SYNC_OP_DECIDE: handleDecide(&r); break;
        case PARAM_SYNC_OP_RECOVERY: handleRecovery(&r); break;
        default: publish(&r, PARAM_SYNC_STATUS_REJECTED, 0, 0, 0); break;
    }
}

```

core1_cyt4bb7/project/code/comm/THVD1450.c

SHA-256

:

f32b00f61dd9fabb1dd83da5f0ad656a4ee1d1c85208b1240c32ce28c67804ee

代码 (c)

```

#include "THVD1450.h"

#include <string.h>

#include "zf_device_type.h"

#include "zf_driver_gpio.h"
#include "zf_driver_uart.h"

#include "pinMap.h" /* RS-485 UART 编号/引脚/方向脚单一真相源 */

#define THVD1450_UART_NUM    (RS485_UART_NUM)
#define THVD1450_UART_TX_PIN (RS485_UART_TX_PIN)
#define THVD1450_UART_RX_PIN (RS485_UART_RX_PIN)

#define THVD1450_DIR_PIN      (RS485_DIR_PIN)
#define THVD1450_RX_FRAME_QUEUE_SIZE (4u)
/* 单次 Poll 排空字节预算: RS-485 未接/悬空/噪声时 RX FIFO 可能持续有字节, 无界 while drain
 * 会拖住前台(饿死 10ms RS-485 命令节拍 / 200ms 核间心跳)。限 64 字节/轮, 余下留下一轮。
 * 帧队列满仍由 thvd1450PushFrame 丢最旧策略兜底。 */
#define THVD1450_POLL_BYTE_BUDGET  (64u)
#if THVD1450_UART_BAUDRATE != 1 000 000u
#error "Core1-Core2 RS-485 baud contract requires 1 Mbps"
#endif

static uint8_t thvd1450TxBuffer[THVD1450_FRAME_LENGTH_MAX];

static thvd1450ParserStruct thvd1450Parser;
static thvd1450FrameStruct thvd1450RxQueue[THVD1450_RX_FRAME_QUEUE_SIZE];

static volatile uint8_t thvd1450RxHead = 0u;
static volatile uint8_t thvd1450RxTail = 0u;
static volatile uint8_t thvd1450RxCount = 0u;

static uint8_t thvd1450NextIndex(uint8_t index)
{
    index++;
    if (index >= THVD1450_RX_FRAME_QUEUE_SIZE)
    {
        index = 0u;
    }

    return index;
}

static void thvd1450PushFrame(const thvd1450FrameStruct *frame)
{
    if (NULL == frame)
    {
        return;
    }

    if (thvd1450RxCount >= THVD1450_RX_FRAME_QUEUE_SIZE)
    {
        thvd1450RxHead = thvd1450NextIndex(thvd1450RxHead);
        thvd1450RxCount--;
    }

    thvd1450RxQueue[thvd1450RxTail] = *frame;
    thvd1450RxTail = thvd1450NextIndex(thvd1450RxTail);
    thvd1450RxCount++;
}

```

```

static void thvd1450HandleByte(uint8_t data)
{
    thvd1450FrameStruct rxFrame;
    thvd1450StatusEnum status;

    status = thvd1450ProtocolInputByte(&thvd1450Parser, data, &rxFrame);
    if (THVD1450_STATUS_FRAME_READY == status)
    {
        thvd1450PushFrame(&rxFrame);
    }
}

void thvd1450Init(void)
{
    thvd1450ProtocolInit(&thvd1450Parser);
    memset(thvd1450RxQueue, 0, sizeof(thvd1450RxQueue));
    thvd1450RxHead = 0u;
    thvd1450RxTail = 0u;
    thvd1450RxCount = 0u;

    uart_init(THVD1450_UART_NUM, THVD1450_UART_BAUDRATE, THVD1450_UART_TX_PIN, THVD1450_UART_RX_PIN);
    gpio_init(THVD1450_DIR_PIN, GPO, GPIO_LOW, GPO_PUSH_PULL);

    thvd1450SetMode(THVD1450_MODE_RECEIVE);

    set_wireless_type(RECEIVER_UART, thvd1450RxIrqHandler);
    uart_rx_interrupt(THVD1450_UART_NUM, 1u);
}

void thvd1450SetMode(thvd1450ModeEnum mode)
{
    if (THVD1450_MODE_TRANSMIT == mode)
    {
        gpio_high(THVD1450_DIR_PIN);
    }
    else
    {
        gpio_low(THVD1450_DIR_PIN);
    }
}

void thvd1450Poll(void)
{
    uint8_t data;
    uint8_t budget = THVD1450_POLL_BYTE_BUDGET;

    uart_rx_interrupt(THVD1450_UART_NUM, 0u);
    /* 限预算排空: 达 THVD1450_POLL_BYTE_BUDGET 即退出本轮, 剩余字节留下一轮 Poll 处理。 */
    while (budget > 0u && uart_query_byte(THVD1450_UART_NUM, &data))
    {
        thvd1450HandleByte(data);
        budget--;
    }
    uart_rx_interrupt(THVD1450_UART_NUM, 1u);
}

thvd1450StatusEnum thvd1450SendBytes(const uint8_t *payload, uint8_t length)
{
    uint8_t frameLength = 0u;
    thvd1450StatusEnum status;

    status = thvd1450ProtocolEncode(
        payload,
        length,
        thvd1450TxBuffer,
        (uint8_t)sizeof(thvd1450TxBuffer),
        &frameLength);
    if (THVD1450_STATUS_OK != status)
    {
        return status;
    }

    thvd1450SetMode(THVD1450_MODE_TRANSMIT);
    uart_write_buffer(THVD1450_UART_NUM, thvd1450TxBuffer, frameLength);
    thvd1450SetMode(THVD1450_MODE_RECEIVE);

    return THVD1450_STATUS_OK;
}

```

```

}

void thvd1450RxIrqHandler(void)
{
    uint8_t data;

    if (uart_query_byte(THVD1450_UART_NUM, &data))
    {
        thvd1450HandleByte(data);
    }
}

thvd1450StatusEnum thvd1450ReadFrame(thvd1450FrameStruct *frame)
{
    if (NULL == frame)
    {
        return THVD1450_STATUS_INVALID_PARAM;
    }

    uart_rx_interrupt(THVD1450_UART_NUM, 0u);
    if (0u == thvd1450RxCount)
    {
        uart_rx_interrupt(THVD1450_UART_NUM, 1u);
        return THVD1450_STATUS_NO_FRAME;
    }

    *frame = thvd1450RxQueue[thvd1450RxHead];
    thvd1450RxHead = thvd1450NextIndex(thvd1450RxHead);
    thvd1450RxCount--;
    uart_rx_interrupt(THVD1450_UART_NUM, 1u);
    return THVD1450_STATUS_OK;
}

```

core1_cyt4bb7/project/code/comm/THVD1450.h

SHA-256

:

be1a0dfdd7f105b85337f2bd5cf3769675f2e7742b9a172927adbf9bee48d875

代码 (c)

```

#ifndef _THVD1450_H_
#define _THVD1450_H_

#include <stdint.h>

#include "THVD1450_protocol.h"

typedef enum
{
    THVD1450_MODE_RECEIVE = 0u,
    THVD1450_MODE_TRANSMIT,
} thvd1450ModeEnum;

void thvd1450Init(void);
void thvd1450SetMode(thvd1450ModeEnum mode);

void thvd1450Poll(void);
void thvd1450RxIrqHandler(void);

thvd1450StatusEnum thvd1450ReadFrame(thvd1450FrameStruct *frame);

thvd1450StatusEnum thvd1450SendBytes(const uint8_t *payload, uint8_t length);

#endif // _THVD1450_H_

```

core1_cyt4bb7/project/code/comm/THVD1450_protocol.c

SHA-256

:

634e0a17539c065b6fda54bd997e2dfbe871fe8480c45d41630e20389016056f

代码 (c)

```
#include "THVD1450_protocol.h"

#include <string.h>

#define THVD1450_PARSE_WAIT_HEAD   (0u)
#define THVD1450_PARSE_WAIT_LENGTH (1u)
#define THVD1450_PARSE_WAIT_PAYLOAD (2u)
#define THVD1450_PARSE_WAIT_CHECKSUM (3u)
#define THVD1450_PARSE_WAIT_TAIL   (4u)

static void thvd1450ProtocolRestartFromHead(thvd1450ParserStruct *parser)
{
    thvd1450ProtocolReset(parser);
    parser->state = THVD1450_PARSE_WAIT_LENGTH;
}

void thvd1450ProtocolInit(thvd1450ParserStruct *parser)
{
    thvd1450ProtocolReset(parser);
}

void thvd1450ProtocolReset(thvd1450ParserStruct *parser)
{
    if (NULL == parser)
    {
        return;
    }

    memset(parser, 0, sizeof(*parser));
    parser->state = THVD1450_PARSE_WAIT_HEAD;
}

uint8_t thvd1450ProtocolChecksum(const uint8_t *payload, uint8_t length)
{
    uint8_t checksum = length;
    uint8_t i;

    for (i = 0u; i < length; i++)
    {
        checksum ^= payload[i];
    }

    return checksum;
}

thvd1450StatusEnum thvd1450ProtocolEncode(
    const uint8_t *payload,
    uint8_t length,
    uint8_t *frameBuffer,
    uint8_t frameBufferSize,
    uint8_t *frameLength)
{
    if ((NULL == frameBuffer) || (NULL == frameLength) || ((0u != length) && (NULL == payload)))
    {
        return THVD1450_STATUS_INVALID_PARAM;
    }

    if (length > THVD1450_PAYLOAD_LENGTH_MAX)
    {
        return THVD1450_STATUS_LENGTH_ERROR;
    }

    if (frameBufferSize < (uint8_t)(length + THVD1450_FRAME_OVERHEAD))
    {
        return THVD1450_STATUS_LENGTH_ERROR;
    }

    frameBuffer[0] = THVD1450_FRAME_HEAD;
    frameBuffer[1] = length;

    if (0u != length)
    {
        memcpy(&frameBuffer[2], payload, length);
    }

    frameBuffer[2u + length] = thvd1450ProtocolChecksum(payload, length);
}
```

```

frameBuffer[3u + length] = THVD1450_FRAME_TAIL;
*frameLength = (uint8_t)(length + THVD1450_FRAME_OVERHEAD);

return THVD1450_STATUS_OK;
}

thvd1450StatusEnum thvd1450ProtocolInputByte(
    thvd1450ParserStruct *parser,
    uint8_t byte,
    thvd1450FrameStruct *outFrame)
{
    if (NULL == parser)
    {
        return THVD1450_STATUS_INVALID_PARAM;
    }

    switch (parser->state)
    {
        case THVD1450_PARSE_WAIT_HEAD:
        {
            if (THVD1450_FRAME_HEAD == byte)
            {
                parser->state = THVD1450_PARSE_WAIT_LENGTH;
            }
            return THVD1450_STATUS_NO_FRAME;
        }

        case THVD1450_PARSE_WAIT_LENGTH:
        {
            if (byte > THVD1450_PAYLOAD_LENGTH_MAX)
            {
                thvd1450ProtocolReset(parser);
                return THVD1450_STATUS_LENGTH_ERROR;
            }

            parser->expectedLength = byte;
            parser->frame.length = byte;
            parser->checksum = byte;
            parser->index = 0u;
            parser->state = (0u == byte) ? THVD1450_PARSE_WAIT_CHECKSUM : THVD1450_PARSE_WAIT_PAYLOAD;
            return THVD1450_STATUS_NO_FRAME;
        }

        case THVD1450_PARSE_WAIT_PAYLOAD:
        {
            parser->frame.payload[parser->index] = byte;
            parser->checksum ^= byte;
            parser->index++;

            if (parser->index >= parser->expectedLength)
            {
                parser->state = THVD1450_PARSE_WAIT_CHECKSUM;
            }
            return THVD1450_STATUS_NO_FRAME;
        }

        case THVD1450_PARSE_WAIT_CHECKSUM:
        {
            if (byte != parser->checksum)
            {
                if (THVD1450_FRAME_HEAD == byte)
                {
                    thvd1450ProtocolRestartFromHead(parser);
                }
                else
                {
                    thvd1450ProtocolReset(parser);
                }
                return THVD1450_STATUS_CHECKSUM_ERROR;
            }

            parser->state = THVD1450_PARSE_WAIT_TAIL;
            return THVD1450_STATUS_NO_FRAME;
        }

        case THVD1450_PARSE_WAIT_TAIL:
        {
            if (THVD1450_FRAME_TAIL == byte)
            {

```

```

        if (NULL != outFrame)
        {
            *outFrame = parser->frame;
        }
        thvd1450ProtocolReset(parser);
        return THVD1450_STATUS_FRAME_READY;
    }

    if (THVD1450_FRAME_HEAD == byte)
    {
        thvd1450ProtocolRestartFromHead(parser);
    }
    else
    {
        thvd1450ProtocolReset(parser);
    }
    return THVD1450_STATUS_TAIL_ERROR;
}

default:
{
    thvd1450ProtocolReset(parser);
    break;
}
}

return THVD1450_STATUS_NO_FRAME;
}

```

core1_cyt4bb7/project/code/comm/THVD1450_protocol.h

SHA-256

:

0cdc3470af32c7fe3808e590f1ac4f4833c1426b514e67b9e228bf5529f56ad6

代码 (c)

```

#ifndef _THVD1450_PROTOCOL_H_
#define _THVD1450_PROTOCOL_H_

#include <stdint.h>

#define THVD1450_FRAME_HEAD      (0x7Au)
#define THVD1450_UART_BAUDRATE  (1 000 000u)
#define THVD1450_FRAME_TAIL      (0x55u)
#define THVD1450_DATA_LENGTH_MAX (16u)
#define THVD1450_PAYLOAD_LENGTH_MAX (THVD1450_DATA_LENGTH_MAX)
#define THVD1450_FRAME_OVERHEAD  (4u)
#define THVD1450_FRAME_LENGTH_MAX (THVD1450_PAYLOAD_LENGTH_MAX + THVD1450_FRAME_OVERHEAD)

typedef enum
{
    THVD1450_STATUS_OK = 0u,
    THVD1450_STATUS_FRAME_READY,
    THVD1450_STATUS_NO_FRAME,
    THVD1450_STATUS_INVALID_PARAM,
    THVD1450_STATUS_LENGTH_ERROR,
    THVD1450_STATUS_CHECKSUM_ERROR,
    THVD1450_STATUS_TAIL_ERROR,
} thvd1450StatusEnum;

typedef struct
{
    uint8_t length;
    uint8_t payload[THVD1450_PAYLOAD_LENGTH_MAX];
} thvd1450FrameStruct;

typedef struct
{
    uint8_t state;
    uint8_t index;
    uint8_t expectedLength;
    uint8_t checksum;
    thvd1450FrameStruct frame;
} thvd1450ParserStruct;

```

```

void thvd1450ProtocolInit(thvd1450ParserStruct *parser);
void thvd1450ProtocolReset(thvd1450ParserStruct *parser);
uint8_t thvd1450ProtocolChecksum(const uint8_t *payload, uint8_t length);

```

```

thvd1450StatusEnum thvd1450ProtocolEncode(
    const uint8_t *payload,
    uint8_t length,
    uint8_t *frameBuffer,
    uint8_t frameBufferSize,
    uint8_t *frameLength);

```

```

thvd1450StatusEnum thvd1450ProtocolInputByte(
    thvd1450ParserStruct *parser,
    uint8_t byte,
    thvd1450FrameStruct *outFrame);

```

```

#endif // _THVD1450_PROTOCOL_H_

```

core1_cyt4bb7/project/code/comm/uartLinkProto.c

SHA-256

:

48b3216267bf537a48b2af3bafdc79827286b7d84d04349d307831fd19fd11e

代码 (c)

```

#include "uartLinkProto.h"

/* float ↔ 4 字节小端互转 (core0/core1 同为小端 ARM, 按字节显式拼装传输) 。 */
typedef union { float f; uint8_t b[4]; } uartLinkFloatBytes;

static uint8_t crc8AtmUpdate(uint8_t crc, uint8_t data)
{
    uint8_t bit;
    crc ^= data;
    for (bit = 0u; bit < 8u; bit++)
    {
        crc = (crc & 0x80u) ? (uint8_t)((crc << 1u) ^ 0x07u) : (uint8_t)(crc << 1u);
    }
    return crc;
}

static uint8_t xorU32Le(uint8_t value, uint32_t data)
{
    return (uint8_t)(value ^ (uint8_t)data ^ (uint8_t)(data >> 8)
        ^ (uint8_t)(data >> 16) ^ (uint8_t)(data >> 24));
}

static uint8_t xorU16Le(uint8_t value, uint16_t data)
{
    return (uint8_t)(value ^ (uint8_t)data ^ (uint8_t)(data >> 8));
}

static void putU32Le(uint8_t *buffer, uint8_t offset, uint32_t value)
{
    buffer[offset] = (uint8_t)value;
    buffer[offset + 1u] = (uint8_t)(value >> 8);
    buffer[offset + 2u] = (uint8_t)(value >> 16);
    buffer[offset + 3u] = (uint8_t)(value >> 24);
}

static uint32_t getU32Le(const uint8_t *buffer, uint8_t offset)
{
    return (uint32_t)buffer[offset]
        | ((uint32_t)buffer[offset + 1u] << 8)
        | ((uint32_t)buffer[offset + 2u] << 16)
        | ((uint32_t)buffer[offset + 3u] << 24);
}

/* 按 type 返回帧总长 (含 HEAD/TAIL) ; 未知 type 返回 0。 */
static uint8_t uartLinkProtoFrameLen(uint8_t type)
{
    switch (type)
    {
        case UART_LINK_TYPE_PING:
        case UART_LINK_TYPE_ACK:

```



```

    case UART_LINK_TYPE_HEARTBEAT:
        return UART_LINK_FRAME_LENGTH;
    case UART_LINK_TYPE_STATUS:
        return UART_LINK_FRAME_LENGTH_STATUS;
    case UART_LINK_TYPE_CORE0_STATUS:
        return UART_LINK_FRAME_LENGTH_C0STATUS;
    case UART_LINK_TYPE_CORE0_POSE:
        return UART_LINK_FRAME_LENGTH_C0POSE;
    case UART_LINK_TYPE_MOTION:
        return UART_LINK_FRAME_LENGTH_MOTION;
    case UART_LINK_TYPE_KVM_CONTROL:
        return UART_LINK_FRAME_LENGTH_KVM_CONTROL;
    case UART_LINK_TYPE_KVM_EVENT:
        return UART_LINK_FRAME_LENGTH_KVM_EVENT;
    case UART_LINK_TYPE_KVM_STATUS:
    case UART_LINK_TYPE_PARAM_REQUEST:
    case UART_LINK_TYPE_PARAM_RESPONSE:
        return UART_LINK_FRAME_LENGTH_KVM_STATUS;
    case UART_LINK_TYPE_PARAM_GLOBAL_REQUEST:
        return UART_LINK_FRAME_LENGTH_PARAM_GLOBAL_REQUEST;
    case UART_LINK_TYPE_PARAM_GLOBAL_STATE:
        return UART_LINK_FRAME_LENGTH_PARAM_GLOBAL_STATE;
    default:
        return 0u;
    }
}

void uartLinkProtoInit(uartLinkParserStruct *parser)
{
    uint8_t i;

    if (0 == parser)
    {
        return;
    }

    parser->index = 0u;
    parser->expectLen = 0u;
    for (i = 0u; i < UART_LINK_FRAME_LENGTH_MAX; ++i)
    {
        parser->buffer[i] = 0u;
    }
}

uint8_t uartLinkProtoChecksum(const uartLinkFrameStruct *frame)
{
    if (0 == frame)
    {
        return 0u;
    }

    if (UART_LINK_TYPE_STATUS == frame->type)
    {
        uint8_t beaconLo = (uint8_t)(frame->beaconCount & 0xFFu);
        uint8_t beaconHi = (uint8_t)((frame->beaconCount >> 8) & 0xFFu);
        return (uint8_t)(frame->type ^ frame->sequence ^ beaconLo ^ beaconHi ^
            frame->linkFlags ^ frame->faultSource ^ frame->faultClass ^
            (uint8_t)frame->producerBootSession ^ (uint8_t)(frame->producerBootSession >> 8) ^
            (uint8_t)(frame->producerBootSession >> 16) ^ (uint8_t)(frame->producerBootSession >> 24));
    }

    if (UART_LINK_TYPE_CORE0_STATUS == frame->type)
    {
        uint8_t crc = 0u;
        crc = crc8AtmUpdate(crc, frame->type);
        crc = crc8AtmUpdate(crc, frame->sequence);
        crc = crc8AtmUpdate(crc, frame->safetyState);
        crc = crc8AtmUpdate(crc, frame->armProgress);
        crc = crc8AtmUpdate(crc, frame->statusFlags);
        crc = crc8AtmUpdate(crc, frame->sysCtrl);
        return crc8AtmUpdate(crc, frame->buzzer);
    }

    if (UART_LINK_TYPE_MOTION == frame->type)
    {
        uartLinkFloatBytes vx, vy, wz;
        uint8_t i;
        uint8_t x = 0u;
        vx.f = frame->motionVx;

```

```

vy.f = frame->motionVy;
wz.f = frame->motionWz;
x = crc8AtmUpdate(x, frame->type);
x = crc8AtmUpdate(x, frame->sequence);
for (i = 0u; i < 4u; i++) x = crc8AtmUpdate(x, vx.b[i]);
for (i = 0u; i < 4u; i++) x = crc8AtmUpdate(x, vy.b[i]);
for (i = 0u; i < 4u; i++) x = crc8AtmUpdate(x, wz.b[i]);
x = crc8AtmUpdate(x, frame->motionFlags);
x = crc8AtmUpdate(x, (uint8_t)frame->producerBootSession);
x = crc8AtmUpdate(x, (uint8_t)(frame->producerBootSession >> 8));
x = crc8AtmUpdate(x, (uint8_t)(frame->producerBootSession >> 16));
return crc8AtmUpdate(x, (uint8_t)(frame->producerBootSession >> 24));
}

if (UART_LINK_TYPE_CORE0_POSE == frame->type)
{
    uartLinkFloatBytes yaw;
    uint8_t i;
    uint8_t x = (uint8_t)(frame->type ^ frame->sequence ^ frame->poseEpoch
        ^ (uint8_t)(frame->poseXQ8 & 0xFFu) ^ (uint8_t)((frame->poseXQ8 >> 8) & 0xFFu)
        ^ (uint8_t)(frame->poseYQ8 & 0xFFu) ^ (uint8_t)((frame->poseYQ8 >> 8) & 0xFFu));
    yaw.f = frame->poseOrientationRad;
    for (i = 0u; i < 4u; i++) x = (uint8_t)(x ^ yaw.b[i]);
    x = (uint8_t)(x ^ frame->poseConfidenceQ8 ^ frame->poseFlags);
    return x;
}

if (UART_LINK_TYPE_KVM_CONTROL == frame->type)
{
    uint8_t x = (uint8_t)(frame->type ^ frame->sequence);
    x = xorU32Le(x, frame->kvmTargetSession);
    x = xorU32Le(x, frame->kvmRouteSession);
    x = xorU32Le(x, frame->kvmRouteEpoch);
    x = (uint8_t)(x ^ frame->kvmTarget);
    return xorU16Le(x, frame->kvmLeaseMs);
}

if (UART_LINK_TYPE_KVM_EVENT == frame->type)
{
    uint8_t x = (uint8_t)(frame->type ^ frame->sequence);
    x = xorU32Le(x, frame->kvmTargetSession);
    x = xorU32Le(x, frame->kvmRouteSession);
    x = xorU32Le(x, frame->kvmRouteEpoch);
    x = xorU32Le(x, frame->kvmEventSeq);
    return (uint8_t)(x ^ frame->kvmTarget ^ frame->kvmKeyId ^ frame->kvmKeyState);
}

if (UART_LINK_TYPE_KVM_STATUS == frame->type)
{
    uint8_t x = (uint8_t)(frame->type ^ frame->sequence);
    x = xorU32Le(x, frame->kvmTargetSession);
    x = xorU32Le(x, frame->kvmRouteSession);
    x = xorU32Le(x, frame->kvmRouteEpoch);
    x = xorU32Le(x, frame->kvmEventSeq);
    x = (uint8_t)(x ^ frame->kvmTarget ^ frame->kvmActiveTarget
        ^ frame->kvmMenuStates ^ frame->kvmFlags ^ frame->kvmAckResult);
    return xorU16Le(x, frame->kvmCore3AgeMs);
}

if ((UART_LINK_TYPE_PARAM_REQUEST == frame->type) ||
    (UART_LINK_TYPE_PARAM_RESPONSE == frame->type))
{
    uint8_t crc = 0u;
    uint8_t i;
    crc = crc8AtmUpdate(crc, frame->type);
    crc = crc8AtmUpdate(crc, frame->sequence);
    crc = crc8AtmUpdate(crc, frame->paramOp);
    crc = crc8AtmUpdate(crc, frame->paramReserved);
    crc = crc8AtmUpdate(crc, frame->paramTarget);
    crc = crc8AtmUpdate(crc, frame->paramStatus);
    for (i = 0u; i < 4u; i++) crc = crc8AtmUpdate(crc,
        (uint8_t)(frame->paramRequestId >> (8u * i)));
    crc = crc8AtmUpdate(crc, (uint8_t)frame->paramOffset);
    crc = crc8AtmUpdate(crc, (uint8_t)(frame->paramOffset >> 8));
    crc = crc8AtmUpdate(crc, frame->paramLength);
    for (i = 0u; i < UART_LINK_PARAM_DATA_BYTES; i++)
        crc = crc8AtmUpdate(crc,
            (i < frame->paramLength) ? frame->paramData[i] : 0u);
    return crc;
}

```

```

}

if (UART_LINK_TYPE_PARAM_GLOBAL_REQUEST == frame->type)
{
    uint8_t crc = 0u;
    uint8_t i;
    crc = crc8AtmUpdate(crc, frame->type);
    crc = crc8AtmUpdate(crc, frame->sequence);
    for (i = 0u; i < 4u; i++) crc = crc8AtmUpdate(crc,
        (uint8_t)(frame->paramGlobalRequest.producerBootGeneration >> (8u * i)));
    for (i = 0u; i < 4u; i++) crc = crc8AtmUpdate(crc,
        (uint8_t)(frame->paramGlobalRequest.sequence >> (8u * i)));
    crc = crc8AtmUpdate(crc, frame->paramGlobalRequest.origin);
    crc = crc8AtmUpdate(crc, frame->paramGlobalRequest.action);
    crc = crc8AtmUpdate(crc, frame->paramGlobalRequest.slot);
    return crc8AtmUpdate(crc, frame->paramGlobalRequest.reserved);
}

if (UART_LINK_TYPE_PARAM_GLOBAL_STATE == frame->type)
{
    const uint32_t words[5] = {
        frame->paramGlobalState.coordinatorBootGeneration,
        frame->paramGlobalState.sequence,
        frame->paramGlobalState.requestBootGeneration,
        frame->paramGlobalState.requestSequence,
        frame->paramGlobalState.transactionId
    };
    const uint8_t bytes[8] = {
        frame->paramGlobalState.origin, frame->paramGlobalState.action,
        frame->paramGlobalState.slot, frame->paramGlobalState.phase,
        frame->paramGlobalState.target, frame->paramGlobalState.error,
        frame->paramGlobalState.readyMask, frame->paramGlobalState.flags
    };
    uint8_t crc = 0u;
    uint8_t i, j;
    crc = crc8AtmUpdate(crc, frame->type);
    crc = crc8AtmUpdate(crc, frame->sequence);
    for (i = 0u; i < 5u; i++)
        for (j = 0u; j < 4u; j++) crc = crc8AtmUpdate(crc,
            (uint8_t)(words[i] >> (8u * j)));
    for (i = 0u; i < 8u; i++) crc = crc8AtmUpdate(crc, bytes[i]);
    return crc;
}

/* 基础帧: type ^ seq ^ value */
return (uint8_t)(frame->type ^ frame->sequence ^ frame->value);
}

uartLinkProtoStatusEnum uartLinkProtoEncode(const uartLinkFrameStruct *frame, uint8_t *buffer, uint8_t bufferSize)
{
    uint8_t frameLen;

    if (0 == frame || 0 == buffer)
    {
        return UART_LINK_PROTO_STATUS_INVALID_PARAM;
    }

    frameLen = uartLinkProtoFrameLen(frame->type);
    if (0u == frameLen)
    {
        return UART_LINK_PROTO_STATUS_TYPE_ERROR;
    }
    if (bufferSize < frameLen)
    {
        return UART_LINK_PROTO_STATUS_INVALID_PARAM;
    }

    buffer[0] = UART_LINK_FRAME_HEAD;
    buffer[1] = frame->type;
    buffer[2] = frame->sequence;

    if (UART_LINK_TYPE_STATUS == frame->type)
    {
        buffer[3] = (uint8_t)(frame->beaconCount & 0xFFu);
        buffer[4] = (uint8_t)((frame->beaconCount >> 8) & 0xFFu);
        buffer[5] = frame->linkFlags;
        buffer[6] = frame->faultSource;
        buffer[7] = frame->faultClass;
        putU32Le(buffer, 8u, frame->producerBootSession);
    }
}

```

```

    buffer[12] = uartLinkProtoChecksum(frame);
    buffer[13] = UART_LINK_FRAME_TAIL;
}
else if (UART_LINK_TYPE_CORE0_STATUS == frame->type)
{
    buffer[3] = frame->safetyState;
    buffer[4] = frame->armProgress;
    buffer[5] = frame->statusFlags;
    buffer[6] = frame->sysCtrl;
    buffer[7] = frame->buzzer;
    buffer[8] = uartLinkProtoChecksum(frame);
    buffer[9] = UART_LINK_FRAME_TAIL;
}
else if (UART_LINK_TYPE_MOTION == frame->type)
{
    uartLinkFloatBytes vx, vy, wz;
    uint8_t i;
    vx.f = frame->motionVx;
    vy.f = frame->motionVy;
    wz.f = frame->motionWz;
    for (i = 0u; i < 4u; i++) buffer[3u + i] = vx.b[i];
    for (i = 0u; i < 4u; i++) buffer[7u + i] = vy.b[i];
    for (i = 0u; i < 4u; i++) buffer[11u + i] = wz.b[i];
    buffer[15] = frame->motionFlags;
    putU32Le(buffer, 16u, frame->producerBootSession);
    buffer[20] = uartLinkProtoChecksum(frame);
    buffer[21] = UART_LINK_FRAME_TAIL;
}
else if (UART_LINK_TYPE_CORE0_POSE == frame->type)
{
    uartLinkFloatBytes yaw;
    uint8_t i;
    yaw.f = frame->poseOrientationRad;
    buffer[3] = frame->poseEpoch;
    buffer[4] = (uint8_t)(frame->poseXQ8 & 0xFFu);
    buffer[5] = (uint8_t)((frame->poseXQ8 >> 8) & 0xFFu);
    buffer[6] = (uint8_t)(frame->poseYQ8 & 0xFFu);
    buffer[7] = (uint8_t)((frame->poseYQ8 >> 8) & 0xFFu);
    for (i = 0u; i < 4u; i++) buffer[8u + i] = yaw.b[i];
    buffer[12] = frame->poseConfidenceQ8;
    buffer[13] = frame->poseFlags;
    buffer[14] = uartLinkProtoChecksum(frame);
    buffer[15] = UART_LINK_FRAME_TAIL;
}
else if (UART_LINK_TYPE_KVM_CONTROL == frame->type)
{
    putU32Le(buffer, 3u, frame->kvmTargetSession);
    putU32Le(buffer, 7u, frame->kvmRouteSession);
    putU32Le(buffer, 11u, frame->kvmRouteEpoch);
    buffer[15] = frame->kvmTarget;
    buffer[16] = (uint8_t)frame->kvmLeaseMs;
    buffer[17] = (uint8_t)(frame->kvmLeaseMs >> 8);
    buffer[18] = uartLinkProtoChecksum(frame);
    buffer[19] = UART_LINK_FRAME_TAIL;
}
else if (UART_LINK_TYPE_KVM_EVENT == frame->type)
{
    putU32Le(buffer, 3u, frame->kvmTargetSession);
    putU32Le(buffer, 7u, frame->kvmRouteSession);
    putU32Le(buffer, 11u, frame->kvmRouteEpoch);
    putU32Le(buffer, 15u, frame->kvmEventSeq);
    buffer[19] = frame->kvmTarget;
    buffer[20] = frame->kvmKeyId;
    buffer[21] = frame->kvmKeyState;
    buffer[22] = uartLinkProtoChecksum(frame);
    buffer[23] = UART_LINK_FRAME_TAIL;
}
else if (UART_LINK_TYPE_KVM_STATUS == frame->type)
{
    putU32Le(buffer, 3u, frame->kvmTargetSession);
    putU32Le(buffer, 7u, frame->kvmRouteSession);
    putU32Le(buffer, 11u, frame->kvmRouteEpoch);
    putU32Le(buffer, 15u, frame->kvmEventSeq);
    buffer[19] = frame->kvmTarget;
    buffer[20] = frame->kvmActiveTarget;
    buffer[21] = frame->kvmMenuStates;
    buffer[22] = frame->kvmFlags;
    buffer[23] = frame->kvmAckResult;
    buffer[24] = (uint8_t)frame->kvmCore3AgeMs;
}

```

```

        buffer[25] = (uint8_t)(frame->kvmCore3AgeMs >> 8);
        buffer[26] = uartLinkProtoChecksum(frame);
        buffer[27] = UART_LINK_FRAME_TAIL;
    }
    else if ((UART_LINK_TYPE_PARAM_REQUEST == frame->type) ||
             (UART_LINK_TYPE_PARAM_RESPONSE == frame->type))
    {
        uint8_t i;
        if (frame->paramLength > UART_LINK_PARAM_DATA_BYTES)
            return UART_LINK_PROTO_STATUS_INVALID_PARAM;
        buffer[3] = frame->paramOp;
        buffer[4] = frame->paramReserved;
        buffer[5] = frame->paramTarget;
        buffer[6] = frame->paramStatus;
        putU32Le(buffer, 7u, frame->paramRequestId);
        buffer[11] = (uint8_t)frame->paramOffset;
        buffer[12] = (uint8_t)(frame->paramOffset >> 8);
        buffer[13] = frame->paramLength;
        for (i = 0u; i < UART_LINK_PARAM_DATA_BYTES; i++)
            buffer[14u + i] = (i < frame->paramLength) ? frame->paramData[i] : 0u;
        buffer[26] = uartLinkProtoChecksum(frame);
        buffer[27] = UART_LINK_FRAME_TAIL;
    }
    else if (UART_LINK_TYPE_PARAM_GLOBAL_REQUEST == frame->type)
    {
        putU32Le(buffer, 3u, frame->paramGlobalRequest.producerBootGeneration);
        putU32Le(buffer, 7u, frame->paramGlobalRequest.sequence);
        buffer[11] = frame->paramGlobalRequest.origin;
        buffer[12] = frame->paramGlobalRequest.action;
        buffer[13] = frame->paramGlobalRequest.slot;
        buffer[14] = frame->paramGlobalRequest.reserved;
        buffer[15] = uartLinkProtoChecksum(frame);
        buffer[16] = UART_LINK_FRAME_TAIL;
    }
    else if (UART_LINK_TYPE_PARAM_GLOBAL_STATE == frame->type)
    {
        putU32Le(buffer, 3u, frame->paramGlobalState.coordinatorBootGeneration);
        putU32Le(buffer, 7u, frame->paramGlobalState.sequence);
        putU32Le(buffer, 11u, frame->paramGlobalState.requestBootGeneration);
        putU32Le(buffer, 15u, frame->paramGlobalState.requestSequence);
        putU32Le(buffer, 19u, frame->paramGlobalState.transactionId);
        buffer[23] = frame->paramGlobalState.origin;
        buffer[24] = frame->paramGlobalState.action;
        buffer[25] = frame->paramGlobalState.slot;
        buffer[26] = frame->paramGlobalState.phase;
        buffer[27] = frame->paramGlobalState.target;
        buffer[28] = frame->paramGlobalState.error;
        buffer[29] = frame->paramGlobalState.readyMask;
        buffer[30] = frame->paramGlobalState.flags;
        buffer[31] = uartLinkProtoChecksum(frame);
        buffer[32] = UART_LINK_FRAME_TAIL;
    }
    else
    {
        buffer[3] = frame->value;
        buffer[4] = uartLinkProtoChecksum(frame);
        buffer[5] = UART_LINK_FRAME_TAIL;
    }

    return UART_LINK_PROTO_STATUS_OK;
}

uartLinkProtoStatusEnum uartLinkProtoInputByte(uartLinkParserStruct *parser, uint8_t data, uartLinkFrameStruct *frame)
{
    uint8_t frameLen;

    if (0 == parser || 0 == frame)
    {
        return UART_LINK_PROTO_STATUS_INVALID_PARAM;
    }

    /* 等待帧头：index==0 时只接受 HEAD，其余字节丢弃。 */
    if (0u == parser->index && UART_LINK_FRAME_HEAD != data)
    {
        return UART_LINK_PROTO_STATUS_OK;
    }

    parser->buffer[parser->index] = data;
    parser->index++;
}

```

```

/* 读到 type (index==2 即第 2 字节落位后) 确定期望帧长。 */
if (2u == parser->index)
{
    parser->expectLen = uartLinkProtoFrameLen(parser->buffer[1]);
    if (0u == parser->expectLen)
    {
        /* 未知 type: 复位重新找帧头, 报类型错。 */
        parser->index = 0u;
        return UART_LINK_PROTO_STATUS_TYPE_ERROR;
    }
}

/* expectLen 未定或未集满, 继续收。 */
if (parser->index < 2u || parser->index < parser->expectLen)
{
    return UART_LINK_PROTO_STATUS_OK;
}

/* 集满一帧, 解析。 */
frameLen = parser->expectLen;
parser->index = 0u;
parser->expectLen = 0u;

frame->type = parser->buffer[1];
frame->sequence = parser->buffer[2];
frame->value = 0u;
frame->beaconCount = 0u;
frame->linkFlags = 0u;
frame->faultSource = 0u;
frame->faultClass = 0u;
frame->producerBootSession = 0u;
frame->safetyState = 0u;
frame->armProgress = 0u;
frame->statusFlags = 0u;
frame->sysCtrl = 0u;
frame->buzzer = 0u;
frame->poseEpoch = 0u;
frame->poseXQ8 = 0u;
frame->poseYQ8 = 0u;
frame->poseOrientationRad = 0.0f;
frame->poseConfidenceQ8 = 0u;
frame->poseFlags = 0u;
frame->motionVx = 0.0f;
frame->motionVy = 0.0f;
frame->motionWz = 0.0f;
frame->motionFlags = 0u;
frame->kvmTargetSession = 0u;
frame->kvmRouteSession = 0u;
frame->kvmRouteEpoch = 0u;
frame->kvmEventSeq = 0u;
frame->kvmLeaseMs = 0u;
frame->kvmCore3AgeMs = 0u;
frame->kvmTarget = UART_LINK_KVM_TARGET_CORE0;
frame->kvmActiveTarget = UART_LINK_KVM_TARGET_CORE0;
frame->kvmKeyId = 0u;
frame->kvmKeyState = 0u;
frame->kvmMenuStates = 0xFFu;
frame->kvmFlags = 0u;
frame->kvmAckResult = UART_LINK_KVM_ACK_NONE;
frame->kvmRequestId = 0u;
frame->paramOffset = 0u;
frame->paramOp = 0u;
frame->paramReserved = 0u;
frame->paramTarget = 0u;
frame->paramStatus = 0u;
frame->paramLength = 0u;
{ uint8_t i; for (i = 0u; i < UART_LINK_PARAM_DATA_BYTES; i++) frame->paramData[i] = 0u; }
frame->paramGlobalRequest.producerBootGeneration = 0u;
frame->paramGlobalRequest.sequence = 0u;
frame->paramGlobalRequest.origin = 0u;
frame->paramGlobalRequest.action = 0u;
frame->paramGlobalRequest.slot = 0u;
frame->paramGlobalRequest.reserved = 0u;
frame->paramGlobalState.coordinatorBootGeneration = 0u;
frame->paramGlobalState.sequence = 0u;
frame->paramGlobalState.requestBootGeneration = 0u;
frame->paramGlobalState.requestSequence = 0u;
frame->paramGlobalState.transactionId = 0u;

```

```

frame->paramGlobalState.origin = 0u;
frame->paramGlobalState.action = 0u;
frame->paramGlobalState.slot = 0u;
frame->paramGlobalState.phase = 0u;
frame->paramGlobalState.target = 0u;
frame->paramGlobalState.error = 0u;
frame->paramGlobalState.readyMask = 0u;
frame->paramGlobalState.flags = 0u;

if (UART_LINK_TYPE_STATUS == frame->type)
{
    frame->beaconCount = (uint16_t)(parser->buffer[3] | ((uint16_t)parser->buffer[4] << 8));
    frame->linkFlags = parser->buffer[5];
    frame->faultSource = parser->buffer[6];
    frame->faultClass = parser->buffer[7];
    frame->producerBootSession = getU32Le(parser->buffer, 8u);
}
else if (UART_LINK_TYPE_CORE0_STATUS == frame->type)
{
    frame->safetyState = parser->buffer[3];
    frame->armProgress = parser->buffer[4];
    frame->statusFlags = parser->buffer[5];
    frame->sysCtrl = parser->buffer[6];
    frame->buzzer = parser->buffer[7];
}
else if (UART_LINK_TYPE_MOTION == frame->type)
{
    uartLinkFloatBytes vx, vy, wz;
    uint8_t i;
    for (i = 0u; i < 4u; i++) vx.b[i] = parser->buffer[3u + i];
    for (i = 0u; i < 4u; i++) vy.b[i] = parser->buffer[7u + i];
    for (i = 0u; i < 4u; i++) wz.b[i] = parser->buffer[11u + i];
    frame->motionVx = vx.f;
    frame->motionVy = vy.f;
    frame->motionWz = wz.f;
    frame->motionFlags = parser->buffer[15];
    frame->producerBootSession = getU32Le(parser->buffer, 16u);
}
else if (UART_LINK_TYPE_CORE0_POSE == frame->type)
{
    uartLinkFloatBytes yaw;
    uint8_t i;
    frame->poseEpoch = parser->buffer[3];
    frame->poseXQ8 = (uint16_t)(parser->buffer[4] | ((uint16_t)parser->buffer[5] << 8));
    frame->poseYQ8 = (uint16_t)(parser->buffer[6] | ((uint16_t)parser->buffer[7] << 8));
    for (i = 0u; i < 4u; i++) yaw.b[i] = parser->buffer[8u + i];
    frame->poseOrientationRad = yaw.f;
    frame->poseConfidenceQ8 = parser->buffer[12];
    frame->poseFlags = parser->buffer[13];
}
else if (UART_LINK_TYPE_KVM_CONTROL == frame->type)
{
    frame->kvmTargetSession = getU32Le(parser->buffer, 3u);
    frame->kvmRouteSession = getU32Le(parser->buffer, 7u);
    frame->kvmRouteEpoch = getU32Le(parser->buffer, 11u);
    frame->kvmTarget = parser->buffer[15];
    frame->kvmLeaseMs = (uint16_t)(parser->buffer[16] | ((uint16_t)parser->buffer[17] << 8));
}
else if (UART_LINK_TYPE_KVM_EVENT == frame->type)
{
    frame->kvmTargetSession = getU32Le(parser->buffer, 3u);
    frame->kvmRouteSession = getU32Le(parser->buffer, 7u);
    frame->kvmRouteEpoch = getU32Le(parser->buffer, 11u);
    frame->kvmEventSeq = getU32Le(parser->buffer, 15u);
    frame->kvmTarget = parser->buffer[19];
    frame->kvmKeyId = parser->buffer[20];
    frame->kvmKeyState = parser->buffer[21];
}
else if (UART_LINK_TYPE_KVM_STATUS == frame->type)
{
    frame->kvmTargetSession = getU32Le(parser->buffer, 3u);
    frame->kvmRouteSession = getU32Le(parser->buffer, 7u);
    frame->kvmRouteEpoch = getU32Le(parser->buffer, 11u);
    frame->kvmEventSeq = getU32Le(parser->buffer, 15u);
    frame->kvmTarget = parser->buffer[19];
    frame->kvmActiveTarget = parser->buffer[20];
    frame->kvmMenuStates = parser->buffer[21];
    frame->kvmFlags = parser->buffer[22];
    frame->kvmAckResult = parser->buffer[23];
}

```

```

        frame->kvmCore3AgeMs = (uint16_t)(parser->buffer[24] | ((uint16_t)parser->buffer[25] << 8));
    }
    else if ((UART_LINK_TYPE_PARAM_REQUEST == frame->type) ||
             (UART_LINK_TYPE_PARAM_RESPONSE == frame->type))
    {
        uint8_t i;
        frame->paramOp = parser->buffer[3];
        frame->paramReserved = parser->buffer[4];
        frame->paramTarget = parser->buffer[5];
        frame->paramStatus = parser->buffer[6];
        frame->paramRequestId = getU32Le(parser->buffer, 7u);
        frame->paramOffset = (uint16_t)(parser->buffer[11] | ((uint16_t)parser->buffer[12] << 8));
        frame->paramLength = parser->buffer[13];
        if (frame->paramLength > UART_LINK_PARAM_DATA_BYTES)
            return UART_LINK_PROTO_STATUS_INVALID_PARAM;
        for (i = 0u; i < UART_LINK_PARAM_DATA_BYTES; i++)
            frame->paramData[i] = parser->buffer[14u + i];
    }
    else if (UART_LINK_TYPE_PARAM_GLOBAL_REQUEST == frame->type)
    {
        frame->paramGlobalRequest.producerBootGeneration = getU32Le(parser->buffer, 3u);
        frame->paramGlobalRequest.sequence = getU32Le(parser->buffer, 7u);
        frame->paramGlobalRequest.origin = parser->buffer[11];
        frame->paramGlobalRequest.action = parser->buffer[12];
        frame->paramGlobalRequest.slot = parser->buffer[13];
        frame->paramGlobalRequest.reserved = parser->buffer[14];
        if (paramSyncGlobalRequestIsValid(&frame->paramGlobalRequest) == 0u)
            return UART_LINK_PROTO_STATUS_INVALID_PARAM;
    }
    else if (UART_LINK_TYPE_PARAM_GLOBAL_STATE == frame->type)
    {
        frame->paramGlobalState.coordinatorBootGeneration = getU32Le(parser->buffer, 3u);
        frame->paramGlobalState.sequence = getU32Le(parser->buffer, 7u);
        frame->paramGlobalState.requestBootGeneration = getU32Le(parser->buffer, 11u);
        frame->paramGlobalState.requestSequence = getU32Le(parser->buffer, 15u);
        frame->paramGlobalState.transactionId = getU32Le(parser->buffer, 19u);
        frame->paramGlobalState.origin = parser->buffer[23];
        frame->paramGlobalState.action = parser->buffer[24];
        frame->paramGlobalState.slot = parser->buffer[25];
        frame->paramGlobalState.phase = parser->buffer[26];
        frame->paramGlobalState.target = parser->buffer[27];
        frame->paramGlobalState.error = parser->buffer[28];
        frame->paramGlobalState.readyMask = parser->buffer[29];
        frame->paramGlobalState.flags = parser->buffer[30];
    }
    else
    {
        frame->value = parser->buffer[3];
    }

    if (uartLinkProtoChecksum(frame) != parser->buffer[frameLen - 2u])
    {
        return UART_LINK_PROTO_STATUS_CHECK_ERROR;
    }
    if (UART_LINK_FRAME_TAIL != parser->buffer[frameLen - 1u])
    {
        return UART_LINK_PROTO_STATUS_TAIL_ERROR;
    }

    return UART_LINK_PROTO_STATUS_FRAME_READY;
}

```

core1_cyt4bb7/project/code/comm/uartLinkProto.h

SHA-256

:

c06329a6c679a4526c3ed656c142cdbcb35a539a0978991bbd1c31f2ecdc75d

代码 (c)

```

#ifndef _UART_LINK_PROTO_H_
#define _UART_LINK_PROTO_H_

```

/*

* SYNC: core0 ↔ core1 两份 uartLinkProto.c/.h 必须逐字节同源。改任一份后 diff 校验另一份。

*

* 帧族（按 type 定长解析，HEAD/TAIL 不变）：

- * 基础帧 6 字节 [HEAD|type|seq|value|xor|TAIL]
- * - PING/ACK/HEARTBEAT：原有，value 留白
- * 状态帧 14 字节 [HEAD|type|seq|beaconLo|beaconHi|linkFlags|faultSource|faultClass|producerBootSession(4)|xor|TAIL]
- * - STATUS(0x40)：core1→core0 回传信标数、链路标志和通用必需视觉故障来源/类别
- * linkFlags bit1=core2LinkLost(core1 测 RS-485↔core2 断联，core0 据此聚合缓降并禁 K5 发车)；
- * linkFlags bit2=mode2LoraLandReq(模式 2 调试 LoRa 双肩键缓降请求，仅转 manualLand，不请求起飞)；
- * linkFlags bit3=requiredVisionFault(任一必需视觉故障；plan 缺失/停更或必需视觉链路、相机、配置、标定故障均 fail-closed，由 core0 聚合缓降并禁 K5 发车)；
- * 与心跳同周期(200ms)常发，不受 MOTION 帧 gotMotion 门控影响(故 core2 断联上报走此帧而非 MOTION flags)
- * core0 状态帧 10 字节 [HEAD|0x44|seq|safetyState|armProgress|statusFlags|sysCtrl|buzzer|crc8|TAIL]
- * - CORE0_STATUS(0x44)：core0→core1 回传 飞控安全态 + arming 进度 + 状态位(供 RGB 状态灯)
- * + sysCtrl(空地系统模式/RGB 意图/到顶标志) + buzzer(蜂鸣模式码，core1 中转 RS-485 给 core2 buzzer 库 pattern 层播放)
- * core0 位姿帧 16 字节 [HEAD|0x42|seq|epoch|xQ8(2)|yQ8(2)|yaw(4)|confQ8|flags|xor|TAIL]
- * - CORE0_POSE(0x42)：core0→core1 回传 core0 视觉灯板同坐标/方向，供 core1 车坐标误差生成和屏幕叠加
- * 运动帧 22 字节 [HEAD|0x51|seq|vx(4)|vy(4)|wz(4)|flags|producerBootSession(4)|crc8|TAIL]
- * - MOTION(0x51)：core1→core0 运动前馈 vx/vy/wz (3×float 小端，来源=core1 规划(beaconNavigator) 或 LoRa 遥控转发)
- * + flags(bit0=remoteTakeoffReq：模式 3 车 LoRa S3+S4 请求，core0 按 TKO 阶段解释为起飞或缓降)
- * CORE0_STATUS/MOTION 使用 CRC-8/ATM(poly=0x07, init=0)，覆盖 type+seq+payload；
- * 其余帧保持 xor = type ^ seq ^ 其余 payload 各字节。
- * 字节序：多字节字段显式按字节小端拼装，不依赖结构体布局。

```
#include <stdint.h>
#include "paramSyncGlobal.h"

#define UART_LINK_RX_SESSION_REJECT (0u)
#define UART_LINK_RX_SESSION_ACCEPT (1u)
#define UART_LINK_RX_SESSION_REBIND (2u)

/* Core0 只在 STATUS 已进入离线栅栏时允许任意非零 producer session 重绑。
 * 在线期间其它 session 一律拒绝，避免延迟旧帧抢占当前 producer。 */
static inline uint8_t uartLinkRxSessionClassify(uint8_t statusFenced,
uint32_t currentSession, uint32_t incomingSession)
{
    if (incomingSession == 0u) return UART_LINK_RX_SESSION_REJECT;
    if (statusFenced != 0u) return UART_LINK_RX_SESSION_REBIND;
    return (incomingSession == currentSession) ? UART_LINK_RX_SESSION_ACCEPT : UART_LINK_RX_SESSION_REJECT;
}

#define UART_LINK_BAUDRATE (4000 000u)

#define UART_LINK_FRAME_HEAD (0xA5u)
#define UART_LINK_FRAME_TAIL (0x5Au)

#define UART_LINK_FRAME_LENGTH      (6u) /* 基础帧 (PING/ACK/HEARTBEAT) */
#define UART_LINK_FRAME_LENGTH_STATUS (14u) /* STATUS 含故障事实及 Core1 启动身份 */
typedef char uartLink_status_frame_length_assert[(UART_LINK_FRAME_LENGTH_STATUS == 14u) ? 1 : -1];
#define UART_LINK_FRAME_LENGTH_C0STATUS (10u) /* core0 状态帧 (CORE0_STATUS 0x44, 含 sysCtrl+buzzer) */
#define UART_LINK_FRAME_LENGTH_C0POSE (16u) /* core0 位姿帧 [HEAD|0x42|seq|epoch|xQ8|yQ8|yaw|conf|flags|xor|TAIL] */
#define UART_LINK_FRAME_LENGTH_MOTION (22u) /* 运动帧含 Core1 启动身份 */
#define UART_LINK_FRAME_LENGTH_KVM_CONTROL (20u)
#define UART_LINK_FRAME_LENGTH_KVM_EVENT (24u)
#define UART_LINK_FRAME_LENGTH_KVM_STATUS (28u)
#define UART_LINK_FRAME_LENGTH_PARAM (28u)
#define UART_LINK_FRAME_LENGTH_PARAM_GLOBAL_REQUEST (17u)
#define UART_LINK_FRAME_LENGTH_PARAM_GLOBAL_STATE (33u)
#define UART_LINK_PARAM_DATA_BYTES (12u)
#define UART_LINK_FRAME_LENGTH_MAX (33u)

#define UART_LINK_TYPE_PING (0x10u)
#define UART_LINK_TYPE_ACK (0x11u)
#define UART_LINK_TYPE_HEARTBEAT (0x20u)
#define UART_LINK_TYPE_KVM_CONTROL (0x31u) /* core0→core1 KVM 路由租约 */
#define UART_LINK_TYPE_KVM_EVENT (0x32u) /* core0→core1 KVM exactly-once 事件 */
#define UART_LINK_TYPE_STATUS (0x40u) /* core1→core0 状态回传 */
#define UART_LINK_TYPE_CORE0_STATUS (0x44u) /* core0→core1 飞控状态；CRC-8/ATM */
#define UART_LINK_TYPE_CORE0_POSE (0x42u) /* core0→core1 灯板同坐标/方向，供 core1 车坐标误差和屏幕叠加 */
#define UART_LINK_TYPE_KVM_STATUS (0x43u) /* core1→core0 KVM 菜单状态与终态 ACK */
#define UART_LINK_TYPE_KVM_MOTION (0x51u) /* core1→core0 运动前馈指令；CRC-8/ATM */
#define UART_LINK_TYPE_PARAM_REQUEST (0x60u)
#define UART_LINK_TYPE_PARAM_RESPONSE (0x61u)
#define UART_LINK_TYPE_PARAM_GLOBAL_STATE (0x62u)
#define UART_LINK_TYPE_PARAM_GLOBAL_REQUEST (0x63u)
#define UART_LINK_PARAM_TRANSPORT_BEGIN (0x80u)
#define UART_LINK_PARAM_TRANSPORT_MORE (0x40u)
#define UART_LINK_PARAM_OWNER_RESTART (0x20u)
#define UART_LINK_PARAM_STATUS_MASK (0x3Fu)
#define UART_LINK_PARAM_OP_POLL (0x7Fu)
```

```

/* 三目标 KVM 线值。Core3 使用物理节点编号 3，禁止压缩成索引 2。 */
#define UART_LINK_KVM_TARGET_CORE0 (0u)
#define UART_LINK_KVM_TARGET_CORE1 (1u)
#define UART_LINK_KVM_TARGET_CORE3 (3u)
#define UART_LINK_KVM_TARGET_CORE4 (4u)
#define UART_LINK_KVM_TARGET_NONE (0xFFu)

/* KVM_STATUS flags。HOME 位只有对应 VALID 且状态未陈旧时才可用于 K5 判定。 */
#define UART_LINK_KVM_FLAG_ROUTE_ACCEPTED (0x01u)
#define UART_LINK_KVM_FLAG_LEASE_VALID (0x02u)
#define UART_LINK_KVM_FLAG_CORE1_VALID (0x04u)
#define UART_LINK_KVM_FLAG_CORE1_HOME (0x08u)
#define UART_LINK_KVM_FLAG_CORE3_VALID (0x10u)
#define UART_LINK_KVM_FLAG_CORE3_HOME (0x20u)
#define UART_LINK_KVM_FLAG_ACK_TERMINAL (0x40u)
#define UART_LINK_KVM_FLAG_PEER_RESTART (0x80u)

/* KVM 终态/拒绝结果。同一 boot session 内重复事件只重放 ACK，不重复副作用。 */
#define UART_LINK_KVM_ACK_NONE (0u)
#define UART_LINK_KVM_ACK_EXECUTED (1u)
#define UART_LINK_KVM_ACK_DUPLICATE (2u)
#define UART_LINK_KVM_ACK_UNHANDLED (3u)
#define UART_LINK_KVM_ACK_BUSY (4u)
#define UART_LINK_KVM_ACK_GLOBAL_EVENT_REJECTED (5u)
#define UART_LINK_KVM_ACK_NO_LEASE (6u)
#define UART_LINK_KVM_ACK_WRONG_TARGET (7u)
#define UART_LINK_KVM_ACK_STALE_ROUTE (8u)
#define UART_LINK_KVM_ACK_GAP (9u)
#define UART_LINK_KVM_ACK_OLD_EVENT (10u)
#define UART_LINK_KVM_ACK_BAD_PAYLOAD (11u)
#define UART_LINK_KVM_ACK_PEER_RESTART (12u)

/* STATUS 的 linkFlags 位定义（core1→core0；core1 cm_7_0 发，core0 crossCoreLinkPoll 取） */
#define UART_LINK_STATUS_FLAG_LINK_OK (0x01u) /* bit0: core1 认为核间链路在线 */
#define UART_LINK_STATUS_FLAG_CORE2_LOST (0x02u) /* bit1: core1 测 RS-485↔core2 断联(>core1 RS485 超时)，core0 缓降聚合用 */
#define UART_LINK_STATUS_FLAG_REQUIRED_VISION_FAULT (0x08u) /* bit2: 模式 2 LoRa 双肩键调试缓降请求 */
#define UART_LINK_STATUS_FLAG_BEACON_SEARCHING (0x10u) /* bit4: Core1 正在执行寻灯 */
#define UART_LINK_STATUS_FLAG_BEACON_SEARCH_TIMEOUT (0x20u) /* bit5: 本次寻灯已超时 */
#define UART_LINK_STATUS_FLAG_BEACON_VALID (0x40u) /* bit6: beaconCount/寻灯字段来自 fresh 计划 */
#define UART_LINK_STATUS_FLAG_CORE2_ATTITUDE_FAULT (0x80u) /* bit7: Core2 姿态诊断故障 */

/* CORE0_STATUS 的 statusFlags 位定义 */
#define UART_LINK_C0_FLAG_CM70_READY (0x01u) /* cm_7_0 初始化完成 */
#define UART_LINK_C0_FLAG_VEL_VALID (0x02u) /* 速度估计有效 */
#define UART_LINK_C0_FLAG_TARGET_VALID (0x04u) /* ^灯板视觉目标锁定 */
#define UART_LINK_C0_FLAG_TOF_HEALTHY (0x08u) /* ToF 定高健康 */
#define UART_LINK_C0_FLAG_LAUNCHED (0x10u) /* 飞控已发车(safetyState==CLOSED_LOOP): core1 读此位中转为 RS-485 cmd LAUNCH 给 core2 */

/* CORE0_STATUS 的 sysCtrl 字节位布局（core0 填 / core1 取）:
 * bit0-1 = systemMode; bit2-4 = rgbIntent; bit5 = altReached;
 * bit6-7 = padding, 发送端恒为 0，接收端不得赋予业务语义。 */
#define UART_LINK_SYS_MODE_MASK (0x03u)
#define UART_LINK_SYS_RGB_SHIFT (2u)
#define UART_LINK_SYS_RGB_MASK (0x07u) /* 取值: (sysCtrl>>SHIFT)&MASK */
#define UART_LINK_SYS_ALT_REACHED (0x20u)
/* 空地系统模式线值（systemMode, CORE0_STATUS sysCtrl bit0-1; core0/core1 共用，本宏为语义单一真相源）:
 * 1=车不动 2=RS-485 自主(车随视觉误差) 3=LoRa 转发(车 LoRa 遥控)。0=未设。 */
#define UART_LINK_SYS_MODE_CAR_STILL (1u)
#define UART_LINK_SYS_MODE_RS485_AUTO (2u)
#define UART_LINK_SYS_MODE_LORA_FWD (3u)

/* rgbIntent 意图码（core0 产生 → core1 rgbAnimator 渲染: 3 bit, 0..7） */
#define UART_LINK_RGB_NORMAL (0u) /* 无动画，走 core1 原状态灯优先级链 */
#define UART_LINK_RGB_TKO_WAIT (1u) /* 起飞 5s 等待: 快闪频率渐增→常亮 */
#define UART_LINK_RGB_TKO_RAMP (2u) /* 起飞 2s 爬高: 常亮→线性渐灭 */
#define UART_LINK_RGB_DISCONNECT_WARN (3u) /* 断联告警: 加速闪烁 */

/* MOTION 帧 flags 字节位定义（core2 → core1 → core0 反向信号; core1 中转填位） */
#define UART_LINK_MOTION_FLAG_TAKEOFF_REQ (0x01u) /* 模式 3 车 LoRa S3+S4 请求: TKO_IDLE 起飞, WAIT/RAMP/CRUISE 缓降 */

/* CORE0_POSE flags 位定义（位置与 heading 独立资格; confidence 仅作 heading 更新门控）。 */
#define UART_LINK_POSE_FLAG_TRACK_LOCKED (0x01u) /* bit0: core0 视觉跟踪锁定 */
#define UART_LINK_POSE_FLAG_TARGET_LOST (0x02u) /* bit1: core0 视觉报告丢失 */
#define UART_LINK_POSE_FLAG_POSITION_NUMERIC_VALID (0x04u) /* bit2: x/y 数值与范围有效 */
#define UART_LINK_POSE_FLAG_POSITION_FRESH (0x08u) /* bit3: 本帧位置可作为 core1 位姿候选 */
#define UART_LINK_POSE_FLAG_HEADING_FRESH (0x10u) /* bit4: 本帧方向样本独立有效 */

```

```

/* buzzer 蜂鸣模式码 (core0 产生 → core1 中转 RS-485 → core2 buzzer 库 pattern 层播放; 与 chassisLink CHASSIS_LINK_BUZZER_* 同值 SYNC) */
#define UART_LINK_BUZZER_SILENT      (0u)
#define UART_LINK_BUZZER_1HZ         (1u) /* 起飞上升期: 每秒一次 */
#define UART_LINK_BUZZER_LONG        (2u) /* 到顶: 长鸣一声 */
#define UART_LINK_BUZZER_3S          (3u) /* 巡航: 每 3s 短促一声 */
#define UART_LINK_BUZZER_WARN        (4u) /* 断联告警: 急促鸣 */
#define UART_LINK_BUZZER_ARMED_IDLE (5u) /* 模式 3 已 arm 待车 S3+S4 请求: 每 ~2s 短鸣警示"已待命"(桨在转) */
#define UART_LINK_BUZZER_WAITING     (6u) /* 等待连接: 每 ~3s 短鸣(core2 本地 WAIT_RS485 用; core0 不下发此码) */
#define UART_LINK_BUZZER_THREE_BEEPS (7u) /* 完整系统资格上升沿: 三声一次 */

```

```

typedef enum
{
    UART_LINK_PROTO_STATUS_OK = 0u,
    UART_LINK_PROTO_STATUS_FRAME_READY,
    UART_LINK_PROTO_STATUS_INVALID_PARAM,
    UART_LINK_PROTO_STATUS_TYPE_ERROR,
    UART_LINK_PROTO_STATUS_CHECK_ERROR,
    UART_LINK_PROTO_STATUS_TAIL_ERROR
} uartLinkProtoStatusEnum;

```

```

typedef struct
{
    uint8_t type;
    uint8_t sequence;
    uint8_t value; /* 基础帧 payload */
    uint16_t beaconCount; /* STATUS 帧: 信标数 */
    uint8_t linkFlags; /* STATUS 帧: 链路标志 */
    uint8_t faultSource; /* STATUS: 必需视觉故障来源, 0=无 */
    uint8_t faultClass; /* STATUS: 必需视觉故障类别, 0=无 */
    uint32_t producerBootSession; /* STATUS/MOTION: Core1 cm7_0 启动身份, 0 非法 */
    uint8_t safetyState; /* CORE0_STATUS 帧: 飞控安全态 */
    uint8_t armProgress; /* CORE0_STATUS 帧: arming 进度 0~100 */
    uint8_t statusFlags; /* CORE0_STATUS 帧: 状态位(UART_LINK_C0_FLAG_*) */
    uint8_t sysCtrl; /* CORE0_STATUS 帧: 空地系统模式/RGB 意图/到顶标志(UART_LINK_SYS_*) */
    uint8_t buzzer; /* CORE0_STATUS 帧: 蜂鸣模式码(UART_LINK_BUZZER_*, 中转 core2 buzzer 库 pattern 层播放) */
    uint8_t poseEpoch; /* CORE0_POSE 帧: core0 链路初始化 epoch, 变更表示 core0 重启/共享区需重新锁存 */
    uint16_t poseXQ8; /* CORE0_POSE 帧: core0 灯板 X, 同坐标 Q8.8 */
    uint16_t poseYQ8; /* CORE0_POSE 帧: core0 灯板 Y, 同坐标 Q8.8 */
    float poseOrientationRad; /* CORE0_POSE 帧: 灯板方向 rad, 范围约 [-pi, pi] */
    uint8_t poseConfidenceQ8; /* CORE0_POSE 帧: 方向置信度 Q0.8 (0.255) */
    uint8_t poseFlags; /* CORE0_POSE 帧: flags(UART_LINK_POSE_FLAG_*) */
    float motionVx; /* MOTION 帧: 速度指令 vx; core0 速度前馈直用, pitch 前馈用其指令微分 */
    float motionVy; /* MOTION 帧: 速度指令 vy; core0 速度前馈直用, roll 前馈用其指令微分 */
    float motionWz; /* MOTION 帧: 角速度指令 wz; core0 yaw 前馈直通 */
    uint8_t motionFlags; /* MOTION 帧: flags(UART_LINK_MOTION_FLAG_*, bit0=remoteTakeoffReq) */
    /* KVM_CONTROL/EVENT/STATUS: 显式字段按帧类型编码, 禁止裸结构体传输。 */
    uint32_t kvmTargetSession; /* 目标路由 owner boot session echo, 重启后旧事件拒绝 */
    uint32_t kvmRouteSession; /* Core0 cm7_1 菜单 boot epoch */
    uint32_t kvmRouteEpoch; /* 每次切目标递增 */
    uint32_t kvmEventSeq; /* 每 route epoch 从 1 起; STATUS 中为最后终态 ACK seq */
    uint16_t kvmLeaseMs; /* CONTROL 租约上限 */
    uint16_t kvmCore3AgeMs; /* STATUS: Core3 菜单状态年龄 */
    uint8_t kvmTarget;
    uint8_t kvmActiveTarget;
    uint8_t kvmKeyId;
    uint8_t kvmKeyState;
    uint8_t kvmMenuStates; /* STATUS: 低 4 位 Core1、高 4 位 Core3 原始菜单态 */
    uint8_t kvmFlags;
    uint8_t kvmAckResult;
    uint32_t paramRequestId;
    uint16_t paramOffset;
    uint8_t paramOp;
    uint8_t paramReserved;
    uint8_t paramTarget;
    uint8_t paramStatus;
    uint8_t paramLength;
    uint8_t paramData[UART_LINK_PARAM_DATA_BYTES];
    paramSyncGlobalRequest paramGlobalRequest;
    paramSyncGlobalState paramGlobalState;
} uartLinkFrameStruct;

```

```

typedef struct
{
    uint8_t index;
    uint8_t expectLen; /* 收到 type 后据此定帧长 (0=未定, 等 type 字节) */
    uint8_t buffer[UART_LINK_FRAME_LENGTH_MAX];
} uartLinkParserStruct;

```

```
void uartLinkProtoInit(uartLinkParserStruct *parser);
uint8_t uartLinkProtoChecksum(const uartLinkFrameStruct *frame);
uartLinkProtoStatusEnum uartLinkProtoEncode(const uartLinkFrameStruct *frame, uint8_t *buffer, uint8_t bufferSize);
uartLinkProtoStatusEnum uartLinkProtoInputByte(uartLinkParserStruct *parser, uint8_t data, uartLinkFrameStruct *frame);

#endif /* _UART_LINK_PROTO_H_ */
```

core1_cyt4bb7/project/code/config/commonDef.h

SHA-256 :
ae4946eeab87b07e01a0956f92f11f0f79652a303cf2e446636abfc986a59bd3

代码 (c)

```
#ifndef COMMON_DEF_H
#define COMMON_DEF_H

#include <stdint.h>

/*
 * 跨模块公共定义：纯常量 / 数学工具 / 图像类型。
 * 本文件不含编译期硬件选择器（在 boardConfig.h，需要者直接包含）、也不含运行时可调参数。
 * 这里全是无副作用、可安全包含的常量与工具。
 */

/* =====
 * 1. 数学宏（钳位 / 角度换算 / 圆周率）
 * ===== */
#define range(x, min, max) (((x) < (min)) ? (min) : (((x) > (max)) ? (max) : (x)))
#define unsignRange(x, max) (((x) > (max)) ? (max) : (x))

#define toRad(x) ((x) * 3.14159265358979323846f / 180.0f) /* 储备：当前无消费者，与在用的 toDeg 对称保留 */
#define toDeg(x) ((x) * 180.0f / 3.14159265358979323846f)

/* PI 可能已由 CMSIS DSP 的 arm_math.h 定义（值在 float 精度下等价），
 * 用 #ifndef 守卫避免 Pa181 宏重定义警告 */
#ifndef PI
#define PI (3.14159265358979323846f)
#endif
#define TWO_PI (2.0f * PI)
#define HALF_PI (0.5f * PI)

/* =====
 * 2. 二值图像颜色
 * ===== */
#define BLACK 0 /* 二值背景色 */
#define WHITE 255 /* 二值前景色 */
#define cReverse(x) ((x) ? BLACK : WHITE) /* 颜色翻转 */

/* =====
 * 3. 跨模块 inline 工具（钳位 / 角度归一化）
 * ===== */
static inline float clampf(float v, float lo, float hi)
{
    if (v < lo) return lo;
    if (v > hi) return hi;
    return v;
}

static inline float wrapToPi(float a)
{
    while (a > PI) a -= TWO_PI;
    while (a < -PI) a += TWO_PI;
    return a;
}

/* =====
 * 4. 通用图像 typedef
 * u8Image 为视觉管线主力类型。
 * ===== */
typedef struct
{
    uint8_t* ptr;

    uint32_t xLen;
```

```
uint32_t yLen;

uint32_t size;
} u8Image;

#endif
```

core1_cyt4bb7/project/code/config/configBeaconDetect.h

SHA-256

:

18acf5315779b46844852e68fdb2b88d743673bec4899d21ecf1dbda200c7e44

代码 (c)

```
#ifndef _CONFIG_BEACON_DETECT_H_
#define _CONFIG_BEACON_DETECT_H_

/*
 * core1 cm_7_1 信标'检测层'调参中心： BinaryCC 与 Legacy Canny/om_beacon 热切换 + 相机曝光。
 * 默认值由 visionParamsLoadDefaults / menuLoadDefaults 加载，运行时菜单可改 + flash 持久化
 * （见 vision/visionParams.c + app/menuModel.c） 。
 *
 * 三层配置头拆分（2026-07 重构，原 configVision.h 拆为 detect/track）：
 * configBeaconDetect.h — 本文件： DetectMode/BinaryCC/Legacy Canny/曝光/信标检测
 * configBeaconTrack.h — 时间一致性跟踪（关联/确认/丢弃/coast/位置/面积 alpha）
 * configBeaconPlanning.h— 路径规划（中心/目标选择/增益/sign/限幅/Min/slew/施密特/TD/派发）
 * 底层 openMorph 缓冲尺寸（OM_*）在 vision/openMorph_config.h，不在此。
 *
 * 图像坐标系： core1 相机 MT9V03X 188×120 灰度。默认值调整后需重新实拍复核；
 * 菜单运行时可调并由 flash 持久化，换硬件或接线后同样必须复核。
 */

/* =====
 * 1. DetectMode + 阈值 + 相机曝光（默认值 + 菜单编辑约束）
 * 规则为 low <= gray <= high； high=255 等价于关闭过曝拒绝。
 * 当前默认使用 Legacy Canny； BinaryCC 保留为菜单热切换回滚/对比。
 * 旧字段名 cannyLow/cannyHigh 仅为兼容 flash 前缀；当前语义为 BinaryCC low/high。
 * ===== */
#define VISION_CANNY_SIGMA (0.75f) /* Legacy Canny 保留项； BinaryCC 不使用 */
#define VISION_DETECT_MODE_BINARYCC (0u)
#define VISION_DETECT_MODE_LEGACY_CANNY (1u)
#define VISION_DETECT_MODE_DEFAULT VISION_DETECT_MODE_LEGACY_CANNY
#define VISION_BINARY_LOW_DEFAULT (40u)
#define VISION_BINARY_HIGH_DEFAULT (255u)
#define VISION_CANNY_LOW_DEFAULT VISION_BINARY_LOW_DEFAULT
#define VISION_CANNY_HIGH_DEFAULT VISION_BINARY_HIGH_DEFAULT
#define VISION_LEGACY_CANNY_LOW_DEFAULT (150u)
#define VISION_LEGACY_CANNY_HIGH_DEFAULT (200u)
#define VISION_LEGACY_CANNY_THRESHOLD_MIN (5.0f)
#define VISION_LEGACY_CANNY_THRESHOLD_MAX (500.0f)
#define VISION_LEGACY_CANNY_THRESHOLD_GAP (5.0f)
#define VISION_CANNY_THRESHOLD_MIN (5.0f)
#define VISION_CANNY_THRESHOLD_MAX (255.0f)
#define VISION_CANNY_THRESHOLD_GAP (0.0f) /* BinaryCC 允许 low == high */
#define VISION_CANNY_STEP_SHORT (1.0f)
#define VISION_CANNY_STEP_LONG (10.0f)

/* 曝光（MT9V03X set_exposure_time 行单位）。MIN/MAX 为菜单编辑约束，上板实拍可微调；
 * 步长采用 SHORT=1/LONG=10，兼顾精调与快速跨区间整定。 */
#define VISION_EXPOSURE_DEFAULT (300u)
#define VISION_EXPOSURE_MIN (1u)
#define VISION_EXPOSURE_MAX (2000u)
#define VISION_EXPOSURE_STEP_SHORT (1.0f)
#define VISION_EXPOSURE_STEP_LONG (10.0f)

/* =====
 * 2. 信标检测默认值（两种 DetectMode 共用，菜单可调 + flash 持久化）
 * 默认按 Legacy Canny 路线保留宽面积门；面积下限覆盖远距离 1.3 像素信标。
 * HuAreaMin 仅用于 BinaryCC，极小远点跳过形状拒绝，避免远距离信标被模板误杀。
 * ===== */
#define VISION_BEACON_SIZE_MIN_DEFAULT (1u)
#define VISION_BEACON_SIZE_MAX_DEFAULT (5000u)
#define VISION_BEACON_ECC_MAX_DEFAULT (8.0f) /* 0=关闭； BinaryCC 连通域容许远距/斜视信标 */
#define VISION_BEACON_BRIGHTNESS_MIN_DEFAULT (40u) /* mean brightness； low=40 时等价于不额外收窄 */
#define VISION_BEACON_MAX_ENDPOINTS_DEFAULT (20u) /* Legacy om_beacon 端点门； BinaryCC 不使用 */
```

```

#define VISION_BEACON_FILL_MIN_DEFAULT      (0.05f) /* 0=关闭；默认只拦大 bbox 低填充干扰 */
#define VISION_BEACON_HU_MAX_DEFAULT        (60.0f) /* 0=关闭；默认宽门，仅拦明显形状干扰 */
#define VISION_BEACON_HU_AREA_MIN_DEFAULT   (1u)
#define VISION_BEACON_HU_MAX_MIN            (0.0f)
#define VISION_BEACON_HU_MAX_MAX            (200.0f)
#define VISION_BEACON_HU_MAX_STEP_SHORT     (1.0f)
#define VISION_BEACON_HU_MAX_STEP_LONG      (10.0f)
#define VISION_BEACON_HU_AREA_MIN_MIN        (1.0f)
#define VISION_BEACON_HU_AREA_MIN_MAX        (100.0f)
#define VISION_BEACON_HU_AREA_MIN_STEP_SHORT (1.0f)
#define VISION_BEACON_HU_AREA_MIN_STEP_LONG  (5.0f)

/* PC 仿真临时 Hu 模板，七阶全量参与距离计算；后续可用更多负样本再收紧 HuMax。 */
#define VISION_BEACON_HU_TEMPLATE_0 (0.180488205803f)
#define VISION_BEACON_HU_TEMPLATE_1 (0.0079012345679f)
#define VISION_BEACON_HU_TEMPLATE_2 (0.000123277089059f)
#define VISION_BEACON_HU_TEMPLATE_3 (0.00000801787369286f)
#define VISION_BEACON_HU_TEMPLATE_4 (0.0f)
#define VISION_BEACON_HU_TEMPLATE_5 (0.0f)
#define VISION_BEACON_HU_TEMPLATE_6 (0.0f)

#endif /* _CONFIG_BEACON_DETECT_H_ */

```

core1_cyt4bb7/project/code/config/configBeaconFusion.h

SHA-256

:

93dbc9c421d2680d135dfc5bd5fe5c811cc35e24207924c1a2b8e5df7c6a45cb

代码 (c)

```

#ifndef CONFIG_BEACON_FUSION_H
#define CONFIG_BEACON_FUSION_H

#define BEACON_MAX_BEACONS_DEFAULT      (3u)
#define BEACON_MAX_BEACONS_MIN          (1u)
#define BEACON_MAX_BEACONS_MAX          (32u)

#define CORE3_OBSERVATION_TIMEOUT_MS_DEFAULT (600u)
#define CORE3_OBSERVATION_TIMEOUT_MS_MIN     (100u)
#define CORE3_OBSERVATION_TIMEOUT_MS_MAX     (2000u)
#define CORE3_OBSERVATION_TIMEOUT_STEP_SHORT (20.0f)
#define CORE3_OBSERVATION_TIMEOUT_STEP_LONG  (100.0f)

/* 三来源候选 fresh 门与健康观察超时。二者不得混作同一故障事实。 */
#define C1_SOURCE_FRESH_GATE_MS_DEFAULT (100u)
#define C3_SOURCE_FRESH_GATE_MS_DEFAULT (100u)
#define C4_SOURCE_FRESH_GATE_MS_DEFAULT (100u)
/* 范围/步长仍为三来源共享策略；默认值拆分以支持逐字段 Flash 回写源码。 */
#define SOURCE_FRESH_GATE_MS_DEFAULT      C1_SOURCE_FRESH_GATE_MS_DEFAULT
#define SOURCE_FRESH_GATE_MS_MIN          (20u)
#define SOURCE_FRESH_GATE_MS_MAX          (600u)
#define SOURCE_FRESH_GATE_STEP_SHORT      (20.0f)
#define SOURCE_FRESH_GATE_STEP_LONG       (100.0f)
#define CORE4_OBSERVATION_TIMEOUT_MS_DEFAULT (600u)
#define CORE4_OBSERVATION_TIMEOUT_MS_MIN     (100u)
#define CORE4_OBSERVATION_TIMEOUT_MS_MAX     (2000u)
#define SOURCE_OBSERVATION_TIMEOUT_STEP_SHORT (20.0f)
#define SOURCE_OBSERVATION_TIMEOUT_STEP_LONG  (100.0f)

#define CORE3_CALIBRATION_CENTER_X_DEFAULT (94.0f)
#define CORE3_CALIBRATION_CENTER_Y_DEFAULT (60.0f)
#define CORE3_CALIBRATION_SCALE_DEFAULT    (1.0f)
#define CORE3_CALIBRATION_YAW_RAD_DEFAULT  (0.0f)
#define CORE3_CALIBRATION_CENTER_X_MIN     (0.0f)
#define CORE3_CALIBRATION_CENTER_X_MAX     (187.0f)
#define CORE3_CALIBRATION_CENTER_Y_MIN     (0.0f)
#define CORE3_CALIBRATION_CENTER_Y_MAX     (119.0f)
#define CORE3_CALIBRATION_SCALE_MIN         (0.10f)
#define CORE3_CALIBRATION_SCALE_MAX         (10.0f)
#define CORE3_CALIBRATION_YAW_RAD_MIN       (-3.1415927f)
#define CORE3_CALIBRATION_YAW_RAD_MAX       (3.1415927f)
#define CORE3_CALIBRATION_CENTER_STEP_SHORT (1.0f)
#define CORE3_CALIBRATION_CENTER_STEP_LONG  (10.0f)
#define CORE3_CALIBRATION_SCALE_STEP_SHORT  (0.01f)
#define CORE3_CALIBRATION_SCALE_STEP_LONG   (0.10f)

```

```

#define CORE3_CALIBRATION_YAW_STEP_SHORT    (0.01f)
#define CORE3_CALIBRATION_YAW_STEP_LONG    (0.10f)

#define CORE3_CALIBRATION_MIRROR_X_DEFAULT    (0u)
#define CORE3_CALIBRATION_FWD_SIGN_DEFAULT    (1)
#define CORE3_CALIBRATION_RIGHT_SIGN_DEFAULT    (1)

/* Core3 独立 Core0Pose 映射偏置；不得随 Core1 偏置默认值联动。 */
#define CORE3_CORE0_POSE_OFFSET_X_DEFAULT    (43.0f)
#define CORE3_CORE0_POSE_OFFSET_Y_DEFAULT    (-115.0f)

/* Core3 活动源的运动输出极性独立于 Core1 下视相机。 */
#define CORE3_PLANNING_VX_SIGN_DEFAULT    (-1)
#define CORE3_PLANNING_VY_SIGN_DEFAULT    (-1)

/* Core4 相机安装方向与旧 Core1 一致：观察坐标旋转+pi/2 后映射到前/右误差。 */
#define CORE4_CALIBRATION_CENTER_X_DEFAULT    (94.0f)
#define CORE4_CALIBRATION_CENTER_Y_DEFAULT    (60.0f)
#define CORE4_CALIBRATION_SCALE_DEFAULT    (1.0f)
#define CORE4_CALIBRATION_YAW_RAD_DEFAULT    (1.57079633f)
#define CORE4_CALIBRATION_MIRROR_X_DEFAULT    (0u)
#define CORE4_CALIBRATION_FWD_SIGN_DEFAULT    (1)
#define CORE4_CALIBRATION_RIGHT_SIGN_DEFAULT    (-1)
#define CORE4_CORE0_POSE_OFFSET_X_DEFAULT    (0.0f)
#define CORE4_CORE0_POSE_OFFSET_Y_DEFAULT    (100.0f)
#define CORE4_PLANNING_KP_VX_DEFAULT    (0.0240000002f)
#define CORE4_PLANNING_KP_VY_DEFAULT    (0.0270000007f)
#define CORE4_PLANNING_VX_SIGN_DEFAULT    (1)
#define CORE4_PLANNING_VY_SIGN_DEFAULT    (-1)

#endif /* CONFIG_BEACON_FUSION_H */

```

core1_cyt4bb7/project/code/config/configBeaconPlanning.h

SHA-256

:

14754293531402b541adb41d860b342eada47cf40c4be99cc449a9113c7a31bc

代码 (c)

```

#ifndef _CONFIG_BEACON_PLANNING_H_
#define _CONFIG_BEACON_PLANNING_H_

/*
 * core1 cm_7_1 路径规划调参中心 (beaconNavigator)。
 * 三层配置头拆分见 config/configBeaconDetect.h 顶注。
 *
 * 两类宏：
 * A. 编译期节拍约束（不入 visionParams/flash，改动需重编）。
 * B. 当前 Navigator 运行参数的默认值、范围和步长，经 visionParams、菜单及 Flash 持久化。
 *
 * 2026-07 重构：删除输出一阶低通(planningOutLpfAlpha)与 LOST_COAST 衰减(COAST_MS/DECAY)；
 * 输出整形未级改跟踪微分器 TD(util/trackingDifferentiator)。极性 sign、派发总开关
 * PLANNING_DISPATCH/SEARCH、派发超时 PLAN_TIMEOUT、SEARCH Wz/Timeout 由编译宏改为
 * 运行时 visionParams 字段（本文件仅留其默认值/范围）。
 */

/* =====
 * A. 编译期静态项
 * ===== */
/* 规划节拍 10ms (cm_7_1 PIT_CH2 分频)。 */
#define PLANNING_DT_S (0.01f)

/* =====
 * B. 运行时默认值 / 范围 / 步长 (visionParams 镜像 + 菜单 + flash)
 * ===== */

/* 平移 P 增益输入为像素误差；TRACK 不生成旋转指令。 */
#define PLANNING_KP_VX_DEFAULT    (0.0390000232f)
#define PLANNING_KP_VY_DEFAULT    (0.0440000147f)
#define CORE3_PLANNING_KP_VX_DEFAULT (0.0240000021f)
#define CORE3_PLANNING_KP_VY_DEFAULT (0.0270000044f)
#define PLANNING_KP_LINEAR_MIN (0.0f)
#define PLANNING_KP_LINEAR_MAX (5.000f)
#define PLANNING_KP_VX_STEP_SHORT (0.001f)
#define PLANNING_KP_VY_STEP_SHORT (0.001f)

```

```

#define PLANNING_KP_VX_STEP_LONG (0.010f)
#define PLANNING_KP_VY_STEP_LONG (0.010f)

/* 离散极性 sign (仅 -1/+1, 上板逐轴标定; 由 visionParams int8 承载, 菜单 step 2 翻转) 。 */
#define PLANNING_VX_SIGN_DEFAULT (1)
#define PLANNING_VY_SIGN_DEFAULT (-1)
#define PLANNING_WZ_SIGN_DEFAULT (-1)
#define PLANNING_SIGN_STEP (2.0f)

/* 径向施密特死区默认值与菜单步长 (像素半径) 。 */
#define PLANNING_DEADZONE_ENTER_DEFAULT (1.0f)
#define PLANNING_DEADZONE_EXIT_DEFAULT (0.0f)
#define PLANNING_DEADZONE_ENTER_MIN (1.0f)
#define PLANNING_DEADZONE_ENTER_MAX (60.0f)
#define PLANNING_DEADZONE_EXIT_MIN (0.0f)
#define PLANNING_DEADZONE_GAP_MIN (0.5f)
#define PLANNING_DEADZONE_STEP_SHORT (0.5f)
#define PLANNING_DEADZONE_STEP_LONG (2.0f)

/* Core0Pose 标定: flags 门控位置更新; 方向置信度只门控 heading 更新。 */
/* 与 Core0 BinaryCC OriConf 默认门槛一致, 避免 Core1/Core3 再次抬高朝向更新门。 */
#define PLANNING_CORE0_POSE_MIN_CONF_DEFAULT (0.05f)
#define PLANNING_CORE0_POSE_MIN_CONF_MIN (0.0f)
#define PLANNING_CORE0_POSE_MIN_CONF_MAX (1.0f)
#define PLANNING_CORE0_POSE_CONF_STEP_SHORT (0.05f)
#define PLANNING_CORE0_POSE_CONF_STEP_LONG (0.20f)
#define PLANNING_CORE0_POSE_OFFSET_X_DEFAULT (2.5f)
#define PLANNING_CORE0_POSE_OFFSET_Y_DEFAULT (-5.0f)
#define PLANNING_CORE0_POSE_OFFSET_X_MIN (-188.0f)
#define PLANNING_CORE0_POSE_OFFSET_X_MAX (188.0f)
#define PLANNING_CORE0_POSE_OFFSET_Y_MIN (-120.0f)
#define PLANNING_CORE0_POSE_OFFSET_Y_MAX (120.0f)
#define PLANNING_CORE0_POSE_OFFSET_STEP_SHORT (0.5f)
#define PLANNING_CORE0_POSE_OFFSET_STEP_LONG (5.0f)
#define PLANNING_CORE0_POSE_HEADING_OFFSET_DEFAULT (0.0f)
#define PLANNING_CORE0_POSE_HEADING_OFFSET_MIN (-3.1416f)
#define PLANNING_CORE0_POSE_HEADING_OFFSET_MAX (3.1416f)
#define PLANNING_CORE0_POSE_HEADING_STEP_SHORT (0.05f)
#define PLANNING_CORE0_POSE_HEADING_STEP_LONG (0.20f)
#define PLANNING_CORE0_POSE_HEADING_SIGN_DEFAULT (1)
#define PLANNING_CORE0_POSE_HOLD_MS_DEFAULT (3000u)
#define PLANNING_CORE0_POSE_HOLD_MS_MIN (0.0f)
#define PLANNING_CORE0_POSE_HOLD_MS_MAX (10 000.0f)
#define PLANNING_CORE0_POSE_HOLD_MS_STEP_SHORT (100.0f)
#define PLANNING_CORE0_POSE_HOLD_MS_STEP_LONG (500.0f)

/* 输出幅值限幅默认值与菜单步长; Core2 仍在本端物理上限处独立 fail-closed。 */
#define PLANNING_VX_MAX_DEFAULT (1.85000 014f)
#define PLANNING_VY_MAX_DEFAULT (1.8499999f)
#define PLANNING_WZ_MAX_DEFAULT (7.80f)
#define PLANNING_VX_MIN_DEFAULT (7.4505806e-09f) /* 完赛整定绝对值, 菜单调绝对值 */
#define PLANNING_VY_MIN_DEFAULT (7.4505806e-09f) /* 完赛整定绝对值, 菜单调绝对值 */
#define PLANNING_WZ_MIN_DEFAULT (0.00f) /* 完赛整定绝对值, 菜单调绝对值 */
#define PLANNING_V_LINEAR_LIMIT_MIN (0.0f)
#define PLANNING_V_LINEAR_LIMIT_MAX (15.00f)
#define PLANNING_WZ_LIMIT_MIN (0.0f)
#define PLANNING_WZ_LIMIT_MAX (70.00f)
#define PLANNING_V_LIMIT_STEP_SHORT (0.05f)
#define PLANNING_V_LIMIT_STEP_LONG (0.20f)
#define PLANNING_WZ_LIMIT_STEP_SHORT (0.10f)
#define PLANNING_WZ_LIMIT_STEP_LONG (0.50f)
#define PLANNING_V_MIN_STEP_SHORT (0.01f)
#define PLANNING_V_MIN_STEP_LONG (0.05f)
#define PLANNING_WZ_MIN_STEP_SHORT (0.01f)
#define PLANNING_WZ_MIN_STEP_LONG (0.05f)

/* 三轴输出斜率限制 (每秒最大变化量); 0 会永久冻结输出, 因此运行时下界严格大于 0。 */
#define PLANNING_VX_SLEW_DEFAULT (30.0f)
#define PLANNING_VY_SLEW_DEFAULT (30.0f)
#define PLANNING_WZ_SLEW_DEFAULT (10.0f)
#define PLANNING_SLEW_MIN (0.1f)
#define PLANNING_SLEW_MAX (100.0f)
#define PLANNING_SLEW_STEP_SHORT (0.5f)
#define PLANNING_SLEW_STEP_LONG (5.0f)

/* 输出跟踪微分器 TD (末级平滑, 替代原一阶低通)。h=节拍(PLANNING_DT_S)。
* R 越大跟踪越快。N 为滤波因子; 默认值按当前上板定点规划联调整定。
* NaN/Inf 回默认再 clamp (见 visionParamsClamp)。三输出轴 R 独立、N 共享。 */
#define PLANNING_TD_RVX_DEFAULT (383.0f)

```



```

#define PLANNING_TD_RVY_DEFAULT (348.0f)
#define PLANNING_TD_RWZ_DEFAULT (170.0f)
#define PLANNING_TD_N_DEFAULT (5.0f)
#define PLANNING_TD_VX_ENABLE_DEFAULT (0u)
#define PLANNING_TD_VY_ENABLE_DEFAULT (0u)
#define PLANNING_TD_WZ_ENABLE_DEFAULT (0u)
#define PLANNING_TD_R_MIN (0.10f)
#define PLANNING_TD_R_MAX (2000.0f)
#define PLANNING_TD_N_MIN (1.0f)
#define PLANNING_TD_N_MAX (500.0f)
#define PLANNING_TD_R_STEP_SHORT (0.5f)
#define PLANNING_TD_R_STEP_LONG (5.0f)
#define PLANNING_TD_N_STEP_SHORT (0.5f)
#define PLANNING_TD_N_STEP_LONG (2.0f)

/* 派发总开关 / SEARCH 旋转使能（运行时 uint8，归一化 0/1；默认与原编译宏一致=1）。
 * dispatchEnable：模式 2 下发运动总门控（跨核经 planToLinkStruct 传给 cm_7_0 派发决策）。
 * searchEnable：SEARCH 态搜索输出使能（最高风险自主运动，可先关仅验 TRACK）。*/
#define PLANNING_DISPATCH_ENABLE_DEFAULT (1u)
#define PLANNING_SEARCH_ENABLE_DEFAULT (1u)
#define PLANNING_ENABLE_STEP (1.0f)

/* SEARCH yaw 目标角速度与搜索超时（ms）。timeout=0 表示不超时；最高风险自主运动参数。
 * yaw 方向由排除上个目标后的 fresh confirmed 信标半画幅多数决定，无统计时按 WzSign。*/
#define PLANNING_SEARCH_WZ_DEFAULT (1.0f)
#define PLANNING_SEARCH_WZ_MIN (0.0f)
#define PLANNING_SEARCH_WZ_MAX (7.00f)
#define PLANNING_SEARCH_WZ_STEP_SHORT (0.10f)
#define PLANNING_SEARCH_WZ_STEP_LONG (0.50f)
#define PLANNING_SEARCH_TIMEOUT_MS_DEFAULT (4000u)
#define PLANNING_SEARCH_TIMEOUT_MS_MIN (0.0f)
#define PLANNING_SEARCH_TIMEOUT_MS_MAX (30 000.0f)
#define PLANNING_SEARCH_TIMEOUT_STEP_SHORT (500.0f)
#define PLANNING_SEARCH_TIMEOUT_STEP_LONG (2000.0f)

/* cm_7_0 派发新鲜度超时（ms）：规划 seq 超此时长未变即判失效，下发零速。运行时，跨核传 cm_7_0。*/
#define PLANNING_PLAN_TIMEOUT_MS_DEFAULT (100u)
#define PLANNING_PLAN_TIMEOUT_MS_MIN (10.0f)
#define PLANNING_PLAN_TIMEOUT_MS_MAX (1000.0f)
#define PLANNING_PLAN_TIMEOUT_STEP_SHORT (10.0f)
#define PLANNING_PLAN_TIMEOUT_STEP_LONG (50.0f)

/* 模式 2 起飞完成后的 one-shot 运动门：先延时，再按开关原地等待或以物理+vx 寻找首灯。*/
#define PLANNING_POST_TAKEOFF_DELAY_MS_DEFAULT (3000u)
#define PLANNING_POST_TAKEOFF_DELAY_MS_MIN (0.0f)
#define PLANNING_POST_TAKEOFF_DELAY_MS_MAX (10 000.0f)
#define PLANNING_POST_TAKEOFF_DELAY_STEP_SHORT (100.0f)
#define PLANNING_POST_TAKEOFF_DELAY_STEP_LONG (500.0f)
#define PLANNING_FIRST_BEACON_FORWARD_SPEED_DEFAULT (1.00f)
#define PLANNING_FIRST_BEACON_FORWARD_SPEED_MIN (0.0f)
#define PLANNING_FIRST_BEACON_FORWARD_SPEED_STEP_SHORT (0.05f)
#define PLANNING_FIRST_BEACON_FORWARD_SPEED_STEP_LONG (0.20f)
#define PLANNING_FIRST_BEACON_FORWARD_ENABLE_DEFAULT (0u)

#endif /* _CONFIG_BEACON_PLANNING_H_ */

```

core1_cyt4bb7/project/code/config/configBeaconTrack.h

SHA-256

:

cdfbd684b725d6ba2c792622690e57a5fa61a20557d6fc689321bd0d7b5badc1

代码 (c)

```

#ifndef _CONFIG_BEACON_TRACK_H_
#define _CONFIG_BEACON_TRACK_H_

/*
 * core1 cm_7_1 信标“跟踪层”调参中心：时间一致性跟踪（抗光关键层）。
 * 连续 confirmHits 帧关联才确认证输出、连续 dropMisses 帧丢失才删除、coastMisses 帧内丢失仍滑行。
 * 关联更新用一阶低通平滑（位置/面积各一 alpha，越小越平滑）；新轨迹直贴测量值。
 * 轨迹内部用坐标速度(px/frame)与面积速度(area pixels/frame)预测下一帧关联残差；
 * 这些速度只作为轨迹事实，不负责 active target 选择。
 * 详见 vision/beaconTrack.c。默认值运行时菜单可调 + flash 持久化。
 *
 * 三层配置头拆分见 config/configBeaconDetect.h 顶注。

```

```

*/

#define VISION_TRACK_MAX_ASSOC_DIST_DEFAULT (14.0f) /* 帧间质心最大关联距离(px) */
#define VISION_TRACK_SIZE_RATIO_MAX_DEFAULT (3.0f) /* 面积连续性: 检测面积/预测面积 比值上限 */
#define VISION_TRACK_CONFIRM_HITS_DEFAULT (3u) /* 连续命中确认帧数 */
#define VISION_TRACK_DROP_MISSES_DEFAULT (4u) /* 已确认轨迹连续丢失删除帧数 */
#define VISION_TRACK_COAST_MISSES_DEFAULT (2u) /* 已确认轨迹丢失时仍输出(滑行)帧数 */

/* 发布边界施密特门: 0/0 表示该边关闭; 非关闭边 Enter>=1 且 Exit<Enter。
* Exit 需连续越界 hold 帧才撤销发布资格, 滤掉边缘单帧检测抖动。
* Core1 因下视位姿默认关闭四边发布门与 EdgeBand; Core3/Core4 保留各自独立边界。 */
#define VISION_TRACK_BORDER_EXIT_HOLD_FRAMES_DEFAULT (2u)
#define VISION_TRACK_EDGE_PREDICTION_HOLD_MS_DEFAULT (500u)
#define VISION_TRACK_EDGE_PREDICTION_HOLD_MS_MIN (0.0f)
#define VISION_TRACK_EDGE_PREDICTION_HOLD_MS_MAX (10 000.0f)
#define VISION_TRACK_EDGE_PREDICTION_HOLD_STEP_SHORT (20.0f)
#define VISION_TRACK_EDGE_PREDICTION_HOLD_STEP_LONG (100.0f)
#define VISION_TRACK_CAMERA_FRAME_PERIOD_MS (20u)
#define VISION_TRACK_EDGE_BAND_C1_LEFT_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_C1_RIGHT_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_C1_TOP_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_C1_BOTTOM_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_C3_LEFT_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_C3_RIGHT_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_C3_TOP_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_C3_BOTTOM_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_C4_LEFT_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_C4_RIGHT_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_C4_TOP_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_C4_BOTTOM_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_PX_MIN (0.0f)
#define VISION_TRACK_EDGE_BAND_X_MAX (93.0f)
#define VISION_TRACK_EDGE_BAND_Y_MAX (59.0f)
#define VISION_TRACK_EDGE_BAND_STEP_SHORT (1.0f)
#define VISION_TRACK_EDGE_BAND_STEP_LONG (4.0f)
#define VISION_TRACK_EDGE_VERTICAL_ASSOC_SCALE (2.0f)
#define VISION_TRACK_EDGE_ASSOC_PREDICT_FRAMES_MAX (3.0f)
#define VISION_TRACK_BORDER_ENTER_LEFT_DEFAULT (0u)
#define VISION_TRACK_BORDER_EXIT_LEFT_DEFAULT (0u)
#define VISION_TRACK_BORDER_ENTER_RIGHT_DEFAULT (0u)
#define VISION_TRACK_BORDER_EXIT_RIGHT_DEFAULT (0u)
#define VISION_TRACK_BORDER_ENTER_TOP_DEFAULT (0u)
#define VISION_TRACK_BORDER_EXIT_TOP_DEFAULT (0u)
#define VISION_TRACK_BORDER_ENTER_BOTTOM_DEFAULT (0u)
#define VISION_TRACK_BORDER_EXIT_BOTTOM_DEFAULT (0u)
#define VISION_TRACK_CORE3_BORDER_ENTER_LEFT_DEFAULT (0u)
#define VISION_TRACK_CORE3_BORDER_EXIT_LEFT_DEFAULT (0u)
#define VISION_TRACK_CORE3_BORDER_ENTER_RIGHT_DEFAULT (0u)
#define VISION_TRACK_CORE3_BORDER_EXIT_RIGHT_DEFAULT (0u)
#define VISION_TRACK_CORE3_BORDER_ENTER_TOP_DEFAULT (0u)
#define VISION_TRACK_CORE3_BORDER_EXIT_TOP_DEFAULT (0u)
#define VISION_TRACK_CORE3_BORDER_ENTER_BOTTOM_DEFAULT (0u)
#define VISION_TRACK_CORE3_BORDER_EXIT_BOTTOM_DEFAULT (0u)
#define VISION_TRACK_CORE4_BORDER_ENTER_LEFT_DEFAULT (0u)
#define VISION_TRACK_CORE4_BORDER_EXIT_LEFT_DEFAULT (0u)
#define VISION_TRACK_CORE4_BORDER_ENTER_RIGHT_DEFAULT (0u)
#define VISION_TRACK_CORE4_BORDER_EXIT_RIGHT_DEFAULT (0u)
#define VISION_TRACK_CORE4_BORDER_ENTER_TOP_DEFAULT (0u)
#define VISION_TRACK_CORE4_BORDER_EXIT_TOP_DEFAULT (0u)
#define VISION_TRACK_CORE4_BORDER_ENTER_BOTTOM_DEFAULT (0u)
#define VISION_TRACK_CORE4_BORDER_EXIT_BOTTOM_DEFAULT (0u)
#define VISION_TRACK_BORDER_X_MAX (93u)
#define VISION_TRACK_BORDER_Y_MAX (59u)
#define VISION_TRACK_BORDER_STEP_SHORT (1.0f)
#define VISION_TRACK_BORDER_STEP_LONG (4.0f)

/* 关联平滑一阶低通  $\alpha$  ( $tr = \alpha * measured + (1 - \alpha) * tr$ , 越小越平滑、越抗抖)。
* 位置/面积独立, 默认 1.00 (直贴测量值; 原硬编码 0.5 IIR 现可调)。上板复标: 移动信标  $\alpha$  太小易掉关联。 */
#define VISION_TRACK_POSITION_ALPHA_DEFAULT (1.00f)
#define VISION_TRACK_SIZE_ALPHA_DEFAULT (1.00f)
#define VISION_TRACK_ALPHA_MIN (0.01f)
#define VISION_TRACK_ALPHA_MAX (1.00f)
#define VISION_TRACK_ALPHA_STEP_SHORT (0.05f)
#define VISION_TRACK_ALPHA_STEP_LONG (0.20f)

/* 轨迹速度预测 (单位 px/frame, 仅用于下一帧关联与事实输出; 不进入控制 ABI)。 */
#define VISION_TRACK_VELOCITY_ALPHA_DEFAULT (0.35f)
#define VISION_TRACK_VELOCITY_MAX_FACTOR_DEFAULT (1.50f)
#define VISION_TRACK_PREDICT_FRAMES_DEFAULT (1.0f)

```

```
/* 面积趋势辅助关联。areaVel 单位为 area pixels/frame。
* 高影响权重/阈值进入 visionParams 菜单 + flash，归一化与 cap 仍为编译期安全约束。
* trackSizeRatioMax 仍是持久化参数，语义保持“面积连续性比值上限”，本轮只把比较基准改为预测面积。 */
#define VISION_TRACK_AREA_VEL_ALPHA_DEFAULT (0.35f)
#define VISION_TRACK_AREA_NORM_MIN (16.0f)
#define VISION_TRACK_ASSOC_SCORE_MAX_DEFAULT (1.75f)
#define VISION_TRACK_ASSOC_AREA_WEIGHT_DEFAULT (0.45f)
#define VISION_TRACK_ASSOC_AREA_VEL_WEIGHT_DEFAULT (0.35f)
#define VISION_TRACK_ASSOC_AREA_ERR_CAP (3.0f)
#define VISION_TRACK_ASSOC_AREA_VEL_ERR_CAP (3.0f)
#define VISION_TRACK_ASSOC_AREA_VEL_NORM (0.35f)
#define VISION_TRACK_ASSOC_AREA_VEL_MIN_ABS (8.0f)
#define VISION_TRACK_AREA_VEL_MAX_RATE (1.0f)

/* 菜单编辑约束（confirm/drop/coast 帧数、关联距离、面积比）。 */
#define VISION_TRACK_ASSOC_DIST_MIN (1.0f)
#define VISION_TRACK_ASSOC_DIST_MAX (100.0f)
#define VISION_TRACK_SIZE_RATIO_MIN (1.1f)
#define VISION_TRACK_SIZE_RATIO_MAX (10.0f)
#define VISION_TRACK_FRAMES_MIN (1.0f)
#define VISION_TRACK_FRAMES_MAX (60.0f)
#define VISION_TRACK_ASSOC_DIST_STEP_SHORT (0.5f)
#define VISION_TRACK_ASSOC_DIST_STEP_LONG (2.0f)
#define VISION_TRACK_SIZE_RATIO_STEP_SHORT (0.1f)
#define VISION_TRACK_SIZE_RATIO_STEP_LONG (0.5f)
#define VISION_TRACK_FRAMES_STEP_SHORT (1.0f)
#define VISION_TRACK_FRAMES_STEP_LONG (2.0f)

/* 面积趋势菜单编辑约束。权重 0=关闭对应 score 项。 */
#define VISION_TRACK_ASSOC_SCORE_MAX_MIN (0.10f)
#define VISION_TRACK_ASSOC_SCORE_MAX_MAX (10.00f)
#define VISION_TRACK_ASSOC_SCORE_MAX_STEP_SHORT (0.05f)
#define VISION_TRACK_ASSOC_SCORE_MAX_STEP_LONG (0.25f)
#define VISION_TRACK_ASSOC_WEIGHT_MIN (0.00f)
#define VISION_TRACK_ASSOC_WEIGHT_MAX (3.00f)
#define VISION_TRACK_ASSOC_WEIGHT_STEP_SHORT (0.05f)
#define VISION_TRACK_ASSOC_WEIGHT_STEP_LONG (0.25f)

#endif /* _CONFIG_BEACON_TRACK_H_ */
```

core1_cyt4bb7/project/code/config/configSerialDebug.h

SHA-256

39566ee68e86b86a929ee6c01edffbcd3ea51be3a3cb12850f2f410b29c54c71

:

代码 (c)

```
#ifndef _CONFIG_SERIAL_DEBUG_H_
#define _CONFIG_SERIAL_DEBUG_H_

/* 逐飞底层断言/异常文本串口配置。UART0 业务字节流仅承载参数网关。 */

/* 默认 debug UART: 若改串口且保留中断接收, 需同步更换 zf_common_debug 的中断 handler。 */
#define SERIAL_DEBUG_UART_INDEX (UART_0)
#define SERIAL_DEBUG_UART_BAUDRATE (1 000 000)
#define SERIAL_DEBUG_UART_TX_PIN (UART0_TX_P00_1)
#define SERIAL_DEBUG_UART_RX_PIN (UART0_RX_P00_0)
#define SERIAL_DEBUG_UART_USE_INTERRUPT (1)

/* 数值可比较的编译期保护; UART 枚举不在预处理期比较。 */
#if (SERIAL_DEBUG_UART_BAUDRATE < 1)
#error "SERIAL_DEBUG_UART_BAUDRATE must be > 0"
#endif

#endif /* _CONFIG_SERIAL_DEBUG_H_ */
```

core1_cyt4bb7/project/code/config/pinMap.h

SHA-256

68d168025791bda85d3758f886975ec653af58d7b1015e1a021d1f4ff259073e

:

代码 (c)

```
#ifndef _PIN_MAP_H_
#define _PIN_MAP_H_

/*
 * core1 飞机副控 / 视觉决策中枢 MCU 引脚集中管理（纯编译期 #define，零依赖）。
 * 右值为逐飞 GPIO/外设复用枚举（UART/TCPWM 通道名或裸 Pxx_y），
 * 仅在各消费 .c 的预处理期展开；本头不 include 任何驱动头，
 * token 的实际定义由消费方自带的 zf_driver_ * 头提供。
 *
 * 硬件事实详见 PROJECT.md §3.5 / §5；引脚汇总见 docs/PINOUT.md。
 */

/* =====
 * 核间双 UART（core0 ↔ core1 同板，两条独立链路，分组交叉相连）
 * 分组 0：core1 P04 ↔ core0 P10
 * 分组 1：core1 P10 ↔ core0 P14
 * 引脚号跨核不同，靠同板线缆相连。建议一条专传算法/遥测、
 * 一条专传人机交互（仅 core0 有按键，交互信息汇于 core0）。
 * *_NUM 为 UART 外设编号（外设复用选择，与引脚同属映射范畴）。
 * ===== */
#define LINK_GROUP0_UART_NUM    (UART_1)
#define LINK_GROUP0_UART_RX_PIN (UART1_RX_P04_0)
#define LINK_GROUP0_UART_TX_PIN (UART1_TX_P04_1)

/* =====
 * core3 后向观察板正式业务专线：
 * core1 P02_1 TX -> core3 P02_0 RX，
 * core3 P02_1 TX -> core1 P02_0 RX。
 * ===== */
#define OBSERVER_CORE3_UART_NUM    (UART_5)
#define OBSERVER_CORE3_UART_RX_PIN (UART5_RX_P02_0)
#define OBSERVER_CORE3_UART_TX_PIN (UART5_TX_P02_1)
#define OBSERVER_CORE3_UART_SCB    (SCB7)

/* core4 前向观察板正式业务专线：core1 UART6/P03。 */
#define OBSERVER_CORE4_UART_NUM    (UART_6)
#define OBSERVER_CORE4_UART_RX_PIN (UART6_RX_P03_0)
#define OBSERVER_CORE4_UART_TX_PIN (UART6_TX_P03_1)
#define OBSERVER_CORE4_UART_SCB    (SCB6)

/* =====
 * RS-485 收发器 THVD1450（UART_4，core1 ↔ core2 飞机-车模链路）
 * RE#/DE 单引脚控制方向：高=发送 / 低=接收；外部下拉默认接收。
 * ===== */
#define RS485_UART_NUM    (UART_4)
#define RS485_UART_RX_PIN (UART4_RX_P14_0) /* THVD1450 RO -> MCU RX */
#define RS485_UART_TX_PIN (UART4_TX_P14_1) /* MCU TX -> THVD1450 DI */
#define RS485_DIR_PIN     (P14_4)          /* RE#/DE 方向控制 */

/* =====
 * RGB LED 灯珠（三路 TCPWM，共阳极：控制脚接阴极，低电平点亮）
 * 右值为 TCPWM 通道令牌（通道+引脚一体，对标 core2 MOTOR_*_EN_PWM），
 * 消费方 pwm_init/pwm_set_duty 直接取用。
 * ===== */
#define RGB_LED_R_PWM (TCPWM_CH10_P05_1)
#define RGB_LED_G_PWM (TCPWM_CH11_P05_2)
#define RGB_LED_B_PWM (TCPWM_CH09_P05_0)

#endif /* _PIN_MAP_H_ */
```

core1_cyt4bb7/project/code/planning/beaconNavigator.c

SHA-256 : 41dfb8c71ac2097c94e88051aabbf3dc4a09c930866556ed3bf56dceebf497f7a

代码 (c)

```
#include "beaconNavigator.h"

#include <math.h>
#include <string.h>
```

```

#include "commonDef.h"
#include "configBeaconPlanning.h"
#include "schmitt.h"
#include "trackingDifferentiator.h"

typedef struct
{
    beaconNavigatorSnapshotStruct snapshot;
    beaconNavigatorParamViewStruct params;
    SchmittTrigger deadzone;
    trackingDifferentiatorStruct tdVx;
    trackingDifferentiatorStruct tdVy;
    trackingDifferentiatorStruct tdWz;
    float slewVx;
    float slewVy;
    float slewWz;
    uint8_t tdVxEnabled;
    uint8_t tdVyEnabled;
    uint8_t tdWzEnabled;
} beaconNavigatorContextStruct;

static beaconNavigatorContextStruct navigatorContext;
#define nav navigatorContext.snapshot
#define params navigatorContext.params

static uint8_t finiteFloat(float value)
{
    return (uint8_t) ((value == value) && (value > -1.0e30f) &&
        (value < 1.0e30f));
}

static float positiveLimit(float value)
{
    return (finiteFloat(value) && value > 0.0f) ? value : 0.0f;
}

static float slewToward(float current, float target, float step)
{
    float delta = target - current;
    if (delta > step) delta = step;
    if (delta < -step) delta = -step;
    return current + delta;
}

static float applyMinimum(float value, float minimum, float maximum)
{
    if (!finiteFloat(value) || !finiteFloat(minimum) || minimum < 0.0f ||
        maximum <= 0.0f) return 0.0f;
    value = clampf(value, -maximum, maximum);
    if (value == 0.0f || minimum == 0.0f) return value;
    minimum = (minimum > maximum) ? maximum : minimum;
    return (value < 0.0f) ? ((value > -minimum) ? -minimum : value) :
        ((value < minimum) ? minimum : value);
}

static void applyPlanarMinimum(float *vx, float *vy, float vxMinimum,
    float vyMinimum, float vxMaximum, float vyMaximum)
{
    float normalizedX;
    float normalizedY;
    float normalizedMagnitude;
    if ((*vx == 0) || (vy == 0) || !finiteFloat(*vx) || !finiteFloat(*vy) ||
        !finiteFloat(vxMinimum) || !finiteFloat(vyMinimum) ||
        vxMinimum < 0.0f || vyMinimum < 0.0f)
    {
        if (vx != 0) *vx = 0.0f;
        if (vy != 0) *vy = 0.0f;
        return;
    }
    *vx = clampf(*vx, -vxMaximum, vxMaximum);
    *vy = clampf(*vy, -vyMaximum, vyMaximum);
    vxMinimum = (vxMinimum > vxMaximum) ? vxMaximum : vxMinimum;
    vyMinimum = (vyMinimum > vyMaximum) ? vyMaximum : vyMinimum;
    if ((*vx == 0.0f) && (*vy == 0.0f)) return;
    if ((vxMinimum == 0.0f) || (vyMinimum == 0.0f))
    {
        *vx = applyMinimum(*vx, vxMinimum, vxMaximum);
        *vy = applyMinimum(*vy, vyMinimum, vyMaximum);
        return;
    }

```

```

    }
    if ((fabsf(*vx) >= vxMinimum) || (fabsf(*vy) >= vyMinimum)) return;
    normalizedX = *vx / vxMinimum;
    normalizedY = *vy / vyMinimum;
    normalizedMagnitude = hypotf(normalizedX, normalizedY);
    if (normalizedMagnitude >= 1.0f) return;
    if (!finiteFloat(normalizedMagnitude) || normalizedMagnitude <= 0.0f)
    {
        *vx = *vy = 0.0f;
        return;
    }
    *vx = clampf(*vx / normalizedMagnitude, -vxMaximum, vxMaximum);
    *vy = clampf(*vy / normalizedMagnitude, -vyMaximum, vyMaximum);
}

static void hardZero(void)
{
    nav.cmdVx = nav.cmdVy = nav.cmdWz = 0.0f;
    nav.vx = nav.vy = nav.wz = 0.0f;
    navigatorContext.slewVx = navigatorContext.slewVy = navigatorContext.slewWz = 0.0f;
    trackingDifferentiatorReset(&navigatorContext.tdVx, 0.0f);
    trackingDifferentiatorReset(&navigatorContext.tdVy, 0.0f);
    trackingDifferentiatorReset(&navigatorContext.tdWz, 0.0f);
}

static void clearCandidate(void)
{
    nav.targetValid = 0u;
    nav.targetFresh = 0u;
    nav.targetHolding = 0u;
    nav.sourceId = BEACON_SOURCE_NONE;
    nav.controlX = nav.controlY = nav.targetSize = nav.targetTrackId = 0u;
    nav.errX = nav.errY = nav.errFwd = nav.errRight = 0.0f;
}

static void copyPose(const core0PoseSupervisorSnapshot *pose)
{
    nav.poseValid = 0u;
    nav.poseHolding = 0u;
    nav.poseHeadingValid = 0u;
    nav.poseHeadingHolding = 0u;
    nav.poseLinkAlive = 0u;
    nav.poseFlags = 0u;
    nav.poseEpoch = 0u;
    nav.poseConfidenceQ8 = 0u;
    nav.poseHoldMs = 0u;
    nav.poseHeadingHoldMs = 0u;
    nav.poseRawX = 0.0f;
    nav.poseRawY = 0.0f;
    nav.poseHeadingRad = 0.0f;
    if (pose == 0) return;
    nav.poseValid = pose->positionValid;
    nav.poseHolding = pose->positionHolding;
    nav.poseHeadingValid = pose->headingValid;
    nav.poseHeadingHolding = pose->headingHolding;
    nav.poseLinkAlive = pose->linkAlive;
    nav.poseFlags = pose->flags;
    nav.poseEpoch = pose->poseEpoch;
    nav.poseConfidenceQ8 = pose->confidenceQ8;
    nav.poseHoldMs = pose->positionAgeMs;
    nav.poseHeadingHoldMs = pose->headingAgeMs;
    nav.poseRawX = pose->x;
    nav.poseRawY = pose->y;
    nav.poseHeadingRad = pose->headingRad;
}

static void syncTd(uint8_t desired, uint8_t *active, float *slew,
    float output, trackingDifferentiatorStruct *td)
{
    desired = (desired != 0u) ? 1u : 0u;
    if (*active == desired) return;
    *slew = output;
    trackingDifferentiatorReset(td, output);
    *active = desired;
}

static void applyOutput(float vx, float vy, float wz, float dtS,
    float intentScale, uint8_t applyMinimumOutput)
{

```

```

float vxMax = positiveLimit(params.planningVxMax);
float vyMax = positiveLimit(params.planningVyMax);
float wzMax = positiveLimit(params.planningWzMax);
if (!finiteFloat(intentScale))
{
    hardZero();
    return;
}
intentScale = clampf(intentScale, 0.0f, 1.0f);
syncTd(params.planningTdVxEnable, &navigatorContext.tdVxEnabled,
    &navigatorContext.slewVx, nav.vx, &navigatorContext.tdVx);
syncTd(params.planningTdVyEnable, &navigatorContext.tdVyEnabled,
    &navigatorContext.slewVy, nav.vy, &navigatorContext.tdVy);
syncTd(params.planningTdWzEnable, &navigatorContext.tdWzEnabled,
    &navigatorContext.slewWz, nav.wz, &navigatorContext.tdWz);
navigatorContext.slewVx = slewToward(navigatorContext.slewVx,
    clampf(vx * intentScale, -vxMax, vxMax), params.planningVxSlewPerS * dtS);
navigatorContext.slewVy = slewToward(navigatorContext.slewVy,
    clampf(vy * intentScale, -vyMax, vyMax), params.planningVySlewPerS * dtS);
navigatorContext.slewWz = slewToward(navigatorContext.slewWz,
    clampf(wz * intentScale, -wzMax, wzMax), params.planningWzSlewPerS * dtS);
nav.cmdVx = clampf(navigatorContext.slewVx, -vxMax, vxMax);
nav.cmdVy = clampf(navigatorContext.slewVy, -vyMax, vyMax);
nav.cmdWz = clampf(navigatorContext.slewWz, -wzMax, wzMax);
trackingDifferentiatorSetParams(&navigatorContext.tdVx, params.planningTdRvx, dtS, params.planningTdN);
trackingDifferentiatorSetParams(&navigatorContext.tdVy, params.planningTdRvy, dtS, params.planningTdN);
trackingDifferentiatorSetParams(&navigatorContext.tdWz, params.planningTdRwz, dtS, params.planningTdN);
nav.vx = navigatorContext.tdVxEnabled ? trackingDifferentiatorUpdate(&navigatorContext.tdVx, nav.cmdVx) : nav.cmdVx;
nav.vy = navigatorContext.tdVyEnabled ? trackingDifferentiatorUpdate(&navigatorContext.tdVy, nav.cmdVy) : nav.cmdVy;
nav.wz = navigatorContext.tdWzEnabled ? trackingDifferentiatorUpdate(&navigatorContext.tdWz, nav.cmdWz) : nav.cmdWz;
if (!finiteFloat(nav.vx) || !finiteFloat(nav.vy) || !finiteFloat(nav.wz))
{
    hardZero();
    return;
}
nav.vx = clampf(nav.vx, -vxMax, vxMax);
nav.vy = clampf(nav.vy, -vyMax, vyMax);
nav.wz = clampf(nav.wz, -wzMax, wzMax);
if (applyMinimumOutput != 0u)
{
    applyPlanarMinimum(&nav.vx, &nav.vy, params.planningVxMin,
        params.planningVyMin, vxMax, vyMax);
    nav.wz = applyMinimum(nav.wz, params.planningWzMin, wzMax);
}
}

static void applyCandidate(const beaconControlCandidate *candidate, float dtS)
{
    float r2;
    float ex;
    float ey;
    int8_t vxSign;
    int8_t vySign;
    float kpVx;
    float kpVy;
    if ((candidate == 0) || (candidate->valid == 0u) ||
        !finiteFloat(candidate->errFwd) || !finiteFloat(candidate->errRight) ||
        !finiteFloat(candidate->centerX) || !finiteFloat(candidate->centerY))
    {
        nav.state = (uint8_t) BEACON_NAV_IDLE;
        nav.valid = 0u;
        clearCandidate();
        hardZero();
        return;
    }
    nav.state = (uint8_t) BEACON_NAV_TRACK;
    nav.valid = 1u;
    nav.sourceId = candidate->sourceId;
    nav.targetValid = 1u;
    nav.targetFresh = candidate->fresh;
    nav.targetHolding = candidate->holding;
    nav.targetTrackId = candidate->trackId;
    nav.targetSize = candidate->size;
    nav.controlX = candidate->fresh ? candidate->centerX : candidate->predictedX;
    nav.controlY = candidate->fresh ? candidate->centerY : candidate->predictedY;
    nav.activeCenterX = candidate->sourceCenterX;
    nav.activeCenterY = candidate->sourceCenterY;
    nav.errFwd = nav.errX = candidate->errFwd;
    nav.errRight = nav.errY = candidate->errRight;
}

```

```

r2 = nav.errFwd * nav.errFwd + nav.errRight * nav.errRight;
schmittSetThresholds(&navigatorContext.deadzone,
    params.planningDeadzoneEnterPx * params.planningDeadzoneEnterPx,
    params.planningDeadzoneExitPx * params.planningDeadzoneExitPx);
schmittUpdate(&navigatorContext.deadzone, r2);
ex = navigatorContext.deadzone.outputState ? nav.errFwd : 0.0f;
ey = navigatorContext.deadzone.outputState ? nav.errRight : 0.0f;
vxSign = params.planningVxSign;
vySign = params.planningVySign;
kpVx = params.planningKpVx;
kpVy = params.planningKpVy;
if (candidate->sourceId == BEACON_SOURCE_CORE3)
{
    vxSign = params.core3PlanningVxSign;
    vySign = params.core3PlanningVySign;
    kpVx = params.core3PlanningKpVx;
    kpVy = params.core3PlanningKpVy;
}
else if (candidate->sourceId == BEACON_SOURCE_CORE4)
{
    vxSign = params.core4PlanningVxSign;
    vySign = params.core4PlanningVySign;
    kpVx = params.core4PlanningKpVx;
    kpVy = params.core4PlanningKpVy;
}
applyOutput((float) vxSign * kpVx * ex, (float) vySign * kpVy * ey,
    0.0f, dtS, candidate->intentScale, 1u);
}

static void applySearch(float dtS, int8_t direction)
{
    float wz;
    if ((params.planningSearchEnable == 0u) ||
        (positiveLimit(params.planningSearchWz) <= 0.0f) ||
        (direction != -1 && direction != 1))
    {
        nav.state = (uint8_t) BEACON_NAV_IDLE;
        nav.valid = 0u;
        hardZero();
        return;
    }
    clearCandidate();
    nav.state = (uint8_t) BEACON_NAV_SEARCH;
    nav.valid = 1u;
    wz = (float) params.planningWzSign * (float) direction * params.planningSearchWz;
    applyOutput(0.0f, 0.0f, wz, dtS, 1.0f, 1u);
    /* SEARCH 只允许 yaw 使用最终 Min; 零线速度必须保持精确零。 */
    nav.vx = nav.vy = 0.0f;
}

static void applyStartup(float dtS)
{
    clearCandidate();
    nav.state = (uint8_t) BEACON_NAV_STARTUP_FORWARD;
    if (params.planningFirstBeaconForwardEnable == 0u)
    {
        nav.valid = 0u;
        hardZero();
        return;
    }
    nav.valid = 1u;
    applyOutput(params.planningFirstBeaconForwardSpeed, 0.0f, 0.0f, dtS, 1.0f,
        1u);
}

void beaconNavigatorApplyParams(const beaconNavigatorParamViewStruct *view)
{
    if (view != 0) params = *view;
}

void beaconNavigatorInit(void)
{
    beaconNavigatorParamViewStruct retainedParams = params;
    memset(&navigatorContext, 0, sizeof(navigatorContext));
    params = retainedParams;
    nav.state = (uint8_t) BEACON_NAV_IDLE;
    trackingDifferentiatorInit(&navigatorContext.tdVx, params.planningTdRvx,
        PLANNING_DT_S, params.planningTdN, 0.0f);
    trackingDifferentiatorInit(&navigatorContext.tdVy, params.planningTdRvy,

```



```

    PLANNING_DT_S, params.planningTdN, 0.0f);
trackingDifferentiatorInit(&navigatorContext.tdWz, params.planningTdRwz,
    PLANNING_DT_S, params.planningTdN, 0.0f);
schmittInit(&navigatorContext.deadzone, 0.0f, 0.0f, 0u);
}

void beaconNavigatorResetForRejoin(void)
{
    memset(&nav, 0, sizeof(nav));
    nav.state = (uint8_t) BEACON_NAV_IDLE;
    hardZero();
    schmittReset(&navigatorContext.deadzone, 0u);
}

void beaconNavigatorGetSnapshot(beaconNavigatorSnapshotStruct *out)
{
    if (out != 0) *out = nav;
}

static void fillPlan(planToLinkStruct *out)
{
    memset(out, 0, sizeof(*out));
    out->magic = CORE1_PLAN_READY_MAGIC;
    out->state = nav.state;
    out->valid = nav.valid;
    out->beaconCount = nav.beaconCount;
    out->targetX = nav.controlX;
    out->targetY = nav.controlY;
    out->targetSize = nav.targetSize;
    out->errX = nav.errX;
    out->errY = nav.errY;
    out->vx = nav.vx;
    out->vy = nav.vy;
    out->wz = (nav.state == (uint8_t) BEACON_NAV_SEARCH) ? nav.wz : 0.0f;
}

void beaconNavigatorStep(const beaconNavigatorCommandStruct *command,
    beaconNavigatorOutputStruct *output)
{
    if ((command == 0) || (output == 0)) return;
    copyPose(command->pose);
    nav.beaconCount = command->qualifiedCount;
    if ((command->kind != BEACON_NAV_COMMAND_BARRIER) &&
        ((command->pose == 0) || (command->pose->motionPermitted == 0u)))
    {
        nav.state = (uint8_t) BEACON_NAV_IDLE;
        nav.valid = 0u;
        clearCandidate();
        hardZero();
        output->snapshot = nav;
        fillPlan(&output->plan);
        return;
    }
    switch (command->kind)
    {
    case BEACON_NAV_COMMAND_BARRIER:
        nav.state = (uint8_t) BEACON_NAV_IDLE;
        nav.valid = 0u;
        clearCandidate();
        hardZero();
        break;
    case BEACON_NAV_COMMAND_STARTUP_FORWARD:
        applyStartup(command->dtS);
        break;
    case BEACON_NAV_COMMAND_CANDIDATE:
        if ((command->candidate != 0) && (command->candidate->valid != 0u))
        {
            applyCandidate(command->candidate, command->dtS);
        }
        else if (command->searchActive != 0u)
        {
            applySearch(command->dtS, command->searchDirection);
        }
        else
        {
            nav.state = (uint8_t) BEACON_NAV_IDLE;
            nav.valid = 0u;
            clearCandidate();
            hardZero();
        }
    }
}

```

```

    }
    break;
case BEACON_NAV_COMMAND_INVALID:
default:
    nav.state = (uint8_t) BEACON_NAV_IDLE;
    nav.valid = 0u;
    clearCandidate();
    hardZero();
    break;
}
output->snapshot = nav;
fillPlan(&output->plan);
}

```

core1_cyt4bb7/project/code/planning/beaconNavigator.h

SHA-256

:

1c65317a3969a8a705ce116297c183552edd754523a56b21538a207d952ed781

代码 (c)

```

#ifndef _BEACON_NAVIGATOR_H_
#define _BEACON_NAVIGATOR_H_

#include <stdint.h>

#include "beaconSourceId.h"
#include "beaconTargetContinuity.h"
#include "core0PoseSupervisor.h"
#include "coreDataTransfer.h"

/* navigator 只负责把已资格化的连续性候选整形为一份 plan，不再持有
 * track selector、来源仲裁或旧双源搜索状态。 */
typedef enum
{
    BEACON_NAV_IDLE = CORE1_PLAN_STATE_IDLE,
    BEACON_NAV_TRACK = CORE1_PLAN_STATE_TRACK,
    BEACON_NAV_STARTUP_FORWARD = CORE1_PLAN_STATE_STARTUP_FORWARD,
    BEACON_NAV_SEARCH = CORE1_PLAN_STATE_SEARCH
} beaconNavigatorStateEnum;

typedef struct
{
    float planningKpVx;
    float planningKpVy;
    float core3PlanningKpVx;
    float core3PlanningKpVy;
    float core4PlanningKpVx;
    float core4PlanningKpVy;
    float planningVxMax;
    float planningVyMax;
    float planningWzMax;
    float planningVxMin;
    float planningVyMin;
    float planningWzMin;
    float planningVxSlewPerS;
    float planningVySlewPerS;
    float planningWzSlewPerS;
    float planningDeadzoneEnterPx;
    float planningDeadzoneExitPx;
    float planningTdRvx;
    float planningTdRvy;
    float planningTdRwz;
    float planningTdN;
    float planningSearchWz;
    float planningFirstBeaconForwardSpeed;
    int8_t planningVxSign;
    int8_t planningVySign;
    int8_t planningWzSign;
    int8_t core3PlanningVxSign;
    int8_t core3PlanningVySign;
    int8_t core4PlanningVxSign;
    int8_t core4PlanningVySign;
    uint8_t planningSearchEnable;
    uint8_t planningTdVxEnable;

```

```

    uint8_t planningTdVyEnable;
    uint8_t planningTdWzEnable;
    uint8_t planningFirstBeaconForwardEnable;
} beaconNavigatorParamViewStruct;

typedef struct
{
    uint8_t state;
    uint8_t valid;
    uint8_t beaconCount;
    uint8_t sourceId;
    uint8_t targetValid;
    uint8_t targetFresh;
    uint8_t targetHolding;
    uint8_t padding0;
    uint16_t controlX;
    uint16_t controlY;
    uint16_t targetSize;
    uint16_t targetTrackId;
    float errX;
    float errY;
    float vx;
    float vy;
    float wz;
    float cmdVx;
    float cmdVy;
    float cmdWz;
    float activeCenterX;
    float activeCenterY;
    float poseHeadingRad;
    uint8_t poseValid;
    uint8_t poseHolding;
    uint8_t poseHeadingValid;
    uint8_t poseHeadingHolding;
    uint8_t poseLinkAlive;
    uint8_t poseFlags;
    uint8_t poseEpoch;
    uint8_t poseConfidenceQ8;
    uint16_t poseHoldMs;
    uint16_t poseHeadingHoldMs;
    float poseRawX;
    float poseRawY;
    float errFwd;
    float errRight;
} beaconNavigatorSnapshotStruct;

typedef enum
{
    BEACON_NAV_COMMAND_BARRIER = 0u,
    BEACON_NAV_COMMAND_STARTUP_FORWARD,
    BEACON_NAV_COMMAND_CANDIDATE,
    BEACON_NAV_COMMAND_INVALID
} beaconNavigatorCommandKindEnum;

typedef struct
{
    float dtS;
    uint8_t kind;
    uint8_t qualifiedCount;
    uint8_t searchActive;
    int8_t searchDirection;
    const core0PoseSupervisorSnapshot *pose;
    const beaconControlCandidate *candidate;
} beaconNavigatorCommandStruct;

typedef struct
{
    beaconNavigatorSnapshotStruct snapshot;
    planToLinkStruct plan;
} beaconNavigatorOutputStruct;

void beaconNavigatorInit(void);
void beaconNavigatorResetForRejoin(void);
void beaconNavigatorApplyParams(const beaconNavigatorParamViewStruct *params);
void beaconNavigatorStep(const beaconNavigatorCommandStruct *command,
    beaconNavigatorOutputStruct *output);
void beaconNavigatorGetSnapshot(beaconNavigatorSnapshotStruct *out);

#endif

```

core1_cyt4bb7/project/code/planning/beaconSourceId.h

SHA-256

:

62db28cb8a8b7f103ef78502b00cbecc857caa2be4453d026aea488136be40ea

代码 (c)

```
#ifndef BEACON_SOURCE_ID_H
#define BEACON_SOURCE_ID_H

#include <stdint.h>

typedef enum
{
    BEACON_SOURCE_NONE = 0u,
    BEACON_SOURCE_CORE1 = 1u,
    BEACON_SOURCE_CORE3 = 3u,
    BEACON_SOURCE_CORE4 = 4u
} beaconSourceIdEnum;

#endif /* BEACON_SOURCE_ID_H */
```

core1_cyt4bb7/project/code/planning/beaconTargetContinuity.c

SHA-256

:

8120d131c23d694cd5f38c7422ec35a15a900990b65a90ea08c95d1dd0f72037

代码 (c)

```
#include "beaconTargetContinuity.h"

#include <math.h>
#include <string.h>

#include "observerLinkProtocol.h"

#define DYNAMIC_CENTER_X_MIN (-188.0f)
#define DYNAMIC_CENTER_X_MAX (375.0f)
#define DYNAMIC_CENTER_Y_MIN (-120.0f)
#define DYNAMIC_CENTER_Y_MAX (239.0f)
#define PI_F (3.1415927f)

static uint8_t finiteFloat(float value)
{
    return (uint8_t) ((value == value) && (value > -3.4028234e38f) &&
        (value < 3.4028234e38f));
}

static uint8_t profileValid(const beaconSourceProfile *profile)
{
    uint8_t commonValid;
    uint8_t headingValid;
    if (profile == 0) return 0u;
    commonValid = (uint8_t) (finiteFloat(profile->centerX) &&
        finiteFloat(profile->centerY) && finiteFloat(profile->scale) &&
        (profile->centerX >= DYNAMIC_CENTER_X_MIN) &&
        (profile->centerX <= DYNAMIC_CENTER_X_MAX) &&
        (profile->centerY >= DYNAMIC_CENTER_Y_MIN) &&
        (profile->centerY <= DYNAMIC_CENTER_Y_MAX) &&
        (profile->scale > 0.0f) && (profile->scale <= 10.0f));
    if (commonValid == 0u) return 0u;
    if (profile->geometryMode == BEACON_SOURCE_GEOMETRY_DOWNWARD_HEADING)
    {
        return (uint8_t) (finiteFloat(profile->bodyHeadingRad) &&
            (profile->bodyHeadingRad >= -PI_F) &&
            (profile->bodyHeadingRad <= PI_F));
    }
    if ((profile->geometryMode != BEACON_SOURCE_GEOMETRY_OBSERVER_FIXED) &&
        (profile->geometryMode != BEACON_SOURCE_GEOMETRY_OBSERVER_HEADING)) return 0u;
    headingValid = (uint8_t) (finiteFloat(profile->bodyHeadingRad) &&
```

```

        (profile->bodyHeadingRad >= -PI_F) &&
        (profile->bodyHeadingRad <= PI_F));
return (uint8_t) (finiteFloat(profile->mountYawRad) &&
        (profile->mountYawRad >= -PI_F) && (profile->mountYawRad <= PI_F) &&
        (profile->mirrorX <= 1u) &&
        ((profile->errorFwdSign == 1) || (profile->errorFwdSign == -1)) &&
        ((profile->errorRightSign == 1) || (profile->errorRightSign == -1)) &&
        ((profile->geometryMode == BEACON_SOURCE_GEOMETRY_OBSERVER_FIXED) ||
        (headingValid != 0u)));
}

static uint8_t convertPoint(const beaconSourceProfile *profile,
        uint16_t x, uint16_t y, float *errFwd, float *errRight)
{
    float u;
    float v;
    float c;
    float s;
    float fixedFwd;
    float fixedRight;
    if ((profileValid(profile) == 0u) || (errFwd == 0) || (errRight == 0))
        return 0u;
    u = ((float) x - profile->centerX) * profile->scale;
    v = ((float) y - profile->centerY) * profile->scale;
    if (profile->geometryMode == BEACON_SOURCE_GEOMETRY_DOWNWARD_HEADING)
    {
        c = cosf(profile->bodyHeadingRad);
        s = sinf(profile->bodyHeadingRad);
        *errFwd = c * u + s * v;
        *errRight = -s * u + c * v;
    }
    else
    {
        if (profile->mirrorX != 0u) u = -u;
        c = cosf(profile->mountYawRad);
        s = sinf(profile->mountYawRad);
        fixedRight = (float) profile->errorRightSign * (c * u - s * v);
        fixedFwd = (float) profile->errorFwdSign * (s * u + c * v);
        if (profile->geometryMode == BEACON_SOURCE_GEOMETRY_OBSERVER_HEADING)
        {
            c = cosf(profile->bodyHeadingRad);
            s = sinf(profile->bodyHeadingRad);
            *errFwd = c * fixedFwd + s * fixedRight;
            *errRight = -s * fixedFwd + c * fixedRight;
        }
        else
        {
            *errFwd = fixedFwd;
            *errRight = fixedRight;
        }
    }
    return (uint8_t) (finiteFloat(*errFwd) && finiteFloat(*errRight));
}

uint8_t beaconTargetContinuityBodyToObserver(const beaconSourceProfile *profile,
        float errFwd, float errRight, float *u, float *v)
{
    float c;
    float s;
    float fixedFwd = errFwd;
    float fixedRight = errRight;
    if ((profileValid(profile) == 0u) || (u == 0) || (v == 0) ||
        (finiteFloat(errFwd) == 0u) || (finiteFloat(errRight) == 0u)) return 0u;
    if (profile->geometryMode == BEACON_SOURCE_GEOMETRY_DOWNWARD_HEADING)
    {
        c = cosf(profile->bodyHeadingRad);
        s = sinf(profile->bodyHeadingRad);
        *u = c * errFwd - s * errRight;
        *v = s * errFwd + c * errRight;
    }
    else
    {
        if (profile->geometryMode == BEACON_SOURCE_GEOMETRY_OBSERVER_HEADING)
        {
            c = cosf(profile->bodyHeadingRad);
            s = sinf(profile->bodyHeadingRad);
            fixedFwd = c * errFwd - s * errRight;
            fixedRight = s * errFwd + c * errRight;
        }
    }
}

```

```

        fixedFwd /= (float) profile->errorFwdSign;
        fixedRight /= (float) profile->errorRightSign;
        c = cosf(profile->mountYawRad);
        s = sinf(profile->mountYawRad);
        *u = c * fixedRight + s * fixedFwd;
        *v = -s * fixedRight + c * fixedFwd;
        if (profile->mirrorX != 0u) *u = -*u;
    }
    return (uint8_t) (finiteFloat(*u) && finiteFloat(*v));
}

static uint8_t convertCandidate(const beaconSourceObservationView *view,
    const beaconConfirmedStruct *track,
    beaconControlCandidate *candidate)
{
    if ((view == 0) || (track == 0) || (candidate == 0) ||
        (profileValid(&view->profile) == 0u) || (track->fresh == 0u) ||
        (track->trackId == 0u) ||
        ((track->flags & BEACON_CONFIRMED_FLAG_BORDER_REJECTED) != 0u)) return 0u;
    memset(candidate, 0, sizeof(*candidate));
    candidate->valid = 1u;
    candidate->fresh = 1u;
    candidate->sourceId = view->sourceId;
    candidate->trackId = track->trackId;
    candidate->size = track->size;
    candidate->centerX = track->centerX;
    candidate->centerY = track->centerY;
    candidate->predictedX = track->predX;
    candidate->predictedY = track->predY;
    candidate->session = view->session;
    candidate->observationSeq = view->observationSeq;
    candidate->intentScale = 1.0f;
    candidate->sourceCenterX = view->profile.centerX;
    candidate->sourceCenterY = view->profile.centerY;
    if ((convertPoint(&view->profile, track->centerX, track->centerY,
        &candidate->errFwd, &candidate->errRight) == 0u) ||
        (convertPoint(&view->profile, track->predX, track->predY,
            &candidate->predictedFwd, &candidate->predictedRight) == 0u))
        return 0u;
    candidate->velocityRight = candidate->predictedRight - candidate->errRight;
    candidate->velocityFwd = candidate->predictedFwd - candidate->errFwd;
    return (uint8_t) (finiteFloat(candidate->errRight) &&
        finiteFloat(candidate->errFwd) && finiteFloat(candidate->predictedRight) &&
        finiteFloat(candidate->predictedFwd));
}

static const beaconSourceObservationView *findCurrentView(
    const beaconSourceObservationView *views, uint8_t viewCount,
    const beaconControlCandidate *last)
{
    uint8_t i;
    if ((views == 0) || (last == 0)) return 0;
    for (i = 0u; i < viewCount; i++)
    {
        if ((views[i].sourceId == last->sourceId) &&
            (views[i].session == last->session) &&
            (profileValid(&views[i].profile) != 0u)) return &views[i];
    }
    return 0;
}

static uint8_t buildEdgeHoldCandidate(
    const beaconSourceObservationView *views, uint8_t viewCount,
    const beaconControlCandidate *last, float intentScale,
    beaconControlCandidate *candidate)
{
    const beaconSourceObservationView *view = findCurrentView(views, viewCount, last);
    if ((view == 0) || (candidate == 0)) return 0u;
    *candidate = *last;
    candidate->fresh = 0u;
    candidate->holding = 1u;
    candidate->intentScale = intentScale;
    candidate->observationSeq = view->observationSeq;
    if (convertPoint(&view->profile, last->predictedX, last->predictedY,
        &candidate->errFwd, &candidate->errRight) == 0u) return 0u;
    candidate->predictedFwd = candidate->errFwd;
    candidate->predictedRight = candidate->errRight;
    candidate->velocityFwd = 0.0f;
    candidate->velocityRight = 0.0f;
}

```

```

        return 1u;
    }

static uint8_t buildScoringReference(
    const beaconSourceObservationView *views, uint8_t viewCount,
    const beaconTargetContinuity *continuity, float *refFwd, float *refRight)
{
    const beaconSourceObservationView *view;
    uint16_t x;
    uint16_t y;
    if ((continuity == 0) || (continuity->last.valid == 0u) ||
        (refFwd == 0) || (refRight == 0)) return 0u;
    view = findCurrentView(views, viewCount, &continuity->last);
    if (view == 0) return 0u;
    x = continuity->trendValid ? continuity->last.predictedX :
        continuity->last.centerX;
    y = continuity->trendValid ? continuity->last.predictedY :
        continuity->last.centerY;
    return convertPoint(&view->profile, x, y, refFwd, refRight);
}

static uint32_t updateClock(beaconTargetContinuity *continuity,
    uint32_t tickMs)
{
    uint32_t elapsedMs;
    if (continuity->clockPrimed == 0u)
    {
        continuity->clockPrimed = 1u;
        continuity->clockTickMs = tickMs;
        return 0u;
    }
    elapsedMs = (uint32_t) (tickMs - continuity->clockTickMs);
    continuity->clockTickMs = tickMs;
    return elapsedMs;
}

static void pauseTargetTimer(beaconTargetContinuity *continuity,
    uint32_t elapsedMs)
{
    if (continuity->last.valid != 0u)
        continuity->lastTickMs += elapsedMs;
}

static void pauseSearchTimer(beaconTargetContinuity *continuity,
    uint32_t elapsedMs)
{
    if (continuity->searchRunning != 0u)
        continuity->searchStartTickMs += elapsedMs;
}

static void advanceSearch(beaconTargetContinuity *continuity,
    uint32_t tickMs, uint16_t searchTimeoutMs, uint8_t searchEnabled,
    uint8_t allowStart)
{
    if (searchEnabled == 0u)
    {
        continuity->searchRunning = 0u;
        continuity->searchTimedOut = 0u;
    }
    else if (continuity->searchTimedOut == 0u)
    {
        if ((continuity->searchRunning == 0u) && (allowStart != 0u))
        {
            continuity->searchRunning = 1u;
            continuity->searchStartTickMs = tickMs;
        }
        if ((continuity->searchRunning != 0u) && (searchTimeoutMs != 0u) &&
            ((uint32_t) (tickMs - continuity->searchStartTickMs) >=
                searchTimeoutMs))
        {
            continuity->searchRunning = 0u;
            continuity->searchTimedOut = 1u;
        }
    }
}

static float edgeHoldIntentScale(uint32_t elapsedMs, uint16_t edgeHoldMs)
{
    if ((edgeHoldMs == 0u) || (elapsedMs >= edgeHoldMs)) return 0.0f;

```

```

        return 1.0f - (float) elapsedMs / (float) edgeHoldMs;
    }

static uint8_t sequenceAdvanced(uint32_t previous, uint32_t current)
{
    uint32_t delta = (uint32_t) (current - previous);
    return (uint8_t) ((delta != 0u) && (delta < 0x80 000 000u));
}

static uint8_t consumeBorderRejection(beaconTargetContinuity *continuity,
    const beaconSourceObservationView *views, uint8_t viewCount)
{
    uint8_t i;
    if ((continuity->last.valid == 0u) || (views == 0)) return 0u;
    for (i = 0u; i < viewCount; i++)
    {
        uint16_t j;
        if ((views[i].tracks == 0) ||
            (views[i].sourceId != continuity->last.sourceId) ||
            (views[i].session != continuity->last.session) ||
            (sequenceAdvanced(continuity->last.observationSeq,
                views[i].observationSeq) == 0u)) continue;
        for (j = 0u; j < views[i].count; j++)
        {
            const beaconConfirmedStruct *track = &views[i].tracks[j];
            if ((track->trackId == continuity->last.trackId) &&
                (track->flags == BEACON_CONFIRMED_FLAG_BORDER_REJECTED))
            {
                memset(&continuity->last, 0, sizeof(continuity->last));
                continuity->trendValid = 0u;
                continuity->edgeHoldActive = 0u;
                return 1u;
            }
        }
    }
    return 0u;
}

static uint8_t sourceRank(uint8_t source, uint8_t current)
{
    if ((current != OBSERVER_LINK_SOURCE_NONE) && (source == current)) return 0u;
    if (source == OBSERVER_LINK_SOURCE_CORE1) return 1u;
    if (source == OBSERVER_LINK_SOURCE_CORE3) return 2u;
    return 3u;
}

static uint8_t tieBefore(const beaconControlCandidate *left,
    const beaconControlCandidate *right,
    uint8_t current)
{
    uint8_t lr = sourceRank(left->sourceId, current);
    uint8_t rr = sourceRank(right->sourceId, current);
    if (lr != rr) return (uint8_t) (lr < rr);
    if (left->session != right->session) return (uint8_t) (left->session < right->session);
    return (uint8_t) (left->trackId < right->trackId);
}

static uint8_t selectExactCurrentCandidate(
    const beaconTargetContinuity *continuity,
    const beaconSourceObservationView *views,
    uint8_t viewCount,
    beaconControlCandidate *candidate)
{
    uint8_t i;
    if ((continuity == 0) || (continuity->last.valid == 0u) ||
        (views == 0) || (candidate == 0)) return 0u;
    for (i = 0u; i < viewCount; i++)
    {
        uint16_t j;
        if ((views[i].fresh == 0u) || (views[i].tracks == 0) ||
            (views[i].sourceId != continuity->last.sourceId) ||
            (views[i].session != continuity->last.session)) continue;
        for (j = 0u; j < views[i].count; j++)
        {
            if ((views[i].tracks[j].trackId == continuity->last.trackId) &&
                (convertCandidate(&views[i], &views[i].tracks[j], candidate) != 0u))
                return 1u;
        }
    }
}

```



```

    return 0u;
}

static uint8_t selectBestCandidate(
    const beaconTargetContinuity *continuity,
    const beaconSourceObservationView *views,
    uint8_t viewCount,
    beaconControlCandidate *best)
{
    float bestScore = 3.4028234e38f;
    float referenceFwd = 0.0f;
    float referenceRight = 0.0f;
    uint8_t haveBest = 0u;
    uint8_t referenceValid = 0u;
    uint8_t currentOwner = OBSERVER_LINK_SOURCE_NONE;
    uint8_t i;
    if (best == 0) return 0u;
    memset(best, 0, sizeof(*best));
    if ((continuity == 0) || (views == 0)) return 0u;
    if (continuity->last.valid != 0u)
    {
        referenceValid = buildScoringReference(views, viewCount, continuity,
            &referenceFwd, &referenceRight);
        if (referenceValid != 0u) currentOwner = continuity->last.sourceId;
    }
    /* 三来源统一评分。历史仍有效时，使用上一目标车体系预测位置
    * 同拍完成跨视野连续交接。 */
    for (i = 0u; i < viewCount; i++)
    {
        uint16_t j;
        if ((views[i].fresh == 0u) || (views[i].tracks == 0)) continue;
        for (j = 0u; j < views[i].count; j++)
        {
            beaconControlCandidate next;
            float df;
            float dr;
            float score;
            if (convertCandidate(&views[i], &views[i].tracks[j], &next) == 0u)
                continue;
            if (referenceValid != 0u)
            {
                df = next.errFwd - referenceFwd;
                dr = next.errRight - referenceRight;
            }
            else
            {
                df = next.errFwd;
                dr = next.errRight;
            }
            score = df * df + dr * dr;
            if ((haveBest == 0u) || (score < bestScore) ||
                ((score == bestScore) && tieBefore(&next, best, currentOwner)))
            {
                *best = next;
                bestScore = score;
                haveBest = 1u;
            }
        }
    }
    return haveBest;
}

void beaconTargetContinuityInit(beaconTargetContinuity *continuity)
{
    if (continuity != 0) memset(continuity, 0, sizeof(*continuity));
}

void beaconTargetContinuityStep(beaconTargetContinuity *continuity,
    const beaconSourceObservationView *views,
    uint8_t viewCount,
    uint32_t tickMs,
    uint16_t edgeHoldMs,
    uint16_t searchTimeoutMs,
    uint8_t searchEnabled,
    uint8_t targetAdvanceAllowed,
    uint8_t searchAdvanceAllowed,
    beaconTargetContinuityOutput *output)
{
    beaconControlCandidate best;

```

```

float referenceFwd = 0.0f;
float referenceRight = 0.0f;
uint32_t clockElapsedMs;
uint8_t haveBest = 0u;
uint8_t referenceValid = 1u;
uint8_t i;
if ((continuity == 0) || (views == 0) || (output == 0)) return;
memset(output, 0, sizeof(*output));
clockElapsedMs = updateClock(continuity, tickMs);
(void) consumeBorderRejection(continuity, views, viewCount);
for (i = 0u; i < viewCount; i++)
{
    uint16_t j;
    if ((views[i].fresh == 0u) || (views[i].tracks == 0)) continue;
    for (j = 0u; j < views[i].count; j++)
    {
        beaconControlCandidate qualified;
        if (convertCandidate(&views[i], &views[i].tracks[j],
            &qualified) == 0u) continue;
        if (output->qualifiedCount < 255u) output->qualifiedCount++;
    }
}
if (continuity->last.valid != 0u)
{
    referenceValid = buildScoringReference(views, viewCount, continuity,
        &referenceFwd, &referenceRight);
}
if (targetAdvanceAllowed == 0u)
{
    if (searchAdvanceAllowed != 0u)
    {
        uint8_t edgeHoldExpired = (uint8_t) ((continuity->last.valid == 0u) ||
            (edgeHoldMs == 0u) ||
            ((uint32_t) (tickMs - continuity->lastTickMs) > edgeHoldMs));
        if (edgeHoldExpired != 0u)
        {
            continuity->edgeHoldActive = 0u;
        }
        advanceSearch(continuity, tickMs, searchTimeoutMs, searchEnabled,
            edgeHoldExpired);
    }
    else
    {
        pauseTargetTimer(continuity, clockElapsedMs);
        pauseSearchTimer(continuity, clockElapsedMs);
    }
    output->searchActive = (uint8_t) (searchEnabled &&
        continuity->searchRunning &&
        !continuity->searchTimedOut);
    output->searchDirection = (continuity->searchDirection == 0) ? 1 :
        continuity->searchDirection;
    return;
}
if (referenceValid == 0u)
{
    continuity->last.valid = 0u;
    continuity->trendValid = 0u;
}
haveBest = selectExactCurrentCandidate(continuity, views, viewCount, &best);
if (haveBest == 0u)
    haveBest = selectBestCandidate(continuity, views, viewCount, &best);
if (haveBest != 0u)
{
    continuity->searchRunning = 0u;
    continuity->searchTimedOut = 0u;
    continuity->trendValid = continuity->last.valid;
    continuity->last = best;
    continuity->lastTickMs = tickMs;
    continuity->edgeHoldActive = 0u;
    continuity->searchDirection = (best.errRight < 0.0f) ? -1 : 1;
    output->candidate = best;
}
else if ((continuity->last.valid != 0u) &&
    ((uint32_t) (tickMs - continuity->lastTickMs) <= edgeHoldMs) &&
    (edgeHoldMs != 0u) &&
    (findCurrentView(views, viewCount, &continuity->last) != 0))
{
    uint32_t elapsedMs = (uint32_t) (tickMs - continuity->lastTickMs);
    float intentScale = edgeHoldIntentScale(elapsedMs, edgeHoldMs);

```

```

continuity->searchRunning = 0u;
continuity->edgeHoldActive = 1u;
if (buildEdgeHoldCandidate(views, viewCount, &continuity->last,
    intentScale, &output->candidate) == 0u)
{
    memset(&output->candidate, 0, sizeof(output->candidate));
    continuity->edgeHoldActive = 0u;
}
}
else
{
    continuity->edgeHoldActive = 0u;
    advanceSearch(continuity, tickMs, searchTimeoutMs, searchEnabled, 1u);
    output->searchActive = (uint8_t) (searchEnabled &&
        continuity->searchRunning && !continuity->searchTimedOut);
}
output->searchDirection = (continuity->searchDirection == 0) ? 1 :
    continuity->searchDirection;
}

```

core1_cyt4bb7/project/code/planning/beaconTargetContinuity.h

SHA-256

:

95d94af70c40b78018b65991993b8418201370c11ceef8b4a3aa3f79acc4eb79

代码 (c)

```

#ifndef BEACON_TARGET_CONTINUITY_H
#define BEACON_TARGET_CONTINUITY_H

#include <stdint.h>
#include "beaconTrack.h"

#define BEACON_CONTINUITY_MAX_SOURCES (3u)

typedef enum
{
    BEACON_SOURCE_GEOMETRY_OBSERVER_FIXED = 0u,
    BEACON_SOURCE_GEOMETRY_DOWNWARD_HEADING = 1u,
    BEACON_SOURCE_GEOMETRY_OBSERVER_HEADING = 2u
} beaconSourceGeometryMode;

typedef struct
{
    float centerX;
    float centerY;
    float scale;
    float mountYawRad;
    float bodyHeadingRad;
    uint8_t mirrorX;
    int8_t errorFwdSign;
    int8_t errorRightSign;
    uint8_t geometryMode;
} beaconSourceProfile;

typedef struct
{
    uint8_t sourceId;
    uint8_t fresh;
    uint16_t trackId;
    uint32_t session;
    uint32_t observationSeq;
    const beaconConfirmedStruct *tracks;
    uint16_t count;
    beaconSourceProfile profile;
} beaconSourceObservationView;

typedef struct
{
    uint8_t valid;
    uint8_t fresh;
    uint8_t holding;
    uint8_t sourceId;
    uint16_t trackId;
    uint16_t size;

```

```

uint16_t centerX;
uint16_t centerY;
uint16_t predictedX;
uint16_t predictedY;
uint32_t session;
uint32_t observationSeq;
float errFwd;
float errRight;
float predictedFwd;
float predictedRight;
float velocityFwd;
float velocityRight;
float intentScale;
float sourceCenterX;
float sourceCenterY;
} beaconControlCandidate;

typedef struct
{
    beaconControlCandidate candidate;
    uint8_t searchActive;
    int8_t searchDirection;
    uint8_t qualifiedCount;
} beaconTargetContinuityOutput;

typedef struct
{
    beaconControlCandidate last;
    uint32_t lastTickMs;
    uint32_t searchStartTickMs;
    uint8_t trendValid;
    uint8_t searchRunning;
    uint8_t searchTimedOut;
    int8_t searchDirection;
    uint8_t edgeHoldActive;
    uint8_t clockPrimed;
    uint32_t clockTickMs;
} beaconTargetContinuity;

void beaconTargetContinuityInit(beaconTargetContinuity *continuity);
uint8_t beaconTargetContinuityBodyToObserver(const beaconSourceProfile *profile,
    float errFwd, float errRight, float *u, float *v);
void beaconTargetContinuityStep(beaconTargetContinuity *continuity,
    const beaconSourceObservationView *views,
    uint8_t viewCount,
    uint32_t tickMs,
    uint16_t edgeHoldMs,
    uint16_t searchTimeoutMs,
    uint8_t searchEnabled,
    uint8_t targetAdvanceAllowed,
    uint8_t searchAdvanceAllowed,
    beaconTargetContinuityOutput *output);

#endif

```

core1_cyt4bb7/project/code/planning/core0PoseSupervisor.c

SHA-256

:

b72dc52839a18740f8b3c4ec0bcb2f01e55da46a44e1f140b95bb974d426be4e

代码 (c)

```

#include "core0PoseSupervisor.h"

#include <string.h>

#include "core1KvmRouter.h"
#include "coreDataTransfer.h"
#include "uartLinkProto.h"

#define POSE_ZERO_WAIT_MS (3000u)
#define POSE_MAX_X_Q8 (187u * 256u)
#define POSE_MAX_Y_Q8 (119u * 256u)

typedef struct

```

```

{
    core0PoseSupervisorSnapshot snapshot;
    uint32_t lastPositionTickMs;
    uint32_t lastHeadingTickMs;
    uint32_t lastSeq;
    uint8_t havePosition;
    uint8_t haveHeading;
    uint8_t poseSeqPrimed;
    uint8_t poseEpochPrimed;
    uint8_t trustedLaunchKnown;
    uint8_t lastTrustedLaunched;
    uint8_t episodePositionQualified;
    uint8_t episodeFencePrimed;
} poseSupervisorContext;

static poseSupervisorContext g_pose;

static float wrapPi(float value)
{
    while (value > 3.14159265358979323846f) value -= 6.28318530717958647692f;
    while (value < -3.14159265358979323846f) value += 6.28318530717958647692f;
    return value;
}

static uint16_t ageU16(uint32_t now, uint32_t then)
{
    uint32_t age = now - then;
    return (age > 0xFFFFu) ? 0xFFFFu : (uint16_t) age;
}

static uint8_t finiteFloat(float value)
{
    return (uint8_t) ((value == value) &&
        (value > -3.4028234e38f) && (value < 3.4028234e38f));
}

static uint8_t seq32Advanced(uint32_t sequence, uint32_t previous)
{
    uint32_t delta = sequence - previous;
    return (uint8_t) ((delta != 0u) && (delta < 0x80000000u));
}

static void beginPositionEpisode(void)
{
    g_pose.episodePositionQualified = 0u;
    g_pose.episodeFencePrimed = (uint8_t) (
        (g_pose.poseEpochPrimed != 0u) &&
        (g_pose.poseSeqPrimed != 0u));
}

void core0PoseSupervisorInit(void)
{
    memset(&g_pose, 0, sizeof(g_pose));
    g_pose.snapshot.stage = CORE0_POSE_STAGE_WAIT_FIRST;
}

void core0PoseSupervisorStep(uint32_t tickMs,
    uint16_t holdMs,
    float minHeadingConfidence,
    int8_t headingSign,
    float headingOffsetRad,
    core0PoseSupervisorSnapshot *snapshot)
{
    core1SystemFactsSnapshot facts;
    core0PoseToVisionStruct pose;
    uint8_t transportAlive;
    uint8_t statusUsable;
    uint8_t launched = 0u;
    uint8_t poseAdvanced = 0u;
    uint8_t positionAccepted = 0u;
    uint8_t epochChanged = 0u;
    uint8_t episodeStarted = 0u;
    core1KvmRouterGetSystemFacts(&facts);
    transportAlive = (uint8_t) ((facts.ready != 0u) &&
        (facts.linkUp != 0u));
    statusUsable = (uint8_t) (transportAlive &&
        (facts.core0StatusValid != 0u));
    g_pose.snapshot.linkAlive = transportAlive;
    if ((transportAlive != 0u) &&

```

```

(coreDataTryReadCore0Pose(&pose) != 0u) &&
(pose.magic == CORE1_C0POSE_READY_MAGIC))
{
    if ((g_pose.poseEpochPrimed != 0u) &&
        (pose.poseEpoch != g_pose.snapshot.poseEpoch))
    {
        epochChanged = 1u;
    }
    if (epochChanged != 0u)
    {
        g_pose.poseSeqPrimed = 0u;
    }
    if ((g_pose.poseEpochPrimed == 0u) || (epochChanged != 0u))
    {
        g_pose.poseEpochPrimed = 1u;
        g_pose.snapshot.poseEpoch = pose.poseEpoch;
    }
    if ((g_pose.poseSeqPrimed == 0u) ||
        (seq32Advanced(pose.seq32, g_pose.lastSeq) != 0u))
    {
        uint8_t sourceOk;
        uint8_t positionOk;
        uint8_t headingOk;
        poseAdvanced = 1u;
        g_pose.poseSeqPrimed = 1u;
        g_pose.lastSeq = pose.seq32;
        g_pose.snapshot.confidenceQ8 = pose.confidenceQ8;
        g_pose.snapshot.flags = pose.flags;
        sourceOk = (uint8_t) (
            ((pose.flags & UART_LINK_POSE_FLAG_TRACK_LOCKED) != 0u) &&
            ((pose.flags & UART_LINK_POSE_FLAG_TARGET_LOST) == 0u));
        positionOk = (uint8_t) (sourceOk &&
            ((pose.flags & UART_LINK_POSE_FLAG_POSITION_NUMERIC_VALID) != 0u) &&
            ((pose.flags & UART_LINK_POSE_FLAG_POSITION_FRESH) != 0u) &&
            (pose.xQ8 <= POSE_MAX_X_Q8) &&
            (pose.yQ8 <= POSE_MAX_Y_Q8));
        headingOk = (uint8_t) (
            ((pose.flags & UART_LINK_POSE_FLAG_HEADING_FRESH) != 0u) &&
            ((float) pose.confidenceQ8 / 255.0f >= minHeadingConfidence) &&
            (finiteFloat(pose.orientationRad) != 0u) &&
            (pose.orientationRad >= -3.1415927f) && (pose.orientationRad <= 3.1415927f));
        if (positionOk != 0u)
        {
            g_pose.snapshot.x = (float) pose.xQ8 / 256.0f;
            g_pose.snapshot.y = (float) pose.yQ8 / 256.0f;
            g_pose.lastPositionTickMs = tickMs;
            g_pose.havePosition = 1u;
            positionAccepted = 1u;
        }
        if (headingOk != 0u)
        {
            g_pose.snapshot.headingRad = wrapPi((float) headingSign *
                pose.orientationRad + headingOffsetRad);
            g_pose.lastHeadingTickMs = tickMs;
            g_pose.haveHeading = 1u;
        }
    }
}
if ((epochChanged != 0u) && (g_pose.trustedLaunchKnown != 0u) &&
    (g_pose.lastTrustedLaunched != 0u))
{
    beginPositionEpisode();
    episodeStarted = 1u;
}
if (statusUsable != 0u)
{
    launched = (uint8_t) ((facts.core0StatusFlags &
        UART_LINK_C0_FLAG_LAUNCHED) != 0u);
    if (g_pose.trustedLaunchKnown == 0u)
    {
        g_pose.trustedLaunchKnown = 1u;
        g_pose.lastTrustedLaunched = launched;
        if (launched != 0u)
        {
            beginPositionEpisode();
            episodeStarted = 1u;
        }
    }
}
else if (launched != g_pose.lastTrustedLaunched)

```

```

{
    g_pose.lastTrustedLaunched = launched;
    if (launched != 0u)
    {
        beginPositionEpisode();
        episodeStarted = 1u;
    }
    else
    {
        g_pose.episodePositionQualified = 0u;
        g_pose.episodeFencePrimed = 0u;
    }
}
if ((launched != 0u) && (episodeStarted == 0u) &&
    (poseAdvanced != 0u))
{
    if (g_pose.episodeFencePrimed == 0u)
    {
        g_pose.episodeFencePrimed = 1u;
    }
    else if (positionAccepted != 0u)
    {
        g_pose.episodePositionQualified = 1u;
    }
}
}
g_pose.snapshot.positionAgeMs = g_pose.havePosition ?
    ageU16(tickMs, g_pose.lastPositionTickMs) : 0xFFFFu;
g_pose.snapshot.headingAgeMs = g_pose.haveHeading ?
    ageU16(tickMs, g_pose.lastHeadingTickMs) : 0xFFFFu;
g_pose.snapshot.positionValid = (uint8_t) (g_pose.havePosition &&
    g_pose.snapshot.positionAgeMs <= holdMs);
g_pose.snapshot.headingValid = (uint8_t) (g_pose.haveHeading &&
    g_pose.snapshot.headingAgeMs <= holdMs);
if (g_pose.snapshot.headingValid == 0u)
{
    g_pose.haveHeading = 0u;
    g_pose.snapshot.headingRad = 0.0f;
}
g_pose.snapshot.positionHolding = (uint8_t) (g_pose.snapshot.positionValid &&
    g_pose.snapshot.positionAgeMs != 0u);
g_pose.snapshot.headingHolding = (uint8_t) (g_pose.snapshot.headingValid &&
    g_pose.snapshot.headingAgeMs != 0u);
if (statusUsable == 0u)
{
    g_pose.snapshot.stage = CORE0_POSE_STAGE_LINK_BYPASS;
    g_pose.snapshot.motionPermitted = 0u;
    if (snapshot != 0) *snapshot = g_pose.snapshot;
    return;
}
if (launched == 0u)
{
    g_pose.snapshot.stage = g_pose.snapshot.positionValid ? CORE0_POSE_STAGE_FRESH :
        CORE0_POSE_STAGE_WAIT_FIRST;
}
else if (g_pose.episodePositionQualified == 0u)
{
    g_pose.snapshot.stage = CORE0_POSE_STAGE_WAIT_FIRST;
}
else
{
    if (g_pose.snapshot.positionAgeMs == 0u) g_pose.snapshot.stage = CORE0_POSE_STAGE_FRESH;
    else if (g_pose.snapshot.positionAgeMs <= holdMs) g_pose.snapshot.stage = CORE0_POSE_STAGE_HOLD;
    else if (g_pose.snapshot.positionAgeMs < (uint32_t) holdMs + POSE_ZERO_WAIT_MS)
        g_pose.snapshot.stage = CORE0_POSE_STAGE_ZERO_WAIT;
    else g_pose.snapshot.stage = CORE0_POSE_STAGE_FAULT;
}
g_pose.snapshot.motionPermitted = (uint8_t) ((g_pose.snapshot.stage == CORE0_POSE_STAGE_FRESH) ||
    (g_pose.snapshot.stage == CORE0_POSE_STAGE_HOLD));
if (snapshot != 0) *snapshot = g_pose.snapshot;
}

```

core1_cyt4bb7/project/code/planning/core0PoseSupervisor.h

SHA-256

:

86e2be32e489ec298a12899a08af9ad1f3ffb966e15417473895a86f13a68602

代码 (c)

```
#ifndef CORE0_POSE_SUPERVISOR_H
#define CORE0_POSE_SUPERVISOR_H

#include <stdint.h>

typedef enum
{
    CORE0_POSE_STAGE_FRESH = 0u,
    CORE0_POSE_STAGE_HOLD,
    CORE0_POSE_STAGE_ZERO_WAIT,
    CORE0_POSE_STAGE_WAIT_FIRST,
    CORE0_POSE_STAGE_FAULT,
    CORE0_POSE_STAGE_LINK_BYPASS
} core0PoseStageEnum;

typedef struct
{
    uint8_t stage;
    uint8_t positionValid;
    uint8_t positionHolding;
    uint8_t headingValid;
    uint8_t headingHolding;
    uint8_t linkAlive;
    uint8_t poseEpoch;
    uint8_t confidenceQ8;
    uint8_t flags;
    uint8_t motionPermitted;
    uint16_t positionAgeMs;
    uint16_t headingAgeMs;
    float x;
    float y;
    float headingRad;
} core0PoseSupervisorSnapshot;

void core0PoseSupervisorInit(void);
void core0PoseSupervisorStep(uint32_t tickMs,
    uint16_t holdMs,
    float minHeadingConfidence,
    int8_t headingSign,
    float headingOffsetRad,
    core0PoseSupervisorSnapshot *snapshot);

#endif
```

core1_cyt4bb7/project/code/planning/takeoffMotionGate.c

SHA-256

:

9791c610151e33d1995ac62ca81df4a962091395e2c4fc7c8bc8427a347777c3

代码 (c)

```
#include "takeoffMotionGate.h"

#include <string.h>

#include "uartLinkProto.h"

typedef struct
{
    uint32_t epoch;
    uint32_t delayElapsedMs;
    uint32_t lastPrepareMs;
    uint16_t delayMs;
}
```



```

uint8_t phase;
uint8_t episodeActive;
uint8_t mode2Selected;
uint8_t runtimePermitted;
uint8_t delayClockPrimed;
} takeoffMotionGateContextStruct;

static takeoffMotionGateContextStruct gateContext;

static void fillDecision(takeoffMotionGateDecisionStruct *decision)
{
    if (decision == 0)
    {
        return;
    }

    decision->epoch = gateContext.epoch;
    decision->phase = gateContext.phase;
    decision->motionUnlocked = (uint8_t) (
        gateContext.runtimePermitted != 0u &&
        (gateContext.phase == TAKEOFF_MOTION_PHASE_PREBEACON ||
         gateContext.phase == TAKEOFF_MOTION_PHASE_NORMAL));
    decision->suppressNavigator = (uint8_t) (gateContext.mode2Selected != 0u &&
        (gateContext.runtimePermitted == 0u ||
         gateContext.phase == TAKEOFF_MOTION_PHASE_INACTIVE ||
         gateContext.phase == TAKEOFF_MOTION_PHASE_DELAY));
    decision->startupForwardActive = (uint8_t) (
        gateContext.runtimePermitted != 0u &&
        gateContext.phase == TAKEOFF_MOTION_PHASE_PREBEACON);
}

static void endEpisode(void)
{
    gateContext.delayElapsedMs = 0u;
    gateContext.lastPrepareMs = 0u;
    gateContext.phase = TAKEOFF_MOTION_PHASE_INACTIVE;
    gateContext.episodeActive = 0u;
    gateContext.runtimePermitted = 0u;
    gateContext.delayClockPrimed = 0u;
}

void takeoffMotionGateInit(void)
{
    (void) memset(&gateContext, 0, sizeof(gateContext));
    gateContext.phase = TAKEOFF_MOTION_PHASE_INACTIVE;
}

void takeoffMotionGateApplyParams(uint16_t postTakeoffDelayMs)
{
    gateContext.delayMs = postTakeoffDelayMs;
}

void takeoffMotionGatePrepare(const takeoffMotionGateInputStruct *input,
                             takeoffMotionGateDecisionStruct *decision)
{
    uint8_t explicitEpisodeExit;
    uint8_t episodeStart;
    uint8_t previousRuntimePermitted;
    uint32_t elapsedStepMs = 0u;

    if (input == 0)
    {
        endEpisode();
        gateContext.mode2Selected = 0u;
        fillDecision(decision);
        return;
    }

    if (input->core0StatusValid != 0u)
    {
        gateContext.mode2Selected = (uint8_t) (
            input->systemMode == UART_LINK_SYS_MODE_RS485_AUTO);
    }
    explicitEpisodeExit = (uint8_t) (input->core0StatusValid != 0u &&
        (gateContext.mode2Selected == 0u || input->launched == 0u));
    if (explicitEpisodeExit != 0u)
    {
        endEpisode();
        fillDecision(decision);
    }
}

```

```

    return;
}

episodeStart = (uint8_t) (gateContext.episodeActive == 0u &&
    input->core0StatusValid != 0u &&
    gateContext.mode2Selected != 0u && input->launched != 0u &&
    input->cruiseQualified != 0u);
if (episodeStart != 0u)
{
    gateContext.episodeActive = 1u;
    gateContext.delayElapsedMs = 0u;
    gateContext.lastPrepareMs = input->nowMs;
    gateContext.delayClockPrimed = 1u;
    gateContext.epoch++;
    if (gateContext.epoch == 0u)
    {
        gateContext.epoch = 1u;
    }
    gateContext.phase = TAKEOFF_MOTION_PHASE_DELAY;
}
else if (gateContext.episodeActive != 0u)
{
    if (gateContext.delayClockPrimed != 0u)
    {
        elapsedStepMs = (uint32_t) (input->nowMs - gateContext.lastPrepareMs);
    }
    gateContext.lastPrepareMs = input->nowMs;
    gateContext.delayClockPrimed = 1u;
}

previousRuntimePermitted = gateContext.runtimePermitted;
gateContext.runtimePermitted = (uint8_t) (
    gateContext.episodeActive != 0u && input->core0StatusValid != 0u &&
    gateContext.mode2Selected != 0u && input->launched != 0u &&
    input->cruiseQualified != 0u && input->runtimePermitted != 0u);

if (gateContext.phase == TAKEOFF_MOTION_PHASE_DELAY &&
    gateContext.runtimePermitted != 0u &&
    (gateContext.delayMs == 0u || previousRuntimePermitted != 0u))
{
    if ((UINT32_MAX - gateContext.delayElapsedMs) < elapsedStepMs)
    {
        gateContext.delayElapsedMs = UINT32_MAX;
    }
    else
    {
        gateContext.delayElapsedMs += elapsedStepMs;
    }
    if (gateContext.delayElapsedMs >= (uint32_t) gateContext.delayMs)
    {
        gateContext.phase = TAKEOFF_MOTION_PHASE_PREBEACON;
    }
}

fillDecision(decision);
}

void takeoffMotionGateFinalize(uint8_t freshTargetAccepted,
    takeoffMotionGateDecisionStruct *decision)
{
    if (gateContext.phase == TAKEOFF_MOTION_PHASE_PREBEACON &&
        gateContext.runtimePermitted != 0u && freshTargetAccepted != 0u)
    {
        gateContext.phase = TAKEOFF_MOTION_PHASE_NORMAL;
    }
    fillDecision(decision);
}

```

core1_cyt4bb7/project/code/planning/takeoffMotionGate.h

SHA-256

:

b5f0ce094357f68878e946b8ba295f4955dff6471fe867b186f8bf3bca5eb6c3

代码 (c)

```

#ifndef _TAKEOFF_MOTION_GATE_H_
#define _TAKEOFF_MOTION_GATE_H_

#include <stdint.h>

typedef enum
{
    TAKEOFF_MOTION_PHASE_INACTIVE = 0u,
    TAKEOFF_MOTION_PHASE_DELAY,
    TAKEOFF_MOTION_PHASE_PREBEACON,
    TAKEOFF_MOTION_PHASE_NORMAL
} takeoffMotionPhaseEnum;

typedef struct
{
    uint32_t nowMs;
    uint8_t systemMode;
    uint8_t core0StatusValid;
    uint8_t launched;
    uint8_t cruiseQualified; /* 本架次已到 CRUISE 且未进入 RAMP/正式缓降。 */
    uint8_t runtimePermitted; /* 本拍链路、相机、身份与屏障资格。 */
} takeoffMotionGateInputStruct;

typedef struct
{
    uint32_t epoch;
    uint8_t phase;
    uint8_t motionUnlocked;
    uint8_t suppressNavigator;
    uint8_t startupForwardActive;
} takeoffMotionGateDecisionStruct;

void takeoffMotionGateInit(void);
void takeoffMotionGateApplyParams(uint16_t postTakeoffDelayMs);
void takeoffMotionGatePrepare(const takeoffMotionGateInputStruct *input,
    takeoffMotionGateDecisionStruct *decision);
void takeoffMotionGateFinalize(uint8_t freshTargetAccepted,
    takeoffMotionGateDecisionStruct *decision);

#endif /* _TAKEOFF_MOTION_GATE_H_ */

```

core1_cyt4bb7/project/code/util/crc32leee.c

SHA-256

:

363e996887ef0e2c68b3518217986f209f0760bcd89ff2ca5cdd3a4801583aba

代码 (c)

```

#include "crc32leee.h"

uint32_t crc32leeeCalc(const uint8_t* data, uint32_t len)
{
    uint32_t crc = 0xFFFFFFFFu;
    uint32_t i;

    for (i = 0u; i < len; i++)
    {
        uint8_t b;
        crc ^= (uint32_t) data[i];
        for (b = 0u; b < 8u; b++)
        {
            uint32_t mask = (uint32_t) (-(int32_t) (crc & 1u));
            crc = (crc >> 1) ^ (0xEDB88320u & mask);
        }
    }
    return ~crc;
}

```

core1_cyt4bb7/project/code/util/crc32ieee.h

SHA-256

:

5442cded78a9802395246ceee07532bb7e5de239f12b419b4ecc309d2c9e1001

代码 (c)

```
#ifndef CRC32_IEEE_H
#define CRC32_IEEE_H

#include <stdint.h>

uint32_t crc32ieeeCalc(const uint8_t* data, uint32_t len);

#endif
```

core1_cyt4bb7/project/code/util/schmitt.c

SHA-256

:

716fd0049a36dd9a35e4b5026ad207b0a28151fea3217e50bf0fcfc72c0d8dac

代码 (c)

```
/*
 * schmitt.c
 *
 * Created: 20 250 718
 * Author: Hecate
 */
#include "schmitt.h"

void schmittInit(SchmittTrigger *obj, float upperThresh, float lowerThresh, uint8_t initState)
{
    initState = (initState > 0u) ? 1u : 0u;

    if (upperThresh < lowerThresh)
    {
        obj->upperThreshold = lowerThresh;
        obj->lowerThreshold = upperThresh;
    }
    else
    {
        obj->upperThreshold = upperThresh;
        obj->lowerThreshold = lowerThresh;
    }

    obj->outputState = initState;
}

void schmittSetThresholds(SchmittTrigger *obj, float upperThresh, float lowerThresh)
{
    if (upperThresh < lowerThresh)
    {
        obj->upperThreshold = lowerThresh;
        obj->lowerThreshold = upperThresh;
    }
    else
    {
        obj->upperThreshold = upperThresh;
        obj->lowerThreshold = lowerThresh;
    }
}

void schmittUpdate(SchmittTrigger *obj, float inputValue)
{
    const float val = inputValue;
    obj->outputState = obj->outputState
        ? ((val <= obj->lowerThreshold) ? 0u : 1u)
        : ((val >= obj->upperThreshold) ? 1u : 0u);
}
```

```
void schmittReset(SchmittTrigger *obj, uint8_t state)
{
    obj->outputState = (state > 0u) ? 1u : 0u;
}
```

core1_cyt4bb7/project/code/util/schmitt.h

SHA-256 :

82a6443d92bd0906ba1fb68ed8f10e8f08e6d3cfac07d82de07141db9a40a24d

代码 (c)

```
/*
 * schmitt.h
 *
 * Created: 20 250 718
 * Author: Hecate
 */
#ifndef CODE_SCHMITT_H_
#define CODE_SCHMITT_H_

#include <stdint.h>

/* 施密特触发器对象 */
typedef struct
{
    float upperThreshold;
    float lowerThreshold;
    uint8_t outputState;
} SchmittTrigger;

/* 初始化触发器阈值和输出状态 */
void schmittInit(SchmittTrigger *obj, float upperThresh, float lowerThresh, uint8_t initState);

/* 重新设置上下阈值 */
void schmittSetThresholds(SchmittTrigger *obj, float upperThresh, float lowerThresh);

/* 输入新采样值并更新输出。迟滞语义（边界含等号）：
 * outputState=1（高）时，输入 <= lowerThreshold 才转 0；
 * outputState=0（低）时，输入 >= upperThreshold 才转 1。 */
void schmittUpdate(SchmittTrigger *obj, float inputValue);

/* 仅复位输出状态（不改阈值），state 归一为 0/1。用于目标丢失等需重置输出而不重设阈值的场景。 */
void schmittReset(SchmittTrigger *obj, uint8_t state);

#endif /* CODE_SCHMITT_H_ */
```

core1_cyt4bb7/project/code/util/trackingDifferentiator.c

SHA-256 :

1e41ffe714bd6c20da2376f7b6ccf920e7d1206f9335eb7b64216eed1e033632

代码 (c)

```
#include "trackingDifferentiator.h"

#include <stdint.h>
#include <math.h>

/*
 * TD/fhan 实现，忠实移植自 common-algorithm-library LADRC.c 的 fhan +
 * trackingDifferentiatorProcess（去掉 firstFlag 首帧贴输入，Init/Reset 显式锚定 x1）。
 * double 字面量统一改 float 后缀（嵌入式规范）；d<=0 除零守卫（R/N 已由调用方 clamp 到正区间，
 * 此为额外数值安全）。fal/powf 不移植（仅 LADRC ESO 用）。
 */

static int8_t tdSign(float x)
{
    return (x > 0.0f) ? (int8_t) 1 : ((x < 0.0f) ? (int8_t) -1 : (int8_t) 0);
}
```

```

}

static float tdFhan(float x1, float x2, float r, float h)
{
    float d = r * h * h;
    float a0 = h * x2;
    float y = x1 + a0;
    float a1;
    float a2;
    float sy;
    float a;
    float sa;
    if (d <= 0.0f) return 0.0f; /* 除零/负参数守卫：R/N clamp 后 d>0，此处仅兜底 */
    a1 = sqrtf(d * (d + 8.0f * fabsf(y)));
    a2 = a0 + (float) tdSign(y) * (a1 - d) * 0.5f;
    sy = (float) (tdSign(y + d) - tdSign(y - d)) * 0.5f;
    a = (a0 + y) * sy + a2 * (1.0f - sy);
    sa = (float) (tdSign(a + d) - tdSign(a - d)) * 0.5f;
    return -r * (a / d) * sa - r * (float) tdSign(a) * (1.0f - sa);
}

void trackingDifferentiatorInit(trackingDifferentiatorStruct* obj, float r, float h, float n, float value)
{
    if (obj == 0) return;
    obj->r = r;
    obj->h = h;
    obj->h0 = n * h;
    obj->x1 = value;
    obj->x2 = 0.0f;
}

void trackingDifferentiatorSetParams(trackingDifferentiatorStruct* obj, float r, float h, float n)
{
    if (obj == 0) return;
    obj->r = r;
    obj->h = h;
    obj->h0 = n * h;
}

void trackingDifferentiatorReset(trackingDifferentiatorStruct* obj, float value)
{
    if (obj == 0) return;
    obj->x1 = value;
    obj->x2 = 0.0f;
}

float trackingDifferentiatorUpdate(trackingDifferentiatorStruct* obj, float v)
{
    if (obj == 0) return 0.0f;
    obj->x1 = obj->x1 + obj->h * obj->x2;
    obj->x2 = obj->x2 + obj->h * tdFhan(obj->x1 - v, obj->x2, obj->r, obj->h0);
    return obj->x1;
}

```

core1_cyt4bb7/project/code/util/trackingDifferentiator.h

SHA-256

:

1e1b4f1d419627f3556b03da809ab13bff6109cfe8d467213a2dfe66f7452fce

代码 (c)

```

#ifndef TRACKING_DIFFERENTIATOR_H_
#define TRACKING_DIFFERENTIATOR_H_

/*
 * trackingDifferentiator - 韩京清跟踪微分器 (TD) 最小子集
 *
 * 从 common-algorithm-library 的 LADRC 只移植 TrackingDifferentiator + fhan 两项，
 * 不移植 LADRC/ESO/LEF/fal/powf。用作 beaconNavigator 输出整形末级平滑（替代原一阶低通）：
 * 对已 Max clamp+slew+Min 的命令做二阶跟踪，输出 x1 平滑收敛到目标、x2 为其微分（本用途只取 x1）。
 *
 * 与原 LADRC 差异（有意）：Init/Reset 锚定 x1=value、x2=0，去掉原 firstFlag“首帧直接贴输入”
 * 行为——输出整形须从当前输出连续起步（复位=0），首帧不得跳到输入值造成阶跃。
 *
 * 全前台调用（cm_7_1 whileProcess，非 ISR），静态无动态内存。R/N 由调用方 clamp 后传入。

```

```

*/

typedef struct
{
    float r; /* 快速因子（越大跟踪越快） */
    float h; /* 离散步长（节拍秒） */
    float h0; /* 滤波因子 = n * h */
    float x1; /* 跟踪输出 */
    float x2; /* 微分输出 */
} trackingDifferentiatorStruct;

/* 初始化：设 r/h/h0(=n*h)，锚定 x1=value、x2=0。 */
void trackingDifferentiatorInit(trackingDifferentiatorStruct* obj, float r, float h, float n, float value);

/* 仅刷新参数 r/h/h0(=n*h)，不动状态 x1/x2（供每拍热重载运行时 R/N）。 */
void trackingDifferentiatorSetParams(trackingDifferentiatorStruct* obj, float r, float h, float n);

/* 仅复位状态：x1=value、x2=0（不改参数）。 */
void trackingDifferentiatorReset(trackingDifferentiatorStruct* obj, float value);

/* 输入目标 v，推进一拍，返回跟踪输出 x1。 */
float trackingDifferentiatorUpdate(trackingDifferentiatorStruct* obj, float v);

#endif /* _TRACKING_DIFFERENTIATOR_H_ */

```

core1_cyt4bb7/project/code/vision/beaconTrack.c

SHA-256

:

cc9b0f74a5786d09b6788297b7fb2b74c6e419f1a84e2bae442a6c69e8c70f94

代码 (c)

```

#include "beaconTrack.h"

#include <string.h>

#include "commonDef.h"
#include "configBeaconTrack.h"

/*
 * 时间一致性跟踪实现。逐帧:
 * 1) 全局选取综合 score 最小且满足连续性(距离+面积趋势+面积比)的轨迹-检测对,
 * 关联则平滑更新+hits++, 否则 misses++, hits 归零。坐标速度单位 px/frame,
 * 面积速度单位 area pixels/frame。
 * 2) 删除丢失超限的轨迹(已确认容忍 DROP_MISSES, 未确认容忍 1 帧)。
 * 3) 未关联的检测生成新轨迹。
 * 4) 输出: 已确认且 misses<=COAST_MISSES 的轨迹(滑行容忍偶发漏检)。
 * 定长数组 + 无动态内存; 纯前台调用(非 ISR)。
 */

static float beaconTrackDist2(float ax, float ay, float bx, float by)
{
    float dx = ax - bx;
    float dy = ay - by;
    return dx * dx + dy * dy;
}

static float beaconTrackAbs(float v)
{
    return (v < 0.0f) ? -v : v;
}

static float beaconTrackMax(float a, float b)
{
    return (a > b) ? a : b;
}

static uint16_t beaconTrackClampCoord(float v, uint16_t maxExclusive)
{
    float maxV = (maxExclusive > 0u) ? (float) (maxExclusive - 1u) : 0.0f;
    v = clampf(v, 0.0f, maxV);
    return (uint16_t) (v + 0.5f);
}

static uint16_t beaconTrackClampSize(float v)

```

```

{
    v = clampf(v, 0.0f, (float) OM_BUFFER_SIZE);
    return (uint16_t) (v + 0.5f);
}

static uint8_t beaconTrackIdActive(const beaconTrackerStruct* tracker, uint16_t id)
{
    if (id == 0u) return 1u;
    for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
    {
        if (tracker->tracks[t].active && tracker->tracks[t].trackId == id) return 1u;
    }
    return 0u;
}

static uint16_t beaconTrackAllocId(beaconTrackerStruct* tracker)
{
    uint16_t id;
    for (uint16_t tries = 0u; tries < 0xFFFFu; tries++)
    {
        id = tracker->nextTrackId;
        tracker->nextTrackId++;
        if (tracker->nextTrackId == 0u) tracker->nextTrackId = 1u;
        if (!beaconTrackIdActive(tracker, id)) return id;
    }
    return 0u;
}

static float beaconTrackPredictFramesForOutput(const beaconTrackStruct* tr)
{
    return (tr->misses > 0u) ? (float) tr->misses : VISION_TRACK_PREDICT_FRAMES_DEFAULT;
}

static float beaconTrackPredX(const beaconTrackStruct* tr, float frames)
{
    return tr->lastRawX + tr->velX * frames;
}

static float beaconTrackPredY(const beaconTrackStruct* tr, float frames)
{
    return tr->lastRawY + tr->velY * frames;
}

static float beaconTrackPredSize(const beaconTrackStruct* tr, float frames)
{
    return clampf(tr->lastRawSize + tr->areaVel * frames, 0.0f, (float) OM_BUFFER_SIZE);
}

static float beaconTrackAreaVelDenom(float size)
{
    float areaBase = beaconTrackMax(size, VISION_TRACK_AREA_NORM_MIN);
    return beaconTrackMax(areaBase * VISION_TRACK_ASSOC_AREA_VEL_NORM,
        VISION_TRACK_ASSOC_AREA_VEL_MIN_ABS);
}

static float beaconTrackAssociationScore(float d2, float maxDist2, float detSize,
    float predSize, float rawAreaVel, float areaVelBaseSize,
    float expectedAreaVel, uint8_t areaVelPrimed,
    float areaWeight, float areaVelWeight)
{
    float predDenom = beaconTrackMax(predSize, VISION_TRACK_AREA_NORM_MIN);
    float areaErr = beaconTrackAbs(detSize - predSize) / predDenom;
    float areaScore;
    float velScore = 0.0f;

    areaErr = clampf(areaErr, 0.0f, VISION_TRACK_ASSOC_AREA_ERR_CAP);
    areaScore = areaErr * areaErr;

    if (areaVelPrimed)
    {
        float velErr = beaconTrackAbs(rawAreaVel - expectedAreaVel) / beaconTrackAreaVelDenom(areaVelBaseSize);
        velErr = clampf(velErr, 0.0f, VISION_TRACK_ASSOC_AREA_VEL_ERR_CAP);
        velScore = velErr * velErr;
    }

    return (d2 / maxDist2)
        + areaWeight * areaScore
        + areaVelWeight * velScore;
}

```



```

static uint8_t beaconTrackPairBetter(float score, float d2, uint16_t trackId, uint16_t detIdx,
    float bestScore, float bestD2,
    uint16_t bestTrackId, uint16_t bestDetIdx)
{
    const float eps = 0.0001f;
    if (score < (bestScore - eps)) return 1u;
    if (score > (bestScore + eps)) return 0u;
    if (d2 < (bestD2 - eps)) return 1u;
    if (d2 > (bestD2 + eps)) return 0u;
    if (trackId < bestTrackId) return 1u;
    if (trackId > bestTrackId) return 0u;
    return (detIdx < bestDetIdx) ? 1u : 0u;
}

```

/* 运行时跟踪参数(默认取 configBeaconTrack.h 的 #define, 菜单可热重载)。

* 仅 cm_7_1 前台读写, 无 ISR 竞争, 无需 volatile。 */

typedef struct

```

{
    float maxAssocDist;
    float sizeRatioMax;
    uint16_t confirmHits;
    uint16_t dropMisses;
    uint16_t coastMisses;
    float positionAlpha; /* 关联更新位置低通系数(0<alpha<=1) */
    float sizeAlpha; /* 关联更新面积低通系数(0<alpha<=1) */
    float areaVelAlpha; /* 面积速度低通系数(0<alpha<=1) */
    float assocScoreMax;
    float assocAreaWeight;
    float assocAreaVelWeight;
    beaconTrackBorderConfigStruct border;
    uint16_t edgePredictionHoldFrames;
    beaconTrackEdgeBandsStruct edgeBands;
} beaconTrackConfigStruct;

```

```

#define BEACON_EDGE_SIDE_LEFT (0x01u)
#define BEACON_EDGE_SIDE_RIGHT (0x02u)
#define BEACON_EDGE_SIDE_TOP (0x04u)
#define BEACON_EDGE_SIDE_BOTTOM (0x08u)

```

```

static beaconTrackConfigStruct beaconTrackConfig = {
    VISION_TRACK_MAX_ASSOC_DIST_DEFAULT,
    VISION_TRACK_SIZE_RATIO_MAX_DEFAULT,
    VISION_TRACK_CONFIRM_HITS_DEFAULT,
    VISION_TRACK_DROP_MISSES_DEFAULT,
    VISION_TRACK_COAST_MISSES_DEFAULT,
    VISION_TRACK_POSITION_ALPHA_DEFAULT,
    VISION_TRACK_SIZE_ALPHA_DEFAULT,
    VISION_TRACK_AREA_VEL_ALPHA_DEFAULT,
    VISION_TRACK_ASSOC_SCORE_MAX_DEFAULT,
    VISION_TRACK_ASSOC_AREA_WEIGHT_DEFAULT,
    VISION_TRACK_ASSOC_AREA_VEL_WEIGHT_DEFAULT,
    {
        {VISION_TRACK_BORDER_ENTER_LEFT_DEFAULT,
        VISION_TRACK_BORDER_ENTER_RIGHT_DEFAULT,
        VISION_TRACK_BORDER_ENTER_TOP_DEFAULT,
        VISION_TRACK_BORDER_ENTER_BOTTOM_DEFAULT},
        {VISION_TRACK_BORDER_EXIT_LEFT_DEFAULT,
        VISION_TRACK_BORDER_EXIT_RIGHT_DEFAULT,
        VISION_TRACK_BORDER_EXIT_TOP_DEFAULT,
        VISION_TRACK_BORDER_EXIT_BOTTOM_DEFAULT}
    },
    0u,
    {VISION_TRACK_EDGE_BAND_C1_LEFT_DEFAULT,
    VISION_TRACK_EDGE_BAND_C1_RIGHT_DEFAULT,
    VISION_TRACK_EDGE_BAND_C1_TOP_DEFAULT,
    VISION_TRACK_EDGE_BAND_C1_BOTTOM_DEFAULT}
};

```

```

static float beaconTrackPredictFramesForAssoc(const beaconTrackStruct* tr)
{
    float frames = (float) tr->misses + VISION_TRACK_PREDICT_FRAMES_DEFAULT;
    if ((tr->edgePredictionSideMask != 0u) &&
        (beaconTrackConfig.edgePredictionHoldFrames != 0u))
    {
        frames = clampf(frames, VISION_TRACK_PREDICT_FRAMES_DEFAULT,
            VISION_TRACK_EDGE_ASSOC_PREDICT_FRAMES_MAX);
    }
    return frames;
}

```

```

}

static uint8_t beaconTrackEdgeSideMask(float x, float y)
{
    uint8_t mask = 0u;
    if (beaconTrackConfig.edgePredictionHoldFrames == 0u) return 0u;
    if ((beaconTrackConfig.edgeBands.left != 0u) &&
        (x < (float) beaconTrackConfig.edgeBands.left)) mask |= BEACON_EDGE_SIDE_LEFT;
    if ((beaconTrackConfig.edgeBands.right != 0u) &&
        (x >= (float) OM_IMAGE_MAX_WIDTH - (float) beaconTrackConfig.edgeBands.right))
        mask |= BEACON_EDGE_SIDE_RIGHT;
    if ((beaconTrackConfig.edgeBands.top != 0u) &&
        (y < (float) beaconTrackConfig.edgeBands.top)) mask |= BEACON_EDGE_SIDE_TOP;
    if ((beaconTrackConfig.edgeBands.bottom != 0u) &&
        (y >= (float) OM_IMAGE_MAX_HEIGHT - (float) beaconTrackConfig.edgeBands.bottom))
        mask |= BEACON_EDGE_SIDE_BOTTOM;
    return mask;
}

static uint8_t beaconTrackSideValid(uint8_t enter, uint8_t exit, uint8_t maximum)
{
    if ((enter == 0u) && (exit == 0u)) return 1u;
    return (uint8_t) ((enter >= 1u) && (enter <= maximum) && (exit < enter));
}

static uint8_t beaconTrackBorderValid(const beaconTrackBorderConfigStruct* margins)
{
    if (margins == 0) return 0u;
    return (uint8_t) (
        beaconTrackSideValid(margins->enter.left, margins->exit.left, VISION_TRACK_BORDER_X_MAX) &&
        beaconTrackSideValid(margins->enter.right, margins->exit.right, VISION_TRACK_BORDER_X_MAX) &&
        beaconTrackSideValid(margins->enter.top, margins->exit.top, VISION_TRACK_BORDER_Y_MAX) &&
        beaconTrackSideValid(margins->enter.bottom, margins->exit.bottom, VISION_TRACK_BORDER_Y_MAX));
}

static uint8_t beaconTrackPointInside(float x, float y,
    const beaconTrackBorderMarginsStruct* margin)
{
    return (uint8_t) (((margin->left == 0u) || (x >= (float) margin->left)) &&
        ((margin->right == 0u) || (x <= (float) (OM_IMAGE_MAX_WIDTH - 1u - margin->right))) &&
        ((margin->top == 0u) || (y >= (float) margin->top)) &&
        ((margin->bottom == 0u) || (y <= (float) (OM_IMAGE_MAX_HEIGHT - 1u - margin->bottom))));
}

static uint8_t beaconTrackUpdateBorderExit(beaconTrackStruct* tr, float x, float y)
{
    if (beaconTrackPointInside(x, y, &beaconTrackConfig.border.exit) != 0u)
    {
        tr->borderOutsideFrames = 0u;
        return 0u;
    }

    if (tr->borderOutsideFrames < 0xFFu) tr->borderOutsideFrames++;
    if (tr->borderOutsideFrames >= VISION_TRACK_BORDER_EXIT_HOLD_FRAMES_DEFAULT)
    {
        tr->borderEligible = 0u;
        return 1u;
    }
    return 0u;
}

void beaconTrackSetBorderMargins(const beaconTrackBorderConfigStruct* margins)
{
    if (beaconTrackBorderValid(margins) != 0u)
    {
        beaconTrackConfig.border = *margins;
    }
}

void beaconTrackSetEdgePredictionConfig(
    const beaconTrackEdgePredictionConfigStruct* config)
{
    uint16_t holdMs;
    if (config == 0) return;
    holdMs = config->holdMs;
    if (holdMs > VISION_TRACK_EDGE_PREDICTION_HOLD_MS_MAX)
        holdMs = VISION_TRACK_EDGE_PREDICTION_HOLD_MS_MAX;
    beaconTrackConfig.edgePredictionHoldFrames = (uint16_t)
        ((holdMs + VISION_TRACK_CAMERA_FRAME_PERIOD_MS - 1u) /

```

```

        VISION_TRACK_CAMERA_FRAME_PERIOD_MS);
beaconTrackConfig.edgeBands = config->bands;
if (beaconTrackConfig.edgeBands.left > (uint8_t) VISION_TRACK_EDGE_BAND_X_MAX)
    beaconTrackConfig.edgeBands.left = (uint8_t) VISION_TRACK_EDGE_BAND_X_MAX;
if (beaconTrackConfig.edgeBands.right > (uint8_t) VISION_TRACK_EDGE_BAND_X_MAX)
    beaconTrackConfig.edgeBands.right = (uint8_t) VISION_TRACK_EDGE_BAND_X_MAX;
if (beaconTrackConfig.edgeBands.top > (uint8_t) VISION_TRACK_EDGE_BAND_Y_MAX)
    beaconTrackConfig.edgeBands.top = (uint8_t) VISION_TRACK_EDGE_BAND_Y_MAX;
if (beaconTrackConfig.edgeBands.bottom > (uint8_t) VISION_TRACK_EDGE_BAND_Y_MAX)
    beaconTrackConfig.edgeBands.bottom = (uint8_t) VISION_TRACK_EDGE_BAND_Y_MAX;
}

void beaconTrackSetParams(float maxAssocDist, float sizeRatioMax,
    uint16_t confirmHits, uint16_t dropMisses, uint16_t coastMisses,
    float positionAlpha, float sizeAlpha,
    float areaVelAlpha, float assocScoreMax,
    float assocAreaWeight, float assocAreaVelWeight)
{
    if (maxAssocDist > 0.0f) beaconTrackConfig.maxAssocDist = maxAssocDist;
    if (sizeRatioMax > 1.0f) beaconTrackConfig.sizeRatioMax = sizeRatioMax;
    if (confirmHits > 0u) beaconTrackConfig.confirmHits = confirmHits;
    if (dropMisses > 0u) beaconTrackConfig.dropMisses = dropMisses;
    if (coastMisses > 0u) beaconTrackConfig.coastMisses = coastMisses;
    if (positionAlpha > 0.0f && positionAlpha <= 1.0f) beaconTrackConfig.positionAlpha = positionAlpha;
    if (sizeAlpha > 0.0f && sizeAlpha <= 1.0f) beaconTrackConfig.sizeAlpha = sizeAlpha;
    if (areaVelAlpha > 0.0f && areaVelAlpha <= 1.0f) beaconTrackConfig.areaVelAlpha = areaVelAlpha;
    if (assocScoreMax > 0.0f) beaconTrackConfig.assocScoreMax = assocScoreMax;
    if (assocAreaWeight >= 0.0f) beaconTrackConfig.assocAreaWeight = assocAreaWeight;
    if (assocAreaVelWeight >= 0.0f) beaconTrackConfig.assocAreaVelWeight = assocAreaVelWeight;
}

void beaconTrackInit(beaconTrackerStruct* tracker)
{
    if (0 == tracker) return;
    for (uint16_t i = 0u; i < OM_MAX_OBJECTS; i++)
    {
        tracker->tracks[i].active = 0u;
    }
    tracker->confirmedCount = 0u;
    tracker->nextTrackId = 1u;
    tracker->frameSeq = 0u;
}

void beaconTrackMarkPublished(beaconTrackerStruct* tracker, uint16_t trackId)
{
    uint16_t t;
    if ((tracker == 0) || (trackId == 0u)) return;
    for (t = 0u; t < OM_MAX_OBJECTS; t++)
    {
        beaconTrackStruct *track = &tracker->tracks[t];
        if ((track->active != 0u) && (track->trackId == trackId))
        {
            track->published = 1u;
            return;
        }
    }
}

void beaconTrackUpdate(beaconTrackerStruct* tracker, const BEACON_DETECTOR_STRUCT* det)
{
    if (0 == tracker || 0 == det) return;

    uint16_t nDet = det->beaconCount;
    uint16_t outCount = 0u;
    if (nDet > OM_MAX_OBJECTS) nDet = OM_MAX_OBJECTS;

    uint8_t detUsed[OM_MAX_OBJECTS];
    uint8_t trackUsed[OM_MAX_OBJECTS];
    for (uint16_t i = 0u; i < nDet; i++) detUsed[i] = 0u;
    for (uint16_t i = 0u; i < OM_MAX_OBJECTS; i++) trackUsed[i] = 0u;
    tracker->frameSeq++;

    const float maxDist2 = beaconTrackConfig.maxAssocDist * beaconTrackConfig.maxAssocDist;
    const float ratioMax = beaconTrackConfig.sizeRatioMax;
    const float ratioMin = 1.0f / beaconTrackConfig.sizeRatioMax;
    const float velMax = beaconTrackConfig.maxAssocDist * VISION_TRACK_VELOCITY_MAX_FACTOR_DEFAULT;

    /* EdgeBand 热重载必须先于本帧关联生效，禁止缩小/关闭后再借旧资格
    * 多使用一次纵向放宽门。扩大参数只影响仍存活轨迹。 */

```

```

for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
{
    beaconTrackStruct* tr = &tracker->tracks[t];
    if (!tr->active) continue;
    tr->edgePredictionSideMask = beaconTrackEdgeSideMask(
        tr->lastRawX, tr->lastRawY);
}

/* 1) 全局选择预测残差最小的兼容轨迹-检测对，避免槽位顺序先抢检测。 */
for (::)
{
    int32_t bestTrack = -1;
    int32_t bestDet = -1;
    uint16_t bestTrackId = 0xFFFFu;
    uint16_t bestDetIdx = 0xFFFFu;
    float bestScore = beaconTrackConfig.assocScoreMax;
    float bestD2 = 1.0f;

    for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
    {
        beaconTrackStruct* tr = &tracker->tracks[t];
        if (!tr->active || trackUsed[t]) continue;

        float frames = beaconTrackPredictFramesForAssoc(tr);
        float predX = beaconTrackPredX(tr, frames);
        float predY = beaconTrackPredY(tr, frames);
        float predSize = beaconTrackPredSize(tr, frames);
        float measurementFrames = (float) tr->misses + 1.0f;
        float predSizeDenom = beaconTrackMax(predSize, VISION_TRACK_AREA_NORM_MIN);
        for (uint16_t d = 0u; d < nDet; d++)
        {
            if (detUsed[d]) continue;
            float dcx = (float) det->beaconData[d].centerX;
            float dcy = (float) det->beaconData[d].centerY;
            float detSize = (float) det->beaconData[d].contourSize;
            float rawAreaVel = (detSize - tr->lastRawSize) / measurementFrames;
            float d2 = beaconTrackDist2(dcx, dcy, predX, predY);
            float positionScore;
            float ratio;
            float score;
            if ((tr->edgePredictionSideMask != 0u) &&
                (beaconTrackConfig.edgePredictionHoldFrames != 0u))
            {
                float dx = dcx - predX;
                float dy = dcy - predY;
                float rx = beaconTrackConfig.maxAssocDist;
                float ry = beaconTrackConfig.maxAssocDist;
                if ((tr->edgePredictionSideMask &
                    (BEACON_EDGE_SIDE_LEFT | BEACON_EDGE_SIDE_RIGHT)) != 0u)
                    rx *= VISION_TRACK_EDGE_VERTICAL_ASSOC_SCALE;
                if ((tr->edgePredictionSideMask &
                    (BEACON_EDGE_SIDE_TOP | BEACON_EDGE_SIDE_BOTTOM)) != 0u)
                    ry *= VISION_TRACK_EDGE_VERTICAL_ASSOC_SCALE;
                positionScore = (dx * dx) / (rx * rx) + (dy * dy) / (ry * ry);
                if (positionScore > 1.0f) continue;
            }
            else
            {
                if (d2 > maxDist2) continue;
                positionScore = d2 / maxDist2;
            }
            ratio = detSize / predSizeDenom;
            if (ratio > ratioMax || ratio < ratioMin) continue;
            score = beaconTrackAssociationScore(positionScore, 1.0f, detSize, predSize, rawAreaVel,
                tr->lastRawSize,
                tr->areaVel, tr->areaVelocityPrimed,
                beaconTrackConfig.assocAreaWeight,
                beaconTrackConfig.assocAreaVelWeight);
            if (score > beaconTrackConfig.assocScoreMax) continue;
            if (bestTrack < 0
                || beaconTrackPairBetter(score, positionScore, tr->trackId, d,
                    bestScore, bestD2, bestTrackId, bestDetIdx))
            {
                bestScore = score;
                bestD2 = positionScore;
                bestTrack = (int32_t) t;
                bestDet = (int32_t) d;
                bestTrackId = tr->trackId;
                bestDetIdx = d;
            }
        }
    }
}

```

```

    }
}
}

if (bestTrack < 0 || bestDet < 0) break;

{
    uint16_t t = (uint16_t) bestTrack;
    uint16_t d = (uint16_t) bestDet;
    beaconTrackStruct* tr = &tracker->tracks[t];
    float aP = beaconTrackConfig.positionAlpha;
    float aS = beaconTrackConfig.sizeAlpha;
    float dcx = (float) det->beaconData[d].centerX;
    float dcy = (float) det->beaconData[d].centerY;
    float detSize = (float) det->beaconData[d].contourSize;
    float measurementFrames = (float) tr->misses + 1.0f;
    float rawVx = (dcx - tr->lastRawX) / measurementFrames;
    float rawVy = (dcy - tr->lastRawY) / measurementFrames;
    float rawAreaVel = (detSize - tr->lastRawSize) / measurementFrames;
    float areaVelMax = beaconTrackMax(tr->lastRawSize * VISION_TRACK_AREA_VEL_MAX_RATE,
        VISION_TRACK_ASSOC_AREA_VEL_MIN_ABS);
    rawVx = clampf(rawVx, -velMax, velMax);
    rawVy = clampf(rawVy, -velMax, velMax);
    rawAreaVel = clampf(rawAreaVel, -areaVelMax, areaVelMax);
    if (tr->velocityPrimed)
    {
        float aV = VISION_TRACK_VELOCITY_ALPHA_DEFAULT;
        tr->velX = aV * rawVx + (1.0f - aV) * tr->velX;
        tr->velY = aV * rawVy + (1.0f - aV) * tr->velY;
    }
    else
    {
        tr->velX = rawVx;
        tr->velY = rawVy;
        tr->velocityPrimed = 1u;
    }
    if (tr->areaVelocityPrimed)
    {
        float aA = beaconTrackConfig.areaVelAlpha;
        tr->areaVel = aA * rawAreaVel + (1.0f - aA) * tr->areaVel;
    }
    else
    {
        tr->areaVel = rawAreaVel;
        tr->areaVelocityPrimed = 1u;
    }
    tr->areaVel = clampf(tr->areaVel, -areaVelMax, areaVelMax);
    tr->velX = clampf(tr->velX, -velMax, velMax);
    tr->velY = clampf(tr->velY, -velMax, velMax);
    tr->lastRawX = dcx;
    tr->lastRawY = dcy;
    tr->lastRawSize = detSize;
    trackUsed[t] = 1u;
    detUsed[d] = 1u;
    /* 关联更新一阶低通(alpha*measured + (1-alpha)*old; 越小越平滑, 替代原硬编码 0.5)。 */
    tr->cx = aP * dcx + (1.0f - aP) * tr->cx;
    tr->cy = aP * dcy + (1.0f - aP) * tr->cy;
    tr->size = aS * detSize + (1.0f - aS) * tr->size;
    if (tr->hits < 0xFFFFu) tr->hits++;
    tr->misses = 0u;
    tr->fresh = 1u;
    tr->edgePredictionSideMask = beaconTrackEdgeSideMask(dcx, dcy);
    if (tr->borderEligible == 0u)
    {
        tr->borderEligible = beaconTrackPointInside(dcx, dcy,
            &beaconTrackConfig.border.enter);
        if (tr->borderEligible != 0u) tr->borderOutsideFrames = 0u;
    }
    tr->frameSeq = tracker->frameSeq;
    if (tr->hits >= beaconTrackConfig.confirmHits) tr->confirmed = 1u;
}
}

for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
{
    beaconTrackStruct* tr = &tracker->tracks[t];
    if (!tr->active || trackUsed[t]) continue;
    if (tr->misses < 0xFFFFu) tr->misses++;
    tr->hits = 0u;

```

```

    tr->fresh = 0u;
    tr->frameSeq = tracker->frameSeq;
}

/* 2) 边界拒绝优先于丢失删除和边缘预测。所有从 Enter 区取得资格的
* 活动轨迹都参与 Exit 确认；只有已经确认发布过的轨迹产生一次拒绝事实。*/
for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
{
    beaconTrackStruct* tr = &tracker->tracks[t];
    float predFrames;
    float predX;
    float predY;
    float qualificationX;
    float qualificationY;
    if (!tr->active || (tr->borderEligible == 0u)) continue;
    predFrames = beaconTrackPredictFramesForOutput(tr);
    predX = beaconTrackPredX(tr, predFrames);
    predY = beaconTrackPredY(tr, predFrames);
    qualificationX = (tr->fresh != 0u) ? tr->lastRawX : predX;
    qualificationY = (tr->fresh != 0u) ? tr->lastRawY : predY;
    if (beaconTrackUpdateBorderExit(tr, qualificationX,
        qualificationY) == 0u) continue;
    if (tr->published != 0u)
    {
        memset(&tracker->confirmed[outCount], 0,
            sizeof(tracker->confirmed[outCount]));
        tracker->confirmed[outCount].trackId = tr->trackId;
        tracker->confirmed[outCount].flags =
            BEACON_CONFIRMED_FLAG_BORDER_REJECTED;
        outCount++;
    }
    tr->edgePredictionSideMask = 0u;
    tr->active = 0u;
    tr->confirmed = 0u;
    tr->fresh = 0u;
}

/* 3) 删除丢失超限轨迹 */
for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
{
    beaconTrackStruct* tr = &tracker->tracks[t];
    if (!tr->active) continue;
    uint16_t limit = tr->confirmed ? beaconTrackConfig.dropMisses : 1u;
    if (tr->confirmed && (tr->edgePredictionSideMask != 0u) &&
        (beaconTrackConfig.edgePredictionHoldFrames != 0u))
    {
        limit = beaconTrackConfig.edgePredictionHoldFrames;
    }
    if (tr->misses > limit)
    {
        tr->active = 0u;
        tr->fresh = 0u;
    }
}

/* 4) 未关联检测生成新轨迹(填首个空槽) */
for (uint16_t d = 0u; d < nDet; d++)
{
    if (detUsed[d]) continue;
    if (beaconTrackPointInside((float) det->beaconData[d].centerX,
        (float) det->beaconData[d].centerY,
        &beaconTrackConfig.border.enter) == 0u) continue;
    for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
    {
        beaconTrackStruct* tr = &tracker->tracks[t];
        uint16_t newTrackId;
        if (tr->active) continue;
        newTrackId = beaconTrackAllocId(tracker);
        if (newTrackId == 0u) break;
        tr->cx = (float) det->beaconData[d].centerX;
        tr->cy = (float) det->beaconData[d].centerY;
        tr->size = (float) det->beaconData[d].contourSize;
        tr->lastRawX = tr->cx;
        tr->lastRawY = tr->cy;
        tr->lastRawSize = tr->size;
        tr->velX = 0.0f;
        tr->velY = 0.0f;
        tr->areaVel = 0.0f;
        tr->frameSeq = tracker->frameSeq;
    }
}

```

```

        tr->trackId = newTrackId;
        tr->hits = 1u;
        tr->misses = 0u;
        tr->confirmed = 0u;
        tr->fresh = 1u;
        tr->velocityPrimed = 0u;
        tr->areaVelocityPrimed = 0u;
        tr->borderEligible = beaconTrackPointInside(tr->cx, tr->cy,
            &beaconTrackConfig.border.enter);
        tr->borderOutsideFrames = 0u;
        tr->edgePredictionSideMask = beaconTrackEdgeSideMask(tr->cx, tr->cy);
        tr->published = 0u;
        tr->active = 1u;
        break;
    }
}

/* 5) 输出已确认且仍新鲜(滑行容忍)的轨迹。拒绝记录已在前面。 */
for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
{
    beaconTrackStruct* tr = &tracker->tracks[t];
    if (!tr->active || !tr->confirmed) continue;
    {
        uint16_t outputLimit = beaconTrackConfig.coastMisses;
        if ((tr->edgePredictionSideMask != 0u) &&
            (beaconTrackConfig.edgePredictionHoldFrames != 0u))
        {
            outputLimit = beaconTrackConfig.edgePredictionHoldFrames;
        }
        if (tr->misses > outputLimit) continue;
    }
    float predFrames = beaconTrackPredictFramesForOutput(tr);
    float predX = beaconTrackPredX(tr, predFrames);
    float predY = beaconTrackPredY(tr, predFrames);
    float predSize = beaconTrackPredSize(tr, predFrames);
    if (tr->borderEligible == 0u) continue;
    tracker->confirmed[outCount].centerX = (uint16_t) (tr->cx + 0.5f);
    tracker->confirmed[outCount].centerY = (uint16_t) (tr->cy + 0.5f);
    tracker->confirmed[outCount].size = beaconTrackClampSize(tr->size);
    tracker->confirmed[outCount].trackId = tr->trackId;
    tracker->confirmed[outCount].predX = beaconTrackClampCoord(predX, OM_IMAGE_MAX_WIDTH);
    tracker->confirmed[outCount].predY = beaconTrackClampCoord(predY, OM_IMAGE_MAX_HEIGHT);
    tracker->confirmed[outCount].predSize = beaconTrackClampSize(predSize);
    tracker->confirmed[outCount].misses = tr->misses;
    tracker->confirmed[outCount].fresh = tr->fresh;
    tracker->confirmed[outCount].flags =
        (tr->fresh ? BEACON_CONFIRMED_FLAG_FRESH : 0u) |
        (tr->misses > 0u ? BEACON_CONFIRMED_FLAG_COAST : 0u) |
        ((!tr->fresh && (tr->edgePredictionSideMask != 0u) &&
            (beaconTrackConfig.edgePredictionHoldFrames != 0u))
            ? BEACON_CONFIRMED_FLAG_EDGE_GRACE : 0u);
    tracker->confirmed[outCount].frameSeq = tr->frameSeq;
    tracker->confirmed[outCount].velX = tr->velX;
    tracker->confirmed[outCount].velY = tr->velY;
    tracker->confirmed[outCount].areaVel = tr->areaVel;
    outCount++;
}
tracker->confirmedCount = outCount;
}

```

core1_cyt4bb7/project/code/vision/beaconTrack.h

SHA-256

:

be2aac15fe9f97bae28426015bdfdbee2030013ce755a010847016ab4245e5df

代码 (c)

```

#ifndef BEACON_TRACK_H
#define BEACON_TRACK_H

#include <stdint.h>

#include "openMorph.h" /* BEACON_DETECTOR_STRUCT / BEACON_DATA_STRUCT / OM_MAX_OBJECTS */

#define BEACON_CONFIRMED_FLAG_FRESH    (0x01u)

```

delxxxv

```

#define BEACON_CONFIRMED_FLAG_COAST    (0x02u)
#define BEACON_CONFIRMED_FLAG_EDGE_GRACE (0x04u)
#define BEACON_CONFIRMED_FLAG_BORDER_REJECTED (0x08u)

/*
 * beaconTrack - 信标时间一致性跟踪（抗光关键层）
 *
 * 单帧检测器（默认 Legacy Canny / 热切换 BinaryCC）可能出现瞬时反光误检(FP)与小目标漏检(FN)。本模块在其上叠加
 * 跨帧关联 + 位置/面积连续性门控 + 稳定性计数：连续 confirmHits 帧关联上才
 * 确认输出、连续 dropMisses 帧丢失才删除。瞬时反光跨帧不连续，过不了确认
 * 门控被滤掉。历史 Legacy Canny 仿真(run_beacon_tracking_test.py): 单帧 4 FP → 跟踪 0 FP。
 *
 * 信标恒亮不闪、可多帧延迟，故时间一致性是合理且有效的抗光手段。
 */

/* 跟踪参数默认值已移至 config/configBeaconTrack.h(VISION_TRACK_*_DEFAULT)。 */

/* 单条轨迹 */
typedef struct
{
    float cx;          /* 平滑后质心 X */
    float cy;          /* 平滑后质心 Y */
    float size;        /* 平滑后像素数 */
    float lastRawX;     /* 上次关联的原始检测 X */
    float lastRawY;     /* 上次关联的原始检测 Y */
    float lastRawSize;  /* 上次关联的原始检测面积 */
    float velX;         /* 原始检测速度(px/frame) */
    float velY;         /* 原始检测速度(px/frame) */
    float areaVel;      /* 原始检测面积速度(area pixels/frame) */
    uint32_t frameSeq;  /* 最近一次输出所属图像帧序号 */
    uint16_t trackId;   /* 轨迹 ID; 0 保留为无效 */
    uint16_t hits;      /* 连续命中帧数 */
    uint16_t misses;    /* 连续丢失帧数 */
    uint8_t confirmed;  /* 1=已确认(连续命中达阈值) */
    uint8_t fresh;      /* 1=本帧关联到真实检测 */
    uint8_t velocityPrimed; /* 1=velX/velY 已由连续两次检测建立 */
    uint8_t areaVelocityPrimed; /* 1=areaVel 已由连续两次检测建立 */
    uint8_t borderEligible; /* 1=已通过发布边界施密特门 */
    uint8_t borderOutsideFrames; /* 连续越过 Exit 边界的输出判定帧数 */
    uint8_t edgePredictionSideMask; /* 上次 fresh 命中的 EdgeBand 边位掩码 */
    uint8_t published; /* 至少产生过一帧普通发布记录 */
    uint8_t active;     /* 1=槽位占用 */
} beaconTrackStruct;

/* 确认输出的信标质心(供决策/规划消费) */
typedef struct
{
    uint16_t centerX;
    uint16_t centerY;
    uint16_t size;
    uint16_t trackId;
    uint16_t predX;
    uint16_t predY;
    uint16_t predSize;
    uint16_t misses;
    uint8_t fresh;
    uint8_t flags;
    uint32_t frameSeq;
    float velX;      /* px/frame */
    float velY;      /* px/frame */
    float areaVel;    /* area pixels/frame */
} beaconConfirmedStruct;

typedef struct
{
    uint8_t left;
    uint8_t right;
    uint8_t top;
    uint8_t bottom;
} beaconTrackBorderMarginsStruct;

typedef struct
{
    beaconTrackBorderMarginsStruct enter;
    beaconTrackBorderMarginsStruct exit;
} beaconTrackBorderConfigStruct;

typedef struct
{

```



```

    uint8_t left;
    uint8_t right;
    uint8_t top;
    uint8_t bottom;
} beaconTrackEdgeBandsStruct;

typedef struct
{
    uint16_t holdMs;
    beaconTrackEdgeBandsStruct bands;
} beaconTrackEdgePredictionConfigStruct;

/* 跟踪器 */
typedef struct
{
    beaconTrackStruct tracks[OM_MAX_OBJECTS];
    beaconConfirmedStruct confirmed[OM_MAX_OBJECTS]; /* update 后的确认输出 */
    uint16_t confirmedCount;
    uint16_t nextTrackId;
    uint32_t frameSeq;
} beaconTrackerStruct;

/* 初始化(清空所有轨迹) */
void beaconTrackInit(beaconTrackerStruct* tracker);

/* 运行时设置跟踪参数(默认取 configBeaconTrack.h #define, 菜单可热重载)。
 * positionAlpha/sizeAlpha: 关联更新一阶低通系数(0<alpha<=1, 越小越平滑; 替代原硬编码 0.5)。
 * 仅 cm_7_1 前台 beaconTrackUpdate 读、同核前台写, 无 ISR 竞争。 */
void beaconTrackSetParams(float maxAssocDist, float sizeRatioMax,
    uint16_t confirmHits, uint16_t dropMisses, uint16_t coastMisses,
    float positionAlpha, float sizeAlpha,
    float areaVelAlpha, float assocScoreMax,
    float assocAreaWeight, float assocAreaVelWeight);

void beaconTrackSetBorderMargins(const beaconTrackBorderConfigStruct* margins);
void beaconTrackSetEdgePredictionConfig(
    const beaconTrackEdgePredictionConfigStruct* config);

/* 用本帧 om_beacon 检测结果更新跟踪器; 更新后确认信标在 tracker->confirmed[0..confirmedCount-1] */
void beaconTrackUpdate(beaconTrackerStruct* tracker, const BEACON_DETECTOR_STRUCT* det);
void beaconTrackMarkPublished(beaconTrackerStruct* tracker, uint16_t trackId);

#endif /* BEACON_TRACK_H */

```

core1_cyt4bb7/project/code/vision/image.c

SHA-256

:

d551e3c0bc9ff807abb0f5f6a63bd9abeb317dec33856cc309081067feeb5dc6

代码 (c)

```

#include "image.h"

#include <string.h>

#include "cy_device_headers.h"
#include "zf_device_mt9v03x.h"

#include "configBeaconDetect.h"
#include "openMorph.h"
#include "om_beaconcc.h"

#define IMAGE_CAMERA_INT_VALID_MASK (1u << 31)

/* 相机 ISR 每帧整帧覆写 SDK 公共缓冲 mt9v03x_image; 视觉管线只读本地一致性快照,
 * 避免拷到半新半旧的撕裂帧。 */
typedef struct
{
    uint8_t frameBuffer[MT9V03X_IMAGE_SIZE];
    uint8_t detectMaskBuffer[MT9V03X_IMAGE_SIZE];
    uint8_t gaussianBlurBuffer[MT9V03X_IMAGE_SIZE];
    uint8_t cannyEdgeBuffer[MT9V03X_IMAGE_SIZE];
    u8Image original;
    u8Image detectMask;
    u8Image gaussianBlur;
}

```

```

    u8Image cannyEdge;
    BEACON_DETECTOR_STRUCT detector;
    beaconTrackerStruct tracker;
    uint32_t frameSeq;
    uint8_t detectMode;
    uint8_t frameValid;
} imageContextStruct;

static imageContextStruct imageContext = {
    .original = {imageContext.frameBuffer, MT9V03X_W, MT9V03X_H, MT9V03X_IMAGE_SIZE},
    .detectMask = {imageContext.detectMaskBuffer, MT9V03X_W, MT9V03X_H, MT9V03X_IMAGE_SIZE},
    .gaussianBlur = {imageContext.gaussianBlurBuffer, MT9V03X_W, MT9V03X_H, MT9V03X_IMAGE_SIZE},
    .cannyEdge = {imageContext.cannyEdgeBuffer, MT9V03X_W, MT9V03X_H, MT9V03X_IMAGE_SIZE},
    .detectMode = VISION_DETECT_MODE_DEFAULT
};

static volatile uint32_t* imageCameraSystemIntControl(void)
{
    #if defined(CY_CORE_CM7_0) && (CY_CORE_CM7_0 != 0)
        return &CPUSS->unCM7_0_SYSTEM_INT_CTL[tcpwm_0_interrupts_59_IRQn].u32Register;
    #else
        return &CPUSS->unCM7_1_SYSTEM_INT_CTL[tcpwm_0_interrupts_59_IRQn].u32Register;
    #endif
}

static uint8_t imageTakeFrameSnapshot(void)
{
    volatile uint32_t* intControl;
    uint32_t savedIntControl;

    if (!mt9v03x_finish_flag) return 0u;

    /* CPUIntIdx3 还承载 UART，只屏蔽 MT9V03X 的 TCPWM59 系统中断源。*/
    intControl = imageCameraSystemIntControl();
    savedIntControl = *intControl;
    *intControl = savedIntControl & ~IMAGE_CAMERA_INT_VALID_MASK;
    (void) *intControl;
    __DSB();
    if (mt9v03x_finish_flag != 0u)
    {
        mt9v03x_finish_flag = 0u;
        const uint8_t* src = mt9v03x_image[0];
        for (uint32_t i = 0u; i < MT9V03X_IMAGE_SIZE; i++)
        {
            imageContext.frameBuffer[i] = src[MT9V03X_IMAGE_SIZE - 1u - i];
        }
    }
    else
    {
        if ((savedIntControl & IMAGE_CAMERA_INT_VALID_MASK) != 0u)
        {
            *intControl = savedIntControl;
            (void) *intControl;
            __DSB();
        }
        return 0u;
    }
    if ((savedIntControl & IMAGE_CAMERA_INT_VALID_MASK) != 0u)
    {
        *intControl = savedIntControl;
        (void) *intControl;
        __DSB();
    }
    return 1u;
}

void imageSetExposure(uint16_t exposure)
{
    {
        if (exposure < VISION_EXPOSURE_MIN) exposure = VISION_EXPOSURE_MIN;
        else if (exposure > VISION_EXPOSURE_MAX) exposure = VISION_EXPOSURE_MAX;
        (void) mt9v03x_set_exposure_time(exposure);
    }
}

void imageSetDetectMode(uint8_t detectMode)
{
    {
        uint8_t nextMode = (detectMode == VISION_DETECT_MODE_LEGACY_CANNY)
            ? VISION_DETECT_MODE_LEGACY_CANNY : VISION_DETECT_MODE_BINARYCC;

        if (nextMode != imageContext.detectMode)

```

```

    {
        imageContext.detectMode = nextMode;
        /* 两检测链接选语义不同，禁止旧轨迹跨模式续接。仅前台参数 Apply 调用。*/
        beaconTrackInit(&imageContext.tracker);
    }
}

uint8_t imageGetDetectMode(void)
{
    return imageContext.detectMode;
}

uint8_t imageInit(void)
{
    static const float defaultBeaconHuTemplate[7] = {
        VISION_BEACON_HU_TEMPLATE_0,
        VISION_BEACON_HU_TEMPLATE_1,
        VISION_BEACON_HU_TEMPLATE_2,
        VISION_BEACON_HU_TEMPLATE_3,
        VISION_BEACON_HU_TEMPLATE_4,
        VISION_BEACON_HU_TEMPLATE_5,
        VISION_BEACON_HU_TEMPLATE_6
    };
    BEACONCC_CONFIG_STRUCT beaconCcConfig;
    uint8_t initState = mt9v03x_init();
    if (initState) return 0u; /* 0=成功，非零=失败 */

    imageContext.detectMode = VISION_DETECT_MODE_DEFAULT;
    imageContext.frameSeq = 0u;
    imageContext.frameValid = 0u;

    beaconCcConfig.thresholdLow = (uint8_t) VISION_CANNY_LOW_DEFAULT;
    beaconCcConfig.thresholdHigh = (uint8_t) VISION_CANNY_HIGH_DEFAULT;
    beaconCcConfig.areaMin = VISION_BEACON_SIZE_MIN_DEFAULT;
    beaconCcConfig.areaMax = VISION_BEACON_SIZE_MAX_DEFAULT;
    beaconCcConfig.brightnessMin = VISION_BEACON_BRIGHTNESS_MIN_DEFAULT;
    beaconCcConfig.eccMax = VISION_BEACON_ECC_MAX_DEFAULT;
    beaconCcConfig.fillMin = VISION_BEACON_FILL_MIN_DEFAULT;
    beaconCcConfig.huMax = VISION_BEACON_HU_MAX_DEFAULT;
    beaconCcConfig.huAreaMin = VISION_BEACON_HU_AREA_MIN_DEFAULT;
    beaconCcInit(&beaconCcConfig);
    beaconCcSetTemplate(defaultBeaconHuTemplate, VISION_BEACON_HU_MAX_DEFAULT);

    cannyInit(&imageContext.original, VISION_CANNY_SIGMA, 0u, 0u,
        imageContext.original.xLen - 1u, imageContext.original.yLen - 1u,
        VISION_LEGACY_CANNY_LOW_DEFAULT, VISION_LEGACY_CANNY_HIGH_DEFAULT);
    beaconDetectInit(VISION_BEACON_SIZE_MIN_DEFAULT, VISION_BEACON_SIZE_MAX_DEFAULT,
        VISION_BEACON_ECC_MAX_DEFAULT, VISION_BEACON_BRIGHTNESS_MIN_DEFAULT,
        VISION_BEACON_MAX_ENDPOINTS_DEFAULT);

    beaconTrackInit(&imageContext.tracker);

    return 1u;
}

uint8_t imageProcess(void)
{
    if (!imageTakeFrameSnapshot()) return 0u;

    if (imageContext.detectMode == VISION_DETECT_MODE_LEGACY_CANNY)
    {
        cannyEdgeDetection(&imageContext.original, &imageContext.gaussianBlur, &imageContext.cannyEdge);
        beaconDetect(&imageContext.cannyEdge, &imageContext.original, &imageContext.detector);
    }
    else
    {
        beaconCcDetect(&imageContext.original, &imageContext.detectMask, &imageContext.detector);
    }
    beaconTrackUpdate(&imageContext.tracker, &imageContext.detector);
    imageContext.frameSeq++;
    imageContext.frameValid = 1u;

    return 1u;
}

uint8_t imageGetFrameView(imageFrameKindEnum kind, imageFrameViewStruct *out)
{
    const u8Image *image;

```

```

if (out == NULL) return 0u;
if (kind == IMAGE_FRAME_ORIGINAL) image = &imageContext.original;
else if (kind == IMAGE_FRAME_CANNY) image = &imageContext.cannyEdge;
else if (kind == IMAGE_FRAME_DETECT_PREVIEW)
{
    image = (imageContext.detectMode == VISION_DETECT_MODE_LEGACY_CANNY)
        ? &imageContext.cannyEdge : &imageContext.detectMask;
}
else
{
    memset(out, 0, sizeof(*out));
    return 0u;
}

out->pixels = image->ptr;
out->width = image->xLen;
out->height = image->yLen;
out->frameSeq = imageContext.frameSeq;
out->valid = imageContext.frameValid;
return out->valid;
}

```

```

void imageGetObservationSnapshot(imageObservationSnapshotStruct *out)
{
    uint16_t count;

    if (out == NULL) return;
    count = imageContext.tracker.confirmedCount;
    if (count > OM_MAX_OBJECTS) count = OM_MAX_OBJECTS;
    memset(out, 0, sizeof(*out));
    if (count != 0u)
    {
        uint16_t i;
        memcpy(out->tracks, imageContext.tracker.confirmed,
            (size_t) count * sizeof(out->tracks[0]));
        for (i = 0u; i < count; i++)
        {
            if ((out->tracks[i].flags &
                BEACON_CONFIRMED_FLAG_BORDER_REJECTED) == 0u)
                beaconTrackMarkPublished(&imageContext.tracker,
                    out->tracks[i].trackId);
        }
    }
    out->frameSeq = imageContext.frameSeq;
    out->count = count;
    out->valid = imageContext.frameValid;
    out->detectMode = imageContext.detectMode;
}

```

core1_cyt4bb7/project/code/vision/image.h

SHA-256

:

f134d5a08cda64137d1ab661daefeb0223cb68e62a0a6df6f7e933e7642c8bef

代码 (c)

```

#ifndef IMAGE_H
#define IMAGE_H

#include <stdint.h>

#include "beaconTrack.h"

/*
 * core1 cm_7_1 信标视觉管线(精简版, 对标 core0 image.c 的取帧机制, 去 ^ 灯板/Hu/标定)。
 * 流程: mt9v03x 取帧(seqlock 快照 + 180° 矫正) → 当前 DetectMode 检测层 → 时间一致性跟踪。
 * 输出确认信标质心列表, 供后续路径规划消费(本阶段仅暂存 RAM, 不下发)。
 */

/* 信标检测 / DetectMode 参数默认值已集中到 config/configBeaconDetect.h、跟踪到 config/configBeaconTrack.h(VISION_* 宏),
 * imageInit / visionParamsLoadDefaults / menuModel 从那里取默认值。 */

uint8_t imageInit(void); /* 初始化相机 + 检测器 + 跟踪器; 返回 1=成功 0=失败 */
uint8_t imageProcess(void); /* 单帧: 取帧→当前 DetectMode 检测→跟踪; 返回 1=新帧处理 0=无新帧 */
void imageSetExposure(uint16_t exposure); /* 相机曝光 owner 入口, 调用前须已 imageInit */

```

```

void imageSetDetectMode(uint8_t detectMode); /* 0=BinaryCC, 1=Legacy Canny, 立即切换检测路由 */
uint8_t imageGetDetectMode(void);

typedef enum
{
    IMAGE_FRAME_ORIGINAL = 0,
    IMAGE_FRAME_CANNY,
    IMAGE_FRAME_DETECT_PREVIEW
} imageFrameKindEnum;

typedef struct
{
    const uint8_t *pixels;
    uint16_t width;
    uint16_t height;
    uint32_t frameSeq;
    uint8_t valid;
} imageFrameViewStruct;

typedef struct
{
    beaconConfirmedStruct tracks[OM_MAX_OBJECTS];
    uint32_t frameSeq;
    uint16_t count;
    uint8_t valid;
    uint8_t detectMode;
} imageObservationSnapshotStruct;

/* view 只读, 生命周期到下一次 imageProcess(); snapshot 按值复制同帧轨迹事实。 */
uint8_t imageGetFrameView(imageFrameKindEnum kind, imageFrameViewStruct *out);
void imageGetObservationSnapshot(imageObservationSnapshotStruct *out);

#endif /* IMAGE_H */

```

core1_cyt4bb7/project/code/vision/om_beacon.c

SHA-256

:

dd2dbf23e6197feccadf973df920c27bfab8102b092ce44f7dd026d37230c0f

代码 (c)

```

/*
 * om_beacon.c - [6] 信标检测 (Beacon Detection)
 *
 * 基于 Canny 边缘 + 简化 BFS, 检测多个红外信标 (闭合椭圆/圆形轮廓)
 * 判据: 闭合性 + 尺寸 + 偏心率上限 + 内部亮度
 * 本文件拥有 legacy/CC 信标检测共用的 BFS 缓冲, 旧 Contour 跟踪器已删除。
 */

#include "om_internal.h"

uint32_t inContourMap[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE];
uint16_t contourGeneration = 0u;
const int8_t contourDx[8] = {0, 1, 1, 1, 0, -1, -1, -1};
const int8_t contourDy[8] = {-1, -1, 0, 1, 1, 0, -1};
uint32_t contourQueue[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE];

// ---- typedef ----

typedef struct
{
    uint16_t sizeMax;
    uint16_t sizeMin;
    float eccMaxRatioK; // 预计算: ((eccMax-1)/(eccMax+1))^2, 用于无 sqrt 偏心率比较
    uint8_t brightnessMin; // 内部亮度下限
    uint16_t maxEndpoints; // 允许的最大端点数 (信标边缘常有断裂)
} BEACON_DATA_CONFIG;

// ---- 内部变量 ----

static BEACON_DATA_CONFIG beaconDataConfig;

// ---- 内部函数声明 ----

static inline uint16_t beaconBFS(const u8Image* edgeSrc, uint16_t startX, uint16_t startY, uint32_t genTag, BEACON_DATA_STRUCT* beacon, float*

```

```

outMxx, float* outMyy, float* outMxy);

// ---- 外部函数实现 ----

void beaconDetectInit(uint16_t sizeMin, uint16_t sizeMax, float eccMax, uint8_t brightnessMin, uint16_t maxEndpoints)
{
    beaconDataConfig.sizeMin = sizeMin;
    beaconDataConfig.sizeMax = sizeMax;
    // 预计算偏心率阈值的平方比值, 运行时只需一次乘法+比较
    float k = (eccMax - 1.0f) / (eccMax + 1.0f);
    beaconDataConfig.eccMaxRatioK = k * k;
    beaconDataConfig.brightnessMin = brightnessMin;
    beaconDataConfig.maxEndpoints = maxEndpoints;
}

uint16_t beaconDetect(const u8Image* edgeSrc, const u8Image* graySrc, BEACON_DETECTOR_STRUCT* obj)
{
    if (edgeSrc->size > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE) return 0;

    // generation 替代 memset (与 contourDetect 共享基础设施)
    contourGeneration++;
    if (contourGeneration == 0)
    {
        memset(inContourMap, 0, sizeof(inContourMap));
        contourGeneration = 1;
    }
    uint32_t genBase = (uint32_t) contourGeneration << 16;

    uint16_t beaconCount = 0;
    uint32_t tagCounter = 0;

    for (uint32_t y = 0; y < edgeSrc->yLen && beaconCount < (OM_MAX_OBJECTS - 1u); y++)
    {
        uint32_t rowOff = y * edgeSrc->xLen;
        for (uint32_t x = 0; x < edgeSrc->xLen && beaconCount < 100; x++)
        {
            uint32_t i = rowOff + x;
            if (edgeSrc->ptr[i] != WHITE || (inContourMap[i] >> 16) == contourGeneration) continue;

            tagCounter++;
            if (tagCounter > 0xFFFFu) goto beaconDone;

            uint32_t genTag = genBase | (uint32_t) tagCounter;

            float curMxx, curMyy, curMxy;
            uint16_t ep = beaconBFS(edgeSrc, (uint16_t) x, (uint16_t) y, genTag,
                                   &obj->beaconData[beaconCount], &curMxx, &curMyy, &curMxy);

            // 闭合轮廓过滤 (信标边缘断裂容忍)
            if (ep > beaconDataConfig.maxEndpoints) continue;

            // 尺寸过滤
            uint16_t sz = obj->beaconData[beaconCount].contourSize;
            if (sz < beaconDataConfig.sizeMin || sz > beaconDataConfig.sizeMax) continue;

            // 偏心率上限过滤: 拒绝过度拉伸的非信标轮廓
            // eigMax/eigMin > eccMax <=> diff^2 > sum^2 * ((eccMax-1)/(eccMax+1))^2
            float eigSum = curMxx + curMyy;
            float eigDiff2 = (curMxx - curMyy) * (curMxx - curMyy) + 4.0f * curMxy * curMxy;
            if (eigSum > 0.0f)
            {
                float bound = eigSum * eigSum * beaconDataConfig.eccMaxRatioK;
                if (eigDiff2 > bound) continue;
            }

            // 质心亮度检查: 填充椭圆的质心必在填充区域内, 单像素即可判定
            uint16_t cx = obj->beaconData[beaconCount].centerX;
            uint16_t cy = obj->beaconData[beaconCount].centerY;
            uint8_t centroidBrightness = graySrc->ptr[(uint32_t) cy * graySrc->xLen + (uint32_t) cx];
            obj->beaconData[beaconCount].brightness = centroidBrightness;
            if (centroidBrightness < beaconDataConfig.brightnessMin) continue;

            beaconCount++;
        }
    }

beaconDone:
    obj->beaconCount = beaconCount;
    for (uint16_t i = beaconCount; i < OM_MAX_OBJECTS; i++) obj->beaconData[i].contourSize = 0;
}

```

```

    return beaconCount;
}

// ---- 内部函数实现 ----

// 简化 BFS: 仅计算二阶矩 (无三阶矩/朝向), 返回端点数
static inline uint16_t beaconBFS(const u8Image* obj, uint16_t startX, uint16_t startY, uint32_t genTag, BEACON_DATA_STRUCT* beacon, float* outMxx,
float* outMyy, float* outMxy)
{
    uint32_t width = obj->xLen;
    uint32_t height = obj->yLen;

    beacon->contourSize = 0;
    beacon->centerX = 0;
    beacon->centerY = 0;
    beacon->brightness = 0;

    uint16_t head = 0;
    uint16_t tail = 0;

    inContourMap[startY * width + startX] = genTag;
    contourQueue[tail++] = ((uint32_t) startY << 16) | (uint32_t) startX;

    uint32_t sumX = 0;
    uint32_t sumY = 0;
    uint32_t sumXX = 0;
    uint32_t sumYY = 0;
    uint32_t sumXY = 0;
    uint16_t totalCount = 0;
    uint16_t endpointCount = 0;
    uint8_t abandon = 0;

    while (head < tail)
    {
        uint16_t cx = (uint16_t) (contourQueue[head] & 0xFFFFu);
        uint16_t cy = (uint16_t) (contourQueue[head] >> 16);
        head++;

        totalCount++;

        if (!abandon)
        {
            sumX += cx;
            sumY += cy;
            sumXX += (uint32_t) cx * cx;
            sumYY += (uint32_t) cy * cy;
            sumXY += (uint32_t) cx * cy;
        }

        uint8_t whiteNeighborCount = 0;

        for (uint8_t i = 0; i < 8; i++)
        {
            int32_t nx = (int32_t) cx + contourDx[i];
            int32_t ny = (int32_t) cy + contourDy[i];

            if (nx < 0 || (uint32_t) nx >= width || ny < 0 || (uint32_t) ny >= height) continue;

            uint16_t nIdx = (uint16_t) ((uint32_t) ny * width + (uint32_t) nx);

            if (obj->ptr[nIdx] == WHITE)
            {
                whiteNeighborCount++;

                if ((inContourMap[nIdx] >> 16) != contourGeneration)
                {
                    inContourMap[nIdx] = genTag;
                    if (tail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
                        contourQueue[tail++] = ((uint32_t) ny << 16) | (uint32_t) nx;
                }
            }
        }

        if (!abandon && whiteNeighborCount < 2) endpointCount++;

        if (!abandon && (totalCount > beaconDataConfig.sizeMax || endpointCount > beaconDataConfig.maxEndpoints)) abandon = 1;
    }

    /* abandon 时置 0: 让调用方尺寸过滤 (sz < sizeMin) 直接拒绝本应丢弃的轮廓 */
}

```

```

beacon->contourSize = abandon ? 0u : totalCount;

if (totalCount > 0 && !abandon)
{
    float fN = (float) totalCount;
    float meanX = (float) sumX / fN;
    float meanY = (float) sumY / fN;
    beacon->centerX = (uint16_t) (sumX / totalCount);
    beacon->centerY = (uint16_t) (sumY / totalCount);

    *outMxx = (float) sumXX / fN - meanX * meanX;
    *outMyy = (float) sumYY / fN - meanY * meanY;
    *outMxy = (float) sumXY / fN - meanX * meanY;
}
else
{
    *outMxx = 0.0f;
    *outMyy = 0.0f;
    *outMxy = 0.0f;
}

return endpointCount;
}

```

core1_cyt4bb7/project/code/vision/om_beaconcc.c

SHA-256

:

565bbc8e60e4e0571d715825d28b5f3feeb97bdcd27d99a61ac0b8d4e3570998

代码 (c)

```

/*
 * om_beaconcc.c - core1 信标 BinaryCC 检测
 *
 * 双阈值二值化 + 八连通连通域 + 矩/Hu 筛选, 输出兼容 BEACON_DETECTOR_STRUCT。
 */

#include "om_beaconcc.h"

#include "om_internal.h"

typedef struct
{
    uint16_t area;
    uint16_t bboxMinX;
    uint16_t bboxMinY;
    uint16_t bboxMaxX;
    uint16_t bboxMaxY;
    float centroidX;
    float centroidY;
    float meanBrightness;
    uint8_t peakBrightness;
    float m20;
    float m02;
    float m11;
    float m30;
    float m03;
    float m21;
    float m12;
    float hu[7];
    float huDistance;
    float ecc;
    float fillRatio;
    uint8_t shapeEligible;
    uint8_t huEligible;
} BEACONCC_CANDIDATE_STRUCT;

static BEACONCC_CONFIG_STRUCT beaconCcConfig;
static uint8_t beaconCcTemplateReady = 0u;
static float beaconCcHuTemplate[7];
static uint8_t beaconCcCandidateCount = 0u;

static void beaconCcThreshold(const u8Image* graySrc, u8Image* maskDst);
static uint8_t beaconCcVisited(uint32_t index);
static void beaconCcMarkVisited(uint32_t index);

```



```

static uint8_t beaconCcScanComponent(const u8Image* graySrc, const u8Image* maskDst, uint32_t startIndex,
                                     BEACONCC_CANDIDATE_STRUCT* out);
static void beaconCcFinishCandidate(BEACONCC_CANDIDATE_STRUCT* c, float sumX, float sumY,
                                    float sumX2, float sumY2, float sumXY,
                                    float sumX3, float sumY3, float sumX2Y, float sumXY2,
                                    float sumGray, uint8_t peakBrightness);
static void beaconCcComputeHu(float area, float mu20, float mu02, float mu11, float mu30, float mu03,
                              float mu21, float mu12, float hu[7]);
static float beaconCcHuDistance(const float hu[7], const float templ[7]);

void beaconCcInit(const BEACONCC_CONFIG_STRUCT* config)
{
    memset(&beaconCcConfig, 0, sizeof(beaconCcConfig));
    memset(beaconCcHuTemplate, 0, sizeof(beaconCcHuTemplate));
    beaconCcTemplateReady = 0u;
    beaconCcCandidateCount = 0u;
    if (config != 0) beaconCcConfig = *config;
}

void beaconCcSetConfig(const BEACONCC_CONFIG_STRUCT* config)
{
    if (config == 0) return;
    beaconCcConfig = *config;
}

void beaconCcSetTemplate(const float hu[7], float huMax)
{
    if (hu == 0 || huMax <= 0.0f)
    {
        beaconCcTemplateReady = 0u;
        beaconCcConfig.huMax = 0.0f;
        return;
    }
    for (uint8_t i = 0u; i < 7u; i++) beaconCcHuTemplate[i] = hu[i];
    beaconCcConfig.huMax = huMax;
    beaconCcTemplateReady = 1u;
}

uint16_t beaconCcDetect(const u8Image* graySrc, u8Image* maskDst, BEACON_DETECTOR_STRUCT* obj)
{
    if (obj == 0 || graySrc == 0 || maskDst == 0 || graySrc->ptr == 0 || maskDst->ptr == 0 ||
        graySrc->size > OM_BUFFER_SIZE || maskDst->size < graySrc->size)
    {
        if (obj != 0) obj->beaconCount = 0u;
        return 0u;
    }

    beaconCcThreshold(graySrc, maskDst);
    beaconCcCandidateCount = 0u;

    contourGeneration++;
    if (contourGeneration == 0u)
    {
        memset(inContourMap, 0, graySrc->size * sizeof(inContourMap[0]));
        contourGeneration = 1u;
    }

    for (uint32_t i = 0u; i < graySrc->size; i++)
    {
        BEACONCC_CANDIDATE_STRUCT c;
        if (maskDst->ptr[i] != WHITE || beaconCcVisited(i)) continue;

        memset(&c, 0, sizeof(c));
        if (!beaconCcScanComponent(graySrc, maskDst, i, &c))
        {
            continue;
        }

        if (c.area < beaconCcConfig.areaMin)
        {
            continue;
        }
        if (beaconCcConfig.areaMax > 0u && c.area > beaconCcConfig.areaMax)
        {
            continue;
        }
        if (beaconCcConfig.brightnessMin > 0u && c.meanBrightness < (float) beaconCcConfig.brightnessMin)
        {
            continue;
        }
    }
}

```

```

    }
    if (c.shapeEligible && beaconCcConfig.eccMax > 0.0f && c.ecc > beaconCcConfig.eccMax)
    {
        continue;
    }
    if (c.shapeEligible && beaconCcConfig.fillMin > 0.0f && c.fillRatio < beaconCcConfig.fillMin)
    {
        continue;
    }
    if (beaconCcTemplateReady && c.huEligible && beaconCcConfig.huMax > 0.0f &&
        c.huDistance > beaconCcConfig.huMax)
    {
        continue;
    }

    if (beaconCcCandidateCount < OM_MAX_OBJECTS)
    {
        uint8_t dst = beaconCcCandidateCount;
        obj->beaconData[dst].centerX = (uint16_t) (c.centroidX + 0.5f);
        obj->beaconData[dst].centerY = (uint16_t) (c.centroidY + 0.5f);
        obj->beaconData[dst].contourSize = c.area;
        obj->beaconData[dst].brightness = (uint8_t) (c.meanBrightness + 0.5f);
        beaconCcCandidateCount++;
    }
}

obj->beaconCount = beaconCcCandidateCount;
for (uint8_t i = beaconCcCandidateCount; i < OM_MAX_OBJECTS; i++) obj->beaconData[i].contourSize = 0u;
return beaconCcCandidateCount;
}

static void beaconCcThreshold(const u8Image* graySrc, u8Image* maskDst)
{
    for (uint32_t i = 0u; i < graySrc->size; i++)
    {
        uint8_t v = graySrc->ptr[i];
        maskDst->ptr[i] = (v >= beaconCcConfig.thresholdLow && v <= beaconCcConfig.thresholdHigh) ? WHITE : BLACK;
    }
}

static uint8_t beaconCcVisited(uint32_t index)
{
    return ((inContourMap[index] >> 16) == contourGeneration) ? 1u : 0u;
}

static void beaconCcMarkVisited(uint32_t index)
{
    inContourMap[index] = ((uint32_t) contourGeneration << 16) | 1u;
}

static uint8_t beaconCcScanComponent(const u8Image* graySrc, const u8Image* maskDst, uint32_t startIndex,
    BEACONCC_CANDIDATE_STRUCT* out)
{
    uint32_t head = 0u;
    uint32_t tail = 0u;
    float sumX = 0.0f, sumY = 0.0f, sumX2 = 0.0f, sumY2 = 0.0f, sumXY = 0.0f;
    float sumX3 = 0.0f, sumY3 = 0.0f, sumX2Y = 0.0f, sumXY2 = 0.0f, sumGray = 0.0f;
    uint8_t peakBrightness = 0u;

    out->bboxMinX = graySrc->xLen;
    out->bboxMinY = graySrc->yLen;
    if (tail >= OM_BUFFER_SIZE) return 0u;
    contourQueue[tail++] = startIndex;
    beaconCcMarkVisited(startIndex);

    while (head < tail)
    {
        uint32_t cur = contourQueue[head++];
        uint32_t x = cur % graySrc->xLen;
        uint32_t y = cur / graySrc->xLen;
        float xf = (float) x;
        float yf = (float) y;
        uint8_t gray8 = graySrc->ptr[cur];
        float gray = (float) gray8;

        out->area++;
        if (x < out->bboxMinX) out->bboxMinX = (uint16_t) x;
        if (y < out->bboxMinY) out->bboxMinY = (uint16_t) y;
        if (x > out->bboxMaxX) out->bboxMaxX = (uint16_t) x;

```

```

if (y > out->bboxMaxY) out->bboxMaxY = (uint16_t) y;
if (gray8 > peakBrightness) peakBrightness = gray8;

sumX += xf;
sumY += yf;
sumX2 += xf * xf;
sumY2 += yf * yf;
sumXY += xf * yf;
sumX3 += xf * xf * xf;
sumY3 += yf * yf * yf;
sumX2Y += xf * xf * yf;
sumXY2 += xf * yf * yf;
sumGray += gray;

for (uint8_t ni = 0u; ni < 8u; ni++)
{
    int32_t nx = (int32_t) x + contourDx[ni];
    int32_t ny = (int32_t) y + contourDy[ni];
    uint32_t next;
    if (nx < 0 || ny < 0 || (uint32_t) nx >= graySrc->xLen || (uint32_t) ny >= graySrc->yLen) continue;
    next = (uint32_t) ny * graySrc->xLen + (uint32_t) nx;
    if (maskDst->ptr[next] == WHITE && !beaconCcVisited(next))
    {
        if (tail >= OM_BUFFER_SIZE) return 0u;
        beaconCcMarkVisited(next);
        contourQueue[tail++] = next;
    }
}
}

if (out->area == 0u) return 0u;
beaconCcFinishCandidate(out, sumX, sumY, sumX2, sumY2, sumXY, sumX3, sumY3,
                        sumX2Y, sumXY2, sumGray, peakBrightness);

return 1u;
}

static void beaconCcFinishCandidate(BEACONCC_CANDIDATE_STRUCT* c, float sumX, float sumY,
                                   float sumX2, float sumY2, float sumXY,
                                   float sumX3, float sumY3, float sumX2Y, float sumXY2,
                                   float sumGray, uint8_t peakBrightness)
{
    float area = (float) c->area;
    float cx = sumX / area;
    float cy = sumY / area;
    float mu20 = sumX2 - cx * sumX;
    float mu02 = sumY2 - cy * sumY;
    float mu11 = sumXY - cx * sumY;
    float mu30 = sumX3 - 3.0f * cx * sumX2 + 3.0f * cx * cx * sumX - cx * cx * cx * area;
    float mu03 = sumY3 - 3.0f * cy * sumY2 + 3.0f * cy * cy * sumY - cy * cy * cy * area;
    float mu21 = sumX2Y - cy * sumX2 - 2.0f * cx * sumXY + 2.0f * cx * cy * sumX + cx * cx * sumY - cx * cx * cy * area;
    float mu12 = sumXY2 - cx * sumY2 - 2.0f * cy * sumXY + 2.0f * cx * cy * sumY + cy * cy * sumX - cx * cy * cy * area;
    float covXX = mu20 / area;
    float covYY = mu02 / area;
    float covXY = mu11 / area;
    float trace = covXX + covYY;
    float delta = sqrtf((covXX - covYY) * (covXX - covYY) + 4.0f * covXY * covXY);
    float lambdaA = (trace + delta) * 0.5f;
    float lambdaB = (trace - delta) * 0.5f;
    float bboxArea = (float) ((c->bboxMaxX - c->bboxMinX + 1u) * (c->bboxMaxY - c->bboxMinY + 1u));

    if (lambdaA < lambdaB)
    {
        float t = lambdaA;
        lambdaA = lambdaB;
        lambdaB = t;
    }

    c->centroidX = cx;
    c->centroidY = cy;
    c->meanBrightness = sumGray / area;
    c->peakBrightness = peakBrightness;
    c->m20 = mu20;
    c->m02 = mu02;
    c->m11 = mu11;
    c->m30 = mu30;
    c->m03 = mu03;
    c->m21 = mu21;
    c->m12 = mu12;

```

```

c->ecc = (lambdaB > 1.0e-6f) ? (lambdaA / lambdaB) : 9999.0f;
c->fillRatio = (bboxArea > 0.0f) ? (area / bboxArea) : 0.0f;
beaconCcComputeHu(area, mu20, mu02, mu11, mu30, mu03, mu21, mu12, c->hu);
c->huDistance = beaconCcTemplateReady ? beaconCcHuDistance(c->hu, beaconCcHuTemplate) : -1.0f;
c->shapeEligible = (c->area >= beaconCcConfig.huAreaMin) ? 1u : 0u;
c->huEligible = (beaconCcTemplateReady && c->shapeEligible) ? 1u : 0u;
}

static void beaconCcComputeHu(float area, float mu20, float mu02, float mu11, float mu30, float mu03,
float mu21, float mu12, float hu[7])
{
    float invArea2 = 1.0f / (area * area);
    float invArea25 = invArea2 / sqrtf(area);
    float eta20 = mu20 * invArea2;
    float eta02 = mu02 * invArea2;
    float eta11 = mu11 * invArea2;
    float eta30 = mu30 * invArea25;
    float eta03 = mu03 * invArea25;
    float eta21 = mu21 * invArea25;
    float eta12 = mu12 * invArea25;
    float a = eta30 + eta12;
    float b = eta21 + eta03;
    float c = eta30 - 3.0f * eta12;
    float d = 3.0f * eta21 - eta03;

    hu[0] = eta20 + eta02;
    hu[1] = (eta20 - eta02) * (eta20 - eta02) + 4.0f * eta11 * eta11;
    hu[2] = c * c + d * d;
    hu[3] = a * a + b * b;
    hu[4] = c * a * (a * a - 3.0f * b * b) + d * b * (3.0f * a * a - b * b);
    hu[5] = (eta20 - eta02) * (a * a - b * b) + 4.0f * eta11 * a * b;
    hu[6] = d * a * (a * a - 3.0f * b * b) - c * b * (3.0f * a * a - b * b);
}

static float beaconCcHuDistance(const float hu[7], const float templ[7])
{
    static const float weights[7] = {2.0f, 2.0f, 1.5f, 1.5f, 0.5f, 0.5f, 0.02f};
    float distance = 0.0f;
    for (uint8_t i = 0u; i < 7u; i++)
    {
        float a = fabsf(hu[i]);
        float b = fabsf(templ[i]);
        if (a < 1.0e-12f) a = 1.0e-12f;
        if (b < 1.0e-12f) b = 1.0e-12f;
        distance += weights[i] * fabsf(logf(a) - logf(b));
    }
    return distance;
}

```

core1_cyt4bb7/project/code/vision/om_beaconcc.h

SHA-256

:

27d217cffe3029e12c16e4f6c01415c0c29e8399466bdf209fce9b653cfeae87

代码 (c)

```

#ifndef OM_BEACONCC_H
#define OM_BEACONCC_H

#include <stdint.h>

#include "openMorph.h"

typedef struct
{
    uint8_t thresholdLow;
    uint8_t thresholdHigh;
    uint16_t areaMin;
    uint16_t areaMax;
    uint8_t brightnessMin;
    float eccMax; /* 0=关闭 */
    float fillMin; /* 0=关闭 */
    float huMax; /* 0=关闭 */
    uint16_t huAreaMin; /* 小于该面积的远点只输出 Hu，不按 Hu 拒绝 */
} BEACONCC_CONFIG_STRUCT;

```

```

void beaconCcInit(const BEACONCC_CONFIG_STRUCT* config);
void beaconCcSetConfig(const BEACONCC_CONFIG_STRUCT* config);
void beaconCcSetTemplate(const float hu[7], float huMax);
uint16_t beaconCcDetect(const u8Image* graySrc, u8Image* maskDst, BEACON_DETECTOR_STRUCT* obj);

#endif /* OM_BEACONCC_H */

```

core1_cyt4bb7/project/code/vision/om_canny.c

SHA-256

:

4c84b294f3345b1e280a99322c1a99e614a8d3b009b2003af2ea2a6e35f095a5

代码 (c)

```

/*
 * om_canny.c - [4] 高斯模糊及 Canny 边缘检测
 *
 * 3x3 定点高斯核 / 逐行流水线 Sobel+NMS+滞后门限 边缘检测
 */

#include "om_internal.h"

// ---- define ----

#define CANNY_RING_WIDTH (OM_CANNY_RING_WIDTH)

// ---- typedef ----

typedef struct
{
    uint32_t width;
    uint32_t height;
    uint32_t pixelCount;
    uint32_t cropStartX;
    uint32_t cropStartY;
    uint32_t cropEndX;
    uint32_t cropEndY;
    uint32_t blurStartX;
    uint32_t blurStartY;
    uint32_t blurEndX;
    uint32_t blurEndY;
} gaussConfigStruct;

typedef struct
{
    uint32_t width;
    uint32_t height;
    uint32_t pixelCount;
    uint32_t sobelStartX;
    uint32_t sobelStartY;
    uint32_t sobelEndX;
    uint32_t sobelEndY;
    uint32_t nmsStartX;
    uint32_t nmsStartY;
    uint32_t nmsEndX;
    uint32_t nmsEndY;
    uint16_t thresholdLow;
    uint16_t thresholdHigh;
    uint32_t thresholdLowSq;
    uint32_t thresholdHighSq;
    int32_t trackOffsets[8];
} cannyConfigStruct;

// ---- 内部变量 ----

static uint16_t gaussKernelPreValueInt0[256];
static uint16_t gaussKernelPreValueInt1[256];
static uint16_t gaussKernelPreValueInt2[256];
static gaussConfigStruct gaussConfig = {0u};
static cannyConfigStruct cannyConfig = {0u};
static uint32_t sobelMapRing[3][CANNY_RING_WIDTH] = {0};
static uint8_t sobelDirMapRing[3][CANNY_RING_WIDTH] = {0};
static uint16_t cannyEdgeQueue[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE] = {0};

```

// ---- 外部函数实现 ----

```
void gaussianInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY)
{
    // gaussianKernelTemp[0] : 四个角点对应的核值
    // gaussianKernelTemp[1] : 四个边点对应的核值
    // gaussianKernelTemp[2] : 中心点对应的核值
    float gaussianKernelTemp[3] = {0.0f, 0.0f, 0.0f};

    memset(&gaussConfig, 0, sizeof(gaussConfig));

    if ((NULL == src) || (NULL == src->ptr) || (0u == src->xLen) || (0u == src->yLen) ||
        (src->xLen > CANNY_RING_WIDTH) || ((src->xLen * src->yLen) > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE))
    {
        memset(gaussKernelPreValueInt0, 0, sizeof(gaussKernelPreValueInt0));
        memset(gaussKernelPreValueInt1, 0, sizeof(gaussKernelPreValueInt1));
        memset(gaussKernelPreValueInt2, 0, sizeof(gaussKernelPreValueInt2));
        return;
    }

    gaussConfig.width = src->xLen;
    gaussConfig.height = src->yLen;
    gaussConfig.pixelCount = src->xLen * src->yLen;

    if (startX > endX)
    {
        uint32_t temp = startX;
        startX = endX;
        endX = temp;
    }
    if (startY > endY)
    {
        uint32_t temp = startY;
        startY = endY;
        endY = temp;
    }

    gaussConfig.cropStartX = unsignRange(startX, gaussConfig.width - 1u);
    gaussConfig.cropStartY = unsignRange(startY, gaussConfig.height - 1u);
    gaussConfig.cropEndX = range(endX, gaussConfig.cropStartX, gaussConfig.width - 1u);
    gaussConfig.cropEndY = range(endY, gaussConfig.cropStartY, gaussConfig.height - 1u);

    gaussConfig.blurStartX = 1u;
    gaussConfig.blurStartY = 1u;
    gaussConfig.blurEndX = 0u;
    gaussConfig.blurEndY = 0u;

    if ((gaussConfig.width >= 3u) && (gaussConfig.height >= 3u) &&
        ((gaussConfig.cropEndX - gaussConfig.cropStartX) >= 2u) &&
        ((gaussConfig.cropEndY - gaussConfig.cropStartY) >= 2u))
    {
        gaussConfig.blurStartX = (0u == gaussConfig.cropStartX) ? 1u : (gaussConfig.cropStartX + 1u);
        gaussConfig.blurStartY = (0u == gaussConfig.cropStartY) ? 1u : (gaussConfig.cropStartY + 1u);
        gaussConfig.blurEndX = (gaussConfig.cropEndX >= (gaussConfig.width - 1u)) ? (gaussConfig.width - 2u) : (gaussConfig.cropEndX - 1u);
        gaussConfig.blurEndY = (gaussConfig.cropEndY >= (gaussConfig.height - 1u)) ? (gaussConfig.height - 2u) : (gaussConfig.cropEndY - 1u);
    }

    if (sigma <= 0.0f)
    {
        memset(gaussKernelPreValueInt0, 0, sizeof(gaussKernelPreValueInt0));
        memset(gaussKernelPreValueInt1, 0, sizeof(gaussKernelPreValueInt1));
        memset(gaussKernelPreValueInt2, 0, sizeof(gaussKernelPreValueInt2));
        return;
    }

    float sigma2 = sigma * sigma;
    float coeff = 1.0f / (TWO_PI * sigma2);
    float sum = 0.0f;

    // 生成 3x3 高斯核，并提取 3 个唯一值（角点/边点/中心）
    for (uint8_t y = 0u; y < 3; y++)
    {
        for (uint8_t x = 0u; x < 3; x++)
        {
            int16_t dx = (int16_t) x - 1;
            int16_t dy = (int16_t) y - 1;
            float exponent = -((float) (dx * dx + dy * dy)) / (2.0f * sigma2);
            float value = (float) exp((double) exponent) * coeff;

```

```

        if (0 == dx && 0 == dy) gaussianKernelTemp[2] = value;
        else if (0 == dx || 0 == dy) gaussianKernelTemp[1] = value;
        else gaussianKernelTemp[0] = value;

        sum += value;
    }
}

if (sum <= 0.0f)
{
    memset(gaussKernelPreValueInt0, 0, sizeof(gaussKernelPreValueInt0));
    memset(gaussKernelPreValueInt1, 0, sizeof(gaussKernelPreValueInt1));
    memset(gaussKernelPreValueInt2, 0, sizeof(gaussKernelPreValueInt2));
    return;
}

// 归一化 3 个唯一核值
gaussianKernelTemp[0] /= sum;
gaussianKernelTemp[1] /= sum;
gaussianKernelTemp[2] /= sum;

// 预计算核值与 8bit 灰度值相乘的定点数 (放缩 256 倍), 消除内层浮点开销
for (uint16_t gray = 0u; gray < 256; gray++)
{
    gaussKernelPreValueInt0[gray] = (uint16_t) (gaussianKernelTemp[0] * (float) gray * 256.0f);
    gaussKernelPreValueInt1[gray] = (uint16_t) (gaussianKernelTemp[1] * (float) gray * 256.0f);
    gaussKernelPreValueInt2[gray] = (uint16_t) (gaussianKernelTemp[2] * (float) gray * 256.0f);
}
}

void cannyInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY, uint16_t thresholdLow, uint16_t thresholdHigh)
{
    gaussianInit(src, sigma, startX, startY, endX, endY);

    memset(&cannyConfig, 0, sizeof(cannyConfig));

    if ((NULL == src) || (NULL == src->ptr) || (0u == src->xLen) || (0u == src->yLen) ||
        (src->xLen > CANNY_RING_WIDTH) || ((src->xLen * src->yLen) > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)) return;

    cannyConfig.width = src->xLen;
    cannyConfig.height = src->yLen;
    cannyConfig.pixelCount = src->xLen * src->yLen;

    if (thresholdLow > thresholdHigh)
    {
        uint16_t temp = thresholdLow;
        thresholdLow = thresholdHigh;
        thresholdHigh = temp;
    }

    cannyConfig.thresholdLow = thresholdLow;
    cannyConfig.thresholdHigh = thresholdHigh;

    // 提前计算平方
    cannyConfig.thresholdLowSq = (uint32_t) thresholdLow * thresholdLow;
    cannyConfig.thresholdHighSq = (uint32_t) thresholdHigh * thresholdHigh;

    cannyConfig.sobelStartX = 1u;
    cannyConfig.sobelStartY = 1u;
    cannyConfig.sobelEndX = 0u;
    cannyConfig.sobelEndY = 0u;
    cannyConfig.nmsStartX = 1u;
    cannyConfig.nmsStartY = 1u;
    cannyConfig.nmsEndX = 0u;
    cannyConfig.nmsEndY = 0u;

    if ((gaussConfig.blurEndX > (gaussConfig.blurStartX + 1u)) &&
        (gaussConfig.blurEndY > (gaussConfig.blurStartY + 1u)))
    {
        cannyConfig.sobelStartX = gaussConfig.blurStartX + 1u;
        cannyConfig.sobelStartY = gaussConfig.blurStartY + 1u;
        cannyConfig.sobelEndX = gaussConfig.blurEndX - 1u;
        cannyConfig.sobelEndY = gaussConfig.blurEndY - 1u;
    }

    if ((cannyConfig.sobelEndX > (cannyConfig.sobelStartX + 1u)) &&
        (cannyConfig.sobelEndY > (cannyConfig.sobelStartY + 1u)))

```

```

{
    cannyConfig.nmsStartX = cannyConfig.sobelStartX + 1u;
    cannyConfig.nmsStartY = cannyConfig.sobelStartY + 1u;
    cannyConfig.nmsEndX = cannyConfig.sobelEndX - 1u;
    cannyConfig.nmsEndY = cannyConfig.sobelEndY - 1u;
}

// 预先计算滞后门限追踪(连边)阶段需要的直接邻信偏移
cannyConfig.trackOffsets[0] = -(int32_t) gaussConfig.width - 1;
cannyConfig.trackOffsets[1] = -(int32_t) gaussConfig.width;
cannyConfig.trackOffsets[2] = -(int32_t) gaussConfig.width + 1;
cannyConfig.trackOffsets[3] = -1;
cannyConfig.trackOffsets[4] = 1;
cannyConfig.trackOffsets[5] = (int32_t) gaussConfig.width - 1;
cannyConfig.trackOffsets[6] = (int32_t) gaussConfig.width;
cannyConfig.trackOffsets[7] = (int32_t) gaussConfig.width + 1;
}

/* 运行时仅更新 Canny 双阈值，不重算高斯核与 ROI，便于按键调参 */
void cannySetThreshold(uint16_t thresholdLow, uint16_t thresholdHigh)
{
    if (thresholdLow > thresholdHigh)
    {
        uint16_t temp = thresholdLow;
        thresholdLow = thresholdHigh;
        thresholdHigh = temp;
    }

    cannyConfig.thresholdLow = thresholdLow;
    cannyConfig.thresholdHigh = thresholdHigh;
    cannyConfig.thresholdLowSq = (uint32_t) thresholdLow * thresholdLow;
    cannyConfig.thresholdHighSq = (uint32_t) thresholdHigh * thresholdHigh;
}

void cannyEdgeDetection(const u8Image* src, u8Image* dstGaussBlur, u8Image* dst)
{
    if (NULL == src || NULL == src->ptr || NULL == dstGaussBlur || NULL == dstGaussBlur->ptr ||
        NULL == dst || NULL == dst->ptr) return;
    if (0u == cannyConfig.pixelCount || 0u == gaussConfig.width) return;

    uint32_t queueHead = 0u;
    uint32_t queueTail = 0u;

    // 清零目标画布、ring buffer 残留、高斯模糊边界脏值
    memset(dst->ptr, BLACK, gaussConfig.width * gaussConfig.height);
    memset(dstGaussBlur->ptr, 0, gaussConfig.width * gaussConfig.height);
    memset(sobelMapRing, 0, sizeof(sobelMapRing));
    memset(sobelDirMapRing, 0, sizeof(sobelDirMapRing));

    for (uint32_t y = gaussConfig.blurStartY; y <= gaussConfig.blurEndY; y++)
    {
        // ===== 第1阶段: 高斯模糊 =====
        const uint8_t* row0 = src->ptr + (y - 1u) * gaussConfig.width;
        const uint8_t* row1 = src->ptr + y * gaussConfig.width;
        const uint8_t* row2 = src->ptr + (y + 1u) * gaussConfig.width;
        uint8_t* dstRow = dstGaussBlur->ptr + y * gaussConfig.width;

        for (uint32_t x = gaussConfig.blurStartX; x <= gaussConfig.blurEndX; x++)
        {
            uint32_t result = gaussKernelPreValueInt0[row0[x - 1u]] + gaussKernelPreValueInt1[row0[x]] + gaussKernelPreValueInt0[row0[x + 1u]] +
                gaussKernelPreValueInt1[row1[x - 1u]] + gaussKernelPreValueInt2[row1[x]] + gaussKernelPreValueInt1[row1[x + 1u]] +
                gaussKernelPreValueInt0[row2[x - 1u]] + gaussKernelPreValueInt1[row2[x]] + gaussKernelPreValueInt0[row2[x + 1u]];

            result = (result + 128) >> 8;
            dstRow[x] = (uint8_t) ((result > 255) ? 255 : result);
        }

        // ===== 第2阶段: Sobel 计算 =====
        if ((cannyConfig.sobelStartX <= cannyConfig.sobelEndX) &&
            (cannyConfig.sobelStartY <= cannyConfig.sobelEndY) &&
            (y >= (cannyConfig.sobelStartY + 1u)) &&
            ((y - 1u) <= cannyConfig.sobelEndY))
        {
            const uint8_t* s_row0 = dstGaussBlur->ptr + (y - 2u) * gaussConfig.width;
            const uint8_t* s_row1 = dstGaussBlur->ptr + (y - 1u) * gaussConfig.width;
            const uint8_t* s_row2 = dstGaussBlur->ptr + y * gaussConfig.width;

            uint32_t* sobelRow = sobelMapRing[(y - 1u) % 3];
            uint8_t* dirRow = sobelDirMapRing[(y - 1u) % 3];

```



```

for (uint32_t x = cannyConfig.sobelStartX; x <= cannyConfig.sobelEndX; x++)
{
    int32_t gx = (int32_t) s_row0[x + 1u] + ((int32_t) s_row1[x + 1u] << 1) + (int32_t) s_row2[x + 1u] - (int32_t) s_row0[x - 1u] - ((int32_t) s_row1[x - 1u] << 1) - (int32_t) s_row2[x - 1u];
    int32_t gy = (int32_t) s_row0[x - 1u] + ((int32_t) s_row0[x] << 1) + (int32_t) s_row0[x + 1u] - (int32_t) s_row2[x - 1u] - ((int32_t) s_row2[x] << 1) - (int32_t) s_row2[x + 1u];

    // 去掉掉极其耗时的 sqrtf 运算
    uint32_t mag_sq = (uint32_t) (gx * gx) + (uint32_t) (gy * gy);
    sobelRow[x] = mag_sq;

    // 完全消除浮点除法，只用整数比较判定 4 大区域方向
    int32_t ax = (gx >= 0) ? gx : -gx;
    int32_t ay = (gy >= 0) ? gy : -gy;
    int32_t ay256 = ay << 8;

    if (ay256 <= ax * 106)
    {
        dirRow[x] = 0; // 水平梯度主导（纵向边缘），沿水平方向抑制
    }
    else if (ay256 >= ax * 618)
    {
        dirRow[x] = 2; // 垂直梯度主导（横向边缘），沿垂直方向抑制
    }
    else if ((gx ^ gy) >= 0)
    {
        dirRow[x] = 1; // 45 度梯度
    }
    else
    {
        dirRow[x] = 3; // 135 度梯度
    }
}

// ===== 第 3 阶段: 非极大值抑制 =====
if ((cannyConfig.nmsStartX <= cannyConfig.nmsEndX) &&
    (cannyConfig.nmsStartY <= cannyConfig.nmsEndY) &&
    (y >= (cannyConfig.nmsStartY + 2u)) &&
    ((y - 2u) <= cannyConfig.nmsEndY))
{
    const uint32_t* n_row0 = sobelMapRing[(y - 3u) % 3];
    const uint32_t* n_row1 = sobelMapRing[(y - 2u) % 3];
    const uint32_t* n_row2 = sobelMapRing[(y - 1u) % 3];
    const uint8_t* dirRow = sobelDirMapRing[(y - 2u) % 3];

    uint8_t* final_dstRow = dst->ptr + (y - 2u) * gaussConfig.width + cannyConfig.nmsStartX;

    for (uint32_t x = cannyConfig.nmsStartX; x <= cannyConfig.nmsEndX; x++)
    {
        uint32_t edge0, edge1;
        uint32_t magnitude = n_row1[x];
        uint8_t dir = dirRow[x];

        if (dir == 0)
        {
            edge0 = n_row1[x - 1u];
            edge1 = n_row1[x + 1u];
        }
        else if (dir == 2)
        {
            edge0 = n_row0[x];
            edge1 = n_row2[x];
        }
        else if (dir == 1)
        {
            // gx, gy 同号 → 梯度方向 ↗ (亮度梯度指向右上) → NMS 比较 ↗ 与 ↘
            edge0 = n_row0[x + 1u];
            edge1 = n_row2[x - 1u];
        }
        else
        {
            // gx, gy 异号 → 梯度方向 ↘ (亮度梯度指向右下) → NMS 比较 ↘ 与 ↗
            edge0 = n_row0[x - 1u];
            edge1 = n_row2[x + 1u];
        }

        if ((magnitude > edge0) && (magnitude >= edge1))
    }
}

```

```

        {
            if (magnitude >= cannyConfig.thresholdHighSq)
            {
                *final_dstRow = WHITE; // 确信强边缘
                if (queueTail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
                {
                    cannyEdgeQueue[queueTail++] = (uint16_t) ((y - 2u) * gaussConfig.width + x);
                }
            }
            else if (magnitude >= cannyConfig.thresholdLowSq)
            {
                *final_dstRow = 127; // 先标记为弱边缘，用于第四阶段追踪，避免大数组分配
            }
        }

        final_dstRow++;
    }
}

// ===== 第4阶段: 滞后门限追踪(连边) =====
while (queueHead < queueTail)
{
    uint32_t index = cannyEdgeQueue[queueHead++];

    for (uint8_t i = 0; i < 8; i++)
    {
        int32_t trackIndex = (int32_t) index + cannyConfig.trackOffsets[i];

        if (trackIndex >= 0 && trackIndex < (int32_t) cannyConfig.pixelCount)
        {
            if (dst->ptr[trackIndex] == 127)
            {
                dst->ptr[trackIndex] = WHITE; // 连通升级为强边缘

                // 防溢出保护
                if (queueTail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
                {
                    cannyEdgeQueue[queueTail++] = (uint16_t) trackIndex;
                }
            }
        }
    }
}

// ===== 收尾: 清理死路 =====
uint32_t totalPixels = gaussConfig.width * gaussConfig.height;
for (uint32_t i = 0; i < totalPixels; i++)
{
    if (dst->ptr[i] == 127) dst->ptr[i] = BLACK; // 无法与强边缘连通被消除
}
}

```

core1_cyt4bb7/project/code/vision/om_internal.h

SHA-256

:

bcfdd795561cd68e2d4d11628decee010070dc0504b92446c93b48d78179c857

代码 (c)

```

/*
 * om_internal.h - openMorph 内部共享头文件 (不对外暴露)
 *
 * 提供各子模块共用的宏定义、标准库包含及共享 BFS 基础设施声明。
 * 外部用户代码只需包含 openMorph.h。
 */
#ifndef OM_INTERNAL_H
#define OM_INTERNAL_H

// =====
// 公共接口 + 基础设施
// =====

#include "openMorph.h" // 公共类型定义与 API 声明
#include "openMorph_config.h" // 可移植尺寸/数量配置 (移植时只改此文件)

```

```

#include "commonDef.h"    // u8Image, BLACK, WHITE, cReverse, range 等

#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

// =====
// 共享宏定义
// =====

#define UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE (OM_BUFFER_SIZE)

// =====
// legacy/CC 信标检测共用 BFS 基础设施 (om_beacon.c 定义)
// =====

extern uint32_t inContourMap[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE];
extern uint16_t contourGeneration;
extern const int8_t contourDx[8];
extern const int8_t contourDy[8];
extern uint32_t contourQueue[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE];

#endif // OM_INTERNAL_H

```

core1_cyt4bb7/project/code/vision/openMorph.h

SHA-256

:

eb7197ab53930ebea0c5238183271afebe3fbd5829beba45676b0877586aac31

代码 (c)

```

/*
 * openMorph.h - 图像处理 头文件
 *
 * Created: 20 250 804
 * Author: Hecate
 *
 * 活跃模块: 高斯/Canny、 legacy 信标与 BinaryCC 信标检测。
 */
#ifndef OPENMORPH_H
#define OPENMORPH_H

// =====
// 包含文件
// =====

#include <stdint.h>

#include "commonDef.h"
#include "openMorph_config.h"
// =====
// [4] 高斯模糊与 Canny 边缘检测
// =====

// ---- 函数声明 ----

void gaussianInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY);
void cannyInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY, uint16_t thresholdLow, uint16_t thresholdHigh);
void cannySetThreshold(uint16_t thresholdLow, uint16_t thresholdHigh);
void cannyEdgeDetection(const u8Image* src, u8Image* dstGaussBlur, u8Image* dst);

// =====
// [6] 信标检测 (Beacon Detection)
// =====

// ---- typedef ----

// 单个信标数据
typedef struct
{
    uint16_t centerX;    // 质心 X
    uint16_t centerY;    // 质心 Y
    uint16_t contourSize; // 轮廓像素数

```

```

    uint8_t brightness; // 质心区域亮度 (用于排查)
} BEACON_DATA_STRUCT;

// 信标检测器数据
typedef struct
{
    BEACON_DATA_STRUCT beaconData[OM_MAX_OBJECTS]; // 信标数据缓存
    uint8_t beaconCount; // 有效信标数量
} BEACON_DETECTOR_STRUCT;

// ---- 函数声明 ----

// eccMax: 偏心率上限 (>=1.0, 1.0=理想圆, 3.0=允许中等椭圆)
// brightnessMin: 信标内部亮度下限 (0~255, 建议大于 120 才视为发光体)
// maxEndpoints: 允许最大端点数 (信标边缘可有断口, 建议 6~10)
void beaconDetectInit(uint16_t sizeMin, uint16_t sizeMax, float eccMax, uint8_t brightnessMin, uint16_t maxEndpoints);

// 检测: edgeSrc=Canny 边缘图, graySrc=灰度原图 (亮度采样)
// 返回检测到的信标数量 (结果存入 obj->beaconData)
uint16_t beaconDetect(const u8Image* edgeSrc, const u8Image* graySrc, BEACON_DETECTOR_STRUCT* obj);

#endif // OPENMORPH_H

```

core1_cyt4bb7/project/code/vision/openMorph_config.h

SHA-256

:

9a3d8e074f1ab1f10ba3ffdd8632db9ef6e688e82751a9f12884da29c8ca4bfd

代码 (c)

```

/*
 * openMorph_config.h - openMorph 库可移植配置头
 *
 * 移植到新工程时只需修改本文件即可适配不同图像分辨率与对象数量上限。
 * openMorph 内部其他源文件不应再出现关于尺寸的字面量。
 */
#ifndef OPENMORPH_CONFIG_H
#define OPENMORPH_CONFIG_H

// =====
// 图像最大尺寸 (静态缓冲区按此分配)
// =====
// core1 实际相机 MT9V03X 为 188 x 120 = 22560 像素 (与 image.c 的 MT9V03X_IMAGE_SIZE 一致)。
// 必须与相机分辨率匹配: 若小于真实分辨率, om_canny/cannyInit 入口保护
// ((xLen*yLen) > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE) 会拒绝处理 → canny 图纯黑、信标失效。
// 注意: core0/core1 当前均为 188x120; openMorph_config.h 仍按各核相机独立维护。
// 像素索引以 uint16 存储 (om_beacon/om_canny), 故 W*H 须 <= 65535; 22560 在范围内。
// 本配置与具体相机驱动解耦, 不引入 SDK 头依赖。
// -----
#define OM_IMAGE_MAX_WIDTH (188u)
#define OM_IMAGE_MAX_HEIGHT (120u)
#define OM_BUFFER_SIZE (OM_IMAGE_MAX_WIDTH * OM_IMAGE_MAX_HEIGHT)

// =====
// 对象检测数量上限 (轮廓 / 信标 / 连通域共用)
// =====
// contourData[] / beaconData[] / sbea[] 等数组的成员数量。
// 超过此上限的候选对象会被忽略 (只保留前 N 个有效结果)。
// -----
#define OM_MAX_OBJECTS (101u)

// =====
// Canny 流水线环形缓冲宽度
// =====
// Canny 边缘检测内部 sobel 环形缓冲的列宽, 必须 >= OM_IMAGE_MAX_WIDTH。
// -----
#define OM_CANNY_RING_WIDTH (256u)

// =====
// 轮廓跟踪状态机参数
// =====
#define OM_LOCK_THRESHOLD (2u) // 连续检测到帧数: SEARCHING → ACQUIRING
#define OM_LOCK_CONFIRM (1u) // 继续确认帧数: ACQUIRING → LOCKED
#define OM_LOST_THRESHOLD (5u) // 连续丢失帧数: LOCKED → LOST_HOLDING
#define OM_LOST_TIMEOUT (30u) // 超时帧数: LOST_HOLDING → SEARCHING

```

```

#define OM_RAY_BALANCE_CONFIDENCE_MIN (0.50f) // ^尖端几何定向最低置信度: 低于此值拒绝更新朝向, 目标位置仍有效
#define OM_ORIENTATION_MIN_SIZE (8u) // 朝向可靠的最小像素数 (与 core0 openMorph 算法同源)
#define OM_ORIENTATION_ALPHA (0.4f) // 朝向低通滤波系数 (喂给 lowPassFilter, 越大越跟随/越小越平滑)
#define OM_POSITION_ALPHA (1.0f) // 质心位置不过滤
#define OM_VELOCITY_ALPHA (0.1f) // 原始质心差分速度低通滤波系数
#define OM_VELOCITY_DT_MIN_S (0.001f) // 有效图像样本间隔下限
#define OM_VELOCITY_DT_MAX_S (0.200f) // 超过此间隔只重建速度基线
#define OM_BORDER_MARGIN (1u) // 画幅边缘安全边距

// =====
// 朝向跳变保护默认值 (运行时可由菜单热重载, 逻辑同 ToF 定高跳变保护)
// NORMAL 态: |Δθ| > JUMP_MAX 视为异常突跳, 拒绝该样本进 SUSPENDED
// SUSPEND 态: |Δθ| < RECOVER_RAD 即时恢复; 介于二者间累计 CONFIRM_FRAMES 帧后
// 接受新朝向为基线; 持续 > JUMP_MAX 则计数清零继续沿用旧朝向
// -----
#define OM_ORIENTATION_JUMP_MAX (0.785398f) // PI/4, 朝向跳变阈值 (rad)
#define OM_ORIENTATION_RECOVER_RAD (0.196349f) // PI/16, 恢复窄带 (rad)
#define OM_ORIENTATION_CONFIRM_FRAMES (3u) // 异常持续这么多帧后接受新朝向

#endif // OPENMORPH_CONFIG_H

```

core1_cyt4bb7/project/code/vision/visionParams.c

SHA-256

:

66966a1ef76771e4612877658943a92233bbdfa4d815de4d5d11dfca44a4dff7

代码 (c)

```

#include "visionParams.h"

#include <string.h>

#include "visionParamsInternal.h"

#include "commonDef.h" /* clampf */
#include "configBeaconDetect.h" /* VISION_CANNY_*/ VISION_EXPOSURE_*/ VISION_BEACON_*/ 默认宏 */
#include "configBeaconTrack.h" /* VISION_TRACK_*/ 默认/范围 + 关联 alpha */
#include "configBeaconPlanning.h" /* PLANNING_*/ 运行时默认/约束 + TD/sign/派发 */
#include "configBeaconFusion.h"
#include "beaconTrack.h" /* beaconTrackSetParams */
#include "beaconNavigator.h"
#include "image.h" /* imageSetExposure */
#include "openMorph.h" /* cannySetThreshold / beaconDetectInit */
#include "om_beaconcc.h" /* beaconCcSetConfig / beaconCcSetTemplate */

/*
 * visionParams.c — core1 视觉参数运行时模型。
 * 持有参数实体、默认加载、sanitize/clamp 与运行时 Apply。
 * 不触屏/不读按键; 全部前台调用。
 */

typedef struct
{
    visionParamsStruct active;
    uint32_t revision;
    uint8_t initialized;
} visionParamsRepositoryStruct;

static visionParamsRepositoryStruct visionParamsRepository;

visionParamsStruct *visionParamsRepositoryMutable(void)
{
    return &visionParamsRepository.active;
}

void visionParamsGetActive(visionParamsStruct *out)
{
    if (out == 0) return;
    *out = visionParamsRepository.active;
}

uint32_t visionParamsGetRevision(void)
{
    return visionParamsRepository.revision;
}

```

```

uint8_t visionParamsCommit(const visionParamsStruct *candidate)
{
    if (candidate == 0) return 0u;
    visionParamsRepository.active = *candidate;
    visionParamsApply();
    return 1u;
}

uint8_t visionParamsUpdateSharedIdentity(uint32_t revision, uint32_t hash)
{
    if (revision == 0u) return 0u;
    if ((visionParams.sharedConfigRevision == revision) &&
        (visionParams.sharedConfigHash == hash))
    {
        return 1u;
    }
    visionParams.sharedConfigRevision = revision;
    visionParams.sharedConfigHash = hash;
    visionParamsRepository.revision++;
    if (visionParamsRepository.revision == 0u) visionParamsRepository.revision = 1u;
    return 1u;
}

/* NaN/Inf 回默认为 clamp (clampf 对 NaN 返回 NaN, 需先剔除; 不引入 math.h) 。 */
static float sanitizeFloat(float v, float def, float lo, float hi)
{
    if (v != v || v > 1.0e30f || v < -1.0e30f) v = def;
    return clampf(v, lo, hi);
}

static void sanitizeBorderPair(uint8_t *enter, uint8_t *exit, uint8_t maxValue,
    uint8_t defaultEnter, uint8_t defaultExit)
{
    if ((enter == 0) || (exit == 0)) return;

    if ((*enter == 0u) && (*exit == 0u))
    {
        return;
    }
    if ((*enter < 1u) || (*enter > maxValue) || (*exit >= *enter))
    {
        *enter = defaultEnter;
        *exit = defaultExit;
    }
}

void visionParamsClamp(void)
{
    visionParams.detectMode = (visionParams.detectMode == VISION_DETECT_MODE_LEGACY_CANNY)
        ? VISION_DETECT_MODE_LEGACY_CANNY
        : VISION_DETECT_MODE_BINARYCC;

    if (visionParams.cannyLow < VISION_CANNY_THRESHOLD_MIN) visionParams.cannyLow = VISION_CANNY_THRESHOLD_MIN;
    if (visionParams.cannyHigh > VISION_CANNY_THRESHOLD_MAX) visionParams.cannyHigh = VISION_CANNY_THRESHOLD_MAX;
    if ((uint16_t) (visionParams.cannyLow + VISION_CANNY_THRESHOLD_GAP) > visionParams.cannyHigh)
    {
        if (visionParams.cannyHigh > (uint16_t) (VISION_CANNY_THRESHOLD_MIN + VISION_CANNY_THRESHOLD_GAP))
        {
            visionParams.cannyLow = (uint16_t) (visionParams.cannyHigh - VISION_CANNY_THRESHOLD_GAP);
        }
        else
        {
            visionParams.cannyLow = VISION_CANNY_THRESHOLD_MIN;
            visionParams.cannyHigh = (uint16_t) (VISION_CANNY_THRESHOLD_MIN + VISION_CANNY_THRESHOLD_GAP);
        }
    }

    if (visionParams.exposure < VISION_EXPOSURE_MIN) visionParams.exposure = VISION_EXPOSURE_MIN;
    if (visionParams.exposure > VISION_EXPOSURE_MAX) visionParams.exposure = VISION_EXPOSURE_MAX;
    if (visionParams.legacyCannyLow < VISION_LEGACY_CANNY_THRESHOLD_MIN)
        visionParams.legacyCannyLow = VISION_LEGACY_CANNY_THRESHOLD_MIN;
    if (visionParams.legacyCannyHigh > VISION_LEGACY_CANNY_THRESHOLD_MAX)
        visionParams.legacyCannyHigh = VISION_LEGACY_CANNY_THRESHOLD_MAX;
    if ((uint16_t) (visionParams.legacyCannyLow + VISION_LEGACY_CANNY_THRESHOLD_GAP) > visionParams.legacyCannyHigh)
    {
        if (visionParams.legacyCannyHigh > (uint16_t) (VISION_LEGACY_CANNY_THRESHOLD_MIN + VISION_LEGACY_CANNY_THRESHOLD_GAP))
        {
            visionParams.legacyCannyLow = (uint16_t) (visionParams.legacyCannyHigh - VISION_LEGACY_CANNY_THRESHOLD_GAP);
        }
    }
}

```

```

else
{
    visionParams.legacyCannyLow = VISION_LEGACY_CANNY_THRESHOLD_MIN;
    visionParams.legacyCannyHigh = (uint16_t) (VISION_LEGACY_CANNY_THRESHOLD_MIN + VISION_LEGACY_CANNY_THRESHOLD_GAP);
}
}

if (visionParams.beaconSizeMin < 1u) visionParams.beaconSizeMin = 1u;
if (visionParams.beaconSizeMin > 59999u) visionParams.beaconSizeMin = 59999u;
if (visionParams.beaconSizeMax <= visionParams.beaconSizeMin)
    visionParams.beaconSizeMax = (uint16_t) (visionParams.beaconSizeMin + 1u);
if (visionParams.beaconSizeMax > 60000u) visionParams.beaconSizeMax = 60000u;
visionParams.beaconEccMax = sanitizeFloat(visionParams.beaconEccMax,
    VISION_BEACON_ECC_MAX_DEFAULT, 0.0f, 20.0f);
if (visionParams.beaconMaxEndpoints > 200u) visionParams.beaconMaxEndpoints = 200u;
visionParams.beaconFillMin = sanitizeFloat(visionParams.beaconFillMin,
    VISION_BEACON_FILL_MIN_DEFAULT, 0.0f, 1.0f);
visionParams.beaconHuMax = sanitizeFloat(visionParams.beaconHuMax,
    VISION_BEACON_HU_MAX_DEFAULT, VISION_BEACON_HU_MAX_MIN, VISION_BEACON_HU_MAX_MAX);
visionParams.beaconHuAreaMin = (uint16_t) clampf((float) visionParams.beaconHuAreaMin,
    VISION_BEACON_HU_AREA_MIN_MIN, VISION_BEACON_HU_AREA_MIN_MAX);

visionParams.trackMaxAssocDist = sanitizeFloat(visionParams.trackMaxAssocDist,
    VISION_TRACK_MAX_ASSOC_DIST_DEFAULT, 1.0f, 100.0f);
visionParams.trackSizeRatioMax = sanitizeFloat(visionParams.trackSizeRatioMax,
    VISION_TRACK_SIZE_RATIO_MAX_DEFAULT, 1.1f, 10.0f);
visionParams.trackConfirmHits = (uint16_t) clampf((float) visionParams.trackConfirmHits, 1.0f, 60.0f);
visionParams.trackDropMisses = (uint16_t) clampf((float) visionParams.trackDropMisses, 1.0f, 60.0f);
visionParams.trackCoastMisses = (uint16_t) clampf((float) visionParams.trackCoastMisses, 1.0f, 60.0f);
visionParams.trackPositionAlpha = sanitizeFloat(visionParams.trackPositionAlpha,
    VISION_TRACK_POSITION_ALPHA_DEFAULT, VISION_TRACK_ALPHA_MIN, VISION_TRACK_ALPHA_MAX);
visionParams.trackSizeAlpha = sanitizeFloat(visionParams.trackSizeAlpha,
    VISION_TRACK_SIZE_ALPHA_DEFAULT, VISION_TRACK_ALPHA_MIN, VISION_TRACK_ALPHA_MAX);
visionParams.planningEdgePredictionHoldMs = (uint16_t) clampf(
    (float) visionParams.planningEdgePredictionHoldMs,
    VISION_TRACK_EDGE_PREDICTION_HOLD_MS_MIN,
    VISION_TRACK_EDGE_PREDICTION_HOLD_MS_MAX);
visionParams.c1EdgeBandLeft = (uint8_t) clampf((float) visionParams.c1EdgeBandLeft, 0.0f, VISION_TRACK_EDGE_BAND_X_MAX);
visionParams.c1EdgeBandRight = (uint8_t) clampf((float) visionParams.c1EdgeBandRight, 0.0f, VISION_TRACK_EDGE_BAND_X_MAX);
visionParams.c1EdgeBandTop = (uint8_t) clampf((float) visionParams.c1EdgeBandTop, 0.0f, VISION_TRACK_EDGE_BAND_Y_MAX);
visionParams.c1EdgeBandBottom = (uint8_t) clampf((float) visionParams.c1EdgeBandBottom, 0.0f, VISION_TRACK_EDGE_BAND_Y_MAX);
visionParams.observerEdgeBandLeft = (uint8_t) clampf((float) visionParams.observerEdgeBandLeft, 0.0f, VISION_TRACK_EDGE_BAND_X_MAX);
visionParams.observerEdgeBandRight = (uint8_t) clampf((float) visionParams.observerEdgeBandRight, 0.0f, VISION_TRACK_EDGE_BAND_X_MAX);
visionParams.observerEdgeBandTop = (uint8_t) clampf((float) visionParams.observerEdgeBandTop, 0.0f, VISION_TRACK_EDGE_BAND_Y_MAX);
visionParams.observerEdgeBandBottom = (uint8_t) clampf((float) visionParams.observerEdgeBandBottom, 0.0f, VISION_TRACK_EDGE_BAND_Y_MAX);
visionParams.c4EdgeBandLeft = (uint8_t) clampf((float) visionParams.c4EdgeBandLeft, 0.0f, VISION_TRACK_EDGE_BAND_X_MAX);
visionParams.c4EdgeBandRight = (uint8_t) clampf((float) visionParams.c4EdgeBandRight, 0.0f, VISION_TRACK_EDGE_BAND_X_MAX);
visionParams.c4EdgeBandTop = (uint8_t) clampf((float) visionParams.c4EdgeBandTop, 0.0f, VISION_TRACK_EDGE_BAND_Y_MAX);
visionParams.c4EdgeBandBottom = (uint8_t) clampf((float) visionParams.c4EdgeBandBottom, 0.0f, VISION_TRACK_EDGE_BAND_Y_MAX);

visionParams.planningKpVx = sanitizeFloat(visionParams.planningKpVx,
    PLANNING_KP_VX_DEFAULT, PLANNING_KP_LINEAR_MIN, PLANNING_KP_LINEAR_MAX);
visionParams.planningKpVy = sanitizeFloat(visionParams.planningKpVy,
    PLANNING_KP_VY_DEFAULT, PLANNING_KP_LINEAR_MIN, PLANNING_KP_LINEAR_MAX);
visionParams.core3PlanningKpVx = sanitizeFloat(visionParams.core3PlanningKpVx,
    CORE3_PLANNING_KP_VX_DEFAULT, PLANNING_KP_LINEAR_MIN, PLANNING_KP_LINEAR_MAX);
visionParams.core3PlanningKpVy = sanitizeFloat(visionParams.core3PlanningKpVy,
    CORE3_PLANNING_KP_VY_DEFAULT, PLANNING_KP_LINEAR_MIN, PLANNING_KP_LINEAR_MAX);
visionParams.core4PlanningKpVx = sanitizeFloat(visionParams.core4PlanningKpVx,
    CORE4_PLANNING_KP_VX_DEFAULT, PLANNING_KP_LINEAR_MIN, PLANNING_KP_LINEAR_MAX);
visionParams.core4PlanningKpVy = sanitizeFloat(visionParams.core4PlanningKpVy,
    CORE4_PLANNING_KP_VY_DEFAULT, PLANNING_KP_LINEAR_MIN, PLANNING_KP_LINEAR_MAX);
visionParams.planningVxMax = sanitizeFloat(visionParams.planningVxMax,
    PLANNING_VX_MAX_DEFAULT, PLANNING_V_LINEAR_LIMIT_MIN, PLANNING_V_LINEAR_LIMIT_MAX);
visionParams.planningVyMax = sanitizeFloat(visionParams.planningVyMax,
    PLANNING_VY_MAX_DEFAULT, PLANNING_V_LINEAR_LIMIT_MIN, PLANNING_V_LINEAR_LIMIT_MAX);
visionParams.planningWzMax = sanitizeFloat(visionParams.planningWzMax,
    PLANNING_WZ_MAX_DEFAULT, PLANNING_WZ_LIMIT_MIN, PLANNING_WZ_LIMIT_MAX);
visionParams.planningVxSlewPerS = sanitizeFloat(visionParams.planningVxSlewPerS,
    PLANNING_VX_SLEW_DEFAULT, PLANNING_SLEW_MIN, PLANNING_SLEW_MAX);
visionParams.planningVySlewPerS = sanitizeFloat(visionParams.planningVySlewPerS,
    PLANNING_VY_SLEW_DEFAULT, PLANNING_SLEW_MIN, PLANNING_SLEW_MAX);
visionParams.planningWzSlewPerS = sanitizeFloat(visionParams.planningWzSlewPerS,
    PLANNING_WZ_SLEW_DEFAULT, PLANNING_SLEW_MIN, PLANNING_SLEW_MAX);
visionParams.planningVxMin = sanitizeFloat(visionParams.planningVxMin,
    PLANNING_VX_MIN_DEFAULT, 0.0f, visionParams.planningVxMax);
visionParams.planningVyMin = sanitizeFloat(visionParams.planningVyMin,
    PLANNING_VY_MIN_DEFAULT, 0.0f, visionParams.planningVyMax);
visionParams.planningWzMin = sanitizeFloat(visionParams.planningWzMin,

```

```

    PLANNING_WZ_MIN_DEFAULT, 0.0f, visionParams.planningWzMax);
visionParams.planningFirstBeaconForwardSpeed = sanitizeFloat(
    visionParams.planningFirstBeaconForwardSpeed,
    PLANNING_FIRST_BEACON_FORWARD_SPEED_DEFAULT,
    PLANNING_FIRST_BEACON_FORWARD_SPEED_MIN,
    visionParams.planningVxMax);
visionParams.planningPostTakeoffDelayMs = (uint16_t) clampf(
    (float) visionParams.planningPostTakeoffDelayMs,
    PLANNING_POST_TAKEOFF_DELAY_MS_MIN,
    PLANNING_POST_TAKEOFF_DELAY_MS_MAX);
visionParams.planningFirstBeaconForwardEnable =
    (visionParams.planningFirstBeaconForwardEnable != 0u) ? 1u : 0u;
/* deadzone: 先钳 enter、后钳 exit (exit 上界依赖已钳位的 enter, 保证 exit <= enter - GAP_MIN), 顺序不可换。 */
visionParams.planningDeadzoneEnterPx = sanitizeFloat(
    visionParams.planningDeadzoneEnterPx, PLANNING_DEADZONE_ENTER_DEFAULT,
    PLANNING_DEADZONE_ENTER_MIN, PLANNING_DEADZONE_ENTER_MAX);
visionParams.planningDeadzoneExitPx = sanitizeFloat(
    visionParams.planningDeadzoneExitPx, PLANNING_DEADZONE_EXIT_DEFAULT,
    PLANNING_DEADZONE_EXIT_MIN,
    visionParams.planningDeadzoneEnterPx - PLANNING_DEADZONE_GAP_MIN);
visionParams.planningCore0PoseMinConfidence = sanitizeFloat(visionParams.planningCore0PoseMinConfidence,
    PLANNING_CORE0_POSE_MIN_CONF_DEFAULT,
    PLANNING_CORE0_POSE_MIN_CONF_MIN,
    PLANNING_CORE0_POSE_MIN_CONF_MAX);
visionParams.c1Core0PoseOffsetX = sanitizeFloat(visionParams.c1Core0PoseOffsetX,
    PLANNING_CORE0_POSE_OFFSET_X_DEFAULT,
    PLANNING_CORE0_POSE_OFFSET_X_MIN,
    PLANNING_CORE0_POSE_OFFSET_X_MAX);
visionParams.c1Core0PoseOffsetY = sanitizeFloat(visionParams.c1Core0PoseOffsetY,
    PLANNING_CORE0_POSE_OFFSET_Y_DEFAULT,
    PLANNING_CORE0_POSE_OFFSET_Y_MIN,
    PLANNING_CORE0_POSE_OFFSET_Y_MAX);
visionParams.planningCore0PoseHeadingOffsetRad = sanitizeFloat(visionParams.planningCore0PoseHeadingOffsetRad,
    PLANNING_CORE0_POSE_HEADING_OFFSET_DEFAULT,
    PLANNING_CORE0_POSE_HEADING_OFFSET_MIN,
    PLANNING_CORE0_POSE_HEADING_OFFSET_MAX);
visionParams.planningCore0PoseHeadingSign =
    (visionParams.planningCore0PoseHeadingSign >= 0) ? (int8_t) 1 : (int8_t) -1;
visionParams.poseHoldMs = (uint16_t) clampf((float) visionParams.poseHoldMs,
    PLANNING_CORE0_POSE_HOLD_MS_MIN, PLANNING_CORE0_POSE_HOLD_MS_MAX);

/* 离散极性 sign: 仅 -1/+1 (>=0 归 +1) 。 */
visionParams.planningVxSign = (visionParams.planningVxSign >= 0) ? (int8_t) 1 : (int8_t) -1;
visionParams.planningVySign = (visionParams.planningVySign >= 0) ? (int8_t) 1 : (int8_t) -1;
visionParams.planningWzSign = (visionParams.planningWzSign >= 0) ? (int8_t) 1 : (int8_t) -1;

/* 派发开关归一化 0/1; 派发超时钳到范围。 */
visionParams.planningDispatchEnable = (visionParams.planningDispatchEnable != 0u) ? 1u : 0u;
visionParams.planningSearchEnable = (visionParams.planningSearchEnable != 0u) ? 1u : 0u;
visionParams.planningPlanTimeoutMs = (uint16_t) clampf((float) visionParams.planningPlanTimeoutMs,
    PLANNING_PLAN_TIMEOUT_MS_MIN, PLANNING_PLAN_TIMEOUT_MS_MAX);
visionParams.planningSearchWz = sanitizeFloat(visionParams.planningSearchWz,
    PLANNING_SEARCH_WZ_DEFAULT, PLANNING_SEARCH_WZ_MIN, PLANNING_SEARCH_WZ_MAX);
visionParams.planningSearchTimeoutMs = (uint16_t) clampf((float) visionParams.planningSearchTimeoutMs,
    PLANNING_SEARCH_TIMEOUT_MS_MIN, PLANNING_SEARCH_TIMEOUT_MS_MAX);

/* 输出 TD: NaN/Inf 回默认再 clamp。 */
visionParams.planningTdRvx = sanitizeFloat(visionParams.planningTdRvx,
    PLANNING_TD_RVX_DEFAULT, PLANNING_TD_R_MIN, PLANNING_TD_R_MAX);
visionParams.planningTdRvy = sanitizeFloat(visionParams.planningTdRvy,
    PLANNING_TD_RVY_DEFAULT, PLANNING_TD_R_MIN, PLANNING_TD_R_MAX);
visionParams.planningTdRwz = sanitizeFloat(visionParams.planningTdRwz,
    PLANNING_TD_RWZ_DEFAULT, PLANNING_TD_R_MIN, PLANNING_TD_R_MAX);
visionParams.planningTdN = sanitizeFloat(visionParams.planningTdN,
    PLANNING_TD_N_DEFAULT, PLANNING_TD_N_MIN, PLANNING_TD_N_MAX);
visionParams.planningTdVxEnable = (visionParams.planningTdVxEnable != 0u) ? 1u : 0u;
visionParams.planningTdVyEnable = (visionParams.planningTdVyEnable != 0u) ? 1u : 0u;
visionParams.planningTdWzEnable = (visionParams.planningTdWzEnable != 0u) ? 1u : 0u;

visionParams.trackAreaVelAlpha = sanitizeFloat(visionParams.trackAreaVelAlpha,
    VISION_TRACK_AREA_VEL_ALPHA_DEFAULT, VISION_TRACK_ALPHA_MIN, VISION_TRACK_ALPHA_MAX);
visionParams.trackAssocScoreMax = sanitizeFloat(visionParams.trackAssocScoreMax,
    VISION_TRACK_ASSOC_SCORE_MAX_DEFAULT,
    VISION_TRACK_ASSOC_SCORE_MAX_MIN,
    VISION_TRACK_ASSOC_SCORE_MAX_MAX);
visionParams.trackAssocAreaWeight = sanitizeFloat(visionParams.trackAssocAreaWeight,
    VISION_TRACK_ASSOC_AREA_WEIGHT_DEFAULT,
    VISION_TRACK_ASSOC_WEIGHT_MIN,
    VISION_TRACK_ASSOC_WEIGHT_MAX);

```



```

visionParams.trackAssocAreaVelWeight = sanitizeFloat(visionParams.trackAssocAreaVelWeight,
    VISION_TRACK_ASSOC_AREA_VEL_WEIGHT_DEFAULT,
    VISION_TRACK_ASSOC_WEIGHT_MIN,
    VISION_TRACK_ASSOC_WEIGHT_MAX);
if (visionParams.sharedConfigRevision == 0u) visionParams.sharedConfigRevision = 1u;
visionParams.core3CenterX = sanitizeFloat(visionParams.core3CenterX,
    CORE3_CALIBRATION_CENTER_X_DEFAULT,
    CORE3_CALIBRATION_CENTER_X_MIN, CORE3_CALIBRATION_CENTER_X_MAX);
visionParams.core3CenterY = sanitizeFloat(visionParams.core3CenterY,
    CORE3_CALIBRATION_CENTER_Y_DEFAULT,
    CORE3_CALIBRATION_CENTER_Y_MIN, CORE3_CALIBRATION_CENTER_Y_MAX);
visionParams.core3Scale = sanitizeFloat(visionParams.core3Scale,
    CORE3_CALIBRATION_SCALE_DEFAULT,
    CORE3_CALIBRATION_SCALE_MIN, CORE3_CALIBRATION_SCALE_MAX);
visionParams.core3MountYawRad = sanitizeFloat(visionParams.core3MountYawRad,
    CORE3_CALIBRATION_YAW_RAD_DEFAULT,
    CORE3_CALIBRATION_YAW_RAD_MIN, CORE3_CALIBRATION_YAW_RAD_MAX);
visionParams.core3ObservationTimeoutMs = (uint16_t) clampf(
    (float) visionParams.core3ObservationTimeoutMs,
    (float) CORE3_OBSERVATION_TIMEOUT_MS_MIN,
    (float) CORE3_OBSERVATION_TIMEOUT_MS_MAX);
if (visionParams.maxBeacons < BEACON_MAX_BEACONS_MIN)
    visionParams.maxBeacons = BEACON_MAX_BEACONS_MIN;
if (visionParams.maxBeacons > BEACON_MAX_BEACONS_MAX)
    visionParams.maxBeacons = BEACON_MAX_BEACONS_MAX;
visionParams.core3MirrorX = (visionParams.core3MirrorX != 0u) ? 1u : 0u;
visionParams.core3ErrorFwdSign = (visionParams.core3ErrorFwdSign >= 0) ? 1 : -1;
visionParams.core3ErrorRightSign = (visionParams.core3ErrorRightSign >= 0) ? 1 : -1;
sanitizeBorderPair(&visionParams.c1BorderEnterLeft, &visionParams.c1BorderExitLeft,
    VISION_TRACK_BORDER_X_MAX,
    VISION_TRACK_BORDER_ENTER_LEFT_DEFAULT, VISION_TRACK_BORDER_EXIT_LEFT_DEFAULT);
sanitizeBorderPair(&visionParams.c1BorderEnterRight, &visionParams.c1BorderExitRight,
    VISION_TRACK_BORDER_X_MAX,
    VISION_TRACK_BORDER_ENTER_RIGHT_DEFAULT, VISION_TRACK_BORDER_EXIT_RIGHT_DEFAULT);
sanitizeBorderPair(&visionParams.c1BorderEnterTop, &visionParams.c1BorderExitTop,
    VISION_TRACK_BORDER_Y_MAX,
    VISION_TRACK_BORDER_ENTER_TOP_DEFAULT, VISION_TRACK_BORDER_EXIT_TOP_DEFAULT);
sanitizeBorderPair(&visionParams.c1BorderEnterBottom, &visionParams.c1BorderExitBottom,
    VISION_TRACK_BORDER_Y_MAX,
    VISION_TRACK_BORDER_ENTER_BOTTOM_DEFAULT, VISION_TRACK_BORDER_EXIT_BOTTOM_DEFAULT);
sanitizeBorderPair(&visionParams.observerBorderEnterLeft, &visionParams.observerBorderExitLeft,
    VISION_TRACK_BORDER_X_MAX,
    VISION_TRACK_CORE3_BORDER_ENTER_LEFT_DEFAULT, VISION_TRACK_CORE3_BORDER_EXIT_LEFT_DEFAULT);
sanitizeBorderPair(&visionParams.observerBorderEnterRight, &visionParams.observerBorderExitRight,
    VISION_TRACK_BORDER_X_MAX,
    VISION_TRACK_CORE3_BORDER_ENTER_RIGHT_DEFAULT, VISION_TRACK_CORE3_BORDER_EXIT_RIGHT_DEFAULT);
sanitizeBorderPair(&visionParams.observerBorderEnterTop, &visionParams.observerBorderExitTop,
    VISION_TRACK_BORDER_Y_MAX,
    VISION_TRACK_CORE3_BORDER_ENTER_TOP_DEFAULT, VISION_TRACK_CORE3_BORDER_EXIT_TOP_DEFAULT);
sanitizeBorderPair(&visionParams.observerBorderEnterBottom, &visionParams.observerBorderExitBottom,
    VISION_TRACK_BORDER_Y_MAX,
    VISION_TRACK_CORE3_BORDER_ENTER_BOTTOM_DEFAULT, VISION_TRACK_CORE3_BORDER_EXIT_BOTTOM_DEFAULT);
visionParams.core3Core0PoseOffsetX = sanitizeFloat(
    visionParams.core3Core0PoseOffsetX,
    CORE3_CORE0_POSE_OFFSET_X_DEFAULT,
    PLANNING_CORE0_POSE_OFFSET_X_MIN,
    PLANNING_CORE0_POSE_OFFSET_X_MAX);
visionParams.core3Core0PoseOffsetY = sanitizeFloat(
    visionParams.core3Core0PoseOffsetY,
    CORE3_CORE0_POSE_OFFSET_Y_DEFAULT,
    PLANNING_CORE0_POSE_OFFSET_Y_MIN,
    PLANNING_CORE0_POSE_OFFSET_Y_MAX);
visionParams.core3PlanningVxSign =
    (visionParams.core3PlanningVxSign >= 0) ? (int8_t) 1 : (int8_t) -1;
visionParams.core3PlanningVySign =
    (visionParams.core3PlanningVySign >= 0) ? (int8_t) 1 : (int8_t) -1;

visionParams.c1FreshGateMs = (uint16_t) clampf((float) visionParams.c1FreshGateMs,
    (float) SOURCE_FRESH_GATE_MS_MIN, (float) SOURCE_FRESH_GATE_MS_MAX);
visionParams.c3FreshGateMs = (uint16_t) clampf((float) visionParams.c3FreshGateMs,
    (float) SOURCE_FRESH_GATE_MS_MIN, (float) SOURCE_FRESH_GATE_MS_MAX);
visionParams.c4FreshGateMs = (uint16_t) clampf((float) visionParams.c4FreshGateMs,
    (float) SOURCE_FRESH_GATE_MS_MIN, (float) SOURCE_FRESH_GATE_MS_MAX);
visionParams.c4ObservationTimeoutMs = (uint16_t) clampf(
    (float) visionParams.c4ObservationTimeoutMs,
    (float) CORE4_OBSERVATION_TIMEOUT_MS_MIN,
    (float) CORE4_OBSERVATION_TIMEOUT_MS_MAX);
visionParams.core4CenterX = sanitizeFloat(visionParams.core4CenterX,
    CORE4_CALIBRATION_CENTER_X_DEFAULT,

```

```

    CORE3_CALIBRATION_CENTER_X_MIN, CORE3_CALIBRATION_CENTER_X_MAX);
visionParams.core4CenterY = sanitizeFloat(visionParams.core4CenterY,
    CORE4_CALIBRATION_CENTER_Y_DEFAULT,
    CORE3_CALIBRATION_CENTER_Y_MIN, CORE3_CALIBRATION_CENTER_Y_MAX);
visionParams.core4Scale = sanitizeFloat(visionParams.core4Scale,
    CORE4_CALIBRATION_SCALE_DEFAULT,
    CORE3_CALIBRATION_SCALE_MIN, CORE3_CALIBRATION_SCALE_MAX);
visionParams.core4MountYawRad = sanitizeFloat(visionParams.core4MountYawRad,
    CORE4_CALIBRATION_YAW_RAD_DEFAULT,
    CORE3_CALIBRATION_YAW_RAD_MIN, CORE3_CALIBRATION_YAW_RAD_MAX);
visionParams.core4MirrorX = (visionParams.core4MirrorX != 0u) ? 1u : 0u;
visionParams.core4ErrorFwdSign = (visionParams.core4ErrorFwdSign >= 0) ? 1 : -1;
visionParams.core4ErrorRightSign = (visionParams.core4ErrorRightSign >= 0) ? 1 : -1;
sanitizeBorderPair(&visionParams.c4BorderEnterLeft, &visionParams.c4BorderExitLeft,
    VISION_TRACK_BORDER_X_MAX,
    VISION_TRACK_CORE4_BORDER_ENTER_LEFT_DEFAULT, VISION_TRACK_CORE4_BORDER_EXIT_LEFT_DEFAULT);
sanitizeBorderPair(&visionParams.c4BorderEnterRight, &visionParams.c4BorderExitRight,
    VISION_TRACK_BORDER_X_MAX,
    VISION_TRACK_CORE4_BORDER_ENTER_RIGHT_DEFAULT, VISION_TRACK_CORE4_BORDER_EXIT_RIGHT_DEFAULT);
sanitizeBorderPair(&visionParams.c4BorderEnterTop, &visionParams.c4BorderExitTop,
    VISION_TRACK_BORDER_Y_MAX,
    VISION_TRACK_CORE4_BORDER_ENTER_TOP_DEFAULT, VISION_TRACK_CORE4_BORDER_EXIT_TOP_DEFAULT);
sanitizeBorderPair(&visionParams.c4BorderEnterBottom, &visionParams.c4BorderExitBottom,
    VISION_TRACK_BORDER_Y_MAX,
    VISION_TRACK_CORE4_BORDER_ENTER_BOTTOM_DEFAULT, VISION_TRACK_CORE4_BORDER_EXIT_BOTTOM_DEFAULT);
visionParams.core4Core0PoseOffsetX = sanitizeFloat(
    visionParams.core4Core0PoseOffsetX, CORE4_CORE0_POSE_OFFSET_X_DEFAULT,
    PLANNING_CORE0_POSE_OFFSET_X_MIN, PLANNING_CORE0_POSE_OFFSET_X_MAX);
visionParams.core4Core0PoseOffsetY = sanitizeFloat(
    visionParams.core4Core0PoseOffsetY, CORE4_CORE0_POSE_OFFSET_Y_DEFAULT,
    PLANNING_CORE0_POSE_OFFSET_Y_MIN, PLANNING_CORE0_POSE_OFFSET_Y_MAX);
visionParams.core4PlanningVxSign =
    (visionParams.core4PlanningVxSign >= 0) ? (int8_t) 1 : (int8_t) -1;
visionParams.core4PlanningVySign =
    (visionParams.core4PlanningVySign >= 0) ? (int8_t) 1 : (int8_t) -1;
}

void visionParamsLoadDefaults(void)
{
    memset(&visionParams, 0, sizeof(visionParams));
    visionParams.detectMode = VISION_DETECT_MODE_DEFAULT;
    visionParams.cannyLow = VISION_CANNY_LOW_DEFAULT;
    visionParams.cannyHigh = VISION_CANNY_HIGH_DEFAULT;
    visionParams.exposure = VISION_EXPOSURE_DEFAULT;
    visionParams.beaconSizeMin = VISION_BEACON_SIZE_MIN_DEFAULT;
    visionParams.beaconSizeMax = VISION_BEACON_SIZE_MAX_DEFAULT;
    visionParams.beaconEccMax = VISION_BEACON_ECC_MAX_DEFAULT;
    visionParams.beaconBrightnessMin = VISION_BEACON_BRIGHTNESS_MIN_DEFAULT;
    visionParams.beaconMaxEndpoints = VISION_BEACON_MAX_ENDPOINTS_DEFAULT;
    visionParams.trackMaxAssocDist = VISION_TRACK_MAX_ASSOC_DIST_DEFAULT;
    visionParams.trackSizeRatioMax = VISION_TRACK_SIZE_RATIO_MAX_DEFAULT;
    visionParams.trackConfirmHits = VISION_TRACK_CONFIRM_HITS_DEFAULT;
    visionParams.trackDropMisses = VISION_TRACK_DROP_MISSES_DEFAULT;
    visionParams.trackCoastMisses = VISION_TRACK_COAST_MISSES_DEFAULT;
    visionParams.trackPositionAlpha = VISION_TRACK_POSITION_ALPHA_DEFAULT;
    visionParams.trackSizeAlpha = VISION_TRACK_SIZE_ALPHA_DEFAULT;
    visionParams.planningEdgePredictionHoldMs = VISION_TRACK_EDGE_PREDICTION_HOLD_MS_DEFAULT;
    visionParams.c1EdgeBandLeft = VISION_TRACK_EDGE_BAND_C1_LEFT_DEFAULT;
    visionParams.c1EdgeBandRight = VISION_TRACK_EDGE_BAND_C1_RIGHT_DEFAULT;
    visionParams.c1EdgeBandTop = VISION_TRACK_EDGE_BAND_C1_TOP_DEFAULT;
    visionParams.c1EdgeBandBottom = VISION_TRACK_EDGE_BAND_C1_BOTTOM_DEFAULT;
    visionParams.observerEdgeBandLeft = VISION_TRACK_EDGE_BAND_C3_LEFT_DEFAULT;
    visionParams.observerEdgeBandRight = VISION_TRACK_EDGE_BAND_C3_RIGHT_DEFAULT;
    visionParams.observerEdgeBandTop = VISION_TRACK_EDGE_BAND_C3_TOP_DEFAULT;
    visionParams.observerEdgeBandBottom = VISION_TRACK_EDGE_BAND_C3_BOTTOM_DEFAULT;
    visionParams.planningKpVx = PLANNING_KP_VX_DEFAULT;
    visionParams.planningKpVy = PLANNING_KP_VY_DEFAULT;
    visionParams.core3PlanningKpVx = CORE3_PLANNING_KP_VX_DEFAULT;
    visionParams.core3PlanningKpVy = CORE3_PLANNING_KP_VY_DEFAULT;
    visionParams.planningVxMax = PLANNING_VX_MAX_DEFAULT;
    visionParams.planningVyMax = PLANNING_VY_MAX_DEFAULT;
    visionParams.planningWzMax = PLANNING_WZ_MAX_DEFAULT;
    visionParams.planningVxSlewPerS = PLANNING_VX_SLEW_DEFAULT;
    visionParams.planningVySlewPerS = PLANNING_VY_SLEW_DEFAULT;
    visionParams.planningWzSlewPerS = PLANNING_WZ_SLEW_DEFAULT;
    visionParams.planningVxMin = PLANNING_VX_MIN_DEFAULT;
    visionParams.planningVyMin = PLANNING_VY_MIN_DEFAULT;
    visionParams.planningWzMin = PLANNING_WZ_MIN_DEFAULT;
    visionParams.planningDeadzoneEnterPx = PLANNING_DEADZONE_ENTER_DEFAULT;

```

```

visionParams.planningDeadzoneExitPx = PLANNING_DEADZONE_EXIT_DEFAULT;
visionParams.planningCore0PoseMinConfidence = PLANNING_CORE0_POSE_MIN_CONF_DEFAULT;
visionParams.c1Core0PoseOffsetX = PLANNING_CORE0_POSE_OFFSET_X_DEFAULT;
visionParams.c1Core0PoseOffsetY = PLANNING_CORE0_POSE_OFFSET_Y_DEFAULT;
visionParams.planningCore0PoseHeadingOffsetRad = PLANNING_CORE0_POSE_HEADING_OFFSET_DEFAULT;
visionParams.planningCore0PoseHeadingSign = (int8_t) PLANNING_CORE0_POSE_HEADING_SIGN_DEFAULT;
visionParams.poseHoldMs = PLANNING_CORE0_POSE_HOLD_MS_DEFAULT;
visionParams.planningVxSign = (int8_t) PLANNING_VX_SIGN_DEFAULT;
visionParams.planningVySign = (int8_t) PLANNING_VY_SIGN_DEFAULT;
visionParams.planningWzSign = (int8_t) PLANNING_WZ_SIGN_DEFAULT;
visionParams.planningDispatchEnable = PLANNING_DISPATCH_ENABLE_DEFAULT;
visionParams.planningSearchEnable = PLANNING_SEARCH_ENABLE_DEFAULT;
visionParams.planningPlanTimeoutMs = PLANNING_PLAN_TIMEOUT_MS_DEFAULT;
visionParams.planningSearchWz = PLANNING_SEARCH_WZ_DEFAULT;
visionParams.planningSearchTimeoutMs = PLANNING_SEARCH_TIMEOUT_MS_DEFAULT;
visionParams.planningTdRvx = PLANNING_TD_RVX_DEFAULT;
visionParams.planningTdRvy = PLANNING_TD_RVY_DEFAULT;
visionParams.planningTdRwz = PLANNING_TD_RWZ_DEFAULT;
visionParams.planningTdN = PLANNING_TD_N_DEFAULT;
visionParams.planningTdVxEnable = PLANNING_TD_VX_ENABLE_DEFAULT;
visionParams.planningTdVyEnable = PLANNING_TD_VY_ENABLE_DEFAULT;
visionParams.planningTdWzEnable = PLANNING_TD_WZ_ENABLE_DEFAULT;
visionParams.trackAreaVelAlpha = VISION_TRACK_AREA_VEL_ALPHA_DEFAULT;
visionParams.trackAssocScoreMax = VISION_TRACK_ASSOC_SCORE_MAX_DEFAULT;
visionParams.trackAssocAreaWeight = VISION_TRACK_ASSOC_AREA_WEIGHT_DEFAULT;
visionParams.trackAssocAreaVelWeight = VISION_TRACK_ASSOC_AREA_VEL_WEIGHT_DEFAULT;
visionParams.beaconFillMin = VISION_BEACON_FILL_MIN_DEFAULT;
visionParams.beaconHuMax = VISION_BEACON_HU_MAX_DEFAULT;
visionParams.beaconHuAreaMin = VISION_BEACON_HU_AREA_MIN_DEFAULT;
visionParams.legacyCannyLow = VISION_LEGACY_CANNY_LOW_DEFAULT;
visionParams.legacyCannyHigh = VISION_LEGACY_CANNY_HIGH_DEFAULT;
visionParams.sharedConfigRevision = 1u;
visionParams.sharedConfigHash = 0u;
visionParams.core3CenterX = CORE3_CALIBRATION_CENTER_X_DEFAULT;
visionParams.core3CenterY = CORE3_CALIBRATION_CENTER_Y_DEFAULT;
visionParams.core3Scale = CORE3_CALIBRATION_SCALE_DEFAULT;
visionParams.core3MountYawRad = CORE3_CALIBRATION_YAW_RAD_DEFAULT;
visionParams.core3ObservationTimeoutMs = CORE3_OBSERVATION_TIMEOUT_MS_DEFAULT;
visionParams.maxBeacons = BEACON_MAX_BEACONS_DEFAULT;
visionParams.core3MirrorX = CORE3_CALIBRATION_MIRROR_X_DEFAULT;
visionParams.core3ErrorFwdSign = CORE3_CALIBRATION_FWD_SIGN_DEFAULT;
visionParams.core3ErrorRightSign = CORE3_CALIBRATION_RIGHT_SIGN_DEFAULT;
visionParams.c1BorderEnterLeft = VISION_TRACK_BORDER_ENTER_LEFT_DEFAULT;
visionParams.c1BorderEnterRight = VISION_TRACK_BORDER_ENTER_RIGHT_DEFAULT;
visionParams.c1BorderEnterTop = VISION_TRACK_BORDER_ENTER_TOP_DEFAULT;
visionParams.c1BorderEnterBottom = VISION_TRACK_BORDER_ENTER_BOTTOM_DEFAULT;
visionParams.c1BorderExitLeft = VISION_TRACK_BORDER_EXIT_LEFT_DEFAULT;
visionParams.c1BorderExitRight = VISION_TRACK_BORDER_EXIT_RIGHT_DEFAULT;
visionParams.c1BorderExitTop = VISION_TRACK_BORDER_EXIT_TOP_DEFAULT;
visionParams.c1BorderExitBottom = VISION_TRACK_BORDER_EXIT_BOTTOM_DEFAULT;
visionParams.observerBorderEnterLeft = VISION_TRACK_CORE3_BORDER_ENTER_LEFT_DEFAULT;
visionParams.observerBorderEnterRight = VISION_TRACK_CORE3_BORDER_ENTER_RIGHT_DEFAULT;
visionParams.observerBorderEnterTop = VISION_TRACK_CORE3_BORDER_ENTER_TOP_DEFAULT;
visionParams.observerBorderEnterBottom = VISION_TRACK_CORE3_BORDER_ENTER_BOTTOM_DEFAULT;
visionParams.observerBorderExitLeft = VISION_TRACK_CORE3_BORDER_EXIT_LEFT_DEFAULT;
visionParams.observerBorderExitRight = VISION_TRACK_CORE3_BORDER_EXIT_RIGHT_DEFAULT;
visionParams.observerBorderExitTop = VISION_TRACK_CORE3_BORDER_EXIT_TOP_DEFAULT;
visionParams.observerBorderExitBottom = VISION_TRACK_CORE3_BORDER_EXIT_BOTTOM_DEFAULT;
visionParams.core3Core0PoseOffsetX = CORE3_CORE0_POSE_OFFSET_X_DEFAULT;
visionParams.core3Core0PoseOffsetY = CORE3_CORE0_POSE_OFFSET_Y_DEFAULT;
visionParams.core3PlanningVxSign = (int8_t) CORE3_PLANNING_VX_SIGN_DEFAULT;
visionParams.core3PlanningVySign = (int8_t) CORE3_PLANNING_VY_SIGN_DEFAULT;
visionParams.planningFirstBeaconForwardSpeed =
    PLANNING_FIRST_BEACON_FORWARD_SPEED_DEFAULT;
visionParams.planningPostTakeoffDelayMs =
    PLANNING_POST_TAKEOFF_DELAY_MS_DEFAULT;
visionParams.planningFirstBeaconForwardEnable =
    PLANNING_FIRST_BEACON_FORWARD_ENABLE_DEFAULT;
visionParams.c1FreshGateMs = C1_SOURCE_FRESH_GATE_MS_DEFAULT;
visionParams.c3FreshGateMs = C3_SOURCE_FRESH_GATE_MS_DEFAULT;
visionParams.c4FreshGateMs = C4_SOURCE_FRESH_GATE_MS_DEFAULT;
visionParams.c4ObservationTimeoutMs = CORE4_OBSERVATION_TIMEOUT_MS_DEFAULT;
visionParams.c4EdgeBandLeft = VISION_TRACK_EDGE_BAND_C4_LEFT_DEFAULT;
visionParams.c4EdgeBandRight = VISION_TRACK_EDGE_BAND_C4_RIGHT_DEFAULT;
visionParams.c4EdgeBandTop = VISION_TRACK_EDGE_BAND_C4_TOP_DEFAULT;
visionParams.c4EdgeBandBottom = VISION_TRACK_EDGE_BAND_C4_BOTTOM_DEFAULT;
visionParams.core4PlanningKpVx = CORE4_PLANNING_KP_VX_DEFAULT;
visionParams.core4PlanningKpVy = CORE4_PLANNING_KP_VY_DEFAULT;
visionParams.core4CenterX = CORE4_CALIBRATION_CENTER_X_DEFAULT;

```

```

visionParams.core4CenterY = CORE4_CALIBRATION_CENTER_Y_DEFAULT;
visionParams.core4Scale = CORE4_CALIBRATION_SCALE_DEFAULT;
visionParams.core4MountYawRad = CORE4_CALIBRATION_YAW_RAD_DEFAULT;
visionParams.core4MirrorX = CORE4_CALIBRATION_MIRROR_X_DEFAULT;
visionParams.core4ErrorFwdSign = CORE4_CALIBRATION_FWD_SIGN_DEFAULT;
visionParams.core4ErrorRightSign = CORE4_CALIBRATION_RIGHT_SIGN_DEFAULT;
visionParams.c4BorderEnterLeft = VISION_TRACK_CORE4_BORDER_ENTER_LEFT_DEFAULT;
visionParams.c4BorderEnterRight = VISION_TRACK_CORE4_BORDER_ENTER_RIGHT_DEFAULT;
visionParams.c4BorderEnterTop = VISION_TRACK_CORE4_BORDER_ENTER_TOP_DEFAULT;
visionParams.c4BorderEnterBottom = VISION_TRACK_CORE4_BORDER_ENTER_BOTTOM_DEFAULT;
visionParams.c4BorderExitLeft = VISION_TRACK_CORE4_BORDER_EXIT_LEFT_DEFAULT;
visionParams.c4BorderExitRight = VISION_TRACK_CORE4_BORDER_EXIT_RIGHT_DEFAULT;
visionParams.c4BorderExitTop = VISION_TRACK_CORE4_BORDER_EXIT_TOP_DEFAULT;
visionParams.c4BorderExitBottom = VISION_TRACK_CORE4_BORDER_EXIT_BOTTOM_DEFAULT;
visionParams.core4Core0PoseOffsetX = CORE4_CORE0_POSE_OFFSET_X_DEFAULT;
visionParams.core4Core0PoseOffsetY = CORE4_CORE0_POSE_OFFSET_Y_DEFAULT;
visionParams.core4PlanningVxSign = (int8_t) CORE4_PLANNING_VX_SIGN_DEFAULT;
visionParams.core4PlanningVySign = (int8_t) CORE4_PLANNING_VY_SIGN_DEFAULT;
}

void visionParamsApply(void)
{
    static const float beaconHuTemplate[7] = {
        VISION_BEACON_HU_TEMPLATE_0,
        VISION_BEACON_HU_TEMPLATE_1,
        VISION_BEACON_HU_TEMPLATE_2,
        VISION_BEACON_HU_TEMPLATE_3,
        VISION_BEACON_HU_TEMPLATE_4,
        VISION_BEACON_HU_TEMPLATE_5,
        VISION_BEACON_HU_TEMPLATE_6};
    BEACONCC_CONFIG_STRUCT beaconCcConfig;
    beaconNavigatorParamViewStruct navigator;
    beaconTrackBorderConfigStruct c1Border;
    beaconTrackEdgePredictionConfigStruct c1Edge;

    visionParamsClamp();

    beaconCcConfig.thresholdLow = (uint8_t) visionParams.cannyLow;
    beaconCcConfig.thresholdHigh = (uint8_t) visionParams.cannyHigh;
    beaconCcConfig.areaMin = visionParams.beaconSizeMin;
    beaconCcConfig.areaMax = visionParams.beaconSizeMax;
    beaconCcConfig.brightnessMin = visionParams.beaconBrightnessMin;
    beaconCcConfig.eccMax = visionParams.beaconEccMax;
    beaconCcConfig.fillMin = visionParams.beaconFillMin;
    beaconCcConfig.huMax = visionParams.beaconHuMax;
    beaconCcConfig.huAreaMin = visionParams.beaconHuAreaMin;
    beaconCcSetConfig(&beaconCcConfig);
    beaconCcSetTemplate(beaconHuTemplate, visionParams.beaconHuMax);
    cannySetThreshold(visionParams.legacyCannyLow, visionParams.legacyCannyHigh);
    beaconDetectInit(visionParams.beaconSizeMin, visionParams.beaconSizeMax,
        visionParams.beaconEccMax, visionParams.beaconBrightnessMin,
        visionParams.beaconMaxEndpoints);
    imageSetDetectMode(visionParams.detectMode);

    /* track: 热重载跟踪门控参数 + 关联平滑 alpha。 */
    beaconTrackSetParams(visionParams.trackMaxAssocDist, visionParams.trackSizeRatioMax,
        visionParams.trackConfirmHits, visionParams.trackDropMisses,
        visionParams.trackCoastMisses,
        visionParams.trackPositionAlpha, visionParams.trackSizeAlpha,
        visionParams.trackAreaVelAlpha, visionParams.trackAssocScoreMax,
        visionParams.trackAssocAreaWeight, visionParams.trackAssocAreaVelWeight);
    c1Border.enter.left = visionParams.c1BorderEnterLeft;
    c1Border.enter.right = visionParams.c1BorderEnterRight;
    c1Border.enter.top = visionParams.c1BorderEnterTop;
    c1Border.enter.bottom = visionParams.c1BorderEnterBottom;
    c1Border.exit.left = visionParams.c1BorderExitLeft;
    c1Border.exit.right = visionParams.c1BorderExitRight;
    c1Border.exit.top = visionParams.c1BorderExitTop;
    c1Border.exit.bottom = visionParams.c1BorderExitBottom;
    beaconTrackSetBorderMargins(&c1Border);
    c1Edge.holdMs = visionParams.planningEdgePredictionHoldMs;
    c1Edge.bands.left = visionParams.c1EdgeBandLeft;
    c1Edge.bands.right = visionParams.c1EdgeBandRight;
    c1Edge.bands.top = visionParams.c1EdgeBandTop;
    c1Edge.bands.bottom = visionParams.c1EdgeBandBottom;
    beaconTrackSetEdgePredictionConfig(&c1Edge);

    navigator.planningKpVx = visionParams.planningKpVx;
    navigator.planningKpVy = visionParams.planningKpVy;

```

```
navigator.core3PlanningKpVx = visionParams.core3PlanningKpVx;
navigator.core3PlanningKpVy = visionParams.core3PlanningKpVy;
navigator.core4PlanningKpVx = visionParams.core4PlanningKpVx;
navigator.core4PlanningKpVy = visionParams.core4PlanningKpVy;
navigator.planningVxMax = visionParams.planningVxMax;
navigator.planningVyMax = visionParams.planningVyMax;
navigator.planningWzMax = visionParams.planningWzMax;
navigator.planningVxMin = visionParams.planningVxMin;
navigator.planningVyMin = visionParams.planningVyMin;
navigator.planningWzMin = visionParams.planningWzMin;
navigator.planningVxSlewPerS = visionParams.planningVxSlewPerS;
navigator.planningVySlewPerS = visionParams.planningVySlewPerS;
navigator.planningWzSlewPerS = visionParams.planningWzSlewPerS;
navigator.planningDeadzoneEnterPx = visionParams.planningDeadzoneEnterPx;
navigator.planningDeadzoneExitPx = visionParams.planningDeadzoneExitPx;
navigator.planningTdRvx = visionParams.planningTdRvx;
navigator.planningTdRvy = visionParams.planningTdRvy;
navigator.planningTdRwz = visionParams.planningTdRwz;
navigator.planningTdN = visionParams.planningTdN;
navigator.planningSearchWz = visionParams.planningSearchWz;
navigator.planningFirstBeaconForwardSpeed = visionParams.planningFirstBeaconForwardSpeed;
navigator.planningVxSign = visionParams.planningVxSign;
navigator.planningVySign = visionParams.planningVySign;
navigator.planningWzSign = visionParams.planningWzSign;
navigator.core3PlanningVxSign = visionParams.core3PlanningVxSign;
navigator.core3PlanningVySign = visionParams.core3PlanningVySign;
navigator.core4PlanningVxSign = visionParams.core4PlanningVxSign;
navigator.core4PlanningVySign = visionParams.core4PlanningVySign;
navigator.planningSearchEnable = visionParams.planningSearchEnable;
navigator.planningTdVxEnable = visionParams.planningTdVxEnable;
navigator.planningTdVyEnable = visionParams.planningTdVyEnable;
navigator.planningTdWzEnable = visionParams.planningTdWzEnable;
navigator.planningFirstBeaconForwardEnable = visionParams.planningFirstBeaconForwardEnable;
beaconNavigatorApplyParams(&navigator);

imageSetExposure(visionParams.exposure);

visionParamsRepository.initialized = 1u;
visionParamsRepository.revision++;
if (visionParamsRepository.revision == 0u) visionParamsRepository.revision = 1u;
}
```

core1_cyt4bb7/project/code/vision/visionParams.h

SHA-256 : 1367b58f444060ff07305571f08e3125d665b63a50cdd21b83c8acc9c3228b63

代码（c）

```
#ifndef _VISION_PARAMS_H_
#define _VISION_PARAMS_H_

#include <stdint.h>

/*
 * visionParams - core1 cm_7_1 视觉参数运行时模型 + flash 持久化
 *
 * 把原本编译期 #define 的 DetectMode / BinaryCC / Legacy Canny / beacon / track 参数集中为运行时可调结构，提供：
 * - 默认值加载(visionParamsLoadDefaults, 取自 configBeacon*.h 的 #define)
 * - 热重载到检测器/相机(visionParamsApply: imageSetDetectMode + beaconCcSetConfig + cannySetThreshold + beaconTrackSetParams + imageSetExposure)
 * - 保存 core1 规划运行时参数，含三来源动态中心/外参/增益/极性/边界/fresh 门/观察超时与共享限幅/slew/TD 整形参数
 * - flash 持久化(load/save/erase, magic+version+CRC, 失败回退默认), 照搬 core0 menuModel 方案
 * - 恢复默认(visionParamsRestoreDefaults: 载默认→应用→擦盘, 擦盘后视觉立即按默认运行)
 *
 * 全部前台调用(cm_7_1 whileProcess / 菜单), 非 ISR, 无 volatile 需求。
 * 远程菜单(core0 透传)与本地开机加载都经本模块改参→apply 生效。
 *
 * 字段名 cannyLow/cannyHigh 为兼容旧 flash 前缀；当前语义是 BinaryCC low/high。
 * Legacy Canny 使用 legacyCannyLow/legacyCannyHigh。
 */

typedef struct
{
    /* v46: 按字段宽度分组, Flash 不包含编译器 padding; 非当前版本完整恢复默认。 */
```

```

/* uint32_t fields */
uint32_t sharedConfigRevision;
uint32_t sharedConfigHash;

/* float fields */
float beaconEccMax;
float trackMaxAssocDist;
float trackSizeRatioMax;
float trackPositionAlpha;
float trackSizeAlpha;
float planningKpVx;
float planningKpVy;
float core3PlanningKpVx;
float core3PlanningKpVy;
float planningVxMax;
float planningVyMax;
float planningWzMax;
float planningDeadzoneEnterPx;
float planningDeadzoneExitPx;
float planningCore0PoseMinConfidence;
float c1Core0PoseOffsetX;
float c1Core0PoseOffsetY;
float planningCore0PoseHeadingOffsetRad;
float planningTdRvx;
float planningTdRvy;
float planningTdRwz;
float planningTdN;
float trackAreaVelAlpha;
float trackAssocScoreMax;
float trackAssocAreaWeight;
float trackAssocAreaVelWeight;
float beaconFillMin;
float beaconHuMax;
float planningVxMin;
float planningVyMin;
float planningWzMin;
float planningSearchWz;
float core3CenterX;
float core3CenterY;
float core3Scale;
float core3MountYawRad;
float core3Core0PoseOffsetX;
float core3Core0PoseOffsetY;
float planningFirstBeaconForwardSpeed;
float core4PlanningKpVx;
float core4PlanningKpVy;
float core4CenterX;
float core4CenterY;
float core4Scale;
float core4MountYawRad;
float core4Core0PoseOffsetX;
float core4Core0PoseOffsetY;
float planningVxSlewPerS;
float planningVySlewPerS;
float planningWzSlewPerS;

/* uint16_t fields */
uint16_t cannyLow;
uint16_t cannyHigh;
uint16_t exposure;
uint16_t beaconSizeMin;
uint16_t beaconSizeMax;
uint16_t beaconMaxEndpoints;
uint16_t trackConfirmHits;
uint16_t trackDropMisses;
uint16_t trackCoastMisses;
uint16_t planningEdgePredictionHoldMs;
uint16_t poseHoldMs;
uint16_t planningPlanTimeoutMs;
uint16_t beaconHuAreaMin;
uint16_t legacyCannyLow;
uint16_t legacyCannyHigh;
uint16_t planningSearchTimeoutMs;
uint16_t core3ObservationTimeoutMs;
uint16_t planningPostTakeoffDelayMs;
uint16_t c1FreshGateMs;
uint16_t c3FreshGateMs;
uint16_t c4FreshGateMs;

```

```

uint16_t c4ObservationTimeoutMs;

/* uint8_t fields */
uint8_t beaconBrightnessMin;
uint8_t c1EdgeBandLeft;
uint8_t c1EdgeBandRight;
uint8_t c1EdgeBandTop;
uint8_t c1EdgeBandBottom;
uint8_t observerEdgeBandLeft;
uint8_t observerEdgeBandRight;
uint8_t observerEdgeBandTop;
uint8_t observerEdgeBandBottom;
uint8_t planningDispatchEnable;
uint8_t planningSearchEnable;
uint8_t planningTdVxEnable;
uint8_t planningTdVyEnable;
uint8_t planningTdWzEnable;
uint8_t detectMode;
uint8_t maxBeacons;
uint8_t core3MirrorX;
uint8_t c1BorderEnterLeft;
uint8_t c1BorderEnterRight;
uint8_t c1BorderEnterTop;
uint8_t c1BorderEnterBottom;
uint8_t c1BorderExitLeft;
uint8_t c1BorderExitRight;
uint8_t c1BorderExitTop;
uint8_t c1BorderExitBottom;
uint8_t observerBorderEnterLeft;
uint8_t observerBorderEnterRight;
uint8_t observerBorderEnterTop;
uint8_t observerBorderEnterBottom;
uint8_t observerBorderExitLeft;
uint8_t observerBorderExitRight;
uint8_t observerBorderExitTop;
uint8_t observerBorderExitBottom;
uint8_t planningFirstBeaconForwardEnable;
uint8_t c4EdgeBandLeft;
uint8_t c4EdgeBandRight;
uint8_t c4EdgeBandTop;
uint8_t c4EdgeBandBottom;
uint8_t core4MirrorX;
uint8_t c4BorderEnterLeft;
uint8_t c4BorderEnterRight;
uint8_t c4BorderEnterTop;
uint8_t c4BorderEnterBottom;
uint8_t c4BorderExitLeft;
uint8_t c4BorderExitRight;
uint8_t c4BorderExitTop;
uint8_t c4BorderExitBottom;

/* int8_t fields */
int8_t planningCore0PoseHeadingSign;
int8_t planningVxSign;
int8_t planningVySign;
int8_t planningWzSign;
int8_t core3ErrorFwdSign;
int8_t core3ErrorRightSign;
int8_t core3PlanningVxSign;
int8_t core3PlanningVySign;
int8_t core4ErrorFwdSign;
int8_t core4ErrorRightSign;
int8_t core4PlanningVxSign;
int8_t core4PlanningVySign;
} visionParamsStruct;

/* 活动参数由 visionParams.c 私有仓库独占；调用方按值取快照或一次提交候选 DTO。 */
void visionParamsGetActive(visionParamsStruct *out);
uint32_t visionParamsGetRevision(void);
uint8_t visionParamsCommit(const visionParamsStruct *candidate);
uint8_t visionParamsUpdateSharedIdentity(uint32_t revision, uint32_t hash);

/* 用 configBeacon.h 的 #define 默认值填充 visionParams(不应用到检测器)。 */
void visionParamsLoadDefaults(void);

/* 把私有活动参数重载到检测模式 / BinaryCC / Legacy Canny / track 检测器。 */
void visionParamsApply(void);

/* 开机加载: flash 校验通过则载入并 apply; 失败回退默认并 apply。须先 flash_init()。 */

```

```

void visionParamsLoad(void);

/* 保存当前 visionParams 到 flash(magic+version+CRC)。仅菜单保存边沿调用，稀疏写防损耗。 */
void visionParamsSave(void);

/* 恢复默认: 载默认→apply(立即按默认运行)→擦 flash。绝不只擦盘留旧值。 */
void visionParamsRestoreDefaults(void);

typedef enum
{
    VISION_PARAMS_SLOT_OK = 0,
    VISION_PARAMS_SLOT_ERROR
} visionParamsSlotStatus;

typedef struct
{
    uint8_t slot;
    uint8_t valid;
    uint8_t active;
    uint8_t bank;
    uint32_t generation;
    uint32_t blobCrc;
    uint32_t transactionId;
} visionParamsSlotMeta;

typedef struct
{
    uint32_t transactionId;
    uint8_t slot;
    uint8_t valid;
    uint8_t activationOnly;
    uint8_t active;
} visionParamsPendingInfo;

typedef struct
{
    uint8_t valid;
    uint8_t bank;
    uint8_t pending;
    uint8_t sourceSlot;
    uint8_t diagnosticFlags;
    uint8_t reserved[3];
    uint32_t generation;
    uint32_t blobCrc;
    uint32_t transactionId;
} visionParamsRuntimeMeta;

#define VISION_PARAMS_RUNTIME_DIAGNOSTIC_DEFAULTS_REWRITTEN (0x01u)

#define VISION_PARAMS_LIBRARY_EMPTY (0u)
#define VISION_PARAMS_LIBRARY_VALID (1u)
#define VISION_PARAMS_LIBRARY_INVALID (2u)
typedef struct
{
    uint8_t state;
    uint8_t bank;
    uint16_t reserved;
    uint32_t generation;
    uint32_t blobCrc;
    uint32_t transactionId;
} visionParamsLibraryMeta;

uint8_t visionParamsGetActiveSlot(void);
visionParamsSlotStatus visionParamsReadSlot(uint8_t slot,
    visionParamsStruct *params, visionParamsSlotMeta *meta);
visionParamsSlotStatus visionParamsInspectSlot(uint8_t slot,
    visionParamsLibraryMeta *meta);
visionParamsSlotStatus visionParamsPrepareSlot(uint8_t slot,
    const visionParamsStruct *params, uint32_t transactionId);
visionParamsSlotStatus visionParamsCommitPreparedSlot(uint8_t slot,
    uint32_t transactionId, uint8_t activate);
visionParamsSlotStatus visionParamsAbortPreparedSlot(uint32_t transactionId);
visionParamsSlotStatus visionParamsFinishPreparedSlot(uint32_t transactionId);
visionParamsSlotStatus visionParamsSaveSlot(uint8_t slot,
    const visionParamsStruct *params, uint32_t transactionId);
visionParamsSlotStatus visionParamsActivateSlot(uint8_t slot);
visionParamsSlotStatus visionParamsActivatePreparedSlot(uint8_t slot,
    uint32_t transactionId);
visionParamsSlotStatus visionParamsActivateSlotPending(uint8_t slot);

```



```
visionParamsSlotStatus visionParamsAbortPendingActivation(void);
visionParamsSlotStatus visionParamsFinishPendingActivation(void);
visionParamsSlotStatus visionParamsGetPending(visionParamsPendingInfo *pending);
visionParamsSlotStatus visionParamsRestoreActiveSlotDefaults(void);
visionParamsSlotStatus visionParamsReadRuntime(visionParamsStruct *params,
    visionParamsRuntimeMeta *meta);
visionParamsSlotStatus visionParamsGetRuntimeMeta(
    visionParamsRuntimeMeta *meta);
visionParamsSlotStatus visionParamsSaveRuntime(
    const visionParamsStruct *params);
visionParamsSlotStatus visionParamsPrepareRuntime(
    const visionParamsStruct *params, uint32_t transactionId,
    uint8_t sourceSlot);
visionParamsSlotStatus visionParamsPrepareRuntimeFromSlot(uint8_t slot,
    uint32_t transactionId);
visionParamsSlotStatus visionParamsApplyPreparedRuntime(
    uint32_t transactionId);
visionParamsSlotStatus visionParamsCommitRuntime(uint32_t transactionId);

#endif /* _VISION_PARAMS_H_ */
```

core1_cyt4bb7/project/code/vision/visionParamsFlashSchemaInternal.h

SHA-256

:

15e919d84a45f48358972de82110c667b374c0838e3909fa14fd22b90db12161

代码 (c)

```
#ifndef VISION_PARAMS_FLASH_SCHEMA_INTERNAL_H
#define VISION_PARAMS_FLASH_SCHEMA_INTERNAL_H

#include <stddef.h>

#include "visionParams.h"

#define VISION_FLASH_PAGE    (0u)
#define VISION_FLASH_MAGIC   (0x43 314 256u) /* C1BV */
#define VISION_FLASH_VERSION (46u)

typedef struct
{
    uint32_t magic;
    uint32_t version;
    uint32_t payloadBytes;
    visionParamsStruct param;
    uint32_t crc;
} visionFlashBlobStruct;

typedef char visionParamsV46_size_assert[(sizeof(visionParamsStruct) == 312u) ? 1 : -1];
typedef char visionFlashBlob_params_offset_assert[(offsetof(visionFlashBlobStruct, param) == 12u) ? 1 : -1];
typedef char visionFlashBlob_crc_offset_assert[(offsetof(visionFlashBlobStruct, crc) == 324u) ? 1 : -1];
typedef char visionFlashBlob_size_assert[(sizeof(visionFlashBlobStruct) == 328u) ? 1 : -1];

#endif
```

core1_cyt4bb7/project/code/vision/visionParamsInternal.h

SHA-256

:

73fb6ff4aae67bd8683dc41e0df5a9acb70fe7e54af4799793b9d68c4d2bd007

代码 (c)

```
#ifndef VISION_PARAMS_INTERNAL_H
#define VISION_PARAMS_INTERNAL_H

#include "visionParams.h"

/* 仅 persistence 与 本实现可用的私有存储入口；业务模块不得包含本头。 */
visionParamsStruct *visionParamsRepositoryMutable(void);
void visionParamsClamp(void);
```

```
#define visionParams (*visionParamsRepositoryMutable())

#endif
```

core1_cyt4bb7/project/code/vision/visionParamSlotStore.c

SHA-256

:

e675a72b8f4fbf8afb8bd391bd5f1a44fcc78106a6c51a711c6dc590821fcb2

代码 (c)

```
/* Core1-local build unit for the repository-wide parameter slot store. */
#include ".././../core0_cyt4bb7/project/code/util/paramSlotStore.c"
```

core1_cyt4bb7/project/code/vision/visionParamsPersistence.c

SHA-256

:

fe9d6ad7573c5c210573510182becfd309ad40b874970a807d60d3f3d8bfd237

代码 (c)

```
#include "visionParams.h"

#include <stddef.h>
#include <string.h>

#include ".././../core0_cyt4bb7/project/code/util/paramSlotStore.h"
#include "crc32Ieee.h"
#include "visionParamsFlashSchemaInternal.h"
#include "visionParamsInternal.h"

static paramSlotStore g_slotStore;
static uint8_t g_slotReady;

static uint8_t visionFlashBlobsValidRaw(const void *data, uint32_t bytes)
{
    const visionFlashBlobStruct *stored = (const visionFlashBlobStruct *) data;
    return (uint8_t) ((bytes == sizeof(*stored)) &&
        (stored->magic == VISION_FLASH_MAGIC) &&
        (stored->version == VISION_FLASH_VERSION) &&
        (stored->payloadBytes == sizeof(*stored)) &&
        (crc32IeeeCalc((const uint8_t *) stored,
            (uint32_t) offsetof(visionFlashBlobStruct, crc)) == stored->crc));
}

static void encodeBlob(const visionParamsStruct *params,
    visionFlashBlobStruct *blob)
{
    memset(blob, 0, sizeof(*blob));
    blob->magic = VISION_FLASH_MAGIC;
    blob->version = VISION_FLASH_VERSION;
    blob->payloadBytes = sizeof(*blob);
    blob->param = *params;
    blob->crc = crc32IeeeCalc((const uint8_t *) blob,
        (uint32_t) offsetof(visionFlashBlobStruct, crc));
}

static void sanitizeCopy(const visionParamsStruct *source,
    visionParamsStruct *sanitized)
{
    visionParamsStruct old = visionParams;
    visionParams = *source;
    visionParamsClamp();
    *sanitized = visionParams;
    visionParams = old;
}

static visionParamsSlotStatus mapStatus(paramSlotStatus status)
{

```

```

    return (status == PARAM_SLOT_OK) ? VISION_PARAMS_SLOT_OK :
        VISION_PARAMS_SLOT_ERROR;
}

void visionParamsLoad(void)
{
    visionFlashBlobStruct defaults;
    visionFlashBlobStruct stored;

    visionParamsLoadDefaults();
    visionParamsClamp();
    encodeBlob(&visionParams, &defaults);
    g_slotReady = (uint8_t) (paramSlotStoreInit(&g_slotStore,
        sizeof(defaults), visionFlashBlobsValidRaw, &defaults) == PARAM_SLOT_OK);
    if (g_slotReady != 0u)
    {
        if (paramSlotStoreReadRuntime(&g_slotStore, &stored,
            sizeof(stored)) == PARAM_SLOT_OK)
        {
            visionParams = stored.param;
            visionParamsClamp();
        }
    }
    visionParamsApply();
}

void visionParamsSave(void)
{
    visionParamsClamp();
    (void) visionParamsSaveRuntime(&visionParams);
}

void visionParamsRestoreDefaults(void)
{
    (void) visionParamsRestoreActiveSlotDefaults();
}

uint8_t visionParamsGetActiveSlot(void)
{
    return (g_slotReady != 0u) ? paramSlotStoreGetActive(&g_slotStore) : 0u;
}

visionParamsSlotStatus visionParamsReadSlot(uint8_t slot,
    visionParamsStruct *params, visionParamsSlotMeta *meta)
{
    visionFlashBlobStruct blob;
    paramSlotMeta raw;
    if ((g_slotReady == 0u) || (params == NULL) ||
        (paramSlotStoreRead(&g_slotStore, slot, &blob, sizeof(blob)) != PARAM_SLOT_OK))
        return VISION_PARAMS_SLOT_ERROR;
    sanitizeCopy(&blob.param, params);
    if (meta != NULL)
    {
        if (paramSlotStoreGetMeta(&g_slotStore, slot, &raw) != PARAM_SLOT_OK)
            return VISION_PARAMS_SLOT_ERROR;
        memcpy(meta, &raw, sizeof(*meta));
    }
    return VISION_PARAMS_SLOT_OK;
}

visionParamsSlotStatus visionParamsPrepareSlot(uint8_t slot,
    const visionParamsStruct *params, uint32_t transactionId)
{
    visionFlashBlobStruct blob;
    visionParamsStruct sanitized;
    if ((g_slotReady == 0u) || (params == NULL)) return VISION_PARAMS_SLOT_ERROR;
    sanitizeCopy(params, &sanitized);
    encodeBlob(&sanitized, &blob);
    return mapStatus(paramSlotStorePrepare(&g_slotStore, slot, &blob,
        sizeof(blob), transactionId));
}

visionParamsSlotStatus visionParamsCommitPreparedSlot(uint8_t slot,
    uint32_t transactionId, uint8_t activate)
{
    return mapStatus(paramSlotStoreCommit(&g_slotStore, slot,
        transactionId, activate));
}

```

```

visionParamsSlotStatus visionParamsAbortPreparedSlot(uint32_t transactionId)
{
    paramSlotPendingInfo pending;
    paramSlotStatus pendingStatus = paramSlotStoreGetPending(&g_slotStore,
        &pending);
    visionParamsStruct candidate;

    if ((pendingStatus != PARAM_SLOT_OK) &&
        (pendingStatus != PARAM_SLOT_EMPTY)) return VISION_PARAMS_SLOT_ERROR;
    if ((pendingStatus == PARAM_SLOT_OK) && (pending.valid != 0u) &&
        (pending.active != 0u) &&
        ((pending.activationOnly != 0u) ||
         (pending.transactionId != transactionId)))
        return VISION_PARAMS_SLOT_ERROR;
    if ((pendingStatus == PARAM_SLOT_OK) && (pending.valid != 0u) &&
        (pending.active != 0u) && (pending.activationOnly == 0u) &&
        (pending.transactionId == transactionId))
    {
        if (visionParamsReadRuntime(&candidate, NULL) != VISION_PARAMS_SLOT_OK ||
            !visionParamsCommit(&candidate)) return VISION_PARAMS_SLOT_ERROR;
    }
    return mapStatus(paramSlotStoreAbort(&g_slotStore, transactionId));
}

visionParamsSlotStatus visionParamsFinishPreparedSlot(uint32_t transactionId)
{
    paramSlotPendingInfo pending;
    paramSlotStatus pendingStatus = paramSlotStoreGetPending(&g_slotStore,
        &pending);

    if ((pendingStatus != PARAM_SLOT_OK) &&
        (pendingStatus != PARAM_SLOT_EMPTY)) return VISION_PARAMS_SLOT_ERROR;
    if ((pendingStatus == PARAM_SLOT_OK) && (pending.valid != 0u) &&
        (pending.active != 0u) &&
        visionParamsApplyPreparedRuntime(transactionId) != VISION_PARAMS_SLOT_OK)
        return VISION_PARAMS_SLOT_ERROR;
    return mapStatus(paramSlotStoreFinish(&g_slotStore, transactionId));
}

visionParamsSlotStatus visionParamsSaveSlot(uint8_t slot,
    const visionParamsStruct *params, uint32_t transactionId)
{
    visionFlashBlobStruct blob;
    visionParamsStruct sanitized;
    if ((g_slotReady == 0u) || (params == NULL)) return VISION_PARAMS_SLOT_ERROR;
    sanitizeCopy(params, &sanitized);
    encodeBlob(&sanitized, &blob);
    return mapStatus(paramSlotStoreSave(&g_slotStore, slot, &blob,
        sizeof(blob), transactionId, 0u));
}

visionParamsSlotStatus visionParamsActivateSlot(uint8_t slot)
{
    visionParamsStruct candidate;
    if (visionParamsReadSlot(slot, &candidate, NULL) != VISION_PARAMS_SLOT_OK)
        return VISION_PARAMS_SLOT_ERROR;
    if (paramSlotStoreSetActive(&g_slotStore, slot) != PARAM_SLOT_OK)
        return VISION_PARAMS_SLOT_ERROR;
    if (visionParamsCommit(&candidate) == 0u) return VISION_PARAMS_SLOT_ERROR;
    return VISION_PARAMS_SLOT_OK;
}

visionParamsSlotStatus visionParamsActivatePreparedSlot(uint8_t slot,
    uint32_t transactionId)
{
    visionParamsStruct candidate;
    if (visionParamsReadSlot(slot, &candidate, NULL) != VISION_PARAMS_SLOT_OK)
        return VISION_PARAMS_SLOT_ERROR;
    if (paramSlotStoreActivatePrepared(&g_slotStore, slot, transactionId) !=
        PARAM_SLOT_OK) return VISION_PARAMS_SLOT_ERROR;
    return visionParamsCommit(&candidate) ? VISION_PARAMS_SLOT_OK :
        VISION_PARAMS_SLOT_ERROR;
}

visionParamsSlotStatus visionParamsActivateSlotPending(uint8_t slot)
{
    visionParamsStruct candidate;
    if (visionParamsReadSlot(slot, &candidate, NULL) != VISION_PARAMS_SLOT_OK)
        return VISION_PARAMS_SLOT_ERROR;

```

```

    if (paramSlotStoreSetActivePending(&g_slotStore, slot) != PARAM_SLOT_OK)
        return VISION_PARAMS_SLOT_ERROR;
    return visionParamsCommit(&candidate) ? VISION_PARAMS_SLOT_OK : VISION_PARAMS_SLOT_ERROR;
}

visionParamsSlotStatus visionParamsAbortPendingActivation(void)
{
    paramSlotPendingInfo pending;
    visionParamsStruct candidate;
    paramSlotStatus status = paramSlotStoreGetPending(&g_slotStore, &pending);
    if (status == PARAM_SLOT_EMPTY) return VISION_PARAMS_SLOT_OK;
    if (status != PARAM_SLOT_OK || !pending.valid || !pending.activationOnly)
        return VISION_PARAMS_SLOT_ERROR;
    if (pending.active &&
        (visionParamsReadRuntime(&candidate, NULL) != VISION_PARAMS_SLOT_OK ||
         !visionParamsCommit(&candidate))) return VISION_PARAMS_SLOT_ERROR;
    return mapStatus(paramSlotStoreAbortPendingActivation(&g_slotStore));
}

visionParamsSlotStatus visionParamsFinishPendingActivation(void)
{
    paramSlotPendingInfo pending;
    if (paramSlotStoreGetPending(&g_slotStore, &pending) != PARAM_SLOT_OK ||
        !pending.valid || !pending.active || !pending.activationOnly ||
        visionParamsApplyPreparedRuntime(UINT32_MAX) != VISION_PARAMS_SLOT_OK)
        return VISION_PARAMS_SLOT_ERROR;
    return mapStatus(paramSlotStoreFinishPendingActivation(&g_slotStore));
}

visionParamsSlotStatus visionParamsGetPending(visionParamsPendingInfo *pending)
{
    paramSlotPendingInfo raw;
    paramSlotStatus status;
    if (pending == NULL) return VISION_PARAMS_SLOT_ERROR;
    memset(pending, 0, sizeof(*pending));
    status = paramSlotStoreGetPending(&g_slotStore, &raw);
    if (status == PARAM_SLOT_EMPTY) return VISION_PARAMS_SLOT_OK;
    if (status != PARAM_SLOT_OK) return VISION_PARAMS_SLOT_ERROR;
    pending->transactionId = raw.transactionId;
    pending->slot = raw.slot;
    pending->valid = raw.valid;
    pending->activationOnly = raw.activationOnly;
    pending->active = raw.active;
    return VISION_PARAMS_SLOT_OK;
}

visionParamsSlotStatus visionParamsRestoreActiveSlotDefaults(void)
{
    visionParamsStruct old = visionParams;
    visionParamsStruct defaults;
    visionParamsLoadDefaults();
    defaults = visionParams;
    visionParams = old;
    if (visionParamsSaveRuntime(&defaults) != VISION_PARAMS_SLOT_OK)
        return VISION_PARAMS_SLOT_ERROR;
    return visionParamsCommit(&defaults) ? VISION_PARAMS_SLOT_OK :
        VISION_PARAMS_SLOT_ERROR;
}

visionParamsSlotStatus visionParamsReadRuntime(visionParamsStruct *params,
    visionParamsRuntimeMeta *meta)
{
    visionFlashBlobStruct blob;
    paramRuntimeMeta raw;
    if ((g_slotReady == 0u) || (params == NULL) ||
        (paramSlotStoreReadRuntime(&g_slotStore, &blob, sizeof(blob)) !=
         PARAM_SLOT_OK)) return VISION_PARAMS_SLOT_ERROR;
    sanitizeCopy(&blob.param, params);
    if (meta != NULL)
    {
        if (paramSlotStoreGetRuntimeMeta(&g_slotStore, &raw) != PARAM_SLOT_OK)
            return VISION_PARAMS_SLOT_ERROR;
        memset(meta, 0, sizeof(*meta));
        meta->valid = raw.valid;
        meta->bank = raw.bank;
        meta->pending = raw.pending;
        meta->sourceSlot = raw.sourceSlot;
        meta->diagnosticFlags = paramSlotStoreRuntimeDefaultsRewritten(&g_slotStore) ?
            VISION_PARAMS_RUNTIME_DIAGNOSTIC_DEFAULTS_REWRITTEN : 0u;
    }
}

```

```

        meta->generation = raw.generation;
        meta->blobCrc = raw.blobCrc;
        meta->transactionId = raw.transactionId;
    }
    return VISION_PARAMS_SLOT_OK;
}

visionParamsSlotStatus visionParamsInspectSlot(uint8_t slot,
visionParamsLibraryMeta *meta)
{
    paramSlotMeta raw;
    visionFlashBlobStruct blob;
    paramSlotStatus status;
    if ((g_slotReady == 0u) || (meta == NULL))
        return VISION_PARAMS_SLOT_ERROR;
    memset(meta, 0, sizeof(*meta));
    status = paramSlotStoreGetMeta(&g_slotStore, slot, &raw);
    if (status == PARAM_SLOT_EMPTY)
    {
        meta->state = VISION_PARAMS_LIBRARY_EMPTY;
        return VISION_PARAMS_SLOT_OK;
    }
    if (status != PARAM_SLOT_OK) return VISION_PARAMS_SLOT_ERROR;
    meta->bank = raw.bank;
    meta->generation = raw.generation;
    meta->blobCrc = raw.blobCrc;
    meta->transactionId = raw.transactionId;
    status = paramSlotStoreRead(&g_slotStore, slot, &blob, sizeof(blob));
    if (status == PARAM_SLOT_OK) meta->state = VISION_PARAMS_LIBRARY_VALID;
    else if ((status == PARAM_SLOT_INVALID) ||
             (status == PARAM_SLOT_VERIFY_FAILED))
        meta->state = VISION_PARAMS_LIBRARY_INVALID;
    else return VISION_PARAMS_SLOT_ERROR;
    return VISION_PARAMS_SLOT_OK;
}

visionParamsSlotStatus visionParamsGetRuntimeMeta(
visionParamsRuntimeMeta *meta)
{
    paramRuntimeMeta raw;
    if ((meta == NULL) ||
        (paramSlotStoreGetRuntimeMeta(&g_slotStore, &raw) != PARAM_SLOT_OK))
        return VISION_PARAMS_SLOT_ERROR;
    memset(meta, 0, sizeof(*meta));
    meta->valid = raw.valid;
    meta->bank = raw.bank;
    meta->pending = raw.pending;
    meta->sourceSlot = raw.sourceSlot;
    meta->diagnosticFlags = paramSlotStoreRuntimeDefaultsRewritten(&g_slotStore) ?
        VISION_PARAMS_RUNTIME_DIAGNOSTIC_DEFAULTS_REWRITTEN : 0u;
    meta->generation = raw.generation;
    meta->blobCrc = raw.blobCrc;
    meta->transactionId = raw.transactionId;
    return VISION_PARAMS_SLOT_OK;
}

visionParamsSlotStatus visionParamsSaveRuntime(
const visionParamsStruct *params)
{
    visionFlashBlobStruct blob;
    visionParamsStruct sanitized;
    if ((g_slotReady == 0u) || (params == NULL)) return VISION_PARAMS_SLOT_ERROR;
    sanitizeCopy(params, &sanitized);
    encodeBlob(&sanitized, &blob);
    return mapStatus(paramSlotStoreSaveRuntime(&g_slotStore, &blob,
        sizeof(blob)));
}

visionParamsSlotStatus visionParamsPrepareRuntime(
const visionParamsStruct *params, uint32_t transactionId,
uint8_t sourceSlot)
{
    visionFlashBlobStruct blob;
    visionParamsStruct sanitized;
    if ((g_slotReady == 0u) || (params == NULL)) return VISION_PARAMS_SLOT_ERROR;
    sanitizeCopy(params, &sanitized);
    encodeBlob(&sanitized, &blob);
    return mapStatus(paramSlotStorePrepareRuntime(&g_slotStore, &blob,
        sizeof(blob), transactionId, sourceSlot));
}

```



```

#include "zf_common_headfile.h"

#include "core0Loop.h"
#include "crossCoreLink.h"
#include "THVD1450.h"
#include "paramSyncGateway.h"

// ***** PIT жМ *****
void pit0_ch0_isr() // 0 жБ
{
    pit_isr_flag_clear(PIT_CH0);

    pitProcess0();
}

void pit0_ch1_isr() // 1 жБ
{
    pit_isr_flag_clear(PIT_CH1);
}

void pit0_ch2_isr() // 2 жБ
{
    pit_isr_flag_clear(PIT_CH2);
}

void pit0_ch10_isr() // 10 жБ
{
    pit_isr_flag_clear(PIT_CH10);
}

void pit0_ch11_isr() // 11 жБ
{
    pit_isr_flag_clear(PIT_CH11);
}

void pit0_ch12_isr() // 12 жБ
{
    pit_isr_flag_clear(PIT_CH12);
}

void pit0_ch13_isr() // 13 жБ
{
    pit_isr_flag_clear(PIT_CH13);
}

void pit0_ch14_isr() // 14 жБ
{
    pit_isr_flag_clear(PIT_CH14);
}

void pit0_ch15_isr() // 15 жБ
{
    pit_isr_flag_clear(PIT_CH15);
}

void pit0_ch16_isr() // 16 жБ
{
    pit_isr_flag_clear(PIT_CH16);
}

void pit0_ch17_isr() // 17 жБ
{
    pit_isr_flag_clear(PIT_CH17);
}

void pit0_ch18_isr() // 18 жБ
{
    pit_isr_flag_clear(PIT_CH18);
}

void pit0_ch19_isr() // 19 жБ
{
    pit_isr_flag_clear(PIT_CH19);
}

void pit0_ch20_isr() // 20 жБ
{
    pit_isr_flag_clear(PIT_CH20);
}

```



```

}

void pit0_ch21_isr() // 21
{
    pit_isr_flag_clear(PIT_CH21);
    tsl1401_collect_pit_handler();
}
// ***** PIT *****

// ***** *****
// 0i
void uart0_isr(void)
{
    if (uart_isr_mask(UART_0)) // 0
    {
        paramSyncGatewayRxIrqHandler();
    }
    else // 0
    {
    }
}

void uart1_isr(void)
{
    if (uart_isr_mask(UART_1)) // 串口 1 接收中断
    {
        crossCoreLinkRxIrqHandler();
    }
    else // 串口 1 发送中断
    {
    }
}

void uart2_isr(void)
{
    if (uart_isr_mask(UART_2)) // 串口 2 接收中断
    {
        /* group1(P10) 单线双向方案下闲置：核间链路仅用 group0(UART_1/P04)。 */
    }
    else // 串口 2 发送中断
    {
    }
}

void uart3_isr(void)
{
    if (uart_isr_mask(UART_3)) // 3
    {
    }
    else // 3
    {
    }
}

void uart4_isr(void)
{
    if (uart_isr_mask(UART_4)) // 4
    {
        thvd1450RxIrqHandler(); // RS-485(UART_4) 收 core2 帧: 仅取字节喂解析器 // 2ص
    }
    else // 4
    {
    }
}

void uart5_isr(void)
{
    if (uart_isr_mask(UART_5)) // 5
    {
    }
    else // 5
    {
    }
}

void uart6_isr(void)
{
    if (uart_isr_mask(UART_6)) // 6

```

```

    {
    }
    else // 6
    {
    }
}
// *****M*****

// *****M*****
void gpio_0_exti_isr() // P
{
}

void gpio_1_exti_isr() // P
{
    if (exti_flag_get(P01_0)) // 'P1_0
    {
    }
    if (exti_flag_get(P01_1))
    {
    }
}

void gpio_2_exti_isr() // P
{
    if (exti_flag_get(P02_0))
    {
    }
    if (exti_flag_get(P02_4))
    {
    }
}

void gpio_3_exti_isr() // P
{
}

void gpio_4_exti_isr() // P
{
}

void gpio_5_exti_isr() // P
{
}

void gpio_6_exti_isr() // P
{
}

void gpio_7_exti_isr() // P
{
}

void gpio_8_exti_isr() // P
{
}

void gpio_9_exti_isr() // P
{
}

void gpio_10_exti_isr() // P
{
}

void gpio_11_exti_isr() // P
{
}

void gpio_12_exti_isr() // P
{
}

void gpio_13_exti_isr() // P
{
}

void gpio_14_exti_isr() // P

```


* 店铺链接 <https://seekfree.taobao.com/>
*
* 修改记录
* 日期 作者 备注
* 2024-1-9 pudding first version
* 2024-5-14 pudding 新增 12 个 pit 周期中断 增加部分注释说明
* 2025-2-4 pudding 优化串口中断逻辑，防止意外干扰导致的卡死问题，优化串口波特率计算逻辑
* 2025-2-4 pudding 新增两个串口接口
*****/

```
#include "zf_common_headfile.h"

#include "core1Loop.h"
#include "core3LinkCore1.h"
#include "core4LinkCore1.h"

// ***** PIT 中断函数 *****
void pit0_ch0_isr() // 定时器通道 0 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH0);
}

void pit0_ch1_isr() // 定时器通道 1 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH1);
}

void pit0_ch2_isr() // 定时器通道 2 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH2);

    pitProcess0(); // 菜单 10ms tick(menuKeyScan 远程按键 + 刷新计数)
}

void pit0_ch10_isr() // 定时器通道 10 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH10);
}

void pit0_ch11_isr() // 定时器通道 11 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH11);
}

void pit0_ch12_isr() // 定时器通道 12 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH12);
}

void pit0_ch13_isr() // 定时器通道 13 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH13);
}

void pit0_ch14_isr() // 定时器通道 14 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH14);
}

void pit0_ch15_isr() // 定时器通道 15 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH15);
}

void pit0_ch16_isr() // 定时器通道 16 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH16);
}

void pit0_ch17_isr() // 定时器通道 17 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH17);
}

void pit0_ch18_isr() // 定时器通道 18 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH18);
}
```

```

void pit0_ch19_isr() // 定时器通道 19 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH19);
}

void pit0_ch20_isr() // 定时器通道 20 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH20);
}

void pit0_ch21_isr() // 定时器通道 21 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH21);
    tsl1401_collect_pit_handler();
}
// ***** PIT 中断函数 *****

// ***** 串口中断函数 *****
// 串口 0 默认作为调试串口
void uart0_isr(void)
{
    if (uart_isr_mask(UART_0)) // 串口 0 接收中断
    {
        #if DEBUG_UART_USE_INTERRUPT // 如果开启 debug 串口中断
            debug_interrupr_handler(); // 调用 debug 串口接收处理函数 数据会被 debug 环形缓冲区读取
        #endif // 如果修改了 DEBUG_UART_INDEX 那这段代码需要放到对应的串口中断去
    }
    else // 串口 0 发送中断
    {
    }
}

void uart1_isr(void)
{
    if (uart_isr_mask(UART_1)) // 串口 1 接收中断
    {
        wireless_module_uart_handler();
    }
    else // 串口 1 发送中断
    {
    }
}

void uart2_isr(void)
{
    if (uart_isr_mask(UART_2)) // 串口 2 接收中断
    {
    }
    else // 串口 2 发送中断
    {
    }
}

void uart3_isr(void)
{
    if (uart_isr_mask(UART_3)) // 串口 3 接收中断
    {
    }
    else // 串口 3 发送中断
    {
    }
}

void uart4_isr(void)
{
    if (uart_isr_mask(UART_4)) // 串口 4 接收中断
    {
        uart_receiver_handler(); // 串口接收机回调函数
    }
    else // 串口 4 发送中断
    {
    }
}

void uart5_isr(void)
{
    if (uart_isr_mask(UART_5)) // 串口 5 接收中断
    {

```

```

        core3LinkCore1RxIsr();
    }
    else // 串口 5 发送中断
    {
    }
}

void uart6_isr(void)
{
    if (uart_isr_mask(UART_6)) // 串口 6 接收中断
    {
        core4LinkCore1RxIsr();
    }
    else // 串口 6 发送中断
    {
    }
}
// ***** 串口中断函数 *****

// ***** 外部中断函数 *****
void gpio_0_exti_isr() // 外部 GPIO_0 中断服务函数
{
}

void gpio_1_exti_isr() // 外部 GPIO_1 中断服务函数
{
    if (exti_flag_get(P01_0)) // 示例 P1_0 端口外部中断判断
    {
    }
    if (exti_flag_get(P01_1))
    {
    }
}

void gpio_2_exti_isr() // 外部 GPIO_2 中断服务函数
{
    if (exti_flag_get(P02_0))
    {
    }
    if (exti_flag_get(P02_4))
    {
    }
}

void gpio_3_exti_isr() // 外部 GPIO_3 中断服务函数
{
}

void gpio_4_exti_isr() // 外部 GPIO_4 中断服务函数
{
}

void gpio_5_exti_isr() // 外部 GPIO_5 中断服务函数
{
}

void gpio_6_exti_isr() // 外部 GPIO_6 中断服务函数
{
}

void gpio_7_exti_isr() // 外部 GPIO_7 中断服务函数
{
}

void gpio_8_exti_isr() // 外部 GPIO_8 中断服务函数
{
}

void gpio_9_exti_isr() // 外部 GPIO_9 中断服务函数
{
}

void gpio_10_exti_isr() // 外部 GPIO_10 中断服务函数
{
}

void gpio_11_exti_isr() // 外部 GPIO_11 中断服务函数
{
}

```

```

}

void gpio_12_exti_isr() // 外部 GPIO_12 中断服务函数
{
}

void gpio_13_exti_isr() // 外部 GPIO_13 中断服务函数
{
}

void gpio_14_exti_isr() // 外部 GPIO_14 中断服务函数
{
}

void gpio_15_exti_isr() // 外部 GPIO_15 中断服务函数
{
}

void gpio_16_exti_isr() // 外部 GPIO_16 中断服务函数
{
}

void gpio_17_exti_isr() // 外部 GPIO_17 中断服务函数
{
}

void gpio_18_exti_isr() // 外部 GPIO_18 中断服务函数
{
}

void gpio_19_exti_isr() // 外部 GPIO_19 中断服务函数
{
}

void gpio_20_exti_isr() // 外部 GPIO_20 中断服务函数
{
}

void gpio_21_exti_isr() // 外部 GPIO_21 中断服务函数
{
}

void gpio_22_exti_isr() // 外部 GPIO_22 中断服务函数
{
}

void gpio_23_exti_isr() // 外部 GPIO_23 中断服务函数
{
}
// ***** 串口中断函数 *****

```

core1_cyt4bb7/project/user/main_cm7_0.c

SHA-256

:

68b421c6f4fb20c0361f25c4d3b1b4035b4102af3991067b4532706b8f738421

代码 (c)

```

/*****
* CYT4BB Opensourec Library 即（CYT4BB 开源库）是一个基于官方 SDK 接口的第三方开源库
* Copyright (c) 2022 SEEKFREE 逐飞科技
*
* 本文件是 CYT4BB 开源库的一部分
*
* CYT4BB 开源库 是免费软件
* 您可以根据自由软件基金会发布的 GPL（GNU General Public License，即 GNU 通用公共许可证）的条款
* 即 GPL 的第 3 版（即 GPL3.0）或（您选择的）任何后来的版本，重新发布和/或修改它
*
* 本开源库的发布是希望它能发挥作用，但并未对其作任何的保证
* 甚至没有隐含的适销性或适合特定用途的保证
* 更多细节请参见 GPL
*
* 您应该在收到本开源库的同时收到一份 GPL 的副本
* 如果没有，请参阅<https://www.gnu.org/licenses/>
*

```

```
* 额外注明:
* 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
* 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
* 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
* 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明 (即本声明)
*
* 文件名称      main_cm7_0
* 公司名称      成都逐飞科技有限公司
* 版本信息      查看 libraries/doc 文件夹内 version 文件 版本说明
* 开发环境      IAR 9.40.1
* 适用平台      CYT4BB
* 店铺链接      https://seekfree.taobao.com/
*
* 修改记录
* 日期          作者          备注
* 2024-1-4      pudding      first version
*****/

#include "zf_common_headfile.h"

#include "core0Loop.h"

// 打开新的工程或者工程移动了位置务必执行以下操作
// 第一步 关闭上面所有打开的文件
// 第二步 project->clean 等待下方进度条走完

// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设

// ***** 代码区域 *****

/* =====
* BEFORE EDITING THIS CORE — READ THESE TWO DOCS FIRST:
* 1. <repo-root>/CLAUDE.md      — multi-core architecture, naming rules, git workflow
* 2. ./project/code/PROJECT.md  — this core's modules, boundaries, hazards
* Each core has the SAME filenames (motor.c / IMU.c / PID.c) but DIFFERENT semantics.
* Do NOT copy patterns across cores. Do NOT edit two cores in one change.
* ===== */
int main(void)
{
    clock_init(SYSTEM_CLOCK_250M); // 时钟配置及系统初始化<务必保留>
    debug_init();                  // 调试串口信息初始化
    // 此处编写用户代码 例如外设初始化代码等

    processInit();

    // 此处编写用户代码 例如外设初始化代码等
    while (true)
    {
        // 此处编写需要循环执行的代码

        whileProcess();

        // 此处编写需要循环执行的代码
    }
}

// ***** 代码区域 *****
```

core1_cyt4bb7/project/user/main_cm7_1.c

SHA-256

:

363e25e84c266e455ba510f21d00be6a76c6b61238c8b3cf827431da3e8084e8

代码 (c)

```
/*=====
* CYT4BB Opensourec Library 即 ( CYT4BB 开源库) 是一个基于官方 SDK 接口的第三方开源库
* Copyright (c) 2022 SEEKFREE 逐飞科技
*
* 本文件是 CYT4BB 开源库的一部分
*
* CYT4BB 开源库 是免费软件
```


* 您可以根据自由软件基金会发布的 GPL（GNU General Public License，即 GNU 通用公共许可证）的条款
* 即 GPL 的第 3 版（即 GPL3.0）或（您选择的）任何后来的版本，重新发布和/或修改它
*
* 本开源库的发布是希望它能发挥作用，但并未对其作任何的保证
* 甚至没有隐含的适销性或适合特定用途的保证
* 更多细节请参见 GPL
*
* 您应该在收到本开源库的同时收到一份 GPL 的副本
* 如果没有，请参阅<<https://www.gnu.org/licenses/>>
*
* 额外注明：
* 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
* 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
* 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
* 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明（即本声明）
*
* 文件名称 main_cm7_1
* 公司名称 成都逐飞科技有限公司
* 版本信息 查看 libraries/doc 文件夹内 version 文件 版本说明
* 开发环境 IAR 9.40.1
* 适用平台 CYT4BB
* 店铺链接 <https://seekfree.taobao.com/>
*
* 修改记录
* 日期 作者 备注
* 2024-1-4 pudding first version
*****/

#include "zf_common_headfile.h"

#include "core1Loop.h"

// 打开新的工程或者工程移动了位置务必执行以下操作
// 第一步 关闭上面所有打开的文件
// 第二步 project->clean 等待下方进度条走完

// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设

// ***** 代码区域 *****

/* =====
* BEFORE EDITING THIS CORE — READ THESE TWO DOCS FIRST:
* 1. <repo-root>/CLAUDE.md — multi-core architecture, naming rules, git workflow
* 2. ./project/code/PROJECT.md — this core's modules, boundaries, hazards
* Each core has the SAME filenames (motor.c / IMU.c / PID.c) but DIFFERENT semantics.
* Do NOT copy patterns across cores. Do NOT edit two cores in one change.
* ===== */

int main(void)
{
 clock_init(SYSTEM_CLOCK_250M); // 时钟配置及系统初始化<务必保留>
 debug_info_init(); // 调试串口信息初始化

 // 此处编写用户代码 例如外设初始化代码等

 processInit();

 // 此处编写用户代码 例如外设初始化代码等

 while (true)
 {
 // 此处编写需要循环执行的代码

 whileProcess();

 // 此处编写需要循环执行的代码

 }

}

// ***** 代码区域 *****

core2_cyt4bb7

core2_cyt4bb7/project/code/app/buzzer.c

SHA-256 :
8fee9d21f0e15396530deee99027c16ea8d43e7e336b51cf40ec9e4416b

代码 (c)

```
#include "buzzer.h"

#include "pinMap.h" /* BEEP_PIN */
#include "configChassisRemote.h" /* BEEP_TICKS_SHORT (短鸣脉宽, 500µs/tick) */
#include "chassisLink.h" /* CHASSIS_LINK_BUZZER_ 码值 */
#include "zf_driver_gpio.h"

/*
 * 蜂鸣器单一所有者本体。详见 buzzer.h 顶注。
 * BEEP_PIN 唯一写者: 仅本文件 gpio_*。仲裁优先级为
 * runaway > WARN > hardware-init-fault > event > ordinary pattern。
 * 节奏相位 rhythmTick 每 buzzerTick(500µs) 自增; 短鸣脉宽复用 BEEP_TICKS_SHORT(100ms=200tick)。
 * 纯 cm_7_0 PIT 上下文 (buzzerInit 在 ISR 启动前 processInit 调), 状态单上上下文, 无需 volatile。
 * 解耦: 不 include motorControl/loraNavControl——失控/事件态经 setter 注入, 无反向依赖。
 */

/* 节奏周期 (500µs/tick) 。 */
#define BUZ_PERIOD_1HZ_TICKS (2000u) /* 1Hz: 每 1000ms 一次短鸣 (起飞等待/爬高) */
#define BUZ_PERIOD_3S_TICKS (6000u) /* 3s: 每 3000ms 一次短鸣 (巡航) */
#define BUZ_PERIOD_WARN_TICKS (400u) /* WARN: 200ms 周期 50% 占空急促鸣 (断联告警) */
#define BUZ_PERIOD_ARMED_TICKS (4000u) /* armed-idle: 每 2000ms 一次短鸣 (模式 3 已 arm 待 S3+S4 请求, 桨在转警示) */
#define BUZ_PERIOD_WAITING_TICKS (6000u) /* 等待连接: 每 3000ms 一次短鸣 (core2 WAIT_LINK 等 RS-485; 与 3S 同节奏但上下文互斥) */
#define BUZ_PERIOD_LORA_IDLE_TICKS (4000u) /* MODE4 本地 LoRa 兜底: 每 2000ms 一声 (0.5Hz, 人耳可辨别) */
#define BUZ_LORA_IDLE_ON_TICKS (300u) /* LoRa 兜底响 150ms (比 SHORT 略长, 舒缓提示而非急促告警) */
#define BUZ_PERIOD_HW_FAIL_TICKS (10000u) /* 5s */
#define BUZ_HW_FAIL_GAP_TICKS (400u) /* 每声起点相隔 200ms */

static uint8_t curCode; /* 当前模式/命令码 */
static uint32_t rhythmTick; /* 节奏相位 (每拍自增) */
static uint16_t eventRemainTicks; /* 一次性事件确认音剩余拍数 (>0 覆盖 pattern) */
static uint8_t runawayActive; /* 失控锁定态 (常鸣, 最高优先) */
static uint8_t hardwareInitFaultActive;

void buzzerInit(void)
{
    gpio_init(BEEP_PIN, GPO, GPIO_LOW, GPO_PUSH_PULL);
    curCode = CHASSIS_LINK_BUZZER_SILENT;
    rhythmTick = 0u;
    eventRemainTicks = 0u;
    runawayActive = 0u;
    hardwareInitFaultActive = 0u;
}

void buzzerSetPattern(uint8_t code)
{
    if (code != curCode)
    {
        curCode = code;
        rhythmTick = 0u; /* 码变化重置节奏相位, 新模式从脉冲起点播放 (短鸣即时触发) */
    }
}

void buzzerBeepEvent(uint16_t durationTicks)
{
    /* 一次性事件确认音: 置剩余拍数, buzzerTick 在 pattern 之上、runaway 之下播满时长。
     * 后到事件覆盖前一事件剩余 (取新时长, 与原 beepRemainTicks 直接赋值语义一致)。 */
    eventRemainTicks = durationTicks;
}

void buzzerSetRunaway(uint8_t active)
{
    runawayActive = active ? 1u : 0u;
}
```

```

void buzzerSetHardwareInitFault(uint8_t active)
{
    hardwareInitFaultActive = active ? 1u : 0u;
}

static uint8_t buzzerComputeHardwareFailOn(void)
{
    uint32_t phase = rhythmTick % BUZ_PERIOD_HW_FAIL_TICKS;
    if (phase >= 4u * BUZ_HW_FAIL_GAP_TICKS) return 0u;
    return ((phase % BUZ_HW_FAIL_GAP_TICKS) < BEEP_TICKS_SHORT) ? 1u : 0u;
}

/* 按当前码与节奏相位算"本拍是否鸣" (pattern 层) 。 */
static uint8_t buzzerComputePatternOn(void)
{
    switch (curCode)
    {
        case CHASSIS_LINK_BUZZER_1HZ:
            return ((rhythmTick % BUZ_PERIOD_1HZ_TICKS) < BEEP_TICKS_SHORT) ? 1u : 0u;
        case CHASSIS_LINK_BUZZER_LONG:
            return 1u; /* 长鸣：码持续期间常鸣 (sequencer 保持 LONG ~800ms 后转 3S) */
        case CHASSIS_LINK_BUZZER_3S:
            return ((rhythmTick % BUZ_PERIOD_3S_TICKS) < BEEP_TICKS_SHORT) ? 1u : 0u;
        case CHASSIS_LINK_BUZZER_WARN:
            return ((rhythmTick % BUZ_PERIOD_WARN_TICKS) < (BUZ_PERIOD_WARN_TICKS / 2u)) ? 1u : 0u;
        case CHASSIS_LINK_BUZZER_ARMED_IDLE:
            return ((rhythmTick % BUZ_PERIOD_ARMED_TICKS) < BEEP_TICKS_SHORT) ? 1u : 0u;
        case CHASSIS_LINK_BUZZER_WAITING:
            return ((rhythmTick % BUZ_PERIOD_WAITING_TICKS) < BEEP_TICKS_SHORT) ? 1u : 0u;
        case CHASSIS_LINK_BUZZER_LORA_IDLE:
            return ((rhythmTick % BUZ_PERIOD_LORA_IDLE_TICKS) < BUZ_LORA_IDLE_ON_TICKS) ? 1u : 0u;
        case CHASSIS_LINK_BUZZER_HW_INIT_FAIL:
            {
                uint32_t phase = rhythmTick % BUZ_PERIOD_HW_FAIL_TICKS;
                if (phase >= 4u * BUZ_HW_FAIL_GAP_TICKS) return 0u;
                return ((phase % BUZ_HW_FAIL_GAP_TICKS) < BEEP_TICKS_SHORT) ? 1u : 0u;
            }
        case CHASSIS_LINK_BUZZER_THREE_BEEPS:
            /* 三连短鸣(一次性): 码变化时 buzzerSetPattern 重置 rhythmTick=0, 按绝对相位播 3 声后恒静默
             * (不取模)。每声 BEEP_TICKS_SHORT(100ms), 声起点间距 250ms (500 tick @500µs; 即鸣 100ms + 静默 150ms)。
             * 码保持不变 rhythmTick 续增但恒静默, 不重播; Core0 完整系统资格 owner 经链路下发
             * 约 700ms 码 7 窗口覆盖全程, Core2 不从局部链路边沿猜测 READY。 */
            if (rhythmTick < BEEP_TICKS_SHORT) return 1u;
            if (rhythmTick >= 500u && rhythmTick < (500u + BEEP_TICKS_SHORT)) return 1u;
            if (rhythmTick >= 1000u && rhythmTick < (1000u + BEEP_TICKS_SHORT)) return 1u;
            return 0u;
        case CHASSIS_LINK_BUZZER_SILENT:
            default:
                return 0u;
    }
}

void buzzerTick(void)
{
    uint8_t on;

    /* 节奏相位无条件推进, 保时间准 (即便本拍被 runaway/event 覆盖, 恢复时相位不漂移) 。 */
    rhythmTick++;

    /* 1. 失控锁定最高优先: 常鸣 (取代原 motorControlOutput 直写 gpio_high; 仅复位经 buzzerInit 解锁) 。 */
    if (runawayActive)
    {
        gpio_high(BEEP_PIN);
        return;
    }

    /* 2. 安全 WARN 高于本机故障和操作事件。 */
    if (curCode == CHASSIS_LINK_BUZZER_WARN)
    {
        on = buzzerComputePatternOn();
        if (on) gpio_high(BEEP_PIN); else gpio_low(BEEP_PIN);
        return;
    }

    /* 3. 本机初始化故障是独立请求, 不能被普通 pattern 覆盖。 */
    if (hardwareInitFaultActive != 0u)
    {
        on = buzzerComputeHardwareFailOn();
        if (on) gpio_high(BEEP_PIN); else gpio_low(BEEP_PIN);
    }
}

```

```

    return;
}

/* 4. 一次性事件确认音：高电平播满剩余时长，覆盖普通 pattern。 */
if (eventRemainTicks > 0u)
{
    eventRemainTicks--;
    gpio_high(BEEP_PIN);
    return;
}

/* 5. 普通模式/命令码节奏。 */
on = buzzerComputePatternOn();
if (on)
{
    gpio_high(BEEP_PIN);
}
else
{
    gpio_low(BEEP_PIN);
}
}

```

core2_cyt4bb7/project/code/app/buzzer.h

SHA-256

:

374992998db50234dea2406013012da4f8d5d3e977d8e4e7e0a062b6cd5448db

代码 (c)

```

#ifndef _BUZZER_H_
#define _BUZZER_H_

#include <stdint.h>

/*
 * core2 蜂鸣器单一所有者（代飞机鸣叫——无人机无蜂鸣器，借车载 BEEP_PIN=P07_2）。
 * 本模块是 BEEP_PIN 的唯一写者：仅此处 gpio_init/gpio_high/gpio_low。其余模块（loraNavControl
 * 事件确认音、motorControl 失控锁定、core0Loop 模式/命令码）只经下方 API 提交“请求”，绝不直写 BEEP，
 * 消除原先三模块各自直写 + 依赖调用顺序的隐式仲裁与冗余双写竞态。
 *
 * 显式优先级仲裁器（buzzerTick 内单点判定，每拍恰写一次）：
 * 1. runaway（失控锁定，buzzerSetRunaway）：常鸣，最高优先（覆盖一切）。
 * 2. WARN：必需链路/缓降急促告警，不能被操作确认音覆盖。
 * 3. HW_INIT_FAIL：Core2 本地显示初始化故障节奏。
 * 4. event：被高优先级覆盖时暂停剩余时长，解除后续播。
 * 5. 其它 pattern。
 * 无 runaway/event 且 pattern=SILENT → gpio_low（静默）。
 *
 * pattern 码值 = CHASSIS_LINK_BUZZER * (SILENT/1HZ/LONG/3S/WARN/ARMED_IDLE/WAITING/THREE_BEEPS,
 * core0 takeoffSequencer 产 → core1 中转 RS-485 sysByte → core2 播放) + core2 本地扩展码
 * LORA_IDLE(=8, 超 sysByte 3-bit, 仅 MODE4 本地置、不经传输)。
 *
 * 调用约定（不变量）：所有 API 仅在 cm_7_0 PIT 上下文（pitProcess0 / loraNavControlPit500us）调用，
 * 单上下文 → 内部状态无需 volatile。buzzerInit 在 processInit (ISR 启动前) 调，恰一次。
 * buzzerTick 每 500μs 调恰一次（PRODUCTION 与 TUNE 均调，使失控常鸣两态都生效）。
 */

void buzzerInit(void);          /* 初始化 BEEP GPIO（唯一所有者）并复位仲裁状态 */
void buzzerSetPattern(uint8_t code); /* 设模式/命令码(CHASSIS_LINK_BUZZER_*)；码变化即重置节奏相位 */
void buzzerBeepEvent(uint16_t durationTicks); /* 请求一次性事件确认音 N 拍（覆盖 pattern；500μs/拍） */
void buzzerSetRunaway(uint8_t active); /* 设失控锁定态（传 motorControlIsRunawayLatched()）：常鸣最高优先 */
void buzzerSetHardwareInitFault(uint8_t active); /* 独立本机故障请求，不与 WARN 共用 pattern 码 */
void buzzerTick(void);          /* 500μs：优先级仲裁后写 BEEP_PIN 恰一次（唯一写者） */

#define CHASSIS_LINK_BUZZER_HW_INIT_FAIL (9u)

#endif /* _BUZZER_H_ */

```

core2_cyt4bb7/project/code/app/chassisMenu.c

SHA-256

:

b8d215679d2c92888fa47d2f3b8388cfcde5c4aba06da11f1c9bdfdf1930eba72

代码 (c)

```
#include "chassisMenu.h"
#include "chassisMenuInternal.h"

#include "chassisLink.h" /* chassisLinkIsAlive: WAIT_RS485 门控(A8) */
#include "core0Loop.h" /* runtime snapshot: 模式 4 结束等待(A8) */
#include "configCore2Build.h"
#include "motorControl.h" /* 失控锁定查询/人工解锁 (HOME K5 长按) */
#include "paramSyncOwnerService.h"
#include "menuKey.h"
#include "lightStrip.h" /* 待发车页亮度快调 lightStripSetBrightness + 步长/钳位宏 */
#include "commonDef.h" /* range 钳位 */
#include "zf_device_ips114.h"
#include "zf_common_font.h"

#include <stdio.h>

/* =====
 * 共享状态唯一定义点 (chassisMenuInternal.h 中 extern)
 * ===== */
chassisMenuContextStruct chassisMenuContext = {state = CHASSIS_MENU_HOME};
static uint8_t paramLockWasActive;
static void localFlashFeedback(uint8_t action, uint8_t success);

/* =====
 * 生命周期
 * ===== */
void chassisMenuInit(void)
{
    chassisMenuBuildPages();

    ips114_set_dir(CORE2_IPS114_DISPLAY_DIR);
    ips114_set_font(IPS114_8X16_FONT);
    ips114_init();
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
    ips114_clear();

    /* 上电先进等待 RS-485 态: 未连 core1 且未进模式 4 时停在等待屏, 不进菜单(A8, 仿 core1 WAIT_LINK)。 */
    chassisMenuState = CHASSIS_MENU_WAIT_RS485;
    chassisPageIdx = 0;
    chassisOptIdx = 0;
    chassisScrollTop = 0u;
    chassisForceRedraw = 1u;
    chassisDirtyMask = CHASSIS_MENU_DIRTY_ALL;
    chassisFeedbackHoldTicks = 0u;
    feedbackWasActive = 0u;
    chassisSurfaceResetPending = 0u;
    chassisFeedbackRenderPending = 0u;
    chassisFeedbackColor = RGB565_BLACK;
    chassisFeedbackAction = PARAM_SYNC_GLOBAL_ACTION_NONE;
    chassisFeedbackSlot = 0u;
    chassisParamSlotInspectPending = 0u;
    chassisParamGlobalLastEvent = 0u;
    chassisBrightnessSaveDelayMs = 0u;
    chassisBrightnessSavePending = 0u;
    chassisFlashActionPending = CHASSIS_MENU_FLASH_ACTION_NONE;
    chassisInitFailedMask = 0u;
    chassisInitUnverifiedMask = 0u;
    chassisInitSummaryMs = 0u;
    chassisInitSummaryLastSecond = 0xFFu;
    chassisParamSlotSelection = 1.0f;
    paramLockWasActive = 0u;
}

/* ISR 1ms: 按键扫描 + 填充反馈倒计时 (轻量, 不触屏) */
void chassisMenuPitTick(void)
{
    menuKeyScan();
```

```

        if (chassisFeedbackHoldTicks > 0u) chassisFeedbackHoldTicks--;
        if (chassisInitSummaryMs > 0u) chassisInitSummaryMs--;
        if (chassisBrightnessSaveDelayMs > 0u) chassisBrightnessSaveDelayMs--;
    }

    /* 输入阶段只请求 surface 切换，实际清屏统一留给渲染阶段。 */
    static void requestSurfaceReset(void)
    {
        chassisSurfaceResetPending = 1u;
        chassisForceRedraw = 1u;
        chassisDirtyMask |= CHASSIS_MENU_DIRTY_SURFACE;
    }

    static void cancelPendingBrightnessSave(void)
    {
        chassisBrightnessSavePending = 0u;
        chassisBrightnessSaveDelayMs = 0u;
    }

    static void requestSettingsSave(void)
    {
        cancelPendingBrightnessSave();
        chassisFlashActionPending = CHASSIS_MENU_FLASH_ACTION_SAVE;
    }

    static void requestSettingsClear(void)
    {
        cancelPendingBrightnessSave();
        chassisFlashActionPending = CHASSIS_MENU_FLASH_ACTION_CLEAR;
    }

    static void serviceBrightnessSave(void)
    {
        if (core2ParamSyncOwnerServiceMutationLocked() != 0u) return;
        if ((chassisBrightnessSavePending != 0u) &&
            (chassisBrightnessSaveDelayMs == 0u))
        {
            chassisBrightnessSavePending = 0u;
            if (core2RuntimeRequestSave() == 0u)
            {
                chassisBrightnessSavePending = 1u;
                chassisBrightnessSaveDelayMs = 100u;
            }
        }
    }

    static void serviceFlashAction(void)
    {
        uint8_t action = chassisFlashActionPending;
        uint8_t success;
        if (core2ParamSyncOwnerServiceMutationLocked() != 0u) return;
        if (action == CHASSIS_MENU_FLASH_ACTION_CLEAR_WAIT)
        {
            if (core2RuntimeParamsApplied() != 0u)
            {
                chassisFlashActionPending = CHASSIS_MENU_FLASH_ACTION_NONE;
                localFlashFeedback(CHASSIS_MENU_FLASH_ACTION_CLEAR, 1u);
            }
            return;
        }
        chassisFlashActionPending = CHASSIS_MENU_FLASH_ACTION_NONE;
        if (action == CHASSIS_MENU_FLASH_ACTION_SAVE)
        {
            success = chassisMenuSaveSettings();
            localFlashFeedback(action, success);
        }
        else if (action == CHASSIS_MENU_FLASH_ACTION_CLEAR)
        {
            success = chassisMenuClearFlash();
            if (success == 0u)
                localFlashFeedback(action, 0u);
            else if (core2RuntimeParamsApplied() != 0u)
                localFlashFeedback(action, 1u);
            else
                chassisFlashActionPending = CHASSIS_MENU_FLASH_ACTION_CLEAR_WAIT;
        }
    }

    /* =====

```

```

/* 按键消费 + 滚动维护 (仿 core0 menuInput)
* ===== */
static uint8_t keyHit(menuKeyIndexEnum k)
{
    if (menuKeyGetState(k) == MENU_KEY_SHORT_PRESS)
    {
        menuKeyClear(k);
        return 1u;
    }
    return 0u;
}

static uint8_t keyLong(menuKeyIndexEnum k)
{
    if (menuKeyGetState(k) == MENU_KEY_LONG_PRESS)
    {
        menuKeyClear(k);
        return 1u;
    }
    return 0u;
}

/* 环形循环 + 滚动窗口维护, 使选中项始终可见 (仿 core0 clampSelAndScroll) */
static void clampSelAndScroll(uint8_t* sel, uint8_t count)
{
    if (count == 0u) { *sel = 0; chassisScrollTop = 0u; return; }
    if (*sel < 0) *sel = (uint8_t) (count - 1u);
    else if (*sel >= (int8_t) count) *sel = 0;

    if ((uint8_t) *sel < chassisScrollTop)
        chassisScrollTop = (uint8_t) *sel;
    else if ((uint8_t) *sel >= (uint8_t) (chassisScrollTop + VISIBLE_ROWS))
        chassisScrollTop = (uint8_t) ((uint8_t) *sel - VISIBLE_ROWS + 1u);
}

static void enterChassisPageList(uint8_t resetSelection)
{
    chassisMenuState = CHASSIS_MENU_PAGE_LIST;
    if (resetSelection)
    {
        chassisPageIdx = 0;
        chassisScrollTop = 0u;
    }
    else
    {
        chassisScrollTop = 0u;
        clampSelAndScroll(&chassisPageIdx, chassisMenuPageCount);
    }
    requestSurfaceReset();
}

static void enterChassisOptionList(void)
{
    if (chassisPageIdx < 0 || (uint8_t) chassisPageIdx >= chassisMenuPageCount
        || chassisMenuPages[chassisPageIdx].optionCount == 0u)
    {
        chassisForceRedraw = 1u;
        return;
    }
    chassisMenuState = CHASSIS_MENU_OPTION_LIST;
    chassisOptIdx = 0;
    chassisScrollTop = 0u;
    if ((uint8_t) chassisPageIdx == chassisParamSlotsPageIndex)
        chassisParamSlotInspectPending = 1u;
    requestSurfaceReset();
}

static uint8_t currentOptionIsValid(const chassisMenuPageStruct* page)
{
    uint8_t i;
    if (page == 0 || chassisOptIdx < 0 || (uint8_t) chassisOptIdx >= page->optionCount) return 0u;
    i = (uint8_t) chassisOptIdx;
    return (uint8_t) (page->opt[i] != 0);
}

/* 填充反馈: 整屏色 + 倒计时 (保存绿/恢复红) */
static void flashFeedback(uint16_t color)
{
    chassisFeedbackColor = color;
}

```

```

        chassisFeedbackAction = PARAM_SYNC_GLOBAL_ACTION_NONE;
        chassisFeedbackSlot = 0u;
        chassisFeedbackRenderPending = 1u;
        chassisFeedbackHoldTicks = CHASSIS_MENU_FEEDBACK_HOLD_TICKS;
        chassisDirtyMask |= CHASSIS_MENU_DIRTY_FEEDBACK;
    }

static void localFlashFeedback(uint8_t action, uint8_t success)
{
    chassisFeedbackColor = success ?
        ((action == CHASSIS_MENU_FLASH_ACTION_SAVE) ?
            RGB565_GREEN : RGB565_RED) : RGB565_YELLOW;
    chassisFeedbackAction = (action == CHASSIS_MENU_FLASH_ACTION_SAVE) ?
        CHASSIS_MENU_FEEDBACK_LOCAL_SAVE :
        CHASSIS_MENU_FEEDBACK_LOCAL_RESET;
    chassisFeedbackSlot = success;
    chassisFeedbackRenderPending = 1u;
    chassisFeedbackHoldTicks = CHASSIS_MENU_FEEDBACK_HOLD_TICKS;
    chassisDirtyMask |= CHASSIS_MENU_DIRTY_FEEDBACK;
}

static void paramFeedback(uint8_t action, uint8_t slot)
{
    chassisFeedbackColor = (action == PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY) ?
        RGB565_GREEN : RGB565_RED;
    chassisFeedbackAction = action;
    chassisFeedbackSlot = slot;
    chassisFeedbackRenderPending = 1u;
    chassisFeedbackHoldTicks = CHASSIS_MENU_FEEDBACK_HOLD_TICKS;
    chassisDirtyMask |= CHASSIS_MENU_DIRTY_FEEDBACK;
}

/* 待发车页(WAIT/HOME)快调灯条亮度：立即热应用，最后一次变化 500ms 后合并落盘。 */
static void chassisMenuAdjustBrightness(int8_t deltaPct)
{
    float prev = chassisMenuWorkingParams.lightStripBrightness;
    float b = range(prev + (float) deltaPct,
        (float) LIGHT_STRIP_BRIGHTNESS_MIN, (float) LIGHT_STRIP_BRIGHTNESS_MAX);
    if (b == prev) return; /* 已在上下限、值未变 → 不重复写 flash（限位处连接防磨损；值为 10 的整数倍，浮点等值比较安全） */
    chassisMenuWorkingParams.lightStripBrightness = b;
    (void) core2RuntimeSubmitParams(&chassisMenuWorkingParams);
    chassisParamsGetActive(&chassisMenuWorkingParams);
    chassisBrightnessSavePending = 1u;
    chassisBrightnessSaveDelayMs = LIGHT_STRIP_SAVE_DELAY_MS;
    chassisDirtyMask |= CHASSIS_MENU_DIRTY_HOME_BRIGHTNESS;
}

/* =====
* 各状态按键 handler
* ===== */
static void handleHomeKeys(void)
{
    if (core2ParamSyncOwnerServiceMutationLocked() != 0u)
    {
        uint8_t key;
        if (keyHit(MENU_KEY_4)) enterChassisPageList(1u);
        for (key = 0u; key < (uint8_t) MENU_KEY_COUNT; key++)
            menuKeyClear((menuKeyIndexEnum) key);
        return;
    }
    /* 新键位(codex 复核点 1)：K4 进菜单(对齐页列表/选项列表 K4=进入)；K1 短=保存 / 长=擦 flash+热重载默认。
    * K2-/K3+ 灯条亮度；K5 短保存、长按人工清失控锁定。 */
    if (keyHit(MENU_KEY_4))
    {
        enterChassisPageList(1u);
    }
    else if (keyHit(MENU_KEY_2)) { chassisMenuAdjustBrightness(-(int8_t) LIGHT_STRIP_STEP_COARSE); } /* 亮度 -10% */
    else if (keyHit(MENU_KEY_3)) { chassisMenuAdjustBrightness(+ (int8_t) LIGHT_STRIP_STEP_COARSE); } /* 亮度 +10% */
    else if (keyHit(MENU_KEY_1)) { requestSettingsSave(); } /* K1 短=保存 */
    else if (keyLong(MENU_KEY_1))
    {
        requestSettingsClear();
    } /* K1 长=擦 flash+热重载默认 */
    else if (keyHit(MENU_KEY_5)) { requestSettingsSave(); } /* K5 短=保存(冗余) */
    else if (keyLong(MENU_KEY_5))
    {
        /* K5 长=失控锁定时人工解锁（免断电恢复重新发车）；未锁定则无操作。解锁后由发车门控正常受控。 */
        if (motorControlsRunawayLatched())
        {

```



```

        motorControlClearRunaway();
        flashFeedback(GB565_YELLOW);
    }
}

static void handlePageListKeys(void)
{
    if (keyHit(MENU_KEY_4))
    {
        enterChassisOptionList();
    }
    else if (keyHit(MENU_KEY_2))
    {
        chassisPageIdx--;
        clampSelAndScroll(&chassisPageIdx, chassisMenuPageCount);
        chassisForceRedraw = 1u;
    }
    else if (keyHit(MENU_KEY_3))
    {
        chassisPageIdx++;
        clampSelAndScroll(&chassisPageIdx, chassisMenuPageCount);
        chassisForceRedraw = 1u;
    }
    else if (keyHit(MENU_KEY_1))
    {
        chassisMenuState = CHASSIS_MENU_HOME;
        requestSurfaceReset();
    }
    else if (keyHit(MENU_KEY_5))
    {
        if (core2ParamSyncOwnerServiceMutationLocked() == 0u)
        {
            requestSettingsSave();
            chassisMenuState = CHASSIS_MENU_HOME;
        }
    }
}

static void handleOptionListKeys(void)
{
    const chassisMenuPageStruct* page;
    if (chassisPageIdx < 0 || (uint8_t) chassisPageIdx >= chassisMenuPageCount)
    {
        enterChassisPageList(0u);
        return;
    }
    page = &chassisMenuPages[chassisPageIdx];

    if ((uint8_t) chassisPageIdx == chassisParamSlotsPageIndex)
    {
        uint8_t slot = (uint8_t) chassisParamSlotSelection;
        if (keyHit(MENU_KEY_5))
        {
            enterChassisPageList(0u);
        }
        else if (keyHit(MENU_KEY_2))
        {
            chassisParamSlotSelection = (float) ((slot > 1u) ? slot - 1u : 5u);
            chassisParamSlotInspectPending = 1u;
            chassisForceRedraw = 1u;
        }
        else if (keyHit(MENU_KEY_3))
        {
            chassisParamSlotSelection = (float) ((slot < 5u) ? slot + 1u : 1u);
            chassisParamSlotInspectPending = 1u;
            chassisForceRedraw = 1u;
        }
        else if (keyHit(MENU_KEY_4))
        {
            core2ParamGlobalUiSnapshot ui;
            uint8_t ok = 0u;
            core2RuntimeGetParamGlobalUi(&ui);
            if (chassisParamSlotInspectPending == 0u &&
                ui.state == CORE2_PARAM_GLOBAL_SUCCESS &&
                ui.action == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY &&
                ui.slot == slot &&
                ui.slotUiState == PARAM_SYNC_SLOT_UI_FULL)
            {

```

```

        ok = core2RuntimeRequestGlobalParamAction(
            PARAM_SYNC_GLOBAL_ACTION_LOAD_RUNTIME, slot);
    }
    (void) ok;
    chassisForceRedraw = 1u;
}
else if (keyLong(MENU_KEY_1))
{
    uint8_t ok = core2RuntimeRequestGlobalParamAction(
        PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY, slot);
    (void) ok;
    chassisForceRedraw = 1u;
}
return;
}

if (keyHit(MENU_KEY_4))
{
    if (core2ParamSyncOwnerServiceMutationLocked() == 0u &&
        currentOptionIsValid(page))
    {
        chassisMenuState = CHASSIS_MENU_EDIT;
        chassisForceRedraw = 1u;
    }
}
else if (keyHit(MENU_KEY_2))
{
    chassisOptIdx--;
    clampSelAndScroll(&chassisOptIdx, page->optionCount);
    chassisForceRedraw = 1u;
}
else if (keyHit(MENU_KEY_3))
{
    chassisOptIdx++;
    clampSelAndScroll(&chassisOptIdx, page->optionCount);
    chassisForceRedraw = 1u;
}
else if (keyHit(MENU_KEY_1))
{
    enterChassisPageList(0u);
}
else if (keyHit(MENU_KEY_5))
{
    if (core2ParamSyncOwnerServiceMutationLocked() == 0u)
    {
        requestSettingsSave();
        chassisMenuState = CHASSIS_MENU_HOME;
    }
}
}

static void handleEditKeys(void)
{
    const chassisMenuPageStruct* page;
    uint8_t i;
    if (chassisPageIdx < 0 || (uint8_t) chassisPageIdx >= chassisMenuPageCount)
    {
        enterChassisPageList(0u);
        return;
    }
    page = &chassisMenuPages[chassisPageIdx];
    if (chassisOptIdx < 0 || (uint8_t) chassisOptIdx >= page->optionCount)
    {
        chassisMenuState = CHASSIS_MENU_OPTION_LIST;
        requestSurfaceReset();
        return;
    }
    i = (uint8_t) chassisOptIdx;
    if (!currentOptionIsValid(page))
    {
        chassisMenuState = CHASSIS_MENU_OPTION_LIST;
        requestSurfaceReset();
        return;
    }
}

if (core2ParamSyncOwnerServiceMutationLocked() != 0u)
{
    chassisMenuState = CHASSIS_MENU_OPTION_LIST;
    requestSurfaceReset();
}

```

```

    return;
}

if (keyHit(MENU_KEY_5))
{
    chassisMenuState = CHASSIS_MENU_OPTION_LIST;
    requestSurfaceReset();
    return;
}

/* 浮点项: core2 菜单按物理方向镜像, K4 +大 K3 +小 K2 -小 K1 -大。 */
{
    float delta = 0.0f;
    uint8_t hasDelta = 0u;
    if (keyHit(MENU_KEY_4)) { delta = page->stepL[i]; hasDelta = 1u; }
    else if (keyHit(MENU_KEY_3)) { delta = page->stepS[i]; hasDelta = 1u; }
    else if (keyHit(MENU_KEY_2)) { delta = -page->stepS[i]; hasDelta = 1u; }
    else if (keyHit(MENU_KEY_1)) { delta = -page->stepL[i]; hasDelta = 1u; }

    if (hasDelta && page->opt[i] != 0)
    {
        *page->opt[i] += delta;
        chassisMenuApplyAfterEdit();
        chassisForceRedraw = 1u;
    }
}

}

/* 前台输入阶段只消费按键和推进页面状态, 不触碰 IPS114 或 Flash。 */
void chassisMenuInputTick(void)
{
    uint8_t k;
    uint8_t paramLocked;
    core2RuntimeSnapshotStruct runtimeSnapshot;
    core2ParamGlobalUiSnapshot paramUi;

    core2RuntimeGetSnapshot(&runtimeSnapshot);
    core2RuntimeGetParamGlobalUi(&paramUi);

    if (paramUi.eventSequence != 0u &&
        paramUi.eventSequence != chassisParamGlobalLastEvent)
    {
        chassisParamGlobalLastEvent = paramUi.eventSequence;
        if (paramUi.state == CORE2_PARAM_GLOBAL_SUCCESS &&
            (paramUi.action == PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY ||
             paramUi.action == PARAM_SYNC_GLOBAL_ACTION_LOAD_RUNTIME))
        {
            paramFeedback(paramUi.action, paramUi.slot);
            chassisParamSlotInspectPending = 1u;
        }
        chassisForceRedraw = 1u;
    }
}

paramLocked = core2ParamSyncOwnerServiceMutationLocked();
if (paramLocked != paramLockWasActive)
{
    for (k = 0u; k < (uint8_t) MENU_KEY_COUNT; k++)
        menuKeyDiscardUntilRelease((menuKeyIndexEnum) k);
    if (paramLocked != 0u)
    {
        uint8_t cancelledAction = chassisFlashActionPending;
        cancelPendingBrightnessSave();
        if (chassisFlashActionPending !=
            CHASSIS_MENU_FLASH_ACTION_CLEAR_WAIT)
            chassisFlashActionPending = CHASSIS_MENU_FLASH_ACTION_NONE;
        if (cancelledAction == CHASSIS_MENU_FLASH_ACTION_SAVE ||
            cancelledAction == CHASSIS_MENU_FLASH_ACTION_CLEAR)
            localFlashFeedback(cancelledAction, 0u);
    }
    paramLockWasActive = paramLocked;
}
if (paramLocked == 0u && chassisMenuSynceWorkingCopy() != 0u)
    chassisForceRedraw = 1u;

if (chassisInitSummaryMs > 0u)
{
    goto input_done;
}
if (chassisInitSummaryLastSecond != 0xFFu)

```

```

{
    chassisInitSummaryLastSecond = 0xFFu;
    requestSurfaceReset();
}

/* 上电等待 RS-485 门控（A8，仿 core1 WAIT_LINK→HOME）：RS-485 未活且未进模式 4 → 停在等待屏、
* 吞按键不进菜单；链路活(core1 已连) 或 进模式 4(失链兜底, 车本地可用) → 迁 HOME。 */
if (chassisMenuState == CHASSIS_MENU_WAIT_RS485)
{
    /* 放行亮度键(K2-/K3+, 500ms 合并存 flash)，其余键吞掉(不进调参菜单) */
    if (paramLocked == 0u)
    {
        if (keyHit(MENU_KEY_2)) chassisMenuAdjustBrightness(-(int8_t) LIGHT_STRIP_STEP_COARSE);
        else if (keyHit(MENU_KEY_3)) chassisMenuAdjustBrightness(+ (int8_t) LIGHT_STRIP_STEP_COARSE);
    }
    for (k = 0u; k < (uint8_t) MENU_KEY_COUNT; k++) menuKeyClear((menuKeyIndexEnum) k);
    /* 阻塞等 core0+core1 全链路：core1(RS-485 活) && core0(经 core1 转告 cmd bit4) 都活 → 进 HOME；
    * 或 MODE4 兜底(RS-485 失链>2s 进本地 LoRa 独立)。否则停等待屏、显示等待对象(仿 core1)。 */
    if ((chassisLinkIsAlive() && chassisLinkGetCore0Alive()) || runtimeSnapshot.standalone)
    {
        chassisMenuState = CHASSIS_MENU_HOME;
        requestSurfaceReset();
    }
    goto input_done;
}

/* 填充反馈保持期：吞按键 + 维持填充，不渲染 */
if (chassisFeedbackHoldTicks > 0u)
{
    feedbackWasActive = 1u;
    for (k = 0u; k < (uint8_t) MENU_KEY_COUNT; k++)
        menuKeyDiscardUntilRelease((menuKeyIndexEnum) k);
    goto input_done;
}
if (feedbackWasActive)
{
    feedbackWasActive = 0u;
    chassisFeedbackRenderPending = 0u;
    requestSurfaceReset();
}

if (chassisParamSlotInspectPending != 0u &&
    chassisFeedbackHoldTicks == 0u &&
    core2RuntimeRequestGlobalParamAction(
        PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY,
        (uint8_t) chassisParamSlotSelection) != 0u)
    chassisParamSlotInspectPending = 0u;

/* 按键状态机（每轮跑，响应及时） */
switch (chassisMenuState)
{
    case CHASSIS_MENU_HOME:    handleHomeKeys(); break;
    case CHASSIS_MENU_PAGE_LIST: handlePageListKeys(); break;
    case CHASSIS_MENU_OPTION_LIST: handleOptionListKeys(); break;
    case CHASSIS_MENU_EDIT:    handleEditKeys(); break;
    default: break;
}

if (chassisFeedbackHoldTicks > 0u)
{
    feedbackWasActive = 1u;
    goto input_done;
}

input_done:
    return;
}

void chassisMenuServiceDeferred(void)
{
    serviceFlashAction();
    serviceBrightnessSave();
}

/* 前台渲染阶段：集中执行清屏/整屏反馈和 dirty 渲染。 */
void chassisMenuRenderTick(uint8_t displayDue)
{
    if (displayDue != 0u) chassisDirtyMask |= CHASSIS_MENU_DIRTY_HOME_FACTS;
    if (chassisInitSummaryMs > 0u)

```

```

{
    uint8_t second = (uint8_t) ((chassisInitSummaryMs + 999u) / 1000u);
    if ((second != chassisInitSummaryLastSecond) || chassisForceRedraw)
    {
        chassisRenderInitFaultSummary(chassisInitFailedMask,
            chassisInitUnverifiedMask, second);
        chassisInitSummaryLastSecond = second;
    }
    return;
}

if (chassisFeedbackHoldTicks > 0u)
{
    if (chassisFeedbackRenderPending != 0u)
    {
        ips114_full(chassisFeedbackColor);
        if (chassisFeedbackAction == PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY ||
            chassisFeedbackAction == PARAM_SYNC_GLOBAL_ACTION_LOAD_RUNTIME)
        {
            char slotText[16];
            ips114_set_color((chassisFeedbackColor == RGB565_GREEN) ?
                RGB565_BLACK : RGB565_WHITE, chassisFeedbackColor);
            ips114_show_string(80u, 42u,
                (chassisFeedbackAction ==
                    PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY) ?
                    "SAVE OK" : "LOAD OK");
            (void) snprintf(slotText, sizeof(slotText), "SLOT %u",
                (unsigned) chassisFeedbackSlot);
            ips114_show_string(88u, 66u, slotText);
            ips114_set_color(RGB565_GREEN, RGB565_BLACK);
        }
        else if (chassisFeedbackAction == CHASSIS_MENU_FEEDBACK_LOCAL_SAVE ||
            chassisFeedbackAction == CHASSIS_MENU_FEEDBACK_LOCAL_RESET)
        {
            const char *text = (chassisFeedbackAction ==
                CHASSIS_MENU_FEEDBACK_LOCAL_SAVE) ?
                (chassisFeedbackSlot ? "SAVE OK" : "SAVE FAIL") :
                (chassisFeedbackSlot ? "RESET OK" : "RESET FAIL");
            ips114_set_color(RGB565_BLACK, chassisFeedbackColor);
            ips114_show_string(72u, 50u, text);
            ips114_set_color(RGB565_GREEN, RGB565_BLACK);
        }
        chassisFeedbackAction = PARAM_SYNC_GLOBAL_ACTION_NONE;
        chassisFeedbackSlot = 0u;
        chassisFeedbackRenderPending = 0u;
    }
    return;
}

if (chassisSurfaceResetPending != 0u)
{
    chassisSurfaceResetPending = 0u;
    chassisMenuClearScreen();
}

if (chassisMenuState == CHASSIS_MENU_WAIT_RS485)
{
    if (displayDue || chassisForceRedraw) chassisRenderWaitRs485();
    chassisDirtyMask = 0u;
    return;
}

/* 渲染节流：仅 20ms 到点、事件 dirty 或强制重绘才画（防闪烁机制®）。
 * HOME 态实时刷新底盘速度，故 displayDue 即画；其余态靠 forceRedraw + 内部变化检测。 */
if (!displayDue && !chassisForceRedraw && (chassisDirtyMask == 0u)) return;

switch (chassisMenuState)
{
    case CHASSIS_MENU_HOME:    chassisRenderHome(); break;
    case CHASSIS_MENU_PAGE_LIST: chassisRenderPageList(); break;
    case CHASSIS_MENU_OPTION_LIST: chassisRenderOptionList(0u); break;
    case CHASSIS_MENU_EDIT:    chassisRenderOptionList(1u); break;
    default: break;
}
chassisDirtyMask = 0u;
}

void chassisMenuSetInitHealth(uint8_t failedMask, uint8_t unverifiedMask)

```

```
{
    chassisInitFailedMask = failedMask;
    chassisInitUnverifiedMask = unverifiedMask;
    if ((failedMask | unverifiedMask) != 0u)
    {
        chassisInitSummaryMs = 3000u;
        chassisInitSummaryLastSecond = 0xFFu;
        chassisForceRedraw = 1u;
    }
}
```

core2_cyt4bb7/project/code/app/chassisMenu.h

SHA-256

:

f050a29e4fb6965db7c719c386a9d21b49107ccdf888dc060c57b95072e109d3

代码 (c)

```
#ifndef _CHASSIS_MENU_H_
#define _CHASSIS_MENU_H_

/*
 * core2 调参/发车菜单 (cm_7_0, PRODUCTION 大类) 公共 API。仿 core0 menu.c。
 *
 * 单核: 菜单与控制同核(cm_7_0), 菜单只编辑私有 working copy; 提交、保存和恢复默认
 * 均通过 core2Runtime 语义命令完成, 不直接持有活动参数或 Flash 后端。
 * 操作: 5 键(KEY_1..5) 状态机 发车页/页列表/选项列表/编辑; 8×16 渲染防闪烁。
 * 参数: 速度/电流/yaw PID + LoRa 映射 + 导航 + 灯条亮度; 运行源由 carRunState 显示, 不在菜单本地切换。
 *
 * 集成(core0Loop.c, PRODUCTION):
 * processInit → chassisMenuInit() (在 chassisParamsInit + 控制 owner init 之后)
 * pitProcess0 → chassisMenuPitTick() (按键扫描/反馈/亮度保存倒计时, 1ms 轻量节拍)
 * whileProcess → chassisMenuInputTick() (前台按键/状态, 不触屏)
 * → chassisMenuRenderTick() (前台渲染, displayFlag 节流)
 */

#include <stdint.h>

void chassisMenuInit(void);
void chassisMenuSetInitHealth(uint8_t failedMask, uint8_t unverifiedMask);
void chassisMenuPitTick(void); /* ISR 1ms: 按键扫描 + 计时器递减, 不触屏/不写 Flash */
void chassisMenuInputTick(void); /* 前台: 按键/页面状态, 不触碰 IPS114 或 Flash */
void chassisMenuRenderTick(uint8_t displayDue); /* 前台: 渲染(displayDue=20ms 到点) */
void chassisMenuServiceDeferred(void); /* 前台尾部: 显式动作及亮度合并保存 */

#endif /* _CHASSIS_MENU_H_ */
```

core2_cyt4bb7/project/code/app/chassisMenuInternal.h

SHA-256

:

98fd90bbfb0063fc28a7c56d8100b07847bf3ef8f8fbc7eaf1554e0a38f0ce1a

代码 (c)

```
#ifndef _CHASSIS_MENU_INTERNAL_H_
#define _CHASSIS_MENU_INTERNAL_H_

/*
 * chassisMenu 模块内部共享头 (不对外, 仅 chassisMenu*.c 引用)。仿 core0 menuInternal.h 简化:
 * chassisMenu.c 状态唯一定义 + 公共 API(chassisMenuInit/PitTick/Tick) + 状态机分发
 * chassisMenuModel.c 页表构建 + applyAfterEdit 热生效 + 保存/恢复(转调 chassisParamsFlash*)
 * chassisMenuRender.c IPS114 8×16 绘制原语 + 各状态渲染
 *
 * 菜单只编辑私有 working copy; 每次业务编辑经 core2RuntimeSubmitParams 一次提交,
 * 不直接持有活动仓库或 owner 内部状态。
 */

#include <stdint.h>

#include "chassisParams.h"
```

```

/* =====
* 屏幕几何 (8×16 字体, IPS114 240×135 横向, 仿 core0)
* ===== */
#define LCD_W      (240u)
#define LCD_H      (135u)
#define FONT_W     (8u)
#define FONT_H     (16u)
#define ROW_Y(r)   ((uint16_t)((r) * FONT_H))
#define COL_X(c)   ((uint16_t)((c) * FONT_W))
#define VISIBLE_ROWS (7u) /* 标题占第 0 行, 列表可见 7 行 */

#define SCROLLBAR_W (4u)
#define SCROLLBAR_X ((uint16_t)(LCD_W - 1u - SCROLLBAR_W))

/* 保存绿/恢复红 整屏填充反馈保持时长。chassisMenuPitTick 由 ISR 1ms 节拍递减。 */
#define CHASSIS_MENU_FEEDBACK_HOLD_TICKS (300u)

#define CHASSIS_MENU_DIRTY_SURFACE      (1u << 0)
#define CHASSIS_MENU_DIRTY_TITLE       (1u << 1)
#define CHASSIS_MENU_DIRTY_SELECTION   (1u << 2)
#define CHASSIS_MENU_DIRTY_HOME_BRIGHTNESS (1u << 3)
#define CHASSIS_MENU_DIRTY_HOME_FACTS   (1u << 4)
#define CHASSIS_MENU_DIRTY_FEEDBACK     (1u << 5)
#define CHASSIS_MENU_DIRTY_ALL          (0x3Fu)

/* =====
* 菜单数据结构
* ===== */
#define CHASSIS_MENU_MAX_OPTIONS (16u)
#define CHASSIS_MENU_MAX_PAGES   (12u) /* 四轮 + Common/Yaw/Heading/LoRa/Nav = 9 页, 留余量 */

typedef struct
{
    const char* name;
    uint8_t optionCount;
    const char* optName[CHASSIS_MENU_MAX_OPTIONS];
    float* opt[CHASSIS_MENU_MAX_OPTIONS]; /* 仅指向菜单 working copy 字段 */
    float stepS[CHASSIS_MENU_MAX_OPTIONS];
    float stepL[CHASSIS_MENU_MAX_OPTIONS];
} chassisMenuPageStruct;

typedef enum
{
    CHASSIS_MENU_HOME = 0,
    CHASSIS_MENU_PAGE_LIST,
    CHASSIS_MENU_OPTION_LIST,
    CHASSIS_MENU_EDIT,
    CHASSIS_MENU_WAIT_RS485 /* 上电等待 RS-485 连 core1: 未连且未进模式 4 时停此态, 不进菜单(A8) */
} chassisMenuStateEnum;

typedef enum
{
    CHASSIS_MENU_FLASH_ACTION_NONE = 0,
    CHASSIS_MENU_FLASH_ACTION_SAVE,
    CHASSIS_MENU_FLASH_ACTION_CLEAR,
    CHASSIS_MENU_FLASH_ACTION_CLEAR_WAIT
} chassisMenuFlashActionEnum;

#define CHASSIS_MENU_FEEDBACK_LOCAL_SAVE (0x80u)
#define CHASSIS_MENU_FEEDBACK_LOCAL_RESET (0x81u)

/* =====
* 跨文件共享状态 (唯一定义在 chassisMenu.c)
* ===== */
typedef struct
{
    chassisMenuStateEnum state;
    int8_t pageldx;
    int8_t optldx;
    uint8_t scrollTop;
    uint8_t forceRedraw;
    uint8_t dirtyMask;
    volatile uint16_t feedbackHoldTicks;
    uint8_t feedbackWasActive;
    uint8_t surfaceResetPending;
    uint8_t feedbackRenderPending;
    uint16_t feedbackColor;
    uint8_t feedbackAction;

```

```

uint8_t feedbackSlot;
uint8_t paramSlotInspectPending;
uint32_t paramGlobalLastEvent;
volatile uint16_t brightnessSaveDelayMs;
uint8_t brightnessSavePending;
uint8_t flashActionPending;
chassisMenuPageStruct pages[CHASSIS_MENU_MAX_PAGES];
uint8_t pageCount;
chassisParamStruct workingParams;
uint32_t workingRevision;
uint8_t initFailedMask;
uint8_t initUnverifiedMask;
volatile uint16_t initSummaryMs;
uint8_t initSummaryLastSecond;
uint8_t paramSlotsPageIndex;
float paramSlotSelection;
} chassisMenuContextStruct;

extern chassisMenuContextStruct chassisMenuContext;

/* 仅限本模块三个实现文件的私有兼容别名，调用方看不到 context。 */
#define chassisMenuState      (chassisMenuContext.state)
#define chassisPageIdx        (chassisMenuContext.pageIdx)
#define chassisOptIdx         (chassisMenuContext.optIdx)
#define chassisScrollTop      (chassisMenuContext.scrollTop)
#define chassisForceRedraw    (chassisMenuContext.forceRedraw)
#define chassisDirtyMask      (chassisMenuContext.dirtyMask)
#define chassisFeedbackHoldTicks (chassisMenuContext.feedbackHoldTicks)
#define feedbackWasActive      (chassisMenuContext.feedbackWasActive)
#define chassisSurfaceResetPending (chassisMenuContext.surfaceResetPending)
#define chassisFeedbackRenderPending (chassisMenuContext.feedbackRenderPending)
#define chassisFeedbackColor    (chassisMenuContext.feedbackColor)
#define chassisFeedbackAction    (chassisMenuContext.feedbackAction)
#define chassisFeedbackSlot      (chassisMenuContext.feedbackSlot)
#define chassisParamSlotInspectPending (chassisMenuContext.paramSlotInspectPending)
#define chassisParamGlobalLastEvent (chassisMenuContext.paramGlobalLastEvent)
#define chassisBrightnessSaveDelayMs (chassisMenuContext.brightnessSaveDelayMs)
#define chassisBrightnessSavePending (chassisMenuContext.brightnessSavePending)
#define chassisFlashActionPending (chassisMenuContext.flashActionPending)
#define chassisMenuPages        (chassisMenuContext.pages)
#define chassisMenuPageCount    (chassisMenuContext.pageCount)
#define chassisMenuWorkingParams (chassisMenuContext.workingParams)
#define chassisMenuWorkingRevision (chassisMenuContext.workingRevision)
#define chassisInitFailedMask    (chassisMenuContext.initFailedMask)
#define chassisInitUnverifiedMask (chassisMenuContext.initUnverifiedMask)
#define chassisInitSummaryMs      (chassisMenuContext.initSummaryMs)
#define chassisInitSummaryLastSecond (chassisMenuContext.initSummaryLastSecond)
#define chassisParamSlotsPageIndex (chassisMenuContext.paramSlotsPageIndex)
#define chassisParamSlotSelection (chassisMenuContext.paramSlotSelection)

/* =====
* 跨文件函数原型
* ===== */
/* chassisMenuModel.c */
void chassisMenuBuildPages(void);
uint8_t chassisMenuSyncWorkingCopy(void);
void chassisMenuApplyAfterEdit(void);
uint8_t chassisMenuSaveSettings(void); /* 经 runtime 保存当前活动参数 */
uint8_t chassisMenuClearFlash(void); /* runtime 恢复源码默认 (HOME K1 长按) */

/* chassisMenuRender.c */
void chassisRenderHome(void);
void chassisRenderPageList(void);
void chassisRenderOptionList(uint8_t editing);
void chassisRenderWaitRs485(void); /* 等待 RS-485 连 core1 等待屏(A8) */
void chassisRenderInitFaultSummary(uint8_t failedMask, uint8_t unverifiedMask, uint8_t secondsLeft);
void chassisMenuClearScreen(void); /* 渲染阶段切状态全清 + forceRedraw */

/* chassisMenuInput.c 已并入 chassisMenu.c (core2 简化, 按键 handler 在状态机文件) */

#endif /* _CHASSIS_MENU_INTERNAL_H_ */

```


core2_cyt4bb7/project/code/app/chassisMenuModel.c

SHA-256

:

da9511b318c065e85178a1ec3f30967c3278ed1793f54133d4575e56a72d185b

代码 (c)

```
#include "chassisMenuInternal.h"

#include "chassisParams.h"
#include "core0Loop.h"
#include "lightStrip.h"

/* ---- 页表构建 helpers ---- */
static chassisMenuPageStruct* curPage = 0;

static void addPage(const char* name)
{
    if (chassisMenuPageCount >= CHASSIS_MENU_MAX_PAGES) { curPage = 0; return; }
    curPage = &chassisMenuPages[chassisMenuPageCount];
    chassisMenuPageCount++;
    curPage->name = name;
    curPage->optionCount = 0u;
}

/* 浮点项: working copy 值指针 + 小/大步长。重建分类由 runtime 对参数差异自行判定。 */
static void addFloat(const char* name, float* val, float stepS, float stepL)
{
    uint8_t i;
    if (curPage == 0) return;
    i = curPage->optionCount;
    if (i >= CHASSIS_MENU_MAX_OPTIONS) return;
    curPage->optName[i] = name;
    curPage->opt[i] = val;
    curPage->stepS[i] = stepS;
    curPage->stepL[i] = stepL;
    curPage->optionCount++;
}

static void addWheelPidPage(
    const char* name,
    chassisPidGainStruct* speedGain,
    chassisPidGainStruct* currentGain)
{
    addPage(name);
    addFloat("Spd Kp", &speedGain->kp, 0.5f, 5.0f);
    addFloat("Spd Ki", &speedGain->ki, 0.002f, 0.01f);
    addFloat("Spd Kd", &speedGain->kd, 0.05f, 0.5f);
    addFloat("Cur Kp", &currentGain->kp, 0.0001f, 0.001f);
    addFloat("Cur Ki", &currentGain->ki, 0.00005f, 0.0005f);
    addFloat("Cur Kd", &currentGain->kd, 0.0001f, 0.001f);
}

void chassisMenuBuildPages(void)
{
    chassisParamsGetActive(&chassisMenuWorkingParams);
    chassisMenuWorkingRevision = chassisParamsGetRevision();
    chassisMenuPageCount = 0u;
    curPage = 0;

    /* 按轮分页: 每页一个轮的全部 PID (速度 kp/ki/kd + 电流 kp/ki/kd) 。 runtime 自动判定需安全重建。
     * 速度/电流环现均四轮独立 (chassisParams 的 speedXx 与 currentXx 各四组) ; 四轮共享积分钳位归 Common 页。 */
    addWheelPidPage("FL Wheel", &chassisMenuWorkingParams.speedFl, &chassisMenuWorkingParams.currentFl);
    addWheelPidPage("FR Wheel", &chassisMenuWorkingParams.speedFr, &chassisMenuWorkingParams.currentFr);
    addWheelPidPage("BL Wheel", &chassisMenuWorkingParams.speedBl, &chassisMenuWorkingParams.currentBl);
    addWheelPidPage("BR Wheel", &chassisMenuWorkingParams.speedBr, &chassisMenuWorkingParams.currentBr);

    /* 公共页: 四轮共享的积分钳位; runtime 自动在安全态重建 PID 应用新钳位。 */
    addPage("Common");
    addFloat("Spd Imax", &chassisMenuWorkingParams.speedIntegralMax, 10.0f, 100.0f);
    addFloat("Cur Imax", &chassisMenuWorkingParams.currentIntegralMax, 10.0f, 100.0f);

    /* 角速度环 PID */
    addPage("Yaw Rate");
}
```

```

addFloat("Yaw Kp", &chassisMenuWorkingParams.yawRate.kp, 0.1f, 1.0f);
addFloat("Yaw Ki", &chassisMenuWorkingParams.yawRate.ki, 0.01f, 0.1f);
addFloat("Yaw Kd", &chassisMenuWorkingParams.yawRate.kd, 0.01f, 0.1f);
addFloat("Yaw Imax", &chassisMenuWorkingParams.yawRateIntegralMax, 0.5f, 5.0f);
addFloat("Yaw OutLim", &chassisMenuWorkingParams.yawRateOutputLimit, 1.0f, 5.0f);

addPage("Heading");
addFloat("Heading Kp", &chassisMenuWorkingParams.headingKp, 0.1f, 1.0f);
addFloat("Heading Wlim", &chassisMenuWorkingParams.headingRateLimit, 0.1f, 0.5f);

/* LoRa 映射（runtime 同次提交热生效；V Max 单位 m/s，同时是摇杆满量程和 LoRa 模式限速） */
addPage("LoRa Map");
addFloat("V Max", &chassisMenuWorkingParams.loraVMaxMps, 0.05f, 0.2f);
addFloat("Wz Max", &chassisMenuWorkingParams.loraWzMaxRadps, 0.1f, 0.5f);
addFloat("Deadzone", &chassisMenuWorkingParams.loraStickDeadzone, 5.0f, 20.0f);

/* 导航 */
addPage("Navigation");
addFloat("NavCorrKp", &chassisMenuWorkingParams.navCorrectionKp, 0.01f, 0.1f);
addFloat("NavCorrLim", &chassisMenuWorkingParams.navCorrectionLimit, 0.5f, 2.0f);
addFloat("NavErrLim", &chassisMenuWorkingParams.navErrorLimitPulses, 50.0f, 200.0f);

chassisParamSlotsPageIndex = chassisMenuPageCount;
chassisParamSlotSelection = 1.0f;
addPage("Param Slots");
addFloat("Slot", &chassisParamSlotSelection, 1.0f, 1.0f);
}

uint8_t chassisMenuSyncWorkingCopy(void)
{
    uint32_t revision = chassisParamsGetRevision();
    if (revision == 0u || revision == chassisMenuWorkingRevision) return 0u;
    chassisParamsGetActive(&chassisMenuWorkingParams);
    chassisMenuWorkingRevision = revision;
    return 1u;
}

/* 编辑后只提交一次完整 working copy；runtime 自行判定动态热生效或安全态重建。 */
void chassisMenuApplyAfterEdit(void)
{
    chassisParamsSanitize(&chassisMenuWorkingParams);
    (void) core2RuntimeSubmitParams(&chassisMenuWorkingParams);
    chassisParamsGetActive(&chassisMenuWorkingParams);
    chassisMenuWorkingRevision = chassisParamsGetRevision();
}

uint8_t chassisMenuSaveSettings(void)
{
    chassisParamsGetActive(&chassisMenuWorkingParams);
    chassisMenuWorkingRevision = chassisParamsGetRevision();
    return core2RuntimeRequestSave();
}

/* runtime 写入源码默认值并热应用（HOME K1 长按）；库槽保持不变。 */
uint8_t chassisMenuClearFlash(void)
{
    uint8_t ok = core2RuntimeRequestFactoryReset();
    chassisParamsGetActive(&chassisMenuWorkingParams);
    chassisMenuWorkingRevision = chassisParamsGetRevision();
    return ok;
}

```

core2_cyt4bb7/project/code/app/chassisMenuRender.c

SHA-256

:

cc3bb4aef337888543c853049cff696224b132c0f17afc4ad6ed33488f630e95

代码 (c)

```

#include "chassisMenuInternal.h"

#include "chassisApplication.h"
#include "core0Loop.h" /* runtime snapshot：HOME 显示真实运行态 */
#include "chassisLink.h" /* chassisLinkIsAlive/GetCore0Alive：等待屏显示 core1/core0 双链路状态 */

```

```

#include "IMU.h" /* HOME YAWrad 行通过应用快照观察 1ms 姿态解算结果 */
#include "loraNavControl.h"
#include "loraRemote.h"
#include "menuFloatDisplayInternal.h"
#include "motorControl.h" /* HOME 失控锁定提示 motorControllsRunawayLatched */
#include "zf_device_ips114.h"
#include "zf_common_font.h"

#include <stdio.h>
#include <string.h>

/* 运行态显示名（与 runtime snapshot/carRunStateEnum 同序：0=WAIT 1=485 2=LoRa(联机) 3=LoRa(脱机兜底)）。
 * 模式 3(LORA_LINKED)与模式 4(MODE4)对操作员同为 LoRa 双摇杆遥控，故同显 "LoRa"（用户要求脱机兜底也显 LoRa）。*/
static const char* const chassisRunStateNames[4] = {"WAIT", "485 ", "LoRa", "LoRa"};

/* 三态列表行配色（统一规范 docs/MENU_UI_CONVENTION.md §3：名/值始终不同前景色）：
 * 普通(非选中) 底黑 名绿 值白
 * 选中(未编辑) 底青 名黑 值蓝
 * 编辑中 底黄 名黑 值红 */
#define MENU_NORM_NAME_FG (RGB565_GREEN)
#define MENU_NORM_VAL_FG (RGB565_WHITE)
#define MENU_SEL_BG (RGB565_CYAN)
#define MENU_SEL_NAME_FG (RGB565_BLACK)
#define MENU_SEL_VAL_FG (RGB565_BLUE)
#define MENU_EDIT_BG (RGB565_YELLOW)
#define MENU_EDIT_NAME_FG (RGB565_BLACK)
#define MENU_EDIT_VAL_FG (RGB565_RED)

#define MENU_TEXT_X (COL_X(1u))
#define MENU_MARK_X (COL_X(1u))
#define MENU_ITEM_X (COL_X(2u))
#define MENU_VAL_X (COL_X(13u))
#define MENU_TITLE_Y (1u)

/* 标题与列表均以 RGB565 矩形覆盖内框；列表右界避开滚动槽和右外框。*/

typedef enum { OPT_ROW_NORMAL = 0, OPT_ROW_SELECTED, OPT_ROW_EDITING } optRowStyleEnum;

static void fillSolidRect(uint16_t left, uint16_t top,
    uint16_t width, uint16_t height, uint16_t color);

static void drawFrameBorder(uint16_t color)
{
    fillSolidRect(0u, 0u, LCD_W, 1u, color);
    fillSolidRect(0u, (uint16_t)(LCD_H - 1u), LCD_W, 1u, color);
    fillSolidRect(0u, 0u, 1u, LCD_H, color);
    fillSolidRect((uint16_t)(LCD_W - 1u), 0u, 1u, LCD_H, color);
}

static void drawSectionDivider(uint16_t y, uint16_t color)
{
    fillSolidRect(1u, y, (uint16_t)(LCD_W - 2u), 1u, color);
}

/* =====
 * 防闪烁机制（仿 core0 menuRender，硬要求）：
 * ① 静态元素首帧绘制 + 变化检测（lastXxx 标记，不变跳过重绘）
 * ② 刷新节流（chassisMenuRenderTick 用 displayDue gate）
 * ③ 局部刷新 + 定宽覆盖（行清只覆盖内框内容区，不擦滚动条/外框；EDIT 值变只重绘当前值）
 * ④ 切状态才 ips114_clear（chassisMenuClearScreen）
 * ===== */

/* 上次渲染快照（变化检测，避免无变化重绘 → 防闪烁核心）*/
static chassisMenuStateEnum lastState = (chassisMenuStateEnum) 0xFF;
static int8_t lastPageIdx = -1;
static int8_t lastOptIdx = -1;
static uint8_t lastScrollTop = 0xFFu;
static uint8_t lastBrightness = 0xFFu;
static uint16_t lastHomeTitleColor = 0xFFFFu;
static uint16_t lastHomeFrameColor = 0xFFFFu;
static char homeTextCache[8][14];
static uint16_t homeTextColor[8];
static int32_t homeNumberCache[6];
static uint16_t homeNumberColor[6];
static char homeFooterCache[40];
static uint16_t homeFooterColor = 0xFFFFu;
static uint8_t lastWaitC1 = 0xFFu;
static uint8_t lastWaitC0Display = 0xFFu;
static uint8_t paramSlotsSurfaceValid;

```

```

static uint8_t paramSlotsLastSelection = 0xFFu;
static core2ParamGlobalUiSnapshot paramSlotsLastUi;

#define HOME_TEXT_COLUMNS (13u)
#define HOME_LEFT_VALUE_X (38u)
#define HOME_RIGHT_VALUE_X (158u)
#define HOME_TILE_MAX_WIDTH (96u)
#define HOME_TILE_MAX_HEIGHT (12u)
#define SOLID_FILL_BUFFER_PIXELS (240u)
static uint16_t homeTileBuffer[HOME_TILE_MAX_WIDTH * HOME_TILE_MAX_HEIGHT];
static uint16_t solidFillBuffer[SOLID_FILL_BUFFER_PIXELS];

static void fillSolidRect(uint16_t left, uint16_t top,
    uint16_t width, uint16_t height, uint16_t color)
{
    uint16_t rowsPerTransfer;
    uint16_t rows;
    uint16_t y = top;
    uint16_t i;

    if ((width == 0u) || (height == 0u) || (width > SOLID_FILL_BUFFER_PIXELS)) return;
    rowsPerTransfer = (uint16_t) (SOLID_FILL_BUFFER_PIXELS / width);
    if (rowsPerTransfer == 0u) rowsPerTransfer = 1u;
    for (i = 0u; i < SOLID_FILL_BUFFER_PIXELS; i +=) solidFillBuffer[i] = color;
    while (height > 0u)
    {
        rows = (height < rowsPerTransfer) ? height : rowsPerTransfer;
        ips114_show_rgb565_image(left, y, solidFillBuffer,
            width, rows, width, rows, 0u);
        y = (uint16_t) (y + rows);
        height = (uint16_t) (height - rows);
    }
}

static void clearPixelRect(uint16_t left, uint16_t top,
    uint16_t right, uint16_t bottom, uint16_t color)
{
    uint16_t width = (uint16_t) (right - left + 1u);
    uint16_t height = (uint16_t) (bottom - top + 1u);
    uint32_t count = (uint32_t) width * height;
    uint32_t i;

    if ((width > HOME_TILE_MAX_WIDTH) || (height > HOME_TILE_MAX_HEIGHT)) return;
    for (i = 0u; i < count; i++)
    {
        homeTileBuffer[i] = color;
    }
    ips114_show_rgb565_image(left, top, homeTileBuffer,
        width, height, width, height, 0u);
}

static void drawHomeTextSlot(uint8_t slot, uint16_t x, uint16_t y,
    const char *value, uint16_t color)
{
    char padded[HOME_TEXT_COLUMNS + 1u];
    uint8_t i = 0u;

    if ((slot >= 8u) || (value == 0)) return;
    if ((strcmp(homeTextCache[slot], value) == 0) &&
        (homeTextColor[slot] == color)) return;

    memset(padded, ' ', HOME_TEXT_COLUMNS);
    while ((i < HOME_TEXT_COLUMNS) && (value[i] != '\0'))
    {
        padded[i] = value[i];
        i++;
    }
    padded[HOME_TEXT_COLUMNS] = '\0';
    ips114_set_color(color, RGB565_BLACK);
    ips114_show_string(x, y, padded);
    strncpy(homeTextCache[slot], value, HOME_TEXT_COLUMNS);
    homeTextCache[slot][HOME_TEXT_COLUMNS] = '\0';
    homeTextColor[slot] = color;
}

static int32_t quantizeTenth(float value)
{
    float scaled = value * 10.0f;
    return (int32_t) ((scaled >= 0.0f) ? (scaled + 0.5f) : (scaled - 0.5f));
}

```

```

}

static void drawHomeNumberSlot(uint8_t slot, uint16_t x, uint16_t y,
    float value, uint8_t valid, uint16_t color)
{
    int32_t quantized;
    if (slot >= 6u) return;
    quantized = (valid != 0u) ? quantizeTenth(value) : INT32_MIN;
    if ((homeNumberCache[slot] == quantized) &&
        (homeNumberColor[slot] == color)) return;

    clearPixelRect(x, y, (uint16_t) (x + 77u), (uint16_t) (y + 7u), RGB565_BLACK);
    ips114_set_color(color, RGB565_BLACK);
    if (valid != 0u)
    {
        ips114_show_float(x, y, (double) value, 4u, 1u);
    }
    else
    {
        ips114_show_string(x, y, "--");
    }
    homeNumberCache[slot] = quantized;
    homeNumberColor[slot] = color;
}

static void drawHomeFooter(const char *text, uint16_t color)
{
    char padded[40];
    uint8_t i = 0u;
    if (text == 0) return;
    if ((strcmp(homeFooterCache, text) == 0) && (homeFooterColor == color)) return;
    memset(padded, '\0', 39u);
    while ((i < 39u) && (text[i] != '\0'))
    {
        padded[i] = text[i];
        i++;
    }
    padded[39] = '\0';
    ips114_set_color(color, RGB565_BLACK);
    ips114_show_string(2u, 122u, padded);
    strncpy(homeFooterCache, text, 39u);
    homeFooterCache[39] = '\0';
    homeFooterColor = color;
}

static void drawBrightnessBar(uint8_t brightness)
{
    uint8_t filled;
    uint8_t cell;
    uint16_t x;
    uint16_t y;
    const uint16_t x0 = 32u;
    const uint16_t y0 = 106u;
    if (brightness > 100u) brightness = 100u;
    if (lastBrightness == brightness) return;
    filled = (uint8_t) (((uint16_t) brightness * 12u + 50u) / 100u);
    ips114_set_font(IPS114_6X8_FONT);
    ips114_set_color(RGB565_WHITE, RGB565_BLACK);
    ips114_show_string(2u, y0, "LIT");
    for (y = 0u; y < HOME_TILE_MAX_HEIGHT; y++)
    {
        for (x = 0u; x < HOME_TILE_MAX_WIDTH; x++)
        {
            {
                cell = (uint8_t) (x / 8u);
                homeTileBuffer[(uint32_t) y * HOME_TILE_MAX_WIDTH + x] =
                    ((x % 8u) < 7u) ?
                        ((cell < filled) ? RGB565_CYAN : RGB565_GRAY) : RGB565_BLACK;
            }
        }
    }
    ips114_show_rgb565_image(x0, y0, homeTileBuffer,
        HOME_TILE_MAX_WIDTH, HOME_TILE_MAX_HEIGHT,
        HOME_TILE_MAX_WIDTH, HOME_TILE_MAX_HEIGHT, 0u);
    ips114_set_color(RGB565_WHITE, RGB565_BLACK);
    ips114_show_string(132u, y0, "K2- K3+");
    lastBrightness = brightness;
}

/* 整行铺底右界：有滚动条→滚动条左(SCROLLBAR_X-1)；无→右内框(LCD_W-2)。渲染前按 total 设。 */
static uint16_t listFillRightX = (uint16_t) (LCD_W - 2u);

```

```

static void fillBottomGutter(uint8_t row, uint16_t color)
{
    uint16_t y = ROW_Y(row);
    if (row != VISIBLE_ROWS) return;
    fillSolidRect(1u, (uint16_t) (y + FONT_H), listFillRightX,
        (uint16_t) ((LCD_H - 1u) - (y + FONT_H)), color);
}

/* 列表空行清理（铺黑到右界，含右侧空隙→消选中条移走后残留的高亮色）。*/
static void clearListRow(uint8_t row)
{
    uint16_t y = ROW_Y(row);
    ips114_set_color(0x000000, 0x000000);
    fillSolidRect(1u, y, listFillRightX, FONT_H, 0x000000);
    fillBottomGutter(row, 0x000000);
}

/* 选中/编辑整行铺底色，顶到右界但不触碰左右外框。
* 命名与 core0/core1 menuRender 的 fillListRowBg 统一（core2 用 row+ROW_Y 入参）。*/
static void fillListRowBg(uint8_t row, uint16_t fg, uint16_t bg)
{
    uint16_t y = ROW_Y(row);
    ips114_set_color(fg, bg);
    fillSolidRect(1u, y, listFillRightX, FONT_H, bg);
    fillBottomGutter(row, bg);
}

void chassisMenuClearScreen(void)
{
    ips114_clear();
    chassisForceRedraw = 1u;
    /* 复位快照，强制下次全画 */
    lastState = (chassisMenuStateEnum) 0xFF;
    lastPageIdx = -1;
    lastOptIdx = -1;
    lastScrollTop = 0xFFu;
    lastBrightness = 0xFFu;
    lastHomeTitleColor = 0xFFFFFu;
    lastHomeFrameColor = 0xFFFFFu;
    memset(homeTextCache, 0, sizeof(homeTextCache));
    memset(homeTextColor, 0xFF, sizeof(homeTextColor));
    memset(homeFooterCache, 0, sizeof(homeFooterCache));
    homeFooterColor = 0xFFFFFu;
    lastWaitC1 = 0xFFu;
    lastWaitC0Display = 0xFFu;
    paramSlotsSurfaceValid = 0u;
    paramSlotsLastSelection = 0xFFu;
    memset(&paramSlotsLastUi, 0xFF, sizeof(paramSlotsLastUi));
    {
        uint8_t i;
        for (i = 0u; i < 6u; i++)
        {
            homeNumberCache[i] = INT32_MIN;
            homeNumberColor[i] = 0xFFFFFu;
        }
    }
}

static void drawTitleEx(const char* title, uint16_t fg, uint16_t bg)
{
    char bounded[40];
    size_t len = strlen(title);
    uint16_t x;

    if (len > 39u) len = 39u;
    memcpy(bounded, title, len);
    bounded[len] = '\0';
    x = (uint16_t) ((LCD_W - (uint16_t) (len * 6u)) / 2u);
    if (x < 2u) x = 2u;

    fillSolidRect(1u, 1u, (uint16_t) (LCD_W - 2u), 15u, bg);
    ips114_set_font(IPS114_6X8_FONT);
    ips114_set_color(fg, bg);
    ips114_show_string(x, 4u, bounded);
    ips114_set_font(IPS114_8X16_FONT);
    ips114_set_color(0x0000FF, 0x000000);
}

```

```

/* 标题栏（静态：仅状态/页变化时重绘） */
static void drawTitle(const char* title)
{
    drawTitleEx(title, RGB565_BLACK, RGB565_GREEN);
}

/* 列表右侧滚动滑块：自清槽（先清 SCROLLBAR_X..LCD_W-2 槽区，消旧滑块残留），再画轨道 + 滑块。
* 不触右外框(x=LCD_W-1)。仅在 total>visible 时绘制。 */
static void drawScrollbar(uint8_t total, uint8_t top)
{
    uint16_t trackTop = ROW_Y(1u);
    uint16_t trackBot = (uint16_t) (LCD_H - 2u);
    uint16_t trackH;
    uint16_t thumbH;
    uint16_t thumbY;

    if (total <= VISIBLE_ROWS) return;
    trackH = (uint16_t) (trackBot - trackTop);

    /* 自清槽：清整个槽区(轨道 + 旧滑块) → 消滚动时旧滑块横线残留 */
    fillSolidRect(SCROLLBAR_X, trackTop,
        (uint16_t) ((LCD_W - 2u) - SCROLLBAR_X + 1u), trackH, RGB565_BLACK);
    fillSolidRect(SCROLLBAR_X, trackTop, 1u, trackH, RGB565_GREEN);
    fillSolidRect((uint16_t) (LCD_W - 2u), trackTop, 1u, trackH, RGB565_GREEN);

    thumbH = (uint16_t) ((uint32_t) trackH * VISIBLE_ROWS / total);
    if (thumbH < 6u) thumbH = 6u;
    thumbY = (uint16_t) (trackTop + (uint32_t) (trackH - thumbH) * top / (uint16_t) (total - VISIBLE_ROWS));

    if ((uint16_t) (thumbY + thumbH) > trackBot)
        thumbH = (uint16_t) (trackBot - thumbY);
    fillSolidRect((uint16_t) (SCROLLBAR_X + 1u), thumbY,
        (uint16_t) ((LCD_W - 3u) - (SCROLLBAR_X + 1u) + 1u), thumbH, RGB565_GREEN);
}

/* ---- 发车页（HOME）：底盘状态总览（仿 core0 renderHome，换底盘量） ---- */
void chassisRenderHome(void)
{
    chassisApplicationSnapshotStruct chassisSnapshot;
    motorControlSnapshotStruct motorSnapshot;
    loraRemoteSnapshotStruct loraSnapshot;
    loraNavControlSnapshotStruct navSnapshot;
    core2RuntimeSnapshotStruct runtimeSnapshot;
    uint16_t titleColor;
    uint16_t frameColor;
    const char *modeText;
    const char *gateText;
    const char *causeText;
    const char *navText;
    const char *wheelText;
    uint8_t motionValid;
    uint8_t yawValid;
    uint8_t runaway;
    uint8_t redrawStatic = (uint8_t) (chassisForceRedraw || lastState != CHASSIS_MENU_HOME);

    chassisApplicationGetSnapshot(&chassisSnapshot);
    motorControlGetSnapshot(&motorSnapshot);
    loraRemoteGetSnapshot(&loraSnapshot);
    loraNavControlGetSnapshot(&navSnapshot);
    core2RuntimeGetSnapshot(&runtimeSnapshot);
    runaway = motorSnapshot.runawayLatched;
    motionValid = (uint8_t) ((chassisSnapshot.valid != 0u) &&
        (runtimeSnapshot.valid != 0u));
    yawValid = (uint8_t) ((chassisSnapshot.valid != 0u) &&
        (chassisSnapshot.attitudeHealthy != 0u) &&
        (chassisSnapshot.attitudeDataFresh != 0u));
    titleColor = (runtimeSnapshot.estop || runaway) ? RGB565_RED :
        ((runtimeSnapshot.runState == 0u) || runtimeSnapshot.standalone || runtimeSnapshot.rejoin ||
        (runtimeSnapshot.commandFresh == 0u)) ? RGB565_YELLOW : RGB565_GREEN;
    frameColor = titleColor;

    if (redrawStatic || (lastHomeTitleColor != titleColor))
    {
        drawTitleEx(" CORE2 CHASSIS ", RGB565_BLACK, titleColor);
        lastHomeTitleColor = titleColor;
    }
    if (redrawStatic)
    {
        static const char *const leftLabels[7] =

```

```

    {"MODE", "CMDX", "CMDY", "CMDW", "ACTX", "ACTY", "YAWrad"};
static const char *const rightLabels[7] =
    {"LINK", "GATE", "LORA", "NAV", "WHEEL", "STATE", "CAUSE"};
uint8_t row;
ips114_set_font(IPS114_6X8_FONT);
ips114_set_color(RGB565_GRAY, RGB565_BLACK);
for (row = 0u; row < 7u; row++)
{
    uint16_t y = (uint16_t) (18u + (uint16_t) row * 12u);
    ips114_show_string(2u, y, leftLabels[row]);
    ips114_show_string(122u, y, rightLabels[row]);
}
ips114_draw_line(119u, 17u, 119u, 103u, RGB565_GRAY);
drawSectionDivider(104u, RGB565_GRAY);
drawSectionDivider(119u, RGB565_GRAY);
}

modeText = (runtimeSnapshot.runState < 4u) ?
    chassisRunStateNames[runtimeSnapshot.runState] : "?";
gateText = runtimeSnapshot.estop ? "ESTOP" :
    (!runtimeSnapshot.enabled ? "BLOCK" :
    (runtimeSnapshot.launched ? "GO" : "ARM WAIT"));
causeText = runaway ? "RUNAWAY" :
    (runtimeSnapshot.estop ? "ESTOP" :
    (runtimeSnapshot.rejoin ? "REJOIN" :
    ((runtimeSnapshot.commandFresh == 0u) ? "CMD STALE" : "--")));
switch (navSnapshot.state)
{
    case LORA_NAV_RECORDING: navText = "RECORD"; break;
    case LORA_NAV_READY: navText = "READY"; break;
    case LORA_NAV_REPLAYING: navText = "REPLAY"; break;
    case LORA_NAV_DONE: navText = "DONE"; break;
    case LORA_NAV_ERROR: navText = "ERROR"; break;
    default: navText = "IDLE"; break;
}
wheelText = runaway ? "SAT LOCK" : "NORMAL";

ips114_set_font(IPS114_6X8_FONT);
drawHomeTextSlot(0u, HOME_LEFT_VALUE_X, 18u, modeText,
    runtimeSnapshot.standalone ? RGB565_YELLOW : RGB565_WHITE);
drawHomeTextSlot(1u, HOME_RIGHT_VALUE_X, 18u,
    (runtimeSnapshot.core1LinkReady && runtimeSnapshot.core0ViaCore1Ready) ?
    "C1 OK C0 OK" :
    (runtimeSnapshot.core1LinkReady ? "C1 OK C0 WAIT" : "C1 LOST C0 --"),
    (runtimeSnapshot.core1LinkReady && runtimeSnapshot.core0ViaCore1Ready) ?
    RGB565_GREEN : RGB565_RED);
drawHomeTextSlot(2u, HOME_RIGHT_VALUE_X, 30u, gateText,
    (runtimeSnapshot.estop || !runtimeSnapshot.enabled) ? RGB565_RED :
    (runtimeSnapshot.launched ? RGB565_GREEN : RGB565_YELLOW));
drawHomeTextSlot(3u, HOME_RIGHT_VALUE_X, 42u,
    !loraSnapshot.linkAlive ? "LOST" :
    (loraSnapshot.unlocked ? "OK UNLOCK" : "OK LOCK"),
    !loraSnapshot.linkAlive ? RGB565_RED :
    (loraSnapshot.unlocked ? RGB565_GREEN : RGB565_YELLOW));
drawHomeTextSlot(4u, HOME_RIGHT_VALUE_X, 54u, navText,
    (navSnapshot.state == LORA_NAV_ERROR) ? RGB565_RED :
    ((navSnapshot.state == LORA_NAV_REPLAYING) ? RGB565_CYAN : RGB565_WHITE));
drawHomeTextSlot(5u, HOME_RIGHT_VALUE_X, 66u, wheelText,
    runaway ? RGB565_RED : RGB565_WHITE);
drawHomeTextSlot(6u, HOME_RIGHT_VALUE_X, 78u,
    runaway ? "LOCKED" : "CONTROL OK", runaway ? RGB565_RED : RGB565_GREEN);
drawHomeTextSlot(7u, HOME_RIGHT_VALUE_X, 90u, causeText,
    (strcmp(causeText, "--") == 0) ? RGB565_WHITE : RGB565_RED);

drawHomeNumberSlot(0u, HOME_LEFT_VALUE_X, 30u,
    chassisSnapshot.targetCommand.vx, motionValid, RGB565_WHITE);
drawHomeNumberSlot(1u, HOME_LEFT_VALUE_X, 42u,
    chassisSnapshot.targetCommand.vy, motionValid, RGB565_WHITE);
drawHomeNumberSlot(2u, HOME_LEFT_VALUE_X, 54u,
    chassisSnapshot.targetCommand.wz, motionValid, RGB565_WHITE);
drawHomeNumberSlot(3u, HOME_LEFT_VALUE_X, 66u,
    chassisSnapshot.actualSpeed.vx, motionValid, RGB565_GREEN);
drawHomeNumberSlot(4u, HOME_LEFT_VALUE_X, 78u,
    chassisSnapshot.actualSpeed.vy, motionValid, RGB565_GREEN);
drawHomeNumberSlot(5u, HOME_LEFT_VALUE_X, 90u,
    chassisSnapshot.imu.yawAngle, yawValid, RGB565_GREEN);
drawBrightnessBar((uint8_t) chassisMenuWorkingParams.lightStripBrightness);

ips114_set_font(IPS114_6X8_FONT);

```



```

if (motorControlsRunawayLatched())
{
    drawHomeFooter("RUNAWAY K5 LONG: UNLOCK", RGB565_RED);
}
else
{
    drawHomeFooter((strcmp(causeText, "--") == 0) ?
        "K4 MENU K1 SAVE K1L CLEAR" : causeText,
        (strcmp(causeText, "--") == 0) ? RGB565_GRAY : RGB565_RED);
}
if (lastHomeFrameColor != frameColor)
{
    drawFrameBorder(frameColor);
    lastHomeFrameColor = frameColor;
}
ips114_set_font(IPS114_8X16_FONT);
ips114_set_color(RGB565_GREEN, RGB565_BLACK);

lastState = CHASSIS_MENU_HOME;
chassisForceRedraw = 0u;
chassisDirtyMask = 0u;
}

/* ---- 等待 RS-485 连接 (WAIT_RS485) : 上电未连 core1 且未进模式 4 时的等待屏 (仿 core1 renderWaitLink) ----
* 红字告警 + 提示; 链路活或进模式 4 后由 chassisMenuInputTick 状态机迁 HOME。静态屏, 变化检测防闪烁。 */
void chassisRenderWaitRs485(void)
{
    uint8_t redrawStatic = (uint8_t) (chassisForceRedraw || lastState != CHASSIS_MENU_WAIT_RS485);
    uint8_t c1 = chassisLinkIsAlive(); /* core1 RS-485 链路活 */
    uint8_t c0 = chassisLinkGetCore0Alive(); /* core0 链路活(经 core1 转告); RS-485 断则 0 */
    uint8_t c0Display = c0 ? 2u : (c1 ? 1u : 0u); /* 0=-- 1=WAIT 2=OK */

    if (redrawStatic)
    {
        drawTitleEx(" WAIT LINK ", RGB565_BLACK, RGB565_RED);
        ips114_set_color(RGB565_GREEN, RGB565_BLACK);
        ips114_show_string(MENU_TEXT_X, ROW_Y(1u), "core1(485) :");
        ips114_show_string(MENU_TEXT_X, ROW_Y(3u), "core0(c1) :");
        drawSectionDivider(104u, RGB565_RED);
        drawSectionDivider(119u, RGB565_RED);
    }

    /* 动态: 两链路状态(绿=OK/红=WAIT, 定宽覆盖) + 亮度。RS-485 断时 core0 未知显 "--"。 */
    if ((redrawStatic != 0u) || (lastWaitC1 != c1))
    {
        ips114_set_color(c1 ? RGB565_GREEN : RGB565_RED, RGB565_BLACK);
        ips114_show_string(COL_X(13u), ROW_Y(1u), c1 ? "OK " : "WAIT");
        lastWaitC1 = c1;
    }
    if ((redrawStatic != 0u) || (lastWaitC0Display != c0Display))
    {
        ips114_set_color(c0 ? RGB565_GREEN : RGB565_RED, RGB565_BLACK);
        ips114_show_string(COL_X(13u), ROW_Y(3u), c0 ? "OK " : (c1 ? "WAIT" : "-- "));
        lastWaitC0Display = c0Display;
    }
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
    drawBrightnessBar((uint8_t) chassisMenuWorkingParams.lightStripBrightness);
    if (redrawStatic != 0u)
    {
        ips114_show_string(MENU_TEXT_X, 121u, "WAIT: local LoRa fallback");
        drawFrameBorder(RGB565_YELLOW);
    }

    lastState = CHASSIS_MENU_WAIT_RS485;
    chassisForceRedraw = 0u;
}

void chassisRenderInitFaultSummary(uint8_t failedMask, uint8_t unverifiedMask, uint8_t secondsLeft)
{
    static const char *const names[2] = {"IMU", "CHASSIS ALG"};
    uint8_t row = 0u;
    uint8_t i;
    if (secondsLeft >= 3u)
    {
        chassisMenuClearScreen();
        drawTitleEx((failedMask != 0u) ? "HW INIT FAIL" : "HW INIT CHECK",
            RGB565_BLACK, (failedMask != 0u) ? RGB565_RED : RGB565_YELLOW);
        for (i = 0u; i < 2u; i++)
        {

```

```

uint8_t bit = (uint8_t) (1u << i);
if ((failedMask & bit) != 0u)
{
    ips114_set_color(RGB565_RED, RGB565_BLACK);
    ips114_show_string(8u, (uint16_t) (34u + row * 16u), names[i]);
    ips114_show_string(128u, (uint16_t) (34u + row * 16u), "FAIL ");
    row++;
}
}
for (i = 0u; i < 2u; i++)
{
    uint8_t bit = (uint8_t) (1u << i);
    if (((unverifiedMask & bit) != 0u) && ((failedMask & bit) == 0u))
    {
        ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
        ips114_show_string(8u, (uint16_t) (34u + row * 16u), names[i]);
        ips114_show_string(128u, (uint16_t) (34u + row * 16u), "CHECK");
        row++;
    }
}
drawFrameBorder((failedMask != 0u) ? RGB565_RED : RGB565_YELLOW);
}
ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
ips114_show_string(8u, 114u, "CONTINUE 0s");
ips114_show_uint(80u, 114u, secondsLeft, 1u);
chassisForceRedraw = 0u;
}

/* ---- 页列表 (PAGE_LIST) ---- */
void chassisRenderPageList(void)
{
    uint8_t i;
    uint8_t redrawStatic = (uint8_t) (lastState != CHASSIS_MENU_PAGE_LIST);
    uint8_t changed = (uint8_t) (redrawStatic
        || lastPageIdx != chassisPageIdx || lastScrollTop != chassisScrollTop);
    uint8_t selectionOnly = (uint8_t) (!(redrawStatic) &&
        (lastScrollTop == chassisScrollTop) && (lastPageIdx != chassisPageIdx));

    /* 无结构变化(displayDue 无变化)直接跳过，避免空刷滚动条/边框造成抖动 */
    if (!changed) return;

    listFillRightX = (chassisMenuPageCount > VISIBLE_ROWS) ? (uint16_t) (SCROLLBAR_X - 1u) : (uint16_t) (LCD_W - 2u);

    if (redrawStatic)
    {
        drawTitle(" SELECT PAGE ");
    }

    for (i = 0u; i < VISIBLE_ROWS; i++)
    {
        uint8_t idx = (uint8_t) (chassisScrollTop + i);
        uint8_t row = (uint8_t) (i + 1u);
        if ((selectionOnly != 0u) && ((int8_t) idx != chassisPageIdx) &&
            ((int8_t) idx != lastPageIdx)) continue;
        if (idx >= chassisMenuPageCount)
        {
            clearListRow(row);
            continue;
        }
        if ((int8_t) idx == chassisPageIdx)
        {
            fillListRowBg(row, MENU_SEL_NAME_FG, MENU_SEL_BG); /* 选中整行青底 */
            ips114_show_string(MENU_MARK_X, ROW_Y(row), ">");
            ips114_show_string(MENU_ITEM_X, ROW_Y(row), chassisMenuPages[idx].name);
            ips114_set_color(RGB565_GREEN, RGB565_BLACK);
        }
        else
        {
            clearListRow(row); /* 普通行：黑底绿名（页列表无数值） */
            ips114_show_string(MENU_ITEM_X, ROW_Y(row), chassisMenuPages[idx].name);
        }
    }

    if (selectionOnly == 0u)
    {
        drawScrollbar(chassisMenuPageCount, chassisScrollTop);
        drawFrameBorder(RGB565_GREEN);
    }
    lastState = CHASSIS_MENU_PAGE_LIST;
}

```

```

        lastPageIdx = chassisPageIdx;
        lastScrollTop = chassisScrollTop;
        chassisForceRedraw = 0u;
    }

/* 把某项的当前浮点值格式化显示，x 起、定宽覆盖。
 * 颜色由调用方(行样式 helper)预设；本函数只在当前前景/背景下清+画，不隐式假设颜色。 */
static void drawOptValue(const chassisMenuPageStruct* page, uint8_t i, uint16_t x, uint16_t y)
{
    float val = (page->opt[i] != 0) ? *page->opt[i] : 0.0f;
    menuFloatDisplayShow(x, y,
        (double) val,
        menuFloatDisplayDecimalsForValueAndStep((double) val, page->stepS[i]));
}

/* 三态行样式表：底色/名色/值色/行首标记。 */
static void optRowStyle(optRowStyleEnum style, uint16_t* bg, uint16_t* nameFg, uint16_t* valFg, const char** mark)
{
    switch (style)
    {
        case OPT_ROW_SELECTED:
            *bg = MENU_SEL_BG; *nameFg = MENU_SEL_NAME_FG; *valFg = MENU_SEL_VAL_FG; *mark = ">"; break;
        case OPT_ROW_EDITING:
            *bg = MENU_EDIT_BG; *nameFg = MENU_EDIT_NAME_FG; *valFg = MENU_EDIT_VAL_FG; *mark = "***"; break;
        case OPT_ROW_NORMAL:
            default:
                *bg = RGB565_BLACK; *nameFg = MENU_NORM_NAME_FG; *valFg = MENU_NORM_VAL_FG; *mark = " "; break;
    }
}

/* 画整行选项（铺底色 + 标记 + 名 + 值，名/值不同前景色）。结构性重用。 */
static void drawOptionRow(uint8_t row, const chassisMenuPageStruct* page, uint8_t idx, optRowStyleEnum style)
{
    uint16_t y = ROW_Y(row);
    uint16_t bg;
    uint16_t nameFg;
    uint16_t valFg;
    const char* mark;
    optRowStyle(style, &bg, &nameFg, &valFg, &mark);

    /* 整行铺底色（只写内框）+ 标记 + 名(名色) */
    ips114_set_color(nameFg, bg);
    fillSolidRect(1u, y, listFillRightX, FONT_H, bg);
    fillBottomGutter(row, bg);
    ips114_show_string(MENU_MARK_X, y, mark);
    ips114_show_string(MENU_ITEM_X, y, page->optName[idx]);

    /* 值(值色，与名不同) */
    ips114_set_color(valFg, bg);
    drawOptValue(page, idx, MENU_VAL_X, y);

    ips114_set_color(RGB565_GREEN, RGB565_BLACK); /* 复原默认 */
}

/* EDIT 态值变局部刷新：仅重绘当前选中项数值(黄底红字)，不动整行/名/边框/滚动条 → 消整页清屏闪烁。 */
static void drawEditValueOnly(const chassisMenuPageStruct* page, uint8_t idx, uint8_t row)
{
    ips114_set_color(MENU_EDIT_VAL_FG, MENU_EDIT_BG);
    drawOptValue(page, idx, MENU_VAL_X, ROW_Y(row));
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
}

/* ---- 选项列表 (OPTION_LIST) / 编辑 (EDIT, editing=1 编辑选中项) ---- */
void chassisRenderOptionList(uint8_t editing)
{
    {
        const chassisMenuPageStruct* page;
        uint8_t i;
        uint8_t enteringList;
        uint8_t editTransition;
        uint8_t structural;
        uint8_t selectionOnly;

        if (chassisPageIdx < 0 || (uint8_t) chassisPageIdx >= chassisMenuPageCount) return;
        page = &chassisMenuPages[chassisPageIdx];
        if ((editing == 0u) &&
            ((uint8_t) chassisPageIdx == chassisParamSlotsPageIndex))
        {
            char line[32];
            core2ParamGlobalUiSnapshot globalUi;

```

```

uint8_t slotUiState;
uint8_t fullRedraw;
core2RuntimeGetParamGlobalUi(&globalUi);
slotUiState = globalUi.slotUiState;
if (chassisParamSlotInspectPending != 0u ||
    globalUi.action != PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY ||
    globalUi.slot != (uint8_t) chassisParamSlotSelection)
    slotUiState = PARAM_SYNC_SLOT_UI_CHECKING;
listFillRightX = (uint16_t) (LCD_W - 2u);
fullRedraw = (uint8_t) (paramSlotsSurfaceValid == 0u);
if (fullRedraw != 0u)
{
    drawTitle("PARAM SLOTS");
    for (i = 1u; i < 8u; i++) clearListRow(i);
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
    ips114_show_string(8u, ROW_Y(5u), "K2/K3 SLOT");
    ips114_show_string(8u, ROW_Y(6u), "K1L SAVE K4 LOAD");
    ips114_show_string(8u, ROW_Y(7u), "K5 BACK");
    drawFrameBorder(RGB565_GREEN);
    paramSlotsSurfaceValid = 1u;
}
if (fullRedraw != 0u || paramSlotsLastSelection !=
    (uint8_t) chassisParamSlotSelection)
{
    fillListRowBg(1u, MENU_SEL_NAME_FG, MENU_SEL_BG);
    (void) snprintf(line, sizeof(line), "> SELECT    SLOT %u",
        (unsigned) ((uint8_t) chassisParamSlotSelection));
    ips114_show_string(MENU_MARK_X, ROW_Y(1u), line);
    paramSlotsLastSelection = (uint8_t) chassisParamSlotSelection;
}
if (fullRedraw != 0u || paramSlotsLastUi.slotUiState != slotUiState)
{
    static const char *const slotText[] = {
        "UNKNOWN", "CHECKING", "FULL", "EMPTY", "PARTIAL",
        "INVALID", "UNAVAILABLE"};
    uint8_t state = slotUiState;
    if (state > PARAM_SYNC_SLOT_UI_UNAVAILABLE)
        state = PARAM_SYNC_SLOT_UI_UNKNOWN;
    clearListRow(2u);
    (void) snprintf(line, sizeof(line), "SLOT: %s", slotText[state]);
    ips114_set_color(state == PARAM_SYNC_SLOT_UI_FULL ? RGB565_GREEN :
        ((state == PARAM_SYNC_SLOT_UI_EMPTY) ? RGB565_CYAN : RGB565_YELLOW),
        RGB565_BLACK);
    ips114_show_string(24u, ROW_Y(2u), line);
}
globalUi.slotUiState = slotUiState;
if (globalUi.state == CORE2_PARAM_GLOBAL_QUEUED)
    (void) snprintf(line, sizeof(line), "TASK: QUEUED");
else if (globalUi.state == CORE2_PARAM_GLOBAL_PROGRESS)
    (void) snprintf(line, sizeof(line), "RUN: C%u PH%u",
        (unsigned) globalUi.target, (unsigned) globalUi.phase);
else if (globalUi.state == CORE2_PARAM_GLOBAL_SUCCESS)
    (void) snprintf(line, sizeof(line), "TASK: SUCCESS");
else if (globalUi.state == CORE2_PARAM_GLOBAL_FAILED)
    (void) snprintf(line, sizeof(line), "FAIL: C%u E%u",
        (unsigned) globalUi.target, (unsigned) globalUi.error);
else
    (void) snprintf(line, sizeof(line), "TASK: READY");
if (fullRedraw != 0u || memcmp(&paramSlotsLastUi, &globalUi,
    sizeof(globalUi)) != 0)
{
    clearListRow(3u);
    ips114_set_color(globalUi.state == CORE2_PARAM_GLOBAL_SUCCESS ? RGB565_GREEN :
        (globalUi.state == CORE2_PARAM_GLOBAL_FAILED ? RGB565_RED : RGB565_CYAN),
        RGB565_BLACK);
    ips114_show_string(24u, ROW_Y(3u), line);
}
paramSlotsLastUi = globalUi;
lastState = CHASSIS_MENU_OPTION_LIST;
lastOptIdx = chassisOptIdx;
lastScrollTop = chassisScrollTop;
chassisForceRedraw = 0u;
return;
}

/* 结构变化判定(不依赖 forceRedraw): 从别态进入 / 进出编辑 / 换选中项 / 滚动 → 整页重绘。
* 编辑态值变(同态同项同滚动)不算结构变化 → 走值局部刷新(drawEditValueOnly),
* 消除原'| editing' 每帧清整页 7 行造成的闪烁(forceRedraw 仅 gate 是否被调, 不再决定整页重绘)。*/
enteringList = (uint8_t) (lastState == CHASSIS_MENU_HOME

```

```

    || lastState == CHASSIS_MENU_PAGE_LIST
    || lastState == (chassisMenuStateEnum) 0xFF);
editTransition = (uint8_t) ((lastState == CHASSIS_MENU_EDIT) != (editing != 0u));
structural = (uint8_t) (enteringList || editTransition
    || lastOptIdx != chassisOptIdx || lastScrollTop != chassisScrollTop);
selectionOnly = (uint8_t) (!(enteringList) && !(editTransition) &&
    (lastScrollTop == chassisScrollTop) && (lastOptIdx != chassisOptIdx));

if (structural)
{
    listFillRightX = (page->optionCount > VISIBLE_ROWS) ? (uint16_t) (SCROLLBAR_X - 1u) : (uint16_t) (LCD_W - 2u);
    if (enteringList)
    {
        drawTitle(page->name);
    }
    for (i = 0u; i < VISIBLE_ROWS; i++)
    {
        uint8_t idx = (uint8_t) (chassisScrollTop + i);
        uint8_t row = (uint8_t) (i + 1u);
        if ((selectionOnly != 0u) && ((int8_t) idx != chassisOptIdx) &&
            ((int8_t) idx != lastOptIdx)) continue;
        if (idx >= page->optionCount)
        {
            clearListRow(row);
            continue;
        }
        {
            uint8_t selected = (uint8_t) ((int8_t) idx == chassisOptIdx);
            optRowStyleEnum style = (!selected) ? OPT_ROW_NORMAL
                : (editing ? OPT_ROW_EDITING : OPT_ROW_SELECTED);
            drawOptionRow(row, page, idx, style);
        }
    }
    if (selectionOnly == 0u)
    {
        drawScrollbar(page->optionCount, chassisScrollTop);
        drawFrameBorder(RGB565_GREEN);
    }
}
else if (editing)
{
    /* 值局部刷新：仅当前选中项数值（选中项在屏行 = optIdx - scrollTop + 1） */
    uint8_t selRow = (uint8_t) ((uint8_t) chassisOptIdx - chassisScrollTop + 1u);
    drawEditValueOnly(page, (uint8_t) chassisOptIdx, selRow);
}
/* else: OPTION_LIST 非结构变化(displayDue 无变化) → 跳过 */

lastState = editing ? CHASSIS_MENU_EDIT : CHASSIS_MENU_OPTION_LIST;
lastOptIdx = chassisOptIdx;
lastScrollTop = chassisScrollTop;
chassisForceRedraw = 0u;
}

```

core2_cyt4bb7/project/code/app/core0Loop.c

SHA-256

:

9bdd06d7c2f1da60d1a83f3d3ff2787a20692812c6128b5cc69bba06e9623d11

代码 (c)

```

#include "core0Loop.h"

#include <math.h>
#include <stddef.h>
#include <string.h>

#include "chassisApplication.h"
#include "chassisParams.h"
#include "configChassisPhysical.h"
#include "configChassisRemote.h"
#include "configCore2Build.h"
#include "currentSense.h"
#include "encoder.h"
#include "IMU.h"
#include "chassisMenu.h"

```

```

#include "lightStrip.h"
#include "menuKey.h"
#include "motor.h"
#include "motorControl.h"
#include "paramSyncOwnerService.h"
#include "paramSyncMotorSafety.h"
#include "chassisLink.h"
#include "coreDataTransfer.h"
#include "loraNavControl.h"
#include "buzzer.h"
#include "loraRemote.h"
#include "tuneMode.h"
#include "zf_device_lora3a22.h"
#include "zf_driver_pit.h"
#include "zf_common_interrupt.h"

/*
 * =====
 * core2 工程配置宏 — 操作指南
 * =====
 *
 * [1] 时序参数（通常不需修改，除非换电机/编码器）
 * -----
 * SAMPLE_PERIOD_US      (50) PIT_CH1 触发周期 (μs)
 *          电流 ADC 过采样：每 50μs 采一次样
 *          PIT_CH1/CH0 共用 CPUIntIdx2，同优先级顺序执行
 *
 * CONTROL_PERIOD_US      (500) PIT_CH0 触发周期 (μs) — 系统“心跳”
 *          每拍执行：电流反馈更新 + 电流 PID + PWM 输出
 *
 * SPEED_LOOP_DIVIDER      (4) 速度环分频：500μs × 4 = 2ms
 *          每 4 拍执行：编码器采样 + 速度 PID + 运动学更新
 *          改此值后需重新整定速度 PID 增益
 *
 * IPS114_DISPLAY_PERIOD_MS(20) LCD HOME 遥测刷新周期 (ms)
 *          不进 ISR，由 whileProcess 消费标志位
 *
 * IPS114_LINE_HEIGHT      (16) LCD 行距(像素)，配合 8×16 横屏字体
 *
 * MOTOR_TUNE_STEP_HIGH/LOW/INTERVAL_S CASCADE/CURRENT 目标阶跃测试：目标每 INTERVAL_S 秒
 *          在高/低值间切换(INTERVAL_S=0 → 恒高值)，单位随模式(速度/电流)，
 *          便于上位机看闭环阶跃响应；低值可为负→双向阶跃
 *
 * [2] 工作模式选择 — 两级体系（详见 configCore2Build.h 与 PROJECT.md §6）
 * -----
 * 第一级（编译期）CORE2_BUILD_MODE：
 * CORE2_BUILD_PRODUCTION 生产固件——空地系统运行态机 carRunState 单点决策(据 RS-485 链路
 * 活否 + core1 下发的 sysByte 源位 + 失链时长，每拍重算)，替代原静态 controlSource 选源：
 * FOLLOW_485 (链路活+源=485, 模式 1/2)：消费 core1 经 RS-485 下发的 vx/vy/wz + 发车门控 + yaw 闭环驱动底盘。
 * LORA_LINKED(链路活+源=LoRa, 模式 3) / MODE4(RS-485 失链 >2s 兜底)：本地 LoRa3A22 双摇杆遥控
 *          + NAV 轨迹录制/回放 (loraNavControl)。左摇杆前后=vx/左右=vy，右摇杆左右=wz；
 *          S3+S4 chord 解锁⇌锁定（锁定即急停），S4 单按=录制，S3 单按=回放；链路丢失 >500ms
 *          软急停；蜂鸣器 P07_2 提示。
 * WAIT_LINK (RS-485 失链未超时)：软急停等待 core1（与原失链急停同语义）。
 * CORE2_BUILD_TUNE 调参固件——编译期选 MOTOR_TUNE_MODE 子模式：
 *
 * 0 MOTOR_TUNE_CASCADE 级联 PID 调试模式
 *
 * 单轮选中调试：编码器 → 速度 PID → 电流 PID → PWM。 |
 * IPS114 显示选中轮子的 6 项数据 (tgt/fbk/spdOut/ |
 * curFbk/curOut)。用途：4 轮逐一标定 PID 参数。 |
 *
 * 1 MOTOR_TUNE_LORA_TEST LoRa 遥控器原始数据监视模式
 *
 * 仅接收 LoRa 帧，IPS114 显示 4 摇杆 + 4 按键 + |
 * 4 拨码开关 + state_flag 原始值。 |
 * 不控制电机、不跑 PID。 |
 * 用途：验证 LoRa 链路, 标定摇杆中位/极性/范围。 |
 *
 * 2 MOTOR_TUNE_CURRENT 电流环单环调参
 * 3 MOTOR_TUNE_OPEN_LOOP 开环 PWM 测试
 *
 * 切换模式：改 configCore2Build.h 的 CORE2_BUILD_MODE / MOTOR_TUNE_MODE，重新编译刷写。
 * =====
 */

```

```

static void core2RuntimeBuildParamViews(
    const chassisParamStruct *params,
    motorControlParamViewStruct *motor,
    chassisApplicationParamViewStruct *chassis,
    loraRemoteParamViewStruct *remote,
    loraNavControlParamViewStruct *nav);
static void core2RuntimeApplyControllerParams(const chassisParamStruct *params);
#if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
static void core2RuntimeApplyDynamicParams(const chassisParamStruct *params);
#endif

/* 空地系统运行态机（单点决策，替代原静态 controlSource 分支）： 每拍 pitProcess0 据
* RS-485 链路活否 + cmd.sysByte 源位 + 失链时长重算，无锁存——RS-485 恢复自动退出模式 4。
* WAIT_LINK : RS-485 失链但未超时——软急停等待 core1
* FOLLOW_485 : RS-485 活 + 源=485(模式 1/2)——跟随 core1 指令(发车门控)
* LORA_LINKED: RS-485 活 + 源=LoRa(模式 3)——本地 LoRa 双摇杆遥控
* MODE4      : RS-485 失链 >AG_CAR_RS485_WAIT_MS——本地兜底, LoRa 独立(车单独可用)
* controlSource 静态镜像已删：源不再由本地 flash 决定，改由 RS-485 链路 + core0 下发的
* sysByte 源位主导；Core2 不再保留第二套控制源参数。 */
typedef enum
{
    CAR_RUN_WAIT_LINK = 0,
    CAR_RUN_FOLLOW_485 = 1,
    CAR_RUN_LORA_LINKED = 2,
    CAR_RUN_MODE4 = 3
} carRunStateEnum;

typedef struct
{
    volatile uint8_t initReady;
    volatile uint8_t displayDue;
    uint32_t displayCountUs;
    volatile uint32_t loopMs;
    uint16_t loopMsAccumUs;
    uint8_t keyTickDivider;
    uint8_t imuTickDivider;
    volatile uint8_t paramApplyPending;
    volatile carRunStateEnum runState;
    volatile uint32_t runStateSeq;
    volatile uint32_t rs485DeadMs;
    volatile uint8_t rs485ControlEnabled;
    volatile uint8_t rs485ControlEstop;
    volatile uint8_t rs485ControlLaunched;
    uint32_t controlPublishSeq;
    volatile uint32_t lastCommSeq;
    uint32_t lastCommTickMs;
    uint8_t commSeen;
    volatile uint8_t commFresh;
    uint8_t speedLoopDivider;
    uint8_t rs485YawDivider;
    uint16_t rejoinReadyMs;
    uint8_t rejoinActive;
    uint8_t rejoinBarrierTicks;
    uint8_t ownerValidPrev;
    uint8_t loraLinkPrev;
    uint8_t loraRearmRequired;
    uint8_t loraLaunchLowSeen;
    volatile uint8_t effectiveLoraLaunchGate;
    uint32_t rejoinFenceSeq;
    uint8_t criticalInitFailed;
    uint8_t initFailedMask;
    uint8_t imuReady;
    uint8_t paramGlobalStateValid;
    paramSyncGlobalState paramGlobalState;
    paramSyncGlobalRequest paramGlobalRequest;
    uint32_t paramGlobalRequestCoordinatorBoot;
    uint32_t paramGlobalRequestSequence;
    uint8_t paramGlobalRequestValid;
    uint8_t paramGlobalUiState;
    uint8_t paramGlobalUiPhase;
    uint8_t paramGlobalUiTarget;
    uint8_t paramGlobalUiError;
    uint8_t paramGlobalUiAction;
    uint8_t paramGlobalUiSlot;
    uint8_t paramGlobalSlotUiState;
    uint32_t paramGlobalUiEventSequence;
} core2ControlRuntimeContextStruct;

static core2ControlRuntimeContextStruct core2ControlRuntime = {0};

```

```

static void cancelPendingParamGlobalRequest(uint8_t error)
{
    core2ParamGlobalUiSnapshot ui;
    ui.state = core2ControlRuntime.paramGlobalUiState;
    ui.phase = core2ControlRuntime.paramGlobalUiPhase;
    ui.target = core2ControlRuntime.paramGlobalUiTarget;
    ui.error = core2ControlRuntime.paramGlobalUiError;
    ui.action = core2ControlRuntime.paramGlobalUiAction;
    ui.slot = core2ControlRuntime.paramGlobalUiSlot;
    ui.slotUiState = core2ControlRuntime.paramGlobalSlotUiState;
    ui.eventSequence = core2ControlRuntime.paramGlobalUiEventSequence;
    if (core2ParamGlobalUiCancel(
        &core2ControlRuntime.paramGlobalRequestValid, &ui, error) == 0u)
        return;
    core2ControlRuntime.paramGlobalRequestCoordinatorBoot = 0u;
    core2ControlRuntime.paramGlobalUiState = ui.state;
    core2ControlRuntime.paramGlobalUiPhase = ui.phase;
    core2ControlRuntime.paramGlobalUiTarget = ui.target;
    core2ControlRuntime.paramGlobalUiError = ui.error;
    core2ControlRuntime.paramGlobalUiAction = ui.action;
    core2ControlRuntime.paramGlobalUiSlot = ui.slot;
    core2ControlRuntime.paramGlobalSlotUiState = ui.slotUiState;
    core2ControlRuntime.paramGlobalUiEventSequence++;
    if (core2ControlRuntime.paramGlobalUiEventSequence == 0u)
        core2ControlRuntime.paramGlobalUiEventSequence = 1u;
}

#if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)

/* FOLLOW 485 控制态 (pitProcess0 控制分支写, whileProcess 遥测读) :
 * bit 语义供遥测 status 上报 core1——使能态 / 急停态 / 已发车态。 */
static void rs485ControlStop(void)
{
    chassisApplicationSetYawRateEnable(0u);
    core2ControlRuntime.speedLoopDivider = 0u;
    core2ControlRuntime.rs485YawDivider = 0u;
    chassisApplicationStop();
    motorControlStopAll();
    chassisApplicationUpdateFeedback();
    /* 遥测态: 急停/等待——使能关、急停置位、未发车 (原两调用点各自重复, 收口于此)。 */
    core2ControlRuntime.rs485ControlEnabled = 0u;
    core2ControlRuntime.rs485ControlEstop = 1u;
    core2ControlRuntime.rs485ControlLaunched = 0u;
}

static void rs485ControlRunSpeedLoop(const chassisLinkCommandStruct *cmd)
{
    chassisMotionCommandStruct motion;

    motion.vx = cmd->vx;
    motion.vy = cmd->vy;
    motion.wz = cmd->wz;

    /* 与 LoRa/NAV 保持相同顺序: 先刷新本拍反馈, 再开启 yaw owner 并应用命令。
     * 姿态不健康时只撤销 yaw 闭环修正并上报独立故障, 合法底盘命令仍继续执行。 */
    chassisApplicationUpdateFeedback();
    chassisApplicationSetYawRateEnable(1u);
    chassisApplicationApplyCommandWithLimit(
        motion,
        CHASSIS_RS485_MAX_LINEAR_SPEED_MPS,
        CHASSIS_RS485_MAX_YAW_RATE_RADPS);

    /* 先下发一次限幅后的目标, 让 yaw PID 读取同一 targetCommand;
     * 6ms tick 产生新 yawCorrection 后, 本拍再重算一次轮速。 */
    core2ControlRuntime.rs485YawDivider++;
    if (core2ControlRuntime.rs485YawDivider >= YAW_RATE_LOOP_DIVIDER)
    {
        core2ControlRuntime.rs485YawDivider = 0u;
        chassisApplicationYawRateTick();
        chassisApplicationApplyCommandWithLimit(
            motion,
            CHASSIS_RS485_MAX_LINEAR_SPEED_MPS,
            CHASSIS_RS485_MAX_YAW_RATE_RADPS);
    }

    motorControlSpeedTick();
}
#endif

```



```

void processInit(void)
{
    chassisParamStruct activeParams;
    uint8_t chassisInitResult;
    uint8_t imuInitResult;

    core2ControlRuntime.runState = CAR_RUN_WAIT_LINK;
    core2ControlRuntime.runStateSeq = 1u;
    core2ControlRuntime.rs485ControlEstop = 1u;
    core2ControlRuntime.ownerValidPrev = 0u;
    core2ControlRuntime.loraLinkPrev = 0u;
    core2ControlRuntime.loraRearmRequired = 1u;
    core2ControlRuntime.loraLaunchLowSeen = 0u;
    core2ControlRuntime.effectiveLoraLaunchGate = 0u;

    /* 控制环参数 RAM 镜像：所有控制 init 前先填宏默认值，再从 flash 载入覆盖（校验通过 +
    * GLOBAL_PARAM_PERSIST_ENABLE），使 motorControlInit/chassisApplicationInit 读到持久化值。
    * 菜单(D-4)运行时改镜像 + 热生效 + 保存。 */
    chassisParamsInit();
    core2ParamSyncOwnerServiceInit();
    chassisParamsGetActive(&activeParams);

    chassisInitResult = chassisApplicationInit();
    imuInitResult = IMUInit();
    if (imuInitResult != 0u) core2ControlRuntime.initFailedMask |= 0x01u;
    else core2ControlRuntime.imuReady = 1u;
    if (chassisInitResult != 0u) core2ControlRuntime.initFailedMask |= 0x02u;
    core2ControlRuntime.criticalInitFailed =
        (core2ControlRuntime.initFailedMask != 0u) ? 1u : 0u;
    motorInit();
    motorControlInit();
    core2RuntimeApplyControllerParams(&activeParams);
    currentSenseInit();
    currentSenseClearOffset(16u);
    encoderInit();
    menuKeyInit(1u); /* menuKeyScan 由 PIT_CH0+KEY_TICK_DIVIDER 以 1ms 周期调用 */
    lightStripInit();
    #if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
        lightStripSetBrightness((uint8_t) activeParams.lightStripBrightness); /* 生产态：恢复持久化/默认亮度（WAIT/HOME 可热调） */
    #else
        lightStripSetBrightness(0u); /* TUNE 调参态 ^ 定位灯条不亮（PWM 占 P08_0 但 0 占空比）；按键改作选轮 */
    #endif
    /* RS-485 链路初始化。PRODUCTION：comm 已搬到 cm_7_1 通信核，本调用是镜像实现的零初始化(不碰 UART4)；
    * TUNE：真实 thvd1450Init(UART_4 + DE=P13_7, comm 仍在本核)。 */
    chassisLinkInit();
    buzzerInit(); /* 蜂鸣器单一所有者：BEEP_PIN gpio_init + 复位仲裁。PRODUCTION/TUNE 都 init(TUNE 失控也鸣) */
    buzzerSetHardwareInitFault(core2ControlRuntime.criticalInitFailed);

    #if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
        /* 空地系统：LoRa 路径需常驻可用——模式 3(源=LoRa)与模式 4(失链兜底)都在运行时切入 LoRa。
        * PRODUCTION：LoRa 收发已搬到 cm_7_1, loraNavControllInit 内的 loraRemoteInit 为镜像零初始化(不碰 UART1)；
        * loraNavControl 仅保留 NAV 录制/回放/落地状态机(电机驱动在本核 500µs ISR)，运动/解锁/边沿读 cm_7_1 共享内存镜像。 */
        loraNavControlInit();
        core2RuntimeApplyDynamicParams(&activeParams);
    #elif (MOTOR_TUNE_MODE == MOTOR_TUNE_LORA_TEST)
        lora3a22_init();
    #endif

    #if (CORE2_BUILD_MODE == CORE2_BUILD_TUNE) && ((MOTOR_TUNE_MODE == MOTOR_TUNE_CASCADE) || (MOTOR_TUNE_MODE ==
    MOTOR_TUNE_CURRENT) || (MOTOR_TUNE_MODE == MOTOR_TUNE_OPEN_LOOP))
        updateTuneTarget();
    #endif

    #if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
        chassisMenuInit(); /* 生产态：板载菜单（发车/调参页）接管 IPS114 + 5 键，替代 tuneDisplay 与灯条键扫描 */
        chassisMenuSetInitHealth(core2ControlRuntime.initFailedMask, 0u);
    #else
        tuneDisplayInit();
    #endif
    pit_us_init(PIT_CH0, CONTROL_PERIOD_US);
    pit_us_init(PIT_CH1, SAMPLE_PERIOD_US);
    core2ControlRuntime.initReady = 1u;
}

uint8_t core2RuntimeSubmitParams(const chassisParamStruct *candidate)
{
    chassisParamStruct previous;
    chassisParamStruct prepared;

```

```

chassisParamStruct active;
chassisApplicationParamViewStruct chassisView;
loraRemoteParamViewStruct remoteView;
loraNavControlParamViewStruct navView;
uint32_t irqWasEnabled;
uint8_t controllerReinitRequired;

if (candidate == 0) return 0u;

prepared = *candidate;
chassisParamsSanitize(&prepared);
chassisParamsGetActive(&previous);

/* 活动仓库只在前台访问，完整 DTO 的 sanitize/复制在 IRQ 开启时完成。 */
if (!chassisParamsCommit(&prepared)) return 0u;
chassisParamsGetActive(&active);
controllerReinitRequired = (memcmp(
    &previous,
    &active,
    offsetof(chassisParamStruct, yawRateOutputLimit)) != 0)
    ? 1u
    : 0u;
core2RuntimeBuildParamViews(&active, 0, &chassisView, &remoteView, &navView);

/* 仅各 ISR owner 的 9 个动态标量与 pending 标志在共享 IRQ 临界区交换；完整 DTO 工作已在上方完成。
 * CH0/CH1 共用 CPUIntIdx2，只保存并屏蔽该 IRQ，禁止逐飞 interrupt_global * 坏计数器。 */
irqWasEnabled = NVIC_GetEnableIRQ(CPUIntIdx2_IRQn);
NVIC_DisableIRQ(CPUIntIdx2_IRQn);
loraRemoteApplyParams(&remoteView);
loraNavControlApplyParams(&navView);
chassisApplicationApplyYawOutputLimit(chassisView.yawRateOutputLimit);
if (controllerReinitRequired) core2ControlRuntime.paramApplyPending = 1u;
if (irqWasEnabled != 0u) NVIC_EnableIRQ(CPUIntIdx2_IRQn);

lightStripSetBrightness((uint8_t) active.lightStripBrightness);
return 1u;
}

uint8_t core2RuntimeRequestSave(void)
{
    chassisParamStruct active;
    if (core2RuntimeParamsApplyReady() == 0u) return 0u;
    chassisParamsGetActive(&active);
    return (uint8_t) (chassisParamsSaveRuntime(&active) ==
        CHASSIS_PARAMS_SLOT_OK);
}

uint8_t core2RuntimeRequestFactoryReset(void)
{
    chassisParamStruct defaults;
    if (core2RuntimeParamsApplyReady() == 0u) return 0u;
    chassisParamsLoadDefaults(&defaults);
    chassisParamsSanitize(&defaults);
    if (chassisParamsSaveRuntime(&defaults) !=
        CHASSIS_PARAMS_SLOT_OK) return 0u;
    return core2RuntimeSubmitParams(&defaults);
}

uint8_t core2RuntimeParamsApplyReady(void)
{
    #if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
        motorControlSnapshotStruct motor;
        motorControlGetSnapshot(&motor);
        if (core2ParamSyncMotorSnapshotIsStopped(&motor) == 0u) return 0u;
        if (core2ControlRuntime.runState == CAR_RUN_WAIT_LINK)
        {
            return 1u;
        }
        if (core2ControlRuntime.runState == CAR_RUN_FOLLOW_485 &&
            (!core2ControlRuntime.rs485ControlEnabled || core2ControlRuntime.rs485ControlEstop ||
                !core2ControlRuntime.rs485ControlLaunched))
        {
            return 1u;
        }
    #endif
    return 0u;
}

#else
    return 1u;
#endif
}

```

```

uint8_t core2RuntimeParamsApplied(void)
{
    return (uint8_t) (core2ControlRuntime.paramApplyPending == 0u);
}

uint8_t core2RuntimeRequestGlobalParamAction(uint8_t action, uint8_t slot)
{
    paramSyncGlobalRequest request;
    if (core2ParamSyncOwnerServiceMutationLocked() != 0u ||
        core2ControlRuntime.paramGlobalRequestValid != 0u ||
        core2ControlRuntime.paramGlobalStateValid == 0u ||
        paramSyncGlobalStateReady(&core2ControlRuntime.paramGlobalState) == 0u)
        return 0u;
    core2ControlRuntime.paramGlobalRequestSequence++;
    if (core2ControlRuntime.paramGlobalRequestSequence == 0u)
        core2ControlRuntime.paramGlobalRequestSequence = 1u;
    request.producerBootGeneration = core2ParamSyncOwnerBootGeneration();
    request.sequence = core2ControlRuntime.paramGlobalRequestSequence;
    request.origin = 2u;
    request.action = action;
    request.slot = slot;
    request.reserved = 0u;
    if (paramSyncGlobalRequestIsValid(&request) == 0u) return 0u;
    core2ControlRuntime.paramGlobalRequest = request;
    core2ControlRuntime.paramGlobalRequestCoordinatorBoot =
        core2ControlRuntime.paramGlobalState.coordinatorBootGeneration;
    core2ControlRuntime.paramGlobalRequestValid = 1u;
    core2ControlRuntime.paramGlobalUiState = CORE2_PARAM_GLOBAL_QUEUED;
    core2ControlRuntime.paramGlobalUiPhase = PARAM_SYNC_GLOBAL_PHASE_REQUESTED;
    core2ControlRuntime.paramGlobalUiTarget = 0xFFu;
    core2ControlRuntime.paramGlobalUiError = PARAM_SYNC_GLOBAL_ERROR_NONE;
    core2ControlRuntime.paramGlobalUiAction = action;
    core2ControlRuntime.paramGlobalUiSlot = slot;
    if (action == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
        core2ControlRuntime.paramGlobalSlotUiState = PARAM_SYNC_SLOT_UI_CHECKING;
    return 1u;
}

void core2RuntimeGetParamGlobalUi(core2ParamGlobalUiSnapshot *out)
{
    if (out == 0) return;
    out->state = core2ControlRuntime.paramGlobalUiState;
    out->phase = core2ControlRuntime.paramGlobalUiPhase;
    out->target = core2ControlRuntime.paramGlobalUiTarget;
    out->error = core2ControlRuntime.paramGlobalUiError;
    out->action = core2ControlRuntime.paramGlobalUiAction;
    out->slot = core2ControlRuntime.paramGlobalUiSlot;
    out->slotUiState = core2ControlRuntime.paramGlobalSlotUiState;
    out->reserved = 0u;
    out->eventSequence = core2ControlRuntime.paramGlobalUiEventSequence;
}

static void core2RuntimeServiceParamApply(void)
{
    if (!core2ControlRuntime.paramApplyPending || !core2RuntimeParamsApplyReady())
    {
        return;
    }

    /* CH0/CH1 共用 CPUIntIdx2: 极短保存并屏蔽该 IRQ 后只停/清 CH0 源, 再恢复原使能状态。
     * 不清 NVIC pending, 不停 CH1; 随后 CH0 已停表, CH1 可继续电流采样而不会触碰控制器状态。 */
    {
        uint32_t irqWasEnabled = NVIC_GetEnableIRQ(CPUIntIdx2_IRQn);
        NVIC_DisableIRQ(CPUIntIdx2_IRQn);
        pit_disable(PIT_CH0);
        pit_isr_flag_clear(PIT_CH0);
        if (irqWasEnabled != 0u) NVIC_EnableIRQ(CPUIntIdx2_IRQn);
    }
    if (core2RuntimeParamsApplyReady())
    {
        chassisParamStruct active;
        chassisParamsGetActive(&active);
        core2RuntimeApplyControllerParams(&active);
        core2ControlRuntime.paramApplyPending = 0u;
    }
    pit_enable(PIT_CH0);
}

```

```

static void core2RuntimeBuildParamViews(
    const chassisParamStruct *params,
    motorControlParamViewStruct *motor,
    chassisApplicationParamViewStruct *chassis,
    loraRemoteParamViewStruct *remote,
    loraNavControlParamViewStruct *nav)
{
    const chassisPidGainStruct *speed[4] = {
        &params->speedFl, &params->speedFr, &params->speedBl, &params->speedBr};
    const chassisPidGainStruct *current[4] = {
        &params->currentFl, &params->currentFr, &params->currentBl, &params->currentBr};
    uint8_t i;

    if (motor != 0)
    {
        for (i = 0u; i < 4u; i++)
        {
            motor->speedKp[i] = speed[i]->kp;
            motor->speedKi[i] = speed[i]->ki;
            motor->speedKd[i] = speed[i]->kd;
            motor->currentKp[i] = current[i]->kp;
            motor->currentKi[i] = current[i]->ki;
            motor->currentKd[i] = current[i]->kd;
        }
        motor->speedIntegralMax = params->speedIntegralMax;
        motor->currentIntegralMax = params->currentIntegralMax;
    }

    if (chassis != 0)
    {
        chassis->yawRateKp = params->yawRate.kp;
        chassis->yawRateKi = params->yawRate.ki;
        chassis->yawRateKd = params->yawRate.kd;
        chassis->yawRateIntegralMax = params->yawRateIntegralMax;
        chassis->headingKp = params->headingKp;
        chassis->headingRateLimit = params->headingRateLimit;
        chassis->yawRateOutputLimit = params->yawRateOutputLimit;
    }

    if (remote != 0)
    {
        remote->vMaxMps = params->loraVMaxMps;
        remote->wzMaxRadps = params->loraWzMaxRadps;
        remote->stickDeadzone = params->loraStickDeadzone;
    }

    if (nav != 0)
    {
        nav->correctionKp = params->navCorrectionKp;
        nav->correctionLimit = params->navCorrectionLimit;
        nav->errorLimitPulses = params->navErrorLimitPulses;
        nav->loraVMaxMps = params->loraVMaxMps;
        nav->loraWzMaxRadps = params->loraWzMaxRadps;
    }
}

static void core2RuntimeApplyControllerParams(const chassisParamStruct *params)
{
    motorControlParamViewStruct motor;
    chassisApplicationParamViewStruct chassis;
    core2RuntimeBuildParamViews(params, &motor, &chassis, 0, 0);
    motorControlApplyParams(&motor);
    (void) chassisApplicationApplyParams(&chassis);
}

#if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
static void core2RuntimeApplyDynamicParams(const chassisParamStruct *params)
{
    chassisApplicationParamViewStruct chassis;
    loraRemoteParamViewStruct remote;
    loraNavControlParamViewStruct nav;

    core2RuntimeBuildParamViews(params, 0, &chassis, &remote, &nav);
    loraRemoteApplyParams(&remote);
    loraNavControlApplyParams(&nav);
    chassisApplicationApplyYawOutputLimit(chassis.yawRateOutputLimit);
}
#endif

```

```

void pitProcess1(void)
{
    if (!core2ControlRuntime.initReady) return;
    currentSenseSample();
}

void pitProcess0(void)
{
    if (!core2ControlRuntime.initReady) return;

    motorControlUpdateCurrentFeedback();

    core2ControlRuntime.imuTickDivider++;
    if (core2ControlRuntime.imuTickDivider >= IMU_TICK_DIVIDER)
    {
        core2ControlRuntime.imuTickDivider = 0u;
        if (core2ControlRuntime.imuReady != 0u)
            IMUUpdate(); /* 1ms (500µs×IMU_TICK_DIVIDER) ; 仅采样、映射并压入前台姿态队列 */
        chassisLinkTickMs(); /* 1ms RS-485 链路年龄计时 (与 IMU 同节拍) */
    }
    #if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
        /* 空地系统：RS-485 连续失链时长累加(活则清零、饱和于等待阈值)，供模式 4 超时兜底决策。
        * 紧随 chassisLinkTickMs 后读 isAlive，两者同拍一致(age 本拍已更新)。ISR 内单字节/字操作，安全。 */
        if (chassisLinksAlive())
        {
            core2ControlRuntime.rs485DeadMs = 0u;
            if ((core2ControlRuntime.rejoinActive != 0u) &&
                (core2ControlRuntime.rejoinReadyMs < 2000u))
                core2ControlRuntime.rejoinReadyMs++;
        }
        else
        {
            if (core2ControlRuntime.rs485DeadMs < AG_CAR_RS485_WAIT_MS)
                core2ControlRuntime.rs485DeadMs++;
            core2ControlRuntime.rejoinReadyMs = 0u;
            core2ControlRuntime.rejoinActive = 0u;
        }
    }
    #endif

    core2ControlRuntime.keyTickDivider++;
    if (core2ControlRuntime.keyTickDivider >= KEY_TICK_DIVIDER)
    {
        core2ControlRuntime.keyTickDivider = 0u;
    }
    #if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
        chassisMenuPitTick(); /* 扫键 + 菜单填充反馈倒计时 (内部含 menuKeyScan, 1ms 节拍) */
    #else
        menuKeyScan();
    #endif

    /* 即使控制硬件初始化失败，UI 刷新和前台通信年龄仍必须继续推进。 */
    core2ControlRuntime.loopMsAccumUs += (uint16_t) CONTROL_PERIOD_US;
    if (core2ControlRuntime.loopMsAccumUs >= 1000u)
    {
        core2ControlRuntime.loopMsAccumUs -= 1000u;
        core2ControlRuntime.loopMs++;
    }
    core2ControlRuntime.displayCountUs += CONTROL_PERIOD_US;
    if (core2ControlRuntime.displayCountUs >= IPS114_DISPLAY_PERIOD_MS * 1000u)
    {
        core2ControlRuntime.displayCountUs = 0u;
        core2ControlRuntime.displayDue = 1u;
    }

    if (core2ControlRuntime.criticalInitFailed != 0u)
    {
        #if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
            rs485ControlStop();
            buzzerSetPattern(CHASSIS_LINK_BUZZER_SILENT);
            buzzerSetRunaway(0u);
            buzzerTick();
        #else
            motorControlStopAll();
        #endif
        return;
    }

    #if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
        /* 空地系统：单点决策 carRunState (据 RS-485 链路活否 + cmd.sysByte 源位 + 失链时长) ，

```

```

*替代原静态 controlSource 分支。每拍 500μs 重算，无锁存——RS-485 恢复自动退出模式 4。
*取最新命令快照(整块、含 sysByte)：既供源位决策、又供 FOLLOW_485 发车门控消费。*/
{
    chassisLinkCommandStruct cmd;
    uint8_t linkAlive = chassisLinkIsAlive();
    carRunStateEnum previousRunState = core2ControlRuntime.runState;
    uint8_t ownerEdge = 0u;

    /* 仅链路活时才在 500μs PIT 内整块读命令快照（chassisLinkGetCommand 含
    * PRIMASK 关中断临界区）：WAIT_LINK/MODE4 失链态的决策只看 rs485DeadMs、
    * 不需要 cmd。未连通时每拍调用会与 RS-485/PIT 时序耦合 → 整机卡在 waiting（板上实测，
    * 屏蔽该调用或仅 isAlive 时读即不卡死）。失链态 cmd 置零：MODE4 launchGate=0、蜂鸣 isAlive
    * 为假不读 cmd 蜂鸣位、SOURCE_IS_LORA 仅在 alive 分支引用，均不受影响。*/
    cmd.vx = 0.0f;
    cmd.vy = 0.0f;
    cmd.wz = 0.0f;
    cmd.cmd = 0u;
    cmd.sysByte = 0u;

    if (linkAlive)
    {
        chassisLinkGetCommand(&cmd);
        if ((previousRunState == CAR_RUN_MODE4) &&
            (core2ControlRuntime.rejoinActive == 0u))
        {
            core2ControlRuntime.rejoinActive = 1u;
            core2ControlRuntime.rejoinReadyMs = 0u;
            core2ControlRuntime.rejoinFenceSeq = core2ControlRuntime.lastCommSeq;
        }
        if (core2ControlRuntime.rejoinActive != 0u)
        {
            if ((core2ControlRuntime.rejoinReadyMs >= 2000u) &&
                (core2ControlRuntime.commFresh != 0u) &&
                (core2ControlRuntime.lastCommSeq != core2ControlRuntime.rejoinFenceSeq))
            {
                core2ControlRuntime.rejoinActive = 0u;
                core2ControlRuntime.rejoinReadyMs = 0u;
                core2ControlRuntime.rejoinBarrierTicks = 1u;
                core2ControlRuntime.runState = CHASSIS_LINK_CMD_SOURCE_IS_LORA(cmd) ? CAR_RUN_LORA_LINKED : CAR_RUN_FOLLOW_485;
            }
            else
            {
                core2ControlRuntime.runState = CAR_RUN_WAIT_LINK;
            }
        }
        else
        {
            core2ControlRuntime.runState = CHASSIS_LINK_CMD_SOURCE_IS_LORA(cmd) ? CAR_RUN_LORA_LINKED : CAR_RUN_FOLLOW_485;
        }
    }
    else
    {
        core2ControlRuntime.runState =
            (core2ControlRuntime.rs485DeadMs >= AG_CAR_RS485_WAIT_MS) ? CAR_RUN_MODE4 : CAR_RUN_WAIT_LINK;
    }
    if (core2ControlRuntime.runState != previousRunState)
    {
        core2ControlRuntime.runStateSeq++;
        if (core2ControlRuntime.runStateSeq == 0u) core2ControlRuntime.runStateSeq = 1u;
        ownerEdge = 1u;
    }
    {
        uint8_t ownerValid = 0u;
        uint8_t loraAlive = loraRemoteIsLinkAlive();
        uint8_t launchGate = (cmd.cmd & CHASSIS_LINK_CMD_LAUNCH) ? 1u : 0u;
        if ((core2ControlRuntime.loraLinkPrev != 0u) && (loraAlive == 0u))
        {
            core2ControlRuntime.loraRearmRequired = 1u;
            core2ControlRuntime.loraLaunchLowSeen = 0u;
        }
        core2ControlRuntime.loraLinkPrev = loraAlive;
        if ((loraAlive != 0u) && (launchGate == 0u))
            core2ControlRuntime.loraLaunchLowSeen = 1u;
        if ((core2ControlRuntime.runState == CAR_RUN_LORA_LINKED) &&
            (loraAlive != 0u) && (launchGate != 0u) &&
            (core2ControlRuntime.loraLaunchLowSeen != 0u))
        {
            core2ControlRuntime.loraRearmRequired = 0u;
        }
    }
}

```

```

core2ControlRuntime.effectiveLoraLaunchGate = (uint8_t) ((loraAlive != 0u) && (launchGate != 0u) &&
    (core2ControlRuntime.loraRearmRequired == 0u));
if (core2ControlRuntime.runState == CAR_RUN_FOLLOW_485)
{
    ownerValid = (uint8_t) (linkAlive &&
        ((cmd.cmd & CHASSIS_LINK_CMD_ENABLE) != 0u) &&
        ((cmd.cmd & CHASSIS_LINK_CMD_ESTOP) == 0u) &&
        ((cmd.cmd & CHASSIS_LINK_CMD_LAUNCH) != 0u));
}
else if (core2ControlRuntime.runState == CAR_RUN_LORA_LINKED)
{
    ownerValid = (uint8_t) (linkAlive &&
        core2ControlRuntime.effectiveLoraLaunchGate);
}
else if (core2ControlRuntime.runState == CAR_RUN_MODE4)
{
    ownerValid = (uint8_t) (loraRemotesLinkAlive() &&
        loraRemotesUnlocked());
}
if (ownerValid != core2ControlRuntime.ownerValidPrev)
{
    core2ControlRuntime.ownerValidPrev = ownerValid;
    ownerEdge = 1u;
}
}
if (ownerEdge != 0u)
{
    core2ControlRuntime.rejoinBarrierTicks = 1u;
    loraNavControlAbortSession();
}

if (core2ControlRuntime.rejoinBarrierTicks != 0u)
{
    /* 热回归只清动态控制历史；恰好一个 500us 拍强制目标与电机输出全零。 */
    core2ControlRuntime.rejoinBarrierTicks--;
    rs485ControlStop();
}
else if (core2ControlRuntime.runState == CAR_RUN_FOLLOW_485)
{
    /* RS-485 跟随 core1（源=485，链路活已由上面决策保证）：消费下发运动指令 + 发车门控。
    * 发车门控（每拍 500us 都查，不分频）：禁止发车的任一条件成立 → 软急停/等待。
    * 条件：收到急停位 / 未收到使能位 / **未收到发车信号(LAUNCH)**（链路失活已在 carRunState
    * 决策中分流到 WAIT_LINK/MODE4，不进本分支）。发车信号来自 core0(经 core1 中转 RS-485
    * cmd LAUNCH 位)——仿 core1 等待上位机发车，上位机未发车则即使有运动指令也禁止发车。
    * 与 LoRa/NAV 急停同语义——等待/急停态跳过本拍 CurrentTick/Output（不再产生 PWM）：
    * 电流环是增量式 PID，StopAll 已清状态，但若同拍再跑 CurrentTick，
    * error=0-feedbackCurrent(非零电流反馈) 会算出非零 motorOutput，形成残余 PWM 脉冲。
    * 故等待/急停态硬停且不输出。 */
    if ((cmd.cmd & CHASSIS_LINK_CMD_ESTOP) || !(cmd.cmd & CHASSIS_LINK_CMD_ENABLE) || !(cmd.cmd & CHASSIS_LINK_CMD_LAUNCH))
    {
        rs485ControlStop(); /* 含遥测态复位（使能关/急停/未发车） */
    }
    else
    {
        /* 使能 + 链路活 + 已发车：按 core1 指令跑级联控制。速度环 2ms 分频，电流环/输出每拍。 */
        core2ControlRuntime.rs485ControlEnabled = 1u;
        core2ControlRuntime.rs485ControlEstop = 0u;
        core2ControlRuntime.rs485ControlLaunched = 1u;
        core2ControlRuntime.speedLoopDivider++;
        if (core2ControlRuntime.speedLoopDivider >= SPEED_LOOP_DIVIDER)
        {
            core2ControlRuntime.speedLoopDivider = 0u;
            rs485ControlRunSpeedLoop(&cmd);
        }
        motorControlCurrentTick();
        motorControlOutput();
    }
}
}
else if ((core2ControlRuntime.runState == CAR_RUN_LORA_LINKED) ||
    (core2ControlRuntime.runState == CAR_RUN_MODE4))
{
    /* 源=LoRa(模式3)或失链兜底(模式4)：本地 LoRa 双摇杆遥控 + NAV 录制/回放。
    * chord 语义按 standalone 分流(B6)：模式4脱机 S3+S4 切换本地解锁；模式2/3联机 S3+S4 发请求，上层按系统模式解释。
    * 驱动门控：模式4=本地解锁；模式3=飞机到顶门控 launchGate(core1 下发 LAUNCH 位)，车禁动直到飞机到顶。 */
    {
        uint8_t standalone = (core2ControlRuntime.runState == CAR_RUN_MODE4) ? 1u : 0u;
        uint8_t launchGate = standalone ? 0u : core2ControlRuntime.effectiveLoraLaunchGate;
        core2ControlRuntime.speedLoopDivider = 0u;
        core2ControlRuntime.rs485YawDivider = 0u;
    }
}

```

```

        loraNavControlPit500us(standalone, launchGate);
    }
}
else /* CAR_RUN_WAIT_LINK: RS-485 失链但未超时——软急停等待 core1(与原失链急停同语义) */
{
    rs485ControlStop(); /* 含遥测态复位（使能关/急停/未发车） */
}

/* 蜂鸣（单点，恰每拍一次，在上方分支 motorControlOutput 之后）：码源 = RS-485 命令的蜂鸣位
 * （链路活才信，否则 SILENT，避免播放陈旧命令码）。BEEP_PIN 由 buzzer 单一所有者库唯一写：
 * 此处只提交 pattern 码 + runaway 态，事件确认音由 loraNavControl 经 buzzerBeepEvent 提交，
 * 仲裁优先级 runaway > WARN > hardware fault > event > pattern 全在 buzzerTick 内。 */
{
    /* 蜂鸣码源：链路活 → core0 经 RS-485 中转的码(起飞/巡航/断联 WARN/armed-idle);
     * 链路失 → 本核本地：模式 4(脱离飞机系统 >2s，已切本地 LoRa 兜底) = LORA_IDLE 0.5Hz 可辨别提示
     * （用户要求：每 2s 响 150ms，人耳可辨别的一声声，非 WARN 急促告警；LoRa 锁/解锁确认音仍可穿插）；
     * WAIT_LINK(等待 RS-485 未超时，尚未进 LoRa) = WAITING 等待连接提示音。 */
    uint8_t buzCode;
    if (chassisLinkIsAlive())
    {
        buzCode = CHASSIS_LINK_CMD_BUZZER(cmd);
    }
    else if (core2ControlRuntime.runState == CAR_RUN_MODE4)
    {
        buzCode = (uint8_t) CHASSIS_LINK_BUZZER_LORA_IDLE; /* 本地 LoRa 兜底：0.5Hz 可辨别提示(非急促告警) */
    }
    else
    {
        buzCode = (uint8_t) CHASSIS_LINK_BUZZER_WAITING; /* WAIT_LINK：等待连接提示音 */
    }
    /* 完整资格三声只接受 Core0 系统资格 owner 下发的码 7；本核不再用局部 C0/C1 链路猜测。 */
    /* 单一所有者：仅 buzzerTick 写 BEEP。失控态注入仲裁器(最高优先常鸣)；LoRa 事件确认音由
     * loraNavControl 经 buzzerBeepEvent 提交(仲裁器 event>pattern 覆盖)，此处不再分 exclusive。 */
    buzzerSetPattern(buzCode);
    buzzerSetRunaway(motorControlIsRunawayLatched());
    buzzerTick();
}
}

#elif (MOTOR_TUNE_MODE == MOTOR_TUNE_CASCADE)
core2ControlRuntime.speedLoopDivider++;
if (core2ControlRuntime.speedLoopDivider >= SPEED_LOOP_DIVIDER)
{
    core2ControlRuntime.speedLoopDivider = 0u;
    chassisApplicationUpdateFeedback();
    updateTuneTarget();
    motorControlSpeedTick();
}
motorControlCurrentTick();
motorControlStopWheelExcept(tuneModeGetSelectedWheel()); /* 仅驱动选中轮，非选中轮空闲 */
motorControlOutput();

#elif (MOTOR_TUNE_MODE == MOTOR_TUNE_LORA_TEST)
core2ControlRuntime.speedLoopDivider++;
if (core2ControlRuntime.speedLoopDivider >= SPEED_LOOP_DIVIDER)
{
    core2ControlRuntime.speedLoopDivider = 0u;
    chassisApplicationUpdateFeedback();
}
}

#elif (MOTOR_TUNE_MODE == MOTOR_TUNE_CURRENT)
core2ControlRuntime.speedLoopDivider++;
if (core2ControlRuntime.speedLoopDivider >= SPEED_LOOP_DIVIDER)
{
    core2ControlRuntime.speedLoopDivider = 0u;
    chassisApplicationUpdateFeedback();
    updateTuneTarget();
}
}
motorControlCurrentTick();
motorControlStopWheelExcept(tuneModeGetSelectedWheel()); /* 仅驱动选中轮，非选中轮空闲 */
motorControlOutput();

#elif (MOTOR_TUNE_MODE == MOTOR_TUNE_OPEN_LOOP)
core2ControlRuntime.speedLoopDivider++;
if (core2ControlRuntime.speedLoopDivider >= SPEED_LOOP_DIVIDER)
{
    core2ControlRuntime.speedLoopDivider = 0u;
    chassisApplicationUpdateFeedback();
    updateTuneTarget();
}
}
motorControlOutput();
#endif

```



```

#if (CORE2_BUILD_MODE == CORE2_BUILD_TUNE)
/* TUNE: pattern 恒 SILENT(未 setPattern), 仅失控时由 buzzer 仲裁器常鸣。每拍 setRunaway+tick 恰一次。
* 本块仅 TUNE 编译, 与 PRODUCTION 在其 carRunState 分支内 tick 互斥; 修复 TUNE 原 BEEP 未 init/失控无鸣缺口。*/
buzzerSetRunaway(motorControlIsRunawayLatched());
buzzerTick();
#endif
}

void whileProcess(void)
{
    if (!core2ControlRuntime.initReady) return;

    core2ParamSyncOwnerServicePoll();

    if (core2ControlRuntime.imuReady != 0u) IMUProcess();

#if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
/* 先消费交互: 参数提交可在同一轮立即进入下方 Apply, 也不被共享镜像/串口处理拖延。*/
chassisMenuInputTick();
#endif
core2RuntimeServiceParamApply();

/* 标志读-请在关中断临界区内完成, 消除 ISR 置位与前台“读-判-清”间的竞态(防丢边沿)。
* 临界区仅含单字节读+清零, 极短; 耗时的发送/刷屏在临界区外执行。
* 用 PRIMASK 存-恢复, 绝不用 SDK interrupt_global_disable/enable (坏计数器, 与 ISR 高频临界区
* 竞争会上漂致中断永久关闭整机卡死; 详见 chassisLink.c 镜像段注释)。*/
{
    uint32_t primask = __get_PRIMASK();
    uint8_t doDisplay;
    __disable_irq();
    doDisplay = core2ControlRuntime.displayDue;
    core2ControlRuntime.displayDue = 0u;
    __set_PRIMASK(primask);
}

#if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
/* 核间共享内存桥接 (RS-485/LoRa 收发已搬到 cm_7_1 通信核) :
* (1) 读 cm_7_1 发布的 commToControl (RS-485 命令 + LoRa 解码), 刷新本地镜像——
*   chassisLink/lorRemote 的 getter 经此镜像供 500µs ISR 读 (SCB cache 操作只在此前台执行)。
* (2) 发布 controlToComm 应答快照: 应答类型按 carRunState (LORA_LINKED→LoRa 运动 / 其它→遥测实速) +
*   遥测实速 + 状态位。cm_7_1 收到 core1 MSG_COMMAND 请求后照发 (请求-应答, 半双工串行化)。
* magic 门控防 __no_init 上电垃圾: cm_7_1 未就绪前镜像保持上电默认(IsAlive=0→WAIT_LINK, 安全)。*/
{
    commToControlStruct commSnap;
    controlToCommStruct ctrlSnap;
    chassisMotionCommandStruct loraCmd;
    chassisApplicationSnapshotStruct chassisSnapshot;
    uint8_t status = 0u;

    chassisApplicationGetSnapshot(&chassisSnapshot);
    loraCmd.vx = 0.0f;
    loraCmd.vy = 0.0f;
    loraCmd.wz = 0.0f;

    if (coreDataReadCommToControl(&commSnap) != 0u &&
        ((core2ControlRuntime.commSeen == 0u) ||
         (commSnap.seq != core2ControlRuntime.lastCommSeq)))
    {
        core2ControlRuntime.lastCommSeq = commSnap.seq;
        core2ControlRuntime.lastCommTickMs = core2ControlRuntime.loopMs;
        core2ControlRuntime.commSeen = 1u;
        core2ControlRuntime.commFresh = 1u;
        chassisLinkMirrorApply(&commSnap);
        loraRemoteMirrorApply(&commSnap);
        core2ControlRuntime.paramGlobalStateValid = (uint8_t) (
            commSnap.paramGlobalStateValid != 0u &&
            paramSyncGlobalStatelsValid(&commSnap.paramGlobalState) != 0u);
        if (core2ControlRuntime.paramGlobalRequestValid != 0u)
        {
            uint8_t cancelReason =
                core2ParamGlobalRequestCancellationReason(
                    &commSnap.paramGlobalState,
                    core2ControlRuntime.paramGlobalStateValid,
                    &core2ControlRuntime.paramGlobalRequest,
                    core2ControlRuntime.paramGlobalRequestCoordinatorBoot);
            if (cancelReason != PARAM_SYNC_GLOBAL_ERROR_NONE)
                cancelPendingParamGlobalRequest(cancelReason);
        }
        if (core2ControlRuntime.paramGlobalStateValid != 0u)
        {

```

```

core2ControlRuntime.paramGlobalState =
    commSnap.paramGlobalState;
if (core2ControlRuntime.paramGlobalRequestValid != 0u &&
    paramSyncGlobalStateMatchesRequest(
        &commSnap.paramGlobalState,
        &core2ControlRuntime.paramGlobalRequest) != 0u)
{
    core2ParamGlobalUiSnapshot ui;
    ui.state = core2ControlRuntime.paramGlobalUiState;
    ui.phase = core2ControlRuntime.paramGlobalUiPhase;
    ui.target = core2ControlRuntime.paramGlobalUiTarget;
    ui.error = core2ControlRuntime.paramGlobalUiError;
    ui.action = core2ControlRuntime.paramGlobalUiAction;
    ui.slot = core2ControlRuntime.paramGlobalUiSlot;
    ui.slotUiState = core2ControlRuntime.paramGlobalSlotUiState;
    ui.reserved = 0u;
    ui.eventSequence = core2ControlRuntime.paramGlobalUiEventSequence;
    if (core2ParamGlobalUiObserve(&commSnap.paramGlobalState,
        &core2ControlRuntime.paramGlobalRequest,
        &core2ControlRuntime.paramGlobalRequestValid,
        &ui) != 0u)
    {
        core2ControlRuntime.paramGlobalUiState = ui.state;
        core2ControlRuntime.paramGlobalUiPhase = ui.phase;
        core2ControlRuntime.paramGlobalUiTarget = ui.target;
        core2ControlRuntime.paramGlobalUiError = ui.error;
        core2ControlRuntime.paramGlobalUiAction = ui.action;
        core2ControlRuntime.paramGlobalUiSlot = ui.slot;
        core2ControlRuntime.paramGlobalSlotUiState = ui.slotUiState;
        if (core2ControlRuntime.paramGlobalRequestValid == 0u)
        {
            core2ControlRuntime.paramGlobalRequestCoordinatorBoot = 0u;
            core2ControlRuntime.paramGlobalUiEventSequence++;
            if (core2ControlRuntime.paramGlobalUiEventSequence == 0u)
                core2ControlRuntime.paramGlobalUiEventSequence = 1u;
        }
    }
}
}
}
if ((core2ControlRuntime.commFresh != 0u) &&
    ((uint32_t) (core2ControlRuntime.loopMs - core2ControlRuntime.lastCommTickMs) >=
        LORA_LINK_TIMEOUT_MS))
{
    chassisLinkMirrorInvalidate();
    loraRemoteMirrorInvalidate();
    core2ControlRuntime.commFresh = 0u;
    core2ControlRuntime.paramGlobalStateValid = 0u;
    cancelPendingParamGlobalRequest(PARAM_SYNC_GLOBAL_ERROR_ROUTE);
}

/* 授权位只反映基础运行门；姿态故障通过独立位上报，不撤销底盘授权。 */
if (core2ControlRuntime.runState == CAR_RUN_FOLLOW_485)
{
    if (core2ControlRuntime.rs485ControlEnabled &&
        core2ControlRuntime.rs485ControlLaunched &&
        !core2ControlRuntime.rs485ControlEstop)
    {
        status |= CHASSIS_LINK_STATUS_ENABLED;
        status |= CHASSIS_LINK_STATUS_LAUNCHED;
    }
    else if (core2ControlRuntime.rs485ControlEstop ||
        core2ControlRuntime.rs485ControlEnabled ||
        core2ControlRuntime.rs485ControlLaunched)
    {
        status |= CHASSIS_LINK_STATUS_ESTOP;
    }
    if (loraRemoteGetRemoteTakeoffReq()) status |= CHASSIS_LINK_STATUS_MODE2_LORA_LAND_REQ;
}
else if (chassisSnapshot.yawRateEnabled)
{
    status |= CHASSIS_LINK_STATUS_ENABLED;
}
if (chassisSnapshot.attitudeFaultActive)
    status |= CHASSIS_LINK_STATUS_ATTITUDE_FAULT;
if (chassisLinkIsAlive()) status |= CHASSIS_LINK_STATUS_LINK_OK;
if ((core2ControlRuntime.runState == CAR_RUN_LORA_LINKED) ||
    (core2ControlRuntime.runState == CAR_RUN_MODE4))
{

```

```

        uint8_t driveGate = 0u;
        if (core2ControlRuntime.runState == CAR_RUN_MODE4)
        {
            driveGate = loraRemoteIsUnlocked();
        }
        else
        {
            driveGate = core2ControlRuntime.effectiveLoraLaunchGate;
        }
        loraCmd = loraRemoteGetMotionForGate(driveGate);
    }

    memset(&ctrlSnap, 0, sizeof(ctrlSnap));
    ctrlSnap.magic = CORE2_CTRL_READY_MAGIC;
    ctrlSnap.seq = ++core2ControlRuntime.controlPublishSeq;
    ctrlSnap.replyType = (core2ControlRuntime.runState == CAR_RUN_LORA_LINKED)
        ? (uint8_t) CHASSIS_LINK_MSG_LORA_MOTION
        : (uint8_t) CHASSIS_LINK_MSG_TELEMETRY;

    ctrlSnap.status = status;
    ctrlSnap.takeoffReq = loraRemoteGetRemoteTakeoffReq(); /* 后备： cm_7_1 通常用本地新鲜值 */
    ctrlSnap.padding0 = 0u;
    ctrlSnap.teleVx = chassisSnapshot.actualSpeed.vx;
    ctrlSnap.teleVy = chassisSnapshot.actualSpeed.vy;
    ctrlSnap.teleWz = chassisSnapshot.actualSpeed.wz;
    ctrlSnap.loraCmdVx = loraCmd.vx;
    ctrlSnap.loraCmdVy = loraCmd.vy;
    ctrlSnap.loraCmdWz = loraCmd.wz;
    ctrlSnap.paramGlobalRequest =
        core2ControlRuntime.paramGlobalRequest;
    ctrlSnap.paramGlobalRequestSeq =
        core2ControlRuntime.paramGlobalRequestSequence;
    ctrlSnap.paramGlobalRequestValid =
        core2ControlRuntime.paramGlobalRequestValid;
    coreDataWriteControlToComm(&ctrlSnap);
}

#else
/* TUNE 调参固件： comm 仍在 cm_7_0 本地（chassisLink 真实实现）。原请求-应答遥测保留。 */
#if (MOTOR_TUNE_MODE == MOTOR_TUNE_LORA_TEST)
    loraRemotePoll(); /* 前台 owner 接收 SDK 已解析的完整 LoRa 帧并发布业务快照 */
#endif
    chassisLinkPoll();
    if (chassisLinkCommandReceived())
    {
        chassisApplicationSnapshotStruct chassisSnapshot;
        uint8_t status = 0u;
        chassisApplicationGetSnapshot(&chassisSnapshot);
        if (chassisSnapshot.yawRateEnabled)
        {
            status |= CHASSIS_LINK_STATUS_ENABLED;
        }
        if (chassisSnapshot.attitudeFaultActive)
            status |= CHASSIS_LINK_STATUS_ATTITUDE_FAULT;
        if (chassisLinkIsAlive()) status |= CHASSIS_LINK_STATUS_LINK_OK;
        chassisLinkSendTelemetry(
            chassisSnapshot.actualSpeed.vx,
            chassisSnapshot.actualSpeed.vy,
            chassisSnapshot.actualSpeed.wz,
            status);
    }
#endif

    #if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
        chassisMenuRenderTick(doDisplay);
    #endif
    #if (CORE2_BUILD_MODE == CORE2_BUILD_TUNE)
        if (doDisplay)
        {
            tuneDisplayUpdate();
        }
    #endif
}

#if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
/* 生产态 5 键已由 chassisMenuInputTick 接管（菜单导航 + HOME 亮度快调）； lightStripKeyScan 不再挂载。
 * LoRa 前台处理(状态机/导航/显示)仅在 LoRa 激活态(模式 3 源=LoRa 或 失链兜底模式 4)运行，
 * 与 pitProcess0 的 loraNavControlPit500us 门控一致(FOLLOW_485/WAIT_LINK 不服务 LoRa)。 */
if ((core2ControlRuntime.runState == CAR_RUN_LORA_LINKED) ||
    (core2ControlRuntime.runState == CAR_RUN_MODE4))
{

```

```

        loraNavControlWhile();
    }
    /* 合并 Flash 保存放到本轮通信、渲染和业务处理之后，避免阻塞新按键反馈。 */
    chassisMenuServiceDeferred();
#elif (MOTOR_TUNE_MODE == MOTOR_TUNE_CASCADE) || (MOTOR_TUNE_MODE == MOTOR_TUNE_CURRENT) || (MOTOR_TUNE_MODE ==
MOTOR_TUNE_OPEN_LOOP)
    tuneWheelKeyScan(); /* 调参选轮态：KEY_2..5 直选 FL/FR/BL/BR，KEY_1 复位默认轮（灯条已灭） */
#endif
}

void core2RuntimeGetSnapshot(core2RuntimeSnapshotStruct *out)
{
    if (out == 0) return;
#if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
    uint32_t irqWasEnabled = NVIC_GetEnableIRQ(CPUIntIdx2_IRQn);
    carRunStateEnum state;
    NVIC_DisableIRQ(CPUIntIdx2_IRQn);
    __DMB();
    state = core2ControlRuntime.runState;
    out->seq = core2ControlRuntime.runStateSeq;
    out->valid = core2ControlRuntime.initReady;
    out->runState = (uint8_t) state;
    out->standalone = (state == CAR_RUN_MODE4) ? 1u : 0u;
    out->core1LinkReady = chassisLinkIsAlive();
    out->core0ViaCore1Ready = chassisLinkGetCore0Alive();
    out->commandFresh = core2ControlRuntime.commFresh;
    out->enabled = core2ControlRuntime.rs485ControlEnabled;
    out->estop = core2ControlRuntime.rs485ControlEstop;
    out->launched = core2ControlRuntime.rs485ControlLaunched;
    out->rejoin = core2ControlRuntime.rejoinActive;
    __DMB();
    if (irqWasEnabled != 0u) NVIC_EnableIRQ(CPUIntIdx2_IRQn);
#else
    out->seq = 0u;
    out->valid = 0u;
    out->runState = 0u;
    out->standalone = 0u;
    out->core1LinkReady = 0u;
    out->core0ViaCore1Ready = 0u;
    out->commandFresh = 0u;
    out->enabled = 0u;
    out->estop = 0u;
    out->launched = 0u;
    out->rejoin = 0u;
#endif
}

```

core2_cyt4bb7/project/code/app/core0Loop.h

SHA-256

:

538b4f5d6406ab7dd766b0094b8b60634f639a69adcd09382b0644efe2133ef8

代码 (c)

```

#ifndef _CORE0LOOP_H_
#define _CORE0LOOP_H_

#include <stdint.h>

#include "chassisParams.h"
#include "paramSyncGlobal.h"

void processInit(void);
void pitProcess0(void);
void pitProcess1(void);
void whileProcess(void);

/* 菜单一次提交完整 working copy；动态字段同拍应用，控制器结构参数在安全态重建。 */
uint8_t core2RuntimeSubmitParams(const chassisParamStruct *candidate);
uint8_t core2RuntimeParamsApplyReady(void);
uint8_t core2RuntimeParamsApplied(void);
uint8_t core2RuntimeRequestSave(void);
uint8_t core2RuntimeRequestFactoryReset(void);
uint8_t core2RuntimeRequestGlobalParamAction(uint8_t action, uint8_t slot);

```

```

typedef enum
{
    CORE2_PARAM_GLOBAL_IDLE = 0u,
    CORE2_PARAM_GLOBAL_QUEUED,
    CORE2_PARAM_GLOBAL_PROGRESS,
    CORE2_PARAM_GLOBAL_SUCCESS,
    CORE2_PARAM_GLOBAL_FAILED
} core2ParamGlobalUiState;

typedef struct
{
    uint8_t state;
    uint8_t phase;
    uint8_t target;
    uint8_t error;
    uint8_t action;
    uint8_t slot;
    uint8_t slotUiState;
    uint8_t reserved;
    uint32_t eventSequence;
} core2ParamGlobalUiSnapshot;

static inline uint8_t core2ParamGlobalUiObserve(
    const paramSyncGlobalState *state,
    const paramSyncGlobalRequest *request,
    uint8_t *requestPending,
    core2ParamGlobalUiSnapshot *ui)
{
    if (state == 0 || request == 0 || requestPending == 0 || ui == 0 ||
        *requestPending == 0u ||
        paramSyncGlobalStateIsValid(state) == 0u ||
        paramSyncGlobalStateMatchesRequest(state, request) == 0u) return 0u;
    ui->phase = state->phase;
    ui->target = state->target;
    ui->error = state->error;
    ui->action = request->action;
    ui->slot = request->slot;
    if ((state->flags & PARAM_SYNC_GLOBAL_FLAG_TERMINAL) != 0u)
    {
        ui->state = ((state->flags & PARAM_SYNC_GLOBAL_FLAG_OK) != 0u) ?
            CORE2_PARAM_GLOBAL_SUCCESS : CORE2_PARAM_GLOBAL_FAILED;
        if (request->action == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
            ui->slotUiState = (ui->state == CORE2_PARAM_GLOBAL_SUCCESS) ?
                paramSyncGlobalClassifySlotSummary(state->transactionId) :
                PARAM_SYNC_SLOT_UI_UNAVAILABLE;
        *requestPending = 0u;
        return 1u;
    }
    if ((state->flags & PARAM_SYNC_GLOBAL_FLAG_BUSY) != 0u)
    {
        ui->state = CORE2_PARAM_GLOBAL_PROGRESS;
        return 1u;
    }
    return 0u;
}

static inline uint8_t core2ParamGlobalUiCancel(uint8_t *requestPending,
    core2ParamGlobalUiSnapshot *ui, uint8_t error)
{
    if (requestPending == 0 || ui == 0 || *requestPending == 0u) return 0u;
    *requestPending = 0u;
    ui->state = CORE2_PARAM_GLOBAL_FAILED;
    ui->phase = PARAM_SYNC_GLOBAL_PHASE_FAILED;
    ui->target = 2u;
    ui->error = error;
    return 1u;
}

static inline uint8_t core2ParamGlobalRequestCancellationReason(
    const paramSyncGlobalState *state, uint8_t stateValid,
    const paramSyncGlobalRequest *request, uint32_t coordinatorBootGeneration)
{
    if (state == 0 || request == 0 || stateValid == 0u ||
        paramSyncGlobalStateIsValid(state) == 0u)
        return PARAM_SYNC_GLOBAL_ERROR_ROUTE;
    if (coordinatorBootGeneration == 0u ||
        state->coordinatorBootGeneration != coordinatorBootGeneration)
        return PARAM_SYNC_GLOBAL_ERROR_PEER_RESTART;
    if (paramSyncGlobalStateReady(state) != 0u ||

```

```

        paramSyncGlobalStateMatchesRequest(state, request) != 0u)
        return PARAM_SYNC_GLOBAL_ERROR_NONE;
    if ((state->flags & PARAM_SYNC_GLOBAL_FLAG_RECOVERY) != 0u)
        return PARAM_SYNC_GLOBAL_ERROR_RECOVERY;
    return PARAM_SYNC_GLOBAL_ERROR_BUSY;
}

void core2RuntimeGetParamGlobalUi(core2ParamGlobalUiSnapshot *out);

typedef struct
{
    uint32_t seq;
    uint8_t valid;
    uint8_t runState; /* 0=WAIT 1=485 2=LoRa linked 3=LoRa standalone */
    uint8_t standalone;
    uint8_t core1LinkReady;
    uint8_t core0ViaCore1Ready;
    uint8_t commandFresh;
    uint8_t enabled;
    uint8_t estop;
    uint8_t launched;
    uint8_t rejoin;
} core2RuntimeSnapshotStruct;

void core2RuntimeGetSnapshot(core2RuntimeSnapshotStruct *out);

#endif

```

core2_cyt4bb7/project/code/app/core1Loop.c

SHA-256

:

56ff5053454be5caa3ad796748b039eea195b60abc76cfbe836881ac9c98acd1

代码 (c)

```

#include "core1Loop.h"

#include "configCore2Build.h" /* CORE2_BUILD_MODE / AG_CAR_RS485_WAIT_MS */
#include "configChassisRemote.h"

#include "zf_driver_pit.h"

/*
 * core2 cm_7_1 = 通信核（生产固件）。
 *
 * 把 RS-485(THVD1450/UART4) 与 LoRa(loraRemote/UART1) 的收发整体放到本空闲核：
 * - 中断驱动 RX： UART4 随 ISR 搬入 THVD 本地 ring， UART1 保留逐飞缓冲，前台 Poll 解析；
 * - RS-485/LoRa 的 RX 中断风暴只拖慢本核， cm_7_0 的 500µs 控制 PIT 被彻底隔离（历史卡死根因消除）。
 * 经 coreDataTransfer 共享内存与 cm_7_0 控制核桥接：
 * - 发布 commToControl： RS-485 命令快照 + LoRa 解码运动/解锁/单按边沿(seq)；
 * - 读取 controlToComm： 应答类型 + 遥测实速，收到 core1 MSG_COMMAND 请求后照发（请求-应答）。
 *
 * 调参固件(CORE2_BUILD_TUNE) 下本核保持空占位（comm 仍在 cm_7_0，调参不联机）。
 */

#if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)

#include <string.h>

#include "chassisLink.h"
#include "loraRemote.h"
#include "coreDataTransfer.h"

#define COMM_TICK_PERIOD_US (1000u) /* PIT_CH2： 1ms tick 驱动链路年龄 + LoRa 超时/chord/edge 状态机 */

typedef struct
{
    volatile uint8_t initReady;
    volatile uint32_t rs485DeadMs;
    volatile uint32_t recordSeq;
    volatile uint32_t replaySeq;
    uint32_t publishSeq;
    volatile uint32_t tickMs;
    controlToCommStruct controlSnapshot;
    uint32_t lastControlSeq;
}

```

```

    uint32_t lastControlTickMs;
    uint8_t controlSeen;
} core2CommRuntimeContextStruct;

static core2CommRuntimeContextStruct core2CommRuntime = {0};

void processInit(void)
{
    chassisLinkInit(); /* 真实实现：thvd1450Init (cm_7_1 内已开 UART4 RX 中断 + DE=P13_7) */
    loraRemoteInit(); /* 真实实现：lora3a22_init + 开 UART1 RX 中断 + 状态复位 */

    pit_us_init(PIT_CH2, COMM_TICK_PERIOD_US); /* 1ms tick → pit0_ch2_isr → pitProcess0 */
    core2CommRuntime.initReady = 1u;
}

void pitProcess0(void)
{
    uint8_t standalone;

    if (!core2CommRuntime.initReady) return;

    chassisLinkTickMs(); /* RS-485 链路年龄 +1ms */

    /* 本核失链计时 (与 cm_7_0 同语义)：用于 standalone(模式 4 脱机) 判定。 */
    if (chassisLinkIsAlive())
    {
        core2CommRuntime.rs485DeadMs = 0u;
    }
    else if (core2CommRuntime.rs485DeadMs < AG_CAR_RS485_WAIT_MS)
    {
        core2CommRuntime.rs485DeadMs++;
    }

    /* standalone=1 (模式 4 脱机)：S3+S4 chord 切换本地解锁；=0 (模式 2/3 联机)：chord 置双肩键请求。
    * 与 cm_7_0 carRunState 的模式 4 判断一致 (RS-485 失链 >AG_CAR_RS485_WAIT_MS)，具体语义由上层系统模式解释。 */
    standalone = (core2CommRuntime.rs485DeadMs >= AG_CAR_RS485_WAIT_MS) ? 1u : 0u;
    loraRemoteTickMs(standalone); /* 超时维护 + chord/edge 状态机 */
    core2CommRuntime.tickMs++;

    /* 单按边沿 (read-and-clear) 转单调 seq，跨核交付给 cm_7_0 (防 read-and-clear 跨核竞态)。 */
    if (loraRemoteRecordEdge()) core2CommRuntime.recordSeq++;
    if (loraRemoteReplayEdge()) core2CommRuntime.replaySeq++;
}

void whileProcess(void)
{
    commToControlStruct out;
    controlToCommStruct reply;
    chassisLinkCommandStruct cmd;
    loraRemoteSnapshotStruct loraSnapshot;
    paramSyncGlobalState paramGlobalState;
    uint8_t controlFresh;

    if (!core2CommRuntime.initReady) return;

    /* 1. 前台排空 RX (UART4 本地 ring/UART1 逐飞缓冲)，解析帧/LoRa 摇杆。 */
    loraRemotePoll();
    chassisLinkPoll();

    /* 2. 构建并发布 commToControl (cm_7_1 → cm_7_0)。 */
    chassisLinkGetCommand(&cmd);
    loraRemoteGetSnapshot(&loraSnapshot);
    memset(&out, 0, sizeof(out));
    out.magic = CORE2_COMM_READY_MAGIC;
    out.seq = ++core2CommRuntime.publishSeq;
    out.cmdVx = cmd.vx;
    out.cmdVy = cmd.vy;
    out.cmdWz = cmd.wz;
    out.cmd = cmd.cmd;
    out.sysByte = cmd.sysByte;
    out.linkAlive = chassisLinkIsAlive();
    out.loraVxRaw = loraSnapshot.joystick[1];
    out.loraVyRaw = loraSnapshot.joystick[0];
    out.loraWzRaw = loraSnapshot.joystick[2];
    out.loraUnlocked = loraSnapshot.unlocked;
    out.loraLinkAlive = loraSnapshot.linkAlive;
    out.takeoffReq = loraSnapshot.takeoffReq;
    out.recordSeq = core2CommRuntime.recordSeq;
    out.replaySeq = core2CommRuntime.replaySeq;

```

```

if (chassisLinkGetParamGlobalState(&paramGlobalState) != 0u)
{
    out.paramGlobalState = paramGlobalState;
    out.paramGlobalStateSeq = paramGlobalState.sequence;
    out.paramGlobalStateValid = 1u;
}
coreDataWriteCommToControl(&out);

/* 前台持续刷新控制核应答缓存；事务失败或停更超过 500ms 时统一按无效处理。 */
if (coreDataReadControlToComm(&reply) != 0u &&
    ((core2CommRuntime.controlSeen == 0u) || (reply.seq != core2CommRuntime.lastControlSeq)))
{
    core2CommRuntime.controlSnapshot = reply;
    core2CommRuntime.lastControlSeq = reply.seq;
    core2CommRuntime.lastControlTickMs = core2CommRuntime.tickMs;
    core2CommRuntime.controlSeen = 1u;
}
controlFresh = (uint8_t) (core2CommRuntime.controlSeen != 0u &&
    (uint32_t) (core2CommRuntime.tickMs -
        core2CommRuntime.lastControlTickMs) < LORA_LINK_TIMEOUT_MS);
if (core2CommRuntime.controlSeen != 0u)
{
    chassisLinkSetParamGlobalRequest(
        &core2CommRuntime.controlSnapshot.paramGlobalRequest,
        (uint8_t) (controlFresh != 0u &&
            core2CommRuntime.controlSnapshot.paramGlobalRequestValid != 0u));
}
/* 3. RS-485 请求-应答：收到 core1 的 MSG_COMMAND 才回一帧应答（半双工总线由 core1 命令串行化）。
* 应答类型/遥测由 cm_7_0 经 controlToComm 决定；LoRa 运动使用 cm_7_0 按本地参数映射后的命令，
* 避免通信核编译期默认值与控制核菜单/flash 参数不一致。
* controlToComm magic 未就绪（cm_7_0 还没发布）时回遥测零值，保持 core1 视为链路在。 */
if (chassisLinkCommandReceived())
{
    uint8_t replyValid;
    replyValid = controlFresh;
    if (replyValid != 0u) reply = core2CommRuntime.controlSnapshot;
    else memset(&reply, 0, sizeof(reply));

    if (replyValid && reply.replyType == CHASSIS_LINK_MSG_LORA_MOTION)
    {
        chassisMotionCommandStruct fallback = loraRemoteGetMotion();
        float vx = reply.loraCmdVx;
        float vy = reply.loraCmdVy;
        float wz = reply.loraCmdWz;
        if (reply.seq == 0u)
        {
            vx = fallback.vx;
            vy = fallback.vy;
            wz = fallback.wz;
        }
        chassisLinkSendLoraMotion(vx, vy, wz, loraSnapshot.takeoffReq,
            (uint8_t) ((reply.status & CHASSIS_LINK_STATUS_ATTITUDE_FAULT) != 0u));
    }
    else
    {
        float vx = replyValid ? reply.teleVx : 0.0f;
        float vy = replyValid ? reply.teleVy : 0.0f;
        float wz = replyValid ? reply.teleWz : 0.0f;
        uint8_t status = replyValid ? reply.status : 0u;
        chassisLinkSendTelemetry(vx, vy, wz, status);
    }
}
}

#else /* CORE2_BUILD_TUNE: cm_7_1 空占位（comm 仍在 cm_7_0） */

static volatile uint8_t core2CommRuntimeInitReady = 0u;

void processInit(void)
{
    core2CommRuntimeInitReady = 1u;
}

void pitProcess0(void)
{
    if (!core2CommRuntimeInitReady) return;
}

void whileProcess(void)

```



```

{
    if (!core2CommRuntimeInitReady) return;
}

#endif

```

core2_cyt4bb7/project/code/app/core1Loop.h

SHA-256

:

4dc1dd25d5d641c5da977e64d819c3d4dd58f39a38bd78cf1795754c3002fa2d

代码 (c)

```

#ifndef _CORE1LOOP_H_
#define _CORE1LOOP_H_

#include <stdint.h>

void processInit(void);
void pitProcess0(void);
void whileProcess(void);

#endif

```

core2_cyt4bb7/project/code/app/lightStrip.c

SHA-256

:

ac9befcbb8fcf7e20585afad73d9bc2b2a065af2cfc66dca84245db80ce43b67

代码 (c)

```

#include "lightStrip.h"

#include "menuKey.h"
#include "pinMap.h"

static volatile uint8_t lightStripBrightness = LIGHT_STRIP_DEFAULT_BRIGHTNESS;

static uint32_t brightnessToDuty(uint8_t pct)
{
    /* PWM_DUTY_MAX = 10 000,  pct 0..100 → 0..10000 */
    return (uint32_t) pct * (PWM_DUTY_MAX / 100u);
}

static uint8_t clampBrightness(int16_t value)
{
    if (value < (int16_t) LIGHT_STRIP_BRIGHTNESS_MIN)
    {
        return LIGHT_STRIP_BRIGHTNESS_MIN;
    }
    if (value > (int16_t) LIGHT_STRIP_BRIGHTNESS_MAX)
    {
        return LIGHT_STRIP_BRIGHTNESS_MAX;
    }
    return (uint8_t) value;
}

void lightStripInit(void)
{
    lightStripBrightness = LIGHT_STRIP_DEFAULT_BRIGHTNESS;
    pwm_init(LIGHT_STRIP_PWM_CHANNEL, LIGHT_STRIP_PWM_FREQ_HZ,
        brightnessToDuty(lightStripBrightness));
}

void lightStripSetBrightness(uint8_t pct)
{
    uint8_t clamped = clampBrightness((int16_t) pct);
    lightStripBrightness = clamped;
    pwm_set_duty(LIGHT_STRIP_PWM_CHANNEL, brightnessToDuty(clamped));
}

```

```

/* 消费一次按键事件（短按或长按均算一次，等价原边沿“按一下调一次”，不连发）。
 * 读到非 RELEASE 即 Clear 并返回 1；长按经 Clear 后进入等释放，不会重复触发。*/
static uint8_t keyConsumed(menuKeyIndexEnum k)
{
    if (menuKeyGetState(k) != MENU_KEY_RELEASE)
    {
        menuKeyClear(k);
        return 1u;
    }
    return 0u;
}

void lightStripKeyScan(void)
{
    int16_t next = (int16_t) lightStripBrightness;
    uint8_t changed = 0u;

    if (keyConsumed(MENU_KEY_1))
    {
        next = (int16_t) LIGHT_STRIP_DEFAULT_BRIGHTNESS;
        changed = 1u;
    }
    if (keyConsumed(MENU_KEY_2))
    {
        next -= (int16_t) LIGHT_STRIP_STEP_COARSE;
        changed = 1u;
    }
    if (keyConsumed(MENU_KEY_3))
    {
        next -= (int16_t) LIGHT_STRIP_STEP_FINE;
        changed = 1u;
    }
    if (keyConsumed(MENU_KEY_4))
    {
        next += (int16_t) LIGHT_STRIP_STEP_FINE;
        changed = 1u;
    }
    if (keyConsumed(MENU_KEY_5))
    {
        next += (int16_t) LIGHT_STRIP_STEP_COARSE;
        changed = 1u;
    }

    if (changed)
    {
        lightStripSetBrightness(clampBrightness(next));
    }
}

```

core2_cyt4bb7/project/code/app/lightStrip.h

SHA-256

:

bbeb85ad464cb3a6548d7069a4c6c16668f572f2c300e6b46603e3fd25ad8736

代码 (c)

```

#ifndef _LIGHT_STRIP_H_
#define _LIGHT_STRIP_H_

#include "zf_common_typedef.h"
#include "zf_driver_pwm.h"

/* core2 cm7_0 灯光 PWM owner: ^ 形定位灯条调光。
 *
 * 默认值以宏定义形式给出，便于实测后修改。
 * ^ 灯条实际引脚仍以 LIGHT_STRIP_PWM_CHANNEL 占位，硬件确认后需同步 PINOUT；
 *
 * 占空比映射：
 * brightness_pct (0~100) → 暗 → 亮
 * 写入寄存器 = brightness_pct * (PWM_DUTY_MAX / 100) = brightness_pct * 100
 *
 * 亮度调节入口（PRODUCTION 大类）：已移入板载菜单 WAIT/HOME 快调（chassisMenu.c），
 * 菜单提交 working copy 后由 core2Runtime 调 lightStripSetBrightness 热生效，
 * 亮度随参数仓库显式保存到 flash；core0Loop.c 上电按活动参数初始化。

```

```
* lightStripKeyScan 下方独立 5 键绑定函数保留为 API，但 PRODUCTION 不再挂载（5 键归菜单导航）。
*
* lightStripKeyScan 独立按键绑定（menuKey 5 键短按，KEY_1=P19_4..KEY_5=P20_3；当前未挂载）：
* KEY_1 = 复位为 LIGHT_STRIP_DEFAULT_BRIGHTNESS
* KEY_2 = -LIGHT_STRIP_STEP_COARSE（粗调减）
* KEY_3 = -LIGHT_STRIP_STEP_FINE（微调减）
* KEY_4 = +LIGHT_STRIP_STEP_FINE（微调加）
* KEY_5 = +LIGHT_STRIP_STEP_COARSE（粗调加）
*/

#define LIGHT_STRIP_PWM_FREQ_HZ (20000u)
#define LIGHT_STRIP_DEFAULT_BRIGHTNESS (60.0f)
#define LIGHT_STRIP_BRIGHTNESS_MIN (0u)
#define LIGHT_STRIP_BRIGHTNESS_MAX (100u)
#define LIGHT_STRIP_STEP_FINE (1u)
#define LIGHT_STRIP_STEP_COARSE (10u)

/* PWM 通道引脚 LIGHT_STRIP_PWM_CHANNEL 见 pinMap.h */

void lightStripInit(void);
void lightStripSetBrightness(uint8_t pct);

/* 独立按键调亮度 API：消费 menuKey 5 键事件更新亮度并刷新 PWM。
* 注：PRODUCTION 已改用菜单 WAIT/HOME 快调亮度，core0Loop.c 不再挂载本函数（防与菜单争 5 键）。保留备用。*/
void lightStripKeyScan(void);

#endif /* _LIGHT_STRIP_H_ */
```

core2_cyt4bb7/project/code/app/menuFloatDisplayInternal.h

SHA-256

:

e5c06fecc348bcaa914fc246d7a0df4c1c07169fdbf396c61be6a168d0ec125f

代码 (c)

```
#ifndef MENU_FLOAT_DISPLAY_INTERNAL_H
#define MENU_FLOAT_DISPLAY_INTERNAL_H

#include <stdint.h>

#include "zf_device_ips114.h"

#define MENU_FLOAT_DISPLAY_DECIMALS_MAX (8u)
#define MENU_FLOAT_DISPLAY_CHARS (15u)

static uint8_t menuFloatDisplayDigitsU64(uint64_t value)
{
    uint8_t digits = 1u;

    while (value >= 10u)
    {
        value /= 10u;
        digits++;
    }
    return digits;
}

static void menuFloatDisplayShowLiteral(uint16_t x,
    uint16_t y,
    const char *literal)
{
    char buf[MENU_FLOAT_DISPLAY_CHARS + 1u];
    uint8_t i;
    uint8_t length = 0u;

    while ((literal[length] != '\0') && (length < MENU_FLOAT_DISPLAY_CHARS))
    {
        length++;
    }
    for (i = 0u; i < MENU_FLOAT_DISPLAY_CHARS; i++)
    {
        buf[i] = ' ';
    }
    for (i = 0u; i < length; i++)
    {

```

```

        buf[MENU_FLOAT_DISPLAY_CHARS - length + i] = literal[i];
    }
    buf[MENU_FLOAT_DISPLAY_CHARS] = '\0';
    ips114_show_string(x, y, buf);
}

static uint8_t menuFloatDisplayDecimalsForStep(float step)
{
    uint8_t decimals = 0u;
    double scaled = (step < 0.0f) ? -(double) step : (double) step;

    if (!(scaled > 0.0))
    {
        return MENU_FLOAT_DISPLAY_DECIMALS_MAX;
    }
    while (decimals < MENU_FLOAT_DISPLAY_DECIMALS_MAX)
    {
        uint64_t rounded;
        double difference;

        if (!(scaled < (double) UINT64_MAX) ||
            !((scaled + 0.5) < (double) UINT64_MAX))
        {
            return MENU_FLOAT_DISPLAY_DECIMALS_MAX;
        }
        rounded = (uint64_t) (scaled + 0.5);
        difference = scaled - (double) rounded;

        if (difference < 0.0)
        {
            difference = -difference;
        }
        if (difference <= 0.000001)
        {
            break;
        }
        scaled *= 10.0;
        decimals++;
    }
    return decimals;
}

static uint8_t menuFloatDisplayDecimalsForValue(double value)
{
    uint8_t decimals;
    double scale = 1.0;
    double tolerance = 0.0000001;

    if (value < 0.0)
    {
        value = -value;
    }
    if (!(value >= 0.0) || !(value < (double) UINT64_MAX))
    {
        return MENU_FLOAT_DISPLAY_DECIMALS_MAX;
    }
    for (decimals = 0u; decimals < MENU_FLOAT_DISPLAY_DECIMALS_MAX; decimals++)
    {
        double scaled = value * scale;
        uint64_t rounded;
        double difference;

        if (!(scaled < (double) UINT64_MAX) ||
            !((scaled + 0.5) < (double) UINT64_MAX))
        {
            return MENU_FLOAT_DISPLAY_DECIMALS_MAX;
        }
        rounded = (uint64_t) (scaled + 0.5);
        difference = value - ((double) rounded / scale);

        if (difference < 0.0)
        {
            difference = -difference;
        }
        if (difference <= tolerance)
        {
            return decimals;
        }
        scale *= 10.0;
    }
}

```

```

    }
    return MENU_FLOAT_DISPLAY_DECIMALS_MAX;
}

static uint8_t menuFloatDisplayDecimalsForValueAndStep(double value, float step)
{
    uint8_t stepDecimals = menuFloatDisplayDecimalsForStep(step);
    uint8_t valueDecimals = menuFloatDisplayDecimalsForValue(value);

    return (stepDecimals > valueDecimals) ? stepDecimals : valueDecimals;
}

static void menuFloatDisplayShow(uint16_t x,
    uint16_t y,
    double value,
    uint8_t requestedDecimals)
{
    static const uint64_t scaleTable[MENU_FLOAT_DISPLAY_DECIMALS_MAX + 1u] =
    {
        1u, 10u, 100u, 1000u, 10 000u,
        100 000u, 1 000 000u, 10 000 000u, 100 000 000u
    };
    char buf[MENU_FLOAT_DISPLAY_CHARS + 1u];
    uint8_t decimals;
    uint8_t selectedDecimals = 0xFFu;
    uint8_t negative = 0u;
    uint8_t pos;
    uint8_t i;
    uint64_t scale = 1u;
    uint64_t scaled = 0u;
    uint64_t intPart;
    uint64_t fracPart;

    if (value != value)
    {
        menuFloatDisplayShowLiteral(x, y, "NaN");
        return;
    }
    if (value < 0.0)
    {
        negative = 1u;
        value = -value;
    }

    decimals = (requestedDecimals <= MENU_FLOAT_DISPLAY_DECIMALS_MAX)
        ? requestedDecimals
        : MENU_FLOAT_DISPLAY_DECIMALS_MAX;
    for (; decimals--> 0)
    {
        double scaledDouble;
        double roundedDouble;
        uint8_t required;

        scale = scaleTable[decimals];
        scaledDouble = value * (double) scale;
        roundedDouble = scaledDouble + 0.5;
        if ((scaledDouble < (double) UINT64_MAX) &&
            (roundedDouble < (double) UINT64_MAX))
        {
            scaled = (uint64_t) roundedDouble;
            intPart = scaled / scale;
            required = (uint8_t) (negative + menuFloatDisplayDigitsU64(intPart));
            if (decimals != 0u)
            {
                required = (uint8_t) (required + 1u + decimals);
            }
            if (required <= MENU_FLOAT_DISPLAY_CHARS)
            {
                selectedDecimals = decimals;
                break;
            }
        }
    }
    if (decimals == 0u)
    {
        break;
    }
}
if (selectedDecimals == 0xFFu)
{

```

```

        menuFloatDisplayShowLiteral(x, y, "OVERFLOW");
        return;
    }

    for (i = 0u; i < MENU_FLOAT_DISPLAY_CHARS; i++)
    {
        buf[i] = ' ';
    }
    buf[MENU_FLOAT_DISPLAY_CHARS] = '\0';
    intPart = scaled / scale;
    fracPart = scaled % scale;
    pos = MENU_FLOAT_DISPLAY_CHARS;
    for (i = 0u; i < selectedDecimals; i++)
    {
        buf[--pos] = (char) ('0' + (fracPart % 10u));
        fracPart /= 10u;
    }
    if (selectedDecimals != 0u)
    {
        buf[--pos] = '.';
    }
    do
    {
        buf[--pos] = (char) ('0' + (intPart % 10u));
        intPart /= 10u;
    } while ((intPart != 0u) && (pos > 0u));
    if ((negative != 0u) && (pos > 0u))
    {
        buf[--pos] = '-';
    }

    ips114_show_string(x, y, buf);
}

#endif

```

core2_cyt4bb7/project/code/app/menuKey.c

SHA-256 :
e0dc39f77f4171ed3d836b5cd53d295fa46e39386508a2818fd0817ab382d473

代码（c）

```

#include "menuKey.h"

#include "pinMap.h"
#include "zf_driver_gpio.h"

/*
 * 实现：5 路 GPI 上拉，按下 = GPIO_LOW（移植自 core0 menuKey，逻辑等价）。
 * menuKeyScan 在 PIT ISR 周期调用，按 zf_device_key 等价语义实现：
 * - KEY_RELEASE_LEVEL = GPIO_HIGH（释放）
 * - SHOCK_PERIOD_MS = 10ms（抖动）
 * - LONG_PERIOD_MS = 1000ms（短按/长按门限）
 * 时间门限按 scanPeriodMs 累计（与调用周期解耦）：core2 以 1ms 调用即 10/1000 拍。
 * 功能：短按锁存到 menuKeyClear；短按超时自动清；长按消费后等物理释放防重复触发。
 * 引脚 core2 顺序：KEY_1=P19_4 ... KEY_5=P20_3（pinMap.h KEY_INPUT_KEYx_PIN）。
 */

#define MENU_KEY_RELEASE_LEVEL (GPIO_HIGH)
#define MENU_KEY_SHOCK_PERIOD_MS (10u)
#define MENU_KEY_LONG_PERIOD_MS (1000u)

static const gpio_pin_enum keyPins[MENU_KEY_COUNT] =
{
    KEY_INPUT_KEY1_PIN, /* KEY_1 = P19_4 */
    KEY_INPUT_KEY2_PIN, /* KEY_2 = P20_0 */
    KEY_INPUT_KEY3_PIN, /* KEY_3 = P20_1 */
    KEY_INPUT_KEY4_PIN, /* KEY_4 = P20_2 */
    KEY_INPUT_KEY5_PIN, /* KEY_5 = P20_3 */
};

#include "menuKeyStateMachineImpl.h"

```

core2_cyt4bb7/project/code/app/menuKey.h

SHA-256

:

9f40d32d19234e8f04bb199df7cae32266fd5f7ba2c7f4b939e2a70ed5028a9e

代码 (c)

```
#ifndef _MENU_KEY_H_
#define _MENU_KEY_H_

#include <stdint.h>

/*
 * core2 板载按键扫描封装（移植自 core0 cm_7_1 menuKey，状态机版）
 *
 * 替代原 keyInput 边沿驱动：提供 短按 / 长按 区分 + 锁存 + 超时清 + 等释放。
 * 实现：
 *   - 5 路 GPIO 直读（pinMap.h KEY_INPUT_KEYx_PIN）+ 周期消抖
 *   - 短按 / 长按 区分（释放时根据持续时长判定）
 *   - 短按锁存：短按事件保留至 menuKeyGetState 读取后由 menuKeyClear 清除
 *   - 短按长时间未消费时自动超时清除（防陈旧事件误触发）
 *   - 长按消费(menuKeyClear)后置“等释放”：按住期间不再重复触发，必须物理释放后才允许下一次事件
 *
 * 引脚顺序：core2 主板 KEY_1=P19_4，KEY_2..5=P20_0..P20_3（与 coreT1/coreT2 反向，
 * 也与 core0 menuKey 的 KEY_1=P20_3 反向）。索引语义见 pinMap.h。
 *
 * 线程模型：
 *   menuKeyScan() 在 PIT ISR 周期调用（core2：500μs PIT ÷ KEY_TICK_DIVIDER = 1ms）；
 *   menuKeyGetState/Clear 在前台消费；状态数组 volatile 跨 ISR/前台共享。
 */

typedef enum
{
    MENU_KEY_1 = 0u,
    MENU_KEY_2,
    MENU_KEY_3,
    MENU_KEY_4,
    MENU_KEY_5,
    MENU_KEY_COUNT
} menuKeyIndexEnum;

typedef enum
{
    MENU_KEY_RELEASE = 0u,
    MENU_KEY_SHORT_PRESS,
    MENU_KEY_LONG_PRESS
} menuKeyStateEnum;

/* 初始化：配置 GPI 上拉 + 清状态。periodMs = 调用 menuKeyScan 的周期(毫秒) */
void menuKeyInit(uint32_t periodMs);

/* 扫描：在 PIT ISR 周期调用 */
void menuKeyScan(void);

/* 查询按键状态（短按事件保留至 menuKeyClear 调用） */
menuKeyStateEnum menuKeyGetState(menuKeyIndexEnum k);

/* 清除指定按键状态（消费事件） */
void menuKeyClear(menuKeyIndexEnum k);
void menuKeyDiscardUntilRelease(menuKeyIndexEnum k);

#endif /* _MENU_KEY_H_ */
```

core2_cyt4bb7/project/code/app/menuKeyStateMachineImpl.h

SHA-256

:

59535b1e043685925409616f368a3a31b77aec3c8e1a5054def43f33e20be6e5

代码 (c)

```

#ifndef MENU_KEY_STATE_MACHINE_IMPL_H
#define MENU_KEY_STATE_MACHINE_IMPL_H

/* 单翻译单元实现片段：由 menuKey.c 在目标专用 keyPins 定义后包含一次。 */
static volatile uint32_t scanPeriodMs = 0u;
static volatile uint32_t pressTimeMs[MENU_KEY_COUNT];
static volatile uint32_t shortHoldMs[MENU_KEY_COUNT];
static volatile menuKeyStateEnum keyState[MENU_KEY_COUNT];
static volatile uint8_t waitRelease[MENU_KEY_COUNT]; /* 消费后等物理释放，防长按按住期间重复触发 */

void menuKeyInit(uint32_t periodMs)
{
    if (periodMs == 0u) periodMs = 1u;
    scanPeriodMs = periodMs;
    for (uint8_t i = 0u; i < MENU_KEY_COUNT; i++)
    {
        gpio_init(keyPins[i], GPI, GPIO_HIGH, GPI_PULL_UP);
        keyState[i] = MENU_KEY_RELEASE;
        pressTimeMs[i] = 0u;
        shortHoldMs[i] = 0u;
        waitRelease[i] = 0u;
    }
}

void menuKeyScan(void)
{
    for (uint8_t i = 0u; i < MENU_KEY_COUNT; i++)
    {
        uint8_t pressed = (gpio_get_level(keyPins[i]) != MENU_KEY_RELEASE_LEVEL) ? 1u : 0u;

        /* 等释放期间：忽略按键直到物理释放，期间不累计、不产状态 */
        if (waitRelease[i])
        {
            if (!pressed)
            {
                waitRelease[i] = 0u;
                keyState[i] = MENU_KEY_RELEASE;
                pressTimeMs[i] = 0u;
                shortHoldMs[i] = 0u;
            }
            continue;
        }

        /* 短按状态锁存超时清除 */
        if (keyState[i] == MENU_KEY_SHORT_PRESS)
        {
            shortHoldMs[i] += scanPeriodMs;
            if (shortHoldMs[i] >= MENU_KEY_LONG_PERIOD_MS)
            {
                keyState[i] = MENU_KEY_RELEASE;
                shortHoldMs[i] = 0u;
            }
        }

        if (pressed)
        {
            /* 按下：累计时长，达长按门限即升级为长按 */
            pressTimeMs[i] += scanPeriodMs;
            if (pressTimeMs[i] >= MENU_KEY_LONG_PERIOD_MS)
            {
                keyState[i] = MENU_KEY_LONG_PRESS;
            }
        }
        else
        {
            /* 释放：根据持续时长判定短按/释放 */
            if (keyState[i] != MENU_KEY_LONG_PRESS &&
                pressTimeMs[i] >= MENU_KEY_SHOCK_PERIOD_MS &&
                pressTimeMs[i] < MENU_KEY_LONG_PERIOD_MS)
            {
                if (keyState[i] != MENU_KEY_SHORT_PRESS)
                {
                    keyState[i] = MENU_KEY_SHORT_PRESS;
                    shortHoldMs[i] = 0u;
                }
            }
            else if ((keyState[i] != MENU_KEY_SHORT_PRESS) &&
                (keyState[i] != MENU_KEY_LONG_PRESS))
            {
            }
        }
    }
}

```



```

        keyState[i] = MENU_KEY_RELEASE;
    }
    pressTimeMs[i] = 0u;
}
}
}

menuKeyStateEnum menuKeyGetState(menuKeyIndexEnum k)
{
    if (k >= MENU_KEY_COUNT) return MENU_KEY_RELEASE;
    return keyState[k];
}

void menuKeyClear(menuKeyIndexEnum k)
{
    if (k >= MENU_KEY_COUNT) return;
    /* 长按消费后置等释放：防按住期间 scan 立刻再次升级为 LONG_PRESS 重复触发。
     * 短按消费正常清零即可（短按事件本就一次性锁存）。 */
    if (keyState[k] == MENU_KEY_LONG_PRESS) waitRelease[k] = 1u;
    keyState[k] = MENU_KEY_RELEASE;
    pressTimeMs[k] = 0u;
    shortHoldMs[k] = 0u;
}

void menuKeyDiscardUntilRelease(menuKeyIndexEnum k)
{
    if (k >= MENU_KEY_COUNT) return;
    waitRelease[k] = 1u;
    keyState[k] = MENU_KEY_RELEASE;
    pressTimeMs[k] = 0u;
    shortHoldMs[k] = 0u;
}

#endif

```

core2_cyt4bb7/project/code/app/paramSyncMotorSafety.h

SHA-256

:

32d12f8f74a4b4893c3d8da2d59ee061b245fb0eb2c2341acb964301fed53eea

代码 (c)

```

#ifndef PARAM_SYNC_MOTOR_SAFETY_H
#define PARAM_SYNC_MOTOR_SAFETY_H

#include <math.h>
#include <stddef.h>
#include <stdint.h>

#include "motorControl.h"

static inline uint8_t core2ParamSyncMotorSnapshotIsStopped(
    const motorControlSnapshotStruct *motor)
{
    if (motor == NULL || motor->valid == 0u ||
        !isfinite(motor->fl.feedbackSpeed) ||
        !isfinite(motor->fr.feedbackSpeed) ||
        !isfinite(motor->bl.feedbackSpeed) ||
        !isfinite(motor->br.feedbackSpeed) ||
        !isfinite(motor->fl.motorOutput) ||
        !isfinite(motor->fr.motorOutput) ||
        !isfinite(motor->bl.motorOutput) ||
        !isfinite(motor->br.motorOutput) ||
        motor->fl.feedbackSpeed != 0.0f ||
        motor->fr.feedbackSpeed != 0.0f ||
        motor->bl.feedbackSpeed != 0.0f ||
        motor->br.feedbackSpeed != 0.0f ||
        motor->fl.motorOutput != 0.0f ||
        motor->fr.motorOutput != 0.0f ||
        motor->bl.motorOutput != 0.0f ||
        motor->br.motorOutput != 0.0f) return 0u;
    return 1u;
}

#endif

```

core2_cyt4bb7/project/code/app/paramSyncOwnerService.c

SHA-256

:

87a819862d49374b42aeca46b565237af00cbd6cef455fadcf3081646485aef

代码 (c)

```
#include "paramSyncOwnerService.h"

#include <stddef.h>
#include <string.h>

#include "chassisParams.h"
#include "core0Loop.h"
#include "crc32Ieee.h"
#include "paramSyncMailbox.h"

#define CORE2_PARAM_SYNC_TARGET    (2u)
#define CORE2_FLASH_MAGIC          (0x4D324B30u)
#define CORE2_FLASH_VERSION        (9u)
#define CORE2_SCHEMA_HASH          (0xF80 812BBu)
#define CORE2_ACTIVATE_ROLLBACK_MAGIC (0xA2B20 000u)
#define PARAM_STAGE_NONE           (0u)
#define PARAM_STAGE_LIBRARY         (1u)
#define PARAM_STAGE_RUNTIME         (2u)

typedef struct
{
    uint32_t magic;
    uint32_t version;
    uint32_t payloadBytes;
    chassisParamStruct params;
    uint32_t crc;
} core2ParamSyncBlob;

typedef char core2ParamSyncBlobSizeAssert[(sizeof(core2ParamSyncBlob) == 176u) ? 1 : -1];

static uint32_t lastBootGeneration, lastRequestSequence, responseSequence;
static uint32_t ownerBootGeneration;
static core2ParamSyncBlob stagedBlob;
static uint32_t stagedTransaction, stagedExpectedCrc;
static uint16_t stagedBytes, stagedReceived;
static uint8_t stagedSlot, stagedActive, stagedPrepared, stagedCommitted;
static uint8_t stagedKind, stagedApplied;
static uint8_t stagedApplyRequestValid, stagedApplySubmitted;
static paramSyncMailboxRequest stagedApplyRequest;
static uint8_t stagedAbortRequestValid, stagedAbortSubmitted;
static paramSyncMailboxRequest stagedAbortRequest;
static uint8_t stagedFinishRequestValid, stagedFinishSubmitted;
static paramSyncMailboxRequest stagedFinishRequest;
static uint8_t activationRecoveryChecked;
static uint8_t syncMutationLocked;
static uint8_t persistentPendingLocked;
static uint8_t syncLockedSlot;
static uint32_t syncLockedTransaction;
static uint32_t lastFinishedTransaction;
static uint32_t lastAbortedTransaction;
#if defined(_ICCARM_)
__no_init uint32_t core2ParamSyncOwnerBootRetained;
__no_init uint32_t core2ParamSyncOwnerBootInverseRetained;
__no_init uint32_t core2ParamSyncActivationRollbackRetained;
__no_init uint32_t core2ParamSyncActivationRollbackInverseRetained;
#else
static uint32_t core2ParamSyncOwnerBootRetained;
static uint32_t core2ParamSyncOwnerBootInverseRetained;
static uint32_t core2ParamSyncActivationRollbackRetained;
static uint32_t core2ParamSyncActivationRollbackInverseRetained;
#endif

static uint8_t activationContext(uint8_t *oldSlot, uint8_t *newSlot)
{
    uint32_t word = core2ParamSyncActivationRollbackRetained;
    if (core2ParamSyncActivationRollbackInverseRetained != ~word ||
        (word & 0xFFFF0000u) != CORE2_ACTIVATE_ROLLBACK_MAGIC)
```

```

        return 0u;
        *oldSlot = (uint8_t) word;
        *newSlot = (uint8_t) (word >> 8);
        return (uint8_t) (*oldSlot >= 1u && *oldSlot <= 5u &&
            *newSlot >= 1u && *newSlot <= 5u);
    }

static void setActivationContext(uint8_t oldSlot, uint8_t newSlot)
{
    uint32_t word = CORE2_ACTIVATE_ROLLBACK_MAGIC |
        (uint32_t) oldSlot | ((uint32_t) newSlot << 8);
    core2ParamSyncActivationRollbackRetained = word;
    core2ParamSyncActivationRollbackInverseRetained = ~word;
}

static void clearActivationContext(void)
{
    core2ParamSyncActivationRollbackRetained = 0u;
    core2ParamSyncActivationRollbackInverseRetained = UINT32_MAX;
}

static uint8_t activatePreparedRuntimeSlot(uint8_t slot, uint32_t transactionId)
{
    chassisParamStruct params;
    return (uint8_t) (chassisParamsActivatePreparedSlot(slot, transactionId,
        &params) == CHASSIS_PARAMS_SLOT_OK &&
        core2RuntimeSubmitParams(&params) != 0u);
}

static uint8_t activateRuntimeSlotPending(uint8_t slot)
{
    chassisParamStruct params;
    return (uint8_t) (chassisParamsActivateSlotPending(slot, &params) ==
        CHASSIS_PARAMS_SLOT_OK &&
        core2RuntimeSubmitParams(&params) != 0u);
}

static uint8_t rollbackActivation(void)
{
    {
        uint8_t oldSlot, newSlot;
        chassisParamsPendingInfo pending;
        (void) activationContext(&oldSlot, &newSlot);
        {
            chassisParamStruct params;
            (void) oldSlot;
            (void) newSlot;
            if (chassisParamsGetPending(&pending) != CHASSIS_PARAMS_SLOT_OK)
                return 0u;
            if (pending.active != 0u && pending.activationOnly != 0u &&
                (chassisParamsReadRuntime(&params, NULL) != CHASSIS_PARAMS_SLOT_OK ||
                core2RuntimeSubmitParams(&params) == 0u))
                return 0u;
            if (chassisParamsAbortPendingActivation(&params) !=
                CHASSIS_PARAMS_SLOT_OK)
                return 0u;
        }
        clearActivationContext();
        return 1u;
    }
}

static uint8_t recoverActivationAfterOwnerBoot(void)
{
    {
        if (activationRecoveryChecked) return 1u;
        activationRecoveryChecked = 1u;
        return 1u;
    }
}

static void putU16(uint8_t *p, uint16_t v)
{
    {
        p[0] = (uint8_t) v;
        p[1] = (uint8_t) (v >> 8);
    }
}

static void putU32(uint8_t *p, uint32_t v)
{
    {
        p[0] = (uint8_t) v;
        p[1] = (uint8_t) (v >> 8);
        p[2] = (uint8_t) (v >> 16);
        p[3] = (uint8_t) (v >> 24);
    }
}

```

```

}

static uint16_t getU16(const uint8_t *p)
{
    return (uint16_t) (p[0] | ((uint16_t) p[1] << 8));
}

static uint32_t getU32(const uint8_t *p)
{
    return (uint32_t) p[0] | ((uint32_t) p[1] << 8) |
        ((uint32_t) p[2] << 16) | ((uint32_t) p[3] << 24);
}

uint32_t core2ParamSyncOwnerBootGeneration(void)
{
    uint32_t previous;
    if (ownerBootGeneration != 0u) return ownerBootGeneration;
    previous = core2ParamSyncOwnerBootRetained;
    if ((previous == 0u) ||
        (core2ParamSyncOwnerBootInverseRetained != ~previous)) previous = 0u;
    previous++;
    if (previous == 0u) previous = 1u;
    core2ParamSyncOwnerBootRetained = previous;
    core2ParamSyncOwnerBootInverseRetained = ~previous;
    ownerBootGeneration = previous;
    return ownerBootGeneration;
}

static void encodeBlob(const chassisParamStruct *params, core2ParamSyncBlob *blob)
{
    memset(blob, 0, sizeof(*blob));
    blob->magic = CORE2_FLASH_MAGIC;
    blob->version = CORE2_FLASH_VERSION;
    blob->payloadBytes = sizeof(*blob);
    blob->params = *params;
    chassisParamsSanitize(&blob->params);
    blob->crc = crc32IeeeCalc((const uint8_t *) blob,
        (uint32_t) offsetof(core2ParamSyncBlob, crc));
}

static void publish(const paramSyncMailboxRequest *request, uint8_t status,
    const uint8_t *data, uint16_t length, uint32_t crc)
{
    paramSyncMailboxResponse response;
    memset(&response, 0, sizeof(response));
    responseSequence++;
    if (responseSequence == 0u) responseSequence = 1u;
    response.producerBootGeneration = core2ParamSyncOwnerBootGeneration();
    response.sequence = responseSequence;
    response.transactionId = request->transactionId;
    response.offset = request->offset;
    response.payloadCrc32 = crc;
    response.dataLength = length;
    response.op = request->op;
    response.slot = request->slot;
    response.target = request->target;
    response.status = status;
    if (data != NULL && length != 0u) memcpy(response.data, data, length);
    (void) paramSyncMailboxPublishResponse(&response);
}

static void clearStaging(void)
{
    memset(&stagedBlob, 0, sizeof(stagedBlob));
    stagedTransaction = 0u;
    stagedExpectedCrc = 0u;
    stagedBytes = 0u;
    stagedActive = 0u;
    stagedPrepared = 0u;
    stagedCommitted = 0u;
    stagedReceived = 0u;
    stagedSlot = 0u;
    stagedKind = PARAM_STAGE_NONE;
    stagedApplied = 0u;
    stagedApplyRequestValid = 0u;
    stagedApplySubmitted = 0u;
    memset(&stagedApplyRequest, 0, sizeof(stagedApplyRequest));
}

```

```

static void clearMutationLock(void)
{
    syncMutationLocked = 0u;
    syncLockedSlot = 0u;
    syncLockedTransaction = 0u;
}

static void serviceStagedApply(void)
{
    chassisParamStruct active;
    if (stagedApplyRequestValid == 0u) return;
    if (stagedApplySubmitted == 0u)
    {
        if (core2RuntimeParamsApplyReady() == 0u) return;
        if (chassisParamsApplyPreparedRuntime(stagedTransaction, &active) !=
            CHASSIS_PARAMS_SLOT_OK ||
            core2RuntimeSubmitParams(&active) == 0u)
        {
            publish(&stagedApplyRequest, PARAM_SYNC_STATUS_FAILED,
                NULL, 0u, 0u);
            stagedApplyRequestValid = 0u;
            return;
        }
        stagedApplySubmitted = 1u;
    }
    if (core2RuntimeParamsApplied() == 0u) return;
    stagedApplied = 1u;
    stagedApplyRequestValid = 0u;
    publish(&stagedApplyRequest, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void finishDeferredAbort(void)
{
    chassisParamStruct params;
    paramSyncMailboxRequest request;
    uint32_t transactionId;
    if (stagedAbortRequestValid == 0u) return;
    if (stagedAbortSubmitted == 0u)
    {
        if (core2RuntimeParamsApplyReady() == 0u) return;
        if (chassisParamsReadRuntime(&params, NULL) != CHASSIS_PARAMS_SLOT_OK ||
            core2RuntimeSubmitParams(&params) == 0u)
        {
            request = stagedAbortRequest;
            stagedAbortRequestValid = 0u;
            publish(&request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
            return;
        }
        stagedAbortSubmitted = 1u;
    }
    if (core2RuntimeParamsApplied() == 0u) return;
    request = stagedAbortRequest;
    transactionId = getU32(request.data);
    if ((transactionId == 0u &&
        chassisParamsAbortPendingActivation(&params) !=
            CHASSIS_PARAMS_SLOT_OK) ||
        (transactionId != 0u &&
        chassisParamsAbortPreparedSlot(transactionId) !=
            CHASSIS_PARAMS_SLOT_OK))
    {
        stagedAbortRequestValid = 0u;
        publish(&request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    stagedAbortRequestValid = 0u;
    stagedAbortSubmitted = 0u;
    lastAbortedTransaction = transactionId;
    clearStaging();
    if (transactionId == 0u) clearActivationContext();
    clearMutationLock();
    persistentPendingLocked = 0u;
    publish(&request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void finishDeferredFinish(void)
{
    chassisParamStruct params;
    paramSyncMailboxRequest request;
    uint32_t transactionId;

```

```

if (stagedFinishRequestValid == 0u) return;
request = stagedFinishRequest;
transactionId = getU32(request.data);
if (stagedFinishSubmitted == 0u)
{
    if (core2RuntimeParamsApplyReady() == 0u) return;
    if (chassisParamsApplyPreparedRuntime(
        (transactionId == 0u) ? UINT32_MAX : transactionId,
        &params) != CHASSIS_PARAMS_SLOT_OK ||
        core2RuntimeSubmitParams(&params) == 0u)
    {
        stagedFinishRequestValid = 0u;
        publish(&request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    stagedFinishSubmitted = 1u;
}
if (core2RuntimeParamsApplied() == 0u) return;
if ((transactionId == 0u &&
    chassisParamsFinishPendingActivation() != CHASSIS_PARAMS_SLOT_OK) ||
    (transactionId != 0u &&
    chassisParamsFinishPreparedSlot(transactionId) !=
    CHASSIS_PARAMS_SLOT_OK))
{
    stagedFinishRequestValid = 0u;
    publish(&request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}
stagedFinishRequestValid = 0u;
stagedFinishSubmitted = 0u;
lastFinishedTransaction = transactionId;
clearStaging();
if (transactionId == 0u) clearActivationContext();
clearMutationLock();
persistentPendingLocked = 0u;
publish(&request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static uint8_t decodeGlobalTransaction(const paramSyncMailboxRequest *request,
    uint8_t allowDefaultSlot, uint8_t *slot, uint32_t *transactionId)
{
    if (request->dataLength != 5u) return 0u;
    *slot = request->data[0];
    *transactionId = getU32(&request->data[1]);
    return (uint8_t) ((*transactionId != 0u) &&
        (*transactionId != UINT32_MAX) &&
        ((*slot >= 1u && *slot <= 5u) ||
        (allowDefaultSlot != 0u && *slot == 0u)));
}

static uint8_t transactionLocked(uint8_t slot, uint32_t transactionId)
{
    return (uint8_t) (syncMutationLocked != 0u &&
        syncLockedSlot == slot && syncLockedTransaction == transactionId);
}

static uint8_t producerBootChangeAllowed(
    const paramSyncMailboxRequest *request)
{
    chassisParamsPendingInfo pending;
    uint8_t oldSlot, newSlot;
    uint8_t activationPending;
    uint8_t transactionPending;
    uint32_t transactionId;

    if (chassisParamsGetPending(&pending) != CHASSIS_PARAMS_SLOT_OK)
        return 0u;
    activationPending = (uint8_t) (activationContext(&oldSlot, &newSlot) ||
        (pending.active != 0u && pending.activationOnly != 0u));
    transactionPending = (uint8_t) (syncMutationLocked || stagedActive ||
        stagedPrepared || stagedCommitted ||
        (pending.active != 0u && pending.activationOnly == 0u));
    if ((activationPending == 0u) &&
        (pending.active == 0u) && (transactionPending == 0u)) return 1u;
    if ((request->op != PARAM_SYNC_OP_FINISH &&
        request->op != PARAM_SYNC_OP_ABORT) ||
        (request->dataLength != 4u)) return 0u;

    transactionId = getU32(request->data);

```

```

    if (activationPending != 0u)
    {
        if ((transactionId != 0u) || stagedActive || stagedPrepared ||
            stagedCommitted) return 0u;
        return 1u;
    }
    if ((pending.active != 0u) &&
        (transactionId != pending.transactionId)) return 0u;
    if ((syncMutationLocked != 0u) &&
        (transactionId != syncLockedTransaction)) return 0u;
    if ((stagedActive || stagedPrepared || stagedCommitted) &&
        (transactionId != stagedTransaction)) return 0u;
    return 1u;
}

uint8_t core2ParamSyncOwnerServiceMutationLocked(void)
{
    return (uint8_t) (syncMutationLocked || stagedActive || stagedPrepared ||
        stagedCommitted || persistentPendingLocked);
}

static void handleReadMeta(const paramSyncMailboxRequest *request)
{
    uint8_t out[26];
    chassisParamsLibraryMeta meta;
    chassisParamStruct params;
    core2ParamSyncBlob blob;
    uint32_t blobCrc = 0u;
    if (request->dataLength != 1u || request->data[0] < 1u || request->data[0] > 5u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    memset(&meta, 0, sizeof(meta));
    if (chassisParamsInspectSlot(request->data[0], &meta) !=
        CHASSIS_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    if (meta.state == CHASSIS_PARAMS_LIBRARY_VALID)
    {
        if (chassisParamsReadSlot(request->data[0], &params, NULL) !=
            CHASSIS_PARAMS_SLOT_OK)
        {
            publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
            return;
        }
        encodeBlob(&params, &blob);
        blobCrc = blob.crc;
    }
    memset(out, 0, sizeof(out));
    out[0] = CORE2_PARAM_SYNC_TARGET;
    out[1] = request->data[0];
    out[2] = meta.state;
    out[3] = 0u;
    putU32(&out[4], CORE2_FLASH_VERSION);
    putU32(&out[8], meta.generation);
    putU32(&out[12], blobCrc);
    putU32(&out[16], meta.transactionId);
    putU32(&out[20], CORE2_SCHEMA_HASH);
    putU16(&out[24], sizeof(core2ParamSyncBlob));
    publish(request, PARAM_SYNC_STATUS_OK, out, sizeof(out), 0u);
}

static void handleReadRuntimeMeta(const paramSyncMailboxRequest *request)
{
    uint8_t out[32];
    chassisParamStruct runtime;
    chassisParamsRuntimeMeta meta;
    core2ParamSyncBlob blob;

    if (request->dataLength != 0u ||
        chassisParamsReadRuntime(&runtime, &meta) != CHASSIS_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    encodeBlob(&runtime, &blob);

```

```

chassisParamsGetActive(&active);
memset(out, 0, sizeof(out));
out[0] = CORE2_PARAM_SYNC_TARGET;
out[1] = meta.valid;
out[2] = meta.bank;
out[3] = meta.pending;
out[4] = (uint8_t) (memcmp(&runtime, &active, sizeof(runtime)) != 0);
out[5] = meta.sourceSlot;
out[6] = meta.dagnosticFlags;
putU32(&out[8], CORE2_FLASH_VERSION);
putU32(&out[12], meta.generation);
putU32(&out[16], blob.crc);
putU32(&out[20], meta.transactionId);
putU32(&out[24], CORE2_SCHEMA_HASH);
putU16(&out[28], sizeof(blob));
publish(request, PARAM_SYNC_STATUS_OK, out, sizeof(out), 0u);
}

static void handleReadRuntimeChunk(const paramSyncMailboxRequest *request)
{
    chassisParamStruct params;
    core2ParamSyncBlob blob;
    uint8_t out[PARAM_SYNC_MAILBOX_DATA_MAX];
    uint16_t offset, length;

    if (request->dataLength != 4u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    offset = getU16(&request->data[0]);
    length = getU16(&request->data[2]);
    if (length == 0u || length > (PARAM_SYNC_MAILBOX_DATA_MAX - 4u) ||
        (uint32_t) offset + length > sizeof(blob) ||
        chassisParamsReadRuntime(&params, NULL) != CHASSIS_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    encodeBlob(&params, &blob);
    putU16(&out[0], offset);
    putU16(&out[2], sizeof(blob));
    memcpy(&out[4], ((const uint8_t *) &blob) + offset, length);
    publish(request, PARAM_SYNC_STATUS_OK, out, (uint16_t) (4u + length),
        blob.crc);
}

static void handleReadChunk(const paramSyncMailboxRequest *request)
{
    chassisParamStruct params;
    core2ParamSyncBlob blob;
    uint8_t out[PARAM_SYNC_MAILBOX_DATA_MAX];
    uint8_t slot;
    uint16_t offset, length;
    if (request->dataLength != 5u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    slot = request->data[0];
    offset = getU16(&request->data[1]);
    length = getU16(&request->data[3]);
    if (slot < 1u || slot > 5u || length == 0u ||
        length > (PARAM_SYNC_MAILBOX_DATA_MAX - 5u) ||
        ((uint32_t) offset + length > sizeof(blob)) ||
        chassisParamsReadSlot(slot, &params, NULL) != CHASSIS_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    encodeBlob(&params, &blob);
    out[0] = slot;
    putU16(&out[1], offset);
    putU16(&out[3], sizeof(blob));
    memcpy(&out[5], ((const uint8_t *) &blob) + offset, length);
    publish(request, PARAM_SYNC_STATUS_OK, out, (uint16_t) (5u + length), blob.crc);
}

static void handlePrepareLibraryFromLive(

```



```

    const paramSyncMailboxRequest *request)
{
    chassisParamStruct active;
    core2ParamSyncBlob blob;
    uint8_t slot;
    uint32_t transactionId;

    if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (stagedPrepared != 0u || stagedCommitted != 0u)
    {
        publish(request, (stagedKind == PARAM_STAGE_LIBRARY &&
            stagedSlot == slot && stagedTransaction == transactionId) ?
            PARAM_SYNC_STATUS_OK : PARAM_SYNC_STATUS_BUSY, NULL, 0u,
            stagedExpectedCrc);
        return;
    }
    chassisParamsGetActive(&active);
    encodeBlob(&active, &blob);
    if (chassisParamsPrepareSlot(slot, &active, transactionId) !=
        CHASSIS_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    clearStaging();
    stagedBlob = blob;
    stagedSlot = slot;
    stagedTransaction = transactionId;
    stagedExpectedCrc = blob.crc;
    stagedBytes = sizeof(blob);
    stagedReceived = sizeof(blob);
    stagedPrepared = 1u;
    stagedKind = PARAM_STAGE_LIBRARY;
    persistentPendingLocked = 1u;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, blob.crc);
}

static void handlePrepareRuntime(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;
    chassisParamsSlotStatus status;

    if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u ||
        slot == 0u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (stagedPrepared != 0u || stagedCommitted != 0u)
    {
        publish(request, (stagedKind == PARAM_STAGE_RUNTIME &&
            stagedSlot == slot && stagedTransaction == transactionId) ?
            PARAM_SYNC_STATUS_OK : PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
        return;
    }
    status = chassisParamsPrepareRuntimeFromSlot(slot, transactionId);
    if (status != CHASSIS_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    clearStaging();
    stagedSlot = slot;
    stagedTransaction = transactionId;
    stagedPrepared = 1u;
    stagedKind = PARAM_STAGE_RUNTIME;
    persistentPendingLocked = 1u;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleCommitRuntime(const paramSyncMailboxRequest *request)
{

```

```

uint8_t slot;
uint32_t transactionId;
if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||
    transactionLocked(slot, transactionId) == 0u ||
    stagedKind != PARAM_STAGE_RUNTIME || stagedPrepared == 0u ||
    stagedSlot != slot || stagedTransaction != transactionId)
{
    publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
    return;
}
if (stagedCommitted == 0u &&
    chassisParamsCommitRuntime(transactionId) != CHASSIS_PARAMS_SLOT_OK)
{
    publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}
stagedCommitted = 1u;
persistentPendingLocked = 1u;
publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleApplyRuntime(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;
    if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u ||
        stagedKind != PARAM_STAGE_RUNTIME || stagedCommitted == 0u ||
        stagedSlot != slot || stagedTransaction != transactionId)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (stagedAbortRequestValid != 0u || stagedFinishRequestValid != 0u)
    {
        publish(request, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
        return;
    }
    if (stagedApplied != 0u)
    {
        publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
        return;
    }
    stagedApplyRequest = *request;
    stagedApplyRequestValid = 1u;
}

static void handlePrepareBegin(const paramSyncMailboxRequest *request)
{
    uint32_t transactionId = request->dataLength == 19u ?
        getU32(&request->data[1]) : 0u;
    if (request->dataLength != 19u || request->data[0] < 1u ||
        request->data[0] > 5u || transactionId == 0u ||
        transactionId == UINT32_MAX ||
        getU32(&request->data[5]) != CORE2_FLASH_VERSION ||
        getU32(&request->data[9]) != CORE2_SCHEMA_HASH ||
        getU16(&request->data[17]) != sizeof(stagedBlob))
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (stagedPrepared || stagedCommitted)
    {
        if (chassisParamsAbortPreparedSlot(stagedTransaction) != CHASSIS_PARAMS_SLOT_OK)
        {
            publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
            return;
        }
    }
    clearStaging();
    memset(&stagedBlob, 0, sizeof(stagedBlob));
    stagedSlot = request->data[0];
    stagedTransaction = transactionId;
    stagedExpectedCrc = getU32(&request->data[13]);
    stagedBytes = getU16(&request->data[17]);
    stagedReceived = 0u;
    stagedPrepared = 0u;
    stagedCommitted = 0u;
}

```

```

    stagedActive = 1u;
    stagedKind = PARAM_STAGE_LIBRARY;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handlePrepareChunk(const paramSyncMailboxRequest *request)
{
    uint16_t offset, length;
    if (!stagedActive || request->dataLength < 5u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    offset = getU16(&request->data[1]);
    length = getU16(&request->data[3]);
    if (request->data[0] != stagedSlot || offset != stagedReceived ||
        length != (uint16_t) (request->dataLength - 5u) ||
        ((uint32_t) offset + length > stagedBytes))
    {
        clearStaging();
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    memcpy(((uint8_t *) &stagedBlob) + offset, &request->data[5], length);
    stagedReceived = (uint16_t) (stagedReceived + length);
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handlePrepareVerify(const paramSyncMailboxRequest *request)
{
    uint32_t crc;
    if (!stagedActive || request->dataLength != 1u || request->data[0] != stagedSlot ||
        stagedReceived != stagedBytes || stagedBlob.magic != CORE2_FLASH_MAGIC ||
        stagedBlob.version != CORE2_FLASH_VERSION || stagedBlob.payloadBytes != sizeof(stagedBlob))
    {
        clearStaging();
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    crc = crc32leecCalc((const uint8_t *) &stagedBlob,
        (uint32_t) offsetof(core2ParamSyncBlob, crc));
    if (crc != stagedBlob.crc || crc != stagedExpectedCrc ||
        chassisParamsPrepareSlot(stagedSlot, &stagedBlob.params, stagedTransaction) !=
            CHASSIS_PARAMS_SLOT_OK)
    {
        clearStaging();
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    stagedActive = 0u;
    stagedPrepared = 1u;
    persistentPendingLocked = 1u;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, crc);
}

static void handleCommit(const paramSyncMailboxRequest *request)
{
    uint32_t transactionId;
    if (request->dataLength != 5u || request->data[0] != stagedSlot ||
        (transactionId = getU32(&request->data[1])) != stagedTransaction ||
        stagedKind != PARAM_STAGE_LIBRARY || !stagedPrepared)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    if (!stagedCommitted && chassisParamsCommitPreparedSlot(stagedSlot,
        transactionId, 0u) != CHASSIS_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    stagedCommitted = 1u;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleActivate(const paramSyncMailboxRequest *request)
{
    uint32_t transactionId;
    uint8_t oldSlot, pendingSlot;

```

```

if (request->dataLength != 5u || request->data[0] < 1u || request->data[0] > 5u)
{
    publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
    return;
}
transactionId = getU32(&request->data[1]);
if (transactionId == 0u)
{
    if (!syncMutationLocked || syncLockedSlot != request->data[0] ||
        stagedActive || stagedPrepared || stagedCommitted)
    {
        publish(request, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
        return;
    }
    if (activationContext(&oldSlot, &pendingSlot))
    {
        publish(request, pendingSlot == request->data[0] ? PARAM_SYNC_STATUS_OK : PARAM_SYNC_STATUS_BUSY,
            NULL, 0u, 0u);
        return;
    }
    oldSlot = chassisParamsGetActiveSlot();
    if (oldSlot < 1u || oldSlot > 5u)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    setActivationContext(oldSlot, request->data[0]);
}
if ((transactionId != 0u && (!stagedCommitted || transactionId != stagedTransaction ||
    request->data[0] != stagedSlot)) ||
    !(transactionId == 0u ? activateRuntimeSlotPending(request->data[0]) :
        activatePreparedRuntimeSlot(request->data[0], transactionId)))
{
    if (transactionId == 0u) (void) rollbackActivation();
    publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}
publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleAbort(const paramSyncMailboxRequest *request)
{
    uint32_t transactionId;
    chassisParamsPendingInfo pending;
    uint8_t runtimePending;
    if (request->dataLength != 4u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    transactionId = getU32(request->data);
    if (stagedFinishRequestValid != 0u)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    if (stagedAbortRequestValid != 0u)
    {
        if (transactionId != getU32(stagedAbortRequest.data))
        {
            publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
            return;
        }
        stagedAbortRequest = *request;
        return;
    }
    if (transactionId == 0u)
    {
        if (chassisParamsGetPending(&pending) != CHASSIS_PARAMS_SLOT_OK)
        {
            publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
            return;
        }
    }
    if (pending.active == 0u || pending.activationOnly == 0u)
    {
        uint8_t ok = rollbackActivation();
        if (ok) clearMutationLock();
        if (ok) persistentPendingLocked = 0u;
        publish(request, ok ? PARAM_SYNC_STATUS_OK :

```

```

        PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}
stagedApplyRequestValid = 0u;
stagedApplySubmitted = 0u;
stagedAbortRequest = *request;
stagedAbortRequestValid = 1u;
stagedAbortSubmitted = 0u;
return;
}
if (!stagedActive && !stagedPrepared && !stagedCommitted &&
    transactionId == lastAbortedTransaction)
{
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
    return;
}
if ((stagedActive || stagedPrepared || stagedCommitted) &&
    transactionId != stagedTransaction)
{
    publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}
if (chassisParamsGetPending(&pending) !=
    CHASSIS_PARAMS_SLOT_OK)
{
    publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}
runtimePending = (uint8_t) (pending.active != 0u &&
    pending.activationOnly == 0u &&
    pending.transactionId == transactionId);
if (runtimePending != 0u)
{
    stagedApplyRequestValid = 0u;
    stagedApplySubmitted = 0u;
    stagedAbortRequest = *request;
    stagedAbortRequestValid = 1u;
    stagedAbortSubmitted = 0u;
    return;
}
if (chassisParamsAbortPreparedSlot(transactionId) != CHASSIS_PARAMS_SLOT_OK)
{
    publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}
lastAbortedTransaction = transactionId;
clearStaging();
clearMutationLock();
persistentPendingLocked = 0u;
publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleFinish(const paramSyncMailboxRequest *request)
{
    uint32_t transactionId;
    chassisParamsPendingInfo pending;
    uint8_t runtimePending;
    if (request->dataLength != 4u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    transactionId = getU32(request->data);
    if (stagedAbortRequestValid != 0u)
    {
        publish(request, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
        return;
    }
    if (stagedFinishRequestValid != 0u)
    {
        if (transactionId != getU32(stagedFinishRequest.data))
        {
            publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
            return;
        }
        stagedFinishRequest = *request;
        return;
    }
    if (transactionId == 0u)

```

```

{
    if (chassisParamsGetPending(&pending) !=
        CHASSIS_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    runtimePending = (uint8_t) (pending.active != 0u &&
        pending.activationOnly != 0u);
    if (runtimePending != 0u)
    {
        stagedFinishRequest = *request;
        stagedFinishRequestValid = 1u;
        stagedFinishSubmitted = 0u;
        return;
    }
    if (chassisParamsFinishPendingActivation() != CHASSIS_PARAMS_SLOT_OK)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    clearActivationContext();
    clearMutationLock();
    persistentPendingLocked = 0u;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
    return;
}

if (!stagedActive && !stagedPrepared && !stagedCommitted &&
    transactionId == lastFinishedTransaction)
{
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
    return;
}

if ((stagedActive || stagedPrepared || stagedCommitted) &&
    ((!stagedPrepared) || transactionId != stagedTransaction ||
    (stagedKind == PARAM_STAGE_LIBRARY && stagedCommitted == 0u) ||
    (stagedKind == PARAM_STAGE_RUNTIME &&
    (stagedCommitted == 0u || stagedApplied == 0u))))
{
    publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}

if (chassisParamsGetPending(&pending) !=
    CHASSIS_PARAMS_SLOT_OK)
{
    publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}

runtimePending = (uint8_t) (pending.active != 0u &&
    pending.activationOnly == 0u &&
    pending.transactionId == transactionId);
if (runtimePending != 0u &&
    !(stagedKind == PARAM_STAGE_RUNTIME && stagedApplied != 0u))
{
    stagedFinishRequest = *request;
    stagedFinishRequestValid = 1u;
    stagedFinishSubmitted = 0u;
    return;
}

if (chassisParamsFinishPreparedSlot(transactionId) != CHASSIS_PARAMS_SLOT_OK)
{
    publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}

lastFinishedTransaction = transactionId;
clearStaging();
clearMutationLock();
persistentPendingLocked = 0u;
publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleLock(const paramSyncMailboxRequest *request)
{
    uint8_t oldSlot, newSlot;
    uint32_t transactionId = (request->dataLength == 5u) ?
        getU32(&request->data[1]) : 0u;
    if (request->dataLength != 5u || request->data[0] > 5u ||
        transactionId == 0u || transactionId == UINT32_MAX ||
        (syncMutationLocked && (syncLockedSlot != request->data[0] ||

```

```

        syncLockedTransaction != transactionId)) ||
        stagedActive || stagedPrepared || stagedCommitted ||
        persistentPendingLocked || activationContext(&oldSlot, &newSlot))
    {
        publish(request, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
        return;
    }
    syncMutationLocked = 1u;
    syncLockedSlot = request->data[0];
    syncLockedTransaction = transactionId;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static uint8_t retainedRequestMayResume(
    const paramSyncMailboxRequest *request)
{
    if (request->target != CORE2_PARAM_SYNC_TARGET) return 0u;
    return (uint8_t) (request->op == PARAM_SYNC_OP_QUERY ||
        request->op == PARAM_SYNC_OP_READ_META ||
        request->op == PARAM_SYNC_OP_READ_CHUNK ||
        request->op == PARAM_SYNC_OP_READ_RUNTIME_META ||
        request->op == PARAM_SYNC_OP_READ_RUNTIME_CHUNK ||
        request->op == PARAM_SYNC_OP_RECOVERY ||
        request->op == PARAM_SYNC_OP_ABORT ||
        request->op == PARAM_SYNC_OP_FINISH);
}

void core2ParamSyncOwnerServiceInit(void)
{
    paramSyncMailboxRequest retainedRequest;
    chassisParamsPendingInfo pending;
    lastBootGeneration = 0u;
    lastRequestSequence = 0u;
    responseSequence = 0u;
    stagedAbortRequestValid = 0u;
    stagedAbortSubmitted = 0u;
    memset(&stagedAbortRequest, 0, sizeof(stagedAbortRequest));
    stagedFinishRequestValid = 0u;
    stagedFinishSubmitted = 0u;
    memset(&stagedFinishRequest, 0, sizeof(stagedFinishRequest));
    persistentPendingLocked = (uint8_t) (
        chassisParamsGetPending(&pending) != CHASSIS_PARAMS_SLOT_OK ||
        pending.valid != 0u);
    (void) core2ParamSyncOwnerBootGeneration();
    if (paramSyncMailboxTryReadRequest(&retainedRequest) != 0u &&
        retainedRequestMayResume(&retainedRequest) == 0u)
    {
        lastBootGeneration = retainedRequest.producerBootGeneration;
        lastRequestSequence = retainedRequest.sequence;
        if (retainedRequest.target == CORE2_PARAM_SYNC_TARGET)
            publish(&retainedRequest, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
    }
}

static void processRequest(const paramSyncMailboxRequest *input)
{
    paramSyncMailboxRequest request = *input;
    if (request.producerBootGeneration == lastBootGeneration &&
        request.sequence == lastRequestSequence) return;
    if (request.producerBootGeneration == lastBootGeneration &&
        (int32_t) (request.sequence - lastRequestSequence) <= 0) return;
    if (lastBootGeneration != 0u &&
        request.producerBootGeneration != lastBootGeneration)
    {
        if (producerBootChangeAllowed(&request) == 0u)
        {
            if (request.target == CORE2_PARAM_SYNC_TARGET)
                publish(&request, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
            return;
        }
    }
    lastBootGeneration = request.producerBootGeneration;
    lastRequestSequence = request.sequence;
    if (request.target != CORE2_PARAM_SYNC_TARGET) return;
    if (!recoverActivationAfterOwnerBoot())
    {
        publish(&request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
}

```

```

switch (request.op)
{
    case PARAM_SYNC_OP_READ_META: handleReadMeta(&request); break;
    case PARAM_SYNC_OP_READ_CHUNK: handleReadChunk(&request); break;
    case PARAM_SYNC_OP_READ_RUNTIME_META:
        handleReadRuntimeMeta(&request); break;
    case PARAM_SYNC_OP_READ_RUNTIME_CHUNK:
        handleReadRuntimeChunk(&request); break;
    case PARAM_SYNC_OP_PREPARE_LIBRARY_FROM_LIVE:
        handlePrepareLibraryFromLive(&request); break;
    case PARAM_SYNC_OP_PREPARE_RUNTIME_FROM_SLOT:
        handlePrepareRuntime(&request); break;
    case PARAM_SYNC_OP_COMMIT_RUNTIME:
        handleCommitRuntime(&request); break;
    case PARAM_SYNC_OP_APPLY_RUNTIME:
        handleApplyRuntime(&request); break;
    case PARAM_SYNC_OP_PREPARE_BEGIN: handlePrepareBegin(&request); break;
    case PARAM_SYNC_OP_PREPARE_CHUNK: handlePrepareChunk(&request); break;
    case PARAM_SYNC_OP_PREPARE_VERIFY: handlePrepareVerify(&request); break;
    case PARAM_SYNC_OP_COMMIT: handleCommit(&request); break;
    case PARAM_SYNC_OP_ACTIVATE: handleActivate(&request); break;
    case PARAM_SYNC_OP_ABORT: handleAbort(&request); break;
    case PARAM_SYNC_OP_FINISH: handleFinish(&request); break;
    case PARAM_SYNC_OP_LOCK: handleLock(&request); break;
    default: publish(&request, PARAM_SYNC_STATUS_REJECTED, NULL, 0u, 0u); break;
}
}

void core2ParamSyncOwnerServicePoll(void)
{
    paramSyncMailboxRequest request;
    if (paramSyncMailboxTryReadRequest(&request) != 0u)
        processRequest(&request);
    finishDeferredAbort();
    finishDeferredFinish();
    serviceStagedApply();
}

```

core2_cyt4bb7/project/code/app/tuneMode.c

SHA-256

:

63912f88e6d300146c22879176f48bc885cead6caed819ec99af3180997bd4a2

代码 (c)

```

#include "tuneMode.h"

#include "chassisApplication.h"
#include "configCore2Build.h"
#include "menuKey.h"
#include "motorControl.h"
#include "loraRemote.h"
#include "zf_device_ips114.h"
#include "zf_common_font.h"

/* 调参选中轮 (0=FL 1=FR 2=BL 3=BR) : tuneWheelKeyScan 改写, 显示/调试/目标设置消费。
 * volatile: whileProcess(前台,经 tuneWheelKeyScan)写、 pitProcess0(ISR,经 updateTuneTarget)读, 单字节原子。 */
static volatile uint8_t motorTuneSelect = MOTOR_TUNE_WHEEL_SELECT;

uint8_t tuneModeGetSelectedWheel(void)
{
    return (uint8_t)(motorTuneSelect & 0x03u);
}

#if (CORE2_BUILD_MODE == CORE2_BUILD_TUNE) \
    && ((MOTOR_TUNE_MODE == MOTOR_TUNE_CASCADE) || (MOTOR_TUNE_MODE == MOTOR_TUNE_CURRENT)) \
    || (MOTOR_TUNE_MODE == MOTOR_TUNE_OPEN_LOOP))
static const char* motorName[4] = {"FL", "FR", "BL", "BR"};
static const motorControlWheelSnapshotStruct* getSelectedWheel(const motorControlSnapshotStruct* snapshot);
#endif

void tuneDisplayInit(void)
{
    #if (CORE2_BUILD_MODE == CORE2_BUILD_TUNE)
        ips114_set_dir(CORE2_IPS114_DISPLAY_DIR);
    #endif
}

```



```

#endif
#if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
    /* PRODUCTION 显示由 chassisMenu 单一所有者接管。 */
    (void) 0;
#elif (MOTOR_TUNE_MODE == MOTOR_TUNE_CASCADE)
    ips114_init();
    ips114_set_font(IPS114_6X8_FONT);
    ips114_set_color(RGB565_WHITE, RGB565_BLACK);
    ips114_clear();
    ips114_show_string(0, 0u * IPS114_LINE_HEIGHT, "Motor Tune   ");
    ips114_show_string(0, 1u * IPS114_LINE_HEIGHT, "Sel:           ");
    ips114_show_string(0, 2u * IPS114_LINE_HEIGHT, "TgtSpd:        ");
    ips114_show_string(0, 3u * IPS114_LINE_HEIGHT, "FbkSpd:        ");
    ips114_show_string(0, 4u * IPS114_LINE_HEIGHT, "SpdOut:        ");
    ips114_show_string(0, 5u * IPS114_LINE_HEIGHT, "FbkCur:        ");
    ips114_show_string(0, 6u * IPS114_LINE_HEIGHT, "CurOut:       ");
#elif (MOTOR_TUNE_MODE == MOTOR_TUNE_CURRENT)
    ips114_init();
    ips114_set_font(IPS114_6X8_FONT);
    ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
    ips114_clear();
    ips114_show_string(0, 0u * IPS114_LINE_HEIGHT, "Motor Tune CUR ");
    ips114_show_string(0, 1u * IPS114_LINE_HEIGHT, "Sel:           ");
    ips114_show_string(0, 2u * IPS114_LINE_HEIGHT, "TgtCur:        ");
    ips114_show_string(0, 3u * IPS114_LINE_HEIGHT, "FbkCur:        ");
    ips114_show_string(0, 4u * IPS114_LINE_HEIGHT, "CurOut:       ");
    ips114_show_string(0, 5u * IPS114_LINE_HEIGHT, "FbkSpd:        ");
#elif (MOTOR_TUNE_MODE == MOTOR_TUNE_OPEN_LOOP)
    ips114_init();
    ips114_set_font(IPS114_6X8_FONT);
    ips114_set_color(RGB565_RED, RGB565_BLACK);
    ips114_clear();
    ips114_show_string(0, 0u * IPS114_LINE_HEIGHT, "Motor Tune OPN ");
    ips114_show_string(0, 1u * IPS114_LINE_HEIGHT, "Sel:           ");
    ips114_show_string(0, 2u * IPS114_LINE_HEIGHT, "Duty:          ");
    ips114_show_string(0, 3u * IPS114_LINE_HEIGHT, "MotOut:        ");
    ips114_show_string(0, 4u * IPS114_LINE_HEIGHT, "FbkSpd:        ");
    ips114_show_string(0, 5u * IPS114_LINE_HEIGHT, "FbkCur:        ");
#elif (MOTOR_TUNE_MODE == MOTOR_TUNE_LORA_TEST)
    ips114_init();
    ips114_set_font(IPS114_6X8_FONT);
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
    ips114_clear();
    ips114_show_string(0, 0u * IPS114_LINE_HEIGHT, "LoRa Test     ");
    ips114_show_string(0, 1u * IPS114_LINE_HEIGHT, "LX/LY:        ");
    ips114_show_string(0, 2u * IPS114_LINE_HEIGHT, "RX/RX:        ");
    ips114_show_string(0, 3u * IPS114_LINE_HEIGHT, "Key0..3:      ");
    ips114_show_string(0, 4u * IPS114_LINE_HEIGHT, "Sw0..3:       ");
    ips114_show_string(0, 5u * IPS114_LINE_HEIGHT, "State:        ");
    ips114_show_string(0, 6u * IPS114_LINE_HEIGHT, "Frames:       ");
#endif
}

void tuneDisplayUpdate(void)
{
    #if (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
        (void) 0;
    #elif (MOTOR_TUNE_MODE == MOTOR_TUNE_CASCADE)
        motorControlSnapshotStruct snapshot;
        const motorControlWheelSnapshotStruct* wheel;
        uint8_t selectedMotor;

        motorControlGetSnapshot(&snapshot);
        selectedMotor = motorTuneSelect & 0x03u;
        wheel = getSelectedWheel(&snapshot);

        ips114_set_color(RGB565_WHITE, RGB565_BLACK);
        ips114_show_string(48, 1u * IPS114_LINE_HEIGHT, motorName[selectedMotor]);
        ips114_show_float(48, 2u * IPS114_LINE_HEIGHT, (double) wheel->targetSpeed, 4, 1);
        ips114_show_float(48, 3u * IPS114_LINE_HEIGHT, (double) wheel->feedbackSpeed, 4, 1);
        ips114_show_float(48, 4u * IPS114_LINE_HEIGHT, (double) wheel->speedControl, 4, 1);
        ips114_show_float(48, 5u * IPS114_LINE_HEIGHT, (double) wheel->feedbackCurrent, 4, 1);
        ips114_show_float(48, 6u * IPS114_LINE_HEIGHT, (double) wheel->currentControl, 4, 2);
    #elif (MOTOR_TUNE_MODE == MOTOR_TUNE_CURRENT)
        motorControlSnapshotStruct snapshot;
        const motorControlWheelSnapshotStruct* wheel;
        uint8_t selectedMotor;

        motorControlGetSnapshot(&snapshot);

```

```

selectedMotor = motorTuneSelect & 0x03u;
wheel = getSelectedWheel(&snapshot);

ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
ips114_show_string(48, 1u * IPS114_LINE_HEIGHT, motorName[selectedMotor]);
ips114_show_float(48, 2u * IPS114_LINE_HEIGHT, (double) wheel->targetCurrent, 4, 1);
ips114_show_float(48, 3u * IPS114_LINE_HEIGHT, (double) wheel->feedbackCurrent, 4, 1);
ips114_show_float(48, 4u * IPS114_LINE_HEIGHT, (double) wheel->currentControl, 4, 2);
ips114_show_float(48, 5u * IPS114_LINE_HEIGHT, (double) wheel->feedbackSpeed, 4, 1);
#elif (MOTOR_TUNE_MODE == MOTOR_TUNE_OPEN_LOOP)
motorControlSnapshotStruct snapshot;
const motorControlWheelSnapshotStruct* wheel;
uint8_t selectedMotor;

motorControlGetSnapshot(&snapshot);
selectedMotor = motorTuneSelect & 0x03u;
wheel = getSelectedWheel(&snapshot);

ips114_set_color(RGB565_RED, RGB565_BLACK);
ips114_show_string(48, 1u * IPS114_LINE_HEIGHT, motorName[selectedMotor]);
ips114_show_float(48, 2u * IPS114_LINE_HEIGHT, (double) MOTOR_TUNE_OPEN_LOOP_DUTY, 1, 3);
ips114_show_float(48, 3u * IPS114_LINE_HEIGHT, (double) wheel->motorOutput, 1, 3);
ips114_show_float(48, 4u * IPS114_LINE_HEIGHT, (double) wheel->feedbackSpeed, 4, 1);
ips114_show_float(48, 5u * IPS114_LINE_HEIGHT, (double) wheel->feedbackCurrent, 4, 1);
#elif (MOTOR_TUNE_MODE == MOTOR_TUNE_LORA_TEST)
static uint32_t displayedFrameCount = 0u;
static uint32_t lastFrameSeq = 0u;
loraRemoteSnapshotStruct snapshot;

loraRemoteGetSnapshot(&snapshot);
if (snapshot.frameSeq != lastFrameSeq)
{
    displayedFrameCount++;
    lastFrameSeq = snapshot.frameSeq;
}

ips114_set_color(RGB565_GREEN, RGB565_BLACK);
ips114_show_int(48, 1u * IPS114_LINE_HEIGHT, (int32_t) snapshot.joystick[0], 6);
ips114_show_int(96, 1u * IPS114_LINE_HEIGHT, (int32_t) snapshot.joystick[1], 6);
ips114_show_int(48, 2u * IPS114_LINE_HEIGHT, (int32_t) snapshot.joystick[2], 6);
ips114_show_int(96, 2u * IPS114_LINE_HEIGHT, (int32_t) snapshot.joystick[3], 6);
ips114_show_uint(48, 3u * IPS114_LINE_HEIGHT, (uint32_t) snapshot.key[0], 1);
ips114_show_uint(60, 3u * IPS114_LINE_HEIGHT, (uint32_t) snapshot.key[1], 1);
ips114_show_uint(72, 3u * IPS114_LINE_HEIGHT, (uint32_t) snapshot.key[2], 1);
ips114_show_uint(84, 3u * IPS114_LINE_HEIGHT, (uint32_t) snapshot.key[3], 1);
ips114_show_uint(48, 4u * IPS114_LINE_HEIGHT, (uint32_t) snapshot.switchKey[0], 1);
ips114_show_uint(60, 4u * IPS114_LINE_HEIGHT, (uint32_t) snapshot.switchKey[1], 1);
ips114_show_uint(72, 4u * IPS114_LINE_HEIGHT, (uint32_t) snapshot.switchKey[2], 1);
ips114_show_uint(84, 4u * IPS114_LINE_HEIGHT, (uint32_t) snapshot.switchKey[3], 1);
ips114_show_uint(48, 5u * IPS114_LINE_HEIGHT, (uint32_t) snapshot.linkAlive, 1);
ips114_show_uint(48, 6u * IPS114_LINE_HEIGHT, displayedFrameCount, 6);
#endif
}

#if (CORE2_BUILD_MODE == CORE2_BUILD_TUNE) \
    && ((MOTOR_TUNE_MODE == MOTOR_TUNE_CASCADE) || (MOTOR_TUNE_MODE == MOTOR_TUNE_CURRENT) \
        || (MOTOR_TUNE_MODE == MOTOR_TUNE_OPEN_LOOP))

#if (MOTOR_TUNE_MODE == MOTOR_TUNE_CASCADE) || (MOTOR_TUNE_MODE == MOTOR_TUNE_CURRENT)
/* 目标阶跃测试值：每 MOTOR_TUNE_STEP_INTERVAL_S 秒在 HIGH/LOW 间切换；间隔=0 恒 HIGH。
 * 每 2ms 调一次(updateTuneTarget @ pitProcess0 ISR)，周期拍数 = INTERVAL_S×500（编译期常量折叠，
 * ISR 内无运行时浮点）。static tickCount/high 仅本 ISR 上下文访问，无需 volatile。*/
static float tuneStepTargetValue(void)
{
    static uint32_t tickCount = 0u;
    static uint8_t high = 1u;
    uint32_t periodTicks = (uint32_t) (MOTOR_TUNE_STEP_INTERVAL_S * 500.0f + 0.5f);

    if (periodTicks > 0u) /* 间隔=0(periodTicks=0) → 不跳变恒 high；>0 周期切换 */
    {
        tickCount++;
        if (tickCount >= periodTicks)
        {
            tickCount = 0u;
            high = high ? 0u : 1u;
        }
    }
    return high ? MOTOR_TUNE_STEP_HIGH : MOTOR_TUNE_STEP_LOW;
}

```

```

#endif

void updateTuneTarget(void)
{
    wheelSpeedStruct target = {0.0f, 0.0f, 0.0f, 0.0f};
    float value;

    #if (MOTOR_TUNE_MODE == MOTOR_TUNE_CASCADE) || (MOTOR_TUNE_MODE == MOTOR_TUNE_CURRENT)
        value = tuneStepTargetValue(); /* 阶跃测试：HIGH/LOW 周期切换或恒 HIGH (INTERVAL_S=0) */
    #else
        value = MOTOR_TUNE_OPEN_LOOP_DUTY;
    #endif

    switch (motorTuneSelect & 0x03u)
    {
        case 0u:
            target.fl = value;
            break;
        case 1u:
            target.fr = value;
            break;
        case 2u:
            target.bl = value;
            break;
        default:
            target.br = value;
            break;
    }

    #if (MOTOR_TUNE_MODE == MOTOR_TUNE_CASCADE)
        chassisApplicationApplyWheelTarget(target);
    #elif (MOTOR_TUNE_MODE == MOTOR_TUNE_CURRENT)
        motorControlSetWheelTargetCurrent(target);
    #else
        motorControlSetOpenLoopOutput(target);
    #endif
}

static const motorControlWheelSnapshotStruct* getSelectedWheel(const motorControlSnapshotStruct *snapshot)
{
    switch (motorTuneSelect & 0x03u)
    {
        case 0u:
            return &snapshot->fl;
        case 1u:
            return &snapshot->fr;
        case 2u:
            return &snapshot->bl;
        default:
            return &snapshot->br;
    }
}

#endif

#if (CORE2_BUILD_MODE == CORE2_BUILD_TUNE) \
    && ((MOTOR_TUNE_MODE == MOTOR_TUNE_CASCADE) || (MOTOR_TUNE_MODE == MOTOR_TUNE_CURRENT) \
        || (MOTOR_TUNE_MODE == MOTOR_TUNE_OPEN_LOOP))
/* 消费一次按键事件（短按/长按均算一次，不连发）：读到非 RELEASE 即 Clear 返回 1。*/
static uint8_t tuneKeyConsumed(menuKeyIndexEnum k)
{
    if (menuKeyGetState(k) != MENU_KEY_RELEASE)
    {
        menuKeyClear(k);
        return 1u;
    }
    return 0u;
}

/* TUNE 调参选轮：KEY_2/3/4/5 直选 FL/FR/BL/BR，KEY_1 复位为头文件默认轮。
 * 命中后立即 updateTuneTarget()——新轮目标生效、其余轮归零（防多轮同转）。前台 whileProcess 调。*/
void tuneWheelKeyScan(void)
{
    uint8_t changed = 0u;

    if (tuneKeyConsumed(MENU_KEY_1)) { motorTuneSelect = MOTOR_TUNE_WHEEL_SELECT; changed = 1u; }
    if (tuneKeyConsumed(MENU_KEY_2)) { motorTuneSelect = 0u; changed = 1u; } /* FL */
    if (tuneKeyConsumed(MENU_KEY_3)) { motorTuneSelect = 1u; changed = 1u; } /* FR */
    if (tuneKeyConsumed(MENU_KEY_4)) { motorTuneSelect = 2u; changed = 1u; } /* BL */
    if (tuneKeyConsumed(MENU_KEY_5)) { motorTuneSelect = 3u; changed = 1u; } /* BR */
}

```

```
        if (changed)
        {
            updateTuneTarget(); /* 新轮目标生效 + 旧轮归零（与 ISR 内调用幂等，单字节 select 切换原子） */
        }
    }
#endif
```

core2_cyt4bb7/project/code/app/tuneMode.h

SHA-256 :

8740f4d91dc576e69f42a3b1e9aca57c3c54a932843a6096993eff8ba400b069

代码 (c)

```
#ifndef _TUNE_MODE_H_
#define _TUNE_MODE_H_

#include "configCore2Build.h" /* 守卫用 CORE2_BUILD_MODE / MOTOR_TUNE_MODE 宏 */

/*
 * core2 调参/显示分发层（从 core0Loop.c 抽出，纯结构迁移，行为不变）。
 *
 * 承接"显示 / 调试 / 选轮 / 调参目标"的模式分发实现：
 * - PRODUCTION 大类：本模块显示/调试入口为 no-op，IPS114 由 chassisMenu 单一所有者接管。
 * - TUNE 大类各子模式（CASCADE/CURRENT/OPEN_LOOP/LORA_TEST）：IPS114 本地显示
 *   + KEY_2.5 选轮（CASCADE/CURRENT/OPEN_LOOP）。
 * 由 core0Loop.c 的 processInit/pitProcess0/whileProcess 在对应时机调用。
 *
 * 注：函数沿用原名（最小改动、行为保持优先）；内部 getSelectedWheel/tuneKeyConsumed 为本 TU static。
 */

void tuneDisplayInit(void); /* TUNE processInit 调用一次：按调参模式初始化 IPS114 */
void tuneDisplayUpdate(void); /* TUNE whileProcess 100ms：刷新选中轮数据 */
uint8_t tuneModeGetSelectedWheel(void); /* 0=FL 1=FR 2=BL 3=BR；隐藏前台写/ISR 读的私有状态 */

#if (MOTOR_TUNE_MODE == MOTOR_TUNE_CASCADE) || (MOTOR_TUNE_MODE == MOTOR_TUNE_CURRENT) \
    || (MOTOR_TUNE_MODE == MOTOR_TUNE_OPEN_LOOP)
void updateTuneTarget(void); /* 按私有选择状态设选中轮目标，其余轮归零 */
#endif
#if (CORE2_BUILD_MODE == CORE2_BUILD_TUNE) \
    && ((MOTOR_TUNE_MODE == MOTOR_TUNE_CASCADE) || (MOTOR_TUNE_MODE == MOTOR_TUNE_CURRENT) \
        || (MOTOR_TUNE_MODE == MOTOR_TUNE_OPEN_LOOP))
void tuneWheelKeyScan(void); /* whileProcess：KEY_2.5 选 FL/FR/BL/BR，KEY_1 复位默认轮 */
#endif

#endif /* _TUNE_MODE_H_ */
```

core2_cyt4bb7/project/code/comm/chassisLink.c

SHA-256 :

0d111f9c700c2d00f8653599882c9752fb5794426c498474f2969229bdcdb31a8

代码 (c)

```
#include "chassisLink.h"

#include "configChassisRemote.h" /* LORA_LINK_TIMEOUT_MS */
#include "configCore2Build.h" /* CORE2_BUILD_MODE */

/*
 * core2 ↔ core1 RS-485 链路层。双实现按「核 + 固件」选择（IAR 预定义 CY_CORE_CM7_0/CM7_1）：
 *
 * [cm_7_0 控制核 + 生产固件] → 镜像实现：不拥有 UART4。RS-485 收发已搬到 cm_7_1 通信核，
 * 本核仅通过 coreDataTransfer 共享内存读 cm_7_1 发布的命令快照。getter API 不变，
 * 由 whileProcess 调 chassisLinkMirrorApply 刷新本地镜像，500μs ISR 临界区读，防撕裂。
 *
 * [cm_7_1 通信核（任何固件）/ cm_7_0 调参固件] → 真实 UART4 实现（THVD1450 帧层）：
 * cm_7_1 中断驱动 RX（ISR 从逐飞暂存搬入 THVD 本地 ring）+ 前台 thvd1450Poll 排空解析；
 * cm_7_0 调参固件沿用原行为（comm 仍在 cm_7_0，调参不依赖联机）。
 */
```

```

*/

#ifndef(CY_CORE_CM7_0) && (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)

/* ===== cm_7_0 生产态：共享内存镜像实现 ===== */
#include "coreDataTransfer.h"
#include "zf_common_interrupt.h"

/* volatile：whileProcess 临界区写、500μs ISR 临界区读；volatile 保证逐字段真实内存访问。
*
* 临界区用 __get_PRIMASK/_disable_irq/_set_PRIMASK（PRIMASK 存-恢复），**绝不用 SDK 的
* interrupt_global_disable/enable**：后者是有 bug 的全局计数器(interrupt_global_flag)而非 PRIMASK 存恢复，
* disable 返回计数器、enable 仅当传 0 才开中断。500μs ISR 高频在此读 linkMirror(GetCommand) 时，若 ISR
* 恰在前台 enable() 的 __enable_irq() 与 flag++ 之间抢入并自做一次 disable/enable，计数器会上漂；漂到 2
* 后下一次 disable/enable 不再开中断 → 中断永久关闭 → PIT ISR 不再触发 → 整机卡死(FOLLOW_485 每拍调
* GetCommand 必现，core0Loop.c 旧注释已实测记录)。PRIMASK 存恢复无此计数器、可重入、ISR 安全。*/
static volatile chassisLinkCommandStruct linkMirror; /* whileProcess 刷新、500μs ISR 临界区读 */
static volatile uint8_t linkAliveMirror = 0u; /* cm_7_1 发布的链路存活，ISR 直读（单字节原子） */

static void chassisLinkMirrorClearUnsafe(void)
{
    linkMirror.vx = 0.0f;
    linkMirror.vy = 0.0f;
    linkMirror.wz = 0.0f;
    linkMirror.cmd = 0u;
    linkMirror.sysByte = 0u;
    linkMirror.fresh = 0u;
    linkMirror.ageMs = LORA_LINK_TIMEOUT_MS;
    linkAliveMirror = 0u;
}

void chassisLinkInit(void)
{
    chassisLinkMirrorClearUnsafe();
}

void chassisLinkMirrorInvalidate(void)
{
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    chassisLinkMirrorClearUnsafe();
    __set_PRIMASK(primask);
}

/* whileProcess 调：把 cm_7_1 共享内存命令快照整块应用到本地镜像（短临界区，防 ISR 读撕裂）。
* 仅在 magic 有效时由 core0Loop 调用。*/
void chassisLinkMirrorApply(const commToControlStruct* s)
{
    uint32_t primask;
    if (0 == s) return;
    primask = __get_PRIMASK();
    __disable_irq();
    linkMirror.vx = s->cmdVx;
    linkMirror.vy = s->cmdVy;
    linkMirror.wz = s->cmdWz;
    linkMirror.cmd = s->cmd;
    linkMirror.sysByte = s->sysByte;
    linkMirror.ageMs = s->linkAlive ? 0u : LORA_LINK_TIMEOUT_MS;
    linkAliveMirror = s->linkAlive ? 1u : 0u;
    __set_PRIMASK(primask);
}

void chassisLinkGetCommand(chassisLinkCommandStruct* out)
{
    uint32_t primask;
    if (0 == out) return;
    primask = __get_PRIMASK();
    __disable_irq();
    *out = linkMirror;
    __set_PRIMASK(primask);
}

uint8_t chassisLinkIsAlive(void)
{
    return linkAliveMirror;
}

uint8_t chassisLinkGetCore0Alive(void)
{

```

```

    return (linkAliveMirror && (linkMirror.cmd & CHASSIS_LINK_CMD_CORE0_ALIVE)) ? 1u : 0u;
}

/* cm_7_0 生产态不拥有 UART4：以下为 API 完整性的空实现（不应被本核调用）。 */
void chassisLinkTickMs(void)
{
}
void chassisLinkPoll(void)
{
}
uint8_t chassisLinkCommandReceived(void)
{
    return 0u;
}
void chassisLinkSendTelemetry(float vx, float vy, float yawRate, uint8_t status)
{
    (void) vx;
    (void) vy;
    (void) yawRate;
    (void) status;
}
void chassisLinkSendLoraMotion(float vx, float vy, float wz,
    uint8_t remoteTakeoffReq, uint8_t attitudeFault)
{
    (void) vx;
    (void) vy;
    (void) wz;
    (void) remoteTakeoffReq;
    (void) attitudeFault;
}

#else /* defined(CY_CORE_CM7_1) || (CORE2_BUILD_MODE == CORE2_BUILD_TUNE) */

/* ===== 真实 UART4 实现（THVD1450 帧层） ===== */

#include <math.h>
#include <string.h>

#include "THVD1450.h"
#include "THVD1450_protocol.h"
#include "chassisLinkInternal.h"
#include "chassisLinkWireInternal.h"
#include "chassisParamProtocol.h"
#include "paramSyncMailbox.h"
#include "zf_common_interrupt.h"

/*
 * 遥测：手填 payload[0]=类型 + 3×float(小端) + status，经 thvd1450SendBytes 发送（半双工 DE 自管理）。
 * 遥控：thvd1450ReadFrame 取帧，校验 payload[0]==MSG_COMMAND，解出 vx/vy/wz + cmd。
 * float↔bytes 用 memcpy（不依赖对齐/不违反严格别名）。
 *
 * 命令快照发布：前台 Poll 解析到本地 pending，再在单个关中断临界区内整块发布到 linkCommand，
 * 读方 GetCommand 同样在临界区内整块快照——消除前台逐字段写 vs 读方逐字段读的撕裂
 * （否则可能读到"新 vx + 旧 wz"的跨帧混合命令）。
 */

#define CHASSIS_LINK_CMD_TIMEOUT_MS (LORA_LINK_TIMEOUT_MS) /* 遥控超时判链路丢失，复用 LoRa 超时常量 */
#define CHASSIS_LINK_CMD_SANITY_LIMIT (100.0f) /* 协议层异常值保护；真实限速在 chassisApplication 层 */
#define CHASSIS_LINK_PARAM_GLOBAL_LOST_MS (300u)
#define CHASSIS_LINK_PARAM_TRANSPORT_TIMEOUT_MS (600u)

static chassisLinkCommandStruct linkCommand; /* 命令快照（前台整块发布，读方整块读，临界区保护） */
static volatile uint32_t linkCommandAgeMs = 0u; /* 距上次有效遥控帧的 ms（TickMs 累加，收到帧清零） */
static uint32_t paramRequestSequence;
static uint32_t paramBridgeBootGeneration;
static paramSyncMailboxRequest paramAssembly;
static paramSyncMailboxResponse paramCachedResponse;
static uint32_t paramWireRequest;
static uint32_t paramOwnerBootGeneration;
static uint32_t paramOwnerResponseSequence;
static uint16_t paramExpectedBytes;
static uint16_t paramReceivedBytes;
static uint8_t paramAssemblyActive;
static uint8_t paramCachedResponseValid;
static volatile uint32_t paramQuietMs;
static uint8_t paramGlobalStateBytes[CHASSIS_PARAM_GLOBAL_STATE_BYTES];
static uint8_t paramGlobalRequestBytes[CHASSIS_PARAM_GLOBAL_REQUEST_BYTES];
static paramSyncGlobalState paramGlobalState;
static paramSyncGlobalRequest paramGlobalRequest;

```

```

static uint32_t paramGlobalStateAgeMs;
static uint8_t paramGlobalStateStreamSequence, paramGlobalStateMask;
static uint8_t paramGlobalRequestStreamSequence, paramGlobalRequestFragmentIndex;
static uint8_t paramGlobalStateValid, paramGlobalRequestValid;
#if defined(__ICCARM__)
__no_init uint32_t core2ParamBridgeBootRetained;
__no_init uint32_t core2ParamBridgeBootInverseRetained;
#else
static uint32_t core2ParamBridgeBootRetained;
static uint32_t core2ParamBridgeBootInverseRetained;
#endif

static void chassisLinkResetParamTransport(void)
{
    memset(&paramAssembly, 0, sizeof(paramAssembly));
    memset(&paramCachedResponse, 0, sizeof(paramCachedResponse));
    paramWireRequest = 0u;
    paramExpectedBytes = 0u;
    paramReceivedBytes = 0u;
    paramAssemblyActive = 0u;
    paramCachedResponseValid = 0u;
    paramQuietMs = CHASSIS_LINK_PARAM_TRANSPORT_TIMEOUT_MS;
}

static uint8_t chassisLinkSequenceAdvanced(uint32_t candidate,
uint32_t reference)
{
    uint32_t delta = candidate - reference;
    return (uint8_t) ((delta != 0u) && (delta < 0x80 000 000u));
}

static void chassisLinkRefreshOwnerResponseBaseline(void)
{
    paramSyncMailboxResponse response;
    if (paramSyncMailboxTryReadResponse(&response) == 0u) return;
    paramOwnerBootGeneration = response.producerBootGeneration;
    paramOwnerResponseSequence = response.sequence;
}

static void chassisLinkSendParamGlobalRequestFragment(void)
{
    chassisParamGlobalFragment fragment;
    uint8_t payload[CHASSIS_PARAM_GLOBAL_FRAME_BYTES];
    uint8_t count = (uint8_t) ((CHASSIS_PARAM_GLOBAL_REQUEST_BYTES +
        CHASSIS_PARAM_GLOBAL_DATA_BYTES - 1u) /
        CHASSIS_PARAM_GLOBAL_DATA_BYTES);
    uint8_t offset, remaining;
    if (paramGlobalRequestValid == 0u) return;
    memset(&fragment, 0, sizeof(fragment));
    fragment.streamSequence = paramGlobalRequestStreamSequence;
    fragment.fragmentIndex = paramGlobalRequestFragmentIndex;
    fragment.fragmentCount = count;
    offset = (uint8_t) (fragment.fragmentIndex * CHASSIS_PARAM_GLOBAL_DATA_BYTES);
    remaining = (uint8_t) (CHASSIS_PARAM_GLOBAL_REQUEST_BYTES - offset);
    fragment.length = (remaining > CHASSIS_PARAM_GLOBAL_DATA_BYTES) ?
        CHASSIS_PARAM_GLOBAL_DATA_BYTES : remaining;
    memcpy(fragment.data, &paramGlobalRequestBytes[offset], fragment.length);
    if ((chassisParamGlobalFragmentEncode(
        CHASSIS_LINK_MSG_PARAM_GLOBAL_REQUEST, &fragment, payload) != 0u) &&
        (thvd1450SendBytes(payload, sizeof(payload)) == THVD1450_STATUS_OK))
    {
        paramGlobalRequestFragmentIndex++;
        if (paramGlobalRequestFragmentIndex >= count)
            paramGlobalRequestFragmentIndex = 0u;
    }
}

static void chassisLinkHandleParamGlobalState(const thvd1450FrameStruct *frame)
{
    chassisParamGlobalFragment fragment;
    uint8_t count = (uint8_t) ((CHASSIS_PARAM_GLOBAL_STATE_BYTES +
        CHASSIS_PARAM_GLOBAL_DATA_BYTES - 1u) /
        CHASSIS_PARAM_GLOBAL_DATA_BYTES);
    uint8_t offset;
    if (chassisParamGlobalFragmentDecode(frame->payload, frame->length,
        CHASSIS_LINK_MSG_PARAM_GLOBAL_STATE, &fragment) == 0u) return;
    if (fragment.streamSequence != paramGlobalStateStreamSequence)
    {
        paramGlobalStateStreamSequence = fragment.streamSequence;
    }
}

```

```

    paramGlobalStateMask = 0u;
}
offset = (uint8_t) (fragment.fragmentIndex * CHASSIS_PARAM_GLOBAL_DATA_BYTES);
if ((fragment.fragmentCount == count) &&
    ((uint16_t) offset + fragment.length <= CHASSIS_PARAM_GLOBAL_STATE_BYTES))
{
    memcpy(&paramGlobalStateBytes[offset], fragment.data, fragment.length);
    paramGlobalStateMask |= (uint8_t) (1u << fragment.fragmentIndex);
    if (paramGlobalStateMask == (uint8_t) ((1u << count) - 1u))
    {
        if (paramSyncGlobalDecodeState(paramGlobalStateBytes,
            &paramGlobalState) != 0u)
        {
            {
                paramGlobalStateValid = 1u;
                paramGlobalStateAgeMs = 0u;
                if (((paramGlobalState.flags & PARAM_SYNC_GLOBAL_FLAG_TERMINAL) != 0u) &&
                    (paramGlobalState.requestBootGeneration ==
                     paramGlobalRequest.producerBootGeneration) &&
                    (paramGlobalState.requestSequence == paramGlobalRequest.sequence))
                    paramGlobalRequestValid = 0u;
            }
            paramGlobalStateMask = 0u;
        }
    }
}
else
{
    paramGlobalStateMask = 0u;
}
chassisLinkSendParamGlobalRequestFragment();
}

static uint8_t chassisLinkPublishParamAssembly(void)
{
    paramAssemblyActive = 0u;
    paramAssembly.dataLength = paramExpectedBytes;
    paramRequestSequence++;
    if (paramRequestSequence == 0u) paramRequestSequence = 1u;
    paramAssembly.producerBootGeneration = paramBridgeBootGeneration;
    paramAssembly.sequence = paramRequestSequence;
    paramAssembly.transactionId = paramRequestSequence;
    paramAssembly.offset = paramRequestSequence;
    paramAssembly.slot = (paramExpectedBytes != 0u) ? paramAssembly.data[0] : 0u;
    if (paramSyncMailboxPublishRequest(&paramAssembly) == 0u)
    {
        chassisLinkResetParamTransport();
        return 0u;
    }
    paramQuietMs = 0u;
    return 1u;
}

static void chassisLinkHandleParamRequest(const thvd1450FrameStruct* frame)
{
    chassisParamFrame wire;
    chassisParamFrame reply;
    paramSyncMailboxResponse response;
    uint8_t payload[CHASSIS_PARAM_FRAME_BYTES];
    uint8_t i;
    if (chassisParamDecode(frame->payload, frame->length,
        CHASSIS_LINK_MSG_PARAM_REQUEST, &wire) == 0u) return;
    paramQuietMs = 0u;
    reply = wire;
    reply.status = CHASSIS_PARAM_STATUS_PENDING;
    reply.length = 0u;
    if (wire.op == CHASSIS_PARAM_OP_POLL)
    {
        uint16_t remaining;
        if ((wire.target != 2u) || (wire.reserved != 0u) ||
            (wire.requestId == 0u) ||
            (wire.requestId != paramWireRequest))
        {
            {
                reply.status = PARAM_SYNC_STATUS_REJECTED;
            }
        }
        else
        {
            if (!paramCachedResponseValid) &&
                paramSyncMailboxTryReadResponse(&response) &&
                (response.transactionId == paramAssembly.transactionId) &&
                (response.offset == paramAssembly.offset) &&

```



```

(response.op == paramAssembly.op) &&
(response.slot == paramAssembly.slot) &&
(response.target == paramAssembly.target))
{
    if ((response.producerBootGeneration == 0u) ||
        ((paramOwnerBootGeneration != 0u) &&
         (response.producerBootGeneration !=
          paramOwnerBootGeneration)))
    {
        if (response.producerBootGeneration != 0u)
        {
            paramOwnerBootGeneration = response.producerBootGeneration;
            paramOwnerResponseSequence = response.sequence;
        }
        chassisLinkResetParamTransport();
        reply.status = CHASSIS_PARAM_OWNER_RESTART;
    }
    else if ((paramOwnerBootGeneration != 0u) &&
             (chassisLinkSequenceAdvanced(response.sequence,
             paramOwnerResponseSequence) == 0u))
    {
        /* Shared response is a non-consuming latest snapshot. The
         * response observed before this request is only a baseline;
         * accepting it would replay an old owner result after either
         * side restarts with repeated wire/request identities. */
    }
    else
    {
        paramOwnerBootGeneration = response.producerBootGeneration;
        paramOwnerResponseSequence = response.sequence;
        paramCachedResponse = response;
        paramCachedResponseValid = 1u;
    }
}
if (paramCachedResponseValid &&
    (paramCachedResponse.transactionId == paramAssembly.transactionId))
{
    reply.op = paramCachedResponse.op;
    reply.reserved = 0u;
    reply.target = paramCachedResponse.target;
    reply.status = (uint8_t) (paramCachedResponse.status &
                             CHASSIS_PARAM_STATUS_MASK);
    reply.offset = wire.offset;
    if (wire.offset > paramCachedResponse.dataLength)
    {
        reply.status = PARAM_SYNC_STATUS_INVALID;
    }
    else
    {
        remaining = (uint16_t) (paramCachedResponse.dataLength - wire.offset);
        reply.length = (remaining > CHASSIS_PARAM_DATA_BYTES) ? CHASSIS_PARAM_DATA_BYTES : (uint8_t) remaining;
        for (i = 0u; i < reply.length; i++)
            reply.data[i] = paramCachedResponse.data[wire.offset + i];
        if (remaining > reply.length)
            reply.status |= CHASSIS_PARAM_TRANSPORT_MORE;
    }
}
}
}
else if ((wire.status & CHASSIS_PARAM_TRANSPORT_BEGIN) != 0u)
{
    if ((wire.target != 2u) || (wire.reserved != 0u) ||
        (wire.offset > PARAM_SYNC_MAILBOX_DATA_MAX))
    {
        chassisLinkResetParamTransport();
        reply.status = PARAM_SYNC_STATUS_REJECTED;
    }
    else if ((wire.requestId == paramWireRequest) &&
             (wire.op == paramAssembly.op) &&
             (wire.reserved == 0u) &&
             (wire.target == paramAssembly.target) &&
             (wire.offset == paramExpectedBytes))
    {
        /* 丢失应答后的 BEGIN 重传必须保持已接收进度与 owner 应答。 */
    }
    else
    {
        /* 新事务发布前只读取一次 owner response 作为反重放基线；禁止在
         * 每轮 UART 排空前扫描 512B 共享邮箱。 */
    }
}

```

```

        chassisLinkRefreshOwnerResponseBaseline();
        chassisLinkResetParamTransport();
        memset(&paramAssembly, 0, sizeof(paramAssembly));
        paramWireRequest = wire.requestId;
        paramExpectedBytes = wire.offset;
        paramReceivedBytes = 0u;
        paramAssembly.op = wire.op;
        paramAssembly.target = wire.target;
        paramAssemblyActive = 1u;
        paramQuietMs = 0u;
        if ((paramExpectedBytes == 0u) &&
            (chassisLinkPublishParamAssembly() == 0u))
            reply.status = PARAM_SYNC_STATUS_FAILED;
    }
}
else if ((wire.requestId == paramWireRequest) &&
        (wire.op == paramAssembly.op) && (wire.reserved == 0u) &&
        (wire.target == paramAssembly.target) &&
        ((uint32_t) wire.offset + wire.length <= paramExpectedBytes))
{
    if (wire.offset < paramReceivedBytes)
    {
        if (((uint32_t) wire.offset + wire.length > paramReceivedBytes) ||
            (memcmp(&paramAssembly.data[wire.offset], wire.data,
                    wire.length) != 0))
        {
            chassisLinkResetParamTransport();
            reply.status = PARAM_SYNC_STATUS_INVALID;
        }
    }
    else if (paramAssemblyActive && wire.offset == paramReceivedBytes)
    {
        memcpy(&paramAssembly.data[paramReceivedBytes], wire.data, wire.length);
        paramReceivedBytes = (uint16_t) (paramReceivedBytes + wire.length);
        if (paramReceivedBytes == paramExpectedBytes)
        {
            if (chassisLinkPublishParamAssembly() == 0u)
                reply.status = PARAM_SYNC_STATUS_FAILED;
        }
    }
    else
    {
        chassisLinkResetParamTransport();
        reply.status = PARAM_SYNC_STATUS_INVALID;
    }
}
else
{
    chassisLinkResetParamTransport();
    reply.status = PARAM_SYNC_STATUS_INVALID;
}
}
if (chassisParamEncode(CHASSIS_LINK_MSG_PARAM_RESPONSE, &reply, payload))
    (void) thvd1450SendBytes(payload, CHASSIS_PARAM_FRAME_BYTES);
}

/* 遥控量有限性 + 粗幅值校验: vx/vy=m/s, wz=rad/s.
 * 这里仅拒绝 NaN/Inf/极端异常帧, 真实 RS-485 独立限速在 chassisApplicationApplyCommandWithLimit. */
uint8_t chassisLinkCommandValid(float vx, float vy, float wz)
{
    if (!isfinite(vx) || !isfinite(vy) || !isfinite(wz)) return 0u;
    if (vx > CHASSIS_LINK_CMD_SANITY_LIMIT || vx < -CHASSIS_LINK_CMD_SANITY_LIMIT) return 0u;
    if (vy > CHASSIS_LINK_CMD_SANITY_LIMIT || vy < -CHASSIS_LINK_CMD_SANITY_LIMIT) return 0u;
    if (wz > CHASSIS_LINK_CMD_SANITY_LIMIT || wz < -CHASSIS_LINK_CMD_SANITY_LIMIT) return 0u;
    return 1u;
}

void chassisLinkDecodeCommand(const thvd1450FrameStruct* frame,
    chassisLinkCommandDecodeStruct* decoded)
{
    decoded->vx = 0.0f;
    decoded->vy = 0.0f;
    decoded->wz = 0.0f;
    decoded->cmd = 0u;
    decoded->sysByte = 0u;
    decoded->valid = 0u;

    if (frame->length < CHASSIS_LINK_COMMAND_BASE_LENGTH || frame->payload[CHASSIS_LINK_TYPE_OFFSET] !=
        CHASSIS_LINK_MSG_COMMAND)
    {

```

```

    return;
}

chassisLinkWireDecodeVector(frame->payload,
    &decoded->vx, &decoded->vy, &decoded->wz);
if (!chassisLinkCommandValid(decoded->vx, decoded->vy, decoded->wz)) return;

decoded->cmd = frame->payload[CHASSIS_LINK_CONTROL_OFFSET];
decoded->sysByte = (frame->length >= CHASSIS_LINK_COMMAND_LENGTH)
    ? frame->payload[CHASSIS_LINK_SYSBYTE_OFFSET]
    : 0u;
decoded->valid = 1u;
}

static void chassisLinkSendReply(uint8_t type, float vx, float vy, float wz, uint8_t control)
{
    uint8_t payload[CHASSIS_LINK_TELEMETRY_LENGTH];
    chassisLinkWireEncodeVector(payload, type, vx, vy, wz);
    payload[CHASSIS_LINK_CONTROL_OFFSET] = control;
    thvd1450SendBytes(payload, (uint8_t) sizeof(payload)); /* 阻塞发送，内部切 DE */
}

void chassisLinkInit(void)
{
    uint32_t nextBootInverse;
    paramSyncMailboxRequest previousRequest;

    thvd1450Init();
    paramBridgeBootGeneration = chassisLinkNextParamBridgeBootGeneration(
        core2ParamBridgeBootRetained, core2ParamBridgeBootInverseRetained,
        &nextBootInverse);
    paramRequestSequence = 0u;
    if (paramSyncMailboxTryReadRequest(&previousRequest) != 0u)
    {
        /* Work RAM survives a single-core reset. Use the actual last
         * published mailbox identity as the new baseline, not only __no_init
         * retained state, so cold/torn bridge restarts cannot reproduce the
         * owner service's (producerBoot, sequence) de-dup tuple. */
        paramBridgeBootGeneration = previousRequest.producerBootGeneration + 1u;
        if (paramBridgeBootGeneration == 0u)
            paramBridgeBootGeneration = 1u;
        nextBootInverse = ~paramBridgeBootGeneration;
        paramRequestSequence = previousRequest.sequence;
    }
    core2ParamBridgeBootRetained = paramBridgeBootGeneration;
    core2ParamBridgeBootInverseRetained = nextBootInverse;
    chassisLinkResetParamTransport();
    paramOwnerBootGeneration = 0u;
    paramOwnerResponseSequence = 0u;
    chassisLinkRefreshOwnerResponseBaseline();
    memset(paramGlobalStateBytes, 0, sizeof(paramGlobalStateBytes));
    memset(paramGlobalRequestBytes, 0, sizeof(paramGlobalRequestBytes));
    memset(&paramGlobalState, 0, sizeof(paramGlobalState));
    memset(&paramGlobalRequest, 0, sizeof(paramGlobalRequest));
    paramGlobalStateAgeMs = CHASSIS_LINK_PARAM_GLOBAL_LOST_MS;
    paramGlobalStateStreamSequence = 0u;
    paramGlobalStateMask = 0u;
    paramGlobalRequestStreamSequence = 0u;
    paramGlobalRequestFragmentIndex = 0u;
    paramGlobalStateValid = 0u;
    paramGlobalRequestValid = 0u;
    linkCommand.vx = 0.0f;
    linkCommand.vy = 0.0f;
    linkCommand.wz = 0.0f;
    linkCommand.cmd = 0u;
    linkCommand.sysByte = 0u;
    linkCommand.fresh = 0u;
    linkCommand.ageMs = CHASSIS_LINK_CMD_TIMEOUT_MS;
    linkCommandAgeMs = CHASSIS_LINK_CMD_TIMEOUT_MS;
}

void chassisLinkSendTelemetry(float vx, float vy, float yawRate, uint8_t status)
{
    chassisLinkSendReply(CHASSIS_LINK_MSG_TELEMETRY, vx, vy, yawRate, status);
}

void chassisLinkSendLoraMotion(float vx, float vy, float wz,
    uint8_t remoteTakeoffReq, uint8_t attitudeFault)
{

```

```

uint8_t flags = remoteTakeoffReq ? CHASSIS_LINK_LORA_FLAG_TAKEOFF_REQ : 0u;
if (attitudeFault != 0u) flags |= CHASSIS_LINK_LORA_FLAG_ATTITUDE_FAULT;
chassisLinkSendReply(CHASSIS_LINK_MSG_LORA_MOTION, vx, vy, wz, flags);
}

void chassisLinkPoll(void)
{
    thvd1450FrameStruct frame;
    chassisLinkCommandStruct pending;
    uint8_t gotValid = 0u;

    /* 参数桥缓存只允许覆盖一次有界的重传窗口。Core1 网关重启时会先
    * 静默等待超过该窗口；因此即使其 __no_init 事务号因冷启动或写裂
    * 从 1 重新开始，也不能命中本核上一会话的缓存响应。控制链失联时
    * 同步撤销缓存，避免无主会话长期保留。这里只在前台改组装状态，
    * 1 ms Tick 仅推进 volatile 年龄，避免 ISR/前台竞争。*/
    if (((paramQuietMs >= CHASSIS_LINK_PARAM_TRANSPORT_TIMEOUT_MS) ||
        (linkCommandAgeMs >= CHASSIS_LINK_CMD_TIMEOUT_MS)) &&
        ((paramWireRequest != 0u) || (paramAssemblyActive != 0u) ||
        (paramCachedResponseValid != 0u)))
    {
        chassisLinkResetParamTransport();
    }
    thvd1450Poll(); /* 排空 UART 接收 ring 喂 parser */

    pending.vx = 0.0f;
    pending.vy = 0.0f;
    pending.wz = 0.0f;
    pending.cmd = 0u;
    pending.sysByte = 0u;

    /* 取尽当前队列中所有帧，保留最新一条有效 MSG_COMMAND（ReadFrame 成功返回 STATUS_OK，无帧返回 NO_FRAME）。
    * 有限性 + 幅值校验任一不过 → 丢弃该帧，不刷新链路年龄（等同未收到，维持失链急停）。*/
    while (thvd1450ReadFrame(&frame) == THVD1450_STATUS_OK)
    {
        chassisLinkCommandDecodeStruct decoded;
        if ((frame.length == CHASSIS_PARAM_FRAME_BYTES) &&
            (frame.payload[0] == CHASSIS_LINK_MSG_PARAM_REQUEST))
        {
            chassisLinkHandleParamRequest(&frame);
            continue; /* 参数帧绝不刷新 command freshness。*/
        }
        if ((frame.length == CHASSIS_PARAM_GLOBAL_FRAME_BYTES) &&
            (frame.payload[0] == CHASSIS_LINK_MSG_PARAM_GLOBAL_STATE))
        {
            chassisLinkHandleParamGlobalState(&frame);
            continue; /* 全核参数状态与控制命令新鲜度正交。*/
        }
        chassisLinkDecodeCommand(&frame, &decoded);
        if (decoded.valid)
        {
            pending.vx = decoded.vx;
            pending.vy = decoded.vy;
            pending.wz = decoded.wz;
            pending.cmd = decoded.cmd;
            pending.sysByte = decoded.sysByte;
            gotValid = 1u;
        }
    }

    if (gotValid)
    {
        /* 整块发布命令快照 + 清链路年龄（单临界区，防读方读到跨帧混合命令）。fresh 仅前台用，临界区外置。
        * PRIMASK 存-恢复，绝不用 SDK interrupt_global_disable/enable（坏计数器：前台 Poll 与 PIT 读
        * 高频交错会令 interrupt_global_flag 上漂致中断永久关闭、整机卡死；详见 cm_7_0 镜像段注释）。*/
        uint32_t primask = __get_PRIMASK();
        __disable_irq();
        linkCommand.vx = pending.vx;
        linkCommand.vy = pending.vy;
        linkCommand.wz = pending.wz;
        linkCommand.cmd = pending.cmd;
        linkCommand.sysByte = pending.sysByte;
        linkCommandAgeMs = 0u;
        __set_PRIMASK(primask);
        linkCommand.fresh = 1u;
    }
    else
    {
        linkCommand.fresh = 0u;
    }
}

```

```

    }
}

void chassisLinkTickMs(void)
{
    /* 1ms 累加链路年龄，饱和于超时阈值（uint32 不回卷）*/
    if (linkCommandAgeMs < CHASSIS_LINK_CMD_TIMEOUT_MS)
    {
        linkCommandAgeMs++;
    }
    if (paramQuietMs < CHASSIS_LINK_PARAM_TRANSPORT_TIMEOUT_MS)
        paramQuietMs++;
    if (paramGlobalStateAgeMs < CHASSIS_LINK_PARAM_GLOBAL_LOST_MS)
        paramGlobalStateAgeMs++;
}

void chassisLinkGetCommand(chassisLinkCommandStruct* out)
{
    uint32_t primask;
    if (out == 0) return;
    /* 整块快照（与 Poll 的整块发布对称），消除读到跨帧混合命令的撕裂。
    * PRIMASK 存-恢复，绝不用 SDK interrupt_global_disable/enable（坏计数器，详见镜像段注释）。*/
    primask = __get_PRIMASK();
    __disable_irq();
    out->vx = linkCommand.vx;
    out->vy = linkCommand.vy;
    out->wz = linkCommand.wz;
    out->cmd = linkCommand.cmd;
    out->sysByte = linkCommand.sysByte;
    out->ageMs = linkCommand.ageMs;
    __set_PRIMASK(primask);
    out->fresh = linkCommand.fresh;
}

uint8_t chassisLinkGetParamGlobalState(paramSyncGlobalState *state)
{
    {
        if ((state == 0) || (paramGlobalStateValid == 0u) ||
            (paramGlobalStateAgeMs >= CHASSIS_LINK_PARAM_GLOBAL_LOST_MS) ||
            (paramSyncGlobalStateIsValid(&paramGlobalState) == 0u)) return 0u;
        *state = paramGlobalState;
        return 1u;
    }
}

void chassisLinkSetParamGlobalRequest(const paramSyncGlobalRequest *request,
uint8_t valid)
{
    {
        if ((valid == 0u) || (request == 0) ||
            (paramSyncGlobalRequestIsValid(request) == 0u) ||
            (request->origin != 2u))
        {
            paramGlobalRequestValid = 0u;
            return;
        }
        if ((paramGlobalRequestValid == 0u) ||
            (request->producerBootGeneration !=
                paramGlobalRequest.producerBootGeneration) ||
            (request->sequence != paramGlobalRequest.sequence))
        {
            paramGlobalRequest = *request;
            paramSyncGlobalEncodeRequest(request, paramGlobalRequestBytes);
            paramGlobalRequestStreamSequence++;
            if (paramGlobalRequestStreamSequence == 0u)
                paramGlobalRequestStreamSequence = 1u;
            paramGlobalRequestFragmentIndex = 0u;
        }
        paramGlobalRequestValid = 1u;
    }
}

uint8_t chassisLinkIsAlive(void)
{
    {
        return (linkCommandAgeMs < CHASSIS_LINK_CMD_TIMEOUT_MS) ? 1u : 0u;
    }
}

uint8_t chassisLinkGetCore0Alive(void)
{
    {
        /* core1 经 RS-485 cmd bit4 转告"core0 核间链路活"。485 失链时不信陈旧位 → 0。
        * linkCommand.cmd 单字节读原子（Poll 在临界区整块发布），无需关中断。*/
        if (!chassisLinkIsAlive()) return 0u;
        return (linkCommand.cmd & CHASSIS_LINK_CMD_CORE0_ALIVE) ? 1u : 0u;
    }
}

```

```
}

uint8_t chassisLinkCommandReceived(void)
{
    /* fresh 由 chassisLinkPoll 每次置：收到有效命令=1，否则=0。供请求-应答触发应答帧。 */
    return linkCommand.fresh;
}

#endif /* gate */
```

core2_cyt4bb7/project/code/comm/chassisLink.h

SHA-256

ea06dcb7fd071e3e9a25dd44ba597d5c25855ed1f5d630953a8e133b1193ab2e

代码 (c)

```
#ifndef _CHASSIS_LINK_H_
#define _CHASSIS_LINK_H_

#include <stdint.h>

#include "chassisLinkProtocol.h"
#include "paramSyncGlobal.h"

/*
 * core2 ↔ core1 RS-485 链路层（基于 THVD1450 帧层）
 *
 * 遥测 core2→core1 (MSG_TELEMETRY, 请求-应答)：编码器正解算实速 vx/vy + 陀螺 yawRate + 状态字。
 * 遥控 core1→core2 (MSG_COMMAND, 前台解析)：目标 vx/vy(m/s)/wz(rad/s) + 指令字（非遥控模式消费）。
 *
 * payload[0] = 消息类型；其后为小端 float32 + 状态/指令字节（手填，不用 PackFloats 以容纳类型字节）。
 * 收发经 thvd1450SendBytes / thvd1450ReadFrame（payload 级），半双工 DE 由 THVD1450 自我管理。
 */

/* core2 本地扩展码：超出 sysByte 3-bit(0-7) 范围，仅本核 MODE4 直接置(不经 RS-485 传输，core0/core1 不产生此码)。
 * 注意：经 CHASSIS_LINK_SYS_BUZZER_MASK(0x07) 提取的链路码永远 ≤7，不会取到 8，二者无冲突。 */
#define CHASSIS_LINK_BUZZER_LORA_IDLE (8u) /* 模式 4 本地 LoRa 兜底提示：0.5Hz 每 2s 响 150ms(人耳可辨别，非告警) */

/* 解析出的遥控指令（非遥控模式下作为运动指令源） */
typedef struct
{
    float vx; /* 目标前向速度 m/s */
    float vy; /* 目标右向速度 m/s */
    float wz; /* 目标角速度 rad/s */
    uint8_t cmd; /* 指令位 (CHASSIS_LINK_CMD_*) */
    uint8_t sysByte; /* sysByte (CHASSIS_LINK_SYS_*：控制源 + 蜂鸣码，core0 下覆盖本地源) */
    uint8_t fresh; /* 1=本次 Poll 收到新指令帧 */
    uint32_t ageMs; /* 距上次有效指令的毫秒数（超时判链路丢失） */
} chassisLinkCommandStruct;

/* sysByte 解读 helper（供 core0Loop 决策控制源/蜂鸣） */
#define CHASSIS_LINK_CMD_SOURCE_IS_LORA(c) (((c).sysByte & CHASSIS_LINK_SYS_SOURCE_LORA) ? 1u : 0u)
#define CHASSIS_LINK_CMD_BUZZER(c) ((uint8_t)((c).sysByte >> CHASSIS_LINK_SYS_BUZZER_SHIFT) & CHASSIS_LINK_SYS_BUZZER_MASK)

void chassisLinkInit(void);

/* 收到有效命令后发送遥测应答：vx/vy(实速 m/s)、yawRate(rad/s)、status 状态位。半双工阻塞发送。 */
void chassisLinkSendTelemetry(float vx, float vy, float yawRate, uint8_t status);

/* 发送 LoRa 遥控值 vx/vy(m/s)/wz(rad/s)(LORA_LINKED 应答, 供 core1→core0 无人机前馈)
 * + remoteTakeoffReq(模式 3 LoRa S3+S4 请求)及姿态故障事实。
 * payload 14 字节=[0x03|3×float|flags(bit0=req, bit1=attitude fault)]。
 * 透传控制核按本地参数实际应用的遥控目标值(非实速)。半双工阻塞发送。 */
void chassisLinkSendLoraMotion(float vx, float vy, float wz,
    uint8_t remoteTakeoffReq, uint8_t attitudeFault);

/* 前台轮询：排空 RX、解析 MSG_COMMAND 帧，更新内部遥控指令快照。 */
void chassisLinkPoll(void);

/* 1ms 调用：链路年龄计时（供超时判定）。 */
void chassisLinkTickMs(void);

/* 读取最近一次遥控指令快照（非遥控模式消费）。 */
void chassisLinkGetCommand(chassisLinkCommandStruct* out);
```

```
/* 最近一次 chassisLinkPoll 是否收到有效 MSG_COMMAND (请求-应答: core1 发命令作"请求",
 * core2 据此回一帧应答, 避免半双工总线碰撞)。1=本轮收到, 0=未收到。*/
uint8_t chassisLinkCommandReceived(void);

/* 485 链路是否活跃 (近期收到过 core1 帧, 未超时)。*/
uint8_t chassisLinkIsAlive(void);

/* core1 经 RS-485 cmd bit4 转告的"core0 核间链路活"。485 失链时返回 0 (不信陈旧位)。
 * 供 core2 等待屏全链路门控 (等 core0+core1) + 全通蜂鸣三声边沿检测。*/
uint8_t chassisLinkGetCore0Alive(void);
uint8_t chassisLinkGetParamGlobalState(paramSyncGlobalState *state);
void chassisLinkSetParamGlobalRequest(const paramSyncGlobalRequest *request,
    uint8_t valid);

#include "configCore2Build.h" /* CORE2_BUILD_MODE */
#if defined(CY_CORE_CM7_0) && (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
/* cm_7_0 生产态镜像实现专用: whileProcess 把 cm_7_1 经共享内存发布的命令快照刷新到本地镜像。
 * 仅在 commToControl.magic 有效时调用。*/
#include "coreDataTransfer.h"
void chassisLinkMirrorApply(const commToControlStruct *s);
void chassisLinkMirrorInvalidate(void);
#endif

#endif /* _CHASSIS_LINK_H_ */
```

core2_cyt4bb7/project/code/comm/chassisLinkInternal.h

SHA-256

:

9e35b6ef4b480aaa363ab82055cbca34df282cca2d895beb3051c1d078d796ea

代码 (c)

```
#ifndef CHASSIS_LINK_INTERNAL_H
#define CHASSIS_LINK_INTERNAL_H

#include <stdint.h>

#include "THVD1450_protocol.h"

typedef struct
{
    float vx;
    float vy;
    float wz;
    uint8_t cmd;
    uint8_t sysByte;
    uint8_t valid;
} chassisLinkCommandDecodeStruct;

static inline uint32_t chassisLinkNextParamBridgeBootGeneration(
    uint32_t retained, uint32_t retainedInverse, uint32_t *nextInverse)
{
    uint32_t next = retained;
    if ((next == 0u) || (retainedInverse != ~next)) next = 0u;
    next++;
    if (next == 0u) next = 1u;
    *nextInverse = ~next;
    return next;
}

uint8_t chassisLinkCommandValid(float vx, float vy, float wz);
void chassisLinkDecodeCommand(const thvd1450FrameStruct* frame,
    chassisLinkCommandDecodeStruct* decoded);

#endif
```

core2_cyt4bb7/project/code/comm/chassisLinkProtocol.h

SHA-256

:

9ad1367a6cbe7fb9c9c9a61ae4dc8d2103ecc74ef0be36c0eb59aa948ad3a5af

代码 (c)

```
#ifndef CHASSIS_LINK_PROTOCOL_H
#define CHASSIS_LINK_PROTOCOL_H

#include <stdint.h>

#define CHASSIS_LINK_MSG_TELEMETRY (0x01u)
#define CHASSIS_LINK_MSG_COMMAND (0x02u)
#define CHASSIS_LINK_MSG_LORA_MOTION (0x03u)

#define CHASSIS_LINK_TYPE_OFFSET (0u)
#define CHASSIS_LINK_VX_OFFSET (1u)
#define CHASSIS_LINK_VY_OFFSET (5u)
#define CHASSIS_LINK_WZ_OFFSET (9u)
#define CHASSIS_LINK_CONTROL_OFFSET (13u)
#define CHASSIS_LINK_SYSBYTE_OFFSET (14u)

#define CHASSIS_LINK_LORA_BASE_LENGTH (13u)
#define CHASSIS_LINK_LORA_LENGTH (14u)
#define CHASSIS_LINK_TELEMETRY_LENGTH (14u)
#define CHASSIS_LINK_COMMAND_BASE_LENGTH (14u)
#define CHASSIS_LINK_COMMAND_LENGTH (15u)

#define CHASSIS_LINK_LORA_FLAG_TAKEOFF_REQ (0x01u)
#define CHASSIS_LINK_LORA_FLAG_ATTITUDE_FAULT (0x02u)

#define CHASSIS_LINK_STATUS_ENABLED (0x01u)
#define CHASSIS_LINK_STATUS_LINK_OK (0x02u)
#define CHASSIS_LINK_STATUS_ESTOP (0x04u)
#define CHASSIS_LINK_STATUS_LAUNCHED (0x08u)
#define CHASSIS_LINK_STATUS_MODE2_LORA_LAND_REQ (0x10u)
#define CHASSIS_LINK_STATUS_ATTITUDE_FAULT (0x20u)
#define CHASSIS_LINK_CMD_ESTOP (0x01u)
#define CHASSIS_LINK_CMD_ENABLE (0x02u)
#define CHASSIS_LINK_CMD_LAUNCH (0x08u)
#define CHASSIS_LINK_CMD_CORE0_ALIVE (0x10u)

#define CHASSIS_LINK_SYS_SOURCE_LORA (0x01u)
#define CHASSIS_LINK_SYS_BUZZER_SHIFT (1u)
#define CHASSIS_LINK_SYS_BUZZER_MASK (0x07u)

#define CHASSIS_LINK_BUZZER_SILENT (0u)
#define CHASSIS_LINK_BUZZER_1HZ (1u)
#define CHASSIS_LINK_BUZZER_LONG (2u)
#define CHASSIS_LINK_BUZZER_3S (3u)
#define CHASSIS_LINK_BUZZER_WARN (4u)
#define CHASSIS_LINK_BUZZER_ARMED_IDLE (5u)
#define CHASSIS_LINK_BUZZER_WAITING (6u)
#define CHASSIS_LINK_BUZZER_THREE_BEEPS (7u)

typedef char chassisLink_float_size_assert[(sizeof(float) == 4u) ? 1 : -1];

#endif
```

core2_cyt4bb7/project/code/comm/chassisLinkWireInternal.h

SHA-256 :
a5d3b8eef02b0730d1659ecf4a429dea95901503427c84a0474bf38daf506cf0

代码 (c)

```
#ifndef CHASSIS_LINK_WIRE_INTERNAL_H
#define CHASSIS_LINK_WIRE_INTERNAL_H

#include <string.h>

#include "chassisLinkProtocol.h"

static void chassisLinkWireEncodeVector(uint8_t* payload, uint8_t type,
float vx, float vy, float wz)
{
    payload[CHASSIS_LINK_TYPE_OFFSET] = type;
    memcpy(&payload[CHASSIS_LINK_VX_OFFSET], &vx, sizeof(vx));
}
```



```

        memcpy(&payload[CHASSIS_LINK_VY_OFFSET], &vy, sizeof(vy));
        memcpy(&payload[CHASSIS_LINK_WZ_OFFSET], &wz, sizeof(wz));
    }

static void chassisLinkWireDecodeVector(const uint8_t* payload,
    float* vx, float* vy, float* wz)
{
    memcpy(vx, &payload[CHASSIS_LINK_VX_OFFSET], sizeof(*vx));
    memcpy(vy, &payload[CHASSIS_LINK_VY_OFFSET], sizeof(*vy));
    memcpy(wz, &payload[CHASSIS_LINK_WZ_OFFSET], sizeof(*wz));
}

#endif

```

core2_cyt4bb7/project/code/comm/coreDataTransfer.c

SHA-256

:

45c0dc8833d027791a24a15117df2ac8e079df775a5c5628169464e0431e8d38

代码 (c)

```

#include "coreDataTransfer.h"

#include "cy_device_headers.h"

#include <string.h>

#define CORE2_CACHE_LINE_SIZE (32u)

/* #pragma location 把共享结构强制放共享 RAM 固定地址(__no_init 绝对放置),
 * 让链接器保留空间。照搬 core0/core1 coreDataTransfer 机制。 */
#pragma location = CORE2_COMM_TO_CTRL_ADDR
__no_init volatile commToControlStruct commToControlRegion;

static uint32_t commPublishGeneration;

static volatile void* cacheLineFor(const volatile void* address)
{
    uintptr_t value = (uintptr_t) address;
    return (volatile void*) (value & ~(uintptr_t) CORE2_CACHE_LINE_SIZE - 1u);
}

static void cleanLine(const volatile void* address)
{
    SCB_CleanDCache_by_Addr(cacheLineFor(address), CORE2_CACHE_LINE_SIZE);
    __DSB();
}

static void invalidateLine(const volatile void* address)
{
    SCB_InvalidateDCache_by_Addr(cacheLineFor(address), CORE2_CACHE_LINE_SIZE);
    __DSB();
}

static uint32_t nextGeneration(uint32_t* generation)
{
    {
        *generation += 2u;
        if (*generation == 0u) *generation = 2u;
        return *generation;
    }
}

void coreDataWriteCommToControl(const commToControlStruct* src)
{
    {
        commToControlStruct frame;
        uint32_t committed;

        if (0 == src) return;

        frame = *src;
        committed = nextGeneration(&commPublishGeneration);

        commToControlRegion.seq = committed | 1u;
        cleanLine(&commToControlRegion.seq);

        frame.magic = CORE2_COMM_READY_MAGIC;
    }
}

```

```

    frame.seq = committed | 1u;
    frame.publishConfirm = committed | 1u;
    memcpy((void*) &commToControlRegion, &frame, sizeof(frame));
    SCB_CleanDCache_by_Addr((volatile void*) &commToControlRegion, sizeof(commToControlStruct));
    __DSB();

    commToControlRegion.publishConfirm = committed;
    cleanLine(&commToControlRegion.publishConfirm);
    commToControlRegion.seq = committed;
    cleanLine(&commToControlRegion.seq);
}

uint8_t coreDataReadCommToControl(commToControlStruct* dst)
{
    uint8_t attempt;

    if (0 == dst) return 0u;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        uint32_t before;
        uint32_t after;
        uint32_t confirm;

        invalidateLine(&commToControlRegion.seq);
        before = commToControlRegion.seq;
        if ((before == 0u) || ((before & 1u) != 0u)) continue;

        SCB_InvalidateDCache_by_Addr((volatile void*) &commToControlRegion, sizeof(commToControlStruct));
        __DSB();
        memcpy(dst, (const void*) &commToControlRegion, sizeof(*dst));

        invalidateLine(&commToControlRegion.publishConfirm);
        confirm = commToControlRegion.publishConfirm;
        invalidateLine(&commToControlRegion.seq);
        after = commToControlRegion.seq;
        if ((before == after) && (before == dst->seq) &&
            (before == dst->publishConfirm) && (before == confirm) &&
            (dst->magic == CORE2_COMM_READY_MAGIC))
        {
            return 1u;
        }
    }

    memset(dst, 0, sizeof(*dst));
    return 0u;
}

/* 反向区: cm_7_0 控制核 → cm_7_1 通信核(应答快照)。另起固定地址, 与 commToControlRegion 无重叠。 */
#pragma location = CORE2_CTRL_TO_COMM_ADDR
__no_init volatile controlToCommStruct controlToCommRegion;

static uint32_t controlPublishGeneration;

void coreDataWriteControlToComm(const controlToCommStruct* src)
{
    controlToCommStruct frame;
    uint32_t committed;

    if (0 == src) return;

    frame = *src;
    committed = nextGeneration(&controlPublishGeneration);

    controlToCommRegion.seq = committed | 1u;
    cleanLine(&controlToCommRegion.seq);

    frame.magic = CORE2_CTRL_READY_MAGIC;
    frame.seq = committed | 1u;
    frame.publishConfirm = committed | 1u;
    memcpy((void*) &controlToCommRegion, &frame, sizeof(frame));
    SCB_CleanDCache_by_Addr((volatile void*) &controlToCommRegion, sizeof(controlToCommStruct));
    __DSB();

    controlToCommRegion.publishConfirm = committed;
    cleanLine(&controlToCommRegion.publishConfirm);
    controlToCommRegion.seq = committed;
    cleanLine(&controlToCommRegion.seq);
}

```

```

uint8_t coreDataReadControlToComm(controlToCommStruct* dst)
{
    uint8_t attempt;

    if (0 == dst) return 0u;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        uint32_t before;
        uint32_t after;
        uint32_t confirm;

        invalidateLine(&controlToCommRegion.seq);
        before = controlToCommRegion.seq;
        if ((before == 0u) || ((before & 1u) != 0u)) continue;

        SCB_InvalidateDCache_by_Addr((volatile void*) &controlToCommRegion, sizeof(controlToCommStruct));
        __DSB();
        memcpy(dst, (const void*) &controlToCommRegion, sizeof(*dst));

        invalidateLine(&controlToCommRegion.publishConfirm);
        confirm = controlToCommRegion.publishConfirm;
        invalidateLine(&controlToCommRegion.seq);
        after = controlToCommRegion.seq;
        if ((before == after) && (before == dst->seq) &&
            (before == dst->publishConfirm) && (before == confirm) &&
            (dst->magic == CORE2_CTRL_READY_MAGIC))
        {
            return 1u;
        }
    }

    memset(dst, 0, sizeof(*dst));
    return 0u;
}

```

core2_cyt4bb7/project/code/comm/coreDataTransfer.h

SHA-256

:

92ccd93183850cd48e9fab8717e18034da3a2de8fab9b1528c327c566c431c51

代码 (c)

```

/*
 * core2 核间共享内存 (cm_7_1 通信核 ↔ cm_7_0 控制核)
 *
 * 物理区域: section .global_ram_data
 * startAddr = 0x28001000, size = 0x2000 (core2 .icf 已定义, 与 core0/core1 同源空闲段)
 *
 * 背景: RS-485(THVD1450/UART4) 与 LoRa(loraRemote/UART1) 的收发整体搬到空闲的 cm_7_1 (通信核),
 * cm_7_1 重新启用中断驱动 RX (库 uart_query_byte 依赖 RX ISR 填软件缓冲), 其 UART 中断风暴只会
 * 拖慢空闲的 cm_7_1, cm_7_0 的 500μs 控制环被彻底隔离。两核经本共享内存桥接高层状态。
 *
 * 方向:
 * commToControl (cm_7_1 → cm_7_0): RS-485 解码命令 + LoRa 原始摇杆/解锁/边沿, 控制核按本地参数映射后驱动电机。
 * controlToComm (cm_7_0 → cm_7_1): 应答快照 (应答类型 + 遥测实速 + 已应用 LoRa 运动命令), 通信核收到 core1 请求后照发。
 *
 * 双核 D-Cache 一致性 (Cortex-M7, cache line=32B), 照搬 core0/core1 coreDataTransfer:
 * 写方 memcpy→SCB_CleanDCache_by_Addr; 读方 SCB_InvalidateDCache_by_Addr→memcpy。
 * 握手 32 位 magic 防 __no_init 上电垃圾; recordSeq/replaySeq 单调计数跨核交付单按边沿 (不可 read-and-clear)。
 * 两区相隔 256B(0x100) 各占独立 cache line, 无伪共享。
 */
#ifndef CORE2_DATA_TRANSFER_H_
#define CORE2_DATA_TRANSFER_H_

#include <stdint.h>

#include "paramSyncMailbox.h"
#include "paramSyncGlobal.h"

/* =====
 * 区 A: cm_7_1 通信核 → cm_7_0 控制核 (commToControl)
 * ===== */
#define CORE2_COMM_TO_CTRL_ADDR (0x28001000u)
#define CORE2_COMM_READY_MAGIC (0x5A5AA5A4u)

```

```

typedef struct
{
    uint32_t magic; /* = CORE2_COMM_READY_MAGIC, cm_7_1 初始化完成后置位（防 __no_init 上电垃圾） */
    uint32_t seq; /* 事务开始代数，由 coreDataTransfer owner 填写 */

    /* ---- RS-485 命令快照（由 core1 MSG_COMMAND 解码） ---- */
    float cmdVx; /* core1 目标 vx */
    float cmdVy; /* core1 目标 vy */
    float cmdWz; /* core1 目标 wz */
    uint8_t cmd; /* CHASSIS_LINK_CMD_* (ESTOP/ENABLE/LAUNCH/CORE0_ALIVE) */
    uint8_t sysByte; /* CHASSIS_LINK_SYS_* (控制源位 + 蜂鸣码) */
    uint8_t linkAlive; /* 1=RS-485 链路活 (cm_7_1 侧 age 未超时) */
    uint8_t padding0;

    /* ---- LoRa 解码快照 (cm_7_1 本地解码 + chord/边沿状态机) ----
     * 只跨核传原始摇杆，cm_7_0 的 loraRemote owner 用 runtime 下发的窄配置映射，避免双核参数不一致。 */
    int16_t loraVxRaw; /* joystick[1]: 左摇杆上下 */
    int16_t loraVyRaw; /* joystick[0]: 左摇杆左右，右推为 +vy */
    int16_t loraWzRaw; /* joystick[2]: 右摇杆左右 */
    int16_t loraRawPad; /* 对齐填充: 3×int16 摇杆后补到 8 字节边界 */
    uint8_t loraUnlocked; /* loraRemotelUnlocked() */
    uint8_t loraLinkAlive; /* loraRemotelLinkAlive() */
    uint8_t takeoffReq; /* loraRemoteGetRemoteTakeoffReq() (联机 S3+S4 请求; 模式 2/3 由上层链路分流) */
    uint8_t padding1;

    /* ---- 单按边沿以单 word seq 计数交付 (不可跨核 read-and-clear) ----
     * cm_7_1 每检测到一次单按上升沿 +1; cm_7_0 保留各自 last-consumed, 正增量触发一次。
     * 容忍发布(~1ms)与消费(~500µs ISR)节拍差, 无丢/无重; boot 首见基线同步防伪边沿。 */
    uint32_t recordSeq; /* S4 单按边沿计数 */
    uint32_t replaySeq; /* S3 单按边沿计数 */

    uint32_t paramGlobalStateSeq;
    paramSyncGlobalState paramGlobalState;
    uint8_t paramGlobalStateValid;
    uint8_t padding2[15];
    uint32_t publishConfirm; /* 事务结束代数，必须与 seq 相等且为偶数 */
} commToControlStruct;

typedef char core2_comm_to_control_size_assert[(sizeof(commToControlStruct) == 96u) ? 1 : -1];

/* cm_7_1 调用: 发布通信→控制快照 (Clean) */
void coreDataWriteCommToControl(const commToControlStruct* src);
/* cm_7_0 调用: 读取通信→控制快照 (Invalidate) */
uint8_t coreDataReadCommToControl(commToControlStruct* dst);

/* =====
 * 区 B: cm_7_0 控制核 → cm_7_1 通信核 (controlToComm)
 * 另起 cache line, 与 64B 的 commToControl(0x28 001 000) 无重叠。
 * ===== */
#define CORE2_CTRL_TO_COMM_ADDR (0x28 001 100u)
#define CORE2_CTRL_READY_MAGIC (0xA5A5A5Bu)

typedef struct
{
    uint32_t magic; /* = CORE2_CTRL_READY_MAGIC */
    uint32_t seq; /* 事务开始代数，由 coreDataTransfer owner 填写 */

    uint8_t replyType; /* 应答类型 = CHASSIS_LINK_MSG_TELEMETRY | CHASSIS_LINK_MSG_LORA_MOTION */
    uint8_t status; /* 遥测状态位 (ENABLED/LINK_OK/ESTOP/LAUNCHED), TELEMETRY 用 */
    uint8_t takeoffReq; /* LORA_MOTION payload[13] bit0 (cm_7_1 通常用本地新鲜值, 此为后备) */
    uint8_t padding0;

    float teleVx; /* TELEMETRY: 编码器正解算实速 vx */
    float teleVy; /* TELEMETRY: 实速 vy */
    float teleWz; /* TELEMETRY: 陀螺 yawRate */

    float loraCmdVx; /* LORA_MOTION: cm_7_0 按本地参数映射并实际应用的 LoRa vx(m/s) */
    float loraCmdVy;
    float loraCmdWz;

    paramSyncGlobalRequest paramGlobalRequest;
    uint32_t paramGlobalRequestSeq;
    uint8_t paramGlobalRequestValid;
    uint8_t padding1[7];
    uint32_t publishConfirm; /* 事务结束代数，必须与 seq 相等且为偶数 */
} controlToCommStruct;

typedef char core2_control_to_comm_size_assert[(sizeof(controlToCommStruct) == 64u) ? 1 : -1];
typedef char core2_comm_to_control_alignment_assert[(((CORE2_COMM_TO_CTRL_ADDR & 31u) == 0u) ? 1 : -1)];

```

```

typedef char core2_control_to_comm_alignment_assert[(((CORE2_CTRL_TO_COMM_ADDR & 31u) == 0u) ? 1 : -1)];
typedef char core2_shared_regions_nonoverlap_assert[
    ((CORE2_COMM_TO_CTRL_ADDR + sizeof(commToControlStruct)) <= CORE2_CTRL_TO_COMM_ADDR) ? 1 : -1];
typedef char core2_control_before_param_sync_assert[
    ((CORE2_CTRL_TO_COMM_ADDR + sizeof(controlToCommStruct)) <=
    PARAM_SYNC_MAILBOX_REGION_ADDR) ? 1 : -1];
typedef char core2_param_sync_region_end_assert[
    ((PARAM_SYNC_MAILBOX_REGION_ADDR + PARAM_SYNC_MAILBOX_REGION_BYTES) <=
    0x28002000u) ? 1 : -1];

/* cm_7_0 调用：发布应答快照 (Clean) */
void coreDataWriteControlToComm(const controlToCommStruct* src);
/* cm_7_1 调用：读取应答快照 (Invalidate) */
uint8_t coreDataReadControlToComm(controlToCommStruct* dst);

#endif /* _CORE2_DATA_TRANSFER_H_ */

```

core2_cyt4bb7/project/code/comm/loraNavControl.c

SHA-256

:

965f68193fbeb1daf8bf619b692f229e0c2ac503c71143607e3dff0b5e060e88

代码 (c)

```

#include "loraNavControl.h"

#include "buzzer.h" /* buzzerBeepEvent：事件确认音改经单一所有者蜂鸣器库提交，不再直写 BEEP */
#include "chassisApplication.h"
#include "commonDef.h"
#include "configChassisRemote.h"
#include "configCore2Build.h"
#include "loraRemote.h"
#include "motorControl.h"
#include "pinMap.h"
#include "zf_driver_gpio.h"
#include "zf_common_interrupt.h" /* snapshot 发布代数内存屏障 */

typedef struct
{
    wheelSpeedStruct baseTargetSpeed;
    float wz;
    int32_t flAccum;
    int32_t frAccum;
    int32_t blAccum;
    int32_t brAccum;
} loraNavControlFrameStruct;

typedef struct
{
    loraNavControlFrameStruct frames[LORA_NAV_MAX_FRAMES];
    loraNavControlSnapshotStruct status;
    uint8_t speedLoopDividerCount;
    uint8_t yawRateLoopDividerCount;
    uint8_t loraTickDividerCount;
    uint8_t lockPrevUnlocked;
    uint8_t linkInitReady;
    uint16_t linkAliveTickCount;
    uint16_t linkLostTickCount;
    float correctionKp;
    float correctionLimit;
    float errorLimitPulses;
    float loraVMaxMps;
    float loraWzMaxRadps;
} loraNavControlContextStruct;

static loraNavControlContextStruct loraNavContext = {0};
static volatile loraNavControlSnapshotStruct loraNavPublished = {.state = LORA_NAV_IDLE};
static volatile uint32_t loraNavPublishGeneration = 0u;

static const chassisMotionCommandStruct NAV_ZERO_COMMAND = {0.0f, 0.0f, 0.0f};

static void loraNavControlPublish(void);

/* 对称限幅薄包装：复用 commonDef.h::clampSym（消除重复实现）。
 * 保留为函数而非内联宏，保证调用点只求值一次。 */

```

```

static float navClampFloat(float value, float limit)
{
    return clampSym(value, limit);
}

static float navRampScale(uint16_t index, uint16_t count)
{
    uint16_t remaining;
    float startScale = 1.0f;
    float endScale = 1.0f;

    if (NAV_RAMP_FRAMES == 0u)
    {
        return 1.0f;
    }

    if (index < NAV_RAMP_FRAMES)
    {
        startScale = (float) (index + 1u) / (float) NAV_RAMP_FRAMES;
    }

    remaining = count - index;
    if (remaining < NAV_RAMP_FRAMES)
    {
        endScale = (float) remaining / (float) NAV_RAMP_FRAMES;
    }

    return (startScale < endScale) ? startScale : endScale;
}

static void navResetActualAccum(void)
{
    loraNavContext.status.flActualAccum = 0;
    loraNavContext.status.frActualAccum = 0;
    loraNavContext.status.blActualAccum = 0;
    loraNavContext.status.brActualAccum = 0;
    loraNavContext.status.flError = 0.0f;
    loraNavContext.status.frError = 0.0f;
    loraNavContext.status.blError = 0.0f;
    loraNavContext.status.brError = 0.0f;
}

static void navStopMotion(loraNavControlStateEnum nextState)
{
    loraNavContext.speedLoopDividerCount = 0u;
    loraNavContext.yawRateLoopDividerCount = 0u;
    chassisApplicationSetYawRateEnable(0u);
    chassisApplicationStop();
    motorControlStopAll();
    loraNavContext.status.state = nextState;
}

static uint8_t navIsManualDriveState(loraNavControlStateEnum state)
{
    return (state == LORA_NAV_IDLE || state == LORA_NAV_READY || state == LORA_NAV_DONE) ? 1u : 0u;
}

static uint8_t navShouldRunOutput(loraNavControlStateEnum state)
{
    return (navIsManualDriveState(state) || state == LORA_NAV_RECORDING || state == LORA_NAV_REPLAYING) ? 1u : 0u;
}

static void navStartRecording(void)
{
    loraNavContext.status.frameCount = 0u;
    loraNavContext.status.frameIndex = 0u;
    navResetActualAccum();
    motorControlStopAll();
    loraNavContext.yawRateLoopDividerCount = 0u;
    chassisApplicationSetYawRateEnable(1u);
    loraNavContext.status.state = LORA_NAV_RECORDING;
    buzzerBeepEvent(BEEP_TICKS_SHORT);
}

static void navFinishRecording(void)
{
    if (loraNavContext.status.frameCount == 0u)
    {
        navStopMotion(LORA_NAV_IDLE);
    }
}

```

```

    }
    else
    {
        navStopMotion(LORA_NAV_READY);
        buzzerBeepEvent(BEEP_TICKS_LONG);
    }
}

static void navStartReplay(void)
{
    if (loraNavContext.status.frameCount == 0u)
    {
        navStopMotion(LORA_NAV_ERROR);
        buzzerBeepEvent(BEEP_TICKS_LONG);
        return;
    }

    loraNavContext.status.frameIndex = 0u;
    navResetActualAccum();
    motorControlStopAll();
    /* 回放保留录制基准轮速/编码器纠偏，并由当前姿态环独占旋转修正。 */
    chassisApplicationSetYawRateEnable(1u);
    loraNavContext.yawRateLoopDividerCount = 0u;
    loraNavContext.status.state = LORA_NAV_REPLAYING;
    buzzerBeepEvent(BEEP_TICKS_SHORT);
}

static void navAccumulateEncoder(encoderDataStruct encoderData)
{
    loraNavContext.status.flActualAccum += encoderData.flCount;
    loraNavContext.status.frActualAccum += encoderData.frCount;
    loraNavContext.status.blActualAccum += encoderData.blCount;
    loraNavContext.status.brActualAccum += encoderData.brCount;
}

static void navRecordFrame(void)
{
    uint16_t frameIndex = loraNavContext.status.frameCount;
    chassisApplicationSnapshotStruct snapshot;

    if (frameIndex >= LORA_NAV_MAX_FRAMES)
    {
        navFinishRecording();
        return;
    }

    chassisApplicationGetSnapshot(&snapshot);
    loraNavContext.frames[frameIndex].baseTargetSpeed = snapshot.baseWheelSpeed;
    loraNavContext.frames[frameIndex].wz = snapshot.targetCommand.wz;
    loraNavContext.frames[frameIndex].flAccum = loraNavContext.status.flActualAccum;
    loraNavContext.frames[frameIndex].frAccum = loraNavContext.status.frActualAccum;
    loraNavContext.frames[frameIndex].blAccum = loraNavContext.status.blActualAccum;
    loraNavContext.frames[frameIndex].brAccum = loraNavContext.status.brActualAccum;
    loraNavContext.status.frameCount = frameIndex + 1u;
}

static void navApplyReplayFrame(void)
{
    uint16_t frameIndex = loraNavContext.status.frameIndex;
    const loraNavControlFrameStruct* frame;
    wheelSpeedStruct baseTargetSpeed;
    wheelSpeedStruct encoderCorrection;
    chassisMotionCommandStruct replayCommand;
    float rampScale;
    float errorLimit = loraNavContext.errorLimitPulses;
    float correctionKp = loraNavContext.correctionKp;
    float correctionLimit = loraNavContext.correctionLimit;
    float rotationProjection;

    if (frameIndex >= loraNavContext.status.frameCount)
    {
        navStopMotion(LORA_NAV_DONE);
        buzzerBeepEvent(BEEP_TICKS_LONG);
        return;
    }

    frame = &loraNavContext.frames[frameIndex];
    loraNavContext.status.flError = (float) (frame->flAccum - loraNavContext.status.flActualAccum);

```

```

loraNavContext.status.frError = (float) (frame->frAccum - loraNavContext.status.frActualAccum);
loraNavContext.status.blError = (float) (frame->blAccum - loraNavContext.status.blActualAccum);
loraNavContext.status.brError = (float) (frame->brAccum - loraNavContext.status.brActualAccum);

/* 历史 yaw 修正属于旋转 owner: 先从累计误差中剔除旋转模式, 再判错和纠偏。 */
rotationProjection = (-loraNavContext.status.flError +
    loraNavContext.status.frError - loraNavContext.status.blError +
    loraNavContext.status.brError) * 0.25f;
loraNavContext.status.flError += rotationProjection;
loraNavContext.status.frError -= rotationProjection;
loraNavContext.status.blError += rotationProjection;
loraNavContext.status.brError -= rotationProjection;

if (absf(loraNavContext.status.flError) > errorLimit ||
    absf(loraNavContext.status.frError) > errorLimit ||
    absf(loraNavContext.status.blError) > errorLimit ||
    absf(loraNavContext.status.brError) > errorLimit)
{
    navStopMotion(LORA_NAV_ERROR);
    buzzerBeepEvent(BEEP_TICKS_LONG);
    return;
}

rampScale = navRampScale(frameIndex, loraNavContext.status.frameCount);
baseTargetSpeed.fl = frame->baseTargetSpeed.fl * rampScale;
baseTargetSpeed.fr = frame->baseTargetSpeed.fr * rampScale;
baseTargetSpeed.bl = frame->baseTargetSpeed.bl * rampScale;
baseTargetSpeed.br = frame->baseTargetSpeed.br * rampScale;
encoderCorrection.fl = navClampFloat(
    correctionKp * loraNavContext.status.flError, correctionLimit);
encoderCorrection.fr = navClampFloat(
    correctionKp * loraNavContext.status.frError, correctionLimit);
encoderCorrection.bl = navClampFloat(
    correctionKp * loraNavContext.status.blError, correctionLimit);
encoderCorrection.br = navClampFloat(
    correctionKp * loraNavContext.status.brError, correctionLimit);

replayCommand.vx = 0.0f;
replayCommand.vy = 0.0f;
replayCommand.wz = frame->wz * rampScale;
chassisApplicationSetMotionCommand(replayCommand);
loraNavContext.yawRateLoopDividerCount++;
if (loraNavContext.yawRateLoopDividerCount >= YAW_RATE_LOOP_DIVIDER)
{
    loraNavContext.yawRateLoopDividerCount = 0u;
    chassisApplicationYawRateTick();
}
chassisApplicationApplyReplayWheelTarget(
    baseTargetSpeed,
    encoderCorrection,
    replayCommand.wz);
loraNavContext.status.frameIndex = frameIndex + 1u;
}

static void navApplyRemoteCommand(void)
{
    chassisMotionCommandStruct cmd;
    /* 上层已完成模式 3/模式 4 driveGate; 这里按 loraRemote owner 缓存的窄配置映射 raw 摇杆。 */
    cmd = loraRemoteGetMotionForGate(1u);
    chassisApplicationSetMotionCommand(cmd);

    /* 角速度环 6ms 节拍 (YAW_RATE_LOOP_DIVIDER) */
    loraNavContext.yawRateLoopDividerCount++;
    if (loraNavContext.yawRateLoopDividerCount >= YAW_RATE_LOOP_DIVIDER)
    {
        loraNavContext.yawRateLoopDividerCount = 0u;
        chassisApplicationYawRateTick();
    }

    chassisApplicationApplyCommandWithLimit(
        cmd,
        loraNavContext.loraVMaxMps,
        loraNavContext.loraWzMaxRadps);
}

static void navDrainPendingEdges(void)
{
    /* 链路未就绪 / 锁定 / 链路丢失 期间, 把残留的单按事件丢弃, 防止恢复后误触发 */
    (void) loraRemoteRecordEdge();
}

```



```

    (void) loraRemoteReplayEdge();
}

void loraNavControlInit(void)
{
    const loraRemoteParamViewStruct remoteParams = {
        LORA_V_MAX_MPS, LORA_WZ_MAX_RADPS, (float) LORA_STICK_DEADZONE};
    const loraNavControlParamViewStruct navParams = {
        NAV_CORRECTION_KP, NAV_CORRECTION_LIMIT, NAV_ERROR_LIMIT_PULSES,
        LORA_V_MAX_MPS, LORA_WZ_MAX_RADPS};

    loraRemoteInit();
    loraRemoteApplyParams(&remoteParams);
    loraNavControlApplyParams(&navParams);

    loraNavContext.speedLoopDividerCount = 0u;
    loraNavContext.yawRateLoopDividerCount = 0u;
    loraNavContext.loraTickDividerCount = 0u;
    loraNavContext.lockPrevUnlocked = 0u;
    loraNavContext.linkInitReady = 0u;
    loraNavContext.linkAliveTickCount = 0u;
    loraNavContext.linkLostTickCount = 0u;
    loraNavContext.status.frameCount = 0u;
    loraNavContext.status.frameIndex = 0u;
    navResetActualAccum();
    chassisApplicationSetYawRateEnable(0u);
    loraNavContext.status.state = LORA_NAV_IDLE;
    loraNavContext.status.seq = 0u;
    loraNavContext.status.valid = 1u;
    loraNavControlPublish();
}

void loraNavControlAbortSession(void)
{
    loraNavContext.status.frameCount = 0u;
    loraNavContext.status.frameIndex = 0u;
    loraNavContext.linkAliveTickCount = 0u;
    loraNavContext.linkLostTickCount = 0u;
    loraNavContext.lockPrevUnlocked = 0u;
    navResetActualAccum();
    navDrainPendingEdges();
    loraNavContext.status.state = LORA_NAV_IDLE;
    loraNavControlPublish();
}

void loraNavControlApplyParams(const loraNavControlParamViewStruct *params)
{
    if (params == 0) return;
    loraNavContext.correctionKp = params->correctionKp;
    loraNavContext.correctionLimit = params->correctionLimit;
    loraNavContext.errorLimitPulses = params->errorLimitPulses;
    loraNavContext.loraVMaxMps = params->loraVMaxMps;
    loraNavContext.loraWzMaxRadps = params->loraWzMaxRadps;
}

void loraNavControlPit500us(uint8_t standalone, uint8_t launchGate)
{
    uint8_t isUnlocked;
    uint8_t linkAlive;

    /* 1ms LoRa tick: 消费帧 + chord/edge 状态机。standalone 传给 chord 分流(模式 4 解锁 / 模式 3 阶段请求)。*/
    loraNavContext.loraTickDividerCount++;
    if (loraNavContext.loraTickDividerCount >= LORA_TICK_DIVIDER)
    {
        loraNavContext.loraTickDividerCount = 0u;
        loraRemoteTickMs(standalone);
    }

    linkAlive = loraRemotesLinkAlive();

    /* 启动初始过滤: 链路连续保持 ~1s 才允许进入正常 workflows;
    * 必须先于锁定检查执行 -- 上电默认锁定, 若 gated 在 isUnlocked 之后,
    * linkAliveTickCount 会一直停在 0, 用户解锁后才开始计 1s,
    * 期间首次 S3/S4 单按会被 navDrainPendingEdges 静默吞掉
    */
    if (!loraNavContext.linkInitReady)
    {
        if (linkAlive)
        {

```

```

    if (loraNavContext.linkAliveTickCount < LORA_INIT_VALID_TICKS)
    {
        loraNavContext.linkAliveTickCount++;
        if (loraNavContext.linkAliveTickCount >= LORA_INIT_VALID_TICKS)
        {
            loraNavContext.linkInitReady = 1u;
        }
    }
}
else
{
    loraNavContext.linkAliveTickCount = 0u;
}
loraNavContext.speedLoopDividerCount = 0u;
chassisApplicationSetYawRateEnable(0u);
motorControlStopAll();
navDrainPendingEdges();
loraNavControlPublish();
return;
}

/* 驱动门控 ('unlocked'=允许跑) 按模式分流(B6):
* 模式 4(脱机 standalone): 本地 S3+S4 解锁状态(原义);
* 模式 3(联机非 standalone): 飞机到顶门控 launchGate(=core1 下发 LAUNCH 位=飞机已发车&&到顶)——
* 飞机起飞爬高期 launchGate=0 → 车锁定不动, 到巡航高后 =1 → 车可驾驶(车禁动直到飞机到顶)。
* 边沿切换均触发长鸣; 锁定 = 急停。 */
isUnlocked = standalone ? loraRemoteIsUnlocked() : launchGate;
if (isUnlocked != loraNavContext.lockPrevUnlocked)
{
    buzzerBeepEvent(BEEP_TICKS_LONG);
    if (!isUnlocked)
    {
        navStopMotion(LORA_NAV_IDLE);
    }
    loraNavContext.lockPrevUnlocked = isUnlocked;
}

if (!isUnlocked)
{
    loraNavContext.speedLoopDividerCount = 0u;
    chassisApplicationSetYawRateEnable(0u);
    motorControlStopAll();
    navDrainPendingEdges();
    loraNavControlPublish();
    return;
}

/* 链路丢失: 强制零速, 保留录制 / 回放进度; 录制中连续断 >2s 后丢弃缓冲 */
if (!linkAlive)
{
    if (loraNavContext.linkLostTickCount < LORA_LINK_LOST_DISCARD_TICKS)
    {
        loraNavContext.linkLostTickCount++;
        if (loraNavContext.linkLostTickCount >= LORA_LINK_LOST_DISCARD_TICKS &&
            loraNavContext.status.state == LORA_NAV_RECORDING)
        {
            navStopMotion(LORA_NAV_IDLE);
            loraNavContext.status.frameCount = 0u;
            buzzerBeepEvent(BEEP_TICKS_LONG);
        }
    }
    chassisApplicationSetYawRateEnable(0u);
    chassisApplicationApplyCommandWithLimit(
        NAV_ZERO_COMMAND,
        loraNavContext.loraVMaxMps,
        loraNavContext.loraWzMaxRadps);
    loraNavContext.speedLoopDividerCount = 0u;
    motorControlStopAll();
    navDrainPendingEdges();
    loraNavControlPublish();
    return;
}
loraNavContext.linkLostTickCount = 0u;

/* S4 单按上升沿: 录制开关 (chord 抑制后才会到达此处) */
if (loraRemoteRecordEdge())
{
    if (loraNavContext.status.state == LORA_NAV_RECORDING)
    {

```

```

        navFinishRecording();
    }
    else if (loraNavContext.status.state != LORA_NAV_ERROR)
    {
        navStartRecording();
    }
    /* ERROR 为故障闭锁态：消费按键但保持停车，只有新的 owner/授权流程可退出。 */
}

/* S3 单按上升沿：回放开关 */
if (loraRemoteReplayEdge())
{
    if (loraNavContext.status.state == LORA_NAV_REPLAYING)
    {
        navStopMotion(LORA_NAV_DONE);
        buzzerBeepEvent(BEEP_TICKS_LONG);
    }
    else if (loraNavContext.status.state == LORA_NAV_READY ||
             loraNavContext.status.state == LORA_NAV_DONE)
    {
        navStartReplay();
    }
}

/* 速度环 @ 2ms */
loraNavContext.speedLoopDividerCount++;
if (loraNavContext.speedLoopDividerCount >= SPEED_LOOP_DIVIDER)
{
    chassisApplicationSnapshotStruct snapshot;

    loraNavContext.speedLoopDividerCount = 0u;
    chassisApplicationUpdateFeedback();
    chassisApplicationGetSnapshot(&snapshot);
    navAccumulateEncoder(snapshot.encoderData);

    if (loraNavContext.status.state == LORA_NAV_RECORDING)
    {
        chassisApplicationSetYawRateEnable(1u);
        navApplyRemoteCommand();
        navRecordFrame();
        motorControlSpeedTick();
    }
    else if (loraNavContext.status.state == LORA_NAV_REPLAYING)
    {
        navApplyReplayFrame();
        if (loraNavContext.status.state == LORA_NAV_REPLAYING)
        {
            motorControlSpeedTick();
        }
    }
    else if (navIsManualDriveState(loraNavContext.status.state))
    {
        chassisApplicationSetYawRateEnable(1u);
        navApplyRemoteCommand();
        motorControlSpeedTick();
    }
    else
    {
        chassisApplicationSetYawRateEnable(0u);
        motorControlStopAll();
    }
}

if (navShouldRunOutput(loraNavContext.status.state))
{
    motorControlCurrentTick();
    motorControlOutput();
}
loraNavControlPublish();
}

void loraNavControlWhile(void)
{
    /* LoRa 帧消费与 chord/edge 状态机由 PIT 内部 loraRemoteTickMs 完成,
    * 调试与 LCD 刷新由 core0Loop 顶层标志位驱动, 此处无周期前台任务
    */
}

void loraNavControlGetSnapshot(loraNavControlSnapshotStruct *out)

```

```

{
    uint8_t retry;

    if (out == 0)
    {
        return;
    }

    for (retry = 0u; retry < 3u; retry++)
    {
        uint32_t before = loraNavPublishGeneration;
        loraNavControlSnapshotStruct snapshot;

        __DMB();
        snapshot = loraNavPublished;
        __DMB();
        if (((before & 1u) == 0u) && (before == loraNavPublishGeneration))
        {
            *out = snapshot;
            return;
        }
    }

    {
        const loraNavControlSnapshotStruct empty = {state = LORA_NAV_IDLE};
        *out = empty;
    }
}

static void loraNavControlPublish(void)
{
    loraNavControlSnapshotStruct snapshot = loraNavContext.status;

    snapshot.seq++;
    snapshot.valid = 1u;
    loraNavContext.status.seq = snapshot.seq;
    loraNavContext.status.valid = snapshot.valid;
    loraNavPublishGeneration++;
    __DMB();
    loraNavPublished = snapshot;
    __DMB();
    loraNavPublishGeneration++;
}

```

core2_cyt4bb7/project/code/comm/loraNavControl.h

SHA-256

:

5c2d4635b5ba0450e1250a16bf422104ad64ae89fee310790cccf7c081920116

代码 (c)

```

#ifndef _LORA_NAV_CONTROL_H_
#define _LORA_NAV_CONTROL_H_

#include <stdint.h>

#define LORA_NAV_MAX_FRAMES (2000u)

typedef struct
{
    float correctionKp;
    float correctionLimit;
    float errorLimitPulses;
    float loraVMaxMps;
    float loraWzMaxRadps;
} loraNavControlParamViewStruct;

typedef enum
{
    LORA_NAV_IDLE = 0u,
    LORA_NAV_RECORDING,
    LORA_NAV_READY,
    LORA_NAV_REPLAYING,
    LORA_NAV_DONE,
    LORA_NAV_ERROR
}

```

```

} loraNavControlStateEnum;

typedef struct
{
    loraNavControlStateEnum state;
    uint16_t frameCount;
    uint16_t frameIndex;
    int32_t flActualAccum;
    int32_t frActualAccum;
    int32_t blActualAccum;
    int32_t brActualAccum;
    float flError;
    float frError;
    float blError;
    float brError;
    uint32_t seq;
    uint8_t valid;
} loraNavControlSnapshotStruct;

void loraNavControlInit(void);
void loraNavControlApplyParams(const loraNavControlParamViewStruct *params);
void loraNavControlAbortSession(void);
/* standalone: 1=模式4 脱机(本地 S3+S4 解锁门控), 0=模式3 联机(飞机到顶 launchGate 门控车驾驶, 车禁动直到飞机到顶)。
 * launchGate: 模式3 时=core1 下发 RS-485 LAUNCH 位(飞机已发车&&到顶); 模式4 时忽略。 */
void loraNavControlPit500us(uint8_t standalone, uint8_t launchGate);
void loraNavControlWhile(void);
void loraNavControlGetSnapshot(loraNavControlSnapshotStruct *out);

#endif

```

core2_cyt4bb7/project/code/comm/loraRemote.c

SHA-256

:

f5d89d6df3cbdb6072fb1176d637755cb7315211aec93291da8bf9b4fd7a665a

代码 (c)

```

#include "loraRemote.h"

#include "configChassisRemote.h"
#include "configCore2Build.h" /* CORE2_BUILD_MODE */

/*
 * LoRa3A22 双摇杆遥控适配层。双实现按「核 + 固件」选择 (IAR 预定义 CY_CORE_CM7_0/CM7_1) :
 *
 * [cm_7_0 控制核 + 生产固件] → 镜像实现: 不拥有 UART1。LoRa 收发/解码/chord 状态机已搬到
 *   cm_7_1 通信核, 本核仅通过 coreDataTransfer 读 cm_7_1 发布的 LoRa 解码快照。getter API 不变,
 *   由 whileProcess 调 loraRemoteMirrorApply 刷新; 单按边沿经 recordSeq/replaySeq 单调计数交付。
 *
 * [cm_7_1 通信核 (任何固件) / cm_7_0 调参固件] → 真实 UART1 实现: cm_7_1 中断驱动 RX
 *   (ISR 填 SeekFree 软件缓冲) + 前台 loraRemotePoll 排空解析 + 1ms tick 维护超时/chord/edge。
 */

/* 摇杆原始值 -> 物理量。
 * 死区内强制 0; 死区外把 [DEADZONE, RANGE] 线性映射到 [0, MAX]; 超出 RANGE 钳位。 */
static float loraRemoteStickToFloat(int16_t raw, float maxOutput, float deadzone)
{
    float magnitude;
    float direction;
    float scale;
    float result;

    if (maxOutput <= 0.0f) return 0.0f;
    if (deadzone < 0.0f) deadzone = 0.0f;
    if (deadzone >= (float) LORA_STICK_RANGE) return 0.0f;

    if (raw >= 0)
    {
        direction = 1.0f;
        magnitude = (float) raw;
    }
    else
    {
        direction = -1.0f;
        magnitude = (float) -(int32_t) raw;
    }
}

```

```

    }

    if (magnitude <= deadzone)
        return 0.0f;
    if (magnitude >= (float) LORA_STICK_RANGE)
        return (direction > 0.0f) ? maxOutput : -maxOutput;

    scale = (magnitude - deadzone) / ((float) LORA_STICK_RANGE - deadzone);
    result = scale * maxOutput;
    return (direction > 0.0f) ? result : -result;
}

chassisMotionCommandStruct loraRemoteMapRawMotion(
    loraRemoteRawMotionStruct raw,
    float vMaxMps,
    float wzMaxRadps,
    float deadzone)
{
    chassisMotionCommandStruct command;
    command.vx = LORA_VX_INPUT_DIR * loraRemoteStickToFloat(raw.vxRaw, vMaxMps, deadzone);
    command.vy = LORA_VY_INPUT_DIR * loraRemoteStickToFloat(raw.vyRaw, vMaxMps, deadzone);
    command.wz = LORA_WZ_INPUT_DIR * loraRemoteStickToFloat(raw.wzRaw, wzMaxRadps, deadzone);
    return command;
}

#if defined(CY_CORE_CM7_0) && (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)

/* ===== cm_7_0 生产态：共享内存镜像实现 ===== */
#include "coreDataTransfer.h"
#include "zf_common_interrupt.h"

/* volatile：whileProcess 临界区写、500μs ISR(loraRemoteGetMotion) 临界区读。临界区已是编译屏障，
 * volatile 再保证逐字节真实内存访问（60A 电机安全裕度，与 chassisLink linkMirror 同处理）。*/
static volatile loraRemoteRawMotionStruct loraRawMirror = {0, 0, 0};
static volatile uint8_t loraUnlockedMirror = 0u;
static volatile uint8_t loraLinkAliveMirror = 0u;
static volatile uint8_t loraTakeoffReqMirror = 0u;
static volatile uint8_t loraRemoteRecordEdgePending = 0u; /* 前台 apply 置，500μs ISR 读-清 */
static volatile uint8_t loraRemoteReplayEdgePending = 0u;
static uint32_t loraLastRecordSeq = 0u; /* 仅前台 apply 访问 */
static uint32_t loraLastReplaySeq = 0u;
static uint8_t loraMirrorInitd = 0u; /* 首次 apply 同步 seq 基线，防 boot 伪边沿 */
static float loraVMaxMps = LORA_V_MAX_MPS;
static float loraWzMaxRadps = LORA_WZ_MAX_RADPS;
static float loraStickDeadzone = (float) LORA_STICK_DEADZONE;

static void loraRemoteMirrorClearUnsafe(void)
{
    {
        loraRawMirror.vxRaw = 0;
        loraRawMirror.vyRaw = 0;
        loraRawMirror.wzRaw = 0;
        loraUnlockedMirror = 0u;
        loraLinkAliveMirror = 0u;
        loraTakeoffReqMirror = 0u;
        loraRemoteRecordEdgePending = 0u;
        loraRemoteReplayEdgePending = 0u;
    }
}

void loraRemoteInit(void)
{
    {
        loraRemoteMirrorClearUnsafe();
        loraLastRecordSeq = 0u;
        loraLastReplaySeq = 0u;
        loraMirrorInitd = 0u;
    }
}

void loraRemoteApplyParams(const loraRemoteParamViewStruct *params)
{
    {
        if (params == 0) return;
        loraVMaxMps = params->vMaxMps;
        loraWzMaxRadps = params->wzMaxRadps;
        loraStickDeadzone = params->stickDeadzone;
    }
}

void loraRemoteMirrorInvalidate(void)
{
    {
        uint32_t primask = __get_PRIMASK();
        __disable_irq();
        loraRemoteMirrorClearUnsafe();
    }
}

```

```

    __set_PRIMASK(primask);
    loraMirrorInitd = 0u;
}

/* whileProcess 调：把 cm_7_1 发布的 LoRa 解码快照刷新到本地镜像。仅 magic 有效时调用。 */
void loraRemoteMirrorApply(const commToControlStruct* s)
{
    uint32_t primask;
    if (0 == s) return;

    /* PRIMASK 存-恢复，绝不用 SDK interrupt_global_disable/enable（坏计数器，ISR 高频用会上漂致中断
    * 永久关闭整机卡死；详见 chassisLink.c 镜像段注释）。 */
    primask = __get_PRIMASK();
    __disable_irq();
    loraRawMirror.vxRaw = s->loraVxRaw;
    loraRawMirror.vyRaw = s->loraVyRaw;
    loraRawMirror.wzRaw = s->loraWzRaw;
    loraUnlockedMirror = s->loraUnlocked;
    loraLinkAliveMirror = s->loraLinkAlive;
    loraTakeoffReqMirror = s->takeoffReq;
    __set_PRIMASK(primask);

    /* 单按边沿：seq 正增量触发一次。首次 apply 同步基线（不触发），防 cm_7_1 早启时 boot 伪边沿。 */
    if (!loraMirrorInitd)
    {
        loraLastRecordSeq = s->recordSeq;
        loraLastReplaySeq = s->replaySeq;
        loraMirrorInitd = 1u;
    }
    else
    {
        if (s->recordSeq != loraLastRecordSeq)
        {
            loraRemoteRecordEdgePending = 1u;
            loraLastRecordSeq = s->recordSeq;
        }
        if (s->replaySeq != loraLastReplaySeq)
        {
            loraRemoteReplayEdgePending = 1u;
            loraLastReplaySeq = s->replaySeq;
        }
    }
}

chassisMotionCommandStruct loraRemoteGetMotion(void)
{
    return loraRemoteGetMotionForGate(loraUnlockedMirror);
}

chassisMotionCommandStruct loraRemoteGetMotionForGate(uint8_t driveGate)
{
    chassisMotionCommandStruct c;
    loraRemoteRawMotionStruct raw;
    c.vx = 0.0f;
    c.vy = 0.0f;
    c.wz = 0.0f;
    if (!driveGate || !loraLinkAliveMirror) return c;
    raw = loraRemoteGetRawMotion();
    c = loraRemoteMapRawMotion(
        raw,
        loraVMaxMps,
        loraWzMaxRadps,
        loraStickDeadzone);
    return c;
}

uint8_t loraRemoteIsLinkAlive(void)
{
    return loraLinkAliveMirror;
}

uint8_t loraRemoteIsUnlocked(void)
{
    return loraUnlockedMirror;
}

uint8_t loraRemoteGetRemoteTakeoffReq(void)
{
    return loraTakeoffReqMirror;
}

```

```

loraRemoteRawMotionStruct loraRemoteGetRawMotion(void)
{
    loraRemoteRawMotionStruct raw;
    uint32_t primask = __get_PRIMASK(); /* 绝不用 SDK interrupt_global * (坏计数器, ISR 高频致卡死) */
    __disable_irq();
    raw.vxRaw = loraRawMirror.vxRaw;
    raw.vyRaw = loraRawMirror.vyRaw;
    raw.wzRaw = loraRawMirror.wzRaw;
    __set_PRIMASK(primask);
    return raw;
}

/* cm_7_0 生产态不拥有 UART1: 以下为 API 完整性的空实现 (不应被本核调用)。 */
void loraRemotePoll(void)
{
}

void loraRemoteTickMs(uint8_t standalone)
{
    (void) standalone;
}

void loraRemoteGetSnapshot(loraRemoteSnapshotStruct *out)
{
    uint8_t i;
    uint32_t primask;

    if (0 == out) return;
    for (i = 0u; i < 4u; i++)
    {
        out->joystick[i] = 0;
        out->key[i] = 0u;
        out->switchKey[i] = 0u;
    }

    primask = __get_PRIMASK();
    __disable_irq();
    out->joystick[0] = loraRawMirror.vyRaw;
    out->joystick[1] = loraRawMirror.vxRaw;
    out->joystick[2] = loraRawMirror.wzRaw;
    out->frameSeq = 0u;
    out->linkAlive = loraLinkAliveMirror;
    out->unlocked = loraUnlockedMirror;
    out->takeoffReq = loraTakeoffReqMirror;
    __set_PRIMASK(primask);
}

#else /* defined(CY_CORE_CM7_1) || (CORE2_BUILD_MODE == CORE2_BUILD_TUNE) */

/* ===== 真实 UART1 实现 ===== */

#include <string.h>

#include "zf_device_lora3a22.h"

#include "zf_common_interrupt.h"

static volatile uint8_t loraRemoteIsUnlockedFlag = 0u;
static volatile uint8_t loraRemoteRecordEdgePending = 0u;
static volatile uint8_t loraRemoteReplayEdgePending = 0u;

static volatile uint8_t loraRemotePrevS3 = 0u;
static volatile uint8_t loraRemotePrevS4 = 0u;
static volatile uint8_t loraRemoteChordArmed = 0u;
static volatile uint8_t loraRemoteKeyFramePrimed = 0u;
static volatile uint8_t loraRemoteModePrimed = 0u;
static volatile uint8_t loraRemoteLastStandalone = 0u;
static volatile uint8_t loraRemoteChordFrameCount = 0u;
static volatile uint8_t loraRemoteReleaseFrameCount = 0u;
static volatile uint32_t loraRemoteLastKeyFrameSeq = 0u;
static volatile uint32_t loraRemoteKeyTickMs = 0u;
static volatile uint32_t loraRemoteChordStartMs = 0u;
static volatile uint32_t loraRemoteReleaseStartMs = 0u;

static volatile uint16_t loraRemoteSilentMs = 0u;

/* 联机双肩键请求保持倒计时(1ms/tick): 覆盖至少三个 200ms STATUS 发布机会, 降低单帧
 * 丢失/调度相位导致模式 2 人工缓降未送达的概率; 归零后由低电平重新武装下一次请求。 */
#define LORA_TAKEOFF_REQ_HOLD_TICKS (700u)
static volatile uint16_t loraRemoteTakeoffReqTicks = 0u;

```



```

/* 前台轮询接收快照（cm_7_1 中断驱动 RX 填 SeekFree 软件缓冲，loraRemotePoll 前台逐字节解析后
 * 整块发布到 loraSnapshot；PIT 侧 Tick/GetMotion 临界区整块读，消除「前台写 18 字节 vs PIT 读」撕裂）。
 * loraParseBuf/Len 为前台解析状态，仅 loraRemotePoll 单上下文访问。 */
static volatile lora3a22_uart_transfer_dat_struct loraSnapshot;
static volatile uint32_t loraRemoteFrameSeq = 0u;
#if !(defined(CY_CORE_CM7_0) && (CORE2_BUILD_MODE == CORE2_BUILD_TUNE) \
    && (MOTOR_TUNE_MODE == MOTOR_TUNE_LORA_TEST))
static uint8_t loraParseBuf[LORA3A22_DATA_LEN];
static uint8_t loraParseLen = 0u;
#endif

void loraRemoteInit(void)
{
    lora3a22_init();

    /* 驱动静态初始化把 state_flag 置 1, 这里强制为 0
     * 避免上电瞬间还未收到第一帧就被当作链路存活
     */
    lora3a22_state_flag = 0u;
    lora3a22_finsh_flag = 0u;
    lora3a22_response_time = 0u;

    loraRemoteIsUnlockedFlag = 0u;
    loraRemoteRecordEdgePending = 0u;
    loraRemoteReplayEdgePending = 0u;
    loraRemotePrevS3 = 0u;
    loraRemotePrevS4 = 0u;
    loraRemoteChordArmed = 0u;
    loraRemoteKeyFramePrimed = 0u;
    loraRemoteModePrimed = 0u;
    loraRemoteLastStandalone = 0u;
    loraRemoteChordFrameCount = 0u;
    loraRemoteReleaseFrameCount = 0u;
    loraRemoteLastKeyFrameSeq = 0u;
    loraRemoteKeyTickMs = 0u;
    loraRemoteChordStartMs = 0u;
    loraRemoteReleaseStartMs = 0u;
    loraRemoteSilentMs = 0u;
    loraRemoteTakeoffReqTicks = 0u;
    loraRemoteFrameSeq = 0u;

    /* 中断驱动 RX：本实现运行在 cm_7_1 通信核（控制环在 cm_7_0，与此 UART 中断完全隔离），
     * 故重新启用 UART1 RX 中断——SeekFree uart_query_byte 依赖 RX ISR 填软件缓冲，否则
     * loraRemotePoll 永远排空不到字节。lora3a22_init 已注册轻量缓冲填充 ISR；前台 loraRemotePoll
     * 排空 + 解析，重活不在 ISR 内。RX 悬空/噪声的中断风暴只拖慢空闲的 cm_7_1，不影响 cm_7_0 500µs PIT。 */
    uart_rx_interrupt(LORA3A22_UART_INDEX, 1u);
#if !(defined(CY_CORE_CM7_0) && (CORE2_BUILD_MODE == CORE2_BUILD_TUNE) \
    && (MOTOR_TUNE_MODE == MOTOR_TUNE_LORA_TEST))
    loraParseLen = 0u;
#endif
    memset((void *) &loraSnapshot, 0, sizeof(loraSnapshot));
}

void loraRemoteApplyParams(const loraRemoteParamViewStruct *params)
{
    (void) params;
}

/* 前台轮询排空 UART1 LoRa FIFO + 逐字节解析（复刻 lora3a22_uart_callback 的起始位/长度/和校验，
 * 但在前台执行、用 uart_query_byte 返回值控制 byte budget）。解析成功整块发布到 loraSnapshot
 * （关中断临界区）+ 置 state/finsh 标志（驱动全局 vuint8，供 loraRemoteTickMs 维护超时）。 */
void loraRemotePoll(void)
{
    #if defined(CY_CORE_CM7_0) && (CORE2_BUILD_MODE == CORE2_BUILD_TUNE) \
    && (MOTOR_TUNE_MODE == MOTOR_TUNE_LORA_TEST)
        uint32_t primask;

    /* cm_7_0 LoRa 调试固件沿用既有 SDK ISR 解析路径；前台 owner 只接收完整帧快照，
     * 不改变 ISR、UART FIFO 或驱动回调所有权。 */
    primask = __get_PRIMASK();
    __disable_irq();
    if (lora3a22_finsh_flag != 0u)
    {
        lora3a22_finsh_flag = 0u;
        memcpy((void *) &loraSnapshot, &lora3a22_uart_transfer, sizeof(loraSnapshot));
        loraRemoteFrameSeq++;
    }
    __set_PRIMASK(primask);

```

```

#else
uint8_t budget = LORA_POLL_BYTE_BUDGET;
uint8_t byte;

while ((budget > 0u) && uart_query_byte(LORA3A22_UART_INDEX, &byte))
{
    budget--;
    loraParseBuf[loraParseLen] = byte;
    loraParseLen++;

    if ((loraParseLen == 1u) && (loraParseBuf[0] != LORA3A22_FRAME_STAR))
    {
        loraParseLen = 0u; /* 起始位不符，丢弃重新对齐 */
        continue;
    }
    if (loraParseLen >= (uint8_t) LORA3A22_DATA_LEN)
    {
        uint8_t parity = loraParseBuf[1];
        uint8_t sum = 0u;
        uint8_t i;
        loraParseBuf[1] = 0u; /* 校验位本身不计入和（与驱动 callback 一致） */
        for (i = 0u; i < (uint8_t) LORA3A22_DATA_LEN; i++)
        {
            sum = (uint8_t) (sum + loraParseBuf[i]);
        }
        if (sum == parity)
        {
            /* PRIMASK 存-恢复，绝不用 SDK interrupt_global_disable/enable（坏计数器：前台 Poll 与
            * PIT 读高频交错令 interrupt_global_flag 上漂致中断永久关闭整机卡死；详见 chassisLink.c 注释）。 */
            uint32_t primask;
            loraParseBuf[1] = parity;
            primask = __get_PRIMASK();
            __disable_irq();
            memcpy((void *) &loraSnapshot, loraParseBuf, sizeof(loraSnapshot));
            loraRemoteFrameSeq++;
            __set_PRIMASK(primask);
            lora3a22_finsh_flag = 1u;
            lora3a22_state_flag = 1u;
        }
        loraParseLen = 0u;
    }
}
#endif
}

void loraRemoteTickMs(uint8_t standalone)
{
    uint8_t curS3;
    uint8_t curS4;
    uint32_t frameSeq;
    lora3a22_uart_transfer_dat_struct snap;

    loraRemoteKeyTickMs++;

    /* 双肩键请求保持倒计时逐拍递减(链路丢失时也衰减，自然落沿)。 */
    if (loraRemoteTakeoffReqTicks > 0u)
    {
        loraRemoteTakeoffReqTicks--;
    }

    /* (1) 链路超时维护: 每收到一帧合法数据, 驱动会把 finsh_flag 与 state_flag
    * 同时置 1, 我们消费 finsh_flag 来重置静默计时;
    * 静默 >= 500ms 则强制 state_flag = 0
    */
    if (lora3a22_finsh_flag != 0u)
    {
        lora3a22_finsh_flag = 0u;
        loraRemoteSilentMs = 0u;
    }
    else if (loraRemoteSilentMs < LORA_LINK_TIMEOUT_MS)
    {
        loraRemoteSilentMs++;
        if (loraRemoteSilentMs >= LORA_LINK_TIMEOUT_MS)
        {
            lora3a22_state_flag = 0u;
        }
    }
}

/* (2) 链路丢失: 强制 chord 抑制 + 清待消费边沿

```

```

* 防止用户在断键时按住的键在恢复链路时产生伪边沿
*/
if (lora3a22_state_flag == 0u)
{
    loraRemoteIsUnlockedFlag = 0u;
    loraRemoteTakeoffReqTicks = 0u;
    loraRemotePrevS3 = 0u;
    loraRemotePrevS4 = 0u;
    loraRemoteChordArmed = 0u;
    loraRemoteKeyFramePrimed = 0u;
    loraRemoteModePrimed = 0u;
    loraRemoteChordFrameCount = 0u;
    loraRemoteReleaseFrameCount = 0u;
    loraRemoteRecordEdgePending = 0u;
    loraRemoteReplayEdgePending = 0u;
    return;
}

/* (3) chord / 单按边沿状态机：快照与发布序号同一临界区读取。只有真实新 LoRa 帧
* 才能推进按键状态，禁止 1ms PIT 把一帧异常 key 值放大成数十个“连续样本”。 */
{
    uint32_t primask = __get_PRIMASK(); /* 绝不用 SDK interrupt_global_* (坏计数器，详见 chassisLink.c 注释) */
    __disable_irq();
    memcpy(&snap, (const void *) &loraSnapshot, sizeof(snap));
    frameSeq = loraRemoteFrameSeq;
    __set_PRIMASK(primask);
}
if ((loraRemoteKeyFramePrimed != 0u) &&
    (frameSeq == loraRemoteLastKeyFrameSeq)) return;
loraRemoteLastKeyFrameSeq = frameSeq;
loraRemoteKeyFramePrimed = 1u;

curS3 = (snap.key[LORA_KEY_S3_INDEX] != 0u) ? 1u : 0u;
curS4 = (snap.key[LORA_KEY_S4_INDEX] != 0u) ? 1u : 0u;

/* standalone 改变会改变组合键业务含义。模式边沿必须撤销旧候选，并重新经过全松武装。 */
if ((loraRemoteModePrimed == 0u) ||
    (loraRemoteLastStandalone != (standalone ? 1u : 0u)))
{
    loraRemoteModePrimed = 1u;
    loraRemoteLastStandalone = standalone ? 1u : 0u;
    loraRemoteChordArmed = 0u;
    loraRemoteChordFrameCount = 0u;
    loraRemoteReleaseFrameCount = 0u;
    loraRemotePrevS3 = curS3;
    loraRemotePrevS4 = curS4;
}

if (curS3 && curS4)
{
    uint32_t confirmMs = standalone ?
        (uint32_t) LORA_CHORD_DEBOUNCE_MS :
        (uint32_t) LORA_LINKED_CHORD_CONFIRM_MS;
    loraRemoteReleaseFrameCount = 0u;
    if (loraRemoteChordArmed != 0u)
    {
        {
            if (loraRemoteChordFrameCount == 0u)
            {
                loraRemoteChordStartMs = loraRemoteKeyTickMs;
                loraRemoteChordFrameCount = 1u;
            }
            else if (loraRemoteChordFrameCount < 0xFFu)
            {
                loraRemoteChordFrameCount++;
            }
            if ((loraRemoteChordFrameCount >= LORA_CHORD_MIN_FRESH_FRAMES) &&
                ((uint32_t) (loraRemoteKeyTickMs - loraRemoteChordStartMs) >= confirmMs))
            {
                if (standalone)
                {
                    {
                        loraRemoteIsUnlockedFlag ^= 1u;
                    }
                    else
                    {
                        {
                            loraRemoteTakeoffReqTicks = LORA_TAKEOFF_REQ_HOLD_TICKS;
                        }
                        loraRemoteChordArmed = 0u;
                        loraRemoteChordFrameCount = 0u;
                        loraRemoteRecordEdgePending = 0u;
                    }
                }
            }
        }
    }
}

```

```

        loraRemoteReplayEdgePending = 0u;
    }
}
else if (!curS3 && !curS4)
{
    /* 上电、重连、模式边沿或一次组合键之后，必须由真实新帧证明全松，
    * 才重装下一次组合键资格。 */
    loraRemoteChordFrameCount = 0u;
    if (loraRemoteChordArmed == 0u)
    {
        if (loraRemoteReleaseFrameCount == 0u)
        {
            loraRemoteReleaseStartMs = loraRemoteKeyTickMs;
            loraRemoteReleaseFrameCount = 1u;
        }
        else if (loraRemoteReleaseFrameCount < 0xFFu)
        {
            loraRemoteReleaseFrameCount++;
        }
        if ((loraRemoteReleaseFrameCount >= LORA_CHORD_MIN_FRESH_FRAMES) &&
            ((uint32_t) (loraRemoteKeyTickMs - loraRemoteReleaseStartMs) >=
             (uint32_t) LORA_CHORD_RELEASE_ARM_MS))
        {
            loraRemoteChordArmed = 1u;
        }
    }
}
else
{
    /* 任一单键打断双键候选；只有已完成全松武装时才产生单按边沿。 */
    loraRemoteChordFrameCount = 0u;
    loraRemoteReleaseFrameCount = 0u;
    if ((loraRemoteChordArmed != 0u) && curS4 && !loraRemotePrevS4 && !curS3)
    {
        loraRemoteRecordEdgePending = 1u;
    }
    if ((loraRemoteChordArmed != 0u) && curS3 && !loraRemotePrevS3 && !curS4)
    {
        loraRemoteReplayEdgePending = 1u;
    }
}

loraRemotePrevS3 = curS3;
loraRemotePrevS4 = curS4;
}

uint8_t loraRemoteIsLinkAlive(void)
{
    return (lora3a22_state_flag != 0u) ? 1u : 0u;
}

uint8_t loraRemoteIsUnlocked(void)
{
    return loraRemoteIsUnlockedFlag;
}

/* 联机双肩键请求电平(保持倒计时未归零即 1)。
* 模式 2 由 TELEMETRY status bit4 上报为调试缓降；模式 3 由 MSG_LORA_MOTION flags bit0 上报。 */
uint8_t loraRemoteGetRemoteTakeoffReq(void)
{
    return (loraRemoteTakeoffReqTicks > 0u) ? 1u : 0u;
}

loraRemoteRawMotionStruct loraRemoteGetRawMotion(void)
{
    loraRemoteRawMotionStruct raw = {0, 0, 0};
    lora3a22_uart_transfer_dat_struct snap;
    uint32_t primask;

    /* loraSnapshot 临界区整块快照（前台 poll 写，本 PIT 读，防撕裂）。
    * PRIMASK 存-恢复，绝不用 SDK interrupt_global_*（坏计数器，详见 chassisLink.c 注释）。 */
    primask = __get_PRIMASK();
    __disable_irq();
    memcpy(&snap, (const void *) &loraSnapshot, sizeof(snap));
    __set_PRIMASK(primask);

    raw.vxRaw = snap.joystick[1];
    raw.vyRaw = snap.joystick[0];

```

```

        raw.wzRaw = snap.joystick[2];
        return raw;
    }

chassisMotionCommandStruct loraRemoteGetMotionForGate(uint8_t driveGate)
{
    chassisMotionCommandStruct command = {0.0f, 0.0f, 0.0f};
    loraRemoteRawMotionStruct raw;

    if (lora3a22_state_flag == 0u || driveGate == 0u)
        return command;

    raw = loraRemoteGetRawMotion();
    return loraRemoteMapRawMotion(raw, LORA_V_MAX_MPS, LORA_WZ_MAX_RADPS, (float) LORA_STICK_DEADZONE);
}

chassisMotionCommandStruct loraRemoteGetMotion(void)
{
    return loraRemoteGetMotionForGate(loraRemoteIsUnlockedFlag);
}

void loraRemoteGetSnapshot(loraRemoteSnapshotStruct *out)
{
    lora3a22_uart_transfer_dat_struct snap;
    uint32_t frameSeq;
    uint8_t linkAlive;
    uint8_t unlocked;
    uint8_t takeoffReq;
    uint32_t primask;
    uint8_t i;

    if (0 == out) return;

    primask = __get_PRIMASK();
    __disable_irq();
    memcpy(&snap, (const void *) &loraSnapshot, sizeof(snap));
    frameSeq = loraRemoteFrameSeq;
    linkAlive = (lora3a22_state_flag != 0u) ? 1u : 0u;
    unlocked = loraRemoteIsUnlockedFlag;
    takeoffReq = (loraRemoteTakeoffReqTicks > 0u) ? 1u : 0u;
    __set_PRIMASK(primask);

    for (i = 0u; i < 4u; i++)
    {
        out->joystick[i] = snap.joystick[i];
        out->key[i] = snap.key[i];
        out->switchKey[i] = snap.switch_key[i];
    }
    out->frameSeq = frameSeq;
    out->linkAlive = linkAlive;
    out->unlocked = unlocked;
    out->takeoffReq = takeoffReq;
}

#endif /* gate */

/* 两种编译分支各自拥有 volatile pending 状态；消费语义统一为单次读清。 */
uint8_t loraRemoteRecordEdge(void)
{
    uint8_t edge = loraRemoteRecordEdgePending;
    loraRemoteRecordEdgePending = 0u;
    return edge;
}

uint8_t loraRemoteReplayEdge(void)
{
    uint8_t edge = loraRemoteReplayEdgePending;
    loraRemoteReplayEdgePending = 0u;
    return edge;
}

```

core2_cyt4bb7/project/code/comm/loraRemote.h

SHA-256

:

0cc4be9964b656741803c8964cc408d15c673c9f5c21f5cab03f198a2af7582a

代码 (c)

```
#ifndef _LORA_REMOTE_H_
#define _LORA_REMOTE_H_

#include <stdint.h>

#include "chassisApplication.h"

/* LoRa3A22 双摇杆遥控适配层
 *
 * 摇杆映射(原始 int16, 中位 0):
 * joystick[1] (左摇杆上下) -> vx (前后)
 * joystick[0] (左摇杆左右) -> vy (左右)
 * joystick[2] (右摇杆左右) -> wz (CCW+/CW-)
 *
 * 按键语义:
 * key[2] = S4 (左肩) 单按 -> 录制开关
 * key[3] = S3 (右肩) 单按 -> 回放开关
 * S3 + S4 同时按下 -> 按 standalone 分流(B6): 脱机(模式 4)=切换 解锁<->锁定;
 * 联机(模式 2/3)=发送双肩键请求, 上层按系统模式解释
 *
 * 安全约束:
 * - 链路丢失 (>500ms 无新帧): GetMotion 强制 (0,0,0), 并清除解锁与起飞请求
 * - 锁定: GetMotion 强制 (0,0,0), 由调用方负责清 PID 状态与停车
 * - chord 抑制: S3+S4 触发 chord 后, 必须先松开两键才能再产生单按事件
 */
typedef struct
{
    int16_t vxRaw; /* joystick[1]: 左摇杆上下 */
    int16_t vyRaw; /* joystick[0]: 左摇杆左右, 右推为 +vy */
    int16_t wzRaw; /* joystick[2]: 右摇杆左右 */
} loraRemoteRawMotionStruct;

/* 按值发布的业务快照, 不暴露逐飞驱动类型或 owner 内部对象地址。
 * frameSeq 仅统计本核真实 UART 实现成功接收的帧; cm_7_0 生产镜像不传输该事实, 保持 0。 */
typedef struct
{
    int16_t joystick[4];
    uint8_t key[4];
    uint8_t switchKey[4];
    uint32_t frameSeq;
    uint8_t linkAlive;
    uint8_t unlocked;
    uint8_t takeoffReq;
} loraRemoteSnapshotStruct;

typedef struct
{
    float vMaxMps;
    float wzMaxRadps;
    float stickDeadzone;
} loraRemoteParamViewStruct;

chassisMotionCommandStruct loraRemoteMapRawMotion(
    loraRemoteRawMotionStruct raw,
    float vMaxMps,
    float wzMaxRadps,
    float deadzone);

/* 调用一次, 完成本模块状态复位; 真实 UART 实现初始化 UART1 并启用 RX 中断。 */
void loraRemoteInit(void);
void loraRemoteApplyParams(const loraRemoteParamViewStruct *params);

/* 前台轮询排空 UART1 LoRa FIFO + 逐字节解析。每轮 whileProcess 调一次, 常驻。
 * 实现按核分流: cm_7_1 通信核中断驱动 RX (ISR 填 SeekFree 软件缓冲, 本函数前台排空解析);
 * cm_7_0 生产固件=镜像空实现 (LoRa 收发已在 cm_7_1)。详见 .claude/rules/core2-safety.md。 */
void loraRemotePoll(void);

/* 必须每 1ms 调用一次(典型由 PIT_CH0 500us /2 分频驱动)
 * 维护链路超时计数与 chord/edge 状态机。standalone: 1=模式 4 脱机(S3+S4 切换解锁),
 * 0=联机模式 2/3(S3+S4 发送双肩键请求, 不锁车)。
 */
void loraRemoteTickMs(uint8_t standalone);

/* 联机双肩键请求电平(保持约 700ms, 覆盖多次 STATUS 发布); 模式 2 走 TELEMETRY status bit4 缓降,
 * 模式 3 走 LORA_MOTION flags bit0 起飞/缓降。接收端每个高电平只消费一次, 低电平重新武装。 */
uint8_t loraRemoteGetRemoteTakeoffReq(void);
```

```
/* 链路是否存活 */
uint8_t loraRemotelIsLinkAlive(void);

/* 当前是否解锁 (chord 切换的内部状态) */
uint8_t loraRemotelIsUnlocked(void);

/* 取摇杆解出的运动指令; 锁定或链路丢失返回 (0,0,0) */
chassisMotionCommandStruct loraRemoteGetMotion(void);
chassisMotionCommandStruct loraRemoteGetMotionForGate(uint8_t driveGate);
loraRemoteRawMotionStruct loraRemoteGetRawMotion(void);

/* S4 单按上升沿(一次性消费, 读后自动清零); chord 抑制期间不触发 */
uint8_t loraRemoteRecordEdge(void);

/* S3 单按上升沿(一次性消费, 读后自动清零); chord 抑制期间不触发 */
uint8_t loraRemoteReplayEdge(void);

/* 一次复制同拍业务事实; out 为空时无操作。 */
void loraRemoteGetSnapshot(loraRemoteSnapshotStruct *out);

#include "configCore2Build.h" /* CORE2_BUILD_MODE */
#if defined(CY_CORE_CM7_0) && (CORE2_BUILD_MODE == CORE2_BUILD_PRODUCTION)
/* cm_7_0 生产态镜像实现专用: whileProcess 把 cm_7_1 经共享内存发布的 LoRa 解码快照刷新到本地镜像。
 * 仅在 commToControl.magic 有效时调用。单按边沿经 recordSeq/replaySeq 在 apply 内做正增量边沿检测。 */
#include "coreDataTransfer.h"
void loraRemoteMirrorApply(const commToControlStruct* s);
void loraRemoteMirrorInvalidate(void);
#endif

#endif
```

core2_cyt4bb7/project/code/comm/paramSyncMailbox.c

SHA-256 :
774a74f34cc5eaab467a692cd9faa2c10c8c111750dc3e38cd1b56c2694384ea

代码 (c)

```
#include "paramSyncMailbox.h"

#include "cy_device_headers.h"

#include <string.h>

#define PARAM_SYNC_REQUEST_MAGIC (0x50535251u) /* PSRQ */
#define PARAM_SYNC_RESPONSE_MAGIC (0x50535253u) /* PSRS */
#define PARAM_SYNC_MAILBOX_ABI (1u)

typedef struct
{
    uint32_t beginSequence;
    uint32_t magic;
    uint32_t abiVersion;
    uint32_t payloadBytes;
    uint32_t producerBootGeneration;
    uint32_t sequence;
    uint32_t transactionId;
    uint32_t offset;
    uint32_t payloadCrc32;
    uint16_t dataLength;
    uint8_t op;
    uint8_t slot;
    uint8_t target;
    uint8_t state;
    uint8_t reservedHeader[6];
    uint8_t data[PARAM_SYNC_MAILBOX_DATA_MAX];
    uint8_t reservedTail[12];
    uint32_t endSequence;
} paramSyncMailboxSharedFrame;

typedef char paramSyncMailbox_frame_size_assert[
    (sizeof(paramSyncMailboxSharedFrame) == PARAM_SYNC_MAILBOX_FRAME_BYTES) ? 1 : -1];

#pragma location = PARAM_SYNC_MAILBOX_REQUEST_ADDR
```

```

__no_init static volatile paramSyncMailboxSharedFrame paramSyncRequestRegion;
#pragma location = PARAM_SYNC_MAILBOX_RESPONSE_ADDR
__no_init static volatile paramSyncMailboxSharedFrame paramSyncResponseRegion;

static uint8_t frameInputValid(uint32_t bootGeneration, uint32_t sequence,
    uint16_t dataLength)
{
    return (uint8_t) ((bootGeneration != 0u) && (sequence != 0u) &&
        (dataLength <= PARAM_SYNC_MAILBOX_DATA_MAX));
}

static void cleanRegion(volatile paramSyncMailboxSharedFrame *region)
{
    SCB_CleanDCache_by_Addr((volatile void *) region, sizeof(*region));
    __DSB();
}

static void invalidateRegion(volatile paramSyncMailboxSharedFrame *region)
{
    SCB_InvalidateDCache_by_Addr((volatile void *) region, sizeof(*region));
    __DSB();
}

static uint8_t publishFrame(volatile paramSyncMailboxSharedFrame *region,
    const paramSyncMailboxSharedFrame *frame)
{
    paramSyncMailboxSharedFrame staged;

    staged = *frame;
    staged.endSequence = 0u;

    region->beginSequence = 0u;
    region->endSequence = 0u;
    cleanRegion(region);

    memcpy((void *) region, &staged, sizeof(staged));
    cleanRegion(region);

    region->endSequence = staged.sequence;
    SCB_CleanDCache_by_Addr((volatile void *)
        ((uintptr_t) &region->endSequence) & ~((uintptr_t) 31u)), 32u);
    __DSB();
    return 1u;
}

static uint8_t sharedFrameValid(const paramSyncMailboxSharedFrame *frame,
    uint32_t magic)
{
    return (uint8_t) ((frame->beginSequence != 0u) &&
        (frame->beginSequence == frame->sequence) &&
        (frame->endSequence == frame->sequence) &&
        (frame->magic == magic) &&
        (frame->abiVersion == PARAM_SYNC_MAILBOX_ABI) &&
        (frame->payloadBytes == frame->dataLength) &&
        frameInputValid(frame->producerBootGeneration, frame->sequence,
            frame->dataLength));
}

static uint8_t tryReadFrame(volatile paramSyncMailboxSharedFrame *region,
    uint32_t magic, paramSyncMailboxSharedFrame *frame)
{
    paramSyncMailboxSharedFrame first;
    paramSyncMailboxSharedFrame second;
    uint8_t attempt;

    for (attempt = 0u; attempt < 3u; attempt++)
    {
        invalidateRegion(region);
        memcpy(&first, (const void *) region, sizeof(first));
        invalidateRegion(region);
        memcpy(&second, (const void *) region, sizeof(second));
        if (sharedFrameValid(&first, magic) &&
            sharedFrameValid(&second, magic) &&
            (memcmp(&first, &second, sizeof(first)) == 0))
        {
            *frame = second;
            return 1u;
        }
    }
    return 0u;
}

```



```

}

uint8_t paramSyncMailboxPublishRequest(const paramSyncMailboxRequest *request)
{
    paramSyncMailboxSharedFrame frame;
    if ((request == 0) || !frameInputValid(request->producerBootGeneration,
        request->sequence, request->dataLength)) return 0u;
    memset(&frame, 0, sizeof(frame));
    frame.beginSequence = request->sequence;
    frame.magic = PARAM_SYNC_REQUEST_MAGIC;
    frame.abiVersion = PARAM_SYNC_MAILBOX_ABI;
    frame.payloadBytes = request->dataLength;
    frame.producerBootGeneration = request->producerBootGeneration;
    frame.sequence = request->sequence;
    frame.transactionId = request->transactionId;
    frame.offset = request->offset;
    frame.payloadCrc32 = request->payloadCrc32;
    frame.dataLength = request->dataLength;
    frame.op = request->op;
    frame.slot = request->slot;
    frame.target = request->target;
    frame.state = request->flags;
    memcpy(frame.data, request->data, request->dataLength);
    return publishFrame(&paramSyncRequestRegion, &frame);
}

uint8_t paramSyncMailboxTryReadRequest(paramSyncMailboxRequest *request)
{
    paramSyncMailboxSharedFrame frame;
    paramSyncMailboxRequest candidate;
    if ((request == 0) || !tryReadFrame(&paramSyncRequestRegion,
        PARAM_SYNC_REQUEST_MAGIC, &frame)) return 0u;
    memset(&candidate, 0, sizeof(candidate));
    candidate.producerBootGeneration = frame.producerBootGeneration;
    candidate.sequence = frame.sequence;
    candidate.transactionId = frame.transactionId;
    candidate.offset = frame.offset;
    candidate.payloadCrc32 = frame.payloadCrc32;
    candidate.dataLength = frame.dataLength;
    candidate.op = frame.op;
    candidate.slot = frame.slot;
    candidate.target = frame.target;
    candidate.flags = frame.state;
    memcpy(candidate.data, frame.data, frame.dataLength);
    *request = candidate;
    return 1u;
}

uint8_t paramSyncMailboxPublishResponse(const paramSyncMailboxResponse *response)
{
    paramSyncMailboxSharedFrame frame;
    if ((response == 0) || !frameInputValid(response->producerBootGeneration,
        response->sequence, response->dataLength)) return 0u;
    memset(&frame, 0, sizeof(frame));
    frame.beginSequence = response->sequence;
    frame.magic = PARAM_SYNC_RESPONSE_MAGIC;
    frame.abiVersion = PARAM_SYNC_MAILBOX_ABI;
    frame.payloadBytes = response->dataLength;
    frame.producerBootGeneration = response->producerBootGeneration;
    frame.sequence = response->sequence;
    frame.transactionId = response->transactionId;
    frame.offset = response->offset;
    frame.payloadCrc32 = response->payloadCrc32;
    frame.dataLength = response->dataLength;
    frame.op = response->op;
    frame.slot = response->slot;
    frame.target = response->target;
    frame.state = response->status;
    memcpy(frame.data, response->data, response->dataLength);
    return publishFrame(&paramSyncResponseRegion, &frame);
}

uint8_t paramSyncMailboxTryReadResponse(paramSyncMailboxResponse *response)
{
    paramSyncMailboxSharedFrame frame;
    paramSyncMailboxResponse candidate;
    if ((response == 0) || !tryReadFrame(&paramSyncResponseRegion,
        PARAM_SYNC_RESPONSE_MAGIC, &frame)) return 0u;
    memset(&candidate, 0, sizeof(candidate));

```

```

candidate.producerBootGeneration = frame.producerBootGeneration;
candidate.sequence = frame.sequence;
candidate.transactionId = frame.transactionId;
candidate.offset = frame.offset;
candidate.payloadCrc32 = frame.payloadCrc32;
candidate.dataLength = frame.dataLength;
candidate.op = frame.op;
candidate.slot = frame.slot;
candidate.target = frame.target;
candidate.status = frame.state;
memcpy(candidate.data, frame.data, frame.dataLength);
*response = candidate;
return 1u;
}

```

core2_cyt4bb7/project/code/comm/THVD1450.c

SHA-256

:

1e76fb6d65a043b69d78f61e00c176076f279d43eb5f022367594f47136d46e2

代码 (c)

```

#include "THVD1450.h"

#include <string.h>

#include "zf_device_type.h"

#include "zf_driver_gpio.h"
#include "zf_driver_uart.h"

#define THVD1450_UART_NUM    (UART_4)
#define THVD1450_UART_TX_PIN (UART4_TX_P14_1)
#define THVD1450_UART_RX_PIN (UART4_RX_P14_0)

#define THVD1450_DIR_PIN      (P13_7) /* core2: RE#/DE 接 P13_7; 与 core1 的驱动并发模型亦有意不同 */
#define THVD1450_RX_FRAME_QUEUE_SIZE (4u)
#define THVD1450_POLL_BYTE_BUDGET   (64u) /* 未接 485 时 RX 悬空可能持续进噪声，前台轮询必须有上限 */
#define THVD1450_RX_BYTE_RING_SIZE  (128u)
#define THVD1450_ISR_BATCH_MAX      (16u)
#define THVD1450_UART_BAUDRATE      1000000
#error "Core1-Core2 RS-485 baud contract requires 1 Mbps"
#endif

static uint8_t thvd1450TxBuffer[THVD1450_FRAME_LENGTH_MAX];

static thvd1450ParserStruct thvd1450Parser;
static thvd1450FrameStruct thvd1450RxQueue[THVD1450_RX_FRAME_QUEUE_SIZE];

static volatile uint8_t thvd1450RxHead = 0u;
static volatile uint8_t thvd1450RxTail = 0u;
static volatile uint8_t thvd1450RxCount = 0u;

#if defined(CY_CORE_CM7_1)
/* UART ISR 单写、前台单读。逐飞 UART 暂存不是 ring，ISR 必须立即搬走本批字节。 */
static uint8_t thvd1450RxByteRing[THVD1450_RX_BYTE_RING_SIZE];
static volatile uint8_t thvd1450RxByteHead = 0u;
static volatile uint8_t thvd1450RxByteTail = 0u;
static volatile uint32_t thvd1450RxOverflowSequence = 0u;
static volatile uint32_t thvd1450RxOverflowAcknowledged = 0u;
static uint8_t thvd1450RxDiscardState = 0u;
#endif

static uint8_t thvd1450NextIndex(uint8_t index)
{
    {
        index++;
        if (index >= THVD1450_RX_FRAME_QUEUE_SIZE)
        {
            index = 0u;
        }
    }

    return index;
}

#if defined(CY_CORE_CM7_1)

```

```

static uint8_t thvd1450NextByteIndex(uint8_t index)
{
    index++;
    if (index >= THVD1450_RX_BYTE_RING_SIZE)
    {
        index = 0u;
    }
    return index;
}

static void thvd1450PushByteFromIsr(uint8_t data)
{
    uint8_t next;

    if (thvd1450RxOverflowSequence != thvd1450RxOverflowAcknowledged)
    {
        return;
    }

    next = thvd1450NextByteIndex(thvd1450RxByteTail);
    if (next == thvd1450RxByteHead)
    {
        thvd1450RxOverflowSequence++;
        return;
    }

    thvd1450RxByteRing[thvd1450RxByteTail] = data;
    __DMB();
    thvd1450RxByteTail = next;
}
#endif

static void thvd1450PushFrame(const thvd1450FrameStruct *frame)
{
    if (NULL == frame)
    {
        return;
    }

    if (thvd1450RxCount >= THVD1450_RX_FRAME_QUEUE_SIZE)
    {
        thvd1450RxHead = thvd1450NextIndex(thvd1450RxHead);
        thvd1450RxCount--;
    }

    thvd1450RxQueue[thvd1450RxTail] = *frame;
    thvd1450RxTail = thvd1450NextIndex(thvd1450RxTail);
    thvd1450RxCount++;
}

static void thvd1450HandleByte(uint8_t data)
{
    thvd1450FrameStruct rxFrame;
    thvd1450StatusEnum status;

    status = thvd1450ProtocolInputByte(&thvd1450Parser, data, &rxFrame);
    if (THVD1450_STATUS_FRAME_READY == status)
    {
        thvd1450PushFrame(&rxFrame);
    }
}

void thvd1450Init(void)
{
    thvd1450ProtocolInit(&thvd1450Parser);
    memset(thvd1450RxQueue, 0, sizeof(thvd1450RxQueue));
    thvd1450RxHead = 0u;
    thvd1450RxTail = 0u;
    thvd1450RxCount = 0u;
#ifdef CY_CORE_CM7_1
    memset(thvd1450RxByteRing, 0, sizeof(thvd1450RxByteRing));
    thvd1450RxByteHead = 0u;
    thvd1450RxByteTail = 0u;
    thvd1450RxOverflowSequence = 0u;
    thvd1450RxOverflowAcknowledged = 0u;
    thvd1450RxDiscardState = 0u;
#endif
}

uart_init(THVD1450_UART_NUM, THVD1450_UART_BAUDRATE, THVD1450_UART_TX_PIN, THVD1450_UART_RX_PIN);

```

```

gpio_init(THVD1450_DIR_PIN, GPO, GPIO_LOW, GPO_PUSH_PULL);

thvd1450SetMode(THVD1450_MODE_RECEIVE);

#if defined(CY_CORE_CM7_1)
/* cm_7_1 通信核：uart4_isr 在 uart_isr_mask 后立即把驱动暂存搬入本地 SPSC ring，
 * 前台 thvd1450Poll 只消费 ring 并解析。控制环在 cm_7_0，与本 UART 中断完全隔离：
 * RS-485 悬空/噪声的 RX 中断风暴只拖慢空闲的 cm_7_1，不影响 cm_7_0 的 500µs 控制 PIT。
 * ISR 只做最多 16 字节的有界搬运，不解析协议、不发送。 */
uart_rx_interrupt(THVD1450_UART_NUM, 1u);
#else
/* cm_7_0 调参固件：保持 RX 中断关闭（原前台轮询范式，避免 485 悬空风暴饿死本核控制 PIT）。
 * 注：cm_7_0 生产固件用镜像实现，根本不编译本 thvd1450Init。 */
uart_rx_interrupt(THVD1450_UART_NUM, 0u);
#endif
}

void thvd1450SetMode(thvd1450ModeEnum mode)
{
    if (THVD1450_MODE_TRANSMIT == mode)
    {
        gpio_high(THVD1450_DIR_PIN);
    }
    else
    {
        gpio_low(THVD1450_DIR_PIN);
    }
}

void thvd1450Poll(void)
{
    uint8_t data;
    uint8_t budget = THVD1450_POLL_BYTE_BUDGET;

#if defined(CY_CORE_CM7_1)
    if (thvd1450RxOverflowSequence != thvd1450RxOverflowAcknowledged)
    {
        uint32_t overflowSequence;

        uart_rx_interrupt(THVD1450_UART_NUM, 0u);
        overflowSequence = thvd1450RxOverflowSequence;
        thvd1450RxByteHead = thvd1450RxByteTail;
        thvd1450ProtocolReset(&thvd1450Parser);
        thvd1450RxDiscardState = 1u;
        thvd1450RxOverflowAcknowledged = overflowSequence;
        uart_rx_interrupt(THVD1450_UART_NUM, 1u);
    }

    while ((budget > 0u) &&
           (thvd1450RxByteHead != thvd1450RxByteTail))
    {
        data = thvd1450RxByteRing[thvd1450RxByteHead];
        __DMB();
        thvd1450RxByteHead = thvd1450NextByteIndex(thvd1450RxByteHead);
        budget--;
        if (thvd1450RxDiscardState != 0u)
        {
            {
                if (thvd1450RxDiscardState == 1u)
                {
                    if (data == THVD1450_FRAME_TAIL)
                        thvd1450RxDiscardState = 2u;
                }
                else if (data == THVD1450_FRAME_HEAD)
                {
                    thvd1450ProtocolReset(&thvd1450Parser);
                    thvd1450HandleByte(data);
                    thvd1450RxDiscardState = 0u;
                }
            }
            continue;
        }
        thvd1450HandleByte(data);
    }
#else
    while ((budget > 0u) && uart_query_byte(THVD1450_UART_NUM, &data))
    {
        budget--;
        thvd1450HandleByte(data);
    }
}

```

```

#endif
}

void thvd1450RxIrqHandler(void)
{
#if defined(CY_CORE_CM7_1)
    uint8_t batch[THVD1450_ISR_BATCH_MAX];
    uint8_t count;
    uint8_t i;

    count = uart_query_buffer(THVD1450_UART_NUM, batch);
    if (count > THVD1450_ISR_BATCH_MAX)
    {
        count = THVD1450_ISR_BATCH_MAX;
    }
    for (i = 0u; i < count; i++)
    {
        thvd1450PushByteFromIsr(batch[i]);
    }
#endif
}

thvd1450StatusEnum thvd1450SendBytes(const uint8_t *payload, uint8_t length)
{
    uint8_t frameLength = 0u;
    thvd1450StatusEnum status;

    status = thvd1450ProtocolEncode(
        payload,
        length,
        thvd1450TxBuffer,
        (uint8_t)sizeof(thvd1450TxBuffer),
        &frameLength);
    if (THVD1450_STATUS_OK != status)
    {
        return status;
    }

    thvd1450SetMode(THVD1450_MODE_TRANSMIT);
    uart_write_buffer(THVD1450_UART_NUM, thvd1450TxBuffer, frameLength);
    thvd1450SetMode(THVD1450_MODE_RECEIVE);

    return THVD1450_STATUS_OK;
}

thvd1450StatusEnum thvd1450ReadFrame(thvd1450FrameStruct *frame)
{
    if (NULL == frame)
    {
        return THVD1450_STATUS_INVALID_PARAM;
    }

    if (0u == thvd1450RxCount)
    {
        return THVD1450_STATUS_NO_FRAME;
    }

    *frame = thvd1450RxQueue[thvd1450RxHead];
    thvd1450RxHead = thvd1450NextIndex(thvd1450RxHead);
    thvd1450RxCount--;
    return THVD1450_STATUS_OK;
}

```

core2_cyt4bb7/project/code/comm/THVD1450.h

SHA-256

:

be1a0dfdd7f105b85337f2bd5cf3769675f2e7742b9a172927adbf9bee48d875

代码 (c)

```

#ifndef_THVD1450_H_
#define_THVD1450_H_

#include <stdint.h>

```

```

#include "THVD1450_protocol.h"

typedef enum
{
    THVD1450_MODE_RECEIVE = 0u,
    THVD1450_MODE_TRANSMIT,
} thvd1450ModeEnum;

void thvd1450Init(void);
void thvd1450SetMode(thvd1450ModeEnum mode);

void thvd1450Poll(void);
void thvd1450RxIrqHandler(void);

thvd1450StatusEnum thvd1450ReadFrame(thvd1450FrameStruct *frame);

thvd1450StatusEnum thvd1450SendBytes(const uint8_t *payload, uint8_t length);

#endif // _THVD1450_H_

```

core2_cyt4bb7/project/code/comm/THVD1450_protocol.c

SHA-256

:

634e0a17539c065b6fda54bd997e2dfbe871fe8480c45d41630e20389016056f

代码 (c)

```

#include "THVD1450_protocol.h"

#include <string.h>

#define THVD1450_PARSE_WAIT_HEAD    (0u)
#define THVD1450_PARSE_WAIT_LENGTH (1u)
#define THVD1450_PARSE_WAIT_PAYLOAD (2u)
#define THVD1450_PARSE_WAIT_CHECKSUM (3u)
#define THVD1450_PARSE_WAIT_TAIL    (4u)

static void thvd1450ProtocolRestartFromHead(thvd1450ParserStruct *parser)
{
    thvd1450ProtocolReset(parser);
    parser->state = THVD1450_PARSE_WAIT_LENGTH;
}

void thvd1450ProtocolInit(thvd1450ParserStruct *parser)
{
    thvd1450ProtocolReset(parser);
}

void thvd1450ProtocolReset(thvd1450ParserStruct *parser)
{
    if (NULL == parser)
    {
        return;
    }

    memset(parser, 0, sizeof(*parser));
    parser->state = THVD1450_PARSE_WAIT_HEAD;
}

uint8_t thvd1450ProtocolChecksum(const uint8_t *payload, uint8_t length)
{
    uint8_t checksum = length;
    uint8_t i;

    for (i = 0u; i < length; i++)
    {
        checksum ^= payload[i];
    }

    return checksum;
}

thvd1450StatusEnum thvd1450ProtocolEncode(
    const uint8_t *payload,
    uint8_t length,

```

```

uint8_t *frameBuffer,
uint8_t frameBufferSize,
uint8_t *frameLength)
{
    if ((NULL == frameBuffer) || (NULL == frameLength) || ((0u != length) && (NULL == payload)))
    {
        return THVD1450_STATUS_INVALID_PARAM;
    }

    if (length > THVD1450_PAYLOAD_LENGTH_MAX)
    {
        return THVD1450_STATUS_LENGTH_ERROR;
    }

    if (frameBufferSize < (uint8_t)(length + THVD1450_FRAME_OVERHEAD))
    {
        return THVD1450_STATUS_LENGTH_ERROR;
    }

    frameBuffer[0] = THVD1450_FRAME_HEAD;
    frameBuffer[1] = length;

    if (0u != length)
    {
        memcpy(&frameBuffer[2], payload, length);
    }

    frameBuffer[2u + length] = thvd1450ProtocolChecksum(payload, length);
    frameBuffer[3u + length] = THVD1450_FRAME_TAIL;
    *frameLength = (uint8_t)(length + THVD1450_FRAME_OVERHEAD);

    return THVD1450_STATUS_OK;
}

thvd1450StatusEnum thvd1450ProtocolInputByte(
    thvd1450ParserStruct *parser,
    uint8_t byte,
    thvd1450FrameStruct *outFrame)
{
    if (NULL == parser)
    {
        return THVD1450_STATUS_INVALID_PARAM;
    }

    switch (parser->state)
    {
        case THVD1450_PARSE_WAIT_HEAD:
        {
            if (THVD1450_FRAME_HEAD == byte)
            {
                parser->state = THVD1450_PARSE_WAIT_LENGTH;
            }
            return THVD1450_STATUS_NO_FRAME;
        }

        case THVD1450_PARSE_WAIT_LENGTH:
        {
            if (byte > THVD1450_PAYLOAD_LENGTH_MAX)
            {
                thvd1450ProtocolReset(parser);
                return THVD1450_STATUS_LENGTH_ERROR;
            }

            parser->expectedLength = byte;
            parser->frame.length = byte;
            parser->checksum = byte;
            parser->index = 0u;
            parser->state = (0u == byte) ? THVD1450_PARSE_WAIT_CHECKSUM : THVD1450_PARSE_WAIT_PAYLOAD;
            return THVD1450_STATUS_NO_FRAME;
        }

        case THVD1450_PARSE_WAIT_PAYLOAD:
        {
            parser->frame.payload[parser->index] = byte;
            parser->checksum ^= byte;
            parser->index++;

            if (parser->index >= parser->expectedLength)
            {

```

```

        parser->state = THVD1450_PARSE_WAIT_CHECKSUM;
    }
    return THVD1450_STATUS_NO_FRAME;
}

case THVD1450_PARSE_WAIT_CHECKSUM:
{
    if (byte != parser->checksum)
    {
        if (THVD1450_FRAME_HEAD == byte)
        {
            thvd1450ProtocolRestartFromHead(parser);
        }
        else
        {
            thvd1450ProtocolReset(parser);
        }
        return THVD1450_STATUS_CHECKSUM_ERROR;
    }

    parser->state = THVD1450_PARSE_WAIT_TAIL;
    return THVD1450_STATUS_NO_FRAME;
}

case THVD1450_PARSE_WAIT_TAIL:
{
    if (THVD1450_FRAME_TAIL == byte)
    {
        if (NULL != outFrame)
        {
            *outFrame = parser->frame;
        }
        thvd1450ProtocolReset(parser);
        return THVD1450_STATUS_FRAME_READY;
    }

    if (THVD1450_FRAME_HEAD == byte)
    {
        thvd1450ProtocolRestartFromHead(parser);
    }
    else
    {
        thvd1450ProtocolReset(parser);
    }
    return THVD1450_STATUS_TAIL_ERROR;
}

default:
{
    thvd1450ProtocolReset(parser);
    break;
}
}

return THVD1450_STATUS_NO_FRAME;
}

```

core2_cyt4bb7/project/code/comm/THVD1450_protocol.h

SHA-256

:

0cdc3470af32c7fe3808e590f1ac4f4833c1426b514e67b9e228bf5529f56ad6

代码 (c)

```

#ifndef THVD1450_PROTOCOL_H
#define THVD1450_PROTOCOL_H

#include <stdint.h>

#define THVD1450_FRAME_HEAD      (0x7Au)
#define THVD1450_UART_BAUDRATE  (1 000 000u)
#define THVD1450_FRAME_TAIL     (0x55u)
#define THVD1450_DATA_LENGTH_MAX (16u)
#define THVD1450_PAYLOAD_LENGTH_MAX (THVD1450_DATA_LENGTH_MAX)
#define THVD1450_FRAME_OVERHEAD  (4u)

```



```

#define THVD1450_FRAME_LENGTH_MAX (THVD1450_PAYLOAD_LENGTH_MAX + THVD1450_FRAME_OVERHEAD)

typedef enum
{
    THVD1450_STATUS_OK = 0u,
    THVD1450_STATUS_FRAME_READY,
    THVD1450_STATUS_NO_FRAME,
    THVD1450_STATUS_INVALID_PARAM,
    THVD1450_STATUS_LENGTH_ERROR,
    THVD1450_STATUS_CHECKSUM_ERROR,
    THVD1450_STATUS_TAIL_ERROR,
} thvd1450StatusEnum;

typedef struct
{
    uint8_t length;
    uint8_t payload[THVD1450_PAYLOAD_LENGTH_MAX];
} thvd1450FrameStruct;

typedef struct
{
    uint8_t state;
    uint8_t index;
    uint8_t expectedLength;
    uint8_t checksum;
    thvd1450FrameStruct frame;
} thvd1450ParserStruct;

void thvd1450ProtocolInit(thvd1450ParserStruct *parser);
void thvd1450ProtocolReset(thvd1450ParserStruct *parser);
uint8_t thvd1450ProtocolChecksum(const uint8_t *payload, uint8_t length);

thvd1450StatusEnum thvd1450ProtocolEncode(
    const uint8_t *payload,
    uint8_t length,
    uint8_t *frameBuffer,
    uint8_t frameBufferSize,
    uint8_t *frameLength);

thvd1450StatusEnum thvd1450ProtocolInputByte(
    thvd1450ParserStruct *parser,
    uint8_t byte,
    thvd1450FrameStruct *outFrame);

#endif // _THVD1450_PROTOCOL_H_

```

core2_cyt4bb7/project/code/config/chassisParams.c

SHA-256

:

0039ec6bd486a948abfa840a3680b9f1b38965cda15400345a5b17b00b1adb6d

代码 (c)

```

#include "chassisParams.h"

#include <string.h>

#include "configChassisPidDefaults.h"
#include "configChassisRemote.h"
#include "lightStrip.h" /* LIGHT_STRIP_DEFAULT_BRIGHTNESS / _MIN / _MAX (亮度默认与钳位单一真相源) */
#include "commonDef.h"

typedef struct
{
    chassisParamStruct active;
    uint32_t revision;
    uint8_t initialized;
} chassisParamsRepositoryStruct;

static chassisParamsRepositoryStruct chassisParamsRepository = {0};

void chassisParamsInit(void)
{
    chassisParamStruct initial;

```

```

    chassisParamsLoadDefaults(&initial);
    chassisParamsFlashLoad(&initial);
    chassisParamsSanitize(&initial);
    chassisParamsRepository.active = initial;
    chassisParamsRepository.revision = 1u;
    chassisParamsRepository.initialized = 1u;
}

void chassisParamsLoadDefaults(chassisParamStruct* p)
{
    if (p == 0) return;

    /* 速度环四轮增益 + 积分钳位 */
    p->speedFl.kp = MOTOR_CONTROL_FL_SPEED_KP;
    p->speedFl.ki = MOTOR_CONTROL_FL_SPEED_KI;
    p->speedFl.kd = MOTOR_CONTROL_FL_SPEED_KD;
    p->speedFr.kp = MOTOR_CONTROL_FR_SPEED_KP;
    p->speedFr.ki = MOTOR_CONTROL_FR_SPEED_KI;
    p->speedFr.kd = MOTOR_CONTROL_FR_SPEED_KD;
    p->speedBl.kp = MOTOR_CONTROL_BL_SPEED_KP;
    p->speedBl.ki = MOTOR_CONTROL_BL_SPEED_KI;
    p->speedBl.kd = MOTOR_CONTROL_BL_SPEED_KD;
    p->speedBr.kp = MOTOR_CONTROL_BR_SPEED_KP;
    p->speedBr.ki = MOTOR_CONTROL_BR_SPEED_KI;
    p->speedBr.kd = MOTOR_CONTROL_BR_SPEED_KD;
    p->speedIntegralMax = MOTOR_CONTROL_SPEED_PID_INTEGRAL_MAX;

    /* 电流环（四轮独立，各读自己一组宏；所有 * CURRENT * 宏均有意义） */
    p->currentFl.kp = MOTOR_CONTROL_FL_CURRENT_KP;
    p->currentFl.ki = MOTOR_CONTROL_FL_CURRENT_KI;
    p->currentFl.kd = MOTOR_CONTROL_FL_CURRENT_KD;
    p->currentFr.kp = MOTOR_CONTROL_FR_CURRENT_KP;
    p->currentFr.ki = MOTOR_CONTROL_FR_CURRENT_KI;
    p->currentFr.kd = MOTOR_CONTROL_FR_CURRENT_KD;
    p->currentBl.kp = MOTOR_CONTROL_BL_CURRENT_KP;
    p->currentBl.ki = MOTOR_CONTROL_BL_CURRENT_KI;
    p->currentBl.kd = MOTOR_CONTROL_BL_CURRENT_KD;
    p->currentBr.kp = MOTOR_CONTROL_BR_CURRENT_KP;
    p->currentBr.ki = MOTOR_CONTROL_BR_CURRENT_KI;
    p->currentBr.kd = MOTOR_CONTROL_BR_CURRENT_KD;
    p->currentIntegralMax = MOTOR_CONTROL_CURRENT_PID_INTEGRAL_MAX;

    /* 角速度环 */
    p->yawRate.kp = CHASSIS_YAW_RATE_KP;
    p->yawRate.ki = CHASSIS_YAW_RATE_KI;
    p->yawRate.kd = CHASSIS_YAW_RATE_KD;
    p->yawRateIntegralMax = CHASSIS_YAW_RATE_INTEGRAL_MAX;
    p->headingKp = CHASSIS_HEADING_KP;
    p->headingRateLimit = CHASSIS_HEADING_RATE_LIMIT_RADPS;
    p->yawRateOutputLimit = CHASSIS_YAW_RATE_OUTPUT_LIMIT;

    /* LoRa 映射（真实物理单位） */
    p->loraVMaxMps = LORA_V_MAX_MPS;
    p->loraWzMaxRadps = LORA_WZ_MAX_RADPS;
    p->loraStickDeadzone = (float) LORA_STICK_DEADZONE;

    /* 导航 */
    p->navCorrectionKp = NAV_CORRECTION_KP;
    p->navCorrectionLimit = NAV_CORRECTION_LIMIT;
    p->navErrorLimitPulses = NAV_ERROR_LIMIT_PULSES;

    /* ^ 定位灯条亮度默认（lightStrip.h 单一真相源） */
    p->lightStripBrightness = (float) LIGHT_STRIP_DEFAULT_BRIGHTNESS;
}

/* 钳位单个 PID 增益组：kp/ki/kd 必须非负（极性靠电机方向系数，增益恒非负，负增益=正反馈危险） */
static void sanitizeGain(chassisPidGainStruct* g)
{
    if (g->kp < 0.0f) g->kp = 0.0f;
    if (g->ki < 0.0f) g->ki = 0.0f;
    if (g->kd < 0.0f) g->kd = 0.0f;
}

static uint8_t chassisParamsAllFinite(const chassisParamStruct* p)
{
    const float* values = &p->speedFl.kp;
    uint32_t count = (uint32_t) (sizeof(chassisParamStruct) / sizeof(float));
    uint32_t i;

```

```

    for (i = 0u; i < count; i++)
    {
        if (floatIsFinite(values[i]) == 0u) return 0u;
    }
    return 1u;
}

void chassisParamsSanitize(chassisParamStruct* p)
{
    if (p == 0) return;

    if (chassisParamsAllFinite(p) == 0u)
    {
        chassisParamsLoadDefaults(p);
    }

    sanitizeGain((chassisPidGainStruct*) &p->speedFl);
    sanitizeGain((chassisPidGainStruct*) &p->speedFr);
    sanitizeGain((chassisPidGainStruct*) &p->speedBl);
    sanitizeGain((chassisPidGainStruct*) &p->speedBr);
    sanitizeGain((chassisPidGainStruct*) &p->currentFl);
    sanitizeGain((chassisPidGainStruct*) &p->currentFr);
    sanitizeGain((chassisPidGainStruct*) &p->currentBl);
    sanitizeGain((chassisPidGainStruct*) &p->currentBr);
    sanitizeGain((chassisPidGainStruct*) &p->yawRate);

    if (p->speedIntegralMax < 0.0f) p->speedIntegralMax = 0.0f;
    if (p->currentIntegralMax < 0.0f) p->currentIntegralMax = 0.0f;
    if (p->yawRateIntegralMax < 0.0f) p->yawRateIntegralMax = 0.0f;
    if (p->headingKp < 0.0f) p->headingKp = 0.0f;
    if (p->headingKp > CHASSIS_HEADING_KP_MAX) p->headingKp = CHASSIS_HEADING_KP_MAX;
    if (p->headingRateLimit < 0.0f) p->headingRateLimit = 0.0f;
    if (p->headingRateLimit > CHASSIS_HEADING_RATE_LIMIT_MAX_RADPS)
    {
        p->headingRateLimit = CHASSIS_HEADING_RATE_LIMIT_MAX_RADPS;
    }
    if (p->yawRateOutputLimit < 0.0f) p->yawRateOutputLimit = 0.0f;

    /* LoRa 速度上限非负；死区须 < 量程（防 loraRemote 除零/负标度） */
    if (p->loraVMaxMps < 0.0f) p->loraVMaxMps = 0.0f;
    if (p->loraWzMaxRadps < 0.0f) p->loraWzMaxRadps = 0.0f;
    if (p->loraStickDeadzone < 0.0f) p->loraStickDeadzone = 0.0f;
    if (p->loraStickDeadzone >= (float) LORA_STICK_RANGE)
    {
        p->loraStickDeadzone = (float) (LORA_STICK_RANGE - 1);
    }

    /* 导航纠正非负 */
    if (p->navCorrectionKp < 0.0f) p->navCorrectionKp = 0.0f;
    if (p->navCorrectionLimit < 0.0f) p->navCorrectionLimit = 0.0f;
    if (p->navErrorLimitPulses < 0.0f) p->navErrorLimitPulses = 0.0f;

    /* 灯条亮度钳到 [MIN,MAX]（防菜单/flash 注入越界；lightStripSetBrightness 收 uint8_t pct） */
    if (p->lightStripBrightness < (float) LIGHT_STRIP_BRIGHTNESS_MIN) p->lightStripBrightness = (float) LIGHT_STRIP_BRIGHTNESS_MIN;
    if (p->lightStripBrightness > (float) LIGHT_STRIP_BRIGHTNESS_MAX) p->lightStripBrightness = (float) LIGHT_STRIP_BRIGHTNESS_MAX;
}

void chassisParamsGetActive(chassisParamStruct* out)
{
    if (out == 0) return;
    if (!chassisParamsRepository.initialized)
    {
        chassisParamsLoadDefaults(out);
        chassisParamsSanitize(out);
        return;
    }
    *out = chassisParamsRepository.active;
}

uint32_t chassisParamsGetRevision(void)
{
    return chassisParamsRepository.initialized ?
        chassisParamsRepository.revision : 0u;
}

uint8_t chassisParamsCommit(const chassisParamStruct* candidate)
{

```

```

chassisParamStruct sanitized;

if (candidate == 0) return 0u;
sanitized = *candidate;
chassisParamsSanitize(&sanitized);
chassisParamsRepository.active = sanitized;
chassisParamsRepository.revision++;
if (chassisParamsRepository.revision == 0u) chassisParamsRepository.revision = 1u;
chassisParamsRepository.initialized = 1u;
return 1u;
}

```

core2_cyt4bb7/project/code/config/chassisParams.h

SHA-256

:

83b7a3a447cf609f30831b54800f70d8c3b03576ef12065240249937a45e1973

代码 (c)

```

#ifndef _CHASSIS_PARAMS_H_
#define _CHASSIS_PARAMS_H_

#include <stdint.h>

/*
 * core2 底盘可调参数 RAM 镜像（仿 core0 flightParams）。
 *
 * 目的：菜单热重载 + flash 持久化的运行时参数载体。控制环从读编译期默认宏
 * 改为读本镜像；默认值仍以 configChassisPidDefaults.h / configChassisRemote.h
 * 宏为单一真相源（chassisParamsLoadDefaults 加载）。
 *
 * 安全只读项不进镜像、保持宏：MOTOR_*_OUTPUT_DIR(极性)、MOTOR_RUNAWAY*(失控保护)、
 * *_DIVIDER(时序)、*_LPF*(滤波)、LORA_LINK_TIMEOUT_MS、CHASSIS_*几何/编码器换算、
 * CHASSIS_RS485_*物理上限。
 *
 * 当前：本结构只作 DTO；活动副本由 chassisParams.c 私有仓库持有。菜单编辑 working copy，
 * runtime 提交后各 owner 缓存所需窄配置，ISR 不直接依赖仓库布局。
 */

/* PID 子参数（对齐 pidInit 入参语义） */
typedef struct
{
    float kp;
    float ki;
    float kd;
} chassisPidGainStruct;

typedef struct
{
    /* $B 速度环（四轮独立增益 + 共用积分钳位；位置式 + D 项 LPF） */
    chassisPidGainStruct speedFl;
    chassisPidGainStruct speedFr;
    chassisPidGainStruct speedBl;
    chassisPidGainStruct speedBr;
    float speedIntegralMax; /* MOTOR_CONTROL_SPEED_PID_INTEGRAL_MAX */

    /* $C 电流环（四轮独立增益 + 共用积分钳位；增量式 integralMax=0） */
    chassisPidGainStruct currentFl;
    chassisPidGainStruct currentFr;
    chassisPidGainStruct currentBl;
    chassisPidGainStruct currentBr;
    float currentIntegralMax; /* MOTOR_CONTROL_CURRENT_PID_INTEGRAL_MAX（四轮共享） */

    /* $D 角速度（yaw）环（位置式） */
    chassisPidGainStruct yawRate;
    float yawRateIntegralMax; /* CHASSIS_YAW_RATE_INTEGRAL_MAX */
    float headingKp; /* CHASSIS_HEADING_KP */
    float headingRateLimit; /* CHASSIS_HEADING_RATE_LIMIT_RADPS */
    float yawRateOutputLimit; /* CHASSIS_YAW_RATE_OUTPUT_LIMIT */

    /* $F LoRa 映射（真实物理单位）：V Max 同时是摇杆满量程与 LoRa 模式线速度限幅 */
    float loraVMaxMps; /* LORA_V_MAX_MPS */
    float loraWzMaxRadps; /* LORA_WZ_MAX_RADPS */
    float loraStickDeadzone; /* LORA_STICK_DEADZONE（float 影子，用时转 int 比较） */
}

```

```

/* 导航 (D-4 菜单接入消费侧后生效) */
float navCorrectionKp; /* NAV_CORRECTION_KP */
float navCorrectionLimit; /* NAV_CORRECTION_LIMIT */
float navErrorLimitPulses; /* NAV_ERROR_LIMIT_PULSES */

/* ^ 定位灯条亮度(0~100 pct, WAIT/HOME 快调 + flash 持久化; 用 lightStripSetBrightness 应用,
 * 默认 LIGHT_STRIP_DEFAULT_BRIGHTNESS. float 影子, 应用时转 uint8_t) */
float lightStripBrightness;

} chassisParamStruct;

/* 仓库生命周期: 默认值 → Flash DTO 覆盖 → sanitize → 活动参数。 */
void chassisParamsInit(void);

/* 从 PID/remote 默认宏加载 DTO (单一真相源)。 */
void chassisParamsLoadDefaults(chassisParamStruct* p);

/* 钳位非法值 (防菜单/flash 注入危险参数: 增益负、积分钳位负、死区越界等)。安全相关。 */
void chassisParamsSanitize(chassisParamStruct* p);

/* 活动参数只按值复制; Commit 成功返回 1, 并递增 revision。 */
void chassisParamsGetActive(chassisParamStruct* out);
uint32_t chassisParamsGetRevision(void);
uint8_t chassisParamsCommit(const chassisParamStruct* candidate);

/* ---- flash 持久化 (仿 core0 menuModel, page 0, magic/version/payloadBytes/CRC32 四校验) ---- */

/* 上电载入: 先 chassisParamsLoadDefaults 填默认, 再本函数从 flash 覆盖 (校验通过 + GLOBAL_PARAM_PERSIST_ENABLE)。
 * 空白页/magic/version/长度/CRC 任一不符 → 保留默认值 (降级安全)。载入后调 chassisParamsSanitize。
 * 必须在控制环 init 之前调用 (chassisParams 先就绪)。 */
void chassisParamsFlashLoad(chassisParamStruct* params);

/* 保存当前镜像到 flash (菜单长按保存触发; GLOBAL_PARAM_PERSIST_ENABLE 关闭时空操作)。
 * 前台调用 (含 flash erase/write 阻塞), 绝不在 ISR。 */
void chassisParamsFlashSave(const chassisParamStruct* params);

/* 擦除 flash 持久化页 (恢复出厂: 下次上电用宏默认值) */
void chassisParamsFlashErase(void);

typedef enum
{
    CHASSIS_PARAMS_SLOT_OK = 0,
    CHASSIS_PARAMS_SLOT_ERROR
} chassisParamsSlotStatus;

typedef struct
{
    uint8_t slot;
    uint8_t valid;
    uint8_t active;
    uint8_t bank;
    uint32_t generation;
    uint32_t blobCrc;
    uint32_t transactionId;
} chassisParamsSlotMeta;

typedef struct
{
    uint8_t valid;
    uint8_t bank;
    uint8_t pending;
    uint8_t sourceSlot;
    uint32_t generation;
    uint32_t blobCrc;
    uint32_t transactionId;
    uint8_t diagnosticFlags;
} chassisParamsRuntimeMeta;

typedef struct
{
    uint8_t valid;
    uint8_t active;
    uint8_t activationOnly;
    uint8_t slot;
    uint32_t transactionId;
} chassisParamsPendingInfo;

#define CHASSIS_PARAMS_LIBRARY_EMPTY (0u)
#define CHASSIS_PARAMS_LIBRARY_VALID (1u)

```

```

#define CHASSIS_PARAMS_LIBRARY_INVALID (2u)
typedef struct
{
    uint8_t state;
    uint8_t bank;
    uint16_t reserved;
    uint32_t generation;
    uint32_t blobCrc;
    uint32_t transactionId;
} chassisParamsLibraryMeta;

uint8_t chassisParamsGetActiveSlot(void);
chassisParamsSlotStatus chassisParamsReadSlot(uint8_t slot,
    chassisParamStruct *params, chassisParamsSlotMeta *meta);
chassisParamsSlotStatus chassisParamsInspectSlot(uint8_t slot,
    chassisParamsLibraryMeta *meta);
chassisParamsSlotStatus chassisParamsPrepareSlot(uint8_t slot,
    const chassisParamStruct *params, uint32_t transactionId);
chassisParamsSlotStatus chassisParamsCommitPreparedSlot(uint8_t slot,
    uint32_t transactionId, uint8_t activate);
chassisParamsSlotStatus chassisParamsAbortPreparedSlot(uint32_t transactionId);
chassisParamsSlotStatus chassisParamsFinishPreparedSlot(uint32_t transactionId);
chassisParamsSlotStatus chassisParamsSaveSlot(uint8_t slot,
    const chassisParamStruct *params, uint32_t transactionId);
chassisParamsSlotStatus chassisParamsActivateSlot(uint8_t slot,
    chassisParamStruct *params);
chassisParamsSlotStatus chassisParamsActivatePreparedSlot(uint8_t slot,
    uint32_t transactionId, chassisParamStruct *params);
chassisParamsSlotStatus chassisParamsActivateSlotPending(uint8_t slot,
    chassisParamStruct *params);
chassisParamsSlotStatus chassisParamsAbortPendingActivation(
    chassisParamStruct *params);
chassisParamsSlotStatus chassisParamsFinishPendingActivation(void);
chassisParamsSlotStatus chassisParamsRestoreActiveSlotDefaults(
    chassisParamStruct *params);
chassisParamsSlotStatus chassisParamsReadRuntime(chassisParamStruct *params,
    chassisParamsRuntimeMeta *meta);
chassisParamsSlotStatus chassisParamsGetRuntimeMeta(
    chassisParamsRuntimeMeta *meta);
chassisParamsSlotStatus chassisParamsGetPending(
    chassisParamsPendingInfo *pending);
chassisParamsSlotStatus chassisParamsSaveRuntime(
    const chassisParamStruct *params);
chassisParamsSlotStatus chassisParamsPrepareRuntime(
    const chassisParamStruct *params, uint32_t transactionId,
    uint8_t sourceSlot);
chassisParamsSlotStatus chassisParamsPrepareRuntimeFromSlot(uint8_t slot,
    uint32_t transactionId);
chassisParamsSlotStatus chassisParamsApplyPreparedRuntime(
    uint32_t transactionId, chassisParamStruct *candidate);
chassisParamsSlotStatus chassisParamsCommitRuntime(uint32_t transactionId);

#endif /* _CHASSIS_PARAMS_H_ */

```

core2_cyt4bb7/project/code/config/chassisParamSlotStore.c

SHA-256

:

4fbfd85b94adb5cc897f8f866f32e835a315c14f26b9158eae0e7530e410791c

代码 (c)

```

/* Core2-local build unit for the repository-wide parameter slot store. */
#include "../..../core0_cyt4bb7/project/code/util/paramSlotStore.c"

```

core2_cyt4bb7/project/code/config/chassisParamsPersistence.c

SHA-256

:

36aaefb57d4d6e07a42114e5f1697e7249d8116c6085c725554fb75d99474cc9

代码 (c)

```

#include "chassisParams.h"

#include <stddef.h>
#include <string.h>

#include ".././../core0_cyt4bb7/project/code/util/paramSlotStore.h"
#include "configCore2Build.h"
#include "crc32leee.h"

#define CHASSIS_FLASH_MAGIC (0x4D324B30u)
#define CHASSIS_FLASH_VERSION (9u)

typedef struct
{
    uint32_t magic;
    uint32_t version;
    uint32_t payloadBytes;
    chassisParamStruct params;
    uint32_t crc;
} chassisFlashBlobStruct;

typedef char chassisParamV9SizeMustRemain160[(sizeof(chassisParamStruct) == 160u) ? 1 : -1];
typedef char chassisBlobV9SizeMustRemain176[(sizeof(chassisFlashBlobStruct) == 176u) ? 1 : -1];

static paramSlotStore g_slotStore;
static uint8_t g_slotReady;

static void fillRuntimeMeta(chassisParamsRuntimeMeta *meta,
    const paramRuntimeMeta *raw)
{
    memset(meta, 0, sizeof(*meta));
    meta->valid = raw->valid;
    meta->bank = raw->bank;
    meta->pending = raw->pending;
    meta->sourceSlot = raw->sourceSlot;
    meta->generation = raw->generation;
    meta->blobCrc = raw->blobCrc;
    meta->transactionId = raw->transactionId;
    meta->diagnosticFlags = paramSlotStoreRuntimeDefaultsRewritten(
        &g_slotStore);
}

static uint8_t blobValid(const void *data, uint32_t bytes)
{
    const chassisFlashBlobStruct *blob = (const chassisFlashBlobStruct *) data;
    return (uint8_t) ((bytes == sizeof(*blob)) &&
        (blob->magic == CHASSIS_FLASH_MAGIC) &&
        (blob->version == CHASSIS_FLASH_VERSION) &&
        (blob->payloadBytes == sizeof(*blob)) &&
        (crc32leeeCalc((const uint8_t *) blob,
            (uint32_t) offsetof(chassisFlashBlobStruct, crc)) == blob->crc));
}

static void encodeBlob(const chassisParamStruct *params,
    chassisFlashBlobStruct *blob)
{
    memset(blob, 0, sizeof(*blob));
    blob->magic = CHASSIS_FLASH_MAGIC;
    blob->version = CHASSIS_FLASH_VERSION;
    blob->payloadBytes = sizeof(*blob);
    blob->params = *params;
    blob->crc = crc32leeeCalc((const uint8_t *) blob,
        (uint32_t) offsetof(chassisFlashBlobStruct, crc));
}

static chassisParamsSlotStatus mapStatus(paramSlotStatus status)
{
    return status == PARAM_SLOT_OK ? CHASSIS_PARAMS_SLOT_OK :
        CHASSIS_PARAMS_SLOT_ERROR;
}

void chassisParamsFlashLoad(chassisParamStruct *params)
{
    chassisFlashBlobStruct defaults;
    chassisFlashBlobStruct stored;
    if (params == NULL) return;
    chassisParamsSanitize(params);
    encodeBlob(params, &defaults);
    #if GLOBAL_PARAM_PERSIST_ENABLE

```

```

g_slotReady = (uint8_t) (paramSlotStoreInit(&g_slotStore,
sizeof(defaults), blobValid, &defaults) == PARAM_SLOT_OK);
if (g_slotReady != 0u)
{
    if (paramSlotStoreReadRuntime(&g_slotStore, &stored,
sizeof(stored)) == PARAM_SLOT_OK)
    {
        *params = stored.params;
        chassisParamsSanitize(params);
    }
}
#else
(void) stored;
g_slotReady = 0u;
#endif
}

void chassisParamsFlashSave(const chassisParamStruct *params)
{
    #if GLOBAL_PARAM_PERSIST_ENABLE
        (void) chassisParamsSaveRuntime(params);
    #else
        (void) params;
    #endif
}

void chassisParamsFlashErase(void)
{
    chassisParamStruct defaults;
    chassisParamsLoadDefaults(&defaults);
    chassisParamsSanitize(&defaults);
    (void) chassisParamsRestoreActiveSlotDefaults(&defaults);
}

uint8_t chassisParamsGetActiveSlot(void)
{
    return (g_slotReady != 0u) ? paramSlotStoreGetActive(&g_slotStore) : 0u;
}

chassisParamsSlotStatus chassisParamsReadSlot(uint8_t slot,
chassisParamStruct *params, chassisParamsSlotMeta *meta)
{
    chassisFlashBlobStruct blob;
    paramSlotMeta raw;
    if ((g_slotReady == 0u) || (params == NULL) ||
(paramSlotStoreRead(&g_slotStore, slot, &blob, sizeof(blob)) != PARAM_SLOT_OK))
        return CHASSIS_PARAMS_SLOT_ERROR;
    *params = blob.params;
    chassisParamsSanitize(params);
    if (meta != NULL)
    {
        if (paramSlotStoreGetMeta(&g_slotStore, slot, &raw) != PARAM_SLOT_OK)
            return CHASSIS_PARAMS_SLOT_ERROR;
        memcpy(meta, &raw, sizeof(*meta));
    }
    return CHASSIS_PARAMS_SLOT_OK;
}

chassisParamsSlotStatus chassisParamsPrepareSlot(uint8_t slot,
const chassisParamStruct *params, uint32_t transactionId)
{
    chassisFlashBlobStruct blob;
    chassisParamStruct sanitized;
    if ((g_slotReady == 0u) || (params == NULL)) return CHASSIS_PARAMS_SLOT_ERROR;
    sanitized = *params;
    chassisParamsSanitize(&sanitized);
    if (memcmp(params, &sanitized, sizeof(sanitized)) != 0)
        return CHASSIS_PARAMS_SLOT_ERROR;
    encodeBlob(&sanitized, &blob);
    return mapStatus(paramSlotStorePrepare(&g_slotStore, slot, &blob,
sizeof(blob), transactionId));
}

chassisParamsSlotStatus chassisParamsCommitPreparedSlot(uint8_t slot,
uint32_t transactionId, uint8_t activate)
{
    return mapStatus(paramSlotStoreCommit(&g_slotStore, slot,
transactionId, activate));
}

```



```

chassisParamsSlotStatus chassisParamsAbortPreparedSlot(uint32_t transactionId)
{
    return mapStatus(paramSlotStoreAbort(&g_slotStore, transactionId));
}

chassisParamsSlotStatus chassisParamsInspectSlot(uint8_t slot,
chassisParamsLibraryMeta *meta)
{
    paramSlotMeta raw;
    chassisFlashBlobStruct blob;
    paramSlotStatus status;
    if ((g_slotReady == 0u) || (meta == NULL))
        return CHASSIS_PARAMS_SLOT_ERROR;
    memset(meta, 0, sizeof(*meta));
    status = paramSlotStoreGetMeta(&g_slotStore, slot, &raw);
    if (status == PARAM_SLOT_EMPTY)
    {
        meta->state = CHASSIS_PARAMS_LIBRARY_EMPTY;
        return CHASSIS_PARAMS_SLOT_OK;
    }
    if (status != PARAM_SLOT_OK) return CHASSIS_PARAMS_SLOT_ERROR;
    meta->bank = raw.bank;
    meta->generation = raw.generation;
    meta->blobCrc = raw.blobCrc;
    meta->transactionId = raw.transactionId;
    status = paramSlotStoreRead(&g_slotStore, slot, &blob, sizeof(blob));
    if (status == PARAM_SLOT_OK) meta->state = CHASSIS_PARAMS_LIBRARY_VALID;
    else if ((status == PARAM_SLOT_INVALID) ||
             (status == PARAM_SLOT_VERIFY_FAILED))
        meta->state = CHASSIS_PARAMS_LIBRARY_INVALID;
    else return CHASSIS_PARAMS_SLOT_ERROR;
    return CHASSIS_PARAMS_SLOT_OK;
}

chassisParamsSlotStatus chassisParamsFinishPreparedSlot(uint32_t transactionId)
{
    return mapStatus(paramSlotStoreFinish(&g_slotStore, transactionId));
}

chassisParamsSlotStatus chassisParamsSaveSlot(uint8_t slot,
const chassisParamStruct *params, uint32_t transactionId)
{
    chassisFlashBlobStruct blob;
    chassisParamStruct sanitized;
    if ((g_slotReady == 0u) || (params == NULL)) return CHASSIS_PARAMS_SLOT_ERROR;
    sanitized = *params;
    chassisParamsSanitize(&sanitized);
    encodeBlob(&sanitized, &blob);
    return mapStatus(paramSlotStoreSave(&g_slotStore, slot, &blob,
sizeof(blob), transactionId, 0u));
}

chassisParamsSlotStatus chassisParamsActivateSlot(uint8_t slot,
chassisParamStruct *params)
{
    if (chassisParamsReadSlot(slot, params, NULL) != CHASSIS_PARAMS_SLOT_OK)
        return CHASSIS_PARAMS_SLOT_ERROR;
    return mapStatus(paramSlotStoreSetActive(&g_slotStore, slot));
}

chassisParamsSlotStatus chassisParamsActivatePreparedSlot(uint8_t slot,
uint32_t transactionId, chassisParamStruct *params)
{
    if (chassisParamsReadSlot(slot, params, NULL) != CHASSIS_PARAMS_SLOT_OK)
        return CHASSIS_PARAMS_SLOT_ERROR;
    return mapStatus(paramSlotStoreActivatePrepared(&g_slotStore, slot,
transactionId));
}

chassisParamsSlotStatus chassisParamsActivateSlotPending(uint8_t slot,
chassisParamStruct *params)
{
    if (chassisParamsReadSlot(slot, params, NULL) != CHASSIS_PARAMS_SLOT_OK)
        return CHASSIS_PARAMS_SLOT_ERROR;
    return mapStatus(paramSlotStoreSetActivePending(&g_slotStore, slot));
}

chassisParamsSlotStatus chassisParamsAbortPendingActivation(

```

```

    chassisParamStruct *params)
{
    if (params == NULL ||
        paramSlotStoreAbortPendingActivation(&g_slotStore) != PARAM_SLOT_OK)
        return CHASSIS_PARAMS_SLOT_ERROR;
    return chassisParamsReadRuntime(params, NULL);
}

chassisParamsSlotStatus chassisParamsFinishPendingActivation(void)
{
    return mapStatus(paramSlotStoreFinishPendingActivation(&g_slotStore));
}

chassisParamsSlotStatus chassisParamsRestoreActiveSlotDefaults(
    chassisParamStruct *params)
{
    if (params == NULL) return CHASSIS_PARAMS_SLOT_ERROR;
    chassisParamsLoadDefaults(params);
    chassisParamsSanitize(params);
    return chassisParamsSaveRuntime(params);
}

chassisParamsSlotStatus chassisParamsReadRuntime(chassisParamStruct *params,
    chassisParamsRuntimeMeta *meta)
{
    chassisFlashBlobStruct blob;
    paramRuntimeMeta raw;
    if ((g_slotReady == 0u) || (params == NULL) ||
        (paramSlotStoreReadRuntime(&g_slotStore, &blob, sizeof(blob)) !=
            PARAM_SLOT_OK)) return CHASSIS_PARAMS_SLOT_ERROR;
    *params = blob.params;
    chassisParamsSanitize(params);
    if (meta != NULL)
    {
        if (paramSlotStoreGetRuntimeMeta(&g_slotStore, &raw) != PARAM_SLOT_OK)
            return CHASSIS_PARAMS_SLOT_ERROR;
        fillRuntimeMeta(meta, &raw);
    }
    return CHASSIS_PARAMS_SLOT_OK;
}

chassisParamsSlotStatus chassisParamsGetRuntimeMeta(
    chassisParamsRuntimeMeta *meta)
{
    paramRuntimeMeta raw;
    if ((meta == NULL) ||
        (paramSlotStoreGetRuntimeMeta(&g_slotStore, &raw) != PARAM_SLOT_OK))
        return CHASSIS_PARAMS_SLOT_ERROR;
    fillRuntimeMeta(meta, &raw);
    return CHASSIS_PARAMS_SLOT_OK;
}

chassisParamsSlotStatus chassisParamsGetPending(
    chassisParamsPendingInfo *pending)
{
    paramSlotPendingInfo raw;
    paramSlotStatus status;
    if ((g_slotReady == 0u) || (pending == NULL))
        return CHASSIS_PARAMS_SLOT_ERROR;
    memset(pending, 0, sizeof(*pending));
    status = paramSlotStoreGetPending(&g_slotStore, &raw);
    if (status == PARAM_SLOT_EMPTY) return CHASSIS_PARAMS_SLOT_OK;
    if (status != PARAM_SLOT_OK) return CHASSIS_PARAMS_SLOT_ERROR;
    pending->valid = raw.valid;
    pending->active = raw.active;
    pending->activationOnly = raw.activationOnly;
    pending->slot = raw.slot;
    pending->transactionId = raw.transactionId;
    return CHASSIS_PARAMS_SLOT_OK;
}

chassisParamsSlotStatus chassisParamsSaveRuntime(
    const chassisParamStruct *params)
{
    chassisFlashBlobStruct blob;
    chassisParamStruct sanitized;
    if ((g_slotReady == 0u) || (params == NULL)) return CHASSIS_PARAMS_SLOT_ERROR;
    sanitized = *params;
    chassisParamsSanitize(&sanitized);

```

```

        encodeBlob(&sanitized, &blob);
        return mapStatus(paramSlotStoreSaveRuntime(&g_slotStore, &blob,
            sizeof(blob)));
    }

chassisParamsSlotStatus chassisParamsPrepareRuntime(
    const chassisParamStruct *params, uint32_t transactionId,
    uint8_t sourceSlot)
{
    chassisFlashBlobStruct blob;
    chassisParamStruct sanitized;
    if ((g_slotReady == 0u) || (params == NULL)) return CHASSIS_PARAMS_SLOT_ERROR;
    sanitized = *params;
    chassisParamsSanitize(&sanitized);
    if (memcmp(params, &sanitized, sizeof(sanitized)) != 0)
        return CHASSIS_PARAMS_SLOT_ERROR;
    encodeBlob(&sanitized, &blob);
    return mapStatus(paramSlotStorePrepareRuntime(&g_slotStore, &blob,
        sizeof(blob), transactionId, sourceSlot));
}

chassisParamsSlotStatus chassisParamsPrepareRuntimeFromSlot(uint8_t slot,
    uint32_t transactionId)
{
    return mapStatus(paramSlotStorePrepareRuntimeFromSlot(&g_slotStore,
        slot, transactionId));
}

chassisParamsSlotStatus chassisParamsApplyPreparedRuntime(
    uint32_t transactionId, chassisParamStruct *candidate)
{
    chassisFlashBlobStruct blob;
    if ((candidate == NULL) ||
        (paramSlotStoreReadPreparedRuntime(&g_slotStore, transactionId,
            &blob, sizeof(blob)) != PARAM_SLOT_OK))
        return CHASSIS_PARAMS_SLOT_ERROR;
    *candidate = blob.params;
    chassisParamsSanitize(candidate);
    return CHASSIS_PARAMS_SLOT_OK;
}

chassisParamsSlotStatus chassisParamsCommitRuntime(uint32_t transactionId)
{
    return mapStatus(paramSlotStoreCommitRuntime(&g_slotStore,
        transactionId));
}

```

core2_cyt4bb7/project/code/config/commonDef.h

SHA-256

:

a6838aeac9bdf807eea5044484c19e2c6b501f26dc0ce0f8df9291993402109c

代码 (c)

```

#ifndef COMMON_DEF_H
#define COMMON_DEF_H

#include <stdint.h>

#define range(x, min, max) (((x) < (min)) ? (min) : (((x) > (max)) ? (max) : (x)))
#define clampSym(x, limit) (range((x), -(limit), (limit))) /* 对称限幅：等价 range(x,-limit,+limit)，limit 须非负 */
#define unsignRange(x, max) (((x) > (max)) ? (max) : (x))
#define absf(x) (((x) < 0.0f) ? -(x) : (x)) /* 浮点绝对值（入参须无副作用，求值两次）*/

#define toRad(x) ((x) * 3.14159265358979323846f / 180.0f)
#define toDeg(x) ((x) * 180.0f / 3.14159265358979323846f)

#define PI (3.14159265358979323846f)
#define TWO_PI (2.0f * PI)
#define HALF_PI (0.5f * PI)

static inline uint8_t floatIsFinite(float v)
{
    return (uint8_t) ((v == v) && ((v - v) == 0.0f));
}

```

```
#endif
```

core2_cyt4bb7/project/code/config/configChassisPhysical.h

SHA-256

:

dc887a02ad3ea092653453e7d2211540781d5a2db8c095345c96fba8baea7d53

代码 (c)

```
#ifndef _CONFIG_CHASSIS_PHYSICAL_H_
#define _CONFIG_CHASSIS_PHYSICAL_H_

#include "configCore2Build.h"

/*
 * core2 底盘物理量、静态限幅、滤波与安全保护。
 * 这些宏不进入 chassisParams/flash，改动需重新编译并实车确认。
 */

#define CHASSIS_SPEED_SAMPLE_PERIOD_S (((float) CONTROL_PERIOD_US * (float) SPEED_LOOP_DIVIDER) / 1000000.0f)

#define CHASSIS_WHEEL_RADIUS      (0.05f)
#define CHASSIS_WHEEL_BASE       (0.169f)
#define CHASSIS_WHEEL_TRACK      (0.21f)
#define CHASSIS_ENCODER_PULSES_PER_REV (2385u)
#define CHASSIS_MECANUM_ROLLER_ANGLE_DEG (45u)

#define CHASSIS_RS485_MAX_LINEAR_SPEED_MPS (10.0f)
#define CHASSIS_RS485_MAX_YAW_RATE_RADPS  (50.0f)

#if (CHASSIS_ENCODER_PULSES_PER_REV == 0u)
#error "CHASSIS_ENCODER_PULSES_PER_REV must be > 0"
#endif

#if (CHASSIS_MECANUM_ROLLER_ANGLE_DEG != 45u)
#error "Current chassisMecanum equations assume 45-degree mecanum rollers"
#endif

#define MOTOR_CONTROL_CURRENT_LIMIT      (1000.0f)
#define MOTOR_CONTROL_OUTPUT_LIMIT      (1.0f)
#define MOTOR_CONTROL_SPEED_DERIVATIVE_LPF_ALPHA (0.1f)
#define MOTOR_CONTROL_SPEED_FEEDBACK_LPF_ALPHA  (1.0f)

#define MOTOR_RUNAWAY_SAT_THRESHOLD (0.999f)
#define MOTOR_RUNAWAY_TICKS      (14000u)

#define CHASSIS_ODOM_LPF_ALPHA (0.1f)

#define CURRENT_SENSE_FILTER_COUNT (4u)
#define CURRENT_SENSE_LPF_ALPHA  (0.1f)

#define IMU_SAMPLE_PERIOD_S (0.001f)

#endif /* _CONFIG_CHASSIS_PHYSICAL_H_ */
```

core2_cyt4bb7/project/code/config/configChassisPidDefaults.h

SHA-256

:

d71aba8d61ac0a44863036bd8e88704e58fa598990c661703e305ee941b1f269

代码 (c)

```
#ifndef _CONFIG_CHASSIS_PID_DEFAULTS_H_
#define _CONFIG_CHASSIS_PID_DEFAULTS_H_

/*
 * core2 底盘 PID 运行时默认值。
 * chassisParamsLoadDefaults 从这里加载，菜单/flash 可覆盖。
 */
```

```
#define MOTOR_CONTROL_SPEED_PID_INTEGRAL_MAX (100.0f)

#define MOTOR_CONTROL_FL_SPEED_KP (16.0f)
#define MOTOR_CONTROL_FL_SPEED_KI (0.012f)
#define MOTOR_CONTROL_FL_SPEED_KD (0.0f)

#define MOTOR_CONTROL_FR_SPEED_KP (16.0f)
#define MOTOR_CONTROL_FR_SPEED_KI (0.012f)
#define MOTOR_CONTROL_FR_SPEED_KD (0.0f)

#define MOTOR_CONTROL_BL_SPEED_KP (16.0f)
#define MOTOR_CONTROL_BL_SPEED_KI (0.012f)
#define MOTOR_CONTROL_BL_SPEED_KD (0.0f)

#define MOTOR_CONTROL_BR_SPEED_KP (16.0f)
#define MOTOR_CONTROL_BR_SPEED_KI (0.012f)
#define MOTOR_CONTROL_BR_SPEED_KD (0.0f)

#define MOTOR_CONTROL_CURRENT_PID_INTEGRAL_MAX (0.0f)

#define MOTOR_CONTROL_FL_CURRENT_KP (0.0004f)
#define MOTOR_CONTROL_FL_CURRENT_KI (0.0004f)
#define MOTOR_CONTROL_FL_CURRENT_KD (0.0f)

#define MOTOR_CONTROL_FR_CURRENT_KP (0.0004f)
#define MOTOR_CONTROL_FR_CURRENT_KI (0.0004f)
#define MOTOR_CONTROL_FR_CURRENT_KD (0.0f)

#define MOTOR_CONTROL_BL_CURRENT_KP (0.0004f)
#define MOTOR_CONTROL_BL_CURRENT_KI (0.0004f)
#define MOTOR_CONTROL_BL_CURRENT_KD (0.0f)

#define MOTOR_CONTROL_BR_CURRENT_KP (0.0004f)
#define MOTOR_CONTROL_BR_CURRENT_KI (0.0004f)
#define MOTOR_CONTROL_BR_CURRENT_KD (0.0f)

#define CHASSIS_YAW_RATE_KP      (6.0f)
#define CHASSIS_YAW_RATE_KI      (0.031f)
#define CHASSIS_YAW_RATE_KD      (0.0f)
#define CHASSIS_YAW_RATE_INTEGRAL_MAX  (15.0f)
#define CHASSIS_YAW_RATE_DERIVATIVE_LPF (1.0f)
#define CHASSIS_YAW_RATE_OUTPUT_LIMIT  (20.0f)

/* 零 wz 航向外环：纯 P，输出为角速度内环目标。 */
#define CHASSIS_HEADING_KP      (9.0f)
#define CHASSIS_HEADING_RATE_LIMIT_RADPS  (1.0f)
#define CHASSIS_HEADING_KP_MAX      (20.0f)
#define CHASSIS_HEADING_RATE_LIMIT_MAX_RADPS (50.0f)

#endif /* _CONFIG_CHASSIS_PID_DEFAULTS_H_ */
```

core2_cyt4bb7/project/code/config/configChassisPolarity.h

SHA-256 :
caabcb24505155493e67a1fb92fe16b0f4afee94361f15963328346c42ec5cac

代码 (c)

```
#ifndef _CONFIG_CHASSIS_POLARITY_H_
#define _CONFIG_CHASSIS_POLARITY_H_

/*
 * core2 极性符号单一真相源。
 * 约定：每轮物理前进 => 正占空比 + 正转速反馈 + 正电流反馈。
 */

#define MOTOR_FL_OUTPUT_DIR (+1.0f)
#define MOTOR_FR_OUTPUT_DIR (-1.0f)
#define MOTOR_BL_OUTPUT_DIR (+1.0f)
#define MOTOR_BR_OUTPUT_DIR (-1.0f)

#define ENCODER_FL_DIR (+1)
#define ENCODER_FR_DIR (-1)
#define ENCODER_BL_DIR (+1)
```

```
#define ENCODER_BR_DIR (-1)

#define CURRENT_SENSE_FL_DIR (+1)
#define CURRENT_SENSE_FR_DIR (-1)
#define CURRENT_SENSE_BL_DIR (+1)
#define CURRENT_SENSE_BR_DIR (-1)

#endif /* _CONFIG_CHASSIS_POLARITY_H_ */
```

core2_cyt4bb7/project/code/config/configChassisRemote.h

SHA-256 :
30f6027a69ab0b4a25f148a94ee533ab92e67f2c260f11062cff398293b887c0

代码 (c)

```
#ifndef _CONFIG_CHASSIS_REMOTE_H_
#define _CONFIG_CHASSIS_REMOTE_H_

/*
 * core2 LoRa 映射、NAV 回放与蜂鸣时长默认值。
 * LoRa V/Wz/死区与 NAV 纠正参数会进入 chassisParams，链路/按键/蜂鸣时长为静态项。
 */

#define LORA_STICK_RANGE      (2000)
#define LORA_STICK_DEADZONE  (80)
#define LORA_V_MAX_MPS       (1.60000 014f)
#define LORA_WZ_MAX_RADPS    (7.0f)
#define LORA_VX_INPUT_DIR    (+1.0f)
#define LORA_VY_INPUT_DIR    (+1.0f)
#define LORA_WZ_INPUT_DIR    (+1.0f)
#define LORA_LINK_TIMEOUT_MS (500u)
#define LORA_KEY_S4_INDEX    (2u)
#define LORA_KEY_S3_INDEX    (3u)
#define LORA_POLL_BYTE_BUDGET (64u)
#define LORA_CHORD_DEBOUNCE_MS (30u) /* 模式 4 本地解锁确认 */
#define LORA_LINKED_CHORD_CONFIRM_MS (200u) /* 模式 2/3 联机请求确认 */
#define LORA_CHORD_RELEASE_ARM_MS (30u) /* 上电/重连/触发后全松重装 */
#define LORA_CHORD_MIN_FRESH_FRAMES (3u) /* 候选必须来自至少 3 个真实新帧 */

#if (LORA_STICK_DEADZONE >= LORA_STICK_RANGE)
#error "LORA_STICK_DEADZONE must be < LORA_STICK_RANGE (stick mapping divides by their difference)"
#endif
#if (LORA_CHORD_DEBOUNCE_MS == 0u)
#error "LORA_CHORD_DEBOUNCE_MS must be > 0"
#endif
#if (LORA_CHORD_DEBOUNCE_MS > 1000u)
#error "LORA_CHORD_DEBOUNCE_MS must be <= 1000"
#endif
#if (LORA_LINKED_CHORD_CONFIRM_MS < LORA_CHORD_DEBOUNCE_MS)
#error "LORA_LINKED_CHORD_CONFIRM_MS must be >= standalone debounce"
#endif
#if (LORA_CHORD_RELEASE_ARM_MS == 0u)
#error "LORA_CHORD_RELEASE_ARM_MS must be > 0"
#endif
#if (LORA_CHORD_MIN_FRESH_FRAMES < 2u)
#error "LORA_CHORD_MIN_FRESH_FRAMES must be >= 2"
#endif

#define NAV_CORRECTION_KP      (0.05f)
#define NAV_CORRECTION_LIMIT   (8.0f)
#define NAV_ERROR_LIMIT_PULSES (600.0f)
#define NAV_RAMP_FRAMES        (100u)
#define LORA_INIT_VALID_TICKS  (2000u)
#define LORA_LINK_LOST_DISCARD_TICKS (4000u)

#define BEEP_TICKS_SHORT (200u)
#define BEEP_TICKS_LONG  (600u)

#endif /* _CONFIG_CHASSIS_REMOTE_H_ */
```

core2_cyt4bb7/project/code/config/configCore2Build.h

SHA-256 :

356ade570b4f43ad21eba8fb40cb1ff93f403f70b83f2ec57d1bd35dcbbcf4f6

代码 (c)

```
#ifndef _CONFIG_CORE2_BUILD_H_
#define _CONFIG_CORE2_BUILD_H_

#include <stdint.h>

/*
 * core2 调度时序与编译期模式。
 * 这里的宏不进入 chassisParams/flash，改动需重新编译。
 */

#define SAMPLE_PERIOD_US      (50u)
#define CONTROL_PERIOD_US    (500u)
#define SPEED_LOOP_DIVIDER    (4u)
#define YAW_RATE_LOOP_DIVIDER (3u)
#define LORA_TICK_DIVIDER     (2u)
#define IPS114_DISPLAY_PERIOD_MS (20u)
#define IPS114_LINE_HEIGHT    (16u)
#define CORE2_IPS114_DISPLAY_DIR IPS114_PORTAIT_180
#define LIGHT_STRIP_SAVE_DELAY_MS (500u)
#define KEY_TICK_DIVIDER      (2u)
#define IMU_TICK_DIVIDER      (2u)

/* RS-485 失链等待阈值：超时后进入模式 4 本地兜底。 */
#define AG_CAR_RS485_WAIT_MS (2000u)

#define CORE2_BUILD_PRODUCTION (0u)
#define CORE2_BUILD_TUNE      (1u)
#define CORE2_BUILD_MODE      CORE2_BUILD_PRODUCTION

#if (CORE2_BUILD_MODE == CORE2_BUILD_TUNE)
#define GLOBAL_PARAM_PERSIST_ENABLE (0u)
#else
#define GLOBAL_PARAM_PERSIST_ENABLE (1u)
#endif

#define MOTOR_TUNE_CASCADE (0u)
#define MOTOR_TUNE_LORA_TEST (1u)
#define MOTOR_TUNE_CURRENT (2u)
#define MOTOR_TUNE_OPEN_LOOP (3u)
#define MOTOR_TUNE_MODE    MOTOR_TUNE_CASCADE

#define MOTOR_TUNE_STEP_HIGH (60.0f)
#define MOTOR_TUNE_STEP_LOW (20.0f)
#define MOTOR_TUNE_STEP_INTERVAL_S (2.5f)
#define MOTOR_TUNE_OPEN_LOOP_DUTY (0.15f)
#define MOTOR_TUNE_WHEEL_SELECT (3u)

#endif /* _CONFIG_CORE2_BUILD_H_ */
```

core2_cyt4bb7/project/code/config/configSerialDebug.h

SHA-256 :

05082182ee930f412c2c2af6abb81ffc03990535805a7943a60c3e0041a3c157

代码 (c)

```
#ifndef _CONFIG_SERIAL_DEBUG_H_
#define _CONFIG_SERIAL_DEBUG_H_

/* 逐飞底层断言/异常文本串口配置；不承载项目级周期调试流。 */

/* 默认 debug UART: 若改串口且保留中断接收, 需同步更换 zf_common_debug 的中断 handler。 */
#define SERIAL_DEBUG_UART_INDEX (UART_0)
```

```

#define SERIAL_DEBUG_UART_BAUDRATE    (1000000)
#define SERIAL_DEBUG_UART_TX_PIN      (UART0_TX_P00_1)
#define SERIAL_DEBUG_UART_RX_PIN      (UART0_RX_P00_0)
#define SERIAL_DEBUG_UART_USE_INTERRUPT (1)

/* 数值可比较的编译期保护；UART 枚举不在预处理期比较。 */
#if (SERIAL_DEBUG_UART_BAUDRATE < 1)
#error "SERIAL_DEBUG_UART_BAUDRATE must be > 0"
#endif

#endif /* _CONFIG_SERIAL_DEBUG_H_ */

```

core2_cyt4bb7/project/code/config/pinMap.h

SHA-256

:

ecbb143c2f83532fc01987b600e91181c14a26dbf2ab1353f8c6aac884a106ab

代码 (c)

```

#ifndef _PIN_MAP_H_
#define _PIN_MAP_H_

/*
 * core2 麦轮车 MCU 引脚集中表（对标 core0 pinMap.h）
 *
 * 集中原散落在 motor.c / encoder.c / currentSense.c / menuKey.c /
 * loraNavControl.c / lightStrip.h 的引脚定义。
 * 零依赖纯 #define：消费 .c 各自 include 对应 zf_driver / zf_device 头展开 token
 * （TCPWM/ADC/编码器/GPIO 枚举值由各驱动头提供，本文件不引头）。
 *
 * 非引脚的频率/步长/时序参数不在此（在 configCore2Build.h / 其它参数子头或各模块本地）。
 */

/* =====
 * 电机驱动（4 路：使能 PWM + 方向 GPIO）
 * ===== */
#define MOTOR_FL_EN_PWM (TCPWM_CH11_P05_2)
#define MOTOR_FR_EN_PWM (TCPWM_CH12_P05_3)
#define MOTOR_BL_EN_PWM (TCPWM_CH30_P10_2)
#define MOTOR_BR_EN_PWM (TCPWM_CH25_P09_1)

#define MOTOR_FL_PH_PIN (P05_0)
#define MOTOR_FR_PH_PIN (P05_1)
#define MOTOR_BL_PH_PIN (P10_3)
#define MOTOR_BR_PH_PIN (P09_0)

/* =====
 * 正交编码器（4 路：通道索引 + A/B 相引脚）
 * ===== */
#define ENCODER_FL_INDEX (TC_CH07_ENCODER)
#define ENCODER_FR_INDEX (TC_CH20_ENCODER)
#define ENCODER_BL_INDEX (TC_CH27_ENCODER)
#define ENCODER_BR_INDEX (TC_CH58_ENCODER)

#define ENCODER_FL_CH1 (TC_CH07_ENCODER_CH1_P07_6)
#define ENCODER_FL_CH2 (TC_CH07_ENCODER_CH2_P07_7)
#define ENCODER_FR_CH1 (TC_CH20_ENCODER_CH1_P08_1)
#define ENCODER_FR_CH2 (TC_CH20_ENCODER_CH2_P08_2)
#define ENCODER_BL_CH1 (TC_CH27_ENCODER_CH1_P19_2)
#define ENCODER_BL_CH2 (TC_CH27_ENCODER_CH2_P19_3)
#define ENCODER_BR_CH1 (TC_CH58_ENCODER_CH1_P17_3)
#define ENCODER_BR_CH2 (TC_CH58_ENCODER_CH2_P17_4)

/* =====
 * 电机电流采样 ADC（4 路）
 * ===== */
#define CURRENT_SENSE_FL_ADC (ADC0_CH01_P06_1)
#define CURRENT_SENSE_FR_ADC (ADC0_CH00_P06_0)
#define CURRENT_SENSE_BL_ADC (ADC1_CH09_P12_5)
#define CURRENT_SENSE_BR_ADC (ADC1_CH08_P12_4)

/* =====
 * 板载 5 键（GPI + 上拉，按下=LOW；丝印顺序与 coreT1/coreT2 反向）
 * ===== */
#define KEY_INPUT_KEY1_PIN (P19_4)

```



```

#define KEY_INPUT_KEY2_PIN (P20_0)
#define KEY_INPUT_KEY3_PIN (P20_1)
#define KEY_INPUT_KEY4_PIN (P20_2)
#define KEY_INPUT_KEY5_PIN (P20_3)

/* =====
* 蜂鸣器 (NPN S8050 高电平驱动) + 灯光 PWM
* ===== */
#define BEEP_PIN          (P07_2)
#define LIGHT_STRIP_PWM_CHANNEL (TCPWM_CH19_P08_0) /* TODO 硬件确认后改实际通道 */

/* =====
* RS-485 收发器 THVD1450 (UART_4, 与 core1 通信)
* 引脚实际定义在 THVD1450.c 内; 两核 THVD 协议层同源, 驱动因 RX 并发模型不同而有意分叉;
* 此处集中登记便于核对: RE#/DE=P13_7、RX=P14_0、TX=P14_1。
* ===== */
#define RS485_DIR_PIN (P13_7) /* RE#/DE 方向控制 (高=发送/低=接收) */
#define RS485_RX_PIN  (P14_0) /* UART4 RX */
#define RS485_TX_PIN  (P14_1) /* UART4 TX */

#endif /* _PIN_MAP_H_ */

```

core2_cyt4bb7/project/code/control/chassisApplication.c

SHA-256

:

683a53b024a161e27cab95c1cf1867afc39d2023b66908963c40aa77869028db

代码 (c)

```

#include "chassisApplication.h"

#include <math.h>

#include "commonDef.h"
#include "configChassisPhysical.h"
#include "configChassisPidDefaults.h"
#include "lowPassFilter.h"
#include "motorControl.h"
#include "PID.h"
#include "zf_common_interrupt.h" /* CPUIntIdx2 命令锁 + snapshot DMB */

#define CHASSIS_IMU_DISPLAY_STALE_FEEDBACK_LIMIT (20u) /* 最快 500us 反馈拍下至少覆盖 10ms */
#define CHASSIS_IMU_PRODUCER_STALE_FEEDBACK_LIMIT (20u) /* 成功读到同一 seq; 仅诊断, 不停车 */
#define CHASSIS_ATTITUDE_FAULT_HOLD_SAMPLES (700u) /* 1ms IMU 样本, 覆盖多次 200ms STATUS 发布 */

typedef struct
{
    chassisMecanumConfigStruct config;
    chassisMotionCommandStruct targetCommand;
    wheelSpeedStruct baseWheelSpeed;
    wheelSpeedStruct targetWheelSpeed;
    wheelSpeedStruct feedbackWheelSpeed;
    encoderDataStruct encoderData;
    imuSnapshotStruct imu;
    PID yawRatePid;
    float yawCorrection;
    float headingKp;
    float headingRateLimit;
    float headingReference;
    float headingError;
    float headingRateTarget;
    float yawRateOutputLimit;
    uint8_t yawRateEnabled;
    uint8_t yawControlMode;
    uint8_t headingReferenceValid;
    uint8_t imuDisplayStaleFeedbackCount;
    uint8_t imuProducerStaleFeedbackCount;
    uint16_t attitudeFaultHoldSamples;
    uint8_t attitudeFaultActive;
    uint32_t lastImuSeq;
    uint32_t lastImuFaultSeq;
    chassisSpeedStruct actualSpeed;
    LowPassFilterData odomVxLpf;
    LowPassFilterData odomVyLpf;
    LowPassFilterData odomYawRateLpf;

```

```

    uint32_t snapshotSeq;
} chassisApplicationContextStruct;

static chassisApplicationContextStruct chassisApplicationContext = {0};
static volatile chassisApplicationSnapshotStruct chassisApplicationPublished = {0};
static volatile uint32_t chassisApplicationPublishGeneration = 0u;

static void chassisApplicationPublish(void);
static void chassisApplicationSetMotionCommandInternal(chassisMotionCommandStruct command);
static void chassisApplicationResetYawState(void);
static void chassisApplicationApplyYawCorrection(wheelSpeedStruct baseWheelSpeed);
static uint32_t chassisApplicationCommandLock(void);
static void chassisApplicationCommandUnlock(uint32_t irqWasEnabled);

static float chassisApplicationWrapPi(float angle)
{
    const float pi = 3.14159265358979323846f;
    const float twoPi = 6.28318530717958647692f;
    while (angle > pi) angle -= twoPi;
    while (angle < -pi) angle += twoPi;
    return angle;
}

static chassisMotionCommandStruct chassisApplicationLimitCommand(
    chassisMotionCommandStruct command,
    float maxLinearSpeedMps,
    float maxYawRateRadps)
{
    if (!isfinite(command.vx) || !isfinite(command.vy) || !isfinite(command.wz))
    {
        command.vx = 0.0f;
        command.vy = 0.0f;
        command.wz = 0.0f;
    }

    if (maxLinearSpeedMps <= 0.0f)
    {
        command.vx = 0.0f;
        command.vy = 0.0f;
    }
    else
    {
        command.vx = clampSym(command.vx, maxLinearSpeedMps);
        command.vy = clampSym(command.vy, maxLinearSpeedMps);
    }

    if (maxYawRateRadps <= 0.0f)
    {
        command.wz = 0.0f;
    }
    else
    {
        command.wz = clampSym(command.wz, maxYawRateRadps);
    }
    return command;
}

uint8_t chassisApplicationInit(void)
{
    const chassisApplicationParamViewStruct params = {
        CHASSIS_YAW_RATE_KP,
        CHASSIS_YAW_RATE_KI,
        CHASSIS_YAW_RATE_KD,
        CHASSIS_YAW_RATE_INTEGRAL_MAX,
        CHASSIS_HEADING_KP,
        CHASSIS_HEADING_RATE_LIMIT_RADPS,
        CHASSIS_YAW_RATE_OUTPUT_LIMIT};
    return chassisApplicationApplyParams(&params);
}

uint8_t chassisApplicationApplyParams(const chassisApplicationParamViewStruct *params)
{
    chassisMecanumConfigStruct config;

    if (params == 0) return 1u;

    chassisMecanumConfigInit(
        &config,
        CHASSIS_WHEEL_RADIUS,

```

```

    CHASSIS_WHEEL_BASE,
    CHASSIS_WHEEL_TRACK,
    CHASSIS_ENCODER_PULSES_PER_REV,
    CHASSIS_SPEED_SAMPLE_PERIOD_S,
    CHASSIS_RS485_MAX_LINEAR_SPEED_MPS);
chassisApplicationContext.config = config;

pidInit(
    &chassisApplicationContext.yawRatePid,
    params->yawRateIntegralMax,
    0.0f,
    params->yawRateKp,
    params->yawRateKi,
    params->yawRateKd,
    CHASSIS_YAW_RATE_DERIVATIVE_LPF);
chassisApplicationContext.yawCorrection = 0.0f;
chassisApplicationContext.headingKp = params->headingKp;
chassisApplicationContext.headingRateLimit = params->headingRateLimit;
chassisApplicationContext.yawRateOutputLimit = params->yawRateOutputLimit;
chassisApplicationContext.yawRateEnabled = 0u;
chassisApplicationContext.yawControlMode = (uint8_t) CHASSIS_YAW_DISABLED;
chassisApplicationContext.headingReferenceValid = 0u;
chassisApplicationContext.imuDisplayStaleFeedbackCount =
    CHASSIS_IMU_DISPLAY_STALE_FEEDBACK_LIMIT;
chassisApplicationContext.imuProducerStaleFeedbackCount = 0u;
chassisApplicationContext.attitudeFaultHoldSamples = 0u;
chassisApplicationContext.attitudeFaultActive = 0u;
chassisApplicationContext.lastImuSeq = chassisApplicationContext.imu.seq;
chassisApplicationContext.lastImuFaultSeq = chassisApplicationContext.imu.faultSeq;

/* 里程计合成速度低通器（遥测用，0.1 系数） */
lowPassFilterInit(&chassisApplicationContext.odomVxLpf, CHASSIS_ODOM_LPF_ALPHA);
lowPassFilterInit(&chassisApplicationContext.odomVyLpf, CHASSIS_ODOM_LPF_ALPHA);
lowPassFilterInit(&chassisApplicationContext.odomYawRateLpf, CHASSIS_ODOM_LPF_ALPHA);
chassisApplicationContext.actualSpeed.vx = 0.0f;
chassisApplicationContext.actualSpeed.vy = 0.0f;
chassisApplicationContext.actualSpeed.wz = 0.0f;

chassisApplicationStop();

return 0u;
}

void chassisApplicationApplyYawOutputLimit(float outputLimit)
{
    chassisApplicationContext.yawRateOutputLimit = (outputLimit > 0.0f) ? outputLimit : 0.0f;
}

void chassisApplicationSetMotionCommand(chassisMotionCommandStruct command)
{
    uint32_t irqWasEnabled = chassisApplicationCommandLock();
    chassisApplicationSetMotionCommandInternal(command);
    chassisApplicationPublish();
    chassisApplicationCommandUnlock(irqWasEnabled);
}

void chassisApplicationApplyWheelTarget(wheelSpeedStruct targetSpeed)
{
    uint32_t irqWasEnabled = chassisApplicationCommandLock();
    chassisApplicationContext.baseWheelSpeed = targetSpeed;
    chassisApplicationContext.targetWheelSpeed = targetSpeed;
    motorControlSetWheelSpeed(targetSpeed);
    chassisApplicationPublish();
    chassisApplicationCommandUnlock(irqWasEnabled);
}

void chassisApplicationApplyReplayWheelTarget(
    wheelSpeedStruct baseTarget,
    wheelSpeedStruct encoderCorrection,
    float wz)
{
    float rotationProjection;
    chassisMotionCommandStruct command = {0.0f, 0.0f, wz};
    wheelSpeedStruct translationalCorrection;
    uint32_t irqWasEnabled = chassisApplicationCommandLock();

    if (!isfinite(wz) ||
        !isfinite(baseTarget.fl) || !isfinite(baseTarget.fr) ||
        !isfinite(baseTarget.bl) || !isfinite(baseTarget.br) ||

```

```

    !isfinite(encoderCorrection.fl) || !isfinite(encoderCorrection.fr) ||
    !isfinite(encoderCorrection.bl) || !isfinite(encoderCorrection.br))
{
    chassisApplicationResetYawState();
    chassisApplicationContext.baseWheelSpeed.fl = 0.0f;
    chassisApplicationContext.baseWheelSpeed.fr = 0.0f;
    chassisApplicationContext.baseWheelSpeed.bl = 0.0f;
    chassisApplicationContext.baseWheelSpeed.br = 0.0f;
    chassisApplicationContext.targetWheelSpeed = chassisApplicationContext.baseWheelSpeed;
    motorControlStopAll();
    chassisApplicationPublish();
    chassisApplicationCommandUnlock(irqWasEnabled);
    return;
}

/* 剔除四轮纠偏中的旋转模式 (-,+,-,+), 由当前 yaw owner 独占旋转。 */
rotationProjection = (-encoderCorrection.fl + encoderCorrection.fr -
    encoderCorrection.bl + encoderCorrection.br) * 0.25f;
translationalCorrection.fl = encoderCorrection.fl + rotationProjection;
translationalCorrection.fr = encoderCorrection.fr - rotationProjection;
translationalCorrection.bl = encoderCorrection.bl + rotationProjection;
translationalCorrection.br = encoderCorrection.br - rotationProjection;
chassisApplicationSetMotionCommandInternal(command);
baseTarget.fl += translationalCorrection.fl;
baseTarget.fr += translationalCorrection.fr;
baseTarget.bl += translationalCorrection.bl;
baseTarget.br += translationalCorrection.br;
chassisApplicationApplyYawCorrection(baseTarget);
chassisApplicationPublish();
chassisApplicationCommandUnlock(irqWasEnabled);
}

void chassisApplicationStop(void)
{
    chassisMotionCommandStruct stopCommand = {0.0f, 0.0f, 0.0f};

    chassisApplicationSetMotionCommandInternal(stopCommand);
    chassisApplicationContext.targetWheelSpeed.fl = 0.0f;
    chassisApplicationContext.targetWheelSpeed.fr = 0.0f;
    chassisApplicationContext.targetWheelSpeed.bl = 0.0f;
    chassisApplicationContext.targetWheelSpeed.br = 0.0f;

    chassisApplicationContext.baseWheelSpeed = chassisApplicationContext.targetWheelSpeed;
    chassisApplicationResetYawState();
    chassisApplicationPublish();
}

void chassisApplicationUpdateFeedback(void)
{
    imuSnapshotStruct imuSnapshot = chassisApplicationContext.imu;
    uint8_t snapshotRead;
    uint8_t sequenceAdvanced;
    uint8_t faultAdvanced;
    uint32_t sequenceAdvanceCount;

    chassisApplicationContext.encoderData = encoderSample();
    chassisApplicationContext.feedbackWheelSpeed = motorControlUpdateSpeedFeedback(chassisApplicationContext.encoderData);

    snapshotRead = IMUTryGetSnapshot(&imuSnapshot);
    sequenceAdvanced = (uint8_t) (snapshotRead != 0u &&
        imuSnapshot.seq != chassisApplicationContext.lastImuSeq);
    sequenceAdvanceCount = sequenceAdvanced ?
        (uint32_t) (imuSnapshot.seq - chassisApplicationContext.lastImuSeq) : 0u;
    faultAdvanced = (uint8_t) (snapshotRead != 0u &&
        imuSnapshot.faultSeq != chassisApplicationContext.lastImuFaultSeq);
    if (sequenceAdvanced != 0u)
    {
        chassisApplicationContext.imuDisplayStaleFeedbackCount = 0u;
    }
    else if (chassisApplicationContext.imuDisplayStaleFeedbackCount <
        CHASSIS_IMU_DISPLAY_STALE_FEEDBACK_LIMIT)
    {
        chassisApplicationContext.imuDisplayStaleFeedbackCount++;
    }
    if (sequenceAdvanced != 0u)
    {
        chassisApplicationContext.imuProducerStaleFeedbackCount = 0u;
    }
    else if ((snapshotRead != 0u) &&

```

```

        (chassisApplicationContext.imuProducerStaleFeedbackCount <
         CHASSIS_IMU_PRODUCER_STALE_FEEDBACK_LIMIT))
    {
        chassisApplicationContext.imuProducerStaleFeedbackCount++;
    }
    imuSnapshot.fresh = sequenceAdvanced;
    imuSnapshot.healthy = (uint8_t) (imuSnapshot.valid != 0u &&
        faultAdvanced == 0u && isfinite(imuSnapshot.yawAngle) &&
        isfinite(imuSnapshot.yawRate) &&
        chassisApplicationContext.imuProducerStaleFeedbackCount <
        CHASSIS_IMU_PRODUCER_STALE_FEEDBACK_LIMIT);
    if (snapshotRead != 0u)
    {
        chassisApplicationContext.lastImuSeq = imuSnapshot.seq;
        chassisApplicationContext.lastImuFaultSeq = imuSnapshot.faultSeq;
    }
    chassisApplicationContext.imu = imuSnapshot;
    if ((snapshotRead != 0u) &&
        ((imuSnapshot.valid == 0u) || (faultAdvanced != 0u) ||
         !isfinite(imuSnapshot.yawAngle) || !isfinite(imuSnapshot.yawRate) ||
         chassisApplicationContext.imuProducerStaleFeedbackCount >=
         CHASSIS_IMU_PRODUCER_STALE_FEEDBACK_LIMIT))
    {
        chassisApplicationContext.attitudeFaultHoldSamples =
            CHASSIS_ATTITUDE_FAULT_HOLD_SAMPLES;
        /* 只撤销依赖姿态的闭环历史；合法底盘运动目标和电机授权保持。*/
        chassisApplicationResetYawState();
    }
    else if ((sequenceAdvanced != 0u) &&
        (chassisApplicationContext.attitudeFaultHoldSamples != 0u))
    {
        if (sequenceAdvanceCount >=
            (uint32_t) chassisApplicationContext.attitudeFaultHoldSamples)
        {
            chassisApplicationContext.attitudeFaultHoldSamples = 0u;
        }
        else
        {
            chassisApplicationContext.attitudeFaultHoldSamples -=
                (uint16_t) sequenceAdvanceCount;
        }
    }
    chassisApplicationContext.attitudeFaultActive = (uint8_t) (
        (imuSnapshot.valid == 0u) ||
        (chassisApplicationContext.attitudeFaultHoldSamples != 0u));

    /* 编码器正解算合成底盘实速(vx/vy) + 陀螺 yawRate(wz)，各过 0.1 低通后存 actualSpeed（遥测用）。
     * wz 取陀螺 yawRate（非正解算的 wz）——遥测以陀螺角速度为准；低通不污染控制用 imu.yawRate。*/
    {
        chassisMecanumConfigStruct configSnapshot = chassisApplicationContext.config;
        wheelSpeedStruct fbSnapshot = chassisApplicationContext.feedbackWheelSpeed;
        chassisSpeedStruct odom = chassisMecanumForward(&configSnapshot, fbSnapshot);
        lowPassFilter(&chassisApplicationContext.odomVxLpf, odom.vx);
        lowPassFilter(&chassisApplicationContext.odomVyLpf, odom.vy);
        lowPassFilter(&chassisApplicationContext.odomYawRateLpf,
            isfinite(imuSnapshot.yawRate) ? imuSnapshot.yawRate : 0.0f);
        chassisApplicationContext.actualSpeed.vx = chassisApplicationContext.odomVxLpf.filteredValue;
        chassisApplicationContext.actualSpeed.vy = chassisApplicationContext.odomVyLpf.filteredValue;
        chassisApplicationContext.actualSpeed.wz = chassisApplicationContext.odomYawRateLpf.filteredValue;
    }
    chassisApplicationPublish();
}

void chassisApplicationGetSnapshot(chassisApplicationSnapshotStruct *out)
{
    uint8_t attempt;
    chassisApplicationSnapshotStruct snapshot = {0};
    const chassisApplicationSnapshotStruct invalid = {0};

    if (0 == out) return;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        uint32_t before = chassisApplicationPublishGeneration;
        uint32_t after;
        if ((before & 1u) != 0u) continue;
        __DMB();
        snapshot.targetCommand.vx = chassisApplicationPublished.targetCommand.vx;
        snapshot.targetCommand.vy = chassisApplicationPublished.targetCommand.vy;
        snapshot.targetCommand.wz = chassisApplicationPublished.targetCommand.wz;
    }
}

```

```

snapshot.baseWheelSpeed.fl = chassisApplicationPublished.baseWheelSpeed.fl;
snapshot.baseWheelSpeed.fr = chassisApplicationPublished.baseWheelSpeed.fr;
snapshot.baseWheelSpeed.bl = chassisApplicationPublished.baseWheelSpeed.bl;
snapshot.baseWheelSpeed.br = chassisApplicationPublished.baseWheelSpeed.br;
snapshot.targetWheelSpeed.fl = chassisApplicationPublished.targetWheelSpeed.fl;
snapshot.targetWheelSpeed.fr = chassisApplicationPublished.targetWheelSpeed.fr;
snapshot.targetWheelSpeed.bl = chassisApplicationPublished.targetWheelSpeed.bl;
snapshot.targetWheelSpeed.br = chassisApplicationPublished.targetWheelSpeed.br;
snapshot.feedbackWheelSpeed.fl = chassisApplicationPublished.feedbackWheelSpeed.fl;
snapshot.feedbackWheelSpeed.fr = chassisApplicationPublished.feedbackWheelSpeed.fr;
snapshot.feedbackWheelSpeed.bl = chassisApplicationPublished.feedbackWheelSpeed.bl;
snapshot.feedbackWheelSpeed.br = chassisApplicationPublished.feedbackWheelSpeed.br;
snapshot.encoderData.flCount = chassisApplicationPublished.encoderData.flCount;
snapshot.encoderData.frCount = chassisApplicationPublished.encoderData.frCount;
snapshot.encoderData.blCount = chassisApplicationPublished.encoderData.blCount;
snapshot.encoderData.brCount = chassisApplicationPublished.encoderData.brCount;
snapshot.imu = chassisApplicationPublished.imu;
snapshot.actualSpeed.vx = chassisApplicationPublished.actualSpeed.vx;
snapshot.actualSpeed.vy = chassisApplicationPublished.actualSpeed.vy;
snapshot.actualSpeed.wz = chassisApplicationPublished.actualSpeed.wz;
snapshot.yawCorrection = chassisApplicationPublished.yawCorrection;
snapshot.headingReference = chassisApplicationPublished.headingReference;
snapshot.headingError = chassisApplicationPublished.headingError;
snapshot.headingRateTarget = chassisApplicationPublished.headingRateTarget;
snapshot.yawRateEnabled = chassisApplicationPublished.yawRateEnabled;
snapshot.yawControlMode = chassisApplicationPublished.yawControlMode;
snapshot.attitudeHealthy = chassisApplicationPublished.attitudeHealthy;
snapshot.attitudeDataFresh = chassisApplicationPublished.attitudeDataFresh;
snapshot.attitudeFaultActive = chassisApplicationPublished.attitudeFaultActive;
snapshot.seq = chassisApplicationPublished.seq;
snapshot.valid = chassisApplicationPublished.valid;
__DMB();
after = chassisApplicationPublishGeneration;
if ((before == after) && ((after & 1u) == 0u))
{
    *out = snapshot;
    return;
}
*out = invalid;
}

void chassisApplicationApplyCommandWithLimit(
    chassisMotionCommandStruct command,
    float maxLinearSpeedMps,
    float maxYawRateRadps)
{
    chassisMecanumConfigStruct configSnapshot = chassisApplicationContext.config;
    wheelSpeedStruct wheelSpeed;
    chassisMotionCommandStruct limitedCommand;

    limitedCommand = chassisApplicationLimitCommand(command, maxLinearSpeedMps, maxYawRateRadps);
    chassisApplicationSetMotionCommandInternal(limitedCommand);
    chassisMecanumConfigSetMaxLinearSpeed(&configSnapshot, maxLinearSpeedMps);

    /* 非零 wz 保留旋转前馈；零 wz 只保留平移，由 heading 外环独占旋转。 */
    wheelSpeed = chassisMecanumInverse(&configSnapshot, limitedCommand);
    chassisApplicationApplyYawCorrection(wheelSpeed);
    chassisApplicationPublish();
}

void chassisApplicationYawRateTick(void)
{
    float targetYawRate;
    float feedbackYawRate;
    uint8_t desiredMode;

    if (!chassisApplicationContext.yawRateEnabled)
    {
        chassisApplicationResetYawState();
        return;
    }
    if (chassisApplicationContext.imu.healthy == 0u ||
        !isfinite(chassisApplicationContext.imu.yawAngle) ||
        !isfinite(chassisApplicationContext.imu.yawRate))
    {
        chassisApplicationResetYawState();
        return;
    }
}

```

```

desiredMode = (chassisApplicationContext.targetCommand.wz == 0.0f) ?
(uint8_t) CHASSIS_YAW_HEADING : (uint8_t) CHASSIS_YAW_RATE;
if (desiredMode != chassisApplicationContext.yawControlMode)
{
    pidReset(&chassisApplicationContext.yawRatePid);
    chassisApplicationContext.yawCorrection = 0.0f;
    chassisApplicationContext.headingError = 0.0f;
    chassisApplicationContext.headingRateTarget = 0.0f;
    if (desiredMode == (uint8_t) CHASSIS_YAW_HEADING)
    {
        chassisApplicationContext.headingReference =
            chassisApplicationContext.imu.yawAngle;
        chassisApplicationContext.headingReferenceValid = 1u;
    }
    else
    {
        chassisApplicationContext.headingReferenceValid = 0u;
    }
    chassisApplicationContext.yawControlMode = desiredMode;
}

if (desiredMode == (uint8_t) CHASSIS_YAW_HEADING)
{
    if (chassisApplicationContext.headingReferenceValid == 0u)
    {
        chassisApplicationContext.headingReference =
            chassisApplicationContext.imu.yawAngle;
        chassisApplicationContext.headingReferenceValid = 1u;
    }
    chassisApplicationContext.headingError = chassisApplicationWrapPi(
        chassisApplicationContext.headingReference -
        chassisApplicationContext.imu.yawAngle);
    chassisApplicationContext.headingRateTarget = clampSym(
        chassisApplicationContext.headingKp *
        chassisApplicationContext.headingError,
        chassisApplicationContext.headingRateLimit);
    targetYawRate = chassisApplicationContext.headingRateTarget;
}
else
{
    chassisApplicationContext.headingError = 0.0f;
    chassisApplicationContext.headingRateTarget =
        chassisApplicationContext.targetCommand.wz;
    targetYawRate = chassisApplicationContext.targetCommand.wz;
}
feedbackYawRate = chassisApplicationContext.imu.yawRate;

chassisApplicationContext.yawRatePid.expectedValue = targetYawRate;
pidProcess(&chassisApplicationContext.yawRatePid, feedbackYawRate);
chassisApplicationContext.yawCorrection = clampSym(
    chassisApplicationContext.yawRatePid.controlValue,
    chassisApplicationContext.yawRateOutputLimit);
}

void chassisApplicationSetYawRateEnable(uint8_t enable)
{
    uint8_t normalized = enable ? 1u : 0u;

    if (normalized == chassisApplicationContext.yawRateEnabled) return;
    chassisApplicationResetYawState();
    chassisApplicationContext.yawRateEnabled = normalized;
    chassisApplicationPublish();
}

static void chassisApplicationSetMotionCommandInternal(chassisMotionCommandStruct command)
{
    chassisApplicationContext.targetCommand = command;
}

static void chassisApplicationResetYawState(void)
{
    pidReset(&chassisApplicationContext.yawRatePid);
    chassisApplicationContext.yawCorrection = 0.0f;
    chassisApplicationContext.headingReference = 0.0f;
    chassisApplicationContext.headingError = 0.0f;
    chassisApplicationContext.headingRateTarget = 0.0f;
    chassisApplicationContext.headingReferenceValid = 0u;
    chassisApplicationContext.yawControlMode = (uint8_t) CHASSIS_YAW_DISABLED;
}

```

```

}

static void chassisApplicationApplyYawCorrection(wheelSpeedStruct baseWheelSpeed)
{
    float yawCorr = chassisApplicationContext.yawRateEnabled ?
        chassisApplicationContext.yawCorrection : 0.0f;
    chassisApplicationContext.baseWheelSpeed = baseWheelSpeed;
    baseWheelSpeed.fl -= yawCorr;
    baseWheelSpeed.fr += yawCorr;
    baseWheelSpeed.bl -= yawCorr;
    baseWheelSpeed.br += yawCorr;
    chassisApplicationContext.targetWheelSpeed = baseWheelSpeed;
    motorControlSetWheelSpeed(baseWheelSpeed);
}

static void chassisApplicationPublish(void)
{
    chassisApplicationContext.snapshotSeq++;
    chassisApplicationPublishGeneration++;
    __DMB();
    chassisApplicationPublished.targetCommand.vx = chassisApplicationContext.targetCommand.vx;
    chassisApplicationPublished.targetCommand.vy = chassisApplicationContext.targetCommand.vy;
    chassisApplicationPublished.targetCommand.wz = chassisApplicationContext.targetCommand.wz;
    chassisApplicationPublished.baseWheelSpeed.fl = chassisApplicationContext.baseWheelSpeed.fl;
    chassisApplicationPublished.baseWheelSpeed.fr = chassisApplicationContext.baseWheelSpeed.fr;
    chassisApplicationPublished.baseWheelSpeed.bl = chassisApplicationContext.baseWheelSpeed.bl;
    chassisApplicationPublished.baseWheelSpeed.br = chassisApplicationContext.baseWheelSpeed.br;
    chassisApplicationPublished.targetWheelSpeed.fl = chassisApplicationContext.targetWheelSpeed.fl;
    chassisApplicationPublished.targetWheelSpeed.fr = chassisApplicationContext.targetWheelSpeed.fr;
    chassisApplicationPublished.targetWheelSpeed.bl = chassisApplicationContext.targetWheelSpeed.bl;
    chassisApplicationPublished.targetWheelSpeed.br = chassisApplicationContext.targetWheelSpeed.br;
    chassisApplicationPublished.feedbackWheelSpeed.fl = chassisApplicationContext.feedbackWheelSpeed.fl;
    chassisApplicationPublished.feedbackWheelSpeed.fr = chassisApplicationContext.feedbackWheelSpeed.fr;
    chassisApplicationPublished.feedbackWheelSpeed.bl = chassisApplicationContext.feedbackWheelSpeed.bl;
    chassisApplicationPublished.feedbackWheelSpeed.br = chassisApplicationContext.feedbackWheelSpeed.br;
    chassisApplicationPublished.encoderData.flCount = chassisApplicationContext.encoderData.flCount;
    chassisApplicationPublished.encoderData.frCount = chassisApplicationContext.encoderData.frCount;
    chassisApplicationPublished.encoderData.blCount = chassisApplicationContext.encoderData.blCount;
    chassisApplicationPublished.encoderData.brCount = chassisApplicationContext.encoderData.brCount;
    chassisApplicationPublished.imu = chassisApplicationContext.imu;
    chassisApplicationPublished.actualSpeed.vx = chassisApplicationContext.actualSpeed.vx;
    chassisApplicationPublished.actualSpeed.vy = chassisApplicationContext.actualSpeed.vy;
    chassisApplicationPublished.actualSpeed.wz = chassisApplicationContext.actualSpeed.wz;
    chassisApplicationPublished.yawCorrection = chassisApplicationContext.yawCorrection;
    chassisApplicationPublished.headingReference = chassisApplicationContext.headingReference;
    chassisApplicationPublished.headingError = chassisApplicationContext.headingError;
    chassisApplicationPublished.headingRateTarget = chassisApplicationContext.headingRateTarget;
    chassisApplicationPublished.yawRateEnabled = chassisApplicationContext.yawRateEnabled;
    chassisApplicationPublished.yawControlMode = chassisApplicationContext.yawControlMode;
    chassisApplicationPublished.attitudeHealthy =
        (uint8_t) (chassisApplicationContext.imu.healthy != 0u);
    chassisApplicationPublished.attitudeDataFresh = (uint8_t) (
        chassisApplicationContext.imu.valid != 0u &&
        chassisApplicationContext.imuDisplayStaleFeedbackCount <
            CHASSIS_IMU_DISPLAY_STALE_FEEDBACK_LIMIT);
    chassisApplicationPublished.attitudeFaultActive =
        chassisApplicationContext.attitudeFaultActive;
    chassisApplicationPublished.seq = chassisApplicationContext.snapshotSeq;
    chassisApplicationPublished.valid = 1u;
    __DMB();
    chassisApplicationPublishGeneration++;
}

static uint32_t chassisApplicationCommandLock(void)
{
    {
        uint32_t irqWasEnabled = NVIC_GetEnableIRQ(CPUIntIdx2_IRQn);
        NVIC_DisableIRQ(CPUIntIdx2_IRQn);
        __DMB();
        return irqWasEnabled;
    }
}

static void chassisApplicationCommandUnlock(uint32_t irqWasEnabled)
{
    {
        __DMB();
        if (irqWasEnabled != 0u)
        {
            NVIC_EnableIRQ(CPUIntIdx2_IRQn);
        }
    }
}

```


core2_cyt4bb7/project/code/control/chassisApplication.h

SHA-256

:

e0de730a2b8b95b3695597e33ce0eed2e5ac65749580e347a8dbdcd746476734

代码 (c)

```
#ifndef _CHASSIS_APPLICATION_H_
#define _CHASSIS_APPLICATION_H_

#include <stdint.h>

#include "IMU.h"
#include "chassisMecanum.h"
#include "encoder.h"

typedef struct
{
    float yawRateKp;
    float yawRateKi;
    float yawRateKd;
    float yawRateIntegralMax;
    float headingKp;
    float headingRateLimit;
    float yawRateOutputLimit;
} chassisApplicationParamViewStruct;

typedef enum
{
    CHASSIS_YAW_DISABLED = 0u,
    CHASSIS_YAW_RATE,
    CHASSIS_YAW_HEADING
} chassisYawControlModeEnum;

typedef chassisSpeedStruct chassisMotionCommandStruct;

typedef struct
{
    chassisMotionCommandStruct targetCommand;
    wheelSpeedStruct baseWheelSpeed;
    wheelSpeedStruct targetWheelSpeed;
    wheelSpeedStruct feedbackWheelSpeed;
    encoderDataStruct encoderData;
    imuSnapshotStruct imu;
    chassisSpeedStruct actualSpeed;
    float yawCorrection;
    float headingReference;
    float headingError;
    float headingRateTarget;
    uint8_t yawRateEnabled;
    uint8_t yawControlMode;
    uint8_t attitudeHealthy;
    uint8_t attitudeDataFresh;
    uint8_t attitudeFaultActive;
    uint32_t seq;
    uint8_t valid;
} chassisApplicationSnapshotStruct;

uint8_t chassisApplicationInit(void);
uint8_t chassisApplicationApplyParams(const chassisApplicationParamViewStruct *params);
void chassisApplicationApplyYawOutputLimit(float outputLimit);
void chassisApplicationSetMotionCommand(chassisMotionCommandStruct command);
void chassisApplicationApplyWheelTarget(wheelSpeedStruct targetSpeed);
void chassisApplicationApplyReplayWheelTarget(
    wheelSpeedStruct baseTarget,
    wheelSpeedStruct encoderCorrection,
    float wz);
void chassisApplicationStop(void);
void chassisApplicationUpdateFeedback(void);
void chassisApplicationGetSnapshot(chassisApplicationSnapshotStruct *out);
void chassisApplicationApplyCommandWithLimit(
    chassisMotionCommandStruct command,
    float maxLinearSpeedMps,
    float maxYawRateRadps);
```

```
void chassisApplicationYawRateTick(void);
void chassisApplicationSetYawRateEnable(uint8_t enable);

#endif
```

core2_cyt4bb7/project/code/control/chassisMecanum.c

SHA-256

:

de40cb5d339ef93825370f851041e934f7a09e31020c52fdbad9f9456e633341

代码 (c)

```
#include "chassisMecanum.h"

#include "commonDef.h"

static float getRotateFactor(const chassisMecanumConfigStruct* config);

void chassisMecanumConfigInit(
    chassisMecanumConfigStruct* config,
    float wheelRadius,
    float wheelBase,
    float wheelTrack,
    uint32_t encoderPulsesPerRev,
    float speedSamplePeriodS,
    float maxWheelLinearSpeedMps)
{
    if (!config) return;

    config->wheelRadius = wheelRadius;
    config->wheelBase = wheelBase;
    config->wheelTrack = wheelTrack;
    config->encoderPulsesPerRev = encoderPulsesPerRev;
    config->speedSamplePeriodS = speedSamplePeriodS;
    config->wheelLinearToPulse = 0.0f;
    config->wheelPulseToLinear = 0.0f;
    config->maxWheelPulseSpeed = 0.0f;

    if (wheelRadius <= 0.0f) return;
    if (encoderPulsesPerRev == 0u) return;
    if (speedSamplePeriodS <= 0.0f) return;

    config->wheelLinearToPulse =
        ((float) encoderPulsesPerRev * speedSamplePeriodS) /
        (TWO_PI * wheelRadius);
    if (config->wheelLinearToPulse <= 0.0f) return;
    config->wheelPulseToLinear = 1.0f / config->wheelLinearToPulse;
    chassisMecanumConfigSetMaxLinearSpeed(config, maxWheelLinearSpeedMps);
}

void chassisMecanumConfigSetMaxLinearSpeed(chassisMecanumConfigStruct* config, float maxWheelLinearSpeedMps)
{
    if (!config) return;
    if (config->wheelLinearToPulse <= 0.0f || maxWheelLinearSpeedMps <= 0.0f)
    {
        config->maxWheelPulseSpeed = 0.0f;
        return;
    }
    config->maxWheelPulseSpeed = maxWheelLinearSpeedMps * config->wheelLinearToPulse;
}

wheelSpeedStruct chassisMecanumInverse(const chassisMecanumConfigStruct* config, chassisSpeedStruct speed)
{
    wheelSpeedStruct wheelSpeed = {0.0f, 0.0f, 0.0f, 0.0f};
    float rotateFactor;

    if (!config) return wheelSpeed;
    if (config->wheelLinearToPulse <= 0.0f) return wheelSpeed;

    rotateFactor = getRotateFactor(config);

    wheelSpeed.fl = (speed.vx - speed.vy - rotateFactor * speed.wz) * config->wheelLinearToPulse;
    wheelSpeed.fr = (speed.vx + speed.vy + rotateFactor * speed.wz) * config->wheelLinearToPulse;
    wheelSpeed.bl = (speed.vx + speed.vy - rotateFactor * speed.wz) * config->wheelLinearToPulse;
    wheelSpeed.br = (speed.vx - speed.vy + rotateFactor * speed.wz) * config->wheelLinearToPulse;
```

```

    if (config->maxWheelPulseSpeed > 0.0f)
    {
        wheelSpeed = chassisMecanumNormalize(wheelSpeed, config->maxWheelPulseSpeed);
    }

    return wheelSpeed;
}

/* 正解算（轮速 pulse/2ms → 底盘速度 m/s）：编码器 4 轮速合成 vx/vy/wz 实际速度。
 * 与逆解算 chassisMecanumInverse 互为逆变换；rotateFactor=0.5(wheelBase+wheelTrack)。 */
chassisSpeedStruct chassisMecanumForward(const chassisMecanumConfigStruct* config, wheelSpeedStruct speed)
{
    chassisSpeedStruct chassisSpeed = {0.0f, 0.0f, 0.0f};
    float rotateFactor;
    float speedFactor;

    if (!config) return chassisSpeed;

    rotateFactor = getRotateFactor(config);
    if (config->wheelPulseToLinear <= 0.0f) return chassisSpeed;
    if (rotateFactor <= 0.0f) return chassisSpeed;

    speedFactor = 0.25f * config->wheelPulseToLinear;

    chassisSpeed.vx = (speed.fl + speed.fr + speed.bl + speed.br) * speedFactor;
    chassisSpeed.vy = (-speed.fl + speed.fr + speed.bl - speed.br) * speedFactor;
    chassisSpeed.wz = (-speed.fl + speed.fr - speed.bl + speed.br) * speedFactor / rotateFactor;

    return chassisSpeed;
}

wheelSpeedStruct chassisMecanumNormalize(wheelSpeedStruct speed, float maxValue)
{
    float currentMax = 0.0f;
    float scale = 1.0f;

    if (maxValue <= 0.0f) return speed;

    currentMax = absf(speed.fl);
    if (absf(speed.fr) > currentMax) currentMax = absf(speed.fr);
    if (absf(speed.bl) > currentMax) currentMax = absf(speed.bl);
    if (absf(speed.br) > currentMax) currentMax = absf(speed.br);

    if (currentMax <= maxValue) return speed;

    scale = maxValue / currentMax;

    speed.fl *= scale;
    speed.fr *= scale;
    speed.bl *= scale;
    speed.br *= scale;

    return speed;
}

static float getRotateFactor(const chassisMecanumConfigStruct* config)
{
    return 0.5f * (config->wheelBase + config->wheelTrack);
}

```

core2_cyt4bb7/project/code/control/chassisMecanum.h

SHA-256

:

e298f7d7c27a1573b81598429f93c2af98f4f82609e6e57732b5705f329335d9

代码 (c)

```

#ifndef _CHASSIS_MECANUM_H_
#define _CHASSIS_MECANUM_H_

/* 麦轮运动学量约定：
 * - 对外接口（chassisSpeedStruct vx/vy/wz）单位：vx/vy = m/s，wz = rad/s。
 * - 内部轮速（wheelSpeedStruct fl/fr/bl/br）单位：脉冲/速度环采样周期（pulse per speedSamplePeriods）。
 * - 转换系数 wheelLinearToPulse = encoderPulsesPerRev * speedSamplePeriodS / (2*pi*wheelRadius)，

```

```

* 由 chassisMecanumConfigInit 据物理参数算出； wheelPulseToLinear 为其倒数（正解算用）。
* 逆解算输入 m/s,rad/s → 输出 pulse； 正解算输入 pulse → 输出 m/s,rad/s。 */

#include <stdint.h>

typedef struct
{
    float vx;
    float vy;
    float wz;
} chassisSpeedStruct;

typedef struct
{
    float fl;
    float fr;
    float bl;
    float br;
} wheelSpeedStruct;

typedef struct
{
    float wheelRadius;
    float wheelBase;
    float wheelTrack;
    uint32_t encoderPulsesPerRev;
    float speedSamplePeriodS;
    float wheelLinearToPulse;
    float wheelPulseToLinear;
    float maxWheelPulseSpeed;
} chassisMecanumConfigStruct;

void chassisMecanumConfigInit(
    chassisMecanumConfigStruct* config,
    float wheelRadius,
    float wheelBase,
    float wheelTrack,
    uint32_t encoderPulsesPerRev,
    float speedSamplePeriodS,
    float maxWheelLinearSpeedMps);
void chassisMecanumConfigSetMaxLinearSpeed(chassisMecanumConfigStruct* config, float maxWheelLinearSpeedMps);
wheelSpeedStruct chassisMecanumInverse(const chassisMecanumConfigStruct* config, chassisSpeedStruct speed);
chassisSpeedStruct chassisMecanumForward(const chassisMecanumConfigStruct* config, wheelSpeedStruct speed);
wheelSpeedStruct chassisMecanumNormalize(wheelSpeedStruct speed, float maxValue);

#endif

```

core2_cyt4bb7/project/code/control/motor.c

SHA-256

:

9b86145279476e93a30775088712b7fca022b0fa823b69b186930e24abfc6294

代码 (c)

```

#include "motor.h"

#include <stdint.h>

#include "zf_driver_gpio.h"
#include "zf_driver_pwm.h"

#include "commonDef.h"
#include "configChassisPolarity.h"
#include "pinMap.h"

#define MOTOR_PWM_FREQ (17 000u)

static uint32_t speedToDuty(float speed);
static void setSingleMotor(pwm_channel_enum pwmChannel, gpio_pin_enum phPin, float speed);

void motorInit(void)
{
    gpio_init(MOTOR_FL_PH_PIN, GPO, GPIO_HIGH, GPO_PUSH_PULL);
    gpio_init(MOTOR_FR_PH_PIN, GPO, GPIO_HIGH, GPO_PUSH_PULL);
    gpio_init(MOTOR_BL_PH_PIN, GPO, GPIO_HIGH, GPO_PUSH_PULL);

```

```

gpio_init(MOTOR_BR_PH_PIN, GPO, GPIO_HIGH, GPO_PUSH_PULL);

pwm_init(MOTOR_FL_EN_PWM, MOTOR_PWM_FREQ, 0u);
pwm_init(MOTOR_FR_EN_PWM, MOTOR_PWM_FREQ, 0u);
pwm_init(MOTOR_BL_EN_PWM, MOTOR_PWM_FREQ, 0u);
pwm_init(MOTOR_BR_EN_PWM, MOTOR_PWM_FREQ, 0u);
}

void motorSetSpeed(float flSpeed, float frSpeed, float blSpeed, float brSpeed)
{
    /* 乘电机方向系数(configChassisPolarity.h): 极性单一真相源, 使 +speed → 四轮物理前进。
    * 各轮系数由实测辨识: FL+1 / FR-1 / BL+1 / BR-1 (左侧 FL/BL +1、右侧 FR/BR -1 物理反向) ;
    * 与速度环 Kp(全 23)成对成立负反馈。 */
    setSingleMotor(MOTOR_FL_EN_PWM, MOTOR_FL_PH_PIN, flSpeed * MOTOR_FL_OUTPUT_DIR);
    setSingleMotor(MOTOR_FR_EN_PWM, MOTOR_FR_PH_PIN, frSpeed * MOTOR_FR_OUTPUT_DIR);
    setSingleMotor(MOTOR_BL_EN_PWM, MOTOR_BL_PH_PIN, blSpeed * MOTOR_BL_OUTPUT_DIR);
    setSingleMotor(MOTOR_BR_EN_PWM, MOTOR_BR_PH_PIN, brSpeed * MOTOR_BR_OUTPUT_DIR);
}

void motorStopAll(void)
{
    motorSetSpeed(0.0f, 0.0f, 0.0f, 0.0f);
}

static uint32_t speedToDuty(float speed)
{
    float speedAbs = range(absf(speed), 0.0f, 1.0f);

    return (uint32_t) (speedAbs * (float) PWM_DUTY_MAX + 0.5f);
}

static void setSingleMotor(pwm_channel_enum pwmChannel, gpio_pin_enum phPin, float speed)
{
    gpio_set_level(phPin, (speed >= 0.0f) ? GPIO_HIGH : GPIO_LOW);
    pwm_set_duty(pwmChannel, speedToDuty(speed));
}

```

core2_cyt4bb7/project/code/control/motor.h

SHA-256

:

5954203520ca9264f1665f6b7f97c57a2ff6517043bb5b2f3cf796506f18d10b

代码 (c)

```

#ifndef _MOTOR_H_
#define _MOTOR_H_

void motorInit(void);
void motorSetSpeed(float flSpeed, float frSpeed, float blSpeed, float brSpeed);
void motorStopAll(void);

#endif

```

core2_cyt4bb7/project/code/control/motorControl.c

SHA-256

:

0e643c890f8b779bb1283bf9662cb5cbbc87a8a44479882d57cc1e293786a102

代码 (c)

```

#include "motorControl.h"

#include "commonDef.h"
#include "configChassisPidDefaults.h"
#include "configChassisPhysical.h"
#include "currentSense.h"
#include "motor.h"
#include "PID.h"
#include "lowPassFilter.h"
#include "zf_common_interrupt.h" /* __get_PRIMASK/ __disable_irq/ __set_PRIMASK (失控解锁临界区) */

```

```

typedef struct
{
    float targetSpeed;
    float feedbackSpeed;
    float targetCurrent;
    float feedbackCurrent;
    float motorOutput;
    PID speedPid;
    PID currentPid;
    LowPassFilterData speedFeedbackLpf;
} motorControlWheelDataStruct;

typedef struct
{
    motorControlWheelDataStruct fl;
    motorControlWheelDataStruct fr;
    motorControlWheelDataStruct bl;
    motorControlWheelDataStruct br;
    volatile uint8_t runawayLatched;
    uint16_t runawaySatCount[4];
    uint32_t snapshotSeq;
    uint8_t snapshotPublishDivider;
} motorControlContextStruct;

static motorControlContextStruct motorControlContext = {
    {0.0f, 0.0f, 0.0f, 0.0f, 0.0f, {0}, {0}, {0}},
    {0.0f, 0.0f, 0.0f, 0.0f, 0.0f, {0}, {0}, {0}},
    {0.0f, 0.0f, 0.0f, 0.0f, 0.0f, {0}, {0}, {0}},
    {0.0f, 0.0f, 0.0f, 0.0f, 0.0f, {0}, {0}, {0}},
    0u,
    {0u, 0u, 0u, 0u},
    0u,
    0u};
static volatile motorControlSnapshotStruct motorControlPublished = {
    {0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f},
    {0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f},
    {0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f},
    {0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 0.0f},
    0u, 0u, 0u};
static volatile uint32_t motorControlPublishGeneration = 0u;

/* 失控保护状态（motorControlOutput 内 ISR 维护；IsRunawayLatched 前台可读）：
* 任一轮 motorOutput 持续饱和达 MOTOR_RUNAWAY_TICKS → latched 持久锁定。
* 解锁路径：复位/重上电后的静态初值，或 motorControlClearRunaway(运行时人工解锁，菜单 K5 长按)。
* latched 被前台查询、motorControlOutput(500μs ISR) 改写，故该成员保持 volatile；
* 计数仅在 ISR 或关中断解锁路径内访问。 */

static void motorControlInitWheel(
    motorControlWheelDataStruct* wheel,
    float speedKp,
    float speedKi,
    float speedKd,
    float speedIntegralMax,
    float currentKp,
    float currentKi,
    float currentKd,
    float currentIntegralMax);
static void motorControlResetWheel(motorControlWheelDataStruct* wheel);
static void motorControlSpeedTickWheel(motorControlWheelDataStruct* wheel);
static void motorControlCurrentTickWheel(motorControlWheelDataStruct* wheel);
static void motorControlUpdateSpeedFeedbackWheel(motorControlWheelDataStruct* wheel, int16_t encoderCount);
static void motorControlPublish(void);
static void motorControlPublishThrottled(void);
static void motorControlPublishWheel(
    volatile motorControlWheelSnapshotStruct *out,
    const motorControlWheelDataStruct *wheel);
static void motorControlReadPublishedWheel(
    motorControlWheelSnapshotStruct *out,
    const volatile motorControlWheelSnapshotStruct *published);
static uint32_t motorControlCommandLock(void);
static void motorControlCommandUnlock(uint32_t irqWasEnabled);

void motorControlInit(void)
{
    const motorControlParamViewStruct params = {
        {MOTOR_CONTROL_FL_SPEED_KP, MOTOR_CONTROL_FR_SPEED_KP, MOTOR_CONTROL_BL_SPEED_KP,
        MOTOR_CONTROL_BR_SPEED_KP},
        {MOTOR_CONTROL_FL_SPEED_KI, MOTOR_CONTROL_FR_SPEED_KI, MOTOR_CONTROL_BL_SPEED_KI, MOTOR_CONTROL_BR_SPEED_KI},

```

```

    {MOTOR_CONTROL_FL_SPEED_KD, MOTOR_CONTROL_FR_SPEED_KD, MOTOR_CONTROL_BL_SPEED_KD,
    MOTOR_CONTROL_BR_SPEED_KD},
    MOTOR_CONTROL_SPEED_PID_INTEGRAL_MAX,
    {MOTOR_CONTROL_FL_CURRENT_KP, MOTOR_CONTROL_FR_CURRENT_KP, MOTOR_CONTROL_BL_CURRENT_KP,
    MOTOR_CONTROL_BR_CURRENT_KP},
    {MOTOR_CONTROL_FL_CURRENT_KI, MOTOR_CONTROL_FR_CURRENT_KI, MOTOR_CONTROL_BL_CURRENT_KI,
    MOTOR_CONTROL_BR_CURRENT_KI},
    {MOTOR_CONTROL_FL_CURRENT_KD, MOTOR_CONTROL_FR_CURRENT_KD, MOTOR_CONTROL_BL_CURRENT_KD,
    MOTOR_CONTROL_BR_CURRENT_KD},
    MOTOR_CONTROL_CURRENT_PID_INTEGRAL_MAX};
    motorControlApplyParams(&params);
}

```

```

void motorControlApplyParams(const motorControlParamViewStruct *params)
{

```

```

    if (params == 0) return;
    motorControlInitWheel(
        &motorControlContext.fl,
        params->speedKp[0], params->speedKi[0], params->speedKd[0],
        params->speedIntegralMax,
        params->currentKp[0], params->currentKi[0], params->currentKd[0],
        params->currentIntegralMax);
    motorControlInitWheel(
        &motorControlContext.fr,
        params->speedKp[1], params->speedKi[1], params->speedKd[1],
        params->speedIntegralMax,
        params->currentKp[1], params->currentKi[1], params->currentKd[1],
        params->currentIntegralMax);
    motorControlInitWheel(
        &motorControlContext.bl,
        params->speedKp[2], params->speedKi[2], params->speedKd[2],
        params->speedIntegralMax,
        params->currentKp[2], params->currentKi[2], params->currentKd[2],
        params->currentIntegralMax);
    motorControlInitWheel(
        &motorControlContext.br,
        params->speedKp[3], params->speedKi[3], params->speedKd[3],
        params->speedIntegralMax,
        params->currentKp[3], params->currentKi[3], params->currentKd[3],
        params->currentIntegralMax);

```

```

/* 参数热重建不解除持久失控锁定；复位/重上电依靠静态初值清 0，运行时只能 K5 人工解锁。*/
motorControlContext.runawaySatCount[0] = 0u;
motorControlContext.runawaySatCount[1] = 0u;
motorControlContext.runawaySatCount[2] = 0u;
motorControlContext.runawaySatCount[3] = 0u;

```

```

    motorControlStopAll();
}

```

```

void motorControlStopAll(void)
{

```

```

    motorControlResetWheel(&motorControlContext.fl);
    motorControlResetWheel(&motorControlContext.fr);
    motorControlResetWheel(&motorControlContext.bl);
    motorControlResetWheel(&motorControlContext.br);
    motorStopAll();
    motorControlPublishThrottled();
}

```

```

/* 调参专用：复位除 selected 外所有轮的控制态（PID 累加器清零 + 目标/输出归零 = 空闲自由），只保留选中轮闭环。
* 防非选中轮零目标极限环震颤（见 motorControl.h 说明）。每输出拍调用，PWM 由后续 motorControlOutput 写 0。
* 只清控制态、**不动 feedbackSpeed/feedbackCurrent**（保留传感器反馈供 16ch 遥测显示非选中轮真实量）。*/
void motorControlStopWheelExcept(uint8_t selected)

```

```

{
    motorControlWheelDataStruct* wheels[4] = {
        &motorControlContext.fl,
        &motorControlContext.fr,
        &motorControlContext.bl,
        &motorControlContext.br};
    uint8_t sel = (uint8_t) (selected & 0x03u);
    uint8_t i;

```

```

    for (i = 0u; i < 4u; i++)
    {
        if (i != sel)
        {
            wheels[i]->targetSpeed = 0.0f;
            wheels[i]->targetCurrent = 0.0f;

```

```

        wheels[i]->motorOutput = 0.0f;
        pidReset(&wheels[i]->speedPid); /* 清积分/累加器，防零目标 windup */
        pidReset(&wheels[i]->currentPid);
    }
}
}

void motorControlSetWheelSpeed(wheelSpeedStruct targetSpeed)
{
    uint32_t irqWasEnabled = motorControlCommandLock();
    motorControlContext.fl.targetSpeed = targetSpeed.fl;
    motorControlContext.fr.targetSpeed = targetSpeed.fr;
    motorControlContext.bl.targetSpeed = targetSpeed.bl;
    motorControlContext.br.targetSpeed = targetSpeed.br;
    motorControlCommandUnlock(irqWasEnabled);
}

void motorControlSetWheelTargetCurrent(wheelSpeedStruct targetCurrent)
{
    uint32_t irqWasEnabled = motorControlCommandLock();
    motorControlContext.fl.targetCurrent = clampSym(targetCurrent.fl, MOTOR_CONTROL_CURRENT_LIMIT);
    motorControlContext.fr.targetCurrent = clampSym(targetCurrent.fr, MOTOR_CONTROL_CURRENT_LIMIT);
    motorControlContext.bl.targetCurrent = clampSym(targetCurrent.bl, MOTOR_CONTROL_CURRENT_LIMIT);
    motorControlContext.br.targetCurrent = clampSym(targetCurrent.br, MOTOR_CONTROL_CURRENT_LIMIT);
    motorControlCommandUnlock(irqWasEnabled);
}

void motorControlSetOpenLoopOutput(wheelSpeedStruct openLoopOutput)
{
    uint32_t irqWasEnabled = motorControlCommandLock();
    motorControlContext.fl.motorOutput = clampSym(openLoopOutput.fl, MOTOR_CONTROL_OUTPUT_LIMIT);
    motorControlContext.fr.motorOutput = clampSym(openLoopOutput.fr, MOTOR_CONTROL_OUTPUT_LIMIT);
    motorControlContext.bl.motorOutput = clampSym(openLoopOutput.bl, MOTOR_CONTROL_OUTPUT_LIMIT);
    motorControlContext.br.motorOutput = clampSym(openLoopOutput.br, MOTOR_CONTROL_OUTPUT_LIMIT);
    motorControlCommandUnlock(irqWasEnabled);
}

void motorControlGetSnapshot(motorControlSnapshotStruct *out)
{
    uint8_t attempt;
    motorControlSnapshotStruct snapshot = {0};
    const motorControlSnapshotStruct invalid = {0};

    if (0 == out) return;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        uint32_t before = motorControlPublishGeneration();
        uint32_t after;
        if ((before & 1u) != 0u) continue;
        __DMB();
        motorControlReadPublishedWheel(&snapshot.fl, &motorControlPublished.fl);
        motorControlReadPublishedWheel(&snapshot.fr, &motorControlPublished.fr);
        motorControlReadPublishedWheel(&snapshot.bl, &motorControlPublished.bl);
        motorControlReadPublishedWheel(&snapshot.br, &motorControlPublished.br);
        snapshot.seq = motorControlPublished.seq;
        snapshot.runawayLatched = motorControlPublished.runawayLatched;
        snapshot.valid = motorControlPublished.valid;
        __DMB();
        after = motorControlPublishGeneration();
        if ((before == after) && ((after & 1u) == 0u))
        {
            *out = snapshot;
            return;
        }
    }
    *out = invalid;
}

wheelSpeedStruct motorControlUpdateSpeedFeedback(encoderDataStruct encoderData)
{
    wheelSpeedStruct speed;

    motorControlUpdateSpeedFeedbackWheel(&motorControlContext.fl, encoderData.flCount);
    motorControlUpdateSpeedFeedbackWheel(&motorControlContext.fr, encoderData.frCount);
    motorControlUpdateSpeedFeedbackWheel(&motorControlContext.bl, encoderData.blCount);
    motorControlUpdateSpeedFeedbackWheel(&motorControlContext.br, encoderData.brCount);
    speed.fl = motorControlContext.fl.feedbackSpeed;
    speed.fr = motorControlContext.fr.feedbackSpeed;
    speed.bl = motorControlContext.bl.feedbackSpeed;

```



```

    speed.br = motorControlContext.br.feedbackSpeed;
    return speed;
}

void motorControlUpdateCurrentFeedback(void)
{
    currentSenseSnapshotStruct snapshot;

    currentSenseGetSnapshot(&snapshot);
    motorControlContext.fl.feedbackCurrent = (float) snapshot.fl;
    motorControlContext.fr.feedbackCurrent = (float) snapshot.fr;
    motorControlContext.bl.feedbackCurrent = (float) snapshot.bl;
    motorControlContext.br.feedbackCurrent = (float) snapshot.br;
}

void motorControlSpeedTick(void)
{
    motorControlSpeedTickWheel(&motorControlContext.fl);
    motorControlSpeedTickWheel(&motorControlContext.fr);
    motorControlSpeedTickWheel(&motorControlContext.bl);
    motorControlSpeedTickWheel(&motorControlContext.br);
}

void motorControlCurrentTick(void)
{
    motorControlCurrentTickWheel(&motorControlContext.fl);
    motorControlCurrentTickWheel(&motorControlContext.fr);
    motorControlCurrentTickWheel(&motorControlContext.bl);
    motorControlCurrentTickWheel(&motorControlContext.br);
}

/* 失控检测：任一轮 motorOutput 持续饱和(|out|≥阈值)达 MOTOR_RUNAWAY_TICKS 拍 → 置 latched。
 * 放 motorControlOutput（所有控制模式唯一必经关口；OPEN_LOOP 不调 currentTick 故不能放那）。*/
static void motorRunawayDetect(void)
{
    const float out[4] = {
        motorControlContext.fl.motorOutput, motorControlContext.fr.motorOutput,
        motorControlContext.bl.motorOutput, motorControlContext.br.motorOutput};
    uint8_t i;

    for (i = 0u; i < 4u; i++)
    {
        if (absf(out[i]) >= MOTOR_RUNAWAY_SAT_THRESHOLD)
        {
            if (motorControlContext.runawaySatCount[i] < MOTOR_RUNAWAY_TICKS)
            {
                motorControlContext.runawaySatCount[i]++;
            }
            if (motorControlContext.runawaySatCount[i] >= MOTOR_RUNAWAY_TICKS)
            {
                motorControlContext.runawayLatched = 1u;
            }
        }
        else
        {
            motorControlContext.runawaySatCount[i] = 0u;
        }
    }
}

void motorControlOutput(void)
{
    uint8_t latchWasSet = motorControlContext.runawayLatched;

    if ((floatIsFinite(motorControlContext.fl.motorOutput) == 0u) ||
        (floatIsFinite(motorControlContext.fr.motorOutput) == 0u) ||
        (floatIsFinite(motorControlContext.bl.motorOutput) == 0u) ||
        (floatIsFinite(motorControlContext.br.motorOutput) == 0u))
    {
        motorControlContext.runawayLatched = 1u;
        motorControlContext.fl.motorOutput = 0.0f;
        motorControlContext.fr.motorOutput = 0.0f;
        motorControlContext.bl.motorOutput = 0.0f;
        motorControlContext.br.motorOutput = 0.0f;
        motorStopAll();
        if (latchWasSet == 0u) motorControlPublish();
        else motorControlPublishThrottled();
        return;
    }
}

```

```

if (!motorControlContext.runawayLatched)
{
    motorRunawayDetect();
}

if (motorControlContext.runawayLatched)
{
    /* 持久失控锁定：全车 PWM=0。蜂鸣由 buzzer 单一所有者库仲裁播放（core0Loop 每拍
    * buzzerSetRunaway(motorControlIsRunawayLatched()) 注入，仲裁器最高优先常鸣）——本模块不再直写 BEEP。
    * 运行时仅 motorControlClearRunaway(菜单 HOME K5 长按) 解锁；复位/重上电依靠静态初值。*/
    motorStopAll();
    if (latchWasSet == 0u) motorControlPublish();
    else motorControlPublishThrottled();
    return;
}

motorSetSpeed(
    motorControlContext.fl.motorOutput,
    motorControlContext.fr.motorOutput,
    motorControlContext.bl.motorOutput,
    motorControlContext.br.motorOutput);
motorControlPublishThrottled();
}

uint8_t motorControlIsRunawayLatched(void)
{
    return motorControlContext.runawayLatched;
}

/* 运行时人工解锁失控锁定（菜单 K5 长按触发）：从静止干净恢复，由发车门控/遥控正常受控。
* 不改 PID 增益/参数；触发门檻(7s 饱和)不变。前台调用，与 500µs ISR(motorControlOutput) 竞争，
* 故用 PRIMASK 临界区原子执行（禁用 SDK interrupt_global_，见 embedded-safety §临界区原语）。
* 先 motorControlStopAll() 把四轮 PID 累加器/目标/feedback/motorOutput 全清 + PWM=0，再清 latched+饱和计数：
* 否则解锁后下一拍 motorControlOutput 会把失控瞬间残留的陈旧饱和 motorOutput 直接写回电机（带电急冲，
* 故障未除时仍会再跑 7s 才重新锁定）。归零后从静止重新闭环，故障仍在则重新累计触发，安全。*/
void motorControlClearRunaway(void)
{
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    motorControlStopAll(); /* 控制态 + 输出归零，避免恢复陈旧饱和和输出 */
    motorControlContext.runawayLatched = 0u;
    motorControlContext.runawaySatCount[0] = 0u;
    motorControlContext.runawaySatCount[1] = 0u;
    motorControlContext.runawaySatCount[2] = 0u;
    motorControlContext.runawaySatCount[3] = 0u;
    motorControlPublish();
    __set_PRIMASK(primask);
}

static void motorControlInitWheel(
    motorControlWheelDataStruct* wheel,
    float speedKp,
    float speedKi,
    float speedKd,
    float speedIntegralMax,
    float currentKp,
    float currentKi,
    float currentKd,
    float currentIntegralMax)
{
    pidInit(&wheel->speedPid, speedIntegralMax, 0.0f, speedKp, speedKi, speedKd, MOTOR_CONTROL_SPEED_DERIVATIVE_LPF_ALPHA);
    pidInit(&wheel->currentPid, currentIntegralMax, 0.0f, currentKp, currentKi, currentKd, 1.0f);
    lowPassFilterInit(&wheel->speedFeedbackLpf, MOTOR_CONTROL_SPEED_FEEDBACK_LPF_ALPHA);
}

static void motorControlResetWheel(motorControlWheelDataStruct* wheel)
{
    {
        wheel->targetSpeed = 0.0f;
        wheel->feedbackSpeed = 0.0f;
        wheel->targetCurrent = 0.0f;
        wheel->feedbackCurrent = 0.0f;
        wheel->motorOutput = 0.0f;

        pidReset(&wheel->speedPid);
        pidReset(&wheel->currentPid);
        lowPassFilterInit(&wheel->speedFeedbackLpf, wheel->speedFeedbackLpf.alpha);
    }
}

```

```

static void motorControlSpeedTickWheel(motorControlWheelDataStruct* wheel)
{
    wheel->speedPid.expectedValue = wheel->targetSpeed;
    pidProcess(&wheel->speedPid, wheel->feedbackSpeed);
    wheel->targetCurrent = clampSym(wheel->speedPid.controlValue, MOTOR_CONTROL_CURRENT_LIMIT);
}

static void motorControlCurrentTickWheel(motorControlWheelDataStruct* wheel)
{
    wheel->currentPid.expectedValue = wheel->targetCurrent;
    pidProcess(&wheel->currentPid, wheel->feedbackCurrent);
    /* 增量式电流环抗饱和: controlValue 是累加器 (每拍 +=), 钳位后回写自身,
    * 防堵转 / 目标电流不可达时 controlValue 无限累积 (windup) ——否则反向命令需
    * 从溢出值逐拍退回, 恢复迟缓。60A 强磁电机安全相关: 截断累加器不影响微分项
    * (lastError/beforeLastError 已在 handler 内更新), 下拍增量基于新 error 叠加在钳位值上。 */
    wheel->currentPid.controlValue = clampSym(wheel->currentPid.controlValue, MOTOR_CONTROL_OUTPUT_LIMIT);
    wheel->motorOutput = wheel->currentPid.controlValue;
}

static void motorControlUpdateSpeedFeedbackWheel(motorControlWheelDataStruct* wheel, int16_t encoderCount)
{
    lowPassFilter(&wheel->speedFeedbackLpf, (float) encoderCount);
    wheel->feedbackSpeed = wheel->speedFeedbackLpf.filteredValue;
}

static void motorControlPublish(void)
{
    motorControlContext.snapshotSeq++;
    motorControlPublishGeneration++;
    __DMB();
    motorControlPublishWheel(&motorControlPublished.fl, &motorControlContext.fl);
    motorControlPublishWheel(&motorControlPublished.fr, &motorControlContext.fr);
    motorControlPublishWheel(&motorControlPublished.bl, &motorControlContext.bl);
    motorControlPublishWheel(&motorControlPublished.br, &motorControlContext.br);
    motorControlPublished.seq = motorControlContext.snapshotSeq;
    motorControlPublished.runawayLatched = motorControlContext.runawayLatched;
    motorControlPublished.valid = 1u;
    __DMB();
    motorControlPublishGeneration++;
}

/* Snapshot 仅供前台显示/遥测, 不参与 500μs 控制反馈。按速度环周期 2ms 发布,
* 避免每个电流环拍复制四轮 28 个 float 并执行 DMB; 首次发布和故障边沿仍由调用方强制 Publish。 */
static void motorControlPublishThrottled(void)
{
    motorControlContext.snapshotPublishDivider++;
    if ((motorControlPublished.valid == 0u) ||
        (motorControlContext.snapshotPublishDivider >= SPEED_LOOP_DIVIDER))
    {
        motorControlContext.snapshotPublishDivider = 0u;
        motorControlPublish();
    }
}

static void motorControlPublishWheel(
    volatile motorControlWheelSnapshotStruct *out,
    const motorControlWheelDataStruct *wheel)
{
    out->targetSpeed = wheel->targetSpeed;
    out->feedbackSpeed = wheel->feedbackSpeed;
    out->targetCurrent = wheel->targetCurrent;
    out->feedbackCurrent = wheel->feedbackCurrent;
    out->motorOutput = wheel->motorOutput;
    out->speedControl = wheel->speedPid.controlValue;
    out->currentControl = wheel->currentPid.controlValue;
}

static void motorControlReadPublishedWheel(
    motorControlWheelSnapshotStruct *out,
    const volatile motorControlWheelSnapshotStruct *published)
{
    out->targetSpeed = published->targetSpeed;
    out->feedbackSpeed = published->feedbackSpeed;
    out->targetCurrent = published->targetCurrent;
    out->feedbackCurrent = published->feedbackCurrent;
    out->motorOutput = published->motorOutput;
    out->speedControl = published->speedControl;
    out->currentControl = published->currentControl;
}

```

```

/* TUNE 前台和 PIT_CH0 都可提交四轮命令。只屏蔽实际承载 PIT_CH0/CH1 的 CPUIntIdx2,
 * 保持四个标量作为一次业务命令发布；不扩大为全局关中断。*/
static uint32_t motorControlCommandLock(void)
{
    uint32_t irqWasEnabled = NVIC_GetEnableIRQ(CPUIntIdx2_IRQn);
    NVIC_DisableIRQ(CPUIntIdx2_IRQn);
    __DMB();
    return irqWasEnabled;
}

static void motorControlCommandUnlock(uint32_t irqWasEnabled)
{
    __DMB();
    if (irqWasEnabled != 0u)
    {
        NVIC_EnableIRQ(CPUIntIdx2_IRQn);
    }
}

```

core2_cyt4bb7/project/code/control/motorControl.h

SHA-256

:

b83571711ff222b5571e47335d07c7762635ef38a691e42c224963863ea654c0

代码 (c)

```

#ifndef _MOTOR_CONTROL_H_
#define _MOTOR_CONTROL_H_

#include <stdint.h>

#include "chassisMecanum.h"
#include "encoder.h"

typedef struct
{
    float speedKp[4];
    float speedKi[4];
    float speedKd[4];
    float speedIntegralMax;
    float currentKp[4];
    float currentKi[4];
    float currentKd[4];
    float currentIntegralMax;
} motorControlParamViewStruct;

typedef struct
{
    float targetSpeed;
    float feedbackSpeed;
    float targetCurrent;
    float feedbackCurrent;
    float motorOutput;
    float speedControl;
    float currentControl;
} motorControlWheelSnapshotStruct;

typedef struct
{
    motorControlWheelSnapshotStruct fl;
    motorControlWheelSnapshotStruct fr;
    motorControlWheelSnapshotStruct bl;
    motorControlWheelSnapshotStruct br;
    uint32_t seq;
    uint8_t runawayLatched;
    uint8_t valid;
} motorControlSnapshotStruct;

void motorControlInit(void);
void motorControlApplyParams(const motorControlParamViewStruct *params);
void motorControlStopAll(void);
/* 调参专用：复位除 selected(0..3) 外所有轮（PID 清零 + 目标/输出归零 = 空闲自由），只保留选中轮闭环。
 * 解决调参时非选中轮在零目标下被主动控到 0 → PI 积分/增量累加器追编码器/电流噪声 + 电机静摩擦
 * 形成零设定点极限环震颤。须在每输出拍(500μs)调用，覆盖 currentTick 对非选中轮的输出。*/

```

```

void motorControlStopWheelExcept(uint8_t selected);

void motorControlSetWheelSpeed(wheelSpeedStruct targetSpeed);
void motorControlSetWheelTargetCurrent(wheelSpeedStruct targetCurrent);
void motorControlSetOpenLoopOutput(wheelSpeedStruct openLoopOutput);
void motorControlGetSnapshot(motorControlSnapshotStruct *out);

wheelSpeedStruct motorControlUpdateSpeedFeedback(encoderDataStruct encoderData);
void motorControlUpdateCurrentFeedback(void);
void motorControlSpeedTick(void);
void motorControlCurrentTick(void);
void motorControlOutput(void);

/* 失控保护：任一轮 motorOutput 持续饱和 MOTOR_RUNAWAY_TICKS(7s) → 持久锁定全车输出 0。
 * 解锁：复位/重上电后的静态初值，或 motorControlClearRunaway(运行时人工，菜单 K5 长按)。
 * 返回 1=已锁定（供显示/遥测状态观测，及 core0Loop 喂 buzzerSetRunaway 由 buzzer 库仲裁常鸣；本模块不再直写 BEEP）。*/
uint8_t motorControlRunawayLatched(void);

/* 运行时人工解锁失控锁定（菜单 HOME K5 长按）：motorControlStopAll(控制态+输出归零)+清 latched/饱和计数，
 * 免断电从静止干净恢复重新发车（不写回失控瞬间陈旧饱和和输出）。PRIMASK 临界区原子；不改参数，触发门槛不变。*/
void motorControlClearRunaway(void);

#endif

```

core2_cyt4bb7/project/code/control/PID.c

SHA-256

:

1266d5bf73d259544a3b7d31f61a1b22adfb1f1ed04eb60974e1cfa3f64b7d01

代码 (c)

```

#include "PID.h"

#include <math.h>

static void incrementalPidProcessHandler(PID *obj, float currentValue)
{
    float error = obj->expectedValue - currentValue;
    float deltaError = error - obj->lastError;

    obj->controlValue +=
        obj->kp * deltaError +
        obj->ki * error +
        obj->kd * (deltaError - (obj->lastError - obj->beforeLastError));

    obj->beforeLastError = obj->lastError;
    obj->lastError = error;
}

static void positionalPidHandler(PID *obj, float currentValue)
{
    float error = obj->expectedValue - currentValue;

    obj->integral += error;

    if (obj->integralMax)
    {
        if (obj->integral > obj->integralMax) obj->integral = obj->integralMax;
        else if (obj->integral < -obj->integralMax) obj->integral = -obj->integralMax;
    }

    obj->controlValue =
        obj->kp * error +
        obj->ki * obj->integral +
        obj->kd * (error - obj->lastError);

    obj->lastError = error;
}

/* 位置式 PID + D 项一阶低通：对 (error - lastError) 过 LPF 后乘 kd */
static void positionalPidWithDLpfHandler(PID *obj, float currentValue)
{
    float error = obj->expectedValue - currentValue;

    obj->integral += error;

```

```

    if (obj->integralMax)
    {
        if (obj->integral > obj->integralMax) obj->integral = obj->integralMax;
        else if (obj->integral < -obj->integralMax) obj->integral = -obj->integralMax;
    }

    lowPassFilter(&obj->dLpf, error - obj->lastError);

    obj->controlValue =
        obj->kp * error +
        obj->ki * obj->integral +
        obj->kd * obj->dLpf.filteredValue;

    obj->lastError = error;
}

void pidInit(PID *obj, float integralMax, float target, float kp, float ki, float kd, float derivativeLpfAlpha)
{
    obj->expectedValue = target;
    obj->kp = kp;
    obj->ki = ki;
    obj->kd = kd;

    if (ki != 0.0f && integralMax != 0.0f) obj->integralMax = fabsf(integralMax / ki);
    else obj->integralMax = 0.0f;

    obj->lastError = 0.0f;
    obj->beforeLastError = 0.0f;
    obj->integral = 0.0f;
    obj->controlValue = 0.0f;

    /* D 项 LPF: 仅在位置式 + alpha∈(0,1) 时启用 */
    if (integralMax != 0.0f && derivativeLpfAlpha > 0.0f && derivativeLpfAlpha < 1.0f)
    {
        lowPassFilterInit(&obj->dLpf, derivativeLpfAlpha);
        obj->dLpfEnabled = 1u;
        obj->process = positionalPidWithDLpfHandler;
    }
    else
    {
        lowPassFilterInit(&obj->dLpf, 1.0f);
        obj->dLpfEnabled = 0u;
        if (integralMax == 0.0f) obj->process = incrementalPidProcessHandler;
        else obj->process = positionalPidHandler;
    }
}

inline void pidProcess(PID *obj, float currentValue)
{
    obj->process(obj, currentValue);
}

void pidReset(PID *obj)
{
    obj->expectedValue = 0.0f;
    obj->lastError = 0.0f;
    obj->beforeLastError = 0.0f;
    obj->integral = 0.0f;
    obj->controlValue = 0.0f;
    lowPassFilterInit(&obj->dLpf, obj->dLpf.alpha);
}

```

core2_cyt4bb7/project/code/control/PID.h

SHA-256

:

03c43f6b80090576d596ebd60803501f199042ce06c312d410a1f0dc31e206e6

代码 (c)

```

#ifndef _PID_H_
#define _PID_H_

#include <stdint.h>

```

```
#include "lowPassFilter.h"

/*
 * 通用 PID 控制器
 * - integralMax == 0 → 增量式（用于电流/速度类无稳态的环；D 项 LPF 强制不启用）
 * - integralMax > 0 → 位置式（积分限幅自动按 ki 折算）
 *
 * D 项低通：
 * - derivativeLpfAlpha >= 1.0f → 不启用（位置式直接乘 kd）
 * - derivativeLpfAlpha ∈ (0,1) 且 位置式 → 启用，对 (error - lastError) 过一阶 LPF 后再乘 kd
 * - 增量式无视 alpha
 *
 * 用法：
 * PID pitchAnglePid;
 * pidInit(&pitchAnglePid, 5.0f, 0.0f, 4.0f, 0.0f, 0.02f, 1.0f); // 位置式无 D 项 LPF
 * pidInit(&pitchRatePid, 5.0f, 0.0f, 17.1f, 0.0f, 17.0f, 0.3f); // 位置式 + D 项 LPF
 * pitchAnglePid.expectedValue = targetAngle;
 * pidProcess(&pitchAnglePid, currentValue);
 * float out = pitchAnglePid.controlValue;
 */

typedef struct PID PID;
typedef void (*pidProcessHandlerFn)(PID *obj, float currentValue);

struct PID
{
    float expectedValue;
    float kp;
    float ki;
    float kd;

    float lastError;
    float beforeLastError;
    float integral;
    float integralMax;

    float controlValue;

    LowPassFilterData dLpf;
    uint8_t dLpfEnabled;

    pidProcessHandlerFn process;
};

/* 初始化：integralMax==0 选增量式，否则位置式。
 * integralMax 的含义：积分项在控制量单位下的最大贡献（内部换算为 |integral| ≤ integralMax/|ki|）
 * derivativeLpfAlpha：D 项一阶 LPF 系数；语义见文件头注释 */
void pidInit(PID *obj, float integralMax, float target, float kp, float ki, float kd, float derivativeLpfAlpha);
void pidProcess(PID *obj, float currentValue);
/* 清零 PID 运行状态（expectedValue/lastError/beforeLastError/integral/controlValue + 重置 D 项 LPF），
 * 保留 kp/ki/kd/integralMax/process 等配置；用于使能切换、急停、回放等需复位累加器的场景。 */
void pidReset(PID *obj);

#endif /* _PID_H_ */
```

core2_cyt4bb7/project/code/sensor/chassisAttitude.c

SHA-256

:

27925a23c80c1183a17748fa95120b0146b20dede06009e634a0488b3ed1f448

代码 (c)

```
#include "chassisAttitude.h"

#include <math.h>
#include <string.h>

#define ATTITUDE_TWO_KP (1.0f)
#define ATTITUDE_TWO_KI (0.0f)
#define ATTITUDE_ACCEL_NORM_SQ_MIN (0.25f)
#define ATTITUDE_ACCEL_NORM_SQ_MAX (2.25f)
#define ATTITUDE_Q_NORM_SQ_MIN (0.25f)
#define ATTITUDE_Q_NORM_SQ_MAX (4.0f)
#define ATTITUDE_COS_PITCH_MIN (0.10f)
```

```

static uint8_t finite3(float x, float y, float z)
{
    return (uint8_t) (isfinite(x) && isfinite(y) && isfinite(z));
}

static float wrapPi(float angle)
{
    const float pi = 3.14159265358979323846f;
    const float twoPi = 6.28318530717958647692f;
    while (angle > pi) angle -= twoPi;
    while (angle < -pi) angle += twoPi;
    return angle;
}

static uint8_t normalizeQuaternion(chassisAttitudeStruct *attitude)
{
    float normSq = attitude->q[0] * attitude->q[0] +
        attitude->q[1] * attitude->q[1] +
        attitude->q[2] * attitude->q[2] +
        attitude->q[3] * attitude->q[3];
    float invNorm;
    uint8_t i;

    if (!isfinite(normSq) || normSq < ATTITUDE_Q_NORM_SQ_MIN ||
        normSq > ATTITUDE_Q_NORM_SQ_MAX) return 0u;
    invNorm = 1.0f / sqrtf(normSq);
    if (!isfinite(invNorm)) return 0u;
    for (i = 0u; i < 4u; i++) attitude->q[i] *= invNorm;
    return 1u;
}

static uint8_t updateEuler(chassisAttitudeStruct *attitude,
    float gyroYRadps, float gyroZRadps)
{
    float q0 = attitude->q[0];
    float q1 = attitude->q[1];
    float q2 = attitude->q[2];
    float q3 = attitude->q[3];
    float sinPitch = 2.0f * (q0 * q2 - q3 * q1);
    float cosPitch;

    if (sinPitch > 1.0f) sinPitch = 1.0f;
    if (sinPitch < -1.0f) sinPitch = -1.0f;
    attitude->roll = atan2f(2.0f * (q0 * q1 + q2 * q3),
        1.0f - 2.0f * (q1 * q1 + q2 * q2));
    attitude->pitch = asinf(sinPitch);
    attitude->yaw = wrapPi(atan2f(2.0f * (q0 * q3 + q1 * q2),
        1.0f - 2.0f * (q2 * q2 + q3 * q3)));
    cosPitch = cosf(attitude->pitch);
    if (!finite3(attitude->roll, attitude->pitch, attitude->yaw) ||
        !isfinite(cosPitch) || fabsf(cosPitch) < ATTITUDE_COS_PITCH_MIN)
        return 0u;
    attitude->yawRate = (sinf(attitude->roll) * gyroYRadps +
        cosf(attitude->roll) * gyroZRadps) / cosPitch;
    return (uint8_t) isfinite(attitude->yawRate);
}

uint8_t chassisAttitudeInit(chassisAttitudeStruct *attitude,
    float accX, float accY, float accZ)
{
    float normSq;
    float roll;
    float pitch;
    float cr;
    float sr;
    float cp;
    float sp;

    if (attitude == 0 || !finite3(accX, accY, accZ)) return 0u;
    memset(attitude, 0, sizeof(*attitude));
    normSq = accX * accX + accY * accY + accZ * accZ;
    if (!isfinite(normSq) || normSq < ATTITUDE_ACCEL_NORM_SQ_MIN ||
        normSq > ATTITUDE_ACCEL_NORM_SQ_MAX) return 0u;
    roll = atan2f(accY, accZ);
    pitch = atan2f(-accX, sqrtf(accY * accY + accZ * accZ));
    cr = cosf(0.5f * roll);
    sr = sinf(0.5f * roll);
    cp = cosf(0.5f * pitch);
    sp = sinf(0.5f * pitch);

```



```

attitude->q[0] = cr * cp;
attitude->q[1] = sr * cp;
attitude->q[2] = cr * sp;
attitude->q[3] = -sr * sp;
if (!normalizeQuaternion(attitude) || !updateEuler(attitude, 0.0f, 0.0f))
    return 0u;
attitude->valid = 1u;
attitude->accelCorrectionActive = 1u;
return 1u;
}

uint8_t chassisAttitudeUpdate(chassisAttitudeStruct *attitude,
    float gyroXRadps, float gyroYRadps, float gyroZRadps,
    float accX, float accY, float accZ, float dtS)
{
    float q0;
    float q1;
    float q2;
    float q3;
    float accNormSq;
    float halfDt;
    float measuredGyroY;
    float measuredGyroZ;

    if (attitude == 0) return 0u;
    if (attitude->valid == 0u ||
        !finite3(gyroXRadps, gyroYRadps, gyroZRadps) ||
        !isfinite(dtS) || dtS <= 0.0f)
    {
        attitude->valid = 0u;
        attitude->accelCorrectionActive = 0u;
        attitude->yawRate = 0.0f;
        return 0u;
    }
    q0 = attitude->q[0]; q1 = attitude->q[1];
    q2 = attitude->q[2]; q3 = attitude->q[3];
    measuredGyroY = gyroYRadps;
    measuredGyroZ = gyroZRadps;
    attitude->accelCorrectionActive = 0u;
    accNormSq = accX * accX + accY * accY + accZ * accZ;
    if (!finite3(accX, accY, accZ) || !isfinite(accNormSq))
    {
        attitude->valid = 0u;
        attitude->yawRate = 0.0f;
        return 0u;
    }
    /* 运行期有限加速度模长越界只表示该拍不适合重力校正：姿态继续由陀螺积分。 */
    if (accNormSq >= ATTITUDE_ACCEL_NORM_SQ_MIN &&
        accNormSq <= ATTITUDE_ACCEL_NORM_SQ_MAX)
    {
        float invAccNorm = 1.0f / sqrtf(accNormSq);
        float halfVx;
        float halfVy;
        float halfVz;
        float halfEx;
        float halfEy;
        float halfEz;
        accX *= invAccNorm; accY *= invAccNorm; accZ *= invAccNorm;
        halfVx = q1 * q3 - q0 * q2;
        halfVy = q0 * q1 + q2 * q3;
        halfVz = q0 * q0 - 0.5f + q3 * q3;
        halfEx = accY * halfVz - accZ * halfVy;
        halfEy = accZ * halfVx - accX * halfVz;
        halfEz = accX * halfVy - accY * halfVx;
        if (ATTITUDE_TWO_KI > 0.0f)
        {
            attitude->integral[0] += ATTITUDE_TWO_KI * halfEx * dtS;
            attitude->integral[1] += ATTITUDE_TWO_KI * halfEy * dtS;
            attitude->integral[2] += ATTITUDE_TWO_KI * halfEz * dtS;
            gyroXRadps += attitude->integral[0];
            gyroYRadps += attitude->integral[1];
            gyroZRadps += attitude->integral[2];
        }
        gyroXRadps += ATTITUDE_TWO_KP * halfEx;
        gyroYRadps += ATTITUDE_TWO_KP * halfEy;
        gyroZRadps += ATTITUDE_TWO_KP * halfEz;
        attitude->accelCorrectionActive = 1u;
    }
}

```

```

halfDt = 0.5f * dtS;
attitude->q[0] += (-q1 * gyroXRadps - q2 * gyroYRadps - q3 * gyroZRadps) * halfDt;
attitude->q[1] += ( q0 * gyroXRadps + q2 * gyroZRadps - q3 * gyroYRadps) * halfDt;
attitude->q[2] += ( q0 * gyroYRadps - q1 * gyroZRadps + q3 * gyroXRadps) * halfDt;
attitude->q[3] += ( q0 * gyroZRadps + q1 * gyroYRadps - q2 * gyroXRadps) * halfDt;
if (!normalizeQuaternion(attitude))
    !updateEuler(attitude, measuredGyroY, measuredGyroZ))
{
    attitude->valid = 0u;
    attitude->yawRate = 0.0f;
    return 0u;
}
return 1u;
}

```

core2_cyt4bb7/project/code/sensor/chassisAttitude.h

SHA-256

:

18eab082211820ab3b0bf6d6d5ca48351c1e1160ba6882c24140fbfe9e8b70fc

代码 (c)

```

#ifndef _CHASSIS_ATTITUDE_H_
#define _CHASSIS_ATTITUDE_H_

#include <stdint.h>

typedef struct
{
    float q[4];
    float integral[3];
    float roll;
    float pitch;
    float yaw;
    float yawRate;
    uint8_t valid;
    uint8_t accelCorrectionActive;
} chassisAttitudeStruct;

uint8_t chassisAttitudeInit(chassisAttitudeStruct *attitude,
    float accX, float accY, float accZ);
uint8_t chassisAttitudeUpdate(chassisAttitudeStruct *attitude,
    float gyroXRadps, float gyroYRadps, float gyroZRadps,
    float accX, float accY, float accZ, float dtS);

#endif

```

core2_cyt4bb7/project/code/sensor/currentSense.c

SHA-256

:

7d06a62559298dc00326fa23fc3940bc09bcf343d528924a787d523419b30b32

代码 (c)

```

#include "currentSense.h"

#include "lowPassFilter.h"
#include "zf_common_interrupt.h" /* 单一 CPUIntIdx2 保存/屏蔽 + __DMB */
#include "zf_driver_adc.h"

#include "configChassisPhysical.h"
#include "configChassisPolarity.h"
#include "pinMap.h"

typedef struct
{
    uint16_t rawValue;
    uint16_t zeroOffset;
    int16_t currentValue;
    LowPassFilterData filter;
}

```

```

} currentSenseChannelStateStruct;

typedef struct
{
    currentSenseChannelStateStruct fl;
    currentSenseChannelStateStruct fr;
    currentSenseChannelStateStruct bl;
    currentSenseChannelStateStruct br;
    uint32_t seq;
    uint8_t valid;
} currentSenseContextStruct;

static currentSenseContextStruct currentSenseContext = {
    {0u, 2048u, 0, {0}},
    {0u, 2048u, 0, {0}},
    {0u, 2048u, 0, {0}},
    {0u, 2048u, 0, {0}},
    0u,
    0u
};

static uint16_t currentSenseSampleChannel(adc_channel_enum channel, uint8_t sampleCount);

void currentSenseInit(void)
{
    adc_init(CURRENT_SENSE_FL_ADC, ADC_12BIT);
    adc_init(CURRENT_SENSE_FR_ADC, ADC_12BIT);
    adc_init(CURRENT_SENSE_BL_ADC, ADC_12BIT);
    adc_init(CURRENT_SENSE_BR_ADC, ADC_12BIT);

    lowPassFilterInit(&currentSenseContext.fl.filter, CURRENT_SENSE_LPF_ALPHA);
    lowPassFilterInit(&currentSenseContext.fr.filter, CURRENT_SENSE_LPF_ALPHA);
    lowPassFilterInit(&currentSenseContext.bl.filter, CURRENT_SENSE_LPF_ALPHA);
    lowPassFilterInit(&currentSenseContext.br.filter, CURRENT_SENSE_LPF_ALPHA);
    currentSenseContext.valid = 1u;
}

void currentSenseClearOffset(uint8_t sampleCount)
{
    uint16_t fl = currentSenseSampleChannel(CURRENT_SENSE_FL_ADC, sampleCount);
    uint16_t fr = currentSenseSampleChannel(CURRENT_SENSE_FR_ADC, sampleCount);
    uint16_t bl = currentSenseSampleChannel(CURRENT_SENSE_BL_ADC, sampleCount);
    uint16_t br = currentSenseSampleChannel(CURRENT_SENSE_BR_ADC, sampleCount);
    currentSenseContext.fl.zeroOffset = fl;
    currentSenseContext.fr.zeroOffset = fr;
    currentSenseContext.bl.zeroOffset = bl;
    currentSenseContext.br.zeroOffset = br;
}

void currentSenseSample(void)
{
    currentSenseContext.fl.rawValue = currentSenseSampleChannel(CURRENT_SENSE_FL_ADC, CURRENT_SENSE_FILTER_COUNT);
    currentSenseContext.fr.rawValue = currentSenseSampleChannel(CURRENT_SENSE_FR_ADC, CURRENT_SENSE_FILTER_COUNT);
    currentSenseContext.bl.rawValue = currentSenseSampleChannel(CURRENT_SENSE_BL_ADC, CURRENT_SENSE_FILTER_COUNT);
    currentSenseContext.br.rawValue = currentSenseSampleChannel(CURRENT_SENSE_BR_ADC, CURRENT_SENSE_FILTER_COUNT);

    /* 乘电流方向系数(configChassisPolarity.h)：前进 → +电流反馈。各轮 ±1 由开环实测辨识。 */
    currentSenseContext.fl.currentValue = (int16_t) (CURRENT_SENSE_FL_DIR * ((int16_t) currentSenseContext.fl.rawValue - (int16_t)
currentSenseContext.fl.zeroOffset));
    currentSenseContext.fr.currentValue = (int16_t) (CURRENT_SENSE_FR_DIR * ((int16_t) currentSenseContext.fr.rawValue - (int16_t)
currentSenseContext.fr.zeroOffset));
    currentSenseContext.bl.currentValue = (int16_t) (CURRENT_SENSE_BL_DIR * ((int16_t) currentSenseContext.bl.rawValue - (int16_t)
currentSenseContext.bl.zeroOffset));
    currentSenseContext.br.currentValue = (int16_t) (CURRENT_SENSE_BR_DIR * ((int16_t) currentSenseContext.br.rawValue - (int16_t)
currentSenseContext.br.zeroOffset));

    lowPassFilter(&currentSenseContext.fl.filter, (float) currentSenseContext.fl.currentValue);
    lowPassFilter(&currentSenseContext.fr.filter, (float) currentSenseContext.fr.currentValue);
    lowPassFilter(&currentSenseContext.bl.filter, (float) currentSenseContext.bl.currentValue);
    lowPassFilter(&currentSenseContext.br.filter, (float) currentSenseContext.br.currentValue);

    currentSenseContext.fl.currentValue = (int16_t) currentSenseContext.fl.filter.filteredValue;
    currentSenseContext.fr.currentValue = (int16_t) currentSenseContext.fr.filter.filteredValue;
    currentSenseContext.bl.currentValue = (int16_t) currentSenseContext.bl.filter.filteredValue;
    currentSenseContext.br.currentValue = (int16_t) currentSenseContext.br.filter.filteredValue;
    currentSenseContext.seq++;
}

void currentSenseGetSnapshot(currentSenseSnapshotStruct *out)
{

```

```

uint32_t irqWasEnabled;

if (0 == out) return;

/* 采样在 PIT_CH1，消费在同一 CPUIntIdx2 上的 CH0 或前台。只屏蔽该共享 IRQ，
 * 一次复制四轮结果；避免在 20kHz 采样 ISR 内额外发布镜像和 DMB。*/
irqWasEnabled = NVIC_GetEnableIRQ(CPUIntIdx2_IRQn);
NVIC_DisableIRQ(CPUIntIdx2_IRQn);
__DMB();
out->fl = currentSenseContext.fl.currentValue;
out->fr = currentSenseContext.fr.currentValue;
out->bl = currentSenseContext.bl.currentValue;
out->br = currentSenseContext.br.currentValue;
out->seq = currentSenseContext.seq;
out->valid = currentSenseContext.valid;
__DMB();
if (irqWasEnabled != 0u) NVIC_EnableIRQ(CPUIntIdx2_IRQn);
}

static uint16_t currentSenseSampleChannel(adc_channel_enum channel, uint8_t sampleCount)
{
    if (sampleCount == 0u)
    {
        return (uint16_t) adc_convert(channel);
    }

    return (uint16_t) adc_mean_filter_convert(channel, (uint8_t) sampleCount);
}

```

core2_cyt4bb7/project/code/sensor/currentSense.h

SHA-256

:

6d0d33deed1e02209322dd6256d60b898cdd210fa03c2e492602c9f83116411a

代码 (c)

```

#ifndef _CURRENT_SENSE_H_
#define _CURRENT_SENSE_H_

#include <stdint.h>

typedef struct
{
    int16_t fl;
    int16_t fr;
    int16_t bl;
    int16_t br;
    uint32_t seq;
    uint8_t valid;
} currentSenseSnapshotStruct;

void currentSenseInit(void);
void currentSenseClearOffset(uint8_t sampleCount);
void currentSenseSample(void);
void currentSenseGetSnapshot(currentSenseSnapshotStruct *out);

#endif

```

core2_cyt4bb7/project/code/sensor/encoder.c

SHA-256

:

b90e1f101ecf4099c5c9f6c13194fa072f9cb9884fb8af14b7ab5b39b4a174e0

代码 (c)

```

#include "encoder.h"

#include "zf_driver_encoder.h"

#include "configChassisPolarity.h"

```

```

#include "pinMap.h"

void encoderInit(void)
{
    encoder_quad_init(ENCODER_FL_INDEX, ENCODER_FL_CH1, ENCODER_FL_CH2);
    encoder_quad_init(ENCODER_FR_INDEX, ENCODER_FR_CH1, ENCODER_FR_CH2);
    encoder_quad_init(ENCODER_BL_INDEX, ENCODER_BL_CH1, ENCODER_BL_CH2);
    encoder_quad_init(ENCODER_BR_INDEX, ENCODER_BR_CH1, ENCODER_BR_CH2);

    encoderClearCount();
}

encoderDataStruct encoderGetCount(void)
{
    encoderDataStruct data = {0};

    /* 乘编码器方向系数(configChassisPolarity.h): 前进 → +转速反馈。各轮 ±1 由开环实测辨识。 */
    data.flCount = (int16_t) (ENCODER_FL_DIR * encoder_get_count(ENCODER_FL_INDEX));
    data.frCount = (int16_t) (ENCODER_FR_DIR * encoder_get_count(ENCODER_FR_INDEX));
    data.blCount = (int16_t) (ENCODER_BL_DIR * encoder_get_count(ENCODER_BL_INDEX));
    data.brCount = (int16_t) (ENCODER_BR_DIR * encoder_get_count(ENCODER_BR_INDEX));

    return data;
}

encoderDataStruct encoderSample(void)
{
    encoderDataStruct data = encoderGetCount();

    encoderClearCount();

    return data;
}

void encoderClearCount(void)
{
    encoder_clear_count(ENCODER_FL_INDEX);
    encoder_clear_count(ENCODER_FR_INDEX);
    encoder_clear_count(ENCODER_BL_INDEX);
    encoder_clear_count(ENCODER_BR_INDEX);
}

```

core2_cyt4bb7/project/code/sensor/encoder.h

SHA-256

:

38eb6eaf93e5b1b60af38b15f604d912af4097e8a6b4b169643b10513154fd04

代码 (c)

```

#ifndef _ENCODER_H_
#define _ENCODER_H_

#include <stdint.h>

typedef struct
{
    int16_t flCount;
    int16_t frCount;
    int16_t blCount;
    int16_t brCount;
} encoderDataStruct;

void encoderInit(void);
encoderDataStruct encoderGetCount(void);
encoderDataStruct encoderSample(void);
void encoderClearCount(void);

#endif

```

core2_cyt4bb7/project/code/sensor/IMU.c

SHA-256

:

9ebab5b76d2aaf635b8d23f083d629d02afcbbb8b58680f72c8484ee2c03d745

代码 (c)

```
#include "IMU.h"

#include "zf_device_imu660rb.h"
#include "zf_common_interrupt.h"
#include "zf_driver_delay.h"

#include "chassisAttitude.h"
#include "commonDef.h"
#include "configChassisPhysical.h"

typedef struct
{
    imuSnapshotStruct snapshot;
    chassisAttitudeStruct attitude;
    int16_t gyroOffset[3];
    uint8_t attitudeFaultReported;
} imuContextStruct;

typedef struct
{
    int16_t gyroX;
    int16_t gyroY;
    int16_t gyroZ;
    int16_t accX;
    int16_t accY;
    int16_t accZ;
} imuRawSampleStruct;

/* 1ms 采样下保留 63ms 前台阻塞裕量；满队列仍 fail-closed 并要求新运动授权。*/
#define IMU_SAMPLE_QUEUE_SIZE (64u)

static imuContextStruct imuContext = {0};
static volatile imuSnapshotStruct imuPublished = {0};
static volatile uint32_t imuPublishGeneration = 0u;
static imuRawSampleStruct imuSampleQueue[IMU_SAMPLE_QUEUE_SIZE];
static volatile uint8_t imuSampleWriteIndex = 0u;
static volatile uint8_t imuSampleReadIndex = 0u;
static volatile uint8_t imuSampleOverflow = 0u;

static void getOffsets(void);
static int16_t imuMapGyroX(void);
static int16_t imuMapGyroY(void);
static int16_t imuMapGyroZ(void);
static int16_t imuMapAccX(void);
static int16_t imuMapAccY(void);
static int16_t imuMapAccZ(void);
static void imuPublish(void);

uint8_t IMUInit(void)
{
    float accX;
    float accY;
    float accZ;
    uint8_t imuInitState = imu660rb_init();

    if (imuInitState != 0u) return imuInitState;
    getOffsets();
    imu660rb_get_acc();
    imuContext.snapshot.accX = imuMapAccX();
    imuContext.snapshot.accY = imuMapAccY();
    imuContext.snapshot.accZ = imuMapAccZ();
    accX = imu660rb_acc_transition(imuContext.snapshot.accX);
    accY = imu660rb_acc_transition(imuContext.snapshot.accY);
    accZ = imu660rb_acc_transition(imuContext.snapshot.accZ);
    if (chassisAttitudeInit(&imuContext.attitude, accX, accY, accZ) == 0u)
    {
        imuContext.snapshot.valid = 0u;
    }
}
```

```

        imuPublish();
        return 1u;
    }
    imuContext.snapshot.valid = 1u;
    imuContext.snapshot.fresh = 1u;
    imuContext.snapshot.healthy = 1u;
    imuContext.snapshot.accelCorrectionActive = 1u;
    imuContext.snapshot.q0 = imuContext.attitude.q[0];
    imuContext.snapshot.q1 = imuContext.attitude.q[1];
    imuContext.snapshot.q2 = imuContext.attitude.q[2];
    imuContext.snapshot.q3 = imuContext.attitude.q[3];
    imuContext.snapshot.roll = imuContext.attitude.roll;
    imuContext.snapshot.pitch = imuContext.attitude.pitch;
    imuContext.snapshot.yawAngle = imuContext.attitude.yaw;
    imuContext.snapshot.yawRate = 0.0f;
    imuPublish();
    return 0u;
}

void IMUUpdate(void)
{
    uint8_t writeIndex = imuSampleWriteIndex;
    uint8_t nextIndex = (uint8_t)((writeIndex + 1u) % IMU_SAMPLE_QUEUE_SIZE);

    /* PIT ISR 只做传感器采样与 SPSC 入队；四元数/三角函数全部在前台 IMUProcess。*/
    imu660rb_get_gyro();
    imu660rb_get_acc();
    if (nextIndex == imuSampleReadIndex)
    {
        imuSampleOverflow = 1u;
        return;
    }
    imuSampleQueue[writeIndex].gyroX = imuMapGyroX();
    imuSampleQueue[writeIndex].gyroY = imuMapGyroY();
    imuSampleQueue[writeIndex].gyroZ = imuMapGyroZ();
    imuSampleQueue[writeIndex].accX = imuMapAccX();
    imuSampleQueue[writeIndex].accY = imuMapAccY();
    imuSampleQueue[writeIndex].accZ = imuMapAccZ();
    __DMB();
    imuSampleWriteIndex = nextIndex;
}

void IMUProcess(void)
{
    float gyroX;
    float gyroY;
    float gyroZ;
    float accX;
    float accY;
    float accZ;
    imuRawSampleStruct sample;
    uint8_t overflowed;
    uint32_t primask;

    primask = __get_PRIMASK();
    __disable_irq();
    overflowed = imuSampleOverflow;
    if (overflowed != 0u)
    {
        imuSampleOverflow = 0u;
        imuSampleReadIndex = imuSampleWriteIndex;
    }
    __set_PRIMASK(primask);
    if (overflowed != 0u)
    {
        imuContext.attitude.valid = 0u;
        imuContext.attitudeFaultReported = 1u;
        imuContext.snapshot.faultSeq++;
        imuContext.snapshot.seq++;
        imuContext.snapshot.valid = 0u;
        imuContext.snapshot.fresh = 0u;
        imuContext.snapshot.healthy = 0u;
        imuContext.snapshot.yawRate = 0.0f;
        imuPublish();
        return;
    }

    while (imuSampleReadIndex != imuSampleWriteIndex)
    {

```

```

uint8_t readIndex = imuSampleReadIndex;
__DMB();
sample = imuSampleQueue[readIndex];
imuSampleReadIndex = (uint8_t)((readIndex + 1u) % IMU_SAMPLE_QUEUE_SIZE);
imuContext.snapshot.gyroX = (int16_t)(sample.gyroX - imuContext.gyroOffset[0]);
imuContext.snapshot.gyroY = (int16_t)(sample.gyroY - imuContext.gyroOffset[1]);
imuContext.snapshot.gyroZ = (int16_t)(sample.gyroZ - imuContext.gyroOffset[2]);
imuContext.snapshot.accX = sample.accX;
imuContext.snapshot.accY = sample.accY;
imuContext.snapshot.accZ = sample.accZ;
gyroX = toRad(imu660rb_gyro_transition(imuContext.snapshot.gyroX));
gyroY = toRad(imu660rb_gyro_transition(imuContext.snapshot.gyroY));
gyroZ = toRad(imu660rb_gyro_transition(imuContext.snapshot.gyroZ));
accX = imu660rb_acc_transition(imuContext.snapshot.accX);
accY = imu660rb_acc_transition(imuContext.snapshot.accY);
accZ = imu660rb_acc_transition(imuContext.snapshot.accZ);
if (imuContext.attitude.valid == 0u)
{
    (void) chassisAttitudeInit(&imuContext.attitude, accX, accY, accZ);
}
else
{
    (void) chassisAttitudeUpdate(&imuContext.attitude,
        gyroX, gyroY, gyroZ, accX, accY, accZ, IMU_SAMPLE_PERIOD_S);
}
if (imuContext.attitude.valid == 0u)
{
    if (imuContext.attitudeFaultReported == 0u)
    {
        imuContext.snapshot.faultSeq++;
        imuContext.attitudeFaultReported = 1u;
    }
    imuContext.snapshot.valid = 0u;
    imuContext.snapshot.fresh = 1u;
    imuContext.snapshot.healthy = 0u;
    imuContext.snapshot.yawRate = 0.0f;
}
else
{
    imuContext.attitudeFaultReported = 0u;
    imuContext.snapshot.q0 = imuContext.attitude.q[0];
    imuContext.snapshot.q1 = imuContext.attitude.q[1];
    imuContext.snapshot.q2 = imuContext.attitude.q[2];
    imuContext.snapshot.q3 = imuContext.attitude.q[3];
    imuContext.snapshot.roll = imuContext.attitude.roll;
    imuContext.snapshot.pitch = imuContext.attitude.pitch;
    imuContext.snapshot.yawAngle = imuContext.attitude.yaw;
    imuContext.snapshot.yawRate = imuContext.attitude.yawRate;
    imuContext.snapshot.accelCorrectionActive =
        imuContext.attitude.accelCorrectionActive;
    imuContext.snapshot.fresh = 1u;
    imuContext.snapshot.healthy = 1u;
    imuContext.snapshot.valid = 1u;
}
imuContext.snapshot.seq++;
imuPublish();
}
}

uint8_t IMUTryGetSnapshot(imuSnapshotStruct *out)
{
    uint8_t attempt;
    imuSnapshotStruct snapshot;

    if (out == 0) return 0u;
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        uint32_t before = imuPublishGeneration;
        uint32_t after;
        if ((before & 1u) != 0u) continue;
        __DMB();
        snapshot = imuPublished;
        __DMB();
        after = imuPublishGeneration;
        if ((before == after) && ((after & 1u) == 0u))
        {
            *out = snapshot;
            return 1u;
        }
    }
}

```



```

    }
    /* 前台 writer 被 PIT 抢占时 generation 会保持奇数；碰撞不能伪造成全零 IMU 事实。 */
    return 0u;
}

#ifdef(CORE2_IMU_HOST_TEST)
void imuHostTestSetPublished(imuSnapshotStruct snapshot, uint32_t generation)
{
    imuPublished = snapshot;
    imuPublishGeneration = generation;
}
#endif

static void getOffsets(void)
{
    system_delay_ms(500);

    int32_t sumX = 0;
    int32_t sumY = 0;
    int32_t sumZ = 0;
    uint16_t i;
    for (i = 0u; i < 200u; i++)
    {
        imu660rb_get_gyro();
        sumX += imuMapGyroX();
        sumY += imuMapGyroY();
        sumZ += imuMapGyroZ();
        system_delay_ms(5);
    }
    imuContext.gyroOffset[0] = (int16_t) (sumX / 200);
    imuContext.gyroOffset[1] = (int16_t) (sumY / 200);
    imuContext.gyroOffset[2] = (int16_t) (sumZ / 200);
}

/* 传感器 Z/X/-Y 映射到车体 前/左/上， gyro 与 acc 共用同一正交变换。 */
static int16_t imuMapGyroX(void) { return imu660rb_gyro_z; }
static int16_t imuMapGyroY(void) { return imu660rb_gyro_x; }
static int16_t imuMapGyroZ(void) { return (int16_t) -imu660rb_gyro_y; }
static int16_t imuMapAccX(void) { return imu660rb_acc_z; }
static int16_t imuMapAccY(void) { return imu660rb_acc_x; }
static int16_t imuMapAccZ(void) { return (int16_t) -imu660rb_acc_y; }

static void imuPublish(void)
{
    imuPublishGeneration++;
    __DMB();
    imuPublished = imuContext.snapshot;
    __DMB();
    imuPublishGeneration++;
}

```

core2_cyt4bb7/project/code/sensor/IMU.h

SHA-256

:

e22789e0c93af7c04b38a739dcb7d01a521f874af724bc6c0295b362746970da

代码 (c)

```

#ifndef _IMU_H_
#define _IMU_H_

#include <stdint.h>

typedef struct
{
    float q0;
    float q1;
    float q2;
    float q3;
    float roll;
    float pitch;
    float yawRate;
    float yawAngle;
    int16_t gyroX;
    int16_t gyroY;

```

```

int16_t gyroZ;
int16_t accX;
int16_t accY;
int16_t accZ;
uint32_t seq;
uint32_t faultSeq;
uint8_t fresh;
uint8_t healthy;
uint8_t valid;
uint8_t accelCorrectionActive;
} imuSnapshotStruct;

uint8_t IMUInit(void);
void IMUUpdate(void);
void IMUProcess(void);
uint8_t IMUTryGetSnapshot(imuSnapshotStruct *out);

#endif

```

core2_cyt4bb7/project/code/util/crc32ieee.c

SHA-256

:

363e996887ef0e2c68b3518217986f209f0760bcd89ff2ca5cdd3a4801583aba

代码 (c)

```

#include "crc32ieee.h"

uint32_t crc32ieeeCalc(const uint8_t* data, uint32_t len)
{
    uint32_t crc = 0xFFFFFFFFu;
    uint32_t i;

    for (i = 0u; i < len; i++)
    {
        uint8_t b;
        crc ^= (uint32_t) data[i];
        for (b = 0u; b < 8u; b++)
        {
            uint32_t mask = (uint32_t) (~((int32_t) (crc & 1u)));
            crc = (crc >> 1) ^ (0xEDB88320u & mask);
        }
    }
    return ~crc;
}

```

core2_cyt4bb7/project/code/util/crc32ieee.h

SHA-256

:

5442cded78a9802395246ceee07532bb7e5de239f12b419b4ecc309d2c9e1001

代码 (c)

```

#ifndef CRC32_IEEE_H
#define CRC32_IEEE_H

#include <stdint.h>

uint32_t crc32ieeeCalc(const uint8_t* data, uint32_t len);

#endif

```

core2_cyt4bb7/project/code/util/lowPassFilter.c

SHA-256

:

d68b6e1056374fb025bb13b6a7e89572c0d122868dc1ec0b5dc70ec6fc9f5f95

代码 (c)

```
/*
 * lowPassFilter.c
 *
 * Created: 20 250 319
 * Author: Hecate
 */
#include "lowPassFilter.h"

void lowPassFilterInit(LowPassFilterData *obj, float alpha)
{
    obj->alpha = alpha;

    obj->filteredValue = 0;
    obj->firstFlag = 1;
    obj->lastOutput = 0;
}

void lowPassFilter(LowPassFilterData *obj, const float value)
{
    if ((obj)->firstFlag == 1u)
    {
        (obj)->firstFlag = 0u;
        (obj)->lastOutput = value;
    }
    (obj)->filteredValue = (obj)->alpha * value + (1.0f - (obj)->alpha) * (obj)->lastOutput;
    (obj)->lastOutput = (obj)->filteredValue;
}
```

core2_cyt4bb7/project/code/util/lowPassFilter.h

SHA-256

:

99c8efecf422784d8604db328540087cc158a454c6afbd6605e4bffc9bf9df5e

代码 (c)

```
/*
 * lowPassFilter.h
 *
 * Created: 20 250 319
 * Author: Hecate
 */
#ifndef CODE_LOWPASSFILTER_H_
#define CODE_LOWPASSFILTER_H_

#include <stdint.h>

// 一阶低通滤波器句柄
typedef struct
{
    float alpha;
    float lastOutput;
    float filteredValue;
    uint8_t firstFlag;
} LowPassFilterData;

// 初始化滤波器参数
void lowPassFilterInit(LowPassFilterData *obj, float alpha);

// 输入新样本并更新滤波值
void lowPassFilter(LowPassFilterData *obj, const float value);

#endif /* CODE_LOWPASSFILTER_H_ */
```

core2_cyt4bb7/project/user/cm7_0_isr.c

SHA-256

:

7ba8e34ca3893a8a16a296f11aa40d76041306e903f2849560a15c956c17b5ab

代码 (c)

```
/*
*****
* CYT4BB Opensourec Library 即 ( CYT4BB 开源库) 是一个基于官方 SDK 接口的第三方开源库
* Copyright (c) 2022 SEEKFREE 逐飞科技
*
* 本文件是 CYT4BB 开源库的一部分
*
* CYT4BB 开源库 是免费软件
* 您可以根据自由软件基金会发布的 GPL (GNU General Public License, 即 GNU 通用公共许可证) 的条款
* 即 GPL 的第 3 版 (即 GPL3.0) 或 (您选择的) 任何后来的版本, 重新发布和/或修改它
*
* 本开源库的发布是希望它能发挥作用, 但并未对其作任何的保证
* 甚至没有隐含的适销性或适合特定用途的保证
* 更多细节请参见 GPL
*
* 您应该在收到本开源库的同时收到一份 GPL 的副本
* 如果没有, 请参阅<https://www.gnu.org/licenses/>
*
* 额外注明:
* 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
* 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
* 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
* 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明 (即本声明)
*
* 文件名称      cm7_0_isr
* 公司名称      成都逐飞科技有限公司
* 版本信息      查看 libraries/doc 文件夹内 version 文件 版本说明
* 开发环境      IAR 9.40.1
* 适用平台      CYT4BB
* 店铺链接      https://seekfree.taobao.com/
*
* 修改记录
* 日期          作者          备注
* 2024-1-9      pudding      first version
* 2024-5-14      pudding      新增 12 个 pit 周期中断 增加部分注释说明
* 2025-2-4      pudding      优化串口中断逻辑, 防止意外干扰导致的卡死问题, 优化串口波特率计算逻辑
* 2025-2-4      pudding      新增两个串口接口
*****
/

#include "zf_common_headfile.h"

#include "core0Loop.h"

// ***** PIT 中断函数 *****
void pit0_ch0_isr() // 定时器通道 0 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH0);

    pitProcess0();
}

void pit0_ch1_isr() // 定时器通道 1 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH1);
    pitProcess1();
}

void pit0_ch2_isr() // 定时器通道 2 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH2);
}

void pit0_ch10_isr() // 定时器通道 10 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH10);
}

void pit0_ch11_isr() // 定时器通道 11 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH11);
}

void pit0_ch12_isr() // 定时器通道 12 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH12);
}
}
```

```

void pit0_ch13_isr() // 定时器通道 13 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH13);
}

void pit0_ch14_isr() // 定时器通道 14 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH14);
}

void pit0_ch15_isr() // 定时器通道 15 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH15);
}

void pit0_ch16_isr() // 定时器通道 16 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH16);
}

void pit0_ch17_isr() // 定时器通道 17 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH17);
}

void pit0_ch18_isr() // 定时器通道 18 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH18);
}

void pit0_ch19_isr() // 定时器通道 19 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH19);
}

void pit0_ch20_isr() // 定时器通道 20 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH20);
}

void pit0_ch21_isr() // 定时器通道 21 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH21);
    tsl1401_collect_pit_handler();
}
// ***** PIT 中断函数 *****

// ***** 串口中断函数 *****
// 串口 0 默认作为调试串口
void uart0_isr(void)
{
    if (uart_isr_mask(UART_0)) // 串口 0 接收中断
    {
        #if DEBUG_UART_USE_INTERRUPT // 如果开启 debug 串口中断
            debug_interrupr_handler(); // 调用 debug 串口接收处理函数 数据会被 debug 环形缓冲区读取
        #endif // 如果修改了 DEBUG_UART_INDEX 那这段代码需要放到对应的串口中断去
    }
    else // 串口 0 发送中断
    {
    }
}

void uart1_isr(void)
{
    if (uart_isr_mask(UART_1)) // 串口 1 接收中断
    {
        wireless_module_uart_handler(); // 无线模块统一回调函数
    }
    else // 串口 1 发送中断
    {
    }
}

void uart2_isr(void)
{
    if (uart_isr_mask(UART_2)) // 串口 2 接收中断
    {

```

```

        gnss_uart_callback(); // GPS 模块回调函数
    }
    else // 串口 2 发送中断
    {
    }
}

void uart3_isr(void)
{
    if (uart_isr_mask(UART_3)) // 串口 3 接收中断
    {
    }
    else // 串口 3 发送中断
    {
    }
}

void uart4_isr(void)
{
    if (uart_isr_mask(UART_4)) // 串口 4 接收中断
    {
        uart_receiver_handler(); // 串口接收机回调函数
    }
    else // 串口 4 发送中断
    {
    }
}

void uart5_isr(void)
{
    if (uart_isr_mask(UART_5)) // 串口 5 接收中断
    {
    }
    else // 串口 5 发送中断
    {
    }
}

void uart6_isr(void)
{
    if (uart_isr_mask(UART_6)) // 串口 6 接收中断
    {
    }
    else // 串口 6 发送中断
    {
    }
}
// ***** 串口中断函数 *****

// ***** 外部中断函数 *****
void gpio_0_exti_isr() // 外部 GPIO_0 中断服务函数
{
}

void gpio_1_exti_isr() // 外部 GPIO_1 中断服务函数
{
    if (exti_flag_get(P01_0)) // 示例 P1_0 端口外部中断判断
    {
    }
    if (exti_flag_get(P01_1))
    {
    }
}

void gpio_2_exti_isr() // 外部 GPIO_2 中断服务函数
{
    if (exti_flag_get(P02_0))
    {
    }
    if (exti_flag_get(P02_4))
    {
    }
}

void gpio_3_exti_isr() // 外部 GPIO_3 中断服务函数
{
}

void gpio_4_exti_isr() // 外部 GPIO_4 中断服务函数

```

```

{
}

void gpio_5_exti_isr() // 外部 GPIO_5 中断服务函数
{
}

void gpio_6_exti_isr() // 外部 GPIO_6 中断服务函数
{
}

void gpio_7_exti_isr() // 外部 GPIO_7 中断服务函数
{
}

void gpio_8_exti_isr() // 外部 GPIO_8 中断服务函数
{
}

void gpio_9_exti_isr() // 外部 GPIO_9 中断服务函数
{
}

void gpio_10_exti_isr() // 外部 GPIO_10 中断服务函数
{
}

void gpio_11_exti_isr() // 外部 GPIO_11 中断服务函数
{
}

void gpio_12_exti_isr() // 外部 GPIO_12 中断服务函数
{
}

void gpio_13_exti_isr() // 外部 GPIO_13 中断服务函数
{
}

void gpio_14_exti_isr() // 外部 GPIO_14 中断服务函数
{
}

void gpio_15_exti_isr() // 外部 GPIO_15 中断服务函数
{
}

void gpio_16_exti_isr() // 外部 GPIO_16 中断服务函数
{
}

void gpio_17_exti_isr() // 外部 GPIO_17 中断服务函数
{
}

void gpio_18_exti_isr() // 外部 GPIO_18 中断服务函数
{
}

void gpio_19_exti_isr() // 外部 GPIO_19 中断服务函数
{
}

void gpio_20_exti_isr() // 外部 GPIO_20 中断服务函数
{
}

void gpio_21_exti_isr() // 外部 GPIO_21 中断服务函数
{
}

void gpio_22_exti_isr() // 外部 GPIO_22 中断服务函数
{
}

void gpio_23_exti_isr() // 外部 GPIO_23 中断服务函数
{
}

```

core2_cyt4bb7/project/user/cm7_1_isr.c

•

•

代码 (c)

```
#include "zf_common_headfile.h"
```

```
#include "core1Loop.h"
#include "THVD1450.h"
```

```
// ***** PIT 中断函数 *****
```

```
void pit0_ch0_isr() // 定时器通道 0 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH0);
}
```

```
void pit0_ch1_isr() // 定时器通道 1 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH1);
}
```

```
void pit0_ch2_isr() // 定时器通道 2 周期中断服务函数（通信核 1ms tick：链路年龄 + LoRa 超时/edge)
{
    pit_isr_flag_clear(PIT_CH2);

    pitProcess0();
}
```

```
void pit0_ch10_isr() // 定时器通道 10 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH10);
}
```



```

void pit0_ch11_isr() // 定时器通道 11 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH11);
}

void pit0_ch12_isr() // 定时器通道 12 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH12);
}

void pit0_ch13_isr() // 定时器通道 13 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH13);
}

void pit0_ch14_isr() // 定时器通道 14 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH14);
}

void pit0_ch15_isr() // 定时器通道 15 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH15);
}

void pit0_ch16_isr() // 定时器通道 16 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH16);
}

void pit0_ch17_isr() // 定时器通道 17 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH17);
}

void pit0_ch18_isr() // 定时器通道 18 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH18);
}

void pit0_ch19_isr() // 定时器通道 19 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH19);
}

void pit0_ch20_isr() // 定时器通道 20 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH20);
}

void pit0_ch21_isr() // 定时器通道 21 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH21);
    tsl1401_collect_pit_handler();
}
// ***** PIT 中断函数 *****

// ***** 串口中断函数 *****
// 串口 0 默认作为调试串口
void uart0_isr(void)
{
    if (uart_isr_mask(UART_0)) // 串口 0 接收中断
    {
        #if DEBUG_UART_USE_INTERRUPT // 如果开启 debug 串口中断
            debug_interrupr_handler(); // 调用 debug 串口接收处理函数 数据会被 debug 环形缓冲区读取
        #endif // 如果修改了 DEBUG_UART_INDEX 那这段代码需要放到对应的串口中断去
    }
    else // 串口 0 发送中断
    {
    }
}

void uart1_isr(void)
{
    /* 通信核 LoRa(UART1): uart_isr_mask 已在条件内把 FIFO 字节填入 SeekFree 软件缓冲,
    * 前台 loraRemotePoll 用 uart_query_byte 排空解析。此处不得再调 wireless_module_uart_handler——
    * 它会在 ISR 内消费 FIFO 并解析到驱动私有缓冲, 导致前台 loraRemotePoll 排空不到字节。*/
}

```

```

    if (uart_isr_mask(UART_1)) // 串口 1 接收中断（仅填缓冲）
    {
    }
    else // 串口 1 发送中断
    {
    }
}

void uart2_isr(void)
{
    if (uart_isr_mask(UART_2)) // 串口 2 接收中断
    {
        gnss_uart_callback();
    }
    else // 串口 2 发送中断
    {
    }
}

void uart3_isr(void)
{
    if (uart_isr_mask(UART_3)) // 串口 3 接收中断
    {
    }
    else // 串口 3 发送中断
    {
    }
}

void uart4_isr(void)
{
    /* 通信核 RS-485：驱动先清 FIFO 到本批暂存，再立即搬入 THVD 本地 SPSC ring。*/
    if (uart_isr_mask(UART_4))
    {
        thvd1450RxIrqHandler();
    }
    else // 串口 4 发送中断
    {
    }
}

void uart5_isr(void)
{
    if (uart_isr_mask(UART_5)) // 串口 5 接收中断
    {
    }
    else // 串口 5 发送中断
    {
    }
}

void uart6_isr(void)
{
    if (uart_isr_mask(UART_6)) // 串口 6 接收中断
    {
    }
    else // 串口 6 发送中断
    {
    }
}
// ***** 串口中断函数 *****

// ***** 外部中断函数 *****
void gpio_0_exti_isr() // 外部 GPIO_0 中断服务函数
{
}

void gpio_1_exti_isr() // 外部 GPIO_1 中断服务函数
{
    if (exti_flag_get(P01_0)) // 示例 P1_0 端口外部中断判断
    {
    }
    if (exti_flag_get(P01_1))
    {
    }
}

void gpio_2_exti_isr() // 外部 GPIO_2 中断服务函数
{
}

```

```

    if (exti_flag_get(P02_0))
    {
    }
    if (exti_flag_get(P02_4))
    {
    }
}

void gpio_3_exti_isr() // 外部 GPIO_3 中断服务函数
{
}

void gpio_4_exti_isr() // 外部 GPIO_4 中断服务函数
{
}

void gpio_5_exti_isr() // 外部 GPIO_5 中断服务函数
{
}

void gpio_6_exti_isr() // 外部 GPIO_6 中断服务函数
{
}

void gpio_7_exti_isr() // 外部 GPIO_7 中断服务函数
{
}

void gpio_8_exti_isr() // 外部 GPIO_8 中断服务函数
{
}

void gpio_9_exti_isr() // 外部 GPIO_9 中断服务函数
{
}

void gpio_10_exti_isr() // 外部 GPIO_10 中断服务函数
{
}

void gpio_11_exti_isr() // 外部 GPIO_11 中断服务函数
{
}

void gpio_12_exti_isr() // 外部 GPIO_12 中断服务函数
{
}

void gpio_13_exti_isr() // 外部 GPIO_13 中断服务函数
{
}

void gpio_14_exti_isr() // 外部 GPIO_14 中断服务函数
{
}

void gpio_15_exti_isr() // 外部 GPIO_15 中断服务函数
{
}

void gpio_16_exti_isr() // 外部 GPIO_16 中断服务函数
{
}

void gpio_17_exti_isr() // 外部 GPIO_17 中断服务函数
{
}

void gpio_18_exti_isr() // 外部 GPIO_18 中断服务函数
{
}

void gpio_19_exti_isr() // 外部 GPIO_19 中断服务函数
{
}

void gpio_20_exti_isr() // 外部 GPIO_20 中断服务函数
{
}

```

```
}

void gpio_21_exti_isr() // 外部 GPIO_21 中断服务函数
{
}

void gpio_22_exti_isr() // 外部 GPIO_22 中断服务函数
{
}

void gpio_23_exti_isr() // 外部 GPIO_23 中断服务函数
{
}

// ***** 串口中断函数 *****
```

core2_cyt4bb7/project/user/main_cm7_0.c

SHA-256 :
68b421c6f4fb20c0361f25c4d3b1b4035b4102af3991067b4532706b8f738421

代码（c）

```
/* *****
 * CYT4BB Opensourec Library 即（CYT4BB 开源库）是一个基于官方 SDK 接口的第三方开源库
 * Copyright (c) 2022 SEEKFREE 逐飞科技
 *
 * 本文件是 CYT4BB 开源库的一部分
 *
 * CYT4BB 开源库 是免费软件
 * 您可以根据自由软件基金会发布的 GPL（GNU General Public License，即 GNU 通用公共许可证）的条款
 * 即 GPL 的第 3 版（即 GPL3.0）或（您选择的）任何后来的版本，重新发布和/或修改它
 *
 * 本开源库的发布是希望它能发挥作用，但并未对其作任何的保证
 * 甚至没有隐含的适销性或适合特定用途的保证
 * 更多细节请参见 GPL
 *
 * 您应该在收到本开源库的同时收到一份 GPL 的副本
 * 如果没有，请参阅<https://www.gnu.org/licenses/>
 *
 * 额外注明：
 * 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
 * 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
 * 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
 * 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明（即本声明）
 *
 * 文件名称      main_cm7_0
 * 公司名称      成都逐飞科技有限公司
 * 版本信息      查看 libraries/doc 文件夹内 version 文件 版本说明
 * 开发环境      IAR 9.40.1
 * 适用平台      CYT4BB
 * 店铺链接      https://seekfree.taobao.com/
 *
 * 修改记录
 * 日期      作者      备注
 * 2024-1-4      pudding      first version
 * ***** /

#include "zf_common_headfile.h"

#include "core0Loop.h"

// 打开新的工程或者工程移动了位置务必执行以下操作
// 第一步 关闭上面所有打开的文件
// 第二步 project->clean 等待下方进度条走完

// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设

// ***** 代码区域 *****

/* =====
 * BEFORE EDITING THIS CORE — READ THESE TWO DOCS FIRST:
 * 1. <repo-root>/CLAUDE.md — multi-core architecture, naming rules, git workflow
```

```

* 2. ./project/code/PROJECT.md    — this core's modules, boundaries, hazards
* Each core has the SAME filenames (motor.c / IMU.c / PID.c) but DIFFERENT semantics.
* Do NOT copy patterns across cores. Do NOT edit two cores in one change.
* ===== */
int main(void)
{
    clock_init(SYSTEM_CLOCK_250M); // 时钟配置及系统初始化<务必保留>
    debug_init();                  // 调试串口信息初始化
    // 此处编写用户代码 例如外设初始化代码等

    processInit();

    // 此处编写用户代码 例如外设初始化代码等
    while (true)
    {
        // 此处编写需要循环执行的代码

        whileProcess();

        // 此处编写需要循环执行的代码
    }
}

// ***** 代码区域 *****

```

core2_cyt4bb7/project/user/main_cm7_1.c

SHA-256

:

363e25e84c266e455ba510f21d00be6a76c6b61238c8b3cf827431da3e8084e8

代码（c）

```

/*****
* CYT4BB Opensourec Library 即（CYT4BB 开源库）是一个基于官方 SDK 接口的第三方开源库
* Copyright (c) 2022 SEEKFREE 逐飞科技
*
* 本文件是 CYT4BB 开源库的一部分
*
* CYT4BB 开源库 是免费软件
* 您可以根据自由软件基金会发布的 GPL（GNU General Public License，即 GNU 通用公共许可证）的条款
* 即 GPL 的第 3 版（即 GPL3.0）或（您选择的）任何后来的版本，重新发布和/或修改它
*
* 本开源库的发布是希望它能发挥作用，但并未对其作任何的保证
* 甚至没有隐含的适销性或适合特定用途的保证
* 更多细节请参见 GPL
*
* 您应该在收到本开源库的同时收到一份 GPL 的副本
* 如果没有，请参阅<https://www.gnu.org/licenses/>
*
* 额外注明：
* 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
* 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
* 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
* 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明（即本声明）
*
* 文件名称      main_cm7_1
* 公司名称      成都逐飞科技有限公司
* 版本信息      查看 libraries/doc 文件夹内 version 文件 版本说明
* 开发环境      IAR 9.40.1
* 适用平台      CYT4BB
* 店铺链接      https://seekfree.taobao.com/
*
* 修改记录
* 日期          作者          备注
* 2024-1-4      pudding      first version
*****/

```

```
#include "zf_common_headfile.h"
```

```
#include "core1Loop.h"
```

```

// 打开新的工程或者工程移动了位置务必执行以下操作
// 第一步 关闭上面所有打开的文件
// 第二步 project->clean 等待下方进度条走完

```

```
// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设

// ***** 代码区域 *****

/* =====
* BEFORE EDITING THIS CORE — READ THESE TWO DOCS FIRST:
* 1. <repo-root>/CLAUDE.md — multi-core architecture, naming rules, git workflow
* 2. ./project/code/PROJECT.md — this core's modules, boundaries, hazards
* Each core has the SAME filenames (motor.c / IMU.c / PID.c) but DIFFERENT semantics.
* Do NOT copy patterns across cores. Do NOT edit two cores in one change.
* ===== */
int main(void)
{
    clock_init(SYSTEM_CLOCK_250M); // 时钟配置及系统初始化<务必保留>
    debug_info_init(); // 调试串口信息初始化

    // 此处编写用户代码 例如外设初始化代码等

    processInit();

    // 此处编写用户代码 例如外设初始化代码等
    while (true)
    {
        // 此处编写需要循环执行的代码

        whileProcess();

        // 此处编写需要循环执行的代码
    }
}

// ***** 代码区域 *****
```

core3_cyt4bb7

core3_cyt4bb7/project/code/app/core0Loop.c

SHA-256

:

a27059140415c287a01950738eaf92e8832a134bd62d8acfae5aebe92a393648

代码 (c)

```
#include "core0Loop.h"

#include <string.h>

#include "core3CoreDataTransfer.h"
#include "core3LinkCore3.h"
#include "configCore3Runtime.h"
#include "zf_driver_pit.h"

static volatile uint32_t g_linkTickMs;
static core3CoreLinkSnapshot g_linkShared;
static uint32_t g_visionGeneration;
static uint32_t g_visionPublishSeq;
static uint32_t g_visionPublishTickMs;
static uint32_t g_lastVisionProbeTickMs;
static uint32_t g_lastLinkPublishTickMs;
static uint8_t g_visionSharedFresh;
static uint8_t g_wirePaused;
static uint8_t g_forceVisionReadyReplay;
static uint32_t g_visionReadySeq;
static uint32_t g_appliedSeq;
static uint32_t g_planningSeq;
static uint32_t g_healthSeq;
static uint32_t g_paramGlobalRequestSeq;
static uint32_t g_paramGlobalStatePublishSeq;
static uint32_t g_paramGlobalStateConsumedSeq;
static core3CoreIngressEvent g_pendingIngress;
static uint8_t g_pendingIngressValid;
static core3CoreEgressEvent g_pendingEgress;
static uint8_t g_pendingEgressValid;
```

```

static core3CoreBorderRejection g_pendingBorderRejection;
static uint8_t g_pendingBorderRejectionValid;

static uint8_t paramGlobalTerminalAwaitingConsume(void)
{
    return (uint8_t) ((g_linkShared.paramGlobalStateSeq != 0u) &&
        (g_paramGlobalStateConsumedSeq !=
            g_linkShared.paramGlobalStateSeq) &&
        (paramSyncGlobalStateIsValid(&g_linkShared.paramGlobalState) != 0u) &&
        ((g_linkShared.paramGlobalState.flags &
            PARAM_SYNC_GLOBAL_FLAG_TERMINAL) != 0u));
}

static void clearSharedParamGlobalState(void)
{
    memset(&g_linkShared.paramGlobalState, 0,
        sizeof(g_linkShared.paramGlobalState));
    g_linkShared.paramGlobalStateSeq = 0u;
}

static void clearSessionPending(void)
{
    memset(&g_pendingIngress, 0, sizeof(g_pendingIngress));
    memset(&g_pendingEgress, 0, sizeof(g_pendingEgress));
    memset(&g_pendingBorderRejection, 0,
        sizeof(g_pendingBorderRejection));
    g_pendingIngressValid = 0u;
    g_pendingEgressValid = 0u;
    g_pendingBorderRejectionValid = 0u;
}

static uint8_t mergePendingBorderRejections(
    observerLinkPlanningTrackSnapshot *snapshot)
{
    if ((snapshot == NULL) || (g_pendingBorderRejectionValid == 0u)) return 0u;
    if ((snapshot->ownerSession != g_pendingBorderRejection.ownerSession) ||
        (snapshot->revision != g_pendingBorderRejection.revision) ||
        (snapshot->hash != g_pendingBorderRejection.hash))
        return 0u;
    return core3CoreBorderRejectionMerge(&g_pendingBorderRejection, snapshot);
}

static void commitPendingBorderRejections(uint8_t delivered)
{
    if ((delivered == 0u) ||
        (delivered > g_pendingBorderRejection.rejectedCount)) return;
    g_pendingBorderRejection.rejectedCount = (uint8_t)
        (g_pendingBorderRejection.rejectedCount - delivered);
    if (g_pendingBorderRejection.rejectedCount == 0u)
    {
        memset(&g_pendingBorderRejection, 0,
            sizeof(g_pendingBorderRejection));
        g_pendingBorderRejectionValid = 0u;
        return;
    }
    memmove(g_pendingBorderRejection.rejectedTrackIds,
        &g_pendingBorderRejection.rejectedTrackIds[delivered],
        (size_t) g_pendingBorderRejection.rejectedCount * sizeof(uint16_t));
}

static void publishLinkSnapshot(uint32_t tickMs, uint8_t force)
{
    observerLinkSharedConfig staged;
    core3LinkCore3Status status;

    if ((force == 0u) && (g_lastLinkPublishTickMs == tickMs)) return;
    g_lastLinkPublishTickMs = tickMs;
    core3LinkCore3GetStatus(&status);
    if ((status.ownerSession != g_linkShared.status.ownerSession) ||
        (status.bindingEpoch != g_linkShared.status.bindingEpoch))
    {
        clearSessionPending();
        core3CoreDataResetIngress(g_linkShared.bootGeneration);
        if (g_visionGeneration != 0u)
            core3CoreDataSyncEgressConsumer(g_visionGeneration);
    }
    g_linkShared.status = status;
    if (paramGlobalTerminalAwaitingConsume() == 0u)
    {

```

```

paramSyncGlobalState state;
if (core3LinkCore3TakeParamGlobalState(&state) != 0u)
{
    g_linkShared.paramGlobalState = state;
    g_paramGlobalStatePublishSeq++;
    if (g_paramGlobalStatePublishSeq == 0u)
    {
        g_paramGlobalStatePublishSeq = 1u;
        g_paramGlobalStateConsumedSeq = 0u;
    }
    g_linkShared.paramGlobalStateSeq =
        g_paramGlobalStatePublishSeq;
}
}
if (((g_wirePaused != 0u) ||
    (core3LinkCore3IsParamGlobalStateValid() == 0u)) &&
    (paramGlobalTerminalAwaitingConsume() == 0u))
    clearSharedParamGlobalState();
if (g_wirePaused != 0u)
{
    uint32_t localSession = g_linkShared.status.localSession;
    memset(&g_linkShared.status, 0, sizeof(g_linkShared.status));
    g_linkShared.status.localSession = localSession;
    g_linkShared.status.stagedMode = 0xFFu;
    g_linkShared.status.appliedMode = 0xFFu;
    memset(&g_linkShared.stagedConfig, 0, sizeof(g_linkShared.stagedConfig));
    g_linkShared.stagedConfigSeq = 0u;
    if (paramGlobalTerminalAwaitingConsume() == 0u)
        clearSharedParamGlobalState();
}
else if (core3LinkCore3GetStagedConfig(&staged) != 0u)
{
    if ((g_linkShared.stagedConfigSeq == 0u) ||
        (memcmp(&staged, &g_linkShared.stagedConfig, sizeof(staged)) != 0))
    {
        g_linkShared.stagedConfig = staged;
        g_linkShared.stagedConfigSeq++;
    }
}
else
{
    memset(&g_linkShared.stagedConfig, 0, sizeof(g_linkShared.stagedConfig));
    g_linkShared.stagedConfigSeq = 0u;
}
g_linkShared.publishSeq++;
g_linkShared.beginGeneration += 2u;
if (g_linkShared.beginGeneration == 0u) g_linkShared.beginGeneration = 2u;
g_linkShared.endGeneration = g_linkShared.beginGeneration;
core3CoreDataWriteLink(&g_linkShared);
}

static void consumeVisionSnapshot(void)
{
    core3CoreVisionSnapshot vision;
    uint32_t bootGeneration;
    uint32_t publishSeq;

    if (g_lastVisionProbeTickMs != g_linkTickMs)
    {
        g_lastVisionProbeTickMs = g_linkTickMs;
        if ((core3CoreDataProbeVision(&bootGeneration, &publishSeq) != 0u) &&
            ((bootGeneration != g_visionGeneration) ||
             (publishSeq != g_visionPublishSeq)) &&
            (core3CoreDataTryReadVision(&vision) != 0u))
        {
            if (vision.bootGeneration != g_visionGeneration)
            {
                uint32_t previousGeneration = g_visionGeneration;
                core3CoreDataSyncEgressConsumer(vision.bootGeneration);
                g_visionGeneration = vision.bootGeneration;
                g_visionPublishSeq = 0u;
                g_visionReadySeq = 0u;
                g_appliedSeq = 0u;
                g_planningSeq = 0u;
                g_healthSeq = 0u;
                g_paramGlobalRequestSeq = 0u;
                g_paramGlobalStateConsumedSeq = 0u;
                clearSessionPending();
                g_forceVisionReadyReplay = 1u;
            }
        }
    }
}

```



```

    g_wirePaused = 0u;
    if (previousGeneration != 0u)
    {
        core3CoreDataNoteSessionReset();
        core3LinkCore3Init(g_linkTickMs);
    }
}
if (vision.publishSeq != g_visionPublishSeq)
{
    if (g_visionSharedFresh == 0u)
    {
        clearSessionPending();
        g_forceVisionReadyReplay = 1u;
        g_wirePaused = 0u;
    }
    g_visionPublishSeq = vision.publishSeq;
    g_visionPublishTickMs = g_linkTickMs;
    g_visionSharedFresh = 1u;
}
if (g_visionSharedFresh != 0u)
{
    if ((g_forceVisionReadyReplay != 0u) ||
        (vision.visionReadySeq != g_visionReadySeq))
    {
        core3LinkCore3SetVisionReady(vision.visionReady);
        g_visionReadySeq = vision.visionReadySeq;
        g_forceVisionReadyReplay = 0u;
    }
    if ((vision.appliedSeq != 0u) &&
        (vision.appliedSeq != g_appliedSeq) &&
        (core3LinkCore3MarkConfigApplied(vision.appliedOwnerSession,
            vision.appliedRevision, vision.appliedHash,
            vision.appliedMode) != 0u))
    {
        g_appliedSeq = vision.appliedSeq;
        g_linkShared.appliedConsumedSeq = g_appliedSeq;
    }
    if ((vision.planningSeq != 0u) &&
        (vision.planningSeq != g_planningSeq))
    {
        uint8_t delivered;
        /* wire 时间戳在 cm7_0 域重锚；相机真实新鲜度由 HEALTH seq/age 独立证明。*/
        vision.planningSnapshot.captureTickMs = g_linkTickMs;
        delivered = mergePendingBorderRejections(
            &vision.planningSnapshot);
        if (core3LinkCore3QueuePlanningTrackSnapshot(
            &vision.planningSnapshot) != 0u)
        {
            g_planningSeq = vision.planningSeq;
            commitPendingBorderRejections(delivered);
        }
    }
    if ((vision.healthSeq != 0u) &&
        (vision.healthSeq != g_healthSeq) &&
        (core3LinkCore3PublishHealth(&vision.health) != 0u))
    {
        g_healthSeq = vision.healthSeq;
    }
    if ((vision.paramGlobalRequestSeq != 0u) &&
        (vision.paramGlobalRequestSeq != g_paramGlobalRequestSeq) &&
        (core3LinkCore3QueueParamGlobalRequest(
            &vision.paramGlobalRequest) != 0u))
    {
        g_paramGlobalRequestSeq = vision.paramGlobalRequestSeq;
    }
    if ((vision.paramGlobalStateConsumedSeq != 0u) &&
        (vision.paramGlobalStateConsumedSeq ==
            g_linkShared.paramGlobalStateSeq))
    {
        g_paramGlobalStateConsumedSeq =
            vision.paramGlobalStateConsumedSeq;
    }
}
}
}
if ((g_visionSharedFresh != 0u) &&
    ((uint32_t) (g_linkTickMs - g_visionPublishTickMs) >=
        CORE3_CORE_SHARED_STALE_MS))
{

```

```

    g_visionSharedFresh = 0u;
    g_wirePaused = 1u;
    g_forceVisionReadyReplay = 1u;
    clearSessionPending();
    core3LinkCore3SetVisionReady(0u);
    core3CoreDataNoteSharedStale();
}
}

static void forwardKvmIngress(void)
{
    if ((g_pendingIngressValid != 0u) &&
        (core3CoreDataPushIngress(&g_pendingIngress) != 0u))
    {
        g_pendingIngressValid = 0u;
    }
    if (g_pendingIngressValid != 0u) return;
    memset(&g_pendingIngress, 0, sizeof(g_pendingIngress));
    if (core3LinkCore3TakeKvmControl(
        &g_pendingIngress.payload.control) != 0u)
    {
        g_pendingIngress.kind = CORE3_CORE_INGRESS_KVM_CONTROL;
        g_pendingIngressValid = 1u;
    }
    else if (core3LinkCore3TakeKvmEvent(
        &g_pendingIngress.payload.event) != 0u)
    {
        g_pendingIngress.kind = CORE3_CORE_INGRESS_KVM_EVENT;
        g_pendingIngressValid = 1u;
    }
    if ((g_pendingIngressValid != 0u) &&
        (core3CoreDataPushIngress(&g_pendingIngress) != 0u))
    {
        g_pendingIngressValid = 0u;
    }
}

static uint8_t retainBorderRejection(
    const core3CoreBorderRejection *event)
{
    return core3CoreBorderRejectionAppend(&g_pendingBorderRejection,
        &g_pendingBorderRejectionValid, event);
}

static void consumeKvmEgress(void)
{
    uint8_t accepted = 0u;
    if (g_pendingEgressValid == 0u)
    {
        g_pendingEgressValid = core3CoreDataPopEgress(&g_pendingEgress,
            g_visionGeneration);
    }
    if (g_pendingEgressValid == 0u) return;
    if (g_pendingEgress.kind == CORE3_CORE_EGRESS_KVM_ACK)
    {
        accepted = core3LinkCore3QueueKvmAck(&g_pendingEgress.payload.ack);
    }
    else if (g_pendingEgress.kind == CORE3_CORE_EGRESS_KVM_MENU)
    {
        accepted = core3LinkCore3QueueKvmMenuStatus(
            &g_pendingEgress.payload.menuStatus);
    }
    else if (g_pendingEgress.kind == CORE3_CORE_EGRESS_BORDER_REJECTION)
    {
        accepted = retainBorderRejection(
            &g_pendingEgress.payload.borderRejection);
    }
    else
    {
        accepted = 1u;
    }
    if (accepted != 0u) g_pendingEgressValid = 0u;
}

void processInit(void)
{
    core3CoreDataTimingInit();
    memset(&g_linkShared, 0, sizeof(g_linkShared));
    g_linkShared.magic = CORE3_CORE_LINK_MAGIC;
}

```

```

g_linkShared.abiVersion = CORE3_CORE_ABI_VERSION;
g_linkShared.payloadBytes = sizeof(g_linkShared);
g_linkShared.bootGeneration = core3CoreDataMakeBootGeneration(0xC3700001u);
g_lastVisionProbeTickMs = UINT32_MAX;
g_lastLinkPublishTickMs = UINT32_MAX;
g_wirePaused = 1u;
g_forceVisionReadyReplay = 1u;
g_paramGlobalStatePublishSeq = 0u;
g_paramGlobalStateConsumedSeq = 0u;
core3CoreDataResetIngress(g_linkShared.bootGeneration);
g_linkShared.beginGeneration = 2u;
g_linkShared.endGeneration = 2u;
core3LinkCore3Init(0u);
publishLinkSnapshot(0u, 1u);
pit_ms_init(PIT_CH0, CORE3_RUNTIME_PIT_MS);
}

void pitProcess0(void)
{
    g_linkTickMs += CORE3_RUNTIME_PIT_MS;
}

void whileProcess(void)
{
    uint32_t tickMs = g_linkTickMs;
    if (g_wirePaused == 0u) consumeKvmEgress();
    consumeVisionSnapshot();
    if (g_wirePaused == 0u)
    {
        consumeKvmEgress();
        core3LinkCore3Poll(tickMs);
        forwardKvmIngress();
        core3LinkCore3FlushControl(tickMs);
        core3LinkCore3FlushTx(tickMs);
    }
    publishLinkSnapshot(tickMs, 0u);
}

```

core3_cyt4bb7/project/code/app/core0Loop.h

SHA-256

:

434bcd13d920fd7c1f0be99c8eb94ff65e1620a5d01b8adec82788be11009b2

代码 (c)

```

#ifndef CORE3_CORE0_LOOP_H
#define CORE3_CORE0_LOOP_H

void processInit(void);
void whileProcess(void);
void pitProcess0(void);

#endif /* CORE3_CORE0_LOOP_H */

```

core3_cyt4bb7/project/code/app/core1Loop.c

SHA-256

:

3d3322c762f89982a803fd84388de0fcdc418181f8f1d7456c09bc0b3a833eac

代码 (c)

```

#include "core1Loop.h"

#include "core3Runtime.h"
#include "paramSyncOwnerService.h"

void processInit(void)
{
    core3ParamSyncOwnerServiceInit();
    core3RuntimeInit();
}

```

```

}

void pitProcess0(void)
{
    core3RuntimeTick10msIsr();
}

void whileProcess(void)
{
    core3ParamSyncOwnerServicePoll();
    core3RuntimeProcess();
}

```

core3_cyt4bb7/project/code/app/core1Loop.h

SHA-256

:

6419ebbece36a10154849cd993adfba74158bff5696a3329d130203fb0f18665

代码 (c)

```

#ifndef CORE3_CORE1_LOOP_H
#define CORE3_CORE1_LOOP_H

void processInit(void);
void whileProcess(void);
void pitProcess0(void);

#endif /* CORE3_CORE1_LOOP_H */

```

core3_cyt4bb7/project/code/app/core3CameraService.c

SHA-256

:

4ee7a6b9f195b52364cc232e9e4160206f348b2ca9a8a7e6d31a8ffa9116b627

代码 (c)

```

#include "core3CameraService.h"

#include <string.h>

#include "cy_device_headers.h"
#include "configCore3Runtime.h"
#include "zf_device_mt9v03x.h"

_Static_assert(MT9V03X_W == CORE3_CAMERA_WIDTH, "Core3 camera width must remain 188");
_Static_assert(MT9V03X_H == CORE3_CAMERA_HEIGHT, "Core3 camera height must remain 120");
_Static_assert(MT9V03X_FPS_DEF == 50, "Core3 camera capture must remain 50fps");

#define CORE3_CAMERA_INT_VALID_MASK (1u << 31)
#define CORE3_CAMERA_CYT2_EQUIVALENT_EXPOSURE (500u)

static volatile uint32_t *cameraSystemIntControl(void)
{
    return &CPUSS->unCM7_1_SYSTEM_INT_CTL[
        tcpwm_0_interrupts_59_IRQn].u32Register;
}

static uint32_t cameraInterruptMask(void)
{
    volatile uint32_t *control = cameraSystemIntControl();
    uint32_t saved = *control;
    *control = saved & ~CORE3_CAMERA_INT_VALID_MASK;
    (void) *control;
    __DSB();
    return saved;
}

static void cameraInterruptRestore(uint32_t saved)
{
    volatile uint32_t *control = cameraSystemIntControl();

```

```

        *control = saved;
        (void) *control;
        __DSB();
    }

static uint8_t g_appImage[CORE3_CAMERA_HEIGHT][CORE3_CAMERA_WIDTH];
static uint32_t g_appFrameSeq;
static uint8_t g_initialized;

uint8_t core3CameraServiceInit(void)
{
    uint8_t result = mt9v03x_init();
    uint32_t savedIntControl;

    if ((result == 0u) &&
        (mt9v03x_set_exposure_time(
            CORE3_CAMERA_CYT2_EQUIVALENT_EXPOSURE) != 0u))
    {
        result = 1u;
    }

    savedIntControl = cameraInterruptMask();
    mt9v03x_finish_flag = 0u;
    cameraInterruptRestore(savedIntControl);
    g_initialized = (uint8_t) (result == 0u);

    return result;
}

uint8_t core3CameraServiceSetExposure(uint16_t exposure)
{
    if (g_initialized == 0u)
    {
        return 1u;
    }
    return mt9v03x_set_exposure_time(exposure);
}

uint8_t core3CameraServiceTryCapture(uint32_t tickMs,
    uint8_t timingValid,
    core3CameraSnapshot *snapshot,
    uint32_t *copyCycles)
{
    uint8_t captured = 0u;
    uint32_t startCycles = 0u;
    uint32_t endCycles = 0u;
    uint32_t savedIntControl;

    if ((snapshot == NULL) || (copyCycles == NULL) || (mt9v03x_finish_flag == 0u))
    {
        return 0u;
    }

    savedIntControl = cameraInterruptMask();

    if (timingValid != 0u)
    {
        startCycles = DWT->CYCCNT;
    }

    if (mt9v03x_finish_flag != 0u)
    {
        mt9v03x_finish_flag = 0u;
        memcpy(g_appImage[0], mt9v03x_image[0], MT9V03X_IMAGE_SIZE);
        captured = 1u;
    }

    if (timingValid != 0u)
    {
        endCycles = DWT->CYCCNT;
    }

    cameraInterruptRestore(savedIntControl);

    if (captured != 0u)
    {
        g_appFrameSeq++;
        if (g_appFrameSeq == 0u) g_appFrameSeq = 1u;
        snapshot->data = g_appImage[0];
    }
}

```

```

        snapshot->width = CORE3_CAMERA_WIDTH;
        snapshot->height = CORE3_CAMERA_HEIGHT;
        snapshot->appFrameSeq = g_appFrameSeq;
        snapshot->tickMs = tickMs;
        *copyCycles = (timingValid != 0u) ? (endCycles - startCycles) : 0u;
    }

    return captured;
}

```

core3_cyt4bb7/project/code/app/core3CameraService.h

SHA-256

:

fd307b59658483ca7331d78cfc159b9e2784103182901ea797b1078dd98982ed

代码 (c)

```

#ifndef CORE3_CAMERA_SERVICE_H
#define CORE3_CAMERA_SERVICE_H

#include <stdint.h>

typedef struct
{
    const uint8_t *data;
    uint16_t width;
    uint16_t height;
    uint32_t appFrameSeq;
    uint32_t tickMs;
} core3CameraSnapshot;

uint8_t core3CameraServiceInit(void);
uint8_t core3CameraServiceSetExposure(uint16_t exposure);
uint8_t core3CameraServiceTryCapture(uint32_t tickMs,
    uint8_t timingValid,
    core3CameraSnapshot *snapshot,
    uint32_t *copyCycles);

#endif /* CORE3_CAMERA_SERVICE_H */

```

core3_cyt4bb7/project/code/app/core3Diagnostics.c

SHA-256

:

df31963e92be43bfd93f288ba934611d2943888001d9e6d9de6369126f2fd8ed

代码 (c)

```

#include "core3Diagnostics.h"

#include <stdarg.h>
#include <stdio.h>
#include <string.h>
#include <math.h>

#include "cy_device_headers.h"
#include "configCore3Runtime.h"
#include "pinMap.h"
#include "observerLinkProtocol.h"
#include "core3VisionParams.h"
#include "zf_common_font.h"
#include "zf_device_ips114.h"
#include "zf_driver_pwm.h"

static uint8_t g_timingValid;
static uint8_t g_rgbReady;
static uint32_t g_lastRedDuty = 0xFFFFFFFFu;
static uint32_t g_lastGreenDuty = 0xFFFFFFFFu;
static uint32_t g_lastBlueDuty = 0xFFFFFFFFu;
#define CORE3_HOME_SLOT_COUNT (15u)
#define CORE3_HOME_SLOT_CHARS (40u)

```

```

#define CORE3_MENU_LINE_COUNT (8u)
#define CORE3_MENU_LINE_CHARS (39u)
#define CORE3_MENU_NAME_X (16u)
#define CORE3_MENU_VALUE_X (112u)
#define CORE3_MENU_SCROLL_X (232u)
#define CORE3_THRESHOLD_IMAGE_Y (18u)
#define CORE3_THRESHOLD_IMAGE_W (110u)
#define CORE3_THRESHOLD_IMAGE_H (70u)
#define CORE3_THRESHOLD_LEFT_X (2u)
#define CORE3_THRESHOLD_RIGHT_X (124u)
#define CORE3_THRESHOLD_TEXT_Y (112u)
#define CORE3_SOLID_FILL_PIXELS (240u)
#define CORE3_THRESHOLD_SLOT_COUNT (3u)
#define CORE3_THRESHOLD_SLOT_LOW_X (2u)
#define CORE3_THRESHOLD_SLOT_HIGH_X (82u)
#define CORE3_THRESHOLD_SLOT_EXPOSURE_X (162u)
#define CORE3_THRESHOLD_SLOT_LOW_WIDTH (72u)
#define CORE3_THRESHOLD_SLOT_HIGH_WIDTH (72u)
#define CORE3_THRESHOLD_SLOT_EXPOSURE_WIDTH (76u)
#define CORE3_THRESHOLD_NAME_LOW_X (10u)
#define CORE3_THRESHOLD_NAME_HIGH_X (90u)
#define CORE3_THRESHOLD_NAME_EXPOSURE_X (170u)
#define CORE3_THRESHOLD_VALUE_LOW_X (42u)
#define CORE3_THRESHOLD_VALUE_HIGH_X (122u)
#define CORE3_THRESHOLD_VALUE_EXPOSURE_X (202u)

#define CORE3_MENU_NORM_NAME_FG (RGB565_GREEN)
#define CORE3_MENU_NORM_VAL_FG (RGB565_WHITE)
#define CORE3_MENU_SEL_BG (RGB565_CYAN)
#define CORE3_MENU_SEL_NAME_FG (RGB565_BLACK)
#define CORE3_MENU_SEL_VAL_FG (RGB565_BLUE)
#define CORE3_MENU_EDIT_BG (RGB565_YELLOW)
#define CORE3_MENU_EDIT_NAME_FG (RGB565_BLACK)
#define CORE3_MENU_EDIT_VAL_FG (RGB565_RED)

typedef enum
{
    CORE3_DISPLAY_SURFACE_UNKNOWN = 0,
    CORE3_DISPLAY_SURFACE_HOME,
    CORE3_DISPLAY_SURFACE_WAIT,
    CORE3_DISPLAY_SURFACE_MENU,
    CORE3_DISPLAY_SURFACE_FEEDBACK,
    CORE3_DISPLAY_SURFACE_INIT_FAULT
} core3DisplaySurface;

typedef enum
{
    CORE3_MENU_ROW_NORMAL = 0,
    CORE3_MENU_ROW_SELECTED,
    CORE3_MENU_ROW_EDIT
} core3MenuRowStyle;

static char g_homeSlotCache[CORE3_HOME_SLOT_COUNT][CORE3_HOME_SLOT_CHARS + 1u];
static uint8_t g_homeSlotValid[CORE3_HOME_SLOT_COUNT];
static uint16_t g_homeSlotColor[CORE3_HOME_SLOT_COUNT];
static uint16_t g_homeSlotBackground[CORE3_HOME_SLOT_COUNT];
static char g_menuLineCache[CORE3_MENU_LINE_COUNT][CORE3_MENU_LINE_CHARS + 1u];
static char g_menuValueCache[CORE3_MENU_LINE_COUNT][CORE3_MENU_VALUE_CHARS + 1u];
static uint8_t g_menuLineValid[CORE3_MENU_LINE_COUNT];
static core3MenuRowStyle g_menuLineStyle[CORE3_MENU_LINE_COUNT];
static uint16_t g_menuFillRightX = 238u;
static uint8_t g_scrollValid;
static uint8_t g_scrollTotal;
static uint8_t g_scrollTop;
static uint16_t g_menuFrameColor = 0xFFFFFu;
static core3DisplaySurface g_displaySurface;
static core3MenuState g_lastMenuState;
static uint32_t g_lastMenuRevision;
static uint16_t g_homeFrameColor;
static uint16_t g_homeTitleColor;
static uint32_t g_thresholdOriginalFrameSeq = 0xFFFFFFFFFu;
static uint32_t g_thresholdPreviewFrameSeq = 0xFFFFFFFFFu;
static uint16_t g_thresholdFrameColor = 0xFFFFFu;
static uint8_t g_thresholdMode = 0xFFu;
static uint8_t g_thresholdOriginalValid;
static uint8_t g_thresholdPreviewValid;
static uint16_t g_thresholdSlotValue[CORE3_THRESHOLD_SLOT_COUNT];
static uint8_t g_thresholdSlotSelected[CORE3_THRESHOLD_SLOT_COUNT];
static uint8_t g_thresholdSlotValid[CORE3_THRESHOLD_SLOT_COUNT];

```

```

static uint8_t g_lastRgbMode = 0xFFu;
static uint32_t g_modeFlashStartMs;
static uint16_t g_solidFillBuffer[CORE3_SOLID_FILL_PIXELS];

static void fillSolidRect(uint16_t left, uint16_t top,
    uint16_t width, uint16_t height, uint16_t color)
{
    uint16_t rowsPerTransfer;
    uint16_t rows;
    uint16_t y = top;
    uint16_t i;

    if ((width == 0u) || (height == 0u) ||
        (width > CORE3_SOLID_FILL_PIXELS)) return;
    rowsPerTransfer = (uint16_t) (CORE3_SOLID_FILL_PIXELS / width);
    for (i = 0u; i < CORE3_SOLID_FILL_PIXELS; i++)
        g_solidFillBuffer[i] = color;
    while (height > 0u)
    {
        rows = (height < rowsPerTransfer) ? height : rowsPerTransfer;
        ips114_show_rgb565_image(left, y, g_solidFillBuffer,
            width, rows, width, rows, 0u);
        y = (uint16_t) (y + rows);
        height = (uint16_t) (height - rows);
    }
}

static const char *core3DiagnosticsStateShortName(core3RuntimeState state)
{
    switch (state)
    {
        case CORE3_RUNTIME_BOOT:
            return "BOOT";
        case CORE3_RUNTIME_CAMERA_STARTING:
            return "START";
        case CORE3_RUNTIME_RUNNING:
            return "RUN";
        case CORE3_RUNTIME_CAMERA_FAULT:
            return "FAULT";
        default:
            return "UNKNOWN";
    }
}

static const char *core3DiagnosticsReasonName(uint8_t reason)
{
    switch (reason)
    {
        case OBSERVER_LINK_REASON_NONE: return "READY";
        case OBSERVER_LINK_REASON_LOCAL_INIT: return "LOCAL INIT";
        case OBSERVER_LINK_REASON_CORE0_LINK: return "C0 LINK";
        case OBSERVER_LINK_REASON_CORE1_LINK: return "C1 LINK";
        case OBSERVER_LINK_REASON_CORE2_LINK: return "C2 LINK";
        case OBSERVER_LINK_REASON_OBSERVER_LINK: return "OBS LINK";
        case OBSERVER_LINK_REASON_PROTOCOL: return "PROTOCOL";
        case OBSERVER_LINK_REASON_CONFIG: return "CONFIG";
        case OBSERVER_LINK_REASON_CAMERA: return "CAMERA";
        case OBSERVER_LINK_REASON_TARGET: return "NO TARGET";
        case OBSERVER_LINK_REASON_BARRIER: return "BARRIER";
        case OBSERVER_LINK_REASON_DISPATCH: return "DISPATCH";
        case OBSERVER_LINK_REASON_SAFETY: return "SAFETY";
        case OBSERVER_LINK_REASON_LANDING: return "LANDING";
        default: return "UNKNOWN";
    }
}

static const char *core3DiagnosticsControlName(uint8_t state)
{
    return (state == OBSERVER_LINK_CAMERA_CONTROL_SAFE) ? "SAFE" : "RUN";
}

static const char *core3DiagnosticsSourceName(const core3RuntimeStatus *status)
{
    if (status->controlClass == OBSERVER_LINK_CONTROL_CLASS_SEARCH)
    {
        return "SEARCH";
    }
    if (status->activeSource == OBSERVER_LINK_SOURCE_CORE1)
    {

```



```

        return "CORE1";
    }
    if (status->activeSource == OBSERVER_LINK_SOURCE_CORE3)
    {
        return "CORE3";
    }
    if (status->activeSource == OBSERVER_LINK_SOURCE_CORE4)
    {
        return "CORE4";
    }
    return "NONE";
}

static const char *core3DiagnosticsNavName(uint8_t state)
{
    switch (state)
    {
        case 2u:
            return "TRACK";
        case 3u:
            return "START";
        case 4u:
            return "SEARCH";
        default:
            return "IDLE";
    }
}

static void core3DiagnosticsFormatQ4(int16_t value, char output[12])
{
    int32_t signedValue = value;
    uint32_t magnitude = (signedValue < 0) ?
        (uint32_t) (-signedValue) : (uint32_t) signedValue;
    uint32_t tenths = (magnitude * 10u + 8u) / 16u;

    (void) snprintf(output, 12u, "%c%lu.%lu",
        (signedValue < 0) ? '-' : '+',
        (unsigned long) (tenths / 10u),
        (unsigned long) (tenths % 10u));
}

static void formatFixedLine(char *line,
    uint8_t width,
    const char *format,
    va_list args)
{
    char formatted[64u];
    size_t length;

    (void) vsnprintf(formatted, sizeof(formatted), format, args);
    memset(line, ' ', width);
    line[width] = '\0';
    length = strlen(formatted);
    if (length > width)
    {
        length = width;
    }
    memcpy(line, formatted, length);
}

static void core3DiagnosticsShowHomeSlot(uint8_t slot,
    ips114_font_size_enum font,
    uint16_t x,
    uint16_t y,
    uint8_t width,
    uint16_t color,
    uint16_t background,
    const char *format,
    ...)
{
    char line[CORE3_HOME_SLOT_CHARS + 1u];
    va_list args;

    if ((slot >= CORE3_HOME_SLOT_COUNT) || (width > CORE3_HOME_SLOT_CHARS))
    {
        return;
    }
    memset(line, 0, sizeof(line));
    va_start(args, format);

```

```

formatFixedLine(line, width, format, args);
va_end(args);

if ((g_homeSlotValid[slot] == 0u) ||
    (g_homeSlotColor[slot] != color) ||
    (g_homeSlotBackground[slot] != background) ||
    (memcmp(g_homeSlotCache[slot], line, sizeof(line)) != 0))
{
    memcpy(g_homeSlotCache[slot], line, sizeof(line));
    g_homeSlotValid[slot] = 1u;
    g_homeSlotColor[slot] = color;
    g_homeSlotBackground[slot] = background;
    ips114_set_font(font);
    ips114_set_color(color, background);
    ips114_show_string(x, y, line);
}
}

static void fillMenuRow(uint8_t row, uint16_t color)
{
    uint16_t y;

    if (row >= CORE3_MENU_LINE_COUNT)
    {
        return;
    }
    y = (uint16_t)(row * 16u);
    if (row == 0u)
    {
        fillSolidRect(1u, 1u, 238u, 15u, color);
        return;
    }
    fillSolidRect(1u, y, g_menuFillRightX, 16u, color);
    if (row == (CORE3_MENU_LINE_COUNT - 1u))
    {
        fillSolidRect(1u, 128u, g_menuFillRightX, 6u, color);
    }
}

static void drawMenuFrame(uint16_t color)
{
    fillSolidRect(0u, 0u, 240u, 1u, color);
    fillSolidRect(0u, 134u, 240u, 1u, color);
    fillSolidRect(0u, 1u, 1u, 133u, color);
    fillSolidRect(239u, 1u, 1u, 133u, color);
}

static void drawMenuScrollbar(uint8_t total, uint8_t top)
{
    uint16_t trackTop = 16u;
    uint16_t trackBottom = 133u;
    uint16_t trackHeight;
    uint16_t thumbHeight;
    uint16_t thumbY;

    g_menuFillRightX = (total > CORE3_MENU_VISIBLE_ROWS)
        ? (CORE3_MENU_SCROLL_X - 1u)
        : 238u;
    if (total <= CORE3_MENU_VISIBLE_ROWS)
    {
        if (g_scrollValid != 0u)
        {
            {
                fillSolidRect(CORE3_MENU_SCROLL_X, trackTop,
                    (uint16_t)(238u - CORE3_MENU_SCROLL_X + 1u),
                    (uint16_t)(trackBottom - trackTop + 1u), RGB565_BLACK);
                memset(g_menuLineValid, 0, sizeof(g_menuLineValid));
            }
            g_scrollValid = 0u;
            return;
        }
    }
    if ((g_scrollValid != 0u) &&
        (g_scrollTotal == total) &&
        (g_scrollTop == top))
    {
        return;
    }

    fillSolidRect(CORE3_MENU_SCROLL_X, trackTop,
        (uint16_t)(238u - CORE3_MENU_SCROLL_X + 1u),

```

```

        (uint16_t) (trackBottom - trackTop + 1u), RGB565_BLACK);
fillSolidRect(CORE3_MENU_SCROLL_X, trackTop, 1u,
    (uint16_t) (trackBottom - trackTop + 1u), RGB565_GREEN);
fillSolidRect(238u, trackTop, 1u,
    (uint16_t) (trackBottom - trackTop + 1u), RGB565_GREEN);
trackHeight = (uint16_t) (trackBottom - trackTop + 1u);
thumbHeight = (uint16_t) (((uint32_t) trackHeight *
    CORE3_MENU_VISIBLE_ROWS) /
    total);
if (thumbHeight < 6u)
{
    thumbHeight = 6u;
}
thumbY = (uint16_t) (trackTop +
    (((uint32_t) (trackHeight - thumbHeight) * top) /
    (total - CORE3_MENU_VISIBLE_ROWS)));
if ((uint16_t) (thumbY + thumbHeight - 1u) > trackBottom)
    thumbHeight = (uint16_t) (trackBottom - thumbY + 1u);
fillSolidRect((uint16_t) (CORE3_MENU_SCROLL_X + 1u), thumbY,
    (uint16_t) (237u - (CORE3_MENU_SCROLL_X + 1u) + 1u),
    thumbHeight, RGB565_GREEN);
g_scrollTotal = total;
g_scrollTop = top;
g_scrollValid = 1u;
}

static void core3DiagnosticsShowMenuTitle(const char *title)
{
    char bounded[40];
    size_t length;
    uint16_t x;

    if (title == NULL)
    {
        title = "";
    }
    length = strlen(title);
    if (length > 39u) length = 39u;
    memcpy(bounded, title, length);
    bounded[length] = '\0';
    if ((g_menuLineValid[0] != 0u) &&
        (strcmp(g_menuLineCache[0], bounded) == 0)) return;
    fillMenuRow(0u, RGB565_GREEN);
    x = (uint16_t) ((240u - (uint16_t) (length * 6u)) / 2u);
    if (x < 2u) x = 2u;
    ips114_set_font(IPS114_6X8_FONT);
    ips114_set_color(RGB565_BLACK, RGB565_GREEN);
    ips114_show_string(x, 4u, bounded);
    ips114_set_font(IPS114_8X16_FONT);
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
    memset(g_menuLineCache[0], 0, sizeof(g_menuLineCache[0]));
    (void) strncpy(g_menuLineCache[0], bounded, CORE3_MENU_LINE_CHARS);
    g_menuLineValid[0] = 1u;
}

static void core3DiagnosticsShowMenuRow(uint8_t row,
    core3MenuRowStyle style,
    const char *name,
    const char *value)
{
    char signature[CORE3_MENU_LINE_CHARS + 1u];
    char fixedValue[CORE3_MENU_VALUE_CHARS + 1u];
    uint16_t background = RGB565_BLACK;
    uint16_t nameForeground = CORE3_MENU_NORM_NAME_FG;
    uint16_t valueForeground = CORE3_MENU_NORM_VAL_FG;
    char marker = ' ';
    uint8_t structureChanged;
    uint8_t valueChanged;
    size_t valueLength;

    if ((row == 0u) || (row >= CORE3_MENU_LINE_COUNT))
    {
        return;
    }
    if (name == NULL)
    {
        name = "";
    }
    if (value == NULL)

```

```

{
    value = "";
}
if (style == CORE3_MENU_ROW_SELECTED)
{
    background = CORE3_MENU_SEL_BG;
    nameForeground = CORE3_MENU_SEL_NAME_FG;
    valueForeground = CORE3_MENU_SEL_VAL_FG;
    marker = '>';
}
else if (style == CORE3_MENU_ROW_EDIT)
{
    background = CORE3_MENU_EDIT_BG;
    nameForeground = CORE3_MENU_EDIT_NAME_FG;
    valueForeground = CORE3_MENU_EDIT_VAL_FG;
    marker = "";
}

(void) snprintf(signature, sizeof(signature), "%c%.11s", marker, name);
memset(fixedValue, ' ', CORE3_MENU_VALUE_CHARS);
fixedValue[CORE3_MENU_VALUE_CHARS] = '\0';
valueLength = strlen(value);
if (valueLength > CORE3_MENU_VALUE_CHARS)
{
    valueLength = CORE3_MENU_VALUE_CHARS;
}
memcpy(fixedValue, value, valueLength);
structureChanged = (uint8_t) ((g_menuLineValid[row] == 0u) ||
    (g_menuLineStyle[row] != style) ||
    (strncmp(g_menuLineCache[row],
        signature,
        CORE3_MENU_LINE_CHARS) != 0));
valueChanged = (uint8_t) ((g_menuLineValid[row] == 0u) ||
    (strncmp(g_menuValueCache[row],
        fixedValue,
        CORE3_MENU_VALUE_CHARS) != 0));

if (structureChanged != 0u)
{
    fillMenuRow(row, background);
    ips114_set_font(IPS114_8X16_FONT);
    ips114_set_color(nameForeground, background);
    ips114_show_char(8u, (uint16_t) (row * 16u), marker);
    ips114_show_string(CORE3_MENU_NAME_X, (uint16_t) (row * 16u), name);
    memset(g_menuLineCache[row], 0, sizeof(g_menuLineCache[row]));
    (void) strncpy(g_menuLineCache[row], signature, CORE3_MENU_LINE_CHARS);
    g_menuLineStyle[row] = style;
}
if ((structureChanged != 0u) || (valueChanged != 0u))
{
    ips114_set_color(valueForeground, background);
    ips114_show_string(CORE3_MENU_VALUE_X,
        (uint16_t) (row * 16u),
        fixedValue);
    memcpy(g_menuValueCache[row], fixedValue, sizeof(fixedValue));
    g_menuLineValid[row] = 1u;
}
}

static uint16_t core3DiagnosticsFrameColor(uint8_t linkReady,
    uint8_t inputOwner)
{
    if ((linkReady == 0u) || (inputOwner == OBSERVER_LINK_KVM_TARGET_NONE))
    {
        return RGB565_YELLOW;
    }
    return (inputOwner == OBSERVER_LINK_KVM_TARGET_CORE3) ? RGB565_GREEN : RGB565_RED;
}

static uint16_t core3DiagnosticsOwnerFrameColor(const core3RuntimeStatus *status)
{
    if ((status == NULL) || (status->kvmLinkReady == 0u) ||
        (status->kvmInputOwner == OBSERVER_LINK_KVM_TARGET_NONE)) return RGB565_YELLOW;
    return (status->kvmInputOwner == OBSERVER_LINK_KVM_TARGET_CORE3)
        ? RGB565_GREEN : RGB565_RED;
}

static void drawHomeFrame(uint16_t color)
{

```

```

    if (g_homeFrameColor != color)
    {
        drawMenuFrame(color);
        fillSolidRect(133u, 16u, 1u, 89u, color);
        fillSolidRect(1u, 104u, 238u, 1u, color);
        fillSolidRect(1u, 119u, 238u, 1u, color);
        g_homeFrameColor = color;
    }
}

static void enterHomeSurface(uint16_t frameColor)
{
    if (g_displaySurface != CORE3_DISPLAY_SURFACE_HOME)
    {
        ips114_set_color(RGB565_GREEN, RGB565_BLACK);
        ips114_clear();
        memset(g_homeSlotValid, 0, sizeof(g_homeSlotValid));
        memset(g_homeSlotColor, 0, sizeof(g_homeSlotColor));
        memset(g_homeSlotBackground, 0, sizeof(g_homeSlotBackground));
        fillMenuRow(0u, RGB565_GREEN);
        g_homeFrameColor = 0xFFFFFu;
        g_homeTitleColor = 0xFFFFFu;
        g_displaySurface = CORE3_DISPLAY_SURFACE_HOME;
    }
    drawHomeFrame(frameColor);
}

static void enterMenuSurface(core3MenuState state)
{
    uint8_t oldThreshold = (uint8_t) (g_lastMenuState == CORE3_MENU_THRESHOLD_EDIT);
    uint8_t newThreshold = (uint8_t) (state == CORE3_MENU_THRESHOLD_EDIT);

    if ((g_displaySurface != CORE3_DISPLAY_SURFACE_MENU) ||
        (oldThreshold != newThreshold))
    {
        ips114_set_color(RGB565_GREEN, RGB565_BLACK);
        ips114_set_font(IPS114_8X16_FONT);
        ips114_clear();
        memset(g_menuLineValid, 0, sizeof(g_menuLineValid));
        g_scrollValid = 0u;
        g_menuFrameColor = 0xFFFFFu;
        g_thresholdOriginalFrameSeq = 0xFFFFFFFFFu;
        g_thresholdPreviewFrameSeq = 0xFFFFFFFFFu;
        g_thresholdFrameColor = 0xFFFFFu;
        g_thresholdMode = 0xFFu;
        g_thresholdOriginalValid = 0u;
        g_thresholdPreviewValid = 0u;
        memset(g_thresholdSlotValid, 0, sizeof(g_thresholdSlotValid));
        g_displaySurface = CORE3_DISPLAY_SURFACE_MENU;
    }
    g_lastMenuState = state;
}

static uint32_t core3DiagnosticsHardwareDuty(uint8_t percent)
{
    uint32_t logicalDuty = ((uint32_t) CORE3_RGB_LOGICAL_BRIGHTNESS * percent) / 100u;

    if (logicalDuty > PWM_DUTY_MAX)
    {
        logicalDuty = PWM_DUTY_MAX;
    }
    return PWM_DUTY_MAX - logicalDuty;
}

static void core3DiagnosticsSetRgb(uint8_t red, uint8_t green, uint8_t blue)
{
    uint32_t redDuty;
    uint32_t greenDuty;
    uint32_t blueDuty;

    if (g_rgbReady == 0u)
    {
        return;
    }
    redDuty = core3DiagnosticsHardwareDuty(red ? 100u : 0u);
    greenDuty = core3DiagnosticsHardwareDuty(green ? 100u : 0u);
    blueDuty = core3DiagnosticsHardwareDuty(blue ? 100u : 0u);

    if (redDuty != g_lastRedDuty)

```

```

    {
        pwm_set_duty(CORE3_RGB_LED_R_PWM, redDuty);
        g_lastRedDuty = redDuty;
    }
    if (greenDuty != g_lastGreenDuty)
    {
        pwm_set_duty(CORE3_RGB_LED_G_PWM, greenDuty);
        g_lastGreenDuty = greenDuty;
    }
    if (blueDuty != g_lastBlueDuty)
    {
        pwm_set_duty(CORE3_RGB_LED_B_PWM, blueDuty);
        g_lastBlueDuty = blueDuty;
    }
}

static void core3DiagnosticsSetRgbLevel(uint8_t red, uint8_t green, uint8_t blue,
    uint8_t level)
{
    uint32_t redDuty;
    uint32_t greenDuty;
    uint32_t blueDuty;
    if (g_rgbReady == 0u) return;
    redDuty = core3DiagnosticsHardwareDuty(red ? level : 0u);
    greenDuty = core3DiagnosticsHardwareDuty(green ? level : 0u);
    blueDuty = core3DiagnosticsHardwareDuty(blue ? level : 0u);
    if (redDuty != g_lastRedDuty) { pwm_set_duty(CORE3_RGB_LED_R_PWM, redDuty); g_lastRedDuty = redDuty; }
    if (greenDuty != g_lastGreenDuty) { pwm_set_duty(CORE3_RGB_LED_G_PWM, greenDuty); g_lastGreenDuty = greenDuty; }
    if (blueDuty != g_lastBlueDuty) { pwm_set_duty(CORE3_RGB_LED_B_PWM, blueDuty); g_lastBlueDuty = blueDuty; }
}

static uint8_t core3DiagnosticsBreatheLevel(uint32_t tickMs)
{
    uint32_t phase = tickMs % 1000u;
    uint32_t triangle = (phase < 500u) ? phase : (1000u - phase);
    return (uint8_t) (20u + (triangle * 80u) / 500u);
}

void core3DiagnosticsDisplayInitEarly(void)
{
    g_timingValid = 0u;
    g_rgbReady = 0u;
    g_lastRedDuty = 0xFFFFFFFFu;
    g_lastGreenDuty = 0xFFFFFFFFu;
    g_lastBlueDuty = 0xFFFFFFFFu;
    memset(g_homeSlotValid, 0, sizeof(g_homeSlotValid));
    memset(g_menuLineValid, 0, sizeof(g_menuLineValid));
    memset(g_thresholdSlotValid, 0, sizeof(g_thresholdSlotValid));
    g_displaySurface = CORE3_DISPLAY_SURFACE_UNKNOWN;
    g_lastMenuState = CORE3_MENU_WAIT_LINK;
    g_lastMenuRevision = 0u;
    g_homeFrameColor = 0xFFFFu;
    g_lastRgbMode = 0xFFu;
    g_modeFlashStartMs = 0u;

    ips114_set_dir(IPS114_PORTAIT);
    ips114_set_font(IPS114_8X16_FONT);
    ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
    ips114_init();
}

void core3DiagnosticsRuntimeInit(void)
{
    uint32_t before;

    g_timingValid = 0u;
    if ((SystemCoreClock != 0u) && ((DWT->CTRL & DWT_CTRL_NOCYCCNT_Msk) == 0u))
    {
        CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
        DWT->CYCCNT = 0u;
        DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
        __DSB();
        __ISB();

        before = DWT->CYCCNT;
        __NOP();
        __NOP();
        __NOP();
        __NOP();
    }
}

```

```

    if (((DWT->CTRL & DWT_CTRL_CYCCNTENA_Msk) != 0u) && (DWT->CYCCNT != before))
    {
        g_timingValid = 1u;
    }
}

pwm_init(CORE3_RGB_LED_R_PWM, CORE3_RGB_PWM_FREQUENCY_HZ, PWM_DUTY_MAX);
pwm_init(CORE3_RGB_LED_G_PWM, CORE3_RGB_PWM_FREQUENCY_HZ, PWM_DUTY_MAX);
pwm_init(CORE3_RGB_LED_B_PWM, CORE3_RGB_PWM_FREQUENCY_HZ, PWM_DUTY_MAX);
g_rgbReady = 1u;
}

uint8_t core3DiagnosticsTimingValid(void)
{
    return g_timingValid;
}

uint32_t core3DiagnosticsCycleNow(void)
{
    return (g_timingValid != 0u) ? DWT->CYCCNT : 0u;
}

uint32_t core3DiagnosticsCyclesToUs(uint32_t cycles)
{
    uint64_t scaled;

    if ((g_timingValid == 0u) || (SystemCoreClock == 0u))
    {
        return CORE3_TIMING_INVALID_US;
    }

    scaled = (uint64_t) cycles * 1000000u;
    return (uint32_t) (scaled / SystemCoreClock);
}

void core3DiagnosticsUpdateRgb(const core3RuntimeStatus *status, uint32_t tickMs)
{
    uint8_t blinkOn;

    if (status == NULL)
    {
        core3DiagnosticsSetRgb(0u, 0u, 0u);
        return;
    }

    switch (status->state)
    {
        case CORE3_RUNTIME_BOOT:
            blinkOn = (uint8_t) (((tickMs / CORE3_RGB_SLOW_HALF_PERIOD_MS) & 1u) == 0u);
            core3DiagnosticsSetRgb(blinkOn, blinkOn, blinkOn);
            break;
        case CORE3_RUNTIME_CAMERA_STARTING:
            blinkOn = (uint8_t) (((tickMs / CORE3_RGB_SLOW_HALF_PERIOD_MS) & 1u) == 0u);
            core3DiagnosticsSetRgb(0u, 0u, blinkOn);
            break;
        case CORE3_RUNTIME_RUNNING:
            if (g_lastRgbMode != status->cameraControlState)
            {
                g_lastRgbMode = status->cameraControlState;
                g_modeFlashStartMs = tickMs;
            }
            if ((status->lastCameraControlState == OBSERVER_LINK_CAMERA_CONTROL_RUN) &&
                (status->core1LinkReady == 0u))
            {
                blinkOn = (uint8_t) (((tickMs / CORE3_RGB_FAST_HALF_PERIOD_MS) & 1u) == 0u);
                core3DiagnosticsSetRgb(blinkOn, blinkOn, 0u);
            }
            else if (((status->controlFlags & OBSERVER_LINK_CONTROL_FLAG_BARRIER_REQUIRED) != 0u) ||
                (status->primaryReason == OBSERVER_LINK_REASON_CAMERA))
            {
                blinkOn = (uint8_t) (((tickMs / CORE3_RGB_FAST_HALF_PERIOD_MS) & 1u) == 0u);
                core3DiagnosticsSetRgb(blinkOn, blinkOn, 0u);
            }
            else if ((status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_REJOIN) ||
                (status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_LINK_WAIT) ||
                (status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_STANDALONE))
            {
                core3DiagnosticsSetRgbLevel(1u, 1u, 0u,
                    core3DiagnosticsBreatheLevel(tickMs));
            }
    }
}

```

```

    }
    else if (status->initFailedMask != 0u)
    {
        uint32_t phase = tickMs % 2000u;
        blinkOn = (uint8_t) ((phase < 100u) ||
            ((phase >= 250u) && (phase < 350u)));
        core3DiagnosticsSetRgb(blinkOn, 0u, 0u);
    }
    else if ((uint32_t) (tickMs - g_modeFlashStartMs) < 500u)
    {
        uint32_t phase = tickMs - g_modeFlashStartMs;
        blinkOn = (uint8_t) ((phase < 100u) ||
            ((phase >= 250u) && (phase < 350u)));
        if (status->cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_RUN)
            core3DiagnosticsSetRgb(0u, blinkOn, blinkOn);
        else core3DiagnosticsSetRgb(blinkOn, blinkOn, 0u);
    }
    else if (status->controlClass == OBSERVER_LINK_CONTROL_CLASS_SEARCH)
    {
        core3DiagnosticsSetRgb(1u, 0u, 1u);
    }
    else if (status->activeSource == OBSERVER_LINK_SOURCE_CORE3)
    {
        core3DiagnosticsSetRgb(0u, 0u, 1u);
    }
    else if (status->activeSource == OBSERVER_LINK_SOURCE_CORE1)
    {
        core3DiagnosticsSetRgb(0u, 1u, 0u);
    }
    else
    {
        if (status->cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_RUN)
            core3DiagnosticsSetRgb(0u, 1u, 1u);
        else core3DiagnosticsSetRgb(1u, 1u, 0u);
    }
    break;
case CORE3_RUNTIME_CAMERA_FAULT:
default:
    blinkOn = (uint8_t) (((tickMs / CORE3_RGB_FAST_HALF_PERIOD_MS) & 1u) == 0u);
    core3DiagnosticsSetRgb(blinkOn, 0u, 0u);
    break;
}
}

uint32_t core3DiagnosticsRenderInitFaultSummary(uint8_t failedMask,
uint8_t unverifiedMask,
uint8_t secondsLeft)
{
    static const char *const names[3] = {"FLASH", "CAMERA", "PIPELINE"};
    uint32_t startCycles = core3DiagnosticsCycleNow();
    uint8_t row = 0u;
    uint8_t i;
    if (g_displaySurface != CORE3_DISPLAY_SURFACE_INIT_FAULT)
    {
        uint16_t titleColor = (failedMask != 0u) ? RGB565_RED : RGB565_YELLOW;
        ips114_set_font(IPS114_8X16_FONT);
        ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
        ips114_clear();
        drawMenuFrame(titleColor);
        fillSolidRect(1u, 1u, 238u, 15u, titleColor);
        ips114_set_font(IPS114_6X8_FONT);
        ips114_set_color(RGB565_BLACK, titleColor);
        ips114_show_string(78u, 4u,
            (failedMask != 0u) ? "HW INIT FAIL" : "HW INIT CHECK");
        ips114_set_font(IPS114_8X16_FONT);
        for (i = 0u; i < 3u; i++)
        {
            uint8_t bit = (uint8_t) (1u << i);
            if ((failedMask & bit) != 0u)
            {
                ips114_set_color(RGB565_RED, RGB565_BLACK);
                ips114_show_string(16u, (uint16_t) (34u + row * 16u), names[i]);
                ips114_show_string(136u, (uint16_t) (34u + row * 16u), "FAIL");
                row++;
            }
        }
    }
    if ((unverifiedMask & CORE3_INIT_CHECK_DWT) != 0u)
    {
        ips114_set_color(RGB565_YELLOW, RGB565_BLACK);

```



```

        ips114_show_string(16u, (uint16_t) (34u + row * 16u), "DWT TIMER");
        ips114_show_string(136u, (uint16_t) (34u + row * 16u), "CHECK");
    }
    g_displaySurface = CORE3_DISPLAY_SURFACE_INIT_FAULT;
}
ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
ips114_show_string(16u, 114u, "CONTINUE 0s");
ips114_show_uint(88u, 114u, secondsLeft, 1u);
return core3DiagnosticsCyclesToUs(core3DiagnosticsCycleNow() - startCycles);
}

static uint16_t previewMapX(uint16_t imageX)
{
    return (uint16_t) (CORE3_DIAGNOSTICS_PREVIEW_X +
        ((uint32_t) imageX * (CORE3_DIAGNOSTICS_PREVIEW_WIDTH - 1u)) /
        (CORE3_CAMERA_WIDTH - 1u));
}

static uint16_t previewMapY(uint16_t imageY)
{
    return (uint16_t) (CORE3_DIAGNOSTICS_PREVIEW_Y +
        ((uint32_t) imageY * (CORE3_DIAGNOSTICS_PREVIEW_HEIGHT - 1u)) /
        (CORE3_CAMERA_HEIGHT - 1u));
}

static void previewCross(uint16_t imageX, uint16_t imageY,
    uint16_t radius, uint16_t color)
{
    uint16_t x = previewMapX(imageX);
    uint16_t y = previewMapY(imageY);
    uint16_t minX = CORE3_DIAGNOSTICS_PREVIEW_X;
    uint16_t maxX = (uint16_t) (minX + CORE3_DIAGNOSTICS_PREVIEW_WIDTH - 1u);
    uint16_t minY = CORE3_DIAGNOSTICS_PREVIEW_Y;
    uint16_t maxY = (uint16_t) (minY + CORE3_DIAGNOSTICS_PREVIEW_HEIGHT - 1u);
    uint16_t x0 = (x > minX + radius) ? (uint16_t) (x - radius) : minX;
    uint16_t x1 = (x + radius < maxX) ? (uint16_t) (x + radius) : maxX;
    uint16_t y0 = (y > minY + radius) ? (uint16_t) (y - radius) : minY;
    uint16_t y1 = (y + radius < maxY) ? (uint16_t) (y + radius) : maxY;
    fillSolidRect(x0, y, (uint16_t) (x1 - x0 + 1u), 1u, color);
    fillSolidRect(x, y0, 1u, (uint16_t) (y1 - y0 + 1u), color);
}

static void previewCircle(uint16_t imageX, uint16_t imageY,
    uint16_t imageRadius, uint16_t color)
{
    int16_t cx = (int16_t) previewMapX(imageX);
    int16_t cy = (int16_t) previewMapY(imageY);
    int16_t radius = (int16_t) (((uint32_t) imageRadius *
        CORE3_DIAGNOSTICS_PREVIEW_WIDTH) / CORE3_CAMERA_WIDTH);
    int16_t x = radius;
    int16_t y = 0;
    int16_t error = 1 - radius;
    int16_t minX = (int16_t) CORE3_DIAGNOSTICS_PREVIEW_X;
    int16_t maxX = (int16_t) (CORE3_DIAGNOSTICS_PREVIEW_X +
        CORE3_DIAGNOSTICS_PREVIEW_WIDTH - 1u);
    int16_t minY = (int16_t) CORE3_DIAGNOSTICS_PREVIEW_Y;
    int16_t maxY = (int16_t) (CORE3_DIAGNOSTICS_PREVIEW_Y +
        CORE3_DIAGNOSTICS_PREVIEW_HEIGHT - 1u);
    if (radius <= 0) return;
    while (x >= y)
    {
        int16_t px[8] = {cx + x, cx + y, cx - y, cx - x,
            cx - x, cx - y, cx + y, cx + x};
        int16_t py[8] = {cy + y, cy + x, cy + x, cy + y,
            cy - y, cy - x, cy - x, cy - y};
        uint8_t i;
        for (i = 0u; i < 8u; i++)
        {
            if ((px[i] >= minX) && (px[i] <= maxX) &&
                (py[i] >= minY) && (py[i] <= maxY))
                ips114_draw_point((uint16_t) px[i], (uint16_t) py[i], color);
        }
        y++;
        if (error < 0) error = (int16_t) (error + 2 * y + 1);
        else
        {
            x--;
            error = (int16_t) (error + 2 * (y - x) + 1);
        }
    }
}

```

```

    }
}

static void previewHeading(uint16_t imageX, uint16_t imageY,
    uint16_t headingQ8, uint16_t color)
{
    float heading = (float) headingQ8 / 256.0f;
    int16_t x0 = (int16_t) previewMapX(imageX);
    int16_t y0 = (int16_t) previewMapY(imageY);
    int16_t x1 = (int16_t) ((float) x0 + cosf(heading) * 9.0f);
    int16_t y1 = (int16_t) ((float) y0 + sinf(heading) * 9.0f);
    int16_t minX = (int16_t) CORE3_DIAGNOSTICS_PREVIEW_X;
    int16_t maxX = (int16_t) (CORE3_DIAGNOSTICS_PREVIEW_X +
        CORE3_DIAGNOSTICS_PREVIEW_WIDTH - 1u);
    int16_t minY = (int16_t) CORE3_DIAGNOSTICS_PREVIEW_Y;
    int16_t maxY = (int16_t) (CORE3_DIAGNOSTICS_PREVIEW_Y +
        CORE3_DIAGNOSTICS_PREVIEW_HEIGHT - 1u);
    if (x1 < minX) x1 = minX; else if (x1 > maxX) x1 = maxX;
    if (y1 < minY) y1 = minY; else if (y1 > maxY) y1 = maxY;
    ips114_draw_line((uint16_t) x0, (uint16_t) y0,
        (uint16_t) x1, (uint16_t) y1, color);
}

static char core3DiagnosticsControlClass(uint8_t controlClass)
{
    switch (controlClass)
    {
        case OBSERVER_LINK_CONTROL_CLASS_CORE1_TRACK: return 'L';
        case OBSERVER_LINK_CONTROL_CLASS_OBSERVER_TRACK: return 'R';
        case OBSERVER_LINK_CONTROL_CLASS_SEARCH: return 'S';
        default: return 'N';
    }
}

static void previewTrackBorder(const beaconTrackBorderMarginsStruct *margin, uint16_t color)
{
    uint16_t left;
    uint16_t right;
    uint16_t top;
    uint16_t bottom;

    if (margin == NULL) return;
    left = previewMapX(margin->left);
    right = previewMapX((uint16_t) (CORE3_CAMERA_WIDTH - 1u - margin->right));
    top = previewMapY(margin->top);
    bottom = previewMapY((uint16_t) (CORE3_CAMERA_HEIGHT - 1u - margin->bottom));
    if (margin->top != 0u) ips114_draw_line(left, top, right, top, color);
    if (margin->bottom != 0u) ips114_draw_line(left, bottom, right, bottom, color);
    if (margin->left != 0u) ips114_draw_line(left, top, left, bottom, color);
    if (margin->right != 0u) ips114_draw_line(right, top, right, bottom, color);
}

static void previewEdgePredictionBand(const beaconTrackEdgeBandsStruct *bands)
{
    uint16_t left = previewMapX(0u);
    uint16_t right = previewMapX(CORE3_CAMERA_WIDTH - 1u);
    uint16_t top = previewMapY(0u);
    uint16_t bottom = previewMapY(CORE3_CAMERA_HEIGHT - 1u);
    if (bands == NULL) return;
    if (bands->left != 0u)
        ips114_draw_line(previewMapX(bands->left), top, previewMapX(bands->left), bottom,
            RGB565_MAGENTA);
    if (bands->right != 0u)
        ips114_draw_line(previewMapX(CORE3_CAMERA_WIDTH - bands->right), top,
            previewMapX(CORE3_CAMERA_WIDTH - bands->right), bottom, RGB565_MAGENTA);
    if (bands->top != 0u)
        ips114_draw_line(left, previewMapY(bands->top), right, previewMapY(bands->top),
            RGB565_MAGENTA);
    if (bands->bottom != 0u)
        ips114_draw_line(left, previewMapY(CORE3_CAMERA_HEIGHT - bands->bottom), right,
            previewMapY(CORE3_CAMERA_HEIGHT - bands->bottom), RGB565_MAGENTA);
}

uint32_t core3DiagnosticsRenderPreview(const core3CameraSnapshot *snapshot,
    const core3RuntimeStatus *status,
    const core3VisionObservation *observation)
{
    uint32_t startCycles;
    uint16_t frameColor;

```

```

if ((snapshot == NULL) || (snapshot->data == NULL) || (status == NULL))
{
    return 0u;
}

startCycles = core3DiagnosticsCycleNow();
frameColor = core3DiagnosticsOwnerFrameColor(status);
enterHomeSurface(frameColor);
ips114_show_gray_image(CORE3_DIAGNOSTICS_PREVIEW_X,
    CORE3_DIAGNOSTICS_PREVIEW_Y,
    snapshot->data,
    snapshot->width,
    snapshot->height,
    CORE3_DIAGNOSTICS_PREVIEW_WIDTH,
    CORE3_DIAGNOSTICS_PREVIEW_HEIGHT,
    0u);
if ((observation != NULL) && (observation->valid != 0u))
{
    previewTrackBorder(&observation->border.exit, RGB565_YELLOW);
    previewTrackBorder(&observation->border.enter, RGB565_CYAN);
    previewEdgePredictionBand(&observation->edgeBands);
}
if ((observation != NULL) && (observation->valid != 0u))
{
    uint16_t i;
    for (i = 0u; i < observation->retainedCount; i++)
    {
        const core3VisionTrack *track = &observation->tracks[i];
        if ((track->flags & BEACON_CONFIRMED_FLAG_BORDER_REJECTED) != 0u)
            continue;
        uint8_t isSelected = (uint8_t)((status->viewOverlayValid != 0u) &&
            (status->activeSource == OBSERVER_LINK_SOURCE_CORE3) &&
            ((status->viewOverlay.flags & OBSERVER_LINK_OVERLAY_FLAG_SELECTED) != 0u) &&
            (status->viewOverlay.sourceFrameSeq == observation->trackerFrameSeq) &&
            (status->viewOverlay.selectedTrackId == track->trackId));
        if (isSelected == 0u)
        {
            uint16_t x = (track->fresh != 0u) ? track->centerX : track->predX;
            uint16_t y = (track->fresh != 0u) ? track->centerY : track->predY;
            previewCross(x, y, 3u, RGB565_YELLOW);
        }
    }
}
if (status->viewOverlayValid != 0u)
{
    const observerLinkViewOverlay *overlay = &status->viewOverlay;
    if ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_CENTER) != 0u)
        previewCross(overlay->activeCenterX, overlay->activeCenterY, 4u, RGB565_CYAN);
    if ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_DEADZONE) != 0u)
    {
        previewCircle(overlay->activeCenterX, overlay->activeCenterY,
            overlay->deadzoneEnterRadius, RGB565_YELLOW);
        previewCircle(overlay->activeCenterX, overlay->activeCenterY,
            overlay->deadzoneExitRadius, RGB565_MAGENTA);
    }
    if ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_SELECTED) != 0u)
        previewCross(overlay->selectedX, overlay->selectedY, 5u,
            (overlay->selectedFlags & OBSERVER_LINK_OVERLAY_SELECTED_COAST) ?
                RGB565_MAGENTA : RGB565_RED);
    if ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_POSE) != 0u)
    {
        uint16_t poseColor = (overlay->poseFlags & OBSERVER_LINK_OVERLAY_POSE_HOLD) ?
            RGB565_BLUE : RGB565_GREEN;
        previewCross(overlay->poseX, overlay->poseY, 5u, poseColor);
        if ((overlay->poseFlags & OBSERVER_LINK_OVERLAY_POSE_HEADING_VALID) != 0u)
        {
            previewHeading(overlay->poseX, overlay->poseY,
                overlay->poseHeadingQ8, poseColor);
        }
    }
}
return core3DiagnosticsCyclesToUs(core3DiagnosticsCycleNow() - startCycles);
}

uint32_t core3DiagnosticsRenderHud(const core3RuntimeStatus *status)
{
    uint32_t startCycles;
    const char *controlName;

```

```

const char *sourceName;
char errX[12];
char errY[12];
uint16_t frameColor;
uint16_t titleColor;
uint16_t coastCount;

if (status == NULL)
{
    return 0u;
}

startCycles = core3DiagnosticsCycleNow();
controlName = core3DiagnosticsControlName(status->cameraControlState);
sourceName = core3DiagnosticsSourceName(status);
core3DiagnosticsFormatQ4(status->errXQ4, errX);
core3DiagnosticsFormatQ4(status->errYQ4, errY);
frameColor = core3DiagnosticsOwnerFrameColor(status);
enterHomeSurface(frameColor);
titleColor = (status->state == CORE3_RUNTIME_CAMERA_FAULT) ? RGB565_RED :
    (((status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_HOME) ||
    (status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_RUNNING)) &&
    (status->primaryReason == OBSERVER_LINK_REASON_NONE)) ?
    RGB565_GREEN : RGB565_YELLOW;
if (g_homeTitleColor != titleColor)
{
    fillMenuRow(0u, titleColor);
    g_homeSlotValid[0] = 0u;
    g_homeTitleColor = titleColor;
    g_homeFrameColor = 0xFFFFu;
    drawHomeFrame(frameColor);
}
coastCount = (status->confirmedBeacons >= status->freshBeacons) ?
    (uint16_t) (status->confirmedBeacons - status->freshBeacons) : 0u;
core3DiagnosticsShowHomeSlot(0u, IPS114_6X8_FONT, 30u, 4u, 30u,
    RGB565_BLACK, titleColor, "CORE3 %s", controlName);
core3DiagnosticsShowHomeSlot(1u, IPS114_6X8_FONT, 135u, 17u, 17u,
    RGB565_WHITE, RGB565_BLACK, "CAM:%s FPS:%lu",
    core3DiagnosticsStateShortName(status->state), (unsigned long) status->appFps);
core3DiagnosticsShowHomeSlot(2u, IPS114_6X8_FONT, 135u, 25u, 17u,
    (status->frameValid != 0u) ? RGB565_WHITE : RGB565_RED, RGB565_BLACK,
    "AGE:%lums", (unsigned long) status->frameAgeMs);
core3DiagnosticsShowHomeSlot(3u, IPS114_6X8_FONT, 135u, 33u, 17u,
    RGB565_YELLOW, RGB565_BLACK, "L:%u F:%u C:%u",
    status->confirmedBeacons, status->freshBeacons,
    coastCount);
core3DiagnosticsShowHomeSlot(4u, IPS114_6X8_FONT, 135u, 41u, 17u,
    (status->cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_SAFE) ?
    RGB565_YELLOW : RGB565_CYAN, RGB565_BLACK,
    "VIS:%s", controlName);
core3DiagnosticsShowHomeSlot(5u, IPS114_6X8_FONT, 135u, 49u, 17u,
    (status->activeSource == OBSERVER_LINK_SOURCE_CORE3) ? RGB565_BLUE :
    ((status->activeSource == OBSERVER_LINK_SOURCE_CORE1) ? RGB565_GREEN :
    RGB565_MAGENTA), RGB565_BLACK,
    "SRC:%s", sourceName);
core3DiagnosticsShowHomeSlot(6u, IPS114_6X8_FONT, 135u, 57u, 17u,
    RGB565_WHITE, RGB565_BLACK, "NAV:%s CLS:%c",
    core3DiagnosticsNavName(status->navState),
    core3DiagnosticsControlClass(status->controlClass));
core3DiagnosticsShowHomeSlot(7u, IPS114_6X8_FONT, 135u, 65u, 17u,
    (status->controlFlags & OBSERVER_LINK_CONTROL_FLAG_BARRIER_REQUIRED) ?
    RGB565_YELLOW : RGB565_GREEN, RGB565_BLACK,
    "BAR:%c DSP:%c",
    (status->controlFlags & OBSERVER_LINK_CONTROL_FLAG_BARRIER_REQUIRED) ? 'Y' : 'N',
    (status->controlFlags & OBSERVER_LINK_CONTROL_FLAG_DISPATCH_ENABLED) ? 'Y' : 'N');
core3DiagnosticsShowHomeSlot(8u, IPS114_6X8_FONT, 135u, 73u, 17u,
    RGB565_WHITE, RGB565_BLACK, "EX:%s", errX);
core3DiagnosticsShowHomeSlot(9u, IPS114_6X8_FONT, 135u, 81u, 17u,
    RGB565_WHITE, RGB565_BLACK, "EY:%s", errY);
core3DiagnosticsShowHomeSlot(10u, IPS114_6X8_FONT, 135u, 89u, 17u,
    (status->core1LinkReady != 0u) ? RGB565_GREEN : RGB565_YELLOW,
    RGB565_BLACK, "1:%s 0:%s 2:%s",
    status->core1LinkOnline ? "ON" : "LOST",
    status->core0LinkUp ? "OK" : "LOST", status->core2LinkUp ? "OK" : "LOST");
core3DiagnosticsShowHomeSlot(12u, IPS114_6X8_FONT, 2u, 106u, 39u,
    (status->kvmInputOwner == OBSERVER_LINK_KVM_TARGET_NONE) ? RGB565_YELLOW :
    RGB565_WHITE, RGB565_BLACK,
    "KEY:%s C1:%s CFG:%s",
    (status->kvmInputOwner == OBSERVER_LINK_KVM_TARGET_CORE3) ? "C3" :

```

```

        ((status->kvmInputOwner == OBSERVER_LINK_KVM_TARGET_CORE1) ? "C1" :
        ((status->kvmInputOwner == OBSERVER_LINK_KVM_TARGET_CORE0) ? "C0" : "NONE")),
        status->core1LinkReady ? "OK" : "WAIT",
        status->cameraControlLocked ? "LOCK" : "OPEN");
core3DiagnosticsShowHomeSlot(13u, IPS114_6X8_FONT, 2u, 121u, 39u,
(status->primaryReason == OBSERVER_LINK_REASON_NONE) ? RGB565_GREEN :
    RGB565_YELLOW, RGB565_BLACK,
"STATE:%s %s",
(status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_REJOIN) ? "REJOIN" :
    ((status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_STANDALONE) ?
    "STANDALONE" : "HOME"),
core3DiagnosticsReasonName(status->primaryReason));

return core3DiagnosticsCyclesToUs(core3DiagnosticsCycleNow() - startCycles);
}

uint32_t core3DiagnosticsRenderMenu(const core3MenuView *view)
{
    uint32_t startCycles;
    uint8_t row;
    uint16_t frameColor;

    if (view == NULL)
    {
        return 0u;
    }
    startCycles = core3DiagnosticsCycleNow();
    frameColor = core3DiagnosticsFrameColor(view->linkReady,
        view->kvmInputOwner);

    if (view->state == CORE3_MENU_WAIT_LINK)
    {
        if (g_displaySurface != CORE3_DISPLAY_SURFACE_WAIT)
        {
            ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
            ips114_set_font(IPS114_8X16_FONT);
            ips114_clear();
            drawMenuFrame(RGB565_YELLOW);
            g_displaySurface = CORE3_DISPLAY_SURFACE_WAIT;
            g_lastMenuRevision = UINT32_MAX;
        }
        if (g_lastMenuRevision != view->screenRevision)
        {
            const char *lastMode = core3DiagnosticsControlName(
                view->lastCameraControlState);

            ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
            ips114_show_string(56u, 18u, "WAIT CORE1 LINK  ");
            ips114_show_string(40u, 50u, "LINK: NOT READY  ");
            ips114_show_string(40u, 66u, "KVM : DISABLED  ");
            ips114_show_string(40u, 82u, "LAST CTRL:      ");
            ips114_show_string(136u, 82u, "      ");
            ips114_show_string(136u, 82u, lastMode);
            ips114_show_string(40u, 106u, "RECONNECT -> HOME  ");
            g_lastMenuRevision = view->screenRevision;
        }
        return core3DiagnosticsCyclesToUs(core3DiagnosticsCycleNow() - startCycles);
    }

    if (view->lastAction != CORE3_MENU_ACTION_NONE)
    {
        if ((g_displaySurface != CORE3_DISPLAY_SURFACE_FEEDBACK) ||
            (g_lastMenuRevision != view->screenRevision))
        {
            uint16_t color = (view->lastAction ==
                CORE3_MENU_ACTION_RUNTIME_LOAD) ? RGB565_RED :
                ((view->lastActionOk != 0u) ? RGB565_GREEN : RGB565_RED);
            char slotText[16];

            ips114_full(color);
            if (view->lastAction == CORE3_MENU_ACTION_LIBRARY_SAVE ||
                view->lastAction == CORE3_MENU_ACTION_RUNTIME_LOAD)
            {
                ips114_set_color((color == RGB565_GREEN) ? RGB565_BLACK :
                    RGB565_WHITE, color);
                ips114_show_string(80u, 42u,
                    (view->lastAction == CORE3_MENU_ACTION_LIBRARY_SAVE) ?
                        "SAVE OK" : "LOAD OK");
                (void) snprintf(slotText, sizeof(slotText), "SLOT %u",

```

```

        (unsigned int) view->lastActionSlot);
    ips114_show_string(88u, 66u, slotText);
    ips114_set_color(GB565_GREEN, GB565_BLACK);
}
g_displaySurface = CORE3_DISPLAY_SURFACE_FEEDBACK;
g_lastMenuRevision = view->screenRevision;
}
return core3DiagnosticsCyclesToUs(core3DiagnosticsCycleNow() - startCycles);
}
if (view->state == CORE3_MENU_HOME)
{
    enterHomeSurface(frameColor);
    g_lastMenuRevision = view->screenRevision;
    return core3DiagnosticsCyclesToUs(core3DiagnosticsCycleNow() - startCycles);
}
if ((g_displaySurface == CORE3_DISPLAY_SURFACE_MENU) &&
    (g_lastMenuState == view->state) &&
    (g_lastMenuRevision == view->screenRevision))
{
    return core3DiagnosticsCyclesToUs(core3DiagnosticsCycleNow() - startCycles);
}

enterMenuSurface(view->state);
if (view->state == CORE3_MENU_PAGE_LIST)
{
    uint8_t pageCount = core3MenuGetPageCount();

    drawMenuScrollbar(pageCount, view->pageScrollTop);
    core3DiagnosticsShowMenuTitle("PAGES");
    for (row = 0u; row < CORE3_MENU_VISIBLE_ROWS; row++)
    {
        uint8_t page = (uint8_t) (view->pageScrollTop + row);

        if (page < pageCount)
        {
            core3DiagnosticsShowMenuRow((uint8_t) (row + 1u),
                (page == (uint8_t) view->pageIndex)
                ? CORE3_MENU_ROW_SELECTED
                : CORE3_MENU_ROW_NORMAL,
                core3MenuGetPageName(page),
                "");
        }
        else
        {
            core3DiagnosticsShowMenuRow((uint8_t) (row + 1u),
                CORE3_MENU_ROW_NORMAL,
                "",
                "");
        }
    }
}
else if ((view->state == CORE3_MENU_OPTION_LIST) &&
    (view->pageIndex == 0))
{
    char value[CORE3_MENU_VALUE_CHARS + 1u];

    drawMenuScrollbar(1u, 0u);
    core3DiagnosticsShowMenuTitle("PARAM SLOTS");
    (void) snprintf(value, sizeof(value), "SLOT %u",
        (unsigned int) (view->selectedParamSlot + 1u));
    core3DiagnosticsShowMenuRow(1u, CORE3_MENU_ROW_SELECTED,
        "SELECT", value);
    switch (view->paramSlotUiState)
    {
        case PARAM_SYNC_SLOT_UI_CHECKING: (void) snprintf(value, sizeof(value), "CHECKING"); break;
        case PARAM_SYNC_SLOT_UI_FULL: (void) snprintf(value, sizeof(value), "FULL"); break;
        case PARAM_SYNC_SLOT_UI_EMPTY: (void) snprintf(value, sizeof(value), "EMPTY"); break;
        case PARAM_SYNC_SLOT_UI_PARTIAL: (void) snprintf(value, sizeof(value), "PARTIAL"); break;
        case PARAM_SYNC_SLOT_UI_INVALID: (void) snprintf(value, sizeof(value), "INVALID"); break;
        case PARAM_SYNC_SLOT_UI_UNAVAILABLE: (void) snprintf(value, sizeof(value), "UNAVAILABLE"); break;
        default: (void) snprintf(value, sizeof(value), "UNKNOWN"); break;
    }
    core3DiagnosticsShowMenuRow(2u, CORE3_MENU_ROW_NORMAL,
        "SLOT", value);
    if (view->paramGlobalUiState == CORE3_MENU_PARAM_GLOBAL_QUEUED)
    {
        (void) snprintf(value, sizeof(value), "QUEUED");
    }
    else if (view->paramGlobalUiState == CORE3_MENU_PARAM_GLOBAL_PROGRESS)

```

```

{
    if (view->paramGlobalTarget <= 4u)
        (void) snprintf(value, sizeof(value),
            (view->paramGlobalExternal != 0u) ?
                "BUSY C%u PH%u" : "RUN C%u PH%u",
                (unsigned int) view->paramGlobalTarget,
                (unsigned int) view->paramGlobalPhase);
    else
        (void) snprintf(value, sizeof(value),
            (view->paramGlobalExternal != 0u) ?
                "BUSY PH%u" : "RUN PH%u",
                (unsigned int) view->paramGlobalPhase);
}
else if (view->paramGlobalUiState == CORE3_MENU_PARAM_GLOBAL_SUCCESS)
{
    (void) snprintf(value, sizeof(value), "SUCCESS");
}
else if (view->paramGlobalUiState == CORE3_MENU_PARAM_GLOBAL_FAILED)
{
    (void) snprintf(value, sizeof(value), "FAIL C%u E%u",
        (unsigned int) view->paramGlobalTarget,
        (unsigned int) view->paramGlobalError);
}
else
{
    (void) snprintf(value, sizeof(value), "IDLE");
}
core3DiagnosticsShowMenuRow(3u, CORE3_MENU_ROW_NORMAL,
    "TASK", value);
core3DiagnosticsShowMenuRow(4u, CORE3_MENU_ROW_NORMAL,
    "K2/K3", "SELECT");
core3DiagnosticsShowMenuRow(5u, CORE3_MENU_ROW_NORMAL,
    "K1 LONG", "SAVE");
core3DiagnosticsShowMenuRow(6u, CORE3_MENU_ROW_NORMAL,
    "K4", "LOAD");
core3DiagnosticsShowMenuRow(7u, CORE3_MENU_ROW_NORMAL,
    "K5", "BACK");
}
else
{
    uint8_t page = (uint8_t) view->pageIndex;
    uint8_t optionCount = core3MenuGetOptionCount(page);

    drawMenuScrollbar(optionCount, view->optionScrollTop);
    core3DiagnosticsShowMenuTitle(core3MenuGetPageName(page));
    for (row = 0u; row < CORE3_MENU_VISIBLE_ROWS; row++)
    {
        uint8_t option = (uint8_t) (view->optionScrollTop + row);

        if (option < optionCount)
        {
            char value[CORE3_MENU_VALUE_CHARS + 1u];
            core3MenuRowStyle style = CORE3_MENU_ROW_NORMAL;

            core3MenuFormatValue(core3MenuGetOptionValue(page, option), value);
            if (option == (uint8_t) view->optionIndex)
            {
                style = (view->state == CORE3_MENU_EDIT)
                    ? CORE3_MENU_ROW_EDIT
                    : CORE3_MENU_ROW_SELECTED;
            }
            core3DiagnosticsShowMenuRow((uint8_t) (row + 1u),
                style,
                core3MenuGetOptionName(page, option),
                value);
        }
        else
        {
            core3DiagnosticsShowMenuRow((uint8_t) (row + 1u),
                CORE3_MENU_ROW_NORMAL,
                "",
                "");
        }
    }
}
}
if (frameColor != g_menuFrameColor)
{
    drawMenuFrame(frameColor);
    g_menuFrameColor = frameColor;
}

```

```

    }
    ips114_set_color(IPS114_GREEN, IPS114_BLACK);
    g_lastMenuRevision = view->screenRevision;
    return core3DiagnosticsCyclesToUs(core3DiagnosticsCycleNow() - startCycles);
}

static void core3DiagnosticsDrawThresholdSlot(core3MenuThresholdItem item,
    uint16_t value, uint8_t selected)
{
    uint16_t slotX;
    uint16_t slotWidth;
    uint16_t nameX;
    uint16_t valueX;
    uint8_t digits;
    const char *name;
    char valueText[5];
    uint16_t background;
    uint16_t nameForeground;
    uint16_t valueForeground;

    if ((uint8_t) item >= CORE3_THRESHOLD_SLOT_COUNT) return;
    if ((g_thresholdSlotValid[item] != 0u) &&
        (g_thresholdSlotValue[item] == value) &&
        (g_thresholdSlotSelected[item] == selected)) return;

    if (item == CORE3_MENU_THRESHOLD_LOW)
    {
        slotX = CORE3_THRESHOLD_SLOT_LOW_X;
        slotWidth = CORE3_THRESHOLD_SLOT_LOW_WIDTH;
        nameX = CORE3_THRESHOLD_NAME_LOW_X;
        valueX = CORE3_THRESHOLD_VALUE_LOW_X;
        digits = 3u;
        name = "Lo.";
    }
    else if (item == CORE3_MENU_THRESHOLD_HIGH)
    {
        slotX = CORE3_THRESHOLD_SLOT_HIGH_X;
        slotWidth = CORE3_THRESHOLD_SLOT_HIGH_WIDTH;
        nameX = CORE3_THRESHOLD_NAME_HIGH_X;
        valueX = CORE3_THRESHOLD_VALUE_HIGH_X;
        digits = 3u;
        name = "Hi.";
    }
    else
    {
        slotX = CORE3_THRESHOLD_SLOT_EXPOSURE_X;
        slotWidth = CORE3_THRESHOLD_SLOT_EXPOSURE_WIDTH;
        nameX = CORE3_THRESHOLD_NAME_EXPOSURE_X;
        valueX = CORE3_THRESHOLD_VALUE_EXPOSURE_X;
        digits = 4u;
        name = "Ex.";
    }

    background = selected ? CORE3_MENU_SEL_BG : IPS114_BLACK;
    nameForeground = selected ? CORE3_MENU_SEL_NAME_FG : CORE3_MENU_NORM_NAME_FG;
    valueForeground = selected ? CORE3_MENU_SEL_VAL_FG : CORE3_MENU_NORM_VAL_FG;
    if (digits == 3u)
        (void) snprintf(valueText, sizeof(valueText), "%3u", (unsigned int) value);
    else
        (void) snprintf(valueText, sizeof(valueText), "%4u", (unsigned int) value);

    fillSolidRect(slotX, CORE3_THRESHOLD_TEXT_Y, slotWidth, 16u, background);
    ips114_set_font(IPS114_8X16_FONT);
    ips114_set_color(nameForeground, background);
    ips114_show_string(nameX, CORE3_THRESHOLD_TEXT_Y, name);
    ips114_set_color(valueForeground, background);
    ips114_show_string(valueX, CORE3_THRESHOLD_TEXT_Y, valueText);
    g_thresholdSlotValue[item] = value;
    g_thresholdSlotSelected[item] = selected;
    g_thresholdSlotValid[item] = 1u;
}

uint32_t core3DiagnosticsRenderThresholdEditor(const core3MenuView *view,
    const core3CameraSnapshot *snapshot,
    const uint8_t *detectPreview,
    uint8_t *updateImages)
{
    const core3VisionParamsStruct *params;
    uint32_t startCycles;

```



```

uint16_t low;
uint16_t high;
uint16_t frameColor;

if ((view == NULL) || (view->state != CORE3_MENU_THRESHOLD_EDIT))
{
    return 0u;
}
startCycles = core3DiagnosticsCycleNow();
frameColor = core3DiagnosticsFrameColor(view->linkReady,
    view->kvmInputOwner);
enterMenuSurface(CORE3_MENU_THRESHOLD_EDIT);
params = core3VisionParamsGetActive();
if (view->detectMode == 1u)
{
    low = params->legacyCannyLow;
    high = params->legacyCannyHigh;
    if (g_thresholdMode != view->detectMode)
        core3DiagnosticsShowMenuTitle("CANNY EDIT");
}
else
{
    low = params->binaryLow;
    high = params->binaryHigh;
    if (g_thresholdMode != view->detectMode)
        core3DiagnosticsShowMenuTitle("BINARY EDIT");
}
if ((updateImages != 0u) && (snapshot != NULL) && (snapshot->data != NULL))
{
    if ((g_thresholdOriginalValid == 0u) ||
        (snapshot->appFrameSeq != g_thresholdOriginalFrameSeq))
    {
        {
            ips114_show_gray_image(CORE3_THRESHOLD_LEFT_X,
                CORE3_THRESHOLD_IMAGE_Y,
                snapshot->data,
                snapshot->width,
                snapshot->height,
                CORE3_THRESHOLD_IMAGE_W,
                CORE3_THRESHOLD_IMAGE_H,
                0u);
            g_thresholdOriginalFrameSeq = snapshot->appFrameSeq;
            g_thresholdOriginalValid = 1u;
        }
    }
    else if ((updateImages != 0u) && (g_thresholdOriginalValid != 0u))
    {
        fillSolidRect(CORE3_THRESHOLD_LEFT_X, CORE3_THRESHOLD_IMAGE_Y,
            CORE3_THRESHOLD_IMAGE_W, CORE3_THRESHOLD_IMAGE_H, RGB565_BLACK);
        g_thresholdOriginalFrameSeq = 0xFFFFFFFFu;
        g_thresholdOriginalValid = 0u;
    }
}
if ((updateImages != 0u) && (detectPreview != NULL) && (snapshot != NULL))
{
    if ((g_thresholdPreviewValid == 0u) ||
        (snapshot->appFrameSeq != g_thresholdPreviewFrameSeq))
    {
        {
            ips114_show_gray_image(CORE3_THRESHOLD_RIGHT_X,
                CORE3_THRESHOLD_IMAGE_Y,
                detectPreview,
                188u,
                120u,
                CORE3_THRESHOLD_IMAGE_W,
                CORE3_THRESHOLD_IMAGE_H,
                0u);
            g_thresholdPreviewFrameSeq = snapshot->appFrameSeq;
            g_thresholdPreviewValid = 1u;
        }
    }
    else if ((updateImages != 0u) && (g_thresholdPreviewValid != 0u))
    {
        fillSolidRect(CORE3_THRESHOLD_RIGHT_X, CORE3_THRESHOLD_IMAGE_Y,
            CORE3_THRESHOLD_IMAGE_W, CORE3_THRESHOLD_IMAGE_H, RGB565_BLACK);
        g_thresholdPreviewFrameSeq = 0xFFFFFFFFu;
        g_thresholdPreviewValid = 0u;
    }
}
if (frameColor != g_thresholdFrameColor)
{
    fillSolidRect(120u, CORE3_THRESHOLD_IMAGE_Y, 1u,
        (uint16_t) (CORE3_THRESHOLD_IMAGE_H + 1u), frameColor);
}

```

```

        fillSolidRect(1u,
            (uint16_t) (CORE3_THRESHOLD_IMAGE_Y + CORE3_THRESHOLD_IMAGE_H + 2u),
            238u, 1u, frameColor);
        drawMenuFrame(frameColor);
        g_thresholdFrameColor = frameColor;
    }
    core3DiagnosticsDrawThresholdSlot(CORE3_MENU_THRESHOLD_LOW,
        low, (uint8_t) (view->thresholdItem == CORE3_MENU_THRESHOLD_LOW));
    core3DiagnosticsDrawThresholdSlot(CORE3_MENU_THRESHOLD_HIGH,
        high, (uint8_t) (view->thresholdItem == CORE3_MENU_THRESHOLD_HIGH));
    core3DiagnosticsDrawThresholdSlot(CORE3_MENU_THRESHOLD_EXPOSURE,
        params->exposure,
        (uint8_t) (view->thresholdItem == CORE3_MENU_THRESHOLD_EXPOSURE));
    g_thresholdMode = view->detectMode;
    g_lastMenuRevision = view->screenRevision;
    return core3DiagnosticsCyclesToUs(core3DiagnosticsCycleNow() - startCycles);
}

```

core3_cyt4bb7/project/code/app/core3Diagnostics.h

SHA-256

:

c1808e5fb87291eb5a518f8cb489d1a4558025e4349d97a9a3061e61d65e13dd

代码 (c)

```

#ifndef CORE3_DIAGNOSTICS_H
#define CORE3_DIAGNOSTICS_H

#include <stdint.h>

#include "core3CameraService.h"
#include "core3Menu.h"
#include "core3Runtime.h"
#include "core3VisionPipeline.h"

void core3DiagnosticsDisplayInitEarly(void);
void core3DiagnosticsRuntimeInit(void);
uint8_t core3DiagnosticsTimingValid(void);
uint32_t core3DiagnosticsCycleNow(void);
uint32_t core3DiagnosticsCyclesToUs(uint32_t cycles);
void core3DiagnosticsUpdateRgb(const core3RuntimeStatus *status, uint32_t tickMs);
uint32_t core3DiagnosticsRenderPreview(const core3CameraSnapshot *snapshot,
    const core3RuntimeStatus *status,
    const core3VisionObservation *observation);
uint32_t core3DiagnosticsRenderHud(const core3RuntimeStatus *status);
uint32_t core3DiagnosticsRenderInitFaultSummary(uint8_t failedMask,
    uint8_t unverifiedMask,
    uint8_t secondsLeft);
uint32_t core3DiagnosticsRenderMenu(const core3MenuView *view);
uint32_t core3DiagnosticsRenderThresholdEditor(const core3MenuView *view,
    const core3CameraSnapshot *snapshot,
    const uint8_t *detectPreview,
    uint8_t updateImages);

#endif /* CORE3_DIAGNOSTICS_H */

```

core3_cyt4bb7/project/code/app/core3Menu.c

SHA-256

:

d1041c9f6f21bbe03dffbd2182269989c9bcfb59a8f6bd8f3324bd425ee47691

代码 (c)

```

#include "core3Menu.h"

#include <stddef.h>
#include <string.h>

#include "configCore3Runtime.h"
#include "configCore3VisionParams.h"

```

```

#include "core3VisionParams.h"
#include "core3VisionParamsPersistence.h"
#include "core3LinkCore3.h"
#include "observerLinkProtocol.h"
#include "paramSyncOwnerService.h"

#define CORE3_MENU_OPTION_COUNT (5u)
#define CORE3_MENU_FEEDBACK_MS (300u)

typedef struct
{
    float *value;
} core3MenuOption;

static float g_workValues[CORE3_MENU_OPTION_COUNT];
static core3MenuOption g_options[CORE3_MENU_OPTION_COUNT] = {
    {&g_workValues[0]},
    {&g_workValues[1]},
    {&g_workValues[2]},
    {&g_workValues[3]},
    {&g_workValues[4]},
};

static core3MenuView g_view;
static uint32_t g_tickMs;
static uint32_t g_feedbackStartMs;
static uint32_t g_workValuesRevision;
static uint8_t g_lastValidDesiredTarget;
static uint8_t g_lastValidDesiredTargetPrimed;

static void markDirty(void)
{
    if (g_view.screenRevision < UINT32_MAX)
    {
        g_view.screenRevision++;
    }
}

static uint32_t floatToU32(float value)
{
    if (value <= 0.0f)
    {
        return 0u;
    }
    if (value >= (float) UINT32_MAX)
    {
        return UINT32_MAX;
    }
    return (uint32_t) (value + 0.5f);
}

static void syncFromActive(void)
{
    const core3VisionParamsStruct *params = core3VisionParamsGetActive();

    g_workValues[0] = (float) params->exposure;
    g_workValues[1] = (float) params->binaryLow;
    g_workValues[2] = (float) params->binaryHigh;
    g_workValues[3] = (float) params->legacyCannyLow;
    g_workValues[4] = (float) params->legacyCannyHigh;
    g_workValuesRevision = core3VisionParamsGetRevision();
}

static uint32_t g_paramGlobalRequestSequence;
static paramSyncGlobalState g_paramGlobalState;
static uint8_t g_paramGlobalStateValid;
static paramSyncGlobalRequest g_paramGlobalPendingRequest;
static uint8_t g_paramGlobalRequestPending;
static uint8_t g_paramSlotInspectPending;
static uint8_t g_externalParamTransactionActive;
static uint32_t g_lastTerminalCoordinatorBoot;
static uint32_t g_lastTerminalRequestBoot;
static uint32_t g_lastTerminalRequestSequence;

static void scheduleParamSlotInspect(void)
{
    g_paramSlotInspectPending = 1u;
    if (g_view.paramSlotUiState != PARAM_SYNC_SLOT_UI_CHECKING)
    {

```

```

        g_view.paramSlotUiState = PARAM_SYNC_SLOT_UI_CHECKING;
        markDirty();
    }
}

static void setParamGlobalUi(core3MenuParamGlobalUiState uiState,
    uint8_t phase, uint8_t target, uint8_t error)
{
    if ((g_view.paramGlobalUiState != uiState) ||
        (g_view.paramGlobalPhase != phase) ||
        (g_view.paramGlobalTarget != target) ||
        (g_view.paramGlobalError != error))
    {
        g_view.paramGlobalUiState = uiState;
        g_view.paramGlobalPhase = phase;
        g_view.paramGlobalTarget = target;
        g_view.paramGlobalError = error;
        markDirty();
    }
}

static void setParamGlobalContext(uint8_t action, uint8_t slot)
{
    if ((g_view.paramGlobalAction != action) ||
        (g_view.paramGlobalSlot != slot))
    {
        g_view.paramGlobalAction = action;
        g_view.paramGlobalSlot = slot;
        markDirty();
    }
}

static void setParamGlobalExternal(uint8_t external)
{
    if (g_view.paramGlobalExternal != external)
    {
        g_view.paramGlobalExternal = external;
        markDirty();
    }
}

static uint8_t observeNewTerminal(const paramSyncGlobalState *state)
{
    if (((state->flags & PARAM_SYNC_GLOBAL_FLAG_TERMINAL) == 0u) ||
        (state->action == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY))
        return 0u;
    if ((g_lastTerminalCoordinatorBoot == state->coordinatorBootGeneration) &&
        (g_lastTerminalRequestBoot == state->requestBootGeneration) &&
        (g_lastTerminalRequestSequence == state->requestSequence))
        return 0u;
    g_lastTerminalCoordinatorBoot = state->coordinatorBootGeneration;
    g_lastTerminalRequestBoot = state->requestBootGeneration;
    g_lastTerminalRequestSequence = state->requestSequence;
    return 1u;
}

static void invalidateGlobalParamRoute(void)
{
    g_paramGlobalStateValid = 0u;
    memset(&g_paramGlobalState, 0, sizeof(g_paramGlobalState));
    if (g_externalParamTransactionActive != 0u)
    {
        g_externalParamTransactionActive = 0u;
        setParamGlobalExternal(0u);
        setParamGlobalContext(PARAM_SYNC_GLOBAL_ACTION_NONE, 0u);
        setParamGlobalUi(CORE3_MENU_PARAM_GLOBAL_FAILED,
            PARAM_SYNC_GLOBAL_PHASE_FAILED, 3u,
            PARAM_SYNC_GLOBAL_ERROR_ROUTE);
    }
    if (g_paramGlobalRequestPending != 0u)
    {
        if (g_paramGlobalPendingRequest.action ==
            PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
            g_view.paramSlotUiState = PARAM_SYNC_SLOT_UI_UNAVAILABLE;
        setParamGlobalUi(CORE3_MENU_PARAM_GLOBAL_FAILED,
            PARAM_SYNC_GLOBAL_PHASE_FAILED, 3u,
            PARAM_SYNC_GLOBAL_ERROR_ROUTE);
        g_paramGlobalRequestPending = 0u;
    }
}

```

```

}

static void pollGlobalParamState(void)
{
    paramSyncGlobalState state;
    uint8_t matchesPending;

    if (core3LinkCore3TakeParamGlobalState(&state) == 0u)
    {
        invalidateGlobalParamRoute();
        return;
    }
    g_paramGlobalState = state;
    g_paramGlobalStateValid = 1u;
    matchesPending = (uint8_t) ((g_paramGlobalRequestPending != 0u) &&
        (paramSyncGlobalStateMatchesRequest(&state,
            &g_paramGlobalPendingRequest) != 0u));
    if (observeNewTerminal(&state) != 0u) scheduleParamSlotInspect();
    if (matchesPending == 0u)
    {
        if ((state.flags & PARAM_SYNC_GLOBAL_FLAG_BUSY) != 0u)
        {
            g_externalParamTransactionActive = 1u;
            setParamGlobalExternal(1u);
            setParamGlobalContext(state.action, state.slot);
            setParamGlobalUi(CORE3_MENU_PARAM_GLOBAL_PROGRESS, state.phase,
                state.target, state.error);
        }
        else if ((state.flags & PARAM_SYNC_GLOBAL_FLAG_TERMINAL) != 0u)
        {
            g_externalParamTransactionActive = 0u;
            setParamGlobalExternal(0u);
            setParamGlobalContext(PARAM_SYNC_GLOBAL_ACTION_NONE, 0u);
            setParamGlobalUi(CORE3_MENU_PARAM_GLOBAL_IDLE, state.phase,
                state.target, state.error);
        }
        else if ((g_externalParamTransactionActive != 0u) &&
            (paramSyncGlobalStateReady(&state) != 0u))
        {
            g_externalParamTransactionActive = 0u;
            setParamGlobalExternal(0u);
            setParamGlobalContext(PARAM_SYNC_GLOBAL_ACTION_NONE, 0u);
            setParamGlobalUi(CORE3_MENU_PARAM_GLOBAL_IDLE, state.phase,
                state.target, state.error);
        }
    }
    return;
}

if (state.phase == PARAM_SYNC_GLOBAL_PHASE_SUCCEEDED)
{
    setParamGlobalExternal(0u);
    g_view.paramGlobalAction = g_paramGlobalPendingRequest.action;
    g_view.paramGlobalSlot = g_paramGlobalPendingRequest.slot;
    if (g_paramGlobalPendingRequest.action ==
        PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
    {
        g_view.paramSlotUiState =
            paramSyncGlobalClassifySlotSummary(state.transactionId);
    }
    else
    {
        g_view.lastAction = (g_paramGlobalPendingRequest.action ==
            PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY) ?
            CORE3_MENU_ACTION_LIBRARY_SAVE : CORE3_MENU_ACTION_RUNTIME_LOAD;
        g_view.lastActionOk = 1u;
        g_view.lastActionSlot = g_paramGlobalPendingRequest.slot;
        g_feedbackStartMs = g_tickMs;
        g_paramSlotInspectPending = 1u;
    }
    setParamGlobalUi(CORE3_MENU_PARAM_GLOBAL_SUCCESS, state.phase,
        0xFFu, PARAM_SYNC_GLOBAL_ERROR_NONE);
    g_paramGlobalRequestPending = 0u;
    return;
}

if (state.phase == PARAM_SYNC_GLOBAL_PHASE_FAILED)
{
    setParamGlobalExternal(0u);
    g_view.paramGlobalAction = g_paramGlobalPendingRequest.action;
    g_view.paramGlobalSlot = g_paramGlobalPendingRequest.slot;

```

```

    if (g_paramGlobalPendingRequest.action ==
        PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
    {
        g_view.paramSlotUiState = PARAM_SYNC_SLOT_UI_UNAVAILABLE;
        setParamGlobalUi(CORE3_MENU_PARAM_GLOBAL_FAILED, state.phase,
            state.target, state.error);
        g_paramGlobalRequestPending = 0u;
        return;
    }
    if ((state.flags & PARAM_SYNC_GLOBAL_FLAG_BUSY) != 0u)
    {
        setParamGlobalUi(CORE3_MENU_PARAM_GLOBAL_PROGRESS, state.phase,
            state.target, state.error);
    }
}

static uint8_t requestGlobalParamAction(uint8_t action, uint8_t slot)
{
    paramSyncGlobalRequest request;
    uint8_t error;
    if (g_paramGlobalRequestPending != 0u)
        error = PARAM_SYNC_GLOBAL_ERROR_BUSY;
    else if (g_paramGlobalStateValid == 0u)
        error = PARAM_SYNC_GLOBAL_ERROR_ROUTE;
    else if (paramSyncGlobalStateReady(&g_paramGlobalState) == 0u)
    {
        if ((g_paramGlobalState.flags & PARAM_SYNC_GLOBAL_FLAG_RECOVERY) != 0u)
            error = PARAM_SYNC_GLOBAL_ERROR_RECOVERY;
        else if (g_paramGlobalState.readyMask != PARAM_SYNC_GLOBAL_TARGET_MASK)
            error = PARAM_SYNC_GLOBAL_ERROR_INCOMPLETE;
        else
            error = PARAM_SYNC_GLOBAL_ERROR_BUSY;
    }
    else
        error = PARAM_SYNC_GLOBAL_ERROR_NONE;
    if (error != PARAM_SYNC_GLOBAL_ERROR_NONE)
    {
        if (action == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
        {
            g_view.paramSlotUiState = PARAM_SYNC_SLOT_UI_UNAVAILABLE;
            setParamGlobalUi(CORE3_MENU_PARAM_GLOBAL_FAILED,
                PARAM_SYNC_GLOBAL_PHASE_FAILED, 3u, error);
            return 0u;
        }
        g_paramGlobalRequestSequence++;
        if (g_paramGlobalRequestSequence == 0u) g_paramGlobalRequestSequence = 1u;
        request.producerBootGeneration = core3LinkCore3GetLocalBootGeneration();
        request.sequence = g_paramGlobalRequestSequence;
        request.origin = 3u;
        request.action = action;
        request.slot = slot;
        request.reserved = 0u;
        if (core3LinkCore3QueueParamGlobalRequest(&request) == 0u)
        {
            if (action == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
            {
                g_view.paramSlotUiState = PARAM_SYNC_SLOT_UI_UNAVAILABLE;
                setParamGlobalUi(CORE3_MENU_PARAM_GLOBAL_FAILED,
                    PARAM_SYNC_GLOBAL_PHASE_FAILED, 3u,
                    PARAM_SYNC_GLOBAL_ERROR_ROUTE);
                return 0u;
            }
            g_paramGlobalPendingRequest = request;
            g_paramGlobalRequestPending = 1u;
            setParamGlobalExternal(0u);
            setParamGlobalUi(CORE3_MENU_PARAM_GLOBAL_QUEUED,
                PARAM_SYNC_GLOBAL_PHASE_REQUESTED, 0xFFu,
                PARAM_SYNC_GLOBAL_ERROR_NONE);
            g_view.paramGlobalAction = action;
            g_view.paramGlobalSlot = slot;
            if (action == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
                g_view.paramSlotUiState = PARAM_SYNC_SLOT_UI_CHECKING;
            return 1u;
        }
    }
}

static uint8_t isValidKvmTarget(uint8_t target)
{
    return (uint8_t) ((target == OBSERVER_LINK_KVM_TARGET_CORE0) ||
        (target == OBSERVER_LINK_KVM_TARGET_CORE1) ||
        (target == OBSERVER_LINK_KVM_TARGET_CORE3) ||
        (target == OBSERVER_LINK_KVM_TARGET_CORE4));
}

```

```

static void returnHomeForKvmSwitch(void)
{
    syncFromActive();
    g_view.state = CORE3_MENU_HOME;
    g_view.pageIndex = 0;
    g_view.optionIndex = 0;
    g_view.pageScrollTop = 0u;
    g_view.optionScrollTop = 0u;
    g_view.thresholdItem = CORE3_MENU_THRESHOLD_LOW;
    g_view.lastAction = CORE3_MENU_ACTION_NONE;
    g_view.lastActionOk = 0u;
    g_view.selectedParamSlot = 0u;
    g_feedbackStartMs = 0u;
}

static void applyWorkValues(void)
{
    core3VisionParamsStruct candidate = *core3VisionParamsGetActive();

    candidate.exposure = (uint16_t) floatToU32(g_workValues[0]);
    candidate.binaryLow = (uint16_t) floatToU32(g_workValues[1]);
    candidate.binaryHigh = (uint16_t) floatToU32(g_workValues[2]);
    candidate.legacyCannyLow = (uint16_t) floatToU32(g_workValues[3]);
    candidate.legacyCannyHigh = (uint16_t) floatToU32(g_workValues[4]);
    core3VisionParamsCommit(&candidate);
    syncFromActive();
}

static void enterHome(void)
{
    g_view.state = CORE3_MENU_HOME;
    markDirty();
}

static void saveSettings(void)
{
    g_view.lastAction = CORE3_MENU_ACTION_SAVE;
    g_view.lastActionOk = (uint8_t) (core3VisionParamsFlashSave(
        core3VisionParamsGetActive()) == CORE3_VISION_PARAMS_SAVE_VERIFIED);
    g_feedbackStartMs = g_tickMs;
    markDirty();
}

static uint8_t handleHome(core3MenuKey key, core3MenuPress press)
{
    if ((key == CORE3_MENU_KEY_3) && (press == CORE3_MENU_PRESS_SHORT))
    {
        g_view.state = CORE3_MENU_THRESHOLD_EDIT;
        g_view.thresholdItem = CORE3_MENU_THRESHOLD_LOW;
        markDirty();
        return 1u;
    }
    if ((key == CORE3_MENU_KEY_4) && (press == CORE3_MENU_PRESS_SHORT))
    {
        g_view.state = CORE3_MENU_PAGE_LIST;
        g_view.pageIndex = 0;
        g_view.pageScrollTop = 0u;
        markDirty();
        return 1u;
    }
    return 0u;
}

static uint8_t handlePageList(core3MenuKey key, core3MenuPress press)
{
    if (press != CORE3_MENU_PRESS_SHORT) return 0u;
    if (key == CORE3_MENU_KEY_1)
    {
        enterHome();
        return 1u;
    }
    if (key == CORE3_MENU_KEY_4)
    {
        g_view.state = CORE3_MENU_OPTION_LIST;
        g_view.optionIndex = 0;
        g_view.optionScrollTop = 0u;
        markDirty();
        scheduleParamSlotInspect();
    }
}

```

```

        return 1u;
    }
    return 0u;
}

static uint8_t handleSlotPage(core3MenuKey key, core3MenuPress press)
{
    if ((key == CORE3_MENU_KEY_5) && (press == CORE3_MENU_PRESS_SHORT))
    {
        enterHome();
        return 1u;
    }
    if (press == CORE3_MENU_PRESS_SHORT)
    {
        if ((key == CORE3_MENU_KEY_2) && (g_view.selectedParamSlot > 0u))
        {
            g_view.selectedParamSlot--;
            scheduleParamSlotInspect();
            return 1u;
        }
        if ((key == CORE3_MENU_KEY_3) &&
            (g_view.selectedParamSlot + 1u < CORE3_VISION_PARAMS_SLOT_COUNT))
        {
            g_view.selectedParamSlot++;
            scheduleParamSlotInspect();
            return 1u;
        }
        if (key == CORE3_MENU_KEY_4)
        {
            if (core3ParamSyncOwnerServiceMutationLocked() != 0u) return 1u;
            if (g_paramGlobalRequestPending == 0u &&
                g_paramSlotInspectPending == 0u &&
                g_view.paramGlobalAction ==
                    PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY &&
                g_view.paramGlobalSlot ==
                    (uint8_t)(g_view.selectedParamSlot + 1u) &&
                g_view.paramSlotUiState == PARAM_SYNC_SLOT_UI_FULL)
            {
                (void) requestGlobalParamAction(
                    PARAM_SYNC_GLOBAL_ACTION_LOAD_RUNTIME,
                    (uint8_t)(g_view.selectedParamSlot + 1u));
            }
            return 1u;
        }
    }
    if ((key == CORE3_MENU_KEY_1) && (press == CORE3_MENU_PRESS_LONG))
    {
        if (core3ParamSyncOwnerServiceMutationLocked() != 0u) return 1u;
        (void) requestGlobalParamAction(PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY,
            (uint8_t)(g_view.selectedParamSlot + 1u));
        return 1u;
    }
    return 0u;
}

static uint8_t handleThresholdEdit(core3MenuKey key, core3MenuPress press)
{
    uint8_t lowIndex = (g_view.detectMode == 1u) ? 3u : 1u;
    uint8_t highIndex = (g_view.detectMode == 1u) ? 4u : 2u;
    uint8_t valueIndex;
    float stepShort;
    float stepLong;
    if (core3ParamSyncOwnerServiceMutationLocked() != 0u)
    {
        enterHome();
        return 1u;
    }
    float delta;

    if ((key == CORE3_MENU_KEY_5) && (press == CORE3_MENU_PRESS_LONG))
    {
        saveSettings();
        enterHome();
        return 1u;
    }
    if (press != CORE3_MENU_PRESS_SHORT)
    {
        return 0u;
    }
}

```



```

if (key == CORE3_MENU_KEY_5)
{
    g_view.thresholdItem = (core3MenuThresholdItem)
        (((uint8_t) g_view.thresholdItem + 1u) % CORE3_MENU_THRESHOLD_COUNT);
    markDirty();
    return 1u;
}

if (g_view.thresholdItem == CORE3_MENU_THRESHOLD_EXPOSURE)
{
    valueIndex = 0u;
    stepShort = CORE3_VISION_EXPOSURE_STEP_S;
    stepLong = CORE3_VISION_EXPOSURE_STEP_L;
}
else
{
    valueIndex = (g_view.thresholdItem == CORE3_MENU_THRESHOLD_LOW)
        ? lowIndex
        : highIndex;
    stepShort = (g_view.detectMode == 1u)
        ? CORE3_VISION_LEGACY_CANNY_STEP_S
        : CORE3_VISION_BINARY_STEP_S;
    stepLong = (g_view.detectMode == 1u)
        ? CORE3_VISION_LEGACY_CANNY_STEP_L
        : CORE3_VISION_BINARY_STEP_L;
}

if (key == CORE3_MENU_KEY_1)
{
    delta = stepLong;
}
else if (key == CORE3_MENU_KEY_2)
{
    delta = stepShort;
}
else if (key == CORE3_MENU_KEY_3)
{
    delta = -stepShort;
}
else if (key == CORE3_MENU_KEY_4)
{
    delta = -stepLong;
}
else
{
    return 0u;
}
*g_options[valueIndex].value += delta;
applyWorkValues();
markDirty();
return 1u;
}

void core3MenuInit(void)
{
    memset(&g_view, 0, sizeof(g_view));
    g_view.state = CORE3_MENU_HOME;
    g_view.pageIndex = 0;
    g_view.optionIndex = 0;
    g_view.screenRevision = 1u;
    g_view.kvmDesiredTarget = OBSERVER_LINK_KVM_TARGET_CORE0;
    g_view.kvmInputOwner = OBSERVER_LINK_KVM_TARGET_NONE;
    g_tickMs = 0u;
    g_feedbackStartMs = 0u;
    g_lastValidDesiredTarget = OBSERVER_LINK_KVM_TARGET_CORE0;
    g_lastValidDesiredTargetPrimed = 1u;
    g_view.selectedParamSlot = 0u;
    g_view.paramGlobalTarget = 0xFFu;
    g_paramGlobalRequestSequence = 0u;
    g_paramGlobalStateValid = 0u;
    g_paramGlobalRequestPending = 0u;
    g_paramSlotInspectPending = 0u;
    g_externalParamTransactionActive = 0u;
    g_lastTerminalCoordinatorBoot = 0u;
    g_lastTerminalRequestBoot = 0u;
    g_lastTerminalRequestSequence = 0u;
    memset(&g_paramGlobalState, 0, sizeof(g_paramGlobalState));
    memset(&g_paramGlobalPendingRequest, 0, sizeof(g_paramGlobalPendingRequest));
    g_view.paramSlotUiState = PARAM_SYNC_SLOT_UI_UNKNOWN;
}

```

```

        syncFromActive();
    }

void core3MenuTick(uint32_t tickMs)
{
    g_tickMs = tickMs;
    pollGlobalParamState();
    if ((core3ParamSyncOwnerServiceMutationLocked() == 0u) &&
        (g_workValuesRevision != core3VisionParamsGetRevision()))
    {
        syncFromActive();
        markDirty();
    }
    if ((g_view.lastAction != CORE3_MENU_ACTION_NONE) &&
        ((uint32_t) (tickMs - g_feedbackStartMs) >= CORE3_MENU_FEEDBACK_MS))
    {
        g_view.lastAction = CORE3_MENU_ACTION_NONE;
        markDirty();
    }
    if ((g_paramSlotInspectPending != 0u) &&
        (g_view.lastAction == CORE3_MENU_ACTION_NONE) &&
        (g_paramGlobalRequestPending == 0u) &&
        (g_paramGlobalStateValid != 0u) &&
        (paramSyncGlobalStateReady(&g_paramGlobalState) != 0u) &&
        requestGlobalParamAction(PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY,
            (uint8_t) (g_view.selectedParamSlot + 1u)) != 0u)
    {
        g_paramSlotInspectPending = 0u;
    }
}

void core3MenuSetDetectMode(uint8_t detectMode)
{
    uint8_t normalized = (detectMode == 1u) ? 1u : 0u;

    if (g_view.detectMode != normalized)
    {
        g_view.detectMode = normalized;
        markDirty();
    }
}

void core3MenuSetLinkState(uint8_t linkReady, uint8_t lastCameraControlState)
{
    uint8_t ready = (linkReady != 0u) ? 1u : 0u;
    uint8_t controlState = (lastCameraControlState <= 2u) ?
        lastCameraControlState : 0u;
    uint8_t changed = (uint8_t) ((g_view.linkReady != ready) ||
        (g_view.lastCameraControlState != controlState));

    if (ready == 0u)
    {
        invalidateGlobalParamRoute();
        if ((g_view.linkReady != 0u) ||
            (g_view.state != CORE3_MENU_HOME) ||
            (g_view.lastAction != CORE3_MENU_ACTION_NONE))
        {
            changed = 1u;
            /* 断链中止编辑且不触发保存；固定回 HOME 继续单板视觉调试。 */
            syncFromActive();
            g_view.state = CORE3_MENU_HOME;
            g_view.thresholdItem = CORE3_MENU_THRESHOLD_LOW;
            g_view.lastAction = CORE3_MENU_ACTION_NONE;
            g_view.lastActionOk = 0u;
            g_feedbackStartMs = 0u;
        }
    }
    g_view.linkReady = ready;
    g_view.lastCameraControlState = controlState;
    if (changed != 0u)
    {
        markDirty();
    }
}

void core3MenuSetControlOwned(uint8_t controlOwned)
{
    uint8_t owned = ((g_view.linkReady != 0u) && (controlOwned != 0u)) ? 1u : 0u;

```

```

    if (g_view.controlOwned != owned)
    {
        g_view.controlOwned = owned;
        markDirty();
    }
}

void core3MenuSetKvmRoute(uint8_t desiredTarget, uint8_t inputOwner)
{
    if ((isValidKvmTarget(desiredTarget) != 0u) &&
        (g_lastValidDesiredTargetPrimed != 0u) &&
        (g_lastValidDesiredTarget == OBSERVER_LINK_KVM_TARGET_CORE3) &&
        (desiredTarget != OBSERVER_LINK_KVM_TARGET_CORE3))
    {
        returnHomeForKvmSwitch();
    }
    if (isValidKvmTarget(desiredTarget) != 0u)
    {
        g_lastValidDesiredTarget = desiredTarget;
        g_lastValidDesiredTargetPrimed = 1u;
    }
    if ((g_view.kvmDesiredTarget != desiredTarget) ||
        (g_view.kvmInputOwner != inputOwner))
    {
        g_view.kvmDesiredTarget = desiredTarget;
        g_view.kvmInputOwner = inputOwner;
        markDirty();
    }
}

uint8_t core3MenuSubmitKey(core3MenuKey key, core3MenuPress press)
{
    if ((key > CORE3_MENU_KEY_5) || (press > CORE3_MENU_PRESS_LONG))
    {
        return 0u;
    }
    if (g_view.lastAction != CORE3_MENU_ACTION_NONE)
    {
        /* 成功提示期间确认并吞掉已送达事件，禁止上游把旧按键重放。 */
        return 1u;
    }
    if (g_view.linkReady == 0u)
    {
        return 0u;
    }
    if ((g_lastValidDesiredTargetPrimed == 0u) ||
        (g_lastValidDesiredTarget != OBSERVER_LINK_KVM_TARGET_CORE3))
    {
        return 0u;
    }

    switch (g_view.state)
    {
    {
        case CORE3_MENU_HOME:
            return handleHome(key, press);
        case CORE3_MENU_PAGE_LIST:
            return handlePageList(key, press);
        case CORE3_MENU_OPTION_LIST:
            return handleSlotPage(key, press);
        case CORE3_MENU_EDIT:
            enterHome();
            return 0u;
        case CORE3_MENU_THRESHOLD_EDIT:
            return handleThresholdEdit(key, press);
        case CORE3_MENU_WAIT_LINK:
            return 0u;
        default:
            return 0u;
    }
    }
}

void core3MenuGetView(core3MenuView *view)
{
    if (view != NULL)
    {
        *view = g_view;
    }
}

```

```

uint8_t core3MenuGetPageCount(void)
{
    return 1u;
}

const char *core3MenuGetPageName(uint8_t pageIndex)
{
    return (pageIndex == 0u) ? "Param Slots" : "";
}

uint8_t core3MenuGetOptionCount(uint8_t pageIndex)
{
    return (pageIndex == 0u) ? 1u : 0u;
}

const char *core3MenuGetOptionName(uint8_t pageIndex, uint8_t optionIndex)
{
    return ((pageIndex == 0u) && (optionIndex == 0u)) ? "Slot" : "";
}

double core3MenuGetOptionValue(uint8_t pageIndex, uint8_t optionIndex)
{
    return ((pageIndex == 0u) && (optionIndex == 0u))
        ? (double) (g_view.selectedParamSlot + 1u) : 0.0;
}

void core3MenuFormatValue(double value,
    char output[CORE3_MENU_VALUE_CHARS + 1u])
{
    uint8_t position;
    uint8_t negative = 0u;
    uint32_t integerPart;

    if (output == NULL)
    {
        return;
    }
    memset(output, ' ', CORE3_MENU_VALUE_CHARS);
    output[CORE3_MENU_VALUE_CHARS] = '\0';

    if (value < 0.0)
    {
        negative = 1u;
        value = -value;
    }
    integerPart = (uint32_t) (value + 0.5);

    position = CORE3_MENU_VALUE_CHARS;
    do
    {
        output[--position] = (char) ('0' + (integerPart % 10u));
        integerPart /= 10u;
    } while ((integerPart != 0u) && (position > 0u));
    if ((negative != 0u) && (position > 0u))
    {
        output[--position] = '-';
    }
}

```

core3_cyt4bb7/project/code/app/core3Menu.h

SHA-256

:

0f42ae7c110878411c9e5f61b84d5f171736a393023a2cad6e9418ce221434af

代码 (c)

```

#ifndef CORE3_MENU_H
#define CORE3_MENU_H

#include <stdint.h>

#define CORE3_MENU_VISIBLE_ROWS (7u)
#define CORE3_MENU_VALUE_CHARS (15u)

typedef enum

```

```

{
    CORE3_MENU_HOME = 0,
    CORE3_MENU_PAGE_LIST,
    CORE3_MENU_OPTION_LIST,
    CORE3_MENU_EDIT,
    CORE3_MENU_THRESHOLD_EDIT,
    CORE3_MENU_WAIT_LINK
} core3MenuState;

typedef enum
{
    CORE3_MENU_THRESHOLD_LOW = 0,
    CORE3_MENU_THRESHOLD_HIGH,
    CORE3_MENU_THRESHOLD_EXPOSURE,
    CORE3_MENU_THRESHOLD_COUNT
} core3MenuThresholdItem;

typedef enum
{
    CORE3_MENU_KEY_1 = 0,
    CORE3_MENU_KEY_2,
    CORE3_MENU_KEY_3,
    CORE3_MENU_KEY_4,
    CORE3_MENU_KEY_5
} core3MenuKey;

typedef enum
{
    CORE3_MENU_PRESS_SHORT = 0,
    CORE3_MENU_PRESS_LONG
} core3MenuPress;

typedef enum
{
    CORE3_MENU_ACTION_NONE = 0,
    CORE3_MENU_ACTION_SAVE,
    CORE3_MENU_ACTION_RESTORE,
    CORE3_MENU_ACTION_LOAD,
    CORE3_MENU_ACTION_LIBRARY_SAVE,
    CORE3_MENU_ACTION_RUNTIME_LOAD
} core3MenuAction;

typedef enum
{
    CORE3_MENU_PARAM_GLOBAL_IDLE = 0,
    CORE3_MENU_PARAM_GLOBAL_QUEUED,
    CORE3_MENU_PARAM_GLOBAL_PROGRESS,
    CORE3_MENU_PARAM_GLOBAL_SUCCESS,
    CORE3_MENU_PARAM_GLOBAL_FAILED
} core3MenuParamGlobalUiState;

typedef struct
{
    core3MenuState state;
    int8_t pageIndex;
    int8_t optionIndex;
    uint8_t pageScrollTop;
    uint8_t optionScrollTop;
    uint8_t detectMode;
    core3MenuThresholdItem thresholdItem;
    uint32_t screenRevision;
    core3MenuAction lastAction;
    uint8_t lastActionOk;
    uint8_t lastActionSlot;
    uint8_t linkReady;
    uint8_t controlOwned;
    uint8_t lastCameraControlState;
    uint8_t kvmDesiredTarget;
    uint8_t kvmInputOwner;
    uint8_t selectedParamSlot;
    core3MenuParamGlobalUiState paramGlobalUiState;
    uint8_t paramGlobalPhase;
    uint8_t paramGlobalTarget;
    uint8_t paramGlobalError;
    uint8_t paramGlobalAction;
    uint8_t paramGlobalSlot;
    uint8_t paramSlotUiState;
    uint8_t paramGlobalExternal;
} core3MenuView;

```

```

void core3MenuInit(void);
void core3MenuTick(uint32_t tickMs);
void core3MenuSetDetectMode(uint8_t detectMode);
void core3MenuSetLinkState(uint8_t linkReady, uint8_t lastCameraControlState);
void core3MenuSetControlOwned(uint8_t controlOwned);
void core3MenuSetKvmRoute(uint8_t desiredTarget, uint8_t inputOwner);
uint8_t core3MenuSubmitKey(core3MenuKey key, core3MenuPress press);
void core3MenuGetView(core3MenuView *view);

uint8_t core3MenuGetPageCount(void);
const char *core3MenuGetPageName(uint8_t pageIndex);
uint8_t core3MenuGetOptionCount(uint8_t pageIndex);
const char *core3MenuGetOptionName(uint8_t pageIndex, uint8_t optionIndex);
double core3MenuGetOptionValue(uint8_t pageIndex, uint8_t optionIndex);

void core3MenuFormatValue(double value,
    char output[CORE3_MENU_VALUE_CHARS + 1u]);

#endif /* CORE3_MENU_H */

```

core3_cyt4bb7/project/code/app/core3Runtime.c

SHA-256

:

a4d1c7137e65d5206eb1b20409137564949212951544a97dee50a16561641d20

代码 (c)

```

#include "core3Runtime.h"

#include <string.h>

#include "configCore3Runtime.h"
#include "configBeaconTrack.h"
#include "core3CameraService.h"
#include "core3Diagnostics.h"
#include "core3Menu.h"
#include "core3LinkCore3.h"
#include "core3KvmCore3.h"
#include "core3VisionParams.h"
#include "core3VisionParamsPersistence.h"
#include "core3VisionPipeline.h"
#include "observerKvmBindingGuard.h"
#include "zf_driver_pit.h"

static volatile uint32_t g_tickMs;
static volatile uint8_t g_tickDue;

static core3RuntimeStatus g_status;
static core3VisionObservation g_latestObservation;
static core3CameraSnapshot g_latestSnapshot;
static uint8_t g_cameraInitialized;
static uint8_t g_recoveryFrameStreak;
static uint32_t g_lastInitAttemptTickMs;
static uint32_t g_lastFrameTickMs;
static uint32_t g_statsWindowStartTickMs;
static uint32_t g_statsWindowFrames;
static uint32_t g_lastHudTickMs;
static uint32_t g_lastPreviewTickMs;
static uint32_t g_lastMenuRenderTickMs;
static uint32_t g_lastRenderedMenuRevision;
static uint32_t g_lastCameraConfigAttemptTickMs;
static uint32_t g_lastCameraConfigAttemptRevision;
static uint32_t g_lastAppliedCameraConfigRevision;
static observerKvmBindingGuard g_kvmBindingGuard;
static uint32_t g_lastKvmLocalSession;
static uint32_t g_lastKvmMenuTickMs;
static uint32_t g_lastKvmMenuRevision;
static observerLinkKvmEvent g_pendingKvmEvent;
static observerLinkKvmAck g_pendingKvmAck;
static observerLinkKvmMenuStatus g_pendingKvmMenu;
static uint8_t g_pendingKvmEventValid;
static uint8_t g_pendingKvmAckValid;
static uint8_t g_pendingKvmMenuValid;
static uint32_t g_linkReadyStableStartMs;

```

```

static uint32_t g_rejoinBarrierUntilMs;
static uint8_t g_localStandalone;
static uint32_t g_initSummaryUntilMs;
static uint8_t g_initSummaryLastSecond;
static uint8_t g_menuConfirmedSwitchedAway;

#define CORE3_INITIAL_LINK_WAIT_MS (2000u)
#define CORE3_REJOIN_STABLE_MS    (2000u)

static void core3RuntimePublishMenuKvmRoute(void)
{
    if (g_status.controlStatusValid != 0u)
    {
        if (g_status.kvmDesiredTarget == OBSERVER_LINK_KVM_TARGET_CORE3)
            g_menuConfirmedSwitchedAway = 0u;
        else if ((g_status.kvmDesiredTarget == OBSERVER_LINK_KVM_TARGET_CORE0) ||
                 (g_status.kvmDesiredTarget == OBSERVER_LINK_KVM_TARGET_CORE1) ||
                 (g_status.kvmDesiredTarget == OBSERVER_LINK_KVM_TARGET_CORE4))
            g_menuConfirmedSwitchedAway = 1u;
        core3MenuSetKvmRoute(g_status.kvmDesiredTarget,
                             g_status.kvmInputOwner);
    }
    else if ((g_status.kvmControlOwned != 0u) &&
             (g_menuConfirmedSwitchedAway == 0u))
    {
        g_status.kvmDesiredTarget = OBSERVER_LINK_KVM_TARGET_CORE3;
        g_status.kvmInputOwner = OBSERVER_LINK_KVM_TARGET_CORE3;
        core3MenuSetKvmRoute(g_status.kvmDesiredTarget,
                             g_status.kvmInputOwner);
    }
}

static void core3RuntimeClearKvmPending(void)
{
    memset(&g_pendingKvmEvent, 0, sizeof(g_pendingKvmEvent));
    memset(&g_pendingKvmAck, 0, sizeof(g_pendingKvmAck));
    memset(&g_pendingKvmMenu, 0, sizeof(g_pendingKvmMenu));
    g_pendingKvmEventValid = 0u;
    g_pendingKvmAckValid = 0u;
    g_pendingKvmMenuValid = 0u;
}

static uint8_t core3RuntimeFlashLoadFailed(core3VisionParamsLoadStatus status)
{
    switch (status)
    {
        case CORE3_VISION_PARAMS_LOAD_VALID:
        case CORE3_VISION_PARAMS_LOAD_BLANK_INITIALIZED:
        case CORE3_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN:
            return 0u;
        default:
            return 1u;
    }
}

static uint8_t core3RuntimeInitSummaryActive(uint32_t tickMs)
{
    return (uint8_t) ((g_initSummaryUntilMs != 0u) &&
                     ((int32_t)(g_initSummaryUntilMs - tickMs) > 0));
}

static void core3RuntimeUpdateLocalPhase(uint32_t tickMs)
{
    if (g_status.core1LinkReady == 0u)
    {
        g_linkReadyStableStartMs = 0u;
        if ((g_localStandalone != 0u) || (tickMs >= CORE3_INITIAL_LINK_WAIT_MS))
        {
            g_localStandalone = 1u;
            g_status.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_STANDALONE;
            g_status.systemFlags |= OBSERVER_LINK_SYSTEM_FLAG_STANDALONE;
            g_status.primaryReason = OBSERVER_LINK_REASON_CORE1_LINK;
        }
        else
        {
            g_status.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_LINK_WAIT;
            g_status.primaryReason = OBSERVER_LINK_REASON_CORE1_LINK;
        }
    }
    return;
}

```

```

    }
    if (g_localStandalone != 0u)
    {
        if (g_linkReadyStableStartMs == 0u) g_linkReadyStableStartMs = tickMs;
        g_status.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_REJOIN;
        g_status.systemFlags |= OBSERVER_LINK_SYSTEM_FLAG_REJOIN;
        g_status.primaryReason = OBSERVER_LINK_REASON_CORE1_LINK;
        if ((uint32_t) (tickMs - g_linkReadyStableStartMs) >= CORE3_REJOIN_STABLE_MS)
        {
            /* 规划重入只建立规划屏障，不得撤销正交的KVM租约或在途ACK。*/
            g_localStandalone = 0u;
            g_linkReadyStableStartMs = 0u;
            g_rejoinBarrierUntilMs = tickMs + CORE3_RUNTIME_PIT_MS;
        }
        return;
    }
    if ((g_rejoinBarrierUntilMs != 0u) &&
        ((int32_t) (g_rejoinBarrierUntilMs - tickMs) > 0))
    {
        g_status.controlFlags |= OBSERVER_LINK_CONTROL_FLAG_BARRIER_REQUIRED;
        g_status.primaryReason = OBSERVER_LINK_REASON_BARRIER;
    }
    else
    {
        g_rejoinBarrierUntilMs = 0u;
    }
}

static void core3RuntimeBuildStandaloneConfig(observerLinkSharedConfig *config)
{
    memset(config, 0, sizeof(*config));
    config->revision = 1u;
    config->detectMode = 1u;
    config->maxBeacons = 8u;
    config->edgePredictionHoldMs = VISION_TRACK_EDGE_PREDICTION_HOLD_MS_DEFAULT;
    config->sourceId = OBSERVER_LINK_SOURCE_CORE3;
    config->profileVersion = 1u;
    config->c1BorderEnterLeft = VISION_TRACK_BORDER_ENTER_LEFT_DEFAULT;
    config->c1BorderEnterRight = VISION_TRACK_BORDER_ENTER_RIGHT_DEFAULT;
    config->c1BorderEnterTop = VISION_TRACK_BORDER_ENTER_TOP_DEFAULT;
    config->c1BorderEnterBottom = VISION_TRACK_BORDER_ENTER_BOTTOM_DEFAULT;
    config->c1BorderExitLeft = VISION_TRACK_BORDER_EXIT_LEFT_DEFAULT;
    config->c1BorderExitRight = VISION_TRACK_BORDER_EXIT_RIGHT_DEFAULT;
    config->c1BorderExitTop = VISION_TRACK_BORDER_EXIT_TOP_DEFAULT;
    config->c1BorderExitBottom = VISION_TRACK_BORDER_EXIT_BOTTOM_DEFAULT;
    config->observerBorderEnterLeft = VISION_TRACK_BORDER_ENTER_LEFT_DEFAULT;
    config->observerBorderEnterRight = VISION_TRACK_BORDER_ENTER_RIGHT_DEFAULT;
    config->observerBorderEnterTop = VISION_TRACK_BORDER_ENTER_TOP_DEFAULT;
    config->observerBorderEnterBottom = VISION_TRACK_BORDER_ENTER_BOTTOM_DEFAULT;
    config->observerBorderExitLeft = VISION_TRACK_BORDER_EXIT_LEFT_DEFAULT;
    config->observerBorderExitRight = VISION_TRACK_BORDER_EXIT_RIGHT_DEFAULT;
    config->observerBorderExitTop = VISION_TRACK_BORDER_EXIT_TOP_DEFAULT;
    config->observerBorderExitBottom = VISION_TRACK_BORDER_EXIT_BOTTOM_DEFAULT;
    config->c1EdgeBandLeft = VISION_TRACK_EDGE_BAND_C1_LEFT_DEFAULT;
    config->c1EdgeBandRight = VISION_TRACK_EDGE_BAND_C1_RIGHT_DEFAULT;
    config->c1EdgeBandTop = VISION_TRACK_EDGE_BAND_C1_TOP_DEFAULT;
    config->c1EdgeBandBottom = VISION_TRACK_EDGE_BAND_C1_BOTTOM_DEFAULT;
    config->observerEdgeBandLeft = VISION_TRACK_EDGE_BAND_C3_LEFT_DEFAULT;
    config->observerEdgeBandRight = VISION_TRACK_EDGE_BAND_C3_RIGHT_DEFAULT;
    config->observerEdgeBandTop = VISION_TRACK_EDGE_BAND_C3_TOP_DEFAULT;
    config->observerEdgeBandBottom = VISION_TRACK_EDGE_BAND_C3_BOTTOM_DEFAULT;
    config->beaconSizeMin = 1u;
    config->beaconSizeMax = 5000u;
    config->beaconEccMax = 8.0f;
    config->beaconBrightnessMin = 40u;
    config->beaconMaxEndpoints = 20u;
    config->beaconFillMin = 0.05f;
    config->beaconHuMax = 60.0f;
    config->beaconHuAreaMin = 1u;
    config->trackMaxAssocDist = 6.0f;
    config->trackSizeRatioMax = 2.0f;
    config->trackConfirmHits = 3u;
    config->trackDropMisses = 4u;
    config->trackCoastMisses = 2u;
    config->trackPositionAlpha = 1.0f;
    config->trackSizeAlpha = 1.0f;
    config->trackAreaVelAlpha = 0.35f;
    config->trackAssocScoreMax = 1.75f;
    config->trackAssocAreaWeight = 0.45f;
}

```



```

config->trackAssocAreaVelWeight = 0.35f;
config->beaconHuTemplate[0] = 0.180488205803f;
config->beaconHuTemplate[1] = 0.0079012345679f;
config->beaconHuTemplate[2] = 0.000123277089059f;
config->beaconHuTemplate[3] = 0.00000801787369286f;
config->hash = observerLinkSharedConfigHash(config);
}

static void core3RuntimeProcessKvm(uint32_t tickMs)
{
    core3LinkCore3Status linkStatus;
    observerLinkKvmControl control;
    observerLinkKvmEvent event;
    core3KvmCore3Status kvmStatus;
    core3MenuView view;
    observerKvmBindingAction bindingAction;
    uint8_t handled;

    core3LinkCore3GetStatus(&linkStatus);
    bindingAction = observerKvmBindingGuardObserve(&g_kvmBindingGuard,
        linkStatus.ownerSession, linkStatus.bindingEpoch);
    if (bindingAction != OBSERVER_KVM_BINDING_UNCHANGED)
    {
        if (bindingAction == OBSERVER_KVM_BINDING_REBOUNDED)
            core3KvmCore3Revoke();
        core3RuntimeClearKvmPending();
    }
    if (g_status.kvmLinkReady == 0u)
    {
        core3KvmCore3Revoke();
        core3RuntimeClearKvmPending();
    }
    core3KvmCore3Poll(tickMs);
    while (core3LinkCore3TakeKvmControl(&control) != 0u)
    {
        if (g_status.kvmLinkReady != 0u)
        {
            (void) core3KvmCore3ApplyControl(&control, tickMs);
        }
    }
    if (g_pendingKvmEventValid == 0u)
    {
        g_pendingKvmEventValid = core3LinkCore3TakeKvmEvent(
            &g_pendingKvmEvent);
    }
    if ((g_pendingKvmEventValid != 0u) &&
        (core3KvmCore3ApplyEvent(&g_pendingKvmEvent, tickMs) != 0u))
    {
        g_pendingKvmEventValid = 0u;
    }
    if (core3KvmCore3TakeEvent(&event) != 0u)
    {
        handled = core3MenuSubmitKey((core3MenuKey) event.keyId,
            (core3MenuPress) event.keyState);
        core3MenuGetView(&view);
        (void) core3KvmCore3CompleteEvent(
            handled ? OBSERVER_LINK_KVM_RESULT_EXECUTED : OBSERVER_LINK_KVM_RESULT_UNHANDLED,
            (uint8_t) view.state,
            (uint8_t) ((g_status.kvmLinkReady != 0u) &&
                (view.state == CORE3_MENU_HOME) &&
                (view.lastAction == CORE3_MENU_ACTION_NONE)),
            (uint8_t) view.lastAction,
            view.lastActionOk,
            view.screenRevision);
    }
    if (g_pendingKvmAckValid == 0u)
    {
        g_pendingKvmAckValid = core3KvmCore3TakeAck(&g_pendingKvmAck);
    }
    if ((g_pendingKvmAckValid != 0u) &&
        (core3LinkCore3QueueKvmAck(&g_pendingKvmAck) != 0u))
    {
        g_pendingKvmAckValid = 0u;
    }
    core3MenuGetView(&view);
    core3KvmCore3UpdateMenu((uint8_t) view.state,
        (uint8_t) ((g_status.kvmLinkReady != 0u) &&
            (view.state == CORE3_MENU_HOME) &&
            (view.lastAction == CORE3_MENU_ACTION_NONE)),

```

```

        (uint8_t) view.lastAction,
        view.lastActionOk,
        (uint8_t) (view.lastAction != CORE3_MENU_ACTION_NONE),
        view.screenRevision);
if ((view.screenRevision != g_lastKvmMenuRevision) ||
    ((uint32_t) (tickMs - g_lastKvmMenuTickMs) >= 50u))
{
    core3KvmCore3GetMenuStatus(&g_pendingKvmMenu);
    g_pendingKvmMenuValid = 1u;
}
if ((g_pendingKvmMenuValid != 0u) &&
    (core3LinkCore3QueueKvmMenuStatus(&g_pendingKvmMenu) != 0u))
{
    g_lastKvmMenuRevision = g_pendingKvmMenu.screenRevision;
    g_lastKvmMenuTickMs = tickMs;
    g_pendingKvmMenuValid = 0u;
}
core3KvmCore3GetStatus(&kvmStatus);
g_status.kvmControlOwned = (uint8_t) ((g_status.kvmLinkReady != 0u) &&
    (kvmStatus.leaseValid != 0u));
/* KVM 租约与规划控制状态正交。CONTROL_STATUS 暂时过期时，仍以直接租约
 * 作为本机按键 owner 事实，避免有效操控期间框线被误判为黄色。 */
core3RuntimePublishMenuKvmRoute();
}

static void core3RuntimeApplyCameraConfig(uint32_t tickMs)
{
    uint32_t revision;

    if (g_cameraInitialized == 0u)
    {
        return;
    }
    revision = core3VisionParamsGetRevision();
    if (revision == g_lastAppliedCameraConfigRevision)
    {
        return;
    }
    if ((revision == g_lastCameraConfigAttemptRevision) &&
        ((uint32_t) (tickMs - g_lastCameraConfigAttemptTickMs) <
            CORE3_CAMERA_CONFIG_RETRY_MS))
    {
        return;
    }
    g_lastCameraConfigAttemptRevision = revision;
    g_lastCameraConfigAttemptTickMs = tickMs;
    if (core3CameraServiceSetExposure(
        core3VisionParamsGetActive()->exposure) != 0u)
    {
        if (g_status.configApplyFailures < UINT32_MAX)
        {
            g_status.configApplyFailures++;
        }
    }
    else
    {
        g_lastAppliedCameraConfigRevision = revision;
    }
}

static void core3RuntimeRecordMaxUs(uint32_t elapsedUs, uint32_t *maximumUs)
{
    if ((elapsedUs != CORE3_TIMING_INVALID_US) && (elapsedUs > *maximumUs))
    {
        *maximumUs = elapsedUs;
    }
}

static void core3RuntimeAttemptCameraInit(uint32_t tickMs)
{
    uint8_t result;

    g_lastInitAttemptTickMs = tickMs;
    g_status.cameraInitAttempts++;
    result = core3CameraServiceInit();
    if (result == 0u)
    {
        g_cameraInitialized = 1u;
        g_status.initFailedMask &= (uint8_t) ~CORE3_INIT_FAIL_CAMERA;
    }
}

```

```

        g_status.state = CORE3_RUNTIME_CAMERA_STARTING;
        g_lastFrameTickMs = tickMs;
        g_recoveryFrameStreak = 0u;
        core3RuntimeApplyCameraConfig(tickMs);
    }
    else
    {
        g_cameraInitialized = 0u;
        g_status.initFailedMask |= CORE3_INIT_FAIL_CAMERA;
        g_status.cameraInitFailures++;
        g_status.state = CORE3_RUNTIME_CAMERA_FAULT;
    }
}

static void core3RuntimeConsumeFrame(const core3CameraSnapshot *snapshot)
{
    uint32_t frameIntervalMs = 0u;

    if (g_status.frameValid != 0u)
    {
        uint32_t expectedFrames;

        frameIntervalMs = snapshot->tickMs - g_lastFrameTickMs;
        expectedFrames =
            (frameIntervalMs + (CORE3_CAMERA_EXPECTED_FRAME_MS / 2u)) /
            CORE3_CAMERA_EXPECTED_FRAME_MS;
        if (expectedFrames > 1u)
        {
            g_status.estimatedMissedFrames += expectedFrames - 1u;
        }
    }

    g_latestSnapshot = *snapshot;
    g_lastFrameTickMs = snapshot->tickMs;
    g_status.frameValid = 1u;
    g_status.appFrameSeq = snapshot->appFrameSeq;
    g_statsWindowFrames++;

    if (g_status.state == CORE3_RUNTIME_CAMERA_STARTING)
    {
        g_status.state = CORE3_RUNTIME_RUNNING;
        g_recoveryFrameStreak = 0u;
    }
    else if (g_status.state == CORE3_RUNTIME_CAMERA_FAULT)
    {
        if ((g_recoveryFrameStreak == 0u) ||
            (frameIntervalMs < CORE3_CAMERA_FRAME_TIMEOUT_MS))
        {
            g_recoveryFrameStreak++;
        }
        else
        {
            g_recoveryFrameStreak = 1u;
        }

        if (g_recoveryFrameStreak >= CORE3_CAMERA_RECOVERY_FRAMES)
        {
            g_status.state = CORE3_RUNTIME_RUNNING;
            g_recoveryFrameStreak = 0u;
        }
    }
}

static void core3RuntimeUpdateStats(uint32_t tickMs)
{
    uint32_t windowMs = tickMs - g_statsWindowStartTickMs;

    if (windowMs >= core3VisionParamsGetActive()->statsPeriodMs)
    {
        g_status.appFps = (g_statsWindowFrames * 1000u) / windowMs;
        g_statsWindowFrames = 0u;
        g_statsWindowStartTickMs = tickMs;
    }

    if (g_status.frameValid != 0u)
    {
        g_status.frameAgeMs = tickMs - g_lastFrameTickMs;
    }
}

```

```

static void core3RuntimeUpdateParamStatus(void)
{
    core3VisionParamsPersistenceStatus persistenceStatus;

    core3VisionParamsPersistenceGetStatus(&persistenceStatus);
    g_status.paramsLoadStatus = (uint8_t) persistenceStatus.lastLoad;
    g_status.flashWriteFailures = persistenceStatus.writeVerifyFailures;
    g_status.configRevision = core3VisionParamsGetRevision();
}

static void core3RuntimeUpdateLinkStatus(void)
{
    core3LinkCore3Status linkStatus;
    observerLinkViewOverlay overlay;
    uint32_t overlayAge = UINT32_MAX;
    uint8_t statusValid;

    core3LinkCore3GetStatus(&linkStatus);
    g_status.core1LinkOnline = linkStatus.online;
    g_status.core1ProtocolCompatible = linkStatus.protocolCompatible;
    g_status.sharedConfigStatus = linkStatus.configStatus;
    g_status.sharedConfigMode = linkStatus.stagedMode;
    g_status.core1LinkReady = (uint8_t) (
        (linkStatus.online != 0u) && (linkStatus.protocolCompatible != 0u) &&
        (linkStatus.planningCompatible != 0u) &&
        (linkStatus.configStatus == OBSERVER_LINK_CONFIG_APPLIED));
    g_status.kvmLinkReady = (uint8_t) ((linkStatus.online != 0u) &&
        (linkStatus.protocolCompatible != 0u) &&
        ((linkStatus.peerCapabilities &
            (OBSERVER_LINK_CAP_KVM | OBSERVER_LINK_CAP_MENU_STATUS)) ==
            (OBSERVER_LINK_CAP_KVM | OBSERVER_LINK_CAP_MENU_STATUS)));
    statusValid = (uint8_t) ((g_status.core1LinkReady != 0u) &&
        (linkStatus.controlStatusValid != 0u) &&
        (linkStatus.cameraControlValid != 0u));
    g_status.controlStatusValid = linkStatus.controlStatusValid;
    g_status.lastCameraControlState = linkStatus.lastKnownCameraControlState;
    g_status.cameraControlState = statusValid ?
        linkStatus.cameraControlState : OBSERVER_LINK_CAMERA_CONTROL_SAFE;
    g_status.cameraControlReady = statusValid;
    g_status.cameraControlLocked = 1u;
    g_status.activeSource = OBSERVER_LINK_SOURCE_NONE;
    g_status.controlClass = OBSERVER_LINK_CONTROL_CLASS_NONE;
    g_status.controlFlags = 0u;
    g_status.navState = 0u;
    g_status.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_LINK_WAIT;
    g_status.systemFlags = 0u;
    g_status.primaryReason = OBSERVER_LINK_REASON_CORE1_LINK;
    g_status.kvmDesiredTarget = OBSERVER_LINK_KVM_TARGET_CORE0;
    g_status.kvmInputOwner = OBSERVER_LINK_KVM_TARGET_NONE;
    g_status.controlBeaconCount = 0u;
    g_status.core0LinkUp = 0u;
    g_status.core2LinkUp = 0u;
    g_status.targetX = 0u;
    g_status.targetY = 0u;
    g_status.targetSize = 0u;
    g_status.errXQ4 = 0;
    g_status.errYQ4 = 0;
    g_status.viewOverlayValid = 0u;
    g_status.viewOverlayAgeMs = UINT32_MAX;
    memset(&g_status.viewOverlay, 0, sizeof(g_status.viewOverlay));
    if (g_status.controlStatusValid != 0u)
    {
        const observerLinkControlStatus *control = &linkStatus.controlStatus;

        g_status.kvmDesiredTarget = control->desiredTarget;
        g_status.kvmInputOwner = control->ownerTarget;
        g_status.core0LinkUp = (uint8_t) ((control->flags &
            OBSERVER_LINK_CONTROL_FLAG_CORE0_LINK_UP) != 0u);
        g_status.core2LinkUp = (uint8_t) ((control->flags &
            OBSERVER_LINK_CONTROL_FLAG_CORE2_LINK_UP) != 0u);
    }
    if (statusValid != 0u)
    {
        const observerLinkControlStatus *control = &linkStatus.controlStatus;

        g_status.activeSource = control->activeSource;
        g_status.controlClass = control->controlClass;
        g_status.controlFlags = control->flags;
        g_status.navState = control->navState;
    }
}

```

```

    g_status.systemPhase = control->systemPhase;
    g_status.systemFlags = control->systemFlags;
    g_status.primaryReason = control->primaryReason;
    g_status.controlBeaconCount = control->beaconCount;
    g_status.cameraControlLocked = (uint8_t)((control->flags &
        OBSERVER_LINK_CONTROL_FLAG_REMOTE_ELIGIBLE) == 0u);
    g_status.targetX = control->targetX;
    g_status.targetY = control->targetY;
    g_status.targetSize = control->targetSize;
    g_status.errXQ4 = control->errXQ4;
    g_status.errYQ4 = control->errYQ4;
}
if (core3LinkCore3GetViewOverlay(&overlay, g_status.tickMs,
    200u, &overlayAge) != 0u)
{
    g_status.viewOverlay = overlay;
    g_status.viewOverlayValid = 1u;
    g_status.viewOverlayAgeMs = overlayAge;
}
g_status.core1Session = linkStatus.ownerSession;
g_status.sharedConfigRevision = linkStatus.stagedRevision;
g_status.sharedConfigHash = linkStatus.stagedHash;
g_status.linkCrcErrors = linkStatus.localDiagnostics.crcErrors;
g_status.linkLengthErrors = linkStatus.localDiagnostics.lengthErrors;
g_status.linkRxOverflowErrors = linkStatus.localDiagnostics.rxOverflowErrors;
g_status.peerRxOverflowErrors = linkStatus.peerDiagnostics.rxOverflowErrors;
g_status.linkTxDeferred = linkStatus.txDeferred;
}

static void core3RuntimeUpdateVisionStatus(void)
{
    core3VisionPipelineStatus pipelineStatus;
    core3VisionObservation observation;

    core3VisionPipelineGetStatus(&pipelineStatus);
    core3VisionPipelineGetObservation(&observation);
    g_status.visionApplied = pipelineStatus.applied;
    g_status.visionDetectMode = pipelineStatus.detectMode;
    if (observation.valid != 0u)
    {
        uint16_t i;
        uint16_t ordinaryRetained = 0u;
        g_latestObservation = observation;
        g_status.visionOverflow = observation.truncated;
        g_status.visionMaxBeacons = observation.maxBeacons;
        g_status.confirmedBeacons = observation.confirmedTotal;
        g_status.freshBeacons = observation.freshTotal;
        for (i = 0u; i < observation.retainedCount; i++)
        {
            if ((observation.tracks[i].flags &
                BEACON_CONFIRMED_FLAG_BORDER_REJECTED) == 0u)
                ordinaryRetained++;
        }
        g_status.retainedBeacons = ordinaryRetained;
    }
    else
    {
        memset(&g_latestObservation, 0, sizeof(g_latestObservation));
        g_status.visionOverflow = 0u;
        g_status.visionMaxBeacons = 0u;
        g_status.confirmedBeacons = 0u;
        g_status.freshBeacons = 0u;
        g_status.retainedBeacons = 0u;
    }
}

static void core3RuntimePublishHealth(uint32_t tickMs)
{
    core3LinkCore3Status linkStatus;
    observerLinkHealth health;
    uint32_t frameAge = (g_status.appFrameSeq != 0u) ?
        (tickMs - g_lastFrameTickMs) : UINT32_MAX;

    core3LinkCore3GetStatus(&linkStatus);
    memset(&health, 0, sizeof(health));
    health.ownerSession = linkStatus.ownerSession;
    health.appFrameSeq = g_status.appFrameSeq;
    health.captureTickMs = g_lastFrameTickMs;
    health.frameAgeMs = (frameAge > UINT16_MAX) ? UINT16_MAX : (uint16_t) frameAge;
}

```

```

health.bridgeAgeMs = UINT16_MAX;
health.sourceId = OBSERVER_LINK_SOURCE_CORE3;
health.configState = linkStatus.configStatus;
if ((g_cameraInitialized != 0u) &&
    (g_status.state == CORE3_RUNTIME_RUNNING))
{
    health.cameraState = OBSERVER_LINK_CAMERA_RUNNING;
}
else if ((g_status.state == CORE3_RUNTIME_CAMERA_FAULT) ||
    (g_status.cameraInitFailures != 0u))
{
    health.cameraState = OBSERVER_LINK_CAMERA_FAULT;
}
else
{
    health.cameraState = OBSERVER_LINK_CAMERA_UNKNOWN;
}
if (frameAge < CORE3_CAMERA_FRAME_TIMEOUT_MS)
{
    health.flags |= OBSERVER_LINK_HEALTH_FLAG_FRAME_FRESH;
}
(void) core3LinkCore3PublishHealth(&health);
}

static uint8_t core3RuntimeBuildPlanningSnapshot(
    const core3VisionObservation *observation,
    observerLinkPlanningTrackSnapshot *snapshot)
{
    uint16_t i;
    uint16_t ordinaryRetained = 0u;

    if ((observation == NULL) || (snapshot == NULL) ||
        (observation->valid == 0u) ||
        (observation->sourceId != OBSERVER_LINK_SOURCE_CORE3) ||
        (observation->detectMode > 1u) ||
        (observation->ownerSession == 0u) ||
        (observation->retainedCount > OBSERVER_LINK_MAX_TRACKS) ||
        (observation->confirmedTotal > UINT8_MAX) ||
        (observation->freshTotal > UINT8_MAX) ||
        (observation->freshLeft > UINT8_MAX) ||
        (observation->freshRight > UINT8_MAX) ||
        (observation->freshTotal > observation->confirmedTotal) ||
        ((uint32_t) observation->freshLeft + observation->freshRight !=
         observation->freshTotal))
    {
        return 0u;
    }

    memset(snapshot, 0, sizeof(*snapshot));
    snapshot->valid = 1u;
    snapshot->sourceId = observation->sourceId;
    snapshot->detectMode = observation->detectMode;
    snapshot->truncated = observation->truncated;
    snapshot->ownerSession = observation->ownerSession;
    snapshot->revision = observation->configRevision;
    snapshot->hash = observation->configHash;
    snapshot->appFrameSeq = observation->appFrameSeq;
    snapshot->captureTickMs = observation->tickMs;
    snapshot->trackerFrameSeq = observation->trackerFrameSeq;
    snapshot->confirmedTotal = (uint8_t) observation->confirmedTotal;
    snapshot->freshTotal = (uint8_t) observation->freshTotal;
    snapshot->freshLeft = (uint8_t) observation->freshLeft;
    snapshot->freshRight = (uint8_t) observation->freshRight;
    snapshot->retainedCount = (uint8_t) observation->retainedCount;

    for (i = 0u; i < observation->retainedCount; i++)
    {
        const core3VisionTrack *source = &observation->tracks[i];
        observerLinkPlanningTrack *destination = &snapshot->tracks[i];
        uint8_t rejected = (uint8_t) ((source->flags &
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) != 0u);
        uint16_t j;

        if ((source->centerX >= 188u) || (source->centerY >= 120u) ||
            (source->predX >= 188u) || (source->predY >= 120u) ||
            (source->size > 22560u) || (source->predSize > 22560u) ||
            (source->trackId == 0u) ||
            (source->misses > OBSERVER_LINK_PLANNING_TRACK_MISSES_MAX) ||
            ((source->flags & (uint8_t) ~(OBSERVER_LINK_TRACK_FLAG_FRESH |

```

```

        OBSERVER_LINK_TRACK_FLAG_COAST |
        OBSERVER_LINK_TRACK_FLAG_EDGE_GRACE |
        OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED))) != 0u) ||
        (((source->flags & OBSERVER_LINK_TRACK_FLAG_FRESH) != 0u) !=
         (source->fresh != 0u)))
    {
        return 0u;
    }
    for (j = 0u; j < i; j++)
    {
        if (observation->tracks[j].trackId == source->trackId)
        {
            return 0u;
        }
    }
    if (rejected != 0u)
    {
        if ((source->flags != OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) ||
            (source->centerX != 0u) || (source->centerY != 0u) ||
            (source->predX != 0u) || (source->predY != 0u) ||
            (source->size != 0u) || (source->predSize != 0u) ||
            (source->misses != 0u) || (source->fresh != 0u) ||
            (source->velX != 0.0f) || (source->velY != 0.0f) ||
            (source->areaVel != 0.0f)) return 0u;
    }
    else
    {
        ordinaryRetained++;
    }
    destination->centerX = (uint8_t) source->centerX;
    destination->centerY = (uint8_t) source->centerY;
    destination->predX = (uint8_t) source->predX;
    destination->predY = (uint8_t) source->predY;
    destination->size = source->size;
    destination->predSize = source->predSize;
    destination->trackId = source->trackId;
    destination->misses = source->misses;
    destination->flags = source->flags;
    if ((observerLinkQuantizeSigned(source->velX, 128.0f,
        -256.0f, 255.9921875f, &destination->velXQ7) == 0u) ||
        (observerLinkQuantizeSigned(source->velY, 128.0f,
        -256.0f, 255.9921875f, &destination->velYQ7) == 0u) ||
        (observerLinkQuantizeSigned(source->areaVel, 1.0f,
        -32768.0f, 32767.0f, &destination->areaVelQ0) == 0u))
    {
        return 0u;
    }
}
return (uint8_t) (ordinaryRetained <= observation->confirmedTotal);
}

```

```

static void core3RuntimeProcessVisionFrame(const core3CameraSnapshot *snapshot)

```

```

{
    core3LinkCore3Status linkStatus;
    observerLinkSharedConfig staged;
    core3VisionObservation observation;
    observerLinkPlanningTrackSnapshot wireSnapshot;
    core3VisionPipelineStatus pipelineStatus;
    uint8_t remoteQualified;

    core3LinkCore3GetStatus(&linkStatus);
    remoteQualified = (uint8_t) ((linkStatus.online != 0u) &&
        (linkStatus.protocolCompatible != 0u) &&
        (linkStatus.ownerSession != 0u) &&
        (core3LinkCore3GetStagedConfig(&staged) != 0u));
    core3VisionPipelineGetStatus(&pipelineStatus);
    if ((remoteQualified == 0u) && (pipelineStatus.applied == 0u))
    {
        core3LinkCore3GetStatus(&linkStatus);
        core3RuntimeBuildStandaloneConfig(&staged);
        if (core3VisionPipelineApplyAtFrameBoundary(
            core3VisionParamsGetActive(),
            core3VisionParamsGetRevision(),
            &staged,
            linkStatus.localSession,
            linkStatus.localSession) == 0u)
        {
            core3RuntimeUpdateVisionStatus();
            return;
        }
    }
}

```

```

    }
}
if ((remoteQualified != 0u) &&
    (core3VisionPipelineApplyAtFrameBoundary(
        core3VisionParamsGetActive(),
        core3VisionParamsGetRevision(),
        &staged,
        linkStatus.localSession,
        linkStatus.ownerSession) == 0u))
{
    if (g_status.configApplyFailures < UINT32_MAX)
    {
        g_status.configApplyFailures++;
    }
    core3VisionPipelineInvalidate();
    core3RuntimeUpdateVisionStatus();
    return;
}
if ((remoteQualified != 0u) &&
    ((linkStatus.configStatus != OBSERVER_LINK_CONFIG_APPLIED) ||
     (linkStatus.appliedRevision != staged.revision) ||
     (linkStatus.appliedHash != staged.hash) ||
     (linkStatus.appliedMode != staged.detectMode)) &&
    (core3LinkCore3MarkConfigApplied(linkStatus.ownerSession,
        staged.revision,
        staged.hash,
        staged.detectMode) == 0u))
{
    if (g_status.configApplyFailures < UINT32_MAX)
    {
        g_status.configApplyFailures++;
    }
    core3RuntimeUpdateVisionStatus();
    return;
}
core3VisionPipelineSetSelectedTrackId(
    ((g_status.viewOverlayValid != 0u) &&
     (g_status.activeSource == OBSERVER_LINK_SOURCE_CORE3) &&
     ((g_status.viewOverlay.flags &
        OBSERVER_LINK_OVERLAY_FLAG_SELECTED) != 0u)) ?
    g_status.viewOverlay.selectedTrackId : 0u);
if (core3VisionPipelineProcess(snapshot) == 0u)
{
    if (g_status.configApplyFailures < UINT32_MAX)
    {
        g_status.configApplyFailures++;
    }
    core3VisionPipelineInvalidate();
}
else
{
    core3VisionPipelineGetObservation(&observation);
    if ((remoteQualified != 0u) &&
        (observation.ownerSession == linkStatus.ownerSession) &&
        (core3RuntimeBuildPlanningSnapshot(&observation,
            &wireSnapshot) != 0u)
        )
    {
        (void) core3LinkCore3QueuePlanningTrackSnapshot(&wireSnapshot);
    }
}
core3RuntimeUpdateVisionStatus();
}

static void core3RuntimeRenderView(const core3MenuView *view, uint32_t tickMs,
    uint8_t updateThresholdImages)
{
    uint32_t displayUs;

    if (view == NULL)
    {
        return;
    }
    if ((view->state != CORE3_MENU_HOME) ||
        (view->lastAction != CORE3_MENU_ACTION_NONE))
    {
        g_lastMenuRenderTickMs = tickMs;
        if (view->state == CORE3_MENU_THRESHOLD_EDIT)
        {

```



```

        displayUs = core3DiagnosticsRenderThresholdEditor(view,
            ((g_status.frameValid != 0u) &&
             (g_status.state != CORE3_RUNTIME_CAMERA_FAULT)) ?
             &g_latestSnapshot : NULL,
            ((g_status.frameValid != 0u) &&
             (g_status.state != CORE3_RUNTIME_CAMERA_FAULT)) ?
             core3VisionPipelineGetPreview() : NULL,
            updateThresholdImages);
    }
    else
    {
        displayUs = core3DiagnosticsRenderMenu(view);
    }
}
else
{
    g_lastHudTickMs = tickMs;
    displayUs = core3DiagnosticsRenderHud(&g_status);
}
g_lastRenderedMenuRevision = view->screenRevision;
if (g_status.timingInvalid == 0u)
{
    core3RuntimeRecordMaxUs(displayUs, &g_status.displayMaxUs);
}
}

void core3RuntimeInit(void)
{
    core3VisionParamsStruct initialParams;
    core3VisionParamsLoadStatus loadStatus;
    uint32_t displayUs;

    g_tickMs = 0u;
    g_tickDue = 0u;
    g_cameraInitialized = 0u;
    g_recoveryFrameStreak = 0u;
    g_lastInitAttemptTickMs = 0u;
    g_lastFrameTickMs = 0u;
    g_statsWindowStartTickMs = 0u;
    g_statsWindowFrames = 0u;
    g_lastHudTickMs = 0u;
    g_lastPreviewTickMs = 0u;
    g_lastMenuRenderTickMs = 0u;
    g_lastRenderedMenuRevision = 0u;
    g_lastCameraConfigAttemptTickMs = 0u;
    g_lastCameraConfigAttemptRevision = 0u;
    g_lastAppliedCameraConfigRevision = 0u;
    observerKvmBindingGuardInit(&g_kvmBindingGuard);
    g_lastKvmLocalSession = 0u;
    g_lastKvmMenuTickMs = 0u;
    g_lastKvmMenuRevision = 0u;
    core3RuntimeClearKvmPending();
    g_linkReadyStableStartMs = 0u;
    g_rejoinBarrierUntilMs = 0u;
    g_localStandalone = 0u;
    g_initSummaryUntilMs = 0u;
    g_initSummaryLastSecond = 0xFFu;
    memset(&g_status, 0, sizeof(g_status));
    g_menuConfirmedSwitchedAway = 0u;
    memset(&g_latestSnapshot, 0, sizeof(g_latestSnapshot));

    g_status.state = CORE3_RUNTIME_BOOT;
    core3DiagnosticsDisplayInitEarly();
    core3VisionParamsLoadDefaults(&initialParams);
    loadStatus = core3VisionParamsFlashLoad(&initialParams);
    g_status.paramsLoadStatus = (uint8_t) loadStatus;
    if (core3RuntimeFlashLoadFailed(loadStatus) != 0u)
        g_status.initFailedMask |= CORE3_INIT_FAIL_FLASH;
    core3VisionParamsSanitize(&initialParams);
    core3VisionParamsSetInitial(&initialParams);
    core3MenuInit();
    core3DiagnosticsRuntimeInit();
    g_status.timingInvalid = (uint8_t) (core3DiagnosticsTimingValid() == 0u);
    if (g_status.timingInvalid != 0u)
    {
        g_status.snapshotMaxUs = CORE3_TIMING_INVALID_US;
        g_status.foregroundMaxUs = CORE3_TIMING_INVALID_US;
        g_status.displayMaxUs = CORE3_TIMING_INVALID_US;
    }
}

```

```

if (core3VisionPipelineInit() == 0u)
    g_status.initFailedMask |= CORE3_INIT_FAIL_PIPELINE;
core3LinkCore3Init(0u);
{
    core3VisionPipelineStatus pipelineStatus;
    core3VisionPipelineGetStatus(&pipelineStatus);
    core3LinkCore3SetVisionReady(pipelineStatus.ready);
}
{
    core3LinkCore3Status linkStatus;
    core3LinkCore3GetStatus(&linkStatus);
    core3KvmCore3Init(linkStatus.localSession);
    g_lastKvmLocalSession = linkStatus.localSession;
}
core3RuntimeUpdateParamStatus();
core3RuntimeUpdateLinkStatus();
core3RuntimeUpdateLocalPhase(0u);
core3MenuSetLinkState(g_status.kvmLinkReady,
    g_status.lastCameraControlState);
core3RuntimePublishMenuKvmRoute();
core3RuntimeUpdateVisionStatus();

core3RuntimeAttemptCameraInit(0u);
if (g_cameraInitialized == 0u)
    g_status.initFailedMask |= CORE3_INIT_FAIL_CAMERA;
if (g_status.timingInvalid != 0u)
    g_status.initUnverifiedMask |= CORE3_INIT_CHECK_DWT;
if ((g_status.initFailedMask | g_status.initUnverifiedMask) != 0u)
{
    g_initSummaryUntilMs = 3000u;
    g_initSummaryLastSecond = 3u;
}
core3DiagnosticsUpdateRgb(&g_status, 0u);

{
    core3MenuView menuView;

    core3MenuGetView(&menuView);
    displayUs = core3RuntimeInitSummaryActive(0u) ?
        core3DiagnosticsRenderInitFaultSummary(g_status.initFailedMask,
            g_status.initUnverifiedMask, 3u) :
        core3DiagnosticsRenderMenu(&menuView);
    g_lastRenderedMenuRevision = menuView.screenRevision;
    if (g_status.timingInvalid == 0u)
    {
        core3RuntimeRecordMaxUs(displayUs, &g_status.displayMaxUs);
    }
}

/* PIT 最后启动，确保 ISR 首次到来前全部 owner 状态已就绪。*/
pit_ms_init(PIT_CH2, CORE3_RUNTIME_PIT_MS);
}

void core3RuntimeTick10msIsr(void)
{
    g_tickMs += CORE3_RUNTIME_PIT_MS;
    g_tickDue = 1u;
}

void core3RuntimeProcess(void)
{
    core3CameraSnapshot snapshot;
    uint32_t copyCycles = 0u;
    uint32_t foregroundStartCycles;
    uint32_t foregroundUs;
    uint32_t displayUs;
    uint32_t tickMs = g_tickMs;
    uint8_t captured = 0u;
    core3MenuView menuView;

    foregroundStartCycles = core3DiagnosticsCycleNow();
    /* 到期位只作观测提示，调度真相是单写者 tick 的无符号差值，因此清零竞态不会丢时间。*/
    g_tickDue = 0u;
    g_status.tickMs = tickMs;
    core3LinkCore3Poll(tickMs);
    {
        core3LinkCore3Status linkStatus;
        core3LinkCore3GetStatus(&linkStatus);
    }
}

```

```

    if ((linkStatus.localSession != 0u) &&
        (linkStatus.localSession != g_lastKvmLocalSession))
    {
        core3KvmCore3Init(linkStatus.localSession);
        g_lastKvmLocalSession = linkStatus.localSession;
        core3RuntimeClearKvmPending();
    }
}
core3RuntimeUpdateLinkStatus();
core3RuntimeUpdateLocalPhase(tickMs);
core3MenuSetLinkState(g_status.kvmLinkReady,
    g_status.lastCameraControlState);
core3RuntimePublishMenuKvmRoute();
core3MenuSetDetectMode((g_status.sharedConfigMode <= 1u)
    ? g_status.sharedConfigMode
    : g_status.visionDetectMode);
core3MenuTick(tickMs);
core3RuntimeProcessKvm(tickMs);
/* ACK/MENU STATUS 在逻辑提交后同拍发送，不等待相机复制或视觉算法。 */
core3LinkCore3FlushControl(tickMs);
core3MenuSetControlOwned(g_status.kvmControlOwned);
core3MenuGetView(&menuView);

if (g_cameraInitialized != 0u)
{
    captured = core3CameraServiceTryCapture(tickMs,
        (uint8_t) (g_status.timingInvalid == 0u),
        &snapshot,
        &copyCycles);
    if (captured != 0u)
    {
        uint32_t copyUs = core3DiagnosticsCyclesToUs(copyCycles);

        core3RuntimeConsumeFrame(&snapshot);
        if (g_status.timingInvalid == 0u)
        {
            core3RuntimeRecordMaxUs(copyUs, &g_status.snapshotMaxUs);
        }
    }
}

/* 快照仍严格早于显示；菜单状态变更在大块视觉算法前优先脏区绘制，
 * 不再叠加 100ms 周期门和一整拍算法耗时。 */
if ((core3RuntimeInitSummaryActive(tickMs) == 0u) &&
    (menuView.screenRevision != g_lastRenderedMenuRevision))
{
    core3RuntimeRenderView(&menuView, tickMs, 0u);
}
if (captured != 0u)
{
    core3RuntimeProcessVisionFrame(&snapshot);
    /* 新规划快照产生后立即使用本 epoch 剩余发送预算；UART 仍由前台单 owner 串行访问。 */
    core3LinkCore3FlushTx(tickMs);
}

if (((g_status.state == CORE3_RUNTIME_CAMERA_STARTING) ||
    (g_status.state == CORE3_RUNTIME_RUNNING)) &&
    ((uint32_t) (tickMs - g_lastFrameTickMs) >= CORE3_CAMERA_FRAME_TIMEOUT_MS))
{
    g_status.state = CORE3_RUNTIME_CAMERA_FAULT;
    g_recoveryFrameStreak = 0u;
}
}
else if ((uint32_t) (tickMs - g_lastInitAttemptTickMs) >= CORE3_CAMERA_INIT_RETRY_MS)
{
    core3RuntimeAttemptCameraInit(tickMs);
}

core3RuntimeUpdateStats(tickMs);
core3RuntimeUpdateParamStatus();
core3RuntimeApplyCameraConfig(tickMs);
core3RuntimePublishHealth(tickMs);
core3DiagnosticsUpdateRgb(&g_status, tickMs);

foregroundUs = core3DiagnosticsCyclesToUs(core3DiagnosticsCycleNow() - foregroundStartCycles);
if (g_status.timingInvalid == 0u)
{
    core3RuntimeRecordMaxUs(foregroundUs, &g_status.foregroundMaxUs);
}

```

```

if (core3RuntimeInitSummaryActive(tickMs) != 0u)
{
    uint8_t second = (uint8_t) ((g_initSummaryUntilMs - tickMs + 999u) / 1000u);
    if (second != g_initSummaryLastSecond)
    {
        g_initSummaryLastSecond = second;
        displayUs = core3DiagnosticsRenderInitFaultSummary(g_status.initFailedMask,
            g_status.initUnverifiedMask, second);
        if (g_status.timingInvalid == 0u)
            core3RuntimeRecordMaxUs(displayUs, &g_status.displayMaxUs);
    }
    return;
}
g_initSummaryUntilMs = 0u;
g_initSummaryLastSecond = 0xFFu;

if ((menuView.state == CORE3_MENU_HOME) &&
    (menuView.lastAction == CORE3_MENU_ACTION_NONE) &&
    (g_status.frameValid != 0u) &&
    ((uint32_t) (tickMs - g_lastPreviewTickMs) >=
        core3VisionParamsGetActive()->previewPeriodMs))
{
    g_lastPreviewTickMs = tickMs;
    displayUs = core3DiagnosticsRenderPreview(&g_latestSnapshot, &g_status,
        &g_latestObservation);
    if (g_status.timingInvalid == 0u)
    {
        core3RuntimeRecordMaxUs(displayUs, &g_status.displayMaxUs);
    }
}

if ((menuView.state != CORE3_MENU_HOME) ||
    (menuView.lastAction != CORE3_MENU_ACTION_NONE))
{
    if (!(menuView.state == CORE3_MENU_THRESHOLD_EDIT) &&
        (menuView.screenRevision != g_lastRenderedMenuRevision) &&
        (captured == 0u) &&
        (g_cameraInitialized != 0u) &&
        (g_status.state == CORE3_RUNTIME_RUNNING) &&
        (g_status.frameValid != 0u)) &&
        ((menuView.screenRevision != g_lastRenderedMenuRevision) ||
        ((uint32_t) (tickMs - g_lastMenuRenderTickMs) >=
            CORE3_MENU_RENDER_PERIOD_MS)))
    {
        uint8_t updateThresholdImages = 0u;
        if ((menuView.state == CORE3_MENU_THRESHOLD_EDIT) &&
            ((uint32_t) (tickMs - g_lastPreviewTickMs) >=
                core3VisionParamsGetActive()->previewPeriodMs))
        {
            g_lastPreviewTickMs = tickMs;
            updateThresholdImages = 1u;
        }
        core3RuntimeRenderView(&menuView, tickMs, updateThresholdImages);
    }
}
else if ((g_lastRenderedMenuRevision != menuView.screenRevision) ||
    ((uint32_t) (tickMs - g_lastHudTickMs) >=
        core3VisionParamsGetActive()->hudPeriodMs))
{
    core3RuntimeRenderView(&menuView, tickMs, 0u);
}
}

```

core3_cyt4bb7/project/code/app/core3Runtime.h

SHA-256

:

af695362cb4193cd9f46d1578dacb0c85303a31ce25f766fa0b602747d69c2d2

代码 (c)

```

#ifndef CORE3_RUNTIME_H
#define CORE3_RUNTIME_H

#include <stdint.h>

```

```

#include "observerLinkProtocol.h"

typedef enum
{
    CORE3_RUNTIME_BOOT = 0,
    CORE3_RUNTIME_CAMERA_STARTING,
    CORE3_RUNTIME_RUNNING,
    CORE3_RUNTIME_CAMERA_FAULT
} core3RuntimeState;

#define CORE3_INIT_FAIL_FLASH    (0x01u)
#define CORE3_INIT_FAIL_CAMERA  (0x02u)
#define CORE3_INIT_FAIL_PIPELINE (0x04u)
#define CORE3_INIT_CHECK_DWT    (0x01u)

typedef struct
{
    core3RuntimeState state;
    uint8_t frameValid;
    uint8_t timingInvalid;
    uint8_t initFailedMask;
    uint8_t initUnverifiedMask;
    uint32_t tickMs;
    uint32_t appFrameSeq;
    uint32_t appFps;
    uint32_t frameAgeMs;
    uint32_t estimatedMissedFrames;
    uint32_t snapshotMaxUs;
    uint32_t foregroundMaxUs;
    uint32_t displayMaxUs;
    uint32_t cameraInitAttempts;
    uint32_t cameraInitFailures;
    uint32_t configRevision;
    uint32_t configApplyFailures;
    uint32_t flashWriteFailures;
    uint8_t paramsLoadStatus;
    uint8_t core1LinkOnline;
    uint8_t core1LinkReady;
    uint8_t kvmLinkReady;
    uint8_t core1ProtocolCompatible;
    uint8_t sharedConfigStatus;
    uint8_t sharedConfigMode;
    uint8_t cameraControlState;
    uint8_t lastCameraControlState;
    uint8_t cameraControlReady;
    uint8_t cameraControlLocked;
    uint8_t kvmControlOwned;
    uint8_t controlStatusValid;
    uint8_t activeSource;
    uint8_t controlClass;
    uint8_t controlFlags;
    uint8_t navState;
    uint8_t systemPhase;
    uint8_t systemFlags;
    uint8_t primaryReason;
    uint8_t kvmDesiredTarget;
    uint8_t kvmInputOwner;
    uint8_t controlBeaconCount;
    uint8_t core0LinkUp;
    uint8_t core2LinkUp;
    uint16_t targetX;
    uint16_t targetY;
    uint16_t targetSize;
    int16_t errXQ4;
    int16_t errYQ4;
    uint8_t viewOverlayValid;
    uint32_t viewOverlayAgeMs;
    observerLinkViewOverlay viewOverlay;
    uint32_t core1Session;
    uint32_t sharedConfigRevision;
    uint32_t sharedConfigHash;
    uint32_t linkCrcErrors;
    uint32_t linkLengthErrors;
    uint32_t linkRxOverflowErrors;
    uint32_t peerRxOverflowErrors;
    uint32_t linkTxDeferred;
    uint8_t visionApplied;
    uint8_t visionDetectMode;
    uint8_t visionOverflow;

```

```

    uint8_t visionMaxBeacons;
    uint16_t confirmedBeacons;
    uint16_t freshBeacons;
    uint16_t retainedBeacons;
} core3RuntimeStatus;

void core3RuntimeInit(void);
void core3RuntimeTick10msIsr(void);
void core3RuntimeProcess(void);

#endif /* CORE3_RUNTIME_H */

```

core3_cyt4bb7/project/code/app/core3VisionPipeline.c

SHA-256

:

ed76e7ec6124e2e42a391a88ca0488126aa15db439c829c4e069baed207ab432

代码 (c)

```

#include "core3VisionPipeline.h"

#include <string.h>

#include "beaconTrack.h"
#include "core3VisionWorkspace.h"
#include "om_beaconcc.h"
#include "openMorph.h"

#define CORE3_VISION_WIDTH (188u)
#define CORE3_VISION_HEIGHT (120u)
#define CORE3_VISION_PIXELS (CORE3_VISION_WIDTH * CORE3_VISION_HEIGHT)
#define CORE3_VISION_CANNY_SIGMA (0.75f)

static uint8_t g_preview[CORE3_VISION_PIXELS];
static BEACON_DETECTOR_STRUCT g_detector;
static beaconTrackerStruct g_tracker;
static observerLinkSharedConfig g_activeShared;
static core3VisionObservation g_observation;
static core3VisionPipelineStatus g_status;
static uint32_t g_ownerSession;
static uint32_t g_sourceBootSession;
static uint32_t g_localRevision;
static uint16_t g_selectedTrackId;

_Static_assert(CORE3_VISION_PIXELS == OM_BUFFER_SIZE,
    "Core3 pipeline and OpenMorph image sizes differ");
_Static_assert(CORE3_VISION_MAX_PUBLISHED_TRACKS <= OM_MAX_OBJECTS,
    "Published track capacity exceeds tracker capacity");

static void clearTracking(void)
{
    memset(&g_detector, 0, sizeof(g_detector));
    beaconTrackInit(&g_tracker);
    memset(&g_observation, 0, sizeof(g_observation));
    memset(g_preview, 0, sizeof(g_preview));
}

static void applyDetectorConfig(const core3VisionParamsStruct *localParams,
    const observerLinkSharedConfig *sharedConfig)
{
    BEACONCC_CONFIG_STRUCT binaryConfig;
    beaconTrackBorderConfigStruct border;
    beaconTrackEdgePredictionConfigStruct edge;

    memset(&binaryConfig, 0, sizeof(binaryConfig));
    binaryConfig.thresholdLow = (uint8_t) localParams->binaryLow;
    binaryConfig.thresholdHigh = (uint8_t) localParams->binaryHigh;
    binaryConfig.areaMin = sharedConfig->beaconSizeMin;
    binaryConfig.areaMax = sharedConfig->beaconSizeMax;
    binaryConfig.brightnessMin = sharedConfig->beaconBrightnessMin;
    binaryConfig.eccMax = sharedConfig->beaconEccMax;
    binaryConfig.fillMin = sharedConfig->beaconFillMin;
    binaryConfig.huMax = sharedConfig->beaconHuMax;
    binaryConfig.huAreaMin = sharedConfig->beaconHuAreaMin;
    beaconCcSetConfig(&binaryConfig);
}

```

```

beaconCcSetTemplate(sharedConfig->beaconHuTemplate,
    sharedConfig->beaconHuMax);
cannySetThreshold(localParams->legacyCannyLow,
    localParams->legacyCannyHigh);
beaconDetectInit(sharedConfig->beaconSizeMin,
    sharedConfig->beaconSizeMax,
    sharedConfig->beaconEccMax,
    sharedConfig->beaconBrightnessMin,
    sharedConfig->beaconMaxEndpoints);
beaconTrackSetParams(sharedConfig->trackMaxAssocDist,
    sharedConfig->trackSizeRatioMax,
    sharedConfig->trackConfirmHits,
    sharedConfig->trackDropMisses,
    sharedConfig->trackCoastMisses,
    sharedConfig->trackPositionAlpha,
    sharedConfig->trackSizeAlpha,
    sharedConfig->trackAreaVelAlpha,
    sharedConfig->trackAssocScoreMax,
    sharedConfig->trackAssocAreaWeight,
    sharedConfig->trackAssocAreaVelWeight);
border.enter.left = sharedConfig->observerBorderEnterLeft;
border.enter.right = sharedConfig->observerBorderEnterRight;
border.enter.top = sharedConfig->observerBorderEnterTop;
border.enter.bottom = sharedConfig->observerBorderEnterBottom;
border.exit.left = sharedConfig->observerBorderExitLeft;
border.exit.right = sharedConfig->observerBorderExitRight;
border.exit.top = sharedConfig->observerBorderExitTop;
border.exit.bottom = sharedConfig->observerBorderExitBottom;
beaconTrackSetBorderMargins(&border);
edge.holdMs = sharedConfig->edgePredictionHoldMs;
edge.bands.left = sharedConfig->observerEdgeBandLeft;
edge.bands.right = sharedConfig->observerEdgeBandRight;
edge.bands.top = sharedConfig->observerEdgeBandTop;
edge.bands.bottom = sharedConfig->observerEdgeBandBottom;
beaconTrackSetEdgePredictionConfig(&edge);
}

static uint8_t trackComesFirst(const beaconConfirmedStruct *left,
    const beaconConfirmedStruct *right)
{
    uint8_t leftRejected = (uint8_t)((left->flags &
        BEACON_CONFIRMED_FLAG_BORDER_REJECTED) != 0u);
    uint8_t rightRejected = (uint8_t)((right->flags &
        BEACON_CONFIRMED_FLAG_BORDER_REJECTED) != 0u);
    uint16_t leftSize;
    uint16_t rightSize;

    if (leftRejected != rightRejected) return leftRejected;
    if ((leftRejected != 0u) &&
        ((left->trackId == g_selectedTrackId) !=
            (right->trackId == g_selectedTrackId)))
        return (uint8_t)(left->trackId == g_selectedTrackId);
    if (left->fresh != right->fresh)
    {
        return left->fresh;
    }
    if (left->misses != right->misses)
    {
        return (uint8_t)(left->misses < right->misses);
    }
    leftSize = (left->fresh != 0u) ? left->size : left->predSize;
    rightSize = (right->fresh != 0u) ? right->size : right->predSize;
    if (leftSize != rightSize)
    {
        return (uint8_t)(leftSize > rightSize);
    }
    return (uint8_t)(left->trackId < right->trackId);
}

static void copyTrack(core3VisionTrack *destination,
    const beaconConfirmedStruct *source)
{
    destination->centerX = source->centerX;
    destination->centerY = source->centerY;
    destination->size = source->size;
    destination->trackId = source->trackId;
    destination->predX = source->predX;
    destination->predY = source->predY;
    destination->predSize = source->predSize;

```

```

destination->misses = source->misses;
destination->fresh = source->fresh;
destination->flags = source->flags;
destination->velX = source->velX;
destination->velY = source->velY;
destination->areaVel = source->areaVel;
}

static void publishObservation(const core3CameraSnapshot *frame)
{
    uint16_t order[OM_MAX_OBJECTS];
    uint16_t count = g_tracker.confirmedCount;
    uint16_t retained = 0u;
    uint16_t ordinaryRetained = 0u;
    uint16_t i;

    memset(&g_observation, 0, sizeof(g_observation));
    g_observation.detectMode = g_activeShared.detectMode;
    g_observation.sourceId = 3u;
    g_observation.maxBeacons = g_activeShared.maxBeacons;
    g_observation.edgeBands.left = g_activeShared.observerEdgeBandLeft;
    g_observation.edgeBands.right = g_activeShared.observerEdgeBandRight;
    g_observation.edgeBands.top = g_activeShared.observerEdgeBandTop;
    g_observation.edgeBands.bottom = g_activeShared.observerEdgeBandBottom;
    g_observation.border.enter.left = g_activeShared.observerBorderEnterLeft;
    g_observation.border.enter.right = g_activeShared.observerBorderEnterRight;
    g_observation.border.enter.top = g_activeShared.observerBorderEnterTop;
    g_observation.border.enter.bottom = g_activeShared.observerBorderEnterBottom;
    g_observation.border.exit.left = g_activeShared.observerBorderExitLeft;
    g_observation.border.exit.right = g_activeShared.observerBorderExitRight;
    g_observation.border.exit.top = g_activeShared.observerBorderExitTop;
    g_observation.border.exit.bottom = g_activeShared.observerBorderExitBottom;
    g_observation.sourceBootSession = g_sourceBootSession;
    g_observation.ownerSession = g_ownerSession;
    g_observation.configRevision = g_activeShared.revision;
    g_observation.configHash = g_activeShared.hash;
    g_observation.appFrameSeq = frame->appFrameSeq;
    g_observation.trackerFrameSeq = g_tracker.frameSeq;
    g_observation.tickMs = frame->tickMs;
    for (i = 0u; i < count; i++)
    {
        uint16_t position = i;

        order[i] = i;
        if ((g_tracker.confirmed[i].flags &
            BEACON_CONFIRMED_FLAG_BORDER_REJECTED) == 0u)
        {
            g_observation.confirmedTotal++;
            if (g_tracker.confirmed[i].fresh != 0u)
            {
                g_observation.freshTotal++;
                if (g_tracker.confirmed[i].centerX < (CORE3_VISION_WIDTH / 2u))
                    g_observation.freshLeft++;
                else
                    g_observation.freshRight++;
            }
        }
        while ((position > 0u) &&
            (trackComesFirst(&g_tracker.confirmed[order[position]],
                &g_tracker.confirmed[order[position - 1u]]) != 0u))
        {
            uint16_t swap = order[position - 1u];
            order[position - 1u] = order[position];
            order[position] = swap;
            position--;
        }
    }
    for (i = 0u; i < count; i++)
    {
        const beaconConfirmedStruct *source = &g_tracker.confirmed[order[i]];
        uint8_t rejected = (uint8_t) ((source->flags &
            BEACON_CONFIRMED_FLAG_BORDER_REJECTED) != 0u);
        if ((retained >= CORE3_VISION_MAX_PUBLISHED_TRACKS) ||
            ((rejected == 0u) &&
            (ordinaryRetained >= g_activeShared.maxBeacons)))
        {
            g_observation.truncated = 1u;
            continue;
        }
    }
}

```



```

        copyTrack(&g_observation.tracks[retained], source);
        retained++;
        if (rejected == 0u)
        {
            ordinaryRetained++;
            beaconTrackMarkPublished(&g_tracker, source->trackId);
        }
    }
    g_observation.retainedCount = retained;
    g_observation.valid = 1u;
}

uint8_t core3VisionPipelineInit(void)
{
    u8Image source;
    BEACONCC_CONFIG_STRUCT binaryConfig;

    memset(&g_status, 0, sizeof(g_status));
    memset(&g_activeShared, 0, sizeof(g_activeShared));
    memset(g_preview, 0, sizeof(g_preview));
    g_ownerSession = 0u;
    g_sourceBootSession = 0u;
    g_localRevision = 0u;
    source.ptr = g_preview;
    source.xLen = CORE3_VISION_WIDTH;
    source.yLen = CORE3_VISION_HEIGHT;
    source.size = CORE3_VISION_PIXELS;
    memset(&binaryConfig, 0, sizeof(binaryConfig));
    beaconCcInit(&binaryConfig);
    cannyInit(&source,
        CORE3_VISION_CANNY_SIGMA,
        0u,
        0u,
        CORE3_VISION_WIDTH - 1u,
        CORE3_VISION_HEIGHT - 1u,
        CORE3_VISION_LEGACY_CANNY_LOW_DEFAULT,
        CORE3_VISION_LEGACY_CANNY_HIGH_DEFAULT);
    clearTracking();
    g_status.ready = 1u;
    return 1u;
}

void core3VisionPipelineSetSelectedTrackId(uint16_t trackId)
{
    g_selectedTrackId = trackId;
}

uint8_t core3VisionPipelineApplyAtFrameBoundary(
    const core3VisionParamsStruct *localParams,
    uint32_t localRevision,
    const observerLinkSharedConfig *sharedConfig,
    uint32_t sourceBootSession,
    uint32_t ownerSession)
{
    uint8_t sharedChanged;
    uint8_t modeChanged;
    uint8_t ownerChanged;

    if ((g_status.ready == 0u) || (localParams == NULL) ||
        (sharedConfig == NULL) || (sourceBootSession == 0u) ||
        (ownerSession == 0u) ||
        (observerLinkSharedConfigIsValid(sharedConfig) == 0u))
    {
        return 0u;
    }
    sharedChanged = (uint8_t) ((g_status.applied == 0u) ||
        (g_activeShared.revision != sharedConfig->revision) ||
        (g_activeShared.hash != sharedConfig->hash));
    modeChanged = (uint8_t) ((g_status.applied != 0u) &&
        (g_activeShared.detectMode != sharedConfig->detectMode));
    ownerChanged = (uint8_t) ((g_status.applied != 0u) &&
        (g_ownerSession != ownerSession));
    if ((sharedChanged != 0u) || (g_localRevision != localRevision) ||
        (ownerChanged != 0u))
    {
        applyDetectorConfig(localParams, sharedConfig);
    }
    if ((modeChanged != 0u) || (ownerChanged != 0u))
    {

```

```

        clearTracking();
        if (modeChanged != 0u)
        {
        }
    }
    else if (sharedChanged != 0u)
    {
        g_observation.valid = 0u;
    }
    g_activeShared = *sharedConfig;
    g_sourceBootSession = sourceBootSession;
    g_ownerSession = ownerSession;
    g_localRevision = localRevision;
    g_status.applied = 1u;
    g_status.detectMode = sharedConfig->detectMode;
    return 1u;
}

uint8_t core3VisionPipelineProcess(const core3CameraSnapshot *frame)
{
    u8Image source;
    u8Image preview;
    u8Image gaussian;

    if ((g_status.applied == 0u) || (frame == NULL) ||
        (frame->data == NULL) || (frame->width != CORE3_VISION_WIDTH) ||
        (frame->height != CORE3_VISION_HEIGHT))
    {
        return 0u;
    }
    source.ptr = (uint8_t *) frame->data;
    source.xLen = frame->width;
    source.yLen = frame->height;
    source.size = CORE3_VISION_PIXELS;
    preview.ptr = g_preview;
    preview.xLen = frame->width;
    preview.yLen = frame->height;
    preview.size = CORE3_VISION_PIXELS;
    if (g_activeShared.detectMode == 1u)
    {
        gaussian.ptr = g_core3VisionWorkspace.scratch.canny.gaussian;
        gaussian.xLen = frame->width;
        gaussian.yLen = frame->height;
        gaussian.size = CORE3_VISION_PIXELS;
        cannyEdgeDetection(&source, &gaussian, &preview);
        (void) beaconDetect(&preview, &source, &g_detector);
    }
    else
    {
        (void) beaconCcDetect(&source, &preview, &g_detector);
    }
    beaconTrackUpdate(&g_tracker, &g_detector);
    publishObservation(frame);
    return 1u;
}

void core3VisionPipelineInvalidate(void)
{
    if (g_status.applied != 0u)
    {
        clearTracking();
    }
    g_status.applied = 0u;
    g_status.detectMode = 0u;
    memset(&g_activeShared, 0, sizeof(g_activeShared));
    g_sourceBootSession = 0u;
    g_ownerSession = 0u;
}

void core3VisionPipelineGetObservation(core3VisionObservation *observation)
{
    if (observation != NULL)
    {
        *observation = g_observation;
    }
}

void core3VisionPipelineGetStatus(core3VisionPipelineStatus *status)
{

```

```

    if (status != NULL)
    {
        *status = g_status;
    }
}

const uint8_t *core3VisionPipelineGetPreview(void)
{
    return g_preview;
}

```

core3_cyt4bb7/project/code/app/core3VisionPipeline.h

SHA-256

:

c9594b06e9b134ada8da43183272ae9782de522b35c2fe6f2e101309380734ca

代码 (c)

```

#ifndef CORE3_VISION_PIPELINE_H
#define CORE3_VISION_PIPELINE_H

#include <stdint.h>

#include "core3CameraService.h"
#include "observerLinkProtocol.h"
#include "core3VisionParams.h"
#include "beaconTrack.h"

#define CORE3_VISION_MAX_PUBLISHED_TRACKS (32u)

typedef struct
{
    uint16_t centerX;
    uint16_t centerY;
    uint16_t size;
    uint16_t trackId;
    uint16_t predX;
    uint16_t predY;
    uint16_t predSize;
    uint16_t misses;
    uint8_t fresh;
    uint8_t flags;
    float velX;
    float velY;
    float areaVel;
} core3VisionTrack;

typedef struct
{
    uint8_t valid;
    uint8_t sourceId;
    uint8_t detectMode;
    uint8_t maxBeacons;
    uint8_t truncated;
    beaconTrackEdgeBandsStruct edgeBands;
    beaconTrackBorderConfigStruct border;
    uint16_t confirmedTotal;
    uint16_t freshTotal;
    uint16_t freshLeft;
    uint16_t freshRight;
    uint16_t retainedCount;
    uint32_t sourceBootSession;
    uint32_t ownerSession;
    uint32_t configRevision;
    uint32_t configHash;
    uint32_t appFrameSeq;
    uint32_t trackerFrameSeq;
    uint32_t tickMs;
    core3VisionTrack tracks[CORE3_VISION_MAX_PUBLISHED_TRACKS];
} core3VisionObservation;

typedef struct
{
    uint8_t ready;
    uint8_t applied;
}

```

```

uint8_t detectMode;
uint8_t padding;
} core3VisionPipelineStatus;

uint8_t core3VisionPipelineInit(void);
uint8_t core3VisionPipelineApplyAtFrameBoundary(
    const core3VisionParamsStruct *localParams,
    uint32_t localRevision,
    const observerLinkSharedConfig *sharedConfig,
    uint32_t sourceBootSession,
    uint32_t ownerSession);
uint8_t core3VisionPipelineProcess(const core3CameraSnapshot *frame);
void core3VisionPipelineSetSelectedTrackId(uint16_t trackId);
void core3VisionPipelineInvalidate(void);
void core3VisionPipelineGetObservation(core3VisionObservation *observation);
void core3VisionPipelineGetStatus(core3VisionPipelineStatus *status);
const uint8_t *core3VisionPipelineGetPreview(void);

#endif /* CORE3_VISION_PIPELINE_H */

```

core3_cyt4bb7/project/code/app/observerKvmBindingGuard.c

SHA-256

:

f103f3edf8549e101a9f14a6722be8922442d65196a6e196336f889c5600c3e5

代码 (c)

```

#include "observerKvmBindingGuard.h"

#include <string.h>

void observerKvmBindingGuardInit(observerKvmBindingGuard *guard)
{
    if (guard != 0) memset(guard, 0, sizeof(*guard));
}

observerKvmBindingAction observerKvmBindingGuardObserve(
    observerKvmBindingGuard *guard,
    uint32_t ownerSession,
    uint32_t bindingEpoch)
{
    if (guard == 0) return OBSERVER_KVM_BINDING_REBOUND;
    if (guard->primed == 0u)
    {
        guard->ownerSession = ownerSession;
        guard->bindingEpoch = bindingEpoch;
        guard->primed = 1u;
        return OBSERVER_KVM_BINDING_INITIAL;
    }
    if ((guard->ownerSession == ownerSession) &&
        (guard->bindingEpoch == bindingEpoch))
        return OBSERVER_KVM_BINDING_UNCHANGED;
    guard->ownerSession = ownerSession;
    guard->bindingEpoch = bindingEpoch;
    return OBSERVER_KVM_BINDING_REBOUND;
}

```

core3_cyt4bb7/project/code/app/observerKvmBindingGuard.h

SHA-256

:

92b8c965b3d809d4fe13be626ccaf26b8fa8ba4e83cf69bfc626d52893f00b5f

代码 (c)

```

#ifndef OBSERVER_KVM_BINDING_GUARD_H
#define OBSERVER_KVM_BINDING_GUARD_H

#include <stdint.h>

typedef enum

```

```

{
    OBSERVER_KVM_BINDING_UNCHANGED = 0u,
    OBSERVER_KVM_BINDING_INITIAL,
    OBSERVER_KVM_BINDING_REBOUND
} observerKvmBindingAction;

typedef struct
{
    uint32_t ownerSession;
    uint32_t bindingEpoch;
    uint8_t primed;
} observerKvmBindingGuard;

void observerKvmBindingGuardInit(observerKvmBindingGuard *guard);
observerKvmBindingAction observerKvmBindingGuardObserve(
    observerKvmBindingGuard *guard,
    uint32_t ownerSession,
    uint32_t bindingEpoch);

#endif /* OBSERVER_KVM_BINDING_GUARD_H */

```

core3_cyt4bb7/project/code/comm/core3CoreDataTransfer.c

SHA-256

:

f181fa92fe9713fbb863927b2ab3285fc5c367df024126929edfcfa3170b40b5

代码 (c)

```

#include "core3CoreDataTransfer.h"

#include <string.h>

#include "cy_device_headers.h"

/* Core-local live snapshot. It is deliberately not placed in shared RAM and
 * is only exposed to business code through the copy-out getter. */
static core3CoreDataDiagnostics g_core3CoreDataDiagnosticsLive;

static uint8_t core3CoreSequenceIsNewer(uint32_t candidate,
    uint32_t reference)
{
    uint32_t delta = candidate - reference;
    return (uint8_t) ((delta != 0u) && (delta < 0x80 000 000u));
}

static void saturatingIncrement(uint32_t *value)
{
    if (*value != UINT32_MAX) (*value)++;
}

static void saturatingAdd(uint32_t *value, uint32_t amount)
{
    if ((UINT32_MAX - *value) < amount) *value = UINT32_MAX;
    else *value += amount;
}

static void saturatingAdd64(uint64_t *value, uint32_t amount)
{
    if ((UINT64_MAX - *value) < amount) *value = UINT64_MAX;
    else *value += amount;
}

void core3CoreDataTimingInit(void)
{
    uint32_t before;

    if ((g_core3CoreDataDiagnosticsLive.timingInitialized != 0u) &&
        (g_core3CoreDataDiagnosticsLive.timingValid != 0u)) return;
    g_core3CoreDataDiagnosticsLive.timingInitialized = 1u;
    g_core3CoreDataDiagnosticsLive.timingValid = 0u;
    if ((DWT->CTRL & DWT_CTRL_NOCYCNT_Msk) != 0u) return;

    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    if ((DWT->CTRL & DWT_CTRL_CYCCNTENA_Msk) == 0u)
    {

```

```

    DWT->CYCCNT = 0u;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}
__DSB();
__ISB();
before = DWT->CYCCNT;
__NOP();
__NOP();
__NOP();
__NOP();
if (((DWT->CTRL & DWT_CTRL_CYCCNTENA_Msk) != 0u) &&
    (DWT->CYCCNT != before))
{
    g_core3CoreDataDiagnosticsLive.timingValid = 1u;
}
}

static uint8_t timingBegin(uint32_t *startCycles)
{
    if ((startCycles == NULL) ||
        (g_core3CoreDataDiagnosticsLive.timingValid == 0u)) return 0u;
    if ((DWT->CTRL & DWT_CTRL_CYCCNTENA_Msk) == 0u)
    {
        g_core3CoreDataDiagnosticsLive.timingValid = 0u;
        return 0u;
    }
    *startCycles = DWT->CYCCNT;
    return 1u;
}

static void timingEnd(uint8_t valid, uint32_t startCycles)
{
    uint32_t elapsed;

    if (valid == 0u)
    {
        saturatingIncrement(
            &g_core3CoreDataDiagnosticsLive.timingInvalidSampleCount);
        return;
    }
    elapsed = DWT->CYCCNT - startCycles;
    if (((DWT->CTRL & DWT_CTRL_CYCCNTENA_Msk) == 0u) || (elapsed == 0u))
    {
        g_core3CoreDataDiagnosticsLive.timingValid = 0u;
        saturatingIncrement(
            &g_core3CoreDataDiagnosticsLive.timingInvalidSampleCount);
        return;
    }
    saturatingIncrement(&g_core3CoreDataDiagnosticsLive.timingSampleCount);
    saturatingAdd64(&g_core3CoreDataDiagnosticsLive.timingTotalCycles, elapsed);
    if (g_core3CoreDataDiagnosticsLive.timingMaxCycles < elapsed)
    {
        g_core3CoreDataDiagnosticsLive.timingMaxCycles = elapsed;
    }
}

/* Each image owns a cache-line-isolated retained pair in core-local RAM. */
typedef struct
{
    uint32_t counter;
    uint32_t counterInverse;
    uint8_t padding[24];
} core3CoreBootState;

#pragma data_alignment = 32
__no_init static core3CoreBootState core3CoreBootStateRetained;

#pragma location = CORE3_CORE_LINK_REGION_ADDR
__no_init volatile core3CoreLinkSnapshot core3CoreLinkRegion;

#pragma location = CORE3_CORE_VISION_REGION_ADDR
__no_init volatile core3CoreVisionSnapshot core3CoreVisionRegion;

#define CORE3_CORE_QUEUE_DEPTH (4u)
#define CORE3_CORE_QUEUE_MASK (CORE3_CORE_QUEUE_DEPTH - 1u)

typedef struct
{
    uint32_t producerBootGeneration;

```

```

    uint32_t sequence;
    uint32_t payloadBytes;
    uint32_t commit;
    uint8_t payload[112];
} core3CoreCommittedSlot;

typedef struct
{
    volatile uint32_t producerBootGeneration;
    volatile uint32_t head;
    uint8_t producerPadding[24];
    volatile uint32_t consumerBootGeneration;
    volatile uint32_t tail;
    uint8_t consumerPadding[24];
    core3CoreCommittedSlot slots[CORE3_CORE_QUEUE_DEPTH];
} core3CoreQueueRegion;

#define CORE3_CORE_QUEUE_COMMIT (0xC35A0000u)

typedef char core3Core_queue_depth_assert[
    ((CORE3_CORE_QUEUE_DEPTH & CORE3_CORE_QUEUE_MASK) == 0u) ? 1 : -1];
typedef char core3Core_queue_region_size_assert[
    (sizeof(core3CoreQueueRegion) == 0x240u) ? 1 : -1];
typedef char core3Core_queue_slot_size_assert[
    (sizeof(core3CoreCommittedSlot) == 128u) ? 1 : -1];
typedef char core3Core_ingress_slot_size_assert[
    (sizeof(core3CoreIngressEvent) <= 112u) ? 1 : -1];
typedef char core3Core_egress_slot_size_assert[
    (sizeof(core3CoreEgressEvent) <= 112u) ? 1 : -1];

#pragma location = CORE3_CORE_INGRESS_QUEUE_ADDR
__no_init volatile core3CoreQueueRegion core3CoreIngressQueue;

#pragma location = CORE3_CORE_EGRESS_QUEUE_ADDR
__no_init volatile core3CoreQueueRegion core3CoreEgressQueue;

static uint8_t snapshotStable(const void *first, const void *second,
    uint32_t size, uint32_t magic, uint32_t begin, uint32_t end)
{
    const core3CoreLinkSnapshot *header = (const core3CoreLinkSnapshot *) second;
    return (uint8_t) ((memcmp(first, second, size) == 0) &&
        (header->magic == magic) &&
        (header->abiVersion == CORE3_CORE_ABI_VERSION) &&
        (header->payloadBytes == size) &&
        (begin != 0u) && (begin == end));
}

static uint8_t probeSnapshot(volatile core3CoreLinkSnapshot *region,
    uint32_t magic, uint32_t payloadBytes, uint32_t *bootGeneration,
    uint32_t *publishSeq, uint32_t *probeCount)
{
    uint32_t begin;
    uint32_t observedMagic;
    uint32_t abiVersion;
    uint32_t observedBytes;
    uint32_t boot;
    uint32_t seq;

    if ((bootGeneration == NULL) || (publishSeq == NULL)) return 0u;
    saturatingIncrement(probeCount);
    SCB_InvalidateDCache_by_Addr((volatile void *) region, 32u);
    begin = region->beginGeneration;
    observedMagic = region->magic;
    abiVersion = region->abiVersion;
    observedBytes = region->payloadBytes;
    boot = region->bootGeneration;
    seq = region->publishSeq;
    if ((begin == 0u) || (observedMagic != magic) ||
        (abiVersion != CORE3_CORE_ABI_VERSION) ||
        (observedBytes != payloadBytes) || (boot == 0u) || (seq == 0u))
    {
        return 0u;
    }
    *bootGeneration = boot;
    *publishSeq = seq;
    return 1u;
}

uint32_t core3CoreDataMakeBootGeneration(uint32_t salt)

```

```

{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    uint32_t generation;

    SCB_InvalidateDCache_by_Addr(
        (volatile void *) &core3CoreBootStateRetained,
        sizeof(core3CoreBootStateRetained));
    if (core3CoreBootStateRetained.counterInverse !=
        ~core3CoreBootStateRetained.counter)
    {
        core3CoreBootStateRetained.counter =
            DWT->CYCCNT ^ salt ^ 0xD2B79F5u;
    }
    core3CoreBootStateRetained.counter++;
    if (core3CoreBootStateRetained.counter == 0u)
    {
        core3CoreBootStateRetained.counter = 1u;
    }
    core3CoreBootStateRetained.counterInverse =
        ~core3CoreBootStateRetained.counter;
    SCB_CleanDCache_by_Addr(
        (volatile void *) &core3CoreBootStateRetained,
        sizeof(core3CoreBootStateRetained));
    __DSB();
    generation = core3CoreBootStateRetained.counter;
    timingEnd(timingValid, startCycles);
    return generation;
}

uint8_t core3CoreDataProbeLink(uint32_t *bootGeneration, uint32_t *publishSeq)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    uint8_t result = probeSnapshot(&core3CoreLinkRegion, CORE3_CORE_LINK_MAGIC,
        sizeof(core3CoreLinkRegion), bootGeneration, publishSeq,
        &g_core3CoreDataDiagnosticsLive.linkProbeCount);
    timingEnd(timingValid, startCycles);
    return result;
}

uint8_t core3CoreDataProbeVision(uint32_t *bootGeneration, uint32_t *publishSeq)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    uint8_t result = probeSnapshot(
        (volatile core3CoreLinkSnapshot *) &core3CoreVisionRegion,
        CORE3_CORE_VISION_MAGIC, sizeof(core3CoreVisionRegion), bootGeneration,
        publishSeq, &g_core3CoreDataDiagnosticsLive.visionProbeCount);
    timingEnd(timingValid, startCycles);
    return result;
}

void core3CoreDataWriteLink(const core3CoreLinkSnapshot *snapshot)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    if (snapshot == NULL)
    {
        timingEnd(timingValid, startCycles);
        return;
    }
    memcpy((void *) &core3CoreLinkRegion, snapshot, sizeof(*snapshot));
    __DMB();
    SCB_CleanDCache_by_Addr((volatile void *) &core3CoreLinkRegion,
        sizeof(core3CoreLinkRegion));
    __DSB();
    saturatingIncrement(&g_core3CoreDataDiagnosticsLive.linkPublishCount);
    saturatingAdd(&g_core3CoreDataDiagnosticsLive.linkPublishBytes,
        sizeof(core3CoreLinkRegion));
    timingEnd(timingValid, startCycles);
}

uint8_t core3CoreDataTryReadLink(core3CoreLinkSnapshot *snapshot)
{
    core3CoreLinkSnapshot first;
    core3CoreLinkSnapshot second;
    uint8_t attempt;
    uint8_t result = 0u;

```



```

uint32_t startCycles;
uint8_t timingValid = timingBegin(&startCycles);
if (snapshot == NULL)
{
    timingEnd(timingValid, startCycles);
    return 0u;
}
saturatingIncrement(&g_core3CoreDataDiagnosticsLive.linkReadAttempts);
for (attempt = 0u; attempt < 3u; attempt++)
{
    SCB_InvalidateDCache_by_Addr((volatile void *) &core3CoreLinkRegion,
        sizeof(core3CoreLinkRegion));
    memcpy(&first, (const void *) &core3CoreLinkRegion, sizeof(first));
    SCB_InvalidateDCache_by_Addr((volatile void *) &core3CoreLinkRegion,
        sizeof(core3CoreLinkRegion));
    memcpy(&second, (const void *) &core3CoreLinkRegion, sizeof(second));
    if (snapshotStable(&first, &second, sizeof(second), CORE3_CORE_LINK_MAGIC,
        second.beginGeneration, second.endGeneration))
    {
        *snapshot = second;
        result = 1u;
        break;
    }
    saturatingIncrement(&g_core3CoreDataDiagnosticsLive.linkReadRetries);
}
if (result == 0u)
{
    saturatingIncrement(&g_core3CoreDataDiagnosticsLive.linkReadFailures);
}
timingEnd(timingValid, startCycles);
return result;
}

void core3CoreDataWriteVision(const core3CoreVisionSnapshot *snapshot)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    if (snapshot == NULL)
    {
        timingEnd(timingValid, startCycles);
        return;
    }
    memcpy((void *) &core3CoreVisionRegion, snapshot, sizeof(*snapshot));
    __DMB();
    SCB_CleanDCache_by_Addr((volatile void *) &core3CoreVisionRegion,
        sizeof(core3CoreVisionRegion));
    __DSB();
    saturatingIncrement(&g_core3CoreDataDiagnosticsLive.visionPublishCount);
    saturatingAdd(&g_core3CoreDataDiagnosticsLive.visionPublishBytes,
        sizeof(core3CoreVisionRegion));
    timingEnd(timingValid, startCycles);
}

uint8_t core3CoreDataTryReadVision(core3CoreVisionSnapshot *snapshot)
{
    core3CoreVisionSnapshot first;
    core3CoreVisionSnapshot second;
    uint8_t attempt;
    uint8_t result = 0u;
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    if (snapshot == NULL)
    {
        timingEnd(timingValid, startCycles);
        return 0u;
    }
    saturatingIncrement(&g_core3CoreDataDiagnosticsLive.visionReadAttempts);
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        SCB_InvalidateDCache_by_Addr((volatile void *) &core3CoreVisionRegion,
            sizeof(core3CoreVisionRegion));
        memcpy(&first, (const void *) &core3CoreVisionRegion, sizeof(first));
        SCB_InvalidateDCache_by_Addr((volatile void *) &core3CoreVisionRegion,
            sizeof(core3CoreVisionRegion));
        memcpy(&second, (const void *) &core3CoreVisionRegion, sizeof(second));
        if (snapshotStable(&first, &second, sizeof(second), CORE3_CORE_VISION_MAGIC,
            second.beginGeneration, second.endGeneration))
        {
            *snapshot = second;

```

```

        result = 1u;
        break;
    }
    saturatingIncrement(&g_core3CoreDataDiagnosticsLive.visionReadRetries);
}
if (result == 0u)
{
    saturatingIncrement(&g_core3CoreDataDiagnosticsLive.visionReadFailures);
}
timingEnd(timingValid, startCycles);
return result;
}

static uint8_t queuePush(volatile core3CoreQueueRegion *queue,
    const void *event, uint32_t eventSize, uint32_t *fullCount,
    uint32_t *highWater)
{
    uint32_t head;
    uint32_t tail;
    uint32_t producerBootGeneration;
    volatile core3CoreCommittedSlot *slot;
    SCB_InvalidateDCache_by_Addr(
        (volatile void *) &queue->consumerBootGeneration, 32u);
    producerBootGeneration = queue->producerBootGeneration;
    if (producerBootGeneration == 0u) return 0u;
    head = queue->head;
    tail = (queue->consumerBootGeneration == producerBootGeneration) ?
        queue->tail : head;
    if ((uint32_t) (head - tail) >= CORE3_CORE_QUEUE_DEPTH)
    {
        saturatingIncrement(fullCount);
        return 0u;
    }
    slot = &queue->slots[head & CORE3_CORE_QUEUE_MASK];
    memset((void *) slot, 0, sizeof(*slot));
    slot->producerBootGeneration = producerBootGeneration;
    slot->sequence = head + 1u;
    slot->payloadBytes = eventSize;
    memcpy((void *) slot->payload, event, eventSize);
    SCB_CleanDCache_by_Addr((volatile void *) slot, sizeof(*slot));
    __DSB();
    slot->commit = CORE3_CORE_QUEUE_COMMIT ^ producerBootGeneration ^ (head + 1u);
    SCB_CleanDCache_by_Addr((volatile void *) slot, 32u);
    __DSB();
    queue->head = head + 1u;
    SCB_CleanDCache_by_Addr(
        (volatile void *) &queue->producerBootGeneration, 32u);
    __DSB();
    if (*highWater < ((uint32_t) (head - tail) + 1u))
    {
        *highWater = (uint32_t) (head - tail) + 1u;
    }
    return 1u;
}

static void queueReset(volatile core3CoreQueueRegion *queue,
    uint32_t producerBootGeneration)
{
    queue->head = 0u;
    queue->producerBootGeneration = producerBootGeneration;
    SCB_CleanDCache_by_Addr(
        (volatile void *) &queue->producerBootGeneration, 32u);
    __DSB();
}

static void queueSyncConsumer(volatile core3CoreQueueRegion *queue,
    uint32_t producerBootGeneration)
{
    SCB_InvalidateDCache_by_Addr(
        (volatile void *) &queue->producerBootGeneration, 32u);
    if (queue->producerBootGeneration != producerBootGeneration) return;
    queue->tail = queue->head;
    queue->consumerBootGeneration = producerBootGeneration;
    SCB_CleanDCache_by_Addr(
        (volatile void *) &queue->consumerBootGeneration, 32u);
    __DSB();
}

void core3CoreDataResetIngress(uint32_t producerBootGeneration)

```

```

{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    queueReset(&core3CoreIngressQueue, producerBootGeneration);
    timingEnd(timingValid, startCycles);
}

void core3CoreDataResetEgress(uint32_t producerBootGeneration)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    queueReset(&core3CoreEgressQueue, producerBootGeneration);
    timingEnd(timingValid, startCycles);
}

void core3CoreDataSyncIngressConsumer(uint32_t producerBootGeneration)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    queueSyncConsumer(&core3CoreIngressQueue, producerBootGeneration);
    timingEnd(timingValid, startCycles);
}

void core3CoreDataSyncEgressConsumer(uint32_t producerBootGeneration)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    queueSyncConsumer(&core3CoreEgressQueue, producerBootGeneration);
    timingEnd(timingValid, startCycles);
}

static uint8_t queuePop(volatile core3CoreQueueRegion *queue,
    void *event, uint32_t eventSize, uint32_t producerBootGeneration)
{
    uint32_t head;
    uint32_t tail;
    uint32_t expectedCommit;
    volatile core3CoreCommittedSlot *slot;
    SCB_InvalidateDCache_by_Addr(
        (volatile void *) &queue->producerBootGeneration, 32u);
    if ((queue->producerBootGeneration != producerBootGeneration) ||
        (queue->consumerBootGeneration != producerBootGeneration)) return 0u;
    head = queue->head;
    tail = queue->tail;
    if (tail == head) return 0u;
    if (((uint32_t) (head - tail) > CORE3_CORE_QUEUE_DEPTH) return 0u;
    slot = &queue->slots[tail & CORE3_CORE_QUEUE_MASK];
    SCB_InvalidateDCache_by_Addr((volatile void *) slot, sizeof(*slot));
    expectedCommit = CORE3_CORE_QUEUE_COMMIT ^ producerBootGeneration ^
        (tail + 1u);
    if ((slot->producerBootGeneration != producerBootGeneration) ||
        (slot->sequence != (tail + 1u)) ||
        (slot->payloadBytes != eventSize) ||
        (slot->commit != expectedCommit)) return 0u;
    memcpy(event, (const void *) slot->payload, eventSize);
    __DMB();
    queue->tail = tail + 1u;
    SCB_CleanDCache_by_Addr(
        (volatile void *) &queue->consumerBootGeneration, 32u);
    __DSB();
    return 1u;
}

uint8_t core3CoreDataPushIngress(const core3CoreIngressEvent *event)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    uint8_t result = 0u;
    if (event != NULL)
    {
        result = queuePush(&core3CoreIngressQueue, event, sizeof(*event),
            &g_core3CoreDataDiagnosticsLive.ingressQueueFull,
            &g_core3CoreDataDiagnosticsLive.ingressQueueHighWater);
    }
    timingEnd(timingValid, startCycles);
    return result;
}

uint8_t core3CoreDataPopIngress(core3CoreIngressEvent *event,

```

```

uint32_t producerBootGeneration)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    uint8_t result = 0u;
    if (event != NULL)
    {
        result = queuePop(&core3CoreIngressQueue, event, sizeof(*event),
            producerBootGeneration);
    }
    timingEnd(timingValid, startCycles);
    return result;
}

uint8_t core3CoreDataPushEgress(const core3CoreEgressEvent *event)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    uint8_t result = 0u;
    if (event != NULL)
    {
        result = queuePush(&core3CoreEgressQueue, event, sizeof(*event),
            &g_core3CoreDataDiagnosticsLive.egressQueueFull,
            &g_core3CoreDataDiagnosticsLive.egressQueueHighWater);
    }
    timingEnd(timingValid, startCycles);
    return result;
}

uint8_t core3CoreDataPopEgress(core3CoreEgressEvent *event,
    uint32_t producerBootGeneration)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    uint8_t result = 0u;
    if (event != NULL)
    {
        result = queuePop(&core3CoreEgressQueue, event, sizeof(*event),
            producerBootGeneration);
    }
    timingEnd(timingValid, startCycles);
    return result;
}

uint8_t core3CoreBorderRejectionMerge(
    const core3CoreBorderRejection *pending,
    observerLinkPlanningTrackSnapshot *snapshot)
{
    uint8_t delivered = 0u;
    uint8_t i;
    if ((pending == NULL) || (snapshot == NULL) ||
        (pending->rejectedCount == 0u) ||
        (snapshot->ownerSession != pending->ownerSession) ||
        (snapshot->revision != pending->revision) ||
        (snapshot->hash != pending->hash) ||
        ((snapshot->trackerFrameSeq != pending->trackerFrameSeq) &&
        (core3CoreSequencesIsNewer(snapshot->trackerFrameSeq,
            pending->trackerFrameSeq) == 0u))) return 0u;
    for (i = 0u; i < pending->rejectedCount; i++)
    {
        uint16_t rejectedId = pending->rejectedTrackIds[i];
        uint8_t j;
        uint8_t replacement = OBSERVER_LINK_MAX_TRACKS;
        for (j = 0u; j < snapshot->retainedCount; j++)
        {
            if (snapshot->tracks[j].trackId == rejectedId)
            {
                memset(&snapshot->tracks[j], 0,
                    sizeof(snapshot->tracks[j]));
                snapshot->tracks[j].trackId = rejectedId;
                snapshot->tracks[j].flags =
                    OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED;
                break;
            }
        }
        if ((snapshot->tracks[j].flags &
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) == 0u)
            replacement = j;
    }
    if (j == snapshot->retainedCount)

```

```

    {
        if (snapshot->retainedCount < OBSERVER_LINK_MAX_TRACKS)
        {
            j = snapshot->retainedCount++;
        }
        else if (replacement < OBSERVER_LINK_MAX_TRACKS)
        {
            j = replacement;
            snapshot->truncated = 1u;
        }
        else
        {
            break;
        }
        memset(&snapshot->tracks[j], 0, sizeof(snapshot->tracks[j]));
        snapshot->tracks[j].trackId = rejectedId;
        snapshot->tracks[j].flags =
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED;
    }
    delivered++;
}
return delivered;
}

uint8_t core3CoreBorderRejectionAppend(core3CoreBorderRejection *pending,
uint8_t *pendingValid, const core3CoreBorderRejection *event)
{
    uint8_t uniqueAdded = 0u;
    uint8_t i;
    if ((pending == NULL) || (pendingValid == NULL) || (event == NULL) ||
        (event->ownerSession == 0u) || (event->rejectedCount == 0u) ||
        (event->rejectedCount > CORE3_CORE_REJECTION_TRACK_MAX)) return 0u;
    for (i = 0u; i < event->rejectedCount; i++)
    {
        uint8_t j;
        if (event->rejectedTrackIds[i] == 0u) return 0u;
        for (j = 0u; j < i; j++)
            if (event->rejectedTrackIds[j] ==
                event->rejectedTrackIds[i]) return 0u;
    }
    if ((*pendingValid == 0u) ||
        (pending->ownerSession != event->ownerSession) ||
        (pending->revision != event->revision) ||
        (pending->hash != event->hash))
    {
        *pending = *event;
        *pendingValid = 1u;
        return 1u;
    }
    for (i = 0u; i < event->rejectedCount; i++)
    {
        uint8_t j;
        for (j = 0u; j < pending->rejectedCount; j++)
            if (pending->rejectedTrackIds[j] ==
                event->rejectedTrackIds[i]) break;
        if (j == pending->rejectedCount) uniqueAdded++;
    }
    if ((uint16_t) pending->rejectedCount + uniqueAdded >
        CORE3_CORE_REJECTION_TRACK_MAX) return 0u;
    for (i = 0u; i < event->rejectedCount; i++)
    {
        uint8_t j;
        for (j = 0u; j < pending->rejectedCount; j++)
            if (pending->rejectedTrackIds[j] ==
                event->rejectedTrackIds[i]) break;
        if (j == pending->rejectedCount)
            pending->rejectedTrackIds[pending->rejectedCount++] =
                event->rejectedTrackIds[i];
    }
    if (core3CoreSequencesIsNewer(event->trackerFrameSeq,
        pending->trackerFrameSeq) != 0u)
        pending->trackerFrameSeq = event->trackerFrameSeq;
    return 1u;
}

uint8_t core3CorePlanningPreserveRejections(
    observerLinkPlanningTrackSnapshot *destination,
    const observerLinkPlanningTrackSnapshot *pending)
{

```

```

uint8_t i;
if ((destination == NULL) || (pending == NULL)) return 0u;
for (i = 0u; i < pending->retainedCount; i++)
{
    const observerLinkPlanningTrack *rejected = &pending->tracks[i];
    uint8_t j;
    uint8_t replacement = OBSERVER_LINK_MAX_TRACKS;
    if ((rejected->flags &
        OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) == 0u) continue;
    for (j = 0u; j < destination->retainedCount; j++)
    {
        if (destination->tracks[j].trackId == rejected->trackId)
        {
            destination->tracks[j] = *rejected;
            replacement = j;
            break;
        }
        if ((destination->tracks[j].flags &
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) == 0u)
            replacement = j;
    }
    if (j < destination->retainedCount) continue;
    if (destination->retainedCount < OBSERVER_LINK_MAX_TRACKS)
        destination->tracks[destination->retainedCount++] = *rejected;
    else if (replacement < OBSERVER_LINK_MAX_TRACKS)
    {
        destination->tracks[replacement] = *rejected;
        destination->truncated = 1u;
    }
    else return 0u;
}
return 1u;
}

void core3CoreDataNoteSharedStale(void)
{
    saturatingIncrement(&g_core3CoreDataDiagnosticsLive.sharedStaleCount);
}

void core3CoreDataNoteSessionReset(void)
{
    saturatingIncrement(&g_core3CoreDataDiagnosticsLive.sessionResetCount);
}

```

core3_cyt4bb7/project/code/comm/core3CoreDataTransfer.h

SHA-256

:

d265b14198f7ac041a336a8d64291a7d0132653909d907b88b699e2c64177de1

代码 (c)

```

#ifndef CORE3_CORE_DATA_TRANSFER_H
#define CORE3_CORE_DATA_TRANSFER_H

#include <stdint.h>

#include "core3LinkCore3.h"
#include "paramSyncMailbox.h"

#define CORE3_CORE_LINK_REGION_ADDR (0x28 001 000u)
#define CORE3_CORE_INGRESS_QUEUE_ADDR (0x28 001 400u)
#define CORE3_CORE_EGRESS_QUEUE_ADDR (0x28 001 800u)
#define CORE3_CORE_VISION_REGION_ADDR (0x28 002 000u)
#define CORE3_CORE_LINK_LIMIT_BYTES (0x0400u)
#define CORE3_CORE_QUEUE_LIMIT_BYTES (0x0400u)
#define CORE3_CORE_VISION_LIMIT_BYTES (0x1000u)
#define CORE3_CORE_LINK_MAGIC (0x43 334C4Bu) /* C3LK */
#define CORE3_CORE_VISION_MAGIC (0x43 335 649u) /* C3VI */
#define CORE3_CORE_ABI_VERSION (7u)

typedef struct
{
    uint32_t beginGeneration;
    uint32_t magic;
    uint32_t abiVersion;
}

```

```

uint32_t payloadBytes;
uint32_t bootGeneration;
uint32_t publishSeq;
core3LinkCore3Status status;
observerLinkSharedConfig stagedConfig;
uint32_t stagedConfigSeq;
uint32_t appliedConsumedSeq;
uint32_t paramGlobalStateSeq;
paramSyncGlobalState paramGlobalState;
uint32_t endGeneration;
} core3CoreLinkSnapshot;

typedef struct
{
    uint32_t beginGeneration;
    uint32_t magic;
    uint32_t abiVersion;
    uint32_t payloadBytes;
    uint32_t bootGeneration;
    uint32_t publishSeq;
    uint32_t visionReadySeq;
    uint8_t visionReady;
    uint8_t padding0[3];
    uint32_t appliedSeq;
    uint32_t appliedOwnerSession;
    uint32_t appliedRevision;
    uint32_t appliedHash;
    uint8_t appliedMode;
    uint8_t padding1[3];
    uint32_t planningSeq;
    observerLinkPlanningTrackSnapshot planningSnapshot;
    uint32_t healthSeq;
    observerLinkHealth health;
    uint32_t paramGlobalRequestSeq;
    paramSyncGlobalRequest paramGlobalRequest;
    uint32_t paramGlobalStateConsumedSeq;
    uint32_t endGeneration;
} core3CoreVisionSnapshot;

typedef enum
{
    CORE3_CORE_INGRESS_KVM_CONTROL = 1,
    CORE3_CORE_INGRESS_KVM_EVENT = 2
} core3CoreIngressKind;

typedef struct
{
    uint32_t kind;
    union
    {
        observerLinkKvmControl control;
        observerLinkKvmEvent event;
    } payload;
} core3CoreIngressEvent;

typedef enum
{
    CORE3_CORE_EGRESS_KVM_ACK = 1,
    CORE3_CORE_EGRESS_KVM_MENU = 2,
    CORE3_CORE_EGRESS_BORDER_REJECTION = 3
} core3CoreEgressKind;

#define CORE3_CORE_REJECTION_TRACK_MAX (44u)

typedef struct
{
    uint32_t ownerSession;
    uint32_t revision;
    uint32_t hash;
    uint32_t trackerFrameSeq;
    uint8_t rejectedCount;
    uint8_t padding[3];
    uint16_t rejectedTrackIds[CORE3_CORE_REJECTION_TRACK_MAX];
} core3CoreBorderRejection;

typedef struct
{
    uint32_t kind;
    union

```

```

    {
        observerLinkKvmAck ack;
        observerLinkKvmMenuStatus menuStatus;
        core3CoreBorderRejection borderRejection;
    } payload;
} core3CoreEgressEvent;

```

```

typedef struct
{
    uint8_t timingInitialized;
    uint8_t timingValid;
    uint8_t timingPadding[2];
    uint32_t timingSampleCount;
    uint32_t timingInvalidSampleCount;
    uint64_t timingTotalCycles;
    uint32_t timingMaxCycles;
    uint32_t linkProbeCount;
    uint32_t visionProbeCount;
    uint32_t linkReadAttempts;
    uint32_t linkReadRetries;
    uint32_t linkReadFailures;
    uint32_t visionReadAttempts;
    uint32_t visionReadRetries;
    uint32_t visionReadFailures;
    uint32_t linkPublishCount;
    uint32_t linkPublishBytes;
    uint32_t visionPublishCount;
    uint32_t visionPublishBytes;
    uint32_t ingressQueueFull;
    uint32_t ingressQueueHighWater;
    uint32_t egressQueueFull;
    uint32_t egressQueueHighWater;
    uint32_t sharedStaleCount;
    uint32_t sessionResetCount;
} core3CoreDataDiagnostics;

```

```

typedef char core3Core_link_region_size_assert[
    (sizeof(core3CoreLinkSnapshot) <= CORE3_CORE_LINK_LIMIT_BYTES) ? 1 : -1];
typedef char core3Core_vision_region_size_assert[
    (sizeof(core3CoreVisionSnapshot) <= CORE3_CORE_VISION_LIMIT_BYTES) ? 1 : -1];
typedef char core3Core_link_region_align_assert[
    ((CORE3_CORE_LINK_REGION_ADDR & 31u) == 0u) ? 1 : -1];
typedef char core3Core_ingress_region_align_assert[
    ((CORE3_CORE_INGRESS_QUEUE_ADDR & 31u) == 0u) ? 1 : -1];
typedef char core3Core_egress_region_align_assert[
    ((CORE3_CORE_EGRESS_QUEUE_ADDR & 31u) == 0u) ? 1 : -1];
typedef char core3Core_vision_region_align_assert[
    ((CORE3_CORE_VISION_REGION_ADDR & 31u) == 0u) ? 1 : -1];
typedef char core3Core_link_before_ingress_assert[
    ((CORE3_CORE_LINK_REGION_ADDR + CORE3_CORE_LINK_LIMIT_BYTES) <=
    CORE3_CORE_INGRESS_QUEUE_ADDR) ? 1 : -1];
typedef char core3Core_ingress_before_egress_assert[
    ((CORE3_CORE_INGRESS_QUEUE_ADDR + CORE3_CORE_QUEUE_LIMIT_BYTES) <=
    CORE3_CORE_EGRESS_QUEUE_ADDR) ? 1 : -1];
typedef char core3Core_egress_before_param_sync_assert[
    ((CORE3_CORE_EGRESS_QUEUE_ADDR + CORE3_CORE_QUEUE_LIMIT_BYTES) <=
    PARAM_SYNC_MAILBOX_REGION_ADDR) ? 1 : -1];
typedef char core3Core_param_sync_before_vision_assert[
    ((PARAM_SYNC_MAILBOX_REGION_ADDR + PARAM_SYNC_MAILBOX_REGION_BYTES) <=
    CORE3_CORE_VISION_REGION_ADDR) ? 1 : -1];
typedef char core3Core_vision_region_end_assert[
    ((CORE3_CORE_VISION_REGION_ADDR + CORE3_CORE_VISION_LIMIT_BYTES) <=
    0x28003000u) ? 1 : -1];

```

```

void core3CoreDataTimingInit(void);
uint32_t core3CoreDataMakeBootGeneration(uint32_t salt);
uint8_t core3CoreDataProbeLink(uint32_t *bootGeneration, uint32_t *publishSeq);
uint8_t core3CoreDataProbeVision(uint32_t *bootGeneration, uint32_t *publishSeq);
void core3CoreDataWriteLink(const core3CoreLinkSnapshot *snapshot);
uint8_t core3CoreDataTryReadLink(core3CoreLinkSnapshot *snapshot);
void core3CoreDataWriteVision(const core3CoreVisionSnapshot *snapshot);
uint8_t core3CoreDataTryReadVision(core3CoreVisionSnapshot *snapshot);
void core3CoreDataResetIngress(uint32_t producerBootGeneration);
void core3CoreDataResetEgress(uint32_t producerBootGeneration);
void core3CoreDataSyncIngressConsumer(uint32_t producerBootGeneration);
void core3CoreDataSyncEgressConsumer(uint32_t producerBootGeneration);
uint8_t core3CoreDataPushIngress(const core3CoreIngressEvent *event);
uint8_t core3CoreDataPopIngress(core3CoreIngressEvent *event,
    uint32_t producerBootGeneration);

```



```

uint8_t core3CoreDataPushEgress(const core3CoreEgressEvent *event);
uint8_t core3CoreDataPopEgress(core3CoreEgressEvent *event,
    uint32_t producerBootGeneration);
uint8_t core3CoreBorderRejectionMerge(
    const core3CoreBorderRejection *pending,
    observerLinkPlanningTrackSnapshot *snapshot);
uint8_t core3CoreBorderRejectionAppend(core3CoreBorderRejection *pending,
    uint8_t *pendingValid, const core3CoreBorderRejection *event);
uint8_t core3CorePlanningPreserveRejections(
    observerLinkPlanningTrackSnapshot *destination,
    const observerLinkPlanningTrackSnapshot *pending);
void core3CoreDataNoteSharedStale(void);
void core3CoreDataNoteSessionReset(void);
#endif /* CORE3_CORE_DATA_TRANSFER_H */

```

core3_cyt4bb7/project/code/comm/core3KvmCore3.c

SHA-256

:

8364aadca4b36bf57ecbfa2e95e38842e9544264a998e0169b90ebb8607fda9

代码 (c)

```

#include "core3KvmCore3.h"

#include <stddef.h>
#include <string.h>

#define CORE3_KVM_KEY_COUNT    (5u)
#define CORE3_KVM_KEY_STATE_COUNT (2u)
#define CORE3_KVM_KEY1        (0u)
#define CORE3_KVM_LONG_PRESS  (1u)
#define CORE3_KVM_ACTION_SAVE (1u)

typedef struct
{
    core3KvmCore3Status publicStatus;
    observerLinkKvmEvent event;
    observerLinkKvmAck ack;
    observerLinkKvmAck completedAck;
    uint32_t lastEventSeq;
    uint32_t screenRevision;
    uint8_t ackReady;
    uint8_t completedAckValid;
    uint8_t menuState;
    uint8_t menuFlags;
    uint8_t lastAckResult;
} core3KvmState;

static core3KvmState g_kvm;

static uint8_t deadlineReached(uint32_t nowMs, uint32_t deadlineMs)
{
    return (uint8_t) (((int32_t) (nowMs - deadlineMs)) >= 0);
}

static uint32_t nextEventSeq(uint32_t value)
{
    value++;
    return (value != 0u) ? value : 1u;
}

static uint8_t eventSeqNewer(uint32_t value, uint32_t reference)
{
    return (uint8_t)((value != reference) &&
        ((uint32_t)(value - reference) < 0x80 000 000u));
}

static uint8_t routeMatchesEvent(const observerLinkKvmEvent *event)
{
    return (uint8_t) ((event->routeSession == g_kvm.publicStatus.routeSession) &&
        (event->routeEpoch == g_kvm.publicStatus.routeEpoch));
}

static uint8_t sameCompletedEvent(const observerLinkKvmEvent *event)
{

```

```

return (uint8_t) ((g_kvm.completedAckValid != 0u) &&
    (event->targetSession == g_kvm.completedAck.targetSession) &&
    (event->routeSession == g_kvm.completedAck.routeSession) &&
    (event->routeEpoch == g_kvm.completedAck.routeEpoch) &&
    (event->eventSeq == g_kvm.completedAck.eventSeq) &&
    (event->target == g_kvm.completedAck.target));
}

static void updatePublicPending(void)
{
    g_kvm.publicStatus.ackPending = g_kvm.ackReady;
}

static void prepareAck(const observerLinkKvmEvent *event, uint8_t result)
{
    memset(&g_kvm.ack, 0, sizeof(g_kvm.ack));
    g_kvm.ack.targetSession = g_kvm.publicStatus.localSession;
    g_kvm.ack.routeSession = event->routeSession;
    g_kvm.ack.routeEpoch = event->routeEpoch;
    g_kvm.ack.eventSeq = event->eventSeq;
    g_kvm.ack.expectedSeq = g_kvm.publicStatus.expectedSeq;
    g_kvm.ack.target = event->target;
    g_kvm.ack.result = result;
    g_kvm.ack.menuState = g_kvm.menuState;
    g_kvm.ackReady = 1u;
    updatePublicPending();
}

static void clearRouteState(void)
{
    g_kvm.publicStatus.expectedSeq = 1u;
    g_kvm.publicStatus.eventPending = 0u;
    g_kvm.ackReady = 0u;
    g_kvm.completedAckValid = 0u;
    g_kvm.lastEventSeq = 0u;
    g_kvm.lastAckResult = OBSERVER_LINK_KVM_RESULT_NONE;
    updatePublicPending();
}

void core3KvmCore3Init(uint32_t localSession)
{
    memset(&g_kvm, 0, sizeof(g_kvm));
    g_kvm.publicStatus.localSession = localSession;
    g_kvm.publicStatus.expectedSeq = 1u;
}

void core3KvmCore3Poll(uint32_t tickMs)
{
    if ((g_kvm.publicStatus.leaseValid != 0u) &&
        (deadlineReached(tickMs, g_kvm.publicStatus.leaseDeadlineMs) != 0u))
    {
        g_kvm.publicStatus.leaseValid = 0u;
        g_kvm.menuFlags &= (uint8_t) ~OBSERVER_LINK_KVM_MENU_FLAG_LEASE_VALID;
    }
}

void core3KvmCore3Revoke(void)
{
    g_kvm.publicStatus.routeSession = 0u;
    g_kvm.publicStatus.routeEpoch = 0u;
    g_kvm.publicStatus.leaseDeadlineMs = 0u;
    g_kvm.publicStatus.leaseValid = 0u;
    g_kvm.menuFlags &= (uint8_t) ~OBSERVER_LINK_KVM_MENU_FLAG_LEASE_VALID;
    clearRouteState();
}

uint8_t core3KvmCore3ApplyControl(const observerLinkKvmControl *control, uint32_t tickMs)
{
    uint8_t routeChanged;
    if ((control == NULL) ||
        (control->targetSession != g_kvm.publicStatus.localSession) ||
        (control->target != OBSERVER_LINK_KVM_TARGET_CORE3) ||
        (control->leaseMs == 0u) ||
        (control->leaseMs > CORE3_KVM_MAX_LEASE_MS))
    {
        return 0u;
    }
    routeChanged = (uint8_t) ((control->routeSession != g_kvm.publicStatus.routeSession) ||
        (control->routeEpoch != g_kvm.publicStatus.routeEpoch));
}

```

```

g_kvm.publicStatus.routeSession = control->routeSession;
g_kvm.publicStatus.routeEpoch = control->routeEpoch;
if (routeChanged != 0u)
{
    clearRouteState();
}
g_kvm.publicStatus.leaseDeadlineMs = tickMs + control->leaseMs;
g_kvm.publicStatus.leaseValid = 1u;
g_kvm.menuFlags |= OBSERVER_LINK_KVM_MENU_FLAG_LEASE_VALID;
return 1u;
}

uint8_t core3KvmCore3ApplyEvent(const observerLinkKvmEvent *event, uint32_t tickMs)
{
    if (event == NULL)
    {
        return 0u;
    }
    core3KvmCore3Poll(tickMs);
    if ((g_kvm.publicStatus.eventPending != 0u) || (g_kvm.ackReady != 0u))
    {
        return 0u;
    }
    if (event->targetSession != g_kvm.publicStatus.localSession)
    {
        prepareAck(event, OBSERVER_LINK_KVM_RESULT_BAD_PAYLOAD);
        return 1u;
    }
    if (event->target != OBSERVER_LINK_KVM_TARGET_CORE3)
    {
        prepareAck(event, OBSERVER_LINK_KVM_RESULT_WRONG_TARGET);
        return 1u;
    }
    if ((g_kvm.publicStatus.routeSession != 0u) && (routeMatchesEvent(event) == 0u))
    {
        prepareAck(event, OBSERVER_LINK_KVM_RESULT_STALE_EPOCH);
        return 1u;
    }
    if ((event->keyId >= CORE3_KVM_KEY_COUNT) ||
        (event->keyState >= CORE3_KVM_KEY_STATE_COUNT) ||
        (event->flags != 0u))
    {
        prepareAck(event, OBSERVER_LINK_KVM_RESULT_BAD_PAYLOAD);
        return 1u;
    }
    if (sameCompletedEvent(event) != 0u)
    {
        g_kvm.ack = g_kvm.completedAck;
        g_kvm.ack.flags |= OBSERVER_LINK_KVM_ACK_FLAG_DUPLICATE;
        g_kvm.ackReady = 1u;
        updatePublicPending();
        return 1u;
    }
    if (g_kvm.publicStatus.leaseValid == 0u)
    {
        prepareAck(event, OBSERVER_LINK_KVM_RESULT_NO_LEASE);
        return 1u;
    }
    if ((event->eventSeq == 0u) ||
        ((event->eventSeq != g_kvm.publicStatus.expectedSeq) &&
         (eventSeqNewer(event->eventSeq, g_kvm.publicStatus.expectedSeq) == 0u)))
    {
        prepareAck(event, OBSERVER_LINK_KVM_RESULT_OLD_SEQ);
        return 1u;
    }
    if (event->eventSeq != g_kvm.publicStatus.expectedSeq)
    {
        prepareAck(event, OBSERVER_LINK_KVM_RESULT_GAP);
        return 1u;
    }
    g_kvm.event = *event;
    g_kvm.publicStatus.eventPending = 1u;
    return 1u;
}

uint8_t core3KvmCore3TakeEvent(observerLinkKvmEvent *event)
{
    if ((event == NULL) || (g_kvm.publicStatus.eventPending == 0u))

```

```

    {
        return 0u;
    }
    *event = g_kvm.event;
    return 1u;
}

uint8_t core3KvmCore3CompleteEvent(uint8_t result, uint8_t menuState,
    uint8_t homeReady, uint8_t action, uint8_t actionOk,
    uint32_t screenRevision)
{
    if (g_kvm.publicStatus.eventPending == 0u)
    {
        return 0u;
    }
    g_kvm.menuState = menuState;
    g_kvm.screenRevision = screenRevision;
    prepareAck(&g_kvm.event, result);
    if (homeReady != 0u)
    {
        g_kvm.ack.flags |= OBSERVER_LINK_KVM_ACK_FLAG_HOME_READY;
    }
    if (actionOk != 0u)
    {
        g_kvm.ack.flags |= OBSERVER_LINK_KVM_ACK_FLAG_ACTION_OK;
    }
    if (action == CORE3_KVM_ACTION_SAVE)
    {
        g_kvm.ack.flags |= OBSERVER_LINK_KVM_ACK_FLAG_ACTION_SAVE;
    }
    g_kvm.publicStatus.expectedSeq = nextEventSeq(g_kvm.publicStatus.expectedSeq);
    g_kvm.ack.expectedSeq = g_kvm.publicStatus.expectedSeq;
    g_kvm.publicStatus.eventPending = 0u;
    g_kvm.completedAck = g_kvm.ack;
    g_kvm.completedAckValid = 1u;
    g_kvm.lastEventSeq = g_kvm.event.eventSeq;
    g_kvm.lastAckResult = result;
    return 1u;
}

uint8_t core3KvmCore3TakeAck(observerLinkKvmAck *ack)
{
    if ((ack == NULL) || (g_kvm.ackReady == 0u))
    {
        return 0u;
    }
    *ack = g_kvm.ack;
    g_kvm.ackReady = 0u;
    updatePublicPending();
    return 1u;
}

void core3KvmCore3UpdateMenu(uint8_t menuState, uint8_t homeReady,
    uint8_t action, uint8_t actionOk, uint8_t feedbackActive,
    uint32_t screenRevision)
{
    uint8_t flags = 0u;
    g_kvm.menuState = menuState;
    g_kvm.screenRevision = screenRevision;
    if (homeReady != 0u)
    {
        flags |= OBSERVER_LINK_KVM_MENU_FLAG_HOME_READY;
    }
    if (feedbackActive != 0u)
    {
        flags |= OBSERVER_LINK_KVM_MENU_FLAG_FEEDBACK_ACTIVE;
    }
    if (g_kvm.publicStatus.leaseValid != 0u)
    {
        flags |= OBSERVER_LINK_KVM_MENU_FLAG_LEASE_VALID;
    }
    if (actionOk != 0u)
    {
        flags |= OBSERVER_LINK_KVM_MENU_FLAG_ACTION_OK;
    }
    if (action == CORE3_KVM_ACTION_SAVE)
    {
        flags |= OBSERVER_LINK_KVM_MENU_FLAG_ACTION_SAVE;
    }
}

```

```

    g_kvm.menuFlags = flags;
}

void core3KvmCore3GetMenuStatus(observerLinkKvmMenuStatus *status)
{
    if (status == NULL)
    {
        return;
    }
    memset(status, 0, sizeof(*status));
    status->targetSession = g_kvm.publicStatus.localSession;
    status->routeSession = g_kvm.publicStatus.routeSession;
    status->routeEpoch = g_kvm.publicStatus.routeEpoch;
    status->lastEventSeq = g_kvm.lastEventSeq;
    status->screenRevision = g_kvm.screenRevision;
    status->target = OBSERVER_LINK_KVM_TARGET_CORE3;
    status->menuState = g_kvm.menuState;
    status->flags = g_kvm.menuFlags;
    status->lastAckResult = g_kvm.lastAckResult;
}

void core3KvmCore3GetStatus(core3KvmCore3Status *status)
{
    if (status != NULL)
    {
        *status = g_kvm.publicStatus;
    }
}

```

core3_cyt4bb7/project/code/comm/core3KvmCore3.h

SHA-256

:

14e17305b889d4ae0b1554ad024b59cafed078e99d379d716b968730a89d1300

代码 (c)

```

#ifndef CORE3_KVM_CORE3_H
#define CORE3_KVM_CORE3_H

#include <stdint.h>

#include "observerLinkProtocol.h"

#define CORE3_KVM_MAX_LEASE_MS (600u)

typedef struct
{
    uint32_t localSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t expectedSeq;
    uint32_t leaseDeadlineMs;
    uint8_t leaseValid;
    uint8_t eventPending;
    uint8_t ackPending;
} core3KvmCore3Status;

void core3KvmCore3Init(uint32_t localSession);
void core3KvmCore3Poll(uint32_t tickMs);
void core3KvmCore3Revoke(void);
uint8_t core3KvmCore3ApplyControl(const observerLinkKvmControl *control, uint32_t tickMs);
uint8_t core3KvmCore3ApplyEvent(const observerLinkKvmEvent *event, uint32_t tickMs);
uint8_t core3KvmCore3TakeEvent(observerLinkKvmEvent *event);
uint8_t core3KvmCore3CompleteEvent(uint8_t result, uint8_t menuState,
    uint8_t homeReady, uint8_t action, uint8_t actionOk,
    uint32_t screenRevision);
uint8_t core3KvmCore3TakeAck(observerLinkKvmAck *ack);
void core3KvmCore3UpdateMenu(uint8_t menuState, uint8_t homeReady,
    uint8_t action, uint8_t actionOk, uint8_t feedbackActive,
    uint32_t screenRevision);
void core3KvmCore3GetMenuStatus(observerLinkKvmMenuStatus *status);
void core3KvmCore3GetStatus(core3KvmCore3Status *status);

#endif /* CORE3_KVM_CORE3_H */

```

core3_cyt4bb7/project/code/comm/core3LinkCore3.c

SHA-256

:

1a98e4d881eaafe1e55e027329480939162acedb4a9f10da9714c969bd6170cf

代码 (c)

```
#include "core3LinkCore3.h"

#include <string.h>

#include "core3CoreDataTransfer.h"
#include "paramSyncMailbox.h"
#include "cy_device_headers.h"
#include "pinMap.h"
#include "zf_driver_uart.h"
#if defined(__ICCARM__)
#include "scb/cy_scb_uart.h"

#if (CORE3_CORE1_UART_NUM != UART_5) || \
    (CORE3_CORE1_UART_RX_PIN != UART5_RX_P02_0) || \
    (CORE3_CORE1_UART_TX_PIN != UART5_TX_P02_1)
#error "Core3-Core1 link must use UART5 P02_0/P02_1"
#endif
#endif

#define OBSERVER_LINK_BAUDRATE (OBSERVER_LINK_UART_BAUDRATE)
#if OBSERVER_LINK_UART_BAUDRATE != 4 000 000u
#error "Core1-Core3 link baud contract requires 4 Mbps"
#endif
#define OBSERVER_LINK_RX_BUDGET (OBSERVER_LINK_RX_BYTE_BUDGET)
#define OBSERVER_LINK_RX_RING_BYTES (2048u)
#define OBSERVER_LINK_RX_RING_MASK (OBSERVER_LINK_RX_RING_BYTES - 1u)
#if (OBSERVER_LINK_RX_RING_BYTES & OBSERVER_LINK_RX_RING_MASK) != 0u
#error "Core1-Core3 RX ring must be a power of two"
#endif
#define OBSERVER_LINK_HELLO_PERIOD_MS (500u)
#define OBSERVER_LINK_HEARTBEAT_PERIOD_MS (200u)
#define OBSERVER_LINK_DIAG_PERIOD_MS (1000u)
#define OBSERVER_LINK_TIMEOUT_MS (600u)
#define OBSERVER_LINK_PLANNING_BURST_MAX (7u)
#define OBSERVER_LINK_PARAM_GLOBAL_RETRY_MS (50u)

static observerLinkParser g_parser;
static core3LinkCore3Status g_status;
static observerLinkSharedConfig g_staged;
static observerLinkConfigAck g_pendingAck;
static uint8_t g_stagedValid;
static uint8_t g_ackPending;
static uint32_t g_packetSeq;
static uint32_t g_rxPacketSeq;
static uint8_t g_rxPacketSeqPrimed;
static uint8_t g_uartInitialized;
static uint32_t g_lastHelloTickMs;
static uint32_t g_lastHeartbeatTickMs;
static uint32_t g_lastDiagTickMs;
static uint32_t g_lastDeferredTickMs;
static uint32_t g_txEpochTickMs;
static uint8_t g_txDataCredits;
static uint8_t g_txControlCredits;
static uint8_t g_planningBurstCount;
static observerLinkKvmControl g_kvmControl;
static observerLinkKvmEvent g_kvmEvent;
static observerLinkKvmAck g_kvmAck;
static observerLinkKvmMenuStatus g_kvmMenuStatus;
static uint8_t g_kvmControlReady;
static uint8_t g_kvmEventReady;
static uint8_t g_kvmAckPending;
static uint8_t g_kvmMenuPending;
static uint8_t g_visionReady;
static observerLinkPlanningTrackSnapshot g_planningActive;
static observerLinkPlanningTrackSnapshot g_planningLatest;
static uint8_t g_planningActiveValid;
static uint8_t g_planningLatestValid;
```

```

static uint8_t g_planningFragmentIndex;
static observerLinkHealth g_health;
static uint8_t g_healthPending;
static volatile uint16_t g_rxHead;
static volatile uint16_t g_rxTail;
static uint8_t g_rxRing[OBSERVER_LINK_RX_RING_BYTES];
static volatile uint32_t g_rxRingOverflow;
static paramSyncMailboxRequest g_paramAssembly;
static paramSyncMailboxResponse g_paramResponse;
static observerParamFrame g_paramReply;
static uint32_t g_paramRequestSequence;
static uint16_t g_paramExpectedBytes, g_paramReceivedBytes;
static uint8_t g_paramAssemblyActive, g_paramResponseValid, g_paramReplyPending;
static uint32_t g_paramOwnerBoot, g_paramOwnerResponseSequence;
static uint32_t g_paramOwnerDeadlineMs, g_paramLastTickMs;
static uint8_t g_paramOwnerBootPrimed, g_paramWaitingOwner;
static paramSyncGlobalState g_paramGlobalState;
static paramSyncGlobalState g_paramGlobalRouteFailure;
static paramSyncGlobalRequest g_paramGlobalRequest;
static uint32_t g_paramGlobalRequestLastSendMs, g_paramGlobalStateLastRxMs;
static uint8_t g_paramGlobalStateReady, g_paramGlobalStateValid;
static uint8_t g_paramGlobalRouteFailureReady, g_paramGlobalRequestPending;

static void failPendingParamGlobalRequest(uint8_t error)
{
    paramSyncGlobalState failure;
    uint32_t sequence;
    if (g_paramGlobalRequestPending == 0u) return;
    memset(&failure, 0, sizeof(failure));
    failure.coordinatorBootGeneration =
        (g_paramGlobalState.coordinatorBootGeneration != 0u) ?
            g_paramGlobalState.coordinatorBootGeneration :
            g_paramGlobalRequest.producerBootGeneration;
    sequence = g_paramGlobalState.sequence + 1u;
    if (sequence == 0u) sequence = 1u;
    failure.sequence = sequence;
    failure.requestBootGeneration =
        g_paramGlobalRequest.producerBootGeneration;
    failure.requestSequence = g_paramGlobalRequest.sequence;
    failure.origin = g_paramGlobalRequest.origin;
    failure.action = g_paramGlobalRequest.action;
    failure.slot = g_paramGlobalRequest.slot;
    failure.phase = PARAM_SYNC_GLOBAL_PHASE_FAILED;
    failure.target = OBSERVER_LINK_SOURCE_CORE3;
    failure.error = error;
    failure.flags = PARAM_SYNC_GLOBAL_FLAG_TERMINAL;
    if (paramSyncGlobalStateIsValid(&failure) != 0u)
    {
        g_paramGlobalRouteFailure = failure;
        g_paramGlobalRouteFailureReady = 1u;
    }
    g_paramGlobalRequestPending = 0u;
}

static void invalidateParamGlobalRoute(void)
{
    failPendingParamGlobalRequest(PARAM_SYNC_GLOBAL_ERROR_ROUTE);
    g_paramGlobalStateValid = 0u;
    g_paramGlobalStateReady = 0u;
}

static uint8_t paramGlobalRouteReady(uint32_t tickMs)
{
    return (uint8_t) (g_status.online != 0u &&
        g_status.protocolCompatible != 0u &&
        (g_status.peerCapabilities & OBSERVER_LINK_CAP_PARAM_SYNC) != 0u &&
        g_paramGlobalStateValid != 0u &&
        (uint32_t) (tickMs - g_paramGlobalStateLastRxMs) <
            CORE3_LINK_PARAM_GLOBAL_STATE_LOST_MS);
}

static uint8_t paramGlobalBusyError(const paramSyncGlobalState *state)
{
    if ((state->flags & PARAM_SYNC_GLOBAL_FLAG_RECOVERY) != 0u)
        return PARAM_SYNC_GLOBAL_ERROR_RECOVERY;
    if (state->readyMask != PARAM_SYNC_GLOBAL_TARGET_MASK)
        return PARAM_SYNC_GLOBAL_ERROR_INCOMPLETE;
    if ((state->flags & (PARAM_SYNC_GLOBAL_FLAG_BUSY |
        PARAM_SYNC_GLOBAL_FLAG_TERMINAL)) != 0u)

```

```

        return PARAM_SYNC_GLOBAL_ERROR_BUSY;
    return PARAM_SYNC_GLOBAL_ERROR_ROUTE;
}

static void clearControlStatus(void)
{
    memset(&g_status.controlStatus, 0, sizeof(g_status.controlStatus));
    g_status.controlStatus.cameraControlState = OBSERVER_LINK_CAMERA_CONTROL_SAFE;
    g_status.cameraControlState = OBSERVER_LINK_CAMERA_CONTROL_SAFE;
    g_status.cameraControlValid = 0u;
    g_status.controlStatusValid = 0u;
    g_status.cameraControlTickMs = 0u;
}

static uint8_t helloIdentifiesCore1(const observerLinkFrame *frame)
{
    uint8_t role;
    uint32_t capabilities;
    const uint32_t required = OBSERVER_LINK_CAP_BINARY_CC |
        OBSERVER_LINK_CAP_LEGACY_CANNY |
        OBSERVER_LINK_CAP_CONFIG_STAGING |
        OBSERVER_LINK_CAP_PLANNING_TRACKS |
        OBSERVER_LINK_CAP_CAMERA_CONTROL |
        OBSERVER_LINK_CAP_CONTROL_STATUS;
    return (uint8_t) ((frame->major == OBSERVER_LINK_PROTOCOL_MAJOR) &&
        (frame->minor == OBSERVER_LINK_PROTOCOL_MINOR) &&
        (observerLinkDecodeHello(frame->payload, frame->payloadLength,
            &role, &capabilities) != 0u) &&
        (role == OBSERVER_LINK_ROLE_CORE1) &&
        ((capabilities & required) == required));
}

static void clearViewOverlay(void)
{
    memset(&g_status.viewOverlay, 0, sizeof(g_status.viewOverlay));
    g_status.viewOverlayValid = 0u;
    g_status.viewOverlayTickMs = 0u;
}

#if defined(_ICCARM_)
__no_init uint32_t core3LinkCore3BootCounter;
__no_init uint32_t core3LinkCore3BootCounterInverse;
#define g_bootCounter core3LinkCore3BootCounter
#define g_bootCounterInverse core3LinkCore3BootCounterInverse
#else
static uint32_t g_bootCounter;
static uint32_t g_bootCounterInverse;
#endif

static uint32_t makeBootSession(uint32_t tickMs)
{
    uint32_t session;
    (void) tickMs;
    if (g_bootCounterInverse != ~g_bootCounter)
    {
        g_bootCounter = 0u;
    }
    g_bootCounter++;
    if (g_bootCounter == 0u) g_bootCounter = 1u;
    g_bootCounterInverse = ~g_bootCounter;
    session = g_bootCounter;
    return session;
}

static void clearOwnerState(uint32_t ownerSession)
{
    invalidateParamGlobalRoute();
    memset(&g_staged, 0, sizeof(g_staged));
    memset(&g_pendingAck, 0, sizeof(g_pendingAck));
    g_stagedValid = 0u;
    g_ackPending = 0u;
    g_status.ownerSession = ownerSession;
    g_status.configStatus = OBSERVER_LINK_CONFIG_NONE;
    g_status.configReason = OBSERVER_LINK_REJECT_NONE;
    g_status.stagedMode = 0xFFu;
    g_status.appliedMode = 0xFFu;
    g_status.stagedRevision = 0u;
    g_status.stagedHash = 0u;
    g_status.appliedRevision = 0u;
}

```



```

g_status.appliedHash = 0u;
clearControlStatus();
clearViewOverlay();
memset(&g_status.peerDiagnostics, 0, sizeof(g_status.peerDiagnostics));
g_status.peerDiagnosticsTickMs = 0u;
g_kvmControlReady = 0u;
g_kvmEventReady = 0u;
g_kvmAckPending = 0u;
g_kvmMenuPending = 0u;
g_planningActiveValid = 0u;
g_planningLatestValid = 0u;
g_planningFragmentIndex = 0u;
memset(&g_health, 0, sizeof(g_health));
g_healthPending = 0u;
g_paramAssemblyActive = 0u;
g_paramResponseValid = 0u;
g_paramReplyPending = 0u;
g_paramWaitingOwner = 0u;
}

static uint8_t sendMessageBudgeted(uint8_t type,
    const uint8_t *payload,
    uint16_t payloadLength,
    uint32_t tickMs,
    uint8_t controlLane)
{
    observerLinkFrame frame;
    uint8_t wire[OBSERVER_LINK_MAX_FRAME_BYTES];
    uint16_t wireLength;

    if (controlLane != 0u)
    {
        if (g_txControlCredits == 0u) return 0u;
    }
    else if (g_txDataCredits == 0u)
    {
        return 0u;
    }

    if ((payloadLength > OBSERVER_LINK_MAX_PAYLOAD_BYTES) ||
        ((payloadLength != 0u) && (payload == NULL)))
    {
        g_status.txFailures++;
        return 0u;
    }
    memset(&frame, 0, sizeof(frame));
    frame.major = OBSERVER_LINK_PROTOCOL_MAJOR;
    frame.minor = OBSERVER_LINK_PROTOCOL_MINOR;
    frame.type = type;
    frame.payloadLength = payloadLength;
    frame.bootSession = g_status.localSession;
    g_packetSeq++;
    if (g_packetSeq == 0u) g_packetSeq = 1u;
    frame.packetSeq = g_packetSeq;
    frame.tickMs = tickMs;
    if (payloadLength != 0u)
    {
        memcpy(frame.payload, payload, payloadLength);
    }
    wireLength = observerLinkEncodeFrame(&frame, wire, (uint16_t) sizeof(wire));
    if (wireLength == 0u)
    {
        g_status.txFailures++;
        return 0u;
    }
    uart_write_buffer(CORE3_CORE1_UART_NUM, wire, wireLength);
    if (controlLane != 0u) g_txControlCredits--;
    else g_txDataCredits--;
    g_status.txFrames++;
    return 1u;
}

static uint8_t sendMessage(uint8_t type,
    const uint8_t *payload,
    uint16_t payloadLength,
    uint32_t tickMs)
{
    return sendMessageBudgeted(type, payload, payloadLength, tickMs, 0u);
}

```

```

static uint8_t sendControlMessage(uint8_t type,
    const uint8_t *payload,
    uint16_t payloadLength,
    uint32_t tickMs)
{
    return sendMessageBudgeted(type, payload, payloadLength, tickMs, 1u);
}

static void publishParamAssembly(void)
{
    g_paramAssemblyActive = 0u;
    g_paramAssembly.dataLength = g_paramExpectedBytes;
    g_paramRequestSequence++;
    if (g_paramRequestSequence == 0u) g_paramRequestSequence = 1u;
    g_paramAssembly.producerBootGeneration = g_status.localSession;
    g_paramAssembly.sequence = g_paramRequestSequence;
    g_paramAssembly.offset = g_paramRequestSequence;
    if (!paramSyncMailboxPublishRequest(&g_paramAssembly))
        g_paramReply.status = PARAM_SYNC_STATUS_FAILED;
    else
    {
        g_paramWaitingOwner = 1u;
        g_paramOwnerDeadlineMs = g_paramLastTickMs + 500u;
    }
}

static void handleParamRequest(const observerLinkFrame *frame, uint32_t tickMs)
{
    observerParamFrame wire;
    paramSyncMailboxResponse response;
    uint16_t remaining;
    g_paramLastTickMs = tickMs;
    if (!observerParamDecode(frame->payload, frame->payloadLength, &wire)) return;
    g_paramReply = wire;
    g_paramReply.payloadCrc32 = 0u;
    g_paramReply.status = PARAM_SYNC_STATUS_PENDING;
    g_paramReply.length = 0u;
    if (wire.op == OBSERVER_PARAM_OP_POLL)
    {
        if (!g_paramResponseValid && paramSyncMailboxTryReadResponse(&response) &&
            response.transactionId == wire.transactionId &&
            response.offset == g_paramAssembly.offset &&
            response.op == g_paramAssembly.op && response.slot == g_paramAssembly.slot &&
            response.target == g_paramAssembly.target)
        {
            if (g_paramOwnerBootPrimed && response.producerBootGeneration != g_paramOwnerBoot)
            {
                g_paramReply.status = PARAM_SYNC_STATUS_FAILED;
                g_paramAssemblyActive = 0u;
                g_paramResponseValid = 0u;
                g_paramWaitingOwner = 0u;
                g_paramOwnerBoot = response.producerBootGeneration;
                g_paramOwnerResponseSequence = response.sequence;
                g_paramOwnerBootPrimed = 1u;
            }
            else if (g_paramOwnerBootPrimed &&
                observerLinkSequenceIsNewer(response.sequence,
                    g_paramOwnerResponseSequence) == 0u)
            {
                /* 保留邮箱旧应答只作基线，不能完成新的线端事务。 */
            }
            else
            {
                g_paramOwnerBoot = response.producerBootGeneration;
                g_paramOwnerResponseSequence = response.sequence;
                g_paramOwnerBootPrimed = 1u;
                g_paramResponse = response;
                g_paramResponseValid = 1u;
                g_paramWaitingOwner = 0u;
            }
        }
    }
    if (g_paramWaitingOwner &&
        (int32_t)(tickMs - g_paramOwnerDeadlineMs) >= 0)
    {
        g_paramWaitingOwner = 0u;
        g_paramReply.status = PARAM_SYNC_STATUS_FAILED;
    }
    if (g_paramResponseValid && g_paramResponse.transactionId == wire.transactionId)

```

```

{
    g_paramReply.op = g_paramResponse.op;
    g_paramReply.slot = g_paramResponse.slot;
    g_paramReply.target = g_paramResponse.target;
    g_paramReply.status = (uint8_t) (g_paramResponse.status & OBSERVER_PARAM_STATUS_MASK);
    g_paramReply.payloadCrc32 = g_paramResponse.payloadCrc32;
    g_paramReply.offset = wire.offset;
    g_paramReply.totalLength = g_paramResponse.dataLength;
    if (wire.offset > g_paramResponse.dataLength)
        g_paramReply.status = PARAM_SYNC_STATUS_INVALID;
    else
    {
        remaining = (uint16_t) (g_paramResponse.dataLength - wire.offset);
        g_paramReply.length = (remaining > OBSERVER_PARAM_DATA_BYTES) ? OBSERVER_PARAM_DATA_BYTES : (uint8_t) remaining;
        memcpy(g_paramReply.data, &g_paramResponse.data[wire.offset], g_paramReply.length);
        if (remaining > g_paramReply.length)
            g_paramReply.status |= OBSERVER_PARAM_TRANSPORT_MORE;
    }
}
}
else if ((wire.status & OBSERVER_PARAM_TRANSPORT_BEGIN) != 0u)
{
    if (wire.target != 3u || wire.totalLength > PARAM_SYNC_MAILBOX_DATA_MAX)
    {
        g_paramAssemblyActive = 0u;
        g_paramResponseValid = 0u;
        g_paramWaitingOwner = 0u;
        g_paramReply.status = PARAM_SYNC_STATUS_INVALID;
        memset(&g_paramAssembly, 0, sizeof(g_paramAssembly));
    }
    else if (wire.transactionId == g_paramAssembly.transactionId &&
        wire.op == g_paramAssembly.op && wire.slot == g_paramAssembly.slot &&
        wire.target == g_paramAssembly.target &&
        wire.payloadCrc32 == g_paramAssembly.payloadCrc32 &&
        wire.totalLength == g_paramExpectedBytes)
    {
        /* 丢失应答后的 BEGIN 重传保留分片进度及 owner 应答。 */
    }
    else
    {
        g_paramAssemblyActive = 0u;
        g_paramResponseValid = 0u;
        g_paramWaitingOwner = 0u;
        memset(&g_paramAssembly, 0, sizeof(g_paramAssembly));
        g_paramExpectedBytes = wire.totalLength;
        g_paramReceivedBytes = 0u;
        g_paramAssembly.op = wire.op;
        g_paramAssembly.slot = wire.slot;
        g_paramAssembly.target = wire.target;
        g_paramAssembly.transactionId = wire.transactionId;
        g_paramAssembly.payloadCrc32 = wire.payloadCrc32;
        g_paramAssemblyActive = 1u;
        if (g_paramExpectedBytes == 0u) publishParamAssembly();
    }
}
else if (wire.transactionId == g_paramAssembly.transactionId &&
    wire.op == g_paramAssembly.op && wire.slot == g_paramAssembly.slot &&
    wire.target == g_paramAssembly.target &&
    wire.payloadCrc32 == g_paramAssembly.payloadCrc32 &&
    ((uint32_t) wire.offset + wire.length <= g_paramExpectedBytes))
{
    if (wire.offset < g_paramReceivedBytes)
    {
        if (((uint32_t) wire.offset + wire.length > g_paramReceivedBytes) ||
            memcmp(&g_paramAssembly.data[wire.offset], wire.data,
                wire.length) != 0)
        {
            g_paramAssemblyActive = 0u;
            g_paramReply.status = PARAM_SYNC_STATUS_INVALID;
        }
    }
    else if (g_paramAssemblyActive && wire.offset == g_paramReceivedBytes)
    {
        memcpy(&g_paramAssembly.data[g_paramReceivedBytes], wire.data, wire.length);
        g_paramReceivedBytes = (uint16_t) (g_paramReceivedBytes + wire.length);
        if (g_paramReceivedBytes == g_paramExpectedBytes) publishParamAssembly();
    }
    else
    {

```

```

        g_paramAssemblyActive = 0u;
        g_paramReply.status = PARAM_SYNC_STATUS_INVALID;
    }
}
else
{
    g_paramAssemblyActive = 0u;
    g_paramReply.status = PARAM_SYNC_STATUS_INVALID;
}
g_paramReplyPending = 1u;
}

static uint8_t sendParamReply(uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_MAX_PAYLOAD_BYTES];
    uint16_t length;
    if (!g_paramReplyPending || !observerParamEncode(&g_paramReply, payload, &length)) return 0u;
    if (!sendMessage(OBSERVER_LINK_MSG_PARAM_RESPONSE, payload, length, tickMs)) return 0u;
    g_paramReplyPending = 0u;
    return 1u;
}

static uint8_t sendParamGlobalRequest(uint32_t tickMs)
{
    uint8_t payload[OBSERVER_PARAM_GLOBAL_REQUEST_BYTES];
    uint16_t length;
    if (g_paramGlobalRequestPending != 0u &&
        paramGlobalRouteReady(tickMs) == 0u)
        failPendingParamGlobalRequest(PARAM_SYNC_GLOBAL_ERROR_ROUTE);
    if ((g_paramGlobalRequestPending == 0u) ||
        ((uint32_t)(tickMs - g_paramGlobalRequestLastSendMs) <
         OBSERVER_LINK_PARAM_GLOBAL_RETRY_MS) ||
        (observerParamGlobalRequestEncode(&g_paramGlobalRequest,
            payload, &length) == 0u) ||
        (sendMessage(OBSERVER_LINK_MSG_PARAM_GLOBAL_REQUEST,
            payload, length, tickMs) == 0u)) return 0u;
    g_paramGlobalRequestLastSendMs = tickMs;
    return 1u;
}

static void queueAck(uint32_t ownerSession,
    const observerLinkSharedConfig *config,
    uint8_t status,
    uint8_t reason)
{
    if ((g_ackPending != 0u) &&
        (g_pendingAck.status == OBSERVER_LINK_CONFIG_REJECTED) &&
        (status != OBSERVER_LINK_CONFIG_REJECTED))
    {
        return;
    }
    memset(&g_pendingAck, 0, sizeof(g_pendingAck));
    g_pendingAck.ownerSession = ownerSession;
    if (config != NULL)
    {
        g_pendingAck.revision = config->revision;
        g_pendingAck.hash = config->hash;
        g_pendingAck.desiredMode = config->detectMode;
    }
    g_pendingAck.appliedMode = g_status.appliedMode;
    g_pendingAck.status = status;
    g_pendingAck.reason = reason;
    g_ackPending = 1u;
}

static void handleHello(const observerLinkFrame *frame)
{
    uint8_t role;
    uint32_t capabilities;

    g_status.peerCapabilities = 0u;
    g_status.protocolCompatible = 0u;
    g_status.planningCompatible = 0u;

    if (observerLinkDecodeHello(frame->payload,
        frame->payloadLength,
        &role,
        &capabilities) == 0u ||
        (role != OBSERVER_LINK_ROLE_CORE1))

```

```

{
    g_status.protocolCompatible = 0u;
    return;
}
g_status.peerCapabilities = capabilities;
g_status.protocolCompatible = (uint8_t) ((frame->minor == OBSERVER_LINK_PROTOCOL_MINOR) &&
    (frame->payload[2] == OBSERVER_LINK_PROTOCOL_MINOR) &&
    ((capabilities & OBSERVER_LINK_CAP_BINARY_CC) != 0u) &&
    ((capabilities & OBSERVER_LINK_CAP_LEGACY_CANNY) != 0u) &&
    ((capabilities & OBSERVER_LINK_CAP_CONFIG_STAGING) != 0u));
g_status.planningCompatible = (uint8_t) ((frame->minor == OBSERVER_LINK_PROTOCOL_MINOR) &&
    (frame->payload[2] == OBSERVER_LINK_PROTOCOL_MINOR) &&
    ((capabilities & OBSERVER_LINK_CAP_PLANNING_TRACKS) != 0u) &&
    ((capabilities & OBSERVER_LINK_CAP_CAMERA_CONTROL) != 0u) &&
    ((capabilities & OBSERVER_LINK_CAP_CONTROL_STATUS) != 0u));
if (g_status.planningCompatible == 0u)
{
    clearControlStatus();
    g_planningActiveValid = 0u;
    g_planningLatestValid = 0u;
    g_planningFragmentIndex = 0u;
}
}

```

```

static void handleConfig(const observerLinkFrame *frame)
{
    observerLinkSharedConfig incoming;

    if ((frame->minor != OBSERVER_LINK_PROTOCOL_MINOR) ||
        (g_status.planningCompatible == 0u))
    {
        queueAck(frame->bootSession,
            NULL,
            OBSERVER_LINK_CONFIG_REJECTED,
            OBSERVER_LINK_REJECT_UNSUPPORTED);
        return;
    }

    if (observerLinkDecodeSharedConfig(frame->payload,
        frame->payloadLength,
        &incoming) == 0u)
    {
        queueAck(frame->bootSession,
            NULL,
            OBSERVER_LINK_CONFIG_REJECTED,
            OBSERVER_LINK_REJECT_BAD_PAYLOAD);
        return;
    }

    if (incoming.hash != observerLinkSharedConfigHash(&incoming))
    {
        queueAck(frame->bootSession,
            &incoming,
            OBSERVER_LINK_CONFIG_REJECTED,
            OBSERVER_LINK_REJECT_BAD_HASH);
        return;
    }

    if ((incoming.sourceId != OBSERVER_LINK_SOURCE_CORE3) ||
        (incoming.profileVersion != 1u) ||
        (g_status.protocolCompatible == 0u) ||
        (observerLinkSharedConfigsValid(&incoming) == 0u))
    {
        queueAck(frame->bootSession,
            &incoming,
            OBSERVER_LINK_CONFIG_REJECTED,
            (g_status.protocolCompatible == 0u)
            ? OBSERVER_LINK_REJECT_UNSUPPORTED
            : OBSERVER_LINK_REJECT_BAD_PAYLOAD);
        return;
    }

    if ((g_stagedValid != 0u) && (incoming.revision != g_staged.revision) &&
        (observerLinkSequencesIsNewer(incoming.revision,
            g_staged.revision) == 0u))
    {
        queueAck(frame->bootSession,
            &g_staged,
            OBSERVER_LINK_CONFIG_REJECTED,
            OBSERVER_LINK_REJECT_OLD_REVISION);
        return;
    }
}

```

```

    }
    if ((g_stagedValid != 0u) && (incoming.revision == g_staged.revision))
    {
        if (incoming.hash != g_staged.hash)
        {
            queueAck(frame->bootSession,
                &g_staged,
                OBSERVER_LINK_CONFIG_REJECTED,
                OBSERVER_LINK_REJECT_REVISION_CONFLICT);
        }
        else
        {
            queueAck(frame->bootSession,
                &g_staged,
                g_status.configStatus,
                g_status.configReason);
        }
        return;
    }

    g_staged = incoming;
    g_stagedValid = 1u;
    g_status.stagedRevision = incoming.revision;
    g_status.stagedHash = incoming.hash;
    g_status.stagedMode = incoming.detectMode;
    g_status.configStatus = OBSERVER_LINK_CONFIG_STAGED;
    g_status.configReason = OBSERVER_LINK_REJECT_NONE;
    queueAck(frame->bootSession,
        &g_staged,
        OBSERVER_LINK_CONFIG_STAGED,
        OBSERVER_LINK_REJECT_NONE);
}

static void handleFrame(const observerLinkFrame *frame, uint32_t tickMs)
{
    uint8_t qualifiedHello = (uint8_t) ((frame->type == OBSERVER_LINK_MSG_HELLO) &&
        (frame->major == OBSERVER_LINK_PROTOCOL_MAJOR) &&
        (frame->minor == OBSERVER_LINK_PROTOCOL_MINOR) &&
        (helloIdentifiesCore1(frame) != 0u));
    uint8_t sessionDecision = observerLinkRxSessionClassify(
        g_status.online, g_status.ownerSession, g_rxPacketSeq,
        g_rxPacketSeqPrimed, frame->bootSession, frame->packetSeq,
        qualifiedHello);
    if (sessionDecision == OBSERVER_LINK_RX_SESSION_REJECT) return;
    if (sessionDecision == OBSERVER_LINK_RX_SESSION_REBIND)
    {
        g_status.bindingEpoch++;
        if (g_status.bindingEpoch == 0u) g_status.bindingEpoch = 1u;
        clearOwnerState(frame->bootSession);
        g_rxPacketSeqPrimed = 0u;
        g_status.protocolCompatible = 0u;
        g_status.peerCapabilities = 0u;
    }
    g_rxPacketSeq = frame->packetSeq;
    g_rxPacketSeqPrimed = 1u;
    g_status.lastValidRxTickMs = tickMs;
    g_status.online = 1u;
    g_status.rxFrames++;

    if ((frame->major != OBSERVER_LINK_PROTOCOL_MAJOR) ||
        (frame->minor != OBSERVER_LINK_PROTOCOL_MINOR))
    {
        invalidateParamGlobalRoute();
        g_status.protocolCompatible = 0u;
        g_status.planningCompatible = 0u;
        g_status.planningActiveValid = 0u;
        g_status.planningLatestValid = 0u;
        g_status.planningFragmentIndex = 0u;
        return;
    }
    if (frame->type == OBSERVER_LINK_MSG_HELLO)
    {
        handleHello(frame);
        if (g_status.protocolCompatible == 0u ||
            (g_status.peerCapabilities & OBSERVER_LINK_CAP_PARAM_SYNC) == 0u)
            invalidateParamGlobalRoute();
    }
    else if (frame->type == OBSERVER_LINK_MSG_CONFIG_SNAPSHOT)

```

```

{
    handleConfig(frame);
}
else if (frame->type == OBSERVER_LINK_MSG_HEARTBEAT)
{
    observerLinkControlStatus controlStatus;
    if ((frame->minor == OBSERVER_LINK_PROTOCOL_MINOR) &&
        ((g_status.peerCapabilities & OBSERVER_LINK_CAP_CAMERA_CONTROL) != 0u) &&
        ((g_status.peerCapabilities & OBSERVER_LINK_CAP_CONTROL_STATUS) != 0u) &&
        (observerLinkDecodeControlStatus(frame->payload,
            frame->payloadLength, &controlStatus) != 0u))
    {
        g_status.controlStatus = controlStatus;
        g_status.controlStatusValid = 1u;
        g_status.cameraControlState = controlStatus.cameraControlState;
        g_status.cameraControlValid = (uint8_t) ((controlStatus.flags & OBSERVER_LINK_CONTROL_FLAG_STATUS_VALID) != 0u);
        if (controlStatus.cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_RUN)
        {
            g_status.lastKnownCameraControlState =
                controlStatus.cameraControlState;
        }
        g_status.cameraControlTickMs = tickMs;
    }
    else
    {
        clearControlStatus();
    }
}
else if (frame->type == OBSERVER_LINK_MSG_DIAGNOSTICS)
{
    if (observerLinkDecodeDiagnostics(frame->payload,
        frame->payloadLength,
        &g_status.peerDiagnostics) != 0u)
    {
        g_status.peerDiagnosticsTickMs = tickMs;
    }
}
else if (frame->type == OBSERVER_LINK_MSG_VIEW_OVERLAY)
{
    observerLinkViewOverlay overlay;
    uint8_t sameIdentity;
    sameIdentity = 0u;
    if ((frame->minor == OBSERVER_LINK_PROTOCOL_MINOR) &&
        ((g_status.peerCapabilities & OBSERVER_LINK_CAP_VIEW_OVERLAY) != 0u) &&
        (observerLinkDecodeViewOverlay(frame->payload,
            frame->payloadLength, &overlay) != 0u) &&
        (overlay.ownerSession == frame->bootSession) &&
        (overlay.ownerSession == g_status.ownerSession) &&
        (overlay.configRevision == g_status.appliedRevision) &&
        (overlay.configHash == g_status.appliedHash) &&
        (overlay.detectMode == g_status.appliedMode))
    {
        sameIdentity = (uint8_t) ((g_status.viewOverlayValid != 0u) &&
            (g_status.viewOverlay.ownerSession == overlay.ownerSession) &&
            (g_status.viewOverlay.configRevision == overlay.configRevision) &&
            (g_status.viewOverlay.configHash == overlay.configHash) &&
            (g_status.viewOverlay.detectMode == overlay.detectMode));
        if ((sameIdentity == 0u) ||
            (observerLinkSequenceIsNewer(overlay.overlaySeq,
                g_status.viewOverlay.overlaySeq) != 0u))
        {
            g_status.viewOverlay = overlay;
            g_status.viewOverlayValid = 1u;
            g_status.viewOverlayTickMs = tickMs;
        }
    }
}
else if (frame->type == OBSERVER_LINK_MSG_KVM_CONTROL)
{
    if (observerLinkDecodeKvmControl(frame->payload,
        frame->payloadLength, &g_kvmControl) != 0u)
    {
        g_kvmControlReady = 1u;
    }
}
else if (frame->type == OBSERVER_LINK_MSG_KVM_EVENT)
{
    if (observerLinkDecodeKvmEvent(frame->payload,
        frame->payloadLength, &g_kvmEvent) != 0u)

```

```

    {
        g_kvmEventReady = 1u;
    }
}
else if ((frame->type == OBSERVER_LINK_MSG_PARAM_REQUEST) &&
        (g_status.peerCapabilities & OBSERVER_LINK_CAP_PARAM_SYNC) != 0u)
{
    handleParamRequest(frame, tickMs);
}
else if (((frame->type == OBSERVER_LINK_MSG_PARAM_GLOBAL_STATE) &&
        ((g_status.peerCapabilities & OBSERVER_LINK_CAP_PARAM_SYNC) != 0u))
{
    paramSyncGlobalState next;
    if (observerParamGlobalStateDecode(frame->payload,
        frame->payloadLength, &next) == 0u)
    {
        invalidateParamGlobalRoute();
    }
    else
    {
        if (g_paramGlobalRequestPending != 0u &&
            g_paramGlobalStateValid != 0u &&
            next.coordinatorBootGeneration !=
                g_paramGlobalState.coordinatorBootGeneration)
            failPendingParamGlobalRequest(
                PARAM_SYNC_GLOBAL_ERROR_PEER_RESTART);
        else if (g_paramGlobalRequestPending != 0u &&
            paramSyncGlobalStateMatchesRequest(&next,
                &g_paramGlobalRequest) == 0u &&
            paramSyncGlobalStateReady(&next) == 0u)
            failPendingParamGlobalRequest(paramGlobalBusyError(&next));
        g_paramGlobalState = next;
        g_paramGlobalStateReady = 1u;
        g_paramGlobalStateValid = 1u;
        g_paramGlobalStateLastRxMs = tickMs;
        if (((next.flags & PARAM_SYNC_GLOBAL_FLAG_TERMINAL) != 0u) &&
            (next.requestBootGeneration ==
                g_paramGlobalRequest.producerBootGeneration) &&
            (next.requestSequence == g_paramGlobalRequest.sequence))
            g_paramGlobalRequestPending = 0u;
    }
}
}

static void pollHardwareRxErrors(void)
{
#ifdef __ICCARM__
    uint32_t status = Cy_SCB_UART_GetRxFifoStatus(CORE3_CORE1_UART_SCB);
    uint32_t errors = status & CY_SCB_UART_RECEIVE_ERR;

    if ((errors & CY_SCB_RX_INTR_OVERFLOW) != 0u)
    {
        g_status.localDiagnostics.rxOverflowErrors++;
    }
    if ((errors & CY_SCB_RX_INTR_UART_FRAME_ERROR) != 0u)
    {
        g_status.localDiagnostics.rxFrameErrors++;
    }
    if ((errors & CY_SCB_RX_INTR_UART_PARITY_ERROR) != 0u)
    {
        g_status.localDiagnostics.rxParityErrors++;
    }
    if (errors != 0u)
    {
        Cy_SCB_UART_ClearRxFifoStatus(CORE3_CORE1_UART_SCB, errors);
    }
#endif
}

static void receiveFrames(uint32_t tickMs)
{
    observerLinkFrame frame;
    uint8_t byte;
    uint16_t budget = OBSERVER_LINK_RX_BUDGET;

    pollHardwareRxErrors();
    while ((budget != 0u) && (g_rxTail != g_rxHead))
    {
        __DMB();

```



```

        byte = g_rxRing[g_rxTail];
        g_rxTail = (uint16_t)((g_rxTail + 1u) & OBSERVER_LINK_RX_RING_MASK);
        budget--;
        if (observerLinkParserFeed(&g_parser, byte, &frame) != 0u)
        {
            handleFrame(&frame, tickMs);
        }
    }
}

static uint8_t sendHello(uint32_t tickMs)
{
    uint8_t payload[8u];
    uint16_t length;
    uint32_t capabilities = OBSERVER_LINK_CAP_CONFIG_STAGING |
        OBSERVER_LINK_CAP_KVM |
        OBSERVER_LINK_CAP_MENU_STATUS |
        OBSERVER_LINK_CAP_PLANNING_TRACKS |
        OBSERVER_LINK_CAP_CAMERA_CONTROL |
        OBSERVER_LINK_CAP_CONTROL_STATUS |
        OBSERVER_LINK_CAP_VIEW_OVERLAY |
        OBSERVER_LINK_CAP_PARAM_SYNC;

    if (g_visionReady != 0u)
    {
        capabilities |= OBSERVER_LINK_CAP_BINARY_CC |
            OBSERVER_LINK_CAP_LEGACY_CANNY |
            OBSERVER_LINK_CAP_CONFIG_APPLY;
    }
    if (observerLinkEncodeHello(OBSERVER_LINK_ROLE_CORE3,
        capabilities,
        payload,
        &length) != 0u)
    {
        return sendMessage(OBSERVER_LINK_MSG_HELLO, payload, length, tickMs);
    }
    return 0u;
}

static uint8_t sendHeartbeat(uint32_t tickMs)
{
    {
        uint8_t payload[4u];
        payload[0] = g_status.protocolCompatible;
        payload[1] = g_status.configStatus;
        payload[2] = g_status.stagedMode;
        payload[3] = g_status.appliedMode;
        return sendMessage(OBSERVER_LINK_MSG_HEARTBEAT,
            payload,
            (uint16_t) sizeof(payload),
            tickMs);
    }
}

static uint8_t sendDiagnostics(uint32_t tickMs)
{
    {
        uint8_t payload[OBSERVER_LINK_DIAGNOSTICS_BYTES];
        uint16_t length;
        observerLinkDiagnostics diagnostics;

        g_status.localDiagnostics.validFrames = g_parser.validFrames;
        g_status.localDiagnostics.crcErrors = g_parser.crcErrors;
        g_status.localDiagnostics.lengthErrors = g_parser.lengthErrors;
        g_status.localDiagnostics.versionErrors = g_parser.versionErrors;
        g_status.localDiagnostics.txDeferred = g_status.txDeferred;
        diagnostics = g_status.localDiagnostics;
        diagnostics.rxOverflowErrors += g_rxRingOverflow;
        if (observerLinkEncodeDiagnostics(&diagnostics,
            payload,
            &length) == 0u)
        {
            return 0u;
        }
        return sendMessage(OBSERVER_LINK_MSG_DIAGNOSTICS,
            payload,
            length,
            tickMs);
    }
}

static uint8_t sendHealth(uint32_t tickMs)
{
    {

```

```

uint8_t payload[OBSERVER_LINK_HEALTH_BYTES];
uint16_t length;
observerLinkHealth health;

if (g_healthPending == 0u) return 0u;
health = g_health;
health.flags |= OBSERVER_LINK_HEALTH_FLAG_BRIDGE_FRESH;
health.bridgeAgeMs = 0u;
if ((observerLinkEncodeHealth(&health, payload, &length) != 0u) &&
    (sendMessage(OBSERVER_LINK_MSG_HEALTH, payload, length, tickMs) != 0u))
{
    g_healthPending = 0u;
    return 1u;
}
return 0u;
}

static uint8_t sendPendingAck(uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_CONFIG_ACK_BYTES];
    uint16_t length;

    if ((g_ackPending != 0u) &&
        (observerLinkEncodeConfigAck(&g_pendingAck, payload, &length) != 0u) &&
        (sendMessage(OBSERVER_LINK_MSG_CONFIG_ACK, payload, length, tickMs) != 0u))
    {
        g_ackPending = 0u;
        return 1u;
    }
    return 0u;
}

static uint8_t sendKvmAck(uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_KVM_ACK_BYTES];
    uint16_t length;
    if ((g_kvmAckPending != 0u) &&
        (observerLinkEncodeKvmAck(&g_kvmAck, payload, &length) != 0u) &&
        (sendControlMessage(OBSERVER_LINK_MSG_KVM_ACK, payload, length, tickMs) != 0u))
    {
        g_kvmAckPending = 0u;
        return 1u;
    }
    return 0u;
}

static uint8_t sendKvmMenuStatus(uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_KVM_MENU_STATUS_BYTES];
    uint16_t length;
    if ((g_kvmMenuPending != 0u) &&
        (observerLinkEncodeKvmMenuStatus(&g_kvmMenuStatus, payload, &length) != 0u) &&
        (sendControlMessage(OBSERVER_LINK_MSG_KVM_MENU_STATUS, payload, length, tickMs) != 0u))
    {
        g_kvmMenuPending = 0u;
        return 1u;
    }
    return 0u;
}

static uint8_t sendPlanningTrackFragment(uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_MAX_BYTES];
    uint16_t length;
    uint8_t fragmentCount;

    if (g_planningActiveValid == 0u)
    {
        if (g_planningLatestValid == 0u)
        {
            return 0u;
        }
        g_planningActive = g_planningLatest;
        g_planningActiveValid = 1u;
        g_planningLatestValid = 0u;
        g_planningFragmentIndex = 0u;
    }
    if ((g_status.online == 0u) || (g_status.protocolCompatible == 0u) ||
        (g_status.planningCompatible == 0u) ||

```

```

    (g_status.configStatus != OBSERVER_LINK_CONFIG_APPLIED) ||
    ((g_status.peerCapabilities & OBSERVER_LINK_CAP_PLANNING_TRACKS) == 0u) ||
    (g_planningActive.ownerSession != g_status.ownerSession) ||
    (g_planningActive.revision != g_status.appliedRevision) ||
    (g_planningActive.hash != g_status.appliedHash) ||
    (g_planningActive.detectMode != g_status.appliedMode))
{
    g_planningActiveValid = 0u;
    return 0u;
}
fragmentCount = observerLinkPlanningTrackFragmentCount(
    g_planningActive.retainedCount);
if (observerLinkEncodePlanningTrackFragment(&g_planningActive,
    g_planningFragmentIndex, payload, &length) == 0u)
{
    g_status.txFailures++;
    g_planningActiveValid = 0u;
    g_planningFragmentIndex = 0u;
    return 0u;
}
if (sendMessage(OBSERVER_LINK_MSG_PLANNING_TRACK_FRAGMENT,
    payload, length, tickMs) == 0u)
{
    return 0u;
}
g_planningFragmentIndex++;
if (g_planningFragmentIndex >= fragmentCount)
{
    g_planningActiveValid = 0u;
    g_planningFragmentIndex = 0u;
}
return 1u;
}

void core3LinkCore3Init(uint32_t tickMs)
{
    paramSyncMailboxRequest retainedRequest;
    paramSyncMailboxResponse retainedResponse;
    if (g_uartInitialized != 0u) uart_rx_interrupt(CORE3_CORE1_UART_NUM, 0u);
    memset(&g_status, 0, sizeof(g_status));
    observerLinkParserInit(&g_parser);
    clearOwnerState(0u);
    g_packetSeq = 0u;
    g_status.localSession = makeBootSession(tickMs);
    g_lastHelloTickMs = tickMs - OBSERVER_LINK_HELLO_PERIOD_MS;
    g_lastHeartbeatTickMs = tickMs - OBSERVER_LINK_HEARTBEAT_PERIOD_MS;
    g_lastDiagTickMs = tickMs - OBSERVER_LINK_DIAG_PERIOD_MS;
    g_lastDeferredTickMs = tickMs - 1u;
    g_txEpochTickMs = tickMs;
    g_txDataCredits = OBSERVER_LINK_TX_FRAMES_PER_EPOCH;
    g_txControlCredits = OBSERVER_LINK_TX_CONTROL_FRAMES_PER_EPOCH;
    g_planningBurstCount = 0u;
    memset(&g_kvmControl, 0, sizeof(g_kvmControl));
    memset(&g_kvmEvent, 0, sizeof(g_kvmEvent));
    memset(&g_kvmAck, 0, sizeof(g_kvmAck));
    memset(&g_kvmMenuStatus, 0, sizeof(g_kvmMenuStatus));
    g_kvmControlReady = 0u;
    g_kvmEventReady = 0u;
    g_kvmAckPending = 0u;
    g_kvmMenuPending = 0u;
    memset(&g_planningActive, 0, sizeof(g_planningActive));
    memset(&g_planningLatest, 0, sizeof(g_planningLatest));
    g_planningActiveValid = 0u;
    g_planningLatestValid = 0u;
    g_planningFragmentIndex = 0u;
    g_rxHead = 0u;
    g_rxTail = 0u;
    g_rxRingOverflow = 0u;
    g_paramAssemblyActive = 0u;
    g_paramResponseValid = 0u;
    g_paramReplyPending = 0u;
    g_paramRequestSequence = 0u;
    if (paramSyncMailboxTryReadRequest(&retainedRequest) != 0u)
        g_paramRequestSequence = retainedRequest.sequence;
    g_paramOwnerBoot = 0u;
    g_paramOwnerResponseSequence = 0u;
    g_paramOwnerBootPrimed = 0u;
    if (paramSyncMailboxTryReadResponse(&retainedResponse) != 0u)
    {

```

```

    g_paramOwnerBoot = retainedResponse.producerBootGeneration;
    g_paramOwnerResponseSequence = retainedResponse.sequence;
    g_paramOwnerBootPrimed = 1u;
}
g_paramWaitingOwner = 0u;
memset(&g_paramGlobalState, 0, sizeof(g_paramGlobalState));
memset(&g_paramGlobalRouteFailure, 0,
    sizeof(g_paramGlobalRouteFailure));
memset(&g_paramGlobalRequest, 0, sizeof(g_paramGlobalRequest));
g_paramGlobalStateReady = 0u;
g_paramGlobalStateValid = 0u;
g_paramGlobalRouteFailureReady = 0u;
g_paramGlobalStateLastRxMs = tickMs;
g_paramGlobalRequestPending = 0u;
g_paramGlobalRequestLastSendMs = tickMs - OBSERVER_LINK_PARAM_GLOBAL_RETRY_MS;
g_visionReady = 0u;
uart_init(CORE3_CORE1_UART_NUM,
    OBSERVER_LINK_BAUDRATE,
    CORE3_CORE1_UART_TX_PIN,
    CORE3_CORE1_UART_RX_PIN);
uart_rx_interrupt(CORE3_CORE1_UART_NUM, 1u);
g_uartInitialized = 1u;
}

void core3LinkCore3RxIsr(void)
{
    uint8_t byte;
    while (uart_query_byte(CORE3_CORE1_UART_NUM, &byte) != 0u)
    {
        uint16_t next = (uint16_t)((g_rxHead + 1u) & OBSERVER_LINK_RX_RING_MASK);
        if (next == g_rxTail)
        {
            g_rxRingOverflow++;
            continue;
        }
        g_rxRing[g_rxHead] = byte;
        __DMB();
        g_rxHead = next;
    }
}

void core3LinkCore3SetVisionReady(uint8_t ready)
{
    g_visionReady = (ready != 0u) ? 1u : 0u;
}

static uint8_t transmitWorkPending(uint32_t tickMs)
{
    const observerLinkPlanningTrackSnapshot *planning =
        (g_planningActiveValid != 0u) ? &g_planningActive : ((g_planningLatestValid != 0u) ? &g_planningLatest : NULL);
    uint8_t planningSendable = (uint8_t)((planning != NULL) &&
        (g_status.online != 0u) && (g_status.protocolCompatible != 0u) &&
        (g_status.planningCompatible != 0u) &&
        (g_status.configStatus == OBSERVER_LINK_CONFIG_APPLIED) &&
        ((g_status.peerCapabilities & OBSERVER_LINK_CAP_PLANNING_TRACKS) != 0u) &&
        (planning->ownerSession == g_status.ownerSession) &&
        (planning->revision == g_status.appliedRevision) &&
        (planning->hash == g_status.appliedHash) &&
        (planning->detectMode == g_status.appliedMode));

    return (uint8_t)((g_ackPending != 0u) ||
        (g_paramReplyPending != 0u) ||
        ((g_paramGlobalRequestPending != 0u) &&
            ((uint32_t)(tickMs - g_paramGlobalRequestLastSendMs) >=
                OBSERVER_LINK_PARAM_GLOBAL_RETRY_MS)) ||
        (planningSendable != 0u) ||
        (g_healthPending != 0u) ||
        ((uint32_t)(tickMs - g_lastHeartbeatTickMs) >=
            OBSERVER_LINK_HEARTBEAT_PERIOD_MS) ||
        ((uint32_t)(tickMs - g_lastHelloTickMs) >=
            OBSERVER_LINK_HELLO_PERIOD_MS) ||
        ((uint32_t)(tickMs - g_lastDiagTickMs) >=
            OBSERVER_LINK_DIAG_PERIOD_MS));
}

static uint8_t transmitOne(uint32_t tickMs)
{
    if (sendPendingAck(tickMs) != 0u) return 1u;
    if (((g_status.online == 0u) || (g_status.protocolCompatible == 0u)) &&

```

```

        ((uint32_t) (tickMs - g_lastHelloTickMs) >= OBSERVER_LINK_HELLO_PERIOD_MS) &&
        (sendHello(tickMs) != 0u))
    {
        g_lastHelloTickMs = tickMs;
        return 1u;
    }
    if (((uint32_t) (tickMs - g_lastHeartbeatTickMs) >= OBSERVER_LINK_HEARTBEAT_PERIOD_MS) &&
        (sendHeartbeat(tickMs) != 0u))
    {
        g_lastHeartbeatTickMs = tickMs;
        return 1u;
    }
    if (sendHealth(tickMs) != 0u) return 1u;
    if ((g_planningBurstCount >= OBSERVER_LINK_PLANNING_BURST_MAX) &&
        ((uint32_t) (tickMs - g_lastHelloTickMs) >= OBSERVER_LINK_HELLO_PERIOD_MS) &&
        (sendHello(tickMs) != 0u))
    {
        g_lastHelloTickMs = tickMs;
        g_planningBurstCount = 0u;
        return 1u;
    }
    if ((g_planningBurstCount >= OBSERVER_LINK_PLANNING_BURST_MAX) &&
        ((uint32_t) (tickMs - g_lastDiagTickMs) >= OBSERVER_LINK_DIAG_PERIOD_MS) &&
        (sendDiagnostics(tickMs) != 0u))
    {
        g_lastDiagTickMs = tickMs;
        g_planningBurstCount = 0u;
        return 1u;
    }
    if (sendPlanningTrackFragment(tickMs) != 0u)
    {
        if (g_planningBurstCount < 0xFFu) g_planningBurstCount++;
        return 1u;
    }
    g_planningBurstCount = 0u;
    if (((uint32_t) (tickMs - g_lastHelloTickMs) >= OBSERVER_LINK_HELLO_PERIOD_MS) &&
        (sendHello(tickMs) != 0u))
    {
        g_lastHelloTickMs = tickMs;
        return 1u;
    }
    if (((uint32_t) (tickMs - g_lastDiagTickMs) >= OBSERVER_LINK_DIAG_PERIOD_MS) &&
        (sendDiagnostics(tickMs) != 0u))
    {
        g_lastDiagTickMs = tickMs;
        return 1u;
    }
    if (sendParamReply(tickMs) != 0u) return 1u;
    if (sendParamGlobalRequest(tickMs) != 0u) return 1u;
    return 0u;
}

uint8_t core3LinkCore3QueueParamGlobalRequest(
    const paramSyncGlobalRequest *request)
{
    if ((request == NULL) || (paramSyncGlobalRequestIsValid(request) == 0u) ||
        (request->origin != OBSERVER_LINK_SOURCE_CORE3)) return 0u;
    if ((g_paramGlobalRequestPending != 0u) &&
        ((g_paramGlobalRequest.producerBootGeneration !=
            request->producerBootGeneration) ||
            (g_paramGlobalRequest.sequence != request->sequence))) return 0u;
    g_paramGlobalRequest = *request;
    g_paramGlobalRequestPending = 1u;
    if (paramGlobalRouteReady(g_paramLastTickMs) == 0u)
    {
        failPendingParamGlobalRequest(PARAM_SYNC_GLOBAL_ERROR_ROUTE);
        return 1u;
    }
    g_paramGlobalRequestLastSendMs -= OBSERVER_LINK_PARAM_GLOBAL_RETRY_MS;
    return 1u;
}

uint8_t core3LinkCore3TakeParamGlobalState(paramSyncGlobalState *state)
{
    if (state != NULL && g_paramGlobalRouteFailureReady != 0u)
    {
        *state = g_paramGlobalRouteFailure;
        g_paramGlobalRouteFailureReady = 0u;
        return 1u;
    }

```

```

    }
    if ((state == NULL) || (g_paramGlobalStateReady == 0u)) return 0u;
    if (paramSyncGlobalStateIsValid(&g_paramGlobalState) == 0u)
    {
        g_paramGlobalStateReady = 0u;
        return 0u;
    }
    *state = g_paramGlobalState;
    g_paramGlobalStateReady = 0u;
    return 1u;
}

uint8_t core3LinkCore3IsParamGlobalStateValid(void)
{
    return paramGlobalRouteReady(g_paramLastTickMs);
}

static void refreshTxEpoch(uint32_t tickMs)
{
    uint32_t elapsedEpochs;
    uint32_t credits;

    elapsedEpochs = (uint32_t) (tickMs - g_txEpochTickMs) /
        OBSERVER_LINK_TX_EPOCH_MS;
    if (elapsedEpochs != 0u)
    {
        g_txEpochTickMs += elapsedEpochs * OBSERVER_LINK_TX_EPOCH_MS;
        credits = (uint32_t) g_txDataCredits +
            elapsedEpochs * OBSERVER_LINK_TX_FRAMES_PER_EPOCH;
        g_txDataCredits = (uint8_t) ((credits > OBSERVER_LINK_TX_DATA_CREDIT_CAP) ? OBSERVER_LINK_TX_DATA_CREDIT_CAP : credits);
        credits = (uint32_t) g_txControlCredits +
            elapsedEpochs * OBSERVER_LINK_TX_CONTROL_FRAMES_PER_EPOCH;
        g_txControlCredits = (uint8_t) ((credits > OBSERVER_LINK_TX_CONTROL_CREDIT_CAP) ? OBSERVER_LINK_TX_CONTROL_CREDIT_CAP :
credits);
    }
}

static void serviceControlTransmit(uint32_t tickMs)
{
    refreshTxEpoch(tickMs);
    while (g_txControlCredits != 0u)
    {
        if (sendKvmAck(tickMs) != 0u) continue;
        if (sendKvmMenuStatus(tickMs) != 0u) continue;
        break;
    }
}

static void serviceTransmit(uint32_t tickMs)
{
    refreshTxEpoch(tickMs);
    while ((g_txDataCredits != 0u) &&
        (transmitOne(tickMs) != 0u))
    {
        /* 有串行发送；完整快照的 active 分片在 latest 之前保持冻结。 */
    }
    if ((transmitWorkPending(tickMs) != 0u) &&
        (g_txDataCredits == 0u) &&
        (g_lastDeferredTickMs != tickMs))
    {
        g_lastDeferredTickMs = tickMs;
        g_status.txDeferred++;
    }
}

void core3LinkCore3Poll(uint32_t tickMs)
{
    g_paramLastTickMs = tickMs;
    receiveFrames(tickMs);
    if (g_paramGlobalStateValid != 0u &&
        (uint32_t) (tickMs - g_paramGlobalStateLastRxMs) >=
            CORE3_LINK_PARAM_GLOBAL_STATE_LOST_MS)
        invalidateParamGlobalRoute();
    if ((g_status.online != 0u) &&
        ((uint32_t) (tickMs - g_status.lastValidRxTickMs) >= OBSERVER_LINK_TIMEOUT_MS))
    {
        uint32_t ownerSession = g_status.ownerSession;
        g_status.online = 0u;
    }
}

```

```

    g_rxPacketSeqPrimed = 0u;
    g_status.protocolCompatible = 0u;
    g_status.peerCapabilities = 0u;
    clearOwnerState(ownerSession);
}
if ((g_status.controlStatusValid != 0u) &&
    ((uint32_t) (tickMs - g_status.cameraControlTickMs) >= OBSERVER_LINK_TIMEOUT_MS))
{
    clearControlStatus();
}
serviceControlTransmit(tickMs);
serviceTransmit(tickMs);
}

void core3LinkCore3FlushControl(uint32_t tickMs)
{
    serviceControlTransmit(tickMs);
}

void core3LinkCore3FlushTx(uint32_t tickMs)
{
    serviceTransmit(tickMs);
}

void core3LinkCore3GetStatus(core3LinkCore3Status *status)
{
    if (status != NULL)
    {
        g_status.localDiagnostics.validFrames = g_parser.validFrames;
        g_status.localDiagnostics.crcErrors = g_parser.crcErrors;
        g_status.localDiagnostics.lengthErrors = g_parser.lengthErrors;
        g_status.localDiagnostics.versionErrors = g_parser.versionErrors;
        g_status.localDiagnostics.txDeferred = g_status.txDeferred;
        *status = g_status;
        status->localDiagnostics.rxOverflowErrors += g_rxRingOverflow;
    }
}

uint32_t core3LinkCore3GetLocalBootGeneration(void)
{
    return g_status.localSession;
}

uint8_t core3LinkCore3GetStagedConfig(observerLinkSharedConfig *config)
{
    if ((config == NULL) || (g_stagedValid == 0u))
    {
        return 0u;
    }
    *config = g_staged;
    return 1u;
}

uint8_t core3LinkCore3MarkConfigApplied(uint32_t ownerSession,
    uint32_t revision,
    uint32_t hash,
    uint8_t appliedMode)
{
    if ((g_stagedValid == 0u) ||
        (g_status.online == 0u) ||
        (g_status.protocolCompatible == 0u) ||
        (ownerSession != g_status.ownerSession) ||
        (revision != g_staged.revision) ||
        (hash != g_staged.hash) ||
        (appliedMode != g_staged.detectMode))
    {
        return 0u;
    }
    if ((g_status.appliedRevision != revision) ||
        (g_status.appliedHash != hash) ||
        (g_status.appliedMode != appliedMode))
    {
        clearViewOverlay();
    }
    g_status.appliedRevision = revision;
    g_status.appliedHash = hash;
    g_status.appliedMode = appliedMode;
    g_status.configStatus = OBSERVER_LINK_CONFIG_APPLIED;
    g_status.configReason = OBSERVER_LINK_REJECT_NONE;
}

```

```

queueAck(ownerSession,
    &g_staged,
    OBSERVER_LINK_CONFIG_APPLIED,
    OBSERVER_LINK_REJECT_NONE);
return 1u;
}

uint8_t core3LinkCore3TakeKvmControl(observerLinkKvmControl *control)
{
    if ((control == NULL) || (g_kvmControlReady == 0u)) return 0u;
    *control = g_kvmControl;
    g_kvmControlReady = 0u;
    return 1u;
}

uint8_t core3LinkCore3TakeKvmEvent(observerLinkKvmEvent *event)
{
    if ((event == NULL) || (g_kvmEventReady == 0u)) return 0u;
    *event = g_kvmEvent;
    g_kvmEventReady = 0u;
    return 1u;
}

uint8_t core3LinkCore3QueueKvmAck(const observerLinkKvmAck *ack)
{
    if (ack == NULL) return 0u;
    g_kvmAck = *ack;
    g_kvmAckPending = 1u;
    return 1u;
}

uint8_t core3LinkCore3QueueKvmMenuStatus(const observerLinkKvmMenuStatus *status)
{
    if (status == NULL) return 0u;
    g_kvmMenuStatus = *status;
    g_kvmMenuPending = 1u;
    return 1u;
}

uint8_t core3LinkCore3QueuePlanningTrackSnapshot(
    const observerLinkPlanningTrackSnapshot *snapshot)
{
    if ((snapshot == NULL) || (snapshot->valid == 0u) ||
        (snapshot->sourceId != OBSERVER_LINK_SOURCE_CORE3) ||
        (snapshot->retainedCount > OBSERVER_LINK_MAX_TRACKS))
    {
        return 0u;
    }
    if (g_planningActiveValid == 0u)
    {
        g_planningActive = *snapshot;
        g_planningActiveValid = 1u;
        g_planningFragmentIndex = 0u;
    }
    else
    {
        observerLinkPlanningTrackSnapshot next = *snapshot;
        if ((g_planningLatestValid != 0u) &&
            (core3CorePlanningPreserveRejections(&next,
                &g_planningLatest) == 0u)) return 0u;
        g_planningLatest = next;
        g_planningLatestValid = 1u;
    }
    return 1u;
}

uint8_t core3LinkCore3PublishHealth(const observerLinkHealth *health)
{
    uint32_t bridgeSeq;
    if ((health == NULL) ||
        (health->sourceId != OBSERVER_LINK_SOURCE_CORE3)) return 0u;
    bridgeSeq = g_health.bridgeSeq + 1u;
    if (bridgeSeq == 0u) bridgeSeq = 1u;
    g_health = *health;
    g_health.bridgeSeq = bridgeSeq;
    g_health.bridgeAgeMs = 0u;
    g_health.flags |= OBSERVER_LINK_HEALTH_FLAG_BRIDGE_FRESH;
    g_healthPending = 1u;
    return 1u;
}

```



```

}

uint8_t core3LinkCore3GetViewOverlay(observerLinkViewOverlay *overlay,
uint32_t tickMs, uint32_t timeoutMs, uint32_t *ageMs)
{
    uint32_t age;

    if ((overlay == NULL) || (g_status.viewOverlayValid == 0u)) return 0u;
    age = tickMs - g_status.viewOverlayTickMs;
    if (ageMs != NULL) *ageMs = age;
    if ((age > timeoutMs) || (g_status.online == 0u) ||
        (g_status.protocolCompatible == 0u) ||
        ((g_status.peerCapabilities & OBSERVER_LINK_CAP_VIEW_OVERLAY) == 0u) ||
        (g_status.viewOverlay.ownerSession != g_status.ownerSession) ||
        (g_status.viewOverlay.configRevision != g_status.appliedRevision) ||
        (g_status.viewOverlay.configHash != g_status.appliedHash) ||
        (g_status.viewOverlay.detectMode != g_status.appliedMode)) return 0u;
    *overlay = g_status.viewOverlay;
    return 1u;
}

```

core3_cyt4bb7/project/code/comm/core3LinkCore3.h

SHA-256

:

c3136b9f12a9b3677bb291c6d2fe150c0150ac4f491ae6bf835d38c1ad23f438

代码 (c)

```

#ifndef OBSERVER_LINK_CORE3_H
#define OBSERVER_LINK_CORE3_H

#include <stdint.h>

#include "observerLinkProtocol.h"

#define CORE3_LINK_PARAM_GLOBAL_STATE_LOST_MS (300u)

typedef struct
{
    uint8_t online;
    uint8_t protocolCompatible;
    uint8_t planningCompatible;
    uint8_t configStatus;
    uint8_t configReason;
    uint8_t stagedMode;
    uint8_t appliedMode;
    uint8_t cameraControlState;
    uint8_t cameraControlValid;
    uint8_t lastKnownCameraControlState;
    uint8_t controlStatusValid;
    uint32_t localSession;
    uint32_t ownerSession;
    uint32_t bindingEpoch;
    uint32_t peerCapabilities;
    uint32_t stagedRevision;
    uint32_t stagedHash;
    uint32_t appliedRevision;
    uint32_t appliedHash;
    uint32_t lastValidRxTickMs;
    uint32_t rxFrames;
    uint32_t txFrames;
    uint32_t txFailures;
    uint32_t txDeferred;
    observerLinkDiagnostics localDiagnostics;
    observerLinkDiagnostics peerDiagnostics;
    uint32_t peerDiagnosticsTickMs;
    uint32_t cameraControlTickMs;
    observerLinkControlStatus controlStatus;
    uint8_t viewOverlayValid;
    uint32_t viewOverlayTickMs;
    observerLinkViewOverlay viewOverlay;
} core3LinkCore3Status;

void core3LinkCore3Init(uint32_t tickMs);
void core3LinkCore3SetVisionReady(uint8_t ready);

```

```

void core3LinkCore3Poll(uint32_t tickMs);
void core3LinkCore3FlushControl(uint32_t tickMs);
void core3LinkCore3FlushTx(uint32_t tickMs);
void core3LinkCore3RxIsr(void);
void core3LinkCore3GetStatus(core3LinkCore3Status *status);
uint32_t core3LinkCore3GetLocalBootGeneration(void);
uint8_t core3LinkCore3GetStagedConfig(observerLinkSharedConfig *config);
uint8_t core3LinkCore3MarkConfigApplied(uint32_t ownerSession,
    uint32_t revision,
    uint32_t hash,
    uint8_t appliedMode);
uint8_t core3LinkCore3TakeKvmControl(observerLinkKvmControl *control);
uint8_t core3LinkCore3TakeKvmEvent(observerLinkKvmEvent *event);
uint8_t core3LinkCore3QueueKvmAck(const observerLinkKvmAck *ack);
uint8_t core3LinkCore3QueueKvmMenuStatus(const observerLinkKvmMenuStatus *status);
uint8_t core3LinkCore3QueuePlanningTrackSnapshot(
    const observerLinkPlanningTrackSnapshot *snapshot);
uint8_t core3LinkCore3PublishHealth(const observerLinkHealth *health);
uint8_t core3LinkCore3GetViewOverlay(observerLinkViewOverlay *overlay,
    uint32_t tickMs, uint32_t timeoutMs, uint32_t *ageMs);
uint8_t core3LinkCore3QueueParamGlobalRequest(
    const paramSyncGlobalRequest *request);
uint8_t core3LinkCore3TakeParamGlobalState(paramSyncGlobalState *state);
uint8_t core3LinkCore3IsParamGlobalStateValid(void);

#endif /* OBSERVER_LINK_CORE3_H */

```

core3_cyt4bb7/project/code/comm/core3LinkProxy.c

SHA-256

:

2ddcd2b6abb0324e17db4701926ac4c2ba7fe48768b486aa67d28ce059f966bd

代码 (c)

```

#include "core3LinkCore3.h"

#include <string.h>

#include "core3CoreDataTransfer.h"
#include "configCore3Runtime.h"

#define CORE3_CORE_LOCAL_KVM_DEPTH (4u)
#define CORE3_CORE_REJECTION_BACKLOG_MAX (1024u)

typedef struct
{
    uint32_t ownerSession;
    uint32_t revision;
    uint32_t hash;
    uint32_t trackerFrameSeq;
    uint16_t count;
    uint16_t trackIds[CORE3_CORE_REJECTION_BACKLOG_MAX];
} core3LocalBorderRejectionBacklog;

static core3CoreLinkSnapshot g_link;
static core3CoreVisionSnapshot g_vision;
static uint32_t g_tickMs;
static uint32_t g_lastLinkPublishSeq;
static uint32_t g_lastLinkPublishTickMs;
static uint32_t g_lastLinkProbeTickMs;
static uint32_t g_lastParamGlobalStateSeq;
static uint32_t g_lastParamGlobalStateTickMs;
static uint32_t g_lastVisionPublishTickMs;
static uint32_t g_lastRemoteRxTick;
static uint32_t g_lastRemoteControlTick;
static uint32_t g_lastRemoteDiagTick;
static uint32_t g_lastRemoteOverlaySeq;
static uint32_t g_localRxTick;
static uint32_t g_localControlTick;
static uint32_t g_localDiagTick;
static uint32_t g_localOverlayTick;
static observerLinkKvmControl g_controlQueue[CORE3_CORE_LOCAL_KVM_DEPTH];
static observerLinkKvmEvent g_eventQueue[CORE3_CORE_LOCAL_KVM_DEPTH];
static uint8_t g_controlHead;
static uint8_t g_controlTail;

```

```

static uint8_t g_controlCount;
static uint8_t g_eventHead;
static uint8_t g_eventTail;
static uint8_t g_eventCount;
static uint8_t g_linkWasFresh;
static uint8_t g_linkBaselinePending;
static uint8_t g_visionDirty;
static core3LocalBorderRejectionBacklog g_pendingBorderRejection;
static uint8_t g_pendingBorderRejectionValid;

static void clearKvmQueues(void)
{
    g_controlHead = 0u;
    g_controlTail = 0u;
    g_controlCount = 0u;
    g_eventHead = 0u;
    g_eventTail = 0u;
    g_eventCount = 0u;
}

static void publishVision(void)
{
    g_vision.publishSeq++;
    g_vision.beginGeneration += 2u;
    if (g_vision.beginGeneration == 0u) g_vision.beginGeneration = 2u;
    g_vision.endGeneration = g_vision.beginGeneration;
    core3CoreDataWriteVision(&g_vision);
    g_lastVisionPublishTickMs = g_tickMs;
    g_visionDirty = 0u;
}

static void markVisionDirty(void)
{
    g_visionDirty = 1u;
}

static void clearPendingBorderRejections(void)
{
    memset(&g_pendingBorderRejection, 0, sizeof(g_pendingBorderRejection));
    g_pendingBorderRejectionValid = 0u;
}

static void flushPendingBorderRejections(void)
{
    while ((g_pendingBorderRejectionValid != 0u) &&
           (g_pendingBorderRejection.count != 0u))
    {
        core3CoreEgressEvent event;
        core3CoreBorderRejection *payload;
        uint16_t sent;
        memset(&event, 0, sizeof(event));
        event.kind = CORE3_CORE_EGRESS_BORDER_REJECTION;
        payload = &event.payload.borderRejection;
        payload->ownerSession = g_pendingBorderRejection.ownerSession;
        payload->revision = g_pendingBorderRejection.revision;
        payload->hash = g_pendingBorderRejection.hash;
        payload->trackerFrameSeq = g_pendingBorderRejection.trackerFrameSeq;
        sent = g_pendingBorderRejection.count;
        if (sent > CORE3_CORE_REJECTION_TRACK_MAX)
            sent = CORE3_CORE_REJECTION_TRACK_MAX;
        payload->rejectedCount = (uint8_t) sent;
        memcpy(payload->rejectedTrackIds, g_pendingBorderRejection.trackIds,
               (size_t) sent * sizeof(uint16_t));
        if (core3CoreDataPushEgress(&event) == 0u) break;
        g_pendingBorderRejection.count = (uint16_t)
            (g_pendingBorderRejection.count - sent);
        if (g_pendingBorderRejection.count != 0u)
            memmove(g_pendingBorderRejection.trackIds,
                    &g_pendingBorderRejection.trackIds[sent],
                    (size_t) g_pendingBorderRejection.count * sizeof(uint16_t));
    }
    if ((g_pendingBorderRejectionValid != 0u) &&
        (g_pendingBorderRejection.count == 0u))
        clearPendingBorderRejections();
}

static void flushVision(uint32_t tickMs)
{
    g_tickMs = tickMs;
}

```

```

if ((g_visionDirty != 0u) ||
    ((uint32_t) (tickMs - g_lastVisionPublishTickMs) >=
     CORE3_CORE_SHARED_HEARTBEAT_MS))
{
    publishVision();
}
}

static void refreshLink(uint32_t tickMs)
{
    core3CoreLinkSnapshot incoming;
    uint32_t bootGeneration;
    uint32_t publishSeq;
    uint8_t bindingChanged;
    uint8_t rebaseline;

    if (g_lastLinkProbeTickMs == tickMs) return;
    g_lastLinkProbeTickMs = tickMs;
    if (core3CoreDataProbeLink(&bootGeneration, &publishSeq) == 0u) return;
    if ((bootGeneration == g_link.bootGeneration) &&
        (publishSeq == g_lastLinkPublishSeq)) return;
    if (core3CoreDataTryReadLink(&incoming) == 0u) return;
    rebaseline = (uint8_t) ((g_linkBaselinePending != 0u) ||
        ((g_link.magic == CORE3_CORE_LINK_MAGIC) &&
         (incoming.bootGeneration != g_link.bootGeneration)));
    bindingChanged = (uint8_t) (
        (g_link.magic == CORE3_CORE_LINK_MAGIC) &&
        ((incoming.status.ownerSession != g_link.status.ownerSession) ||
         (incoming.status.bindingEpoch != g_link.status.bindingEpoch)));
    if ((g_link.magic != CORE3_CORE_LINK_MAGIC) ||
        (incoming.bootGeneration != g_link.bootGeneration))
    {
        if (g_link.magic == CORE3_CORE_LINK_MAGIC)
        {
            core3CoreDataNoteSessionReset();
        }
        core3CoreDataSyncIngressConsumer(incoming.bootGeneration);
        if (g_vision.appliedSeq != 0u)
        {
            g_vision.appliedSeq++;
            if (g_vision.appliedSeq == 0u) g_vision.appliedSeq = 1u;
            markVisionDirty();
        }
        clearKvmQueues();
        clearPendingBorderRejections();
        g_linkWasFresh = 0u;
        g_lastRemoteRxTick = 0u;
        g_lastRemoteControlTick = 0u;
        g_lastRemoteDiagTick = 0u;
        g_lastRemoteOverlaySeq = 0u;
    }
    else if (bindingChanged != 0u)
    {
        core3CoreDataSyncIngressConsumer(incoming.bootGeneration);
        g_vision.appliedOwnerSession = 0u;
        clearKvmQueues();
        clearPendingBorderRejections();
        g_lastRemoteRxTick = 0u;
        g_lastRemoteControlTick = 0u;
        g_lastRemoteDiagTick = 0u;
        g_lastRemoteOverlaySeq = 0u;
        g_lastParamGlobalStateSeq = incoming.paramGlobalStateSeq;
        g_lastParamGlobalStateTickMs =
            tickMs - CORE3_LINK_PARAM_GLOBAL_STATE_LOST_MS;
    }
    if (rebaseline != 0u)
    {
        /* Shared RAM is __no_init. The first coherent sample establishes only
         * producer identity and sequence; an advancing outer publication must
         * follow before it can become fresh. READY additionally requires an
         * advancing parameter-state sequence. */
        g_lastLinkPublishSeq = incoming.publishSeq;
        g_lastLinkPublishTickMs = tickMs - CORE3_CORE_SHARED_STALE_MS;
        g_lastParamGlobalStateSeq = incoming.paramGlobalStateSeq;
        g_lastParamGlobalStateTickMs =
            tickMs - CORE3_LINK_PARAM_GLOBAL_STATE_LOST_MS;
        g_lastRemoteRxTick = incoming.status.lastValidRxTickMs;
        g_lastRemoteControlTick = incoming.status.cameraControlTickMs;
        g_lastRemoteDiagTick = incoming.status.peerDiagnosticsTickMs;
    }
}

```

```

    g_lastRemoteOverlaySeq = (incoming.status.viewOverlayValid != 0u) ?
        incoming.status.viewOverlay.overlaySeq : 0u;
    g_localRxTick = tickMs;
    g_localControlTick = tickMs;
    g_localDiagTick = tickMs;
    g_localOverlayTick = tickMs;
    incoming.status.lastValidRxTickMs = g_localRxTick;
    incoming.status.cameraControlTickMs = g_localControlTick;
    incoming.status.peerDiagnosticsTickMs = g_localDiagTick;
    incoming.status.viewOverlayTickMs = g_localOverlayTick;
    g_link = incoming;
    g_linkBaselinePending = 0u;
    return;
}
if (incoming.publishSeq != g_lastLinkPublishSeq)
{
    g_lastLinkPublishSeq = incoming.publishSeq;
    g_lastLinkPublishTickMs = tickMs;
    if (incoming.paramGlobalStateSeq == 0u)
    {
        g_lastParamGlobalStateSeq = 0u;
        g_lastParamGlobalStateTickMs =
            tickMs - CORE3_LINK_PARAM_GLOBAL_STATE_LOST_MS;
    }
    else if ((incoming.paramGlobalStateSeq !=
        g_lastParamGlobalStateSeq) ||
        ((g_linkWasFresh == 0u) &&
        (paramSyncGlobalStateIsValid(
            &incoming.paramGlobalState) != 0u) &&
        ((incoming.paramGlobalState.flags &
            PARAM_SYNC_GLOBAL_FLAG_TERMINAL) != 0u)))
    {
        g_lastParamGlobalStateSeq = incoming.paramGlobalStateSeq;
        g_lastParamGlobalStateTickMs = tickMs;
    }
}
if (incoming.status.lastValidRxTickMs != g_lastRemoteRxTick)
{
    g_lastRemoteRxTick = incoming.status.lastValidRxTickMs;
    g_localRxTick = tickMs;
}
if (incoming.status.cameraControlTickMs != g_lastRemoteControlTick)
{
    g_lastRemoteControlTick = incoming.status.cameraControlTickMs;
    g_localControlTick = tickMs;
}
if (incoming.status.peerDiagnosticsTickMs != g_lastRemoteDiagTick)
{
    g_lastRemoteDiagTick = incoming.status.peerDiagnosticsTickMs;
    g_localDiagTick = tickMs;
}
if ((incoming.status.viewOverlayValid != 0u) &&
    (incoming.status.viewOverlay.overlaySeq != g_lastRemoteOverlaySeq))
{
    g_lastRemoteOverlaySeq = incoming.status.viewOverlay.overlaySeq;
    g_localOverlayTick = tickMs;
}
incoming.status.lastValidRxTickMs = g_localRxTick;
incoming.status.cameraControlTickMs = g_localControlTick;
incoming.status.peerDiagnosticsTickMs = g_localDiagTick;
incoming.status.viewOverlayTickMs = g_localOverlayTick;
g_link = incoming;
}

static uint8_t linkFresh(void)
{
    return (uint8_t) ((g_link.magic == CORE3_CORE_LINK_MAGIC) &&
        ((uint32_t) (g_tickMs - g_lastLinkPublishTickMs) <
            CORE3_CORE_SHARED_STALE_MS));
}

static uint8_t paramGlobalStateFresh(void)
{
    return (uint8_t) ((linkFresh() != 0u) &&
        (g_link.paramGlobalStateSeq != 0u) &&
        (g_link.paramGlobalStateSeq == g_lastParamGlobalStateSeq) &&
        ((uint32_t) (g_tickMs - g_lastParamGlobalStateTickMs) <
            CORE3_LINK_PARAM_GLOBAL_STATE_LOST_MS) &&
        (paramSyncGlobalStateIsValid(&g_link.paramGlobalState) != 0u));
}

```

```

}

void core3LinkCore3Init(uint32_t tickMs)
{
    core3CoreDataTimingInit();
    memset(&g_link, 0, sizeof(g_link));
    memset(&g_vision, 0, sizeof(g_vision));
    g_tickMs = tickMs;
    g_lastLinkProbeTickMs = UINT32_MAX;
    g_lastVisionPublishTickMs = tickMs;
    g_lastParamGlobalStateSeq = 0u;
    g_lastParamGlobalStateTickMs =
        tickMs - CORE3_LINK_PARAM_GLOBAL_STATE_LOST_MS;
    g_visionDirty = 0u;
    clearPendingBorderRejections();
    g_linkWasFresh = 0u;
    g_linkBaselinePending = 1u;
    clearKvmQueues();
    g_vision.magic = CORE3_CORE_VISION_MAGIC;
    g_vision.abiVersion = CORE3_CORE_ABI_VERSION;
    g_vision.payloadBytes = sizeof(g_vision);
    g_vision.bootGeneration = core3CoreDataMakeBootGeneration(0xC3 710 001u);
    core3CoreDataResetEgress(g_vision.bootGeneration);
    g_vision.beginGeneration = 2u;
    g_vision.endGeneration = 2u;
    publishVision();
    refreshLink(tickMs);
}

void core3LinkCore3SetVisionReady(uint8_t ready)
{
    ready = (uint8_t) (ready != 0u);
    if ((g_vision.visionReadySeq == 0u) || (g_vision.visionReady != ready))
    {
        g_vision.visionReady = ready;
        g_vision.visionReadySeq++;
        markVisionDirty();
    }
}

void core3LinkCore3Poll(uint32_t tickMs)
{
    core3CoreIngressEvent event;
    g_tickMs = tickMs;
    refreshLink(tickMs);
    if (linkFresh() == 0u)
    {
        if (g_linkWasFresh != 0u)
        {
            core3CoreDataNoteSharedState();
            core3CoreDataSyncIngressConsumer(g_link.bootGeneration);
            clearKvmQueues();
            clearPendingBorderRejections();
        }
        g_linkWasFresh = 0u;
        return;
    }
    g_linkWasFresh = 1u;
    flushPendingBorderRejections();
    while ((g_controlCount < CORE3_CORE_LOCAL_KVM_DEPTH) &&
        (g_eventCount < CORE3_CORE_LOCAL_KVM_DEPTH) &&
        (core3CoreDataPopIngress(&event, g_link.bootGeneration) != 0u))
    {
        if (event.kind == CORE3_CORE_INGRESS_KVM_CONTROL)
        {
            g_controlQueue[g_controlHead] = event.payload.control;
            g_controlHead = (uint8_t) ((g_controlHead + 1u) %
                CORE3_CORE_LOCAL_KVM_DEPTH);
            g_controlCount++;
        }
        else if (event.kind == CORE3_CORE_INGRESS_KVM_EVENT)
        {
            g_eventQueue[g_eventHead] = event.payload.event;
            g_eventHead = (uint8_t) ((g_eventHead + 1u) %
                CORE3_CORE_LOCAL_KVM_DEPTH);
            g_eventCount++;
        }
    }
}

```

```

void core3LinkCore3FlushControl(uint32_t tickMs)
{
    flushVision(tickMs);
}

void core3LinkCore3FlushTx(uint32_t tickMs)
{
    flushVision(tickMs);
}

void core3LinkCore3RxIsr(void)
{
}

void core3LinkCore3GetStatus(core3LinkCore3Status *status)
{
    if (status == NULL) return;
    if (linkFresh() != 0u)
    {
        *status = g_link.status;
    }
    else
    {
        memset(status, 0, sizeof(*status));
        status->stagedMode = 0xFFu;
        status->appliedMode = 0xFFu;
    }
}

uint32_t core3LinkCore3GetLocalBootGeneration(void)
{
    return g_vision.bootGeneration;
}

uint8_t core3LinkCore3GetStagedConfig(observerLinkSharedConfig *config)
{
    if ((config == NULL) || (linkFresh() == 0u) ||
        (g_link.stagedConfigSeq == 0u)) return 0u;
    *config = g_link.stagedConfig;
    return 1u;
}

uint8_t core3LinkCore3MarkConfigApplied(uint32_t ownerSession,
    uint32_t revision, uint32_t hash, uint8_t appliedMode)
{
    if ((linkFresh() == 0u) ||
        ((g_vision.appliedSeq != 0u) &&
         (g_link.appliedConsumedSeq != g_vision.appliedSeq) &&
         (g_vision.appliedOwnerSession == ownerSession))) return 0u;
    g_vision.appliedOwnerSession = ownerSession;
    g_vision.appliedRevision = revision;
    g_vision.appliedHash = hash;
    g_vision.appliedMode = appliedMode;
    g_vision.appliedSeq++;
    if (g_vision.appliedSeq == 0u) g_vision.appliedSeq = 1u;
    markVisionDirty();
    return 1u;
}

uint8_t core3LinkCore3TakeKvmControl(observerLinkKvmControl *control)
{
    if ((control == NULL) || (linkFresh() == 0u) ||
        (g_controlCount == 0u)) return 0u;
    *control = g_controlQueue[g_controlTail];
    g_controlTail = (uint8_t)((g_controlTail + 1u) %
        CORE3_CORE_LOCAL_KVM_DEPTH);
    g_controlCount--;
    return 1u;
}

uint8_t core3LinkCore3TakeKvmEvent(observerLinkKvmEvent *event)
{
    if ((event == NULL) || (linkFresh() == 0u) ||
        (g_eventCount == 0u)) return 0u;
    *event = g_eventQueue[g_eventTail];
    g_eventTail = (uint8_t)((g_eventTail + 1u) %
        CORE3_CORE_LOCAL_KVM_DEPTH);
    g_eventCount--;
}

```

```

    return 1u;
}

uint8_t core3LinkCore3QueueKvmAck(const observerLinkKvmAck *ack)
{
    core3CoreEgressEvent event;
    if (ack == NULL) return 0u;
    memset(&event, 0, sizeof(event));
    event.kind = CORE3_CORE_EGRESS_KVM_ACK;
    event.payload.ack = *ack;
    return core3CoreDataPushEgress(&event);
}

uint8_t core3LinkCore3QueueKvmMenuStatus(const observerLinkKvmMenuStatus *status)
{
    core3CoreEgressEvent event;
    if (status == NULL) return 0u;
    memset(&event, 0, sizeof(event));
    event.kind = CORE3_CORE_EGRESS_KVM_MENU;
    event.payload.menuStatus = *status;
    return core3CoreDataPushEgress(&event);
}

uint8_t core3LinkCore3QueuePlanningTrackSnapshot(
    const observerLinkPlanningTrackSnapshot *snapshot)
{
    core3CoreEgressEvent rejection;
    core3CoreBorderRejection *event;
    uint8_t i;
    if (snapshot == NULL) return 0u;

    flushPendingBorderRejections();
    if (g_pendingBorderRejectionValid != 0u)
    {
        if ((g_pendingBorderRejection.ownerSession != snapshot->ownerSession) ||
            (g_pendingBorderRejection.revision != snapshot->revision) ||
            (g_pendingBorderRejection.hash != snapshot->hash))
        {
            clearPendingBorderRejections();
        }
    }

    memset(&rejection, 0, sizeof(rejection));
    rejection.kind = CORE3_CORE_EGRESS_BORDER_REJECTION;
    event = &rejection.payload.borderRejection;
    event->ownerSession = snapshot->ownerSession;
    event->revision = snapshot->revision;
    event->hash = snapshot->hash;
    event->trackerFrameSeq = snapshot->trackerFrameSeq;
    for (i = 0u; i < snapshot->retainedCount; i++)
    {
        const observerLinkPlanningTrack *track = &snapshot->tracks[i];
        if ((track->flags &
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) == 0u) continue;
        if ((track->trackId == 0u) ||
            (event->rejectedCount >= CORE3_CORE_REJECTION_TRACK_MAX))
            return 0u;
        event->rejectedTrackIds[event->rejectedCount++] = track->trackId;
    }
    if (event->rejectedCount != 0u)
    {
        uint16_t uniqueAdded = 0u;
        if (g_pendingBorderRejectionValid == 0u)
        {
            g_pendingBorderRejection.ownerSession = event->ownerSession;
            g_pendingBorderRejection.revision = event->revision;
            g_pendingBorderRejection.hash = event->hash;
            g_pendingBorderRejection.trackerFrameSeq = event->trackerFrameSeq;
            g_pendingBorderRejectionValid = 1u;
        }
        for (i = 0u; i < event->rejectedCount; i++)
        {
            uint16_t j;
            for (j = 0u; j < g_pendingBorderRejection.count; j++)
                if (g_pendingBorderRejection.trackIds[j] ==
                    event->rejectedTrackIds[i]) break;
            if (j == g_pendingBorderRejection.count) uniqueAdded++;
        }
        if ((uint32_t) g_pendingBorderRejection.count + uniqueAdded >

```



```

    CORE3_CORE_REJECTION_BACKLOG_MAX) return 0u;
    for (i = 0u; i < event->rejectedCount; i++)
    {
        uint16_t j;
        for (j = 0u; j < g_pendingBorderRejection.count; j++)
            if (g_pendingBorderRejection.trackIds[j] ==
                event->rejectedTrackIds[i]) break;
        if (j == g_pendingBorderRejection.count)
            g_pendingBorderRejection.trackIds[
                g_pendingBorderRejection.count++] =
                    event->rejectedTrackIds[i];
    }
    if (observerLinkSequenceIsNewer(event->trackerFrameSeq,
        g_pendingBorderRejection.trackerFrameSeq) != 0u)
        g_pendingBorderRejection.trackerFrameSeq = event->trackerFrameSeq;
    flushPendingBorderRejections();
}
g_vision.planningSnapshot = *snapshot;
g_vision.planningSeq++;
markVisionDirty();
return 1u;
}

uint8_t core3LinkCore3PublishHealth(const observerLinkHealth *health)
{
    if ((health == NULL) ||
        (health->sourceId != OBSERVER_LINK_SOURCE_CORE3)) return 0u;
    g_vision.health = *health;
    g_vision.healthSeq++;
    if (g_vision.healthSeq == 0u) g_vision.healthSeq++;
    markVisionDirty();
    return 1u;
}

uint8_t core3LinkCore3GetViewOverlay(observerLinkViewOverlay *overlay,
    uint32_t tickMs, uint32_t timeoutMs, uint32_t *ageMs)
{
    uint32_t age;
    if ((overlay == NULL) || (linkFresh() == 0u) ||
        (g_link.status.viewOverlayValid == 0u)) return 0u;
    age = tickMs - g_localOverlayTick;
    if (ageMs != NULL) *ageMs = age;
    if (age > timeoutMs) return 0u;
    *overlay = g_link.status.viewOverlay;
    return 1u;
}

uint8_t core3LinkCore3QueueParamGlobalRequest(
    const paramSyncGlobalRequest *request)
{
    if ((request == NULL) || (paramSyncGlobalRequestIsValid(request) == 0u) ||
        (request->origin != OBSERVER_LINK_SOURCE_CORE3)) return 0u;
    if ((g_vision.paramGlobalRequestSeq != 0u) &&
        (g_vision.paramGlobalRequest.producerBootGeneration ==
            request->producerBootGeneration) &&
        (g_vision.paramGlobalRequest.sequence == request->sequence)) return 1u;
    g_vision.paramGlobalRequest = *request;
    g_vision.paramGlobalRequestSeq++;
    if (g_vision.paramGlobalRequestSeq == 0u)
        g_vision.paramGlobalRequestSeq = 1u;
    markVisionDirty();
    return 1u;
}

uint8_t core3LinkCore3TakeParamGlobalState(paramSyncGlobalState *state)
{
    if ((state == NULL) ||
        (core3LinkCore3IsParamGlobalStateValid() == 0u)) return 0u;
    *state = g_link.paramGlobalState;
    if (g_vision.paramGlobalStateConsumedSeq !=
        g_link.paramGlobalStateSeq)
    {
        g_vision.paramGlobalStateConsumedSeq =
            g_link.paramGlobalStateSeq;
        markVisionDirty();
    }
    return 1u;
}

```

```
uint8_t core3LinkCore3IsParamGlobalStateValid(void)
{
    if (paramGlobalStateFresh() == 0u) return 0u;
    if ((g_link.paramGlobalState.flags &
        PARAM_SYNC_GLOBAL_FLAG_TERMINAL) != 0u) return 1u;
    return (uint8_t) (g_link.status.online != 0u);
}
```

core3_cyt4bb7/project/code/comm/core3LinkSemantic.c

SHA-256

:

58792bd9fc36f13f5020e457b27d80ca7b08ba0c34aa2038357a8aacf7254f0d

代码 (c)

```
#include "observerLinkProtocol.h"

#include <string.h>

#include "crc32Ieee.h"

static void putU16(uint8_t *dst, uint16_t value)
{
    dst[0] = (uint8_t) value;
    dst[1] = (uint8_t) (value >> 8);
}

static void putFloat(uint8_t *dst, float value)
{
    uint32_t bits;
    memcpy(&bits, &value, sizeof(bits));
    dst[0] = (uint8_t) bits;
    dst[1] = (uint8_t) (bits >> 8);
    dst[2] = (uint8_t) (bits >> 16);
    dst[3] = (uint8_t) (bits >> 24);
}

static uint8_t finiteInRange(float value, float minimum, float maximum)
{
    if ((value != value) || (value > 1.0e30f) || (value < -1.0e30f))
    {
        return 0u;
    }
    return (uint8_t) ((value >= minimum) && (value <= maximum));
}

uint8_t observerLinkQuantizeSigned(float value, float scale,
    float minimum, float maximum, int16_t *quantized)
{
    float scaled;

    if ((quantized == NULL) ||
        (finiteInRange(value, minimum, maximum) == 0u) ||
        (finiteInRange(scale, 0.000001f, 1.0e30f) == 0u))
    {
        return 0u;
    }
    scaled = value * scale;
    if (finiteInRange(scaled, -32768.0f, 32767.0f) == 0u)
    {
        return 0u;
    }
    scaled += (scaled >= 0.0f) ? 0.5f : -0.5f;
    *quantized = (int16_t) scaled;
    return 1u;
}

static void packSharedConfig(const observerLinkSharedConfig *config,
    uint8_t payload[OBSERVER_LINK_SHARED_CONFIG_BYTES])
{
    uint8_t i;
    memset(payload, 0, OBSERVER_LINK_SHARED_CONFIG_BYTES);
    payload[8] = config->detectMode;
    payload[9] = config->maxBeacons;
    putU16(&payload[10], config->edgePredictionHoldMs);
}
```

```

    putU16(&payload[12], config->beaconSizeMin);
    putU16(&payload[14], config->beaconSizeMax);
    putFloat(&payload[16], config->beaconEccMax);
    payload[20] = config->beaconBrightnessMin;
    payload[21] = config->sourceId;
    putU16(&payload[22], config->beaconMaxEndpoints);
    putFloat(&payload[24], config->beaconFillMin);
    putFloat(&payload[28], config->beaconHuMax);
    putU16(&payload[32], config->beaconHuAreaMin);
    payload[34] = config->profileVersion;
    putFloat(&payload[36], config->trackMaxAssocDist);
    putFloat(&payload[40], config->trackSizeRatioMax);
    putU16(&payload[44], config->trackConfirmHits);
    putU16(&payload[46], config->trackDropMisses);
    putU16(&payload[48], config->trackCoastMisses);
    putFloat(&payload[52], config->trackPositionAlpha);
    putFloat(&payload[56], config->trackSizeAlpha);
    putFloat(&payload[60], config->trackAreaVelAlpha);
    putFloat(&payload[64], config->trackAssocScoreMax);
    putFloat(&payload[68], config->trackAssocAreaWeight);
    putFloat(&payload[72], config->trackAssocAreaVelWeight);
    for (i = 0u; i < 7u; i++)
    {
        putFloat(&payload[76u + (uint16_t) i * 4u],
            config->beaconHuTemplate[i]);
    }
    payload[104] = config->c1BorderEnterLeft;
    payload[105] = config->c1BorderEnterRight;
    payload[106] = config->c1BorderEnterTop;
    payload[107] = config->c1BorderEnterBottom;
    payload[108] = config->c1BorderExitLeft;
    payload[109] = config->c1BorderExitRight;
    payload[110] = config->c1BorderExitTop;
    payload[111] = config->c1BorderExitBottom;
    payload[112] = config->observerBorderEnterLeft;
    payload[113] = config->observerBorderEnterRight;
    payload[114] = config->observerBorderEnterTop;
    payload[115] = config->observerBorderEnterBottom;
    payload[116] = config->observerBorderExitLeft;
    payload[117] = config->observerBorderExitRight;
    payload[118] = config->observerBorderExitTop;
    payload[119] = config->observerBorderExitBottom;
    payload[120] = config->c1EdgeBandLeft;
    payload[121] = config->c1EdgeBandRight;
    payload[122] = config->c1EdgeBandTop;
    payload[123] = config->c1EdgeBandBottom;
    payload[124] = config->observerEdgeBandLeft;
    payload[125] = config->observerEdgeBandRight;
    payload[126] = config->observerEdgeBandTop;
    payload[127] = config->observerEdgeBandBottom;
}

uint32_t observerLinkSharedConfigHash(const observerLinkSharedConfig *config)
{
    uint8_t payload[OBSERVER_LINK_SHARED_CONFIG_BYTES];
    if (config == NULL) return 0u;
    packSharedConfig(config, payload);
    return crc32leecCalc(&payload[8], OBSERVER_LINK_SHARED_CONFIG_BYTES - 8u);
}

static uint8_t borderSideValid(uint8_t enter, uint8_t exit, uint8_t maximum)
{
    if ((enter == 0u) && (exit == 0u)) return 1u;
    return (uint8_t) ((enter >= 1u) && (enter <= maximum) && (exit < enter));
}

static uint8_t sharedConfigBorderValid(const observerLinkSharedConfig *config)
{
    return (uint8_t) (
        (config->edgePredictionHoldMs <= OBSERVER_LINK_EDGE_PREDICTION_HOLD_MS_MAX) &&
        (config->sourceId == OBSERVER_LINK_SOURCE_CORE3) &&
        (config->profileVersion == 1u) &&
        (config->c1EdgeBandLeft <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
        (config->c1EdgeBandRight <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
        (config->c1EdgeBandTop <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        (config->c1EdgeBandBottom <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        (config->observerEdgeBandLeft <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
        (config->observerEdgeBandRight <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
    );
}

```

```

        (config->observerEdgeBandTop <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        (config->observerEdgeBandBottom <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        borderSideValid(config->c1BorderEnterLeft, config->c1BorderExitLeft, 93u) &&
        borderSideValid(config->c1BorderEnterRight, config->c1BorderExitRight, 93u) &&
        borderSideValid(config->c1BorderEnterTop, config->c1BorderExitTop, 59u) &&
        borderSideValid(config->c1BorderEnterBottom, config->c1BorderExitBottom, 59u) &&
        borderSideValid(config->observerBorderEnterLeft, config->observerBorderExitLeft, 93u) &&
        borderSideValid(config->observerBorderEnterRight, config->observerBorderExitRight, 93u) &&
        borderSideValid(config->observerBorderEnterTop, config->observerBorderExitTop, 59u) &&
        borderSideValid(config->observerBorderEnterBottom, config->observerBorderExitBottom, 59u));
    }

uint8_t observerLinkSharedConfigIsValid(const observerLinkSharedConfig *config)
{
    uint8_t i;
    if ((config == NULL) || (config->revision == 0u) ||
        (config->hash != observerLinkSharedConfigHash(config)) ||
        (config->detectMode > 1u) ||
        (config->maxBeacons < 1u) || (config->maxBeacons > 32u) ||
        (sharedConfigBorderValid(config) == 0u) ||
        (config->beaconSizeMin < 1u) ||
        (config->beaconSizeMax <= config->beaconSizeMin) ||
        (config->beaconSizeMax > 60000u) ||
        (config->beaconMaxEndpoints > 200u) ||
        (config->beaconHuAreaMin < 1u) ||
        (config->beaconHuAreaMin > 100u) ||
        (config->trackConfirmHits < 1u) || (config->trackConfirmHits > 60u) ||
        (config->trackDropMisses < 1u) || (config->trackDropMisses > 60u) ||
        (config->trackCoastMisses < 1u) || (config->trackCoastMisses > 60u))
    {
        return 0u;
    }
    for (i = 0u; i < 7u; i++)
    {
        if (finiteInRange(config->beaconHuTemplate[i],
            -1000000.0f, 1000000.0f) == 0u)
        {
            return 0u;
        }
    }
}

return (uint8_t) (
    finiteInRange(config->beaconEccMax, 0.0f, 20.0f) &&
    finiteInRange(config->beaconFillMin, 0.0f, 1.0f) &&
    finiteInRange(config->beaconHuMax, 0.0f, 200.0f) &&
    finiteInRange(config->trackMaxAssocDist, 1.0f, 100.0f) &&
    finiteInRange(config->trackSizeRatioMax, 1.1f, 10.0f) &&
    finiteInRange(config->trackPositionAlpha, 0.01f, 1.0f) &&
    finiteInRange(config->trackSizeAlpha, 0.01f, 1.0f) &&
    finiteInRange(config->trackAreaVelAlpha, 0.01f, 1.0f) &&
    finiteInRange(config->trackAssocScoreMax, 0.1f, 10.0f) &&
    finiteInRange(config->trackAssocAreaWeight, 0.0f, 3.0f) &&
    finiteInRange(config->trackAssocAreaVelWeight, 0.0f, 3.0f));
}

```

core3_cyt4bb7/project/code/comm/observerLinkProtocol.c

SHA-256

:

e019e39df722dc745661f3a59859e5cbeb121fd354d13928214448bcab16b271

代码 (c)

```

#include "observerLinkProtocol.h"

#include <string.h>

#include "crc32Ieee.h"

static void putU16(uint8_t *dst, uint16_t value)
{
    dst[0] = (uint8_t) value;
    dst[1] = (uint8_t) (value >> 8);
}

static void putU32(uint8_t *dst, uint32_t value)
{

```

```

    dst[0] = (uint8_t) value;
    dst[1] = (uint8_t) (value >> 8);
    dst[2] = (uint8_t) (value >> 16);
    dst[3] = (uint8_t) (value >> 24);
}

static uint16_t getU16(const uint8_t *src)
{
    return (uint16_t) ((uint16_t) src[0] | ((uint16_t) src[1] << 8));
}

static uint32_t getU32(const uint8_t *src)
{
    return (uint32_t) src[0] |
        ((uint32_t) src[1] << 8) |
        ((uint32_t) src[2] << 16) |
        ((uint32_t) src[3] << 24);
}

static void putFloat(uint8_t *dst, float value)
{
    uint32_t bits;
    memcpy(&bits, &value, sizeof(bits));
    putU32(dst, bits);
}

static float getFloat(const uint8_t *src)
{
    uint32_t bits = getU32(src);
    float value;
    memcpy(&value, &bits, sizeof(value));
    return value;
}

static uint8_t finiteInRange(float value, float minimum, float maximum)
{
    if ((value != value) || (value > 1.0e30f) || (value < -1.0e30f))
    {
        return 0u;
    }
    return (uint8_t) ((value >= minimum) && (value <= maximum));
}

uint8_t observerLinkQuantizeSigned(float value, float scale,
    float minimum, float maximum, int16_t *quantized)
{
    float scaled;

    if ((quantized == NULL) ||
        (finiteInRange(value, minimum, maximum) == 0u) ||
        (finiteInRange(scale, 0.000001f, 1.0e30f) == 0u))
    {
        return 0u;
    }
    scaled = value * scale;
    if (finiteInRange(scaled, -32768.0f, 32767.0f) == 0u)
    {
        return 0u;
    }
    scaled += (scaled >= 0.0f) ? 0.5f : -0.5f;
    *quantized = (int16_t) scaled;
    return 1u;
}

static uint8_t updateExpected(observerLinkParser *parser)
{
    uint16_t payloadLength;

    parser->expected = 0u;
    if (parser->used < 8u)
    {
        return 1u;
    }
    payloadLength = getU16(&parser->bytes[6]);
    if (payloadLength > OBSERVER_LINK_MAX_PAYLOAD_BYTES)
    {
        return 0u;
    }
    parser->expected = (uint16_t) (OBSERVER_LINK_HEADER_BYTES +

```

```

        payloadLength +
        OBSERVER_LINK_CRC_BYTES);
    return 1u;
}

static void recoverParser(observerLinkParser *parser)
{
    uint16_t i;
    uint8_t found;

    for (;;)
    {
        found = 0u;
        for (i = 1u; (uint16_t) (i + 1u) < parser->used; i++)
        {
            if ((parser->bytes[i] == OBSERVER_LINK_SOF0) &&
                (parser->bytes[i + 1u] == OBSERVER_LINK_SOF1))
            {
                memmove(parser->bytes, &parser->bytes[i], parser->used - i);
                parser->used = (uint16_t) (parser->used - i);
                if (updateExpected(parser) != 0u)
                {
                    return;
                }
                parser->lengthErrors++;
                found = 1u;
                break;
            }
        }
        if (found != 0u)
        {
            continue;
        }
        if ((parser->used != 0u) &&
            (parser->bytes[parser->used - 1u] == OBSERVER_LINK_SOF0))
        {
            parser->bytes[0] = OBSERVER_LINK_SOF0;
            parser->used = 1u;
        }
        else
        {
            parser->used = 0u;
        }
        parser->expected = 0u;
        return;
    }
}

void observerLinkParserInit(observerLinkParser *parser)
{
    if (parser != NULL)
    {
        memset(parser, 0, sizeof(*parser));
    }
}

uint16_t observerLinkEncodeFrame(const observerLinkFrame *frame, uint8_t *output, uint16_t capacity)
{
    uint16_t total;
    uint32_t crc;

    if ((frame == NULL) || (output == NULL) ||
        (frame->payloadLength > OBSERVER_LINK_MAX_PAYLOAD_BYTES))
    {
        return 0u;
    }
    total = (uint16_t) (OBSERVER_LINK_HEADER_BYTES + frame->payloadLength +
        OBSERVER_LINK_CRC_BYTES);
    if (capacity < total)
    {
        return 0u;
    }

    output[0] = OBSERVER_LINK_SOF0;
    output[1] = OBSERVER_LINK_SOF1;
    output[2] = frame->major;
    output[3] = frame->minor;
    output[4] = frame->type;
    output[5] = frame->flags;

```

```

    putU16(&output[6], frame->payloadLength);
    putU32(&output[8], frame->bootSession);
    putU32(&output[12], frame->packetSeq);
    putU32(&output[16], frame->tickMs);
    if (frame->payloadLength != 0u)
    {
        memcpy(&output[OBSERVER_LINK_HEADER_BYTES],
            frame->payload,
            frame->payloadLength);
    }
    crc = crc32IeeeCalc(&output[2],
        (uint32_t) (OBSERVER_LINK_HEADER_BYTES - 2u + frame->payloadLength));
    putU32(&output[OBSERVER_LINK_HEADER_BYTES + frame->payloadLength], crc);
    return total;
}

uint8_t observerLinkParserFeed(observerLinkParser *parser, uint8_t byte, observerLinkFrame *frame)
{
    uint16_t payloadLength;
    uint32_t expectedCrc;
    uint32_t actualCrc;

    if ((parser == NULL) || (frame == NULL))
    {
        return 0u;
    }

    if (parser->used == 0u)
    {
        if (byte == OBSERVER_LINK_SOF0)
        {
            {
                parser->bytes[0] = byte;
                parser->used = 1u;
            }
            return 0u;
        }
    }
    if (parser->used == 1u)
    {
        if (byte == OBSERVER_LINK_SOF1)
        {
            {
                parser->bytes[1] = byte;
                parser->used = 2u;
            }
        }
        else if (byte != OBSERVER_LINK_SOF0)
        {
            {
                parser->used = 0u;
            }
            return 0u;
        }
    }

    if (parser->used >= OBSERVER_LINK_MAX_FRAME_BYTES)
    {
        parser->lengthErrors++;
        parser->used = 0u;
        parser->expected = 0u;
        return 0u;
    }
    parser->bytes[parser->used++] = byte;

    if ((parser->used >= 8u) && (parser->expected == 0u))
    {
        if (updateExpected(parser) == 0u)
        {
            {
                parser->lengthErrors++;
                recoverParser(parser);
                return 0u;
            }
        }
    }
    if ((parser->expected == 0u) || (parser->used < parser->expected))
    {
        return 0u;
    }
}

validateFrame:
payloadLength = getU16(&parser->bytes[6]);
expectedCrc = getU32(&parser->bytes[OBSERVER_LINK_HEADER_BYTES + payloadLength]);
actualCrc = crc32IeeeCalc(&parser->bytes[2],
    (uint32_t) (OBSERVER_LINK_HEADER_BYTES - 2u + payloadLength));
if (expectedCrc != actualCrc)

```

```

{
    parser->crcErrors++;
    recoverParser(parser);
    if ((parser->expected != 0u) && (parser->used >= parser->expected))
    {
        goto validateFrame;
    }
    return 0u;
}

frame->major = parser->bytes[2];
frame->minor = parser->bytes[3];
frame->type = parser->bytes[4];
frame->flags = parser->bytes[5];
frame->payloadLength = payloadLength;
frame->bootSession = getU32(&parser->bytes[8]);
frame->packetSeq = getU32(&parser->bytes[12]);
frame->tickMs = getU32(&parser->bytes[16]);
if (payloadLength != 0u)
{
    memcpy(frame->payload,
           &parser->bytes[OBSERVER_LINK_HEADER_BYTES],
           payloadLength);
}
if (frame->major != OBSERVER_LINK_PROTOCOL_MAJOR)
{
    parser->versionErrors++;
}
parser->validFrames++;
if (parser->used > parser->expected)
{
    uint16_t remaining = (uint16_t) (parser->used - parser->expected);
    memmove(parser->bytes, &parser->bytes[parser->expected], remaining);
    parser->used = remaining;
    if (updateExpected(parser) == 0u)
    {
        parser->lengthErrors++;
        recoverParser(parser);
    }
}
else
{
    parser->used = 0u;
    parser->expected = 0u;
}
return 1u;
}

uint8_t observerLinkEncodeHello(uint8_t role, uint32_t capabilities, uint8_t *payload, uint16_t *length)
{
    if ((payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    payload[0] = role;
    payload[1] = OBSERVER_LINK_PROTOCOL_MAJOR;
    payload[2] = OBSERVER_LINK_PROTOCOL_MINOR;
    payload[3] = 0u;
    putU32(&payload[4], capabilities);
    *length = 8u;
    return 1u;
}

uint8_t observerLinkDecodeHello(const uint8_t *payload, uint16_t length, uint8_t *role, uint32_t *capabilities)
{
    if ((payload == NULL) || (role == NULL) || (capabilities == NULL) ||
        (length != 8u))
    {
        return 0u;
    }
    if ((payload[1] != OBSERVER_LINK_PROTOCOL_MAJOR) ||
        (payload[2] != OBSERVER_LINK_PROTOCOL_MINOR))
    {
        return 0u;
    }
    *role = payload[0];
    *capabilities = getU32(&payload[4]);
    return 1u;
}

```



```

uint8_t observerLinkEncodeSharedConfig(const observerLinkSharedConfig *config, uint8_t *payload, uint16_t *length)
{
    uint8_t i;
    if ((config == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    memset(payload, 0, OBSERVER_LINK_SHARED_CONFIG_BYTES);
    putU32(&payload[0], config->revision);
    putU32(&payload[4], config->hash);
    payload[8] = config->detectMode;
    payload[9] = config->maxBeacons;
    putU16(&payload[10], config->edgePredictionHoldMs);
    putU16(&payload[12], config->beaconSizeMin);
    putU16(&payload[14], config->beaconSizeMax);
    putFloat(&payload[16], config->beaconEccMax);
    payload[20] = config->beaconBrightnessMin;
    payload[21] = config->sourceId;
    putU16(&payload[22], config->beaconMaxEndpoints);
    putFloat(&payload[24], config->beaconFillMin);
    putFloat(&payload[28], config->beaconHuMax);
    putU16(&payload[32], config->beaconHuAreaMin);
    payload[34] = config->profileVersion;
    putFloat(&payload[36], config->trackMaxAssocDist);
    putFloat(&payload[40], config->trackSizeRatioMax);
    putU16(&payload[44], config->trackConfirmHits);
    putU16(&payload[46], config->trackDropMisses);
    putU16(&payload[48], config->trackCoastMisses);
    putFloat(&payload[52], config->trackPositionAlpha);
    putFloat(&payload[56], config->trackSizeAlpha);
    putFloat(&payload[60], config->trackAreaVelAlpha);
    putFloat(&payload[64], config->trackAssocScoreMax);
    putFloat(&payload[68], config->trackAssocAreaWeight);
    putFloat(&payload[72], config->trackAssocAreaVelWeight);
    for (i = 0u; i < 7u; i++)
    {
        putFloat(&payload[76u + (uint16_t) i * 4u], config->beaconHuTemplate[i]);
    }
    payload[104] = config->c1BorderEnterLeft;
    payload[105] = config->c1BorderEnterRight;
    payload[106] = config->c1BorderEnterTop;
    payload[107] = config->c1BorderEnterBottom;
    payload[108] = config->c1BorderExitLeft;
    payload[109] = config->c1BorderExitRight;
    payload[110] = config->c1BorderExitTop;
    payload[111] = config->c1BorderExitBottom;
    payload[112] = config->observerBorderEnterLeft;
    payload[113] = config->observerBorderEnterRight;
    payload[114] = config->observerBorderEnterTop;
    payload[115] = config->observerBorderEnterBottom;
    payload[116] = config->observerBorderExitLeft;
    payload[117] = config->observerBorderExitRight;
    payload[118] = config->observerBorderExitTop;
    payload[119] = config->observerBorderExitBottom;
    payload[120] = config->c1EdgeBandLeft;
    payload[121] = config->c1EdgeBandRight;
    payload[122] = config->c1EdgeBandTop;
    payload[123] = config->c1EdgeBandBottom;
    payload[124] = config->observerEdgeBandLeft;
    payload[125] = config->observerEdgeBandRight;
    payload[126] = config->observerEdgeBandTop;
    payload[127] = config->observerEdgeBandBottom;
    *length = OBSERVER_LINK_SHARED_CONFIG_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeSharedConfig(const uint8_t *payload, uint16_t length, observerLinkSharedConfig *config)
{
    uint8_t i;
    if ((payload == NULL) || (config == NULL) ||
        (length != OBSERVER_LINK_SHARED_CONFIG_BYTES))
    {
        return 0u;
    }
    memset(config, 0, sizeof(*config));
    config->revision = getU32(&payload[0]);
    config->hash = getU32(&payload[4]);
    config->detectMode = payload[8];

```

```

config->maxBeacons = payload[9];
config->edgePredictionHoldMs = getU16(&payload[10]);
config->beaconSizeMin = getU16(&payload[12]);
config->beaconSizeMax = getU16(&payload[14]);
config->beaconEccMax = getFloat(&payload[16]);
config->beaconBrightnessMin = payload[20];
config->sourceId = payload[21];
config->beaconMaxEndpoints = getU16(&payload[22]);
config->beaconFillMin = getFloat(&payload[24]);
config->beaconHuMax = getFloat(&payload[28]);
config->beaconHuAreaMin = getU16(&payload[32]);
config->profileVersion = payload[34];
config->trackMaxAssocDist = getFloat(&payload[36]);
config->trackSizeRatioMax = getFloat(&payload[40]);
config->trackConfirmHits = getU16(&payload[44]);
config->trackDropMisses = getU16(&payload[46]);
config->trackCoastMisses = getU16(&payload[48]);
config->trackPositionAlpha = getFloat(&payload[52]);
config->trackSizeAlpha = getFloat(&payload[56]);
config->trackAreaVelAlpha = getFloat(&payload[60]);
config->trackAssocScoreMax = getFloat(&payload[64]);
config->trackAssocAreaWeight = getFloat(&payload[68]);
config->trackAssocAreaVelWeight = getFloat(&payload[72]);
for (i = 0u; i < 7u; i++)
{
    config->beaconHuTemplate[i] = getFloat(&payload[76u + (uint16_t) i * 4u]);
}
config->c1BorderEnterLeft = payload[104];
config->c1BorderEnterRight = payload[105];
config->c1BorderEnterTop = payload[106];
config->c1BorderEnterBottom = payload[107];
config->c1BorderExitLeft = payload[108];
config->c1BorderExitRight = payload[109];
config->c1BorderExitTop = payload[110];
config->c1BorderExitBottom = payload[111];
config->observerBorderEnterLeft = payload[112];
config->observerBorderEnterRight = payload[113];
config->observerBorderEnterTop = payload[114];
config->observerBorderEnterBottom = payload[115];
config->observerBorderExitLeft = payload[116];
config->observerBorderExitRight = payload[117];
config->observerBorderExitTop = payload[118];
config->observerBorderExitBottom = payload[119];
config->c1EdgeBandLeft = payload[120];
config->c1EdgeBandRight = payload[121];
config->c1EdgeBandTop = payload[122];
config->c1EdgeBandBottom = payload[123];
config->observerEdgeBandLeft = payload[124];
config->observerEdgeBandRight = payload[125];
config->observerEdgeBandTop = payload[126];
config->observerEdgeBandBottom = payload[127];
return 1u;
}

static uint8_t borderSideValid(uint8_t enter, uint8_t exit, uint8_t maxVal)
{
    if ((enter == 0u) && (exit == 0u)) return 1u;
    return (uint8_t) ((enter >= 1u) && (enter <= maxVal) && (exit < enter));
}

static uint8_t sharedConfigBorderValid(const observerLinkSharedConfig *config)
{
    if (config == NULL) return 0u;
    return (uint8_t) (
        (config->edgePredictionHoldMs <= OBSERVER_LINK_EDGE_PREDICTION_HOLD_MS_MAX) &&
        ((config->sourceId == OBSERVER_LINK_SOURCE_CORE3) ||
         (config->sourceId == OBSERVER_LINK_SOURCE_CORE4)) &&
        (config->profileVersion == 1u) &&
        (config->c1EdgeBandLeft <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
        (config->c1EdgeBandRight <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
        (config->c1EdgeBandTop <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        (config->c1EdgeBandBottom <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        (config->observerEdgeBandLeft <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
        (config->observerEdgeBandRight <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
        (config->observerEdgeBandTop <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        (config->observerEdgeBandBottom <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        borderSideValid(config->c1BorderEnterLeft, config->c1BorderExitLeft, 93u) &&
        borderSideValid(config->c1BorderEnterRight, config->c1BorderExitRight, 93u) &&
        borderSideValid(config->c1BorderEnterTop, config->c1BorderExitTop, 59u) &&
    )
}

```

```

        borderSideValid(config->c1BorderEnterBottom, config->c1BorderExitBottom, 59u) &&
        borderSideValid(config->observerBorderEnterLeft, config->observerBorderExitLeft, 93u) &&
        borderSideValid(config->observerBorderEnterRight, config->observerBorderExitRight, 93u) &&
        borderSideValid(config->observerBorderEnterTop, config->observerBorderExitTop, 59u) &&
        borderSideValid(config->observerBorderEnterBottom, config->observerBorderExitBottom, 59u));
    }

uint32_t observerLinkSharedConfigHash(const observerLinkSharedConfig *config)
{
    uint8_t payload[OBSERVER_LINK_SHARED_CONFIG_BYTES];
    uint16_t length;

    if ((config == NULL) ||
        (observerLinkEncodeSharedConfig(config, payload, &length) == 0u))
    {
        return 0u;
    }
    return crc32IeeeCalc(&payload[8], (uint32_t) (length - 8u));
}

uint8_t observerLinkSharedConfigIsValid(const observerLinkSharedConfig *config)
{
    uint8_t i;
    if ((config == NULL) || (config->revision == 0u) ||
        (config->hash != observerLinkSharedConfigHash(config)) ||
        (config->detectMode > 1u) ||
        (config->maxBeacons < 1u) || (config->maxBeacons > 32u) ||
        (sharedConfigBorderValid(config) == 0u) ||
        (config->beaconSizeMin < 1u) ||
        (config->beaconSizeMax <= config->beaconSizeMin) ||
        (config->beaconSizeMax > 60000u) ||
        (config->beaconMaxEndpoints > 200u) ||
        (config->beaconHuAreaMin < 1u) ||
        (config->beaconHuAreaMin > 100u) ||
        (config->trackConfirmHits < 1u) || (config->trackConfirmHits > 60u) ||
        (config->trackDropMisses < 1u) || (config->trackDropMisses > 60u) ||
        (config->trackCoastMisses < 1u) || (config->trackCoastMisses > 60u))
    {
        return 0u;
    }
    for (i = 0u; i < 7u; i++)
    {
        if (finiteInRange(config->beaconHuTemplate[i], -1000000.0f, 1000000.0f) == 0u)
        {
            return 0u;
        }
    }
    return (uint8_t) (
        finiteInRange(config->beaconEccMax, 0.0f, 20.0f) &&
        finiteInRange(config->beaconFillMin, 0.0f, 1.0f) &&
        finiteInRange(config->beaconHuMax, 0.0f, 200.0f) &&
        finiteInRange(config->trackMaxAssocDist, 1.0f, 100.0f) &&
        finiteInRange(config->trackSizeRatioMax, 1.1f, 10.0f) &&
        finiteInRange(config->trackPositionAlpha, 0.01f, 1.0f) &&
        finiteInRange(config->trackSizeAlpha, 0.01f, 1.0f) &&
        finiteInRange(config->trackAreaVelAlpha, 0.01f, 1.0f) &&
        finiteInRange(config->trackAssocScoreMax, 0.1f, 10.0f) &&
        finiteInRange(config->trackAssocAreaWeight, 0.0f, 3.0f) &&
        finiteInRange(config->trackAssocAreaVelWeight, 0.0f, 3.0f));
}

uint8_t observerLinkEncodeConfigAck(const observerLinkConfigAck *ack, uint8_t *payload, uint16_t *length)
{
    if ((ack == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], ack->ownerSession);
    putU32(&payload[4], ack->revision);
    putU32(&payload[8], ack->hash);
    payload[12] = ack->desiredMode;
    payload[13] = ack->appliedMode;
    payload[14] = ack->status;
    payload[15] = ack->reason;
    *length = OBSERVER_LINK_CONFIG_ACK_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeConfigAck(const uint8_t *payload, uint16_t length, observerLinkConfigAck *ack)

```

```

{
    if ((payload == NULL) || (ack == NULL) ||
        (length != OBSERVER_LINK_CONFIG_ACK_BYTES))
    {
        return 0u;
    }
    ack->ownerSession = getU32(&payload[0]);
    ack->revision = getU32(&payload[4]);
    ack->hash = getU32(&payload[8]);
    ack->desiredMode = payload[12];
    ack->appliedMode = payload[13];
    ack->status = payload[14];
    ack->reason = payload[15];
    return 1u;
}

uint8_t observerLinkEncodeDiagnostics(const observerLinkDiagnostics *diagnostics, uint8_t *payload, uint16_t *length)
{
    if ((diagnostics == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], diagnostics->validFrames);
    putU32(&payload[4], diagnostics->crcErrors);
    putU32(&payload[8], diagnostics->lengthErrors);
    putU32(&payload[12], diagnostics->versionErrors);
    putU32(&payload[16], diagnostics->rxOverflowErrors);
    putU32(&payload[20], diagnostics->rxFrameErrors);
    putU32(&payload[24], diagnostics->rxParityErrors);
    putU32(&payload[28], diagnostics->txDeferred);
    *length = OBSERVER_LINK_DIAGNOSTICS_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeDiagnostics(const uint8_t *payload, uint16_t length, observerLinkDiagnostics *diagnostics)
{
    if ((payload == NULL) || (diagnostics == NULL) ||
        (length != OBSERVER_LINK_DIAGNOSTICS_BYTES))
    {
        return 0u;
    }
    diagnostics->validFrames = getU32(&payload[0]);
    diagnostics->crcErrors = getU32(&payload[4]);
    diagnostics->lengthErrors = getU32(&payload[8]);
    diagnostics->versionErrors = getU32(&payload[12]);
    diagnostics->rxOverflowErrors = getU32(&payload[16]);
    diagnostics->rxFrameErrors = getU32(&payload[20]);
    diagnostics->rxParityErrors = getU32(&payload[24]);
    diagnostics->txDeferred = getU32(&payload[28]);
    return 1u;
}

uint8_t observerLinkEncodeKvmControl(const observerLinkKvmControl *control, uint8_t *payload, uint16_t *length)
{
    if ((control == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], control->targetSession);
    putU32(&payload[4], control->routeSession);
    putU32(&payload[8], control->routeEpoch);
    putU32(&payload[12], control->leaseMs);
    payload[16] = control->target;
    *length = OBSERVER_LINK_KVM_CONTROL_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeKvmControl(const uint8_t *payload, uint16_t length, observerLinkKvmControl *control)
{
    if ((payload == NULL) || (control == NULL) || (length != OBSERVER_LINK_KVM_CONTROL_BYTES))
    {
        return 0u;
    }
    memset(control, 0, sizeof(*control));
    control->targetSession = getU32(&payload[0]);
    control->routeSession = getU32(&payload[4]);
    control->routeEpoch = getU32(&payload[8]);
    control->leaseMs = getU32(&payload[12]);
    control->target = payload[16];
}

```

```

    return 1u;
}

uint8_t observerLinkEncodeKvmEvent(const observerLinkKvmEvent *event, uint8_t *payload, uint16_t *length)
{
    if ((event == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], event->targetSession);
    putU32(&payload[4], event->routeSession);
    putU32(&payload[8], event->routeEpoch);
    putU32(&payload[12], event->eventSeq);
    payload[16] = event->target;
    payload[17] = event->keyId;
    payload[18] = event->keyState;
    payload[19] = event->flags;
    *length = OBSERVER_LINK_KVM_EVENT_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeKvmEvent(const uint8_t *payload, uint16_t length, observerLinkKvmEvent *event)
{
    if ((payload == NULL) || (event == NULL) || (length != OBSERVER_LINK_KVM_EVENT_BYTES))
    {
        return 0u;
    }
    memset(event, 0, sizeof(*event));
    event->targetSession = getU32(&payload[0]);
    event->routeSession = getU32(&payload[4]);
    event->routeEpoch = getU32(&payload[8]);
    event->eventSeq = getU32(&payload[12]);
    event->target = payload[16];
    event->keyId = payload[17];
    event->keyState = payload[18];
    event->flags = payload[19];
    return 1u;
}

uint8_t observerLinkEncodeKvmAck(const observerLinkKvmAck *ack, uint8_t *payload, uint16_t *length)
{
    if ((ack == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], ack->targetSession);
    putU32(&payload[4], ack->routeSession);
    putU32(&payload[8], ack->routeEpoch);
    putU32(&payload[12], ack->eventSeq);
    putU32(&payload[16], ack->expectedSeq);
    payload[20] = ack->target;
    payload[21] = ack->result;
    payload[22] = ack->menuState;
    payload[23] = ack->flags;
    *length = OBSERVER_LINK_KVM_ACK_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeKvmAck(const uint8_t *payload, uint16_t length, observerLinkKvmAck *ack)
{
    if ((payload == NULL) || (ack == NULL) || (length != OBSERVER_LINK_KVM_ACK_BYTES))
    {
        return 0u;
    }
    memset(ack, 0, sizeof(*ack));
    ack->targetSession = getU32(&payload[0]);
    ack->routeSession = getU32(&payload[4]);
    ack->routeEpoch = getU32(&payload[8]);
    ack->eventSeq = getU32(&payload[12]);
    ack->expectedSeq = getU32(&payload[16]);
    ack->target = payload[20];
    ack->result = payload[21];
    ack->menuState = payload[22];
    ack->flags = payload[23];
    return 1u;
}

uint8_t observerLinkEncodeKvmMenuStatus(const observerLinkKvmMenuStatus *status, uint8_t *payload, uint16_t *length)
{

```

```

if ((status == NULL) || (payload == NULL) || (length == NULL))
{
    return 0u;
}
putU32(&payload[0], status->targetSession);
putU32(&payload[4], status->routeSession);
putU32(&payload[8], status->routeEpoch);
putU32(&payload[12], status->lastEventSeq);
putU32(&payload[16], status->screenRevision);
payload[20] = status->target;
payload[21] = status->menuState;
payload[22] = status->flags;
payload[23] = status->lastAckResult;
*length = OBSERVER_LINK_KVM_MENU_STATUS_BYTES;
return 1u;
}

uint8_t observerLinkDecodeKvmMenuStatus(const uint8_t *payload, uint16_t length, observerLinkKvmMenuStatus *status)
{
    if ((payload == NULL) || (status == NULL) || (length != OBSERVER_LINK_KVM_MENU_STATUS_BYTES))
    {
        return 0u;
    }
    memset(status, 0, sizeof(*status));
    status->targetSession = getU32(&payload[0]);
    status->routeSession = getU32(&payload[4]);
    status->routeEpoch = getU32(&payload[8]);
    status->lastEventSeq = getU32(&payload[12]);
    status->screenRevision = getU32(&payload[16]);
    status->target = payload[20];
    status->menuState = payload[21];
    status->flags = payload[22];
    status->lastAckResult = payload[23];
    return 1u;
}

static uint8_t controlStatusIsValid(const observerLinkControlStatus *status)
{
    uint8_t statusValid;
    uint8_t decisionValid;
    uint8_t barrier;
    uint8_t dispatch;
    uint8_t core0LinkUp;
    uint8_t core2LinkUp;
    uint8_t requiredVision;

    if ((status == NULL) || (status->detectMode > 1u) ||
        (status->protocolCompatible > 1u) || (status->algorithmsReady > 1u) ||
        (status->configCompatible > 1u) ||
        (status->cameraControlState > OBSERVER_LINK_CAMERA_CONTROL_RUN) ||
        ((status->activeSource != OBSERVER_LINK_SOURCE_NONE) &&
         (status->activeSource != OBSERVER_LINK_SOURCE_CORE1) &&
         (status->activeSource != OBSERVER_LINK_SOURCE_CORE3) &&
         (status->activeSource != OBSERVER_LINK_SOURCE_CORE4)) ||
        (status->controlClass > OBSERVER_LINK_CONTROL_CLASS_SEARCH) ||
        ((status->navState != 0u) && (status->navState != 2u) &&
         (status->navState != 3u) && (status->navState != 4u)) ||
        (status->systemPhase > OBSERVER_LINK_SYSTEM_PHASE_RUNNING) ||
        ((status->systemFlags & 0x80u) != 0u) ||
        (status->primaryReason > OBSERVER_LINK_REASON_LANDING) ||
        ((status->desiredTarget != OBSERVER_LINK_KVM_TARGET_CORE0) &&
         (status->desiredTarget != OBSERVER_LINK_KVM_TARGET_CORE1) &&
         (status->desiredTarget != OBSERVER_LINK_KVM_TARGET_CORE3) &&
         (status->desiredTarget != OBSERVER_LINK_KVM_TARGET_CORE4) &&
         (status->desiredTarget != OBSERVER_LINK_KVM_TARGET_NONE)) ||
        ((status->ownerTarget != OBSERVER_LINK_KVM_TARGET_CORE0) &&
         (status->ownerTarget != OBSERVER_LINK_KVM_TARGET_CORE1) &&
         (status->ownerTarget != OBSERVER_LINK_KVM_TARGET_CORE3) &&
         (status->ownerTarget != OBSERVER_LINK_KVM_TARGET_CORE4) &&
         (status->ownerTarget != OBSERVER_LINK_KVM_TARGET_NONE)) ||
        (status->targetX >= 188u) ||
        (status->targetY >= 120u) || (status->targetSize > 22 560u))
    {
        return 0u;
    }

    statusValid = (uint8_t) ((status->flags &
        OBSERVER_LINK_CONTROL_FLAG_STATUS_VALID) != 0u);
    decisionValid = (uint8_t) ((status->flags &

```

```

    OBSERVER_LINK_CONTROL_FLAG_DECISION_VALID) != 0u);
barrier = (uint8_t)((status->flags &
    OBSERVER_LINK_CONTROL_FLAG_BARRIER_REQUIRED) != 0u);
dispatch = (uint8_t)((status->flags &
    OBSERVER_LINK_CONTROL_FLAG_DISPATCH_ENABLED) != 0u);
core0LinkUp = (uint8_t)((status->flags &
    OBSERVER_LINK_CONTROL_FLAG_CORE0_LINK_UP) != 0u);
core2LinkUp = (uint8_t)((status->flags &
    OBSERVER_LINK_CONTROL_FLAG_CORE2_LINK_UP) != 0u);
requiredVision = (uint8_t)((status->flags &
    OBSERVER_LINK_CONTROL_FLAG_REQUIRED_VISION) != 0u);

if (((status->systemFlags & OBSERVER_LINK_SYSTEM_FLAG_STANDALONE) != 0u) !=
    (status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_STANDALONE)) ||
    ((status->systemFlags & OBSERVER_LINK_SYSTEM_FLAG_REJOIN) != 0u) !=
    (status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_REJOIN)) ||
    ((status->systemFlags & OBSERVER_LINK_SYSTEM_FLAG_LANDING) != 0u) &&
    ((status->systemFlags & OBSERVER_LINK_SYSTEM_FLAG_LAUNCH_ELIGIBLE) != 0u)))
{
    return 0u;
}

if (statusValid == 0u)
{
    return (uint8_t)((status->cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_SAFE) &&
        (status->activeSource == OBSERVER_LINK_SOURCE_NONE) &&
        (status->controlClass == OBSERVER_LINK_CONTROL_CLASS_NONE) &&
        (status->navState == 0u) && (status->beaconCount == 0u) &&
        (status->targetX == 0u) && (status->targetY == 0u) &&
        (status->targetSize == 0u) && (status->errXQ4 == 0) &&
        (status->errYQ4 == 0) && (status->flags == 0u));
}

if ((status->cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_SAFE) ||
    ((status->cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_RUN) !=
    (requiredVision != 0u)) ||
    ((core2LinkUp != 0u) && (core0LinkUp == 0u)) ||
    ((decisionValid != 0u) &&
    (status->controlClass == OBSERVER_LINK_CONTROL_CLASS_NONE)) ||
    ((status->controlClass == OBSERVER_LINK_CONTROL_CLASS_NONE) &&
    (status->activeSource != OBSERVER_LINK_SOURCE_NONE)) ||
    ((status->controlClass == OBSERVER_LINK_CONTROL_CLASS_CORE1_TRACK) &&
    (status->activeSource != OBSERVER_LINK_SOURCE_CORE1)) ||
    ((status->controlClass == OBSERVER_LINK_CONTROL_CLASS_OBSERVER_TRACK) &&
    (status->activeSource != OBSERVER_LINK_SOURCE_CORE3) &&
    (status->activeSource != OBSERVER_LINK_SOURCE_CORE4)) ||
    ((status->controlClass == OBSERVER_LINK_CONTROL_CLASS_SEARCH) &&
    (status->activeSource != OBSERVER_LINK_SOURCE_NONE)) ||
    ((dispatch != 0u) && ((decisionValid == 0u) || (barrier != 0u)) ||
    (status->controlClass == OBSERVER_LINK_CONTROL_CLASS_NONE))))
{
    return 0u;
}

return 1u;
}

uint8_t observerLinkEncodeControlStatus(const observerLinkControlStatus *status,
uint8_t *payload, uint16_t *length)
{
    if ((payload == NULL) || (length == NULL) ||
        (controlStatusIsValid(status) == 0u))
    {
        return 0u;
    }
    payload[0] = status->detectMode;
    payload[1] = status->protocolCompatible;
    payload[2] = status->algorithmsReady;
    payload[3] = status->configCompatible;
    payload[4] = status->cameraControlState;
    payload[5] = status->activeSource;
    payload[6] = status->controlClass;
    payload[7] = status->flags;
    payload[8] = status->navState;
    payload[9] = status->beaconCount;
    putU16(&payload[10], status->targetX);
    putU16(&payload[12], status->targetY);
    putU16(&payload[14], status->targetSize);
    putU16(&payload[16], (uint16_t) status->errXQ4);
    putU16(&payload[18], (uint16_t) status->errYQ4);
    payload[20] = status->systemPhase;

```

```

    payload[21] = status->systemFlags;
    payload[22] = status->primaryReason;
    payload[23] = status->desiredTarget;
    payload[24] = status->ownerTarget;
    *length = OBSERVER_LINK_CORE1_HEARTBEAT_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeControlStatus(const uint8_t *payload, uint16_t length,
    observerLinkControlStatus *status)
{
    if ((payload == NULL) || (status == NULL) ||
        (length != OBSERVER_LINK_CORE1_HEARTBEAT_BYTES))
    {
        return 0u;
    }
    memset(status, 0, sizeof(*status));
    status->detectMode = payload[0];
    status->protocolCompatible = payload[1];
    status->algorithmsReady = payload[2];
    status->configCompatible = payload[3];
    status->cameraControlState = payload[4];
    status->activeSource = payload[5];
    status->controlClass = payload[6];
    status->flags = payload[7];
    status->navState = payload[8];
    status->beaconCount = payload[9];
    status->targetX = getU16(&payload[10]);
    status->targetY = getU16(&payload[12]);
    status->targetSize = getU16(&payload[14]);
    status->errXQ4 = (int16_t) getU16(&payload[16]);
    status->errYQ4 = (int16_t) getU16(&payload[18]);
    status->systemPhase = payload[20];
    status->systemFlags = payload[21];
    status->primaryReason = payload[22];
    status->desiredTarget = payload[23];
    status->ownerTarget = payload[24];
    return controlStatusIsValid(status);
}

static uint8_t observerSourceIsValid(uint8_t sourceId)
{
    return (uint8_t) ((sourceId == OBSERVER_LINK_SOURCE_CORE3) ||
        (sourceId == OBSERVER_LINK_SOURCE_CORE4));
}

uint8_t observerLinkEncodeHealth(const observerLinkHealth *health,
    uint8_t *payload, uint16_t *length)
{
    if ((health == NULL) || (payload == NULL) || (length == NULL) ||
        (observerSourceIsValid(health->sourceId) == 0u) ||
        (health->cameraState > 2u) ||
        (health->configState > OBSERVER_LINK_CONFIG_REJECTED))
    {
        return 0u;
    }
    putU32(&payload[0], health->ownerSession);
    putU32(&payload[4], health->appFrameSeq);
    putU32(&payload[8], health->captureTickMs);
    putU32(&payload[12], health->bridgeSeq);
    putU16(&payload[16], health->frameAgeMs);
    putU16(&payload[18], health->bridgeAgeMs);
    payload[20] = health->sourceId;
    payload[21] = health->cameraState;
    payload[22] = health->configState;
    payload[23] = health->flags;
    *length = OBSERVER_LINK_HEALTH_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeHealth(const uint8_t *payload, uint16_t length,
    observerLinkHealth *health)
{
    if ((payload == NULL) || (health == NULL) ||
        (length != OBSERVER_LINK_HEALTH_BYTES))
    {
        return 0u;
    }
    health->ownerSession = getU32(&payload[0]);

```



```

health->appFrameSeq = getU32(&payload[4]);
health->captureTickMs = getU32(&payload[8]);
health->bridgeSeq = getU32(&payload[12]);
health->frameAgeMs = getU16(&payload[16]);
health->bridgeAgeMs = getU16(&payload[18]);
health->sourceId = payload[20];
health->cameraState = payload[21];
health->configState = payload[22];
health->flags = payload[23];
return (uint8_t) ((observerSourceIsValid(health->sourceId) != 0u) &&
    (health->cameraState <= 2u) &&
    (health->configState <= OBSERVER_LINK_CONFIG_REJECTED));
}

static uint8_t viewOverlayIsValid(const observerLinkViewOverlay *overlay)
{
    uint16_t knownFlags = OBSERVER_LINK_OVERLAY_FLAG_VALID |
        OBSERVER_LINK_OVERLAY_FLAG_CENTER | OBSERVER_LINK_OVERLAY_FLAG_DEADZONE |
        OBSERVER_LINK_OVERLAY_FLAG_SELECTED | OBSERVER_LINK_OVERLAY_FLAG_POSE;
    uint8_t selectedPresent;
    uint8_t posePresent;

    if ((overlay == NULL) || (overlay->overlaySeq == 0u) ||
        (overlay->ownerSession == 0u) || (overlay->configRevision == 0u) ||
        (overlay->detectMode > 1u) || ((overlay->flags & ~knownFlags) != 0u) ||
        ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_VALID) == 0u)) return 0u;
    selectedPresent = (uint8_t) ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_SELECTED) != 0u);
    posePresent = (uint8_t) ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_POSE) != 0u);
    if ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_CENTER) != 0u)
    {
        if ((overlay->activeCenterX >= 188u) || (overlay->activeCenterY >= 120u)) return 0u;
    }
    else if ((overlay->activeCenterX != 0u) || (overlay->activeCenterY != 0u)) return 0u;
    if ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_DEADZONE) != 0u)
    {
        if ((overlay->deadzoneEnterRadius > 188u) ||
            (overlay->deadzoneExitRadius > 188u) ||
            (overlay->deadzoneExitRadius > overlay->deadzoneEnterRadius)) return 0u;
    }
    else if ((overlay->deadzoneEnterRadius != 0u) ||
        (overlay->deadzoneExitRadius != 0u)) return 0u;
    if (selectedPresent != 0u)
    {
        if ((overlay->sourceFrameSeq == 0u) ||
            (overlay->selectedTrackId == 0u) || (overlay->selectedX >= 188u) ||
            (overlay->selectedY >= 120u) || (overlay->selectedSize > 22560u) ||
            ((overlay->selectedFlags & (uint8_t) ~(OBSERVER_LINK_OVERLAY_SELECTED_FRESH |
                OBSERVER_LINK_OVERLAY_SELECTED_COAST)) != 0u) ||
            (overlay->selectedFlags == 0u) ||
            (overlay->selectedFlags == (OBSERVER_LINK_OVERLAY_SELECTED_FRESH |
                OBSERVER_LINK_OVERLAY_SELECTED_COAST))) return 0u;
    }
    else if ((overlay->selectedTrackId != 0u) || (overlay->selectedX != 0u) ||
        (overlay->selectedY != 0u) || (overlay->selectedSize != 0u) ||
        (overlay->selectedFlags != 0u)) return 0u;
    if (posePresent != 0u)
    {
        uint8_t poseAgeFlags = (uint8_t) (overlay->poseFlags &
            (OBSERVER_LINK_OVERLAY_POSE_FRESH | OBSERVER_LINK_OVERLAY_POSE_HOLD));
        if ((overlay->poseX >= 188u) || (overlay->poseY >= 120u) ||
            ((overlay->poseFlags & (uint8_t) ~(OBSERVER_LINK_OVERLAY_POSE_FRESH |
                OBSERVER_LINK_OVERLAY_POSE_HOLD |
                OBSERVER_LINK_OVERLAY_POSE_HEADING_VALID)) != 0u) ||
            (poseAgeFlags == 0u) ||
            (poseAgeFlags == (OBSERVER_LINK_OVERLAY_POSE_FRESH |
                OBSERVER_LINK_OVERLAY_POSE_HOLD))) return 0u;
        if ((overlay->poseFlags & OBSERVER_LINK_OVERLAY_POSE_HEADING_VALID) != 0u)
        {
            if ((overlay->poseHeadingQ8 < -805) || (overlay->poseHeadingQ8 > 805)) return 0u;
        }
        else if (overlay->poseHeadingQ8 != 0) return 0u;
    }
    else if ((overlay->poseX != 0u) || (overlay->poseY != 0u) ||
        (overlay->poseHeadingQ8 != 0) || (overlay->poseFlags != 0u)) return 0u;
    return 1u;
}

uint8_t observerLinkEncodeViewOverlay(const observerLinkViewOverlay *overlay,
    uint8_t *payload, uint16_t *length)

```

```

{
    if ((payload == NULL) || (length == NULL) ||
        (viewOverlayIsValid(overlay) == 0u)) return 0u;
    putU32(&payload[0], overlay->overlaySeq);
    putU32(&payload[4], overlay->sourceFrameSeq);
    putU32(&payload[8], overlay->ownerSession);
    putU32(&payload[12], overlay->configRevision);
    putU32(&payload[16], overlay->configHash);
    putU16(&payload[20], overlay->flags);
    putU16(&payload[22], overlay->selectedTrackId);
    putU16(&payload[24], overlay->activeCenterX);
    putU16(&payload[26], overlay->activeCenterY);
    putU16(&payload[28], overlay->deadzoneEnterRadius);
    putU16(&payload[30], overlay->deadzoneExitRadius);
    putU16(&payload[32], overlay->selectedX);
    putU16(&payload[34], overlay->selectedY);
    putU16(&payload[36], overlay->selectedSize);
    putU16(&payload[38], overlay->poseX);
    putU16(&payload[40], overlay->poseY);
    putU16(&payload[42], (uint16_t) overlay->poseHeadingQ8);
    payload[44] = overlay->selectedFlags;
    payload[45] = overlay->poseFlags;
    payload[46] = overlay->detectMode;
    *length = OBSERVER_LINK_VIEW_OVERLAY_BYTES;
    return 1u;
}

```

```

uint8_t observerLinkDecodeViewOverlay(const uint8_t *payload, uint16_t length,
    observerLinkViewOverlay *overlay)

```

```

{
    if ((payload == NULL) || (overlay == NULL) ||
        (length != OBSERVER_LINK_VIEW_OVERLAY_BYTES)) return 0u;
    memset(overlay, 0, sizeof(*overlay));
    overlay->overlaySeq = getU32(&payload[0]);
    overlay->sourceFrameSeq = getU32(&payload[4]);
    overlay->ownerSession = getU32(&payload[8]);
    overlay->configRevision = getU32(&payload[12]);
    overlay->configHash = getU32(&payload[16]);
    overlay->flags = getU16(&payload[20]);
    overlay->selectedTrackId = getU16(&payload[22]);
    overlay->activeCenterX = getU16(&payload[24]);
    overlay->activeCenterY = getU16(&payload[26]);
    overlay->deadzoneEnterRadius = getU16(&payload[28]);
    overlay->deadzoneExitRadius = getU16(&payload[30]);
    overlay->selectedX = getU16(&payload[32]);
    overlay->selectedY = getU16(&payload[34]);
    overlay->selectedSize = getU16(&payload[36]);
    overlay->poseX = getU16(&payload[38]);
    overlay->poseY = getU16(&payload[40]);
    overlay->poseHeadingQ8 = (int16_t) getU16(&payload[42]);
    overlay->selectedFlags = payload[44];
    overlay->poseFlags = payload[45];
    overlay->detectMode = payload[46];
    return viewOverlayIsValid(overlay);
}

```

```

uint8_t observerLinkPlanningTrackFragmentCount(uint8_t retainedCount)

```

```

{
    if (retainedCount > OBSERVER_LINK_MAX_TRACKS)
    {
        return 0u;
    }
    return (retainedCount == 0u) ? 1u :
        (uint8_t) ((retainedCount + OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT - 1u) /
            OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT);
}

```

```

static uint8_t planningTrackIsValid(const observerLinkPlanningTrack *track)

```

```

{
    uint8_t fresh;
    uint8_t coast;
    uint8_t edgeGrace;
    uint8_t rejected;

    if ((track == NULL) || (track->centerX >= 188u) ||
        (track->centerY >= 120u) || (track->predX >= 188u) ||
        (track->predY >= 120u) || (track->size > 22 560u) ||
        (track->predSize > 22 560u) || (track->trackId == 0u) ||
        (track->misses > OBSERVER_LINK_PLANNING_TRACK_MISSES_MAX))

```

```

        ((track->flags & (uint8_t) ~(OBSERVER_LINK_TRACK_FLAG_FRESH |
            OBSERVER_LINK_TRACK_FLAG_COAST |
            OBSERVER_LINK_TRACK_FLAG_EDGE_GRACE |
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED)) != 0u))
    {
        return 0u;
    }
    fresh = (uint8_t) ((track->flags & OBSERVER_LINK_TRACK_FLAG_FRESH) != 0u);
    coast = (uint8_t) ((track->flags & OBSERVER_LINK_TRACK_FLAG_COAST) != 0u);
    edgeGrace = (uint8_t) ((track->flags & OBSERVER_LINK_TRACK_FLAG_EDGE_GRACE) != 0u);
    rejected = (uint8_t) ((track->flags &
        OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) != 0u);
    if (rejected != 0u)
    {
        return (uint8_t) ((track->flags ==
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) &&
            (track->centerX == 0u) && (track->centerY == 0u) &&
            (track->predX == 0u) && (track->predY == 0u) &&
            (track->size == 0u) && (track->predSize == 0u) &&
            (track->misses == 0u) && (track->velXQ7 == 0) &&
            (track->velYQ7 == 0) && (track->areaVelQ0 == 0));
    }
    if ((edgeGrace != 0u) && ((fresh != 0u) || (coast == 0u))) return 0u;
    return (uint8_t) (((fresh != 0u) && (track->misses == 0u) && (coast == 0u)) ||
        ((fresh == 0u) && (track->misses != 0u) && (coast != 0u)));
}

uint8_t observerLinkEncodePlanningTrackFragment(
    const observerLinkPlanningTrackSnapshot *snapshot,
    uint8_t fragmentIndex, uint8_t *payload, uint16_t *length)
{
    uint8_t fragmentCount;
    uint8_t first;
    uint8_t count;
    uint8_t i;
    uint8_t ordinaryRetained = 0u;
    uint16_t offset;

    if ((snapshot == NULL) || (payload == NULL) || (length == NULL) ||
        (snapshot->valid == 0u) || (observerSourceIsValid(snapshot->sourceId) == 0u) ||
        (snapshot->detectMode > 1u) ||
        (snapshot->retainedCount > OBSERVER_LINK_MAX_TRACKS) ||
        (snapshot->freshTotal > snapshot->confirmedTotal) ||
        ((uint16_t) snapshot->freshLeft + snapshot->freshRight != snapshot->freshTotal))
    {
        return 0u;
    }
    for (i = 0u; i < snapshot->retainedCount; i++)
    {
        uint8_t j;
        if (planningTrackIsValid(&snapshot->tracks[i]) == 0u) return 0u;
        if ((snapshot->tracks[i].flags &
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) == 0u)
            ordinaryRetained++;
        for (j = 0u; j < i; j++)
        {
            if (snapshot->tracks[j].trackId == snapshot->tracks[i].trackId)
                return 0u;
        }
    }
    if (ordinaryRetained > snapshot->confirmedTotal) return 0u;
    fragmentCount = observerLinkPlanningTrackFragmentCount(snapshot->retainedCount);
    if (fragmentIndex >= fragmentCount)
    {
        return 0u;
    }
    first = (uint8_t) (fragmentIndex * OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT);
    count = (uint8_t) (snapshot->retainedCount - first);
    if (count > OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT)
    {
        count = OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT;
    }
    memset(payload, 0, OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_MAX_BYTES);
    payload[0] = snapshot->sourceId;
    payload[1] = snapshot->detectMode;
    payload[2] = (uint8_t) (OBSERVER_LINK_TRACK_SNAPSHOT_VALID |
        ((snapshot->truncated != 0u) ? OBSERVER_LINK_TRACK_SNAPSHOT_TRUNCATED : 0u));
    payload[3] = fragmentIndex;
    payload[4] = fragmentCount;

```

```

payload[5] = first;
payload[6] = count;
payload[7] = snapshot->retainedCount;
putU32(&payload[8], snapshot->ownerSession);
putU32(&payload[12], snapshot->revision);
putU32(&payload[16], snapshot->hash);
putU32(&payload[20], snapshot->appFrameSeq);
putU32(&payload[24], snapshot->captureTickMs);
payload[28] = snapshot->confirmedTotal;
payload[29] = snapshot->freshTotal;
payload[30] = snapshot->freshLeft;
payload[31] = snapshot->freshRight;
putU32(&payload[32], snapshot->trackerFrameSeq);
offset = OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES;
for (i = 0u; i < count; i++)
{
    const observerLinkPlanningTrack *track = &snapshot->tracks[first + i];
    uint8_t j;
    if (planningTrackIsValid(track) == 0u)
    {
        return 0u;
    }
    for (j = 0u; j < (uint8_t) (first + i); j++)
    {
        if (snapshot->tracks[j].trackId == track->trackId)
        {
            return 0u;
        }
    }
    payload[offset + 0u] = track->centerX;
    payload[offset + 1u] = track->centerY;
    payload[offset + 2u] = track->predX;
    payload[offset + 3u] = track->predY;
    putU16(&payload[offset + 4u], track->size);
    putU16(&payload[offset + 6u], track->predSize);
    putU16(&payload[offset + 8u], track->trackId);
    putU16(&payload[offset + 10u], (uint16_t) (track->misses |
        ((uint16_t) track->flags << 9u)));
    putU16(&payload[offset + 12u], (uint16_t) track->velXQ7);
    putU16(&payload[offset + 14u], (uint16_t) track->velYQ7);
    putU16(&payload[offset + 16u], (uint16_t) track->areaVelQ0);
    offset = (uint16_t) (offset + OBSERVER_LINK_PLANNING_TRACK_BYTES);
}
*length = offset;
return 1u;
}

uint8_t observerLinkDecodePlanningTrackFragment(const uint8_t *payload,
uint16_t length, observerLinkPlanningTrackFragment *fragment)
{
    uint8_t expectedFragments;
    uint8_t expectedFirst;
    uint8_t expectedCount;
    uint8_t i;
    uint16_t offset;

    if ((payload == NULL) || (fragment == NULL) ||
        (length < OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES) ||
        (observerSourceIsValid(payload[0]) == 0u) || (payload[1] > 1u) ||
        ((payload[2] & OBSERVER_LINK_TRACK_SNAPSHOT_VALID) == 0u) ||
        ((payload[2] & (uint8_t) ~(OBSERVER_LINK_TRACK_SNAPSHOT_VALID |
            OBSERVER_LINK_TRACK_SNAPSHOT_TRUNCATED)) != 0u) ||
        (payload[7] > OBSERVER_LINK_MAX_TRACKS) ||
        (payload[29] > payload[28]) ||
        ((uint16_t) payload[30] + payload[31] != payload[29]))
    {
        return 0u;
    }
    expectedFragments = observerLinkPlanningTrackFragmentCount(payload[7]);
    if ((payload[4] != expectedFragments) || (payload[3] >= expectedFragments))
    {
        return 0u;
    }
    expectedFirst = (uint8_t) (payload[3] * OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT);
    expectedCount = (uint8_t) (payload[7] - expectedFirst);
    if (expectedCount > OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT)
    {
        expectedCount = OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT;
    }
}

```

```

if ((payload[5] != expectedFirst) || (payload[6] != expectedCount) ||
    (length != (uint16_t) (OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES +
        expectedCount * OBSERVER_LINK_PLANNING_TRACK_BYTES)))
{
    return 0u;
}
memset(fragment, 0, sizeof(*fragment));
fragment->sourceId = payload[0];
fragment->detectMode = payload[1];
fragment->truncated = (uint8_t) ((payload[2] & OBSERVER_LINK_TRACK_SNAPSHOT_TRUNCATED) != 0u);
fragment->fragmentIndex = payload[3];
fragment->fragmentCount = payload[4];
fragment->firstTrack = payload[5];
fragment->trackCount = payload[6];
fragment->retainedCount = payload[7];
fragment->ownerSession = getU32(&payload[8]);
fragment->revision = getU32(&payload[12]);
fragment->hash = getU32(&payload[16]);
fragment->appFrameSeq = getU32(&payload[20]);
fragment->captureTickMs = getU32(&payload[24]);
fragment->confirmedTotal = payload[28];
fragment->freshTotal = payload[29];
fragment->freshLeft = payload[30];
fragment->freshRight = payload[31];
fragment->trackerFrameSeq = getU32(&payload[32]);
offset = OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES;
for (i = 0u; i < expectedCount; i++)
{
    observerLinkPlanningTrack *track = &fragment->tracks[i];
    uint8_t j;
    track->centerX = payload[offset + 0u];
    track->centerY = payload[offset + 1u];
    track->predX = payload[offset + 2u];
    track->predY = payload[offset + 3u];
    track->size = getU16(&payload[offset + 4u]);
    track->predSize = getU16(&payload[offset + 6u]);
    track->trackId = getU16(&payload[offset + 8u]);
    {
        uint16_t state = getU16(&payload[offset + 10u]);
        if ((state & 0xe000u) != 0u)
        {
            return 0u;
        }
        track->misses = (uint16_t) (state & 0x01ffu);
        track->flags = (uint8_t) ((state >> 9u) & 0x0fu);
    }
    track->velXQ7 = (int16_t) getU16(&payload[offset + 12u]);
    track->velYQ7 = (int16_t) getU16(&payload[offset + 14u]);
    track->areaVelQ0 = (int16_t) getU16(&payload[offset + 16u]);
    if (planningTrackIsValid(track) == 0u)
    {
        return 0u;
    }
    for (j = 0u; j < i; j++)
    {
        if (fragment->tracks[j].trackId == track->trackId)
        {
            return 0u;
        }
    }
    offset = (uint16_t) (offset + OBSERVER_LINK_PLANNING_TRACK_BYTES);
}
return 1u;
}

```

core3_cyt4bb7/project/code/comm/observerLinkProtocol.h

SHA-256

:

f96f51f0dcdaa2d9b76945ae34e036142bae8d76aa9e0262593d15c5ed3e8695

代码 (c)

```

#ifndef OBSERVER_LINK_PROTOCOL_H
#define OBSERVER_LINK_PROTOCOL_H

```

```

#include <stdint.h>
#include <string.h>
#include "paramSyncGlobal.h"

#define OBSERVER_LINK_SOF0 (0xA5u)
#define OBSERVER_LINK_SOF1 (0x5Au)
#define OBSERVER_LINK_PROTOCOL_MAJOR (3u)
#define OBSERVER_LINK_PROTOCOL_MINOR (2u)
#define OBSERVER_LINK_UART_BAUDRATE (4000000u)
#define OBSERVER_LINK_TX_EPOCH_MS (10u)
#define OBSERVER_LINK_TX_FRAMES_PER_EPOCH (4u)
#define OBSERVER_LINK_TX_CONTROL_FRAMES_PER_EPOCH (2u)
#define OBSERVER_LINK_TX_DATA_CREDIT_CAP (8u)
#define OBSERVER_LINK_TX_CONTROL_CREDIT_CAP (4u)
#define OBSERVER_LINK_RX_BYTE_BUDGET (1536u)
#define OBSERVER_LINK_MAX_PAYLOAD_BYTES (128u)
#define OBSERVER_LINK_HEADER_BYTES (20u)
#define OBSERVER_LINK_CRC_BYTES (4u)
#define OBSERVER_LINK_MAX_FRAME_BYTES (OBSERVER_LINK_HEADER_BYTES + OBSERVER_LINK_MAX_PAYLOAD_BYTES +
OBSERVER_LINK_CRC_BYTES)
#define OBSERVER_LINK_SHARED_CONFIG_BYTES (128u)
#define OBSERVER_LINK_EDGE_PREDICTION_HOLD_MS_MAX (10000u)
#define OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX (93u)
#define OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX (59u)
#define OBSERVER_LINK_CONFIG_ACK_BYTES (16u)
#define OBSERVER_LINK_DIAGNOSTICS_BYTES (32u)
#define OBSERVER_LINK_KVM_CONTROL_BYTES (17u)
#define OBSERVER_LINK_KVM_EVENT_BYTES (20u)
#define OBSERVER_LINK_KVM_ACK_BYTES (24u)
#define OBSERVER_LINK_KVM_MENU_STATUS_BYTES (24u)
#define OBSERVER_LINK_VIEW_OVERLAY_BYTES (47u)
#define OBSERVER_LINK_MAX_TRACKS (32u)
#define OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES (36u)
#define OBSERVER_LINK_PLANNING_TRACK_BYTES (18u)
#define OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT (5u)
#define OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_MAX_BYTES (OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES +
OBSERVER_LINK_PLANNING_TRACK_BYTES * OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT)
#define OBSERVER_LINK_PLANNING_TRACK_MISSES_MAX (500u)

/* RFC1982 式 32 位序号比较：0 保留给未初始态，跨 UINT32_MAX 时仍保持半窗单调。*/
static inline uint8_t observerLinkSequenceIsNewer(
    uint32_t value, uint32_t reference)
{
    uint32_t delta = value - reference;
    return (uint8_t) ((value != 0u) && (delta != 0u) &&
        (delta < 0x80000000u));
}

#define OBSERVER_LINK_RX_SESSION_REJECT (0u)
#define OBSERVER_LINK_RX_SESSION_ACCEPT (1u)
#define OBSERVER_LINK_RX_SESSION_REBIND (2u)

/* 在线时身份不可被其它 session 抢占；只有接收端已进入离线栅栏后，合格 HELLO
* 才能用任意非零 session（含冷启动回退/同值）重建 packet 序号基线。*/
static inline uint8_t observerLinkRxSessionClassify(uint8_t online,
    uint32_t currentSession, uint32_t currentPacketSeq,
    uint8_t packetSeqPrimed, uint32_t incomingSession,
    uint32_t incomingPacketSeq, uint8_t qualifiedHello)
{
    if ((incomingSession == 0u) || (incomingPacketSeq == 0u))
        return OBSERVER_LINK_RX_SESSION_REJECT;
    if (online == 0u)
        return (qualifiedHello != 0u) ? OBSERVER_LINK_RX_SESSION_REBIND : OBSERVER_LINK_RX_SESSION_REJECT;
    if (incomingSession != currentSession)
        return OBSERVER_LINK_RX_SESSION_REJECT;
    if ((packetSeqPrimed != 0u) &&
        (observerLinkSequenceIsNewer(incomingPacketSeq,
            currentPacketSeq) == 0u))
        return OBSERVER_LINK_RX_SESSION_REJECT;
    return OBSERVER_LINK_RX_SESSION_ACCEPT;
}

#define OBSERVER_LINK_CAP_BINARY_CC (1u << 0)
#define OBSERVER_LINK_CAP_LEGACY_CANNY (1u << 1)
#define OBSERVER_LINK_CAP_CONFIG_STAGING (1u << 2)
#define OBSERVER_LINK_CAP_CONFIG_APPLY (1u << 3)
#define OBSERVER_LINK_CAP_KVM (1u << 4)
#define OBSERVER_LINK_CAP_MENU_STATUS (1u << 5)
#define OBSERVER_LINK_CAP_PLANNING_TRACKS (1u << 6)

```

```

#define OBSERVER_LINK_CAP_CAMERA_CONTROL (1u << 7)
#define OBSERVER_LINK_CAP_CONTROL_STATUS (1u << 8)
#define OBSERVER_LINK_CAP_VIEW_OVERLAY (1u << 9)
#define OBSERVER_LINK_CAP_PARAM_SYNC (1u << 10)
#define OBSERVER_PARAM_HEADER_BYTES (17u)
#define OBSERVER_PARAM_DATA_BYTES (OBSERVER_LINK_MAX_PAYLOAD_BYTES - OBSERVER_PARAM_HEADER_BYTES)
#define OBSERVER_PARAM_TRANSPORT_BEGIN (0x80u)
#define OBSERVER_PARAM_TRANSPORT_MORE (0x40u)
#define OBSERVER_PARAM_STATUS_MASK (0x3Fu)
#define OBSERVER_PARAM_OP_POLL (0x7Fu)

typedef struct
{
    uint32_t transactionId;
    uint32_t payloadCrc32;
    uint16_t offset;
    uint16_t totalLength;
    uint8_t op;
    uint8_t slot;
    uint8_t target;
    uint8_t status;
    uint8_t length;
    uint8_t data[OBSERVER_PARAM_DATA_BYTES];
} observerParamFrame;

static inline uint8_t observerParamEncode(const observerParamFrame *frame,
    uint8_t *payload, uint16_t *length)
{
    if (frame == NULL || payload == NULL || length == NULL ||
        frame->length > OBSERVER_PARAM_DATA_BYTES) return 0u;
    payload[0] = frame->op;
    payload[1] = frame->slot;
    payload[2] = frame->target;
    payload[3] = frame->status;
    payload[4] = (uint8_t) frame->transactionId;
    payload[5] = (uint8_t) (frame->transactionId >> 8);
    payload[6] = (uint8_t) (frame->transactionId >> 16);
    payload[7] = (uint8_t) (frame->transactionId >> 24);
    payload[8] = (uint8_t) frame->offset;
    payload[9] = (uint8_t) (frame->offset >> 8);
    payload[10] = (uint8_t) frame->totalLength;
    payload[11] = (uint8_t) (frame->totalLength >> 8);
    payload[12] = (uint8_t) frame->payloadCrc32;
    payload[13] = (uint8_t) (frame->payloadCrc32 >> 8);
    payload[14] = (uint8_t) (frame->payloadCrc32 >> 16);
    payload[15] = (uint8_t) (frame->payloadCrc32 >> 24);
    payload[16] = frame->length;
    if (frame->length != 0u) memcpy(&payload[17], frame->data, frame->length);
    *length = (uint16_t) (OBSERVER_PARAM_HEADER_BYTES + frame->length);
    return 1u;
}

static inline uint8_t observerParamDecode(const uint8_t *payload,
    uint16_t length, observerParamFrame *frame)
{
    observerParamFrame candidate;
    if (payload == NULL || frame == NULL || length < OBSERVER_PARAM_HEADER_BYTES ||
        payload[16] > OBSERVER_PARAM_DATA_BYTES ||
        length != (uint16_t) (OBSERVER_PARAM_HEADER_BYTES + payload[16])) return 0u;
    memset(&candidate, 0, sizeof(candidate));
    candidate.op = payload[0];
    candidate.slot = payload[1];
    candidate.target = payload[2];
    candidate.status = payload[3];
    candidate.transactionId = (uint32_t) payload[4] | ((uint32_t) payload[5] << 8) |
        ((uint32_t) payload[6] << 16) | ((uint32_t) payload[7] << 24);
    candidate.offset = (uint16_t) (payload[8] | ((uint16_t) payload[9] << 8));
    candidate.totalLength = (uint16_t) (payload[10] | ((uint16_t) payload[11] << 8));
    candidate.payloadCrc32 = (uint32_t) payload[12] |
        ((uint32_t) payload[13] << 8) | ((uint32_t) payload[14] << 16) |
        ((uint32_t) payload[15] << 24);
    candidate.length = payload[16];
    if (candidate.length != 0u) memcpy(candidate.data, &payload[17], candidate.length);
    *frame = candidate;
    return 1u;
}

#define OBSERVER_PARAM_GLOBAL_REQUEST_BYTES (12u)
#define OBSERVER_PARAM_GLOBAL_STATE_BYTES (28u)

```

```

typedef char observerParamGlobalRequestSizeAssert[
    (sizeof(paramSyncGlobalRequest) == OBSERVER_PARAM_GLOBAL_REQUEST_BYTES) ? 1 : -1];
typedef char observerParamGlobalStateSizeAssert[
    (sizeof(paramSyncGlobalState) == OBSERVER_PARAM_GLOBAL_STATE_BYTES) ? 1 : -1];
static inline void observerParamGlobalPutU32(uint8_t *p, uint32_t value)
{
    p[0] = (uint8_t) value; p[1] = (uint8_t) (value >> 8);
    p[2] = (uint8_t) (value >> 16); p[3] = (uint8_t) (value >> 24);
}
static inline uint32_t observerParamGlobalGetU32(const uint8_t *p)
{
    return (uint32_t) p[0] | ((uint32_t) p[1] << 8) |
        ((uint32_t) p[2] << 16) | ((uint32_t) p[3] << 24);
}
static inline uint8_t observerParamGlobalRequestEncode(
    const paramSyncGlobalRequest *value, uint8_t *payload, uint16_t *length)
{
    if ((value == NULL) || (payload == NULL) || (length == NULL) ||
        (paramSyncGlobalRequestIsValid(value) == 0u)) return 0u;
    observerParamGlobalPutU32(&payload[0], value->producerBootGeneration);
    observerParamGlobalPutU32(&payload[4], value->sequence);
    payload[8] = value->origin; payload[9] = value->action;
    payload[10] = value->slot; payload[11] = 0u;
    *length = OBSERVER_PARAM_GLOBAL_REQUEST_BYTES;
    return 1u;
}
static inline uint8_t observerParamGlobalRequestDecode(const uint8_t *payload,
    uint16_t length, paramSyncGlobalRequest *value)
{
    paramSyncGlobalRequest candidate;
    if ((payload == NULL) || (value == NULL) ||
        (length != OBSERVER_PARAM_GLOBAL_REQUEST_BYTES)) return 0u;
    candidate.producerBootGeneration = observerParamGlobalGetU32(&payload[0]);
    candidate.sequence = observerParamGlobalGetU32(&payload[4]);
    candidate.origin = payload[8]; candidate.action = payload[9];
    candidate.slot = payload[10]; candidate.reserved = payload[11];
    if (paramSyncGlobalRequestIsValid(&candidate) == 0u) return 0u;
    *value = candidate;
    return 1u;
}
static inline uint8_t observerParamGlobalStateEncode(
    const paramSyncGlobalState *value, uint8_t *payload, uint16_t *length)
{
    if ((value == NULL) || (payload == NULL) || (length == NULL) ||
        paramSyncGlobalStateIsValid(value) == 0u) return 0u;
    observerParamGlobalPutU32(&payload[0], value->coordinatorBootGeneration);
    observerParamGlobalPutU32(&payload[4], value->sequence);
    observerParamGlobalPutU32(&payload[8], value->requestBootGeneration);
    observerParamGlobalPutU32(&payload[12], value->requestSequence);
    observerParamGlobalPutU32(&payload[16], value->transactionId);
    payload[20] = value->origin; payload[21] = value->action;
    payload[22] = value->slot; payload[23] = value->phase;
    payload[24] = value->target; payload[25] = value->error;
    payload[26] = value->readyMask; payload[27] = value->flags;
    *length = OBSERVER_PARAM_GLOBAL_STATE_BYTES;
    return paramSyncGlobalStateIsValid(value);
}
static inline uint8_t observerParamGlobalStateDecode(const uint8_t *payload,
    uint16_t length, paramSyncGlobalState *value)
{
    paramSyncGlobalState candidate;
    if ((payload == NULL) || (value == NULL) ||
        (length != OBSERVER_PARAM_GLOBAL_STATE_BYTES)) return 0u;
    memset(&candidate, 0, sizeof(candidate));
    candidate.coordinatorBootGeneration = observerParamGlobalGetU32(&payload[0]);
    candidate.sequence = observerParamGlobalGetU32(&payload[4]);
    candidate.requestBootGeneration = observerParamGlobalGetU32(&payload[8]);
    candidate.requestSequence = observerParamGlobalGetU32(&payload[12]);
    candidate.transactionId = observerParamGlobalGetU32(&payload[16]);
    candidate.origin = payload[20]; candidate.action = payload[21];
    candidate.slot = payload[22]; candidate.phase = payload[23];
    candidate.target = payload[24]; candidate.error = payload[25];
    candidate.readyMask = payload[26]; candidate.flags = payload[27];
    if (paramSyncGlobalStateIsValid(&candidate) == 0u) return 0u;
    *value = candidate;
    return 1u;
}
#define OBSERVER_LINK_CAMERA_CONTROL_SAFE    (0u)

```



```

#define OBSERVER_LINK_CAMERA_CONTROL_RUN      (1u)
#define OBSERVER_LINK_CORE1_HEARTBEAT_BYTES   (25u)
#define OBSERVER_LINK_HEALTH_BYTES             (24u)
#define OBSERVER_LINK_CAMERA_UNKNOWN          (0u)
#define OBSERVER_LINK_CAMERA_RUNNING          (1u)
#define OBSERVER_LINK_CAMERA_FAULT            (2u)
#define OBSERVER_LINK_HEALTH_FLAG_FRAME_FRESH (1u << 0)
#define OBSERVER_LINK_HEALTH_FLAG_BRIDGE_FRESH (1u << 1)

#define OBSERVER_LINK_SYSTEM_PHASE_INIT        (0u)
#define OBSERVER_LINK_SYSTEM_PHASE_INIT_FAULT (1u)
#define OBSERVER_LINK_SYSTEM_PHASE_LINK_WAIT  (2u)
#define OBSERVER_LINK_SYSTEM_PHASE_HOME       (3u)
#define OBSERVER_LINK_SYSTEM_PHASE_REJOIN     (4u)
#define OBSERVER_LINK_SYSTEM_PHASE_STANDALONE (5u)
#define OBSERVER_LINK_SYSTEM_PHASE_RUNNING    (6u)

#define OBSERVER_LINK_SYSTEM_FLAG_LAUNCHED     (1u << 0)
#define OBSERVER_LINK_SYSTEM_FLAG_LANDING      (1u << 1)
#define OBSERVER_LINK_SYSTEM_FLAG_VEL_VALID    (1u << 2)
#define OBSERVER_LINK_SYSTEM_FLAG_TARGET_VALID (1u << 3)
#define OBSERVER_LINK_SYSTEM_FLAG_STANDALONE    (1u << 4)
#define OBSERVER_LINK_SYSTEM_FLAG_REJOIN       (1u << 5)
#define OBSERVER_LINK_SYSTEM_FLAG_LAUNCH_ELIGIBLE (1u << 6)

#define OBSERVER_LINK_REASON_NONE              (0u)
#define OBSERVER_LINK_REASON_LOCAL_INIT        (1u)
#define OBSERVER_LINK_REASON_CORE0_LINK        (2u)
#define OBSERVER_LINK_REASON_CORE1_LINK        (3u)
#define OBSERVER_LINK_REASON_CORE2_LINK        (4u)
#define OBSERVER_LINK_REASON_OBSERVER_LINK     (5u)
#define OBSERVER_LINK_REASON_PROTOCOL          (6u)
#define OBSERVER_LINK_REASON_CONFIG            (7u)
#define OBSERVER_LINK_REASON_CAMERA            (8u)
#define OBSERVER_LINK_REASON_TARGET            (9u)
#define OBSERVER_LINK_REASON_BARRIER          (10u)
#define OBSERVER_LINK_REASON_DISPATCH          (11u)
#define OBSERVER_LINK_REASON_SAFETY            (12u)
#define OBSERVER_LINK_REASON_LANDING           (13u)

#define OBSERVER_LINK_OVERLAY_FLAG_VALID        (1u << 0)
#define OBSERVER_LINK_OVERLAY_FLAG_CENTER       (1u << 1)
#define OBSERVER_LINK_OVERLAY_FLAG_DEADZONE     (1u << 2)
#define OBSERVER_LINK_OVERLAY_FLAG_SELECTED     (1u << 3)
#define OBSERVER_LINK_OVERLAY_FLAG_POSE         (1u << 4)
#define OBSERVER_LINK_OVERLAY_SELECTED_FRESH    (1u << 0)
#define OBSERVER_LINK_OVERLAY_SELECTED_COAST    (1u << 1)
#define OBSERVER_LINK_OVERLAY_POSE_FRESH        (1u << 0)
#define OBSERVER_LINK_OVERLAY_POSE_HOLD         (1u << 1)
#define OBSERVER_LINK_OVERLAY_POSE_HEADING_VALID (1u << 2)

#define OBSERVER_LINK_CONTROL_FLAG_STATUS_VALID (1u << 0)
#define OBSERVER_LINK_CONTROL_FLAG_DECISION_VALID (1u << 1)
#define OBSERVER_LINK_CONTROL_FLAG_BARRIER_REQUIRED (1u << 2)
#define OBSERVER_LINK_CONTROL_FLAG_DISPATCH_ENABLED (1u << 3)
#define OBSERVER_LINK_CONTROL_FLAG_REMOTE_ELIGIBLE (1u << 4)
#define OBSERVER_LINK_CONTROL_FLAG_CORE0_LINK_UP (1u << 5)
#define OBSERVER_LINK_CONTROL_FLAG_CORE2_LINK_UP (1u << 6)
#define OBSERVER_LINK_CONTROL_FLAG_REQUIRED_VISION (1u << 7)

#define OBSERVER_LINK_SOURCE_NONE (0u)
#define OBSERVER_LINK_SOURCE_CORE1 (1u)
#define OBSERVER_LINK_SOURCE_CORE3 (3u)
#define OBSERVER_LINK_SOURCE_CORE4 (4u)

#define OBSERVER_LINK_CONTROL_CLASS_NONE (0u)
#define OBSERVER_LINK_CONTROL_CLASS_CORE1_TRACK (1u)
#define OBSERVER_LINK_CONTROL_CLASS_OBSERVER_TRACK (2u)
#define OBSERVER_LINK_CONTROL_CLASS_SEARCH (3u)

#define OBSERVER_LINK_TRACK_FLAG_FRESH (1u << 0)
#define OBSERVER_LINK_TRACK_FLAG_COAST (1u << 1)
#define OBSERVER_LINK_TRACK_FLAG_EDGE_GRACE (1u << 2)
#define OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED (1u << 3)
#define OBSERVER_LINK_TRACK_SNAPSHOT_VALID (1u << 0)
#define OBSERVER_LINK_TRACK_SNAPSHOT_TRUNCATED (1u << 1)

#define OBSERVER_LINK_KVM_TARGET_CORE0 (0u)
#define OBSERVER_LINK_KVM_TARGET_CORE1 (1u)

```

```

#define OBSERVER_LINK_KVM_TARGET_CORE3 (3u)
#define OBSERVER_LINK_KVM_TARGET_CORE4 (4u)
#define OBSERVER_LINK_KVM_TARGET_NONE (0xFFu)

#define OBSERVER_LINK_KVM_ACK_FLAG_HOME_READY (1u << 0)
#define OBSERVER_LINK_KVM_ACK_FLAG_ACTION_OK (1u << 1)
#define OBSERVER_LINK_KVM_ACK_FLAG_ACTION_SAVE (1u << 2)
#define OBSERVER_LINK_KVM_ACK_FLAG_DUPLICATE (1u << 3)

#define OBSERVER_LINK_KVM_MENU_FLAG_HOME_READY (1u << 0)
#define OBSERVER_LINK_KVM_MENU_FLAG_FEEDBACK_ACTIVE (1u << 1)
#define OBSERVER_LINK_KVM_MENU_FLAG_LEASE_VALID (1u << 2)
#define OBSERVER_LINK_KVM_MENU_FLAG_ACTION_OK (1u << 3)
#define OBSERVER_LINK_KVM_MENU_FLAG_ACTION_SAVE (1u << 4)

typedef enum
{
    OBSERVER_LINK_ROLE_CORE1 = 1,
    OBSERVER_LINK_ROLE_CORE3 = 3,
    OBSERVER_LINK_ROLE_CORE4 = 4
} observerLinkRole;

typedef enum
{
    OBSERVER_LINK_MSG_HELLO = 1,
    OBSERVER_LINK_MSG_HEARTBEAT = 2,
    OBSERVER_LINK_MSG_DIAGNOSTICS = 3,
    OBSERVER_LINK_MSG_CONFIG_SNAPSHOT = 4,
    OBSERVER_LINK_MSG_CONFIG_ACK = 5,
    OBSERVER_LINK_MSG_KVM_CONTROL = 6,
    OBSERVER_LINK_MSG_KVM_EVENT = 7,
    OBSERVER_LINK_MSG_KVM_ACK = 8,
    OBSERVER_LINK_MSG_KVM_MENU_STATUS = 9,
    OBSERVER_LINK_MSG_PLANNING_TRACK_FRAGMENT = 10,
    OBSERVER_LINK_MSG_VIEW_OVERLAY = 11,
    OBSERVER_LINK_MSG_HEALTH = 12,
    OBSERVER_LINK_MSG_PARAM_REQUEST = 13,
    OBSERVER_LINK_MSG_PARAM_RESPONSE = 14,
    OBSERVER_LINK_MSG_PARAM_GLOBAL_STATE = 15,
    OBSERVER_LINK_MSG_PARAM_GLOBAL_REQUEST = 16
} observerLinkMessageType;

typedef enum
{
    OBSERVER_LINK_KVM_RESULT_NONE = 0,
    OBSERVER_LINK_KVM_RESULT_EXECUTED = 1,
    OBSERVER_LINK_KVM_RESULT_UNHANDLED = 2,
    OBSERVER_LINK_KVM_RESULT_NO_LEASE = 3,
    OBSERVER_LINK_KVM_RESULT_BAD_PAYLOAD = 4,
    OBSERVER_LINK_KVM_RESULT_WRONG_TARGET = 5,
    OBSERVER_LINK_KVM_RESULT_STALE_EPOCH = 6,
    OBSERVER_LINK_KVM_RESULT_GLOBAL_EVENT_REJECTED = 7,
    OBSERVER_LINK_KVM_RESULT_GAP = 8,
    OBSERVER_LINK_KVM_RESULT_OLD_SEQ = 9
} observerLinkKvmResult;

typedef enum
{
    OBSERVER_LINK_CONFIG_NONE = 0,
    OBSERVER_LINK_CONFIG_RECEIVED = 1,
    OBSERVER_LINK_CONFIG_STAGED = 2,
    OBSERVER_LINK_CONFIG_APPLIED = 3,
    OBSERVER_LINK_CONFIG_REJECTED = 4
} observerLinkConfigStatus;

typedef enum
{
    OBSERVER_LINK_REJECT_NONE = 0,
    OBSERVER_LINK_REJECT_BAD_PAYLOAD = 1,
    OBSERVER_LINK_REJECT_BAD_HASH = 2,
    OBSERVER_LINK_REJECT_OLD_REVISION = 3,
    OBSERVER_LINK_REJECT_REVISION_CONFLICT = 4,
    OBSERVER_LINK_REJECT_UNSUPPORTED = 5
} observerLinkRejectReason;

typedef struct
{
    uint8_t major;
    uint8_t minor;

```

```

uint8_t type;
uint8_t flags;
uint16_t payloadLength;
uint32_t bootSession;
uint32_t packetSeq;
uint32_t tickMs;
uint8_t payload[OBSERVER_LINK_MAX_PAYLOAD_BYTES];
} observerLinkFrame;

```

```

typedef struct
{
    uint8_t bytes[OBSERVER_LINK_MAX_FRAME_BYTES];
    uint16_t used;
    uint16_t expected;
    uint32_t validFrames;
    uint32_t crcErrors;
    uint32_t lengthErrors;
    uint32_t versionErrors;
} observerLinkParser;

```

```

typedef struct
{
    uint32_t revision;
    uint32_t hash;
    uint8_t detectMode;
    uint8_t maxBeacons;
    uint16_t edgePredictionHoldMs;
    uint8_t sourceId;
    uint8_t profileVersion;
    uint16_t beaconSizeMin;
    uint16_t beaconSizeMax;
    float beaconEccMax;
    uint8_t beaconBrightnessMin;
    uint16_t beaconMaxEndpoints;
    float beaconFillMin;
    float beaconHuMax;
    uint16_t beaconHuAreaMin;
    float trackMaxAssocDist;
    float trackSizeRatioMax;
    uint16_t trackConfirmHits;
    uint16_t trackDropMisses;
    uint16_t trackCoastMisses;
    float trackPositionAlpha;
    float trackSizeAlpha;
    float trackAreaVelAlpha;
    float trackAssocScoreMax;
    float trackAssocAreaWeight;
    float trackAssocAreaVelWeight;
    float beaconHuTemplate[7];
    uint8_t c1BorderEnterLeft;
    uint8_t c1BorderEnterRight;
    uint8_t c1BorderEnterTop;
    uint8_t c1BorderEnterBottom;
    uint8_t c1BorderExitLeft;
    uint8_t c1BorderExitRight;
    uint8_t c1BorderExitTop;
    uint8_t c1BorderExitBottom;
    uint8_t observerBorderEnterLeft;
    uint8_t observerBorderEnterRight;
    uint8_t observerBorderEnterTop;
    uint8_t observerBorderEnterBottom;
    uint8_t observerBorderExitLeft;
    uint8_t observerBorderExitRight;
    uint8_t observerBorderExitTop;
    uint8_t observerBorderExitBottom;
    uint8_t c1EdgeBandLeft;
    uint8_t c1EdgeBandRight;
    uint8_t c1EdgeBandTop;
    uint8_t c1EdgeBandBottom;
    uint8_t observerEdgeBandLeft;
    uint8_t observerEdgeBandRight;
    uint8_t observerEdgeBandTop;
    uint8_t observerEdgeBandBottom;
} observerLinkSharedConfig;

```

```

typedef struct
{
    uint32_t ownerSession;
    uint32_t revision;
}

```

```

uint32_t hash;
uint8_t desiredMode;
uint8_t appliedMode;
uint8_t status;
uint8_t reason;
} observerLinkConfigAck;

typedef struct
{
    uint32_t validFrames;
    uint32_t crcErrors;
    uint32_t lengthErrors;
    uint32_t versionErrors;
    uint32_t rxOverflowErrors;
    uint32_t rxFrameErrors;
    uint32_t rxParityErrors;
    uint32_t txDeferred;
} observerLinkDiagnostics;

typedef struct
{
    uint32_t targetSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t leaseMs;
    uint8_t target;
} observerLinkKvmControl;

typedef struct
{
    uint32_t targetSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t eventSeq;
    uint8_t target;
    uint8_t keyId;
    uint8_t keyState;
    uint8_t flags;
} observerLinkKvmEvent;

typedef struct
{
    uint32_t targetSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t eventSeq;
    uint32_t expectedSeq;
    uint8_t target;
    uint8_t result;
    uint8_t menuState;
    uint8_t flags;
} observerLinkKvmAck;

typedef struct
{
    uint32_t targetSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t lastEventSeq;
    uint32_t screenRevision;
    uint8_t target;
    uint8_t menuState;
    uint8_t flags;
    uint8_t lastAckResult;
} observerLinkKvmMenuStatus;

typedef struct
{
    uint8_t detectMode;
    uint8_t protocolCompatible;
    uint8_t algorithmsReady;
    uint8_t configCompatible;
    uint8_t cameraControlState;
    uint8_t activeSource;
    uint8_t controlClass;
    uint8_t flags;
    uint8_t navState;
    uint8_t beaconCount;
    uint16_t targetX;

```

```

    uint16_t targetY;
    uint16_t targetSize;
    int16_t errXQ4;
    int16_t errYQ4;
    uint8_t systemPhase;
    uint8_t systemFlags;
    uint8_t primaryReason;
    uint8_t desiredTarget;
    uint8_t ownerTarget;
} observerLinkControlStatus;

typedef struct
{
    uint32_t ownerSession;
    uint32_t appFrameSeq;
    uint32_t captureTickMs;
    uint32_t bridgeSeq;
    uint16_t frameAgeMs;
    uint16_t bridgeAgeMs;
    uint8_t sourceId;
    uint8_t cameraState;
    uint8_t configState;
    uint8_t flags;
} observerLinkHealth;

/* ownerSession 是发送端 Core1 boot session; 接收端只接受与当前 peer session 一致的 DTO。 */
typedef struct
{
    uint32_t overlaySeq;
    uint32_t sourceFrameSeq;
    uint32_t ownerSession;
    uint32_t configRevision;
    uint32_t configHash;
    uint16_t flags;
    uint16_t selectedTrackId;
    uint16_t activeCenterX;
    uint16_t activeCenterY;
    uint16_t deadzoneEnterRadius;
    uint16_t deadzoneExitRadius;
    uint16_t selectedX;
    uint16_t selectedY;
    uint16_t selectedSize;
    uint16_t poseX;
    uint16_t poseY;
    int16_t poseHeadingQ8;
    uint8_t selectedFlags;
    uint8_t poseFlags;
    uint8_t detectMode;
} observerLinkViewOverlay;

typedef struct
{
    uint8_t centerX;
    uint8_t centerY;
    uint8_t predX;
    uint8_t predY;
    uint16_t size;
    uint16_t predSize;
    uint16_t trackId;
    uint16_t misses;
    uint8_t flags;
    int16_t velXQ7;
    int16_t velYQ7;
    int16_t areaVelQ0;
} observerLinkPlanningTrack;

typedef struct
{
    uint8_t valid;
    uint8_t sourceId;
    uint8_t detectMode;
    uint8_t truncated;
    uint32_t ownerSession;
    uint32_t revision;
    uint32_t hash;
    uint32_t appFrameSeq;
    uint32_t captureTickMs;
    uint32_t trackerFrameSeq;
    uint8_t confirmedTotal;

```

```

    uint8_t freshTotal;
    uint8_t freshLeft;
    uint8_t freshRight;
    uint8_t retainedCount;
    observerLinkPlanningTrack tracks[OBSERVER_LINK_MAX_TRACKS];
} observerLinkPlanningTrackSnapshot;

typedef struct
{
    uint8_t sourceId;
    uint8_t detectMode;
    uint8_t truncated;
    uint8_t fragmentIndex;
    uint8_t fragmentCount;
    uint8_t firstTrack;
    uint8_t trackCount;
    uint8_t retainedCount;
    uint32_t ownerSession;
    uint32_t revision;
    uint32_t hash;
    uint32_t appFrameSeq;
    uint32_t captureTickMs;
    uint32_t trackerFrameSeq;
    uint8_t confirmedTotal;
    uint8_t freshTotal;
    uint8_t freshLeft;
    uint8_t freshRight;
    observerLinkPlanningTrack tracks[OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT];
} observerLinkPlanningTrackFragment;

void observerLinkParserInit(observerLinkParser *parser);
uint8_t observerLinkParserFeed(observerLinkParser *parser, uint8_t byte, observerLinkFrame *frame);
uint16_t observerLinkEncodeFrame(const observerLinkFrame *frame, uint8_t *output, uint16_t capacity);
uint8_t observerLinkEncodeHello(uint8_t role, uint32_t capabilities, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeHello(const uint8_t *payload, uint16_t length, uint8_t *role, uint32_t *capabilities);
uint8_t observerLinkEncodeSharedConfig(const observerLinkSharedConfig *config, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeSharedConfig(const uint8_t *payload, uint16_t length, observerLinkSharedConfig *config);
uint32_t observerLinkSharedConfigHash(const observerLinkSharedConfig *config);
uint8_t observerLinkSharedConfigIsValid(const observerLinkSharedConfig *config);
uint8_t observerLinkEncodeConfigAck(const observerLinkConfigAck *ack, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeConfigAck(const uint8_t *payload, uint16_t length, observerLinkConfigAck *ack);
uint8_t observerLinkEncodeDiagnostics(const observerLinkDiagnostics *diagnostics, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeDiagnostics(const uint8_t *payload, uint16_t length, observerLinkDiagnostics *diagnostics);
uint8_t observerLinkEncodeKvmControl(const observerLinkKvmControl *control, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeKvmControl(const uint8_t *payload, uint16_t length, observerLinkKvmControl *control);
uint8_t observerLinkEncodeKvmEvent(const observerLinkKvmEvent *event, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeKvmEvent(const uint8_t *payload, uint16_t length, observerLinkKvmEvent *event);
uint8_t observerLinkEncodeKvmAck(const observerLinkKvmAck *ack, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeKvmAck(const uint8_t *payload, uint16_t length, observerLinkKvmAck *ack);
uint8_t observerLinkEncodeKvmMenuStatus(const observerLinkKvmMenuStatus *status, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeKvmMenuStatus(const uint8_t *payload, uint16_t length, observerLinkKvmMenuStatus *status);
uint8_t observerLinkEncodeControlStatus(const observerLinkControlStatus *status,
    uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeControlStatus(const uint8_t *payload, uint16_t length,
    observerLinkControlStatus *status);
uint8_t observerLinkEncodeHealth(const observerLinkHealth *health,
    uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeHealth(const uint8_t *payload, uint16_t length,
    observerLinkHealth *health);
uint8_t observerLinkEncodeViewOverlay(const observerLinkViewOverlay *overlay,
    uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeViewOverlay(const uint8_t *payload, uint16_t length,
    observerLinkViewOverlay *overlay);
uint8_t observerLinkPlanningTrackFragmentCount(uint8_t retainedCount);
uint8_t observerLinkEncodePlanningTrackFragment(
    const observerLinkPlanningTrackSnapshot *snapshot,
    uint8_t fragmentIndex, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodePlanningTrackFragment(const uint8_t *payload,
    uint16_t length, observerLinkPlanningTrackFragment *fragment);
uint8_t observerLinkQuantizeSigned(float value, float scale,
    float minimum, float maximum, int16_t *quantized);

#endif /* OBSERVER_LINK_PROTOCOL_H */

```

core3_cyt4bb7/project/code/comm/paramSyncGlobal.h

SHA-256

:

16bd76824b594d1c1737b56b4d97f5d90393ccf24573bf1b42af32c3cb5e8f67

代码 (c)

```
#ifndef PARAM_SYNC_GLOBAL_H
#define PARAM_SYNC_GLOBAL_H

#include <stdint.h>

#define PARAM_SYNC_GLOBAL_TARGET_MASK (0x1Fu)

typedef enum
{
    PARAM_SYNC_GLOBAL_ACTION_NONE = 0u,
    PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY = 1u,
    PARAM_SYNC_GLOBAL_ACTION_LOAD_RUNTIME = 2u,
    PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY = 3u
} paramSyncGlobalAction;

#define PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_VALID (0u)
#define PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_EMPTY (5u)
#define PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_INVALID (10u)

static inline uint32_t paramSyncGlobalPackSlotSummary(uint8_t validMask,
    uint8_t emptyMask, uint8_t invalidMask)
{
    return ((uint32_t) (validMask & PARAM_SYNC_GLOBAL_TARGET_MASK) <<
        PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_VALID) |
        ((uint32_t) (emptyMask & PARAM_SYNC_GLOBAL_TARGET_MASK) <<
        PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_EMPTY) |
        ((uint32_t) (invalidMask & PARAM_SYNC_GLOBAL_TARGET_MASK) <<
        PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_INVALID);
}

static inline uint8_t paramSyncGlobalSlotSummaryMask(uint32_t summary,
    uint8_t shift)
{
    return (uint8_t) ((summary >> shift) & PARAM_SYNC_GLOBAL_TARGET_MASK);
}

typedef enum
{
    PARAM_SYNC_SLOT_UI_UNKNOWN = 0u,
    PARAM_SYNC_SLOT_UI_CHECKING,
    PARAM_SYNC_SLOT_UI_FULL,
    PARAM_SYNC_SLOT_UI_EMPTY,
    PARAM_SYNC_SLOT_UI_PARTIAL,
    PARAM_SYNC_SLOT_UI_INVALID,
    PARAM_SYNC_SLOT_UI_UNAVAILABLE
} paramSyncSlotUiState;

static inline uint8_t paramSyncGlobalClassifySlotSummary(uint32_t summary)
{
    uint8_t validMask = paramSyncGlobalSlotSummaryMask(summary,
        PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_VALID);
    uint8_t emptyMask = paramSyncGlobalSlotSummaryMask(summary,
        PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_EMPTY);
    uint8_t invalidMask = paramSyncGlobalSlotSummaryMask(summary,
        PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_INVALID);
    if (invalidMask != 0u) return PARAM_SYNC_SLOT_UI_INVALID;
    if (validMask == PARAM_SYNC_GLOBAL_TARGET_MASK && emptyMask == 0u)
        return PARAM_SYNC_SLOT_UI_FULL;
    if (emptyMask == PARAM_SYNC_GLOBAL_TARGET_MASK && validMask == 0u)
        return PARAM_SYNC_SLOT_UI_EMPTY;
    if ((uint8_t) (validMask | emptyMask) == PARAM_SYNC_GLOBAL_TARGET_MASK)
        return PARAM_SYNC_SLOT_UI_PARTIAL;
    return PARAM_SYNC_SLOT_UI_UNAVAILABLE;
}

typedef enum
{

```

```

PARAM_SYNC_GLOBAL_PHASE_UNINITIALIZED = 0u,
PARAM_SYNC_GLOBAL_PHASE_READY = 1u,
PARAM_SYNC_GLOBAL_PHASE_REQUESTED = 2u,
PARAM_SYNC_GLOBAL_PHASE_LOCKING = 3u,
PARAM_SYNC_GLOBAL_PHASE_PREPARING = 4u,
PARAM_SYNC_GLOBAL_PHASE_COMMITTING = 5u,
PARAM_SYNC_GLOBAL_PHASE_APPLYING = 6u,
PARAM_SYNC_GLOBAL_PHASE_DECIDING = 7u,
PARAM_SYNC_GLOBAL_PHASE_FINISHING = 8u,
PARAM_SYNC_GLOBAL_PHASE_ABORTING = 9u,
PARAM_SYNC_GLOBAL_PHASE_RECOVERING = 10u,
PARAM_SYNC_GLOBAL_PHASE_SUCCEEDED = 11u,
PARAM_SYNC_GLOBAL_PHASE_FAILED = 12u
} paramSyncGlobalPhase;

typedef enum
{
    PARAM_SYNC_GLOBAL_ERROR_NONE = 0u,
    PARAM_SYNC_GLOBAL_ERROR_BUSY = 1u,
    PARAM_SYNC_GLOBAL_ERROR_REJECTED = 2u,
    PARAM_SYNC_GLOBAL_ERROR_INCOMPLETE = 3u,
    PARAM_SYNC_GLOBAL_ERROR_ROUTE = 4u,
    PARAM_SYNC_GLOBAL_ERROR_STORAGE = 5u,
    PARAM_SYNC_GLOBAL_ERROR_RECOVERY = 6u,
    PARAM_SYNC_GLOBAL_ERROR_PEER_RESTART = 7u,
    PARAM_SYNC_GLOBAL_ERROR_INVALID = 8u
} paramSyncGlobalError;

#define PARAM_SYNC_GLOBAL_FLAG_READY (1u << 0)
#define PARAM_SYNC_GLOBAL_FLAG_BUSY (1u << 1)
#define PARAM_SYNC_GLOBAL_FLAG_RECOVERY (1u << 2)
#define PARAM_SYNC_GLOBAL_FLAG_TERMINAL (1u << 3)
#define PARAM_SYNC_GLOBAL_FLAG_OK (1u << 4)
#define PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY (1u << 5)

typedef struct
{
    uint32_t producerBootGeneration;
    uint32_t sequence;
    uint8_t origin;
    uint8_t action;
    uint8_t slot;
    uint8_t reserved;
} paramSyncGlobalRequest;

typedef struct
{
    uint32_t coordinatorBootGeneration;
    uint32_t sequence;
    uint32_t requestBootGeneration;
    uint32_t requestSequence;
    uint32_t transactionId;
    uint8_t origin;
    uint8_t action;
    uint8_t slot;
    uint8_t phase;
    uint8_t target;
    uint8_t error;
    uint8_t readyMask;
    uint8_t flags;
} paramSyncGlobalState;

static inline uint8_t paramSyncGlobalStateIsValid(
    const paramSyncGlobalState *state)
{
    const uint8_t allowedFlags = PARAM_SYNC_GLOBAL_FLAG_READY |
        PARAM_SYNC_GLOBAL_FLAG_BUSY | PARAM_SYNC_GLOBAL_FLAG_RECOVERY |
        PARAM_SYNC_GLOBAL_FLAG_TERMINAL | PARAM_SYNC_GLOBAL_FLAG_OK |
        PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY;
    uint8_t identityPresent;
    if (state == 0 || state->coordinatorBootGeneration == 0u ||
        state->sequence == 0u || state->phase > PARAM_SYNC_GLOBAL_PHASE_FAILED ||
        (state->target != 0xFFu && state->target > 4u) ||
        state->error > PARAM_SYNC_GLOBAL_ERROR_INVALID ||
        (state->readyMask & (uint8_t)~PARAM_SYNC_GLOBAL_TARGET_MASK) != 0u ||
        (state->flags & (uint8_t)~allowedFlags) != 0u) return 0u;
    identityPresent = (uint8_t) (state->requestBootGeneration != 0u);
    if (identityPresent != (uint8_t) (state->requestSequence != 0u)) return 0u;
    if (state->action == PARAM_SYNC_GLOBAL_ACTION_NONE)

```



```

{
    if (state->slot != 0u || state->origin != 0xFFu) return 0u;
}
else
{
    if (state->action > PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY ||
        (state->origin > 4u && state->origin != 0xFFu)) return 0u;
    if (state->slot < 1u || state->slot > 5u) return 0u;
}
if (state->phase == PARAM_SYNC_GLOBAL_PHASE_UNINITIALIZED)
{
    return (uint8_t) ((state->flags == 0u ||
        state->flags == PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY) &&
        state->action == PARAM_SYNC_GLOBAL_ACTION_NONE &&
        identityPresent == 0u && state->transactionId == 0u &&
        state->target == 0xFFu && state->error == PARAM_SYNC_GLOBAL_ERROR_NONE);
}
if (state->phase == PARAM_SYNC_GLOBAL_PHASE_READY)
{
    return (uint8_t) (state->flags ==
        (PARAM_SYNC_GLOBAL_FLAG_READY |
        PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY) &&
        state->readyMask == PARAM_SYNC_GLOBAL_TARGET_MASK &&
        state->action == PARAM_SYNC_GLOBAL_ACTION_NONE &&
        identityPresent == 0u && state->transactionId == 0u &&
        state->target == 0xFFu && state->error == PARAM_SYNC_GLOBAL_ERROR_NONE);
}
if (state->phase == PARAM_SYNC_GLOBAL_PHASE_RECOVERING)
{
    return (uint8_t) (state->flags == (PARAM_SYNC_GLOBAL_FLAG_BUSY |
        PARAM_SYNC_GLOBAL_FLAG_RECOVERY) &&
        state->action == PARAM_SYNC_GLOBAL_ACTION_NONE &&
        identityPresent == 0u && state->target == 0xFFu &&
        (state->error == PARAM_SYNC_GLOBAL_ERROR_NONE ||
        state->error == PARAM_SYNC_GLOBAL_ERROR_RECOVERY));
}
if (state->phase == PARAM_SYNC_GLOBAL_PHASE_SUCCEEDED)
{
    return (uint8_t) ((state->flags == (PARAM_SYNC_GLOBAL_FLAG_TERMINAL |
        PARAM_SYNC_GLOBAL_FLAG_OK) ||
        state->flags == (PARAM_SYNC_GLOBAL_FLAG_TERMINAL |
        PARAM_SYNC_GLOBAL_FLAG_OK |
        PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY)) &&
        state->action != PARAM_SYNC_GLOBAL_ACTION_NONE &&
        identityPresent != 0u && state->transactionId != 0u &&
        state->target == 0xFFu && state->error == PARAM_SYNC_GLOBAL_ERROR_NONE);
}
if (state->phase == PARAM_SYNC_GLOBAL_PHASE_FAILED)
{
    return (uint8_t) ((state->flags == PARAM_SYNC_GLOBAL_FLAG_TERMINAL ||
        state->flags == (PARAM_SYNC_GLOBAL_FLAG_TERMINAL |
        PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY)) &&
        state->action != PARAM_SYNC_GLOBAL_ACTION_NONE &&
        identityPresent != 0u &&
        state->error != PARAM_SYNC_GLOBAL_ERROR_NONE);
}
return (uint8_t) (state->phase >= PARAM_SYNC_GLOBAL_PHASE_REQUESTED &&
    state->phase <= PARAM_SYNC_GLOBAL_PHASE_ABORTING &&
    state->flags == PARAM_SYNC_GLOBAL_FLAG_BUSY &&
    state->action != PARAM_SYNC_GLOBAL_ACTION_NONE &&
    identityPresent != 0u && state->transactionId != 0u);
}

static inline uint8_t paramSyncGlobalRequestIsValid(
    const paramSyncGlobalRequest *request)
{
    if (request == 0 || request->producerBootGeneration == 0u ||
        request->sequence == 0u || request->origin > 4u ||
        request->reserved != 0u ||
        request->action < PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY ||
        request->action > PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
        return 0u;
    return (uint8_t) (request->slot >= 1u && request->slot <= 5u);
}

static inline uint8_t paramSyncGlobalStateReady(
    const paramSyncGlobalState *state)
{
    return (uint8_t) (paramSyncGlobalStateIsValid(state) != 0u &&

```

```

        state->phase == PARAM_SYNC_GLOBAL_PHASE_READY &&
        state->action == PARAM_SYNC_GLOBAL_ACTION_NONE &&
        state->target == 0xFFu && state->error == PARAM_SYNC_GLOBAL_ERROR_NONE &&
        state->readyMask == PARAM_SYNC_GLOBAL_TARGET_MASK &&
        state->flags == (PARAM_SYNC_GLOBAL_FLAG_READY |
                        PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY));
    }

static inline uint8_t paramSyncGlobalStateRuntimeReady(
    const paramSyncGlobalState *state)
{
    return (uint8_t) (paramSyncGlobalStateIsValid(state) != 0u &&
        (state->flags & PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY) != 0u);
}

static inline uint8_t paramSyncGlobalStateMatchesRequest(
    const paramSyncGlobalState *state, const paramSyncGlobalRequest *request)
{
    return (uint8_t) (paramSyncGlobalStateIsValid(state) != 0u &&
        paramSyncGlobalRequestIsValid(request) != 0u &&
        state->requestBootGeneration == request->producerBootGeneration &&
        state->requestSequence == request->sequence &&
        state->origin == request->origin && state->action == request->action &&
        state->slot == request->slot);
}

static inline void paramSyncGlobalPutU32(uint8_t *p, uint32_t value)
{
    p[0] = (uint8_t) value; p[1] = (uint8_t) (value >> 8);
    p[2] = (uint8_t) (value >> 16); p[3] = (uint8_t) (value >> 24);
}

static inline uint32_t paramSyncGlobalGetU32(const uint8_t *p)
{
    return (uint32_t) p[0] | ((uint32_t) p[1] << 8) |
        ((uint32_t) p[2] << 16) | ((uint32_t) p[3] << 24);
}

static inline void paramSyncGlobalEncodeRequest(
    const paramSyncGlobalRequest *value, uint8_t out[12])
{
    paramSyncGlobalPutU32(&out[0], value->producerBootGeneration);
    paramSyncGlobalPutU32(&out[4], value->sequence);
    out[8] = value->origin; out[9] = value->action;
    out[10] = value->slot; out[11] = value->reserved;
}

static inline uint8_t paramSyncGlobalDecodeRequest(const uint8_t in[12],
    paramSyncGlobalRequest *value)
{
    value->producerBootGeneration = paramSyncGlobalGetU32(&in[0]);
    value->sequence = paramSyncGlobalGetU32(&in[4]);
    value->origin = in[8]; value->action = in[9];
    value->slot = in[10]; value->reserved = in[11];
    return paramSyncGlobalRequestIsValid(value);
}

static inline void paramSyncGlobalEncodeState(
    const paramSyncGlobalState *value, uint8_t out[28])
{
    paramSyncGlobalPutU32(&out[0], value->coordinatorBootGeneration);
    paramSyncGlobalPutU32(&out[4], value->sequence);
    paramSyncGlobalPutU32(&out[8], value->requestBootGeneration);
    paramSyncGlobalPutU32(&out[12], value->requestSequence);
    paramSyncGlobalPutU32(&out[16], value->transactionId);
    out[20] = value->origin; out[21] = value->action; out[22] = value->slot;
    out[23] = value->phase; out[24] = value->target; out[25] = value->error;
    out[26] = value->readyMask; out[27] = value->flags;
}

static inline uint8_t paramSyncGlobalDecodeState(const uint8_t in[28],
    paramSyncGlobalState *value)
{
    value->coordinatorBootGeneration = paramSyncGlobalGetU32(&in[0]);
    value->sequence = paramSyncGlobalGetU32(&in[4]);
    value->requestBootGeneration = paramSyncGlobalGetU32(&in[8]);
    value->requestSequence = paramSyncGlobalGetU32(&in[12]);
    value->transactionId = paramSyncGlobalGetU32(&in[16]);
    value->origin = in[20]; value->action = in[21]; value->slot = in[22];
    value->phase = in[23]; value->target = in[24]; value->error = in[25];
    value->readyMask = in[26]; value->flags = in[27];
    return paramSyncGlobalStateIsValid(value);
}

```

```
typedef char paramSyncGlobal_request_size_assert[
    (sizeof(paramSyncGlobalRequest) == 12u) ? 1 : -1];
typedef char paramSyncGlobal_state_size_assert[
    (sizeof(paramSyncGlobalState) == 28u) ? 1 : -1];
```

```
#endif
```

core3_cyt4bb7/project/code/comm/paramSyncMailbox.c

SHA-256

:

774a74f34cc5eaab467a692cd9faa2c10c8c111750dc3e38cd1b56c2694384ea

代码 (c)

```
#include "paramSyncMailbox.h"

#include "cy_device_headers.h"

#include <string.h>

#define PARAM_SYNC_REQUEST_MAGIC (0x50535251u) /* PSRQ */
#define PARAM_SYNC_RESPONSE_MAGIC (0x50535253u) /* PSRS */
#define PARAM_SYNC_MAILBOX_ABI (1u)

typedef struct
{
    uint32_t beginSequence;
    uint32_t magic;
    uint32_t abiVersion;
    uint32_t payloadBytes;
    uint32_t producerBootGeneration;
    uint32_t sequence;
    uint32_t transactionId;
    uint32_t offset;
    uint32_t payloadCrc32;
    uint16_t dataLength;
    uint8_t op;
    uint8_t slot;
    uint8_t target;
    uint8_t state;
    uint8_t reservedHeader[6];
    uint8_t data[PARAM_SYNC_MAILBOX_DATA_MAX];
    uint8_t reservedTail[12];
    uint32_t endSequence;
} paramSyncMailboxSharedFrame;

typedef char paramSyncMailbox_frame_size_assert[
    (sizeof(paramSyncMailboxSharedFrame) == PARAM_SYNC_MAILBOX_FRAME_BYTES) ? 1 : -1];

#pragma location = PARAM_SYNC_MAILBOX_REQUEST_ADDR
__no_init static volatile paramSyncMailboxSharedFrame paramSyncRequestRegion;
#pragma location = PARAM_SYNC_MAILBOX_RESPONSE_ADDR
__no_init static volatile paramSyncMailboxSharedFrame paramSyncResponseRegion;

static uint8_t frameInputValid(uint32_t bootGeneration, uint32_t sequence,
    uint16_t dataLength)
{
    return (uint8_t)((bootGeneration != 0u) && (sequence != 0u) &&
        (dataLength <= PARAM_SYNC_MAILBOX_DATA_MAX));
}

static void cleanRegion(volatile paramSyncMailboxSharedFrame *region)
{
    SCB_CleanDCache_by_Addr((volatile void *) region, sizeof(*region));
    __DSB();
}

static void invalidateRegion(volatile paramSyncMailboxSharedFrame *region)
{
    SCB_InvalidateDCache_by_Addr((volatile void *) region, sizeof(*region));
    __DSB();
}

static uint8_t publishFrame(volatile paramSyncMailboxSharedFrame *region,
    const paramSyncMailboxSharedFrame *frame)
{

```

```

paramSyncMailboxSharedFrame staged;

staged = *frame;
staged.endSequence = 0u;

region->beginSequence = 0u;
region->endSequence = 0u;
cleanRegion(region);

memcpy((void *) region, &staged, sizeof(staged));
cleanRegion(region);

region->endSequence = staged.sequence;
SCB_CleanDCache_by_Addr((volatile void *)
    ((uintptr_t) &region->endSequence) & ~(uintptr_t) 31u), 32u);
__DSB();
return 1u;
}

static uint8_t sharedFrameValid(const paramSyncMailboxSharedFrame *frame,
    uint32_t magic)
{
    return (uint8_t) ((frame->beginSequence != 0u) &&
        (frame->beginSequence == frame->sequence) &&
        (frame->endSequence == frame->sequence) &&
        (frame->magic == magic) &&
        (frame->abiVersion == PARAM_SYNC_MAILBOX_ABI) &&
        (frame->payloadBytes == frame->dataLength) &&
        frameInputValid(frame->producerBootGeneration, frame->sequence,
            frame->dataLength));
}

static uint8_t tryReadFrame(volatile paramSyncMailboxSharedFrame *region,
    uint32_t magic, paramSyncMailboxSharedFrame *frame)
{
    paramSyncMailboxSharedFrame first;
    paramSyncMailboxSharedFrame second;
    uint8_t attempt;

    for (attempt = 0u; attempt < 3u; attempt++)
    {
        invalidateRegion(region);
        memcpy(&first, (const void *) region, sizeof(first));
        invalidateRegion(region);
        memcpy(&second, (const void *) region, sizeof(second));
        if (sharedFrameValid(&first, magic) &&
            sharedFrameValid(&second, magic) &&
            (memcmp(&first, &second, sizeof(first)) == 0))
        {
            *frame = second;
            return 1u;
        }
    }
    return 0u;
}

uint8_t paramSyncMailboxPublishRequest(const paramSyncMailboxRequest *request)
{
    paramSyncMailboxSharedFrame frame;
    if ((request == 0) || !frameInputValid(request->producerBootGeneration,
        request->sequence, request->dataLength)) return 0u;
    memset(&frame, 0, sizeof(frame));
    frame.beginSequence = request->sequence;
    frame.magic = PARAM_SYNC_REQUEST_MAGIC;
    frame.abiVersion = PARAM_SYNC_MAILBOX_ABI;
    frame.payloadBytes = request->dataLength;
    frame.producerBootGeneration = request->producerBootGeneration;
    frame.sequence = request->sequence;
    frame.transactionId = request->transactionId;
    frame.offset = request->offset;
    frame.payloadCrc32 = request->payloadCrc32;
    frame.dataLength = request->dataLength;
    frame.op = request->op;
    frame.slot = request->slot;
    frame.target = request->target;
    frame.state = request->flags;
    memcpy(frame.data, request->data, request->dataLength);
    return publishFrame(&paramSyncRequestRegion, &frame);
}

```

```

uint8_t paramSyncMailboxTryReadRequest(paramSyncMailboxRequest *request)
{
    paramSyncMailboxSharedFrame frame;
    paramSyncMailboxRequest candidate;
    if ((request == 0) || !tryReadFrame(&paramSyncRequestRegion,
        PARAM_SYNC_REQUEST_MAGIC, &frame)) return 0u;
    memset(&candidate, 0, sizeof(candidate));
    candidate.producerBootGeneration = frame.producerBootGeneration;
    candidate.sequence = frame.sequence;
    candidate.transactionId = frame.transactionId;
    candidate.offset = frame.offset;
    candidate.payloadCrc32 = frame.payloadCrc32;
    candidate.dataLength = frame.dataLength;
    candidate.op = frame.op;
    candidate.slot = frame.slot;
    candidate.target = frame.target;
    candidate.flags = frame.state;
    memcpy(candidate.data, frame.data, frame.dataLength);
    *request = candidate;
    return 1u;
}

uint8_t paramSyncMailboxPublishResponse(const paramSyncMailboxResponse *response)
{
    paramSyncMailboxSharedFrame frame;
    if ((response == 0) || !frameInputValid(response->producerBootGeneration,
        response->sequence, response->dataLength)) return 0u;
    memset(&frame, 0, sizeof(frame));
    frame.beginSequence = response->sequence;
    frame.magic = PARAM_SYNC_RESPONSE_MAGIC;
    frame.abiVersion = PARAM_SYNC_MAILBOX_ABI;
    frame.payloadBytes = response->dataLength;
    frame.producerBootGeneration = response->producerBootGeneration;
    frame.sequence = response->sequence;
    frame.transactionId = response->transactionId;
    frame.offset = response->offset;
    frame.payloadCrc32 = response->payloadCrc32;
    frame.dataLength = response->dataLength;
    frame.op = response->op;
    frame.slot = response->slot;
    frame.target = response->target;
    frame.state = response->status;
    memcpy(frame.data, response->data, response->dataLength);
    return publishFrame(&paramSyncResponseRegion, &frame);
}

uint8_t paramSyncMailboxTryReadResponse(paramSyncMailboxResponse *response)
{
    paramSyncMailboxSharedFrame frame;
    paramSyncMailboxResponse candidate;
    if ((response == 0) || !tryReadFrame(&paramSyncResponseRegion,
        PARAM_SYNC_RESPONSE_MAGIC, &frame)) return 0u;
    memset(&candidate, 0, sizeof(candidate));
    candidate.producerBootGeneration = frame.producerBootGeneration;
    candidate.sequence = frame.sequence;
    candidate.transactionId = frame.transactionId;
    candidate.offset = frame.offset;
    candidate.payloadCrc32 = frame.payloadCrc32;
    candidate.dataLength = frame.dataLength;
    candidate.op = frame.op;
    candidate.slot = frame.slot;
    candidate.target = frame.target;
    candidate.status = frame.state;
    memcpy(candidate.data, frame.data, frame.dataLength);
    *response = candidate;
    return 1u;
}

```

core3_cyt4bb7/project/code/comm/paramSyncOwnerService.c

SHA-256

:

7f2ac3684ebcca71a1daebbc832c54d7d83c68b1b7a72247da2ebe22e21127df

代码 (c)

```
#include "paramSyncOwnerService.h"

#include <string.h>

#include "core3CoreDataTransfer.h"
#include "core3VisionParams.h"
#include "core3VisionParamsBlob.h"
#include "core3VisionParamsPersistence.h"
#include "crc32Ieee.h"
#include "paramSyncMailbox.h"

#define CORE3_PARAM_SYNC_TARGET      (3u)
#define CORE3_PARAM_SYNC_SCHEMA_HASH (0x321CE9D7u)
#define CORE3_ACTIVATE_ROLLBACK_MAGIC (0xA3B20 000u)
#define PARAM_STAGE_NONE             (0u)
#define PARAM_STAGE_LIBRARY           (1u)
#define PARAM_STAGE_RUNTIME           (2u)

static uint32_t g_lastBootGeneration;
static uint32_t g_lastRequestSequence;
static uint32_t g_responseSequence;
static uint32_t g_ownerBootGeneration;
static core3VisionParamsFlashBlob g_stagedBlob;
static uint32_t g_stagedTransactionId;
static uint32_t g_stagedExpectedCrc;
static uint16_t g_stagedBytes;
static uint16_t g_stagedReceived;
static uint8_t g_stagedSlot;
static uint8_t g_stagedValid;
static uint8_t g_flashPrepared;
static uint8_t g_flashCommitted;
static uint8_t g_stageKind;
static uint8_t g_runtimeApplied;
static uint8_t g_activationRecoveryChecked;
static uint8_t g_syncMutationLocked;
static uint8_t g_syncLockedSlot;
static uint32_t g_syncLockedTransaction;
static uint32_t g_lastFinishedTransaction;
static uint32_t g_lastAbortedTransaction;
#if defined(__ICCARM__)
__no_init uint32_t core3ParamSyncActivationRollbackRetained;
__no_init uint32_t core3ParamSyncActivationRollbackInverseRetained;
#else
static uint32_t core3ParamSyncActivationRollbackRetained;
static uint32_t core3ParamSyncActivationRollbackInverseRetained;
#endif

static uint8_t activationContext(uint8_t *oldSlotExternal,
    uint8_t *newSlotExternal)
{
    uint32_t word = core3ParamSyncActivationRollbackRetained;
    if (core3ParamSyncActivationRollbackInverseRetained != ~word ||
        (word & 0xFFFF0000u) != CORE3_ACTIVATE_ROLLBACK_MAGIC)
        return 0u;
    *oldSlotExternal = (uint8_t) word;
    *newSlotExternal = (uint8_t) (word >> 8);
    return (uint8_t) (*oldSlotExternal >= 1u &&
        *oldSlotExternal <= CORE3_VISION_PARAMS_SLOT_COUNT &&
        *newSlotExternal >= 1u &&
        *newSlotExternal <= CORE3_VISION_PARAMS_SLOT_COUNT);
}

static void setActivationContext(uint8_t oldSlotExternal,
    uint8_t newSlotExternal)
{
    uint32_t word = CORE3_ACTIVATE_ROLLBACK_MAGIC |
        ((uint32_t) oldSlotExternal |
        ((uint32_t) newSlotExternal << 8);
    core3ParamSyncActivationRollbackRetained = word;
    core3ParamSyncActivationRollbackInverseRetained = ~word;
}

static void clearActivationContext(void)
{
    core3ParamSyncActivationRollbackRetained = 0u;
    core3ParamSyncActivationRollbackInverseRetained = UINT32_MAX;
}
```

```

}

static uint8_t activatePreparedRuntimeSlot(uint8_t slotExternal,
uint32_t transactionId)
{
    core3VisionParamsStruct params;
    if (core3VisionParamsFlashActivatePrepared((uint8_t) (slotExternal - 1u),
transactionId, &params) != CORE3_VISION_PARAMS_LOAD_VALID)
        return 0u;
    core3VisionParamsCommit(&params);
    return 1u;
}

static uint8_t activateRuntimeSlotPending(uint8_t slotExternal)
{
    core3VisionParamsStruct params;
    if (core3VisionParamsFlashActivateSlotPending((uint8_t) (slotExternal - 1u),
&params) != CORE3_VISION_PARAMS_LOAD_VALID) return 0u;
    core3VisionParamsCommit(&params);
    return 1u;
}

static uint8_t rollbackActivation(void)
{
    uint8_t oldSlotExternal, newSlotExternal;
    (void) activationContext(&oldSlotExternal, &newSlotExternal);
    {
        core3VisionParamsStruct params;
        (void) oldSlotExternal;
        (void) newSlotExternal;
        if (core3VisionParamsFlashAbortPendingActivation(&params) !=
CORE3_VISION_PARAMS_LOAD_VALID) return 0u;
        core3VisionParamsCommit(&params);
    }
    clearActivationContext();
    return 1u;
}

static uint8_t recoverActivationAfterOwnerBoot(void)
{
    if (g_activationRecoveryChecked != 0u) return 1u;
    g_activationRecoveryChecked = 1u;
    return 1u;
}

static void putU16(uint8_t *output, uint16_t value)
{
    output[0] = (uint8_t) value;
    output[1] = (uint8_t) (value >> 8);
}

static void putU32(uint8_t *output, uint32_t value)
{
    output[0] = (uint8_t) value;
    output[1] = (uint8_t) (value >> 8);
    output[2] = (uint8_t) (value >> 16);
    output[3] = (uint8_t) (value >> 24);
}

static uint16_t getU16(const uint8_t *input)
{
    return (uint16_t) (input[0] | ((uint16_t) input[1] << 8));
}

static uint32_t getU32(const uint8_t *input)
{
    return (uint32_t) input[0] | ((uint32_t) input[1] << 8) |
((uint32_t) input[2] << 16) | ((uint32_t) input[3] << 24);
}

static uint8_t externalSlotValid(uint8_t slot)
{
    return (uint8_t) ((slot >= 1u) &&
(slot <= CORE3_VISION_PARAMS_SLOT_COUNT));
}

static void clearMutationLock(void)
{
    g_syncMutationLocked = 0u;
}

```

```

    g_syncLockedSlot = 0u;
    g_syncLockedTransaction = 0u;
}

static uint8_t decodeGlobalTransaction(const paramSyncMailboxRequest *request,
    uint8_t allowDefaultSlot, uint8_t *slot, uint32_t *transactionId)
{
    if (request->dataLength != 5u) return 0u;
    *slot = request->data[0];
    *transactionId = getU32(&request->data[1]);
    return (uint8_t) ((*transactionId != 0u) &&
        (*transactionId != UINT32_MAX) &&
        (externalSlotValid(*slot) != 0u ||
            (allowDefaultSlot != 0u && *slot == 0u)));
}

static uint8_t transactionLocked(uint8_t slot, uint32_t transactionId)
{
    return (uint8_t) (g_syncMutationLocked != 0u &&
        g_syncLockedSlot == slot &&
        g_syncLockedTransaction == transactionId);
}

static uint8_t producerBootChangeAllowed(
    const paramSyncMailboxRequest *request)
{
    core3VisionParamsRuntimeMeta runtimeMeta;
    uint8_t oldSlot, newSlot;
    uint8_t activationPending;
    uint8_t transactionPending;
    uint32_t transactionId;

    if (core3VisionParamsFlashGetRuntimeMeta(&runtimeMeta) !=
        CORE3_VISION_PARAMS_LOAD_VALID) return 0u;
    activationPending = (uint8_t) (activationContext(&oldSlot, &newSlot) ||
        (runtimeMeta.pending != 0u &&
            runtimeMeta.transactionId == UINT32_MAX));
    transactionPending = (uint8_t) (g_syncMutationLocked || g_stagedValid ||
        g_flashPrepared || g_flashCommitted || runtimeMeta.pending);
    if ((activationPending == 0u) && (transactionPending == 0u)) return 1u;
    if ((request->op != PARAM_SYNC_OP_FINISH &&
        request->op != PARAM_SYNC_OP_ABORT) ||
        (request->dataLength != 4u)) return 0u;

    transactionId = getU32(request->data);
    if (activationPending != 0u)
    {
        if ((transactionId != 0u) || g_stagedValid || g_flashPrepared ||
            g_flashCommitted) return 0u;
        return 1u;
    }
    if ((g_syncMutationLocked != 0u) &&
        (transactionId != g_syncLockedTransaction)) return 0u;
    if ((g_stagedValid || g_flashPrepared || g_flashCommitted) &&
        (transactionId != g_stagedTransactionId)) return 0u;
    if ((runtimeMeta.pending != 0u) &&
        (transactionId != runtimeMeta.transactionId)) return 0u;
    return 1u;
}

uint8_t core3ParamSyncOwnerServiceMutationLocked(void)
{
    return (uint8_t) (g_syncMutationLocked || g_stagedValid ||
        g_flashPrepared || g_flashCommitted ||
        core3VisionParamsFlashMutationPending());
}

static void clearStaging(void)
{
    memset(&g_stagedBlob, 0, sizeof(g_stagedBlob));
    g_stagedTransactionId = 0u;
    g_stagedExpectedCrc = 0u;
    g_stagedBytes = 0u;
    g_stagedReceived = 0u;
    g_stagedSlot = 0u;
    g_stagedValid = 0u;
    g_flashPrepared = 0u;
    g_flashCommitted = 0u;
    g_stageKind = PARAM_STAGE_NONE;
}

```



```

    g_runtimeApplied = 0u;
}

static uint8_t rollbackTransaction(uint32_t transactionId)
{
    core3VisionParamsStruct params;
    if (core3VisionParamsFlashAbortTransaction(transactionId) !=
        CORE3_VISION_PARAMS_SAVE_VERIFIED) return 0u;
    if (core3VisionParamsFlashReadRuntime(&params) !=
        CORE3_VISION_PARAMS_LOAD_VALID) return 0u;
    core3VisionParamsCommit(&params);
    return 1u;
}

static void publish(const paramSyncMailboxRequest *request, uint8_t status,
    const uint8_t *data, uint16_t length, uint32_t crc)
{
    paramSyncMailboxResponse response;
    memset(&response, 0, sizeof(response));
    g_responseSequence++;
    if (g_responseSequence == 0u) g_responseSequence = 1u;
    response.producerBootGeneration = g_ownerBootGeneration;
    response.sequence = g_responseSequence;
    response.transactionId = request->transactionId;
    response.offset = request->offset;
    response.payloadCrc32 = crc;
    response.dataLength = length;
    response.op = request->op;
    response.slot = request->slot;
    response.target = request->target;
    response.status = status;
    if ((data != NULL) && (length != 0u)) memcpy(response.data, data, length);
    (void) paramSyncMailboxPublishResponse(&response);
}

static void handleReadMeta(const paramSyncMailboxRequest *request)
{
    uint8_t output[26];
    uint8_t externalSlot;
    uint8_t internalSlot;
    core3VisionParamsLibraryMeta meta;
    core3VisionParamsStruct params;
    core3VisionParamsFlashBlob blob;
    uint32_t blobCrc = 0u;

    if ((request->dataLength != 1u) ||
        (externalSlotValid(request->data[0]) == 0u))
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    externalSlot = request->data[0];
    internalSlot = (uint8_t) (externalSlot - 1u);
    memset(&meta, 0, sizeof(meta));
    if (core3VisionParamsFlashInspectSlot(internalSlot, &meta) !=
        CORE3_VISION_PARAMS_LOAD_VALID)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    if (meta.state == CORE3_VISION_PARAMS_LIBRARY_VALID)
    {
        if (core3VisionParamsFlashReadSlot(internalSlot, &params, NULL) !=
            CORE3_VISION_PARAMS_LOAD_VALID)
        {
            publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
            return;
        }
        core3VisionParamsBlobEncode(&params, &blob);
        blobCrc = getU32(&blob.bytes[CORE3_VISION_PARAMS_BLOB_CRC_BYTES]);
    }
    memset(output, 0, sizeof(output));
    output[0] = CORE3_PARAM_SYNC_TARGET;
    output[1] = externalSlot;
    output[2] = meta.state;
    output[3] = 0u;
    putU32(&output[4], CORE3_VISION_PARAMS_FLASH_VERSION);
    putU32(&output[8], meta.generation);
    putU32(&output[12], blobCrc);
}

```

```

        putU32(&output[16], meta.transactionId);
        putU32(&output[20], CORE3_PARAM_SYNC_SCHEMA_HASH);
        putU16(&output[24], CORE3_VISION_PARAMS_BLOB_BYTES);
        publish(request, PARAM_SYNC_STATUS_OK, output, sizeof(output), 0u);
    }

static void handleReadRuntimeMeta(const paramSyncMailboxRequest *request)
{
    uint8_t output[32];
    core3VisionParamsStruct runtime;
    core3VisionParamsRuntimeMeta meta;
    core3VisionParamsPersistenceStatus persistence;
    core3VisionParamsFlashBlob blob;
    uint32_t blobCrc;

    if (request->dataLength != 0u ||
        core3VisionParamsFlashReadRuntime(&runtime) !=
            CORE3_VISION_PARAMS_LOAD_VALID ||
        core3VisionParamsFlashGetRuntimeMeta(&meta) !=
            CORE3_VISION_PARAMS_LOAD_VALID)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }

    core3VisionParamsBlobEncode(&runtime, &blob);
    core3VisionParamsPersistenceGetStatus(&persistence);
    blobCrc = getU32(&blob.bytes[CORE3_VISION_PARAMS_BLOB_CRC_BYTES]);
    memset(output, 0, sizeof(output));
    output[0] = CORE3_PARAM_SYNC_TARGET;
    output[1] = meta.valid;
    output[2] = meta.bank;
    output[3] = meta.pending;
    output[4] = (uint8_t) (memcmp(&runtime, core3VisionParamsGetActive(),
        sizeof(runtime)) != 0);
    output[5] = meta.sourceSlot;
    output[6] = persistence.runtimeDefaultsRewritten != 0u ? 1u : 0u;
    putU32(&output[8], CORE3_VISION_PARAMS_FLASH_VERSION);
    putU32(&output[12], meta.generation);
    putU32(&output[16], blobCrc);
    putU32(&output[20], meta.transactionId);
    putU32(&output[24], CORE3_PARAM_SYNC_SCHEMA_HASH);
    putU16(&output[28], CORE3_VISION_PARAMS_BLOB_BYTES);
    publish(request, PARAM_SYNC_STATUS_OK, output, sizeof(output), 0u);
}

static void handleReadRuntimeChunk(const paramSyncMailboxRequest *request)
{
    uint8_t output[PARAM_SYNC_MAILBOX_DATA_MAX];
    uint16_t offset;
    uint16_t length;
    uint32_t blobCrc;
    core3VisionParamsStruct params;
    core3VisionParamsFlashBlob blob;

    if (request->dataLength != 4u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }

    offset = getU16(&request->data[0]);
    length = getU16(&request->data[2]);
    if (length == 0u || length > (PARAM_SYNC_MAILBOX_DATA_MAX - 4u) ||
        (uint32_t) offset + length > CORE3_VISION_PARAMS_BLOB_BYTES ||
        core3VisionParamsFlashReadRuntime(&params) !=
            CORE3_VISION_PARAMS_LOAD_VALID)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }

    core3VisionParamsBlobEncode(&params, &blob);
    blobCrc = getU32(&blob.bytes[CORE3_VISION_PARAMS_BLOB_CRC_BYTES]);
    putU16(&output[0], offset);
    putU16(&output[2], CORE3_VISION_PARAMS_BLOB_BYTES);
    memcpy(&output[4], &blob.bytes[offset], length);
    publish(request, PARAM_SYNC_STATUS_OK, output,
        (uint16_t) (4u + length), blobCrc);
}

static void handleReadChunk(const paramSyncMailboxRequest *request)

```

```

{
    uint8_t output[PARAM_SYNC_MAILBOX_DATA_MAX];
    uint8_t externalSlot;
    uint8_t internalSlot;
    uint16_t offset;
    uint16_t length;
    uint32_t blobCrc;
    core3VisionParamsStruct params;
    core3VisionParamsSlotMeta meta;
    core3VisionParamsFlashBlob blob;

    if (request->dataLength != 5u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    externalSlot = request->data[0];
    offset = getU16(&request->data[1]);
    length = getU16(&request->data[3]);
    if ((externalSlotValid(externalSlot) == 0u) || (length == 0u) ||
        (length > (PARAM_SYNC_MAILBOX_DATA_MAX - 5u)) ||
        (offset >= CORE3_VISION_PARAMS_BLOB_BYTES) ||
        ((uint32_t) offset + length > CORE3_VISION_PARAMS_BLOB_BYTES))
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    internalSlot = (uint8_t) (externalSlot - 1u);
    if (core3VisionParamsFlashReadSlot(internalSlot, &params, &meta) !=
        CORE3_VISION_PARAMS_LOAD_VALID)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    core3VisionParamsBlobEncode(&params, &blob);
    blobCrc = getU32(&blob.bytes[CORE3_VISION_PARAMS_BLOB_CRC_BYTES]);
    output[0] = externalSlot;
    putU16(&output[1], offset);
    putU16(&output[3], CORE3_VISION_PARAMS_BLOB_BYTES);
    memcpy(&output[5], &blob.bytes[offset], length);
    publish(request, PARAM_SYNC_STATUS_OK, output,
        (uint16_t) (5u + length), blobCrc);
}

static void handlePrepareLibraryFromLive(
    const paramSyncMailboxRequest *request)
{
    core3VisionParamsFlashBlob blob;
    const core3VisionParamsStruct *active;
    uint8_t slot;
    uint32_t transactionId;
    uint32_t blobCrc;

    if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (g_flashPrepared != 0u || g_flashCommitted != 0u)
    {
        publish(request, (g_stageKind == PARAM_STAGE_LIBRARY &&
            g_stagedSlot == slot &&
            g_stagedTransactionId == transactionId) ? PARAM_SYNC_STATUS_OK :
            PARAM_SYNC_STATUS_BUSY, NULL, 0u, g_stagedExpectedCrc);
        return;
    }
    active = core3VisionParamsGetActive();
    core3VisionParamsBlobEncode(active, &blob);
    blobCrc = getU32(&blob.bytes[CORE3_VISION_PARAMS_BLOB_CRC_BYTES]);
    if (core3VisionParamsFlashPrepareSlot((uint8_t) (slot - 1u), active,
        transactionId) != CORE3_VISION_PARAMS_SAVE_VERIFIED)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    clearStaging();
    g_stagedBlob = blob;
    g_stagedSlot = slot;

```

```

    g_stagedTransactionId = transactionId;
    g_stagedExpectedCrc = blobCrc;
    g_stagedBytes = CORE3_VISION_PARAMS_BLOB_BYTES;
    g_stagedReceived = CORE3_VISION_PARAMS_BLOB_BYTES;
    g_stagedValid = 1u;
    g_flashPrepared = 1u;
    g_stageKind = PARAM_STAGE_LIBRARY;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, blobCrc);
}

static void handlePrepareRuntime(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;
    core3VisionParamsSaveStatus status;

    if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u || slot == 0u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (g_flashPrepared != 0u || g_flashCommitted != 0u)
    {
        publish(request, (g_stageKind == PARAM_STAGE_RUNTIME &&
            g_stagedSlot == slot &&
            g_stagedTransactionId == transactionId) ? PARAM_SYNC_STATUS_OK :
            PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
        return;
    }
    status = core3VisionParamsFlashPrepareRuntimeFromSlot(
        (uint8_t) (slot - 1u), transactionId);
    if (status != CORE3_VISION_PARAMS_SAVE_VERIFIED)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    clearStaging();
    g_stagedSlot = slot;
    g_stagedTransactionId = transactionId;
    g_stagedValid = 1u;
    g_flashPrepared = 1u;
    g_stageKind = PARAM_STAGE_RUNTIME;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleCommitRuntime(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;
    if (decodeGlobalTransaction(request, 1u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u ||
        g_stageKind != PARAM_STAGE_RUNTIME || g_flashPrepared == 0u ||
        g_stagedSlot != slot || g_stagedTransactionId != transactionId)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (g_flashCommitted == 0u &&
        core3VisionParamsFlashCommitRuntime(transactionId) !=
            CORE3_VISION_PARAMS_SAVE_VERIFIED)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    g_flashCommitted = 1u;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleApplyRuntime(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;
    if (decodeGlobalTransaction(request, 1u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u ||
        g_stageKind != PARAM_STAGE_RUNTIME || g_flashCommitted == 0u ||
        g_stagedSlot != slot || g_stagedTransactionId != transactionId)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
    }
}

```

```

        return;
    }
    if (g_runtimeApplied == 0u &&
        core3VisionParamsFlashApplyPreparedRuntime(transactionId) !=
            CORE3_VISION_PARAMS_SAVE_VERIFIED)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    g_runtimeApplied = 1u;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handlePrepareBegin(const paramSyncMailboxRequest *request)
{
    uint32_t transactionId;
    if ((request->dataLength != 19u) ||
        (externalSlotValid(request->data[0]) == 0u) ||
        (getU32(&request->data[5]) != CORE3_VISION_PARAMS_FLASH_VERSION) ||
        (getU32(&request->data[9]) != CORE3_PARAM_SYNC_SCHEMA_HASH) ||
        (getU16(&request->data[17]) != CORE3_VISION_PARAMS_BLOB_BYTES))
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    transactionId = getU32(&request->data[1]);
    if (transactionId == 0u || transactionId == UINT32_MAX)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (g_flashPrepared != 0u && rollbackTransaction(g_stagedTransactionId) == 0u)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    clearStaging();
    g_stagedSlot = request->data[0];
    g_stagedTransactionId = transactionId;
    g_stagedExpectedCrc = getU32(&request->data[13]);
    g_stagedBytes = getU16(&request->data[17]);
    g_stagedValid = 1u;
    g_stageKind = PARAM_STAGE_LIBRARY;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handlePrepareChunk(const paramSyncMailboxRequest *request)
{
    uint16_t offset;
    uint16_t length;
    if ((g_stagedValid == 0u) || (request->dataLength < 5u))
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    offset = getU16(&request->data[1]);
    length = getU16(&request->data[3]);
    if ((request->data[0] != g_stagedSlot) ||
        (offset != g_stagedReceived) ||
        (length != (uint16_t) (request->dataLength - 5u)) ||
        ((uint32_t) offset + length > g_stagedBytes))
    {
        clearStaging();
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    memcpy(&g_stagedBlob.bytes[offset], &request->data[5], length);
    g_stagedReceived = (uint16_t) (g_stagedReceived + length);
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handlePrepareFinish(const paramSyncMailboxRequest *request)
{
    uint32_t internalCrc;
    core3VisionParamsStruct params;
    core3VisionParamsStruct sanitized;
    if ((g_stagedValid == 0u) || (request->dataLength != 1u) ||
        (request->data[0] != g_stagedSlot) ||

```

```

    (g_stagedReceived != g_stagedBytes))
{
    clearStaging();
    publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
    return;
}
internalCrc = getU32(
    &g_stagedBlob.bytes[CORE3_VISION_PARAMS_BLOB_CRC_BYTES]);
if (g_flashPrepared != 0u)
{
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, internalCrc);
    return;
}
if (core3VisionParamsBlobDecode(&g_stagedBlob, &params) ==
    CORE3_VISION_PARAMS_BLOB_VALID)
{
    sanitized = params;
    core3VisionParamsSanitize(&sanitized);
}
else
{
    clearStaging();
    publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}
if ((internalCrc != g_stagedExpectedCrc) ||
    (crc32IeeeCalc(g_stagedBlob.bytes,
        CORE3_VISION_PARAMS_BLOB_CRC_BYTES) != internalCrc) ||
    (memcmp(&sanitized, &params, sizeof(params)) != 0) ||
    (core3VisionParamsFlashPrepareSlot((uint8_t) (g_stagedSlot - 1u),
        &params, g_stagedTransactionId) !=
        CORE3_VISION_PARAMS_SAVE_VERIFIED))
{
    clearStaging();
    publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}
g_flashPrepared = 1u;
publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, internalCrc);
}

static void handleCommit(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;
    if ((request->dataLength != 5u) ||
        (externalSlotValid(request->data[0]) == 0u))
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    slot = request->data[0];
    transactionId = getU32(&request->data[1]);
    if ((g_stageKind != PARAM_STAGE_LIBRARY) ||
        (g_stagedValid == 0u) || (g_flashPrepared == 0u) ||
        (slot != g_stagedSlot) || (transactionId != g_stagedTransactionId))
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    if (g_flashCommitted != 0u)
    {
        publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
        return;
    }
    if (core3VisionParamsFlashCommitPrepared((uint8_t) (slot - 1u),
        transactionId) != CORE3_VISION_PARAMS_SAVE_VERIFIED)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    g_flashCommitted = 1u;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleActivate(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint8_t oldSlotExternal, pendingSlot;

```

```

uint32_t transactionId;
if ((request->dataLength != 5u) ||
    (externalSlotValid(request->data[0]) == 0u))
{
    publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
    return;
}
slot = request->data[0];
transactionId = getU32(&request->data[1]);
if (((transactionId != 0u) && ((g_stagedValid == 0u) ||
    (g_flashCommitted == 0u) ||
    (transactionId != g_stagedTransactionId) ||
    (slot != g_stagedSlot))))
{
    publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}
if (transactionId == 0u)
{
    if (!g_syncMutationLocked || g_syncLockedSlot != slot ||
        g_stagedValid || g_flashPrepared ||
        g_flashCommitted)
    {
        publish(request, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
        return;
    }
    if (activationContext(&oldSlotExternal, &pendingSlot))
    {
        publish(request, pendingSlot == slot ? PARAM_SYNC_STATUS_OK : PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
        return;
    }
    oldSlotExternal = (uint8_t) (core3VisionParamsFlashGetActiveSlot() + 1u);
    if (!externalSlotValid(oldSlotExternal))
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    setActivationContext(oldSlotExternal, slot);
}
if (!(transactionId == 0u ? activateRuntimeSlotPending(slot) :
    activatePreparedRuntimeSlot(slot, transactionId)))
{
    if (transactionId == 0u) (void) rollbackActivation();
    publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}
publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleAbort(const paramSyncMailboxRequest *request)
{
    uint32_t transactionId;
    if (request->dataLength != 4u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    transactionId = getU32(request->data);
    if (transactionId == UINT32_MAX)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (transactionId == 0u)
    {
        uint8_t ok = rollbackActivation();
        if (ok) clearMutationLock();
        publish(request, ok ? PARAM_SYNC_STATUS_OK : PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    if (!g_stagedValid && !g_flashPrepared && !g_flashCommitted &&
        transactionId == g_lastAbortedTransaction)
    {
        publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
        return;
    }
    if (((g_stagedValid != 0u) &&
        (transactionId != g_stagedTransactionId)) ||
        (rollbackTransaction(transactionId) == 0u))

```

```

    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    g_lastAbortedTransaction = transactionId;
    clearStaging();
    clearMutationLock();
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleFinish(const paramSyncMailboxRequest *request)
{
    uint32_t transactionId;
    if (request->dataLength != 4u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    transactionId = getU32(request->data);
    if (transactionId == 0u)
    {
        if (core3VisionParamsFlashFinishPendingActivation() !=
            CORE3_VISION_PARAMS_SAVE_VERIFIED)
        {
            publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
            return;
        }
        clearActivationContext();
        clearMutationLock();
        publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
        return;
    }
    if (!g_stagedValid && !g_flashPrepared && !g_flashCommitted &&
        transactionId == g_lastFinishedTransaction)
    {
        publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
        return;
    }
    if ((g_stagedValid != 0u) &&
        (((g_stageKind == PARAM_STAGE_LIBRARY) &&
          (g_flashCommitted == 0u)) ||
         ((g_stageKind == PARAM_STAGE_RUNTIME) &&
          ((g_flashCommitted == 0u) || (g_runtimeApplied == 0u))) ||
          (transactionId != g_stagedTransactionId)))
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    if (core3VisionParamsFlashFinishTransaction(transactionId) !=
        CORE3_VISION_PARAMS_SAVE_VERIFIED)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    g_lastFinishedTransaction = transactionId;
    clearStaging();
    clearMutationLock();
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleLock(const paramSyncMailboxRequest *request)
{
    uint8_t oldSlot, newSlot;
    uint32_t transactionId = (request->dataLength == 5u) ?
        getU32(&request->data[1]) : 0u;
    if (request->dataLength != 5u || request->data[0] > 5u ||
        transactionId == 0u || transactionId == UINT32_MAX ||
        (g_syncMutationLocked && (g_syncLockedSlot != request->data[0] ||
          g_syncLockedTransaction != transactionId)) ||
        g_stagedValid || g_flashPrepared || g_flashCommitted ||
        core3VisionParamsFlashMutationPending() ||
        activationContext(&oldSlot, &newSlot))
    {
        publish(request, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
        return;
    }
    g_syncMutationLocked = 1u;
    g_syncLockedSlot = request->data[0];
    g_syncLockedTransaction = transactionId;

```



```

    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static uint8_t retainedRequestMayResume(
    const paramSyncMailboxRequest *request)
{
    if (request->target != CORE3_PARAM_SYNC_TARGET) return 0u;
    return (uint8_t) (request->op == PARAM_SYNC_OP_QUERY ||
        request->op == PARAM_SYNC_OP_READ_META ||
        request->op == PARAM_SYNC_OP_READ_CHUNK ||
        request->op == PARAM_SYNC_OP_READ_RUNTIME_META ||
        request->op == PARAM_SYNC_OP_READ_RUNTIME_CHUNK ||
        request->op == PARAM_SYNC_OP_RECOVERY ||
        request->op == PARAM_SYNC_OP_ABORT ||
        request->op == PARAM_SYNC_OP_FINISH);
}

void core3ParamSyncOwnerServiceInit(void)
{
    paramSyncMailboxRequest retainedRequest;
    g_lastBootGeneration = 0u;
    g_lastRequestSequence = 0u;
    g_responseSequence = 0u;
    core3CoreDataTimingInit();
    if (g_ownerBootGeneration == 0u)
        g_ownerBootGeneration = core3CoreDataMakeBootGeneration(0xC3 735 053u);
    if (paramSyncMailboxTryReadRequest(&retainedRequest) != 0u &&
        retainedRequestMayResume(&retainedRequest) == 0u)
    {
        g_lastBootGeneration = retainedRequest.producerBootGeneration;
        g_lastRequestSequence = retainedRequest.sequence;
        if (retainedRequest.target == CORE3_PARAM_SYNC_TARGET)
            publish(&retainedRequest, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
    }
}

void core3ParamSyncOwnerServicePoll(void)
{
    paramSyncMailboxRequest request;
    if (g_ownerBootGeneration == 0u)
        g_ownerBootGeneration = core3CoreDataMakeBootGeneration(0xC3 735 053u);
    if (paramSyncMailboxTryReadRequest(&request) == 0u) return;
    if ((request.producerBootGeneration == g_lastBootGeneration) &&
        (request.sequence == g_lastRequestSequence)) return;
    if (request.producerBootGeneration == g_lastBootGeneration &&
        (int32_t) (request.sequence - g_lastRequestSequence) <= 0) return;
    if ((g_lastBootGeneration != 0u) &&
        (request.producerBootGeneration != g_lastBootGeneration) &&
        (producerBootChangeAllowed(&request) == 0u))
    {
        if (request.target == CORE3_PARAM_SYNC_TARGET)
            publish(&request, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
        return;
    }
    g_lastBootGeneration = request.producerBootGeneration;
    g_lastRequestSequence = request.sequence;
    if (request.target != CORE3_PARAM_SYNC_TARGET) return;
    if (recoverActivationAfterOwnerBoot() == 0u)
    {
        publish(&request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    switch (request.op)
    {
    case PARAM_SYNC_OP_READ_META: handleReadMeta(&request); break;
    case PARAM_SYNC_OP_READ_CHUNK: handleReadChunk(&request); break;
    case PARAM_SYNC_OP_READ_RUNTIME_META:
        handleReadRuntimeMeta(&request); break;
    case PARAM_SYNC_OP_READ_RUNTIME_CHUNK:
        handleReadRuntimeChunk(&request); break;
    case PARAM_SYNC_OP_PREPARE_LIBRARY_FROM_LIVE:
        handlePrepareLibraryFromLive(&request); break;
    case PARAM_SYNC_OP_PREPARE_RUNTIME_FROM_SLOT:
        handlePrepareRuntime(&request); break;
    case PARAM_SYNC_OP_COMMIT_RUNTIME:
        handleCommitRuntime(&request); break;
    case PARAM_SYNC_OP_APPLY_RUNTIME:
        handleApplyRuntime(&request); break;
    case PARAM_SYNC_OP_PREPARE_BEGIN: handlePrepareBegin(&request); break;

```

```

        case PARAM_SYNC_OP_PREPARE_CHUNK: handlePrepareChunk(&request); break;
        case PARAM_SYNC_OP_PREPARE_VERIFY: handlePrepareFinish(&request); break;
        case PARAM_SYNC_OP_COMMIT: handleCommit(&request); break;
        case PARAM_SYNC_OP_ACTIVATE: handleActivate(&request); break;
        case PARAM_SYNC_OP_ABORT: handleAbort(&request); break;
        case PARAM_SYNC_OP_FINISH: handleFinish(&request); break;
        case PARAM_SYNC_OP_LOCK: handleLock(&request); break;
        default:
            publish(&request, PARAM_SYNC_STATUS_REJECTED, NULL, 0u, 0u);
            break;
    }
}

```

core3_cyt4bb7/project/code/comm/paramSyncOwnerService.h

SHA-256

:

d4ac1bc8189e06307a99a96b86921bfe63e95b4a592fab580f519f1a7789f16d

代码 (c)

```

#ifndef CORE3_PARAM_SYNC_OWNER_SERVICE_H
#define CORE3_PARAM_SYNC_OWNER_SERVICE_H

#include <stdint.h>

void core3ParamSyncOwnerServiceInit(void);
void core3ParamSyncOwnerServicePoll(void);
uint8_t core3ParamSyncOwnerServiceMutationLocked(void);

#endif

```

core3_cyt4bb7/project/code/config/commonDef.h

SHA-256

:

1050cf30dc102938618387d9616381a7405e30ae19ba35586bb68f6322da5ec1

代码 (c)

```

#ifndef CORE3_COMMON_DEF_H
#define CORE3_COMMON_DEF_H

#include <stdint.h>

#define range(x, min, max)      (((x) < (min)) ? (min) : (((x) > (max)) ? (max) : (x)))
#define unsignRange(x, max)     (((x) > (max)) ? (max) : (x))

#define toRad(x)                ((x) * 0.017453292519943295f)
#define toDeg(x)                ((x) * 57.29577951308232f)

#ifndef PI
#define PI                      (3.14159265358979323846f)
#endif
#define TWO_PI                  (6.28318530717958647692f)
#define HALF_PI                 (1.57079632679489661923f)

#define BLACK                   (0)
#define WHITE                   (255)
#define cReverse(x)             ((x) ? BLACK : WHITE)

static inline float clampf(float value, float minValue, float maxValue)
{
    return (value < minValue) ? minValue : ((value > maxValue) ? maxValue : value);
}

static inline float wrapToPi(float angle)
{
    while (angle > PI)
    {
        angle -= TWO_PI;
    }
}

```

```

        while (angle < -PI)
        {
            angle += TWO_PI;
        }
        return angle;
    }

typedef struct
{
    uint8_t* ptr;
    uint32_t xLen;
    uint32_t yLen;
    uint32_t size;
} u8Image;

#endif /* CORE3_COMMON_DEF_H */

```

core3_cyt4bb7/project/code/config/configBeaconTrack.h

SHA-256

:

ceba4379d718229a747d517b7abc0d129509210661d45de9320541e98f0020b4

代码 (c)

```

#ifndef _CONFIG_BEACON_TRACK_H_
#define _CONFIG_BEACON_TRACK_H_

/*
 * Core3 信标"跟踪层"调参中心：时间一致性跟踪（抗光关键层）。
 * 连续 confirmHits 帧关联才确认输出、连续 dropMisses 帧丢失才删除、coastMisses 帧内丢失仍滑行。
 * 关联更新用一阶低通平滑（位置/面积各一 alpha，越小越平滑）；新轨迹直贴测量值。
 * 轨迹内部用坐标速度(px/frame)与面积速度(area pixels/frame)预测下一帧关联残差；
 * 这些速度只作为轨迹事实，不负责 active target 选择。
 * 详见 vision/beaconTrack.c。共享跟踪参数由 Core1 配置下发并在帧边界生效。
 *
 * Core3 相机局部阈值不在本文件，由 configCore3VisionParams.h 管理。
 */

#define VISION_TRACK_MAX_ASSOC_DIST_DEFAULT (6.0f) /* 帧间质心最大关联距离(px) */
#define VISION_TRACK_SIZE_RATIO_MAX_DEFAULT (2.0f) /* 面积连续性: 检测面积/预测面积 比值上限 */
#define VISION_TRACK_CONFIRM_HITS_DEFAULT (3u) /* 连续命中确认帧数 */
#define VISION_TRACK_DROP_MISSES_DEFAULT (4u) /* 已确认轨迹连续丢失删除帧数 */
#define VISION_TRACK_COAST_MISSES_DEFAULT (2u) /* 已确认轨迹丢失时仍输出(滑行)帧数 */

/* 发布边界施密特门：0/0 表示该边关闭；非关闭边 Enter>=1 且 Exit<Enter。
 * Exit 需连续越界 hold 帧才撤销发布资格，滤掉边缘单帧检测抖动。
 * Core3 默认关闭下边；正式运行值由 Core1 共享配置同步。 */
#define VISION_TRACK_BORDER_EXIT_HOLD_FRAMES_DEFAULT (2u)
#define VISION_TRACK_EDGE_PREDICTION_HOLD_MS_DEFAULT (500u)
#define VISION_TRACK_EDGE_PREDICTION_HOLD_MS_MAX (10 000u)
#define VISION_TRACK_CAMERA_FRAME_PERIOD_MS (20u)
#define VISION_TRACK_EDGE_BAND_C1_LEFT_DEFAULT (4u)
#define VISION_TRACK_EDGE_BAND_C1_RIGHT_DEFAULT (10u)
#define VISION_TRACK_EDGE_BAND_C1_TOP_DEFAULT (4u)
#define VISION_TRACK_EDGE_BAND_C1_BOTTOM_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_C3_LEFT_DEFAULT (4u)
#define VISION_TRACK_EDGE_BAND_C3_RIGHT_DEFAULT (4u)
#define VISION_TRACK_EDGE_BAND_C3_TOP_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_C3_BOTTOM_DEFAULT (10u)
#define VISION_TRACK_EDGE_BAND_X_MAX (93.0f)
#define VISION_TRACK_EDGE_BAND_Y_MAX (59.0f)
#define VISION_TRACK_EDGE_VERTICAL_ASSOC_SCALE (2.0f)
#define VISION_TRACK_EDGE_ASSOC_PREDICT_FRAMES_MAX (3.0f)
#define VISION_TRACK_BORDER_ENTER_LEFT_DEFAULT (24u)
#define VISION_TRACK_BORDER_EXIT_LEFT_DEFAULT (9u)
#define VISION_TRACK_BORDER_ENTER_RIGHT_DEFAULT (24u)
#define VISION_TRACK_BORDER_EXIT_RIGHT_DEFAULT (9u)
#define VISION_TRACK_BORDER_ENTER_TOP_DEFAULT (8u)
#define VISION_TRACK_BORDER_EXIT_TOP_DEFAULT (3u)
#define VISION_TRACK_BORDER_ENTER_BOTTOM_DEFAULT (0u)
#define VISION_TRACK_BORDER_EXIT_BOTTOM_DEFAULT (0u)
#define VISION_TRACK_BORDER_X_MAX (93u)
#define VISION_TRACK_BORDER_Y_MAX (59u)

/* 关联平滑一阶低通 alpha （tr = alpha*measured + (1-alpha)*tr，越小越平滑、越抗抖）。

```

```

* 位置/面积独立，默认 1.00（直贴测量值；原硬编码 0.5 IIR 现可调）。上板复标：移动信标 alpha 太小易掉关联。 */
#define VISION_TRACK_POSITION_ALPHA_DEFAULT (1.00f)
#define VISION_TRACK_SIZE_ALPHA_DEFAULT (1.00f)
#define VISION_TRACK_ALPHA_MIN (0.01f)
#define VISION_TRACK_ALPHA_MAX (1.00f)
#define VISION_TRACK_ALPHA_STEP_SHORT (0.05f)
#define VISION_TRACK_ALPHA_STEP_LONG (0.20f)

/* 轨迹速度预测（单位 px/frame，仅用于下一帧关联与事实输出；不进入控制 ABI）。 */
#define VISION_TRACK_VELOCITY_ALPHA_DEFAULT (0.35f)
#define VISION_TRACK_VELOCITY_MAX_FACTOR_DEFAULT (1.50f)
#define VISION_TRACK_PREDICT_FRAMES_DEFAULT (1.0f)

/* 面积趋势辅助关联。areaVel 单位为 area pixels/frame。
* 高影响权重/阈值进入 visionParams 菜单 + flash，归一化与 cap 仍为编译期安全约束。
* trackSizeRatioMax 仍是持久化参数，语义保持“面积连续性比值上限”，本轮只把比较基准改为预测面积。 */
#define VISION_TRACK_AREA_VEL_ALPHA_DEFAULT (0.35f)
#define VISION_TRACK_AREA_NORM_MIN (16.0f)
#define VISION_TRACK_ASSOC_SCORE_MAX_DEFAULT (1.75f)
#define VISION_TRACK_ASSOC_AREA_WEIGHT_DEFAULT (0.45f)
#define VISION_TRACK_ASSOC_AREA_VEL_WEIGHT_DEFAULT (0.35f)
#define VISION_TRACK_ASSOC_AREA_ERR_CAP (3.0f)
#define VISION_TRACK_ASSOC_AREA_VEL_ERR_CAP (3.0f)
#define VISION_TRACK_ASSOC_AREA_VEL_NORM (0.35f)
#define VISION_TRACK_ASSOC_AREA_VEL_MIN_ABS (8.0f)
#define VISION_TRACK_AREA_VEL_MAX_RATE (1.0f)

/* 菜单编辑约束（confirm/drop/coast 帧数、关联距离、面积比）。 */
#define VISION_TRACK_ASSOC_DIST_MIN (1.0f)
#define VISION_TRACK_ASSOC_DIST_MAX (100.0f)
#define VISION_TRACK_SIZE_RATIO_MIN (1.1f)
#define VISION_TRACK_SIZE_RATIO_MAX (10.0f)
#define VISION_TRACK_FRAMES_MIN (1.0f)
#define VISION_TRACK_FRAMES_MAX (60.0f)
#define VISION_TRACK_ASSOC_DIST_STEP_SHORT (0.5f)
#define VISION_TRACK_ASSOC_DIST_STEP_LONG (2.0f)
#define VISION_TRACK_SIZE_RATIO_STEP_SHORT (0.1f)
#define VISION_TRACK_SIZE_RATIO_STEP_LONG (0.5f)
#define VISION_TRACK_FRAMES_STEP_SHORT (1.0f)
#define VISION_TRACK_FRAMES_STEP_LONG (2.0f)

/* 面积趋势菜单编辑约束。权重 0=关闭对应 score 项。 */
#define VISION_TRACK_ASSOC_SCORE_MAX_MIN (0.10f)
#define VISION_TRACK_ASSOC_SCORE_MAX_MAX (10.00f)
#define VISION_TRACK_ASSOC_SCORE_MAX_STEP_SHORT (0.05f)
#define VISION_TRACK_ASSOC_SCORE_MAX_STEP_LONG (0.25f)
#define VISION_TRACK_ASSOC_WEIGHT_MIN (0.00f)
#define VISION_TRACK_ASSOC_WEIGHT_MAX (3.00f)
#define VISION_TRACK_ASSOC_WEIGHT_STEP_SHORT (0.05f)
#define VISION_TRACK_ASSOC_WEIGHT_STEP_LONG (0.25f)

#endif /* _CONFIG_BEACON_TRACK_H_ */
```

core3_cyt4bb7/project/code/config/configCore3Runtime.h

SHA-256 :
13507db170c3d74dbfe70208624413a50aeaae05970e252ca9f7cb10a1674002

代码 (c)

```
#ifndef CORE3_CONFIG_CORE3_RUNTIME_H
#define CORE3_CONFIG_CORE3_RUNTIME_H

#define CORE3_RUNTIME_PIT_MS (10u)

#define CORE3_CORE_SHARED_HEARTBEAT_MS (50u)
#define CORE3_CORE_SHARED_STALE_MS (600u)

#define CORE3_CAMERA_WIDTH (188u)
#define CORE3_CAMERA_HEIGHT (120u)
#define CORE3_CAMERA_EXPECTED_FRAME_MS (20u)
#define CORE3_CAMERA_FRAME_TIMEOUT_MS (200u)
#define CORE3_CAMERA_INIT_RETRY_MS (1000u)
#define CORE3_CAMERA_CONFIG_RETRY_MS (1000u)
#define CORE3_CAMERA_RECOVERY_FRAMES (3u)
```

```

#define CORE3_DIAGNOSTICS_PREVIEW_X    (2u)
#define CORE3_DIAGNOSTICS_PREVIEW_Y    (17u)
#define CORE3_DIAGNOSTICS_PREVIEW_WIDTH (130u)
#define CORE3_DIAGNOSTICS_PREVIEW_HEIGHT (67u)
#define CORE3_DIAGNOSTICS_HUD_CHARS    (24u)
#define CORE3_MENU_RENDER_PERIOD_MS    (20u)

#define CORE3_RGB_PWM_FREQUENCY_HZ     (1000u)
#define CORE3_RGB_LOGICAL_BRIGHTNESS  (1000u)
#define CORE3_RGB_SLOW_HALF_PERIOD_MS (500u)
#define CORE3_RGB_FAST_HALF_PERIOD_MS (200u)

#define CORE3_TIMING_INVALID_US (0xFFFFFFFFu)

#endif /* CORE3_CONFIG_CORE3_RUNTIME_H */

```

core3_cyt4bb7/project/code/config/configCore3VisionParams.h

SHA-256

:

538214221beb58f5775f356e2b695b99de7ec7f99e2da074e5201128379638fe

代码 (c)

```

#ifndef CORE3_CONFIG_CORE3_VISION_PARAMS_H
#define CORE3_CONFIG_CORE3_VISION_PARAMS_H

/* 启动先建立旧 CYT2 等价的 500 曝光；活动参数随后按本默认值应用。 */
#define CORE3_VISION_EXPOSURE_DEFAULT (300u)
#define CORE3_VISION_EXPOSURE_MIN     (1u)
#define CORE3_VISION_EXPOSURE_MAX     (2000u)
#define CORE3_VISION_EXPOSURE_STEP_S  (1.0f)
#define CORE3_VISION_EXPOSURE_STEP_L  (10.0f)

#define CORE3_VISION_BINARY_LOW_DEFAULT (40u)
#define CORE3_VISION_BINARY_HIGH_DEFAULT (255u)
#define CORE3_VISION_BINARY_MIN         (5u)
#define CORE3_VISION_BINARY_MAX         (255u)
#define CORE3_VISION_BINARY_STEP_S      (1.0f)
#define CORE3_VISION_BINARY_STEP_L      (10.0f)

#define CORE3_VISION_LEGACY_CANNY_LOW_DEFAULT (150u)
#define CORE3_VISION_LEGACY_CANNY_HIGH_DEFAULT (200u)
#define CORE3_VISION_LEGACY_CANNY_MIN         (5u)
#define CORE3_VISION_LEGACY_CANNY_MAX         (500u)
#define CORE3_VISION_LEGACY_CANNY_GAP         (5u)
#define CORE3_VISION_LEGACY_CANNY_STEP_S      (1.0f)
#define CORE3_VISION_LEGACY_CANNY_STEP_L      (10.0f)

#define CORE3_VISION_HUD_PERIOD_DEFAULT_MS (200u)
#define CORE3_VISION_HUD_PERIOD_MIN_MS     (20u)
#define CORE3_VISION_HUD_PERIOD_MAX_MS     (200u)
#define CORE3_VISION_HUD_PERIOD_STEP_S      (10.0f)
#define CORE3_VISION_HUD_PERIOD_STEP_L      (100.0f)

#define CORE3_VISION_PREVIEW_PERIOD_DEFAULT_MS (40u)
#define CORE3_VISION_PREVIEW_PERIOD_MIN_MS     (4u)
#define CORE3_VISION_PREVIEW_PERIOD_MAX_MS     (2000u)
#define CORE3_VISION_PREVIEW_PERIOD_STEP_S      (50.0f)
#define CORE3_VISION_PREVIEW_PERIOD_STEP_L      (500.0f)

#define CORE3_VISION_STATS_PERIOD_DEFAULT_MS (1000u)
#define CORE3_VISION_STATS_PERIOD_MIN_MS     (100u)
#define CORE3_VISION_STATS_PERIOD_MAX_MS     (5000u)
#define CORE3_VISION_STATS_PERIOD_STEP_S      (100.0f)
#define CORE3_VISION_STATS_PERIOD_STEP_L      (500.0f)

#endif /* CORE3_CONFIG_CORE3_VISION_PARAMS_H */

```

core3_cyt4bb7/project/code/config/configSerialDebug.h

SHA-256

:

12550932f48bb5df2d76c4c0d852d36ded5c8d36b3f79c4882a8ebcac8da5d9a

代码 (c)

```
#ifndef CORE3_CONFIG_SERIAL_DEBUG_H
#define CORE3_CONFIG_SERIAL_DEBUG_H

/* cm7_0 启动日志使用独立 UART1; Core1 业务专线固定由 UART5 承载。 */
#define SERIAL_DEBUG_UART_INDEX      (UART_1)
#define SERIAL_DEBUG_UART_BAUDRATE   (115200)
#define SERIAL_DEBUG_UART_TX_PIN     (UART1_TX_P04_1)
#define SERIAL_DEBUG_UART_RX_PIN     (UART1_RX_P04_0)
#define SERIAL_DEBUG_UART_USE_INTERRUPT (1)

#if (SERIAL_DEBUG_UART_BAUDRATE < 1)
#error "SERIAL_DEBUG_UART_BAUDRATE must be > 0"
#endif

#endif /* CORE3_CONFIG_SERIAL_DEBUG_H */
```

core3_cyt4bb7/project/code/config/core3VisionParams.c

SHA-256

:

54643113f7770958f45bb3ae2608f571d88c909a1a15f0f4863a3516f133becc

代码 (c)

```
#include "core3VisionParams.h"

#include <string.h>

static core3VisionParamsStruct g_activeParams;
static uint32_t g_revision;

static uint32_t clampU32(uint32_t value, uint32_t minimum, uint32_t maximum)
{
    if (value < minimum)
    {
        return minimum;
    }
    if (value > maximum)
    {
        return maximum;
    }
    return value;
}

static uint16_t clampU16(uint16_t value, uint16_t minimum, uint16_t maximum)
{
    if (value < minimum)
    {
        return minimum;
    }
    if (value > maximum)
    {
        return maximum;
    }
    return value;
}

void core3VisionParamsLoadDefaults(core3VisionParamsStruct *params)
{
    if (params == NULL)
    {
        return;
    }
}
```

```

memset(params, 0, sizeof(*params));
params->exposure = CORE3_VISION_EXPOSURE_DEFAULT;
params->hudPeriodMs = CORE3_VISION_HUD_PERIOD_DEFAULT_MS;
params->previewPeriodMs = CORE3_VISION_PREVIEW_PERIOD_DEFAULT_MS;
params->statsPeriodMs = CORE3_VISION_STATS_PERIOD_DEFAULT_MS;
params->binaryLow = CORE3_VISION_BINARY_LOW_DEFAULT;
params->binaryHigh = CORE3_VISION_BINARY_HIGH_DEFAULT;
params->legacyCannyLow = CORE3_VISION_LEGACY_CANNY_LOW_DEFAULT;
params->legacyCannyHigh = CORE3_VISION_LEGACY_CANNY_HIGH_DEFAULT;
}

void core3VisionParamsSanitize(core3VisionParamsStruct *params)
{
    if (params == NULL)
    {
        return;
    }

    if (params->exposure < CORE3_VISION_EXPOSURE_MIN)
    {
        params->exposure = CORE3_VISION_EXPOSURE_MIN;
    }
    if (params->exposure > CORE3_VISION_EXPOSURE_MAX)
    {
        params->exposure = CORE3_VISION_EXPOSURE_MAX;
    }
    params->hudPeriodMs = clampU32(params->hudPeriodMs,
        CORE3_VISION_HUD_PERIOD_MIN_MS,
        CORE3_VISION_HUD_PERIOD_MAX_MS);
    params->previewPeriodMs = clampU32(params->previewPeriodMs,
        CORE3_VISION_PREVIEW_PERIOD_MIN_MS,
        CORE3_VISION_PREVIEW_PERIOD_MAX_MS);
    params->statsPeriodMs = clampU32(params->statsPeriodMs,
        CORE3_VISION_STATS_PERIOD_MIN_MS,
        CORE3_VISION_STATS_PERIOD_MAX_MS);
    params->binaryLow = clampU16(params->binaryLow,
        CORE3_VISION_BINARY_MIN,
        CORE3_VISION_BINARY_MAX);
    params->binaryHigh = clampU16(params->binaryHigh,
        CORE3_VISION_BINARY_MIN,
        CORE3_VISION_BINARY_MAX);
    if (params->binaryLow > params->binaryHigh)
    {
        params->binaryLow = params->binaryHigh;
    }
    params->legacyCannyLow = clampU16(params->legacyCannyLow,
        CORE3_VISION_LEGACY_CANNY_MIN,
        CORE3_VISION_LEGACY_CANNY_MAX);
    params->legacyCannyHigh = clampU16(params->legacyCannyHigh,
        CORE3_VISION_LEGACY_CANNY_MIN,
        CORE3_VISION_LEGACY_CANNY_MAX);
    if ((uint32_t) params->legacyCannyLow + CORE3_VISION_LEGACY_CANNY_GAP >
        params->legacyCannyHigh)
    {
        if (params->legacyCannyHigh >=
            (CORE3_VISION_LEGACY_CANNY_MIN + CORE3_VISION_LEGACY_CANNY_GAP))
        {
            params->legacyCannyLow = (uint16_t) (params->legacyCannyHigh -
                CORE3_VISION_LEGACY_CANNY_GAP);
        }
        else
        {
            params->legacyCannyLow = CORE3_VISION_LEGACY_CANNY_MIN;
            params->legacyCannyHigh = (uint16_t) (CORE3_VISION_LEGACY_CANNY_MIN +
                CORE3_VISION_LEGACY_CANNY_GAP);
        }
    }
}

void core3VisionParamsSetInitial(const core3VisionParamsStruct *params)
{
    if (params == NULL)
    {
        return;
    }

    g_activeParams = *params;
    core3VisionParamsSanitize(&g_activeParams);
    g_revision = 1u;
}

```

```

}

void core3VisionParamsCommit(const core3VisionParamsStruct *params)
{
    if (params == NULL)
    {
        return;
    }

    g_activeParams = *params;
    core3VisionParamsSanitize(&g_activeParams);
    g_revision++;
    if (g_revision == 0u) g_revision = 1u;
}

const core3VisionParamsStruct *core3VisionParamsGetActive(void)
{
    return &g_activeParams;
}

uint32_t core3VisionParamsGetRevision(void)
{
    return g_revision;
}

```

core3_cyt4bb7/project/code/config/core3VisionParams.h

SHA-256

:

bdaa54d2e1b85dbf615f60dadba3252057790e283a23179ca0cb0138bcfb3654

代码 (c)

```

#ifndef CORE3_VISION_PARAMS_H
#define CORE3_VISION_PARAMS_H

#include <stdint.h>

#include "configCore3VisionParams.h"

typedef struct
{
    uint16_t exposure;
    uint32_t hudPeriodMs;
    uint32_t previewPeriodMs;
    uint32_t statsPeriodMs;
    uint16_t binaryLow;
    uint16_t binaryHigh;
    uint16_t legacyCannyLow;
    uint16_t legacyCannyHigh;
} core3VisionParamsStruct;

void core3VisionParamsLoadDefaults(core3VisionParamsStruct *params);
void core3VisionParamsSanitize(core3VisionParamsStruct *params);
void core3VisionParamsSetInitial(const core3VisionParamsStruct *params);
void core3VisionParamsCommit(const core3VisionParamsStruct *params);
const core3VisionParamsStruct *core3VisionParamsGetActive(void);
uint32_t core3VisionParamsGetRevision(void);

#endif /* CORE3_VISION_PARAMS_H */

```

core3_cyt4bb7/project/code/config/core3VisionParamsBlob.c

SHA-256

:

6758c253fdcbff7b42ea5218aad6a72f50baf91781dff83b4107ad90dba90e7

代码 (c)

```

#include "core3VisionParamsBlob.h"
#include <string.h>
#include "crc32Ieee.h"

```



```
static void put16(uint8_t *p, uint16_t v) { p[0] = (uint8_t)v; p[1] = (uint8_t)(v >> 8); }
static void put32(uint8_t *p, uint32_t v) { p[0] = (uint8_t)v; p[1] = (uint8_t)(v >> 8); p[2] = (uint8_t)(v >> 16); p[3] = (uint8_t)(v >> 24); }
static uint16_t get16(const uint8_t *p) { return (uint16_t)((uint16_t)p[0] | ((uint16_t)p[1] << 8)); }
static uint32_t get32(const uint8_t *p) { return (uint32_t)p[0] | ((uint32_t)p[1] << 8) | ((uint32_t)p[2] << 16) | ((uint32_t)p[3] << 24); }
void core3VisionParamsBlobEncode(const core3VisionParamsStruct *p, core3VisionParamsFlashBlob *b) { if (!p || !b) return; memset(b, 0, sizeof(*b)); put32(&b->bytes[0], CORE3_VISION_PARAMS_FLASH_MAGIC); put32(&b->bytes[4], CORE3_VISION_PARAMS_FLASH_VERSION); put32(&b->bytes[8], CORE3_VISION_PARAMS_BLOB_BYTES); put16(&b->bytes[12], p->exposure); put32(&b->bytes[14], p->hudPeriodMs); put32(&b->bytes[18], p->previewPeriodMs); put32(&b->bytes[22], p->statsPeriodMs); put16(&b->bytes[26], p->binaryLow); put16(&b->bytes[28], p->binaryHigh); put16(&b->bytes[30], p->legacyCannyLow); put16(&b->bytes[32], p->legacyCannyHigh); put32(&b->bytes[34], crc32IeeeCalc(b->bytes, CORE3_VISION_PARAMS_BLOB_CRC_BYTES)); }
core3VisionParamsBlobStatus core3VisionParamsBlobDecode(const core3VisionParamsFlashBlob *b, core3VisionParamsStruct *p) { if (!b || !p) return CORE3_VISION_PARAMS_BLOB_BAD_LENGTH; if (get32(&b->bytes[0]) != CORE3_VISION_PARAMS_FLASH_MAGIC) return CORE3_VISION_PARAMS_BLOB_BAD_MAGIC; if (get32(&b->bytes[4]) != CORE3_VISION_PARAMS_FLASH_VERSION) return CORE3_VISION_PARAMS_BLOB_UNSUPPORTED_VERSION; if (get32(&b->bytes[8]) != CORE3_VISION_PARAMS_BLOB_BYTES) return CORE3_VISION_PARAMS_BLOB_BAD_LENGTH; if (get32(&b->bytes[34]) != crc32IeeeCalc(b->bytes, CORE3_VISION_PARAMS_BLOB_CRC_BYTES)) return CORE3_VISION_PARAMS_BLOB_BAD_CRC; memset(p, 0, sizeof(*p)); p->exposure = get16(&b->bytes[12]); p->hudPeriodMs = get32(&b->bytes[14]); p->previewPeriodMs = get32(&b->bytes[18]); p->statsPeriodMs = get32(&b->bytes[22]); p->binaryLow = get16(&b->bytes[26]); p->binaryHigh = get16(&b->bytes[28]); p->legacyCannyLow = get16(&b->bytes[30]); p->legacyCannyHigh = get16(&b->bytes[32]); return CORE3_VISION_PARAMS_BLOB_VALID; }
```

core3_cyt4bb7/project/code/config/core3VisionParamsBlob.h

SHA-256

:

7c6b2c88bc92e8c2377beed155630012a2535dbf760168d5d22894db1e2b7291

代码 (c)

```
#ifndef CORE3_VISION_PARAMS_BLOB_H
#define CORE3_VISION_PARAMS_BLOB_H

#include <stdint.h>
#include "core3VisionParams.h"

#define CORE3_VISION_PARAMS_FLASH_MAGIC (0x43335650u)
#define CORE3_VISION_PARAMS_FLASH_VERSION (8u)
#define CORE3_VISION_PARAMS_PARAM_BYTES (22u)
#define CORE3_VISION_PARAMS_BLOB_CRC_BYTES (34u)
#define CORE3_VISION_PARAMS_BLOB_BYTES (38u)

typedef struct { uint8_t bytes[CORE3_VISION_PARAMS_BLOB_BYTES]; } core3VisionParamsFlashBlob;
typedef enum { CORE3_VISION_PARAMS_BLOB_VALID=0, CORE3_VISION_PARAMS_BLOB_BAD_MAGIC, CORE3_VISION_PARAMS_BLOB_BAD_LENGTH, CORE3_VISION_PARAMS_BLOB_BAD_CRC, CORE3_VISION_PARAMS_BLOB_UNSUPPORTED_VERSION } core3VisionParamsBlobStatus;
void core3VisionParamsBlobEncode(const core3VisionParamsStruct *params, core3VisionParamsFlashBlob *blob);
core3VisionParamsBlobStatus core3VisionParamsBlobDecode(const core3VisionParamsFlashBlob *blob, core3VisionParamsStruct *params);
#endif
```

core3_cyt4bb7/project/code/config/core3VisionParamsPersistence.c

SHA-256

:

83a5ab63d06020b03ad7c9fba4c08776b87ecac2bbf7a11c915c7c72adba9ed9

代码 (c)

```
#include "core3VisionParamsPersistence.h"

#include <stddef.h>
#include <string.h>

#include "crc32Ieee.h"
#include "zf_driver_flash.h"

#define CORE3_SLOT_STORE_VERSION (2u)
#define CORE3_SLOT_RECORD_MAGIC (0x33534C54u) /* 3SLT */
#define CORE3_SLOT_SELECTOR_MAGIC (0x3353454Cu) /* 3SEL */
#define CORE3_SLOT_SELECTOR_PAGE_A (10u)
#define CORE3_SLOT_SELECTOR_PAGE_B (11u)
#define CORE3_SLOT_DATA_PAGE_FIRST (0u)
#define CORE3_SLOT_DATA_PAGE_LAST (CORE3_VISION_PARAMS_SLOT_COUNT * 2u - 1u)
#define CORE3_COORDINATOR_PAGE_A (12u)
#define CORE3_COORDINATOR_PAGE_B (13u)
```

```

#define CORE3_SLOT_VALID_RECORD      (1u)
#define CORE3_SLOT_VALID_LEGACY      (2u)
#define CORE3_SLOT_MAX_WORDS         (32u)
#define CORE3_FLASH_PAGE_WORDS       (FLASH_PAGE_SIZE / sizeof(uint32_t))
#define CORE3_SELECTOR_WORDS          ((sizeof(core3SlotSelector) + 3u) / 4u)
#define CORE3_SELECTOR_RECORDS        (CORE3_FLASH_PAGE_WORDS / CORE3_SELECTOR_WORDS)
#define CORE3_RUNTIME_SELECTOR_MAGIC (0x33544E52u) /* RNT3 */
#define CORE3_RUNTIME_SELECTOR_VERSION (1u)
#define CORE3_RUNTIME_DATA_PAGE_A     (14u)
#define CORE3_RUNTIME_DATA_PAGE_B     (15u)
#define CORE3_RUNTIME_SELECTOR_PAGE_A (16u)
#define CORE3_RUNTIME_SELECTOR_PAGE_B (17u)
#define CORE3_RUNTIME_SELECTOR_BYTES   (64u)
#define CORE3_RUNTIME_SELECTOR_WORDS   (CORE3_RUNTIME_SELECTOR_BYTES / 4u)
#define CORE3_RUNTIME_SELECTOR_RECORDS \
    (CORE3_FLASH_PAGE_WORDS / CORE3_RUNTIME_SELECTOR_WORDS)

```

```

typedef struct
{
    uint32_t magic;
    uint32_t version;
    uint32_t slot;
    uint32_t generation;
    uint32_t transactionId;
    uint32_t payloadBytes;
    core3VisionParamsFlashBlob payload;
    uint32_t crc;
} core3SlotRecord;

```

```

typedef struct
{
    uint32_t magic;
    uint32_t version;
    uint32_t generation;
    uint32_t activeSlot;
    uint32_t bank[CORE3_VISION_PARAMS_SLOT_COUNT];
    uint32_t valid[CORE3_VISION_PARAMS_SLOT_COUNT];
    uint32_t slotGeneration[CORE3_VISION_PARAMS_SLOT_COUNT];
    uint32_t transactionId[CORE3_VISION_PARAMS_SLOT_COUNT];
    uint32_t pendingTransactionId;
    uint32_t rollbackActiveSlot;
    uint32_t rollbackSlot;
    uint32_t rollbackBank;
    uint32_t rollbackValid;
    uint32_t rollbackGeneration;
    uint32_t rollbackTransactionId;
    uint32_t crc;
} core3SlotSelector;

```

```

typedef struct
{
    uint32_t magic;
    uint32_t version;
    uint32_t recordBytes;
    uint32_t sequence;
    uint8_t bank;
    uint8_t valid;
    uint8_t pending;
    uint8_t sourceSlot;
    uint32_t generation;
    uint32_t blobCrc;
    uint32_t transactionId;
    uint8_t rollbackBank;
    uint8_t rollbackValid;
    uint8_t rollbackSourceSlot;
    uint8_t reserved0;
    uint32_t rollbackGeneration;
    uint32_t rollbackBlobCrc;
    uint32_t rollbackTransactionId;
    uint32_t reserved[3u];
    uint32_t crc;
} core3RuntimeSelector;

```

```

typedef enum
{
    CORE3_FLASH_RANGE_BLANK = 0,
    CORE3_FLASH_RANGE_PROGRAMMED,
    CORE3_FLASH_RANGE_IO_FAILED
} core3FlashRangeState;

```

```

typedef enum
{
    CORE3_SELECTOR_NONE = 0,
    CORE3_SELECTOR_VALID,
    CORE3_SELECTOR_CORRUPT,
    CORE3_SELECTOR_REPAIR_LAST_GOOD,
    CORE3_SELECTOR_REBUILD_FROM_DATA
} core3SelectorLoadState;

static uint32_t g_flashWords[CORE3_SLOT_MAX_WORDS];
static uint8_t g_flashReady;
static uint8_t g_selectorReady;
static uint8_t g_selectorPage;
static uint8_t g_selectorRecord;
static uint8_t g_selectorUncertain;
static uint8_t g_selectorUncertainHadDurable;
static core3SlotSelector g_selector;
static core3RuntimeSelector g_runtimeSelector;
static uint8_t g_runtimeSelectorReady;
static uint8_t g_runtimeSelectorPage;
static uint8_t g_runtimeSelectorRecord;
static uint8_t g_runtimeSelectorUncertain;
static uint8_t g_runtimeSelectorUncertainHadDurable;
static core3VisionParamsPersistenceStatus g_status;
static struct
{
    core3SlotSelector before;
    uint32_t transactionId;
    uint32_t generation;
    uint8_t slot;
    uint8_t bank;
    uint8_t valid;
    uint8_t committed;
} g_prepared;
static struct
{
    uint32_t transactionId;
    uint32_t generation;
    uint32_t blobCrc;
    uint8_t sourceSlot;
    uint8_t bank;
    uint8_t valid;
    uint8_t committed;
} g_runtimePrepared;

_Static_assert(FLASH_PAGE_NUM >= 18u,
    "Core3 parameter slots exceed Work Flash");
_Static_assert(sizeof(core3SlotRecord) <= FLASH_PAGE_SIZE,
    "Core3 slot record must fit one page");
_Static_assert(sizeof(core3SlotSelector) <= FLASH_PAGE_SIZE,
    "Core3 slot selector must fit one page");
_Static_assert(((sizeof(core3SlotSelector) + 3u) / 4u) <= CORE3_SLOT_MAX_WORDS,
    "Core3 slot scratch buffer too small");
_Static_assert(sizeof(core3RuntimeSelector) == CORE3_RUNTIME_SELECTOR_BYTES,
    "Core3 runtime selector size drift");
_Static_assert(CORE3_SLOT_DATA_PAGE_FIRST <= CORE3_SLOT_DATA_PAGE_LAST &&
    CORE3_SLOT_DATA_PAGE_LAST < CORE3_SLOT_SELECTOR_PAGE_A,
    "Core3 slot data overlaps selector journal");
_Static_assert(CORE3_SLOT_SELECTOR_PAGE_A + 1u ==
    CORE3_SLOT_SELECTOR_PAGE_B &&
    CORE3_SLOT_SELECTOR_PAGE_B < CORE3_COORDINATOR_PAGE_A,
    "Core3 slot selector overlaps coordinator reserve");
_Static_assert(CORE3_COORDINATOR_PAGE_A + 1u == CORE3_COORDINATOR_PAGE_B &&
    CORE3_COORDINATOR_PAGE_B < CORE3_RUNTIME_DATA_PAGE_A,
    "Core3 coordinator reserve overlaps runtime data");
_Static_assert(CORE3_RUNTIME_DATA_PAGE_A + 1u ==
    CORE3_RUNTIME_DATA_PAGE_B &&
    CORE3_RUNTIME_DATA_PAGE_B < CORE3_RUNTIME_SELECTOR_PAGE_A &&
    CORE3_RUNTIME_SELECTOR_PAGE_A + 1u ==
    CORE3_RUNTIME_SELECTOR_PAGE_B &&
    CORE3_RUNTIME_SELECTOR_PAGE_B < FLASH_PAGE_NUM,
    "Core3 runtime pages overlap or exceed Work Flash");
_Static_assert(CORE3_SELECTOR_RECORDS > 0u &&
    CORE3_RUNTIME_SELECTOR_RECORDS > 0u,
    "Core3 selector records must fit one Work Flash page");

static void saturatingIncrement(uint32_t *value)
{

```

```

        if (*value < UINT32_MAX) (*value)++;
    }

static void ensureFlashReady(void)
{
    if (g_flashReady == 0u)
    {
        flash_init();
        g_flashReady = 1u;
    }
}

static uintptr_t flashPageAddress(uint32_t page)
{
    return (uintptr_t) FLASH_BASE_ADDR + (uintptr_t) page * FLASH_PAGE_SIZE;
}

static const uint32_t *flashWordAddress(uint32_t page, uint32_t wordIndex)
{
    return (const uint32_t *) (uintptr_t)
        (flashPageAddress(page) + wordIndex * sizeof(uint32_t));
}

static core3FlashRangeState blankCheckRange(uint32_t page,
uint32_t firstWord,
uint32_t wordCount)
{
    cy_stc_flash_blankcheck_config_t config = {0};
    cy_en_flashdrv_status_t status;

    if ((page >= FLASH_PAGE_NUM) || (wordCount == 0u) ||
        (firstWord >= CORE3_FLASH_PAGE_WORDS) ||
        (wordCount > (CORE3_FLASH_PAGE_WORDS - firstWord)))
    {
        return CORE3_FLASH_RANGE_IO_FAILED;
    }
    config.addrToBeChecked = flashWordAddress(page, firstWord);
    config.numOfWordsToBeChecked = wordCount;
    status = Cy_Flash_BlankCheck(NULL, &config, CY_FLASH_DRIVER_BLOCKING);
    if (status == CY_FLASH_DRV_SUCCESS) return CORE3_FLASH_RANGE_BLANK;
    if (status == CY_FLASH_DRV_FLASH_NOT_ERASED) return CORE3_FLASH_RANGE_PROGRAMMED;
    return CORE3_FLASH_RANGE_IO_FAILED;
}

static void invalidateFlashWordCache(const uint32_t *wordAddress)
{
    #if CY_CORE_CM7_0 || CY_CORE_CM7_1
        uintptr_t cacheLine = ((uintptr_t) wordAddress) & ~(uintptr_t) 31u;
        SCB_InvalidateDCache_by_Addr((uint32_t *) cacheLine, 32);
    #else
        (void) wordAddress;
    #endif
}

static uint8_t readFlashWordSafely(uint32_t page,
uint32_t wordIndex,
uint32_t *value)
{
    const uint32_t *address;
    core3FlashRangeState state;

    if ((value == NULL) || (wordIndex >= CORE3_FLASH_PAGE_WORDS)) return 1u;
    state = blankCheckRange(page, wordIndex, 1u);
    if (state == CORE3_FLASH_RANGE_BLANK)
    {
        *value = UINT32_MAX;
        return 0u;
    }
    if (state != CORE3_FLASH_RANGE_PROGRAMMED) return 1u;
    address = flashWordAddress(page, wordIndex);
    invalidateFlashWordCache(address);
    *value = *((const volatile uint32_t *) address);
    return 0u;
}

static uint8_t readPageBytes(uint32_t page, void *data, uint32_t length)
{
    uint32_t words;
    uint32_t i;

```

```

    if ((data == NULL) || (length == 0u)) return 1u;
    words = (length + 3u) / 4u;
    if (words > CORE3_SLOT_MAX_WORDS) return 1u;
    memset(g_flashWords, 0xFF, sizeof(g_flashWords));
    for (i = 0u; i < words; i++)
    {
        if (readFlashWordSafely(page, i, &g_flashWords[i]) != 0u) return 1u;
    }
    if (data != g_flashWords)
    {
        memcpy(data, g_flashWords, length);
    }
    return 0u;
}

static uint8_t erasePage(uint32_t page)
{
    cy_stc_flash_erasesector_config_t config = {0};
    cy_en_flashdrv_status_t status;

    config.blocking = CY_FLASH_ERASESECTOR_BLOCKING;
    config.intrMask = CY_FLASH_ERASESECTOR_NOT_SET_INTR_MASK;
    config.Addr = flashWordAddress(page, 0u);
    status = Cy_Flash_EraseSector(NULL, &config, CY_FLASH_DRIVER_BLOCKING);
    if (status != CY_FLASH_DRV_SUCCESS) return 1u;
    return (uint8_t) (blankCheckRange(page, 0u, CORE3_FLASH_PAGE_WORDS) ==
        CORE3_FLASH_RANGE_BLANK ? 0u : 1u);
}

static uint8_t programFlashWord(uint32_t page,
    uint32_t wordIndex,
    const uint32_t *value)
{
    cy_stc_flash_programrow_config_t config = {0};
    cy_en_flashdrv_status_t status;

    if ((value == NULL) || (wordIndex >= CORE3_FLASH_PAGE_WORDS)) return 1u;
    config.blocking = CY_FLASH_PROGRAMROW_BLOCKING;
    config.skipBC = CY_FLASH_PROGRAMROW_BLANK_CHECK;
    config.dataSize = CY_FLASH_PROGRAMROW_DATA_SIZE_32BIT;
    config.dataLoc = CY_FLASH_PROGRAMROW_DATA_LOCATION_SRAM;
    config.intrMask = CY_FLASH_PROGRAMROW_NOT_SET_INTR_MASK;
    config.destAddr = flashWordAddress(page, wordIndex);
    config.dataAddr = value;

#ifdef CY_CORE_CM7_0 || CY_CORE_CM7_1
    {
        uintptr_t start = ((uintptr_t) value) & ~(uintptr_t) 31u;
        uintptr_t end = (((uintptr_t) value + sizeof(*value) + 31u) &
            ~(uintptr_t) 31u);
        SCB_CleanDCache_by_Addr((volatile void *) start,
            (int32_t) (end - start));
    }
#endif
    status = Cy_Flash_ProgramRow(NULL, &config, CY_FLASH_DRIVER_BLOCKING);
    return (uint8_t) (status == CY_FLASH_DRV_SUCCESS ? 0u : 1u);
}

static uint8_t writePageAtomic(uint32_t page, const void *data, uint32_t length)
{
    {
        uint32_t words;
        uint32_t readback;
        uint32_t i;

        if ((data == NULL) || (length == 0u)) return 1u;
        words = (length + 3u) / 4u;
        if (words > CORE3_SLOT_MAX_WORDS) return 1u;
        if (erasePage(page) != 0u) return 1u;
        memset(g_flashWords, 0xFF, sizeof(g_flashWords));
        memcpy(g_flashWords, data, length);
        for (i = 1u; i < words; i++)
        {
            if ((programFlashWord(page, i, &g_flashWords[i]) != 0u) ||
                (readFlashWordSafely(page, i, &readback) != 0u) ||
                (readback != g_flashWords[i])) return 1u;
        }
        if (programFlashWord(page, 0u, &g_flashWords[0]) != 0u) return 1u;
        memset(g_flashWords, 0xFF, sizeof(g_flashWords));
        if (readPageBytes(page, g_flashWords, length) != 0u) return 1u;
    }
}

```

```

        return (uint8_t) (memcmp(data, g_flashWords, length) == 0 ? 0u : 1u);
    }

static uint8_t selectorSlotStateValid(uint32_t slot, uint32_t bank,
    uint32_t valid, uint32_t generation, uint32_t transactionId)
{
    if ((slot >= CORE3_VISION_PARAMS_SLOT_COUNT) || (bank > 1u) ||
        (valid > CORE3_SLOT_VALID_LEGACY)) return 0u;
    if (valid == 0u)
        return (uint8_t) ((bank == 0u) && (generation == 0u) &&
            (transactionId == 0u));
    if (generation == 0u) return 0u;
    if (valid == CORE3_SLOT_VALID_LEGACY)
        return (uint8_t) ((slot == 0u) && (bank == 0u) &&
            (generation == 1u) &&
            (transactionId == 0u));
    return (uint8_t) (transactionId != UINT32_MAX);
}

static uint8_t selectorValid(const core3SlotSelector *selector)
{
    uint32_t i;
    uint32_t crc;
    uint8_t anyValid = 0u;

    if ((selector->magic != CORE3_SLOT_SELECTOR_MAGIC) ||
        (selector->version != CORE3_SLOT_STORE_VERSION) ||
        (selector->generation == 0u) ||
        (selector->activeSlot > CORE3_VISION_PARAMS_SLOT_COUNT)) return 0u;
    for (i = 0u; i < CORE3_VISION_PARAMS_SLOT_COUNT; i++)
    {
        if (selectorSlotStateValid(i, selector->bank[i],
            selector->valid[i], selector->slotGeneration[i],
            selector->transactionId[i]) == 0u) return 0u;
        if (selector->valid[i] != 0u) anyValid = 1u;
    }
    if ((anyValid == 0u &&
        selector->activeSlot != CORE3_VISION_PARAMS_SLOT_COUNT) ||
        (anyValid != 0u &&
        (selector->activeSlot >= CORE3_VISION_PARAMS_SLOT_COUNT ||
        selector->valid[selector->activeSlot] == 0u))) return 0u;
    if (selector->pendingTransactionId != 0u)
    {
        uint32_t slot = selector->rollbackSlot;
        if ((slot >= CORE3_VISION_PARAMS_SLOT_COUNT) ||
            (selector->rollbackActiveSlot >
                CORE3_VISION_PARAMS_SLOT_COUNT) ||
            (selector->valid[slot] == 0u) ||
            (selector->slotGeneration[slot] == 0u) ||
            (selectorSlotStateValid(slot, selector->rollbackBank,
                selector->rollbackValid, selector->rollbackGeneration,
                selector->rollbackTransactionId) == 0u)) return 0u;
        if (selector->rollbackActiveSlot == slot)
        {
            if (selector->rollbackValid == 0u) return 0u;
        }
        else if ((selector->rollbackActiveSlot <
            CORE3_VISION_PARAMS_SLOT_COUNT) &&
            (selector->valid[selector->rollbackActiveSlot] == 0u))
            return 0u;
        if (selector->pendingTransactionId == UINT32_MAX)
        {
            if ((selector->activeSlot != slot) ||
                (selector->rollbackBank != selector->bank[slot]) ||
                (selector->rollbackValid != selector->valid[slot]) ||
                (selector->rollbackGeneration !=
                    selector->slotGeneration[slot]) ||
                (selector->rollbackTransactionId !=
                    selector->transactionId[slot])) return 0u;
        }
        else if (((selector->activeSlot !=
            selector->rollbackActiveSlot) &&
            (selector->activeSlot != slot)) ||
            (selector->valid[slot] != CORE3_SLOT_VALID_RECORD) ||
            (selector->transactionId[slot] !=
                selector->pendingTransactionId)) return 0u;
    }
    else
    {
        uint32_t expectedGeneration =

```

```

        selector->rollbackGeneration + 1u;
uint32_t expectedBank = 0u;
if (expectedGeneration == 0u) expectedGeneration = 1u;
if (selector->rollbackValid == CORE3_SLOT_VALID_RECORD)
    expectedBank = selector->rollbackBank ^ 1u;
else if (selector->rollbackValid == CORE3_SLOT_VALID_LEGACY)
    expectedBank = 1u;
if ((selector->bank[slot] != expectedBank) ||
    (selector->slotGeneration[slot] != expectedGeneration))
    return 0u;
    }
}
else if ((selector->rollbackActiveSlot != 0u) ||
    (selector->rollbackSlot != 0u) ||
    (selector->rollbackBank != 0u) ||
    (selector->rollbackValid != 0u) ||
    (selector->rollbackGeneration != 0u) ||
    (selector->rollbackTransactionId != 0u))
{
    return 0u;
}
crc = crc32leeeCalc((const uint8_t *) selector,
    (uint32_t) offsetof(core3SlotSelector, crc));
return (uint8_t) (crc == selector->crc ? 1u : 0u);
}

static uint8_t selectorNewer(uint32_t candidate, uint32_t current)
{
    return (uint8_t) ((int32_t) (candidate - current) > 0 ? 1u : 0u);
}

static core3SelectorLoadState loadSelector(void)
{
    const uint32_t pages[2u] = {CORE3_SLOT_SELECTOR_PAGE_A, CORE3_SLOT_SELECTOR_PAGE_B};
    uint8_t found = 0u;
    uint8_t blankFound = 0u;
    uint8_t programmedInvalid = 0u;
    uint8_t committedInvalid = 0u;
    uint32_t p, record, word;
    for (p = 0u; p < 2u; p++)
    {
        for (record = 0u; record < CORE3_SELECTOR_RECORDS; record++)
        {
            core3SlotSelector candidate;
            uint32_t *words = (uint32_t *) (void *) &candidate;
            core3FlashRangeState state = blankCheckRange(pages[p],
                record * CORE3_SELECTOR_WORDS, CORE3_SELECTOR_WORDS);
            if (state == CORE3_FLASH_RANGE_IO_FAILED) return CORE3_SELECTOR_CORRUPT;
            if (state == CORE3_FLASH_RANGE_BLANK)
            {
                blankFound = 1u;
                continue;
            }
            memset(&candidate, 0xFF, sizeof(candidate));
            for (word = 0u; word < CORE3_SELECTOR_WORDS; word++)
                if (readFlashWordSafely(pages[p], record * CORE3_SELECTOR_WORDS + word,
                    &words[word]) != 0u) return CORE3_SELECTOR_CORRUPT;
            if (selectorValid(&candidate) == 0u)
            {
                programmedInvalid = 1u;
                if (candidate.magic == CORE3_SLOT_SELECTOR_MAGIC)
                    committedInvalid = 1u;
            }
            else if (!found ||
                selectorNewer(candidate.generation, g_selector.generation))
            {
                g_selector = candidate;
                g_selectorPage = (uint8_t) pages[p];
                g_selectorRecord = (uint8_t) record;
                found = 1u;
            }
        }
    }
}
if (committedInvalid != 0u && found != 0u)
{
    g_selectorReady = 1u;
    return CORE3_SELECTOR_REPAIR_LAST_GOOD;
}

```

```

    g_selectorReady = found;
    if (found) return CORE3_SELECTOR_VALID;
    if (programmedInvalid) return CORE3_SELECTOR_REBUILD_FROM_DATA;
    return blankFound ? CORE3_SELECTOR_NONE : CORE3_SELECTOR_CORRUPT;
}

static uint8_t resolveSelectorUncertainty(void)
{
    core3SelectorLoadState state;
    if (g_selectorUncertain == 0u) return 0u;
    state = loadSelector();
    if ((state != CORE3_SELECTOR_VALID) &&
        (state != CORE3_SELECTOR_NONE)) return 1u;
    if (state == CORE3_SELECTOR_NONE)
    {
        if (g_selectorUncertainHadDurable != 0u) return 1u;
        memset(&g_selector, 0, sizeof(g_selector));
        g_selectorReady = 0u;
        g_selectorPage = 0u;
        g_selectorRecord = 0u;
    }
    g_selectorUncertain = 0u;
    g_selectorUncertainHadDurable = 0u;
    return 0u;
}

static uint8_t reconcileSelectorWrite(const core3SlotSelector *expected)
{
    g_selectorUncertain = 1u;
    if (resolveSelectorUncertainty() != 0u) return 1u;
    return (uint8_t) ((g_selectorReady != 0u) &&
        (memcmp(&g_selector, expected, sizeof(*expected)) == 0) ? 0u : 1u);
}

static uint8_t writeSelector(const core3SlotSelector *candidate)
{
    core3SlotSelector stored = *candidate;
    core3SlotSelector verified;
    const uint32_t *words;
    uint32_t page = CORE3_SLOT_SELECTOR_PAGE_A;
    uint32_t record = 0u;
    uint32_t word;

    if (g_selectorUncertain != 0u)
    {
        if (resolveSelectorUncertainty() != 0u) return 1u;
        return 1u;
    }

    stored.magic = CORE3_SLOT_SELECTOR_MAGIC;
    stored.version = CORE3_SLOT_STORE_VERSION;
    stored.crc = crc32leecalc((const uint8_t *) &stored,
        (uint32_t) offsetof(core3SlotSelector, crc));
    g_selectorUncertainHadDurable = g_selectorReady;
    if (g_selectorReady)
    {
        page = g_selectorPage;
        for (record = (uint32_t) g_selectorRecord + 1u;
            record < CORE3_SELECTOR_RECORDS; record++)
            if (blankCheckRange(page, record * CORE3_SELECTOR_WORDS,
                CORE3_SELECTOR_WORDS) == CORE3_FLASH_RANGE_BLANK) break;
        if (record >= CORE3_SELECTOR_RECORDS)
        {
            page = (page == CORE3_SLOT_SELECTOR_PAGE_A) ?
                CORE3_SLOT_SELECTOR_PAGE_B : CORE3_SLOT_SELECTOR_PAGE_A;
            if (erasePage(page) != 0u) return 1u;
            record = 0u;
        }
    }
    else if (blankCheckRange(page, 0u, CORE3_SELECTOR_WORDS) != CORE3_FLASH_RANGE_BLANK &&
        erasePage(page) != 0u) return 1u;
    words = (const uint32_t *) (const void *) &stored;
    for (word = 1u; word < CORE3_SELECTOR_WORDS; word++)
        if (programFlashWord(page, record * CORE3_SELECTOR_WORDS + word,
            &words[word]) != 0u) return 1u;
    if (programFlashWord(page, record * CORE3_SELECTOR_WORDS, &words[0]) != 0u)
        return reconcileSelectorWrite(&stored);
    {
        uint32_t *verifyWords = (uint32_t *) (void *) &verified;

```



```

        for (word = 0u; word < CORE3_SELECTOR_WORDS; word++)
            if (readFlashWordSafely(page, record * CORE3_SELECTOR_WORDS + word,
                &verifyWords[word]) != 0u) return reconcileSelectorWrite(&stored);
    }
    if (!selectorValid(&verified) || memcmp(&verified, &stored, sizeof(stored)) != 0)
        return reconcileSelectorWrite(&stored);
    g_selector = stored;
    g_selectorPage = (uint8_t) page;
    g_selectorRecord = (uint8_t) record;
    g_selectorReady = 1u;
    g_selectorUncertain = 0u;
    g_selectorUncertainHadDurable = 0u;
    return 0u;
}

static uint8_t runtimeSelectorValid(const core3RuntimeSelector *selector)
{
    uint32_t crc;
    uint32_t expectedGeneration;
    if ((selector->magic != CORE3_RUNTIME_SELECTOR_MAGIC) ||
        (selector->version != CORE3_RUNTIME_SELECTOR_VERSION) ||
        (selector->recordBytes != CORE3_RUNTIME_SELECTOR_BYTES) ||
        (selector->sequence == 0u) ||
        (selector->bank > 1u) || (selector->valid != 1u) ||
        (selector->pending > 1u) ||
        (selector->sourceSlot > CORE3_VISION_PARAMS_SLOT_COUNT) ||
        (selector->rollbackBank > 1u) ||
        (selector->rollbackValid > 1u) ||
        (selector->rollbackSourceSlot > CORE3_VISION_PARAMS_SLOT_COUNT) ||
        (selector->generation == 0u) ||
        (selector->reserved0 != 0u) || (selector->reserved[0u] != 0u) ||
        (selector->reserved[1u] != 0u) || (selector->reserved[2u] != 0u))
        return 0u;
    if (selector->pending == 0u)
    {
        if ((selector->rollbackBank != 0u) ||
            (selector->rollbackValid != 0u) ||
            (selector->rollbackSourceSlot != 0u) ||
            (selector->rollbackGeneration != 0u) ||
            (selector->rollbackBlobCrc != 0u) ||
            (selector->rollbackTransactionId != 0u)) return 0u;
    }
    else
    {
        expectedGeneration = selector->rollbackGeneration + 1u;
        if (expectedGeneration == 0u) expectedGeneration = 1u;
        if ((selector->transactionId == 0u) ||
            (selector->rollbackValid != 1u) ||
            (selector->rollbackGeneration == 0u) ||
            (selector->bank == selector->rollbackBank) ||
            (selector->generation != expectedGeneration)) return 0u;
    }
    crc = crc32leeeCalc((const uint8_t *) selector,
        (uint32_t) offsetof(core3RuntimeSelector, crc));
    return (uint8_t) (crc == selector->crc ? 1u : 0u);
}

static core3SelectorLoadState loadRuntimeSelector(void)
{
    const uint32_t pages[2u] = {CORE3_RUNTIME_SELECTOR_PAGE_A,
        CORE3_RUNTIME_SELECTOR_PAGE_B};
    uint8_t found = 0u;
    uint8_t blankFound = 0u;
    uint8_t programmedInvalid = 0u;
    uint8_t committedInvalid = 0u;
    uint32_t p, record, word;
    for (p = 0u; p < 2u; p++)
    {
        for (record = 0u; record < CORE3_RUNTIME_SELECTOR_RECORDS; record++)
        {
            core3RuntimeSelector candidate;
            uint32_t *words = (uint32_t *) (void *) &candidate;
            core3FlashRangeState state = blankCheckRange(pages[p],
                record * CORE3_RUNTIME_SELECTOR_WORDS,
                CORE3_RUNTIME_SELECTOR_WORDS);
            if (state == CORE3_FLASH_RANGE_IO_FAILED) return CORE3_SELECTOR_CORRUPT;
            if (state == CORE3_FLASH_RANGE_BLANK)
            {
                blankFound = 1u;
            }

```

```

        continue;
    }
    memset(&candidate, 0xFF, sizeof(candidate));
    for (word = 0u; word < CORE3_RUNTIME_SELECTOR_WORDS; word++)
        if (readFlashWordSafely(pages[p],
            record * CORE3_RUNTIME_SELECTOR_WORDS + word,
            &words[word]) != 0u) return CORE3_SELECTOR_CORRUPT;
    if (runtimeSelectorValid(&candidate) == 0u)
    {
        programmedInvalid = 1u;
        if (candidate.magic == CORE3_RUNTIME_SELECTOR_MAGIC)
            committedInvalid = 1u;
    }
    else if (!found || selectorNewer(candidate.sequence,
        g_runtimeSelector.sequence))
    {
        g_runtimeSelector = candidate;
        g_runtimeSelectorPage = (uint8_t) pages[p];
        g_runtimeSelectorRecord = (uint8_t) record;
        found = 1u;
    }
    }
    }
    if (committedInvalid != 0u)
    {
        g_runtimeSelectorReady = 0u;
        return CORE3_SELECTOR_CORRUPT;
    }
    g_runtimeSelectorReady = found;
    if (found) return CORE3_SELECTOR_VALID;
    if (programmedInvalid) return CORE3_SELECTOR_CORRUPT;
    return blankFound ? CORE3_SELECTOR_NONE : CORE3_SELECTOR_CORRUPT;
}

static uint8_t resolveRuntimeSelectorUncertainty(void)
{
    core3SelectorLoadState state;
    if (g_runtimeSelectorUncertain == 0u) return 0u;
    state = loadRuntimeSelector();
    if (state == CORE3_SELECTOR_CORRUPT) return 1u;
    if (state == CORE3_SELECTOR_NONE)
    {
        if (g_runtimeSelectorUncertainHadDurable != 0u) return 1u;
        memset(&g_runtimeSelector, 0, sizeof(g_runtimeSelector));
        g_runtimeSelectorReady = 0u;
        g_runtimeSelectorPage = 0u;
        g_runtimeSelectorRecord = 0u;
    }
    g_runtimeSelectorUncertain = 0u;
    g_runtimeSelectorUncertainHadDurable = 0u;
    return 0u;
}

static uint8_t reconcileRuntimeSelectorWrite(
    const core3RuntimeSelector *expected)
{
    g_runtimeSelectorUncertain = 1u;
    if (resolveRuntimeSelectorUncertainty() != 0u) return 1u;
    return (uint8_t) ((g_runtimeSelectorReady != 0u) &&
        (memcmp(&g_runtimeSelector, expected, sizeof(*expected)) == 0) ? 0u : 1u);
}

static uint8_t writeRuntimeSelector(const core3RuntimeSelector *candidate)
{
    core3RuntimeSelector stored = *candidate;
    core3RuntimeSelector verified;
    const uint32_t *words;
    uint32_t page = CORE3_RUNTIME_SELECTOR_PAGE_A;
    uint32_t record = 0u;
    uint32_t word;

    if (g_runtimeSelectorUncertain != 0u)
    {
        if (resolveRuntimeSelectorUncertainty() != 0u) return 1u;
        return 1u;
    }
    stored.magic = CORE3_RUNTIME_SELECTOR_MAGIC;
    stored.version = CORE3_RUNTIME_SELECTOR_VERSION;
    stored.recordBytes = CORE3_RUNTIME_SELECTOR_BYTES;

```

```

stored.sequence = g_runtimeSelectorReady ?
    g_runtimeSelector.sequence + 1u : 1u;
if (stored.sequence == 0u) stored.sequence = 1u;
stored.crc = crc32IeeeCalc((const uint8_t *) &stored,
    (uint32_t) offsetof(core3RuntimeSelector, crc));
g_runtimeSelectorUncertainHadDurable = g_runtimeSelectorReady;
if (g_runtimeSelectorReady)
{
    page = g_runtimeSelectorPage;
    for (record = (uint32_t) g_runtimeSelectorRecord + 1u;
        record < CORE3_RUNTIME_SELECTOR_RECORDS; record++)
        if (blankCheckRange(page,
            record * CORE3_RUNTIME_SELECTOR_WORDS,
            CORE3_RUNTIME_SELECTOR_WORDS) == CORE3_FLASH_RANGE_BLANK) break;
    if (record >= CORE3_RUNTIME_SELECTOR_RECORDS)
    {
        page = (page == CORE3_RUNTIME_SELECTOR_PAGE_A) ?
            CORE3_RUNTIME_SELECTOR_PAGE_B : CORE3_RUNTIME_SELECTOR_PAGE_A;
        if (erasePage(page) != 0u) return 1u;
        record = 0u;
    }
}
else if (blankCheckRange(page, 0u, CORE3_RUNTIME_SELECTOR_WORDS) !=
    CORE3_FLASH_RANGE_BLANK && erasePage(page) != 0u) return 1u;
words = (const uint32_t *) (const void *) &stored;
for (word = 1u; word < CORE3_RUNTIME_SELECTOR_WORDS; word++)
    if (programFlashWord(page,
        record * CORE3_RUNTIME_SELECTOR_WORDS + word,
        &words[word]) != 0u) return 1u;
if (programFlashWord(page, record * CORE3_RUNTIME_SELECTOR_WORDS,
    &words[0]) != 0u) return reconcileRuntimeSelectorWrite(&stored);
{
    uint32_t *verifyWords = (uint32_t *) (void *) &verified;
    for (word = 0u; word < CORE3_RUNTIME_SELECTOR_WORDS; word++)
        if (readFlashWordSafely(page,
            record * CORE3_RUNTIME_SELECTOR_WORDS + word,
            &verifyWords[word]) != 0u)
            return reconcileRuntimeSelectorWrite(&stored);
}
if (!runtimeSelectorValid(&verified) ||
    memcmp(&verified, &stored, sizeof(stored)) != 0)
    return reconcileRuntimeSelectorWrite(&stored);
g_runtimeSelector = stored;
g_runtimeSelectorPage = (uint8_t) page;
g_runtimeSelectorRecord = (uint8_t) record;
g_runtimeSelectorReady = 1u;
g_runtimeSelectorUncertain = 0u;
g_runtimeSelectorUncertainHadDurable = 0u;
return 0u;
}

static uint32_t runtimeDataPage(uint8_t bank)
{
    return bank != 0u ? CORE3_RUNTIME_DATA_PAGE_B :
        CORE3_RUNTIME_DATA_PAGE_A;
}

static uint8_t readRuntimeBank(uint8_t bank, uint32_t blobCrc,
    core3VisionParamsStruct *params)
{
    core3VisionParamsFlashBlob blob;
    if ((bank > 1u) || (params == NULL) ||
        (readPageBytes(runtimeDataPage(bank), &blob, sizeof(blob)) != 0u) ||
        (crc32IeeeCalc(blob.bytes, CORE3_VISION_PARAMS_BLOB_BYTES) != blobCrc) ||
        (core3VisionParamsBlobDecode(&blob, params) !=
            CORE3_VISION_PARAMS_BLOB_VALID)) return 1u;
    core3VisionParamsSanitize(params);
    return 0u;
}

static uint8_t writeRuntimeBank(uint8_t bank,
    const core3VisionParamsStruct *params, uint32_t *blobCrc)
{
    core3VisionParamsFlashBlob blob;
    core3VisionParamsStruct sanitized;
    core3VisionParamsStruct verified;
    if ((bank > 1u) || (params == NULL) || (blobCrc == NULL)) return 1u;
    sanitized = *params;
    core3VisionParamsSanitize(&sanitized);

```

```

core3VisionParamsBlobEncode(&sanitized, &blob);
*blobCrc = crc32IeeeCalc(blob.bytes, CORE3_VISION_PARAMS_BLOB_BYTES);
if ((writePageAtomic(runtimeDataPage(bank), &blob, sizeof(blob)) != 0u) ||
    (readRuntimeBank(bank, *blobCrc, &verified) != 0u) ||
    (memcmp(&sanitized, &verified, sizeof(sanitized)) != 0)) return 1u;
return 0u;
}

static void clearRuntimePrepared(void)
{
    memset(&g_runtimePrepared, 0, sizeof(g_runtimePrepared));
}

static uint8_t committedRuntimeSelector(core3RuntimeSelector *committed)
{
    if ((committed == NULL) || (g_runtimeSelectorReady == 0u)) return 1u;
    *committed = g_runtimeSelector;
    if (committed->pending != 0u)
    {
        if (committed->rollbackValid == 0u) return 1u;
        committed->bank = committed->rollbackBank;
        committed->valid = committed->rollbackValid;
        committed->pending = 0u;
        committed->sourceSlot = committed->rollbackSourceSlot;
        committed->generation = committed->rollbackGeneration;
        committed->blobCrc = committed->rollbackBlobCrc;
        committed->transactionId = committed->rollbackTransactionId;
    }
    return 0u;
}

static core3VisionParamsLoadStatus readRuntimeCommitted(
    core3VisionParamsStruct *params)
{
    core3RuntimeSelector committed;
    if ((params == NULL) || (resolveRuntimeSelectorUncertainty() != 0u) ||
        (committedRuntimeSelector(&committed) != 0u) ||
        (readRuntimeBank(committed.bank, committed.blobCrc, params) != 0u))
        return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
    return CORE3_VISION_PARAMS_LOAD_VALID;
}

static core3VisionParamsSaveStatus prepareRuntimeInternal(
    const core3VisionParamsStruct *params, uint32_t transactionId,
    uint8_t sourceSlot)
{
    uint8_t bank;
    if ((params == NULL) ||
        (sourceSlot > CORE3_VISION_PARAMS_SLOT_COUNT) ||
        (resolveRuntimeSelectorUncertainty() != 0u) ||
        (g_runtimePrepared.valid != 0u) ||
        ((g_runtimeSelectorReady != 0u) &&
         (g_runtimeSelector.pending != 0u)))
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    bank = (g_runtimeSelectorReady != 0u) ?
        (uint8_t) (g_runtimeSelector.bank ^ 1u) : 0u;
    memset(&g_runtimePrepared, 0, sizeof(g_runtimePrepared));
    if (writeRuntimeBank(bank, params, &g_runtimePrepared.blobCrc) != 0u)
        return CORE3_VISION_PARAMS_SAVE_VERIFY_FAILED;
    g_runtimePrepared.transactionId = transactionId;
    g_runtimePrepared.generation = (g_runtimeSelectorReady != 0u) ?
        g_runtimeSelector.generation + 1u : 1u;
    if (g_runtimePrepared.generation == 0u)
        g_runtimePrepared.generation = 1u;
    g_runtimePrepared.sourceSlot = sourceSlot;
    g_runtimePrepared.bank = bank;
    g_runtimePrepared.valid = 1u;
    return CORE3_VISION_PARAMS_SAVE_VERIFIED;
}

static core3VisionParamsSaveStatus commitRuntimeInternal(
    uint32_t transactionId, uint8_t pending)
{
    core3RuntimeSelector next;
    if ((g_runtimePrepared.valid == 0u) ||
        (g_runtimePrepared.committed != 0u) ||
        (g_runtimePrepared.transactionId != transactionId) ||
        ((pending != 0u) && ((transactionId == 0u) ||
         (g_runtimeSelectorReady == 0u))))

```

```

        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
memset(&next, 0, sizeof(next));
if (g_runtimeSelectorReady != 0u) next = g_runtimeSelector;
if (pending != 0u)
{
    next.rollbackBank = g_runtimeSelector.bank;
    next.rollbackValid = g_runtimeSelector.valid;
    next.rollbackSourceSlot = g_runtimeSelector.sourceSlot;
    next.rollbackGeneration = g_runtimeSelector.generation;
    next.rollbackBlobCrc = g_runtimeSelector.blobCrc;
    next.rollbackTransactionId = g_runtimeSelector.transactionId;
}
else
{
    next.rollbackBank = 0u;
    next.rollbackValid = 0u;
    next.rollbackSourceSlot = 0u;
    next.rollbackGeneration = 0u;
    next.rollbackBlobCrc = 0u;
    next.rollbackTransactionId = 0u;
}
next.bank = g_runtimePrepared.bank;
next.valid = 1u;
next.pending = (uint8_t) (pending != 0u);
next.sourceSlot = g_runtimePrepared.sourceSlot;
next.generation = g_runtimePrepared.generation;
next.blobCrc = g_runtimePrepared.blobCrc;
next.transactionId = transactionId;
memset(next.reserved, 0, sizeof(next.reserved));
if (writeRuntimeSelector(&next) != 0u)
    return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
g_runtimePrepared.committed = 1u;
return CORE3_VISION_PARAMS_SAVE_VERIFIED;
}

static core3VisionParamsSaveStatus saveRuntimeFinal(
    const core3VisionParamsStruct *params, uint8_t sourceSlot)
{
    core3VisionParamsSaveStatus status = prepareRuntimeInternal(params, 0u,
        sourceSlot);
    if (status == CORE3_VISION_PARAMS_SAVE_VERIFIED)
        status = commitRuntimeInternal(0u, 0u);
    if ((status == CORE3_VISION_PARAMS_SAVE_VERIFIED) ||
        (g_runtimePrepared.committed == 0u)) clearRuntimePrepared();
    return status;
}

static core3VisionParamsSaveStatus abortRuntimeTransaction(
    uint32_t transactionId)
{
    core3RuntimeSelector next;
    if (resolveRuntimeSelectorUncertainty() != 0u)
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    if ((g_runtimeSelectorReady != 0u) &&
        (g_runtimeSelector.pending != 0u))
    {
        if ((g_runtimeSelector.transactionId != transactionId) ||
            (g_runtimeSelector.rollbackValid == 0u))
            return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
        next = g_runtimeSelector;
        next.bank = next.rollbackBank;
        next.valid = next.rollbackValid;
        next.pending = 0u;
        next.sourceSlot = next.rollbackSourceSlot;
        next.generation = next.rollbackGeneration;
        next.blobCrc = next.rollbackBlobCrc;
        next.transactionId = next.rollbackTransactionId;
        next.rollbackBank = 0u;
        next.rollbackValid = 0u;
        next.rollbackSourceSlot = 0u;
        next.rollbackGeneration = 0u;
        next.rollbackBlobCrc = 0u;
        next.rollbackTransactionId = 0u;
        memset(next.reserved, 0, sizeof(next.reserved));
        if (writeRuntimeSelector(&next) != 0u)
            return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    }
    else if ((g_runtimePrepared.valid != 0u) &&
        (g_runtimePrepared.transactionId != transactionId))

```

```

    {
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    }
    clearRuntimePrepared();
    return CORE3_VISION_PARAMS_SAVE_VERIFIED;
}

static core3VisionParamsSaveStatus finishRuntimeTransaction(
    uint32_t transactionId)
{
    core3RuntimeSelector next;
    core3VisionParamsStruct verified;
    if (resolveRuntimeSelectorUncertainty() != 0u)
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    if ((g_runtimeSelectorReady != 0u) &&
        (g_runtimeSelector.pending != 0u))
    {
        if ((g_runtimeSelector.transactionId != transactionId) ||
            (readRuntimeBank(g_runtimeSelector.bank,
                g_runtimeSelector.blobCrc, &verified) != 0u))
            return CORE3_VISION_PARAMS_SAVE_VERIFY_FAILED;
        next = g_runtimeSelector;
        next.pending = 0u;
        next.rollbackBank = 0u;
        next.rollbackValid = 0u;
        next.rollbackSourceSlot = 0u;
        next.rollbackGeneration = 0u;
        next.rollbackBlobCrc = 0u;
        next.rollbackTransactionId = 0u;
        memset(next.reserved, 0, sizeof(next.reserved));
        if (writeRuntimeSelector(&next) != 0u)
            return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    }
    else if ((g_runtimePrepared.valid != 0u) &&
        (g_runtimePrepared.transactionId != transactionId))
    {
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    }
    clearRuntimePrepared();
    return CORE3_VISION_PARAMS_SAVE_VERIFIED;
}

static core3VisionParamsSaveStatus applyCommittedRuntime(void)
{
    core3VisionParamsStruct params;
    if (readRuntimeCommitted(&params) != CORE3_VISION_PARAMS_LOAD_VALID)
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    core3VisionParamsCommit(&params);
    return CORE3_VISION_PARAMS_SAVE_VERIFIED;
}

static uint8_t recoverPendingSelector(void)
{
    core3SlotSelector rollback;
    uint32_t slot;
    if (g_selector.pendingTransactionId == 0u) return 0u;
    slot = g_selector.rollbackSlot;
    if (slot >= CORE3_VISION_PARAMS_SLOT_COUNT ||
        g_selector.rollbackActiveSlot > CORE3_VISION_PARAMS_SLOT_COUNT ||
        g_selector.rollbackBank > 1u ||
        g_selector.rollbackValid > CORE3_SLOT_VALID_LEGACY) return 1u;
    rollback = g_selector;
    rollback.generation++;
    if (rollback.generation == 0u) rollback.generation = 1u;
    rollback.activeSlot = g_selector.rollbackActiveSlot;
    rollback.bank[slot] = g_selector.rollbackBank;
    rollback.valid[slot] = g_selector.rollbackValid;
    rollback.slotGeneration[slot] = g_selector.rollbackGeneration;
    rollback.transactionId[slot] = g_selector.rollbackTransactionId;
    rollback.pendingTransactionId = 0u;
    rollback.rollbackActiveSlot = 0u;
    rollback.rollbackSlot = 0u;
    rollback.rollbackBank = 0u;
    rollback.rollbackValid = 0u;
    rollback.rollbackGeneration = 0u;
    rollback.rollbackTransactionId = 0u;
    return writeSelector(&rollback);
}

```

```

static uint32_t slotPage(uint8_t slot, uint8_t bank)
{
    return (uint32_t) slot * 2u + (uint32_t) bank;
}

static uint8_t decodeLegacyPage(core3VisionParamsStruct *params)
{
    core3VisionParamsFlashBlob blob;

    if (readPageBytes(0u, &blob, sizeof(blob)) != 0u) return 0u;
    return (uint8_t) (core3VisionParamsBlobDecode(&blob, params) ==
        CORE3_VISION_PARAMS_BLOB_VALID ? 1u : 0u);
}

static uint8_t decodeSlotRecord(uint8_t slot,
    uint8_t bank,
    core3VisionParamsStruct *params)
{
    core3SlotRecord record;
    uint32_t crc;

    if (readPageBytes(slotPage(slot, bank), &record, sizeof(record)) != 0u) return 0u;
    crc = crc32leeeCalc((const uint8_t *) &record,
        (uint32_t) offsetof(core3SlotRecord, crc));
    if ((record.magic != CORE3_SLOT_RECORD_MAGIC) ||
        (record.version != CORE3_SLOT_STORE_VERSION) ||
        (record.slot != slot) ||
        (record.payloadBytes != CORE3_VISION_PARAMS_BLOB_BYTES) ||
        (record.crc != crc) ||
        (core3VisionParamsBlobDecode(&record.payload, params) !=
            CORE3_VISION_PARAMS_BLOB_VALID)) return 0u;
    return 1u;
}

static uint8_t readRecoverableSlotRecord(uint8_t slot, uint8_t bank,
    uint8_t *valid, uint32_t *generation)
{
    core3SlotRecord record;
    core3VisionParamsStruct params;
    uint32_t crc;

    *valid = 0u;
    *generation = 0u;
    if (readPageBytes(slotPage(slot, bank), &record, sizeof(record)) != 0u)
        return 1u;
    crc = crc32leeeCalc((const uint8_t *) &record,
        (uint32_t) offsetof(core3SlotRecord, crc));
    if ((record.magic == CORE3_SLOT_RECORD_MAGIC) &&
        (record.version == CORE3_SLOT_STORE_VERSION) &&
        (record.slot == slot) && (record.generation != 0u) &&
        (record.transactionId != UINT32_MAX) &&
        (record.payloadBytes == CORE3_VISION_PARAMS_BLOB_BYTES) &&
        (record.crc == crc) &&
        (core3VisionParamsBlobDecode(&record.payload, &params) ==
            CORE3_VISION_PARAMS_BLOB_VALID))
    {
        *valid = 1u;
        *generation = record.generation;
    }
    return 0u;
}

static uint8_t readRecoverableLegacy(uint8_t *valid)
{
    core3VisionParamsFlashBlob blob;
    core3VisionParamsStruct params;

    *valid = 0u;
    if (readPageBytes(0u, &blob, sizeof(blob)) != 0u) return 1u;
    if (core3VisionParamsBlobDecode(&blob, &params) ==
        CORE3_VISION_PARAMS_BLOB_VALID) *valid = 1u;
    return 0u;
}

static uint8_t rebuildLibrarySelector(core3SelectorLoadState state)
{
    core3SlotSelector recovered;
    uint8_t slot;

```

```

if (state == CORE3_SELECTOR_REPAIR_LAST_GOOD)
{
    recovered = g_selector;
}
else if (state == CORE3_SELECTOR_REBUILD_FROM_DATA)
{
    memset(&recovered, 0, sizeof(recovered));
    recovered.generation = 1u;
    recovered.activeSlot = CORE3_VISION_PARAMS_SLOT_COUNT;
    for (slot = 0u; slot < CORE3_VISION_PARAMS_SLOT_COUNT; slot++)
    {
        uint8_t bank;
        uint8_t selectedValid = 0u;
        uint8_t selectedBank = 0u;
        uint32_t selectedGeneration = 0u;
        for (bank = 0u; bank < 2u; bank++)
        {
            uint8_t valid;
            uint32_t generation;
            if (readRecoverableSlotRecord(slot, bank, &valid,
                &generation) != 0u)
                return 1u;
            if ((valid != 0u) && ((selectedValid == 0u) ||
                (selectorNewer(generation, selectedGeneration) != 0u)))
            {
                selectedValid = 1u;
                selectedBank = bank;
                selectedGeneration = generation;
            }
        }
        if (selectedValid != 0u)
        {
            recovered.bank[slot] = selectedBank;
            recovered.valid[slot] = CORE3_SLOT_VALID_RECORD;
            recovered.slotGeneration[slot] = selectedGeneration;
            recovered.transactionId[slot] = 0u;
            if (recovered.activeSlot == CORE3_VISION_PARAMS_SLOT_COUNT)
                recovered.activeSlot = slot;
        }
    }
    if (recovered.valid[0u] == 0u)
    {
        uint8_t valid;
        if (readRecoverableLegacy(&valid) != 0u) return 1u;
        if (valid != 0u)
        {
            recovered.valid[0u] = CORE3_SLOT_VALID_LEGACY;
            recovered.slotGeneration[0u] = 1u;
            recovered.activeSlot = 0u;
        }
    }
}
else
{
    return 1u;
}

if ((erasePage(CORE3_SLOT_SELECTOR_PAGE_A) != 0u) ||
    (erasePage(CORE3_SLOT_SELECTOR_PAGE_B) != 0u)) return 1u;
memset(&g_selector, 0, sizeof(g_selector));
g_selectorReady = 0u;
g_selectorPage = 0u;
g_selectorRecord = 0u;
g_selectorUncertain = 0u;
g_selectorUncertainHadDurable = 0u;
return writeSelector(&recovered);
}

static uint8_t preparedRecordStillValid(uint8_t slot, uint8_t bank,
    uint32_t generation, uint32_t transactionId)
{
    core3SlotRecord record;
    core3VisionParamsStruct params;
    uint32_t crc;
    if (readPageBytes(slotPage(slot, bank), &record, sizeof(record)) != 0u)
        return 0u;
    crc = crc32IeeeCalc((const uint8_t *) &record,
        (uint32_t) offsetof(core3SlotRecord, crc));
    return (uint8_t) ((record.magic == CORE3_SLOT_RECORD_MAGIC) &&

```



```

        (record.version == CORE3_SLOT_STORE_VERSION) &&
        (record.slot == slot) && (record.generation == generation) &&
        (record.transactionId == transactionId) && (record.crc == crc) &&
        (record.payloadBytes == CORE3_VISION_PARAMS_BLOB_BYTES) &&
        (core3VisionParamsBlobDecode(&record.payload, &params) ==
         CORE3_VISION_PARAMS_BLOB_VALID));
    }

static core3VisionParamsSaveStatus saveSlotInternal(uint8_t slot,
const core3VisionParamsStruct *params,
uint32_t transactionId)
{
    core3SlotRecord record;
    core3SlotSelector next;
    core3VisionParamsStruct sanitized;
    uint8_t bank;

    if ((slot >= CORE3_VISION_PARAMS_SLOT_COUNT) || (params == NULL))
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    if (g_prepared.valid != 0u || g_selector.pendingTransactionId != 0u)
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    sanitized = *params;
    core3VisionParamsSanitize(&sanitized);
    if (g_selector.valid[slot] == CORE3_SLOT_VALID_RECORD)
        bank = (uint8_t) (g_selector.bank[slot] ^ 1u);
    else if (g_selector.valid[slot] == CORE3_SLOT_VALID_LEGACY)
        bank = 1u;
    else
        bank = 0u;
    memset(&record, 0, sizeof(record));
    record.magic = CORE3_SLOT_RECORD_MAGIC;
    record.version = CORE3_SLOT_STORE_VERSION;
    record.slot = slot;
    record.generation = g_selector.slotGeneration[slot] + 1u;
    if (record.generation == 0u) record.generation = 1u;
    record.transactionId = transactionId;
    record.payloadBytes = CORE3_VISION_PARAMS_BLOB_BYTES;
    core3VisionParamsBlobEncode(&sanitized, &record.payload);
    record.crc = crc32IeeeCalc((const uint8_t *) &record,
        (uint32_t) offsetof(core3SlotRecord, crc));
    if (writePageAtomic(slotPage(slot, bank), &record, sizeof(record)) != 0u)
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    if (decodeSlotRecord(slot, bank, &sanitized) == 0u)
        return CORE3_VISION_PARAMS_SAVE_VERIFY_FAILED;
    next = g_selector;
    next.generation++;
    if (next.generation == 0u) next.generation = 1u;
    next.bank[slot] = bank;
    next.valid[slot] = CORE3_SLOT_VALID_RECORD;
    next.slotGeneration[slot] = record.generation;
    next.transactionId[slot] = transactionId;
    next.activeSlot = slot;
    if (writeSelector(&next) != 0u) return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    return CORE3_VISION_PARAMS_SAVE_VERIFIED;
}

static core3VisionParamsLoadStatus initializeRuntime(
const core3VisionParamsStruct *defaults,
core3VisionParamsStruct *params)
{
    core3SelectorLoadState runtimeState = loadRuntimeSelector();
    core3VisionParamsStruct source;
    uint8_t sourceSlot = 0u;

    if (runtimeState == CORE3_SELECTOR_CORRUPT)
        return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
    if (runtimeState == CORE3_SELECTOR_VALID)
    {
        if (readRuntimeCommitted(params) == CORE3_VISION_PARAMS_LOAD_VALID)
            return CORE3_VISION_PARAMS_LOAD_VALID;
        /* A pending candidate is deliberately not repaired here: its rollback
         * must stay available for the coordinator's explicit ABORT. */
        if (g_runtimeSelector.pending != 0u)
            return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
        if ((defaults == NULL) ||
            (saveRuntimeFinal(defaults, 0u) !=
             CORE3_VISION_PARAMS_SAVE_VERIFIED))
            return CORE3_VISION_PARAMS_LOAD_REWRITE_FAILED;
        *params = *defaults;
    }
}

```

```

    core3VisionParamsSanitize(params);
    return CORE3_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN;
}

/* Upgrade from the former active-slot selector. If it had already
 * switched to a pending candidate, first materialize the rollback source
 * and then recreate the runtime candidate as pending. */
if ((g_selector.pendingTransactionId != 0u) &&
    (g_selector.activeSlot == g_selector.rollbackSlot) &&
    (g_selector.rollbackSlot < CORE3_VISION_PARAMS_SLOT_COUNT))
{
    uint8_t target = (uint8_t) g_selector.rollbackSlot;
    uint8_t oldActive = (uint8_t) g_selector.rollbackActiveSlot;
    uint32_t transactionId = g_selector.pendingTransactionId;
    uint8_t oldOk = 0u;
    if (oldActive < CORE3_VISION_PARAMS_SLOT_COUNT)
    {
        if (oldActive == target)
        {
            if (g_selector.rollbackValid == CORE3_SLOT_VALID_LEGACY)
            {
                oldOk = decodeLegacyPage(&source);
            }
            else if (g_selector.rollbackValid == CORE3_SLOT_VALID_RECORD)
            {
                oldOk = decodeSlotRecord(target,
                    (uint8_t) g_selector.rollbackBank, &source);
            }
        }
        else if (core3VisionParamsFlashReadSlot(oldActive, &source,
            NULL) == CORE3_VISION_PARAMS_LOAD_VALID)
        {
            oldOk = 1u;
        }
    }
    if ((oldOk == 0u) && (defaults != NULL))
    {
        source = *defaults;
        core3VisionParamsSanitize(&source);
        oldActive = CORE3_VISION_PARAMS_SLOT_COUNT;
        oldOk = 1u;
    }
    if ((oldOk == 0u) ||
        (saveRuntimeFinal(&source,
            oldActive < CORE3_VISION_PARAMS_SLOT_COUNT ?
                (uint8_t) (oldActive + 1u) : 0u) !=
            CORE3_VISION_PARAMS_SAVE_VERIFIED))
    {
        return CORE3_VISION_PARAMS_LOAD_REWRITE_FAILED;
    }
    *params = source;
    if (core3VisionParamsFlashReadSlot(target, &source, NULL) ==
        CORE3_VISION_PARAMS_LOAD_VALID)
    {
        if ((prepareRuntimeInternal(&source, transactionId,
            (uint8_t) (target + 1u)) !=
            CORE3_VISION_PARAMS_SAVE_VERIFIED) ||
            (commitRuntimeInternal(transactionId, 1u) !=
            CORE3_VISION_PARAMS_SAVE_VERIFIED))
        {
            return CORE3_VISION_PARAMS_LOAD_REWRITE_FAILED;
        }
    }
    return CORE3_VISION_PARAMS_LOAD_VALID;
}

if ((g_selectorReady != 0u) &&
    (g_selector.activeSlot < CORE3_VISION_PARAMS_SLOT_COUNT) &&
    (core3VisionParamsFlashReadSlot((uint8_t) g_selector.activeSlot,
        &source, NULL) == CORE3_VISION_PARAMS_LOAD_VALID))
{
    sourceSlot = (uint8_t) (g_selector.activeSlot + 1u);
}
else if (defaults != NULL)
{
    source = *defaults;
    core3VisionParamsSanitize(&source);
}
else
{
    return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
}
if (saveRuntimeFinal(&source, sourceSlot) !=
    CORE3_VISION_PARAMS_SAVE_VERIFIED)
{
    return CORE3_VISION_PARAMS_LOAD_REWRITE_FAILED;
}
*params = source;
return CORE3_VISION_PARAMS_LOAD_VALID;

```

```

}

core3VisionParamsLoadStatus core3VisionParamsFlashLoad(core3VisionParamsStruct *params)
{
    core3SelectorLoadState selectorState;
    core3FlashRangeState legacyState;
    core3VisionParamsLoadStatus libraryStatus = CORE3_VISION_PARAMS_LOAD_VALID;
    core3VisionParamsLoadStatus runtimeStatus;
    core3VisionParamsStruct defaults;
    core3VisionParamsStruct loaded;

    if (params == NULL) return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
    defaults = *params;
    core3VisionParamsSanitize(&defaults);
    ensureFlashReady();
    selectorState = loadSelector();
    if ((selectorState == CORE3_SELECTOR_REPAIR_LAST_GOOD) ||
        (selectorState == CORE3_SELECTOR_REBUILD_FROM_DATA))
    {
        if (rebuildLibrarySelector(selectorState) == 0u)
            selectorState = CORE3_SELECTOR_VALID;
        else
            selectorState = CORE3_SELECTOR_CORRUPT;
    }
    if (selectorState == CORE3_SELECTOR_CORRUPT)
    {
        /* Runtime is the sole startup source. Keep the damaged library
        * unavailable for diagnostics and writes, but still boot a verified
        * committed runtime (or rebuild runtime from defaults). */
        memset(&g_selector, 0, sizeof(g_selector));
        memset(&g_prepared, 0, sizeof(g_prepared));
        g_selectorReady = 0u;
        libraryStatus = CORE3_VISION_PARAMS_LOAD_READ_FAILED;
    }
    else if (selectorState == CORE3_SELECTOR_VALID)
    {
        if (g_selector.pendingTransactionId != 0u &&
            g_selector.pendingTransactionId != UINT32_MAX)
        {
            uint8_t slot = (uint8_t) g_selector.rollbackSlot;
            memset(&g_prepared, 0, sizeof(g_prepared));
            g_prepared.before = g_selector;
            g_prepared.before.activeSlot = g_selector.rollbackActiveSlot;
            g_prepared.before.bank[slot] = g_selector.rollbackBank;
            g_prepared.before.valid[slot] = g_selector.rollbackValid;
            g_prepared.before.slotGeneration[slot] =
                g_selector.rollbackGeneration;
            g_prepared.before.transactionId[slot] =
                g_selector.rollbackTransactionId;
            g_prepared.before.pendingTransactionId = 0u;
            g_prepared.transactionId = g_selector.pendingTransactionId;
            g_prepared.generation = g_selector.slotGeneration[slot];
            g_prepared.slot = slot;
            g_prepared.bank = (uint8_t) g_selector.bank[slot];
            g_prepared.valid = 1u;
            g_prepared.committed = 1u;
        }
    }
    else
    {
        memset(&g_selector, 0, sizeof(g_selector));
        g_selector.activeSlot = CORE3_VISION_PARAMS_SLOT_COUNT;
        legacyState = blankCheckRange(0u, 0u, CORE3_FLASH_PAGE_WORDS);
        if (legacyState == CORE3_FLASH_RANGE_IO_FAILED)
        {
            libraryStatus = CORE3_VISION_PARAMS_LOAD_READ_FAILED;
        }
        else if ((legacyState == CORE3_FLASH_RANGE_PROGRAMMED) &&
            (decodeLegacyPage(&loaded) != 0u))
        {
            g_selector.activeSlot = 0u;
            g_selector.valid[0] = CORE3_SLOT_VALID_LEGACY;
            g_selector.slotGeneration[0] = 1u;
            g_selector.generation = 1u;
            if (writeSelector(&g_selector) != 0u)
            {
                saturatingIncrement(&g_status.writeVerifyFailures);
                libraryStatus = CORE3_VISION_PARAMS_LOAD_REWRITE_FAILED;
            }
        }
    }
}

```

```

    }
    else
    {
        g_selector.generation = 1u;
        if (legacyState == CORE3_FLASH_RANGE_PROGRAMMED)
        {
            g_selector.activeSlot = 0u;
            g_selector.valid[0u] = CORE3_SLOT_VALID_LEGACY;
            g_selector.slotGeneration[0u] = 1u;
        }
        if (writeSelector(&g_selector) != 0u)
        {
            saturatingIncrement(&g_status.writeVerifyFailures);
            libraryStatus = (legacyState == CORE3_FLASH_RANGE_BLANK)
                ? CORE3_VISION_PARAMS_LOAD_BLANK_INIT_FAILED
                : CORE3_VISION_PARAMS_LOAD_REWRITE_FAILED;
        }
        else
        {
            libraryStatus = (legacyState == CORE3_FLASH_RANGE_BLANK)
                ? CORE3_VISION_PARAMS_LOAD_BLANK_INITIALIZED
                : CORE3_VISION_PARAMS_LOAD_READ_FAILED;
        }
    }
}

runtimeStatus = initializeRuntime(&defaults, params);
g_status.libraryLoad = libraryStatus;
g_status.runtimeLoad = runtimeStatus;
if (runtimeStatus == CORE3_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN)
    g_status.runtimeDefaultsRewritten = 1u;
if ((runtimeStatus != CORE3_VISION_PARAMS_LOAD_VALID) &&
    (runtimeStatus != CORE3_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN))
    g_status.lastLoad = runtimeStatus;
else if (runtimeStatus == CORE3_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN)
    g_status.lastLoad = runtimeStatus;
else if ((libraryStatus != CORE3_VISION_PARAMS_LOAD_VALID) &&
    (libraryStatus != CORE3_VISION_PARAMS_LOAD_BLANK_INITIALIZED) &&
    (libraryStatus != CORE3_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN))
    g_status.lastLoad = CORE3_VISION_PARAMS_LOAD_VALID;
else
    g_status.lastLoad = libraryStatus;
return g_status.lastLoad;
}

core3VisionParamsLoadStatus core3VisionParamsFlashReadSlot(uint8_t slot,
core3VisionParamsStruct *params,
core3VisionParamsSlotMeta *meta)
{
    uint8_t ok;

    if ((g_selectorReady == 0u) || (slot >= CORE3_VISION_PARAMS_SLOT_COUNT) ||
        (params == NULL) || (g_selector.valid[slot] == 0u))
        return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
    ok = (g_selector.valid[slot] == CORE3_SLOT_VALID_LEGACY)
        ? decodeLegacyPage(params)
        : decodeSlotRecord(slot, (uint8_t) g_selector.bank[slot], params);
    if (ok == 0u) return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
    core3VisionParamsSanitize(params);
    if (meta != NULL)
    {
        meta->generation = g_selector.slotGeneration[slot];
        meta->transactionId = g_selector.transactionId[slot];
        meta->valid = 1u;
        meta->active = 0u;
        meta->legacy = (uint8_t) (g_selector.valid[slot] == CORE3_SLOT_VALID_LEGACY ? 1u : 0u);
    }
    return CORE3_VISION_PARAMS_LOAD_VALID;
}

core3VisionParamsLoadStatus core3VisionParamsFlashInspectSlot(uint8_t slot,
core3VisionParamsLibraryMeta *meta)
{
    core3VisionParamsStruct params;
    core3VisionParamsFlashBlob blob;
    if ((meta == NULL) || (g_selectorReady == 0u) ||
        (slot >= CORE3_VISION_PARAMS_SLOT_COUNT))
        return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
    memset(meta, 0, sizeof(*meta));
    meta->bank = (uint8_t) g_selector.bank[slot];

```

```

    meta->generation = g_selector.slotGeneration[slot];
    meta->transactionId = g_selector.transactionId[slot];
    if (g_selector.valid[slot] == 0u)
    {
        meta->state = CORE3_VISION_PARAMS_LIBRARY_EMPTY;
        return CORE3_VISION_PARAMS_LOAD_VALID;
    }
    if (core3VisionParamsFlashReadSlot(slot, &params, NULL) !=
        CORE3_VISION_PARAMS_LOAD_VALID)
    {
        meta->state = CORE3_VISION_PARAMS_LIBRARY_INVALID;
        return CORE3_VISION_PARAMS_LOAD_VALID;
    }
    core3VisionParamsBlobEncode(&params, &blob);
    meta->blobCrc = crc32IeeeCalc(blob.bytes,
        CORE3_VISION_PARAMS_BLOB_BYTES);
    meta->state = CORE3_VISION_PARAMS_LIBRARY_VALID;
    return CORE3_VISION_PARAMS_LOAD_VALID;
}

core3VisionParamsLoadStatus core3VisionParamsFlashReadRuntime(
    core3VisionParamsStruct *params)
{
    ensureFlashReady();
    return readRuntimeCommitted(params);
}

core3VisionParamsLoadStatus core3VisionParamsFlashGetRuntimeMeta(
    core3VisionParamsRuntimeMeta *meta)
{
    core3RuntimeSelector committed;
    if ((meta == NULL) || (resolveRuntimeSelectorUncertainty() != 0u) ||
        (committedRuntimeSelector(&committed) != 0u))
        return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
    memset(meta, 0, sizeof(*meta));
    meta->generation = committed.generation;
    meta->blobCrc = committed.blobCrc;
    meta->transactionId = committed.transactionId;
    meta->sourceSlot = committed.sourceSlot;
    meta->valid = committed.valid;
    meta->pending = g_runtimeSelector.pending;
    meta->bank = committed.bank;
    return CORE3_VISION_PARAMS_LOAD_VALID;
}

core3VisionParamsSaveStatus core3VisionParamsFlashSaveRuntime(
    const core3VisionParamsStruct *params)
{
    ensureFlashReady();
    g_status.lastSave = saveRuntimeFinal(params, 0u);
    if (g_status.lastSave != CORE3_VISION_PARAMS_SAVE_VERIFIED)
        saturatingIncrement(&g_status.writeVerifyFailures);
    return g_status.lastSave;
}

core3VisionParamsSaveStatus core3VisionParamsFlashPrepareRuntime(
    const core3VisionParamsStruct *params, uint32_t transactionId,
    uint8_t sourceSlot)
{
    ensureFlashReady();
    if ((transactionId == 0u) || (transactionId == UINT32_MAX))
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    return prepareRuntimeInternal(params, transactionId, sourceSlot);
}

core3VisionParamsSaveStatus core3VisionParamsFlashPrepareRuntimeFromSlot(
    uint8_t slot, uint32_t transactionId)
{
    core3VisionParamsStruct params;
    if ((transactionId == 0u) || (transactionId == UINT32_MAX) ||
        (core3VisionParamsFlashReadSlot(slot, &params, NULL) !=
            CORE3_VISION_PARAMS_LOAD_VALID))
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    return prepareRuntimeInternal(&params, transactionId,
        (uint8_t) (slot + 1u));
}

core3VisionParamsSaveStatus core3VisionParamsFlashApplyPreparedRuntime(
    uint32_t transactionId)

```

```

{
    core3VisionParamsStruct params;
    if ((transactionId == 0u) ||
        (resolveRuntimeSelectorUncertainty() != 0u) ||
        (g_runtimeSelectorReady == 0u) ||
        (g_runtimeSelector.pending == 0u) ||
        (g_runtimeSelector.transactionId != transactionId) ||
        (readRuntimeBank(g_runtimeSelector.bank, g_runtimeSelector.blobCrc,
            &params) != 0u))
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    core3VisionParamsCommit(&params);
    return CORE3_VISION_PARAMS_SAVE_VERIFIED;
}

core3VisionParamsSaveStatus core3VisionParamsFlashCommitRuntime(
    uint32_t transactionId)
{
    if ((transactionId == 0u) || (transactionId == UINT32_MAX))
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    return commitRuntimeInternal(transactionId, 1u);
}

core3VisionParamsSaveStatus core3VisionParamsFlashSaveSlot(uint8_t slot,
    const core3VisionParamsStruct *params,
    uint32_t transactionId)
{
    ensureFlashReady();
    g_status.lastSave = (transactionId == UINT32_MAX) ?
        CORE3_VISION_PARAMS_SAVE_WRITE_FAILED :
        saveSlotInternal(slot, params, transactionId);
    if (g_status.lastSave != CORE3_VISION_PARAMS_SAVE_VERIFIED)
        saturatingIncrement(&g_status.writeVerifyFailures);
    return g_status.lastSave;
}

core3VisionParamsSaveStatus core3VisionParamsFlashPrepareSlot(uint8_t slot,
    const core3VisionParamsStruct *params,
    uint32_t transactionId)
{
    core3SlotRecord record;
    core3VisionParamsStruct sanitized;
    uint8_t bank;

    ensureFlashReady();
    if ((g_selectorReady == 0u) || (slot >= CORE3_VISION_PARAMS_SLOT_COUNT) ||
        (params == NULL) || (transactionId == 0u) ||
        (transactionId == UINT32_MAX) || (g_prepared.valid != 0u))
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    sanitized = *params;
    core3VisionParamsSanitize(&sanitized);
    if (g_selector.valid[slot] == CORE3_SLOT_VALID_RECORD)
        bank = (uint8_t) (g_selector.bank[slot] ^ 1u);
    else if (g_selector.valid[slot] == CORE3_SLOT_VALID_LEGACY)
        bank = 1u;
    else
        bank = 0u;
    memset(&record, 0, sizeof(record));
    record.magic = CORE3_SLOT_RECORD_MAGIC;
    record.version = CORE3_SLOT_STORE_VERSION;
    record.slot = slot;
    record.generation = g_selector.slotGeneration[slot] + 1u;
    if (record.generation == 0u) record.generation = 1u;
    record.transactionId = transactionId;
    record.payloadBytes = CORE3_VISION_PARAMS_BLOB_BYTES;
    core3VisionParamsBlobEncode(&sanitized, &record.payload);
    record.crc = crc32leecalc((const uint8_t *) &record,
        (uint32_t) offsetof(core3SlotRecord, crc));
    if ((writePageAtomic(slotPage(slot, bank), &record, sizeof(record)) != 0u) ||
        (decodeSlotRecord(slot, bank, &sanitized) == 0u))
        return CORE3_VISION_PARAMS_SAVE_VERIFY_FAILED;
    memset(&g_prepared, 0, sizeof(g_prepared));
    g_prepared.before = g_selector;
    g_prepared.transactionId = transactionId;
    g_prepared.generation = record.generation;
    g_prepared.slot = slot;
    g_prepared.bank = bank;
    g_prepared.valid = 1u;
    return CORE3_VISION_PARAMS_SAVE_VERIFIED;
}

```

```

core3VisionParamsSaveStatus core3VisionParamsFlashCommitPrepared(uint8_t slot,
uint32_t transactionId)
{
    core3SlotSelector next;
    if ((transactionId == 0u) || (transactionId == UINT32_MAX) ||
        (g_prepared.valid == 0u) || (g_prepared.committed != 0u) ||
        (g_prepared.slot != slot) || (g_prepared.transactionId != transactionId) ||
        !preparedRecordStillValid(slot, g_prepared.bank, g_prepared.generation,
            transactionId))
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    next = g_selector;
    next.generation++;
    if (next.generation == 0u) next.generation = 1u;
    next.bank[slot] = g_prepared.bank;
    next.valid[slot] = CORE3_SLOT_VALID_RECORD;
    next.slotGeneration[slot] = g_prepared.generation;
    next.transactionId[slot] = transactionId;
    next.activeSlot = slot;
    next.pendingTransactionId = transactionId;
    next.rollbackActiveSlot = g_prepared.before.activeSlot;
    next.rollbackSlot = slot;
    next.rollbackBank = g_prepared.before.bank[slot];
    next.rollbackValid = g_prepared.before.valid[slot];
    next.rollbackGeneration = g_prepared.before.slotGeneration[slot];
    next.rollbackTransactionId = g_prepared.before.transactionId[slot];
    if (writeSelector(&next) != 0u) return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    g_prepared.committed = 1u;
    return CORE3_VISION_PARAMS_SAVE_VERIFIED;
}

core3VisionParamsSaveStatus core3VisionParamsFlashAbortTransaction(
uint32_t transactionId)
{
    core3SlotSelector rollback;
    core3VisionParamsSaveStatus runtimeStatus;
    uint8_t runtimePending;
    if (transactionId == UINT32_MAX)
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    runtimePending = (uint8_t) (g_runtimeSelectorReady != 0u &&
        g_runtimeSelector.pending != 0u &&
        g_runtimeSelector.transactionId == transactionId);
    if (runtimePending != 0u &&
        applyCommittedRuntime() != CORE3_VISION_PARAMS_SAVE_VERIFIED)
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    runtimeStatus = abortRuntimeTransaction(transactionId);
    if (runtimeStatus != CORE3_VISION_PARAMS_SAVE_VERIFIED)
        return runtimeStatus;
    if (resolveSelectorUncertainty() != 0u)
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    if (g_selector.pendingTransactionId != 0u)
    {
        if (g_selector.pendingTransactionId != transactionId ||
            recoverPendingSelector() != 0u)
            return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
        memset(&g_prepared, 0, sizeof(g_prepared));
        return CORE3_VISION_PARAMS_SAVE_VERIFIED;
    }
    if ((g_prepared.valid == 0u) || (g_prepared.transactionId != transactionId))
    {
        return CORE3_VISION_PARAMS_SAVE_VERIFIED;
    }
    if (g_prepared.committed != 0u)
    {
        rollback = g_prepared.before;
        rollback.generation = g_selector.generation + 1u;
        if (rollback.generation == 0u) rollback.generation = 1u;
        if (writeSelector(&rollback) != 0u) return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    }
    memset(&g_prepared, 0, sizeof(g_prepared));
    return CORE3_VISION_PARAMS_SAVE_VERIFIED;
}

core3VisionParamsSaveStatus core3VisionParamsFlashFinishTransaction(uint32_t transactionId)
{
    core3VisionParamsSaveStatus runtimeStatus;
    uint8_t runtimePending;
    if (transactionId == UINT32_MAX)
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;

```

```

runtimePending = (uint8_t) (g_runtimeSelectorReady != 0u &&
    g_runtimeSelector.pending != 0u &&
    g_runtimeSelector.transactionId == transactionId);
if (runtimePending != 0u &&
    core3VisionParamsFlashApplyPreparedRuntime(transactionId) !=
        CORE3_VISION_PARAMS_SAVE_VERIFIED)
    return CORE3_VISION_PARAMS_SAVE_VERIFY_FAILED;
if (resolveSelectorUncertainty() != 0u)
    return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
if (g_selector.pendingTransactionId == transactionId)
{
    uint8_t slot = (uint8_t) g_selector.rollbackSlot;
    core3SlotSelector next = g_selector;
    if ((slot >= CORE3_VISION_PARAMS_SLOT_COUNT) ||
        (g_selector.valid[slot] != CORE3_SLOT_VALID_RECORD) ||
        (preparedRecordStillValid(slot, (uint8_t) g_selector.bank[slot],
            g_selector.slotGeneration[slot], transactionId) == 0u))
        return CORE3_VISION_PARAMS_SAVE_VERIFY_FAILED;
    next.generation++;
    if (next.generation == 0u) next.generation = 1u;
    next.pendingTransactionId = 0u;
    next.rollbackActiveSlot = 0u;
    next.rollbackSlot = 0u;
    next.rollbackBank = 0u;
    next.rollbackValid = 0u;
    next.rollbackGeneration = 0u;
    next.rollbackTransactionId = 0u;
    if (writeSelector(&next) != 0u) return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
}
if ((g_prepared.valid != 0u) && (g_prepared.transactionId == transactionId))
    memset(&g_prepared, 0, sizeof(g_prepared));
else if (g_selector.pendingTransactionId != 0u)
    return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
runtimeStatus = finishRuntimeTransaction(transactionId);
if (runtimeStatus != CORE3_VISION_PARAMS_SAVE_VERIFIED)
    return runtimeStatus;
return CORE3_VISION_PARAMS_SAVE_VERIFIED;
}

core3VisionParamsSaveStatus core3VisionParamsFlashSave(
    const core3VisionParamsStruct *params)
{
    return core3VisionParamsFlashSaveRuntime(params);
}

core3VisionParamsLoadStatus core3VisionParamsFlashActivateSlot(uint8_t slot,
    core3VisionParamsStruct *params)
{
    core3VisionParamsLoadStatus status = core3VisionParamsFlashReadSlot(slot, params, NULL);

    if ((status != CORE3_VISION_PARAMS_LOAD_VALID) ||
        (saveRuntimeFinal(params, (uint8_t) (slot + 1u)) !=
            CORE3_VISION_PARAMS_SAVE_VERIFIED))
        return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
    return CORE3_VISION_PARAMS_LOAD_VALID;
}

core3VisionParamsLoadStatus core3VisionParamsFlashActivatePrepared(uint8_t slot,
    uint32_t transactionId, core3VisionParamsStruct *params)
{
    core3VisionParamsLoadStatus status;
    if (params == NULL || transactionId == 0u || transactionId == UINT32_MAX ||
        g_prepared.valid == 0u || g_prepared.committed == 0u ||
        g_prepared.slot != slot || g_prepared.transactionId != transactionId ||
        g_selector.pendingTransactionId != transactionId ||
        !preparedRecordStillValid(slot, g_prepared.bank, g_prepared.generation,
            transactionId)) return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
    status = core3VisionParamsFlashReadSlot(slot, params, NULL);
    if (status != CORE3_VISION_PARAMS_LOAD_VALID) return status;
    if ((prepareRuntimeInternal(params, transactionId,
        (uint8_t) (slot + 1u)) != CORE3_VISION_PARAMS_SAVE_VERIFIED) ||
        (commitRuntimeInternal(transactionId, 1u) !=
            CORE3_VISION_PARAMS_SAVE_VERIFIED))
        return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
    return CORE3_VISION_PARAMS_LOAD_VALID;
}

core3VisionParamsLoadStatus core3VisionParamsFlashActivateSlotPending(uint8_t slot,
    core3VisionParamsStruct *params)

```



```

{
    core3VisionParamsLoadStatus status = core3VisionParamsFlashReadSlot(slot, params, NULL);
    if ((status != CORE3_VISION_PARAMS_LOAD_VALID) ||
        (prepareRuntimeInternal(params, UINT32_MAX,
            (uint8_t) (slot + 1u)) != CORE3_VISION_PARAMS_SAVE_VERIFIED) ||
        (commitRuntimeInternal(UINT32_MAX, 1u) !=
            CORE3_VISION_PARAMS_SAVE_VERIFIED))
        return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
    return CORE3_VISION_PARAMS_LOAD_VALID;
}

core3VisionParamsLoadStatus core3VisionParamsFlashAbortPendingActivation(
    core3VisionParamsStruct *params)
{
    if (params == NULL) return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
    if ((g_runtimeSelectorReady != 0u) &&
        (g_runtimeSelector.pending != 0u) &&
        (g_runtimeSelector.transactionId == UINT32_MAX))
    {
        if (readRuntimeCommitted(params) != CORE3_VISION_PARAMS_LOAD_VALID)
            return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
        core3VisionParamsCommit(params);
    }
    if (abortRuntimeTransaction(UINT32_MAX) !=
        CORE3_VISION_PARAMS_SAVE_VERIFIED)
        return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
    if ((g_selector.pendingTransactionId == UINT32_MAX) &&
        (recoverPendingSelector() != 0u))
        return CORE3_VISION_PARAMS_LOAD_READ_FAILED;
    return CORE3_VISION_PARAMS_LOAD_VALID;
}

core3VisionParamsSaveStatus core3VisionParamsFlashFinishPendingActivation(void)
{
    core3VisionParamsSaveStatus status;
    if ((g_selector.pendingTransactionId == UINT32_MAX) &&
        ((g_runtimeSelectorReady == 0u) ||
         (g_runtimeSelector.pending == 0u) ||
         (g_runtimeSelector.transactionId != UINT32_MAX)))
        return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    if ((g_runtimeSelectorReady != 0u) &&
        (g_runtimeSelector.pending != 0u) &&
        (g_runtimeSelector.transactionId == UINT32_MAX) &&
        (core3VisionParamsFlashApplyPreparedRuntime(UINT32_MAX) !=
            CORE3_VISION_PARAMS_SAVE_VERIFIED))
        return CORE3_VISION_PARAMS_SAVE_VERIFY_FAILED;
    if (g_selector.pendingTransactionId == UINT32_MAX)
    {
        core3SlotSelector next = g_selector;
        next.generation++;
        if (next.generation == 0u) next.generation = 1u;
        next.pendingTransactionId = 0u;
        next.rollbackActiveSlot = 0u;
        next.rollbackSlot = 0u;
        next.rollbackBank = 0u;
        next.rollbackValid = 0u;
        next.rollbackGeneration = 0u;
        next.rollbackTransactionId = 0u;
        if (writeSelector(&next) != 0u)
            return CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;
    }
    status = finishRuntimeTransaction(UINT32_MAX);
    return status;
}

uint8_t core3VisionParamsFlashGetActiveSlot(void)
{
    return 0u;
}

uint8_t core3VisionParamsFlashMutationPending(void)
{
    ensureFlashReady();
    return (uint8_t) ((g_selectorReady == 0u) ||
        (g_runtimeSelectorReady == 0u) ||
        (g_selectorUncertain != 0u) ||
        (g_runtimeSelectorUncertain != 0u) ||
        (g_selector.pendingTransactionId != 0u) ||
        (g_runtimeSelector.pending != 0u));
}

```

```

}

void core3VisionParamsPersistenceGetStatus(
    core3VisionParamsPersistenceStatus *status)
{
    if (status != NULL) *status = g_status;
}

```

core3_cyt4bb7/project/code/config/core3VisionParamsPersistence.h

SHA-256

:

1e3e39d655dac8769a5d1a3c6a744474df91854ea1984157d78c9cf87a194006

代码 (c)

```

#ifndef CORE3_VISION_PARAMS_PERSISTENCE_H
#define CORE3_VISION_PARAMS_PERSISTENCE_H

#include "core3VisionParamsStorage.h"

typedef struct
{
    core3VisionParamsLoadStatus lastLoad;
    core3VisionParamsSaveStatus lastSave;
    uint32_t writeVerifyFailures;
    core3VisionParamsLoadStatus libraryLoad;
    core3VisionParamsLoadStatus runtimeLoad;
    uint8_t runtimeDefaultsRewritten;
} core3VisionParamsPersistenceStatus;

#define CORE3_VISION_PARAMS_SLOT_COUNT (5u)

typedef struct
{
    uint32_t generation;
    uint32_t transactionId;
    uint8_t valid;
    uint8_t active;
    uint8_t legacy;
} core3VisionParamsSlotMeta;

typedef struct
{
    uint32_t generation;
    uint32_t blobCrc;
    uint32_t transactionId;
    uint8_t sourceSlot;
    uint8_t valid;
    uint8_t pending;
    uint8_t bank;
} core3VisionParamsRuntimeMeta;

#define CORE3_VISION_PARAMS_LIBRARY_EMPTY (0u)
#define CORE3_VISION_PARAMS_LIBRARY_VALID (1u)
#define CORE3_VISION_PARAMS_LIBRARY_INVALID (2u)
typedef struct
{
    uint8_t state;
    uint8_t bank;
    uint16_t reserved;
    uint32_t generation;
    uint32_t blobCrc;
    uint32_t transactionId;
} core3VisionParamsLibraryMeta;

core3VisionParamsLoadStatus core3VisionParamsFlashLoad(
    core3VisionParamsStruct *params);
core3VisionParamsSaveStatus core3VisionParamsFlashSave(
    const core3VisionParamsStruct *params);
core3VisionParamsLoadStatus core3VisionParamsFlashReadRuntime(
    core3VisionParamsStruct *params);
core3VisionParamsLoadStatus core3VisionParamsFlashGetRuntimeMeta(
    core3VisionParamsRuntimeMeta *meta);
core3VisionParamsSaveStatus core3VisionParamsFlashSaveRuntime(
    const core3VisionParamsStruct *params);

```

```
core3VisionParamsSaveStatus core3VisionParamsFlashPrepareRuntime(  
    const core3VisionParamsStruct *params,  
    uint32_t transactionId,  
    uint8_t sourceSlot);  
core3VisionParamsSaveStatus core3VisionParamsFlashPrepareRuntimeFromSlot(  
    uint8_t slot,  
    uint32_t transactionId);  
core3VisionParamsSaveStatus core3VisionParamsFlashApplyPreparedRuntime(  
    uint32_t transactionId);  
core3VisionParamsSaveStatus core3VisionParamsFlashCommitRuntime(  
    uint32_t transactionId);  
core3VisionParamsLoadStatus core3VisionParamsFlashReadSlot(uint8_t slot,  
    core3VisionParamsStruct *params,  
    core3VisionParamsSlotMeta *meta);  
core3VisionParamsLoadStatus core3VisionParamsFlashInspectSlot(uint8_t slot,  
    core3VisionParamsLibraryMeta *meta);  
core3VisionParamsSaveStatus core3VisionParamsFlashSaveSlot(uint8_t slot,  
    const core3VisionParamsStruct *params,  
    uint32_t transactionId);  
core3VisionParamsSaveStatus core3VisionParamsFlashPrepareSlot(uint8_t slot,  
    const core3VisionParamsStruct *params,  
    uint32_t transactionId);  
core3VisionParamsSaveStatus core3VisionParamsFlashCommitPrepared(uint8_t slot,  
    uint32_t transactionId);  
core3VisionParamsSaveStatus core3VisionParamsFlashAbortTransaction(  
    uint32_t transactionId);  
core3VisionParamsSaveStatus core3VisionParamsFlashFinishTransaction(uint32_t transactionId);  
core3VisionParamsLoadStatus core3VisionParamsFlashActivateSlot(uint8_t slot,  
    core3VisionParamsStruct *params);  
core3VisionParamsLoadStatus core3VisionParamsFlashActivatePrepared(uint8_t slot,  
    uint32_t transactionId, core3VisionParamsStruct *params);  
core3VisionParamsLoadStatus core3VisionParamsFlashActivateSlotPending(uint8_t slot,  
    core3VisionParamsStruct *params);  
core3VisionParamsLoadStatus core3VisionParamsFlashAbortPendingActivation(  
    core3VisionParamsStruct *params);  
core3VisionParamsSaveStatus core3VisionParamsFlashFinishPendingActivation(void);  
uint8_t core3VisionParamsFlashGetActiveSlot(void);  
uint8_t core3VisionParamsFlashMutationPending(void);  
void core3VisionParamsPersistenceGetStatus(  
    core3VisionParamsPersistenceStatus *status);  
  
#endif /* CORE3_VISION_PARAMS_PERSISTENCE_H */
```

core3_cyt4bb7/project/code/config/core3VisionParamsStorage.c

SHA-256 :
dd7dec6e293de6c37f15110d979741cd7402926f483ae611f33cd4e94d283748

代码 (c)

```
#include "core3VisionParamsStorage.h"  
#include <string.h>  
static uint8_t validBackend(const core3VisionParamsStorageBackend*b){return(uint8_t)(b&&b->read&&b->erase&&b->write);}  
static uint8_t blank(const uint8_t*p){uint32_t i;for(i=0;i<CORE3_VISION_PARAMS_BLOB_BYTES;i++)if(p[i]!=0xFFu)return 0u;return 1u;}  
static uint8_t match(const core3VisionParamsFlashBlob*e,const core3VisionParamsFlashBlob*a,const core3VisionParamsStruct*p){core3VisionParamsStruct  
d;return(uint8_t)  
(memcmp(e,a,CORE3_VISION_PARAMS_BLOB_BYTES)==0&&core3VisionParamsBlobDecode(a,&d)==CORE3_VISION_PARAMS_BLOB_VALID&&memc  
mp(p,&d,sizeof(d))==0);}  
core3VisionParamsSaveStatus core3VisionParamsStorageSave(const core3VisionParamsStorageBackend*b,const core3VisionParamsStruct*p)  
{core3VisionParamsFlashBlob e,a;if(!validBackend(b))!p)return  
CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;core3VisionParamsBlobEncode(p,&e);if(b->erase(b->context))return  
CORE3_VISION_PARAMS_SAVE_ERASE_FAILED;if(b->write(b->context,e.bytes,CORE3_VISION_PARAMS_BLOB_BYTES))return  
CORE3_VISION_PARAMS_SAVE_WRITE_FAILED;memset(&a,0,sizeof(a));if(b->read(b->context,a.bytes,CORE3_VISION_PARAMS_BLOB_BYTES))return  
CORE3_VISION_PARAMS_SAVE_READ_FAILED;return match(&e,&a,p)?  
CORE3_VISION_PARAMS_SAVE_VERIFIED:CORE3_VISION_PARAMS_SAVE_VERIFY_FAILED;}  
core3VisionParamsLoadStatus core3VisionParamsStorageInitializeBlank(const core3VisionParamsStorageBackend*b,const  
core3VisionParamsStruct*d,core3VisionParamsStruct*l){if(!validBackend(b))!d!lreturn  
CORE3_VISION_PARAMS_LOAD_BLANK_INIT_FAILED;*l=*d;core3VisionParamsSanitize(l);return  
core3VisionParamsStorageSave(b,l)==CORE3_VISION_PARAMS_SAVE_VERIFIED?  
CORE3_VISION_PARAMS_LOAD_BLANK_INITIALIZED:CORE3_VISION_PARAMS_LOAD_BLANK_INIT_FAILED;}  
core3VisionParamsLoadStatus core3VisionParamsStorageLoad(const core3VisionParamsStorageBackend*b,const  
core3VisionParamsStruct*d,core3VisionParamsStruct*l){core3VisionParamsFlashBlob x;core3VisionParamsBlobStatus s;if(!validBackend(b))!d!lreturn  
CORE3_VISION_PARAMS_LOAD_READ_FAILED;*l=*d;memset(&x,0,sizeof(x));if(b->read(b-  
>context,x.bytes,CORE3_VISION_PARAMS_BLOB_BYTES))return CORE3_VISION_PARAMS_LOAD_READ_FAILED;if(blank(x.bytes))return  
CORE3_VISION_PARAMS_LOAD_BLANK;s=core3VisionParamsBlobDecode(&x,l);if(s==CORE3_VISION_PARAMS_BLOB_VALID)return
```

```
CORE3_VISION_PARAMS_LOAD_VALID; *l = *d; core3VisionParamsSanitize(l); return
core3VisionParamsStorageSave(b, l) == CORE3_VISION_PARAMS_SAVE_VERIFIED?
CORE3_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN: CORE3_VISION_PARAMS_LOAD_REWRITE_FAILED; }
```

core3_cyt4bb7/project/code/config/core3VisionParamsStorage.h

SHA-256 :

b08696ff916ab5a9e93e15522d53deca74bc786a3df08048044dadcbadd57f40

代码 (c)

```
#ifndef CORE3_VISION_PARAMS_STORAGE_H
#define CORE3_VISION_PARAMS_STORAGE_H
#include <stdint.h>
#include "core3VisionParamsBlob.h"
#define CORE3_VISION_PARAMS_STORAGE_READ_BYTES CORE3_VISION_PARAMS_BLOB_BYTES
typedef uint8_t (*core3VisionParamsStorageRead)(void*, uint8_t*, uint32_t);
typedef uint8_t (*core3VisionParamsStorageEraseFn)(void*);
typedef uint8_t (*core3VisionParamsStorageWrite)(void*, const uint8_t*, uint32_t);
typedef struct { void* context; core3VisionParamsStorageRead read; core3VisionParamsStorageEraseFn erase; core3VisionParamsStorageWrite write; }
core3VisionParamsStorageBackend;
typedef
enum { CORE3_VISION_PARAMS_LOAD_BLANK = 0, CORE3_VISION_PARAMS_LOAD_VALID, CORE3_VISION_PARAMS_LOAD_READ_FAILED, CORE3_VI
SION_PARAMS_LOAD_REWRITE_FAILED, CORE3_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN, CORE3_VISION_PARAMS_LOAD_BLANK_INITI
ALIZED, CORE3_VISION_PARAMS_LOAD_BLANK_INIT_FAILED, CORE3_VISION_PARAMS_LOAD_PAGE_CONFLICT } core3VisionParamsLoadStatus;
typedef
enum { CORE3_VISION_PARAMS_SAVE_VERIFIED = 0, CORE3_VISION_PARAMS_SAVE_ERASE_FAILED, CORE3_VISION_PARAMS_SAVE_WRITE_FAILED,
CORE3_VISION_PARAMS_SAVE_READ_FAILED, CORE3_VISION_PARAMS_SAVE_VERIFY_FAILED } core3VisionParamsSaveStatus;
core3VisionParamsLoadStatus core3VisionParamsStorageLoad(const core3VisionParamsStorageBackend*, const
core3VisionParamsStruct*, core3VisionParamsStruct*);
core3VisionParamsLoadStatus core3VisionParamsStorageInitializeBlank(const core3VisionParamsStorageBackend*, const
core3VisionParamsStruct*, core3VisionParamsStruct*);
core3VisionParamsSaveStatus core3VisionParamsStorageSave(const core3VisionParamsStorageBackend*, const core3VisionParamsStruct*);
#endif
```

core3_cyt4bb7/project/code/config/pinMap.h

SHA-256 :

f0cf579212ec34d7846058e20130c37e2b67f24730c2bf60ca2d57333b0652a0

代码 (c)

```
#ifndef CORE3_PIN_MAP_H
#define CORE3_PIN_MAP_H

/* Core3 只登记本工程需要选择或约束的引脚；MT9V03X 与 IPS114 使用 CYT4 逐飞默认引脚。
* 业务 UART5 使用 P02，屏幕使用 SPI1，二者互不复用。
*/

/* Core1 业务专线：4 Mbps，Core1 P02_1 TX -> Core3 P02_0 RX。 */
#define CORE3_CORE1_UART_NUM (UART_5)
#define CORE3_CORE1_UART_RX_PIN (UART5_RX_P02_0)
#define CORE3_CORE1_UART_TX_PIN (UART5_TX_P02_1)
#define CORE3_CORE1_UART_SCB (SCB7)

/* 共阳极 RGB 灯珠，低电平点亮。 */
#define CORE3_RGB_LED_R_PWM (TCPWM_CH09_P05_0)
#define CORE3_RGB_LED_G_PWM (TCPWM_CH11_P05_2)
#define CORE3_RGB_LED_B_PWM (TCPWM_CH10_P05_1)

#endif /* CORE3_PIN_MAP_H */
```

core3_cyt4bb7/project/code/platform/core3InterruptPlatform.c

SHA-256 :

c32e0e413cf9d5d523b4d5df04e8a8034e99c49937a8f5b636c4566148850e4c

代码 (c)

```
#include "zf_common_interrupt.h"

void interrupt_global_enable(uint32 primask)
{
    __set_PRIMASK(primask);
}

uint32 interrupt_global_disable(void)
{
    uint32 primask = __get_PRIMASK();
    __disable_irq();
    return primask;
}

void assert_interrupt_config(void)
{
    __disable_irq();
}

void interrupt_set_priority(IRQn_Type irqn, uint8 priority)
{
    NVIC_SetPriority(irqn, priority);
}

void interrupt_init(cy_stc_sysint_irq_t *isrConfig,
    cy_systemIntr_Handler userIsrFunc, uint8 priority)
{
    Cy_SysInt_InitIRQ(isrConfig);
    Cy_SysInt_SetSystemIrqVector(isrConfig->sysIntSrc, userIsrFunc);
    interrupt_set_priority(isrConfig->intIdx, priority);
    NVIC_ClearPendingIRQ(isrConfig->intIdx);
    NVIC_EnableIRQ(isrConfig->intIdx);
}
```

core3_cyt4bb7/project/code/util/crc32leee.c

SHA-256

:

363e996887ef0e2c68b3518217986f209f0760bcd89ff2ca5cdd3a4801583aba

代码 (c)

```
#include "crc32leee.h"

uint32_t crc32leeeCalc(const uint8_t* data, uint32_t len)
{
    uint32_t crc = 0xFFFFFFFFu;
    uint32_t i;

    for (i = 0u; i < len; i++)
    {
        uint8_t b;
        crc ^= (uint32_t) data[i];
        for (b = 0u; b < 8u; b++)
        {
            uint32_t mask = (uint32_t) ~(int32_t) (crc & 1u);
            crc = (crc >> 1) ^ (0xEDB88320u & mask);
        }
    }
    return ~crc;
}
```

core3_cyt4bb7/project/code/util/crc32leee.h

SHA-256

:

5442cded78a9802395246ceee07532bb7e5de239f12b419b4ecc309d2c9e1001

代码 (c)

```

#ifndef CRC32_IEEE_H
#define CRC32_IEEE_H

#include <stdint.h>

uint32_t crc32IeeeCalc(const uint8_t* data, uint32_t len);

#endif

```

core3_cyt4bb7/project/code/vision/beaconTrack.c

SHA-256

:

fff71ab1956b76286bce86a0af9a96b601007153eed0a5c8c6e8f65abc764043

代码 (c)

```

#include "beaconTrack.h"

#include <string.h>

#include "commonDef.h"
#include "configBeaconTrack.h"

/*
 * 时间一致性跟踪实现。逐帧:
 * 1) 全局选取综合 score 最小且满足连续性(距离+面积趋势+面积比)的轨迹-检测对,
 * 关联则平滑更新+hits++, 否则 misses++, hits 归零。坐标速度单位 px/frame,
 * 面积速度单位 area pixels/frame。
 * 2) 删除丢失超限的轨迹(已确认容忍 DROP_MISSES, 未确认容忍 1 帧)。
 * 3) 未关联的检测生成新轨迹。
 * 4) 输出: 已确认且 misses<=COAST_MISSES 的轨迹(滑行容忍偶发漏检)。
 * 定长数组 + 无动态内存; 纯前台调用(非 ISR)。
 */

static float beaconTrackDist2(float ax, float ay, float bx, float by)
{
    float dx = ax - bx;
    float dy = ay - by;
    return dx * dx + dy * dy;
}

static float beaconTrackAbs(float v)
{
    return (v < 0.0f) ? -v : v;
}

static float beaconTrackMax(float a, float b)
{
    return (a > b) ? a : b;
}

static uint16_t beaconTrackClampCoord(float v, uint16_t maxExclusive)
{
    float maxV = (maxExclusive > 0u) ? (float) (maxExclusive - 1u) : 0.0f;
    v = clampf(v, 0.0f, maxV);
    return (uint16_t) (v + 0.5f);
}

static uint16_t beaconTrackClampSize(float v)
{
    v = clampf(v, 0.0f, (float) OM_BUFFER_SIZE);
    return (uint16_t) (v + 0.5f);
}

static uint8_t beaconTrackIdActive(const beaconTrackerStruct* tracker, uint16_t id)
{
    if (id == 0u) return 1u;
    for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
    {
        if (tracker->tracks[t].active && tracker->tracks[t].trackId == id) return 1u;
    }
    return 0u;
}

static uint16_t beaconTrackAllocId(beaconTrackerStruct* tracker)

```

```

{
    uint16_t id;
    for (uint16_t tries = 0u; tries < 0xFFFFu; tries++)
    {
        id = tracker->nextTrackId;
        tracker->nextTrackId++;
        if (tracker->nextTrackId == 0u) tracker->nextTrackId = 1u;
        if (!beaconTrackIdActive(tracker, id)) return id;
    }
    return 0u;
}

static float beaconTrackPredictFramesForOutput(const beaconTrackStruct* tr)
{
    return (tr->misses > 0u) ? (float) tr->misses : VISION_TRACK_PREDICT_FRAMES_DEFAULT;
}

static float beaconTrackPredX(const beaconTrackStruct* tr, float frames)
{
    return tr->lastRawX + tr->velX * frames;
}

static float beaconTrackPredY(const beaconTrackStruct* tr, float frames)
{
    return tr->lastRawY + tr->velY * frames;
}

static float beaconTrackPredSize(const beaconTrackStruct* tr, float frames)
{
    return clampf(tr->lastRawSize + tr->areaVel * frames, 0.0f, (float) OM_BUFFER_SIZE);
}

static float beaconTrackAreaVelDenom(float size)
{
    float areaBase = beaconTrackMax(size, VISION_TRACK_AREA_NORM_MIN);
    return beaconTrackMax(areaBase * VISION_TRACK_ASSOC_AREA_VEL_NORM,
        VISION_TRACK_ASSOC_AREA_VEL_MIN_ABS);
}

static float beaconTrackAssociationScore(float d2, float maxDist2, float detSize,
    float predSize, float rawAreaVel, float areaVelBaseSize,
    float expectedAreaVel, uint8_t areaVelPrimed,
    float areaWeight, float areaVelWeight)
{
    float predDenom = beaconTrackMax(predSize, VISION_TRACK_AREA_NORM_MIN);
    float areaErr = beaconTrackAbs(detSize - predSize) / predDenom;
    float areaScore;
    float velScore = 0.0f;

    areaErr = clampf(areaErr, 0.0f, VISION_TRACK_ASSOC_AREA_ERR_CAP);
    areaScore = areaErr * areaErr;

    if (areaVelPrimed)
    {
        float velErr = beaconTrackAbs(rawAreaVel - expectedAreaVel) / beaconTrackAreaVelDenom(areaVelBaseSize);
        velErr = clampf(velErr, 0.0f, VISION_TRACK_ASSOC_AREA_VEL_ERR_CAP);
        velScore = velErr * velErr;
    }

    return (d2 / maxDist2)
        + areaWeight * areaScore
        + areaVelWeight * velScore;
}

static uint8_t beaconTrackPairBetter(float score, float d2, uint16_t trackId, uint16_t detIdx,
    float bestScore, float bestD2,
    uint16_t bestTrackId, uint16_t bestDetIdx)
{
    const float eps = 0.0001f;
    if (score < (bestScore - eps)) return 1u;
    if (score > (bestScore + eps)) return 0u;
    if (d2 < (bestD2 - eps)) return 1u;
    if (d2 > (bestD2 + eps)) return 0u;
    if (trackId < bestTrackId) return 1u;
    if (trackId > bestTrackId) return 0u;
    return (detIdx < bestDetIdx) ? 1u : 0u;
}

```

/* 运行时跟踪参数(默认取 configBeaconTrack.h 的 #define, 菜单可热重载)。

```

* 仅 Core3 前台读写, 无 ISR 竞争, 无需 volatile。 */
typedef struct
{
    float maxAssocDist;
    float sizeRatioMax;
    uint16_t confirmHits;
    uint16_t dropMisses;
    uint16_t coastMisses;
    float positionAlpha; /* 关联更新位置低通系数(0<alpha<=1) */
    float sizeAlpha; /* 关联更新面积低通系数(0<alpha<=1) */
    float areaVelAlpha; /* 面积速度低通系数(0<alpha<=1) */
    float assocScoreMax;
    float assocAreaWeight;
    float assocAreaVelWeight;
    beaconTrackBorderConfigStruct border;
    uint16_t edgePredictionHoldFrames;
    beaconTrackEdgeBandsStruct edgeBands;
} beaconTrackConfigStruct;

#define BEACON_EDGE_SIDE_LEFT (0x01u)
#define BEACON_EDGE_SIDE_RIGHT (0x02u)
#define BEACON_EDGE_SIDE_TOP (0x04u)
#define BEACON_EDGE_SIDE_BOTTOM (0x08u)

static beaconTrackConfigStruct beaconTrackConfig = {
    VISION_TRACK_MAX_ASSOC_DIST_DEFAULT,
    VISION_TRACK_SIZE_RATIO_MAX_DEFAULT,
    VISION_TRACK_CONFIRM_HITS_DEFAULT,
    VISION_TRACK_DROP_MISSES_DEFAULT,
    VISION_TRACK_COAST_MISSES_DEFAULT,
    VISION_TRACK_POSITION_ALPHA_DEFAULT,
    VISION_TRACK_SIZE_ALPHA_DEFAULT,
    VISION_TRACK_AREA_VEL_ALPHA_DEFAULT,
    VISION_TRACK_ASSOC_SCORE_MAX_DEFAULT,
    VISION_TRACK_ASSOC_AREA_WEIGHT_DEFAULT,
    VISION_TRACK_ASSOC_AREA_VEL_WEIGHT_DEFAULT,
    {
        {VISION_TRACK_BORDER_ENTER_LEFT_DEFAULT,
         VISION_TRACK_BORDER_ENTER_RIGHT_DEFAULT,
         VISION_TRACK_BORDER_ENTER_TOP_DEFAULT,
         VISION_TRACK_BORDER_ENTER_BOTTOM_DEFAULT},
        {VISION_TRACK_BORDER_EXIT_LEFT_DEFAULT,
         VISION_TRACK_BORDER_EXIT_RIGHT_DEFAULT,
         VISION_TRACK_BORDER_EXIT_TOP_DEFAULT,
         VISION_TRACK_BORDER_EXIT_BOTTOM_DEFAULT}
    },
    0u,
    {VISION_TRACK_EDGE_BAND_C3_LEFT_DEFAULT,
     VISION_TRACK_EDGE_BAND_C3_RIGHT_DEFAULT,
     VISION_TRACK_EDGE_BAND_C3_TOP_DEFAULT,
     VISION_TRACK_EDGE_BAND_C3_BOTTOM_DEFAULT}
};

static float beaconTrackPredictFramesForAssoc(const beaconTrackStruct* tr)
{
    float frames = (float) tr->misses + VISION_TRACK_PREDICT_FRAMES_DEFAULT;
    if ((tr->edgePredictionSideMask != 0u) &&
        (beaconTrackConfig.edgePredictionHoldFrames != 0u))
    {
        frames = clampf(frames, VISION_TRACK_PREDICT_FRAMES_DEFAULT,
                        VISION_TRACK_EDGE_ASSOC_PREDICT_FRAMES_MAX);
    }
    return frames;
}

static uint8_t beaconTrackEdgeSideMask(float x, float y)
{
    uint8_t mask = 0u;
    if (beaconTrackConfig.edgePredictionHoldFrames == 0u) return 0u;
    if ((beaconTrackConfig.edgeBands.left != 0u) &&
        (x < (float) beaconTrackConfig.edgeBands.left)) mask |= BEACON_EDGE_SIDE_LEFT;
    if ((beaconTrackConfig.edgeBands.right != 0u) &&
        (x >= (float) OM_IMAGE_MAX_WIDTH - (float) beaconTrackConfig.edgeBands.right))
        mask |= BEACON_EDGE_SIDE_RIGHT;
    if ((beaconTrackConfig.edgeBands.top != 0u) &&
        (y < (float) beaconTrackConfig.edgeBands.top)) mask |= BEACON_EDGE_SIDE_TOP;
    if ((beaconTrackConfig.edgeBands.bottom != 0u) &&
        (y >= (float) OM_IMAGE_MAX_HEIGHT - (float) beaconTrackConfig.edgeBands.bottom))
        mask |= BEACON_EDGE_SIDE_BOTTOM;
}

```



```

    return mask;
}

static uint8_t beaconTrackSideValid(uint8_t enter, uint8_t exit, uint8_t maximum)
{
    if ((enter == 0u) && (exit == 0u)) return 1u;
    return (uint8_t) ((enter >= 1u) && (enter <= maximum) && (exit < enter));
}

static uint8_t beaconTrackBorderValid(const beaconTrackBorderConfigStruct* margins)
{
    if (margins == 0) return 0u;
    return (uint8_t) (
        beaconTrackSideValid(margins->enter.left, margins->exit.left, VISION_TRACK_BORDER_X_MAX) &&
        beaconTrackSideValid(margins->enter.right, margins->exit.right, VISION_TRACK_BORDER_X_MAX) &&
        beaconTrackSideValid(margins->enter.top, margins->exit.top, VISION_TRACK_BORDER_Y_MAX) &&
        beaconTrackSideValid(margins->enter.bottom, margins->exit.bottom, VISION_TRACK_BORDER_Y_MAX));
}

static uint8_t beaconTrackPointInside(float x, float y,
    const beaconTrackBorderMarginsStruct* margin)
{
    return (uint8_t) (((margin->left == 0u) || (x >= (float) margin->left)) &&
        ((margin->right == 0u) || (x <= (float) (OM_IMAGE_MAX_WIDTH - 1u - margin->right))) &&
        ((margin->top == 0u) || (y >= (float) margin->top)) &&
        ((margin->bottom == 0u) || (y <= (float) (OM_IMAGE_MAX_HEIGHT - 1u - margin->bottom))));
}

static uint8_t beaconTrackUpdateBorderExit(beaconTrackStruct* tr, float x, float y)
{
    if (beaconTrackPointInside(x, y, &beaconTrackConfig.border.exit) != 0u)
    {
        tr->borderOutsideFrames = 0u;
        return 0u;
    }

    if (tr->borderOutsideFrames < 0xFFu) tr->borderOutsideFrames++;
    if (tr->borderOutsideFrames >= VISION_TRACK_BORDER_EXIT_HOLD_FRAMES_DEFAULT)
    {
        tr->borderEligible = 0u;
        return 1u;
    }
    return 0u;
}

void beaconTrackSetBorderMargins(const beaconTrackBorderConfigStruct* margins)
{
    if (beaconTrackBorderValid(margins) != 0u)
    {
        beaconTrackConfig.border = *margins;
    }
}

void beaconTrackSetEdgePredictionConfig(
    const beaconTrackEdgePredictionConfigStruct* config)
{
    uint16_t holdMs;
    if (config == 0) return;
    holdMs = config->holdMs;
    if (holdMs > VISION_TRACK_EDGE_PREDICTION_HOLD_MS_MAX)
        holdMs = VISION_TRACK_EDGE_PREDICTION_HOLD_MS_MAX;
    beaconTrackConfig.edgePredictionHoldFrames = (uint16_t)
        ((holdMs + VISION_TRACK_CAMERA_FRAME_PERIOD_MS - 1u) /
            VISION_TRACK_CAMERA_FRAME_PERIOD_MS);
    beaconTrackConfig.edgeBands = config->bands;
    if (beaconTrackConfig.edgeBands.left > (uint8_t) VISION_TRACK_EDGE_BAND_X_MAX)
        beaconTrackConfig.edgeBands.left = (uint8_t) VISION_TRACK_EDGE_BAND_X_MAX;
    if (beaconTrackConfig.edgeBands.right > (uint8_t) VISION_TRACK_EDGE_BAND_X_MAX)
        beaconTrackConfig.edgeBands.right = (uint8_t) VISION_TRACK_EDGE_BAND_X_MAX;
    if (beaconTrackConfig.edgeBands.top > (uint8_t) VISION_TRACK_EDGE_BAND_Y_MAX)
        beaconTrackConfig.edgeBands.top = (uint8_t) VISION_TRACK_EDGE_BAND_Y_MAX;
    if (beaconTrackConfig.edgeBands.bottom > (uint8_t) VISION_TRACK_EDGE_BAND_Y_MAX)
        beaconTrackConfig.edgeBands.bottom = (uint8_t) VISION_TRACK_EDGE_BAND_Y_MAX;
}

void beaconTrackSetParams(float maxAssocDist, float sizeRatioMax,
    uint16_t confirmHits, uint16_t dropMisses, uint16_t coastMisses,
    float positionAlpha, float sizeAlpha,
    float areaVelAlpha, float assocScoreMax,

```

```

        float assocAreaWeight, float assocAreaVelWeight)
{
    if (maxAssocDist > 0.0f) beaconTrackConfig.maxAssocDist = maxAssocDist;
    if (sizeRatioMax > 1.0f) beaconTrackConfig.sizeRatioMax = sizeRatioMax;
    if (confirmHits > 0u) beaconTrackConfig.confirmHits = confirmHits;
    if (dropMisses > 0u) beaconTrackConfig.dropMisses = dropMisses;
    if (coastMisses > 0u) beaconTrackConfig.coastMisses = coastMisses;
    if (positionAlpha > 0.0f && positionAlpha <= 1.0f) beaconTrackConfig.positionAlpha = positionAlpha;
    if (sizeAlpha > 0.0f && sizeAlpha <= 1.0f) beaconTrackConfig.sizeAlpha = sizeAlpha;
    if (areaVelAlpha > 0.0f && areaVelAlpha <= 1.0f) beaconTrackConfig.areaVelAlpha = areaVelAlpha;
    if (assocScoreMax > 0.0f) beaconTrackConfig.assocScoreMax = assocScoreMax;
    if (assocAreaWeight >= 0.0f) beaconTrackConfig.assocAreaWeight = assocAreaWeight;
    if (assocAreaVelWeight >= 0.0f) beaconTrackConfig.assocAreaVelWeight = assocAreaVelWeight;
}

void beaconTrackInit(beaconTrackerStruct* tracker)
{
    if (0 == tracker) return;
    for (uint16_t i = 0u; i < OM_MAX_OBJECTS; i++)
    {
        tracker->tracks[i].active = 0u;
    }
    tracker->confirmedCount = 0u;
    tracker->nextTrackId = 1u;
    tracker->frameSeq = 0u;
}

void beaconTrackMarkPublished(beaconTrackerStruct* tracker, uint16_t trackId)
{
    uint16_t t;
    if ((tracker == 0) || (trackId == 0u)) return;
    for (t = 0u; t < OM_MAX_OBJECTS; t++)
    {
        beaconTrackStruct *track = &tracker->tracks[t];
        if ((track->active != 0u) && (track->trackId == trackId))
        {
            track->published = 1u;
            return;
        }
    }
}

void beaconTrackUpdate(beaconTrackerStruct* tracker, const BEACON_DETECTOR_STRUCT* det)
{
    if (0 == tracker || 0 == det) return;

    uint16_t nDet = det->beaconCount;
    uint16_t outCount = 0u;
    if (nDet > OM_MAX_OBJECTS) nDet = OM_MAX_OBJECTS;

    uint8_t detUsed[OM_MAX_OBJECTS];
    uint8_t trackUsed[OM_MAX_OBJECTS];
    for (uint16_t i = 0u; i < nDet; i++) detUsed[i] = 0u;
    for (uint16_t i = 0u; i < OM_MAX_OBJECTS; i++) trackUsed[i] = 0u;
    tracker->frameSeq++;

    const float maxDist2 = beaconTrackConfig.maxAssocDist * beaconTrackConfig.maxAssocDist;
    const float ratioMax = beaconTrackConfig.sizeRatioMax;
    const float ratioMin = 1.0f / beaconTrackConfig.sizeRatioMax;
    const float velMax = beaconTrackConfig.maxAssocDist * VISION_TRACK_VELOCITY_MAX_FACTOR_DEFAULT;

    /* EdgeBand 热重载必须先于本帧关联生效，禁止缩小/关闭后再借旧资格
    * 多使用一次纵向放宽门。扩大参数只影响仍存活轨迹。 */
    for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
    {
        beaconTrackStruct* tr = &tracker->tracks[t];
        if (!tr->active) continue;
        tr->edgePredictionSideMask = beaconTrackEdgeSideMask(
            tr->lastRawX, tr->lastRawY);
    }

    /* 1) 全局选择预测残差最小的兼容轨迹-检测对，避免槽位顺序先抢检测。 */
    for (;;)
    {
        int32_t bestTrack = -1;
        int32_t bestDet = -1;
        uint16_t bestTrackId = 0xFFFFu;
        uint16_t bestDetIdx = 0xFFFFu;
        float bestScore = beaconTrackConfig.assocScoreMax;

```

```

float bestD2 = 1.0f;

for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
{
    beaconTrackStruct* tr = &tracker->tracks[t];
    if (!tr->active || trackUsed[t]) continue;

    float frames = beaconTrackPredictFramesForAssoc(tr);
    float predX = beaconTrackPredX(tr, frames);
    float predY = beaconTrackPredY(tr, frames);
    float predSize = beaconTrackPredSize(tr, frames);
    float measurementFrames = (float) tr->misses + 1.0f;
    float predSizeDenom = beaconTrackMax(predSize, VISION_TRACK_AREA_NORM_MIN);
    for (uint16_t d = 0u; d < nDet; d++)
    {
        if (detUsed[d]) continue;
        float dcx = (float) det->beaconData[d].centerX;
        float dcy = (float) det->beaconData[d].centerY;
        float detSize = (float) det->beaconData[d].contourSize;
        float rawAreaVel = (detSize - tr->lastRawSize) / measurementFrames;
        float d2 = beaconTrackDist2(dcx, dcy, predX, predY);
        float positionScore;
        float ratio;
        float score;
        if ((tr->edgePredictionSideMask != 0u) &&
            (beaconTrackConfig.edgePredictionHoldFrames != 0u))
        {
            float dx = dcx - predX;
            float dy = dcy - predY;
            float rx = beaconTrackConfig.maxAssocDist;
            float ry = beaconTrackConfig.maxAssocDist;
            if ((tr->edgePredictionSideMask &
                (BEACON_EDGE_SIDE_LEFT | BEACON_EDGE_SIDE_RIGHT)) != 0u)
                rx *= VISION_TRACK_EDGE_VERTICAL_ASSOC_SCALE;
            if ((tr->edgePredictionSideMask &
                (BEACON_EDGE_SIDE_TOP | BEACON_EDGE_SIDE_BOTTOM)) != 0u)
                ry *= VISION_TRACK_EDGE_VERTICAL_ASSOC_SCALE;
            positionScore = (dx * dx) / (rx * rx) + (dy * dy) / (ry * ry);
            if (positionScore > 1.0f) continue;
        }
        else
        {
            if (d2 > maxDist2) continue;
            positionScore = d2 / maxDist2;
        }
        ratio = detSize / predSizeDenom;
        if (ratio > ratioMax || ratio < ratioMin) continue;
        score = beaconTrackAssociationScore(positionScore, 1.0f, detSize, predSize, rawAreaVel,
            tr->lastRawSize,
            tr->areaVel, tr->areaVelocityPrimed,
            beaconTrackConfig.assocAreaWeight,
            beaconTrackConfig.assocAreaVelWeight);
        if (score > beaconTrackConfig.assocScoreMax) continue;
        if (bestTrack < 0
            || beaconTrackPairBetter(score, positionScore, tr->trackId, d,
                bestScore, bestD2, bestTrackId, bestDetIdx))
        {
            bestScore = score;
            bestD2 = positionScore;
            bestTrack = (int32_t) t;
            bestDet = (int32_t) d;
            bestTrackId = tr->trackId;
            bestDetIdx = d;
        }
    }
}

if (bestTrack < 0 || bestDet < 0) break;

{
    uint16_t t = (uint16_t) bestTrack;
    uint16_t d = (uint16_t) bestDet;
    beaconTrackStruct* tr = &tracker->tracks[t];
    float aP = beaconTrackConfig.positionAlpha;
    float aS = beaconTrackConfig.sizeAlpha;
    float dcx = (float) det->beaconData[d].centerX;
    float dcy = (float) det->beaconData[d].centerY;
    float detSize = (float) det->beaconData[d].contourSize;
    float measurementFrames = (float) tr->misses + 1.0f;

```

```

float rawVx = (dcx - tr->lastRawX) / measurementFrames;
float rawVy = (dcy - tr->lastRawY) / measurementFrames;
float rawAreaVel = (detSize - tr->lastRawSize) / measurementFrames;
float areaVelMax = beaconTrackMax(tr->lastRawSize * VISION_TRACK_AREA_VEL_MAX_RATE,
    VISION_TRACK_ASSOC_AREA_VEL_MIN_ABS);
rawVx = clampf(rawVx, -velMax, velMax);
rawVy = clampf(rawVy, -velMax, velMax);
rawAreaVel = clampf(rawAreaVel, -areaVelMax, areaVelMax);
if (tr->velocityPrimed)
{
    float aV = VISION_TRACK_VELOCITY_ALPHA_DEFAULT;
    tr->velX = aV * rawVx + (1.0f - aV) * tr->velX;
    tr->velY = aV * rawVy + (1.0f - aV) * tr->velY;
}
else
{
    tr->velX = rawVx;
    tr->velY = rawVy;
    tr->velocityPrimed = 1u;
}
if (tr->areaVelocityPrimed)
{
    float aA = beaconTrackConfig.areaVelAlpha;
    tr->areaVel = aA * rawAreaVel + (1.0f - aA) * tr->areaVel;
}
else
{
    tr->areaVel = rawAreaVel;
    tr->areaVelocityPrimed = 1u;
}
tr->areaVel = clampf(tr->areaVel, -areaVelMax, areaVelMax);
tr->velX = clampf(tr->velX, -velMax, velMax);
tr->velY = clampf(tr->velY, -velMax, velMax);
tr->lastRawX = dcx;
tr->lastRawY = dcy;
tr->lastRawSize = detSize;
trackUsed[t] = 1u;
detUsed[d] = 1u;
/* 关联更新一阶低通(alpha*measured + (1-alpha)*old; 越小越平滑, 替代原硬编码 0.5)。 */
tr->cx = aP * dcx + (1.0f - aP) * tr->cx;
tr->cy = aP * dcy + (1.0f - aP) * tr->cy;
tr->size = aS * detSize + (1.0f - aS) * tr->size;
if (tr->hits < 0xFFFFu) tr->hits++;
tr->misses = 0u;
tr->fresh = 1u;
tr->edgePredictionSideMask = beaconTrackEdgeSideMask(dcx, dcy);
if (tr->borderEligible == 0u)
{
    tr->borderEligible = beaconTrackPointInside(dcx, dcy,
        &beaconTrackConfig.border.enter);
    if (tr->borderEligible != 0u) tr->borderOutsideFrames = 0u;
}
tr->frameSeq = tracker->frameSeq;
if (tr->hits >= beaconTrackConfig.confirmHits) tr->confirmed = 1u;
}
}

for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
{
    beaconTrackStruct* tr = &tracker->tracks[t];
    if (!tr->active || trackUsed[t]) continue;
    if (tr->misses < 0xFFFFu) tr->misses++;
    tr->hits = 0u;
    tr->fresh = 0u;
    tr->frameSeq = tracker->frameSeq;
}

/* 2) 边界拒绝优先于丢失删除和边缘预测。所有从 Enter 区取得资格的
* 活动轨迹都参与 Exit 确认；只有已经确认发布过的轨迹产生一次拒绝事实。 */
for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
{
    beaconTrackStruct* tr = &tracker->tracks[t];
    float predFrames;
    float predX;
    float predY;
    float qualificationX;
    float qualificationY;
    if (!tr->active || (tr->borderEligible == 0u)) continue;
    predFrames = beaconTrackPredictFramesForOutput(tr);

```

```

predX = beaconTrackPredX(tr, predFrames);
predY = beaconTrackPredY(tr, predFrames);
qualificationX = (tr->fresh != 0u) ? tr->lastRawX : predX;
qualificationY = (tr->fresh != 0u) ? tr->lastRawY : predY;
if (beaconTrackUpdateBorderExit(tr, qualificationX,
    qualificationY) == 0u) continue;
if (tr->published != 0u)
{
    memset(&tracker->confirmed[outCount], 0,
        sizeof(tracker->confirmed[outCount]));
    tracker->confirmed[outCount].trackId = tr->trackId;
    tracker->confirmed[outCount].flags =
        BEACON_CONFIRMED_FLAG_BORDER_REJECTED;
    outCount++;
}
tr->edgePredictionSideMask = 0u;
tr->active = 0u;
tr->confirmed = 0u;
tr->fresh = 0u;
}

/* 3) 删除丢失超限轨迹 */
for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
{
    beaconTrackStruct* tr = &tracker->tracks[t];
    if (!tr->active) continue;
    uint16_t limit = tr->confirmed ? beaconTrackConfig.dropMisses : 1u;
    if (tr->confirmed && (tr->edgePredictionSideMask != 0u) &&
        (beaconTrackConfig.edgePredictionHoldFrames != 0u))
    {
        limit = beaconTrackConfig.edgePredictionHoldFrames;
    }
    if (tr->misses > limit)
    {
        tr->active = 0u;
        tr->fresh = 0u;
    }
}

/* 4) 未关联检测生成新轨迹(填首个空槽) */
for (uint16_t d = 0u; d < nDet; d++)
{
    if (detUsed[d]) continue;
    if (beaconTrackPointInside((float) det->beaconData[d].centerX,
        (float) det->beaconData[d].centerY,
        &beaconTrackConfig.border.enter) == 0u) continue;
    for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
    {
        beaconTrackStruct* tr = &tracker->tracks[t];
        uint16_t newTrackId;
        if (tr->active) continue;
        newTrackId = beaconTrackAllocId(tracker);
        if (newTrackId == 0u) break;
        tr->cx = (float) det->beaconData[d].centerX;
        tr->cy = (float) det->beaconData[d].centerY;
        tr->size = (float) det->beaconData[d].contourSize;
        tr->lastRawX = tr->cx;
        tr->lastRawY = tr->cy;
        tr->lastRawSize = tr->size;
        tr->velX = 0.0f;
        tr->velY = 0.0f;
        tr->areaVel = 0.0f;
        tr->frameSeq = tracker->frameSeq;
        tr->trackId = newTrackId;
        tr->hits = 1u;
        tr->misses = 0u;
        tr->confirmed = 0u;
        tr->fresh = 1u;
        tr->velocityPrimed = 0u;
        tr->areaVelocityPrimed = 0u;
        tr->borderEligible = beaconTrackPointInside(tr->cx, tr->cy,
            &beaconTrackConfig.border.enter);
        tr->borderOutsideFrames = 0u;
        tr->edgePredictionSideMask = beaconTrackEdgeSideMask(tr->cx, tr->cy);
        tr->published = 0u;
        tr->active = 1u;
        break;
    }
}
}

```

```

/* 5) 输出已确认且仍新鲜(滑行容忍)的轨迹。拒绝记录已在前面。 */
for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
{
    beaconTrackStruct* tr = &tracker->tracks[t];
    if (!tr->active || !tr->confirmed) continue;
    {
        uint16_t outputLimit = beaconTrackConfig.coastMisses;
        if ((tr->edgePredictionSideMask != 0u) &&
            (beaconTrackConfig.edgePredictionHoldFrames != 0u))
        {
            outputLimit = beaconTrackConfig.edgePredictionHoldFrames;
        }
        if (tr->misses > outputLimit) continue;
    }
    float predFrames = beaconTrackPredictFramesForOutput(tr);
    float predX = beaconTrackPredX(tr, predFrames);
    float predY = beaconTrackPredY(tr, predFrames);
    float predSize = beaconTrackPredSize(tr, predFrames);
    if (tr->borderEligible == 0u) continue;
    tracker->confirmed[outCount].centerX = (uint16_t) (tr->cx + 0.5f);
    tracker->confirmed[outCount].centerY = (uint16_t) (tr->cy + 0.5f);
    tracker->confirmed[outCount].size = beaconTrackClampSize(tr->size);
    tracker->confirmed[outCount].trackId = tr->trackId;
    tracker->confirmed[outCount].predX = beaconTrackClampCoord(predX, OM_IMAGE_MAX_WIDTH);
    tracker->confirmed[outCount].predY = beaconTrackClampCoord(predY, OM_IMAGE_MAX_HEIGHT);
    tracker->confirmed[outCount].predSize = beaconTrackClampSize(predSize);
    tracker->confirmed[outCount].misses = tr->misses;
    tracker->confirmed[outCount].fresh = tr->fresh;
    tracker->confirmed[outCount].flags =
        (tr->fresh ? BEACON_CONFIRMED_FLAG_FRESH : 0u) |
        (tr->misses > 0u ? BEACON_CONFIRMED_FLAG_COAST : 0u) |
        ((!tr->fresh && (tr->edgePredictionSideMask != 0u) &&
            (beaconTrackConfig.edgePredictionHoldFrames != 0u))
            ? BEACON_CONFIRMED_FLAG_EDGE_GRACE : 0u);
    tracker->confirmed[outCount].frameSeq = tr->frameSeq;
    tracker->confirmed[outCount].velX = tr->velX;
    tracker->confirmed[outCount].velY = tr->velY;
    tracker->confirmed[outCount].areaVel = tr->areaVel;
    outCount++;
}
tracker->confirmedCount = outCount;
}

```

core3_cyt4bb7/project/code/vision/beaconTrack.h

SHA-256

:

52d199e7661cf05c6fb29544bf88261d4cad7f591c70c3c7820df1123e

代码 (c)

```

#ifndef BEACON_TRACK_H
#define BEACON_TRACK_H

#include <stdint.h>

#include "openMorph.h" /* BEACON_DETECTOR_STRUCT / BEACON_DATA_STRUCT / OM_MAX_OBJECTS */

#define BEACON_CONFIRMED_FLAG_FRESH    (0x01u)
#define BEACON_CONFIRMED_FLAG_COAST    (0x02u)
#define BEACON_CONFIRMED_FLAG_EDGE_GRACE (0x04u)
#define BEACON_CONFIRMED_FLAG_BORDER_REJECTED (0x08u)

/*
 * beaconTrack - 信标时间一致性跟踪（抗光关键层）
 *
 * 单帧检测器（运行时 BinaryCC / Legacy Canny 二选一）可能出现瞬时反光误检(FP)与小目标漏检(FN)。本模块在其上叠加
 * 跨帧关联 + 位置/面积连续性门控 + 稳定性计数：连续 confirmHits 帧关联上才
 * 确认输出。连续 dropMisses 帧丢失才删除。瞬时反光跨帧不连续，过不了确认
 * 门控被滤掉。历史 Legacy Canny 仿真(run_beacon_tracking_test.py): 单帧 4 FP → 跟踪 0 FP。
 *
 * 信标恒亮不闪、可多帧延迟，故时间一致性是合理且有效的抗光手段。
 */

/* 跟踪参数默认值已移至 config/configBeaconTrack.h(VISION_TRACK_*_DEFAULT)。 */

```

```

/* 单条轨迹 */
typedef struct
{
    float cx;          /* 平滑后质心 X */
    float cy;          /* 平滑后质心 Y */
    float size;         /* 平滑后像素数 */
    float lastRawX;     /* 上次关联的原始检测 X */
    float lastRawY;     /* 上次关联的原始检测 Y */
    float lastRawSize;  /* 上次关联的原始检测面积 */
    float velX;         /* 原始检测速度(px/frame) */
    float velY;         /* 原始检测速度(px/frame) */
    float areaVel;      /* 原始检测面积速度(area pixels/frame) */
    uint32_t frameSeq;  /* 最近一次输出所属图像帧序号 */
    uint16_t trackId;   /* 轨迹 ID; 0 保留为无效 */
    uint16_t hits;      /* 连续命中帧数 */
    uint16_t misses;    /* 连续丢失帧数 */
    uint8_t confirmed;  /* 1=已确认(连续命中达阈值) */
    uint8_t fresh;      /* 1=本帧关联到真实检测 */
    uint8_t velocityPrimed; /* 1=velX/velY 已由连续两次检测建立 */
    uint8_t areaVelocityPrimed; /* 1=areaVel 已由连续两次检测建立 */
    uint8_t borderEligible; /* 1=已通过发布边界施密特门 */
    uint8_t borderOutsideFrames; /* 连续越过 Exit 边界的输出判定帧数 */
    uint8_t edgePredictionSideMask; /* 上次 fresh 命中的 EdgeBand 边位掩码 */
    uint8_t published; /* 至少产生过一帧普通发布记录 */
    uint8_t active;     /* 1=槽位占用 */
} beaconTrackStruct;

/* 确认输出的信标质心(供决策/规划消费) */
typedef struct
{
    uint16_t centerX;
    uint16_t centerY;
    uint16_t size;
    uint16_t trackId;
    uint16_t predX;
    uint16_t predY;
    uint16_t predSize;
    uint16_t misses;
    uint8_t fresh;
    uint8_t flags;
    uint32_t frameSeq;
    float velX;      /* px/frame */
    float velY;      /* px/frame */
    float areaVel;   /* area pixels/frame */
} beaconConfirmedStruct;

typedef struct
{
    uint8_t left;
    uint8_t right;
    uint8_t top;
    uint8_t bottom;
} beaconTrackBorderMarginsStruct;

typedef struct
{
    beaconTrackBorderMarginsStruct enter;
    beaconTrackBorderMarginsStruct exit;
} beaconTrackBorderConfigStruct;

typedef struct
{
    uint8_t left;
    uint8_t right;
    uint8_t top;
    uint8_t bottom;
} beaconTrackEdgeBandsStruct;

typedef struct
{
    uint16_t holdMs;
    beaconTrackEdgeBandsStruct bands;
} beaconTrackEdgePredictionConfigStruct;

/* 跟踪器 */
typedef struct
{
    beaconTrackStruct tracks[OM_MAX_OBJECTS];

```

```

    beaconConfirmedStruct confirmed[OM_MAX_OBJECTS]; /* update 后的确认输出 */
    uint16_t confirmedCount;
    uint16_t nextTrackId;
    uint32_t frameSeq;
} beaconTrackerStruct;

/* 初始化(清空所有轨迹) */
void beaconTrackInit(beaconTrackerStruct* tracker);

/* 运行时设置跟踪参数(默认取 configBeaconTrack.h #define, 菜单可热重载)。
 * positionAlpha/sizeAlpha: 关联更新一阶低通系数(0<alpha<=1, 越小越平滑; 替代原硬编码 0.5)。
 * 仅 Core3 前台 beaconTrackUpdate 读写, 无 ISR 竞争。 */
void beaconTrackSetParams(float maxAssocDist, float sizeRatioMax,
    uint16_t confirmHits, uint16_t dropMisses, uint16_t coastMisses,
    float positionAlpha, float sizeAlpha,
    float areaVelAlpha, float assocScoreMax,
    float assocAreaWeight, float assocAreaVelWeight);

void beaconTrackSetBorderMargins(const beaconTrackBorderConfigStruct* margins);
void beaconTrackSetEdgePredictionConfig(
    const beaconTrackEdgePredictionConfigStruct* config);

/* 用本帧 om_beacon 检测结果更新跟踪器; 更新后确认信标在 tracker->confirmed[0..confirmedCount-1] */
void beaconTrackUpdate(beaconTrackerStruct* tracker, const BEACON_DETECTOR_STRUCT* det);
void beaconTrackMarkPublished(beaconTrackerStruct* tracker, uint16_t trackId);

#endif /* BEACON_TRACK_H */

```

core3_cyt4bb7/project/code/vision/core3VisionWorkspace.c

SHA-256

:

7654ce7b768c1786dd907250ba0662632a01f3033118aacc04494e064d298585

代码 (c)

```

#include "core3VisionWorkspace.h"

core3VisionWorkspaceStruct g_core3VisionWorkspace;

uint16_t contourGeneration = 0u;
const int8_t contourDx[8] = {-1, 0, 1, 1, 1, 0, -1, -1};
const int8_t contourDy[8] = {-1, -1, -1, 0, 1, 1, 1, 0};

```

core3_cyt4bb7/project/code/vision/core3VisionWorkspace.h

SHA-256

:

c725e10e9605890472400fec4dec874cab87c00b314e4ec8153682ec9fa2a1cf

代码 (c)

```

#ifndef CORE3_VISION_WORKSPACE_H
#define CORE3_VISION_WORKSPACE_H

#include <stdint.h>

#include "openMorph_config.h"

typedef struct
{
    uint32_t visitMap[OM_BUFFER_SIZE];
    union
    {
        {
            uint32_t bfsQueue[OM_BUFFER_SIZE];
            struct
            {
                uint8_t gaussian[OM_BUFFER_SIZE];
                uint16_t edgeQueue[OM_BUFFER_SIZE];
            } canny;
        } scratch;
    }
} core3VisionWorkspaceStruct;

```



```

extern core3VisionWorkspaceStruct g_core3VisionWorkspace;

_Static_assert(OM_BUFFER_SIZE == (188u * 120u), "Core3 vision pixel count mismatch");
_Static_assert(sizeof(uint8_t) == 1u, "Core3 gray pixel must remain one byte");
_Static_assert(sizeof(uint16_t) == 2u, "Canny pixel index must remain two bytes");
_Static_assert(OM_BUFFER_SIZE <= UINT16_MAX, "Canny pixel index must fit uint16_t");
_Static_assert(sizeof(((core3VisionWorkspaceStruct*) 0)->visitMap[0]) == sizeof(uint32_t),
    "Beacon generation tag must remain uint32_t");
_Static_assert(sizeof(((core3VisionWorkspaceStruct*) 0)->scratch.canny) <=
    sizeof(((core3VisionWorkspaceStruct*) 0)->scratch.bfsQueue),
    "Canny scratch must fit the shared BFS queue storage");
_Static_assert(sizeof(core3VisionWorkspaceStruct) == (OM_BUFFER_SIZE * sizeof(uint32_t) * 2u),
    "Core3 shared vision workspace must remain 180 480 bytes");

#endif /* CORE3_VISION_WORKSPACE_H */

```

core3_cyt4bb7/project/code/vision/om_beacon.c

SHA-256

:

d8ffe1e4726cbd6fad7f66b174f1c5adaac019c314988e5d8df3d1443b4f34e2

代码 (c)

```

/*
 * om_beacon.c - [6] 信标检测 (Beacon Detection)
 *
 * 基于 Canny 边缘 + 简化 BFS, 检测多个红外信标 (闭合椭圆/圆形轮廓)
 * 判据: 闭合性 + 尺寸 + 偏心率上限 + 内部亮度
 * 复用 Core3 vision workspace 的 inContourMap / contourQueue / contourGeneration 基础设施
 */

#include "om_internal.h"

// ---- typedef ----

typedef struct
{
    uint16_t sizeMax;
    uint16_t sizeMin;
    float eccMaxRatioK; // 预计算: ((eccMax-1)/(eccMax+1))^2, 用于无 sqrt 偏心率比较
    uint8_t brightnessMin; // 内部亮度下限
    uint16_t maxEndpoints; // 允许的最大端点数 (信标边缘常有断裂)
} BEACON_DATA_CONFIG;

// ---- 内部变量 ----

static BEACON_DATA_CONFIG beaconDataConfig;

// ---- 内部函数声明 ----

static inline uint16_t beaconBFS(const u8Image* edgeSrc, uint16_t startX, uint16_t startY, uint32_t genTag, BEACON_DATA_STRUCT* beacon, float*
outMxx, float* outMyy, float* outMxy);

// ---- 外部函数实现 ----

void beaconDetectInit(uint16_t sizeMin, uint16_t sizeMax, float eccMax, uint8_t brightnessMin, uint16_t maxEndpoints)
{
    beaconDataConfig.sizeMin = sizeMin;
    beaconDataConfig.sizeMax = sizeMax;
    // 预计算偏心率阈值的平方比值, 运行时只需一次乘法+比较
    float k = (eccMax - 1.0f) / (eccMax + 1.0f);
    beaconDataConfig.eccMaxRatioK = k * k;
    beaconDataConfig.brightnessMin = brightnessMin;
    beaconDataConfig.maxEndpoints = maxEndpoints;
}

uint16_t beaconDetect(const u8Image* edgeSrc, const u8Image* graySrc, BEACON_DETECTOR_STRUCT* obj)
{
    if (edgeSrc->size > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE) return 0;

    // generation 替代 memset (与 contourDetect 共享基础设施)
    contourGeneration++;
    if (contourGeneration == 0)
    {

```

```

        memset(inContourMap, 0, sizeof(inContourMap));
        contourGeneration = 1;
    }
    uint32_t genBase = (uint32_t) contourGeneration << 16;

    uint16_t beaconCount = 0;
    uint32_t tagCounter = 0;

    for (uint32_t y = 0; y < edgeSrc->yLen && beaconCount < (OM_MAX_OBJECTS - 1u); y++)
    {
        uint32_t rowOff = y * edgeSrc->xLen;
        for (uint32_t x = 0; x < edgeSrc->xLen && beaconCount < 100; x++)
        {
            uint32_t i = rowOff + x;
            if (edgeSrc->ptr[i] != WHITE || (inContourMap[i] >> 16) == contourGeneration) continue;

            tagCounter++;
            if (tagCounter > 0xFFFFu) goto beaconDone;

            uint32_t genTag = genBase | (uint32_t) tagCounter;

            float curMxx, curMyy, curMxy;
            uint16_t ep = beaconBFS(edgeSrc, (uint16_t) x, (uint16_t) y, genTag,
                                    &obj->beaconData[beaconCount], &curMxx, &curMyy, &curMxy);

            // 闭合轮廓过滤 (信标边缘断裂容忍)
            if (ep > beaconDataConfig.maxEndpoints) continue;

            // 尺寸过滤
            uint16_t sz = obj->beaconData[beaconCount].contourSize;
            if (sz < beaconDataConfig.sizeMin || sz > beaconDataConfig.sizeMax) continue;

            // 偏心率上限过滤: 拒绝过度拉伸的非信标轮廓
            // eigMax/eigMin > eccMax  <=>  diff^2 > sum^2 * ((eccMax-1)/(eccMax+1))^2
            float eigSum = curMxx + curMyy;
            float eigDiff2 = (curMxx - curMyy) * (curMxx - curMyy) + 4.0f * curMxy * curMxy;
            if (eigSum > 0.0f)
            {
                float bound = eigSum * eigSum * beaconDataConfig.eccMaxRatioK;
                if (eigDiff2 > bound) continue;
            }

            // 质心亮度检查: 填充椭圆的质心必在填充区域内, 单像素即可判定
            uint16_t cx = obj->beaconData[beaconCount].centerX;
            uint16_t cy = obj->beaconData[beaconCount].centerY;
            uint8_t centroidBrightness = graySrc->ptr[(uint32_t) cy * graySrc->xLen + (uint32_t) cx];
            obj->beaconData[beaconCount].brightness = centroidBrightness;
            if (centroidBrightness < beaconDataConfig.brightnessMin) continue;

            beaconCount++;
        }
    }

beaconDone:
    obj->beaconCount = beaconCount;
    for (uint16_t i = beaconCount; i < OM_MAX_OBJECTS; i++) obj->beaconData[i].contourSize = 0;
    return beaconCount;
}

// ---- 内部函数实现 ----

// 简化 BFS: 仅计算二阶矩 (无三阶矩/朝向), 返回端点数
static inline uint16_t beaconBFS(const u8Image* obj, uint16_t startX, uint16_t startY, uint32_t genTag, BEACON_DATA_STRUCT* beacon, float* outMxx,
float* outMyy, float* outMxy)
{
    uint32_t width = obj->xLen;
    uint32_t height = obj->yLen;

    beacon->contourSize = 0;
    beacon->centerX = 0;
    beacon->centerY = 0;
    beacon->brightness = 0;

    uint16_t head = 0;
    uint16_t tail = 0;

    inContourMap[startY * width + startX] = genTag;
    contourQueue[tail++] = ((uint32_t) startY << 16) | (uint32_t) startX;

```

```

uint32_t sumX = 0;
uint32_t sumY = 0;
uint32_t sumXX = 0;
uint32_t sumYY = 0;
uint32_t sumXY = 0;
uint16_t totalCount = 0;
uint16_t endpointCount = 0;
uint8_t abandon = 0;

while (head < tail)
{
    uint16_t cx = (uint16_t) (contourQueue[head] & 0xFFFFu);
    uint16_t cy = (uint16_t) (contourQueue[head] >> 16);
    head++;

    totalCount++;

    if (!abandon)
    {
        sumX += cx;
        sumY += cy;
        sumXX += (uint32_t) cx * cx;
        sumYY += (uint32_t) cy * cy;
        sumXY += (uint32_t) cx * cy;
    }

    uint8_t whiteNeighborCount = 0;

    for (uint8_t i = 0; i < 8; i++)
    {
        int32_t nx = (int32_t) cx + contourDx[i];
        int32_t ny = (int32_t) cy + contourDy[i];

        if (nx < 0 || (uint32_t) nx >= width || ny < 0 || (uint32_t) ny >= height) continue;

        uint16_t nIndex = (uint16_t) ((uint32_t) ny * width + (uint32_t) nx);

        if (obj->ptr[nIndex] == WHITE)
        {
            whiteNeighborCount++;

            if ((inContourMap[nIndex] >> 16) != contourGeneration)
            {
                inContourMap[nIndex] = genTag;
                if (tail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
                    contourQueue[tail++] = ((uint32_t) ny << 16) | (uint32_t) nx;
            }
        }
    }

    if (!abandon && whiteNeighborCount < 2) endpointCount++;

    if (!abandon && (totalCount > beaconDataConfig.sizeMax || endpointCount > beaconDataConfig.maxEndpoints)) abandon = 1;
}

/* abandon 时置 0: 让调用方尺寸过滤 (sz < sizeMin) 直接拒绝本应丢弃的轮廓 */
beacon->contourSize = abandon ? 0u : totalCount;

if (totalCount > 0 && !abandon)
{
    float fN = (float) totalCount;
    float meanX = (float) sumX / fN;
    float meanY = (float) sumY / fN;
    beacon->centerX = (uint16_t) (sumX / totalCount);
    beacon->centerY = (uint16_t) (sumY / totalCount);

    *outMxx = (float) sumXX / fN - meanX * meanX;
    *outMyy = (float) sumYY / fN - meanY * meanY;
    *outMxy = (float) sumXY / fN - meanX * meanY;
}
else
{
    *outMxx = 0.0f;
    *outMyy = 0.0f;
    *outMxy = 0.0f;
}

return endpointCount;
}

```

core3_cyt4bb7/project/code/vision/om_beaconcc.c

SHA-256

:

565bbc8e60e4e0571d715825d28b5f3feeb97bdcd27d99a61ac0b8d4e3570998

代码 (c)

```
/*
 * om_beaconcc.c - core1 信标 BinaryCC 检测
 *
 * 双阈值二值化 + 八连连通域 + 矩/Hu 筛选, 输出兼容 BEACON_DETECTOR_STRUCT。
 */

#include "om_beaconcc.h"

#include "om_internal.h"

typedef struct
{
    uint16_t area;
    uint16_t bboxMinX;
    uint16_t bboxMinY;
    uint16_t bboxMaxX;
    uint16_t bboxMaxY;
    float centroidX;
    float centroidY;
    float meanBrightness;
    uint8_t peakBrightness;
    float m20;
    float m02;
    float m11;
    float m30;
    float m03;
    float m21;
    float m12;
    float hu[7];
    float huDistance;
    float ecc;
    float fillRatio;
    uint8_t shapeEligible;
    uint8_t huEligible;
} BEACONCC_CANDIDATE_STRUCT;

static BEACONCC_CONFIG_STRUCT beaconCcConfig;
static uint8_t beaconCcTemplateReady = 0u;
static float beaconCcHuTemplate[7];
static uint8_t beaconCcCandidateCount = 0u;

static void beaconCcThreshold(const u8Image* graySrc, u8Image* maskDst);
static uint8_t beaconCcVisited(uint32_t index);
static void beaconCcMarkVisited(uint32_t index);
static uint8_t beaconCcScanComponent(const u8Image* graySrc, const u8Image* maskDst, uint32_t startIndex,
    BEACONCC_CANDIDATE_STRUCT* out);
static void beaconCcFinishCandidate(BEACONCC_CANDIDATE_STRUCT* c, float sumX, float sumY,
    float sumX2, float sumY2, float sumXY,
    float sumX3, float sumY3, float sumX2Y, float sumXY2,
    float sumGray, uint8_t peakBrightness);
static void beaconCcComputeHu(float area, float mu20, float mu02, float mu11, float mu30, float mu03,
    float mu21, float mu12, float hu[7]);
static float beaconCcHuDistance(const float hu[7], const float templ[7]);

void beaconCcInit(const BEACONCC_CONFIG_STRUCT* config)
{
    memset(&beaconCcConfig, 0, sizeof(beaconCcConfig));
    memset(beaconCcHuTemplate, 0, sizeof(beaconCcHuTemplate));
    beaconCcTemplateReady = 0u;
    beaconCcCandidateCount = 0u;
    if (config != 0) beaconCcConfig = *config;
}

void beaconCcSetConfig(const BEACONCC_CONFIG_STRUCT* config)
{
    if (config == 0) return;
    beaconCcConfig = *config;
```

```

}

void beaconCcSetTemplate(const float hu[7], float huMax)
{
    if (hu == 0 || huMax <= 0.0f)
    {
        beaconCcTemplateReady = 0u;
        beaconCcConfig.huMax = 0.0f;
        return;
    }
    for (uint8_t i = 0u; i < 7u; i++) beaconCcHuTemplate[i] = hu[i];
    beaconCcConfig.huMax = huMax;
    beaconCcTemplateReady = 1u;
}

uint16_t beaconCcDetect(const u8Image* graySrc, u8Image* maskDst, BEACON_DETECTOR_STRUCT* obj)
{
    if (obj == 0 || graySrc == 0 || maskDst == 0 || graySrc->ptr == 0 || maskDst->ptr == 0 ||
        graySrc->size > OM_BUFFER_SIZE || maskDst->size < graySrc->size)
    {
        if (obj != 0) obj->beaconCount = 0u;
        return 0u;
    }

    beaconCcThreshold(graySrc, maskDst);
    beaconCcCandidateCount = 0u;

    contourGeneration++;
    if (contourGeneration == 0u)
    {
        memset(inContourMap, 0, graySrc->size * sizeof(inContourMap[0]));
        contourGeneration = 1u;
    }

    for (uint32_t i = 0u; i < graySrc->size; i++)
    {
        BEACONCC_CANDIDATE_STRUCT c;
        if (maskDst->ptr[i] != WHITE || beaconCcVisited(i)) continue;

        memset(&c, 0, sizeof(c));
        if (!beaconCcScanComponent(graySrc, maskDst, i, &c))
        {
            continue;
        }

        if (c.area < beaconCcConfig.areaMin)
        {
            continue;
        }
        if (beaconCcConfig.areaMax > 0u && c.area > beaconCcConfig.areaMax)
        {
            continue;
        }
        if (beaconCcConfig.brightnessMin > 0u && c.meanBrightness < (float) beaconCcConfig.brightnessMin)
        {
            continue;
        }
        if (c.shapeEligible && beaconCcConfig.eccMax > 0.0f && c.ecc > beaconCcConfig.eccMax)
        {
            continue;
        }
        if (c.shapeEligible && beaconCcConfig.fillMin > 0.0f && c.fillRatio < beaconCcConfig.fillMin)
        {
            continue;
        }
        if (beaconCcTemplateReady && c.huEligible && beaconCcConfig.huMax > 0.0f &&
            c.huDistance > beaconCcConfig.huMax)
        {
            continue;
        }

        if (beaconCcCandidateCount < OM_MAX_OBJECTS)
        {
            uint8_t dst = beaconCcCandidateCount;
            obj->beaconData[dst].centerX = (uint16_t) (c.centroidX + 0.5f);
            obj->beaconData[dst].centerY = (uint16_t) (c.centroidY + 0.5f);
            obj->beaconData[dst].contourSize = c.area;
            obj->beaconData[dst].brightness = (uint8_t) (c.meanBrightness + 0.5f);
            beaconCcCandidateCount++;
        }
    }
}

```

```

    }
}

obj->beaconCount = beaconCcCandidateCount;
for (uint8_t i = beaconCcCandidateCount; i < OM_MAX_OBJECTS; i++) obj->beaconData[i].contourSize = 0u;
return beaconCcCandidateCount;
}

static void beaconCcThreshold(const u8Image* graySrc, u8Image* maskDst)
{
    for (uint32_t i = 0u; i < graySrc->size; i++)
    {
        uint8_t v = graySrc->ptr[i];
        maskDst->ptr[i] = (v >= beaconCcConfig.thresholdLow && v <= beaconCcConfig.thresholdHigh) ? WHITE : BLACK;
    }
}

static uint8_t beaconCcVisited(uint32_t index)
{
    return ((inContourMap[index] >> 16) == contourGeneration) ? 1u : 0u;
}

static void beaconCcMarkVisited(uint32_t index)
{
    inContourMap[index] = ((uint32_t) contourGeneration << 16) | 1u;
}

static uint8_t beaconCcScanComponent(const u8Image* graySrc, const u8Image* maskDst, uint32_t startIndex,
    BEACONCC_CANDIDATE_STRUCT* out)
{
    uint32_t head = 0u;
    uint32_t tail = 0u;
    float sumX = 0.0f, sumY = 0.0f, sumX2 = 0.0f, sumY2 = 0.0f, sumXY = 0.0f;
    float sumX3 = 0.0f, sumY3 = 0.0f, sumX2Y = 0.0f, sumXY2 = 0.0f, sumGray = 0.0f;
    uint8_t peakBrightness = 0u;

    out->bboxMinX = graySrc->xLen;
    out->bboxMinY = graySrc->yLen;
    if (tail >= OM_BUFFER_SIZE) return 0u;
    contourQueue[tail++] = startIndex;
    beaconCcMarkVisited(startIndex);

    while (head < tail)
    {
        uint32_t cur = contourQueue[head++];
        uint32_t x = cur % graySrc->xLen;
        uint32_t y = cur / graySrc->xLen;
        float xf = (float) x;
        float yf = (float) y;
        uint8_t gray8 = graySrc->ptr[cur];
        float gray = (float) gray8;

        out->area++;
        if (x < out->bboxMinX) out->bboxMinX = (uint16_t) x;
        if (y < out->bboxMinY) out->bboxMinY = (uint16_t) y;
        if (x > out->bboxMaxX) out->bboxMaxX = (uint16_t) x;
        if (y > out->bboxMaxY) out->bboxMaxY = (uint16_t) y;
        if (gray8 > peakBrightness) peakBrightness = gray8;

        sumX += xf;
        sumY += yf;
        sumX2 += xf * xf;
        sumY2 += yf * yf;
        sumXY += xf * yf;
        sumX3 += xf * xf * xf;
        sumY3 += yf * yf * yf;
        sumX2Y += xf * xf * yf;
        sumXY2 += xf * yf * yf;
        sumGray += gray;

        for (uint8_t ni = 0u; ni < 8u; ni++)
        {
            int32_t nx = (int32_t) x + contourDx[ni];
            int32_t ny = (int32_t) y + contourDy[ni];
            uint32_t next;
            if (nx < 0 || ny < 0 || (uint32_t) nx >= graySrc->xLen || (uint32_t) ny >= graySrc->yLen) continue;
            next = (uint32_t) ny * graySrc->xLen + (uint32_t) nx;
            if (maskDst->ptr[next] == WHITE && !beaconCcVisited(next))
            {

```

```

        if (tail >= OM_BUFFER_SIZE) return 0u;
        beaconCcMarkVisited(next);
        contourQueue[tail++] = next;
    }
}

if (out->area == 0u) return 0u;
beaconCcFinishCandidate(out, sumX, sumY, sumX2, sumY2, sumXY, sumX3, sumY3,
    sumX2Y, sumXY2, sumGray, peakBrightness);
return 1u;
}

static void beaconCcFinishCandidate(BEACONCC_CANDIDATE_STRUCT* c, float sumX, float sumY,
    float sumX2, float sumY2, float sumXY,
    float sumX3, float sumY3, float sumX2Y, float sumXY2,
    float sumGray, uint8_t peakBrightness)
{
    float area = (float) c->area;
    float cx = sumX / area;
    float cy = sumY / area;
    float mu20 = sumX2 - cx * sumX;
    float mu02 = sumY2 - cy * sumY;
    float mu11 = sumXY - cx * sumY;
    float mu30 = sumX3 - 3.0f * cx * sumX2 + 3.0f * cx * cx * sumX - cx * cx * cx * area;
    float mu03 = sumY3 - 3.0f * cy * sumY2 + 3.0f * cy * cy * sumY - cy * cy * cy * area;
    float mu21 = sumX2Y - cy * sumX2 - 2.0f * cx * sumXY + 2.0f * cx * cy * sumX + cx * cx * sumY - cx * cx * cy * area;
    float mu12 = sumXY2 - cx * sumY2 - 2.0f * cy * sumXY + 2.0f * cx * cy * sumY + cy * cy * sumX - cx * cy * cy * area;
    float covXX = mu20 / area;
    float covYY = mu02 / area;
    float covXY = mu11 / area;
    float trace = covXX + covYY;
    float delta = sqrtf((covXX - covYY) * (covXX - covYY) + 4.0f * covXY * covXY);
    float lambdaA = (trace + delta) * 0.5f;
    float lambdaB = (trace - delta) * 0.5f;
    float bboxArea = (float) ((c->bboxMaxX - c->bboxMinX + 1u) * (c->bboxMaxY - c->bboxMinY + 1u));

    if (lambdaA < lambdaB)
    {
        float t = lambdaA;
        lambdaA = lambdaB;
        lambdaB = t;
    }

    c->centroidX = cx;
    c->centroidY = cy;
    c->meanBrightness = sumGray / area;
    c->peakBrightness = peakBrightness;
    c->m20 = mu20;
    c->m02 = mu02;
    c->m11 = mu11;
    c->m30 = mu30;
    c->m03 = mu03;
    c->m21 = mu21;
    c->m12 = mu12;
    c->ecc = (lambdaB > 1.0e-6f) ? (lambdaA / lambdaB) : 9999.0f;
    c->fillRatio = (bboxArea > 0.0f) ? (area / bboxArea) : 0.0f;
    beaconCcComputeHu(area, mu20, mu02, mu11, mu30, mu03, mu21, mu12, c->hu);
    c->huDistance = beaconCcTemplateReady ? beaconCcHuDistance(c->hu, beaconCcHuTemplate) : -1.0f;
    c->shapeEligible = (c->area >= beaconCcConfig.huAreaMin) ? 1u : 0u;
    c->huEligible = (beaconCcTemplateReady && c->shapeEligible) ? 1u : 0u;
}

static void beaconCcComputeHu(float area, float mu20, float mu02, float mu11, float mu30, float mu03,
    float mu21, float mu12, float hu[7])
{
    float invArea2 = 1.0f / (area * area);
    float invArea25 = invArea2 / sqrtf(area);
    float eta20 = mu20 * invArea2;
    float eta02 = mu02 * invArea2;
    float eta11 = mu11 * invArea2;
    float eta30 = mu30 * invArea25;
    float eta03 = mu03 * invArea25;
    float eta21 = mu21 * invArea25;
    float eta12 = mu12 * invArea25;
    float a = eta30 + eta12;
    float b = eta21 + eta03;
    float c = eta30 - 3.0f * eta12;
    float d = 3.0f * eta21 - eta03;

```

```
    hu[0] = eta20 + eta02;
    hu[1] = (eta20 - eta02) * (eta20 - eta02) + 4.0f * eta11 * eta11;
    hu[2] = c * c + d * d;
    hu[3] = a * a + b * b;
    hu[4] = c * a * (a * a - 3.0f * b * b) + d * b * (3.0f * a * a - b * b);
    hu[5] = (eta20 - eta02) * (a * a - b * b) + 4.0f * eta11 * a * b;
    hu[6] = d * a * (a * a - 3.0f * b * b) - c * b * (3.0f * a * a - b * b);
}

static float beaconCcHuDistance(const float hu[7], const float templ[7])
{
    static const float weights[7] = {2.0f, 2.0f, 1.5f, 1.5f, 0.5f, 0.5f, 0.02f};
    float distance = 0.0f;
    for (uint8_t i = 0u; i < 7u; i++)
    {
        float a = fabsf(hu[i]);
        float b = fabsf(templ[i]);
        if (a < 1.0e-12f) a = 1.0e-12f;
        if (b < 1.0e-12f) b = 1.0e-12f;
        distance += weights[i] * fabsf(logf(a) - logf(b));
    }
    return distance;
}
```

core3_cyt4bb7/project/code/vision/om_beaconcc.h

SHA-256

:

27d217cffe3029e12c16e4f6c01415c0c29e8399466bdf209fce9b653cfeae87

代码 (c)

```
#ifndef OM_BEACONCC_H
#define OM_BEACONCC_H

#include <stdint.h>

#include "openMorph.h"

typedef struct
{
    uint8_t thresholdLow;
    uint8_t thresholdHigh;
    uint16_t areaMin;
    uint16_t areaMax;
    uint8_t brightnessMin;
    float eccMax;      /* 0=关闭 */
    float fillMin;     /* 0=关闭 */
    float huMax;       /* 0=关闭 */
    uint16_t huAreaMin; /* 小于该面积的远点只输出 Hu，不按 Hu 拒绝 */
} BEACONCC_CONFIG_STRUCT;

void beaconCcInit(const BEACONCC_CONFIG_STRUCT* config);
void beaconCcSetConfig(const BEACONCC_CONFIG_STRUCT* config);
void beaconCcSetTemplate(const float hu[7], float huMax);
uint16_t beaconCcDetect(const u8Image* graySrc, u8Image* maskDst, BEACON_DETECTOR_STRUCT* obj);

#endif /* OM_BEACONCC_H */
```

core3_cyt4bb7/project/code/vision/om_canny.c

SHA-256

:

ae68fd41321e73a4cb679f23232d8c3cb8d95a62a2f180f78091b02386e24f3c

代码 (c)

```
/*
 * om_canny.c - [4] 高斯模糊及 Canny 边缘检测
 *
 * 3x3 定点高斯核 / 逐行流水线 Sobel+NMS+滞后门限 边缘检测
```



```

*/

#include "om_internal.h"

// ---- define ----

#define CANNY_RING_WIDTH (OM_CANNY_RING_WIDTH)

// ---- typedef ----

typedef struct
{
    uint32_t width;
    uint32_t height;
    uint32_t pixelCount;
    uint32_t cropStartX;
    uint32_t cropStartY;
    uint32_t cropEndX;
    uint32_t cropEndY;
    uint32_t blurStartX;
    uint32_t blurStartY;
    uint32_t blurEndX;
    uint32_t blurEndY;
} gaussConfigStruct;

typedef struct
{
    uint32_t width;
    uint32_t height;
    uint32_t pixelCount;
    uint32_t sobelStartX;
    uint32_t sobelStartY;
    uint32_t sobelEndX;
    uint32_t sobelEndY;
    uint32_t nmsStartX;
    uint32_t nmsStartY;
    uint32_t nmsEndX;
    uint32_t nmsEndY;
    uint16_t thresholdLow;
    uint16_t thresholdHigh;
    uint32_t thresholdLowSq;
    uint32_t thresholdHighSq;
    int32_t trackOffsets[8];
} cannyConfigStruct;

// ---- 内部变量 ----

static uint16_t gaussKernelPreValueInt0[256];
static uint16_t gaussKernelPreValueInt1[256];
static uint16_t gaussKernelPreValueInt2[256];
static gaussConfigStruct gaussConfig = {0u};
static cannyConfigStruct cannyConfig = {0u};
static uint32_t sobelMapRing[3][CANNY_RING_WIDTH] = {0};
static uint8_t sobelDirMapRing[3][CANNY_RING_WIDTH] = {0};

// ---- 外部函数实现 ----

void gaussianInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY)
{
    // gaussianKernelTemp[0] : 四个角点对应的核值
    // gaussianKernelTemp[1] : 四个边点对应的核值
    // gaussianKernelTemp[2] : 中心点对应的核值
    float gaussianKernelTemp[3] = {0.0f, 0.0f, 0.0f};

    memset(&gaussConfig, 0, sizeof(gaussConfig));

    if ((NULL == src) || (NULL == src->ptr) || (0u == src->xLen) || (0u == src->yLen) ||
        (src->xLen > CANNY_RING_WIDTH) || ((src->xLen * src->yLen) > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE))
    {
        memset(gaussKernelPreValueInt0, 0, sizeof(gaussKernelPreValueInt0));
        memset(gaussKernelPreValueInt1, 0, sizeof(gaussKernelPreValueInt1));
        memset(gaussKernelPreValueInt2, 0, sizeof(gaussKernelPreValueInt2));
        return;
    }

    gaussConfig.width = src->xLen;
    gaussConfig.height = src->yLen;
    gaussConfig.pixelCount = src->xLen * src->yLen;

```

```

if (startX > endX)
{
    uint32_t temp = startX;
    startX = endX;
    endX = temp;
}
if (startY > endY)
{
    uint32_t temp = startY;
    startY = endY;
    endY = temp;
}

gaussConfig.cropStartX = unsignRange(startX, gaussConfig.width - 1u);
gaussConfig.cropStartY = unsignRange(startY, gaussConfig.height - 1u);
gaussConfig.cropEndX = range(endX, gaussConfig.cropStartX, gaussConfig.width - 1u);
gaussConfig.cropEndY = range(endY, gaussConfig.cropStartY, gaussConfig.height - 1u);

gaussConfig.blurStartX = 1u;
gaussConfig.blurStartY = 1u;
gaussConfig.blurEndX = 0u;
gaussConfig.blurEndY = 0u;

if ((gaussConfig.width >= 3u) && (gaussConfig.height >= 3u) &&
    ((gaussConfig.cropEndX - gaussConfig.cropStartX) >= 2u) &&
    ((gaussConfig.cropEndY - gaussConfig.cropStartY) >= 2u))
{
    gaussConfig.blurStartX = (0u == gaussConfig.cropStartX) ? 1u : (gaussConfig.cropStartX + 1u);
    gaussConfig.blurStartY = (0u == gaussConfig.cropStartY) ? 1u : (gaussConfig.cropStartY + 1u);
    gaussConfig.blurEndX = (gaussConfig.cropEndX >= (gaussConfig.width - 1u)) ? (gaussConfig.width - 2u) : (gaussConfig.cropEndX - 1u);
    gaussConfig.blurEndY = (gaussConfig.cropEndY >= (gaussConfig.height - 1u)) ? (gaussConfig.height - 2u) : (gaussConfig.cropEndY - 1u);
}

if (sigma <= 0.0f)
{
    memset(gaussKernelPreValueInt0, 0, sizeof(gaussKernelPreValueInt0));
    memset(gaussKernelPreValueInt1, 0, sizeof(gaussKernelPreValueInt1));
    memset(gaussKernelPreValueInt2, 0, sizeof(gaussKernelPreValueInt2));
    return;
}

float sigma2 = sigma * sigma;
float coeff = 1.0f / (TWO_PI * sigma2);
float sum = 0.0f;

// 生成 3x3 高斯核，并提取 3 个唯一值（角点/边点/中心）
for (uint8_t y = 0u; y < 3; y++)
{
    for (uint8_t x = 0u; x < 3; x++)
    {
        int16_t dx = (int16_t) x - 1;
        int16_t dy = (int16_t) y - 1;
        float exponent = -((float) (dx * dx + dy * dy)) / (2.0f * sigma2);
        float value = (float) exp((double) exponent) * coeff;

        if (0 == dx && 0 == dy) gaussianKernelTemp[2] = value;
        else if (0 == dx || 0 == dy) gaussianKernelTemp[1] = value;
        else gaussianKernelTemp[0] = value;

        sum += value;
    }
}

if (sum <= 0.0f)
{
    memset(gaussKernelPreValueInt0, 0, sizeof(gaussKernelPreValueInt0));
    memset(gaussKernelPreValueInt1, 0, sizeof(gaussKernelPreValueInt1));
    memset(gaussKernelPreValueInt2, 0, sizeof(gaussKernelPreValueInt2));
    return;
}

// 归一化 3 个唯一核值
gaussianKernelTemp[0] /= sum;
gaussianKernelTemp[1] /= sum;
gaussianKernelTemp[2] /= sum;

// 预计算核值与 8bit 灰度值相乘的定点数(放缩 256 倍)，消除内层浮点开销
for (uint16_t gray = 0u; gray < 256; gray++)
{

```

```

        gaussKernelPreValueInt0[gray] = (uint16_t) (gaussianKernelTemp[0] * (float) gray * 256.0f);
        gaussKernelPreValueInt1[gray] = (uint16_t) (gaussianKernelTemp[1] * (float) gray * 256.0f);
        gaussKernelPreValueInt2[gray] = (uint16_t) (gaussianKernelTemp[2] * (float) gray * 256.0f);
    }
}

void cannyInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY, uint16_t thresholdLow, uint16_t thresholdHigh)
{
    gaussianInit(src, sigma, startX, startY, endX, endY);

    memset(&cannyConfig, 0, sizeof(cannyConfig));

    if ((NULL == src) || (NULL == src->ptr) || (0u == src->xLen) || (0u == src->yLen) ||
        (src->xLen > CANNY_RING_WIDTH) || ((src->xLen * src->yLen) > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)) return;

    cannyConfig.width = src->xLen;
    cannyConfig.height = src->yLen;
    cannyConfig.pixelCount = src->xLen * src->yLen;

    if (thresholdLow > thresholdHigh)
    {
        uint16_t temp = thresholdLow;
        thresholdLow = thresholdHigh;
        thresholdHigh = temp;
    }

    cannyConfig.thresholdLow = thresholdLow;
    cannyConfig.thresholdHigh = thresholdHigh;

    // 提前计算平方
    cannyConfig.thresholdLowSq = (uint32_t) thresholdLow * thresholdLow;
    cannyConfig.thresholdHighSq = (uint32_t) thresholdHigh * thresholdHigh;

    cannyConfig.sobelStartX = 1u;
    cannyConfig.sobelStartY = 1u;
    cannyConfig.sobelEndX = 0u;
    cannyConfig.sobelEndY = 0u;
    cannyConfig.nmsStartX = 1u;
    cannyConfig.nmsStartY = 1u;
    cannyConfig.nmsEndX = 0u;
    cannyConfig.nmsEndY = 0u;

    if ((gaussConfig.blurEndX > (gaussConfig.blurStartX + 1u)) &&
        (gaussConfig.blurEndY > (gaussConfig.blurStartY + 1u)))
    {
        cannyConfig.sobelStartX = gaussConfig.blurStartX + 1u;
        cannyConfig.sobelStartY = gaussConfig.blurStartY + 1u;
        cannyConfig.sobelEndX = gaussConfig.blurEndX - 1u;
        cannyConfig.sobelEndY = gaussConfig.blurEndY - 1u;
    }

    if ((cannyConfig.sobelEndX > (cannyConfig.sobelStartX + 1u)) &&
        (cannyConfig.sobelEndY > (cannyConfig.sobelStartY + 1u)))
    {
        cannyConfig.nmsStartX = cannyConfig.sobelStartX + 1u;
        cannyConfig.nmsStartY = cannyConfig.sobelStartY + 1u;
        cannyConfig.nmsEndX = cannyConfig.sobelEndX - 1u;
        cannyConfig.nmsEndY = cannyConfig.sobelEndY - 1u;
    }

    // 预先计算滞后门限追踪(连边)阶段需要的直接邻信偏移
    cannyConfig.trackOffsets[0] = -(int32_t) gaussConfig.width - 1;
    cannyConfig.trackOffsets[1] = -(int32_t) gaussConfig.width;
    cannyConfig.trackOffsets[2] = -(int32_t) gaussConfig.width + 1;
    cannyConfig.trackOffsets[3] = -1;
    cannyConfig.trackOffsets[4] = 1;
    cannyConfig.trackOffsets[5] = (int32_t) gaussConfig.width - 1;
    cannyConfig.trackOffsets[6] = (int32_t) gaussConfig.width;
    cannyConfig.trackOffsets[7] = (int32_t) gaussConfig.width + 1;
}

/* 运行时仅更新 Canny 双阈值，不重算高斯核与 ROI，便于按键调参 */
void cannySetThreshold(uint16_t thresholdLow, uint16_t thresholdHigh)
{
    if (thresholdLow > thresholdHigh)
    {
        uint16_t temp = thresholdLow;
        thresholdLow = thresholdHigh;
        thresholdHigh = temp;
    }
}

```

```

}

cannyConfig.thresholdLow = thresholdLow;
cannyConfig.thresholdHigh = thresholdHigh;
cannyConfig.thresholdLowSq = (uint32_t) thresholdLow * thresholdLow;
cannyConfig.thresholdHighSq = (uint32_t) thresholdHigh * thresholdHigh;
}

void cannyEdgeDetection(const u8Image* src, u8Image* dstGaussBlur, u8Image* dst)
{
    if (NULL == src || NULL == src->ptr || NULL == dstGaussBlur || NULL == dstGaussBlur->ptr ||
        NULL == dst || NULL == dst->ptr) return;
    if (0u == cannyConfig.pixelCount || 0u == gaussConfig.width) return;

    uint32_t queueHead = 0u;
    uint32_t queueTail = 0u;

    // 清零目标画布、ring buffer 残留、高斯模糊边界脏值
    memset(dst->ptr, BLACK, gaussConfig.width * gaussConfig.height);
    memset(dstGaussBlur->ptr, 0, gaussConfig.width * gaussConfig.height);
    memset(sobelMapRing, 0, sizeof(sobelMapRing));
    memset(sobelDirMapRing, 0, sizeof(sobelDirMapRing));

    for (uint32_t y = gaussConfig.blurStartY; y <= gaussConfig.blurEndY; y++)
    {
        // ===== 第 1 阶段: 高斯模糊 =====
        const uint8_t* row0 = src->ptr + (y - 1u) * gaussConfig.width;
        const uint8_t* row1 = src->ptr + y * gaussConfig.width;
        const uint8_t* row2 = src->ptr + (y + 1u) * gaussConfig.width;
        uint8_t* dstRow = dstGaussBlur->ptr + y * gaussConfig.width;

        for (uint32_t x = gaussConfig.blurStartX; x <= gaussConfig.blurEndX; x++)
        {
            uint32_t result = gaussKernelPreValueInt0[row0[x - 1u]] + gaussKernelPreValueInt1[row0[x]] + gaussKernelPreValueInt0[row0[x + 1u]] +
                gaussKernelPreValueInt1[row1[x - 1u]] + gaussKernelPreValueInt2[row1[x]] + gaussKernelPreValueInt1[row1[x + 1u]] +
                gaussKernelPreValueInt0[row2[x - 1u]] + gaussKernelPreValueInt1[row2[x]] + gaussKernelPreValueInt0[row2[x + 1u]];

            result = (result + 128) >> 8;
            dstRow[x] = (uint8_t) ((result > 255) ? 255 : result);
        }

        // ===== 第 2 阶段: Sobel 计算 =====
        if ((cannyConfig.sobelStartX <= cannyConfig.sobelEndX) &&
            (cannyConfig.sobelStartY <= cannyConfig.sobelEndY) &&
            (y >= (cannyConfig.sobelStartY + 1u)) &&
            ((y - 1u) <= cannyConfig.sobelEndY))
        {
            const uint8_t* s_row0 = dstGaussBlur->ptr + (y - 2u) * gaussConfig.width;
            const uint8_t* s_row1 = dstGaussBlur->ptr + (y - 1u) * gaussConfig.width;
            const uint8_t* s_row2 = dstGaussBlur->ptr + y * gaussConfig.width;

            uint32_t* sobelRow = sobelMapRing[(y - 1u) % 3];
            uint8_t* dirRow = sobelDirMapRing[(y - 1u) % 3];

            for (uint32_t x = cannyConfig.sobelStartX; x <= cannyConfig.sobelEndX; x++)
            {
                int32_t gx = (int32_t) s_row0[x + 1u] + ((int32_t) s_row1[x + 1u] << 1) + (int32_t) s_row2[x + 1u] - (int32_t) s_row0[x - 1u] - ((int32_t) s_row1[x -
                    1u] << 1) - (int32_t) s_row2[x - 1u];
                int32_t gy = (int32_t) s_row0[x - 1u] + ((int32_t) s_row0[x] << 1) + (int32_t) s_row0[x + 1u] - (int32_t) s_row2[x - 1u] - ((int32_t) s_row2[x] << 1)
                    - (int32_t) s_row2[x + 1u];

                // 去除掉极其耗时的 sqrtf 运算
                uint32_t mag_sq = (uint32_t) (gx * gx) + (uint32_t) (gy * gy);
                sobelRow[x] = mag_sq;

                // 完全消除浮点除法，只用整数比较判定 4 大区域方向
                int32_t ax = (gx >= 0) ? gx : -gx;
                int32_t ay = (gy >= 0) ? gy : -gy;
                int32_t ay256 = ay << 8;

                if (ay256 <= ax * 106)
                {
                    dirRow[x] = 0; // 水平梯度主导（纵向边缘），沿水平方向抑制
                }
                else if (ay256 >= ax * 618)
                {
                    dirRow[x] = 2; // 垂直梯度主导（横向边缘），沿垂直方向抑制
                }
                else if ((gx ^ gy) >= 0)
            }
        }
    }
}

```

```

    {
        dirRow[x] = 1; // 45 度梯度
    }
    else
    {
        dirRow[x] = 3; // 135 度梯度
    }
}
}

// ===== 第3阶段: 非极大值抑制 =====
if ((cannyConfig.nmsStartX <= cannyConfig.nmsEndX) &&
    (cannyConfig.nmsStartY <= cannyConfig.nmsEndY) &&
    (y >= (cannyConfig.nmsStartY + 2u)) &&
    ((y - 2u) <= cannyConfig.nmsEndY))
{
    const uint32_t* n_row0 = sobelMapRing[(y - 3u) % 3];
    const uint32_t* n_row1 = sobelMapRing[(y - 2u) % 3];
    const uint32_t* n_row2 = sobelMapRing[(y - 1u) % 3];
    const uint8_t* dirRow = sobelDirMapRing[(y - 2u) % 3];

    uint8_t* final_dstRow = dst->ptr + (y - 2u) * gaussConfig.width + cannyConfig.nmsStartX;

    for (uint32_t x = cannyConfig.nmsStartX; x <= cannyConfig.nmsEndX; x++)
    {
        uint32_t edge0, edge1;
        uint32_t magnitude = n_row1[x];
        uint8_t dir = dirRow[x];

        if (dir == 0)
        {
            edge0 = n_row1[x - 1u];
            edge1 = n_row1[x + 1u];
        }
        else if (dir == 2)
        {
            edge0 = n_row0[x];
            edge1 = n_row2[x];
        }
        else if (dir == 1)
        {
            // gx, gy 同号 → 梯度方向 ↗ (亮度梯度指向右上) → NMS 比较 ↗ 与 ↘
            edge0 = n_row0[x + 1u];
            edge1 = n_row2[x - 1u];
        }
        else
        {
            // gx, gy 异号 → 梯度方向 ↘ (亮度梯度指向右下) → NMS 比较 ↘ 与 ↗
            edge0 = n_row0[x - 1u];
            edge1 = n_row2[x + 1u];
        }

        if ((magnitude > edge0) && (magnitude >= edge1))
        {
            if (magnitude >= cannyConfig.thresholdHighSq)
            {
                *final_dstRow = WHITE; // 确信强边缘
                if (queueTail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
                {
                    cannyEdgeQueue[queueTail++] = (uint16_t)((y - 2u) * gaussConfig.width + x);
                }
            }
            else if (magnitude >= cannyConfig.thresholdLowSq)
            {
                *final_dstRow = 127; // 先标记为弱边缘, 用于第四阶段追踪, 避免大数组分配
            }
        }

        final_dstRow++;
    }
}

// ===== 第4阶段: 滞后门限追踪(连边) =====
while (queueHead < queueTail)
{
    uint32_t index = cannyEdgeQueue[queueHead++];

    for (uint8_t i = 0; i < 8; i++)

```

```

    {
        int32_t trackIndex = (int32_t) index + cannyConfig.trackOffsets[i];

        if (trackIndex >= 0 && trackIndex < (int32_t) cannyConfig.pixelCount)
        {
            if (dst->ptr[trackIndex] == 127)
            {
                dst->ptr[trackIndex] = WHITE; // 连通升级为强边缘

                // 防溢出保护
                if (queueTail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
                {
                    cannyEdgeQueue[queueTail++] = (uint16_t) trackIndex;
                }
            }
        }
    }
}

// ===== 收尾: 清理死路 =====
uint32_t totalPixels = gaussConfig.width * gaussConfig.height;
for (uint32_t i = 0; i < totalPixels; i++)
{
    if (dst->ptr[i] == 127) dst->ptr[i] = BLACK; // 无法与强边缘连通被消除
}
}

```

core3_cyt4bb7/project/code/vision/om_internal.h

SHA-256

:

2bdaac56dde7c65646ced94a0a72c26e53b6d9fd7a1ee855489a1ab8ffecb58

代码 (c)

```

/*
 * om_internal.h - openMorph 内部共享头文件 (不对外暴露)
 *
 * 提供各子模块共用的宏定义、标准库包含及共享 BFS 基础设施声明。
 * 外部用户代码只需包含 openMorph.h。
 */
#ifndef OM_INTERNAL_H
#define OM_INTERNAL_H

// =====
// 公共接口 + 基础设施
// =====

#include "openMorph.h" // 公共类型定义与 API 声明
#include "openMorph_config.h" // 可移植尺寸/数量配置 (移植时只改此文件)
#include "commonDef.h" // u8Image, BLACK, WHITE, cReverse, range 等
#include "core3VisionWorkspace.h"

#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

// =====
// 共享宏定义
// =====

#define UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE (OM_BUFFER_SIZE)

// =====
// 共享工作区基础设施 (Canny 与 Beacon 串行复用同一 union)
// =====

extern uint16_t contourGeneration;
extern const int8_t contourDx[8];
extern const int8_t contourDy[8];

#define inContourMap (g_core3VisionWorkspace.visitMap)
#define contourQueue (g_core3VisionWorkspace.scratch.bfsQueue)
#define cannyEdgeQueue (g_core3VisionWorkspace.scratch.canny.edgeQueue)

```

```
#endif // OM_INTERNAL_H
```

core3_cyt4bb7/project/code/vision/openMorph.h

SHA-256

:

886f365fcf5e1585cec1b25f01f54b965ff34c40033f05a2f20705a9c9f6b371

代码 (c)

```
/*
 * Core3 openMorph 子集：保留 Core1 生产版 Canny 与两套信标检测所需 ABI。
 * Core3 不移植 image.c、形态学、Two-Pass 与轮廓跟踪，避免重复相机 owner 和无消费者代码。
 */
#ifndef OPENMORPH_H
#define OPENMORPH_H

#include <stdint.h>

#include "commonDef.h"
#include "openMorph_config.h"

void gaussianInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY,
    uint32_t endX, uint32_t endY);
void cannyInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY,
    uint32_t endX, uint32_t endY, uint16_t thresholdLow, uint16_t thresholdHigh);
void cannySetThreshold(uint16_t thresholdLow, uint16_t thresholdHigh);
void cannyEdgeDetection(const u8Image* src, u8Image* dstGaussBlur, u8Image* dst);

typedef struct
{
    uint16_t centerX;
    uint16_t centerY;
    uint16_t contourSize;
    uint8_t brightness;
} BEACON_DATA_STRUCT;

typedef struct
{
    BEACON_DATA_STRUCT beaconData[OM_MAX_OBJECTS];
    uint8_t beaconCount;
} BEACON_DETECTOR_STRUCT;

void beaconDetectInit(uint16_t sizeMin, uint16_t sizeMax, float eccMax,
    uint8_t brightnessMin, uint16_t maxEndpoints);
uint16_t beaconDetect(const u8Image* edgeSrc, const u8Image* graySrc,
    BEACON_DETECTOR_STRUCT* obj);

#endif /* OPENMORPH_H */
```

core3_cyt4bb7/project/code/vision/openMorph_config.h

SHA-256

:

9fb6374567ec5a208349fdddee2d0280eb91858d730add80076358068184750c

代码 (c)

```
/*
 * openMorph_config.h - openMorph 库可移植配置头
 *
 * 移植到新工程时只需修改本文件即可适配不同图像分辨率与对象数量上限。
 * openMorph 内部其他源文件不应再出现关于尺寸的字面量。
 */
#ifndef OPENMORPH_CONFIG_H
#define OPENMORPH_CONFIG_H

// =====
// 图像最大尺寸 (静态缓冲区按此分配)
// =====
// Core3 实际相机 MT9V03X 为 188 x 120 = 22560 像素。
```

```
// 必须与相机分辨率匹配: 若小于真实分辨率, om_canny/cannyInit 入口保护
// ((xLen*yLen) > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE) 会拒绝处理 → canny 图纯黑、信标失效。
// 本副本只服务 Core3 后视相机, 不引入或接管相机驱动。
// 像素索引以 uint16 存储 (om_beacon/om_canny), 故 W*H 须 <= 65535; 22560 在范围内。
// 本配置与具体相机驱动解耦, 不引入 SDK 头依赖。
// -----
#define OM_IMAGE_MAX_WIDTH (188u)
#define OM_IMAGE_MAX_HEIGHT (120u)
#define OM_BUFFER_SIZE (OM_IMAGE_MAX_WIDTH * OM_IMAGE_MAX_HEIGHT)

// =====
// 对象检测数量上限 (轮廓 / 信标 / 连通域共用)
// =====
// contourData[] / beaconData[] / sbea[] 等数组的成员数量。
// 超过此上限的候选对象会被忽略 (只保留前 N 个有效结果)。
// -----
#define OM_MAX_OBJECTS (101u)

// =====
// Canny 流水线环形缓冲宽度
// =====
// Canny 边缘检测内部 sobel 环形缓冲的列宽, 必须 >= OM_IMAGE_MAX_WIDTH。
// -----
#define OM_CANNY_RING_WIDTH (256u)

// =====
// 轮廓跟踪状态机参数
// =====
#define OM_LOCK_THRESHOLD (2u) // 连续检测到帧数: SEARCHING → ACQUIRING
#define OM_LOCK_CONFIRM (1u) // 继续确认帧数: ACQUIRING → LOCKED
#define OM_LOST_THRESHOLD (5u) // 连续丢失帧数: LOCKED → LOST_HOLDING
#define OM_LOST_TIMEOUT (30u) // 超时帧数: LOST_HOLDING → SEARCHING
#define OM_RAY_BALANCE_CONFIDENCE_MIN (0.50f) // ^ 尖端几何定向最低置信度: 低于此值拒绝更新朝向, 目标位置仍有效
#define OM_ORIENTATION_MIN_SIZE (8u) // 朝向可靠的最小像素数 (与 core0 openMorph 算法同源)
#define OM_ORIENTATION_ALPHA (0.4f) // 朝向低通滤波系数 (喂给 lowPassFilter, 越大越跟随/越小越平滑)
#define OM_POSITION_ALPHA (1.0f) // 质心位置不过滤
#define OM_VELOCITY_ALPHA (0.1f) // 原始质心差分速度低通滤波系数
#define OM_VELOCITY_DT_MIN_S (0.001f) // 有效图像样本间隔下限
#define OM_VELOCITY_DT_MAX_S (0.200f) // 超过此间隔只重建速度基线
#define OM_BORDER_MARGIN (1u) // 画幅边缘安全边距

// =====
// 朝向跳变保护默认值 (运行时可由菜单热重载, 逻辑同 ToF 定高跳变保护)
// NORMAL 态: |Δθ| > JUMP_MAX 视为异常突跳, 拒绝该样本进 SUSPENDED
// SUSPENDED 态: |Δθ| < RECOVER_RAD 即时恢复; 介于二者间累计 CONFIRM_FRAMES 帧后
// 接受新朝向为基线; 持续 > JUMP_MAX 则计数清零继续沿用旧朝向
// -----
#define OM_ORIENTATION_JUMP_MAX (0.785398f) // PI/4, 朝向跳变阈值 (rad)
#define OM_ORIENTATION_RECOVER_RAD (0.196349f) // PI/16, 恢复窄带 (rad)
#define OM_ORIENTATION_CONFIRM_FRAMES (3u) // 异常持续这么多帧后接受新朝向

#endif // OPENMORPH_CONFIG_H
```

core3_cyt4bb7/project/user/cm7_0_isr.c

SHA-256

:

339c2e0cd649351cc4fcc1200cb2d282441f22113bd12c460f9cd44170dfe762

代码 (c)

```
#include "zf_common_headfile.h"

#include "core0Loop.h"
#include "core3LinkCore3.h"
void pit0_ch0_isr()
{
    pit_isr_flag_clear(PIT_CH0);

    pitProcess0();
}

void pit0_ch1_isr()
{
    pit_isr_flag_clear(PIT_CH1);
}
```



```

void pit0_ch2_isr()
{
    pit_isr_flag_clear(PIT_CH2);
}

void pit0_ch10_isr()
{
    pit_isr_flag_clear(PIT_CH10);
}

void pit0_ch11_isr()
{
    pit_isr_flag_clear(PIT_CH11);
}

void pit0_ch12_isr()
{
    pit_isr_flag_clear(PIT_CH12);
}

void pit0_ch13_isr()
{
    pit_isr_flag_clear(PIT_CH13);
}

void pit0_ch14_isr()
{
    pit_isr_flag_clear(PIT_CH14);
}

void pit0_ch15_isr()
{
    pit_isr_flag_clear(PIT_CH15);
}

void pit0_ch16_isr()
{
    pit_isr_flag_clear(PIT_CH16);
}

void pit0_ch17_isr()
{
    pit_isr_flag_clear(PIT_CH17);
}

void pit0_ch18_isr()
{
    pit_isr_flag_clear(PIT_CH18);
}

void pit0_ch19_isr()
{
    pit_isr_flag_clear(PIT_CH19);
}

void pit0_ch20_isr()
{
    pit_isr_flag_clear(PIT_CH20);
}

void pit0_ch21_isr()
{
    pit_isr_flag_clear(PIT_CH21);
}

void uart0_isr(void)
{
    if (uart_isr_mask(UART_0))
    {
    }
    else
    {
    }
}

void uart1_isr(void)
{
    if (uart_isr_mask(UART_1))
    {

```

```

#if DEBUG_UART_USE_INTERRUPT
    debug_interrupr_handler();
#endif
}
else
{
}
}

void uart2_isr(void)
{
    if (uart_isr_mask(UART_2))
    {
        /* group1(P10) 单线双向方案下闲置：核间链路仅用 group0(UART_1/P04)。 */
    }
    else
    {
    }
}

void uart3_isr(void)
{
    if (uart_isr_mask(UART_3))
    {
    }
    else
    {
    }
}

void uart4_isr(void)
{
    if (uart_isr_mask(UART_4))
    {
    }
    else
    {
    }
}

void uart5_isr(void)
{
    if (uart_isr_mask(UART_5))
    {
        core3LinkCore3RxIsr();
    }
    else
    {
    }
}

void uart6_isr(void)
{
    if (uart_isr_mask(UART_6))
    {
    }
    else
    {
    }
}

void gpio_0_exti_isr()
{
}

void gpio_1_exti_isr()
{
    if (exti_flag_get(P01_0))
    {
    }
    if (exti_flag_get(P01_1))
    {
    }
}

void gpio_2_exti_isr()
{
    if (exti_flag_get(P02_0))
    {
    }
}

```

```

    if (exti_flag_get(P02_4))
    {
    }
}

void gpio_3_exti_isr()
{
}

void gpio_4_exti_isr()
{
}

void gpio_5_exti_isr()
{
}

void gpio_6_exti_isr()
{
}

void gpio_7_exti_isr()
{
}

void gpio_8_exti_isr()
{
}

void gpio_9_exti_isr()
{
}

void gpio_10_exti_isr()
{
}

void gpio_11_exti_isr()
{
}

void gpio_12_exti_isr()
{
}

void gpio_13_exti_isr()
{
}

void gpio_14_exti_isr()
{
}

void gpio_15_exti_isr()
{
}

void gpio_16_exti_isr()
{
}

void gpio_17_exti_isr()
{
}

void gpio_18_exti_isr()
{
}

void gpio_19_exti_isr()
{
}

void gpio_20_exti_isr()
{
}

void gpio_21_exti_isr()

```

```
{
}

void gpio_22_exti_isr()
{
}

void gpio_23_exti_isr()
{
}
```

core3_cyt4bb7/project/user/cm7_1_isr.c

SHA-256

:

cf6adeb3e70bb87c7af14269a4d0ea7e11ea67061a86beb8e93b6826254be65f

代码（c）

```
/*
*****
* CYT4BB Opensourec Library 即（CYT4BB 开源库）是一个基于官方 SDK 接口的第三方开源库
* Copyright (c) 2022 SEEKFREE 逐飞科技
*
* 本文件是 CYT4BB 开源库的一部分
*
* CYT4BB 开源库 是免费软件
* 您可以根据自由软件基金会发布的 GPL（GNU General Public License，即 GNU 通用公共许可证）的条款
* 即 GPL 的第 3 版（即 GPL3.0）或（您选择的）任何后来的版本，重新发布和/或修改它
*
* 本开源库的发布是希望它能发挥作用，但并未对其作任何的保证
* 甚至没有隐含的适销性或适合特定用途的保证
* 更多细节请参见 GPL
*
* 您应该在收到本开源库的同时收到一份 GPL 的副本
* 如果没有，请参阅<https://www.gnu.org/licenses/>
*
* 额外注明：
* 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
* 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
* 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
* 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明（即本声明）
*
* 文件名称      cm7_1_isr
* 公司名称      成都逐飞科技有限公司
* 版本信息      查看 libraries/doc 文件夹内 version 文件 版本说明
* 开发环境      IAR 9.40.1
* 适用平台      CYT4BB
* 店铺链接      https://seekfree.taobao.com/
*
* 修改记录
* 日期          作者          备注
* 2024-1-9      pudding      first version
* 2024-5-14      pudding      新增 12 个 pit 周期中断 增加部分注释说明
* 2025-2-4      pudding      优化串口中断逻辑，防止意外干扰导致的卡死问题，优化串口波特率计算逻辑
* 2025-2-4      pudding      新增两个串口接口
*****
*/

#include "zf_common_headfile.h"

#include "core1Loop.h"

// ***** PIT 中断函数 *****
void pit0_ch0_isr() // 定时器通道 0 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH0);
}

void pit0_ch1_isr() // 定时器通道 1 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH1);
}

void pit0_ch2_isr() // 定时器通道 2 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH2);
}
```

```

    pitProcess0(); // 菜单 10ms tick(menuKeyScan 远程按键 + 刷新计数)
}

void pit0_ch10_isr() // 定时器通道 10 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH10);
}

void pit0_ch11_isr() // 定时器通道 11 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH11);
}

void pit0_ch12_isr() // 定时器通道 12 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH12);
}

void pit0_ch13_isr() // 定时器通道 13 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH13);
}

void pit0_ch14_isr() // 定时器通道 14 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH14);
}

void pit0_ch15_isr() // 定时器通道 15 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH15);
}

void pit0_ch16_isr() // 定时器通道 16 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH16);
}

void pit0_ch17_isr() // 定时器通道 17 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH17);
}

void pit0_ch18_isr() // 定时器通道 18 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH18);
}

void pit0_ch19_isr() // 定时器通道 19 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH19);
}

void pit0_ch20_isr() // 定时器通道 20 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH20);
}

void pit0_ch21_isr() // 定时器通道 21 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH21);
}
// ***** PIT 中断函数 *****

// ***** 串口中断函数 *****
// 串口 0 默认作为调试串口
void uart0_isr(void)
{
    if (uart_isr_mask(UART_0)) // 串口 0 接收中断
    {
    }
    else // 串口 0 发送中断
    {
    }
}

void uart1_isr(void)
{

```

```

        if (uart_isr_mask(UART_1)) // 串口 1 接收中断
        {
#ifdef DEBUG_UART_USE_INTERRUPT
            debug_interrupr_handler();
#endif
        }
        else // 串口 1 发送中断
        {
        }
    }

void uart2_isr(void)
{
    if (uart_isr_mask(UART_2)) // 串口 2 接收中断
    {
    }
    else // 串口 2 发送中断
    {
    }
}

void uart3_isr(void)
{
    if (uart_isr_mask(UART_3)) // 串口 3 接收中断
    {
    }
    else // 串口 3 发送中断
    {
    }
}

void uart4_isr(void)
{
    if (uart_isr_mask(UART_4)) // 串口 4 接收中断
    {
    }
    else // 串口 4 发送中断
    {
    }
}

void uart5_isr(void)
{
    if (uart_isr_mask(UART_5)) // 串口 5 接收中断
    {
    }
    else // 串口 5 发送中断
    {
    }
}

void uart6_isr(void)
{
    if (uart_isr_mask(UART_6)) // 串口 6 接收中断
    {
    }
    else // 串口 6 发送中断
    {
    }
}
// ***** 串口中断函数 *****

// ***** 外部中断函数 *****
void gpio_0_exti_isr() // 外部 GPIO_0 中断服务函数
{
}

void gpio_1_exti_isr() // 外部 GPIO_1 中断服务函数
{
    if (exti_flag_get(P01_0)) // 示例 P1_0 端口外部中断判断
    {
    }
    if (exti_flag_get(P01_1))
    {
    }
}

void gpio_2_exti_isr() // 外部 GPIO_2 中断服务函数
{
}

```

```

    if (exti_flag_get(P02_0))
    {
    }
    if (exti_flag_get(P02_4))
    {
    }
}

void gpio_3_exti_isr() // 外部 GPIO_3 中断服务函数
{
}

void gpio_4_exti_isr() // 外部 GPIO_4 中断服务函数
{
}

void gpio_5_exti_isr() // 外部 GPIO_5 中断服务函数
{
}

void gpio_6_exti_isr() // 外部 GPIO_6 中断服务函数
{
}

void gpio_7_exti_isr() // 外部 GPIO_7 中断服务函数
{
}

void gpio_8_exti_isr() // 外部 GPIO_8 中断服务函数
{
}

void gpio_9_exti_isr() // 外部 GPIO_9 中断服务函数
{
}

void gpio_10_exti_isr() // 外部 GPIO_10 中断服务函数
{
}

void gpio_11_exti_isr() // 外部 GPIO_11 中断服务函数
{
}

void gpio_12_exti_isr() // 外部 GPIO_12 中断服务函数
{
}

void gpio_13_exti_isr() // 外部 GPIO_13 中断服务函数
{
}

void gpio_14_exti_isr() // 外部 GPIO_14 中断服务函数
{
}

void gpio_15_exti_isr() // 外部 GPIO_15 中断服务函数
{
}

void gpio_16_exti_isr() // 外部 GPIO_16 中断服务函数
{
}

void gpio_17_exti_isr() // 外部 GPIO_17 中断服务函数
{
}

void gpio_18_exti_isr() // 外部 GPIO_18 中断服务函数
{
}

void gpio_19_exti_isr() // 外部 GPIO_19 中断服务函数
{
}

void gpio_20_exti_isr() // 外部 GPIO_20 中断服务函数
{
}

```

```
}

void gpio_21_exti_isr() // 外部 GPIO_21 中断服务函数
{
}

void gpio_22_exti_isr() // 外部 GPIO_22 中断服务函数
{
}

void gpio_23_exti_isr() // 外部 GPIO_23 中断服务函数
{
}

// ***** 串口中断函数 *****
```

core3_cyt4bb7/project/user/main_cm7_0.c

SHA-256

:

68b421c6f4fb20c0361f25c4d3b1b4035b4102af3991067b4532706b8f738421

代码 (c)

```
/* *****
 * CYT4BB Opensourec Library 即 ( CYT4BB 开源库) 是一个基于官方 SDK 接口的第三方开源库
 * Copyright (c) 2022 SEEKFREE 逐飞科技
 *
 * 本文件是 CYT4BB 开源库的一部分
 *
 * CYT4BB 开源库 是免费软件
 * 您可以根据自由软件基金会发布的 GPL (GNU General Public License, 即 GNU 通用公共许可证) 的条款
 * 即 GPL 的第 3 版 (即 GPL3.0) 或 (您选择的) 任何后来的版本, 重新发布和/或修改它
 *
 * 本开源库的发布是希望它能发挥作用, 但并未对其作任何的保证
 * 甚至没有隐含的适销性或适合特定用途的保证
 * 更多细节请参见 GPL
 *
 * 您应该在收到本开源库的同时收到一份 GPL 的副本
 * 如果没有, 请参阅<https://www.gnu.org/licenses/>
 *
 * 额外注明:
 * 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
 * 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
 * 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
 * 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明 (即本声明)
 *
 * 文件名称      main_cm7_0
 * 公司名称      成都逐飞科技有限公司
 * 版本信息      查看 libraries/doc 文件夹内 version 文件 版本说明
 * 开发环境      IAR 9.40.1
 * 适用平台      CYT4BB
 * 店铺链接      https://seekfree.taobao.com/
 *
 * 修改记录
 * 日期          作者          备注
 * 2024-1-4      pudding      first version
 * *****/

#include "zf_common_headfile.h"

#include "core0Loop.h"

// 打开新的工程或者工程移动了位置务必执行以下操作
// 第一步 关闭上面所有打开的文件
// 第二步 project->clean 等待下方进度条走完

// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设

// ***** 代码区域 *****

/* =====
 * BEFORE EDITING THIS CORE — READ THESE TWO DOCS FIRST:
 * 1. <repo-root>/CLAUDE.md — multi-core architecture, naming rules, git workflow
```



```
* 2. ./project/code/PROJECT.md    — this core's modules, boundaries, hazards
* Each core has the SAME filenames (motor.c / IMU.c / PID.c) but DIFFERENT semantics.
* Do NOT copy patterns across cores. Do NOT edit two cores in one change.
* ===== */
```

```
int main(void)
{
    clock_init(SYSTEM_CLOCK_250M); // 时钟配置及系统初始化<务必保留>
    debug_init();                  // 调试串口信息初始化
    // 此处编写用户代码 例如外设初始化代码等

    processInit();

    // 此处编写用户代码 例如外设初始化代码等
    while (true)
    {
        // 此处编写需要循环执行的代码

        whileProcess();

        // 此处编写需要循环执行的代码
    }
}

// ***** 代码区域 *****
```

core3_cyt4bb7/project/user/main_cm7_1.c

SHA-256

:

363e25e84c266e455ba510f21d00be6a76c6b61238c8b3cf827431da3e8084e8

代码 (c)

```
/* *****
 * CYT4BB Opensourec Library 即（CYT4BB 开源库）是一个基于官方 SDK 接口的第三方开源库
 * Copyright (c) 2022 SEEKFREE 逐飞科技
 *
 * 本文件是 CYT4BB 开源库的一部分
 *
 * CYT4BB 开源库 是免费软件
 * 您可以根据自由软件基金会发布的 GPL（GNU General Public License，即 GNU 通用公共许可证）的条款
 * 即 GPL 的第 3 版（即 GPL3.0）或（您选择的）任何后来的版本，重新发布和/或修改它
 *
 * 本开源库的发布是希望它能发挥作用，但并未对其作任何的保证
 * 甚至没有隐含的适销性或适合特定用途的保证
 * 更多细节请参见 GPL
 *
 * 您应该在收到本开源库的同时收到一份 GPL 的副本
 * 如果没有，请参阅<https://www.gnu.org/licenses/>
 *
 * 额外注明：
 * 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
 * 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
 * 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
 * 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明（即本声明）
 *
 * 文件名称      main_cm7_1
 * 公司名称      成都逐飞科技有限公司
 * 版本信息      查看 libraries/doc 文件夹内 version 文件 版本说明
 * 开发环境      IAR 9.40.1
 * 适用平台      CYT4BB
 * 店铺链接      https://seekfree.taobao.com/
 *
 * 修改记录
 * 日期      作者      备注
 * 2024-1-4    pudding    first version
 * ***** */
```

```
#include "zf_common_headfile.h"
```

```
#include "core1Loop.h"
```

```
// 打开新的工程或者工程移动了位置务必执行以下操作
// 第一步 关闭上面所有打开的文件
// 第二步 project->clean 等待下方进度条走完
```

```
// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设

// ***** 代码区域 *****

/* =====
* BEFORE EDITING THIS CORE — READ THESE TWO DOCS FIRST:
* 1. <repo-root>/CLAUDE.md — multi-core architecture, naming rules, git workflow
* 2. ./project/code/PROJECT.md — this core's modules, boundaries, hazards
* Each core has the SAME filenames (motor.c / IMU.c / PID.c) but DIFFERENT semantics.
* Do NOT copy patterns across cores. Do NOT edit two cores in one change.
* ===== */
int main(void)
{
    clock_init(SYSTEM_CLOCK_250M); // 时钟配置及系统初始化<务必保留>
    debug_info_init(); // 调试串口信息初始化

    // 此处编写用户代码 例如外设初始化代码等

    processInit();

    // 此处编写用户代码 例如外设初始化代码等
    while (true)
    {
        // 此处编写需要循环执行的代码

        whileProcess();

        // 此处编写需要循环执行的代码
    }
}

// ***** 代码区域 *****
```

core4_cyt4bb7

core4_cyt4bb7/project/code/app/core0Loop.c

SHA-256

:

6e79b7257334190f146a14951fa121570066a7c0e268e081210b8796998d6aad

代码 (c)

```
#include "core0Loop.h"

#include <string.h>

#include "core4CoreDataTransfer.h"
#include "core4LinkCore4.h"
#include "configCore4Runtime.h"
#include "zf_driver_pit.h"

static volatile uint32_t g_linkTickMs;
static core4CoreLinkSnapshot g_linkShared;
static uint32_t g_visionGeneration;
static uint32_t g_visionPublishSeq;
static uint32_t g_visionPublishTickMs;
static uint32_t g_lastVisionProbeTickMs;
static uint32_t g_lastLinkPublishTickMs;
static uint8_t g_visionSharedFresh;
static uint8_t g_wirePaused;
static uint8_t g_forceVisionReadyReplay;
static uint32_t g_visionReadySeq;
static uint32_t g_appliedSeq;
static uint32_t g_planningSeq;
static uint32_t g_healthSeq;
static uint32_t g_paramGlobalRequestSeq;
static uint32_t g_paramGlobalStatePublishSeq;
static uint32_t g_paramGlobalStateConsumedSeq;
static core4CoreIngressEvent g_pendingIngress;
static uint8_t g_pendingIngressValid;
static core4CoreEgressEvent g_pendingEgress;
static uint8_t g_pendingEgressValid;
```

```

static core4CoreBorderRejection g_pendingBorderRejection;
static uint8_t g_pendingBorderRejectionValid;

static uint8_t paramGlobalTerminalAwaitingConsume(void)
{
    return (uint8_t) ((g_linkShared.paramGlobalStateSeq != 0u) &&
        (g_paramGlobalStateConsumedSeq !=
            g_linkShared.paramGlobalStateSeq) &&
        (paramSyncGlobalStateIsValid(&g_linkShared.paramGlobalState) != 0u) &&
        ((g_linkShared.paramGlobalState.flags &
            PARAM_SYNC_GLOBAL_FLAG_TERMINAL) != 0u));
}

static void clearSharedParamGlobalState(void)
{
    memset(&g_linkShared.paramGlobalState, 0,
        sizeof(g_linkShared.paramGlobalState));
    g_linkShared.paramGlobalStateSeq = 0u;
}

static void clearSessionPending(void)
{
    memset(&g_pendingIngress, 0, sizeof(g_pendingIngress));
    memset(&g_pendingEgress, 0, sizeof(g_pendingEgress));
    memset(&g_pendingBorderRejection, 0,
        sizeof(g_pendingBorderRejection));
    g_pendingIngressValid = 0u;
    g_pendingEgressValid = 0u;
    g_pendingBorderRejectionValid = 0u;
}

static uint8_t mergePendingBorderRejections(
    observerLinkPlanningTrackSnapshot *snapshot)
{
    if ((snapshot == NULL) || (g_pendingBorderRejectionValid == 0u)) return 0u;
    if ((snapshot->ownerSession != g_pendingBorderRejection.ownerSession) ||
        (snapshot->revision != g_pendingBorderRejection.revision) ||
        (snapshot->hash != g_pendingBorderRejection.hash))
        return 0u;
    return core4CoreBorderRejectionMerge(&g_pendingBorderRejection, snapshot);
}

static void commitPendingBorderRejections(uint8_t delivered)
{
    if ((delivered == 0u) ||
        (delivered > g_pendingBorderRejection.rejectedCount)) return;
    g_pendingBorderRejection.rejectedCount = (uint8_t)
        (g_pendingBorderRejection.rejectedCount - delivered);
    if (g_pendingBorderRejection.rejectedCount == 0u)
    {
        memset(&g_pendingBorderRejection, 0,
            sizeof(g_pendingBorderRejection));
        g_pendingBorderRejectionValid = 0u;
        return;
    }
    memmove(g_pendingBorderRejection.rejectedTrackIds,
        &g_pendingBorderRejection.rejectedTrackIds[delivered],
        (size_t) g_pendingBorderRejection.rejectedCount * sizeof(uint16_t));
}

static void publishLinkSnapshot(uint32_t tickMs, uint8_t force)
{
    observerLinkSharedConfig staged;
    core4LinkCore4Status status;

    if ((force == 0u) && (g_lastLinkPublishTickMs == tickMs)) return;
    g_lastLinkPublishTickMs = tickMs;
    core4LinkCore4GetStatus(&status);
    if ((status.ownerSession != g_linkShared.status.ownerSession) ||
        (status.bindingEpoch != g_linkShared.status.bindingEpoch))
    {
        clearSessionPending();
        core4CoreDataResetIngress(g_linkShared.bootGeneration);
        if (g_visionGeneration != 0u)
            core4CoreDataSyncEgressConsumer(g_visionGeneration);
    }
    g_linkShared.status = status;
    if (paramGlobalTerminalAwaitingConsume() == 0u)
    {

```

```

paramSyncGlobalState state;
if (core4LinkCore4TakeParamGlobalState(&state) != 0u)
{
    g_linkShared.paramGlobalState = state;
    g_paramGlobalStatePublishSeq++;
    if (g_paramGlobalStatePublishSeq == 0u)
    {
        g_paramGlobalStatePublishSeq = 1u;
        g_paramGlobalStateConsumedSeq = 0u;
    }
    g_linkShared.paramGlobalStateSeq =
        g_paramGlobalStatePublishSeq;
}
}
if (((g_wirePaused != 0u) ||
    (core4LinkCore4IsParamGlobalStateValid() == 0u)) &&
    (paramGlobalTerminalAwaitingConsume() == 0u))
    clearSharedParamGlobalState();
if (g_wirePaused != 0u)
{
    uint32_t localSession = g_linkShared.status.localSession;
    memset(&g_linkShared.status, 0, sizeof(g_linkShared.status));
    g_linkShared.status.localSession = localSession;
    g_linkShared.status.stagedMode = 0xFFu;
    g_linkShared.status.appliedMode = 0xFFu;
    memset(&g_linkShared.stagedConfig, 0, sizeof(g_linkShared.stagedConfig));
    g_linkShared.stagedConfigSeq = 0u;
    if (paramGlobalTerminalAwaitingConsume() == 0u)
        clearSharedParamGlobalState();
}
else if (core4LinkCore4GetStagedConfig(&staged) != 0u)
{
    if ((g_linkShared.stagedConfigSeq == 0u) ||
        (memcmp(&staged, &g_linkShared.stagedConfig, sizeof(staged)) != 0))
    {
        g_linkShared.stagedConfig = staged;
        g_linkShared.stagedConfigSeq++;
    }
}
else
{
    memset(&g_linkShared.stagedConfig, 0, sizeof(g_linkShared.stagedConfig));
    g_linkShared.stagedConfigSeq = 0u;
}
g_linkShared.publishSeq++;
g_linkShared.beginGeneration += 2u;
if (g_linkShared.beginGeneration == 0u) g_linkShared.beginGeneration = 2u;
g_linkShared.endGeneration = g_linkShared.beginGeneration;
core4CoreDataWriteLink(&g_linkShared);
}

static void consumeVisionSnapshot(void)
{
    core4CoreVisionSnapshot vision;
    uint32_t bootGeneration;
    uint32_t publishSeq;

    if (g_lastVisionProbeTickMs != g_linkTickMs)
    {
        g_lastVisionProbeTickMs = g_linkTickMs;
        if ((core4CoreDataProbeVision(&bootGeneration, &publishSeq) != 0u) &&
            ((bootGeneration != g_visionGeneration) ||
             (publishSeq != g_visionPublishSeq)) &&
            (core4CoreDataTryReadVision(&vision) != 0u))
        {
            if (vision.bootGeneration != g_visionGeneration)
            {
                uint32_t previousGeneration = g_visionGeneration;
                core4CoreDataSyncEgressConsumer(vision.bootGeneration);
                g_visionGeneration = vision.bootGeneration;
                g_visionPublishSeq = 0u;
                g_visionReadySeq = 0u;
                g_appliedSeq = 0u;
                g_planningSeq = 0u;
                g_healthSeq = 0u;
                g_paramGlobalRequestSeq = 0u;
                g_paramGlobalStateConsumedSeq = 0u;
                clearSessionPending();
                g_forceVisionReadyReplay = 1u;
            }
        }
    }
}

```

```

    g_wirePaused = 0u;
    if (previousGeneration != 0u)
    {
        core4CoreDataNoteSessionReset();
        core4LinkCore4Init(g_linkTickMs);
    }
}
if (vision.publishSeq != g_visionPublishSeq)
{
    if (g_visionSharedFresh == 0u)
    {
        clearSessionPending();
        g_forceVisionReadyReplay = 1u;
        g_wirePaused = 0u;
    }
    g_visionPublishSeq = vision.publishSeq;
    g_visionPublishTickMs = g_linkTickMs;
    g_visionSharedFresh = 1u;
}
if (g_visionSharedFresh != 0u)
{
    if ((g_forceVisionReadyReplay != 0u) ||
        (vision.visionReadySeq != g_visionReadySeq))
    {
        core4LinkCore4SetVisionReady(vision.visionReady);
        g_visionReadySeq = vision.visionReadySeq;
        g_forceVisionReadyReplay = 0u;
    }
    if ((vision.appliedSeq != 0u) &&
        (vision.appliedSeq != g_appliedSeq) &&
        (core4LinkCore4MarkConfigApplied(vision.appliedOwnerSession,
            vision.appliedRevision, vision.appliedHash,
            vision.appliedMode) != 0u))
    {
        g_appliedSeq = vision.appliedSeq;
        g_linkShared.appliedConsumedSeq = g_appliedSeq;
    }
    if ((vision.planningSeq != 0u) &&
        (vision.planningSeq != g_planningSeq))
    {
        uint8_t delivered;
        /* wire 时间戳在 cm7_0 域重锚；相机真实新鲜度由 HEALTH seq/age 独立证明。*/
        vision.planningSnapshot.captureTickMs = g_linkTickMs;
        delivered = mergePendingBorderRejections(
            &vision.planningSnapshot);
        if (core4LinkCore4QueuePlanningTrackSnapshot(
            &vision.planningSnapshot) != 0u)
        {
            g_planningSeq = vision.planningSeq;
            commitPendingBorderRejections(delivered);
        }
    }
    if ((vision.healthSeq != 0u) &&
        (vision.healthSeq != g_healthSeq) &&
        (core4LinkCore4PublishHealth(&vision.health) != 0u))
    {
        g_healthSeq = vision.healthSeq;
    }
    if ((vision.paramGlobalRequestSeq != 0u) &&
        (vision.paramGlobalRequestSeq != g_paramGlobalRequestSeq) &&
        (core4LinkCore4QueueParamGlobalRequest(
            &vision.paramGlobalRequest) != 0u))
    {
        g_paramGlobalRequestSeq = vision.paramGlobalRequestSeq;
    }
    if ((vision.paramGlobalStateConsumedSeq != 0u) &&
        (vision.paramGlobalStateConsumedSeq ==
            g_linkShared.paramGlobalStateSeq))
    {
        g_paramGlobalStateConsumedSeq =
            vision.paramGlobalStateConsumedSeq;
    }
}
}
}
if ((g_visionSharedFresh != 0u) &&
    ((uint32_t) (g_linkTickMs - g_visionPublishTickMs) >=
        CORE4_CORE_SHARED_STALE_MS))
{

```

```

    g_visionSharedFresh = 0u;
    g_wirePaused = 1u;
    g_forceVisionReadyReplay = 1u;
    clearSessionPending();
    core4LinkCore4SetVisionReady(0u);
    core4CoreDataNoteSharedStale();
}
}

static void forwardKvmIngress(void)
{
    if ((g_pendingIngressValid != 0u) &&
        (core4CoreDataPushIngress(&g_pendingIngress) != 0u))
    {
        g_pendingIngressValid = 0u;
    }
    if (g_pendingIngressValid != 0u) return;
    memset(&g_pendingIngress, 0, sizeof(g_pendingIngress));
    if (core4LinkCore4TakeKvmControl(
        &g_pendingIngress.payload.control) != 0u)
    {
        g_pendingIngress.kind = CORE4_CORE_INGRESS_KVM_CONTROL;
        g_pendingIngressValid = 1u;
    }
    else if (core4LinkCore4TakeKvmEvent(
        &g_pendingIngress.payload.event) != 0u)
    {
        g_pendingIngress.kind = CORE4_CORE_INGRESS_KVM_EVENT;
        g_pendingIngressValid = 1u;
    }
    if ((g_pendingIngressValid != 0u) &&
        (core4CoreDataPushIngress(&g_pendingIngress) != 0u))
    {
        g_pendingIngressValid = 0u;
    }
}

static uint8_t retainBorderRejection(
    const core4CoreBorderRejection *event)
{
    return core4CoreBorderRejectionAppend(&g_pendingBorderRejection,
        &g_pendingBorderRejectionValid, event);
}

static void consumeKvmEgress(void)
{
    uint8_t accepted = 0u;
    if (g_pendingEgressValid == 0u)
    {
        g_pendingEgressValid = core4CoreDataPopEgress(&g_pendingEgress,
            g_visionGeneration);
    }
    if (g_pendingEgressValid == 0u) return;
    if (g_pendingEgress.kind == CORE4_CORE_EGRESS_KVM_ACK)
    {
        accepted = core4LinkCore4QueueKvmAck(&g_pendingEgress.payload.ack);
    }
    else if (g_pendingEgress.kind == CORE4_CORE_EGRESS_KVM_MENU)
    {
        accepted = core4LinkCore4QueueKvmMenuStatus(
            &g_pendingEgress.payload.menuStatus);
    }
    else if (g_pendingEgress.kind == CORE4_CORE_EGRESS_BORDER_REJECTION)
    {
        accepted = retainBorderRejection(
            &g_pendingEgress.payload.borderRejection);
    }
    else
    {
        accepted = 1u;
    }
    if (accepted != 0u) g_pendingEgressValid = 0u;
}

void processInit(void)
{
    core4CoreDataTimingInit();
    memset(&g_linkShared, 0, sizeof(g_linkShared));
    g_linkShared.magic = CORE4_CORE_LINK_MAGIC;
}

```

```

g_linkShared.abiVersion = CORE4_CORE_ABI_VERSION;
g_linkShared.payloadBytes = sizeof(g_linkShared);
g_linkShared.bootGeneration = core4CoreDataMakeBootGeneration(0xC4700001u);
g_lastVisionProbeTickMs = UINT32_MAX;
g_lastLinkPublishTickMs = UINT32_MAX;
g_wirePaused = 1u;
g_forceVisionReadyReplay = 1u;
g_paramGlobalStatePublishSeq = 0u;
g_paramGlobalStateConsumedSeq = 0u;
core4CoreDataResetIngress(g_linkShared.bootGeneration);
g_linkShared.beginGeneration = 2u;
g_linkShared.endGeneration = 2u;
core4LinkCore4Init(0u);
publishLinkSnapshot(0u, 1u);
pit_ms_init(PIT_CH0, CORE4_RUNTIME_PIT_MS);
}

void pitProcess0(void)
{
    g_linkTickMs += CORE4_RUNTIME_PIT_MS;
}

void whileProcess(void)
{
    uint32_t tickMs = g_linkTickMs;
    if (g_wirePaused == 0u) consumeKvmEgress();
    consumeVisionSnapshot();
    if (g_wirePaused == 0u)
    {
        consumeKvmEgress();
        core4LinkCore4Poll(tickMs);
        forwardKvmIngress();
        core4LinkCore4FlushControl(tickMs);
        core4LinkCore4FlushTx(tickMs);
    }
    publishLinkSnapshot(tickMs, 0u);
}

```

core4_cyt4bb7/project/code/app/core0Loop.h

SHA-256

:

ba1b962e1e8b87dd6c2efb47ffd5c650eb1a53829c8779fa68d196435e0a51c8

代码 (c)

```

#ifndef CORE4_CORE0_LOOP_H
#define CORE4_CORE0_LOOP_H

void processInit(void);
void whileProcess(void);
void pitProcess0(void);

#endif /* CORE4_CORE0_LOOP_H */

```

core4_cyt4bb7/project/code/app/core1Loop.c

SHA-256

:

f8c58595e5547f579b1ac2361a35867b4c437d64a5c339605684cb39f4c2a70a

代码 (c)

```

#include "core1Loop.h"

#include "core4Runtime.h"
#include "paramSyncOwnerService.h"

void processInit(void)
{
    core4ParamSyncOwnerServiceInit();
    core4RuntimeInit();
}

```

```

}

void pitProcess0(void)
{
    core4RuntimeTick10msIsr();
}

void whileProcess(void)
{
    core4ParamSyncOwnerServicePoll();
    core4RuntimeProcess();
}

```

core4_cyt4bb7/project/code/app/core1Loop.h

SHA-256

:

a599e587142bb105a5a9c2af0c3e9adcc56659f2d0a9d928bb085ef77124e2e6

代码 (c)

```

#ifndef CORE4_CORE1_LOOP_H
#define CORE4_CORE1_LOOP_H

void processInit(void);
void whileProcess(void);
void pitProcess0(void);

#endif /* CORE4_CORE1_LOOP_H */

```

core4_cyt4bb7/project/code/app/core4CameraService.c

SHA-256

:

618288ba7d26459d1192e196bc1d3b15001dc5e93173c20ec1807727863a78f2

代码 (c)

```

#include "core4CameraService.h"

#include "cy_device_headers.h"
#include "configCore4Runtime.h"
#include "zf_device_mt9v03x.h"

_Static_assert(MT9V03X_W == CORE4_CAMERA_WIDTH, "Core4 camera width must remain 188");
_Static_assert(MT9V03X_H == CORE4_CAMERA_HEIGHT, "Core4 camera height must remain 120");
_Static_assert(MT9V03X_FPS_DEF == 50, "Core4 camera capture must remain 50fps");

#define CORE4_CAMERA_INT_VALID_MASK (1u << 31)
#define CORE4_CAMERA_CYT2_EQUIVALENT_EXPOSURE (500u)

static volatile uint32_t *cameraSystemIntControl(void)
{
    return &CPUSS->unCM7_1_SYSTEM_INT_CTL[
        tcpwm_0_interrupts_59_IRQn].u32Register;
}

static uint32_t cameraInterruptMask(void)
{
    volatile uint32_t *control = cameraSystemIntControl();
    uint32_t saved = *control;
    *control = saved & ~CORE4_CAMERA_INT_VALID_MASK;
    (void) *control;
    __DSB();
    return saved;
}

static void cameraInterruptRestore(uint32_t saved)
{
    volatile uint32_t *control = cameraSystemIntControl();
    *control = saved;
    (void) *control;
}

```



```

    __DSB();
}

static uint8_t g_appImage[CORE4_CAMERA_HEIGHT][CORE4_CAMERA_WIDTH];
static uint32_t g_appFrameSeq;
static uint8_t g_initialized;

uint8_t core4CameraServiceInit(void)
{
    uint8_t result = mt9v03x_init();
    uint32_t savedIntControl;

    if ((result == 0u) &&
        (mt9v03x_set_exposure_time(
            CORE4_CAMERA_CYT2_EQUIVALENT_EXPOSURE) != 0u))
    {
        result = 1u;
    }

    savedIntControl = cameraInterruptMask();
    mt9v03x_finish_flag = 0u;
    cameraInterruptRestore(savedIntControl);
    g_initialized = (uint8_t) (result == 0u);

    return result;
}

uint8_t core4CameraServiceSetExposure(uint16_t exposure)
{
    if (g_initialized == 0u)
    {
        return 1u;
    }
    return mt9v03x_set_exposure_time(exposure);
}

uint8_t core4CameraServiceTryCapture(uint32_t tickMs,
    uint8_t timingValid,
    core4CameraSnapshot *snapshot,
    uint32_t *copyCycles)
{
    uint8_t captured = 0u;
    uint32_t startCycles = 0u;
    uint32_t endCycles = 0u;
    uint32_t savedIntControl;

    if ((snapshot == NULL) || (copyCycles == NULL) || (mt9v03x_finish_flag == 0u))
    {
        return 0u;
    }

    savedIntControl = cameraInterruptMask();

    if (timingValid != 0u)
    {
        startCycles = DWT->CYCCNT;
    }

    if (mt9v03x_finish_flag != 0u)
    {
        uint16_t y;
        mt9v03x_finish_flag = 0u;
        for (y = 0u; y < CORE4_CAMERA_HEIGHT; y++)
        {
            uint16_t x;
            for (x = 0u; x < CORE4_CAMERA_WIDTH; x++)
            {
                g_appImage[y][x] = mt9v03x_image
                    [CORE4_CAMERA_HEIGHT - 1u - y]
                    [CORE4_CAMERA_WIDTH - 1u - x];
            }
        }
        captured = 1u;
    }

    if (timingValid != 0u)
    {
        endCycles = DWT->CYCCNT;
    }
}

```

```

cameraInterruptRestore(savedIntControl);

if (captured != 0u)
{
    g_appFrameSeq++;
    if (g_appFrameSeq == 0u) g_appFrameSeq = 1u;
    snapshot->data = g_appImage[0];
    snapshot->width = CORE4_CAMERA_WIDTH;
    snapshot->height = CORE4_CAMERA_HEIGHT;
    snapshot->appFrameSeq = g_appFrameSeq;
    snapshot->tickMs = tickMs;
    *copyCycles = (timingValid != 0u) ? (endCycles - startCycles) : 0u;
}

return captured;
}

```

core4_cyt4bb7/project/code/app/core4CameraService.h

SHA-256

:

6ba927d5c6a468a8ce772c3be25e42dee99318baf2ba46007f32c5f4b04ca96c

代码 (c)

```

#ifndef CORE4_CAMERA_SERVICE_H
#define CORE4_CAMERA_SERVICE_H

#include <stdint.h>

typedef struct
{
    const uint8_t *data;
    uint16_t width;
    uint16_t height;
    uint32_t appFrameSeq;
    uint32_t tickMs;
} core4CameraSnapshot;

uint8_t core4CameraServiceInit(void);
uint8_t core4CameraServiceSetExposure(uint16_t exposure);
uint8_t core4CameraServiceTryCapture(uint32_t tickMs,
    uint8_t timingValid,
    core4CameraSnapshot *snapshot,
    uint32_t *copyCycles);

#endif /* CORE4_CAMERA_SERVICE_H */

```

core4_cyt4bb7/project/code/app/core4Diagnostics.c

SHA-256

:

988745bba46f638a73d9ea7f515a91b58ff19ab992c80e5cb7320cc30ba89577

代码 (c)

```

#include "core4Diagnostics.h"

#include <stdarg.h>
#include <stdio.h>
#include <string.h>
#include <math.h>

#include "cy_device_headers.h"
#include "configCore4Runtime.h"
#include "pinMap.h"
#include "observerLinkProtocol.h"
#include "core4VisionParams.h"
#include "zf_common_font.h"
#include "zf_device_ips114.h"
#include "zf_driver_pwm.h"

```

```

static uint8_t g_timingValid;
static uint8_t g_rgbReady;
static uint32_t g_lastRedDuty = 0xFFFFFFFFFu;
static uint32_t g_lastGreenDuty = 0xFFFFFFFFFu;
static uint32_t g_lastBlueDuty = 0xFFFFFFFFFu;
#define CORE4_HOME_SLOT_COUNT (15u)
#define CORE4_HOME_SLOT_CHARS (40u)
#define CORE4_MENU_LINE_COUNT (8u)
#define CORE4_MENU_LINE_CHARS (39u)
#define CORE4_MENU_NAME_X (16u)
#define CORE4_MENU_VALUE_X (112u)
#define CORE4_MENU_SCROLL_X (232u)
#define CORE4_THRESHOLD_IMAGE_Y (18u)
#define CORE4_THRESHOLD_IMAGE_W (110u)
#define CORE4_THRESHOLD_IMAGE_H (70u)
#define CORE4_THRESHOLD_LEFT_X (2u)
#define CORE4_THRESHOLD_RIGHT_X (124u)
#define CORE4_THRESHOLD_TEXT_Y (112u)
#define CORE4_SOLID_FILL_PIXELS (240u)
#define CORE4_THRESHOLD_SLOT_COUNT (3u)
#define CORE4_THRESHOLD_SLOT_LOW_X (2u)
#define CORE4_THRESHOLD_SLOT_HIGH_X (82u)
#define CORE4_THRESHOLD_SLOT_EXPOSURE_X (162u)
#define CORE4_THRESHOLD_SLOT_LOW_WIDTH (72u)
#define CORE4_THRESHOLD_SLOT_HIGH_WIDTH (72u)
#define CORE4_THRESHOLD_SLOT_EXPOSURE_WIDTH (76u)
#define CORE4_THRESHOLD_NAME_LOW_X (10u)
#define CORE4_THRESHOLD_NAME_HIGH_X (90u)
#define CORE4_THRESHOLD_NAME_EXPOSURE_X (170u)
#define CORE4_THRESHOLD_VALUE_LOW_X (42u)
#define CORE4_THRESHOLD_VALUE_HIGH_X (122u)
#define CORE4_THRESHOLD_VALUE_EXPOSURE_X (202u)

#define CORE4_MENU_NORM_NAME_FG (RGB565_GREEN)
#define CORE4_MENU_NORM_VAL_FG (RGB565_WHITE)
#define CORE4_MENU_SEL_BG (RGB565_CYAN)
#define CORE4_MENU_SEL_NAME_FG (RGB565_BLACK)
#define CORE4_MENU_SEL_VAL_FG (RGB565_BLUE)
#define CORE4_MENU_EDIT_BG (RGB565_YELLOW)
#define CORE4_MENU_EDIT_NAME_FG (RGB565_BLACK)
#define CORE4_MENU_EDIT_VAL_FG (RGB565_RED)

typedef enum
{
    CORE4_DISPLAY_SURFACE_UNKNOWN = 0,
    CORE4_DISPLAY_SURFACE_HOME,
    CORE4_DISPLAY_SURFACE_WAIT,
    CORE4_DISPLAY_SURFACE_MENU,
    CORE4_DISPLAY_SURFACE_FEEDBACK,
    CORE4_DISPLAY_SURFACE_INIT_FAULT
} core4DisplaySurface;

typedef enum
{
    CORE4_MENU_ROW_NORMAL = 0,
    CORE4_MENU_ROW_SELECTED,
    CORE4_MENU_ROW_EDIT
} core4MenuRowStyle;

static char g_homeSlotCache[CORE4_HOME_SLOT_COUNT][CORE4_HOME_SLOT_CHARS + 1u];
static uint8_t g_homeSlotValid[CORE4_HOME_SLOT_COUNT];
static uint16_t g_homeSlotColor[CORE4_HOME_SLOT_COUNT];
static uint16_t g_homeSlotBackground[CORE4_HOME_SLOT_COUNT];
static char g_menuLineCache[CORE4_MENU_LINE_COUNT][CORE4_MENU_LINE_CHARS + 1u];
static char g_menuValueCache[CORE4_MENU_LINE_COUNT][CORE4_MENU_VALUE_CHARS + 1u];
static uint8_t g_menuLineValid[CORE4_MENU_LINE_COUNT];
static core4MenuRowStyle g_menuLineStyle[CORE4_MENU_LINE_COUNT];
static uint16_t g_menuFillRightX = 238u;
static uint8_t g_scrollValid;
static uint8_t g_scrollTotal;
static uint8_t g_scrollTop;
static uint16_t g_menuFrameColor = 0xFFFFFu;
static core4DisplaySurface g_displaySurface;
static core4MenuState g_lastMenuState;
static uint32_t g_lastMenuRevision;
static uint16_t g_homeFrameColor;
static uint16_t g_homeTitleColor;
static uint32_t g_thresholdOriginalFrameSeq = 0xFFFFFFFFFu;

```

```

static uint32_t g_thresholdPreviewFrameSeq = 0xFFFFFFFFFu;
static uint16_t g_thresholdFrameColor = 0xFFFFu;
static uint8_t g_thresholdMode = 0xFFu;
static uint8_t g_thresholdOriginalValid;
static uint8_t g_thresholdPreviewValid;
static uint16_t g_thresholdSlotValue[CORE4_THRESHOLD_SLOT_COUNT];
static uint8_t g_thresholdSlotSelected[CORE4_THRESHOLD_SLOT_COUNT];
static uint8_t g_thresholdSlotValid[CORE4_THRESHOLD_SLOT_COUNT];
static uint8_t g_lastRgbMode = 0xFFu;
static uint32_t g_modeFlashStartMs;
static uint16_t g_solidFillBuffer[CORE4_SOLID_FILL_PIXELS];

static void fillSolidRect(uint16_t left, uint16_t top,
    uint16_t width, uint16_t height, uint16_t color)
{
    uint16_t rowsPerTransfer;
    uint16_t rows;
    uint16_t y = top;
    uint16_t i;

    if ((width == 0u) || (height == 0u))
        (width > CORE4_SOLID_FILL_PIXELS) return;
    rowsPerTransfer = (uint16_t) (CORE4_SOLID_FILL_PIXELS / width);
    for (i = 0u; i < CORE4_SOLID_FILL_PIXELS; i++)
        g_solidFillBuffer[i] = color;
    while (height > 0u)
    {
        rows = (height < rowsPerTransfer) ? height : rowsPerTransfer;
        ips114_show_rgb565_image(left, y, g_solidFillBuffer,
            width, rows, width, rows, 0u);
        y = (uint16_t) (y + rows);
        height = (uint16_t) (height - rows);
    }
}

static const char *core4DiagnosticsStateShortName(core4RuntimeState state)
{
    switch (state)
    {
        case CORE4_RUNTIME_BOOT:
            return "BOOT";
        case CORE4_RUNTIME_CAMERA_STARTING:
            return "START";
        case CORE4_RUNTIME_RUNNING:
            return "RUN";
        case CORE4_RUNTIME_CAMERA_FAULT:
            return "FAULT";
        default:
            return "UNKNOWN";
    }
}

static const char *core4DiagnosticsReasonName(uint8_t reason)
{
    switch (reason)
    {
        case OBSERVER_LINK_REASON_NONE: return "READY";
        case OBSERVER_LINK_REASON_LOCAL_INIT: return "LOCAL INIT";
        case OBSERVER_LINK_REASON_CORE0_LINK: return "C0 LINK";
        case OBSERVER_LINK_REASON_CORE1_LINK: return "C1 LINK";
        case OBSERVER_LINK_REASON_CORE2_LINK: return "C2 LINK";
        case OBSERVER_LINK_REASON_OBSERVER_LINK: return "OBS LINK";
        case OBSERVER_LINK_REASON_PROTOCOL: return "PROTOCOL";
        case OBSERVER_LINK_REASON_CONFIG: return "CONFIG";
        case OBSERVER_LINK_REASON_CAMERA: return "CAMERA";
        case OBSERVER_LINK_REASON_TARGET: return "NO TARGET";
        case OBSERVER_LINK_REASON_BARRIER: return "BARRIER";
        case OBSERVER_LINK_REASON_DISPATCH: return "DISPATCH";
        case OBSERVER_LINK_REASON_SAFETY: return "SAFETY";
        case OBSERVER_LINK_REASON_LANDING: return "LANDING";
        default: return "UNKNOWN";
    }
}

static const char *core4DiagnosticsControlName(uint8_t state)
{
    return (state == OBSERVER_LINK_CAMERA_CONTROL_SAFE) ? "SAFE" : "RUN";
}

```

```

static const char *core4DiagnosticsSourceName(const core4RuntimeStatus *status)
{
    if (status->controlClass == OBSERVER_LINK_CONTROL_CLASS_SEARCH)
    {
        return "SEARCH";
    }
    if (status->activeSource == OBSERVER_LINK_SOURCE_CORE1)
    {
        return "CORE1";
    }
    if (status->activeSource == OBSERVER_LINK_SOURCE_CORE3)
    {
        return "CORE3";
    }
    if (status->activeSource == OBSERVER_LINK_SOURCE_CORE4)
    {
        return "CORE4";
    }
    return "NONE";
}

```

```

static const char *core4DiagnosticsNavName(uint8_t state)
{
    switch (state)
    {
        case 2u:
            return "TRACK";
        case 3u:
            return "START";
        case 4u:
            return "SEARCH";
        default:
            return "IDLE";
    }
}

```

```

static void core4DiagnosticsFormatQ4(int16_t value, char output[12])
{
    int32_t signedValue = value;
    uint32_t magnitude = (signedValue < 0) ?
        (uint32_t) (-signedValue) : (uint32_t) signedValue;
    uint32_t tenths = (magnitude * 10u + 8u) / 16u;

    (void) snprintf(output, 12u, "%c%lu.%lu",
        (signedValue < 0) ? '-' : '+',
        (unsigned long) (tenths / 10u),
        (unsigned long) (tenths % 10u));
}

```

```

static void formatFixedLine(char *line,
    uint8_t width,
    const char *format,
    va_list args)
{
    char formatted[64u];
    size_t length;

    (void) vsnprintf(formatted, sizeof(formatted), format, args);
    memset(line, '\0', width);
    line[width] = '\0';
    length = strlen(formatted);
    if (length > width)
    {
        length = width;
    }
    memcpy(line, formatted, length);
}

```

```

static void core4DiagnosticsShowHomeSlot(uint8_t slot,
    ips114_font_size_enum font,
    uint16_t x,
    uint16_t y,
    uint8_t width,
    uint16_t color,
    uint16_t background,
    const char *format,
    ...)
{
    char line[CORE4_HOME_SLOT_CHARS + 1u];
}

```

```

va_list args;

if ((slot >= CORE4_HOME_SLOT_COUNT) || (width > CORE4_HOME_SLOT_CHARS))
{
    return;
}
memset(line, 0, sizeof(line));
va_start(args, format);
formatFixedLine(line, width, format, args);
va_end(args);

if ((g_homeSlotValid[slot] == 0u) ||
    (g_homeSlotColor[slot] != color) ||
    (g_homeSlotBackground[slot] != background) ||
    (memcmp(g_homeSlotCache[slot], line, sizeof(line)) != 0))
{
    memcpy(g_homeSlotCache[slot], line, sizeof(line));
    g_homeSlotValid[slot] = 1u;
    g_homeSlotColor[slot] = color;
    g_homeSlotBackground[slot] = background;
    ips114_set_font(font);
    ips114_set_color(color, background);
    ips114_show_string(x, y, line);
}
}

static void fillMenuRow(uint8_t row, uint16_t color)
{
    uint16_t y;

    if (row >= CORE4_MENU_LINE_COUNT)
    {
        return;
    }
    y = (uint16_t)(row * 16u);
    if (row == 0u)
    {
        fillSolidRect(1u, 1u, 238u, 15u, color);
        return;
    }
    fillSolidRect(1u, y, g_menuFillRightX, 16u, color);
    if (row == (CORE4_MENU_LINE_COUNT - 1u))
    {
        fillSolidRect(1u, 128u, g_menuFillRightX, 6u, color);
    }
}

static void drawMenuFrame(uint16_t color)
{
    fillSolidRect(0u, 0u, 240u, 1u, color);
    fillSolidRect(0u, 134u, 240u, 1u, color);
    fillSolidRect(0u, 1u, 1u, 133u, color);
    fillSolidRect(239u, 1u, 1u, 133u, color);
}

static void drawMenuScrollbar(uint8_t total, uint8_t top)
{
    uint16_t trackTop = 16u;
    uint16_t trackBottom = 133u;
    uint16_t trackHeight;
    uint16_t thumbHeight;
    uint16_t thumbY;

    g_menuFillRightX = (total > CORE4_MENU_VISIBLE_ROWS)
        ? (CORE4_MENU_SCROLL_X - 1u)
        : 238u;
    if (total <= CORE4_MENU_VISIBLE_ROWS)
    {
        if (g_scrollValid != 0u)
        {
            fillSolidRect(CORE4_MENU_SCROLL_X, trackTop,
                (uint16_t)(238u - CORE4_MENU_SCROLL_X + 1u),
                (uint16_t)(trackBottom - trackTop + 1u), RGB565_BLACK);
            memset(g_menuLineValid, 0, sizeof(g_menuLineValid));
        }
        g_scrollValid = 0u;
        return;
    }
    if ((g_scrollValid != 0u) &&

```

```

        (g_scrollTotal == total) &&
        (g_scrollTop == top))
    {
        return;
    }

    fillSolidRect(CORE4_MENU_SCROLL_X, trackTop,
        (uint16_t) (238u - CORE4_MENU_SCROLL_X + 1u),
        (uint16_t) (trackBottom - trackTop + 1u), RGB565_BLACK);
    fillSolidRect(CORE4_MENU_SCROLL_X, trackTop, 1u,
        (uint16_t) (trackBottom - trackTop + 1u), RGB565_GREEN);
    fillSolidRect(238u, trackTop, 1u,
        (uint16_t) (trackBottom - trackTop + 1u), RGB565_GREEN);
    trackHeight = (uint16_t) (trackBottom - trackTop + 1u);
    thumbHeight = (uint16_t) (((uint32_t) trackHeight *
        CORE4_MENU_VISIBLE_ROWS) /
        total);
    if (thumbHeight < 6u)
    {
        thumbHeight = 6u;
    }
    thumbY = (uint16_t) (trackTop +
        (((uint32_t) (trackHeight - thumbHeight) * top) /
        (total - CORE4_MENU_VISIBLE_ROWS)));
    if ((uint16_t) (thumbY + thumbHeight - 1u) > trackBottom)
        thumbHeight = (uint16_t) (trackBottom - thumbY + 1u);
    fillSolidRect((uint16_t) (CORE4_MENU_SCROLL_X + 1u), thumbY,
        (uint16_t) (237u - (CORE4_MENU_SCROLL_X + 1u) + 1u),
        thumbHeight, RGB565_GREEN);
    g_scrollTotal = total;
    g_scrollTop = top;
    g_scrollValid = 1u;
}

static void core4DiagnosticsShowMenuTitle(const char *title)
{
    char bounded[40];
    size_t length;
    uint16_t x;

    if (title == NULL)
    {
        title = "";
    }
    length = strlen(title);
    if (length > 39u) length = 39u;
    memcpy(bounded, title, length);
    bounded[length] = '\0';
    if ((g_menuLineValid[0] != 0u) &&
        (strcmp(g_menuLineCache[0], bounded) == 0)) return;
    fillMenuRow(0u, RGB565_GREEN);
    x = (uint16_t) ((240u - (uint16_t) (length * 6u)) / 2u);
    if (x < 2u) x = 2u;
    ips114_set_font(IPS114_6X8_FONT);
    ips114_set_color(RGB565_BLACK, RGB565_GREEN);
    ips114_show_string(x, 4u, bounded);
    ips114_set_font(IPS114_8X16_FONT);
    ips114_set_color(RGB565_GREEN, RGB565_BLACK);
    memset(g_menuLineCache[0], 0, sizeof(g_menuLineCache[0]));
    (void) strncpy(g_menuLineCache[0], bounded, CORE4_MENU_LINE_CHARS);
    g_menuLineValid[0] = 1u;
}

static void core4DiagnosticsShowMenuRow(uint8_t row,
    core4MenuRowStyle style,
    const char *name,
    const char *value)
{
    char signature[CORE4_MENU_LINE_CHARS + 1u];
    char fixedValue[CORE4_MENU_VALUE_CHARS + 1u];
    uint16_t background = RGB565_BLACK;
    uint16_t nameForeground = CORE4_MENU_NORM_NAME_FG;
    uint16_t valueForeground = CORE4_MENU_NORM_VAL_FG;
    char marker = ' ';
    uint8_t structureChanged;
    uint8_t valueChanged;
    size_t valueLength;

    if ((row == 0u) || (row >= CORE4_MENU_LINE_COUNT))

```

```

{
    return;
}
if (name == NULL)
{
    name = "";
}
if (value == NULL)
{
    value = "";
}
if (style == CORE4_MENU_ROW_SELECTED)
{
    background = CORE4_MENU_SEL_BG;
    nameForeground = CORE4_MENU_SEL_NAME_FG;
    valueForeground = CORE4_MENU_SEL_VAL_FG;
    marker = '>';
}
else if (style == CORE4_MENU_ROW_EDIT)
{
    background = CORE4_MENU_EDIT_BG;
    nameForeground = CORE4_MENU_EDIT_NAME_FG;
    valueForeground = CORE4_MENU_EDIT_VAL_FG;
    marker = '*';
}

(void) snprintf(signature, sizeof(signature), "%c%.11s", marker, name);
memset(fixedValue, ' ', CORE4_MENU_VALUE_CHARS);
fixedValue[CORE4_MENU_VALUE_CHARS] = '\0';
valueLength = strlen(value);
if (valueLength > CORE4_MENU_VALUE_CHARS)
{
    valueLength = CORE4_MENU_VALUE_CHARS;
}
memcpy(fixedValue, value, valueLength);
structureChanged = (uint8_t)((g_menuLineValid[row] == 0u) ||
    (g_menuLineStyle[row] != style) ||
    (strcmp(g_menuLineCache[row],
        signature,
        CORE4_MENU_LINE_CHARS) != 0));
valueChanged = (uint8_t)((g_menuLineValid[row] == 0u) ||
    (strcmp(g_menuValueCache[row],
        fixedValue,
        CORE4_MENU_VALUE_CHARS) != 0));

if (structureChanged != 0u)
{
    fillMenuRow(row, background);
    ips114_set_font(IPS114_8X16_FONT);
    ips114_set_color(nameForeground, background);
    ips114_show_char(8u, (uint16_t)(row * 16u), marker);
    ips114_show_string(CORE4_MENU_NAME_X, (uint16_t)(row * 16u), name);
    memset(g_menuLineCache[row], 0, sizeof(g_menuLineCache[row]));
    (void) strncpy(g_menuLineCache[row], signature, CORE4_MENU_LINE_CHARS);
    g_menuLineStyle[row] = style;
}
if ((structureChanged != 0u) || (valueChanged != 0u))
{
    ips114_set_color(valueForeground, background);
    ips114_show_string(CORE4_MENU_VALUE_X,
        (uint16_t)(row * 16u),
        fixedValue);
    memcpy(g_menuValueCache[row], fixedValue, sizeof(fixedValue));
    g_menuLineValid[row] = 1u;
}
}

static uint16_t core4DiagnosticsFrameColor(uint8_t linkReady,
    uint8_t inputOwner)
{
    if ((linkReady == 0u) || (inputOwner == OBSERVER_LINK_KVM_TARGET_NONE))
    {
        return RGB565_YELLOW;
    }
    return (inputOwner == OBSERVER_LINK_KVM_TARGET_CORE4) ? RGB565_GREEN : RGB565_RED;
}

static uint16_t core4DiagnosticsOwnerFrameColor(const core4RuntimeStatus *status)

```



```

{
    if ((status == NULL) || (status->kvmLinkReady == 0u) ||
        (status->kvmInputOwner == OBSERVER_LINK_KVM_TARGET_NONE)) return RGB565_YELLOW;
    return (status->kvmInputOwner == OBSERVER_LINK_KVM_TARGET_CORE4
        ? RGB565_GREEN : RGB565_RED;
    }

static void drawHomeFrame(uint16_t color)
{
    if (g_homeFrameColor != color)
    {
        drawMenuFrame(color);
        fillSolidRect(133u, 16u, 1u, 89u, color);
        fillSolidRect(1u, 104u, 238u, 1u, color);
        fillSolidRect(1u, 119u, 238u, 1u, color);
        g_homeFrameColor = color;
    }
}

static void enterHomeSurface(uint16_t frameColor)
{
    if (g_displaySurface != CORE4_DISPLAY_SURFACE_HOME)
    {
        ips114_set_color(RGB565_GREEN, RGB565_BLACK);
        ips114_clear();
        memset(g_homeSlotValid, 0, sizeof(g_homeSlotValid));
        memset(g_homeSlotColor, 0, sizeof(g_homeSlotColor));
        memset(g_homeSlotBackground, 0, sizeof(g_homeSlotBackground));
        fillMenuRow(0u, RGB565_GREEN);
        g_homeFrameColor = 0xFFFFFu;
        g_homeTitleColor = 0xFFFFFu;
        g_displaySurface = CORE4_DISPLAY_SURFACE_HOME;
    }
    drawHomeFrame(frameColor);
}

static void enterMenuSurface(core4MenuState state)
{
    uint8_t oldThreshold = (uint8_t) (g_lastMenuState == CORE4_MENU_THRESHOLD_EDIT);
    uint8_t newThreshold = (uint8_t) (state == CORE4_MENU_THRESHOLD_EDIT);

    if ((g_displaySurface != CORE4_DISPLAY_SURFACE_MENU) ||
        (oldThreshold != newThreshold))
    {
        ips114_set_color(RGB565_GREEN, RGB565_BLACK);
        ips114_set_font(IPS114_8X16_FONT);
        ips114_clear();
        memset(g_menuLineValid, 0, sizeof(g_menuLineValid));
        g_scrollValid = 0u;
        g_menuFrameColor = 0xFFFFFu;
        g_thresholdOriginalFrameSeq = 0xFFFFFFFFFu;
        g_thresholdPreviewFrameSeq = 0xFFFFFFFFFu;
        g_thresholdFrameColor = 0xFFFFFu;
        g_thresholdMode = 0xFFu;
        g_thresholdOriginalValid = 0u;
        g_thresholdPreviewValid = 0u;
        memset(g_thresholdSlotValid, 0, sizeof(g_thresholdSlotValid));
        g_displaySurface = CORE4_DISPLAY_SURFACE_MENU;
    }
    g_lastMenuState = state;
}

static uint32_t core4DiagnosticsHardwareDuty(uint8_t percent)
{
    uint32_t logicalDuty = ((uint32_t) CORE4_RGB_LOGICAL_BRIGHTNESS * percent) / 100u;

    if (logicalDuty > PWM_DUTY_MAX)
    {
        logicalDuty = PWM_DUTY_MAX;
    }
    return PWM_DUTY_MAX - logicalDuty;
}

static void core4DiagnosticsSetRgb(uint8_t red, uint8_t green, uint8_t blue)
{
    uint32_t redDuty;
    uint32_t greenDuty;
    uint32_t blueDuty;

```

```

if (g_rgbReady == 0u)
{
    return;
}
redDuty = core4DiagnosticsHardwareDuty(red ? 100u : 0u);
greenDuty = core4DiagnosticsHardwareDuty(green ? 100u : 0u);
blueDuty = core4DiagnosticsHardwareDuty(blue ? 100u : 0u);

if (redDuty != g_lastRedDuty)
{
    pwm_set_duty(CORE4_RGB_LED_R_PWM, redDuty);
    g_lastRedDuty = redDuty;
}
if (greenDuty != g_lastGreenDuty)
{
    pwm_set_duty(CORE4_RGB_LED_G_PWM, greenDuty);
    g_lastGreenDuty = greenDuty;
}
if (blueDuty != g_lastBlueDuty)
{
    pwm_set_duty(CORE4_RGB_LED_B_PWM, blueDuty);
    g_lastBlueDuty = blueDuty;
}
}

static void core4DiagnosticsSetRgbLevel(uint8_t red, uint8_t green, uint8_t blue,
uint8_t level)
{
    uint32_t redDuty;
    uint32_t greenDuty;
    uint32_t blueDuty;
    if (g_rgbReady == 0u) return;
    redDuty = core4DiagnosticsHardwareDuty(red ? level : 0u);
    greenDuty = core4DiagnosticsHardwareDuty(green ? level : 0u);
    blueDuty = core4DiagnosticsHardwareDuty(blue ? level : 0u);
    if (redDuty != g_lastRedDuty) { pwm_set_duty(CORE4_RGB_LED_R_PWM, redDuty); g_lastRedDuty = redDuty; }
    if (greenDuty != g_lastGreenDuty) { pwm_set_duty(CORE4_RGB_LED_G_PWM, greenDuty); g_lastGreenDuty = greenDuty; }
    if (blueDuty != g_lastBlueDuty) { pwm_set_duty(CORE4_RGB_LED_B_PWM, blueDuty); g_lastBlueDuty = blueDuty; }
}

static uint8_t core4DiagnosticsBreatheLevel(uint32_t tickMs)
{
    uint32_t phase = tickMs % 1000u;
    uint32_t triangle = (phase < 500u) ? phase : (1000u - phase);
    return (uint8_t) (20u + (triangle * 80u) / 500u);
}

void core4DiagnosticsDisplayInitEarly(void)
{
    g_timingValid = 0u;
    g_rgbReady = 0u;
    g_lastRedDuty = 0xFFFFFFFFu;
    g_lastGreenDuty = 0xFFFFFFFFu;
    g_lastBlueDuty = 0xFFFFFFFFu;
    memset(g_homeSlotValid, 0, sizeof(g_homeSlotValid));
    memset(g_menuLineValid, 0, sizeof(g_menuLineValid));
    memset(g_thresholdSlotValid, 0, sizeof(g_thresholdSlotValid));
    g_displaySurface = CORE4_DISPLAY_SURFACE_UNKNOWN;
    g_lastMenuState = CORE4_MENU_WAIT_LINK;
    g_lastMenuRevision = 0u;
    g_homeFrameColor = 0xFFFFu;
    g_lastRgbMode = 0xFFu;
    g_modeFlashStartMs = 0u;

    ips114_set_dir(IPS114_PORTAIT);
    ips114_set_font(IPS114_8X16_FONT);
    ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
    ips114_init();
}

void core4DiagnosticsRuntimeInit(void)
{
    uint32_t before;

    g_timingValid = 0u;
    if ((SystemCoreClock != 0u) && ((DWT->CTRL & DWT_CTRL_NOCYCCNT_Msk) == 0u))
    {
        CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
        DWT->CYCCNT = 0u;
    }
}

```

```

DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
__DSB();
__ISB();

before = DWT->CYCCNT;
__NOP();
__NOP();
__NOP();
__NOP();
if (((DWT->CTRL & DWT_CTRL_CYCCNTENA_Msk) != 0u) && (DWT->CYCCNT != before))
{
    g_timingValid = 1u;
}
}

pwm_init(CORE4_RGB_LED_R_PWM, CORE4_RGB_PWM_FREQUENCY_HZ, PWM_DUTY_MAX);
pwm_init(CORE4_RGB_LED_G_PWM, CORE4_RGB_PWM_FREQUENCY_HZ, PWM_DUTY_MAX);
pwm_init(CORE4_RGB_LED_B_PWM, CORE4_RGB_PWM_FREQUENCY_HZ, PWM_DUTY_MAX);
g_rgbReady = 1u;
}

uint8_t core4DiagnosticsTimingValid(void)
{
    return g_timingValid;
}

uint32_t core4DiagnosticsCycleNow(void)
{
    return (g_timingValid != 0u) ? DWT->CYCCNT : 0u;
}

uint32_t core4DiagnosticsCyclesToUs(uint32_t cycles)
{
    uint64_t scaled;

    if ((g_timingValid == 0u) || (SystemCoreClock == 0u))
    {
        return CORE4_TIMING_INVALID_US;
    }

    scaled = (uint64_t) cycles * 1000000u;
    return (uint32_t) (scaled / SystemCoreClock);
}

void core4DiagnosticsUpdateRgb(const core4RuntimeStatus *status, uint32_t tickMs)
{
    uint8_t blinkOn;

    if (status == NULL)
    {
        core4DiagnosticsSetRgb(0u, 0u, 0u);
        return;
    }

    switch (status->state)
    {
        case CORE4_RUNTIME_BOOT:
            blinkOn = (uint8_t) (((tickMs / CORE4_RGB_SLOW_HALF_PERIOD_MS) & 1u) == 0u);
            core4DiagnosticsSetRgb(blinkOn, blinkOn, blinkOn);
            break;
        case CORE4_RUNTIME_CAMERA_STARTING:
            blinkOn = (uint8_t) (((tickMs / CORE4_RGB_SLOW_HALF_PERIOD_MS) & 1u) == 0u);
            core4DiagnosticsSetRgb(0u, 0u, blinkOn);
            break;
        case CORE4_RUNTIME_RUNNING:
            if (g_lastRgbMode != status->cameraControlState)
            {
                g_lastRgbMode = status->cameraControlState;
                g_modeFlashStartMs = tickMs;
            }
            if ((status->lastCameraControlState == OBSERVER_LINK_CAMERA_CONTROL_RUN) &&
                (status->coreLinkReady == 0u))
            {
                blinkOn = (uint8_t) (((tickMs / CORE4_RGB_FAST_HALF_PERIOD_MS) & 1u) == 0u);
                core4DiagnosticsSetRgb(blinkOn, blinkOn, 0u);
            }
            else if (((status->controlFlags & OBSERVER_LINK_CONTROL_FLAG_BARRIER_REQUIRED) != 0u) ||
                (status->primaryReason == OBSERVER_LINK_REASON_CAMERA))
            {

```

```

        blinkOn = (uint8_t) (((tickMs / CORE4_RGB_FAST_HALF_PERIOD_MS) & 1u) == 0u);
        core4DiagnosticsSetRgb(blinkOn, blinkOn, 0u);
    }
    else if ((status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_REJOIN) ||
             (status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_LINK_WAIT) ||
             (status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_STANDALONE))
    {
        core4DiagnosticsSetRgbLevel(1u, 1u, 0u,
                                     core4DiagnosticsBreatheLevel(tickMs));
    }
    else if (status->initFailedMask != 0u)
    {
        uint32_t phase = tickMs % 2000u;
        blinkOn = (uint8_t) ((phase < 100u) ||
                              ((phase >= 250u) && (phase < 350u)));
        core4DiagnosticsSetRgb(blinkOn, 0u, 0u);
    }
    else if ((uint32_t) (tickMs - g_modeFlashStartMs) < 500u)
    {
        uint32_t phase = tickMs - g_modeFlashStartMs;
        blinkOn = (uint8_t) ((phase < 100u) ||
                              ((phase >= 250u) && (phase < 350u)));
        if (status->cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_RUN)
            core4DiagnosticsSetRgb(0u, blinkOn, blinkOn);
        else core4DiagnosticsSetRgb(blinkOn, blinkOn, 0u);
    }
    else if (status->controlClass == OBSERVER_LINK_CONTROL_CLASS_SEARCH)
    {
        core4DiagnosticsSetRgb(1u, 0u, 1u);
    }
    else if (status->activeSource == OBSERVER_LINK_SOURCE_CORE4)
    {
        core4DiagnosticsSetRgb(0u, 0u, 1u);
    }
    else if (status->activeSource == OBSERVER_LINK_SOURCE_CORE1)
    {
        core4DiagnosticsSetRgb(0u, 1u, 0u);
    }
    else
    {
        if (status->cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_RUN)
            core4DiagnosticsSetRgb(0u, 1u, 1u);
        else core4DiagnosticsSetRgb(1u, 1u, 0u);
    }
    break;
case CORE4_RUNTIME_CAMERA_FAULT:
default:
    blinkOn = (uint8_t) (((tickMs / CORE4_RGB_FAST_HALF_PERIOD_MS) & 1u) == 0u);
    core4DiagnosticsSetRgb(blinkOn, 0u, 0u);
    break;
}
}

```

```

uint32_t core4DiagnosticsRenderInitFaultSummary(uint8_t failedMask,
uint8_t unverifiedMask,
uint8_t secondsLeft)
{
    static const char *const names[3] = {"FLASH", "CAMERA", "PIPELINE"};
    uint32_t startCycles = core4DiagnosticsCycleNow();
    uint8_t row = 0u;
    uint8_t i;
    if (g_displaySurface != CORE4_DISPLAY_SURFACE_INIT_FAULT)
    {
        uint16_t titleColor = (failedMask != 0u) ? RGB565_RED : RGB565_YELLOW;
        ips114_set_font(IPS114_8X16_FONT);
        ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
        ips114_clear();
        drawMenuFrame(titleColor);
        fillSolidRect(1u, 1u, 238u, 15u, titleColor);
        ips114_set_font(IPS114_6X8_FONT);
        ips114_set_color(RGB565_BLACK, titleColor);
        ips114_show_string(78u, 4u,
                          (failedMask != 0u) ? "HW INIT FAIL" : "HW INIT CHECK");
        ips114_set_font(IPS114_8X16_FONT);
        for (i = 0u; i < 3u; i++)
        {
            uint8_t bit = (uint8_t) (1u << i);
            if ((failedMask & bit) != 0u)
            {

```

```

        ips114_set_color(GB565_RED, GB565_BLACK);
        ips114_show_string(16u, (uint16_t) (34u + row * 16u), names[i]);
        ips114_show_string(136u, (uint16_t) (34u + row * 16u), "FAIL ");
        row++;
    }
}
if ((unverifiedMask & CORE4_INIT_CHECK_DWT) != 0u)
{
    ips114_set_color(GB565_YELLOW, GB565_BLACK);
    ips114_show_string(16u, (uint16_t) (34u + row * 16u), "DWT TIMER");
    ips114_show_string(136u, (uint16_t) (34u + row * 16u), "CHECK");
}
g_displaySurface = CORE4_DISPLAY_SURFACE_INIT_FAULT;
}
ips114_set_color(GB565_YELLOW, GB565_BLACK);
ips114_show_string(16u, 114u, "CONTINUE 0s");
ips114_show_uint(88u, 114u, secondsLeft, 1u);
return core4DiagnosticsCyclesToUs(core4DiagnosticsCycleNow() - startCycles);
}

static uint16_t previewMapX(uint16_t imageX)
{
    return (uint16_t) (CORE4_DIAGNOSTICS_PREVIEW_X +
        ((uint32_t) imageX * (CORE4_DIAGNOSTICS_PREVIEW_WIDTH - 1u)) /
        (CORE4_CAMERA_WIDTH - 1u));
}

static uint16_t previewMapY(uint16_t imageY)
{
    return (uint16_t) (CORE4_DIAGNOSTICS_PREVIEW_Y +
        ((uint32_t) imageY * (CORE4_DIAGNOSTICS_PREVIEW_HEIGHT - 1u)) /
        (CORE4_CAMERA_HEIGHT - 1u));
}

static void previewCross(uint16_t imageX, uint16_t imageY,
    uint16_t radius, uint16_t color)
{
    uint16_t x = previewMapX(imageX);
    uint16_t y = previewMapY(imageY);
    uint16_t minX = CORE4_DIAGNOSTICS_PREVIEW_X;
    uint16_t maxX = (uint16_t) (minX + CORE4_DIAGNOSTICS_PREVIEW_WIDTH - 1u);
    uint16_t minY = CORE4_DIAGNOSTICS_PREVIEW_Y;
    uint16_t maxY = (uint16_t) (minY + CORE4_DIAGNOSTICS_PREVIEW_HEIGHT - 1u);
    uint16_t x0 = (x > minX + radius) ? (uint16_t) (x - radius) : minX;
    uint16_t x1 = (x + radius < maxX) ? (uint16_t) (x + radius) : maxX;
    uint16_t y0 = (y > minY + radius) ? (uint16_t) (y - radius) : minY;
    uint16_t y1 = (y + radius < maxY) ? (uint16_t) (y + radius) : maxY;
    fillSolidRect(x0, y, (uint16_t) (x1 - x0 + 1u), 1u, color);
    fillSolidRect(x, y0, 1u, (uint16_t) (y1 - y0 + 1u), color);
}

static void previewCircle(uint16_t imageX, uint16_t imageY,
    uint16_t imageRadius, uint16_t color)
{
    int16_t cx = (int16_t) previewMapX(imageX);
    int16_t cy = (int16_t) previewMapY(imageY);
    int16_t radius = (int16_t) (((uint32_t) imageRadius *
        CORE4_DIAGNOSTICS_PREVIEW_WIDTH) / CORE4_CAMERA_WIDTH);
    int16_t x = radius;
    int16_t y = 0;
    int16_t error = 1 - radius;
    int16_t minX = (int16_t) CORE4_DIAGNOSTICS_PREVIEW_X;
    int16_t maxX = (int16_t) (CORE4_DIAGNOSTICS_PREVIEW_X +
        CORE4_DIAGNOSTICS_PREVIEW_WIDTH - 1u);
    int16_t minY = (int16_t) CORE4_DIAGNOSTICS_PREVIEW_Y;
    int16_t maxY = (int16_t) (CORE4_DIAGNOSTICS_PREVIEW_Y +
        CORE4_DIAGNOSTICS_PREVIEW_HEIGHT - 1u);
    if (radius <= 0) return;
    while (x >= y)
    {
        int16_t px[8] = {cx + x, cx + y, cx - y, cx - x,
            cx - x, cx - y, cx + y, cx + x};
        int16_t py[8] = {cy + y, cy + x, cy + x, cy + y,
            cy - y, cy - x, cy - x, cy - y};
        uint8_t i;
        for (i = 0u; i < 8u; i++)
        {
            if ((px[i] >= minX) && (px[i] <= maxX) &&
                (py[i] >= minY) && (py[i] <= maxY))

```

```

        ips114_draw_point((uint16_t) px[i], (uint16_t) py[i], color);
    }
    y++;
    if (error < 0) error = (int16_t) (error + 2 * y + 1);
    else
    {
        x--;
        error = (int16_t) (error + 2 * (y - x) + 1);
    }
}
}

static void previewHeading(uint16_t imageX, uint16_t imageY,
    int16_t headingQ8, uint16_t color)
{
    float heading = (float) headingQ8 / 256.0f;
    int16_t x0 = (int16_t) previewMapX(imageX);
    int16_t y0 = (int16_t) previewMapY(imageY);
    int16_t x1 = (int16_t) ((float) x0 + cosf(heading) * 9.0f);
    int16_t y1 = (int16_t) ((float) y0 + sinf(heading) * 9.0f);
    int16_t minX = (int16_t) CORE4_DIAGNOSTICS_PREVIEW_X;
    int16_t maxX = (int16_t) (CORE4_DIAGNOSTICS_PREVIEW_X +
        CORE4_DIAGNOSTICS_PREVIEW_WIDTH - 1u);
    int16_t minY = (int16_t) CORE4_DIAGNOSTICS_PREVIEW_Y;
    int16_t maxY = (int16_t) (CORE4_DIAGNOSTICS_PREVIEW_Y +
        CORE4_DIAGNOSTICS_PREVIEW_HEIGHT - 1u);
    if (x1 < minX) x1 = minX; else if (x1 > maxX) x1 = maxX;
    if (y1 < minY) y1 = minY; else if (y1 > maxY) y1 = maxY;
    ips114_draw_line((uint16_t) x0, (uint16_t) y0,
        (uint16_t) x1, (uint16_t) y1, color);
}

static char core4DiagnosticsControlClass(uint8_t controlClass)
{
    switch (controlClass)
    {
        case OBSERVER_LINK_CONTROL_CLASS_CORE1_TRACK: return 'L';
        case OBSERVER_LINK_CONTROL_CLASS_OBSERVER_TRACK: return 'R';
        case OBSERVER_LINK_CONTROL_CLASS_SEARCH: return 'S';
        default: return 'N';
    }
}

static void previewTrackBorder(const beaconTrackBorderMarginsStruct *margin, uint16_t color)
{
    uint16_t left;
    uint16_t right;
    uint16_t top;
    uint16_t bottom;

    if (margin == NULL) return;
    left = previewMapX(margin->left);
    right = previewMapX((uint16_t) (CORE4_CAMERA_WIDTH - 1u - margin->right));
    top = previewMapY(margin->top);
    bottom = previewMapY((uint16_t) (CORE4_CAMERA_HEIGHT - 1u - margin->bottom));
    if (margin->top != 0u) ips114_draw_line(left, top, right, top, color);
    if (margin->bottom != 0u) ips114_draw_line(left, bottom, right, bottom, color);
    if (margin->left != 0u) ips114_draw_line(left, top, left, bottom, color);
    if (margin->right != 0u) ips114_draw_line(right, top, right, bottom, color);
}

static void previewEdgePredictionBand(const beaconTrackEdgeBandsStruct *bands)
{
    uint16_t left = previewMapX(0u);
    uint16_t right = previewMapX(CORE4_CAMERA_WIDTH - 1u);
    uint16_t top = previewMapY(0u);
    uint16_t bottom = previewMapY(CORE4_CAMERA_HEIGHT - 1u);
    if (bands == NULL) return;
    if (bands->left != 0u)
        ips114_draw_line(previewMapX(bands->left), top, previewMapX(bands->left), bottom,
            RGB565_MAGENTA);
    if (bands->right != 0u)
        ips114_draw_line(previewMapX(CORE4_CAMERA_WIDTH - bands->right), top,
            previewMapX(CORE4_CAMERA_WIDTH - bands->right), bottom, RGB565_MAGENTA);
    if (bands->top != 0u)
        ips114_draw_line(left, previewMapY(bands->top), right, previewMapY(bands->top),
            RGB565_MAGENTA);
    if (bands->bottom != 0u)
        ips114_draw_line(left, previewMapY(CORE4_CAMERA_HEIGHT - bands->bottom), right,

```

```

        previewMapY(CORE4_CAMERA_HEIGHT - bands->bottom), RGB565_MAGENTA);
    }

uint32_t core4DiagnosticsRenderPreview(const core4CameraSnapshot *snapshot,
    const core4RuntimeStatus *status,
    const core4VisionObservation *observation)
{
    uint32_t startCycles;
    uint16_t frameColor;

    if ((snapshot == NULL) || (snapshot->data == NULL) || (status == NULL))
    {
        return 0u;
    }

    startCycles = core4DiagnosticsCycleNow();
    frameColor = core4DiagnosticsOwnerFrameColor(status);
    enterHomeSurface(frameColor);
    ips114_show_gray_image(CORE4_DIAGNOSTICS_PREVIEW_X,
        CORE4_DIAGNOSTICS_PREVIEW_Y,
        snapshot->data,
        snapshot->width,
        snapshot->height,
        CORE4_DIAGNOSTICS_PREVIEW_WIDTH,
        CORE4_DIAGNOSTICS_PREVIEW_HEIGHT,
        0u);
    if ((observation != NULL) && (observation->valid != 0u))
    {
        previewTrackBorder(&observation->border.exit, RGB565_YELLOW);
        previewTrackBorder(&observation->border.enter, RGB565_CYAN);
        previewEdgePredictionBand(&observation->edgeBands);
    }
    if ((observation != NULL) && (observation->valid != 0u))
    {
        uint16_t i;
        for (i = 0u; i < observation->retainedCount; i++)
        {
            const core4VisionTrack *track = &observation->tracks[i];
            if ((track->flags & BEACON_CONFIRMED_FLAG_BORDER_REJECTED) != 0u)
                continue;
            uint8_t isSelected = (uint8_t)((status->viewOverlayValid != 0u) &&
                (status->activeSource == OBSERVER_LINK_SOURCE_CORE4) &&
                ((status->viewOverlay.flags & OBSERVER_LINK_OVERLAY_FLAG_SELECTED) != 0u) &&
                (status->viewOverlay.sourceFrameSeq == observation->trackerFrameSeq) &&
                (status->viewOverlay.selectedTrackId == track->trackId));
            if (isSelected == 0u)
            {
                uint16_t x = (track->fresh != 0u) ? track->centerX : track->predX;
                uint16_t y = (track->fresh != 0u) ? track->centerY : track->predY;
                previewCross(x, y, 3u, RGB565_YELLOW);
            }
        }
    }
    if (status->viewOverlayValid != 0u)
    {
        const observerLinkViewOverlay *overlay = &status->viewOverlay;
        if ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_CENTER) != 0u)
            previewCross(overlay->activeCenterX, overlay->activeCenterY, 4u, RGB565_CYAN);
        if ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_DEADZONE) != 0u)
        {
            previewCircle(overlay->activeCenterX, overlay->activeCenterY,
                overlay->deadzoneEnterRadius, RGB565_YELLOW);
            previewCircle(overlay->activeCenterX, overlay->activeCenterY,
                overlay->deadzoneExitRadius, RGB565_MAGENTA);
        }
        if ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_SELECTED) != 0u)
            previewCross(overlay->selectedX, overlay->selectedY, 5u,
                (overlay->selectedFlags & OBSERVER_LINK_OVERLAY_SELECTED_COAST) ?
                RGB565_MAGENTA : RGB565_RED);
        if ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_POSE) != 0u)
        {
            uint16_t poseColor = (overlay->poseFlags & OBSERVER_LINK_OVERLAY_POSE_HOLD) ?
                RGB565_BLUE : RGB565_GREEN;
            previewCross(overlay->poseX, overlay->poseY, 5u, poseColor);
            if ((overlay->poseFlags & OBSERVER_LINK_OVERLAY_POSE_HEADING_VALID) != 0u)
            {
                previewHeading(overlay->poseX, overlay->poseY,
                    overlay->poseHeadingQ8, poseColor);
            }
        }
    }
}

```

```

    }
}
return core4DiagnosticsCyclesToUs(core4DiagnosticsCycleNow() - startCycles);
}

uint32_t core4DiagnosticsRenderHud(const core4RuntimeStatus *status)
{
    uint32_t startCycles;
    const char *controlName;
    const char *sourceName;
    char errX[12];
    char errY[12];
    uint16_t frameColor;
    uint16_t titleColor;
    uint16_t coastCount;

    if (status == NULL)
    {
        return 0u;
    }

    startCycles = core4DiagnosticsCycleNow();
    controlName = core4DiagnosticsControlName(status->cameraControlState);
    sourceName = core4DiagnosticsSourceName(status);
    core4DiagnosticsFormatQ4(status->errXQ4, errX);
    core4DiagnosticsFormatQ4(status->errYQ4, errY);
    frameColor = core4DiagnosticsOwnerFrameColor(status);
    enterHomeSurface(frameColor);
    titleColor = (status->state == CORE4_RUNTIME_CAMERA_FAULT) ? RGB565_RED :
        (((status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_HOME) ||
          (status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_RUNNING)) &&
         (status->primaryReason == OBSERVER_LINK_REASON_NONE)) ?
        RGB565_GREEN : RGB565_YELLOW;
    if (g_homeTitleColor != titleColor)
    {
        fillMenuRow(0u, titleColor);
        g_homeSlotValid[0] = 0u;
        g_homeTitleColor = titleColor;
        g_homeFrameColor = 0xFFFFFu;
        drawHomeFrame(frameColor);
    }
    coastCount = (status->confirmedBeacons >= status->freshBeacons) ?
        (uint16_t) (status->confirmedBeacons - status->freshBeacons) : 0u;
    core4DiagnosticsShowHomeSlot(0u, IPS114_6X8_FONT, 30u, 4u, 30u,
        RGB565_BLACK, titleColor, "CORE4 %s", controlName);
    core4DiagnosticsShowHomeSlot(1u, IPS114_6X8_FONT, 135u, 17u, 17u,
        RGB565_WHITE, RGB565_BLACK, "CAM:%s FPS:%lu",
        core4DiagnosticsStateShortName(status->state), (unsigned long) status->appFps);
    core4DiagnosticsShowHomeSlot(2u, IPS114_6X8_FONT, 135u, 25u, 17u,
        (status->frameValid != 0u) ? RGB565_WHITE : RGB565_RED, RGB565_BLACK,
        "AGE:%lums", (unsigned long) status->frameAgeMs);
    core4DiagnosticsShowHomeSlot(3u, IPS114_6X8_FONT, 135u, 33u, 17u,
        RGB565_YELLOW, RGB565_BLACK, "L:%u F:%u C:%u",
        status->confirmedBeacons, status->freshBeacons,
        coastCount);
    core4DiagnosticsShowHomeSlot(4u, IPS114_6X8_FONT, 135u, 41u, 17u,
        (status->cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_SAFE) ?
        RGB565_YELLOW : RGB565_CYAN, RGB565_BLACK,
        "VIS:%s", controlName);
    core4DiagnosticsShowHomeSlot(5u, IPS114_6X8_FONT, 135u, 49u, 17u,
        (status->activeSource == OBSERVER_LINK_SOURCE_CORE4) ? RGB565_BLUE :
        ((status->activeSource == OBSERVER_LINK_SOURCE_CORE1) ? RGB565_GREEN :
         RGB565_MAGENTA), RGB565_BLACK,
        "SRC:%s", sourceName);
    core4DiagnosticsShowHomeSlot(6u, IPS114_6X8_FONT, 135u, 57u, 17u,
        RGB565_WHITE, RGB565_BLACK, "NAV:%s CLS:%c",
        core4DiagnosticsNavName(status->navState),
        core4DiagnosticsControlClass(status->controlClass));
    core4DiagnosticsShowHomeSlot(7u, IPS114_6X8_FONT, 135u, 65u, 17u,
        (status->controlFlags & OBSERVER_LINK_CONTROL_FLAG_BARRIER_REQUIRED) ?
        RGB565_YELLOW : RGB565_GREEN, RGB565_BLACK,
        "BAR:%c DSP:%c",
        (status->controlFlags & OBSERVER_LINK_CONTROL_FLAG_BARRIER_REQUIRED) ? 'Y' : 'N',
        (status->controlFlags & OBSERVER_LINK_CONTROL_FLAG_DISPATCH_ENABLED) ? 'Y' : 'N');
    core4DiagnosticsShowHomeSlot(8u, IPS114_6X8_FONT, 135u, 73u, 17u,
        RGB565_WHITE, RGB565_BLACK, "EX:%s", errX);
    core4DiagnosticsShowHomeSlot(9u, IPS114_6X8_FONT, 135u, 81u, 17u,
        RGB565_WHITE, RGB565_BLACK, "EY:%s", errY);
    core4DiagnosticsShowHomeSlot(10u, IPS114_6X8_FONT, 135u, 89u, 17u,

```



```

(status->core1LinkReady != 0u) ? RGB565_GREEN : RGB565_YELLOW,
RGB565_BLACK, "1:%s 0:%s 2:%s",
status->core1LinkOnline ? "ON" : "LOST",
status->core0LinkUp ? "OK" : "LOST", status->core2LinkUp ? "OK" : "LOST");
core4DiagnosticsShowHomeSlot(12u, IPS114_6X8_FONT, 2u, 106u, 39u,
(status->kvmInputOwner == OBSERVER_LINK_KVM_TARGET_NONE) ? RGB565_YELLOW :
RGB565_WHITE, RGB565_BLACK,
"KEY:%s C1:%s CFG:%s",
(status->kvmInputOwner == OBSERVER_LINK_KVM_TARGET_CORE4) ? "C4" :
((status->kvmInputOwner == OBSERVER_LINK_KVM_TARGET_CORE1) ? "C1" :
((status->kvmInputOwner == OBSERVER_LINK_KVM_TARGET_CORE0) ? "C0" : "NONE")),
status->core1LinkReady ? "OK" : "WAIT",
status->cameraControlLocked ? "LOCK" : "OPEN");
core4DiagnosticsShowHomeSlot(13u, IPS114_6X8_FONT, 2u, 121u, 39u,
(status->primaryReason == OBSERVER_LINK_REASON_NONE) ? RGB565_GREEN :
RGB565_YELLOW, RGB565_BLACK,
"STATE:%s %s",
(status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_REJOIN) ? "REJOIN" :
((status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_STANDALONE) ?
"STANDALONE" : "HOME"),
core4DiagnosticsReasonName(status->primaryReason));

return core4DiagnosticsCyclesToUs(core4DiagnosticsCycleNow() - startCycles);
}

uint32_t core4DiagnosticsRenderMenu(const core4MenuView *view)
{
uint32_t startCycles;
uint8_t row;
uint16_t frameColor;

if (view == NULL)
{
return 0u;
}

startCycles = core4DiagnosticsCycleNow();
frameColor = core4DiagnosticsFrameColor(view->linkReady,
view->kvmInputOwner);

if (view->state == CORE4_MENU_WAIT_LINK)
{
if (g_displaySurface != CORE4_DISPLAY_SURFACE_WAIT)
{
ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
ips114_set_font(IPS114_8X16_FONT);
ips114_clear();
drawMenuFrame(RGB565_YELLOW);
g_displaySurface = CORE4_DISPLAY_SURFACE_WAIT;
g_lastMenuRevision = UINT32_MAX;
}
if (g_lastMenuRevision != view->screenRevision)
{
const char *lastMode = core4DiagnosticsControlName(
view->lastCameraControlState);

ips114_set_color(RGB565_YELLOW, RGB565_BLACK);
ips114_show_string(56u, 18u, "WAIT CORE1 LINK ");
ips114_show_string(40u, 50u, "LINK: NOT READY ");
ips114_show_string(40u, 66u, "KVM : DISABLED ");
ips114_show_string(40u, 82u, "LAST CTRL: ");
ips114_show_string(136u, 82u, " ");
ips114_show_string(136u, 82u, lastMode);
ips114_show_string(40u, 106u, "RECONNECT -> HOME ");
g_lastMenuRevision = view->screenRevision;
}
}
return core4DiagnosticsCyclesToUs(core4DiagnosticsCycleNow() - startCycles);
}

if (view->lastAction != CORE4_MENU_ACTION_NONE)
{
if ((g_displaySurface != CORE4_DISPLAY_SURFACE_FEEDBACK) ||
(g_lastMenuRevision != view->screenRevision))
{
uint16_t color = (view->lastAction ==
CORE4_MENU_ACTION_RUNTIME_LOAD) ? RGB565_RED :
((view->lastActionOk != 0u) ? RGB565_GREEN : RGB565_RED);
char slotText[16];

ips114_full(color);

```

```

    if (view->lastAction == CORE4_MENU_ACTION_LIBRARY_SAVE ||
        view->lastAction == CORE4_MENU_ACTION_RUNTIME_LOAD)
    {
        ips114_set_color((color == RGB565_GREEN) ? RGB565_BLACK :
            RGB565_WHITE, color);
        ips114_show_string(80u, 42u,
            (view->lastAction == CORE4_MENU_ACTION_LIBRARY_SAVE) ?
                "SAVE OK" : "LOAD OK");
        (void) snprintf(slotText, sizeof(slotText), "SLOT %u",
            (unsigned int) view->lastActionSlot);
        ips114_show_string(88u, 66u, slotText);
        ips114_set_color(RGB565_GREEN, RGB565_BLACK);
    }
    g_displaySurface = CORE4_DISPLAY_SURFACE_FEEDBACK;
    g_lastMenuRevision = view->screenRevision;
}
return core4DiagnosticsCyclesToUs(core4DiagnosticsCycleNow() - startCycles);
}
if (view->state == CORE4_MENU_HOME)
{
    enterHomeSurface(frameColor);
    g_lastMenuRevision = view->screenRevision;
    return core4DiagnosticsCyclesToUs(core4DiagnosticsCycleNow() - startCycles);
}
if ((g_displaySurface == CORE4_DISPLAY_SURFACE_MENU) &&
    (g_lastMenuState == view->state) &&
    (g_lastMenuRevision == view->screenRevision))
{
    return core4DiagnosticsCyclesToUs(core4DiagnosticsCycleNow() - startCycles);
}

enterMenuSurface(view->state);
if (view->state == CORE4_MENU_PAGE_LIST)
{
    uint8_t pageCount = core4MenuGetPageCount();

    drawMenuScrollbar(pageCount, view->pageScrollTop);
    core4DiagnosticsShowMenuTitle("PAGES");
    for (row = 0u; row < CORE4_MENU_VISIBLE_ROWS; row++)
    {
        uint8_t page = (uint8_t) (view->pageScrollTop + row);

        if (page < pageCount)
        {
            core4DiagnosticsShowMenuRow((uint8_t) (row + 1u),
                (page == (uint8_t) view->pageIndex)
                    ? CORE4_MENU_ROW_SELECTED
                    : CORE4_MENU_ROW_NORMAL,
                core4MenuGetPageName(page),
                "");
        }
        else
        {
            core4DiagnosticsShowMenuRow((uint8_t) (row + 1u),
                CORE4_MENU_ROW_NORMAL,
                "",
                "");
        }
    }
}
else if ((view->state == CORE4_MENU_OPTION_LIST) &&
    (view->pageIndex == 0))
{
    char value[CORE4_MENU_VALUE_CHARS + 1u];

    drawMenuScrollbar(1u, 0u);
    core4DiagnosticsShowMenuTitle("PARAM SLOTS");
    (void) snprintf(value, sizeof(value), "SLOT %u",
        (unsigned int) (view->selectedParamSlot + 1u));
    core4DiagnosticsShowMenuRow(1u, CORE4_MENU_ROW_SELECTED,
        "SELECT", value);
    switch (view->paramSlotUiState)
    {
        case PARAM_SYNC_SLOT_UI_CHECKING: (void) snprintf(value, sizeof(value), "CHECKING"); break;
        case PARAM_SYNC_SLOT_UI_FULL: (void) snprintf(value, sizeof(value), "FULL"); break;
        case PARAM_SYNC_SLOT_UI_EMPTY: (void) snprintf(value, sizeof(value), "EMPTY"); break;
        case PARAM_SYNC_SLOT_UI_PARTIAL: (void) snprintf(value, sizeof(value), "PARTIAL"); break;
        case PARAM_SYNC_SLOT_UI_INVALID: (void) snprintf(value, sizeof(value), "INVALID"); break;
        case PARAM_SYNC_SLOT_UI_UNAVAILABLE: (void) snprintf(value, sizeof(value), "UNAVAILABLE"); break;
    }
}

```

```

        default: (void) snprintf(value, sizeof(value), "UNKNOWN"); break;
    }
    core4DiagnosticsShowMenuRow(2u, CORE4_MENU_ROW_NORMAL,
        "SLOT", value);
    if (view->paramGlobalUiState == CORE4_MENU_PARAM_GLOBAL_QUEUED)
    {
        (void) snprintf(value, sizeof(value), "QUEUED");
    }
    else if (view->paramGlobalUiState == CORE4_MENU_PARAM_GLOBAL_PROGRESS)
    {
        if (view->paramGlobalTarget <= 4u)
            (void) snprintf(value, sizeof(value),
                (view->paramGlobalExternal != 0u) ?
                    "BUSY C%u PH%u" : "RUN C%u PH%u",
                    (unsigned int) view->paramGlobalTarget,
                    (unsigned int) view->paramGlobalPhase);
        else
            (void) snprintf(value, sizeof(value),
                (view->paramGlobalExternal != 0u) ?
                    "BUSY PH%u" : "RUN PH%u",
                    (unsigned int) view->paramGlobalPhase);
    }
    else if (view->paramGlobalUiState == CORE4_MENU_PARAM_GLOBAL_SUCCESS)
    {
        (void) snprintf(value, sizeof(value), "SUCCESS");
    }
    else if (view->paramGlobalUiState == CORE4_MENU_PARAM_GLOBAL_FAILED)
    {
        (void) snprintf(value, sizeof(value), "FAIL C%u E%u",
            (unsigned int) view->paramGlobalTarget,
            (unsigned int) view->paramGlobalError);
    }
    else
    {
        (void) snprintf(value, sizeof(value), "IDLE");
    }
    core4DiagnosticsShowMenuRow(3u, CORE4_MENU_ROW_NORMAL,
        "TASK", value);
    core4DiagnosticsShowMenuRow(4u, CORE4_MENU_ROW_NORMAL,
        "K2/K3", "SELECT");
    core4DiagnosticsShowMenuRow(5u, CORE4_MENU_ROW_NORMAL,
        "K1 LONG", "SAVE");
    core4DiagnosticsShowMenuRow(6u, CORE4_MENU_ROW_NORMAL,
        "K4", "LOAD");
    core4DiagnosticsShowMenuRow(7u, CORE4_MENU_ROW_NORMAL,
        "K5", "BACK");
}
else
{
    uint8_t page = (uint8_t) view->pageIndex;
    uint8_t optionCount = core4MenuGetOptionCount(page);

    drawMenuScrollbar(optionCount, view->optionScrollTop);
    core4DiagnosticsShowMenuTitle(core4MenuGetPageName(page));
    for (row = 0u; row < CORE4_MENU_VISIBLE_ROWS; row++)
    {
        uint8_t option = (uint8_t) (view->optionScrollTop + row);

        if (option < optionCount)
        {
            char value[CORE4_MENU_VALUE_CHARS + 1u];
            core4MenuRowStyle style = CORE4_MENU_ROW_NORMAL;

            core4MenuFormatValue(core4MenuGetOptionValue(page, option), value);
            if (option == (uint8_t) view->optionIndex)
            {
                style = (view->state == CORE4_MENU_EDIT)
                    ? CORE4_MENU_ROW_EDIT
                    : CORE4_MENU_ROW_SELECTED;
            }
            core4DiagnosticsShowMenuRow((uint8_t) (row + 1u),
                style,
                core4MenuGetOptionName(page, option),
                value);
        }
        else
        {
            core4DiagnosticsShowMenuRow((uint8_t) (row + 1u),
                CORE4_MENU_ROW_NORMAL,

```

```

        ""
        ,
        "");
    }
}
}
if (frameColor != g_menuFrameColor)
{
    drawMenuFrame(frameColor);
    g_menuFrameColor = frameColor;
}
ips114_set_color(RGB565_GREEN, RGB565_BLACK);
g_lastMenuRevision = view->screenRevision;
return core4DiagnosticsCyclesToUs(core4DiagnosticsCycleNow() - startCycles);
}

static void core4DiagnosticsDrawThresholdSlot(core4MenuThresholdItem item,
uint16_t value, uint8_t selected)
{
    uint16_t slotX;
    uint16_t slotWidth;
    uint16_t nameX;
    uint16_t valueX;
    uint8_t digits;
    const char *name;
    char valueText[5];
    uint16_t background;
    uint16_t nameForeground;
    uint16_t valueForeground;

    if ((uint8_t) item >= CORE4_THRESHOLD_SLOT_COUNT) return;
    if ((g_thresholdSlotValid[item] != 0u) &&
        (g_thresholdSlotValue[item] == value) &&
        (g_thresholdSlotSelected[item] == selected)) return;

    if (item == CORE4_MENU_THRESHOLD_LOW)
    {
        slotX = CORE4_THRESHOLD_SLOT_LOW_X;
        slotWidth = CORE4_THRESHOLD_SLOT_LOW_WIDTH;
        nameX = CORE4_THRESHOLD_NAME_LOW_X;
        valueX = CORE4_THRESHOLD_VALUE_LOW_X;
        digits = 3u;
        name = "Lo.";
    }
    else if (item == CORE4_MENU_THRESHOLD_HIGH)
    {
        slotX = CORE4_THRESHOLD_SLOT_HIGH_X;
        slotWidth = CORE4_THRESHOLD_SLOT_HIGH_WIDTH;
        nameX = CORE4_THRESHOLD_NAME_HIGH_X;
        valueX = CORE4_THRESHOLD_VALUE_HIGH_X;
        digits = 3u;
        name = "Hi.";
    }
    else
    {
        slotX = CORE4_THRESHOLD_SLOT_EXPOSURE_X;
        slotWidth = CORE4_THRESHOLD_SLOT_EXPOSURE_WIDTH;
        nameX = CORE4_THRESHOLD_NAME_EXPOSURE_X;
        valueX = CORE4_THRESHOLD_VALUE_EXPOSURE_X;
        digits = 4u;
        name = "Ex.";
    }

    background = selected ? CORE4_MENU_SEL_BG : RGB565_BLACK;
    nameForeground = selected ? CORE4_MENU_SEL_NAME_FG : CORE4_MENU_NORM_NAME_FG;
    valueForeground = selected ? CORE4_MENU_SEL_VAL_FG : CORE4_MENU_NORM_VAL_FG;
    if (digits == 3u)
        (void) snprintf(valueText, sizeof(valueText), "%3u", (unsigned int) value);
    else
        (void) snprintf(valueText, sizeof(valueText), "%4u", (unsigned int) value);

    fillSolidRect(slotX, CORE4_THRESHOLD_TEXT_Y, slotWidth, 16u, background);
    ips114_set_font(IPS114_8X16_FONT);
    ips114_set_color(nameForeground, background);
    ips114_show_string(nameX, CORE4_THRESHOLD_TEXT_Y, name);
    ips114_set_color(valueForeground, background);
    ips114_show_string(valueX, CORE4_THRESHOLD_TEXT_Y, valueText);
    g_thresholdSlotValue[item] = value;
    g_thresholdSlotSelected[item] = selected;
    g_thresholdSlotValid[item] = 1u;
}

```

```

}

uint32_t core4DiagnosticsRenderThresholdEditor(const core4MenuView *view,
const core4CameraSnapshot *snapshot,
const uint8_t *detectPreview,
uint8_t updateImages)
{
    const core4VisionParamsStruct *params;
    uint32_t startCycles;
    uint16_t low;
    uint16_t high;
    uint16_t frameColor;

    if ((view == NULL) || (view->state != CORE4_MENU_THRESHOLD_EDIT))
    {
        return 0u;
    }
    startCycles = core4DiagnosticsCycleNow();
    frameColor = core4DiagnosticsFrameColor(view->linkReady,
        view->kvmInputOwner);
    enterMenuSurface(CORE4_MENU_THRESHOLD_EDIT);
    params = core4VisionParamsGetActive();
    if (view->detectMode == 1u)
    {
        low = params->legacyCannyLow;
        high = params->legacyCannyHigh;
        if (g_thresholdMode != view->detectMode)
            core4DiagnosticsShowMenuTitle("CANNY EDIT");
    }
    else
    {
        low = params->binaryLow;
        high = params->binaryHigh;
        if (g_thresholdMode != view->detectMode)
            core4DiagnosticsShowMenuTitle("BINARY EDIT");
    }
    if ((updateImages != 0u) && (snapshot != NULL) && (snapshot->data != NULL))
    {
        if ((g_thresholdOriginalValid == 0u) ||
            (snapshot->appFrameSeq != g_thresholdOriginalFrameSeq))
        {
            {
                ips114_show_gray_image(CORE4_THRESHOLD_LEFT_X,
                    CORE4_THRESHOLD_IMAGE_Y,
                    snapshot->data,
                    snapshot->width,
                    snapshot->height,
                    CORE4_THRESHOLD_IMAGE_W,
                    CORE4_THRESHOLD_IMAGE_H,
                    0u);
                g_thresholdOriginalFrameSeq = snapshot->appFrameSeq;
                g_thresholdOriginalValid = 1u;
            }
        }
        else if ((updateImages != 0u) && (g_thresholdOriginalValid != 0u))
        {
            fillSolidRect(CORE4_THRESHOLD_LEFT_X, CORE4_THRESHOLD_IMAGE_Y,
                CORE4_THRESHOLD_IMAGE_W, CORE4_THRESHOLD_IMAGE_H, RGB565_BLACK);
            g_thresholdOriginalFrameSeq = 0xFFFFFFFFu;
            g_thresholdOriginalValid = 0u;
        }
    }
    if ((updateImages != 0u) && (detectPreview != NULL) && (snapshot != NULL))
    {
        if ((g_thresholdPreviewValid == 0u) ||
            (snapshot->appFrameSeq != g_thresholdPreviewFrameSeq))
        {
            {
                ips114_show_gray_image(CORE4_THRESHOLD_RIGHT_X,
                    CORE4_THRESHOLD_IMAGE_Y,
                    detectPreview,
                    188u,
                    120u,
                    CORE4_THRESHOLD_IMAGE_W,
                    CORE4_THRESHOLD_IMAGE_H,
                    0u);
                g_thresholdPreviewFrameSeq = snapshot->appFrameSeq;
                g_thresholdPreviewValid = 1u;
            }
        }
        else if ((updateImages != 0u) && (g_thresholdPreviewValid != 0u))
        {

```

```

        fillSolidRect(CORE4_THRESHOLD_RIGHT_X, CORE4_THRESHOLD_IMAGE_Y,
            CORE4_THRESHOLD_IMAGE_W, CORE4_THRESHOLD_IMAGE_H, RGB565_BLACK);
        g_thresholdPreviewFrameSeq = 0xFFFFFFFFu;
        g_thresholdPreviewValid = 0u;
    }
    if (frameColor != g_thresholdFrameColor)
    {
        fillSolidRect(120u, CORE4_THRESHOLD_IMAGE_Y, 1u,
            (uint16_t) (CORE4_THRESHOLD_IMAGE_H + 1u), frameColor);
        fillSolidRect(1u,
            (uint16_t) (CORE4_THRESHOLD_IMAGE_Y + CORE4_THRESHOLD_IMAGE_H + 2u),
            238u, 1u, frameColor);
        drawMenuFrame(frameColor);
        g_thresholdFrameColor = frameColor;
    }
    core4DiagnosticsDrawThresholdSlot(CORE4_MENU_THRESHOLD_LOW,
        low, (uint8_t) (view->thresholdItem == CORE4_MENU_THRESHOLD_LOW));
    core4DiagnosticsDrawThresholdSlot(CORE4_MENU_THRESHOLD_HIGH,
        high, (uint8_t) (view->thresholdItem == CORE4_MENU_THRESHOLD_HIGH));
    core4DiagnosticsDrawThresholdSlot(CORE4_MENU_THRESHOLD_EXPOSURE,
        params->exposure,
        (uint8_t) (view->thresholdItem == CORE4_MENU_THRESHOLD_EXPOSURE));
    g_thresholdMode = view->detectMode;
    g_lastMenuRevision = view->screenRevision;
    return core4DiagnosticsCyclesToUs(core4DiagnosticsCycleNow() - startCycles);
}

```

core4_cyt4bb7/project/code/app/core4Diagnostics.h

SHA-256

:

06b6ab80db26e2960d47b62231880cb482428e72b7a3640cc772150fd7c28054

代码 (c)

```

#ifndef CORE4_DIAGNOSTICS_H
#define CORE4_DIAGNOSTICS_H

#include <stdint.h>

#include "core4CameraService.h"
#include "core4Menu.h"
#include "core4Runtime.h"
#include "core4VisionPipeline.h"

void core4DiagnosticsDisplayInitEarly(void);
void core4DiagnosticsRuntimeInit(void);
uint8_t core4DiagnosticsTimingValid(void);
uint32_t core4DiagnosticsCycleNow(void);
uint32_t core4DiagnosticsCyclesToUs(uint32_t cycles);
void core4DiagnosticsUpdateRgb(const core4RuntimeStatus *status, uint32_t tickMs);
uint32_t core4DiagnosticsRenderPreview(const core4CameraSnapshot *snapshot,
    const core4RuntimeStatus *status,
    const core4VisionObservation *observation);
uint32_t core4DiagnosticsRenderHud(const core4RuntimeStatus *status);
uint32_t core4DiagnosticsRenderInitFaultSummary(uint8_t failedMask,
    uint8_t unverifiedMask,
    uint8_t secondsLeft);
uint32_t core4DiagnosticsRenderMenu(const core4MenuView *view);
uint32_t core4DiagnosticsRenderThresholdEditor(const core4MenuView *view,
    const core4CameraSnapshot *snapshot,
    const uint8_t *detectPreview,
    uint8_t updateImages);

#endif /* CORE4_DIAGNOSTICS_H */

```

core4_cyt4bb7/project/code/app/core4Menu.c

SHA-256

:

68b879f50c2e76ebfa2008823d7a10936ba7691d14eb568207c3cfeecafe477

代码 (c)

```
#include "core4Menu.h"

#include <stddef.h>
#include <string.h>

#include "configCore4Runtime.h"
#include "configCore4VisionParams.h"
#include "core4VisionParams.h"
#include "core4VisionParamsPersistence.h"
#include "core4LinkCore4.h"
#include "observerLinkProtocol.h"
#include "paramSyncOwnerService.h"

#define CORE4_MENU_OPTION_COUNT (5u)
#define CORE4_MENU_FEEDBACK_MS (300u)

typedef struct
{
    float *value;
} core4MenuOption;

static float g_workValues[CORE4_MENU_OPTION_COUNT];
static core4MenuOption g_options[CORE4_MENU_OPTION_COUNT] = {
    {&g_workValues[0]},
    {&g_workValues[1]},
    {&g_workValues[2]},
    {&g_workValues[3]},
    {&g_workValues[4]},
};

static core4MenuView g_view;
static uint32_t g_tickMs;
static uint32_t g_feedbackStartMs;
static uint32_t g_workValuesRevision;
static uint8_t g_lastValidDesiredTarget;
static uint8_t g_lastValidDesiredTargetPrimed;

static void markDirty(void)
{
    {
        if (g_view.screenRevision < UINT32_MAX)
        {
            g_view.screenRevision++;
        }
    }
}

static uint32_t floatToU32(float value)
{
    {
        if (value <= 0.0f)
        {
            return 0u;
        }
        if (value >= (float) UINT32_MAX)
        {
            return UINT32_MAX;
        }
        return (uint32_t) (value + 0.5f);
    }
}

static void syncFromActive(void)
{
    {
        const core4VisionParamsStruct *params = core4VisionParamsGetActive();

        g_workValues[0] = (float) params->exposure;
        g_workValues[1] = (float) params->binaryLow;
        g_workValues[2] = (float) params->binaryHigh;
        g_workValues[3] = (float) params->legacyCannyLow;
        g_workValues[4] = (float) params->legacyCannyHigh;
        g_workValuesRevision = core4VisionParamsGetRevision();
    }
}

static uint32_t g_paramGlobalRequestSequence;
static paramSyncGlobalState g_paramGlobalState;
static uint8_t g_paramGlobalStateValid;
static paramSyncGlobalRequest g_paramGlobalPendingRequest;
static uint8_t g_paramGlobalRequestPending;
static uint8_t g_paramSlotInspectPending;
```

```

static uint8_t g_externalParamTransactionActive;
static uint32_t g_lastTerminalCoordinatorBoot;
static uint32_t g_lastTerminalRequestBoot;
static uint32_t g_lastTerminalRequestSequence;

static void scheduleParamSlotInspect(void)
{
    g_paramSlotInspectPending = 1u;
    if (g_view.paramSlotUiState != PARAM_SYNC_SLOT_UI_CHECKING)
    {
        g_view.paramSlotUiState = PARAM_SYNC_SLOT_UI_CHECKING;
        markDirty();
    }
}

static void setParamGlobalUi(core4MenuParamGlobalUiState uiState,
    uint8_t phase, uint8_t target, uint8_t error)
{
    if ((g_view.paramGlobalUiState != uiState) ||
        (g_view.paramGlobalPhase != phase) ||
        (g_view.paramGlobalTarget != target) ||
        (g_view.paramGlobalError != error))
    {
        g_view.paramGlobalUiState = uiState;
        g_view.paramGlobalPhase = phase;
        g_view.paramGlobalTarget = target;
        g_view.paramGlobalError = error;
        markDirty();
    }
}

static void setParamGlobalContext(uint8_t action, uint8_t slot)
{
    if ((g_view.paramGlobalAction != action) ||
        (g_view.paramGlobalSlot != slot))
    {
        g_view.paramGlobalAction = action;
        g_view.paramGlobalSlot = slot;
        markDirty();
    }
}

static void setParamGlobalExternal(uint8_t external)
{
    if (g_view.paramGlobalExternal != external)
    {
        g_view.paramGlobalExternal = external;
        markDirty();
    }
}

static uint8_t observeNewTerminal(const paramSyncGlobalState *state)
{
    if (((state->flags & PARAM_SYNC_GLOBAL_FLAG_TERMINAL) == 0u) ||
        (state->action == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY))
        return 0u;
    if ((g_lastTerminalCoordinatorBoot == state->coordinatorBootGeneration) &&
        (g_lastTerminalRequestBoot == state->requestBootGeneration) &&
        (g_lastTerminalRequestSequence == state->requestSequence))
        return 0u;
    g_lastTerminalCoordinatorBoot = state->coordinatorBootGeneration;
    g_lastTerminalRequestBoot = state->requestBootGeneration;
    g_lastTerminalRequestSequence = state->requestSequence;
    return 1u;
}

static void invalidateGlobalParamRoute(void)
{
    g_paramGlobalStateValid = 0u;
    memset(&g_paramGlobalState, 0, sizeof(g_paramGlobalState));
    if (g_externalParamTransactionActive != 0u)
    {
        g_externalParamTransactionActive = 0u;
        setParamGlobalExternal(0u);
        setParamGlobalContext(PARAM_SYNC_GLOBAL_ACTION_NONE, 0u);
        setParamGlobalUi(CORE4_MENU_PARAM_GLOBAL_FAILED,
            PARAM_SYNC_GLOBAL_PHASE_FAILED, 4u,
            PARAM_SYNC_GLOBAL_ERROR_ROUTE);
    }
}

```



```

if (g_paramGlobalRequestPending != 0u)
{
    if (g_paramGlobalPendingRequest.action ==
        PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
        g_view.paramSlotUiState = PARAM_SYNC_SLOT_UI_UNAVAILABLE;
    setParamGlobalUi(CORE4_MENU_PARAM_GLOBAL_FAILED,
        PARAM_SYNC_GLOBAL_PHASE_FAILED, 4u,
        PARAM_SYNC_GLOBAL_ERROR_ROUTE);
    g_paramGlobalRequestPending = 0u;
}
}

static void pollGlobalParamState(void)
{
    paramSyncGlobalState state;
    uint8_t matchesPending;

    if (core4LinkCore4TakeParamGlobalState(&state) == 0u)
    {
        invalidateGlobalParamRoute();
        return;
    }
    g_paramGlobalState = state;
    g_paramGlobalStateValid = 1u;
    matchesPending = (uint8_t) ((g_paramGlobalRequestPending != 0u) &&
        (paramSyncGlobalStateMatchesRequest(&state,
            &g_paramGlobalPendingRequest) != 0u));
    if (observeNewTerminal(&state) != 0u) scheduleParamSlotInspect();
    if (matchesPending == 0u)
    {
        if ((state.flags & PARAM_SYNC_GLOBAL_FLAG_BUSY) != 0u)
        {
            g_externalParamTransactionActive = 1u;
            setParamGlobalExternal(1u);
            setParamGlobalContext(state.action, state.slot);
            setParamGlobalUi(CORE4_MENU_PARAM_GLOBAL_PROGRESS, state.phase,
                state.target, state.error);
        }
        else if ((state.flags & PARAM_SYNC_GLOBAL_FLAG_TERMINAL) != 0u)
        {
            g_externalParamTransactionActive = 0u;
            setParamGlobalExternal(0u);
            setParamGlobalContext(PARAM_SYNC_GLOBAL_ACTION_NONE, 0u);
            setParamGlobalUi(CORE4_MENU_PARAM_GLOBAL_IDLE, state.phase,
                state.target, state.error);
        }
        else if ((g_externalParamTransactionActive != 0u) &&
            (paramSyncGlobalStateReady(&state) != 0u))
        {
            g_externalParamTransactionActive = 0u;
            setParamGlobalExternal(0u);
            setParamGlobalContext(PARAM_SYNC_GLOBAL_ACTION_NONE, 0u);
            setParamGlobalUi(CORE4_MENU_PARAM_GLOBAL_IDLE, state.phase,
                state.target, state.error);
        }
    }
    return;
}

if (state.phase == PARAM_SYNC_GLOBAL_PHASE_SUCCEEDED)
{
    setParamGlobalExternal(0u);
    g_view.paramGlobalAction = g_paramGlobalPendingRequest.action;
    g_view.paramGlobalSlot = g_paramGlobalPendingRequest.slot;
    if (g_paramGlobalPendingRequest.action ==
        PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
    {
        g_view.paramSlotUiState =
            paramSyncGlobalClassifySlotSummary(state.transactionId);
    }
    else
    {
        g_view.lastAction = (g_paramGlobalPendingRequest.action ==
            PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY) ?
            CORE4_MENU_ACTION_LIBRARY_SAVE : CORE4_MENU_ACTION_RUNTIME_LOAD;
        g_view.lastActionOk = 1u;
        g_view.lastActionSlot = g_paramGlobalPendingRequest.slot;
        g_feedbackStartMs = g_tickMs;
        g_paramSlotInspectPending = 1u;
    }
}

```

```

        setParamGlobalUi(CORE4_MENU_PARAM_GLOBAL_SUCCESS, state.phase,
            0xFFu, PARAM_SYNC_GLOBAL_ERROR_NONE);
        g_paramGlobalRequestPending = 0u;
        return;
    }
    if (state.phase == PARAM_SYNC_GLOBAL_PHASE_FAILED)
    {
        setParamGlobalExternal(0u);
        g_view.paramGlobalAction = g_paramGlobalPendingRequest.action;
        g_view.paramGlobalSlot = g_paramGlobalPendingRequest.slot;
        if (g_paramGlobalPendingRequest.action ==
            PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
            g_view.paramSlotUiState = PARAM_SYNC_SLOT_UI_UNAVAILABLE;
        setParamGlobalUi(CORE4_MENU_PARAM_GLOBAL_FAILED, state.phase,
            state.target, state.error);
        g_paramGlobalRequestPending = 0u;
        return;
    }
    if ((state.flags & PARAM_SYNC_GLOBAL_FLAG_BUSY) != 0u)
    {
        setParamGlobalUi(CORE4_MENU_PARAM_GLOBAL_PROGRESS, state.phase,
            state.target, state.error);
    }
}

static uint8_t requestGlobalParamAction(uint8_t action, uint8_t slot)
{
    paramSyncGlobalRequest request;
    uint8_t error;
    if (g_paramGlobalRequestPending != 0u)
        error = PARAM_SYNC_GLOBAL_ERROR_BUSY;
    else if (g_paramGlobalStateValid == 0u)
        error = PARAM_SYNC_GLOBAL_ERROR_ROUTE;
    else if (paramSyncGlobalStateReady(&g_paramGlobalState) == 0u)
    {
        if ((g_paramGlobalState.flags & PARAM_SYNC_GLOBAL_FLAG_RECOVERY) != 0u)
            error = PARAM_SYNC_GLOBAL_ERROR_RECOVERY;
        else if (g_paramGlobalState.readyMask != PARAM_SYNC_GLOBAL_TARGET_MASK)
            error = PARAM_SYNC_GLOBAL_ERROR_INCOMPLETE;
        else
            error = PARAM_SYNC_GLOBAL_ERROR_BUSY;
    }
    else
        error = PARAM_SYNC_GLOBAL_ERROR_NONE;
    if (error != PARAM_SYNC_GLOBAL_ERROR_NONE)
    {
        if (action == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
            g_view.paramSlotUiState = PARAM_SYNC_SLOT_UI_UNAVAILABLE;
        setParamGlobalUi(CORE4_MENU_PARAM_GLOBAL_FAILED,
            PARAM_SYNC_GLOBAL_PHASE_FAILED, 4u, error);
        return 0u;
    }
    g_paramGlobalRequestSequence++;
    if (g_paramGlobalRequestSequence == 0u) g_paramGlobalRequestSequence = 1u;
    request.producerBootGeneration = core4LinkCore4GetLocalBootGeneration();
    request.sequence = g_paramGlobalRequestSequence;
    request.origin = 4u;
    request.action = action;
    request.slot = slot;
    request.reserved = 0u;
    if (core4LinkCore4QueueParamGlobalRequest(&request) == 0u)
    {
        if (action == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
            g_view.paramSlotUiState = PARAM_SYNC_SLOT_UI_UNAVAILABLE;
        setParamGlobalUi(CORE4_MENU_PARAM_GLOBAL_FAILED,
            PARAM_SYNC_GLOBAL_PHASE_FAILED, 4u,
            PARAM_SYNC_GLOBAL_ERROR_ROUTE);
        return 0u;
    }
    g_paramGlobalPendingRequest = request;
    g_paramGlobalRequestPending = 1u;
    setParamGlobalExternal(0u);
    setParamGlobalUi(CORE4_MENU_PARAM_GLOBAL_QUEUED,
        PARAM_SYNC_GLOBAL_PHASE_REQUESTED, 0xFFu,
        PARAM_SYNC_GLOBAL_ERROR_NONE);
    g_view.paramGlobalAction = action;
    g_view.paramGlobalSlot = slot;
    if (action == PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
        g_view.paramSlotUiState = PARAM_SYNC_SLOT_UI_CHECKING;

```

```

        return 1u;
    }

static uint8_t isValidKvmTarget(uint8_t target)
{
    return (uint8_t) ((target == OBSERVER_LINK_KVM_TARGET_CORE0) ||
        (target == OBSERVER_LINK_KVM_TARGET_CORE1) ||
        (target == OBSERVER_LINK_KVM_TARGET_CORE3) ||
        (target == OBSERVER_LINK_KVM_TARGET_CORE4));
}

static void returnHomeForKvmSwitch(void)
{
    syncFromActive();
    g_view.state = CORE4_MENU_HOME;
    g_view.pageIndex = 0;
    g_view.optionIndex = 0;
    g_view.pageScrollTop = 0u;
    g_view.optionScrollTop = 0u;
    g_view.thresholdItem = CORE4_MENU_THRESHOLD_LOW;
    g_view.lastAction = CORE4_MENU_ACTION_NONE;
    g_view.lastActionOk = 0u;
    g_view.selectedParamSlot = 0u;
    g_feedbackStartMs = 0u;
}

static void applyWorkValues(void)
{
    core4VisionParamsStruct candidate = *core4VisionParamsGetActive();

    candidate.exposure = (uint16_t) floatToU32(g_workValues[0]);
    candidate.binaryLow = (uint16_t) floatToU32(g_workValues[1]);
    candidate.binaryHigh = (uint16_t) floatToU32(g_workValues[2]);
    candidate.legacyCannyLow = (uint16_t) floatToU32(g_workValues[3]);
    candidate.legacyCannyHigh = (uint16_t) floatToU32(g_workValues[4]);
    core4VisionParamsCommit(&candidate);
    syncFromActive();
}

static void enterHome(void)
{
    g_view.state = CORE4_MENU_HOME;
    markDirty();
}

static void saveSettings(void)
{
    g_view.lastAction = CORE4_MENU_ACTION_SAVE;
    g_view.lastActionOk = (uint8_t) (core4VisionParamsFlashSave(
        core4VisionParamsGetActive()) == CORE4_VISION_PARAMS_SAVE_VERIFIED);
    g_feedbackStartMs = g_tickMs;
    markDirty();
}

static uint8_t handleHome(core4MenuKey key, core4MenuPress press)
{
    if ((key == CORE4_MENU_KEY_3) && (press == CORE4_MENU_PRESS_SHORT))
    {
        g_view.state = CORE4_MENU_THRESHOLD_EDIT;
        g_view.thresholdItem = CORE4_MENU_THRESHOLD_LOW;
        markDirty();
        return 1u;
    }
    if ((key == CORE4_MENU_KEY_4) && (press == CORE4_MENU_PRESS_SHORT))
    {
        g_view.state = CORE4_MENU_PAGE_LIST;
        g_view.pageIndex = 0;
        g_view.pageScrollTop = 0u;
        markDirty();
        return 1u;
    }
    return 0u;
}

static uint8_t handlePageList(core4MenuKey key, core4MenuPress press)
{
    if (press != CORE4_MENU_PRESS_SHORT) return 0u;
    if (key == CORE4_MENU_KEY_1)
    {

```

```

        enterHome();
        return 1u;
    }
    if (key == CORE4_MENU_KEY_4)
    {
        g_view.state = CORE4_MENU_OPTION_LIST;
        g_view.optionIndex = 0;
        g_view.optionScrollTop = 0u;
        markDirty();
        scheduleParamSlotInspect();
        return 1u;
    }
    return 0u;
}

static uint8_t handleSlotPage(core4MenuKey key, core4MenuPress press)
{
    if ((key == CORE4_MENU_KEY_5) && (press == CORE4_MENU_PRESS_SHORT))
    {
        enterHome();
        return 1u;
    }
    if (press == CORE4_MENU_PRESS_SHORT)
    {
        if ((key == CORE4_MENU_KEY_2) && (g_view.selectedParamSlot > 0u))
        {
            g_view.selectedParamSlot--;
            scheduleParamSlotInspect();
            return 1u;
        }
        if ((key == CORE4_MENU_KEY_3) &&
            (g_view.selectedParamSlot + 1u < CORE4_VISION_PARAMS_SLOT_COUNT))
        {
            g_view.selectedParamSlot++;
            scheduleParamSlotInspect();
            return 1u;
        }
    }
    if (key == CORE4_MENU_KEY_4)
    {
        if (core4ParamSyncOwnerServiceMutationLocked() != 0u) return 1u;
        if (g_paramGlobalRequestPending == 0u &&
            g_paramSlotInspectPending == 0u &&
            g_view.paramGlobalAction ==
                PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY &&
            g_view.paramGlobalSlot ==
                (uint8_t) (g_view.selectedParamSlot + 1u) &&
            g_view.paramSlotUiState == PARAM_SYNC_SLOT_UI_FULL)
        {
            (void) requestGlobalParamAction(
                PARAM_SYNC_GLOBAL_ACTION_LOAD_RUNTIME,
                (uint8_t) (g_view.selectedParamSlot + 1u));
        }
        return 1u;
    }
}
if ((key == CORE4_MENU_KEY_1) && (press == CORE4_MENU_PRESS_LONG))
{
    if (core4ParamSyncOwnerServiceMutationLocked() != 0u) return 1u;
    (void) requestGlobalParamAction(PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY,
        (uint8_t) (g_view.selectedParamSlot + 1u));
    return 1u;
}
return 0u;
}

static uint8_t handleThresholdEdit(core4MenuKey key, core4MenuPress press)
{
    uint8_t lowIndex = (g_view.detectMode == 1u) ? 3u : 1u;
    uint8_t highIndex = (g_view.detectMode == 1u) ? 4u : 2u;
    uint8_t valueIndex;
    float stepShort;
    float stepLong;
    if (core4ParamSyncOwnerServiceMutationLocked() != 0u)
    {
        enterHome();
        return 1u;
    }
    float delta;

```

```

if ((key == CORE4_MENU_KEY_5) && (press == CORE4_MENU_PRESS_LONG))
{
    saveSettings();
    enterHome();
    return 1u;
}
if (press != CORE4_MENU_PRESS_SHORT)
{
    return 0u;
}
if (key == CORE4_MENU_KEY_5)
{
    g_view.thresholdItem = (core4MenuThresholdItem)
        (((uint8_t) g_view.thresholdItem + 1u) % CORE4_MENU_THRESHOLD_COUNT);
    markDirty();
    return 1u;
}

if (g_view.thresholdItem == CORE4_MENU_THRESHOLD_EXPOSURE)
{
    valueIndex = 0u;
    stepShort = CORE4_VISION_EXPOSURE_STEP_S;
    stepLong = CORE4_VISION_EXPOSURE_STEP_L;
}
else
{
    valueIndex = (g_view.thresholdItem == CORE4_MENU_THRESHOLD_LOW)
        ? lowIndex
        : highIndex;
    stepShort = (g_view.detectMode == 1u)
        ? CORE4_VISION_LEGACY_CANNY_STEP_S
        : CORE4_VISION_BINARY_STEP_S;
    stepLong = (g_view.detectMode == 1u)
        ? CORE4_VISION_LEGACY_CANNY_STEP_L
        : CORE4_VISION_BINARY_STEP_L;
}

if (key == CORE4_MENU_KEY_1)
{
    delta = stepLong;
}
else if (key == CORE4_MENU_KEY_2)
{
    delta = stepShort;
}
else if (key == CORE4_MENU_KEY_3)
{
    delta = -stepShort;
}
else if (key == CORE4_MENU_KEY_4)
{
    delta = -stepLong;
}
else
{
    return 0u;
}
*g_options[valueIndex].value += delta;
applyWorkValues();
markDirty();
return 1u;
}

void core4MenuInit(void)
{
    memset(&g_view, 0, sizeof(g_view));
    g_view.state = CORE4_MENU_HOME;
    g_view.pageIndex = 0;
    g_view.optionIndex = 0;
    g_view.screenRevision = 1u;
    g_view.kvmDesiredTarget = OBSERVER_LINK_KVM_TARGET_CORE0;
    g_view.kvmInputOwner = OBSERVER_LINK_KVM_TARGET_NONE;
    g_tickMs = 0u;
    g_feedbackStartMs = 0u;
    g_lastValidDesiredTarget = OBSERVER_LINK_KVM_TARGET_CORE0;
    g_lastValidDesiredTargetPrimed = 1u;
    g_view.selectedParamSlot = 0u;
    g_view.paramGlobalTarget = 0xFFu;
    g_paramGlobalRequestSequence = 0u;
}

```

```

g_paramGlobalStateValid = 0u;
g_paramGlobalRequestPending = 0u;
g_paramSlotInspectPending = 0u;
g_externalParamTransactionActive = 0u;
g_lastTerminalCoordinatorBoot = 0u;
g_lastTerminalRequestBoot = 0u;
g_lastTerminalRequestSequence = 0u;
memset(&g_paramGlobalState, 0, sizeof(g_paramGlobalState));
memset(&g_paramGlobalPendingRequest, 0, sizeof(g_paramGlobalPendingRequest));
g_view.paramSlotUiState = PARAM_SYNC_SLOT_UI_UNKNOWN;
syncFromActive();
}

void core4MenuTick(uint32_t tickMs)
{
    g_tickMs = tickMs;
    pollGlobalParamState();
    if ((core4ParamSyncOwnerServiceMutationLocked() == 0u) &&
        (g_workValuesRevision != core4VisionParamsGetRevision()))
    {
        syncFromActive();
        markDirty();
    }
    if ((g_view.lastAction != CORE4_MENU_ACTION_NONE) &&
        ((uint32_t)(tickMs - g_feedbackStartMs) >= CORE4_MENU_FEEDBACK_MS))
    {
        g_view.lastAction = CORE4_MENU_ACTION_NONE;
        markDirty();
    }
    if ((g_paramSlotInspectPending != 0u) &&
        (g_view.lastAction == CORE4_MENU_ACTION_NONE) &&
        (g_paramGlobalRequestPending == 0u) &&
        (g_paramGlobalStateValid != 0u) &&
        (paramSyncGlobalStateReady(&g_paramGlobalState) != 0u) &&
        requestGlobalParamAction(PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY,
            (uint8_t)(g_view.selectedParamSlot + 1u)) != 0u)
    {
        g_paramSlotInspectPending = 0u;
    }
}

void core4MenuSetDetectMode(uint8_t detectMode)
{
    uint8_t normalized = (detectMode == 1u) ? 1u : 0u;

    if (g_view.detectMode != normalized)
    {
        g_view.detectMode = normalized;
        markDirty();
    }
}

void core4MenuSetLinkState(uint8_t linkReady, uint8_t lastCameraControlState)
{
    uint8_t ready = (linkReady != 0u) ? 1u : 0u;
    uint8_t controlState = (lastCameraControlState <= 2u) ?
        lastCameraControlState : 0u;
    uint8_t changed = (uint8_t)((g_view.linkReady != ready) ||
        (g_view.lastCameraControlState != controlState));

    if (ready == 0u)
    {
        invalidateGlobalParamRoute();
        if ((g_view.linkReady != 0u) ||
            (g_view.state != CORE4_MENU_HOME) ||
            (g_view.lastAction != CORE4_MENU_ACTION_NONE))
        {
            changed = 1u;
            /* 断链中止编辑且不触发保存；固定回 HOME 继续单板视觉调试。 */
            syncFromActive();
            g_view.state = CORE4_MENU_HOME;
            g_view.thresholdItem = CORE4_MENU_THRESHOLD_LOW;
            g_view.lastAction = CORE4_MENU_ACTION_NONE;
            g_view.lastActionOk = 0u;
            g_feedbackStartMs = 0u;
        }
    }
    g_view.linkReady = ready;
}

```

```

g_view.lastCameraControlState = controlState;
if (changed != 0u)
{
    markDirty();
}
}

void core4MenuSetControlOwned(uint8_t controlOwned)
{
    uint8_t owned = ((g_view.linkReady != 0u) && (controlOwned != 0u)) ? 1u : 0u;

    if (g_view.controlOwned != owned)
    {
        g_view.controlOwned = owned;
        markDirty();
    }
}

void core4MenuSetKvmRoute(uint8_t desiredTarget, uint8_t inputOwner)
{
    if ((isValidKvmTarget(desiredTarget) != 0u) &&
        (g_lastValidDesiredTargetPrimed != 0u) &&
        (g_lastValidDesiredTarget == OBSERVER_LINK_KVM_TARGET_CORE4) &&
        (desiredTarget != OBSERVER_LINK_KVM_TARGET_CORE4))
    {
        returnHomeForKvmSwitch();
    }
    if (isValidKvmTarget(desiredTarget) != 0u)
    {
        g_lastValidDesiredTarget = desiredTarget;
        g_lastValidDesiredTargetPrimed = 1u;
    }
    if ((g_view.kvmDesiredTarget != desiredTarget) ||
        (g_view.kvmInputOwner != inputOwner))
    {
        g_view.kvmDesiredTarget = desiredTarget;
        g_view.kvmInputOwner = inputOwner;
        markDirty();
    }
}

uint8_t core4MenuSubmitKey(core4MenuKey key, core4MenuPress press)
{
    if ((key > CORE4_MENU_KEY_5) || (press > CORE4_MENU_PRESS_LONG))
    {
        return 0u;
    }
    if (g_view.lastAction != CORE4_MENU_ACTION_NONE)
    {
        /* 成功提示期间确认并吞掉已送达事件，禁止上游把旧按键重放。 */
        return 1u;
    }
    if (g_view.linkReady == 0u)
    {
        return 0u;
    }
    if ((g_lastValidDesiredTargetPrimed == 0u) ||
        (g_lastValidDesiredTarget != OBSERVER_LINK_KVM_TARGET_CORE4))
    {
        return 0u;
    }

    switch (g_view.state)
    {
    case CORE4_MENU_HOME:
        return handleHome(key, press);
    case CORE4_MENU_PAGE_LIST:
        return handlePageList(key, press);
    case CORE4_MENU_OPTION_LIST:
        return handleSlotPage(key, press);
    case CORE4_MENU_EDIT:
        enterHome();
        return 0u;
    case CORE4_MENU_THRESHOLD_EDIT:
        return handleThresholdEdit(key, press);
    case CORE4_MENU_WAIT_LINK:
        return 0u;
    default:
        return 0u;
    }
}

```

```

    }
}

void core4MenuGetView(core4MenuView *view)
{
    if (view != NULL)
    {
        *view = g_view;
    }
}

uint8_t core4MenuGetPageCount(void)
{
    return 1u;
}

const char *core4MenuGetPageName(uint8_t pageIndex)
{
    return (pageIndex == 0u) ? "Param Slots" : "";
}

uint8_t core4MenuGetOptionCount(uint8_t pageIndex)
{
    return (pageIndex == 0u) ? 1u : 0u;
}

const char *core4MenuGetOptionName(uint8_t pageIndex, uint8_t optionIndex)
{
    return ((pageIndex == 0u) && (optionIndex == 0u)) ? "Slot" : "";
}

double core4MenuGetOptionValue(uint8_t pageIndex, uint8_t optionIndex)
{
    return ((pageIndex == 0u) && (optionIndex == 0u))
        ? (double) (g_view.selectedParamSlot + 1u) : 0.0;
}

void core4MenuFormatValue(double value,
    char output[CORE4_MENU_VALUE_CHARS + 1u])
{
    uint8_t position;
    uint8_t negative = 0u;
    uint32_t integerPart;

    if (output == NULL)
    {
        return;
    }
    memset(output, ' ', CORE4_MENU_VALUE_CHARS);
    output[CORE4_MENU_VALUE_CHARS] = '\0';

    if (value < 0.0)
    {
        negative = 1u;
        value = -value;
    }
    integerPart = (uint32_t) (value + 0.5);

    position = CORE4_MENU_VALUE_CHARS;
    do
    {
        output[--position] = (char) ('0' + (integerPart % 10u));
        integerPart /= 10u;
    } while ((integerPart != 0u) && (position > 0u));
    if ((negative != 0u) && (position > 0u))
    {
        output[--position] = '-';
    }
}

```

core4_cyt4bb7/project/code/app/core4Menu.h

SHA-256

:

87d20eb70d64d4724d019d8da0e812aa88b3dc0dbb0cbd0ca7005190516de1b7

代码 (c)

```
#ifndef CORE4_MENU_H
#define CORE4_MENU_H

#include <stdint.h>

#define CORE4_MENU_VISIBLE_ROWS (7u)
#define CORE4_MENU_VALUE_CHARS (15u)

typedef enum
{
    CORE4_MENU_HOME = 0,
    CORE4_MENU_PAGE_LIST,
    CORE4_MENU_OPTION_LIST,
    CORE4_MENU_EDIT,
    CORE4_MENU_THRESHOLD_EDIT,
    CORE4_MENU_WAIT_LINK
} core4MenuState;

typedef enum
{
    CORE4_MENU_THRESHOLD_LOW = 0,
    CORE4_MENU_THRESHOLD_HIGH,
    CORE4_MENU_THRESHOLD_EXPOSURE,
    CORE4_MENU_THRESHOLD_COUNT
} core4MenuThresholdItem;

typedef enum
{
    CORE4_MENU_KEY_1 = 0,
    CORE4_MENU_KEY_2,
    CORE4_MENU_KEY_3,
    CORE4_MENU_KEY_4,
    CORE4_MENU_KEY_5
} core4MenuKey;

typedef enum
{
    CORE4_MENU_PRESS_SHORT = 0,
    CORE4_MENU_PRESS_LONG
} core4MenuPress;

typedef enum
{
    CORE4_MENU_ACTION_NONE = 0,
    CORE4_MENU_ACTION_SAVE,
    CORE4_MENU_ACTION_RESTORE,
    CORE4_MENU_ACTION_LOAD,
    CORE4_MENU_ACTION_LIBRARY_SAVE,
    CORE4_MENU_ACTION_RUNTIME_LOAD
} core4MenuAction;

typedef enum
{
    CORE4_MENU_PARAM_GLOBAL_IDLE = 0,
    CORE4_MENU_PARAM_GLOBAL_QUEUED,
    CORE4_MENU_PARAM_GLOBAL_PROGRESS,
    CORE4_MENU_PARAM_GLOBAL_SUCCESS,
    CORE4_MENU_PARAM_GLOBAL_FAILED
} core4MenuParamGlobalUiState;

typedef struct
{
    core4MenuState state;
    int8_t pageIndex;
    int8_t optionIndex;
    uint8_t pageScrollTop;
    uint8_t optionScrollTop;
    uint8_t detectMode;
    core4MenuThresholdItem thresholdItem;
    uint32_t screenRevision;
    core4MenuAction lastAction;
    uint8_t lastActionOk;
    uint8_t lastActionSlot;
    uint8_t linkReady;
    uint8_t controlOwned;
    uint8_t lastCameraControlState;
```

```

uint8_t kvmDesiredTarget;
uint8_t kvmInputOwner;
uint8_t selectedParamSlot;
core4MenuParamGlobalUiState paramGlobalUiState;
uint8_t paramGlobalPhase;
uint8_t paramGlobalTarget;
uint8_t paramGlobalError;
uint8_t paramGlobalAction;
uint8_t paramGlobalSlot;
uint8_t paramSlotUiState;
uint8_t paramGlobalExternal;
} core4MenuView;

void core4MenuInit(void);
void core4MenuTick(uint32_t tickMs);
void core4MenuSetDetectMode(uint8_t detectMode);
void core4MenuSetLinkState(uint8_t linkReady, uint8_t lastCameraControlState);
void core4MenuSetControlOwned(uint8_t controlOwned);
void core4MenuSetKvmRoute(uint8_t desiredTarget, uint8_t inputOwner);
uint8_t core4MenuSubmitKey(core4MenuKey key, core4MenuPress press);
void core4MenuGetView(core4MenuView *view);

uint8_t core4MenuGetPageCount(void);
const char *core4MenuGetPageName(uint8_t pageIndex);
uint8_t core4MenuGetOptionCount(uint8_t pageIndex);
const char *core4MenuGetOptionName(uint8_t pageIndex, uint8_t optionIndex);
double core4MenuGetOptionValue(uint8_t pageIndex, uint8_t optionIndex);

void core4MenuFormatValue(double value,
    char output[CORE4_MENU_VALUE_CHARS + 1u]);

#endif /* CORE4_MENU_H */

```

core4_cyt4bb7/project/code/app/core4Runtime.c

SHA-256

:

b7c5203ddef2bc7401d1af2e927f7f20cf3e5e348e91058ca045465a29d471c6

代码 (c)

```

#include "core4Runtime.h"

#include <string.h>

#include "configCore4Runtime.h"
#include "configBeaconTrack.h"
#include "core4CameraService.h"
#include "core4Diagnostics.h"
#include "core4Menu.h"
#include "core4LinkCore4.h"
#include "core4KvmCore4.h"
#include "core4VisionParams.h"
#include "core4VisionParamsPersistence.h"
#include "core4VisionPipeline.h"
#include "observerKvmBindingGuard.h"
#include "zf_driver_pit.h"

static volatile uint32_t g_tickMs;
static volatile uint8_t g_tickDue;

static core4RuntimeStatus g_status;
static core4VisionObservation g_latestObservation;
static core4CameraSnapshot g_latestSnapshot;
static uint8_t g_cameraInitialized;
static uint8_t g_recoveryFrameStreak;
static uint32_t g_lastInitAttemptTickMs;
static uint32_t g_lastFrameTickMs;
static uint32_t g_statsWindowStartTickMs;
static uint32_t g_statsWindowFrames;
static uint32_t g_lastHudTickMs;
static uint32_t g_lastPreviewTickMs;
static uint32_t g_lastMenuRenderTickMs;
static uint32_t g_lastRenderedMenuRevision;
static uint32_t g_lastCameraConfigAttemptTickMs;
static uint32_t g_lastCameraConfigAttemptRevision;

```

```

static uint32_t g_lastAppliedCameraConfigRevision;
static observerKvmBindingGuard g_kvmBindingGuard;
static uint32_t g_lastKvmLocalSession;
static uint32_t g_lastKvmMenuTickMs;
static uint32_t g_lastKvmMenuRevision;
static observerLinkKvmEvent g_pendingKvmEvent;
static observerLinkKvmAck g_pendingKvmAck;
static observerLinkKvmMenuStatus g_pendingKvmMenu;
static uint8_t g_pendingKvmEventValid;
static uint8_t g_pendingKvmAckValid;
static uint8_t g_pendingKvmMenuValid;
static uint32_t g_linkReadyStableStartMs;
static uint32_t g_rejoinBarrierUntilMs;
static uint8_t g_localStandalone;
static uint32_t g_initSummaryUntilMs;
static uint8_t g_initSummaryLastSecond;
static uint8_t g_menuConfirmedSwitchedAway;

#define CORE4_INITIAL_LINK_WAIT_MS (2000u)
#define CORE4_REJOIN_STABLE_MS    (2000u)

static void core4RuntimePublishMenuKvmRoute(void)
{
    if (g_status.controlStatusValid != 0u)
    {
        if (g_status.kvmDesiredTarget == OBSERVER_LINK_KVM_TARGET_CORE4)
        {
            g_menuConfirmedSwitchedAway = 0u;
        }
        else if ((g_status.kvmDesiredTarget == OBSERVER_LINK_KVM_TARGET_CORE0) ||
                 (g_status.kvmDesiredTarget == OBSERVER_LINK_KVM_TARGET_CORE1) ||
                 (g_status.kvmDesiredTarget == OBSERVER_LINK_KVM_TARGET_CORE3))
        {
            g_menuConfirmedSwitchedAway = 1u;
            core4MenuSetKvmRoute(g_status.kvmDesiredTarget,
                                g_status.kvmInputOwner);
        }
        else if ((g_status.kvmControlOwned != 0u) &&
                 (g_menuConfirmedSwitchedAway == 0u))
        {
            g_status.kvmDesiredTarget = OBSERVER_LINK_KVM_TARGET_CORE4;
            g_status.kvmInputOwner = OBSERVER_LINK_KVM_TARGET_CORE4;
            core4MenuSetKvmRoute(g_status.kvmDesiredTarget,
                                g_status.kvmInputOwner);
        }
    }
}

static void core4RuntimeClearKvmPending(void)
{
    memset(&g_pendingKvmEvent, 0, sizeof(g_pendingKvmEvent));
    memset(&g_pendingKvmAck, 0, sizeof(g_pendingKvmAck));
    memset(&g_pendingKvmMenu, 0, sizeof(g_pendingKvmMenu));
    g_pendingKvmEventValid = 0u;
    g_pendingKvmAckValid = 0u;
    g_pendingKvmMenuValid = 0u;
}

static uint8_t core4RuntimeFlashLoadFailed(core4VisionParamsLoadStatus status)
{
    switch (status)
    {
        case CORE4_VISION_PARAMS_LOAD_VALID:
        case CORE4_VISION_PARAMS_LOAD_BLANK_INITIALIZED:
        case CORE4_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN:
            return 0u;
        default:
            return 1u;
    }
}

static uint8_t core4RuntimeInitSummaryActive(uint32_t tickMs)
{
    return (uint8_t) ((g_initSummaryUntilMs != 0u) &&
                     ((int32_t) (g_initSummaryUntilMs - tickMs) > 0));
}

static void core4RuntimeUpdateLocalPhase(uint32_t tickMs)
{
    if (g_status.core1LinkReady == 0u)
    {
        g_linkReadyStableStartMs = 0u;
        if ((g_localStandalone != 0u) || (tickMs >= CORE4_INITIAL_LINK_WAIT_MS))
    }
}

```

```

    {
        g_localStandalone = 1u;
        g_status.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_STANDALONE;
        g_status.systemFlags |= OBSERVER_LINK_SYSTEM_FLAG_STANDALONE;
        g_status.primaryReason = OBSERVER_LINK_REASON_CORE1_LINK;
    }
    else
    {
        g_status.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_LINK_WAIT;
        g_status.primaryReason = OBSERVER_LINK_REASON_CORE1_LINK;
    }
    return;
}
if (g_localStandalone != 0u)
{
    if (g_linkReadyStableStartMs == 0u) g_linkReadyStableStartMs = tickMs;
    g_status.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_REJOIN;
    g_status.systemFlags |= OBSERVER_LINK_SYSTEM_FLAG_REJOIN;
    g_status.primaryReason = OBSERVER_LINK_REASON_CORE1_LINK;
    if ((uint32_t)(tickMs - g_linkReadyStableStartMs) >= CORE4_REJOIN_STABLE_MS)
    {
        /* 规划重入只建立规划屏障，不得撤销正交的KVM租约或在途ACK。 */
        g_localStandalone = 0u;
        g_linkReadyStableStartMs = 0u;
        g_rejoinBarrierUntilMs = tickMs + CORE4_RUNTIME_PIT_MS;
    }
    return;
}
if ((g_rejoinBarrierUntilMs != 0u) &&
    ((int32_t)(g_rejoinBarrierUntilMs - tickMs) > 0))
{
    g_status.controlFlags |= OBSERVER_LINK_CONTROL_FLAG_BARRIER_REQUIRED;
    g_status.primaryReason = OBSERVER_LINK_REASON_BARRIER;
}
else
{
    {
        g_rejoinBarrierUntilMs = 0u;
    }
}
}

static void core4RuntimeBuildStandaloneConfig(observerLinkSharedConfig *config)
{
    memset(config, 0, sizeof(*config));
    config->revision = 1u;
    config->detectMode = 1u;
    config->maxBeacons = 8u;
    config->edgePredictionHoldMs = VISION_TRACK_EDGE_PREDICTION_HOLD_MS_DEFAULT;
    config->sourceId = OBSERVER_LINK_SOURCE_CORE4;
    config->profileVersion = 1u;
    config->c1BorderEnterLeft = VISION_TRACK_BORDER_ENTER_LEFT_DEFAULT;
    config->c1BorderEnterRight = VISION_TRACK_BORDER_ENTER_RIGHT_DEFAULT;
    config->c1BorderEnterTop = VISION_TRACK_BORDER_ENTER_TOP_DEFAULT;
    config->c1BorderEnterBottom = VISION_TRACK_BORDER_ENTER_BOTTOM_DEFAULT;
    config->c1BorderExitLeft = VISION_TRACK_BORDER_EXIT_LEFT_DEFAULT;
    config->c1BorderExitRight = VISION_TRACK_BORDER_EXIT_RIGHT_DEFAULT;
    config->c1BorderExitTop = VISION_TRACK_BORDER_EXIT_TOP_DEFAULT;
    config->c1BorderExitBottom = VISION_TRACK_BORDER_EXIT_BOTTOM_DEFAULT;
    config->observerBorderEnterLeft = VISION_TRACK_BORDER_ENTER_LEFT_DEFAULT;
    config->observerBorderEnterRight = VISION_TRACK_BORDER_ENTER_RIGHT_DEFAULT;
    config->observerBorderEnterTop = VISION_TRACK_BORDER_ENTER_TOP_DEFAULT;
    config->observerBorderEnterBottom = VISION_TRACK_BORDER_ENTER_BOTTOM_DEFAULT;
    config->observerBorderExitLeft = VISION_TRACK_BORDER_EXIT_LEFT_DEFAULT;
    config->observerBorderExitRight = VISION_TRACK_BORDER_EXIT_RIGHT_DEFAULT;
    config->observerBorderExitTop = VISION_TRACK_BORDER_EXIT_TOP_DEFAULT;
    config->observerBorderExitBottom = VISION_TRACK_BORDER_EXIT_BOTTOM_DEFAULT;
    config->c1EdgeBandLeft = VISION_TRACK_EDGE_BAND_C1_LEFT_DEFAULT;
    config->c1EdgeBandRight = VISION_TRACK_EDGE_BAND_C1_RIGHT_DEFAULT;
    config->c1EdgeBandTop = VISION_TRACK_EDGE_BAND_C1_TOP_DEFAULT;
    config->c1EdgeBandBottom = VISION_TRACK_EDGE_BAND_C1_BOTTOM_DEFAULT;
    config->observerEdgeBandLeft = VISION_TRACK_EDGE_BAND_OBSERVER_LEFT_DEFAULT;
    config->observerEdgeBandRight = VISION_TRACK_EDGE_BAND_OBSERVER_RIGHT_DEFAULT;
    config->observerEdgeBandTop = VISION_TRACK_EDGE_BAND_OBSERVER_TOP_DEFAULT;
    config->observerEdgeBandBottom = VISION_TRACK_EDGE_BAND_OBSERVER_BOTTOM_DEFAULT;
    config->beaconSizeMin = 1u;
    config->beaconSizeMax = 5000u;
    config->beaconEccMax = 8.0f;
    config->beaconBrightnessMin = 40u;
    config->beaconMaxEndpoints = 20u;
    config->beaconFillMin = 0.05f;
}

```

```

config->beaconHuMax = 60.0f;
config->beaconHuAreaMin = 1u;
config->trackMaxAssocDist = 6.0f;
config->trackSizeRatioMax = 2.0f;
config->trackConfirmHits = 3u;
config->trackDropMisses = 4u;
config->trackCoastMisses = 2u;
config->trackPositionAlpha = 1.0f;
config->trackSizeAlpha = 1.0f;
config->trackAreaVelAlpha = 0.35f;
config->trackAssocScoreMax = 1.75f;
config->trackAssocAreaWeight = 0.45f;
config->trackAssocAreaVelWeight = 0.35f;
config->beaconHuTemplate[0] = 0.180488 205 803f;
config->beaconHuTemplate[1] = 0.0079 012 345 679f;
config->beaconHuTemplate[2] = 0.000123 277 089 059f;
config->beaconHuTemplate[3] = 0.0000 080 178 736 9286f;
config->hash = observerLinkSharedConfigHash(config);
}

static void core4RuntimeProcessKvm(uint32_t tickMs)
{
    core4LinkCore4Status linkStatus;
    observerLinkKvmControl control;
    observerLinkKvmEvent event;
    core4KvmCore4Status kvmStatus;
    core4MenuView view;
    observerKvmBindingAction bindingAction;
    uint8_t handled;

    core4LinkCore4GetStatus(&linkStatus);
    bindingAction = observerKvmBindingGuardObserve(&g_kvmBindingGuard,
        linkStatus.ownerSession, linkStatus.bindingEpoch);
    if (bindingAction != OBSERVER_KVM_BINDING_UNCHANGED)
    {
        if (bindingAction == OBSERVER_KVM_BINDING_REBOUND)
            core4KvmCore4Revoke();
        core4RuntimeClearKvmPending();
    }
    if (g_status.kvmLinkReady == 0u)
    {
        core4KvmCore4Revoke();
        core4RuntimeClearKvmPending();
    }
    core4KvmCore4Poll(tickMs);
    while (core4LinkCore4TakeKvmControl(&control) != 0u)
    {
        if (g_status.kvmLinkReady != 0u)
        {
            (void) core4KvmCore4ApplyControl(&control, tickMs);
        }
    }
    if (g_pendingKvmEventValid == 0u)
    {
        g_pendingKvmEventValid = core4LinkCore4TakeKvmEvent(
            &g_pendingKvmEvent);
    }
    if ((g_pendingKvmEventValid != 0u) &&
        (core4KvmCore4ApplyEvent(&g_pendingKvmEvent, tickMs) != 0u))
    {
        g_pendingKvmEventValid = 0u;
    }
    if (core4KvmCore4TakeEvent(&event) != 0u)
    {
        handled = core4MenuSubmitKey((core4MenuKey) event.keyId,
            (core4MenuPress) event.keyState);
        core4MenuGetView(&view);
        (void) core4KvmCore4CompleteEvent(
            handled ? OBSERVER_LINK_KVM_RESULT_EXECUTED : OBSERVER_LINK_KVM_RESULT_UNHANDLED,
            (uint8_t) view.state,
            (uint8_t) ((g_status.kvmLinkReady != 0u) &&
                (view.state == CORE4_MENU_HOME) &&
                (view.lastAction == CORE4_MENU_ACTION_NONE)),
            (uint8_t) view.lastAction,
            view.lastActionOk,
            view.screenRevision);
    }
    if (g_pendingKvmAckValid == 0u)
    {

```

```

        g_pendingKvmAckValid = core4KvmCore4TakeAck(&g_pendingKvmAck);
    }
    if ((g_pendingKvmAckValid != 0u) &&
        (core4LinkCore4QueueKvmAck(&g_pendingKvmAck) != 0u))
    {
        g_pendingKvmAckValid = 0u;
    }
    core4MenuGetView(&view);
    core4KvmCore4UpdateMenu((uint8_t) view.state,
        (uint8_t) ((g_status.kvmLinkReady != 0u) &&
            (view.state == CORE4_MENU_HOME) &&
            (view.lastAction == CORE4_MENU_ACTION_NONE)),
        (uint8_t) view.lastAction,
        view.lastActionOk,
        (uint8_t) (view.lastAction != CORE4_MENU_ACTION_NONE),
        view.screenRevision);
    if ((view.screenRevision != g_lastKvmMenuRevision) ||
        ((uint32_t) (tickMs - g_lastKvmMenuTickMs) >= 50u))
    {
        core4KvmCore4GetMenuStatus(&g_pendingKvmMenu);
        g_pendingKvmMenuValid = 1u;
    }
    if ((g_pendingKvmMenuValid != 0u) &&
        (core4LinkCore4QueueKvmMenuStatus(&g_pendingKvmMenu) != 0u))
    {
        g_lastKvmMenuRevision = g_pendingKvmMenu.screenRevision;
        g_lastKvmMenuTickMs = tickMs;
        g_pendingKvmMenuValid = 0u;
    }
    core4KvmCore4GetStatus(&kvmStatus);
    g_status.kvmControlOwned = (uint8_t) ((g_status.kvmLinkReady != 0u) &&
        (kvmStatus.leaseValid != 0u));
    /* KVM 租约与规划控制状态正交。CONTROL_STATUS 暂时过期时，仍以直接租约
    * 作为本机按键 owner 事实，避免有效操控期间框线被误判为黄色。*/
    core4RuntimePublishMenuKvmRoute();
}

static void core4RuntimeApplyCameraConfig(uint32_t tickMs)
{
    uint32_t revision;

    if (g_cameraInitialized == 0u)
    {
        return;
    }
    revision = core4VisionParamsGetRevision();
    if (revision == g_lastAppliedCameraConfigRevision)
    {
        return;
    }
    if ((revision == g_lastCameraConfigAttemptRevision) &&
        ((uint32_t) (tickMs - g_lastCameraConfigAttemptTickMs) <
            CORE4_CAMERA_CONFIG_RETRY_MS))
    {
        return;
    }
    g_lastCameraConfigAttemptRevision = revision;
    g_lastCameraConfigAttemptTickMs = tickMs;
    if (core4CameraServiceSetExposure(
        core4VisionParamsGetActive()->exposure) != 0u)
    {
        if (g_status.configApplyFailures < UINT32_MAX)
        {
            g_status.configApplyFailures++;
        }
    }
    else
    {
        g_lastAppliedCameraConfigRevision = revision;
    }
}

static void core4RuntimeRecordMaxUs(uint32_t elapsedUs, uint32_t *maximumUs)
{
    if ((elapsedUs != CORE4_TIMING_INVALID_US) && (elapsedUs > *maximumUs))
    {
        *maximumUs = elapsedUs;
    }
}

```

```

}

static void core4RuntimeAttemptCameraInit(uint32_t tickMs)
{
    uint8_t result;

    g_lastInitAttemptTickMs = tickMs;
    g_status.cameraInitAttempts++;
    result = core4CameraServiceInit();
    if (result == 0u)
    {
        g_cameraInitialized = 1u;
        g_status.initFailedMask &= (uint8_t) ~CORE4_INIT_FAIL_CAMERA;
        g_status.state = CORE4_RUNTIME_CAMERA_STARTING;
        g_lastFrameTickMs = tickMs;
        g_recoveryFrameStreak = 0u;
        core4RuntimeApplyCameraConfig(tickMs);
    }
    else
    {
        g_cameraInitialized = 0u;
        g_status.initFailedMask |= CORE4_INIT_FAIL_CAMERA;
        g_status.cameraInitFailures++;
        g_status.state = CORE4_RUNTIME_CAMERA_FAULT;
    }
}

static void core4RuntimeConsumeFrame(const core4CameraSnapshot *snapshot)
{
    uint32_t frameIntervalMs = 0u;

    if (g_status.frameValid != 0u)
    {
        uint32_t expectedFrames;

        frameIntervalMs = snapshot->tickMs - g_lastFrameTickMs;
        expectedFrames =
            (frameIntervalMs + (CORE4_CAMERA_EXPECTED_FRAME_MS / 2u)) /
            CORE4_CAMERA_EXPECTED_FRAME_MS;
        if (expectedFrames > 1u)
        {
            g_status.estimatedMissedFrames += expectedFrames - 1u;
        }
    }

    g_latestSnapshot = *snapshot;
    g_lastFrameTickMs = snapshot->tickMs;
    g_status.frameValid = 1u;
    g_status.appFrameSeq = snapshot->appFrameSeq;
    g_statsWindowFrames++;

    if (g_status.state == CORE4_RUNTIME_CAMERA_STARTING)
    {
        g_status.state = CORE4_RUNTIME_RUNNING;
        g_recoveryFrameStreak = 0u;
    }
    else if (g_status.state == CORE4_RUNTIME_CAMERA_FAULT)
    {
        if ((g_recoveryFrameStreak == 0u) ||
            (frameIntervalMs < CORE4_CAMERA_FRAME_TIMEOUT_MS))
        {
            g_recoveryFrameStreak++;
        }
        else
        {
            g_recoveryFrameStreak = 1u;
        }

        if (g_recoveryFrameStreak >= CORE4_CAMERA_RECOVERY_FRAMES)
        {
            g_status.state = CORE4_RUNTIME_RUNNING;
            g_recoveryFrameStreak = 0u;
        }
    }
}

static void core4RuntimeUpdateStats(uint32_t tickMs)
{
    uint32_t windowMs = tickMs - g_statsWindowStartTickMs;

```

```

if (windowMs >= core4VisionParamsGetActive()->statsPeriodMs)
{
    g_status.appFps = (g_statsWindowFrames * 1000u) / windowMs;
    g_statsWindowFrames = 0u;
    g_statsWindowStartTickMs = tickMs;
}

if (g_status.frameValid != 0u)
{
    g_status.frameAgeMs = tickMs - g_lastFrameTickMs;
}
}

static void core4RuntimeUpdateParamStatus(void)
{
    core4VisionParamsPersistenceStatus persistenceStatus;

    core4VisionParamsPersistenceGetStatus(&persistenceStatus);
    g_status.paramsLoadStatus = (uint8_t) persistenceStatus.lastLoad;
    g_status.flashWriteFailures = persistenceStatus.writeVerifyFailures;
    g_status.configRevision = core4VisionParamsGetRevision();
}

static void core4RuntimeUpdateLinkStatus(void)
{
    core4LinkCore4Status linkStatus;
    observerLinkViewOverlay overlay;
    uint32_t overlayAge = UINT32_MAX;
    uint8_t statusValid;

    core4LinkCore4GetStatus(&linkStatus);
    g_status.core1LinkOnline = linkStatus.online;
    g_status.core1ProtocolCompatible = linkStatus.protocolCompatible;
    g_status.sharedConfigStatus = linkStatus.configStatus;
    g_status.sharedConfigMode = linkStatus.stagedMode;
    g_status.core1LinkReady = (uint8_t) (
        (linkStatus.online != 0u) && (linkStatus.protocolCompatible != 0u) &&
        (linkStatus.planningCompatible != 0u) &&
        (linkStatus.configStatus == OBSERVER_LINK_CONFIG_APPLIED));
    g_status.kvmLinkReady = (uint8_t) ((linkStatus.online != 0u) &&
        (linkStatus.protocolCompatible != 0u) &&
        ((linkStatus.peerCapabilities &
            (OBSERVER_LINK_CAP_KVM | OBSERVER_LINK_CAP_MENU_STATUS)) ==
            (OBSERVER_LINK_CAP_KVM | OBSERVER_LINK_CAP_MENU_STATUS)));
    statusValid = (uint8_t) ((g_status.core1LinkReady != 0u) &&
        (linkStatus.controlStatusValid != 0u) &&
        (linkStatus.cameraControlValid != 0u));
    g_status.controlStatusValid = linkStatus.controlStatusValid;
    g_status.lastCameraControlState = linkStatus.lastKnownCameraControlState;
    g_status.cameraControlState = statusValid ?
        linkStatus.cameraControlState : OBSERVER_LINK_CAMERA_CONTROL_SAFE;
    g_status.cameraControlReady = statusValid;
    g_status.cameraControlLocked = 1u;
    g_status.activeSource = OBSERVER_LINK_SOURCE_NONE;
    g_status.controlClass = OBSERVER_LINK_CONTROL_CLASS_NONE;
    g_status.controlFlags = 0u;
    g_status.navState = 0u;
    g_status.systemPhase = OBSERVER_LINK_SYSTEM_PHASE_LINK_WAIT;
    g_status.systemFlags = 0u;
    g_status.primaryReason = OBSERVER_LINK_REASON_CORE1_LINK;
    g_status.kvmDesiredTarget = OBSERVER_LINK_KVM_TARGET_CORE0;
    g_status.kvmInputOwner = OBSERVER_LINK_KVM_TARGET_NONE;
    g_status.controlBeaconCount = 0u;
    g_status.core0LinkUp = 0u;
    g_status.core2LinkUp = 0u;
    g_status.targetX = 0u;
    g_status.targetY = 0u;
    g_status.targetSize = 0u;
    g_status.errXQ4 = 0;
    g_status.errYQ4 = 0;
    g_status.viewOverlayValid = 0u;
    g_status.viewOverlayAgeMs = UINT32_MAX;
    memset(&g_status.viewOverlay, 0, sizeof(g_status.viewOverlay));
    if (g_status.controlStatusValid != 0u)
    {
        const observerLinkControlStatus *control = &linkStatus.controlStatus;

        g_status.kvmDesiredTarget = control->desiredTarget;
        g_status.kvmInputOwner = control->ownerTarget;
    }
}

```



```

    g_status.core0LinkUp = (uint8_t)((control->flags &
        OBSERVER_LINK_CONTROL_FLAG_CORE0_LINK_UP) != 0u);
    g_status.core2LinkUp = (uint8_t)((control->flags &
        OBSERVER_LINK_CONTROL_FLAG_CORE2_LINK_UP) != 0u);
}
if (statusValid != 0u)
{
    const observerLinkControlStatus *control = &linkStatus.controlStatus;

    g_status.activeSource = control->activeSource;
    g_status.controlClass = control->controlClass;
    g_status.controlFlags = control->flags;
    g_status.navState = control->navState;
    g_status.systemPhase = control->systemPhase;
    g_status.systemFlags = control->systemFlags;
    g_status.primaryReason = control->primaryReason;
    g_status.controlBeaconCount = control->beaconCount;
    g_status.cameraControlLocked = (uint8_t)((control->flags &
        OBSERVER_LINK_CONTROL_FLAG_REMOTE_ELIGIBLE) == 0u);
    g_status.targetX = control->targetX;
    g_status.targetY = control->targetY;
    g_status.targetSize = control->targetSize;
    g_status.errXQ4 = control->errXQ4;
    g_status.errYQ4 = control->errYQ4;
}
if (core4LinkCore4GetViewOverlay(&overlay, g_status.tickMs,
    200u, &overlayAge) != 0u)
{
    g_status.viewOverlay = overlay;
    g_status.viewOverlayValid = 1u;
    g_status.viewOverlayAgeMs = overlayAge;
}
g_status.core1Session = linkStatus.ownerSession;
g_status.sharedConfigRevision = linkStatus.stagedRevision;
g_status.sharedConfigHash = linkStatus.stagedHash;
g_status.linkCrcErrors = linkStatus.localDiagnostics.crcErrors;
g_status.linkLengthErrors = linkStatus.localDiagnostics.lengthErrors;
g_status.linkRxOverflowErrors = linkStatus.localDiagnostics.rxOverflowErrors;
g_status.peerRxOverflowErrors = linkStatus.peerDiagnostics.rxOverflowErrors;
g_status.linkTxDeferred = linkStatus.txDeferred;
}

static void core4RuntimeUpdateVisionStatus(void)
{
    core4VisionPipelineStatus pipelineStatus;
    core4VisionObservation observation;

    core4VisionPipelineGetStatus(&pipelineStatus);
    core4VisionPipelineGetObservation(&observation);
    g_status.visionApplied = pipelineStatus.applied;
    g_status.visionDetectMode = pipelineStatus.detectMode;
    if (observation.valid != 0u)
    {
        uint16_t i;
        uint16_t ordinaryRetained = 0u;
        g_latestObservation = observation;
        g_status.visionOverflow = observation.truncated;
        g_status.visionMaxBeacons = observation.maxBeacons;
        g_status.confirmedBeacons = observation.confirmedTotal;
        g_status.freshBeacons = observation.freshTotal;
        for (i = 0u; i < observation.retainedCount; i++)
        {
            if ((observation.tracks[i].flags &
                BEACON_CONFIRMED_FLAG_BORDER_REJECTED) == 0u)
                ordinaryRetained++;
        }
        g_status.retainedBeacons = ordinaryRetained;
    }
    else
    {
        memset(&g_latestObservation, 0, sizeof(g_latestObservation));
        g_status.visionOverflow = 0u;
        g_status.visionMaxBeacons = 0u;
        g_status.confirmedBeacons = 0u;
        g_status.freshBeacons = 0u;
        g_status.retainedBeacons = 0u;
    }
}

```

```

static void core4RuntimePublishHealth(uint32_t tickMs)
{
    core4LinkCore4Status linkStatus;
    observerLinkHealth health;
    uint32_t frameAge = (g_status.appFrameSeq != 0u) ?
        (tickMs - g_lastFrameTickMs) : UINT32_MAX;

    core4LinkCore4GetStatus(&linkStatus);
    memset(&health, 0, sizeof(health));
    health.ownerSession = linkStatus.ownerSession;
    health.appFrameSeq = g_status.appFrameSeq;
    health.captureTickMs = g_lastFrameTickMs;
    health.frameAgeMs = (frameAge > UINT16_MAX) ? UINT16_MAX : (uint16_t) frameAge;
    health.bridgeAgeMs = UINT16_MAX;
    health.sourceId = OBSERVER_LINK_SOURCE_CORE4;
    health.configState = linkStatus.configStatus;
    if ((g_cameraInitialized != 0u) &&
        (g_status.state == CORE4_RUNTIME_RUNNING))
    {
        health.cameraState = OBSERVER_LINK_CAMERA_RUNNING;
    }
    else if ((g_status.state == CORE4_RUNTIME_CAMERA_FAULT) ||
        (g_status.cameraInitFailures != 0u))
    {
        health.cameraState = OBSERVER_LINK_CAMERA_FAULT;
    }
    else
    {
        health.cameraState = OBSERVER_LINK_CAMERA_UNKNOWN;
    }
    if (frameAge < CORE4_CAMERA_FRAME_TIMEOUT_MS)
    {
        health.flags |= OBSERVER_LINK_HEALTH_FLAG_FRAME_FRESH;
    }
    (void) core4LinkCore4PublishHealth(&health);
}

static uint8_t core4RuntimeBuildPlanningSnapshot(
    const core4VisionObservation *observation,
    observerLinkPlanningTrackSnapshot *snapshot)
{
    uint16_t i;
    uint16_t ordinaryRetained = 0u;

    if ((observation == NULL) || (snapshot == NULL) ||
        (observation->valid == 0u) ||
        (observation->sourceId != OBSERVER_LINK_SOURCE_CORE4) ||
        (observation->detectMode > 1u) ||
        (observation->ownerSession == 0u) ||
        (observation->retainedCount > OBSERVER_LINK_MAX_TRACKS) ||
        (observation->confirmedTotal > UINT8_MAX) ||
        (observation->freshTotal > UINT8_MAX) ||
        (observation->freshLeft > UINT8_MAX) ||
        (observation->freshRight > UINT8_MAX) ||
        (observation->freshTotal > observation->confirmedTotal) ||
        ((uint32_t) observation->freshLeft + observation->freshRight !=
            observation->freshTotal))
    {
        return 0u;
    }

    memset(snapshot, 0, sizeof(*snapshot));
    snapshot->valid = 1u;
    snapshot->sourceId = observation->sourceId;
    snapshot->detectMode = observation->detectMode;
    snapshot->truncated = observation->truncated;
    snapshot->ownerSession = observation->ownerSession;
    snapshot->revision = observation->configRevision;
    snapshot->hash = observation->configHash;
    snapshot->appFrameSeq = observation->appFrameSeq;
    snapshot->captureTickMs = observation->tickMs;
    snapshot->trackerFrameSeq = observation->trackerFrameSeq;
    snapshot->confirmedTotal = (uint8_t) observation->confirmedTotal;
    snapshot->freshTotal = (uint8_t) observation->freshTotal;
    snapshot->freshLeft = (uint8_t) observation->freshLeft;
    snapshot->freshRight = (uint8_t) observation->freshRight;
    snapshot->retainedCount = (uint8_t) observation->retainedCount;

    for (i = 0u; i < observation->retainedCount; i++)

```

```

{
    const core4VisionTrack *source = &observation->tracks[i];
    observerLinkPlanningTrack *destination = &snapshot->tracks[i];
    uint8_t rejected = (uint8_t)((source->flags &
        OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) != 0u);
    uint16_t j;

    if ((source->centerX >= 188u) || (source->centerY >= 120u) ||
        (source->predX >= 188u) || (source->predY >= 120u) ||
        (source->size > 22560u) || (source->predSize > 22560u) ||
        (source->trackId == 0u) ||
        (source->misses > OBSERVER_LINK_PLANNING_TRACK_MISSES_MAX) ||
        ((source->flags & (uint8_t) ~(OBSERVER_LINK_TRACK_FLAG_FRESH |
            OBSERVER_LINK_TRACK_FLAG_COAST |
            OBSERVER_LINK_TRACK_FLAG_EDGE_GRACE |
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED)) != 0u) ||
        (((source->flags & OBSERVER_LINK_TRACK_FLAG_FRESH) != 0u) !=
            (source->fresh != 0u)))
    {
        return 0u;
    }
    for (j = 0u; j < i; j++)
    {
        if (observation->tracks[j].trackId == source->trackId)
        {
            return 0u;
        }
    }
    if (rejected != 0u)
    {
        if ((source->flags != OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) ||
            (source->centerX != 0u) || (source->centerY != 0u) ||
            (source->predX != 0u) || (source->predY != 0u) ||
            (source->size != 0u) || (source->predSize != 0u) ||
            (source->misses != 0u) || (source->fresh != 0u) ||
            (source->velX != 0.0f) || (source->velY != 0.0f) ||
            (source->areaVel != 0.0f)) return 0u;
    }
    else
    {
        ordinaryRetained++;
    }
    destination->centerX = (uint8_t) source->centerX;
    destination->centerY = (uint8_t) source->centerY;
    destination->predX = (uint8_t) source->predX;
    destination->predY = (uint8_t) source->predY;
    destination->size = source->size;
    destination->predSize = source->predSize;
    destination->trackId = source->trackId;
    destination->misses = source->misses;
    destination->flags = source->flags;
    if ((observerLinkQuantizeSigned(source->velX, 128.0f,
        -256.0f, 255.9921875f, &destination->velXQ7) == 0u) ||
        (observerLinkQuantizeSigned(source->velY, 128.0f,
        -256.0f, 255.9921875f, &destination->velYQ7) == 0u) ||
        (observerLinkQuantizeSigned(source->areaVel, 1.0f,
        -32768.0f, 32767.0f, &destination->areaVelQ0) == 0u))
    {
        return 0u;
    }
}
return (uint8_t) (ordinaryRetained <= observation->confirmedTotal);
}

static void core4RuntimeProcessVisionFrame(const core4CameraSnapshot *snapshot)
{
    core4LinkCore4Status linkStatus;
    observerLinkSharedConfig staged;
    core4VisionObservation observation;
    observerLinkPlanningTrackSnapshot wireSnapshot;
    core4VisionPipelineStatus pipelineStatus;
    uint8_t remoteQualified;

    core4LinkCore4GetStatus(&linkStatus);
    remoteQualified = (uint8_t) ((linkStatus.online != 0u) &&
        (linkStatus.protocolCompatible != 0u) &&
        (linkStatus.ownerSession != 0u) &&
        (core4LinkCore4GetStagedConfig(&staged) != 0u));
    core4VisionPipelineGetStatus(&pipelineStatus);

```

```

if ((remoteQualified == 0u) && (pipelineStatus.applied == 0u))
{
    core4LinkCore4GetStatus(&linkStatus);
    core4RuntimeBuildStandaloneConfig(&staged);
    if (core4VisionPipelineApplyAtFrameBoundary(
        core4VisionParamsGetActive(),
        core4VisionParamsGetRevision(),
        &staged,
        linkStatus.localSession,
        linkStatus.localSession) == 0u)
    {
        core4RuntimeUpdateVisionStatus();
        return;
    }
}
if ((remoteQualified != 0u) &&
    (core4VisionPipelineApplyAtFrameBoundary(
        core4VisionParamsGetActive(),
        core4VisionParamsGetRevision(),
        &staged,
        linkStatus.localSession,
        linkStatus.ownerSession) == 0u))
{
    if (g_status.configApplyFailures < UINT32_MAX)
    {
        g_status.configApplyFailures++;
    }
    core4VisionPipelineInvalidate();
    core4RuntimeUpdateVisionStatus();
    return;
}
if ((remoteQualified != 0u) &&
    ((linkStatus.configStatus != OBSERVER_LINK_CONFIG_APPLIED) ||
     (linkStatus.appliedRevision != staged.revision) ||
     (linkStatus.appliedHash != staged.hash) ||
     (linkStatus.appliedMode != staged.detectMode)) &&
    (core4LinkCore4MarkConfigApplied(linkStatus.ownerSession,
        staged.revision,
        staged.hash,
        staged.detectMode) == 0u))
{
    if (g_status.configApplyFailures < UINT32_MAX)
    {
        g_status.configApplyFailures++;
    }
    core4RuntimeUpdateVisionStatus();
    return;
}
core4VisionPipelineSetSelectedTrackId(
    ((g_status.viewOverlayValid != 0u) &&
     (g_status.activeSource == OBSERVER_LINK_SOURCE_CORE4) &&
     ((g_status.viewOverlay.flags &
        OBSERVER_LINK_OVERLAY_FLAG_SELECTED) != 0u)) ?
    g_status.viewOverlay.selectedTrackId : 0u);
if (core4VisionPipelineProcess(snapshot) == 0u)
{
    if (g_status.configApplyFailures < UINT32_MAX)
    {
        g_status.configApplyFailures++;
    }
    core4VisionPipelineInvalidate();
}
else
{
    core4VisionPipelineGetObservation(&observation);
    if ((remoteQualified != 0u) &&
        (observation.ownerSession == linkStatus.ownerSession) &&
        (core4RuntimeBuildPlanningSnapshot(&observation,
            &wireSnapshot) != 0u)
        )
    {
        (void) core4LinkCore4QueuePlanningTrackSnapshot(&wireSnapshot);
    }
}
core4RuntimeUpdateVisionStatus();
}

static void core4RuntimeRenderView(const core4MenuView *view, uint32_t tickMs,
    uint8_t updateThresholdImages)

```

```

{
    uint32_t displayUs;

    if (view == NULL)
    {
        return;
    }
    if ((view->state != CORE4_MENU_HOME) ||
        (view->lastAction != CORE4_MENU_ACTION_NONE))
    {
        g_lastMenuRenderTickMs = tickMs;
        if (view->state == CORE4_MENU_THRESHOLD_EDIT)
        {
            displayUs = core4DiagnosticsRenderThresholdEditor(view,
                ((g_status.frameValid != 0u) &&
                 (g_status.state != CORE4_RUNTIME_CAMERA_FAULT)) ?
                 &g_latestSnapshot : NULL,
                ((g_status.frameValid != 0u) &&
                 (g_status.state != CORE4_RUNTIME_CAMERA_FAULT)) ?
                 core4VisionPipelineGetPreview() : NULL,
                updateThresholdImages);
        }
        else
        {
            displayUs = core4DiagnosticsRenderMenu(view);
        }
    }
    else
    {
        g_lastHudTickMs = tickMs;
        displayUs = core4DiagnosticsRenderHud(&g_status);
    }
    g_lastRenderedMenuRevision = view->screenRevision;
    if (g_status.timingInvalid == 0u)
    {
        core4RuntimeRecordMaxUs(displayUs, &g_status.displayMaxUs);
    }
}

```

```

void core4RuntimeInit(void)
{
    core4VisionParamsStruct initialParams;
    core4VisionParamsLoadStatus loadStatus;
    uint32_t displayUs;

    g_tickMs = 0u;
    g_tickDue = 0u;
    g_cameraInitialized = 0u;
    g_recoveryFrameStreak = 0u;
    g_lastInitAttemptTickMs = 0u;
    g_lastFrameTickMs = 0u;
    g_statsWindowStartTickMs = 0u;
    g_statsWindowFrames = 0u;
    g_lastHudTickMs = 0u;
    g_lastPreviewTickMs = 0u;
    g_lastMenuRenderTickMs = 0u;
    g_lastRenderedMenuRevision = 0u;
    g_lastCameraConfigAttemptTickMs = 0u;
    g_lastCameraConfigAttemptRevision = 0u;
    g_lastAppliedCameraConfigRevision = 0u;
    observerKvmBindingGuardInit(&g_kvmBindingGuard);
    g_lastKvmLocalSession = 0u;
    g_lastKvmMenuTickMs = 0u;
    g_lastKvmMenuRevision = 0u;
    core4RuntimeClearKvmPending();
    g_linkReadyStableStartMs = 0u;
    g_rejoinBarrierUntilMs = 0u;
    g_localStandalone = 0u;
    g_initSummaryUntilMs = 0u;
    g_initSummaryLastSecond = 0xFFu;
    memset(&g_status, 0, sizeof(g_status));
    g_menuConfirmedSwitchedAway = 0u;
    memset(&g_latestSnapshot, 0, sizeof(g_latestSnapshot));

    g_status.state = CORE4_RUNTIME_BOOT;
    core4DiagnosticsDisplayInitEarly();
    core4VisionParamsLoadDefaults(&initialParams);
    loadStatus = core4VisionParamsFlashLoad(&initialParams);
    g_status.paramsLoadStatus = (uint8_t) loadStatus;
}

```

```

if (core4RuntimeFlashLoadFailed(loadStatus) != 0u)
    g_status.initFailedMask |= CORE4_INIT_FAIL_FLASH;
core4VisionParamsSanitize(&initialParams);
core4VisionParamsSetInitial(&initialParams);
core4MenuInit();
core4DiagnosticsRuntimeInit();
g_status.timingInvalid = (uint8_t) (core4DiagnosticsTimingValid() == 0u);
if (g_status.timingInvalid != 0u)
{
    g_status.snapshotMaxUs = CORE4_TIMING_INVALID_US;
    g_status.foregroundMaxUs = CORE4_TIMING_INVALID_US;
    g_status.displayMaxUs = CORE4_TIMING_INVALID_US;
}

if (core4VisionPipelineInit() == 0u)
    g_status.initFailedMask |= CORE4_INIT_FAIL_PIPELINE;
core4LinkCore4Init(0u);
{
    core4VisionPipelineStatus pipelineStatus;
    core4VisionPipelineGetStatus(&pipelineStatus);
    core4LinkCore4SetVisionReady(pipelineStatus.ready);
}
{
    core4LinkCore4Status linkStatus;
    core4LinkCore4GetStatus(&linkStatus);
    core4KvmCore4Init(linkStatus.localSession);
    g_lastKvmLocalSession = linkStatus.localSession;
}
core4RuntimeUpdateParamStatus();
core4RuntimeUpdateLinkStatus();
core4RuntimeUpdateLocalPhase(0u);
core4MenuSetLinkState(g_status.kvmLinkReady,
    g_status.lastCameraControlState);
core4RuntimePublishMenuKvmRoute();
core4RuntimeUpdateVisionStatus();

core4RuntimeAttemptCameraInit(0u);
if (g_cameraInitialized == 0u)
    g_status.initFailedMask |= CORE4_INIT_FAIL_CAMERA;
if (g_status.timingInvalid != 0u)
    g_status.initUnverifiedMask |= CORE4_INIT_CHECK_DWT;
if ((g_status.initFailedMask | g_status.initUnverifiedMask) != 0u)
{
    g_initSummaryUntilMs = 3000u;
    g_initSummaryLastSecond = 3u;
}
core4DiagnosticsUpdateRgb(&g_status, 0u);

{
    core4MenuView menuView;

    core4MenuGetView(&menuView);
    displayUs = core4RuntimeInitSummaryActive(0u) ?
        core4DiagnosticsRenderInitFaultSummary(g_status.initFailedMask,
            g_status.initUnverifiedMask, 3u) :
        core4DiagnosticsRenderMenu(&menuView);
    g_lastRenderedMenuRevision = menuView.screenRevision;
    if (g_status.timingInvalid == 0u)
    {
        core4RuntimeRecordMaxUs(displayUs, &g_status.displayMaxUs);
    }
}

/* PIT 最后启动，确保 ISR 首次到来前全部 owner 状态已就绪。 */
pit_ms_init(PIT_CH2, CORE4_RUNTIME_PIT_MS);
}

void core4RuntimeTick10msIsr(void)
{
    g_tickMs += CORE4_RUNTIME_PIT_MS;
    g_tickDue = 1u;
}

void core4RuntimeProcess(void)
{
    core4CameraSnapshot snapshot;
    uint32_t copyCycles = 0u;
    uint32_t foregroundStartCycles;
    uint32_t foregroundUs;

```

```

uint32_t displayUs;
uint32_t tickMs = g_tickMs;
uint8_t captured = 0u;
core4MenuView menuView;

foregroundStartCycles = core4DiagnosticsCycleNow();
/* 到期位只作观测提示，调度真相是单写者 tick 的无符号差值，因此清零竞态不会丢时间。 */
g_tickDue = 0u;
g_status.tickMs = tickMs;
core4LinkCore4Poll(tickMs);
{
    core4LinkCore4Status linkStatus;
    core4LinkCore4GetStatus(&linkStatus);
    if ((linkStatus.localSession != 0u) &&
        (linkStatus.localSession != g_lastKvmLocalSession))
    {
        core4KvmCore4Init(linkStatus.localSession);
        g_lastKvmLocalSession = linkStatus.localSession;
        core4RuntimeClearKvmPending();
    }
}
core4RuntimeUpdateLinkStatus();
core4RuntimeUpdateLocalPhase(tickMs);
core4MenuSetLinkState(g_status.kvmLinkReady,
    g_status.lastCameraControlState);
core4RuntimePublishMenuKvmRoute();
core4MenuSetDetectMode((g_status.sharedConfigMode <= 1u)
    ? g_status.sharedConfigMode
    : g_status.visionDetectMode);
core4MenuTick(tickMs);
core4RuntimeProcessKvm(tickMs);
/* ACK/MENU_STATUS 在逻辑提交后同拍发送，不等待相机复制或视觉算法。 */
core4LinkCore4FlushControl(tickMs);
core4MenuSetControlOwned(g_status.kvmControlOwned);
core4MenuGetView(&menuView);

if (g_cameraInitialized != 0u)
{
    captured = core4CameraServiceTryCapture(tickMs,
        (uint8_t) (g_status.timingInvalid == 0u),
        &snapshot,
        &copyCycles);
    if (captured != 0u)
    {
        uint32_t copyUs = core4DiagnosticsCyclesToUs(copyCycles);

        core4RuntimeConsumeFrame(&snapshot);
        if (g_status.timingInvalid == 0u)
        {
            core4RuntimeRecordMaxUs(copyUs, &g_status.snapshotMaxUs);
        }
    }
}

/* 快照仍严格早于显示；菜单状态变更在大块视觉算法前优先脏区绘制，
 * 不再叠加 100ms 周期门和一整拍算法耗时。 */
if ((core4RuntimeInitSummaryActive(tickMs) == 0u) &&
    (menuView.screenRevision != g_lastRenderedMenuRevision))
{
    core4RuntimeRenderView(&menuView, tickMs, 0u);
}
if (captured != 0u)
{
    core4RuntimeProcessVisionFrame(&snapshot);
    /* 新规划快照产生后立即使用本 epoch 剩余发送预算；UART 仍由前台单 owner 串行访问。 */
    core4LinkCore4FlushTx(tickMs);
}

if (((g_status.state == CORE4_RUNTIME_CAMERA_STARTING) ||
    (g_status.state == CORE4_RUNTIME_RUNNING)) &&
    ((uint32_t) (tickMs - g_lastFrameTickMs) >= CORE4_CAMERA_FRAME_TIMEOUT_MS))
{
    g_status.state = CORE4_RUNTIME_CAMERA_FAULT;
    g_recoveryFrameStreak = 0u;
}
}
else if ((uint32_t) (tickMs - g_lastInitAttemptTickMs) >= CORE4_CAMERA_INIT_RETRY_MS)
{
    core4RuntimeAttemptCameraInit(tickMs);
}

```

```

core4RuntimeUpdateStats(tickMs);
core4RuntimeUpdateParamStatus();
core4RuntimeApplyCameraConfig(tickMs);
core4RuntimePublishHealth(tickMs);
core4DiagnosticsUpdateRgb(&g_status, tickMs);

foregroundUs = core4DiagnosticsCyclesToUs(core4DiagnosticsCycleNow() - foregroundStartCycles);
if (g_status.timingInvalid == 0u)
{
    core4RuntimeRecordMaxUs(foregroundUs, &g_status.foregroundMaxUs);
}

if (core4RuntimeInitSummaryActive(tickMs) != 0u)
{
    uint8_t second = (uint8_t) ((g_initSummaryUntilMs - tickMs + 999u) / 1000u);
    if (second != g_initSummaryLastSecond)
    {
        g_initSummaryLastSecond = second;
        displayUs = core4DiagnosticsRenderInitFaultSummary(g_status.initFailedMask,
            g_status.initUnverifiedMask, second);
        if (g_status.timingInvalid == 0u)
            core4RuntimeRecordMaxUs(displayUs, &g_status.displayMaxUs);
    }
    return;
}
g_initSummaryUntilMs = 0u;
g_initSummaryLastSecond = 0xFFu;

if ((menuView.state == CORE4_MENU_HOME) &&
    (menuView.lastAction == CORE4_MENU_ACTION_NONE) &&
    (g_status.frameValid != 0u) &&
    ((uint32_t) (tickMs - g_lastPreviewTickMs) >=
        core4VisionParamsGetActive()->previewPeriodMs))
{
    g_lastPreviewTickMs = tickMs;
    displayUs = core4DiagnosticsRenderPreview(&g_latestSnapshot, &g_status,
        &g_latestObservation);
    if (g_status.timingInvalid == 0u)
    {
        core4RuntimeRecordMaxUs(displayUs, &g_status.displayMaxUs);
    }
}

if ((menuView.state != CORE4_MENU_HOME) ||
    (menuView.lastAction != CORE4_MENU_ACTION_NONE))
{
    if (!((menuView.state == CORE4_MENU_THRESHOLD_EDIT) &&
        (menuView.screenRevision != g_lastRenderedMenuRevision) &&
        (captured == 0u) &&
        (g_cameraInitialized != 0u) &&
        (g_status.state == CORE4_RUNTIME_RUNNING) &&
        (g_status.frameValid != 0u)) &&
        ((menuView.screenRevision != g_lastRenderedMenuRevision) ||
        ((uint32_t) (tickMs - g_lastMenuRenderTickMs) >=
            CORE4_MENU_RENDER_PERIOD_MS)))
    {
        uint8_t updateThresholdImages = 0u;
        if ((menuView.state == CORE4_MENU_THRESHOLD_EDIT) &&
            ((uint32_t) (tickMs - g_lastPreviewTickMs) >=
                core4VisionParamsGetActive()->previewPeriodMs))
        {
            g_lastPreviewTickMs = tickMs;
            updateThresholdImages = 1u;
        }
        core4RuntimeRenderView(&menuView, tickMs, updateThresholdImages);
    }
}
else if ((g_lastRenderedMenuRevision != menuView.screenRevision) ||
    ((uint32_t) (tickMs - g_lastHudTickMs) >=
        core4VisionParamsGetActive()->hudPeriodMs))
{
    core4RuntimeRenderView(&menuView, tickMs, 0u);
}
}

```


core4_cyt4bb7/project/code/app/core4Runtime.h

SHA-256

:

66b8ca7c02dec143dc9e4d23f7d38a0f730113e536ba18794eef039b21d19302

代码 (c)

```
#ifndef CORE4_RUNTIME_H
#define CORE4_RUNTIME_H

#include <stdint.h>

#include "observerLinkProtocol.h"

typedef enum
{
    CORE4_RUNTIME_BOOT = 0,
    CORE4_RUNTIME_CAMERA_STARTING,
    CORE4_RUNTIME_RUNNING,
    CORE4_RUNTIME_CAMERA_FAULT
} core4RuntimeState;

#define CORE4_INIT_FAIL_FLASH    (0x01u)
#define CORE4_INIT_FAIL_CAMERA  (0x02u)
#define CORE4_INIT_FAIL_PIPELINE (0x04u)
#define CORE4_INIT_CHECK_DWT     (0x01u)

typedef struct
{
    core4RuntimeState state;
    uint8_t frameValid;
    uint8_t timingInvalid;
    uint8_t initFailedMask;
    uint8_t initUnverifiedMask;
    uint32_t tickMs;
    uint32_t appFrameSeq;
    uint32_t appFps;
    uint32_t frameAgeMs;
    uint32_t estimatedMissedFrames;
    uint32_t snapshotMaxUs;
    uint32_t foregroundMaxUs;
    uint32_t displayMaxUs;
    uint32_t cameraInitAttempts;
    uint32_t cameraInitFailures;
    uint32_t configRevision;
    uint32_t configApplyFailures;
    uint32_t flashWriteFailures;
    uint8_t paramsLoadStatus;
    uint8_t core1LinkOnline;
    uint8_t core1LinkReady;
    uint8_t kvmLinkReady;
    uint8_t core1ProtocolCompatible;
    uint8_t sharedConfigStatus;
    uint8_t sharedConfigMode;
    uint8_t cameraControlState;
    uint8_t lastCameraControlState;
    uint8_t cameraControlReady;
    uint8_t cameraControlLocked;
    uint8_t kvmControlOwned;
    uint8_t controlStatusValid;
    uint8_t activeSource;
    uint8_t controlClass;
    uint8_t controlFlags;
    uint8_t navState;
    uint8_t systemPhase;
    uint8_t systemFlags;
    uint8_t primaryReason;
    uint8_t kvmDesiredTarget;
    uint8_t kvmInputOwner;
    uint8_t controlBeaconCount;
    uint8_t core0LinkUp;
    uint8_t core2LinkUp;
    uint16_t targetX;
    uint16_t targetY;
}
```

```

uint16_t targetSize;
int16_t errXQ4;
int16_t errYQ4;
uint8_t viewOverlayValid;
uint32_t viewOverlayAgeMs;
observerLinkViewOverlay viewOverlay;
uint32_t core1Session;
uint32_t sharedConfigRevision;
uint32_t sharedConfigHash;
uint32_t linkCrcErrors;
uint32_t linkLengthErrors;
uint32_t linkRxOverflowErrors;
uint32_t peerRxOverflowErrors;
uint32_t linkTxDeferred;
uint8_t visionApplied;
uint8_t visionDetectMode;
uint8_t visionOverflow;
uint8_t visionMaxBeacons;
uint16_t confirmedBeacons;
uint16_t freshBeacons;
uint16_t retainedBeacons;
} core4RuntimeStatus;

void core4RuntimeInit(void);
void core4RuntimeTick10msIsr(void);
void core4RuntimeProcess(void);

```

```
#endif /* CORE4_RUNTIME_H */
```

core4_cyt4bb7/project/code/app/core4VisionPipeline.c

SHA-256

:

64baf81521452ef8e844f6c09703c008775161898c801aa28e6c68daf0524238

代码 (c)

```

#include "core4VisionPipeline.h"

#include <string.h>

#include "beaconTrack.h"
#include "core4VisionWorkspace.h"
#include "om_beaconcc.h"
#include "openMorph.h"

#define CORE4_VISION_WIDTH (188u)
#define CORE4_VISION_HEIGHT (120u)
#define CORE4_VISION_PIXELS (CORE4_VISION_WIDTH * CORE4_VISION_HEIGHT)
#define CORE4_VISION_CANNY_SIGMA (0.75f)

static uint8_t g_preview[CORE4_VISION_PIXELS];
static BEACON_DETECTOR_STRUCT g_detector;
static beaconTrackerStruct g_tracker;
static observerLinkSharedConfig g_activeShared;
static core4VisionObservation g_observation;
static core4VisionPipelineStatus g_status;
static uint32_t g_ownerSession;
static uint32_t g_sourceBootSession;
static uint32_t g_localRevision;
static uint16_t g_selectedTrackId;

_Static_assert(CORE4_VISION_PIXELS == OM_BUFFER_SIZE,
    "Core4 pipeline and OpenMorph image sizes differ");
_Static_assert(CORE4_VISION_MAX_PUBLISHED_TRACKS <= OM_MAX_OBJECTS,
    "Published track capacity exceeds tracker capacity");

static void clearTracking(void)
{
    memset(&g_detector, 0, sizeof(g_detector));
    beaconTrackInit(&g_tracker);
    memset(&g_observation, 0, sizeof(g_observation));
    memset(g_preview, 0, sizeof(g_preview));
}

static void applyDetectorConfig(const core4VisionParamsStruct *localParams,

```

```

const observerLinkSharedConfig *sharedConfig)
{
    BEACONCC_CONFIG_STRUCT binaryConfig;
    beaconTrackBorderConfigStruct border;
    beaconTrackEdgePredictionConfigStruct edge;

    memset(&binaryConfig, 0, sizeof(binaryConfig));
    binaryConfig.thresholdLow = (uint8_t) localParams->binaryLow;
    binaryConfig.thresholdHigh = (uint8_t) localParams->binaryHigh;
    binaryConfig.areaMin = sharedConfig->beaconSizeMin;
    binaryConfig.areaMax = sharedConfig->beaconSizeMax;
    binaryConfig.brightnessMin = sharedConfig->beaconBrightnessMin;
    binaryConfig.eccMax = sharedConfig->beaconEccMax;
    binaryConfig.fillMin = sharedConfig->beaconFillMin;
    binaryConfig.huMax = sharedConfig->beaconHuMax;
    binaryConfig.huAreaMin = sharedConfig->beaconHuAreaMin;
    beaconCcSetConfig(&binaryConfig);
    beaconCcSetTemplate(sharedConfig->beaconHuTemplate,
        sharedConfig->beaconHuMax);
    cannySetThreshold(localParams->legacyCannyLow,
        localParams->legacyCannyHigh);
    beaconDetectInit(sharedConfig->beaconSizeMin,
        sharedConfig->beaconSizeMax,
        sharedConfig->beaconEccMax,
        sharedConfig->beaconBrightnessMin,
        sharedConfig->beaconMaxEndpoints);
    beaconTrackSetParams(sharedConfig->trackMaxAssocDist,
        sharedConfig->trackSizeRatioMax,
        sharedConfig->trackConfirmHits,
        sharedConfig->trackDropMisses,
        sharedConfig->trackCoastMisses,
        sharedConfig->trackPositionAlpha,
        sharedConfig->trackSizeAlpha,
        sharedConfig->trackAreaVelAlpha,
        sharedConfig->trackAssocScoreMax,
        sharedConfig->trackAssocAreaWeight,
        sharedConfig->trackAssocAreaVelWeight);
    border.enter.left = sharedConfig->observerBorderEnterLeft;
    border.enter.right = sharedConfig->observerBorderEnterRight;
    border.enter.top = sharedConfig->observerBorderEnterTop;
    border.enter.bottom = sharedConfig->observerBorderEnterBottom;
    border.exit.left = sharedConfig->observerBorderExitLeft;
    border.exit.right = sharedConfig->observerBorderExitRight;
    border.exit.top = sharedConfig->observerBorderExitTop;
    border.exit.bottom = sharedConfig->observerBorderExitBottom;
    beaconTrackSetBorderMargins(&border);
    edge.holdMs = sharedConfig->edgePredictionHoldMs;
    edge.bands.left = sharedConfig->observerEdgeBandLeft;
    edge.bands.right = sharedConfig->observerEdgeBandRight;
    edge.bands.top = sharedConfig->observerEdgeBandTop;
    edge.bands.bottom = sharedConfig->observerEdgeBandBottom;
    beaconTrackSetEdgePredictionConfig(&edge);
}

```

```

static uint8_t trackComesFirst(const beaconConfirmedStruct *left,
    const beaconConfirmedStruct *right)
{
    uint8_t leftRejected = (uint8_t) ((left->flags &
        BEACON_CONFIRMED_FLAG_BORDER_REJECTED) != 0u);
    uint8_t rightRejected = (uint8_t) ((right->flags &
        BEACON_CONFIRMED_FLAG_BORDER_REJECTED) != 0u);
    uint16_t leftSize;
    uint16_t rightSize;

    if (leftRejected != rightRejected) return leftRejected;
    if ((leftRejected != 0u) &&
        ((left->trackId == g_selectedTrackId) !=
            (right->trackId == g_selectedTrackId)))
        return (uint8_t) (left->trackId == g_selectedTrackId);
    if (left->fresh != right->fresh)
    {
        return left->fresh;
    }
    if (left->misses != right->misses)
    {
        return (uint8_t) (left->misses < right->misses);
    }
    leftSize = (left->fresh != 0u) ? left->size : left->predSize;
    rightSize = (right->fresh != 0u) ? right->size : right->predSize;
}

```

```

    if (leftSize != rightSize)
    {
        return (uint8_t) (leftSize > rightSize);
    }
    return (uint8_t) (left->trackId < right->trackId);
}

static void copyTrack(core4VisionTrack *destination,
    const beaconConfirmedStruct *source)
{
    destination->centerX = source->centerX;
    destination->centerY = source->centerY;
    destination->size = source->size;
    destination->trackId = source->trackId;
    destination->predX = source->predX;
    destination->predY = source->predY;
    destination->predSize = source->predSize;
    destination->misses = source->misses;
    destination->fresh = source->fresh;
    destination->flags = source->flags;
    destination->velX = source->velX;
    destination->velY = source->velY;
    destination->areaVel = source->areaVel;
}

static void publishObservation(const core4CameraSnapshot *frame)
{
    uint16_t order[OM_MAX_OBJECTS];
    uint16_t count = g_tracker.confirmedCount;
    uint16_t retained = 0u;
    uint16_t ordinaryRetained = 0u;
    uint16_t i;

    memset(&g_observation, 0, sizeof(g_observation));
    g_observation.detectMode = g_activeShared.detectMode;
    g_observation.sourceId = OBSERVER_LINK_SOURCE_CORE4;
    g_observation.maxBeacons = g_activeShared.maxBeacons;
    g_observation.edgeBands.left = g_activeShared.observerEdgeBandLeft;
    g_observation.edgeBands.right = g_activeShared.observerEdgeBandRight;
    g_observation.edgeBands.top = g_activeShared.observerEdgeBandTop;
    g_observation.edgeBands.bottom = g_activeShared.observerEdgeBandBottom;
    g_observation.border.enter.left = g_activeShared.observerBorderEnterLeft;
    g_observation.border.enter.right = g_activeShared.observerBorderEnterRight;
    g_observation.border.enter.top = g_activeShared.observerBorderEnterTop;
    g_observation.border.enter.bottom = g_activeShared.observerBorderEnterBottom;
    g_observation.border.exit.left = g_activeShared.observerBorderExitLeft;
    g_observation.border.exit.right = g_activeShared.observerBorderExitRight;
    g_observation.border.exit.top = g_activeShared.observerBorderExitTop;
    g_observation.border.exit.bottom = g_activeShared.observerBorderExitBottom;
    g_observation.sourceBootSession = g_sourceBootSession;
    g_observation.ownerSession = g_ownerSession;
    g_observation.configRevision = g_activeShared.revision;
    g_observation.configHash = g_activeShared.hash;
    g_observation.appFrameSeq = frame->appFrameSeq;
    g_observation.trackerFrameSeq = g_tracker.frameSeq;
    g_observation.tickMs = frame->tickMs;
    for (i = 0u; i < count; i++)
    {
        uint16_t position = i;

        order[i] = i;
        if ((g_tracker.confirmed[i].flags &
            BEACON_CONFIRMED_FLAG_BORDER_REJECTED) == 0u)
        {
            g_observation.confirmedTotal++;
            if (g_tracker.confirmed[i].fresh != 0u)
            {
                g_observation.freshTotal++;
                if (g_tracker.confirmed[i].centerX < (CORE4_VISION_WIDTH / 2u))
                    g_observation.freshLeft++;
                else
                    g_observation.freshRight++;
            }
        }
    }
    while ((position > 0u) &&
        (trackComesFirst(&g_tracker.confirmed[order[position]],
            &g_tracker.confirmed[order[position - 1u]]) != 0u))
    {
        uint16_t swap = order[position - 1u];

```

```

        order[position - 1u] = order[position];
        order[position] = swap;
        position--;
    }
}
for (i = 0u; i < count; i++)
{
    const beaconConfirmedStruct *source = &g_tracker.confirmed[order[i]];
    uint8_t rejected = (uint8_t) ((source->flags &
        BEACON_CONFIRMED_FLAG_BORDER_REJECTED) != 0u);
    if ((retained >= CORE4_VISION_MAX_PUBLISHED_TRACKS) ||
        ((rejected == 0u) &&
            (ordinaryRetained >= g_activeShared.maxBeacons)))
    {
        g_observation.truncated = 1u;
        continue;
    }
    copyTrack(&g_observation.tracks[retained], source);
    retained++;
    if (rejected == 0u)
    {
        ordinaryRetained++;
        beaconTrackMarkPublished(&g_tracker, source->trackId);
    }
}
g_observation.retainedCount = retained;
g_observation.valid = 1u;
}

uint8_t core4VisionPipelineInit(void)
{
    u8Image source;
    BEACONCC_CONFIG_STRUCT binaryConfig;

    memset(&g_status, 0, sizeof(g_status));
    memset(&g_activeShared, 0, sizeof(g_activeShared));
    memset(g_preview, 0, sizeof(g_preview));
    g_ownerSession = 0u;
    g_sourceBootSession = 0u;
    g_localRevision = 0u;
    source.ptr = g_preview;
    source.xLen = CORE4_VISION_WIDTH;
    source.yLen = CORE4_VISION_HEIGHT;
    source.size = CORE4_VISION_PIXELS;
    memset(&binaryConfig, 0, sizeof(binaryConfig));
    beaconCclnit(&binaryConfig);
    cannyInit(&source,
        CORE4_VISION_CANNY_SIGMA,
        0u,
        0u,
        CORE4_VISION_WIDTH - 1u,
        CORE4_VISION_HEIGHT - 1u,
        CORE4_VISION_LEGACY_CANNY_LOW_DEFAULT,
        CORE4_VISION_LEGACY_CANNY_HIGH_DEFAULT);
    clearTracking();
    g_status.ready = 1u;
    return 1u;
}

void core4VisionPipelineSetSelectedTrackId(uint16_t trackId)
{
    g_selectedTrackId = trackId;
}

uint8_t core4VisionPipelineApplyAtFrameBoundary(
    const core4VisionParamsStruct *localParams,
    uint32_t localRevision,
    const observerLinkSharedConfig *sharedConfig,
    uint32_t sourceBootSession,
    uint32_t ownerSession)
{
    uint8_t sharedChanged;
    uint8_t modeChanged;
    uint8_t ownerChanged;

    if ((g_status.ready == 0u) || (localParams == NULL) ||
        (sharedConfig == NULL) || (sourceBootSession == 0u) ||
        (ownerSession == 0u) ||
        (observerLinkSharedConfigIsValid(sharedConfig) == 0u))

```

```

{
    return 0u;
}
sharedChanged = (uint8_t) ((g_status.applied == 0u) ||
    (g_activeShared.revision != sharedConfig->revision) ||
    (g_activeShared.hash != sharedConfig->hash));
modeChanged = (uint8_t) ((g_status.applied != 0u) &&
    (g_activeShared.detectMode != sharedConfig->detectMode));
ownerChanged = (uint8_t) ((g_status.applied != 0u) &&
    (g_ownerSession != ownerSession));
if ((sharedChanged != 0u) || (g_localRevision != localRevision) ||
    (ownerChanged != 0u))
{
    applyDetectorConfig(localParams, sharedConfig);
}
if ((modeChanged != 0u) || (ownerChanged != 0u))
{
    clearTracking();
    if (modeChanged != 0u)
    {
    }
}
else if (sharedChanged != 0u)
{
    g_observation.valid = 0u;
}
g_activeShared = *sharedConfig;
g_sourceBootSession = sourceBootSession;
g_ownerSession = ownerSession;
g_localRevision = localRevision;
g_status.applied = 1u;
g_status.detectMode = sharedConfig->detectMode;
return 1u;
}

uint8_t core4VisionPipelineProcess(const core4CameraSnapshot *frame)
{
    u8Image source;
    u8Image preview;
    u8Image gaussian;

    if ((g_status.applied == 0u) || (frame == NULL) ||
        (frame->data == NULL) || (frame->width != CORE4_VISION_WIDTH) ||
        (frame->height != CORE4_VISION_HEIGHT))
    {
        return 0u;
    }
    source.ptr = (uint8_t *) frame->data;
    source.xLen = frame->width;
    source.yLen = frame->height;
    source.size = CORE4_VISION_PIXELS;
    preview.ptr = g_preview;
    preview.xLen = frame->width;
    preview.yLen = frame->height;
    preview.size = CORE4_VISION_PIXELS;
    if (g_activeShared.detectMode == 1u)
    {
        gaussian.ptr = g_core4VisionWorkspace.scratch.canny.gaussian;
        gaussian.xLen = frame->width;
        gaussian.yLen = frame->height;
        gaussian.size = CORE4_VISION_PIXELS;
        cannyEdgeDetection(&source, &gaussian, &preview);
        (void) beaconDetect(&preview, &source, &g_detector);
    }
    else
    {
        (void) beaconCcDetect(&source, &preview, &g_detector);
    }
    beaconTrackUpdate(&g_tracker, &g_detector);
    publishObservation(frame);
    return 1u;
}

void core4VisionPipelineInvalidate(void)
{
    if (g_status.applied != 0u)
    {
        clearTracking();
    }
}

```

```

    g_status.applied = 0u;
    g_status.detectMode = 0u;
    memset(&g_activeShared, 0, sizeof(g_activeShared));
    g_sourceBootSession = 0u;
    g_ownerSession = 0u;
}

void core4VisionPipelineGetObservation(core4VisionObservation *observation)
{
    if (observation != NULL)
    {
        *observation = g_observation;
    }
}

void core4VisionPipelineGetStatus(core4VisionPipelineStatus *status)
{
    if (status != NULL)
    {
        *status = g_status;
    }
}

const uint8_t *core4VisionPipelineGetPreview(void)
{
    return g_preview;
}

```

core4_cyt4bb7/project/code/app/core4VisionPipeline.h

SHA-256

:

7e2e2854794acd4fd87ec467f670cb1618c28dab2e66a5176e7106557a85982f

代码 (c)

```

#ifndef CORE4_VISION_PIPELINE_H
#define CORE4_VISION_PIPELINE_H

#include <stdint.h>

#include "core4CameraService.h"
#include "observerLinkProtocol.h"
#include "core4VisionParams.h"
#include "beaconTrack.h"

#define CORE4_VISION_MAX_PUBLISHED_TRACKS (32u)

typedef struct
{
    uint16_t centerX;
    uint16_t centerY;
    uint16_t size;
    uint16_t trackId;
    uint16_t predX;
    uint16_t predY;
    uint16_t predSize;
    uint16_t misses;
    uint8_t fresh;
    uint8_t flags;
    float velX;
    float velY;
    float areaVel;
} core4VisionTrack;

typedef struct
{
    uint8_t valid;
    uint8_t sourceId;
    uint8_t detectMode;
    uint8_t maxBeacons;
    uint8_t truncated;
    beaconTrackEdgeBandsStruct edgeBands;
    beaconTrackBorderConfigStruct border;
    uint16_t confirmedTotal;
    uint16_t freshTotal;
}

```

```

uint16_t freshLeft;
uint16_t freshRight;
uint16_t retainedCount;
uint32_t sourceBootSession;
uint32_t ownerSession;
uint32_t configRevision;
uint32_t configHash;
uint32_t appFrameSeq;
uint32_t trackerFrameSeq;
uint32_t tickMs;
core4VisionTrack tracks[CORE4_VISION_MAX_PUBLISHED_TRACKS];
} core4VisionObservation;

typedef struct
{
    uint8_t ready;
    uint8_t applied;
    uint8_t detectMode;
    uint8_t padding;
} core4VisionPipelineStatus;

uint8_t core4VisionPipelineInit(void);
uint8_t core4VisionPipelineApplyAtFrameBoundary(
    const core4VisionParamsStruct *localParams,
    uint32_t localRevision,
    const observerLinkSharedConfig *sharedConfig,
    uint32_t sourceBootSession,
    uint32_t ownerSession);
uint8_t core4VisionPipelineProcess(const core4CameraSnapshot *frame);
void core4VisionPipelineSetSelectedTrackId(uint16_t trackId);
void core4VisionPipelineInvalidate(void);
void core4VisionPipelineGetObservation(core4VisionObservation *observation);
void core4VisionPipelineGetStatus(core4VisionPipelineStatus *status);
const uint8_t *core4VisionPipelineGetPreview(void);

#endif /* CORE4_VISION_PIPELINE_H */

```

core4_cyt4bb7/project/code/app/observerKvmBindingGuard.c

SHA-256

:

f103f3edf8549e101a9f14a6722be8922442d65196a6e196336f889c5600c3e5

代码 (c)

```

#include "observerKvmBindingGuard.h"

#include <string.h>

void observerKvmBindingGuardInit(observerKvmBindingGuard *guard)
{
    if (guard != 0) memset(guard, 0, sizeof(*guard));
}

observerKvmBindingAction observerKvmBindingGuardObserve(
    observerKvmBindingGuard *guard,
    uint32_t ownerSession,
    uint32_t bindingEpoch)
{
    if (guard == 0) return OBSERVER_KVM_BINDING_REBOUND;
    if (guard->primed == 0u)
    {
        guard->ownerSession = ownerSession;
        guard->bindingEpoch = bindingEpoch;
        guard->primed = 1u;
        return OBSERVER_KVM_BINDING_INITIAL;
    }
    if ((guard->ownerSession == ownerSession) &&
        (guard->bindingEpoch == bindingEpoch))
        return OBSERVER_KVM_BINDING_UNCHANGED;
    guard->ownerSession = ownerSession;
    guard->bindingEpoch = bindingEpoch;
    return OBSERVER_KVM_BINDING_REBOUND;
}

```


core4_cyt4bb7/project/code/app/observerKvmBindingGuard.h

SHA-256

:

92b8c965b3d809d4fe13be626ccaf26b8fa8ba4e83cf69bfc626d52893f00b5f

代码 (c)

```
#ifndef OBSERVER_KVM_BINDING_GUARD_H
#define OBSERVER_KVM_BINDING_GUARD_H

#include <stdint.h>

typedef enum
{
    OBSERVER_KVM_BINDING_UNCHANGED = 0u,
    OBSERVER_KVM_BINDING_INITIAL,
    OBSERVER_KVM_BINDING_REBOUND
} observerKvmBindingAction;

typedef struct
{
    uint32_t ownerSession;
    uint32_t bindingEpoch;
    uint8_t primed;
} observerKvmBindingGuard;

void observerKvmBindingGuardInit(observerKvmBindingGuard *guard);
observerKvmBindingAction observerKvmBindingGuardObserve(
    observerKvmBindingGuard *guard,
    uint32_t ownerSession,
    uint32_t bindingEpoch);

#endif /* OBSERVER_KVM_BINDING_GUARD_H */
```

core4_cyt4bb7/project/code/comm/core4CoreDataTransfer.c

SHA-256

:

a758997333f4fee475c0f7ecde6668743829363d472a0a5335376079e9e6cc1a

代码 (c)

```
#include "core4CoreDataTransfer.h"

#include <string.h>

#include "cy_device_headers.h"

/* Core-local live snapshot. It is deliberately not placed in shared RAM and
 * is only exposed to business code through the copy-out getter. */
static core4CoreDataDiagnostics g_core4CoreDataDiagnosticsLive;

static uint8_t core4CoreSequenceIsNewer(uint32_t candidate,
    uint32_t reference)
{
    uint32_t delta = candidate - reference;
    return (uint8_t) ((delta != 0u) && (delta < 0x80 000 000u));
}

static void saturatingIncrement(uint32_t *value)
{
    if (*value != UINT32_MAX) (*value)++;
}

static void saturatingAdd(uint32_t *value, uint32_t amount)
{
    if ((UINT32_MAX - *value) < amount) *value = UINT32_MAX;
    else *value += amount;
}

static void saturatingAdd64(uint64_t *value, uint32_t amount)
```

```

{
    if (((UINT64_MAX - *value) < amount) *value = UINT64_MAX;
        else *value += amount;
    }
}

void core4CoreDataTimingInit(void)
{
    uint32_t before;

    if ((g_core4CoreDataDiagnosticsLive.timingInitialized != 0u) &&
        (g_core4CoreDataDiagnosticsLive.timingValid != 0u)) return;
    g_core4CoreDataDiagnosticsLive.timingInitialized = 1u;
    g_core4CoreDataDiagnosticsLive.timingValid = 0u;
    if ((DWT->CTRL & DWT_CTRL_NOCYCCNT_Msk) != 0u) return;

    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    if ((DWT->CTRL & DWT_CTRL_CYCCNTENA_Msk) == 0u)
    {
        DWT->CYCCNT = 0u;
        DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
    }
    __DSB();
    __ISB();
    before = DWT->CYCCNT;
    __NOP();
    __NOP();
    __NOP();
    __NOP();
    if (((DWT->CTRL & DWT_CTRL_CYCCNTENA_Msk) != 0u) &&
        (DWT->CYCCNT != before))
    {
        g_core4CoreDataDiagnosticsLive.timingValid = 1u;
    }
}

static uint8_t timingBegin(uint32_t *startCycles)
{
    if ((startCycles == NULL) ||
        (g_core4CoreDataDiagnosticsLive.timingValid == 0u)) return 0u;
    if ((DWT->CTRL & DWT_CTRL_CYCCNTENA_Msk) == 0u)
    {
        g_core4CoreDataDiagnosticsLive.timingValid = 0u;
        return 0u;
    }
    *startCycles = DWT->CYCCNT;
    return 1u;
}

static void timingEnd(uint8_t valid, uint32_t startCycles)
{
    uint32_t elapsed;

    if (valid == 0u)
    {
        saturatingIncrement(
            &g_core4CoreDataDiagnosticsLive.timingInvalidSampleCount);
        return;
    }
    elapsed = DWT->CYCCNT - startCycles;
    if (((DWT->CTRL & DWT_CTRL_CYCCNTENA_Msk) == 0u) || (elapsed == 0u))
    {
        g_core4CoreDataDiagnosticsLive.timingValid = 0u;
        saturatingIncrement(
            &g_core4CoreDataDiagnosticsLive.timingInvalidSampleCount);
        return;
    }
    saturatingIncrement(&g_core4CoreDataDiagnosticsLive.timingSampleCount);
    saturatingAdd64(&g_core4CoreDataDiagnosticsLive.timingTotalCycles, elapsed);
    if (g_core4CoreDataDiagnosticsLive.timingMaxCycles < elapsed)
    {
        g_core4CoreDataDiagnosticsLive.timingMaxCycles = elapsed;
    }
}

/* Each image owns a cache-line-isolated retained pair in core-local RAM. */
typedef struct
{
    uint32_t counter;
    uint32_t counterInverse;
}

```

```

    uint8_t padding[24];
} core4CoreBootState;

#pragma data_alignment = 32
__no_init static core4CoreBootState core4CoreBootStateRetained;

#pragma location = CORE4_CORE_LINK_REGION_ADDR
__no_init volatile core4CoreLinkSnapshot core4CoreLinkRegion;

#pragma location = CORE4_CORE_VISION_REGION_ADDR
__no_init volatile core4CoreVisionSnapshot core4CoreVisionRegion;

#define CORE4_CORE_QUEUE_DEPTH (4u)
#define CORE4_CORE_QUEUE_MASK (CORE4_CORE_QUEUE_DEPTH - 1u)

typedef struct
{
    uint32_t producerBootGeneration;
    uint32_t sequence;
    uint32_t payloadBytes;
    uint32_t commit;
    uint8_t payload[112];
} core4CoreCommittedSlot;

typedef struct
{
    volatile uint32_t producerBootGeneration;
    volatile uint32_t head;
    uint8_t producerPadding[24];
    volatile uint32_t consumerBootGeneration;
    volatile uint32_t tail;
    uint8_t consumerPadding[24];
    core4CoreCommittedSlot slots[CORE4_CORE_QUEUE_DEPTH];
} core4CoreQueueRegion;

#define CORE4_CORE_QUEUE_COMMIT (0xC45A0000u)

typedef char core4Core_queue_depth_assert[
    ((CORE4_CORE_QUEUE_DEPTH & CORE4_CORE_QUEUE_MASK) == 0u) ? 1 : -1];
typedef char core4Core_queue_region_size_assert[
    (sizeof(core4CoreQueueRegion) == 0x240u) ? 1 : -1];
typedef char core4Core_queue_slot_size_assert[
    (sizeof(core4CoreCommittedSlot) == 128u) ? 1 : -1];
typedef char core4Core_ingress_slot_size_assert[
    (sizeof(core4CoreIngressEvent) <= 112u) ? 1 : -1];
typedef char core4Core_egress_slot_size_assert[
    (sizeof(core4CoreEgressEvent) <= 112u) ? 1 : -1];

#pragma location = CORE4_CORE_INGRESS_QUEUE_ADDR
__no_init volatile core4CoreQueueRegion core4CoreIngressQueue;

#pragma location = CORE4_CORE_EGRESS_QUEUE_ADDR
__no_init volatile core4CoreQueueRegion core4CoreEgressQueue;

static uint8_t snapshotStable(const void *first, const void *second,
    uint32_t size, uint32_t magic, uint32_t begin, uint32_t end)
{
    const core4CoreLinkSnapshot *header = (const core4CoreLinkSnapshot *) second;
    return (uint8_t) ((memcmp(first, second, size) == 0) &&
        (header->magic == magic) &&
        (header->abiVersion == CORE4_CORE_ABI_VERSION) &&
        (header->payloadBytes == size) &&
        (begin != 0u) && (begin == end));
}

static uint8_t probeSnapshot(volatile core4CoreLinkSnapshot *region,
    uint32_t magic, uint32_t payloadBytes, uint32_t *bootGeneration,
    uint32_t *publishSeq, uint32_t *probeCount)
{
    uint32_t begin;
    uint32_t observedMagic;
    uint32_t abiVersion;
    uint32_t observedBytes;
    uint32_t boot;
    uint32_t seq;

    if ((bootGeneration == NULL) || (publishSeq == NULL)) return 0u;
    saturatingIncrement(probeCount);
    SCB_InvalidateDCache_by_Addr((volatile void *) region, 32u);

```

```

begin = region->beginGeneration;
observedMagic = region->magic;
abiVersion = region->abiVersion;
observedBytes = region->payloadBytes;
boot = region->bootGeneration;
seq = region->publishSeq;
if ((begin == 0u) || (observedMagic != magic) ||
    (abiVersion != CORE4_CORE_ABI_VERSION) ||
    (observedBytes != payloadBytes) || (boot == 0u) || (seq == 0u))
{
    return 0u;
}
*bootGeneration = boot;
*publishSeq = seq;
return 1u;
}

uint32_t core4CoreDataMakeBootGeneration(uint32_t salt)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    uint32_t generation;

    SCB_InvalidateDCache_by_Addr(
        (volatile void *) &core4CoreBootStateRetained,
        sizeof(core4CoreBootStateRetained));
    if (core4CoreBootStateRetained.counterInverse !=
        ~core4CoreBootStateRetained.counter)
    {
        core4CoreBootStateRetained.counter =
            DWT->CYCCNT ^ salt ^ 0x6D2B79F5u;
    }
    core4CoreBootStateRetained.counter++;
    if (core4CoreBootStateRetained.counter == 0u)
    {
        core4CoreBootStateRetained.counter = 1u;
    }
    core4CoreBootStateRetained.counterInverse =
        ~core4CoreBootStateRetained.counter;
    SCB_CleanDCache_by_Addr(
        (volatile void *) &core4CoreBootStateRetained,
        sizeof(core4CoreBootStateRetained));
    __DSB();
    generation = core4CoreBootStateRetained.counter;
    timingEnd(timingValid, startCycles);
    return generation;
}

uint8_t core4CoreDataProbeLink(uint32_t *bootGeneration, uint32_t *publishSeq)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    uint8_t result = probeSnapshot(&core4CoreLinkRegion, CORE4_CORE_LINK_MAGIC,
        sizeof(core4CoreLinkRegion), bootGeneration, publishSeq,
        &g_core4CoreDataDiagnosticsLive.linkProbeCount);
    timingEnd(timingValid, startCycles);
    return result;
}

uint8_t core4CoreDataProbeVision(uint32_t *bootGeneration, uint32_t *publishSeq)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    uint8_t result = probeSnapshot(
        (volatile core4CoreLinkSnapshot *) &core4CoreVisionRegion,
        CORE4_CORE_VISION_MAGIC, sizeof(core4CoreVisionRegion), bootGeneration,
        publishSeq, &g_core4CoreDataDiagnosticsLive.visionProbeCount);
    timingEnd(timingValid, startCycles);
    return result;
}

void core4CoreDataWriteLink(const core4CoreLinkSnapshot *snapshot)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    if (snapshot == NULL)
    {
        timingEnd(timingValid, startCycles);
        return;
    }

```

```

    }
    memcpy((void *) &core4CoreLinkRegion, snapshot, sizeof(*snapshot));
    __DMB();
    SCB_CleanDCache_by_Addr((volatile void *) &core4CoreLinkRegion,
        sizeof(core4CoreLinkRegion));
    __DSB();
    saturatingIncrement(&g_core4CoreDataDiagnosticsLive.linkPublishCount);
    saturatingAdd(&g_core4CoreDataDiagnosticsLive.linkPublishBytes,
        sizeof(core4CoreLinkRegion));
    timingEnd(timingValid, startCycles);
}

uint8_t core4CoreDataTryReadLink(core4CoreLinkSnapshot *snapshot)
{
    core4CoreLinkSnapshot first;
    core4CoreLinkSnapshot second;
    uint8_t attempt;
    uint8_t result = 0u;
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    if (snapshot == NULL)
    {
        timingEnd(timingValid, startCycles);
        return 0u;
    }
    saturatingIncrement(&g_core4CoreDataDiagnosticsLive.linkReadAttempts);
    for (attempt = 0u; attempt < 3u; attempt++)
    {
        SCB_InvalidateDCache_by_Addr((volatile void *) &core4CoreLinkRegion,
            sizeof(core4CoreLinkRegion));
        memcpy(&first, (const void *) &core4CoreLinkRegion, sizeof(first));
        SCB_InvalidateDCache_by_Addr((volatile void *) &core4CoreLinkRegion,
            sizeof(core4CoreLinkRegion));
        memcpy(&second, (const void *) &core4CoreLinkRegion, sizeof(second));
        if (snapshotStable(&first, &second, sizeof(second), CORE4_CORE_LINK_MAGIC,
            second.beginGeneration, second.endGeneration))
        {
            *snapshot = second;
            result = 1u;
            break;
        }
        saturatingIncrement(&g_core4CoreDataDiagnosticsLive.linkReadRetries);
    }
    if (result == 0u)
    {
        saturatingIncrement(&g_core4CoreDataDiagnosticsLive.linkReadFailures);
    }
    timingEnd(timingValid, startCycles);
    return result;
}

void core4CoreDataWriteVision(const core4CoreVisionSnapshot *snapshot)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    if (snapshot == NULL)
    {
        timingEnd(timingValid, startCycles);
        return;
    }
    memcpy((void *) &core4CoreVisionRegion, snapshot, sizeof(*snapshot));
    __DMB();
    SCB_CleanDCache_by_Addr((volatile void *) &core4CoreVisionRegion,
        sizeof(core4CoreVisionRegion));
    __DSB();
    saturatingIncrement(&g_core4CoreDataDiagnosticsLive.visionPublishCount);
    saturatingAdd(&g_core4CoreDataDiagnosticsLive.visionPublishBytes,
        sizeof(core4CoreVisionRegion));
    timingEnd(timingValid, startCycles);
}

uint8_t core4CoreDataTryReadVision(core4CoreVisionSnapshot *snapshot)
{
    core4CoreVisionSnapshot first;
    core4CoreVisionSnapshot second;
    uint8_t attempt;
    uint8_t result = 0u;
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);

```

```

if (snapshot == NULL)
{
    timingEnd(timingValid, startCycles);
    return 0u;
}
saturatingIncrement(&g_core4CoreDataDiagnosticsLive.visionReadAttempts);
for (attempt = 0u; attempt < 3u; attempt++)
{
    SCB_InvalidateDCache_by_Addr((volatile void *) &core4CoreVisionRegion,
        sizeof(core4CoreVisionRegion));
    memcpy(&first, (const void *) &core4CoreVisionRegion, sizeof(first));
    SCB_InvalidateDCache_by_Addr((volatile void *) &core4CoreVisionRegion,
        sizeof(core4CoreVisionRegion));
    memcpy(&second, (const void *) &core4CoreVisionRegion, sizeof(second));
    if (snapshotStable(&first, &second, sizeof(second), CORE4_CORE_VISION_MAGIC,
        second.beginGeneration, second.endGeneration))
    {
        *snapshot = second;
        result = 1u;
        break;
    }
    saturatingIncrement(&g_core4CoreDataDiagnosticsLive.visionReadRetries);
}
if (result == 0u)
{
    saturatingIncrement(&g_core4CoreDataDiagnosticsLive.visionReadFailures);
}
timingEnd(timingValid, startCycles);
return result;
}

static uint8_t queuePush(volatile core4CoreQueueRegion *queue,
    const void *event, uint32_t eventSize, uint32_t *fullCount,
    uint32_t *highWater)
{
    uint32_t head;
    uint32_t tail;
    uint32_t producerBootGeneration;
    volatile core4CoreCommittedSlot *slot;
    SCB_InvalidateDCache_by_Addr(
        (volatile void *) &queue->consumerBootGeneration, 32u);
    producerBootGeneration = queue->producerBootGeneration;
    if (producerBootGeneration == 0u) return 0u;
    head = queue->head;
    tail = (queue->consumerBootGeneration == producerBootGeneration) ?
        queue->tail : head;
    if ((uint32_t) (head - tail) >= CORE4_CORE_QUEUE_DEPTH)
    {
        saturatingIncrement(fullCount);
        return 0u;
    }
    slot = &queue->slots[head & CORE4_CORE_QUEUE_MASK];
    memset((void *) slot, 0, sizeof(*slot));
    slot->producerBootGeneration = producerBootGeneration;
    slot->sequence = head + 1u;
    slot->payloadBytes = eventSize;
    memcpy((void *) slot->payload, event, eventSize);
    SCB_CleanDCache_by_Addr((volatile void *) slot, sizeof(*slot));
    __DSB();
    slot->commit = CORE4_CORE_QUEUE_COMMIT ^ producerBootGeneration ^ (head + 1u);
    SCB_CleanDCache_by_Addr((volatile void *) slot, 32u);
    __DSB();
    queue->head = head + 1u;
    SCB_CleanDCache_by_Addr(
        (volatile void *) &queue->producerBootGeneration, 32u);
    __DSB();
    if (*highWater < ((uint32_t) (head - tail) + 1u))
    {
        *highWater = (uint32_t) (head - tail) + 1u;
    }
    return 1u;
}

static void queueReset(volatile core4CoreQueueRegion *queue,
    uint32_t producerBootGeneration)
{
    queue->head = 0u;
    queue->producerBootGeneration = producerBootGeneration;
    SCB_CleanDCache_by_Addr(

```

```

        (volatile void *) &queue->producerBootGeneration, 32u);
__DSB();
}

static void queueSyncConsumer(volatile core4CoreQueueRegion *queue,
uint32_t producerBootGeneration)
{
    SCB_InvalidateDCache_by_Addr(
        (volatile void *) &queue->producerBootGeneration, 32u);
    if (queue->producerBootGeneration != producerBootGeneration) return;
    queue->tail = queue->head;
    queue->consumerBootGeneration = producerBootGeneration;
    SCB_CleanDCache_by_Addr(
        (volatile void *) &queue->consumerBootGeneration, 32u);
    __DSB();
}

void core4CoreDataResetIngress(uint32_t producerBootGeneration)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    queueReset(&core4CoreIngressQueue, producerBootGeneration);
    timingEnd(timingValid, startCycles);
}

void core4CoreDataResetEgress(uint32_t producerBootGeneration)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    queueReset(&core4CoreEgressQueue, producerBootGeneration);
    timingEnd(timingValid, startCycles);
}

void core4CoreDataSyncIngressConsumer(uint32_t producerBootGeneration)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    queueSyncConsumer(&core4CoreIngressQueue, producerBootGeneration);
    timingEnd(timingValid, startCycles);
}

void core4CoreDataSyncEgressConsumer(uint32_t producerBootGeneration)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    queueSyncConsumer(&core4CoreEgressQueue, producerBootGeneration);
    timingEnd(timingValid, startCycles);
}

static uint8_t queuePop(volatile core4CoreQueueRegion *queue,
void *event, uint32_t eventSize, uint32_t producerBootGeneration)
{
    uint32_t head;
    uint32_t tail;
    uint32_t expectedCommit;
    volatile core4CoreCommittedSlot *slot;
    SCB_InvalidateDCache_by_Addr(
        (volatile void *) &queue->producerBootGeneration, 32u);
    if ((queue->producerBootGeneration != producerBootGeneration) ||
        (queue->consumerBootGeneration != producerBootGeneration)) return 0u;
    head = queue->head;
    tail = queue->tail;
    if (tail == head) return 0u;
    if (((uint32_t) (head - tail) > CORE4_CORE_QUEUE_DEPTH) return 0u;
    slot = &queue->slots[tail & CORE4_CORE_QUEUE_MASK];
    SCB_InvalidateDCache_by_Addr((volatile void *) slot, sizeof(*slot));
    expectedCommit = CORE4_CORE_QUEUE_COMMIT ^ producerBootGeneration ^
        (tail + 1u);
    if ((slot->producerBootGeneration != producerBootGeneration) ||
        (slot->sequence != (tail + 1u)) ||
        (slot->payloadBytes != eventSize) ||
        (slot->commit != expectedCommit)) return 0u;
    memcpy(event, (const void *) slot->payload, eventSize);
    __DMB();
    queue->tail = tail + 1u;
    SCB_CleanDCache_by_Addr(
        (volatile void *) &queue->consumerBootGeneration, 32u);
    __DSB();
    return 1u;
}

```

```

}

uint8_t core4CoreDataPushIngress(const core4CoreIngressEvent *event)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    uint8_t result = 0u;
    if (event != NULL)
    {
        result = queuePush(&core4CoreIngressQueue, event, sizeof(*event),
            &g_core4CoreDataDiagnosticsLive.ingressQueueFull,
            &g_core4CoreDataDiagnosticsLive.ingressQueueHighWater);
    }
    timingEnd(timingValid, startCycles);
    return result;
}

uint8_t core4CoreDataPopIngress(core4CoreIngressEvent *event,
    uint32_t producerBootGeneration)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    uint8_t result = 0u;
    if (event != NULL)
    {
        result = queuePop(&core4CoreIngressQueue, event, sizeof(*event),
            producerBootGeneration);
    }
    timingEnd(timingValid, startCycles);
    return result;
}

uint8_t core4CoreDataPushEgress(const core4CoreEgressEvent *event)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    uint8_t result = 0u;
    if (event != NULL)
    {
        result = queuePush(&core4CoreEgressQueue, event, sizeof(*event),
            &g_core4CoreDataDiagnosticsLive.egressQueueFull,
            &g_core4CoreDataDiagnosticsLive.egressQueueHighWater);
    }
    timingEnd(timingValid, startCycles);
    return result;
}

uint8_t core4CoreDataPopEgress(core4CoreEgressEvent *event,
    uint32_t producerBootGeneration)
{
    uint32_t startCycles;
    uint8_t timingValid = timingBegin(&startCycles);
    uint8_t result = 0u;
    if (event != NULL)
    {
        result = queuePop(&core4CoreEgressQueue, event, sizeof(*event),
            producerBootGeneration);
    }
    timingEnd(timingValid, startCycles);
    return result;
}

uint8_t core4CoreBorderRejectionMerge(
    const core4CoreBorderRejection *pending,
    observerLinkPlanningTrackSnapshot *snapshot)
{
    uint8_t delivered = 0u;
    uint8_t i;
    if ((pending == NULL) || (snapshot == NULL) ||
        (pending->rejectedCount == 0u) ||
        (snapshot->ownerSession != pending->ownerSession) ||
        (snapshot->revision != pending->revision) ||
        (snapshot->hash != pending->hash) ||
        ((snapshot->trackerFrameSeq != pending->trackerFrameSeq) &&
            (core4CoreSequenceIsNewer(snapshot->trackerFrameSeq,
                pending->trackerFrameSeq) == 0u))) return 0u;
    for (i = 0u; i < pending->rejectedCount; i++)
    {
        uint16_t rejectedId = pending->rejectedTrackIds[i];

```



```

uint8_t j;
uint8_t replacement = OBSERVER_LINK_MAX_TRACKS;
for (j = 0u; j < snapshot->retainedCount; j++)
{
    if (snapshot->tracks[j].trackId == rejectedId)
    {
        memset(&snapshot->tracks[j], 0,
            sizeof(snapshot->tracks[j]));
        snapshot->tracks[j].trackId = rejectedId;
        snapshot->tracks[j].flags =
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED;
        break;
    }
    if ((snapshot->tracks[j].flags &
        OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) == 0u)
        replacement = j;
}
if (j == snapshot->retainedCount)
{
    if (snapshot->retainedCount < OBSERVER_LINK_MAX_TRACKS)
    {
        j = snapshot->retainedCount++;
    }
    else if (replacement < OBSERVER_LINK_MAX_TRACKS)
    {
        j = replacement;
        snapshot->truncated = 1u;
    }
    else
    {
        break;
    }
    memset(&snapshot->tracks[j], 0, sizeof(snapshot->tracks[j]));
    snapshot->tracks[j].trackId = rejectedId;
    snapshot->tracks[j].flags =
        OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED;
}
delivered++;
}
return delivered;
}

uint8_t core4CoreBorderRejectionAppend(core4CoreBorderRejection *pending,
uint8_t *pendingValid, const core4CoreBorderRejection *event)
{
    uint8_t uniqueAdded = 0u;
    uint8_t i;
    if ((pending == NULL) || (pendingValid == NULL) || (event == NULL) ||
        (event->ownerSession == 0u) || (event->rejectedCount == 0u) ||
        (event->rejectedCount > CORE4_CORE_REJECTION_TRACK_MAX)) return 0u;
    for (i = 0u; i < event->rejectedCount; i++)
    {
        uint8_t j;
        if (event->rejectedTrackIds[i] == 0u) return 0u;
        for (j = 0u; j < i; j++)
            if (event->rejectedTrackIds[j] ==
                event->rejectedTrackIds[i]) return 0u;
    }
    if ((*pendingValid == 0u) ||
        (pending->ownerSession != event->ownerSession) ||
        (pending->revision != event->revision) ||
        (pending->hash != event->hash))
    {
        *pending = *event;
        *pendingValid = 1u;
        return 1u;
    }
    for (i = 0u; i < event->rejectedCount; i++)
    {
        uint8_t j;
        for (j = 0u; j < pending->rejectedCount; j++)
            if (pending->rejectedTrackIds[j] ==
                event->rejectedTrackIds[i]) break;
        if (j == pending->rejectedCount) uniqueAdded++;
    }
    if ((uint16_t) pending->rejectedCount + uniqueAdded >
        CORE4_CORE_REJECTION_TRACK_MAX) return 0u;
    for (i = 0u; i < event->rejectedCount; i++)
    {

```

```

        uint8_t j;
        for (j = 0u; j < pending->rejectedCount; j++)
            if (pending->rejectedTrackIds[j] ==
                event->rejectedTrackIds[i]) break;
        if (j == pending->rejectedCount)
            pending->rejectedTrackIds[pending->rejectedCount++] =
                event->rejectedTrackIds[i];
    }
    if (core4CoreSequenceIsNewer(event->trackerFrameSeq,
        pending->trackerFrameSeq) != 0u)
        pending->trackerFrameSeq = event->trackerFrameSeq;
    return 1u;
}

uint8_t core4CorePlanningPreserveRejections(
    observerLinkPlanningTrackSnapshot *destination,
    const observerLinkPlanningTrackSnapshot *pending)
{
    uint8_t i;
    if ((destination == NULL) || (pending == NULL)) return 0u;
    for (i = 0u; i < pending->retainedCount; i++)
    {
        const observerLinkPlanningTrack *rejected = &pending->tracks[i];
        uint8_t j;
        uint8_t replacement = OBSERVER_LINK_MAX_TRACKS;
        if ((rejected->flags &
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) == 0u) continue;
        for (j = 0u; j < destination->retainedCount; j++)
        {
            if (destination->tracks[j].trackId == rejected->trackId)
            {
                destination->tracks[j] = *rejected;
                replacement = j;
                break;
            }
            if ((destination->tracks[j].flags &
                OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) == 0u)
                replacement = j;
        }
        if (j < destination->retainedCount) continue;
        if (destination->retainedCount < OBSERVER_LINK_MAX_TRACKS)
            destination->tracks[destination->retainedCount++] = *rejected;
        else if (replacement < OBSERVER_LINK_MAX_TRACKS)
        {
            destination->tracks[replacement] = *rejected;
            destination->truncated = 1u;
        }
        else return 0u;
    }
    return 1u;
}

void core4CoreDataNoteSharedStale(void)
{
    saturatingIncrement(&g_core4CoreDataDiagnosticsLive.sharedStaleCount);
}

void core4CoreDataNoteSessionReset(void)
{
    saturatingIncrement(&g_core4CoreDataDiagnosticsLive.sessionResetCount);
}

```

core4_cyt4bb7/project/code/comm/core4CoreDataTransfer.h

SHA-256

:

4a91ee5a0aaf32a1b4a8ec16a0b3d4cdc2d40b6a508c45d9eea9ae87b05a8d65

代码 (c)

```

#ifndef CORE4_CORE_DATA_TRANSFER_H
#define CORE4_CORE_DATA_TRANSFER_H

#include <stdint.h>

#include "core4LinkCore4.h"

```

```

#include "paramSyncMailbox.h"

#define CORE4_CORE_LINK_REGION_ADDR (0x28 001 000u)
#define CORE4_CORE_INGRESS_QUEUE_ADDR (0x28 001 400u)
#define CORE4_CORE_EGRESS_QUEUE_ADDR (0x28 001 800u)
#define CORE4_CORE_VISION_REGION_ADDR (0x28 002 000u)
#define CORE4_CORE_LINK_LIMIT_BYTES (0x0400u)
#define CORE4_CORE_QUEUE_LIMIT_BYTES (0x0400u)
#define CORE4_CORE_VISION_LIMIT_BYTES (0x1000u)
#define CORE4_CORE_LINK_MAGIC (0x43 344C4Bu) /* C4LK */
#define CORE4_CORE_VISION_MAGIC (0x43 345 649u) /* C4VI */
#define CORE4_CORE_ABI_VERSION (7u)

typedef struct
{
    uint32_t beginGeneration;
    uint32_t magic;
    uint32_t abiVersion;
    uint32_t payloadBytes;
    uint32_t bootGeneration;
    uint32_t publishSeq;
    core4LinkCore4Status status;
    observerLinkSharedConfig stagedConfig;
    uint32_t stagedConfigSeq;
    uint32_t appliedConsumedSeq;
    uint32_t paramGlobalStateSeq;
    paramSyncGlobalState paramGlobalState;
    uint32_t endGeneration;
} core4CoreLinkSnapshot;

typedef struct
{
    uint32_t beginGeneration;
    uint32_t magic;
    uint32_t abiVersion;
    uint32_t payloadBytes;
    uint32_t bootGeneration;
    uint32_t publishSeq;
    uint32_t visionReadySeq;
    uint8_t visionReady;
    uint8_t padding0[3];
    uint32_t appliedSeq;
    uint32_t appliedOwnerSession;
    uint32_t appliedRevision;
    uint32_t appliedHash;
    uint8_t appliedMode;
    uint8_t padding1[3];
    uint32_t planningSeq;
    observerLinkPlanningTrackSnapshot planningSnapshot;
    uint32_t healthSeq;
    observerLinkHealth health;
    uint32_t paramGlobalRequestSeq;
    paramSyncGlobalRequest paramGlobalRequest;
    uint32_t paramGlobalStateConsumedSeq;
    uint32_t endGeneration;
} core4CoreVisionSnapshot;

typedef enum
{
    CORE4_CORE_INGRESS_KVM_CONTROL = 1,
    CORE4_CORE_INGRESS_KVM_EVENT = 2
} core4CoreIngressKind;

typedef struct
{
    uint32_t kind;
    union
    {
        {
            observerLinkKvmControl control;
            observerLinkKvmEvent event;
        } payload;
    }
} core4CoreIngressEvent;

typedef enum
{
    CORE4_CORE_EGRESS_KVM_ACK = 1,
    CORE4_CORE_EGRESS_KVM_MENU = 2,
    CORE4_CORE_EGRESS_BORDER_REJECTION = 3
} core4CoreEgressKind;

```

```

#define CORE4_CORE_REJECTION_TRACK_MAX (44u)

typedef struct
{
    uint32_t ownerSession;
    uint32_t revision;
    uint32_t hash;
    uint32_t trackerFrameSeq;
    uint8_t rejectedCount;
    uint8_t padding[3];
    uint16_t rejectedTrackIds[CORE4_CORE_REJECTION_TRACK_MAX];
} core4CoreBorderRejection;

typedef struct
{
    uint32_t kind;
    union
    {
        {
            observerLinkKvmAck ack;
            observerLinkKvmMenuStatus menuStatus;
            core4CoreBorderRejection borderRejection;
        } payload;
    }
} core4CoreEgressEvent;

typedef struct
{
    {
        uint8_t timingInitialized;
        uint8_t timingValid;
        uint8_t timingPadding[2];
        uint32_t timingSampleCount;
        uint32_t timingInvalidSampleCount;
        uint64_t timingTotalCycles;
        uint32_t timingMaxCycles;
        uint32_t linkProbeCount;
        uint32_t visionProbeCount;
        uint32_t linkReadAttempts;
        uint32_t linkReadRetries;
        uint32_t linkReadFailures;
        uint32_t visionReadAttempts;
        uint32_t visionReadRetries;
        uint32_t visionReadFailures;
        uint32_t linkPublishCount;
        uint32_t linkPublishBytes;
        uint32_t visionPublishCount;
        uint32_t visionPublishBytes;
        uint32_t ingressQueueFull;
        uint32_t ingressQueueHighWater;
        uint32_t egressQueueFull;
        uint32_t egressQueueHighWater;
        uint32_t sharedStaleCount;
        uint32_t sessionResetCount;
    }
} core4CoreDataDiagnostics;

typedef char core4Core_link_region_size_assert[
    (sizeof(core4CoreLinkSnapshot) <= CORE4_CORE_LINK_LIMIT_BYTES) ? 1 : -1];
typedef char core4Core_vision_region_size_assert[
    (sizeof(core4CoreVisionSnapshot) <= CORE4_CORE_VISION_LIMIT_BYTES) ? 1 : -1];
typedef char core4Core_link_region_align_assert[
    ((CORE4_CORE_LINK_REGION_ADDR & 31u) == 0u) ? 1 : -1];
typedef char core4Core_ingress_region_align_assert[
    ((CORE4_CORE_INGRESS_QUEUE_ADDR & 31u) == 0u) ? 1 : -1];
typedef char core4Core_egress_region_align_assert[
    ((CORE4_CORE_EGRESS_QUEUE_ADDR & 31u) == 0u) ? 1 : -1];
typedef char core4Core_vision_region_align_assert[
    ((CORE4_CORE_VISION_REGION_ADDR & 31u) == 0u) ? 1 : -1];
typedef char core4Core_link_before_ingress_assert[
    ((CORE4_CORE_LINK_REGION_ADDR + CORE4_CORE_LINK_LIMIT_BYTES) <=
    CORE4_CORE_INGRESS_QUEUE_ADDR) ? 1 : -1];
typedef char core4Core_ingress_before_egress_assert[
    ((CORE4_CORE_INGRESS_QUEUE_ADDR + CORE4_CORE_QUEUE_LIMIT_BYTES) <=
    CORE4_CORE_EGRESS_QUEUE_ADDR) ? 1 : -1];
typedef char core4Core_egress_before_param_sync_assert[
    ((CORE4_CORE_EGRESS_QUEUE_ADDR + CORE4_CORE_QUEUE_LIMIT_BYTES) <=
    PARAM_SYNC_MAILBOX_REGION_ADDR) ? 1 : -1];
typedef char core4Core_param_sync_before_vision_assert[
    ((PARAM_SYNC_MAILBOX_REGION_ADDR + PARAM_SYNC_MAILBOX_REGION_BYTES) <=
    CORE4_CORE_VISION_REGION_ADDR) ? 1 : -1];
typedef char core4Core_vision_region_end_assert[

```

```

((CORE4_CORE_VISION_REGION_ADDR + CORE4_CORE_VISION_LIMIT_BYTES) <=
0x28003000u) ? 1 : -1];

void core4CoreDataTimingInit(void);
uint32_t core4CoreDataMakeBootGeneration(uint32_t salt);
uint8_t core4CoreDataProbeLink(uint32_t *bootGeneration, uint32_t *publishSeq);
uint8_t core4CoreDataProbeVision(uint32_t *bootGeneration, uint32_t *publishSeq);
void core4CoreDataWriteLink(const core4CoreLinkSnapshot *snapshot);
uint8_t core4CoreDataTryReadLink(core4CoreLinkSnapshot *snapshot);
void core4CoreDataWriteVision(const core4CoreVisionSnapshot *snapshot);
uint8_t core4CoreDataTryReadVision(core4CoreVisionSnapshot *snapshot);
void core4CoreDataResetIngress(uint32_t producerBootGeneration);
void core4CoreDataResetEgress(uint32_t producerBootGeneration);
void core4CoreDataSyncIngressConsumer(uint32_t producerBootGeneration);
void core4CoreDataSyncEgressConsumer(uint32_t producerBootGeneration);
uint8_t core4CoreDataPushIngress(const core4CoreIngressEvent *event);
uint8_t core4CoreDataPopIngress(core4CoreIngressEvent *event,
    uint32_t producerBootGeneration);
uint8_t core4CoreDataPushEgress(const core4CoreEgressEvent *event);
uint8_t core4CoreDataPopEgress(core4CoreEgressEvent *event,
    uint32_t producerBootGeneration);
uint8_t core4CoreBorderRejectionMerge(
    const core4CoreBorderRejection *pending,
    observerLinkPlanningTrackSnapshot *snapshot);
uint8_t core4CoreBorderRejectionAppend(core4CoreBorderRejection *pending,
    uint8_t *pendingValid, const core4CoreBorderRejection *event);
uint8_t core4CorePlanningPreserveRejections(
    observerLinkPlanningTrackSnapshot *destination,
    const observerLinkPlanningTrackSnapshot *pending);
void core4CoreDataNoteSharedStale(void);
void core4CoreDataNoteSessionReset(void);
#endif /* CORE4_CORE_DATA_TRANSFER_H */

```

core4_cyt4bb7/project/code/comm/core4KvmCore4.c

SHA-256

:

eb784c0425812e642e5acb7604768f035f47a7a8e088c748c481b729b2799d03

代码 (c)

```

#include "core4KvmCore4.h"

#include <stddef.h>
#include <string.h>

#define CORE4_KVM_KEY_COUNT    (5u)
#define CORE4_KVM_KEY_STATE_COUNT (2u)
#define CORE4_KVM_KEY1        (0u)
#define CORE4_KVM_LONG_PRESS  (1u)
#define CORE4_KVM_ACTION_SAVE  (1u)

typedef struct
{
    core4KvmCore4Status publicStatus;
    observerLinkKvmEvent event;
    observerLinkKvmAck ack;
    observerLinkKvmAck completedAck;
    uint32_t lastEventSeq;
    uint32_t screenRevision;
    uint8_t ackReady;
    uint8_t completedAckValid;
    uint8_t menuState;
    uint8_t menuFlags;
    uint8_t lastAckResult;
} core4KvmState;

static core4KvmState g_kvm;

static uint8_t deadlineReached(uint32_t nowMs, uint32_t deadlineMs)
{
    return (uint8_t) (((int32_t) (nowMs - deadlineMs)) >= 0);
}

static uint32_t nextEventSeq(uint32_t value)
{

```

```

        value++;
        return (value != 0u) ? value : 1u;
    }

static uint8_t eventSeqNewer(uint32_t value, uint32_t reference)
{
    return (uint8_t)((value != reference) &&
        ((uint32_t)(value - reference) < 0x80 000 000u));
}

static uint8_t routeMatchesEvent(const observerLinkKvmEvent *event)
{
    return (uint8_t) ((event->routeSession == g_kvm.publicStatus.routeSession) &&
        (event->routeEpoch == g_kvm.publicStatus.routeEpoch));
}

static uint8_t sameCompletedEvent(const observerLinkKvmEvent *event)
{
    return (uint8_t) ((g_kvm.completedAckValid != 0u) &&
        (event->targetSession == g_kvm.completedAck.targetSession) &&
        (event->routeSession == g_kvm.completedAck.routeSession) &&
        (event->routeEpoch == g_kvm.completedAck.routeEpoch) &&
        (event->eventSeq == g_kvm.completedAck.eventSeq) &&
        (event->target == g_kvm.completedAck.target));
}

static void updatePublicPending(void)
{
    g_kvm.publicStatus.ackPending = g_kvm.ackReady;
}

static void prepareAck(const observerLinkKvmEvent *event, uint8_t result)
{
    memset(&g_kvm.ack, 0, sizeof(g_kvm.ack));
    g_kvm.ack.targetSession = g_kvm.publicStatus.localSession;
    g_kvm.ack.routeSession = event->routeSession;
    g_kvm.ack.routeEpoch = event->routeEpoch;
    g_kvm.ack.eventSeq = event->eventSeq;
    g_kvm.ack.expectedSeq = g_kvm.publicStatus.expectedSeq;
    g_kvm.ack.target = event->target;
    g_kvm.ack.result = result;
    g_kvm.ack.menuState = g_kvm.menuState;
    g_kvm.ackReady = 1u;
    updatePublicPending();
}

static void clearRouteState(void)
{
    g_kvm.publicStatus.expectedSeq = 1u;
    g_kvm.publicStatus.eventPending = 0u;
    g_kvm.ackReady = 0u;
    g_kvm.completedAckValid = 0u;
    g_kvm.lastEventSeq = 0u;
    g_kvm.lastAckResult = OBSERVER_LINK_KVM_RESULT_NONE;
    updatePublicPending();
}

void core4KvmCore4Init(uint32_t localSession)
{
    memset(&g_kvm, 0, sizeof(g_kvm));
    g_kvm.publicStatus.localSession = localSession;
    g_kvm.publicStatus.expectedSeq = 1u;
}

void core4KvmCore4Poll(uint32_t tickMs)
{
    if ((g_kvm.publicStatus.leaseValid != 0u) &&
        (deadlineReached(tickMs, g_kvm.publicStatus.leaseDeadlineMs) != 0u))
    {
        g_kvm.publicStatus.leaseValid = 0u;
        g_kvm.menuFlags &= (uint8_t) ~OBSERVER_LINK_KVM_MENU_FLAG_LEASE_VALID;
    }
}

void core4KvmCore4Revoke(void)
{
    g_kvm.publicStatus.routeSession = 0u;
    g_kvm.publicStatus.routeEpoch = 0u;
    g_kvm.publicStatus.leaseDeadlineMs = 0u;
}

```

```

    g_kvm.publicStatus.leaseValid = 0u;
    g_kvm.menuFlags &= (uint8_t) ~OBSERVER_LINK_KVM_MENU_FLAG_LEASE_VALID;
    clearRouteState();
}

uint8_t core4KvmCore4ApplyControl(const observerLinkKvmControl *control, uint32_t tickMs)
{
    uint8_t routeChanged;
    if ((control == NULL) ||
        (control->targetSession != g_kvm.publicStatus.localSession) ||
        (control->target != OBSERVER_LINK_KVM_TARGET_CORE4) ||
        (control->leaseMs == 0u) ||
        (control->leaseMs > CORE4_KVM_MAX_LEASE_MS))
    {
        return 0u;
    }
    routeChanged = (uint8_t) ((control->routeSession != g_kvm.publicStatus.routeSession) ||
        (control->routeEpoch != g_kvm.publicStatus.routeEpoch));
    g_kvm.publicStatus.routeSession = control->routeSession;
    g_kvm.publicStatus.routeEpoch = control->routeEpoch;
    if (routeChanged != 0u)
    {
        clearRouteState();
    }
    g_kvm.publicStatus.leaseDeadlineMs = tickMs + control->leaseMs;
    g_kvm.publicStatus.leaseValid = 1u;
    g_kvm.menuFlags |= OBSERVER_LINK_KVM_MENU_FLAG_LEASE_VALID;
    return 1u;
}

uint8_t core4KvmCore4ApplyEvent(const observerLinkKvmEvent *event, uint32_t tickMs)
{
    if (event == NULL)
    {
        return 0u;
    }
    core4KvmCore4Poll(tickMs);
    if ((g_kvm.publicStatus.eventPending != 0u) || (g_kvm.ackReady != 0u))
    {
        return 0u;
    }
    if (event->targetSession != g_kvm.publicStatus.localSession)
    {
        prepareAck(event, OBSERVER_LINK_KVM_RESULT_BAD_PAYLOAD);
        return 1u;
    }
    if (event->target != OBSERVER_LINK_KVM_TARGET_CORE4)
    {
        prepareAck(event, OBSERVER_LINK_KVM_RESULT_WRONG_TARGET);
        return 1u;
    }
    if ((g_kvm.publicStatus.routeSession != 0u) && (routeMatchesEvent(event) == 0u))
    {
        prepareAck(event, OBSERVER_LINK_KVM_RESULT_STALE_EPOCH);
        return 1u;
    }
    if ((event->keyId >= CORE4_KVM_KEY_COUNT) ||
        (event->keyState >= CORE4_KVM_KEY_STATE_COUNT) ||
        (event->flags != 0u))
    {
        prepareAck(event, OBSERVER_LINK_KVM_RESULT_BAD_PAYLOAD);
        return 1u;
    }
    if (sameCompletedEvent(event) != 0u)
    {
        g_kvm.ack = g_kvm.completedAck;
        g_kvm.ack.flags |= OBSERVER_LINK_KVM_ACK_FLAG_DUPLICATE;
        g_kvm.ackReady = 1u;
        updatePublicPending();
        return 1u;
    }
    if (g_kvm.publicStatus.leaseValid == 0u)
    {
        prepareAck(event, OBSERVER_LINK_KVM_RESULT_NO_LEASE);
        return 1u;
    }
    if ((event->eventSeq == 0u) ||
        ((event->eventSeq != g_kvm.publicStatus.expectedSeq) &&
        (eventSeqNewer(event->eventSeq, g_kvm.publicStatus.expectedSeq) == 0u)))

```

```

    {
        prepareAck(event, OBSERVER_LINK_KVM_RESULT_OLD_SEQ);
        return 1u;
    }
    if (event->eventSeq != g_kvm.publicStatus.expectedSeq)
    {
        prepareAck(event, OBSERVER_LINK_KVM_RESULT_GAP);
        return 1u;
    }
    g_kvm.event = *event;
    g_kvm.publicStatus.eventPending = 1u;
    return 1u;
}

uint8_t core4KvmCore4TakeEvent(observerLinkKvmEvent *event)
{
    if ((event == NULL) || (g_kvm.publicStatus.eventPending == 0u))
    {
        return 0u;
    }
    *event = g_kvm.event;
    return 1u;
}

uint8_t core4KvmCore4CompleteEvent(uint8_t result, uint8_t menuState,
    uint8_t homeReady, uint8_t action, uint8_t actionOk,
    uint32_t screenRevision)
{
    if (g_kvm.publicStatus.eventPending == 0u)
    {
        return 0u;
    }
    g_kvm.menuState = menuState;
    g_kvm.screenRevision = screenRevision;
    prepareAck(&g_kvm.event, result);
    if (homeReady != 0u)
    {
        g_kvm.ack.flags |= OBSERVER_LINK_KVM_ACK_FLAG_HOME_READY;
    }
    if (actionOk != 0u)
    {
        g_kvm.ack.flags |= OBSERVER_LINK_KVM_ACK_FLAG_ACTION_OK;
    }
    if (action == CORE4_KVM_ACTION_SAVE)
    {
        g_kvm.ack.flags |= OBSERVER_LINK_KVM_ACK_FLAG_ACTION_SAVE;
    }
    g_kvm.publicStatus.expectedSeq = nextEventSeq(g_kvm.publicStatus.expectedSeq);
    g_kvm.ack.expectedSeq = g_kvm.publicStatus.expectedSeq;
    g_kvm.publicStatus.eventPending = 0u;
    g_kvm.completedAck = g_kvm.ack;
    g_kvm.completedAckValid = 1u;
    g_kvm.lastEventSeq = g_kvm.event.eventSeq;
    g_kvm.lastAckResult = result;
    return 1u;
}

uint8_t core4KvmCore4TakeAck(observerLinkKvmAck *ack)
{
    if ((ack == NULL) || (g_kvm.ackReady == 0u))
    {
        return 0u;
    }
    *ack = g_kvm.ack;
    g_kvm.ackReady = 0u;
    updatePublicPending();
    return 1u;
}

void core4KvmCore4UpdateMenu(uint8_t menuState, uint8_t homeReady,
    uint8_t action, uint8_t actionOk, uint8_t feedbackActive,
    uint32_t screenRevision)
{
    uint8_t flags = 0u;
    g_kvm.menuState = menuState;
    g_kvm.screenRevision = screenRevision;
    if (homeReady != 0u)
    {
        flags |= OBSERVER_LINK_KVM_MENU_FLAG_HOME_READY;
    }

```



```

    }
    if (feedbackActive != 0u)
    {
        flags |= OBSERVER_LINK_KVM_MENU_FLAG_FEEDBACK_ACTIVE;
    }
    if (g_kvm.publicStatus.leaseValid != 0u)
    {
        flags |= OBSERVER_LINK_KVM_MENU_FLAG_LEASE_VALID;
    }
    if (actionOk != 0u)
    {
        flags |= OBSERVER_LINK_KVM_MENU_FLAG_ACTION_OK;
    }
    if (action == CORE4_KVM_ACTION_SAVE)
    {
        flags |= OBSERVER_LINK_KVM_MENU_FLAG_ACTION_SAVE;
    }
    g_kvm.menuFlags = flags;
}

void core4KvmCore4GetMenuStatus(observerLinkKvmMenuStatus *status)
{
    if (status == NULL)
    {
        return;
    }
    memset(status, 0, sizeof(*status));
    status->targetSession = g_kvm.publicStatus.localSession;
    status->routeSession = g_kvm.publicStatus.routeSession;
    status->routeEpoch = g_kvm.publicStatus.routeEpoch;
    status->lastEventSeq = g_kvm.lastEventSeq;
    status->screenRevision = g_kvm.screenRevision;
    status->target = OBSERVER_LINK_KVM_TARGET_CORE4;
    status->menuState = g_kvm.menuState;
    status->flags = g_kvm.menuFlags;
    status->lastAckResult = g_kvm.lastAckResult;
}

void core4KvmCore4GetStatus(core4KvmCore4Status *status)
{
    if (status != NULL)
    {
        *status = g_kvm.publicStatus;
    }
}

```

core4_cyt4bb7/project/code/comm/core4KvmCore4.h

SHA-256

:

fa7aa6941f47fdf1ffdc44ca9fc7abdf915995eeb50b122b91461561d392664f

代码 (c)

```

#ifndef CORE4_KVM_CORE4_H
#define CORE4_KVM_CORE4_H

#include <stdint.h>

#include "observerLinkProtocol.h"

#define CORE4_KVM_MAX_LEASE_MS (600u)

typedef struct
{
    uint32_t localSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t expectedSeq;
    uint32_t leaseDeadlineMs;
    uint8_t leaseValid;
    uint8_t eventPending;
    uint8_t ackPending;
} core4KvmCore4Status;

void core4KvmCore4Init(uint32_t localSession);

```

```

void core4KvmCore4Poll(uint32_t tickMs);
void core4KvmCore4Revoke(void);
uint8_t core4KvmCore4ApplyControl(const observerLinkKvmControl *control, uint32_t tickMs);
uint8_t core4KvmCore4ApplyEvent(const observerLinkKvmEvent *event, uint32_t tickMs);
uint8_t core4KvmCore4TakeEvent(observerLinkKvmEvent *event);
uint8_t core4KvmCore4CompleteEvent(uint8_t result, uint8_t menuState,
    uint8_t homeReady, uint8_t action, uint8_t actionOk,
    uint32_t screenRevision);
uint8_t core4KvmCore4TakeAck(observerLinkKvmAck *ack);
void core4KvmCore4UpdateMenu(uint8_t menuState, uint8_t homeReady,
    uint8_t action, uint8_t actionOk, uint8_t feedbackActive,
    uint32_t screenRevision);
void core4KvmCore4GetMenuStatus(observerLinkKvmMenuStatus *status);
void core4KvmCore4GetStatus(core4KvmCore4Status *status);

#endif /* CORE4_KVM_CORE4_H */

```

core4_cyt4bb7/project/code/comm/core4LinkCore4.c

SHA-256

:

9fb7d17c61b5361e23b32197abbe3d1a2d84e95aff36c4cc6bdeb64fb1cbaa5a

代码 (c)

```

#include "core4LinkCore4.h"

#include <string.h>

#include "core4CoreDataTransfer.h"
#include "paramSyncMailbox.h"
#include "cy_device_headers.h"
#include "pinMap.h"
#include "zf_driver_uart.h"
#if defined(__ICCARM__)
#include "scb/cy_scb_uart.h"

#if (CORE4_CORE1_UART_NUM != UART_5) || \
    (CORE4_CORE1_UART_RX_PIN != UART5_RX_P02_0) || \
    (CORE4_CORE1_UART_TX_PIN != UART5_TX_P02_1)
#error "Core4-Core1 link must use UART5 P02_0/P02_1"
#endif
#endif

#define OBSERVER_LINK_BAUDRATE (OBSERVER_LINK_UART_BAUDRATE)
#if OBSERVER_LINK_UART_BAUDRATE != 4 000 000u
#error "Core1-Core4 link baud contract requires 4 Mbps"
#endif
#define OBSERVER_LINK_RX_BUDGET (OBSERVER_LINK_RX_BYTE_BUDGET)
#define OBSERVER_LINK_RX_RING_BYTES (2048u)
#define OBSERVER_LINK_RX_RING_MASK (OBSERVER_LINK_RX_RING_BYTES - 1u)
#if (OBSERVER_LINK_RX_RING_BYTES & OBSERVER_LINK_RX_RING_MASK) != 0u
#error "Core1-Core4 RX ring must be a power of two"
#endif
#define OBSERVER_LINK_HELLO_PERIOD_MS (500u)
#define OBSERVER_LINK_HEARTBEAT_PERIOD_MS (200u)
#define OBSERVER_LINK_DIAG_PERIOD_MS (1000u)
#define OBSERVER_LINK_TIMEOUT_MS (600u)
#define OBSERVER_LINK_PLANNING_BURST_MAX (7u)
#define OBSERVER_LINK_PARAM_GLOBAL_RETRY_MS (50u)

static observerLinkParser g_parser;
static core4LinkCore4Status g_status;
static observerLinkSharedConfig g_staged;
static observerLinkConfigAck g_pendingAck;
static uint8_t g_stagedValid;
static uint8_t g_ackPending;
static uint32_t g_packetSeq;
static uint32_t g_rxPacketSeq;
static uint8_t g_rxPacketSeqPrimed;
static uint8_t g_uartInitialized;
static uint32_t g_lastHelloTickMs;
static uint32_t g_lastHeartbeatTickMs;
static uint32_t g_lastDiagTickMs;
static uint32_t g_lastDeferredTickMs;
static uint32_t g_txEpochTickMs;

```

```

static uint8_t g_txDataCredits;
static uint8_t g_txControlCredits;
static uint8_t g_planningBurstCount;
static observerLinkKvmControl g_kvmControl;
static observerLinkKvmEvent g_kvmEvent;
static observerLinkKvmAck g_kvmAck;
static observerLinkKvmMenuStatus g_kvmMenuStatus;
static uint8_t g_kvmControlReady;
static uint8_t g_kvmEventReady;
static uint8_t g_kvmAckPending;
static uint8_t g_kvmMenuPending;
static uint8_t g_visionReady;
static observerLinkPlanningTrackSnapshot g_planningActive;
static observerLinkPlanningTrackSnapshot g_planningLatest;
static uint8_t g_planningActiveValid;
static uint8_t g_planningLatestValid;
static uint8_t g_planningFragmentIndex;
static observerLinkHealth g_health;
static uint8_t g_healthPending;
static volatile uint16_t g_rxHead;
static volatile uint16_t g_rxTail;
static uint8_t g_rxRing[OBSERVER_LINK_RX_RING_BYTES];
static volatile uint32_t g_rxRingOverflow;
static paramSyncMailboxRequest g_paramAssembly;
static paramSyncMailboxResponse g_paramResponse;
static observerParamFrame g_paramReply;
static uint32_t g_paramRequestSequence;
static uint16_t g_paramExpectedBytes, g_paramReceivedBytes;
static uint8_t g_paramAssemblyActive, g_paramResponseValid, g_paramReplyPending;
static uint32_t g_paramOwnerBoot, g_paramOwnerResponseSequence;
static uint32_t g_paramOwnerDeadlineMs, g_paramLastTickMs;
static uint8_t g_paramOwnerBootPrimed, g_paramWaitingOwner;
static paramSyncGlobalState g_paramGlobalState;
static paramSyncGlobalState g_paramGlobalRouteFailure;
static paramSyncGlobalRequest g_paramGlobalRequest;
static uint32_t g_paramGlobalRequestLastSendMs, g_paramGlobalStateLastRxMs;
static uint8_t g_paramGlobalStateReady, g_paramGlobalStateValid;
static uint8_t g_paramGlobalRouteFailureReady, g_paramGlobalRequestPending;

static void failPendingParamGlobalRequest(uint8_t error)
{
    paramSyncGlobalState failure;
    uint32_t sequence;
    if (g_paramGlobalRequestPending == 0u) return;
    memset(&failure, 0, sizeof(failure));
    failure.coordinatorBootGeneration =
        (g_paramGlobalState.coordinatorBootGeneration != 0u) ?
            g_paramGlobalState.coordinatorBootGeneration :
            g_paramGlobalRequest.producerBootGeneration;
    sequence = g_paramGlobalState.sequence + 1u;
    if (sequence == 0u) sequence = 1u;
    failure.sequence = sequence;
    failure.requestBootGeneration =
        g_paramGlobalRequest.producerBootGeneration;
    failure.requestSequence = g_paramGlobalRequest.sequence;
    failure.origin = g_paramGlobalRequest.origin;
    failure.action = g_paramGlobalRequest.action;
    failure.slot = g_paramGlobalRequest.slot;
    failure.phase = PARAM_SYNC_GLOBAL_PHASE_FAILED;
    failure.target = OBSERVER_LINK_SOURCE_CORE4;
    failure.error = error;
    failure.flags = PARAM_SYNC_GLOBAL_FLAG_TERMINAL;
    if (paramSyncGlobalStateValid(&failure) != 0u)
    {
        g_paramGlobalRouteFailure = failure;
        g_paramGlobalRouteFailureReady = 1u;
    }
    g_paramGlobalRequestPending = 0u;
}

static void invalidateParamGlobalRoute(void)
{
    failPendingParamGlobalRequest(PARAM_SYNC_GLOBAL_ERROR_ROUTE);
    g_paramGlobalStateValid = 0u;
    g_paramGlobalStateReady = 0u;
}

static uint8_t paramGlobalRouteReady(uint32_t tickMs)
{

```

```

return (uint8_t) (g_status.online != 0u &&
    g_status.protocolCompatible != 0u &&
    (g_status.peerCapabilities & OBSERVER_LINK_CAP_PARAM_SYNC) != 0u &&
    g_paramGlobalStateValid != 0u &&
    (uint32_t) (tickMs - g_paramGlobalStateLastRxMs) <
        CORE4_LINK_PARAM_GLOBAL_STATE_LOST_MS);
}

static uint8_t paramGlobalBusyError(const paramSyncGlobalState *state)
{
    if ((state->flags & PARAM_SYNC_GLOBAL_FLAG_RECOVERY) != 0u)
        return PARAM_SYNC_GLOBAL_ERROR_RECOVERY;
    if (state->readyMask != PARAM_SYNC_GLOBAL_TARGET_MASK)
        return PARAM_SYNC_GLOBAL_ERROR_INCOMPLETE;
    if ((state->flags & (PARAM_SYNC_GLOBAL_FLAG_BUSY |
        PARAM_SYNC_GLOBAL_FLAG_TERMINAL)) != 0u)
        return PARAM_SYNC_GLOBAL_ERROR_BUSY;
    return PARAM_SYNC_GLOBAL_ERROR_ROUTE;
}

static void clearControlStatus(void)
{
    memset(&g_status.controlStatus, 0, sizeof(g_status.controlStatus));
    g_status.controlStatus.cameraControlState = OBSERVER_LINK_CAMERA_CONTROL_SAFE;
    g_status.cameraControlState = OBSERVER_LINK_CAMERA_CONTROL_SAFE;
    g_status.cameraControlValid = 0u;
    g_status.controlStatusValid = 0u;
    g_status.cameraControlTickMs = 0u;
}

static uint8_t helloIdentifiesCore1(const observerLinkFrame *frame)
{
    uint8_t role;
    uint32_t capabilities;
    const uint32_t required = OBSERVER_LINK_CAP_BINARY_CC |
        OBSERVER_LINK_CAP_LEGACY_CANNY |
        OBSERVER_LINK_CAP_CONFIG_STAGING |
        OBSERVER_LINK_CAP_PLANNING_TRACKS |
        OBSERVER_LINK_CAP_CAMERA_CONTROL |
        OBSERVER_LINK_CAP_CONTROL_STATUS;
    return (uint8_t) ((frame->major == OBSERVER_LINK_PROTOCOL_MAJOR) &&
        (frame->minor == OBSERVER_LINK_PROTOCOL_MINOR) &&
        (observerLinkDecodeHello(frame->payload, frame->payloadLength,
            &role, &capabilities) != 0u) &&
        (role == OBSERVER_LINK_ROLE_CORE1) &&
        ((capabilities & required) == required));
}

static void clearViewOverlay(void)
{
    memset(&g_status.viewOverlay, 0, sizeof(g_status.viewOverlay));
    g_status.viewOverlayValid = 0u;
    g_status.viewOverlayTickMs = 0u;
}

#if defined(_ICCARM_)
__no_init uint32_t core4LinkCore4BootCounter;
__no_init uint32_t core4LinkCore4BootCounterInverse;
#define g_bootCounter    core4LinkCore4BootCounter
#define g_bootCounterInverse core4LinkCore4BootCounterInverse
#else
static uint32_t g_bootCounter;
static uint32_t g_bootCounterInverse;
#endif

static uint32_t makeBootSession(uint32_t tickMs)
{
    uint32_t session;
    (void) tickMs;
    if (g_bootCounterInverse != ~g_bootCounter)
    {
        g_bootCounter = 0u;
    }
    g_bootCounter++;
    if (g_bootCounter == 0u) g_bootCounter = 1u;
    g_bootCounterInverse = ~g_bootCounter;
    session = g_bootCounter;
    return session;
}

```

```

}

static void clearOwnerState(uint32_t ownerSession)
{
    invalidateParamGlobalRoute();
    memset(&g_staged, 0, sizeof(g_staged));
    memset(&g_pendingAck, 0, sizeof(g_pendingAck));
    g_stagedValid = 0u;
    g_ackPending = 0u;
    g_status.ownerSession = ownerSession;
    g_status.configStatus = OBSERVER_LINK_CONFIG_NONE;
    g_status.configReason = OBSERVER_LINK_REJECT_NONE;
    g_status.stagedMode = 0xFFu;
    g_status.appliedMode = 0xFFu;
    g_status.stagedRevision = 0u;
    g_status.stagedHash = 0u;
    g_status.appliedRevision = 0u;
    g_status.appliedHash = 0u;
    clearControlStatus();
    clearViewOverlay();
    memset(&g_status.peerDiagnostics, 0, sizeof(g_status.peerDiagnostics));
    g_status.peerDiagnosticsTickMs = 0u;
    g_kvmControlReady = 0u;
    g_kvmEventReady = 0u;
    g_kvmAckPending = 0u;
    g_kvmMenuPending = 0u;
    g_planningActiveValid = 0u;
    g_planningLatestValid = 0u;
    g_planningFragmentIndex = 0u;
    memset(&g_health, 0, sizeof(g_health));
    g_healthPending = 0u;
    g_paramAssemblyActive = 0u;
    g_paramResponseValid = 0u;
    g_paramReplyPending = 0u;
    g_paramWaitingOwner = 0u;
}

static uint8_t sendMessageBudgeted(uint8_t type,
    const uint8_t *payload,
    uint16_t payloadLength,
    uint32_t tickMs,
    uint8_t controlLane)
{
    observerLinkFrame frame;
    uint8_t wire[OBSERVER_LINK_MAX_FRAME_BYTES];
    uint16_t wireLength;

    if (controlLane != 0u)
    {
        if (g_txControlCredits == 0u) return 0u;
    }
    else if (g_txDataCredits == 0u)
    {
        return 0u;
    }

    if ((payloadLength > OBSERVER_LINK_MAX_PAYLOAD_BYTES) ||
        ((payloadLength != 0u) && (payload == NULL)))
    {
        g_status.txFailures++;
        return 0u;
    }
    memset(&frame, 0, sizeof(frame));
    frame.major = OBSERVER_LINK_PROTOCOL_MAJOR;
    frame.minor = OBSERVER_LINK_PROTOCOL_MINOR;
    frame.type = type;
    frame.payloadLength = payloadLength;
    frame.bootSession = g_status.localSession;
    g_packetSeq++;
    if (g_packetSeq == 0u) g_packetSeq = 1u;
    frame.packetSeq = g_packetSeq;
    frame.tickMs = tickMs;
    if (payloadLength != 0u)
    {
        memcpy(frame.payload, payload, payloadLength);
    }
    wireLength = observerLinkEncodeFrame(&frame, wire, (uint16_t) sizeof(wire));
    if (wireLength == 0u)
    {

```

```

        g_status.txFailures++;
        return 0u;
    }
    uart_write_buffer(CORE4_CORE1_UART_NUM, wire, wireLength);
    if (controlLane != 0u) g_txControlCredits--;
    else g_txDataCredits--;
    g_status.txFrames++;
    return 1u;
}

static uint8_t sendMessage(uint8_t type,
    const uint8_t *payload,
    uint16_t payloadLength,
    uint32_t tickMs)
{
    return sendMessageBudgeted(type, payload, payloadLength, tickMs, 0u);
}

static uint8_t sendControlMessage(uint8_t type,
    const uint8_t *payload,
    uint16_t payloadLength,
    uint32_t tickMs)
{
    return sendMessageBudgeted(type, payload, payloadLength, tickMs, 1u);
}

static void publishParamAssembly(void)
{
    g_paramAssemblyActive = 0u;
    g_paramAssembly.dataLength = g_paramExpectedBytes;
    g_paramRequestSequence++;
    if (g_paramRequestSequence == 0u) g_paramRequestSequence = 1u;
    g_paramAssembly.producerBootGeneration = g_status.localSession;
    g_paramAssembly.sequence = g_paramRequestSequence;
    g_paramAssembly.offset = g_paramRequestSequence;
    if (!paramSyncMailboxPublishRequest(&g_paramAssembly))
        g_paramReply.status = PARAM_SYNC_STATUS_FAILED;
    else
    {
        g_paramWaitingOwner = 1u;
        g_paramOwnerDeadlineMs = g_paramLastTickMs + 500u;
    }
}

static void handleParamRequest(const observerLinkFrame *frame, uint32_t tickMs)
{
    observerParamFrame wire;
    paramSyncMailboxResponse response;
    uint16_t remaining;
    g_paramLastTickMs = tickMs;
    if (!observerParamDecode(frame->payload, frame->payloadLength, &wire)) return;
    g_paramReply = wire;
    g_paramReply.payloadCrc32 = 0u;
    g_paramReply.status = PARAM_SYNC_STATUS_PENDING;
    g_paramReply.length = 0u;
    if (wire.op == OBSERVER_PARAM_OP_POLL)
    {
        if (!g_paramResponseValid && paramSyncMailboxTryReadResponse(&response) &&
            response.transactionId == wire.transactionId &&
            response.offset == g_paramAssembly.offset &&
            response.op == g_paramAssembly.op && response.slot == g_paramAssembly.slot &&
            response.target == g_paramAssembly.target)
        {
            if (g_paramOwnerBootPrimed && response.producerBootGeneration != g_paramOwnerBoot)
            {
                g_paramReply.status = PARAM_SYNC_STATUS_FAILED;
                g_paramAssemblyActive = 0u;
                g_paramResponseValid = 0u;
                g_paramWaitingOwner = 0u;
                g_paramOwnerBoot = response.producerBootGeneration;
                g_paramOwnerResponseSequence = response.sequence;
                g_paramOwnerBootPrimed = 1u;
            }
            else if (g_paramOwnerBootPrimed &&
                observerLinkSequenceIsNewer(response.sequence,
                    g_paramOwnerResponseSequence) == 0u)
            {
                /* 保留邮箱旧应答只作基线，不能完成新的线端事务。 */
            }
        }
    }
}

```

```

else
{
    g_paramOwnerBoot = response.producerBootGeneration;
    g_paramOwnerResponseSequence = response.sequence;
    g_paramOwnerBootPrimed = 1u;
    g_paramResponse = response;
    g_paramResponseValid = 1u;
    g_paramWaitingOwner = 0u;
}
}
if (g_paramWaitingOwner &&
    (int32_t)(tickMs - g_paramOwnerDeadlineMs) >= 0)
{
    g_paramWaitingOwner = 0u;
    g_paramReply.status = PARAM_SYNC_STATUS_FAILED;
}
if (g_paramResponseValid && g_paramResponse.transactionId == wire.transactionId)
{
    g_paramReply.op = g_paramResponse.op;
    g_paramReply.slot = g_paramResponse.slot;
    g_paramReply.target = g_paramResponse.target;
    g_paramReply.status = (uint8_t)(g_paramResponse.status & OBSERVER_PARAM_STATUS_MASK);
    g_paramReply.payloadCrc32 = g_paramResponse.payloadCrc32;
    g_paramReply.offset = wire.offset;
    g_paramReply.totalLength = g_paramResponse.dataLength;
    if (wire.offset > g_paramResponse.dataLength)
        g_paramReply.status = PARAM_SYNC_STATUS_INVALID;
    else
    {
        remaining = (uint16_t)(g_paramResponse.dataLength - wire.offset);
        g_paramReply.length = (remaining > OBSERVER_PARAM_DATA_BYTES) ? OBSERVER_PARAM_DATA_BYTES : (uint8_t)remaining;
        memcpy(g_paramReply.data, &g_paramResponse.data[wire.offset], g_paramReply.length);
        if (remaining > g_paramReply.length)
            g_paramReply.status |= OBSERVER_PARAM_TRANSPORT_MORE;
    }
}
}
else if ((wire.status & OBSERVER_PARAM_TRANSPORT_BEGIN) != 0u)
{
    if (wire.target != 4u || wire.totalLength > PARAM_SYNC_MAILBOX_DATA_MAX)
    {
        g_paramAssemblyActive = 0u;
        g_paramResponseValid = 0u;
        g_paramWaitingOwner = 0u;
        g_paramReply.status = PARAM_SYNC_STATUS_INVALID;
        memset(&g_paramAssembly, 0, sizeof(g_paramAssembly));
    }
    else if (wire.transactionId == g_paramAssembly.transactionId &&
        wire.op == g_paramAssembly.op && wire.slot == g_paramAssembly.slot &&
        wire.target == g_paramAssembly.target &&
        wire.payloadCrc32 == g_paramAssembly.payloadCrc32 &&
        wire.totalLength == g_paramExpectedBytes)
    {
        /* 丢失应答后的 BEGIN 重传保留分片进度及 owner 应答。 */
    }
    else
    {
        g_paramAssemblyActive = 0u;
        g_paramResponseValid = 0u;
        g_paramWaitingOwner = 0u;
        memset(&g_paramAssembly, 0, sizeof(g_paramAssembly));
        g_paramExpectedBytes = wire.totalLength;
        g_paramReceivedBytes = 0u;
        g_paramAssembly.op = wire.op;
        g_paramAssembly.slot = wire.slot;
        g_paramAssembly.target = wire.target;
        g_paramAssembly.transactionId = wire.transactionId;
        g_paramAssembly.payloadCrc32 = wire.payloadCrc32;
        g_paramAssemblyActive = 1u;
        if (g_paramExpectedBytes == 0u) publishParamAssembly();
    }
}
else if (wire.transactionId == g_paramAssembly.transactionId &&
    wire.op == g_paramAssembly.op && wire.slot == g_paramAssembly.slot &&
    wire.target == g_paramAssembly.target &&
    wire.payloadCrc32 == g_paramAssembly.payloadCrc32 &&
    ((uint32_t)wire.offset + wire.length <= g_paramExpectedBytes))
{
    if (wire.offset < g_paramReceivedBytes)

```

```

    {
        if (((uint32_t) wire.offset + wire.length > g_paramReceivedBytes) ||
            memcmp(&g_paramAssembly.data[wire.offset], wire.data,
                wire.length) != 0)
        {
            g_paramAssemblyActive = 0u;
            g_paramReply.status = PARAM_SYNC_STATUS_INVALID;
        }
    }
    else if (g_paramAssemblyActive && wire.offset == g_paramReceivedBytes)
    {
        memcpy(&g_paramAssembly.data[g_paramReceivedBytes], wire.data, wire.length);
        g_paramReceivedBytes = (uint16_t) (g_paramReceivedBytes + wire.length);
        if (g_paramReceivedBytes == g_paramExpectedBytes) publishParamAssembly();
    }
    else
    {
        g_paramAssemblyActive = 0u;
        g_paramReply.status = PARAM_SYNC_STATUS_INVALID;
    }
}
else
{
    g_paramAssemblyActive = 0u;
    g_paramReply.status = PARAM_SYNC_STATUS_INVALID;
}
g_paramReplyPending = 1u;
}

static uint8_t sendParamReply(uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_MAX_PAYLOAD_BYTES];
    uint16_t length;
    if (!g_paramReplyPending || !observerParamEncode(&g_paramReply, payload, &length)) return 0u;
    if (!sendMessage(OBSERVER_LINK_MSG_PARAM_RESPONSE, payload, length, tickMs)) return 0u;
    g_paramReplyPending = 0u;
    return 1u;
}

static uint8_t sendParamGlobalRequest(uint32_t tickMs)
{
    uint8_t payload[OBSERVER_PARAM_GLOBAL_REQUEST_BYTES];
    uint16_t length;
    if (g_paramGlobalRequestPending != 0u &&
        paramGlobalRouteReady(tickMs) == 0u)
        failPendingParamGlobalRequest(PARAM_SYNC_GLOBAL_ERROR_ROUTE);
    if ((g_paramGlobalRequestPending == 0u) ||
        ((uint32_t) (tickMs - g_paramGlobalRequestLastSendMs) <
            OBSERVER_LINK_PARAM_GLOBAL_RETRY_MS) ||
        (observerParamGlobalRequestEncode(&g_paramGlobalRequest,
            payload, &length) == 0u) ||
        (sendMessage(OBSERVER_LINK_MSG_PARAM_GLOBAL_REQUEST,
            payload, length, tickMs) == 0u)) return 0u;
    g_paramGlobalRequestLastSendMs = tickMs;
    return 1u;
}

static void queueAck(uint32_t ownerSession,
    const observerLinkSharedConfig *config,
    uint8_t status,
    uint8_t reason)
{
    if ((g_ackPending != 0u) &&
        (g_pendingAck.status == OBSERVER_LINK_CONFIG_REJECTED) &&
        (status != OBSERVER_LINK_CONFIG_REJECTED))
    {
        return;
    }
    memset(&g_pendingAck, 0, sizeof(g_pendingAck));
    g_pendingAck.ownerSession = ownerSession;
    if (config != NULL)
    {
        g_pendingAck.revision = config->revision;
        g_pendingAck.hash = config->hash;
        g_pendingAck.desiredMode = config->detectMode;
    }
    g_pendingAck.appliedMode = g_status.appliedMode;
    g_pendingAck.status = status;
    g_pendingAck.reason = reason;

```



```

    g_ackPending = 1u;
}

static void handleHello(const observerLinkFrame *frame)
{
    uint8_t role;
    uint32_t capabilities;

    g_status.peerCapabilities = 0u;
    g_status.protocolCompatible = 0u;
    g_status.planningCompatible = 0u;

    if (observerLinkDecodeHello(frame->payload,
        frame->payloadLength,
        &role,
        &capabilities) == 0u ||
        (role != OBSERVER_LINK_ROLE_CORE1))
    {
        g_status.protocolCompatible = 0u;
        return;
    }
    g_status.peerCapabilities = capabilities;
    g_status.protocolCompatible = (uint8_t) ((frame->minor == OBSERVER_LINK_PROTOCOL_MINOR) &&
        (frame->payload[2] == OBSERVER_LINK_PROTOCOL_MINOR) &&
        ((capabilities & OBSERVER_LINK_CAP_BINARY_CC) != 0u) &&
        ((capabilities & OBSERVER_LINK_CAP_LEGACY_CANNY) != 0u) &&
        ((capabilities & OBSERVER_LINK_CAP_CONFIG_STAGING) != 0u));
    g_status.planningCompatible = (uint8_t) ((frame->minor == OBSERVER_LINK_PROTOCOL_MINOR) &&
        (frame->payload[2] == OBSERVER_LINK_PROTOCOL_MINOR) &&
        ((capabilities & OBSERVER_LINK_CAP_PLANNING_TRACKS) != 0u) &&
        ((capabilities & OBSERVER_LINK_CAP_CAMERA_CONTROL) != 0u) &&
        ((capabilities & OBSERVER_LINK_CAP_CONTROL_STATUS) != 0u));
    if (g_status.planningCompatible == 0u)
    {
        clearControlStatus();
        g_planningActiveValid = 0u;
        g_planningLatestValid = 0u;
        g_planningFragmentIndex = 0u;
    }
}

static void handleConfig(const observerLinkFrame *frame)
{
    observerLinkSharedConfig incoming;

    if ((frame->minor != OBSERVER_LINK_PROTOCOL_MINOR) ||
        (g_status.planningCompatible == 0u))
    {
        queueAck(frame->bootSession,
            NULL,
            OBSERVER_LINK_CONFIG_REJECTED,
            OBSERVER_LINK_REJECT_UNSUPPORTED);
        return;
    }

    if (observerLinkDecodeSharedConfig(frame->payload,
        frame->payloadLength,
        &incoming) == 0u)
    {
        queueAck(frame->bootSession,
            NULL,
            OBSERVER_LINK_CONFIG_REJECTED,
            OBSERVER_LINK_REJECT_BAD_PAYLOAD);
        return;
    }
    if (incoming.hash != observerLinkSharedConfigHash(&incoming))
    {
        queueAck(frame->bootSession,
            &incoming,
            OBSERVER_LINK_CONFIG_REJECTED,
            OBSERVER_LINK_REJECT_BAD_HASH);
        return;
    }
    if ((incoming.sourceId != OBSERVER_LINK_SOURCE_CORE4) ||
        (incoming.profileVersion != 1u) ||
        (g_status.protocolCompatible == 0u) ||
        (observerLinkSharedConfigsValid(&incoming) == 0u))
    {
        queueAck(frame->bootSession,

```

```

        &incoming,
        OBSERVER_LINK_CONFIG_REJECTED,
        (g_status.protocolCompatible == 0u)
        ? OBSERVER_LINK_REJECT_UNSUPPORTED
        : OBSERVER_LINK_REJECT_BAD_PAYLOAD);
    return;
}

if ((g_stagedValid != 0u) && (incoming.revision != g_staged.revision) &&
    (observerLinkSequencesIsNewer(incoming.revision,
    g_staged.revision) == 0u))
{
    queueAck(frame->bootSession,
        &g_staged,
        OBSERVER_LINK_CONFIG_REJECTED,
        OBSERVER_LINK_REJECT_OLD_REVISION);
    return;
}
if ((g_stagedValid != 0u) && (incoming.revision == g_staged.revision))
{
    if (incoming.hash != g_staged.hash)
    {
        queueAck(frame->bootSession,
            &g_staged,
            OBSERVER_LINK_CONFIG_REJECTED,
            OBSERVER_LINK_REJECT_REVISION_CONFLICT);
    }
    else
    {
        queueAck(frame->bootSession,
            &g_staged,
            g_status.configStatus,
            g_status.configReason);
    }
    return;
}

g_staged = incoming;
g_stagedValid = 1u;
g_status.stagedRevision = incoming.revision;
g_status.stagedHash = incoming.hash;
g_status.stagedMode = incoming.detectMode;
g_status.configStatus = OBSERVER_LINK_CONFIG_STAGED;
g_status.configReason = OBSERVER_LINK_REJECT_NONE;
queueAck(frame->bootSession,
    &g_staged,
    OBSERVER_LINK_CONFIG_STAGED,
    OBSERVER_LINK_REJECT_NONE);
}

static void handleFrame(const observerLinkFrame *frame, uint32_t tickMs)
{
    uint8_t qualifiedHello = (uint8_t) ((frame->type == OBSERVER_LINK_MSG_HELLO) &&
        (frame->major == OBSERVER_LINK_PROTOCOL_MAJOR) &&
        (frame->minor == OBSERVER_LINK_PROTOCOL_MINOR) &&
        (helloIdentifiesCore1(frame) != 0u));
    uint8_t sessionDecision = observerLinkRxSessionClassify(
        g_status.online, g_status.ownerSession, g_rxPacketSeq,
        g_rxPacketSeqPrimed, frame->bootSession, frame->packetSeq,
        qualifiedHello);
    if (sessionDecision == OBSERVER_LINK_RX_SESSION_REJECT) return;
    if (sessionDecision == OBSERVER_LINK_RX_SESSION_REBIND)
    {
        g_status.bindingEpoch++;
        if (g_status.bindingEpoch == 0u) g_status.bindingEpoch = 1u;
        clearOwnerState(frame->bootSession);
        g_rxPacketSeqPrimed = 0u;
        g_status.protocolCompatible = 0u;
        g_status.peerCapabilities = 0u;
    }
    g_rxPacketSeq = frame->packetSeq;
    g_rxPacketSeqPrimed = 1u;
    g_status.lastValidRxTickMs = tickMs;
    g_status.online = 1u;
    g_status.rxFrames++;

    if ((frame->major != OBSERVER_LINK_PROTOCOL_MAJOR) ||
        (frame->minor != OBSERVER_LINK_PROTOCOL_MINOR))
    {

```

```

invalidateParamGlobalRoute();
g_status.protocolCompatible = 0u;
g_status.planningCompatible = 0u;
g_planningActiveValid = 0u;
g_planningLatestValid = 0u;
g_planningFragmentIndex = 0u;
return;
}
if (frame->type == OBSERVER_LINK_MSG_HELLO)
{
    handleHello(frame);
    if (g_status.protocolCompatible == 0u ||
        (g_status.peerCapabilities & OBSERVER_LINK_CAP_PARAM_SYNC) == 0u)
        invalidateParamGlobalRoute();
}
else if (frame->type == OBSERVER_LINK_MSG_CONFIG_SNAPSHOT)
{
    handleConfig(frame);
}
else if (frame->type == OBSERVER_LINK_MSG_HEARTBEAT)
{
    observerLinkControlStatus controlStatus;
    if ((frame->minor == OBSERVER_LINK_PROTOCOL_MINOR) &&
        ((g_status.peerCapabilities & OBSERVER_LINK_CAP_CAMERA_CONTROL) != 0u) &&
        ((g_status.peerCapabilities & OBSERVER_LINK_CAP_CONTROL_STATUS) != 0u) &&
        (observerLinkDecodeControlStatus(frame->payload,
            frame->payloadLength, &controlStatus) != 0u))
    {
        g_status.controlStatus = controlStatus;
        g_status.controlStatusValid = 1u;
        g_status.cameraControlState = controlStatus.cameraControlState;
        g_status.cameraControlValid = (uint8_t) ((controlStatus.flags & OBSERVER_LINK_CONTROL_FLAG_STATUS_VALID) != 0u);
        if (controlStatus.cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_RUN)
        {
            g_status.lastKnownCameraControlState =
                controlStatus.cameraControlState;
        }
        g_status.cameraControlTickMs = tickMs;
    }
    else
    {
        clearControlStatus();
    }
}
else if (frame->type == OBSERVER_LINK_MSG_DIAGNOSTICS)
{
    if (observerLinkDecodeDiagnostics(frame->payload,
        frame->payloadLength,
        &g_status.peerDiagnostics) != 0u)
    {
        g_status.peerDiagnosticsTickMs = tickMs;
    }
}
else if (frame->type == OBSERVER_LINK_MSG_VIEW_OVERLAY)
{
    observerLinkViewOverlay overlay;
    uint8_t sameIdentity;
    sameIdentity = 0u;
    if ((frame->minor == OBSERVER_LINK_PROTOCOL_MINOR) &&
        ((g_status.peerCapabilities & OBSERVER_LINK_CAP_VIEW_OVERLAY) != 0u) &&
        (observerLinkDecodeViewOverlay(frame->payload,
            frame->payloadLength, &overlay) != 0u) &&
        (overlay.ownerSession == frame->bootSession) &&
        (overlay.ownerSession == g_status.ownerSession) &&
        (overlay.configRevision == g_status.appliedRevision) &&
        (overlay.configHash == g_status.appliedHash) &&
        (overlay.detectMode == g_status.appliedMode))
    {
        sameIdentity = (uint8_t) ((g_status.viewOverlayValid != 0u) &&
            (g_status.viewOverlay.ownerSession == overlay.ownerSession) &&
            (g_status.viewOverlay.configRevision == overlay.configRevision) &&
            (g_status.viewOverlay.configHash == overlay.configHash) &&
            (g_status.viewOverlay.detectMode == overlay.detectMode));
    }
    if ((sameIdentity == 0u) ||
        (observerLinkSequenceIsNewer(overlay.overlaySeq,
            g_status.viewOverlay.overlaySeq) != 0u))
    {
        g_status.viewOverlay = overlay;
        g_status.viewOverlayValid = 1u;
    }
}

```

```

        g_status.viewOverlayTickMs = tickMs;
    }
}
}
else if (frame->type == OBSERVER_LINK_MSG_KVM_CONTROL)
{
    if (observerLinkDecodeKvmControl(frame->payload,
        frame->payloadLength, &g_kvmControl) != 0u)
    {
        g_kvmControlReady = 1u;
    }
}
else if (frame->type == OBSERVER_LINK_MSG_KVM_EVENT)
{
    if (observerLinkDecodeKvmEvent(frame->payload,
        frame->payloadLength, &g_kvmEvent) != 0u)
    {
        g_kvmEventReady = 1u;
    }
}
else if (frame->type == OBSERVER_LINK_MSG_PARAM_REQUEST &&
    (g_status.peerCapabilities & OBSERVER_LINK_CAP_PARAM_SYNC) != 0u)
{
    handleParamRequest(frame, tickMs);
}
else if ((frame->type == OBSERVER_LINK_MSG_PARAM_GLOBAL_STATE) &&
    (g_status.peerCapabilities & OBSERVER_LINK_CAP_PARAM_SYNC) != 0u))
{
    paramSyncGlobalState next;
    if (observerParamGlobalStateDecode(frame->payload,
        frame->payloadLength, &next) == 0u)
    {
        invalidateParamGlobalRoute();
    }
}
else
{
    {
        if (g_paramGlobalRequestPending != 0u &&
            g_paramGlobalStateValid != 0u &&
            next.coordinatorBootGeneration !=
                g_paramGlobalState.coordinatorBootGeneration)
            failPendingParamGlobalRequest(
                PARAM_SYNC_GLOBAL_ERROR_PEER_RESTART);
        else if (g_paramGlobalRequestPending != 0u &&
            paramSyncGlobalStateMatchesRequest(&next,
                &g_paramGlobalRequest) == 0u &&
            paramSyncGlobalStateReady(&next) == 0u)
            failPendingParamGlobalRequest(paramGlobalBusyError(&next));
        g_paramGlobalState = next;
        g_paramGlobalStateReady = 1u;
        g_paramGlobalStateValid = 1u;
        g_paramGlobalStateLastRxMs = tickMs;
        if (((next.flags & PARAM_SYNC_GLOBAL_FLAG_TERMINAL) != 0u) &&
            (next.requestBootGeneration ==
                g_paramGlobalRequest.producerBootGeneration) &&
            (next.requestSequence == g_paramGlobalRequest.sequence))
            g_paramGlobalRequestPending = 0u;
    }
}
}

static void pollHardwareRxErrors(void)
{
#ifdef __ICCARM__
    uint32_t status = Cy_SCB_UART_GetRxFifoStatus(CORE4_CORE1_UART_SCB);
    uint32_t errors = status & CY_SCB_UART_RECEIVE_ERR;

    if ((errors & CY_SCB_RX_INTR_OVERFLOW) != 0u)
    {
        g_status.localDiagnostics.rxOverflowErrors++;
    }
    if ((errors & CY_SCB_RX_INTR_UART_FRAME_ERROR) != 0u)
    {
        g_status.localDiagnostics.rxFrameErrors++;
    }
    if ((errors & CY_SCB_RX_INTR_UART_PARITY_ERROR) != 0u)
    {
        g_status.localDiagnostics.rxParityErrors++;
    }
    if (errors != 0u)

```

```

    {
        Cy_SCB_UART_ClearRxFifoStatus(CORE4_CORE1_UART_SCB, errors);
    }
#endif
}

static void receiveFrames(uint32_t tickMs)
{
    observerLinkFrame frame;
    uint8_t byte;
    uint16_t budget = OBSERVER_LINK_RX_BUDGET;

    pollHardwareRxErrors();
    while ((budget != 0u) && (g_rxTail != g_rxHead))
    {
        __DMB();
        byte = g_rxRing[g_rxTail];
        g_rxTail = (uint16_t)((g_rxTail + 1u) & OBSERVER_LINK_RX_RING_MASK);
        budget--;
        if (observerLinkParserFeed(&g_parser, byte, &frame) != 0u)
        {
            handleFrame(&frame, tickMs);
        }
    }
}

static uint8_t sendHello(uint32_t tickMs)
{
    uint8_t payload[8u];
    uint16_t length;
    uint32_t capabilities = OBSERVER_LINK_CAP_CONFIG_STAGING |
        OBSERVER_LINK_CAP_KVM |
        OBSERVER_LINK_CAP_MENU_STATUS |
        OBSERVER_LINK_CAP_PLANNING_TRACKS |
        OBSERVER_LINK_CAP_CAMERA_CONTROL |
        OBSERVER_LINK_CAP_CONTROL_STATUS |
        OBSERVER_LINK_CAP_VIEW_OVERLAY |
        OBSERVER_LINK_CAP_PARAM_SYNC;

    if (g_visionReady != 0u)
    {
        capabilities |= OBSERVER_LINK_CAP_BINARY_CC |
            OBSERVER_LINK_CAP_LEGACY_CANNY |
            OBSERVER_LINK_CAP_CONFIG_APPLY;
    }
    if (observerLinkEncodeHello(OBSERVER_LINK_ROLE_CORE4,
        capabilities,
        payload,
        &length) != 0u)
    {
        return sendMessage(OBSERVER_LINK_MSG_HELLO, payload, length, tickMs);
    }
    return 0u;
}

static uint8_t sendHeartbeat(uint32_t tickMs)
{
    {
        uint8_t payload[4u];
        payload[0] = g_status.protocolCompatible;
        payload[1] = g_status.configStatus;
        payload[2] = g_status.stagedMode;
        payload[3] = g_status.appliedMode;
        return sendMessage(OBSERVER_LINK_MSG_HEARTBEAT,
            payload,
            (uint16_t) sizeof(payload),
            tickMs);
    }
}

static uint8_t sendDiagnostics(uint32_t tickMs)
{
    {
        uint8_t payload[OBSERVER_LINK_DIAGNOSTICS_BYTES];
        uint16_t length;
        observerLinkDiagnostics diagnostics;

        g_status.localDiagnostics.validFrames = g_parser.validFrames;
        g_status.localDiagnostics.crcErrors = g_parser.crcErrors;
        g_status.localDiagnostics.lengthErrors = g_parser.lengthErrors;
        g_status.localDiagnostics.versionErrors = g_parser.versionErrors;
        g_status.localDiagnostics.txDeferred = g_status.txDeferred;
    }
}

```

```

    diagnostics = g_status.localDiagnostics;
    diagnostics.rxOverflowErrors += g_rxRingOverflow;
    if (observerLinkEncodeDiagnostics(&diagnostics,
        payload,
        &length) == 0u)
    {
        return 0u;
    }
    return sendMessage(OBSERVER_LINK_MSG_DIAGNOSTICS,
        payload,
        length,
        tickMs);
}

static uint8_t sendHealth(uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_HEALTH_BYTES];
    uint16_t length;
    observerLinkHealth health;

    if (g_healthPending == 0u) return 0u;
    health = g_health;
    health.flags |= OBSERVER_LINK_HEALTH_FLAG_BRIDGE_FRESH;
    health.bridgeAgeMs = 0u;
    if ((observerLinkEncodeHealth(&health, payload, &length) != 0u) &&
        (sendMessage(OBSERVER_LINK_MSG_HEALTH, payload, length, tickMs) != 0u))
    {
        g_healthPending = 0u;
        return 1u;
    }
    return 0u;
}

static uint8_t sendPendingAck(uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_CONFIG_ACK_BYTES];
    uint16_t length;

    if ((g_ackPending != 0u) &&
        (observerLinkEncodeConfigAck(&g_pendingAck, payload, &length) != 0u) &&
        (sendMessage(OBSERVER_LINK_MSG_CONFIG_ACK, payload, length, tickMs) != 0u))
    {
        g_ackPending = 0u;
        return 1u;
    }
    return 0u;
}

static uint8_t sendKvmAck(uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_KVM_ACK_BYTES];
    uint16_t length;
    if ((g_kvmAckPending != 0u) &&
        (observerLinkEncodeKvmAck(&g_kvmAck, payload, &length) != 0u) &&
        (sendControlMessage(OBSERVER_LINK_MSG_KVM_ACK, payload, length, tickMs) != 0u))
    {
        g_kvmAckPending = 0u;
        return 1u;
    }
    return 0u;
}

static uint8_t sendKvmMenuStatus(uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_KVM_MENU_STATUS_BYTES];
    uint16_t length;
    if ((g_kvmMenuPending != 0u) &&
        (observerLinkEncodeKvmMenuStatus(&g_kvmMenuStatus, payload, &length) != 0u) &&
        (sendControlMessage(OBSERVER_LINK_MSG_KVM_MENU_STATUS, payload, length, tickMs) != 0u))
    {
        g_kvmMenuPending = 0u;
        return 1u;
    }
    return 0u;
}

static uint8_t sendPlanningTrackFragment(uint32_t tickMs)
{
    uint8_t payload[OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_MAX_BYTES];

```

```

uint16_t length;
uint8_t fragmentCount;

if (g_planningActiveValid == 0u)
{
    if (g_planningLatestValid == 0u)
    {
        return 0u;
    }
    g_planningActive = g_planningLatest;
    g_planningActiveValid = 1u;
    g_planningLatestValid = 0u;
    g_planningFragmentIndex = 0u;
}
if ((g_status.online == 0u) || (g_status.protocolCompatible == 0u) ||
    (g_status.planningCompatible == 0u) ||
    (g_status.configStatus != OBSERVER_LINK_CONFIG_APPLIED) ||
    ((g_status.peerCapabilities & OBSERVER_LINK_CAP_PLANNING_TRACKS) == 0u) ||
    (g_planningActive.ownerSession != g_status.ownerSession) ||
    (g_planningActive.revision != g_status.appliedRevision) ||
    (g_planningActive.hash != g_status.appliedHash) ||
    (g_planningActive.detectMode != g_status.appliedMode))
{
    g_planningActiveValid = 0u;
    return 0u;
}
fragmentCount = observerLinkPlanningTrackFragmentCount(
    g_planningActive.retainedCount);
if (observerLinkEncodePlanningTrackFragment(&g_planningActive,
    g_planningFragmentIndex, payload, &length) == 0u)
{
    g_status.txFailures++;
    g_planningActiveValid = 0u;
    g_planningFragmentIndex = 0u;
    return 0u;
}
if (sendMessage(OBSERVER_LINK_MSG_PLANNING_TRACK_FRAGMENT,
    payload, length, tickMs) == 0u)
{
    return 0u;
}
g_planningFragmentIndex++;
if (g_planningFragmentIndex >= fragmentCount)
{
    g_planningActiveValid = 0u;
    g_planningFragmentIndex = 0u;
}
return 1u;
}

void core4LinkCore4Init(uint32_t tickMs)
{
    paramSyncMailboxRequest retainedRequest;
    paramSyncMailboxResponse retainedResponse;
    if (g_uartInitialized != 0u) uart_rx_interrupt(CORE4_CORE1_UART_NUM, 0u);
    memset(&g_status, 0, sizeof(g_status));
    observerLinkParserInit(&g_parser);
    clearOwnerState(0u);
    g_packetSeq = 0u;
    g_status.localSession = makeBootSession(tickMs);
    g_lastHelloTickMs = tickMs - OBSERVER_LINK_HELLO_PERIOD_MS;
    g_lastHeartbeatTickMs = tickMs - OBSERVER_LINK_HEARTBEAT_PERIOD_MS;
    g_lastDiagTickMs = tickMs - OBSERVER_LINK_DIAG_PERIOD_MS;
    g_lastDeferredTickMs = tickMs - 1u;
    g_txEpochTickMs = tickMs;
    g_txDataCredits = OBSERVER_LINK_TX_FRAMES_PER_EPOCH;
    g_txControlCredits = OBSERVER_LINK_TX_CONTROL_FRAMES_PER_EPOCH;
    g_planningBurstCount = 0u;
    memset(&g_kvmControl, 0, sizeof(g_kvmControl));
    memset(&g_kvmEvent, 0, sizeof(g_kvmEvent));
    memset(&g_kvmAck, 0, sizeof(g_kvmAck));
    memset(&g_kvmMenuStatus, 0, sizeof(g_kvmMenuStatus));
    g_kvmControlReady = 0u;
    g_kvmEventReady = 0u;
    g_kvmAckPending = 0u;
    g_kvmMenuPending = 0u;
    memset(&g_planningActive, 0, sizeof(g_planningActive));
    memset(&g_planningLatest, 0, sizeof(g_planningLatest));
    g_planningActiveValid = 0u;
}

```

```

g_planningLatestValid = 0u;
g_planningFragmentIndex = 0u;
g_rxHead = 0u;
g_rxTail = 0u;
g_rxRingOverflow = 0u;
g_paramAssemblyActive = 0u;
g_paramResponseValid = 0u;
g_paramReplyPending = 0u;
g_paramRequestSequence = 0u;
if (paramSyncMailboxTryReadRequest(&retainedRequest) != 0u)
    g_paramRequestSequence = retainedRequest.sequence;
g_paramOwnerBoot = 0u;
g_paramOwnerResponseSequence = 0u;
g_paramOwnerBootPrimed = 0u;
if (paramSyncMailboxTryReadResponse(&retainedResponse) != 0u)
{
    g_paramOwnerBoot = retainedResponse.producerBootGeneration;
    g_paramOwnerResponseSequence = retainedResponse.sequence;
    g_paramOwnerBootPrimed = 1u;
}
g_paramWaitingOwner = 0u;
memset(&g_paramGlobalState, 0, sizeof(g_paramGlobalState));
memset(&g_paramGlobalRouteFailure, 0,
    sizeof(g_paramGlobalRouteFailure));
memset(&g_paramGlobalRequest, 0, sizeof(g_paramGlobalRequest));
g_paramGlobalStateReady = 0u;
g_paramGlobalStateValid = 0u;
g_paramGlobalRouteFailureReady = 0u;
g_paramGlobalStateLastRxMs = tickMs;
g_paramGlobalRequestPending = 0u;
g_paramGlobalRequestLastSendMs = tickMs - OBSERVER_LINK_PARAM_GLOBAL_RETRY_MS;
g_visionReady = 0u;
uart_init(CORE4_CORE1_UART_NUM,
    OBSERVER_LINK_BAUDRATE,
    CORE4_CORE1_UART_TX_PIN,
    CORE4_CORE1_UART_RX_PIN);
uart_rx_interrupt(CORE4_CORE1_UART_NUM, 1u);
g_uartInitialized = 1u;
}

void core4LinkCore4RxIsr(void)
{
    uint8_t byte;
    while (uart_query_byte(CORE4_CORE1_UART_NUM, &byte) != 0u)
    {
        uint16_t next = (uint16_t)((g_rxHead + 1u) & OBSERVER_LINK_RX_RING_MASK);
        if (next == g_rxTail)
        {
            g_rxRingOverflow++;
            continue;
        }
        g_rxRing[g_rxHead] = byte;
        __DMB();
        g_rxHead = next;
    }
}

void core4LinkCore4SetVisionReady(uint8_t ready)
{
    g_visionReady = (ready != 0u) ? 1u : 0u;
}

static uint8_t transmitWorkPending(uint32_t tickMs)
{
    const observerLinkPlanningTrackSnapshot *planning =
        (g_planningActiveValid != 0u) ? &g_planningActive : ((g_planningLatestValid != 0u) ? &g_planningLatest : NULL);
    uint8_t planningSendable = (uint8_t)((planning != NULL) &&
        (g_status.online != 0u) && (g_status.protocolCompatible != 0u) &&
        (g_status.planningCompatible != 0u) &&
        (g_status.configStatus == OBSERVER_LINK_CONFIG_APPLIED) &&
        ((g_status.peerCapabilities & OBSERVER_LINK_CAP_PLANNING_TRACKS) != 0u) &&
        (planning->ownerSession == g_status.ownerSession) &&
        (planning->revision == g_status.appliedRevision) &&
        (planning->hash == g_status.appliedHash) &&
        (planning->detectMode == g_status.appliedMode));

    return (uint8_t)((g_ackPending != 0u) ||
        (g_paramReplyPending != 0u) ||
        ((g_paramGlobalRequestPending != 0u) &&

```



```

        ((uint32_t) (tickMs - g_paramGlobalRequestLastSendMs) >=
         OBSERVER_LINK_PARAM_GLOBAL_RETRY_MS)) ||
        (planningSendable != 0u) ||
        (g_healthPending != 0u) ||
        ((uint32_t) (tickMs - g_lastHeartbeatTickMs) >=
         OBSERVER_LINK_HEARTBEAT_PERIOD_MS) ||
        ((uint32_t) (tickMs - g_lastHelloTickMs) >=
         OBSERVER_LINK_HELLO_PERIOD_MS) ||
        ((uint32_t) (tickMs - g_lastDiagTickMs) >=
         OBSERVER_LINK_DIAG_PERIOD_MS));
    }

static uint8_t transmitOne(uint32_t tickMs)
{
    if (sendPendingAck(tickMs) != 0u) return 1u;
    if (((g_status.online == 0u) || (g_status.protocolCompatible == 0u)) &&
        ((uint32_t) (tickMs - g_lastHelloTickMs) >= OBSERVER_LINK_HELLO_PERIOD_MS) &&
        (sendHello(tickMs) != 0u))
    {
        g_lastHelloTickMs = tickMs;
        return 1u;
    }
    if (((uint32_t) (tickMs - g_lastHeartbeatTickMs) >= OBSERVER_LINK_HEARTBEAT_PERIOD_MS) &&
        (sendHeartbeat(tickMs) != 0u))
    {
        g_lastHeartbeatTickMs = tickMs;
        return 1u;
    }
    if (sendHealth(tickMs) != 0u) return 1u;
    if ((g_planningBurstCount >= OBSERVER_LINK_PLANNING_BURST_MAX) &&
        ((uint32_t) (tickMs - g_lastHelloTickMs) >= OBSERVER_LINK_HELLO_PERIOD_MS) &&
        (sendHello(tickMs) != 0u))
    {
        g_lastHelloTickMs = tickMs;
        g_planningBurstCount = 0u;
        return 1u;
    }
    if ((g_planningBurstCount >= OBSERVER_LINK_PLANNING_BURST_MAX) &&
        ((uint32_t) (tickMs - g_lastDiagTickMs) >= OBSERVER_LINK_DIAG_PERIOD_MS) &&
        (sendDiagnostics(tickMs) != 0u))
    {
        g_lastDiagTickMs = tickMs;
        g_planningBurstCount = 0u;
        return 1u;
    }
    if (sendPlanningTrackFragment(tickMs) != 0u)
    {
        if (g_planningBurstCount < 0xFFu) g_planningBurstCount++;
        return 1u;
    }
    g_planningBurstCount = 0u;
    if (((uint32_t) (tickMs - g_lastHelloTickMs) >= OBSERVER_LINK_HELLO_PERIOD_MS) &&
        (sendHello(tickMs) != 0u))
    {
        g_lastHelloTickMs = tickMs;
        return 1u;
    }
    if (((uint32_t) (tickMs - g_lastDiagTickMs) >= OBSERVER_LINK_DIAG_PERIOD_MS) &&
        (sendDiagnostics(tickMs) != 0u))
    {
        g_lastDiagTickMs = tickMs;
        return 1u;
    }
    if (sendParamReply(tickMs) != 0u) return 1u;
    if (sendParamGlobalRequest(tickMs) != 0u) return 1u;
    return 0u;
}

uint8_t core4LinkCore4QueueParamGlobalRequest(
    const paramSyncGlobalRequest *request)
{
    if ((request == NULL) || (paramSyncGlobalRequestIsValid(request) == 0u) ||
        (request->origin != OBSERVER_LINK_SOURCE_CORE4)) return 0u;
    if ((g_paramGlobalRequestPending != 0u) &&
        ((g_paramGlobalRequest.producerBootGeneration !=
         request->producerBootGeneration) ||
         (g_paramGlobalRequest.sequence != request->sequence))) return 0u;
    g_paramGlobalRequest = *request;
    g_paramGlobalRequestPending = 1u;
}

```

```

if (paramGlobalRouteReady(g_paramLastTickMs) == 0u)
{
    failPendingParamGlobalRequest(PARAM_SYNC_GLOBAL_ERROR_ROUTE);
    return 1u;
}
g_paramGlobalRequestLastSendMs -= OBSERVER_LINK_PARAM_GLOBAL_RETRY_MS;
return 1u;
}

uint8_t core4LinkCore4TakeParamGlobalState(paramSyncGlobalState *state)
{
    if (state != NULL && g_paramGlobalRouteFailureReady != 0u)
    {
        *state = g_paramGlobalRouteFailure;
        g_paramGlobalRouteFailureReady = 0u;
        return 1u;
    }
    if ((state == NULL) || (g_paramGlobalStateReady == 0u)) return 0u;
    if (paramSyncGlobalStateIsValid(&g_paramGlobalState) == 0u)
    {
        g_paramGlobalStateReady = 0u;
        return 0u;
    }
    *state = g_paramGlobalState;
    g_paramGlobalStateReady = 0u;
    return 1u;
}

uint8_t core4LinkCore4IsParamGlobalStateValid(void)
{
    return paramGlobalRouteReady(g_paramLastTickMs);
}

static void refreshTxEpoch(uint32_t tickMs)
{
    uint32_t elapsedEpochs;
    uint32_t credits;

    elapsedEpochs = (uint32_t) (tickMs - g_txEpochTickMs) /
        OBSERVER_LINK_TX_EPOCH_MS;
    if (elapsedEpochs != 0u)
    {
        g_txEpochTickMs += elapsedEpochs * OBSERVER_LINK_TX_EPOCH_MS;
        credits = (uint32_t) g_txDataCredits +
            elapsedEpochs * OBSERVER_LINK_TX_FRAMES_PER_EPOCH;
        g_txDataCredits = (uint8_t) ((credits > OBSERVER_LINK_TX_DATA_CREDIT_CAP) ? OBSERVER_LINK_TX_DATA_CREDIT_CAP : credits);
        credits = (uint32_t) g_txControlCredits +
            elapsedEpochs * OBSERVER_LINK_TX_CONTROL_FRAMES_PER_EPOCH;
        g_txControlCredits = (uint8_t) ((credits > OBSERVER_LINK_TX_CONTROL_CREDIT_CAP) ? OBSERVER_LINK_TX_CONTROL_CREDIT_CAP :
credits);
    }
}

static void serviceControlTransmit(uint32_t tickMs)
{
    refreshTxEpoch(tickMs);
    while (g_txControlCredits != 0u)
    {
        if (sendKvmAck(tickMs) != 0u) continue;
        if (sendKvmMenuStatus(tickMs) != 0u) continue;
        break;
    }
}

static void serviceTransmit(uint32_t tickMs)
{
    refreshTxEpoch(tickMs);
    while ((g_txDataCredits != 0u) &&
        (transmitOne(tickMs) != 0u))
    {
        /* 有界串行发送；完整快照的 active 分片在 latest 之前保持冻结。 */
    }
    if ((transmitWorkPending(tickMs) != 0u) &&
        (g_txDataCredits == 0u) &&
        (g_lastDeferredTickMs != tickMs))
    {
        g_lastDeferredTickMs = tickMs;
        g_status.txDeferred++;
    }
}

```

```

}

void core4LinkCore4Poll(uint32_t tickMs)
{
    g_paramLastTickMs = tickMs;
    receiveFrames(tickMs);
    if (g_paramGlobalStateValid != 0u &&
        (uint32_t) (tickMs - g_paramGlobalStateLastRxMs) >=
            CORE4_LINK_PARAM_GLOBAL_STATE_LOST_MS)
        invalidateParamGlobalRoute();
    if ((g_status.online != 0u) &&
        ((uint32_t) (tickMs - g_status.lastValidRxTickMs) >= OBSERVER_LINK_TIMEOUT_MS))
    {
        uint32_t ownerSession = g_status.ownerSession;
        g_status.online = 0u;
        g_rxPacketSeqPrimed = 0u;
        g_status.protocolCompatible = 0u;
        g_status.peerCapabilities = 0u;
        clearOwnerState(ownerSession);
    }
    if ((g_status.controlStatusValid != 0u) &&
        ((uint32_t) (tickMs - g_status.cameraControlTickMs) >= OBSERVER_LINK_TIMEOUT_MS))
    {
        clearControlStatus();
    }
    serviceControlTransmit(tickMs);
    serviceTransmit(tickMs);
}

void core4LinkCore4FlushControl(uint32_t tickMs)
{
    serviceControlTransmit(tickMs);
}

void core4LinkCore4FlushTx(uint32_t tickMs)
{
    serviceTransmit(tickMs);
}

void core4LinkCore4GetStatus(core4LinkCore4Status *status)
{
    if (status != NULL)
    {
        g_status.localDiagnostics.validFrames = g_parser.validFrames;
        g_status.localDiagnostics.crcErrors = g_parser.crcErrors;
        g_status.localDiagnostics.lengthErrors = g_parser.lengthErrors;
        g_status.localDiagnostics.versionErrors = g_parser.versionErrors;
        g_status.localDiagnostics.txDeferred = g_status.txDeferred;
        *status = g_status;
        status->localDiagnostics.rxOverflowErrors += g_rxRingOverflow;
    }
}

uint32_t core4LinkCore4GetLocalBootGeneration(void)
{
    return g_status.localSession;
}

uint32_t core4LinkCore4GetStagedConfig(observerLinkSharedConfig *config)
{
    if ((config == NULL) || (g_stagedValid == 0u))
    {
        return 0u;
    }
    *config = g_staged;
    return 1u;
}

uint8_t core4LinkCore4MarkConfigApplied(uint32_t ownerSession,
    uint32_t revision,
    uint32_t hash,
    uint8_t appliedMode)
{
    if ((g_stagedValid == 0u) ||
        (g_status.online == 0u) ||
        (g_status.protocolCompatible == 0u) ||
        (ownerSession != g_status.ownerSession) ||
        (revision != g_staged.revision) ||
        (hash != g_staged.hash) ||

```

```

        (appliedMode != g_staged.detectMode))
    {
        return 0u;
    }
    if ((g_status.appliedRevision != revision) ||
        (g_status.appliedHash != hash) ||
        (g_status.appliedMode != appliedMode))
    {
        clearViewOverlay();
    }
    g_status.appliedRevision = revision;
    g_status.appliedHash = hash;
    g_status.appliedMode = appliedMode;
    g_status.configStatus = OBSERVER_LINK_CONFIG_APPLIED;
    g_status.configReason = OBSERVER_LINK_REJECT_NONE;
    queueAck(ownerSession,
        &g_staged,
        OBSERVER_LINK_CONFIG_APPLIED,
        OBSERVER_LINK_REJECT_NONE);
    return 1u;
}

uint8_t core4LinkCore4TakeKvmControl(observerLinkKvmControl *control)
{
    if ((control == NULL) || (g_kvmControlReady == 0u)) return 0u;
    *control = g_kvmControl;
    g_kvmControlReady = 0u;
    return 1u;
}

uint8_t core4LinkCore4TakeKvmEvent(observerLinkKvmEvent *event)
{
    if ((event == NULL) || (g_kvmEventReady == 0u)) return 0u;
    *event = g_kvmEvent;
    g_kvmEventReady = 0u;
    return 1u;
}

uint8_t core4LinkCore4QueueKvmAck(const observerLinkKvmAck *ack)
{
    if (ack == NULL) return 0u;
    g_kvmAck = *ack;
    g_kvmAckPending = 1u;
    return 1u;
}

uint8_t core4LinkCore4QueueKvmMenuStatus(const observerLinkKvmMenuStatus *status)
{
    if (status == NULL) return 0u;
    g_kvmMenuStatus = *status;
    g_kvmMenuPending = 1u;
    return 1u;
}

uint8_t core4LinkCore4QueuePlanningTrackSnapshot(
    const observerLinkPlanningTrackSnapshot *snapshot)
{
    if ((snapshot == NULL) || (snapshot->valid == 0u) ||
        (snapshot->sourceId != OBSERVER_LINK_SOURCE_CORE4) ||
        (snapshot->retainedCount > OBSERVER_LINK_MAX_TRACKS))
    {
        return 0u;
    }
    if (g_planningActiveValid == 0u)
    {
        g_planningActive = *snapshot;
        g_planningActiveValid = 1u;
        g_planningFragmentIndex = 0u;
    }
    else
    {
        observerLinkPlanningTrackSnapshot next = *snapshot;
        if ((g_planningLatestValid != 0u) &&
            (core4CorePlanningPreserveRejections(&next,
                &g_planningLatest) == 0u)) return 0u;
        g_planningLatest = next;
        g_planningLatestValid = 1u;
    }
}

```

```

        return 1u;
    }

uint8_t core4LinkCore4PublishHealth(const observerLinkHealth *health)
{
    uint32_t bridgeSeq;
    if ((health == NULL) ||
        (health->sourceId != OBSERVER_LINK_SOURCE_CORE4)) return 0u;
    bridgeSeq = g_health.bridgeSeq + 1u;
    if (bridgeSeq == 0u) bridgeSeq = 1u;
    g_health = *health;
    g_health.bridgeSeq = bridgeSeq;
    g_health.bridgeAgeMs = 0u;
    g_health.flags |= OBSERVER_LINK_HEALTH_FLAG_BRIDGE_FRESH;
    g_healthPending = 1u;
    return 1u;
}

uint8_t core4LinkCore4GetViewOverlay(observerLinkViewOverlay *overlay,
    uint32_t tickMs, uint32_t timeoutMs, uint32_t *ageMs)
{
    uint32_t age;

    if ((overlay == NULL) || (g_status.viewOverlayValid == 0u)) return 0u;
    age = tickMs - g_status.viewOverlayTickMs;
    if (ageMs != NULL) *ageMs = age;
    if ((age > timeoutMs) || (g_status.online == 0u) ||
        (g_status.protocolCompatible == 0u) ||
        ((g_status.peerCapabilities & OBSERVER_LINK_CAP_VIEW_OVERLAY) == 0u) ||
        (g_status.viewOverlay.ownerSession != g_status.ownerSession) ||
        (g_status.viewOverlay.configRevision != g_status.appliedRevision) ||
        (g_status.viewOverlay.configHash != g_status.appliedHash) ||
        (g_status.viewOverlay.detectMode != g_status.appliedMode)) return 0u;
    *overlay = g_status.viewOverlay;
    return 1u;
}

```

core4_cyt4bb7/project/code/comm/core4LinkCore4.h

SHA-256

:

c56407d642abfb4d5e8ee1cb0ed2016361b53678a1c0983f9230f593349b18b2

代码 (c)

```

#ifndef OBSERVER_LINK_CORE4_H
#define OBSERVER_LINK_CORE4_H

#include <stdint.h>

#include "observerLinkProtocol.h"

#define CORE4_LINK_PARAM_GLOBAL_STATE_LOST_MS (300u)

typedef struct
{
    uint8_t online;
    uint8_t protocolCompatible;
    uint8_t planningCompatible;
    uint8_t configStatus;
    uint8_t configReason;
    uint8_t stagedMode;
    uint8_t appliedMode;
    uint8_t cameraControlState;
    uint8_t cameraControlValid;
    uint8_t lastKnownCameraControlState;
    uint8_t controlStatusValid;
    uint32_t localSession;
    uint32_t ownerSession;
    uint32_t bindingEpoch;
    uint32_t peerCapabilities;
    uint32_t stagedRevision;
    uint32_t stagedHash;
    uint32_t appliedRevision;
    uint32_t appliedHash;
    uint32_t lastValidRxTickMs;
}

```

```

uint32_t rxFrames;
uint32_t txFrames;
uint32_t txFailures;
uint32_t txDeferred;
observerLinkDiagnostics localDiagnostics;
observerLinkDiagnostics peerDiagnostics;
uint32_t peerDiagnosticsTickMs;
uint32_t cameraControlTickMs;
observerLinkControlStatus controlStatus;
uint8_t viewOverlayValid;
uint32_t viewOverlayTickMs;
observerLinkViewOverlay viewOverlay;
} core4LinkCore4Status;

void core4LinkCore4Init(uint32_t tickMs);
void core4LinkCore4SetVisionReady(uint8_t ready);
void core4LinkCore4Poll(uint32_t tickMs);
void core4LinkCore4FlushControl(uint32_t tickMs);
void core4LinkCore4FlushTx(uint32_t tickMs);
void core4LinkCore4RxIsr(void);
void core4LinkCore4GetStatus(core4LinkCore4Status *status);
uint32_t core4LinkCore4GetLocalBootGeneration(void);
uint8_t core4LinkCore4GetStagedConfig(observerLinkSharedConfig *config);
uint8_t core4LinkCore4MarkConfigApplied(uint32_t ownerSession,
    uint32_t revision,
    uint32_t hash,
    uint8_t appliedMode);
uint8_t core4LinkCore4TakeKvmControl(observerLinkKvmControl *control);
uint8_t core4LinkCore4TakeKvmEvent(observerLinkKvmEvent *event);
uint8_t core4LinkCore4QueueKvmAck(const observerLinkKvmAck *ack);
uint8_t core4LinkCore4QueueKvmMenuStatus(const observerLinkKvmMenuStatus *status);
uint8_t core4LinkCore4QueuePlanningTrackSnapshot(
    const observerLinkPlanningTrackSnapshot *snapshot);
uint8_t core4LinkCore4PublishHealth(const observerLinkHealth *health);
uint8_t core4LinkCore4GetViewOverlay(observerLinkViewOverlay *overlay,
    uint32_t tickMs, uint32_t timeoutMs, uint32_t *ageMs);
uint8_t core4LinkCore4QueueParamGlobalRequest(
    const paramSyncGlobalRequest *request);
uint8_t core4LinkCore4TakeParamGlobalState(paramSyncGlobalState *state);
uint8_t core4LinkCore4IsParamGlobalStateValid(void);

#endif /* OBSERVER_LINK_CORE4_H */

```

core4_cyt4bb7/project/code/comm/core4LinkProxy.c

SHA-256

:

a2c9d6622e9ca6824bad6298f10e42f89493eca17863fbfddb517c8be177677d

代码 (c)

```

#include "core4LinkCore4.h"

#include <string.h>

#include "core4CoreDataTransfer.h"
#include "configCore4Runtime.h"

#define CORE4_CORE_LOCAL_KVM_DEPTH (4u)
#define CORE4_CORE_REJECTION_BACKLOG_MAX (1024u)

typedef struct
{
    uint32_t ownerSession;
    uint32_t revision;
    uint32_t hash;
    uint32_t trackerFrameSeq;
    uint16_t count;
    uint16_t trackIds[CORE4_CORE_REJECTION_BACKLOG_MAX];
} core4LocalBorderRejectionBacklog;

static core4CoreLinkSnapshot g_link;
static core4CoreVisionSnapshot g_vision;
static uint32_t g_tickMs;
static uint32_t g_lastLinkPublishSeq;
static uint32_t g_lastLinkPublishTickMs;

```

```

static uint32_t g_lastLinkProbeTickMs;
static uint32_t g_lastParamGlobalStateSeq;
static uint32_t g_lastParamGlobalStateTickMs;
static uint32_t g_lastVisionPublishTickMs;
static uint32_t g_lastRemoteRxTick;
static uint32_t g_lastRemoteControlTick;
static uint32_t g_lastRemoteDiagTick;
static uint32_t g_lastRemoteOverlaySeq;
static uint32_t g_localRxTick;
static uint32_t g_localControlTick;
static uint32_t g_localDiagTick;
static uint32_t g_localOverlayTick;
static observerLinkKvmControl g_controlQueue[CORE4_CORE_LOCAL_KVM_DEPTH];
static observerLinkKvmEvent g_eventQueue[CORE4_CORE_LOCAL_KVM_DEPTH];
static uint8_t g_controlHead;
static uint8_t g_controlTail;
static uint8_t g_controlCount;
static uint8_t g_eventHead;
static uint8_t g_eventTail;
static uint8_t g_eventCount;
static uint8_t g_linkWasFresh;
static uint8_t g_linkBaselinePending;
static uint8_t g_visionDirty;
static core4LocalBorderRejectionBacklog g_pendingBorderRejection;
static uint8_t g_pendingBorderRejectionValid;

static void clearKvmQueues(void)
{
    g_controlHead = 0u;
    g_controlTail = 0u;
    g_controlCount = 0u;
    g_eventHead = 0u;
    g_eventTail = 0u;
    g_eventCount = 0u;
}

static void publishVision(void)
{
    g_vision.publishSeq++;
    g_vision.beginGeneration += 2u;
    if (g_vision.beginGeneration == 0u) g_vision.beginGeneration = 2u;
    g_vision.endGeneration = g_vision.beginGeneration;
    core4CoreDataWriteVision(&g_vision);
    g_lastVisionPublishTickMs = g_tickMs;
    g_visionDirty = 0u;
}

static void markVisionDirty(void)
{
    g_visionDirty = 1u;
}

static void clearPendingBorderRejections(void)
{
    memset(&g_pendingBorderRejection, 0, sizeof(g_pendingBorderRejection));
    g_pendingBorderRejectionValid = 0u;
}

static void flushPendingBorderRejections(void)
{
    while ((g_pendingBorderRejectionValid != 0u) &&
           (g_pendingBorderRejection.count != 0u))
    {
        core4CoreEgressEvent event;
        core4CoreBorderRejection *payload;
        uint16_t sent;
        memset(&event, 0, sizeof(event));
        event.kind = CORE4_CORE_EGRESS_BORDER_REJECTION;
        payload = &event.payload.borderRejection;
        payload->ownerSession = g_pendingBorderRejection.ownerSession;
        payload->revision = g_pendingBorderRejection.revision;
        payload->hash = g_pendingBorderRejection.hash;
        payload->trackerFrameSeq = g_pendingBorderRejection.trackerFrameSeq;
        sent = g_pendingBorderRejection.count;
        if (sent > CORE4_CORE_REJECTION_TRACK_MAX)
            sent = CORE4_CORE_REJECTION_TRACK_MAX;
        payload->rejectedCount = (uint8_t) sent;
        memcpy(payload->rejectedTrackIds, g_pendingBorderRejection.trackIds,
               (size_t) sent * sizeof(uint16_t));
    }
}

```

```

        if (core4CoreDataPushEgress(&event) == 0u) break;
        g_pendingBorderRejection.count = (uint16_t)
            (g_pendingBorderRejection.count - sent);
        if (g_pendingBorderRejection.count != 0u)
            memmove(g_pendingBorderRejection.trackIds,
                    &g_pendingBorderRejection.trackIds[sent],
                    (size_t) g_pendingBorderRejection.count * sizeof(uint16_t));
    }
    if ((g_pendingBorderRejectionValid != 0u) &&
        (g_pendingBorderRejection.count == 0u))
        clearPendingBorderRejections();
}

static void flushVision(uint32_t tickMs)
{
    g_tickMs = tickMs;
    if ((g_visionDirty != 0u) ||
        ((uint32_t) (tickMs - g_lastVisionPublishTickMs) >=
         CORE4_CORE_SHARED_HEARTBEAT_MS))
    {
        publishVision();
    }
}

static void refreshLink(uint32_t tickMs)
{
    core4CoreLinkSnapshot incoming;
    uint32_t bootGeneration;
    uint32_t publishSeq;
    uint8_t bindingChanged;
    uint8_t rebaseline;

    if (g_lastLinkProbeTickMs == tickMs) return;
    g_lastLinkProbeTickMs = tickMs;
    if (core4CoreDataProbeLink(&bootGeneration, &publishSeq) == 0u) return;
    if ((bootGeneration == g_link.bootGeneration) &&
        (publishSeq == g_lastLinkPublishSeq)) return;
    if (core4CoreDataTryReadLink(&incoming) == 0u) return;
    rebaseline = (uint8_t) ((g_linkBaselinePending != 0u) ||
        ((g_link.magic == CORE4_CORE_LINK_MAGIC) &&
         (incoming.bootGeneration != g_link.bootGeneration)));
    bindingChanged = (uint8_t) (
        (g_link.magic == CORE4_CORE_LINK_MAGIC) &&
        ((incoming.status.ownerSession != g_link.status.ownerSession) ||
         (incoming.status.bindingEpoch != g_link.status.bindingEpoch)));
    if ((g_link.magic != CORE4_CORE_LINK_MAGIC) ||
        (incoming.bootGeneration != g_link.bootGeneration))
    {
        if (g_link.magic == CORE4_CORE_LINK_MAGIC)
        {
            core4CoreDataNoteSessionReset();
        }
        core4CoreDataSyncIngressConsumer(incoming.bootGeneration);
        if (g_vision.appliedSeq != 0u)
        {
            g_vision.appliedSeq++;
            if (g_vision.appliedSeq == 0u) g_vision.appliedSeq = 1u;
            markVisionDirty();
        }
        clearKvmQueues();
        clearPendingBorderRejections();
        g_linkWasFresh = 0u;
        g_lastRemoteRxTick = 0u;
        g_lastRemoteControlTick = 0u;
        g_lastRemoteDiagTick = 0u;
        g_lastRemoteOverlaySeq = 0u;
    }
    else if (bindingChanged != 0u)
    {
        core4CoreDataSyncIngressConsumer(incoming.bootGeneration);
        g_vision.appliedOwnerSession = 0u;
        clearKvmQueues();
        clearPendingBorderRejections();
        g_lastRemoteRxTick = 0u;
        g_lastRemoteControlTick = 0u;
        g_lastRemoteDiagTick = 0u;
        g_lastRemoteOverlaySeq = 0u;
        g_lastParamGlobalStateSeq = incoming.paramGlobalStateSeq;
        g_lastParamGlobalStateTickMs =

```



```

        tickMs - CORE4_LINK_PARAM_GLOBAL_STATE_LOST_MS;
    }
    if (rebaseline != 0u)
    {
        /* Shared RAM is __no_init. The first coherent sample establishes only
        * producer identity and sequence; an advancing outer publication must
        * follow before it can become fresh. READY additionally requires an
        * advancing parameter-state sequence. */
        g_lastLinkPublishSeq = incoming.publishSeq;
        g_lastLinkPublishTickMs = tickMs - CORE4_CORE_SHARED_STALE_MS;
        g_lastParamGlobalStateSeq = incoming.paramGlobalStateSeq;
        g_lastParamGlobalStateTickMs =
            tickMs - CORE4_LINK_PARAM_GLOBAL_STATE_LOST_MS;
        g_lastRemoteRxTick = incoming.status.lastValidRxTickMs;
        g_lastRemoteControlTick = incoming.status.cameraControlTickMs;
        g_lastRemoteDiagTick = incoming.status.peerDiagnosticsTickMs;
        g_lastRemoteOverlaySeq = (incoming.status.viewOverlayValid != 0u) ?
            incoming.status.viewOverlay.overlaySeq : 0u;
        g_localRxTick = tickMs;
        g_localControlTick = tickMs;
        g_localDiagTick = tickMs;
        g_localOverlayTick = tickMs;
        incoming.status.lastValidRxTickMs = g_localRxTick;
        incoming.status.cameraControlTickMs = g_localControlTick;
        incoming.status.peerDiagnosticsTickMs = g_localDiagTick;
        incoming.status.viewOverlayTickMs = g_localOverlayTick;
        g_link = incoming;
        g_linkBaselinePending = 0u;
        return;
    }
    if (incoming.publishSeq != g_lastLinkPublishSeq)
    {
        g_lastLinkPublishSeq = incoming.publishSeq;
        g_lastLinkPublishTickMs = tickMs;
        if (incoming.paramGlobalStateSeq == 0u)
        {
            g_lastParamGlobalStateSeq = 0u;
            g_lastParamGlobalStateTickMs =
                tickMs - CORE4_LINK_PARAM_GLOBAL_STATE_LOST_MS;
        }
        else if ((incoming.paramGlobalStateSeq !=
            g_lastParamGlobalStateSeq) ||
            ((g_linkWasFresh == 0u) &&
            (paramSyncGlobalStatesValid(
                &incoming.paramGlobalState) != 0u) &&
            ((incoming.paramGlobalState.flags &
                PARAM_SYNC_GLOBAL_FLAG_TERMINAL) != 0u)))
        {
            g_lastParamGlobalStateSeq = incoming.paramGlobalStateSeq;
            g_lastParamGlobalStateTickMs = tickMs;
        }
    }
    if (incoming.status.lastValidRxTickMs != g_lastRemoteRxTick)
    {
        g_lastRemoteRxTick = incoming.status.lastValidRxTickMs;
        g_localRxTick = tickMs;
    }
    if (incoming.status.cameraControlTickMs != g_lastRemoteControlTick)
    {
        g_lastRemoteControlTick = incoming.status.cameraControlTickMs;
        g_localControlTick = tickMs;
    }
    if (incoming.status.peerDiagnosticsTickMs != g_lastRemoteDiagTick)
    {
        g_lastRemoteDiagTick = incoming.status.peerDiagnosticsTickMs;
        g_localDiagTick = tickMs;
    }
    if ((incoming.status.viewOverlayValid != 0u) &&
        (incoming.status.viewOverlay.overlaySeq != g_lastRemoteOverlaySeq))
    {
        g_lastRemoteOverlaySeq = incoming.status.viewOverlay.overlaySeq;
        g_localOverlayTick = tickMs;
    }
    incoming.status.lastValidRxTickMs = g_localRxTick;
    incoming.status.cameraControlTickMs = g_localControlTick;
    incoming.status.peerDiagnosticsTickMs = g_localDiagTick;
    incoming.status.viewOverlayTickMs = g_localOverlayTick;
    g_link = incoming;
}

```

```

static uint8_t linkFresh(void)
{
    return (uint8_t) ((g_link.magic == CORE4_CORE_LINK_MAGIC) &&
        ((uint32_t) (g_tickMs - g_lastLinkPublishTickMs) <
            CORE4_CORE_SHARED_STALE_MS));
}

static uint8_t paramGlobalStateFresh(void)
{
    return (uint8_t) ((linkFresh() != 0u) &&
        (g_link.paramGlobalStateSeq != 0u) &&
        (g_link.paramGlobalStateSeq == g_lastParamGlobalStateSeq) &&
        ((uint32_t) (g_tickMs - g_lastParamGlobalStateTickMs) <
            CORE4_LINK_PARAM_GLOBAL_STATE_LOST_MS) &&
        (paramSyncGlobalStateIsValid(&g_link.paramGlobalState) != 0u));
}

void core4LinkCore4Init(uint32_t tickMs)
{
    core4CoreDataTimingInit();
    memset(&g_link, 0, sizeof(g_link));
    memset(&g_vision, 0, sizeof(g_vision));
    g_tickMs = tickMs;
    g_lastLinkProbeTickMs = UINT32_MAX;
    g_lastVisionPublishTickMs = tickMs;
    g_lastParamGlobalStateSeq = 0u;
    g_lastParamGlobalStateTickMs =
        tickMs - CORE4_LINK_PARAM_GLOBAL_STATE_LOST_MS;
    g_visionDirty = 0u;
    clearPendingBorderRejections();
    g_linkWasFresh = 0u;
    g_linkBaselinePending = 1u;
    clearKvmQueues();
    g_vision.magic = CORE4_CORE_VISION_MAGIC;
    g_vision.abiVersion = CORE4_CORE_ABI_VERSION;
    g_vision.payloadBytes = sizeof(g_vision);
    g_vision.bootGeneration = core4CoreDataMakeBootGeneration(0xC4710001u);
    core4CoreDataResetEgress(g_vision.bootGeneration);
    g_vision.beginGeneration = 2u;
    g_vision.endGeneration = 2u;
    publishVision();
    refreshLink(tickMs);
}

void core4LinkCore4SetVisionReady(uint8_t ready)
{
    ready = (uint8_t) (ready != 0u);
    if ((g_vision.visionReadySeq == 0u) || (g_vision.visionReady != ready))
    {
        g_vision.visionReady = ready;
        g_vision.visionReadySeq++;
        markVisionDirty();
    }
}

void core4LinkCore4Poll(uint32_t tickMs)
{
    core4CoreIngressEvent event;
    g_tickMs = tickMs;
    refreshLink(tickMs);
    if (linkFresh() == 0u)
    {
        if (g_linkWasFresh != 0u)
        {
            core4CoreDataNoteSharedState();
            core4CoreDataSyncIngressConsumer(g_link.bootGeneration);
            clearKvmQueues();
            clearPendingBorderRejections();
        }
        g_linkWasFresh = 0u;
        return;
    }
    g_linkWasFresh = 1u;
    flushPendingBorderRejections();
    while ((g_controlCount < CORE4_CORE_LOCAL_KVM_DEPTH) &&
        (g_eventCount < CORE4_CORE_LOCAL_KVM_DEPTH) &&
        (core4CoreDataPopIngress(&event, g_link.bootGeneration) != 0u))
    {

```

```

        if (event.kind == CORE4_CORE_INGRESS_KVM_CONTROL)
        {
            g_controlQueue[g_controlHead] = event.payload.control;
            g_controlHead = (uint8_t)((g_controlHead + 1u) %
                CORE4_CORE_LOCAL_KVM_DEPTH);
            g_controlCount++;
        }
        else if (event.kind == CORE4_CORE_INGRESS_KVM_EVENT)
        {
            g_eventQueue[g_eventHead] = event.payload.event;
            g_eventHead = (uint8_t)((g_eventHead + 1u) %
                CORE4_CORE_LOCAL_KVM_DEPTH);
            g_eventCount++;
        }
    }
}

void core4LinkCore4FlushControl(uint32_t tickMs)
{
    flushVision(tickMs);
}

void core4LinkCore4FlushTx(uint32_t tickMs)
{
    flushVision(tickMs);
}

void core4LinkCore4RxIsr(void)
{
}

void core4LinkCore4GetStatus(core4LinkCore4Status *status)
{
    if (status == NULL) return;
    if (linkFresh() != 0u)
    {
        *status = g_link.status;
    }
    else
    {
        memset(status, 0, sizeof(*status));
        status->stagedMode = 0xFFu;
        status->appliedMode = 0xFFu;
    }
}

uint32_t core4LinkCore4GetLocalBootGeneration(void)
{
    return g_vision.bootGeneration;
}

uint8_t core4LinkCore4GetStagedConfig(observerLinkSharedConfig *config)
{
    if ((config == NULL) || (linkFresh() == 0u) ||
        (g_link.stagedConfigSeq == 0u)) return 0u;
    *config = g_link.stagedConfig;
    return 1u;
}

uint8_t core4LinkCore4MarkConfigApplied(uint32_t ownerSession,
    uint32_t revision, uint32_t hash, uint8_t appliedMode)
{
    if ((linkFresh() == 0u) ||
        ((g_vision.appliedSeq != 0u) &&
            (g_link.appliedConsumedSeq != g_vision.appliedSeq) &&
            (g_vision.appliedOwnerSession == ownerSession))) return 0u;
    g_vision.appliedOwnerSession = ownerSession;
    g_vision.appliedRevision = revision;
    g_vision.appliedHash = hash;
    g_vision.appliedMode = appliedMode;
    g_vision.appliedSeq++;
    if (g_vision.appliedSeq == 0u) g_vision.appliedSeq = 1u;
    markVisionDirty();
    return 1u;
}

uint8_t core4LinkCore4TakeKvmControl(observerLinkKvmControl *control)
{
    if ((control == NULL) || (linkFresh() == 0u) ||

```

```

        (g_controlCount == 0u)) return 0u;
    *control = g_controlQueue[g_controlTail];
    g_controlTail = (uint8_t)((g_controlTail + 1u) %
        CORE4_CORE_LOCAL_KVM_DEPTH);
    g_controlCount--;
    return 1u;
}

uint8_t core4LinkCore4TakeKvmEvent(observerLinkKvmEvent *event)
{
    if ((event == NULL) || (linkFresh() == 0u) ||
        (g_eventCount == 0u)) return 0u;
    *event = g_eventQueue[g_eventTail];
    g_eventTail = (uint8_t)((g_eventTail + 1u) %
        CORE4_CORE_LOCAL_KVM_DEPTH);
    g_eventCount--;
    return 1u;
}

uint8_t core4LinkCore4QueueKvmAck(const observerLinkKvmAck *ack)
{
    core4CoreEgressEvent event;
    if (ack == NULL) return 0u;
    memset(&event, 0, sizeof(event));
    event.kind = CORE4_CORE_EGRESS_KVM_ACK;
    event.payload.ack = *ack;
    return core4CoreDataPushEgress(&event);
}

uint8_t core4LinkCore4QueueKvmMenuStatus(const observerLinkKvmMenuStatus *status)
{
    core4CoreEgressEvent event;
    if (status == NULL) return 0u;
    memset(&event, 0, sizeof(event));
    event.kind = CORE4_CORE_EGRESS_KVM_MENU;
    event.payload.menuStatus = *status;
    return core4CoreDataPushEgress(&event);
}

uint8_t core4LinkCore4QueuePlanningTrackSnapshot(
    const observerLinkPlanningTrackSnapshot *snapshot)
{
    core4CoreEgressEvent rejection;
    core4CoreBorderRejection *event;
    uint8_t i;
    if (snapshot == NULL) return 0u;

    flushPendingBorderRejections();
    if (g_pendingBorderRejectionValid != 0u)
    {
        if ((g_pendingBorderRejection.ownerSession != snapshot->ownerSession) ||
            (g_pendingBorderRejection.revision != snapshot->revision) ||
            (g_pendingBorderRejection.hash != snapshot->hash))
        {
            clearPendingBorderRejections();
        }
    }

    memset(&rejection, 0, sizeof(rejection));
    rejection.kind = CORE4_CORE_EGRESS_BORDER_REJECTION;
    event = &rejection.payload.borderRejection;
    event->ownerSession = snapshot->ownerSession;
    event->revision = snapshot->revision;
    event->hash = snapshot->hash;
    event->trackerFrameSeq = snapshot->trackerFrameSeq;
    for (i = 0u; i < snapshot->retainedCount; i++)
    {
        const observerLinkPlanningTrack *track = &snapshot->tracks[i];
        if ((track->flags &
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) == 0u) continue;
        if ((track->trackId == 0u) ||
            (event->rejectedCount >= CORE4_CORE_REJECTION_TRACK_MAX))
            return 0u;
        event->rejectedTrackIds[event->rejectedCount++] = track->trackId;
    }
    if (event->rejectedCount != 0u)
    {
        uint16_t uniqueAdded = 0u;
        if (g_pendingBorderRejectionValid == 0u)

```

```

    {
        g_pendingBorderRejection.ownerSession = event->ownerSession;
        g_pendingBorderRejection.revision = event->revision;
        g_pendingBorderRejection.hash = event->hash;
        g_pendingBorderRejection.trackerFrameSeq = event->trackerFrameSeq;
        g_pendingBorderRejectionValid = 1u;
    }
    for (i = 0u; i < event->rejectedCount; i++)
    {
        uint16_t j;
        for (j = 0u; j < g_pendingBorderRejection.count; j++)
            if (g_pendingBorderRejection.trackIds[j] ==
                event->rejectedTrackIds[i]) break;
        if (j == g_pendingBorderRejection.count) uniqueAdded++;
    }
    if ((uint32_t) g_pendingBorderRejection.count + uniqueAdded >
        CORE4_CORE_REJECTION_BACKLOG_MAX) return 0u;
    for (i = 0u; i < event->rejectedCount; i++)
    {
        uint16_t j;
        for (j = 0u; j < g_pendingBorderRejection.count; j++)
            if (g_pendingBorderRejection.trackIds[j] ==
                event->rejectedTrackIds[i]) break;
        if (j == g_pendingBorderRejection.count)
            g_pendingBorderRejection.trackIds[
                g_pendingBorderRejection.count++] =
                    event->rejectedTrackIds[i];
    }
    if (observerLinkSequenceIsNewer(event->trackerFrameSeq,
        g_pendingBorderRejection.trackerFrameSeq) != 0u)
        g_pendingBorderRejection.trackerFrameSeq = event->trackerFrameSeq;
    flushPendingBorderRejections();
}
g_vision.planningSnapshot = *snapshot;
g_vision.planningSeq++;
markVisionDirty();
return 1u;
}

uint8_t core4LinkCore4PublishHealth(const observerLinkHealth *health)
{
    if ((health == NULL) ||
        (health->sourceId != OBSERVER_LINK_SOURCE_CORE4)) return 0u;
    g_vision.health = *health;
    g_vision.healthSeq++;
    if (g_vision.healthSeq == 0u) g_vision.healthSeq++;
    markVisionDirty();
    return 1u;
}

uint8_t core4LinkCore4GetViewOverlay(observerLinkViewOverlay *overlay,
    uint32_t tickMs, uint32_t timeoutMs, uint32_t *ageMs)
{
    uint32_t age;
    if ((overlay == NULL) || (linkFresh() == 0u) ||
        (g_link.status.viewOverlayValid == 0u)) return 0u;
    age = tickMs - g_localOverlayTick;
    if (ageMs != NULL) *ageMs = age;
    if (age > timeoutMs) return 0u;
    *overlay = g_link.status.viewOverlay;
    return 1u;
}

uint8_t core4LinkCore4QueueParamGlobalRequest(
    const paramSyncGlobalRequest *request)
{
    if ((request == NULL) || (paramSyncGlobalRequestIsValid(request) == 0u) ||
        (request->origin != OBSERVER_LINK_SOURCE_CORE4)) return 0u;
    if ((g_vision.paramGlobalRequestSeq != 0u) &&
        (g_vision.paramGlobalRequest.producerBootGeneration ==
            request->producerBootGeneration) &&
        (g_vision.paramGlobalRequest.sequence == request->sequence)) return 1u;
    g_vision.paramGlobalRequest = *request;
    g_vision.paramGlobalRequestSeq++;
    if (g_vision.paramGlobalRequestSeq == 0u)
        g_vision.paramGlobalRequestSeq = 1u;
    markVisionDirty();
    return 1u;
}

```

```

uint8_t core4LinkCore4TakeParamGlobalState(paramSyncGlobalState *state)
{
    if ((state == NULL) ||
        (core4LinkCore4IsParamGlobalStateValid() == 0u)) return 0u;
    *state = g_link.paramGlobalState;
    if (g_vision.paramGlobalStateConsumedSeq !=
        g_link.paramGlobalStateSeq)
    {
        g_vision.paramGlobalStateConsumedSeq =
            g_link.paramGlobalStateSeq;
        markVisionDirty();
    }
    return 1u;
}

uint8_t core4LinkCore4IsParamGlobalStateValid(void)
{
    if (paramGlobalStateFresh() == 0u) return 0u;
    if ((g_link.paramGlobalState.flags &
        PARAM_SYNC_GLOBAL_FLAG_TERMINAL) != 0u) return 1u;
    return (uint8_t) (g_link.status.online != 0u);
}

```

core4_cyt4bb7/project/code/comm/core4LinkSemantic.c

SHA-256

:

5fcd0a31c03de367f41367b88f468880eb2f93700188063823cb398619cb3abe

代码 (c)

```

#include "observerLinkProtocol.h"

#include <string.h>

#include "crc32leee.h"

static void putU16(uint8_t *dst, uint16_t value)
{
    dst[0] = (uint8_t) value;
    dst[1] = (uint8_t) (value >> 8);
}

static void putFloat(uint8_t *dst, float value)
{
    uint32_t bits;
    memcpy(&bits, &value, sizeof(bits));
    dst[0] = (uint8_t) bits;
    dst[1] = (uint8_t) (bits >> 8);
    dst[2] = (uint8_t) (bits >> 16);
    dst[3] = (uint8_t) (bits >> 24);
}

static uint8_t finiteInRange(float value, float minimum, float maximum)
{
    if ((value != value) || (value > 1.0e30f) || (value < -1.0e30f))
    {
        return 0u;
    }
    return (uint8_t) ((value >= minimum) && (value <= maximum));
}

uint8_t observerLinkQuantizeSigned(float value, float scale,
    float minimum, float maximum, int16_t *quantized)
{
    float scaled;

    if ((quantized == NULL) ||
        (finiteInRange(value, minimum, maximum) == 0u) ||
        (finiteInRange(scale, 0.000001f, 1.0e30f) == 0u))
    {
        return 0u;
    }
    scaled = value * scale;
    if (finiteInRange(scaled, -32768.0f, 32767.0f) == 0u)

```

```

    {
        return 0u;
    }
    scaled += (scaled >= 0.0f) ? 0.5f : -0.5f;
    *quantized = (int16_t) scaled;
    return 1u;
}

static void packSharedConfig(const observerLinkSharedConfig *config,
    uint8_t payload[OBSERVER_LINK_SHARED_CONFIG_BYTES])
{
    uint8_t i;
    memset(payload, 0, OBSERVER_LINK_SHARED_CONFIG_BYTES);
    payload[8] = config->detectMode;
    payload[9] = config->maxBeacons;
    putU16(&payload[10], config->edgePredictionHoldMs);
    putU16(&payload[12], config->beaconSizeMin);
    putU16(&payload[14], config->beaconSizeMax);
    putFloat(&payload[16], config->beaconEccMax);
    payload[20] = config->beaconBrightnessMin;
    payload[21] = config->sourceId;
    putU16(&payload[22], config->beaconMaxEndpoints);
    putFloat(&payload[24], config->beaconFillMin);
    putFloat(&payload[28], config->beaconHuMax);
    putU16(&payload[32], config->beaconHuAreaMin);
    payload[34] = config->profileVersion;
    putFloat(&payload[36], config->trackMaxAssocDist);
    putFloat(&payload[40], config->trackSizeRatioMax);
    putU16(&payload[44], config->trackConfirmHits);
    putU16(&payload[46], config->trackDropMisses);
    putU16(&payload[48], config->trackCoastMisses);
    putFloat(&payload[52], config->trackPositionAlpha);
    putFloat(&payload[56], config->trackSizeAlpha);
    putFloat(&payload[60], config->trackAreaVelAlpha);
    putFloat(&payload[64], config->trackAssocScoreMax);
    putFloat(&payload[68], config->trackAssocAreaWeight);
    putFloat(&payload[72], config->trackAssocAreaVelWeight);
    for (i = 0u; i < 7u; i++)
    {
        putFloat(&payload[76u + (uint16_t) i * 4u],
            config->beaconHuTemplate[i]);
    }
    payload[104] = config->c1BorderEnterLeft;
    payload[105] = config->c1BorderEnterRight;
    payload[106] = config->c1BorderEnterTop;
    payload[107] = config->c1BorderEnterBottom;
    payload[108] = config->c1BorderExitLeft;
    payload[109] = config->c1BorderExitRight;
    payload[110] = config->c1BorderExitTop;
    payload[111] = config->c1BorderExitBottom;
    payload[112] = config->observerBorderEnterLeft;
    payload[113] = config->observerBorderEnterRight;
    payload[114] = config->observerBorderEnterTop;
    payload[115] = config->observerBorderEnterBottom;
    payload[116] = config->observerBorderExitLeft;
    payload[117] = config->observerBorderExitRight;
    payload[118] = config->observerBorderExitTop;
    payload[119] = config->observerBorderExitBottom;
    payload[120] = config->c1EdgeBandLeft;
    payload[121] = config->c1EdgeBandRight;
    payload[122] = config->c1EdgeBandTop;
    payload[123] = config->c1EdgeBandBottom;
    payload[124] = config->observerEdgeBandLeft;
    payload[125] = config->observerEdgeBandRight;
    payload[126] = config->observerEdgeBandTop;
    payload[127] = config->observerEdgeBandBottom;
}

uint32_t observerLinkSharedConfigHash(const observerLinkSharedConfig *config)
{
    uint8_t payload[OBSERVER_LINK_SHARED_CONFIG_BYTES];
    if (config == NULL) return 0u;
    packSharedConfig(config, payload);
    return crc32IeeeCalc(&payload[8], OBSERVER_LINK_SHARED_CONFIG_BYTES - 8u);
}

static uint8_t borderSideValid(uint8_t enter, uint8_t exit, uint8_t maximum)
{
    if ((enter == 0u) && (exit == 0u)) return 1u;

```

```

        return (uint8_t) ((enter >= 1u) && (enter <= maximum) && (exit < enter));
    }

static uint8_t sharedConfigBorderValid(const observerLinkSharedConfig *config)
{
    return (uint8_t) (
        (config->edgePredictionHoldMs <= OBSERVER_LINK_EDGE_PREDICTION_HOLD_MS_MAX) &&
        (config->sourceId == OBSERVER_LINK_SOURCE_CORE4) &&
        (config->profileVersion == 1u) &&
        (config->c1EdgeBandLeft <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
        (config->c1EdgeBandRight <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
        (config->c1EdgeBandTop <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        (config->c1EdgeBandBottom <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        (config->observerEdgeBandLeft <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
        (config->observerEdgeBandRight <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
        (config->observerEdgeBandTop <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        (config->observerEdgeBandBottom <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
        borderSideValid(config->c1BorderEnterLeft, config->c1BorderExitLeft, 93u) &&
        borderSideValid(config->c1BorderEnterRight, config->c1BorderExitRight, 93u) &&
        borderSideValid(config->c1BorderEnterTop, config->c1BorderExitTop, 59u) &&
        borderSideValid(config->c1BorderEnterBottom, config->c1BorderExitBottom, 59u) &&
        borderSideValid(config->observerBorderEnterLeft, config->observerBorderExitLeft, 93u) &&
        borderSideValid(config->observerBorderEnterRight, config->observerBorderExitRight, 93u) &&
        borderSideValid(config->observerBorderEnterTop, config->observerBorderExitTop, 59u) &&
        borderSideValid(config->observerBorderEnterBottom, config->observerBorderExitBottom, 59u));
    }

uint8_t observerLinkSharedConfigsValid(const observerLinkSharedConfig *config)
{
    uint8_t i;
    if ((config == NULL) || (config->revision == 0u) ||
        (config->hash != observerLinkSharedConfigHash(config)) ||
        (config->detectMode > 1u) ||
        (config->maxBeacons < 1u) || (config->maxBeacons > 32u) ||
        (sharedConfigBorderValid(config) == 0u) ||
        (config->beaconSizeMin < 1u) ||
        (config->beaconSizeMax <= config->beaconSizeMin) ||
        (config->beaconSizeMax > 60 000u) ||
        (config->beaconMaxEndpoints > 200u) ||
        (config->beaconHuAreaMin < 1u) ||
        (config->beaconHuAreaMin > 100u) ||
        (config->trackConfirmHits < 1u) || (config->trackConfirmHits > 60u) ||
        (config->trackDropMisses < 1u) || (config->trackDropMisses > 60u) ||
        (config->trackCoastMisses < 1u) || (config->trackCoastMisses > 60u))
    {
        return 0u;
    }
    for (i = 0u; i < 7u; i++)
    {
        if (finiteInRange(config->beaconHuTemplate[i],
            -1 000 000.0f, 1 000 000.0f) == 0u)
        {
            return 0u;
        }
    }
}

return (uint8_t) (
    finiteInRange(config->beaconEccMax, 0.0f, 20.0f) &&
    finiteInRange(config->beaconFillMin, 0.0f, 1.0f) &&
    finiteInRange(config->beaconHuMax, 0.0f, 200.0f) &&
    finiteInRange(config->trackMaxAssocDist, 1.0f, 100.0f) &&
    finiteInRange(config->trackSizeRatioMax, 1.1f, 10.0f) &&
    finiteInRange(config->trackPositionAlpha, 0.01f, 1.0f) &&
    finiteInRange(config->trackSizeAlpha, 0.01f, 1.0f) &&
    finiteInRange(config->trackAreaVelAlpha, 0.01f, 1.0f) &&
    finiteInRange(config->trackAssocScoreMax, 0.1f, 10.0f) &&
    finiteInRange(config->trackAssocAreaWeight, 0.0f, 3.0f) &&
    finiteInRange(config->trackAssocAreaVelWeight, 0.0f, 3.0f));
}

```

core4_cyt4bb7/project/code/comm/observerLinkProtocol.c

SHA-256

:

e019e39df722dc745661f3a59859e5cbeb121fd354d13928214448bcab16b271

代码 (c)

```
#include "observerLinkProtocol.h"

#include <string.h>

#include "crc32Ieee.h"

static void putU16(uint8_t *dst, uint16_t value)
{
    dst[0] = (uint8_t) value;
    dst[1] = (uint8_t) (value >> 8);
}

static void putU32(uint8_t *dst, uint32_t value)
{
    dst[0] = (uint8_t) value;
    dst[1] = (uint8_t) (value >> 8);
    dst[2] = (uint8_t) (value >> 16);
    dst[3] = (uint8_t) (value >> 24);
}

static uint16_t getU16(const uint8_t *src)
{
    return (uint16_t) ((uint16_t) src[0] | ((uint16_t) src[1] << 8));
}

static uint32_t getU32(const uint8_t *src)
{
    return (uint32_t) src[0] |
        ((uint32_t) src[1] << 8) |
        ((uint32_t) src[2] << 16) |
        ((uint32_t) src[3] << 24);
}

static void putFloat(uint8_t *dst, float value)
{
    uint32_t bits;
    memcpy(&bits, &value, sizeof(bits));
    putU32(dst, bits);
}

static float getFloat(const uint8_t *src)
{
    uint32_t bits = getU32(src);
    float value;
    memcpy(&value, &bits, sizeof(value));
    return value;
}

static uint8_t finiteInRange(float value, float minimum, float maximum)
{
    if ((value != value) || (value > 1.0e30f) || (value < -1.0e30f))
    {
        return 0u;
    }
    return (uint8_t) ((value >= minimum) && (value <= maximum));
}

uint8_t observerLinkQuantizeSigned(float value, float scale,
    float minimum, float maximum, int16_t *quantized)
{
    float scaled;

    if ((quantized == NULL) ||
        (finiteInRange(value, minimum, maximum) == 0u) ||
        (finiteInRange(scale, 0.000001f, 1.0e30f) == 0u))
    {
        return 0u;
    }
    scaled = value * scale;
    if (finiteInRange(scaled, -32768.0f, 32767.0f) == 0u)
    {
        return 0u;
    }
    scaled += (scaled >= 0.0f) ? 0.5f : -0.5f;
    *quantized = (int16_t) scaled;
    return 1u;
}
```

```

}

static uint8_t updateExpected(observerLinkParser *parser)
{
    uint16_t payloadLength;

    parser->expected = 0u;
    if (parser->used < 8u)
    {
        return 1u;
    }
    payloadLength = getU16(&parser->bytes[6]);
    if (payloadLength > OBSERVER_LINK_MAX_PAYLOAD_BYTES)
    {
        return 0u;
    }
    parser->expected = (uint16_t) (OBSERVER_LINK_HEADER_BYTES +
                                payloadLength +
                                OBSERVER_LINK_CRC_BYTES);
    return 1u;
}

static void recoverParser(observerLinkParser *parser)
{
    uint16_t i;
    uint8_t found;

    for (;;)
    {
        found = 0u;
        for (i = 1u; (uint16_t) (i + 1u) < parser->used; i++)
        {
            if ((parser->bytes[i] == OBSERVER_LINK_SOF0) &&
                (parser->bytes[i + 1u] == OBSERVER_LINK_SOF1))
            {
                memmove(parser->bytes, &parser->bytes[i], parser->used - i);
                parser->used = (uint16_t) (parser->used - i);
                if (updateExpected(parser) != 0u)
                {
                    return;
                }
                parser->lengthErrors++;
                found = 1u;
                break;
            }
        }
        if (found != 0u)
        {
            continue;
        }
        if ((parser->used != 0u) &&
            (parser->bytes[parser->used - 1u] == OBSERVER_LINK_SOF0))
        {
            parser->bytes[0] = OBSERVER_LINK_SOF0;
            parser->used = 1u;
        }
        else
        {
            parser->used = 0u;
        }
        parser->expected = 0u;
        return;
    }
}

void observerLinkParserInit(observerLinkParser *parser)
{
    if (parser != NULL)
    {
        memset(parser, 0, sizeof(*parser));
    }
}

uint16_t observerLinkEncodeFrame(const observerLinkFrame *frame, uint8_t *output, uint16_t capacity)
{
    uint16_t total;
    uint32_t crc;

    if ((frame == NULL) || (output == NULL) ||

```

```

    (frame->payloadLength > OBSERVER_LINK_MAX_PAYLOAD_BYTES))
{
    return 0u;
}
total = (uint16_t) (OBSERVER_LINK_HEADER_BYTES + frame->payloadLength +
    OBSERVER_LINK_CRC_BYTES);
if (capacity < total)
{
    return 0u;
}

output[0] = OBSERVER_LINK_SOF0;
output[1] = OBSERVER_LINK_SOF1;
output[2] = frame->major;
output[3] = frame->minor;
output[4] = frame->type;
output[5] = frame->flags;
putU16(&output[6], frame->payloadLength);
putU32(&output[8], frame->bootSession);
putU32(&output[12], frame->packetSeq);
putU32(&output[16], frame->tickMs);
if (frame->payloadLength != 0u)
{
    memcpy(&output[OBSERVER_LINK_HEADER_BYTES],
        frame->payload,
        frame->payloadLength);
}
crc = crc32leeeCalc(&output[2],
    (uint32_t) (OBSERVER_LINK_HEADER_BYTES - 2u + frame->payloadLength));
putU32(&output[OBSERVER_LINK_HEADER_BYTES + frame->payloadLength], crc);
return total;
}

uint8_t observerLinkParserFeed(observerLinkParser *parser, uint8_t byte, observerLinkFrame *frame)
{
    uint16_t payloadLength;
    uint32_t expectedCrc;
    uint32_t actualCrc;

    if ((parser == NULL) || (frame == NULL))
    {
        return 0u;
    }

    if (parser->used == 0u)
    {
        if (byte == OBSERVER_LINK_SOF0)
        {
            {
                parser->bytes[0] = byte;
                parser->used = 1u;
            }
            return 0u;
        }
    }
    if (parser->used == 1u)
    {
        if (byte == OBSERVER_LINK_SOF1)
        {
            {
                parser->bytes[1] = byte;
                parser->used = 2u;
            }
        }
        else if (byte != OBSERVER_LINK_SOF0)
        {
            {
                parser->used = 0u;
            }
        }
        return 0u;
    }

    if (parser->used >= OBSERVER_LINK_MAX_FRAME_BYTES)
    {
        {
            parser->lengthErrors++;
            parser->used = 0u;
            parser->expected = 0u;
            return 0u;
        }
    }
    parser->bytes[parser->used++] = byte;

    if ((parser->used >= 8u) && (parser->expected == 0u))
    {
        if (updateExpected(parser) == 0u)
    
```

```

    {
        parser->lengthErrors++;
        recoverParser(parser);
        return 0u;
    }
}
if ((parser->expected == 0u) || (parser->used < parser->expected))
{
    return 0u;
}

validateFrame:
payloadLength = getU16(&parser->bytes[6]);
expectedCrc = getU32(&parser->bytes[OBSERVER_LINK_HEADER_BYTES + payloadLength]);
actualCrc = crc32IeeeCalc(&parser->bytes[2],
    (uint32_t) (OBSERVER_LINK_HEADER_BYTES - 2u + payloadLength));
if (expectedCrc != actualCrc)
{
    parser->crcErrors++;
    recoverParser(parser);
    if ((parser->expected != 0u) && (parser->used >= parser->expected))
    {
        goto validateFrame;
    }
    return 0u;
}

frame->major = parser->bytes[2];
frame->minor = parser->bytes[3];
frame->type = parser->bytes[4];
frame->flags = parser->bytes[5];
frame->payloadLength = payloadLength;
frame->bootSession = getU32(&parser->bytes[8]);
frame->packetSeq = getU32(&parser->bytes[12]);
frame->tickMs = getU32(&parser->bytes[16]);
if (payloadLength != 0u)
{
    memcpy(frame->payload,
        &parser->bytes[OBSERVER_LINK_HEADER_BYTES],
        payloadLength);
}
if (frame->major != OBSERVER_LINK_PROTOCOL_MAJOR)
{
    parser->versionErrors++;
}
parser->validFrames++;
if (parser->used > parser->expected)
{
    uint16_t remaining = (uint16_t) (parser->used - parser->expected);
    memmove(parser->bytes, &parser->bytes[parser->expected], remaining);
    parser->used = remaining;
    if (updateExpected(parser) == 0u)
    {
        parser->lengthErrors++;
        recoverParser(parser);
    }
}
else
{
    parser->used = 0u;
    parser->expected = 0u;
}
return 1u;
}

uint8_t observerLinkEncodeHello(uint8_t role, uint32_t capabilities, uint8_t *payload, uint16_t *length)
{
    if ((payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    payload[0] = role;
    payload[1] = OBSERVER_LINK_PROTOCOL_MAJOR;
    payload[2] = OBSERVER_LINK_PROTOCOL_MINOR;
    payload[3] = 0u;
    putU32(&payload[4], capabilities);
    *length = 8u;
    return 1u;
}

```

```

uint8_t observerLinkDecodeHello(const uint8_t *payload, uint16_t length, uint8_t *role, uint32_t *capabilities)
{
    if ((payload == NULL) || (role == NULL) || (capabilities == NULL) ||
        (length != 8u))
    {
        return 0u;
    }
    if ((payload[1] != OBSERVER_LINK_PROTOCOL_MAJOR) ||
        (payload[2] != OBSERVER_LINK_PROTOCOL_MINOR))
    {
        return 0u;
    }
    *role = payload[0];
    *capabilities = getU32(&payload[4]);
    return 1u;
}

uint8_t observerLinkEncodeSharedConfig(const observerLinkSharedConfig *config, uint8_t *payload, uint16_t *length)
{
    uint8_t i;
    if ((config == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    memset(payload, 0, OBSERVER_LINK_SHARED_CONFIG_BYTES);
    putU32(&payload[0], config->revision);
    putU32(&payload[4], config->hash);
    payload[8] = config->detectMode;
    payload[9] = config->maxBeacons;
    putU16(&payload[10], config->edgePredictionHoldMs);
    putU16(&payload[12], config->beaconSizeMin);
    putU16(&payload[14], config->beaconSizeMax);
    putFloat(&payload[16], config->beaconEccMax);
    payload[20] = config->beaconBrightnessMin;
    payload[21] = config->sourceId;
    putU16(&payload[22], config->beaconMaxEndpoints);
    putFloat(&payload[24], config->beaconFillMin);
    putFloat(&payload[28], config->beaconHuMax);
    putU16(&payload[32], config->beaconHuAreaMin);
    payload[34] = config->profileVersion;
    putFloat(&payload[36], config->trackMaxAssocDist);
    putFloat(&payload[40], config->trackSizeRatioMax);
    putU16(&payload[44], config->trackConfirmHits);
    putU16(&payload[46], config->trackDropMisses);
    putU16(&payload[48], config->trackCoastMisses);
    putFloat(&payload[52], config->trackPositionAlpha);
    putFloat(&payload[56], config->trackSizeAlpha);
    putFloat(&payload[60], config->trackAreaVelAlpha);
    putFloat(&payload[64], config->trackAssocScoreMax);
    putFloat(&payload[68], config->trackAssocAreaWeight);
    putFloat(&payload[72], config->trackAssocAreaVelWeight);
    for (i = 0u; i < 7u; i++)
    {
        putFloat(&payload[76u + (uint16_t) i * 4u], config->beaconHuTemplate[i]);
    }
    payload[104] = config->c1BorderEnterLeft;
    payload[105] = config->c1BorderEnterRight;
    payload[106] = config->c1BorderEnterTop;
    payload[107] = config->c1BorderEnterBottom;
    payload[108] = config->c1BorderExitLeft;
    payload[109] = config->c1BorderExitRight;
    payload[110] = config->c1BorderExitTop;
    payload[111] = config->c1BorderExitBottom;
    payload[112] = config->observerBorderEnterLeft;
    payload[113] = config->observerBorderEnterRight;
    payload[114] = config->observerBorderEnterTop;
    payload[115] = config->observerBorderEnterBottom;
    payload[116] = config->observerBorderExitLeft;
    payload[117] = config->observerBorderExitRight;
    payload[118] = config->observerBorderExitTop;
    payload[119] = config->observerBorderExitBottom;
    payload[120] = config->c1EdgeBandLeft;
    payload[121] = config->c1EdgeBandRight;
    payload[122] = config->c1EdgeBandTop;
    payload[123] = config->c1EdgeBandBottom;
    payload[124] = config->observerEdgeBandLeft;
    payload[125] = config->observerEdgeBandRight;
    payload[126] = config->observerEdgeBandTop;
}

```

```

payload[127] = config->observerEdgeBandBottom;
*length = OBSERVER_LINK_SHARED_CONFIG_BYTES;
return 1u;
}

uint8_t observerLinkDecodeSharedConfig(const uint8_t *payload, uint16_t length, observerLinkSharedConfig *config)
{
    uint8_t i;
    if ((payload == NULL) || (config == NULL) ||
        (length != OBSERVER_LINK_SHARED_CONFIG_BYTES))
    {
        return 0u;
    }
    memset(config, 0, sizeof(*config));
    config->revision = getU32(&payload[0]);
    config->hash = getU32(&payload[4]);
    config->detectMode = payload[8];
    config->maxBeacons = payload[9];
    config->edgePredictionHoldMs = getU16(&payload[10]);
    config->beaconSizeMin = getU16(&payload[12]);
    config->beaconSizeMax = getU16(&payload[14]);
    config->beaconEccMax = getFloat(&payload[16]);
    config->beaconBrightnessMin = payload[20];
    config->sourceId = payload[21];
    config->beaconMaxEndpoints = getU16(&payload[22]);
    config->beaconFillMin = getFloat(&payload[24]);
    config->beaconHuMax = getFloat(&payload[28]);
    config->beaconHuAreaMin = getU16(&payload[32]);
    config->profileVersion = payload[34];
    config->trackMaxAssocDist = getFloat(&payload[36]);
    config->trackSizeRatioMax = getFloat(&payload[40]);
    config->trackConfirmHits = getU16(&payload[44]);
    config->trackDropMisses = getU16(&payload[46]);
    config->trackCoastMisses = getU16(&payload[48]);
    config->trackPositionAlpha = getFloat(&payload[52]);
    config->trackSizeAlpha = getFloat(&payload[56]);
    config->trackAreaVelAlpha = getFloat(&payload[60]);
    config->trackAssocScoreMax = getFloat(&payload[64]);
    config->trackAssocAreaWeight = getFloat(&payload[68]);
    config->trackAssocAreaVelWeight = getFloat(&payload[72]);
    for (i = 0u; i < 7u; i++)
    {
        config->beaconHuTemplate[i] = getFloat(&payload[76u + (uint16_t) i * 4u]);
    }
    config->c1BorderEnterLeft = payload[104];
    config->c1BorderEnterRight = payload[105];
    config->c1BorderEnterTop = payload[106];
    config->c1BorderEnterBottom = payload[107];
    config->c1BorderExitLeft = payload[108];
    config->c1BorderExitRight = payload[109];
    config->c1BorderExitTop = payload[110];
    config->c1BorderExitBottom = payload[111];
    config->observerBorderEnterLeft = payload[112];
    config->observerBorderEnterRight = payload[113];
    config->observerBorderEnterTop = payload[114];
    config->observerBorderEnterBottom = payload[115];
    config->observerBorderExitLeft = payload[116];
    config->observerBorderExitRight = payload[117];
    config->observerBorderExitTop = payload[118];
    config->observerBorderExitBottom = payload[119];
    config->c1EdgeBandLeft = payload[120];
    config->c1EdgeBandRight = payload[121];
    config->c1EdgeBandTop = payload[122];
    config->c1EdgeBandBottom = payload[123];
    config->observerEdgeBandLeft = payload[124];
    config->observerEdgeBandRight = payload[125];
    config->observerEdgeBandTop = payload[126];
    config->observerEdgeBandBottom = payload[127];
    return 1u;
}

static uint8_t borderSideValid(uint8_t enter, uint8_t exit, uint8_t maxValue)
{
    if ((enter == 0u) && (exit == 0u)) return 1u;
    return (uint8_t) ((enter >= 1u) && (enter <= maxValue) && (exit < enter));
}

static uint8_t sharedConfigBorderValid(const observerLinkSharedConfig *config)
{

```

```

if (config == NULL) return 0u;
return (uint8_t) (
    (config->edgePredictionHoldMs <= OBSERVER_LINK_EDGE_PREDICTION_HOLD_MS_MAX) &&
    ((config->sourceId == OBSERVER_LINK_SOURCE_CORE3) ||
    (config->sourceId == OBSERVER_LINK_SOURCE_CORE4)) &&
    (config->profileVersion == 1u) &&
    (config->c1EdgeBandLeft <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
    (config->c1EdgeBandRight <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
    (config->c1EdgeBandTop <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
    (config->c1EdgeBandBottom <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
    (config->observerEdgeBandLeft <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
    (config->observerEdgeBandRight <= OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX) &&
    (config->observerEdgeBandTop <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
    (config->observerEdgeBandBottom <= OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX) &&
    borderSideValid(config->c1BorderEnterLeft, config->c1BorderExitLeft, 93u) &&
    borderSideValid(config->c1BorderEnterRight, config->c1BorderExitRight, 93u) &&
    borderSideValid(config->c1BorderEnterTop, config->c1BorderExitTop, 59u) &&
    borderSideValid(config->c1BorderEnterBottom, config->c1BorderExitBottom, 59u) &&
    borderSideValid(config->observerBorderEnterLeft, config->observerBorderExitLeft, 93u) &&
    borderSideValid(config->observerBorderEnterRight, config->observerBorderExitRight, 93u) &&
    borderSideValid(config->observerBorderEnterTop, config->observerBorderExitTop, 59u) &&
    borderSideValid(config->observerBorderEnterBottom, config->observerBorderExitBottom, 59u));
}

uint32_t observerLinkSharedConfigHash(const observerLinkSharedConfig *config)
{
    uint8_t payload[OBSERVER_LINK_SHARED_CONFIG_BYTES];
    uint16_t length;

    if ((config == NULL) ||
        (observerLinkEncodeSharedConfig(config, payload, &length) == 0u))
    {
        return 0u;
    }
    return crc32IeeeCalc(&payload[8], (uint32_t) (length - 8u));
}

uint8_t observerLinkSharedConfigsValid(const observerLinkSharedConfig *config)
{
    uint8_t i;
    if ((config == NULL) || (config->revision == 0u) ||
        (config->hash != observerLinkSharedConfigHash(config)) ||
        (config->detectMode > 1u) ||
        (config->maxBeacons < 1u) || (config->maxBeacons > 32u) ||
        (sharedConfigBorderValid(config) == 0u) ||
        (config->beaconSizeMin < 1u) ||
        (config->beaconSizeMax <= config->beaconSizeMin) ||
        (config->beaconSizeMax > 60000u) ||
        (config->beaconMaxEndpoints > 200u) ||
        (config->beaconHuAreaMin < 1u) ||
        (config->beaconHuAreaMin > 100u) ||
        (config->trackConfirmHits < 1u) || (config->trackConfirmHits > 60u) ||
        (config->trackDropMisses < 1u) || (config->trackDropMisses > 60u) ||
        (config->trackCoastMisses < 1u) || (config->trackCoastMisses > 60u))
    {
        return 0u;
    }
    for (i = 0u; i < 7u; i++)
    {
        if (finiteInRange(config->beaconHuTemplate[i], -1 000 000.0f, 1 000 000.0f) == 0u)
        {
            return 0u;
        }
    }
}

return (uint8_t) (
    finiteInRange(config->beaconEccMax, 0.0f, 20.0f) &&
    finiteInRange(config->beaconFillMin, 0.0f, 1.0f) &&
    finiteInRange(config->beaconHuMax, 0.0f, 200.0f) &&
    finiteInRange(config->trackMaxAssocDist, 1.0f, 100.0f) &&
    finiteInRange(config->trackSizeRatioMax, 1.1f, 10.0f) &&
    finiteInRange(config->trackPositionAlpha, 0.01f, 1.0f) &&
    finiteInRange(config->trackSizeAlpha, 0.01f, 1.0f) &&
    finiteInRange(config->trackAreaVelAlpha, 0.01f, 1.0f) &&
    finiteInRange(config->trackAssocScoreMax, 0.1f, 10.0f) &&
    finiteInRange(config->trackAssocAreaWeight, 0.0f, 3.0f) &&
    finiteInRange(config->trackAssocAreaVelWeight, 0.0f, 3.0f));
}

uint8_t observerLinkEncodeConfigAck(const observerLinkConfigAck *ack, uint8_t *payload, uint16_t *length)

```

```

{
    if ((ack == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], ack->ownerSession);
    putU32(&payload[4], ack->revision);
    putU32(&payload[8], ack->hash);
    payload[12] = ack->desiredMode;
    payload[13] = ack->appliedMode;
    payload[14] = ack->status;
    payload[15] = ack->reason;
    *length = OBSERVER_LINK_CONFIG_ACK_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeConfigAck(const uint8_t *payload, uint16_t length, observerLinkConfigAck *ack)
{
    if ((payload == NULL) || (ack == NULL) ||
        (length != OBSERVER_LINK_CONFIG_ACK_BYTES))
    {
        return 0u;
    }
    ack->ownerSession = getU32(&payload[0]);
    ack->revision = getU32(&payload[4]);
    ack->hash = getU32(&payload[8]);
    ack->desiredMode = payload[12];
    ack->appliedMode = payload[13];
    ack->status = payload[14];
    ack->reason = payload[15];
    return 1u;
}

uint8_t observerLinkEncodeDiagnostics(const observerLinkDiagnostics *diagnostics, uint8_t *payload, uint16_t *length)
{
    if ((diagnostics == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], diagnostics->validFrames);
    putU32(&payload[4], diagnostics->crcErrors);
    putU32(&payload[8], diagnostics->lengthErrors);
    putU32(&payload[12], diagnostics->versionErrors);
    putU32(&payload[16], diagnostics->rxOverflowErrors);
    putU32(&payload[20], diagnostics->rxFrameErrors);
    putU32(&payload[24], diagnostics->rxParityErrors);
    putU32(&payload[28], diagnostics->txDeferred);
    *length = OBSERVER_LINK_DIAGNOSTICS_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeDiagnostics(const uint8_t *payload, uint16_t length, observerLinkDiagnostics *diagnostics)
{
    if ((payload == NULL) || (diagnostics == NULL) ||
        (length != OBSERVER_LINK_DIAGNOSTICS_BYTES))
    {
        return 0u;
    }
    diagnostics->validFrames = getU32(&payload[0]);
    diagnostics->crcErrors = getU32(&payload[4]);
    diagnostics->lengthErrors = getU32(&payload[8]);
    diagnostics->versionErrors = getU32(&payload[12]);
    diagnostics->rxOverflowErrors = getU32(&payload[16]);
    diagnostics->rxFrameErrors = getU32(&payload[20]);
    diagnostics->rxParityErrors = getU32(&payload[24]);
    diagnostics->txDeferred = getU32(&payload[28]);
    return 1u;
}

uint8_t observerLinkEncodeKvmControl(const observerLinkKvmControl *control, uint8_t *payload, uint16_t *length)
{
    if ((control == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], control->targetSession);
    putU32(&payload[4], control->routeSession);
    putU32(&payload[8], control->routeEpoch);

```



```

        putU32(&payload[12], control->leaseMs);
        payload[16] = control->target;
        *length = OBSERVER_LINK_KVM_CONTROL_BYTES;
        return 1u;
    }

uint8_t observerLinkDecodeKvmControl(const uint8_t *payload, uint16_t length, observerLinkKvmControl *control)
{
    if ((payload == NULL) || (control == NULL) || (length != OBSERVER_LINK_KVM_CONTROL_BYTES))
    {
        return 0u;
    }
    memset(control, 0, sizeof(*control));
    control->targetSession = getU32(&payload[0]);
    control->routeSession = getU32(&payload[4]);
    control->routeEpoch = getU32(&payload[8]);
    control->leaseMs = getU32(&payload[12]);
    control->target = payload[16];
    return 1u;
}

uint8_t observerLinkEncodeKvmEvent(const observerLinkKvmEvent *event, uint8_t *payload, uint16_t *length)
{
    if ((event == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], event->targetSession);
    putU32(&payload[4], event->routeSession);
    putU32(&payload[8], event->routeEpoch);
    putU32(&payload[12], event->eventSeq);
    payload[16] = event->target;
    payload[17] = event->keyId;
    payload[18] = event->keyState;
    payload[19] = event->flags;
    *length = OBSERVER_LINK_KVM_EVENT_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeKvmEvent(const uint8_t *payload, uint16_t length, observerLinkKvmEvent *event)
{
    if ((payload == NULL) || (event == NULL) || (length != OBSERVER_LINK_KVM_EVENT_BYTES))
    {
        return 0u;
    }
    memset(event, 0, sizeof(*event));
    event->targetSession = getU32(&payload[0]);
    event->routeSession = getU32(&payload[4]);
    event->routeEpoch = getU32(&payload[8]);
    event->eventSeq = getU32(&payload[12]);
    event->target = payload[16];
    event->keyId = payload[17];
    event->keyState = payload[18];
    event->flags = payload[19];
    return 1u;
}

uint8_t observerLinkEncodeKvmAck(const observerLinkKvmAck *ack, uint8_t *payload, uint16_t *length)
{
    if ((ack == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], ack->targetSession);
    putU32(&payload[4], ack->routeSession);
    putU32(&payload[8], ack->routeEpoch);
    putU32(&payload[12], ack->eventSeq);
    putU32(&payload[16], ack->expectedSeq);
    payload[20] = ack->target;
    payload[21] = ack->result;
    payload[22] = ack->menuState;
    payload[23] = ack->flags;
    *length = OBSERVER_LINK_KVM_ACK_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeKvmAck(const uint8_t *payload, uint16_t length, observerLinkKvmAck *ack)
{
    if ((payload == NULL) || (ack == NULL) || (length != OBSERVER_LINK_KVM_ACK_BYTES))

```

```

    {
        return 0u;
    }
    memset(ack, 0, sizeof(*ack));
    ack->targetSession = getU32(&payload[0]);
    ack->routeSession = getU32(&payload[4]);
    ack->routeEpoch = getU32(&payload[8]);
    ack->eventSeq = getU32(&payload[12]);
    ack->expectedSeq = getU32(&payload[16]);
    ack->target = payload[20];
    ack->result = payload[21];
    ack->menuState = payload[22];
    ack->flags = payload[23];
    return 1u;
}

uint8_t observerLinkEncodeKvmMenuStatus(const observerLinkKvmMenuStatus *status, uint8_t *payload, uint16_t *length)
{
    if ((status == NULL) || (payload == NULL) || (length == NULL))
    {
        return 0u;
    }
    putU32(&payload[0], status->targetSession);
    putU32(&payload[4], status->routeSession);
    putU32(&payload[8], status->routeEpoch);
    putU32(&payload[12], status->lastEventSeq);
    putU32(&payload[16], status->screenRevision);
    payload[20] = status->target;
    payload[21] = status->menuState;
    payload[22] = status->flags;
    payload[23] = status->lastAckResult;
    *length = OBSERVER_LINK_KVM_MENU_STATUS_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeKvmMenuStatus(const uint8_t *payload, uint16_t length, observerLinkKvmMenuStatus *status)
{
    if ((payload == NULL) || (status == NULL) || (length != OBSERVER_LINK_KVM_MENU_STATUS_BYTES))
    {
        return 0u;
    }
    memset(status, 0, sizeof(*status));
    status->targetSession = getU32(&payload[0]);
    status->routeSession = getU32(&payload[4]);
    status->routeEpoch = getU32(&payload[8]);
    status->lastEventSeq = getU32(&payload[12]);
    status->screenRevision = getU32(&payload[16]);
    status->target = payload[20];
    status->menuState = payload[21];
    status->flags = payload[22];
    status->lastAckResult = payload[23];
    return 1u;
}

static uint8_t controlStatusIsValid(const observerLinkControlStatus *status)
{
    uint8_t statusValid;
    uint8_t decisionValid;
    uint8_t barrier;
    uint8_t dispatch;
    uint8_t core0LinkUp;
    uint8_t core2LinkUp;
    uint8_t requiredVision;

    if ((status == NULL) || (status->detectMode > 1u) ||
        (status->protocolCompatible > 1u) || (status->algorithmsReady > 1u) ||
        (status->configCompatible > 1u) ||
        (status->cameraControlState > OBSERVER_LINK_CAMERA_CONTROL_RUN) ||
        ((status->activeSource != OBSERVER_LINK_SOURCE_NONE) &&
         (status->activeSource != OBSERVER_LINK_SOURCE_CORE1) &&
         (status->activeSource != OBSERVER_LINK_SOURCE_CORE3) &&
         (status->activeSource != OBSERVER_LINK_SOURCE_CORE4)) ||
        (status->controlClass > OBSERVER_LINK_CONTROL_CLASS_SEARCH) ||
        ((status->navState != 0u) && (status->navState != 2u) &&
         (status->navState != 3u) && (status->navState != 4u)) ||
        (status->systemPhase > OBSERVER_LINK_SYSTEM_PHASE_RUNNING) ||
        ((status->systemFlags & 0x80u) != 0u) ||
        (status->primaryReason > OBSERVER_LINK_REASON_LANDING) ||
        ((status->desiredTarget != OBSERVER_LINK_KVM_TARGET_CORE0) &&

```

```

(status->desiredTarget != OBSERVER_LINK_KVM_TARGET_CORE1) &&
(status->desiredTarget != OBSERVER_LINK_KVM_TARGET_CORE3) &&
(status->desiredTarget != OBSERVER_LINK_KVM_TARGET_CORE4) &&
(status->desiredTarget != OBSERVER_LINK_KVM_TARGET_NONE)) ||
((status->ownerTarget != OBSERVER_LINK_KVM_TARGET_CORE0) &&
(status->ownerTarget != OBSERVER_LINK_KVM_TARGET_CORE1) &&
(status->ownerTarget != OBSERVER_LINK_KVM_TARGET_CORE3) &&
(status->ownerTarget != OBSERVER_LINK_KVM_TARGET_CORE4) &&
(status->ownerTarget != OBSERVER_LINK_KVM_TARGET_NONE)) ||
(status->targetX >= 188u) ||
(status->targetY >= 120u) || (status->targetSize > 22 560u))
{
    return 0u;
}

statusValid = (uint8_t) ((status->flags &
    OBSERVER_LINK_CONTROL_FLAG_STATUS_VALID) != 0u);
decisionValid = (uint8_t) ((status->flags &
    OBSERVER_LINK_CONTROL_FLAG_DECISION_VALID) != 0u);
barrier = (uint8_t) ((status->flags &
    OBSERVER_LINK_CONTROL_FLAG_BARRIER_REQUIRED) != 0u);
dispatch = (uint8_t) ((status->flags &
    OBSERVER_LINK_CONTROL_FLAG_DISPATCH_ENABLED) != 0u);
core0LinkUp = (uint8_t) ((status->flags &
    OBSERVER_LINK_CONTROL_FLAG_CORE0_LINK_UP) != 0u);
core2LinkUp = (uint8_t) ((status->flags &
    OBSERVER_LINK_CONTROL_FLAG_CORE2_LINK_UP) != 0u);
requiredVision = (uint8_t) ((status->flags &
    OBSERVER_LINK_CONTROL_FLAG_REQUIRED_VISION) != 0u);

if (((status->systemFlags & OBSERVER_LINK_SYSTEM_FLAG_STANDALONE) != 0u) !=
    (status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_STANDALONE)) ||
    ((status->systemFlags & OBSERVER_LINK_SYSTEM_FLAG_REJOIN) != 0u) !=
    (status->systemPhase == OBSERVER_LINK_SYSTEM_PHASE_REJOIN)) ||
    ((status->systemFlags & OBSERVER_LINK_SYSTEM_FLAG_LANDING) != 0u) &&
    ((status->systemFlags & OBSERVER_LINK_SYSTEM_FLAG_LAUNCH_ELIGIBLE) != 0u)))
{
    return 0u;
}

if (statusValid == 0u)
{
    return (uint8_t) ((status->cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_SAFE) &&
        (status->activeSource == OBSERVER_LINK_SOURCE_NONE) &&
        (status->controlClass == OBSERVER_LINK_CONTROL_CLASS_NONE) &&
        (status->navState == 0u) && (status->beaconCount == 0u) &&
        (status->targetX == 0u) && (status->targetY == 0u) &&
        (status->targetSize == 0u) && (status->errXQ4 == 0) &&
        (status->errYQ4 == 0) && (status->flags == 0u));
}
if ((status->cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_SAFE) ||
    (status->cameraControlState == OBSERVER_LINK_CAMERA_CONTROL_RUN) !=
    (requiredVision != 0u)) ||
    ((core2LinkUp != 0u) && (core0LinkUp == 0u)) ||
    ((decisionValid != 0u) &&
    (status->controlClass == OBSERVER_LINK_CONTROL_CLASS_NONE)) ||
    ((status->controlClass == OBSERVER_LINK_CONTROL_CLASS_NONE) &&
    (status->activeSource != OBSERVER_LINK_SOURCE_NONE)) ||
    ((status->controlClass == OBSERVER_LINK_CONTROL_CLASS_CORE1_TRACK) &&
    (status->activeSource != OBSERVER_LINK_SOURCE_CORE1)) ||
    ((status->controlClass == OBSERVER_LINK_CONTROL_CLASS_OBSERVER_TRACK) &&
    (status->activeSource != OBSERVER_LINK_SOURCE_CORE3) &&
    (status->activeSource != OBSERVER_LINK_SOURCE_CORE4)) ||
    ((status->controlClass == OBSERVER_LINK_CONTROL_CLASS_SEARCH) &&
    (status->activeSource != OBSERVER_LINK_SOURCE_NONE)) ||
    (dispatch != 0u) && ((decisionValid == 0u) || (barrier != 0u) ||
    (status->controlClass == OBSERVER_LINK_CONTROL_CLASS_NONE)))
{
    return 0u;
}
return 1u;
}

uint8_t observerLinkEncodeControlStatus(const observerLinkControlStatus *status,
    uint8_t *payload, uint16_t *length)
{
    if ((payload == NULL) || (length == NULL) ||
        (controlStatusIsValid(status) == 0u))
    {

```

```

        return 0u;
    }
    payload[0] = status->detectMode;
    payload[1] = status->protocolCompatible;
    payload[2] = status->algorithmsReady;
    payload[3] = status->configCompatible;
    payload[4] = status->cameraControlState;
    payload[5] = status->activeSource;
    payload[6] = status->controlClass;
    payload[7] = status->flags;
    payload[8] = status->navState;
    payload[9] = status->beaconCount;
    putU16(&payload[10], status->targetX);
    putU16(&payload[12], status->targetY);
    putU16(&payload[14], status->targetSize);
    putU16(&payload[16], (uint16_t) status->errXQ4);
    putU16(&payload[18], (uint16_t) status->errYQ4);
    payload[20] = status->systemPhase;
    payload[21] = status->systemFlags;
    payload[22] = status->primaryReason;
    payload[23] = status->desiredTarget;
    payload[24] = status->ownerTarget;
    *length = OBSERVER_LINK_CORE1_HEARTBEAT_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeControlStatus(const uint8_t *payload, uint16_t length,
    observerLinkControlStatus *status)
{
    if ((payload == NULL) || (status == NULL) ||
        (length != OBSERVER_LINK_CORE1_HEARTBEAT_BYTES))
    {
        return 0u;
    }
    memset(status, 0, sizeof(*status));
    status->detectMode = payload[0];
    status->protocolCompatible = payload[1];
    status->algorithmsReady = payload[2];
    status->configCompatible = payload[3];
    status->cameraControlState = payload[4];
    status->activeSource = payload[5];
    status->controlClass = payload[6];
    status->flags = payload[7];
    status->navState = payload[8];
    status->beaconCount = payload[9];
    status->targetX = getU16(&payload[10]);
    status->targetY = getU16(&payload[12]);
    status->targetSize = getU16(&payload[14]);
    status->errXQ4 = (int16_t) getU16(&payload[16]);
    status->errYQ4 = (int16_t) getU16(&payload[18]);
    status->systemPhase = payload[20];
    status->systemFlags = payload[21];
    status->primaryReason = payload[22];
    status->desiredTarget = payload[23];
    status->ownerTarget = payload[24];
    return controlStatusIsValid(status);
}

static uint8_t observerSourceIsValid(uint8_t sourceId)
{
    return (uint8_t) ((sourceId == OBSERVER_LINK_SOURCE_CORE3) ||
        (sourceId == OBSERVER_LINK_SOURCE_CORE4));
}

uint8_t observerLinkEncodeHealth(const observerLinkHealth *health,
    uint8_t *payload, uint16_t *length)
{
    if ((health == NULL) || (payload == NULL) || (length == NULL) ||
        (observerSourceIsValid(health->sourceId) == 0u) ||
        (health->cameraState > 2u) ||
        (health->configState > OBSERVER_LINK_CONFIG_REJECTED))
    {
        return 0u;
    }
    putU32(&payload[0], health->ownerSession);
    putU32(&payload[4], health->appFrameSeq);
    putU32(&payload[8], health->captureTickMs);
    putU32(&payload[12], health->bridgeSeq);
    putU16(&payload[16], health->frameAgeMs);

```

```

    putU16(&payload[18], health->bridgeAgeMs);
    payload[20] = health->sourceId;
    payload[21] = health->cameraState;
    payload[22] = health->configState;
    payload[23] = health->flags;
    *length = OBSERVER_LINK_HEALTH_BYTES;
    return 1u;
}

uint8_t observerLinkDecodeHealth(const uint8_t *payload, uint16_t length,
    observerLinkHealth *health)
{
    if ((payload == NULL) || (health == NULL) ||
        (length != OBSERVER_LINK_HEALTH_BYTES))
    {
        return 0u;
    }
    health->ownerSession = getU32(&payload[0]);
    health->appFrameSeq = getU32(&payload[4]);
    health->captureTickMs = getU32(&payload[8]);
    health->bridgeSeq = getU32(&payload[12]);
    health->frameAgeMs = getU16(&payload[16]);
    health->bridgeAgeMs = getU16(&payload[18]);
    health->sourceId = payload[20];
    health->cameraState = payload[21];
    health->configState = payload[22];
    health->flags = payload[23];
    return (uint8_t) ((observerSourceIsValid(health->sourceId) != 0u) &&
        (health->cameraState <= 2u) &&
        (health->configState <= OBSERVER_LINK_CONFIG_REJECTED));
}

static uint8_t viewOverlayIsValid(const observerLinkViewOverlay *overlay)
{
    uint16_t knownFlags = OBSERVER_LINK_OVERLAY_FLAG_VALID |
        OBSERVER_LINK_OVERLAY_FLAG_CENTER | OBSERVER_LINK_OVERLAY_FLAG_DEADZONE |
        OBSERVER_LINK_OVERLAY_FLAG_SELECTED | OBSERVER_LINK_OVERLAY_FLAG_POSE;
    uint8_t selectedPresent;
    uint8_t posePresent;

    if ((overlay == NULL) || (overlay->overlaySeq == 0u) ||
        (overlay->ownerSession == 0u) || (overlay->configRevision == 0u) ||
        (overlay->detectMode > 1u) || (overlay->flags & ~knownFlags) != 0u) ||
        ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_VALID) == 0u)) return 0u;
    selectedPresent = (uint8_t) ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_SELECTED) != 0u);
    posePresent = (uint8_t) ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_POSE) != 0u);
    if ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_CENTER) != 0u)
    {
        if ((overlay->activeCenterX >= 188u) || (overlay->activeCenterY >= 120u)) return 0u;
    }
    else if ((overlay->activeCenterX != 0u) || (overlay->activeCenterY != 0u)) return 0u;
    if ((overlay->flags & OBSERVER_LINK_OVERLAY_FLAG_DEADZONE) != 0u)
    {
        if ((overlay->deadzoneEnterRadius > 188u) ||
            (overlay->deadzoneExitRadius > 188u) ||
            (overlay->deadzoneExitRadius > overlay->deadzoneEnterRadius)) return 0u;
    }
    else if ((overlay->deadzoneEnterRadius != 0u) ||
        (overlay->deadzoneExitRadius != 0u)) return 0u;
    if (selectedPresent != 0u)
    {
        if ((overlay->sourceFrameSeq == 0u) ||
            (overlay->selectedTrackId == 0u) || (overlay->selectedX >= 188u) ||
            (overlay->selectedY >= 120u) || (overlay->selectedSize > 22 560u) ||
            ((overlay->selectedFlags & (uint8_t) ~(OBSERVER_LINK_OVERLAY_SELECTED_FRESH |
                OBSERVER_LINK_OVERLAY_SELECTED_COAST)) != 0u) ||
            (overlay->selectedFlags == 0u) ||
            (overlay->selectedFlags == (OBSERVER_LINK_OVERLAY_SELECTED_FRESH |
                OBSERVER_LINK_OVERLAY_SELECTED_COAST))) return 0u;
    }
    else if ((overlay->selectedTrackId != 0u) || (overlay->selectedX != 0u) ||
        (overlay->selectedY != 0u) || (overlay->selectedSize != 0u) ||
        (overlay->selectedFlags != 0u)) return 0u;
    if (posePresent != 0u)
    {
        uint8_t poseAgeFlags = (uint8_t) (overlay->poseFlags &
            (OBSERVER_LINK_OVERLAY_POSE_FRESH | OBSERVER_LINK_OVERLAY_POSE_HOLD));
        if ((overlay->poseX >= 188u) || (overlay->poseY >= 120u) ||
            ((overlay->poseFlags & (uint8_t) ~(OBSERVER_LINK_OVERLAY_POSE_FRESH |

```

```

        OBSERVER_LINK_OVERLAY_POSE_HOLD |
        OBSERVER_LINK_OVERLAY_POSE_HEADING_VALID)) != 0u) ||
        (poseAgeFlags == 0u) ||
        (poseAgeFlags == (OBSERVER_LINK_OVERLAY_POSE_FRESH |
        OBSERVER_LINK_OVERLAY_POSE_HOLD))) return 0u;
    if ((overlay->poseFlags & OBSERVER_LINK_OVERLAY_POSE_HEADING_VALID) != 0u)
    {
        if ((overlay->poseHeadingQ8 < -805) || (overlay->poseHeadingQ8 > 805)) return 0u;
    }
    else if (overlay->poseHeadingQ8 != 0) return 0u;
}
else if ((overlay->poseX != 0u) || (overlay->poseY != 0u) ||
        (overlay->poseHeadingQ8 != 0) || (overlay->poseFlags != 0u)) return 0u;
return 1u;
}

```

```

uint8_t observerLinkEncodeViewOverlay(const observerLinkViewOverlay *overlay,
uint8_t *payload, uint16_t *length)

```

```

{
    if ((payload == NULL) || (length == NULL) ||
        (viewOverlayIsValid(overlay) == 0u)) return 0u;
    putU32(&payload[0], overlay->overlaySeq);
    putU32(&payload[4], overlay->sourceFrameSeq);
    putU32(&payload[8], overlay->ownerSession);
    putU32(&payload[12], overlay->configRevision);
    putU32(&payload[16], overlay->configHash);
    putU16(&payload[20], overlay->flags);
    putU16(&payload[22], overlay->selectedTrackId);
    putU16(&payload[24], overlay->activeCenterX);
    putU16(&payload[26], overlay->activeCenterY);
    putU16(&payload[28], overlay->deadzoneEnterRadius);
    putU16(&payload[30], overlay->deadzoneExitRadius);
    putU16(&payload[32], overlay->selectedX);
    putU16(&payload[34], overlay->selectedY);
    putU16(&payload[36], overlay->selectedSize);
    putU16(&payload[38], overlay->poseX);
    putU16(&payload[40], overlay->poseY);
    putU16(&payload[42], (uint16_t) overlay->poseHeadingQ8);
    payload[44] = overlay->selectedFlags;
    payload[45] = overlay->poseFlags;
    payload[46] = overlay->detectMode;
    *length = OBSERVER_LINK_VIEW_OVERLAY_BYTES;
    return 1u;
}

```

```

uint8_t observerLinkDecodeViewOverlay(const uint8_t *payload, uint16_t length,
observerLinkViewOverlay *overlay)

```

```

{
    if ((payload == NULL) || (overlay == NULL) ||
        (length != OBSERVER_LINK_VIEW_OVERLAY_BYTES)) return 0u;
    memset(overlay, 0, sizeof(*overlay));
    overlay->overlaySeq = getU32(&payload[0]);
    overlay->sourceFrameSeq = getU32(&payload[4]);
    overlay->ownerSession = getU32(&payload[8]);
    overlay->configRevision = getU32(&payload[12]);
    overlay->configHash = getU32(&payload[16]);
    overlay->flags = getU16(&payload[20]);
    overlay->selectedTrackId = getU16(&payload[22]);
    overlay->activeCenterX = getU16(&payload[24]);
    overlay->activeCenterY = getU16(&payload[26]);
    overlay->deadzoneEnterRadius = getU16(&payload[28]);
    overlay->deadzoneExitRadius = getU16(&payload[30]);
    overlay->selectedX = getU16(&payload[32]);
    overlay->selectedY = getU16(&payload[34]);
    overlay->selectedSize = getU16(&payload[36]);
    overlay->poseX = getU16(&payload[38]);
    overlay->poseY = getU16(&payload[40]);
    overlay->poseHeadingQ8 = (int16_t) getU16(&payload[42]);
    overlay->selectedFlags = payload[44];
    overlay->poseFlags = payload[45];
    overlay->detectMode = payload[46];
    return viewOverlayIsValid(overlay);
}

```

```

uint8_t observerLinkPlanningTrackFragmentCount(uint8_t retainedCount)

```

```

{
    if (retainedCount > OBSERVER_LINK_MAX_TRACKS)
    {
        return 0u;
    }
}

```

```

    }
    return (retainedCount == 0u) ? 1u :
        (uint8_t) ((retainedCount + OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT - 1u) /
            OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT);
}

static uint8_t planningTrackIsValid(const observerLinkPlanningTrack *track)
{
    uint8_t fresh;
    uint8_t coast;
    uint8_t edgeGrace;
    uint8_t rejected;

    if ((track == NULL) || (track->centerX >= 188u) ||
        (track->centerY >= 120u) || (track->predX >= 188u) ||
        (track->predY >= 120u) || (track->size > 22 560u) ||
        (track->predSize > 22 560u) || (track->trackId == 0u) ||
        (track->misses > OBSERVER_LINK_PLANNING_TRACK_MISSES_MAX) ||
        ((track->flags & (uint8_t) ~(OBSERVER_LINK_TRACK_FLAG_FRESH |
            OBSERVER_LINK_TRACK_FLAG_COAST |
            OBSERVER_LINK_TRACK_FLAG_EDGE_GRACE |
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED)) != 0u))
    {
        return 0u;
    }
    fresh = (uint8_t) ((track->flags & OBSERVER_LINK_TRACK_FLAG_FRESH) != 0u);
    coast = (uint8_t) ((track->flags & OBSERVER_LINK_TRACK_FLAG_COAST) != 0u);
    edgeGrace = (uint8_t) ((track->flags & OBSERVER_LINK_TRACK_FLAG_EDGE_GRACE) != 0u);
    rejected = (uint8_t) ((track->flags &
        OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) != 0u);
    if (rejected != 0u)
    {
        return (uint8_t) ((track->flags ==
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) &&
            (track->centerX == 0u) && (track->centerY == 0u) &&
            (track->predX == 0u) && (track->predY == 0u) &&
            (track->size == 0u) && (track->predSize == 0u) &&
            (track->misses == 0u) && (track->velXQ7 == 0) &&
            (track->velYQ7 == 0) && (track->areaVelQ0 == 0));
    }
    if ((edgeGrace != 0u) && ((fresh != 0u) || (coast == 0u))) return 0u;
    return (uint8_t) (((fresh != 0u) && (track->misses == 0u) && (coast == 0u)) ||
        ((fresh == 0u) && (track->misses != 0u) && (coast != 0u)));
}

uint8_t observerLinkEncodePlanningTrackFragment(
    const observerLinkPlanningTrackSnapshot *snapshot,
    uint8_t fragmentIndex, uint8_t *payload, uint16_t *length)
{
    uint8_t fragmentCount;
    uint8_t first;
    uint8_t count;
    uint8_t i;
    uint8_t ordinaryRetained = 0u;
    uint16_t offset;

    if ((snapshot == NULL) || (payload == NULL) || (length == NULL) ||
        (snapshot->valid == 0u) || (observerSourceIsValid(snapshot->sourceId) == 0u) ||
        (snapshot->detectMode > 1u) ||
        (snapshot->retainedCount > OBSERVER_LINK_MAX_TRACKS) ||
        (snapshot->freshTotal > snapshot->confirmedTotal) ||
        ((uint16_t) snapshot->freshLeft + snapshot->freshRight != snapshot->freshTotal))
    {
        return 0u;
    }
    for (i = 0u; i < snapshot->retainedCount; i++)
    {
        uint8_t j;
        if (planningTrackIsValid(&snapshot->tracks[i]) == 0u) return 0u;
        if ((snapshot->tracks[i].flags &
            OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED) == 0u)
            ordinaryRetained++;
        for (j = 0u; j < i; j++)
        {
            if (snapshot->tracks[j].trackId == snapshot->tracks[i].trackId)
                return 0u;
        }
    }
    if (ordinaryRetained > snapshot->confirmedTotal) return 0u;

```

```

fragmentCount = observerLinkPlanningTrackFragmentCount(snapshot->retainedCount);
if (fragmentIndex >= fragmentCount)
{
    return 0u;
}
first = (uint8_t) (fragmentIndex * OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT);
count = (uint8_t) (snapshot->retainedCount - first);
if (count > OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT)
{
    count = OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT;
}
memset(payload, 0, OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_MAX_BYTES);
payload[0] = snapshot->sourceId;
payload[1] = snapshot->detectMode;
payload[2] = (uint8_t) (OBSERVER_LINK_TRACK_SNAPSHOT_VALID |
    ((snapshot->truncated != 0u) ? OBSERVER_LINK_TRACK_SNAPSHOT_TRUNCATED : 0u));
payload[3] = fragmentIndex;
payload[4] = fragmentCount;
payload[5] = first;
payload[6] = count;
payload[7] = snapshot->retainedCount;
putU32(&payload[8], snapshot->ownerSession);
putU32(&payload[12], snapshot->revision);
putU32(&payload[16], snapshot->hash);
putU32(&payload[20], snapshot->appFrameSeq);
putU32(&payload[24], snapshot->captureTickMs);
payload[28] = snapshot->confirmedTotal;
payload[29] = snapshot->freshTotal;
payload[30] = snapshot->freshLeft;
payload[31] = snapshot->freshRight;
putU32(&payload[32], snapshot->trackerFrameSeq);
offset = OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES;
for (i = 0u; i < count; i++)
{
    const observerLinkPlanningTrack *track = &snapshot->tracks[first + i];
    uint8_t j;
    if (planningTrackIsValid(track) == 0u)
    {
        return 0u;
    }
    for (j = 0u; j < (uint8_t) (first + i); j++)
    {
        if (snapshot->tracks[j].trackId == track->trackId)
        {
            return 0u;
        }
    }
    payload[offset + 0u] = track->centerX;
    payload[offset + 1u] = track->centerY;
    payload[offset + 2u] = track->predX;
    payload[offset + 3u] = track->predY;
    putU16(&payload[offset + 4u], track->size);
    putU16(&payload[offset + 6u], track->predSize);
    putU16(&payload[offset + 8u], track->trackId);
    putU16(&payload[offset + 10u], (uint16_t) (track->misses |
        ((uint16_t) track->flags << 9u)));
    putU16(&payload[offset + 12u], (uint16_t) track->velXQ7);
    putU16(&payload[offset + 14u], (uint16_t) track->velYQ7);
    putU16(&payload[offset + 16u], (uint16_t) track->areaVelQ0);
    offset = (uint16_t) (offset + OBSERVER_LINK_PLANNING_TRACK_BYTES);
}
*length = offset;
return 1u;
}

uint8_t observerLinkDecodePlanningTrackFragment(const uint8_t *payload,
    uint16_t length, observerLinkPlanningTrackFragment *fragment)
{
    uint8_t expectedFragments;
    uint8_t expectedFirst;
    uint8_t expectedCount;
    uint8_t i;
    uint16_t offset;

    if ((payload == NULL) || (fragment == NULL) ||
        (length < OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES) ||
        (observerSourceIsValid(payload[0]) == 0u) || (payload[1] > 1u) ||
        ((payload[2] & OBSERVER_LINK_TRACK_SNAPSHOT_VALID) == 0u) ||
        ((payload[2] & (uint8_t) ~(OBSERVER_LINK_TRACK_SNAPSHOT_VALID |

```



```

        OBSERVER_LINK_TRACK_SNAPSHOT_TRUNCATED)) != 0u) ||
(payload[7] > OBSERVER_LINK_MAX_TRACKS) ||
(payload[29] > payload[28]) ||
((uint16_t) payload[30] + payload[31] != payload[29]))
{
    return 0u;
}
expectedFragments = observerLinkPlanningTrackFragmentCount(payload[7]);
if ((payload[4] != expectedFragments) || (payload[3] >= expectedFragments))
{
    return 0u;
}
expectedFirst = (uint8_t) (payload[3] * OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT);
expectedCount = (uint8_t) (payload[7] - expectedFirst);
if (expectedCount > OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT)
{
    expectedCount = OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT;
}
if ((payload[5] != expectedFirst) || (payload[6] != expectedCount) ||
(length != (uint16_t) (OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES +
expectedCount * OBSERVER_LINK_PLANNING_TRACK_BYTES)))
{
    return 0u;
}
memset(fragment, 0, sizeof(*fragment));
fragment->sourceId = payload[0];
fragment->detectMode = payload[1];
fragment->truncated = (uint8_t) ((payload[2] & OBSERVER_LINK_TRACK_SNAPSHOT_TRUNCATED) != 0u);
fragment->fragmentIndex = payload[3];
fragment->fragmentCount = payload[4];
fragment->firstTrack = payload[5];
fragment->trackCount = payload[6];
fragment->retainedCount = payload[7];
fragment->ownerSession = getU32(&payload[8]);
fragment->revision = getU32(&payload[12]);
fragment->hash = getU32(&payload[16]);
fragment->appFrameSeq = getU32(&payload[20]);
fragment->captureTickMs = getU32(&payload[24]);
fragment->confirmedTotal = payload[28];
fragment->freshTotal = payload[29];
fragment->freshLeft = payload[30];
fragment->freshRight = payload[31];
fragment->trackerFrameSeq = getU32(&payload[32]);
offset = OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES;
for (i = 0u; i < expectedCount; i++)
{
    observerLinkPlanningTrack *track = &fragment->tracks[i];
    uint8_t j;
    track->centerX = payload[offset + 0u];
    track->centerY = payload[offset + 1u];
    track->predX = payload[offset + 2u];
    track->predY = payload[offset + 3u];
    track->size = getU16(&payload[offset + 4u]);
    track->predSize = getU16(&payload[offset + 6u]);
    track->trackId = getU16(&payload[offset + 8u]);
    {
        uint16_t state = getU16(&payload[offset + 10u]);
        if ((state & 0xe000u) != 0u)
        {
            return 0u;
        }
        track->misses = (uint16_t) (state & 0x01ffu);
        track->flags = (uint8_t) ((state >> 9u) & 0x0fu);
    }
    track->velXQ7 = (int16_t) getU16(&payload[offset + 12u]);
    track->velYQ7 = (int16_t) getU16(&payload[offset + 14u]);
    track->areaVelQ0 = (int16_t) getU16(&payload[offset + 16u]);
    if (planningTrackIsValid(track) == 0u)
    {
        return 0u;
    }
    for (j = 0u; j < i; j++)
    {
        if (fragment->tracks[j].trackId == track->trackId)
        {
            return 0u;
        }
    }
}
offset = (uint16_t) (offset + OBSERVER_LINK_PLANNING_TRACK_BYTES);

```

```

    }
    return 1u;
}

```

core4_cyt4bb7/project/code/comm/observerLinkProtocol.h

SHA-256

:

f96f51f0dcdaa2d9b76945ae34e036142bae8d76aa9e0262593d15c5ed3e8695

代码 (c)

```

#ifndef OBSERVER_LINK_PROTOCOL_H
#define OBSERVER_LINK_PROTOCOL_H

#include <stdint.h>
#include <string.h>
#include "paramSyncGlobal.h"

#define OBSERVER_LINK_SOF0 (0xA5u)
#define OBSERVER_LINK_SOF1 (0x5Au)
#define OBSERVER_LINK_PROTOCOL_MAJOR (3u)
#define OBSERVER_LINK_PROTOCOL_MINOR (2u)
#define OBSERVER_LINK_UART_BAUDRATE (4000 000u)
#define OBSERVER_LINK_TX_EPOCH_MS (10u)
#define OBSERVER_LINK_TX_FRAMES_PER_EPOCH (4u)
#define OBSERVER_LINK_TX_CONTROL_FRAMES_PER_EPOCH (2u)
#define OBSERVER_LINK_TX_DATA_CREDIT_CAP (8u)
#define OBSERVER_LINK_TX_CONTROL_CREDIT_CAP (4u)
#define OBSERVER_LINK_RX_BYTE_BUDGET (1536u)
#define OBSERVER_LINK_MAX_PAYLOAD_BYTES (128u)
#define OBSERVER_LINK_HEADER_BYTES (20u)
#define OBSERVER_LINK_CRC_BYTES (4u)
#define OBSERVER_LINK_MAX_FRAME_BYTES (OBSERVER_LINK_HEADER_BYTES + OBSERVER_LINK_MAX_PAYLOAD_BYTES +
OBSERVER_LINK_CRC_BYTES)
#define OBSERVER_LINK_SHARED_CONFIG_BYTES (128u)
#define OBSERVER_LINK_EDGE_PREDICTION_HOLD_MS_MAX (10 000u)
#define OBSERVER_LINK_EDGE_PREDICTION_BAND_X_MAX (93u)
#define OBSERVER_LINK_EDGE_PREDICTION_BAND_Y_MAX (59u)
#define OBSERVER_LINK_CONFIG_ACK_BYTES (16u)
#define OBSERVER_LINK_DIAGNOSTICS_BYTES (32u)
#define OBSERVER_LINK_KVM_CONTROL_BYTES (17u)
#define OBSERVER_LINK_KVM_EVENT_BYTES (20u)
#define OBSERVER_LINK_KVM_ACK_BYTES (24u)
#define OBSERVER_LINK_KVM_MENU_STATUS_BYTES (24u)
#define OBSERVER_LINK_VIEW_OVERLAY_BYTES (47u)
#define OBSERVER_LINK_MAX_TRACKS (32u)
#define OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES (36u)
#define OBSERVER_LINK_PLANNING_TRACK_BYTES (18u)
#define OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT (5u)
#define OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_MAX_BYTES (OBSERVER_LINK_PLANNING_TRACK_FRAGMENT_HEADER_BYTES +
OBSERVER_LINK_PLANNING_TRACK_BYTES * OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT)
#define OBSERVER_LINK_PLANNING_TRACK_MISSES_MAX (500u)

/* RFC1982 式 32 位序号比较：0 保留给未初始化态，跨 UINT32_MAX 时仍保持半窗单调。*/
static inline uint8_t observerLinkSequencelsNewer(
    uint32_t value, uint32_t reference)
{
    {
        uint32_t delta = value - reference;
        return (uint8_t) ((value != 0u) && (delta != 0u) &&
            (delta < 0x80 000 000u));
    }
}

#define OBSERVER_LINK_RX_SESSION_REJECT (0u)
#define OBSERVER_LINK_RX_SESSION_ACCEPT (1u)
#define OBSERVER_LINK_RX_SESSION_REBIND (2u)

/* 在线时身份不可被其它 session 抢占；只有接收端已进入离线栅栏后，合格 HELLO
* 才能用任意非零 session（含冷启动回退/同值）重建 packet 序号基线。*/
static inline uint8_t observerLinkRxSessionClassify(uint8_t online,
    uint32_t currentSession, uint32_t currentPacketSeq,
    uint8_t packetSeqPrimed, uint32_t incomingSession,
    uint32_t incomingPacketSeq, uint8_t qualifiedHello)
{
    {
        if ((incomingSession == 0u) || (incomingPacketSeq == 0u))
            return OBSERVER_LINK_RX_SESSION_REJECT;
    }
}

```

```

    if (online == 0u)
        return (qualifiedHello != 0u) ? OBSERVER_LINK_RX_SESSION_REBIND : OBSERVER_LINK_RX_SESSION_REJECT;
    if (incomingSession != currentSession)
        return OBSERVER_LINK_RX_SESSION_REJECT;
    if ((packetSeqPrimed != 0u) &&
        (observerLinkSequenceIsNewer(incomingPacketSeq,
            currentPacketSeq) == 0u))
        return OBSERVER_LINK_RX_SESSION_REJECT;
    return OBSERVER_LINK_RX_SESSION_ACCEPT;
}

#define OBSERVER_LINK_CAP_BINARY_CC      (1u << 0)
#define OBSERVER_LINK_CAP_LEGACY_CANNY   (1u << 1)
#define OBSERVER_LINK_CAP_CONFIG_STAGING (1u << 2)
#define OBSERVER_LINK_CAP_CONFIG_APPLY   (1u << 3)
#define OBSERVER_LINK_CAP_KVM            (1u << 4)
#define OBSERVER_LINK_CAP_MENU_STATUS    (1u << 5)
#define OBSERVER_LINK_CAP_PLANNING_TRACKS (1u << 6)
#define OBSERVER_LINK_CAP_CAMERA_CONTROL (1u << 7)
#define OBSERVER_LINK_CAP_CONTROL_STATUS (1u << 8)
#define OBSERVER_LINK_CAP_VIEW_OVERLAY   (1u << 9)
#define OBSERVER_LINK_CAP_PARAM_SYNC     (1u << 10)
#define OBSERVER_PARAM_HEADER_BYTES      (17u)
#define OBSERVER_PARAM_DATA_BYTES        (OBSERVER_LINK_MAX_PAYLOAD_BYTES - OBSERVER_PARAM_HEADER_BYTES)
#define OBSERVER_PARAM_TRANSPORT_BEGIN   (0x80u)
#define OBSERVER_PARAM_TRANSPORT_MORE    (0x40u)
#define OBSERVER_PARAM_STATUS_MASK        (0x3Fu)
#define OBSERVER_PARAM_OP_POLL            (0x7Fu)

typedef struct
{
    uint32_t transactionId;
    uint32_t payloadCrc32;
    uint16_t offset;
    uint16_t totalLength;
    uint8_t op;
    uint8_t slot;
    uint8_t target;
    uint8_t status;
    uint8_t length;
    uint8_t data[OBSERVER_PARAM_DATA_BYTES];
} observerParamFrame;

static inline uint8_t observerParamEncode(const observerParamFrame *frame,
    uint8_t *payload, uint16_t *length)
{
    if (frame == NULL || payload == NULL || length == NULL ||
        frame->length > OBSERVER_PARAM_DATA_BYTES) return 0u;
    payload[0] = frame->op;
    payload[1] = frame->slot;
    payload[2] = frame->target;
    payload[3] = frame->status;
    payload[4] = (uint8_t) frame->transactionId;
    payload[5] = (uint8_t) (frame->transactionId >> 8);
    payload[6] = (uint8_t) (frame->transactionId >> 16);
    payload[7] = (uint8_t) (frame->transactionId >> 24);
    payload[8] = (uint8_t) frame->offset;
    payload[9] = (uint8_t) (frame->offset >> 8);
    payload[10] = (uint8_t) frame->totalLength;
    payload[11] = (uint8_t) (frame->totalLength >> 8);
    payload[12] = (uint8_t) frame->payloadCrc32;
    payload[13] = (uint8_t) (frame->payloadCrc32 >> 8);
    payload[14] = (uint8_t) (frame->payloadCrc32 >> 16);
    payload[15] = (uint8_t) (frame->payloadCrc32 >> 24);
    payload[16] = frame->length;
    if (frame->length != 0u) memcpy(&payload[17], frame->data, frame->length);
    *length = (uint16_t) (OBSERVER_PARAM_HEADER_BYTES + frame->length);
    return 1u;
}

static inline uint8_t observerParamDecode(const uint8_t *payload,
    uint16_t length, observerParamFrame *frame)
{
    observerParamFrame candidate;
    if (payload == NULL || frame == NULL || length < OBSERVER_PARAM_HEADER_BYTES ||
        payload[16] > OBSERVER_PARAM_DATA_BYTES ||
        length != (uint16_t) (OBSERVER_PARAM_HEADER_BYTES + payload[16])) return 0u;
    memset(&candidate, 0, sizeof(candidate));

```

```

candidate.op = payload[0];
candidate.slot = payload[1];
candidate.target = payload[2];
candidate.status = payload[3];
candidate.transactionId = (uint32_t) payload[4] | ((uint32_t) payload[5] << 8) |
    ((uint32_t) payload[6] << 16) | ((uint32_t) payload[7] << 24);
candidate.offset = (uint16_t) (payload[8] | ((uint16_t) payload[9] << 8));
candidate.totalLength = (uint16_t) (payload[10] | ((uint16_t) payload[11] << 8));
candidate.payloadCrc32 = (uint32_t) payload[12] |
    ((uint32_t) payload[13] << 8) | ((uint32_t) payload[14] << 16) |
    ((uint32_t) payload[15] << 24);
candidate.length = payload[16];
if (candidate.length != 0u) memcpy(candidate.data, &payload[17], candidate.length);
*frame = candidate;
return 1u;
}

#define OBSERVER_PARAM_GLOBAL_REQUEST_BYTES (12u)
#define OBSERVER_PARAM_GLOBAL_STATE_BYTES (28u)
typedef char observerParamGlobalRequestSizeAssert[
    (sizeof(paramSyncGlobalRequest) == OBSERVER_PARAM_GLOBAL_REQUEST_BYTES) ? 1 : -1];
typedef char observerParamGlobalStateSizeAssert[
    (sizeof(paramSyncGlobalState) == OBSERVER_PARAM_GLOBAL_STATE_BYTES) ? 1 : -1];
static inline void observerParamGlobalPutU32(uint8_t *p, uint32_t value)
{
    p[0] = (uint8_t) value; p[1] = (uint8_t) (value >> 8);
    p[2] = (uint8_t) (value >> 16); p[3] = (uint8_t) (value >> 24);
}
static inline uint32_t observerParamGlobalGetU32(const uint8_t *p)
{
    return (uint32_t) p[0] | ((uint32_t) p[1] << 8) |
        ((uint32_t) p[2] << 16) | ((uint32_t) p[3] << 24);
}
static inline uint8_t observerParamGlobalRequestEncode(
    const paramSyncGlobalRequest *value, uint8_t *payload, uint16_t *length)
{
    if ((value == NULL) || (payload == NULL) || (length == NULL) ||
        (paramSyncGlobalRequestIsValid(value) == 0u)) return 0u;
    observerParamGlobalPutU32(&payload[0], value->producerBootGeneration);
    observerParamGlobalPutU32(&payload[4], value->sequence);
    payload[8] = value->origin; payload[9] = value->action;
    payload[10] = value->slot; payload[11] = 0u;
    *length = OBSERVER_PARAM_GLOBAL_REQUEST_BYTES;
    return 1u;
}
static inline uint8_t observerParamGlobalRequestDecode(const uint8_t *payload,
    uint16_t length, paramSyncGlobalRequest *value)
{
    paramSyncGlobalRequest candidate;
    if ((payload == NULL) || (value == NULL) ||
        (length != OBSERVER_PARAM_GLOBAL_REQUEST_BYTES)) return 0u;
    candidate.producerBootGeneration = observerParamGlobalGetU32(&payload[0]);
    candidate.sequence = observerParamGlobalGetU32(&payload[4]);
    candidate.origin = payload[8]; candidate.action = payload[9];
    candidate.slot = payload[10]; candidate.reserved = payload[11];
    if (paramSyncGlobalRequestIsValid(&candidate) == 0u) return 0u;
    *value = candidate;
    return 1u;
}
static inline uint8_t observerParamGlobalStateEncode(
    const paramSyncGlobalState *value, uint8_t *payload, uint16_t *length)
{
    if ((value == NULL) || (payload == NULL) || (length == NULL) ||
        paramSyncGlobalStateIsValid(value) == 0u) return 0u;
    observerParamGlobalPutU32(&payload[0], value->coordinatorBootGeneration);
    observerParamGlobalPutU32(&payload[4], value->sequence);
    observerParamGlobalPutU32(&payload[8], value->requestBootGeneration);
    observerParamGlobalPutU32(&payload[12], value->requestSequence);
    observerParamGlobalPutU32(&payload[16], value->transactionId);
    payload[20] = value->origin; payload[21] = value->action;
    payload[22] = value->slot; payload[23] = value->phase;
    payload[24] = value->target; payload[25] = value->error;
    payload[26] = value->readyMask; payload[27] = value->flags;
    *length = OBSERVER_PARAM_GLOBAL_STATE_BYTES;
    return paramSyncGlobalStateIsValid(value);
}
static inline uint8_t observerParamGlobalStateDecode(const uint8_t *payload,
    uint16_t length, paramSyncGlobalState *value)
{

```

```

paramSyncGlobalState candidate;
if ((payload == NULL) || (value == NULL) ||
    (length != OBSERVER_PARAM_GLOBAL_STATE_BYTES)) return 0u;
memset(&candidate, 0, sizeof(candidate));
candidate.coordinatorBootGeneration = observerParamGlobalGetU32(&payload[0]);
candidate.sequence = observerParamGlobalGetU32(&payload[4]);
candidate.requestBootGeneration = observerParamGlobalGetU32(&payload[8]);
candidate.requestSequence = observerParamGlobalGetU32(&payload[12]);
candidate.transactionId = observerParamGlobalGetU32(&payload[16]);
candidate.origin = payload[20]; candidate.action = payload[21];
candidate.slot = payload[22]; candidate.phase = payload[23];
candidate.target = payload[24]; candidate.error = payload[25];
candidate.readyMask = payload[26]; candidate.flags = payload[27];
if (paramSyncGlobalStateIsValid(&candidate) == 0u) return 0u;
*value = candidate;
return 1u;
}

#define OBSERVER_LINK_CAMERA_CONTROL_SAFE    (0u)
#define OBSERVER_LINK_CAMERA_CONTROL_RUN    (1u)
#define OBSERVER_LINK_CORE1_HEARTBEAT_BYTES (25u)
#define OBSERVER_LINK_HEALTH_BYTES          (24u)
#define OBSERVER_LINK_CAMERA_UNKNOWN        (0u)
#define OBSERVER_LINK_CAMERA_RUNNING        (1u)
#define OBSERVER_LINK_CAMERA_FAULT          (2u)
#define OBSERVER_LINK_HEALTH_FLAG_FRAME_FRESH (1u << 0)
#define OBSERVER_LINK_HEALTH_FLAG_BRIDGE_FRESH (1u << 1)

#define OBSERVER_LINK_SYSTEM_PHASE_INIT      (0u)
#define OBSERVER_LINK_SYSTEM_PHASE_INIT_FAULT (1u)
#define OBSERVER_LINK_SYSTEM_PHASE_LINK_WAIT (2u)
#define OBSERVER_LINK_SYSTEM_PHASE_HOME      (3u)
#define OBSERVER_LINK_SYSTEM_PHASE_REJOIN    (4u)
#define OBSERVER_LINK_SYSTEM_PHASE_STANDALONE (5u)
#define OBSERVER_LINK_SYSTEM_PHASE_RUNNING    (6u)

#define OBSERVER_LINK_SYSTEM_FLAG_LAUNCHED    (1u << 0)
#define OBSERVER_LINK_SYSTEM_FLAG_LANDING    (1u << 1)
#define OBSERVER_LINK_SYSTEM_FLAG_VEL_VALID   (1u << 2)
#define OBSERVER_LINK_SYSTEM_FLAG_TARGET_VALID (1u << 3)
#define OBSERVER_LINK_SYSTEM_FLAG_STANDALONE   (1u << 4)
#define OBSERVER_LINK_SYSTEM_FLAG_REJOIN      (1u << 5)
#define OBSERVER_LINK_SYSTEM_FLAG_LAUNCH_ELIGIBLE (1u << 6)

#define OBSERVER_LINK_REASON_NONE             (0u)
#define OBSERVER_LINK_REASON_LOCAL_INIT       (1u)
#define OBSERVER_LINK_REASON_CORE0_LINK       (2u)
#define OBSERVER_LINK_REASON_CORE1_LINK       (3u)
#define OBSERVER_LINK_REASON_CORE2_LINK       (4u)
#define OBSERVER_LINK_REASON_OBSERVER_LINK    (5u)
#define OBSERVER_LINK_REASON_PROTOCOL          (6u)
#define OBSERVER_LINK_REASON_CONFIG            (7u)
#define OBSERVER_LINK_REASON_CAMERA            (8u)
#define OBSERVER_LINK_REASON_TARGET            (9u)
#define OBSERVER_LINK_REASON_BARRIER          (10u)
#define OBSERVER_LINK_REASON_DISPATCH          (11u)
#define OBSERVER_LINK_REASON_SAFETY            (12u)
#define OBSERVER_LINK_REASON_LANDING           (13u)

#define OBSERVER_LINK_OVERLAY_FLAG_VALID       (1u << 0)
#define OBSERVER_LINK_OVERLAY_FLAG_CENTER      (1u << 1)
#define OBSERVER_LINK_OVERLAY_FLAG_DEADZONE    (1u << 2)
#define OBSERVER_LINK_OVERLAY_FLAG_SELECTED    (1u << 3)
#define OBSERVER_LINK_OVERLAY_FLAG_POSE        (1u << 4)
#define OBSERVER_LINK_OVERLAY_SELECTED_FRESH   (1u << 0)
#define OBSERVER_LINK_OVERLAY_SELECTED_COAST   (1u << 1)
#define OBSERVER_LINK_OVERLAY_POSE_FRESH       (1u << 0)
#define OBSERVER_LINK_OVERLAY_POSE_HOLD        (1u << 1)
#define OBSERVER_LINK_OVERLAY_POSE_HEADING_VALID (1u << 2)

#define OBSERVER_LINK_CONTROL_FLAG_STATUS_VALID (1u << 0)
#define OBSERVER_LINK_CONTROL_FLAG_DECISION_VALID (1u << 1)
#define OBSERVER_LINK_CONTROL_FLAG_BARRIER_REQUIRED (1u << 2)
#define OBSERVER_LINK_CONTROL_FLAG_DISPATCH_ENABLED (1u << 3)
#define OBSERVER_LINK_CONTROL_FLAG_REMOTE_ELIGIBLE (1u << 4)
#define OBSERVER_LINK_CONTROL_FLAG_CORE0_LINK_UP (1u << 5)
#define OBSERVER_LINK_CONTROL_FLAG_CORE2_LINK_UP (1u << 6)
#define OBSERVER_LINK_CONTROL_FLAG_REQUIRED_VISION (1u << 7)

```

```

#define OBSERVER_LINK_SOURCE_NONE (0u)
#define OBSERVER_LINK_SOURCE_CORE1 (1u)
#define OBSERVER_LINK_SOURCE_CORE3 (3u)
#define OBSERVER_LINK_SOURCE_CORE4 (4u)

#define OBSERVER_LINK_CONTROL_CLASS_NONE (0u)
#define OBSERVER_LINK_CONTROL_CLASS_CORE1_TRACK (1u)
#define OBSERVER_LINK_CONTROL_CLASS_OBSERVER_TRACK (2u)
#define OBSERVER_LINK_CONTROL_CLASS_SEARCH (3u)

#define OBSERVER_LINK_TRACK_FLAG_FRESH (1u << 0)
#define OBSERVER_LINK_TRACK_FLAG_COAST (1u << 1)
#define OBSERVER_LINK_TRACK_FLAG_EDGE_GRACE (1u << 2)
#define OBSERVER_LINK_TRACK_FLAG_BORDER_REJECTED (1u << 3)
#define OBSERVER_LINK_TRACK_SNAPSHOT_VALID (1u << 0)
#define OBSERVER_LINK_TRACK_SNAPSHOT_TRUNCATED (1u << 1)

#define OBSERVER_LINK_KVM_TARGET_CORE0 (0u)
#define OBSERVER_LINK_KVM_TARGET_CORE1 (1u)
#define OBSERVER_LINK_KVM_TARGET_CORE3 (3u)
#define OBSERVER_LINK_KVM_TARGET_CORE4 (4u)
#define OBSERVER_LINK_KVM_TARGET_NONE (0xFFu)

#define OBSERVER_LINK_KVM_ACK_FLAG_HOME_READY (1u << 0)
#define OBSERVER_LINK_KVM_ACK_FLAG_ACTION_OK (1u << 1)
#define OBSERVER_LINK_KVM_ACK_FLAG_ACTION_SAVE (1u << 2)
#define OBSERVER_LINK_KVM_ACK_FLAG_DUPLICATE (1u << 3)

#define OBSERVER_LINK_KVM_MENU_FLAG_HOME_READY (1u << 0)
#define OBSERVER_LINK_KVM_MENU_FLAG_FEEDBACK_ACTIVE (1u << 1)
#define OBSERVER_LINK_KVM_MENU_FLAG_LEASE_VALID (1u << 2)
#define OBSERVER_LINK_KVM_MENU_FLAG_ACTION_OK (1u << 3)
#define OBSERVER_LINK_KVM_MENU_FLAG_ACTION_SAVE (1u << 4)

typedef enum
{
    OBSERVER_LINK_ROLE_CORE1 = 1,
    OBSERVER_LINK_ROLE_CORE3 = 3,
    OBSERVER_LINK_ROLE_CORE4 = 4
} observerLinkRole;

typedef enum
{
    OBSERVER_LINK_MSG_HELLO = 1,
    OBSERVER_LINK_MSG_HEARTBEAT = 2,
    OBSERVER_LINK_MSG_DIAGNOSTICS = 3,
    OBSERVER_LINK_MSG_CONFIG_SNAPSHOT = 4,
    OBSERVER_LINK_MSG_CONFIG_ACK = 5,
    OBSERVER_LINK_MSG_KVM_CONTROL = 6,
    OBSERVER_LINK_MSG_KVM_EVENT = 7,
    OBSERVER_LINK_MSG_KVM_ACK = 8,
    OBSERVER_LINK_MSG_KVM_MENU_STATUS = 9,
    OBSERVER_LINK_MSG_PLANNING_TRACK_FRAGMENT = 10,
    OBSERVER_LINK_MSG_VIEW_OVERLAY = 11,
    OBSERVER_LINK_MSG_HEALTH = 12,
    OBSERVER_LINK_MSG_PARAM_REQUEST = 13,
    OBSERVER_LINK_MSG_PARAM_RESPONSE = 14,
    OBSERVER_LINK_MSG_PARAM_GLOBAL_STATE = 15,
    OBSERVER_LINK_MSG_PARAM_GLOBAL_REQUEST = 16
} observerLinkMessageType;

typedef enum
{
    OBSERVER_LINK_KVM_RESULT_NONE = 0,
    OBSERVER_LINK_KVM_RESULT_EXECUTED = 1,
    OBSERVER_LINK_KVM_RESULT_UNHANDLED = 2,
    OBSERVER_LINK_KVM_RESULT_NO_LEASE = 3,
    OBSERVER_LINK_KVM_RESULT_BAD_PAYLOAD = 4,
    OBSERVER_LINK_KVM_RESULT_WRONG_TARGET = 5,
    OBSERVER_LINK_KVM_RESULT_STALE_EPOCH = 6,
    OBSERVER_LINK_KVM_RESULT_GLOBAL_EVENT_REJECTED = 7,
    OBSERVER_LINK_KVM_RESULT_GAP = 8,
    OBSERVER_LINK_KVM_RESULT_OLD_SEQ = 9
} observerLinkKvmResult;

typedef enum
{
    OBSERVER_LINK_CONFIG_NONE = 0,
    OBSERVER_LINK_CONFIG_RECEIVED = 1,

```

```

OBSERVER_LINK_CONFIG_STAGED = 2,
OBSERVER_LINK_CONFIG_APPLIED = 3,
OBSERVER_LINK_CONFIG_REJECTED = 4
} observerLinkConfigStatus;

typedef enum
{
    OBSERVER_LINK_REJECT_NONE = 0,
    OBSERVER_LINK_REJECT_BAD_PAYLOAD = 1,
    OBSERVER_LINK_REJECT_BAD_HASH = 2,
    OBSERVER_LINK_REJECT_OLD_REVISION = 3,
    OBSERVER_LINK_REJECT_REVISION_CONFLICT = 4,
    OBSERVER_LINK_REJECT_UNSUPPORTED = 5
} observerLinkRejectReason;

typedef struct
{
    uint8_t major;
    uint8_t minor;
    uint8_t type;
    uint8_t flags;
    uint16_t payloadLength;
    uint32_t bootSession;
    uint32_t packetSeq;
    uint32_t tickMs;
    uint8_t payload[OBSERVER_LINK_MAX_PAYLOAD_BYTES];
} observerLinkFrame;

typedef struct
{
    uint8_t bytes[OBSERVER_LINK_MAX_FRAME_BYTES];
    uint16_t used;
    uint16_t expected;
    uint32_t validFrames;
    uint32_t crcErrors;
    uint32_t lengthErrors;
    uint32_t versionErrors;
} observerLinkParser;

typedef struct
{
    uint32_t revision;
    uint32_t hash;
    uint8_t detectMode;
    uint8_t maxBeacons;
    uint16_t edgePredictionHoldMs;
    uint8_t sourceId;
    uint8_t profileVersion;
    uint16_t beaconSizeMin;
    uint16_t beaconSizeMax;
    float beaconEccMax;
    uint8_t beaconBrightnessMin;
    uint16_t beaconMaxEndpoints;
    float beaconFillMin;
    float beaconHuMax;
    uint16_t beaconHuAreaMin;
    float trackMaxAssocDist;
    float trackSizeRatioMax;
    uint16_t trackConfirmHits;
    uint16_t trackDropMisses;
    uint16_t trackCoastMisses;
    float trackPositionAlpha;
    float trackSizeAlpha;
    float trackAreaVelAlpha;
    float trackAssocScoreMax;
    float trackAssocAreaWeight;
    float trackAssocAreaVelWeight;
    float beaconHuTemplate[7];
    uint8_t c1BorderEnterLeft;
    uint8_t c1BorderEnterRight;
    uint8_t c1BorderEnterTop;
    uint8_t c1BorderEnterBottom;
    uint8_t c1BorderExitLeft;
    uint8_t c1BorderExitRight;
    uint8_t c1BorderExitTop;
    uint8_t c1BorderExitBottom;
    uint8_t observerBorderEnterLeft;
    uint8_t observerBorderEnterRight;
    uint8_t observerBorderEnterTop;

```

```

uint8_t observerBorderEnterBottom;
uint8_t observerBorderExitLeft;
uint8_t observerBorderExitRight;
uint8_t observerBorderExitTop;
uint8_t observerBorderExitBottom;
uint8_t c1EdgeBandLeft;
uint8_t c1EdgeBandRight;
uint8_t c1EdgeBandTop;
uint8_t c1EdgeBandBottom;
uint8_t observerEdgeBandLeft;
uint8_t observerEdgeBandRight;
uint8_t observerEdgeBandTop;
uint8_t observerEdgeBandBottom;
} observerLinkSharedConfig;

```

```

typedef struct
{
    uint32_t ownerSession;
    uint32_t revision;
    uint32_t hash;
    uint8_t desiredMode;
    uint8_t appliedMode;
    uint8_t status;
    uint8_t reason;
} observerLinkConfigAck;

```

```

typedef struct
{
    uint32_t validFrames;
    uint32_t crcErrors;
    uint32_t lengthErrors;
    uint32_t versionErrors;
    uint32_t rxOverflowErrors;
    uint32_t rxFrameErrors;
    uint32_t rxParityErrors;
    uint32_t txDeferred;
} observerLinkDiagnostics;

```

```

typedef struct
{
    uint32_t targetSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t leaseMs;
    uint8_t target;
} observerLinkKvmControl;

```

```

typedef struct
{
    uint32_t targetSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t eventSeq;
    uint8_t target;
    uint8_t keyId;
    uint8_t keyState;
    uint8_t flags;
} observerLinkKvmEvent;

```

```

typedef struct
{
    uint32_t targetSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t eventSeq;
    uint32_t expectedSeq;
    uint8_t target;
    uint8_t result;
    uint8_t menuState;
    uint8_t flags;
} observerLinkKvmAck;

```

```

typedef struct
{
    uint32_t targetSession;
    uint32_t routeSession;
    uint32_t routeEpoch;
    uint32_t lastEventSeq;
    uint32_t screenRevision;
}

```



```

    uint8_t target;
    uint8_t menuState;
    uint8_t flags;
    uint8_t lastAckResult;
} observerLinkKvmMenuStatus;

```

```

typedef struct
{
    uint8_t detectMode;
    uint8_t protocolCompatible;
    uint8_t algorithmsReady;
    uint8_t configCompatible;
    uint8_t cameraControlState;
    uint8_t activeSource;
    uint8_t controlClass;
    uint8_t flags;
    uint8_t navState;
    uint8_t beaconCount;
    uint16_t targetX;
    uint16_t targetY;
    uint16_t targetSize;
    int16_t errXQ4;
    int16_t errYQ4;
    uint8_t systemPhase;
    uint8_t systemFlags;
    uint8_t primaryReason;
    uint8_t desiredTarget;
    uint8_t ownerTarget;
} observerLinkControlStatus;

```

```

typedef struct
{
    uint32_t ownerSession;
    uint32_t appFrameSeq;
    uint32_t captureTickMs;
    uint32_t bridgeSeq;
    uint16_t frameAgeMs;
    uint16_t bridgeAgeMs;
    uint8_t sourceId;
    uint8_t cameraState;
    uint8_t configState;
    uint8_t flags;
} observerLinkHealth;

```

/* ownerSession 是发送端 Core1 boot session; 接收端只接受与当前 peer session 一致的 DTO。 */

```

typedef struct
{
    uint32_t overlaySeq;
    uint32_t sourceFrameSeq;
    uint32_t ownerSession;
    uint32_t configRevision;
    uint32_t configHash;
    uint16_t flags;
    uint16_t selectedTrackId;
    uint16_t activeCenterX;
    uint16_t activeCenterY;
    uint16_t deadzoneEnterRadius;
    uint16_t deadzoneExitRadius;
    uint16_t selectedX;
    uint16_t selectedY;
    uint16_t selectedSize;
    uint16_t poseX;
    uint16_t poseY;
    int16_t poseHeadingQ8;
    uint8_t selectedFlags;
    uint8_t poseFlags;
    uint8_t detectMode;
} observerLinkViewOverlay;

```

```

typedef struct
{
    uint8_t centerX;
    uint8_t centerY;
    uint8_t predX;
    uint8_t predY;
    uint16_t size;
    uint16_t predSize;
    uint16_t trackId;
    uint16_t misses;
}

```

```

uint8_t flags;
int16_t velXQ7;
int16_t velYQ7;
int16_t areaVelQ0;
} observerLinkPlanningTrack;

```

```

typedef struct
{
    uint8_t valid;
    uint8_t sourceId;
    uint8_t detectMode;
    uint8_t truncated;
    uint32_t ownerSession;
    uint32_t revision;
    uint32_t hash;
    uint32_t appFrameSeq;
    uint32_t captureTickMs;
    uint32_t trackerFrameSeq;
    uint8_t confirmedTotal;
    uint8_t freshTotal;
    uint8_t freshLeft;
    uint8_t freshRight;
    uint8_t retainedCount;
    observerLinkPlanningTrack tracks[OBSERVER_LINK_MAX_TRACKS];
} observerLinkPlanningTrackSnapshot;

```

```

typedef struct
{
    uint8_t sourceId;
    uint8_t detectMode;
    uint8_t truncated;
    uint8_t fragmentIndex;
    uint8_t fragmentCount;
    uint8_t firstTrack;
    uint8_t trackCount;
    uint8_t retainedCount;
    uint32_t ownerSession;
    uint32_t revision;
    uint32_t hash;
    uint32_t appFrameSeq;
    uint32_t captureTickMs;
    uint32_t trackerFrameSeq;
    uint8_t confirmedTotal;
    uint8_t freshTotal;
    uint8_t freshLeft;
    uint8_t freshRight;
    observerLinkPlanningTrack tracks[OBSERVER_LINK_PLANNING_TRACKS_PER_FRAGMENT];
} observerLinkPlanningTrackFragment;

```

```

void observerLinkParserInit(observerLinkParser *parser);
uint8_t observerLinkParserFeed(observerLinkParser *parser, uint8_t byte, observerLinkFrame *frame);
uint16_t observerLinkEncodeFrame(const observerLinkFrame *frame, uint8_t *output, uint16_t capacity);
uint8_t observerLinkEncodeHello(uint8_t role, uint32_t capabilities, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeHello(const uint8_t *payload, uint16_t length, uint8_t *role, uint32_t *capabilities);
uint8_t observerLinkEncodeSharedConfig(const observerLinkSharedConfig *config, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeSharedConfig(const uint8_t *payload, uint16_t length, observerLinkSharedConfig *config);
uint32_t observerLinkSharedConfigHash(const observerLinkSharedConfig *config);
uint8_t observerLinkSharedConfigIsValid(const observerLinkSharedConfig *config);
uint8_t observerLinkEncodeConfigAck(const observerLinkConfigAck *ack, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeConfigAck(const uint8_t *payload, uint16_t length, observerLinkConfigAck *ack);
uint8_t observerLinkEncodeDiagnostics(const observerLinkDiagnostics *diagnostics, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeDiagnostics(const uint8_t *payload, uint16_t length, observerLinkDiagnostics *diagnostics);
uint8_t observerLinkEncodeKvmControl(const observerLinkKvmControl *control, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeKvmControl(const uint8_t *payload, uint16_t length, observerLinkKvmControl *control);
uint8_t observerLinkEncodeKvmEvent(const observerLinkKvmEvent *event, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeKvmEvent(const uint8_t *payload, uint16_t length, observerLinkKvmEvent *event);
uint8_t observerLinkEncodeKvmAck(const observerLinkKvmAck *ack, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeKvmAck(const uint8_t *payload, uint16_t length, observerLinkKvmAck *ack);
uint8_t observerLinkEncodeKvmMenuStatus(const observerLinkKvmMenuStatus *status, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeKvmMenuStatus(const uint8_t *payload, uint16_t length, observerLinkKvmMenuStatus *status);
uint8_t observerLinkEncodeControlStatus(const observerLinkControlStatus *status,
    uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeControlStatus(const uint8_t *payload, uint16_t length,
    observerLinkControlStatus *status);
uint8_t observerLinkEncodeHealth(const observerLinkHealth *health,
    uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeHealth(const uint8_t *payload, uint16_t length,
    observerLinkHealth *health);
uint8_t observerLinkEncodeViewOverlay(const observerLinkViewOverlay *overlay,

```

```

    uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodeViewOverlay(const uint8_t *payload, uint16_t length,
    observerLinkViewOverlay *overlay);
uint8_t observerLinkPlanningTrackFragmentCount(uint8_t retainedCount);
uint8_t observerLinkEncodePlanningTrackFragment(
    const observerLinkPlanningTrackSnapshot *snapshot,
    uint8_t fragmentIndex, uint8_t *payload, uint16_t *length);
uint8_t observerLinkDecodePlanningTrackFragment(const uint8_t *payload,
    uint16_t length, observerLinkPlanningTrackFragment *fragment);
uint8_t observerLinkQuantizeSigned(float value, float scale,
    float minimum, float maximum, int16_t *quantized);

#endif /* OBSERVER_LINK_PROTOCOL_H */

```

core4_cyt4bb7/project/code/comm/paramSyncGlobal.h

SHA-256

:

16bd76824b594d1c1737b56b4d97f5d90393ccf24573bf1b42af32c3cb5e8f67

代码 (c)

```

#ifndef PARAM_SYNC_GLOBAL_H
#define PARAM_SYNC_GLOBAL_H

#include <stdint.h>

#define PARAM_SYNC_GLOBAL_TARGET_MASK (0x1Fu)

typedef enum
{
    PARAM_SYNC_GLOBAL_ACTION_NONE = 0u,
    PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY = 1u,
    PARAM_SYNC_GLOBAL_ACTION_LOAD_RUNTIME = 2u,
    PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY = 3u
} paramSyncGlobalAction;

#define PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_VALID (0u)
#define PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_EMPTY (5u)
#define PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_INVALID (10u)

static inline uint32_t paramSyncGlobalPackSlotSummary(uint8_t validMask,
    uint8_t emptyMask, uint8_t invalidMask)
{
    return ((uint32_t) (validMask & PARAM_SYNC_GLOBAL_TARGET_MASK) <<
        PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_VALID) |
        ((uint32_t) (emptyMask & PARAM_SYNC_GLOBAL_TARGET_MASK) <<
        PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_EMPTY) |
        ((uint32_t) (invalidMask & PARAM_SYNC_GLOBAL_TARGET_MASK) <<
        PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_INVALID);
}

static inline uint8_t paramSyncGlobalSlotSummaryMask(uint32_t summary,
    uint8_t shift)
{
    return (uint8_t) ((summary >> shift) & PARAM_SYNC_GLOBAL_TARGET_MASK);
}

typedef enum
{
    PARAM_SYNC_SLOT_UI_UNKNOWN = 0u,
    PARAM_SYNC_SLOT_UI_CHECKING,
    PARAM_SYNC_SLOT_UI_FULL,
    PARAM_SYNC_SLOT_UI_EMPTY,
    PARAM_SYNC_SLOT_UI_PARTIAL,
    PARAM_SYNC_SLOT_UI_INVALID,
    PARAM_SYNC_SLOT_UI_UNAVAILABLE
} paramSyncSlotUiState;

static inline uint8_t paramSyncGlobalClassifySlotSummary(uint32_t summary)
{
    uint8_t validMask = paramSyncGlobalSlotSummaryMask(summary,
        PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_VALID);
    uint8_t emptyMask = paramSyncGlobalSlotSummaryMask(summary,
        PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_EMPTY);
    uint8_t invalidMask = paramSyncGlobalSlotSummaryMask(summary,

```

```

    PARAM_SYNC_GLOBAL_SLOT_MASK_SHIFT_INVALID);
if (invalidMask != 0u) return PARAM_SYNC_SLOT_UI_INVALID;
if (validMask == PARAM_SYNC_GLOBAL_TARGET_MASK && emptyMask == 0u)
    return PARAM_SYNC_SLOT_UI_FULL;
if (emptyMask == PARAM_SYNC_GLOBAL_TARGET_MASK && validMask == 0u)
    return PARAM_SYNC_SLOT_UI_EMPTY;
if ((uint8_t) (validMask | emptyMask) == PARAM_SYNC_GLOBAL_TARGET_MASK)
    return PARAM_SYNC_SLOT_UI_PARTIAL;
return PARAM_SYNC_SLOT_UI_UNAVAILABLE;
}

typedef enum
{
    PARAM_SYNC_GLOBAL_PHASE_UNINITIALIZED = 0u,
    PARAM_SYNC_GLOBAL_PHASE_READY = 1u,
    PARAM_SYNC_GLOBAL_PHASE_REQUESTED = 2u,
    PARAM_SYNC_GLOBAL_PHASE_LOCKING = 3u,
    PARAM_SYNC_GLOBAL_PHASE_PREPARING = 4u,
    PARAM_SYNC_GLOBAL_PHASE_COMMITTING = 5u,
    PARAM_SYNC_GLOBAL_PHASE_APPLYING = 6u,
    PARAM_SYNC_GLOBAL_PHASE_DECIDING = 7u,
    PARAM_SYNC_GLOBAL_PHASE_FINISHING = 8u,
    PARAM_SYNC_GLOBAL_PHASE_ABORTING = 9u,
    PARAM_SYNC_GLOBAL_PHASE_RECOVERING = 10u,
    PARAM_SYNC_GLOBAL_PHASE_SUCCEEDED = 11u,
    PARAM_SYNC_GLOBAL_PHASE_FAILED = 12u
} paramSyncGlobalPhase;

typedef enum
{
    PARAM_SYNC_GLOBAL_ERROR_NONE = 0u,
    PARAM_SYNC_GLOBAL_ERROR_BUSY = 1u,
    PARAM_SYNC_GLOBAL_ERROR_REJECTED = 2u,
    PARAM_SYNC_GLOBAL_ERROR_INCOMPLETE = 3u,
    PARAM_SYNC_GLOBAL_ERROR_ROUTE = 4u,
    PARAM_SYNC_GLOBAL_ERROR_STORAGE = 5u,
    PARAM_SYNC_GLOBAL_ERROR_RECOVERY = 6u,
    PARAM_SYNC_GLOBAL_ERROR_PEER_RESTART = 7u,
    PARAM_SYNC_GLOBAL_ERROR_INVALID = 8u
} paramSyncGlobalError;

#define PARAM_SYNC_GLOBAL_FLAG_READY (1u << 0)
#define PARAM_SYNC_GLOBAL_FLAG_BUSY (1u << 1)
#define PARAM_SYNC_GLOBAL_FLAG_RECOVERY (1u << 2)
#define PARAM_SYNC_GLOBAL_FLAG_TERMINAL (1u << 3)
#define PARAM_SYNC_GLOBAL_FLAG_OK (1u << 4)
#define PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY (1u << 5)

typedef struct
{
    uint32_t producerBootGeneration;
    uint32_t sequence;
    uint8_t origin;
    uint8_t action;
    uint8_t slot;
    uint8_t reserved;
} paramSyncGlobalRequest;

typedef struct
{
    uint32_t coordinatorBootGeneration;
    uint32_t sequence;
    uint32_t requestBootGeneration;
    uint32_t requestSequence;
    uint32_t transactionId;
    uint8_t origin;
    uint8_t action;
    uint8_t slot;
    uint8_t phase;
    uint8_t target;
    uint8_t error;
    uint8_t readyMask;
    uint8_t flags;
} paramSyncGlobalState;

static inline uint8_t paramSyncGlobalStateIsValid(
    const paramSyncGlobalState *state)
{
    const uint8_t allowedFlags = PARAM_SYNC_GLOBAL_FLAG_READY |

```

```

PARAM_SYNC_GLOBAL_FLAG_BUSY | PARAM_SYNC_GLOBAL_FLAG_RECOVERY |
PARAM_SYNC_GLOBAL_FLAG_TERMINAL | PARAM_SYNC_GLOBAL_FLAG_OK |
PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY;
uint8_t identityPresent;
if (state == 0 || state->coordinatorBootGeneration == 0u ||
    state->sequence == 0u || state->phase > PARAM_SYNC_GLOBAL_PHASE_FAILED ||
    (state->target != 0xFFu && state->target > 4u) ||
    state->error > PARAM_SYNC_GLOBAL_ERROR_INVALID ||
    (state->readyMask & (uint8_t) ~PARAM_SYNC_GLOBAL_TARGET_MASK) != 0u ||
    (state->flags & (uint8_t) ~allowedFlags) != 0u) return 0u;
identityPresent = (uint8_t) (state->requestBootGeneration != 0u);
if (identityPresent != (uint8_t) (state->requestSequence != 0u)) return 0u;
if (state->action == PARAM_SYNC_GLOBAL_ACTION_NONE)
{
    if (state->slot != 0u || state->origin != 0xFFu) return 0u;
}
else
{
    if (state->action > PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY ||
        (state->origin > 4u && state->origin != 0xFFu)) return 0u;
    if (state->slot < 1u || state->slot > 5u) return 0u;
}
if (state->phase == PARAM_SYNC_GLOBAL_PHASE_UNINITIALIZED)
{
    return (uint8_t) ((state->flags == 0u ||
        state->flags == PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY) &&
        state->action == PARAM_SYNC_GLOBAL_ACTION_NONE &&
        identityPresent == 0u && state->transactionId == 0u &&
        state->target == 0xFFu && state->error == PARAM_SYNC_GLOBAL_ERROR_NONE);
}
if (state->phase == PARAM_SYNC_GLOBAL_PHASE_READY)
{
    return (uint8_t) (state->flags ==
        (PARAM_SYNC_GLOBAL_FLAG_READY |
        PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY) &&
        state->readyMask == PARAM_SYNC_GLOBAL_TARGET_MASK &&
        state->action == PARAM_SYNC_GLOBAL_ACTION_NONE &&
        identityPresent == 0u && state->transactionId == 0u &&
        state->target == 0xFFu && state->error == PARAM_SYNC_GLOBAL_ERROR_NONE);
}
if (state->phase == PARAM_SYNC_GLOBAL_PHASE_RECOVERING)
{
    return (uint8_t) (state->flags == (PARAM_SYNC_GLOBAL_FLAG_BUSY |
        PARAM_SYNC_GLOBAL_FLAG_RECOVERY) &&
        state->action == PARAM_SYNC_GLOBAL_ACTION_NONE &&
        identityPresent == 0u && state->target == 0xFFu &&
        (state->error == PARAM_SYNC_GLOBAL_ERROR_NONE ||
        state->error == PARAM_SYNC_GLOBAL_ERROR_RECOVERY));
}
if (state->phase == PARAM_SYNC_GLOBAL_PHASE_SUCCEEDED)
{
    return (uint8_t) ((state->flags == (PARAM_SYNC_GLOBAL_FLAG_TERMINAL |
        PARAM_SYNC_GLOBAL_FLAG_OK) ||
        state->flags == (PARAM_SYNC_GLOBAL_FLAG_TERMINAL |
        PARAM_SYNC_GLOBAL_FLAG_OK |
        PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY)) &&
        state->action != PARAM_SYNC_GLOBAL_ACTION_NONE &&
        identityPresent != 0u && state->transactionId != 0u &&
        state->target == 0xFFu && state->error == PARAM_SYNC_GLOBAL_ERROR_NONE);
}
if (state->phase == PARAM_SYNC_GLOBAL_PHASE_FAILED)
{
    return (uint8_t) ((state->flags == PARAM_SYNC_GLOBAL_FLAG_TERMINAL ||
        state->flags == (PARAM_SYNC_GLOBAL_FLAG_TERMINAL |
        PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY)) &&
        state->action != PARAM_SYNC_GLOBAL_ACTION_NONE &&
        identityPresent != 0u &&
        state->error != PARAM_SYNC_GLOBAL_ERROR_NONE);
}
return (uint8_t) (state->phase >= PARAM_SYNC_GLOBAL_PHASE_REQUESTED &&
    state->phase <= PARAM_SYNC_GLOBAL_PHASE_ABORTING &&
    state->flags == PARAM_SYNC_GLOBAL_FLAG_BUSY &&
    state->action != PARAM_SYNC_GLOBAL_ACTION_NONE &&
    identityPresent != 0u && state->transactionId != 0u);
}

static inline uint8_t paramSyncGlobalRequestIsValid(
    const paramSyncGlobalRequest *request)
{

```

```

    if (request == 0 || request->producerBootGeneration == 0u ||
        request->sequence == 0u || request->origin > 4u ||
        request->reserved != 0u ||
        request->action < PARAM_SYNC_GLOBAL_ACTION_SAVE_LIBRARY ||
        request->action > PARAM_SYNC_GLOBAL_ACTION_INSPECT_LIBRARY)
        return 0u;
    return (uint8_t) (request->slot >= 1u && request->slot <= 5u);
}

static inline uint8_t paramSyncGlobalStateReady(
    const paramSyncGlobalState *state)
{
    return (uint8_t) (paramSyncGlobalStateIsValid(state) != 0u &&
        state->phase == PARAM_SYNC_GLOBAL_PHASE_READY &&
        state->action == PARAM_SYNC_GLOBAL_ACTION_NONE &&
        state->target == 0xFFu && state->error == PARAM_SYNC_GLOBAL_ERROR_NONE &&
        state->readyMask == PARAM_SYNC_GLOBAL_TARGET_MASK &&
        state->flags == (PARAM_SYNC_GLOBAL_FLAG_READY |
            PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY));
}

static inline uint8_t paramSyncGlobalStateRuntimeReady(
    const paramSyncGlobalState *state)
{
    return (uint8_t) (paramSyncGlobalStateIsValid(state) != 0u &&
        (state->flags & PARAM_SYNC_GLOBAL_FLAG_RUNTIME_READY) != 0u);
}

static inline uint8_t paramSyncGlobalStateMatchesRequest(
    const paramSyncGlobalState *state, const paramSyncGlobalRequest *request)
{
    return (uint8_t) (paramSyncGlobalStateIsValid(state) != 0u &&
        paramSyncGlobalRequestIsValid(request) != 0u &&
        state->requestBootGeneration == request->producerBootGeneration &&
        state->requestSequence == request->sequence &&
        state->origin == request->origin && state->action == request->action &&
        state->slot == request->slot);
}

static inline void paramSyncGlobalPutU32(uint8_t *p, uint32_t value)
{
    p[0] = (uint8_t) value; p[1] = (uint8_t) (value >> 8);
    p[2] = (uint8_t) (value >> 16); p[3] = (uint8_t) (value >> 24);
}

static inline uint32_t paramSyncGlobalGetU32(const uint8_t *p)
{
    return (uint32_t) p[0] | ((uint32_t) p[1] << 8) |
        ((uint32_t) p[2] << 16) | ((uint32_t) p[3] << 24);
}

static inline void paramSyncGlobalEncodeRequest(
    const paramSyncGlobalRequest *value, uint8_t out[12])
{
    paramSyncGlobalPutU32(&out[0], value->producerBootGeneration);
    paramSyncGlobalPutU32(&out[4], value->sequence);
    out[8] = value->origin; out[9] = value->action;
    out[10] = value->slot; out[11] = value->reserved;
}

static inline uint8_t paramSyncGlobalDecodeRequest(const uint8_t in[12],
    paramSyncGlobalRequest *value)
{
    value->producerBootGeneration = paramSyncGlobalGetU32(&in[0]);
    value->sequence = paramSyncGlobalGetU32(&in[4]);
    value->origin = in[8]; value->action = in[9];
    value->slot = in[10]; value->reserved = in[11];
    return paramSyncGlobalRequestIsValid(value);
}

static inline void paramSyncGlobalEncodeState(
    const paramSyncGlobalState *value, uint8_t out[28])
{
    paramSyncGlobalPutU32(&out[0], value->coordinatorBootGeneration);
    paramSyncGlobalPutU32(&out[4], value->sequence);
    paramSyncGlobalPutU32(&out[8], value->requestBootGeneration);
    paramSyncGlobalPutU32(&out[12], value->requestSequence);
    paramSyncGlobalPutU32(&out[16], value->transactionId);
    out[20] = value->origin; out[21] = value->action; out[22] = value->slot;
    out[23] = value->phase; out[24] = value->target; out[25] = value->error;
    out[26] = value->readyMask; out[27] = value->flags;
}

static inline uint8_t paramSyncGlobalDecodeState(const uint8_t in[28],

```

```

    paramSyncGlobalState *value)
{
    value->coordinatorBootGeneration = paramSyncGlobalGetU32(&in[0]);
    value->sequence = paramSyncGlobalGetU32(&in[4]);
    value->requestBootGeneration = paramSyncGlobalGetU32(&in[8]);
    value->requestSequence = paramSyncGlobalGetU32(&in[12]);
    value->transactionId = paramSyncGlobalGetU32(&in[16]);
    value->origin = in[20]; value->action = in[21]; value->slot = in[22];
    value->phase = in[23]; value->target = in[24]; value->error = in[25];
    value->readyMask = in[26]; value->flags = in[27];
    return paramSyncGlobalStateIsValid(value);
}

typedef char paramSyncGlobal_request_size_assert[
    (sizeof(paramSyncGlobalRequest) == 12u) ? 1 : -1];
typedef char paramSyncGlobal_state_size_assert[
    (sizeof(paramSyncGlobalState) == 28u) ? 1 : -1];

#endif

```

core4_cyt4bb7/project/code/comm/paramSyncMailbox.c

SHA-256

:

774a74f34cc5eaab467a692cd9faa2c10c8c111750dc3e38cd1b56c2694384ea

代码 (c)

```

#include "paramSyncMailbox.h"

#include "cy_device_headers.h"

#include <string.h>

#define PARAM_SYNC_REQUEST_MAGIC (0x50535251u) /* PSRQ */
#define PARAM_SYNC_RESPONSE_MAGIC (0x50535253u) /* PSRS */
#define PARAM_SYNC_MAILBOX_ABI (1u)

typedef struct
{
    uint32_t beginSequence;
    uint32_t magic;
    uint32_t abiVersion;
    uint32_t payloadBytes;
    uint32_t producerBootGeneration;
    uint32_t sequence;
    uint32_t transactionId;
    uint32_t offset;
    uint32_t payloadCrc32;
    uint16_t dataLength;
    uint8_t op;
    uint8_t slot;
    uint8_t target;
    uint8_t state;
    uint8_t reservedHeader[6];
    uint8_t data[PARAM_SYNC_MAILBOX_DATA_MAX];
    uint8_t reservedTail[12];
    uint32_t endSequence;
} paramSyncMailboxSharedFrame;

typedef char paramSyncMailbox_frame_size_assert[
    (sizeof(paramSyncMailboxSharedFrame) == PARAM_SYNC_MAILBOX_FRAME_BYTES) ? 1 : -1];

#pragma location = PARAM_SYNC_MAILBOX_REQUEST_ADDR
__no_init static volatile paramSyncMailboxSharedFrame paramSyncRequestRegion;
#pragma location = PARAM_SYNC_MAILBOX_RESPONSE_ADDR
__no_init static volatile paramSyncMailboxSharedFrame paramSyncResponseRegion;

static uint8_t frameInputValid(uint32_t bootGeneration, uint32_t sequence,
    uint16_t dataLength)
{
    return (uint8_t) ((bootGeneration != 0u) && (sequence != 0u) &&
        (dataLength <= PARAM_SYNC_MAILBOX_DATA_MAX));
}

static void cleanRegion(volatile paramSyncMailboxSharedFrame *region)
{

```

```

        SCB_CleanDCache_by_Addr((volatile void *) region, sizeof(*region));
        __DSB();
    }

static void invalidateRegion(volatile paramSyncMailboxSharedFrame *region)
{
    SCB_InvalidateDCache_by_Addr((volatile void *) region, sizeof(*region));
    __DSB();
}

static uint8_t publishFrame(volatile paramSyncMailboxSharedFrame *region,
                           const paramSyncMailboxSharedFrame *frame)
{
    paramSyncMailboxSharedFrame staged;

    staged = *frame;
    staged.endSequence = 0u;

    region->beginSequence = 0u;
    region->endSequence = 0u;
    cleanRegion(region);

    memcpy((void *) region, &staged, sizeof(staged));
    cleanRegion(region);

    region->endSequence = staged.sequence;
    SCB_CleanDCache_by_Addr((volatile void *)
        (((uintptr_t) &region->endSequence) & ~(uintptr_t) 31u), 32u);
    __DSB();
    return 1u;
}

static uint8_t sharedFrameValid(const paramSyncMailboxSharedFrame *frame,
                                uint32_t magic)
{
    return (uint8_t) ((frame->beginSequence != 0u) &&
        (frame->beginSequence == frame->sequence) &&
        (frame->endSequence == frame->sequence) &&
        (frame->magic == magic) &&
        (frame->abiVersion == PARAM_SYNC_MAILBOX_ABI) &&
        (frame->payloadBytes == frame->dataLength) &&
        frameInputValid(frame->producerBootGeneration, frame->sequence,
            frame->dataLength));
}

static uint8_t tryReadFrame(volatile paramSyncMailboxSharedFrame *region,
                            uint32_t magic, paramSyncMailboxSharedFrame *frame)
{
    paramSyncMailboxSharedFrame first;
    paramSyncMailboxSharedFrame second;
    uint8_t attempt;

    for (attempt = 0u; attempt < 3u; attempt++)
    {
        invalidateRegion(region);
        memcpy(&first, (const void *) region, sizeof(first));
        invalidateRegion(region);
        memcpy(&second, (const void *) region, sizeof(second));
        if (sharedFrameValid(&first, magic) &&
            sharedFrameValid(&second, magic) &&
            (memcmp(&first, &second, sizeof(first)) == 0))
        {
            *frame = second;
            return 1u;
        }
    }
    return 0u;
}

uint8_t paramSyncMailboxPublishRequest(const paramSyncMailboxRequest *request)
{
    paramSyncMailboxSharedFrame frame;
    if ((request == 0) || !frameInputValid(request->producerBootGeneration,
        request->sequence, request->dataLength)) return 0u;
    memset(&frame, 0, sizeof(frame));
    frame.beginSequence = request->sequence;
    frame.magic = PARAM_SYNC_REQUEST_MAGIC;
    frame.abiVersion = PARAM_SYNC_MAILBOX_ABI;
    frame.payloadBytes = request->dataLength;

```



```

    frame.producerBootGeneration = request->producerBootGeneration;
    frame.sequence = request->sequence;
    frame.transactionId = request->transactionId;
    frame.offset = request->offset;
    frame.payloadCrc32 = request->payloadCrc32;
    frame.dataLength = request->dataLength;
    frame.op = request->op;
    frame.slot = request->slot;
    frame.target = request->target;
    frame.state = request->flags;
    memcpy(frame.data, request->data, request->dataLength);
    return publishFrame(&paramSyncRequestRegion, &frame);
}

uint8_t paramSyncMailboxTryReadRequest(paramSyncMailboxRequest *request)
{
    paramSyncMailboxSharedFrame frame;
    paramSyncMailboxRequest candidate;
    if ((request == 0) || !tryReadFrame(&paramSyncRequestRegion,
        PARAM_SYNC_REQUEST_MAGIC, &frame)) return 0u;
    memset(&candidate, 0, sizeof(candidate));
    candidate.producerBootGeneration = frame.producerBootGeneration;
    candidate.sequence = frame.sequence;
    candidate.transactionId = frame.transactionId;
    candidate.offset = frame.offset;
    candidate.payloadCrc32 = frame.payloadCrc32;
    candidate.dataLength = frame.dataLength;
    candidate.op = frame.op;
    candidate.slot = frame.slot;
    candidate.target = frame.target;
    candidate.flags = frame.state;
    memcpy(candidate.data, frame.data, frame.dataLength);
    *request = candidate;
    return 1u;
}

uint8_t paramSyncMailboxPublishResponse(const paramSyncMailboxResponse *response)
{
    paramSyncMailboxSharedFrame frame;
    if ((response == 0) || !frameInputValid(response->producerBootGeneration,
        response->sequence, response->dataLength)) return 0u;
    memset(&frame, 0, sizeof(frame));
    frame.beginSequence = response->sequence;
    frame.magic = PARAM_SYNC_RESPONSE_MAGIC;
    frame.abiVersion = PARAM_SYNC_MAILBOX_ABI;
    frame.payloadBytes = response->dataLength;
    frame.producerBootGeneration = response->producerBootGeneration;
    frame.sequence = response->sequence;
    frame.transactionId = response->transactionId;
    frame.offset = response->offset;
    frame.payloadCrc32 = response->payloadCrc32;
    frame.dataLength = response->dataLength;
    frame.op = response->op;
    frame.slot = response->slot;
    frame.target = response->target;
    frame.state = response->status;
    memcpy(frame.data, response->data, response->dataLength);
    return publishFrame(&paramSyncResponseRegion, &frame);
}

uint8_t paramSyncMailboxTryReadResponse(paramSyncMailboxResponse *response)
{
    paramSyncMailboxSharedFrame frame;
    paramSyncMailboxResponse candidate;
    if ((response == 0) || !tryReadFrame(&paramSyncResponseRegion,
        PARAM_SYNC_RESPONSE_MAGIC, &frame)) return 0u;
    memset(&candidate, 0, sizeof(candidate));
    candidate.producerBootGeneration = frame.producerBootGeneration;
    candidate.sequence = frame.sequence;
    candidate.transactionId = frame.transactionId;
    candidate.offset = frame.offset;
    candidate.payloadCrc32 = frame.payloadCrc32;
    candidate.dataLength = frame.dataLength;
    candidate.op = frame.op;
    candidate.slot = frame.slot;
    candidate.target = frame.target;
    candidate.status = frame.state;
    memcpy(candidate.data, frame.data, frame.dataLength);
    *response = candidate;
}

```

```
    return 1u;
}
```

core4_cyt4bb7/project/code/comm/paramSyncOwnerService.c

SHA-256

:

031b72a9e15d63cc390fb2a2993a76db7fa90ac3d3d495068243d16638affd6f

代码 (c)

```
#include "paramSyncOwnerService.h"

#include <string.h>

#include "core4CoreDataTransfer.h"
#include "core4VisionParams.h"
#include "core4VisionParamsBlob.h"
#include "core4VisionParamsPersistence.h"
#include "crc32leee.h"
#include "paramSyncMailbox.h"

#define CORE4_PARAM_SYNC_TARGET    (4u)
#define CORE4_PARAM_SYNC_SCHEMA_HASH (0xE30FC075u)
#define CORE4_ACTIVATE_ROLLBACK_MAGIC (0xA4B20000u)
#define PARAM_STAGE_NONE          (0u)
#define PARAM_STAGE_LIBRARY        (1u)
#define PARAM_STAGE_RUNTIME        (2u)

static uint32_t g_lastBootGeneration;
static uint32_t g_lastRequestSequence;
static uint32_t g_responseSequence;
static uint32_t g_ownerBootGeneration;
static core4VisionParamsFlashBlob g_stagedBlob;
static uint32_t g_stagedTransactionId;
static uint32_t g_stagedExpectedCrc;
static uint16_t g_stagedBytes;
static uint16_t g_stagedReceived;
static uint8_t g_stagedSlot;
static uint8_t g_stagedValid;
static uint8_t g_flashPrepared;
static uint8_t g_flashCommitted;
static uint8_t g_stageKind;
static uint8_t g_runtimeApplied;
static uint8_t g_activationRecoveryChecked;
static uint8_t g_syncMutationLocked;
static uint8_t g_syncLockedSlot;
static uint32_t g_syncLockedTransaction;
static uint32_t g_lastFinishedTransaction;
static uint32_t g_lastAbortedTransaction;
#if defined(__ICCARM__)
__no_init uint32_t core4ParamSyncActivationRollbackRetained;
__no_init uint32_t core4ParamSyncActivationRollbackInverseRetained;
#else
static uint32_t core4ParamSyncActivationRollbackRetained;
static uint32_t core4ParamSyncActivationRollbackInverseRetained;
#endif

static uint8_t activationContext(uint8_t *oldSlotExternal,
                                uint8_t *newSlotExternal)
{
    uint32_t word = core4ParamSyncActivationRollbackRetained;
    if (core4ParamSyncActivationRollbackInverseRetained != ~word ||
        (word & 0xFFFF0000u) != CORE4_ACTIVATE_ROLLBACK_MAGIC)
        return 0u;
    *oldSlotExternal = (uint8_t) word;
    *newSlotExternal = (uint8_t) (word >> 8);
    return (uint8_t) (*oldSlotExternal >= 1u &&
                     *oldSlotExternal <= CORE4_VISION_PARAMS_SLOT_COUNT &&
                     *newSlotExternal >= 1u &&
                     *newSlotExternal <= CORE4_VISION_PARAMS_SLOT_COUNT);
}

static void setActivationContext(uint8_t oldSlotExternal,
                                uint8_t newSlotExternal)
{

```

```

uint32_t word = CORE4_ACTIVATE_ROLLBACK_MAGIC |
    (uint32_t) oldSlotExternal |
    ((uint32_t) newSlotExternal << 8);
core4ParamSyncActivationRollbackRetained = word;
core4ParamSyncActivationRollbackInverseRetained = ~word;
}

static void clearActivationContext(void)
{
    core4ParamSyncActivationRollbackRetained = 0u;
    core4ParamSyncActivationRollbackInverseRetained = UINT32_MAX;
}

static uint8_t activatePreparedRuntimeSlot(uint8_t slotExternal,
uint32_t transactionId)
{
    core4VisionParamsStruct params;
    if (core4VisionParamsFlashActivatePrepared((uint8_t) (slotExternal - 1u),
        transactionId, &params) != CORE4_VISION_PARAMS_LOAD_VALID)
        return 0u;
    core4VisionParamsCommit(&params);
    return 1u;
}

static uint8_t activateRuntimeSlotPending(uint8_t slotExternal)
{
    core4VisionParamsStruct params;
    if (core4VisionParamsFlashActivateSlotPending((uint8_t) (slotExternal - 1u),
        &params) != CORE4_VISION_PARAMS_LOAD_VALID) return 0u;
    core4VisionParamsCommit(&params);
    return 1u;
}

static uint8_t rollbackActivation(void)
{
    uint8_t oldSlotExternal, newSlotExternal;
    (void) activationContext(&oldSlotExternal, &newSlotExternal);
    {
        core4VisionParamsStruct params;
        (void) oldSlotExternal;
        (void) newSlotExternal;
        if (core4VisionParamsFlashAbortPendingActivation(&params) !=
            CORE4_VISION_PARAMS_LOAD_VALID) return 0u;
        core4VisionParamsCommit(&params);
    }
    clearActivationContext();
    return 1u;
}

static uint8_t recoverActivationAfterOwnerBoot(void)
{
    if (g_activationRecoveryChecked != 0u) return 1u;
    g_activationRecoveryChecked = 1u;
    return 1u;
}

static void putU16(uint8_t *output, uint16_t value)
{
    output[0] = (uint8_t) value;
    output[1] = (uint8_t) (value >> 8);
}

static void putU32(uint8_t *output, uint32_t value)
{
    output[0] = (uint8_t) value;
    output[1] = (uint8_t) (value >> 8);
    output[2] = (uint8_t) (value >> 16);
    output[3] = (uint8_t) (value >> 24);
}

static uint16_t getU16(const uint8_t *input)
{
    return (uint16_t) (input[0] | ((uint16_t) input[1] << 8));
}

static uint32_t getU32(const uint8_t *input)
{
    return (uint32_t) input[0] | ((uint32_t) input[1] << 8) |
        ((uint32_t) input[2] << 16) | ((uint32_t) input[3] << 24);
}

```

```

}

static uint8_t externalSlotValid(uint8_t slot)
{
    return (uint8_t) ((slot >= 1u) &&
        (slot <= CORE4_VISION_PARAMS_SLOT_COUNT));
}

static void clearMutationLock(void)
{
    g_syncMutationLocked = 0u;
    g_syncLockedSlot = 0u;
    g_syncLockedTransaction = 0u;
}

static uint8_t decodeGlobalTransaction(const paramSyncMailboxRequest *request,
    uint8_t allowDefaultSlot, uint8_t *slot, uint32_t *transactionId)
{
    if (request->dataLength != 5u) return 0u;
    *slot = request->data[0];
    *transactionId = getU32(&request->data[1]);
    return (uint8_t) ((*transactionId != 0u) &&
        (*transactionId != UINT32_MAX) &&
        (externalSlotValid(*slot) != 0u ||
            (allowDefaultSlot != 0u && *slot == 0u)));
}

static uint8_t transactionLocked(uint8_t slot, uint32_t transactionId)
{
    return (uint8_t) (g_syncMutationLocked != 0u &&
        g_syncLockedSlot == slot &&
        g_syncLockedTransaction == transactionId);
}

static uint8_t producerBootChangeAllowed(
    const paramSyncMailboxRequest *request)
{
    core4VisionParamsRuntimeMeta runtimeMeta;
    uint8_t oldSlot, newSlot;
    uint8_t activationPending;
    uint8_t transactionPending;
    uint32_t transactionId;

    if (core4VisionParamsFlashGetRuntimeMeta(&runtimeMeta) !=
        CORE4_VISION_PARAMS_LOAD_VALID) return 0u;
    activationPending = (uint8_t) (activationContext(&oldSlot, &newSlot) ||
        (runtimeMeta.pending != 0u &&
            runtimeMeta.transactionId == UINT32_MAX));
    transactionPending = (uint8_t) (g_syncMutationLocked || g_stagedValid ||
        g_flashPrepared || g_flashCommitted || runtimeMeta.pending);
    if ((activationPending == 0u) && (transactionPending == 0u)) return 1u;
    if ((request->op != PARAM_SYNC_OP_FINISH &&
        request->op != PARAM_SYNC_OP_ABORT) ||
        (request->dataLength != 4u)) return 0u;

    transactionId = getU32(request->data);
    if (activationPending != 0u)
    {
        if ((transactionId != 0u) || g_stagedValid || g_flashPrepared ||
            g_flashCommitted) return 0u;
        return 1u;
    }
    if ((g_syncMutationLocked != 0u) &&
        (transactionId != g_syncLockedTransaction)) return 0u;
    if ((g_stagedValid || g_flashPrepared || g_flashCommitted) &&
        (transactionId != g_stagedTransactionId)) return 0u;
    if ((runtimeMeta.pending != 0u) &&
        (transactionId != runtimeMeta.transactionId)) return 0u;
    return 1u;
}

uint8_t core4ParamSyncOwnerServiceMutationLocked(void)
{
    return (uint8_t) (g_syncMutationLocked || g_stagedValid ||
        g_flashPrepared || g_flashCommitted ||
        core4VisionParamsFlashMutationPending());
}

static void clearStaging(void)

```

```

{
    memset(&g_stagedBlob, 0, sizeof(g_stagedBlob));
    g_stagedTransactionId = 0u;
    g_stagedExpectedCrc = 0u;
    g_stagedBytes = 0u;
    g_stagedReceived = 0u;
    g_stagedSlot = 0u;
    g_stagedValid = 0u;
    g_flashPrepared = 0u;
    g_flashCommitted = 0u;
    g_stageKind = PARAM_STAGE_NONE;
    g_runtimeApplied = 0u;
}

static uint8_t rollbackTransaction(uint32_t transactionId)
{
    core4VisionParamsStruct params;
    if (core4VisionParamsFlashAbortTransaction(transactionId) !=
        CORE4_VISION_PARAMS_SAVE_VERIFIED) return 0u;
    if (core4VisionParamsFlashReadRuntime(&params) !=
        CORE4_VISION_PARAMS_LOAD_VALID) return 0u;
    core4VisionParamsCommit(&params);
    return 1u;
}

static void publish(const paramSyncMailboxRequest *request, uint8_t status,
    const uint8_t *data, uint16_t length, uint32_t crc)
{
    paramSyncMailboxResponse response;
    memset(&response, 0, sizeof(response));
    g_responseSequence++;
    if (g_responseSequence == 0u) g_responseSequence = 1u;
    response.producerBootGeneration = g_ownerBootGeneration;
    response.sequence = g_responseSequence;
    response.transactionId = request->transactionId;
    response.offset = request->offset;
    response.payloadCrc32 = crc;
    response.dataLength = length;
    response.op = request->op;
    response.slot = request->slot;
    response.target = request->target;
    response.status = status;
    if ((data != NULL) && (length != 0u)) memcpy(response.data, data, length);
    (void) paramSyncMailboxPublishResponse(&response);
}

static void handleReadMeta(const paramSyncMailboxRequest *request)
{
    uint8_t output[26];
    uint8_t externalSlot;
    uint8_t internalSlot;
    core4VisionParamsLibraryMeta meta;
    core4VisionParamsStruct params;
    core4VisionParamsFlashBlob blob;
    uint32_t blobCrc = 0u;

    if ((request->dataLength != 1u) ||
        (externalSlotValid(request->data[0]) == 0u))
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    externalSlot = request->data[0];
    internalSlot = (uint8_t) (externalSlot - 1u);
    memset(&meta, 0, sizeof(meta));
    if (core4VisionParamsFlashInspectSlot(internalSlot, &meta) !=
        CORE4_VISION_PARAMS_LOAD_VALID)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    if (meta.state == CORE4_VISION_PARAMS_LIBRARY_VALID)
    {
        if (core4VisionParamsFlashReadSlot(internalSlot, &params, NULL) !=
            CORE4_VISION_PARAMS_LOAD_VALID)
        {
            publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
            return;
        }
    }
}

```

```

        core4VisionParamsBlobEncode(&params, &blob);
        blobCrc = getU32(&blob.bytes[CORE4_VISION_PARAMS_BLOB_CRC_BYTES]);
    }
    memset(output, 0, sizeof(output));
    output[0] = CORE4_PARAM_SYNC_TARGET;
    output[1] = externalSlot;
    output[2] = meta.state;
    output[3] = 0u;
    putU32(&output[4], CORE4_VISION_PARAMS_FLASH_VERSION);
    putU32(&output[8], meta.generation);
    putU32(&output[12], blobCrc);
    putU32(&output[16], meta.transactionId);
    putU32(&output[20], CORE4_PARAM_SYNC_SCHEMA_HASH);
    putU16(&output[24], CORE4_VISION_PARAMS_BLOB_BYTES);
    publish(request, PARAM_SYNC_STATUS_OK, output, sizeof(output), 0u);
}

```

```

static void handleReadRuntimeMeta(const paramSyncMailboxRequest *request)

```

```

{
    uint8_t output[32];
    core4VisionParamsStruct runtime;
    core4VisionParamsRuntimeMeta meta;
    core4VisionParamsPersistenceStatus persistence;
    core4VisionParamsFlashBlob blob;
    uint32_t blobCrc;

    if (request->dataLength != 0u ||
        core4VisionParamsFlashReadRuntime(&runtime) !=
            CORE4_VISION_PARAMS_LOAD_VALID ||
        core4VisionParamsFlashGetRuntimeMeta(&meta) !=
            CORE4_VISION_PARAMS_LOAD_VALID)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    core4VisionParamsBlobEncode(&runtime, &blob);
    core4VisionParamsPersistenceGetStatus(&persistence);
    blobCrc = getU32(&blob.bytes[CORE4_VISION_PARAMS_BLOB_CRC_BYTES]);
    memset(output, 0, sizeof(output));
    output[0] = CORE4_PARAM_SYNC_TARGET;
    output[1] = meta.valid;
    output[2] = meta.bank;
    output[3] = meta.pending;
    output[4] = (uint8_t) (memcmp(&runtime, core4VisionParamsGetActive(),
        sizeof(runtime)) != 0);
    output[5] = meta.sourceSlot;
    output[6] = persistence.runtimeDefaultsRewritten != 0u ? 1u : 0u;
    putU32(&output[8], CORE4_VISION_PARAMS_FLASH_VERSION);
    putU32(&output[12], meta.generation);
    putU32(&output[16], blobCrc);
    putU32(&output[20], meta.transactionId);
    putU32(&output[24], CORE4_PARAM_SYNC_SCHEMA_HASH);
    putU16(&output[28], CORE4_VISION_PARAMS_BLOB_BYTES);
    publish(request, PARAM_SYNC_STATUS_OK, output, sizeof(output), 0u);
}

```

```

static void handleReadRuntimeChunk(const paramSyncMailboxRequest *request)

```

```

{
    uint8_t output[PARAM_SYNC_MAILBOX_DATA_MAX];
    uint16_t offset;
    uint16_t length;
    uint32_t blobCrc;
    core4VisionParamsStruct params;
    core4VisionParamsFlashBlob blob;

    if (request->dataLength != 4u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    offset = getU16(&request->data[0]);
    length = getU16(&request->data[2]);
    if (length == 0u || length > (PARAM_SYNC_MAILBOX_DATA_MAX - 4u) ||
        (uint32_t) offset + length > CORE4_VISION_PARAMS_BLOB_BYTES ||
        core4VisionParamsFlashReadRuntime(&params) !=
            CORE4_VISION_PARAMS_LOAD_VALID)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
}

```

```

    }
    core4VisionParamsBlobEncode(&params, &blob);
    blobCrc = getU32(&blob.bytes[CORE4_VISION_PARAMS_BLOB_CRC_BYTES]);
    putU16(&output[0], offset);
    putU16(&output[2], CORE4_VISION_PARAMS_BLOB_BYTES);
    memcpy(&output[4], &blob.bytes[offset], length);
    publish(request, PARAM_SYNC_STATUS_OK, output,
            (uint16_t) (4u + length), blobCrc);
}

static void handleReadChunk(const paramSyncMailboxRequest *request)
{
    uint8_t output[PARAM_SYNC_MAILBOX_DATA_MAX];
    uint8_t externalSlot;
    uint8_t internalSlot;
    uint16_t offset;
    uint16_t length;
    uint32_t blobCrc;
    core4VisionParamsStruct params;
    core4VisionParamsSlotMeta meta;
    core4VisionParamsFlashBlob blob;

    if (request->dataLength != 5u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    externalSlot = request->data[0];
    offset = getU16(&request->data[1]);
    length = getU16(&request->data[3]);
    if ((externalSlotValid(externalSlot) == 0u) || (length == 0u) ||
        (length > (PARAM_SYNC_MAILBOX_DATA_MAX - 5u)) ||
        (offset >= CORE4_VISION_PARAMS_BLOB_BYTES) ||
        ((uint32_t) offset + length > CORE4_VISION_PARAMS_BLOB_BYTES))
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    internalSlot = (uint8_t) (externalSlot - 1u);
    if (core4VisionParamsFlashReadSlot(internalSlot, &params, &meta) !=
        CORE4_VISION_PARAMS_LOAD_VALID)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    core4VisionParamsBlobEncode(&params, &blob);
    blobCrc = getU32(&blob.bytes[CORE4_VISION_PARAMS_BLOB_CRC_BYTES]);
    output[0] = externalSlot;
    putU16(&output[1], offset);
    putU16(&output[3], CORE4_VISION_PARAMS_BLOB_BYTES);
    memcpy(&output[5], &blob.bytes[offset], length);
    publish(request, PARAM_SYNC_STATUS_OK, output,
            (uint16_t) (5u + length), blobCrc);
}

static void handlePrepareLibraryFromLive(
    const paramSyncMailboxRequest *request)
{
    core4VisionParamsFlashBlob blob;
    const core4VisionParamsStruct *active;
    uint8_t slot;
    uint32_t transactionId;
    uint32_t blobCrc;

    if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (g_flashPrepared != 0u || g_flashCommitted != 0u)
    {
        publish(request, (g_stageKind == PARAM_STAGE_LIBRARY &&
            g_stagedSlot == slot &&
            g_stagedTransactionId == transactionId) ? PARAM_SYNC_STATUS_OK :
            PARAM_SYNC_STATUS_BUSY, NULL, 0u, g_stagedExpectedCrc);
        return;
    }
    active = core4VisionParamsGetActive();

```

```

core4VisionParamsBlobEncode(active, &blob);
blobCrc = getU32(&blob.bytes[CORE4_VISION_PARAMS_BLOB_CRC_BYTES]);
if (core4VisionParamsFlashPrepareSlot((uint8_t) (slot - 1u), active,
    transactionId) != CORE4_VISION_PARAMS_SAVE_VERIFIED)
{
    publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    return;
}
clearStaging();
g_stagedBlob = blob;
g_stagedSlot = slot;
g_stagedTransactionId = transactionId;
g_stagedExpectedCrc = blobCrc;
g_stagedBytes = CORE4_VISION_PARAMS_BLOB_BYTES;
g_stagedReceived = CORE4_VISION_PARAMS_BLOB_BYTES;
g_stagedValid = 1u;
g_flashPrepared = 1u;
g_stageKind = PARAM_STAGE_LIBRARY;
publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, blobCrc);
}

static void handlePrepareRuntime(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;
    core4VisionParamsSaveStatus status;

    if (decodeGlobalTransaction(request, 0u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u || slot == 0u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (g_flashPrepared != 0u || g_flashCommitted != 0u)
    {
        publish(request, (g_stageKind == PARAM_STAGE_RUNTIME &&
            g_stagedSlot == slot &&
            g_stagedTransactionId == transactionId) ? PARAM_SYNC_STATUS_OK :
            PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
        return;
    }
    status = core4VisionParamsFlashPrepareRuntimeFromSlot(
        (uint8_t) (slot - 1u), transactionId);
    if (status != CORE4_VISION_PARAMS_SAVE_VERIFIED)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    clearStaging();
    g_stagedSlot = slot;
    g_stagedTransactionId = transactionId;
    g_stagedValid = 1u;
    g_flashPrepared = 1u;
    g_stageKind = PARAM_STAGE_RUNTIME;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleCommitRuntime(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;
    if (decodeGlobalTransaction(request, 1u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u ||
        g_stageKind != PARAM_STAGE_RUNTIME || g_flashPrepared == 0u ||
        g_stagedSlot != slot || g_stagedTransactionId != transactionId)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (g_flashCommitted == 0u &&
        core4VisionParamsFlashCommitRuntime(transactionId) !=
            CORE4_VISION_PARAMS_SAVE_VERIFIED)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    g_flashCommitted = 1u;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

```



```

static void handleApplyRuntime(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;
    if (decodeGlobalTransaction(request, 1u, &slot, &transactionId) == 0u ||
        transactionLocked(slot, transactionId) == 0u ||
        g_stageKind != PARAM_STAGE_RUNTIME || g_flashCommitted == 0u ||
        g_stagedSlot != slot || g_stagedTransactionId != transactionId)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (g_runtimeApplied == 0u &&
        core4VisionParamsFlashApplyPreparedRuntime(transactionId) !=
        CORE4_VISION_PARAMS_SAVE_VERIFIED)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    g_runtimeApplied = 1u;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handlePrepareBegin(const paramSyncMailboxRequest *request)
{
    uint32_t transactionId;
    if ((request->dataLength != 19u) ||
        (externalSlotValid(request->data[0]) == 0u) ||
        (getU32(&request->data[5]) != CORE4_VISION_PARAMS_FLASH_VERSION) ||
        (getU32(&request->data[9]) != CORE4_PARAM_SYNC_SCHEMA_HASH) ||
        (getU16(&request->data[17]) != CORE4_VISION_PARAMS_BLOB_BYTES))
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    transactionId = getU32(&request->data[1]);
    if (transactionId == 0u || transactionId == UINT32_MAX)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (g_flashPrepared != 0u && rollbackTransaction(g_stagedTransactionId) == 0u)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    clearStaging();
    g_stagedSlot = request->data[0];
    g_stagedTransactionId = transactionId;
    g_stagedExpectedCrc = getU32(&request->data[13]);
    g_stagedBytes = getU16(&request->data[17]);
    g_stagedValid = 1u;
    g_stageKind = PARAM_STAGE_LIBRARY;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handlePrepareChunk(const paramSyncMailboxRequest *request)
{
    uint16_t offset;
    uint16_t length;
    if ((g_stagedValid == 0u) || (request->dataLength < 5u))
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    offset = getU16(&request->data[1]);
    length = getU16(&request->data[3]);
    if ((request->data[0] != g_stagedSlot) ||
        (offset != g_stagedReceived) ||
        (length != (uint16_t) (request->dataLength - 5u)) ||
        ((uint32_t) offset + length > g_stagedBytes))
    {
        clearStaging();
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    memcpy(&g_stagedBlob.bytes[offset], &request->data[5], length);
    g_stagedReceived = (uint16_t) (g_stagedReceived + length);
}

```

```

        publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
    }

static void handlePrepareFinish(const paramSyncMailboxRequest *request)
{
    uint32_t internalCrc;
    core4VisionParamsStruct params;
    core4VisionParamsStruct sanitized;
    if ((g_stagedValid == 0u) || (request->dataLength != 1u) ||
        (request->data[0] != g_stagedSlot) ||
        (g_stagedReceived != g_stagedBytes))
    {
        clearStaging();
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    internalCrc = getU32(
        &g_stagedBlob.bytes[CORE4_VISION_PARAMS_BLOB_CRC_BYTES]);
    if (g_flashPrepared != 0u)
    {
        publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, internalCrc);
        return;
    }
    if (core4VisionParamsBlobDecode(&g_stagedBlob, &params) ==
        CORE4_VISION_PARAMS_BLOB_VALID)
    {
        sanitized = params;
        core4VisionParamsSanitize(&sanitized);
    }
    else
    {
        clearStaging();
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    if ((internalCrc != g_stagedExpectedCrc) ||
        (crc32IeeeCalc(g_stagedBlob.bytes,
            CORE4_VISION_PARAMS_BLOB_CRC_BYTES) != internalCrc) ||
        (memcmp(&sanitized, &params, sizeof(params)) != 0) ||
        (core4VisionParamsFlashPrepareSlot((uint8_t) (g_stagedSlot - 1u),
            &params, g_stagedTransactionId) !=
            CORE4_VISION_PARAMS_SAVE_VERIFIED))
    {
        clearStaging();
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    g_flashPrepared = 1u;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, internalCrc);
}

static void handleCommit(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint32_t transactionId;
    if ((request->dataLength != 5u) ||
        (externalSlotValid(request->data[0]) == 0u))
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    slot = request->data[0];
    transactionId = getU32(&request->data[1]);
    if ((g_stageKind != PARAM_STAGE_LIBRARY) ||
        (g_stagedValid == 0u) || (g_flashPrepared == 0u) ||
        (slot != g_stagedSlot) || (transactionId != g_stagedTransactionId))
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    if (g_flashCommitted != 0u)
    {
        publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
        return;
    }
    if (core4VisionParamsFlashCommitPrepared((uint8_t) (slot - 1u),
        transactionId) != CORE4_VISION_PARAMS_SAVE_VERIFIED)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
    }
}

```

```

        return;
    }
    g_flashCommitted = 1u;
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleActivate(const paramSyncMailboxRequest *request)
{
    uint8_t slot;
    uint8_t oldSlotExternal, pendingSlot;
    uint32_t transactionId;
    if ((request->dataLength != 5u) ||
        (externalSlotValid(request->data[0]) == 0u))
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    slot = request->data[0];
    transactionId = getU32(&request->data[1]);
    if (((transactionId != 0u) && ((g_stagedValid == 0u) ||
        (g_flashCommitted == 0u) ||
        (transactionId != g_stagedTransactionId) ||
        (slot != g_stagedSlot))))
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    if (transactionId == 0u)
    {
        if (!g_syncMutationLocked || g_syncLockedSlot != slot ||
            g_stagedValid || g_flashPrepared ||
            g_flashCommitted)
        {
            publish(request, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
            return;
        }
        if (activationContext(&oldSlotExternal, &pendingSlot))
        {
            publish(request, pendingSlot == slot ? PARAM_SYNC_STATUS_OK : PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
            return;
        }
        oldSlotExternal = (uint8_t) (core4VisionParamsFlashGetActiveSlot() + 1u);
        if (!externalSlotValid(oldSlotExternal))
        {
            publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
            return;
        }
        setActivationContext(oldSlotExternal, slot);
    }
    if (!(transactionId == 0u ? activateRuntimeSlotPending(slot) :
        activatePreparedRuntimeSlot(slot, transactionId)))
    {
        if (transactionId == 0u) (void) rollbackActivation();
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleAbort(const paramSyncMailboxRequest *request)
{
    uint32_t transactionId;
    if (request->dataLength != 4u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    transactionId = getU32(request->data);
    if (transactionId == UINT32_MAX)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    if (transactionId == 0u)
    {
        uint8_t ok = rollbackActivation();
        if (ok) clearMutationLock();
        publish(request, ok ? PARAM_SYNC_STATUS_OK : PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
}

```

```

    }
    if (!g_stagedValid && !g_flashPrepared && !g_flashCommitted &&
        transactionId == g_lastAbortedTransaction)
    {
        publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
        return;
    }
    if (((g_stagedValid != 0u) &&
        (transactionId != g_stagedTransactionId)) ||
        (rollbackTransaction(transactionId) == 0u))
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    g_lastAbortedTransaction = transactionId;
    clearStaging();
    clearMutationLock();
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleFinish(const paramSyncMailboxRequest *request)
{
    uint32_t transactionId;
    if (request->dataLength != 4u)
    {
        publish(request, PARAM_SYNC_STATUS_INVALID, NULL, 0u, 0u);
        return;
    }
    transactionId = getU32(request->data);
    if (transactionId == 0u)
    {
        if (core4VisionParamsFlashFinishPendingActivation() !=
            CORE4_VISION_PARAMS_SAVE_VERIFIED)
        {
            publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
            return;
        }
        clearActivationContext();
        clearMutationLock();
        publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
        return;
    }
    if (!g_stagedValid && !g_flashPrepared && !g_flashCommitted &&
        transactionId == g_lastFinishedTransaction)
    {
        publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
        return;
    }
    if ((g_stagedValid != 0u) &&
        ((g_stageKind == PARAM_STAGE_LIBRARY) &&
         (g_flashCommitted == 0u)) ||
        ((g_stageKind == PARAM_STAGE_RUNTIME) &&
         ((g_flashCommitted == 0u) || (g_runtimeApplied == 0u))) ||
        (transactionId != g_stagedTransactionId))
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    if (core4VisionParamsFlashFinishTransaction(transactionId) !=
        CORE4_VISION_PARAMS_SAVE_VERIFIED)
    {
        publish(request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    g_lastFinishedTransaction = transactionId;
    clearStaging();
    clearMutationLock();
    publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static void handleLock(const paramSyncMailboxRequest *request)
{
    uint8_t oldSlot, newSlot;
    uint32_t transactionId = (request->dataLength == 5u) ?
        getU32(&request->data[1]) : 0u;
    if (request->dataLength != 5u || request->data[0] > 5u ||
        transactionId == 0u || transactionId == UINT32_MAX ||
        (g_syncMutationLocked && (g_syncLockedSlot != request->data[0] ||
            g_syncLockedTransaction != transactionId)) ||

```

```

    g_stagedValid || g_flashPrepared || g_flashCommitted ||
    core4VisionParamsFlashMutationPending() ||
    activationContext(&oldSlot, &newSlot))
{
    publish(request, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
    return;
}
g_syncMutationLocked = 1u;
g_syncLockedSlot = request->data[0];
g_syncLockedTransaction = transactionId;
publish(request, PARAM_SYNC_STATUS_OK, NULL, 0u, 0u);
}

static uint8_t retainedRequestMayResume(
    const paramSyncMailboxRequest *request)
{
    if (request->target != CORE4_PARAM_SYNC_TARGET) return 0u;
    return (uint8_t) (request->op == PARAM_SYNC_OP_QUERY ||
        request->op == PARAM_SYNC_OP_READ_META ||
        request->op == PARAM_SYNC_OP_READ_CHUNK ||
        request->op == PARAM_SYNC_OP_READ_RUNTIME_META ||
        request->op == PARAM_SYNC_OP_READ_RUNTIME_CHUNK ||
        request->op == PARAM_SYNC_OP_RECOVERY ||
        request->op == PARAM_SYNC_OP_ABORT ||
        request->op == PARAM_SYNC_OP_FINISH);
}

void core4ParamSyncOwnerServiceInit(void)
{
    paramSyncMailboxRequest retainedRequest;
    g_lastBootGeneration = 0u;
    g_lastRequestSequence = 0u;
    g_responseSequence = 0u;
    core4CoreDataTimingInit();
    if (g_ownerBootGeneration == 0u)
        g_ownerBootGeneration = core4CoreDataMakeBootGeneration(0xC4745053u);
    if (paramSyncMailboxTryReadRequest(&retainedRequest) != 0u &&
        retainedRequestMayResume(&retainedRequest) == 0u)
    {
        g_lastBootGeneration = retainedRequest.producerBootGeneration;
        g_lastRequestSequence = retainedRequest.sequence;
        if (retainedRequest.target == CORE4_PARAM_SYNC_TARGET)
            publish(&retainedRequest, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
    }
}

void core4ParamSyncOwnerServicePoll(void)
{
    paramSyncMailboxRequest request;
    if (g_ownerBootGeneration == 0u)
        g_ownerBootGeneration = core4CoreDataMakeBootGeneration(0xC4745053u);
    if (paramSyncMailboxTryReadRequest(&request) == 0u) return;
    if ((request.producerBootGeneration == g_lastBootGeneration) &&
        (request.sequence == g_lastRequestSequence)) return;
    if (request.producerBootGeneration == g_lastBootGeneration &&
        (int32_t) (request.sequence - g_lastRequestSequence) <= 0) return;
    if ((g_lastBootGeneration != 0u) &&
        (request.producerBootGeneration != g_lastBootGeneration) &&
        (producerBootChangeAllowed(&request) == 0u))
    {
        if (request.target == CORE4_PARAM_SYNC_TARGET)
            publish(&request, PARAM_SYNC_STATUS_BUSY, NULL, 0u, 0u);
        return;
    }
    g_lastBootGeneration = request.producerBootGeneration;
    g_lastRequestSequence = request.sequence;
    if (request.target != CORE4_PARAM_SYNC_TARGET) return;
    if (recoverActivationAfterOwnerBoot() == 0u)
    {
        publish(&request, PARAM_SYNC_STATUS_FAILED, NULL, 0u, 0u);
        return;
    }
    switch (request.op)
    {
        case PARAM_SYNC_OP_READ_META: handleReadMeta(&request); break;
        case PARAM_SYNC_OP_READ_CHUNK: handleReadChunk(&request); break;
        case PARAM_SYNC_OP_READ_RUNTIME_META:
            handleReadRuntimeMeta(&request); break;
        case PARAM_SYNC_OP_READ_RUNTIME_CHUNK:

```

```
        handleReadRuntimeChunk(&request); break;
    case PARAM_SYNC_OP_PREPARE_LIBRARY_FROM_LIVE:
        handlePrepareLibraryFromLive(&request); break;
    case PARAM_SYNC_OP_PREPARE_RUNTIME_FROM_SLOT:
        handlePrepareRuntime(&request); break;
    case PARAM_SYNC_OP_COMMIT_RUNTIME:
        handleCommitRuntime(&request); break;
    case PARAM_SYNC_OP_APPLY_RUNTIME:
        handleApplyRuntime(&request); break;
    case PARAM_SYNC_OP_PREPARE_BEGIN: handlePrepareBegin(&request); break;
    case PARAM_SYNC_OP_PREPARE_CHUNK: handlePrepareChunk(&request); break;
    case PARAM_SYNC_OP_PREPARE_VERIFY: handlePrepareFinish(&request); break;
    case PARAM_SYNC_OP_COMMIT: handleCommit(&request); break;
    case PARAM_SYNC_OP_ACTIVATE: handleActivate(&request); break;
    case PARAM_SYNC_OP_ABORT: handleAbort(&request); break;
    case PARAM_SYNC_OP_FINISH: handleFinish(&request); break;
    case PARAM_SYNC_OP_LOCK: handleLock(&request); break;
    default:
        publish(&request, PARAM_SYNC_STATUS_REJECTED, NULL, 0u, 0u);
        break;
    }
}
```

core4_cyt4bb7/project/code/comm/paramSyncOwnerService.h

SHA-256

:

cb4900f08e571a34abd5f933cc3a8958adc4dc86bab8fb1dee91d60ee1e8bb16

代码 (c)

```
#ifndef CORE4_PARAM_SYNC_OWNER_SERVICE_H
#define CORE4_PARAM_SYNC_OWNER_SERVICE_H

#include <stdint.h>

void core4ParamSyncOwnerServiceInit(void);
void core4ParamSyncOwnerServicePoll(void);
uint8_t core4ParamSyncOwnerServiceMutationLocked(void);

#endif
```

core4_cyt4bb7/project/code/config/commonDef.h

SHA-256

:

4e78e15f4c288ebe9906ffc19bea1f0d1331a3eaa04c7dad14a76d3b12bce98d

代码 (c)

```
#ifndef CORE4_COMMON_DEF_H
#define CORE4_COMMON_DEF_H

#include <stdint.h>

#define range(x, min, max)      (((x) < (min)) ? (min) : (((x) > (max)) ? (max) : (x)))
#define unsignRange(x, max)     (((x) > (max)) ? (max) : (x))

#define toRad(x)                ((x) * 0.017453292519943295f)
#define toDeg(x)                ((x) * 57.29577951308232f)

#ifndef PI
#define PI                      (3.14159265358979323846f)
#endif
#define TWO_PI                  (6.28318530717958647692f)
#define HALF_PI                 (1.57079632679489661923f)

#define BLACK                   (0)
#define WHITE                   (255)
#define cReverse(x)             ((x) ? BLACK : WHITE)

static inline float clampf(float value, float minValue, float maxValue)
```

```

{
    return (value < minValue) ? minValue : ((value > maxValue) ? maxValue : value);
}

static inline float wrapToPi(float angle)
{
    while (angle > PI)
    {
        angle -= TWO_PI;
    }
    while (angle < -PI)
    {
        angle += TWO_PI;
    }
    return angle;
}

typedef struct
{
    uint8_t* ptr;
    uint32_t xLen;
    uint32_t yLen;
    uint32_t size;
} u8Image;

#endif /* CORE4_COMMON_DEF_H */

```

core4_cyt4bb7/project/code/config/configBeaconTrack.h

SHA-256

:

438acf0956aa5f911a59f24a5e08ad712ae5510f6d9b30618c205ef4b4ebf585

代码 (c)

```

#ifndef CONFIG_BEACON_TRACK_H_
#define CONFIG_BEACON_TRACK_H_

/*
 * Core4 信标"跟踪层"调参中心：时间一致性跟踪（抗光关键层）。
 * 连续 confirmHits 帧关联才确认输出、连续 dropMisses 帧丢失才删除、coastMisses 帧内丢失仍滑行。
 * 关联更新用一阶低通平滑（位置/面积各一 alpha，越小越平滑）；新轨迹直贴测量值。
 * 轨迹内部用坐标速度(px/frame)与面积速度(area pixels/frame)预测下一帧关联残差；
 * 这些速度只作为轨迹事实，不负责 active target 选择。
 * 详见 vision/beaconTrack.c。共享跟踪参数由 Core1 配置下发并在帧边界生效。
 */

/* Core4 相机局部阈值不在本文件，由 configCore4VisionParams.h 管理。 */

#define VISION_TRACK_MAX_ASSOC_DIST_DEFAULT (6.0f) /* 帧间质心最大关联距离(px) */
#define VISION_TRACK_SIZE_RATIO_MAX_DEFAULT (2.0f) /* 面积连续性: 检测面积/预测面积 比值上限 */
#define VISION_TRACK_CONFIRM_HITS_DEFAULT (3u) /* 连续命中确认帧数 */
#define VISION_TRACK_DROP_MISSES_DEFAULT (4u) /* 已确认轨迹连续丢失删除帧数 */
#define VISION_TRACK_COAST_MISSES_DEFAULT (2u) /* 已确认轨迹丢失时仍输出(滑行)帧数 */

/* 发布边界施密特门：0/0 表示该边关闭；非关闭边 Enter>=1 且 Exit<Enter。
 * Exit 需连续越界 hold 帧才撤销发布资格，滤掉边缘单帧检测抖动。
 * Core4 默认关闭下边；正式运行值由 Core1 共享配置同步。 */
#define VISION_TRACK_BORDER_EXIT_HOLD_FRAMES_DEFAULT (2u)
#define VISION_TRACK_EDGE_PREDICTION_HOLD_MS_DEFAULT (500u)
#define VISION_TRACK_EDGE_PREDICTION_HOLD_MS_MAX (10 000u)
#define VISION_TRACK_CAMERA_FRAME_PERIOD_MS (20u)
#define VISION_TRACK_EDGE_BAND_C1_LEFT_DEFAULT (4u)
#define VISION_TRACK_EDGE_BAND_C1_RIGHT_DEFAULT (10u)
#define VISION_TRACK_EDGE_BAND_C1_TOP_DEFAULT (4u)
#define VISION_TRACK_EDGE_BAND_C1_BOTTOM_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_OBSERVER_LEFT_DEFAULT (4u)
#define VISION_TRACK_EDGE_BAND_OBSERVER_RIGHT_DEFAULT (4u)
#define VISION_TRACK_EDGE_BAND_OBSERVER_TOP_DEFAULT (0u)
#define VISION_TRACK_EDGE_BAND_OBSERVER_BOTTOM_DEFAULT (10u)
#define VISION_TRACK_EDGE_BAND_X_MAX (93.0f)
#define VISION_TRACK_EDGE_BAND_Y_MAX (59.0f)
#define VISION_TRACK_EDGE_VERTICAL_ASSOC_SCALE (2.0f)
#define VISION_TRACK_EDGE_ASSOC_PREDICT_FRAMES_MAX (3.0f)
#define VISION_TRACK_BORDER_ENTER_LEFT_DEFAULT (24u)
#define VISION_TRACK_BORDER_EXIT_LEFT_DEFAULT (9u)

```

```
#define VISION_TRACK_BORDER_ENTER_RIGHT_DEFAULT    (24u)
#define VISION_TRACK_BORDER_EXIT_RIGHT_DEFAULT     (9u)
#define VISION_TRACK_BORDER_ENTER_TOP_DEFAULT      (8u)
#define VISION_TRACK_BORDER_EXIT_TOP_DEFAULT       (3u)
#define VISION_TRACK_BORDER_ENTER_BOTTOM_DEFAULT   (0u)
#define VISION_TRACK_BORDER_EXIT_BOTTOM_DEFAULT    (0u)
#define VISION_TRACK_BORDER_X_MAX                  (93u)
#define VISION_TRACK_BORDER_Y_MAX                  (59u)

/* 关联平滑一阶低通 alpha (tr = alpha*measured + (1-alpha)*tr, 越小越平滑、越抗抖) 。
 * 位置/面积独立, 默认 1.00 (直贴测量值; 原硬编码 0.5 IIR 现可调) 。上板复标: 移动信标 alpha 太小易掉关联。 */
#define VISION_TRACK_POSITION_ALPHA_DEFAULT (1.00f)
#define VISION_TRACK_SIZE_ALPHA_DEFAULT    (1.00f)
#define VISION_TRACK_ALPHA_MIN             (0.01f)
#define VISION_TRACK_ALPHA_MAX             (1.00f)
#define VISION_TRACK_ALPHA_STEP_SHORT      (0.05f)
#define VISION_TRACK_ALPHA_STEP_LONG       (0.20f)

/* 轨迹速度预测 (单位 px/frame, 仅用于下一帧关联与事实输出; 不进入控制 ABI) 。 */
#define VISION_TRACK_VELOCITY_ALPHA_DEFAULT (0.35f)
#define VISION_TRACK_VELOCITY_MAX_FACTOR_DEFAULT (1.50f)
#define VISION_TRACK_PREDICT_FRAMES_DEFAULT (1.0f)

/* 面积趋势辅助关联。areaVel 单位为 area pixels/frame。
 * 高影响权重/阈值进入 visionParams 菜单 + flash, 归一化与 cap 仍为编译期安全约束。
 * trackSizeRatioMax 仍是持久化参数, 语义保持“面积连续性比值上限”, 本轮只把比较基准改为预测面积。 */
#define VISION_TRACK_AREA_VEL_ALPHA_DEFAULT (0.35f)
#define VISION_TRACK_AREA_VEL_NORM_MIN      (16.0f)
#define VISION_TRACK_ASSOC_SCORE_MAX_DEFAULT (1.75f)
#define VISION_TRACK_ASSOC_AREA_WEIGHT_DEFAULT (0.45f)
#define VISION_TRACK_ASSOC_AREA_VEL_WEIGHT_DEFAULT (0.35f)
#define VISION_TRACK_ASSOC_AREA_ERR_CAP      (3.0f)
#define VISION_TRACK_ASSOC_AREA_VEL_ERR_CAP  (3.0f)
#define VISION_TRACK_ASSOC_AREA_VEL_NORM     (0.35f)
#define VISION_TRACK_ASSOC_AREA_VEL_MIN_ABS  (8.0f)
#define VISION_TRACK_AREA_VEL_MAX_RATE       (1.0f)

/* 菜单编辑约束 (confirm/drop/coast 帧数、关联距离、面积比) 。 */
#define VISION_TRACK_ASSOC_DIST_MIN          (1.0f)
#define VISION_TRACK_ASSOC_DIST_MAX         (100.0f)
#define VISION_TRACK_SIZE_RATIO_MIN         (1.1f)
#define VISION_TRACK_SIZE_RATIO_MAX         (10.0f)
#define VISION_TRACK_FRAMES_MIN             (1.0f)
#define VISION_TRACK_FRAMES_MAX             (60.0f)
#define VISION_TRACK_ASSOC_DIST_STEP_SHORT  (0.5f)
#define VISION_TRACK_ASSOC_DIST_STEP_LONG   (2.0f)
#define VISION_TRACK_SIZE_RATIO_STEP_SHORT  (0.1f)
#define VISION_TRACK_SIZE_RATIO_STEP_LONG   (0.5f)
#define VISION_TRACK_FRAMES_STEP_SHORT      (1.0f)
#define VISION_TRACK_FRAMES_STEP_LONG       (2.0f)

/* 面积趋势菜单编辑约束。权重 0=关闭对应 score 项。 */
#define VISION_TRACK_ASSOC_SCORE_MAX_MIN     (0.10f)
#define VISION_TRACK_ASSOC_SCORE_MAX_MAX     (10.00f)
#define VISION_TRACK_ASSOC_SCORE_MAX_STEP_SHORT (0.05f)
#define VISION_TRACK_ASSOC_SCORE_MAX_STEP_LONG (0.25f)
#define VISION_TRACK_ASSOC_WEIGHT_MIN        (0.00f)
#define VISION_TRACK_ASSOC_WEIGHT_MAX        (3.00f)
#define VISION_TRACK_ASSOC_WEIGHT_STEP_SHORT (0.05f)
#define VISION_TRACK_ASSOC_WEIGHT_STEP_LONG  (0.25f)

#endif /* _CONFIG_BEACON_TRACK_H_ */
```

core4_cyt4bb7/project/code/config/configCore4Runtime.h

SHA-256 :
9564a2dbb060266d9a446d3237994c43d54af9446616ebed7c6315e3504a5458

代码 (c)

```
#ifndef CORE4_CONFIG_CORE4_RUNTIME_H
#define CORE4_CONFIG_CORE4_RUNTIME_H

#define CORE4_RUNTIME_PIT_MS (10u)
```



```
#define CORE4_CORE_SHARED_HEARTBEAT_MS (50u)
#define CORE4_CORE_SHARED_STALE_MS   (600u)

#define CORE4_CAMERA_WIDTH           (188u)
#define CORE4_CAMERA_HEIGHT          (120u)
#define CORE4_CAMERA_EXPECTED_FRAME_MS (20u)
#define CORE4_CAMERA_FRAME_TIMEOUT_MS (200u)
#define CORE4_CAMERA_INIT_RETRY_MS   (1000u)
#define CORE4_CAMERA_CONFIG_RETRY_MS (1000u)
#define CORE4_CAMERA_RECOVERY_FRAMES (3u)

#define CORE4_DIAGNOSTICS_PREVIEW_X   (2u)
#define CORE4_DIAGNOSTICS_PREVIEW_Y   (17u)
#define CORE4_DIAGNOSTICS_PREVIEW_WIDTH (130u)
#define CORE4_DIAGNOSTICS_PREVIEW_HEIGHT (67u)
#define CORE4_DIAGNOSTICS_HUD_CHARS    (24u)
#define CORE4_MENU_RENDER_PERIOD_MS    (20u)

#define CORE4_RGB_PWM_FREQUENCY_HZ     (1000u)
#define CORE4_RGB_LOGICAL_BRIGHTNESS   (1000u)
#define CORE4_RGB_SLOW_HALF_PERIOD_MS  (500u)
#define CORE4_RGB_FAST_HALF_PERIOD_MS  (200u)

#define CORE4_TIMING_INVALID_US (0xFFFFFFFFu)

#endif /* CORE4_CONFIG_CORE4_RUNTIME_H */
```

core4_cyt4bb7/project/code/config/configCore4VisionParams.h

SHA-256 :
c6ce7011615eb0bfdb6076b6768b7540416efafa2e828bbacaff67ce35421cfd

代码 (c)

```
#ifndef CORE4_CONFIG_CORE4_VISION_PARAMS_H
#define CORE4_CONFIG_CORE4_VISION_PARAMS_H

/* 启动先建立旧 CYT2 等价的 500 曝光；活动参数随后按本默认值应用。 */
#define CORE4_VISION_EXPOSURE_DEFAULT (300u)
#define CORE4_VISION_EXPOSURE_MIN      (1u)
#define CORE4_VISION_EXPOSURE_MAX      (2000u)
#define CORE4_VISION_EXPOSURE_STEP_S   (1.0f)
#define CORE4_VISION_EXPOSURE_STEP_L   (10.0f)

#define CORE4_VISION_BINARY_LOW_DEFAULT (40u)
#define CORE4_VISION_BINARY_HIGH_DEFAULT (255u)
#define CORE4_VISION_BINARY_MIN         (5u)
#define CORE4_VISION_BINARY_MAX         (255u)
#define CORE4_VISION_BINARY_STEP_S      (1.0f)
#define CORE4_VISION_BINARY_STEP_L      (10.0f)

#define CORE4_VISION_LEGACY_CANNY_LOW_DEFAULT (150u)
#define CORE4_VISION_LEGACY_CANNY_HIGH_DEFAULT (200u)
#define CORE4_VISION_LEGACY_CANNY_MIN         (5u)
#define CORE4_VISION_LEGACY_CANNY_MAX         (500u)
#define CORE4_VISION_LEGACY_CANNY_GAP         (5u)
#define CORE4_VISION_LEGACY_CANNY_STEP_S      (1.0f)
#define CORE4_VISION_LEGACY_CANNY_STEP_L      (10.0f)

#define CORE4_VISION_HUD_PERIOD_DEFAULT_MS (200u)
#define CORE4_VISION_HUD_PERIOD_MIN_MS     (20u)
#define CORE4_VISION_HUD_PERIOD_MAX_MS     (200u)
#define CORE4_VISION_HUD_PERIOD_STEP_S      (10.0f)
#define CORE4_VISION_HUD_PERIOD_STEP_L      (100.0f)

#define CORE4_VISION_PREVIEW_PERIOD_DEFAULT_MS (40u)
#define CORE4_VISION_PREVIEW_PERIOD_MIN_MS     (40u)
#define CORE4_VISION_PREVIEW_PERIOD_MAX_MS     (2000u)
#define CORE4_VISION_PREVIEW_PERIOD_STEP_S      (50.0f)
#define CORE4_VISION_PREVIEW_PERIOD_STEP_L      (500.0f)

#define CORE4_VISION_STATS_PERIOD_DEFAULT_MS (1000u)
#define CORE4_VISION_STATS_PERIOD_MIN_MS     (1000u)
#define CORE4_VISION_STATS_PERIOD_MAX_MS     (5000u)
#define CORE4_VISION_STATS_PERIOD_STEP_S      (100.0f)
```

```
#define CORE4_VISION_STATS_PERIOD_STEP_L    (500.0f)

#endif /* CORE4_CONFIG_CORE4_VISION_PARAMS_H */
```

core4_cyt4bb7/project/code/config/configSerialDebug.h

SHA-256

:

5bb3ef9aeaa12f4bb0cc367b562f26ef66a81f2c98d949b6cc36368f2c02a64e

代码 (c)

```
#ifndef CORE4_CONFIG_SERIAL_DEBUG_H
#define CORE4_CONFIG_SERIAL_DEBUG_H

/* cm7_0 启动日志使用独立 UART1; Core1 业务专线固定由 UART5 承载。 */
#define SERIAL_DEBUG_UART_INDEX      (UART_1)
#define SERIAL_DEBUG_UART_BAUDRATE   (115200)
#define SERIAL_DEBUG_UART_TX_PIN     (UART1_TX_P04_1)
#define SERIAL_DEBUG_UART_RX_PIN     (UART1_RX_P04_0)
#define SERIAL_DEBUG_UART_USE_INTERRUPT (1)

#if (SERIAL_DEBUG_UART_BAUDRATE < 1)
#error "SERIAL_DEBUG_UART_BAUDRATE must be > 0"
#endif

#endif /* CORE4_CONFIG_SERIAL_DEBUG_H */
```

core4_cyt4bb7/project/code/config/core4VisionParams.c

SHA-256

:

3042320db117cecceb15094d9447a42f6caa666564b7f590fc375fa02cffffa4b

代码 (c)

```
#include "core4VisionParams.h"

#include <string.h>

static core4VisionParamsStruct g_activeParams;
static uint32_t g_revision;

static uint32_t clampU32(uint32_t value, uint32_t minimum, uint32_t maximum)
{
    if (value < minimum)
    {
        return minimum;
    }
    if (value > maximum)
    {
        return maximum;
    }
    return value;
}

static uint16_t clampU16(uint16_t value, uint16_t minimum, uint16_t maximum)
{
    if (value < minimum)
    {
        return minimum;
    }
    if (value > maximum)
    {
        return maximum;
    }
    return value;
}

void core4VisionParamsLoadDefaults(core4VisionParamsStruct *params)
{
    if (params == NULL)
```

```

{
    return;
}

memset(params, 0, sizeof(*params));
params->exposure = CORE4_VISION_EXPOSURE_DEFAULT;
params->hudPeriodMs = CORE4_VISION_HUD_PERIOD_DEFAULT_MS;
params->previewPeriodMs = CORE4_VISION_PREVIEW_PERIOD_DEFAULT_MS;
params->statsPeriodMs = CORE4_VISION_STATS_PERIOD_DEFAULT_MS;
params->binaryLow = CORE4_VISION_BINARY_LOW_DEFAULT;
params->binaryHigh = CORE4_VISION_BINARY_HIGH_DEFAULT;
params->legacyCannyLow = CORE4_VISION_LEGACY_CANNY_LOW_DEFAULT;
params->legacyCannyHigh = CORE4_VISION_LEGACY_CANNY_HIGH_DEFAULT;
}

void core4VisionParamsSanitize(core4VisionParamsStruct *params)
{
    if (params == NULL)
    {
        return;
    }

    if (params->exposure < CORE4_VISION_EXPOSURE_MIN)
    {
        params->exposure = CORE4_VISION_EXPOSURE_MIN;
    }
    if (params->exposure > CORE4_VISION_EXPOSURE_MAX)
    {
        params->exposure = CORE4_VISION_EXPOSURE_MAX;
    }
    params->hudPeriodMs = clampU32(params->hudPeriodMs,
        CORE4_VISION_HUD_PERIOD_MIN_MS,
        CORE4_VISION_HUD_PERIOD_MAX_MS);
    params->previewPeriodMs = clampU32(params->previewPeriodMs,
        CORE4_VISION_PREVIEW_PERIOD_MIN_MS,
        CORE4_VISION_PREVIEW_PERIOD_MAX_MS);
    params->statsPeriodMs = clampU32(params->statsPeriodMs,
        CORE4_VISION_STATS_PERIOD_MIN_MS,
        CORE4_VISION_STATS_PERIOD_MAX_MS);
    params->binaryLow = clampU16(params->binaryLow,
        CORE4_VISION_BINARY_MIN,
        CORE4_VISION_BINARY_MAX);
    params->binaryHigh = clampU16(params->binaryHigh,
        CORE4_VISION_BINARY_MIN,
        CORE4_VISION_BINARY_MAX);
    if (params->binaryLow > params->binaryHigh)
    {
        params->binaryLow = params->binaryHigh;
    }
    params->legacyCannyLow = clampU16(params->legacyCannyLow,
        CORE4_VISION_LEGACY_CANNY_MIN,
        CORE4_VISION_LEGACY_CANNY_MAX);
    params->legacyCannyHigh = clampU16(params->legacyCannyHigh,
        CORE4_VISION_LEGACY_CANNY_MIN,
        CORE4_VISION_LEGACY_CANNY_MAX);
    if ((uint32_t) params->legacyCannyLow + CORE4_VISION_LEGACY_CANNY_GAP >
        params->legacyCannyHigh)
    {
        if (params->legacyCannyHigh >=
            (CORE4_VISION_LEGACY_CANNY_MIN + CORE4_VISION_LEGACY_CANNY_GAP))
        {
            params->legacyCannyLow = (uint16_t) (params->legacyCannyHigh -
                CORE4_VISION_LEGACY_CANNY_GAP);
        }
        else
        {
            params->legacyCannyLow = CORE4_VISION_LEGACY_CANNY_MIN;
            params->legacyCannyHigh = (uint16_t) (CORE4_VISION_LEGACY_CANNY_MIN +
                CORE4_VISION_LEGACY_CANNY_GAP);
        }
    }
}

void core4VisionParamsSetInitial(const core4VisionParamsStruct *params)
{
    if (params == NULL)
    {
        return;
    }
}

```

```

    g_activeParams = *params;
    core4VisionParamsSanitize(&g_activeParams);
    g_revision = 1u;
}

void core4VisionParamsCommit(const core4VisionParamsStruct *params)
{
    if (params == NULL)
    {
        return;
    }

    g_activeParams = *params;
    core4VisionParamsSanitize(&g_activeParams);
    g_revision++;
    if (g_revision == 0u) g_revision = 1u;
}

const core4VisionParamsStruct *core4VisionParamsGetActive(void)
{
    return &g_activeParams;
}

uint32_t core4VisionParamsGetRevision(void)
{
    return g_revision;
}

```

core4_cyt4bb7/project/code/config/core4VisionParams.h

SHA-256

:

a8b139cc35db8aef3882d297e913ee57ed60346dd2684d71bcccbdf1c33eb245

代码 (c)

```

#ifndef CORE4_VISION_PARAMS_H
#define CORE4_VISION_PARAMS_H

#include <stdint.h>

#include "configCore4VisionParams.h"

typedef struct
{
    uint16_t exposure;
    uint32_t hudPeriodMs;
    uint32_t previewPeriodMs;
    uint32_t statsPeriodMs;
    uint16_t binaryLow;
    uint16_t binaryHigh;
    uint16_t legacyCannyLow;
    uint16_t legacyCannyHigh;
} core4VisionParamsStruct;

void core4VisionParamsLoadDefaults(core4VisionParamsStruct *params);
void core4VisionParamsSanitize(core4VisionParamsStruct *params);
void core4VisionParamsSetInitial(const core4VisionParamsStruct *params);
void core4VisionParamsCommit(const core4VisionParamsStruct *params);
const core4VisionParamsStruct *core4VisionParamsGetActive(void);
uint32_t core4VisionParamsGetRevision(void);

#endif /* CORE4_VISION_PARAMS_H */

```

core4_cyt4bb7/project/code/config/core4VisionParamsBlob.c

SHA-256

:

b241ed903462069a62c2b4ecd0d7e0b0f1026167589bc98a5ec3289f23e1eef4

代码 (c)

```
#include "core4VisionParamsBlob.h"
#include <string.h>
#include "crc32leee.h"

static void put16(uint8_t *p,uint16_t v){p[0]=(uint8_t)v;p[1]=(uint8_t)(v>>8);}
static void put32(uint8_t *p,uint32_t v){p[0]=(uint8_t)v;p[1]=(uint8_t)(v>>8);p[2]=(uint8_t)(v>>16);p[3]=(uint8_t)(v>>24);}
static uint16_t get16(const uint8_t *p){return (uint16_t)((uint16_t)p[0]|((uint16_t)p[1]<<8));}
static uint32_t get32(const uint8_t *p){return (uint32_t)p[0]|((uint32_t)p[1]<<8)|((uint32_t)p[2]<<16)|((uint32_t)p[3]<<24);}
void core4VisionParamsBlobEncode(const core4VisionParamsStruct *p,core4VisionParamsFlashBlob *b){if(!p||!b)return;memset(b,0,sizeof(*b));put32(&b->bytes[0],CORE4_VISION_PARAMS_FLASH_MAGIC);put32(&b->bytes[4],CORE4_VISION_PARAMS_FLASH_VERSION);put32(&b->bytes[8],CORE4_VISION_PARAMS_BLOB_BYTES);put16(&b->bytes[12],p->exposure);put32(&b->bytes[14],p->hudPeriodMs);put32(&b->bytes[18],p->previewPeriodMs);put32(&b->bytes[22],p->statsPeriodMs);put16(&b->bytes[26],p->binaryLow);put16(&b->bytes[28],p->binaryHigh);put16(&b->bytes[30],p->legacyCannyLow);put16(&b->bytes[32],p->legacyCannyHigh);put32(&b->bytes[34],crc32leeeCalc(b->bytes,CORE4_VISION_PARAMS_BLOB_CRC_BYTES));}
core4VisionParamsBlobStatus core4VisionParamsBlobDecode(const core4VisionParamsFlashBlob *b,core4VisionParamsStruct *p){if(!b||!p)return CORE4_VISION_PARAMS_BLOB_BAD_LENGTH;if(get32(&b->bytes[0])!=CORE4_VISION_PARAMS_FLASH_MAGIC)return CORE4_VISION_PARAMS_BLOB_BAD_MAGIC;if(get32(&b->bytes[4])!=CORE4_VISION_PARAMS_FLASH_VERSION)return CORE4_VISION_PARAMS_BLOB_UNSUPPORTED_VERSION;if(get32(&b->bytes[8])!=CORE4_VISION_PARAMS_BLOB_BYTES)return CORE4_VISION_PARAMS_BLOB_BAD_LENGTH;if(get32(&b->bytes[34])!=crc32leeeCalc(b->bytes,CORE4_VISION_PARAMS_BLOB_CRC_BYTES))return CORE4_VISION_PARAMS_BLOB_BAD_CRC;memset(p,0,sizeof(*p));p->exposure=get16(&b->bytes[12]);p->hudPeriodMs=get32(&b->bytes[14]);p->previewPeriodMs=get32(&b->bytes[18]);p->statsPeriodMs=get32(&b->bytes[22]);p->binaryLow=get16(&b->bytes[26]);p->binaryHigh=get16(&b->bytes[28]);p->legacyCannyLow=get16(&b->bytes[30]);p->legacyCannyHigh=get16(&b->bytes[32]);return CORE4_VISION_PARAMS_BLOB_VALID;}
```

core4_cyt4bb7/project/code/config/core4VisionParamsBlob.h

SHA-256

:

3ac3353748317af3a53c2d5ed5a88bbb80a574cea78d9b6944aab5a8e7898ea3

代码 (c)

```
#ifndef CORE4_VISION_PARAMS_BLOB_H
#define CORE4_VISION_PARAMS_BLOB_H

#include <stdint.h>
#include "core4VisionParams.h"

#define CORE4_VISION_PARAMS_FLASH_MAGIC (0x43345650u)
#define CORE4_VISION_PARAMS_FLASH_VERSION (2u)
#define CORE4_VISION_PARAMS_PARAM_BYTES (22u)
#define CORE4_VISION_PARAMS_BLOB_CRC_BYTES (34u)
#define CORE4_VISION_PARAMS_BLOB_BYTES (38u)

typedef struct { uint8_t bytes[CORE4_VISION_PARAMS_BLOB_BYTES]; } core4VisionParamsFlashBlob;
typedef enum { CORE4_VISION_PARAMS_BLOB_VALID=0, CORE4_VISION_PARAMS_BLOB_BAD_MAGIC, CORE4_VISION_PARAMS_BLOB_BAD_LENGTH, CORE4_VISION_PARAMS_BLOB_BAD_CRC, CORE4_VISION_PARAMS_BLOB_UNSUPPORTED_VERSION } core4VisionParamsBlobStatus;
void core4VisionParamsBlobEncode(const core4VisionParamsStruct *params, core4VisionParamsFlashBlob *blob);
core4VisionParamsBlobStatus core4VisionParamsBlobDecode(const core4VisionParamsFlashBlob *blob, core4VisionParamsStruct *params);
#endif
```

core4_cyt4bb7/project/code/config/core4VisionParamsPersistence.c

SHA-256

:

e1799df6e3abe8020db618ee74904514eafd04e5c1735d21caf4d847ac667883

代码 (c)

```
#include "core4VisionParamsPersistence.h"

#include <stddef.h>
#include <string.h>

#include "crc32leee.h"
#include "zf_driver_flash.h"

#define CORE4_SLOT_STORE_VERSION (2u)
#define CORE4_SLOT_RECORD_MAGIC (0x34534C54u) /* 4SLT */
#define CORE4_SLOT_SELECTOR_MAGIC (0x3453454Cu) /* 4SEL */
#define CORE4_SLOT_SELECTOR_PAGE_A (10u)
#define CORE4_SLOT_SELECTOR_PAGE_B (11u)
#define CORE4_SLOT_DATA_PAGE_FIRST (0u)
```

```

#define CORE4_SLOT_DATA_PAGE_LAST    (CORE4_VISION_PARAMS_SLOT_COUNT * 2u - 1u)
#define CORE4_COORDINATOR_PAGE_A     (12u)
#define CORE4_COORDINATOR_PAGE_B     (13u)
#define CORE4_SLOT_VALID_RECORD       (1u)
#define CORE4_SLOT_VALID_LEGACY       (2u)
#define CORE4_SLOT_MAX_WORDS          (32u)
#define CORE4_FLASH_PAGE_WORDS        (FLASH_PAGE_SIZE / sizeof(uint32_t))
#define CORE4_SELECTOR_WORDS          ((sizeof(core4SlotSelector) + 3u) / 4u)
#define CORE4_SELECTOR_RECORDS        (CORE4_FLASH_PAGE_WORDS / CORE4_SELECTOR_WORDS)
#define CORE4_RUNTIME_SELECTOR_MAGIC  (0x34544E52u) /* RNT4 */
#define CORE4_RUNTIME_SELECTOR_VERSION (1u)
#define CORE4_RUNTIME_DATA_PAGE_A     (14u)
#define CORE4_RUNTIME_DATA_PAGE_B     (15u)
#define CORE4_RUNTIME_SELECTOR_PAGE_A (16u)
#define CORE4_RUNTIME_SELECTOR_PAGE_B (17u)
#define CORE4_RUNTIME_SELECTOR_BYTES   (64u)
#define CORE4_RUNTIME_SELECTOR_WORDS   (CORE4_RUNTIME_SELECTOR_BYTES / 4u)
#define CORE4_RUNTIME_SELECTOR_RECORDS \
    (CORE4_FLASH_PAGE_WORDS / CORE4_RUNTIME_SELECTOR_WORDS)

```

```

typedef struct
{
    uint32_t magic;
    uint32_t version;
    uint32_t slot;
    uint32_t generation;
    uint32_t transactionId;
    uint32_t payloadBytes;
    core4VisionParamsFlashBlob payload;
    uint32_t crc;
} core4SlotRecord;

```

```

typedef struct
{
    uint32_t magic;
    uint32_t version;
    uint32_t generation;
    uint32_t activeSlot;
    uint32_t bank[CORE4_VISION_PARAMS_SLOT_COUNT];
    uint32_t valid[CORE4_VISION_PARAMS_SLOT_COUNT];
    uint32_t slotGeneration[CORE4_VISION_PARAMS_SLOT_COUNT];
    uint32_t transactionId[CORE4_VISION_PARAMS_SLOT_COUNT];
    uint32_t pendingTransactionId;
    uint32_t rollbackActiveSlot;
    uint32_t rollbackSlot;
    uint32_t rollbackBank;
    uint32_t rollbackValid;
    uint32_t rollbackGeneration;
    uint32_t rollbackTransactionId;
    uint32_t crc;
} core4SlotSelector;

```

```

typedef struct
{
    uint32_t magic;
    uint32_t version;
    uint32_t recordBytes;
    uint32_t sequence;
    uint8_t bank;
    uint8_t valid;
    uint8_t pending;
    uint8_t sourceSlot;
    uint32_t generation;
    uint32_t blobCrc;
    uint32_t transactionId;
    uint8_t rollbackBank;
    uint8_t rollbackValid;
    uint8_t rollbackSourceSlot;
    uint8_t reserved0;
    uint32_t rollbackGeneration;
    uint32_t rollbackBlobCrc;
    uint32_t rollbackTransactionId;
    uint32_t reserved[3u];
    uint32_t crc;
} core4RuntimeSelector;

```

```

typedef enum
{
    CORE4_FLASH_RANGE_BLANK = 0,

```

```

    CORE4_FLASH_RANGE_PROGRAMMED,
    CORE4_FLASH_RANGE_IO_FAILED
} core4FlashRangeState;

typedef enum
{
    CORE4_SELECTOR_NONE = 0,
    CORE4_SELECTOR_VALID,
    CORE4_SELECTOR_CORRUPT,
    CORE4_SELECTOR_REPAIR_LAST_GOOD,
    CORE4_SELECTOR_REBUILD_FROM_DATA
} core4SelectorLoadState;

static uint32_t g_flashWords[CORE4_SLOT_MAX_WORDS];
static uint8_t g_flashReady;
static uint8_t g_selectorReady;
static uint8_t g_selectorPage;
static uint8_t g_selectorRecord;
static uint8_t g_selectorUncertain;
static uint8_t g_selectorUncertainHadDurable;
static core4SlotSelector g_selector;
static core4RuntimeSelector g_runtimeSelector;
static uint8_t g_runtimeSelectorReady;
static uint8_t g_runtimeSelectorPage;
static uint8_t g_runtimeSelectorRecord;
static uint8_t g_runtimeSelectorUncertain;
static uint8_t g_runtimeSelectorUncertainHadDurable;
static core4VisionParamsPersistenceStatus g_status;
static struct
{
    core4SlotSelector before;
    uint32_t transactionId;
    uint32_t generation;
    uint8_t slot;
    uint8_t bank;
    uint8_t valid;
    uint8_t committed;
} g_prepared;
static struct
{
    uint32_t transactionId;
    uint32_t generation;
    uint32_t blobCrc;
    uint8_t sourceSlot;
    uint8_t bank;
    uint8_t valid;
    uint8_t committed;
} g_runtimePrepared;

_Static_assert(FLASH_PAGE_NUM >= 18u,
    "Core4 parameter slots exceed Work Flash");
_Static_assert(sizeof(core4SlotRecord) <= FLASH_PAGE_SIZE,
    "Core4 slot record must fit one page");
_Static_assert(sizeof(core4SlotSelector) <= FLASH_PAGE_SIZE,
    "Core4 slot selector must fit one page");
_Static_assert(((sizeof(core4SlotSelector) + 3u) / 4u) <= CORE4_SLOT_MAX_WORDS,
    "Core4 slot scratch buffer too small");
_Static_assert(sizeof(core4RuntimeSelector) == CORE4_RUNTIME_SELECTOR_BYTES,
    "Core4 runtime selector size drift");
_Static_assert(CORE4_SLOT_DATA_PAGE_FIRST <= CORE4_SLOT_DATA_PAGE_LAST &&
    CORE4_SLOT_DATA_PAGE_LAST < CORE4_SLOT_SELECTOR_PAGE_A,
    "Core4 slot data overlaps selector journal");
_Static_assert(CORE4_SLOT_SELECTOR_PAGE_A + 1u ==
    CORE4_SLOT_SELECTOR_PAGE_B &&
    CORE4_SLOT_SELECTOR_PAGE_B < CORE4_COORDINATOR_PAGE_A,
    "Core4 slot selector overlaps coordinator reserve");
_Static_assert(CORE4_COORDINATOR_PAGE_A + 1u == CORE4_COORDINATOR_PAGE_B &&
    CORE4_COORDINATOR_PAGE_B < CORE4_RUNTIME_DATA_PAGE_A,
    "Core4 coordinator reserve overlaps runtime data");
_Static_assert(CORE4_RUNTIME_DATA_PAGE_A + 1u ==
    CORE4_RUNTIME_DATA_PAGE_B &&
    CORE4_RUNTIME_DATA_PAGE_B < CORE4_RUNTIME_SELECTOR_PAGE_A &&
    CORE4_RUNTIME_SELECTOR_PAGE_A + 1u ==
    CORE4_RUNTIME_SELECTOR_PAGE_B &&
    CORE4_RUNTIME_SELECTOR_PAGE_B < FLASH_PAGE_NUM,
    "Core4 runtime pages overlap or exceed Work Flash");
_Static_assert(CORE4_SELECTOR_RECORDS > 0u &&
    CORE4_RUNTIME_SELECTOR_RECORDS > 0u,
    "Core4 selector records must fit one Work Flash page");

```

```

static void saturatingIncrement(uint32_t *value)
{
    if (*value < UINT32_MAX) (*value)++;
}

static void ensureFlashReady(void)
{
    if (g_flashReady == 0u)
    {
        flash_init();
        g_flashReady = 1u;
    }
}

static uintptr_t flashPageAddress(uint32_t page)
{
    return (uintptr_t) FLASH_BASE_ADDR + (uintptr_t) page * FLASH_PAGE_SIZE;
}

static const uint32_t *flashWordAddress(uint32_t page, uint32_t wordIndex)
{
    return (const uint32_t *) (uintptr_t)
        (flashPageAddress(page) + wordIndex * sizeof(uint32_t));
}

static core4FlashRangeState blankCheckRange(uint32_t page,
uint32_t firstWord,
uint32_t wordCount)
{
    cy_stc_flash_blankcheck_config_t config = {0};
    cy_en_flashdrv_status_t status;

    if ((page >= FLASH_PAGE_NUM) || (wordCount == 0u) ||
        (firstWord >= CORE4_FLASH_PAGE_WORDS) ||
        (wordCount > (CORE4_FLASH_PAGE_WORDS - firstWord)))
    {
        return CORE4_FLASH_RANGE_IO_FAILED;
    }
    config.addrToBeChecked = flashWordAddress(page, firstWord);
    config.numOfWordsToBeChecked = wordCount;
    status = Cy_Flash_BlankCheck(NULL, &config, CY_FLASH_DRIVER_BLOCKING);
    if (status == CY_FLASH_DRV_SUCCESS) return CORE4_FLASH_RANGE_BLANK;
    if (status == CY_FLASH_DRV_FLASH_NOT_ERASED) return CORE4_FLASH_RANGE_PROGRAMMED;
    return CORE4_FLASH_RANGE_IO_FAILED;
}

static void invalidateFlashWordCache(const uint32_t *wordAddress)
{
    #if CY_CORE_CM7_0 || CY_CORE_CM7_1
        uintptr_t cacheLine = ((uintptr_t) wordAddress) & ~(uintptr_t) 31u;
        SCB_InvalidateDCache_by_Addr((uint32_t *) cacheLine, 32);
    #else
        (void) wordAddress;
    #endif
}

static uint8_t readFlashWordSafely(uint32_t page,
uint32_t wordIndex,
uint32_t *value)
{
    const uint32_t *address;
    core4FlashRangeState state;

    if ((value == NULL) || (wordIndex >= CORE4_FLASH_PAGE_WORDS)) return 1u;
    state = blankCheckRange(page, wordIndex, 1u);
    if (state == CORE4_FLASH_RANGE_BLANK)
    {
        *value = UINT32_MAX;
        return 0u;
    }
    if (state != CORE4_FLASH_RANGE_PROGRAMMED) return 1u;
    address = flashWordAddress(page, wordIndex);
    invalidateFlashWordCache(address);
    *value = *((const volatile uint32_t *) address);
    return 0u;
}

static uint8_t readPageBytes(uint32_t page, void *data, uint32_t length)

```



```

{
    uint32_t words;
    uint32_t i;

    if ((data == NULL) || (length == 0u)) return 1u;
    words = (length + 3u) / 4u;
    if (words > CORE4_SLOT_MAX_WORDS) return 1u;
    memset(g_flashWords, 0xFF, sizeof(g_flashWords));
    for (i = 0u; i < words; i++)
    {
        if (readFlashWordSafely(page, i, &g_flashWords[i]) != 0u) return 1u;
    }
    if (data != g_flashWords)
    {
        memcpy(data, g_flashWords, length);
    }
    return 0u;
}

static uint8_t erasePage(uint32_t page)
{
    cy_stc_flash_erasesector_config_t config = {0};
    cy_en_flashdrv_status_t status;

    config.blocking = CY_FLASH_ERASESECTOR_BLOCKING;
    config.intrMask = CY_FLASH_ERASESECTOR_NOT_SET_INTR_MASK;
    config.Addr = flashWordAddress(page, 0u);
    status = Cy_Flash_EraseSector(NULL, &config, CY_FLASH_DRIVER_BLOCKING);
    if (status != CY_FLASH_DRV_SUCCESS) return 1u;
    return (uint8_t) (blankCheckRange(page, 0u, CORE4_FLASH_PAGE_WORDS) ==
        CORE4_FLASH_RANGE_BLANK ? 0u : 1u);
}

static uint8_t programFlashWord(uint32_t page,
    uint32_t wordIndex,
    const uint32_t *value)
{
    cy_stc_flash_programrow_config_t config = {0};
    cy_en_flashdrv_status_t status;

    if ((value == NULL) || (wordIndex >= CORE4_FLASH_PAGE_WORDS)) return 1u;
    config.blocking = CY_FLASH_PROGRAMROW_BLOCKING;
    config.skipBC = CY_FLASH_PROGRAMROW_BLANK_CHECK;
    config.dataSize = CY_FLASH_PROGRAMROW_DATA_SIZE_32BIT;
    config.dataLoc = CY_FLASH_PROGRAMROW_DATA_LOCATION_SRAM;
    config.intrMask = CY_FLASH_PROGRAMROW_NOT_SET_INTR_MASK;
    config.destAddr = flashWordAddress(page, wordIndex);
    config.dataAddr = value;

#ifdef CY_CORE_CM7_0 || CY_CORE_CM7_1
    {
        uintptr_t start = ((uintptr_t) value) & ~(uintptr_t) 31u;
        uintptr_t end = (((uintptr_t) value + sizeof(*value) + 31u) &
            ~(uintptr_t) 31u);
        SCB_CleanDCache_by_Addr((volatile void *) start,
            (int32_t) (end - start));
    }
#endif
    status = Cy_Flash_ProgramRow(NULL, &config, CY_FLASH_DRIVER_BLOCKING);
    return (uint8_t) (status == CY_FLASH_DRV_SUCCESS ? 0u : 1u);
}

static uint8_t writePageAtomic(uint32_t page, const void *data, uint32_t length)
{
    {
        uint32_t words;
        uint32_t readback;
        uint32_t i;

        if ((data == NULL) || (length == 0u)) return 1u;
        words = (length + 3u) / 4u;
        if (words > CORE4_SLOT_MAX_WORDS) return 1u;
        if (erasePage(page) != 0u) return 1u;
        memset(g_flashWords, 0xFF, sizeof(g_flashWords));
        memcpy(g_flashWords, data, length);
        for (i = 1u; i < words; i++)
        {
            if ((programFlashWord(page, i, &g_flashWords[i]) != 0u) ||
                (readFlashWordSafely(page, i, &readback) != 0u) ||
                (readback != g_flashWords[i])) return 1u;
        }
    }
}

```

```

    if (programFlashWord(page, 0u, &g_flashWords[0]) != 0u) return 1u;
    memset(g_flashWords, 0xFF, sizeof(g_flashWords));
    if (readPageBytes(page, g_flashWords, length) != 0u) return 1u;
    return (uint8_t) (memcmp(data, g_flashWords, length) == 0 ? 0u : 1u);
}

static uint8_t selectorSlotStateValid(uint32_t slot, uint32_t bank,
    uint32_t valid, uint32_t generation, uint32_t transactionId)
{
    if ((slot >= CORE4_VISION_PARAMS_SLOT_COUNT) || (bank > 1u) ||
        (valid > CORE4_SLOT_VALID_LEGACY)) return 0u;
    if (valid == 0u)
        return (uint8_t) ((bank == 0u) && (generation == 0u) &&
            (transactionId == 0u));
    if (generation == 0u) return 0u;
    if (valid == CORE4_SLOT_VALID_LEGACY)
        return (uint8_t) ((slot == 0u) && (bank == 0u) &&
            (generation == 1u) &&
            (transactionId == 0u));
    return (uint8_t) (transactionId != UINT32_MAX);
}

static uint8_t selectorValid(const core4SlotSelector *selector)
{
    uint32_t i;
    uint32_t crc;
    uint8_t anyValid = 0u;

    if ((selector->magic != CORE4_SLOT_SELECTOR_MAGIC) ||
        (selector->version != CORE4_SLOT_STORE_VERSION) ||
        (selector->generation == 0u) ||
        (selector->activeSlot > CORE4_VISION_PARAMS_SLOT_COUNT)) return 0u;
    for (i = 0u; i < CORE4_VISION_PARAMS_SLOT_COUNT; i++)
    {
        if (selectorSlotStateValid(i, selector->bank[i],
            selector->valid[i], selector->slotGeneration[i],
            selector->transactionId[i]) == 0u) return 0u;
        if (selector->valid[i] != 0u) anyValid = 1u;
    }
    if ((anyValid == 0u &&
        selector->activeSlot != CORE4_VISION_PARAMS_SLOT_COUNT) ||
        (anyValid != 0u &&
        (selector->activeSlot >= CORE4_VISION_PARAMS_SLOT_COUNT ||
        selector->valid[selector->activeSlot] == 0u))) return 0u;
    if (selector->pendingTransactionId != 0u)
    {
        uint32_t slot = selector->rollbackSlot;
        if ((slot >= CORE4_VISION_PARAMS_SLOT_COUNT) ||
            (selector->rollbackActiveSlot >
                CORE4_VISION_PARAMS_SLOT_COUNT) ||
            (selector->valid[slot] == 0u) ||
            (selector->slotGeneration[slot] == 0u) ||
            (selectorSlotStateValid(slot, selector->rollbackBank,
                selector->rollbackValid, selector->rollbackGeneration,
                selector->rollbackTransactionId) == 0u)) return 0u;
        if (selector->rollbackActiveSlot == slot)
        {
            if (selector->rollbackValid == 0u) return 0u;
        }
        else if ((selector->rollbackActiveSlot <
            CORE4_VISION_PARAMS_SLOT_COUNT) &&
            (selector->valid[selector->rollbackActiveSlot] == 0u))
            return 0u;
        if (selector->pendingTransactionId == UINT32_MAX)
        {
            if ((selector->activeSlot != slot) ||
                (selector->rollbackBank != selector->bank[slot]) ||
                (selector->rollbackValid != selector->valid[slot]) ||
                (selector->rollbackGeneration !=
                    selector->slotGeneration[slot]) ||
                (selector->rollbackTransactionId !=
                    selector->transactionId[slot])) return 0u;
        }
        else if (((selector->activeSlot !=
            selector->rollbackActiveSlot) &&
            (selector->activeSlot != slot)) ||
            (selector->valid[slot] != CORE4_SLOT_VALID_RECORD) ||
            (selector->transactionId[slot] !=
                selector->pendingTransactionId)) return 0u;
    }
}

```

```

else
{
    uint32_t expectedGeneration =
        selector->rollbackGeneration + 1u;
    uint32_t expectedBank = 0u;
    if (expectedGeneration == 0u) expectedGeneration = 1u;
    if (selector->rollbackValid == CORE4_SLOT_VALID_RECORD)
        expectedBank = selector->rollbackBank ^ 1u;
    else if (selector->rollbackValid == CORE4_SLOT_VALID_LEGACY)
        expectedBank = 1u;
    if ((selector->bank[slot] != expectedBank) ||
        (selector->slotGeneration[slot] != expectedGeneration))
        return 0u;
}
}
else if ((selector->rollbackActiveSlot != 0u) ||
        (selector->rollbackSlot != 0u) ||
        (selector->rollbackBank != 0u) ||
        (selector->rollbackValid != 0u) ||
        (selector->rollbackGeneration != 0u) ||
        (selector->rollbackTransactionId != 0u))
{
    return 0u;
}
}
crc = crc32leeeCalc((const uint8_t *) selector,
    (uint32_t) offsetof(core4SlotSelector, crc));
return (uint8_t) (crc == selector->crc ? 1u : 0u);
}

static uint8_t selectorNewer(uint32_t candidate, uint32_t current)
{
    return (uint8_t) ((int32_t) (candidate - current) > 0 ? 1u : 0u);
}

static core4SelectorLoadState loadSelector(void)
{
    const uint32_t pages[2u] = {CORE4_SLOT_SELECTOR_PAGE_A, CORE4_SLOT_SELECTOR_PAGE_B};
    uint8_t found = 0u;
    uint8_t blankFound = 0u;
    uint8_t programmedInvalid = 0u;
    uint8_t committedInvalid = 0u;
    uint32_t p, record, word;
    for (p = 0u; p < 2u; p++)
    {
        for (record = 0u; record < CORE4_SELECTOR_RECORDS; record++)
        {
            core4SlotSelector candidate;
            uint32_t *words = (uint32_t *) (void *) &candidate;
            core4FlashRangeState state = blankCheckRange(pages[p],
                record * CORE4_SELECTOR_WORDS, CORE4_SELECTOR_WORDS);
            if (state == CORE4_FLASH_RANGE_IO_FAILED) return CORE4_SELECTOR_CORRUPT;
            if (state == CORE4_FLASH_RANGE_BLANK)
            {
                blankFound = 1u;
                continue;
            }
            memset(&candidate, 0xFF, sizeof(candidate));
            for (word = 0u; word < CORE4_SELECTOR_WORDS; word++)
                if (readFlashWordSafely(pages[p], record * CORE4_SELECTOR_WORDS + word,
                    &words[word]) != 0u) return CORE4_SELECTOR_CORRUPT;
            if (selectorValid(&candidate) == 0u)
            {
                programmedInvalid = 1u;
                if (candidate.magic == CORE4_SLOT_SELECTOR_MAGIC)
                    committedInvalid = 1u;
            }
            else if (!found ||
                selectorNewer(candidate.generation, g_selector.generation))
            {
                g_selector = candidate;
                g_selectorPage = (uint8_t) pages[p];
                g_selectorRecord = (uint8_t) record;
                found = 1u;
            }
        }
    }
}
if (committedInvalid != 0u && found != 0u)
{
    g_selectorReady = 1u;
}

```

```

        return CORE4_SELECTOR_REPAIR_LAST_GOOD;
    }
    g_selectorReady = found;
    if (found) return CORE4_SELECTOR_VALID;
    if (programmedInvalid) return CORE4_SELECTOR_REBUILD_FROM_DATA;
    return blankFound ? CORE4_SELECTOR_NONE : CORE4_SELECTOR_CORRUPT;
}

static uint8_t resolveSelectorUncertainty(void)
{
    core4SelectorLoadState state;
    if (g_selectorUncertain == 0u) return 0u;
    state = loadSelector();
    if ((state != CORE4_SELECTOR_VALID) &&
        (state != CORE4_SELECTOR_NONE)) return 1u;
    if (state == CORE4_SELECTOR_NONE)
    {
        if (g_selectorUncertainHadDurable != 0u) return 1u;
        memset(&g_selector, 0, sizeof(g_selector));
        g_selectorReady = 0u;
        g_selectorPage = 0u;
        g_selectorRecord = 0u;
    }
    g_selectorUncertain = 0u;
    g_selectorUncertainHadDurable = 0u;
    return 0u;
}

static uint8_t reconcileSelectorWrite(const core4SlotSelector *expected)
{
    g_selectorUncertain = 1u;
    if (resolveSelectorUncertainty() != 0u) return 1u;
    return (uint8_t)((g_selectorReady != 0u) &&
        (memcmp(&g_selector, expected, sizeof(*expected)) == 0) ? 0u : 1u);
}

static uint8_t writeSelector(const core4SlotSelector *candidate)
{
    core4SlotSelector stored = *candidate;
    core4SlotSelector verified;
    const uint32_t *words;
    uint32_t page = CORE4_SLOT_SELECTOR_PAGE_A;
    uint32_t record = 0u;
    uint32_t word;

    if (g_selectorUncertain != 0u)
    {
        if (resolveSelectorUncertainty() != 0u) return 1u;
        return 1u;
    }

    stored.magic = CORE4_SLOT_SELECTOR_MAGIC;
    stored.version = CORE4_SLOT_STORE_VERSION;
    stored.crc = crc32IeeeCalc((const uint8_t *) &stored,
        (uint32_t) offsetof(core4SlotSelector, crc));
    g_selectorUncertainHadDurable = g_selectorReady;
    if (g_selectorReady)
    {
        page = g_selectorPage;
        for (record = (uint32_t) g_selectorRecord + 1u;
            record < CORE4_SELECTOR_RECORDS; record++)
            if (blankCheckRange(page, record * CORE4_SELECTOR_WORDS,
                CORE4_SELECTOR_WORDS) == CORE4_FLASH_RANGE_BLANK) break;
        if (record >= CORE4_SELECTOR_RECORDS)
        {
            page = (page == CORE4_SLOT_SELECTOR_PAGE_A) ?
                CORE4_SLOT_SELECTOR_PAGE_B : CORE4_SLOT_SELECTOR_PAGE_A;
            if (erasePage(page) != 0u) return 1u;
            record = 0u;
        }
    }
    else if (blankCheckRange(page, 0u, CORE4_SELECTOR_WORDS) != CORE4_FLASH_RANGE_BLANK &&
        erasePage(page) != 0u) return 1u;
    words = (const uint32_t *) (const void *) &stored;
    for (word = 1u; word < CORE4_SELECTOR_WORDS; word++)
        if (programFlashWord(page, record * CORE4_SELECTOR_WORDS + word,
            &words[word]) != 0u) return 1u;
    if (programFlashWord(page, record * CORE4_SELECTOR_WORDS, &words[0]) != 0u)
        return reconcileSelectorWrite(&stored);
}

```

```

{
    uint32_t *verifyWords = (uint32_t *) (void *) &verified;
    for (word = 0u; word < CORE4_SELECTOR_WORDS; word++)
        if (readFlashWordSafely(page, record * CORE4_SELECTOR_WORDS + word,
            &verifyWords[word]) != 0u) return reconcileSelectorWrite(&stored);
}

if (!selectorValid(&verified) || memcmp(&verified, &stored, sizeof(stored)) != 0)
    return reconcileSelectorWrite(&stored);
g_selector = stored;
g_selectorPage = (uint8_t) page;
g_selectorRecord = (uint8_t) record;
g_selectorReady = 1u;
g_selectorUncertain = 0u;
g_selectorUncertainHadDurable = 0u;
return 0u;
}

static uint8_t runtimeSelectorValid(const core4RuntimeSelector *selector)
{
    uint32_t crc;
    uint32_t expectedGeneration;
    if ((selector->magic != CORE4_RUNTIME_SELECTOR_MAGIC) ||
        (selector->version != CORE4_RUNTIME_SELECTOR_VERSION) ||
        (selector->recordBytes != CORE4_RUNTIME_SELECTOR_BYTES) ||
        (selector->sequence == 0u) ||
        (selector->bank > 1u) || (selector->valid != 1u) ||
        (selector->pending > 1u) ||
        (selector->sourceSlot > CORE4_VISION_PARAMS_SLOT_COUNT) ||
        (selector->rollbackBank > 1u) ||
        (selector->rollbackValid > 1u) ||
        (selector->rollbackSourceSlot > CORE4_VISION_PARAMS_SLOT_COUNT) ||
        (selector->generation == 0u) ||
        (selector->reserved0 != 0u) || (selector->reserved[0u] != 0u) ||
        (selector->reserved[1u] != 0u) || (selector->reserved[2u] != 0u))
        return 0u;
    if (selector->pending == 0u)
    {
        if ((selector->rollbackBank != 0u) ||
            (selector->rollbackValid != 0u) ||
            (selector->rollbackSourceSlot != 0u) ||
            (selector->rollbackGeneration != 0u) ||
            (selector->rollbackBlobCrc != 0u) ||
            (selector->rollbackTransactionId != 0u)) return 0u;
    }
    else
    {
        expectedGeneration = selector->rollbackGeneration + 1u;
        if (expectedGeneration == 0u) expectedGeneration = 1u;
        if ((selector->transactionId == 0u) ||
            (selector->rollbackValid != 1u) ||
            (selector->rollbackGeneration == 0u) ||
            (selector->bank == selector->rollbackBank) ||
            (selector->generation != expectedGeneration)) return 0u;
    }
    crc = crc32IeeeCalc((const uint8_t *) selector,
        (uint32_t) offsetof(core4RuntimeSelector, crc));
    return (uint8_t) (crc == selector->crc ? 1u : 0u);
}

static core4SelectorLoadState loadRuntimeSelector(void)
{
    const uint32_t pages[2u] = {CORE4_RUNTIME_SELECTOR_PAGE_A,
        CORE4_RUNTIME_SELECTOR_PAGE_B};
    uint8_t found = 0u;
    uint8_t blankFound = 0u;
    uint8_t programmedInvalid = 0u;
    uint8_t committedInvalid = 0u;
    uint32_t p, record, word;
    for (p = 0u; p < 2u; p++)
    {
        for (record = 0u; record < CORE4_RUNTIME_SELECTOR_RECORDS; record++)
        {
            core4RuntimeSelector candidate;
            uint32_t *words = (uint32_t *) (void *) &candidate;
            core4FlashRangeState state = blankCheckRange(pages[p],
                record * CORE4_RUNTIME_SELECTOR_WORDS,
                CORE4_RUNTIME_SELECTOR_WORDS);
            if (state == CORE4_FLASH_RANGE_IO_FAILED) return CORE4_SELECTOR_CORRUPT;
            if (state == CORE4_FLASH_RANGE_BLANK)

```

```

    {
        blankFound = 1u;
        continue;
    }
    memset(&candidate, 0xFF, sizeof(candidate));
    for (word = 0u; word < CORE4_RUNTIME_SELECTOR_WORDS; word++)
        if (readFlashWordSafely(pages[p],
            record * CORE4_RUNTIME_SELECTOR_WORDS + word,
            &words[word]) != 0u) return CORE4_SELECTOR_CORRUPT;
    if (runtimeSelectorValid(&candidate) == 0u)
    {
        programmedInvalid = 1u;
        if (candidate.magic == CORE4_RUNTIME_SELECTOR_MAGIC)
            committedInvalid = 1u;
    }
    else if (!found || selectorNewer(candidate.sequence,
        g_runtimeSelector.sequence))
    {
        g_runtimeSelector = candidate;
        g_runtimeSelectorPage = (uint8_t) pages[p];
        g_runtimeSelectorRecord = (uint8_t) record;
        found = 1u;
    }
}
}
if (committedInvalid != 0u)
{
    g_runtimeSelectorReady = 0u;
    return CORE4_SELECTOR_CORRUPT;
}
g_runtimeSelectorReady = found;
if (found) return CORE4_SELECTOR_VALID;
if (programmedInvalid) return CORE4_SELECTOR_CORRUPT;
return blankFound ? CORE4_SELECTOR_NONE : CORE4_SELECTOR_CORRUPT;
}

static uint8_t resolveRuntimeSelectorUncertainty(void)
{
    core4SelectorLoadState state;
    if (g_runtimeSelectorUncertain == 0u) return 0u;
    state = loadRuntimeSelector();
    if (state == CORE4_SELECTOR_CORRUPT) return 1u;
    if (state == CORE4_SELECTOR_NONE)
    {
        if (g_runtimeSelectorUncertainHadDurable != 0u) return 1u;
        memset(&g_runtimeSelector, 0, sizeof(g_runtimeSelector));
        g_runtimeSelectorReady = 0u;
        g_runtimeSelectorPage = 0u;
        g_runtimeSelectorRecord = 0u;
    }
    g_runtimeSelectorUncertain = 0u;
    g_runtimeSelectorUncertainHadDurable = 0u;
    return 0u;
}

static uint8_t reconcileRuntimeSelectorWrite(
    const core4RuntimeSelector *expected)
{
    g_runtimeSelectorUncertain = 1u;
    if (resolveRuntimeSelectorUncertainty() != 0u) return 1u;
    return (uint8_t) ((g_runtimeSelectorReady != 0u) &&
        (memcmp(&g_runtimeSelector, expected, sizeof(*expected)) == 0) ? 0u : 1u);
}

static uint8_t writeRuntimeSelector(const core4RuntimeSelector *candidate)
{
    core4RuntimeSelector stored = *candidate;
    core4RuntimeSelector verified;
    const uint32_t *words;
    uint32_t page = CORE4_RUNTIME_SELECTOR_PAGE_A;
    uint32_t record = 0u;
    uint32_t word;

    if (g_runtimeSelectorUncertain != 0u)
    {
        if (resolveRuntimeSelectorUncertainty() != 0u) return 1u;
        return 1u;
    }
    stored.magic = CORE4_RUNTIME_SELECTOR_MAGIC;

```

```

stored.version = CORE4_RUNTIME_SELECTOR_VERSION;
stored.recordBytes = CORE4_RUNTIME_SELECTOR_BYTES;
stored.sequence = g_runtimeSelectorReady ?
    g_runtimeSelector.sequence + 1u : 1u;
if (stored.sequence == 0u) stored.sequence = 1u;
stored.crc = crc32IeeeCalc((const uint8_t *) &stored,
    (uint32_t) offsetof(core4RuntimeSelector, crc));
g_runtimeSelectorUncertainHadDurable = g_runtimeSelectorReady;
if (g_runtimeSelectorReady)
{
    page = g_runtimeSelectorPage;
    for (record = (uint32_t) g_runtimeSelectorRecord + 1u;
        record < CORE4_RUNTIME_SELECTOR_RECORDS; record++)
        if (blankCheckRange(page,
            record * CORE4_RUNTIME_SELECTOR_WORDS,
            CORE4_RUNTIME_SELECTOR_WORDS) == CORE4_FLASH_RANGE_BLANK) break;
    if (record >= CORE4_RUNTIME_SELECTOR_RECORDS)
    {
        page = (page == CORE4_RUNTIME_SELECTOR_PAGE_A) ?
            CORE4_RUNTIME_SELECTOR_PAGE_B : CORE4_RUNTIME_SELECTOR_PAGE_A;
        if (erasePage(page) != 0u) return 1u;
        record = 0u;
    }
}
else if (blankCheckRange(page, 0u, CORE4_RUNTIME_SELECTOR_WORDS) !=
    CORE4_FLASH_RANGE_BLANK && erasePage(page) != 0u) return 1u;
words = (const uint32_t *) (const void *) &stored;
for (word = 1u; word < CORE4_RUNTIME_SELECTOR_WORDS; word++)
    if (programFlashWord(page,
        record * CORE4_RUNTIME_SELECTOR_WORDS + word,
        &words[word]) != 0u) return 1u;
if (programFlashWord(page, record * CORE4_RUNTIME_SELECTOR_WORDS,
    &words[0]) != 0u) return reconcileRuntimeSelectorWrite(&stored);
{
    uint32_t *verifyWords = (uint32_t *) (void *) &verified;
    for (word = 0u; word < CORE4_RUNTIME_SELECTOR_WORDS; word++)
        if (readFlashWordSafely(page,
            record * CORE4_RUNTIME_SELECTOR_WORDS + word,
            &verifyWords[word]) != 0u)
            return reconcileRuntimeSelectorWrite(&stored);
}
if (!runtimeSelectorValid(&verified) ||
    memcmp(&verified, &stored, sizeof(stored)) != 0)
    return reconcileRuntimeSelectorWrite(&stored);
g_runtimeSelector = stored;
g_runtimeSelectorPage = (uint8_t) page;
g_runtimeSelectorRecord = (uint8_t) record;
g_runtimeSelectorReady = 1u;
g_runtimeSelectorUncertain = 0u;
g_runtimeSelectorUncertainHadDurable = 0u;
return 0u;
}

static uint32_t runtimeDataPage(uint8_t bank)
{
    return bank != 0u ? CORE4_RUNTIME_DATA_PAGE_B :
        CORE4_RUNTIME_DATA_PAGE_A;
}

static uint8_t readRuntimeBank(uint8_t bank, uint32_t blobCrc,
    core4VisionParamsStruct *params)
{
    core4VisionParamsFlashBlob blob;
    if ((bank > 1u) || (params == NULL) ||
        (readPageBytes(runtimeDataPage(bank), &blob, sizeof(blob)) != 0u) ||
        (crc32IeeeCalc(blob.bytes, CORE4_VISION_PARAMS_BLOB_BYTES) != blobCrc) ||
        (core4VisionParamsBlobDecode(&blob, params) !=
            CORE4_VISION_PARAMS_BLOB_VALID)) return 1u;
    core4VisionParamsSanitize(params);
    return 0u;
}

static uint8_t writeRuntimeBank(uint8_t bank,
    const core4VisionParamsStruct *params, uint32_t *blobCrc)
{
    core4VisionParamsFlashBlob blob;
    core4VisionParamsStruct sanitized;
    core4VisionParamsStruct verified;
    if ((bank > 1u) || (params == NULL) || (blobCrc == NULL)) return 1u;

```

```

    sanitized = *params;
    core4VisionParamsSanitize(&sanitized);
    core4VisionParamsBlobEncode(&sanitized, &blob);
    *blobCrc = crc32leeeCalc(blob.bytes, CORE4_VISION_PARAMS_BLOB_BYTES);
    if ((writePageAtomic(runtimeDataPage(bank), &blob, sizeof(blob)) != 0u) ||
        (readRuntimeBank(bank, *blobCrc, &verified) != 0u) ||
        (memcmp(&sanitized, &verified, sizeof(sanitized)) != 0)) return 1u;
    return 0u;
}

static void clearRuntimePrepared(void)
{
    memset(&g_runtimePrepared, 0, sizeof(g_runtimePrepared));
}

static uint8_t committedRuntimeSelector(core4RuntimeSelector *committed)
{
    if ((committed == NULL) || (g_runtimeSelectorReady == 0u)) return 1u;
    *committed = g_runtimeSelector;
    if (committed->pending != 0u)
    {
        if (committed->rollbackValid == 0u) return 1u;
        committed->bank = committed->rollbackBank;
        committed->valid = committed->rollbackValid;
        committed->pending = 0u;
        committed->sourceSlot = committed->rollbackSourceSlot;
        committed->generation = committed->rollbackGeneration;
        committed->blobCrc = committed->rollbackBlobCrc;
        committed->transactionId = committed->rollbackTransactionId;
    }
    return 0u;
}

static core4VisionParamsLoadStatus readRuntimeCommitted(
    core4VisionParamsStruct *params)
{
    core4RuntimeSelector committed;
    if ((params == NULL) || (resolveRuntimeSelectorUncertainty() != 0u) ||
        (committedRuntimeSelector(&committed) != 0u) ||
        (readRuntimeBank(committed.bank, committed.blobCrc, params) != 0u))
        return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
    return CORE4_VISION_PARAMS_LOAD_VALID;
}

static core4VisionParamsSaveStatus prepareRuntimeInternal(
    const core4VisionParamsStruct *params, uint32_t transactionId,
    uint8_t sourceSlot)
{
    uint8_t bank;
    if ((params == NULL) ||
        (sourceSlot > CORE4_VISION_PARAMS_SLOT_COUNT) ||
        (resolveRuntimeSelectorUncertainty() != 0u) ||
        (g_runtimePrepared.valid != 0u) ||
        ((g_runtimeSelectorReady != 0u) &&
         (g_runtimeSelector.pending != 0u)))
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    bank = (g_runtimeSelectorReady != 0u) ?
        (uint8_t) (g_runtimeSelector.bank ^ 1u) : 0u;
    memset(&g_runtimePrepared, 0, sizeof(g_runtimePrepared));
    if (writeRuntimeBank(bank, params, &g_runtimePrepared.blobCrc) != 0u)
        return CORE4_VISION_PARAMS_SAVE_VERIFY_FAILED;
    g_runtimePrepared.transactionId = transactionId;
    g_runtimePrepared.generation = (g_runtimeSelectorReady != 0u) ?
        g_runtimeSelector.generation + 1u : 1u;
    if (g_runtimePrepared.generation == 0u)
        g_runtimePrepared.generation = 1u;
    g_runtimePrepared.sourceSlot = sourceSlot;
    g_runtimePrepared.bank = bank;
    g_runtimePrepared.valid = 1u;
    return CORE4_VISION_PARAMS_SAVE_VERIFIED;
}

static core4VisionParamsSaveStatus commitRuntimeInternal(
    uint32_t transactionId, uint8_t pending)
{
    core4RuntimeSelector next;
    if ((g_runtimePrepared.valid == 0u) ||
        (g_runtimePrepared.committed != 0u) ||
        (g_runtimePrepared.transactionId != transactionId) ||

```



```

        ((pending != 0u) && ((transactionId == 0u) ||
        (g_runtimeSelectorReady == 0u))))
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    memset(&next, 0, sizeof(next));
    if (g_runtimeSelectorReady != 0u) next = g_runtimeSelector;
    if (pending != 0u)
    {
        next.rollbackBank = g_runtimeSelector.bank;
        next.rollbackValid = g_runtimeSelector.valid;
        next.rollbackSourceSlot = g_runtimeSelector.sourceSlot;
        next.rollbackGeneration = g_runtimeSelector.generation;
        next.rollbackBlobCrc = g_runtimeSelector.blobCrc;
        next.rollbackTransactionId = g_runtimeSelector.transactionId;
    }
    else
    {
        next.rollbackBank = 0u;
        next.rollbackValid = 0u;
        next.rollbackSourceSlot = 0u;
        next.rollbackGeneration = 0u;
        next.rollbackBlobCrc = 0u;
        next.rollbackTransactionId = 0u;
    }
    next.bank = g_runtimePrepared.bank;
    next.valid = 1u;
    next.pending = (uint8_t) (pending != 0u);
    next.sourceSlot = g_runtimePrepared.sourceSlot;
    next.generation = g_runtimePrepared.generation;
    next.blobCrc = g_runtimePrepared.blobCrc;
    next.transactionId = transactionId;
    memset(next.reserved, 0, sizeof(next.reserved));
    if (writeRuntimeSelector(&next) != 0u)
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    g_runtimePrepared.committed = 1u;
    return CORE4_VISION_PARAMS_SAVE_VERIFIED;
}

static core4VisionParamsSaveStatus saveRuntimeFinal(
    const core4VisionParamsStruct *params, uint8_t sourceSlot)
{
    core4VisionParamsSaveStatus status = prepareRuntimeInternal(params, 0u,
        sourceSlot);
    if (status == CORE4_VISION_PARAMS_SAVE_VERIFIED)
        status = commitRuntimeInternal(0u, 0u);
    if ((status == CORE4_VISION_PARAMS_SAVE_VERIFIED) ||
        (g_runtimePrepared.committed == 0u)) clearRuntimePrepared();
    return status;
}

static core4VisionParamsSaveStatus abortRuntimeTransaction(
    uint32_t transactionId)
{
    core4RuntimeSelector next;
    if (resolveRuntimeSelectorUncertainty() != 0u)
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    if ((g_runtimeSelectorReady != 0u) &&
        (g_runtimeSelector.pending != 0u))
    {
        if ((g_runtimeSelector.transactionId != transactionId) ||
            (g_runtimeSelector.rollbackValid == 0u))
            return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
        next = g_runtimeSelector;
        next.bank = next.rollbackBank;
        next.valid = next.rollbackValid;
        next.pending = 0u;
        next.sourceSlot = next.rollbackSourceSlot;
        next.generation = next.rollbackGeneration;
        next.blobCrc = next.rollbackBlobCrc;
        next.transactionId = next.rollbackTransactionId;
        next.rollbackBank = 0u;
        next.rollbackValid = 0u;
        next.rollbackSourceSlot = 0u;
        next.rollbackGeneration = 0u;
        next.rollbackBlobCrc = 0u;
        next.rollbackTransactionId = 0u;
        memset(next.reserved, 0, sizeof(next.reserved));
        if (writeRuntimeSelector(&next) != 0u)
            return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    }
}

```

```

else if ((g_runtimePrepared.valid != 0u) &&
         (g_runtimePrepared.transactionId != transactionId))
{
    return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
}
clearRuntimePrepared();
return CORE4_VISION_PARAMS_SAVE_VERIFIED;
}

static core4VisionParamsSaveStatus finishRuntimeTransaction(
    uint32_t transactionId)
{
    core4RuntimeSelector next;
    core4VisionParamsStruct verified;
    if (resolveRuntimeSelectorUncertainty() != 0u)
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    if ((g_runtimeSelectorReady != 0u) &&
        (g_runtimeSelector.pending != 0u))
    {
        if ((g_runtimeSelector.transactionId != transactionId) ||
            (readRuntimeBank(g_runtimeSelector.bank,
                            g_runtimeSelector.blobCrc, &verified) != 0u))
            return CORE4_VISION_PARAMS_SAVE_VERIFY_FAILED;
        next = g_runtimeSelector;
        next.pending = 0u;
        next.rollbackBank = 0u;
        next.rollbackValid = 0u;
        next.rollbackSourceSlot = 0u;
        next.rollbackGeneration = 0u;
        next.rollbackBlobCrc = 0u;
        next.rollbackTransactionId = 0u;
        memset(next.reserved, 0, sizeof(next.reserved));
        if (writeRuntimeSelector(&next) != 0u)
            return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    }
    else if ((g_runtimePrepared.valid != 0u) &&
            (g_runtimePrepared.transactionId != transactionId))
    {
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    }
    clearRuntimePrepared();
    return CORE4_VISION_PARAMS_SAVE_VERIFIED;
}

static core4VisionParamsSaveStatus applyCommittedRuntime(void)
{
    core4VisionParamsStruct params;
    if (readRuntimeCommitted(&params) != CORE4_VISION_PARAMS_LOAD_VALID)
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    core4VisionParamsCommit(&params);
    return CORE4_VISION_PARAMS_SAVE_VERIFIED;
}

static uint8_t recoverPendingSelector(void)
{
    core4SlotSelector rollback;
    uint32_t slot;
    if (g_selector.pendingTransactionId == 0u) return 0u;
    slot = g_selector.rollbackSlot;
    if (slot >= CORE4_VISION_PARAMS_SLOT_COUNT ||
        g_selector.rollbackActiveSlot > CORE4_VISION_PARAMS_SLOT_COUNT ||
        g_selector.rollbackBank > 1u ||
        g_selector.rollbackValid > CORE4_SLOT_VALID_LEGACY) return 1u;
    rollback = g_selector;
    rollback.generation++;
    if (rollback.generation == 0u) rollback.generation = 1u;
    rollback.activeSlot = g_selector.rollbackActiveSlot;
    rollback.bank[slot] = g_selector.rollbackBank;
    rollback.valid[slot] = g_selector.rollbackValid;
    rollback.slotGeneration[slot] = g_selector.rollbackGeneration;
    rollback.transactionId[slot] = g_selector.rollbackTransactionId;
    rollback.pendingTransactionId = 0u;
    rollback.rollbackActiveSlot = 0u;
    rollback.rollbackSlot = 0u;
    rollback.rollbackBank = 0u;
    rollback.rollbackValid = 0u;
    rollback.rollbackGeneration = 0u;
    rollback.rollbackTransactionId = 0u;
    return writeSelector(&rollback);
}

```

```

}

static uint32_t slotPage(uint8_t slot, uint8_t bank)
{
    return (uint32_t) slot * 2u + (uint32_t) bank;
}

static uint8_t decodeLegacyPage(core4VisionParamsStruct *params)
{
    core4VisionParamsFlashBlob blob;

    if (readPageBytes(0u, &blob, sizeof(blob)) != 0u) return 0u;
    return (uint8_t) (core4VisionParamsBlobDecode(&blob, params) ==
        CORE4_VISION_PARAMS_BLOB_VALID ? 1u : 0u);
}

static uint8_t decodeSlotRecord(uint8_t slot,
    uint8_t bank,
    core4VisionParamsStruct *params)
{
    core4SlotRecord record;
    uint32_t crc;

    if (readPageBytes(slotPage(slot, bank), &record, sizeof(record)) != 0u) return 0u;
    crc = crc32leeeCalc((const uint8_t *) &record,
        (uint32_t) offsetof(core4SlotRecord, crc));
    if ((record.magic != CORE4_SLOT_RECORD_MAGIC) ||
        (record.version != CORE4_SLOT_STORE_VERSION) ||
        (record.slot != slot) ||
        (record.payloadBytes != CORE4_VISION_PARAMS_BLOB_BYTES) ||
        (record.crc != crc) ||
        (core4VisionParamsBlobDecode(&record.payload, params) !=
            CORE4_VISION_PARAMS_BLOB_VALID)) return 0u;
    return 1u;
}

static uint8_t readRecoverableSlotRecord(uint8_t slot, uint8_t bank,
    uint8_t *valid, uint32_t *generation)
{
    core4SlotRecord record;
    core4VisionParamsStruct params;
    uint32_t crc;

    *valid = 0u;
    *generation = 0u;
    if (readPageBytes(slotPage(slot, bank), &record, sizeof(record)) != 0u)
        return 1u;
    crc = crc32leeeCalc((const uint8_t *) &record,
        (uint32_t) offsetof(core4SlotRecord, crc));
    if ((record.magic == CORE4_SLOT_RECORD_MAGIC) &&
        (record.version == CORE4_SLOT_STORE_VERSION) &&
        (record.slot == slot) && (record.generation != 0u) &&
        (record.transactionId != UINT32_MAX) &&
        (record.payloadBytes == CORE4_VISION_PARAMS_BLOB_BYTES) &&
        (record.crc == crc) &&
        (core4VisionParamsBlobDecode(&record.payload, &params) ==
            CORE4_VISION_PARAMS_BLOB_VALID))
    {
        *valid = 1u;
        *generation = record.generation;
    }
    return 0u;
}

static uint8_t readRecoverableLegacy(uint8_t *valid)
{
    core4VisionParamsFlashBlob blob;
    core4VisionParamsStruct params;

    *valid = 0u;
    if (readPageBytes(0u, &blob, sizeof(blob)) != 0u) return 1u;
    if (core4VisionParamsBlobDecode(&blob, &params) ==
        CORE4_VISION_PARAMS_BLOB_VALID) *valid = 1u;
    return 0u;
}

static uint8_t rebuildLibrarySelector(core4SelectorLoadState state)
{
    core4SlotSelector recovered;

```

```

uint8_t slot;

if (state == CORE4_SELECTOR_REPAIR_LAST_GOOD)
{
    recovered = g_selector;
}
else if (state == CORE4_SELECTOR_REBUILD_FROM_DATA)
{
    memset(&recovered, 0, sizeof(recovered));
    recovered.generation = 1u;
    recovered.activeSlot = CORE4_VISION_PARAMS_SLOT_COUNT;
    for (slot = 0u; slot < CORE4_VISION_PARAMS_SLOT_COUNT; slot++)
    {
        uint8_t bank;
        uint8_t selectedValid = 0u;
        uint8_t selectedBank = 0u;
        uint32_t selectedGeneration = 0u;
        for (bank = 0u; bank < 2u; bank++)
        {
            uint8_t valid;
            uint32_t generation;
            if (readRecoverableSlotRecord(slot, bank, &valid,
                &generation) != 0u)
                return 1u;

            if ((valid != 0u) && ((selectedValid == 0u) ||
                (selectorNewer(generation, selectedGeneration) != 0u)))
            {
                selectedValid = 1u;
                selectedBank = bank;
                selectedGeneration = generation;
            }
        }
        if (selectedValid != 0u)
        {
            recovered.bank[slot] = selectedBank;
            recovered.valid[slot] = CORE4_SLOT_VALID_RECORD;
            recovered.slotGeneration[slot] = selectedGeneration;
            recovered.transactionId[slot] = 0u;
            if (recovered.activeSlot == CORE4_VISION_PARAMS_SLOT_COUNT)
                recovered.activeSlot = slot;
        }
    }
    if (recovered.valid[0u] == 0u)
    {
        uint8_t valid;
        if (readRecoverableLegacy(&valid) != 0u) return 1u;
        if (valid != 0u)
        {
            recovered.valid[0u] = CORE4_SLOT_VALID_LEGACY;
            recovered.slotGeneration[0u] = 1u;
            recovered.activeSlot = 0u;
        }
    }
}
else
{
    return 1u;
}

if ((erasePage(CORE4_SLOT_SELECTOR_PAGE_A) != 0u) ||
    (erasePage(CORE4_SLOT_SELECTOR_PAGE_B) != 0u)) return 1u;
memset(&g_selector, 0, sizeof(g_selector));
g_selectorReady = 0u;
g_selectorPage = 0u;
g_selectorRecord = 0u;
g_selectorUncertain = 0u;
g_selectorUncertainHadDurable = 0u;
return writeSelector(&recovered);
}

static uint8_t preparedRecordStillValid(uint8_t slot, uint8_t bank,
    uint32_t generation, uint32_t transactionId)
{
    core4SlotRecord record;
    core4VisionParamsStruct params;
    uint32_t crc;
    if (readPageBytes(slotPage(slot, bank), &record, sizeof(record)) != 0u)
        return 0u;
    crc = crc32leeeCalc((const uint8_t *) &record,

```

```

        (uint32_t) offsetof(core4SlotRecord, crc));
return (uint8_t) ((record.magic == CORE4_SLOT_RECORD_MAGIC) &&
    (record.version == CORE4_SLOT_STORE_VERSION) &&
    (record.slot == slot) && (record.generation == generation) &&
    (record.transactionId == transactionId) && (record.crc == crc) &&
    (record.payloadBytes == CORE4_VISION_PARAMS_BLOB_BYTES) &&
    (core4VisionParamsBlobDecode(&record.payload, &params) ==
        CORE4_VISION_PARAMS_BLOB_VALID));
}

static core4VisionParamsSaveStatus saveSlotInternal(uint8_t slot,
    const core4VisionParamsStruct *params,
    uint32_t transactionId)
{
    core4SlotRecord record;
    core4SlotSelector next;
    core4VisionParamsStruct sanitized;
    uint8_t bank;

    if ((slot >= CORE4_VISION_PARAMS_SLOT_COUNT) || (params == NULL))
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    if (g_prepared.valid != 0u || g_selector.pendingTransactionId != 0u)
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    sanitized = *params;
    core4VisionParamsSanitize(&sanitized);
    if (g_selector.valid[slot] == CORE4_SLOT_VALID_RECORD)
        bank = (uint8_t) (g_selector.bank[slot] ^ 1u);
    else if (g_selector.valid[slot] == CORE4_SLOT_VALID_LEGACY)
        bank = 1u;
    else
        bank = 0u;
    memset(&record, 0, sizeof(record));
    record.magic = CORE4_SLOT_RECORD_MAGIC;
    record.version = CORE4_SLOT_STORE_VERSION;
    record.slot = slot;
    record.generation = g_selector.slotGeneration[slot] + 1u;
    if (record.generation == 0u) record.generation = 1u;
    record.transactionId = transactionId;
    record.payloadBytes = CORE4_VISION_PARAMS_BLOB_BYTES;
    core4VisionParamsBlobEncode(&sanitized, &record.payload);
    record.crc = crc32IeeCalc((const uint8_t *) &record,
        (uint32_t) offsetof(core4SlotRecord, crc));
    if (writePageAtomic(slotPage(slot, bank), &record, sizeof(record)) != 0u)
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    if (decodeSlotRecord(slot, bank, &sanitized) == 0u)
        return CORE4_VISION_PARAMS_SAVE_VERIFY_FAILED;
    next = g_selector;
    next.generation++;
    if (next.generation == 0u) next.generation = 1u;
    next.bank[slot] = bank;
    next.valid[slot] = CORE4_SLOT_VALID_RECORD;
    next.slotGeneration[slot] = record.generation;
    next.transactionId[slot] = transactionId;
    next.activeSlot = slot;
    if (writeSelector(&next) != 0u) return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    return CORE4_VISION_PARAMS_SAVE_VERIFIED;
}

static core4VisionParamsLoadStatus initializeRuntime(
    const core4VisionParamsStruct *defaults,
    core4VisionParamsStruct *params)
{
    core4SelectorLoadState runtimeState = loadRuntimeSelector();
    core4VisionParamsStruct source;
    uint8_t sourceSlot = 0u;

    if (runtimeState == CORE4_SELECTOR_CORRUPT)
        return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
    if (runtimeState == CORE4_SELECTOR_VALID)
    {
        if (readRuntimeCommitted(params) == CORE4_VISION_PARAMS_LOAD_VALID)
            return CORE4_VISION_PARAMS_LOAD_VALID;
        /* A pending candidate is deliberately not repaired here: its rollback
         * must stay available for the coordinator's explicit ABORT. */
        if (g_runtimeSelector.pending != 0u)
            return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
        if ((defaults == NULL) ||
            (saveRuntimeFinal(defaults, 0u) !=
                CORE4_VISION_PARAMS_SAVE_VERIFIED))
    }
}

```

```

        return CORE4_VISION_PARAMS_LOAD_REWRITE_FAILED;
    *params = *defaults;
    core4VisionParamsSanitize(params);
    return CORE4_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN;
}

/* Upgrade from the former active-slot selector. If it had already
 * switched to a pending candidate, first materialize the rollback source
 * and then recreate the runtime candidate as pending. */
if ((g_selector.pendingTransactionId != 0u) &&
    (g_selector.activeSlot == g_selector.rollbackSlot) &&
    (g_selector.rollbackSlot < CORE4_VISION_PARAMS_SLOT_COUNT))
{
    uint8_t target = (uint8_t) g_selector.rollbackSlot;
    uint8_t oldActive = (uint8_t) g_selector.rollbackActiveSlot;
    uint32_t transactionId = g_selector.pendingTransactionId;
    uint8_t oldOk = 0u;
    if (oldActive < CORE4_VISION_PARAMS_SLOT_COUNT)
    {
        if (oldActive == target)
        {
            if (g_selector.rollbackValid == CORE4_SLOT_VALID_LEGACY)
                oldOk = decodeLegacyPage(&source);
            else if (g_selector.rollbackValid == CORE4_SLOT_VALID_RECORD)
                oldOk = decodeSlotRecord(target,
                                         (uint8_t) g_selector.rollbackBank, &source);
        }
        else if (core4VisionParamsFlashReadSlot(oldActive, &source,
                                                NULL) == CORE4_VISION_PARAMS_LOAD_VALID)
        {
            oldOk = 1u;
        }
    }
    if ((oldOk == 0u) && (defaults != NULL))
    {
        source = *defaults;
        core4VisionParamsSanitize(&source);
        oldActive = CORE4_VISION_PARAMS_SLOT_COUNT;
        oldOk = 1u;
    }
    if ((oldOk == 0u) ||
        (saveRuntimeFinal(&source,
                         oldActive < CORE4_VISION_PARAMS_SLOT_COUNT ?
                         (uint8_t) (oldActive + 1u) : 0u) !=
         CORE4_VISION_PARAMS_SAVE_VERIFIED))
        return CORE4_VISION_PARAMS_LOAD_REWRITE_FAILED;
    *params = source;
    if (core4VisionParamsFlashReadSlot(target, &source, NULL) ==
        CORE4_VISION_PARAMS_LOAD_VALID)
    {
        if ((prepareRuntimeInternal(&source, transactionId,
                                   (uint8_t) (target + 1u)) !=
            CORE4_VISION_PARAMS_SAVE_VERIFIED) ||
            (commitRuntimeInternal(transactionId, 1u) !=
             CORE4_VISION_PARAMS_SAVE_VERIFIED))
            return CORE4_VISION_PARAMS_LOAD_REWRITE_FAILED;
    }
    return CORE4_VISION_PARAMS_LOAD_VALID;
}

if ((g_selectorReady != 0u) &&
    (g_selector.activeSlot < CORE4_VISION_PARAMS_SLOT_COUNT) &&
    (core4VisionParamsFlashReadSlot((uint8_t) g_selector.activeSlot,
                                    &source, NULL) == CORE4_VISION_PARAMS_LOAD_VALID))
{
    sourceSlot = (uint8_t) (g_selector.activeSlot + 1u);
}
else if (defaults != NULL)
{
    source = *defaults;
    core4VisionParamsSanitize(&source);
}
else
{
    return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
}
if (saveRuntimeFinal(&source, sourceSlot) !=
    CORE4_VISION_PARAMS_SAVE_VERIFIED)
    return CORE4_VISION_PARAMS_LOAD_REWRITE_FAILED;

```

```

*params = source;
return CORE4_VISION_PARAMS_LOAD_VALID;
}

core4VisionParamsLoadStatus core4VisionParamsFlashLoad(core4VisionParamsStruct *params)
{
    core4SelectorLoadState selectorState;
    core4FlashRangeState legacyState;
    core4VisionParamsLoadStatus libraryStatus = CORE4_VISION_PARAMS_LOAD_VALID;
    core4VisionParamsLoadStatus runtimeStatus;
    core4VisionParamsStruct defaults;
    core4VisionParamsStruct loaded;

    if (params == NULL) return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
    defaults = *params;
    core4VisionParamsSanitize(&defaults);
    ensureFlashReady();
    selectorState = loadSelector();
    if ((selectorState == CORE4_SELECTOR_REPAIR_LAST_GOOD) ||
        (selectorState == CORE4_SELECTOR_REBUILD_FROM_DATA))
    {
        if (rebuildLibrarySelector(selectorState) == 0u)
            selectorState = CORE4_SELECTOR_VALID;
        else
            selectorState = CORE4_SELECTOR_CORRUPT;
    }
    if (selectorState == CORE4_SELECTOR_CORRUPT)
    {
        /* Runtime is the sole startup source. Keep the damaged library
        * unavailable for diagnostics and writes, but still boot a verified
        * committed runtime (or rebuild runtime from defaults). */
        memset(&g_selector, 0, sizeof(g_selector));
        memset(&g_prepared, 0, sizeof(g_prepared));
        g_selectorReady = 0u;
        libraryStatus = CORE4_VISION_PARAMS_LOAD_READ_FAILED;
    }
    else if (selectorState == CORE4_SELECTOR_VALID)
    {
        if (g_selector.pendingTransactionId != 0u &&
            g_selector.pendingTransactionId != UINT32_MAX)
        {
            uint8_t slot = (uint8_t) g_selector.rollbackSlot;
            memset(&g_prepared, 0, sizeof(g_prepared));
            g_prepared.before = g_selector;
            g_prepared.before.activeSlot = g_selector.rollbackActiveSlot;
            g_prepared.before.bank[slot] = g_selector.rollbackBank;
            g_prepared.before.valid[slot] = g_selector.rollbackValid;
            g_prepared.before.slotGeneration[slot] =
                g_selector.rollbackGeneration;
            g_prepared.before.transactionId[slot] =
                g_selector.rollbackTransactionId;
            g_prepared.before.pendingTransactionId = 0u;
            g_prepared.transactionId = g_selector.pendingTransactionId;
            g_prepared.generation = g_selector.slotGeneration[slot];
            g_prepared.slot = slot;
            g_prepared.bank = (uint8_t) g_selector.bank[slot];
            g_prepared.valid = 1u;
            g_prepared.committed = 1u;
        }
    }
    else
    {
        memset(&g_selector, 0, sizeof(g_selector));
        g_selector.activeSlot = CORE4_VISION_PARAMS_SLOT_COUNT;
        legacyState = blankCheckRange(0u, 0u, CORE4_FLASH_PAGE_WORDS);
        if (legacyState == CORE4_FLASH_RANGE_IO_FAILED)
        {
            libraryStatus = CORE4_VISION_PARAMS_LOAD_READ_FAILED;
        }
        else if ((legacyState == CORE4_FLASH_RANGE_PROGRAMMED) &&
            (decodeLegacyPage(&loaded) != 0u))
        {
            g_selector.activeSlot = 0u;
            g_selector.valid[0] = CORE4_SLOT_VALID_LEGACY;
            g_selector.slotGeneration[0] = 1u;
            g_selector.generation = 1u;
            if (writeSelector(&g_selector) != 0u)
            {
                saturatingIncrement(&g_status.writeVerifyFailures);
            }
        }
    }
}

```

```

        libraryStatus = CORE4_VISION_PARAMS_LOAD_REWRITE_FAILED;
    }
}
else
{
    g_selector.generation = 1u;
    if (legacyState == CORE4_FLASH_RANGE_PROGRAMMED)
    {
        g_selector.activeSlot = 0u;
        g_selector.valid[0u] = CORE4_SLOT_VALID_LEGACY;
        g_selector.slotGeneration[0u] = 1u;
    }
    if (writeSelector(&g_selector) != 0u)
    {
        saturatingIncrement(&g_status.writeVerifyFailures);
        libraryStatus = (legacyState == CORE4_FLASH_RANGE_BLANK
            ? CORE4_VISION_PARAMS_LOAD_BLANK_INIT_FAILED
            : CORE4_VISION_PARAMS_LOAD_REWRITE_FAILED;
        )
    }
    else
    {
        libraryStatus = (legacyState == CORE4_FLASH_RANGE_BLANK
            ? CORE4_VISION_PARAMS_LOAD_BLANK_INITIALIZED
            : CORE4_VISION_PARAMS_LOAD_READ_FAILED;
        )
    }
}
}
runtimeStatus = initializeRuntime(&defaults, params);
g_status.libraryLoad = libraryStatus;
g_status.runtimeLoad = runtimeStatus;
if (runtimeStatus == CORE4_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN)
    g_status.runtimeDefaultsRewritten = 1u;
if ((runtimeStatus != CORE4_VISION_PARAMS_LOAD_VALID) &&
    (runtimeStatus != CORE4_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN))
    g_status.lastLoad = runtimeStatus;
else if (runtimeStatus == CORE4_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN)
    g_status.lastLoad = runtimeStatus;
else if ((libraryStatus != CORE4_VISION_PARAMS_LOAD_VALID) &&
    (libraryStatus != CORE4_VISION_PARAMS_LOAD_BLANK_INITIALIZED) &&
    (libraryStatus != CORE4_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN))
    g_status.lastLoad = CORE4_VISION_PARAMS_LOAD_VALID;
else
    g_status.lastLoad = libraryStatus;
return g_status.lastLoad;
}

core4VisionParamsLoadStatus core4VisionParamsFlashReadSlot(uint8_t slot,
    core4VisionParamsStruct *params,
    core4VisionParamsSlotMeta *meta)
{
    uint8_t ok;

    if ((g_selectorReady == 0u) || (slot >= CORE4_VISION_PARAMS_SLOT_COUNT) ||
        (params == NULL) || (g_selector.valid[slot] == 0u))
        return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
    ok = (g_selector.valid[slot] == CORE4_SLOT_VALID_LEGACY)
        ? decodeLegacyPage(params)
        : decodeSlotRecord(slot, (uint8_t) g_selector.bank[slot], params);
    if (ok == 0u) return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
    core4VisionParamsSanitize(params);
    if (meta != NULL)
    {
        meta->generation = g_selector.slotGeneration[slot];
        meta->transactionId = g_selector.transactionId[slot];
        meta->valid = 1u;
        meta->active = 0u;
        meta->legacy = (uint8_t) (g_selector.valid[slot] == CORE4_SLOT_VALID_LEGACY ? 1u : 0u);
    }
    return CORE4_VISION_PARAMS_LOAD_VALID;
}

core4VisionParamsLoadStatus core4VisionParamsFlashInspectSlot(uint8_t slot,
    core4VisionParamsLibraryMeta *meta)
{
    core4VisionParamsStruct params;
    core4VisionParamsFlashBlob blob;
    if ((meta == NULL) || (g_selectorReady == 0u) ||
        (slot >= CORE4_VISION_PARAMS_SLOT_COUNT))
        return CORE4_VISION_PARAMS_LOAD_READ_FAILED;

```



```

memset(meta, 0, sizeof(*meta));
meta->bank = (uint8_t) g_selector.bank[slot];
meta->generation = g_selector.slotGeneration[slot];
meta->transactionId = g_selector.transactionId[slot];
if (g_selector.valid[slot] == 0u)
{
    meta->state = CORE4_VISION_PARAMS_LIBRARY_EMPTY;
    return CORE4_VISION_PARAMS_LOAD_VALID;
}
if (core4VisionParamsFlashReadSlot(slot, &params, NULL) !=
    CORE4_VISION_PARAMS_LOAD_VALID)
{
    meta->state = CORE4_VISION_PARAMS_LIBRARY_INVALID;
    return CORE4_VISION_PARAMS_LOAD_VALID;
}
core4VisionParamsBlobEncode(&params, &blob);
meta->blobCrc = crc32leecalc(blob.bytes,
    CORE4_VISION_PARAMS_BLOB_BYTES);
meta->state = CORE4_VISION_PARAMS_LIBRARY_VALID;
return CORE4_VISION_PARAMS_LOAD_VALID;
}

core4VisionParamsLoadStatus core4VisionParamsFlashReadRuntime(
    core4VisionParamsStruct *params)
{
    ensureFlashReady();
    return readRuntimeCommitted(params);
}

core4VisionParamsLoadStatus core4VisionParamsFlashGetRuntimeMeta(
    core4VisionParamsRuntimeMeta *meta)
{
    core4RuntimeSelector committed;
    if ((meta == NULL) || (resolveRuntimeSelectorUncertainty() != 0u) ||
        (committedRuntimeSelector(&committed) != 0u))
        return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
    memset(meta, 0, sizeof(*meta));
    meta->generation = committed.generation;
    meta->blobCrc = committed.blobCrc;
    meta->transactionId = committed.transactionId;
    meta->sourceSlot = committed.sourceSlot;
    meta->valid = committed.valid;
    meta->pending = g_runtimeSelector.pending;
    meta->bank = committed.bank;
    return CORE4_VISION_PARAMS_LOAD_VALID;
}

core4VisionParamsSaveStatus core4VisionParamsFlashSaveRuntime(
    const core4VisionParamsStruct *params)
{
    ensureFlashReady();
    g_status.lastSave = saveRuntimeFinal(params, 0u);
    if (g_status.lastSave != CORE4_VISION_PARAMS_SAVE_VERIFIED)
        saturatingIncrement(&g_status.writeVerifyFailures);
    return g_status.lastSave;
}

core4VisionParamsSaveStatus core4VisionParamsFlashPrepareRuntime(
    const core4VisionParamsStruct *params, uint32_t transactionId,
    uint8_t sourceSlot)
{
    ensureFlashReady();
    if ((transactionId == 0u) || (transactionId == UINT32_MAX))
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    return prepareRuntimeInternal(params, transactionId, sourceSlot);
}

core4VisionParamsSaveStatus core4VisionParamsFlashPrepareRuntimeFromSlot(
    uint8_t slot, uint32_t transactionId)
{
    core4VisionParamsStruct params;
    if ((transactionId == 0u) || (transactionId == UINT32_MAX) ||
        (core4VisionParamsFlashReadSlot(slot, &params, NULL) !=
            CORE4_VISION_PARAMS_LOAD_VALID))
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    return prepareRuntimeInternal(&params, transactionId,
        (uint8_t) (slot + 1u));
}

```

```

core4VisionParamsSaveStatus core4VisionParamsFlashApplyPreparedRuntime(
    uint32_t transactionId)
{
    core4VisionParamsStruct params;
    if ((transactionId == 0u) ||
        (resolveRuntimeSelectorUncertainty() != 0u) ||
        (g_runtimeSelectorReady == 0u) ||
        (g_runtimeSelector.pending == 0u) ||
        (g_runtimeSelector.transactionId != transactionId) ||
        (readRuntimeBank(g_runtimeSelector.bank, g_runtimeSelector.blobCrc,
            &params) != 0u))
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    core4VisionParamsCommit(&params);
    return CORE4_VISION_PARAMS_SAVE_VERIFIED;
}

core4VisionParamsSaveStatus core4VisionParamsFlashCommitRuntime(
    uint32_t transactionId)
{
    if ((transactionId == 0u) || (transactionId == UINT32_MAX))
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    return commitRuntimeInternal(transactionId, 1u);
}

core4VisionParamsSaveStatus core4VisionParamsFlashSaveSlot(uint8_t slot,
    const core4VisionParamsStruct *params,
    uint32_t transactionId)
{
    ensureFlashReady();
    g_status.lastSave = (transactionId == UINT32_MAX) ?
        CORE4_VISION_PARAMS_SAVE_WRITE_FAILED :
        saveSlotInternal(slot, params, transactionId);
    if (g_status.lastSave != CORE4_VISION_PARAMS_SAVE_VERIFIED)
        saturatingIncrement(&g_status.writeVerifyFailures);
    return g_status.lastSave;
}

core4VisionParamsSaveStatus core4VisionParamsFlashPrepareSlot(uint8_t slot,
    const core4VisionParamsStruct *params,
    uint32_t transactionId)
{
    core4SlotRecord record;
    core4VisionParamsStruct sanitized;
    uint8_t bank;

    ensureFlashReady();
    if ((g_selectorReady == 0u) || (slot >= CORE4_VISION_PARAMS_SLOT_COUNT) ||
        (params == NULL) || (transactionId == 0u) ||
        (transactionId == UINT32_MAX) || (g_prepared.valid != 0u))
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    sanitized = *params;
    core4VisionParamsSanitize(&sanitized);
    if (g_selector.valid[slot] == CORE4_SLOT_VALID_RECORD)
        bank = (uint8_t) (g_selector.bank[slot] ^ 1u);
    else if (g_selector.valid[slot] == CORE4_SLOT_VALID_LEGACY)
        bank = 1u;
    else
        bank = 0u;
    memset(&record, 0, sizeof(record));
    record.magic = CORE4_SLOT_RECORD_MAGIC;
    record.version = CORE4_SLOT_STORE_VERSION;
    record.slot = slot;
    record.generation = g_selector.slotGeneration[slot] + 1u;
    if (record.generation == 0u) record.generation = 1u;
    record.transactionId = transactionId;
    record.payloadBytes = CORE4_VISION_PARAMS_BLOB_BYTES;
    core4VisionParamsBlobEncode(&sanitized, &record.payload);
    record.crc = crc32IeeeCalc((const uint8_t *) &record,
        (uint32_t) offsetof(core4SlotRecord, crc));
    if ((writePageAtomic(slotPage(slot, bank), &record, sizeof(record)) != 0u) ||
        (decodeSlotRecord(slot, bank, &sanitized) == 0u))
        return CORE4_VISION_PARAMS_SAVE_VERIFY_FAILED;
    memset(&g_prepared, 0, sizeof(g_prepared));
    g_prepared.before = g_selector;
    g_prepared.transactionId = transactionId;
    g_prepared.generation = record.generation;
    g_prepared.slot = slot;
    g_prepared.bank = bank;
    g_prepared.valid = 1u;
}

```

```

        return CORE4_VISION_PARAMS_SAVE_VERIFIED;
    }

core4VisionParamsSaveStatus core4VisionParamsFlashCommitPrepared(uint8_t slot,
    uint32_t transactionId)
{
    core4SlotSelector next;
    if ((transactionId == 0u) || (transactionId == UINT32_MAX) ||
        (g_prepared.valid == 0u) || (g_prepared.committed != 0u) ||
        (g_prepared.slot != slot) || (g_prepared.transactionId != transactionId) ||
        !preparedRecordStillValid(slot, g_prepared.bank, g_prepared.generation,
            transactionId))
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    next = g_selector;
    next.generation++;
    if (next.generation == 0u) next.generation = 1u;
    next.bank[slot] = g_prepared.bank;
    next.valid[slot] = CORE4_SLOT_VALID_RECORD;
    next.slotGeneration[slot] = g_prepared.generation;
    next.transactionId[slot] = transactionId;
    next.activeSlot = slot;
    next.pendingTransactionId = transactionId;
    next.rollbackActiveSlot = g_prepared.before.activeSlot;
    next.rollbackSlot = slot;
    next.rollbackBank = g_prepared.before.bank[slot];
    next.rollbackValid = g_prepared.before.valid[slot];
    next.rollbackGeneration = g_prepared.before.slotGeneration[slot];
    next.rollbackTransactionId = g_prepared.before.transactionId[slot];
    if (writeSelector(&next) != 0u) return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    g_prepared.committed = 1u;
    return CORE4_VISION_PARAMS_SAVE_VERIFIED;
}

core4VisionParamsSaveStatus core4VisionParamsFlashAbortTransaction(
    uint32_t transactionId)
{
    core4SlotSelector rollback;
    core4VisionParamsSaveStatus runtimeStatus;
    uint8_t runtimePending;
    if (transactionId == UINT32_MAX)
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    runtimePending = (uint8_t) (g_runtimeSelectorReady != 0u &&
        g_runtimeSelector.pending != 0u &&
        g_runtimeSelector.transactionId == transactionId);
    if (runtimePending != 0u &&
        applyCommittedRuntime() != CORE4_VISION_PARAMS_SAVE_VERIFIED)
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    runtimeStatus = abortRuntimeTransaction(transactionId);
    if (runtimeStatus != CORE4_VISION_PARAMS_SAVE_VERIFIED)
        return runtimeStatus;
    if (resolveSelectorUncertainty() != 0u)
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    if (g_selector.pendingTransactionId != 0u)
    {
        if (g_selector.pendingTransactionId != transactionId ||
            recoverPendingSelector() != 0u)
            return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
        memset(&g_prepared, 0, sizeof(g_prepared));
        return CORE4_VISION_PARAMS_SAVE_VERIFIED;
    }
    if ((g_prepared.valid == 0u) || (g_prepared.transactionId != transactionId))
    {
        return CORE4_VISION_PARAMS_SAVE_VERIFIED;
    }
    if (g_prepared.committed != 0u)
    {
        rollback = g_prepared.before;
        rollback.generation = g_selector.generation + 1u;
        if (rollback.generation == 0u) rollback.generation = 1u;
        if (writeSelector(&rollback) != 0u) return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    }
    memset(&g_prepared, 0, sizeof(g_prepared));
    return CORE4_VISION_PARAMS_SAVE_VERIFIED;
}

core4VisionParamsSaveStatus core4VisionParamsFlashFinishTransaction(uint32_t transactionId)
{
    core4VisionParamsSaveStatus runtimeStatus;
    uint8_t runtimePending;

```

```

if (transactionId == UINT32_MAX)
    return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
runtimePending = (uint8_t) (g_runtimeSelectorReady != 0u &&
    g_runtimeSelector.pending != 0u &&
    g_runtimeSelector.transactionId == transactionId);
if (runtimePending != 0u &&
    core4VisionParamsFlashApplyPreparedRuntime(transactionId) !=
        CORE4_VISION_PARAMS_SAVE_VERIFIED)
    return CORE4_VISION_PARAMS_SAVE_VERIFY_FAILED;
if (resolveSelectorUncertainty() != 0u)
    return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
if (g_selector.pendingTransactionId == transactionId)
{
    uint8_t slot = (uint8_t) g_selector.rollbackSlot;
    core4SlotSelector next = g_selector;
    if ((slot >= CORE4_VISION_PARAMS_SLOT_COUNT) ||
        (g_selector.valid[slot] != CORE4_SLOT_VALID_RECORD) ||
        (preparedRecordStillValid(slot, (uint8_t) g_selector.bank[slot],
            g_selector.slotGeneration[slot], transactionId) == 0u))
        return CORE4_VISION_PARAMS_SAVE_VERIFY_FAILED;
    next.generation++;
    if (next.generation == 0u) next.generation = 1u;
    next.pendingTransactionId = 0u;
    next.rollbackActiveSlot = 0u;
    next.rollbackSlot = 0u;
    next.rollbackBank = 0u;
    next.rollbackValid = 0u;
    next.rollbackGeneration = 0u;
    next.rollbackTransactionId = 0u;
    if (writeSelector(&next) != 0u) return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
}
if ((g_prepared.valid != 0u) && (g_prepared.transactionId == transactionId))
    memset(&g_prepared, 0, sizeof(g_prepared));
else if (g_selector.pendingTransactionId != 0u)
    return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
runtimeStatus = finishRuntimeTransaction(transactionId);
if (runtimeStatus != CORE4_VISION_PARAMS_SAVE_VERIFIED)
    return runtimeStatus;
return CORE4_VISION_PARAMS_SAVE_VERIFIED;
}

core4VisionParamsSaveStatus core4VisionParamsFlashSave(
    const core4VisionParamsStruct *params)
{
    return core4VisionParamsFlashSaveRuntime(params);
}

core4VisionParamsLoadStatus core4VisionParamsFlashActivateSlot(uint8_t slot,
    core4VisionParamsStruct *params)
{
    core4VisionParamsLoadStatus status = core4VisionParamsFlashReadSlot(slot, params, NULL);

    if ((status != CORE4_VISION_PARAMS_LOAD_VALID) ||
        (saveRuntimeFinal(params, (uint8_t) (slot + 1u)) !=
            CORE4_VISION_PARAMS_SAVE_VERIFIED))
        return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
    return CORE4_VISION_PARAMS_LOAD_VALID;
}

core4VisionParamsLoadStatus core4VisionParamsFlashActivatePrepared(uint8_t slot,
    uint32_t transactionId, core4VisionParamsStruct *params)
{
    core4VisionParamsLoadStatus status;
    if (params == NULL || transactionId == 0u || transactionId == UINT32_MAX ||
        g_prepared.valid == 0u || g_prepared.committed == 0u ||
        g_prepared.slot != slot || g_prepared.transactionId != transactionId ||
        g_selector.pendingTransactionId != transactionId ||
        !preparedRecordStillValid(slot, g_prepared.bank, g_prepared.generation,
            transactionId)) return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
    status = core4VisionParamsFlashReadSlot(slot, params, NULL);
    if (status != CORE4_VISION_PARAMS_LOAD_VALID) return status;
    if ((prepareRuntimeInternal(params, transactionId,
        (uint8_t) (slot + 1u)) != CORE4_VISION_PARAMS_SAVE_VERIFIED) ||
        (commitRuntimeInternal(transactionId, 1u) !=
            CORE4_VISION_PARAMS_SAVE_VERIFIED))
        return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
    return CORE4_VISION_PARAMS_LOAD_VALID;
}

```

```

core4VisionParamsLoadStatus core4VisionParamsFlashActivateSlotPending(uint8_t slot,
    core4VisionParamsStruct *params)
{
    core4VisionParamsLoadStatus status = core4VisionParamsFlashReadSlot(slot, params, NULL);
    if ((status != CORE4_VISION_PARAMS_LOAD_VALID) ||
        (prepareRuntimeInternal(params, UINT32_MAX,
            (uint8_t) (slot + 1u)) != CORE4_VISION_PARAMS_SAVE_VERIFIED) ||
        (commitRuntimeInternal(UINT32_MAX, 1u) !=
            CORE4_VISION_PARAMS_SAVE_VERIFIED))
        return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
    return CORE4_VISION_PARAMS_LOAD_VALID;
}

```

```

core4VisionParamsLoadStatus core4VisionParamsFlashAbortPendingActivation(
    core4VisionParamsStruct *params)
{
    if (params == NULL) return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
    if ((g_runtimeSelectorReady != 0u) &&
        (g_runtimeSelector.pending != 0u) &&
        (g_runtimeSelector.transactionId == UINT32_MAX))
    {
        if (readRuntimeCommitted(params) != CORE4_VISION_PARAMS_LOAD_VALID)
            return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
        core4VisionParamsCommit(params);
    }
    if (abortRuntimeTransaction(UINT32_MAX) !=
        CORE4_VISION_PARAMS_SAVE_VERIFIED)
        return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
    if ((g_selector.pendingTransactionId == UINT32_MAX) &&
        (recoverPendingSelector() != 0u))
        return CORE4_VISION_PARAMS_LOAD_READ_FAILED;
    return CORE4_VISION_PARAMS_LOAD_VALID;
}

```

```

core4VisionParamsSaveStatus core4VisionParamsFlashFinishPendingActivation(void)
{
    core4VisionParamsSaveStatus status;
    if ((g_selector.pendingTransactionId == UINT32_MAX) &&
        ((g_runtimeSelectorReady == 0u) ||
            (g_runtimeSelector.pending == 0u) ||
            (g_runtimeSelector.transactionId != UINT32_MAX)))
        return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    if ((g_runtimeSelectorReady != 0u) &&
        (g_runtimeSelector.pending != 0u) &&
        (g_runtimeSelector.transactionId == UINT32_MAX) &&
        (core4VisionParamsFlashApplyPreparedRuntime(UINT32_MAX) !=
            CORE4_VISION_PARAMS_SAVE_VERIFIED))
        return CORE4_VISION_PARAMS_SAVE_VERIFY_FAILED;
    if (g_selector.pendingTransactionId == UINT32_MAX)
    {
        core4SlotSelector next = g_selector;
        next.generation++;
        if (next.generation == 0u) next.generation = 1u;
        next.pendingTransactionId = 0u;
        next.rollbackActiveSlot = 0u;
        next.rollbackSlot = 0u;
        next.rollbackBank = 0u;
        next.rollbackValid = 0u;
        next.rollbackGeneration = 0u;
        next.rollbackTransactionId = 0u;
        if (writeSelector(&next) != 0u)
            return CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;
    }
    status = finishRuntimeTransaction(UINT32_MAX);
    return status;
}

```

```

uint8_t core4VisionParamsFlashGetActiveSlot(void)
{
    return 0u;
}

```

```

uint8_t core4VisionParamsFlashMutationPending(void)
{
    ensureFlashReady();
    return (uint8_t) ((g_selectorReady == 0u) ||
        (g_runtimeSelectorReady == 0u) ||
        (g_selectorUncertain != 0u) ||
        (g_runtimeSelectorUncertain != 0u) ||

```

```

        (g_selector.pendingTransactionId != 0u) ||
        (g_runtimeSelector.pending != 0u));
    }

void core4VisionParamsPersistenceGetStatus(
    core4VisionParamsPersistenceStatus *status)
{
    if (status != NULL) *status = g_status;
}

```

core4_cyt4bb7/project/code/config/core4VisionParamsPersistence.h

SHA-256

:

e4aa20312307088ccf4f7ee9e3207e74549800e9732085f7eb93cda5dbc91a0e

代码 (c)

```

#ifndef CORE4_VISION_PARAMS_PERSISTENCE_H
#define CORE4_VISION_PARAMS_PERSISTENCE_H

#include "core4VisionParamsStorage.h"

typedef struct
{
    core4VisionParamsLoadStatus lastLoad;
    core4VisionParamsSaveStatus lastSave;
    uint32_t writeVerifyFailures;
    core4VisionParamsLoadStatus libraryLoad;
    core4VisionParamsLoadStatus runtimeLoad;
    uint8_t runtimeDefaultsRewritten;
} core4VisionParamsPersistenceStatus;

#define CORE4_VISION_PARAMS_SLOT_COUNT (5u)

typedef struct
{
    uint32_t generation;
    uint32_t transactionId;
    uint8_t valid;
    uint8_t active;
    uint8_t legacy;
} core4VisionParamsSlotMeta;

typedef struct
{
    uint32_t generation;
    uint32_t blobCrc;
    uint32_t transactionId;
    uint8_t sourceSlot;
    uint8_t valid;
    uint8_t pending;
    uint8_t bank;
} core4VisionParamsRuntimeMeta;

#define CORE4_VISION_PARAMS_LIBRARY_EMPTY (0u)
#define CORE4_VISION_PARAMS_LIBRARY_VALID (1u)
#define CORE4_VISION_PARAMS_LIBRARY_INVALID (2u)
typedef struct
{
    uint8_t state;
    uint8_t bank;
    uint16_t reserved;
    uint32_t generation;
    uint32_t blobCrc;
    uint32_t transactionId;
} core4VisionParamsLibraryMeta;

core4VisionParamsLoadStatus core4VisionParamsFlashLoad(
    core4VisionParamsStruct *params);
core4VisionParamsSaveStatus core4VisionParamsFlashSave(
    const core4VisionParamsStruct *params);
core4VisionParamsLoadStatus core4VisionParamsFlashReadRuntime(
    core4VisionParamsStruct *params);
core4VisionParamsLoadStatus core4VisionParamsFlashGetRuntimeMeta(
    core4VisionParamsRuntimeMeta *meta);

```

```
core4VisionParamsSaveStatus core4VisionParamsFlashSaveRuntime(  
    const core4VisionParamsStruct *params);  
core4VisionParamsSaveStatus core4VisionParamsFlashPrepareRuntime(  
    const core4VisionParamsStruct *params,  
    uint32_t transactionId,  
    uint8_t sourceSlot);  
core4VisionParamsSaveStatus core4VisionParamsFlashPrepareRuntimeFromSlot(  
    uint8_t slot,  
    uint32_t transactionId);  
core4VisionParamsSaveStatus core4VisionParamsFlashApplyPreparedRuntime(  
    uint32_t transactionId);  
core4VisionParamsSaveStatus core4VisionParamsFlashCommitRuntime(  
    uint32_t transactionId);  
core4VisionParamsLoadStatus core4VisionParamsFlashReadSlot(uint8_t slot,  
    core4VisionParamsStruct *params,  
    core4VisionParamsSlotMeta *meta);  
core4VisionParamsLoadStatus core4VisionParamsFlashInspectSlot(uint8_t slot,  
    core4VisionParamsLibraryMeta *meta);  
core4VisionParamsSaveStatus core4VisionParamsFlashSaveSlot(uint8_t slot,  
    const core4VisionParamsStruct *params,  
    uint32_t transactionId);  
core4VisionParamsSaveStatus core4VisionParamsFlashPrepareSlot(uint8_t slot,  
    const core4VisionParamsStruct *params,  
    uint32_t transactionId);  
core4VisionParamsSaveStatus core4VisionParamsFlashCommitPrepared(uint8_t slot,  
    uint32_t transactionId);  
core4VisionParamsSaveStatus core4VisionParamsFlashAbortTransaction(  
    uint32_t transactionId);  
core4VisionParamsSaveStatus core4VisionParamsFlashFinishTransaction(uint32_t transactionId);  
core4VisionParamsLoadStatus core4VisionParamsFlashActivateSlot(uint8_t slot,  
    core4VisionParamsStruct *params);  
core4VisionParamsLoadStatus core4VisionParamsFlashActivatePrepared(uint8_t slot,  
    uint32_t transactionId, core4VisionParamsStruct *params);  
core4VisionParamsLoadStatus core4VisionParamsFlashActivateSlotPending(uint8_t slot,  
    core4VisionParamsStruct *params);  
core4VisionParamsLoadStatus core4VisionParamsFlashAbortPendingActivation(  
    core4VisionParamsStruct *params);  
core4VisionParamsSaveStatus core4VisionParamsFlashFinishPendingActivation(void);  
uint8_t core4VisionParamsFlashGetActiveSlot(void);  
uint8_t core4VisionParamsFlashMutationPending(void);  
void core4VisionParamsPersistenceGetStatus(  
    core4VisionParamsPersistenceStatus *status);  
  
#endif /* CORE4_VISION_PARAMS_PERSISTENCE_H */
```

core4_cyt4bb7/project/code/config/core4VisionParamsStorage.c

SHA-256 :
c0a549e52cd1833f0bdec72bbabab8a4d7ef8351d33434003e50cdff8921677a

代码 (c)

```
#include "core4VisionParamsStorage.h"  
#include <string.h>  
static uint8_t validBackend(const core4VisionParamsStorageBackend*b){return(uint8_t)(b->b->read&&b->erase&&b->write);}  
static uint8_t blank(const uint8_t*p){uint32_t i;for(i=0;i<CORE4_VISION_PARAMS_BLOB_BYTES;i++)if(p[i]!=0xFFu)return 0u;return 1u;}  
static uint8_t match(const core4VisionParamsFlashBlob*e,const core4VisionParamsFlashBlob*a,const core4VisionParamsStruct*p){core4VisionParamsStruct  
d;return(uint8_t)  
(memcmp(e,a,CORE4_VISION_PARAMS_BLOB_BYTES)==0&&core4VisionParamsBlobDecode(a,&d)==CORE4_VISION_PARAMS_BLOB_VALID&&memc  
mp(p,&d,sizeof(d))==0);}  
core4VisionParamsSaveStatus core4VisionParamsStorageSave(const core4VisionParamsStorageBackend*b,const core4VisionParamsStruct*p)  
{core4VisionParamsFlashBlob e,a;if(!validBackend(b))!p)return  
CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;core4VisionParamsBlobEncode(p,&e);if(b->erase(b->context))return  
CORE4_VISION_PARAMS_SAVE_ERASE_FAILED;if(b->write(b->context,e.bytes,CORE4_VISION_PARAMS_BLOB_BYTES))return  
CORE4_VISION_PARAMS_SAVE_WRITE_FAILED;memset(&a,0,sizeof(a));if(b->read(b->context,a.bytes,CORE4_VISION_PARAMS_BLOB_BYTES))return  
CORE4_VISION_PARAMS_SAVE_READ_FAILED;return match(&e,&a,p)?  
CORE4_VISION_PARAMS_SAVE_VERIFIED:CORE4_VISION_PARAMS_SAVE_VERIFY_FAILED;}  
core4VisionParamsLoadStatus core4VisionParamsStorageInitializeBlank(const core4VisionParamsStorageBackend*b,const  
core4VisionParamsStruct*d,core4VisionParamsStruct*l){if(!validBackend(b))!d!!l)return  
CORE4_VISION_PARAMS_LOAD_BLANK_INIT_FAILED;*l=*d;core4VisionParamsSanitize(l);return  
core4VisionParamsStorageSave(b,l)==CORE4_VISION_PARAMS_SAVE_VERIFIED?  
CORE4_VISION_PARAMS_LOAD_BLANK_INITIALIZED:CORE4_VISION_PARAMS_LOAD_BLANK_INIT_FAILED;}  
core4VisionParamsLoadStatus core4VisionParamsStorageLoad(const core4VisionParamsStorageBackend*b,const  
core4VisionParamsStruct*d,core4VisionParamsStruct*l){core4VisionParamsFlashBlob x;core4VisionParamsBlobStatus s;if(!validBackend(b))!d!!l)return  
CORE4_VISION_PARAMS_LOAD_READ_FAILED;*l=*d;memset(&x,0,sizeof(x));if(b->read(b-
```

```
>context,x.bytes,CORE4_VISION_PARAMS_BLOB_BYTES))return CORE4_VISION_PARAMS_LOAD_READ_FAILED;if(blank(x.bytes))return
CORE4_VISION_PARAMS_LOAD_BLANK;s=core4VisionParamsBlobDecode(&x,l);if(s==CORE4_VISION_PARAMS_BLOB_VALID)return
CORE4_VISION_PARAMS_LOAD_VALID;"l="d;core4VisionParamsSanitize(l);return
core4VisionParamsStorageSave(b,l)==CORE4_VISION_PARAMS_SAVE_VERIFIED?
CORE4_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN:CORE4_VISION_PARAMS_LOAD_REWRITE_FAILED;}
```

core4_cyt4bb7/project/code/config/core4VisionParamsStorage.h

SHA-256

:

2c059b447bf77c6409be4a5f7d4d22eeb29c91b32e68c19ec99f03a4b10140da

代码 (c)

```
#ifndef CORE4_VISION_PARAMS_STORAGE_H
#define CORE4_VISION_PARAMS_STORAGE_H
#include <stdint.h>
#include "core4VisionParamsBlob.h"
#define CORE4_VISION_PARAMS_STORAGE_READ_BYTES CORE4_VISION_PARAMS_BLOB_BYTES
typedef uint8_t (*core4VisionParamsStorageRead)(void*,uint8_t*,uint32_t);
typedef uint8_t (*core4VisionParamsStorageEraseFn)(void*);
typedef uint8_t (*core4VisionParamsStorageWrite)(void*,const uint8_t*,uint32_t);
typedef struct{void*context;core4VisionParamsStorageRead read;core4VisionParamsStorageEraseFn erase;core4VisionParamsStorageWrite write;}
core4VisionParamsStorageBackend;
typedef
enum{CORE4_VISION_PARAMS_LOAD_BLANK=0,CORE4_VISION_PARAMS_LOAD_VALID,CORE4_VISION_PARAMS_LOAD_READ_FAILED,CORE4_VI
SION_PARAMS_LOAD_REWRITE_FAILED,CORE4_VISION_PARAMS_LOAD_DEFAULTS_REWRITTEN,CORE4_VISION_PARAMS_LOAD_BLANK_INITI
ALIZED,CORE4_VISION_PARAMS_LOAD_BLANK_INIT_FAILED,CORE4_VISION_PARAMS_LOAD_PAGE_CONFLICT} core4VisionParamsLoadStatus;
typedef
enum{CORE4_VISION_PARAMS_SAVE_VERIFIED=0,CORE4_VISION_PARAMS_SAVE_ERASE_FAILED,CORE4_VISION_PARAMS_SAVE_WRITE_FAILED,
CORE4_VISION_PARAMS_SAVE_READ_FAILED,CORE4_VISION_PARAMS_SAVE_VERIFY_FAILED} core4VisionParamsSaveStatus;
core4VisionParamsLoadStatus core4VisionParamsStorageLoad(const core4VisionParamsStorageBackend*,const
core4VisionParamsStruct*,core4VisionParamsStruct*);
core4VisionParamsLoadStatus core4VisionParamsStorageInitializeBlank(const core4VisionParamsStorageBackend*,const
core4VisionParamsStruct*,core4VisionParamsStruct*);
core4VisionParamsSaveStatus core4VisionParamsStorageSave(const core4VisionParamsStorageBackend*,const core4VisionParamsStruct*);
#endif
```

core4_cyt4bb7/project/code/config/pinMap.h

SHA-256

:

5f2c4018d419d1df91e6472ad1ad54e4c5905c1429aeab37b89f5722787cc842

代码 (c)

```
#ifndef CORE4_PIN_MAP_H
#define CORE4_PIN_MAP_H

/* Core4 只登记本工程需要选择或约束的引脚：MT9V03X 与 IPS114 使用 CYT4 逐飞默认引脚。
* 业务 UART5 使用 P02，屏幕使用 SPI1，二者互不复用。
*/

/* Core1 业务专线：4 Mbps，Core1 P03_1 TX -> Core4 P02_0 RX。 */
#define CORE4_CORE1_UART_NUM (UART_5)
#define CORE4_CORE1_UART_RX_PIN (UART5_RX_P02_0)
#define CORE4_CORE1_UART_TX_PIN (UART5_TX_P02_1)
#define CORE4_CORE1_UART_SCB (SCB7)

/* 共阳极 RGB 灯珠，低电平点亮。 */
#define CORE4_RGB_LED_R_PWM (TCPWM_CH09_P05_0)
#define CORE4_RGB_LED_G_PWM (TCPWM_CH11_P05_2)
#define CORE4_RGB_LED_B_PWM (TCPWM_CH10_P05_1)

#endif /* CORE4_PIN_MAP_H */
```


core4_cyt4bb7/project/code/platform/core4InterruptPlatform.c

SHA-256

:

c32e0e413cf9d5d523b4d5df04e8a8034e99c49937a8f5b636c4566148850e4c

代码 (c)

```
#include "zf_common_interrupt.h"

void interrupt_global_enable(uint32 primask)
{
    __set_PRIMASK(primask);
}

uint32 interrupt_global_disable(void)
{
    uint32 primask = __get_PRIMASK();
    __disable_irq();
    return primask;
}

void assert_interrupt_config(void)
{
    __disable_irq();
}

void interrupt_set_priority(IRQn_Type irqn, uint8 priority)
{
    NVIC_SetPriority(irqn, priority);
}

void interrupt_init(cy_stc_sysint_irq_t *isrConfig,
    cy_systemIntr_Handler userIsrFunc, uint8 priority)
{
    Cy_SysInt_InitIRQ(isrConfig);
    Cy_SysInt_SetSystemIrqVector(isrConfig->sysIntSrc, userIsrFunc);
    interrupt_set_priority(isrConfig->intIdx, priority);
    NVIC_ClearPendingIRQ(isrConfig->intIdx);
    NVIC_EnableIRQ(isrConfig->intIdx);
}
```

core4_cyt4bb7/project/code/util/crc32leee.c

SHA-256

:

363e996887ef0e2c68b3518217986f209f0760bcd89ff2ca5cdd3a4801583aba

代码 (c)

```
#include "crc32leee.h"

uint32_t crc32leeeCalc(const uint8_t* data, uint32_t len)
{
    uint32_t crc = 0xFFFFFFFFu;
    uint32_t i;

    for (i = 0u; i < len; i++)
    {
        uint8_t b;
        crc ^= (uint32_t) data[i];
        for (b = 0u; b < 8u; b++)
        {
            uint32_t mask = (uint32_t) (~((int32_t) (crc & 1u)));
            crc = (crc >> 1) ^ (0xEDB88320u & mask);
        }
    }
    return ~crc;
}
```

core4_cyt4bb7/project/code/util/crc32ieee.h

SHA-256

:

5442cded78a9802395246ceee07532bb7e5de239f12b419b4ecc309d2c9e1001

代码 (c)

```
#ifndef CRC32_IEEE_H
#define CRC32_IEEE_H

#include <stdint.h>

uint32_t crc32IeeeCalc(const uint8_t* data, uint32_t len);

#endif
```

core4_cyt4bb7/project/code/vision/beaconTrack.c

SHA-256

:

65ae1cb42782586e2583a21a7320b5ec9be268a374c2f318997e90ed6fdf9346

代码 (c)

```
#include "beaconTrack.h"

#include <string.h>

#include "commonDef.h"
#include "configBeaconTrack.h"

/*
 * 时间一致性跟踪实现。逐帧:
 * 1) 全局选取综合 score 最小且满足连续性(距离+面积趋势+面积比)的轨迹-检测对,
 * 关联则平滑更新+hits++, 否则 misses++, hits 归零。坐标速度单位 px/frame,
 * 面积速度单位 area pixels/frame。
 * 2) 删除丢失超限的轨迹(已确认容忍 DROP_MISSES, 未确认容忍 1 帧)。
 * 3) 未关联的检测生成新轨迹。
 * 4) 输出: 已确认且 misses<=COAST_MISSES 的轨迹(滑行容忍偶发漏检)。
 * 定长数组 + 无动态内存; 纯前台调用(非 ISR)。
 */

static float beaconTrackDist2(float ax, float ay, float bx, float by)
{
    float dx = ax - bx;
    float dy = ay - by;
    return dx * dx + dy * dy;
}

static float beaconTrackAbs(float v)
{
    return (v < 0.0f) ? -v : v;
}

static float beaconTrackMax(float a, float b)
{
    return (a > b) ? a : b;
}

static uint16_t beaconTrackClampCoord(float v, uint16_t maxExclusive)
{
    float maxV = (maxExclusive > 0u) ? (float) (maxExclusive - 1u) : 0.0f;
    v = clampf(v, 0.0f, maxV);
    return (uint16_t) (v + 0.5f);
}

static uint16_t beaconTrackClampSize(float v)
{
    v = clampf(v, 0.0f, (float) OM_BUFFER_SIZE);
    return (uint16_t) (v + 0.5f);
}
```

```

static uint8_t beaconTrackIdActive(const beaconTrackerStruct* tracker, uint16_t id)
{
    if (id == 0u) return 1u;
    for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
    {
        if (tracker->tracks[t].active && tracker->tracks[t].trackId == id) return 1u;
    }
    return 0u;
}

static uint16_t beaconTrackAllocId(beaconTrackerStruct* tracker)
{
    uint16_t id;
    for (uint16_t tries = 0u; tries < 0xFFFFu; tries++)
    {
        id = tracker->nextTrackId;
        tracker->nextTrackId++;
        if (tracker->nextTrackId == 0u) tracker->nextTrackId = 1u;
        if (!beaconTrackIdActive(tracker, id)) return id;
    }
    return 0u;
}

static float beaconTrackPredictFramesForOutput(const beaconTrackStruct* tr)
{
    return (tr->misses > 0u) ? (float) tr->misses : VISION_TRACK_PREDICT_FRAMES_DEFAULT;
}

static float beaconTrackPredX(const beaconTrackStruct* tr, float frames)
{
    return tr->lastRawX + tr->velX * frames;
}

static float beaconTrackPredY(const beaconTrackStruct* tr, float frames)
{
    return tr->lastRawY + tr->velY * frames;
}

static float beaconTrackPredSize(const beaconTrackStruct* tr, float frames)
{
    return clampf(tr->lastRawSize + tr->areaVel * frames, 0.0f, (float) OM_BUFFER_SIZE);
}

static float beaconTrackAreaVelDenom(float size)
{
    float areaBase = beaconTrackMax(size, VISION_TRACK_AREA_NORM_MIN);
    return beaconTrackMax(areaBase * VISION_TRACK_ASSOC_AREA_VEL_NORM,
        VISION_TRACK_ASSOC_AREA_VEL_MIN_ABS);
}

static float beaconTrackAssociationScore(float d2, float maxDist2, float detSize,
    float predSize, float rawAreaVel, float areaVelBaseSize,
    float expectedAreaVel, uint8_t areaVelPrimed,
    float areaWeight, float areaVelWeight)
{
    float predDenom = beaconTrackMax(predSize, VISION_TRACK_AREA_NORM_MIN);
    float areaErr = beaconTrackAbs(detSize - predSize) / predDenom;
    float areaScore;
    float velScore = 0.0f;

    areaErr = clampf(areaErr, 0.0f, VISION_TRACK_ASSOC_AREA_ERR_CAP);
    areaScore = areaErr * areaErr;

    if (areaVelPrimed)
    {
        float velErr = beaconTrackAbs(rawAreaVel - expectedAreaVel) / beaconTrackAreaVelDenom(areaVelBaseSize);
        velErr = clampf(velErr, 0.0f, VISION_TRACK_ASSOC_AREA_VEL_ERR_CAP);
        velScore = velErr * velErr;
    }

    return (d2 / maxDist2)
        + areaWeight * areaScore
        + areaVelWeight * velScore;
}

static uint8_t beaconTrackPairBetter(float score, float d2, uint16_t trackId, uint16_t detIdx,
    float bestScore, float bestD2,
    uint16_t bestTrackId, uint16_t bestDetIdx)

```

```

{
    const float eps = 0.0001f;
    if (score < (bestScore - eps)) return 1u;
    if (score > (bestScore + eps)) return 0u;
    if (d2 < (bestD2 - eps)) return 1u;
    if (d2 > (bestD2 + eps)) return 0u;
    if (trackId < bestTrackId) return 1u;
    if (trackId > bestTrackId) return 0u;
    return (detIdx < bestDetIdx) ? 1u : 0u;
}

/* 运行时跟踪参数(默认取 configBeaconTrack.h 的 #define, 菜单可热重载)。
* 仅 Core4 前台读写, 无 ISR 竞争, 无需 volatile。 */
typedef struct
{
    float maxAssocDist;
    float sizeRatioMax;
    uint16_t confirmHits;
    uint16_t dropMisses;
    uint16_t coastMisses;
    float positionAlpha; /* 关联更新位置低通系数(0<alpha<=1) */
    float sizeAlpha; /* 关联更新面积低通系数(0<alpha<=1) */
    float areaVelAlpha; /* 面积速度低通系数(0<alpha<=1) */
    float assocScoreMax;
    float assocAreaWeight;
    float assocAreaVelWeight;
    beaconTrackBorderConfigStruct border;
    uint16_t edgePredictionHoldFrames;
    beaconTrackEdgeBandsStruct edgeBands;
} beaconTrackConfigStruct;

#define BEACON_EDGE_SIDE_LEFT (0x01u)
#define BEACON_EDGE_SIDE_RIGHT (0x02u)
#define BEACON_EDGE_SIDE_TOP (0x04u)
#define BEACON_EDGE_SIDE_BOTTOM (0x08u)

static beaconTrackConfigStruct beaconTrackConfig = {
    VISION_TRACK_MAX_ASSOC_DIST_DEFAULT,
    VISION_TRACK_SIZE_RATIO_MAX_DEFAULT,
    VISION_TRACK_CONFIRM_HITS_DEFAULT,
    VISION_TRACK_DROP_MISSES_DEFAULT,
    VISION_TRACK_COAST_MISSES_DEFAULT,
    VISION_TRACK_POSITION_ALPHA_DEFAULT,
    VISION_TRACK_SIZE_ALPHA_DEFAULT,
    VISION_TRACK_AREA_VEL_ALPHA_DEFAULT,
    VISION_TRACK_ASSOC_SCORE_MAX_DEFAULT,
    VISION_TRACK_ASSOC_AREA_WEIGHT_DEFAULT,
    VISION_TRACK_ASSOC_AREA_VEL_WEIGHT_DEFAULT,
    {
        {VISION_TRACK_BORDER_ENTER_LEFT_DEFAULT,
        VISION_TRACK_BORDER_ENTER_RIGHT_DEFAULT,
        VISION_TRACK_BORDER_ENTER_TOP_DEFAULT,
        VISION_TRACK_BORDER_ENTER_BOTTOM_DEFAULT},
        {VISION_TRACK_BORDER_EXIT_LEFT_DEFAULT,
        VISION_TRACK_BORDER_EXIT_RIGHT_DEFAULT,
        VISION_TRACK_BORDER_EXIT_TOP_DEFAULT,
        VISION_TRACK_BORDER_EXIT_BOTTOM_DEFAULT}
    },
    0u,
    {VISION_TRACK_EDGE_BAND_OBSERVER_LEFT_DEFAULT,
    VISION_TRACK_EDGE_BAND_OBSERVER_RIGHT_DEFAULT,
    VISION_TRACK_EDGE_BAND_OBSERVER_TOP_DEFAULT,
    VISION_TRACK_EDGE_BAND_OBSERVER_BOTTOM_DEFAULT}
};

static float beaconTrackPredictFramesForAssoc(const beaconTrackStruct* tr)
{
    float frames = (float) tr->misses + VISION_TRACK_PREDICT_FRAMES_DEFAULT;
    if ((tr->edgePredictionSideMask != 0u) &&
        (beaconTrackConfig.edgePredictionHoldFrames != 0u))
    {
        frames = clampf(frames, VISION_TRACK_PREDICT_FRAMES_DEFAULT,
            VISION_TRACK_EDGE_ASSOC_PREDICT_FRAMES_MAX);
    }
    return frames;
}

static uint8_t beaconTrackEdgeSideMask(float x, float y)
{

```

```

uint8_t mask = 0u;
if ((beaconTrackConfig.edgePredictionHoldFrames == 0u) return 0u;
if ((beaconTrackConfig.edgeBands.left != 0u) &&
    (x < (float) beaconTrackConfig.edgeBands.left)) mask |= BEACON_EDGE_SIDE_LEFT;
if ((beaconTrackConfig.edgeBands.right != 0u) &&
    (x >= (float) OM_IMAGE_MAX_WIDTH - (float) beaconTrackConfig.edgeBands.right))
    mask |= BEACON_EDGE_SIDE_RIGHT;
if ((beaconTrackConfig.edgeBands.top != 0u) &&
    (y < (float) beaconTrackConfig.edgeBands.top)) mask |= BEACON_EDGE_SIDE_TOP;
if ((beaconTrackConfig.edgeBands.bottom != 0u) &&
    (y >= (float) OM_IMAGE_MAX_HEIGHT - (float) beaconTrackConfig.edgeBands.bottom))
    mask |= BEACON_EDGE_SIDE_BOTTOM;
return mask;
}

static uint8_t beaconTrackSideValid(uint8_t enter, uint8_t exit, uint8_t maximum)
{
    if ((enter == 0u) && (exit == 0u)) return 1u;
    return (uint8_t) ((enter >= 1u) && (enter <= maximum) && (exit < enter));
}

static uint8_t beaconTrackBorderValid(const beaconTrackBorderConfigStruct* margins)
{
    if (margins == 0) return 0u;
    return (uint8_t) (
        beaconTrackSideValid(margins->enter.left, margins->exit.left, VISION_TRACK_BORDER_X_MAX) &&
        beaconTrackSideValid(margins->enter.right, margins->exit.right, VISION_TRACK_BORDER_X_MAX) &&
        beaconTrackSideValid(margins->enter.top, margins->exit.top, VISION_TRACK_BORDER_Y_MAX) &&
        beaconTrackSideValid(margins->enter.bottom, margins->exit.bottom, VISION_TRACK_BORDER_Y_MAX));
}

static uint8_t beaconTrackPointInside(float x, float y,
    const beaconTrackBorderMarginsStruct* margin)
{
    return (uint8_t) (((margin->left == 0u) || (x >= (float) margin->left)) &&
        ((margin->right == 0u) || (x <= (float) (OM_IMAGE_MAX_WIDTH - 1u - margin->right))) &&
        ((margin->top == 0u) || (y >= (float) margin->top)) &&
        ((margin->bottom == 0u) || (y <= (float) (OM_IMAGE_MAX_HEIGHT - 1u - margin->bottom))));
}

static uint8_t beaconTrackUpdateBorderExit(beaconTrackStruct* tr, float x, float y)
{
    if (beaconTrackPointInside(x, y, &beaconTrackConfig.border.exit) != 0u)
    {
        tr->borderOutsideFrames = 0u;
        return 0u;
    }

    if (tr->borderOutsideFrames < 0xFFu) tr->borderOutsideFrames++;
    if (tr->borderOutsideFrames >= VISION_TRACK_BORDER_EXIT_HOLD_FRAMES_DEFAULT)
    {
        tr->borderEligible = 0u;
        return 1u;
    }
    return 0u;
}

void beaconTrackSetBorderMargins(const beaconTrackBorderConfigStruct* margins)
{
    if (beaconTrackBorderValid(margins) != 0u)
    {
        beaconTrackConfig.border = *margins;
    }
}

void beaconTrackSetEdgePredictionConfig(
    const beaconTrackEdgePredictionConfigStruct* config)
{
    uint16_t holdMs;
    if (config == 0) return;
    holdMs = config->holdMs;
    if (holdMs > VISION_TRACK_EDGE_PREDICTION_HOLD_MS_MAX)
        holdMs = VISION_TRACK_EDGE_PREDICTION_HOLD_MS_MAX;
    beaconTrackConfig.edgePredictionHoldFrames = (uint16_t)
        ((holdMs + VISION_TRACK_CAMERA_FRAME_PERIOD_MS - 1u) /
            VISION_TRACK_CAMERA_FRAME_PERIOD_MS);
    beaconTrackConfig.edgeBands = config->bands;
    if (beaconTrackConfig.edgeBands.left > (uint8_t) VISION_TRACK_EDGE_BAND_X_MAX)
        beaconTrackConfig.edgeBands.left = (uint8_t) VISION_TRACK_EDGE_BAND_X_MAX;

```

```

    if (beaconTrackConfig.edgeBands.right > (uint8_t) VISION_TRACK_EDGE_BAND_X_MAX)
        beaconTrackConfig.edgeBands.right = (uint8_t) VISION_TRACK_EDGE_BAND_X_MAX;
    if (beaconTrackConfig.edgeBands.top > (uint8_t) VISION_TRACK_EDGE_BAND_Y_MAX)
        beaconTrackConfig.edgeBands.top = (uint8_t) VISION_TRACK_EDGE_BAND_Y_MAX;
    if (beaconTrackConfig.edgeBands.bottom > (uint8_t) VISION_TRACK_EDGE_BAND_Y_MAX)
        beaconTrackConfig.edgeBands.bottom = (uint8_t) VISION_TRACK_EDGE_BAND_Y_MAX;
}

void beaconTrackSetParams(float maxAssocDist, float sizeRatioMax,
    uint16_t confirmHits, uint16_t dropMisses, uint16_t coastMisses,
    float positionAlpha, float sizeAlpha,
    float areaVelAlpha, float assocScoreMax,
    float assocAreaWeight, float assocAreaVelWeight)
{
    if (maxAssocDist > 0.0f) beaconTrackConfig.maxAssocDist = maxAssocDist;
    if (sizeRatioMax > 1.0f) beaconTrackConfig.sizeRatioMax = sizeRatioMax;
    if (confirmHits > 0u) beaconTrackConfig.confirmHits = confirmHits;
    if (dropMisses > 0u) beaconTrackConfig.dropMisses = dropMisses;
    if (coastMisses > 0u) beaconTrackConfig.coastMisses = coastMisses;
    if (positionAlpha > 0.0f && positionAlpha <= 1.0f) beaconTrackConfig.positionAlpha = positionAlpha;
    if (sizeAlpha > 0.0f && sizeAlpha <= 1.0f) beaconTrackConfig.sizeAlpha = sizeAlpha;
    if (areaVelAlpha > 0.0f && areaVelAlpha <= 1.0f) beaconTrackConfig.areaVelAlpha = areaVelAlpha;
    if (assocScoreMax > 0.0f) beaconTrackConfig.assocScoreMax = assocScoreMax;
    if (assocAreaWeight >= 0.0f) beaconTrackConfig.assocAreaWeight = assocAreaWeight;
    if (assocAreaVelWeight >= 0.0f) beaconTrackConfig.assocAreaVelWeight = assocAreaVelWeight;
}

void beaconTrackInit(beaconTrackerStruct* tracker)
{
    if (0 == tracker) return;
    for (uint16_t i = 0u; i < OM_MAX_OBJECTS; i++)
    {
        tracker->tracks[i].active = 0u;
    }
    tracker->confirmedCount = 0u;
    tracker->nextTrackId = 1u;
    tracker->frameSeq = 0u;
}

void beaconTrackMarkPublished(beaconTrackerStruct* tracker, uint16_t trackId)
{
    uint16_t t;
    if ((tracker == 0) || (trackId == 0u)) return;
    for (t = 0u; t < OM_MAX_OBJECTS; t++)
    {
        beaconTrackStruct *track = &tracker->tracks[t];
        if ((track->active != 0u) && (track->trackId == trackId))
        {
            track->published = 1u;
            return;
        }
    }
}

void beaconTrackUpdate(beaconTrackerStruct* tracker, const BEACON_DETECTOR_STRUCT* det)
{
    if (0 == tracker || 0 == det) return;

    uint16_t nDet = det->beaconCount;
    uint16_t outCount = 0u;
    if (nDet > OM_MAX_OBJECTS) nDet = OM_MAX_OBJECTS;

    uint8_t detUsed[OM_MAX_OBJECTS];
    uint8_t trackUsed[OM_MAX_OBJECTS];
    for (uint16_t i = 0u; i < nDet; i++) detUsed[i] = 0u;
    for (uint16_t i = 0u; i < OM_MAX_OBJECTS; i++) trackUsed[i] = 0u;
    tracker->frameSeq++;

    const float maxDist2 = beaconTrackConfig.maxAssocDist * beaconTrackConfig.maxAssocDist;
    const float ratioMax = beaconTrackConfig.sizeRatioMax;
    const float ratioMin = 1.0f / beaconTrackConfig.sizeRatioMax;
    const float velMax = beaconTrackConfig.maxAssocDist * VISION_TRACK_VELOCITY_MAX_FACTOR_DEFAULT;

    /* EdgeBand 热重载必须先于本帧关联生效，禁止缩小\关闭后再借旧资格
    * 多使用一次纵向放宽门。扩大参数只影响仍存活轨迹。 */
    for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
    {
        beaconTrackStruct* tr = &tracker->tracks[t];
        if (!tr->active) continue;

```

```

tr->edgePredictionSideMask = beaconTrackEdgeSideMask(
    tr->lastRawX, tr->lastRawY);
}

/* 1) 全局选择预测残差最小的兼容轨迹-检测对，避免槽位顺序先抢检测。 */
for (;;)
{
    int32_t bestTrack = -1;
    int32_t bestDet = -1;
    uint16_t bestTrackId = 0xFFFFu;
    uint16_t bestDetIdx = 0xFFFFu;
    float bestScore = beaconTrackConfig.assocScoreMax;
    float bestD2 = 1.0f;

    for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
    {
        beaconTrackStruct* tr = &tracker->tracks[t];
        if (!tr->active || trackUsed[t]) continue;

        float frames = beaconTrackPredictFramesForAssoc(tr);
        float predX = beaconTrackPredX(tr, frames);
        float predY = beaconTrackPredY(tr, frames);
        float predSize = beaconTrackPredSize(tr, frames);
        float measurementFrames = (float) tr->misses + 1.0f;
        float predSizeDenom = beaconTrackMax(predSize, VISION_TRACK_AREA_NORM_MIN);
        for (uint16_t d = 0u; d < nDet; d++)
        {
            if (detUsed[d]) continue;
            float dcx = (float) det->beaconData[d].centerX;
            float dcy = (float) det->beaconData[d].centerY;
            float detSize = (float) det->beaconData[d].contourSize;
            float rawAreaVel = (detSize - tr->lastRawSize) / measurementFrames;
            float d2 = beaconTrackDist2(dcx, dcy, predX, predY);
            float positionScore;
            float ratio;
            float score;
            if ((tr->edgePredictionSideMask != 0u) &&
                (beaconTrackConfig.edgePredictionHoldFrames != 0u))
            {
                float dx = dcx - predX;
                float dy = dcy - predY;
                float rx = beaconTrackConfig.maxAssocDist;
                float ry = beaconTrackConfig.maxAssocDist;
                if ((tr->edgePredictionSideMask &
                    (BEACON_EDGE_SIDE_LEFT | BEACON_EDGE_SIDE_RIGHT)) != 0u)
                    rx *= VISION_TRACK_EDGE_VERTICAL_ASSOC_SCALE;
                if ((tr->edgePredictionSideMask &
                    (BEACON_EDGE_SIDE_TOP | BEACON_EDGE_SIDE_BOTTOM)) != 0u)
                    ry *= VISION_TRACK_EDGE_VERTICAL_ASSOC_SCALE;
                positionScore = (dx * dx) / (rx * rx) + (dy * dy) / (ry * ry);
                if (positionScore > 1.0f) continue;
            }
            else
            {
                if (d2 > maxDist2) continue;
                positionScore = d2 / maxDist2;
            }
            ratio = detSize / predSizeDenom;
            if (ratio > ratioMax || ratio < ratioMin) continue;
            score = beaconTrackAssociationScore(positionScore, 1.0f, detSize, predSize, rawAreaVel,
                tr->lastRawSize,
                tr->areaVel, tr->areaVelocityPrimed,
                beaconTrackConfig.assocAreaWeight,
                beaconTrackConfig.assocAreaVelWeight);
            if (score > beaconTrackConfig.assocScoreMax) continue;
            if (bestTrack < 0
                || beaconTrackPairBetter(score, positionScore, tr->trackId, d,
                    bestScore, bestD2, bestTrackId, bestDetIdx))
            {
                bestScore = score;
                bestD2 = positionScore;
                bestTrack = (int32_t) t;
                bestDet = (int32_t) d;
                bestTrackId = tr->trackId;
                bestDetIdx = d;
            }
        }
    }
}

```

```

if (bestTrack < 0 || bestDet < 0) break;

{
    uint16_t t = (uint16_t) bestTrack;
    uint16_t d = (uint16_t) bestDet;
    beaconTrackStruct* tr = &tracker->tracks[t];
    float aP = beaconTrackConfig.positionAlpha;
    float aS = beaconTrackConfig.sizeAlpha;
    float dcx = (float) det->beaconData[d].centerX;
    float dcy = (float) det->beaconData[d].centerY;
    float detSize = (float) det->beaconData[d].contourSize;
    float measurementFrames = (float) tr->misses + 1.0f;
    float rawVx = (dcx - tr->lastRawX) / measurementFrames;
    float rawVy = (dcy - tr->lastRawY) / measurementFrames;
    float rawAreaVel = (detSize - tr->lastRawSize) / measurementFrames;
    float areaVelMax = beaconTrackMax(tr->lastRawSize * VISION_TRACK_AREA_VEL_MAX_RATE,
        VISION_TRACK_ASSOC_AREA_VEL_MIN_ABS);
    rawVx = clampf(rawVx, -velMax, velMax);
    rawVy = clampf(rawVy, -velMax, velMax);
    rawAreaVel = clampf(rawAreaVel, -areaVelMax, areaVelMax);
    if (tr->velocityPrimed)
    {
        float aV = VISION_TRACK_VELOCITY_ALPHA_DEFAULT;
        tr->velX = aV * rawVx + (1.0f - aV) * tr->velX;
        tr->velY = aV * rawVy + (1.0f - aV) * tr->velY;
    }
    else
    {
        tr->velX = rawVx;
        tr->velY = rawVy;
        tr->velocityPrimed = 1u;
    }
    if (tr->areaVelocityPrimed)
    {
        float aA = beaconTrackConfig.areaVelAlpha;
        tr->areaVel = aA * rawAreaVel + (1.0f - aA) * tr->areaVel;
    }
    else
    {
        tr->areaVel = rawAreaVel;
        tr->areaVelocityPrimed = 1u;
    }
    tr->areaVel = clampf(tr->areaVel, -areaVelMax, areaVelMax);
    tr->velX = clampf(tr->velX, -velMax, velMax);
    tr->velY = clampf(tr->velY, -velMax, velMax);
    tr->lastRawX = dcx;
    tr->lastRawY = dcy;
    tr->lastRawSize = detSize;
    trackUsed[t] = 1u;
    detUsed[d] = 1u;
    /* 关联更新一阶低通(alpha*measured + (1-alpha)*old; 越小越平滑, 替代原硬编码 0.5)。 */
    tr->cx = aP * dcx + (1.0f - aP) * tr->cx;
    tr->cy = aP * dcy + (1.0f - aP) * tr->cy;
    tr->size = aS * detSize + (1.0f - aS) * tr->size;
    if (tr->hits < 0xFFFFu) tr->hits++;
    tr->misses = 0u;
    tr->fresh = 1u;
    tr->edgePredictionSideMask = beaconTrackEdgeSideMask(dcx, dcy);
    if (tr->borderEligible == 0u)
    {
        tr->borderEligible = beaconTrackPointInside(dcx, dcy,
            &beaconTrackConfig.border.enter);
        if (tr->borderEligible != 0u) tr->borderOutsideFrames = 0u;
    }
    tr->frameSeq = tracker->frameSeq;
    if (tr->hits >= beaconTrackConfig.confirmHits) tr->confirmed = 1u;
}
}

for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
{
    beaconTrackStruct* tr = &tracker->tracks[t];
    if (!tr->active || trackUsed[t]) continue;
    if (tr->misses < 0xFFFFu) tr->misses++;
    tr->hits = 0u;
    tr->fresh = 0u;
    tr->frameSeq = tracker->frameSeq;
}

```



```

/* 2) 边界拒绝优先于丢失删除和边缘预测。所有从 Enter 区取得资格的
* 活动轨迹都参与 Exit 确认；只有已经确认发布过的轨迹产生一次拒绝事实。*/
for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
{
    beaconTrackStruct* tr = &tracker->tracks[t];
    float predFrames;
    float predX;
    float predY;
    float qualificationX;
    float qualificationY;
    if (!tr->active || (tr->borderEligible == 0u)) continue;
    predFrames = beaconTrackPredictFramesForOutput(tr);
    predX = beaconTrackPredX(tr, predFrames);
    predY = beaconTrackPredY(tr, predFrames);
    qualificationX = (tr->fresh != 0u) ? tr->lastRawX : predX;
    qualificationY = (tr->fresh != 0u) ? tr->lastRawY : predY;
    if (beaconTrackUpdateBorderExit(tr, qualificationX,
        qualificationY) == 0u) continue;
    if (tr->published != 0u)
    {
        memset(&tracker->confirmed[outCount], 0,
            sizeof(tracker->confirmed[outCount]));
        tracker->confirmed[outCount].trackId = tr->trackId;
        tracker->confirmed[outCount].flags =
            BEACON_CONFIRMED_FLAG_BORDER_REJECTED;
        outCount++;
    }
    tr->edgePredictionSideMask = 0u;
    tr->active = 0u;
    tr->confirmed = 0u;
    tr->fresh = 0u;
}

/* 3) 删除丢失超限轨迹 */
for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
{
    beaconTrackStruct* tr = &tracker->tracks[t];
    if (!tr->active) continue;
    uint16_t limit = tr->confirmed ? beaconTrackConfig.dropMisses : 1u;
    if (tr->confirmed && (tr->edgePredictionSideMask != 0u) &&
        (beaconTrackConfig.edgePredictionHoldFrames != 0u))
    {
        limit = beaconTrackConfig.edgePredictionHoldFrames;
    }
    if (tr->misses > limit)
    {
        tr->active = 0u;
        tr->fresh = 0u;
    }
}

/* 4) 未关联检测生成新轨迹(填首个空槽) */
for (uint16_t d = 0u; d < nDet; d++)
{
    if (detUsed[d]) continue;
    if (beaconTrackPointInside((float) det->beaconData[d].centerX,
        (float) det->beaconData[d].centerY,
        &beaconTrackConfig.border.enter) == 0u) continue;
    for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
    {
        beaconTrackStruct* tr = &tracker->tracks[t];
        uint16_t newTrackId;
        if (tr->active) continue;
        newTrackId = beaconTrackAllocId(tracker);
        if (newTrackId == 0u) break;
        tr->cx = (float) det->beaconData[d].centerX;
        tr->cy = (float) det->beaconData[d].centerY;
        tr->size = (float) det->beaconData[d].contourSize;
        tr->lastRawX = tr->cx;
        tr->lastRawY = tr->cy;
        tr->lastRawSize = tr->size;
        tr->velX = 0.0f;
        tr->velY = 0.0f;
        tr->areaVel = 0.0f;
        tr->frameSeq = tracker->frameSeq;
        tr->trackId = newTrackId;
        tr->hits = 1u;
        tr->misses = 0u;
        tr->confirmed = 0u;
    }
}

```

```

        tr->fresh = 1u;
        tr->velocityPrimed = 0u;
        tr->areaVelocityPrimed = 0u;
        tr->borderEligible = beaconTrackPointInside(tr->cx, tr->cy,
            &beaconTrackConfig.border.enter);
        tr->borderOutsideFrames = 0u;
        tr->edgePredictionSideMask = beaconTrackEdgeSideMask(tr->cx, tr->cy);
        tr->published = 0u;
        tr->active = 1u;
        break;
    }
}

/* 5) 输出已确认且仍新鲜(滑行容忍)的轨迹。拒绝记录已在前面。 */
for (uint16_t t = 0u; t < OM_MAX_OBJECTS; t++)
{
    beaconTrackStruct* tr = &tracker->tracks[t];
    if (!tr->active || !tr->confirmed) continue;
    {
        uint16_t outputLimit = beaconTrackConfig.coastMisses;
        if ((tr->edgePredictionSideMask != 0u) &&
            (beaconTrackConfig.edgePredictionHoldFrames != 0u))
        {
            outputLimit = beaconTrackConfig.edgePredictionHoldFrames;
        }
        if (tr->misses > outputLimit) continue;
    }
    float predFrames = beaconTrackPredictFramesForOutput(tr);
    float predX = beaconTrackPredX(tr, predFrames);
    float predY = beaconTrackPredY(tr, predFrames);
    float predSize = beaconTrackPredSize(tr, predFrames);
    if (tr->borderEligible == 0u) continue;
    tracker->confirmed[outCount].centerX = (uint16_t) (tr->cx + 0.5f);
    tracker->confirmed[outCount].centerY = (uint16_t) (tr->cy + 0.5f);
    tracker->confirmed[outCount].size = beaconTrackClampSize(tr->size);
    tracker->confirmed[outCount].trackId = tr->trackId;
    tracker->confirmed[outCount].predX = beaconTrackClampCoord(predX, OM_IMAGE_MAX_WIDTH);
    tracker->confirmed[outCount].predY = beaconTrackClampCoord(predY, OM_IMAGE_MAX_HEIGHT);
    tracker->confirmed[outCount].predSize = beaconTrackClampSize(predSize);
    tracker->confirmed[outCount].misses = tr->misses;
    tracker->confirmed[outCount].fresh = tr->fresh;
    tracker->confirmed[outCount].flags =
        (tr->fresh ? BEACON_CONFIRMED_FLAG_FRESH : 0u) |
        (tr->misses > 0u ? BEACON_CONFIRMED_FLAG_COAST : 0u) |
        ((!tr->fresh && (tr->edgePredictionSideMask != 0u) &&
            (beaconTrackConfig.edgePredictionHoldFrames != 0u))
            ? BEACON_CONFIRMED_FLAG_EDGE_GRACE : 0u);
    tracker->confirmed[outCount].frameSeq = tr->frameSeq;
    tracker->confirmed[outCount].velX = tr->velX;
    tracker->confirmed[outCount].velY = tr->velY;
    tracker->confirmed[outCount].areaVel = tr->areaVel;
    outCount++;
}
tracker->confirmedCount = outCount;
}

```

core4_cyt4bb7/project/code/vision/beaconTrack.h

SHA-256

:

11c7aec427c49971f12e26ea0373a59a1df794dfa56bc68227ab2018e427a2cf

代码 (c)

```

#ifndef BEACON_TRACK_H
#define BEACON_TRACK_H

#include <stdint.h>

#include "openMorph.h" /* BEACON_DETECTOR_STRUCT / BEACON_DATA_STRUCT / OM_MAX_OBJECTS */

#define BEACON_CONFIRMED_FLAG_FRESH    (0x01u)
#define BEACON_CONFIRMED_FLAG_COAST    (0x02u)
#define BEACON_CONFIRMED_FLAG_EDGE_GRACE (0x04u)
#define BEACON_CONFIRMED_FLAG_BORDER_REJECTED (0x08u)

```

```

/*
 * beaconTrack - 信标时间一致性跟踪（抗光关键层）
 *
 * 单帧检测器（运行时 BinaryCC / Legacy Canny 二选一）可能出现瞬时反光误检(FP)与小目标漏检(FN)。本模块在其上叠加
 * 跨帧关联 + 位置/面积连续性门控 + 稳定性计数：连续 confirmHits 帧关联上才
 * 确认输出、连续 dropMisses 帧丢失才删除。瞬时反光跨帧不连续，过不了确认
 * 门控被滤掉。历史 Legacy Canny 仿真(run_beacon_tracking_test.py): 单帧 4 FP → 跟踪 0 FP。
 *
 * 信标恒亮不闪、可多帧延迟，故时间一致性是合理且有效的抗光手段。
 */

/* 跟踪参数默认值已移至 config/configBeaconTrack.h(VISION_TRACK_*_DEFAULT)。 */

/* 单条轨迹 */
typedef struct
{
    float cx;          /* 平滑后质心 X */
    float cy;          /* 平滑后质心 Y */
    float size;        /* 平滑后像素数 */
    float lastRawX;     /* 上次关联的原始检测 X */
    float lastRawY;     /* 上次关联的原始检测 Y */
    float lastRawSize;  /* 上次关联的原始检测面积 */
    float velX;         /* 原始检测速度(px/frame) */
    float velY;         /* 原始检测速度(px/frame) */
    float areaVel;      /* 原始检测面积速度(area pixels/frame) */
    uint32_t frameSeq;  /* 最近一次输出所属图像帧序号 */
    uint16_t trackId;   /* 轨迹 ID；0 保留为无效 */
    uint16_t hits;      /* 连续命中帧数 */
    uint16_t misses;    /* 连续丢失帧数 */
    uint8_t confirmed;  /* 1=已确认(连续命中达阈值) */
    uint8_t fresh;      /* 1=本帧关联到真实检测 */
    uint8_t velocityPrimed; /* 1=velX/velY 已由连续两次检测建立 */
    uint8_t areaVelocityPrimed; /* 1=areaVel 已由连续两次检测建立 */
    uint8_t borderEligible; /* 1=已通过发布边界施密特门 */
    uint8_t borderOutsideFrames; /* 连续越过 Exit 边界的输出判定帧数 */
    uint8_t edgePredictionSideMask; /* 上次 fresh 命中的 EdgeBand 边位掩码 */
    uint8_t published; /* 至少产生过一帧普通发布记录 */
    uint8_t active;     /* 1=槽位占用 */
} beaconTrackStruct;

/* 确认输出的信标质心(供决策/规划消费) */
typedef struct
{
    uint16_t centerX;
    uint16_t centerY;
    uint16_t size;
    uint16_t trackId;
    uint16_t predX;
    uint16_t predY;
    uint16_t predSize;
    uint16_t misses;
    uint8_t fresh;
    uint8_t flags;
    uint32_t frameSeq;
    float velX;      /* px/frame */
    float velY;      /* px/frame */
    float areaVel;   /* area pixels/frame */
} beaconConfirmedStruct;

typedef struct
{
    uint8_t left;
    uint8_t right;
    uint8_t top;
    uint8_t bottom;
} beaconTrackBorderMarginsStruct;

typedef struct
{
    beaconTrackBorderMarginsStruct enter;
    beaconTrackBorderMarginsStruct exit;
} beaconTrackBorderConfigStruct;

typedef struct
{
    uint8_t left;
    uint8_t right;
    uint8_t top;
    uint8_t bottom;

```

```

} beaconTrackEdgeBandsStruct;

typedef struct
{
    uint16_t holdMs;
    beaconTrackEdgeBandsStruct bands;
} beaconTrackEdgePredictionConfigStruct;

/* 跟踪器 */
typedef struct
{
    beaconTrackStruct tracks[OM_MAX_OBJECTS];
    beaconConfirmedStruct confirmed[OM_MAX_OBJECTS]; /* update 后的确认输出 */
    uint16_t confirmedCount;
    uint16_t nextTrackId;
    uint32_t frameSeq;
} beaconTrackerStruct;

/* 初始化(清空所有轨迹) */
void beaconTrackInit(beaconTrackerStruct* tracker);

/* 运行时设置跟踪参数(默认取 configBeaconTrack.h #define, 菜单可热重载)。
 * positionAlpha/sizeAlpha: 关联更新一阶低通系数(0<alpha<=1, 越小越平滑; 替代原硬编码 0.5)。
 * 仅 Core4 前台 beaconTrackUpdate 读写, 无 ISR 竞争。 */
void beaconTrackSetParams(float maxAssocDist, float sizeRatioMax,
    uint16_t confirmHits, uint16_t dropMisses, uint16_t coastMisses,
    float positionAlpha, float sizeAlpha,
    float areaVelAlpha, float assocScoreMax,
    float assocAreaWeight, float assocAreaVelWeight);

void beaconTrackSetBorderMargins(const beaconTrackBorderConfigStruct* margins);
void beaconTrackSetEdgePredictionConfig(
    const beaconTrackEdgePredictionConfigStruct* config);

/* 用本帧 om_beacon 检测结果更新跟踪器; 更新后确认信标在 tracker->confirmed[0..confirmedCount-1] */
void beaconTrackUpdate(beaconTrackerStruct* tracker, const BEACON_DETECTOR_STRUCT* det);
void beaconTrackMarkPublished(beaconTrackerStruct* tracker, uint16_t trackId);

#endif /* BEACON_TRACK_H */

```

core4_cyt4bb7/project/code/vision/core4VisionWorkspace.c

SHA-256

:

4deecaf410949e750be46d828443f84e5880f6b5ccf00d051cc171c746bc1df

代码 (c)

```

#include "core4VisionWorkspace.h"

core4VisionWorkspaceStruct g_core4VisionWorkspace;

uint16_t contourGeneration = 0u;
const int8_t contourDx[8] = {-1, 0, 1, 1, 1, 0, -1, -1};
const int8_t contourDy[8] = {-1, -1, -1, 0, 1, 1, 1, 0};

```

core4_cyt4bb7/project/code/vision/core4VisionWorkspace.h

SHA-256

:

c78ac5fbec8dfe64d3312dc84b8d17f5f71c910ec70e9ea0b96f91a903a3a250

代码 (c)

```

#ifndef CORE4_VISION_WORKSPACE_H
#define CORE4_VISION_WORKSPACE_H

#include <stdint.h>

#include "openMorph_config.h"

typedef struct

```

```

{
    uint32_t visitMap[OM_BUFFER_SIZE];
    union
    {
        {
            uint32_t bfsQueue[OM_BUFFER_SIZE];
            struct
            {
                uint8_t gaussian[OM_BUFFER_SIZE];
                uint16_t edgeQueue[OM_BUFFER_SIZE];
            } canny;
        } scratch;
    } core4VisionWorkspaceStruct;

extern core4VisionWorkspaceStruct g_core4VisionWorkspace;

_Static_assert(OM_BUFFER_SIZE == (188u * 120u), "Core4 vision pixel count mismatch");
_Static_assert(sizeof(uint8_t) == 1u, "Core4 gray pixel must remain one byte");
_Static_assert(sizeof(uint16_t) == 2u, "Canny pixel index must remain two bytes");
_Static_assert(OM_BUFFER_SIZE <= UINT16_MAX, "Canny pixel index must fit uint16_t");
_Static_assert(sizeof(((core4VisionWorkspaceStruct*) 0)->visitMap[0]) == sizeof(uint32_t),
    "Beacon generation tag must remain uint32_t");
_Static_assert(sizeof(((core4VisionWorkspaceStruct*) 0)->scratch.canny) <=
    sizeof(((core4VisionWorkspaceStruct*) 0)->scratch.bfsQueue),
    "Canny scratch must fit the shared BFS queue storage");
_Static_assert(sizeof(core4VisionWorkspaceStruct) == (OM_BUFFER_SIZE * sizeof(uint32_t) * 2u),
    "Core4 shared vision workspace must remain 180 480 bytes");

#endif /* CORE4_VISION_WORKSPACE_H */

```

core4_cyt4bb7/project/code/vision/om_beacon.c

SHA-256

:

df368f9762041e2374379957307f4e1ce4ce3cf67ee754bd0e3acf077eedcb99

代码 (c)

```

/*
 * om_beacon.c - [6] 信标检测 (Beacon Detection)
 *
 * 基于 Canny 边缘 + 简化 BFS, 检测多个红外信标 (闭合椭圆/圆形轮廓)
 * 判据: 闭合性 + 尺寸 + 偏心率上限 + 内部亮度
 * 复用 Core4 vision workspace 的 inContourMap / contourQueue / contourGeneration 基础设施
 */

#include "om_internal.h"

// ---- typedef ----

typedef struct
{
    uint16_t sizeMax;
    uint16_t sizeMin;
    float eccMaxRatioK; // 预计算: ((eccMax-1)/(eccMax+1))^2, 用于无 sqrt 偏心率比较
    uint8_t brightnessMin; // 内部亮度下限
    uint16_t maxEndpoints; // 允许的最大端点数 (信标边缘常有断裂)
} BEACON_DATA_CONFIG;

// ---- 内部变量 ----

static BEACON_DATA_CONFIG beaconDataConfig;

// ---- 内部函数声明 ----

static inline uint16_t beaconBFS(const u8Image* edgeSrc, uint16_t startX, uint16_t startY, uint32_t genTag, BEACON_DATA_STRUCT* beacon, float*
outMxx, float* outMyy, float* outMxy);

// ---- 外部函数实现 ----

void beaconDetectInit(uint16_t sizeMin, uint16_t sizeMax, float eccMax, uint8_t brightnessMin, uint16_t maxEndpoints)
{
    beaconDataConfig.sizeMin = sizeMin;
    beaconDataConfig.sizeMax = sizeMax;
    // 预计算偏心率阈值的平方比值, 运行时只需一次乘法+比较
    float k = (eccMax - 1.0f) / (eccMax + 1.0f);
    beaconDataConfig.eccMaxRatioK = k * k;

```

```

        beaconDataConfig.brightnessMin = brightnessMin;
        beaconDataConfig.maxEndpoints = maxEndpoints;
    }

uint16_t beaconDetect(const u8Image* edgeSrc, const u8Image* graySrc, BEACON_DETECTOR_STRUCT* obj)
{
    if (edgeSrc->size > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE) return 0;

    // generation 替代 memset (与 contourDetect 共享基础设施)
    contourGeneration++;
    if (contourGeneration == 0)
    {
        memset(inContourMap, 0, sizeof(inContourMap));
        contourGeneration = 1;
    }
    uint32_t genBase = (uint32_t) contourGeneration << 16;

    uint16_t beaconCount = 0;
    uint32_t tagCounter = 0;

    for (uint32_t y = 0; y < edgeSrc->yLen && beaconCount < (OM_MAX_OBJECTS - 1u); y++)
    {
        uint32_t rowOff = y * edgeSrc->xLen;
        for (uint32_t x = 0; x < edgeSrc->xLen && beaconCount < 100; x++)
        {
            uint32_t i = rowOff + x;
            if (edgeSrc->ptr[i] != WHITE || (inContourMap[i] >> 16) == contourGeneration) continue;

            tagCounter++;
            if (tagCounter > 0xFFFFu) goto beaconDone;

            uint32_t genTag = genBase | (uint32_t) tagCounter;

            float curMxx, curMyy, curMxy;
            uint16_t ep = beaconBFS(edgeSrc, (uint16_t) x, (uint16_t) y, genTag,
                                    &obj->beaconData[beaconCount], &curMxx, &curMyy, &curMxy);

            // 闭合轮廓过滤 (信标边缘断裂容忍)
            if (ep > beaconDataConfig.maxEndpoints) continue;

            // 尺寸过滤
            uint16_t sz = obj->beaconData[beaconCount].contourSize;
            if (sz < beaconDataConfig.sizeMin || sz > beaconDataConfig.sizeMax) continue;

            // 偏心率上限过滤: 拒绝过度拉伸的非信标轮廓
            // eigMax/eigMin > eccMax  <=>  diff^2 > sum^2 * ((eccMax-1)/(eccMax+1))^2
            float eigSum = curMxx + curMyy;
            float eigDiff2 = (curMxx - curMyy) * (curMxx - curMyy) + 4.0f * curMxy * curMxy;
            if (eigSum > 0.0f)
            {
                float bound = eigSum * eigSum * beaconDataConfig.eccMaxRatioK;
                if (eigDiff2 > bound) continue;
            }

            // 质心亮度检查: 填充椭圆的质心必在填充区域内, 单像素即可判定
            uint16_t cx = obj->beaconData[beaconCount].centerX;
            uint16_t cy = obj->beaconData[beaconCount].centerY;
            uint8_t centroidBrightness = graySrc->ptr[(uint32_t) cy * graySrc->xLen + (uint32_t) cx];
            obj->beaconData[beaconCount].brightness = centroidBrightness;
            if (centroidBrightness < beaconDataConfig.brightnessMin) continue;

            beaconCount++;
        }
    }

beaconDone:
    obj->beaconCount = beaconCount;
    for (uint16_t i = beaconCount; i < OM_MAX_OBJECTS; i++) obj->beaconData[i].contourSize = 0;
    return beaconCount;
}

// ---- 内部函数实现 ----

// 简化 BFS: 仅计算二阶矩 (无三阶矩/朝向), 返回端点数
static inline uint16_t beaconBFS(const u8Image* obj, uint16_t startX, uint16_t startY, uint32_t genTag, BEACON_DATA_STRUCT* beacon, float* outMxx,
float* outMyy, float* outMxy)
{
    uint32_t width = obj->xLen;
    uint32_t height = obj->yLen;

```

```

beacon->contourSize = 0;
beacon->centerX = 0;
beacon->centerY = 0;
beacon->brightness = 0;

uint16_t head = 0;
uint16_t tail = 0;

inContourMap[startY * width + startX] = genTag;
contourQueue[tail++] = ((uint32_t) startY << 16) | (uint32_t) startX;

uint32_t sumX = 0;
uint32_t sumY = 0;
uint32_t sumXX = 0;
uint32_t sumYY = 0;
uint32_t sumXY = 0;
uint16_t totalCount = 0;
uint16_t endpointCount = 0;
uint8_t abandon = 0;

while (head < tail)
{
    uint16_t cx = (uint16_t) (contourQueue[head] & 0xFFFFu);
    uint16_t cy = (uint16_t) (contourQueue[head] >> 16);
    head++;

    totalCount++;

    if (!abandon)
    {
        sumX += cx;
        sumY += cy;
        sumXX += (uint32_t) cx * cx;
        sumYY += (uint32_t) cy * cy;
        sumXY += (uint32_t) cx * cy;
    }

    uint8_t whiteNeighborCount = 0;

    for (uint8_t i = 0; i < 8; i++)
    {
        int32_t nx = (int32_t) cx + contourDx[i];
        int32_t ny = (int32_t) cy + contourDy[i];

        if (nx < 0 || (uint32_t) nx >= width || ny < 0 || (uint32_t) ny >= height) continue;

        uint16_t nIndex = (uint16_t) ((uint32_t) ny * width + (uint32_t) nx);

        if (obj->ptr[nIndex] == WHITE)
        {
            whiteNeighborCount++;

            if ((inContourMap[nIndex] >> 16) != contourGeneration)
            {
                inContourMap[nIndex] = genTag;
                if (tail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
                    contourQueue[tail++] = ((uint32_t) ny << 16) | (uint32_t) nx;
            }
        }
    }

    if (!abandon && whiteNeighborCount < 2) endpointCount++;

    if (!abandon && (totalCount > beaconDataConfig.sizeMax || endpointCount > beaconDataConfig.maxEndpoints)) abandon = 1;
}

/* abandon 时置 0： 让调用方尺寸过滤 (sz < sizeMin) 直接拒绝本应丢弃的轮廓 */
beacon->contourSize = abandon ? 0u : totalCount;

if (totalCount > 0 && !abandon)
{
    float fN = (float) totalCount;
    float meanX = (float) sumX / fN;
    float meanY = (float) sumY / fN;
    beacon->centerX = (uint16_t) (sumX / totalCount);
    beacon->centerY = (uint16_t) (sumY / totalCount);

    *outMxx = (float) sumXX / fN - meanX * meanX;
}

```

```

        *outMyy = (float) sumYY / fN - meanY * meanY;
        *outMxy = (float) sumXY / fN - meanX * meanY;
    }
    else
    {
        *outMxx = 0.0f;
        *outMyy = 0.0f;
        *outMxy = 0.0f;
    }

    return endpointCount;
}

```

core4_cyt4bb7/project/code/vision/om_beaconcc.c

SHA-256

:

565bbc8e60e4e0571d715825d28b5f3feeb97bdcd27d99a61ac0b8d4e3570998

代码 (c)

```

/*
 * om_beaconcc.c - core1 信标 BinaryCC 检测
 *
 * 双阈值二值化 + 八连连通域 + 矩/Hu 筛选, 输出兼容 BEACON_DETECTOR_STRUCT。
 */

#include "om_beaconcc.h"

#include "om_internal.h"

typedef struct
{
    uint16_t area;
    uint16_t bboxMinX;
    uint16_t bboxMinY;
    uint16_t bboxMaxX;
    uint16_t bboxMaxY;
    float centroidX;
    float centroidY;
    float meanBrightness;
    uint8_t peakBrightness;
    float m20;
    float m02;
    float m11;
    float m30;
    float m03;
    float m21;
    float m12;
    float hu[7];
    float huDistance;
    float ecc;
    float fillRatio;
    uint8_t shapeEligible;
    uint8_t huEligible;
} BEACONCC_CANDIDATE_STRUCT;

static BEACONCC_CONFIG_STRUCT beaconCcConfig;
static uint8_t beaconCcTemplateReady = 0u;
static float beaconCcHuTemplate[7];
static uint8_t beaconCcCandidateCount = 0u;

static void beaconCcThreshold(const u8Image* graySrc, u8Image* maskDst);
static uint8_t beaconCcVisited(uint32_t index);
static void beaconCcMarkVisited(uint32_t index);
static uint8_t beaconCcScanComponent(const u8Image* graySrc, const u8Image* maskDst, uint32_t startIndex,
    BEACONCC_CANDIDATE_STRUCT* out);
static void beaconCcFinishCandidate(BEACONCC_CANDIDATE_STRUCT* c, float sumX, float sumY,
    float sumX2, float sumY2, float sumXY,
    float sumX3, float sumY3, float sumX2Y, float sumXY2,
    float sumGray, uint8_t peakBrightness);
static void beaconCcComputeHu(float area, float mu20, float mu02, float mu11, float mu30, float mu03,
    float mu21, float mu12, float hu[7]);
static float beaconCcHuDistance(const float hu[7], const float templ[7]);

void beaconCcInit(const BEACONCC_CONFIG_STRUCT* config)

```



```

{
    memset(&beaconCcConfig, 0, sizeof(beaconCcConfig));
    memset(beaconCcHuTemplate, 0, sizeof(beaconCcHuTemplate));
    beaconCcTemplateReady = 0u;
    beaconCcCandidateCount = 0u;
    if (config != 0) beaconCcConfig = *config;
}

void beaconCcSetConfig(const BEACONCC_CONFIG_STRUCT* config)
{
    if (config == 0) return;
    beaconCcConfig = *config;
}

void beaconCcSetTemplate(const float hu[7], float huMax)
{
    if (hu == 0 || huMax <= 0.0f)
    {
        beaconCcTemplateReady = 0u;
        beaconCcConfig.huMax = 0.0f;
        return;
    }
    for (uint8_t i = 0u; i < 7u; i++) beaconCcHuTemplate[i] = hu[i];
    beaconCcConfig.huMax = huMax;
    beaconCcTemplateReady = 1u;
}

uint16_t beaconCcDetect(const u8Image* graySrc, u8Image* maskDst, BEACON_DETECTOR_STRUCT* obj)
{
    if (obj == 0 || graySrc == 0 || maskDst == 0 || graySrc->ptr == 0 || maskDst->ptr == 0 ||
        graySrc->size > OM_BUFFER_SIZE || maskDst->size < graySrc->size)
    {
        if (obj != 0) obj->beaconCount = 0u;
        return 0u;
    }

    beaconCcThreshold(graySrc, maskDst);
    beaconCcCandidateCount = 0u;

    contourGeneration++;
    if (contourGeneration == 0u)
    {
        memset(inContourMap, 0, graySrc->size * sizeof(inContourMap[0]));
        contourGeneration = 1u;
    }

    for (uint32_t i = 0u; i < graySrc->size; i++)
    {
        BEACONCC_CANDIDATE_STRUCT c;
        if (maskDst->ptr[i] != WHITE || beaconCcVisited(i)) continue;

        memset(&c, 0, sizeof(c));
        if (!beaconCcScanComponent(graySrc, maskDst, i, &c))
        {
            continue;
        }

        if (c.area < beaconCcConfig.areaMin)
        {
            continue;
        }
        if (beaconCcConfig.areaMax > 0u && c.area > beaconCcConfig.areaMax)
        {
            continue;
        }
        if (beaconCcConfig.brightnessMin > 0u && c.meanBrightness < (float) beaconCcConfig.brightnessMin)
        {
            continue;
        }
        if (c.shapeEligible && beaconCcConfig.eccMax > 0.0f && c.ecc > beaconCcConfig.eccMax)
        {
            continue;
        }
        if (c.shapeEligible && beaconCcConfig.fillMin > 0.0f && c.fillRatio < beaconCcConfig.fillMin)
        {
            continue;
        }
        if (beaconCcTemplateReady && c.huEligible && beaconCcConfig.huMax > 0.0f &&
            c.huDistance > beaconCcConfig.huMax)

```

```

    {
        continue;
    }

    if (beaconCcCandidateCount < OM_MAX_OBJECTS)
    {
        uint8_t dst = beaconCcCandidateCount;
        obj->beaconData[dst].centerX = (uint16_t) (c.centroidX + 0.5f);
        obj->beaconData[dst].centerY = (uint16_t) (c.centroidY + 0.5f);
        obj->beaconData[dst].contourSize = c.area;
        obj->beaconData[dst].brightness = (uint8_t) (c.meanBrightness + 0.5f);
        beaconCcCandidateCount++;
    }
}

obj->beaconCount = beaconCcCandidateCount;
for (uint8_t i = beaconCcCandidateCount; i < OM_MAX_OBJECTS; i++) obj->beaconData[i].contourSize = 0u;
return beaconCcCandidateCount;
}

static void beaconCcThreshold(const u8Image* graySrc, u8Image* maskDst)
{
    for (uint32_t i = 0u; i < graySrc->size; i++)
    {
        uint8_t v = graySrc->ptr[i];
        maskDst->ptr[i] = (v >= beaconCcConfig.thresholdLow && v <= beaconCcConfig.thresholdHigh) ? WHITE : BLACK;
    }
}

static uint8_t beaconCcVisited(uint32_t index)
{
    return ((inContourMap[index] >> 16) == contourGeneration) ? 1u : 0u;
}

static void beaconCcMarkVisited(uint32_t index)
{
    inContourMap[index] = ((uint32_t) contourGeneration << 16) | 1u;
}

static uint8_t beaconCcScanComponent(const u8Image* graySrc, const u8Image* maskDst, uint32_t startIndex,
    BEACONCC_CANDIDATE_STRUCT* out)
{
    uint32_t head = 0u;
    uint32_t tail = 0u;
    float sumX = 0.0f, sumY = 0.0f, sumX2 = 0.0f, sumY2 = 0.0f, sumXY = 0.0f;
    float sumX3 = 0.0f, sumY3 = 0.0f, sumX2Y = 0.0f, sumXY2 = 0.0f, sumGray = 0.0f;
    uint8_t peakBrightness = 0u;

    out->bboxMinX = graySrc->xLen;
    out->bboxMinY = graySrc->yLen;
    if (tail >= OM_BUFFER_SIZE) return 0u;
    contourQueue[tail++] = startIndex;
    beaconCcMarkVisited(startIndex);

    while (head < tail)
    {
        uint32_t cur = contourQueue[head++];
        uint32_t x = cur % graySrc->xLen;
        uint32_t y = cur / graySrc->xLen;
        float xf = (float) x;
        float yf = (float) y;
        uint8_t gray8 = graySrc->ptr[cur];
        float gray = (float) gray8;

        out->area++;
        if (x < out->bboxMinX) out->bboxMinX = (uint16_t) x;
        if (y < out->bboxMinY) out->bboxMinY = (uint16_t) y;
        if (x > out->bboxMaxX) out->bboxMaxX = (uint16_t) x;
        if (y > out->bboxMaxY) out->bboxMaxY = (uint16_t) y;
        if (gray8 > peakBrightness) peakBrightness = gray8;

        sumX += xf;
        sumY += yf;
        sumX2 += xf * xf;
        sumY2 += yf * yf;
        sumXY += xf * yf;
        sumX3 += xf * xf * xf;
        sumY3 += yf * yf * yf;
        sumX2Y += xf * xf * yf;
    }
}

```

```

sumXY2 += xf * yf * yf;
sumGray += gray;

for (uint8_t ni = 0u; ni < 8u; ni++)
{
    int32_t nx = (int32_t) x + contourDx[ni];
    int32_t ny = (int32_t) y + contourDy[ni];
    uint32_t next;
    if (nx < 0 || ny < 0 || (uint32_t) nx >= graySrc->xLen || (uint32_t) ny >= graySrc->yLen) continue;
    next = (uint32_t) ny * graySrc->xLen + (uint32_t) nx;
    if (maskDst->ptr[next] == WHITE && !beaconCcVisited(next))
    {
        if (tail >= OM_BUFFER_SIZE) return 0u;
        beaconCcMarkVisited(next);
        contourQueue[tail++] = next;
    }
}

if (out->area == 0u) return 0u;
beaconCcFinishCandidate(out, sumX, sumY, sumX2, sumY2, sumXY, sumX3, sumY3,
                        sumX2Y, sumXY2, sumGray, peakBrightness);
return 1u;
}

static void beaconCcFinishCandidate(BEACONCC_CANDIDATE_STRUCT* c, float sumX, float sumY,
                                   float sumX2, float sumY2, float sumXY,
                                   float sumX3, float sumY3, float sumX2Y, float sumXY2,
                                   float sumGray, uint8_t peakBrightness)
{
    float area = (float) c->area;
    float cx = sumX / area;
    float cy = sumY / area;
    float mu20 = sumX2 - cx * sumX;
    float mu02 = sumY2 - cy * sumY;
    float mu11 = sumXY - cx * sumY;
    float mu30 = sumX3 - 3.0f * cx * sumX2 + 3.0f * cx * cx * sumX - cx * cx * cx * area;
    float mu03 = sumY3 - 3.0f * cy * sumY2 + 3.0f * cy * cy * sumY - cy * cy * cy * area;
    float mu21 = sumX2Y - cy * sumX2 - 2.0f * cx * sumXY + 2.0f * cx * cy * sumX + cx * cx * sumY - cx * cx * cy * area;
    float mu12 = sumXY2 - cx * sumY2 - 2.0f * cy * sumXY + 2.0f * cx * cy * sumY + cy * cy * sumX - cx * cy * cy * area;
    float covXX = mu20 / area;
    float covYY = mu02 / area;
    float covXY = mu11 / area;
    float trace = covXX + covYY;
    float delta = sqrtf((covXX - covYY) * (covXX - covYY) + 4.0f * covXY * covXY);
    float lambdaA = (trace + delta) * 0.5f;
    float lambdaB = (trace - delta) * 0.5f;
    float bboxArea = (float) ((c->bboxMaxX - c->bboxMinX + 1u) * (c->bboxMaxY - c->bboxMinY + 1u));

    if (lambdaA < lambdaB)
    {
        float t = lambdaA;
        lambdaA = lambdaB;
        lambdaB = t;
    }

    c->centroidX = cx;
    c->centroidY = cy;
    c->meanBrightness = sumGray / area;
    c->peakBrightness = peakBrightness;
    c->m20 = mu20;
    c->m02 = mu02;
    c->m11 = mu11;
    c->m30 = mu30;
    c->m03 = mu03;
    c->m21 = mu21;
    c->m12 = mu12;
    c->ecc = (lambdaB > 1.0e-6f) ? (lambdaA / lambdaB) : 9999.0f;
    c->fillRatio = (bboxArea > 0.0f) ? (area / bboxArea) : 0.0f;
    beaconCcComputeHu(area, mu20, mu02, mu11, mu30, mu03, mu21, mu12, c->hu);
    c->huDistance = beaconCcTemplateReady ? beaconCcHuDistance(c->hu, beaconCcHuTemplate) : -1.0f;
    c->shapeEligible = (c->area >= beaconCcConfig.huAreaMin) ? 1u : 0u;
    c->huEligible = (beaconCcTemplateReady && c->shapeEligible) ? 1u : 0u;
}

static void beaconCcComputeHu(float area, float mu20, float mu02, float mu11, float mu30, float mu03,
                              float mu21, float mu12, float hu[7])
{
    float invArea2 = 1.0f / (area * area);

```

```

float invArea25 = invArea2 / sqrtf(area);
float eta20 = mu20 * invArea2;
float eta02 = mu02 * invArea2;
float eta11 = mu11 * invArea2;
float eta30 = mu30 * invArea25;
float eta03 = mu03 * invArea25;
float eta21 = mu21 * invArea25;
float eta12 = mu12 * invArea25;
float a = eta30 + eta12;
float b = eta21 + eta03;
float c = eta30 - 3.0f * eta12;
float d = 3.0f * eta21 - eta03;

hu[0] = eta20 + eta02;
hu[1] = (eta20 - eta02) * (eta20 - eta02) + 4.0f * eta11 * eta11;
hu[2] = c * c + d * d;
hu[3] = a * a + b * b;
hu[4] = c * a * (a * a - 3.0f * b * b) + d * b * (3.0f * a * a - b * b);
hu[5] = (eta20 - eta02) * (a * a - b * b) + 4.0f * eta11 * a * b;
hu[6] = d * a * (a * a - 3.0f * b * b) - c * b * (3.0f * a * a - b * b);
}

static float beaconCcHuDistance(const float hu[7], const float templ[7])
{
    static const float weights[7] = {2.0f, 2.0f, 1.5f, 1.5f, 0.5f, 0.5f, 0.02f};
    float distance = 0.0f;
    for (uint8_t i = 0u; i < 7u; i++)
    {
        float a = fabsf(hu[i]);
        float b = fabsf(templ[i]);
        if (a < 1.0e-12f) a = 1.0e-12f;
        if (b < 1.0e-12f) b = 1.0e-12f;
        distance += weights[i] * fabsf(logf(a) - logf(b));
    }
    return distance;
}

```

core4_cyt4bb7/project/code/vision/om_beaconcc.h

SHA-256

:

27d217cffe3029e12c16e4f6c01415c0c29e8399466bdf209fce9b653cfeae87

代码 (c)

```

#ifndef OM_BEACONCC_H
#define OM_BEACONCC_H

#include <stdint.h>

#include "openMorph.h"

typedef struct
{
    uint8_t thresholdLow;
    uint8_t thresholdHigh;
    uint16_t areaMin;
    uint16_t areaMax;
    uint8_t brightnessMin;
    float eccMax;      /* 0=关闭 */
    float fillMin;     /* 0=关闭 */
    float huMax;       /* 0=关闭 */
    uint16_t huAreaMin; /* 小于该面积的远点只输出 Hu，不按 Hu 拒绝 */
} BEACONCC_CONFIG_STRUCT;

void beaconCcInit(const BEACONCC_CONFIG_STRUCT* config);
void beaconCcSetConfig(const BEACONCC_CONFIG_STRUCT* config);
void beaconCcSetTemplate(const float hu[7], float huMax);
uint16_t beaconCcDetect(const u8Image* graySrc, u8Image* maskDst, BEACON_DETECTOR_STRUCT* obj);

#endif /* OM_BEACONCC_H */

```

core4_cyt4bb7/project/code/vision/om_canny.c

SHA-256

:

ae68fd41321e73a4cb679f23232d8c3cb8d95a62a2f180f78091b02386e24f3c

代码 (c)

```
/*
 * om_canny.c - [4] 高斯模糊及 Canny 边缘检测
 *
 * 3x3 定点高斯核 / 逐行流水线 Sobel+NMS+滞后门限 边缘检测
 */

#include "om_internal.h"

// ---- define ----

#define CANNY_RING_WIDTH (OM_CANNY_RING_WIDTH)

// ---- typedef ----

typedef struct
{
    uint32_t width;
    uint32_t height;
    uint32_t pixelCount;
    uint32_t cropStartX;
    uint32_t cropStartY;
    uint32_t cropEndX;
    uint32_t cropEndY;
    uint32_t blurStartX;
    uint32_t blurStartY;
    uint32_t blurEndX;
    uint32_t blurEndY;
} gaussConfigStruct;

typedef struct
{
    uint32_t width;
    uint32_t height;
    uint32_t pixelCount;
    uint32_t sobelStartX;
    uint32_t sobelStartY;
    uint32_t sobelEndX;
    uint32_t sobelEndY;
    uint32_t nmsStartX;
    uint32_t nmsStartY;
    uint32_t nmsEndX;
    uint32_t nmsEndY;
    uint16_t thresholdLow;
    uint16_t thresholdHigh;
    uint32_t thresholdLowSq;
    uint32_t thresholdHighSq;
    int32_t trackOffsets[8];
} cannyConfigStruct;

// ---- 内部变量 ----

static uint16_t gaussKernelPreValueInt0[256];
static uint16_t gaussKernelPreValueInt1[256];
static uint16_t gaussKernelPreValueInt2[256];
static gaussConfigStruct gaussConfig = {0u};
static cannyConfigStruct cannyConfig = {0u};
static uint32_t sobelMapRing[3][CANNY_RING_WIDTH] = {0};
static uint8_t sobelDirMapRing[3][CANNY_RING_WIDTH] = {0};

// ---- 外部函数实现 ----

void gaussianInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY)
{
    // gaussianKernelTemp[0] : 四个角点对应的核值
    // gaussianKernelTemp[1] : 四个边点对应的核值
    // gaussianKernelTemp[2] : 中心点对应的核值
    float gaussianKernelTemp[3] = {0.0f, 0.0f, 0.0f};
```

```

memset(&gaussConfig, 0, sizeof(gaussConfig));

if ((NULL == src) || (NULL == src->ptr) || (0u == src->xLen) || (0u == src->yLen) ||
    (src->xLen > CANNY_RING_WIDTH) || ((src->xLen * src->yLen) > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE))
{
    memset(gaussKernelPreValueInt0, 0, sizeof(gaussKernelPreValueInt0));
    memset(gaussKernelPreValueInt1, 0, sizeof(gaussKernelPreValueInt1));
    memset(gaussKernelPreValueInt2, 0, sizeof(gaussKernelPreValueInt2));
    return;
}

gaussConfig.width = src->xLen;
gaussConfig.height = src->yLen;
gaussConfig.pixelCount = src->xLen * src->yLen;

if (startX > endX)
{
    uint32_t temp = startX;
    startX = endX;
    endX = temp;
}
if (startY > endY)
{
    uint32_t temp = startY;
    startY = endY;
    endY = temp;
}

gaussConfig.cropStartX = unsignRange(startX, gaussConfig.width - 1u);
gaussConfig.cropStartY = unsignRange(startY, gaussConfig.height - 1u);
gaussConfig.cropEndX = range(endX, gaussConfig.cropStartX, gaussConfig.width - 1u);
gaussConfig.cropEndY = range(endY, gaussConfig.cropStartY, gaussConfig.height - 1u);

gaussConfig.blurStartX = 1u;
gaussConfig.blurStartY = 1u;
gaussConfig.blurEndX = 0u;
gaussConfig.blurEndY = 0u;

if ((gaussConfig.width >= 3u) && (gaussConfig.height >= 3u) &&
    ((gaussConfig.cropEndX - gaussConfig.cropStartX) >= 2u) &&
    ((gaussConfig.cropEndY - gaussConfig.cropStartY) >= 2u))
{
    gaussConfig.blurStartX = (0u == gaussConfig.cropStartX) ? 1u : (gaussConfig.cropStartX + 1u);
    gaussConfig.blurStartY = (0u == gaussConfig.cropStartY) ? 1u : (gaussConfig.cropStartY + 1u);
    gaussConfig.blurEndX = (gaussConfig.cropEndX >= (gaussConfig.width - 1u)) ? (gaussConfig.width - 2u) : (gaussConfig.cropEndX - 1u);
    gaussConfig.blurEndY = (gaussConfig.cropEndY >= (gaussConfig.height - 1u)) ? (gaussConfig.height - 2u) : (gaussConfig.cropEndY - 1u);
}

if (sigma <= 0.0f)
{
    memset(gaussKernelPreValueInt0, 0, sizeof(gaussKernelPreValueInt0));
    memset(gaussKernelPreValueInt1, 0, sizeof(gaussKernelPreValueInt1));
    memset(gaussKernelPreValueInt2, 0, sizeof(gaussKernelPreValueInt2));
    return;
}

float sigma2 = sigma * sigma;
float coeff = 1.0f / (TWO_PI * sigma2);
float sum = 0.0f;

// 生成 3x3 高斯核，并提取 3 个唯一值（角点/边点/中心）
for (uint8_t y = 0u; y < 3; y++)
{
    for (uint8_t x = 0u; x < 3; x++)
    {
        int16_t dx = (int16_t) x - 1;
        int16_t dy = (int16_t) y - 1;
        float exponent = -((float) (dx * dx + dy * dy)) / (2.0f * sigma2);
        float value = (float) exp((double) exponent) * coeff;

        if (0 == dx && 0 == dy) gaussianKernelTemp[2] = value;
        else if (0 == dx || 0 == dy) gaussianKernelTemp[1] = value;
        else gaussianKernelTemp[0] = value;

        sum += value;
    }
}

```

```

if (sum <= 0.0f)
{
    memset(gaussKernelPreValueInt0, 0, sizeof(gaussKernelPreValueInt0));
    memset(gaussKernelPreValueInt1, 0, sizeof(gaussKernelPreValueInt1));
    memset(gaussKernelPreValueInt2, 0, sizeof(gaussKernelPreValueInt2));
    return;
}

// 归一化 3 个唯一核值
gaussianKernelTemp[0] /= sum;
gaussianKernelTemp[1] /= sum;
gaussianKernelTemp[2] /= sum;

// 预计算核值与 8bit 灰度值相乘的定点数 (放缩 256 倍), 消除内层浮点开销
for (uint16_t gray = 0u; gray < 256; gray++)
{
    gaussKernelPreValueInt0[gray] = (uint16_t) (gaussianKernelTemp[0] * (float) gray * 256.0f);
    gaussKernelPreValueInt1[gray] = (uint16_t) (gaussianKernelTemp[1] * (float) gray * 256.0f);
    gaussKernelPreValueInt2[gray] = (uint16_t) (gaussianKernelTemp[2] * (float) gray * 256.0f);
}
}

void cannyInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY, uint16_t thresholdLow, uint16_t thresholdHigh)
{
    gaussianInit(src, sigma, startX, startY, endX, endY);

    memset(&cannyConfig, 0, sizeof(cannyConfig));

    if ((NULL == src) || (NULL == src->ptr) || (0u == src->xLen) || (0u == src->yLen) ||
        (src->xLen > CANNY_RING_WIDTH) || ((src->xLen * src->yLen) > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)) return;

    cannyConfig.width = src->xLen;
    cannyConfig.height = src->yLen;
    cannyConfig.pixelCount = src->xLen * src->yLen;

    if (thresholdLow > thresholdHigh)
    {
        uint16_t temp = thresholdLow;
        thresholdLow = thresholdHigh;
        thresholdHigh = temp;
    }

    cannyConfig.thresholdLow = thresholdLow;
    cannyConfig.thresholdHigh = thresholdHigh;

    // 提前计算平方
    cannyConfig.thresholdLowSq = (uint32_t) thresholdLow * thresholdLow;
    cannyConfig.thresholdHighSq = (uint32_t) thresholdHigh * thresholdHigh;

    cannyConfig.sobelStartX = 1u;
    cannyConfig.sobelStartY = 1u;
    cannyConfig.sobelEndX = 0u;
    cannyConfig.sobelEndY = 0u;
    cannyConfig.nmsStartX = 1u;
    cannyConfig.nmsStartY = 1u;
    cannyConfig.nmsEndX = 0u;
    cannyConfig.nmsEndY = 0u;

    if ((gaussConfig.blurEndX > (gaussConfig.blurStartX + 1u)) &&
        (gaussConfig.blurEndY > (gaussConfig.blurStartY + 1u)))
    {
        cannyConfig.sobelStartX = gaussConfig.blurStartX + 1u;
        cannyConfig.sobelStartY = gaussConfig.blurStartY + 1u;
        cannyConfig.sobelEndX = gaussConfig.blurEndX - 1u;
        cannyConfig.sobelEndY = gaussConfig.blurEndY - 1u;
    }

    if ((cannyConfig.sobelEndX > (cannyConfig.sobelStartX + 1u)) &&
        (cannyConfig.sobelEndY > (cannyConfig.sobelStartY + 1u)))
    {
        cannyConfig.nmsStartX = cannyConfig.sobelStartX + 1u;
        cannyConfig.nmsStartY = cannyConfig.sobelStartY + 1u;
        cannyConfig.nmsEndX = cannyConfig.sobelEndX - 1u;
        cannyConfig.nmsEndY = cannyConfig.sobelEndY - 1u;
    }
}

// 预先计算滞后门限追踪(连边)阶段需要的直接邻信偏移
cannyConfig.trackOffsets[0] = -(int32_t) gaussConfig.width - 1;
cannyConfig.trackOffsets[1] = -(int32_t) gaussConfig.width;

```

```

cannyConfig.trackOffsets[2] = -(int32_t) gaussConfig.width + 1;
cannyConfig.trackOffsets[3] = -1;
cannyConfig.trackOffsets[4] = 1;
cannyConfig.trackOffsets[5] = (int32_t) gaussConfig.width - 1;
cannyConfig.trackOffsets[6] = (int32_t) gaussConfig.width;
cannyConfig.trackOffsets[7] = (int32_t) gaussConfig.width + 1;
}

/* 运行时仅更新 Canny 双阈值，不重算高斯核与 ROI，便于按键调参 */
void cannySetThreshold(uint16_t thresholdLow, uint16_t thresholdHigh)
{
    if (thresholdLow > thresholdHigh)
    {
        uint16_t temp = thresholdLow;
        thresholdLow = thresholdHigh;
        thresholdHigh = temp;
    }

    cannyConfig.thresholdLow = thresholdLow;
    cannyConfig.thresholdHigh = thresholdHigh;
    cannyConfig.thresholdLowSq = (uint32_t) thresholdLow * thresholdLow;
    cannyConfig.thresholdHighSq = (uint32_t) thresholdHigh * thresholdHigh;
}

void cannyEdgeDetection(const u8Image* src, u8Image* dstGaussBlur, u8Image* dst)
{
    if (NULL == src || NULL == src->ptr || NULL == dstGaussBlur || NULL == dstGaussBlur->ptr ||
        NULL == dst || NULL == dst->ptr) return;
    if (0u == cannyConfig.pixelCount || 0u == gaussConfig.width) return;

    uint32_t queueHead = 0u;
    uint32_t queueTail = 0u;

    // 清零目标画布、ring buffer 残留、高斯模糊边界脏值
    memset(dst->ptr, BLACK, gaussConfig.width * gaussConfig.height);
    memset(dstGaussBlur->ptr, 0, gaussConfig.width * gaussConfig.height);
    memset(sobelMapRing, 0, sizeof(sobelMapRing));
    memset(sobelDirMapRing, 0, sizeof(sobelDirMapRing));

    for (uint32_t y = gaussConfig.blurStartY; y <= gaussConfig.blurEndY; y++)
    {
        // ===== 第 1 阶段: 高斯模糊 =====
        const uint8_t* row0 = src->ptr + (y - 1u) * gaussConfig.width;
        const uint8_t* row1 = src->ptr + y * gaussConfig.width;
        const uint8_t* row2 = src->ptr + (y + 1u) * gaussConfig.width;
        uint8_t* dstRow = dstGaussBlur->ptr + y * gaussConfig.width;

        for (uint32_t x = gaussConfig.blurStartX; x <= gaussConfig.blurEndX; x++)
        {
            uint32_t result = gaussKernelPreValueInt0[row0[x - 1u]] + gaussKernelPreValueInt1[row0[x]] + gaussKernelPreValueInt0[row0[x + 1u]] +
                gaussKernelPreValueInt1[row1[x - 1u]] + gaussKernelPreValueInt2[row1[x]] + gaussKernelPreValueInt1[row1[x + 1u]] +
                gaussKernelPreValueInt0[row2[x - 1u]] + gaussKernelPreValueInt1[row2[x]] + gaussKernelPreValueInt0[row2[x + 1u]];

            result = (result + 128) >> 8;
            dstRow[x] = (uint8_t) ((result > 255) ? 255 : result);
        }

        // ===== 第 2 阶段: Sobel 计算 =====
        if ((cannyConfig.sobelStartX <= cannyConfig.sobelEndX) &&
            (cannyConfig.sobelStartY <= cannyConfig.sobelEndY) &&
            (y >= (cannyConfig.sobelStartY + 1u)) &&
            ((y - 1u) <= cannyConfig.sobelEndY))
        {
            const uint8_t* s_row0 = dstGaussBlur->ptr + (y - 2u) * gaussConfig.width;
            const uint8_t* s_row1 = dstGaussBlur->ptr + (y - 1u) * gaussConfig.width;
            const uint8_t* s_row2 = dstGaussBlur->ptr + y * gaussConfig.width;

            uint32_t sobelRow = sobelMapRing[(y - 1u) % 3];
            uint8_t dirRow = sobelDirMapRing[(y - 1u) % 3];

            for (uint32_t x = cannyConfig.sobelStartX; x <= cannyConfig.sobelEndX; x++)
            {
                int32_t gx = (int32_t) s_row0[x + 1u] + ((int32_t) s_row1[x + 1u] << 1) + (int32_t) s_row2[x + 1u] - (int32_t) s_row0[x - 1u] - ((int32_t) s_row1[x - 1u] << 1) - (int32_t) s_row2[x - 1u];
                int32_t gy = (int32_t) s_row0[x - 1u] + ((int32_t) s_row0[x] << 1) + (int32_t) s_row0[x + 1u] - (int32_t) s_row2[x - 1u] - ((int32_t) s_row2[x] << 1) - (int32_t) s_row2[x + 1u];

                // 去除掉极其耗时的 sqrtf 运算
                uint32_t mag_sq = (uint32_t) (gx * gx) + (uint32_t) (gy * gy);
            }
        }
    }
}

```



```

sobelRow[x] = mag_sq;

// 完全消除浮点除法，只用整数比较判定4大区面方向
int32_t ax = (gx >= 0) ? gx : -gx;
int32_t ay = (gy >= 0) ? gy : -gy;
int32_t ay256 = ay << 8;

if (ay256 <= ax * 106)
{
    dirRow[x] = 0; // 水平梯度主导（纵向边缘），沿水平方向抑制
}
else if (ay256 >= ax * 618)
{
    dirRow[x] = 2; // 垂直梯度主导（横向边缘），沿垂直方向抑制
}
else if ((gx ^ gy) >= 0)
{
    dirRow[x] = 1; // 45 度梯度
}
else
{
    dirRow[x] = 3; // 135 度梯度
}
}
}

// ===== 第3阶段: 非极大值抑制 =====
if ((cannyConfig.nmsStartX <= cannyConfig.nmsEndX) &&
    (cannyConfig.nmsStartY <= cannyConfig.nmsEndY) &&
    (y >= (cannyConfig.nmsStartY + 2u)) &&
    ((y - 2u) <= cannyConfig.nmsEndY))
{
    const uint32_t* n_row0 = sobelMapRing[(y - 3u) % 3];
    const uint32_t* n_row1 = sobelMapRing[(y - 2u) % 3];
    const uint32_t* n_row2 = sobelMapRing[(y - 1u) % 3];
    const uint8_t* dirRow = sobelDirMapRing[(y - 2u) % 3];

    uint8_t* final_dstRow = dst->ptr + (y - 2u) * gaussConfig.width + cannyConfig.nmsStartX;

    for (uint32_t x = cannyConfig.nmsStartX; x <= cannyConfig.nmsEndX; x++)
    {
        uint32_t edge0, edge1;
        uint32_t magnitude = n_row1[x];
        uint8_t dir = dirRow[x];

        if (dir == 0)
        {
            edge0 = n_row1[x - 1u];
            edge1 = n_row1[x + 1u];
        }
        else if (dir == 2)
        {
            edge0 = n_row0[x];
            edge1 = n_row2[x];
        }
        else if (dir == 1)
        {
            // gx, gy 同号 → 梯度方向 ↗ (亮度梯度指向右上) → NMS 比较 ↗ 与 ↘
            edge0 = n_row0[x + 1u];
            edge1 = n_row2[x - 1u];
        }
        else
        {
            // gx, gy 异号 → 梯度方向 ↘ (亮度梯度指向右下) → NMS 比较 ↘ 与 ↗
            edge0 = n_row0[x - 1u];
            edge1 = n_row2[x + 1u];
        }

        if ((magnitude > edge0) && (magnitude >= edge1))
        {
            if (magnitude >= cannyConfig.thresholdHighSq)
            {
                *final_dstRow = WHITE; // 确信强边缘
                if (queueTail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
                {
                    cannyEdgeQueue[queueTail++] = (uint16_t)((y - 2u) * gaussConfig.width + x);
                }
            }
            else if (magnitude >= cannyConfig.thresholdLowSq)

```

```

        {
            *final_dstRow = 127; // 先标记为弱边缘，用于第四阶段追踪，避免大数组分配
        }
    }

    final_dstRow++;
}
}

// ===== 第4阶段: 滞后门限追踪(连边) =====
while (queueHead < queueTail)
{
    uint32_t index = cannyEdgeQueue[queueHead++];

    for (uint8_t i = 0; i < 8; i++)
    {
        int32_t trackIndex = (int32_t) index + cannyConfig.trackOffsets[i];

        if (trackIndex >= 0 && trackIndex < (int32_t) cannyConfig.pixelCount)
        {
            if (dst->ptr[trackIndex] == 127)
            {
                dst->ptr[trackIndex] = WHITE; // 连通升级为强边缘

                // 防溢出保护
                if (queueTail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
                {
                    cannyEdgeQueue[queueTail++] = (uint16_t) trackIndex;
                }
            }
        }
    }
}

// ===== 收尾: 清理死路 =====
uint32_t totalPixels = gaussConfig.width * gaussConfig.height;
for (uint32_t i = 0; i < totalPixels; i++)
{
    if (dst->ptr[i] == 127) dst->ptr[i] = BLACK; // 无法与强边缘连通被消除
}
}

```

core4_cyt4bb7/project/code/vision/om_internal.h

SHA-256

:

5f04c5057c05fcc501a27943ea278a3e0e220de9cad93a9d1f47701dde93417d

代码 (c)

```

/*
 * om_internal.h - openMorph 内部共享头文件 (不对外暴露)
 *
 * 提供各子模块共用的宏定义、标准库包含及共享 BFS 基础设施声明。
 * 外部用户代码只需包含 openMorph.h。
 */
#ifndef OM_INTERNAL_H
#define OM_INTERNAL_H

// =====
// 公共接口 + 基础设施
// =====

#include "openMorph.h" // 公共类型定义与 API 声明
#include "openMorph_config.h" // 可移植尺寸/数量配置 (移植时只改此文件)
#include "commonDef.h" // u8Image, BLACK, WHITE, cReverse, range 等
#include "core4VisionWorkspace.h"

#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

// =====
// 共享宏定义

```

```
// =====

#define UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE (OM_BUFFER_SIZE)

// =====
// 共享工作区基础设施（Canny 与 Beacon 串行复用同一 union）
// =====

extern uint16_t contourGeneration;
extern const int8_t contourDx[8];
extern const int8_t contourDy[8];

#define inContourMap (g_core4VisionWorkspace.visitMap)
#define contourQueue (g_core4VisionWorkspace.scratch.bfsQueue)
#define cannyEdgeQueue (g_core4VisionWorkspace.scratch.canny.edgeQueue)

#endif // OM_INTERNAL_H
```

core4_cyt4bb7/project/code/vision/openMorph.h

SHA-256

:

92a40b379a6b348c966165aab1ab18adbf43833c8e5aa4f346ac19aea2398177

代码（c）

```
/*
 * Core4 openMorph 子集：保留 Core1 生产版 Canny 与两套信标检测所需 ABI。
 * Core4 不移植 image.c、形态学、Two-Pass 与轮廓跟踪，避免重复相机 owner 和无消费者代码。
 */
#ifdef OPENMORPH_H
#define OPENMORPH_H

#include <stdint.h>

#include "commonDef.h"
#include "openMorph_config.h"

void gaussianInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY,
    uint32_t endX, uint32_t endY);
void cannyInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY,
    uint32_t endX, uint32_t endY, uint16_t thresholdLow, uint16_t thresholdHigh);
void cannySetThreshold(uint16_t thresholdLow, uint16_t thresholdHigh);
void cannyEdgeDetection(const u8Image* src, u8Image* dstGaussBlur, u8Image* dst);

typedef struct
{
    uint16_t centerX;
    uint16_t centerY;
    uint16_t contourSize;
    uint8_t brightness;
} BEACON_DATA_STRUCT;

typedef struct
{
    BEACON_DATA_STRUCT beaconData[OM_MAX_OBJECTS];
    uint8_t beaconCount;
} BEACON_DETECTOR_STRUCT;

void beaconDetectInit(uint16_t sizeMin, uint16_t sizeMax, float eccMax,
    uint8_t brightnessMin, uint16_t maxEndpoints);
uint16_t beaconDetect(const u8Image* edgeSrc, const u8Image* graySrc,
    BEACON_DETECTOR_STRUCT* obj);

#endif /* OPENMORPH_H */
```

core4_cyt4bb7/project/code/vision/openMorph_config.h

SHA-256

:

0f579ab84ea8603c68c5b05fe3bea64f97cab9886a039fe7f82e17f9bc8b95c7

代码 (c)

```
/*
 * openMorph_config.h - openMorph 库可移植配置头
 *
 * 移植到新工程时只需修改本文件即可适配不同图像分辨率与对象数量上限。
 * openMorph 内部其他源文件不应再出现关于尺寸的字面量。
 */
#ifndef OPENMORPH_CONFIG_H
#define OPENMORPH_CONFIG_H

// =====
// 图像最大尺寸 (静态缓冲区按此分配)
// =====
// Core4 实际相机 MT9V03X 为 188 x 120 = 22560 像素。
// 必须与相机分辨率匹配: 若小于真实分辨率, om_canny/cannyInit 入口保护
// ((xLen*yLen) > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE) 会拒绝处理 → canny 图纯黑、信标失效。
// 本副本只服务 Core4 前视相机, 不引入或接管相机驱动。
// 像素索引以 uint16 存储 (om_beacon/om_canny), 故 W*H 须 <= 65535; 22560 在范围内。
// 本配置与具体相机驱动解耦, 不引入 SDK 头依赖。
// -----
#define OM_IMAGE_MAX_WIDTH (188u)
#define OM_IMAGE_MAX_HEIGHT (120u)
#define OM_BUFFER_SIZE (OM_IMAGE_MAX_WIDTH * OM_IMAGE_MAX_HEIGHT)

// =====
// 对象检测数量上限 (轮廓 / 信标 / 连通域共用)
// =====
// contourData[] / beaconData[] / sbea[] 等数组的成员数量。
// 超过此上限的候选对象会被忽略 (只保留前 N 个有效结果)。
// -----
#define OM_MAX_OBJECTS (101u)

// =====
// Canny 流水线环形缓冲宽度
// =====
// Canny 边缘检测内部 sobel 环形缓冲的列宽, 必须 >= OM_IMAGE_MAX_WIDTH。
// -----
#define OM_CANNY_RING_WIDTH (256u)

// =====
// 轮廓跟踪状态机参数
// =====
#define OM_LOCK_THRESHOLD (2u) // 连续检测到帧数: SEARCHING → ACQUIRING
#define OM_LOCK_CONFIRM (1u) // 继续确认帧数: ACQUIRING → LOCKED
#define OM_LOST_THRESHOLD (5u) // 连续丢失帧数: LOCKED → LOST_HOLDING
#define OM_LOST_TIMEOUT (30u) // 超时帧数: LOST_HOLDING → SEARCHING
#define OM_RAY_BALANCE_CONFIDENCE_MIN (0.50f) // ^ 尖端几何定向最低置信度: 低于此值拒绝更新朝向, 目标位置仍有效
#define OM_ORIENTATION_MIN_SIZE (8u) // 朝向可靠的最小像素数 (与 core0 openMorph 算法同源)
#define OM_ORIENTATION_ALPHA (0.4f) // 朝向低通滤波系数 (喂给 lowPassFilter, 越大越跟随/越小越平滑)
#define OM_POSITION_ALPHA (1.0f) // 质心位置不过滤
#define OM_VELOCITY_ALPHA (0.1f) // 原始质心差分速度低通滤波系数
#define OM_VELOCITY_DT_MIN_S (0.001f) // 有效图像样本间隔下限
#define OM_VELOCITY_DT_MAX_S (0.200f) // 超过此间隔只重建速度基线
#define OM_BORDER_MARGIN (1u) // 画幅边缘安全边距

// =====
// 朝向跳变保护默认值 (运行时可由菜单热重载, 逻辑同 ToF 定高跳变保护)
// NORMAL 态: |Δθ| > JUMP_MAX 视为异常突跳, 拒绝该样本进 SUSPENDED
// SUSPEND 态: |Δθ| < RECOVER_RAD 即时恢复; 介于二者间累计 CONFIRM_FRAMES 帧后
// 接受新朝向为基线; 持续 > JUMP_MAX 则计数清零继续沿用旧朝向
// -----
#define OM_ORIENTATION_JUMP_MAX (0.785398f) // PI/4, 朝向跳变阈值 (rad)
#define OM_ORIENTATION_RECOVER_RAD (0.196349f) // PI/16, 恢复窄带 (rad)
#define OM_ORIENTATION_CONFIRM_FRAMES (3u) // 异常持续这么多帧后接受新朝向

#endif // OPENMORPH_CONFIG_H
```

core4_cyt4bb7/project/user/cm7_0_isr.c

SHA-256

:

05ac45402d21fac897f9fdffa41bb622f54f534d603661b9d0991d77758d857a

代码 (c)

```
#include "zf_common_headfile.h"

#include "core0Loop.h"
#include "core4LinkCore4.h"
void pit0_ch0_isr()
{
    pit_isr_flag_clear(PIT_CH0);

    pitProcess0();
}

void pit0_ch1_isr()
{
    pit_isr_flag_clear(PIT_CH1);
}

void pit0_ch2_isr()
{
    pit_isr_flag_clear(PIT_CH2);
}

void pit0_ch10_isr()
{
    pit_isr_flag_clear(PIT_CH10);
}

void pit0_ch11_isr()
{
    pit_isr_flag_clear(PIT_CH11);
}

void pit0_ch12_isr()
{
    pit_isr_flag_clear(PIT_CH12);
}

void pit0_ch13_isr()
{
    pit_isr_flag_clear(PIT_CH13);
}

void pit0_ch14_isr()
{
    pit_isr_flag_clear(PIT_CH14);
}

void pit0_ch15_isr()
{
    pit_isr_flag_clear(PIT_CH15);
}

void pit0_ch16_isr()
{
    pit_isr_flag_clear(PIT_CH16);
}

void pit0_ch17_isr()
{
    pit_isr_flag_clear(PIT_CH17);
}

void pit0_ch18_isr()
{
    pit_isr_flag_clear(PIT_CH18);
}

void pit0_ch19_isr()
{
    pit_isr_flag_clear(PIT_CH19);
}

void pit0_ch20_isr()
{
    pit_isr_flag_clear(PIT_CH20);
}
```

```

void pit0_ch21_isr()
{
    pit_isr_flag_clear(PIT_CH21);
}
void uart0_isr(void)
{
    if (uart_isr_mask(UART_0))
    {
    }
    else
    {
    }
}

void uart1_isr(void)
{
    if (uart_isr_mask(UART_1))
    {
#ifdef DEBUG_UART_USE_INTERRUPT
        debug_interrupr_handler();
#endif
    }
    else
    {
    }
}

void uart2_isr(void)
{
    if (uart_isr_mask(UART_2))
    {
        /* group1(P10) 单线双向方案下闲置：核间链路仅用 group0(UART_1/P04)。 */
    }
    else
    {
    }
}

void uart3_isr(void)
{
    if (uart_isr_mask(UART_3))
    {
    }
    else
    {
    }
}

void uart4_isr(void)
{
    if (uart_isr_mask(UART_4))
    {
    }
    else
    {
    }
}

void uart5_isr(void)
{
    if (uart_isr_mask(UART_5))
    {
        core4LinkCore4RxIsr();
    }
    else
    {
    }
}

void uart6_isr(void)
{
    if (uart_isr_mask(UART_6))
    {
    }
    else
    {
    }
}

void gpio_0_exti_isr()

```

```

{
}

void gpio_1_exti_isr()
{
    if (exti_flag_get(P01_0))
    {
    }
    if (exti_flag_get(P01_1))
    {
    }
}

void gpio_2_exti_isr()
{
    if (exti_flag_get(P02_0))
    {
    }
    if (exti_flag_get(P02_4))
    {
    }
}

void gpio_3_exti_isr()
{
}

void gpio_4_exti_isr()
{
}

void gpio_5_exti_isr()
{
}

void gpio_6_exti_isr()
{
}

void gpio_7_exti_isr()
{
}

void gpio_8_exti_isr()
{
}

void gpio_9_exti_isr()
{
}

void gpio_10_exti_isr()
{
}

void gpio_11_exti_isr()
{
}

void gpio_12_exti_isr()
{
}

void gpio_13_exti_isr()
{
}

void gpio_14_exti_isr()
{
}

void gpio_15_exti_isr()
{
}

void gpio_16_exti_isr()
{
}

```

```
void gpio_17_exti_isr()
{
}

void gpio_18_exti_isr()
{
}

void gpio_19_exti_isr()
{
}

void gpio_20_exti_isr()
{
}

void gpio_21_exti_isr()
{
}

void gpio_22_exti_isr()
{
}

void gpio_23_exti_isr()
{
}
```

core4_cyt4bb7/project/user/cm7_1_isr.c

SHA-256 :
cf6adeb3e70bb87c7af14269a4d0ea7e11ea67061a86beb8e93b6826254be65f

代码 (c)

```
/*
* CYT4BB Opensourec Library 即（CYT4BB 开源库）是一个基于官方 SDK 接口的第三方开源库
* Copyright (c) 2022 SEEKFREE 逐飞科技
*
* 本文件是 CYT4BB 开源库的一部分
*
* CYT4BB 开源库 是免费软件
* 您可以根据自由软件基金会发布的 GPL（GNU General Public License，即 GNU 通用公共许可证）的条款
* 即 GPL 的第 3 版（即 GPL3.0）或（您选择的）任何后来的版本，重新发布和/或修改它
*
* 本开源库的发布是希望它能发挥作用，但并未对其作任何的保证
* 甚至没有隐含的适销性或适合特定用途的保证
* 更多细节请参见 GPL
*
* 您应该在收到本开源库的同时收到一份 GPL 的副本
* 如果没有，请参阅<https://www.gnu.org/licenses/>
*
* 额外注明：
* 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
* 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
* 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
* 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明（即本声明）
*
* 文件名称      cm7_1_isr
* 公司名称      成都逐飞科技有限公司
* 版本信息      查看 libraries/doc 文件夹内 version 文件 版本说明
* 开发环境      IAR 9.40.1
* 适用平台      CYT4BB
* 店铺链接      https://seekfree.taobao.com/
*
* 修改记录
* 日期      作者      备注
* 2024-1-9      pudding      first version
* 2024-5-14      pudding      新增 12 个 pit 周期中断 增加部分注释说明
* 2025-2-4      pudding      优化串口中断逻辑，防止意外干扰导致的卡死问题，优化串口波特率计算逻辑
* 2025-2-4      pudding      新增两个串口接口
*/

#include "zf_common_headfile.h"
```



```

#include "core1Loop.h"

// ***** PIT 中断函数 *****
void pit0_ch0_isr() // 定时器通道 0 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH0);
}

void pit0_ch1_isr() // 定时器通道 1 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH1);
}

void pit0_ch2_isr() // 定时器通道 2 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH2);

    pitProcess0(); // 菜单 10ms tick(menuKeyScan 远程按键 + 刷新计数)
}

void pit0_ch10_isr() // 定时器通道 10 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH10);
}

void pit0_ch11_isr() // 定时器通道 11 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH11);
}

void pit0_ch12_isr() // 定时器通道 12 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH12);
}

void pit0_ch13_isr() // 定时器通道 13 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH13);
}

void pit0_ch14_isr() // 定时器通道 14 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH14);
}

void pit0_ch15_isr() // 定时器通道 15 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH15);
}

void pit0_ch16_isr() // 定时器通道 16 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH16);
}

void pit0_ch17_isr() // 定时器通道 17 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH17);
}

void pit0_ch18_isr() // 定时器通道 18 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH18);
}

void pit0_ch19_isr() // 定时器通道 19 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH19);
}

void pit0_ch20_isr() // 定时器通道 20 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH20);
}

void pit0_ch21_isr() // 定时器通道 21 周期中断服务函数
{
    pit_isr_flag_clear(PIT_CH21);
}

```

```

}
// ***** PIT 中断函数 *****

// ***** 串口中断函数 *****
// 串口 0 默认作为调试串口
void uart0_isr(void)
{
    if (uart_isr_mask(UART_0)) // 串口 0 接收中断
    {
    }
    else // 串口 0 发送中断
    {
    }
}

void uart1_isr(void)
{
    if (uart_isr_mask(UART_1)) // 串口 1 接收中断
    {
    }
    #if DEBUG_UART_USE_INTERRUPT
        debug_interrupr_handler();
    #endif
    else // 串口 1 发送中断
    {
    }
}

void uart2_isr(void)
{
    if (uart_isr_mask(UART_2)) // 串口 2 接收中断
    {
    }
    else // 串口 2 发送中断
    {
    }
}

void uart3_isr(void)
{
    if (uart_isr_mask(UART_3)) // 串口 3 接收中断
    {
    }
    else // 串口 3 发送中断
    {
    }
}

void uart4_isr(void)
{
    if (uart_isr_mask(UART_4)) // 串口 4 接收中断
    {
    }
    else // 串口 4 发送中断
    {
    }
}

void uart5_isr(void)
{
    if (uart_isr_mask(UART_5)) // 串口 5 接收中断
    {
    }
    else // 串口 5 发送中断
    {
    }
}

void uart6_isr(void)
{
    if (uart_isr_mask(UART_6)) // 串口 6 接收中断
    {
    }
    else // 串口 6 发送中断
    {
    }
}
// ***** 串口中断函数 *****

```

```
// ***** 外部中断函数 *****
void gpio_0_exti_isr() // 外部 GPIO_0 中断服务函数
{
}

void gpio_1_exti_isr() // 外部 GPIO_1 中断服务函数
{
    if (exti_flag_get(P01_0)) // 示例 P1_0 端口外部中断判断
    {
    }
    if (exti_flag_get(P01_1))
    {
    }
}

void gpio_2_exti_isr() // 外部 GPIO_2 中断服务函数
{
    if (exti_flag_get(P02_0))
    {
    }
    if (exti_flag_get(P02_4))
    {
    }
}

void gpio_3_exti_isr() // 外部 GPIO_3 中断服务函数
{
}

void gpio_4_exti_isr() // 外部 GPIO_4 中断服务函数
{
}

void gpio_5_exti_isr() // 外部 GPIO_5 中断服务函数
{
}

void gpio_6_exti_isr() // 外部 GPIO_6 中断服务函数
{
}

void gpio_7_exti_isr() // 外部 GPIO_7 中断服务函数
{
}

void gpio_8_exti_isr() // 外部 GPIO_8 中断服务函数
{
}

void gpio_9_exti_isr() // 外部 GPIO_9 中断服务函数
{
}

void gpio_10_exti_isr() // 外部 GPIO_10 中断服务函数
{
}

void gpio_11_exti_isr() // 外部 GPIO_11 中断服务函数
{
}

void gpio_12_exti_isr() // 外部 GPIO_12 中断服务函数
{
}

void gpio_13_exti_isr() // 外部 GPIO_13 中断服务函数
{
}

void gpio_14_exti_isr() // 外部 GPIO_14 中断服务函数
{
}

void gpio_15_exti_isr() // 外部 GPIO_15 中断服务函数
{
}
```

```
void gpio_16_exti_isr() // 外部 GPIO_16 中断服务函数
{
}

void gpio_17_exti_isr() // 外部 GPIO_17 中断服务函数
{
}

void gpio_18_exti_isr() // 外部 GPIO_18 中断服务函数
{
}

void gpio_19_exti_isr() // 外部 GPIO_19 中断服务函数
{
}

void gpio_20_exti_isr() // 外部 GPIO_20 中断服务函数
{
}

void gpio_21_exti_isr() // 外部 GPIO_21 中断服务函数
{
}

void gpio_22_exti_isr() // 外部 GPIO_22 中断服务函数
{
}

void gpio_23_exti_isr() // 外部 GPIO_23 中断服务函数
{
}
// ***** 串口中断函数 *****
```

core4_cyt4bb7/project/user/main_cm7_0.c

SHA-256 :
68b421c6f4fb20c0361f25c4d3b1b4035b4102af3991067b4532706b8f738421

代码 (c)

```
/* *****
 * CYT4BB Opensource Library 即 ( CYT4BB 开源库) 是一个基于官方 SDK 接口的第三方开源库
 * Copyright (c) 2022 SEEKFREE 逐飞科技
 *
 * 本文件是 CYT4BB 开源库的一部分
 *
 * CYT4BB 开源库 是免费软件
 * 您可以根据自由软件基金会发布的 GPL (GNU General Public License, 即 GNU 通用公共许可证) 的条款
 * 即 GPL 的第 3 版 (即 GPL3.0) 或 (您选择的) 任何后来的版本, 重新发布和/或修改它
 *
 * 本开源库的发布是希望它能发挥作用, 但并未对其作任何的保证
 * 甚至没有隐含的适销性或适合特定用途的保证
 * 更多细节请参见 GPL
 *
 * 您应该在收到本开源库的同时收到一份 GPL 的副本
 * 如果没有, 请参阅<https://www.gnu.org/licenses/>
 *
 * 额外注明:
 * 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
 * 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
 * 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
 * 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明 (即本声明)
 *
 * 文件名称      main_cm7_0
 * 公司名称      成都逐飞科技有限公司
 * 版本信息      查看 libraries/doc 文件夹内 version 文件 版本说明
 * 开发环境      IAR 9.40.1
 * 适用平台      CYT4BB
 * 店铺链接      https://seekfree.taobao.com/
 *
 * 修改记录
 * 日期          作者          备注
 * 2024-1-4      pudding      first version
 * ***** */
```

```
#include "zf_common_headfile.h"

#include "core0Loop.h"

// 打开新的工程或者工程移动了位置务必执行以下操作
// 第一步 关闭上面所有打开的文件
// 第二步 project->clean 等待下方进度条走完

// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设

// ***** 代码区域 *****

/* =====
* BEFORE EDITING THIS CORE — READ THESE TWO DOCS FIRST:
* 1. <repo-root>/CLAUDE.md — multi-core architecture, naming rules, git workflow
* 2. ./project/code/PROJECT.md — this core's modules, boundaries, hazards
* Each core has the SAME filenames (motor.c / IMU.c / PID.c) but DIFFERENT semantics.
* Do NOT copy patterns across cores. Do NOT edit two cores in one change.
* ===== */
int main(void)
{
    clock_init(SYSTEM_CLOCK_250M); // 时钟配置及系统初始化<务必保留>
    debug_init(); // 调试串口信息初始化
    // 此处编写用户代码 例如外设初始化代码等

    processInit();

    // 此处编写用户代码 例如外设初始化代码等
    while (true)
    {
        // 此处编写需要循环执行的代码

        whileProcess();

        // 此处编写需要循环执行的代码
    }
}

// ***** 代码区域 *****
```

core4_cyt4bb7/project/user/main_cm7_1.c

SHA-256

:

363e25e84c266e455ba510f21d00be6a76c6b61238c8b3cf827431da3e8084e8

代码（c）

```
/*=====
* CYT4BB Opensourec Library 即（CYT4BB 开源库）是一个基于官方 SDK 接口的第三方开源库
* Copyright (c) 2022 SEEKFREE 逐飞科技
*
* 本文件是 CYT4BB 开源库的一部分
*
* CYT4BB 开源库 是免费软件
* 您可以根据自由软件基金会发布的 GPL（GNU General Public License，即 GNU 通用公共许可证）的条款
* 即 GPL 的第 3 版（即 GPL3.0）或（您选择的）任何后来的版本，重新发布和/或修改它
*
* 本开源库的发布是希望它能发挥作用，但并未对其作任何的保证
* 甚至没有隐含的适销性或适合特定用途的保证
* 更多细节请参见 GPL
*
* 您应该在收到本开源库的同时收到一份 GPL 的副本
* 如果没有，请参阅<https://www.gnu.org/licenses/>
*
* 额外注明：
* 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
* 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
* 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
* 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明（即本声明）
*
* 文件名称      main_cm7_1
* 公司名称      成都逐飞科技有限公司
```

```
* 版本信息      查看 libraries/doc 文件夹内 version 文件 版本说明
* 开发环境      IAR 9.40.1
* 适用平台      CYT4BB
* 店铺链接      https://seekfree.taobao.com/
*
* 修改记录
* 日期          作者          备注
* 2024-1-4      pudding      first version
*****/

#include "zf_common_headfile.h"

#include "core1Loop.h"

// 打开新的工程或者工程移动了位置务必执行以下操作
// 第一步 关闭上面所有打开的文件
// 第二步 project->clean 等待下方进度条走完

// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设

// ***** 代码区域 *****

/* =====
* BEFORE EDITING THIS CORE — READ THESE TWO DOCS FIRST:
* 1. <repo-root>/CLAUDE.md      — multi-core architecture, naming rules, git workflow
* 2. ./project/code/PROJECT.md  — this core's modules, boundaries, hazards
* Each core has the SAME filenames (motor.c / IMU.c / PID.c) but DIFFERENT semantics.
* Do NOT copy patterns across cores. Do NOT edit two cores in one change.
* ===== */
int main(void)
{
    clock_init(SYSTEM_CLOCK_250M); // 时钟配置及系统初始化<务必保留>
    debug_info_init();              // 调试串口信息初始化

    // 此处编写用户代码 例如外设初始化代码等

    processInit();

    // 此处编写用户代码 例如外设初始化代码等
    while (true)
    {
        // 此处编写需要循环执行的代码

        whileProcess();

        // 此处编写需要循环执行的代码
    }
}

// ***** 代码区域 *****
```

coreT3_cyt4bb7

coreT3_cyt4bb7/project/code/commonDef.h

SHA-256

:

0f6202a64cd4714e209191d46d0989aca91b360ed08f647fe4b8fd3cddab2075

代码 (c)

```
#ifndef COMMON_DEF_H
#define COMMON_DEF_H

#include <stdint.h>

/* coreT3 工作模式: cm_7_0 采集 MT9V03X 原始灰度帧并经 WiFi SPI 上行。 */
#define CORET3_MODE_RAW_IMAGE_STREAM (0)
#define CORET3_MODE (CORET3_MODE_RAW_IMAGE_STREAM)

#define range(x, min, max) (((x) < (min)) ? (min) : (((x) > (max)) ? (max) : (x)))
#define unsignRange(x, max) (((x) > (max)) ? (max) : (x))
```

```

#define toRad(x) ((x) * 3.1415 926 535 897 9 323 846f / 180.0f)
#define toDeg(x) ((x) * 180.0f / 3.1415 926 535 897 9 323 846f)

#define BLACK 0
#define WHITE 255
#define cReverse(x) ((x) ? BLACK : WHITE)

#ifndef PI
#define PI (3.1415 926 535 897 9 323 846f)
#endif
#define TWO_PI (2.0f * PI)
#define HALF_PI (0.5f * PI)

typedef struct
{
    uint8_t* ptr;
    uint32_t xLen;
    uint32_t yLen;
    uint32_t size;
} u8Image;

typedef struct
{
    uint16_t* ptr;
    uint32_t xLen;
    uint32_t yLen;
    uint32_t size;
} u16Image;

typedef struct
{
    float* ptr;
    uint32_t xLen;
    uint32_t yLen;
    uint32_t size;
} floatKernel;

typedef union
{
    uint8_t data[4];
    float value;
} floatByteUnion;

#endif /* COMMON_DEF_H */

```

coreT3_cyt4bb7/project/code/config/configImageStream.h

SHA-256

:

3a39d782de255ce2f00fe3896f3f5507603bec87d40b542e075d5441da30c7d1

代码 (c)

```

#ifndef CORET3_CONFIG_IMAGE_STREAM_H
#define CORET3_CONFIG_IMAGE_STREAM_H

#include <stdint.h>

/* 默认匹配 active core0/core1 的 MT9V03X 几何。相机按 core0 已验证 50fps 采集。 */
#define CORET3_IMAGE_STREAM_WIDTH (188u)
#define CORET3_IMAGE_STREAM_HEIGHT (120u)
#define CORET3_IMAGE_CAMERA_FPS (50u)
#define CORET3_IMAGE_NO_FRAME_PERIOD_MS (100u)
#define CORET3_IMAGE_STREAM_AUTO_EXP (0u)
#define CORET3_IMAGE_STREAM_EXP_TIME (101u)
#define CORET3_IMAGE_STREAM_GAIN (32u)

#if (CORET3_IMAGE_STREAM_WIDTH < 1u) || (CORET3_IMAGE_STREAM_WIDTH > 188u)
#error "CORET3_IMAGE_STREAM_WIDTH must be in 1..188"
#endif
#if (CORET3_IMAGE_STREAM_HEIGHT < 1u) || (CORET3_IMAGE_STREAM_HEIGHT > 120u)
#error "CORET3_IMAGE_STREAM_HEIGHT must be in 1..120"
#endif
#if ((CORET3_IMAGE_STREAM_WIDTH * CORET3_IMAGE_STREAM_HEIGHT) > 65 535u)
#error "CORET3 image size must fit MT9V03X driver limit"
#endif

```

```
#if (CORET3_IMAGE_CAMERA_FPS < 1u)
#error "CORET3_IMAGE_CAMERA_FPS must be > 0"
#endif
#if (CORET3_IMAGE_NO_FRAME_PERIOD_MS < 1u)
#error "CORET3_IMAGE_NO_FRAME_PERIOD_MS must be > 0"
#endif

#endif /* CORET3_CONFIG_IMAGE_STREAM_H */
```

coreT3_cyt4bb7/project/code/config/configSerialDebug.h

SHA-256

:

48805c2ec7d45b0ae6383a1d4641d4279468fc091659ea75ce7fc1b2934249b2

代码 (c)

```
#ifndef CORET3_CONFIG_SERIAL_DEBUG_H
#define CORET3_CONFIG_SERIAL_DEBUG_H

#include "config/pinMap.h"

/* coreT3 debug UART 只用于 printf 日志；图像上行走 WiFi SPI。 */
#define SERIAL_DEBUG_UART_INDEX      CORET3_DEBUG_UART_INDEX
#define SERIAL_DEBUG_UART_BAUDRATE   (115200)
#define SERIAL_DEBUG_UART_TX_PIN     CORET3_DEBUG_UART_TX_PIN
#define SERIAL_DEBUG_UART_RX_PIN     CORET3_DEBUG_UART_RX_PIN
#define SERIAL_DEBUG_UART_USE_INTERRUPT (0)

#if (SERIAL_DEBUG_UART_BAUDRATE < 1)
#error "SERIAL_DEBUG_UART_BAUDRATE must be > 0"
#endif

#endif /* CORET3_CONFIG_SERIAL_DEBUG_H */
```

coreT3_cyt4bb7/project/code/config/pinMap.h

SHA-256

:

27daf74557f2e6822acb3505be30f7bb0479a7217f77ada041de15b739531f18

代码 (c)

```
#ifndef CORET3_PIN_MAP_H
#define CORET3_PIN_MAP_H

/* coreT3 硬件映射集中头: 只放引脚/外设常量, 不放业务逻辑。 */
#define CORET3_DEBUG_UART_INDEX      (UART_0)
#define CORET3_DEBUG_UART_TX_PIN     (UART0_TX_P00_1)
#define CORET3_DEBUG_UART_RX_PIN     (UART0_RX_P00_0)

#define CORET3_MT9V03X_COF_IIC_SCL    (P17_1)
#define CORET3_MT9V03X_COF_IIC_SDA    (P17_2)
#define CORET3_MT9V03X_PCLK_PIN      (P06_5)
#define CORET3_MT9V03X_VSYNC_PIN     (P06_6)
#define CORET3_MT9V03X_DATA_PIN      (P18_0)

#define CORET3_KEY_1_PIN              (P20_3)
#define CORET3_KEY_2_PIN              (P20_2)
#define CORET3_KEY_3_PIN              (P20_1)
#define CORET3_KEY_4_PIN              (P20_0)
#define CORET3_KEY_5_PIN              (P19_4)

#endif /* CORET3_PIN_MAP_H */
```

coreT3_cyt4bb7/project/code/core0Loop.c

SHA-256

:

9b58bb23132cf5e341a0fc8dcbd0c71f5317446c37beb90c3e53e03f1e436e69

代码 (c)

```
#include "core0Loop.h"

#include <stdint.h>
#include <string.h>

#include "seekfree_assistant.h"
#include "seekfree_assistant_interface.h"
#include "zf_device_ips114.h"
#include "zf_device_mt9v03x.h"
#include "zf_device_wifi_spi.h"
#include "zf_driver_delay.h"

#include "config/configImageStream.h"
#include "config/configWifiSpi.h"

/* coreT3 极简固件：只做两件事——
 * 1. MT9V03X 采集的整帧经 WiFi SPI 逐飞助手协议发到 PC 上位机；
 * 2. 同一帧在本板 IPS114 显示。
 * 前台单循环、无 PIT/按键/状态计数（参考 SeekFree E6_12 已验证结构；
 * 之前加 10ms PIT 会抢占阻塞式 WiFi SPI 发送，发 1 帧即卡死）。*/

/* 发送前先拷贝到本地缓冲，发送与显示读同一快照，避免与相机 DMA 撕裂。*/
static uint8_t imageCopy[MT9V03X_H][MT9V03X_W];

static void wifiSpiInitBlock(void)
{
    while (wifi_spi_init(CORET3_WIFI_SSID, CORET3_WIFI_PASSWORD))
    {
        ips114_show_string(0u, 0u, "WiFi connecting..");
        system_delay_ms(100u);
    }

    if (1 != WIFI_SPI_AUTO_CONNECT)
    {
        while (wifi_spi_socket_connect("TCP",
                                         CORET3_WIFI_TARGET_IP,
                                         CORET3_WIFI_TARGET_PORT,
                                         CORET3_WIFI_LOCAL_PORT))
        {
            {
                ips114_show_string(0u, 0u, "TCP connecting..");
                system_delay_ms(100u);
            }
        }
    }
}

void processInit(void)
{
    /* 本驱动方向命名反直觉：PORTAIT=240x135(宽 x 高)、CROSSWISE=135x240。
     * 188 宽需要 240 宽的绘制区，故用 PORTAIT_180 才容得下 188x120，
     * 用 CROSSWISE(宽 135) 会因 x+188-1>135 触发 set_region 超边界断言。*/
    ips114_set_dir(IPS114_PORTAIT_180);
    ips114_init();
    ips114_clear();
    ips114_show_string(0u, 0u, "coreT3 WiFi SPI");

    wifiSpiInitBlock();

    mt9v03x_init();
    seekfree_assistant_interface_init(SEEKFREE_ASSISTANT_WIFI_SPI);
    seekfree_assistant_camera_information_config(SEEKFREE_ASSISTANT_MT9V03X,
                                                imageCopy[0],
                                                MT9V03X_W,
                                                MT9V03X_H);

    ips114_clear();
}

void pitProcess0(void)
{
}

void pitProcess1(void)
{
}

void whileProcess(void)
```

```

{
    if (mt9v03x_finish_flag)
    {
        mt9v03x_finish_flag = 0u;
        memcpy(imageCopy[0], mt9v03x_image[0], MT9V03X_IMAGE_SIZE);

        seekfree_assistant_camera_send();
        ips114_show_gray_image(0u, 0u, imageCopy[0], MT9V03X_W, MT9V03X_H, MT9V03X_W, MT9V03X_H, 0u);
    }
}

```

coreT3_cyt4bb7/project/code/core0Loop.h

SHA-256

:

439701ddaf047773a4179076bea41afedb1708f90b47244ff7cf4de10c0b7058

代码 (c)

```

#ifndef _CORE0LOOP_H_
#define _CORE0LOOP_H_

#include <stdint.h>

#include "commonDef.h"

void processInit(void);
void pitProcess0(void);
void pitProcess1(void);
void whileProcess(void);

#endif /* _CORE0LOOP_H_ */

```

coreT3_cyt4bb7/project/code/core1Loop.c

SHA-256

:

f7fd37b926e39f8ae78749f2fd7c94f331960e6aceca669b62d77392da74e62e

代码 (c)

```

#include "core1Loop.h"

#include "commonDef.h"

static volatile uint8_t initReadyFlag = 0u;

void processInit(void)
{
    initReadyFlag = 1u;
}

void pitProcess0(void)
{
    if (!initReadyFlag) return;
}

void whileProcess(void)
{
    if (!initReadyFlag) return;
}

```

coreT3_cyt4bb7/project/code/core1Loop.h

SHA-256

:

030adc393b85cebd984ca84f8e555a05274425c2f79499f0902079bc7db9f319

代码 (c)

```
#ifndef _CORE1LOOP_H_
#define _CORE1LOOP_H_

#include <stdint.h>

#include "cy_device_headers.h"

void processInit(void);
void pitProcess0(void);
void whileProcess(void);

#endif /* _CORE1LOOP_H_ */
```

coreT3_cyt4bb7/project/code/lowPassFilter.c

SHA-256

:

36e5dbbded0830f3b91308821132c824fa42a1411f3817fc1e7c4ce7a4a2ba85

代码 (c)

```
/*
 * lowPassFilter.c
 *
 * Created: 20 250 319
 * Author: Hecate
 */
#include "lowPassFilter.h"

void lowPassFilterInit(LowPassFilterData *obj, float alpha)
{
    obj->alpha = alpha;

    obj->filteredValue = 0;
    obj->firstFlag = 1;
    obj->lastOutput = 0;
}

void lowPassFilterReset(LowPassFilterData *obj, float value)
{
    obj->firstFlag = 0u;
    obj->lastOutput = value;
    obj->filteredValue = value;
}

void lowPassFilter(LowPassFilterData *obj, const float value)
{
    if ((obj)->firstFlag == 1u)
    {
        (obj)->firstFlag = 0u;
        (obj)->lastOutput = value;
    }
    (obj)->filteredValue = (obj)->alpha * value + (1.0f - (obj)->alpha) * (obj)->lastOutput;
    (obj)->lastOutput = (obj)->filteredValue;
}
```

coreT3_cyt4bb7/project/code/lowPassFilter.h

SHA-256

:

e5deedc753466e971994dc6d6a19c97a2853d175c5a17578994096c17b64ec50

代码 (c)

```
/*
 * lowPassFilter.h
 *
 * Created: 20 250 319
 * Author: Hecate
 */
```

```

#ifndef CODE_LOWPASSFILTER_H_
#define CODE_LOWPASSFILTER_H_

#include <stdint.h>

// 一阶低通滤波器句柄
typedef struct
{
    float alpha;
    float lastOutput;
    float filteredValue;
    uint8_t firstFlag;
} LowPassFilterData;

// 初始化滤波器参数
void lowPassFilterInit(LowPassFilterData *obj, float alpha);

// 将滤波器状态强制锚定到指定值 (firstFlag 清零, 下一拍从此值连续滤波)
// 用于"非锁定态直接跟随原始值"场景, 保证后续进入滤波时无突跳
void lowPassFilterReset(LowPassFilterData *obj, float value);

// 输入新样本并更新滤波值
void lowPassFilter(LowPassFilterData *obj, const float value);

#endif /* CODE_LOWPASSFILTER_H_ */

```

coreT3_cyt4bb7/project/code/om_beacon.c

SHA-256

:

0b347f9b7acd7150d28431bcb9f97b57bbd2bb8fda5ddd052e129de152db153f

代码 (c)

```

/*
 * om_beacon.c - [6] 信标检测 (Beacon Detection)
 *
 * 基于 Canny 边缘 + 简化 BFS, 检测多个红外信标 (闭合椭圆/圆形轮廓)
 * 判据: 闭合性 + 尺寸 + 偏心率上限 + 内部亮度
 * 复用 om_contour.c 的 inContourMap / contourQueue / contourGeneration 基础设施
 */

#include "om_internal.h"

// ---- typedef ----

typedef struct
{
    uint16_t sizeMax;
    uint16_t sizeMin;
    float eccMaxRatioK; // 预计算: ((eccMax-1)/(eccMax+1))^2, 用于无 sqrt 偏心率比较
    uint8_t brightnessMin; // 内部亮度下限
    uint16_t maxEndpoints; // 允许的最大端点数 (信标边缘常有断裂)
} BEACON_DATA_CONFIG;

// ---- 内部变量 ----

static BEACON_DATA_CONFIG beaconDataConfig;

// ---- 内部函数声明 ----

static inline uint16_t beaconBFS(const u8Image* edgeSrc, uint16_t startX, uint16_t startY, uint32_t genTag, BEACON_DATA_STRUCT* beacon, float* outMxx, float* outMyy, float* outMxy);

// ---- 外部函数实现 ----

void beaconDetectInit(uint16_t sizeMin, uint16_t sizeMax, float eccMax, uint8_t brightnessMin, uint16_t maxEndpoints)
{
    beaconDataConfig.sizeMin = sizeMin;
    beaconDataConfig.sizeMax = sizeMax;
    // 预计算偏心率阈值的平方比值, 运行时只需一次乘法+比较
    float k = (eccMax - 1.0f) / (eccMax + 1.0f);
    beaconDataConfig.eccMaxRatioK = k * k;
    beaconDataConfig.brightnessMin = brightnessMin;
    beaconDataConfig.maxEndpoints = maxEndpoints;
}

```

```

uint16_t beaconDetect(const u8Image* edgeSrc, const u8Image* graySrc, BEACON_DETECTOR_STRUCT* obj)
{
    if (edgeSrc->size > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE) return 0;

    // generation 替代 memset (与 contourDetect 共享基础设施)
    contourGeneration++;
    if (contourGeneration == 0)
    {
        memset(inContourMap, 0, sizeof(inContourMap));
        contourGeneration = 1;
    }
    uint32_t genBase = (uint32_t) contourGeneration << 16;

    uint16_t beaconCount = 0;
    uint32_t tagCounter = 0;

    for (uint32_t y = 0; y < edgeSrc->yLen && beaconCount < 100; y++)
    {
        uint32_t rowOff = y * edgeSrc->xLen;
        for (uint32_t x = 0; x < edgeSrc->xLen && beaconCount < 100; x++)
        {
            uint32_t i = rowOff + x;
            if (edgeSrc->ptr[i] != WHITE || (inContourMap[i] >> 16) == contourGeneration) continue;

            tagCounter++;
            if (tagCounter > 0xFFFFu) goto beaconDone;

            uint32_t genTag = genBase | (uint32_t) tagCounter;

            float curMxx, curMyy, curMxy;
            uint16_t ep = beaconBFS(edgeSrc, (uint16_t) x, (uint16_t) y, genTag,
                                    &obj->beaconData[beaconCount], &curMxx, &curMyy, &curMxy);

            // 闭合轮廓过滤 (信标边缘断裂容忍)
            if (ep > beaconDataConfig.maxEndpoints) continue;

            // 尺寸过滤
            uint16_t sz = obj->beaconData[beaconCount].contourSize;
            if (sz < beaconDataConfig.sizeMin || sz > beaconDataConfig.sizeMax) continue;

            // 偏心率上限过滤: 拒绝过度拉伸的非信标轮廓
            // eigMax/eigMin > eccMax  <=>  diff^2 > sum^2 * ((eccMax-1)/(eccMax+1))^2
            float eigSum = curMxx + curMyy;
            float eigDiff2 = (curMxx - curMyy) * (curMxx - curMyy) + 4.0f * curMxy * curMxy;
            if (eigSum > 0.0f)
            {
                float bound = eigSum * eigSum * beaconDataConfig.eccMaxRatioK;
                if (eigDiff2 > bound) continue;
            }

            // 质心亮度检查: 填充椭圆的质心必在填充区域内, 单像素即可判定
            uint16_t cx = obj->beaconData[beaconCount].centerX;
            uint16_t cy = obj->beaconData[beaconCount].centerY;
            uint8_t centroidBrightness = graySrc->ptr[(uint32_t) cy * graySrc->xLen + (uint32_t) cx];
            obj->beaconData[beaconCount].brightness = centroidBrightness;
            if (centroidBrightness < beaconDataConfig.brightnessMin) continue;

            beaconCount++;
        }
    }

beaconDone:
    obj->beaconCount = beaconCount;
    for (uint16_t i = beaconCount; i < 101; i++) obj->beaconData[i].contourSize = 0;
    return beaconCount;
}

// ---- 内部函数实现 ----

// 简化 BFS: 仅计算二阶矩 (无三阶矩/朝向), 返回端点数
static inline uint16_t beaconBFS(const u8Image* obj, uint16_t startX, uint16_t startY, uint32_t genTag, BEACON_DATA_STRUCT* beacon, float* outMxx,
float* outMyy, float* outMxy)
{
    uint32_t width = obj->xLen;
    uint32_t height = obj->yLen;

    beacon->contourSize = 0;
    beacon->centerX = 0;

```

```

beacon->centerY = 0;
beacon->brightness = 0;

uint16_t head = 0;
uint16_t tail = 0;

inContourMap[startY * width + startX] = genTag;
contourQueue[tail++] = ((uint32_t) startY << 16) | (uint32_t) startX;

uint32_t sumX = 0;
uint32_t sumY = 0;
uint32_t sumXX = 0;
uint32_t sumYY = 0;
uint32_t sumXY = 0;
uint16_t totalCount = 0;
uint16_t endpointCount = 0;
uint8_t abandon = 0;

while (head < tail)
{
    uint16_t cx = (uint16_t) (contourQueue[head] & 0xFFFFu);
    uint16_t cy = (uint16_t) (contourQueue[head] >> 16);
    head++;

    totalCount++;

    if (!abandon)
    {
        sumX += cx;
        sumY += cy;
        sumXX += (uint32_t) cx * cx;
        sumYY += (uint32_t) cy * cy;
        sumXY += (uint32_t) cx * cy;
    }

    uint8_t whiteNeighborCount = 0;

    for (uint8_t i = 0; i < 8; i++)
    {
        int32_t nx = (int32_t) cx + contourDx[i];
        int32_t ny = (int32_t) cy + contourDy[i];

        if (nx < 0 || (uint32_t) nx >= width || ny < 0 || (uint32_t) ny >= height) continue;

        uint16_t nIndex = (uint16_t) ((uint32_t) ny * width + (uint32_t) nx);

        if (obj->ptr[nIndex] == WHITE)
        {
            whiteNeighborCount++;

            if ((inContourMap[nIndex] >> 16) != contourGeneration)
            {
                inContourMap[nIndex] = genTag;
                if (tail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
                    contourQueue[tail++] = ((uint32_t) ny << 16) | (uint32_t) nx;
            }
        }
    }

    if (!abandon && whiteNeighborCount < 2) endpointCount++;

    if (!abandon && (totalCount > beaconDataConfig.sizeMax || endpointCount > beaconDataConfig.maxEndpoints)) abandon = 1;
}

beacon->contourSize = totalCount;

if (totalCount > 0 && !abandon)
{
    float fN = (float) totalCount;
    float meanX = (float) sumX / fN;
    float meanY = (float) sumY / fN;
    beacon->centerX = (uint16_t) (sumX / totalCount);
    beacon->centerY = (uint16_t) (sumY / totalCount);

    *outMxx = (float) sumXX / fN - meanX * meanX;
    *outMyy = (float) sumYY / fN - meanY * meanY;
    *outMxy = (float) sumXY / fN - meanX * meanY;
}
else

```

```

    {
        *outMxx = 0.0f;
        *outMyy = 0.0f;
        *outMxy = 0.0f;
    }

    return endpointCount;
}

```

coreT3_cyt4bb7/project/code/om_binary.c

SHA-256

:

1db51de7e87df6f34d5052594bff59289da1cfe494054af95691d78cc7ff8763

代码 (c)

```

/*
 * om_binary.c - [2] 二值化
 * 单阈值 / 双阈值 / RGB565 二值化
 */

#include "om_internal.h"

// ---- typedef ----

typedef struct
{
    uint8_t threshold;
    uint8_t thresholdUp;
    uint8_t thresholdDown;
    uint8_t background;
    uint8_t r_min, r_max;
    uint8_t g_min, g_max;
    uint8_t b_min, b_max;
} binaryConfigStruct;

// ---- 内部变量 ----

static binaryConfigStruct binaryConfig;

// ---- 外部函数实现 ----

void toBinaryInit(uint8_t threshold)
{
    binaryConfig.threshold = threshold;
}

void toBinaryDoubleThresholdInit(uint8_t thresholdUp, uint8_t thresholdDown)
{
    binaryConfig.thresholdUp = thresholdUp;
    binaryConfig.thresholdDown = thresholdDown;
}

void rgb565toBinaryInit(uint8_t background, uint8_t r_min, uint8_t r_max, uint8_t g_min, uint8_t g_max, uint8_t b_min, uint8_t b_max)
{
    binaryConfig.background = background;
    binaryConfig.r_min = r_min;
    binaryConfig.r_max = r_max;
    binaryConfig.g_min = g_min;
    binaryConfig.g_max = g_max;
    binaryConfig.b_min = b_min;
    binaryConfig.b_max = b_max;
}

inline void toBinary(const u8Image* src, u8Image* dst)
{
    for (uint16_t i = 0; i < src->xLen * src->yLen; i++) dst->ptr[i] = (src->ptr[i] > binaryConfig.threshold) ? WHITE : BLACK;
}

inline void toBinaryDoubleThreshold(const u8Image* src, u8Image* dst)
{
    for (uint16_t i = 0; i < src->xLen * src->yLen; i++) dst->ptr[i] = ((src->ptr[i] < binaryConfig.thresholdDown) ? BLACK : ((src->ptr[i] <=
binaryConfig.thresholdUp) ? WHITE : BLACK));
}

```

```

inline void rgb565toBinary(const u8Image* src, u8Image* dst)
{
    // 注意: src->ptr 按小端存储 RGB565, 每像素 2 字节
    // src->xLen/yLen 为像素维度, 迭代像素总数
    for (uint32_t i = 0; i < src->xLen * src->yLen; i++)
    {
        uint16_t pixel = (uint16_t) src->ptr[i * 2u] | ((uint16_t) src->ptr[i * 2u + 1u] << 8);

        // 从 RGB565 格式中提取 RGB 分量
        uint8_t r = (pixel >> 11) << 3;    // 红色分量 (5 位 -> 8 位)
        uint8_t g = ((pixel >> 5) & 0x3F) << 2; // 绿色分量 (6 位 -> 8 位)
        uint8_t b = (pixel & 0x1F) << 3;    // 蓝色分量 (5 位 -> 8 位)

        // 根据范围判断是否为前景
        dst->ptr[i] = (r >= binaryConfig.r_min && r <= binaryConfig.r_max && g >= binaryConfig.g_min && g <= binaryConfig.g_max && b >=
        binaryConfig.b_min && b <= binaryConfig.b_max) ? cReverse(binaryConfig.background) : binaryConfig.background;
    }
}

```

coreT3_cyt4bb7/project/code/om_canny.c

SHA-256

:

a0b3b3156172737bd61c873e467e0ca6d7a654b17c3bacb5556a560e43d5d759

代码 (c)

```

/*
 * om_canny.c - [4] 高斯模糊及 Canny 边缘检测
 *
 * 3x3 定点高斯核 / 逐行流水线 Sobel+NMS+滞后门限 边缘检测
 */

#include "om_internal.h"

// ---- define ----

#define CANNY_RING_WIDTH (OM_CANNY_RING_WIDTH)

// ---- typedef ----

typedef struct
{
    uint32_t width;
    uint32_t height;
    uint32_t pixelCount;
    uint32_t cropStartX;
    uint32_t cropStartY;
    uint32_t cropEndX;
    uint32_t cropEndY;
    uint32_t blurStartX;
    uint32_t blurStartY;
    uint32_t blurEndX;
    uint32_t blurEndY;
} gaussConfigStruct;

typedef struct
{
    uint32_t width;
    uint32_t height;
    uint32_t pixelCount;
    uint32_t sobelStartX;
    uint32_t sobelStartY;
    uint32_t sobelEndX;
    uint32_t sobelEndY;
    uint32_t nmsStartX;
    uint32_t nmsStartY;
    uint32_t nmsEndX;
    uint32_t nmsEndY;
    uint16_t thresholdLow;
    uint16_t thresholdHigh;
    uint32_t thresholdLowSq;
    uint32_t thresholdHighSq;
    int32_t trackOffsets[8];
} cannyConfigStruct;

```



```

// ---- 内部变量 ----

static uint16_t gaussKernelPreValueInt0[256];
static uint16_t gaussKernelPreValueInt1[256];
static uint16_t gaussKernelPreValueInt2[256];
static gaussConfigStruct gaussConfig = {0u};
static cannyConfigStruct cannyConfig = {0u};
static uint32_t sobelMapRing[3][CANNY_RING_WIDTH] = {0};
static uint8_t sobelDirMapRing[3][CANNY_RING_WIDTH] = {0};
static uint16_t cannyEdgeQueue[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE] = {0};

// ---- 外部函数实现 ----

// kernelSize = 3u;
void gaussianInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY)
{
    // gaussianKernelTemp[0] : 四个角点对应的核值
    // gaussianKernelTemp[1] : 四个边点对应的核值
    // gaussianKernelTemp[2] : 中心点对应的核值
    float gaussianKernelTemp[3] = {0.0f, 0.0f, 0.0f};

    memset(&gaussConfig, 0, sizeof(gaussConfig));

    if ((NULL == src) || (NULL == src->ptr) || (0u == src->xLen) || (0u == src->yLen) ||
        (src->xLen > CANNY_RING_WIDTH) || ((src->xLen * src->yLen) > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE))
    {
        memset(gaussKernelPreValueInt0, 0, sizeof(gaussKernelPreValueInt0));
        memset(gaussKernelPreValueInt1, 0, sizeof(gaussKernelPreValueInt1));
        memset(gaussKernelPreValueInt2, 0, sizeof(gaussKernelPreValueInt2));
        return;
    }

    gaussConfig.width = src->xLen;
    gaussConfig.height = src->yLen;
    gaussConfig.pixelCount = src->xLen * src->yLen;

    if (startX > endX)
    {
        uint32_t temp = startX;
        startX = endX;
        endX = temp;
    }
    if (startY > endY)
    {
        uint32_t temp = startY;
        startY = endY;
        endY = temp;
    }

    gaussConfig.cropStartX = unsignRange(startX, gaussConfig.width - 1u);
    gaussConfig.cropStartY = unsignRange(startY, gaussConfig.height - 1u);
    gaussConfig.cropEndX = range(endX, gaussConfig.cropStartX, gaussConfig.width - 1u);
    gaussConfig.cropEndY = range(endY, gaussConfig.cropStartY, gaussConfig.height - 1u);

    gaussConfig.blurStartX = 1u;
    gaussConfig.blurStartY = 1u;
    gaussConfig.blurEndX = 0u;
    gaussConfig.blurEndY = 0u;

    if ((gaussConfig.width >= 3u) && (gaussConfig.height >= 3u) &&
        ((gaussConfig.cropEndX - gaussConfig.cropStartX) >= 2u) &&
        ((gaussConfig.cropEndY - gaussConfig.cropStartY) >= 2u))
    {
        gaussConfig.blurStartX = (0u == gaussConfig.cropStartX) ? 1u : (gaussConfig.cropStartX + 1u);
        gaussConfig.blurStartY = (0u == gaussConfig.cropStartY) ? 1u : (gaussConfig.cropStartY + 1u);
        gaussConfig.blurEndX = (gaussConfig.cropEndX >= (gaussConfig.width - 1u)) ? (gaussConfig.width - 2u) : (gaussConfig.cropEndX - 1u);
        gaussConfig.blurEndY = (gaussConfig.cropEndY >= (gaussConfig.height - 1u)) ? (gaussConfig.height - 2u) : (gaussConfig.cropEndY - 1u);
    }

    if (sigma <= 0.0f)
    {
        memset(gaussKernelPreValueInt0, 0, sizeof(gaussKernelPreValueInt0));
        memset(gaussKernelPreValueInt1, 0, sizeof(gaussKernelPreValueInt1));
        memset(gaussKernelPreValueInt2, 0, sizeof(gaussKernelPreValueInt2));
        return;
    }

    float sigma2 = sigma * sigma;

```

```

float coeff = 1.0f / (TWO_PI * sigma2);
float sum = 0.0f;

// 生成 3x3 高斯核，并提取 3 个唯一值（角点/边点/中心）
for (uint8_t y = 0u; y < 3; y++)
{
    for (uint8_t x = 0u; x < 3; x++)
    {
        int16_t dx = (int16_t) x - 1;
        int16_t dy = (int16_t) y - 1;
        float exponent = -((float) (dx * dx + dy * dy)) / (2.0f * sigma2);
        float value = (float) exp((double) exponent) * coeff;

        if (0 == dx && 0 == dy) gaussianKernelTemp[2] = value;
        else if (0 == dx || 0 == dy) gaussianKernelTemp[1] = value;
        else gaussianKernelTemp[0] = value;

        sum += value;
    }
}

if (sum <= 0.0f)
{
    memset(gaussKernelPreValueInt0, 0, sizeof(gaussKernelPreValueInt0));
    memset(gaussKernelPreValueInt1, 0, sizeof(gaussKernelPreValueInt1));
    memset(gaussKernelPreValueInt2, 0, sizeof(gaussKernelPreValueInt2));
    return;
}

// 归一化 3 个唯一核值
gaussianKernelTemp[0] /= sum;
gaussianKernelTemp[1] /= sum;
gaussianKernelTemp[2] /= sum;

// 预计算核值与 8bit 灰度值相乘的定点数(放缩 256 倍)，消除内层浮点开销
for (uint16_t gray = 0u; gray < 256; gray++)
{
    gaussKernelPreValueInt0[gray] = (uint16_t) (gaussianKernelTemp[0] * (float) gray * 256.0f);
    gaussKernelPreValueInt1[gray] = (uint16_t) (gaussianKernelTemp[1] * (float) gray * 256.0f);
    gaussKernelPreValueInt2[gray] = (uint16_t) (gaussianKernelTemp[2] * (float) gray * 256.0f);
}
}

void gaussianBlur(const u8Image* src, u8Image* dst)
{
    if (NULL == src || NULL == src->ptr || NULL == dst || NULL == dst->ptr) return;
    if (0u == gaussConfig.width || gaussConfig.blurStartX > gaussConfig.blurEndX) return;

    for (uint32_t y = gaussConfig.blurStartY; y <= gaussConfig.blurEndY; y++)
    {
        const uint8_t* row0 = src->ptr + (y - 1u) * gaussConfig.width;
        const uint8_t* row1 = src->ptr + y * gaussConfig.width;
        const uint8_t* row2 = src->ptr + (y + 1u) * gaussConfig.width;
        uint8_t* dstRow = dst->ptr + y * gaussConfig.width;

        for (uint32_t x = gaussConfig.blurStartX; x <= gaussConfig.blurEndX; x++)
        {
            uint32_t result = gaussKernelPreValueInt0[row0[x - 1u]] + gaussKernelPreValueInt1[row0[x]] + gaussKernelPreValueInt0[row0[x + 1u]] +
                gaussKernelPreValueInt1[row1[x - 1u]] + gaussKernelPreValueInt2[row1[x]] + gaussKernelPreValueInt1[row1[x + 1u]] +
                gaussKernelPreValueInt0[row2[x - 1u]] + gaussKernelPreValueInt1[row2[x]] + gaussKernelPreValueInt0[row2[x + 1u]];

            result = (result + 128) >> 8;
            dstRow[x] = (uint8_t) ((result > 255) ? 255 : result);
        }
    }
}

void cannyInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY, uint16_t thresholdLow, uint16_t thresholdHigh)
{
    gaussianInit(src, sigma, startX, startY, endX, endY);

    memset(&cannyConfig, 0, sizeof(cannyConfig));

    if ((NULL == src) || (NULL == src->ptr) || (0u == src->xLen) || (0u == src->yLen) ||
        (src->xLen > CANNY_RING_WIDTH) || ((src->xLen * src->yLen) > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)) return;

    cannyConfig.width = src->xLen;
    cannyConfig.height = src->yLen;
}

```

```

cannyConfig.pixelCount = src->xLen * src->yLen;

if (thresholdLow > thresholdHigh)
{
    uint16_t temp = thresholdLow;
    thresholdLow = thresholdHigh;
    thresholdHigh = temp;
}

cannyConfig.thresholdLow = thresholdLow;
cannyConfig.thresholdHigh = thresholdHigh;

// 提前计算平方
cannyConfig.thresholdLowSq = (uint32_t) thresholdLow * thresholdLow;
cannyConfig.thresholdHighSq = (uint32_t) thresholdHigh * thresholdHigh;

cannyConfig.sobelStartX = 1u;
cannyConfig.sobelStartY = 1u;
cannyConfig.sobelEndX = 0u;
cannyConfig.sobelEndY = 0u;
cannyConfig.nmsStartX = 1u;
cannyConfig.nmsStartY = 1u;
cannyConfig.nmsEndX = 0u;
cannyConfig.nmsEndY = 0u;

if ((gaussConfig.blurEndX > (gaussConfig.blurStartX + 1u)) &&
    (gaussConfig.blurEndY > (gaussConfig.blurStartY + 1u)))
{
    cannyConfig.sobelStartX = gaussConfig.blurStartX + 1u;
    cannyConfig.sobelStartY = gaussConfig.blurStartY + 1u;
    cannyConfig.sobelEndX = gaussConfig.blurEndX - 1u;
    cannyConfig.sobelEndY = gaussConfig.blurEndY - 1u;
}

if ((cannyConfig.sobelEndX > (cannyConfig.sobelStartX + 1u)) &&
    (cannyConfig.sobelEndY > (cannyConfig.sobelStartY + 1u)))
{
    cannyConfig.nmsStartX = cannyConfig.sobelStartX + 1u;
    cannyConfig.nmsStartY = cannyConfig.sobelStartY + 1u;
    cannyConfig.nmsEndX = cannyConfig.sobelEndX - 1u;
    cannyConfig.nmsEndY = cannyConfig.sobelEndY - 1u;
}

// 预先计算滞后门限追踪(连边)阶段需要的直接邻信偏移
cannyConfig.trackOffsets[0] = -(int32_t) gaussConfig.width - 1;
cannyConfig.trackOffsets[1] = -(int32_t) gaussConfig.width;
cannyConfig.trackOffsets[2] = -(int32_t) gaussConfig.width + 1;
cannyConfig.trackOffsets[3] = -1;
cannyConfig.trackOffsets[4] = 1;
cannyConfig.trackOffsets[5] = (int32_t) gaussConfig.width - 1;
cannyConfig.trackOffsets[6] = (int32_t) gaussConfig.width;
cannyConfig.trackOffsets[7] = (int32_t) gaussConfig.width + 1;
}

/* 运行时仅更新 Canny 双阈值，不重算高斯核与 ROI，便于按键调参 */
void cannySetThreshold(uint16_t thresholdLow, uint16_t thresholdHigh)
{
    if (thresholdLow > thresholdHigh)
    {
        uint16_t temp = thresholdLow;
        thresholdLow = thresholdHigh;
        thresholdHigh = temp;
    }

    cannyConfig.thresholdLow = thresholdLow;
    cannyConfig.thresholdHigh = thresholdHigh;
    cannyConfig.thresholdLowSq = (uint32_t) thresholdLow * thresholdLow;
    cannyConfig.thresholdHighSq = (uint32_t) thresholdHigh * thresholdHigh;
}

void cannyEdgeDetection(const u8Image* src, u8Image* dstGaussBlur, u8Image* dst)
{
    if (NULL == src || NULL == src->ptr || NULL == dstGaussBlur || NULL == dstGaussBlur->ptr ||
        NULL == dst || NULL == dst->ptr) return;
    if (0u == cannyConfig.pixelCount || 0u == gaussConfig.width) return;

    uint32_t queueHead = 0u;
    uint32_t queueTail = 0u;

```

```

// 清零目标画布、ring buffer 残留、高斯模糊边界脏值
memset(dst->ptr, BLACK, gaussConfig.width * gaussConfig.height);
memset(dstGaussBlur->ptr, 0, gaussConfig.width * gaussConfig.height);
memset(sobelMapRing, 0, sizeof(sobelMapRing));
memset(sobelDirMapRing, 0, sizeof(sobelDirMapRing));

for (uint32_t y = gaussConfig.blurStartY; y <= gaussConfig.blurEndY; y++)
{
    // ===== 第1阶段: 高斯模糊 =====
    const uint8_t* row0 = src->ptr + (y - 1u) * gaussConfig.width;
    const uint8_t* row1 = src->ptr + y * gaussConfig.width;
    const uint8_t* row2 = src->ptr + (y + 1u) * gaussConfig.width;
    uint8_t* dstRow = dstGaussBlur->ptr + y * gaussConfig.width;

    for (uint32_t x = gaussConfig.blurStartX; x <= gaussConfig.blurEndX; x++)
    {
        uint32_t result = gaussKernelPreValueInt0[row0[x - 1u]] + gaussKernelPreValueInt1[row0[x]] + gaussKernelPreValueInt0[row0[x + 1u]] +
            gaussKernelPreValueInt1[row1[x - 1u]] + gaussKernelPreValueInt2[row1[x]] + gaussKernelPreValueInt1[row1[x + 1u]] +
            gaussKernelPreValueInt0[row2[x - 1u]] + gaussKernelPreValueInt1[row2[x]] + gaussKernelPreValueInt0[row2[x + 1u]];

        result = (result + 128) >> 8;
        dstRow[x] = (uint8_t) ((result > 255) ? 255 : result);
    }

    // ===== 第2阶段: Sobel 计算 =====
    if ((cannyConfig.sobelStartX <= cannyConfig.sobelEndX) &&
        (cannyConfig.sobelStartY <= cannyConfig.sobelEndY) &&
        (y >= (cannyConfig.sobelStartY + 1u)) &&
        ((y - 1u) <= cannyConfig.sobelEndY))
    {
        const uint8_t* s_row0 = dstGaussBlur->ptr + (y - 2u) * gaussConfig.width;
        const uint8_t* s_row1 = dstGaussBlur->ptr + (y - 1u) * gaussConfig.width;
        const uint8_t* s_row2 = dstGaussBlur->ptr + y * gaussConfig.width;

        uint32_t* sobelRow = sobelMapRing[(y - 1u) % 3];
        uint8_t* dirRow = sobelDirMapRing[(y - 1u) % 3];

        for (uint32_t x = cannyConfig.sobelStartX; x <= cannyConfig.sobelEndX; x++)
        {
            int32_t gx = (int32_t) s_row0[x + 1u] + ((int32_t) s_row1[x + 1u] << 1) + (int32_t) s_row2[x + 1u] - (int32_t) s_row0[x - 1u] - ((int32_t) s_row1[x - 1u] << 1) - (int32_t) s_row2[x - 1u];
            int32_t gy = (int32_t) s_row0[x - 1u] + ((int32_t) s_row0[x] << 1) + (int32_t) s_row0[x + 1u] - (int32_t) s_row2[x - 1u] - ((int32_t) s_row2[x] << 1) - (int32_t) s_row2[x + 1u];

            // 去除掉极其耗时的 sqrtf 运算
            uint32_t mag_sq = (uint32_t) (gx * gx) + (uint32_t) (gy * gy);
            sobelRow[x] = mag_sq;

            // 完全消除浮点除法, 只用整数比较判定 4 大区域方向
            int32_t ax = (gx >= 0) ? gx : -gx;
            int32_t ay = (gy >= 0) ? gy : -gy;
            int32_t ay256 = ay << 8;

            if (ay256 <= ax * 106)
            {
                dirRow[x] = 0; // 水平梯度主导 (纵向边缘), 沿水平方向抑制
            }
            else if (ay256 >= ax * 618)
            {
                dirRow[x] = 2; // 垂直梯度主导 (横向边缘), 沿垂直方向抑制
            }
            else if ((gx ^ gy) >= 0)
            {
                dirRow[x] = 1; // 45 度梯度
            }
            else
            {
                dirRow[x] = 3; // 135 度梯度
            }
        }
    }

    // ===== 第3阶段: 非极大值抑制 =====
    if ((cannyConfig.nmsStartX <= cannyConfig.nmsEndX) &&
        (cannyConfig.nmsStartY <= cannyConfig.nmsEndY) &&
        (y >= (cannyConfig.nmsStartY + 2u)) &&
        ((y - 2u) <= cannyConfig.nmsEndY))
    {
        const uint32_t* n_row0 = sobelMapRing[(y - 3u) % 3];
    }
}

```

```

const uint32_t* n_row1 = sobelMapRing[(y - 2u) % 3];
const uint32_t* n_row2 = sobelMapRing[(y - 1u) % 3];
const uint8_t* dirRow = sobelDirMapRing[(y - 2u) % 3];

uint8_t* final_dstRow = dst->ptr + (y - 2u) * gaussConfig.width + cannyConfig.nmsStartX;

for (uint32_t x = cannyConfig.nmsStartX; x <= cannyConfig.nmsEndX; x++)
{
    uint32_t edge0, edge1;
    uint32_t magnitude = n_row1[x];
    uint8_t dir = dirRow[x];

    if (dir == 0)
    {
        edge0 = n_row1[x - 1u];
        edge1 = n_row1[x + 1u];
    }
    else if (dir == 2)
    {
        edge0 = n_row0[x];
        edge1 = n_row2[x];
    }
    else if (dir == 1)
    {
        // gx, gy 同号 → 梯度方向 ↗ (亮度梯度指向右上) → NMS 比较 ↗ 与 ↘
        edge0 = n_row0[x + 1u];
        edge1 = n_row2[x - 1u];
    }
    else
    {
        // gx, gy 异号 → 梯度方向 ↘ (亮度梯度指向右下) → NMS 比较 ↘ 与 ↗
        edge0 = n_row0[x - 1u];
        edge1 = n_row2[x + 1u];
    }

    if ((magnitude > edge0) && (magnitude >= edge1))
    {
        if (magnitude >= cannyConfig.thresholdHighSq)
        {
            *final_dstRow = WHITE; // 确信强边缘
            if (queueTail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
            {
                cannyEdgeQueue[queueTail++] = (uint16_t)((y - 2u) * gaussConfig.width + x);
            }
        }
        else if (magnitude >= cannyConfig.thresholdLowSq)
        {
            *final_dstRow = 127; // 先标记为弱边缘，用于第四阶段追踪，避免大数组分配
        }
    }

    final_dstRow++;
}
}

// ===== 第4阶段: 滞后门限追踪(连边) =====
while (queueHead < queueTail)
{
    uint32_t index = cannyEdgeQueue[queueHead++];

    for (uint8_t i = 0; i < 8; i++)
    {
        int32_t trackIndex = (int32_t) index + cannyConfig.trackOffsets[i];

        if (trackIndex >= 0 && trackIndex < (int32_t) cannyConfig.pixelCount)
        {
            if (dst->ptr[trackIndex] == 127)
            {
                dst->ptr[trackIndex] = WHITE; // 连通升级为强边缘

                // 防溢出保护
                if (queueTail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
                {
                    cannyEdgeQueue[queueTail++] = (uint16_t) trackIndex;
                }
            }
        }
    }
}
}

```

```

    }

    // ===== 收尾: 清理死路 =====
    uint32_t totalPixels = gaussConfig.width * gaussConfig.height;
    for (uint32_t i = 0; i < totalPixels; i++)
    {
        if (dst->ptr[i] == 127) dst->ptr[i] = BLACK; // 无法与强边缘连通被消除
    }
}

```

coreT3_cyt4bb7/project/code/om_connected.c

SHA-256

:

d78e1bb765ad301048475dee479f462863547f12126fced35c4a68c83f097631

代码 (c)

```

/*
 * om_connected.c - [3] 连通域检测
 *
 * 四邻域 / 八邻域 Two-Pass 连通域标记
 */

#include "om_internal.h"

// ---- typedef ----

typedef struct
{
    uint8_t background;
    u8Image* data;
    u8Image* show;
    float areaTopMin;
    float areaTopMax;
    float kMin;
    float kMax;
    uint16_t xRange[2];
    uint16_t yRange[2];
} connectedAreaConfigStruct;

// ---- 内部变量 ----

static connectedAreaConfigStruct connectedAreaConfig;
static uint16_t g_labMap[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE];
static uint16_t g_labelEquivalence[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE];
static SBEA g_tempBea[OM_MAX_OBJECTS];

// ---- 外部函数实现 ----

void twoPassConnectedAreaInit(uint8_t background, u8Image* src, u8Image* sho, float areaMinLow, float areaMaxLow, float areaMinHigh, float areaMaxHigh, float leftDeadZone, float rightDeadZone, float topDeadZone, float bottomDeadZone)
{
    connectedAreaConfig.background = background;
    connectedAreaConfig.data = src;
    connectedAreaConfig.show = sho;
    connectedAreaConfig.areaTopMax = areaMaxHigh;
    connectedAreaConfig.areaTopMin = areaMinHigh;
    connectedAreaConfig.xRange[0] = (uint16_t) (leftDeadZone * src->xLen);
    connectedAreaConfig.xRange[1] = (uint16_t) ((1.0f - rightDeadZone) * src->xLen);
    connectedAreaConfig.yRange[0] = (uint16_t) (topDeadZone * src->yLen);
    connectedAreaConfig.yRange[1] = (uint16_t) ((1.0f - bottomDeadZone) * src->yLen);
    connectedAreaConfig.kMin = (areaMinLow - areaMinHigh) / ((float) connectedAreaConfig.yRange[1] - (float) connectedAreaConfig.yRange[0]);
    connectedAreaConfig.kMax = (areaMaxLow - areaMaxHigh) / ((float) connectedAreaConfig.yRange[1] - (float) connectedAreaConfig.yRange[0]);
}

inline void twoPassFourConnectedAreaProcess(BEAINF* obj)
{
    // first pass
    if (connectedAreaConfig.data->size > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE) return;
    uint16_t lab = 0;

    // 清零全局数组
    memset(g_labMap, 0, connectedAreaConfig.data->size * sizeof(uint16_t));
    memset(g_labelEquivalence, 0, UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE * sizeof(uint16_t));
}

```

```

for (uint16_t i = 0; i < connectedAreaConfig.data->size; i++)
{
    if (lab >= UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE) continue;

    if (connectedAreaConfig.data->ptr[i] == connectedAreaConfig.background)
    {
        g_labMap[i] = 0; // 背景像素标记为 0
        continue;
    }

    uint16_t left = (i % connectedAreaConfig.data->xLen == 0) ? 0 : g_labMap[i - 1];
    uint16_t up = (i < connectedAreaConfig.data->xLen) ? 0 : g_labMap[i - connectedAreaConfig.data->xLen];

    if (!left && !up)
    {
        g_labMap[i] = ++lab; // 新标签
        g_labelEquivalence[lab] = lab;
    }
    else if (left && !up) g_labMap[i] = left; // 使用左边标签
    else if (!left && up) g_labMap[i] = up; // 使用上边标签
    else
    {
        // 两个标签合并
        uint16_t minLabel = (left < up) ? left : up;
        uint16_t maxLabel = (left < up) ? up : left;
        g_labMap[i] = minLabel;
        if (minLabel < g_labelEquivalence[maxLabel]) g_labelEquivalence[maxLabel] = minLabel;
    }
}

// second pass
// 清零全局临时 BEA 数组
memset(g_tempBea, 0, 101 * sizeof(SBEA));
uint16_t tempBeaCount = 0;

// 路径压缩优化标签等价表
for (uint16_t i = 1; i <= lab; i++)
{
    uint16_t root = i;
    while (root != g_labelEquivalence[root]) root = g_labelEquivalence[root];
    g_labelEquivalence[i] = root;
}

for (uint16_t i = 0; i < connectedAreaConfig.data->size; i++)
{
    uint16_t curr_label = g_labMap[i];

    if (!curr_label) continue;

    // 使用优化后的等价表直接查找
    curr_label = g_labelEquivalence[curr_label];

    if (curr_label > 100) continue; // 防止数组越界

    tempBeaCount = (curr_label > tempBeaCount) ? curr_label : tempBeaCount;

    g_tempBea[curr_label - 1].beaX += (i % connectedAreaConfig.data->xLen);
    g_tempBea[curr_label - 1].beaY += (i / connectedAreaConfig.data->xLen);
    g_tempBea[curr_label - 1].beaArea++;
}

// filter pass
memcpy(connectedAreaConfig.show->ptr, connectedAreaConfig.data->ptr, connectedAreaConfig.data->size * sizeof(uint8_t));
obj->beaCount = 0; // 重置计数器
obj->selectedIndex = 100; // 100 表示未选中任何区域
float minDistance = -1.0f; // 使用-1 表示未初始化

for (uint16_t i = 0; i < tempBeaCount; i++)
{
    // 跳过空区域
    if (g_tempBea[i].beaArea == 0 || obj->beaCount >= 100) continue;

    // 计算质心
    g_tempBea[i].beaX = (uint32_t)((float) g_tempBea[i].beaX / (float) g_tempBea[i].beaArea);
    g_tempBea[i].beaY = (uint32_t)((float) g_tempBea[i].beaY / (float) g_tempBea[i].beaArea);

    // 检查面积和位置约束
    if (g_tempBea[i].beaX < connectedAreaConfig.xRange[0] || g_tempBea[i].beaX >= connectedAreaConfig.xRange[1] ||
        g_tempBea[i].beaY < connectedAreaConfig.yRange[0] || g_tempBea[i].beaY >= connectedAreaConfig.yRange[1] ||

```

```

        g_tempBea[i].beaArea > (uint16_t) (connectedAreaConfig.areaTopMax + connectedAreaConfig.kMax * (float) (g_tempBea[i].beaY -
connectedAreaConfig.yRange[0])) || g_tempBea[i].beaArea < (uint16_t) (connectedAreaConfig.areaTopMin + connectedAreaConfig.kMin * (float)
(g_tempBea[i].beaY - connectedAreaConfig.yRange[0])))
        continue;

// 将有效信标添加到结果数组
obj->sbea[obj->beaCount].beaX = g_tempBea[i].beaX;
obj->sbea[obj->beaCount].beaY = g_tempBea[i].beaY;
obj->sbea[obj->beaCount].beaArea = g_tempBea[i].beaArea;

// 选择逻辑：分两种情况处理
if (obj->lastFrameBeaExist)
{
    // 上帧有灯：选择距离上帧选中的灯距离最近的信标
    float distance = sqrtf(powf((float) obj->sbea[obj->beaCount].beaX - (float) obj->lastFrameSelectedBeaXY[0], 2) +
        powf((float) obj->sbea[obj->beaCount].beaY - (float) obj->lastFrameSelectedBeaXY[1], 2));

    if (minDistance < 0.0f || distance < minDistance)
    {
        obj->selectedIndex = obj->beaCount;
        minDistance = distance;
    }
}
// 上帧无灯：选择面积最大的信标
else if (obj->selectedIndex == 100 || obj->sbea[obj->beaCount].beaArea > obj->sbea[obj->selectedIndex].beaArea) obj->selectedIndex = obj->beaCount;

// 标记水平线
for (uint16_t x = 0; x < connectedAreaConfig.data->xLen; x++) connectedAreaConfig.show->ptr[obj->sbea[obj->beaCount].beaY *
connectedAreaConfig.data->xLen + x] = cReverse(connectedAreaConfig.background);

// 标记垂直线
for (uint16_t y = 0; y < connectedAreaConfig.data->yLen; y++) connectedAreaConfig.show->ptr[y * connectedAreaConfig.data->xLen + obj->sbea[obj-
>beaCount].beaX] = cReverse(connectedAreaConfig.background);

    obj->beaCount++;
}

if (obj->beaCount)
{
    obj->lastFrameBeaExist = 1; // 标记当前帧有有效区域
    obj->lastFrameSelectedBeaXY[0] = obj->sbea[obj->selectedIndex].beaX;
    obj->lastFrameSelectedBeaXY[1] = obj->sbea[obj->selectedIndex].beaY;
}
else obj->lastFrameBeaExist = 0;
}

inline void twoPassEightConnectedAreaProcess(BEAINF* obj)
{
    // first pass
    if (connectedAreaConfig.data->size > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE) return;
    uint16_t lab = 0;

    // 清零全局数组
    memset(g_labMap, 0, connectedAreaConfig.data->size * sizeof(uint16_t));
    memset(g_labelEquivalence, 0, UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE * sizeof(uint16_t));
    for (uint16_t i = 0; i < connectedAreaConfig.data->size; i++)
    {
        if (lab >= UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE) continue;

        if (connectedAreaConfig.data->ptr[i] == connectedAreaConfig.background)
        {
            g_labMap[i] = 0; // 背景像素标记为 0
            continue;
        }

        // 收集所有标签
        uint16_t labels[4];
        labels[0] = (i % connectedAreaConfig.data->xLen == 0) ? 0 : g_labMap[i - 1];
        labels[1] = (i % connectedAreaConfig.data->xLen == 0 || i < connectedAreaConfig.data->xLen) ? 0 : g_labMap[i - connectedAreaConfig.data->xLen - 1];
        labels[2] = (i < connectedAreaConfig.data->xLen) ? 0 : g_labMap[i - connectedAreaConfig.data->xLen];
        labels[3] = (i % connectedAreaConfig.data->xLen == connectedAreaConfig.data->xLen - 1 || i < connectedAreaConfig.data->xLen) ? 0 : g_labMap[i -
connectedAreaConfig.data->xLen + 1];
        uint16_t validLabels[4] = {0};
        uint8_t validCount = 0;

        for (uint16_t k = 0; k < 4; k++)
            if (labels[k] != 0) validLabels[validCount++] = labels[k];

        if (!validCount)

```



```

{
    g_labMap[i] = ++lab; // 新标签
    g_labelEquivalence[lab] = lab;
}
else if (validCount == 1) g_labMap[i] = validLabels[0];
else
{
    // 找到最小标签
    uint16_t minLabel = validLabels[0];
    for (uint16_t k = 1; k < validCount; k++) minLabel = (validLabels[k] < minLabel) ? validLabels[k] : minLabel;

    // 使用最小标签
    g_labMap[i] = minLabel;

    // 将所有其他标签指向最小标签
    for (uint16_t k = 0; k < validCount; k++)
        if (minLabel < g_labelEquivalence[validLabels[k]]) g_labelEquivalence[validLabels[k]] = minLabel;
}
}

// second pass
// 清零全局临时 BEA 数组 (复用同一个全局数组)
memset(g_tempBea, 0, 101 * sizeof(SBEA));
uint16_t tempBeaCount = 0;

// 路径压缩优化标签等价表
for (uint16_t i = 1; i <= lab; i++)
{
    uint16_t root = i;
    while (root != g_labelEquivalence[root]) root = g_labelEquivalence[root];
    g_labelEquivalence[i] = root;
}

for (uint16_t i = 0; i < connectedAreaConfig.data->size; i++)
{
    uint16_t curr_label = g_labMap[i];

    if (!curr_label) continue;

    // 使用优化后的等价表直接查找
    curr_label = g_labelEquivalence[curr_label];

    if (curr_label > 100) continue; // 防止数组越界

    tempBeaCount = (curr_label > tempBeaCount) ? curr_label : tempBeaCount;

    g_tempBea[curr_label - 1].beaX += (i % connectedAreaConfig.data->xLen);
    g_tempBea[curr_label - 1].beaY += (i / connectedAreaConfig.data->xLen);
    g_tempBea[curr_label - 1].beaArea++;
}

// filter pass
memcpy(connectedAreaConfig.show->ptr, connectedAreaConfig.data->ptr, connectedAreaConfig.data->size * sizeof(uint8_t));
obj->beaCount = 0; // 重置计数器
obj->selectedIndex = 100; // 100 表示未选中任何区域
float minDistance = -1.0f; // 使用-1表示未初始化

for (uint16_t i = 0; i < tempBeaCount; i++)
{
    // 跳过空区域
    if (g_tempBea[i].beaArea == 0 || obj->beaCount >= 100) continue;

    // 计算质心
    g_tempBea[i].beaX = (uint32_t)((float) g_tempBea[i].beaX / (float) g_tempBea[i].beaArea);
    g_tempBea[i].beaY = (uint32_t)((float) g_tempBea[i].beaY / (float) g_tempBea[i].beaArea);

    // 检查面积和位置约束
    if (g_tempBea[i].beaX < connectedAreaConfig.xRange[0] || g_tempBea[i].beaX >= connectedAreaConfig.xRange[1] ||
        g_tempBea[i].beaY < connectedAreaConfig.yRange[0] || g_tempBea[i].beaY >= connectedAreaConfig.yRange[1] ||
        g_tempBea[i].beaArea > (uint16_t)(connectedAreaConfig.areaTopMax + connectedAreaConfig.kMax * (float)(g_tempBea[i].beaY -
            connectedAreaConfig.yRange[0])) || g_tempBea[i].beaArea < (uint16_t)(connectedAreaConfig.areaTopMin + connectedAreaConfig.kMin * (float)
            (g_tempBea[i].beaY - connectedAreaConfig.yRange[0])))
        continue;

    // 将有效信标添加到结果数组
    obj->sbea[obj->beaCount].beaX = g_tempBea[i].beaX;
    obj->sbea[obj->beaCount].beaY = g_tempBea[i].beaY;
    obj->sbea[obj->beaCount].beaArea = g_tempBea[i].beaArea;
}

```

```

// 选择逻辑：分两种情况处理
if (obj->lastFrameBeaExist)
{
    // 上帧有灯：选择距离上帧选中的灯距离最近的信标
    float distance = sqrtf(powf((float) obj->sbea[obj->beaCount].beaX - (float) obj->lastFrameSelectedBeaXY[0], 2) +
        powf((float) obj->sbea[obj->beaCount].beaY - (float) obj->lastFrameSelectedBeaXY[1], 2));

    if (minDistance < 0.0f || distance < minDistance)
    {
        obj->selectedIndex = obj->beaCount;
        minDistance = distance;
    }
}
// 上帧无灯：选择面积最大的信标
else if (obj->selectedIndex == 100 || obj->sbea[obj->beaCount].beaArea > obj->sbea[obj->selectedIndex].beaArea) obj->selectedIndex = obj->beaCount;

// 标记水平线
for (uint16_t x = 0; x < connectedAreaConfig.data->xLen; x++) connectedAreaConfig.show->ptr[obj->sbea[obj->beaCount].beaY *
connectedAreaConfig.data->xLen + x] = cReverse(connectedAreaConfig.background);

// 标记垂直线
for (uint16_t y = 0; y < connectedAreaConfig.data->yLen; y++) connectedAreaConfig.show->ptr[y * connectedAreaConfig.data->xLen + obj->sbea[obj-
>beaCount].beaX] = cReverse(connectedAreaConfig.background);

    obj->beaCount++;
}

if (obj->beaCount)
{
    obj->lastFrameBeaExist = 1; // 标记当前帧有有效区域
    obj->lastFrameSelectedBeaXY[0] = obj->sbea[obj->selectedIndex].beaX;
    obj->lastFrameSelectedBeaXY[1] = obj->sbea[obj->selectedIndex].beaY;
}
else obj->lastFrameBeaExist = 0;
}

```

coreT3_cyt4bb7/project/code/om_contour.c

SHA-256

:

2bd899aa569cba2ff3e1d76bdd91a822d63c7aa8d32b77e6b4d98222f8ffacf2

代码（c）

```

/*
 * om_contour.c - [5] 轮廓提取
 *
 * 8 邻域 BFS 闭合轮廓检测、Hu 矩特征匹配与几何特征
 * 本文件同时对外暴露 om_beacon.c 共用的 BFS 基础设施 (inContourMap 等)
 */

#include "om_internal.h"

// ---- define ----

#define CONTOUR_ECC_MIN    (1.5f)
#define CONTOUR_ECC_MAX    (6.0f) /* 偏心率上限: ^ 笔画粗 ecc<=4, 细长开放曲线 ecc>=20 */
/* CONTOUR_HU_TOPK 已废弃: 现对所有候选全量计算 Hu, 不做 Top-K 预筛 */
#define CONTOUR_USE_FAST_LOG (1u)

// ---- typedef ----

typedef struct
{
    uint16_t sizeMax;
    uint16_t sizeMin;

    uint16_t visionCenterX; // 视觉中心 X
    uint16_t visionCenterY; // 视觉中心 Y

    uint16_t roiRadius;     // ROI 检测半径 (0=全图)
    uint16_t earlyStopRadius; // 质心距视觉中心 <= 此值时立即停止
    uint32_t earlyStopRadiusSq;

    uint8_t centroidFeature; // 0: 质心在内 1: 质心在外
}

```

```

uint8_t huTemplateReady; // Hu 模板是否有效: 1=已设置, 0=未设置

float huTemplate[7]; // 目标形状 Hu 矩模板
float shapeWeight; // 形状匹配权重 (0=纯距离, >0 启用 Hu 矩匹配)
float huLostThreshold; // 目标丢失 Hu 距离阈值 (选中轮廓 Hu 距离>阈值 → targetLost)
uint8_t brightnessMax; // 质心亮度允许上限 (255=禁用, ^笔画位于暗背景上)
} CONTOUR_DATA_CONFIG;

// =====
// 共用 BFS 基础设施 (om_beacon.c 通过 extern 复用)
// =====

uint32_t inContourMap[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE];
uint16_t contourGeneration = 0;
const int8_t contourDx[8] = {0, 1, 1, 1, 0, -1, -1, -1};
const int8_t contourDy[8] = {-1, -1, 0, 1, 1, 1, 0, -1};
uint32_t contourQueue[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE];

// ---- 内部数据 ----

static CONTOUR_DATA_CONFIG contourDataConfig;

// ---- 轮廓选择缓冲区 (内存换性能: 全帧候选对应 Hu 特征) ----
static float contourHuBuf[OM_MAX_OBJECTS]; // 每条轮廓的 Hu 加权匹配距离
static uint32_t contourDistSqBuf[OM_MAX_OBJECTS]; // 每条轮廓质心距视觉中心的距离平方
static float contourMomBuf[OM_MAX_OBJECTS * 7]; // 每条轮廓的中心矩 [mxx,myy,mxy,m30,m03,m21,m12]
static float contourOrientBuf[OM_MAX_OBJECTS]; /* 朝向: 主轴方向 + 三阶矩符号定向 (-PI ~ PI) */
static float contourM3ConfBuf[OM_MAX_OBJECTS]; /* 三阶矩定向置信度 */

/* Hu 选择策略: candidate 与最佳 Hu 距离差 < CONTOUR_HU_TIE_THRESHOLD 时
 * 使用位置距离 tiebreak; 收紧则更依赖 Hu */
#define CONTOUR_HU_TIE_THRESHOLD 1.0f

// ---- 内部函数声明 ----

static inline uint16_t eightNeighborhoodIterativeCall(const u8Image* obj, uint16_t startX, uint16_t startY, uint32_t genTag, CONTOUR_DATA_STRUCT*
contour, float* outMxx, float* outMyy, float* outMxy, float* outM30, float* outM03, float* outM21, float* outM12, float* outOrientation, float*
outM3Confidence);
static void computeHuMoments(float mxx, float myy, float mxy, float m30, float m03, float m21, float m12, uint16_t n, float hu[7]);
static float contourHuDistanceByIndex(const CONTOUR_DETECTOR_STRUCT* obj, uint16_t contourIdx);
static float huMatchDistance(const float huA[7], const float huB[7]);
static inline float fastLogPositive(float x);

/* ---- 跟踪滤波: 复用通用 lowPassFilter 模块, 系数由 OM_*_ALPHA 宏调节 ---- */

// 标量一阶低通 (位置): 每拍刷新 alpha 后喂给通用 IIR
static void contourLowPass(LowPassFilterData* f, float sample, float alpha)
{
    f->alpha = alpha;
    lowPassFilter(f, sample);
}

// 角度域一阶低通 (朝向): 解卷绕喂给通用 IIR, 再卷绕回 [-PI, PI]
static void contourAngleLowPass(LowPassFilterData* f, float sample, float alpha)
{
    f->alpha = alpha;
    if (f->firstFlag == 1u)
    {
        lowPassFilterReset(f, sample);
        return;
    }
    float diff = sample - f->lastOutput;
    while (diff > PI) diff -= TWO_PI;
    while (diff < -PI) diff += TWO_PI;
    lowPassFilter(f, f->lastOutput + diff);
    float out = f->filteredValue;
    while (out > PI) out -= TWO_PI;
    while (out < -PI) out += TWO_PI;
    f->filteredValue = out;
    f->lastOutput = out;
}

// ---- 外部函数实现 ----

void contourDetectInit(uint16_t sizeMin, uint16_t sizeMax, uint8_t centroidFeature, uint16_t visionCenterX, uint16_t visionCenterY, uint16_t roiRadius,
uint16_t earlyStopRadius, uint8_t brightnessMax)
{
    contourDataConfig.sizeMin = sizeMin;
    contourDataConfig.sizeMax = sizeMax;
}

```

```

    contourDataConfig.centroidFeature = centroidFeature;
    contourDataConfig.visionCenterX = visionCenterX;
    contourDataConfig.visionCenterY = visionCenterY;
    contourDataConfig.roiRadius = roiRadius;
    contourDataConfig.earlyStopRadius = earlyStopRadius;
    contourDataConfig.brightnessMax = brightnessMax;
    contourDataConfig.earlyStopRadiusSq = (uint32_t) earlyStopRadius * earlyStopRadius;
    contourDataConfig.huTemplateReady = 0u;
    contourDataConfig.shapeWeight = 0.0f;
    contourDataConfig.huLostThreshold = 0.0f;
}

void contourDetectSetSizeRange(uint16_t sizeMin, uint16_t sizeMax)
{
    contourDataConfig.sizeMin = sizeMin;
    contourDataConfig.sizeMax = sizeMax;
}

void contourDetectSetTemplate(const float* huMoments, float shapeWeight, float huLostThreshold)
{
    if (NULL == huMoments)
    {
        for (uint8_t i = 0; i < 7; i++) contourDataConfig.huTemplate[i] = 0.0f;
        contourDataConfig.huTemplateReady = 0u;
    }
    else
    {
        for (uint8_t i = 0; i < 7; i++) contourDataConfig.huTemplate[i] = huMoments[i];
        contourDataConfig.huTemplateReady = 1u;
    }
    contourDataConfig.shapeWeight = shapeWeight;
    contourDataConfig.huLostThreshold = huLostThreshold;
}

uint16_t contourDetect(const u8Image* src, const u8Image* graySrc, CONTOUR_DETECTOR_STRUCT* obj)
{
    if (src->size > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
    {
        obj->contourCount = 0;
        obj->selectedIndex = 0;
        obj->targetLost = 1;
        obj->orientation = 0.0f;
        return 0;
    }

    // generation 替代 memset: 每帧自增, 匹配当前 generation 才视为已访问
    contourGeneration++;
    if (contourGeneration == 0)
    {
        memset(inContourMap, 0, sizeof(inContourMap));
        contourGeneration = 1;
    }
    uint32_t genBase = (uint32_t) contourGeneration << 16;

    uint16_t contourCount = 0;
    uint32_t tagCounter = 0;
    obj->selectedIndex = 0;
    obj->orientation = 0.0f;
    obj->targetLost = 1;

    /* huEnabled 仅以模板是否就绪为准 (shapeWeight 保留作为变量, 但不再参与打分) */
    uint8_t huEnabled = (contourDataConfig.huTemplateReady) ? 1u : 0u;

    // ROI 边界
    uint32_t roiStartX = 0, roiStartY = 0;
    uint32_t roiEndX = src->xLen - 1, roiEndY = src->yLen - 1;
    if (contourDataConfig.roiRadius > 0)
    {
        int32_t rs = (int32_t) contourDataConfig.visionCenterX - (int32_t) contourDataConfig.roiRadius;
        int32_t re = (int32_t) contourDataConfig.visionCenterX + (int32_t) contourDataConfig.roiRadius;
        int32_t ts = (int32_t) contourDataConfig.visionCenterY - (int32_t) contourDataConfig.roiRadius;
        int32_t te = (int32_t) contourDataConfig.visionCenterY + (int32_t) contourDataConfig.roiRadius;
        roiStartX = (rs < 0) ? 0 : (uint32_t) rs;
        roiStartY = (ts < 0) ? 0 : (uint32_t) ts;
        roiEndX = ((uint32_t) re >= src->xLen) ? src->xLen - 1 : (uint32_t) re;
        roiEndY = ((uint32_t) te >= src->yLen) ? src->yLen - 1 : (uint32_t) te;
    }

    // ===== 第一阶段: 扫描全部轮廓并收集特征 =====

```

```

for (uint32_t y = roiStartY; y <= roiEndY && contourCount < OM_MAX_OBJECTS - 1; y++)
{
    uint32_t rowOff = y * src->xLen;
    for (uint32_t x = roiStartX; x <= roiEndX && contourCount < OM_MAX_OBJECTS - 1; x++)
    {
        uint32_t i = rowOff + x;
        if (src->ptr[i] != WHITE || (inContourMap[i] >> 16) == contourGeneration) continue;

        tagCounter++;
        if (tagCounter > 0xFFFFu) goto phaseTwoSelect;

        uint32_t genTag = genBase | (uint32_t) tagCounter;

        float curMxx = 0.0f, curMyy = 0.0f, curMxy = 0.0f;
        float curM30 = 0.0f, curM03 = 0.0f, curM21 = 0.0f, curM12 = 0.0f;
        uint16_t endpointCount = eightNeighborhoodIterativeCall(src, (uint16_t) x, (uint16_t) y, genTag, &obj->contourData[contourCount], &curMxx,
        &curMyy, &curMxy, &curM30, &curM03, &curM21, &curM12, &contourOrientBuf[contourCount], &contourM3ConfBuf[contourCount]);

        /* 闭合判定收紧: 真闭环=0 端点, ^=2 端点; 其余开放噪声曲线一律弃 */
        if (endpointCount != 0u && endpointCount != 2u) continue;

        /* BFS 内 abandon (触边/超尺寸/超端点) 时 contourSize 已被置 0, 下方 sizeMin 检查会拒绝 */

        // 尺寸过滤
        uint16_t sz = obj->contourData[contourCount].contourSize;
        if (sz < contourDataConfig.sizeMin || sz > contourDataConfig.sizeMax) continue;

        // ^形状几何预筛: 偏心率检查 (^主轴比/次轴比 在[1.5, 6])
        float eigSum = curMxx + curMyy;
        float eigDiff2 = (curMxx - curMyy) * (curMxx - curMyy) + 4.0f * curMxy * curMxy;
        if (eigSum > 0.0f && eigDiff2 > 0.0f)
        {
            // 免 sqrt 偏心率判定:
            // eigMax/eigMin < r <=> diff^2 < sum^2 * ((r-1)/(r+1))^2
            // eigMax/eigMin > r <=> diff^2 > sum^2 * ((r-1)/(r+1))^2
            const float ratioKMin = (CONTOUR_ECC_MIN - 1.0f) / (CONTOUR_ECC_MIN + 1.0f);
            const float ratioKMax = (CONTOUR_ECC_MAX - 1.0f) / (CONTOUR_ECC_MAX + 1.0f);
            float boundMin = eigSum * ratioKMin;
            float boundMax = eigSum * ratioKMax;
            if (eigDiff2 < boundMin * boundMin) continue;
            if (eigDiff2 > boundMax * boundMax) continue;
        }

        // 质心亮度上限过滤: ^笔画必处于暗背景上, 中心过亮必非^
        if (contourDataConfig.brightnessMax < 255)
        {
            uint16_t ccx = obj->contourData[contourCount].centerX;
            uint16_t ccy = obj->contourData[contourCount].centerY;
            uint8_t centroidBright = graySrc->ptr[(uint32_t) ccy * graySrc->xLen + (uint32_t) ccx];
            if (centroidBright > contourDataConfig.brightnessMax) continue;
        }

        // 质心内外特征 (射线法: 由质心向右发射水平射线, 统计与本轮廓边缘的交点奇偶)
        if (contourDataConfig.centroidFeature <= 1)
        {
            uint16_t crossings = 0;
            uint8_t prevEdge = 0;
            for (uint16_t cx = obj->contourData[contourCount].centerX + 1; cx < src->xLen; cx++)
            {
                uint16_t idx = (uint16_t) (obj->contourData[contourCount].centerY * src->xLen + cx);
                uint8_t isEdge = ((inContourMap[idx] & 0xFFFFu) == tagCounter && (inContourMap[idx] >> 16) == contourGeneration) ? 1u : 0u;
                if (isEdge && !prevEdge) crossings++;
                prevEdge = isEdge;
            }
            uint8_t centroidInside = crossings & 1u;
            if (contourDataConfig.centroidFeature == 0 && !centroidInside) continue;
            if (contourDataConfig.centroidFeature == 1 && centroidInside) continue;
        }

        // 存储距离平方
        int32_t dx = (int32_t) obj->contourData[contourCount].centerX - (int32_t) contourDataConfig.visionCenterX;
        int32_t dy = (int32_t) obj->contourData[contourCount].centerY - (int32_t) contourDataConfig.visionCenterY;
        contourDistSqBuf[contourCount] = (uint32_t) (dx * dx + dy * dy);

        // 存储中心矩 (第二阶段成败选择需要)
        uint32_t mBase = (uint32_t) contourCount * 7u;
        contourMomBuf[mBase] = curMxx;
        contourMomBuf[mBase + 1] = curMyy;
        contourMomBuf[mBase + 2] = curMxy;
    }
}

```

```

        contourMomBuf[mBase + 3] = curM30;
        contourMomBuf[mBase + 4] = curM03;
        contourMomBuf[mBase + 5] = curM21;
        contourMomBuf[mBase + 6] = curM12;

        // Hu 距离在第二阶段进入 Top-K 候选后才算, 第一阶段仅收集几何特征
        obj->contourData[contourCount].huDistance = 0.0f;

        contourCount++;

        // 非 Hu 模式下 earlyStop (Hu 模式需收集全部轮廓, 不能早停)
        if (!huEnabled && contourDataConfig.earlyStopRadius > 0 && contourDistSqBuf[contourCount - 1] <= contourDataConfig.earlyStopRadiusSq)
            goto phaseTwoSelect;
    }
}

phaseTwoSelect:
    obj->contourCount = contourCount;

    // ===== 第二阶段: 在已收集轮廓中选出最佳轮廓 =====
    if (contourCount > 0)
    {
        if (huEnabled && contourCount > 1)
        {
            // Hu-first: 对所有候选计算 Hu 距离, 不做 Top-K 预筛
            float huMin = 3.402823 466e+38f;
            for (uint16_t k = 0; k < contourCount; k++)
            {
                float dHu = contourHuDistanceByIndex(obj, k);
                contourHuBuf[k] = dHu;
                obj->contourData[k].huDistance = dHu;
                if (dHu < huMin) huMin = dHu;
            }

            /* Hu 优先选择: 最小 Hu; 同 Hu 带内 (差 < TIE_THRESHOLD) 按位置 tiebreak */
            uint32_t bestTieDist = 0xFFFFFFFFu;
            for (uint16_t k = 0; k < contourCount; k++)
            {
                if ((contourHuBuf[k] - huMin) < CONTOUR_HU_TIE_THRESHOLD)
                {
                    if (contourDistSqBuf[k] < bestTieDist)
                    {
                        bestTieDist = contourDistSqBuf[k];
                        obj->selectedIndex = (uint8_t) k;
                    }
                }
            }
        }
        else if (!huEnabled)
        {
            // 纯距离选择
            uint32_t bestDist = contourDistSqBuf[0];
            obj->selectedIndex = 0;
            for (uint16_t k = 1; k < contourCount; k++)
            {
                if (contourDistSqBuf[k] < bestDist)
                {
                    bestDist = contourDistSqBuf[k];
                    obj->selectedIndex = (uint8_t) k;
                }
            }
        }
        // contourCount==1 时 selectedIndex 已为 0

        // 目标丢失判定: Hu 距离超阈值 → 形状不匹配模板
        if (huEnabled && contourDataConfig.huLostThreshold > 0.0f)
        {
            float selHu = contourHuDistanceByIndex(obj, obj->selectedIndex);
            contourHuBuf[obj->selectedIndex] = selHu;
            obj->contourData[obj->selectedIndex].huDistance = selHu;
            obj->targetLost = (selHu > contourDataConfig.huLostThreshold) ? 1u : 0u;
        }
        else
        {
            obj->targetLost = 0;
        }

        /* 选中轮廓朝向: BFS 末尾用三阶矩符号定向, 无跨帧状态 */
        obj->orientation = contourOrientBuf[obj->selectedIndex];
    }

```

```

}

// 清空后段未使用位置
for (uint16_t i = contourCount; i < OM_MAX_OBJECTS; i++) obj->contourData[i].contourSize = 0;

// ===== 跟踪状态机 =====
{
    uint8_t detected = (obj->targetLost == 0u && contourCount > 0u) ? 1u : 0u;
    uint16_t selCx = obj->contourData[obj->selectedIndex].centerX;
    uint16_t selCy = obj->contourData[obj->selectedIndex].centerY;
    float selOrient = contourOrientBuf[obj->selectedIndex];
    float selM3Conf = contourM3ConfBuf[obj->selectedIndex];
    uint16_t selSize = obj->contourData[obj->selectedIndex].contourSize;

    // 画幅边缘接触检测: 质心过于靠近边缘视为丢失
    if (detected)
    {
        if (selCx <= OM_BORDER_MARGIN || selCx >= (src->xLen - 1u - OM_BORDER_MARGIN) ||
            selCy <= OM_BORDER_MARGIN || selCy >= (src->yLen - 1u - OM_BORDER_MARGIN))
        {
            detected = 0u;
            obj->targetLost = 1u;
        }
    }

    // 写入当前帧 m3Confidence
    obj->m3Confidence = detected ? selM3Conf : 0.0f;

    if (detected)
    {
        obj->lostFrameCount = 0u;
        obj->lockFrameCount = (obj->lockFrameCount < 255u) ? obj->lockFrameCount + 1u : 255u;

        switch (obj->trackingState)
        {
            case OM_TRACK_SEARCHING:
                if (obj->lockFrameCount >= OM_LOCK_THRESHOLD)
                    obj->trackingState = OM_TRACK_ACQUIRING;
                break;
            case OM_TRACK_ACQUIRING:
                if (obj->lockFrameCount >= (OM_LOCK_THRESHOLD + OM_LOCK_CONFIRM))
                    obj->trackingState = OM_TRACK_LOCKED;
                break;
            case OM_TRACK_LOCKED:
                break;
            case OM_TRACK_LOST_HOLDING:
                obj->trackingState = OM_TRACK_LOCKED;
                break;
            default:
                obj->trackingState = OM_TRACK_SEARCHING;
                break;
        }

        // 朝向鲁棒性: 主轴方向(PCA, 二阶矩决定)始终实时跟随; 三阶矩符号弱(尖端
        // 不可信)时仅把尖端对齐上一帧消除 180 度乱跳, 不冻结主轴角。
        // 旧实现低置信度时整体沿用 lastValidOrientation, 而实测 m3Conf<=0.001 恒
        // 低于阈值 -> 每帧冻结 + 回写自锁, 导致旋转时朝向锁死在首帧。
        float finalOrient = selOrient;
        if (obj->trackingState >= OM_TRACK_ACQUIRING && obj->lastValidSize > 0u)
        {
            if (selSize < OM_ORIENTATION_MIN_SIZE || selM3Conf < OM_M3_CONFIDENCE_MIN)
            {
                float diff = selOrient - obj->lastValidOrientation;
                while (diff > PI) diff -= TWO_PI;
                while (diff < -PI) diff += TWO_PI;
                // selOrient 与上一帧反向(尖端选反) -> 翻转 180 度对齐, 主轴角仍随实时
                if (diff > HALF_PI)
                    finalOrient = selOrient - PI;
                else if (diff < -HALF_PI)
                    finalOrient = selOrient + PI;
                while (finalOrient > PI) finalOrient -= TWO_PI;
                while (finalOrient < -PI) finalOrient += TWO_PI;
            }
            // selM3Conf 足够: 信任 selOrient (含实时尖端定向), 不干预
        }

        // 更新 lastValid
        obj->lastValidCenterX = selCx;
        obj->lastValidCenterY = selCy;
    }
}

```

```

obj->lastValidOrientation = finalOrient;
obj->lastValidHuDistance = obj->contourData[obj->selectedIndex].huDistance;
obj->lastValidSize = selSize;

// 低通滤波 (仅 LOCKED 态), 系数由 OM_*_ALPHA 宏调节
if (obj->trackingState == OM_TRACK_LOCKED)
{
    contourLowPass(&obj->centerXFilter, (float)selCx, OM_POSITION_ALPHA);
    contourLowPass(&obj->centerYFilter, (float)selCy, OM_POSITION_ALPHA);
    contourAngleLowPass(&obj->orientFilter, finalOrient, OM_ORIENTATION_ALPHA);
}
else
{
    // 非 LOCKED 态直接跟随, 锚定滤波器状态防止入锁瞬间突跳
    lowPassFilterReset(&obj->centerXFilter, (float)selCx);
    lowPassFilterReset(&obj->centerYFilter, (float)selCy);
    lowPassFilterReset(&obj->orientFilter, finalOrient);
}

obj->orientation = obj->orientFilter.filteredValue;
}
else
{
    // 目标丢失
    obj->lockFrameCount = 0u;
    obj->lostFrameCount = (obj->lostFrameCount < 255u) ? obj->lostFrameCount + 1u : 255u;

    switch (obj->trackingState)
    {
        case OM_TRACK_SEARCHING:
            break;
        case OM_TRACK_ACQUIRING:
            obj->trackingState = OM_TRACK_SEARCHING;
            break;
        case OM_TRACK_LOCKED:
            if (obj->lostFrameCount >= OM_LOST_THRESHOLD)
                obj->trackingState = OM_TRACK_LOST_HOLDING;
            break;
        case OM_TRACK_LOST_HOLDING:
            if (obj->lostFrameCount >= OM_LOST_TIMEOUT)
                obj->trackingState = OM_TRACK_SEARCHING;
            break;
        default:
            obj->trackingState = OM_TRACK_SEARCHING;
            break;
    }

    // 保持 lastValid 不覆盖, 输出滤波值不变
    if (obj->lastValidSize > 0u)
        obj->orientation = obj->orientFilter.filteredValue;
    else
        obj->orientation = 0.0f;
}
}

return contourCount;
}

// ---- 提取 Hu 实现 ----

uint8_t contourExtractHuMoments(const CONTOUR_DETECTOR_STRUCT* obj, uint8_t contourIndex, float huOut[7])
{
    if (obj == NULL || huOut == NULL) return 0u;
    if (contourIndex >= obj->contourCount || obj->contourData[contourIndex].contourSize == 0) return 0u;
    uint32_t mBase = (uint32_t) contourIndex * 7u;
    computeHuMoments(contourMomBuf[mBase], contourMomBuf[mBase + 1], contourMomBuf[mBase + 2],
        contourMomBuf[mBase + 3], contourMomBuf[mBase + 4], contourMomBuf[mBase + 5], contourMomBuf[mBase + 6],
        obj->contourData[contourIndex].contourSize, huOut);
    return 1u;
}

// ---- 内部函数实现 ----

// 8 邻域 BFS: 统计连通区域大小、质心、二阶矩、三阶矩及端点数; 返回端点数
static inline uint16_t eightNeighborhoodIterativeCall(const u8Image* obj, uint16_t startX, uint16_t startY, uint32_t genTag, CONTOUR_DATA_STRUCT*
    contour, float* outMxx, float* outMyy, float* outMxy, float* outM30, float* outM03, float* outM21, float* outM12, float* outOrientation, float*
    outM3Confidence)
{
    uint32_t width = obj->xLen;

```



```

uint32_t height = obj->yLen;

contour->contourSize = 0;
contour->centerX = 0;
contour->centerY = 0;

uint16_t head = 0;
uint16_t tail = 0;

// 起点入队并标记 (packed: high16=y, low16=x)
inContourMap[startY * width + startX] = genTag;
contourQueue[tail++] = ((uint32_t) startY << 16) | (uint32_t) startX;

uint32_t sumX = 0;
uint32_t sumY = 0;
uint32_t sumXX = 0;
uint32_t sumYY = 0;
uint32_t sumXY = 0;
uint64_t sumXXX = 0;
uint64_t sumYYY = 0;
uint64_t sumXXY = 0;
uint64_t sumXYY = 0;
uint16_t totalCount = 0;
uint16_t endpointCount = 0;
uint8_t abandon = 0;

while (head < tail)
{
    uint16_t cx = (uint16_t) (contourQueue[head] & 0xFFFFu);
    uint16_t cy = (uint16_t) (contourQueue[head] >> 16);
    head++;

    totalCount++;

    if (!abandon)
    {
        sumX += cx;
        sumY += cy;
        uint32_t cxx = (uint32_t) cx * cx;
        uint32_t cyy = (uint32_t) cy * cy;
        sumXX += cxx;
        sumYY += cyy;
        sumXY += (uint32_t) cx * cy;
        sumXXX += (uint64_t) cxx * cx;
        sumYYY += (uint64_t) cyy * cy;
        sumXXY += (uint64_t) cxx * cy;
        sumXYY += (uint64_t) cx * cyy;
    }

    // 统计原图中当前像素的白色 s 邻域数量 (用于端点检测)
    uint8_t whiteNeighborCount = 0;

    for (uint8_t i = 0; i < 8; i++)
    {
        int32_t nx = (int32_t) cx + contourDx[i];
        int32_t ny = (int32_t) cy + contourDy[i];

        if (nx < 0 || (uint32_t) nx >= width || ny < 0 || (uint32_t) ny >= height) continue;

        uint16_t nIndex = (uint16_t) ((uint32_t) ny * width + (uint32_t) nx);

        if (obj->ptr[nIndex] == WHITE)
        {
            whiteNeighborCount++;

            // 未访问的白色像素入队
            if ((inContourMap[nIndex] >> 16) != contourGeneration)
            {
                inContourMap[nIndex] = genTag;
                if (tail < UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE)
                    contourQueue[tail++] = ((uint32_t) ny << 16) | (uint32_t) nx;
            }
        }
    }
}

// Canny 后边缘上端点像素白色邻居数 < 2
if (!abandon && whiteNeighborCount < 2) endpointCount++;

/* 触图像边界即 abandon: 边缘截断的开放曲线刚好有 2 个端点会冒充闭合 ^ */

```

```

        if (!abandon && (cx == 0u || cy == 0u || (uint32_t) cx == width - 1u || (uint32_t) cy == height - 1u)) abandon = 1;

        if (!abandon && (totalCount > contourDataConfig.sizeMax || endpointCount > 2)) abandon = 1;
    }

    /* abandon 时 contourSize 置 0, 让 caller 的 sizeMin 检查直接拒绝该轮廓 */
    contour->contourSize = abandon ? 0u : totalCount;

    if (totalCount > 0 && !abandon)
    {
        float fN = (float) totalCount;
        float meanX = (float) sumX / fN;
        float meanY = (float) sumY / fN;
        contour->centerX = (uint16_t) (sumX / totalCount);
        contour->centerY = (uint16_t) (sumY / totalCount);

        *outMxx = (float) sumXX / fN - meanX * meanX;
        *outMyy = (float) sumYY / fN - meanY * meanY;
        *outMxy = (float) sumXY / fN - meanX * meanY;

        // 计算中心矩
        *outM30 = (float) sumXXX / fN - 3.0f * meanX * (float) sumXX / fN + 2.0f * meanX * meanX * meanX;
        *outM03 = (float) sumYYY / fN - 3.0f * meanY * (float) sumYY / fN + 2.0f * meanY * meanY * meanY;
        *outM21 = (float) sumXXY / fN - 2.0f * meanX * (float) sumXY / fN - meanY * (float) sumXX / fN + 2.0f * meanX * meanX * meanY;
        *outM12 = (float) sumXY / fN - 2.0f * meanY * (float) sumXY / fN - meanX * (float) sumYY / fN + 2.0f * meanX * meanY * meanY;

        /* ^ 尖端朝向: 主轴方向 + 三阶矩符号定向
        * theta = 0.5*atan2(2*mxy, mxx-myy) 是次轴; 主轴 axis = theta + PI/2
        * 沿主轴投影的三阶矩 m3Axis 符号指向长尾 (^ 尖端)
        * m3Axis = c^3*m30 + 3*c^2*s*m21 + 3*c*s^2*m12 + s^3*m03
        */
        float orientTheta = 0.5f * atan2f(2.0f * (*outMxy), (*outMxx) - (*outMyy));
        float orientAxis = orientTheta + HALF_PI;
        float c = cosf(orientAxis);
        float s = sinf(orientAxis);
        float c2 = c * c;
        float s2 = s * s;
        float m3Axis = c2 * c * (*outM30) + 3.0f * c2 * s * (*outM21) + 3.0f * c * s2 * (*outM12) + s2 * s * (*outM03);
        float eigSumLocal = (*outMxx) + (*outMyy);
        *outM3Confidence = (eigSumLocal > 1e-6f) ? (m3Axis * m3Axis) / (eigSumLocal * eigSumLocal * eigSumLocal + 1e-6f) : 0.0f;
        if (*outM3Confidence < 0.0f) *outM3Confidence = -*outM3Confidence;
        float result = (m3Axis > 0.0f) ? orientAxis : (orientAxis + PI);
        while (result > PI) result -= TWO_PI;
        while (result < -PI) result += TWO_PI;
        *outOrientation = result;
    }
    else
    {
        *outMxx = 0.0f;
        *outMyy = 0.0f;
        *outMxy = 0.0f;
        *outM30 = 0.0f;
        *outM03 = 0.0f;
        *outM21 = 0.0f;
        *outM12 = 0.0f;
        *outOrientation = 0.0f;
        *outM3Confidence = 0.0f;
    }

    return endpointCount;
}

// 计算 7 个 Hu 不变矩 (输入为中心矩, n 为像素数)
static void computeHuMoments(float mxx, float myy, float mxy, float m30, float m03, float m21, float m12, uint16_t n, float hu[7])
{
    float fN = (float) n;
    float sqrtN = sqrtf(fN);
    float invN = 1.0f / fN;
    float invN15 = invN / sqrtN;

    // 归一化中心矩
    float eta20 = mxx * invN;
    float eta02 = myy * invN;
    float eta11 = mxy * invN;
    float eta30 = m30 * invN15;
    float eta03 = m03 * invN15;
    float eta21 = m21 * invN15;
    float eta12 = m12 * invN15;

```

```

// 7 个 Hu 不变矩
hu[0] = eta20 + eta02;
hu[1] = (eta20 - eta02) * (eta20 - eta02) + 4.0f * eta11 * eta11;

float t1 = eta30 - 3.0f * eta12;
float t2 = 3.0f * eta21 - eta03;
hu[2] = t1 * t1 + t2 * t2;

float t3 = eta30 + eta12;
float t4 = eta21 + eta03;
hu[3] = t3 * t3 + t4 * t4;

float t3sq = t3 * t3;
float t4sq = t4 * t4;
hu[4] = t1 * t3 * (t3sq - 3.0f * t4sq) + t2 * t4 * (3.0f * t3sq - t4sq);
hu[5] = (eta20 - eta02) * (t3sq - t4sq) + 4.0f * eta11 * t3 * t4;
hu[6] = t2 * t3 * (t3sq - 3.0f * t4sq) - t1 * t4 * (3.0f * t3sq - t4sq);
}

static float contourHuDistanceByIndex(const CONTOUR_DETECTOR_STRUCT* obj, uint16_t contourIdx)
{
    float hu[7];
    uint32_t mBase = (uint32_t) contourIdx * 7u;
    computeHuMoments(contourMomBuf[mBase], contourMomBuf[mBase + 1], contourMomBuf[mBase + 2],
        contourMomBuf[mBase + 3], contourMomBuf[mBase + 4], contourMomBuf[mBase + 5], contourMomBuf[mBase + 6],
        obj->contourData[contourIdx].contourSize, hu);
    return huMatchDistance(hu, contourDataConfig.huTemplate);
}

static inline float fastLogPositive(float x)
{
    #if (CONTOUR_USE_FAST_LOG)
        union
        {
            float f;
            uint32_t u;
        } v;
        v.f = x;

        int32_t exp2 = (int32_t) ((v.u >> 23) & 0xFFu) - 127;
        uint32_t mant = (v.u & 0x7FFFFFFu) | 0x3F800000u; // 1.m
        union
        {
            float f;
            uint32_t u;
        } vm;
        vm.u = mant;
        float m = vm.f;

        float y = m - 1.0f; // y in [0,1)
        // ln(1+y) 三阶截断
        float ln_m = y - 0.5f * y * y + (1.0f / 3.0f) * y * y * y;
        return ln_m + (float) exp2 * 0.69314718056f;
    #else
        return logf(x);
    #endif
}

// Hu 矩加权匹配距离 (log 空间加权 L1, 低阶高权重, h6 符号无意义)
static float huMatchDistance(const float huA[7], const float huB[7])
{
    float a0 = fabsf(huA[0]);
    if (a0 < 1e-12f) a0 = 1e-12f;
    float b0 = fabsf(huB[0]);
    if (b0 < 1e-12f) b0 = 1e-12f;
    float a1 = fabsf(huA[1]);
    if (a1 < 1e-12f) a1 = 1e-12f;
    float b1 = fabsf(huB[1]);
    if (b1 < 1e-12f) b1 = 1e-12f;
    float a2 = fabsf(huA[2]);
    if (a2 < 1e-12f) a2 = 1e-12f;
    float b2 = fabsf(huB[2]);
    if (b2 < 1e-12f) b2 = 1e-12f;
    float a3 = fabsf(huA[3]);
    if (a3 < 1e-12f) a3 = 1e-12f;
    float b3 = fabsf(huB[3]);
    if (b3 < 1e-12f) b3 = 1e-12f;
    float a4 = fabsf(huA[4]);
    if (a4 < 1e-12f) a4 = 1e-12f;

```

```

float b4 = fabsf(huB[4]);
if (b4 < 1e-12f) b4 = 1e-12f;
float a5 = fabsf(huA[5]);
if (a5 < 1e-12f) a5 = 1e-12f;
float b5 = fabsf(huB[5]);
if (b5 < 1e-12f) b5 = 1e-12f;

float d0 = fastLogPositive(a0) - fastLogPositive(b0);
float d1 = fastLogPositive(a1) - fastLogPositive(b1);
float d2 = fastLogPositive(a2) - fastLogPositive(b2);
float d3 = fastLogPositive(a3) - fastLogPositive(b3);
float d4 = fastLogPositive(a4) - fastLogPositive(b4);
float d5 = fastLogPositive(a5) - fastLogPositive(b5);

float dist = 2.0f * fabsf(d0) + 2.0f * fabsf(d1) + 1.5f * fabsf(d2) + 1.5f * fabsf(d3) + 0.5f * fabsf(d4) + 0.5f * fabsf(d5);
return dist;
}

```

coreT3_cyt4bb7/project/code/om_internal.h

SHA-256

:

e2e73c91a1fef2535f5ffb038efcee4f2566a907cd4fafd11515113ceec46fdf

代码 (c)

```

/*
 * om_internal.h - openMorph 内部共享头文件 (不对外暴露)
 *
 * 提供各子模块共用的宏定义、标准库包含及共享 BFS 基础设施声明。
 * 外部用户代码只需包含 openMorph.h。
 */
#ifndef OM_INTERNAL_H
#define OM_INTERNAL_H

// =====
// 公共接口 + 基础设施
// =====

#include "openMorph.h" // 公共类型定义与 API 声明
#include "openMorph_config.h" // 可移植尺寸/数量配置 (移植时只改此文件)
#include "commonDef.h" // u8Image, BLACK, WHITE, cReverse, range 等

#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

// =====
// 共享宏定义
// =====

#define UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE (OM_BUFFER_SIZE)

// =====
// 共享 BFS 基础设施 (om_contour.c 定义, om_beacon.c 使用)
// =====

extern uint32_t inContourMap[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE];
extern uint16_t contourGeneration;
extern const int8_t contourDx[8];
extern const int8_t contourDy[8];
extern uint32_t contourQueue[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE];

#endif // OM_INTERNAL_H

```

coreT3_cyt4bb7/project/code/om_morph.c

SHA-256

:

63f29c08c6b1d43276c861a0c20801a15a6c4dd174f7dc395c9fc1011ed665f4

代码 (c)

```
/*
 * om_morph.c - [1] 形态学运算
 *
 * 腐蚀 / 膨胀 / 开运算 / 闭运算
 */

#include "om_internal.h"

// ---- typedef ----

typedef struct
{
    uint8_t background;
} morphConfigStruct;

// ---- 内部变量 ----

static morphConfigStruct morphConfig;
static uint8_t g_morphBuffer[UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE];

// ---- 外部函数实现 ----

void morphInit(uint8_t background)
{
    morphConfig.background = background;
}

inline void morphErode(const u8Image* src, const u8Image* kernel, u8Image* dst)
{
    for (uint16_t i = 0; i < src->size; i++)
    {
        uint16_t y = i / src->xLen;
        uint16_t x = i % src->xLen;

        uint8_t result = cReverse(morphConfig.background);

        // 遍历内核，但减少不必要的计算
        for (uint16_t j = 0; j < kernel->size && result == cReverse(morphConfig.background); j++)
        {
            if (kernel->ptr[j] == cReverse(morphConfig.background))
            {
                uint16_t kx = j % kernel->xLen;
                uint16_t ky = j / kernel->xLen;

                int32_t px = (int32_t) x + (int32_t) kx - (int32_t) (kernel->xLen >> 1);
                int32_t py = (int32_t) y + (int32_t) ky - (int32_t) (kernel->yLen >> 1);

                result = (px < 0 || px >= src->xLen || py < 0 || py >= src->yLen) ? morphConfig.background : (src->ptr[py * src->xLen + px] == morphConfig.background ? morphConfig.background : result);
            }
        }

        dst->ptr[i] = result;
    }
}

inline void morphDilate(const u8Image* src, const u8Image* kernel, u8Image* dst)
{
    for (uint16_t i = 0; i < src->size; i++)
    {
        uint16_t y = i / src->xLen;
        uint16_t x = i % src->xLen;

        uint8_t result = morphConfig.background;

        // 遍历内核，但减少不必要的计算
        for (uint16_t j = 0; j < kernel->size && result == morphConfig.background; j++)
        {
            if (kernel->ptr[j] == cReverse(morphConfig.background))
            {
                uint16_t kx = j % kernel->xLen;
                uint16_t ky = j / kernel->xLen;

                int32_t px = (int32_t) x + (int32_t) kx - (int32_t) (kernel->xLen >> 1);
                int32_t py = (int32_t) y + (int32_t) ky - (int32_t) (kernel->yLen >> 1);
```

```

        result = (px < 0 || px >= src->xLen || py < 0 || py >= src->yLen) ? morphConfig.background : (src->ptr[py * src->xLen + px] ==
morphConfig.background ? result : cReverse(morphConfig.background));
    }
}

dst->ptr[i] = result;
}
}

inline void morphOpen(const u8Image* src, const u8Image* kernel, u8Image* dst)
{
    if (src->xLen * src->yLen > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE) return;

    // 使用全局缓冲区，避免栈分配
    memset(g_morphBuffer, 0, src->xLen * src->yLen * sizeof(uint8_t));
    u8Image tempImg = {g_morphBuffer, src->xLen, src->yLen, src->xLen * src->yLen};

    morphErode(src, kernel, &tempImg);
    morphDilate(&tempImg, kernel, dst);
}

inline void morphClose(const u8Image* src, const u8Image* kernel, u8Image* dst)
{
    if (src->xLen * src->yLen > UNIFORMLY_PRE_ALLOCATED_BUFFER_SIZE) return;

    // 使用全局缓冲区，避免栈分配
    memset(g_morphBuffer, 0, src->xLen * src->yLen * sizeof(uint8_t));
    u8Image tempImg = {g_morphBuffer, src->xLen, src->yLen, src->xLen * src->yLen};

    morphDilate(src, kernel, &tempImg);
    morphErode(&tempImg, kernel, dst);
}

```

coreT3_cyt4bb7/project/code/openMorph.h

SHA-256

:

fc6133f6488cd166830479405fd181bf518681bb192ec30a7115a2729261e748

代码 (c)

```

/*
 * openMorph.h - 图像处理 头文件
 *
 * Created: 20 250 804
 * Author: Hecate
 *
 * 模块组成:
 * [1] 形态学运算    --- 腐蚀 / 膨胀 / 开运算 / 闭运算
 * [2] 二值化      --- 单阈值 / 双阈值 / RGB565
 * [3] 连通域处理   --- 四连通 / 八连通 Two-Pass
 * [4] 高斯模糊与 Canny --- 3x3 定点高斯核 / 逐行流水线边缘检测
 * [5] 轮廓提取     --- 8 邻域 BFS 闭合轮廓检测与几何特征
 * [6] 信标检测     --- Canny+BFS 的目标点亮信标检测
 */
#ifndef OPENMORPH_H
#define OPENMORPH_H

// =====
// 包含文件
// =====

#include <stdint.h>

#include "commonDef.h"
#include "openMorph_config.h"
#include "lowPassFilter.h"

// =====
// [1] 形态学运算
// =====

// ---- 函数声明 ----

void morphInit(uint8_t background);

```

```

void morphErode(const u8Image* src, const u8Image* kernel, u8Image* dst);
void morphDilate(const u8Image* src, const u8Image* kernel, u8Image* dst);
void morphOpen(const u8Image* src, const u8Image* kernel, u8Image* dst);
void morphClose(const u8Image* src, const u8Image* kernel, u8Image* dst);

// =====
// [2] 二值化
// =====

// ---- 函数声明 ----

void toBinaryInit(uint8_t threshold);
void toBinaryDoubleThresholdInit(uint8_t thresholdUp, uint8_t thresholdDown);
void rgb565toBinaryInit(uint8_t background, uint8_t r_min, uint8_t r_max, uint8_t g_min, uint8_t g_max, uint8_t b_min, uint8_t b_max);

void toBinary(const u8Image* src, u8Image* dst);
void toBinaryDoubleThreshold(const u8Image* src, u8Image* dst);
void rgb565toBinary(const u8Image* src, u8Image* dst);

// =====
// [3] 连通域处理
// =====

// ---- typedef ----

// 单个信标区域数据
typedef struct
{
    uint32_t beaX;    // 质心 X (累加和平均)
    uint32_t beaY;    // 质心 Y (累加和平均)
    uint16_t beaArea; // 区域像素数
} SBEA;

// 连通域检测器数据
typedef struct
{
    uint16_t beaCount;    // 有效区域数量
    uint8_t lastFrameBeaExist;    // 上帧是否存在
    uint32_t lastFrameSelectedBeaXY[2]; // 上帧选中区域质心
    uint16_t selectedIndex;    // 当前选中区域索引
    SBEA sbea[OM_MAX_OBJECTS]; // 区域缓存
} BEAINF;

// ---- 函数声明 ----

void twoPassConnectedAreaInit(uint8_t background, u8Image* src, u8Image* sho, float areaMinLow, float areaMaxLow, float areaMinHigh, float areaMaxHigh, float leftDeadZone, float rightDeadZone, float topDeadZone, float bottomDeadZone);

// 处理四连通区域
void twoPassFourConnectedAreaProcess(BEAINF* obj);

// 处理八连通区域
void twoPassEightConnectedAreaProcess(BEAINF* obj);

// =====
// [4] 高斯模糊与 Canny 边缘检测
// =====

// ---- 函数声明 ----

void gaussianInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY);
void gaussianBlur(const u8Image* src, u8Image* dst);
void cannyInit(const u8Image* src, float sigma, uint32_t startX, uint32_t startY, uint32_t endX, uint32_t endY, uint16_t thresholdLow, uint16_t thresholdHigh);
void cannySetThreshold(uint16_t thresholdLow, uint16_t thresholdHigh);
void cannyEdgeDetection(const u8Image* src, u8Image* dstGaussBlur, u8Image* dst);

// =====
// [5] 轮廓提取
// =====

// ---- typedef ----

// 单个轮廓数据
typedef struct
{
    uint16_t centerX;    // 质心 X
    uint16_t centerY;    // 质心 Y
    uint16_t contourSize; // 全部像素数

```

```

    float huDistance;    /* Hu 距离: 0 = 未参与 Hu (未进 Top-K); >0 = 实际匹配距离 */
} CONTOUR_DATA_STRUCT;

// 跟踪状态枚举
#define OM_TRACK_SEARCHING   (0u)
#define OM_TRACK_ACQUIRING   (1u)
#define OM_TRACK_LOCKED      (2u)
#define OM_TRACK_LOST_HOLDING (3u)

// 轮廓检测器数据
typedef struct
{
    CONTOUR_DATA_STRUCT contourData[OM_MAX_OBJECTS]; // 轮廓数据缓存

    uint8_t selectedIndex; // 当前选中轮廓索引
    uint8_t_t contourCount; // 有效闭合轮廓数量
    uint8_t_t targetLost;    // 1=目标丢失(无候选或 Hu 距离超阈值), 0=锁定中
    float orientation;    // 选中轮廓的几何朝向 (弧度, -PI ~ PI)

    /* 跨帧状态保持 */
    uint16_t lastValidCenterX;
    uint16_t lastValidCenterY;
    float lastValidOrientation;
    float lastValidHuDistance;
    uint16_t lastValidSize;
    uint8_t_t lostFrameCount;
    uint8_t_t lockFrameCount;
    uint8_t_t trackingState;
    float m3Confidence;    // 三阶矩定向置信度 (用于朝向鲁棒性)
    LowPassFilterData orientFilter; // 朝向低通 (角度域, 系数 OM_ORIENTATION_ALPHA)
    LowPassFilterData centerXFilter; // 位置 X 低通 (系数 OM_POSITION_ALPHA)
    LowPassFilterData centerYFilter; // 位置 Y 低通 (系数 OM_POSITION_ALPHA)
} CONTOUR_DETECTOR_STRUCT;

// ---- 函数声明 ----

void contourDetectInit(uint16_t sizeMin, uint16_t sizeMax, uint8_t centroidFeature, uint16_t visionCenterX, uint16_t visionCenterY, uint16_t roiRadius,
    uint16_t earlyStopRadius, uint8_t brightnessMax);

// 运行时更新尺寸过滤范围 (标定后收紧)
void contourDetectSetSizeRange(uint16_t sizeMin, uint16_t sizeMax);

// 设置目标形状 Hu 矩模板
// shapeWeight: 0=纯距离选择, >0=Hu 距离+距离微调权重, 建议 0.15
// huLostThreshold: 选中轮廓 Hu 距离超阈值则置 targetLost=1, 建议 20.0
// huMoments: 可传 NULL 以禁用 Hu 模板(此时退化为纯距离逻辑)
void contourDetectSetTemplate(const float* huMoments, float shapeWeight, float huLostThreshold);

// 检测: src=Canny 边缘图, graySrc=灰度原图 (用于亮度剔除), 返回检测到的闭合轮廓数量
// brightnessMax=255 时跳过亮度剔除
uint16_t_t contourDetect(const u8Image* src, const u8Image* graySrc, CONTOUR_DETECTOR_STRUCT* obj);

// 从最近一次 contourDetect() 结果中提取指定轮廓的 7 维 Hu 矩
// 返回 1=成功, 0=索引越界或无数据 (必须在同一帧 contourDetect 之后调用)
uint8_t_t contourExtractHuMoments(const CONTOUR_DETECTOR_STRUCT* obj, uint8_t_t contourIndex, float huOut[7]);

// =====
// [6] 信标检测 (Beacon Detection)
// =====

// ---- typedef ----

// 单个信标数据
typedef struct
{
    uint16_t centerX;    // 质心 X
    uint16_t centerY;    // 质心 Y
    uint16_t contourSize; // 轮廓像素数
    uint8_t_t brightness; // 质心区域亮度 (用于排查)
} BEACON_DATA_STRUCT;

// 信标检测器数据
typedef struct
{
    BEACON_DATA_STRUCT beaconData[OM_MAX_OBJECTS]; // 信标数据缓存
    uint8_t_t beaconCount;    // 有效信标数量
} BEACON_DETECTOR_STRUCT;

// ---- 函数声明 ----

```



```

// eccMax: 偏心率上限 (>=1.0, 1.0=理想圆, 3.0=允许中等椭圆)
// brightnessMin: 信标内部亮度下限 (0~255, 建议大于 120 才视为发光体)
// maxEndpoints: 允许最大端点数 (信标边缘可有断口, 建议 6~10)
void beaconDetectInit(uint16_t sizeMin, uint16_t sizeMax, float eccMax, uint8_t brightnessMin, uint16_t maxEndpoints);

// 检测: edgeSrc=Canny 边缘图, graySrc=灰度原图 (亮度采样)
// 返回检测到的信标数量 (结果存入 obj->beaconData)
uint16_t beaconDetect(const u8Image* edgeSrc, const u8Image* graySrc, BEACON_DETECTOR_STRUCT* obj);

#endif // OPENMORPH_H

```

coreT3_cyt4bb7/project/code/openMorph_config.h

SHA-256

:

61c14c89639ca97b4b8698f1ceac7211b3b17d84261bc96a4a7ab4dfd8b61869

代码 (c)

```

/*
 * openMorph_config.h - openMorph 库可移植配置头
 *
 * 移植到新工程时只需修改本文件即可适配不同图像分辨率与对象数量上限。
 * openMorph 内部其他源文件不应再出现关于尺寸的字面量。
 */
#ifndef OPENMORPH_CONFIG_H
#define OPENMORPH_CONFIG_H

// =====
// 图像最大尺寸 (静态缓冲区按此分配)
// =====
// 当前默认 94 x 60 = 5640 像素 (与本工程实际相机分辨率匹配)
// 如需适配 188x120 / 320x240 等更高分辨率, 按比例上调即可。
// 本配置与具体相机驱动解耦, 不引入 SDK 头依赖。
// -----
#define OM_IMAGE_MAX_WIDTH   (94u)
#define OM_IMAGE_MAX_HEIGHT (60u)
#define OM_BUFFER_SIZE      (OM_IMAGE_MAX_WIDTH * OM_IMAGE_MAX_HEIGHT)

// =====
// 对象检测数量上限 (轮廓 / 信标 / 连通域共用)
// =====
// contourData[] / beaconData[] / sbea[] 等数组的成员数量。
// 超过此上限的候选对象会被忽略 (只保留前 N 个有效结果)。
// -----
#define OM_MAX_OBJECTS      (101u)

// =====
// Canny 流水线环形缓冲宽度
// =====
// Canny 边缘检测内部 sobel 环形缓冲的列宽, 必须 >= OM_IMAGE_MAX_WIDTH。
// -----
#define OM_CANNY_RING_WIDTH (256u)

// =====
// 轮廓跟踪状态机参数
// =====
#define OM_LOCK_THRESHOLD   (3u) // 连续检测到帧数: SEARCHING → ACQUIRING
#define OM_LOCK_CONFIRM    (2u) // 继续确认帧数: ACQUIRING → LOCKED
#define OM_LOST_THRESHOLD  (5u) // 连续丢失帧数: LOCKED → LOST_HOLDING
#define OM_LOST_TIMEOUT    (30u) // 超时帧数: LOST_HOLDING → SEARCHING
#define OM_ORIENTATION_JUMP_MAX (0.785398f) // PI/4, 帧间朝向跳变上限 (预留, 当前未启用硬跳变保护)
#define OM_ORIENTATION_CONFIRM_FRAMES (3u) // 新朝向确认帧数
#define OM_M3_CONFIDENCE_MIN (0.02f) // 三阶矩尖端定向最低置信度: 低于此值仅按上帧对齐尖端 180 度, 不冻结主轴
#define OM_ORIENTATION_MIN_SIZE (8u) // 朝向可靠的最小像素数 (94×60 下按面积比例从 188×120 的 30 下调)
#define OM_ORIENTATION_ALPHA (0.4f) // 朝向低通滤波系数 (喂给 lowPassFilter, 越大越跟随/越小越平滑)
#define OM_POSITION_ALPHA (0.5f) // 位置低通滤波系数 (喂给 lowPassFilter, 越大越跟随/越小越平滑)
#define OM_BORDER_MARGIN (1u) // 画幅边缘安全边距

#endif // OPENMORPH_CONFIG_H

```

SHA-256 : 91578c94c1b2ecd0083a92955d75937490f84654b4249a366b677c7cdd21690b

```

/*
 * CYT4BB Opensource Library ***** CYT4BB ***** SDK **** J*****
 * Copyright (c) 2022 SEEKFREE *****15****
 */

***** CYT4BB *****n*****

/*
 * CYT4BB *****
 * ***** GPL*****GNU General Public License*****
 GNUü*****
 ***** GPLj3榮*****GPL3.0*****gKK*****İ汾*****/?***/
 */
*****k;*****w*****ä*****δ*****KE1*****
*****û*****حص*****;ا*****
*****ب*****μ***** GPL
 */
*****Ö*****7*****6*****n***** GPL *****K*****
*****ü*****Y*****<https://www.gnu.org/licenses/>*
 */
*****Y*****?***
*****7***** GPL3.0 *****é*****İ汾*****
*****Йfi*****libraries/doc *****μ***** GPL3_permission_statement.txt|*****
*****libraries *****|*****μ***** LICENSE|*****
*****эλδ*****h*****r5dI*****E*****
 */
*****!***** cm7_0_isr ***** ?*****
*****æ*****r5***** _*****
*****汾*****ω*****終libraries/doc *****version *****汾*****
*****IAR 9.40.1
*****3***** CYT4BB
***** ***** https://seekfree.taobao.com/
 */
***** !%*****
***** ***** V*****
***** 2024-1-9 pudding first version
***** 2024-5-14 pudding *****12*****pit*****ж*****ÿ*****V*****-*****
***** 2025-2-4 pudding *****Z*****Ж***** *****W*****µL*****::Z*****ي***** *****
***** 2025-2-4 pudding *****ن*****x*****
*****

```

```
#include "core0Loop.h"
```

mcdviii

```
}  
  
void pit0_ch11_isr() // 6 11 жP  
{  
    pit_isr_flag_clear(PIT_CH11);  
}  
  
void pit0_ch12_isr() // 6 12 жP  
{  
    pit_isr_flag_clear(PIT_CH12);  
}  
  
void pit0_ch13_isr() // 6 13 жP  
{  
    pit_isr_flag_clear(PIT_CH13);  
}  
  
void pit0_ch14_isr() // 6 14 жP  
{  
    pit_isr_flag_clear(PIT_CH14);  
}  
  
void pit0_ch15_isr() // 6 15 жP  
{  
    pit_isr_flag_clear(PIT_CH15);  
}  
  
void pit0_ch16_isr() // 6 16 жP  
{  
    pit_isr_flag_clear(PIT_CH16);  
}  
  
void pit0_ch17_isr() // 6 17 жP  
{  
    pit_isr_flag_clear(PIT_CH17);  
}  
  
void pit0_ch18_isr() // 6 18 жP  
{  
    pit_isr_flag_clear(PIT_CH18);  
}  
  
void pit0_ch19_isr() // 6 19 жP  
{  
    pit_isr_flag_clear(PIT_CH19);  
}  
  
void pit0_ch20_isr() // 6 20 жP  
{  
    pit_isr_flag_clear(PIT_CH20);  
}  
  
void pit0_ch21_isr() // 6 21 жP  
{  
    pit_isr_flag_clear(PIT_CH21);  
    tsl1401_collect_pit_handler();  
}  
// ***** PIT жM *****  
  
// ***** жM *****  
// 0I i r  
void uart0_isr(void)  
{  
    if(uart_isr_mask(UART_0)) // 0 ж  
    {  
#if DEBUG_UART_USE_INTERRUPT        // debug ж  
        debug_interrupr_handler(); // debug ъ u ђ     debug λ d.  
#endif                                //      DEBГG_UAТ_RT_INDEХ δ Ć w Ő ] ж z,  
    }  
    else // 0 ж  
    {  
    }  
}  
  
void uart1_isr(void)  
{  
    if(uart_isr_mask(UART_1)) // 1 ж
```

```

        wireless_module_uart_handler(); // 0x00000000gTx ص 00000000
    }
    else // 0x0000000100000000Ж
    {
    }
}

void uart2_isr(void)
{
    if (uart_isr_mask(UART_2)) // 0x0000000200000000Ж
    {
        gnss_uart_callback(); // GPSg ص 00000000
    }
    else // 0x0000000200000000Ж
    {
    }
}

void uart3_isr(void)
{
    if (uart_isr_mask(UART_3)) // 0x0000000300000000Ж
    {
    }
    else // 0x0000000300000000Ж
    {
    }
}

void uart4_isr(void)
{
    if (uart_isr_mask(UART_4)) // 0x0000000400000000Ж
    {
        uart_receiver_handler(); // 0x00000000 00000000g ص 00000000
    }
    else // 0x0000000400000000Ж
    {
    }
}

void uart5_isr(void)
{
    if (uart_isr_mask(UART_5)) // 0x0000000500000000Ж
    {
    }
    else // 0x0000000500000000Ж
    {
    }
}

void uart6_isr(void)
{
    if (uart_isr_mask(UART_6)) // 0x0000000600000000Ж
    {
    }
    else // 0x0000000600000000Ж
    {
    }
}

// ***** 0x00000000ЖM0000 *****

// ***** 0M 0000 *****
void gpio_0_exti_isr() // 0P 00000000Ж
{
}

void gpio_1_exti_isr() // 0P 00000000Ж
{
    if (exti_flag_get(P01_0)) // '0000P1_0. 0000ЖЖЖЖЖ
    {
    }
    if (exti_flag_get(P01_1))
    {
    }
}

void gpio_2_exti_isr() // 0P 00000000Ж
{
    if (exti_flag_get(P02_0))
    {
    }
}

```

```

    }
    if (exti_flag_get(P02_4))
    {
    }
}

void gpio_3_exti_isr() // ⚡P ⚡⚡⚡⚡⚡K
{
}

void gpio_4_exti_isr() // ⚡P ⚡⚡⚡⚡⚡K
{
}

void gpio_5_exti_isr() // ⚡P ⚡⚡⚡⚡⚡K
{
}

void gpio_6_exti_isr() // ⚡P ⚡⚡⚡⚡⚡K
{
}

void gpio_7_exti_isr() // ⚡P ⚡⚡⚡⚡⚡K
{
}

void gpio_8_exti_isr() // ⚡P ⚡⚡⚡⚡⚡K
{
}

void gpio_9_exti_isr() // ⚡P ⚡⚡⚡⚡⚡K
{
}

void gpio_10_exti_isr() // ⚡P ⚡⚡⚡⚡⚡Ж
{
}

void gpio_11_exti_isr() // ⚡P ⚡⚡⚡⚡⚡Ж
{
}

void gpio_12_exti_isr() // ⚡P ⚡⚡⚡⚡⚡Ж
{
}

void gpio_13_exti_isr() // ⚡P ⚡⚡⚡⚡⚡Ж
{
}

void gpio_14_exti_isr() // ⚡P ⚡⚡⚡⚡⚡Ж
{
}

void gpio_15_exti_isr() // ⚡P ⚡⚡⚡⚡⚡Ж
{
}

void gpio_16_exti_isr() // ⚡P ⚡⚡⚡⚡⚡Ж
{
}

void gpio_17_exti_isr() // ⚡P ⚡⚡⚡⚡⚡Ж
{
}

void gpio_18_exti_isr() // ⚡P ⚡⚡⚡⚡⚡Ж
{
}

void gpio_19_exti_isr() // ⚡P ⚡⚡⚡⚡⚡Ж
{
}

void gpio_20_exti_isr() // ⚡P ⚡⚡⚡⚡⚡Ж
{
}

```

coreT3_cyt4bb7/project/user/cm7_1_isr.c

SHA-256

2b5414a2d192587d35adaeab22c30c7f61873d1944dedbb1d655f4a4db0e3d23

代码 (c)

[illegible]

```
#include "zf_common_headfile.h"
```

```
#include "core1Loop.h"
```

```
// *****PIT*****
void pit0_ch1_isr() // 0
{
    pit_isr_flag_clear(PIT_CH0);
}

void pit0_ch1_isr() // 1
{
    pit_isr_flag_clear(PIT_CH1);

    pitProcess0();
}
```


mcdxiv


```

    {
    }
    if (exti_flag_get(P01_1))
    {
    }
}

void gpio_2_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
    if (exti_flag_get(P02_0))
    {
    }
    if (exti_flag_get(P02_4))
    {
    }
}

void gpio_3_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
}

void gpio_4_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
}

void gpio_5_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
}

void gpio_6_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
}

void gpio_7_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
}

void gpio_8_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
}

void gpio_9_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
}

void gpio_10_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
}

void gpio_11_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
}

void gpio_12_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
}

void gpio_13_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
}

void gpio_14_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
}

void gpio_15_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
}

void gpio_16_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
}

void gpio_17_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K
{
}

void gpio_18_exti_isr() // ⚡P      ⚡⚡⚡⚡⚡⚡K

```

```

{
}

void gpio_19_exti_isr() // 中断
{
}

void gpio_20_exti_isr() // 中断
{
}

void gpio_21_exti_isr() // 中断
{
}

void gpio_22_exti_isr() // 中断
{
}

void gpio_23_exti_isr() // 中断
{
}
// ***** 逐飞科技 *****

```

coreT3_cyt4bb7/project/user/main_cm7_0.c

SHA-256

:

242b7ae29f849aa3e046f4825283205c56b64b525bd46c859bf86110f750abf1

代码 (c)

```

/*****
 * CYT4BB Opensource Library 即（CYT4BB 开源库）是一个基于官方 SDK 接口的第三方开源库
 * Copyright (c) 2022 SEEKFREE 逐飞科技
 *
 * 本文件是 CYT4BB 开源库的一部分
 *
 * CYT4BB 开源库 是免费软件
 * 您可以根据自由软件基金会发布的 GPL（GNU General Public License，即 GNU 通用公共许可证）的条款
 * 即 GPL 的第 3 版（即 GPL3.0）或（您选择的）任何后来的版本，重新发布和/或修改它
 *
 * 本开源库的发布是希望它能发挥作用，但并未对其作任何的保证
 * 甚至没有隐含的适销性或适合特定用途的保证
 * 更多细节请参见 GPL
 *
 * 您应该在收到本开源库的同时收到一份 GPL 的副本
 * 如果没有，请参阅<https://www.gnu.org/licenses/>
 *
 * 额外注明：
 * 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
 * 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
 * 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
 * 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明（即本声明）
 *
 * 文件名称      main_cm7_0
 * 公司名称      成都逐飞科技有限公司
 * 版本信息      查看 libraries/doc 文件夹内 version 文件 版本说明
 * 开发环境      IAR 9.40.1
 * 适用平台      CYT4BB
 * 店铺链接      https://seekfree.taobao.com/
 *
 * 修改记录
 * 日期          作者          备注
 * 2024-1-4      pudding      first version
 *****/

#include "zf_common_headfile.h"

#include "core0Loop.h"

// 打开新的工程或者工程移动了位置务必执行以下操作
// 第一步 关闭上面所有打开的文件
// 第二步 project->clean 等待下方进度条走完

// 本例程是开源库空工程 可用作移植或者测试各类内外设

```

```
// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设

// ***** 代码区域 *****

/* =====
* BEFORE EDITING THIS CORE — READ THESE TWO DOCS FIRST:
* 1. <repo-root>/CLAUDE.md — multi-core architecture, naming rules, git workflow
* 2. ./project/code/PROJECT.md — this core's modules, boundaries, hazards
* Each core has the SAME filenames (motor.c / IMU.c / PID.c) but DIFFERENT semantics.
* Do NOT copy patterns across cores. Do NOT edit two cores in one change.
* ===== */
int main(void)
{
    clock_init(SYSTEM_CLOCK_250M); // 时钟配置及系统初始化<务必保留>
    debug_init(); // 调试串口信息初始化
    // 此处编写用户代码 例如外设初始化代码等

    processInit();

    // 此处编写用户代码 例如外设初始化代码等
    while (true)
    {
        // 此处编写需要循环执行的代码

        whileProcess();

        // 此处编写需要循环执行的代码
    }
}

// ***** 代码区域 *****
```

coreT3_cyt4bb7/project/user/main_cm7_1.c

SHA-256

:

363e25e84c266e455ba510f21d00be6a76c6b61238c8b3cf827431da3e8084e8

代码（c）

```
/* =====
* CYT4BB Opensource Library 即（CYT4BB 开源库）是一个基于官方 SDK 接口的第三方开源库
* Copyright (c) 2022 SEEKFREE 逐飞科技
*
* 本文件是 CYT4BB 开源库的一部分
*
* CYT4BB 开源库 是免费软件
* 您可以根据自由软件基金会发布的 GPL（GNU General Public License，即 GNU 通用公共许可证）的条款
* 即 GPL 的第 3 版（即 GPL3.0）或（您选择的）任何后来的版本，重新发布和/或修改它
*
* 本开源库的发布是希望它能发挥作用，但并未对其作任何的保证
* 甚至没有隐含的适销性或适合特定用途的保证
* 更多细节请参见 GPL
*
* 您应该在收到本开源库的同时收到一份 GPL 的副本
* 如果没有，请参阅<https://www.gnu.org/licenses/>
*
* 额外注明：
* 本开源库使用 GPL3.0 开源许可证协议 以上许可申明为译文版本
* 许可申明英文版在 libraries/doc 文件夹下的 GPL3_permission_statement.txt 文件中
* 许可证副本在 libraries 文件夹下 即该文件夹下的 LICENSE 文件
* 欢迎各位使用并传播本程序 但修改内容时必须保留逐飞科技的版权声明（即本声明）
*
* 文件名称      main_cm7_1
* 公司名称      成都逐飞科技有限公司
* 版本信息      查看 libraries/doc 文件夹内 version 文件 版本说明
* 开发环境      IAR 9.40.1
* 适用平台      CYT4BB
* 店铺链接      https://seekfree.taobao.com/
*
* 修改记录
* 日期          作者          备注
* 2024-1-4      pudding      first version
* ===== */
```

```

#include "zf_common_headfile.h"

#include "core1Loop.h"

// 打开新的工程或者工程移动了位置务必执行以下操作
// 第一步 关闭上面所有打开的文件
// 第二步 project->clean 等待下方进度条走完

// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设
// 本例程是开源库空工程 可用作移植或者测试各类内外设

// ***** 代码区域 *****

/* =====
* BEFORE EDITING THIS CORE — READ THESE TWO DOCS FIRST:
* 1. <repo-root>/CLAUDE.md — multi-core architecture, naming rules, git workflow
* 2. ./project/code/PROJECT.md — this core's modules, boundaries, hazards
* Each core has the SAME filenames (motor.c / IMU.c / PID.c) but DIFFERENT semantics.
* Do NOT copy patterns across cores. Do NOT edit two cores in one change.
* ===== */
int main(void)
{
    clock_init(SYSTEM_CLOCK_250M); // 时钟配置及系统初始化<务必保留>
    debug_info_init(); // 调试串口信息初始化

    // 此处编写用户代码 例如外设初始化代码等

    processInit();

    // 此处编写用户代码 例如外设初始化代码等
    while (true)
    {
        // 此处编写需要循环执行的代码

        whileProcess();

        // 此处编写需要循环执行的代码
    }
}

// ***** 代码区域 *****

```